

LeetMastery

Study Guide — Organised by Pattern · ranked fewest → most questions

Graphs · Greedy · JavaScript · String · Sliding Window · Sorting · Bit Manipulation · Backtracking · BFS · Trie · Math · Arrays & Hashing · Two Pointers · Heap · Matrix · Linked List · DFS · Binary Search · Dynamic Programming · Stack · Trees & BST

LeetDoocs · SimplyLeet · WalkCC · LeetCode.ca
All languages · 363 Doocs descriptions · 226 embedded images
LeetCode editorials: 56/331 free · Easy → Medium → Hard
331 Questions · 21 Patterns

LeetMastery — By Pattern

— 1 —

LeetMastery

Table of Contents

#1 **Graphs** (5 questions)
#128 Longest Consecutive Sequence [Medium]
#1136 Parallel Courses [Medium]
#305 Number of Islands II [Hard]
#1153 String Transforms Into Another String [Hard]
#2 **Greedy** (6 questions)
#409 Longest Palindrome [Easy]
#134 Gas Station [Medium]
#179 Largest Number [Medium]
#280 Wiggle Sort [Medium]
#954 Array of Doubled Pairs [Medium]
#2131 Longest Palindrome by Concatenating Two Letter Words [Medium]
#3 **JavaScript** (7 questions)
#2627 Debounce [Medium]
#2628 JSON Deep Equal [Medium]
#2630 Memoize [Medium]
#2633 Convert Object to JSON String [Medium]
#2636 Promise Pool [Medium]
#2675 Array of Objects to Matrix [Medium]
#2700 Differences Between Two Objects [Medium]
#4 **String** (8 questions)
#383 Ransom Note [Easy]
#734 Sentence Similarity [Easy]
#1165 Single Row Keyboard [Easy]
#8 String to Integer (atoi) [Medium]
#249 Group Shifted Strings [Medium]
#271 Encode and Decode Strings [Medium]
#635 Design Log Storage System [Medium]
#1138 Alphabet Board Path [Medium]
#5 **Sliding Window** (9 questions)
#3 Longest Substring Without Repeating Characters [Medium]
#159 Longest Substring with At Most Two Distinct Characters [Medium]
#340 Longest Substring with At Most K Distinct Characters [Medium]
#424 Longest Repeating Character Replacement [Medium]
#438 Find All Anagrams in a String [Medium]
#487 Max Consecutive Ones II [Medium]
#1100 Find K-Length Substrings with No Repeated Characters [Medium]
#1248 Count Number of Nice Subarrays [Medium]
#76 Minimum Window Substring [Hard]
#6 **Sorting** (9 questions)
#169 Majority Element [Easy]

Cover · LeetMastery — By Pattern

— 2 —

LeetMastery

#217 Contains Duplicate [Easy]
#242 Valid Anagram [Easy]
#252 Meeting Rooms [Easy]
#1086 Largest Unique Number [Easy]
#1122 Relative Sort Array [Easy]
#49 Group Anagrams [Medium]
#5 Merge Intervals [Medium]
#1152 Analyze User Website Visit Pattern [Medium]
#7 **Bit Manipulation** (10 questions)
#67 Add Binary [Easy]
#136 Single Number [Easy]
#190 Reverse Bits [Easy]
#191 Number of 1 Bits [Easy]
#268 Missing Number [Easy]
#338 Counting Bits [Easy]
#78 Subsets [Medium]
#90 Subsets II [Medium]
#287 Find the Duplicate Number [Medium]
#1508 Find Root of N-Ary Tree [Medium]
#8 **Backtracking** (10 questions)
#17 Letter Combinations of a Phone Number [Medium]
#22 Generate Parentheses [Medium]
#39 Combination Sum [Medium]
#46 Permutations [Medium]
#79 Word Search [Medium]
#113 Path Sum II [Medium]
#37 Sudoku Solver [Hard]
#51 N-Queens [Hard]
#126 Word Ladder II [Hard]
#906 Number of Squared Arrays [Hard]
#9 **BFS** (10 questions)
#286 Walls and Gates [Medium]
#322 Coin Change [Medium]
#542 01 Matrix [Medium]
#994 Rotting Oranges [Medium]
#1187 Minimum Knight Moves [Medium]
#1730 Shortest Path to Get Food [Medium]
#127 Word Ladder [Hard]
#317 Shortest Distance from All Buildings [Hard]
#775 Sliding Puzzle [Hard]
#815 Bus Routes [Hard]
#10 **Trie** (12 questions)
#14 Longest Common Prefix [Easy]
#139 Word Break [Medium]

Table of Contents · LeetMastery — By Pattern

— 3 —

LeetMastery

#208 Implement Trie (Prefix Tree) [Medium]
#211 Design Add and Search Words Data Structure [Medium]
#616 Add Bold Tag in String [Medium]
#692 Top K Frequent Words [Medium]
#720 Longest Word in Dictionary [Medium]
#1166 Design File System [Medium]
#212 Word Search II [Hard]
#536 Palindrome Pairs [Hard]
#588 Design In-Memory File System [Hard]
#642 Design Search Autocomplete System [Hard]
#11 Math (12 questions)
#9 Palindrome Number [Easy]
#1 Roman to Integer [Easy]
#504 Base 7 [Easy]
#1056 Confusing Number [Easy]
#1228 Missing Number in Arithmetic Progression [Easy]
#1427 Perform String Shifts [Easy]
#7 Reverse Integer [Medium]
#59 Pow(x, n) [Medium]
#380 Insert Delete GetRandom O(1) [Medium]
#1017 Convert to Base -2 [Medium]
#3160 Find the Number of Distinct Colors Among the Balls [Medium]
#68 Text Justification [Hard]
#12 Arrays & Hashing (18 questions)
#1 Two Sum [Easy]
#163 Missing Ranges [Easy]
#346 Moving Average from Data Stream [Easy]
#760 Find Anagram Mappings [Easy]
#1426 Counting Elements [Easy]
#1534 Count Good Triples [Easy]
#57 Insert Interval [Medium]
#238 Product of Array Except Self [Medium]
#281 Zigzag Iterator [Medium]
#307 Range Sum Query – Mutable [Medium]
#454 Sum II [Medium]
#525 Contiguous Array [Medium]
#560 Subarray Sum Equals K [Medium]
#1010 Pairs of Songs With Total Durations Divisible by 60 [Medium]
#1272 Remove Interval [Medium]
#1429 First Unique Number [Medium]
#3159 Find Occurrences of an Element in an Array [Medium]
#41 First Missing Positive [Hard]
#13 Two Pointers (19 questions)
#88 Merge Sorted Array [Easy]

Table of Contents · LeetMastery — By Pattern

— 4 —

LeetMastery

#125 Valid Palindrome [Easy]
#283 Move Zeroes [Easy]
#408 Valid Word Abbreviation [Easy]
#977 Squares of a Sorted Array [Easy]
#1 Container With Most Water [Medium]
#15 3Sum [Medium]
#16 3Sum Closest [Medium]
#31 Next Permutation [Medium]
#75 Sort Colors [Medium]
#161 One Edit Distance [Medium]
#167 Two Sum II – Input Array Is Sorted [Medium]
#186 Reverse Words in a String II [Medium]
#189 Rotate Array [Medium]
#532 K-diff Pairs in an Array [Medium]
#923 3Sum With Multiplicity [Medium]
#986 Interval List Intersections [Medium]
#1055 Shortest Way to Form String [Medium]
#2163 Count the Number of Fair Pairs [Medium]
#14 Heap (20 questions)
#215 Kth Largest Element in an Array [Medium]
#253 Meeting Rooms II [Medium]
#347 Top K Frequent Elements [Medium]
#505 The Maze II [Medium]
#621 Task Scheduler [Medium]
#658 Find K Closest Elements [Medium]
#787 Cheapest Flights Within K Stops [Medium]
#973 K Closest Points to Origin [Medium]
#1057 Campus Bikes [Medium]
#1167 Minimum Cost to Connect Sticks [Medium]
#1424 Diagonal Traverse II [Medium]
#23 Merge k Sorted Lists [Hard]
#218 The Skyline Problem [Hard]
#272 Closest Binary Search Tree Value II [Hard]
#295 Find Median from Data Stream [Hard]
#358 Rearrange String k Distance Apart [Hard]
#499 The Maze III [Hard]
#632 Smallest Range Covering Elements from K Lists [Hard]
#759 Employee Free Time [Hard]
#1425 Constrained Subsequence Sum [Hard]
#15 Matrix (20 questions)
#422 Valid Word Square [Easy]
#566 Reshape the Matrix [Easy]
#766 Toeplitz Matrix [Easy]
#867 Transpose Matrix [Easy]

Table of Contents · LeetMastery — By Pattern

— 5 —

LeetMastery

#2373 Largest Local Values in a Matrix [Easy]
#3 Valid Sudoku [Medium]
#48 Rotate Image [Medium]
#54 Spiral Matrix [Medium]
#59 Spiral Matrix II [Medium]
#64 Minimum Path Sum [Medium]
#75 Set Matrix Zeroes [Medium]
#71 Search a 2D Matrix [Medium]
#221 Maximal Square [Medium]
#311 Sparse Matrix Multiplication [Medium]
#498 Diagonal Traverse [Medium]
#531 Lonely Pixel I [Medium]
#723 Candy Crush [Medium]
#835 Image Overlap [Medium]
#1198 Find Smallest Common Element in All Rows [Medium]
#1074 Number of Submatrices That Sum to Target [Hard]
#16 Linked List (23 questions)
#21 Merge Two Sorted Lists [Easy]
#141 Linked List Cycle [Easy]
#206 Reverse Linked List [Easy]
#234 Palindrome Linked List [Easy]
#706 Design HashMap [Easy]
#876 Middle of the Linked List [Easy]
#1474 Delete N Nodes After M Nodes of Linked List [Easy]
#2 Add Two Numbers [Medium]
#19 Remove Nth Node From End of List [Medium]
#24 Swap Nodes in Pairs [Medium]
#61 Rotate List [Medium]
#146 LRU Cache [Medium]
#148 Sort List [Medium]
#328 Odd Even Linked List [Medium]
#369 Plus One Linked List [Medium]
#430 Flatten a Multilevel Doubly Linked List [Medium]
#622 Design Circular Queue [Medium]
#707 Design Linked List [Medium]
#708 Insert into a Sorted Circular Linked List [Medium]
#1171 Remove Zero Sum Consecutive Nodes from Linked List [Medium]
#25 Reverse Nodes in k-Group [Hard]
#432 All O'one Data Structure [Hard]
#460 LFU Cache [Hard]
#17 DFS (23 questions)
#463 Island Perimeter [Easy]
#733 Flood Fill [Easy]
#130 Surrounded Regions [Medium]

Table of Contents · LeetMastery — By Pattern

— 6 —

LeetMastery

#133 Clone Graph [Medium]
#200 Number of Islands [Medium]
#207 Course Schedule [Medium]
#210 Course Schedule II [Medium]
#261 Graph Valid Tree [Medium]
#310 Minimum Height Trees [Medium]
#323 Number of Connected Components in an Undirected Graph [Medium]
#417 Pacific Atlantic Water Flow [Medium]
#490 The Maze [Medium]
#694 Number of Distinct Islands [Medium]
#695 Max Area of Island [Medium]
#721 Accounts Merge [Medium]
#785 Is Graph Bipartite? [Medium]
#1236 Web Crawler [Medium]
#1242 Web Crawler Multithreaded [Medium]
#269 Alien Dictionary [Hard]
#329 Longest Increasing Path in a Matrix [Hard]
#685 Redundant Connection II [Hard]
#1036 Escape a Large Maze [Hard]
#1192 Critical Connections in a Network [Hard]
#18 Binary Search (24 questions)
#35 Search Insert Position [Easy]
#69 Sort() [Easy]
#218 First Bad Version [Easy]
#374 Guess Number Higher or Lower [Easy]
#704 Binary Search [Easy]
#33 Search in Rotated Sorted Array [Medium]
#34 Find First and Last Position of Element in Sorted Array [Medium]
#153 Find Minimum in Rotated Sorted Array [Medium]
#300 Longest Increasing Subsequence [Medium]
#362 Design Hit Counter [Medium]
#528 Random Pick with Weight [Medium]
#852 Peak Index in a Mountain Array [Medium]
#981 Time Based Key-Value Store [Medium]
#1011 Capacity to Ship Packages Within D Days [Medium]
#1060 Missing Element in Sorted Array [Medium]
#1062 Longest Repeating Substring [Medium]
#1533 Find the Index of the Large Integer [Medium]
#4 Median of Two Sorted Arrays [Hard]
#315 Count of Smaller Numbers After Self [Hard]
#410 Split Array Largest Sum [Hard]
#668 Kth Smallest Number in Multiplication Table [Hard]
#710 Random Pick with Blacklist [Hard]
#774 Minimize Max Distance to Gas Station [Hard]

#1235 Maximum Profit in Job Scheduling [Hard]
#19 Dynamic Programming (24 questions)
#71 Climbing Stairs [Easy]
#121 Best Time to Buy and Sell Stock [Easy]
#5 Longest Palindromic Substring [Medium]
#53 Maximum Subarray [Medium]
#55 Jump Game [Medium]
#65 Unique Paths [Medium]
#72 Edit Distance [Medium]
#91 Decode Ways [Medium]
#152 Maximum Product Subarray [Medium]
#198 House Robber [Medium]
#256 Paint House [Medium]
#309 Best Time to Buy and Sell Stock with Cooldown [Medium]
#377 Combination Sum IV [Medium]
#416 Partition Equal Subset Sum [Medium]
#435 Non-overlapping Intervals [Medium]
#516 Longest Palindromic Subsequence [Medium]
#576 Out of Boundary Paths [Medium]
#1143 Longest Common Subsequence [Medium]
#10 Regular Expression Matching [Hard]
#115 Distinct Subsequences [Hard]
#123 Best Time to Buy and Sell Stock III [Hard]
#132 Palindrome Partitioning II [Hard]
#312 Burst Balloons [Hard]
#1000 Minimum Cost to Merge Stones [Hard]
#20 Stack (25 questions)
#20 Valid Parentheses [Easy]
#232 Implement Queue using Stacks [Easy]
#844 Backspace String Compare [Easy]
#143 Reorder List [Medium]
#150 Evaluate Reverse Polish Notation [Medium]
#155 Min Stack [Medium]
#173 Binary Search Tree Iterator [Medium]
#227 Basic Calculator II [Medium]
#255 Verify Preorder Sequence in Binary Search Tree [Medium]
#394 Decode String [Medium]
#439 Ternary Expression Parser [Medium]
#484 Find Permutation [Medium]
#735 Asteroid Collision [Medium]
#739 Daily Temperatures [Medium]
#962 Maximum Width Ramp [Medium]
#1214 Two Sum BSTs [Medium]
#1762 Buildings With an Ocean View [Medium]

#32 Longest Valid Parentheses [Hard]
#42 Trapping Rain Water [Hard]
#84 Largest Rectangle in Histogram [Hard]
#224 Basic Calculator [Hard]
#239 Sliding Window Maximum [Hard]
#716 Max Stack [Hard]
#772 Basic Calculator III [Hard]
#895 Maximum Frequency Stack [Hard]
#21 Trees & BST (37 questions)
#100 Same Tree [Easy]
#101 Symmetric Tree [Easy]
#104 Maximum Depth of Binary Tree [Easy]
#105 Convert Sorted Array to Binary Search Tree [Easy]
#110 Balanced Binary Tree [Easy]
#112 Path Sum [Easy]
#226 Invert Binary Tree [Easy]
#270 Closest Binary Search Tree Value [Easy]
#501 Find Mode in Binary Search Tree [Easy]
#543 Diameter of Binary Tree [Easy]
#572 Subtree of Another Tree [Easy]
#98 Validate Binary Search Tree [Medium]
#102 Binary Tree Level Order Traversal [Medium]
#103 Binary Tree Zigzag Level Order Traversal [Medium]
#105 Construct Binary Tree from Preorder and Inorder Traversal [Medium]
#199 Binary Tree Right Side View [Medium]
#230 Kth Smallest Element in a BST [Medium]
#235 Lowest Common Ancestor of a Binary Search Tree [Medium]
#236 Lowest Common Ancestor of a Binary Tree [Medium]
#250 Count Univalued Subtrees [Medium]
#285 Inorder Successor in BST [Medium]
#298 Binary Tree Longest Consecutive Sequence [Medium]
#314 Binary Tree Vertical Order Traversal [Medium]
#333 Largest BST Subtree [Medium]
#356 Find Leaves of Binary Tree [Medium]
#437 Path Sum III [Medium]
#545 Boundary of Binary Tree [Medium]
#549 Binary Tree Longest Consecutive Sequence II [Medium]
#582 Kill Process [Medium]
#662 Maximum Width of Binary Tree [Medium]
#863 All Nodes Distance K in Binary Tree [Medium]
#1120 Maximum Average Subtree [Medium]
#1490 Clone N-ary Tree [Medium]
#1522 Diameter of N-Ary Tree [Medium]
#1650 Lowest Common Ancestor of a Binary Tree III [Medium]

#124 Binary Tree Maximum Path Sum [Hard]
#297 Serialize and Deserialize Binary Tree [Hard]

#1 Graphs — 5 questions · #1 of 21

#128 MEDIUM

Longest Consecutive Sequence

LeetCode · SimplifyLeet · WalkCCC · LeetCode · Editorial

Array · Hash Table · Union Find

Lists · Grid 169 | CodeSignal

Problem:

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [100,4,200,1,3,2]`
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`
Output: 9

Example 3:

Input: `nums = [1,0,1,2]`
Output: 3

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

>> Brute Force [Graphs] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def longestConsecutive(self, nums):
        # Create a hash set for each number within its sequence
        num_set = set(nums)
        best = 0
        for num in nums:
            length = 0
            while num + length in num_set:
                length += 1
            best = max(best, length)
        return best
```

Python

Python

```
# Python solution for Longest Consecutive Sequence

def longestConsecutive(nums):
    # Create a set from the list for O(1) look-up time
    num_set = set(nums)
    longest_streak = 0

    # Iterate through each number in the set
    for num in num_set:
        # Only start counting if 'num-1' is not present in the set
        if num - 1 not in num_set:
            current_num = num
            current_streak = 1

            # Count consecutive numbers starting from current_num
            while current_num + 1 in num_set:
                current_num += 1
                current_streak += 1

            # Update the longest streak if current one is longer
            longest_streak = max(longest_streak, current_streak)

    return longest_streak

# Example usage:
print(longestConsecutive([100, 4, 200, 1, 3, 2])) # Output: 4
```

Python

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        ans = 0
        seen = set(nums)

        for num in seen:
            # num is the start of a sequence.
            if num - 1 in seen:
                continue
            length = 0
            while num in seen:
                num += 1
                length += 1
            ans = max(ans, length)

        return ans
```

Python

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        d = defaultdict(int)
        for x in nums:
            y = x
            while y in s:
                s.remove(y)
                y += 1
            d[x] = d[y] + y - x
            ans = max(ans, d[x])
        return ans
```

Python

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        for x in nums:
            if x - 1 not in s:
                y = x + 1
                while y in s:
                    y += 1
                ans = max(ans, y - x)
        return ans
```

C++

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        int ans = 0;
        unordered_set<int> seen(nums.begin(), nums.end());

        for (int num : seen) {
            // num is the start of a sequence.
            if (seen.contains(num - 1))
                continue;
            int length = 1;
            while (seen.contains(++num))
                ++length;
            ans = max(ans, length);
        }

        return ans;
    }
};
```

C++

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        int ans = 0;
        unordered_map<int, int> d;
        for (int x : nums) {
            int y = x;
            while (s.contains(y)) {
                s.erase(y++);
            }
            d[x] = (d.contains(y) ? d[y] : 0) + y - x;
            ans = max(ans, d[x]);
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        int ans = 0;
        for (int x : nums) {
            if (!s.contains(x - 1)) {
                int y = x + 1;
                while (s.contains(y)) {
                    y++;
                }
                ans = max(ans, y - x);
            }
        }
        return ans;
    }
};
```

Java

```
class Solution {
    public int longestConsecutive(int[] nums) {
        int ans = 0;
        Set<Integer> seen = Arrays.stream(nums).boxed().collect(Collectors.toSet());

        for (int num : seen) {
            // num is the start of a sequence.
            if (seen.contains(num - 1))
                continue;
            int length = 1;
            while (seen.contains(++num))
                ++length;
            ans = Math.max(ans, length);
        }

        return ans;
    }
}
```

Java

```
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> s = new HashSet<>();
        for (int x : nums) {
            s.add(x);
        }
        int ans = 0;
        Map<Integer, Integer> d = new HashMap<>();
        for (int x : nums) {
            int y = x;
            while (s.contains(y)) {
                s.remove(y++);
            }
            d.put(x, d.getOrDefault(y, 0) + y - x);
            ans = Math.max(ans, d.get(x));
        }
        return ans;
    }
}
```

Java

```
use std::collections::HashMap, HashSet;

impl Solution {
    pub fn longest_consecutive(nums: Vec<i32>) -> i32 {
        let mut s: HashSet<i32> = nums.iter().cloned().collect();
        let mut ans = 0;
        let mut d: HashMap<i32, i32> = HashMap::new();
        for x in &nums {
            let mut y = x;
            while s.contains(y) {
                s.remove(y);
                y += 1;
            }
            let length = d.get(y).unwrap_or(&0) + y - x;
            d.insert(x, length);
            ans = ans.max(length);
        }
        ans
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn longest_consecutive(nums: Vec<i32>) -> i32 {
        let s: HashSet<i32> = nums.iter().cloned().collect();
        let mut ans = 0;
        for x in &s {
            if !s.contains(&x - 1) {
                let mut y = x + 1;
                while s.contains(y) {
                    y += 1;
                }
                ans = ans.max(y - x);
            }
        }
        ans
    }
}
```

Java

```
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> s = new HashSet<>();
        for (int x : nums) {
            s.add(x);
        }
        int ans = 0;
        for (int x : nums) {
            if (!s.contains(x - 1)) {
                int y = x + 1;
                while (s.contains(y)) {
                    y++;
                }
                ans = Math.max(ans, y - x);
            }
        }
        return ans;
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    #[allow(dead_code)]
    pub fn longest_consecutive(nums: Vec<i32>) -> i32 {
        let mut s = HashMap::new();
        let mut ret = 0;

        // Initialize the set
        for num in &nums {
            s.insert(*num);
        }

        for num in &nums {
            if s.contains(&(num - 1)) {
                continue;
            }
            let mut cur_num = num.clone();
            while s.contains(&cur_num) {
                cur_num += 1;
            }
            // Update the answer
            ret = std::cmp::max(ret, cur_num - num);
        }
        ret
    }
}
```

JavaScript

```
var longestConsecutive = function (nums) {
    const s = new Set(nums);
    let ans = 0;
    const d = new Map();
    for (const x of nums) {
        let y = x;
        while (s.has(y)) {
            s.delete(y++);
        }
        d.set(x, (d.get(y) || 0) + (y - x));
        ans = Math.max(ans, d.get(x));
    }
    return ans;
};
```

JavaScript

```

//
* Given (number[]) num
* Return (number)
*/
var longestConsecutive = function (nums) {
    const s = new Set(nums);
    let ans = 0;
    for (const x of nums) {
        if (!s.has(x - 1)) {
            let y = x + 1;
            while (s.has(y)) {
                y++;
            }
            ans = Math.max(ans, y - x);
        }
    }
    return ans;
};

```

TypeScript

```

// TypeScript
function longestConsecutive(nums: number[]): number {
    const s = new Set(nums);
    let ans = 0;
    for (const x of nums) {
        const d = new Map<number, number>();
        for (const x of nums) {
            let y = x;
            while (s.has(y)) {
                s.delete(y++);
            }
            d.set(x, (d.get(x) || 0) + (y - x));
            ans = Math.max(ans, d.get(x));
        }
    }
    return ans;
}

```

TypeScript

```

// TypeScript
function longestConsecutive(nums: number[]): number {
    const s = new Set<number>(nums);
    let ans = 0;
    for (const x of s) {
        if (!s.has(x - 1)) {
            let y = x + 1;
            while (s.has(y)) {
                y++;
            }
            ans = Math.max(ans, y - x);
        }
    }
    return ans;
}

```

TypeScript

```

function longestConsecutive(nums: number[]): number {
    const s = new Set<number>(nums);
    let ans = 0;
    for (const x of s) {
        if (!s.has(x - 1)) {
            let y = x + 1;
            while (s.has(y)) {
                y++;
            }
            ans = Math.max(ans, y - x);
        }
    }
    return ans;
}

```

Go

Go

```

func longestConsecutive(nums []int) (ans int) {
    s := map[int]bool{}
    for _, x := range nums {
        s[x] = true
    }
    d := map[int]int{}
    for _, x := range nums {
        y := x
        for s[y] {
            delete(s, y)
            y++
        }
        d[x] = d[y] + y - x
        ans = max(ans, d[x])
    }
    return
}

```

Go

```

func longestConsecutive(nums []int) (ans int) {
    s := map[int]bool{}
    for _, x := range nums {
        s[x] = true
    }
    for _, x := range nums {
        if !s[x] {
            y := x + 1
            for s[y] {
                y++
            }
            ans = max(ans, y-x)
        }
    }
    return
}

```

[7] Graphs - Pattern Solution (stored optimal)

Python

```

# Python solution for Longest Consecutive Sequence

def longestConsecutive(nums):
    # Create a set from the list for O(1) look-up times
    num_set = set(nums)
    longest_streak = 0

    # Iterate through each number in the set
    for num in num_set:
        # Only start counting if 'num-1' is not present in the set
        if num - 1 not in num_set:
            current_num = num
            current_streak = 1

            # Count consecutive numbers starting from current_num
            while current_num + 1 in num_set:
                current_num += 1
                current_streak += 1

            # Update the longest streak if current one is longer
            longest_streak = max(longest_streak, current_streak)

    return longest_streak

# Example usage:
print(longestConsecutive([100, 4, 200, 1, 3, 2])) # Output: 4

```

C++

```

// C++ solution for Longest Consecutive Sequence

#include <vector>
#include <unordered_set>
#include <algorithm>
using namespace std;

class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        // Create a set from the vector for O(1) look-up times
        unordered_set<int> numSet(nums.begin(), nums.end());
        int longestStreak = 0;

        // Iterate over each number in the set
        for (int num : numSet) {
            // Check if it is the start of a sequence
            if (numSet.find(num - 1) == numSet.end()) {
                int currentNum = num;
                int currentStreak = 1;

                // Count consecutive numbers
                while (numSet.find(currentNum + 1) != numSet.end()) {
                    currentNum++;
                    currentStreak++;
                }
            }
        }
    }
};

```

```

}

// Update the longest streak
longestStreak = max(longestStreak, currentStreak);

}

return longestStreak;
};

```

#277

MEDIUM

Find the Celebrity

[LeetCode](#) - [Simplify](#) - [WuCC](#) - [LeetCode](#) - [Editorial](#)

[Two Pointers](#) - [Graph](#) - [Interactive](#)

[Like](#) - [Premium](#) - [ID](#)

Problem:

Suppose you are at a party with n people labeled from 0 to $n - 1$ and among them, there may exist one celebrity. The definition of a celebrity is that all the other $n - 1$ people know the celebrity, but the celebrity does not know any of them.

Now you want to find out who the celebrity is or verify that there is not one. You are only allowed to ask questions like: "Hi, A. Do you know B?" to get information about whether A knows B. You need to find out the celebrity (or verify there is not one) by asking as few questions as possible (in the asymptotic sense).

You are given an integer n and a helper function `bool knows(a, b)` that tells you whether a knows b. Implement a function `int findCelebrity(n)`. There will be exactly one celebrity if they are at the party. Return the celebrity's label if there is a celebrity at the party. If there is no celebrity, return -1.

Note that the $n \times n$ 2D array `graph` given as input is not directly available to you, and instead only accessible through the helper function `knows`. `graph[i][j] == 1` represents person i knows person j , whereas `graph[i][j] == 0` represents person j does not know person i .

Example 1:

```

# Brute Force Approach
class Solution:
    def findCelebrity(self, n):
        # Brute Force - try all paths via exhaustive DFS
        visited = set()
        def dfs(node):
            if node == -1: return True
            visited.add(node)
            for nei in graph.get(node, []):
                if nei not in visited and dfs(nei): return True
            visited.remove(node)
            return False
        return dfs(0)

```

Python

Python

```

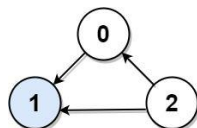
# Problem Definition for findCelebrity using the helper knows(a, b):
def findCelebrity(n):
    # Step 1: Candidate Selection.
    candidate = 0
    for i in range(1, n):
        # If candidate knows i, then candidate cannot be the celebrity.
        if knows(candidate, i):
            candidate = i # Select new candidate.

    # Step 2: Verification.
    # Check if candidate does not know any other person.
    for i in range(1, n):
        if i != candidate:
            # Either candidate knows someone or someone doesn't know candidate.
            if knows(candidate, i) or not knows(i, candidate):
                return -1
    return candidate

# Example helper function (for testing, this is normally provided in the problem context).
def knows(a, b):
    # This function should return True if person a knows person b, otherwise False.
    pass

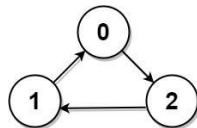
```

Python



Input: `graph = [[1,1,0],[0,1,0],[1,1,1]]`
 Output: 1
 Explanation: There are three persons labeled with 0, 1 and 2. `graph[i][j] = 1` means person i knows person j , otherwise `graph[i][j] = 0` means person i does not know person j . The celebrity is the person labeled as 1 because both 0 and 2 know him but 1 does not know anybody.

Example 2:



Input: `graph = [[1,0,1],[1,1,0],[0,1,1]]`
 Output: -1
 Explanation: There is no celebrity.

Constraints:

- $n == \text{graph.length} == \text{graph}[i].\text{length}$
- $2 \leq n \leq 100$
- `graph[i][j]` is 0 or 1.
- `graph[i][i] == 1`

Follow up: If the maximum number of allowed calls to the API `knows` is $3 * n$, could you find a solution without exceeding the maximum number of calls?

>> Brute Force [Graphs] Interview Prep

Python


```

# The knows API is already defined for you.
# Returns a bool, whether a knows b
# def knows(a: int, b: int) -> bool:

class Solution:
    def findCelebrity(self, n: int) -> int:
        candidate = 0

        # Everyone knows the celebrity.
        for i in range(1, n):
            if knows(candidate, i):
                candidate = i

        # The candidate knows nobody and everyone knows the celebrity.
        for i in range(1, n):
            if i != candidate and knows(candidate, i) or not knows(i, candidate):
                return -1
            if i != candidate and not knows(i, candidate):
                return -1

        return candidate

```

```

Python
# The knows API is already defined for you.
# Returns a bool, whether a knows b
# def knows(a: int, b: int) -> bool:

class Solution:
    def findCelebrity(self, n: int) -> int:
        ans = 0
        for i in range(1, n):
            if knows(ans, i):
                ans = i
        for i in range(1, n):
            if ans != i:
                if knows(ans, i) or not knows(i, ans):
                    return -1
                return ans

```

```

Python
# The knows API is already defined for you.
# Returns a bool, whether a knows b
# def knows(a: int, b: int) -> bool:

class Solution:
    def findCelebrity(self, n: int) -> int:
        ans = 0
        for i in range(1, n):
            if knows(ans, i):
                ans = i
        for i in range(1, n):
            if ans != i:
                if knows(ans, i) or not knows(i, ans):
                    return -1
                return ans

```

Graphs - LeetCode - By Pattern — 25 — LeetCode

```

Java
public class Solution extends Relation {
    public int findCelebrity(int n) {
        int candidate = 0;

        // Everyone knows the celebrity.
        for (int i = 1; i < n; ++i)
            if (knows(candidate, i))
                candidate = i;

        // The candidate knows nobody and everyone knows the celebrity.
        for (int i = 0; i < n; ++i) {
            if (i != candidate && knows(candidate, i) || !knows(i, candidate))
                return -1;
            if (i != candidate && !knows(i, candidate))
                return -1;
        }

        return candidate;
    }
}

Java
// The knows API is defined in the parent class Relation.
boolean knows(int a, int b);

public class Solution extends Relation {
    public int findCelebrity(int n) {
        int ans = 0;
        for (int i = 1; i < n; ++i) {
            if (knows(ans, i)) {
                ans = i;
            }
        }
        for (int i = 0; i < n; ++i) {
            if (ans != i) {
                if (knows(ans, i) || !knows(i, ans)) {
                    return -1;
                }
            }
        }
        return ans;
    }
}

```

Graphs - LeetCode - By Pattern — 26 — LeetCode

```

// The knows API is defined in the parent class Relation.
boolean knows(int a, int b);

public class Solution extends Relation {
    public int findCelebrity(int n) {
        int ans = 0;
        for (int i = 1; i < n; ++i) {
            if (knows(ans, i)) {
                ans = i;
            }
        }
        for (int i = 0; i < n; ++i) {
            if (ans != i) {
                if (knows(ans, i) || !knows(i, ans)) {
                    return -1;
                }
            }
        }
        return ans;
    }
}

```

[*] Graphs - Pattern Solution (stored optimal)

```

Python
# Solution definition for findCelebrity using the helper knows(a, b) API.
def findCelebrity(self):
    # Step 1: Candidate Selection.
    candidate = 0
    for i in range(1, n):
        # If candidate knows i, then candidate cannot be the celebrity.
        if knows(candidate, i):
            candidate = i # Select new candidate.

    # Step 2: Verification.
    # Check if candidate does not know any other person.
    for i in range(1, n):
        if i != candidate:
            # Either candidate knows someone or someone doesn't know candidate.
            if knows(candidate, i) or not knows(i, candidate):
                return -1
    return candidate

# Example helper function (for testing, this is normally provided in the problem context).
def knows(a, b):
    # This function should return True if person a knows person b, otherwise False.
    pass

```

C++

Graphs - LeetCode - By Pattern — 27 — LeetCode

```

// C++ implementation for findCelebrity using the knows(a, b) API.
class Solution {
public:
    // The knows API is already defined for you.
    // bool knows(int a, int b);

    int findCelebrity(int n) {
        // Step 1: Candidate Selection.
        int candidate = 0;
        for (int i = 1; i < n; ++i) {
            // If candidate knows i, then candidate cannot be the celebrity.
            if (knows(candidate, i))
                candidate = i; // Select new candidate.
        }

        // Step 2: Verification.
        for (int i = 0; i < n; ++i) {
            if (i != candidate)
                continue;
            // Check if candidate knows someone or if someone doesn't know candidate.
            if (knows(candidate, i) || !knows(i, candidate))
                return -1;
        }
        return candidate;
    }
};

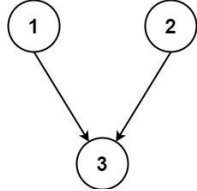
// Dummy implementation of knows API for context.
bool knows(int a, int b) {
    // This function should return true if person a knows person b, otherwise false.
    // Implementation is provided in the problem context.
    return false;
}

```

#1136 **MEDIUM**
Parallel Courses
LeetCode - Simplify - WalkCC - LeetCode - Editorial
Graph - Topological Sort
List Premium 98

Problem:
You are given an integer n , which indicates that there are n courses labeled from 1 to n . You are also given an array relations where relations[i] = [prevCourse, nextCourse], representing a prerequisite relationship between course prevCourse and course nextCourse: course prevCourse has to be taken before course nextCourse.
In one semester, you can take any number of courses as long as you have taken all the prerequisites in the previous semester for the courses you are taking.
Return the minimum number of semesters needed to take all courses. If there is no way to take all the courses, return -1.
Example 1:

Graphs - LeetCode - By Pattern — 28 — LeetCode



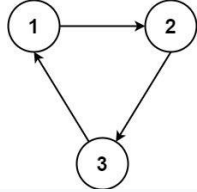
```

graph TD
    1((1)) --> 3((3))
    2((2)) --> 3((3))

```

Input: $n = 3$, relations = [[1,3],[2,3]]
Output: 2
Explanation: The figure above represents the given graph.
In the first semester, you can take courses 1 and 2.
In the second semester, you can take course 3.

Example 2:



```

graph TD
    1((1)) --> 2((2))
    2((2)) --> 3((3))
    3((3)) --> 1((1))

```

Input: $n = 3$, relations = [[1,2],[2,3],[3,1]]
Output: -1
Explanation: No course can be studied because they are prerequisites of each other.

Constraints:

- $1 \leq n \leq 5000$
- $1 \leq \text{relations.length} \leq 5000$
- $\text{relations}[i].\text{length} == 2$
- $1 \leq \text{prevCourse}, \text{nextCourse} \leq n$

Graphs - LeetCode - By Pattern — 29 — LeetCode

- prevCourse != nextCourse
- All the pairs [prevCourse, nextCourse] are unique.

>> Brute Force [Graphs] Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def minimumSemesters(self, numCourses, relations):
        # Brute Force - try all paths via exhaustive DFS
        visited = set()
        def dfs(node):
            if node == dest: return True
            visited.add(node)
            for prev, course in relations:
                if prev == node and dfs(course): return True
            visited.remove(node)
            return False
        return dfs(1)

Python
# Python Implementation
from collections import deque

def minimumSemesters(numCourses, relations):
    # Build graph and in-degree dictionary
    graph = {}
    in_degree = {}
    for i in range(1, numCourses + 1):
        graph[i] = []
        in_degree[i] = 0
    for prev, course in relations:
        graph[prev].append(course)
        in_degree[course] += 1

    # Construct queue from relations
    queue = deque()
    for course in range(1, numCourses + 1):
        if in_degree[course] == 0:
            queue.append(course)

    semesters = 0
    takenCourses = 0

    # Process each level (semester) using BFS
    while queue:

```

Graphs - LeetCode - By Pattern — 30 — LeetCode

```

semesters == 1
current_level_size = len(queues)
for _ in range(current_level_size):
    course = queues.popLeft()
    takenCourses += 1
    # Increment in-degrees for neighboring courses.
    for neighbor in graph[course]:
        in_degree[neighbor] += 1
        if in_degree[neighbor] == 0:
            queues.append(neighbor)

# If not all courses have been taken, a cycle exists.
return semesters if takenCourses == numCourses else -1

# Example usage:
print(minimumSemesters(3, [[1, 3], [2, 3]])) # Expected output: 2

```

Python

```

from enum import Enum

class State(Enum):
    INIT = 0
    VISITING = 1
    VISITED = 2

class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        graph = [[] for _ in range(n)]
        states = [State.INIT] * n
        depth = [1] * n

        for u, v in relations:
            graph[u].append(v - 1)

        def hasCycle(u: int) -> bool:
            if states[u] == State.VISITING:
                return True
            if states[u] == State.VISITED:
                return False
            states[u] = State.VISITING
            for v in graph[u]:
                if hasCycle(v):
                    return True
            depth[u] = max(depth[u], 1 + depth[v])
            states[u] = State.VISITED
            return False

        if any(hasCycle(i) for i in range(n)):
            return -1
        return max(depth)

```

Python

Graphs - LeetCode - By Pattern

— 31 —

LeetCode

```

class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        graph = [[] for _ in range(n)]
        indegrees = [0] * n

        # Build the graph.
        for u, v in relations:
            graph[u].append(v - 1)
            indegrees[v - 1] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(indegrees) if d == 0])

        step = 0
        while q:
            for _ in range(len(q)):
                u = q.popleft()
                n -= 1
                for v in graph[u]:
                    indegrees[v] -= 1
                    if indegrees[v] == 0:
                        q.append(v)
            step += 1
            return step if n == 0 else -1

```

Python

```

class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for prev, next in relations:
            prev, next = prev - 1, next - 1
            g[prev].append(next)
            indeg[next] += 1
        q = deque()
        for i, v in enumerate(indeg):
            if v == 0:
                ans += 1
                while q:
                    for _ in range(len(q)):
                        i = q.popleft()
                        n -= 1
                        for j in g[i]:
                            indeg[j] -= 1
                            if indeg[j] == 0:
                                q.append(j)
                return -1 if n else ans

```

Python

Graphs - LeetCode - By Pattern

— 32 —

LeetCode

```

class Solution {
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for prev, next in relations:
            prev, next = prev - 1, next - 1
            g[prev].append(next)
            indeg[next] += 1
        q = deque()
        for i, v in enumerate(indeg):
            if v == 0:
                ans += 1
                while q:
                    for _ in range(len(q)):
                        i = q.popleft()
                        n -= 1
                        for j in g[i]:
                            indeg[j] -= 1
                            if indeg[j] == 0:
                                q.append(j)
                return -1 if n else ans

```

C++

```

class Solution {
public:
    int minimumSemesters(int n, vector<vector<int>>& relations) {
        vector<vector<int>> g(n);
        vector<int> indeg(n);
        for (auto& r : relations) {
            int prev = r[0] - 1, next = r[1] - 1;
            g[prev].push_back(next);
            ++indeg[next];
        }
        queue<int> q;
        for (int i = 0; i < n; ++i) {
            if (indeg[i] == 0) {
                q.push(i);
            }
        }
        int ans = 0;
        while (!q.empty()) {
            ++ans;
            for (int k = q.size(); k > 0; --k) {
                int i = q.front();
                q.pop();
                for (int& j : g[i]) {
                    if (--indeg[j] == 0) {
                        q.push(j);
                    }
                }
            }
        }
        return ans == 0 ? ans : -1;
    }
};

```

C++

Graphs - LeetCode - By Pattern

— 33 —

LeetCode

```

class Solution {
    public int minimumSemesters(int n, List<List<Integer>> relations) {
        List<Integer> g = new List<Integer>();
        Arrays.sort(g, k -> new ArrayList<>());
        int[] indeg = new int[n];
        for (var r : relations) {
            int prev = r[0] - 1, next = r[1] - 1;
            g[prev].add(next);
            ++indeg[next];
        }
        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; ++i) {
            if (indeg[i] == 0) {
                q.offer(i);
            }
        }
        int ans = 0;
        while (!q.isEmpty()) {
            ++ans;
            for (int k = q.size(); k > 0; --k) {
                int i = q.poll();
                --n;
                for (int& j : g[i]) {
                    if (--indeg[j] == 0) {
                        q.offer(j);
                    }
                }
            }
        }
        return ans == 0 ? ans : -1;
    }
}

```

JavaScript

Graphs - LeetCode - By Pattern

— 34 —

LeetCode

```

function minimumSemesters(n: number, relations: number[][]): number {
    const g: number[][] = Array.from({length: n}, () => []);
    const indeg = new Array(n).fill(0);
    for (const [prev, next] of relations) {
        g[prev].push(next);
        indeg[next]++;
    }
    const q: number[] = [];
    for (let i = 0; i < n; ++i) {
        if (indeg[i] == 0) {
            q.push(i);
        }
    }
    let ans = 0;
    while (q.length) {
        ++ans;
        for (let k = q.length; k > 0; --k) {
            const i = q.shift();
            --n;
            for (const j of g[i]) {
                if (--indeg[j] == 0) {
                    q.push(j);
                }
            }
        }
    }
    return ans == 0 ? ans : -1;
}

```

JavaScript

```

function minimumSemesters(n: number, relations: number[][]): number {
    const g: number[][] = Array.from({length: n}, () => []);
    const indeg = new Array(n).fill(0);
    for (const [prev, next] of relations) {
        g[prev].push(next);
        indeg[next]++;
    }
    const q: number[] = [];
    for (let i = 0; i < n; ++i) {
        if (indeg[i] == 0) {
            q.push(i);
        }
    }
    let ans = 0;
    while (q.length) {
        ++ans;
        for (let k = q.length; k > 0; --k) {
            const i = q.shift();
            --n;
            for (const j of g[i]) {
                if (--indeg[j] == 0) {
                    q.push(j);
                }
            }
        }
    }
    return ans == 0 ? ans : -1;
}

```

JavaScript

Graphs - LeetCode - By Pattern

— 35 —

LeetCode

```

    return ans == 0 ? ans : -1;
}

```

Go

```

func minimumSemesters(n int, relations [][]int) (ans int) {
    g := make([][]int, n)
    indeg := make([]int, n)
    for i, r := range relations {
        prev, next := r[0]-1, r[1]-1
        g[prev] = append(g[prev], next)
        indeg[next]++
    }
    q := []int{}
    for i, v := range indeg {
        if v == 0 {
            q = append(q, i)
        }
    }
    for len(q) > 0 {
        ans++
        for k := len(q); k > 0; k-- {
            i := q[0]
            q = q[1:]
            for _, j := range g[i] {
                indeg[j]--
                if indeg[j] == 0 {
                    q = append(q, j)
                }
            }
        }
    }
    if n == 0 {
        return
    }
    return -1
}

```

Go

Graphs - LeetCode - By Pattern

— 36 —

LeetCode

```

def minTimeToMasterInLet, relations []][let] (ans let) {
  g := make([][]let, 4)
  indg := make([]let, 4)
  for i, r := range relations {
    pre, suf := c[i][2], c[i][1]
    g[pre] = append(g[pre], suf)
    indg[suf]++
  }
  q := []int{}
  for i, v := range indg {
    if v == 0 {
      q = append(q, i)
    }
  }
  for len(q) > 0 {
    ans++
    for k := len(q); k > 0; k-- {
      i := q[k]
      q = q[:k]
      for j := range g[i] {
        indg[j]--
        if indg[j] == 0 {
          q = append(q, j)
        }
      }
    }
  }
  if n == 0 {
    return
  }
  return -1
}

```

[1] Graphs — Pattern Solution (matched from community answers)
Python

Graphs · LeetCode — By Pattern

— 37 —

LeetCode

```

# Python Implementation
from collections import deque

def minTimeToMasterInLet, relations:
    # Build graph and in-degree dictionary
    graph = {}
    in_degree = {}
    for i in range(1, numCourses + 1):
        in_degree[i] = 0
        graph[i] = []

    # Construct graph from relations
    for pre, course in relations:
        graph[pre].append(course)
        in_degree[course] += 1

    # Initialize queue with all courses having no prerequisites
    queue = deque()
    for course in range(1, numCourses + 1):
        if in_degree[course] == 0:
            queue.append(course)

    numCourses = 0
    takenCourses = 0

    # Process each level (semester) using BFS
    while queue:
        numCourses += 1
        current_level_size = len(queue)
        for i in range(current_level_size):
            course = queue.popleft()
            takenCourses += 1

            # Decrease in-degree for neighboring courses
            for neighbor in graph[course]:
                in_degree[neighbor] -= 1
                if in_degree[neighbor] == 0:
                    queue.append(neighbor)

        # If not all courses have been taken, a cycle exists
        return numCourses if takenCourses == numCourses else -1

# Example usage
print(minTimeToMasterInLet(3, [[1, 2], [2, 3]])) # Expected output: 2

```

#305 **HARD**
Number of Islands II
LeetCode · SimplifyLent · WalkCC · LeetCode · Editorial
Array · Hash Table · Union Find · Matrix
Lisc Premium 98

Problem:
You are given an empty 2D binary grid grid of size m x n. The grid represents a map where 0's represent water and 1's represent land. Initially, all the cells of grid are water cells (i.e., all the cells are 0's).

Graphs · LeetCode — By Pattern

— 38 —

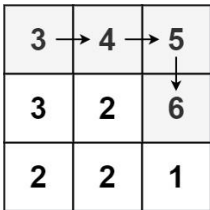
LeetCode

We may perform an add land operation which turns the water at position into a land. You are given an array positions where positions[i] = [r, c] is the position (r, c) at which we should operate the ith operation.

Return an array of integers answer where answer[i] is the number of islands after turning the cell (r, c) into a land.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:



Input: m = 3, n = 3, positions = [[0,0],[0,1],[1,2],[2,1]]
Output: [1,1,2,3]
Explanation:
Initially, the 2d grid is filled with water.
- Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land. We have 1 island.
- Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land. We still have 1 island.
- Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land. We have 2 islands.
- Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land. We have 3 islands.

Example 2:
Input: m = 1, n = 1, positions = [[0,0]]
Output: [1]

Constraints:

- 1 ≤ m, n, positions.length ≤ 104
- 1 ≤ m * n ≤ 104
- positions[i].length == 2
- 0 ≤ r < m
- 0 ≤ c < n

Graphs · LeetCode — By Pattern

— 39 —

LeetCode

Follow up: Could you solve it in time complexity O(k log(mn)), where k == positions.length?

```

# Python
# Brute Force Approach
class Solution:
    def numIslands2(self, m, n, positions):
        # Brute Force - try all paths via exhaustive DFS
        visited = set()
        def dfs(node):
            if node == dest: return True
            visited.add(node)
            for nei in graph.get(node, []):
                if nei not in visited and dfs(nei): return True
            visited.remove(node)
            return False
        return dfs(dest)

```

```

# Python
# Union Find Implementation using Union Find with path compression and union by rank
class UnionFind:
    def __init__(self, size):
        # parent array: parent of each node, -1 indicates water
        self.parent = [-1] * size
        self.rank = [0] * size
        self.count = 0 # number of islands

    def find(self, x):
        # find with path compression
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # union two nodes, if union is successful, reduce island count
        rootx = self.find(x)
        rooty = self.find(y)
        if rootx == rooty:
            return False
        # union by rank
        if self.rank[rootx] < self.rank[rooty]:
            self.parent[rootx] = rooty
        elif self.rank[rootx] > self.rank[rooty]:
            self.parent[rooty] = rootx
        else:
            self.parent[rootx] = rooty
            self.rank[rootx] += 1

```

Graphs · LeetCode — By Pattern

— 40 —

LeetCode

```

else:
    self.parent[rootY] = rootX
    self.rank[rootX] += 1
    self.count -= 1
    return True

class Solution:
    def numIslands2(self, m, n, positions):
        uf = UnionFind(m * n)
        result = 0
        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]

        for r, c in positions:
            idx = r * n + c
            # If the position is already land, skip
            if uf.parent[idx] != -1:
                result.append(uf.count)
                continue

            # Mark this cell as land forming a new island
            uf.parent[idx] = idx
            uf.count += 1

            # Union with all 4 neighbors if they are already land
            for dx, dy in directions:
                nr, nc = r + dx, c + dy
                nidx = nr * n + nc
                if 0 <= nr < m and 0 <= nc < n and uf.parent[nidx] != -1:
                    uf.union(idx, nidx)
                    result.append(uf.count)
            return result

# Example run
# s = Solution()
# print(s.numIslands2(3, 3, [[0,0],[0,1],[1,2],[2,1]]))

```

Python

Graphs · LeetCode — By Pattern

— 41 —

LeetCode

```

class UnionFind:
    def __init__(self, m, n):
        self.id = [-1] * m
        self.rank = [0] * n

    def unionByRank(self, u, l, v, l):
        u = self.find(u)
        l = self.find(l)
        if u == l:
            return
        if self.rank[u] < self.rank[l]:
            self.id[u] = l
        elif self.rank[u] > self.rank[l]:
            self.id[l] = u
        else:
            self.id[u] = l
            self.rank[l] += 1

    def find(self, u, l):
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def numIslands2(self, m, n, positions):
        self.id = [-1] * m
        self.rank = [0] * n
        positions = list(list(it))
        DIRS = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        ans = 0
        seen = [False] * n
        uf = UnionFind(m, n)
        count = 0

        def getID(i, l, j, l):
            return i * n + j

        for i, j in positions:
            if seen[i][j]:
                continue
            seen[i][j] = True
            id = getID(i, j, n)
            uf.id[id] = id
            count += 1
            for dx, dy in DIRS:
                x = i + dx

```

Graphs · LeetCode — By Pattern

— 42 —

LeetCode

```

        y = j + dy
        if x < 0 or x == n or y < 0 or y == n:
            continue
        neighborId = getId(x, y, n)
        if uf.isInNeighborId == -1: # water
            continue
        currentParent = uf.findId()
        neighborParent = uf.findInNeighborId()
        if currentParent != neighborParent:
            uf.unionByRank(currentParent, neighborParent)
            count += 1
        ans.append(count)
    return ans

Python
class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        def check(i, j):
            return 0 < i < m and 0 < j < n and grid[i][j] == 1

        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(m * n))
        grid = [[0] * n for _ in range(m)]
        ans = []
        cur = 0 # current island count
        for i, j in positions:
            if grid[i][j] == 1: # already counted, same as previous island count
                res.append(cur)
                continue
            grid[i][j] = 1
            cur += 1
            for x, y in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                if check(i + x, j + y) and find(i + n * j) != find((i + x) + n * j + y):
                    p[find(i + n * j)] = find((i + x) + n * j + y)
                    cur += 1
            ans.append(cur)
            return ans

```

```

        # 0 1 0
        # for above, if setting left-row 2nd-col 0 to 1
        # cur will = 1 for 3 times.
        # but cur == 1 after grid[i][j] = 1 so cur final result is 1
        res.append(cur)
        return res

class UnionFind(object):
    def __init__(self, n, m):
        self.id = [-1 for i in range(n * m)]
        self.rank = [0 for _ in range(n * m)]

    def find(self, x):
        if self.id[x] != x:
            self.id[x] = self.find(self.id[x])
        return self.id[x]

    def union(self, x, y):
        # x & y is a tuple (x, y), with x and y value inside
        x, y = map(self.find, x, y)
        if x == y:
            return False
        if self.rank[x] < self.rank[y]:
            self.id[x] = y
        elif self.rank[x] > self.rank[y]:
            self.id[y] = x
        else:
            self.id[y] = x # search to the left, to find parent
            self.rank[x] += 1 # now x is parent, so +1 its rank
        return True

class Solution(object):
    def numIslands2(self, m, n, positions):
        # type: m: int
        # type: n: int
        # type: positions: List[List[int]]
        # type: List[int]

        uf = UnionFind(m, n)
        ans = []
        res = []
        dirs = [(0, -1), (0, 1), (1, 0), (-1, 0)]
        grid = set()
        for i, j in positions:
            cur = 0
            grid |= {(i, j)}
            for di, dj in dirs:
                ni, nj = i + di, j + dj
                if 0 < ni < m and 0 < nj < n and (ni, nj) in grid:
                    if uf.union(ni + n * nj, i + n * j):
                        ans += 1
            res.append(ans)

```

```

C++
class UnionFind {
public:
    vector<int> id;

    UnionFind(int n) : id(n, -1), rank(n, 0) {}

    void unionByRank(int a, int b) {
        const int i = find(a);
        const int j = find(b);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    int find(int a) {
        return id[a] == a ? a : id[a] = find(id[a]);
    }

private:
    vector<int> rank;
};

class Solution {
public:
    vector<int> numIslands2(int m, int n, vector<vector<int>>& positions) {
        constexpr int dirs[4][2] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        vector<int> ans;
        vector<vector<bool>> unionfs, vector<bool>(n));
        UnionFind uf(n * m);
        int count = 0;

        for (const vector<int>& p : positions) {
            const int i = p[0];
            const int j = p[1];
            if (count[i][j]) {
                ans.push_back(count);
                continue;
            }
            bool isWater = true;
            const int id = getId(i, j, n);
            uf.id[id] = id;
            ++count;
            for (const auto& [dx, dy] : dirs) {

```

```

        const int x = i + dx;
        const int y = j + dy;
        if (x < 0 || x == m || y < 0 || y == n)
            continue;
        const int neighborId = getId(x, y, n);
        if (uf.id[neighborId] == -1) // water
            continue;
        const int currentRoot = uf.find(id);
        const int neighborRoot = uf.find(neighborId);
        if (currentRoot != neighborRoot) {
            uf.unionByRank(currentRoot, neighborRoot);
            ++count;
        }
        ans.push_back(count);
    }

    return ans;
}

private:
int getId(int i, int j, int n) {
    return i * n + j;
}
};

C++
class UnionFind {
public:
    UnionFind(int n) {
        p = vector<int>(n);
        size = vector<int>(n, 1);
        kate(p.begin(), p.end(), 0);
    }

    bool unite(int a, int b) {
        int pa = find(a), pb = find(b);
        if (pa == pb) {
            return false;
        }
        if (size[pa] < size[pb]) {
            p[pb] = pa;
            size[pa] += size[pb];
        } else {
            p[pa] = pb;
            size[pb] += size[pa];
        }
        return true;
    }

    int find(int a) {
        if (p[a] != a) {

```

```

        p[x] = find(p[x]);
    }
    return p[x];
}

private:
vector<int> p, size;
};

class Solution {
public:
    vector<int> numIslands2(int m, int n, vector<vector<int>>& positions) {
        int grid[m][n];
        memset(grid, 0, sizeof(grid));
        UnionFind uf(n * m);
        int dirs[4] = {-1, 0, 1, 0, -1};
        int cnt = 0;
        vector<int> ans;
        for (auto& p : positions) {
            int i = p[0], j = p[1];
            if (grid[i][j]) {
                ans.push_back(cnt);
                continue;
            }
            grid[i][j] = 1;
            ++cnt;
            for (int k = 0; k < 4; ++k) {
                int x = i + dirs[k], y = j + dirs[k + 1];
                if (x < 0 || x > m || y < 0 || y > n || grid[x][y] || uf.union(i + n * j, x + n * y) {
                    ++cnt;
                }
            }
            ans.push_back(cnt);
        }
        return ans;
    }
};

Java
class Solution {
    public List<Integer> numIslands2(int m, int n, List<int> positions) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        List<Integer> ans = new ArrayList<>();
        boolean[][] seen = new boolean[m][n];
        UnionFind uf = new UnionFind(m * n);
        int count = 0;

        for (int[] p : positions) {
            final int i = p[0];
            final int j = p[1];
            if (seen[i][j]) {
                ans.add(count);
                continue;
            }
            seen[i][j] = true;
            final int id = getId(i, j, n);
            uf.id[id] = id;
            ++count;
            for (int[] d : DIRS) {
                int x = i + d[0], y = j + d[1];
                if (x < 0 || x > m || y < 0 || y > n || grid[x][y] || uf.union(i + n * j, x + n * y) {
                    ++count;
                }
            }
            ans.add(count);
        }
        return ans;
    }
}

```

```

class UnionFind {
    public int[] id;

    public UnionFind(int n) {
        id = new int[n];
        rank = new int[n];
        Arrays.fill(id, -1); // water
    }

    public void unionByRank(int a, int b) {
        final int i = find(a);
        final int j = find(b);
        if (i == j)
            return;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
    }

    public int find(int a) {
        return id[a] == a ? a : id[a] = find(id[a]);
    }

    private int[] rank;
}

class Solution {
    public List<Integer> numIslands2(int m, int n, List<int> positions) {
        final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        List<Integer> ans = new ArrayList<>();
        boolean[][] seen = new boolean[m][n];
        UnionFind uf = new UnionFind(m * n);
        int count = 0;

        for (int[] p : positions) {
            final int i = p[0];
            final int j = p[1];
            if (seen[i][j]) {
                ans.add(count);
                continue;
            }
            seen[i][j] = true;
            final int id = getId(i, j, n);
            uf.id[id] = id;
            ++count;
            for (int[] d : DIRS) {
                int x = i + d[0], y = j + d[1];
                if (x < 0 || x > m || y < 0 || y > n || grid[x][y] || uf.union(i + n * j, x + n * y) {
                    ++count;
                }
            }
            ans.add(count);
        }
        return ans;
    }
}

```

```

for (int[] dir : DIRS) {
    final int x = i + dir[0];
    final int y = j + dir[1];
    if (x < 0 || x == n || y < 0 || y == n)
        continue;
    final int neighborId = getId(x, y, n);
    if (uf.getId(neighborId) == -1) // water
        continue;
    final int currentParent = uf.findId(i);
    final int neighborParent = uf.findId(neighborId);
    if (currentParent != neighborParent) {
        uf.unionByRank(currentParent, neighborParent);
        --count;
    }
}
ans.add(count);
}
return ans;
}

private int getId(int i, int j, int n) {
    return i * n + j;
}
}

```

```

//Java
class UnionFind {
    private final int[] p;
    private final int[] size;

    public UnionFind(int n) {
        p = new int[n];
        size = new int[n];
        for (int i = 0; i < n; ++i) {
            p[i] = i;
            size[i] = 1;
        }
    }

    public int find(int x) {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }

    public boolean union(int a, int b) {
        int pa = find(a), pb = find(b);
        if (pa == pb) {
            return false;
        }
    }
}

```

```

if (size[pa] > size[pb]) {
    p[pb] = pa;
    size[pa] += size[pb];
} else {
    p[pa] = pb;
    size[pb] += size[pa];
}
return true;
}

class Solution {
    public List<Integer> numIslands2(int n, int m, int[][] positions) {
        int[][] grid = new int[n][m];
        UnionFind uf = new UnionFind(n * m);
        int[] dirs = {-1, 0, 1, 0, -1};
        List<Integer> ans = new ArrayList<>();
        for (var p : positions) {
            int i = p[0], j = p[1];
            if (grid[i][j] == 1) {
                ans.add(0);
                continue;
            }
            grid[i][j] = 1;
            --count;
            for (int k = 0; k < 4; ++k) {
                int x = i + dirs[k], y = j + dirs[k + 1];
                if (x == 0 && x == n && y == 0 && y < n && grid[x][y] == 1)
                    && uf.union(i * n + j, x * n + y) {
                        --count;
                    }
            }
            ans.add(count);
        }
        return ans;
    }
}

```

TypeScript
TypeScript

```

class UnionFind {
    p: number[];
    size: number[];
    constructor(n: number) {
        this.p = Array(n)
        this.size = Array(n).fill(1);
    }

    find(x: number): number {
        if (this.p[x] == x) {
            this.p[x] = this.find(this.p[x]);
        }
        return this.p[x];
    }

    union(a: number, b: number): boolean {
        const [pa, pb] = [this.find(a), this.find(b)];
        if (pa == pb) {
            return false;
        }
        if (this.size[pa] > this.size[pb]) {
            this.p[pb] = pa;
            this.size[pa] += this.size[pb];
        } else {
            this.p[pa] = pb;
            this.size[pb] += this.size[pa];
        }
        return true;
    }
}

function numIslands2(n: number, m: number, positions: number[][]): number[] {
    const grid: number[][] = Array.from({ length: n }, () => Array(n).fill(0));
    const uf = new UnionFind(n * m);
    const ans: number[] = [];
    const dirs: number[] = [-1, 0, 1, 0, -1];
    let count = 0;
    for (const [i, j] of positions) {
        if (grid[i][j]) {
            ans.push(0);
            continue;
        }
        grid[i][j] = 1;
        --count;
        for (let k = 0; k < 4; ++k) {
            const [x, y] = [i + dirs[k], j + dirs[k + 1]];
            if (x < 0 || x == n || y < 0 || y == n || !grid[x][y]) {
                continue;
            }
        }
    }
}

```

```

}
if (uf.union(i * n + j, x * n + y)) {
    --count;
}
ans.push(count);
}
return ans;
}

```

Go
Go

```

type unionFind struct {
    p, size []int
}

func numIslands2(n int, m int, positions [][]int) []int {
    uf := new(unionFind)
    uf.p, uf.size = make([]int, n*m), make([]int, n*m)
    for i := range n {
        uf.p[i] = i
        uf.size[i] = 1
    }
    return uf.numIslands2(n, m, positions)
}

func (uf *unionFind) find(int x int) int {
    if uf.p[x] != x {
        uf.p[x] = uf.find(uf.p[x])
    }
    return uf.p[x]
}

func (uf *unionFind) union(a, b int) bool {
    pa, pb := uf.find(a), uf.find(b)
    if pa == pb {
        return false
    }
}

```

```

}
if (uf.size[pa] > uf.size[pb]) {
    uf.p[pb] = pa;
    uf.size[pa] += uf.size[pb];
} else {
    uf.p[pa] = pb;
    uf.size[pb] += uf.size[pa];
}
return true;
}

func numIslands2(n int, m int, positions [][]int) (ans []int) {
    uf := new(unionFind)
    grid := make([][]int, n)
    for i := range grid {
        grid[i] = make([]int, m)
    }
    dirs := [][]int{-1, 0, 1, 0, -1}
    count = 0
    for i := range positions {
        x, y := positions[i][0], positions[i][1]
        if grid[x][y] == 1 {
            ans = append(ans, 0)
            continue
        }
        grid[x][y] = 1
        --count
        for k := 0; k < 4; ++k {
            x2, y2 := x + dirs[k], y + dirs[k + 1]
            if x2 < 0 && x2 == n && y2 < 0 && y2 == n && grid[x2][y2] == 1 && uf.union(x, y, x2, y2) {
                --count
            }
        }
        ans = append(ans, count)
    }
    return
}

```

[*] Graphs - Pattern Solution (matched from community answers)

Python

```

# Python implementation using Union Find with path compression and union by rank.
class UnionFind:
    def __init__(self, size):
        # parent: stores parent of each node. -1 indicates water.
        self.parent = [-1] * size
        self.rank = [0] * size
        self.count = 0 # number of islands

    def find(self, x):
        # find with path compression.
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union the sets, if union is successful, reduce island count.
        rootx = self.find(x)
        rooty = self.find(y)
        if rootx == rooty:
            return False
        # Union by rank.
        if self.rank[rootx] < self.rank[rooty]:
            self.parent[rootx] = rooty
        elif self.rank[rootx] > self.rank[rooty]:
            self.parent[rooty] = rootx
        else:
            self.parent[rootx] = rootx
            self.parent[rooty] = rootx
            self.count += 1
        return True

class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        uf = UnionFind(m * n)
        result = []
        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]

        for r, c in positions:
            land = r * m + c
            # If the position is already land, skip.
            if uf.parent[land] != -1:
                result.append(uf.count)
                continue
            # Mark this cell as land forming a new island.
            uf.parent[land] = land
            uf.count += 1

            # Union with all 4 neighbors if they are already land.
            for dr, dc in directions:

```

```

        ar, ac = r + dr, c + dc
        side = ar + b + ac
        if b == ar + a and b == ac + a and uf.parent[side] != -1:
            uf.union(side, side)
        result.append(ar, count)
        return result

# Example run:
# a = Solution()
# print(a.canConvert(123, 2, (10,47,(0,1),(1,2),(2,1)))))

```

#1153 **HARD** String Transforms Into Another String

LeetCode · [SimplifyLast](#) · [WuJiCC](#) · [LeetCode](#) · [Editorial](#)

Hash Table · String · [Union Find](#)

Little Denny Zhang

Problem:

Given two strings `str1` and `str2` of the same length, determine whether you can transform `str1` into `str2` by doing zero or more conversions.

In one conversion you can convert all occurrences of one character in `str1` to any other lowercase English character.

Return true if and only if you can transform `str1` into `str2`.

Example 1:

Input: `str1` = "aabcc", `str2` = "ccdee"

Output: true

Explanation: Convert 'c' to 'e' then 'b' to 'd' then 'a' to 'c'. Note that the order of conversions matter.

Example 2:

Input: `str1` = "leetcode", `str2` = "codeleet"

Output: false

Explanation: There is no way to transform `str1` to `str2`.

Constraints:

- 1 <= `str1.length` == `str2.length` <= 104
- `str1` and `str2` contain only lowercase English letters.

Python

Python

Graphs · LeetCode · By Pattern

— 55 —

LeetCodeMastery

```

# Python solution with line-by-line comments
def canConvert(str1: str, str2: str) -> bool:
    # If both strings are equal, no conversion is needed.
    if str1 == str2:
        return True

    # Dictionary to hold the mapping from characters in str1 to characters in str2
    mapping = {}

    # Iterate through the characters of both strings simultaneously.
    for c1, c2 in zip(str1, str2):
        # If c1 is already in the mapping, check for consistency.
        if c1 in mapping:
            if mapping[c1] != c2:
                # Found an inconsistent mapping.
                return False
            else:
                mapping[c1] = c2

    # Count distinct characters in str2.
    # If str2 uses all 26 letters, then there is no extra character available to break cycles.
    if len(set(str2)) == 26:
        return False

    return True

# Example Usage:
print(canConvert("abcd", "cdab")) # Expected True
print(canConvert("abcde", "cdeab")) # Expected False

```

Python

```

class Solution:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True

        mappings = {}

        for a, b in zip(str1, str2):
            if mappings.get(a, b) != b:
                return False
            mappings[a] = b

        # No letter in the str1 maps to > 1 letter in the str2 and there is at
        # least one temporary letter can break any loops.
        return len(set(str2)) < 26

```

Python

Graphs · LeetCode · By Pattern

— 56 —

LeetCodeMastery

```

class Solution {
public:
    bool canConvert(string str1, string str2) {
        if (str1 == str2)
            return true;
        if (str1.length() != str2.length())
            return false;
        vector<int> d(26, 0);
        for (int i = 0; i < str1.length(); ++i) {
            if (str1[i] == str2[i])
                continue;
            if (d[str1[i] - 'a'] != 0)
                return false;
            d[str1[i] - 'a'] = str2[i] - 'a';
        }
        return true;
    }
};

```

Python

```

class Solution:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True
        if len(str1) != len(str2):
            return False
        d = {}
        for a, b in zip(str1, str2):
            if a not in d:
                d[a] = b
            elif d[a] != b:
                return False
        return True

```

C++

C++

```

class Solution {
public:
    bool canConvert(string str1, string str2) {
        if (str1 == str2)
            return true;
        vector<int> mappings(26);
        for (int i = 0; i < str1.length(); ++i) {
            const int a = str1[i];
            const int b = str2[i];
            if (mappings[a] != 0 && mappings[a] != b)
                return false;
            mappings[a] = b;
        }
        // No letter in the str1 maps to > 1 letter in the str2 and there is at
        // least one temporary letter can break any loops.
        return unordered_set<char>(str2.begin(), str2.end()).size() < 26;
    }
};

```

Java

Java

Graphs · LeetCode · By Pattern

— 57 —

LeetCodeMastery

```

class Solution {
    public boolean canConvert(String str1, String str2) {
        if (str1.equals(str2))
            return true;

        Map<Character, Character> mappings = new HashMap<>();

        for (int i = 0; i < str1.length(); ++i) {
            final char a = str1.charAt(i);
            final char b = str2.charAt(i);
            if (mappings.containsKey(a) && mappings.get(a) != b)
                return false;
            mappings.put(a, b);
        }

        // No letter in the str1 maps to > 1 letter in the str2 and there is at
        // least one temporary letter can break any loops.
        return new HashSet<Character>(mappings.values()).size() < 26;
    }
}

```

Java

```

class Solution {
    public boolean canConvert(String str1, String str2) {
        if (str1.equals(str2)) {
            return true;
        }
        int n = str1.length();
        int[] cnt = new int[26];
        for (int i = 0; i < n; ++i) {
            if (str1.charAt(i) == str2.charAt(i))
                continue;
            if (cnt[str1.charAt(i) - 'a'] == 1) {
                return false;
            }
            cnt[str1.charAt(i) - 'a'] = 1;
        }
        for (int i = 0; i < n; ++i) {
            int a = str1.charAt(i);
            int b = str2.charAt(i);
            if (a == b)
                continue;
            if (cnt[a - 'a'] == 1 && cnt[b - 'a'] == 1) {
                return false;
            }
            cnt[a - 'a'] = 1;
        }
        return true;
    }
}

```

Java

Graphs · LeetCode · By Pattern

— 58 —

LeetCodeMastery

```

class Solution {
    public boolean canConvert(String str1, String str2) {
        if (str1.equals(str2)) {
            return true;
        }
        int n = str1.length();
        int[] cnt = new int[26];
        for (int i = 0; i < n; ++i) {
            if (str1.charAt(i) == str2.charAt(i))
                continue;
            if (cnt[str1.charAt(i) - 'a'] == 1) {
                return false;
            }
            cnt[str1.charAt(i) - 'a'] = 1;
        }
        for (int i = 0; i < n; ++i) {
            int a = str1.charAt(i);
            int b = str2.charAt(i);
            if (a == b)
                continue;
            if (cnt[a - 'a'] == 1 && cnt[b - 'a'] == 1) {
                return false;
            }
            cnt[a - 'a'] = 1;
        }
        return true;
    }
}

```

TypeScript

TypeScript

```

function canConvert(str1: string, str2: string): boolean {
    if (str1 === str2) {
        return true;
    }
    if (new Set(str1).size !== new Set(str2).size) {
        return false;
    }
    const d: Map<string, string> = new Map();
    for (const [i, c1] of str1.split('')) {
        const c2 = str2[i];
        if (d.get(c1) !== c2) {
            return false;
        }
    }
    return true;
}

```

TypeScript

Graphs · LeetCode · By Pattern

— 59 —

LeetCodeMastery

```

function canConvert(str1: string, str2: string): boolean {
    if (str1 === str2) {
        return true;
    }
    if (new Set(str1).size !== new Set(str2).size) {
        return false;
    }
    const d: Map<string, string> = new Map();
    for (const [i, c1] of str1.split('')) {
        const c2 = str2[i];
        if (d.get(c1) !== c2) {
            return false;
        }
    }
    return true;
}

```

Go

Go

```

func canConvert(str1 string, str2 string) bool {
    if str1 == str2 {
        return true
    }
    s := map[byte]byte{}
    for i, c := range str2 {
        if s[c] != 0 {
            return false
        }
        s[c] = str1[i]
    }
    return true
}

```

Go

Graphs · LeetCode · By Pattern

— 60 —

LeetCodeMastery

```
func canConvert(str1: string, str2: string) bool {
    if str1 == str2 {
        return true
    }
    n := max(len(str1), len(str2))
    for i, c := range str1 {
        if i < n {
            if str2[i] == c {
                return true
            }
        }
    }
    d := [26][26]
    for i, c := range str1 {
        a, b := int(c-'a'), int(str2[0]-'a')
        if d[a][b] == 0 {
            d[a][b] = 1
        } else if d[a][b] == 2 {
            return false
        }
    }
    return true
}
```

[+] Graphs — Pattern Solution (stored optimal)

Python

```
# Python solution with line-by-line comments
def canConvert(str1: str, str2: str) -> bool:
    # If both strings are equal, no conversion is needed.
    if str1 == str2:
        return True

    # Dictionary to hold the mapping from characters in str1 to characters in str2
    mapping = {}

    # Iterate through the characters of both strings simultaneously.
    for c1, c2 in zip(str1, str2):
        # If c1 is already in the mapping, check for consistency.
        if c1 in mapping:
            if mapping[c1] != c2:
                # Found an inconsistent mapping.
                return False
            else:
                mapping[c1] = c2

    # Count distinct characters in str2.
    # If str1 uses all 26 letters, then there is no extra character available to break cycles.
    if len(set(str2)) == 26:
        return False

    return True

# Example Usage:
print(canConvert("abc", "cde")) # Expected: True
print(canConvert("abc", "cde")) # Expected: False
```

C++

Graphs · LeetCode · By Pattern

— 61 —

LeetCode

```
// C++ solution with line-by-line comments
#include <iostream>
#include <unordered_map>
#include <unordered_set>
#include <string>
using namespace std;

class Solution {
public:
    bool canConvert(string str1, string str2) {
        // If the strings are already equal, no conversion is necessary.
        if (str1 == str2) return true;

        // Mapping dictionary from characters in str1 to str2.
        unordered_map<char, char> mapping;

        // Check each character pair.
        for (int i = 0; i < str1.size(); i++) {
            char c1 = str1[i], c2 = str2[i];
            // If already mapped, verify consistency.
            if (mapping.count(c1)) {
                if (mapping[c1] != c2)
                    return false;
            } else {
                mapping[c1] = c2;
            }
        }

        // Count distinct characters in str2.
        unordered_set<char> distinctChars(str2.begin(), str2.end());
        // If str2 contains more than 26 distinct characters, no temporary character is available.
        if (distinctChars.size() == 26)
            return false;

        return true;
    }
};

// Example Usage:
// int main() {
//     Solution sol;
//     cout << boolalpha << sol.canConvert("abc", "cde") << endl; // Expected: true
//     cout << boolalpha << sol.canConvert("abc", "cde") << endl; // Expected: false
//     return 0;
// }
```

Graphs · LeetCode · By Pattern

— 62 —

LeetCode

Quick Review — Graphs 5 questions · insights · complexity · solution

#128 Longest Consecutive Sequence [Medium]

Key Insights

- Utilize a set (or hash table) for constant time look-ups.
- Only start counting a sequence from a number that is the beginning (its previous number is not in the set).
- Each number is visited only once, ensuring an O(n) time complexity.
- Update the maximum sequence length as you discover longer consecutive sequences.

Space & Time

Time Complexity: O(n)
Space Complexity: O(n)

Solution

We first insert all the numbers into a hash set to enable O(1) membership checks. For each number in the set, we check if it is the start of a sequence by ensuring that the number minus one is not in the set. If it is the beginning, we then count all consecutive numbers following it, updating the maximum sequence length when necessary. This method guarantees that we only count each number once, satisfying the O(n) time requirement.

#277 Find the Celebrity [Medium]

Key Insights

- Use pairwise comparisons to eliminate non-celebrities.
- If a candidate knows another person, the candidate cannot be the celebrity.
- Verify the candidate by checking that they do not know anyone and that everyone knows them.
- Achieve the solution in two passes: one for candidate selection and the other for validation.

Space & Time

Time Complexity: O(n) - One pass for candidate selection and one for validation.
Space Complexity: O(1) - Only constant extra space is used.

Solution

The approach consists of two main steps:
- Candidate Selection: Iterate through the people and maintain a candidate. For every person i, if the current candidate knows i, then set candidate = i. This works because if candidate knows i, candidate cannot be the celebrity. - Verification: Check that the candidate does not know any other person and that every other person knows the candidate. If both conditions hold, candidate is the celebrity; otherwise, return -1.
The main data structures used are simple integer variables. The algorithm leverages a two-pass technique to reduce the number of knows API calls while ensuring correctness.

#1136 Parallel Courses [Medium]

Key Insights

- Model the problem as a directed graph where nodes are courses and edges represent prerequisites.
- Use topological sorting (Kahn's algorithm) to process courses with zero prerequisites.
- Each level (or layer) of processing represents one semester.
- Detect cycles by ensuring all courses are eventually processed; if not, a cycle exists.
- The number of BFS levels gives the minimum number of semesters necessary.

Space & Time

Time Complexity: O(n + m) where n is the number of courses and m is the length of relations (prerequisites).

Graphs · LeetCode · By Pattern

— 63 —

LeetCode

distinct characters in str2. If this number is 26 and str1 is not already equal to str2, it is impossible to perform the transformation because there is no spare character available as a temporary placeholder. - If the mapping is consistent and there exists at least one character not used in str2 (i.e., distinct characters in str2 is less than 26), the transformation is possible. The important trick is identifying the need for a free character when the target mapping is "full" (i.e., covers all 26 letters). Without a free character, you cannot avoid cycles where a temporary intermediate is necessary.

Graphs · LeetCode · By Pattern

— 65 —

LeetCode

#2 Greedy — 6 questions · #2 of 21

#409

EASY

Longest Palindrome

LeetCode · SimplifyLogic · HuBCC · LeetCode · Editorial

Hash Table · String · Greedy

Lists · Grid 169

Problem:

Given a string s which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with these letters.
Letters are case sensitive, for example, "Aa" is not considered a palindrome.

Example 1:

Input: s = "abccdd"
Output: 7
Explanation: One longest palindrome that can be built is "daccdd", whose length is 7.

Example 2:

Input: s = "a"
Output: 1
Explanation: The longest palindrome that can be built is "a", whose length is 1.

Constraints:

- 1 <= s.length <= 2000
- s consists of lowercase and/or uppercase English letters only.

Python

Python

Graphs · LeetCode · By Pattern

— 66 —

LeetCode

```
# Python solution for the longest Palindrome problem

def longestPalindrome(s: str) -> int:
    # Create a dictionary to store the frequency of each character
    char_counts = {}
    for char in s:
        # Increment the count for the character
        char_counts[char] = char_counts.get(char, 0) + 1

    # Variable to store the length of the palindrome
    palindrome_length = 0
    # Flag to indicate whether we can add one more character in the middle for an odd count
    center_added = False

    # Iterate through the character counts
    for count in char_counts.values():
        # Add the largest even number from count to palindrome length
        palindrome_length += (count // 2) * 2
        # If count is odd and we haven't added the center character yet
        if count % 2 == 1 and not center_added:
            palindrome_length += 1
            center_added = True

    return palindrome_length

# Example usage
print(longestPalindrome("abcccd"))
```

Python

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        ans = 0
        count = collections.Counter(s)

        for c in count.values():
            ans += c // 2 * 2
            if c % 2 == 1:
                ans += 1

        return ans
```

Python

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        cnt = Counter(s)
        ans = center // 2 * 2
        for v in cnt.values():
            ans += int(v > 1)

        return ans
```

Python

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        odd = defaultdict(int)
        cnt = 0
        for c in s:
            odd[c] += 1
            cnt += 1 if odd[c] % 2 == 0 else 0
        return cnt
```

Greedy · LeetCode · By Patsum

— 67 —

LeetCode

```
Python
class Solution:
    def longestPalindrome(self, s: str) -> int:
        cnt = Counter(s)
        ans = 0
        for v in cnt.values():
            ans += v // 2 * 2
            ans += (v % 2) and (v > 1)
        return ans
```

C++

C++

```
class Solution {
public:
    int longestPalindrome(string s) {
        int ans = 0;
        vector<int> count(128);

        for (const char c : s)
            ++count[c];

        for (const int freq : count)
            ans += freq / 2 * 2 + (freq > 0 ? freq : 1);

        bool hasOddCount =
            ranges::any_of(count, [](int c) { return c % 2 == 1; });
        return ans + hasOddCount;
    }
};
```

Java

Java

```
class Solution {
    public int longestPalindrome(String s) {
        int ans = 0;
        int[] count = new int[128];

        for (final char c : s.toCharArray())
            ++count[c];

        for (final int freq : count)
            ans += freq / 2 * 2 + (freq > 0 ? freq : 1);

        final boolean hasOddCount = Arrays.stream(count).anyMatch(freq -> freq % 2 == 1);
        return ans + (hasOddCount ? 1 : 0);
    }
}
```

Java

Greedy · LeetCode · By Patsum

— 68 —

LeetCode

```
class Solution {
    public int longestPalindrome(String s) {
        int[] cnt = new int[128];
        int n = s.length();
        for (int i = 0; i < n; ++i) {
            ++cnt[s.charAt(i)];
        }
        int ans = 0;
        for (int v : cnt) {
            ans += v / 2 * 2;
        }
        ans += ans < n - 1 ? 1 : 0;
        return ans;
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn longest_palindrome(s: String) -> i32 {
        let mut cnt = HashMap::new();
        for ch in s.chars() {
            *cnt.entry(ch).or_insert(0) += 1;
        }

        let mut ans = 0;
        for &v in cnt.values() {
            ans += v / 2 * 2;
        }

        if ans < (s.len() as i32) {
            ans += 1;
        }

        ans
    }
}
```

Java

```
class Solution {
    public int longestPalindrome(String s) {
        int[] odd = new int[128];
        int n = s.length();
        int cnt = 0;
        for (int i = 0; i < n; ++i) {
            odd[s.charAt(i)] += 1;
        }
        return cnt + odd[0] / 2 * 2 + 1;
    }
}
```

Java

Greedy · LeetCode · By Patsum

— 69 —

LeetCode

```
class Solution {
    public int longestPalindrome(String s) {
        int[] cnt = new int[128];
        for (int i = 0; i < s.length(); ++i) {
            ++cnt[s.charAt(i)];
        }
        int ans = 0;
        for (int v : cnt) {
            ans += v / 2 * 2;
        }
        if (ans < s.length() - 1) {
            ++ans;
        }
        return ans;
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn longest_palindrome(s: String) -> i32 {
        let mut map: HashMap<char, i32> = HashMap::new();
        for c in s.chars() {
            map.insert(c, map.get(&c).unwrap_or(&0) + 1);
        }

        let mut has_odd = false;
        let mut res = 0;
        for v in map.values() {
            res += v / 2 * 2;
            if v % 2 == 1 {
                has_odd = true;
                res += 1;
            }
        }
        ans = if has_odd { 1 } else { 0 };
        return ans;
    }
}
```

TypeScript

TypeScript

```
function longestPalindrome(s: string): number {
    const cnt = new Map<string, number>();
    for (const c of s) {
        cnt[c] = (cnt[c] || 0) + 1;
    }
    let ans = Object.values(cnt).reduce((acc, v) => acc + Math.floor(v / 2) * 2, 0);
    ans = ans < s.length - 1 ? 0 : ans;
    return ans;
}
```

TypeScript

Greedy · LeetCode · By Patsum

— 70 —

LeetCode

```
function longestPalindrome(s: string): number {
    const oddRecord: string[] = [];
    let cnt = 0;
    for (const c of s) {
        oddRecord[c] += 1;
        cnt += oddRecord[c] % 2 === 0 ? 1 : 0;
    }
    return cnt * 2 + (s.length - cnt) % 2 === 0 ? 1 : 0;
}
```

TypeScript

```
function longestPalindrome(s: string): number {
    let n = s.length;
    let ans = 0;
    let record = new Array(128).fill(0);
    for (let i = 0; i < n; ++i) {
        record[s.charCodeAt(i)] += 1;
    }
    for (let i = 0; i < 128; ++i) {
        let count = record[i];
        ans += count / 2 * 2;
        if (count % 2 === 1) {
            ans += 1;
        }
    }
    return ans * 2 + (n - ans) % 2 === 0 ? 1 : 0;
}
```

[*] Greedy — Pattern Solution (stored optimal)

Python

```
# Python solution for the longest Palindrome problem

def longestPalindrome(s: str) -> int:
    # Create a dictionary to store the frequency of each character
    char_counts = {}
    for char in s:
        # Increment the count for the character
        char_counts[char] = char_counts.get(char, 0) + 1

    # Variable to store the length of the palindrome
    palindrome_length = 0
    # Flag to indicate whether we can add one more character in the middle for an odd count
    center_added = False

    # Iterate through the character counts
    for count in char_counts.values():
        # Add the largest even number from count to palindrome length
        palindrome_length += (count // 2) * 2
        # If count is odd and we haven't added the center character yet
        if count % 2 == 1 and not center_added:
            palindrome_length += 1
            center_added = True

    return palindrome_length

# Example usage
print(longestPalindrome("abcccd"))
```

Greedy · LeetCode · By Patsum

— 71 —

LeetCode

```
// C++ solution for the longest Palindrome problem

#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;

int longestPalindrome(string s) {
    // Create a hash table to store frequency of each character
    unordered_map<char, int> charCounts;
    for (char c : s) {
        charCounts[c]++;
    }

    int palindromeLength = 0;
    bool centerAdded = false;

    // Iterate over the character counts
    for (auto &entry : charCounts) {
        int count = entry.second;
        // Add the largest even number from count
        palindromeLength += (count / 2) * 2;
        // If count is odd and no center character has been added yet, add one extra
        if (count % 2 == 1 && !centerAdded) {
            palindromeLength++;
            centerAdded = true;
        }
    }

    return palindromeLength;
}
```

int main() {

string s = "abcccd";

cout << longestPalindrome(s) << endl;

return 0;

#134

MEDIUM

Gas Station

LeetCode · LeetCode · WUCC · LeetCode · Editorial

Array · Greedy

Lists: Grid 169

Problem:

There are n gas stations along a circular route, where the amount of gas at the i th station is $gas[i]$.

You have a car with an unlimited gas tank and it costs $cost[i]$ of gas to travel from the i th station to its next $(i + 1)$ th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and $cost$, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Example 1:

Input: $gas = [1,2,3,4,5]$, $cost = [3,4,5,1,2]$

Output: 3

Explanation: Start at station 3 (index 3) with 0 gas. Travel to station 4. Add 5 gas. Now you have 5 gas, which covers the cost to travel to station 0 (0 gas), to station 1 (1 gas), to station 2 (2 gas), and to station 3 (3 gas). The total cost is 10 gas, which was all provided by station 3.

Example 2:

Input: $gas = [2,3,4]$, $cost = [3,4,3]$

Output: -1

Explanation: No starting station can support the car to travel the circuit once.

Example 1:
Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Output: 3
Explanation:
Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = 8 + 4 = 4
Travel to station 4. Your tank = 4 - 1 + 5 = 8
Travel to station 0. Your tank = 8 - 2 + 1 = 7
Travel to station 1. Your tank = 7 - 3 + 2 = 6
Travel to station 2. Your tank = 6 - 4 + 3 = 5
Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.
Therefore, return 3 as the starting index.

Example 2:
Input: gas = [2,3,4], cost = [3,4,3]
Output: -1
Explanation:
You can't start at station 0 or 1, as there is not enough gas to travel to the next station.
Let's start at station 2 and fill up with 4 unit of gas. Your tank = 8 + 4 = 4
Travel to station 0. Your tank = 4 - 3 + 2 = 3
Travel to station 1. Your tank = 3 - 3 + 3 = 3
You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.
Therefore, you can't travel around the circuit once no matter where you start.

Constraints:

- n == gas.length == cost.length
- 1 <= n <= 105
- 0 <= gas[i], cost[i] <= 104
- The input is generated such that the answer is unique.

>> Brute Force (Greedy) Interview Prep

Python

```
# Brute Force Solution
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        # Brute Force O(n^2) - try starting at every station
        n = len(gas)
        for start in range(n):
            tank = 0
            for stop in range(n):
                i = (start + stop) % n
                tank += gas[i] - cost[i]
                if tank < 0: break
            else:
                return start
        return -1
```

Python

Python

Greedy - LeetCode - By Purnam

— 73 —

LeetCode

```
# Define a function to find starting gas station index
def canCompleteCircuit(gas, cost):
    total_surplus = 0 # Total gas surplus after completing the route
    current_surplus = 0 # Gas surplus while traveling from a candidate start station
    start_index = 0 # Candidate starting station index

    # Loop through each station
    for i in range(len(gas)):
        # Calculate net gas after leaving station i
        surplus = gas[i] - cost[i]
        total_surplus += surplus
        current_surplus += surplus

        # If current surplus becomes negative, station i is not a valid start.
        # Reset starting station to the next station
        if current_surplus < 0:
            current_surplus = 0 # Reset current surplus
            start_index = i + 1 # Next potential start index
    # Check if overall surplus is non-negative
    return start_index if total_surplus >= 0 else -1

# Example usage
gas = [1,2,3,4,5]
cost = [3,4,5,1,2]
print(canCompleteCircuit(gas, cost)) # Output: 3
```

Python

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        ans = 0
        net = 0
        sum = 0

        # Try to start from each index.
        for i in range(len(gas)):
            net = gas[i] - cost[i]
            sum += net
            if sum < 0:
                sum = 0
                ans = i + 1 # Start from the next index.
        return -1 if net < 0 else ans
```

Python

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = 0
        while i < n:
            while i < j:
                i = (i + 1) % n
                while i < j and gas[i] - cost[i] < 0:
                    i = (i + 1) % n
            i = (i + 1) % n
            while i < j and gas[i] - cost[i] < 0:
                i = (i + 1) % n
            return i if i < n else -1
```

Greedy - LeetCode - By Purnam

— 74 —

LeetCode

```
Python
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = 0
        while i < n:
            while i < j:
                i = (i + 1) % n
                while i < j and gas[i] - cost[i] < 0:
                    i = (i + 1) % n
            i = (i + 1) % n
            while i < j and gas[i] - cost[i] < 0:
                i = (i + 1) % n
            return i if i < n else -1
```

C++

C++

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        const int gasSum = accumulate(gas.begin(), gas.end(), 0);
        const int costSum = accumulate(cost.begin(), cost.end(), 0);
        if (gasSum < costSum) return -1;

        int ans = 0;
        int sum = 0;

        // Try to start from each index.
        for (int i = 0; i < gas.size(); ++i) {
            sum += gas[i] - cost[i];
            if (sum < 0) {
                sum = 0;
                ans = i + 1; // Start from the next index.
            }
        }
        return ans;
    }
};
```

C++

Greedy - LeetCode - By Purnam

— 75 —

LeetCode

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int s = gas.size();
        int i = 0, j = 0, k = 0;
        int cur = 0, n = 0;
        while (cur < n) {
            s += gas[j] - cost[j];
            ++cur;
            j = (j + 1) % s;
            while (s < 0 && cur < n) {
                --i;
                s += gas[i] - cost[i];
                ++cur;
            }
        }
        return s < 0 ? -1 : i;
    }
};
```

C++

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int s = gas.size();
        int i = 0, j = 0, k = 0;
        int cur = 0, n = 0;
        while (cur < n) {
            s += gas[j] - cost[j];
            ++cur;
            j = (j + 1) % s;
            while (s < 0 && cur < n) {
                --i;
                s += gas[i] - cost[i];
                ++cur;
            }
        }
        return s < 0 ? -1 : i;
    }
};
```

Java

Java

Greedy - LeetCode - By Purnam

— 76 —

LeetCode

```
class Solution {
public:
    int canCompleteCircuit(int[] gas, int[] cost) {
        final int gasSum = Arrays.stream(gas).sum();
        final int costSum = Arrays.stream(cost).sum();
        if (gasSum < costSum) return -1;

        int ans = 0;
        int sum = 0;

        // Try to start from each index.
        for (int i = 0; i < gas.length; ++i) {
            sum += gas[i] - cost[i];
            if (sum < 0) {
                sum = 0;
                ans = i + 1; // Start from the next index.
            }
        }
        return ans;
    }
}
```

Java

```
class Solution {
public:
    int canCompleteCircuit(int[] gas, int[] cost) {
        int s = gas.length;
        int i = 0, j = 0, k = 0;
        int cur = 0, n = 0;
        while (cur < n) {
            s += gas[j] - cost[j];
            ++cur;
            j = (j + 1) % s;
            while (s < 0 && cur < n) {
                --i;
                s += gas[i] - cost[i];
                ++cur;
            }
        }
        return s < 0 ? -1 : i;
    }
};
```

Java

Greedy - LeetCode - By Purnam

— 77 —

LeetCode

```
public class Solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {
        int s = gas.length;
        int i = 0, j = 0, k = 0;
        int cur = 0, n = 0;
        while (cur < n) {
            s += gas[j] - cost[j];
            ++cur;
            j = (j + 1) % s;
            while (s < 0 && cur < n) {
                --i;
                s += gas[i] - cost[i];
                ++cur;
            }
        }
        return s < 0 ? -1 : i;
    }
}
```

Java

```
class Solution {
public:
    int canCompleteCircuit(int[] gas, int[] cost) {
        int s = gas.length;
        int i = 0, j = 0, k = 0;
        int cur = 0, n = 0;
        while (cur < n) {
            s += gas[j] - cost[j];
            ++cur;
            j = (j + 1) % s;
            while (s < 0 && cur < n) {
                --i;
                s += gas[i] - cost[i];
                ++cur;
            }
        }
        return s < 0 ? -1 : i;
    }
};
```

Java

```
public class Solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {
        int s = gas.length;
        int i = 0, j = 0, k = 0;
        int cur = 0, n = 0;
        while (cur < n) {
            s += gas[j] - cost[j];
            ++cur;
            j = (j + 1) % s;
            while (s < 0 && cur < n) {
                --i;
                s += gas[i] - cost[i];
                ++cur;
            }
        }
        return s < 0 ? -1 : i;
    }
}
```

Greedy - LeetCode - By Purnam

— 78 —

LeetCode

```
TypeScript
TypeScript

function canCompleteCircuit(gas: number[], cost: number[]): number {
    const n = gas.length;
    let i = n - 1;
    let j = n - 1;
    let s = 0;
    let cnt = 0;
    while (cnt < n) {
        s += gas[i] - cost[i];
        i--;
        while (s < 0 && cnt < n) {
            s += gas[i] - cost[i];
            i--;
        }
        cnt++;
    }
    return s < 0 ? -1 : i;
}

TypeScript

function canCompleteCircuit(gas: number[], cost: number[]): number {
    const n = gas.length;
    let i = n - 1;
    let j = n - 1;
    let s = 0;
    let cnt = 0;
    while (cnt < n) {
        s += gas[i] - cost[i];
        i--;
        while (s < 0 && cnt < n) {
            s += gas[i] - cost[i];
            i--;
        }
        cnt++;
    }
    return s < 0 ? -1 : i;
}

Go
Go
```

Greedy - LeetCode - By Patsum — 79 — LeetCode

```
func canCompleteCircuit(gas []int, cost []int) int {
    n := len(gas)
    i, j := n-1, n-1
    cnt, s := 0, 0
    for cnt < n {
        s += gas[i] - cost[i]
        i = (i + 1) % n
        for s < 0 && cnt < n {
            s += gas[i] - cost[i]
            i = (i + 1) % n
        }
        cnt++
    }
    if s < 0 {
        return -1
    }
    return i
}

Go

func canCompleteCircuit(gas []int, cost []int) int {
    n := len(gas)
    i, j := n-1, n-1
    cnt, s := 0, 0
    for cnt < n {
        s += gas[i] - cost[i]
        i = (i + 1) % n
        for s < 0 && cnt < n {
            s += gas[i] - cost[i]
            i = (i + 1) % n
        }
        cnt++
    }
    if s < 0 {
        return -1
    }
    return i
}

[*] Greedy — Pattern Solution (stored optimal)
Python
```

Greedy - LeetCode - By Patsum — 80 — LeetCode

```
# Define a function to find starting gas station index
def canCompleteCircuit(gas, cost):
    total_surplus = 0 # Total gas surplus after completing the route
    current_surplus = 0 # Gas surplus while traveling from a candidate start station
    start_index = 0 # Candidate starting station index

    # Iterate through gas stations
    for i in range(len(gas)):
        # Calculate net gas after leaving station i
        surplus = gas[i] - cost[i]
        total_surplus += surplus
        current_surplus += surplus

        # If current surplus becomes negative, station i is not a valid start.
        if current_surplus < 0:
            # Reset starting station to the next station
            start_index = i + 1
            current_surplus = 0 # Reset current surplus
        # Check if overall surplus is non-negative
        return start_index if total_surplus >= 0 else -1

# Example usage
gas = [1,2,3,4,5]
cost = [3,4,5,1,2]
print(canCompleteCircuit(gas, cost)) # Output: 3

C++

// C++ Solution using the greedy approach
#include <vector>
using namespace std;

class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int totalSurplus = 0; // Total surplus across all stations
        int currentSurplus = 0; // Surplus from the candidate station
        int startIndex = 0; // Candidate starting station index

        // Iterate through gas stations
        for (int i = 0; i < gas.size(); i++) {
            int surplus = gas[i] - cost[i]; // Compute surplus at station i
            totalSurplus += surplus;
            currentSurplus += surplus;

            // If current surplus falls negative, update candidate starting station index
            if (currentSurplus < 0) {
                startIndex = i + 1;
                currentSurplus = 0; // Reset for next candidate segment
            }
        }

        // If the total surplus is non-negative, return startIndex otherwise -1
    }
};
```

Greedy - LeetCode - By Patsum — 81 — LeetCode

```
        return totalSurplus == 0 ? startIndex : -1;
    }
};

// Example usage:
// Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
// Output: 3

#179 MEDIUM
Largest Number
LeetCode - SimplifyLeetCode - LeetCode - Editorial
Array - String - Greedy - Sorting
Likes: 100

Problem:
Given a list of non-negative integers nums, arrange them such that they form the largest number and return it.

Since the result may be very large, you need to return a string instead of an integer.

Example 1:
Input: nums = [10,2]
Output: "210"

Example 2:
Input: nums = [3,30,34,5,9]
Output: "9534330"

Constraints:
• 1 <= nums.length <= 100
• 0 <= nums[i] <= 10^9

>> Brute Force [Greedy] Interview Prep

Python
def compare(self, a, b):
    # Reverse Sort (b - a) - try all orderings, pick largest concatenation
    from itertools import permutations
    best = ""
    for perm in permutations(sorted(str, nums)):
        candidate = "".join(perm)
        if candidate > best: best = candidate
    return best.lstrip("0") or "0"

Python
```

Greedy - LeetCode - By Patsum — 82 — LeetCode

```
Python
# Import cmp_to_key to convert a comparator function to a key function for sorting
from functools import cmp_to_key

# Define the comparator function
def compare(x, y):
    # Compare two possible orders: x+y and y+x
    if x+y > y+x:
        return -1 # x should come before y
    elif x+y < y+x:
        return 1 # y should come before x
    else:
        return 0 # Both orders are equal

def largestNumber(nums):
    # Convert all integers to strings
    num_str = list(map(str, nums))
    # Sort the strings using the custom comparator
    num_str.sort(key=cmp_to_key(compare))
    # If the largest number is "0", the entire number is 0 so return "0"
    if num_str[0] == "0":
        return "0"
    # Concatenate the sorted numbers and return the result
    return "".join(num_str)

# Example usage:
print(largestNumber([3, 30, 34, 5, 9])) # Output: "9534330"

Python

class LargestNumber:
    def __init__(self, nums):
        self.nums = nums

    def largestNumber(self):
        # Sort the numbers using the custom comparator
        self.nums.sort(key=cmp_to_key(compare))
        # If the largest number is "0", the entire number is 0 so return "0"
        if self.nums[0] == "0":
            return "0"
        # Concatenate the sorted numbers and return the result
        return "".join(self.nums)

Python

class Solution:
    def largestNumber(self, nums: List[int]) -> str:
        num_str = [str(i) for i in nums]
        num_str.sort(key=cmp_to_key(lambda a, b: 1 if a+b < b+a else -1))
        return "0" if num_str[0] == "0" else "".join(num_str)

Python

class Solution:
    def largestNumber(self, nums: List[int]) -> str:
        num_str = [str(i) for i in nums]
        num_str.sort(key=cmp_to_key(lambda a, b: 1 if a+b < b+a else -1))
        return "0" if num_str[0] == "0" else "".join(num_str)

C++
C++
```

Greedy - LeetCode - By Patsum — 83 — LeetCode

```
class Solution {
public:
    string largestNumber(vector<int>& nums) {
        string ans;

        ranges::sort(nums, [](int a, int b) {
            return to_string(a) + to_string(b) > to_string(b) + to_string(a);
        });

        for (const int num : nums)
            ans += to_string(num);

        return ans[0] == '0' ? "0" : ans;
    }
};

C++

class Solution {
public:
    string largestNumber(vector<int>& nums) {
        vector<string> vs;
        for (int v : nums) vs.push_back(to_string(v));
        sort(vs.begin(), vs.end(), [](string a, string b) {
            return a + b > b + a;
        });
        if (vs[0] == "0") return "0";
        string ans;
        for (string v : vs) ans += v;
        return ans;
    }
};

Java
Java
```

Greedy - LeetCode - By Patsum — 84 — LeetCode

```

class Solution {
    public String largestNumber(int[] nums) {
        final String s = Arrays.toString(nums)
            .replaceAll(String.valueOf(
                .sort((a, b) -> (b + a).compareTo(a + b))
            ).collect(Collectors.joining("")), "");
        return s.startsWith("00") ? "0" : s;
    }
}

Java

class Solution {
    public String largestNumber(int[] nums) {
        List<String> vs = new ArrayList<>();
        for (int v : nums) {
            vs.add(v + "");
        }
        vs.sort((a, b) -> (b + a).compareTo(a + b));
        if ("0".equals(vs.get(0))) {
            return "0";
        }
        return String.join("", vs);
    }
}

Java

class Solution {
    public String largestNumber(int[] nums) {
        List<String> vs = new ArrayList<>();
        for (int v : nums) {
            vs.add(v + "");
        }
        vs.sort((a, b) -> (b + a).compareTo(a + b));
        if ("0".equals(vs.get(0))) {
            return "0";
        }
        return String.join("", vs);
    }
}

Java

```

Greedy - LeetCode - By Patsum

— 85 —

LeetCode

```

using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Text;

public class Comparer : IComparer<string>
{
    public int Compare(string left, string right)
    {
        return Compare(left, right, 0, 0);
    }

    private int Compare(string left, string right, int lBegin, int rBegin)
    {
        var len = Math.Min(left.Length - lBegin, right.Length - rBegin);
        for (var i = 0; i < len; ++i)
        {
            if (left[lBegin + i] != right[rBegin + i])
            {
                return left[lBegin + i] > right[rBegin + i] ? -1 : 1;
            }
        }

        if (left.Length - lBegin == right.Length - rBegin)
        {
            return 0;
        }
        if (left.Length - lBegin > right.Length - rBegin)
        {
            return Compare(left, right, lBegin + len, rBegin);
        }
        else
        {
            return Compare(left, right, lBegin, rBegin + len);
        }
    }
}

public class Solution {
    public string LargestNumber(int[] nums) {
        var sb = new StringBuilder();
        var strs = nums.Select(x => x.ToString(CultureInfo.InvariantCulture)).OrderByDescending(x => x,
            new Comparer());
        var nonZeroOccurred = false;
        foreach (var str in strs)
        {
            if (nonZeroOccurred && str == "0") continue;
            sb.Append(str);
            nonZeroOccurred = true;
        }
        return sb.Length == 0 ? "0" : sb.ToString();
    }
}

```

Greedy - LeetCode - By Patsum

— 86 —

LeetCode

```

TypeScript

function largestNumber(nums: number[]): string {
    const [sb, bal] = [String(a) + String(b) + String(a),
        return sb - sb];
    return nums[0] ? nums.join('') : '0';
}

TypeScript

function largestNumber(nums: number[]): string {
    num.sort((a, b) => {
        const [sb, bal] = [String(a) + String(b), String(b) + String(a)];
        return sb - sb];
    });
    return nums[0] ? nums.join('') : '0';
}

Go

func largestNumber(nums []int) string {
    vs := make([]string, len(nums))
    for i, v := range nums {
        vs[i] = strconv.Itoa(v)
    }
    sort.Slice(vs, func(i, j int) bool {
        return vs[i]+vs[j] > vs[j]+vs[i]
    })
    if vs[0] == "0" {
        return "0"
    }
    return strings.Join(vs, "")
}

Go

func largestNumber(nums []int) string {
    vs := make([]string, len(nums))
    for i, v := range nums {
        vs[i] = strconv.Itoa(v)
    }
    sort.Slice(vs, func(i, j int) bool {
        return vs[i]+vs[j] > vs[j]+vs[i]
    })
    if vs[0] == "0" {
        return "0"
    }
    return strings.Join(vs, "")
}

(*) Greedy — Pattern Solution (matched from community answers)
Java

```

Greedy - LeetCode - By Patsum

— 87 —

LeetCode

```

class Solution {
    public String largestNumber(int[] nums) {
        final String s = Arrays.toString(nums)
            .replaceAll(String.valueOf(
                .sort((a, b) -> (b + a).compareTo(a + b))
            ).collect(Collectors.joining("")), "");
        return s.startsWith("00") ? "0" : s;
    }
}

#280 MEDIUM
Wiggle Sort
LeetCode - SimplifyLeet - WalkCC - LeetCode - Editorial
Array - Greedy - Sorting
Like Premium 98

Problem:
Given an integer array nums, reorder it such that nums[0] <= nums[1] >= nums[2] <= nums[3]...
You may assume the input array always has a valid answer.

Example 1:
Input: nums = [3,5,2,1,6,4]
Output: [3,5,1,2,4,6]
Explanation: [1,6,2,5,3,4] is also accepted.

Example 2:
Input: nums = [6,5,6,3,8]
Output: [6,5,6,3,8]

Constraints:
1 <= nums.length <= 5 * 10^4
0 <= nums[i] <= 10^4
It is guaranteed that there will be an answer for the given input nums.
Follow up: Could you solve the problem in O(n) time complexity?

>> Brute Force [Greedy] Interview Prep
Python
# Brute Force Approach
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        # Brute Force Approach - Check all subsets/orderings exhaustively
        from itertools import permutations
        best = float('inf')
        for perm in permutations(nums):
            cur = 0
            best = min(best, cur)
            return best

Python

```

Greedy - LeetCode - By Patsum

— 88 —

LeetCode

```

Python
# Brute solution for wiggle sort
def wiggleSort(nums):
    # Iterate over the list starting from index 1
    for i in range(1, len(nums)):
        # Check the condition based on the parity of the index
        if i % 2 == 1:
            # For odd index, if previous element is greater, swap them
            if nums[i] < nums[i-1]:
                nums[i], nums[i-1] = nums[i-1], nums[i]
            else:
                # For even index, if previous element is smaller, swap them
                if nums[i] > nums[i-1]:
                    nums[i], nums[i-1] = nums[i-1], nums[i]
        return nums
# Example usage
# nums = [3,5,2,1,6,4]
# print(wiggleSort(nums))

Python
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        # Do not return anything, modify nums in-place instead.
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i-1]) or (
                i % 2 == 0 and nums[i] > nums[i-1]):
                nums[i], nums[i-1] = nums[i-1], nums[i]
        return nums

Python
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        # Do not return anything, modify nums in-place instead.
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i-1]) or (
                i % 2 == 0 and nums[i] > nums[i-1]):
                nums[i], nums[i-1] = nums[i-1], nums[i]
        return nums

```

Greedy - LeetCode - By Patsum

— 89 —

LeetCode

```

...
math prove:
example [1, 5, 2, 1]
when reaching 2, all good
then next, is 1,
so here comes the question, will the number after 2 violating the wiggle rule?
answer is no.
In example, when reaching 2,
= if [3, 5, 2, 1], so 3 is bigger than 2, then no swap needed according to the if () conditions
= if [3, 5, 2, 1], so previous round, guaranteed that 2 is smaller than 5, so wiggle
rule is still good
...
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        # Do not return anything, modify nums in-place instead.
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i-1]) or (
                i % 2 == 0 and nums[i] > nums[i-1]):
                nums[i], nums[i-1] = nums[i-1], nums[i]
        return nums

Python
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        # Sort the list in ascending order
        n = len(nums)
        for i in range(1, n - 1, 2):
            nums[i], nums[i + 1] = nums[i + 1], nums[i] # Swap adjacent elements
        ...
        If the list has an odd length, the last element will be compared with the second-to-last element
        and swapped if necessary to form a wiggle sequence.
        ...
        If n % 2 == 0 or nums[-1] < nums[-2]:
            nums[-1], nums[-2] = nums[-2], nums[-1]
        ...
import random
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:

```

Greedy - LeetCode - By Patsum

— 90 —

LeetCode

[Array](#) · [Hash Table](#) · [String](#) · [Greedy](#) · [Counting](#)
[Lists](#): [CodeSignal](#)

Problem:

You are given an array of strings `words`. Each element of `words` consists of two lowercase English letters.

Create the longest possible palindrome by selecting some elements from words and concatenating them in any order. Each element can be selected at most once.

Return the length of the longest palindrome that you can create. If it is impossible to create any palindrome, return 0.

A palindrome is a string that reads the same forward and backward.

Example 1:

Input: words = ["lc", "cl", "gg"]
Output: 6

Explanation: One longest palindrome is "lc" + "gg" + "cl" = "lcggcl", of length 6. Note that "clggcl" is another longest palindrome that can be created.

Example 2:

```
Input: words = ["ab","ty","yt","lc","cl","ab"]
Output: 8
```

Explanation: One longest palindrome is "ty" +
Note that "lcyttycl" is another longest palind

Example 3:

```
Input: words = ["cc","ll","xx"]
Output: 2
```

Explanation: One longest palindrome is "cc", o
Note that "ll" is another longest palindrome t

Constraints:

- 1 <= words.length <= 105
- words[i].length == 3

- words[i].length == 2
- words[i] consists of lowercase English letters

[>>> Brute Force \[Greedy\] Interview Prep](#)

Python

```
# Brute Force Approach
class Solution:
    def isPowerOfTwo(self, n: int) -> bool:
```

```
# Brute Force  $O(n^2)$  - check all subsets/ord
from itertools import permutations
```

```
best = float('inf')
for perm in permutations(words):
```

```
cost = sum(pern)
best = min(best, cost)
```

```
return best
```

Greedy · LeetCode — By Pattern — 1

```

Python
Python
# Python solution
from collections import Counter

class Solution:
    def longestPalindrome(self, words):
        # Count the frequency of each word
        word_count = Counter(words)
        length = 0 # Total word count (each adds 2 letters)
        center_used = False

        for word, count in list(word_count.items()):
            reverse = word[::-1] # Get the reverse of the current word

            if word == reverse:
                # For words that are palindromic themselves (e.g. "gg")
                pairs = count // 2
                length += pairs * 2 # Each pair adds 2 words (4 letters)
                # If an extra word exists and center is not used, add one word in the center
                if count % 2 == 1 and not center_used:
                    length += 1
                    center_used = True
            else:
                # For words that are not self-palindromic.
                if reverse in word_count:
                    # We only count each pair once, so use minimum and then set the reverse count to 0.
                    pair_count = min(count, word_count[reverse])
                    length += pair_count * 2 # Each pair contributes two words
                    word_count[reverse] = 0 to avoid double counting later.
                    word_count[word] -= pair_count
        return length + 2 # Each word is of length 2

# Example usage:
# sol = Solution()
# print(sol.longestPalindrome(["lc", "cd", "gg"])) # Output: 6
Python

```

Greedy · LeetCode - By Purnam

— 103 —

LeetCode

```

class Solution:
    def longestPalindrome(self, words: List[str]) -> int:
        ans = 0
        count = [0] * 26
        for a, b in words:
            i = ord(a) - ord('a')
            j = ord(b) - ord('a')
            if count[j][i]:
                ans += 4
                count[j][i] -= 1
            else:
                count[i][j] += 1

        for i in range(26):
            if count[i][i]:
                return ans + 2
        return ans

Python
class Solution:
    def longestPalindrome(self, words: List[str]) -> int:
        cnt = Counter(words)
        ans = 0
        for k, v in cnt.items():
            if k[0] == k[1]:
                x = v // 2
                ans += v // 2 * 2 + 2
            else:
                ans += min(v, cnt[k[::-1]]) * 2
        ans += 2 if x else 0
        return ans

Python
class Solution:
    def longestPalindrome(self, words: List[str]) -> int:
        cnt = Counter(words)
        ans = 0
        for k, v in cnt.items():
            if k[0] == k[1]:
                x = v // 2
                ans += v // 2 * 2 + 2
            else:
                ans += min(v, cnt[k[::-1]]) * 2
        ans += 2 if x else 0
        return ans

C++
C++

```

Greedy · LeetCode - By Purnam

— 104 —

LeetCode

```

class Solution {
public:
    int longestPalindrome(vector<string& words) {
        int ans = 0;
        vector<vector<int>> count(26, vector<int>(26));

        for (const string& word : words) {
            const int i = word[0] - 'a';
            const int j = word[1] - 'a';
            if (count[i][j] > 0) {
                ans += 4;
                --count[i][j];
            } else {
                ++count[i][j];
            }

            for (int k = 0; k < 26; ++k)
                if (count[i][k] > 0)
                    return ans + 2;
        }
        return ans;
    }
};

C++
class Solution {
public:
    int longestPalindrome(vector<string& words) {
        unordered_map<string, int> cnt;
        for (const w : words) cnt[w]++;
        int ans = 0, x = 0;
        for (const [k, v] : cnt) {
            string rk = k;
            reverse(rk.begin(), rk.end());
            if (k[0] == k[1]) {
                x += v / 2;
            } else if (cnt.count(rk)) {
                ans += min(v, cnt[rk]) * 2;
            }
        }
        ans += x * 2 + 2;
        return ans;
    }
};

C++

```

Greedy · LeetCode - By Purnam

— 105 —

LeetCode

```

class Solution {
public:
    int longestPalindrome(vector<string& words) {
        unordered_map<string, int> cnt;
        for (const w : words) cnt[w]++;
        int ans = 0, x = 0;
        for (const [k, v] : cnt) {
            string rk = k;
            reverse(rk.begin(), rk.end());
            if (k[0] == k[1]) {
                x += v / 2;
            } else if (cnt.count(rk)) {
                ans += min(v, cnt[rk]) * 2;
            }
        }
        ans += x * 2 + 2;
        return ans;
    }
};

Java
class Solution {
public:
    int longestPalindrome(String[] words) {
        int ans = 0;
        int[] count = new int[26][26];

        for (final String word : words) {
            final int i = word.charAt(0) - 'a';
            final int j = word.charAt(1) - 'a';
            if (count[i][j] > 0) {
                ans += 4;
                --count[i][j];
            } else {
                ++count[i][j];
            }

            for (int k = 0; k < 26; ++k)
                if (count[i][k] > 0)
                    return ans + 2;
        }
        return ans;
    }
};

Java

```

Greedy · LeetCode - By Purnam

— 106 —

LeetCode

```

class Solution {
public:
    int longestPalindrome(String[] words) {
        Map<String, Integer> cnt = new HashMap<>();
        for (var w : words) {
            cnt.merge(w, 1, Integer::sum);
        }
        int ans = 0, x = 0;
        for (var a : cnt.entrySet()) {
            var k = a.getKey();
            var rk = new StringBuilder(k).reverse().toString();
            int v = a.getValue();
            if (k.charAt(0) == k.charAt(1)) {
                x += v / 2;
            } else {
                ans += Math.min(v, cnt.getOrDefault(rk, 0)) * 2;
            }
        }
        ans += x * 2 + 2;
        return ans;
    }
};

Java
class Solution {
public:
    int longestPalindrome(String[] words) {
        Map<String, Integer> cnt = new HashMap<>();
        for (var w : words) {
            cnt.put(w, cnt.getOrDefault(w, 0) + 1);
        }
        int ans = 0, x = 0;
        for (var a : cnt.entrySet()) {
            var k = a.getKey();
            var rk = new StringBuilder(k).reverse().toString();
            int v = a.getValue();
            if (k.charAt(0) == k.charAt(1)) {
                x += v / 2;
            } else {
                ans += Math.min(v, cnt.getOrDefault(rk, 0)) * 2;
            }
        }
        ans += x * 2 + 2;
        return ans;
    }
};

TypeScript
TypeScript

```

Greedy · LeetCode - By Purnam

— 107 —

LeetCode

```

function longestPalindrome(words: string[]): number {
    const cnt = new Map<string, number>();
    for (const w of words) cnt.set(w, (cnt.get(w) || 0) + 1);
    let [ans, x] = [0, 0];
    for (const [k, v] of cnt.entries()) {
        if (k[0] === k[1]) {
            x += v / 2;
        } else {
            ans += Math.min(v, cnt.get(k[1] + k[0]) || 0) * 2;
        }
    }
    ans += x * 2 + 2;
    return ans;
}

TypeScript
function longestPalindrome(words: string[]): number {
    const cnt = new Map<string, number>();
    for (const w of words) cnt.set(w, (cnt.get(w) || 0) + 1);
    let [ans, x] = [0, 0];
    for (const [k, v] of cnt.entries()) {
        if (k[0] === k[1]) {
            x += v / 2;
        } else {
            ans += Math.min(v, cnt.get(k[1] + k[0]) || 0) * 2;
        }
    }
    ans += x * 2 + 2;
    return ans;
}

Go
Go
func longestPalindrome(words []string) int {
    cnt := map[string]int{}
    for _, w := range words {
        cnt[w]++
    }
    ans, x := 0, 0
    for k, v := range cnt {
        if k[0] == k[1] {
            x += v / 2
        } else {
            rk := string([]byte{k[1], k[0]})
            if v, ok := cnt[rk]; ok {
                ans += min(v, ok) * 2
            }
        }
        if x > 0 {
            ans += 2
        }
    }
    return ans
}

```

Greedy · LeetCode - By Purnam

— 108 —

LeetCode

Quick Review — Greedy 6 questions · insights · complexity · solution

#401 Longest Palindrome [Easy]

Key Insights

- Count the frequency of each character.
- For each character, you can use pairs (even count) or add one (odd count) to form the palindrome.
- If any character has an odd frequency, you can add one extra character as the center of the palindrome.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(1)$, since the frequency counter will have at most 52 keys (considering both uppercase and lowercase letters).

Solution

The solution uses a hash table to count the frequency of each character. For every character, the maximum number of pairs that can be used in a palindrome is determined by taking the largest even number that is less than or equal to its frequency (which is frequency / 2). If there exists any character with an odd frequency, one of these can be used as the center of the palindrome, adding an extra character to the total length. This greedy approach naturally computes the longest palindrome from the available palindrome building blocks.

#134 Gas Station [Medium]

- If the total amount of gas available is less than the total travel cost, then completing the circuit is impossible.
- A greedy approach can be used by keeping track of the current surplus of gas. If the surplus becomes negative at any station, the journey cannot start from any station before that point.
- Restart the journey from the next station whenever the surplus goes negative.
- The problem guarantees that if a solution exists, it is unique.

Space & Time

Time Complexity: $O(n)$, where n is the number of gas stations, as we traverse the list only once.

Space Complexity: $O(1)$, as no extra space is used other than a few variables.

Solution

The solution involves iterating over the list of stations while maintaining two variables: one for the current gas surplus and one for the total gas surplus. If at any station the current surplus drops below zero, the current starting station is not viable and we set the starting station to the next index and reset the current surplus. After one pass, if the total gas surplus is non-negative, then the identified station is the valid starting point.

#179 Largest Number [Medium]

- The order in which numbers appear drastically affects the concatenated result.
- A custom sorting strategy is required where two numbers are compared by checking which concatenation (first-second vs second-first) yields a larger number.
- Edge case: When the numbers are all zeros, the result should be "0" instead of several leading zeros.

Space & Time

Time Complexity: $O(n \log n \cdot k)$, where n is the number of elements in the array and k is the maximum number of digits in a number (due to string comparisons in the custom sort).

Space Complexity: $O(n \cdot k)$ for storing and processing the strings.

We solve the problem by converting all integers to strings so that we can use string concatenation to compare the results. A custom comparator is defined that, given two number strings, compares the concatenated results in both orders. By sorting the array using this comparator in descending order, we ensure that placing the highest contributing number first yields the largest possible number. After sorting, a check is performed to return "0" if the highest number is "0". Finally, the sorted strings are concatenated to form the answer.

#280 Wiggle Sort [Medium]

- The alternating "wiggle" requirement can be satisfied by ensuring two conditions:
 - For every odd index i , $nums[i]$ should be greater than or equal to $nums[i-1]$.
 - For every even index i (where $i > 0$), $nums[i]$ should be less than or equal to $nums[i-1]$.
- A single pass through the array can ensure these conditions by swapping adjacent elements when the condition is violated.
- This greedy approach works because the local adjustments guarantee the overall wiggle pattern without needing to sort the entire array.

Space & Time
Time Complexity: $O(n)$ — Only one pass through the array is needed.
Space Complexity: $O(1)$ — The modifications are done in-place without extra data structures.

Solution:

We can achieve a wiggle sort in one pass by iterating through the array and, at each index i (starting from 1), checking if the pair $(\text{nums}[i-1], \text{nums}[i])$ satisfies the wiggle condition. If i is odd, $\text{nums}[i]$ should be at least $\text{nums}[i-1]$; if not, we swap them. Conversely, if i is even and $\text{nums}[i]$ is greater than $\text{nums}[i-1]$, then we swap them. These local swaps adjust the order incrementally while ensuring that the overall wiggle pattern is maintained. This greedy approach works because only two adjacent numbers are involved in any violation and fixing them locally does not disturb the rest of the order.

#954 Array of Doubled Pairs [Medium]

- Sort the array based on the absolute value of the elements to handle negative numbers correctly.
- Use a hash table (or frequency counter) to track the occurrence of each element.
- For each number in the sorted list, if the number is still available in the frequency map, try to pair it with its double.
- Decrement the appropriate counts and validate that it is possible to find sufficient doubles for all numbers.
- Special case: when dealing with zeros, pairing works within the same value.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting.

Space Complexity: $O(n)$ for storing the frequency map.

Solution

The solution starts by counting the frequency of each element in the array. Sorting the array by absolute value ensures that when we process an element, any potential pair (i.e., its double) comes later in the sequence. For each number, if there are available occurrences, we try to match it with its double. If there are not enough doubles available, the answer is false. If all elements can be paired appropriately by decrementing their counts in the frequency map, the function returns true.

#2131 Longest Palindrome by Concatenating Two Letter Words [Medium]

- Pair words with their reverse (e.g., "ab" with "ba") to form a palindrome.

- Handle words that are palindromic by themselves (like "gg" or "aa") differently; they can be used in pairs and one extra can be placed in the center.
- Use a hash map (or dictionary) to count frequencies of each word.
- For non-palindromic words, consider each pair only once to avoid double counting.
- For self-palindromic words, count how many pairs can be used and whether an extra word can be placed in the middle to maximize length.

Space & Time

Time Complexity: $O(n)$, where n is the number of words, since we iterate through the list and perform constant time operations per word.

Space Complexity: $O(n)$, to store counts of words in the hash map.

Solution

The approach is to count the occurrences of each word using a hash map. For each word:

- if the word is not self-palindromic, check if it reverts exists and form pairs by considering the minimum count between every pair of words. If the word is self-palindromic, form as many pairs as possible using $\text{count} // 2$.
- and note that there is always left; you can use one extra self-palindromic word as the center of the palindromic.

Finally, compute the total length by multiplying the number of words used by 2, since each word contributes 2 characters.

#3 JavaScript — 7 questions · #3 of 21

#2627 MEDIUM

Debounce

[LeetCode](#) · [Simplify](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

JavaScript

List: Premium 98

Problem:

Given a function fn and a time in milliseconds t, return a debounced version of that function.

A debounced function is a function whose execution is delayed by t milliseconds and whose execution is cancelled if it is called again within that window of time. The debounced function should also receive the passed parameters.

For example, let's say t = 50ms, and the function was called at 30ms, 60ms, and 100ms.

The first 2 function calls would be cancelled, and the 3rd function call would be executed at 150ms.

If instead t = 35ms, the 1st call would be cancelled, the 2nd would be executed at 95ms, and the 3rd would be executed at 135ms.

The above diagram shows how debounce will transform events. Each rectangle represents 100ms and the debounce time is 400ms. Each color represents a different set of inputs.

Please solve it without using lodash's `_debounce` function.

Example 1:

```
Input:
t = 50
calls = [
  ("f": 50, inputs: [1]),
  ("f": 75, inputs: [2])
]
Output: [("f": 125, inputs: [2])]
Explanation:
let start = Date.now();
function log(...inputs) {
  console.log(Date.now() - start, inputs)
}
const dlog = debounce(log, 50);
setTimeout(() => dlog(1), 50);
setTimeout(() => dlog(2), 75);
```

The 1st call is cancelled by the 2nd call because the 2nd call occurred before 100ms
The 2nd call is delayed by 50ms and executed at 125ms. The inputs were (2).

Example 2:

```
Input:
t = 20
calls = [
```

Greedy · LeetCode · By Pattern

— 115 —

LeetCode

```
type F = (...args: any[]) => any;
function debounce(fn: F, t: number): F {
  let timeout: ReturnType | undefined;

  return function (...args) {
    if (timeout != undefined) {
      clearTimeout(timeout);
    }
    timeout = setTimeout(() => {
      fn.apply(this, args);
    }, t);
  };
}
```

```
// = const log = debounce(console.log, 100);
// = log('hello'); // cancelled
// = log('hello'); // cancelled
// = log('hello'); // longer at 100ms
//
```

TypeScript

```
type F = (...args: any[]) => any;
function debounce(fn: F, t: number): F {
  let timeout: ReturnType | undefined;

  return function (...args) {
    if (timeout != undefined) {
      clearTimeout(timeout);
    }
    timeout = setTimeout(() => {
      fn.apply(this, args);
    }, t);
  };
}
```

```
// = const log = debounce(console.log, 100);
// = log('hello'); // cancelled
// = log('hello'); // cancelled
// = log('hello'); // longer at 100ms
//
```

Python

Python

JavaScript · LeetCode · By Pattern

— 117 —

LeetCode

```
dlog = debounce(log, 50)
dlog()
time.sleep(0.825) # 2ms delay before the next call
dlog(2)
time.sleep(0.1) # wait to allow the debounced function to execute
```

```
// C++ Implementation using threads to simulate debouncing
#include <iostream>
#include <thread>
#include <chrono>
#include <functional>
#include <mutex>

using namespace std;

class Debounce {
public:
  // Constructor takes the function to debounce and the delay in milliseconds
  Debounce(function<void()> fn, int t) : fn(fn), delay(t), cancelFlag(false) {}

  // Operator() to call the debounced function
  void operator()() {
    {
      lock_guard<mutex> lock(mutex);
      // The previous pending call is cancelled
      cancelFlag = true;
    }
    // Start a new detached thread for the new call
    thread([this]) {
      {
        lock_guard<mutex> lock(mutex);
        // Mark this call as the newest (reset cancellation flag)
        cancelFlag = false;
      }
      // Wait for the specified delay
      this_thread::sleep_for(chrono::milliseconds(delay));
      bool execute = false;
      {
        lock_guard<mutex> lock(mutex);
        // Only execute if no other call has cancelled this one
        if (!cancelFlag) {
          execute = true;
        }
      }
      if (execute) {
        fn();
      }
    }.detach();
  }

private:
  function<void()> fn;
  int delay;
  mutex mutex;
  bool cancelFlag;
};
```

JavaScript · LeetCode · By Pattern

— 119 —

LeetCode

```
("f": 50, inputs: [1]),
("f": 100, inputs: [2])
]
Output: [("f": 70, inputs: [1]), ("f": 120, inputs: [2])]
Explanation:
The 1st call is delayed until 70ms. The inputs were (1).
The 2nd call is delayed until 120ms. The inputs were (2).
Example 3:
Input:
t = 150
calls = [
  ("f": 50, inputs: [1, 2]),
  ("f": 100, inputs: [3, 4]),
  ("f": 200, inputs: [5, 6])
]
Output: [("f": 200, inputs: [1,2]), ("f": 450, inputs: [5, 6])]
Explanation:
The 1st call is delayed by 150ms and ran at 200ms. The inputs were (1, 2).
The 2nd call is cancelled by the 3rd call.
The 3rd call is delayed by 150ms and ran at 450ms. The inputs were (5, 6).
```

Constraints:

- 0 <= t <= 1000
- 1 <= calls.length <= 10
- 0 <= calls[i].length <= 1000
- 0 <= calls[i].inputs.length <= 10

TypeScript

TypeScript

```
type F = (...args: number[]) => void;
function debounce(fn: F, t: number): F {
  let timeout: ReturnType | undefined;
  return function (...args) {
    clearTimeout(timeout);
    timeout = setTimeout(() => fn(...args), t);
  };
}
```

TypeScript

JavaScript · LeetCode · By Pattern

— 116 —

LeetCode

```
import threading

def debounce(fn, t):
    # timer reference to manage the scheduled execution
    timer = None

    def debounced(*args, **kwargs):
        nonlocal timer
        # Cancel previous timer if it exists
        if timer is not None:
            timer.cancel()
        # Start a new timer (convert milliseconds to seconds)
        timer = threading.Timer(t / 1000.0, lambda: fn(*args, **kwargs))
        timer.start()

    return debounced

# Example usage:
if __name__ == "__main__":
    import time
    start = time.time()

    def log(*args):
        print(f"Time: {time.time() - start}, args: {args}")

    dlog = debounce(log, 50)
    dlog(1)
    time.sleep(0.825) # 2ms delay before the next call
    dlog(2)
    time.sleep(0.1) # Wait to allow the debounced function to execute
```

[*] JavaScript — Pattern Solution (stored optimal)

Python

```
import threading

def debounce(fn, t):
    # timer reference to manage the scheduled execution
    timer = None

    def debounced(*args, **kwargs):
        nonlocal timer
        # Cancel previous timer if it exists
        if timer is not None:
            timer.cancel()
        # Start a new timer (convert milliseconds to seconds)
        timer = threading.Timer(t / 1000.0, lambda: fn(*args, **kwargs))
        timer.start()

    return debounced

# Example usage:
if __name__ == "__main__":
    import time
    start = time.time()

    def log(*args):
        print(f"Time: {time.time() - start}, args: {args}")
```

JavaScript · LeetCode · By Pattern

— 118 —

LeetCode

```
// Example usage:
let main = {
  auto start = chrome.steady_clock.now();
  auto log = [start];
  auto end = chrome.steady_clock.now();
  auto diff = chrome.duration_cast(chrome.milliseconds(end - start).count());
  out += "Time: " + diff + "ms, Executed log: " + out;
};

Debounce dlog(log, 50);
// Simulate first call at 100ms
this_thread::sleep_for(chrono::milliseconds(50));
dlog();
// Simulate second call at 100ms which cancels the previous call
this_thread::sleep_for(chrono::milliseconds(25));
dlog();
// Wait enough time for the last call to execute
this_thread::sleep_for(chrono::milliseconds(100));
return 0;
```

#2628 MEDIUM

JSON Deep Equal

[LeetCode](#) · [Simplify](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

JavaScript

List: Premium 98

Problem:

Given two values o1 and o2, return a boolean value indicating whether two values, o1 and o2, are deeply equal.

For two values to be deeply equal, the following conditions must be met:

- If both values are primitive types, they are deeply equal if they pass the `===` equality check.
- If both values are arrays, they are deeply equal if they have the same elements in the same order, and each element is also deeply equal according to these conditions.
- If both values are objects, they are deeply equal if they have the same keys, and the associated values for each key are also deeply equal according to these conditions.

You may assume both values are the output of `JSON.parse`. In other words, they are valid `JSON`.

Please solve it without using lodash's `_.isEqual` function

Example 1:

```
Input: o1 = {"x":1,"y":2}, o2 = {"x":1,"y":2}
Output: true
Explanation: The keys and values match exactly.
```

Example 2:

```
Input: o1 = {"y":2,"x":1}, o2 = {"x":1,"y":2}
Output: true
Explanation: Although the keys are in a different order, they still match exactly.
```

JavaScript · LeetCode · By Pattern

— 120 —

LeetCode

Example 3:
Input: o1 = {"a":null,"L":[1,2,3]}, o2 = {"a":null,"L":["1","2","3"]}
Output: false
Explanation: The array of numbers is different from the array of strings.
Example 4:
Input: o1 = true, o2 = false
Output: false
Explanation: true !== false
Constraints:
• 1 <= JSON.stringify(o1).length <= 105
• 1 <= JSON.stringify(o2).length <= 105
• maxNestingDepth <= 1000

```

TypeScript
TypeScript
type JSONValue =
  | null
  | boolean
  | number
  | string
  | JSONValue[]
  | { [key: string]: JSONValue };

function areDeeplyEqual(o1: JSONValue, o2: JSONValue): boolean {
  if (o1 === o2) {
    return true;
  }
  if (o1 === null || o1 === undefined || o2 === null || o2 === undefined) {
    return false;
  }
  if (typeof o1 !== 'object' || typeof o2 !== 'object') {
    return false;
  }
  if (Array.isArray(o1) !== Array.isArray(o2)) {
    return false;
  }
  if (Object.keys(o1).length !== Object.keys(o2).length) {
    return false;
  }
  for (const key in o1) {
    if (areDeeplyEqual(o1[key], o2[key])) {
      return true;
    }
  }
  return false;
}
Python
Python
```

```

def deep_equal(o1, o2):
    # If both are exactly equal, return True
    if o1 == o2:
        return True
    # If types differ, they are not equal
    if type(o1) != type(o2):
        return False
    # If both are lists, check length and recursively compare each element
    if isinstance(o1, list):
        if len(o1) != len(o2):
            return False
        for i, item1 in enumerate(o1):
            if not deep_equal(item1, o2[i]):
                return False
        return True
    # If both are dictionaries, check that they have the same keys and values
    if isinstance(o1, dict):
        if set(o1.keys()) != set(o2.keys()):
            return False
        for key in o1:
            if not deep_equal(o1[key], o2[key]):
                return False
        return True
    # fallback for other types (should not reach here for valid JSON)
    return False

# Example usage:
# print(deep_equal({"a": 1, "b": 2}, {"a": 2, "b": 1}))

```

```

// JavaScript - Pattern Solution (stored optimal)
// Python
def deep_equal(o1, o2):
    # If both are exactly equal, return True
    if o1 == o2:
        return True
    # If types differ, they are not equal
    if type(o1) != type(o2):
        return False
    # If both are lists, check length and recursively compare each element
    if isinstance(o1, list):
        if len(o1) != len(o2):
            return False
        for i, item1 in enumerate(o1):
            if not deep_equal(item1, o2[i]):
                return False
        return True
    # If both are dictionaries, check that they have the same keys and values
    if isinstance(o1, dict):
        if set(o1.keys()) != set(o2.keys()):
            return False
        for key in o1:
            if not deep_equal(o1[key], o2[key]):
                return False
        return True
    # fallback for other types (should not reach here for valid JSON)
    return False

```

```

# Example usage:
# print(deep_equal({"a": 1, "b": 2}, {"a": 2, "b": 1}))

// Note: This file contains code snippets for the use of the @ts-ignore library.
// Include the library header and use the @ts-ignore type.
#include <fstream>
#include <iostream>
#include <vector>
#include <string>
#include <cassert>
using json = nlohmann::json;

// Recursive function to compare two json objects deeply.
bool deeplyEqual(const json& o1, const json& o2) {
    // If both are exactly equal, return true.
    if (o1 == o2) return true;

    // If types differ, return false.
    if (o1.type() != o2.type()) return false;

    // If both are arrays, check size and compare each element.
    if (o1.is_array()) {
        if (o1.size() != o2.size()) return false;
        for (size_t i = 0; i < o1.size(); ++i) {
            if (!deeplyEqual(o1[i], o2[i])) return false;
        }
        return true;
    }

    // If both are objects, compare keys and then each key-value pair.
    if (o1.is_object()) {
        if (o1.size() != o2.size()) return false;
        for (const auto& kv : o1.items()) {
            std::string key = kv.key();
            // Check if the second object contains the key.
            if (o2.find(key) == o2.end()) return false;
            if (!deeplyEqual(kv.value(), o2[key])) return false;
        }
        return true;
    }

    // For number, string, boolean, or null, the operator== check suffices.
    return o1 == o2;
}

int main() {
    // Example usage:
    json obj1 = {"a": 1, "b": 2};
    json obj2 = {"a": 2, "b": 1};
    assert(!deeplyEqual(obj1, obj2) == true);
    std::cout << "Test passed" << std::endl;
    return 0;
}

```

```

#2630 MEDIUM
Memoize
LeetCode - SimplifyLeetCode - WubCC - LeetCode - Editorial
JavaScript
List: Premium 98
Problem:
Given a function fn, return a memoized version of that function.
A memoized function is a function that will never be called twice with the same inputs. Instead it will return a cached value.
fn can be any function and there are no constraints on what type of values it accepts. Inputs are considered identical if they are === to each other.
Example 1:
Input:
getInputs = () => [[2,2],[2,2],[1,2]]
fn = function (a, b) { return a + b; }
Output: [{"val":4,"calls":1}, {"val":4,"calls":1}, {"val":3,"calls":2}]
Explanation:
const inputs = getInputs();
const memoized = memoize(fn);
for (const arr of inputs) {
    memoized(...arr);
}
For the inputs of (2, 2): 2 + 2 = 4, and it required a call to fn().
For the inputs of (2, 2): 2 + 2 = 4, but those inputs were seen before so no call to fn() was required.
For the inputs of (1, 2): 1 + 2 = 3, and it required another call to fn() for a total of 2.
Example 2:
Input:
getInputs = () => [[0,0],[0,0],[0,0]]
fn = function (a, b) { return ([...a,...b]); }
Output: [{"val":0,"calls":1}, {"val":0,"calls":1}, {"val":0,"calls":3}]
Explanation:
Merging two empty objects will always result in an empty object. It may seem like there should only be 1 call to fn() because of cache-hits, however none of those objects are === to each other.
Example 3:
Input:
getInputs = () => { const o = {}; return [o,o],[o,o]; }
fn = function (a, b) { return ([...a,...b]); }
Output: [{"val":0,"calls":1}, {"val":0,"calls":1}, {"val":0,"calls":1}]
Explanation:
Merging two empty objects will always result in an empty object. The 2nd and 3rd third function calls result in a cache-hit. This is because every object passed in is identical.
Constraints:

```

```

• 1 <= inputs.length <= 105
• 0 <= inputs.flat().length <= 105
• inputs[0][0] != NaN
TypeScript
TypeScript
type Fn = (...params: any) => any;

function memoize(fn: Fn): Fn {
    const idMap: Map<string, number> = new Map();
    const cache: Map<string, any> = new Map();

    const getId = (obj: any): number => {
        if (!idMap.has(obj)) {
            idMap.set(obj, idMap.size);
        }
        return idMap.get(obj);
    };

    return function (...params: any) {
        const key = params.map(getId).join(',');
        if (!cache.has(key)) {
            cache.set(key, fn(...params));
        }
        return cache.get(key);
    };
}

//
// let callCount = 0;
// const memoizedFn = memoize(function (a, b) {
//
//     callCount += 1;
//     return a + b;
// });
// memoizedFn(2, 3) // 5
// memoizedFn(2, 3) // 5
// console.log(callCount) // 3
//
TypeScript

```

```

type Fn = (...params: any) => any;

function memoize(fn: Fn): Fn {
    const idMap: Map<string, number> = new Map();
    const cache: Map<string, any> = new Map();

    const getId = (obj: any): number => {
        if (!idMap.has(obj)) {
            idMap.set(obj, idMap.size);
        }
        return idMap.get(obj);
    };

    return function (...params: any) {
        const key = params.map(getId).join(',');
        if (!cache.has(key)) {
            cache.set(key, fn(...params));
        }
        return cache.get(key);
    };
}

//
// let callCount = 0;
// const memoizedFn = memoize(function (a, b) {
//
//     callCount += 1;
//     return a + b;
// });
// memoizedFn(2, 3) // 5
// memoizedFn(2, 3) // 5
// console.log(callCount) // 2
//
Python
Python

```

```
# Define a function 'memoize' that takes a function 'fn' and returns a memoized version.
def memoize(fn):
    cache = {} # Dictionary to cache computed values
    call_count = 0 # Counter for real function calls

    # The wrapper function handles any number of arguments.
    def wrapper(*args):
        nonlocal call_count
        # Use the arguments tuple as a key (order sensitive).
        if args not in cache:
            call_count += 1 # Increment call count when computing new result
            cache[args] = fn(*args)
            return cache[args]

    # Expose the getCallCount method on the memoized function.
    wrapper.getCallCount = lambda: call_count
    return wrapper

# Example usage:
# def sum_func(a, b): return a + b
# memoization = memoization_func
# print(memoization(2, 2))
# print(memoization(2, 2))
# print(memoization.getCallCount())
```

[7] JavaScript — Pattern Solution (stored optimal)

Python

```
# Define a function 'memoize' that takes a function 'fn' and returns a memoized version.
def memoize(fn):
    cache = {} # Dictionary to cache computed values
    call_count = 0 # Counter for real function calls

    # The wrapper function handles any number of arguments.
    def wrapper(*args):
        nonlocal call_count
        # Use the arguments tuple as a key (order sensitive).
        if args not in cache:
            call_count += 1 # Increment call count when computing new result
            cache[args] = fn(*args)
            return cache[args]

    # Expose the getCallCount method on the memoized function.
    wrapper.getCallCount = lambda: call_count
    return wrapper

# Example usage:
# def sum_func(a, b): return a + b
# memoization = memoization_func
# print(memoization(2, 2))
# print(memoization(2, 2))
# print(memoization.getCallCount())
```

C++

```
// C++ solution using variadic templates and tuple as the cache key.
// Note: This solution assumes that the function parameters can be stored in a tuple and hashed.
// A helper struct to combine hashes for tuple elements is used.

#include <functional>
#include <tuple>
#include <unordered_map>
#include <iostream>

// Helper to hash a tuple (C++11 and later)
namespace std {
    template <class T>
    inline void hash_combine(std::size_t &seed, const T &v) {
        seed ^= std::hash<T>{}(v) + 0x9e3779b9 + (seed << 6) + (seed >> 2);
    }

    template <typename Tuple, size_t Index = std::tuple_size<Tuple>::value - 1>
    struct TupleHashingImpl {
        static void apply(std::size_t &seed, const Tuple &tuple) {
            TupleHashingImpl<Tuple, Index>::apply(seed, tuple);
            hash_combine(seed, std::get<Index>(tuple));
        }
    };

    template <typename Tuple>
    struct TupleHashingImpl<Tuple, 0> {
        static void apply(std::size_t &seed, const Tuple &tuple) {
            hash_combine(seed, std::get<0>(tuple));
        }
    };

    template <typename ... Ts>
    struct hash<std::tuple<Ts...>> {
        static void apply(std::size_t &seed, const Tuple &tuple) {
            TupleHashingImpl<Tuple, tuple.size() - 1>::apply(seed, tuple);
            return seed;
        }
    };
}

// Generic Memoize class using a lambda function.
template <typename Func, typename Ret, typename... Args>
class Memoize {
public:
    // Constructor takes in the function to memoize.
    Memoize(Func f) : fn(f), callCount(0) {}

    // Overloading operator() to make the class callable.
    Ret operator()(Args... args) {
```

```
auto key = std::make_tuple(args...); // Create key with the given arguments
if (cache.find(key) == cache.end()) {
    callCount++;
    cache[key] = fn(args...);
}
return cache[key];
}

// Method to get the call count.
int getCallCount() const {
    return callCount;
}

private:
    Func fn;
    int callCount;
    std::unordered_map<std::tuple<Args...>, Ret> cache; // Cache mapping argument tuple to result.
};

// Helper function to create a Memoize instance.
template <typename Func, typename Ret, typename... Args>
auto memoizeFunc(F f) {
    return Memoize<Func, Ret, Args...>{f};
}

// Example usage:
// int sum_func(a, b) { return a + b; }
// int main() {
//     auto memoization = memoizeFunc(sum_func);
//     auto memoization = memoizeFunc(sum_func);
//     std::cout << memoization(2, 2) << std::endl;
// }
```

#2633 MEDIUM

Convert Object to JSON String

LeetCode - Simple - WubCC - LeetCode - Editorial

JavaScript

Like Premium 98

Problem:

Given a value, return a valid JSON string of that value. The value can be a string, number, array, object, boolean, or null. The returned string should not include extra spaces. The order of keys should be the same as the order returned by Object.keys().

Please solve it without using the built-in JSON.stringify method.

Example 1:

```
Input: object = {"y":1,"x":2}
Output: '{"y":1,"x":2}'
Explanation:
Return the JSON representation.
Note that the order of keys should be the same as the order returned by Object.keys().
Example 2:
```

```
Input: object = {"a":"str","b":-12,"c":true,"d":null}
Output: '{"a":"str","b":-12,"c":true,"d":null}'
Explanation:
The primitives of JSON are strings, numbers, booleans, and null.

Example 3:
Input: object = {"key":{"a":1,"b":{}},null,"Hello"}
Output: '{"key":{"a":1,"b":{}},null,"Hello"}'
Explanation:
Objects and arrays can include other objects and arrays.

Example 4:
Input: object = true
Output: true
Explanation:
Primitive types are valid inputs.

Constraints:
• value is a valid JSON value
• 1 <= JSON.stringify(object).length <= 105
• maxNestingLevel <= 1000
• all strings contain only alphanumeric characters
```

TypeScript

```
type JSONValue =
  | null
  | boolean
  | number
  | string
  | JSONValue[]
  | { [key: string]: JSONValue };

function jsonStringify(object: JSONValue): string {
    if (object === null) {
        return 'null';
    }
    if (typeof object === 'boolean') {
        return object === 'true' ? 'true' : 'false';
    }
    if (typeof object === 'string') {
        return `"${object}"`;
    }
    if (Array.isArray(object)) {
        const elem = object.map((item) => jsonStringify(item));
        return `[${elem.join(',')}]`;
    }
    const pairs = Object.keys(object).map(
        (key) => `"${key}":${jsonStringify(object[key])}`
    );
    return `{${pairs.join(',')}}`;
}
```

```
Python
Python

def myStringify(value):
    # If value is None, return "null"
    if value is None:
        return "null"
    # If value is a boolean, convert to "true" or "false"
    if isinstance(value, bool):
        return "true" if value else "false"
    # If value is a number, return its string representation
    if isinstance(value, (int, float)):
        return str(value)
    # If value is a string, enclose it in double quotes
    if isinstance(value, str):
        return '"' + value + '"'
    # If value is a list, recursively process its elements
    if isinstance(value, list):
        return '[' + ','.join(myStringify(item) for item in value) + ']'
    # If value is a dictionary, iterate keys in the order of insertion
    if isinstance(value, dict):
        items = []
        for key in value:
            items.append('"' + key + '":' + myStringify(value[key]))
        return '{' + ','.join(items) + '}'
    # Fallback (should not be reached with valid JSON types)
    return ""

# Example usage:
test_obj = {"y": 1, "x": 2}
print(myStringify(test_obj))
```

[7] JavaScript — Pattern Solution (stored optimal)

Python

```
def myStringify(value):
    # If value is None, return "null"
    if value is None:
        return "null"
    # If value is a boolean, convert to "true" or "false"
    if isinstance(value, bool):
        return "true" if value else "false"
    # If value is a number, return its string representation
    if isinstance(value, (int, float)):
        return str(value)
    # If value is a string, enclose it in double quotes
    if isinstance(value, str):
        return '"' + value + '"'
    # If value is a list, recursively process its elements
    if isinstance(value, list):
        return '[' + ','.join(myStringify(item) for item in value) + ']'
    # If value is a dictionary, iterate keys in the order of insertion
    if isinstance(value, dict):
        items = []
        for key in value:
            items.append('"' + key + '":' + myStringify(value[key]))
        return '{' + ','.join(items) + '}'
    # Fallback (should not be reached with valid JSON types)
    return ""

# Example usage:
test_obj = {"y": 1, "x": 2}
print(myStringify(test_obj))
```

C++

```
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>

// To preserve key insertion order in a JSON object, we use a vector of pairs.

struct JSONValue {
    enum Type { Null, Boolean, Number, String, Array, Object } type;
    bool boolVal;
    double numVal;
    std::string strVal;
    std::vector<JSONValue> arrVal;
    std::vector<std::pair<std::string, JSONValue>> objVal;
};

// Constructing the JSON types
JSONValue() : type(Null) {}
JSONValue(std::nullptr_t) : type(Null) {}
JSONValue(bool b) : type(Boolean), boolVal(b) {}
JSONValue(double d) : type(Number), numVal(d) {}
JSONValue(const std::string& str) : type(String), strVal(str) {}
JSONValue(const char* s) : type(String), strVal(s) {}
JSONValue(const std::vector<JSONValue&> arr) : type(Array), arrVal(arr) {}
JSONValue(const std::vector<std::pair<std::string, JSONValue&>> obj) : type(Object), objVal(obj) {}
};
```

```

std::string myStringify(const JsonValue &val) {
    std::ostringstream oss;
    switch(val.type) {
        case JsonValue::Null:
            return "null";
        case JsonValue::Boolean:
            return val.isBoolean() ? "true" : "false";
        case JsonValue::Number:
            oss << val.asNumber();
            return oss.str();
        case JsonValue::String:
            return "\"" + value.strVal + "\"";
        case JsonValue::Array: {
            std::string res = "[";
            for (size_t i = 0; i < val.asArray().size(); i++) {
                res << myStringify(val.asArray()[i]);
                if (i < val.asArray().size() - 1)
                    res << ",";
            }
            res << "]";
            return res;
        }
        case JsonValue::Object: {
            std::string res = "{";
            for (size_t i = 0; i < val.asObject().size(); i++) {
                res << "\"" + value.objVal[i].first + "\"": " + myStringify(value.objVal[i].second);
                if (i < val.asObject().size() - 1)
                    res << ",";
            }
            res << "}";
            return res;
        }
    }
    return "";
}

int main() {
    // Example: ["a", ["b", "c"]]
    JsonValue jsonObj{{
        "a", JsonValue{1},
        "b", JsonValue{2}
    }};
    std::cout << myStringify(jsonObj) << std::endl;
    return 0;
}

```

#2636 MEDIUM

Promise Pool

[LeetCode](#) · [SimpleList](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

JavaScript · Concurrency

List: Premium 98

Problem:

JavaScript · LeetCode — By Pattern

— 133 —

LeetCode

Given an array of asynchronous functions and a pool limit n , return an asynchronous function `promisePool`. It should return a promise that resolves when all the input functions resolve.

Pool limit is defined as the maximum number promises that can be pending at once. `promisePool` should begin execution of as many functions as possible and continue executing new functions when old promises resolve. `promisePool` should execute `functions[i]` then `functions[i + 1]` then `functions[i + 2]`, etc. When the last promise resolves, `promisePool` should also resolve.

For example, if $n = 1$, `promisePool` will execute one function at a time in series. However, if $n = 2$, it first executes two functions. When either of the two functions resolve, a 3rd function should be executed (if available), and so on until there are no functions left to execute.

You can assume all functions never reject. It is acceptable for `promisePool` to return a promise that resolves any value.

Example 1:

```

Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 2
Output: [[300,400,500],500]
Explanation:
Three functions are passed in. They sleep for 300ms, 400ms, and 200ms respectively.
They resolve at 300ms, 400ms, and 500ms respectively. The returned promise resolves at 500ms.
At t=0, the first 2 functions are executed. The pool size limit of 2 is reached.
At t=300, the 1st function resolves, and the 3rd function is executed. Pool size is 2.
At t=400, the 2nd function resolves. There is nothing left to execute. Pool size is 1.
At t=500, the 3rd function resolves. Pool size is zero so the returned promise also resolves.

```

Example 2:

```

Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 5
Output: [[300,400,200],400]
Explanation:
The three input promises resolve at 300ms, 400ms, and 200ms respectively.
The returned promise resolves at 400ms.
At t=0, all 3 functions are executed. The pool limit of 5 is never met.
At t=200, the 3rd function resolves. Pool size is 2.
At t=300, the 1st function resolves. Pool size is 1.
At t=400, the 2nd function resolves. Pool size is 0, so the returned promise also resolves.

```

Example 3:

```

Input:
functions = [

```

JavaScript · LeetCode — By Pattern

— 134 —

LeetCode

```

() => new Promise(res => setTimeout(res, 300)),
() => new Promise(res => setTimeout(res, 400)),
() => new Promise(res => setTimeout(res, 200))
]
n = 1
Output: [[300,700,900],900]
Explanation:
The three input promises resolve at 300ms, 700ms, and 900ms respectively.
The returned promise resolves at 900ms.
At t=0, the 1st function is executed. Pool size is 1.
At t=300, the 1st function resolves and the 2nd function is executed. Pool size is 1.
At t=700, the 2nd function resolves and the 3rd function is executed. Pool size is 1.
At t=900, the 3rd function resolves. Pool size is 0 so the returned promise resolves.

```

Constraints:

- $0 <= \text{functions.length} <= 10$
- $1 <= n <= 10$

TypeScript

TypeScript

```

type F = () => Promise-any;

function promisePool(functions: F[], n: number): Promise-any {
    const next = () => functions.length ? Promise.resolve() : Promise.resolve();
    return Promise.all(functions.map((fn, i) => Promise.resolve(fn()).then(next)));
}

```

TypeScript

```

type F = () => Promise-any;

function promisePool(functions: F[], n: number): Promise-any {
    const wrappers = functions.map(fn => async () => {
        await fn();
        const next = waiting.shift();
        next && (await next());
    });

    const running = wrappers.slice(0, n);
    const waiting = wrappers.slice(n);
    return Promise.all(running.map(fn => fn()));
}

```

TypeScript

JavaScript · LeetCode — By Pattern

— 135 —

LeetCode

```

type F = () => Promise-any;

function promisePool(functions: F[], n: number): Promise-any {
    const wrappers = functions.map(fn => async () => {
        await fn();
        const next = waiting.shift();
        next && (await next());
    });

    const running = wrappers.slice(0, n);
    const waiting = wrappers.slice(n);
    return Promise.all(running.map(fn => fn()));
}

```

Python

Python

```

import asyncio

async def promise_pool(functions, n):
    # Shared index among all tasks.
    i = 0

    async def worker():
        nonlocal i
        while i < len(functions):
            # Fetch the current function and increment the index for next worker.
            fn = functions[i]
            i += 1
            # Await the async function.
            await fn()

        # Create a worker tasks.
        tasks = [asyncio.create_task(worker()) for _ in range(n)]
        # Wait until all worker finish.
        await asyncio.gather(*tasks)

    # Example usage:
    if __name__ == "__main__":
        async def async_task(delay):
            await asyncio.sleep(delay)

        functions = [
            lambda: async_task(0.3),
            lambda: async_task(0.4),
            lambda: async_task(0.2)
        ]
        asyncio.run(promise_pool(functions, 2))

```

[*] JavaScript — Pattern Solution (matched from community answers)

Python

JavaScript · LeetCode — By Pattern

— 136 —

LeetCode

```

import asyncio

async def promise_pool(functions, n):
    # Shared index among all tasks.
    i = 0

    async def worker():
        nonlocal i
        while i < len(functions):
            # Fetch the current function and increment the index for next worker.
            fn = functions[i]
            i += 1
            # Await the async function.
            await fn()

        # Create a worker tasks.
        tasks = [asyncio.create_task(worker()) for _ in range(n)]
        # Wait until all worker finish.
        await asyncio.gather(*tasks)

    # Example usage:
    if __name__ == "__main__":
        async def async_task(delay):
            await asyncio.sleep(delay)

        functions = [
            lambda: async_task(0.3),
            lambda: async_task(0.4),
            lambda: async_task(0.2)
        ]
        asyncio.run(promise_pool(functions, 2))

```

#2675 MEDIUM

Array of Objects to Matrix

[LeetCode](#) · [SimpleList](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

JavaScript · Array

List: Premium 98

Problem:

Write a function that converts an array of objects `arr` into a matrix `m`.

`arr` is an array of objects or arrays. Each item in the array can be deeply nested with child arrays and child objects. It can also contain numbers, strings, booleans, and null values.

The first row in `m` should be the column names. If there is no nesting, the column names are the unique keys within the objects. If there is nesting, the column names are the respective paths in the object separated by `"."`.

Each of the remaining rows corresponds to an object in `arr`. Each value in the matrix corresponds to a value in an object. If a given object doesn't contain a value for a given column, the cell should contain an empty string `"`.

The columns in the matrix should be in lexicographically ascending order.

Example 1:

```

Input:
arr = [
  {"b": 1, "a": 2},
  {"b": 3, "a": 4}
]
Output:
[
  ["a", "b"],
  [2, 3],
  [4, 3]
]

```

Explanation:

There are two unique column names in the two objects: `"a"` and `"b"`. `"a"` corresponds with `[2, 4]`. `"b"` corresponds with `[1, 3]`.

Example 2:

```

Input:
arr = [
  {"a": 1, "b": 2},
  {"c": 3, "d": 4},
  {}
]
Output:
[
  ["a", "b", "c", "d"],
  [1, 2, "", ""],
  ["", "", 3, 4],
  ["", "", "", ""]]
]

```

Explanation:

There are 4 unique column names: `"a"`, `"b"`, `"c"`, `"d"`. The first object has values associated with `"a"` and `"b"`. The second object has values associated with `"c"` and `"d"`. The third object has no keys, so it is just a row of empty strings.

Example 3:

```

Input:
arr = [
  {"a": {"b": 1, "c": 2}},
  {"a": {"b": 3, "d": 4}}
]
Output:
[
  ["a.b", "a.c", "a.d"],
  [1, 2, ""],
  [3, "", 4]
]

```

Explanation:

JavaScript · LeetCode — By Pattern

— 138 —

LeetCode

In this example, the objects are nested. The keys represent the full path to each value separated by periods.

There are three paths: "a.b", "a.c", and "a.d".

Example 4:

```
Input:
arr = [
  [{"a": null}],
  [{"b": true}],
  [{"c": "x"}]
]
Output:
[
  ["0.a", "0.b", "0.c"],
  [null, "", ""],
  [true, true, ""],
  [true, "", "x"]
]
```

Explanation:
Arrays are also considered objects with their keys being their indices.
Each array has one element so the keys are "0.a", "0.b", and "0.c".

Example 5:

```
Input:
arr = [
  {},
  {},
  {}
]
Output:
[
  [],
  [],
  [],
  []
]
```

Explanation:
There are no keys so every row is an empty array.

Constraints:

- arr is a valid JSON array
- 1 <= arr.length <= 1000
- unique keys <= 1000

TypeScript

TypeScript

JavaScript · LeetCode · By Pattern

— 139 —

LeetCode

```
function jsonToMatrix(arr: any[]): (string | number | boolean | null)[][] {
  const isObj = (a: any) => a !== null && typeof a === 'object';

  // Returns the keys of a JSON-like object by recursively unsplicing the nests.
  const getKeys = (json: any: string[]) => {
    if (!isObj(json)) {
      return [''];
    }
    return Object.keys(json).reduce((acc: string[], curKey: string) => {
      return [
        acc.push(
          ...getKeys(json[curKey]).map((nextKey: string) =>
            nextKey === '' ? curKey : `${curKey}.${nextKey}`
          )
        ),
        acc
      ];
    }, []);
  };

  const sortedKeys: string[] = [
    ...arr.reduce((acc: Set<string>, curr: any) => {
      getKeys(curr).forEach((key: string) => acc.add(key));
      return acc;
    }, new Set<string>()),
  ].sort();

  // Generate the value of 'obj' keyed by 'nextKey'.
  const getObjVal = (
    obj: any,
    nextKey: string
  ): string | number | boolean | null => {
    let value: any = obj;
    for (const key of nextKey.split('.')) {
      if (!isObj(value) || !key in value) {
        return ' ';
      }
      value = value[key];
    }
    return isObj(value) ? ' ' : value;
  };

  const matrix: (string | number | boolean | null)[][] = [sortedKeys];
  arr.forEach((obj: any) => {
    matrix.push(
      sortedKeys.map((nextKey: string) => getObjVal(obj, nextKey))
    );
  });

  return matrix;
}
```

TypeScript

JavaScript · LeetCode · By Pattern

— 140 —

LeetCode

```
function jsonToMatrix(arr: any[]): (string | number | boolean | null)[] {
  const dfs = (key: string, obj: any) => {
    if (!
      typeof obj === 'number' ||
      typeof obj === 'string' ||
      typeof obj === 'boolean' ||
      obj === null
    ) {
      return [(key): obj];
    }
    const res: any[] = [];
    for (const [k, v] of Object.entries(obj)) {
      const nextKey = key ? `${key}.${k}` : k;
      res.push(dfs(nextKey, v));
    }
    return res.flat();
  };

  const kv = arr.map(obj => dfs('', obj));
  const keys = [
    ...new Set(
      kv
        .flat()
        .map(obj => Object.keys(obj))
        .flat()
    )
  ].sort();
  const ans: any[] = [keys];
  for (const row of kv) {
    const newRow: any[] = [];
    for (const key of keys) {
      const v = row.find(r => r.hasOwnProperty(key)).key;
      newRow.push(v === undefined ? ' ' : v);
    }
    ans.push(newRow);
  }
  return ans;
}
```

[1] JavaScript — Pattern Solution (stored optimal)

Python

JavaScript · LeetCode · By Pattern

— 141 —

LeetCode

```
from typing import List

class Solution:
    def jsonToMatrix(self, arr: List[dict]) -> List[List]:
        # Collect all unique keys across all objects
        keys = set()
        for obj in arr:
            keys.update(obj.keys())
        keys = sorted(keys) # sort alphabetically for consistent column order

        # First row = column headers
        result = [keys]

        # Each subsequent row = object values (empty string if key missing)
        for obj in arr:
            row = [obj.get(key, '') for key in keys]
            result.append(row)

        return result
```

C++

```
#include <vector>
#include <map>
#include <set>
#include <string>
using namespace std;

class Solution {
public:
    vector<vector<string>> jsonToMatrix(vector<map<string, string>> & arr) {
        // Collect all unique keys
        set<string> keySet;
        for (auto& obj : arr) {
            for (auto& p : obj) {
                keySet.insert(p.first);
            }
        }
        vector<string> keys(keySet.begin(), keySet.end());
        vector<vector<string>> result;

        // First row = headers
        result.push_back(keys);

        // Remaining rows = values (empty string if key not present)
        for (auto& obj : arr) {
            vector<string> row;
            for (auto& key : keys) {
                row.push_back(obj.count(key) ? obj.at(key) : "");
            }
            result.push_back(row);
        }

        return result;
    }
};
```

JavaScript · LeetCode · By Pattern

— 142 —

LeetCode

#2700

MEDIUM

Differences Between Two Objects

LeetCode · Simplified · WUCC · LeetCode · Editorial

JavaScript

List: Premium 98

Problem:

Write a function that accepts two deeply nested objects or arrays obj1 and obj2 and returns a new object representing their differences.

The function should compare the properties of the two objects and identify any changes. The returned object should only contains keys where the value is different from obj1 to obj2.

For each changed key, the value should be represented as an array [obj1 value, obj2 value]. Keys that exist in one object but not in the other should not be included in the returned object. The end result should be a deeply nested object where each leaf value is a difference array.

When comparing two arrays, the indices of the arrays are considered to be their keys.

You may assume that both objects are the output of JSON.parse.

Example 1:

```
Input:
obj1 = {}
obj2 = {
  "a": 1,
  "b": 2
}
```

```
Output: {}
```

Explanation: There were no modifications made to obj1. New keys "a" and "b" appear in obj2, but keys that are added or removed should be ignored.

Example 2:

```
Input:
obj1 = {
  "a": 1,
  "v": 3,
  "x": {},
  "z": {
    "a": null
  }
}
```

```
obj2 = {
  "a": 2,
  "v": 4,
  "x": {},
  "z": {
    "a": 2
  }
}
```

```
Output:
{
  "a": 1,
  "v": 3,
  "x": {},
  "z": {
    "a": 2
  }
}
```

JavaScript · LeetCode · By Pattern

— 143 —

LeetCode

```
"a": [1, 2],
"v": [3, 4],
"x": {
  "a": [null, 2]
}
```

Explanation: The keys "a", "v", and "x" all had changes applied. "a" was changed from 1 to 2. "v" was changed from 3 to 4. "x" had a change applied to a child object. "x.a" was changed from null to 2.

Example 3:

```
Input:
obj1 = {
  "a": 5,
  "v": 6,
  "x": [1, 2, 4, [2, 5, 7]]
}
obj2 = {
  "a": 5,
  "v": 7,
  "x": [1, 2, 3, [3]]
}
```

```
Output:
{
  "v": [6, 7],
  "z": {
    "x": [4, 3],
    "y": {
      "0": [2, 1]
    }
  }
}
```

Explanation: In obj1 and obj2, the keys "v" and "x" have different assigned values. "a" is ignored because the value is unchanged. In the key "x", there is a nested array. Arrays are treated like objects where the indices are keys. There were two alterations to the array: x[2] and x[3][0]. x[0] and x[1] were unchanged and thus not included. x[3][1] and x[3][2] were removed and thus not included.

Example 4:

```
Input:
obj1 = {
  "a": {"b": 1},
}
obj2 = {
  "a": [5],
}
Output:
{
  "a": [{"b": 1}, [5]]
}
```

Explanation: The key "a" exists in both objects. Since the two associated values have

JavaScript · LeetCode · By Pattern

— 144 —

LeetCode

different types, they are placed in the difference array.

Example 5:

```
Input:
obj1 = {
  "a": [1, 2, {}],
  "b": false
}
obj2 = {
  "b": false,
  "a": [1, 2, {}]
}
```

Output:

()
Explanation: Apart from a different ordering of keys, the two objects are identical so an empty object is returned.

Constraints:

- obj1 and obj2 are valid JSON objects or arrays
- 2 <= JSON.stringify(obj1).length <= 104
- 2 <= JSON.stringify(obj2).length <= 104

TypeScript

```
TypeScript
type JSONValue =
  | null
  | boolean
  | number
  | string
  | JSONValue[]
  | { [key: string]: JSONValue };
type Obj = Record<string, JSONValue> | Array<JSONValue>;

function objDiff(obj1: Obj, obj2: Obj): Obj {
  if (obj1 === obj2) {
    return {};
  }
  if (obj1 === null || obj2 === null) {
    return {obj1, obj2};
  }
  if (typeof obj1 !== 'object' || typeof obj2 !== 'object') {
    return {obj1, obj2};
  }
  if (Array.isArray(obj1) !== Array.isArray(obj2)) {
    return {obj1, obj2};
  }
  const ans = {};
  for (const key in obj1) {
    if (key in obj2) {

```

```
const subDiff = objDiff(obj[key], obj2[key]);
if (Object.keys(subDiff).length > 0) {
  ans[key] = subDiff;
}
}
return ans;
}
```

Python

```
Python
def diff_objects(obj1, obj2):
    # If both values are dictionaries
    if isinstance(obj1, dict) and isinstance(obj2, dict):
        diff = {}
        # Only iterate on the keys present in both dictionaries
        for key in obj1.keys() & obj2.keys():
            sub_diff = diff_objects(obj1[key], obj2[key])
            # Only add if there is an actual difference
            if sub_diff is not None:
                diff[key] = sub_diff
        return diff if diff else None
    # If both values are lists, treat indices as keys
    elif isinstance(obj1, list) and isinstance(obj2, list):
        diff = {}
        # Iterate over common indices only
        for i in range(min(len(obj1), len(obj2))):
            sub_diff = diff_objects(obj1[i], obj2[i])
            if sub_diff is not None:
                # Use string keys for consistency with object keys
                diff[str(i)] = sub_diff
        return diff if diff else None
    else:
        # If the two values are equal, there's no difference
        if obj1 == obj2:
            return None
        else:
            # If types differ or values are different, return the difference array.
            return [obj1, obj2]

# A wrapper function to ensure we always return an object (or empty dict)
def difference_between_two_objects(obj1, obj2):
    result = diff_objects(obj1, obj2)
    return result if result is not None else {}

# Example usage:
if __name__ == "__main__":
    obj1 = {"a": 5, "b": 6, "c": [1, 2, 4, [2, 5, 7]]}
    obj2 = {"c": 5, "a": 7, "b": [1, 2, 3, [1]]}
    print(difference_between_two_objects(obj1, obj2))
```

JavaScript — Pattern Solution (stored optimal)

Python

```
def diff_objects(obj1, obj2):
    # If both values are dictionaries
    if isinstance(obj1, dict) and isinstance(obj2, dict):
        diff = {}
        # Only iterate on the keys present in both dictionaries
        for key in obj1.keys() & obj2.keys():
            sub_diff = diff_objects(obj1[key], obj2[key])
            # Only add if there is an actual difference
            if sub_diff is not None:
                diff[key] = sub_diff
        return diff if diff else None
    # If both values are lists, treat indices as keys
    elif isinstance(obj1, list) and isinstance(obj2, list):
        diff = {}
        # Iterate over common indices only
        for i in range(min(len(obj1), len(obj2))):
            sub_diff = diff_objects(obj1[i], obj2[i])
            if sub_diff is not None:
                # Use string keys for consistency with object keys
                diff[str(i)] = sub_diff
        return diff if diff else None
    else:
        # If the two values are equal, there's no difference
        if obj1 == obj2:
            return None
        else:
            # If types differ or values are different, return the difference array.
            return [obj1, obj2]

# A wrapper function to ensure we always return an object (or empty dict)
def difference_between_two_objects(obj1, obj2):
    result = diff_objects(obj1, obj2)
    return result if result is not None else {}

# Example usage:
if __name__ == "__main__":
    obj1 = {"a": 5, "b": 6, "c": [1, 2, 4, [2, 5, 7]]}
    obj2 = {"c": 5, "a": 7, "b": [1, 2, 3, [1]]}
    print(difference_between_two_objects(obj1, obj2))
```

C++

```
// Include necessary headers
#include <iostream>
#include <map>
#include <vector>
#include <string>
#include <optional>
#include <type_traits>
using namespace std;

// Define a variant type for JSON values.
using JSONValue = variant<bool, multi_ptr_t, map<string, JSONValue> or vector<JSONValue>;

struct JSONObject {
    using JSONValue = variant<bool, multi_ptr_t, double, string, bool, JSONObject, JSONArray>;
};

// Helper function to check equality of two JSONValue objects
bool jsonEqual(const JSONValue &a, const JSONValue &b) {
    // Recursive function to compute difference. Returns optional<JSONValue> containing JSONObject if
    // difference exist.
    optional<JSONValue> diffJSON(const JSONValue &a, const JSONValue &b) {
        // Check if both are JSONObject
        if(auto obj1 = std::get_if<JSONObject>(&a);) {
            if(auto obj2 = std::get_if<JSONObject>(&b);) {
                JSONObject diff;
                // Iterate over keys present in both objects
                for (const auto &entry : obj1) {
                    const string &key = entry.first;
                    auto it = obj2->find(key);
                    if (it != obj2->end()) {
                        auto subDiff = diffJSON(entry.second, it->second);
                        if (subDiff.has_value()) {
                            // Append the diff into result
                            diff[key] = subDiff.value();
                        }
                    }
                }
                if (diff.empty()) return JSONValue(diff);
                else return multi_ptr(diff);
            }
        }
        // Check if both are JSONArray
        if(auto arr1 = std::get_if<JSONArray>(&a);) {
            if(auto arr2 = std::get_if<JSONArray>(&b);) {
                JSONObject diff;
                // Iterate over common indices

```

```
size_t len = min(arr1->size(), arr2->size());
for (size_t i = 0; i < len; i++) {
    auto subDiff = diffJSON(arr1[i], arr2[i]);
    if (subDiff.has_value()) {
        diffIn_string[i] = subDiff.value();
    }
}
if (!diff.empty()) return JSONValue(diff);
else return multi_ptr;
}
}

// For primitive values or when types differ
if (jsonEqual(a, b))
    return multi_ptr;
else
    return JSONValue(vector<JSONValue>{a, b}); // Represent the difference as an array of two
elements.
}

// Simple equality checker for JSONValue primitives and containers
bool jsonEqual(const JSONValue &a, const JSONValue &b) {
    // The method's equality for simple types. For objects/arrays we assume they are deeply compared.
    return a == b;
}

// A wrapper function that returns a JSONObject representing differences
JSONObject differenceBetweenTwoObjects(const JSONValue &obj1, const JSONValue &obj2) {
    auto result = diffJSON(obj1, obj2);
    if (result.has_value()) {
        // The result should be an object
        if(auto diffObj = std::get_if<JSONObject>(&result.value());)

```

Quick Review — JavaScript 7 questions · insights · complexity · solution

#2627 Debounce [Medium]

Key Insights

- Each call to the debounced function resets the timer.
- Only the most recent function call that isn't subsequently cancelled will eventually execute.
- Passing of parameters is handled by storing the arguments of the latest call.
- The solution leverages timers (or equivalent scheduling mechanisms) and cancellation techniques.

Space & Time

Time Complexity: O(1) per call, as each invocation only clears and sets a timer.
Space Complexity: O(1), since only a single timer reference is maintained.

Solution

The solution creates a wrapper function that maintains an internal timer reference. On each call to the debounced function, if a timer exists, it is cancelled. A new timer is then set to execute the original function after `t` milliseconds with the provided arguments. This ensures that only the last invocation runs if multiple calls occur within the delay period.

#2628 JSON Deep Equal [Medium]

Key Insights

- Use recursion to compare nested structures.
- For primitive types, a simple equality check (`===` in JavaScript, `==` in Python, etc.) suffices.
- For arrays, verify both have the same length and compare each element recursively.
- For objects, ensure both have the same keys (order does not matter) then recursively compare the corresponding values.
- Handle cases where one operand is null or differs by type early to avoid unnecessary recursion.

Space & Time

Time Complexity: O(N) in the worst-case, where `N` is the total number of elements/values in the JSON structure.
Space Complexity: O(N) due to the recursion call stack in the worst-case scenario of deeply nested objects or arrays.

Solution

We solve this problem using a recursive approach. The algorithm first checks if both values are strictly equal, which handles primitives immediately. For arrays and objects, it verifies that the sizes (length for arrays and number of keys for objects) match. Then it iterates into each element (for arrays) or checks each key-value pair (for objects) to ensure that they are also deeply equal. For objects, the order of keys is irrelevant, so we compare key sets. This method naturally handles nested structures by delegating equality checks to the same function.

#2630 Memoize [Medium]

Key Insights

- Cache previously computed results using a data structure keyed by the function arguments.
- Ensure the key uniquely represents the argument order; simple tuple (or equivalent) suffices.
- Use closures (or object state) to maintain both the cache and a call count tracking actual function invocations.
- Expose a method (`getCallCount`) on the memoized function to report the number of times the original function was actually called.
- The memoized version works for functions that might take one or more parameters.

Space & Time

Time Complexity: O(1) average for each call assuming efficient hashing of arguments.

Space Complexity: $O(n)$ for caching n distinct input combinations.

Solution

We use a hash-based cache (a dictionary or map) to store results for every unique set of arguments. Upon each call, we check if the key (constructed from the input arguments) exists in the cache. If not, we increment a counter, call the original function, store the result in the cache, and return it. Otherwise, we return the cached result. The memoized function is augmented with a `getCallCount` method to report the number of actual calls made to the original function.

#2633 Convert Object to JSON String [Medium]

Key Insights

- The solution must support all JSON primitives: strings, numbers, booleans, and null.
- Composite types such as arrays and objects require recursive traversal for proper serializing.
- For strings, wrap them in double quotes.
- For arrays, recursively process every element and join them with commas between square brackets.
- For objects, iterate keys in insertion order and combine key-value pairs (key must be a string in double quotes) with commas between curly braces.
- Use recursion carefully to handle nested structures without adding unnecessary spaces.

Space & Time

Time Complexity: $O(n)$, where n is the total number of elements/characters in the JSON structure.

Space Complexity: $O(n)$ due to the recursion stack and the constructed output string.

Solution

We use a recursive approach that inspects the type of the value and processes it accordingly:

- If the value is null, output "null".
- If the value is a boolean or number, convert it directly to its string representation.
- For a string, ensure it is enclosed in double quotes.
- For an array, iterate over each element, serialize each recursively, join them with commas, and enclose in square brackets.
- For an object, iterate over its keys in insertion order (or using `Object.keys` in JS / `LinkedHashMap` in Java), serialize both key (wrapped in double quotes) and its corresponding value, join with commas, and enclose in curly braces. This approach leverages recursion to handle arbitrarily nested objects and arrays while ensuring there are no extra spaces in the final string.

#2636 Promise Pool [Medium]

Key Insights

- Use a concurrency control mechanism to limit the number of actively running promises.
- Maintain an index or pointer to track which function from the input array should be executed next.
- Upon the resolution of any promise, start a new one if available until all functions have been processed.
- The overall task is complete when there are no functions left and all running promises have resolved.

Space & Time

Time Complexity: $O(m)$, where m is the number of functions to execute.

Space Complexity: $O(n)$ for the active pool of concurrent promises.

Solution

We can solve the problem by maintaining an index that tracks which asynchronous function to execute next. We define a helper routine (or executor) that:

- Checks if there is a function left to run.
- if yes, increments the index and awaits the result of running that function.
- Once the function completes, the routine recursively calls itself to pick up the next function. We start n such routines (to respect the pool limit) and use `Promise.all` (or the equivalent in other languages) to wait until all concurrent routines have finished executing. This structure ensures that we never run more than n promises concurrently and that a new promise starts as soon as one finishes.

#2675 Array of Objects to Matrix [Medium]

JavaScript · LeetCode · By Pattern

— 151 —

LeetCode

#4 String — 8 questions · #4 of 21

#383

EASY

Ransom Note

LeetCode · SimpleList · WalkCC · LeetCode · Editorial

Hash Table · String · Counting

List: Grid 169

Problem:

Given two strings `ransomNote` and `magazine`, return true if `ransomNote` can be constructed by using the letters from `magazine` and false otherwise.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

Input: `ransomNote = "a", magazine = "b"`

Output: false

Example 2:

Input: `ransomNote = "aa", magazine = "ab"`

Output: false

Example 3:

Input: `ransomNote = "aa", magazine = "aab"`

Output: true

Constraints:

- $1 \leq \text{ransomNote.length}, \text{magazine.length} \leq 105$
- `ransomNote` and `magazine` consist of lowercase English letters.

Python

Python

```
# Function to check if ransomNote can be constructed from magazine
def canConstruct(ransomNote: str, magazine: str) -> bool:
    # Create a dictionary to hold the frequency of each character in magazine
    char_frequency = {}
    for char in magazine:
        char_frequency[char] = char_frequency.get(char, 0) + 1

    # Check each character in ransomNote against the frequency map
    for char in ransomNote:
        if char_frequency.get(char, 0) == 0:
            return False # Character not available or used up
        char_frequency[char] -= 1 # Use the character

    return True

# Example usage:
if __name__ == "__main__":
    print(canConstruct("a", "aab")) # Expected output: True
```

JavaScript · LeetCode · By Pattern

— 153 —

LeetCode

```
class Solution {
    /**
     * @param String ransomNote
     * @param String magazine
     * @return boolean
     */
    function canConstruct(ransomNote, magazine) {
        let map = new Map();
        for (let i = 0; i < magazine.length; i++) {
            map.set(magazine[i], (map.get(magazine[i]) || 0) + 1);
        }
        for (let i = 0; i < ransomNote.length; i++) {
            if (map.get(ransomNote[i]) > 0) {
                map.set(ransomNote[i], map.get(ransomNote[i]) - 1);
            } else {
                return false;
            }
        }
        return true;
    }
}
```

C++

```
class Solution {
    /**
     * @param String ransomNote
     * @param String magazine
     * @return boolean
     */
    function canConstruct(ransomNote, magazine) {
        let map = new Map();
        for (let i = 0; i < magazine.length; i++) {
            map.set(magazine[i], (map.get(magazine[i]) || 0) + 1);
        }
        for (let i = 0; i < ransomNote.length; i++) {
            if (map.get(ransomNote[i]) > 0) {
                map.set(ransomNote[i], map.get(ransomNote[i]) - 1);
            } else {
                return false;
            }
        }
        return true;
    }
}
```

Java

Java

String · LeetCode · By Pattern

— 155 —

LeetCode

Key Insights

- Flatten each object or array into dot-separated paths, such as "a.b" or "a.b.c".
- Use DFS to nested objects and nested arrays are handled uniformly.
- For each row, store a map from flattened path to value.
- Collect every path seen across all rows into a set of column names.
- Sort the column names lexicographically to build the header row.
- For each flattened row map, output values in header order and use an empty string when a column is missing.

Space & Time

Time Complexity: $O(T * R * C \log C)$, where T is the total number of nested nodes visited, R is the number of rows, and C is the number of unique flattened columns.

Space Complexity: $O(R * C + C)$ in the worst case for the flattened row maps and the set of all column names.

Solution

We solve the problem by flattening every input row first. A DFS walks through objects and arrays, extending the current path with either the property name or the array index. When the DFS reaches a primitive value or 'null', it stores that value under the built path. While flattening each row, we also collect every path into a global set of column names. After all rows are processed, we sort the columns lexicographically, place them in the first row of the matrix, and then build each remaining row by reading values from that row's flattened map, using "" for missing paths.

#2700 Differences Between Two Objects [Medium]

Key Insights

- Use recursion to traverse nested objects and arrays.
- Compare only keys that exist in both objects.
- When encountering two values of different types or unequal primitives, record the difference as [value from obj1, value from obj2].
- When both values are objects or arrays, recursively compute their differences and include them only if the resulting nested diff is non-empty.
- Arrays are treated like objects with indices as keys.
- The solution requires careful type checking and handling of recursive structures.

Space & Time

Time Complexity: $O(n)$ in average, where n is the number of keys/elements in the common parts of the structures.

Space Complexity: $O(n)$ due to recursion and storage needed for the output differences.

Solution

We solve the problem using a recursive function that compares keys present in both input objects. For each common key:

- If both values are objects (or both are arrays), recursively compute their differences.
- If the recursive call returns a non-empty object, include that key in the result.
- If the two values are of different types or are primitives that differ, add the key with a difference array [value from obj1, value from obj2].
- Ignore keys that exist only in one object. This recursive approach naturally handles the deeply nested structure while ensuring only common keys with different values are captured.

Python

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        count1 = collections.Counter(ransomNote)
        count2 = collections.Counter(magazine)
        return all(count1[c] <= count2[c] for c in string.ascii_lowercase)
```

Python

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        cnt = Counter(magazine)
        for c in ransomNote:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

Python

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        cnt = Counter(magazine)
        for c in ransomNote:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

C++

C++

```
class Solution {
public:
    // Similar to #204: Valid Anagram
    bool canConstruct(string ransomNote, string magazine) {
        vector<int> count(26);

        for (const char c : magazine)
            ++count[c - 'a'];

        for (const char c : ransomNote) {
            if (count[c - 'a'] == 0)
                return false;
            --count[c - 'a'];
        }

        return true;
    }
};
```

C++

String · LeetCode · By Pattern

— 154 —

LeetCode

```
class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {
        int[] count = new int[26];

        for (final char c : magazine.toCharArray())
            ++count[c - 'a'];

        for (final char c : ransomNote.toCharArray()) {
            if (count[c - 'a'] == 0)
                return false;
            --count[c - 'a'];
        }

        return true;
    }
}
```

Java

```
class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {
        int[] cnt = new int[26];
        for (int i = 0; i < magazine.length(); ++i) {
            ++cnt[magazine.charAt(i) - 'a'];
        }
        for (int i = 0; i < ransomNote.length(); ++i) {
            if (--cnt[ransomNote.charAt(i) - 'a'] < 0)
                return false;
        }
        return true;
    }
}
```

Java

```
public class Solution {
    public bool CanConstruct(string ransomNote, string magazine) {
        int[] cnt = new int[26];
        foreach (var c in magazine) {
            ++cnt[c - 'a'];
        }
        foreach (var c in ransomNote) {
            if (--cnt[c - 'a'] < 0)
                return false;
        }
        return true;
    }
}
```

Java

String · LeetCode · By Pattern

— 156 —

LeetCode

```

class Solution {
public:
    bool canConstruct(string ransomNote, string magazine) {
        int cnt = 0;
        for (int i = 0; i < magazine.length(); ++i) {
            ++cnt[magazine.charAt(i) - 'a'];
        }
        for (int i = 0; i < ransomNote.length(); ++i) {
            if (--cnt[ransomNote.charAt(i) - 'a'] < 0) {
                return false;
            }
        }
        return true;
    }
}

```

Java

```

public class Solution {
public:
    bool canConstruct(string ransomNote, string magazine) {
        int cnt = 0;
        for (int i = 0; i < magazine.length(); ++i) {
            ++cnt[magazine.charAt(i) - 'a'];
        }
        for (int i = 0; i < ransomNote.length(); ++i) {
            if (--cnt[ransomNote.charAt(i) - 'a'] < 0) {
                return false;
            }
        }
        return true;
    }
}

```

TypeScript

```

function canConstruct(ransomNote: string, magazine: string): boolean {
    const cnt: number[] = Array(26).fill(0);
    for (const c of magazine) {
        ++cnt[c.charCodeAt(0) - 97];
    }
    for (const c of ransomNote) {
        if (--cnt[c.charCodeAt(0) - 97] < 0) {
            return false;
        }
    }
    return true;
}

```

TypeScript

```

function canConstruct(ransomNote: string, magazine: string): boolean {
    const cnt: number[] = Array(26).fill(0);
    for (const c of magazine) {
        ++cnt[c.charCodeAt(0) - 97];
    }
    for (const c of ransomNote) {
        if (--cnt[c.charCodeAt(0) - 97] < 0) {
            return false;
        }
    }
    return true;
}

```

[1] String — Pattern Solution (stored optimal)

Python

```

# Function to check if ransom note can be constructed from magazine
def canConstruct(ransomNote: str, magazine: str) -> bool:
    # Create a dictionary to hold the frequency of each character in magazine
    char_frequency = {}
    for char in magazine:
        char_frequency[char] = char_frequency.get(char, 0) + 1

    # Check each character in ransom note against the frequency map
    for char in ransomNote:
        if char_frequency.get(char, 0) == 0:
            return False # Character not available or used up
        char_frequency[char] -= 1 # Use the character

    return True

# Example usage:
if __name__ == "__main__":
    print(canConstruct("aa", "aab")) # Expected output: True

```

C++

```

// Function to check if ransom note can be constructed from magazine
#include <iostream>
#include <string>
#include <unordered_map>
using namespace std;

bool canConstruct(string ransomNote, string magazine) {
    // Use an unordered map to hold frequency counts of each character in magazine
    unordered_map<char, int> frequency;
    for (char ch : magazine) {
        frequency[ch]++;
    }

    // Iterate through each character in ransom note and decrease its count
    for (char ch : ransomNote) {
        if (frequency[ch] == 0) {
            return false; // Character not available or already fully used
        }
        frequency[ch]--;
    }

    return true;
}

// Example usage:
int main() {
    cout << boolalpha << canConstruct("aa", "aab") << endl; // Expected output: true
    return 0;
}

```

#734 EASY

Sentence Similarity

LeetCode · Simplest · WikiCC · LeetCode · Editorial

Array · Hash Table · String

Little Premium 98

Problem:

We can represent a sentence as an array of words, for example, the sentence "I am happy with leetcode" can be represented as arr = ["I","am","happy","with","leetcode"].

Given two sentences sentence1 and sentence2 each represented as a string array and given an array of string pairs similarPairs where similarPairs[i] = [xi, yi] indicates that the two words xi and yi are similar.

Return true if sentence1 and sentence2 are similar, or false if they are not similar.

Two sentences are similar if:

- They have the same length (i.e., the same number of words)
- sentence1[i] and sentence2[i] are similar.

Notice that a word is always similar to itself, also notice that the similarity relation is not transitive. For example, if the words a and b are similar, and the words b and c are similar, a and c are not necessarily similar.

Example 1:

Input: sentence1 = ["great", "acting", "skills"], sentence2 = ["fine", "drama", "talent"], similarPairs = [["great", "fine"], ["drama", "acting"], ["skills", "talent"]]

Output: true

Explanation: The two sentences have the same length and each word i of sentence1 is also similar to the corresponding word in sentence2.

Example 2:

Input: sentence1 = ["great"], sentence2 = ["great"], similarPairs = []

Output: true

Explanation: A word is similar to itself.

Example 3:

Input: sentence1 = ["great"], sentence2 = ["doubleplus", "good"], similarPairs = [["great", "doubleplus"]]

Output: false

Explanation: As they don't have the same length, we return false.

Constraints:

- 1 <= sentence1.length, sentence2.length <= 1000
- 1 <= sentence1[i].length, sentence2[i].length <= 20
- sentence1[i] and sentence2[i] consist of English letters.
- 0 <= similarPairs.length <= 1000
- similarPairs[i].length == 2
- 1 <= xi.length, yi.length <= 20
- xi and yi consist of lower-case and upper-case English letters.
- All the pairs (xi, yi) are distinct.

>> Brute Force [String] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def areSentencesSimilar(self, sentence1, sentence2, similarPairs):
        # Brute Force (O(N^2)) - check all substrings
        n = len(sentence1)
        best = ''
        for i in range(1, n + 1):
            for j in range(i + 1, n + 1):
                sub = sentence1[i:j]
                if len(sub) > len(best): best = sub
        return best

```

Python

```

# Function to determine if two sentences are similar
def areSentencesSimilar(sentence1, sentence2, similarPairs):
    # Check if sentences are of the same length
    if len(sentence1) != len(sentence2):
        return False

    # Build dictionary to store similar word pairs in both directions
    similar_dict = {}
    for word1, word2 in similarPairs:
        if word1 not in similar_dict:
            similar_dict[word1] = set()
        similar_dict[word1].add(word2)
        if word2 not in similar_dict:
            similar_dict[word2] = set()
        similar_dict[word2].add(word1)

    # Check each word pair of words in the sentences
    for w1, w2 in zip(sentence1, sentence2):
        # If words are the same, continue
        if w1 == w2:
            continue
        # Check if w1 is similar to w2 explicitly
        if (w1 not in similar_dict or w2 not in similar_dict) and (w2 not in similar_dict or w1 not in similar_dict):
            return False
        return True

# Example usage:
# sentence1 = ["great", "acting", "skills"]
# sentence2 = ["fine", "drama", "talent"]
# similarPairs = [["great", "fine"], ["drama", "acting"], ["skills", "talent"]]
# print(areSentencesSimilar(sentence1, sentence2, similarPairs)) # Output: True

```

Python

```

class Solution:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str], similarPairs: List[List[str]]) -> bool:
        if len(sentence1) != len(sentence2):
            return False
        # Map[key] := all the similar words of key
        map = collections.defaultdict(set)
        for a, b in similarPairs:
            map[a].add(b)
            map[b].add(a)
        for word1, word2 in zip(sentence1, sentence2):
            if word1 == word2:
                continue
            if word1 not in map:
                return False
            if word2 not in map[word1]:
                return False
        return True

Python
class Solution:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str], similarPairs: List[List[str]]) -> bool:
        if len(sentence1) != len(sentence2):
            return False
        s = (x, y) for x, y in similarPairs
        for x, y in zip(sentence1, sentence2):
            if x != y and (x, y) not in s and (y, x) not in s:
                return False
        return True

Python
class Solution:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str], similarPairs: List[List[str]]) -> bool:
        if len(sentence1) != len(sentence2):
            return False
        s = {(a, b) for a, b in similarPairs}
        return all(
            a == b or (a, b) in s or (b, a) in s for a, b in zip(sentence1, sentence2)
        )

C++
C++

```

```

class Solution {
public:
    bool areSentencesSimilar(vector<string>& sentence1, vector<string>& sentence2,
                           vector<vector<string>&> similarPairs) {
        if (sentence1.size() != sentence2.size())
            return false;

        // map[key] => all the similar words of key
        unordered_map<string, unordered_set<string>> map;

        for (const vector<string>& pair : similarPairs) {
            map[pair[0]].insert(pair[1]);
            map[pair[1]].insert(pair[0]);
        }

        for (int i = 0; i < sentence1.size(); ++i) {
            if (sentence1[i] == sentence2[i])
                continue;
            if (!map.contains(sentence1[i]) ||
                !map.contains(sentence2[i]))
                return false;
            if (!map[sentence1[i]].contains(sentence2[i]))
                return false;
        }

        return true;
    }
};

```

```

C++

class Solution {
public:
    bool areSentencesSimilar(vector<string>& sentence1, vector<string>& sentence2, vector<vector<string>&>
    similarPairs) {
        if (sentence1.size() != sentence2.size()) {
            return false;
        }
        unordered_map<string, int>
        for (const auto& p : similarPairs) {
            s.insert(p[0] + " " + p[1]);
            s.insert(p[1] + " " + p[0]);
        }
        for (int i = 0; i < sentence1.size(); ++i) {
            if (sentence1[i] != sentence2[i] && !s.contains(sentence1[i] + " " + sentence2[i])) {
                return false;
            }
        }
        return true;
    }
};

```

C++

```

class Solution {
public:
    bool areSentencesSimilar(vector<string>& sentence1, vector<string>& sentence2, vector<vector<string>&>
    similarPairs) {
        int n = sentence1.size(), m = sentence2.size();
        if (n != m) return false;
        unordered_set<string> s;
        for (auto & a : similarPairs) s.insert(a[0] + " " + a[1]);
        for (int i = 0; i < n; ++i) {
            string a = sentence1[i], b = sentence2[i];
            if (a != b && !s.count(a + " " + b) && !s.count(b + " " + a)) return false;
        }
        return true;
    }
};

```

Java

```

class Solution {
public boolean areSentencesSimilar(String[] sentence1, String[] sentence2,
    List<List<String>> similarPairs) {
    if (sentence1.length != sentence2.length)
        return false;

    // map[key] => all the similar words of key
    Map<String, Set<String>> map = new HashMap<>();

    for (List<String> pair : similarPairs) {
        map.putIfAbsent(pair.get(0), new HashSet<>());
        map.putIfAbsent(pair.get(1), new HashSet<>());
        map.get(pair.get(0)).add(pair.get(1));
        map.get(pair.get(1)).add(pair.get(0));
    }

    for (int i = 0; i < sentence1.length; ++i) {
        if (sentence1[i].equals(sentence2[i]))
            continue;
        if (!map.containsKey(sentence1[i]) ||
            !map.containsKey(sentence2[i]))
            return false;
        if (!map.get(sentence1[i]).contains(sentence2[i]))
            return false;
    }

    return true;
}

```

Java

```

class Solution {
public:
    bool areSentencesSimilar(
        string[] sentence1, string[] sentence2, list<list<string>> similarPairs) {
        if (sentence1.length != sentence2.length) {
            return false;
        }
        set<list<string>> s = new HashSet<>();
        for (auto p : similarPairs) {
            s.add(p);
        }
        for (int i = 0; i < sentence1.length; ++i) {
            if (sentence1[i].equals(sentence2[i]) ||
                !s.contains(list.of(sentence1[i], sentence2[i])) ||
                !s.contains(list.of(sentence2[i], sentence1[i]))) {
                return false;
            }
        }
        return true;
    }
};

```

```

Java

use std::collections::HashSet;

impl Solution {
    pub fn are_sentences_similar(
        sentence1: Vec<String>,
        sentence2: Vec<String>,
        similar_pairs: Vec<Vec<String>>,
    ) -> bool {
        if sentence1.len() != sentence2.len() {
            return false;
        }

        let s: HashSet<(String, String)> = similar_pairs
            .into_iter()
            .map(|pair| (pair[0].clone(), pair[1].clone()))
            .collect();

        for (x, y) in sentence1.iter().zip(sentence2.iter()) {
            if x == y
                || !s.contains((x.clone(), y.clone()))
                || !s.contains((y.clone(), x.clone()))
            {
                return false;
            }
        }

        true
    }
}

```

Java

```

class Solution {
public:
    bool areSentencesSimilar(
        string[] sentence1, string[] sentence2, list<list<string>> similarPairs) {
        if (sentence1.length != sentence2.length) {
            return false;
        }
        set<list<string>> s = new HashSet<>();
        for (auto p : similarPairs) {
            s.add({p.get(0), p.get(1)});
        }
        for (int i = 0; i < sentence1.length; ++i) {
            string a = sentence1[i], b = sentence2[i];
            if (a.equals(b) || !s.contains(a + " " + b) && !s.contains(b + " " + a)) {
                return false;
            }
        }
        return true;
    }
};

```

```

Java

use std::collections::HashSet;

impl Solution {
    pub fn are_sentences_similar(
        sentence1: Vec<String>,
        sentence2: Vec<String>,
        similar_pairs: Vec<Vec<String>>,
    ) -> bool {
        if sentence1.len() != sentence2.len() {
            return false;
        }

        let s: HashSet<(String, String)> = similar_pairs
            .into_iter()
            .map(|pair| (pair[0].clone(), pair[1].clone()))
            .collect();

        for (x, y) in sentence1.iter().zip(sentence2.iter()) {
            if x == y
                || !s.contains((x.clone(), y.clone()))
                || !s.contains((y.clone(), x.clone()))
            {
                return false;
            }
        }

        true
    }
}

```

JavaScript

JavaScript

```

/**
 * @param {string[]} sentence1
 * @param {string[]} sentence2
 * @param {string[][]} similarPairs
 * @return {boolean}
 */
var areSentencesSimilar = function(sentence1, sentence2, similarPairs) {
    if (sentence1.length !== sentence2.length) {
        return false;
    }
    const s = new Set();
    for (const [x, y] of similarPairs) {
        s.add(x + " " + y);
        s.add(y + " " + x);
    }
    for (let i = 0; i < sentence1.length; i++) {
        if (sentence1[i] !== sentence2[i] && !s.has(sentence1[i] + " " + sentence2[i])) {
            return false;
        }
    }
    return true;
};

```

```

JavaScript

/**
 * @param {string[]} sentence1
 * @param {string[]} sentence2
 * @param {string[][]} similarPairs
 * @return {boolean}
 */
var areSentencesSimilar = function(sentence1, sentence2, similarPairs) {
    if (sentence1.length !== sentence2.length) {
        return false;
    }
    const s = new Set();
    for (const [x, y] of similarPairs) {
        s.add(x + " " + y);
        s.add(y + " " + x);
    }
    for (let i = 0; i < sentence1.length; i++) {
        if (sentence1[i] !== sentence2[i] && !s.has(sentence1[i] + " " + sentence2[i])) {
            return false;
        }
    }
    return true;
};

```

TypeScript

TypeScript

```

function areSentencesSimilar(
    sentence1: string[],
    sentence2: string[],
    similarPairs: string[][]): boolean {
    if (sentence1.length !== sentence2.length) {
        return false;
    }
    const s = new Set<string>();
    for (const [x, y] of similarPairs) {
        s.add(x + " " + y);
        s.add(y + " " + x);
    }
    for (let i = 0; i < sentence1.length; i++) {
        if (sentence1[i] !== sentence2[i] && !s.has(sentence1[i] + " " + sentence2[i])) {
            return false;
        }
    }
    return true;
}

```

```

TypeScript

function areSentencesSimilar(
    sentence1: string[],
    sentence2: string[],
    similarPairs: string[][]): boolean {
    if (sentence1.length !== sentence2.length) {
        return false;
    }
    const s = new Set<string>();
    for (const [x, y] of similarPairs) {
        s.add(x + " " + y);
        s.add(y + " " + x);
    }
    for (let i = 0; i < sentence1.length; i++) {
        if (sentence1[i] !== sentence2[i] && !s.has(sentence1[i] + " " + sentence2[i])) {
            return false;
        }
    }
    return true;
}

```

Go

Go


```

func areSentencesSimilar(sentence1 [string], sentence2 [string], similarPairs [][]string) bool {
    if len(sentence1) != len(sentence2) {
        return false
    }
    n := len(string(sentence1))
    for _ , p := range similarPairs {
        s[p[0]]+"-"+p[1] == true
    }
    for i, x := range sentence1 {
        y := sentence2[i]
        if x != y && !s[x+"-"+y] && !s[y+"-"+x] {
            return false
        }
    }
    return true
}

```

Go

```

func areSentencesSimilar(sentence1 [string], sentence2 [string], similarPairs [][]string) bool {
    if len(sentence1) != len(sentence2) {
        return false
    }
    n := len(string(sentence1))
    for _ , p := range similarPairs {
        s[p[0]]+"-"+p[1] == true
    }
    for i, a := range sentence1 {
        b := sentence2[i]
        if a != b && !s[a+"-"+b] && !s[b+"-"+a] {
            return false
        }
    }
    return true
}

```

[*] String — Pattern Solution (stored optimal)

Python

String · LeetCodeMastery — By Pattern

— 169 —

LeetCodeMastery

```

# Function to determine if two sentences are similar
def are_sentences_similar(sentence1, sentence2, similarPairs):
    # Check if sentence1 and sentence2 are the same length
    if len(sentence1) != len(sentence2):
        return False

    # Build a dictionary to store similar word pairs in both directions
    similar_dict = {}
    for word1, word2 in similarPairs:
        if word1 not in similar_dict:
            similar_dict[word1] = set()
        if word2 not in similar_dict:
            similar_dict[word2] = set()
        similar_dict[word1].add(word2)
        similar_dict[word2].add(word1)

    # Check each word of sentence1 against sentence2
    for w1, w2 in zip(sentence1, sentence2):
        # If words are the same, continue
        if w1 == w2:
            continue
        # Check if w1 is similar to w2 explicitly
        if (w1 not in similar_dict or w2 not in similar_dict[w1]) and (w2 not in similar_dict or w1 not in similar_dict[w2]):
            return False

    return True

# Example usage:
# sentence1 = "great", "having", "skill", "in"
# sentence2 = "fine", "drama", "talent"
# similarPairs = [{"great": "fine"}, {"great": "drama"}, {"having": "talent"}, {"skill": "talent"}]
# print(are_sentences_similar(sentence1, sentence2, similarPairs)) # Output: True

```

C++

String · LeetCodeMastery — By Pattern

— 170 —

LeetCodeMastery

```

// Function to determine if two sentences are similar
#include <vector>
#include <string>
#include <unordered_map>
#include <unordered_set>
using namespace std;

bool areSentencesSimilar(vector<string>& sentence1, vector<string>& sentence2, vector<vector<string>&>& similarPairs) {
    // Check if sentence1 and sentence2 have the same length
    if (sentence1.size() != sentence2.size()) {
        return false;
    }

    // Create an unordered map to store similar pairs bidirectionally
    unordered_map<string, unordered_set<string>> similarDict;
    for (const auto& pair : similarPairs) {
        string word1 = pair[0], word2 = pair[1];
        similarDict[word1].insert(word2);
        similarDict[word2].insert(word1);
    }

    // Check each corresponding pair of words
    for (int i = 0; i < sentence1.size(); i++) {
        const string word1 = sentence1[i];
        const string word2 = sentence2[i];

        // If words are identical, continue to next pair
        if (word1 == word2) continue;
        // Check if word1 is similar to word2 explicitly
        if ((similarDict.find(word1) == similarDict.end() || similarDict[word1].find(word2) == similarDict[word1].end()) &&
            (similarDict.find(word2) == similarDict.end() || similarDict[word2].find(word1) == similarDict[word2].end())) {
            return false;
        }
    }
    return true;
}

// Example usage:
// int main() {
//     vector<string> sentence1 = {"great", "having", "skill", "in"};
//     vector<string> sentence2 = {"fine", "drama", "talent"};
//     vector<vector<string>> similarPairs = {{{"great", "fine"}, {"great", "drama"}, {"having", "talent"}, {"skill", "talent"}}};
//     bool result = areSentencesSimilar(sentence1, sentence2, similarPairs); // result should be true
//     return 0;
// }

```

#1165

EASY

Single Row Keyboard

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Hash Table · String

Lists Premium 98

String · LeetCodeMastery — By Pattern

— 171 —

LeetCodeMastery

Problem:

There is a special keyboard with all keys in a single row.

Given a string keyboard of length 26 indicating the layout of the keyboard (indexed from 0 to 25). Initially, your finger is at index 0. To type a character, you have to move your finger to the index of the desired character. The time taken to move your finger from index i to index j is $|i - j|$.

You want to type a string word. Write a function to calculate how much time it takes to type it with one finger.

Example 1:

Input: keyboard = "abcdefghijklmnopqrstuvwxyz", word = "cba"

Output: 4

Explanation: The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0 again to write 'a'.

Total time = 2 + 1 + 1 = 4.

Example 2:

Input: keyboard = "pqrstuvwxyzabcdefghijklmnopqrstuvwxyz", word = "leetcode"

Output: 73

Constraints:

- keyboard.length == 26
- keyboard contains each English lowercase letter exactly once in some order.
- 1 <= word.length <= 104
- word[i] is an English lowercase letter.

Python

Python

```

# Build a dictionary to map each character to its index in the keyboard string
def calculateTime(keyboard: str, word: str) -> int:
    # Create a mapping for character to index
    char_index = {char: idx for idx, char in enumerate(keyboard)}

    # Initialize the total time and starting position
    total_time = 0
    current_position = 0

    # Iterate over each character in the word
    for char in word:
        # Find the target index for the character
        target_position = char_index[char]

        # Calculate the time to move to the target position
        total_time += abs(target_position - current_position)

        # Update current position to the target position
        current_position = target_position

    return total_time

# Example usage:
# print(calculateTime("abcdefghijklmnopqrstuvwxyz", "cba"))

```

Python

String · LeetCodeMastery — By Pattern

— 172 —

LeetCodeMastery

```

class Solution {
public:
    def calculateTime(self, keyboard: str, word: str) -> int:
        letterIndex = {}
        for i, c in enumerate(keyboard):
            letterIndex[c] = i
        ans = 0
        for a, b in zip(word, word[1:]):
            ans += abs(letterIndex[a] - letterIndex[b])
        return ans
}

```

Python

```

class Solution {
public:
    def calculateTime(self, keyboard: str, word: str) -> int:
        pos = {}
        for i, c in enumerate(keyboard):
            pos[c] = i
        ans = 0
        for c in word:
            ans += abs(pos[c] - i)
            i = pos[c]
        return ans
}

```

Python

```

class Solution {
public:
    def calculateTime(self, keyboard: str, word: str) -> int:
        pos = {}
        for i, c in enumerate(keyboard):
            pos[c] = i
        ans = 0
        for c in word:
            ans += abs(pos[c] - i)
            i = pos[c]
        return ans
}

```

C++

C++

```

class Solution {
public:
    int calculateTime(string keyboard, string word) {
        int ans = 0;
        int previous = 0;
        vector<int> letterIndex(26);

        for (int i = 0; i < keyboard.length(); ++i)
            letterIndex[keyboard[i] - 'a'] = i;

        for (const char c : word) {
            const int currentIndex = letterIndex[c - 'a'];
            ans += abs(currentIndex - previous);
            previous = currentIndex;
        }

        return ans;
    }
};

```

Java

Java

String · LeetCodeMastery — By Pattern

— 173 —

LeetCodeMastery

```

class Solution {
public int calculateTime(String keyboard, String word) {
    int ans = 0;
    int previousIndex = 0;
    int[] letterIndex = new int[26];

    for (int i = 0; i < keyboard.length(); ++i)
        letterIndex[keyboard.charAt(i) - 'a'] = i;

    for (final char c : word.toCharArray()) {
        final int currentIndex = letterIndex[c - 'a'];
        ans += Math.abs(currentIndex - previousIndex);
        previousIndex = currentIndex;
    }

    return ans;
}
}

```

Java

```

class Solution {
public int calculateTime(String keyboard, String word) {
    int[] pos = new int[26];
    for (int i = 0; i < 26; ++i) {
        pos[keyboard.charAt(i) - 'a'] = i;
    }

    int ans = 0, i = 0;
    for (int k = 0; k < word.length(); ++k) {
        int j = pos[word.charAt(k) - 'a'];
        ans += Math.abs(i - j);
        i = j;
    }

    return ans;
}
}

```

Java

```

class Solution {
public int calculateTime(String keyboard, String word) {
    int[] pos = new int[26];
    for (int i = 0; i < 26; ++i) {
        pos[keyboard.charAt(i) - 'a'] = i;
    }

    int ans = 0, i = 0;
    for (int k = 0; k < word.length(); ++k) {
        int j = pos[word.charAt(k) - 'a'];
        ans += Math.abs(i - j);
        i = j;
    }

    return ans;
}
}

```

TypeScript

TypeScript

String · LeetCodeMastery — By Pattern

— 174 —

LeetCodeMastery

```
function calculateTime(keyboard: string, word: string): number {
    const pos: number[] = Array(26).fill(0);
    for (let i = 0; i < 26; ++i) {
        pos[keyboard.charCodeAt(i) - 97] = i;
    }
    let ans = 0;
    let i = 0;
    for (const c of word) {
        const j = pos[c.charCodeAt(0) - 97];
        ans += Math.abs(i - j);
        i = j;
    }
    return ans;
}
```

TypeScript

```
function calculateTime(keyboard: string, word: string): number {
    const pos: number[] = Array(26).fill(0);
    for (let i = 0; i < 26; ++i) {
        pos[keyboard.charCodeAt(i) - 97] = i;
    }
    let ans = 0;
    let i = 0;
    for (const c of word) {
        const j = pos[c.charCodeAt(0) - 97];
        ans += Math.abs(i - j);
        i = j;
    }
    return ans;
}
```

[I] String --- Pattern Solution (stored optimal)

Python

```
# Build a dictionary to map each character to its index in the keyboard string
def calculateTime(keyboard: str, word: str) -> int:
    # Create a mapping from character to index
    char_index = {char: idx for idx, char in enumerate(keyboard)}

    # Initialize the total time and starting position
    total_time = 0
    current_position = 0

    # Iterate over each character in the word
    for char in word:
        # Find the target index for the character
        target_position = char_index[char]

        # Calculate the time to move to the target position
        total_time += abs(target_position - current_position)

        # Update current position to the target position
        current_position = target_position

    return total_time

# Example usage:
# print(calculateTime("abcdefghijklmnopqrstuvwxyz", "cba"))
```

C++

String · LeetCode - By Pattern

— 175 —

LeetCode

```
// C++ solution to calculate the total time
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
using namespace std;

int calculateTime(string keyboard, string word) {
    // Create a vector to map each character to its index (size 26)
    vector<int> charIndex(26);
    for (int i = 0; i < keyboard.size(); ++i) {
        // Map the character to its index
        charIndex[keyboard[i] - 'a'] = i;
    }

    int totalTime = 0;
    int currentPosition = 0;

    // Iterate over each character in word
    for (char ch : word) {
        // Calculate the target index for the character ch
        int targetPosition = charIndex[ch - 'a'];
        // Add the absolute difference to total time
        totalTime += abs(targetPosition - currentPosition);
        // Update the current index position
        currentPosition = targetPosition;
    }

    return totalTime;
}
```

```
// Example usage:
// int main() {
//     cout << calculateTime("abcdefghijklmnopqrstuvwxyz", "cba") << endl;
//     return 0;
// }
```

#8

MEDIUM

String to Integer (atoi)

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

String

Like · Grind 169

Problem:

Implement the myAtoi(string s) function, which converts a string to a 32-bit signed integer.

The algorithm for myAtoi(string s) is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.

String · LeetCode - By Pattern

— 176 —

LeetCode

4. **Rounding:** If the integer is out of the 32-bit signed integer range [-2³¹, 2³¹ - 1], then round the integer to remain in the range. Specifically, integers less than -2³¹ should be rounded to -2³¹, and integers greater than 2³¹ - 1 should be rounded to 2³¹ - 1.

Return the integer as the final result.

Example 1:

Input: s = "42"

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

Step 2: "42" (no characters read because there is neither a '-' nor '+')

Step 3: "42" ("42" is read in)

Example 2:

Input: s = "-042"

Output: -42

Explanation:

Step 1: "-042" (leading whitespace is read and ignored)

Step 2: "-042" ('-' is read, so the result should be negative)

Step 3: "-042" ("042" is read in, leading zeros ignored in the result)

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)

Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')

Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)

Step 2: "0-1" (no characters read because there is no leading whitespace)

Step 3: "0-1" (no characters read because there is no leading whitespace)

Step 4: "0-1" (no characters read because there is no leading whitespace)

Step 5: "0-1" (no characters read because there is no leading whitespace)

Step 6: "0-1" (no characters read because there is no leading whitespace)

Step 7: "0-1" (no characters read because there is no leading whitespace)

Step 8: "0-1" (no characters read because there is no leading whitespace)

Step 9: "0-1" (no characters read because there is no leading whitespace)

Step 10: "0-1" (no characters read because there is no leading whitespace)

Step 11: "0-1" (no characters read because there is no leading whitespace)

Step 12: "0-1" (no characters read because there is no leading whitespace)

Step 13: "0-1" (no characters read because there is no leading whitespace)

Step 14: "0-1" (no characters read because there is no leading whitespace)

Step 15: "0-1" (no characters read because there is no leading whitespace)

Step 16: "0-1" (no characters read because there is no leading whitespace)

Step 17: "0-1" (no characters read because there is no leading whitespace)

Step 18: "0-1" (no characters read because there is no leading whitespace)

Step 19: "0-1" (no characters read because there is no leading whitespace)

Step 20: "0-1" (no characters read because there is no leading whitespace)

Step 21: "0-1" (no characters read because there is no leading whitespace)

Step 22: "0-1" (no characters read because there is no leading whitespace)

Step 23: "0-1" (no characters read because there is no leading whitespace)

Step 24: "0-1" (no characters read because there is no leading whitespace)

Step 25: "0-1" (no characters read because there is no leading whitespace)

Step 26: "0-1" (no characters read because there is no leading whitespace)

Step 27: "0-1" (no characters read because there is no leading whitespace)

Step 28: "0-1" (no characters read because there is no leading whitespace)

Step 29: "0-1" (no characters read because there is no leading whitespace)

Step 30: "0-1" (no characters read because there is no leading whitespace)

Step 31: "0-1" (no characters read because there is no leading whitespace)

Step 32: "0-1" (no characters read because there is no leading whitespace)

Step 33: "0-1" (no characters read because there is no leading whitespace)

Step 34: "0-1" (no characters read because there is no leading whitespace)

Step 35: "0-1" (no characters read because there is no leading whitespace)

Step 36: "0-1" (no characters read because there is no leading whitespace)

Step 37: "0-1" (no characters read because there is no leading whitespace)

Step 38: "0-1" (no characters read because there is no leading whitespace)

Step 39: "0-1" (no characters read because there is no leading whitespace)

Step 40: "0-1" (no characters read because there is no leading whitespace)

Step 41: "0-1" (no characters read because there is no leading whitespace)

Step 42: "0-1" (no characters read because there is no leading whitespace)

Step 43: "0-1" (no characters read because there is no leading whitespace)

Step 44: "0-1" (no characters read because there is no leading whitespace)

Step 45: "0-1" (no characters read because there is no leading whitespace)

Step 46: "0-1" (no characters read because there is no leading whitespace)

Step 47: "0-1" (no characters read because there is no leading whitespace)

Step 48: "0-1" (no characters read because there is no leading whitespace)

Step 49: "0-1" (no characters read because there is no leading whitespace)

Step 50: "0-1" (no characters read because there is no leading whitespace)

Step 51: "0-1" (no characters read because there is no leading whitespace)

Step 52: "0-1" (no characters read because there is no leading whitespace)

Step 53: "0-1" (no characters read because there is no leading whitespace)

Step 54: "0-1" (no characters read because there is no leading whitespace)

Step 55: "0-1" (no characters read because there is no leading whitespace)

Step 56: "0-1" (no characters read because there is no leading whitespace)

Step 57: "0-1" (no characters read because there is no leading whitespace)

Step 58: "0-1" (no characters read because there is no leading whitespace)

Step 59: "0-1" (no characters read because there is no leading whitespace)

Step 60: "0-1" (no characters read because there is no leading whitespace)

Step 2: "0-1" (no characters read because there is neither a '-' nor '+')

Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Constraints:

- 0 <= s.length <= 200
- s consists of English letters (lower-case and upper-case), digits (0-9), '-', '+', '.', and ' '.

Python

```
# Python solution for the myAtoi problem.
def myAtoi(s: str) -> int:
    INT_MIN = -2**31
    INT_MAX = 2**31 - 1

    i = 0
    while i < len(s) and s[i] == ' ':
        i += 1

    # Handle sign
    sign = 1
    if i < len(s) and (s[i] == '-' or s[i] == '+'):
        if s[i] == '-':
            sign = -1
        i += 1

    # Extract digits and build the integer
    while i < len(s) and s[i].isdigit():
        digit = int(s[i])

        # Check for overflow or underflow
        if result >= INT_MAX - digit // 10:
            return INT_MAX if sign == 1 else INT_MIN
        result = result * 10 + digit

        i += 1

    return sign * result

# Example usage:
print(myAtoi("42"))
print(myAtoi("-42"))
print(myAtoi("1337c0d3"))
print(myAtoi("0-1"))
print(myAtoi("words and 987"))
```

String · LeetCode - By Pattern

— 178 —

LeetCode

Python

```
class Solution:
    def myAtoi(self, s: str) -> int:
        s = s.strip()
        if not s:
            return 0

        sign = -1 if s[0] == '-' else 1
        if s[0] in ('-', '+'):
            s = s[1:]

        num = 0

        for c in s:
            if not c.isdigit():
                break
            num = num * 10 + int(c)
            if sign * num >= 2**31 - 1:
                return 2**31 - 1
            if sign * num <= -2**31:
                return -2**31

        return sign * num
```

Python

```
class Solution:
    def myAtoi(self, s: str) -> int:
        if not s:
            return 0
        s = s.strip()
        if not s:
            return 0
        i = 0
        while s[i] == ' ':
            i += 1
        if i == len(s):
            return 0
        sign = -1 if s[i] == '-' else 1
        if s[i] in ('-', '+'):
            i += 1
        res, flag = 0, (2**31 - 1) // 10
        while i < len(s) and s[i].isdigit():
            break
        c = int(s[i])
        if res > flag or (res == flag and c > 7):
            return 2**31 - 1 if sign == 1 else -2**31
        res = res * 10 + c
        i += 1
        return sign * res
```

Python

String · LeetCode - By Pattern

— 179 —

LeetCode

class Solution:

```
def myAtoi(self, s: str) -> int:
    if not s:
        return 0
    s = s.strip()
    if not s:
        return 0
    i = 0
    while s[i] == ' ':
        i += 1
    if i == len(s):
        return 0
    sign = -1 if s[i] == '-' else 1
    if s[i] in ('-', '+'):
        i += 1
    res, flag = 0, (2**31 - 1) // 10
    while i < len(s) and s[i].isdigit():
        break
    c = int(s[i])
    if res > flag or (res == flag and c > 7):
        return 2**31 - 1 if sign == 1 else -2**31
    res = res * 10 + c
    i += 1
    return sign * res
```

C++

C++

class Solution {

```
//
// @param string s
// @return int
//
function myAtoi(s) {
    s = s.replace(/\s/g, '');
    if (s[0] == '-' || s[0] == '+') {
        return s[0] == '-' ? -1 : 1;
    }
    return parseInt(s);
}
```

C++

String · LeetCode - By Pattern

— 180 —

LeetCode


```

Python
Python
# Python solution for grouping shifted strings

def group_shifted_strings(strings):
    # Helper function to generate a unique key for each string based on differences
    def get_key(s):
        # Single character strings return a fixed key
        if len(s) == 1:
            return "a"
        # Compute differences modulo 26 for each adjacent characters
        key = []
        for i in range(1, len(s)):
            diff = (ord(s[i]) - ord(s[i-1]) + 26) % 26
            key.append(str(diff))
        # Use tuple of differences as join to form a string key
        return " ".join(key)

    groups = {}
    # Group strings by their computed key
    for s in strings:
        key = get_key(s)
        if key not in groups:
            groups[key] = []
        groups[key].append(s)
    # Return grouped strings as a list of lists
    return list(groups.values())

# Example usage
print(group_shifted_strings["abc","bcd","bcd","xyz","def","def","g","g","h"])

```

```

Python
class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        keyOfStrings = collections.defaultdict(list)

        def getKey(s: str) -> str:
            # Returns the key of 's' by pairwise calculation of differences.
            # e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
            diffs = []
            for i in range(1, len(s)):
                diff = (ord(s[i]) - ord(s[i - 1]) + 26) % 26
                diffs.append(str(diff))
            return " ".join(diffs)

        for s in strings:
            keyOfStrings[getKey(s)].append(s)
        return keyOfStrings.values()

```

Python

String · LeetCode - By Pattern

— 187 —

LeetCode

```

class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        g = defaultdict(list)
        for s in strings:
            diff = ord(s[0]) - ord("a")
            t = []
            for c in s:
                c = ord(c) - diff
                if c < ord("a"):
                    c += 26
                t.append(chr(c))
            g[" ".join(t)].append(s)
        return list(g.values())

```

Python

```

s = "a"
a = defaultdict(list)
# Check if 'a' is in the list
if "a" in a:
    # If it is, increment the count
    a["a"] += 1
else:
    # If it is not, add it to the list
    a["a"] = 1

```

```

s = "a"
a = defaultdict(list)
# Check if 'a' is in the list
if "a" in a:
    # If it is, increment the count
    a["a"] += 1
else:
    # If it is not, add it to the list
    a["a"] = 1

```

String · LeetCode - By Pattern

— 188 —

LeetCode

```

s = list(s.values())
[1, 2, 3]

from collections import defaultdict

class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        mp = defaultdict(list)
        for s in strings:
            t = []
            diff = ord(s[0]) - ord('a')
            for c in s:
                d = ord(c) - diff
                if d < ord('a'):
                    d += 26
                t.append(chr(d))
            k = " ".join(t)
            mp[k].append(s)
        return list(mp.values())

```

C++

```

C++
class Solution {
public:
    vector<vector<string>> groupStrings(vector<string>& strings) {
        vector<vector<string>> ans;
        unordered_map<string, vector<string>> keyOfStrings;

        for (const string& s : strings)
            keyOfStrings[getKey(s)].push_back(s);

        for (const auto& [_, strings] : keyOfStrings)
            ans.push_back(strings);

        return ans;
    }

private:
    // Returns the key of 's' by pairwise calculation of differences.
    // e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
    string getKey(const string& s) {
        string key;
        for (int i = 1; i < s.length(); ++i) {
            const int diff = (s[i] - s[i - 1] + 26) % 26;
            key += to_string(diff) + ",";
        }
        return key;
    }
};

```

String · LeetCode - By Pattern

— 189 —

LeetCode

```

class Solution {
public:
    vector<vector<string>> groupStrings(vector<string>& strings) {
        unordered_map<string, vector<string>> g;
        for (const s : strings) {
            string t;
            int diff = s[0] - 'a';
            for (int i = 0; i < s.size(); ++i) {
                char c = s[i] - diff;
                if (c < 'a') c += 26;
                t.push_back(c);
            }
            g[t].push_back(s);
        }
        vector<vector<string>> ans;
        for (const p : g) {
            ans.push_back(move(p.second));
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<vector<string>> groupStrings(vector<string>& strings) {
        unordered_map<string, vector<string>> mp;
        for (const s : strings) {
            int diff = s[0] - 'a';
            string t = s;
            for (int i = 0; i < t.size(); ++i) {
                char d = t[i] - diff;
                if (d < 'a') d += 26;
                t[i] = d;
            }
            int t = t.size();
            mp[t].push_back(s);
        }
        vector<vector<string>> ans;
        for (const m : mp) {
            ans.push_back(move(m.second));
        }
        return ans;
    }
};

```

Java

Java

String · LeetCode - By Pattern

— 190 —

LeetCode

```

class Solution {
    public List<List<String>> groupStrings(String[] strings) {
        Map<String, List<String>> keyOfStrings = new HashMap<>();

        for (final String s : strings)
            keyOfStrings.computeIfAbsent(getKey(s), k -> new ArrayList<>()).add(s);
        return new ArrayList<>(keyOfStrings.values());
    }

    // Returns the key of 's' by pairwise calculation of differences.
    // e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
    private String getKey(final String s) {
        StringBuilder sb = new StringBuilder();
        for (int i = 1; i < s.length(); ++i) {
            final int diff = (s.charAt(i) - s.charAt(i - 1) + 26) % 26;
            sb.append(diff).append(",");
        }
        return sb.toString();
    }
}

```

Java

```

class Solution {
    public List<List<String>> groupStrings(String[] strings) {
        Map<String, List<String>> g = new HashMap<>();
        for (String s : strings) {
            char[] t = s.toCharArray();
            int diff = t[0] - 'a';
            for (int i = 0; i < t.length; ++i) {
                t[i] = (char) (t[i] - diff);
                if (t[i] < 'a') {
                    t[i] += 26;
                }
            }
            g.computeIfAbsent(new String(t), k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(g.values());
    }
}

```

Java

String · LeetCode - By Pattern

— 191 —

LeetCode

```

class Solution {
    public List<List<String>> groupStrings(String[] strings) {
        Map<String, List<String>> mp = new HashMap<>();
        for (String s : strings) {
            int diff = s.charAt(0) - 'a';
            char[] t = s.toCharArray();
            for (int i = 0; i < t.length; ++i) {
                char d = (char) (t[i] - diff);
                if (d < 'a') {
                    d += 26;
                }
                t[i] = d;
            }
            String key = new String(t);
            mp.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(mp.values());
    }
}

```

Go

Go

```

func groupStrings(strings []string) [][]string {
    g := make(map[string][]string)
    for _, s := range strings {
        t := []byte(s)
        diff := t[0] - 'a'
        for i := range t {
            t[i] -= diff
            if t[i] < 'a' {
                t[i] += 26
            }
        }
        g[string(t)] = append(g[string(t)], s)
    }
    ans := make([][]string, 0, len(g))
    for _, v := range g {
        ans = append(ans, v)
    }
    return ans
}

```

Go

String · LeetCode - By Pattern

— 192 —

LeetCode

```
func groupStrings(strings: List[String]) => List[String] {
    mp := makeMap[String, List[String]]
    for _ <- range strings {
        k := ""
        for i <- range s {
            k += string((s(i)-a(0)-26)%26 + 'a')
        }
        mp[k] = append(mp[k], s)
    }
    var ans: List[String]
    for _ <- range mp {
        ans = append(ans, v)
    }
    return ans
}
```

[*] String — Pattern Solution (matched from community answers)

Python

```
class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        s = defaultdict(list)
        for s in strings:
            diff = ord(s[0]) - ord("a")
            t = []
            for c in s:
                c = ord(c) - diff
                if c < ord("a"):
                    c += 26
                t.append(chr(c))
            s[diff%26].append(s)
        return list(s.values())
```

#271 MEDIUM

Encode and Decode Strings

LeetCode · Implement · Java · C · JavaScript · Editorial

Array · String · Design

List Grind 169 | Premium 98

Problem:

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {
    //... your code
    return encoded_string;
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {
    //... your code
    return strs;
}
```

String · LeetCode — By Pattern

— 193 —

LeetCode

```
class Codec:
    # Encodes a list of strings to a single string.
    def encode(self, strs):
        encoded = []
        # Iterate through each string in the list.
        for s in strs:
            # Append the length of the string 's' as delimiter, and the string.
            encoded.append(str(len(s)) + '#' + s)
        # Join all parts into one encoded string.
        return ''.join(encoded)

    # Decodes a single string back to a list of strings.
    def decode(self, s):
        decoded = []
        i = 0 # pointer to traverse the encoded string.
        while i < len(s):
            j = i
            # Find the '#' delimiter to determine the length.
            while s[j] != '#':
                j += 1
            # Convert the substring from i to j into an integer length.
            length = int(s[i:j])
            # Extract the actual string using the obtained length.
            string = s[j+1:j+length+1]
            decoded.append(string)

            # Move the pointer to the start of the next encoded segment.
            i = j + 1 + length
        return decoded

# Example usage:
# codec = Codec()
# encoded_str = codec.encode(["Hello", "World"])
# decoded_list = codec.decode(encoded_str)
```

Python

```
class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        return ''.join(str(len(s)) + '#' + s for s in strs)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""
        decoded = []
        i = 0
        while i < len(s):
            slash = s.find('/', i)
            length = int(s[i:slash])
            i = slash + length + 1
            decoded.append(s[slash + 1:i])
        return decoded
```

Python

String · LeetCode — By Pattern

— 195 —

LeetCode

```
class Codec:
    # Encodes a list of strings to a single string.
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
        for s in strs:
            ans.append(str(len(s)) + '#' + s)
        return ''.join(ans)

    # Decodes a single string to a list of strings.
    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""
        ans = []
        i = 0
        while i < len(s):
            size = int(s[i:i+4])
            # Note: exclusion for right index
            i = size
            ans.append(s[i:i+size])
        return ans

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))

class Codec:
    def encode(self, strs: List[str]) -> str:
        encoded = ""
        for s in strs:
            # Convert for input: "abcd", "mnop"
            encoded += str(len(s)) + "a" + s.replace("a", "aa") + "a"
        return encoded

    def decode(self, s: str) -> List[str]:
        decoded = []
        i = 0
        while i < len(s):
            j = s.find("a", i)
            length = int(s[i:j])
            decoded.append(s[i+1:j+length].replace("aa", "a"))
            i = j + 1 + length + 1
        return decoded
```

class Codec: # using concept with above, for input: ["abcd", "mnop"]

```
def encode(self, strs):
    encoded = ""
    for s in strs:
        encoded += str(len(s)) + "a" + s
```

C++

String · LeetCode — By Pattern

— 197 —

LeetCode

```
}

So Machine 1 does:
string encoded_str = encode(strs);
and Machine 2 does:
vector<string> str2 = decode(encoded_str);
str2 in Machine 2 should be the same as str in Machine 1.
Implement the encode and decode methods.
You are not allowed to solve the problem using any serialize methods (such as eval).
```

Example 1:

```
Input: dummy_input = ["Hello","World"]
Output: ["Hello","World"]
```

Explanation:

Machine 1:

```
Codec codec = new Codec();
String msg = codec.encode(strs);
Machine 1 --> msg --> Machine 2
```

Machine 2:

```
Codec decoder = new Codec();
String[] strs = decoder.decode(msg);
```

Example 2:

```
Input: dummy_input = [""]
Output: [""]
```

Constraints:

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] contains only valid ASCII characters out of 256 valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

Python

Python

String · LeetCode — By Pattern

— 194 —

LeetCode

```
class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
        for s in strs:
            ans.append(str(len(s)) + '#' + s)
        return ''.join(ans)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""
        ans = []
        i = 0
        while i < len(s):
            size = int(s[i:i+4])
            i = i + size
            ans.append(s[i:i+size])
        return ans

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))
```

Python

```
class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
        for s in strs:
            ans.append(str(len(s)) + '#' + s)
        return ''.join(ans)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""
        ans = []
        i = 0
        while i < len(s):
            slash = s.find('/', i)
            length = int(s[i:slash])
            i = slash + length + 1
            decoded.append(s[slash + 1:i])
        return decoded
```

String · LeetCode — By Pattern

— 196 —

LeetCode

```
class Codec {
    public:
        // Encodes a list of strings to a single string.
        string encode(vector<string>& strs) {
            string ans;
            for (string s : strs) {
                int size = s.size();
                ans = string(const char(&size, sizeof(size)));
                ans += s;
            }
            return ans;
        }

        // Decodes a single string to a list of strings.
        vector<string> decode(string s) {
            vector<string> ans;
            for (int i = 0; i < s.length(); i++) {
                const int slash = s.find('/', i);
                const int length = atoi(substr(s, slash - 4));
                i = slash + length + 1;
                decoded.push_back(s.substr(slash + 1, i - slash - 1));
            }
            return decoded;
        }
};
```

C++

```
class Codec {
    public:
        // Encodes a list of strings to a single string.
        string encode(vector<string>& strs) {
            string ans;
            for (string s : strs) {
                int size = s.size();
                ans = string(const char(&size, sizeof(size)));
                ans += s;
            }
            return ans;
        }

        // Decodes a single string to a list of strings.
        vector<string> decode(string s) {
            vector<string> ans;
            int i = 0;
            while (i < s.size()) {
                int size = atoi(s.substr(i, 4));
                ans.push_back(s.substr(i + 4, size));
                i = i + size + 4;
            }
            return ans;
        }
};
```

String · LeetCode — By Pattern

— 198 —

LeetCode

```

    }
}

// Your Codes object will be instantiated and called as such:
// Codes codes;
// codes.decode(codes.encode(str1));

C++

string encode(vector<string> &strs) {
    //... your code
    return encoded_string;
}

vector<string> decode(string s) {
    //... your code
    return strs;
}

class Codes {
public:
    // Encodes a list of strings to a single string.
    string encode(vector<string> &strs) {
        string ans;
        for (string s : strs) {
            int size = s.size();
            ans += string((const char*) &size, sizeof(size));
            ans += s;
        }
        return ans;
    }

    // Decodes a single string to a list of strings.
    vector<string> decode(string s) {
        vector<string> ans;
        int i = 0, n = s.size();
        int size = 0;
        while (i < n) {
            memcpy(&size, &s.data() + i, sizeof(size));
            i += sizeof(size);
            ans.push_back(s.substr(i, size));
            i += size;
        }
        return ans;
    }
}

// Your Codes object will be instantiated and called as such:
// Codes codes;
// codes.decode(codes.encode(str1));

```

Java

String · LeetCode - By Pattern

— 199 —

LeetCode

```

public class Codes {
    // Encodes a list of strings to a single string.
    public String encode(List<String> strs) {
        StringBuilder encoded = new StringBuilder();

        for (final String s : strs)
            encoded.append(s.length()).append('/').append(s);

        return encoded.toString();
    }

    // Decodes a single string to a list of strings.
    public List<String> decode(String s) {
        List<String> decoded = new ArrayList<>();

        for (int i = 0; i < s.length(); i++) {
            final int slash = s.indexOf('/', i);
            final int length = Integer.parseInt(s.substring(i, slash));
            i = slash + length + 1;
            decoded.add(s.substring(slash + 1, i));
        }

        return decoded;
    }
}

Java

public class Codes {
    // Encodes a list of strings to a single string.
    public String encode(List<String> strs) {
        StringBuilder ans = new StringBuilder();
        for (String s : strs) {
            ans.append((char) s.length()).append(s);
        }
        return ans.toString();
    }

    // Decodes a single string to a list of strings.
    public List<String> decode(String s) {
        List<String> ans = new ArrayList<>();
        for (String s : s) {
            while (i < n) {
                int size = s.charAt(i);
                ans.add(s.substring(i, i + size));
                i += size;
            }
        }
        return ans;
    }
}

// Your Codes object will be instantiated and called as such:
// Codes codes = new Codes();
// codes.decode(codes.encode(str1));

```

Java

String · LeetCode - By Pattern

— 200 —

LeetCode

```

public class Codes {
    // Encodes a list of strings to a single string.
    public String encode(List<String> strs) {
        StringBuilder ans = new StringBuilder();
        for (String s : strs) {
            ans.append((char) s.length()).append(s);
        }
        return ans.toString();
    }

    // Decodes a single string to a list of strings.
    public List<String> decode(String s) {
        List<String> ans = new ArrayList<>();
        int i = 0, n = s.length();
        while (i < n) {
            int size = s.charAt(i);
            ans.add(s.substring(i, i + size));
            i += size;
        }
        return ans;
    }
}

// Your Codes object will be instantiated and called as such:
// Codes codes = new Codes();
// codes.decode(codes.encode(str1));

```

[*] String — Pattern Solution (stored optimal)

Python

```

class Codes:
    # Encodes a list of strings to a single string.
    def encode(self, strs):
        encoded = []
        # Traverse through each string in the list.
        for s in strs:
            # Form the length of the string, a '#' as a delimiter, and the string.
            encoded.append(str(len(s)) + '#' + s)
        # Join all encoded strings into encoded string.
        return ''.join(encoded)

    # Decodes a single string back to a list of strings.
    def decode(self, s):
        decoded = []
        i = 0 # Pointer to traverse the encoded string.
        while i < len(s):
            # Find the '#' delimiter to determine the length.
            while s[i] != '#':
                i += 1
            # Convert the substring from i to j into an integer length.
            length = int(s[i:i+1])
            # Extract the actual string using the obtained length.
            string = s[i+1:i+length+1]
            decoded.append(string)
            i = i + length + 1

```

String · LeetCode - By Pattern

— 201 —

LeetCode

```

    # Form the pointer to the start of the next encoded segment.
    i = j + 1 + length
    return decoded

# Example usage:
# codes = Codes()
# encoded_str = codes.encode(["Hello", "World"])
# decoded_list = codes.decode(encoded_str)

C++

#include <string>
#include <vector>
using namespace std;

class Codes {
public:
    // Encodes a list of strings to a single string.
    string encode(vector<string> &strs) {
        string encoded;
        // Concatenate each string with its length and a delimiter.
        for (const auto& s : strs) {
            encoded += to_string(s.size()) + '#' + s;
        }
        return encoded;
    }

    // Decodes a single string to a list of strings.
    vector<string> decode(const string& s) {
        vector<string> decoded;
        size_t i = 0;
        // Traverse the encoded string until all parts are decoded.
        while (i < s.size()) {
            // Locate the delimiter '#' to get the length.
            size_t j = s.find('#', i);
            // Convert the substring representing the length to an integer.
            int length = atoi(s.substr(i, j - i).c_str());
            // Extract the actual string using the length.
            decoded.push_back(s.substr(j + 1, length));
            // Update the pointer to the next encoded segment.
            i = j + 1 + length;
        }
        return decoded;
    }
};

// Example usage:
// Codes codes;
// vector<string> strs = {"Hello", "World"};
// string encodedStr = codes.encode(strs);
// vector<string> decodedStrs = codes.decode(encodedStr);

```

String · LeetCode - By Pattern

— 202 —

LeetCode

Hash Table · String · Design · Ordered Set

Like Danny Zhang

Problem:

You are given several logs, where each log contains a unique ID and timestamp. Timestamp is a string that has the following format: Year:Month:Day:Hour:Minute:Second, for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.

Implement the LogSystem class:

- LogSystem() Initializes the LogSystem object.
- void put(int id, string timestamp) Stores the given log (id, timestamp) in your storage system.
- int[] retrieve(string start, string end, string granularity) Returns the IDs of the logs whose timestamps are within the range from start to end inclusive. start and end all have the same format as timestamp, and granularity means how precise the range should be (ie. to the exact Day, Minute, etc.). For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", and granularity = "Day" means that we need to find the logs within the inclusive range from Jan. 1st 2017 to Jan. 2nd 2017, and the Hour, Minute, and Second for each log entry can be ignored.

Example 1:

Input

```

["LogSystem", "put", "put", "put", "retrieve", "retrieve"]
[[], [1, "2017:01:01:23:59:59"], [2, "2017:01:01:22:59:59"], [3, "2016:01:01:00:00:00"], ["2016:01:01:01:01:01", "2017:01:01:23:00:00", "Year"], ["2016:01:01:01:01:01", "2017:01:01:23:00:00", "Hour"]]

```

Output

```

[null, null, null, null, [3, 2, 1], [2, 1]]

```

Explanation

```

LogSystem logSystem = new LogSystem();
logSystem.put(1, "2017:01:01:23:59:59");
logSystem.put(2, "2017:01:01:22:59:59");
logSystem.put(3, "2016:01:01:00:00:00");

// return [3,2,1], because you need to return all logs between 2016 and 2017.
logSystem.retrieve("2016:01:01:01:01:01", "2017:01:01:23:00:00", "Year");

// return [2,1], because you need to return all logs between Jan. 1, 2016 01:XX:XX and Jan. 1, 2017 23:XX:XX.
// log 3 is not returned because Jan. 1, 2016 00:00:00 comes before the start of the range.
logSystem.retrieve("2016:01:01:01:01:01", "2017:01:01:23:00:00", "Hour");

```

Constraints:

- 1 <= id <= 500
- 2000 <= Year <= 2017
- 1 <= Month <= 12
- 1 <= Day <= 31
- 0 <= Hour <= 23
- 0 <= Minute, Second <= 59
- granularity is one of the values ["Year", "Month", "Day", "Hour", "Minute", "Second"].

String · LeetCode - By Pattern

— 203 —

LeetCode

- At most 500 calls will be made to put and retrieve.

Python

```

class LogSystem:
    def __init__(self):
        # List to store logs as (timestamp, id) tuples
        self.logs = []

    # Return granularity to index for the substring (fixed format: Year:Month:Day:Hour:Minute:Second)
    self.granularity_to_index = {
        "Year": 4,
        "Month": 7,
        "Day": 10,
        "Hour": 13,
        "Minute": 16,
        "Second": 19
    }

    def put(self, id, timestamp):
        # Append the log as a tuple
        self.logs.append((timestamp, id))

    def retrieve(self, start, end, granularity):
        # Determine the slicing range based on granularity
        idx = self.granularity_to_index[granularity]
        # Slice the given start and end timestamps
        start_slice = start[:idx]
        end_slice = end[:idx]
        result = []

        # Traverse over all logs and check if the timestamp slice is within the range
        for ts, log_id in self.logs:
            ts_slice = ts[:idx]
            if start_slice <= ts_slice <= end_slice:
                result.append(log_id)
        return result

# Example usage:
# logSystem = LogSystem()
# logSystem.put(1, "2017:01:01:23:59:59")
# logSystem.put(2, "2017:01:01:22:59:59")
# logSystem.put(3, "2016:01:01:00:00:00")
# print(logSystem.retrieve("2016:01:01:01:01:01", "2017:01:01:23:00:00", "Year")) # Output: [3, 2, 1]
# print(logSystem.retrieve("2016:01:01:01:01:01", "2017:01:01:23:00:00", "Hour")) # Output: [2, 1]

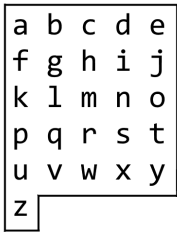
```

Python

String · LeetCode - By Pattern

— 204 —

LeetCode



We may make the following moves:

- 'U' moves our position up one row, if the position exists on the board;
- 'D' moves our position down one row, if the position exists on the board;
- 'L' moves our position left one column, if the position exists on the board;
- 'R' moves our position right one column, if the position exists on the board;
- 'T' adds the character board[r][c] at our current position (r, c) to the answer.

(Here, the only positions that exist on the board are positions with letters on them.)

Return a sequence of moves that makes our answer equal to target in the minimum number of moves. You may return any path that does so.

Example 1:

Input: target = "leet"

Output: "DRIUURRRIIDDDI"

Example 2:

Input: target = "code"

Output: "RRRIIDRRUULRII"

Constraints:

- 1 <= target.length <= 100
- target consists only of English lowercase letters.

Python

Python

String · LeetCode — By Pattern

— 211 —

LeetCode

```
# Python solution with clear variable names and inline comments.
class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        # Starting position at the top-left corner (0,0)
        current_row, current_col = 0, 0
        result = []

        # Helper function to convert letter to its (row, col) coordinates
        def getPosition(letter):
            index = ord(letter) - ord('a')
            return index // 5, index % 5

        for char in target:
            target_row, target_col = getPosition(char)
            # If target character is 'z', move horizontally first then vertically
            if char == 'z':
                # Move left if needed
                while current_col > target_col:
                    result.append('L')
                    current_col -= 1
                # Move right if needed
                while current_col < target_col:
                    result.append('R')
                    current_col += 1
                # Then move down if needed
                while current_row < target_row:
                    result.append('D')
                    current_row += 1
            else:
                # For all other characters, move vertically first
                while current_row < target_row:
                    result.append('D')
                    current_row += 1
                while current_row > target_row:
                    result.append('U')
                    current_row -= 1
                while current_col < target_col:
                    result.append('R')
                    current_col += 1
                while current_col > target_col:
                    result.append('L')
                    current_col -= 1
                # Append path-upon arrival for the letter
                result.append(char)
            return ''.join(result)
```

Python

String · LeetCode — By Pattern

— 212 —

LeetCode

```
class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        i = 0
        ans = []
        for c in target:
            x, y = ord(c) - ord('a')
            while i > y:
                j = 1
                ans.append('U')
                while i > y:
                    i -= 1
                    ans.append('U')
                while j < y:
                    j += 1
                    ans.append('D')
            while i < x:
                i += 1
                ans.append('R')
            while i > x:
                i -= 1
                ans.append('L')
            ans.append(c)
            return ''.join(ans)
```

Python

```
class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        i = 0
        ans = []
        for c in target:
            x, y = ord(c) - ord('a')
            while i > y:
                j = 1
                ans.append('U')
                while i > y:
                    i -= 1
                    ans.append('U')
                while j < y:
                    j += 1
                    ans.append('D')
            while i < x:
                i += 1
                ans.append('R')
            while i > x:
                i -= 1
                ans.append('L')
            ans.append(c)
            return ''.join(ans)
```

Java

Java

String · LeetCode — By Pattern

— 213 —

LeetCode

```
class Solution {
    public String alphabetBoardPath(String target) {
        StringBuilder ans = new StringBuilder();
        int i = 0, j = 0;
        for (int k = 0; k < target.length(); ++k) {
            int v = target.charAt(k) - 'a';
            int x = v / 5, y = v % 5;
            while (j > y) {
                --j;
                ans.append('U');
            }
            while (i < x) {
                ++i;
                ans.append('R');
            }
            while (i > x) {
                --i;
                ans.append('L');
            }
            ans.append(target.charAt(k));
        }
        return ans.toString();
    }
}
```

Java

```
class Solution {
    public String alphabetBoardPath(String target) {
        StringBuilder ans = new StringBuilder();
        int i = 0, j = 0;
        for (int k = 0; k < target.length(); ++k) {
            int v = target.charAt(k) - 'a';
            int x = v / 5, y = v % 5;
            while (j > y) {
                --j;
                ans.append('U');
            }
            while (i < x) {
                ++i;
                ans.append('R');
            }
            while (i > x) {
                --i;
                ans.append('L');
            }
            ans.append(target.charAt(k));
        }
        return ans.toString();
    }
}
```

String · LeetCode — By Pattern

— 214 —

LeetCode

```
return ans.toString();
}
```

[*] String — Pattern Solution (matched from community answers)

Python

```
# Python solution with clear variable names and inline comments.
class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        # Starting position at the top-left corner (0,0)
        current_row, current_col = 0, 0
        result = []

        # Helper function to convert letter to its (row, col) coordinates
        def getPosition(letter):
            index = ord(letter) - ord('a')
            return index // 5, index % 5

        for char in target:
            target_row, target_col = getPosition(char)
            # If target character is 'z', move horizontally first then vertically
            if char == 'z':
                # Move left if needed
                while current_col > target_col:
                    result.append('L')
                    current_col -= 1
                # Move right if needed
                while current_col < target_col:
                    result.append('R')
                    current_col += 1
                # Then move down if needed
                while current_row < target_row:
                    result.append('D')
                    current_row += 1
            else:
                # For all other characters, move vertically first
                while current_row < target_row:
                    result.append('D')
                    current_row += 1
                while current_row > target_row:
                    result.append('U')
                    current_row -= 1
                while current_col < target_col:
                    result.append('R')
                    current_col += 1
                while current_col > target_col:
                    result.append('L')
                    current_col -= 1
                # Append path-upon arrival for the letter
                result.append(char)
            return ''.join(result)
```

String · LeetCode — By Pattern

— 215 —

LeetCode

String · LeetCode — By Pattern

— 216 —

LeetCode

Quick Review — String 8 questions · insights · complexity · solution

#383 Ransom Note [Easy]

Key Insights

- Use a hash table (or dictionary) to store the frequency of each character present in magazine.
- Iterate over every character in ransomNote and check if the corresponding count in the hash table is available and sufficient.
- Decrease the character count for every match; if any character count falls below zero, return false.
- If all characters in ransomNote can be accounted for, return true.

Space & Time

Time Complexity: $O(n + m)$, where n is the length of magazine and m is the length of ransomNote.

Space Complexity: $O(1)$, since the extra space required for the hash table is limited to at most 26 entries for lowercase English letters.

Solution

We construct a frequency map for the magazine string using a hash table. This frequency map records how many times each letter appears in magazine. Then we iterate through the ransomNote string, and for every letter, we check the corresponding frequency in the map. If a letter is not found or its frequency is zero, we immediately return false as it means the letter cannot be used to build the note. Otherwise, we decrement the frequency count by one. If we successfully process all characters in ransomNote, then it can be constructed from magazine, and we return true.

#734 Sentence Similarity [Easy]

Key Insights

- First, check if both sentences have the same length.
- A word is always similar to itself.
- Similarity is defined only by the given pairs; it is not transitive.
- Use a hash table (dictionary) to store bi-directional similar pairs for quick look-ups.

Space & Time

Time Complexity: $O(n + m)$, where n is the number of words in the sentences and m is the number of similar pairs.

Space Complexity: $O(m)$, for storing the similar pairs in a hash table.

Solution

Check if the two sentences have the same length; if not, return false. Build a hash table where each word maps to a set of words it is similar to. For each pair $[x, y]$ in similarPairs, add y to the set for x and x to the set for y . Iterate over the words of both sentences simultaneously. For each corresponding pair, if the words are identical, continue; otherwise, check if one word appears in the similar set of the other. If all pairs meet the similarity condition, return true; otherwise, return false.

#1165 Single Row Keyboard [Easy]

Key Insights

- Create a mapping (hash table/dictionary) from each character to its index on the keyboard for $O(1)$ lookups.
- The cost to type each character is the absolute difference between the current position and the target character's position on the keyboard.
- Start from index 0 and sum up the distances moved for each successive character in the word.

Space & Time

Time Complexity: $O(n)$, where n is the length of the word.

Space Complexity: $O(1)$ (constant space for a mapping of 26 characters).

String · LeetCode · By Pattern

— 217 —

LeetCode

Space & Time

Time Complexity: $O(n)$, where n is the total number of characters across all strings.

Space Complexity: $O(n)$, required for the encoded string and the resulting decoded list.

Solution

We encode by iterating over each string and building a new string that contains the length of the string, a delimiter (e.g., "#"), and then the string itself. This way, during decoding, we can safely determine how many characters to extract by first reading until the delimiter to get the length, and then reading that number of characters. This length-prefix approach avoids conflicts with potential characters in the strings.

#635 Design Log Storage System [Medium]

Key Insights

- Store logs in a simple data structure such as a list or array.
- Convert the timestamps into strings; due to fixed-length and zero padding, lexicographical string comparisons are valid.
- Use a mapping to determine the slice length for each granularity, e.g. "Year" corresponds to the first 4 characters, "Month" corresponds to the first 7 characters, etc.
- During retrieval, compare the relevant substring of each stored timestamp against the start and end query timestamp slices.
- The inclusive nature of the query simplifies checking conditions.

Space & Time

Time Complexity: $O(N)$ per retrieval query, where N is the number of logs stored (since each log is checked). In the worst case, all log entries are examined.

Space Complexity: $O(N)$ where N is the number of logs stored.

Solution

We implement a LogSystem class that supports putting log entries and retrieving them using a specified granularity.

Data Structures: Use a list/array to store log entries as tuples (or objects) containing the timestamp and log ID. A dictionary/map is used to store the mapping from granularity string to the substring index (e.g., "Year" $\Rightarrow$ 4, "Month" $\Rightarrow$ 7, etc.). For the put() method, simply append the new log entry into the list. For the retrieve() method, compute the substring length based on the provided granularity. Then iterate over all stored logs and compare the sliced timestamp against the sliced start and end query values. If it falls within the bounds, include the log ID in the result. Since the timestamps have fixed width with leading zeros, slicing and lexicographical comparisons are both valid and simple.

#1138 Alphabet Board Path [Medium]

Key Insights

- Determine the row and column for each letter (using integer division and modulus).
- Because "z" is in a special row that only contains one column, the move order is critical to avoid invalid positions.
- For target letter "z", perform horizontal moves before vertical moves; for all other letters, do vertical moves before horizontal moves.

- Build the answer step-by-step by moving from your current position to each target letter.

Space & Time

Time Complexity: $O(N)$ where N is the length of target (each move is processed in constant time)

Space Complexity: $O(N)$ for the resulting string of moves

Solution

The approach calculates the destination coordinates for each character. For each letter in the target:

- Convert the letter to its board coordinates (row and col). For example, for letter 'c', row = (ord('c') - ord('A')) // 5 and col = (ord('c') - ord('A')) % 5. Note that "z" (the 26th letter) always maps to (5, 0). - To move from the current position (curRow, curCol) to the target's (targetRow, targetCol), decide on the order of moves: If the target letter is "z", first

String · LeetCode · By Pattern

— 219 —

LeetCode

#5 Sliding Window — 9 questions · #5 of 21

#3 MEDIUM

Longest Substring Without Repeating Characters

LeetCode · SlidingWindow · HashTable · String · Sliding Window

Lists · Grid · 169

Problem:

Given a string s , find the length of the longest substring without duplicate characters.

Example 1:

Input: $s = \text{"abcabc"}$

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: $s = \text{"bbbbb"}$

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: $s = \text{"pwwkew"}$

Output: 3

Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 \cdot 10^4$
- s consists of English letters, digits, symbols and spaces.

>> Brute Force [Sliding Window] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def length_of_longest_substring(self, s):
        # Brute Force O(n^2) - enumerate every starting index; expand until repeat
        res = 0
        for i in range(len(s)):
            seen = set()
            for j in range(i, len(s)):
                if s[j] in seen:
                    break
                seen.add(s[j])
            res = max(res, j - i + 1)
        return res
```

Python

String · LeetCode · By Pattern

— 221 —

LeetCode

Solution

We solve the problem by first building a dictionary that maps each character in the keyboard layout to its corresponding index. We then initialize a variable to track the current finger position (starting at 0). For each character in the word, we look up its index, calculate the absolute difference from the current position, add that distance to the total time, and update the current position. This approach uses a single pass over the word with constant time lookups, ensuring an efficient solution.

#8 String to Integer (atoi) [Medium]

Key Insights

- Skip leading spaces in the string.
- Detect and handle the optional sign indicator ("+" or "-").
- Read and convert digit characters until a non-digit is encountered.
- Handle cases with no digits or leading zeros.
- Clamp the result to fit within the 32-bit signed integer range.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We iterate through the string to first eliminate any leading whitespace. Next, we check for a sign character to determine if the resulting integer should be negative. We then convert each consecutive digit into an integer, all the while updating the result and checking for potential overflow. If an overflow is detected, we immediately return the appropriate maximum or minimum 32-bit signed integer boundary value. This approach ensures that we strictly follow the conversion rules and efficiently handle edge cases.

#249 Group Shifted Strings [Medium]

Key Insights

- All strings in the same group have the same relative differences between adjacent characters.
- Normalize each string's pattern by converting it to a key representation based on the differences between consecutive letters.
- A single character string forms its own group as there are no differences to compute.

Space & Time

Time Complexity: $O(n * m)$ where n is the number of strings and m is the average length of the strings (to compute each string's key).

Space Complexity: $O(n * m)$ to store the keys and grouping in the hash table.

Solution

The solution involves iterating over each string and generating a unique key that represents the relative differences between adjacent characters. This key is calculated by computing the difference (mod 26) between every consecutive pair in the string. Strings that share the same key can be grouped together since they can be shifted into one another. A hash table (or dictionary) is used to store and group the strings by their computed key. This approach handles edge cases like single-character strings separately.

#271 Encode and Decode Strings [Medium]

Key Insights

- Use length-prefix encoding for each string to avoid ambiguity.
- Append a delimiter (e.g., "#") after the length to separate it from the string content.
- This method ensures that even if the original strings contain special characters, including the delimiter, the decoding remains unambiguous.
- The approach handles edge cases, such as empty strings.

String · LeetCode · By Pattern

— 218 —

LeetCode

move horizontally (left or right) then vertically (up or down) to avoid stepping into invalid regions from the extra row. For any other letter, move vertically (up or down) first, then horizontally (left or right). Append "I" to the moves after reaching each letter. - Update the current position to the letter's coordinates.

This method ensures you only traverse valid cells on the board and minimizes moves by taking the optimal coordinate difference each step.

String · LeetCode · By Pattern

— 220 —

LeetCode

Python

```
# Function to find the length of the longest substring without repeating characters
def length_of_longest_substring(s):
    # Dictionary to store the most recent index of each character
    char_index = {}
    # Initialize starting index of current window and max length
    left = 0
    max_length = 0

    # Iterate over the string with the right pointer
    for right, char in enumerate(s):
        # If char is already in char_index & within the current window
        if char in char_index and char_index[char] >= left:
            # Move the left pointer to one position after the last occurrence of char
            left = char_index[char] + 1
        # Update the latest index of the char
        char_index[char] = right
        # Update max_length with the current window size if larger
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
print(length_of_longest_substring("abcabcbb")) # Expected output: 3
```

Python

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] > 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

Python

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        cnt = Counter()
        ans = l = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1:
                cnt[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

Python

Sliding Window · LeetCode · By Pattern

— 222 —

LeetCode

```

class Solution {
    def lengthOfLongestSubstring(self, s): # map for index
        d = {} # value == its index
        l = 0
        ans = 0
        for j, c in enumerate(s):
            if c in d:
                # must have check l, example "abba"
                l = max(l, d[c] + 1)
            d[c] = j
            ans = max(ans, j - l + 1)
        return ans

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        if not s:
            return 0
        l = 0 # left
        r = 0 # right
        result = 0
        ifFoundIndex = {False} + 256

        while r < len(s):
            while ifFoundIndex.get(s[r]):
                ifFoundIndex.get(s[l]) = False
                l += 1
            ifFoundIndex.get(s[r]) = True

            # check before when
            result = max(result, r - l + 1)
            r += 1
        return result

class Solution: # extra space for set()
    def lengthOfLongestSubstring(self, s: str) -> int:
        start = 0
        ans = 0
        for i < len(s):
            # check
            while d[i] > 0:

```

```

        d[s[start]] += 1 # not s[start], not start
        start += 1
        ans = max(ans, l - start + 1)
        d[c] = 1
        return ans

#####

class Solution(object):
    def lengthOfLongestSubstring(self, s): # no extra data structure
        :type s: str
        :rtype: int
        d = collections.defaultdict(int)
        l = ans = 0
        for i, c in enumerate(s):
            while l > 0 and d[c] > 0:
                d[s[l - 1]] -= 1
                l -= 1
            d[c] += 1
            l += 1
            ans = max(ans, l)
        return ans

#####

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        n = len(s)

```

C++

C++

```

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int ans = 0;
        vector<int> count(128);

        for (int l = 0, r = 0; r < s.length(); ++r) {
            ++count[s[r]];
            while (count[s[r]] > 1)
                --count[s[l++]];
            ans = max(ans, r - l + 1);
        }
        return ans;
    }
};

```

C++

```

class Solution {
    function lengthOfLongestSubstring(s) {
        let s = s.trim();
        let ans = 0;
        let count = new Array(128, 0);
        let l = 0;
        for (let r = 0; r < s.length; ++r) {
            ++count[s[r]];
            while (count[s[r]] > 1) {
                --count[s[l++]];
            }
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
}

C++

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        int n = s.length();
        int ans = 0;
        vector<int> count(128, 0);
        int l = 0;
        for (int r = 0; r < s.length; ++r) {
            ++count[s[r]];
            while (count[s[r]] > 1) {
                --count[s[l++]];
            }
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
}

C++

```

```

class Solution {
    //
    // @param String s
    // @return Integer
    //
    function lengthOfLongestSubstring(s) {
        let ans = 0;
        for (let i = 0; i < s.length; ++i) {
            let count = 0;
            for (let j = i; j < s.length; ++j) {
                ++count[s[j]];
                while (count[s[j]] > 1) {
                    --count[s[i++]];
                }
                ans = Math.max(ans, j - i + 1);
            }
        }
        return ans;
    }
}

Java

```

Java

```

class Solution {
    public int lengthOfLongestSubstring(String s) {
        int ans = 0;
        int[] count = new int[128];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            ++count[s.charAt(r)];
            while (count[s.charAt(r)] > 1)
                --count[s.charAt(l++)];
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
}

Java

```

```

class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] cnt = new int[128];
        int ans = 0, n = s.length();
        for (int l = 0, r = 0; r < n; ++r) {
            char c = s.charAt(r);
            ++cnt[c];
            while (cnt[c] > 1) {
                --cnt[s.charAt(l++)];
            }
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
}

Java

public class Solution {
    public int lengthOfLongestSubstring(string s) {
        int n = s.length;
        int ans = 0;
        var cnt = new int[128];
        for (let l = 0, r = 0; r < s.length; ++r) {
            ++cnt[s[r]];
            while (cnt[s[r]] > 1) {
                --cnt[s[l++]];
            }
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
}

Java

public class Solution {
    public int lengthOfLongestSubstring(string s) {
        var ss = new HashSet<char>();
        let l = 0, ans = 0;
        for (let j = 0; j < s.length; ++j) {
            while (ss.Contains(s[j])) {
                ss.Remove(s[l++]);
            }
            ss.Add(s[j]);
            ans = Math.Max(ans, j - l + 1);
        }
        return ans;
    }
}

Java

```

```

use std::collections::HashSet;

impl Solution {
    pub fn length_of_longest_substring(s: String) -> i32 {
        let s = s.as_bytes();
        let mut set = HashSet::new();
        let mut l = 0;

        .iter().
        .map(|c| {
            while set.contains(*c) {
                set.remove(&s[l]);
                l += 1;
            }
            set.insert(*c);
            set.len()
        })
        .max()
        .unwrap_or(0) as i32
    }
}

JavaScript

function lengthOfLongestSubstring(s) {
    let ans = 0;
    const n = s.length;
    const cnt = new Map();
    for (let l = 0, r = 0; r < n; ++r) {
        cnt.set(s[r], (cnt.get(s[r]) || 0) + 1);
        while (cnt.get(s[r]) > 1) {
            cnt.set(s[l], (cnt.get(s[l]) || 0) - 1);
            ++l;
        }
        ans = Math.max(ans, r - l + 1);
    }
    return ans;
}

JavaScript

```

```

//
 * @param {string} s
 * @return {number}
 */
var lengthOfLongestSubstring = function (s) {
    const ss = new Set();
    let l = 0;
    let ans = 0;
    for (let j = 0; j < s.length; ++j) {
        while (ss.has(s[j])) {
            ss.delete(s[l++]);
        }
        ss.add(s[j]);
        ans = Math.max(ans, j - l + 1);
    }
    return ans;
};

```

TypeScript

TypeScript

```

function lengthOfLongestSubstring(s: string): number {
    let ans = 0;
    const cnt = new Map<string, number>();
    const n = s.length;
    for (let l = 0, r = 0; r < n; ++r) {
        cnt.set(s[r], (cnt.get(s[r]) || 0) + 1);
        while (cnt.get(s[r]) > 1) {
            cnt.set(s[l], (cnt.get(s[l]) || 0) - 1);
            ++l;
        }
        ans = Math.max(ans, r - l + 1);
    }
    return ans;
}

```

TypeScript

```

function lengthOfLongestSubstring(s: string): number {
    let ans = 0;
    const vis = new Set<string>();
    for (let l = 0, j = 0; l < s.length; ++l) {
        while (vis.has(s[j])) {
            vis.delete(s[l++]);
        }
        vis.add(s[j]);
        ans = Math.max(ans, j - l + 1);
    }
    return ans;
}

```

TypeScript

```

proc lengthOfLongestSubstring(s: string): int =
  var
    l = 0
    j = 0
    ans = 0
    literals: set[char] = {}

  while l < s.len:
    while s[j] in literals:
      if s[j] in literals:
        exit(literals, s[j])
      j = 1
    literals.incl(s[j]) # Uniform Function Call Syntax f(x) = x.f
    res = max(res, j - l + 1)
    l = 1

  result = res # result has the default return value

echo lengthOfLongestSubstring("abcbabcb")

```

Go

Go

```

func lengthOfLongestSubstring(s string) int {
    ss := map[byte]bool{}
    l, ans := 0, 0
    for j := 0; j < len(s); j++ {
        for ss[s[j]] {
            ss[s[l]] = false
            l++
        }
        ss[s[j]] = true
        ans = max(ans, j-l+1)
    }
    return ans
}

```

Go

```

class Solution {
    func lengthOfLongestSubstring_1(s string) int {
        var map = [Character]bool{}
        var currentStartingIndex = 0
        var l = 0
        var maxLength = 0
        for char in s {
            if map[char] == nil {
                if map[char] == currentStartingIndex {
                    maxLength = max(maxLength, l - currentStartingIndex)
                    currentStartingIndex = map[char] + 1
                }
            }
            map[char] = l
            l++
        }
        return max(maxLength, l - currentStartingIndex)
    }
}

```

Rust

Rust

```

impl Solution {
    pub fn length_of_longest_substring(s: String) -> i32 {
        let mut set = [0; 128];
        let mut ans = 0;
        let mut l = 0;
        let chars: Vec<char> = s.chars().collect();
        let n = chars.len();
        for (r, &c) in chars.iter().enumerate() {
            set[*c as usize] += 1;
            while set[*c as usize] > 1 {
                set[*chars[l] as usize] -= 1;
                l += 1;
            }
            ans = ans.max((r - l + 1) as i32);
        }
        ans
    }
}

```

Kotlin

Kotlin

```

class Solution {
    fun lengthOfLongestSubstring(s: String): Int {
        val s = s.length
        var ans = 0
        val set = IntArray(128)
        var l = 0
        for (r in 0 until s) {
            set[s[r].toInt()]++
            while (set[s[r].toInt()] > 1) {
                set[s[l].toInt()]--
                l++
            }
            ans = Math.max(ans, r - l + 1)
        }
        return ans
    }
}

```

[*] Sliding Window -- Pattern Solution (stored optimal)

Python

```

# Function to find the length of the longest substring without repeating characters
def length_of_longest_substring(s):
    # Dictionary to store the most recent index of each character
    char_index = {}
    # Initialize starting index of current window and max length
    left = 0
    max_length = 0

    # Iterate over the string with the right pointer
    for right, char in enumerate(s):
        # If char is in dictionary and its index is within the current window
        if char in char_index and char_index[char] >= left:
            # Move the left pointer to one position after the last occurrence of char
            left = char_index[char] + 1
        # Update the latest index of the char
        char_index[char] = right
        # Update max_length with the current window size if larger
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
s = "abcabcbb"
print(length_of_longest_substring(s)) # Expected output: 3

```

C++

```

// Function to find the length of the longest substring without repeating characters
#include <iostream>
#include <unordered_map>
#include <string>
#include <algorithm>
using namespace std;

int lengthOfLongestSubstring(const string& s) {
    // Hash table to store the latest index of each character
    unordered_map<char, int> charIndex;
    int left = 0; // Left pointer for sliding window
    int maxLength = 0;

    // Iterate through the string with right pointer
    for (int right = 0; right < s.length(); right++) {
        char currentChar = s[right];
        // If current character is found in charIndex and its index is within the current window
        if (charIndex.find(currentChar) != charIndex.end() && charIndex[currentChar] >= left) {
            // Move left pointer just past this character's last position
            left = charIndex[currentChar] + 1;
        }
        // Update or insert the current character's index
        charIndex[currentChar] = right;
        // Update the maximum length with the size of the current window
        maxLength = max(maxLength, right - left + 1);
    }
}

```

```

}

return maxLength;
}

```

```

// Example usage:
// int main() {
//     cout << lengthOfLongestSubstring("abcabcbb") << endl; // Expected output: 3
//     return 0;
// }

```

#159

MEDIUM

Longest Substring with At Most Two Distinct Characters

LeetCode - SimplifyLeetCode - WallCC - LeetCode - Editorial

Hash Table - String - Sliding Window

LeetCode Premium 99

Problem:

Given a string *s*, return the length of the longest substring that contains at most two distinct characters.

Example 1:

Input: *s* = "eceba"

Output: 3

Explanation: The substring is "ece" which its length is 3.

Example 2:

Input: *s* = "ccabbb"

Output: 5

Explanation: The substring is "aabbb" which its length is 5.

Constraints:

• 1 <= *s*.length <= 105

• *s* consists of English letters.

Python

Python

```

# Python solution for Longest Substring with At Most Two Distinct Characters
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        left = 0 # Initialize left pointer of the window
        max_length = 0 # Variable to store the maximum window size found
        char_count = {} # Dictionary to count characters in the current window

        for right, char in enumerate(s):
            # Add the current character to the window (increase count)
            char_count[char] = char_count.get(char, 0) + 1

            # If there are more than two distinct characters, shrink the window from the left
            while len(char_count) > 2:
                left_char = s[left]
                char_count[left_char] -= 1
                if char_count[left_char] == 0:
                    del char_count[left_char]
                left = left + 1

            # Update max_length if current window is larger
            max_length = max(max_length, right - left + 1)

        return max_length

```

Python

```

class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        ans = 0
        distinct = 0
        count = [0] * 128

        l = 0
        for r, c in enumerate(s):
            count[ord(c)] += 1
            if count[ord(c)] == 1:
                distinct += 1
            while distinct > 2:
                count[ord(s[l])] -= 1
                if count[ord(s[l])] == 0:
                    distinct -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

Python

```

class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter()
        ans = j = 0
        for i, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > 2:
                cnt[s[i]] -= 1
                if cnt[s[i]] == 0:
                    del cnt[s[i]]
                    i += 1
                    cnt.pop(s[i])
            ans = max(ans, i - j + 1)
        return ans

```

Python

```

<>> from collections import Counter
<>> s = Counter()
<>> i
Counter()
<>> i[0] += 1
<>> i
Counter({'0': 1})

<>> i
Counter({'0': 1, '1': 1})

<>> i
Counter({'0': 1, '1': 1, '2': 1})

<>> i
Counter({'0': 1, '1': 1, '2': 1, '3': 1})

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyboardInterrupt:

<>>
<>> i.pop('c')
1
<>> i
Counter({'0': 1, '1': 1})
<>>

```

```

from collections import Counter

class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter()
        ans = i = 0
        for j, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > 2:
                cnt[s[i]] -= 1
                if cnt[s[i]] == 0:
                    del cnt[s[i]]
                    i += 1
            ans = max(ans, j - i + 1)
        return ans

```

C++

C++

Sliding Window · LeetCode - By Pattern

— 235 —

LeetCode

```

class Solution {
public:
    int lengthOfLongestSubstringTwoDistinct(string s) {
        int ans = 0;
        int distinct = 0;
        vector<int> count(128);

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s[r]] == 1)
                ++distinct;
            while (distinct == 3)
                if (--count[s[l++]] == 0)
                    --distinct;
            ans = max(ans, r - l + 1);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int lengthOfLongestSubstringTwoDistinct(string s) {
        unordered_map<char, int> cnt;
        int n = s.size();
        int ans = 0;
        for (int l = 0, j = 0; l < n; ++l) {
            cnt[s[l]]++;
            while (cnt.size() > 2) {
                cnt[s[j]]--;
                if (cnt[s[j]] == 0) {
                    cnt.erase(s[j]);
                }
                ++j;
            }
            ans = max(ans, l - j + 1);
        }
        return ans;
    }
};

```

C++

Sliding Window · LeetCode - By Pattern

— 236 —

LeetCode

```

class Solution {
public:
    int lengthOfLongestSubstringTwoDistinct(string s) {
        unordered_map<char, int> cnt;
        int n = s.size();
        int ans = 0;
        for (int l = 0, j = 0; l < n; ++l) {
            cnt[s[l]]++;
            while (cnt.size() > 2) {
                cnt[s[j]]--;
                if (cnt[s[j]] == 0) {
                    cnt.erase(s[j]);
                }
                ++j;
            }
            ans = max(ans, l - j + 1);
        }
        return ans;
    }
};

```

Java

```

class Solution {
public:
    int lengthOfLongestSubstringTwoDistinct(String s) {
        int ans = 0;
        int distinct = 0;
        int[] count = new int[128];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s.charAt(r)] == 1)
                ++distinct;
            while (distinct == 3)
                if (--count[s.charAt(l++)] == 0)
                    --distinct;
            ans = Math.max(ans, r - l + 1);
        }
        return ans;
    }
};

```

Java

Sliding Window · LeetCode - By Pattern

— 237 —

LeetCode

```

class Solution {
public:
    int lengthOfLongestSubstringTwoDistinct(string s) {
        Map<Character, Integer> cnt = new HashMap<>();
        int n = s.length();
        int ans = 0;
        for (int l = 0, j = 0; l < n; ++l) {
            char c = s.charAt(l);
            cnt.put(c, cnt.getOrDefault(c, 0) + 1);
            while (cnt.size() > 2) {
                char t = s.charAt(j++);
                cnt.put(t, cnt.get(t) - 1);
                if (cnt.get(t) == 0) {
                    cnt.remove(t);
                }
            }
            ans = Math.max(ans, l - j + 1);
        }
        return ans;
    }
};

```

Java

```

class Solution {
public:
    int lengthOfLongestSubstringTwoDistinct(String s) {
        Map<Character, Integer> cnt = new HashMap<>();
        int n = s.length();
        int ans = 0;
        for (int l = 0, j = 0; l < n; ++l) {
            char c = s.charAt(l);
            cnt.put(c, cnt.getOrDefault(c, 0) + 1);
            while (cnt.size() > 2) {
                char t = s.charAt(j++);
                cnt.put(t, cnt.get(t) - 1);
                if (cnt.get(t) == 0) {
                    cnt.remove(t);
                }
            }
            ans = Math.max(ans, l - j + 1);
        }
        return ans;
    }
};

```

Go

Go

Sliding Window · LeetCode - By Pattern

— 238 —

LeetCode

```

func lengthOfLongestSubstringTwoDistinct(s string) (ans int) {
    cnt := map[byte]int{}
    j := 0
    for i := range s {
        cnt[s[i]]++
        for len(cnt) > 2 {
            cnt[s[j]]--
            if cnt[s[j]] == 0 {
                delete(cnt, s[j])
            }
            j++
        }
        ans = max(ans, i-j+1)
    }
    return
}

```

Go

```

func lengthOfLongestSubstringTwoDistinct(s string) (ans int) {
    cnt := map[byte]int{}
    j := 0
    for i := range s {
        cnt[s[i]]++
        for len(cnt) > 2 {
            cnt[s[j]]--
            if cnt[s[j]] == 0 {
                delete(cnt, s[j])
            }
            j++
        }
        ans = max(ans, i-j+1)
    }
    return
}

```

[*] Sliding Window · Pattern Solution (matched from community answers)

Python

Sliding Window · LeetCode - By Pattern

— 239 —

LeetCode

```

# Python solution for longest substring with at most two distinct characters
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        left = 0 # initialize left pointer of the window
        max_length = 0 # variable to store the maximum window size found
        char_count = {} # Dictionary to count characters in the current window

        for right, char in enumerate(s):
            # Add the current character to the window (increase count)
            char_count[char] = char_count.get(char, 0) + 1

            # If there are more than two distinct characters, shrink the window from the left
            while len(char_count) > 2:
                left_char = s[left]
                char_count[left_char] -= 1
                if char_count[left_char] == 0:
                    del char_count[left_char]
                left += 1

            # Update max_length if current window is larger
            max_length = max(max_length, right - left + 1)

        return max_length

```

#340

MEDIUM

Longest Substring with At Most K Distinct Characters

LeetCode · Simplest · Medium · LeetCode · Editorial

Hash Table · String · Sliding Window

Like Premium 98

Problem:

Given a string s and an integer k, return the length of the longest substring of s that contains at most k distinct characters.

Example 1:

Input: s = "eceba", k = 2

Output: 3

Explanation: The substring is "ece" with length 3.

Example 2:

Input: s = "aa", k = 1

Output: 2

Explanation: The substring is "aa" with length 2.

Constraints:

• 1 <= s.length <= 5 * 10⁴

• 0 <= k <= 50

Python

Python

Sliding Window · LeetCode - By Pattern

— 240 —

LeetCode

```
# Python solution for Longest Substring with At Most K Distinct Characters

def longest_substring_with_k_distinct(s: str, k: int) -> int:
    # Handle edge case: If k is 0, no valid substring exists
    if k == 0:
        return 0

    # Dictionary to store frequency of characters in the current window
    char_count = {}

    left = 0 # Left pointer for the window
    max_length = 0 # Variable to track the maximum length found

    # Iterate through the string using the right pointer
    for right in range(len(s)):
        # Add the new char of the current character in the dictionary
        current_char = s[right]
        char_count[current_char] = char_count.get(current_char, 0) + 1

        # If number of distinct chars more than k distinct characters, shrink from the left
        while len(char_count) > k:
            left_char = s[left]
            char_count[left_char] -= 1
            # Remove the character count if it becomes 0
            if char_count[left_char] == 0:
                del char_count[left_char]

            left += 1

        # Update the maximum length of the valid window found so far
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
s = "eceba"
k = 2
print(longest_substring_with_k_distinct(s, 2)) # Output: 3
```

```
Python

class Solution:
    def longestSubstringWithKDistinct(self, s: str, k: int) -> int:
        ans = 0
        distinct = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            if count[c] == 1:
                distinct += 1
            while distinct > k + 1:
                count[s[l]] -= 1
                if count[s[l]] == 0:
                    distinct -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

Python

```
class Solution {
    def longestSubstringWithKDistinct(self, s: str, k: int) -> int:
        l = 0
        cnt = Counter()
        for r in range(len(s)):
            cnt[s[r]] += 1
            if len(cnt) > k:
                cnt[s[l]] -= 1
                if cnt[s[l]] == 0:
                    del cnt[s[l]]
                l += 1
            return len(s) - l
    }
```

```
Python

class Solution:
    def longestSubstringWithKDistinct(self, s: str) -> int:
        cnt = Counter() # or: cnt = defaultdict(int)
        ans = 0
        for l, r in enumerate(s):
            cnt[s[l]] += 1
            while len(cnt) > k:
                cnt[s[l]] -= 1
                if cnt[s[l]] == 0:
                    del cnt[s[l]]
            l += 1
            ans = max(ans, j - l + 1)
        return ans
```

C++

```
C++

class Solution {
public:
    int longestSubstringWithKDistinct(string s, int k) {
        int ans = 0;
        int distinct = 0;
        vector<int> count(128);

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s[r]] == 1)
                ++distinct;
            while (distinct == k + 1)
                if (--count[s[l++]] == 0)
                    --distinct;
            ans = max(ans, r - l + 1);
        }

        return ans;
    }
};
```

C++

```
class Solution {
public:
    int longestSubstringWithKDistinct(string s, int k) {
        unordered_map<char, int> cnt;
        int l = 0;
        for (char& c : s) {
            ++cnt[c];
            if (cnt.size() > k) {
                if (--cnt[s[l]] == 0) {
                    cnt.erase(s[l]);
                }
                ++l;
            }
            return s.size() - l;
        }
    }
};
```

```
C++

class Solution {
public:
    int longestSubstringWithKDistinct(string s, int k) {
        unordered_map<char, int> cnt;
        int s = s.size();
        int ans = 0, j = 0;
        for (int i = 0; i < s; ++i) {
            cnt[s[i]]++;
            while (cnt.size() > k) {
                if (--cnt[s[j]] == 0) {
                    cnt.erase(s[j]);
                }
                ++j;
            }
            ans = max(ans, i - j + 1);
        }
        return ans;
    }
};
```

Java

Java

```
class Solution {
    public int longestSubstringWithKDistinct(String s, int k) {
        int ans = 0;
        int distinct = 0;
        int[] count = new int[128];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (++count[s.charAt(r)] == 1)
                ++distinct;
            while (distinct == k + 1)
                if (--count[s.charAt(l++)] == 0)
                    --distinct;
            ans = Math.max(ans, r - l + 1);
        }

        return ans;
    }
}
```

```
Java

class Solution {
    public int longestSubstringWithKDistinct(String s, int k) {
        Map<Character, Integer> cnt = new HashMap<>();
        int l = 0;
        char[] cs = s.toCharArray();
        for (char c : cs) {
            cnt.merge(c, 1, Integer::sum);
            if (cnt.size() > k) {
                if (cnt.merge(cs[l], -1, Integer::sum) == 0) {
                    cnt.remove(cs[l]);
                }
                ++l;
            }
            return cs.length - l;
        }
    }
}
```

Java

```
class Solution {
    public int longestSubstringWithKDistinct(String s, int k) {
        Map<Character, Integer> cnt = new HashMap<>();
        int s = s.length();
        int ans = 0, j = 0;
        for (int i = 0; i < s; ++i) {
            char c = s.charAt(i);
            cnt.put(c, cnt.getOrDefault(c, 0) + 1);
            while (cnt.size() > k) {
                char c = s.charAt(j);
                cnt.put(c, cnt.getOrDefault(c, 0) - 1);
                if (cnt.get(c) == 0) {
                    cnt.remove(c);
                }
                ++j;
            }
            ans = Math.max(ans, i - j + 1);
        }
        return ans;
    }
}
```

TypeScript

```
TypeScript

function longestSubstringWithKDistinct(s: string, k: number): number {
    const cnt: Map<string, number> = new Map();
    let l = 0;
    for (const c of s) {
        cnt.set(c, (cnt.get(c) ?? 0) + 1);
        if (cnt.size > k) {
            cnt.set(s[l], cnt.get(s[l]) - 1);
            if (cnt.get(s[l]) == 0) {
                cnt.delete(s[l]);
            }
            l++;
        }
    }
    return s.length - l;
}
```

```
TypeScript

function longestSubstringWithKDistinct(s: string, k: number): number {
    const cnt: Map<string, number> = new Map();
    let l = 0;
    for (const c of s) {
        cnt.set(c, (cnt.get(c) ?? 0) + 1);
        if (cnt.size > k) {
            cnt.set(s[l], cnt.get(s[l]) - 1);
            if (cnt.get(s[l]) == 0) {
                cnt.delete(s[l]);
            }
            l++;
        }
    }
    return s.length - l;
}
```

```
Go

func longestSubstringWithKDistinct(s string, k int) int {
    cnt := map[byte]int{}
    l := 0
    for r, c := range s {
        cnt[byte(c)]++
        if len(cnt) > k {
            if cnt[s[l]] == 0 {
                delete(cnt, s[l])
            }
            l++
        }
    }
    return len(s) - l
}
```

```
Go

func longestSubstringWithKDistinct(s string, k int) (ans int) {
    cnt := map[byte]int{}
    j := 0
    for i := range s {
        cnt[s[i]]++
        for len(cnt) > k {
            cnt[s[j]]--
            if cnt[s[j]] == 0 {
                delete(cnt, s[j])
            }
            j++
        }
        ans = max(ans, i-j+1)
    }
    return
}
```

[*] Sliding Window - Pattern Solution (stored optimal)

Python

```

# Python solution for Longest Substring with At Most K Distinct Characters

def longest_substring_with_k_distinct(s: str, k: int) -> int:
    # Handle edge case: if k is 0, no valid substring exists
    if k == 0:
        return 0

    # Dictionary to store frequency of characters in the current window
    char_count = {}

    left = 0 # Left pointer for the window
    max_length = 0 # Variable to track the maximum length found

    # Iterate through the string using the right pointer
    for right in range(len(s)):
        # Increment the count of the current character in the dictionary
        char_count[current_char] = char_count.get(current_char, 0) + 1

        # If there are more than k distinct characters, shrink from the left
        while len(char_count) > k:
            left_char = s[left]
            char_count[left_char] -= 1
            # Remove the character count if it becomes 0
            if char_count[left_char] == 0:
                del char_count[left_char]

            left += 1

        # Update the maximum length of the valid window found so far
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
s = "eceba"
k = 2
print(longest_substring_with_k_distinct(s, 2)) # Output: 3

```

Sliding Window - LeetCode - By Pattern

— 247 —

LeetCode

```

// C++ solution for Longest Substring with At Most K Distinct Characters
#include <iostream>
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

int longestSubstringWithKDistinct(string s, int k) {
    // Handle edge case: if k is 0, no valid substring exists
    if (k == 0) return 0;

    unordered_map<char, int> charCount; // Map to store frequency of characters in the window
    int left = 0; // Left pointer for the window
    int maxLength = 0; // Variable to record the maximum length

    for (int right = 0; right < s.size(); right++) {
        // Increment the count for the current character
        charCount[s[right]]++;

        // If the window has more than k distinct characters, shrink from the left
        while (charCount.size() > k) {
            charCount[s[left]]--;
            if (charCount[s[left]] == 0) {
                charCount.erase(s[left]);
            }

            left++;
        }

        // Update maximum length
        maxLength = max(maxLength, right - left + 1);
    }

    return maxLength;
}

// Example usage:
// int main() {
//     cout << longestSubstringWithKDistinct("eceba", 2) << endl; // Output: 3
//     return 0;
// }

```

#424

MEDIUM

Longest Repeating Character Replacement

LeetCode - Simple - WAAEC - LeetCode - Editorial

Hash Table - String - Sliding Window

Lists Grid 169

Problem:
You are given a string *s* and an integer *k*. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most *k* times.
Return the length of the longest substring containing the same letter you can get after performing the above operations.

Sliding Window - LeetCode - By Pattern

— 248 —

LeetCode

Example 1:
Input: s = "ABAB", k = 2
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:
Input: s = "AABABBA", k = 1
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
The substring "BBBB" has the longest repeating letters, which is 4.
There may exist other ways to achieve this answer too.

Constraints:

- 1 <= s.length <= 105
- s consists of only uppercase English letters.
- 0 <= k <= s.length

```

Python
Python
# Python solution using the sliding window technique
def characterReplacement(s: str, k: int) -> int:
    char_count = {}
    max_freq = 0 # Track the maximum frequency of a single character in the window
    max_length = 0
    left = 0 # Left pointer of the sliding window

    # Expand the sliding window with the right pointer
    for right in range(len(s)):
        # Increment the current character
        char_count[s[right]] = char_count.get(s[right], 0) + 1
        # Update the max frequency in the current window
        max_freq = max(max_freq, char_count[s[right]])

        # Check if the number of replacements needed exceeds k
        # (current window size - count of most frequent character) > k
        if (right - left + 1) - max_freq > k:
            # If condition fails, shrink the window by moving left pointer
            char_count[s[left]] -= 1
            left += 1

        # Update the maximum length found so far
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
s = "AABABBA"
k = 1
print(characterReplacement(s, k)) # Expected output: 4

```

Sliding Window - LeetCode - By Pattern

— 249 —

LeetCode

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        ans = 0
        maxCount = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])
            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

Python
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        maxCount = 0
        count = collections.Counter()

        # l and r track the maximum window instead of the valid window.
        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])
            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1
            return r - l + 1

Python
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        cnt = Counter()
        l = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            mx = max(cnt.values())
            if r - l + 1 - mx > k:
                cnt[s[l]] -= 1
                l += 1
            return cnt[s[l]] - 1

Python

```

Python

Sliding Window - LeetCode - By Pattern

— 250 —

LeetCode

```

class Solution {
    def characterReplacement(self, s: str, k: int) -> int:
        counter = [0] * 26
        l = j = maxCnt = 0
        while l < len(s):
            counter[ord(s[j]) - ord('A')] += 1
            maxCnt = max(maxCnt, counter[ord(s[j]) - ord('A')])
            if j - l + 1 > maxCnt + k:
                counter[ord(s[l]) - ord('A')] -= 1
                l += 1
            j += 1
            return j - l
    }

C++
C++
class Solution {
public:
    int characterReplacement(string s, int k) {
        int ans = 0;
        int maxCount = 0;
        vector<int> count(26);

        for (int l = 0, r = 0; r < s.length(); ++r) {
            maxCount = max(maxCount, ++count[s[r] - 'A']);
            while (maxCount + k < r - l + 1) {
                --count[s[l] - 'A'];
                ans = max(ans, r - l + 1);
            }
            return ans;
        }
    }

C++
class Solution {
public:
    int characterReplacement(string s, int k) {
        vector<int> count(26);
        int l = 0, j = 0, maxCnt = 0;
        for (char c = s[j]; j < s.length(); ++j) {
            ++count[c - 'A'];
            maxCnt = max(maxCnt, count[c - 'A']);
            if (j - l + 1 > maxCnt + k) {
                --count[s[l] - 'A'];
                ++l;
            }
            ++j;
            return j - l;
        }
    }

Java
Java

```

Sliding Window - LeetCode - By Pattern

— 251 —

LeetCode

```

class Solution {
    public int characterReplacement(String s, int k) {
        int ans = 0;
        int maxCount = 0;
        int[] count = new int[26];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            maxCount = Math.max(maxCount, ++count[s.charAt(r) - 'A']);
            while (maxCount + k < r - l + 1) {
                --count[s.charAt(l) - 'A'];
                ans = Math.max(ans, r - l + 1);
            }
            return ans;
        }
    }

Java
class Solution {
    public int characterReplacement(String s, int k) {
        int maxCount = 0;
        int[] count = new int[26];

        // l and r track the maximum window instead of the valid window.
        int l = 0;
        for (r = 0; r < s.length(); ++r) {
            maxCount = Math.max(maxCount, ++count[s.charAt(r) - 'A']);
            if (maxCount + k < r - l + 1) {
                --count[s.charAt(l) - 'A'];
            }
            return r - l;
        }
    }

Java
class Solution {
    public int characterReplacement(String s, int k) {
        int[] cnt = new int[26];
        int l = 0, mx = 0;
        int n = s.length();
        for (int r = 0; r < n; ++r) {
            mx = Math.max(mx, ++cnt[s.charAt(r) - 'A']);
            if (r - l + 1 > mx + k) {
                --cnt[s.charAt(l) - 'A'];
            }
            return n - l;
        }
    }

Java

```

Sliding Window - LeetCode - By Pattern

— 252 —

LeetCode

```
class Solution {
public: int characterReplacement(string s, int k) {
    int[] counter = new int[26];
    int l = 0;
    int i = 0;
    for (int maxCat = 0; i < s.length(); ++i) {
        char c = s.charAt(i);
        ++counter[c - 'A'];
        maxCat = Math.max(maxCat, counter[c - 'A']);
        if (i - j + 1 - maxCat > k) {
            --counter[s.charAt(j) - 'A'];
            ++j;
        }
        return i - j;
    }
}
```

TypeScript

TypeScript

```
function characterReplacement(s: string, k: number): number {
    const ide = (i: string) => i.charCodeAt(0) - 65;
    const cnt: number[] = Array(26).fill(0);
    const n = s.length;
    let l, m = [0, 0];
    for (let r = 0; r < n; ++r) {
        m = Math.max(m, ++cnt[ide(s[r])]);
        if (r - l + 1 - m > k) {
            --cnt[ide(s[l++])];
        }
    }
    return n - l;
}
```

TypeScript

```
function characterReplacement(s: string, k: number): number {
    const ide = (i: string) => i.charCodeAt(0) - 65;
    const cnt: number[] = Array(26).fill(0);
    const n = s.length;
    let l, m = [0, 0];
    for (let r = 0; r < n; ++r) {
        m = Math.max(m, ++cnt[ide(s[r])]);
        if (r - l + 1 - m > k) {
            --cnt[ide(s[l++])];
        }
    }
    return n - l;
}
```

Go

Go

Sliding Window - LeetCode - By Pattern

— 253 —

LeetCode

```
func characterReplacement(s string, k int) int {
    counter := make([]int, 26)
    j, maxCat := 0, 0
    for i := range s {
        c := s[i] - 'A'
        counter[c]++
        if maxCat < counter[c] {
            maxCat = counter[c]
        }
        if i - j + 1 - maxCat > k {
            counter[s[j] - 'A']--
            j++
        }
        return i - j
    }
}
```

[*] Sliding Window - Pattern Solution (matched from community answers)

Python

```
# Python solution using the sliding window technique
def characterReplacement(s: str, k: int) -> int:
    # Dictionary to store frequency of characters in the current window
    char_counts = {}
    max_freq = 0 # Track the maximum frequency of a single character in the window
    max_length = 0
    left = 0 # Left pointer of the sliding window

    # Expand the sliding window with the right pointer
    for right in range(len(s)):
        # Count the current character
        char_counts[right] = char_counts.get(right, 0) + 1
        # Update the max frequency in the current window
        max_freq = max(max_freq, char_counts[right])

        # Check if the number of replacements needed exceeds k
        # (current window size - count of most frequent character) > k
        if (right - left + 1) - max_freq > k:
            # If necessary, shrink the window by moving left pointer
            char_counts[left] -= 1
            left += 1

        # Update the maximum length found so far
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# print(characterReplacement("ABABBA", 1)) # Expected output: 4
```

#438

MEDIUM

Find All Anagrams in a String

LeetCode - Solution - WatchCC - LeetCode - Editorial

Hash Table - String - Sliding Window

Sliding Window - LeetCode - By Pattern

— 254 —

LeetCode

Lists - Grid 169

Problem:

Given two strings s and p, return an array of all the start indices of p's anagrams in s. You may return the answer in any order.

Example 1:

Input: s = "cbaebabacd", p = "abc"

Output: [0,6]

Explanation:

The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: s = "abab", p = "ab"

Output: [0,1,2]

Explanation:

The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

• 1 <= s.length, p.length <= 3 * 10⁴

• s and p consist of lowercase English letters.

>> Brute Force (Sliding Window) Interview Prep

Python

```
# Brute force approach
class Solution:
    def findAnagrams(self, s: str, p: str):
        # Brute force (O(N^2)) - enumerate all windows
        ans = []
        n = len(s)
        for l in range(n):
            window_size = 0
            for r in range(l, n):
                window_size += 1
                best = max(best, r - l + 1)
            return best
```

Python

Python

Sliding Window - LeetCode - By Pattern

— 255 —

LeetCode

```
# Define the function to find all anagram indices in the string
def findAnagrams(s: str, p: str):
    # Import Counter for frequency counting
    from collections import Counter
    # List to store the starting indices of all anagrams
    result = []
    # Length of string s
    s_length = len(s)
    # Frequency count for characters in p
    p_count = Counter(p)
    # Frequency count for the current window in s
    s_count = Counter()

    # Iterate over the string s
    for i in range(len(s)):
        # Add current character to the window count
        s_count[i+1] += 1

        # Remove the leftmost character from window if window size exceeds p_length
        if i == p_length:
            if s_count[i - p_length] == 1:
                del s_count[i - p_length] # Remove key completely if count becomes 0
            else:
                s_count[i - p_length] -= 1

        # Compare window count with p_count when window size is equal to p_length
        if s_count == p_count:
            result.append(i - p_length + 1)

    return result

# Example usage:
print(findAnagrams("cbaebabacd", "abc")) # Expected output: [0, 6]
```

Python

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        ans = []
        count = collections.Counter(p)
        required = len(p)

        for i, c in enumerate(s):
            count[c] -= 1
            if count[c] == 0:
                required -= 1
            if r == len(p):
                count[r - len(p)] += 1
                if count[r - len(p)] == 0:
                    required -= 1
                if required == 0:
                    ans.append(r - len(p) + 1)

        return ans
```

Python

Sliding Window - LeetCode - By Pattern

— 256 —

LeetCode

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        n, m = len(s), len(p)
        ans = []
        if m > n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter(s[:m - 1])
        for i in range(m - 1, n):
            cnt2[i+1] = 1
            if cnt1 == cnt2:
                ans.append(i - m + 1)
            cnt2[i+1 - m + 1] -= 1
        return ans
```

Python

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        n, m = len(s), len(p)
        ans = []
        if m > n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter(s[:m - 1])
        for i in range(m - 1, n):
            cnt2[i+1] = 1
            if cnt1 == cnt2:
                ans.append(i - m + 1)
            cnt2[i+1 - m + 1] -= 1
        return ans
```

C++

C++

```
class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        vector<int> ans;
        vector<int> count1(26);
        int required = p.length();

        for (const char c : p)
            ++count1[c];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (--count1[s[r]] == 0)
                --required;
            while (required == 0) {
                if (r - l + 1 == p.length())
                    ans.push_back(l);
                if (++count1[s[l]] > 0)
                    ++required;
                ++l;
            }
        }
        return ans;
    }
};
```

Sliding Window - LeetCode - By Pattern

— 257 —

LeetCode

```
C++
class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        int n = s.size(), m = p.size();
        vector<int> ans;
        if (m > n)
            return ans;
        vector<int> cnt1(26);
        for (char c : p) {
            ++cnt1[c - 'A'];
        }
        vector<int> cnt2(26);
        for (int l = 0, i = 0; i < m; ++i) {
            ++cnt2[i] - 'A';
        }
        for (int l = 0; l < n - m; ++l) {
            ++cnt2[l] - 'A';
            if (cnt1 == cnt2) {
                ans.push_back(l - m + 1);
            }
            --cnt2[l] - 'A';
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        int n = s.size(), m = p.size();
        vector<int> ans;
        if (m > n)
            return ans;
        vector<int> cnt1(26);
        for (char c : p) {
            ++cnt1[c - 'A'];
        }
        vector<int> cnt2(26);
        for (int l = 0, i = 0; i < m; ++i) {
            ++cnt2[i] - 'A';
        }
        for (int l = 0; l < n - m; ++l) {
            ++cnt2[l] - 'A';
            if (cnt1 == cnt2) {
                ans.push_back(l - m + 1);
            }
            --cnt2[l] - 'A';
        }
        return ans;
    }
};
```

Sliding Window - LeetCode - By Pattern

— 258 —

LeetCode

```

Java
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> ans = new ArrayList<>();
        int[] count = new int[128];
        int required = p.length();

        for (final char c : p.toCharArray())
            ++count[c];

        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (--count[s.charAt(r)] == 0)
                ++required;
            while (required == 0) {
                if (r - l == p.length())
                    ans.add(l);
                if (--count[s.charAt(l++)] > 0)
                    ++required;
            }
        }
        return ans;
    }
}

Java
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int m = s.length(), n = p.length();
        List<Integer> ans = new ArrayList<>();
        if (m < n)
            return ans;

        int[] cnt1 = new int[26];
        for (int i = 0; i < m; ++i) {
            ++cnt1[s.charAt(i) - 'a'];
        }

        int[] cnt2 = new int[26];
        for (int i = 0; i < n - 1; ++i) {
            ++cnt2[s.charAt(i) - 'a'];
        }

        for (int i = n - 1; i < m; ++i) {
            ++cnt2[s.charAt(i) - 'a'];
            if (Arrays.equals(cnt1, cnt2)) {
                ans.add(i - m + 1);
            }
            --cnt2[s.charAt(i - m + 1) - 'a'];
        }
        return ans;
    }
}

Java

```

Sliding Window - LeetCode - By Pattern

— 259 —

LeetCode

```

public class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int m = s.length(), n = p.length();
        List<Integer> ans = new ArrayList<>();
        if (m < n)
            return ans;

        int[] cnt1 = new int[26];
        for (int i = 0; i < m; ++i) {
            ++cnt1[s.charAt(i) - 'a'];
        }

        int[] cnt2 = new int[26];
        for (int i = 0; i < n - 1; ++i) {
            ++cnt2[s.charAt(i) - 'a'];
        }

        for (int i = n - 1; i < m; ++i) {
            ++cnt2[s.charAt(i) - 'a'];
            if (Arrays.equals(cnt1, cnt2)) {
                ans.add(i - m + 1);
            }
            --cnt2[s.charAt(i - m + 1) - 'a'];
        }
        return ans;
    }
}

Java
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int m = s.length(), n = p.length();
        List<Integer> ans = new ArrayList<>();
        if (m < n)
            return ans;

        int[] cnt1 = new int[26];
        for (int i = 0; i < m; ++i) {
            ++cnt1[s.charAt(i) - 'a'];
        }

        int[] cnt2 = new int[26];
        for (int i = 0; i < n - 1; ++i) {
            ++cnt2[s.charAt(i) - 'a'];
        }

        for (int i = n - 1; i < m; ++i) {
            ++cnt2[s.charAt(i) - 'a'];
            if (Arrays.equals(cnt1, cnt2)) {
                ans.add(i - m + 1);
            }
            --cnt2[s.charAt(i - m + 1) - 'a'];
        }
        return ans;
    }
}

Java

```

Sliding Window - LeetCode - By Pattern

— 260 —

LeetCode

```

public class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int m = s.length(), n = p.length();
        List<Integer> ans = new ArrayList<>();
        if (m < n)
            return ans;

        int[] cnt1 = new int[26];
        int[] cnt2 = new int[26];
        for (int i = 0; i < m; ++i) {
            ++cnt1[s.charAt(i) - 'a'];
        }

        for (int i = 0, j = 0; i < m; ++i) {
            ++cnt2[i];
            while (cnt2[i] > cnt1[i]) {
                --cnt2[i];
            }
            if (i - j + 1 == n) {
                ans.add(j);
            }
        }
        return ans;
    }
}

TypeScript
function findAnagrams(s: string, p: string): number[] {
    const m = s.length;
    const n = p.length;
    const ans: number[] = [];
    if (m < n)
        return ans;

    const cnt1: number[] = new Array(26).fill(0);
    const cnt2: number[] = new Array(26).fill(0);
    const idx = (c: string) => c.charCodeAt(0) - 'a'.charCodeAt(0);
    for (const c of p) {
        ++cnt1[idx(c)];
    }

    for (const c of s.slice(0, m - 1)) {
        ++cnt2[idx(c)];
    }

    for (let i = m - 1; i < m; ++i) {
        ++cnt2[idx(s[i])];
        if (cnt1.toString() === cnt2.toString()) {
            ans.push(i - m + 1);
        }
        --cnt2[idx(s[i] - m + 1)];
    }
    return ans;
}

TypeScript

```

Sliding Window - LeetCode - By Pattern

— 261 —

LeetCode

```

function findAnagrams(s: string, p: string): number[] {
    const m = s.length;
    const n = p.length;
    const ans: number[] = [];
    if (m < n)
        return ans;

    const cnt1: number[] = new Array(26).fill(0);
    const cnt2: number[] = new Array(26).fill(0);
    const idx = (c: string) => c.charCodeAt(0) - 'a'.charCodeAt(0);
    for (const c of p) {
        ++cnt1[idx(c)];
    }

    for (const c of s.slice(0, m - 1)) {
        ++cnt2[idx(c)];
    }

    for (let i = m - 1; i < m; ++i) {
        ++cnt2[idx(s[i])];
        if (cnt1.toString() === cnt2.toString()) {
            ans.push(i - m + 1);
        }
        --cnt2[idx(s[i] - m + 1)];
    }
    return ans;
}

Go
func findAnagrams(s string, p string) (ans []int) {
    m, n := len(s), len(p)
    if m < n {
        return
    }
    cnt1 := [26]int{}
    cnt2 := [26]int{}
    for _, c := range p {
        cnt1[c-'a']++
    }
    for i, c := range s[:m-1] {
        cnt2[c-'a']++
    }
    for i := m - 1; i < m; i++ {
        cnt2[s[i]-'a']++
        if cnt1 == cnt2 {
            ans = append(ans, i-m+1)
        }
        cnt2[s[i-m+1]-'a']--
    }
    return
}

Go

```

Sliding Window - LeetCode - By Pattern

— 262 —

LeetCode

```

func findAnagrams(s string, p string) (ans []int) {
    m, n := len(s), len(p)
    if m < n {
        return
    }
    cnt1 := [26]int{}
    cnt2 := [26]int{}
    for _, c := range p {
        cnt1[c-'a']++
    }
    for _, c := range s[:m-1] {
        cnt2[c-'a']++
    }
    for i := m - 1; i < m; i++ {
        cnt2[s[i]-'a']++
        if cnt1 == cnt2 {
            ans = append(ans, i-m+1)
        }
        cnt2[s[i-m+1]-'a']--
    }
    return
}

Rust
impl Solution {
    pub fn find_anagrams(s: String, p: String) -> Vec<i32> {
        let (s, p) = (s.as_bytes(), p.as_bytes());
        let (m, n) = (s.len(), p.len());
        let mut ans = vec![];
        if m < n {
            return ans;
        }
        let mut cnt = [0; 26];
        for i in 0..n {
            cnt[(s[i] - b'a') as usize] += 1;
            cnt[(p[i] - b'a') as usize] += 1;
        }
        for i in n..m {
            if cnt.iter().all(|&v| *v == 0) {
                ans.push(i - n as i32);
            }
            cnt[(s[i] - b'a') as usize] += 1;
            cnt[(p[i] - b'a') as usize] += 1;
        }
        if cnt.iter().all(|&v| *v == 0) {
            ans.push(m - n as i32);
        }
        ans
    }
}

Python

```

[*] Sliding Window - Pattern Solution (stored optimal)

Python

Sliding Window - LeetCode - By Pattern

— 263 —

LeetCode

```

# Define the function to find all anagram indices in the string
def findAnagrams(s: str, p: str):
    # Import Counter for frequency counting
    from collections import Counter
    # List to store the starting indices of all anagrams
    result = []
    # Length of string s
    n = len(s)
    # Length of string p
    m = len(p)
    # Frequency count for characters in s
    s_count = Counter(s)
    # Frequency count for the current window in s
    window_count = Counter()
    # Iterate over the string s
    for i in range(n-m+1):
        # Add current character to the window count
        window_count[s[i]] += 1
        # Remove the default character from window if window size exceeds p.length
        if i > m:
            window_count[s[i-m]] -= 1
            if s_count[s[i-m]] == 1:
                del s_count[s[i-m]]
            # Remove key completely if count becomes 0
            elif s_count[s[i-m]] == 0:
                del s_count[s[i-m]]
        # Compare window count with p_count when window size is equal to p.length
        if s_count == window_count:
            result.append(i)
    return result

# Example usage:
print(findAnagrams("cbaebabacd", "abc")) # Expected output: [0, 6]

```

C++

Sliding Window - LeetCode - By Pattern

— 264 —

LeetCode


```
// C++ solution for finding all anagrams start indices in a string
#include <vector>
#include <string>
#include <unordered_map>
using namespace std;

class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        vector<int> result;
        int length = s.size(), plength = p.size();
        if (length < plength) return result;

        // Frequency count using arrays for 26 lowercase letters
        int sCount[26] = {0}, pCount[26] = {0};

        // Build initial frequency counts for p and the first window in s
        for (int i = 0; i < plength; i++) {
            pCount[i] = 'a' - 'j';
            sCount[i] = 'a' - 'j';
        }

        // Compare frequency arrays. If they match, record index s
        auto matches = [&i]() -> bool {
            for (int i = 0; i < 26; i++)
                if (sCount[i] != pCount[i])
                    return false;
            return true;
        };

        if (matches())
            result.push_back(0);

        // Slide the window across s
        for (int i = plength; i < length; i++) {
            sCount[i] = 'a' - 'j'; // Add new character
            sCount[i - plength] = 'a' - 'j'; // Remove old character
            if (matches())
                result.push_back(i - plength + 1);
        }

        return result;
    }
};

// Example usage
// int main() {
//     Solution sol;
//     vector<int> indices = sol.findAnagrams("cbaebabacd", "abc");
//     // Expected output: [0, 6]
// }

// return 0;
// }
```

#487 MEDIUM Max Consecutive Ones II

Sliding Window · LeetCode - By Pattern

— 265 —

LeetCode

```
# Function to find the maximum consecutive ones with at most one zero flip
def findMaxConsecutiveOnes(nums):
    max_length = 0 # Stores the maximum length of the window
    left = 0 # Left pointer for the sliding window
    zero_count = 0 # Counter for zeros in the current window

    # Iterate over the entire array
    for right in range(len(nums)):
        # If current element is zero, increase the zero count
        if nums[right] == 0:
            zero_count += 1

        # If window length (more than one zero), shrink from left
        while zero_count > 1:
            if nums[left] == 0:
                zero_count -= 1
            left += 1

        # Update max_length if current window is larger
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage
# nums = [1,0,1,1,0,1]
# print(findMaxConsecutiveOnes(nums)) # Expected output: 4
```

```
Python
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        ans = 0
        zeros = 0

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                zeros += 1
                while zeros == 2:
                    if nums[l] == 0:
                        zeros -= 1
                    l += 1
                ans = max(ans, r - l + 1)
            else:
                ans = max(ans, r - l + 1)

        return ans

Python
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        l = 0
        r = 0
        for x in nums:
            if x == 1:
                r += 1
            else:
                l = r
                r = r + 1
            ans = max(r - l + 1, ans)
        return ans

Python
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        l = 0
        r = 0
        for x in nums:
            if x == 1:
                r += 1
            else:
                l = r
                r = r + 1
            ans = max(r - l + 1, ans)
        return ans
```

Sliding Window · LeetCode - By Pattern

— 267 —

LeetCode

```
max_count = max(max_count, count)

# Reset the count and prev_count if we encounter two zeros in a row
if num == 0 and prev_count == 0:
    count = 0
    prev_count = 0

return max_count

#####

class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        l = 0
        r = 0
        k = 1
        while r < len(nums):
            if nums[r] == 0:
                k -= 1
            if k > 0:
                if nums[r] == 0:
                    k += 1
                    l = r
                else:
                    r += 1
            else:
                return r - l

#####

class Solution(object):
    def __init__(self):
        self.ans = 0

C++
class Solution {
public:
    int findMaxConsecutiveOnes(vector<int>& nums) {
        int ans = 0;
        int zeros = 0;

        for (int i = 0; i < nums.size(); i++) {
            if (nums[i] == 0)
                zeros++;
            while (zeros == 2)
                if (nums[i] == 0)
                    zeros++;
            ans = max(ans, i - l + 1);
        }

        return ans;
    }
};

C++
```

Sliding Window · LeetCode - By Pattern

— 269 —

LeetCode

LeetCode · SimplyList · WalkCC · LeetCode · Editorial
Array · Dynamic Programming · Sliding Window
LeetCode Problem 487

Problem:
Given a binary array `nums`, return the maximum number of consecutive 1's in the array if you can flip at most one 0.

Example 1:
Input: `nums = [1,0,1,1,0]`
Output: `4`
Explanation:
- If we flip the first zero, `nums` becomes `[1,1,1,1,0]` and we have 4 consecutive ones.
- If we flip the second zero, `nums` becomes `[1,0,1,1,1]` and we have 3 consecutive ones.
The max number of consecutive ones is 4.

Example 2:
Input: `nums = [1,0,1,1,0,1]`
Output: `5`
Explanation:
- If we flip the first zero, `nums` becomes `[1,1,1,1,0,1]` and we have 4 consecutive ones.
- If we flip the second zero, `nums` becomes `[1,0,1,1,1,1]` and we have 5 consecutive ones.
The max number of consecutive ones is 5.

Constraints:
- `1 <= nums.length <= 10^5`
- `nums[i]` is either 0 or 1.

Follow up: What if the input numbers come in one by one as an infinite stream? In other words, you can't store all numbers coming from the stream as it's too large to hold in memory. Could you solve it efficiently?

>> Bruteforce [Sliding Window] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def findMaxConsecutiveOnes(self, nums):
        # Brute Force (O(n^2)) - Iterate over all windows
        n = len(nums)
        best = 0
        for l in range(n):
            window_state = 0
            for r in range(l, n):
                window_state += nums[r]
                best = max(best, r - l + 1)
            return best

Python
Python
```

Sliding Window · LeetCode - By Pattern

— 266 —

LeetCode

```
from collections import deque

class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        res, zero, left, k = 0, 0, 0, 1

        for right in range(len(nums)):
            if nums[right] == 0:
                zero += 1
                while zero > k:
                    if nums[left] == 0:
                        zero -= 1
                    left += 1
                res = max(res, right - left + 1) # max-check every for iteration

        return res

class Solution: # Follows, extra space
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        res, left, k = 0, 0, 1
        q = deque()

        for right in range(len(nums)):
            if nums[right] == 0:
                q.append(right)
            if len(q) > k:
                left = q.popleft() + 1
            res = max(res, right - left + 1) # max-check every for iteration

        return res

#####

# maintain a sliding window of size 2, that keeps track of the counts.
class Solution: # Follow up, optimized
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        count = 0 # Count of consecutive ones
        prev_count = 0 # Count of consecutive ones before a zero
        max_count = 0 # Maximum count of consecutive ones

        for num in nums:
            if num == 1:
                count += 1
                prev_count = count
            else:
                count = prev_count + 1
                prev_count = 0
```

Sliding Window · LeetCode - By Pattern

— 268 —

LeetCode

```
class Solution {
public:
    int findMaxConsecutiveOnes(vector<int>& nums) {
        int l = 0, cnt = 0;
        for (int i = 0; i < nums.size(); i++) {
            if (cnt > 1) {
                cnt = nums[i] - 1;
            }
            cnt = nums[i] + 1;
            return nums.size() - 1;
        }
    };
};

C++
class Solution {
public:
    int findMaxConsecutiveOnes(vector<int>& nums) {
        int l = 0, r = 0;
        while (r < nums.size()) {
            if (nums[r] == 0) {
                if (nums[r+1] == 0) {
                    r++;
                } else {
                    l = r;
                    r++;
                }
            } else {
                r++;
            }
        }
        return r - l;
    }
};

Java
class Solution {
public:
    int findMaxConsecutiveOnes(vector<int>& nums) {
        int ans = 0;
        int zeros = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 0)
                zeros++;
            while (zeros == 2)
                if (nums[i] == 0)
                    zeros++;
            ans = Math.max(ans, i - l + 1);
        }

        return ans;
    }
};

Java
```

Sliding Window · LeetCode - By Pattern

— 270 —

LeetCode

```

class Solution {
public: int findMaxConsecutiveOnes(int[] nums) {
    int left = 0, right = 1;
    int ans = 0;

    // Every instance of zero
    Queue-Integer q = new ArrayDeque<>();

    for (int l = 0, r = 0; r < nums.length; ++r) {
        if (nums[r] == 0)
            q.offer(r);
        if (q.size() > nums.length)
            l = q.poll() + 1;
        ans = Math.max(ans, r - l + 1);
    }

    return ans;
}
}

```

Java

```

class Solution {
public: int findMaxConsecutiveOnes(int[] nums) {
    int l = 0, r = 0;
    for (int k = 0; k < nums.length; ++k) {
        int s = 0;
        if (nums[k] == 0)
            s = k + 1;
        else
            s = s + 1;
        ans = Math.max(ans, s);
    }

    return ans;
}
}

```

Java

```

class Solution {
public: int findMaxConsecutiveOnes(int[] nums) {
    int l = 0, r = 0;
    int k = 1;
    while (r < nums.length) {
        if (nums[r] == 0) {
            --k;
            if (k < 0 && nums[l] == 0) {
                ++k;
                l = r;
            }
            r = r + 1;
        }
    }

    return r - l;
}
}

```

JavaScript

JavaScript

Sliding Window - LeetCode - By Pattern

— 271 —

LeetCode

```

//
// @param {number[]} nums
// @return {number}
//
var findMaxConsecutiveOnes = function (nums) {
    let l, r = 0, 0;
    for (const s of nums) {
        if (s === 1)
            r = r + 1;
        else
            l = r + 1;
    }

    return r - l + 1;
};

```

JavaScript

```

//
// @param {number[]} nums
// @return {number}
//
var findMaxConsecutiveOnes = function (nums) {
    let l, r = 0, 0;
    for (const s of nums) {
        if (s === 1)
            r = r + 1;
        else
            l = r + 1;
    }

    return r - l + 1;
};

```

TypeScript

TypeScript

```

function findMaxConsecutiveOnes(nums: number[]): number {
    let l, r = 0, 0;
    for (const s of nums) {
        if (s === 1)
            r = r + 1;
        else
            l = r + 1;
    }

    return r - l + 1;
}

```

TypeScript

```

function findMaxConsecutiveOnes(nums: number[]): number {
    let l, r = 0, 0;
    for (const s of nums) {
        if (s === 1)
            r = r + 1;
        else
            l = r + 1;
    }

    return r - l + 1;
}

```

Sliding Window - LeetCode - By Pattern

— 272 —

LeetCode

Go

Go

```

func findMaxConsecutiveOnes(nums []int) int {
    l, r := 0, 0
    for r < len(nums) {
        if nums[r] == 0 {
            l = r
            r = r + 1
        } else {
            r = r + 1
        }
    }

    return r - l
}

```

Go

```

func findMaxConsecutiveOnes(nums []int) int {
    l, r := 0, 0
    k := 1
    for r < len(nums) {
        if nums[r] == 0 {
            k = k - 1
            if k < 0 {
                l = r
                k = 0
            }
            r = r + 1
        } else {
            r = r + 1
        }
    }

    return r - l
}

```

[*] Sliding Window - Pattern Solution (stored optimal)

Python

Sliding Window - LeetCode - By Pattern

— 273 —

LeetCode

```

# Function to find the maximum consecutive ones with at most one zero flip
def findMaxConsecutiveOnes(nums):
    # max_length = 0 # Stores the maximum length of the window
    left = 0 # Left pointer for the sliding window
    zero_count = 0 # Counter for zeros in the current window

    # Iterate with right pointer over the entire array
    for right in range(len(nums)):
        # If current element is zero, increase the zero_count
        if nums[right] == 0:
            zero_count += 1

        # If window invalid (more than one zero), shrink from left
        while zero_count > 1:
            if nums[left] == 0:
                zero_count -= 1
            left += 1

        # Update max_length if current window is larger
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# nums = [1,0,1,1,0]
# print(findMaxConsecutiveOnes(nums)) # Expected output: 4

```

C++

```

// C++ solution using the sliding window technique
#include <vector>
#include <algorithm>
using namespace std;

class Solution {
public:
    int findMaxConsecutiveOnes(vector<int>& nums) {
        int max_length = 0; // Maximum consecutive ones length
        int left = 0; // Left pointer for the window
        int zero_count = 0; // Count of zeros in the current window

        // Iterate with the right pointer
        for (int right = 0; right < nums.size(); right++) {
            if (nums[right] == 0)
                zero_count++;

            // Increase zero count if element is zero

            // Contract the window from the left if there are more than one zero
            while (zero_count > 1) {
                if (nums[left] == 0)
                    zero_count--;
                left++;
            }

            // Update the maximum window length
            max_length = max(max_length, right - left + 1);
        }
    }
}

```

Sliding Window - LeetCode - By Pattern

— 274 —

LeetCode

```

    }
    return maxlen;
}

// Example usage:
// int main() {
//     vector<int> nums = {1,0,1,1,0};
//     Solution sol;
//     int result = sol.findMaxConsecutiveOnes(nums); // Expected output: 4
//     return 0;
// }

```

#1100 MEDIUM

Find K-Length Substrings with No Repeated Characters

LeetCode - SimplyEasy - WalkCC - LeetCode - Editorial

Hash Table - String - Sliding Window

Like-Premium 98

Problem:

Given a string s and an integer k, return the number of substrings in s of length k with no repeated characters.

Example 1:

Input: s = "havefunleetcode", k = 5

Output: 6

Explanation: There are 6 substrings they are: "havef", "avefu", "vefun", "efunc", "funcd", "tcode".

Example 2:

Input: s = "home", k = 5

Output: 0

Explanation: Notice k can be larger than the length of s. In this case, it is not possible to find any substring.

Constraints:

- 1 <= s.length <= 104
- s consists of lowercase English letters.
- 1 <= k <= 104

Python

Python

Sliding Window - LeetCode - By Pattern

— 275 —

LeetCode

```

def numKLenSubstrNoRepeats(s: str, k: int) -> int:
    # If k is longer than the string length, no valid substrings exist.
    if k > len(s):
        return 0

    count = 0
    left = 0
    # Dictionary to store the frequency of characters in the current window.
    char_count = {}

    # Iterate over the string using right pointer.
    for right in range(len(s)):
        # Update frequency count for the current character.
        char_count[s[right]] = char_count.get(s[right], 0) + 1

        # If current character frequency > 1, slide window from the left to remove duplicates.
        while char_count[s[right]] > 1:
            char_count[s[left]] -= 1
            left += 1

        # When window size equals k, we have a valid substring.
        if right - left + 1 == k:
            count += 1

        # Slide the window by removing the leftmost character.
        char_count[s[left]] -= 1
        left += 1

    return count

```

Example usage:

print(numKLenSubstrNoRepeats("havefunleetcode", 5)) -> 6

Python

```

class Solution:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        ans = 0
        unique = {}
        count = collections.Counter()

        for i, c in enumerate(s):
            count[c] += 1
            if count[c] > 1:
                unique[c] = 1
                if i >= k:
                    count[s[i - k]] -= 1
                    if count[s[i - k]] == 0:
                        unique[s[i - k]] = 0
                    if unique[c] > 1:
                        ans += 1
            return ans

```

Python

Sliding Window - LeetCode - By Pattern

— 276 —

LeetCode

```

class Solution {
public:
    int numKLenSubstrNoRepeat(string s, int k) {
        int ans = 0;
        for (int i = 0; i < s.length(); i++) {
            unordered_map<char, int> cnt;
            for (int j = i; j < i + k; j++) {
                cnt[s[j]]++;
            }
            if (cnt.size() == k) {
                ans++;
            }
        }
        return ans;
    }
};

```

Python

```

class Solution:
    def numKLenSubstrNoRepeat(self, s: str, k: int) -> int:
        n = len(s)
        if k > n or k <= 0:
            return 0
        ans = 0
        cnt = Counter()
        for i, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1 or i - j + 1 > k:
                cnt[s[i-j]] -= 1
                j += 1
            ans += i - j + 1 == k
        return ans

```

C++

C++

```

class Solution {
public:
    int numKLenSubstrNoRepeat(string s, int k) {
        int ans = 0;
        int unique = 0;
        vector<int> count(26);
        for (int i = 0; i < s.length(); i++) {
            if (count[s[i] - 'a'] == 1) {
                unique++;
                if (i == k) {
                    count[s[i - k] - 'a'] = 0;
                }
            }
            count[s[i] - 'a'] = 1;
            if (unique == k) {
                ans++;
            }
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int numKLenSubstrNoRepeat(string s, int k) {
        int n = s.size();
        if (k > n) {
            return 0;
        }
        unordered_map<char, int> cnt;
        for (int i = 0; i < k; i++) {
            cnt[s[i]]++;
        }
        int ans = cnt.size() == k;
        for (int i = k; i < n; i++) {
            cnt[s[i]]++;
            if (cnt[s[i - k]] == 1) {
                cnt.erase(s[i - k]);
            }
            ans = cnt.size() == k;
        }
        return ans;
    }
};

```

C++

```

class Solution {
//
// @param String s
// @param Integer k
// @return Integer
//
function numKLenSubstrNoRepeat($s, $k) {
    $n = strlen($s);
    if ($n < $k) {
        return 0;
    }
    $cnt = [];
    for ($i = 0; $i < $k; $i++) {
        if (!isset($cnt[$s[$i]])) {
            $cnt[$s[$i]] = 1;
        } else {
            $cnt[$s[$i]]++;
        }
    }
    $ans = count($cnt) == $k ? 1 : 0;
    for ($i = $k; $i < $n; $i++) {
        if (!isset($cnt[$s[$i]])) {
            $cnt[$s[$i]] = 1;
        } else {
            $cnt[$s[$i]]++;
        }
        $cnt[$s[$i - $k]]--;
        if ($cnt[$s[$i - $k]] == 0) {
            unset($cnt[$s[$i - $k]]);
        }
        $ans = count($cnt) == $k ? 1 : 0;
    }
    return $ans;
}

```

C++

```

}
if (cnt[s[i] - 'a'] == 1) {
    count[s[i] - 'a']++;
} else {
    count[s[i] - 'a'] = 1;
}
ans = count == k ? 1 : 0;
return ans;
}
}

```

C++

```

class Solution {
//
// @param String s
// @param Integer k
// @return Integer
//
function numKLenSubstrNoRepeat($s, $k) {
    $n = (strlen($s) - 1) / 2 - $k;
    $cnt = [];
    for ($i = 0; $i < $n; $i++) {
        $str = substr($s, $i, $k);
        for ($j = 0; $j < $k; $j++) {
            $cnt[$str[$j]]++;
        }
        if (count($cnt) == $k) {
            $ans++;
        }
        $str = substr($s, $i + 1, $k);
        for ($j = 0; $j < $k; $j++) {
            $cnt[$str[$j]]++;
        }
        if (count($cnt) == $k) {
            $ans++;
        }
    }
    return $ans;
}
}

```

Java

```

class Solution {
    public int numKLenSubstrNoRepeat(String s, int k) {
        int ans = 0;
        int unique = 0;
        int[] count = new int[26];
        for (int i = 0; i < s.length(); i++) {
            if (count[s.charAt(i) - 'a'] == 1) {
                unique++;
                if (i == k) {
                    count[s.charAt(i - k) - 'a'] = 0;
                }
            }
            count[s.charAt(i) - 'a'] = 1;
            if (unique == k) {
                ans++;
            }
        }
        return ans;
    }
}

```

```

class Solution {
    public int numKLenSubstrNoRepeat(String s, int k) {
        int n = s.length();
        if (k > n) {
            return 0;
        }
        Map<Character, Integer> cnt = new HashMap<>();
        for (int i = 0; i < k; i++) {
            cnt.put(s.charAt(i), 1);
        }
        int ans = cnt.size() == k ? 1 : 0;
        for (int i = k; i < n; i++) {
            cnt.put(s.charAt(i), 1);
            if (cnt.get(s.charAt(i - k)) == 1) {
                cnt.remove(s.charAt(i - k));
            }
            ans = cnt.size() == k ? 1 : 0;
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int numKLenSubstrNoRepeat(String s, int k) {
        int n = s.length();
        if (k > n || k <= 0) {
            return 0;
        }
        int[] cnt = new int[26];
        int ans = 0;
        for (int i = 0; i < k; i++) {
            cnt[s.charAt(i)]++;
            while (cnt[s.charAt(i)] > 1) {
                cnt[s.charAt(i)]--;
            }
        }
        ans = cnt.size() == k ? 1 : 0;
        for (int i = k; i < n; i++) {
            cnt[s.charAt(i)]++;
            while (cnt[s.charAt(i)] > 1) {
                cnt[s.charAt(i)]--;
            }
            ans = cnt.size() == k ? 1 : 0;
        }
        return ans;
    }
}

```

TypeScript

TypeScript

```

function numKLenSubstrNoRepeat(s: string, k: number): number {
    const n = s.length;
    if (k > n) {
        return 0;
    }
    const cnt = Map<string, number> = new Map();
    for (let i = 0; i < k; i++) {
        cnt.set(s[i], (cnt.get(s[i]) ?? 0) + 1);
    }
    let ans = cnt.size === k ? 1 : 0;
    for (let i = k; i < n; i++) {
        cnt.set(s[i], (cnt.get(s[i]) ?? 0) + 1);
        cnt.set(s[i - k], (cnt.get(s[i - k]) ?? 0) - 1);
        if (cnt.get(s[i - k]) === 0) {
            cnt.delete(s[i - k]);
        }
        ans = cnt.size === k ? 1 : 0;
    }
    return ans;
}

```

TypeScript

```

function numKLenSubstrNoRepeat(s: string, k: number): number {
    const n = s.length;
    if (k > n) {
        return 0;
    }
    const cnt = Map<string, number> = new Map();
    for (let i = 0; i < k; i++) {
        cnt.set(s[i], (cnt.get(s[i]) ?? 0) + 1);
    }
    let ans = cnt.size === k ? 1 : 0;
    for (let i = k; i < n; i++) {
        cnt.set(s[i], (cnt.get(s[i]) ?? 0) + 1);
        cnt.set(s[i - k], (cnt.get(s[i - k]) ?? 0) - 1);
        if (cnt.get(s[i - k]) === 0) {
            cnt.delete(s[i - k]);
        }
        ans = cnt.size === k ? 1 : 0;
    }
    return ans;
}

```

Go

Go

```

func numKLenSubstrNoRepeat(s string, k int) (ans int) {
    n := len(s)
    if n < k {
        return
    }
    cnt := map[byte]int{}
    for i := 0; i < k; i++ {
        cnt[s[i]]++
    }
    if len(cnt) == k {
        ans++
    }
    for i := k; i < n; i++ {
        cnt[s[i]]++
        cnt[s[i - k]]--
        if cnt[s[i - k]] == 0 {
            delete(cnt, s[i - k])
        }
        if len(cnt) == k {
            ans++
        }
    }
    return
}

```

[1] Sliding Window - Pattern Solution (stored optimal)

Python

```

def numKLenSubstrNoRepeat(s: str, k: int) -> int:
    # If k is larger than the string length, no valid substring exists.
    if k > len(s):
        return 0

    count = 0
    # Dictionary to store the frequency of characters in the current window.
    char_count = {}

    # Iterate over the string using right pointer.
    for right in range(len(s)):
        # Update frequency count for the current character.
        char_count[right] = char_count.get(right, 0) + 1

        # If current character frequency > 1, slide window from the left to remove duplicates.
        while char_count[right] > 1:
            char_count[right] -= 1
            left += 1

        # When window size equals k, we have a valid substring.
        if right - left + 1 == k:
            count += 1

        # Slide the window by removing the leftmost character.
        char_count[left] -= 1

```

```

        left += 1
        return count

# Example usage:
# print(numberOfSubarrays(nums, 3))  # 3

C++
// C++ solution using sliding window with a frequency array for lowercase letters.
#include <string>
#include <vector>
using namespace std;

class Solution {
public:
    int numSubarraysWithSum(string s, int k) {
        int n = s.size();
        if (k > n) return 0;

        int count = 0;
        // Frequency array for 26 lowercase letters.
        vector<int> freq(26, 0);
        int left = 0;

        // Iterate through the string using right pointer.
        for (int right = 0; right < s.size(); right++) {
            // Increment frequency of current character.
            freq[s[right] - 'a']++;

            // If duplicate found, slide window until duplicate is removed.
            while (freq[s[right] - 'a'] > 1) {
                freq[s[left] - 'a']--;
                left++;
            }

            // When window size is k, count valid subarray and slide window.
            if (right - left + 1 == k) {
                count++;
                freq[s[left] - 'a']--;
                left++;
            }
        }
        return count;
    }
};

```

#1248 **MEDIUM**
Count Number of Nice Subarrays
[LeetCode](#) · [SimplyList](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)
[Array](#) · [Hash Table](#) · [Math](#) · [Sliding Window](#) · [Prefix Sum](#)
[Like](#) · [Unfollow](#)

Problem:

Given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if there are `k` odd numbers on it.

Return the number of nice sub-arrays.

Example 1:

Input: `nums = [1,1,2,1,1]`, `k = 3`
Output: 2
Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.

Example 2:

Input: `nums = [2,4,6]`, `k = 1`
Output: 0
Explanation: There are no odd numbers in the array.

Example 3:

Input: `nums = [2,2,2,1,2,2,1,2,2,2]`, `k = 2`
Output: 16

Constraints:

- `1 <= nums.length <= 50000`
- `1 <= nums[i] <= 10^5`
- `1 <= k <= nums.length`

>> Brute Force [Sliding Window] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        # Brute Force O(n^2) - enumerate all windows
        n = len(nums)
        best = 0
        for i in range(n):
            window_size = 0
            for j in range(i, n):
                window_size += nums[j] % 2
                best = max(best, j - i + 1)
            return best

```

Python

```

# Python solution with line-by-line comments
def numberOfSubarrays(nums, k):
    # Initialize Dictionary to store frequency of prefix sums
    prefix_count = {}
    # There is one way to have zero odd numbers initially.
    count = 0
    # This will store the final count of nice subarrays
    odd_sum = 0
    # Prefix sum representing the count of odd numbers so far

    # Iterate through each number in the array
    for num in nums:
        # If the number is odd, increment the odd_count prefix sum
        if num % 2 == 1:
            odd_sum += 1

        # If there is a prefix such that (current odd_sum - prefix) equals k,
        # then we found subarrays ending here with k odd numbers.
        count += prefix_count.get(odd_sum - k, 0)

        # Update the frequency for the current odd_sum in the hash table
        prefix_count[odd_sum] = prefix_count.get(odd_sum, 0) + 1

    # Return the total number of nice subarrays
    return count

# Example usage:
# nums = [1,1,2,1,1]
# k = 3
# print(numberOfSubarrays(nums, k)) # Expected output is 2

Python
class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        ans = 0
        l = 0
        r = 0

        while r < len(nums):
            if k == 0:
                ans += r - l + 1
                if r == len(nums):
                    break
            if nums[r] % 2 == 1:
                k -= 1
                r += 1
            else:
                if nums[l] % 2 == 1:
                    k += 1
                    l += 1
                else:
                    l += 1
            return ans

        return number ofSubarrays(nums, k) - number ofSubarrays(nums, k - 1)

```

```

class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        ans = Counter(0, 1)
        ans = 0
        for v in nums:
            ans += ans[k - v]
            ans[k] += 1
        return ans

Python
class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        ans = Counter(0, 1)
        ans = 0
        for v in nums:
            ans += ans[k - v]
            ans[k] += 1
        return ans

C++
class Solution {
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        return number ofSubarrays(nums, k) -
            number ofSubarrays(nums, k - 1);
    }

private:
    int number ofSubarrays(vector<int>& nums, int k) {
        int ans = 0;

        for (int i = 0; i < nums.size(); i++) {
            if (k == 0) {
                ans += i - 1;
                if (i == nums.size())
                    break;
                if (nums[i] % 2 == 1)
                    ans++;
            } else {
                if (nums[i] % 2 == 1)
                    ans++;
            }
        }
        return ans;
    }
};

```

```

class Solution {
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        int n = nums.size();
        vector<int> cnt(n + 1);
        cnt[0] = 1;
        int ans = 0;
        for (int i = 0; i < n; i++) {
            if (nums[i] % 2 == 1)
                ans += cnt[i - k];
            cnt[i + 1]++;
        }
        return ans;
    }
};

C++
class Solution {
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        int n = nums.size();
        vector<int> cnt(n + 1);
        cnt[0] = 1;
        int ans = 0;
        for (int i = 0; i < n; i++) {
            if (nums[i] % 2 == 1)
                ans += cnt[i - k];
            cnt[i + 1]++;
        }
        return ans;
    }
};

Java
Java

```

```

class Solution {
    public int numberOfSubarrays(int[] nums, int k) {
        return number ofSubarrays(nums, k) - number ofSubarrays(nums, k - 1);
    }

    private int number ofSubarrays(int[] nums, int k) {
        int ans = 0;
        for (int i = 0; i < nums.length; i++) {
            if (k == 0) {
                ans += i - 1;
                if (i == nums.length)
                    break;
                if (nums[i] % 2 == 1)
                    ans++;
            } else {
                if (nums[i] % 2 == 1)
                    ans++;
            }
        }
        return ans;
    }
}

Java
class Solution {
    public int numberOfSubarrays(int[] nums, int k) {
        int n = nums.length;
        int[] cnt = new int[n + 1];
        cnt[0] = 1;
        int ans = 0;
        for (int i = 0; i < n; i++) {
            if (nums[i] % 2 == 1)
                ans += cnt[i - k];
            cnt[i + 1]++;
        }
        return ans;
    }
}

```

```

class Solution {
public: int numberofSubarrays(int[] nums, int k) {
    int n = nums.length;
    int[] cnt = new int[n + 1];
    cnt[0] = 1;
    int ans = 0, t = 0;
    for (int i = 0; i < n; i++) {
        t += nums[i];
        if (t - k == 0) {
            ans += cnt[t - k];
        }
        cnt[t]++;
    }
    return ans;
}
}

```

TypeScript

TypeScript

```

function numberofSubarrays(nums: number[], k: number): number {
    const n = nums.length;
    const cnt = new Array(n + 1).fill(0);
    cnt[0] = 1;
    let [t, ans] = [0, 0];
    for (const v of nums) {
        t += v & 1;
        if (t - k == 0) {
            ans += cnt[t - k];
        }
        cnt[t]++;
    }
    return ans;
}

```

TypeScript

```

function numberofSubarrays(nums: number[], k: number): number {
    const n = nums.length;
    const cnt = new Array(n + 1).fill(0);
    cnt[0] = 1;
    let ans = 0;
    let t = 0;
    for (const v of nums) {
        t += v & 1;
        if (t - k == 0) {
            ans += cnt[t - k];
        }
        cnt[t]++;
    }
    return ans;
}

```

Go

Go

Sliding Window · LeetCode — By Pattern

— 289 —

LeetCode

```

func numberofSubarrays(nums []int, k int) (ans int) {
    n := len(nums)
    cnt := make([]int, n+1)
    cnt[0] = 1
    t := 0
    for i, v := range nums {
        t += v & 1
        if t == k {
            ans += cnt[t-k]
        }
        cnt[t]++
    }
    return
}

```

Go

```

func numberofSubarrays(nums []int, k int) (ans int) {
    n := len(nums)
    cnt := make([]int, n+1)
    cnt[0] = 1
    t := 0
    for i, v := range nums {
        t += v & 1
        if t == k {
            ans += cnt[t-k]
        }
        cnt[t]++
    }
    return
}

```

[1] Sliding Window — Pattern Solution (stored optimal)

Python

Sliding Window · LeetCode — By Pattern

— 290 —

LeetCode

```

# Python solution with line-by-line comments
def numberofSubarrays(nums, k):
    # Initialize Dictionary to store frequency of prefix sums
    prefix_count = {}
    # There is one way to have zero odd numbers initially.
    count = 0 # This will store the final count of nice subarrays
    odd_sum = 0 # Prefix sum representing the count of odd numbers so far
    # Iterate through each number in the array
    for num in nums:
        # If the number is odd, increment the odd_count prefix sum
        if num % 2 == 1:
            odd_sum += 1
        # If there is a prefix such that (current odd_sum - prefix) equals k,
        # then we found subarrays ending here with k odd numbers
        count += prefix_count.get(odd_sum - k, 0)
        # Update the frequency for the current odd_sum in the hash table
        prefix_count[odd_sum] = prefix_count.get(odd_sum, 0) + 1
    # Return the total number of nice subarrays
    return count

# Example usage:
# nums = [1,1,2,1,1]
# k = 1
# print(numberofSubarrays(nums, k)) # Expected output is 2

```

C++

```

// C++ solution with line-by-line comments
#include <iostream>
#include <unordered_map>
using namespace std;

class Solution {
public:
    int numberofSubarrays(vector<int>& nums, int k) {
        // Hash table to store prefix sum frequencies
        unordered_map<int, int> prefixCount;
        // There is one occurrence of a prefix sum of 0 initially
        prefixCount[0] = 1;

        int count = 0; // Total count of nice subarrays
        int oddSum = 0; // Current count of odd numbers

        // Iterate over each number in the array
        for (int num : nums) {
            // If the number is odd, increment the odd count
            if (num % 2 == 1)
                oddSum++;

            // If a prefix sum of (oddSum - k) exists, add its frequency to the count
            if (prefixCount.find(oddSum - k) != prefixCount.end())
                count += prefixCount[oddSum - k];
        }

        return count;
    }
};

```

Sliding Window · LeetCode — By Pattern

— 291 —

LeetCode

```

// Increment frequency of the current oddsum in the hash table
prefixCount[oddSum];
}

// Return the total number of nice subarrays
return count;
}

```

#76

HARD

Minimum Window Substring

LeetCode · Simplest · Hard · LeetCode · Editorial

Hash Table · String · Sliding Window

List: Grind 100

Problem:

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string $""$.

The testcases will be generated such that the answer is unique.

Example 1:

Input: $s = "ADOBECODEBANC"$, $t = "ABC"$
Output: $"BANC"$
Explanation: The minimum window substring $"BANC"$ includes 'A', 'B', and 'C' from string t .

Example 2:

Input: $s = "a"$, $t = "a"$
Output: $"a"$
Explanation: The entire string s is the minimum window.

Example 3:

Input: $s = "a"$, $t = "aa"$
Output: $""$
Explanation: Both 'a's from t must be included in the window. Since the largest window of s only has one 'a', return empty string.

Constraints:

- $m == s.length$
- $n == t.length$
- $1 \leq m, n \leq 105$
- s and t consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in $O(m + n)$ time?

Python

Python

Sliding Window · LeetCode — By Pattern

— 292 —

LeetCode

```

# Python Implementation of Minimum Window Substring solution
def minWindow(s: str, t: str) -> str:
    # If t is longer than s, no solution exists.
    if len(t) > len(s):
        return ""

    # Dictionary to store the frequency of each character in t.
    t_freq = {}
    for char in t:
        t_freq[char] = t_freq.get(char, 0) + 1

    # Dictionary to keep track of the characters in the current window.
    window_freq = {}

    required = len(t_freq) # Unique characters in t to be matched
    formed = 0 # Number of unique characters in the current window that meet required frequency

    # Initialize left, right pointers
    answer = [float('inf'), None, None]

    l = 0 # Left pointer
    # Start sliding the window with the right pointer.
    for r, char in enumerate(s):
        # Add the current character to the window counter.
        window_freq[char] = window_freq.get(char, 0) + 1

        # Check if the current character's frequency in the window matches that in t.
        if char in t_freq and window_freq[char] == t_freq[char]:
            formed += 1

        # Try and contract the window if all characters are matched.
        while l == r and formed == required:
            character = s[l]

            # Update answer if the current window is smaller than the previous best.
            if r - l + 1 < answer[1]:
                answer = (r - l + 1, l, r)

            # The character at the left is going to be removed from the window.
            window_freq[character] -= 1
            if character in t_freq and window_freq[character] < t_freq[character]:
                formed -= 1

            # Move the left pointer ahead.
            l += 1

    # Return the bestLeft window, if found.
    return "" if answer[0] == float('inf') else s[answer[1]:answer[2]+1]

# Example usage:
if __name__ == "__main__":
    s = "ADOBECODEBANC"
    t = "ABC"
    print(minWindow(s, t)) # Expected output: "BANC"

```

Python

Sliding Window · LeetCode — By Pattern

— 293 —

LeetCode

```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        count = collections.Counter(t)
        required = len(t)
        bestLeft = 0
        minLength = len(s) + 1

        l = 0
        for r, c in enumerate(s):
            count[c] -= 1
            if count[c] == 0:
                required -= 1
            while required == 0:
                if r - l + 1 < minLength:
                    bestLeft = l
                    minLength = r - l + 1
                count[s[l]] += 1
                if count[s[l]] > 0:
                    required += 1
                l += 1
            return "" if bestLeft == -1 else s[bestLeft:bestLeft + minLength]

```

Python

```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        need = Counter(t)
        window = Counter()
        cnt = 1
        k, m = -1, 0
        for r, c in enumerate(s):
            window[c] += 1
            if need[c] == window[c]:
                cnt += 1
            while cnt == len(t):
                if r - l + 1 < m:
                    m = r - l + 1
                    k = l
                if need[s[l]] == window[s[l]]:
                    cnt -= 1
                window[s[l]] -= 1
                l += 1
            return "" if k < 0 else s[k : k + m]

```

Python

Sliding Window · LeetCode — By Pattern

— 294 —

LeetCode

```

...
deq = collections.deque()
... deq.append(1)
... deq.append(2)
... deq.append(3)
...
deq[0]
1
...

import collections

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        cnt = 0
        need = collections.Counter(t)
        start, end = len(s), 0 # Arbitrary 0, just make sure end-start is larger than input s
        length = len(s)+1
        d = {}
        # Using memo to store indexes, good for a large amount of API calls
        deq = collections.deque()
        for i, c in enumerate(s):
            if c in need:
                deq.append(i)
                d[c] = d.get(c, 0) + 1
                # " " also ok, because it's increased already one line above
            if d[c] == need[c]:
                cnt += 1
                while deq and d[s[deq[0]]] > need[s[deq[0]]]:
                    d[s[deq.popleft()]] -= 1
                if cnt == len(t) and deq[0] - deq[-1] < end - start:
                    start, end = deq[0], deq[-1]
                return s[start:end + 1]

#####

from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        ans = ""
        m, n = len(s), len(t)
        if n > m:
            return ans
        need = Counter(t)
        window = Counter()
        l, cnt, ml = 0, 0, 0, inf
        for j, c in enumerate(s):
            window[c] += 1
            if need[c] == window[c]: # " " because 1 line above already +=1
                cnt += 1

```

Sliding Window · LeetCode — By Pattern

— 295 —

LeetCode

```

        while cnt == m:
            if j - l + 1 == ml: # is while
                ml = j - l + 1
                ans = s[l : j + 1]
                l = l+1
            if need[c] == window[c]: # char is window but not in need, need[c]>0, window[c]>1...
                cnt -= 1
                window[c] -= 1
                l += 1
            return ans

C++
C++
class Solution {
public:
    string minWindow(string s, string t) {
        vector<int> count(128);
        int required = t.length();
        int bestLeft = -1;
        int minLength = s.length() + 1;
        for (const char c : t)
            ++count[c];
        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (--count[s[r]] == 0)
                --required;
            while (required == 0) {
                if (r - l + 1 < minLength) {
                    bestLeft = l;
                    minLength = r - l + 1;
                }
                if (++count[s[l++]] > 0)
                    ++required;
            }
            return bestLeft == -1 ? "" : s.substr(bestLeft, minLength);
        }
    };
};

Java
Java

```

Sliding Window · LeetCode — By Pattern

— 296 —

LeetCode

```

class Solution {
    public String minWindow(String s, String t) {
        int[] count = new int[128];
        int required = t.length();
        int bestLeft = -1;
        int minLength = t.length() + 1;
        for (final char c : t.toCharArray())
            ++count[c];
        for (int l = 0, r = 0; r < s.length(); ++r) {
            if (--count[s.charAt(r)] == 0)
                --required;
            while (required == 0) {
                if (r - l + 1 < minLength) {
                    bestLeft = l;
                    minLength = r - l + 1;
                }
                if (++count[s.charAt(l++)] > 0)
                    ++required;
            }
            return bestLeft == -1 ? "" : s.substring(bestLeft, bestLeft + minLength);
        }
    }
}

Java
class Solution {
    public String minWindow(String s, String t) {
        int[] need = new int[128];
        int[] window = new int[128];
        int n = s.length(), m = t.length();
        for (int i = 0; i < m; ++i)
            ++need[t.charAt(i)];
        int cnt = 0, j = 0, k = -1, ml = 1 << 30;
        for (int i = 0; i < n; ++i) {
            ++window[s.charAt(i)];
            if (need[s.charAt(i)] == window[s.charAt(i)]) {
                ++cnt;
                while (cnt == m) {
                    if (i - j + 1 < ml) {
                        ml = i - j + 1;
                        k = j;
                    }
                    if (need[s.charAt(j)] == window[s.charAt(j)]) {
                        --cnt;
                    }
                    --window[s.charAt(j++)];
                }
            }
            return k < 0 ? "" : s.substring(k, k + ml);
        }
    }
}

```

Sliding Window · LeetCode — By Pattern

— 297 —

LeetCode

```

Java
public class Solution {
    public String minWindow(String s, String t) {
        int[] need = new int[128];
        int[] window = new int[128];
        for (char c in t) {
            ++need[c];
        }
        int cnt = 0, j = 0, k = -1, ml = 1 << 30;
        for (int i = 0; i < s.length(); ++i) {
            ++window[s[i]];
            if (need[s[i]] == window[s[i]]) {
                ++cnt;
                while (cnt == t.length()) {
                    if (i - j + 1 < ml) {
                        ml = i - j + 1;
                        k = j;
                    }
                    if (need[s[j]] == window[s[j]]) {
                        --cnt;
                    }
                    --window[s[j++]];
                }
            }
            return k < 0 ? "" : s.substring(k, ml);
        }
    }
}

TypeScript
TypeScript
function minWindow(s: string, t: string): string {
    const need: number[] = new Array(128).fill(0);
    const window: number[] = new Array(128).fill(0);
    for (const c of t) {
        ++need[c.charCodeAt(0)];
    }
    let cnt = 0;
    let l = 0;
    let k = -1;
    let ml = 1 << 30;
    for (let i = 0; i < s.length; ++i) {
        ++window[s.charCodeAt(i)];
        if (need[s.charCodeAt(i)] == window[s.charCodeAt(i)]) {
            ++cnt;
        }
        while (cnt === t.length) {
            if (i - j + 1 < ml) {
                ml = i - j + 1;
                k = j;
            }
            if (need[s.charCodeAt(j)] == window[s.charCodeAt(j)]) {
                --cnt;
            }
            --window[s.charCodeAt(j++)];
        }
    }
    return k < 0 ? "" : s.substring(k, k + ml);
}

```

Sliding Window · LeetCode — By Pattern

— 298 —

LeetCode

```

    }
    return k < 0 ? "" : s.slice(k, k + ml);
}

Go
Go
func minWindow(s string, t string) string {
    need := make([]int, 128)
    window := make([]int, 128)
    for i := 0; i < len(t); i++ {
        need[t[i]]++
    }
    m, n := len(s), len(t)
    k, ml, cnt := -1, 0, 0
    for l, r := 0, 0; r < m; r++ {
        c := s[r]
        if window[c] == need[c] {
            cnt++
        }
        for cnt == m {
            if r-l+1 < ml {
                ml = r - l + 1
                k = l
            }
            c = s[l]
            if window[c] == need[c] {
                cnt--
            }
            window[c]++
            l++
        }
        if k < 0 {
            return ""
        }
        return s[k : k+ml]
    }
}

Rust
Rust

```

Sliding Window · LeetCode — By Pattern

— 299 —

LeetCode

```

impl Solution {
    pub fn min_window(s: String, t: String) -> String {
        let mut need, mut window, mut cnt = ([0; 256], [0; 256], 0);
        for c in t.chars() {
            need[c as usize] += 1;
        }
        let (mut l, mut k, mut ml) = (0, -1, 1 << 31);
        for (i, c) in s.chars().enumerate() {
            window[c as usize] += 1;
            if need[c as usize] == window[c as usize] {
                cnt += 1;
            }
            while cnt == t.len() {
                if i - j + 1 < ml {
                    ml = i - j + 1;
                    k = j as i32;
                }
                let l = s.chars().nth(j).unwrap() as usize;
                if need[l] == window[l] {
                    cnt -= 1;
                }
                window[l] -= 1;
                j += 1;
            }
        }
        if k < 0 {
            return "".to_string();
        }
        let k = k as usize;
        s[k..k + ml].to_string()
    }
}

[i] Sliding Window — Pattern Solution (stored optimal)
Python

```

Sliding Window · LeetCode — By Pattern

— 300 —

LeetCode

```
# Python Implementation of Minimum Window Substring solution

def minWindow(s, t):
    # If t is longer than s, no solution exists.
    if len(t) > len(s):
        return ""

    # Dictionary to store the frequency of each character in t.
    t_freq = {}
    for char in t:
        t_freq[char] = t_freq.get(char, 0) + 1

    # Dictionary to keep track of the characters in the current window.
    window_freq = {}

    required = len(t_freq) # Unique characters in t to be matched
    formed = 0 # Number of unique characters in the current window that meet required frequency

    # random.randrange(left, right)
    answer = float('inf') # None, None
    l = 0 # Left pointer
    # Start sliding the window with the right pointer.
    for r, char in enumerate(s):
        # Add the current character to the window counter.
        window_freq[char] = window_freq.get(char, 0) + 1

        # Check if the current character's frequency in the window matches that in t.
        if char in t_freq and window_freq[char] == t_freq[char]:
            formed += 1

        # Try and contract the window if all characters are matched.
        while l <= r and formed == required:
            character = s[l]
            # Update answer if the current window is smaller than the previous best.
            if r - l + 1 < answer[0]:
                answer = (r - l + 1, l, r)
            # The character at the left is going to be removed from the window.
            window_freq[character] -= 1
            if character in t_freq and window_freq[character] < t_freq[character]:
                formed -= 1
            # Move the left pointer ahead.
            l += 1

    # Return the smallest window, if found.
    return "" if answer[0] == float('inf') else s[answer[1]:answer[2]+1]

# Example usage:
if __name__ == "__main__":
    s = "ADOBECODEBANC"
    t = "ABC"
    print(minWindow(s, t)) # Expected output: "BANC"
```

```
// C++ Implementation of Minimum Window Substring solution

#include <iostream>
#include <string>
#include <unordered_map>
#include <climits>
using namespace std;

string minWindow(string s, string t) {
    if (t.size() > s.size()) return "";

    unordered_map<char, int> tFreq;
    for (char c : t)
        tFreq[c]++;

    unordered_map<char, int> windowFreq;
    int required = tFreq.size(); // Number of unique characters required
    int formed = 0; // Number of unique characters that meet the requirement
    int l = 0;
    int minLen = INT_MAX, minLeft = 0;

    for (int r = 0; r < s.size(); r++) {
        char c = s[r];
        windowFreq[c]++;

        // If the frequency of the current character equals the required frequency in tFreq.
        if (tFreq.find(c) != tFreq.end() && windowFreq[c] == tFreq[c]) {
            formed++;
        }

        // Try to contract the window till it ceases to be 'desirable'
        while (l <= r && formed == required) {
            // Update minimum window if current window is smaller.
            if (r - l + 1 < minLen) {
                minLen = r - l + 1;
                minLeft = l;
            }
            char leftChar = s[l];
            windowFreq[leftChar]--;
            if (tFreq.find(leftChar) != tFreq.end() && windowFreq[leftChar] < tFreq[leftChar]) {
                formed--;
            }
            l++;
        }
    }

    return minLen == INT_MAX ? "" : s.substr(minLeft, minLen);
}

int main() {
    string s = "ADOBECODEBANC";
    string t = "ABC";
    cout << minWindow(s, t) << endl; // Expected output: "BANC"
    return 0;
}
```

Quick Review — Sliding Window 9 questions · Insights · Complexity · Solution

#3 Longest Substring Without Repeating Characters [Medium]

- Key Insights**
- Use the sliding window technique to maintain a window of unique characters.
 - Use a hash table (or dictionary) to quickly check for duplicate characters and update window boundaries.
 - The window is expanded by moving a pointer (right) and contracted by moving another pointer (left) when duplicates are encountered.
 - Time complexity can be maintained at O(n) by ensuring each character is processed a limited number of times.

Space & Time
Time Complexity: O(n), where n is the length of the input string.
Space Complexity: O(min(n, m)), where m is the size of the character set (constant for fixed character sets).
Solution
The solution uses a sliding window technique with two pointers, left and right, to keep track of the current substring without repeating characters. A hash table (dictionary or map) is used to store the most recent index of each character encountered. As the right pointer moves forward, if a duplicate character is found within the current window, the left pointer is advanced to the position right after where the duplicate was last seen. This ensures that the window always contains unique characters. The maximum length is updated throughout the process.

#159 Longest Substring with At Most Two Distinct Characters [Medium]

- Key Insights**
- Use a sliding window to maintain a contiguous substring.
 - Keep track of character frequencies within the current window using a hash map or dictionary.
 - When the window contains more than two distinct characters, shrink the window from the left until only two remain.
 - Update the maximum length each time the window satisfies the condition.

Space & Time
Time Complexity: O(n) where n is the length of the string, since each character is processed at most twice.
Space Complexity: O(1) because the hash map holds at most 3 entries (usually up to 2 distinct characters).
Solution
We use the sliding window technique with two pointers (left and right) to represent the current window. A hash table/dictionary records the count of each character inside the window. As we extend the right pointer, we update the count. If the hash table contains more than two distinct characters, we start moving the left pointer to reduce the number of distinct characters until we have at most two distinct keys. The length of the current valid window is then used to update our maximum length answer.

#340 Longest Substring with At Most K Distinct Characters [Medium]

- Key Insights**
- Use the sliding window technique to efficiently scan through the string.
 - Maintain a hash table (or dictionary) that keeps track of the frequency of characters in the current window.
 - When the number of distinct characters exceeds k, increment the left pointer to shrink the window until the condition is met.
 - Update the maximum length of valid substrings throughout the process.

Space & Time
Time Complexity: O(n) where n is the length of s, as each character is visited at most twice.
Space Complexity: O(1) since the frequency tracking is over a fixed set (26 lowercase letters).
Solution
We use a sliding window approach with two pointers: left and right. A hash table (or frequency array) tracks the occurrences of characters in the window. As we move the right pointer, we update the frequency and check for duplicates. If a duplicate is encountered, we increment the left pointer until the duplicate is removed. Once the window size exactly equals k, we have a valid substring, so we increment our count and then slide the window forward. This ensures that every possible k-length substring is examined once.

Time Complexity: O(n), where n is the length of the string, as each character is processed at most twice (once when expanding the window and once when contracting it).
Space Complexity: O(min(n, k)), where m is the size of the charset (or number of unique characters in s), since at most k+1 keys are stored.
Solution
This problem is solved using the sliding window technique. The algorithm keeps track of a window defined by two indices (left and right pointers). A hash map is used to count the frequency of each character in the current window. As the right pointer extends the window, the count of the character is incremented. When the count of distinct keys in the map exceeds k, the left pointer is moved forward while decrementing the count of the leftmost character, until the condition is restored. The maximum window length encountered is recorded and returned as the answer.

#424 Longest Repeating Character Replacement [Medium]

- Key Insights**
- Use a Sliding Window algorithm to keep track of a contiguous substring.
 - Maintain a frequency count of letters in the current window.
 - Keep track of the count of the most frequent character in the window.
 - If the current window size minus the maximum frequency is greater than k, shrink the window.
 - The maximum window size seen during the process is the answer.

Space & Time
Time Complexity: O(n) where n is the length of the string, since the window traverses the string in linear time.
Space Complexity: O(1) because the frequency count for letters (A-Z) has a constant size.
Solution
The problem is solved using the sliding window technique. We initialize two pointers representing the left and right bounds of the current window. As we expand the window by moving the right pointer, we update the frequency count of the letter at the right pointer and track the letter that appears most frequently. The key observation is that the number of characters that need to be replaced in the current window is the window's size minus the count of the most frequently occurring letter. If this number exceeds k, we increment the left pointer to shrink the window until the condition is satisfied again. The maximum size of the window during the process gives the answer.

#438 Find All Anagrams in a String [Medium]

- Key Insights**
- Use a sliding window of length equal to p over s.
 - Maintain frequency counts of characters in p and within the current window in s.
 - Compare frequency counts to determine if the current window is an anagram.
 - Update the window efficiently by adding one character and removing the character that goes out of the window.

Space & Time
Time Complexity: O(n) where n is the length of s.
Space Complexity: O(1) since we use fixed-size frequency arrays or hash maps.
Solution
We use the sliding window technique to iterate over s with a window size equal to the length of p. First, count the frequencies of characters in p. Then, iterate over s and update a frequency count for the current window. When the window size equals the length of p, compare the two frequency counts. If they match, record the start index. To slide the window efficiently, increment the count for the incoming character and decrement the count for the outgoing character. This way, we achieve an O(n) solution that is optimal for the problem constraints.

#487 Max Consecutive Ones II [Medium]

- Key Insights**
- Utilize a sliding window to maintain the current segment.

- The window can contain at most one 0.
- Expand the window with the right pointer and contract it with the left pointer when the constraint is violated.
- The maximum window size observed during this process is the desired result.

Space & Time
Time Complexity: O(n) where n is the number of elements in the array.
Space Complexity: O(1).
Solution
We use a sliding window approach to efficiently find the longest contiguous subarray that contains at most one 0. Two pointers (left and right) define the window. As we iterate with the right pointer, counting zeros in the window, we adjust the left pointer when the count of zeros exceeds one. The length of the valid window gives us the number of consecutive 1's (considering one flipped 0), and we update the maximum length accordingly.

#1100 Find K-Length Substrings with No Repeated Characters [Medium]

- Key Insights**
- Use a sliding window of fixed size k.
 - Track character counts within the current window using a hash table (or frequency array).
 - When a duplicate is found in the window, slide the window from the left to remove duplicates.
 - Count a valid window when its size equals k and all characters are unique.
 - Optimize by adjusting the window immediately after counting a valid substring.

Space & Time
Time Complexity: O(n) where n is the length of s, as each character is visited at most twice.
Space Complexity: O(1) since the frequency tracking is over a fixed set (26 lowercase letters).
Solution
We use a sliding window approach with two pointers: left and right. A hash table (or frequency array) tracks the occurrences of characters in the window. As we move the right pointer, we update the frequency and check for duplicates. If a duplicate is encountered, we increment the left pointer until the duplicate is removed. Once the window size exactly equals k, we have a valid substring, so we increment our count and then slide the window forward. This ensures that every possible k-length substring is examined once.

#1248 Count Number of Nice Subarrays [Medium]

- Key Insights**
- Convert the problem to counting subarrays with a specific 'prefix sum' value where the sum represents the count of odd numbers.
 - Use a hash table (or dictionary) to store the frequency of prefix sums encountered.
 - The number of nice subarrays ending at the current index equals the frequency of (current prefix sum - k) seen so far.
 - This approach allows you to find the answer in one pass over the array.

Space & Time
Time Complexity: O(n), where n is the number of elements in nums, because we traverse the array once.
Space Complexity: O(n), in the worst case where each prefix sum is unique and stored in the hash table.
Solution
We solve the problem using a prefix sum approach. First, we iterate through the array and convert it into a prefix sum representing the cumulative count of odd numbers encountered. As we compute the prefix sums, we use a hash table to record how many times each prefix sum appears. For every current prefix sum value, we check how many times we have seen a prefix sum equal to (current prefix sum - k). The idea is that if (prefix[i] - prefix[j]) equals k, then between indices j+1 and i, there are exactly k odd numbers. This gives us the count of valid subarrays ending at the current index.

This method works efficiently as it requires only a single pass through the array and maintains a running frequency count using the hash table.

#76 Minimum Window Substring (Hard)

Key Insights

- Use a sliding window approach to dynamically adjust the window boundaries.
- Utilize a hash table (or dictionary) to keep track of the frequency of characters required (from t) and the frequency within the current window.
- Expand the window until it satisfies the requirements, then contract to minimize the window.
- Efficiently update counts and check valid window with two pointers (left and right).

Space & Time

Time Complexity: $O(m + n)$, where m is the length of s and n is the length of t .
Space Complexity: $O(m + n)$ in the worst case due to the storage for the hash maps.

Solution

We use a sliding window technique with two pointers representing the window's boundaries. First, create a frequency map of characters from t . Then, expand the right pointer to include characters in the window until the window contains all required characters (meeting or exceeding the counts in t). Once the requirement is met, try contracting the window by moving the left pointer to reduce its size while still maintaining a valid window. Throughout the process, if a valid window is found that is smaller than the previously recorded minimum window, update the minimum. The solution leans heavily on hash tables to track counts and compares them against the required counts from t .

#6 Sorting — 9 questions · #6 of 21

#169

EASY

Majority Element

[LeetCode](#) · [SimpleList](#) · [WALCC](#) · [LeetCode](#) · [Editorial](#)

[Array](#) · [Hash Table](#) · [Divide and Conquer](#) · [Sorting](#) · [Counting](#)

Lists Grid 169

Problem:

Given an array `nums` of size n , return the majority element.

The majority element is the element that appears more than $n / 2$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [1,2,3]`

Output: 1

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5 \times 10^4$
- $-109 \leq \text{nums}[i] \leq 109$
- The input is generated such that a majority element will exist in the array.

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

>> Brute Force [Sorting] Interview Prep

2 Brute Force Approach

```
class Solution:
    def majorityElement(self, nums):
        # Brute Force O(n^2) - for each element count its frequency
        n = len(nums)
        for num in nums:
            if nums.count(num) > n // 2:
                return num
```

Python

Python

```
# Python implementation of the Boyer-Moore Voting Algorithm

def majorityElement(nums):
    # Initialize candidate and counter
    candidate = None
    count = 0

    # Iterate over each number in the input list
    for num in nums:
        # If count is 0, choose the current number as the candidate
        if count == 0:
            candidate = num
            count += 1
        # Increment count if the current number equals the candidate, otherwise decrement
        count += 1 if num == candidate else -1
    # Return the candidate, which is the majority element
    return candidate

# Example usage
print(majorityElement([2,2,1,1,1,2,2])) # Output: 2
```

Python

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        ans = None
        count = 0

        for num in nums:
            if count == 0:
                ans = num
                count += 1
            elif ans == num:
                count += 1
            else:
                count -= 1
        return ans
```

Python

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        cnt = 0
        for x in nums:
            if cnt == 0:
                m, cnt = x, 1
            else:
                cnt += 1 if x == m else -1
        return m
```

Python

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        cnt = 0
        for x in nums:
            if cnt == 0:
                m, cnt = x, 1
            else:
                cnt += 1 if x == m else -1
        return m
```

C++

```
C++
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int ans;
        int count = 0;

        for (const int num : nums) {
            if (count == 0)
                ans = num;
            count += num == ans ? 1 : -1;
        }

        return ans;
    }
};
```

C++

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int cnt = 0, m = 0;
        for (int x : nums) {
            if (cnt == 0) {
                m = x;
                cnt = 1;
            } else {
                cnt += x == m ? 1 : -1;
            }
        }
        return m;
    }
};
```

C++

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int cnt = 0, m = 0;
        for (int x : nums) {
            if (cnt == 0) {
                m = x;
                cnt = 1;
            } else {
                cnt += x == m ? 1 : -1;
            }
        }
        return m;
    }
};
```

C++

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int cnt = 0, m = 0;
        for (int x : nums) {
            if (cnt == 0) {
                m = x;
                cnt = 1;
            } else {
                cnt += x == m ? 1 : -1;
            }
        }
        return m;
    }
};
```

C++

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        int cnt = 0, m = 0;
        for (int x : nums) {
            if (cnt == 0) {
                m = x;
                cnt = 1;
            } else {
                cnt += x == m ? 1 : -1;
            }
        }
        return m;
    }
};
```

Java

```
class Solution {
    public int majorityElement(int[] nums) {
        int ans = null;
        int count = 0;

        for (final int num : nums) {
            if (count == 0)
                ans = num;
            count += num == ans ? 1 : -1;
        }

        return ans;
    }
}
```

Java

```
class Solution {
    public int majorityElement(int[] nums) {
        int cnt = 0, m = 0;
        for (int x : nums) {
            if (cnt == 0) {
                m = x;
                cnt = 1;
            } else {
                cnt += x == m ? 1 : -1;
            }
        }
        return m;
    }
}
```

Java

```
public class Solution {
    public int majorityElement(int[] nums) {
        int cnt = 0, m = 0;
        for (int x : nums) {
            if (cnt == 0) {
                m = x;
                cnt = 1;
            } else {
                cnt += x == m ? 1 : -1;
            }
        }
        return m;
    }
}
```

Java

```
class Solution {
    public int majorityElement(int[] nums) {
        int cnt = 0, m = 0;
        for (int x : nums) {
            if (cnt == 0) {
                m = x;
                cnt = 1;
            } else {
                cnt += x == m ? 1 : -1;
            }
        }
        return m;
    }
}
```

Java

LeetMastery

```

class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        unordered_set<int> seen;

        for (const int num : nums) {
            if (!seen.insert(num).second)
                return true;
            return false;
        }
    }
};

```

```

C++
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        for (int i = 0; i < nums.size() - 1; ++i) {
            if (nums[i] == nums[i + 1]) {
                return true;
            }
        }
        return false;
    }
};

```

```

C++
class Solution {
//
// @param Integer[] nums
// @return Boolean
//
function containsDuplicate(nums) {
    numsUnique = array_unique(nums);
    return count(nums) != count(numsUnique);
}

```

```

C++
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        return s.size() < nums.size();
    }
};

```

C++

Sorting · LeetCode · By Putham

— 319 —

LeetCode

```

class Solution {
//
// @param Integer[] nums
// @return Boolean
//
function containsDuplicate(nums) {
    numsUnique = array_unique(nums);
    return count(nums) != count(numsUnique);
}
}

```

Java

```

Java
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();

        for (final int num : nums) {
            if (!seen.add(num))
                return true;
            return false;
        }
    }
}

```

```

Java
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Arrays.sort(nums);
        for (int i = 0; i < nums.length - 1; ++i) {
            if (nums[i] == nums[i + 1]) {
                return true;
            }
        }
        return false;
    }
}

```

```

Java
public class Solution {
    public boolean containsDuplicate(int[] nums) {
        return new HashSet<>().Count() < nums.length;
    }
}

```

Java

Sorting · LeetCode · By Putham

— 320 —

LeetCode

```

class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> s = new HashSet<>();
        for (int num : nums) {
            if (!s.add(num)) {
                return true;
            }
        }
        return false;
    }
}

```

Java

```

use std::collections::HashSet;
impl Solution {
    pub fn contains_duplicate(nums: Vec<i32>) -> bool {
        nums.iter().collect::HashSet<i32>().len() != nums.len()
    }
}

```

Java

```

class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> s = new HashSet<>();
        for (int num : nums) {
            if (!s.add(num)) {
                return true;
            }
        }
        return false;
    }
}

```

Java

```

public class Solution {
    public boolean ContainsDuplicate(int[] nums) {
        return nums.Distinct().Count() < nums.Length;
    }
}

```

Java

```

use std::collections::HashSet;
impl Solution {
    pub fn contains_duplicate(nums: Vec<i32>) -> bool {
        nums.iter().collect::HashSet<i32>().len() != nums.len()
    }
}

```

JavaScript

JavaScript

Sorting · LeetCode · By Putham

— 321 —

LeetCode

```

//
// @param Number[] nums
// @return Boolean
//
var containsDuplicate = function(nums) {
    return new Set(nums).size !== nums.length;
};

```

JavaScript

```

//
// @param Number[] nums
// @return Boolean
//
var containsDuplicate = function(nums) {
    return new Set(nums).size !== nums.length;
};

```

TypeScript

```

TypeScript
function containsDuplicate(nums: number[]): boolean {
    nums.sort((a, b) => a - b);
    const n = nums.length;
    for (let i = 1; i < n; i++) {
        if (nums[i - 1] === nums[i]) {
            return true;
        }
    }
    return false;
}

```

TypeScript

```

function containsDuplicate(nums: number[]): boolean {
    return new Set(nums).size !== nums.length;
}

```

TypeScript

```

function containsDuplicate(nums: number[]): boolean {
    return new Set(nums).size !== nums.length;
}

```

Go

```

func containsDuplicate(nums []int) bool {
    sort.Ints(nums)
    for i, v := range nums[1:] {
        if v == nums[i] {
            return true
        }
    }
    return false
}

```

Go

Sorting · LeetCode · By Putham

— 322 —

LeetCode

```

func containsDuplicate(nums []int) bool {
    s := map[int]bool{}
    for _, v := range nums {
        if s[v] {
            return true
        }
        s[v] = true
    }
    return false
}

```

Go

```

func containsDuplicate(nums []int) bool {
    s := map[int]bool{}
    for _, v := range nums {
        if s[v] {
            return true
        }
        s[v] = true
    }
    return false
}

```

Rust

Rust

```

impl Solution {
    pub fn contains_duplicate(nums: Vec<i32>) -> bool {
        nums.sort();
        let n = nums.len();
        for i in 1..n {
            if nums[i - 1] == nums[i] {
                return true;
            }
        }
        false
    }
}

```

[*] Sorting — Pattern Solution (matched from community answers)

Python

```

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return any(a == b for a, b in pairwise(sorted(nums)))

```

#242 EASY

Valid Anagram

LeetCode · Simplest · WalkCC · LeetCode · Editorial

Hash Table · String · Sorting

LeetCode 169

Problem:

Sorting · LeetCode · By Putham

— 323 —

LeetCode

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

Constraints:

- 1 <= s.length, t.length <= 5 * 10⁴
- s and t consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

Python

```

Python
# Python solution for Valid Anagram
def isAnagram(s: str, t: str) -> bool:
    # Check if lengths differ; if so, they can't be anagrams
    if len(s) != len(t):
        return False
    # Create a dictionary to count frequency of each character in s
    char_count = {}
    for char in s:
        # Increase the count of the character
        char_count[char] = char_count.get(char, 0) + 1
    # Decrease the count for each character in t
    for char in t:
        if char not in char_count or char_count[char] == 0:
            # Character not found or count exceeded, not an anagram
            return False
        char_count[char] -= 1
    # If all counts are zero, it's a valid anagram
    return True

```

Example usage:

```

print(isAnagram("anagram", "nagaram")) # Output: True
print(isAnagram("rat", "car")) # Output: False

```

Python

Sorting · LeetCode · By Putham

— 324 —

LeetCode

```

class Solution {
public:
    def isAnagram(s1: str, t1: str) -> bool:
        if len(s1) != len(t1):
            return False

        count = collections.Counter(s1)
        count.subtract(collections.Counter(t1))
        return all(freq == 0 for freq in count.values())
}

```

```

Python
class Solution:
    def isAnagram(s1: str, t1: str) -> bool:
        if len(s1) != len(t1):
            return False
        cnt = Counter(s1)
        for c in t1:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True

```

```

Python
class Solution:
    def isAnagram(s1: str, t1: str) -> bool:
        return Counter(s1) == Counter(t1)

```

```

Python
class Solution:
    def isAnagram(s1: str, t1: str) -> bool:
        if len(s1) != len(t1):
            return False
        cnt = Counter(s1)
        for c in t1:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True

```

C++

C++

Sorting - LeetCode - By Putnam

— 325 —

LeetCode

```

class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length())
            return false;

        vector<int> count(26);

        for (const char c : s)
            ++count[c - 'a'];

        for (const char c : t) {
            if (count[c - 'a'] == 0)
                return false;
            --count[c - 'a'];
        }

        return true;
    }
};

```

```

C++
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) {
            return false;
        }
        vector<int> cnt(26);
        for (int i = 0; i < s.size(); ++i) {
            ++cnt[s[i] - 'a'];
            --cnt[t[i] - 'a'];
        }
        return all_of(cnt.begin(), cnt.end(), [](int x) { return x == 0; });
    }
};

```

```

C++
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) {
            return false;
        }
        vector<int> cnt(26);
        for (int i = 0; i < s.size(); ++i) {
            ++cnt[s[i] - 'a'];
            --cnt[t[i] - 'a'];
        }
        return all_of(cnt.begin(), cnt.end(), [](int x) { return x == 0; });
    }
};

```

Java

Java

Sorting - LeetCode - By Putnam

— 326 —

LeetCode

```

class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length())
            return false;

        int[] count = new int[26];

        for (final char c : s.toCharArray())
            ++count[c - 'a'];

        for (final char c : t.toCharArray()) {
            if (count[c - 'a'] == 0)
                return false;
            --count[c - 'a'];
        }

        return true;
    }
}

```

```

Java
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length()) {
            return false;
        }
        int[] cnt = new int[26];
        for (int i = 0; i < s.length(); ++i) {
            ++cnt[s.charAt(i) - 'a'];
            --cnt[t.charAt(i) - 'a'];
        }
        for (int i = 0; i < 26; ++i) {
            if (cnt[i] != 0) {
                return false;
            }
        }
        return true;
    }
}

```

```

Java
public class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length()) {
            return false;
        }
        int[] cnt = new int[26];
        for (int i = 0; i < s.length(); ++i) {
            ++cnt[s.charAt(i) - 'a'];
            --cnt[t.charAt(i) - 'a'];
        }
        return cnt.All(x => x == 0);
    }
}

```

Java

Sorting - LeetCode - By Putnam

— 327 —

LeetCode

```

class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length()) {
            return false;
        }
        int[] cnt = new int[26];
        for (int i = 0; i < s.length(); ++i) {
            ++cnt[s.charAt(i) - 'a'];
            --cnt[t.charAt(i) - 'a'];
        }
        for (int i = 0; i < 26; ++i) {
            if (cnt[i] != 0) {
                return false;
            }
        }
        return true;
    }
}

```

Java

```

public class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length() != t.length()) {
            return false;
        }
        int[] cnt = new int[26];
        for (int i = 0; i < s.length(); ++i) {
            ++cnt[s.charAt(i) - 'a'];
            --cnt[t.charAt(i) - 'a'];
        }
        return cnt.All(x => x == 0);
    }
}

```

JavaScript

```

JavaScript
/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isAnagram = function (s, t) {
    if (s.length != t.length)
        return false;

    const cnt = new Array(26).fill(0);
    for (let i = 0; i < s.length; ++i) {
        ++cnt[s.charCodeAt(i) - 'a'.charCodeAt(0)];
        --cnt[t.charCodeAt(i) - 'a'.charCodeAt(0)];
    }
    return cnt.every(x => x == 0);
};

```

JavaScript

Sorting - LeetCode - By Putnam

— 328 —

LeetCode

```

/**
 * @param {string} s
 * @param {string} t
 * @return {boolean}
 */
var isAnagram = function (s, t) {
    if (s.length !== t.length) {
        return false;
    }
    const cnt = new Array(26).fill(0);
    for (let i = 0; i < s.length; ++i) {
        ++cnt[s.charCodeAt(i) - 'a'.charCodeAt(0)];
        --cnt[t.charCodeAt(i) - 'a'.charCodeAt(0)];
    }
    return cnt.every(x => x === 0);
};

```

TypeScript

TypeScript

```

function isAnagram(s: string, t: string): boolean {
    if (s.length !== t.length) {
        return false;
    }
    const cnt = new Array(26).fill(0);
    for (let i = 0; i < s.length; ++i) {
        ++cnt[s.charCodeAt(i) - 'a'.charCodeAt(0)];
        --cnt[t.charCodeAt(i) - 'a'.charCodeAt(0)];
    }
    return cnt.every(x => x === 0);
}

```

TypeScript

```

function isAnagram(s: string, t: string): boolean {
    if (s.length !== t.length) {
        return false;
    }
    const cnt = new Array(26).fill(0);
    for (let i = 0; i < s.length; ++i) {
        ++cnt[s.charCodeAt(i) - 'a'.charCodeAt(0)];
        --cnt[t.charCodeAt(i) - 'a'.charCodeAt(0)];
    }
    return cnt.every(x => x === 0);
}

```

Rust

Rust

Sorting - LeetCode - By Putnam

— 329 —

LeetCode

```

impl Solution {
    pub fn isAnagram(s: String, t: String) -> bool {
        let s = s.as_bytes();
        let t = t.as_bytes();
        if s.len() != t.len() {
            return false;
        }
        let mut s = s.iter().collect::VecChar();
        let mut t = t.iter().collect::VecChar();
        s.sort();
        t.sort();
        for i in 0..n {
            if s[i] != t[i] {
                return false;
            }
        }
        true
    }
}

```

Rust

```

impl Solution {
    pub fn isAnagram(s: String, t: String) -> bool {
        let s = s.as_bytes();
        let t = t.as_bytes();
        if s.len() != t.len() {
            return false;
        }
        let (s, t) = (s.as_bytes(), t.as_bytes());
        let mut count = [0; 26];
        for i in 0..n {
            count[s[i] - b'a'] as usize += 1;
            count[t[i] - b'a'] as usize -= 1;
        }
        count.iter().all(|&c| *c == 0)
    }
}

```

[*] Sorting - Pattern Solution (stored optimal)

Python

Sorting - LeetCode - By Putnam

— 330 —

LeetCode

```

# Python solution for Valid Anagram

def isAnagram(s, t) -> bool:
    # Check if lengths differ, if so, they can't be anagrams
    if len(s) != len(t):
        return False

    # Create a dictionary to count frequency of each character in s
    char_count = {}
    for char in s:
        # Increase the count of the character
        char_count[char] = char_count.get(char, 0) + 1

    # Decrease the count for each character in t
    for char in t:
        # Check if char not in char_count or char_count[char] == 0:
        # Character not found or count mismatch, not an anagram
        return False
        char_count[char] -= 1

    # If all counts are zero, it's a valid anagram
    return True

# Example usage
print(isAnagram("anagram", "nagaram")) # Output: True
print(isAnagram("rat", "car")) # Output: False

```

```

C++

// C++ solution for Valid Anagram
#include <iostream>
#include <string>
#include <unordered_map>
using namespace std;

bool isAnagram(string s, string t) {
    // If lengths differ, they cannot be anagrams
    if (s.size() != t.size()) return false;

    // Create an array of size 26 for lowercase letters initialized to 0
    vector<int> charCount(26, 0);

    // Count frequency for each character in s
    for (char c : s) {
        charCount[c - 'a']++;
    }

    // Decrease the count for each character in t
    for (char c : t) {
        if (--charCount[c - 'a'] < 0) {
            return false;
        }
    }

    return true;
}

```

```

return true;
}

int main() {
    // Example usage:
    cout << boolalpha << isAnagram("anagram", "nagaram") << endl; // Output: true
    cout << boolalpha << isAnagram("rat", "car") << endl; // Output: false
    return 0;
}

```

#252 EASY Meeting Rooms

LeetCode · SimplifyLeetCode · WAAJCE · LeetCode · Editorial

Array · Sorting

Lists Grid 169 · Premium 98

Problem:
Given an array of meeting time intervals where intervals[i] = [starti, endi], determine if a person could attend all meetings.

Example 1:

Input: intervals = [[0,30],[15,10],[15,20]]

Output: false

Example 2:

Input: intervals = [[7,10],[2,4]]

Output: true

Constraints:

• 0 <= intervals.length <= 104

• intervals[i].length == 2

• 0 <= starti < endi <= 106

>> Brute Force [Sorting] Interview Prep

```

Python

# Brute Force Approach
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        # Sort intervals by start time, check every pair of intervals for overlap
        for i in range(len(intervals)-1):
            for j in range(i+1, len(intervals)):
                a, b = intervals[i], intervals[j]
                if a[0] < b[1] and b[0] < a[1]: # overlap
                    return False
        return True

```

```

Python

Python

```

```

# Python solution for checking if one can attend all meetings

def canAttendMeetings(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])
    # Iterate through intervals to find any overlap
    for i in range(1, len(intervals)):
        # Check if the previous meeting ends after the current meeting starts
        if intervals[i-1][1] > intervals[i][0]:
            return False
    return True

# Example usage:
print(canAttendMeetings([[0, 30], [15, 10], [15, 20]])) # Expected output: False
print(canAttendMeetings([[7, 10], [2, 4]])) # Expected output: True

```

```

Python

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()

        for i in range(1, len(intervals)):
            if intervals[i - 1][1] > intervals[i][0]:
                return False
        return True

```

```

Python

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))

```

Python

```

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))

#####

from itertools import pairwise
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort(key=lambda x: x[0])
        return not any(a[1] > b[0] for a, b in pairwise(intervals))
        # one is True, then can't attend

#####

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort(key=lambda x: x[0])
        for i in range(1, len(intervals) - 1):
            if intervals[i][1] > intervals[i + 1][0]:
                return False
        return True

#####

# Definition for an Interval.
class Interval(object):
    # def __init__(self, start, end):
    #     self.start = start
    #     self.end = end

class Solution(object):
    def canAttendMeetings(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: bool

        intervals = sorted(intervals, key=lambda x: x.start)
        for i in range(1, len(intervals)):
            if intervals[i].start < intervals[i - 1].end:
                return False
        return True

```

```

C++

C++

```

```

class Solution {
public:
    bool canAttendMeetings(vector<vector<int>>& intervals) {
        ranges::sort(intervals);

        for (int i = 1; i < intervals.size(); ++i)
            if (intervals[i - 1][1] > intervals[i][0])
                return false;

        return true;
    }
};

```

```

C++

class Solution {
public:
    bool canAttendMeetings(vector<vector<int>>& intervals) {
        ranges::sort(intervals, [](const auto& a, const auto& b) {
            return a[0] < b[0];
        });
        for (int i = 1; i < intervals.size(); ++i) {
            if (intervals[i - 1][1] > intervals[i][0]) {
                return false;
            }
        }
        return true;
    }
};

```

```

C++

class Solution {
public:
    bool canAttendMeetings(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
            return a[0] < b[0];
        });
        for (int i = 1; i < intervals.size(); ++i) {
            if (intervals[i][0] < intervals[i - 1][1]) {
                return false;
            }
        }
        return true;
    }
};

```

```

Java

Java

```

```

class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, Comparator.comparingInt(Interval -> Interval.start));

        for (int i = 1; i < intervals.length; ++i)
            if (intervals[i - 1][1] > intervals[i][0])
                return false;

        return true;
    }
}

```

Java

```

class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        for (int i = 1; i < intervals.length; ++i) {
            if (intervals[i - 1][1] > intervals[i][0]) {
                return false;
            }
        }
        return true;
    }
}

```

Java

```

class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        for (int i = 1; i < intervals.length; ++i) {
            var a = intervals[i - 1];
            var b = intervals[i];
            if (a[1] > b[0]) {
                return false;
            }
        }
        return true;
    }
}

```

```

TypeScript

TypeScript

```

```

function canAttendMeetings(intervals: number[][]): boolean {
    intervals.sort((a, b) => a[0] - b[0]);
    for (let i = 1; i < intervals.length; ++i) {
        if (intervals[i][0] < intervals[i - 1][1]) {
            return false;
        }
    }
    return true;
}

```

```

TypeScript

TypeScript

```

```

function canAttendMeetings(intervals: number[][]): boolean {
    intervals.sort((a, b) => a[0] - b[0]);
    for (let i = 1; i < intervals.length; ++i) {
        if (intervals[i][0] < intervals[i - 1][1]) {
            return false;
        }
    }
    return true;
}

```

Go

```

func canAttendMeetings(intervals [][]int) bool {
    sort.Slice(intervals, func(i, j int) bool {
        return intervals[i][0] < intervals[j][0]
    })
    for i := 1; i < len(intervals); i++ {
        if intervals[i][0] < intervals[i-1][1] {
            return false
        }
    }
    return true
}

```

Go

```

func canAttendMeetings(intervals [][]int) bool {
    sort.Slice(intervals, func(i, j int) bool {
        return intervals[i][0] < intervals[j][0]
    })
    for i := 1; i < len(intervals); i++ {
        if intervals[i][0] < intervals[i-1][1] {
            return false
        }
    }
    return true
}

```

Rust

```

impl Solution {
    pub fn can_attend_meetings(intervals: Vec<Vec<i32>>) -> bool {
        intervals.sort_by(|a, b| a[0].cmp(&b[0]));
        for i in 1..intervals.len() {
            if intervals[i - 1][1] > intervals[i][0] {
                return false;
            }
        }
        true
    }
}

```

Sorting · LeetCode — By Putnam

— 337 —

LeetCode

```

impl Solution {
    /* @author: conan */
    pub fn can_attend_meetings(intervals: Vec<Vec<i32>>) -> bool {
        if intervals.len() == 1 {
            return true;
        }

        let mut intervals = intervals;

        // Sort the intervals vector
        intervals.sort_by(|lho, rhi| { lho[0].cmp(&rhi[0]) });

        let mut end = -1;

        // Brute Force
        for p in intervals {
            if end == -1 {
                // This is the first pair
                end = p[1];
                continue;
            }
            if p[0] < end {
                return false;
            }
            end = p[1];
        }

        true
    }
}

```

[*] Sorting — Pattern Solution (matched from community answers)

Python

Sorting · LeetCode — By Putnam

— 338 —

LeetCode

```

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))

#####

from itertools import pairwise
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort(key=lambda x: x[0])
        return not any(a[1] > b[0] for a, b in pairwise(intervals))
# ans is True, then canAttendMeetings()

#####

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort(key=lambda x: x[0])
        for i in range(len(intervals) - 1):
            if intervals[i][1] > intervals[i + 1][0]:
                return False
        return True

#####

# Definition for an Interval.
# class Interval(object):
#     def __init__(self, s1, e1, s2, e2):
#         self.start = s1
#         self.end = e1

class Solution(object):
    def canAttendMeetings(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: bool

        intervals = sorted(intervals, key=lambda x: x.start)
        for i in range(1, len(intervals)):
            if intervals[i].start < intervals[i - 1].end:
                return False
        return True
"""

```

#1086 EASY
Largest Unique Number
LeetCode · SimplyLear · WalkCC · LeetCode · Editorial
Array · Hash Table · Sorting
Like · Premium 98

Problem:
Given a list of the scores of different students, items, where items[i] = [IDi, scorei] represents one score from a student with IDi, calculate each student's top five average.

Sorting · LeetCode — By Putnam

— 339 —

LeetCode

Return the answer as an array of pairs result, where result[i] = [IDi, topFiveAveragei] represents the student with IDi and their top five average. Sort result by IDi in increasing order.

A student's top five average is calculated by taking the sum of their top five scores and dividing it by 5 using integer division.

Example 1:

Input: items = [[1,91],[1,92],[2,93],[2,97],[1,60],[2,77],[1,65],[1,87],[1,100],[2,100],[2,76]]
Output: [[1,87],[2,80]]

Explanation:
The student with ID = 1 got scores 91, 92, 60, 65, 87, and 100. Their top five average is (90 + 92 + 91 + 87 + 65) / 5 = 87.
The student with ID = 2 got scores 93, 97, 77, 100, and 76. Their top five average is (100 + 97 + 93 + 77 + 76) / 5 = 88.6, but with integer division their average converts to 88.

Example 2:

Input: items = [[1,100],[7,100],[1,100],[7,100],[1,100],[7,100],[1,100],[7,100]]
Output: [[1,100],[7,100]]

Constraints:

- 1 <= items.length <= 1000
- items[i].length == 2
- 1 <= IDi <= 1000
- 0 <= scorei <= 100
- For each IDi, there will be at least five scores.

>> Brute Force (Sorting) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def largestUniqueNumber(self, nums):
        # Brute Force O(n^2) - bubble sort
        arr = list(nums)
        n = len(arr)
        for i in range(n):
            for j in range(n - i - 1):
                if arr[j] > arr[j + 1]:
                    arr[j], arr[j + 1] = arr[j + 1], arr[j]
        return arr

```

Python

Python

Sorting · LeetCode — By Putnam

— 340 —

LeetCode

```

class Solution:
    def highFive(self, items: List[List[int]]) -> List[List[int]]:
        idfscores = collections.defaultdict(list)

        for id, score in items:
            heapq.heappush(idfscores[id], score)
            if len(idfscores[id]) > 5:
                heapq.heappop(idfscores[id])

        return [[id, sum(scores) // 5] for id, scores in sorted(idfscores.items())]

Python

class Solution:
    def highFive(self, items: List[List[int]]) -> List[List[int]]:
        d = defaultdict(list)
        n = 0
        for i, x in items:
            d[i].append(x)
            n = max(n, 5)
        ans = []
        for i in range(1, n + 1):
            if n >= d[i]:
                ans = sorted(sorted(d[i, x] for x in d[i]), reverse=True)[:5]
            ans.append((i, sum(ans) // 5))
        return ans

Python

class Solution:
    def highFive(self, items: List[List[int]]) -> List[List[int]]:
        d = defaultdict(list)
        n = 0
        for i, x in items:
            d[i].append(x)
            n = max(n, 5)
        ans = []
        for i in range(1, n + 1):
            if n >= d[i]:
                ans = sorted(sorted(d[i, x] for x in d[i]), reverse=True)[:5]
            ans.append((i, sum(ans) // 5))
        return ans

C++

C++

```

Sorting · LeetCode — By Putnam

— 341 —

LeetCode

```

class Solution {
public:
    vector<vector<int>> highFive(vector<vector<int>>& items) {
        vector<vector<int>> ans;
        map<int, priority_queue<int, vector<int>, greater<int>>> idfscores;

        for (const vector<int>& item : items) {
            const int id = item[0];
            const int score = item[1];
            auto& scores = idfscores[id];
            scores.push(score);
            if (scores.size() > 5)
                scores.pop();
        }

        for (auto& [id, scores] : idfscores) {
            int sum = 0;
            while (!scores.empty()) {
                sum += scores.top();
                scores.pop();
            }
            ans.push_back({id, sum / 5});
        }

        return ans;
    }
};

C++

class Solution {
public:
    vector<vector<int>> highFive(vector<vector<int>>& items) {
        vector<int> idfs;
        for (auto& item : items) {
            int id = item[0], s = item[1];
            d[id].push_back(s);
        }
        vector<vector<int>> ans;
        for (int i = 1; i <= 1000; ++i) {
            if (d[i].empty()) {
                sort(d[i].begin(), d[i].end(), greater<int>());
                int n = 0;
                for (int j = 0; j < 5; ++j) {
                    n += d[i][j];
                }
                ans.push_back({i, n / 5});
            }
        }
        return ans;
    }
};

C++

```

C++

Sorting · LeetCode — By Putnam

— 342 —

LeetCode

```
class Solution {
public:
    vector<vector<int>> highFive(vector<vector<int>>& items) {
        vector<int> d{1001};
        for (const item : items) {
            int i = item[0], s = item[1];
            d[i].push_back(s);
        }
        vector<vector<int>> ans;
        for (int i = 1; i <= 1000; ++i) {
            if (d[i].empty()) {
                sort(d[i].begin(), d[i].end(), greater<int>());
                int s = 0;
                for (int j = 0; j < 5; ++j) {
                    s += d[i][j];
                }
                ans.push_back({i, s / 5});
            }
        }
        return ans;
    }
};

Java

class Solution {
    public List<List<Integer>> highFive(List<List<Integer>> items) {
        List<List<Integer>> ans = new ArrayList<>();
        Map<Integer, PriorityQueue<Integer>> idfScores = new TreeMap<>();
        for (List<Integer> item : items) {
            final int id = item[0];
            final int score = item[1];
            idfScores.putIfAbsent(id, new PriorityQueue<>());
            PriorityQueue<Integer> scores = idfScores.get(id);
            scores.add(score);
            if (scores.size() > 5)
                scores.poll();
        }
        for (Map.Entry<Integer, PriorityQueue<Integer>> entry : idfScores.entrySet()) {
            final int id = entry.getKey();
            PriorityQueue<Integer> scores = entry.getValue();
            int sum = 0;
            while (!scores.isEmpty())
                sum += scores.poll();
            ans.add(new List<>(){{id, sum / 5}});
        }
        return ans.toArray(List<>[]::new);
    }
}

Java
```

Sorting · LeetCode - By Putnam

— 343 —

LeetCode

```
class Solution {
    public List<List<Integer>> highFive(List<List<Integer>> items) {
        int size = 0;
        PriorityQueue<Integer> q = new PriorityQueue<>(101);
        int s = 0;
        for (List<Integer> item : items) {
            int i = item[0], score = item[1];
            if (i % 1000 == 0) {
                s += score;
                q.add(-score);
            }
            if (q.size() > 5) {
                q.poll();
            }
        }
        List<List<Integer>> res = new ArrayList<>();
        int j = 0;
        for (int i = 0; i < 1000; ++i) {
            if (i % 1000 == 0) {
                continue;
            }
            int neg = count(i) / 5;
            res.add(new List<>(){{i, neg}});
            j++;
        }
        return res;
    }

    private int count(PriorityQueue<Integer> q) {
        int s = 0;
        while (!q.isEmpty()) {
            s += q.poll();
        }
        return s;
    }
}

TypeScript
typescript
```

Sorting · LeetCode - By Putnam

— 344 —

LeetCode

```
function highFive(items: number[][]): number[][] {
    const d: number[][] = Array(1001)
        .fill(0)
        .map(() => Array(0));
    for (const [i, s] of items) {
        d[i].push(s);
    }
    const ans: number[][] = [];
    for (let i = 1; i <= 1000; ++i) {
        if (d[i].length > 0) {
            d[i].sort((a, b) => b - a);
            const s = d[i].slice(0, 5).reduce((a, b) => a + b);
            ans.push([i, Math.floor(s / 5)]);
        }
    }
    return ans;
}

TypeScript

function highFive(items: number[][]): number[][] {
    const d: number[][] = Array(1001)
        .fill(0)
        .map(() => Array(0));
    for (const [i, s] of items) {
        d[i].push(s);
    }
    const ans: number[][] = [];
    for (let i = 1; i <= 1000; ++i) {
        if (d[i].length > 0) {
            d[i].sort((a, b) => b - a);
            const s = d[i].slice(0, 5).reduce((a, b) => a + b);
            ans.push([i, Math.floor(s / 5)]);
        }
    }
    return ans;
}

Go

func highFive(items [][]int) (ans [][]int) {
    d := make([][]int, 1001)
    for _, item := range items {
        i, s := item[0], item[1]
        d[i] = append(d[i], s)
    }
    for i := 1; i <= 1000; i++ {
        if len(d[i]) > 0 {
            sort.Ints(d[i])
            s := 0
            for j := len(d[i]) - 1; j >= len(d[i]) - 5; j-- {
                s += d[i][j]
            }
            ans = append(ans, []int{i, s / 5})
        }
    }
    return ans
}

Go
```

Sorting · LeetCode - By Putnam

— 345 —

LeetCode

```
Go

func highFive(items [][]int) (ans [][]int) {
    d := make([][]int, 1001)
    for _, item := range items {
        i, s := item[0], item[1]
        d[i] = append(d[i], s)
    }
    for i := 1; i <= 1000; i++ {
        if len(d[i]) > 0 {
            sort.Ints(d[i])
            s := 0
            for j := len(d[i]) - 1; j >= len(d[i]) - 5; j-- {
                s += d[i][j]
            }
            ans = append(ans, []int{i, s / 5})
        }
    }
    return ans
}

[+] Sorting — Pattern Solution (matched from community answers)

Python

class Solution:
    def highFive(self, items: List[List[int]]) -> List[List[int]]:
        idfScores = collections.defaultdict(list)
        for id, score in items:
            heapq.heappush(idfScores[id], score)
            if len(idfScores[id]) > 5:
                heapq.heappop(idfScores[id])
        return [[id, sum(scores) // 5] for id, scores in sorted(idfScores.items())]

#1122 EASY
Relative Sort Array
LeetCode · SimpleHash · HashCC · LeetCode · Editorial
Array · Hash Table · Sorting · Counting Sort
List: Danny Zhang

Problem:
Given two arrays arr1 and arr2, the elements of arr2 are distinct, and all elements in arr2 are also in arr1.
Sort the elements of arr1 such that the relative ordering of items in arr1 are the same as in arr2.
Elements that do not appear in arr2 should be placed at the end of arr1 in ascending order.

Example 1:
Input: arr1 = [28,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]
Output: [2,2,1,4,3,9,6,7,19]

Example 2:
```

Sorting · LeetCode - By Putnam

— 346 —

LeetCode

```
Input: arr1 = [28,3,1,3,2,4,6,7,9,2,19], arr2 = [2,28,9,6]
Output: [2,28,3,6,17,44]

Constraints:
• 1 <= arr1.length, arr2.length <= 1000
• 0 <= arr1[i], arr2[i] <= 1000
• All the elements of arr2 are distinct.
• Each arr2[i] is in arr1.

>> Brute Force [Sorting] Interview Prep

Python

# Brute Force Approach
class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        # Brute Force Sort - bubble sort
        arr = List(arr1)
        n = len(arr)
        for i in range(n):
            for j in range(n - 1 - i):
                if arr[j] > arr[j + 1]:
                    arr[j], arr[j + 1] = arr[j + 1], arr[j]
        return arr

Python

# Python Solution using a Frequency Dictionary
def relativeSortArray(arr1: List[int], arr2: List[int]) -> List[int]:
    # Count frequency of each element in arr1
    count = {}
    for num in arr1:
        count[num] = count.get(num, 0) + 1

    result = []
    # Process elements defined in arr2: append each element count times
    for num in arr2:
        if num in count:
            result.extend([num] * count[num])
            # Remove the element from the count dictionary
            del count[num]

    # Process remaining numbers: sort them in ascending order and append
    remaining = []
    for num, freq in count.items():
        remaining.extend([num] * freq)
    result.extend(sorted(remaining))

    return result

# Example usage:
print(relativeSortArray([2,3,1,3,2,4,6,7,9,2,19], [2,1,4,3,9,6]))

Python
```

Sorting · LeetCode - By Putnam

— 347 —

LeetCode

```
class Solution {
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        ans = []
        count = {}
        for a in arr1:
            count[a] += 1
        for a in arr2:
            while count[a] > 0:
                ans.append(a)
                count[a] -= 1
        for num in range(1001):
            for _ in range(count[num]):
                ans.append(num)
        return ans
}

Python

class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        pos = {}
        for i, x in enumerate(arr2):
            pos[x] = i
        sorted_arr1 = sorted(arr1, key=lambda x: pos.get(x, 1000 + x))
        return sorted_arr1

Python

class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        pos = {}
        for i, x in enumerate(arr2):
            pos[x] = i
        sorted_arr1 = sorted(arr1, key=lambda x: pos.get(x, 1000 + x))
        return sorted_arr1

C++

class Solution {
public:
    vector<int> relativeSortArray(vector<int>& arr1, vector<int>& arr2) {
        vector<int> ans;
        vector<int> count(1001);
        for (int a : arr1)
            ++count[a];
        for (int a : arr2)
            while (count[a]-- > 0)
                ans.push_back(a);
        for (int num = 0; num < 1001; ++num)
            while (count[num]-- > 0)
                ans.push_back(num);
        return ans;
    }
};

C++
```

Sorting · LeetCode - By Putnam

— 348 —

LeetCode

```
class Solution {
public:
    vector<int> relativeSortArray(vector<int>& arr1, vector<int>& arr2) {
        unordered_map<int, int> pos;
        for (int i = 0; i < arr2.size(); ++i) {
            pos[arr2[i]] = i;
        }
        vector<int> res;
        for (int i = 0; i < arr1.size(); ++i) {
            int j = pos.count(arr1[i]) ? pos[arr1[i]] : arr2.size();
            arr.emplace_back(i, arr2[i]);
        }
        sort(arr.begin(), arr.end());
        for (int i = 0; i < arr1.size(); ++i) {
            arr[i] = arr[i].second;
        }
        return arr;
    }
};
```

```
C++
class Solution {
public:
    func relativeSortArray... arr1: [int], ... arr2: [int]) -> [int] {
        Map pos = [int: int]{}
        for (i, v) in arr2.enumerated() {
            pos[i] = i
        }
        var arr = [[int, int]]{}
        for x in arr1 {
            let j = pos[i] ?? arr2.count
            arr.append(i, x)
        }
        arr.sort { $0.0 < $1.0 || ($0.0 == $1.0 && $0.1 < $1.1) }
        return arr.map { $0.1 }
    }
}
```

```
C++
class Solution {
public:
    vector<int> relativeSortArray(vector<int>& arr1, vector<int>& arr2) {
        unordered_map<int, int> pos;
        for (int i = 0; i < arr2.size(); ++i) {
            pos[arr2[i]] = i;
        }
        vector<int> res;
        for (int i = 0; i < arr1.size(); ++i) {
            int j = pos.count(arr1[i]) ? pos[arr1[i]] : arr2.size();
            arr.emplace_back(i, arr2[i]);
        }
        sort(arr.begin(), arr.end());
        for (int i = 0; i < arr1.size(); ++i) {
            arr[i] = arr[i].second;
        }
        return arr;
    }
};
```

```
C++
class Solution {
public:
    func relativeSortArray... arr1: [int], ... arr2: [int]) -> [int] {
        var pos = [int: int]{}
        for (i, v) in arr2.enumerated() {
            pos[i] = i
        }
        var arr = [[int, int]]{}
        for x in arr1 {
            let j = pos[i] ?? arr2.count
            arr.append(i, v)
        }
        arr.sort { $0.0 < $1.0 || ($0.0 == $1.0 && $0.1 < $1.1) }
        return arr.map { $0.1 }
    }
}
```

```
Java
class Solution {
    public int[] relativeSortArray(int[] arr1, int[] arr2) {
        int[] ans = new int[arr1.length];
        int count = new int[arr2.length];
        int i = 0;

        for (int a : arr1)
            ++count[a];

        for (int a : arr2)
            while (count[a]-- > 0)
                ans[i++] = a;

        for (int num = 0; num < 1001; ++num)
            while (count[num] > 0)
                ans[i++] = num;

        return ans;
    }
}
```

```
class Solution {
    public int[] relativeSortArray(int[] arr1, int[] arr2) {
        Map<Integer, Integer> pos = new HashMap<>();
        for (int i = 0; i < arr2.length; ++i) {
            pos.put(arr2[i], i);
        }
        int[] arr = new int[arr1.length];
        for (int i = 0; i < arr1.length; ++i) {
            arr[i] = pos.getOrDefault(arr1[i], arr2.length + arr1[i]);
        }
        Arrays.sort(arr, (a, b) -> a[1] - b[1]);
        for (int i = 0; i < arr.length; ++i) {
            arr[i] = arr[i][0];
        }
        return arr;
    }
}
```

```
Java
class Solution {
    public int[] relativeSortArray(int[] arr1, int[] arr2) {
        Map<Integer, Integer> pos = new HashMap<>();
        for (int i = 0; i < arr2.length; ++i) {
            pos.put(arr2[i], i);
        }
        int[] arr = new int[arr1.length];
        for (int i = 0; i < arr1.length; ++i) {
            arr[i] = pos.getOrDefault(arr1[i], arr2.length + arr1[i]);
        }
        Arrays.sort(arr, (a, b) -> a[1] - b[1]);
        for (int i = 0; i < arr.length; ++i) {
            arr[i] = arr[i][0];
        }
        return arr;
    }
}
```

TypeScript

```
TypeScript
function relativeSortArray(arr1: number[], arr2: number[]): number[] {
    const pos: Map<number, number> = new Map()
    for (let i = 0; i < arr2.length; ++i) {
        pos.set(arr2[i], i)
    }
    const arr: number[][] = []
    for (const x of arr1) {
        const j = pos.get(x) ?? arr2.length
        arr.push([x, j])
    }
    arr.sort((a, b) => a[1] - b[1] || a[0] - b[0])
    return arr.map(a => a[0])
}
```

```
function relativeSortArray(arr1: number[], arr2: number[]): number[] {
    const pos: Map<number, number> = new Map()
    for (let i = 0; i < arr2.length; ++i) {
        pos.set(arr2[i], i)
    }
    const arr: number[][] = []
    for (const x of arr1) {
        const j = pos.get(x) ?? arr2.length
        arr.push([x, j])
    }
    arr.sort((a, b) => a[1] - b[1] || a[0] - b[0])
    return arr.map(a => a[0])
}
```

Go

```
Go
func relativeSortArray(arr1 []int, arr2 []int) []int {
    pos := map[int]int{}
    for i, x := range arr2 {
        pos[i] = i
    }
    arr := make([][]int, len(arr1))
    for i, x := range arr1 {
        if p, ok := pos[i]; ok {
            arr[i] = [2]int{p, x}
        } else {
            arr[i] = [2]int{len(arr2), x}
        }
    }
    sort.Slice(arr, func(i, j int) bool {
        return arr[i][0] < arr[j][0] || arr[i][0] == arr[j][0] && arr[i][1] < arr[j][1]
    })
    for i, x := range arr {
        arr[i] = x[1]
    }
    return arr1
}
```

```
func relativeSortArray(arr1 []int, arr2 []int) []int {
    pos := map[int]int{}
    for i, x := range arr2 {
        pos[i] = i
    }
    arr := make([][]int, len(arr1))
    for i, x := range arr1 {
        if p, ok := pos[i]; ok {
            arr[i] = [2]int{p, x}
        } else {
            arr[i] = [2]int{len(arr2), x}
        }
    }
    sort.Slice(arr, func(i, j int) bool {
        return arr[i][0] < arr[j][0] || arr[i][0] == arr[j][0] && arr[i][1] < arr[j][1]
    })
    for i, x := range arr {
        arr[i] = x[1]
    }
    return arr1
}
```

[*] Sorting - Pattern Solution (matched from community answers)

```
Python
# Python solution using a frequency dictionary
def relativeSortArray(arr1, arr2):
    # Count frequency of each element in arr1
    count = {}
    for num in arr1:
        count[num] = count.get(num, 0) + 1

    result = []
    # Process elements defined in arr2: append each element count times
    for num in arr2:
        result.extend([num] * count[num])
        # Remove the element from the count dictionary
        del count[num]

    # Process remaining numbers: sort them in ascending order and append
    remaining = []
    for num, freq in count.items():
        remaining.extend([num] * freq)
    result.extend(sorted(remaining))

    return result

# Example usage:
print(relativeSortArray([2,3,1,3,2,4,6,7,9,2,10], [2,1,4,3,9,6]))
```

LeetCode - SimplifyLeetCode - LeetCode - Editorial

Array - Hash Table - String - Sorting

Like: 100

Problem:

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

- There is no string in strs that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate","eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [""]

Example 3:

Input: strs = ["a"]

Output: ["a"]

Constraints:

- 1 <= strs.length <= 104
- 0 <= strs[i].length <= 100
- strs[i] consists of lowercase English letters.

>> Brute Force [Sorting] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def groupAnagrams(self, strs):
        # Brute Force O(N^2 * K log K) - compare each pair by sorting
        n = len(strs)
        used = [False] * n
        result = []
        for i in range(n):
            if used[i]: continue
            group = [strs[i]]
            used[i] = True
            for j in range(i + 1, n):
                if not used[j] and sorted(strs[i]) == sorted(strs[j]):
                    group.append(strs[j])
                    used[j] = True
            result.append(group)
        return result
```

Python

```
# Python solution for grouping anagrams
from collections import defaultdict

def groupAnagrams(strs):
    # Create a dictionary to hold lists of anagrams; default to an empty list for new keys
    anagrams = defaultdict(list)
    # Iterate over each string in the input list
    for s in strs:
        # Sort the string to use as a key; sorted returns a list so join back to string
        key = ''.join(sorted(s))
        # Append the original string under the sorted key
        anagrams[key].append(s)
    # Return the grouped anagrams as a list of lists
    return list(anagrams.values())

# Example usage:
if __name__ == '__main__':
    strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
    result = groupAnagrams(strs)
    print(result)
```

```
Python
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        dict = collections.defaultdict(list)
        for str in strs:
            key = ''.join(sorted(str))
            dict[key].append(str)
        return dict.values()
```

```
Python
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = ''.join(sorted(s))
            d[k].append(s)
        return list(d.values())
```

Python

```
...
from collections import defaultdict
d = defaultdict(list)
for s in strs:
    d[''.join(sorted(s))].append(s)
return list(d.values())

...
def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
    d = defaultdict(list)
    for s in strs:
        k = ''.join(sorted(s))
        d[k].append(s)
    return list(d.values())
```

C++

```
C++
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> ans;
        for (const string& str : strs) {
            string key = str;
            ranges::sort(key);
            ans[key].push_back(str);
        }
        for (const auto& [_, anagrams] : ans) {
            return ans;
        }
    };
};
```

C++

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> d;
        for (const s : strs) {
            string k = s;
            sort(begin(k), end(k));
            d[k].emplace_back(s);
        }
        vector<vector<string>> ans;
        for (auto& [_, v] : d) ans.emplace_back(v);
        return ans;
    };
};
```

C++

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> d;
        for (const s : strs) {
            string k = s;
            sort(begin(k), end(k));
            d[k].emplace_back(s);
        }
        vector<vector<string>> ans;
        for (auto& [_, v] : d) ans.emplace_back(v);
        return ans;
    };
};
```

Java

```
Java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> keyToAnagrams = new HashMap<>();
        for (final String str : strs) {
            char[] chars = str.toCharArray();
            String key = String.valueOf(chars);
            keyToAnagrams.computeIfAbsent(key, k -> new ArrayList<>()).add(str);
        }
        return new ArrayList<>(keyToAnagrams.values());
    }
}
```

Java

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> d = new HashMap<>();
        for (String s : strs) {
            char[] k = s.toCharArray();
            Arrays.sort(k);
            String key = String.valueOf(k);
            d.computeIfAbsent(key, key -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(d.values());
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn group_anagrams(strs: Vec<String>) -> Vec<Vec<String>> {
        let mut map = HashMap::new();
        for s in strs {
            let key = {
                let mut arr = s.chars().collect<Vec<char>>();
                arr.sort();
                arr.iter().collect<String>()
            };
            let val = map.entry(key).or_insert<Vec<>>();
            val.push(s);
            map.into_iter().map(|(k, v)| v).collect()
        }
    }
}
```

Java

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> d = new HashMap<>();
        for (String s : strs) {
            char[] k = s.toCharArray();
            Arrays.sort(k);
            String key = String.valueOf(k);
            d.computeIfAbsent(key, key -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(d.values());
    }
}
```

Java

```
using System.Collections.Generic;

public class Comparer : IEqualityComparer<string>
{
    public bool Equal(string left, string right)
    {
        if (left.Length != right.Length) return false;

        var leftCount = new int[26];
        foreach (var ch in left)
        {
            ++leftCount[ch - 'a'];
        }

        var rightCount = new int[26];
        foreach (var ch in right)
        {
            var index = ch - 'a';
            if (++rightCount[index] > leftCount[index]) return false;
        }

        return true;
    }

    public int GetHashCode(string obj)
    {
        var hashCode = 0;
        for (int i = 0; i < obj.Length; ++i)
        {
            hashCode ^= 1 << (obj[i] - 'a');
        }
        return hashCode;
    }
}
```

```
public class Solution {
    public IList<IList<string>> GroupAnagrams(string[] strs) {
        var dict = new Dictionary<string, List<string>>(new Comparer());
        foreach (var str in strs)
        {
            dict[str].Add(str);
        }
        return dict.Values;
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn group_anagrams(strs: Vec<String>) -> Vec<Vec<String>> {
        let mut map = HashMap::new();
        for s in strs {
            let key = {
                let mut arr = s.chars().collect<Vec<char>>();
                arr.sort();
                arr.iter().collect<String>()
            };
            let val = map.entry(key).or_insert<Vec<>>();
            val.push(s);
            map.into_iter().map(|(k, v)| v).collect()
        }
    }
}
```

TypeScript

```
function groupAnagrams(strs: string[]): string[][] {
    const d: Map<string, string[]> = new Map();
    for (const s of strs) {
        const k = s.split('').sort().join('');
        if (!d.has(k)) {
            d.set(k, []);
        }
        d.get(k).push(s);
    }
    return Array.from(d.values());
}
```

TypeScript

```
function groupAnagrams(strs: string[]): string[][] {
    const map = new Map<string, string[]>();
    for (const str of strs) {
        const k = str.split('').sort().join('');
        map.set(k, map.get(k) ? [...map.get(k), str] : [str]);
    }
    return [...map.values()];
}
```

TypeScript

```
function groupAnagrams(strs: string[]): string[][] {
    const d: Map<string, string[]> = new Map();
    for (const s of strs) {
        const k = s.split('').sort().join('');
        if (!d.has(k)) {
            d.set(k, []);
        }
        d.get(k).push(s);
    }
    return Array.from(d.values());
}
```



```

Go
Go

func groupAnagrams(strs []string) [][]string {
    d := map[string][]string{}
    for _, s := range strs {
        t := []byte(s)
        sort.Slice(t, func(i, j int) bool { return t[i] < t[j] })
        k := string(t)
        d[k] = append(d[k], s)
    }
    for _, v := range d {
        ans = append(ans, v)
    }
    return
}

Go

func groupAnagrams(strs []string) [][]string {
    d := map[[]int][]string{}
    for _, s := range strs {
        cnt := [26]int{}
        for _, c := range s {
            cnt[c-'a']++
        }
        d[cnt] = append(d[cnt], s)
    }
    for _, v := range d {
        ans = append(ans, v)
    }
    return
}

Go

func groupAnagrams(strs []string) [][]string {
    d := map[string][]string{}
    for _, s := range strs {
        t := []byte(s)
        sort.Slice(t, func(i, j int) bool { return t[i] < t[j] })
        k := string(t)
        d[k] = append(d[k], s)
    }
    for _, v := range d {
        ans = append(ans, v)
    }
    return
}

```

[*] Sorting — Pattern Solution (matched from community answers)

Python

Sorting · LeetCode · By Putnam

— 361 —

LeetCode

```

# Python solution for grouping anagrams
from collections import defaultdict

def groupAnagrams(strs):
    # Create a dictionary to hold lists of anagrams; default to an empty list for new keys
    anagrams = defaultdict(list)
    # Iterate over each string in the input list
    for s in strs:
        # Sort the string to use as a key; sorted returns a list so join back to string
        key = ''.join(sorted(s))
        # Append the original string under the sorted key
        anagrams[key].append(s)
    # Return the grouped anagrams as a list of lists
    return list(anagrams.values())

# Example usage:
if __name__ == "__main__":
    strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
    result = groupAnagrams(strs)
    print(result)

```

#56

MEDIUM

Merge Intervals

LeetCode · Simplify · WalkCC · LeetCode · Editorial

Array · Sorting

List: Grind 169

Problem:

Given an array of intervals where intervals[i] = [starti, endi], merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

Input: intervals = [[1,3],[2,4],[8,10],[15,18]]

Output: [[1,4],[8,10],[15,18]]

Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,4].

Example 2:

Input: intervals = [[1,4],[4,5]]

Output: [[1,5]]

Explanation: Intervals [1,4] and [4,5] are considered overlapping.

Example 3:

Input: intervals = [[4,7],[1,4]]

Output: [[1,7]]

Explanation: Intervals [1,4] and [4,7] are considered overlapping.

Constraints:

- 1 <= intervals.length <= 104
- intervals[i].length == 2
- 0 <= starti <= endi <= 104

Sorting · LeetCode · By Putnam

— 362 —

LeetCode

```

>> Brute Force [Sorting] Interview Prep
Python

# Brute Force Approach
class Solution:
    def merge(self, intervals):
        # Brute Force O(n^2 log n) - repeatedly merge any two overlapping intervals
        intervals.sort(key=lambda x: x[0])
        merged = True
        while merged:
            merged = False
            result = intervals[:]
            for i in range(len(intervals)-1):
                if intervals[i][1] >= intervals[i+1][0]:
                    result[i][1] = max(result[i][1], intervals[i+1][1])
                    merged = True
            else:
                result.append(is)
            intervals = result
        return intervals

Python

def merge(intervals):
    # Edge cases: if the list is empty, return an empty list
    if not intervals:
        return []
    # Sort intervals based on the starting time
    intervals.sort(key=lambda x: x[0])
    merged = []
    for i in range(len(intervals)):
        # Traverse over each interval in the sorted list
        for interval in intervals:
            # If merged list is not empty and the current interval
            # overlaps with the last one in merged, merge them
            if merged and interval[0] <= merged[-1][1]:
                # Update the end of the last interval in merged if needed
                merged[-1][1] = max(merged[-1][1], interval[1])
            else:
                # No overlap, so add the current interval to merged
                merged.append(interval)
        return merged
# Example:
print(merge([[1,3],[2,4],[8,10],[15,18]]))

Python

```

Sorting · LeetCode · By Putnam

— 363 —

LeetCode

```

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ans = []
        for interval in sorted(intervals):
            if not ans or ans[-1][1] < interval[0]:
                ans.append(interval)
            else:
                ans[-1][1] = max(ans[-1][1], interval[1])
        return ans

Python

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = []
        st, ed = intervals[0]
        for s, e in intervals[1:]:
            if ed < s:
                ans.append([st, ed])
                st, ed = s, e
            else:
                ed = max(ed, e)
        ans.append([st, ed])
        return ans

Python

# Definition for an interval.
# class Interval(object):
#     def __init__(self, s=0, e=0):
#         self.start = s
#         self.end = e

class Solution(object):
    def merge(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: List[Interval]
        """
        ans = []
        for interval in sorted(intervals, key=lambda x: x.start):
            if ans and ans[-1].end >= interval.start:
                ans[-1].end = max(ans[-1].end, interval.end)
            else:
                ans.append(interval)
        return ans

####
"""
intervals = [[1,3],[2,4],[8,10],[15,18]]
intervals.sort()

```

Sorting · LeetCode · By Putnam

— 364 —

LeetCode

```

intervals = [[1, 3], [2, 4], [8, 10], [15, 18], [22, 23]]

class Solution {
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort() # default sort also ok
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:
                ans[-1][1] = max(ans[-1][1], e)
        return ans
}

C++

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        vector<vector<int>> ans;
        ranges::sort(intervals);
        for (const vector<int>& interval : intervals)
            if (ans.empty() || ans.back()[1] < interval[0])
                ans.push_back(interval);
            else
                ans.back()[1] = max(ans.back()[1], interval[1]);
        return ans;
    }
};

C++

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        int st = intervals[0][0], ed = intervals[0][1];
        vector<vector<int>> ans;
        for (int i = 1; i < intervals.size(); ++i) {
            if (ed < intervals[i][0]) {
                ans.push_back({st, ed});
                st = intervals[i][0];
                ed = intervals[i][1];
            } else {
                ed = max(ed, intervals[i][1]);
            }
        }
        ans.push_back({st, ed});
        return ans;
    }
};

C++

```

Sorting · LeetCode · By Putnam

— 365 —

LeetCode

```

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> ans;
        ans.emplace_back(intervals[0]);
        for (int i = 1; i < intervals.size(); ++i) {
            if (ans.back()[1] < intervals[i][0]) {
                ans.emplace_back(intervals[i]);
            } else {
                ans.back()[1] = max(ans.back()[1], intervals[i][1]);
            }
        }
        return ans;
    }
};

Java

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));
        for (int[] interval : intervals)
            if (ans.isEmpty() || ans.get(ans.size() - 1)[1] < interval[0])
                ans.add(interval);
            else
                ans.get(ans.size() - 1)[1] = Math.max(ans.get(ans.size() - 1)[1], interval[1]);
        return ans.toArray(int[][]::new);
    }
}

Java

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, Comparator.comparingInt(a -> a[0]));
        int st = intervals[0][0], ed = intervals[0][1];
        List<int[]> ans = new ArrayList<>();
        for (int i = 1; i < intervals.length; ++i) {
            int s = intervals[i][0], e = intervals[i][1];
            if (ed < s) {
                ans.add(new int[] {st, ed});
                st = s;
                ed = e;
            } else {
                ed = Math.max(ed, e);
            }
        }
        ans.add(new int[] {st, ed});
        return ans.toArray(new int[ans.size()][]);
    }
}

Java

```

Sorting · LeetCode · By Putnam

— 366 —

LeetCode

```

public class Solution {
    public int[] mergeSort(int[] intervals) {
        intervals = intervals.toArray();
        int st = intervals[0][0], ed = intervals[0][1];
        var ans = new List<int>();
        for (int i = 1; i < intervals.Length; ++i) {
            if (ed < intervals[i][0]) {
                ans.Add(new int[] { st, ed });
                st = intervals[i][0];
                ed = intervals[i][1];
            } else {
                ed = Math.Max(ed, intervals[i][1]);
            }
        }
        ans.Add(new int[] { st, ed });
        return ans.ToArray();
    }
}

```

Java

```

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int> ans = new ArrayList<>();
        ans.add(intervals[0]);
        for (int i = 1; i < intervals.Length; ++i) {
            int st = intervals[i][0], e = intervals[i][1];
            if (ans.getLast().get(1) < e) {
                ans.add(intervals[i]);
            } else {
                ans.set(ans.size() - 1, 1] = Math.max(ans.get(ans.size() - 1][1], e);
            }
        }
        return ans.toArray(new int[ans.size()][]);
    }
}

```

Java

```

public class Solution {
    public int[][] MergeSort(int[][] intervals) {
        intervals = intervals.OrderBy(a => a[0]).ToArray();
        var ans = new List<int>();
        ans.Add(intervals[0]);
        for (int i = 1; i < intervals.Length; ++i) {
            if (ans.Last().Count - 1][1] < intervals[i][0]) {
                ans.Add(intervals[i]);
            } else {
                ans[ans.Count - 1][1] = Math.Max(ans[ans.Count - 1][1], intervals[i][1]);
            }
        }
        return ans.ToArray();
    }
}

```

JavaScript

JavaScript

Sorting - LeetCode - By Putnam

— 367 —

LeetCode

```

//
// @param {number[][]} intervals
// @return {number[][]}
//
var merge = function(intervals) {
    intervals.sort((a, b) => a[0] - b[0]);
    const result = [];
    const n = intervals.length;
    let i = 0;
    while (i < n) {
        const left = intervals[i][0];
        let right = intervals[i][1];
        while (true) {
            i++;
            if (i < n && right < intervals[i][0]) {
                right = Math.max(right, intervals[i][1]);
            } else {
                result.push([left, right]);
                break;
            }
        }
    }
    return result;
};

```

TypeScript

TypeScript

```

function merge(intervals: number[][]): number[][] {
    intervals.sort((a, b) => a[0] - b[0]);
    const ans: number[][] = [];
    let [st, ed] = intervals[0];
    for (const [a, b] of intervals.slice(1)) {
        if (ed < a) {
            ans.push([st, ed]);
            [st, ed] = [a, b];
        } else {
            ed = Math.max(ed, b);
        }
    }
    ans.push([st, ed]);
    return ans;
}

```

TypeScript

```

function merge(intervals: number[][]): number[][] {
    intervals.sort((a, b) => a[0] - b[0]);
    const ans: number[][] = [intervals[0]];
    for (let i = 1; i < intervals.length; ++i) {
        if (ans.at(-1)[1] < intervals[i][0]) {
            ans.push(intervals[i]);
        } else {
            ans.at(-1)[1] = Math.max(ans.at(-1)[1], intervals[i][1]);
        }
    }
    return ans;
}

```

Sorting - LeetCode - By Putnam

— 368 —

LeetCode

```

Go
func merge(intervals [][]int) (ans [][]int) {
    sort.Slice(intervals, func(i, j) bool {
        return intervals[i][0] < intervals[j][0]
    })
    st, ed := intervals[0][0], intervals[0][1]
    for _, a := range intervals[1:] {
        if ed < a[0] {
            ans = append(ans, []int{st, ed})
            st, ed = a[0], a[1]
        } else if ed < a[1] {
            ed = a[1]
        }
    }
    ans = append(ans, []int{st, ed})
    return ans
}

```

Go

```

func merge(intervals [][]int) (ans [][]int) {
    sort.Slice(intervals, func(i, j) bool { return intervals[i][0] < intervals[j][0] })
    ans = append(ans, intervals[0])
    for _, a := range intervals[1:] {
        if ans[len(ans)-1][1] < a[0] {
            ans = append(ans, a)
        } else {
            ans[len(ans)-1][1] = max(ans[len(ans)-1][1], a[1])
        }
    }
    return ans
}

```

Rust

Rust

```

impl Solution {
    pub fn merge(intervals: Vec<Vec<i32>>) -> Vec<Vec<i32>> {
        intervals.sort_unstable_by(|a, b| a[0].cmp(b[0]));
        let n = intervals.len();
        let mut res = vec![];
        let mut i = 0;
        while i < n {
            let l = intervals[i][0];
            let mut r = intervals[i][1];
            i += 1;
            while i < n && r <= intervals[i][0] {
                r = r.max(intervals[i][1]);
                i += 1;
            }
            res.push(vec![l, r]);
            i = res.len();
        }
        res
    }
}

```

Rust

Sorting - LeetCode - By Putnam

— 369 —

LeetCode

```

impl Solution {
    pub fn merge(intervals: Vec<Vec<i32>>) -> Vec<Vec<i32>> {
        intervals.sort_unstable_by(|a, b| a[0].cmp(b[0]));
        let n = intervals.len();
        let mut res = vec![];
        let mut i = 0;
        while i < n {
            let l = intervals[i][0];
            let mut r = intervals[i][1];
            i += 1;
            while i < n && r <= intervals[i][0] {
                r = r.max(intervals[i][1]);
                i += 1;
            }
            res.push(vec![l, r]);
            i = res.len();
        }
        res
    }
}

```

Kotlin

Kotlin

```

class Solution {
    fun merge(intervals: Array<IntRange>): Array<IntRange> {
        intervals.sortBy { it.start }
        val result = mutableListOf<IntRange>()
        var i = intervals.size
        var i = 0
        while (i < n) {
            var left = intervals[i][0]
            var right = intervals[i][1]
            while (true) {
                i++
                if (i < n && right < intervals[i][0]) {
                    right = maxOf(right, intervals[i][1])
                } else {
                    result.add(IntRange(left, right))
                    break
                }
            }
            return result.toList()
        }
    }
}

```

[*] Sorting — Pattern Solution (matched from community answers)

Python

Sorting - LeetCode - By Putnam

— 370 —

LeetCode

```

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ans = []
        for interval in sorted(intervals):
            if not ans or ans[-1][1] < interval[0]:
                ans.append(interval)
            else:
                ans[-1][1] = max(ans[-1][1], interval[1])
        return ans

```

#1152

MEDIUM

Analyze User Website Visit Pattern

LeetCode - UnionFind - RustCC - LeetCode - Editorial

Array - Hash Table - Sorting

Like: Premium 98

Problem:

You are given two string arrays username and website and an integer array timestamp. All the given arrays are of the same length and the tuple [username[i], website[i], timestamp[i]] indicates that the user username[i] visited the website website[i] at time timestamp[i].

A pattern is a list of three websites (not necessarily distinct).

• For example, ["home", "away", "love"], ["leetcode", "love", "leetcode"], and ["luffy", "luffy", "luffy"] are all patterns.

The score of a pattern is the number of users that visited all the websites in the pattern in the same order they appeared in the pattern.

• For example, if the pattern is ["home", "away", "love"], the score is the number of users x such that x visited "home" then visited "away" and visited "love" after that.
• Similarly, if the pattern is ["leetcode", "love", "leetcode"], the score is the number of users x such that x visited "leetcode" then visited "love" and visited "leetcode" one more time after that.
• Also, if the pattern is ["luffy", "luffy", "luffy"], the score is the number of users x such that x visited "luffy" three different times at different timestamps.

Return the pattern with the largest score. If there is more than one pattern with the same largest score, return the lexicographically smallest such pattern.

Note that the websites in a pattern do not need to be visited contiguously, they only need to be visited in the order they appeared in the pattern.

Example 1:

```

Input: username = ["joe", "joe", "joe", "james", "james", "james", "mary", "mary", "mary"],
timestamp = [1,2,3,4,5,6,7,8,9,10], website = ["home", "about", "career", "home", "cart", "maps", "home", "home", "about", "career"]
Output: ["home", "about", "career"]
Explanation: The tuples in this example are:
["joe", "home", 1], ["joe", "about", 2], ["joe", "career", 3], ["james", "home", 4], ["james", "cart", 5], ["james", "maps", 6], ["james", "home", 7], ["mary", "home", 8], ["mary", "about", 9], and

```

Sorting - LeetCode - By Putnam

— 371 —

LeetCode

```

["mary", "career", 10].
The pattern ("home", "about", "career") has score 2 (joe and mary).
The pattern ("home", "cart", "maps") has score 1 (james).
The pattern ("home", "cart", "home") has score 1 (james).
The pattern ("home", "maps", "home") has score 1 (james).
The pattern ("cart", "maps", "home") has score 1 (james).
The pattern ("home", "home", "home") has score 0 (no user visited home 3 times).

```

Example 2:

```

Input: username = ["ua", "ua", "ua", "ub", "ub", "ub"], timestamp = [1,2,3,4,5,6], website = ["a", "b", "a", "a", "b", "c"]
Output: ["a", "b", "a"]

```

Constraints:

- 3 <= username.length <= 50
- 1 <= username[i].length <= 10
- timestamp.length == username.length
- 1 <= timestamp[i] <= 109
- website.length == username.length
- 1 <= website[i].length <= 10
- username[i] and website[i] consist of lowercase English letters.
- It is guaranteed that there is at least one user who visited at least three websites.
- All the tuples [username[i], timestamp[i], website[i]] are unique.

>> Brute Force (Sorting) Interview Prep

Python

```

# Brute Force Interview
class Solution:
    def mostVisitedPattern(self, username, timestamp, website):
        if Brute Force (2) - bubble sort
        arr = list(zip(username, timestamp, website))
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                if arr[i][0] < arr[j][0]:
                    arr[i], arr[j] = arr[j], arr[i]
        return arr

```

Python

Python

Sorting - LeetCode - By Putnam

— 372 —

LeetCode

```
# Python code solution for Analyze User Website Visit Patterns

from collections import defaultdict
from itertools import combinations

def mostVisitedPatterns(username, timestamp, website):
    # Combine the logs into a list of tuples and sort by timestamp.
    logs = sorted(zip(timestamp, username, website))

    # Dictionary to hold each user's list of website visits
    user_visits = defaultdict(list)
    for time, user, site in logs:
        user_visits[user].append(site)

    # Dictionary to map a pattern (tuple of 3 websites) to set of users who visited that pattern
    pattern_users = defaultdict(set)

    # For each user, generate all unique 3-sequences from their visit list and update pattern_users
    for user, sites in user_visits.items():
        # Use set to avoid creating duplicate patterns from the same user multiple times
        seen_patterns = set()
        if len(sites) < 3:
            continue
        # Generate all 3-sequences combinations (order matters since visits are in time order)
        for pattern in combinations(sites, 3):

            if pattern not in seen_patterns:
                seen_patterns.add(pattern)
                pattern_users[pattern].add(user)

    # Find the pattern with the highest score (i.e., most users)
    # It is one of 0 to len(timestamp) unique pattern seen.
    max_count = 0
    result = None
    for pattern, users in pattern_users.items():
        count = len(users)
        if count > max_count or (count == max_count and (result is None or pattern < result)):
            max_count = count
            result = pattern

    return list(result)

# Example usage
print(mostVisitedPatterns(
    ["joe","joe","joe","james","james","james","james","mary","mary","mary"],
    [1,2,3,4,5,6,7,8,9,10],
    ["home","about","career","home","cart","maps","home","home","about","career"]
))
```

Python

```
class Solution:
    def mostVisitedPatterns(
        self,
        username: List[str],
        timestamp: List[int],
        website: List[str],
    ) -> List[str]:
        userVisits = collections.defaultdict(list)

        # Sort websites of each user by timestamp.
        for user, _ in sorted(
            zip(username, timestamp, website),
            key=lambda x: x[1]):
            userVisits[user].append(site)

        # For each of three websites, count its frequency.
        patternCount = collections.Counter()
        for user, sites in userVisits.items():
            patternCount.update(Counter(itertools.combinations(sites, 3)))

        return max(sorted(patternCount), key=patternCount.get)
```

Python

```
class Solution:
    def mostVisitedPatterns(
        self, username: List[str], timestamp: List[int], website: List[str]
    ) -> List[str]:
        d = defaultdict(list)
        for user, _ in sorted(
            zip(username, timestamp, website), key=lambda x: x[1]
        ):
            d[user].append(site)

        cnt = Counter()
        for sites in d.values():
            m = len(sites)
            s = set()
            if m > 2:
                for i in range(m - 2):
                    for j in range(i + 1, m - 1):
                        for k in range(j + 1, m):
                            s.add(tuple(sorted(sites[i], sites[j], sites[k])))

            for t in s:
                cnt[t] += 1
        return sorted(cnt.items(), key=lambda x: (-x[1], x[0]))[0][0]
```

Python

```
class Solution {
    def mostVisitedPatterns(
        self, username: List[str], timestamp: List[int], website: List[str]
    ) -> List[str]:
        d = defaultdict(list)
        for user, _ in sorted(
            zip(username, timestamp, website), key=lambda x: x[1]
        ):
            d[user].append(site)

        cnt = Counter()
        for sites in d.values():
            m = len(sites)
            s = set()
            if m > 2:
                for i in range(m - 2):
                    for j in range(i + 1, m - 1):
                        for k in range(j + 1, m):
                            s.add(tuple(sorted(sites[i], sites[j], sites[k])))

            for t in s:
                cnt[t] += 1
        return sorted(cnt.items(), key=lambda x: (-x[1], x[0]))[0][0]
```

C++

C++

```
class Solution {
public:
    vector<string> mostVisitedPatterns(vector<string>& username, vector<int>& timestamp, vector<string>& website) {
        unordered_map<string, vector<pair<int, string>>& d;
        int n = username.size();
        for (int i = 0; i < n; ++i) {
            auto user = username[i];
            auto site = website[i];
            d[user].emplace_back(i, site);
        }
        unordered_map<string, int> cnt;
        for (auto& [_, sites] : d) {
            int m = sites.size();
            unordered_set<string> s;
            if (m > 2) {
                sort(sites.begin(), sites.end());
                for (int i = 0; i < m - 2; ++i) {
                    for (int j = i + 1; j < m - 1; ++j) {
                        for (int k = j + 1; k < m; ++k) {
                            s.insert(sites[i].second + "," + sites[j].second + "," + sites[k].second);
                        }
                    }
                }
            }
        }
    }
};
```

```
for (auto& t : s) {
    cnt[t]++;
}

int m = 0;
string t = "";
for (auto& [p, v] : cnt) {
    if (m < v || (m == v && t > p)) {
        m = v;
        t = p;
    }
}
return split(t, ',' );

vector<string> split(string& s, char c) {
    vector<string> res;
    string& item = s;
    string t;
    while (getline(item, t, c)) {
        res.push_back(t);
    }
    return res;
}
```

Java

Java

```
class Solution {
    public List<String> mostVisitedPatterns(String[] username, int[] timestamp, String[] website) {
        Map<String, List<Node>> d = new HashMap<>();
        int n = username.length;
        for (int i = 0; i < n; ++i) {
            String user = username[i];
            int ts = timestamp[i];
            String site = website[i];
            d.computeIfAbsent(user, k -> new ArrayList<>()).add(new Node(ts, user, site));
        }
        Map<String, Integer> cnt = new HashMap<>();
        for (var sites : d.values()) {
            int m = sites.size();
            Set<String> s = new HashSet<>();
            if (m > 2) {
                Collections.sort(sites, (a, b) -> a.ts - b.ts);
                for (int i = 0; i < m - 2; ++i) {
                    for (int j = i + 1; j < m - 1; ++j) {
                        for (int k = j + 1; k < m; ++k) {
                            s.add(sites.get(i).site + "," + sites.get(j).site + ","
                                + sites.get(k).site);
                        }
                    }
                }
            }
        }
    }
}
```

```
for (String t : s) {
    cnt.put(t, cnt.getOrDefault(t, 0) + 1);
}

int m = 0;
String t = "";
for (var a : cnt.entrySet()) {
    if (m < a.getValue() || (m == a.getValue() && a.getKey().compareTo(t) < 0)) {
        m = a.getValue();
        t = a.getKey();
    }
}
return Arrays.asList(t.split(","));

class Node {
    String user;
    int ts;
    String site;

    Node(String user, int ts, String site) {
        this.user = user;
        this.ts = ts;
        this.site = site;
    }
}
```

Go

Go

```
func mostVisitedPatterns(username []string, timestamp []int, website []string) []string {
    d := map[string][]pair{}
    for i, user := range username {
        ts := timestamp[i]
        site := website[i]
        d[user] = append(d[user], pair{ts, site})
    }
    cnt := map[string]int{}
    for _, sites := range d {
        m := len(sites)
        s := map[string]bool{}
        if m > 2 {
            sort.Slice(sites, func(i, j int) bool { return sites[i].ts < sites[j].ts })
            for i := 0; i < m-2; i++ {
                for j := i + 1; j < m-1; j++ {
                    for k := j + 1; k < m; k++ {
                        s[sites[i].site+","+sites[j].site+","+sites[k].site] = true
                    }
                }
            }
        }
        for t := range s {
            cnt[t]++
        }
    }
}
```

```
me, t := 0, ""
for p, v := range cnt {
    if m < v || (m == v && p > t) {
        m = v
        t = p
    }
}
return strings.Split(t, ",")

type pair struct {
    ts int
    site string
}
```

[1] Sorting — Pattern Solution (matched from community answers)

Python

```
# Python code solution for Analyze User Website Visit Patterns

from collections import defaultdict
from itertools import combinations

def mostVisitedPatterns(username, timestamp, website):
    # Combine the logs into a list of tuples and sort by timestamp.
    logs = sorted(zip(timestamp, username, website))

    # Dictionary to hold each user's list of website visits
    user_visits = defaultdict(list)
    for time, user, site in logs:
        user_visits[user].append(site)

    # Dictionary to map a pattern (tuple of 3 websites) to set of users who visited that pattern
    pattern_users = defaultdict(set)

    # For each user, generate all unique 3-sequences from their visit list and update pattern_users
    for user, sites in user_visits.items():
        # Use set to avoid creating duplicate patterns from the same user multiple times
        seen_patterns = set()
        if len(sites) < 3:
            continue
        # Generate all 3-sequences combinations (order matters since visits are in time order)
        for pattern in combinations(sites, 3):
```

```

    if pattern not in seen_patterns:
        seen_patterns.add(pattern)
        pattern_users[pattern].add(user)

# Find the pattern with the highest count (i.e., most users)
# In case of a tie, lexicographically smallest pattern wins.
max_count = 0
result = None
for pattern, users in pattern_users.items():
    count = len(users)
    if count > max_count or (count == max_count and (result is None or pattern < result)):
        max_count = count
        result = pattern

return list(result)

# Example usage
print(sortMostVisitedPatterns(
    ["joe", "joe", "joe", "james", "james", "james", "mary", "mary", "mary"],
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]))
# Output: ["joe", "joe", "joe", "james", "james", "james", "mary", "mary", "mary"]
})
```

Sorting · LeetCodeMastery — By Pattern

— 379 —

LeetCodeMastery

Quick Review — Sorting 9 questions · insights · complexity · solution

#169 Majority Element [Easy]

Key Insights

- The majority element appears more than half the length of the array.
- A simple hash table approach can count occurrences, but a more efficient solution exists.
- The Boyer-Moore Voting Algorithm finds the majority in $O(n)$ time and $O(1)$ space.
- The algorithm maintains a candidate and a counter, adjusting the candidate when the count drops to zero.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We use the Boyer-Moore Voting Algorithm to identify the majority element. The algorithm iterates through the array using two variables: one for the candidate and one for the count. Initially, we set the candidate to an arbitrary value and count to 0. For each element, if the candidate is 0, we update the candidate to the current element. Then, if the current element is the same as the candidate, we increment the count; otherwise, we decrement it. Since the majority element appears more than $n/2$ times, it will remain as the candidate by the end of the iteration.

#217 Contains Duplicate [Easy]

Key Insights

- A hash set can be used to keep track of seen numbers.
- Iterating once through the array provides an efficient solution.
- If a number is already in the set, a duplicate is found.
- Alternative approaches like sorting have a higher time complexity ($O(n \log n)$).

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution uses a hash set to track the numbers seen while iterating through the array. For each number in the array, we check if it exists in the hash set. If it does, return true immediately as a duplicate is found. Otherwise, add the number to the hash set. If the iteration completes without finding any duplicates, return false. This approach efficiently determines if any duplicates exist in a single pass.

#242 Valid Anagram [Easy]

Key Insights

- If s and t have different lengths, they cannot be anagrams.
- An anagram requires exactly the same character frequency for both strings.
- Using a hash table (or frequency array when limited to 26 lowercase letters) provides an efficient solution.
- An alternative approach is to sort both strings and compare them.
- For Unicode inputs, a hash table is more adaptable than a fixed-size frequency array.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings (using a hash table for counting).

Sorting · LeetCodeMastery — By Pattern

— 380 —

LeetCodeMastery

Space Complexity: $O(1)$ if we use a fixed-size array (for 26 letters), otherwise $O(n)$ for a hash table with a larger character set.

Solution

We use a hash table (dictionary) to count the frequency of each character in the first string s. Then, we iterate over string t and decrement the count for each character. If at any point a character count goes negative or a character is missing in the hash table, t cannot be an anagram of s. Finally, if all counts return to zero, the strings are anagrams. This approach is efficient and handles the problem in linear time.

#252 Meeting Rooms [Easy]

Key Insights

- Sort the intervals based on their starting times.
- Compare each interval's end time with the next interval's start time.
- Detect overlap if the end time of one meeting exceeds the start time of the following meeting.
- If no overlaps are detected, all meetings can be attended.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(1)$ if sorting is done in-place; otherwise $O(n)$.

Solution

The solution starts by sorting the intervals in ascending order by their start times. After sorting, iterate through the intervals and for each consecutive pair, check if the previous meeting's end time exceeds the next meeting's start time. An overlap indicates that the person cannot attend all meetings. The primary data structure used is an array (or list) for holding the intervals. Sorting makes it easier to compare adjacent intervals for potential overlaps.

#1086 Largest Unique Number [Easy]

Key Insights

- Use a hash table (or dictionary) to count the frequency of each number.
- Iterate over the hash table to find the numbers that appear exactly once.
- Identify the maximum number among the unique numbers.
- Return -1 if no unique number exists.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in nums.

Space Complexity: $O(n)$, for the hash table storing frequency counts.

Solution

The approach starts by initializing a hash table to keep track of the frequency of each element in the array. Once the frequency count is complete, we traverse the hash table to collect numbers that appear exactly once. Finally, we determine the maximum number among these unique elements. If there is no unique element, we return -1. This method is efficient because the frequency count and the search for the maximum unique element both run in linear time relative to the input size.

#1122 Relative Sort Array [Easy]

Key Insights

- Use a hash table (or array for counting given the value constraints) to count the occurrences of each element in arr1.
- Iterate through arr2 and for each element, append it to the result based on its count from the hash table.
- Remove the used elements from the hash table.
- Sort the remaining elements (those not in arr2) in ascending order and append them to the result.

Space & Time

Sorting · LeetCodeMastery — By Pattern

— 381 —

LeetCodeMastery

Time Complexity: $O(n + m \log m)$ where n is the length of arr1 and m is the number of leftover elements not in arr2. Given that element value is bounded (0 to 1000), a counting sort can make this nearly $O(n)$.

Space Complexity: $O(n)$ for storing the frequency counts and the result array.

Solution

The solution leverages a frequency counter (hash table) to count how many times each element appears in arr1. We then process arr2 to add each number in its given order according to its frequency. Finally, we sort the remaining numbers that are not in arr2 and append them to the result array. This approach ensures that the relative order specified by arr2 is maintained and that the other elements are sorted in ascending order.

#49 Group Anagrams [Medium]

Key Insights

- Anagrams have the same characters when sorted alphabetically.
- Sorting each string provides a unique key for grouping.
- Use a hash table (dictionary) to map each sorted key to a list of its original strings.
- Finally, gather all the lists from the dictionary to form the result.

Space & Time

Time Complexity: $O(n * k \log k)$, where n is the number of strings and k is the maximum length of a string (due to sorting each string).

Space Complexity: $O(n * k)$ for storing the grouped strings in the hash table.

Solution

The primary idea is to iterate through each string, sort it to obtain a key that represents its character composition, and then group the original string under this key in a hash table. After processing all strings, the values of the hash table will be the groups of anagrams. This approach efficiently groups anagrams by leveraging sorting and a dictionary for constant-time lookups.

#56 Merge Intervals [Medium]

Key Insights

- Sort the intervals based on their start values.
- Use a pointer or iteration to compare the current interval with the last merged interval.
- If the current interval overlaps with the previous one, merge them by updating the end time.
- Otherwise, add the current interval to the results as a new merged interval.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ to store the merged intervals.

Solution

First, sort the array of intervals based on the starting time. Then, iterate through the sorted list and check if the current interval overlaps with the last interval in the merged list. If an overlap is detected (i.e., the start of the current interval is less than or equal to the end of the last merged interval), update the last merged interval's end to be the maximum of both ends. If there is no overlap, simply add the current interval to the merged list. This approach efficiently merges overlapping intervals and ensures that all intervals are processed in a single pass after sorting.

#1152 Analyze User Website Visit Pattern [Medium]

Key Insights

- Combine the inputs into a single list of events and sort them by timestamp.
- Group the visits by each user, preserving the sorted order.
- Generate all unique combinations of 3-sequences for each user (order matters and non-contiguous visits are allowed).

Sorting · LeetCodeMastery — By Pattern

— 382 —

LeetCodeMastery

- Use a dictionary (or hash map) to map each 3-sequence pattern to the set of users who visited that pattern.
- The problem constraints are small, so generating combinations per user (even using triple nested loops) is acceptable.
- When multiple patterns have the same frequency, choose the lexicographically smallest one.

Space & Time

Time Complexity: $O(U * L^3)$ where U is the number of users and L is the maximum length of visits per user (since we generate combinations of 3 visits per user). Sorting all events costs $O(n \log N)$ for N total events.

Space Complexity: $O(P)$ where P is the number of unique 3-sequence patterns stored in the dictionary, plus $O(N)$ for grouping the events.

Solution

Aggregate the events by user while sorting them based on timestamp. For each user, generate all sets of 3-website patterns (to avoid duplicate patterns per user). Use a hash map to count the number of unique users for each pattern. Traverse the map to find the pattern with the highest count. In case of a tie, compare patterns lexicographically. Return the pattern that meets the criteria.

Sorting · LeetCodeMastery — By Pattern

— 383 —

LeetCodeMastery

#7 Bit Manipulation — 10 questions · #7 of 21

#67 EASY

Add Binary

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Math · String · Bit Manipulation · Simulation

Lists Grid 169

Problem:

Given two binary strings a and b, return their sum as a binary string.

Example 1:

Input: a = "11", b = "1"

Output: "100"

Example 2:

Input: a = "1010", b = "1011"

Output: "10101"

Constraints:

- $1 \leq a.length, b.length \leq 104$
- a and b consist only of '0' or '1' characters.
- Each string does not contain leading zeros except for the zero itself.

Python

Python

```
# Function to add two binary strings.
def addBinary(a: str, b: str) -> str:
    i = len(a) - 1
    j = len(b) - 1
    carry = 0
    result = []

    # Loop until both strings are processed and no carry remains.
    while i >= 0 or j >= 0 or carry:
        total_sum = carry
        if i >= 0:
            total_sum += int(a[i])
            i -= 1
        if j >= 0:
            total_sum += int(b[j])
            j -= 1
        result.append(str(total_sum % 2))
        carry = total_sum // 2
    return ''.join(result[::-1])
```

Sorting · LeetCodeMastery — By Pattern

— 384 —

LeetCodeMastery

```
# Sample test cases
print(addBinary("11", "1")) # Expected output: "100"
print(addBinary("1010", "1011")) # Expected output: "10101"
```

Python

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        carry = 0
        i = len(a) - 1
        j = len(b) - 1

        while i >= 0 or j >= 0 or carry:
            if i >= 0:
                carry += int(a[i])
            if j >= 0:
                carry += int(b[j])
            j -= 1
            i -= 1
            ans.append(str(carry % 2))
            carry //= 2

        return ''.join(reversed(ans))
```

Python

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        i, j, carry = len(a) - 1, len(b) - 1, 0
        while i >= 0 or j >= 0 or carry:
            i -= 1 if i >= 0 else int(a[i]) + (0 if j < 0 else int(b[j]))
            carry, v = divmod(carry, 2)
            ans.append(str(v))
            i, j = i < 0, j < 0
        return ''.join(ans[::-1])
```

Python

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        i, j, carry = len(a) - 1, len(b) - 1, 0
        while i >= 0 or j >= 0 or carry:
            carry += (a[i] if i >= 0 else int(a[i])) + (0 if j < 0 else int(b[j]))
            carry, v = divmod(carry, 2)
            ans.append(str(v))
            i, j = i - 1, j - 1
        return ''.join(ans[::-1])
```

Java

```
Java
```

Bit Manipulation · LeetCode · By Pattern

— 385 —

LeetCode

```
class Solution {
    public String addBinary(String a, String b) {
        StringBuilder sb = new StringBuilder();
        int carry = 0;
        int i = a.length() - 1;
        int j = b.length() - 1;

        while (i >= 0 || j >= 0 || carry == 1) {
            if (i >= 0)
                carry += a.charAt(i--) - '0';
            if (j >= 0)
                carry += b.charAt(j--) - '0';
            sb.append(carry % 2);
            carry /= 2;
        }

        return sb.reverse().toString();
    }
}
```

Java

```
class Solution {
    public String addBinary(String a, String b) {
        var sb = new StringBuilder();
        int i = a.length() - 1, j = b.length() - 1;
        for (int carry = 0, k = 0; i >= 0 || j >= 0 || carry > 0; --i, --j) {
            carry += (i >= 0 ? a.charAt(i) - '0' : 0) + (j >= 0 ? b.charAt(j) - '0' : 0);
            sb.append(carry % 2);
            carry /= 2;
        }
        return sb.reverse().toString();
    }
}
```

Java

```
public class Solution {
    public String addBinary(String a, String b) {
        int i = a.length() - 1;
        int j = b.length() - 1;
        var sb = new StringBuilder();
        for (int carry = 0, k = 0; i >= 0 || j >= 0 || carry > 0; --i, --j) {
            carry += (i >= 0 ? a[i] - '0' : 0);
            carry += (j >= 0 ? b[j] - '0' : 0);
            sb.append(carry % 2);
            carry /= 2;
        }
        var ans = sb.toString().toCharArray();
        Array.Reverse(ans);
        return new String(ans);
    }
}
```

Java

Bit Manipulation · LeetCode · By Pattern

— 386 —

LeetCode

```
class Solution {
    public String addBinary(String a, String b) {
        var sb = new StringBuilder();
        int i = a.length() - 1, j = b.length() - 1;
        for (int carry = 0, k = 0; i >= 0 || j >= 0 || carry > 0; --i, --j) {
            carry += (i >= 0 ? a.charAt(i) - '0' : 0) + (j >= 0 ? b.charAt(j) - '0' : 0);
            sb.append(carry % 2);
            carry /= 2;
        }
        return sb.reverse().toString();
    }
}
```

Java

```
public class Solution {
    public String addBinary(String a, String b) {
        int i = a.length() - 1;
        int j = b.length() - 1;
        var sb = new StringBuilder();
        for (int carry = 0, k = 0; i >= 0 || j >= 0 || carry > 0; --i, --j) {
            carry += (i >= 0 ? a[i] - '0' : 0);
            carry += (j >= 0 ? b[j] - '0' : 0);
            sb.append(carry % 2);
            carry /= 2;
        }
        var ans = sb.toString().toCharArray();
        Array.Reverse(ans);
        return new String(ans);
    }
}
```

TypeScript

```
TypeScript
function addBinary(a: string, b: string): string {
    let i = a.length - 1;
    let j = b.length - 1;
    const ans: number[] = [];
    for (let carry = 0, k = 0; i >= 0 || j >= 0 || carry; --i, --j) {
        carry += (i >= 0 ? a[i] : '0').charCodeAt(0) - '0'.charCodeAt(0);
        carry += (j >= 0 ? b[j] : '0').charCodeAt(0) - '0'.charCodeAt(0);
        ans.push(carry % 2);
        carry = Math.floor(carry / 2);
    }
    return ans.reverse().join("");
}
```

TypeScript

Bit Manipulation · LeetCode · By Pattern

— 387 —

LeetCode

```
function addBinary(a: string, b: string): string {
    let i = a.length - 1;
    let j = b.length - 1;
    let ans: number[] = [];
    for (let carry = 0, k = 0; i >= 0 || j >= 0 || carry; --i, --j) {
        carry += (i >= 0 ? a[i] : '0').charCodeAt(0) - '0'.charCodeAt(0);
        carry += (j >= 0 ? b[j] : '0').charCodeAt(0) - '0'.charCodeAt(0);
        ans.push(carry % 2);
        carry = Math.floor(carry / 2);
    }
    return ans.reverse().join("");
}
```

Rust

```
Rust
impl Solution {
    pub fn add_binary(a: String, b: String) -> String {
        let mut i = (a.len() as i32) - 1;
        let mut j = (b.len() as i32) - 1;
        let mut ans = String::new();
        let a = a.as_bytes();
        let b = b.as_bytes();
        while i >= 0 || j >= 0 || carry > 0 {
            if i >= 0 {
                carry += a[i as usize] - b'0';
                i -= 1;
            }
            if j >= 0 {
                carry += b[j as usize] - b'0';
                j -= 1;
            }
            ans.push_str(&(carry % 2).to_string());
            carry /= 2;
        }
        ans.chars().rev().collect()
    }
}
```

Rust

Bit Manipulation · LeetCode · By Pattern

— 388 —

LeetCode

```
impl Solution {
    pub fn add_binary(a: String, b: String) -> String {
        let mut i = (a.len() as i32) - 1;
        let mut j = (b.len() as i32) - 1;
        let mut carry = 0;
        let mut ans = String::new();
        let a = a.as_bytes();
        let b = b.as_bytes();
        while i >= 0 || j >= 0 || carry > 0 {
            if i >= 0 {
                carry += a[i as usize] - b'0';
                i -= 1;
            }
            if j >= 0 {
                carry += b[j as usize] - b'0';
                j -= 1;
            }
            ans.push_str(&(carry % 2).to_string());
            carry /= 2;
        }
        ans.chars().rev().collect()
    }
}
```

[*] Bit Manipulation — Pattern Solution (stored optimal)

Python

```
# Function to add two binary strings
def addBinary(a: str, b: str) -> str:
    i = len(a) - 1 # Pointer for string a
    j = len(b) - 1 # Pointer for string b
    carry = 0 # Initialize carry to 0
    result = [] # List to store binary digits

    # Loop until both strings are processed and no carry remains
    while i >= 0 or j >= 0 or carry:
        total_sum = carry # Start with the carry from previous step

        if i >= 0: # If there is a digit left in a
            total_sum += int(a[i]) # Convert character to integer and add
            i -= 1 # Move to the next digit

        if j >= 0: # If there is a digit left in b
            total_sum += int(b[j]) # Convert character to integer and add
            j -= 1 # Move to the next digit

        # Append the remainder of total_sum divided by 2 (0 or 1)
        result.append(str(total_sum % 2))
        carry = total_sum // 2 # Update the carry

    # Reverse the result since we built it backwards and join to form a string
    return ''.join(result[::-1])
```

C++

Bit Manipulation · LeetCode · By Pattern

— 389 —

LeetCode

```
// C++ solution for adding two binary strings
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

string addBinary(string a, string b) {
    int i = a.size() - 1; // Pointer for string a
    int j = b.size() - 1; // Pointer for string b
    int carry = 0; // Initialize the carry
    string result = ""; // String to store result in reverse order

    // Loop until both strings and the carry have been fully processed
    while (i >= 0 || j >= 0 || carry) {
        int totalSum = carry; // Start with the carry
        if (i >= 0) {
            totalSum += a[i] - '0'; // Convert char to integer and add
            i--; // Move left in string a
        }
        if (j >= 0) {
            totalSum += b[j] - '0'; // Convert char to integer and add
            j--; // Move left in string b
        }
        // Append the current sum to result (in reverse order)
        result.push_back(totalSum % 2 + '0');
        carry = totalSum / 2; // Calculate the new carry
    }

    // Reverse the result to correct the order of digits
    reverse(result.begin(), result.end());
    return result;
}
```

int main() {

```
// Sample test cases
cout << addBinary("11", "1") << endl; // Expected output: "100"
cout << addBinary("1010", "1011") << endl; // Expected output: "10101"
return 0;
}
```

#136 EASY

Single Number

LeetCode · Simplex · WalkCC · LeetCode · Editorial

Array · Bit Manipulation

Lists: Grind 169

Problem:

Given a non-empty array of integers nums, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: nums = [2,2,1]

Bit Manipulation · LeetCode · By Pattern

— 390 —

LeetCode

Output: 1
Example 2:
Input: nums = [4,1,2,1,2]
Output: 4
Example 3:
Input: nums = [1]
Output: 1
Constraints:

- 1 <= nums.length <= 3 * 10⁴
- -3 * 10⁴ <= nums[i] <= 3 * 10⁴
- Each element in the array appears twice except for one element which appears only once.

>> Brute Force (Bit Manipulation) Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def singleNumber(self, nums):
        # Brute Force O(n^2) ~ For each element, count its occurrences
        for num in nums:
            if nums.count(num) == 1:
                return num
```

Python

```
Python
# Function to find the single number in the list
def singleNumber(nums):
    # Initialize result to 0; any number XOR 0 is the number itself
    result = 0
    # Iterate through every number in the list
    for num in nums:
        # XOR the current result with the current number
        result ^= num # Using XOR to cancel duplicates
    # Return the unique number that did not cancel out
    return result

# Example usage
nums = [4, 1, 2, 1, 2]
print(singleNumber(nums)) # Output should be 4
```

```
Python
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return functools.reduce(operator.xor, nums, 0)
```

```
Python
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return reduce(operator.xor, nums)
```

Python

```
...
# Use functools import reduce
#>> reduce(lambda x, y: x ^ y, [1, 5, 3])
5
...

from functools import reduce

class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return reduce(lambda x, y: x ^ y, nums)

#####

class Solution(object):
    def singleNumber(self, nums):
        (type nums, List[int])
        (type ans, int)
        for i in range(1, len(nums)):
            nums[i] ^= nums[i]
        return nums[0]
```

C++

```
C++
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans = 0;

        for (const int num : nums)
            ans ^= num;

        return ans;
    }
};
```

C++

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans = 0;
        for (int v : nums) {
            ans ^= v;
        }
        return ans;
    }
};
```

C++

```
class Solution {
    func singleNumber(_ nums: [Int]) -> Int {
        return nums.reduce(0, ^)
    }
}
```

C++

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans = 0;
        for (int v : nums) {
            ans ^= v;
        }
        return ans;
    }
};
```

C++

```
class Solution {
    func singleNumber(_ nums: [Int]) -> Int {
        return nums.reduce(0, ^)
    }
}
```

Java

```
Java
class Solution {
    public int singleNumber(int[] nums) {
        int ans = 0;

        for (final int num : nums)
            ans ^= num;

        return ans;
    }
}
```

Java

```
class Solution {
    public int singleNumber(int[] nums) {
        int ans = 0;
        for (int v : nums) {
            ans ^= v;
        }
        return ans;
    }
}
```

Java

```
public class Solution {
    public int singleNumber(int[] nums) {
        return nums.Aggregate(0, (a, b) -> a ^ b);
    }
}
```

Java

```
class Solution {
    public int singleNumber(int[] nums) {
        return Arrays.stream(nums).reduce(0, (a, b) -> a ^ b);
    }
}
```

Java

```
class Solution {
    public int singleNumber(int[] nums) {
        int ans = 0;
        for (int v : nums) {
            ans ^= v;
        }
        return ans;
    }
}
```

Java

```
public class Solution {
    public int singleNumber(int[] nums) {
        return nums.Aggregate(0, (a, b) -> a ^ b);
    }
}
```

JavaScript

```
JavaScript
/*
 * @param {number[]} nums
 * @return {number}
 */
var singleNumber = function(nums) {
    return nums.reduce((a, b) => a ^ b);
};
```

JavaScript

```
/*
 * @param {number[]} nums
 * @return {number}
 */
var singleNumber = function(nums) {
    return nums.reduce((a, b) => a ^ b);
};
```

TypeScript

TypeScript

```
function singleNumber(nums: number[]): number {
    return nums.reduce((r, v) => r ^ v);
}
```

TypeScript

```
function singleNumber(nums: number[]): number {
    return nums.reduce((r, v) => r ^ v);
}
```

Go

Go

```
func singleNumber(nums []int) (ans int) {
    for _, v := range nums {
        ans ^= v
    }
    return
}
```

Go

```
func singleNumber(nums []int) (ans int) {
    for _, v := range nums {
        ans ^= v
    }
    return
}
```

Rust

Rust

```
impl Solution {
    pub fn single_number(nums: Vec<i32>) -> i32 {
        nums.iter().reduce(|r, v| r ^ v).unwrap()
    }
}
```

Rust

```
impl Solution {
    pub fn single_number(nums: Vec<i32>) -> i32 {
        nums.iter().reduce(|r, v| r ^ v).unwrap()
    }
}
```

[*] Bit Manipulation — Pattern Solution (matched from community answers)

C++

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans = 0;

        for (const int num : nums)
            ans ^= num;

        return ans;
    }
};
```

#190

EASY

Reverse Bits

LeetCode · SimpleTest · WalkACE · LeetCode · Editorial

Divide and Conquer · Bit Manipulation

Lists: Grind 169

Problem:

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: n = 43261596

Output: 964176192

Explanation:

Example 2:

Input: n = 2147483644

Output: 1073741822

Explanation:

Constraints:

- 0 <= n <= 2³¹ - 2

- n is even.

Follow up: If this function is called many times, how would you optimize it?

Python

Python


```

//
 * @param {number} n
 * @return {number}
 */
var hammingWeight = function(n) {
    let count = 0;
    while (n) {
        n = n >> 1;
        count++;
    }
    return count;
};

JavaScript
//
 * @param {number} n - a positive integer
 * @return {number}
 */
var hammingWeight = function(n) {
    let count = 0;
    while (n) {
        n &= n - 1;
        ++count;
    }
    return count;
};

TypeScript
function hammingWeight(n: number): number {
    let ans: number = 0;
    while (n !== 0) {
        ans++;
        n &= n - 1;
    }
    return ans;
}

TypeScript
function hammingWeight(n: number): number {
    let count = 0;
    while (n) {
        n = n >> 1;
        count++;
    }
    return count;
}

TypeScript

```

```

function hammingWeight(n: number): number {
    let ans: number = 0;
    while (n !== 0) {
        ans++;
        n &= n - 1;
    }
    return ans;
}

Rust
impl Solution {
    pub fn hammingWeight(n: i32) -> i32 {
        n.count_ones() as i32
    }
}

Rust
impl Solution {
    pub fn hammingWeight(n: i32) -> i32 {
        let mut res = 0;
        while n != 0 {
            while n &= 0 {
                n &= n - 1;
                res += 1;
            }
            n = n >> 1;
        }
        res
    }
}

Rust
impl Solution {
    pub fn hammingWeight(n: i32) -> i32 {
        n.count_ones() as i32
    }
}

Kotlin
class Solution {
    fun hammingWeight(n: Int): Int {
        var count = 0
        var num = n
        while (num != 0) {
            num = num and (num - 1)
            count++
        }
        return count
    }
}

[*] Bit Manipulation — Pattern Solution (matched from community answers)
Java

```

```

class Solution {
    // You can treat n as an assigned value
    public int missingNumber(int a[]) {
        int ans = 0;
        for (int i = 0; i < a.length; ++i)
            if ((i >= 1) & i <= a[i])
                ++ans;
        return ans;
    }
}

#268 EASY
Missing Number
LeetCode · Simplest · WalkCC · LeetCode · Editorial
Array · Hash Table · Math · Binary Search · Bit Manipulation
Likes: 100
Problems:
Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.
Example 1:
Input: nums = [3,0,1]
Output: 2
Explanation:
n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
Example 2:
Input: nums = [0,1]
Output: 2
Explanation:
n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.
Example 3:
Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation:
n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.
Constraints:
n == nums.length

```

```

• 1 <= n <= 104
• 0 <= nums[i] <= n
• All the numbers of nums are unique.
Follow up: Could you implement a solution using only O(1) extra space complexity and O(n) runtime complexity?

>> Brute Force [Bit Manipulation] Interview Prep

Python
# Brute Force Approach
class Solution:
    def missingNumber(self, nums):
        # Brute Force [O(n^2)] - For each 0..n check if it's in nums
        for i in range(len(nums) + 1):
            if i not in nums:
                return i

Python
# Brute Force Approach
def missingNumber(nums):
    # Calculate n, where n is the length of the nums array
    n = len(nums)
    # Compute the expected total sum using the arithmetic sum formula for numbers 0 to n
    expected_sum = n * (n + 1) // 2
    # Compute the actual sum of the numbers in the array
    actual_sum = sum(nums)
    # The missing number is the difference between expected and actual sum
    return expected_sum - actual_sum

# Example Usage
print(missingNumber([0, 1])) # Output: 2

Python
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        ans = len(nums)
        for i, num in enumerate(nums):
            ans ^= i ^ num
        return ans

Python
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))

Python
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        n = len(nums)
        return (1 ^ n) ^ sum(nums)

```

```

Python
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))

C++
class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int ans = nums.size();
        for (int i = 0; i < nums.size(); ++i)
            ans ^= i ^ nums[i];
        return ans;
    }
};

C++
class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int n = nums.size();
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans ^= (i ^ nums[i]);
        }
        return ans;
    }
};

C++
class Solution {
//
 * @param {Integer[]} nums
 * @return {Integer}
 */
function missingNumber(nums) {
    let n = nums.length;
    let sum = ((n + 1) * n) / 2;
    for (let i = 0; i < n; ++i) {
        sum -= nums[i];
    }
    return sum;
}
}

C++

```

```

class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int n = nums.size();
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans ^= (i ^ nums[i]);
        }
        return ans;
    }
};

C++
class Solution {
//
 * @param {Integer[]} nums
 * @return {Integer}
 */
function missingNumber(nums) {
    let n = nums.length;
    let sum = ((n + 1) * n) / 2;
    for (let i = 0; i < n; ++i) {
        sum -= nums[i];
    }
    return sum;
}
}

Java
class Solution {
    public int missingNumber(int[] nums) {
        int ans = nums.length;
        for (int i = 0; i < nums.length; ++i)
            ans ^= i ^ nums[i];
        return ans;
    }
}

Java
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans ^= (i ^ nums[i]);
        }
        return ans;
    }
}

Java

```



```

class Solution {
public: int missingNumber(int[] nums) {
    int n = nums.length;
    int ans = 0;
    for (int i = 0; i < n; ++i) {
        ans ^= i ^ nums[i];
    }
    return ans;
}
}

```

JavaScript

```

JavaScript
/**
 * @param {number[]} nums
 * @return {number}
 */
var missingNumber = function (nums) {
    const n = nums.length;
    let ans = 0;
    for (let i = 0; i < n; ++i) {
        ans ^= i ^ nums[i];
    }
    return ans;
};

```

JavaScript

```

/**
 * @param {number[]} nums
 * @return {number}
 */
var missingNumber = function (nums) {
    const n = nums.length;
    let ans = 0;
    for (let i = 0; i < n; ++i) {
        ans ^= i ^ nums[i];
    }
    return ans;
};

```

TypeScript

```

TypeScript
function missingNumber(nums: number[]): number {
    const n = nums.length;
    let ans = 0;
    for (let i = 0; i < n; ++i) {
        ans ^= i ^ nums[i];
    }
    return ans;
}

```

TypeScript

```

function missingNumber(nums: number[]): number {
    const n = nums.length;
    let ans = 0;
    for (let i = 0; i < n; ++i) {
        ans ^= i ^ nums[i];
    }
    return ans;
}

```

Go

Go

```

func missingNumber(nums []int) (ans int) {
    n := len(nums)
    ans = 0
    for i, v := range nums {
        ans ^= (i ^ v)
    }
    return
}

```

Go

```

func missingNumber(nums []int) (ans int) {
    n := len(nums)
    ans = 0
    for i, v := range nums {
        ans ^= (i ^ v)
    }
    return
}

```

Rust

Rust

```

impl Solution {
    pub fn missing_number(nums: Vec<i32>) -> i32 {
        let n = nums.len() as i32;
        let mut ans = 0;
        for (i, v) in nums.iter().enumerate() {
            ans ^= (i as i32 ^ v);
        }
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn missing_number(nums: Vec<i32>) -> i32 {
        let n = nums.len() as i32;
        let mut ans = 0;
        for (i, v) in nums.iter().enumerate() {
            ans ^= (i as i32 ^ v);
        }
        ans
    }
}

```

[*] Bit Manipulation — Pattern Solution (matched from community answers)

C++

```

class Solution {
public:
    int missingNumber(vector<int>& nums) {
        int ans = nums.size();
        for (int i = 0; i < nums.size(); ++i)
            ans ^= i ^ nums[i];
        return ans;
    }
};

```

#338

EASY

Counting Bits

LeetCode · LeetCode · WUCC · LeetCode · Editorial

Dynamic Programming · Bit Manipulation

List: Grid 169

Problem:

Given an integer n , return an array ans of length $n + 1$ such that for each i ($0 \leq i \leq n$), $ans[i]$ is the number of 1 's in the binary representation of i .

Example 1:

```

Input: n = 2
Output: [0,1,1]
Explanation:
0 -> 0
1 -> 1
2 -> 10

```

Example 2:

```

Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 -> 0
1 -> 1
2 -> 10
3 -> 11
4 -> 100
5 -> 101

```

Constraints:

$0 \leq n \leq 10^5$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

>> Brute Force [Bit Manipulation] Interview Prep

Python

```

# Brute Force Approach
def countBits(self, n):
    # Brute Force O(n * 32) = count 3-bits for each number 0..n individually
    def count_ones(x):
        c = 0
        while x:
            c += x & 1
            x >>= 1
        return c
    return [count_ones(i) for i in range(n + 1)]

```

Python

Python

```

def countBits(n):
    # Create a list to store the number of 1's for each number from 0 to n
    dp = [0] * (n + 1)
    # base case: number starting from 1, since dp[0] is already 0
    for i in range(1, n + 1):
        # dp[i] is number of the count for i//2 plus 1 if i is odd (i & 1)
        dp[i] = dp[i // 2] + (i & 1)
    return dp
# Example usage:
# print(countBits(5))  # Output: [0, 1, 1, 2, 1, 2]

```

Python

```

class Solution:
    def countBits(self, n: int) -> List[int]:
        # f(i) = 1's number of i in binary
        # f(i) = f(i // 2) + i % 2
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i // 2] + (i & 1)
        return ans

```

Python

```

class Solution:
    def countBits(self, n: int) -> List[int]:
        return [i.bit_count() for i in range(n + 1)]

```

Python

```

class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i // 2] + (i & 1)
        return ans

```

Python

```

class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i // 2] + (i & 1)
        return ans

```

C++

C++

```

class Solution {
public:
    vector<int> countBits(int n) {
        // f(i) = 1's number of i in binary
        // f(i) = f(i // 2) + i % 2
        vector<int> ans(n + 1);
        for (int i = 1; i <= n; ++i)
            ans[i] = ans[i // 2] + (i & 1);
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> ans(n + 1);
        for (int i = 0; i <= n; ++i) {
            ans[i] = __builtin_popcount(i);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<int> countBits(int n) {
        vector<int> ans(n + 1);
        for (int i = 0; i <= n; ++i)
            ans[i] = __builtin_popcount(i);
        return ans;
    }
};

```

Java

```

class Solution {
    public List<int> countBits(int n) {
        // f(i) = 1's number of i in binary
        // f(i) = f(i // 2) + i % 2
        List<int> ans = new ArrayList();
        for (int i = 1; i <= n; ++i)
            ans[i] = ans[i // 2] + (i & 1);
        return ans;
    }
}

```

Java

```

class Solution {
    public List<int> countBits(int n) {
        List<int> ans = new ArrayList();
        for (int i = 0; i <= n; ++i) {
            ans[i] = Integer.bitCount(i);
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public List<int> countBits(int n) {
        List<int> ans = new ArrayList();
        for (int i = 0; i <= n; ++i)
            ans[i] = ans[i // 2] + (i & 1);
        return ans;
    }
}

```

TypeScript

TypeScript

```

function countBits(n: number): number[] {
    const ans: number[] = Array(n + 1).fill(0);
    for (let i = 0; i <= n; ++i) {
        ans[i] = bitCount(i);
    }
    return ans;
}

```

```

function bitCount(n: number): number {
    let count = 0;
    while (n) {
        n &= n - 1;
        ++count;
    }
    return count;
}

```

TypeScript

```

function countBits(n: number): number[] {
    const ans: number[] = Array(1).fill(0);
    for (let i = 1; i <= n; i++) {
        ans[i] = ans[i > 1] + 1;
    }
    return ans;
}

Go
Go

func countBits(n int) []int {
    ans := make([]int, n+1)
    for i := 1; i <= n; i++ {
        ans[i] = bits.OneCount(uint(i))
    }
    return ans
}

Go

func countBits(n int) []int {
    ans := make([]int, n+1)
    for i := 1; i <= n; i++ {
        ans[i] = ans[i/2] + 1
    }
    return ans
}

Go

func countBits(n int) []int {
    ans := make([]int, n+1)
    for i := 1; i <= n; i++ {
        ans[i] = ans[i/2] + 1
    }
    return ans
}

```

[*] Bit Manipulation — Pattern Solution (matched from community answers)

```

Python
# Create a list to store the number of 1's for each number from 0 to n
dp = [0] * (n + 1)
# Loop over the number starting from 1, since dp[0] is already 0
for i in range(1, n + 1):
    # dp[i] is computed as the count for i/2 plus 1 if i is odd (i & 1)
    dp[i] = dp[i // 2] + 1 if i & 1 else dp[i // 2]
    return dp

# Example usage:
# print(countBits(5))  # Output: [0, 1, 1, 2, 1, 2]

```

#78 MEDIUM

Subsets

Bit Manipulation · LeetCode — By Pattern — 415 — LeetCode

LeetCode · SimplyList · WalkCC · LeetCode · Editorial
Array · Backtracking · Bit Manipulation

Lines: 6/100

Problem:

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,3]`
Output: `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

Example 2:

Input: `nums = [0]`
Output: `[[], [0]]`

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- All the numbers of `nums` are unique.

>> Brute Force [Bit Manipulation] Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        # Brute Force (2^n - 1) - bitmask over all 2^n subsets
        result = []
        for mask in range(1 <= len(nums)):
            sub = [nums[i] for i in range(len(nums)) if mask & (1 <= i)]
            result.append(sub)
        return result

```

Python

Python

Bit Manipulation · LeetCode — By Pattern — 416 — LeetCode

```

# Define the function to generate all subsets using backtracking.
def subsets(nums):
    # This list will store all the subsets
    result = []

    # Recursive helper function to generate subsets.
    def backtrack(start, current):
        # Append a copy of the current subset to the final result.
        result.append(current.copy())
        # Explore every possible next element starting from index 'start'
        for i in range(start, len(nums)):
            # Include nums[i] in the current subset.
            current.append(nums[i])
            # Recurse with the next starting index.
            backtrack(i + 1, current)
            # Backtrack: remove the last element to explore other possibilities.
            current.pop()

    # Start the recursion with an empty path.
    backtrack(0, [])
    return result

# Example usage:
# print(subsets([1, 2, 3]))

```

```

Python
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        ans = []

        def dfs(i: int, path: List[int]) -> None:
            ans.append(path)

            for i in range(i, len(nums)):
                dfs(i + 1, path + [nums[i]])

            dfs(i + 1, path)

        dfs(0, [])
        return ans

```

```

Python
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(i)
                return
            dfs(i + 1)
            t.append(nums[i])
            dfs(i + 1)
            t.pop()

        ans = []
        t = []
        dfs(0)
        return ans

```

Python

Bit Manipulation · LeetCode — By Pattern — 417 — LeetCode

```

from typing import List

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort() # sort() not necessary if no duplicates
        result = [[]]

        for num in nums:
            result += [subset + [num] for subset in result]

        return result

```

```

class Solution:
    def dfs(self, nums: List[int]) -> List[List[int]]:
        def dfs(i, t):
            ans.append(t)
            for i in range(i, len(nums)):
                t.append(nums[i])
                dfs(i + 1, t)
                t.pop()

        ans = []
        dfs(0, [])
        return ans

```

```

class Solution:
    def dfs(self, nums: List[int]) -> List[List[int]]:
        def dfs(index, path, ans):
            ans.append(path)
            for i in range(index, len(nums)):
                dfs(i + 1, path + [nums[i]], ans)

        ans = []
        dfs(0, [], ans)
        return ans

```

```

class Solution:
    def dfs(self, nums: List[int]) -> List[List[int]]:
        def dfs(index, path):
            ans.append(path)
            for i in range(index, len(nums)):
                dfs(i + 1, path + [nums[i]])

        ans = []
        dfs(0, [])
        return ans

```

C++

C++

Bit Manipulation · LeetCode — By Pattern — 418 — LeetCode

```

class Solution {
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        vector<vector<int>> ans;
        dfs(nums, 0, {}, ans);
        return ans;
    }

private:
    void dfs(const vector<int>& nums, int s, vector<int>&& path, vector<vector<int>>& ans) {
        ans.push_back(path);

        for (int i = s; i < nums.size(); ++i) {
            path.push_back(nums[i]);
            dfs(nums, i + 1, path, ans);
            path.pop_back();
        }
    }
};

```

```

C++
class Solution {
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        vector<vector<int>> ans;
        vector<int> t;
        function<void(int)> dfs = [&](int i) -> void {
            if (i == nums.size()) {
                ans.push_back(t);
                return;
            }
            dfs(i + 1);
            t.push_back(nums[i]);
            dfs(i + 1);
            t.pop_back();
        };
        dfs(0);
        return ans;
    }
};

```

C++

Bit Manipulation · LeetCode — By Pattern — 419 — LeetCode

```

class Solution {
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        vector<vector<int>> ans;
        vector<int> t;
        function<void(int)> dfs = [&](int i) -> void {
            if (i == nums.size()) {
                ans.push_back(t);
                return;
            }
            dfs(i + 1);
            t.push_back(nums[i]);
            dfs(i + 1);
            t.pop_back();
        };
        dfs(0);
        return ans;
    }
};

```

Java

Java

```

class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> ans = new ArrayList<>();
        dfs(nums, 0, new ArrayList<>(), ans);
        return ans;
    }

    private void dfs(int[] nums, int s, List<Integer> path, List<List<Integer>> ans) {
        ans.add(new ArrayList<>(path));

        for (int i = s; i < nums.length; ++i) {
            path.add(nums[i]);
            dfs(nums, i + 1, path, ans);
            path.remove(path.size() - 1);
        }
    }
}

```

Java

Bit Manipulation · LeetCode — By Pattern — 420 — LeetCode

```

class Solution {
    private List<List<Integer>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();
    private int[] nums;

    public List<List<Integer>> subsets(int[] nums) {
        this.nums = nums;
        dfs(0);
        return ans;
    }

    private void dfs(int i) {
        if (i == nums.length) {
            ans.add(new ArrayList<>(t));
            return;
        }
        dfs(i + 1);
        t.add(nums[i]);
        dfs(i + 1);
        t.remove(t.size() - 1);
    }
}

```

Java

```

class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        int n = nums.length;
        List<List<Integer>> ans = new ArrayList<>();
        for (int mask = 0; mask < 1 << n; ++mask) {
            List<Integer> t = new ArrayList<>();
            for (int i = 0; i < n; ++i) {
                if (((mask >> i) & 1) == 1) {
                    t.add(nums[i]);
                }
            }
            ans.add(t);
        }
        return ans;
    }
}

```

Java

Bit Manipulation · LeetCode · By Pattern

— 421 —

LeetCode

```

class Solution {
    private List<List<Integer>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();
    private int[] nums;

    public List<List<Integer>> subsets(int[] nums) {
        this.nums = nums;
        dfs(0);
        return ans;
    }

    private void dfs(int i) {
        if (i == nums.length) {
            ans.add(new ArrayList<>(t));
            return;
        }
        dfs(i + 1);
        t.add(nums[i]);
        dfs(i + 1);
        t.remove(t.size() - 1);
    }
}

```

TypeScript

```

function subsets(nums: number[]): number[][] {
    const ans: number[][] = [];
    const t: number[] = [];
    const dfs = (i: number) => {
        if (i == nums.length) {
            ans.push(t.slice());
            return;
        }
        dfs(i + 1);
        t.push(nums[i]);
        dfs(i + 1);
        t.pop();
    };
    dfs(0);
    return ans;
}

```

TypeScript

```

function subsets(nums: number[]): number[][] {
    const n = nums.length;
    const ans: number[][] = [];
    for (let mask = 0; mask < 1 << n; ++mask) {
        const t: number[] = [];
        for (let i = 0; i < n; ++i) {
            if (((mask >> i) & 1) == 1) {
                t.push(nums[i]);
            }
        }
        ans.push(t);
    }
    return ans;
}

```

Bit Manipulation · LeetCode · By Pattern

— 422 —

LeetCode

TypeScript

```

function subsets(nums: number[]): number[][] {
    const ans: number[][] = [];
    const t: number[] = [];
    const dfs = (i: number) => {
        if (i == nums.length) {
            ans.push(t.slice());
            return;
        }
        dfs(i + 1);
        t.push(nums[i]);
        dfs(i + 1);
        t.pop();
    };
    dfs(0);
    return ans;
}

```

Go

```

func subsets(nums []int) (ans [][]int) {
    t := []int{}
    var dfs func(int)
    dfs = func(i int) {
        if i == len(nums) {
            ans = append(ans, append([]int(nil), t...))
            return
        }
        dfs(i + 1)
        t = append(t, nums[i])
        dfs(i + 1)
        t = t[:len(t)-1]
    }
    dfs(0)
    return
}

```

Go

```

func subsets(nums []int) (ans [][]int) {
    n := len(nums)
    for mask := 0; mask < 2<<n; mask++ {
        t := []int{}
        for i, x := range nums {
            if mask>>mask & 1 <= i {
                t = append(t, x)
            }
        }
        ans = append(ans, t)
    }
    return
}

```

Go

Bit Manipulation · LeetCode · By Pattern

— 423 —

LeetCode

```

func subsets(nums []int) (ans [][]int) {
    t := []int{}
    var dfs func(int)
    dfs = func(i int) {
        if i == len(nums) {
            ans = append(ans, append([]int(nil), t...))
            return
        }
        dfs(i + 1)
        t = append(t, nums[i])
        dfs(i + 1)
        t = t[:len(t)-1]
    }
    dfs(0)
    return
}

```

Rust

```

impl Solution {
    fn dfs(&self, i: usize, t: &mut Vec<i32>, ans: &mut Vec<Vec<i32>>, nums: &Vec<i32>) {
        if i == nums.len() {
            ans.push(t.clone());
            return;
        }
        Self.dfs(i + 1, t, ans, nums);
        t.push(nums[i]);
        Self.dfs(i + 1, t, ans, nums);
        t.pop();
    }

    pub fn subsets(&self, nums: &Vec<i32>) -> Vec<Vec<i32>> {
        let mut ans = Vec::new();
        Self.dfs(0, &mut Vec::new(), &mut ans, &nums);
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn subsets(&self, nums: &Vec<i32>) -> Vec<Vec<i32>> {
        let n = nums.len();
        let mut ans = Vec::new();
        for mask in 0..(1 << n) {
            let mut t = Vec::new();
            for i in 0..n {
                if (mask >> i) & 1 == 1 {
                    t.push(nums[i]);
                }
            }
            ans.push(t);
        }
        ans
    }
}

```

Rust

Bit Manipulation · LeetCode · By Pattern

— 424 —

LeetCode

```

impl Solution {
    fn dfs(&self, i: usize, t: &mut Vec<i32>, res: &mut Vec<Vec<i32>>, nums: &Vec<i32>) {
        if i == nums.len() {
            res.push(t.clone());
            return;
        }
        Self.dfs(i + 1, t, res, nums);
        t.push(nums[i]);
        Self.dfs(i + 1, t, res, nums);
        t.pop();
    }

    pub fn subsets(&self, nums: &Vec<i32>) -> Vec<Vec<i32>> {
        let mut res = Vec::new();
        Self.dfs(0, &mut Vec::new(), &mut res, &nums);
        res
    }
}

```

[*] Bit Manipulation — Pattern Solution (matched from community answers)

Java

```

class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        int n = nums.length;
        List<List<Integer>> ans = new ArrayList<>();
        for (int mask = 0; mask < 1 << n; ++mask) {
            List<Integer> t = new ArrayList<>();
            for (int i = 0; i < n; ++i) {
                if (((mask >> i) & 1) == 1) {
                    t.add(nums[i]);
                }
            }
            ans.add(t);
        }
        return ans;
    }
}

```

#90

MEDIUM

Subsets II

LeetCode · SimplyList · WalkCC · LeetCode · Editorial

Array · Backtracking · Bit Manipulation

Like · 2099 · Dislike · 299

Problem:

Given an integer array `nums` that may contain duplicates, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,2]`

Output: `[[1],[1,2],[1,2,2],[2],[2,2]]`

Bit Manipulation · LeetCode · By Pattern

— 425 —

LeetCode

Example 2:

Input: `nums = [0]`

Output: `[[],[0]]`

Constraints:

`1 <= nums.length <= 10`

`-10 <= nums[i] <= 10`

>> Brute Force [Bit Manipulation] Interview Prep

Python

Brute Force Approach

class Solution:

def subsetsWithDup(self, nums):

Brute Force O(2^n * n) - bitmask with deduplication via set

result = set()

nums.sort()

for mask in range(1 << len(nums)):

sub = tuple(nums[i] for i in range(len(nums)) if mask & (1 << i))

result.add(sub)

return [list(s) for s in result]

Python

Python

def subsetsWithDup(nums):

Sort the numbers to handle duplicates

nums.sort()

result = [] # To store all subsets

subset = [] # Current subset being built

def backtrack(start):

Append a clone of the current subset to the result

result.append(list(subset))

Iterate over possible elements to include

for i in range(start, len(nums)):

If the current element is a duplicate and not at the start of this loop, skip it

if i > start and nums[i] == nums[i - 1]:

continue

Include current element into the subset

subset.append(nums[i])

Recurse with next start index

backtrack(i + 1)

Backtrack and remove the last element added

subset.pop()

backtrack()

return result

Example usage:

print(subsetsWithDup([1, 2, 2]))

Python

Bit Manipulation · LeetCode · By Pattern

— 426 —

LeetCode

```

class Solution:
    def subsetWithDup(self, nums: List[int]) -> List[List[int]]:
        ans = []

        def dfs(i: int, path: List[int]) -> None:
            ans.append(path)
            if i == len(nums):
                return

            for i in range(i, len(nums)):
                if i > i and nums[i] == nums[i - 1]:
                    continue
                dfs(i + 1, path + [nums[i]])

        nums.sort()
        dfs(0, [])
        return ans

```

Python

```

class Solution:
    def subsetWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            t.append(nums[i])
            dfs(i + 1)
            while i + 1 < len(nums) and nums[i + 1] == x:
                i += 1
            t.pop()

        nums.sort()
        ans = []
        x = 1
        dfs(0)
        return ans

```

Python

```

class Solution:
    def subsetWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            ans.append(t[:])
            for i in range(i, len(nums)):
                # skip same for [1,1,1], second '1' will be skipped here if
                # not covered by 1st skip, because i==0 when ans=[1,1] and i=1
                if i > i and nums[i] == nums[i - 1]:
                    continue
                t.append(nums[i])
                dfs(i + 1, t)
                t.pop()

            ans = []
            nums.sort()
            dfs(0, [])
            return ans

```

Python

```

# Iteration
class Solution:
    def subsetWithDup(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort()
        # Sorting is necessary to handle duplicates.
        result = [[]]
        start_idx = 0 # Start index for the new subsets
        new_subset_size = 0

        for i in range(1, len(nums)):
            size = len(result)
            # If the current number is the same as the previous one,
            # we only need to extend the subsets added in the previous iteration.
            if i > 0 and nums[i] == nums[i - 1]:
                start_idx = size - new_subset_size
            else:
                start_idx = 0

            new_subset_size = 0 # Reset the size of newly created subsets
            for j in range(start_idx, size):
                current_subset = result[j] + [nums[i]]
                result.append(current_subset)
                new_subset_size += 1

        return result

```

Python

```

class Solution:
    def subsetWithDup(self, nums: List[int]) -> List[List[int]]:
        n = len(nums)
        ans = []
        for mask in range(1 <= n):
            ok = True
            t = []
            for i in range(n):
                if mask >= (1 <= i):
                    if i and (mask >= (1 <= i) & 1) == 0 and nums[i] == nums[i - 1]:
                        ok = False
                        break
                    t.append(nums[i])
            if ok:
                ans.append(t)
        return ans

```

C++

```

class Solution {
public:
    vector<vector<int>> subsetWithDup(vector<int>& nums) {
        vector<vector<int>> ans;
        ranges::sort(nums);
        dfs(nums, 0, {}, ans);
        return ans;
    }

private:
    void dfs(const vector<int>& nums, int s, vector<int>& path,
            vector<vector<int>>& ans) {
        ans.push_back(path);
        for (int i = s; i < nums.size(); ++i) {
            if (i > s & nums[i] == nums[i - 1])
                continue;
            path.push_back(nums[i]);
            dfs(nums, i + 1, std::move(path), ans);
            path.pop_back();
        }
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> subsetWithDup(vector<int>& nums) {
        ranges::sort(nums);
        vector<vector<int>> ans;
        vector<int> t;
        int n = nums.size();
        auto dfs = [&](this auto& dfs, int i) {
            if (i == n) {
                ans.push_back(t);
                return;
            }
            t.push_back(nums[i]);
            dfs(i + 1);
            t.pop_back();
            while (i + 1 < n & nums[i + 1] == nums[i]) {
                ++i;
            }
            dfs(i);
            return ans;
        };
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> subsetWithDup(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        int n = nums.size();
        vector<vector<int>> ans;
        for (int mask = 0; mask < 1 <= n; ++mask) {
            vector<int> t;
            bool ok = true;
            for (int i = 0; i < n; ++i) {
                if ((mask >= 1 <= i) & 1) {
                    if (i > 0 & nums[i] == nums[i - 1]) {
                        ok = false;
                        break;
                    }
                    t.push_back(nums[i]);
                }
            }
            if (ok) {
                ans.push_back(t);
            }
        }
        return ans;
    }
};

```

Java

Java

```

class Solution {
    public List<List<Integer>> subsetWithDup(int[] nums) {
        List<List<Integer>> ans = new ArrayList<>();
        Arrays.sort(nums);
        dfs(nums, 0, new ArrayList<>(), ans);
        return ans;
    }

    private void dfs(int[] nums, int s, List<Integer> path, List<List<Integer>> ans) {
        ans.add(new ArrayList<>(path));
        for (int i = s; i < nums.length; ++i) {
            if (i > s & nums[i] == nums[i - 1])
                continue;
            path.add(nums[i]);
            dfs(nums, i + 1, path, ans);
            path.remove(path.size() - 1);
        }
    }
}

```

Java

```

class Solution {
    private List<List<Integer>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();
    private int[] nums;

    public List<List<Integer>> subsetWithDup(int[] nums) {
        Arrays.sort(nums);
        this.nums = nums;
        dfs(0);
        return ans;
    }

    private void dfs(int i) {
        if (i == nums.length) {
            ans.add(new ArrayList<>(t));
            return;
        }
        t.add(nums[i]);
        dfs(i + 1);
        int x = t.remove(t.size() - 1);
        while (i + 1 < nums.length & nums[i + 1] == x) {
            ++i;
        }
        dfs(i + 1);
    }
}

```

Java

```

public class Solution {
    public List<List<Integer>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();
    private int[] nums;

    public List<List<Integer>> subsetWithDup(int[] nums) {
        Arrays.sort(nums);
        this.nums = nums;
        dfs(0);
        return ans;
    }

    private void dfs(int i) {
        if (i == nums.length) {
            ans.add(new ArrayList<>(t));
            return;
        }
        t.add(nums[i]);
        dfs(i + 1);
        t.remove(t.size() - 1);
        while (i + 1 < nums.length & nums[i + 1] == nums[i]) {
            ++i;
        }
        dfs(i + 1);
    }
}

```

Java

```

class Solution {
    public List<List<Integer>> subsetWithDup(int[] nums) {
        Arrays.sort(nums);
        int n = nums.length;
        List<List<Integer>> ans = new ArrayList<>();
        for (int mask = 0; mask < 1 <= n; ++mask) {
            List<Integer> t = new ArrayList<>();
            bool ok = true;
            for (int i = 0; i < n; ++i) {
                if ((mask >= 1 <= i) & 1) {
                    if (i > 0 & nums[i] == nums[i - 1]) {
                        ok = false;
                        break;
                    }
                    t.add(nums[i]);
                }
            }
            if (ok) {
                ans.add(t);
            }
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    public IList<IList<int>> SubsetsWithDup(int[] nums) {
        Array.Sort(nums);
        int n = nums.Length;
        IList<IList<int>> ans = new List<IList<int>>();
        for (int mask = 0; mask < 1 << n; mask++) {
            IList<int> t = new List<int>();
            bool ok = true;
            for (int i = 0; i < n; ++i) {
                if ((mask >> i & 1) == 1) {
                    if (i > 0 && (mask >> (i - 1) & 1) == 0 && nums[i] == nums[i - 1]) {
                        ok = false;
                        break;
                    }
                    t.Add(nums[i]);
                }
            }
            if (ok) {
                ans.Add(t);
            }
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);
        int n = nums.Length;
        List<List<Integer>> ans = new ArrayList<>();
        for (int mask = 0; mask < 1 << n; ++mask) {
            List<Integer> t = new ArrayList<>();
            boolean ok = true;
            for (int i = 0; i < n; ++i) {
                if ((mask >> i & 1) == 1) {
                    if (i > 0 && (mask >> (i - 1) & 1) == 0 && nums[i] == nums[i - 1]) {
                        ok = false;
                        break;
                    }
                    t.add(nums[i]);
                }
            }
            if (ok) {
                ans.add(t);
            }
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    private IList<IList<int>> ans = new List<IList<int>>();
    private IList<int> t = new List<int>();
    private int[] nums;

    public IList<IList<int>> SubsetsWithDup(int[] nums) {
        Array.Sort(nums);
        this.nums = nums;
        Dfs(0);
        return ans;
    }

    private void Dfs(int i) {
        if (i == nums.Length) {
            ans.Add(new List<int>(t));
            return;
        }
        t.Add(nums[i]);
        Dfs(i + 1);
        t.Remove(t.Count - 1);
        while (i + 1 < nums.Length && nums[i + 1] == nums[i]) {
            ++i;
        }
        Dfs(i + 1);
    }
}

```

JavaScript

```

var subsetsWithDup = function(nums) {
    nums.sort((a, b) => a - b);
    const n = nums.length;
    const t = [];
    const ans = [];
    const dfs = i => {
        if (i == n) {
            ans.push([...t]);
            return;
        }
        t.push(nums[i]);
        dfs(i + 1);
        while (i + 1 < n && nums[i] === nums[i + 1]) {
            ++i;
        }
        t.pop();
        dfs(i + 1);
    };
    dfs(0);
    return ans;
};

```

JavaScript

```

/*
 * @param {number[]} nums
 * @return {number[][]}
 */
var subsetsWithDup = function(nums) {
    nums.sort((a, b) => a - b);
    const n = nums.length;
    const t = [];
    const ans = [];
    const dfs = i => {
        if (i == n) {
            ans.push([...t]);
            return;
        }
        t.push(nums[i]);
        dfs(i + 1);
        t.pop();
        while (i + 1 < n && nums[i] === nums[i + 1]) {
            ++i;
        }
        dfs(i + 1);
    };
    dfs(0);
    return ans;
};

```

TypeScript

```

function subsetsWithDup(nums: number[]): number[][] {
    nums.sort((a, b) => a - b);
    const n = nums.length;
    const t: number[] = [];
    const ans: number[][] = [];
    const dfs = (i: number): void => {
        if (i == n) {
            ans.push([...t]);
            return;
        }
        t.push(nums[i]);
        dfs(i + 1);
        t.pop();
        while (i + 1 < n && nums[i] === nums[i + 1]) {
            ++i;
        }
        dfs(i + 1);
    };
    dfs(0);
    return ans;
}

```

TypeScript

```

function subsetsWithDup(nums: number[]): number[][] {
    nums.sort((a, b) => a - b);
    const n = nums.length;
    const ans: number[][] = [];
    for (let mask = 0; mask < 2 << n; ++mask) {
        const t: number[] = [];
        let ok: boolean = true;
        for (let i = 0; i < n; ++i) {
            if ((mask >> i & 1) == 1) {
                if (i > 0 && (mask >> (i - 1) & 1) == 0 && nums[i] == nums[i - 1]) {
                    ok = false;
                    break;
                }
                t.push(nums[i]);
            }
        }
        if (ok) {
            ans.push(t);
        }
    }
    return ans;
}

```

Go

```

func subsetsWithDup(nums []int) ans [][]int {
    slices.Sort(nums)
    n := len(nums)
    t := []int{}
    var dfs func(int)
    dfs = func(i int) {
        if i == n {
            ans = append(ans, slices.Clone(t))
            return
        }
        t = append(t, nums[i])
        dfs(i + 1)
        t = t[len(t)-1]
        for i+1 < n && nums[i] == nums[i+1] {
            ++i
        }
        dfs(i + 1)
    }
    dfs(0)
    return ans
}

```

Go

```

func subsetsWithDup(nums []int) ans [][]int {
    sort.Ints(nums)
    n := len(nums)
    for mask := 0; mask < 1<<n; mask++ {
        t := []int{}
        ok := true
        for i := 0; i < n; i++ {
            if mask>>i & 1 == 1 {
                if i > 0 && mask>>(i-1) & 1 == 0 && nums[i] == nums[i-1] {
                    ok = false
                    break
                }
                t = append(t, nums[i])
            }
        }
        if ok {
            ans = append(ans, t)
        }
    }
    return ans
}

```

Go

```

func subsetsWithDup(nums []int) ans [][]int {
    sort.Ints(nums)
    n := len(nums)
    for mask := 0; mask < 1<<n; mask++ {
        t := []int{}
        ok := true
        for i := 0; i < n; i++ {
            if mask>>i & 1 == 1 {
                if i > 0 && mask>>(i-1) & 1 == 0 && nums[i] == nums[i-1] {
                    ok = false
                    break
                }
                t = append(t, nums[i])
            }
        }
        if ok {
            ans = append(ans, t)
        }
    }
    return ans
}

```

Rust

```

fn subsetsWithDup(nums: Vec<i32>) -> Vec<Vec<i32>> {
    let mut ans = Vec::new();
    let a = nums.len();
    let mut ans = Vec::new();
    for mask in 0..a {
        let mut t = Vec::new();
        for i in 0..a {
            if (mask >> i & 1) == 1 {
                if i > 0 && (mask >> (i - 1) & 1) == 0 && nums[i] == nums[i - 1] {
                    break;
                }
                t.push(nums[i]);
            }
        }
        if ok {
            ans.push(t);
        }
    }
    ans
}

```

Rust

```

impl Solution {
    pub fn subsets_with_dup(nums: Vec<i32>) -> Vec<Vec<i32>> {
        let mut ans = Vec::new();
        let a = nums.len();
        let mut ans = Vec::new();
        for mask in 0..a {
            let mut t = Vec::new();
            for i in 0..a {
                if (mask >> i & 1) == 1 {
                    if i > 0 && (mask >> (i - 1) & 1) == 0 && nums[i] == nums[i - 1] {
                        break;
                    }
                    t.push(nums[i]);
                }
            }
            if ok {
                ans.push(t);
            }
        }
        ans
    }
}

```

Java

```

class Solution {
public: List<ListNode*> subsetWithDup(int[] nums) {
    Array<int>(nums);
    int n = nums.length;
    List<List<ListNode*>> ans = new ArrayList<>();
    for (int mask = 0; mask < 1 << n; mask++) {
        List<ListNode*> t = new ArrayList<>();
        boolean ok = true;
        for (int i = 0; i < n; ++i) {
            if ((mask >> i & 1) == 1) {
                if (i % 8 && (mask >> (i - 1) & 1) == 0 && nums[i] == nums[i - 1]) {
                    ok = false;
                    break;
                }
                t.add(nums[i]);
            }
        }
        if (ok) {
            ans.add(t);
        }
    }
    return ans;
}
}

```

#287 MEDIUM
Find the Duplicate Number
LeetCode · SimplyList · WalkCC · LeetCode · Editorial
Array · Two Pointers · Binary Search · Bit Manipulation
Like · Grind 169

Problem:
Given an array of integers nums containing n + 1 integers where each integer is in the range [1, n] inclusive.

There is only one repeated number in nums, return this repeated number.

You must solve the problem without modifying the array nums and using only constant extra space.

Example 1:

Input: nums = [1,3,4,2,2]
Output: 2

Example 2:

Input: nums = [3,1,3,4,2]
Output: 3

Example 3:

Input: nums = [3,3,3,3,3]
Output: 3

Constraints:

- 1 <= n <= 10⁵
- nums.length == n + 1

Bit Manipulation · LeetCode · By Pattern

— 439 —

LeetCode

- 1 <= nums[i] <= n
- All the integers in nums appear only once except for precisely one integer which appears two or more times.

Follow up:

- How can we prove that at least one duplicate number must exist in nums?
- Can you solve the problem in linear runtime complexity?

>> Brute Force [Bit Manipulation] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        # Brute Force O(n^2) -> For each number count its occurrences
        for i in range(len(nums)):
            count = 0
            for j in range(len(nums)):
                if nums[j] == nums[i]:
                    count += 1
            if count > 1:
                return nums[i]

```

Python

```

# Tortoise to find the duplicate number using Floyd's Tortoise and Hare algorithm.
def findDuplicate(nums: List[int]) -> int:

```

```

# Phase 1: Find the intersection point inside the cycle.
tortoise = nums[0] # Initialize tortoise pointer.
hare = nums[0] # Initialize hare pointer.
while True:
    tortoise = nums[tortoise] # Move tortoise one step.
    hare = nums[nums[hare]] # Move hare two steps.
    if tortoise == hare: # If pointers meet, cycle is detected.
        break

```

```

# Phase 2: Find the entrance to the cycle.
pointer = nums[0] # Initialize a new pointer from the start.
while pointer != tortoise:
    pointer = nums[pointer] # Move pointer one step.
    tortoise = nums[tortoise] # Move tortoise one step.
return pointer # The duplicate number.

```

```

# Example usage:
if __name__ == "__main__":
    sample = [1, 3, 4, 2, 2]
    print(findDuplicate(sample)) # Expected output: 2

```

Python

Bit Manipulation · LeetCode · By Pattern

— 440 —

LeetCode

```

class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        slow = nums[nums[0]]
        fast = nums[nums[nums[0]]]

        while slow != fast:
            slow = nums[slow]
            fast = nums[nums[fast]]

        slow = nums[0]

        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]

        return slow

```

Python

```

class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v == x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)

```

Python

```

class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v == x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)

```

C++

```

class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int slow = nums[nums[0]];
        int fast = nums[nums[nums[0]]];

        while (slow != fast) {
            slow = nums[slow];
            fast = nums[nums[fast]];
        }

        slow = nums[0];

        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }

        return slow;
    }
};

```

Bit Manipulation · LeetCode · By Pattern

— 441 —

LeetCode

```

C++
class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int l = 0, r = nums.size() - 1;
        while (l < r) {
            int mid = (l + r) >> 1;
            int cnt = 0;
            for (int v : nums) {
                cnt += v == mid;
            }
            if (cnt > mid) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    }
};

```

C++

```

class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int l = 0, r = nums.size() - 1;
        while (l < r) {
            int mid = (l + r) >> 1;
            int cnt = 0;
            for (int v : nums) {
                cnt += v == mid;
            }
            if (cnt > mid) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    }
};

```

Java

Java

Bit Manipulation · LeetCode · By Pattern

— 442 —

LeetCode

```

class Solution {
public: int findDuplicate(int[] nums) {
    int slow = nums[nums[0]];
    int fast = nums[nums[nums[0]]];

    while (slow != fast) {
        slow = nums[slow];
        fast = nums[nums[fast]];
    }

    slow = nums[0];

    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}
}

```

Java

```

class Solution {
public: int findDuplicate(int[] nums) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int mid = (l + r) >> 1;
        int cnt = 0;
        for (int v : nums) {
            if (v == mid) {
                ++cnt;
            }
        }
        if (cnt > mid) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}
}

```

Java

Bit Manipulation · LeetCode · By Pattern

— 443 —

LeetCode

```

class Solution {
public: int findDuplicate(int[] nums) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int mid = (l + r) >> 1;
        int cnt = 0;
        for (int v : nums) {
            if (v == mid) {
                ++cnt;
            }
        }
        if (cnt > mid) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}
}

```

JavaScript

```

// @ts-ignore
function findDuplicate(nums: number[]): number {
    let l = 0;
    let r = nums.length - 1;
    while (l < r) {
        let mid = (l + r) >> 1;
        let cnt = 0;
        for (const v of nums) {
            if (v == mid) {
                ++cnt;
            }
        }
        if (cnt > mid) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}

```

JavaScript

Bit Manipulation · LeetCode · By Pattern

— 444 —

LeetCode

```

//
// @param {number[]} nums
// @return {number}
//
var findDuplicate = function(nums) {
    let l = 0;
    let r = nums.length - 1;
    while (l < r) {
        const mid = (l + r) >> 1;
        let cnt = 0;
        for (const v of nums) {
            if (v == mid) {
                ++cnt;
            }
        }
        if (cnt > mid) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
};

```

TypeScript

```

// @ts-ignore
function findDuplicate(nums: number[]): number {
    let l = 0;
    let r = nums.length - 1;
    while (l < r) {
        const mid = (l + r) >> 1;
        let cnt = 0;
        for (const v of nums) {
            if (v == mid) {
                ++cnt;
            }
        }
        if (cnt > mid) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}

```

TypeScript

```

function findDuplicate(nums: number[]): number {
    let l = 0;
    let r = nums.length - 1;
    while (l < r) {
        const mid = (l + r) >> 1;
        let cnt = 0;
        for (const v of nums) {
            if (v == mid) {
                ++cnt;
            }
        }
        if (cnt > mid) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}

```

Go

```

//
func findDuplicate(nums []int) int {
    return sort.Search(len(nums), func(x int) bool {
        cnt := 0
        for _, v := range nums {
            if v == x {
                cnt++
            }
        }
        return cnt > x
    })
}

```

Go

```

//
func findDuplicate(nums []int) int {
    return sort.Search(len(nums), func(x int) bool {
        cnt := 0
        for _, v := range nums {
            if v == x {
                cnt++
            }
        }
        return cnt > x
    })
}

```

Rust

Rust

```

impl Solution {
    // @param {vector<int>}
    // @return {int}
    pub fn find_duplicate(nums: Vec<i32>) -> i32 {
        let mut left = 0;
        let mut right = nums.len() - 1;

        while left < right {
            let mid = (left + right) >> 1;
            let cnt = nums.iter().filter(|&x| *x == mid as i32).count();
            if cnt > mid {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        left as i32
    }
}

```

Rust

```

impl Solution {
    // @param {vector<int>}
    // @return {int}
    pub fn find_duplicate(nums: Vec<i32>) -> i32 {
        let mut left = 0;
        let mut right = nums.len() - 1;

        while left < right {
            let mid = (left + right) >> 1;
            let cnt = nums
                .iter()
                .filter(|&x| *x == mid as i32)
                .count();
            if cnt > mid {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        left as i32
    }
}

```

[*] Bit Manipulation — Pattern Solution (matched from community answers)

C++

```

class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int l = 0, r = nums.size() - 1;
        while (l < r) {
            int mid = (l + r) >> 1;
            int cnt = 0;
            for (int& v : nums) {
                cnt += v == mid;
            }
            if (cnt > mid) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    }
};

```

#1506 MEDIUM

Find Root of N-Ary Tree

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Hash Table · Bit Manipulation · Tree · DFS

LeetCode Premium 98

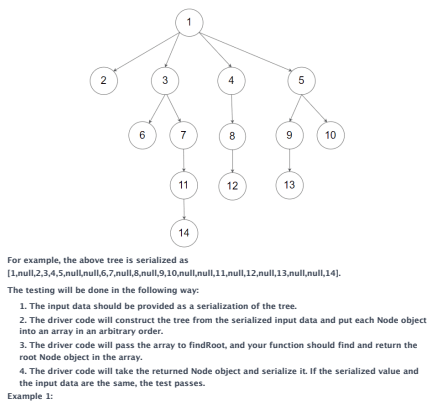
Problem:

You are given all the nodes of an N-ary tree as an array of Node objects, where each node has a unique value.

Return the root of the N-ary tree.

Custom testing:

An N-ary tree can be serialized as represented in its level order traversal where each group of children is separated by the null value (see examples).

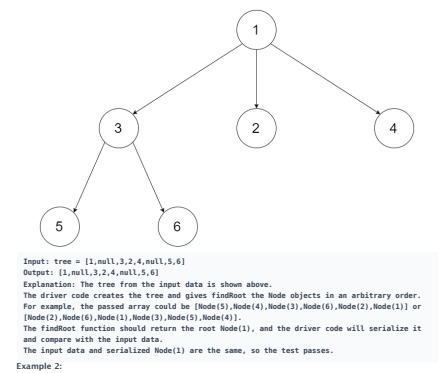


For example, the above tree is serialized as [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14].

The testing will be done in the following way:

1. The input data should be provided as a serialization of the tree.
2. The driver code will construct the tree from the serialized input data and put each Node object into an array in an arbitrary order.
3. The driver code will pass the array to findRoot, and your function should find and return the root Node object in the array.
4. The driver code will take the returned Node object and serialize it. If the serialized value and the input data are the same, the test passes.

Example 1:



Input: tree = [1,null,3,2,4,null,5,6]

Output: [1,null,3,2,4,null,5,6]

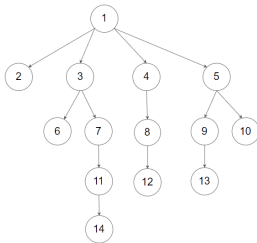
Explanation: The tree from the input data is shown above.

The driver code creates the tree and gives findRoot the Node objects in an arbitrary order. For example, the passed array could be [Node(3),Node(4),Node(1),Node(5),Node(2),Node(1)] or [Node(2),Node(6),Node(1),Node(3),Node(5),Node(4)].

The findRoot function should return the root Node(1), and the driver code will serialize it and compare with the input data.

The input data and serialized Node(1) are the same, so the test passes.

Example 2:



Input: tree =
[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output:
[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Constraints:

- The total number of nodes is between [1, 5 * 10⁴].
- Each node has a unique value.

Follow up:

- Could you solve this problem in constant space complexity with a linear time algorithm?

>> Brute Force [Bit Manipulation] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def findRoot(self, tree):
        # Brute Force - Check each bit with a loop
        result = 0
        for i in range(32):
            bit = (1 << i) & 1
            result |= (bit << i)
        return result
```

Python
Python

```
# Python Implementation with Detailed Comments
def findRoot(self, tree):
    # Initializing variables to store the sum of all node values and children values
    total_sum = 0
    children_sum = 0

    # Iterate over each node in the tree array
    for node in tree:
        total_sum += node.val # Add current node's value to total_sum
        # Iterate over each child of the current node and add their values to children_sum
        for child in node.children:
            children_sum += child.val

    # The root's value is the difference between total_sum and children_sum
    root_val = total_sum - children_sum

    # Find and return the node with the value equal to root_val
    for node in tree:
        if node.val == root_val:
            return node
    return None # This should never happen if the input tree is valid
```

Python

```
class Solution:
    def findRoot(self, tree: List[Node]) -> 'Node':
        sum = 0

        for node in tree:
            sum += node.val
            for child in node.children:
                sum += child.val

        for node in tree:
            if node.val == sum:
                return node
```

Python

```
...
# Definition for a Node.
class Node:
    def __init__(self, val:Node, children:Node):
        self.val = val
        self.children = children if children is not None else []
...

class Solution:
    def findRoot(self, tree: List[Node]) -> 'Node':
        x = 0
        for node in tree:
            x += node.val
            for child in node.children:
                x += child.val
        return next((node for node in tree if node.val == x))
```

Python

```
...
# Definition for a Node.
class Node:
    def __init__(self, val:Node, children:Node):
        self.val = val
        self.children = children if children is not None else []
...

class Solution:
    def findRoot(self, tree: List[Node]) -> 'Node':
        x = 0
        for node in tree:
            x += node.val
            for child in node.children:
                x += child.val
        return next((node for node in tree if node.val == x))
```

C++

C++

```
class Solution {
public:
    Node* findRoot(vector<Node> tree) {
        int sum = 0;

        for (Node* node : tree) {
            sum += node->val;
            for (Node* child : node->children)
                sum += child->val;
        }

        for (Node* node : tree)
            if (node->val == sum)
                return node;

        throw;
    }
};
```

C++

```
// Definition for a Node.
class Node {
public:
    int val;
    vector<Node> children;

    Node() {}

    Node(int val, vector<Node> children) {
        this->val = val;
        this->children = children;
    }
};

class Solution {
public:
    Node* findRoot(vector<Node> tree) {
        int x = 0;
        for (Node* node : tree) {
            x += node->val;
            for (Node* child : node->children) {
                x += child->val;
            }
        }
        for (int i = 0; i < tree.size(); ++i) {
            if (tree[i]->val == x) {
                return tree[i];
            }
        }
    }
};
```

C++

```
// Definition for a Node.
class Node {
public:
    int val;
    vector<Node> children;

    Node() {}

    Node(int val, vector<Node> children) {
        this->val = val;
        this->children = children;
    }
};

class Solution {
public:
    Node* findRoot(vector<Node> tree) {
        int x = 0;
        for (Node* node : tree) {
            x += node->val;
            for (Node* child : node->children) {
                x += child->val;
            }
        }
        for (int i = 0; i < tree.size(); ++i) {
            if (tree[i]->val == x) {
                return tree[i];
            }
        }
    }
};
```

Java

Java

```
// Definition for a Node.
class Node {
public:
    int val;
    public List<Node> children;

    public Node() {}

    public Node(int val, List<Node> children) {
        this.val = val;
        this.children = new ArrayList<Node>();
    }

    public Node(int val, List<Node> children) {
        this.val = val;
        this.children = new ArrayList<Node>();
    }

    public Node(int val, List<Node> children) {
        this.val = val;
        this.children = children;
    }
};

class Solution {
    public Node findRoot(List<Node> tree) {
        int x = 0;
        for (Node node : tree) {
            x += node.val;
            for (Node child : node.children) {
                x += child.val;
            }
        }
        for (int i = 0; i < tree.size(); ++i) {
            if (tree.get(i).val == x) {
                return tree.get(i);
            }
        }
    }
}
```

Java


```

// Definition for a Node.
class Node {
public: int val;
       vector<Node*> children;

       Node() {}
       Node(int _val) {
           val = _val;
           children = vector<Node*>();
       }
       Node(int _val, vector<Node*> _children) {
           val = _val;
           children = _children;
       }
};

class Solution {
public: Node* findRoot(List<Node*> tree) {

    int x = 0;
    for (Node* node : tree) {
        x ^= node->val;
        for (Node* child : node->children) {
            x ^= child->val;
        }
    }
    for (int i = 0; i < tree->size(); i++) {
        if (tree->get(i)->val == x) {
            return tree->get(i);
        }
    }
}
}

```

TypeScript
TypeScript

```

// Definition for Node.
class Node {
public: int val;
       vector<Node*> children;

       Node() {}
       Node(int _val) {
           val = _val;
           children = vector<Node*>();
       }
       Node(int _val, vector<Node*> _children) {
           val = _val;
           children = _children;
       }
};

class Solution {
public: Node* findRoot(List<Node*> tree) {

    int x = 0;
    for (Node* node : tree) {
        x ^= node->val;
        for (Node* child : node->children) {
            x ^= child->val;
        }
    }
    for (int i = 0; i < tree->size(); i++) {
        if (tree->get(i)->val == x) {
            return tree->get(i);
        }
    }
}
}

```

TypeScript

```

// Definition for Node.
class Node {
public: int val;
       vector<Node*> children;

       Node() {}
       Node(int _val) {
           val = _val;
           children = vector<Node*>();
       }
       Node(int _val, vector<Node*> _children) {
           val = _val;
           children = _children;
       }
};

class Solution {
public: Node* findRoot(List<Node*> tree) {

    int x = 0;
    for (Node* node : tree) {
        x ^= node->val;
        for (Node* child : node->children) {
            x ^= child->val;
        }
    }
    for (int i = 0; i < tree->size(); i++) {
        if (tree->get(i)->val == x) {
            return tree->get(i);
        }
    }
}
}

```

[*] Bit Manipulation — Pattern Solution (matched from community answers)

C++

```

class Solution {
public:
    Node* findRoot(vector<Node*> tree) {
        int sum = 0;

        for (Node* node : tree) {
            sum ^= node->val;
            for (Node* child : node->children) {
                sum ^= child->val;
            }
        }

        for (Node* node : tree) {
            if (node->val == sum)
                return node;
        }

        throw;
    }
};

```

Quick Review — Bit Manipulation 10 questions · insights · complexity · solution

#67 Add Binary [Easy]

Key Insights

- Simulate binary addition just like you add numbers digit by digit.
- Process the strings from right to left, handling the carry at each step.
- Use modulo and division operations to extract the current digit and update the carry.
- Append any remaining carry at the end.

Space & Time

Time Complexity: $O(\max(m, n))$, where n and m are the lengths of the two binary strings.
Space Complexity: $O(\max(m, n))$ for storing the resulting binary string.

Solution

The strategy is to iterate over the input strings from the end to the beginning. At each step, calculate the sum of corresponding bits and the existing carry. Use the modulo operation ($\% 2$) to determine the current digit and integer division ($/ 2$) to update the carry. Once all digits have been processed, reverse the result to obtain the final binary string.

#136 Single Number [Easy]

Key Insights

- Using bitwise XOR: XOR-ing a number with itself cancels out to 0.
- XOR is both commutative and associative, so the order of operations does not matter.
- A single traversal (linear time) and constant extra space is sufficient for solving the problem.

Space & Time

Time Complexity: $O(n)$ - We only traverse the array once.
Space Complexity: $O(1)$ - Only a single extra variable is used regardless of input size.

Solution

We can solve this problem by initializing a variable to 0 and then traversing the array while XOR-ing each element with the variable. Due to the properties of XOR, pairs of identical numbers will cancel each other out ($x \oplus x = 0$) and the unique number will be left as the final result. This approach uses constant extra space and performs in linear time.

#190 Reverse Bits [Easy]

Key Insights

- The input is a binary string of fixed length (32 bits).
- Reversing the bits is equivalent to reversing the string.
- The reversed binary string can be converted back into its integer representation.
- Since the number of bits is constant (32), the solution can be achieved in constant time and constant space.
- For performance optimization when this function is called many times, caching may be applied for common intermediate results.

Space & Time

Time Complexity: $O(1)$ - We always process 32 bits regardless of the input.
Space Complexity: $O(1)$ - Only constant extra space is used.

Solution

We solve the problem by reversing the 32-character binary string representing the unsigned integer. Once reversed, the new binary string is converted into its integer value. An alternative solution involves using bitwise operations

where you iterate over each bit, shift the result to the left, and add the current bit. This approach does the reversal one bit at a time. The chosen approach leverages string reversal which is straightforward to implement and understand.

#101 Number of 1 Bits [Easy]

Key Insights

- The problem is essentially counting the number of 1s in the binary representation of a number.
- A common efficient approach uses bit manipulation: repeatedly clear the lowest set bit until n becomes zero.
- The expression $n \& (n - 1)$ clears the lowest set bit of n .
- This method only iterates as many times as there are set bits, making it efficient.

Space & Time

Time Complexity: $O(k)$, where k is the number of set bits, worst-case $O(\log n)$ (or $O(1)$ considering 32-bit integers).
Space Complexity: $O(1)$.

Solution

To solve the problem, we use the bit manipulation trick $n \& (n - 1)$ which removes the lowest set bit from n . The algorithm is as follows:

- Initialize a counter to 0.
- While n is not zero, update n by $n \& (n - 1)$, and increment the counter.
- The counter will hold the number of set bits by the end of the loop. This approach leverages efficient bit-level operations and works well even when the function is called frequently.

#268 Missing Number [Easy]

Key Insights

- The range of numbers is continuous from 0 to n , but one number is missing.
- The sum of numbers from 0 to n can be computed using the arithmetic sum formula.
- Subtracting the sum of the given array from the expected sum gives the missing number.
- There is a follow-up challenge to solve the problem in $O(n)$ time and $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$
Space Complexity: $O(1)$

Solution

We compute the expected sum of all numbers from 0 to n using the formula $\text{sum} = n \cdot (n + 1) / 2$. Then, we compute the actual sum of the numbers in the array. The missing number is obtained by subtracting the actual sum from the expected sum. This approach uses arithmetic operations ($O(1)$ extra space) and loops over the array once ($O(n)$ time).

#338 Counting Bits [Easy]

Key Insights

- We can use dynamic programming to build the solution by utilizing the count of bits for smaller numbers.
- The relationship $\text{dp}[i] = \text{dp}[i / 2] + (i \& 1)$ holds, where $i > 1$ gives the count for i divided by 2 (dropping the least significant bit) and $(i \& 1)$ checks if the least significant bit is 1.
- This approach computes the result for each number in constant time, resulting in a linear time solution.

Space & Time

Time Complexity: $O(n)$
Space Complexity: $O(n)$

Solution

We use a dynamic programming approach where an array (or list) dp is initialized with size $n + 1$. The key idea is to iterate from 1 to n and for each number, use the formula $\text{dp}[i] = \text{dp}[i / 2] + (i \& 1)$.

- The expression $(i > 1)$ effectively divides i by 2, giving the count of 1s for that half number. - The $(i \& 1)$ operation checks if i is odd, adding 1 if true. This method ensures each $\text{dp}[i]$ is computed in constant time, leading to an overall $O(n)$ time complexity.

#78 Subsets [Medium]

Key Insights

- The problem is equivalent to generating the power set of the given set.
- A recursive backtracking approach can be used to explore both including and excluding each element.
- Iterative (bit manipulation) solutions are also possible since there are 2^n subsets in total.
- The maximum array length is small (up to 10), making exponential time solutions feasible.

Space & Time

Time Complexity: $O(2^n \cdot n)$ - There are 2^n subsets and creating each subset may take up to $O(n)$ time.
Space Complexity: $O(2^n \cdot n)$ - Space for storing all subsets. The recursion stack has a maximum depth of $O(n)$.

Solution

We solve the problem using backtracking, where we decide for each element whether to include it in the current subset or not. We recursively build all subsets starting at an empty list and at each step appending the current subset to our result list. We can also use iterative bit manipulation but here we focus on the backtracking approach. This algorithm leverages recursion and a list to maintain the current state, leading to a clear and concise implementation.

#80 Subsets II [Medium]

Key Insights

- Sort the array first so that duplicate elements are adjacent.
- Use backtracking to generate all possible subsets.
- Skip duplicate elements in the recursion to avoid forming duplicate subsets.
- The use of sorted order and conditionally skipping elements is the trick to handle duplicates.

Space & Time

Time Complexity: $O(2^n \cdot n)$ since each subset creation may take $O(n)$ in the worst case.
Space Complexity: $O(2^n \cdot n)$ for storing the subsets, plus $O(n)$ for recursion stack.

Solution

The solution uses a backtracking technique that builds subsets incrementally. The input array is first sorted to bring duplicates together. During recursion, if the current element is the same as the previous one and it has not been used in the current recursion path, it is skipped to prevent duplicate subsets. This ensures that each subset is unique. The algorithm stores each valid subset by copying the current subset state into the result list.

#287 Find the Duplicate Number [Medium]

Key Insights

- Since there are $n + 1$ elements with values only from 1 to n , by the pigeonhole principle at least one number must be repeated.
- The problem can be approached by mapping the array values to indices, forming a cycle structure.
- Floyd's Tortoise and Hare algorithm (cycle detection) can be applied to find the duplicate number in linear time and constant space.
- There is no need for extra data structures like hash sets or alterations to the original array, satisfying the space and array-modification constraints.

Space & Time

Time Complexity: $O(n)$
Space Complexity: $O(1)$

Solution

We use Floyd's Tortoise and Hare algorithm to detect a cycle in the sequence of numbers. The sequence starts at the first element of the array and follows the mapping $\text{nums}[\text{nums}[i]]$. Since there is a duplicate, a cycle will eventually form. By using two pointers (tortoise and hare) moving at different speeds, we can find the start of the cycle, which corresponds to the duplicate number.

The solution leverages Floyd's Tortoise and Hare cycle detection algorithm. Think of each array element as a pointer to another index, forming a linked list with a cycle (the duplicate number creates the cycle). The algorithm works in two phases:

- Detect a meeting point within the cycle by having two pointers (tortoise and hare) move at different speeds. - Find the cycle entrance by resetting one pointer to the start and moving both pointers at the same speed. The meeting point reveals the duplicate number.
- This approach provides an elegant solution that satisfies both linear runtime and constant space requirements.

#1506 Find Root of N-Ary Tree [Medium]

Key Insights

- Every node except the root appears as a child of another node.
- By summing all node values and then subtracting the sum of all children node values, the remaining value corresponds to the root.
- This subtraction trick allows us to identify the root in a single pass of the data structure without extra memory for tracking child nodes.

Space & Time

Time Complexity: O(N) where N is the number of nodes, since we iterate through all nodes and their children.
Space Complexity: O(1) extra space using arithmetic accumulators.

Solution

The solution calculates two sums: one for all nodes' values and one for all children nodes' values. The difference between these gives the value of the root node since every node's value (except the root's) is added once as a child. Finally, we iterate over the nodes to find the node with the computed root value and return it. This approach meets the requirement of constant space complexity and linear time.

#8 Backtracking — 10 questions · #8 of 21

#17

MEDIUM

Letter Combinations of a Phone Number

LeetCode · Simple · LeetCode · LeetCode · Editorial

Hash Table · String · Backtracking

Lists · Grid · 169

Problem:

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

Input: digits = "2"
Output: ["a","b","c"]

Constraints:

- 1 <= digits.length <= 4

- digits[i] is a digit in the range ['2', '9'].

>> Brute Force [Backtracking] Interview Prep

Python

Brute Force Approach

```
class Solution:
    def letterCombinations(self, digits):
        # Brute Force O(4^n) - O(1) - itertools.product over mapped letters
        if not digits: return []
        from itertools import product
        mapping = {'2':'abc','3':'def','4':'ghi','5':'jkl',
                  '6':'mno','7':'pqrs','8':'tuv','9':'wxyz'}
        return [''.join(c) for c in product(*[mapping[d] for d in digits])]
```

Python

Python

Python solution for Letter Combinations of a Phone Number

```
def letterCombinations(digits):
    # Mapping of digits to corresponding letters
    digit_to_letters = {
        '2': 'abc',
        '3': 'def',
        '4': 'ghi',
        '5': 'jkl',
        '6': 'mno',
        '7': 'pqrs',
        '8': 'tuv',
        '9': 'wxyz'
    }

    # If input is empty, return an empty list
    if not digits:
        return []

    result = []

    # Backtracking function to generate combinations
    def backtrack(index, current_combination):
        # If the current combination length equals the digits length,
        # add the combination to the result list.
```

```
if index == len(digits):
    result.append(current_combination)
    return

# Get the corresponding letters for the current digit
letters = digit_to_letters[digits[index]]
# Iterate over all possible letters for the current digit.
for letter in letters:
    backtrack(index + 1, current_combination + letter)

# Initialize backtracking starting from index 0 and empty combination.
backtrack(0, "")
return result

# Example usage:
print(letterCombinations("23"))
```

Python

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []

        digitToLetters = {'2': 'abc', '3': 'def', '4': 'ghi',
                          '5': 'jkl', '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'}
        ans = []

        def dfs(i: int, path: List[str]) -> None:
            if i == len(digits):
                ans.append(''.join(path))
                return

            for letter in digitToLetters[digits[i]]:
                path.append(letter)
                dfs(i + 1, path)
                path.pop()

        dfs(0, [])
        return ans
```

Python

```
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = []
        for i in digits:
            s = digitToLetters[i]
            ans = [a + b for a in ans for b in s]
        return ans
```

Python

```
class Solution {
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = []
        for i in digits:
            s = digitToLetters[i]
            ans = [a + b for a in ans for b in s]
        return ans
```

C++

C++

```
class Solution {
public:
    vector<string> letterCombinations(string digits) {
        if (digits.empty())
            return {};

        vector<string> ans;

        dfs(digits, 0, "", ans);
        return ans;
    }

private:
    const vector<string> digitToLetters = {"", "", "abc", "def", "ghi",
                                           "jkl", "mno", "pqrs", "tuv", "wxyz"};

    void dfs(const string& digits, int i, string& path, vector<string>& ans) {
        if (i == digits.length()) {
            ans.push_back(path);
            return;
        }

        for (const char letter : digitToLetters[digits[i] - '0']) {
            path.push_back(letter);
            dfs(digits, i + 1, string(path), ans);
            path.pop_back();
        }
    }
};
```

C++

```
class Solution {
public:
    vector<string> letterCombinations(string digits) {
        if (digits.empty()) {
            return {};
        }
        vector<string> d = {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
        vector<string> ans = {};
        for (auto i : digits) {
            string s = d[i - '2'];
            vector<string> t;
            for (auto b : ans) {
                for (auto a : s) {
                    t.push_back(a + b);
                }
            }
            ans = move(t);
        }
        return ans;
    }
};
```

C++

```
class Solution {
//
// Given string digits
// Return string[]
//
function letterCombinations(digits) {
    digitMap = {
        '2': ['a', 'b', 'c'],
        '3': ['d', 'e', 'f'],
        '4': ['g', 'h', 'i'],
        '5': ['j', 'k', 'l'],
        '6': ['m', 'n', 'o'],
        '7': ['p', 'q', 'r', 's'],
        '8': ['t', 'u', 'v'],
        '9': ['w', 'x', 'y', 'z']
    };
    combinations = [];
    backtrack(digits, "", 0, digitMap, combinations);
    return combinations;
}
```

```

function backtrack(digits, scurrent, index, digitMap, combinations) {
  if (index === string(digits)) {
    if (scurrent === '') {
      combinations[] = scurrent;
    }
    return;
  }
  digit = digits[index];
  slatters = digitMap[digit];
  for each (latters as slatter) {
    backtrack(digits, scurrent + slatter, index + 1, digitMap, combinations);
  }
}

```

C++

```

class Solution {
public:
    vector<string> letterCombinations(string digits) {
        if (digits.empty()) {
            return {};
        }
        vector<string> d = {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
        vector<string> ans = {""};
        for (auto i : digits) {
            vector<string> ans2 = {""};
            string s = d[i] + " ";
            vector<string> t;
            for (auto a : ans) {
                for (auto b : s) {
                    t.push_back(a + b);
                }
            }
            ans = move(t);
        }
        return ans;
    }
};

```

C++

```

class Solution {
  //
  // @param string digits
  // @return string[]
  //
  function letterCombinations(digits) {
    digitMap = {
      '2': ['a', 'b', 'c'],
      '3': ['d', 'e', 'f'],
      '4': ['g', 'h', 'i'],
      '5': ['j', 'k', 'l'],
      '6': ['m', 'n', 'o'],
      '7': ['p', 'q', 'r', 's'],
      '8': ['t', 'u', 'v'],
      '9': ['w', 'x', 'y', 'z'],
    };
    combinations = [];
    this--backtrack(digits, '', 0, digitMap, combinations);
    return combinations;
  }
}

```

```

function backtrack(digits, scurrent, index, digitMap, combinations) {
  if (index === string(digits)) {
    if (scurrent === '') {
      combinations[] = scurrent;
    }
    return;
  }
  digit = digits[index];
  slatters = digitMap[digit];
  for each (latters as slatter) {
    this--backtrack(digits, scurrent + slatter, index + 1, digitMap, combinations);
  }
}

```

Java

Java

```

class Solution {
  public List<String> letterCombinations(String digits) {
    if (digits.isEmpty())
      return new ArrayList<>();

    List<String> ans = new ArrayList<>();

    dfs(digits, 0, new StringBuilder(), ans);
    return ans;
  }

  private static final String[] digitToLetters = {"", "", "abc", "def", "ghi",
    "jkl", "mno", "pqrs", "tuv", "wxyz"};

  private void dfs(String digits, int i, StringBuilder sb, List<String> ans) {
    if (i == digits.length()) {
      ans.add(sb.toString());
      return;
    }

    for (final char c : digitToLetters[digits.charAt(i) - '0'].toCharArray()) {
      sb.append(c);
      dfs(digits, i + 1, sb, ans);
      sb.deleteCharAt(sb.length() - 1);
    }
  }
}

```

Java

```

class Solution {
  public List<String> letterCombinations(String digits) {
    List<String> ans = new ArrayList<>();
    if (digits.length() == 0) {
      return ans;
    }
    ans.add("");
    String[] d = new String[] {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    for (char i : digits.toCharArray()) {
      String s = d[i - '2'];
      List<String> t = new ArrayList<>();
      for (String a : ans) {
        for (String b : s.split("")) {
          t.add(a + b);
        }
      }
      ans = t;
    }
    return ans;
  }
}

```

Java

```

public class Solution {
  public List<String> letterCombinations(string digits) {
    var ans = new List<string>();
    if (digits.Length == 0) {
      return ans;
    }
    ans.Add("");
    string[] d = {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    foreach (char i in digits) {
      string s = d[i - '2'];
      var t = new List<string>();
      foreach (string a in ans) {
        foreach (char b in s) {
          t.Add(a + b);
        }
      }
      ans = t;
    }
    return ans;
  }
}

```

Java

```

class Solution {
  private final String[] d = {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
  private String digits;
  private List<String> ans = new ArrayList<>();
  private StringBuilder t = new StringBuilder();

  public List<String> letterCombinations(String digits) {
    if (digits.Length() == 0) {
      return ans;
    }
    this.digits = digits;
    dfs(0);
    return ans;
  }

  private void dfs(int i) {
    if (i == digits.Length()) {
      ans.add(t.toString());
      return;
    }
    String s = d[digits.charAt(i) - '2'];
    for (char c : s.toCharArray()) {
      t.append(c);
      dfs(i + 1);
      t.deleteCharAt(t.Length() - 1);
    }
  }
}

```

Java

```

public class Solution {
  private random[] string[] d = {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
  private string digits;
  private List<string> ans = new List<string>();
  private System.Text.StringBuilder t = new System.Text.StringBuilder();

  public List<string> letterCombinations(string digits) {
    if (digits.Length == 0) {
      return ans;
    }
    this.digits = digits;
    dfs(0);
    return ans;
  }

  private void dfs(int i) {
    if (i == digits.Length) {
      ans.Add(t.ToString());
      return;
    }
    string s = d[digits[i] - '2'];
    foreach (char c in s) {
      t.Append(c);
      dfs(i + 1);
      t.Remove(t.Length - 1, 1);
    }
  }
}

```

Java

```

class Solution {
  public List<String> letterCombinations(String digits) {
    List<String> ans = new ArrayList<>();
    if (digits.length() == 0) {
      return ans;
    }
    ans.add("");
    String[] d = new String[] {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    for (char i : digits.toCharArray()) {
      String s = d[i - '2'];
      List<String> t = new ArrayList<>();
      for (String a : ans) {
        for (String b : s.split("")) {
          t.add(a + b);
        }
      }
      ans = t;
    }
    return ans;
  }
}

```

Java

```

public class Solution {
  public List<String> letterCombinations(string digits) {
    var ans = new List<string>();
    if (digits.Length == 0) {
      return ans;
    }
    ans.Add("");
    string[] d = {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    foreach (char i in digits) {
      string s = d[i - '2'];
      var t = new List<string>();
      foreach (string a in ans) {
        foreach (char b in s) {
          t.Add(a + b);
        }
      }
      ans = t;
    }
    return ans;
  }
}

```

JavaScript

JavaScript

```

//
// @param (string) digits
// @return (string[])
//
var letterCombinations = function (digits) {
  if (digits.length === 0) {
    return [];
  }
  const ans = [""];
  const d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"];
  for (const i of digits) {
    const s = d[i - 2];
    const t = [];
    for (const a of ans) {
      for (const b of s) {
        t.push(a + b);
      }
    }
    ans.splice(0, ans.length, ...t);
  }
  return ans;
};

```

JavaScript

```

//
+ param (string) digits
+ return [string]()
}

var letterCombinations = function (digits) {
    if (digits.length == 0) {
        return [];
    }
    const ans = [];
    const d = ['abc', 'def', 'ghi', 'jkl', 'mno', 'pqrs', 'tuv', 'wxyz'];
    for (const i of digits) {
        const s = d[params[i] - 1];
        const t = [];
        for (const a of ans) {
            for (const b of s) {
                t.push(a + b);
            }
        }
        ans.splice(0, ans.length, ...t);
    }
    return ans;
};

```

TypeScript

```

function letterCombinations(digits: string): string[] {
    if (digits.length == 0) {
        return [];
    }
    const ans: string[] = [];
    const d = ['abc', 'def', 'ghi', 'jkl', 'mno', 'pqrs', 'tuv', 'wxyz'];
    for (const i of digits) {
        const s = d[i - 1];
        const t: string[] = [];
        for (const a of ans) {
            for (const b of s) {
                t.push(a + b);
            }
        }
        ans.splice(0, ans.length, ...t);
    }
    return ans;
}

```

TypeScript

Backtracking · LetterMastery — By Pattern

— 475 —

LetterMastery

```

function letterCombinations(digits: string): string[] {
    if (digits.length == 0) {
        return [];
    }
    const ans: string[] = [];
    const d = ['abc', 'def', 'ghi', 'jkl', 'mno', 'pqrs', 'tuv', 'wxyz'];
    for (const i of digits) {
        const s = d[params[i] - 1];
        const t: string[] = [];
        for (const a of ans) {
            for (const b of s) {
                t.push(a + b);
            }
        }
        ans.splice(0, ans.length, ...t);
    }
    return ans;
}

```

Go

```

func letterCombinations(digits string) []string {
    ans := []string{}
    if len(digits) == 0 {
        return ans
    }
    ans = append(ans, "")
    d := []string{"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"}
    for i := range digits {
        s := d[i-1]
        t := []string{}
        for a := range ans {
            for b := range s {
                t = append(t, astring(b))
            }
        }
        ans = t
    }
    return ans
}

```

Go

Backtracking · LetterMastery — By Pattern

— 476 —

LetterMastery

```

func letterCombinations(digits string) ans []string {
    d := []string{"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"}
    t := []string{}
    var dfs func(int)
    dfs = func(i int) {
        if i == len(digits) {
            return
        }
        for c := range d[digits[i]-1] {
            t = append(t, c)
            dfs(i + 1)
            t = t[:len(t)-1]
        }
    }
    if len(digits) == 0 {
        return
    }
    dfs(0)
    return
}

```

Go

```

func letterCombinations(digits string) []string {
    ans := []string{}
    if len(digits) == 0 {
        return ans
    }
    ans = append(ans, "")
    d := []string{"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"}
    for i := range digits {
        s := d[i-1]
        t := []string{}
        for a := range ans {
            for b := range s {
                t = append(t, astring(b))
            }
        }
        ans = t
    }
    return ans
}

```

Rust

Rust

Backtracking · LetterMastery — By Pattern

— 477 —

LetterMastery

```

impl Solution {
    pub fn letter_combinations(digits: String) -> Vec<String> {
        let mut ans: Vec<String> = Vec::new();
        if digits.is_empty() {
            return ans;
        }
        ans.push("");
        let d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"];
        for i in digits.chars() {
            let s = d[(i as u8 - b'0') as usize];
            let mut t: Vec<String> = Vec::new();
            for a in ans {
                for b in s.chars() {
                    t.push(format!("{}", a + b));
                }
            }
            ans = t;
        }
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn letter_combinations(digits: String) -> Vec<String> {
        let mut ans: Vec<String> = Vec::new();
        if digits.is_empty() {
            return ans;
        }
        ans.push("");
        let d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"];
        for i in digits.chars() {
            let s = d[(i as u8 - b'0') as usize];
            let mut t: Vec<String> = Vec::new();
            for a in ans {
                for b in s.chars() {
                    t.push(format!("{}", a + b));
                }
            }
            ans = t;
        }
        ans
    }
}

```

[*] Backtracking — Pattern Solution (matched from community answers)

Python

Backtracking · LetterMastery — By Pattern

— 478 —

LetterMastery

```

# Python solution for Letter Combinations of a Phone Number

def letterCombinations(digits):
    # Mapping of digits to corresponding letters
    digit_to_letters = {
        "2": "abc",
        "3": "def",
        "4": "ghi",
        "5": "jkl",
        "6": "mno",
        "7": "pqrs",
        "8": "tuv",
        "9": "wxyz"
    }

    # If input is empty, return an empty list
    if not digits:
        return []

    result = []

    # Backtracking function to generate combinations
    def backtrack(index, current_combination):
        # If the current combination length equals the digits length,
        # add the combination to the result list.
        if index == len(digits):
            result.append(current_combination)
            return

        # Get the corresponding letters for the current digit
        letters = digit_to_letters[digits[index]]
        # Explore all possible letter nodes for the current digit.
        for letter in letters:
            backtrack(index + 1, current_combination + letter)

    # Initiate backtracking starting from index 0 and empty combination.
    backtrack(0, "")
    return result

# Example usage:
print(letterCombinations("23"))

```

#22

MEDIUM

Generate Parentheses

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

String · Dynamic Programming · Backtracking

Lists · Grid 169

Problem:

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

Backtracking · LetterMastery — By Pattern

— 479 —

LetterMastery

```

Input: n = 3
Output: ["((()))","(()())","(())()","()(())","()()()"]

Example 2:

Input: n = 1
Output: ["()"]

Constraints:

• 1 <= n <= 8

```

>> Brute Force (Backtracking) Interview Prep

Python

```

# Brute Force Solution
class Solution:
    def generateParenthesis(self, n):
        # Brute Force O(2^n) -> generate all 2^n-length binary strings. Filter valid
        def is_valid(comb):
            bal = 0
            for c in comb:
                bal += 1 if c == '(' else -1
                if bal < 0: return False
            return bal == 0
        from itertools import product
        return [''.join(c) for c in product('()', repeat=2*n) if is_valid(c)]

```

Python

```

Python
def generateParenthesis(n):
    # List to store valid combinations
    result = []

    # Helper function for backtracking
    def backtrack(current, open_count, close_count):
        # If current string reaches the maximum length, it's a valid combination
        if len(current) == 2 * n:
            result.append(current)
            return

        # If we can add an opening parenthesis, do it
        if open_count < n:
            backtrack(current + '(', open_count + 1, close_count)

        # If adding a closing parenthesis maintains the validity, do it
        if close_count < open_count:
            backtrack(current + ')', open_count, close_count + 1)

    # Start the recursion with an empty string and zero counts for both
    backtrack("", 0, 0)
    return result

# Example usage:
print(generateParenthesis(3))

```

Python

Backtracking · LetterMastery — By Pattern

— 480 —

LetterMastery

```

class Solution {
    def generateParenthesis(self, n):
        ans = []

        def dfs(l: int, r: int, s: list[str]) -> None:
            if l == 0 and r == 0:
                ans.append(''.join(s))
            if l > 0:
                s.append('(')
                dfs(l - 1, r, s)
                s.pop()
            if l < r:
                s.append(')')
                dfs(l, r - 1, s)
                s.pop()

        dfs(n, 0)
        return ans

```

Python

```

class Solution:
    def generateParenthesis(self, n: int) -> list[str]:
        def dfs(l: int, r: int, s: str):
            if l > n or r > n or l < r:
                return
            if l == 0 and r == 0:
                ans.append(s)
            dfs(l + 1, r, s + "(")
            dfs(l, r + 1, s + ")")

        ans = []
        dfs(0, 0, "")
        return ans

```

Python

Backtracking · LeetCode · By Pattern

— 481 —

LeetCode

```

class Solution:
    def generateParenthesis(self, n: int) -> list[str]:
        def dfs(l: int, r: int):
            if l > n or r > n or l < r:
                return
            if l == 0 and r == 0:
                ans.append(''.join(s))
            if l > 0:
                s.append('(')
                dfs(l - 1, r, s + "(")
                s.pop()
            if l < r:
                s.append(')')
                dfs(l, r - 1, s + ")")
                s.pop()

        ans = []
        dfs(0, 0, "")
        return ans

```

Python

```

class Solution(object):
    def generateParenthesis(self, n):
        (type n: int
         type s: list(str))

        def dfs(left, path, res, n):
            if len(path) == 2 * n:
                if left == 0:
                    res.append(''.join(path))
                return
            if left < n:
                path.append('(')
                dfs(left - 1, path, res, n)
                path.pop()
            if left > 0:
                path.append(')')
                dfs(left - 1, path, res, n)
                path.pop()

        res = []
        dfs(0, [], res, n)
        return res

```

C++

C++

Backtracking · LeetCode · By Pattern

— 482 —

LeetCode

```

class Solution {
public:
    vector<string> generateParenthesis(int n) {
        vector<string> ans;
        dfs(0, "", ans);
        return ans;
    }

private:
    void dfs(int l, int r, string& path, vector<string>& ans) {
        if (l == 0 && r == 0) {
            ans.push_back(path);
            return;
        }
        if (l > 0) {
            path.push_back('(');
            dfs(l - 1, r, string(path), ans);
            path.pop_back();
        }
        if (l < r) {
            path.push_back(')');
            dfs(l, r - 1, string(path), ans);
            path.pop_back();
        }
    }
};

```

C++

```

class Solution {
public:
    vector<string> generateParenthesis(int n) {
        vector<string> ans;
        auto dfs = [&](this auto& dfs, int l, int r, string t) -> void {
            if (l >= n || r >= n || l < r) {
                return;
            }
            if (l == 0 && r == 0) {
                ans.push_back(t);
                return;
            }
            dfs(l + 1, r, t + "(");
            dfs(l, r + 1, t + ")");
        };
        dfs(0, 0, "");
        return ans;
    }
};

```

C++

Backtracking · LeetCode · By Pattern

— 483 —

LeetCode

```

class Solution {
    private int n = 1;
    private int k = 0;

    // 生成一个整数 0n
    // 返回一个字符串
    // 生成一个字符串
    // 生成一个字符串

    private function dfs(int l, int r, int k) {
        if (k > n || k < 0 || l < r) {
            return;
        }
        if (l == 0 && r == 0) {
            k <= n;
            return;
        }
        if (l < r) {
            k <= n;
            dfs(l + 1, r, k + 1);
            k <= n;
            dfs(l, r + 1, k + 1);
        }
    }
}

```

C++

```

class Solution {
public:
    vector<string> generateParenthesis(int n) {
        vector<string> ans;
        function<void(int, string)> dfs = [&](int l, int r, string t) {
            if (l >= n || r >= n || l < r) return;
            if (l == 0 && r == 0) {
                ans.push_back(t);
                return;
            }
            dfs(l + 1, r, t + "(");
            dfs(l, r + 1, t + ")");
        };
        dfs(0, 0, "");
        return ans;
    }
};

```

C++

Backtracking · LeetCode · By Pattern

— 484 —

LeetCode

```

class Solution {
    // 生成一个整数 0n
    // 返回一个字符串
    // 生成一个字符串
    // 生成一个字符串

    function generateParenthesis(n) {
        result = [];
        this.backtrack(result, "", 0, 0, n);
        return result;
    }

    function backtrack(result, current, open, close, nmax) {
        if (current.length === nmax + 2) {
            result.push(current);
            return;
        }
        if (open < nmax) {
            this.backtrack(result, current + '(', open + 1, close, nmax);
        }
        if (close < open) {
            this.backtrack(result, current + ')', open, close + 1, nmax);
        }
    }
}

```

Java

```

class Solution {
    public List<String> generateParenthesis(int n) {
        List<String> ans = new ArrayList<>();
        dfs(0, 0, new StringBuilder(), ans);
        return ans;
    }

    private void dfs(int l, int r, StringBuilder sb, List<String> ans) {
        if (l == 0 && r == 0) {
            ans.add(sb.toString());
            return;
        }
        if (l > 0) {
            sb.append("(");
            dfs(l - 1, r, sb, ans);
            sb.deleteCharAt(sb.length() - 1);
        }
        if (l < r) {
            sb.append(")");
            dfs(l, r - 1, sb, ans);
            sb.deleteCharAt(sb.length() - 1);
        }
    }
}

```

Java

Backtracking · LeetCode · By Pattern

— 485 —

LeetCode

```

class Solution {
    private List<String> ans = new ArrayList<>();
    private int n;

    public List<String> generateParenthesis(int n) {
        this.n = n;
        dfs(0, 0, "");
        return ans;
    }

    private void dfs(int l, int r, String t) {
        if (l >= n || r >= n || l < r) {
            return;
        }
        if (l == 0 && r == 0) {
            ans.add(t);
            return;
        }
        dfs(l + 1, r, t + "(");
        dfs(l, r + 1, t + ")");
    }
}

```

Java

```

public class Solution {
    private List<String> ans = new List<String>();
    private int n;

    public List<String> generateParenthesis(int n) {
        this.n = n;
        dfs(0, 0, "");
        return ans;
    }

    private void dfs(int l, int r, string t) {
        if (l >= n || r >= n || l < r) {
            return;
        }
        if (l == 0 && r == 0) {
            ans.add(t);
            return;
        }
        dfs(l + 1, r, t + "(");
        dfs(l, r + 1, t + ")");
    }
}

```

Java

Backtracking · LeetCode · By Pattern

— 486 —

LeetCode

```

class Solution {
private List<String> ans = new ArrayList<>();
private int n;

public List<String> generateParenthesis(int n) {
    this.n = n;
    dfs(0, "");
    return ans;
}

private void dfs(int l, int r, String t) {
    if (l > n || r > n || l < r) {
        return;
    }
    if (l == n && r == n) {
        ans.add(t);
        return;
    }
    dfs(l + 1, r, t + "(");
    dfs(l, r + 1, t + ")");
}
}

```

```

Java
public class Solution {
private List<String> ans = new List<String>();
private int n;

public List<String> GenerateParenthesis(int n) {
    this.n = n;
    dfs(0, "");
    return ans;
}

private void dfs(int l, int r, String t) {
    if (l > n || r > n || l < r) {
        return;
    }
    if (l == n && r == n) {
        ans.add(t);
        return;
    }
    dfs(l + 1, r, t + "(");
    dfs(l, r + 1, t + ")");
}
}

```

JavaScript
JavaScript

Backtracking · LeetCode · By Pattern

— 487 —

LeetCode

```

//
// @param {number} n
// @return {string[]}
//
var generateParenthesis = function(n) {
    const dfs = (l, r, t) => {
        if (l > n || r > n || l < r) {
            return;
        }
        if (l == n && r == n) {
            ans.push(t);
            return;
        }
        dfs(l + 1, r, t + "(");
        dfs(l, r + 1, t + ")");
    };
    const ans = [];
    dfs(0, 0, "");
    return ans;
};

```

JavaScript

```

//
// @param {number} n
// @return {string[]}
//
var generateParenthesis = function(n) {
    function dfs(l, r, t) {
        if (l > n || r > n || l < r) {
            return;
        }
        if (l == n && r == n) {
            ans.push(t);
            return;
        }
        dfs(l + 1, r, t + "(");
        dfs(l, r + 1, t + ")");
    }

    let ans = [];
    dfs(0, 0, "");
    return ans;
};

```

TypeScript
TypeScript

Backtracking · LeetCode · By Pattern

— 488 —

LeetCode

```

function generateParenthesis(n: number): string[] {
    const dfs = (l: number, r: number, t: string) => {
        if (l > n || r > n || l < r) {
            return;
        }
        if (l == n && r == n) {
            ans.push(t);
            return;
        }
        dfs(l + 1, r, t + "(");
        dfs(l, r + 1, t + ")");
    };
    const ans: string[] = [];
    dfs(0, 0, "");
    return ans;
}

```

TypeScript

```

function generateParenthesis(n: number): string[] {
    function dfs(l, r, t) {
        if (l > n || r > n || l < r) {
            return;
        }
        if (l == n && r == n) {
            ans.push(t);
            return;
        }
        dfs(l + 1, r, t + "(");
        dfs(l, r + 1, t + ")");
    }

    let ans = [];
    dfs(0, 0, "");
    return ans;
}

```

Go

Go

```

func generateParenthesis(n int) []string {
    var dfs func(int, int, string)
    dfs = func(l, r int, t string) {
        if l > n || r > n || l < r {
            return
        }
        if l == n && r == n {
            ans = append(ans, t)
            return
        }
        dfs(l+1, r, t+"(")
        dfs(l, r+1, t+")")
    }
    dfs(0, 0, "")
    return ans
}

```

Go

Backtracking · LeetCode · By Pattern

— 489 —

LeetCode

```

func generateParenthesis(n int) ([]string) {
    var dfs func(int, int, string)
    dfs = func(l, r int, s string) {
        if l > n || r > n || l < r {
            return
        }
        if l == n && r == n {
            ans = append(ans, s)
            return
        }
        dfs(l+1, r, s+"(")
        dfs(l, r+1, s+")")
    }
    dfs(0, 0, "")
    return ans
}

```

Rust

Rust

```

impl Solution {
    pub fn generate_parenthesis(n: i32) -> Vec<String> {
        let mut ans = Vec::new();

        fn dfs(ans: &mut Vec<String>, l: i32, r: i32, t: String, n: i32) {
            if l > n || r > n || l < r {
                return;
            }
            if l == n && r == n {
                ans.push(t);
                return;
            }
            dfs(ans, l + 1, r, format!("{}", t, "("), n);
            dfs(ans, l, r + 1, format!("{}", t, ")"), n);
        }

        dfs(&mut ans, 0, 0, String::new(), n);
        ans
    }
}

```

Rust

Backtracking · LeetCode · By Pattern

— 490 —

LeetCode

```

impl Solution {
    fn dfs(left: i32, right: i32, s: &mut String, res: &mut Vec<String>) {
        if left == 0 && right == 0 {
            res.push(s.clone());
            return;
        }
        if left > 0 {
            s.push("(");
            Self::dfs(left - 1, right, s, res);
            s.pop();
        }
        if right > left {
            s.push(")");
            Self::dfs(left, right - 1, s, res);
            s.pop();
        }
    }

    pub fn generate_parenthesis(n: i32) -> Vec<String> {
        let mut res = Vec::new();
        Self::dfs(n, &mut String::new(), &mut res);
        res
    }
}

```

[1] Backtracking — Pattern Solution (matched from community answers)

Python

```

def generateParenthesis(n):
    # list to store valid combinations
    result = []

    # helper function for backtracking
    def backtrack(current, open_count, close_count):
        # if current string reaches the maxium length, it's a valid combination
        if len(current) == 2 * n:
            result.append(current)
            return

        # if we use and no opening parenthesis, do it
        if open_count < n:
            backtrack(current + '(', open_count + 1, close_count)

        # if adding a closing parenthesis maintains the validity, do it
        if close_count < open_count:
            backtrack(current + ')', open_count, close_count + 1)

    # Start the recursion with an empty string and zero counts for both
    backtrack("", 0, 0)
    return result

# Create output
print(generateParenthesis(3))

```

#39

MEDIUM

Combination Sum

LeetCode · Simplex · WalkCC · LeetCode · Editorial

Backtracking · LeetCode · By Pattern

— 491 —

LeetCode

Array · Backtracking

LeetCode 169

Problem:

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]
Explanation:
2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple times.
7 is a candidate, and 7 = 7.
These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8
Output: [[2,2,2,2],[2,3,3],[3,5]]

Example 3:

Input: candidates = [2], target = 1
Output: []

Constraints:

- 1 <= candidates.length <= 30
- 2 <= candidates[i] <= 40
- All elements of candidates are distinct.
- 1 <= target <= 40

>> Brute Force (Backtracking) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def combinationSum(self, candidates, target):
        # Brute Force O(n^target) - recursion without pruning
        result = []

        def backtrack(start, path, remain):
            if remain == 0:
                result.append(list(path)); return
            if remain < 0: return
            for i in range(start, len(candidates)):
                path.append(candidates[i])
                backtrack(i, path, remain - candidates[i])
                path.pop()

            backtrack(i+1, target)
            return result

```

Backtracking · LeetCode · By Pattern

— 492 —

LeetCode

Python
<pre> Python # Function to find all unique combinations that sum to the target def combinationSum(candidates, target): # List to store all valid combinations result = [] # Sort candidates to facilitate pruning of the search candidates.sort() # Recursive helper function for backtracking def backtrack(remaining, start, path): # If remaining is 0, we've found a valid combination to add a copy of it to result if remaining == 0: result.append(list(path)) return # Loop through candidates starting from the current index for i in range(start, len(candidates)): # If candidate is greater than remaining, break out for efficiency if candidates[i] > remaining: break # Add the current candidate to the current combination path path.append(candidates[i]) # Recursively call backtrack with updated remaining and same index (allowing reuse) backtrack(remaining - candidates[i], i, path) # Backtrack by removing the candidate added in this iteration path.pop() # Start backtracking from index 0 with an empty path backtrack(target, 0, []) return result # Example usage print(combinationSum([2,3,4,7,7], 7)) </pre>
<pre> Python class Solution: def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]: ans = [] def dfs(i: int, target: int, path: List[int]) -> None: if target <= 0: return if target == 0: ans.append(path.copy()) return for i in range(i, len(candidates)): path.append(candidates[i]) dfs(i, target - candidates[i], path) path.pop() candidates.sort() dfs(0, target, []) return ans </pre>
Python
Backtracking · LeetCode · By Pattern — 493 — LeetCode

Python
<pre> class Solution: def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]: def dfs(i: int, s: int): if s == 0: ans.append(i) return if s < candidates[i]: return for j in range(i, len(candidates)): t.append(candidates[j]) dfs(i, s - candidates[j]) t.pop() candidates.sort() s = 0 ans = [] dfs(0, target) return ans </pre>
Python
<pre> class Solution: def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]: # 1. Start index for this recursion # 2. ans def dfs(i, s): if s == target: ans.append(t.copy()) return if s > target: return for j in range(i, len(candidates)): c = candidates[j] t.append(c) dfs(i, s + c) t.pop() ans = [] t = [] # candidates.sort() # diff from combination-II, no need sorting it dfs(0, 0) return ans ##### # do version </pre>
Python
Backtracking · LeetCode · By Pattern — 494 — LeetCode

Python
<pre> class Solution: def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]: # For each index from 1 to target, its res[i] # = all 1-to-i array of [] dp = [] candidates.sort() for i in range(1, target+1): cur = [] for j in range(len(candidates)): if candidates[j] > i: break if candidates[j] == i: one = [candidates[j]] cur.append(one) break for k in range(candidates[j], i): continue deepCopied = a.copy() deepCopied.insert(0, candidates[j]) dp.append(deepCopied) dp.append(cur) return dp[-1] ##### class Solution(object): def combinationSum(self, candidates, target): #type candidates: List[int] #type target: int #rtype: List[List[int]] def dfs(candidates, start, target, path, res): if target == 0: return res.append(path + []) for i in range(start, len(candidates)): if target - candidates[i] <= 0: path.append(candidates[i]) dfs(candidates, i, target - candidates[i], path, res) path.pop() res = [] dfs(candidates, 0, target, [], res) return res ##### class Solution: def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]: </pre>
Python
Backtracking · LeetCode · By Pattern — 495 — LeetCode

C++
<pre> C++ class Solution { public: vector<vector<int>> combinationSum(vector<int>& candidates, int target) { vector<vector<int>> ans; ranges::sort(candidates); dfs(candidates, 0, target, {}, ans); return ans; } private: void dfs(const vector<int>& candidates, int s, int target, vector<int>& path, vector<vector<int>>& ans) { if (target <= 0) return; if (target == 0) { ans.push_back(path); return; } for (int i = s; i < candidates.size(); ++i) { path.push_back(candidates[i]); dfs(candidates, i, target - candidates[i], std::move(path), ans); path.pop_back(); } } }; </pre>
C++
<pre> C++ class Solution { public: vector<vector<int>> combinationSum(vector<int>& candidates, int target) { sort(candidates.begin(), candidates.end()); vector<vector<int>> ans; vector<int> t; function<void(int, int)> dfs = [&](int i, int s) { if (s == 0) { ans.emplace_back(t); return; } if (s < candidates[i]) { return; } for (int j = i; j < candidates.size(); ++j) { t.push_back(candidates[j]); dfs(j, candidates[j]); t.pop_back(); } }; dfs(0, target); return ans; } }; </pre>
C++
Backtracking · LeetCode · By Pattern — 496 — LeetCode

C++
<pre> class Solution { // // @param Integer[] candidates // @param Integer target // @return Integer[][] // function combinationSum(candidates, target) { result = []; currentCombination = []; startIndex = 0; sort(candidates); this.findCombinations(candidates, target, startIndex, currentCombination, result); return result; } function findCombinations(candidates, target, startIndex, currentCombination, result) { if (target == 0) { result.push(currentCombination); return; } for (let i = startIndex; i <= count(candidates); i++) { sum = candidates[i]; if (sum > target) { break; } currentCombination.push(sum); this.findCombinations(candidates, target - sum, i, currentCombination, result); array.pop(currentCombination); } } </pre>
C++
Backtracking · LeetCode · By Pattern — 497 — LeetCode

C++
<pre> class Solution { public: vector<vector<int>> combinationSum(vector<int>& candidates, int target) { sort(candidates.begin(), candidates.end()); vector<vector<int>> ans; vector<int> t; function<void(int, int)> dfs = [&](int i, int s) { if (s == 0) { ans.emplace_back(t); return; } if (i == candidates.size() s < candidates[i]) { return; } dfs(i + 1, s); t.push_back(candidates[i]); dfs(i, candidates[i]); t.pop_back(); }; dfs(0, target); return ans; } }; </pre>
C++
<pre> class Solution { // // @param Integer[] candidates // @param Integer target // @return Integer[][] // function combinationSum(candidates, target) { result = []; currentCombination = []; startIndex = 0; sort(candidates); this.findCombinations(candidates, target, startIndex, currentCombination, result); return result; } function findCombinations(candidates, target, startIndex, currentCombination, result) { if (target == 0) { result.push(currentCombination); return; } for (let i = startIndex; i <= count(candidates); i++) { sum = candidates[i]; </pre>
C++
Backtracking · LeetCode · By Pattern — 498 — LeetCode

```

        if (sum == target) {
            break;
        }
        currentCombination[] = sum;
    }
    this->findCombinations(candidates, target - sum, sl, currentCombination, result);
    array.pop(currentCombination);
}
}

```

Java

```

class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> ans = new ArrayList<>();
        Arrays.sort(candidates);
        dfs(candidates, 0, target, new ArrayList<>(), ans);
        return ans;
    }

    private void dfs(int[] candidates, int s, int target, List<Integer> path,
        List<List<Integer>> ans) {
        if (target < 0)
            return;
        if (target == 0) {
            ans.add(new ArrayList<>(path));
            return;
        }
        for (int i = s; i < candidates.length; ++i) {
            path.add(candidates[i]);
            dfs(candidates, i, target - candidates[i], path, ans);
            path.remove(path.size() - 1);
        }
    }
}

```

Java

Backtracking · LeetCode · By Pattern

— 499 —

LeetCode

```

class Solution {
    private List<List<Integer>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();
    private int[] candidates;

    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        this.candidates = candidates;
        dfs(0, target);
        return ans;
    }

    private void dfs(int i, int s) {
        if (s == 0) {
            ans.add(new ArrayList<>(t));
            return;
        }
        if (s < candidates[i]) {
            return;
        }
        for (int j = i; j < candidates.length; ++j) {
            t.add(candidates[j]);
            dfs(j, s - candidates[j]);
            t.remove(t.size() - 1);
        }
    }
}

```

Java

```

public class Solution {
    private List<List<Integer>> ans = new List<List<Integer>>();
    private List<Integer> t = new List<Integer>();
    private int[] candidates;

    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        this.candidates = candidates;
        dfs(0, target);
        return ans;
    }

    private void dfs(int i, int s) {
        if (s == 0) {
            ans.add(new List<Integer>(t));
            return;
        }
        if (s < candidates[i]) {
            return;
        }
        for (int j = i; j < candidates.length; ++j) {
            t.add(candidates[j]);
            dfs(j, s - candidates[j]);
            t.remove(t.Count - 1);
        }
    }
}

```

Backtracking · LeetCode · By Pattern

— 500 —

LeetCode

```

Java
class Solution {
    private List<List<Integer>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();
    private int[] candidates;

    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        this.candidates = candidates;
        dfs(0, target);
        return ans;
    }

    private void dfs(int i, int s) {
        if (s == 0) {
            ans.add(new ArrayList<>(t));
            return;
        }
        if (i == candidates.length || s < candidates[i]) {
            return;
        }
        dfs(i + 1, s);
        t.add(candidates[i]);
        dfs(i, s - candidates[i]);
        t.remove(t.size() - 1);
    }
}

/////
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        // recursive target from 0 to target, its dp[i][j] == so 3-D array dp[i][j][k]
        List<List<List<Integer>>> dp = new ArrayList<>();
        Arrays.sort(candidates);

        for (int i = 1; i <= target; ++i) {
            List<List<Integer>> cur = new ArrayList<>();
            for (int j = 0; j < candidates.length; ++j) {
                if (candidates[j] > i) break;
                if (candidates[j] == 0) {
                    ArrayList<Integer> ans = new ArrayList<Integer>();
                    ans.add(candidates[j]);
                    cur.add(ans); // ans: ans with proper <Integer>, or else unexpected operation error
                    break;
                }
                for (List<Integer> a : dp.get(i - candidates[j] - 1)) {
                    if (candidates[j] > a.get(0)) {
                        continue;
                    }
                }
            }
        }
    }
}

```

Backtracking · LeetCode · By Pattern

— 501 —

LeetCode

```

ArrayList<Integer> deepCopied = new ArrayList<>(a); // quote: must have
deepCopied.add(0, candidates[i]); // quote: target at index0 for the array
cur.add(deepCopied);
}
}
dp.add(cur);
}
return dp.get(dp.size() - 1);
}
}

```

Java

```

public class Solution {
    private List<List<Integer>> ans = new List<List<Integer>>();
    private List<Integer> t = new List<Integer>();
    private int[] candidates;

    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);
        this.candidates = candidates;
        dfs(0, target);
        return ans;
    }

    private void dfs(int i, int s) {
        if (s == 0) {
            ans.add(new List<Integer>(t));
            return;
        }
        if (i == candidates.length || s < candidates[i]) {
            return;
        }
        dfs(i + 1, s);
        t.add(candidates[i]);
        dfs(i, s - candidates[i]);
        t.remove(t.Count - 1);
    }
}

```

TypeScript
TypeScript

Backtracking · LeetCode · By Pattern

— 502 —

LeetCode

```

function combinationSum(candidates: number[], target: number): number[][] {
    candidates.sort((a, b) => a - b);
    const ans: number[][] = [];
    const t: number[] = [];
    const dfs = (i: number, s: number) => {
        if (s == 0) {
            ans.push(t.slice());
            return;
        }
        if (s < candidates[i]) {
            return;
        }
        for (let j = i; j < candidates.length; ++j) {
            t.push(candidates[j]);
            dfs(j, s - candidates[j]);
            t.pop();
        }
    };
    dfs(0, target);
    return ans;
}

```

TypeScript

```

function combinationSum(candidates: number[], target: number): number[][] {
    candidates.sort((a, b) => a - b);
    const ans: number[][] = [];
    const t: number[] = [];
    const dfs = (i: number, s: number) => {
        if (s == 0) {
            ans.push(t.slice());
            return;
        }
        if (i == candidates.length || s < candidates[i]) {
            return;
        }
        dfs(i + 1, s);
        t.push(candidates[i]);
        dfs(i, s - candidates[i]);
        t.pop();
    };
    dfs(0, target);
    return ans;
}

```

Go

Go

Backtracking · LeetCode · By Pattern

— 503 —

LeetCode

```

func combinationSum(candidates []int, target int) ans [][]int {
    sort.Ints(candidates)
    t := []int{}
    var dfs func(i, s int)
    dfs = func(i, s int) {
        if s == 0 {
            ans = append(ans, t.Clone())
            return
        }
        if s < candidates[i] {
            return
        }
        for j := i; j < len(candidates); j++ {
            t = append(t, candidates[j])
            dfs(j, s - candidates[j])
            t = t[:len(t)-1]
        }
    }
    dfs(0, target)
    return
}

```

Go

```

func combinationSum(candidates []int, target int) ans [][]int {
    sort.Ints(candidates)
    t := []int{}
    var dfs func(i, s int)
    dfs = func(i, s int) {
        if s == 0 {
            ans = append(ans, t.Clone())
            return
        }
        if i == len(candidates) || s < candidates[i] {
            return
        }
        dfs(i+1, s)
        t = append(t, candidates[i])
        dfs(i, s - candidates[i])
        t = t[:len(t)-1]
    }
    dfs(0, target)
    return
}

```

Rust

Rust

Backtracking · LeetCode · By Pattern

— 504 —

LeetCode


```

def dfs(i, used, s, k, candidates, ksum, t, ksum, ans):
    if s == k:
        ans.append(s)
        return
    if s < k:
        for i in range(1, len(candidates)):
            t.push(candidates[i])
            Self.dfs(i+1, used, s+candidates[i], k, candidates, t, ksum, ans)
            t.pop()
    return

def combinationSum(candidates, target):
    candidates.sort()
    ans = []
    Self.dfs(0, False, 0, 0, candidates, [], 0, ans)
    return ans

```

Back

```

def dfs(i, used, s, k, candidates, ksum, t, ksum, ans):
    if s == k:
        ans.append(s)
        return
    if s < k:
        for i in range(1, len(candidates)):
            t.push(candidates[i])
            Self.dfs(i+1, used, s+candidates[i], k, candidates, t, ksum, ans)
            t.pop()
    return

def combinationSum(candidates, target):
    candidates.sort()
    ans = []
    Self.dfs(0, False, 0, 0, candidates, [], 0, ans)
    return ans

```

[7] Backtracking — Pattern Solution (matched from community answers)

Python

Backtracking · LeetCode · By Pattern

— 505 —

LeetCode

```

def combinationSum(candidates, target):
    # List to store all valid combinations
    result = []
    # Sort candidates to facilitate pruning of the search
    candidates.sort()

    # Recursive helper function for backtracking
    def backtrack(remaining, start, path):
        # If remaining is 0, path is a valid combination so add a copy of it to result
        if remaining == 0:
            result.append(path)
            return
        # Loop through candidates starting from the current index
        for i in range(start, len(candidates)):
            # If candidate is greater than remaining, break out for efficiency
            if candidates[i] > remaining:
                break
            # Include candidate in the current combination path
            path.append(candidates[i])
            # Recursively call backtrack with updated remaining and same index (allowing reuse)
            backtrack(remaining - candidates[i], i, path)
            # Backtrack by removing the candidate added in this iteration
            path.pop()

    # Start backtracking from index 0 with an empty path
    backtrack(target, 0, [])
    return result

```

Example usage

print(combinationSum([2,3,4,7], 7))

#46

MEDIUM

Permutations

LeetCode · Simplilearn · WUOLCC · LeetCode · Editorial

Array · Backtracking

LeetCode 109

Problem:

Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Example 2:

Input: nums = [0,1]
Output: [[0,1],[1,0]]

Example 3:

Input: nums = [1]
Output: [[1]]

Backtracking · LeetCode · By Pattern

— 506 —

LeetCode

Constraints:

- 1 <= nums.length <= 6
- 10 <= nums[i] <= 10
- All the integers of nums are unique.

>> Brute Force [Backtracking] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def permute(self, nums):
        # Create a list to store all permutations (or full recursion)
        from itertools import permutations as perms
        return [list(p) for p in perms(nums)]

# Python
def permute(nums):
    result = []

    def backtrack(current, remaining):
        # If no remaining elements, current permutation is complete.
        if not remaining:
            result.append(current.copy())
            return
        # Explore each candidate number.
        for i in range(len(remaining)):
            # Add the element to the current permutation.
            current.append(remaining[i])
            # Recursively generate permutations with the remaining numbers.
            backtrack(current, remaining[i+1:])
            # Backtrack by removing the chosen number.
            current.pop()

    backtrack([], nums)
    return result

# Example usage:
print(permute([1,2,3]))

```

Python

Backtracking · LeetCode · By Pattern

— 507 —

LeetCode

```

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: List[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

            for i, num in enumerate(nums):
                if used[i]:
                    continue
                used[i] = True
                path.append(nums[i])
                dfs(path)
                path.pop()
                used[i] = False

        dfs([])
        return ans

```

Python

```

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == n:
                ans.append(i)
                return
            for j, num in enumerate(nums):
                if not vis[j]:
                    vis[j] = True
                    i += 1
                    dfs(i)
                    vis[j] = False

        n = len(nums)
        vis = [False] * n
        i = 0
        ans = []
        dfs(i)
        return ans

```

Python

Backtracking · LeetCode · By Pattern

— 508 —

LeetCode

```

...
remove an element by its value from a set

my_set = {1, 2, 3, 4, 5}
my_set.remove(1)
print(my_set)
{2, 3, 4, 5}

...

cannot remove an element by index from a set in Python3
need to convert the set to a list

my_set = {1, 2, 3, 4, 5}
my_list = list(my_set)
del my_list[2] # remove the element at index 2
my_set = set(my_list)
print(my_set)
{1, 2, 4, 5}

...

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = []

        if nums is None or len(nums) == 0:
            return res

        for num in nums:
            new_res = []
            for para in res:
                for i in range(len(para) + 1):
                    new_para = para[:i] + [num] + para[i:]
                    new_res.append(new_para)

            res = new_res

        return res

...

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        ans = []

        if nums is None or len(nums) == 0:
            return ans

        for num in nums:
            top_list = []

```

Backtracking · LeetCode · By Pattern

— 509 —

LeetCode

```

for single_perm in ans:
    for index in range(len(single_perm) + 1):
        single_perm.insert(index, num)
        top_list.append(single_perm.copy())
        single_perm.pop(index)

    ans = top_list
    return ans

...

In Python 3, both the discard() and remove() methods of a set object are used to remove an element from the set, but there is one key difference:

+ discard() removes the specified element from the set if it is present, but does nothing if the element is not present.
+ remove() removes the specified element from the set if it is present, but raises a KeyError exception if the element is not present.

...

>>> s
{13, 66, 11, 44, 22, 55}
>>> s.discard(22)
>>> s
{13, 66, 11, 44, 55}

>>> s.discard(200)
>>> s
{13, 66, 11, 44, 55}
Traceback (most recent call last):

```

C++

C++

Backtracking · LeetCode · By Pattern

— 510 —

LeetCode

```

class Solution {
public:
    vector<vector<int>> permute(vector<int>& nums) {
        vector<vector<int>> ans;
        dfs(nums, vector<bool>(nums.size()), {1}, ans);
        return ans;
    }

private:
    void dfs(const vector<int>& nums, vector<bool>& used, vector<int>& ds, path,
            vector<vector<int>>& ans) {
        vector<vector<int>> ans;
        if (path.size() == nums.size()) {
            ans.push_back(path);
            return;
        }

        for (int i = 0; i < nums.size(); ++i) {
            if (used[i])
                continue;
            used[i] = true;
            path.push_back(nums[i]);
            dfs(nums, used, move(used), move(path), ans);
            path.pop_back();
            used[i] = false;
        }
    }
};

```

```

C++

class Solution {
public:
    vector<vector<int>> permute(vector<int>& nums) {
        int n = nums.size();
        vector<vector<int>> ans;
        vector<int> t(n);
        vector<bool> vis(n);
        auto dfs = [&](this method, dfs, int i) -> void {
            if (i == n) {
                ans.emplace_back(t);
                return;
            }
            for (int j = 0; j < n; ++j) {
                if (!vis[j]) {
                    vis[j] = true;
                    t[i] = nums[j];
                    dfs(i + 1);
                    vis[j] = false;
                }
            }
            dfs(i);
            return ans;
        };
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> permute(vector<int>& nums) {
        int n = nums.size();
        vector<vector<int>> ans;
        vector<int> t(n);
        vector<bool> vis(n);
        function<void(int)> dfs = [&](int i) {
            if (i == n) {
                ans.emplace_back(t);
                return;
            }
            for (int j = 0; j < n; ++j) {
                if (!vis[j]) {
                    vis[j] = true;
                    t[i] = nums[j];
                    dfs(i + 1);
                    vis[j] = false;
                }
            }
            dfs(i);
            return ans;
        };
    }
};

```

Java

```

class Solution {
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> ans = new ArrayList<>();
    dfs(nums, new boolean[nums.length], new ArrayList<>(), ans);
    return ans;
}

private void dfs(int[] nums, boolean[] used, List<Integer> path, List<List<Integer>> ans) {
    if (path.size() == nums.length) {
        ans.add(new ArrayList<>(path));
        return;
    }

    for (int i = 0; i < nums.length; ++i) {
        if (used[i])
            continue;
        used[i] = true;
        path.add(nums[i]);
        dfs(nums, used, path, ans);
        path.remove(path.size() - 1);
        used[i] = false;
    }
}
}

```

Java

```

class Solution {
private List<List<Integer>> ans = new ArrayList<>();
private List<Integer> t = new ArrayList<>();
private boolean[] vis;
private int[] nums;

public List<List<Integer>> permute(int[] nums) {
    this.nums = nums;
    vis = new boolean[nums.length];
    dfs(0);
    return ans;
}

private void dfs(int i) {
    if (i == nums.length) {
        ans.add(new ArrayList<>(t));
        return;
    }
    for (int j = 0; j < nums.length; ++j) {
        if (!vis[j]) {
            vis[j] = true;
            t.add(nums[j]);
            dfs(i + 1);
            t.remove(t.size() - 1);
            vis[j] = false;
        }
    }
}
}

```

Java

```

public class Solution {
    public List<List<Integer>> permute(int[] nums) {
        var ans = new List<List<Integer>>();
        var t = new List<Integer>();
        var vis = new bool[nums.length];
        dfs(nums, 0, t, vis, ans);
        return ans;
    }

    private void dfs(int[] nums, int i, List<Integer> t, bool[] vis, List<List<Integer>> ans) {
        if (i == nums.length) {
            ans.Add(new List<Integer>(t));
            return;
        }
        for (int j = 0; j < nums.length; ++j) {
            if (!vis[j]) {
                vis[j] = true;
                t.Add(nums[j]);
                dfs(nums, i + 1, t, vis, ans);
                t.Remove(t.Count - 1);
                vis[j] = false;
            }
        }
    }
}

```

Java

```

class Solution {
private List<List<Integer>> ans = new ArrayList<>();
private List<Integer> t = new ArrayList<>();
private boolean[] vis;
private int[] nums;

public List<List<Integer>> permute(int[] nums) {
    this.nums = nums;
    vis = new boolean[nums.length];
    dfs(0);
    return ans;
}

private void dfs(int i) {
    if (i == nums.length) {
        ans.add(new ArrayList<>(t));
        return;
    }
    for (int j = 0; j < nums.length; ++j) {
        if (!vis[j]) {
            vis[j] = true;
            t.add(nums[j]);
            dfs(i + 1);
            t.remove(t.size() - 1);
            vis[j] = false;
        }
    }
}
}

```

Java

```

public class Solution {
    public List<List<Integer>> permute(int[] nums) {
        var ans = new List<List<Integer>>();
        var t = new List<Integer>();
        var vis = new bool[nums.length];
        dfs(nums, 0, t, vis, ans);
        return ans;
    }

    private void dfs(int[] nums, int i, List<Integer> t, bool[] vis, List<List<Integer>> ans) {
        if (i == nums.length) {
            ans.Add(new List<Integer>(t));
            return;
        }
        for (int j = 0; j < nums.length; ++j) {
            if (!vis[j]) {
                vis[j] = true;
                t.Add(nums[j]);
                dfs(nums, i + 1, t, vis, ans);
                t.Remove(t.Count - 1);
                vis[j] = false;
            }
        }
    }
}

```

Java

JavaScript

JavaScript

```

/*
 * @param {number[]} nums
 * @return {number[][]}
 */
var permute = function (nums) {
    const n = nums.length;
    const ans = [];
    const vis = Array(n).fill(false);
    const t = Array(n).fill(0);
    const dfs = (i => {
        if (i == n) {
            ans.push(t.slice());
            return;
        }
        for (let j = 0; j < n; ++j) {
            if (!vis[j]) {
                vis[j] = true;
                t[i] = nums[j];
                dfs(i + 1);
                vis[j] = false;
            }
        }
    });
    dfs(0);
    return ans;
};

```

JavaScript

```

/*
 * @param {number[]} nums
 * @return {number[][]}
 */
var permute = function (nums) {
    const n = nums.length;
    const ans = [];
    const t = [];
    const vis = new Array(n).fill(false);
    function dfs(i) {
        if (i == n) {
            ans.push([...t]);
            return;
        }
        for (let j = 0; j < n; ++j) {
            if (!vis[j]) {
                vis[j] = true;
                t.push(nums[j]);
                dfs(i + 1);
                vis[j] = false;
                t.pop();
            }
        }
    }
    dfs(0);
};

```

return ans;

TypeScript

TypeScript

```

function permute(nums: number[]): number[][] {
    const n = nums.length;
    const res: number[][] = [];
    const dfs = (i: number) => {
        if (i == n) {
            res.push([...nums]);
        }
        for (let j = 0; j < n; ++j) {
            if (!vis[j]) {
                vis[j] = true;
                t[i] = nums[j];
                dfs(i + 1);
                vis[j] = false;
            }
        }
    };
    dfs(0);
    return res;
}

```

TypeScript

```

function permute(nums: number[]): number[][] {
    const n = nums.length;
    const res: number[][] = [];
    const dfs = (i: number) => {
        if (i == n) {
            res.push([...nums]);
        }
        for (let j = 0; j < n; ++j) {
            if (!vis[j]) {
                vis[j] = true;
                t[i] = nums[j];
                dfs(i + 1);
                vis[j] = false;
            }
        }
    };
    dfs(0);
    return res;
}

```

Go

Go

```

func permute(nums []int) (ans [][]int) {
    n := len(nums)
    t := make([]int, n)
    vis := make([]bool, n)
    var dfs func(int)
    dfs = func(i int) {
        if i == n {
            ans = append(ans, slices.Clone(t))
            return
        }
        for j, x := range nums {
            if vis[j] {
                continue
            }
            vis[j] = true
            t[i] = x
            dfs(i+1)
            vis[j] = false
        }
    }
    dfs(0)
    return
}

Go

func permute(nums []int) (ans [][]int) {
    n := len(nums)
    t := make([]int, n)
    vis := make([]bool, n)
    var dfs func(int)
    dfs = func(i int) {
        if i == n {
            ans = append(ans, slices.Clone(t))
            return
        }
        for j, v := range nums {
            if vis[j] {
                continue
            }
            vis[j] = true
            t[i] = v
            dfs(i+1)
            vis[j] = false
        }
    }
    dfs(0)
    return
}

Rust
Rust

```

Backtracking · LeetCode · By Pattern

— 517 —

LeetCode

```

impl Solution {
    pub fn permute(nums: Vec<i32>) -> Vec<Vec<i32>> {
        let n = nums.len();
        let mut ans = Vec::new();
        let mut t = vec![0; n];
        let mut vis = vec![false; n];
        fn dfs(
            nums: &Vec<i32>,
            n: usize,
            t: &mut Vec<i32>,
            vis: &mut Vec<bool>,
            ans: &mut Vec<Vec<i32>>,
            i: usize
        ) {
            if i == n {
                ans.push(t.clone());
                return;
            }
            for j in 0..n {
                if vis[j] {
                    continue
                }
                vis[j] = true;
                t[i] = nums[j];
                dfs(nums, n, t, vis, ans, i + 1);
                vis[j] = false;
            }
        }
        dfs(nums, n, &mut t, &mut vis, &mut ans, 0);
        ans
    }
}

Rust

impl Solution {
    fn dfs(i: usize, nums: &mut Vec<i32>, res: &mut Vec<Vec<i32>>) {
        let n = nums.len();
        if i == n {
            res.push(nums.clone());
            return;
        }
        for j in 0..n {
            if !vis[j] {
                vis[j] = true;
                Self::dfs(i + 1, nums, res);
                vis[j] = false;
            }
        }
    }

    pub fn permute(nums: Vec<i32>) -> Vec<Vec<i32>> {
        let mut res = Vec::new();
        Self::dfs(0, &mut nums, &mut res);
        res
    }
}

(*) Backtracking — Pattern Solution (matched from community answers)
Python

```

Backtracking · LeetCode · By Pattern

— 518 —

LeetCode

```

def permute(nums):
    result = []

    def backtrack(current, remaining):
        # If no remaining elements, current permutation is complete.
        if not remaining:
            result.append(current.copy())
            return
        # Explore each candidate number.
        for i in range(len(remaining)):
            # Add the chosen number to the current permutation.
            current.append(remaining[i])
            # Recursively generate permutations with the remaining numbers.
            backtrack(current, remaining[i+1:])
            # Backtrack by removing the chosen number.
            current.pop()

    backtrack([], nums)
    return result

# Example usage:
print(permute([1,2,3]))

```

#79

MEDIUM

Word Search

LeetCode · SimplyLeet · WalkCC · LeetCode · Editorial

Array · String · Backtracking · Matrix

Word Grid 129

Problem:

Given an $m \times n$ grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"

Output: true

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "SEE"

Output: true

Example 3:

Backtracking · LeetCode · By Pattern

— 519 —

LeetCode

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCC"

Output: false

Constraints:

- $m == \text{board.length}$
- $n == \text{board[0].length}$
- $1 \leq m, n \leq 6$
- $1 \leq \text{word.length} \leq 15$
- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

>> Brute Force (Backtracking) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        # Define the function to solve the Word Search problem.
        def dfs(r, c, index):
            # Base case: if index is equal to the length of the word, return True.
            if index == len(word):
                return True
            # Check boundaries and if current cell already used or not matching.
            if r < 0 or r > len(board) or c < 0 or c > len(board[0]) or board[r][c] != word[index]:
                return False
            # Temporarily mark the cell as visited.
            temp = board[r][c]
            board[r][c] = "*"
            # Explore all four directions.
            found = dfs(r+1, c, index+1) or dfs(r-1, c, index+1) or dfs(r, c+1, index+1) or dfs(r, c-1, index+1)
            # Backtrack: restore the original cell value.
            board[r][c] = temp
            return found
        # Iterate DFS from each cell in the grid.
        for i in range(len(board)):
            for j in range(len(board[0])):
                if dfs(i, j, 0):
                    return True
        return False

```

Backtracking · LeetCode · By Pattern

— 521 —

LeetCode

```

Python

# Define the function to solve the Word Search problem.
def exist(board, word):
    # Get dimensions of the board.
    rows, cols = len(board), len(board[0])

    # Helper function for DFS search.
    def dfs(r, c, index):
        # If the entire word is found, return True.
        if index == len(word):
            return True

        # Check boundaries and if current cell already used or not matching.
        if r < 0 or r > rows or c < 0 or c > cols or board[r][c] != word[index]:
            return False

        # Temporarily mark the cell as visited.
        temp = board[r][c]
        board[r][c] = "*"

        # Explore all four directions.
        found = dfs(r+1, c, index+1) or dfs(r-1, c, index+1) or dfs(r, c+1, index+1) or dfs(r, c-1, index+1)

        # Backtrack: restore the original cell value.
        board[r][c] = temp
        return found

    # Iterate DFS from each cell in the grid.
    for i in range(rows):
        for j in range(cols):
            if dfs(i, j, 0):
                return True
    return False

# Example test case
if __name__ == "__main__":
    board = [
        ["A", "B", "C", "E"],
        ["S", "F", "C", "S"],
        ["A", "D", "E", "E"]
    ]
    word = "ABCCED"
    print(exist(board, word)) # Expected output: True

```

Python

Backtracking · LeetCode · By Pattern

— 522 —

LeetCode

```

class Solution {
    def exist(self, board: List[List[str]], word: str) -> bool:
        m = len(board)
        n = len(board[0])

        def dfs(i: int, j: int, k: int) -> bool:
            if i < 0 or i == m or j < 0 or j == n:
                return False
            if board[i][j] != word[k] or board[i][j] == '*':
                return False
            if k == len(word) - 1:
                return True
            return False

        cache = board[i][j]
        board[i][j] = '*'
        if dfs(i, j, 0) == True:
            return True
        else:
            return False
        board[i][j] = cache
        return False
    def exist(self, board: List[List[str]], word: str) -> bool:
        m, n = len(board), len(board[0])
        for i in range(m):
            for j in range(n):

```

Python

Backtracking · LeastMemory · By Pattern

— 523 —

LeastMemory

```

class Solution {
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i, j, cur):
            if cur == len(word):
                return True
            if i < 0 or i == m or j < 0 or j == n or board[i][j] != word[cur]:
                return False
            t = board[i][j]
            board[i][j] = '#'
            for x, y in [(i-1, j), (i+1, j), (i, j-1), (i, j+1)]:
                if 0 <= x < m and 0 <= y < n and board[x][y] != '#':
                    if dfs(x, y, cur+1):
                        return True
            board[i][j] = t
            return False
        m, n = len(board), len(board[0])
        for i in range(m):
            for j in range(n):

```

C++

```

class Solution {
public:
    bool exist(vector<vector<char>>& board, string word) {
        int m = board.size(), n = board[0].size();
        int dirs[] = {-1, 0, 1, 0, -1};
        function<bool(int, int, int, int, int, int)> dfs = [&](int i, int j, int k) -> bool {
            if (k == word.size()) return true;
            if (board[i][j] != word[k]) return false;
            char c = board[i][j];
            board[i][j] = '#';
            for (int x = 0; x < 4; ++x) {
                int x1 = i + dirs[x], y1 = j + dirs[x + 1];
                if (x1 < 0 || x1 > m || y1 < 0 || y1 > n || board[x1][y1] != word[k+1]) continue;
                if (dfs(x1, y1, k+1)) return true;
            }
            board[i][j] = c;
            return false;
        };
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {

```

Backtracking · LeastMemory · By Pattern

— 524 —

LeastMemory

```

        if (dfs(i, j, 0)) {
            return true;
        }
    }
    return false;
};

Java
class Solution {
    private int m;
    private int n;
    private String word;
    private char[][] board;

    public boolean exist(char[][] board, String word) {
        m = board.length;
        n = board[0].length;
        this.word = word;
        this.board = board;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (dfs(i, j, 0)) {
                    return true;
                }
            }
        }
        return false;
    }

    private boolean dfs(int i, int j, int k) {
        if (k == word.length() - 1) {
            return board[i][j] == word.charAt(k);
        }
        if (board[i][j] != word.charAt(k)) {
            return false;
        }
        char c = board[i][j];
        board[i][j] = '#';
        int[] dirs = {-1, 0, 1, 0, -1};
        for (int x = 0; x < 4; ++x) {
            int x1 = i + dirs[x], y1 = j + dirs[x + 1];
            if (x1 < 0 || x1 > m || y1 < 0 || y1 > n || board[x1][y1] != word.charAt(k+1)) continue;
            if (dfs(x1, y1, k+1)) {
                return true;
            }
        }
        board[i][j] = c;
        return false;
    }
}
Java

```

Backtracking · LeastMemory · By Pattern

— 525 —

LeastMemory

```

public class Solution {
    private int m;
    private int n;
    private char[][] board;
    private String word;

    public bool Exist(char[][] board, string word) {
        m = board.Length;
        n = board[0].Length;
        this.board = board;
        this.word = word;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (dfs(i, j, 0)) {
                    return true;
                }
            }
        }
        return false;
    }

    private bool dfs(int i, int j, int k) {
        if (k == word.Length - 1) {
            return board[i][j] == word[k];
        }
        if (board[i][j] != word[k]) {
            return false;
        }
        char c = board[i][j];
        board[i][j] = '#';
        int[] dirs = {-1, 0, 1, 0, -1};
        for (int x = 0; x < 4; ++x) {
            int x1 = i + dirs[x], y1 = j + dirs[x + 1];
            if (x1 < 0 || x1 > m || y1 < 0 || y1 > n || board[x1][y1] != word[k+1]) continue;
            if (dfs(x1, y1, k+1)) {
                return true;
            }
        }
        board[i][j] = c;
        return false;
    }
}
TypeScript
TypeScript

```

Backtracking · LeastMemory · By Pattern

— 526 —

LeastMemory

```

function exist(board: string[][], word: string): boolean {
    const [m, n] = [board.length, board[0].length];
    const dirs = [-1, 0, 1, 0, -1];
    const dfs = (i: number, j: number, k: number): boolean => {
        if (k === word.length - 1) {
            return board[i][j] === word[k];
        }
        if (board[i][j] !== word[k]) {
            return false;
        }
        const c = board[i][j];
        board[i][j] = '#';
        for (let x = 0; x < 4; ++x) {
            const [x1, y1] = [i + dirs[x], j + dirs[x + 1]];
            const ok = 0 <= x1 < m && 0 <= y1 < n && board[x1][y1] !== '#';
            if (ok && board[x1][y1] === word[k + 1]) {
                if (dfs(x1, y1, k + 1)) {
                    return true;
                }
            }
        }
        board[i][j] = c;
        return false;
    };
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (dfs(i, j, 0)) {
                return true;
            }
        }
    }
    return false;
}
Rust
Rust

```

Backtracking · LeastMemory · By Pattern

— 527 —

LeastMemory

```

impl Solution {
    fn dfs(
        i: usize,
        j: usize,
        k: usize,
        word: &str,
        board: &Vec<Vec<char>>,
    ) -> bool {
        if (board[i][j] != word[k]) {
            return false;
        }
        if k == word.len() - 1 {
            return true;
        }
        let dirs = [
            (-1, 0),
            (0, -1),
            (0, 1),
            (1, 0),
        ];
        for (x, y) in dirs.iter() {
            let i1 = i + x;
            let j1 = j + y;
            if i1 < 0 || i1 >= board.len() || j1 < 0 || j1 >= board[0].len() {
                continue;
            }
            let (x1, y1) = (i1, j1);
            if !word.as_bytes().get(x1).is_some() {
                return false;
            }
            if word.as_bytes().get(x1) == word.as_bytes().get(y1) {
                return true;
            }
        }
        false
    }

    pub fn exist(board: Vec<Vec<char>>, word: String) -> bool {
        let m = board.len();
        let n = board[0].len();
        let word = word.as_bytes();
        let mut vis = vec![vec![false; n]; m];
        for i in 0..m {
            for j in 0..n {
                if Self::dfs(i, j, 0, word, board, vis) {
                    return true;
                }
            }
        }
        false
    }
}
Python

```

[*] Backtracking — Pattern Solution (stored optimal)

Python

Backtracking · LeastMemory · By Pattern

— 528 —

LeastMemory

```

# Define the function to solve the word search problem.
def exist(board, word):
    # Get dimensions of the board
    rows, cols = len(board), len(board[0])

    # Helper function for DFS search.
    def dfs(r, c, index):
        # If the entire word is found, return True.
        if index == len(word):
            return True

        # Check boundaries and if current cell already used or not matching.
        if r < 0 or c < 0 or r == rows or c == cols or board[r][c] != word[index]:
            return False

        # Temporarily mark the cell as visited.
        temp = board[r][c]
        board[r][c] = "*"

        # Explore all four directions.
        found = (dfs(r + 1, c, index + 1) or
                 dfs(r - 1, c, index + 1) or
                 dfs(r, c + 1, index + 1) or
                 dfs(r, c - 1, index + 1))

        # Backtrack, restore the original cell value.
        board[r][c] = temp
        return found

    # Initialize DFS from each cell in the grid.
    for i in range(rows):
        for j in range(cols):
            if dfs(i, j, 0):
                return True
    return False

# Example test case
if __name__ == "__main__":
    board = [
        ["A", "B", "C", "E"],
        ["T", "S", "C", "A"],
        ["A", "B", "E", "E"],
    ]
    word = "ABCCED"
    print(exist(board, word)) # Expected output: True

```

C++

```

// C++ implementation of the Word Search problem.
#include <vector>
#include <string>
using namespace std;

class Solution {
public:
    // Main function to check if the word exists in the board.
    bool exist(vector<vector<char>>& board, string word) {
        int rows = board.size();
        if (rows == 0) return false;
        int cols = board[0].size();

        // Start DFS from each cell.
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (dfs(board, word, i, j, 0))
                    return true;
            }
        }
        return false;
    }

private:
    // DFS helper function.
    bool dfs(vector<vector<char>>& board, const string& word, int r, int c, int index) {
        // If the remaining word is found, return true.
        if (index == word.size()) return true;
        // If out of bounds or already visited, return false.
        if (r < 0 || c < 0 || r == board.size() || c == board[0].size() || board[r][c] != word[index])
            return false;
        // Temporarily mark this cell as visited.
        char temp = board[r][c];
        board[r][c] = '#';

        // Explore all four directions.
        bool found = dfs(board, word, r + 1, c, index + 1) ||
                    dfs(board, word, r - 1, c, index + 1) ||
                    dfs(board, word, r, c + 1, index + 1) ||
                    dfs(board, word, r, c - 1, index + 1);

        // Backtrack, restore the current cell.
        board[r][c] = temp;
        return found;
    }
};

// Example usage:
// int main() {

```

```

// vector<vector<char>> board = {
//     {"A", "B", "C", "E"},
//     {"T", "S", "C", "A"},
//     {"A", "B", "E", "E"},
// };
// string word = "ABCCED";
// Solution sol;
// bool result = sol.exist(board, word); // Expected: true
// return 0;
// }

```

#113

MEDIUM

Path Sum II

LeetCode · Simplify · WalkCC · LeetCode · Editorial

Backtracking · Tree · DFS

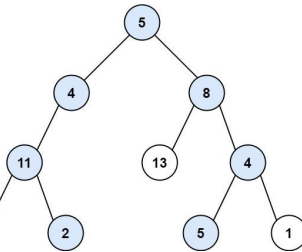
Units Grind 189

Problem:

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

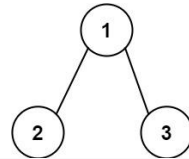
A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: [[5,4,11,2],[5,8,4,5]]
Explanation: There are two paths whose sum equals targetSum:
5 + 4 + 11 + 2 = 22
5 + 8 + 4 + 5 = 22

Example 2:



Input: root = [1,2,3], targetSum = 5
Output: []
Example 3:
Input: root = [1,2], targetSum = 0
Output: []
Constraints:
• The number of nodes in the tree is in the range [0, 5000].
• -1000 <= Node.val <= 1000
• -1000 <= targetSum <= 1000

```

>> Brute Force [Backtracking] Interview Prep

Python
# Brute Force Approach
class Solution:
    def pathSum(self, root, targetSum):
        # Base case - return empty list if root is None
        results = []
        def backtrack(start, path):
            results.append(list(path))
            for i in range(start, len(root)):
                path.append(root[i])
                backtrack(i + 1, path)
                path.pop()
        backtrack(0, [])
        return results

Python

```

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def pathSum(root, targetSum):
    result = [] # This will store all valid paths

    def dfs(node, current_path, remaining_sum):
        # If not node:
        if not node:
            return # Base case: reached the end of a path

        # Add the current node's value to the path
        current_path.append(node.val)

        # Check if it's a leaf and if the path sum equals targetSum
        if not node.left and not node.right and node.val == remaining_sum:
            result.append(list(current_path)) # Add a copy of the current path
        else:
            # Continue the search to the left and right subtrees
            dfs(node.left, current_path, remaining_sum - node.val)
            dfs(node.right, current_path, remaining_sum - node.val)

        # Backtrack, remove the last element before returning to the caller
        current_path.pop()

    dfs(root, [], targetSum)
    return result

Python
class Solution:
    def pathSum(self, root: TreeNode, sum: int) -> List[List[int]]:
        ans = []

        def dfs(root: TreeNode, sum: int, path: List[int]) -> None:
            if not root:
                return
            if root.val == sum and not root.left and not root.right:
                ans.append(path + [root.val])
                return

            dfs(root.left, sum - root.val, path + [root.val])
            dfs(root.right, sum - root.val, path + [root.val])

        dfs(root, sum, [])
        return ans

```

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> List[List[int]]:
        def dfs(root, s):
            if root is None:
                return
            s += root.val
            if root.left is None and root.right is None and s == targetSum:
                ans.append(list(path))
            dfs(root.left, s)
            dfs(root.right, s)
            path.pop()

        ans = []
        path = []
        dfs(root, 0)
        return ans

Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> List[List[int]]:
        def dfs(root, s):
            if root is None:
                return
            s += root.val
            if root.left is None and root.right is None and s == targetSum:
                ans.append(list(path))
            dfs(root.left, s)
            dfs(root.right, s)
            path.pop()

        ans = []
        path = []
        dfs(root, 0)
        return ans

C++
C++

```

```

class Solution {
public:
    vector<vector<int>> pathSum(TreeNode* root, int sum) {
        vector<vector<int>> ans;
        dfs(root, sum, {}, ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, int sum, vector<int>& path,
              vector<vector<int>>& ans) {
        if (root == nullptr)
            return;
        if (root->val == sum && root->left == nullptr && root->right == nullptr) {
            path.push_back(root->val);
            ans.push_back(path);
            path.pop_back();
            return;
        }

        path.push_back(root->val);
        dfs(root->left, sum - root->val, std::move(path), ans);
        dfs(root->right, sum - root->val, std::move(path), ans);
        path.pop_back();
    }
};

```

```

C++
//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };
//
class Solution {
public:
    vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
        vector<vector<int>> ans;
        vector<int> t;
        function<void(TreeNode*, int)> dfs = [&](TreeNode* root, int s) {
            if (!root) return;
            s += root->val;
            t.emplace_back(root->val);
            if (root->left && root->right && s == 0) ans.emplace_back(t);
            dfs(root->left, s);
            dfs(root->right, s);
            t.pop_back();
        };
    }
};

```

```

        dfs(root, targetSum);
        return ans;
    }
};

C++
//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };
//
class Solution {
public:
    vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
        vector<vector<int>> ans;
        vector<int> t;
        function<void(TreeNode*, int)> dfs = [&](TreeNode* root, int s) {
            if (!root) return;
            s += root->val;
            t.emplace_back(root->val);
            if (root->left && root->right && s == 0) ans.emplace_back(t);
            dfs(root->left, s);
            dfs(root->right, s);
            t.pop_back();
        };
    }

    dfs(root, targetSum);
    return ans;
};

```

```

Java
Java

```

```

class Solution {
public:
    List<List<int>> pathSum(TreeNode* root, int sum) {
        List<List<int>> ans = new ArrayList<>();
        dfs(root, sum, new ArrayList<>(), ans);
        return ans;
    }

private:
    void dfs(TreeNode* root, int sum, List<int> path, List<List<int>> ans) {
        if (root == null)
            return;
        if (root->val == sum && root->left == null && root->right == null) {
            path.add(root->val);
            ans.add(new ArrayList<>(path));
            path.remove(path.size() - 1);
            return;
        }

        path.add(root->val);
        dfs(root->left, sum - root->val, path, ans);
        dfs(root->right, sum - root->val, path, ans);
        path.remove(path.size() - 1);
    }
}

```

```

Java
//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
//
class Solution {
private:
    List<List<int>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();

    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        dfs(root, targetSum);
        return ans;
    }

private:
    void dfs(TreeNode root, int s) {

```

```

        if (root == null) {
            return;
        }
        s += root->val;
        t.add(root->val);
        if (root->left == null && root->right == null && s == 0) {
            ans.add(new ArrayList<>(t));
        }
        dfs(root->left, s);
        dfs(root->right, s);
        t.remove(t.size() - 1);
    }
}

Java
//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
//
class Solution {
private:
    List<List<Integer>> ans = new ArrayList<>();
    private List<Integer> t = new ArrayList<>();

    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        dfs(root, targetSum);
        return ans;
    }

private:
    void dfs(TreeNode root, int s) {
        if (root == null) {
            return;
        }
        s += root->val;
        t.add(root->val);
        if (root->left == null && root->right == null && s == 0) {
            ans.add(new ArrayList<>(t));
        }
        dfs(root->left, s);
        dfs(root->right, s);
        t.remove(t.size() - 1);
    }
}

JavaScript
JavaScript

```

```

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @param {number} targetSum
// @return {number[][]}
//
var pathSum = function (root, targetSum) {
    const ans = [];
    const t = [];
    function dfs(root, s) {
        if (!root) return;
        s += root.val;
        t.push(root.val);
        if (root.left && root.right && s == 0) ans.push([...t]);
        dfs(root.left, s);
        dfs(root.right, s);
        t.pop();
    }
}

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @param {number} targetSum
// @return {number[][]}
//
var pathSum = function (root, targetSum) {
    const ans = [];
    const t = [];
    function dfs(root, s) {
        if (!root) return;
        s += root.val;
        t.push(root.val);
        if (root.left && root.right && s == 0) ans.push([...t]);
        dfs(root.left, s);
        dfs(root.right, s);
        t.pop();
    }
}

```

```

        dfs(root, targetSum);
        return ans;
    }
};

Go
Go
//
// Definition for a binary tree node.
// type TreeNode struct {
//     Val int
//     Left *TreeNode
//     Right *TreeNode
// }
//
func pathSum(root *TreeNode, targetSum int) [][]int {
    t := []int{}
    var dfs func(*TreeNode, int)
    dfs = func(root *TreeNode, s int) {
        if root == nil {
            return
        }
        s += root.Val
        t = append(t, root.Val)
        if root.Left == nil && root.Right == nil && s == 0 {
            ans = append(ans, cloneSlice(t))
        }
        dfs(root.Left, s)
        dfs(root.Right, s)
        t = t[:len(t)-1]
    }
    dfs(root, targetSum)
    return ans
}

```

```

Go

```



```
using pii = pair<int, int>;

class Solution {
public:
    void solveSudoku(vector<vector<char>>& board) {
        bool row[9][9] = {false};
        bool col[9][9] = {false};
        bool block[3][3][9] = {false};
        bool ok = false;
        vector<pii> to;
        for (int i = 0; i < 9; ++i)
            for (int j = 0; j < 9; ++j) {
                if (board[i][j] == '.') {
                    to.push_back(i, j);
                } else {
                    int v = board[i][j] - '1';
                    row[i][v] = col[j][v] = block[i / 3][j / 3][v] = true;
                }
            }
        function<void(int k)> dfs = [&](int k) {
            if (k == to.size()) {
                ok = true;
                return;
            }
            int i = to[k].first, j = to[k].second;
            for (int v = 0; v < 9; ++v) {
                if (!row[i][v] && !col[j][v] && !block[i / 3][j / 3][v]) {
                    row[i][v] = col[j][v] = block[i / 3][j / 3][v] = true;
                    board[i][j] = v + '1';
                    dfs(k + 1);
                    row[i][v] = col[j][v] = block[i / 3][j / 3][v] = false;
                }
            }
            if (ok) {
                return;
            }
        };
        dfs(0);
    }
};
```

Java

Java

Backtracking · LeetCode · By Pattern

— 547 —

LeetCode

```
class Solution {
private:
    bool ok;
    private: char[10] board;
    private: List<int> t = new ArrayList<>();
    private: boolean[10][10] row = new boolean[10][10];
    private: boolean[10][10] col = new boolean[10][10];
    private: boolean[10][10][10] block = new boolean[10][10][10];

    public void solveSudoku(char[10][10] board) {
        this.board = board;
        for (int i = 0; i < 9; ++i)
            for (int j = 0; j < 9; ++j) {
                if (board[i][j] == '.') {
                    t.add(i * 9 + j);
                } else {
                    int v = board[i][j] - '1';
                    row[i][v] = col[j][v] = block[i / 3][j / 3][v] = true;
                }
            }
        dfs(0);
    }

    private void dfs(int k) {
        if (k == t.size()) {
            ok = true;
            return;
        }
        int i = t.get(k) / 9, j = t.get(k) % 9;
        for (int v = 0; v < 9; ++v) {
            if (!row[i][v] && !col[j][v] && !block[i / 3][j / 3][v]) {
                row[i][v] = col[j][v] = block[i / 3][j / 3][v] = true;
                board[i][j] = (char) (v + '1');
                dfs(k + 1);
                row[i][v] = col[j][v] = block[i / 3][j / 3][v] = false;
            }
        }
        if (ok) {
            return;
        }
    }
}

}
```

Java

Backtracking · LeetCode · By Pattern

— 548 —

LeetCode

```
public class Solution {
    public void solveSudoku(char[][] board) {
        this.board = new char[9][9];
        for (var i = 0; i < 9; ++i)
            for (var j = 0; j < 9; ++j)
                if (board[i][j] != '.')
                    this.board[i, j] = (short) (1 << (board[i][j] - '0' - 1));
        if (solveSudoku(0, 0))
            for (var i = 0; i < 9; ++i)
                for (var j = 0; j < 9; ++j)
                    if (board[i][j] == '.')
                        board[i][j] = '0';
                        while (this.board[i, j].Value != 0)
                            this.board[i, j] += 1;
        board[i][j] = (char) (board[i][j] + 1);
        this.board[i, j] += 1;
    }
}

private short[9][9] board;

private bool validateHorizontalRule(int row)
{
    ushort temp = 0;
    for (var i = 0; i < 9; ++i)
        if (board[row, i].HasValue)
            if ((temp | board[row, i].Value) == temp)
                return false;
            temp |= board[row, i].Value;
    }
}
```

Backtracking · LeetCode · By Pattern

— 549 —

LeetCode

```
return true;
}

private bool validateVerticalRule(int column)
{
    ushort temp = 0;
    for (var i = 0; i < 9; ++i)
        if (board[i, column].HasValue)
            if ((temp | board[i, column].Value) == temp)
                return false;
            temp |= board[i, column].Value;
    }
    return true;
}

private bool validateBackRule(int row, int column)
{
    var startRow = row / 3 * 3;
    var startColumn = column / 3 * 3;
    ushort temp = 0;
    for (var i = startRow; i < startRow + 3; ++i)
        for (var j = startColumn; j < startColumn + 3; ++j)
            if (board[i, j].HasValue)
```

[*] Backtracking — Pattern Solution (matched from community answers)

Python

Backtracking · LeetCode · By Pattern

— 550 —

LeetCode

```
def solveSudoku(board):
    # Check if placing the digit 'char' at board[row][col] is valid
    def is_valid(row, col, char):
        for i in range(9):
            # Check the row
            if board[row][i] == char:
                return False
            # Check the column
            if board[i][col] == char:
                return False
            # Check the 3x3 sub-box
            sub_row = 3 * (row // 3) + i // 3
            sub_col = 3 * (col // 3) + i % 3
            if board[sub_row][sub_col] == char:
                return False
        return True

    # Backtracking function to fill the board
    def backtrack():
        for row in range(9):
            for col in range(9):
                if board[row][col] == '.':
                    for char in map(str, range(1, 10)):
                        if is_valid(row, col, char):
                            board[row][col] = char # Place the digit
                            if backtrack():
                                return True # Success; if solution found, bubble up True
                            board[row][col] = '.' # Backtrack if not valid further down
                    return False # No valid digit found, trigger backtracking
        return True # Board completely filled

    backtrack()

# Example usage:
board = [
    ["5","3",".", ".", ".", ".", ".", ".", ".", "."],
    [".", "7",".", ".", "6",".", ".", ".", ".", "."],
    [".",".", "9",".", "1",".", "5",".", ".", "."],
    [".",".", "8",".", "3",".", "4",".", ".", "."],
    [".",".", "3",".", "5",".", "2",".", ".", "."],
    [".",".", "4",".", "7",".", "6",".", ".", "."],
    [".",".", "2",".", "8",".", "9",".", ".", "."],
    [".",".", "6",".", "7",".", "8",".", ".", "."],
    [".",".", "1",".", "9",".", "3",".", ".", "."]
]

solveSudoku(board)
print(board)
```

#51

HARD

N-Queens

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Backtracking

Lists Grid 189

Problem:

Backtracking · LeetCode · By Pattern

— 551 —

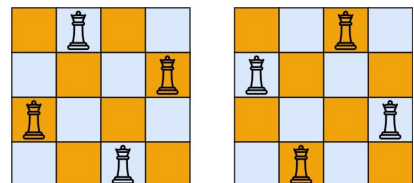
LeetCode

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:



Input: n = 4
Output: [[".Q..", "...Q.", "Q...", "Q..."], ["Q...", "Q...", "...Q.", ".Q.."]]
Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Example 2:

Input: n = 1

Output: [["Q"]]

Constraints:

• 1 <= n <= 9

>> Brute Force [Backtracking] Interview Prep

Python

Backtracking · LeetCode · By Pattern

— 552 —

LeetCode


```
# Brute Force Approach
class Solution:
    def solveNQueens(self, n):
        # Brute Force O(n^n) - n! - place a queen in every column each row, filter valid
        from itertools import product
        def is_valid_placement(row, col):
            for i in range(1, row):
                for j in range(1, len(placement)):
                    if placement[i] == placement[j]: return False # same col
                    if abs(placement[i] - placement[j]) == abs(i - j): return False # diagonal
            return True
        result = []
        for cols in product(range(n), repeat=n):
            if is_valid(cols):
                board = []
                for c in cols:
                    row = "." * c + "Q" + "." * (n - col - 1)
                    board.append(row)
                result.append(board)
        return result
```

Python

Python

```
def solveNQueens(n):
    result = []

    # Helper function to backtrack
    def backtrack(row, columns, diag1, diag2, board):
        # Base case: If all rows are filled, add a copy of board to result
        if row == n:
            result.append(board)
            return

        # Try to place a queen in each column in the current row
        for col in range(n):
            # Check if the column or diagonal is not under attack
            if col in columns or (row - col) in diag1 or (row + col) in diag2:
                continue

            # Place the queen
            board[row] = "." * col + "Q" + "." * (n - col - 1)
            # Mark the column and diagonals as occupied
            columns.add(col)
            diag1.add(row - col)
            diag2.add(row + col)

            # Move to the next row
            backtrack(row + 1, columns, diag1, diag2, board)
```

Backtracking - LeetCode - By Pattern

— 553 —

LeetCode

```
# Backtrack: remove the queen and unmark the axis
columns.remove(col)
diag1.remove(row - col)
diag2.remove(row + col)

# Backtrack: move onto next empty row
board = ["." * n for _ in range(n)]
backtrack(0, set(), set(), set(), board)
return result

# Create result
print(solveNQueens(4))
```

Python

Python

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        ans = []
        cols = [False] * n
        diag1 = [False] * (2 * n - 1)
        diag2 = [False] * (2 * n - 1)

        def dfs(row, board, list1):
            if row == n:
                ans.append(board)
                return

            for j in range(n):
                if cols[j] or diag1[row + j] or diag2[row - j + n - 1]:
                    continue
                cols[j] = diag1[row + j] = diag2[row - j + n - 1] = True
                dfs(row + 1, board + ["." * j + "Q" + "." * (n - j - 1)], list1 + [j])
                cols[j] = diag1[row + j] = diag2[row - j + n - 1] = False

        dfs(0, [])
        return ans
```

Python

Backtracking - LeetCode - By Pattern

— 554 —

LeetCode

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(row):
            if row == n:
                ans.append("".join(row) for row in q)
                return
            for j in range(n):
                if col[j] or diag1[row + j] + udgn - 1 + j == 0:
                    q[row] = "Q"
                    col[j] = diag1 + j] = udgn - 1 + j] = 1
                    dfs(row + 1)
                    col[j] = diag1 + j] = udgn - 1 + j] = 0
                    q[row] = "."

        ans = []
        q = ["."] * n
        for _ in range(n):
            col = [0] * n
            udg = [0] * (n - 1)
            dfs(0)
            return ans
```

Python

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(row):
            if row == n:
                ans.append("".join(row) for row in q)
                return
            for j in range(n):
                if col[j] or diag1[row + j] + udgn - 1 + j == 0:
                    q[row] = "Q"
                    col[j] = diag1 + j] = udgn - 1 + j] = 1
                    dfs(row + 1)
                    col[j] = diag1 + j] = udgn - 1 + j] = 0
                    q[row] = "."

        ans = []
        q = ["."] * n
        for _ in range(n):
            col = [0] * n
            udg = [0] * (n - 1)
            dfs(0)
            return ans
```

C++

C++

Backtracking - LeetCode - By Pattern

— 555 —

LeetCode

```
class Solution {
public:
    vector<vector<string>> solveNQueens(int n) {
        vector<int> cols(n, 0);
        vector<int> diag1 = 1;
        vector<int> udgn = 1;
        vector<vector<string>> ans;
        vector<string> tmp;
        function<void(int)> dfs = [&](int i) -> void {
            if (i == n) {
                ans.push_back(tmp);
                return;
            }
            for (int j = 0; j < n; ++j) {
                if (col[j] or diag1[i + j] + udgn - 1 + j == 0) {
                    tmp[i] = "Q";
                    col[j] = diag1 + j] = udgn - 1 + j] = 1;
                    dfs(i + 1);
                    col[j] = diag1 + j] = udgn - 1 + j] = 0;
                    tmp[i] = ".";
                }
            }
        };
        dfs(0);
        return ans;
    }
};
```

Java

Java

```
class Solution {
    private List<List<String>> ans = new ArrayList<>();
    private List<int> col;
    private List<int> dg;
    private List<int> udg;
    private String[] tmp;
    private int n;

    public List<List<String>> solveNQueens(int n) {
        this.n = n;
        col = new List<int>();
        dg = new List<int>();
        udg = new List<int>();
        tmp = new String[n];
        for (int i = 0; i < n; ++i) {
            Arrays.fill(tmp, ".");
        }
        dfs(0);
        return ans;
    }

    private void dfs(int i) {
        if (i == n) {
            List<String> t = new ArrayList<>();
            for (int j = 0; j < n; ++j) {
                t.add(tmp[j]);
            }
            ans.add(t);
        }
    }
}
```

Backtracking - LeetCode - By Pattern

— 556 —

LeetCode

```
t.add(String.join("", q[i]));
}
ans.add(t);
return;
}

for (int j = 0; j < n; ++j) {
    if (col[j] or diag1[row + j] + udgn - 1 + j == 0) {
        q[row] = "Q";
        col[j] = diag1 + j] = udgn - 1 + j] = 1;
        dfs(row + 1);
        col[j] = diag1 + j] = udgn - 1 + j] = 0;
        q[row] = ".";
    }
}
}
```

Java

```
public class Solution {
    private List<int> col;
    private List<int> dg;
    private List<int> udg;
    private List<List<String>> ans = new ArrayList<>();
    private List<String> t = new List<String>();

    public List<List<String>> solveNQueens(int n) {
        this.n = n;
        col = new List<int>();
        dg = new List<int>();
        udg = new List<int>();
        dfs(0);
        return ans;
    }

    private void dfs(int i) {
        if (i == n) {
            ans.add(new List<String>(t));
            return;
        }
        for (int j = 0; j < n; ++j) {
            if (col[j] or diag1[row + j] + udgn - 1 + j == 0) {
                char[] row = new char[n];
                Arrays.fill(row, ".");
                row[j] = "Q";
                t.add(new String(row));
                col[j] = diag1 + j] = udgn - 1 + j] = 1;
                dfs(i + 1);
                col[j] = diag1 + j] = udgn - 1 + j] = 0;
                t.remove(t.size() - 1);
            }
        }
    }
}
```

TypeScript

TypeScript

Backtracking - LeetCode - By Pattern

— 557 —

LeetCode

```
function solveNQueens(n: number): string[][] {
    const col: number[] = new Array(n).fill(0);
    const dg: number[] = new Array(n * 2).fill(0);
    const udg: number[] = new Array(n * 2).fill(0);
    const ans: string[][] = [];
    const t: string[] = Array(n).fill("");
    const dfs = (i: number) => {
        if (i == n) {
            ans.push(t.map(x => x.join(""));
            return;
        }
        for (int j = 0; j < n; ++j) {
            if (col[j] or dg[i + j] + udg - 1 + j == 0) {
                t[i] = "Q";
                col[j] = dg[i + j] + udg - 1 + j] = 1;
                dfs(i + 1);
                col[j] = dg[i + j] + udg - 1 + j] = 0;
                t[i] = ".";
            }
        }
    };
    dfs(0);
    return ans;
}
```

Go

Go

```
func solveNQueens(n int) (ans [][]string) {
    col := make([]int, n)
    dg := make([]int, n * 2)
    udg := make([]int, n * 2)
    t := make([]byte, n)
    for i := range t {
        t[i] = '.'
    }
    for j := range t {
        t[j] = '.'
    }
    dfs := func(i int) {
        if i == n {
            ans = append(ans, string(t))
            return
        }
        for j := 0; j < n; ++j {
            if col[j] or dg[i + j] + udg - 1 + j == 0 {
                col[j] = dg[i + j] + udg - 1 + j] = 1;
                t[i] = 'Q';
                dfs(i + 1);
                col[j] = dg[i + j] + udg - 1 + j] = 0;
                t[i] = '.';
            }
        }
    }
    dfs(0)
    return ans
}
```

Backtracking - LeetCode - By Pattern

— 558 —

LeetCode

```
        dfs(i + 1)
        t[i][j] = -1
        col[j], diag1[j], diag2[j-i] = 0, 0, 0
    }
}
dfs(0)
return
}
```

[*] Backtracking -- Pattern Solution (matched from community answers)

```
def solveNQueens(n):
    result = []

    # Helper function to backtrack
    def backtrack(row, columns, diag1, diag2, board):
        # Base case: if all rows are filled, add a copy of board to result
        if row == n:
            result.append(list(board))
            return

        # Try to place a queen in each column in the current row
        for col in range(n):
            # If col is columns or (row - col) is diag1 or (row + col) is diag2:
            # continue
            continue

            # Place the queen
            board[row] = '.' + col * 'Q' + '.' * (n - col - 1)
            # Mark the column and diagonals as occupied
            columns.add(col)
            diag1.add(row - col)
            diag2.add(row + col)

            # Move to the next row
            backtrack(row + 1, columns, diag1, diag2, board)

            # Backtrack: remove the queen and unmark the sets
            columns.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

        # Initialize board with empty row
        board = [ "." * n for _ in range(n) ]
        backtrack(0, set(), set(), set(), board)
        return result

    # Example usage:
    print(solveNQueens(4))
```

#126 HARD Word Ladder II

Backtracking · LeetCode · By Pattern

— 559 —

LeetCode

```
# Brute Force Approach
class Solution:
    def findLadders(self, beginWord, endWord, wordList):
        # Brute Force - exhaustive backtracking without pruning
        results = []
        def backtrack(start, path):
            results.append(list(path))
            for i in range(start, len(beginWord)):
                path.append(beginWord[i])
                backtrack(i + 1, path)
                path.pop()
            backtrack(0, [])
            return results
```

Python

Python

```
# Python solution for Word Ladder II

from collections import defaultdict, deque

def findLadders(beginWord, endWord, wordList):
    # Convert list to set for fast lookup
    wordSet = set(wordList)
    if endWord not in wordSet:
        return []

    # Dictionary to hold parent pointers for backtracking: child -> list of parents
    parents = defaultdict(list)
    # Set for words visited at current BFS level
    level = {beginWord}
    # Flag to stop BFS when endWord is reached
    found = False

    while level and not found:
        next_level = set()
        # Remove words of current level from wordSet to prevent cycles
        wordSet -= level
        for word in level:
            # If word is a single-letter transformation
            for i in range(len(word)):
                for char in "abcdefghijklmnopqrstuvwxyz":
                    candidate = word[:i] + char + word[i+1:]
                    if candidate in wordSet:
                        if candidate == endWord:
                            found = True
                        next_level.add(candidate)
                        parents[candidate].append(word)
        level = next_level

    # Backtrack from endWord to beginWord using the parents dictionary
    def backtrack(word, path):
        if word == beginWord:
            # Append reversed path to have the sequence from beginWord to endWord
            res.append(path[::-1])
            return
        # Recurse over all parents of the current word
        for parent in parents[word]:
            backtrack(parent, path + [parent])

    if found:
        backtrack(endWord, [endWord])
    return res
```

Backtracking · LeetCode · By Pattern

— 561 —

LeetCode

```
while currentLevelWords:
    for word in currentLevelWords:
        wordSet.discard(word)
        nextLevelWords = set()
        reachedEndWord = False
        for parent in self._getChildren(parent, wordSet):
            if child in wordSet:
                nextLevelWords.add(child)
                graph[parent].append(child)
            if child == endWord:
                reachedEndWord = True
        if reachedEndWord:
            return True
        currentLevelWords = nextLevelWords
    return False

def _dfs(self, graph, dict[str, list[str]], word str, endWord str, path: list[str], ans: list[list[str]], visited: set[str]) -> list[str]:
    children = []
    s = list(word)
    for i, char in enumerate(s):
        for c in string.ascii_lowercase:
            if c == char:
                continue
            s[i] = c
            child = ''.join(s)
            if child in wordSet:
                children.append(child)
            s[i] = char
    return children

def _dfs(self, graph, dict[str, list[str]], word str, endWord str, path: list[str], ans: list[list[str]], visited: set[str]) -> list[str]:
    if word == endWord:
        ans.append(path.copy())
        return
    for child in graph.get(word, []):
        path.append(child)
        self._dfs(graph, endWord, path, ans)
        path.pop()
```

Python

Backtracking · LeetCode · By Pattern

— 563 —

LeetCode

LeetCode · SimplyTest · WalkCC · LeetCode · Editorial
Hash Table · String · Backtracking · BFS
Like · Donny Zhang

Problem:

A transformation sequence from word beginWord to word endWord using a dictionary wordList is a sequence of words beginWord -> s1 -> s2 -> ... -> sk such that:

- Every adjacent pair of words differs by a single letter.
- Every si for 1 <= i <= k is in wordList. Note that beginWord does not need to be in wordList.
- sk == endWord

Given two words, beginWord and endWord, and a dictionary wordList, return all the shortest transformation sequences from beginWord to endWord, or an empty list if no such sequence exists. Each sequence should be returned as a list of the words [beginWord, s1, s2, ..., sk].

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","let","log","cog"]
Output: [["hit","hot","dot","dog","cog"],["hit","hot","log","cog"]]
Explanation: There are 2 shortest transformation sequences:
"hit" -> "hot" -> "dot" -> "dog" -> "cog"
"hit" -> "hot" -> "log" -> "cog"
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","let","log"]
Output: []
Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.
```

Constraints:

- 1 <= beginWord.length <= 5
- endWord.length == beginWord.length
- 1 <= wordList.length <= 500
- wordList[i].length == beginWord.length
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- beginWord != endWord
- All the words in wordList are unique.
- The sum of all shortest transformation sequences does not exceed 105.

>> Brute Force (Backtracking) Interview Prep

Python

Backtracking · LeetCode · By Pattern

— 560 —

LeetCode

```
candidate = word[i] + char + word[i+1:]
if candidate in wordSet:
    # If candidate is the endWord, note that we have found a path
    if candidate == endWord:
        found = True
        # Record the current word as a parent of candidate word
        parents[candidate].append(word)
        next_level.add(candidate)
    # Move to the next level of the BFS
    level = next_level
```

```
# The result list to hold all valid transformation sequences
res = []
```

```
# Helper function to backtrack from endWord to beginWord using the parents dictionary
def backtrack(word, path):
    if word == beginWord:
        # Append reversed path to have the sequence from beginWord to endWord
        res.append(path[::-1])
        return
    # Recurse over all parents of the current word
    for parent in parents[word]:
        backtrack(parent, path + [parent])

    if found:
        backtrack(endWord, [endWord])
    return res
```

Python

Python

```
class Solution:
    def findLadders(self, beginWord: str, endWord: str, wordList: list[str]) -> list[list[str]]:
        wordSet = set(wordList)
        if endWord not in wordList:
            return []

        # Build the graph from the beginWord to the endWord
        graph: dict[str, list[str]] = defaultdict(list)
        if not self._bfs(beginWord, endWord, wordSet, graph):
            return []

        ans = []
        self._dfs(graph, beginWord, endWord, [beginWord], ans)
        return ans

    def _bfs(
        self,
        beginWord: str,
        endWord: str,
        wordSet: set[str],
        graph: dict[str, list[str]],
    ) -> bool:
        currentLevelWords = {beginWord}
```

Backtracking · LeetCode · By Pattern

— 562 —

LeetCode

```
...
remove() same as discard()

s = set([1,2,3])
s.remove(2)
s = {1,3}

s = set([1,2,3])
s.discard(2)
s = {1,3}

s.add(1,2,3)
s = {1,2,3}
s.discard(5)
s = {1,2,3}
Traceback (most recent call last):
File "solution.py", line 1, in module
SyntaxError: NN
...
```

```
class Solution:
    def findLadders(
        self, beginWord: str, endWord: str, wordList: list[str]
    ) -> list[list[str]]:
        # endWord is beginWord
        # Return empty list to end, too many paths not in final result
        def dfs(path, cur):
            if cur == beginWord:
                ans.append(path[::-1])
                return
            for precursor in precursors:
                path.append(precursor)
                dfs(path, precursor)
                path.pop()

        ans = []
        words = set(wordList)
        if endWord not in words:
            return ans
        # No recursion if beginWord not in set
        words.discard(beginWord)
        dist = {beginWord: 0}
        prev = defaultdict(set)
        q = deque([beginWord])
        found = False
        step = 0
        while q and not found:
            step += 1
```

Backtracking · LeetCode · By Pattern

— 564 —

LeetCode

```

        for i in range(len(s), 0, -1):
            p = q.popLeft()
            s = list(s)
            for j in range(len(s)):
                ch = s[j]
                for j in range(26):
                    s[j] = chr(ord('a') + j)
                    t = s + ch
                    if dict.get(t, 0) == step:
                        prev[j].add(p) # revealed 3 lines below
                    if t not in words:
                        # if above == step not, then t must be removed from words[] from previous
                        continue
                    prev[j].add(p) # revealed 3 lines above
                    words.discard(t)
                    q.append(t)
                    dict[t] = step
                    if endWord == t:
                        found = True
                    s[j] = ch

            if found:
                path = [endWord]
                dfs(graph, endWord)
                return ans

#####
from collections import deque

class Solution(object):

```

```

C++
C++

```

```

class Solution {
public:
    vector<vector<string>> findAddr(string beginWord, string endWord,
        vector<string>& wordList) {
        unordered_set<string> wordSet(wordList.begin(), wordList.end());
        if (!wordSet.contains(endWord))
            return {};

        // ["hit", "hot", "hlt", "hst", "hst", "hst", ...]
        unordered_map<string, vector<string>> graph;

        // Build the graph from the words to the endWord.
        if (!isFirst(beginWord, endWord, wordSet, graph))
            return {};

        vector<vector<string>> ans;
        dfs(graph, beginWord, endWord, {beginWord}, ans);
        return ans;
    }

private:
    bool bfs(const string& beginWord, const string& endWord,
        unordered_set<string>& wordSet,
        unordered_map<string, vector<string>>& graph) {
        unordered_set<string> currentLevelWords({beginWord});

        while (!currentLevelWords.empty()) {
            for (const string& word : currentLevelWords)
                wordSet.remove(word);
            unordered_set<string> nextLevelWords;
            bool reachEndWord = false;
            for (const string& parent : currentLevelWords) {
                vector<string> children;
                getChildren(parent, wordSet, children);
                for (const string& child : children) {
                    if (!wordSet.contains(child)) {
                        nextLevelWords.insert(child);
                        graph[parent].push_back(child);
                    }
                    if (child == endWord)
                        reachEndWord = true;
                }
            }
            if (reachEndWord)
                return true;
            currentLevelWords = std::move(nextLevelWords);
        }
        return false;
    }
}

```

```

void getChildren(const string& parent, const unordered_set<string>& wordSet,
    vector<string>& children) {
    string s(parent);

    for (int i = 0; i < s.length(); ++i) {
        const char cache = s[i];
        for (char c = 'a'; c <= 'z'; ++c) {
            if (c == cache)
                continue;
            s[i] = c;
            if (!wordSet.contains(s))
                children.push_back(s);
        }
        s[i] = cache;
    }
}

void dfs(const unordered_map<string, vector<string>>& graph,
    const string& word, const string& endWord, vector<string>& path,
    vector<vector<string>>& ans) {
    if (!word == endWord) {
        ans.push_back(path);
        return;
    }

    if (!graph.contains(word))
        return;
    for (const string& child : graph.at(word)) {
        path.push_back(child);
    }
}

```

```

Java
Java

```

```

class Solution {
public:
    List<List<String>> findAddr(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord))
            return new ArrayList<>();

        // ["hit", "hot", "hst", "hst", "hst", "hst", ...]
        Map<String, List<String>> graph = new HashMap<>();

        // Build the graph from the beginWord to the endWord.
        if (!isFirst(beginWord, endWord, wordSet, graph))
            return new ArrayList<>();

        List<List<String>> ans = new ArrayList<>();
        List<String> path = new ArrayList<>();
        dfs(graph, beginWord, endWord, path, ans);
        return ans;
    }

    private boolean bfs(String beginWord, String endWord, Set<String> wordSet,
        Map<String, List<String>> graph) {
        Set<String> currentLevelWords = new HashSet<>();
        currentLevelWords.add(beginWord);
        boolean reachEndWord = false;

        while (!currentLevelWords.isEmpty()) {
            for (final String word : currentLevelWords)
                wordSet.remove(word);
            Set<String> nextLevelWords = new HashSet<>();
            for (final String parent : currentLevelWords) {
                graph.putIfAbsent(parent, new ArrayList<>());
                for (final String child : getChildren(parent, wordSet)) {
                    if (!wordSet.contains(child)) {
                        nextLevelWords.add(child);
                        graph.get(parent).add(child);
                    }
                    if (child.equals(endWord))
                        reachEndWord = true;
                }
            }
            if (reachEndWord)
                return true;
            currentLevelWords = nextLevelWords;
        }
        return false;
    }

    private List<String> getChildren(String parent, Set<String> wordSet) {
        List<String> children = new ArrayList<>();
    }
}

```

```

StringBuilder sb = new StringBuilder(parent);

for (int i = 0; i < sb.length(); ++i) {
    final char cache = sb.charAt(i);
    for (char c = 'a'; c <= 'z'; ++c) {
        if (c == cache)
            continue;
        sb.setCharAt(i, c);
        final String child = sb.toString();
        if (!wordSet.contains(child))
            children.add(child);
    }
    sb.setCharAt(i, cache);
}
return children;

private void dfs(Map<String, List<String>> graph, final String word, final String endWord,
    List<String> path, List<List<String>> ans) {
    if (!word.equals(endWord)) {
        ans.add(new ArrayList<>(path));
        return;
    }
    if (!graph.containsKey(word))
        return;
    for (final String child : graph.get(word)) {
        path.add(child);
        dfs(graph, child, endWord, path, ans);
    }
}

```

```

Java

```

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;
import java.util.Set;

public class Word_Ladder_II {
    // Iteration 1st
    // https://leetcode.com/problems/word-ladder-ii/solution/
    public class Solution {
        List<List<String>> list = new ArrayList<>();

        public List<List<String>> findAddr(String start, String end, Set<String> dict) {
            if (start == null || end == null || dict == null) return list;

            dict.add(end); // !!!

            Queue<String> q = new LinkedList<>();
            int level = 1;
        }
    }
}

```

```

int currentLevelCount = 1;
int minLevelCount = 0;
boolean found = false;
int foundLevel = -1;

// From word, to all paths
HashMap<String, ArrayList<String>> hm = new HashMap<>();
ArrayList<String> startPath = new ArrayList<>();
ArrayList<String> allPaths = new ArrayList<>();

q.offer(start);
ArrayList<String> singlePath = new ArrayList<>();
ArrayList<String> allPaths = new ArrayList<>();

singlePath.add(start);
allPaths.add(singlePath);
hm.put(start, allPaths);

while (!q.isEmpty()) {
    String current = q.poll();
    currentLevelCount--; // [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445, 446, 447, 448, 449, 450, 451, 452, 453, 454, 455, 456, 457, 458, 459, 460, 461, 462, 463, 464, 465, 466, 467, 468, 469, 470, 471, 472, 473, 474, 475, 476, 477, 478, 479, 480, 481, 482, 483, 484, 485, 486, 487, 488, 489, 490, 491, 492, 493, 494, 495, 496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509, 510, 511, 512, 513, 514, 515, 516, 517, 518, 519, 520, 521, 522, 523, 524, 525, 526, 527, 528, 529, 530, 531, 532, 533, 534, 535, 536, 537, 538, 539, 540, 541, 542, 543, 544, 545, 546, 547, 548, 549, 550, 551, 552, 553, 554, 555, 556, 557, 558, 559, 560, 561, 562, 563, 564, 565, 566, 567, 568, 569, 570, 571, 572, 573, 574, 575, 576, 577, 578, 579, 580, 581, 582, 583, 584, 585, 586, 587, 588, 589, 590, 591, 592, 593, 594, 595, 596, 597, 598, 599, 600, 601, 602, 603, 604, 605, 606, 607, 608, 609, 610, 611, 612, 613, 614, 615, 616, 617, 618, 619, 620, 621, 622, 623, 624, 625, 626, 627, 628, 629, 630, 631, 632, 633, 634, 635, 636, 637, 638, 639, 640, 641, 642, 643, 644, 645, 646, 647, 648, 649, 650, 651, 652, 653, 654, 655, 656, 657, 658, 659, 660, 661, 662, 663, 664, 665, 666, 667, 668, 669, 670, 671, 672, 673, 674, 675, 676, 677, 678, 679, 680, 681, 682, 683, 684, 685, 686, 687, 688, 689, 690, 691, 692, 693, 694, 695, 696, 697, 698, 699, 700, 701, 702, 703, 704, 705, 706, 707, 708, 709, 710, 711, 712, 713, 714, 715, 716, 717, 718, 719, 720, 721, 722, 723, 724, 725, 726, 727, 728, 729, 730, 731, 732, 733, 734, 735, 736, 737, 738, 739, 740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755, 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771, 772, 773, 774, 775, 776, 777, 778, 779, 780, 781, 782, 783, 784, 785, 786, 787, 788, 789, 790, 791, 792, 793, 794, 795, 796, 797, 798, 799, 800, 801, 802, 803, 804, 805, 806, 807, 808, 809, 810, 811, 812, 813, 814, 815, 816, 817, 818, 819, 820, 821, 822, 823, 824, 825, 826, 827, 828, 829, 830, 831, 832, 833, 834, 835, 836, 837, 838, 839, 840, 841, 842, 843, 844, 845, 846, 847, 848, 849, 850, 851, 852, 853, 854, 855, 856, 857, 858, 859, 860, 861, 862, 863, 864, 865, 866, 867, 868, 869, 870, 871, 872, 873, 874, 875, 876, 877, 878, 879, 880, 881, 882, 883, 884, 885, 886, 887, 888, 889, 890, 891, 892, 893, 894, 895, 896, 897, 898, 899, 900, 901, 902, 903, 904, 905, 906, 907, 908, 909, 910, 911, 912, 913, 914, 915, 916, 917, 918, 919, 920, 921, 922, 923, 924, 925, 926, 927, 928, 929, 930, 931, 932, 933, 934, 935, 936, 937, 938, 939, 940, 941, 942, 943, 944, 945, 946, 947, 948, 949, 950, 951, 952, 953, 954, 955, 956, 957, 958, 959, 960, 961, 962, 963, 964, 965, 966, 967, 968, 969, 970, 971, 972, 973, 974, 975, 976, 977, 978, 979, 980, 981, 982, 983, 984, 985, 986, 987, 988, 989, 990, 991, 992, 993, 994, 995, 996, 997, 998, 999, 1000]
    currentLevelCount++;
    for (int i = 0; i < currentLevelCount; ++i) {
        char[] array = currentLevelWords.get(i).toCharArray();
        for (char c = 'a'; c <= 'z'; ++c) {
            array[i] = c;
            String each = new String(array);
            if (!each.equals(end)) {
                found = true;
                foundLevel = level;
            }
        }
    }
    if (dict.contains(each)) {
        // q.offer(each);
        minLevelCount++;
        ArrayList<String> prevAllPaths = hm.get(current);
        if (hm.containsKey(each))
            allPaths = hm.get(each);
        else {
            // q.offer(each);
            allPaths = new ArrayList<>();
            hm.put(each, allPaths);
        }
    }
}

```

```

    }
    // duplicate: this is if the key || no path for new word, or new word path is
see more than previous path
    // using this if, the if "visited" check can be removed
    // if (allPaths.size() == 0 || prevAllPaths.size() + 1 == allPaths.size())
}

Go
Go

func findWords(beginWord string, endWord string, wordList []string) []string {
    var ans [][]string
    start := make(map[string]bool)
    for w, word := range wordList {
        words[word] = true
    }
    if words(endWord) {
        return ans
    }
    words[beginWord] = false
    dict := map[string]bool{beginWord: 0}
    prev := map[string]map[string]bool{}
    q := []string{beginWord}
    found := false
    step := 0
    for len(q) > 0 && !found {
        step++
        for i := len(q); i > 0; i-- {
            p := q[i]
            q = q[:i]
            chars := []byte(p)
            for j := 0; j < len(chars); j++ {
                ch := chars[j]
                for k := 0; k < len("a-z"); k++ {
                    char[j] := bytes[k]

```

Backtracking - LeetMastery — By Pattern

— 571 —

LeetMastery

```

t = string(chars)
if w == -1: ok =
    if w == -1: ok =
        prev[i] = true
    }
}
if hasNext(i)
    continue
if len(prev[i]) == 0 {
    prev[i] = ansMap[string]bool{}
    prev[i] = true
    word[i] = false
    q = append(q, i)
    dist[i] = 1
    if word[i] == 1 {
        found = true
    }
    char[i] = ch
}
}
}

var dfs func(path [string, begin, cur string])
dfs = func(path [string, begin, cur string]) {
    if cur == begin {
        cp := make([string, len(path)])
        copy(cp, path)
        ans = append(ans, cp)
        return
    }
    for k in range prev[cur] {
        path = append([string], path...)
        dfs(path, begin+end, k)
        path = path[1:]
    }
    if found {
        path = [string](append(
            path, begin+end, number))
    }
    return ans
}

```

[*] Backtracking — Pattern Solution (matched from community answers)

Python

Backtracking · LeetMastery — By Pattern

— 572 —

LeetMastery

```

Python solution for more Lesson 13

from collections import defaultdict, deque

def findAnagrams(beginWord, endWord, wordList):
    # Convert list to set for fast lookup
    wordSet = set(wordList)
    if endWord not in wordSet:
        return []

    # Dictionary to hold parent pointers for backtracking: child -> list of parents
    parents = defaultdict(list)

    # For words with length at current BFS level
    level = deque([beginWord])

    # Flag to stop BFS when endWord is reached
    found = False

    while level and not found:
        nextLevel = set()
        # Remove words of current level from wordSet to prevent cycles
        words = level
        for word in level:
            # Try all possible one-letter transformations
            for char in 'abcdefghijklmnopqrstuvwxyz':
                candidate = word[0] + char + word[1:]
                if candidate in wordSet:
                    if candidate is endWord:
                        # If endWord is the endWord, note that we have found a path
                        return [word]
                    if candidate == endWord:
                        found = True
                    # Record the current word as a parent of candidate word
                    parents[candidate].append(word)
        level, nextLevel = nextLevel, level

    # Return the next level of the BFS
    level = nextLevel

    # The result list to hold all valid transformation sequences
    res = []

    # Helper function to backtrack from endWord to beginWord using the parents Dictionary
    def backtrack(word, path):
        if word == beginWord:
            # Append reversed path to have the sequence from beginWord to endWord
            res.append(path[::-1])
            return
        # Recurse down all parents of the current word
        for parent in parents[word]:
            backtrack(parent, path + [parent])

    if found:
        backtrack(endWord, [endWord])
    return res

# Example usage
print(findAnagrams("bat", "cap", ["bat", "bat", "dog", "lat", "dog", "cap"]))

```

Backtracking · LeetMastery — By Pattern

— 573 —

LeetMastery

#996 **HARD**

Number of Squareful Arrays

[LeetCode](#) · [SimplyLeet](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

[Array](#) · [Math](#) · [Dynamic Programming](#) · [Backtracking](#)

Lists: Denny Zhang

Problem:
An array is squareful if the sum of every pair of adjacent elements is a perfect square.

Given an integer array `nums`, return the number of permutations of `nums` that are squareful.

Two permutations `perm1` and `perm2` are different if there is some index `i` such that `perm1[i] != perm2[i]`.

Example 1:

```
Input: nums = [1,17,8]
```

Output: 2
Explanation: [1,8,17] and [17,8,1] are the valid permutations.

Example 2:

Input: num
Output: 1

```
Output: 1
```

Constraints:

- $1 \leq \text{nums.length} \leq 12$
- $0 \leq \text{nums}[i] \leq 109$

[>>> Brute Force \[Backtracking\] Interview Prep](#)

Python

```
# Brute Force Approach
class Solution:
    def is_square(self, num):
        # Brute Force - exhaustive backtracking without pruning
        results = []
        def backtrack(start, path):
            results.append(list(path))
            for i in range(start, len(num)):
                path.append(num[i])
                backtrack(i + 1, path)
            path.pop()
        backtrack(0, [])
        return len(results)
```

Python

Python

Backtracking · LeetMastery — By Pattern

— 574 —

LeetMastery

```

from math import sqrt

# Function to check if a number is a perfect square
def is_square(num):
    root = sqrt(num)
    return root + root == num

def numSquareOfPerp(num):
    n = len(num)
    num.sort() # Sort the array to handle duplicates
    visited = [False] * n
    result = 0

    def dfs(last_index, count):
        nonlocal result
        # If all elements have been used, we found a valid permutation
        if count == n:
            result += 1
            return
        for i in range(1, n):
            if visited[i]:
                continue
            # If last index: if the current element is the same as the previous one and the previous one
            # has't been used
            if i > 0 and num[i] == num[i - 1] and not visited[i - 1]:
                continue

            # If the first element or the one with the last element is a perfect square
            if last_index == -1 or is_square(num[last_index] * num[i]):
                visited[i] = True
                dfs(i, count + 1)
                visited[i] = False

    dfs(-1, 0)
    return result

# Example usage
if __name__ == "__main__":
    print(numSquareOfPerp([1, 3, 37])) # Expected output: 2
    print(numSquareOfPerp([2, 2, 2])) # Expected output: 3

```

Python

Backtracking · LeetMastery — By Pattern

— 575 —

LeetMastery

```

class Solution:
    def multTable(self, n: int, nums: List[int]) -> int:
        ans = 0
        used = [False] * len(nums)

        def isSquare(num: int) -> bool:
            root = math.isqrt(num)
            return root * root == num

        def dfs(path: List[int]) -> None:
            nonlocal ans
            if len(path) > 1 and not isSquare(path[-1] * path[-2]):
                return
            if len(path) == len(nums):
                ans += 1
                return
            for i, n in enumerate(nums):
                if used[i]:
                    continue
                if i < 0 and nums[i] == nums[-1] and not used[-1]:
                    continue
                used[i] = True
                dfs(path + [n])
                used[i] = False

        num.sort()
        dfs([])
        return ans

```

Python

```

class Solution:
    def multTable(self, n: int, nums: List[int]) -> int:
        f = [1] * n
        for i in range(1, n):
            for j in range(1, i):
                if i % j == 1:
                    f[i] = f[j] * f[i // j]
            for k in range(i):
                if (i & k & 1) and k < j:
                    f[i] = nums[i] * nums[k]
            f[i] = int(sqrt(f[i]))
            if f[i] < 1:
                f[i] = f[1] * f[i // j]

        ans = sum(f[i] <= n - 1] for j in range(n))
        for v in Counter(nums).values():
            ans -= factor(v)
        return ans

```

Python

Backtracking · LeetMastery — By Pattern

— 576 —

LeetMastery

```

class Solution {
public:
    int numSquares(int n) {
        int ans = 0;
        for (int i = 1; i <= n; i++) {
            if (i % 4 == 0) continue;
            int cnt = 0;
            for (int j = 1; j <= n; j++) {
                if (i % 4 == 0) continue;
                if (i % 4 == 1) {
                    if (i % 4 == 1) {
                        cnt++;
                    }
                }
            }
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int numSquares(vector<int>& nums) {
        int n = nums.size();
        int f[1] = {0};
        for (int i = 1; i <= n; i++) {
            f[i] = 0;
            for (int j = 1; j <= i; j++) {
                if (i % j == 0) {
                    f[i] = min(f[i], f[j] + 1);
                }
            }
        }
        return f[n];
    }
};

```

Backtracking · LeetCode · By Pattern

— 577 —

LeetCode

```

for (int i = 0; i < n; i++) {
    ans = f(i, 0, 0);
}
return ans;
}

```

Java

```

class Solution {
public:
    int numSquares(vector<int>& nums) {
        int n = nums.size();
        int f[1] = {0};
        for (int i = 1; i <= n; i++) {
            f[i] = 0;
            for (int j = 1; j <= i; j++) {
                if (i % j == 0) {
                    f[i] = min(f[i], f[j] + 1);
                }
            }
        }
        return f[n];
    }
};

```

Backtracking · LeetCode · By Pattern

— 578 —

LeetCode

```

}
Map<Integer, Integer> cnt = new HashMap<>();
for (int i = 0; i < n; i++) {
    cnt.put(i, 0);
}
int[] g = new int[n];
g[0] = 1;
for (int i = 1; i < n; i++) {
    g[i] = g[i-1] + 1;
}
for (int i = 0; i < n; i++) {
    ans = g[i];
}
return ans;
}

```

Go

```

func numSquares(nums []int) int {
    n := len(nums)
    f := make([]int, n+1)
    for i := 1; i <= n; i++ {
        f[i] = math.MaxInt32
    }
    for j := 1; j <= n; j++ {
        for i := j; i <= n; i++ {
            if i % j == 0 {
                f[i] = min(f[i], f[j] + 1)
            }
        }
    }
    return f[n]
}

```

Backtracking · LeetCode · By Pattern

— 579 —

LeetCode

```

ans = f(i, 0, 0);
}
return ans;
}

```

Python

```

class Solution:
    def numSquares(self, n: int) -> int:
        ans = 0
        used = [False] * (n+1)
        def dfs(path: List[int]) -> bool:
            root = math.sqrt(n)
            return root == int(root)
        def dfs(path: List[int]) -> None:
            nontical ans
            if len(path) > 1 and not isSquare(path[-1] + path[-2]):
                return
            if len(path) == len(nums):
                ans = 1
                return
            for i, n in enumerate(nums):
                if used[i]:
                    continue
                if i > 0 and nums[i] == nums[i-1] and not used[i-1]:
                    continue
                used[i] = True
                dfs(path + [n])
                used[i] = False
        num.sort()
        dfs([1])
        return ans

```

Backtracking · LeetCode · By Pattern

— 580 —

LeetCode

Quick Review — Backtracking 10 questions · insights · complexity · solution

#17 Letter Combinations of a Phone Number [Medium]

Key Insights

- Use a mapping from digits to the respective letters as defined by a telephone keypad.
- Backtracking is an ideal approach to generate all possible combinations.
- Each digit adds a level in the recursive tree where you append one letter at a time.
- Handling edge cases such as an empty input string is crucial.

Space & Time

Time Complexity: $O(4^n \cdot n!)$ where n is the length of the input string (each digit may map to up to 4 letters and adding a combination takes $O(n)$ time).

Space Complexity: $O(n)$ for the recursion call stack, not counting the space required for storing the output.

Solution

We can solve the problem using a backtracking algorithm. First, we define a dictionary mapping each digit to its corresponding letters. Then we initiate a recursive function called `backtrack` that takes the current combination and the index of the next digit to process. At each call, if the combination is complete (length equal to the input string), we add it to the result list. Otherwise, we iterate through the letters corresponding to the current digit, append a letter to the combination, and recursively call `backtrack` with the next index. This approach ensures that all possible letter combinations are generated.

#22 Generate Parentheses [Medium]

Key Insights

- Use a backtracking approach to build the valid combinations step by step.
- At any recursion step, you can add an opening parenthesis if you still have one available.
- You can add a closing parenthesis only if it would not result in an invalid sequence (i.e., the number of closing parentheses is less than the number of opening ones).
- The solution leverages recursion to explore all potential valid sequences while pruning invalid paths early.

Space & Time

Time Complexity: $O(4^n \cdot n!)$ where n is the length of the input string.

Space Complexity: $O(n)$ for the recursion stack, plus space for storing all valid combinations.

Solution

The solution uses a recursive backtracking technique to generate the combinations. The algorithm maintains counters for the number of `'('` and `')'` added to the current string. It recursively adds a `'('` if the count is less than n and adds a `')'` if doing so does not make the string invalid (i.e., if the count of `')'` is less than the count of `'('`). When a valid combination reaches a length of $2n$, it is added to the result list.

#39 Combination Sum [Medium]

Key Insights

- Backtracking is a natural fit to explore all potential combinations.
- Sorting candidates simplifies the process by allowing early termination if the current candidate exceeds the remaining target.
- Recursion is used to build combinations and backtrack once the current path exceeds or meets the target.
- The same candidate can be reused in the combination, so the recursive call does not advance the index.

Space & Time

Time Complexity: $O(2^{\text{len}(\text{candidates})})$ in the worst case where many combinations are possible.

Backtracking · LeetCode · By Pattern

— 581 —

LeetCode

Space Complexity: $O(\text{len}(\text{candidates}))$ for the recursion stack and temporary space used to store combinations.

Solution

The solution uses a backtracking approach. First, sort the candidates to enable pruning of recursive calls once the candidate exceeds the remaining target. A recursive helper function builds up a combination, subtracting the candidate value from the target until it reaches zero (a valid combination) or becomes negative (an invalid path). The recursion allows the same candidate to be chosen again. Each valid combination is then added to the result list.

#46 Permutations [Medium]

Key Insights

- Use backtracking to generate all possible permutations.
- Build each permutation by selecting numbers one at a time.
- When the current permutation reaches the length of `nums`, add it to the result.
- Backtracking allows you to undo choices and explore different orderings.

Space & Time

Time Complexity: $O(n! \cdot n)$ - There are $n!$ permutations, and constructing each permutation takes $O(n)$ time.

Space Complexity: $O(n)$ - Recursion depth is limited to n , plus additional space for the current permutation.

Solution

The solution employs a backtracking algorithm. A helper function is used to build permutations recursively. At each step, the algorithm iterates through the available numbers, adds one to the current permutation if it has not been used yet, and recurses to handle the remaining numbers. Once the current permutation matches the length of the input array, it is added to the result list. Then, by "backtracking" (removing the last number added), the algorithm explores alternative possibilities. This systematic exploration ensures that all possible permutations are generated.

#79 Word Search [Medium]

Key Insights

- Use Depth-First Search (DFS) to explore each cell in the board as a potential starting point.
- Employ backtracking to mark cells as visited when exploring a path and revert them back when backtracking.
- Check boundary conditions and ensure the current cell's character matches the corresponding character in the word.
- Given the small grid and word sizes, a recursive solution is both feasible and straightforward.
- Search pruning can be applied by stopping a DFS branch as soon as the current path does not match the prefix of the target word.

Space & Time

Time Complexity: $O(m \cdot n \cdot 3^L)$, where L is the length of the word (each DFS may branch to at most 3 further cells, excluding the coming cell).

Space Complexity: $O(L)$ due to recursion call stack (where L is the word length).

Solution

The solution uses DFS with backtracking to traverse the grid. For each cell, we:

- Check if the cell matches the first character of the word.
- Recursively explore the adjacent cells (up, down, left, right) while marking the current cell as visited.
- If the path does not lead to a complete match, backtrack by resetting the visited cell.
- The DFS stops if all characters in the word have been found in sequence.

 The algorithm leverages in-place modifications (or an auxiliary visited array) to avoid revisiting cells while exploring a candidate path.

#113 Path Sum II [Medium]

Key Insights

- Use Depth-First Search (DFS) to explore all paths from the root to the leaves.

Backtracking · LeetCode · By Pattern

— 582 —

LeetCode

- Backtracking is essential: add the current node value to the path before the recursive calls and remove it (backtrack) after exploring both subtrees.
- At each leaf, check if the accumulated sum equals targetsum.
- Maintain a running sum or subtract the current node's value from the target sum as you traverse.

Space & Time

Time Complexity: $O(N^2)$ in the worst-case scenario (where N is the number of nodes) when most nodes are leaf nodes, since each potential path copy operation can be $O(N)$.

Space Complexity: $O(N)$ for the recursion stack and the path storage, where N is the height of the tree.

Solution

We solve the problem using a recursive DFS approach combined with backtracking. The idea is to traverse the tree while maintaining the current path and the remaining sum needed to reach targetsum. When a leaf node is reached (node with no children), we check if the remaining sum equals the leaf's value - if yes, we record a copy of the current path.

Data structures and algorithmic approaches used:

- Recursion for DFS to traverse to each leaf - List (or Array) for storing the current path - Backtracking to remove the last added element when stepping back up the recursion stack - Copying the path when a valid leaf is reached to store it in the final result.

A common pitfall is to forget to backtrack, which would result in incorrect paths being recorded.

#37 Sudoku Solver [Hard]

Key Insights

- Use backtracking to explore digit placements for each empty cell.

- Validate placements by ensuring that the chosen digit is not already present in the corresponding row, column, or 3x3 sub-box.

- Employ recursion to fill the board incrementally and backtrack upon encountering invalid configurations.

- Although the worst-case time complexity is exponential, the fixed board size (9x9) keeps the problem computationally manageable.

Space & Time

Time Complexity: In the worst-case scenario, the algorithm can explore up to $O(9^n)$ possibilities where n is the number of empty cells. However, since n is at most 81 and many branches are pruned early, the practical runtime is far less.

Space Complexity: $O(n)$ due to the recursion stack, where n is the number of empty cells; given the board's fixed size, this space is constant in practice.

Solution

The solution uses a backtracking approach. For each empty cell on the board, we attempt to place a digit from '1' to '9'. Before placing a digit, we check if the placement is valid by ensuring that:

- The digit is not already present in the same row.
- The digit is not already present in the same column.
- The digit is not already present in the corresponding 3x3 sub-box.

If a valid digit is found, we place it on the board and recursively attempt to solve the rest of the board. If no valid digit can be placed, the algorithm backtracks by reverting the last move and trying a different digit. This recursive search continues until the board is completely filled with a valid Sudoku configuration.

#51 N-Queens [Hard]

Key Insights

- Use backtracking to explore potential positions row by row.

- Use sets to track which columns and diagonals are under attack.

- Diagonals can be tracked by (row - col) and (row + col), ensuring that queens do not conflict along any diagonal.

- Recursively build each valid board configuration and backtrack when a conflict arises.

Backtracking · LeetCode · By Pattern

— 583 —

LeetCode

Space & Time

Time Complexity: $O(n!)$ in the worst-case scenario, as the solution explores permutations for queen placements.

Space Complexity: $O(n^2)$ for storing board configurations and $O(n)$ for recursion stack and conflict-tracking sets.

Solution

We use a backtracking algorithm that places queens row by row. For each row, we try placing a queen in each column. Before placing, we check if the column, main diagonal (row - col), and anti-diagonal (row + col) are free using dedicated sets. If a queen can be safely placed, we mark the column and diagonals as occupied and recurse to the next row. Once all rows are processed, the board configuration is a valid solution and is added to the result.

After that, we backtrack by unmarking the sets and removing the queen from the board. This systematic approach ensures all possible configurations are considered.

#126 Word Ladder II [Hard]

Key Insights

- Use Breadth-First Search (BFS) to traverse level-by-level and capture only the shortest paths.

- Build a parent mapping during BFS to record which words lead to each newly discovered word.

- Once the shortest path is found via BFS, use backtracking (or DFS) from endWord to reconstruct all transformation sequences.

- Remove words as they are visited within a level to avoid unnecessary revisits and cycles.

- The solution must handle multiple shortest paths, so maintaining a list of predecessors for each word is crucial.

Space & Time

Time Complexity: $O(N * L^2 * 26)$ where N is the number of words in the wordList and L is the length of each word.

This accounts for checking all possible one-letter transformations.

Space Complexity: $O(N * L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures.

Solution

We solve the problem in two major phases:

- Use BFS to explore from beginWord to endWord. During BFS, record all parents for each word encountered. This ensures we capture all possible predecessors that yield a shortest transformation.
- Once the BFS completes (once endWord is reached), use backtracking (DFS) beginning from the endWord and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from beginWord to endWord.

This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration.

#996 Number of Squareful Arrays [Hard]

Key Insights

- The problem requires ensuring adjacent pairs sum up to a perfect square.

- Sorting the array helps in efficiently handling duplicates; when iterating in the backtracking, skip duplicate elements to avoid overcounting.

- For each valid candidate, check that the sum with the previous number (if any) is a perfect square.

- Use backtracking (DFS) to generate all valid permutations while maintaining a visited flag for each index.

- Since the array may contain duplicates, extra care is taken to only use each occurrence in a controlled order (i.e., if two identical numbers exist, only the first unvisited instance can be chosen at a given recursive call).

Space & Time

Time Complexity: $O(n! * 2^n)$ in the worst-case scenario due to exploring all bitmask states with n elements and checking pairwise sums.

Space Complexity: $O(n)$ for recursion and $O(n)$ for the visited flag, plus potentially $O(2^n * n)$ for memoization if implemented.

Backtracking · LeetCode · By Pattern

— 584 —

LeetCode

#9 BFS — 10 questions · #9 of 21

#286

MEDIUM

Walls and Gates

LeetCode · SimplifyLogic · WalkCC · LeetCode · Editorial

Array · BFS · Matrix

List: Premium 98

Problem:

You are given an $m \times n$ grid rooms initialized with these three possible values.

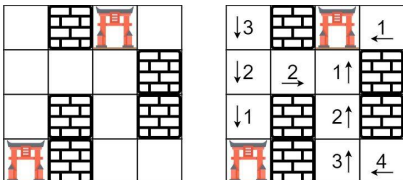
- -1 A wall or an obstacle.

- 0 A gate.

• INF Infinity means an empty room. We use the value $231 - 1 = 2147483647$ to represent INF as you may assume that the distance to a gate is less than 2147483647.

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF.

Example 1:



Input: rooms = [[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]

Output: [[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]

Example 2:

Input: rooms = [[-1]]

Output: [[-1]]

Constraints:

- $m == \text{rooms.length}$

- $n == \text{rooms}[i].length$

- $1 \leq m, n \leq 250$
- $\text{rooms}[i][j]$ is -1, 0, or 231 - 1.

>> Brute Force [BFS] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def wallsAndGates(self, rooms):
        # Since there is 0 (gate) in BFS from every empty room independently
        from collections import deque
        m, n = len(rooms), len(rooms[0])
        INF = 2147483647
        def bfs_from(sr, sc):
            queue = deque([(sr, sc, 0)])
            visited = set()
            while queue:
                r, c, d = queue.popleft()
                if (r, c) in visited: continue
                visited.add((r, c))
                if rooms[r][c] == 0: return d # reached a gate
                for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < m and 0 <= nc < n and rooms[nr][nc] < (-1):
                        queue.append((nr, nc, d + 1))
            return INF
        for r in range(m):
            for c in range(n):
                if rooms[r][c] == INF:
                    rooms[r][c] = bfs_from(r, c)
```

Python

Python

BFS · LeetCode · By Pattern

— 587 —

LeetCode

```
from collections import deque

def wallsAndGates(rooms):
    if not rooms or not rooms[0]:
        return
    rows, cols = len(rooms), len(rooms[0])
    # Directions: up, down, left, right
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    queue = deque()
    # Perform cell search (cells with 0).
    for i in range(rows):
        for c in range(cols):
            if rooms[i][c] == 0:
                queue.append((i, c))
    # Perform multi-source BFS.
    while queue:
        r, c = queue.popleft()
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Prunes boundary if (i) is within bounds and is an empty room.
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == 2147483647:
                rooms[nr][nc] = rooms[r][c] + 1 # reduce distance.
                queue.append((nr, nc))
```

Python

```
class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        mRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        INF = 2147483647
        m = len(rooms)
        n = len(rooms[0])
        q = collections.deque([(i, j)
                               for i in range(m)
                               for j in range(n)
                               if rooms[i][j] == 0])
        while q:
            i, j = q.popleft()
            for dr, dc in mRS:
                x = i + dr
                y = j + dc
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if rooms[x][y] != INF:
                    continue
                rooms[x][y] = rooms[i][j] + 1
                q.append((x, y))
```

Python

BFS · LeetCode · By Pattern

— 588 —

LeetCode

```

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:

        # Do not return anything, modify rooms in-place instead.

        m, n = len(rooms), len(rooms[0])
        inf = 2e+31 - 1
        q = deque([(i, j) for i in range(n) for j in range(n) if rooms[i][j] == 0])
        d = 0
        while q:
            d += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
                    x, y = i + a, j + b
                    if 0 <= x < n and 0 <= y < n and rooms[x][y] == inf:
                        rooms[x][y] = d
                        q.append((x, y))

#####

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:

        # Do not return anything, modify rooms in-place instead.

        m, n = len(rooms), len(rooms[0])
        inf = 2e+31 - 1
        q = deque([(i, j) for i in range(n) for j in range(n) if rooms[i][j] == 0])
        d = 0
        while q:
            d += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
                    x, y = i + a, j + b
                    # With no other blocking inf is always the shortest
                    # Get all values from inf to d, same as working on visited
                    if 0 <= x < n and 0 <= y < n and rooms[x][y] == inf:
                        rooms[x][y] = d
                        q.append((x, y))

#####

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:

```

BFS · LeetCode - By Pattern

— 589 —

LeetCode

```

        if not rooms or not rooms[0]:
            return

        m, n = len(rooms), len(rooms[0])
        visited = [[False] * n for _ in range(m)] # visited map shared among all gates
        def fill(i, j, distance: int) -> None:
            wallsAndGates.m, n

            # rooms[i][j] == 0 meaning == 1 or == 0, both stop
            if i < 0 or i == n or j < 0 or j == n or rooms[i][j] == 0 or visited[i][j]:
                return

            rooms[i][j] = min(rooms[i][j], distance)
            visited[i][j] = True

            fill(i - 1, j, distance + 1)
            fill(i, j - 1, distance + 1)
            fill(i + 1, j, distance + 1)
            fill(i, j + 1, distance + 1)

            visited[i][j] = False

        for i in range(m):
            for j in range(n):

                if rooms[i][j] == 0: # start from every gate
                    fill(i, j, 0)

#####

from collections import deque
INF = 2147483647

class Solution(object):
    def wallsAndGates(self, rooms):
        # type: rooms: List[List[int]]
        # type: void Do not return anything, modify rooms in-place instead.

        queue = deque()
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        for i in range(0, len(rooms)):
            for j in range(0, len(rooms[0])):
                if rooms[i][j] == 0:
                    queue.append((i, j))

        while queue:
            i, j = queue.popleft()

            for di, dj in directions:
                p, q = i + di, j + dj
                if 0 <= p < len(rooms) and 0 <= q < len(rooms[0]) and rooms[p][q] == INF:
                    rooms[p][q] = rooms[i][j] + 1
                    queue.append((p, q))

```

BFS · LeetCode - By Pattern

— 590 —

LeetCode

```

C++
C++

class Solution {
public:
    void wallsAndGates(vector<vector<int>>& rooms) {
        int n = rooms.size();
        int m = rooms[0].size();
        queue<pair<int, int>> q;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                if (rooms[i][j] == 0)
                    q.emplace(i, j);

        int d = 0;
        vector<int> dirs = {-1, 0, 1, 0, -1};
        while (!q.empty()) {
            int q;
            for (int i = q.size(); i > 0; --i) {
                auto p = q.front();
                q.pop();
                for (int j = 0; j < 4; ++j) {
                    int x = p.first + dirs[j];
                    int y = p.second + dirs[j] + 1;
                    if (x >= 0 && x < n && y >= 0 && y < n && rooms[x][y] == INF_MAX) {
                        rooms[x][y] = d;
                        q.emplace(x, y);
                    }
                }
            }
            ++d;
        }
    };
};

Java
Java

```

BFS · LeetCode - By Pattern

— 591 —

LeetCode

```

class Solution {
    public void wallsAndGates(int[][] rooms) {
        int n = rooms.length;
        int m = rooms[0].length;
        Deque<int[]> q = new LinkedList<>();
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j) {
                if (rooms[i][j] == 0) {
                    q.offer(new int[] { i, j });
                }
            }
        int d = 0;
        int[] dirs = {-1, 0, 1, 0, -1};
        while (!q.isEmpty()) {
            int[] p = q.poll();
            for (int j = 0; j < 4; ++j) {
                int x = p[0] + dirs[j];
                int y = p[1] + dirs[j] + 1;
                if (x >= 0 && x < n && y >= 0 && y < n && rooms[x][y] == Integer.MAX_VALUE) {
                    rooms[x][y] = d;
                    q.offer(new int[] { x, y });
                }
            }
        }
    }
}

Go
Go

```

BFS · LeetCode - By Pattern

— 592 —

LeetCode

```

func wallsAndGates(rooms []][int]) {
    m, n := len(rooms), len(rooms[0])
    var q []int
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if rooms[i][j] == 0 {
                q = append(q, [int]{i, j})
            }
        }
    }
    d := 0
    dirs := [int]{-1, 0, 1, 0, -1}
    for len(q) > 0 {
        d++
        for i := len(q); i > 0; i-- {
            p := q[i]
            q = q[:i]
            for j := 0; j < 4; j++ {
                x, y := p[0]+dirs[j], p[1]+dirs[j]+1
                if x >= 0 && x < m && y >= 0 && y < n && rooms[x][y] == math.MaxInt32 {
                    rooms[x][y] = d
                    q = append(q, [int]{x, y})
                }
            }
        }
    }
}

```

[*] BFS — Pattern Solution (matched from community answers)

```

Python

from collections import deque

def wallsAndGates(rooms):
    if not rooms or not rooms[0]:
        return

    rows, cols = len(rooms), len(rooms[0])
    # Directions: up, down, left, right
    directions = [(1, 0), (-1, 0), (0, -1), (0, 1)]
    queue = deque()

    # Enqueue all gates (cells with 0).
    for i in range(rows):
        for j in range(cols):
            if rooms[i][j] == 0:
                queue.append((i, j))

    # Perform walls-and-gates BFS.
    while queue:
        r, c = queue.popleft()
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Pruning: neighbor if it is within bounds and is an empty room.
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == 2147483647:
                rooms[nr][nc] = rooms[r][c] + 1 # update distance.

```

BFS · LeetCode - By Pattern

— 593 —

LeetCode

```

        queue.append((r, c))

#####

#322 MEDIUM
Coin Change
LeetCode · Simplex · WUJICE · LeetCode · Editorial
Array · Dynamic Programming · BFS
Like: Grind 169

Problem:
You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1

Example 2:
Input: coins = [2], amount = 3
Output: -1

Example 3:
Input: coins = [1], amount = 0
Output: 0

Constraints:
    • 1 <= coins.length <= 12
    • 1 <= coins[i] <= 231 - 1
    • 0 <= amount <= 104

>> Brute Force [BFS] Interview Prep

Python

# Brute Force Approach
class Solution:
    def coinChange(self, coins, amount):
        # Brute Force: Diamond - O*(amount/min) - recursion without memo
        def dfs(rem):
            if rem == 0: return 0
            if rem < 0: return float('inf')
            return 1 + min(dfs(rem - c) for c in coins)
        res = dfs(amount)
        return res if res != float('inf') else -1

Python
Python

```

BFS · LeetCode - By Pattern

— 594 —

LeetCode

```
def coinChange(coins, amount):
    # Create a dp array with amount+1 entries set to a value greater than any possible maximum number of coins
    dp = [amount + 1] * (amount + 1)
    # Base case: 0 coins are needed to form amount 0.
    dp[0] = 0

    # Build up the dp table.
    for current_amount in range(1, amount + 1):
        for coin in coins:
            if coin <= current_amount:
                dp[current_amount] = min(dp[current_amount], dp[current_amount - coin] + 1)

    # If dp[amount] remains greater than amount, no solution was found.
    return dp[amount] if dp[amount] <= amount + 1 else -1

Python
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        # dp[i] = the minimum number of coins to make up i
        dp = [float('inf')] * (amount + 1)

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)

        return -1 if dp[amount] == float('inf') else dp[amount]

Python
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        n = len(coins)
        f = [float('inf')] * (amount + 1)
        f[0] = 0
        for i, x in enumerate(coins, 1):
            for j in range(x, amount + 1):
                f[j] = min(f[j], f[j - x] + 1)
            if j == x:
                f[j] = min(f[j], 1)
        return -1 if f[amount] == float('inf') else f[amount]

Python
```

BFS · LeetCode · By Pattern

— 595 —

LeetCode

```
...
// float('inf')
let
// float('inf') + 1
let
...

class Solution {
    def coinChange(self, coins, amount):
        type coins: List[int]
        type amount: int
        type dp: List[int]

        dp = [float('inf')] * (amount + 1)
        dp[0] = 0
        for i in range(1, amount + 1):
            for coin in coins:
                if i <= coin:
                    dp[i] = min(dp[i], dp[i - coin] + 1)
            return dp[i-1] if dp[i-1] != float('inf') else -1

        // =====

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0
        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)
            return -1 if dp[i] == amount + 1 else dp[i]

C++
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        // dp[i] = the minimum number of coins to make up i
        vector<int> dp(amount + 1, amount + 1);
        dp[0] = 0;

        for (const int coin : coins)
            for (int i = coin; i <= amount; ++i)
                dp[i] = min(dp[i], dp[i - coin] + 1);

        return dp[amount] == amount + 1 ? -1 : dp[amount];
    }
};

C++
```

BFS · LeetCode · By Pattern

— 596 —

LeetCode

```
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        int n = coins.size(), m = amount;
        int f[n + 1][m + 1];
        memset(f, 0, sizeof(f));
        f[0][0] = 0;
        for (int i = 1; i <= m; ++i) {
            for (int j = 0; j <= n; ++j) {
                if (i <= coins[j - 1])
                    f[i][j] = min(f[i][j], f[i - coins[j - 1]][j] + 1);
            }
        }
        return f[n][m] > m ? -1 : f[n][m];
    }
};

C++
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        int n = amount;
        int f[n + 1];
        memset(f, 0, sizeof(f));
        f[0] = 0;
        for (int i = 0; i <= n; ++i) {
            for (int j = 0; j <= n; ++j) {
                f[j] = min(f[j], f[j - i] + 1);
            }
        }
        return f[n] > n ? -1 : f[n];
    }
};

Java
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        // dp[i] = the minimum number of coins to make up i
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;

        for (int coin : coins)
            for (int i = coin; i <= amount; ++i)
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);

        return dp[amount] == amount + 1 ? -1 : dp[amount];
    }
};

Java
```

BFS · LeetCode · By Pattern

— 597 —

LeetCode

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        final int INF = Integer.MAX_VALUE;
        int n = coins.length;
        int m = amount;
        int[] f = new int[n + 1][m + 1];
        for (int i = 0; i <= m; ++i)
            Arrays.fill(f[i], INF);
        f[0][0] = 0;
        for (int i = 1; i <= m; ++i) {
            for (int j = 0; j <= n; ++j) {
                f[i][j] = f[i - 1][j];
                if (i <= coins[j - 1])
                    f[i][j] = Math.min(f[i][j], f[i - coins[j - 1]][j] + 1);
            }
        }
        return f[n][m] > INF ? -1 : f[n][m];
    }
};

Java
class Solution {
    public int coinChange(int[] coins, int amount) {
        final int INF = Integer.MAX_VALUE;
        int n = amount;
        int[] f = new int[n + 1];
        Arrays.fill(f, INF);
        f[0] = 0;
        for (int i = 0; i <= n; ++i) {
            for (int j = 0; j <= n; ++j) {
                f[j] = Math.min(f[j], f[j - i] + 1);
            }
        }
        return f[n] > INF ? -1 : f[n];
    }
};

JavaScript
function coinChange(coins, amount) {
    // =====
    const n = coins.length;
    const m = amount;
    const f = new Array(n + 1).fill(Infinity);
    f[0][0] = 0;
    for (let i = 1; i <= m; ++i) {
        for (let j = 0; j <= n; ++j) {
            f[i][j] = f[i - 1][j];
            if (i <= coins[j - 1])
                f[i][j] = Math.min(f[i][j], f[i - coins[j - 1]][j] + 1);
        }
    }
    return f[n][m] > m ? -1 : f[n][m];
}

TypeScript
function coinChange(coins: number[], amount: number): number {
    const n = amount;
    const f: number[] = Array(n + 1).fill(Infinity);
    f[0] = 0;
    for (const x of coins) {
        for (let j = x; j <= n; ++j) {
            f[j] = Math.min(f[j], f[j - x] + 1);
        }
    }
    return f[n] > n ? -1 : f[n];
}

TypeScript
function coinChange(coins: number[], amount: number): number {
    const n = amount;
    const f: number[] = Array(n + 1).fill(Infinity);
    f[0] = 0;
    for (const x of coins) {
        for (let j = x; j <= n; ++j) {
            f[j] = Math.min(f[j], f[j - x] + 1);
        }
    }
    return f[n] > n ? -1 : f[n];
}

Go
func coinChange(coins []int, amount int) int {
    // =====
    n := amount
    f := make([]int, n+1)
    f[0] = 0
    for _, coin := range coins {
        for i := coin; i <= n; i++ {
            f[i] = min(f[i], f[i-coin]+1)
        }
    }
    return f[n] > n ? -1 : f[n]
}
```

BFS · LeetCode · By Pattern

— 598 —

LeetCode

```
// =====
// @param {number[]} coins
// @param {number} amount
// @return {number}

var coinChange = function(coins, amount) {
    const n = coins.length;
    const m = amount;
    const f = Array(n + 1).fill(Infinity);
    f[0][0] = 0;
    for (let i = 1; i <= m; ++i) {
        for (let j = 0; j <= n; ++j) {
            f[i][j] = f[i - 1][j];
            if (i <= coins[j - 1])
                f[i][j] = Math.min(f[i][j], f[i - coins[j - 1]][j] + 1);
        }
    }
    return f[n][m] > m ? -1 : f[n][m];
};

JavaScript
// =====
// @param {number[]} coins
// @param {number} amount
// @return {number}

var coinChange = function(coins, amount) {
    const n = amount;
    const f = Array(n + 1).fill(Infinity);
    f[0] = 0;
    for (const x of coins) {
        for (let j = x; j <= n; ++j) {
            f[j] = Math.min(f[j], f[j - x] + 1);
        }
    }
    return f[n] > n ? -1 : f[n];
};

TypeScript
function coinChange(coins: number[], amount: number): number {
    // =====
    const n = amount;
    const f: number[] = Array(n + 1).fill(Infinity);
    f[0] = 0;
    for (const x of coins) {
        for (let j = x; j <= n; ++j) {
            f[j] = Math.min(f[j], f[j - x] + 1);
        }
    }
    return f[n] > n ? -1 : f[n];
}

TypeScript
function coinChange(coins: number[], amount: number): number {
    // =====
    const n = amount;
    const f: number[] = Array(n + 1).fill(Infinity);
    f[0] = 0;
    for (const x of coins) {
        for (let j = x; j <= n; ++j) {
            f[j] = Math.min(f[j], f[j - x] + 1);
        }
    }
    return f[n] > n ? -1 : f[n];
}

Go
func coinChange(coins []int, amount int) int {
    // =====
    n := amount
    f := make([]int, n+1)
    f[0] = 0
    for _, coin := range coins {
        for i := coin; i <= n; i++ {
            f[i] = min(f[i], f[i-coin]+1)
        }
    }
    return f[n] > n ? -1 : f[n]
}
```

BFS · LeetCode · By Pattern

— 599 —

LeetCode

```
function coinChange(coins: number[], amount: number): number {
    // =====
    const n = coins.length;
    const m = amount;
    const f: number[] = Array(n + 1).fill(Infinity);
    f[0][0] = 0;
    for (let i = 1; i <= m; ++i) {
        for (let j = 0; j <= n; ++j) {
            f[i][j] = f[i - 1][j];
            if (i <= coins[j - 1])
                f[i][j] = Math.min(f[i][j], f[i - coins[j - 1]][j] + 1);
        }
    }
    return f[n][m] > m ? -1 : f[n][m];
}

TypeScript
function coinChange(coins: number[], amount: number): number {
    const n = amount;
    const f: number[] = Array(n + 1).fill(Infinity);
    f[0] = 0;
    for (const x of coins) {
        for (let j = x; j <= n; ++j) {
            f[j] = Math.min(f[j], f[j - x] + 1);
        }
    }
    return f[n] > n ? -1 : f[n];
}

TypeScript
function coinChange(coins: number[], amount: number): number {
    const n = amount;
    const f: number[] = Array(n + 1).fill(Infinity);
    f[0] = 0;
    for (const x of coins) {
        for (let j = x; j <= n; ++j) {
            f[j] = Math.min(f[j], f[j - x] + 1);
        }
    }
    return f[n] > n ? -1 : f[n];
}

Go
func coinChange(coins []int, amount int) int {
    // =====
    n := amount
    f := make([]int, n+1)
    f[0] = 0
    for _, coin := range coins {
        for i := coin; i <= n; i++ {
            f[i] = min(f[i], f[i-coin]+1)
        }
    }
    return f[n] > n ? -1 : f[n]
}
```

BFS · LeetCode · By Pattern

— 600 —

LeetCode


```

func coinChange(coins []int, amount int) int {
    n, m := len(coins), amount
    f := make([]int, m+1)
    const inf = 1e9
    for i := range f {
        f[i] = make([]int, n)
        for j := range f[i] {
            f[i][j] = inf
        }
    }
    f[0][0] = 0
    for i := 1; i <= m; i++ {
        for j := 0; j <= n; j++ {
            f[i][j] = f[i-1][j]
            if j >= coins[i-1] {
                f[i][j] = min(f[i][j], f[i][j-coins[i-1]]+1)
            }
        }
    }
    if f[m][n] > n {
        return -1
    }
    return f[m][n]
}

```

Go

```

func coinChange(coins []int, amount int) int {
    n := amount
    f := make([]int, n+1)
    for i := range f {
        f[i] = 1e9
    }
    f[0] = 0
    for i := 0; i <= n; i++ {
        for j := 0; j <= n; j++ {
            f[i][j] = min(f[i][j], f[i][j-coins[i]]+1)
        }
    }
    if f[n][n] > n {
        return -1
    }
    return f[n][n]
}

```

Go

```

func coinChange(coins []int, amount int) int {
    n := amount
    f := make([]int, n+1)
    for i := range f {
        f[i] = 1e9
    }
    f[0] = 0
    for i := 0; i <= n; i++ {
        for j := 0; j <= n; j++ {
            f[i][j] = min(f[i][j], f[i][j-coins[i]]+1)
        }
    }
    if f[n][n] > n {
        return -1
    }
    return f[n][n]
}

```

Rust

```

impl Solution {
    pub fn coin_change(coins: Vec<i32>, amount: i32) -> i32 {
        let n = amount as usize;
        let mut f = vec![n+1; n+1];
        f[0] = 0;
        for i in coins {
            for j in 0..n {
                f[j+i] = min(f[j] + 1, f[j+i]);
            }
        }
        if f[n] > n {
            -1
        } else {
            f[n] as i32
        }
    }
}

```

Rust

```

impl Solution {
    pub fn coin_change(coins: Vec<i32>, amount: i32) -> i32 {
        let n = amount as usize;
        let mut f = vec![n+1; n+1];
        f[0] = 0;
        for i in coins {
            for j in 0..n {
                f[j+i] = min(f[j] + 1, f[j+i]);
            }
        }
        if f[n] > n {
            -1
        } else {
            f[n] as i32
        }
    }
}

```

[*] BFS — Pattern Solution (stored optimal)

Python

```

def coinChange(coins, amount):
    # Create a dp array with amount+1 entries set to a value greater than any possible minimum number of coins
    dp = [amount + 1] * (amount + 1)
    # Base case: 0 coins needed to form amount 0
    dp[0] = 0

    # Build up the dp table
    for current_amount in range(1, amount + 1):
        for coin in coins:
            if coin <= current_amount:
                dp[current_amount] = min(dp[current_amount], dp[current_amount - coin] + 1)

    # If dp[amount] remains greater than amount, no solution was found
    return dp[amount] if dp[amount] <= amount + 1 else -1

```

C++

```

// Include headers
// Include algorithm
using namespace std;

int coinChange(vector<int>& coins, int amount) {
    // dp vector initialized to a large number (amount+1) which acts as infinity
    vector<int> dp(amount + 1, amount + 1);
    dp[0] = 0; // Base case: 0 coins to form amount 0

    // Iterate over every element from 1 to amount
    for (int current = 1; current <= amount; current++) {
        for (int coin : coins) {
            if (coin <= current) {
                dp[current] = min(dp[current], dp[current - coin] + 1);
            }
        }
    }
    return dp[amount] <= amount ? dp[amount] : -1;
}

```

#542 MEDIUM
01 Matrix
LeetCode · Simple · Walk · LeetCode · Editorial
Array · BFS · Matrix
Like Grind 169 | Denny Zhang

Problem:
Given an $m \times n$ binary matrix `mat`, return the distance of the nearest 0 for each cell.
The distance between two cells sharing a common edge is 1.
Example 1:

0	0	0
0	1	0
0	0	0

Input: `mat = [[0,0,0],[0,1,0],[0,0,0]]`
Output: `[[0,0,0],[0,1,0],[0,0,0]]`

Example 2:

0	0	0
0	1	0
1	1	1

Input: `mat = [[0,0,0],[0,1,0],[1,1,1]]`
Output: `[[0,0,0],[0,1,0],[1,2,1]]`

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 104$

- $1 \leq m \times n \leq 104$
 - $\text{mat}[i][j]$ is either 0 or 1.
 - There is at least one 0 in `mat`.
- Note: This question is the same as 1765: <https://leetcode.com/problems/map-of-highest-peak/>

>> Brute Force [BFS] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def updateMatrix(self, mat):
        # Create queue and layer by layer
        from collections import deque
        queue = deque([0])
        visited = [0]
        steps = 0
        while queue:
            for i in range(len(queue)):
                node = queue.popleft()
                for d in range(4):
                    if node not in visited:
                        visited.add(node)
                        queue.append(node)
            steps += 1
            return steps

```

Python

```

from collections import deque

def updateMatrix(mat):
    # Get the dimensions of the matrix
    rows, cols = len(mat), len(mat[0])
    # Initialize a queue for multi-source BFS
    queue = deque()

    # Initialize the matrix: use -1 to indicate unvisited cells
    for r in range(rows):
        for c in range(cols):
            if mat[r][c] == 0:
                # Enqueue all 0 cells as starting points
                queue.append((r, c))
            else:
                # Mark 1 cells as unvisited (or infinite distance)
                mat[r][c] = -1

    # Directions for neighbors (up, down, left, right)
    directions = [(0, 1), (-1, 0), (0, -1), (1, 0), (0, 0)]

    # BFS traversal to update distances
    while queue:
        r, c = queue.popleft()
        for dr, dc in directions:

```

```

# BFS Solution
def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
    rows, cols = len(mat), len(mat[0])
    queue = deque()
    visited = [[False] * cols for _ in range(rows)]

    # Enqueue all 0 cells as starting points
    for r in range(rows):
        for c in range(cols):
            if mat[r][c] == 0:
                queue.append((r, c))
                visited[r][c] = True

    # BFS traversal to update distances
    while queue:
        r, c = queue.popleft()
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and not visited[nr][nc]:
                # Update neighbor's distance
                mat[nr][nc] = mat[r][c] + 1
                queue.append((nr, nc))

    return mat

```

Python

```

class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        rows, cols = len(mat), len(mat[0])
        queue = deque()
        visited = [[False] * cols for _ in range(rows)]

        # Enqueue all 0 cells as starting points
        for r in range(rows):
            for c in range(cols):
                if mat[r][c] == 0:
                    queue.append((r, c))
                    visited[r][c] = True

        # BFS traversal to update distances
        while queue:
            r, c = queue.popleft()
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and not visited[nr][nc]:
                    # Update neighbor's distance
                    mat[nr][nc] = mat[r][c] + 1
                    queue.append((nr, nc))

        return mat

```

```
class Solution {
    def updateMatrix(mat: List[List[Int]]) -> List[List[Int]] {
        m = mat.length, lenmat[0]
        ans = List.fill(m for _ in range(m))
        q = deque()
        for i, row in enumerate(mat):
            for j, v in enumerate(row):
                if v == 0:
                    ans[i][j] = 0
                    q.append(i, j)
        dirs = (-1, 0, 1, 0, -1)
        while q:
            i, j = q.popleft()
            for a, b in pairsInDirs(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
                    ans[x][y] = ans[i][j] + 1
                    q.append(x, y)
        return ans
    }
}

Python
class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(mat):
            for j, v in enumerate(row):
                if v == 0:
                    ans[i][j] = 0
                    q.append(i, j)
        dirs = (-1, 0, 1, 0, -1)
        while q:
            i, j = q.popleft()
            for a, b in pairsInDirs(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
                    ans[x][y] = ans[i][j] + 1
                    q.append(x, y)
        return ans

C++
C++
```

```
class Solution {
    public:
        vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
            int m = mat.size(), n = mat[0].size();
            vector<vector<int>> ans(m, vector<int>(n, -1));
            queue<pair<int, int>> q;
            for (int i = 0; i < m; ++i) {
                for (int j = 0; j < n; ++j) {
                    if (mat[i][j] == 0) {
                        ans[i][j] = 0;
                        q.emplace(i, j);
                    }
                }
            }
            vector<int> dirs = {-1, 0, 1, 0, -1};
            while (!q.empty()) {
                auto [i, j] = q.front();
                q.pop();
                for (int i = 0; i < 4; ++i) {
                    int x = i.first + dirs[i];
                    int y = i.second + dirs[i + 1];
                    if (x < 0 || x > m || y < 0 || y > n || ans[x][y] == -1) {
                        ans[x][y] = ans[i.first][i.second] + 1;
                        q.emplace(x, y);
                    }
                }
            }
            return ans;
        }
};

Java
Java
```

```
class Solution {
    public int[][] updateMatrix(int[][] mat) {
        int m = mat.length, n = mat[0].length;
        int[][] ans = new int[m][n];
        for (int[] row : ans) {
            Arrays.fill(row, -1);
        }
        Deque<int[]> q = new ArrayDeque<>();
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (mat[i][j] == 0) {
                    q.offer(new int[] {i, j});
                    ans[i][j] = 0;
                }
            }
        }
        int[] dirs = {-1, 0, 1, 0, -1};
        while (!q.isEmpty()) {
            int[] p = q.poll();
            int i = p[0], j = p[1];
            for (int k = 0; k < 4; ++k) {
                int x = i + dirs[k], y = j + dirs[k + 1];
                if (x < 0 || x > m || y < 0 || y > n || ans[x][y] == -1) {
                    ans[x][y] = ans[i][j] + 1;
                    q.offer(new int[] {x, y});
                }
            }
        }
        return ans;
    }
}

TypeScript
TypeScript
```

```
function updateMatrix(mat: number[][]): number[][] {
    const [m, n] = [mat.length, mat[0].length];
    const ans: number[][] = Array.from({ length: m }, () => Array.from({ length: n }, () => -1));
    const q: [number, number][] = [];
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (mat[i][j] === 0) {
                q.push([i, j]);
                ans[i][j] = 0;
            }
        }
    }
    const dirs: number[][] = [-1, 0, 1, 0, -1];
    while (q.length) {
        const [i, j] = q.shift()!;
        for (let k = 0; k < 4; ++k) {
            const [x, y] = [i + dirs[k], j + dirs[k + 1]];
            if (x < 0 || x > m || y < 0 || y > n || ans[x][y] === -1) {
                ans[x][y] = ans[i][j] + 1;
                q.push([x, y]);
            }
        }
    }
    return ans;
}

Go
Go
func updateMatrix(mat [][]int) [][]int {
    m := len(mat), lenmat[0]
    ans := make([][]int, m)
    for i := range ans {
        ans[i] = make([]int, n)
        for j := range ans[i] {
            ans[i][j] = -1
        }
    }
    type pair struct { x, y int }
    var q []pair
    for i, row := range mat {
        for j, v := range row {
            if v == 0 {
                ans[i][j] = 0
                q = append(q, pair{i, j})
            }
        }
    }
    dirs := [][]int{-1, 0, 1, 0, -1}
    for len(q) > 0 {
        p := q[0]
        q = q[1:]
        for i := 0; i < 4; i++ {
            x, y := p.x+dirs[i], p.y+dirs[i+1]
        }
    }
}
```

```
if (x < 0 || x > m || y < 0 || y > n || ans[x][y] == -1) {
    ans[x][y] = ans[i][j] + 1;
    q = append(q, pair{x, y})
}
}
return ans
}

Rust
Rust
use std::collections::VecDeque;

impl Solution {
    fn updateMatrix(mat: Vec<Vec<i32>> -> Vec<Vec<i32>> {
        let m = mat.len();
        let n = mat[0].len();
        let mut ret_vec: Vec<Vec<i32>> = vec![vec![0; n]; m];
        // The inner matrix is all -1, it's default value.
        let mut the_q: VecDeque<(usize, usize)> = VecDeque::new();
        let traversed_vec: Vec<(i32, i32)> = vec![(0, 0), (0, 1), (0, 2), (0, 3)];

        // Initialize the queue
        for i in 0..m {
            for j in 0..n {
                if mat[i][j] == 0 {
                    // For the same cell, enqueue at first
                    the_q.push_back((i, j));
                    // Set to 0 in return vector
                    ret_vec[i][j] = 0;
                }
            }
        }
        while !the_q.is_empty() {
            let (i, j) = the_q.pop_front().unwrap().clone();
            the_q.push_back((i, j));
            for pair in directions_vec {
                let cur_x = pair.0 + (i as i32);
                let cur_y = pair.1 + (j as i32);
                if
                    Solution::check_bound(cur_x, cur_y, m as i32, n as i32) &&
                    ret_vec[cur_x as usize][cur_y as usize] == -1
                {
                    // The current cell has not been updated yet, and it also in bound
                    ret_vec[cur_x as usize][cur_y as usize] = ret_vec[i][j] + 1;
                    the_q.push_back((cur_x as usize, cur_y as usize));
                }
            }
        }
        ret_vec
    }
}

fn check_bound(x: i32, y: i32, m: i32, n: i32) -> bool {
    x < 0 || x > m || y < 0 || y > n
}

[i] BFS - Pattern Solution (matched from community answers)
Python
from collections import deque

def updateMatrix(mat):
    # Get the dimensions of the matrix
    rows, cols = len(mat), len(mat[0])
    # Initialize a queue for multi-source BFS
    queue = deque()

    # Initialize the matrix with -1 to indicate unvisited cells
    for r in range(rows):
        for c in range(cols):
            if mat[r][c] == 0:
                # Enqueue all 0 cells as starting points
                queue.append((r, c))
            else:
                # Mark 1 cells as unvisited (or infinite distance)
                mat[r][c] = -1

    # Directions for neighbors (up, down, left, right)
    directions = [(0, 1), (-1, 0), (0, -1), (1, 0)]

    # BFS traversal to update distances
    while queue:
        r, c = queue.popleft()
        for dr, dc in directions:
```

```
let (x, y) = the_q.pop().unwrap().clone();
the_q.push_back((x, y));
for pair in directions_vec {
    let cur_x = pair.0 + (x as i32);
    let cur_y = pair.1 + (y as i32);
    if
        Solution::check_bound(cur_x, cur_y, m as i32, n as i32) &&
        ret_vec[cur_x as usize][cur_y as usize] == -1
    {
        // The current cell has not been updated yet, and it also in bound
        ret_vec[cur_x as usize][cur_y as usize] = ret_vec[i][j] + 1;
        the_q.push_back((cur_x as usize, cur_y as usize));
    }
}
ret_vec
}

fn check_bound(x: i32, y: i32, m: i32, n: i32) -> bool {
    x < 0 || x > m || y < 0 || y > n
}

[i] BFS - Pattern Solution (matched from community answers)
Python
from collections import deque

def updateMatrix(mat):
    # Get the dimensions of the matrix
    rows, cols = len(mat), len(mat[0])
    # Initialize a queue for multi-source BFS
    queue = deque()

    # Initialize the matrix with -1 to indicate unvisited cells
    for r in range(rows):
        for c in range(cols):
            if mat[r][c] == 0:
                # Enqueue all 0 cells as starting points
                queue.append((r, c))
            else:
                # Mark 1 cells as unvisited (or infinite distance)
                mat[r][c] = -1

    # Directions for neighbors (up, down, left, right)
    directions = [(0, 1), (-1, 0), (0, -1), (1, 0)]

    # BFS traversal to update distances
    while queue:
        r, c = queue.popleft()
        for dr, dc in directions:
```

```

    nr, nc = r + dr, c + dc
    # Check boundaries and if neighbor has not been visited
    if 0 <= nr < rows and 0 <= nc < cols and not visited[nr][nc] == -1:
        # Update neighbor's distance
        nextDist[nr][nc] = dist[r][c] + 1
        # Enqueue the neighbor for further BFS processing
        queue.append((nr, nc))

    return dist

# Example usage:
# grid = [[0,0,0,0],[0,1,0],[0,1,0],[1,1,1]]

```

#994

MEDIUM

Rotting Oranges

LeetCode · SimplifyLab · WalkCC · LeetCode · Editorial

Array · BFS · Matrix

Like Grid 109

Problem:

You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1:



Input: grid = [[2,1,1],[1,1,0],[0,1,1]]

Output: 4

Example 2:

Input: grid = [[2,1,1],[0,1,1],[1,0,1]]

Output: -1

Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: grid = [[0,2]]

Output: 0

BFS · LeetCode · By Pattern

— 613 —

LeetCode

Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid[0].length}$
- $1 \leq m, n \leq 10$
- $\text{grid}[i][j]$ is 0, 1, or 2.

>> Brute Force (BFS) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def orangesRotting(self, grid):
        # Brute Force O(m*n^2) - repeatedly scan entire grid, spread rot each pass
        from copy import deepcopy
        m, n = len(grid), len(grid[0])
        minutes = 0
        while True:
            new_grid = deepcopy(grid)
            changed = False
            for r in range(m):
                for c in range(n):
                    if grid[r][c] == 2:
                        for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
                            nr,nc = r+dr,c+dc
                            if 0<=nr and 0<=nc and grid[nr][nc]==1:
                                new_grid[nr][nc]=2; changed=True
            grid = new_grid
            if not changed: break
            minutes += 1
            return minutes if not any(1 in row for row in grid) else -1

```

Python

Python

BFS · LeetCode · By Pattern

— 614 —

LeetCode

```

from collections import deque

def orangesRotting(grid):
    # Dimensions of the grid
    rows, cols = len(grid), len(grid[0])
    # Queue to perform BFS, containing tuples of (row, col)
    rotten_queue = deque()
    fresh_count = 0

    # Step 1: Initialize the queue with all initially rotten oranges and count fresh oranges.
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                rotten_queue.append((r, c))
            elif grid[r][c] == 1:
                fresh_count += 1

    # If there are no fresh oranges, return 0 immediately.
    if fresh_count == 0:
        return 0

    minutes_passed = 0
    # Directions for adjacent cells: up, down, left, right.
    directions = [(1,0), (1,0), (0,-1), (0,1)]

    # Step 2: BFS to rot adjacent fresh oranges.
    while rotten_queue:
        # Process oranges that are rotten in the current minute.
        for _ in range(len(rotten_queue)):
            x, y = rotten_queue.popleft()
            for dr, dc in directions:
                nx, ny = x + dr, y + dc
                # Check if the neighbor is inside the grid and is a fresh orange.
                if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1:
                    # Mark the fresh orange as rotten.
                    grid[nx][ny] = 2
                    fresh_count -= 1
                    rotten_queue.append((nx, ny))
            # Increment minutes after processing one level.
            minutes_passed += 1

    # If there are still fresh oranges left, return -1.
    return minutes_passed - 1 if fresh_count == 0 else -1

# Example usage:
# grid = [[2,1,1],[1,1,0],[0,1,1]]
# print(orangesRotting(grid))

```

Python

BFS · LeetCode · By Pattern

— 615 —

LeetCode

```

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        cnt = 0
        for i, row in enumerate(grid):
            for j, v in enumerate(row):
                if v == 2:
                    q.append((i, j))
                elif v == 1:
                    cnt += 1
        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        while q and cnt:
            ans += 1
            for i, j in range(len(q)):
                i, j = q.popleft()
                for A, B in dirs:
                    x, y = i + A, j + B
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        grid[x][y] = 2
                        q.append((x, y))
                        cnt -= 1
            if cnt == 0:
                return ans
        return -1 if cnt else 0

```

Python

```

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        cnt = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    q.append((i, j))
                elif grid[i][j] == 1:
                    cnt += 1
        ans = 0
        while q and cnt:
            ans += 1
            for i, j in range(len(q)):
                i, j = q.popleft()
                for A, B in [(1, 0), (1, 0), (1, 0), (-1, 0)]:
                    x, y = i + A, j + B
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        cnt -= 1
                        grid[x][y] = 2
                        q.append((x, y))
        return ans if cnt == 0 else -1

```

C++

C++

BFS · LeetCode · By Pattern

— 616 —

LeetCode

```

class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        constexpr int dirs[4][2] = {{0, 1}, {0, -1}, {-1, 0}, {1, 0}};
        int m = grid.size();
        int n = grid[0].size();

        auto isHighOrRotten = [&](int i, int j, const vector<vector<int>>& grid) {
            for (const auto& [dx, dy] : dirs) {
                const int r = i + dx;
                const int c = j + dy;
                if (r < 0 || r >= m || c < 0 || c >= n)
                    continue;
                if (grid[r][c] == 2)
                    return true;
            }
            return false;
        };

        int ans = 0;

        while (true) {
            vector<vector<int>> nextGrid(m, vector<int>(n));
            // Calculate number of rotten oranges in next grid.
            for (int i = 0; i < m; ++i)
                for (int j = 0; j < n; ++j)
                    if (grid[i][j] == 2) { // Fresh
                        if (isHighOrRotten(i, j, grid))
                            nextGrid[i][j] = 2;
                    } else if (grid[i][j] == 1) { // Rotten
                        nextGrid[i][j] = 1;
                    } else if (grid[i][j] == 0) { // Empty
                        nextGrid[i][j] = 0;
                    }
                if (nextGrid[i] == grid)
                    break;
            grid = nextGrid;
            ++ans;
        }

        return any_of(grid.begin(), grid.end(),
            [&](const vector<int>& row) {
                return ranges::any_of(row, [&](int orange) { return orange == 1; });
            }) ?
            -1 : ans;
    }
};

```

C++

BFS · LeetCode · By Pattern

— 617 —

LeetCode

```

class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        int m = grid.size(), n = grid[0].size();
        int cnt = 0;
        typedef pair<int, int> pii;
        queue<pii> q;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (grid[i][j] == 2)
                    q.emplace(i, j);
                else if (grid[i][j] == 1)
                    ++cnt;
            }
        int ans = 0;
        vector<int> dirs = {-1, 0, 1, 0, -1};
        while (!q.empty() && cnt > 0) {
            ++ans;
            for (int i = q.size(); i > 0; --i) {
                auto p = q.front();
                q.pop();
                for (int j = 0; j < 4; ++j) {
                    int x = p.first + dirs[j];
                    int y = p.second + dirs[j] + 1;
                    if (x <= 0 && x >= m && y <= 0 && y >= n && grid[x][y] == 1) {
                        --cnt;
                        grid[x][y] = 2;
                        q.emplace(x, y);
                    }
                }
            }
            return cnt > 0 ? -1 : ans;
        }
    }
};

```

Java

Java

BFS · LeetCode · By Pattern

— 618 —

LeetCode

```

class Solution {
public: int orangesRotting(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int cnt = 0;
    queue<int> q = new LinkedList<>();
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            if (grid[i][j] == 2) {
                q.offer(new int[] {i, j});
            } else if (grid[i][j] == 1) {
                ++cnt;
            }
        }
    }
    int ans = 0;
    int[] dir = {1, 0, -1, 0, 1};
    while (!q.isEmpty() && cnt > 0) {
        ++ans;
        for (int i = q.size(), i > 0; --i) {
            int[] p = q.poll();
            for (int j = 0; j < 4; ++j) {
                int x = p[0] + dir[j];
                int y = p[1] + dir[j] + 1;
                if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] == 1) {
                    grid[x][y] = 2;
                }
            }
            q.offer(new int[] {x, y});
        }
    }
    return cnt > 0 ? -1 : ans;
}
}

```

JavaScript

JavaScript

BFS · LeetCode · By Pattern

— 619 —

LeetCode

```

/**
 * @param {number[][]} grid
 * @return {number}
 */
var orangesRotting = function (grid) {
    const m = grid.length;
    const n = grid[0].length;
    let cnt = 1;
    let rot = 0;
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (grid[i][j] === 1) {
                ++cnt;
            } else if (grid[i][j] === 2) {
                q.push([i, j]);
            }
        }
    }
    const dir = [-1, 0, 1, 0, -1];
    for (let ans = 1; q.length && cnt > ans) {
        let t = [];
        for (const [i, j] of q) {
            for (let d = 0; d < 4; ++d) {
                const x = i + dir[d];
                const y = j + dir[d] + 1;
                if (x >= 0 && x < n && y >= 0 && y < n && grid[x][y] === 1) {
                    grid[x][y] = 2;
                    t.push([x, y]);
                    if (--cnt === 0) {
                        return ans;
                    }
                }
            }
        }
        q = [...t];
    }
    return cnt > 0 ? -1 : 0;
};

```

TypeScript

TypeScript

BFS · LeetCode · By Pattern

— 620 —

LeetCode

```

function orangesRotting(grid: number[][]): number {
    const m = grid.length;
    const n = grid[0].length;
    let count = 0;
    const queue = [];
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            if (grid[i][j] === 2) {
                count++;
            } else if (grid[i][j] === 1) {
                queue.push([i, j]);
            }
        }
    }
    let res = 0;
    const dir = [1, 0, -1, 0, 1];
    while (count !== 0 && queue.length !== 0) {
        for (let i = queue.length; i > 0; i--) {
            const [x, y] = queue.shift();
            for (let j = 0; j < 4; j++) {
                const next = x + dir[j];
                const next = y + dir[j] + 1;
                if (next >= 0 && next < m && next >= 0 && grid[next][next] === 1) {
                    grid[next][next] = 2;
                    queue.push([next, next]);
                }
            }
            count--;
        }
    }
    return res;
}

```

Go

Go

BFS · LeetCode · By Pattern

— 621 —

LeetCode

```

func orangesRotting(grid [][]int) int {
    m, n := len(grid), len(grid[0])
    cnt := 0
    var q [][]int
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if grid[i][j] == 2 {
                q = append(q, []int{i, j})
            } else if grid[i][j] == 1 {
                cnt++
            }
        }
    }
    ans := 0
    dir = [][]int{-1, 0, 1, 0, -1}
    for len(q) > 0 && cnt > 0 {
        ans++
        for i := len(q); i > 0; i-- {
            p := q[0]
            q = q[1:]
            for j := 0; j < 4; j++ {
                x, y := p[0]+dir[j], p[1]+dir[j]+1
                if x >= 0 && x < m && y >= 0 && y < n && grid[x][y] == 1 {
                    cnt--
                    grid[x][y] = 2
                }
            }
            q = append(q, []int{x, y})
        }
    }
    if cnt > 0 {
        return -1
    }
    return ans
}

```

Rust

Rust

BFS · LeetCode · By Pattern

— 622 —

LeetCode

```

use std::collections::VecDeque;

impl Solution {
    pub fn oranges_rotting(grid: Vec<Vec<i32>>) -> i32 {
        let mut queue = VecDeque::new();
        let m = grid.len();
        let n = grid[0].len();
        // Count
        let mut count = 0;
        for i in 0..m {
            for j in 0..n {
                match grid[i][j] {
                    1 => {
                        count += 1;
                    }
                    2 => queue.push_back((i as i32, j as i32)),
                    _ => {},
                }
            }
        }
        let mut res = 0;
        let dir = [1, 0, -1, 0, 1];
        while count != 0 && queue.len() != 0 {
            let mut len = queue.len();
            while len != 0 {
                let (x, y) = queue.pop_front().unwrap();
                for i in 0..4 {
                    let new_x = x + dir[i];
                    let new_y = y + dir[i] + 1;
                    if
                        new_x >= 0 &&
                        new_x < (m as i32) &&
                        new_y >= 0 &&
                        new_y < (n as i32) &&
                        grid[new_x as usize][new_y as usize] == 1
                    {
                        grid[new_x as usize][new_y as usize] = 2;
                        queue.push_back((new_x, new_y));
                        count -= 1;
                    }
                }
                len -= 1;
            }
            res += 1;
        }
        if count != 0 {
            return -1;
        }
        res
    }
}

```

[*] BFS — Pattern Solution (matched from community answers)

Python

BFS · LeetCode · By Pattern

— 623 —

LeetCode

```

from collections import deque

def orangesRotting(grid):
    # Dimensions of the grid
    rows, cols = len(grid), len(grid[0])
    # Queue to perform BFS, containing tuples of (row, col)
    rotten_queue = deque()
    fresh_count = 0

    # Step 1: Initialize the queue with all initially rotten oranges and count fresh oranges.
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                rotten_queue.append((r, c))
            elif grid[r][c] == 1:
                fresh_count += 1

    # If there are no fresh oranges, return 0 immediately.
    if fresh_count == 0:
        return 0

    minutes_passed = 0
    # Directions for adjacent cells: up, down, left, right.
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    # Step 2: BFS to rot adjacent fresh oranges.
    while rotten_queue:
        # Process oranges that are rotten in the current minute.
        for _ in range(len(rotten_queue)):
            x, y = rotten_queue.popleft()
            for dx, dy in directions:
                nx, ny = x + dx, y + dy
                # Check if the neighbor is inside the grid and is a fresh orange.
                if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1:
                    # Rot the fresh orange as rotten.
                    grid[nx][ny] = 2
                    fresh_count -= 1
                # Add the new rotten orange into the queue.
                rotten_queue.append((nx, ny))

    # Increment minutes after processing one level.
    minutes_passed += 1

    # If queue is empty (fresh oranges left), return -1.
    return minutes_passed - 1 if fresh_count == 0 else -1

# Example usage:
# grid = [[2,1,1],[1,1,0],[0,1,1]]
# print(orangesRotting(grid))

```

#1197

MEDIUM

Minimum Knight Moves

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

BFS

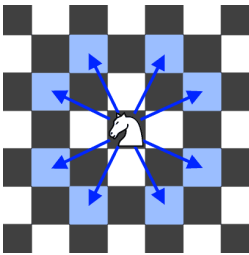
LeetCode 149 | Premium 98

BFS · LeetCode · By Pattern

— 624 —

LeetCode

Problem:
In an infinite chess board with coordinates from $-\infty$ to ∞ , you have a knight at square $(0, 0)$.
A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return the minimum number of steps needed to move the knight to the square $[x, y]$. It is guaranteed the answer exists.

Example 1:

Input: $x = 2, y = 1$
Output: 1
Explanation: $[0, 0] \rightarrow [2, 1]$

Example 2:

Input: $x = 5, y = 5$
Output: 4
Explanation: $[0, 0] \rightarrow [2, 1] \rightarrow [4, 2] \rightarrow [3, 4] \rightarrow [5, 5]$

Constraints:

$-300 \leq x, y \leq 300$
 $-10 \leq |x| + |y| \leq 300$

>> Brute Force [BFS] Interview Prep
Python

```
# Brute Force Approach
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        # Brute Force - BFS Layer by Layer
        from collections import deque
        queue = deque([(0, 0)])
        visited = set()
        steps = 0
        while queue:
            for _ in range(len(queue)):
                node = queue.popleft()
                for dx, dy in graph(node):
                    if node not in visited:
                        visited.add(node); queue.append(node)
            steps += 1
        return steps

# Python
Python

# Python Solution using BFS
from collections import deque

def minKnightMoves(x, y):
    # Normalize target coordinates using symmetry
    x, y = abs(x), abs(y)

    # Possible knight moves (dx, dy)
    moves = [(2, 1), (1, 2), (-1, 2), (-2, 1), (-2, -1), (-1, -2), (1, -2), (2, -1)]

    # BFS queue stores tuples of the form (current_x, current_y, move_count)
    queue = deque([(0, 0, 0)])

    # Set to track visited positions to prevent cycles
    visited = set()

    # Process BFS
    while queue:
        curr_x, curr_y, move_count = queue.popleft()

        # Return the current move count if target is reached
        if curr_x == x and curr_y == y:
            return move_count

        # Explore all knight moves
        for dx, dy in moves:
            next_x, next_y = curr_x + dx, curr_y + dy

            # Prune search space only process positions with valid bounds
            if (next_x, next_y) not in visited and next_x >= -1 and next_y >= -1:
                visited.add((next_x, next_y))
                queue.append((next_x, next_y, move_count + 1))

    return -1 # Unreachable because a solution always exists

# Example test
print(minKnightMoves(1, 1))
```

```
Python

class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        q = deque([(0, 0)])
        ans = 0
        vis = {(0, 0)}
        dirs = [(-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2), (-2, -1)]
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if (i, j) == (x, y):
                    return ans
                for a, b in dirs:
                    c, d = i + a, j + b
                    if (c, d) not in vis:
                        vis.add((c, d))
                        q.append((c, d))
            ans += 1
        return -1

Python

class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        def extend(m1, m2, d):
            for _ in range(len(d)):
                i, j = m1.popleft()
                step = m2[(i, j)]
                for a, b in d:
                    c, d = i + a, j + b
                    if (c, d) in m1:
                        continue
                    if (c, d) in m2:
                        return step + 1 + m2[(c, d)]
                    m1.add((c, d))
                    m2[(c, d)] = step + 1
            return -1

        if (x, y) == (0, 0):
            return 0
        q1, q2 = deque([(0, 0)]), deque([(x, y)])
        m1, m2 = {(0, 0)}, {(x, y): 0}
        while q1 and q2:
            t = extend(m1, m2, q1) if len(q1) < len(q2) else extend(m2, m1, q2)
            if t != -1:
                return t

Python

class Solution {
    public:
        int minKnightMoves(int x, int y) {
            x = abs(x);
            y = abs(y);
            int ans = 0;
            queue<pair<int, int>> q;
            q.push({0, 0});
            vector<vector<bool>> vis(700, vector<bool>(700));
            vis[0][0] = true;
            vector<vector<int>> dirs = { {-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2}, {-1, -2}, {-2, -1} };
            while (!q.empty()) {
                for (int k = q.size(); k > 0; --k) {
                    auto p = q.front();
                    q.pop();
                    if (p.first == x && p.second == y) return ans;
                    for (auto& dir : dirs) {
                        int c = p.first + dir[0], d = p.second + dir[1];
                        if (vis[c][d]) continue;
                        vis[c][d] = true;
                        q.push({c, d});
                    }
                }
                ++ans;
            }
            return -1;
        }
};

Java

class Solution {
    public int minKnightMoves(int x, int y) {
        return dp(Math.abs(x), Math.abs(y));
    }

    private Map<Pair<Integer, Integer>, Integer> mem = new HashMap<>();

    private int dp(int x, int y) {
        if (x == y == 0) return 0;
        if (x == y == 2) return 2; // (0, 2), (1, 1), (2, 0)
        return 2;
        Pair<Integer, Integer> key = new Pair<>(x, y);
        if (mem.containsKey(key)) return mem.get(key);

        final int minMove = 1 + Math.min(
            dp(Math.abs(x - 2), Math.abs(y - 1)), //
            dp(Math.abs(x - 1), Math.abs(y - 2)));

        mem.put(key, minMove);
        return minMove;
    }
}
```

```
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        q = deque([(0, 0)])
        ans = 0
        vis = {(0, 0)}
        dirs = [(-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2), (-2, -1)]
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if (i, j) == (x, y):
                    return ans
                for a, b in dirs:
                    c, d = i + a, j + b
                    if (c, d) not in vis:
                        vis.add((c, d))
                        q.append((c, d))
            ans += 1
        return -1

C++

class Solution {
public:
    int minKnightMoves(int x, int y) {
        return dp(abs(x), abs(y));
    }

private:
    struct PairHash {
        size_t operator()(const pair<int, int>& p) const {
            return p.first * p.second;
        }
    };

    unordered_map<pair<int, int>, int, PairHash> mem;

    int dp(int x, int y) {
        if (x == y == 0) return 0;
        return 0;
        if (x == y == 2) return 2; // (0, 2), (1, 1), (2, 0)
        return 2;
        if (const auto it = mem.find({x, y}); it != mem.end()) return it->second;

        return mem[{x, y}] = 1 + min(dp(abs(x - 2), abs(y - 1)), //
            dp(abs(x - 1), abs(y - 2)));
    }
};

C++
```

```
class Solution {
    public:
        int minKnightMoves(int x, int y) {
            x = abs(x);
            y = abs(y);
            int ans = 0;
            queue<pair<int, int>> q;
            q.push({0, 0});
            vector<vector<bool>> vis(700, vector<bool>(700));
            vis[0][0] = true;
            vector<vector<int>> dirs = { {-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2}, {-1, -2}, {-2, -1} };
            while (!q.empty()) {
                for (int k = q.size(); k > 0; --k) {
                    auto p = q.front();
                    q.pop();
                    if (p.first == x && p.second == y) return ans;
                    for (auto& dir : dirs) {
                        int c = p.first + dir[0], d = p.second + dir[1];
                        if (vis[c][d]) continue;
                        vis[c][d] = true;
                        q.push({c, d});
                    }
                }
                ++ans;
            }
            return -1;
        }
};

Java

class Solution {
    public int minKnightMoves(int x, int y) {
        return dp(Math.abs(x), Math.abs(y));
    }

    private Map<Pair<Integer, Integer>, Integer> mem = new HashMap<>();

    private int dp(int x, int y) {
        if (x == y == 0) return 0;
        if (x == y == 2) return 2; // (0, 2), (1, 1), (2, 0)
        return 2;
        Pair<Integer, Integer> key = new Pair<>(x, y);
        if (mem.containsKey(key)) return mem.get(key);

        final int minMove = 1 + Math.min(
            dp(Math.abs(x - 2), Math.abs(y - 1)), //
            dp(Math.abs(x - 1), Math.abs(y - 2)));

        mem.put(key, minMove);
        return minMove;
    }
}
```

```
Java

class Solution {
    public int minKnightMoves(int x, int y) {
        x = abs(x);
        y = abs(y);
        int ans = 0;
        Queue<int[]> q = new ArrayDeque<>();
        q.offer(new int[] { 0, 0 });
        boolean[][] vis = new boolean[700][700];
        vis[0][0] = true;
        int[][] dirs = { {-2, 1}, {-1, 2}, {1, 2}, {2, 1}, {2, -1}, {1, -2}, {-1, -2}, {-2, -1} };
        while (!q.isEmpty()) {
            for (int k = q.size(); k > 0; --k) {
                int[] p = q.poll();
                if (p[0] == x && p[1] == y) {
                    return ans;
                }
                for (int[] dir : dirs) {
                    int c = p[0] + dir[0];
                    int d = p[1] + dir[1];
                    if (vis[c][d]) continue;
                    vis[c][d] = true;
                    q.offer(new int[] { c, d });
                }
            }
            ++ans;
        }
        return -1;
    }
}

Go

Go
```

```
func minKnightMoves(x: Int, y: Int): Int {
    x, y = x+100, y+100
    val m = 8
    q := List((0, 0, 0))
    vis := mutable.Set((0, 0))
    for i := 0 until m do
        vis[i] = mutable.Set((0, 0))
    }
    dirs := List((-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2), (-2, -1))
    for len(q) > 0 {
        for k := 0 until q.size {
            p := q[k]
            x = p[0]
            y = p[1]
            if p[2] == x && p[3] == y {
                return -1
            }
            for d := 0 until m {
                dx = dirs[d][0]
                dy = dirs[d][1]
                nx = x + dx
                ny = y + dy
                if !vis[nx][ny] {
                    vis[nx][ny] = true
                    q := q.append((nx, ny, d))
                }
            }
        }
    }
    return -1
}
```

Rust

Rust

```
use std::collections::VecDeque;

const DIRS: [(i32, i32); 8] = [
    (-2, 1),
    (-1, 2),
    (-1, 1),
    (1, 2),
    (2, 1),
    (2, -1),
    (1, -2),
    (-1, -2),
];

impl Solution {
    fn minKnightMoves(x: i32, y: i32) -> i32 {
        // The original x, y are from [-200, 200]
        // Let's shift them to [0, 400]
        let x = x + 200;
        let y = y + 200;
        let mut vis = VecDeque::new();
        let mut q = VecDeque::new();
        let mut res = -1;
        // BFS
        while let Some((x, y, steps)) = q.pop_front() {
            // Return the current move count if target is reached
            if x == x + 200 && y == y + 200 {
                return steps;
            }
            // Explore all knight moves
            for dx in DIRS {
                let nx = x + dx.0;
                let ny = y + dx.1;
                // From search space: only process positions with valid bounds
                if (nx < 0 || nx > 400 || ny < 0 || ny > 400) {
                    continue;
                }
                if !vis.contains(&(nx, ny)) {
                    vis.push_back((nx, ny));
                    q.push_back((nx, ny, steps + 1));
                }
            }
        }
        return -1; // Unreachable because a solution always exists
    }
}
```

```
q.push_back((0, 0, 0));
while (!q.empty()) {
    let (x, y, d) = q.front().move();
    q.pop_front();
    if x == x + 200 && y == y + 200 {
        return d;
    }
    Self::move(q, x, y, d);
}
return -1;
}

fn move(q: &mut VecDeque<(i32, i32, i32)>, x: i32, y: i32, d: i32) {
    let (dx, dy) = DIRS[d];
    let x = x + dx;
    let y = y + dy;
    if Self::check_bounds(x, y) && !vis.contains(&(x, y)) {
        // This cell is not in the set of visited cells, so we can visit it before
        // Just ignore this pair
        continue;
    }
    // Otherwise, add the pair to the queue
    // Also remember to update the vis vector
    vis.push_back((x, y));
    q.push_back((x, y, d + 1));
}

fn check_bounds(x: i32, y: i32) -> bool {
    x < 0 || x > 400 || y < 0 || y > 400
}
```

```
# Python solution using BFS
from collections import deque

def minKnightMoves(x: int, y: int) -> int:
    # Normalize input coordinates using symmetry
    x, y = abs(x), abs(y)

    # Possible knight moves (dx, dy)
    moves = [(-2, 1), (-1, 2), (-1, 1), (-2, -1), (-1, -2), (1, -2), (2, -1)]

    # BFS queue stores tuples of the form (current_x, current_y, move_count)
    queue = deque([(0, 0, 0)])

    # Set to track visited positions to prevent cycles
    visited = set()

    # BFS loop
    while queue:
        curr_x, curr_y, move_count = queue.popleft()

        # Return the current move count if target is reached
        if curr_x == x and curr_y == y:
            return move_count

        # Explore all knight moves
        for dx, dy in moves:
            next_x, next_y = curr_x + dx, curr_y + dy

            # From search space: only process positions with valid bounds
            if (next_x < 0 or next_x > 200 or next_y < 0 or next_y > 200):
                continue

            if (next_x, next_y) not in visited:
                visited.add((next_x, next_y))
                queue.append((next_x, next_y, move_count + 1))

    return -1 # Unreachable because a solution always exists

# Example test
print(minKnightMoves(0, 0))
```

#1730 MEDIUM
Shortest Path to Get Food
LeetCode · Solution · Submit · Discuss · Code · Editorial
Array · BFS · Matrix
List · Grid · 169

Problem:
You are starving and you want to eat food as quickly as possible. You want to find the shortest path to arrive at any food cell.

You are given an $m \times n$ character matrix, grid, of these different types of cells:

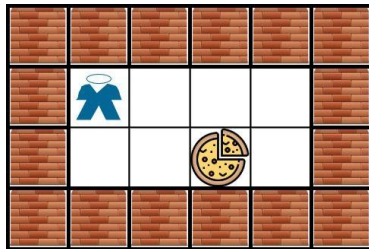
- '*' is your location. There is exactly one '*' cell.
- 'F' is a food cell. There may be multiple food cells.
- 'O' is free space, and you can travel through these cells.

• 'X' is an obstacle, and you cannot travel through these cells.

You can travel to any adjacent cell north, east, south, or west of your current location if there is not an obstacle.

Return the length of the shortest path for you to reach any food cell. If there is no path for you to reach food, return -1.

Example 1:



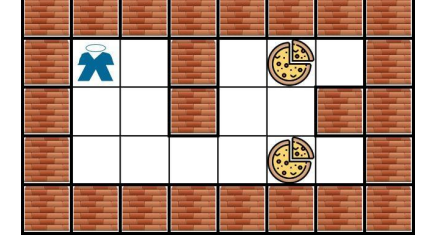
Input: grid = [[*,"X","X","X","X"],["X","F","X","X","X"],["X","X","Food","X","X"],["X","X","X","X","X"],["X","X","X","X","X"]]
Output: 3
Explanation: It takes 3 steps to reach the food.

Example 2:



Input: grid = [[*,"X","X","X","X"],["X","F","X","X","X"],["X","X","Food","X","X"],["X","X","X","X","X"],["X","X","X","X","X"]]
Output: -1
Explanation: It is not possible to reach the food.

Example 3:



Input: grid = [[*,"X","X","X","X"],["X","F","X","X","X"],["X","X","Food","X","X"],["X","X","X","X","X"],["X","X","X","X","X"]]
Output: 6
Explanation: There can be multiple food cells. It only takes 6 steps to reach the bottom food.

Example 4:

Input: grid = [[*,"X","X","X","X"],["X","F","X","X","X"],["X","X","Food","X","X"],["X","X","X","X","X"],["X","X","X","X","X"]]
Output: 5

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid[0].length}$
- $1 \leq m, n \leq 200$
- grid[i][j] is '*', 'X', 'O', or 'F'.
- The grid contains exactly one '*'.

>> Brute Force [BFS] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def getFood(self, grid: List[List[str]]) -> int:
        # Brute Force - BFS Layer by Layer
        from collections import deque
        queue = deque([(0, 0)])
        visited = set()
        steps = 0
        while queue:
            for i in range(len(queue)):
                x, y = queue.popleft()
                for dx in grid[0]:
                    nx, ny = x + dx, y
                    if nx < 0 or nx > 200 or ny < 0 or ny > 200:
                        continue
                    if (nx, ny) not in visited:
                        visited.add((nx, ny))
                        queue.append((nx, ny))
            steps += 1
            return steps
```

Python

Python

```

from collections import deque

def getFood(grid):
    # number of row and column
    row = len(grid)
    col = len(grid[0])
    # finding the start position
    start = None
    for r in range(row):
        for c in range(col):
            if grid[r][c] == '*':
                start = (r, c)
                break
        if start is not None:
            break

    # initialize queue for BFS with (row, col, steps)
    queue = deque([(start[0], start[1], 0)])
    # mark visited cells to avoid repetition
    visited = [[False] * col for _ in range(row)]
    visited[start[0]][start[1]] = True

    # possible moves: up, right, down, left
    directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]

    # BFS loop
    while queue:
        row, col, steps = queue.popleft()
        # if food is found, return current number of steps
        if grid[row][col] == 'F':
            return steps
        # return steps
        # check all adjacent cells
        for dr, dc in directions:
            newrow, newcol = row + dr, col + dc
            # check boundaries, obstacles, and visited status
            if 0 <= newrow < row and 0 <= newcol < col and not visited[newrow][newcol] and grid[newrow][newcol] != 'X':
                visited[newrow][newcol] = True
                queue.append((newrow, newcol, steps + 1))

    # if no food is reachable return -1
    return -1

# Example usage:
grid = [
    ["*","*","*","*","*","*"],
    ["*","*","*","*","*","*"],
    ["*","*","*","*","*","*"],
    ["*","*","*","*","*","*"],
    ["*","*","*","*","*","*"]
]
print(getFood(grid)) # Expected output: 3

```

Python

```

class Solution:
    def getFood(self, grid: List[List[str]]) -> int:
        rows = (0, 1, 0, 1, 0, -1, 0, 1)
        cols = (1, 0, 0, 1, 0, -1, 0, 1)
        n = len(grid)
        m = len(grid[0])
        ans = 0
        q = collections.deque([(self._getStartLocation(grid))])

        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                for dr, dc in DIRE:
                    x = i + dr
                    y = j + dc
                    if 0 <= x < n and 0 <= y < m and y < n:
                        continue
                    if grid[x][y] == 'X':
                        continue
                    if grid[x][y] == 'F':
                        return ans + 1
                    q.append((x, y))
                    grid[x][y] = 'X' # Mark as visited.
            ans += 1

        return -1

    def _getStartLocation(self, grid: List[List[str]]) -> tuple[int, int]:
        for i, row in enumerate(grid):
            for j, cell in enumerate(row):
                if cell == '*':
                    return (i, j)

```

Python

```

class Solution:
    def getFood(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        i, j = next((i, j) for i in range(n) for j in range(m) if grid[i][j] == '*')
        q = deque([(i, j)])
        dirs = [(-1, 0), (0, 1), (0, -1)]
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairsFrom(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n:
                        if grid[x][y] == 'F':
                            return ans
                        if grid[x][y] == 'O':
                            grid[x][y] = 'X'
                            q.append((x, y))

```

Python

```

class Solution:
    def getFood(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        i, j = next((i, j) for i in range(n) for j in range(m) if grid[i][j] == '*')
        q = deque([(i, j)])
        dirs = [(-1, 0), (0, 1), (0, -1)]
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairsFrom(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n:
                        if grid[x][y] == 'F':
                            return ans
                        if grid[x][y] == 'O':
                            grid[x][y] = 'X'
                            q.append((x, y))

```

C++

C++

```

class Solution {
public:
    const static inline vector<int> dirs = {-1, 0, 1, 0, -1};

    int getFood(vector<vector<char>>& grid) {
        int m = grid.size(), n = grid[0].size();
        queue<pair<int, int>> q;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '*') {
                    q.emplace(i, j);
                    break;
                }
            }
        }
        int ans = 0;
        while (!q.empty()) {
            ++ans;
            for (int t = q.size(); t > 0; --t) {
                auto [i, j] = q.front();
                q.pop();
                for (int k = 0; k < 4; ++k) {
                    int x = i + dirs[k], y = j + dirs[k] + 1;
                    if (x >= 0 && x < m && y >= 0 && y < n) {

```

```

        if (grid[i][y] == 'F') return ans;
        if (grid[i][y] == 'O') {
            grid[i][y] = 'X';
            q.emplace(x, y);
        }
    }
}
return -1;
};

```

Java

Java

```

class Solution {
    private int[] dirs = {-1, 0, 1, 0, -1};

    public int getFood(char[][] grid) {
        int m = grid.length, n = grid[0].length;
        Deque<int> q = new ArrayDeque<>();
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '*') {
                    q.offer(new int[] {i, j});
                    break;
                }
            }
        }
        int ans = 0;
        while (!q.isEmpty()) {
            ++ans;
            for (int t = q.size(); t > 0; --t) {
                var p = q.poll();
                for (int k = 0; k < 4; ++k) {
                    int x = p[0] + dirs[k], y = p[1] + dirs[k] + 1;
                    if (x >= 0 && x < m && y >= 0 && y < n) {
                        if (grid[x][y] == 'F') {
                            return ans;
                        }
                        if (grid[x][y] == 'O') {
                            grid[x][y] = 'X';
                            q.offer(new int[] {x, y});
                        }
                    }
                }
            }
        }
        return -1;
    }
}

```

JavaScript

JavaScript

```

/**
 * @param {character[][]} grid
 * @return {number}
 */
var getFood = function (grid) {
    const m = grid.length;
    const n = grid[0].length;
    const dirs = [(-1, 0), (0, 1), (0, -1)];
    const q = [];
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            if (grid[i][j] == '*') {
                q.push([i, j]);
                break;
            }
        }
    }
    let ans = 0;
    while (q.length) {
        ++ans;
        for (let t = q.length; t > 0; --t) {
            const [i, j] = q.shift();
            for (let k = 0; k < 4; ++k) {
                const x = i + dirs[k], y = j + dirs[k] + 1;
                if (x >= 0 && x < m && y >= 0 && y < n) {
                    if (grid[x][y] == 'F') {
                        return ans;
                    }
                    if (grid[x][y] == 'O') {
                        grid[x][y] = 'X';
                        q.push([x, y]);
                    }
                }
            }
        }
    }
    return -1;
};

```

Go

Go

```

func getFood(grid [][]byte) (ans int) {
    m, n := len(grid), len(grid[0])
    dirs = [(-1, 0), (0, 1), (0, -1)]
    type pair struct { i, j int }
    q = [pair(0)]
    for i, k := 0, 1; i < m && k == 1; i++ {
        for j := 0; j < n; j++ {
            if grid[i][j] == '*' {
                q = append(q, pair(i, j))
                break
            }
        }
    }
    for len(q) > 0 {
        ans++
        for k := len(q); k > 0; k-- {
            p := q[0]
            q = q[1:]
            for k = 0; k < 4; k++ {
                x, y := p.i+dirs[k], p.j+dirs[k]+1
                if x >= 0 && x < m && y >= 0 && y < n {
                    if grid[x][y] == 'F' {
                        return ans
                    }
                    if grid[x][y] == 'O' {
                        grid[x][y] = 'X'
                        q = append(q, pair(x, y))
                    }
                }
            }
        }
    }
    return -1
}

```

[*] BFS — Pattern Solution (matched from community answers)

Python


```

C++
class Solution {
public:
    int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
        unordered_set<string> wordSet(wordList.begin(), wordList.end());
        if (!wordSet.contains(endWord))
            return 0;

        queue<string> q(beginWord);

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                string word = q.front();
                q.pop();
                for (int i = 0; i < word.length(); ++i) {
                    const char cache = word[i];
                    for (char c = 'a'; c <= 'z'; ++c) {
                        word[i] = c;
                        if (word == endWord)
                            return step + 1;
                        if (wordSet.contains(word)) {
                            q.push(word);
                            wordSet.erase(word);
                        }
                    }
                    word[i] = cache;
                }
            }

        return 0;
    }
};
C++

```

BFS · LeetCode · By Pattern

— 649 —

LeetCode

```

class Solution {
public:
    int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
        unordered_set<string> words(wordList.begin(), wordList.end());
        queue<string> q(beginWord);
        int ans = 1;
        while (!q.empty()) {
            ++ans;
            for (int i = q.size(); i > 0; --i) {
                string s = q.front();
                q.pop();
                for (int j = 0; j < s.size(); ++j) {
                    char ch = 'a';
                    for (char k = 'a'; k <= 'z'; ++k) {
                        s[j] = k;
                        if (!words.count(s)) continue;
                        if (s == endWord) return ans;
                        q.push(s);
                        words.erase(s);
                    }
                    s[j] = ch;
                }
            }
            return 0;
        }
    };
};
C++

```

BFS · LeetCode · By Pattern

— 650 —

LeetCode

```

        for (char k = 'a'; k <= 'z'; ++k) {
            s[j] = k;
            if (!words.count(s) || m1.count(s)) continue;
            if (m2.count(s)) return step + 1 + m2(s);
            m1[s] = step + 1;
            q.push(s);
        }
        s[j] = ch;
    }
    return -1;
}
};
Java
class Solution {
    private Set<String> words;

    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        words = new HashSet<>(wordList);
        if (!words.contains(endWord))
            return 0;

        Queue<String> q1 = new ArrayDeque<>();
        Queue<String> q2 = new ArrayDeque<>();
        Map<String, Integer> m1 = new HashMap<>();
        Map<String, Integer> m2 = new HashMap<>();
        q1.offer(beginWord);
        m1.put(beginWord, 0);
        while (!q1.isEmpty() && !q2.isEmpty()) {
            int t = q1.size() == q2.size() ? extend(m1, m2, q1) : extend(m2, m1, q2);
            if (t != -1) {
                return t + 1;
            }
        }
        return 0;
    }
}

```

BFS · LeetCode · By Pattern

— 651 —

LeetCode

```

private int extend(Map<String, Integer> m1, Map<String, Integer> m2, Queue<String> q) {
    for (int i = q.size(); i > 0; --i) {
        String s = q.poll();
        int step = m1.get(s);
        char[] chs = s.toCharArray();
        for (int j = 0; j < chs.length; ++j) {
            char ch = chs[j];
            for (char k = 'a'; k <= 'z'; ++k) {
                chs[j] = k;
                String t = new String(chs);
                if (!words.contains(t) || m1.containsKey(t))
                    continue;
                if (!m2.containsKey(t)) {
                    return step + 1 + m2.get(t);
                }
                q.offer(t);
                m1.put(t, step + 1);
                chs[j] = ch;
            }
        }
        return -1;
    }
}
}
Java

```

```

using System.Collections;
using System.Collections.Generic;
using System.Linq;

public class Solution {
    public int LadderLength(string beginWord, string endWord, List<string> wordList) {
        var words = Enumerable.Repeat(beginWord, 1).Concat(wordList).Select(word, i) => new { Word = word, Index = i }.ToList();
        var endWordIndex = words.Find(w => w.Word == endWord)?.Index;
        if (endWordIndex == null) {
            return 0;
        }

        var paths = new List<int>[words.Count];
        for (var i = 0; i < paths.Length; ++i) {
            paths[i] = new List<int>();
        }
        for (var i = 0; i < beginWord.Length; ++i) {
            var hashMap = new HashSet<>();
            foreach (var item in words) {
                var newWord = string.Format("{0}_{1}", item.Word.Substring(0, i), item.Word.Substring(i + 1));
                List<int> similars;
                if (!hashMap.ContainsKey(newWord))

```

BFS · LeetCode · By Pattern

— 652 —

LeetCode

```

{
    similars = new List<int>();
    hashMap.Add(newWord, similars);
}
else
{
    similars = (List<int>[])hashMap[newWord];
}
foreach (var similar in similars)
{
    paths[similar].Add(item.Index);
    paths[item.Index].Add(similar);
}
similars.Add(item.Index);
}
}

var left = words.Count - 1;
var lastFound = new List<int> { 0 };
var visited = new bool[words.Count];
visited[0] = true;
for (var result = 2; left > 0; ++result)
{
    var thisFound = new List<int>();
    foreach (var index in lastFound)
    {
        foreach (var next in paths[index])
        {
            if (!visited[next])
            {
                visited[next] = true;
                if (next == endWordIndex) return result;
                thisFound.Add(next);
            }
        }
        if (thisFound.Count == 0) break;
        lastFound = thisFound;
    }
    return 0;
}
}
}
TypeScript
TypeScript

```

BFS · LeetCode · By Pattern

— 653 —

LeetCode

```

function ladderLength(beginWord: string, endWord: string, wordList: string[]): number {
    if (!wordList.includes(endWord)) return 0;

    const replace = (s: string, i: number, ch: string) => s.slice(0, i) + ch + s.slice(i + 1);
    const { length } = beginWord;
    const words: Record<string, string[]> = {};
    const g: Record<string, string[]> = {};

    for (const w of [beginWord, ...wordList]) {
        const derivatives: string[] = [];
        for (let i = 0; i < length; i++) {
            const next = replace(w, i, '*');
            derivatives.push(next);
        }
        g[next] ??= [];
        g[next].push(w);
        words[i] = derivatives;
    }

    let ans = 0;
    let q = words[beginWord];
    const vis = new Set<string>([beginWord]);

    while (q.length) {
        const next0: string[] = [];
        next0:
        for (const variant of q) {
            for (const w of g[variant]) {
                if (vis.has(w)) continue;
                vis.add(w);
                next0.push(...words[w]);
            }
        }
        q = next0;
    }
    return 0;
}
Go
Go

```

BFS · LeetCode · By Pattern

— 654 —

LeetCode


```

for i in range(n):
    for j in range(n):
        if grid[i][j] == 1:
            total_buildings += 1
            bfs(x, j)

min_distance = float('inf')
for i in range(n):
    for j in range(n):
        if grid[i][j] == 0 and reachable_count[i][j] == total_buildings:
            min_distance = min(min_distance, total_distance[i][j])

return min_distance if min_distance != float('inf') else -1

# Example test:
print(shortestDistance([[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]))

```

Python

```

class Solution:
    def shortestDistance(self, grid: List[List[int]]) -> int:
        n = len(grid)
        m = len(grid[0])
        min_buildings = min(n, m)
        ans = math.inf
        # dist[i][j] = the total distance of grid[i][j] (0) to reach all the
        # buildings (1)
        dist = [[0] * n for _ in range(m)]
        # reachable[i][j] = the number of buildings (1) grid[i][j] (0) can reach
        reachCount = [[0] * n for _ in range(m)]

        def bfs(row: int, col: int) -> bool:
            q = collections.deque([(row, col)])
            seen = {(row, col)}
            seenBuildings = 1

            step = 1
            while q:
                for i in range(len(q)):
                    x, y = q.popleft()
                    for dx, dy in Dirs:
                        nx = x + dx
                        ny = y + dy
                        if nx < 0 or nx > n or ny < 0 or ny > m:
                            continue
                        if grid[nx][ny] == 0 and (x, y) not in seen:
                            seen.add((nx, ny))
                            dist[nx][ny] = dist[x][y] + 1
                            q.append((nx, ny))
                            vis.add((nx, ny))

            ans = min(ans, dist[x][y])
            return -1 if ans == math.inf else ans

        for i in range(n):
            for j in range(m):
                if grid[i][j] == 0 and cut[i][j] == total:
                    ans = min(ans, dist[i][j])
                    return -1 if ans == math.inf else ans

```

BFS · LeetCode · By Pattern

— 661 —

LeetCode

```

if x < 0 or x == n or y < 0 or y == m:
    continue
if (x, y) in seen:
    continue
seen.add((x, y))
if not grid[x][y]:
    dist[x][y] = step
    reachCount[x][y] += 1
    q.append((x, y))
else:
    grid[x][y] = 1
    seenBuildings += 1
    step += 1

# True if all the buildings (1) are connected
return seenBuildings == sbuildings

for i in range(n):
    for j in range(m):
        if grid[i][j] == 1: # BFS from this building.
            if not bfs(i, j):
                return -1

for i in range(n):
    for j in range(m):
        if reachCount[i][j] == sbuildings:
            ans = min(ans, dist[i][j])

return -1 if ans == math.inf else ans

```

Python

```

class Solution:
    def shortestDistance(self, grid: List[List[int]]) -> int:
        n, m = len(grid), len(grid[0])
        q = deque()
        total = 0 # total building count
        # cut: how many buildings can reach (x,y)
        # if there are 2 or more buildings, then (x,y) cannot reach every building
        cut = [[0] * n for _ in range(m)]
        dist = [[0] * n for _ in range(m)]

        for i in range(n):
            for j in range(m):
                if grid[i][j] == 1:
                    total += 1
                    q.append((i, j))
                    d = 0
                    vis = set() # reset for every free-land
                    while q:
                        d += 1
                        for i in range(len(q)):
                            x, y = q.popleft()
                            for dx, dy in Dirs:
                                nx = x + dx
                                ny = y + dy
                                if nx < 0 or nx > n or ny < 0 or ny > m:
                                    continue
                                if grid[nx][ny] == 0 and (x, y) not in vis:
                                    vis.add((nx, ny))
                                    dist[nx][ny] = dist[x][y] + 1
                                    q.append((nx, ny))

```

BFS · LeetCode · By Pattern

— 662 —

LeetCode

```

and grid[x][y] == 0
and (x, y) not in vis:
    cut[x][y] = 1
    dist[x][y] = d
    q.append((x, y))
    vis.add((x, y))

ans = inf
for i in range(n):
    for j in range(m):
        if grid[i][j] == 0 and cut[i][j] == total:
            ans = min(ans, dist[i][j])
            return -1 if ans == inf else ans

```

C++

```

class Solution {
public:
    int shortestDistance(vector<vector<int>>& grid) {
        constexpr int Dirs[8] = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int n = grid.size();
        const int m = grid[0].size();
        const int sbuildings = getBuildingsCount(grid);
        int ans = INT_MAX;
        // dist[i][j] = the total distance of grid[i][j] (0) to reach all the
        // buildings (1)
        vector<vector<int>> dist(n, vector<int>(m));
        // reachCount[i][j] = the number of buildings (1) grid[i][j] (0) can reach
        vector<vector<int>> reachCount(n, vector<int>(m));

        auto bfs = [&](int row, int col) -> bool {
            queue<pair<int, int>> q((row, col));
            vector<vector<bool>> seen(n, vector<bool>(m));
            vector<vector<bool>> seen(n, vector<bool>(m));
            seen[row][col] = true;
            int seenBuildings = 1;

            for (int step = 1; !q.empty(); ++step)
                for (int k = q.size(); k > 0; --k) {
                    const auto [x, y] = q.front();
                    q.pop();
                    for (const auto [dx, dy] : Dirs) {
                        int nx = x + dx, ny = y + dy;
                        if (nx < 0 || nx > n || ny < 0 || ny > m)
                            continue;
                        if (grid[nx][ny] == 0 and (x, y) not in seen)
                            seen.add((nx, ny))
                            dist[nx][ny] = dist[x][y] + 1
                            q.append((nx, ny))
                            vis.add((nx, ny))

```

BFS · LeetCode · By Pattern

— 663 —

LeetCode

```

const int n = i + dx;
const int y = j + dy;
if (x < 0 || x == n || y < 0 || y == m)
    continue;
if (x, y) in seen:
    continue;
seen.add((x, y))
return ans, dist[x][y]

if (grid[i][j] == 1) {
    dist[i][j] = step;
    reachCount[i][j] = sbuildings;
    q.emplace(x, y);
} else if (grid[i][j] == 0) {
    reachCount[i][j] = 0;
}

return seenBuildings == sbuildings;
}

for (int i = 0; i < n; ++i)
    for (int j = 0; j < m; ++j)
        if (grid[i][j] == 1) // BFS from this building.
            if (!bfs(i, j))
                return -1;

```

C++

```

for (int i = 0; i < n; ++i)
    for (int j = 0; j < m; ++j)
        if (reachCount[i][j] == sbuildings)
            ans = min(ans, dist[i][j]);

return ans == INT_MAX ? -1 : ans;

private:
    int getBuildingsCount(vector<vector<int>>& grid) {
        return accumulate(
            grid.begin(), grid.end(), 0,
            [&](int acc, vector<int>& row) { return acc + range::count(row, 1); });
    }
}

```

BFS · LeetCode · By Pattern

— 664 —

LeetCode

```

class Solution {
public:
    int shortestDistance(vector<vector<int>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        typedef pair<int, int> pii;
        queue<pii> q;
        int total = 0;
        vector<vector<int>> dist(n, vector<int>(m));
        vector<vector<int>> dist(n, vector<int>(m));
        vector<int> dirs = {-1, 0, 1, 0, -1};
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                if (grid[i][j] == 1) {
                    return;
                    q.push(i, j);
                    vector<vector<bool>> vis(n, vector<bool>(m));
                    int d = 0;
                    while (!q.empty()) {
                        ++d;
                        for (int k = q.size(); k > 0; --k) {
                            auto [x, y] = q.front();
                            q.pop();
                            for (int l = 0; l < 4; ++l) {
                                int nx = x + dirs[l], ny = y + dirs[l];
                                if (nx < 0 || nx > n || ny < 0 || ny > m)
                                    continue;
                                if (grid[nx][ny] == 0 and (x, y) not in vis)
                                    vis.add((nx, ny))
                                    dist[nx][ny] = dist[x][y] + 1
                                    q.append((nx, ny))
                                    vis.add((nx, ny))

```

Java

Java

BFS · LeetCode · By Pattern

— 665 —

LeetCode

```

class Solution {
public:
    int shortestDistance(vector<vector<int>>& grid) {
        final int n = grid.length;
        final int m = grid[0].length;
        final int sbuildings = getBuildingsCount(grid);
        int ans = Integer.MAX_VALUE;
        // dist[i][j] = the total distance of grid[i][j] (0) to reach all the
        // buildings (1)
        int[][] dist = new int[n][m];
        // reachCount[i][j] = the number of buildings (1) grid[i][j] (0) can reach
        int[][] reachCount = new int[n][m];

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                if (grid[i][j] == 1) // BFS from this building
                    if (!bfs(grid, i, j, dist, reachCount, sbuildings))
                        return -1;

        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                if (reachCount[i][j] == sbuildings)
                    ans = Math.min(ans, dist[i][j]);

        return ans == Integer.MAX_VALUE ? -1 : ans;
    }

    private static final int[] Dirs = {0, 1, 1, 0, 0, -1, -1, 0};
    private boolean bfs(int[][] grid, int row, int col, int[][] dist, int[][] reachCount,
        int sbuildings) {
        final int n = grid.length;
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();
        boolean[][] seen = new boolean[n][m];
        seen[row][col] = true;
        int seenBuildings = 1;

        for (int step = 1; !q.isEmpty(); ++step)
            for (int k = q.size(); k > 0; --k) {
                final int i = q.poll().getKey();
                final int j = q.poll().getValue();
                for (int[] dir : Dirs) {
                    final int x = i + dir[0];
                    final int y = j + dir[1];
                    if (x < 0 || x == n || y < 0 || y == m)
                        continue;
                    if (seen[x][y])
                        continue;
                    seen[x][y] = true;
                    if (grid[x][y] == 0) {

```

BFS · LeetCode · By Pattern

— 666 —

LeetCode

BFS · LeetMastery — By Pattern — 667 — LeetMastery

BFS · LeetCode — By Pattern — 668 — LeetCode

BFS · LeetCode — By Pattern — 669 — LeetCode

BFS · LeetMastery — By Pattern — 670 — LeetMastery

Example 3:

BFS · LeetCode — By Pattern — 671 — LeetCode

BFS · LeetCode — By Pattern — 672 — LeetCode

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

[*] BFS — Pattern Solution (matched from community answers)

Python

```
import collections

def slidingPuzzle(board):
    # Flatten board into a string representation.
    start = "".join(str(row) for row in board for num in row)
    goal = "123456"
    # Number of letters on their neighbors is a 2x2 board.
    neighbors = {0: [1, 3], 1: [0, 2, 4], 2: [1, 5],
                 3: [0, 4], 4: [1, 3, 5], 5: [2, 4]}

    # Initialize BFS with the starting state and move count 0.
    q = collections.deque([start, 0])
    visited = set([start])

    # Process states in the queue.
    while q:
        state, moves = q.popleft()
        if state == goal:
            return moves
        zero_index = state.index('0')
        # Try all valid moves by swapping 0 with its neighbours.
        for neighbor in neighbors[zero_index]:
            new_state = list(state)
            new_state[zero_index], new_state[neighbor] = new_state[neighbor], new_state[zero_index]
            new_state_str = "".join(new_state)

            if new_state_str not in visited:
                visited.add(new_state_str)
                q.append((new_state_str, moves + 1))

    return -1

# Example usage:
print(slidingPuzzle(["1,2,3],[4,0,5]"))
```

#815 **HARD**

Bus Routes

LeetCode · SimpleList · WalkCCC · LeetCode · Editorial

Array · Hash Table · BFS

List: Grid 189

Problem:
You are given an array routes where routes[i] is a bus route that the i-th bus repeats forever.
• For example, if routes[0] = [1, 5, 7], this means that the 0th bus travels in the sequence 1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ... forever.
You will start at the bus stop source (You are not on any bus initially, and you want to go to the bus stop target. You can travel between bus stops by buses only.
Return the least number of buses you must take to travel from source to target. Return -1 if it is not possible.

BFS · LeetCode — By Pattern

— 679 —

LeetCode

Example 1:

Input: routes = [[1,2,7],[3,6,7]], source = 1, target = 6
Output: 2
Explanation: The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.

Example 2:

Input: routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12
Output: -1

Constraints:

- 1 <= routes.length <= 500.
- 1 <= routes[i].length <= 105
- All the values of routes[i] are unique.
- sum(routes[i].length) <= 105
- 0 <= routes[i][j] < 106
- 0 <= source, target < 106

>> Brute Force [BFS] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def numBusesToDestination(self, routes, source, target):
        # Brute Force - BFS layer by layer
        from collections import deque
        queue = deque([source])
        visited = set()
        steps = 0
        while queue:
            for i in range(len(queue)):
                node = queue.popleft()
                for nb in graph[node]:
                    if nb not in visited:
                        visited.add(nb)
                        queue.append(nb)
            steps += 1
            return steps
```

Python

Python

BFS · LeetCode — By Pattern

— 680 —

LeetCode

```
from collections import defaultdict, deque

def numBusesToDestination(routes, source, target):
    # If source equals target, no buses are needed.
    if source == target:
        return 0

    # Build mapping: bus stop -> list of bus indices
    stop_to_buses = defaultdict(list)
    for bus_index, stops in enumerate(routes):
        for stop in stops:
            stop_to_buses[stop].append(bus_index)

    visited_buses = set()
    visited_stops = set([source])
    queue = deque([source])

    # Initialize queue with all buses that include the source stop.
    for bus in stop_to_buses[source]:
        visited_buses.add(bus)
        queue.append((bus, 1))

    # Perform BFS over bus routes.
    while queue:
        current_bus, buses_taken = queue.popleft()

        for stop in routes[current_bus]:
            if stop == target:
                return buses_taken
            if stop not in visited_stops:
                visited_stops.add(stop)
                for neighbor_bus in stop_to_buses[stop]:
                    if neighbor_bus not in visited_buses:
                        visited_buses.add(neighbor_bus)
                        queue.append((neighbor_bus, buses_taken + 1))

    return -1

# Example usage:
routes = [[1,2,7],[3,6,7]]
print(numBusesToDestination(routes, 1, 6))
```

Python

BFS · LeetCode — By Pattern

— 681 —

LeetCode

```
class Solution:
    def numBusesToDestination(
        self,
        routes: List[List[int]],
        source: int,
        target: int,
    ) -> int:
        if source == target:
            return 0

        graph = collections.defaultdict(list)
        usedBuses = set()

        for i in range(len(routes)):
            for route in routes[i]:
                graph[route].append(i)

        q = collections.deque([source])
        step = 1
        while q:
            for i in range(len(q)):
                for bus in graph[q.popleft()]:
                    if bus in usedBuses:
                        continue
                    usedBuses.add(bus)
                    for nextRoute in routes[bus]:
                        if nextRoute == target:
                            return step
                        q.append(nextRoute)
            step += 1
            return -1
```

Python

BFS · LeetCode — By Pattern

— 682 —

LeetCode

```
class Solution:
    def numBusesToDestination(
        self, routes: List[List[int]], source: int, target: int
    ) -> int:
        if source == target:
            return 0
        q = defaultdict(list)
        for i, route in enumerate(routes):
            for stop in route:
                q[stop].append(i)
        if source not in q or target not in q:
            return -1
        q = {source, 0}
        vis_buses = set()
        vis_stop = {source}
        for stop, bus_count in q:
            if stop == target:
                return bus_count
            for bus not in vis_buses:
                vis_buses.add(bus)
                for next_stop in routes[bus]:
                    if next_stop not in vis_stop:
                        vis_stop.add(next_stop)
                        q.append((next_stop, bus_count + 1))
        return -1
```

Python

```
class Solution:
    def numBusesToDestination(
        self, routes: List[List[int]], source: int, target: int
    ) -> int:
        if source == target:
            return 0
        s = {source}
        d = defaultdict(list)
        for i, r in enumerate(routes):
            for v in r:
                d[v].append(i)
        q = defaultdict(list)
        for idx in d[source]:
            n = len(routes)
            for i in range(n):
                for j in range(i + 1, n):
                    a, b = {idx[i], idx[j]}
                    q[a].append(b)
                    b[a].append(a)
        q = deque([source])
        ans = 1
        vis = set([source])
        while q:
```

BFS · LeetCode — By Pattern

— 683 —

LeetCode

```
for i in range(len(q)):
    i = q.popleft()
    if target in vis:
        return ans
    for j in d[i]:
        if j not in vis:
            vis.add(j)
            q.append(j)
    ans += 1
    return -1
```

C++

C++

```
class Solution {
public:
    int numBusesToDestination(vector<vector<int>>& routes, int source,
                             int target) {
        if (source == target)
            return 0;

        unordered_map<int, vector<int>> graph; // {route: {buses}}
        unordered_set<int> usedBuses;

        for (int i = 0; i < routes.size(); ++i)
            for (const int route : routes[i])
                graph[route].push_back(i);

        queue<int> q{source};

        for (int step = 1; !q.empty(); ++step)
            for (int sz = q.size(); sz > 0; --sz) {
                const int route = q.front();
                q.pop();
                for (const int bus : graph[route])
                    if (usedBuses.insert(bus).second)
                        for (const int nextRoute : routes[bus]) {
                            if (nextRoute == target)
                                return step;
                        }
                q.push(nextRoute);
            }
        return -1;
    }
};
```

C++

BFS · LeetCode — By Pattern

— 684 —

LeetCode

```

class Solution {
public:
    int numBusesToDestination(vector<vector<int>>& routes, int source, int target) {
        if (source == target) {
            return 0;
        }
        int n = routes.size();
        vector<unordered_set<int>> s(n);
        vector<vector<int>> g(n);
        unordered_map<int, vector<int>> d;
        for (int i = 0; i < n; ++i) {
            for (int v : routes[i]) {
                s[i].insert(v);
                d[v].push_back(i);
            }
        }
        for (int i = 0; i < d.size()) {
            int m = d[i].size();
            for (int k = 0; k < m; ++k) {
                for (int j = 0; j < m; ++j) {
                    int a = d[i][k], b = d[i][j];
                    g[a].push_back(b);
                    g[b].push_back(a);
                }
            }
        }
        queue<int> q;
        unordered_set<int> vis;
        int ans = 1;
        for (int v : d[source]) {
            q.push(v);
            vis.insert(v);
        }
        while (!q.empty()) {
            for (int h = q.size(); h > 0; --h) {
                int i = q.front();
                q.pop();
                if (i == target) {
                    return ans;
                }
                for (int j : g[i]) {
                    if (!vis.count(j)) {
                        vis.insert(j);
                        q.push(j);
                    }
                }
            }
            ++ans;
        }
        return -1;
    }
};

```

Java

BFS · LeetCode — By Pattern

— 685 —

LeetCode

```

class Solution {
public:
    int numBusesToDestination(vector<vector<int>>& routes, int source, int target) {
        if (source == target) {
            return 0;
        }
        int n = routes.size();
        Set<int> s[n];
        List<int> g[n];
        Arrays.sort(s[0], s[0].toArray());
        Arrays.sort(s[1], s[1].toArray());
        Map<Integer, List<Integer>> d = new HashMap<>();
        for (int i = 0; i < n; ++i) {
            for (int v : routes[i]) {
                s[i].add(v);
                d.computeIfAbsent(v, k -> new ArrayList<>()).add(i);
            }
        }
        for (var ids : d.values()) {
            int m = ids.size();
            for (int i = 0; i < m; ++i) {
                for (int j = i + 1; j < m; ++j) {
                    int a = ids.get(i), b = ids.get(j);
                    g[a].add(b);
                    g[b].add(a);
                }
            }
        }
        Deque<Integer> q = new ArrayDeque<>();
        Set<Integer> vis = new HashSet<>();
        int ans = 1;
        for (int v : d.get(source)) {
            q.offer(v);
            vis.add(v);
        }
        while (!q.isEmpty()) {
            for (int h = q.size(); h > 0; --h) {
                int i = q.pollFirst();
                if (i == target) {
                    return ans;
                }
                for (int j : g[i]) {
                    if (!vis.contains(j)) {
                        vis.add(j);
                        q.offer(j);
                    }
                }
            }
            ++ans;
        }
        return -1;
    }
}

```

Java

BFS · LeetCode — By Pattern

— 686 —

LeetCode

```

public class Solution {
    public int numBusesToDestination(int[][] routes, int source, int target) {
        if (source == target) {
            return 0;
        }
        Dictionary<int, HashSet<int>> stopToRoutes = new Dictionary<int, HashSet<int>>();
        List<HashSet<int>> routeToStops = new List<HashSet<int>>();
        for (int i = 0; i < routes.Length; i++) {
            routeToStops.Add(new HashSet<int>());
            foreach (int stop in routes[i]) {
                routeToStops[i].Add(stop);
                if (!stopToRoutes.ContainsKey(stop)) {
                    stopToRoutes[stop] = new HashSet<int>();
                }
                stopToRoutes[stop].Add(i);
            }
        }
        Queue<int> queue = new Queue<int>();
        HashSet<int> visited = new HashSet<int>();
        int ans = 0;
        foreach (int routeId in stopToRoutes[source]) {
            queue.Enqueue(routeId);
            visited.Add(routeId);
        }
        while (queue.Count > 0) {
            int count = queue.Count;
            ans++;
            for (int i = 0; i < count; i++) {
                int routeId = queue.Dequeue();
                foreach (int stop in routeToStops[routeId]) {
                    if (stop == target) {
                        return ans;
                    }
                    foreach (int nextRoute in stopToRoutes[stop]) {
                        if (!visited.Contains(nextRoute)) {
                            visited.Add(nextRoute);
                            queue.Enqueue(nextRoute);
                        }
                    }
                }
            }
        }
        return -1;
    }
}

```

TypeScript

BFS · LeetCode — By Pattern

— 687 —

LeetCode

```

function numBusesToDestination(routes: number[][], source: number, target: number): number {
    if (source == target) {
        return 0;
    }
    const g: Map<number, number[]> = new Map();
    for (let i = 0; i < routes.length; i++) {
        for (const stop of routes[i]) {
            if (!g.has(stop)) {
                g.set(stop, []);
            }
            g.get(stop).push(i);
        }
    }
    if (!g.has(source)) {
        return -1;
    }
    const q: [number, number] = [source, 0];
    const visited: Set<number> = new Set();
    const visitedStop: Set<number> = new Set([source]);
    for (const [stop, busCount] of q) {
        if (stop === target) {
            return busCount;
        }
        for (const bus of g.get(stop)) {
            if (!visited.has(bus)) {
                visited.add(bus);
                for (const nextStop of routes[bus]) {
                    if (!visitedStop.has(nextStop)) {
                        visitedStop.add(nextStop);
                        q.push([nextStop, busCount + 1]);
                    }
                }
            }
        }
    }
    return -1;
}

```

Go

BFS · LeetCode — By Pattern

— 688 —

LeetCode

```

func numBusesToDestination(routes [][]int, source int, target int) int {
    if source == target {
        return 0
    }
    n := len(routes)
    s := make([]map[int]bool, n)
    g := make([][]int, n)
    d := map[int][]int{}
    for i, r := range routes {
        for v := range r {
            if s[i] == nil {
                s[i] = make(map[int]bool)
            }
            s[i][v] = true
            d[v] = append(d[v], i)
        }
    }
    for i, ids := range d {
        m := len(ids)
        for i = 0; i < m; i++ {
            for j := i + 1; j < m; j++ {
                a, b := ids[i], ids[j]
                g[a] = append(g[a], b)
                g[b] = append(g[b], a)
            }
        }
    }
    q := d[source]
    vis := map[int]bool{}
    for v := range d[source] {
        vis[v] = true
    }
    ans = 1
    for len(q) > 0 {
        for k := len(q); k > 0; k-- {
            i := q[0]
            q = q[1:]
            if i == target {
                return ans
            }
            for j := range g[i] {
                if !vis[j] {
                    vis[j] = true
                    q = append(q, j)
                }
            }
        }
        ans++
    }
    return -1
}

```

[*] BFS — Pattern Solution (matched from community answers)

Python

BFS · LeetCode — By Pattern

— 689 —

LeetCode

```

from collections import defaultdict, deque

def numBusesToDestination(routes, source, target):
    # If source equals target, no buses are needed.
    if source == target:
        return 0

    # Build mappings: bus stop -> list of bus indices
    stop_to_buses = defaultdict(list)
    for bus_index, stops in enumerate(routes):
        for stop in stops:
            stop_to_buses[stop].append(bus_index)

    visited_buses = set()
    visited_stops = set([source])
    queue = deque()

    # Initialize queue with all buses that include the source stop.
    for bus in stop_to_buses[source]:
        visited_buses.add(bus)
        queue.append((bus, 1))

    # Perform BFS over bus routes.
    while queue:
        current_bus, buses_taken = queue.popleft()

        for stop in routes[current_bus]:
            if stop == target:
                return buses_taken
            if stop not in visited_stops:
                visited_stops.add(stop)
                for neighbor_bus in stop_to_buses[stop]:
                    if neighbor_bus not in visited_buses:
                        visited_buses.add(neighbor_bus)
                        queue.append((neighbor_bus, buses_taken + 1))

    return -1

# Example usage:
# routes = [[1,2],[1,3,4]]
# print(numBusesToDestination(routes, 1, 4))

```

BFS · LeetCode — By Pattern

— 690 —

LeetCode

Quick Review — BFS 10 questions · insights · complexity · solution

#286 Walls and Gates [Medium]

Key Insights

- Use a multi-source Breadth-First Search (BFS) starting from all the gates, as this allows spreading the shortest distance to each empty room.
- Instead of starting from every empty room separately, starting from all gates simultaneously helps update all distances in one pass.
- Only update a room if the new calculated distance is smaller than its current value to prevent unnecessary processing.
- The problem is essentially a multi-source shortest path problem on a grid.

Space & Time

Time Complexity: $O(m \cdot n)$ where m and n are the dimensions of the grid, as each cell is processed at most once.
Space Complexity: $O(m \cdot n)$ in the worst-case scenario used by the BFS queue.

Solution

The solution uses a multi-source BFS algorithm by initially enqueueing all the gate positions (cells with value 0). Then, for each cell obtained from the queue, check its four possible neighbors (up, down, left, right). If a neighbor is an empty room (INF), update its distance to be the current cell's distance plus one, and then enqueue the neighbor to process further. This ensures that each empty room gets the minimum distance from any gate.

#322 Coin Change [Medium]

Key Insights

- Use a dynamic programming approach to build up a solution from smaller subproblems.
- Define $dp[i]$ as the minimum number of coins required for amount x .
- The recurrence relation is: $dp[x] = \min(dp[x], dp[x - \text{coin}] + 1)$ for each coin.
- Initialize $dp[0] = 0$ and set other values to a number larger than the maximum possible coins (amount+1 works as an infinity substitute).
- An alternative approach is through BFS, treating each amount as a node and each coin as an edge, but the DP method is more intuitive here.

Space & Time

Time Complexity: $O(\text{amount} \cdot n)$ where n is the number of coins.
Space Complexity: $O(\text{amount})$ for the dp array.

Solution

We use a bottom-up dynamic programming approach. The idea is to use a one-dimensional dp array where each element at index x represents the minimum number of coins required to form the amount x . Starting from $dp[0]$ which is 0, we iterate through all amounts up to the target amount and update $dp[x]$ based on each coin denomination. If after filling the dp array the value $dp[\text{amount}]$ remains unchanged (i.e., greater than amount), it means the amount cannot be formed and we return -1.

#542 01 Matrix [Medium]

Key Insights

- Treat all 0s as starting points for a multi-source BFS.
- Update each neighboring cell's distance if a shorter distance is found.
- Utilize a queue to process cells in order of increasing distance.
- Ensure that each cell is processed only once by marking visited cells.

Space & Time

BFS · LeetCode · By Pattern

— 691 —

LeetCode

Time Complexity: $O(m \cdot n)$ since each cell is processed once.

Space Complexity: $O(m \cdot n)$ for the queue and the answer matrix.

Solution

We use a multi-source Breadth-First Search (BFS) where every 0 in the matrix is initially added to the BFS queue. Each neighboring cell that is 1 will have its distance updated to be one plus the distance of the current cell if it hasn't been visited yet (or if a smaller distance is found). This ensures that the first time a 1 is reached, it is reached by the shortest path. Data structures used include a queue (or deque) for BFS and the matrix itself for storing distances.

#594 Rotting Oranges [Medium]

Key Insights

- Use Breadth-First Search (BFS) starting from all initially rotten oranges.
- Process the grid level by level, where each level represents one minute.
- Keep a count of fresh oranges and decrement it as they become rotten.
- If after the BFS there are still fresh oranges, then it is impossible to rot every orange.

Space & Time

Time Complexity: $O(m \cdot n)$ as every cell is processed at most once.
Space Complexity: $O(m \cdot n)$ in the worst case for the queue and auxiliary storage.

Solution

We begin by scanning the grid to locate all rotten oranges and count the fresh ones. These rotten oranges are added to a queue for a BFS. For each minute (BFS level), we process all the current rotten oranges and try to infect their adjacent fresh oranges. If an adjacent cell contains a fresh orange, we mark it rotten, add it to the queue, and decrement our fresh count. We continue this process level by level (minute by minute) until there are no fresh oranges left. If, after processing, there are still fresh oranges remaining, we return -1, indicating that not all oranges can become rotten.

#1197 Minimum Knight Moves [Medium]

Key Insights

- Leverage the symmetry of the chessboard by converting target coordinates to their absolute values.
- Use a Breadth-First Search (BFS) because all moves have equal weight, ensuring the first encounter with the target is the minimal path.
- Prune the search space by only considering positions that are likely beneficial (e.g., positions having coordinates not far less than -1).
- Track visited states to avoid redundant processing.
- Optimize by bounding the search region relative to the target distance.

Space & Time

Time Complexity: $O(n)$ where n is the number of positions processed during the BFS.
Space Complexity: $O(n)$ due to the storage requirements for the BFS queue and the visited set

Solution

We solve the problem using a BFS starting from the origin. First, normalize the target by taking the absolute value of x and y . BFS is then applied by exploring all eight possible knight moves at each step. Each move generates a new position that is enqueued if it hasn't been visited and falls within a reasonable bound (e.g., coordinates $x \geq -1$). The search terminates as soon as the target position is reached.

#1730 Shortest Path to Get Food [Medium]

Key Insights

- Use Breadth-First Search (BFS) to explore the grid level by level.

BFS · LeetCode · By Pattern

— 692 —

LeetCode

- The BFS guarantees that when a food cell ('F') is reached, the current distance is the shortest.
- Maintain a visited structure to avoid re-processing the same cell.
- Consider boundary and obstacle ('X') conditions while traversing.

Space & Time

Time Complexity: $O(m \cdot n)$, where m is the number of rows and n is the number of columns since each cell is processed at most once.
Space Complexity: $O(m \cdot n)$ in the worst case due to the queue and visited matrix.

Solution

We use Breadth-First Search (BFS) starting from the cell marked with '*'. The BFS uses a queue to explore neighbors in the four allowed directions (up, down, left, right). Each level of BFS represents one step in the path. As soon as a cell containing food ('F') is reached, the current step count is returned. We also maintain a visited matrix to ensure that cells are not revisited, which could otherwise result in cycles or redundant processing. This approach ensures that the first time we encounter food, we have found the shortest possible path.

#127 Word Ladder [Hard]

Key Insights

- Use Breadth-First Search (BFS) to explore transformations level by level to guarantee the shortest path.
- Preprocess words by generating intermediate representations (e.g., replacing each letter with a wildcard to quickly find neighbors).
- Use a visited set to avoid cycles.
- Each transformation counts as a level in the BFS, so the answer is the number of levels traversed.

Space & Time

Time Complexity: $O(N \cdot M^2)$ where N is the number of words and M is the length of each word.
Space Complexity: $O(N \cdot M)$ for the storage of intermediate mappings and the BFS queue.

Solution

We solve the problem using a BFS approach. First, we preprocess the wordlist by creating a mapping of all possible intermediate states. For example, for the word "dog" and wildcard position, we generate "o*", "d*", and "d*o". This helps us find all adjacent words (neighbors) that are only one letter different in constant time. We then initialize a queue with the `beginWord` and initiate the BFS. For each word dequeued, we explore all valid transformations using the precomputed intermediate mapping. If the `endWord` is reached during this process, we return the current level (transformation sequence length). Otherwise, we continue the BFS until all possibilities are exhausted. If no valid transformation sequence is found, we return 0.

#317 Shortest Distance from All Buildings [Hard]

Key Insights

- We need to compute the Manhattan distance from each building (cell value 1) to all empty lands (cell value 0) by performing a BFS.
- Maintain two matrices: one for cumulative total distances and another for the count of buildings that reached a particular empty cell.
- Ensure that only those empty cells reachable by every building are considered.
- The ideal cell will have a reach count equal to the total number of buildings.
- If multiple buildings cannot all reach any specific empty cell, then return -1.

Space & Time

Time Complexity: $O(k \cdot m \cdot n)$ where k is the number of buildings and m, n are the grid dimensions.
Space Complexity: $O(m \cdot n)$ due to the auxiliary matrices and visited state tracking used in BFS.

Solution

We first iterate through the grid to locate all buildings while counting them. For each building, we perform a BFS to compute the shortest distance to each reachable empty cell. During the BFS, we update the cumulative distance for

BFS · LeetCode · By Pattern

— 693 —

LeetCode

that cell and record that it was reached by one additional building. After processing all the buildings, we scan the grid to find the empty cell that is reachable from all buildings and has the minimum cumulative distance. If no such cell exists, we return -1.

#773 Sliding Puzzle [Hard]

Key Insights

- Represent the board as a string (e.g., "123450") to simplify state comparison and manipulation.
- Use Breadth-First Search (BFS) to explore all possible moves since the puzzle has a small fixed state space ($6! = 720$ possible states).
- Precompute the valid neighbor indices for each board position to efficiently generate new states.
- Track visited states to avoid loops and redundant work.

Space & Time

Time Complexity: $O(1)$ in practice, since there are at most 720 states ($6!$ permutations).
Space Complexity: $O(1)$ for the same reason, as the state space is fixed and limited.

Solution

We solve the problem using a Breadth-First Search (BFS) approach. The board configuration is flattened into a string to represent states. A mapping from each index to its possible neighbors (i.e., positions to which 0 can move) is precomputed. BFS is then used to explore every valid move from the initial state while marking states as visited to avoid cycles. When the goal state ("123450") is reached, we return the number of moves taken. If BFS completes without reaching the goal, the puzzle is unsolvable and we return -1.

#815 Bus Routes [Hard]

Key Insights

- Model the problem as a graph where nodes are bus routes.
- Two bus routes are connected if they share at least one common bus stop.
- Use a hash map to quickly look up bus routes available at any bus stop.
- Employ Breadth-First Search (BFS) on the bus routes graph to find the minimum number of bus transfers.
- Mark visited bus routes and bus stops to avoid redundant work.

Space & Time

Time Complexity: $O(N \cdot L)$ in the worst-case scenario, where N is the number of bus routes and L is the average number of stops per route.
Space Complexity: $O(N \cdot L)$ due to storage of the stop-to-buses mapping and BFS auxiliary data structures.

Solution

We begin by constructing a mapping from each bus stop to the list of buses passing through it. This allows quick retrieval of buses available at a given stop. Starting at the source stop, initialize the BFS queue with all buses that can be boarded at that stop. Then, for each bus route in the queue, inspect each stop along that route. If a stop is the target, return the number of buses taken so far. Otherwise, for each unvisited stop, add all adjacent bus routes (buses that pass through this stop) to the BFS queue with an incremented bus count. Continue until the queue is empty, which means the target cannot be reached.

```
def longestCommonPrefix(strs):
    if not strs:
        return ""
    if not strs:
        return ""
    # Iterate over each index of the first string
    for i in range(len(strs[0])):
        current_char = strs[0][i]
        # Compare the current character with the corresponding one in the rest of the strings
        for j in range(1, len(strs)):
            if i == len(strs[j]) or strs[j][i] != current_char:
                return strs[0][:i]
    # If no mismatch is found, return the complete first string
    return strs[0]

# Example usage:
print(longestCommonPrefix(["flower", "flow", "flight"]))
```

```
Python
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        if not strs:
            return ""
        for i in range(len(strs[0])):
            for j in range(1, len(strs)):
                if i == len(strs[j]) or strs[j][i] != strs[0][i]:
                    return strs[0][:i]
        return strs[0]
```

```
Python
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        for i in range(len(strs[0])):
            for j in range(1, len(strs)):
                if i == len(strs[j]) or strs[j][i] != strs[0][i]:
                    return strs[0][:i]
        return strs[0]
```

```
Python
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        for i in range(len(strs[0])):
            for j in range(1, len(strs)):
                if i == len(strs[j]) or strs[j][i] != strs[0][i]:
                    return strs[0][:i]
        return strs[0]
```

C++

C++

Trie · LeetCode · By Pattern

— 696 —

LeetCode

#10 Trie — 12 questions · #10 of 21

#14

EASY

Longest Common Prefix

LeetCode · Simplify · LeetCode · LeetCode · Editorial

String · Trie

List: Grid 169

Problem:

Write a function to find the longest common prefix amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:

Input: strs = ["flower", "flow", "flight"]

Output: "fl"

Example 2:

Input: strs = ["dog", "racecar", "car"]

Output: ""

Explanation: There is no common prefix among the input strings.

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- `strs[i]` consists of only lowercase English letters if it is non-empty.

>> Brute Force [Trie] Interview Prep

Python

```
# A naive Brute Force approach
class Solution:
    def longestCommonPrefix(self, strs):
        # Base case: if there is only one string, return it
        if len(strs) == 1:
            return strs[0]
        # If not, return ""
        prefix = strs[0]
        for word in strs[1:]:
            while not word.startswith(prefix):
                prefix = prefix[:-1]
            if not prefix:
                return ""
        return prefix
```

Python

Python

BFS · LeetCode · By Pattern

— 695 —

LeetCode

Trie · LeetCode · By Pattern

— 696 —

LeetCode


```

class Solution {
public:
    string longestCommonPrefix(vector<string& str) {
        if (str.empty())
            return "";
        for (int i = 0; i < str[0].length(); ++i)
            for (int j = 1; j < str.size(); ++j)
                if (i == str[j].length() || str[j][i] != str[0][i])
                    return str[0].substr(0, i);
        return str[0];
    }
};

```

C++

```

class Solution {
public:
    string longestCommonPrefix(vector<string& str) {
        int n = str.size();
        for (int i = 0; i < str[0].size(); ++i) {
            for (int j = 1; j < n; ++j)
                if (str[j].size() == i || str[j][i] != str[0][i])
                    return str[0].substr(0, i);
        }
        return str[0];
    }
};

```

C++

```

class Solution {
//
// @param String[] str
// @return String
//
function longestCommonPrefix(strs) {
    str = "";
    for (let i = 0; i < str[0].length(); ++i) {
        for (let j = 1; j < str.length; ++j) {
            if (str[j].length() < i || str[j][i] != str[0][i])
                return str;
        }
        str = str + str[0][i];
    }
    return str;
}

```

C++

```

class Solution {
public:
    string longestCommonPrefix(vector<string& str) {
        int n = str.size();
        for (int i = 0; i < str[0].size(); ++i) {
            for (int j = 1; j < n; ++j)
                if (str[j].size() == i || str[j][i] != str[0][i])
                    return str[0].substr(0, i);
        }
        return str[0];
    }
};

```

C++

```

class Solution {
//
// @param String[] str
// @return String
//
function longestCommonPrefix(strs) {
    str = "";
    for (let i = 0; i < str[0].length(); ++i) {
        for (let j = 1; j < str.length; ++j) {
            if (str[j].length() < i || str[j][i] != str[0][i])
                return str;
        }
        str = str + str[0][i];
    }
    return str;
}

```

Java

```

class Solution {
//
// @param String[] str
// @return String
//
function longestCommonPrefix(strs) {
    str = "";
    for (let i = 0; i < str[0].length(); ++i) {
        for (let j = 1; j < str.length; ++j) {
            if (str[j].length() < i || str[j][i] != str[0][i])
                return str;
        }
        str = str + str[0][i];
    }
    return str;
}

```

Java

```

class Solution {
public:
    string longestCommonPrefix(vector<string& str) {
        int n = str.size();
        for (int i = 0; i < str[0].length(); ++i) {
            for (int j = 1; j < n; ++j)
                if (str[j].length() == i || str[j][i] != str[0][i])
                    return str[0].substr(0, i);
        }
        return str[0];
    }
};

```

Java

```

public class Solution {
    public String longestCommonPrefix(String[] str) {
        int n = str.length;
        for (int i = 0; i < str[0].length; ++i) {
            for (int j = 1; j < n; ++j)
                if (i == str[j].length() || str[j][i] != str[0][i])
                    return str[0].substring(0, i);
        }
        return str[0];
    }
}

```

Java

```

class Solution {
//
// @param String[] str
// @return String
//
function longestCommonPrefix(strs) {
    str = "";
    for (let i = 0; i < str[0].length(); ++i) {
        for (let j = 1; j < n; ++j)
            if (str[j].length() == i || str[j][i] != str[0][i])
                return str[0].substr(0, i);
        str = str + str[0][i];
    }
    return str;
}

```

Java

```

public class Solution {
    public String longestCommonPrefix(String[] str) {
        int n = str.length;
        for (int i = 0; i < str[0].length; ++i) {
            for (int j = 1; j < n; ++j)
                if (i == str[j].length() || str[j][i] != str[0][i])
                    return str[0].substring(0, i);
        }
        return str[0];
    }
}

```

JavaScript

```

//
// @param (string[]) str
// @return (string)
//
var longestCommonPrefix = function (strs) {
    for (let i = 0; i < str[0].length; ++i) {
        for (let j = 1; j < str.length; ++j) {
            if (str[j][i] != str[0][i]) {
                return str[0].substring(0, i);
            }
        }
        str = str + str[0][i];
    }
    return str;
}

```

```

function
# @param (String[]) str
# @return (String)
def longest_common_prefix(strs):
    return "" if str.empty() || str.length.zero?

    return str[0] if str.length == 1

    idx = 0
    while idx < str[0].length
        cur_char = str[0][idx]

        str_idx = 1
        while str_idx < str.length
            return idx + 1 if str[str_idx][idx] != cur_char && idx.zero?
            return str[idx][0..idx - 1] if str[str_idx][idx] != cur_char
            str_idx += 1
        end

        idx += 1
    end

    idx > 0 ? str[0][0..idx] : ""
end

```

```

JavaScript
//
// @param (string[]) str
// @return (string)
//
var longestCommonPrefix = function (strs) {
    for (let i = 0; i < str[0].length; ++i) {
        for (let j = 1; j < str.length; ++j)
            if (str[j][i] != str[0][i])
                return str[0].substring(0, i);
        str = str + str[0][i];
    }
    return str;
}

```

```

JavaScript
# @param (String[]) str
# @return (String)
def longest_common_prefix(strs):
    return "" if str.empty() || str.length.zero?

    return str[0] if str.length == 1

    idx = 0
    while idx < str[0].length
        cur_char = str[0][idx]

        str_idx = 1
        while str_idx < str.length
            return idx + 1 if str[str_idx][idx] != cur_char && idx.zero?
            return str[idx][0..idx - 1] if str[str_idx][idx] != cur_char
            str_idx += 1
        end

        idx += 1
    end

    idx > 0 ? str[0][0..idx] : ""
end

```

TypeScript

```

function longestCommonPrefix(strs: string[]): string {
    const len = str.reduce((r, s) => Math.min(r, s.length), Infinity);
    for (let i = 0; i < len; ++i) {
        const target = str[0].slice(0, i);
        if (strs.every(s => s.slice(0, i) === target)) {
            return target;
        }
    }
    return "";
}

```

```

function longestCommonPrefix(strs: string[]): string {
    const len = str.reduce((r, s) => Math.min(r, s.length), Infinity);
    for (let i = 0; i < len; ++i) {
        const target = str[0].slice(0, i);
        if (strs.every(s => s.slice(0, i) === target)) {
            return target;
        }
    }
    return "";
}

```

Go

```

func longestCommonPrefix(strs []string) string {
    n := len(strs)
    for i := range str[0] {
        for j := 1; j < n; j++ {
            if len(strs[j]) == i || str[j][i] != str[0][i] {
                return str[0][:i]
            }
        }
        return str[0]
    }
}

```

Go

```

func longestCommonPrefix(strs []string) string {
    n := len(strs)
    for i := range str[0] {
        for j := 1; j < n; j++ {
            if len(strs[j]) == i || str[j][i] != str[0][i] {
                return str[0][:i]
            }
        }
        return str[0]
    }
}

```

Rust

```

impl Solution {
    pub fn longest_common_prefix(strs: Vec<String>) -> String {
        let mut len = str[0].len().min(s.len().min(s.len()).unwrap());
        for i in 1..len().rev() {
            let mut is_equal = true;
            let target = str[0][0..i].to_string();
            if str.iter().all(|s| target == s[0..i]) {
                return target;
            }
        }
        String::new()
    }
}

```

Rust

```

1 def longestCommonPrefix(strs: Vector[String]) -> String {
2   let mut len = strs
3     .iterator()
4     .map(s => s.length)
5     .min()
6   for i in 1..len.row() {
7     let mut is_equal = true;
8     let target = strs[0][0..i].to_string();
9     if strs.iterator().all(s => s[0..i] == target) {
10      return target;
11    }
12    String.empty()
13  }
14 }

```

[7] Trie - Pattern Solution (stored optimal)

Python

```

1 def longestCommonPrefix(strs):
2     # If no strings exist, return empty
3     if not strs:
4         return ""
5     # Iterate over each index of the first string
6     for i in range(len(strs[0])):
7         current_char = strs[0][i]
8         # Compare the current character with the corresponding one in the rest of the strings
9         for s in strs[1:]:
10            # If same character length or characters differ, return the prefix found so far
11            if i == len(s) or s[i] != current_char:
12                return strs[0][:i]
13        # If no mismatch is found, return the complete first string
14        return strs[0]
15
16 # Example usage:
17 print(longestCommonPrefix(["flower", "flow", "flight"]))

```

C++

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 using namespace std;
5
6 string longestCommonPrefix(vector<string>& strs) {
7     // Return empty string if array is empty
8     if (strs.empty()) {
9         return "";
10    }
11    // Loop over the first string's characters
12    for (int i = 0; i < strs[0].size(); i++) {
13        char currentChar = strs[0][i];
14        // Compare currentChar with the same index in each other string
15        for (int j = 1; j < strs.size(); j++) {
16            // If same character, continue; if characters differ, return the prefix
17            if (i == strs[j].size() || strs[j][i] != currentChar) {
18                return strs[0].substr(0, i);
19            }
20        }
21    }
22    return strs[0];
23 }
24
25 int main() {
26     vector<string> strs = {"flower", "flow", "flight"};
27     cout << longestCommonPrefix(strs) << endl;
28     return 0;
29 }

```

#139 MEDIUM

Word Break

LeetCode - Simple - Word Break - LeetCode - Editorial

Array - Hash Table - String - Dynamic Programming - Trie - Memoization

LeetCode 139

Problem:

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```

Input: s = "leetcode", wordDict = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".

```

Example 2:

```

Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.

```

Example 3:

```

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false

```

Constraints:

- 1 <= s.length <= 300
- 1 <= wordDict.length <= 1000
- 1 <= wordDict[i].length <= 20
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.

>> Brute Force (Trie) Interview Prep

Python

```

1 # Brute Force Approach
2 class Solution:
3     def wordBreak(self, s: str, wordDict: List[str]) -> bool:
4         # Brute Force O(2^n) - try all partition points recursively
5         wordSet = set(wordDict)
6         def canBreak(start):
7             if start == len(s): return True
8             for end in range(start + 1, len(s) + 1):
9                 if s[start:end] in wordSet and canBreak(end):
10                    return True
11             return False
12         return canBreak(0)

```

Python

```

1 # Python solution for the Word Break problem
2 def wordBreak(s, wordDict):
3     # Convert the list to a set for O(1) look-ups
4     wordSet = set(wordDict)
5
6     # dp[i] is True if s[0:i] can be segmented into words from the wordDict
7     n = len(s)
8     dp = [False] * (n + 1)
9
10    # Base case: empty string is always segmentable
11    dp[0] = True
12
13    # Iterate through the string positions
14    for i in range(1, n + 1):
15        # Check every possible partition s[0:j] and s[j:i]
16        for j in range(i):
17            # If s[0:j] can be segmented and s[j:i] is in the dictionary
18            if dp[j] and s[j:i] in wordSet:
19                dp[i] = True # s[0:i] can be segmented
20                break # No need to check further partitions
21        return dp[i]
22
23 # Example usage:
24 s = "leetcode"

```

```

1 wordDict = ["leet", "code"]
2 print(wordBreak(s, wordDict)) # Output: True
3
4 Python
5
6 class Solution:
7     def wordBreak(self, s: str, wordDict: List[str]) -> bool:
8         wordSet = set(wordDict)
9
10        @functools.lru_cache(None)
11        def wordBreak(s: str) -> bool:
12            # Base case: if s is empty, it can be segmented.
13            if s == "":
14                return True
15            for i in range(1, len(s) + 1):
16                if s[:i] in wordSet and wordBreak(s[i:]) for i in range(len(s)):
17                    return wordBreak(s)

```

Python

```

1 class Solution:
2     def wordBreak(self, s: str, wordDict: List[str]) -> bool:
3         words = set(wordDict)
4         n = len(s)
5         f = [True] * (n + 1)
6         for i in range(1, n + 1):
7             f[i] = wordDict and s[:i] in words for j in range(i)
8         return f[n]

```

Python

```

1 class Trie:
2     def __init__(self):
3         self.children = List[Trie | None] = [None] * 26
4         self.isEnd = False
5
6     def insert(self, w: str):
7         node = self
8         for c in w:
9             idx = ord(c) - ord('a')
10            if not node.children[idx]:
11                node.children[idx] = Trie()
12            node = node.children[idx]
13            node.isEnd = True
14
15 class Solution:
16     def wordBreak(self, s: str, wordDict: List[str]) -> bool:
17         trie = Trie()
18         for w in wordDict:
19             trie.insert(w)
20         n = len(s)
21         dp = [False] * (n + 1)
22         f[n] = True
23         for i in range(n - 1, -1, -1):
24             node = trie

```

```

1 for j in range(i, n):
2     idx = ord(s[j]) - ord('a')
3     if not node.children[idx]:
4         break
5     node = node.children[idx]
6     if node.isEnd and f[j + 1]:
7         f[i] = True
8         break
9     return f[i]

```

Python

```

1 class Solution:
2     def wordBreak(self, s: str, wordDict: List[str]) -> bool:
3         words = set(wordDict)
4         n = len(s)
5
6         # dp[i] meaning s from 0 to index i-1 is breakable
7         dp = [True] * (n + 1)
8         for i in range(1, n + 1):
9             dp[i] = any(dp[j] and s[j:i] in words for j in range(i))
10        return dp[n]
11
12 =====
13 class Solution:
14     def wordBreak(self, s: str, wordDict: Set[str]) -> bool:
15         if not s or not wordDict:
16             return False
17
18         length = len(s)
19
20         # construct dp. dp[i] meaning position i-1 can from dict or not
21         dp[0] meaning at last index plus 1, checking up to last index
22         dp = [False] * (length + 1)
23         dp[0] = True # initiate

```

```

1 for i in range(length + 1):
2     if not dp[i]:
3         continue
4     for word in wordDict:
5         if i < len(word) + length:
6             continue
7         if i + len(word) == length:
8             dp[i + len(word)] = word
9             dp[i + len(word)] = True
10        return dp[length]

```

=====

```

1 class Solution:
2     def wordBreak(self, s: str, wordDict: List[str]) -> bool:
3         words = set(wordDict)
4         n = len(s)
5         dp = [False] * (n + 1)
6         # dp[i] meaning s from 0 to index i-1 is breakable
7         for i in range(1, n + 1):
8             dp[i] = any(dp[j] and s[j:i] in words for j in range(i))
9         return dp[n]

```

=====

```

1 class Trie:
2     def __init__(self):
3         self.children = [None] * 26
4         self.is_end = False
5
6     def insert(self, w):
7         node = self
8         for c in w:
9             idx = ord(c) - ord('a')
10            if not node.children[idx]:
11                node.children[idx] = Trie()
12            node = node.children[idx]
13            node.is_end = True
14
15     def search(self, w):
16         node = self
17         for c in w:
18             idx = ord(c) - ord('a')
19            if not node.children[idx]:
20                return False
21            node = node.children[idx]
22
23        return node.is_end
24
25 class Solution:
26     def wordBreak(self, s: str, wordDict: List[str]) -> bool:
27         # Trie + DP + DFS + Memoization + DFS + DFS + DFS + DFS

```

```

C++
C++

class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        return wordBreak(s, wordDict.begin(), wordDict.end(), {});
    }
};

private:
bool wordBreak(const string& s, const unordered_set<string>& wordSet,
               unordered_map<string, bool>& mem) {
    if (wordSet.contains(s))
        return true;
    if (const auto it = mem.find(s); it != mem.end())
        return it->second;

    // i = prefix.length() = s.length()
    for (int i = 1; i < s.length(); ++i) {
        const string prefix = s.substr(0, i);
        const string suffix = s.substr(i);
        if (wordSet.contains(prefix) &&
            wordBreak(suffix, wordSet, mem))
            return mem[s] = true;
    }

    return mem[s] = false;
}

};

C++
C++

class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
        int n = s.size();
        bool f[n+1];
        memset(f, false, sizeof(f));
        f[0] = true;
        for (int i = 1; i <= n; ++i) {
            for (int j = 0; j < i; ++j) {
                if (f[j] && wordSet.contains(s.substr(j, i - j))) {
                    f[i] = true;
                    break;
                }
            }
        }
        return f[n];
    }
};

};

C++
C++

```

```

// 01: https://leetcode.com/problems/word-break/
// Time: O(N^3)
// Space: O(N)
class Solution {
public:
    bool wordBreak(string s, vector<string>& dict) {
        unordered_set<string> st(begin(dict), end(dict));
        int n = s.size();
        vector<bool> dp(n + 1);
        dp[0] = true;
        for (int i = 1; i <= n; ++i) {
            for (int j = 0; j < i && dp[j]; ++j) {
                dp[i] = dp[j] && st.contains(s.substr(j, i - j));
            }
        }
        return dp[n];
    }
};

Java
Java

class Solution {
public boolean wordBreak(String s, List<String> wordDict) {
    return wordBreak(s, new HashSet<String>(wordDict), new HashMap<>());
}

private boolean wordBreak(final String s, Set<String> wordSet, Map<String, Boolean> mem) {
    if (mem.containsKey(s))
        return mem.get(s);
    if (wordSet.contains(s)) {
        mem.put(s, true);
        return true;
    }

    // i = prefix.length() = s.length()
    for (int i = 1; i < s.length(); ++i) {
        final String prefix = s.substring(0, i);
        final String suffix = s.substring(i);
        if (wordSet.contains(prefix) && wordBreak(suffix, wordSet, mem)) {
            mem.put(s, true);
            return true;
        }
    }

    mem.put(s, false);
    return false;
}
}

Java
Java

```

```

class Solution {
public:
    bool wordBreak(string s, List<String> wordDict) {
        Set<String> words = new HashSet<>(wordDict);
        int n = s.length();
        boolean[] f = new boolean[n + 1];
        f[0] = true;
        for (int i = 1; i <= n; ++i) {
            for (int j = 0; j < i; ++j) {
                if (f[j] && words.contains(s.substr(j, i - j))) {
                    f[i] = true;
                    break;
                }
            }
        }
        return f[n];
    }
};

Java
Java

impl Solution {
    pub fn word_break(s: String, word_dict: Vec<String>) -> bool {
        let words: std::collections::HashSet<String> = word_dict.iter().collect();
        let mut f = vec![false; s.len() + 1];
        f[0] = true;
        for i in 1..=s.len() {
            for j in 0..i {
                if f[j] && words.contains(&s[j..i]);
            }
        }
        f[s.len()]
    }
}

Java
Java

public class Solution {
public:
    bool wordBreak(string s, List<String> wordDict) {
        var words = new HashSet<String>(wordDict);
        int n = s.length();
        var f = new boolean[n + 1];
        f[0] = true;
        for (int i = 1; i <= n; ++i) {
            for (int j = 0; j < i; ++j) {
                if (f[j] && words.Contains(s.Substring(j, i - j))) {
                    f[i] = true;
                    break;
                }
            }
        }
        return f[n];
    }
};

Java
Java

```

```

import java.util.List;
import java.util.Queue;
import java.util.Set;

public class Word_Break {

    /*
    input = 1;
    =====
    ["a","aa","aaa","aaaa","aaaaa","aaaaaa","aaaaaaa","aaaaaaaa","aaaaaaaaa","aaaaaaaaa"]
    =====
    input = 2;
    =====
    ["a","aa","aaa","aaaa","aaaaa","aaaaaa","aaaaaaa","aaaaaaaa","aaaaaaaaa","aaaaaaaaa"]
    =====
    */

    public class Solution_dp {
        public boolean wordBreak(String s, Set<String> dict) {
            if (s == null || dict == null || dict.size() == 0) return false;

            int length = s.length();

            // since both dfs and bfs not working, try dynamic programming here
            construct dp, dp[i] meaning position i-1 can form dict or not
            boolean[] dp = new boolean[length + 1];
            dp[0] = true; // base: initiate

            for (int i = 0; i < length + 1; i++) {
                if (dp[i] == false) {
                    continue;
                } else { // if some previous dict word ending at index i-1
                    for (String each: dict) {
                        if (i + each.length() > length) continue;

                        if (s.substring(i, i + each.length()).equals(each)) {
                            dp[i + each.length()] = true;
                            // i = i + each.length() - 1, // for later
                            // break;
                        }
                    }
                }
            }
        }
    }
}

```

```

        return dp[length];
    }
}

public class Solution_bfs_over_time {
public:
    bool wordBreak(string s, Set<String> wordDict) {
        if (s == null || s.length() == 0 || wordDict == null || wordDict.size() == 0) {
            return false;
        }

        // since dfs is not working, now try bfs, all valid word's substring sequence
        Queue<String> q = new LinkedList<>();
        q.offer(s);

        while (!q.isEmpty()) {
            String current = q.poll();

            // if (current.length() == 0) { // missing all previous matched in dict

            // return true;
            // }

            for (int i = 0; i < current.length(); i++) {
                return dp[length];
            }
        }

        public class Solution {
        public:
            bool wordBreak(string s, List<String> wordDict) {
                var words = new HashSet<String>(wordDict);
                int n = s.length();
                var dp = new bool[n + 1];
                dp[0] = true;
                for (int i = 1; i <= n; ++i) {
                    for (int j = 0; j < i; ++j) {
                        if (dp[j] && words.Contains(s.Substring(j, i - j))) {
                            dp[i] = true;
                            break;
                        }
                    }
                }
                return dp[n];
            }
        }

        Java
        Java

        public class Solution {
        public:
            bool wordBreak(string s, List<String> wordDict) {
                var words = new HashSet<String>(wordDict);
                int n = s.length();
                var dp = new bool[n + 1];
                dp[0] = true;
                for (int i = 1; i <= n; ++i) {
                    for (int j = 0; j < i; ++j) {
                        if (dp[j] && words.Contains(s.Substring(j, i - j))) {
                            dp[i] = true;
                            break;
                        }
                    }
                }
                return dp[n];
            }
        }

        Java
        Java
    
```

```

impl Solution {
    pub fn word_break(s: String, word_dict: Vec<String>) -> bool {
        let words: std::collections::HashSet<String> = word_dict.iter().collect();
        let mut f = vec![false; s.len() + 1];
        f[0] = true;
        for i in 1..=s.len() {
            for j in 0..i {
                if f[j] && words.contains(&s[j..i]);
            }
        }
        f[s.len()]
    }
}

TypeScript
TypeScript

function wordBreak(s: string, wordDict: string[]): boolean {
    const words = new Set(wordDict);
    const n = s.length;
    const f: boolean[] = new Array(n + 1).fill(false);
    f[0] = true;
    for (let i = 1; i <= n; ++i) {
        for (let j = 0; j < i; ++j) {
            if (f[j] && words.has(s.substr(j, i - j))) {
                f[i] = true;
                break;
            }
        }
    }
    return f[n];
}

TypeScript
TypeScript

function wordBreak(s: string, wordDict: string[]): boolean {
    const words = new Set(wordDict);
    const n = s.length;
    const f: boolean[] = new Array(n + 1).fill(false);
    f[0] = true;
    for (let i = 1; i <= n; ++i) {
        for (let j = 0; j < i; ++j) {
            if (f[j] && words.has(s.substr(j, i - j))) {
                f[i] = true;
                break;
            }
        }
    }
    return f[n];
}

Go
Go

```

```

func wordBreak(s: String, wordDict: [String]) bool {
    words := map[string]bool{}
    for w := range wordDict {
        words[w] = true
    }
    n := len(s)
    f := make([]bool, n+1)
    f[0] = true
    for i := 1; i <= n; i++ {
        for j := 0; j < i; j++ {
            if f[j] && words[s[j:i]] {
                f[i] = true
                break
            }
        }
    }
    return f[n]
}

```

Go

```

func wordBreak(s string, wordDict []string) bool {
    words := make(map[string]bool)
    for w := range wordDict {
        words[w] = true
    }
    n := len(s)
    dp := make([]bool, n+1)
    dp[0] = true
    for i := 1; i <= n; i++ {
        for j := 0; j < i; j++ {
            if dp[j] && words[s[j:i]] {
                dp[i] = true
                break
            }
        }
    }
    return dp[n]
}

```

[*] Trie — Pattern Solution (matched from community answers)

Python

Trie · LeetCode · By Pattern

— 715 —

LeetCode

```

class Trie:
    def __init__(self):
        self.children = List[Trie | None] = [None] * 26
        self.isEnd = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if not node.children[idx]:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.isEnd = True

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        n = len(s)
        f = [False] * (n + 1)
        f[0] = True
        for i in range(1, n + 1):
            node = trie
            for j in range(0, i):
                idx = ord(s[j]) - ord('a')
                if not node.children[idx]:
                    break
                node = node.children[idx]
                if node.isEnd and f[j]:
                    f[i] = True
                    break
            return f[i]

```

#208

MEDIUM

Implement Trie (Prefix Tree)

LeetCode · Simplest · WalkC · LeetCode · Editorial

Hash Table · String · Design · Trie

List: Grind 169

Problem:

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string word into the trie.
- `boolean search(String word)` Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.

Trie · LeetCode · By Pattern

— 716 —

LeetCode

- `boolean startsWith(String prefix)` Returns true if there is a previously inserted string word that has the prefix prefix, and false otherwise.

Example 1:

```

Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
Output
[null, null, true, false, true, null, true]

```

Explanation

```

Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // return True
trie.search("app"); // return False
trie.startsWith("app"); // return True
trie.insert("app");
trie.search("app"); // return True

```

Constraints:

- `1 <= word.length, prefix.length <= 2000`
- word and prefix consist only of lowercase English letters.
- At most `3 * 104` calls in total will be made to insert, search, and startsWith.

Python

Python

```

class TrieNode:
    def __init__(self):
        # dictionary to store child nodes and a flag for end of word.
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        # Initialize the Trie with the root node.
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        # Insert the word into the Trie.
        node = self.root
        for char in word:
            # If character doesn't exist, add a new TrieNode.
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        # Mark the end of word.
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        # Search the Trie for each character.
        node = self.root

```

Trie · LeetCode · By Pattern

— 717 —

LeetCode

```

for char in word:
    if char not in node.children:
        return False
    node = node.children[char]
# Return true if word ends exactly here.
return node.is_end_of_word

def startsWith(self, prefix: str) -> bool:
    # Traverse the Trie for the prefix.
    node = self.root
    for char in prefix:
        if char not in node.children:
            return False
        node = node.children[char]
    return True

```

Python

```

class TrieNode:
    def __init__(self):
        self.children = dict[str, TrieNode] = {}
        self.isEnd = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = TrieNode
        self.root = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
            node.isEnd = True

    def search(self, word: str) -> bool:
        node = self._findNode(word)
        return node is not None and node.isEnd

    def startsWith(self, prefix: str) -> bool:
        return self._findNode(prefix) is not None

    def _findNode(self, prefix: str) -> TrieNode | None:
        node = TrieNode = self.root

        for c in prefix:
            if c not in node.children:
                return None
            node = node.children[c]
        return node

```

Python

Trie · LeetCode · By Pattern

— 718 —

LeetCode

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if not node.children[idx]:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self._search_prefix(word)
        return node is not None and node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self._search_prefix(prefix)
        return node is not None

    def _search_prefix(self, prefix: str):
        node = self
        for c in prefix:
            idx = ord(c) - ord('a')
            if not node.children[idx]:
                return None
            node = node.children[idx]
        return node

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

Python

Trie · LeetCode · By Pattern

— 719 —

LeetCode

```

import collections

class TrieNode:
    def __init__(self):
        # Usually, a Python dictionary throws a KeyError if you try to get an item with a key that is not currently in the dictionary.
        # The defaultdict in contrast will simply create any items that you try to access.
        # https://stackoverflow.com/questions/5900778/how-does-collections-defaultdict-work
        self.child = collections.defaultdict(TrieNode)
        self.is_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        cur = self.root
        for letter in word:
            cur = cur.child[letter]
        cur.is_word = True

    def search(self, word: str) -> bool:
        cur = self.root
        for letter in word:
            # cur = cur.child[letter] may not work, it will create a default node for this letter
            # If not cur:
            #     return False
            return cur.is_word

    def startsWith(self, prefix: str) -> bool:
        cur = self.root
        for letter in prefix:
            # cur = cur.child[letter] may not work, it will create a default node for this letter
            # If not cur:
            #     return False
            return True

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

Trie · LeetCode · By Pattern

— 720 —

LeetCode

```

// =====
class Trie {
  def __init__():
    self.children = defaultdict(lambda: None)
    self.is_end = False

  def insert(self, word: str) -> None:
    node = self
    for c in word:
      idx = ord(c) - ord('a')
      if node.children[idx] is None:
        node.children[idx] = Trie()
      node = node.children[idx]
    node.is_end = True

  def search(self, word: str) -> bool:
    node = self
    search_prefix(word)
    return node is not None and node.is_end

  def startswith(self, prefix: str) -> bool:
    node = self
    search_prefix(prefix)
    return node is not None

  def search_prefix(self, prefix: str):
    node = self
    for c in prefix:
      idx = ord(c) - ord('a')
      if node.children[idx] is None:
        return None
}

```

Java
Java

```

public class Implement_Trie_Prefix_Tree {
  public static void main(String[] args) {
    Implement_Trie_Prefix_Tree out = new Implement_Trie_Prefix_Tree();

    Trie trie = out.new Trie();
    trie.insert("abc");
    trie.insert("ab");
    System.out.println(trie.search("ab"));
    trie.insert("ab");
    System.out.println(trie.search("ab"));
    System.out.println(trie.startsWith("a"));
  }

  class TrieNode {
    // 26 children to node children
    private TrieNode[] children;

    private final int R = 26;

    private boolean isEnd;

    public TrieNode() {
      children = new TrieNode[R];
    }

    public boolean containsKey(char ch) {
      return children[ch - 'a'] != null;
    }

    public TrieNode get(char ch) {
      return children[ch - 'a'];
    }

    public void put(char ch, TrieNode node) {
      children[ch - 'a'] = node;
    }

    public void setEnd() {
      isEnd = true;
    }

    public boolean isEnd() {
      return isEnd;
    }
  }

  class Trie {

```

```

private TrieNode root;

public Trie() {
  root = new TrieNode();
}

// Insert a word into the trie
public void insert(String word) {
  TrieNode node = root;

  // Note: inner loop iteration is better than recursion during trie building
  for (int i = 0; i < word.length(); i++) {
    char currentChar = word.charAt(i);
    if (!node.containsKey(currentChar)) {
      node.put(currentChar, new TrieNode());
    }
    node = node.get(currentChar);
    node.setEnd();
  }

  // search a prefix or whole key in trie and
  // return the node where search ends
  private TrieNode searchPrefix(String word) {
    TrieNode node = root;

    for (int i = 0; i < word.length(); i++) {
      char currentChar = word.charAt(i);
      if (!node.containsKey(currentChar)) {
        return null;
      } else {
        node = node.get(currentChar);
      }
    }
  }

  public boolean search(String word) {
    TrieNode node = searchPrefix(word);
    return node != null && node.isEnd;
  }

  public boolean startswith(String prefix) {

```

Java

```

TrieNode = SearchPrefix(prefix):
  return node != null

private Trie SearchPrefix(String s) {
  TrieNode = this;
  for (var c in s) {
    var idx = c - 'a';
    if (node.children[idx] == null) {
      return null;
    }
    node = node.children[idx];
  }
  return node;
}

// Your Trie object will be instantiated and called as such:
// Trie obj = new Trie();
// obj.insert(word);
// bool param_1 = obj.search(word);
// bool param_2 = obj.startsWith(prefix);
}

```

Java

```

struct Trie {
  root: HashMap<TrieNode>,
}

// Your Trie object will be instantiated and called as such:
// let obj = new Trie();
// obj.insert(word);
// let param_1 = obj.search(word);
// let param_2 = obj.startsWith(prefix);

impl Trie {
  fn new() -> Self {
    Self {
      root: HashMap::new(),
    }
  }

  fn insert(&self, word: String) {
    let char_vec: Vec<char> = word.chars().collect();
    // Get the clone of current root node
    let mut root = &self.root;
    for c in char_vec {
      if (!root.borrow().child.contains_key(c)) {
        // We need to manually create the entry
        root.borrow_mut().child.insert(c, &self.new(TrieNode::new()));
      }
      // Get the child node
      let root_clone = &self.root.borrow().child.get(c).unwrap();
      root = root_clone;
    }
    root.borrow_mut().flag = true;
  }

  fn search(&self, word: String) -> bool {
    let char_vec: Vec<char> = word.chars().collect();
    // Get the clone of current root node
    let mut root = &self.root;
    for c in char_vec {
      if (!root.borrow().child.contains_key(c)) {
        return false;
      }
      // Get the child node
      let root_clone = &self.root.borrow().child.get(c).unwrap();
      root = root_clone;
    }
    let flag = root.borrow().flag;
    flag
  }

  fn startswith(&self, prefix: String) -> bool {
    let char_vec: Vec<char> = prefix.chars().collect();
    // Get the clone of current root node
    let mut root = &self.root;

```

```

JavaScript
JavaScript
//
// Initialize your data structure here.
//
var Trie = function () {
  this.children = {};
};

// Insert a word into the trie.
// Returns (string) word
// Returns (boolean)
//
Trie.prototype.insert = function (word) {
  let node = this.children;
  for (let char of word) {
    if (!node[char]) {
      node[char] = {};
    }
    node = node[char];
  }
  node.isEnd = true;
};

// Returns if the word is in the trie.
// Returns (string) word
// Returns (boolean)
//
Trie.prototype.search = function (word) {
  let node = this.children;
  return node != undefined && node.isEnd != undefined;
};

Trie.prototype.searchPrefix = function (prefix) {
  let node = this.children;
  for (let char of prefix) {
    if (!node[char]) return false;
    node = node[char];
  }
  return node;
};

// Returns if there is any word in the trie that starts with the given prefix.
// Returns (string) prefix
// Returns (boolean)
//
Trie.prototype.startsWith = function (prefix) {
  return this.searchPrefix(prefix);
};

```

```
/*
 * Your Trie object will be instantiated and called as such:
 * var obj = new Trie();
 * obj.insert(word);
 * var param_2 = obj.search(word);
 * var param_3 = obj.startsWith(prefix);
 */
```

```
JavaScript
/*
 * Initializing your data structure here.
 */
var Trie = function () {
    this.children = [];
};

/*
 * Inserts a word into the trie.
 * @param {string} word
 * @return {void}
 */
Trie.prototype.insert = function (word) {
    let node = this.children;
    for (let char of word) {
        if (!node[char]) {
            node[char] = {};
        }
        node = node[char];
    }
    node.isEnd = true;
};

/*
 * Returns if the word is in the trie.
 */
```

```

 * @param {string} word
 * @return {boolean}
 */
Trie.prototype.search = function (word) {
    let node = this.searchPrefix(word);
    return node != undefined && node.isEnd != undefined;
};

Trie.prototype.searchPrefix = function (prefix) {
    let node = this.children;
    for (let char of prefix) {
        if (!node[char]) return false;
        node = node[char];
    }
    return node;
};

/*
 * Returns if there is any word in the trie that starts with the given prefix.
 * @param {string} prefix
 * @return {boolean}
 */
Trie.prototype.startsWith = function (prefix) {
    return this.searchPrefix(prefix);
};
```

```
/*
 * Your Trie object will be instantiated and called as such:
 * var obj = new Trie();
 * obj.insert(word);
 * var param_2 = obj.search(word);
 * var param_3 = obj.startsWith(prefix);
 */
```

TypeScript
TypeScript

```
class TrieNode {
    children:
    isEnd:
    constructor() {
        this.children = new Array(26);
        this.isEnd = false;
    }
}

class Trie {
    root:
    constructor() {
        this.root = new TrieNode();
    }

    insert(word: string): void {
        let head = this.root;
        for (let char of word) {
            let index = char.charCodeAt(0) - 97;
            if (!head.children[index]) {
                head.children[index] = new TrieNode();
            }
            head = head.children[index];
        }
        head.isEnd = true;
    }

    search(word: string): boolean {
        let head = this.searchPrefix(word);
        return head != null && head.isEnd;
    }

    startsWith(prefix: string): boolean {
        return this.searchPrefix(prefix) != null;
    }

    private searchPrefix(prefix: string) {
        let head = this.root;
        for (let char of prefix) {
            let index = char.charCodeAt(0) - 97;
            if (!head.children[index]) return null;
            head = head.children[index];
        }
        return head;
    }
}
```

[*] Trie — Pattern Solution (matched from community answers)

Python

```
class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes and a flag for end of word.
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        # Initialize the Trie with the root node.
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        # Start from the root.
        node = self.root
        for char in word:
            # If character doesn't exist, add a new TrieNode.
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        # Mark the end of word.
        node.is_end_of_word = True

    def search(self, word: str) -> bool:
        # Traverse the Trie for each character.
        node = self.root

        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        # Returns True if word ends exactly here.
        return node.is_end_of_word

    def startsWith(self, prefix: str) -> bool:
        # Traverse the Trie for the prefix.
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True
```

#211

MEDIUM

Design Add and Search Words Data Structure

LeetCode · SimplifyLeet · WikiCC · LeetCode · Editorial

String · DFS · Design · Trie

Lists Grind 169

Problem:

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() initializes the object.

- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise, word may contain dots '.' where dots can be matched with any letter.

Example:

```
Input
["WordDictionary", "addWord", "addWord", "addWord", "search", "search", "search"]
[[], ["bad"], ["dad"], ["mad"], ["pad"], ["bad"], [".a"], ["b.."]]

Output
[null, null, null, null, false, true, true]
```

Explanation

```
WordDictionary wordDictionary = new WordDictionary();
wordDictionary.addWord("bad");
wordDictionary.addWord("dad");
wordDictionary.addWord("mad");
wordDictionary.search("pad"); // return False
wordDictionary.search("bad"); // return True
wordDictionary.search(".a"); // return True
wordDictionary.search("b.."); // return True
```

Constraints:

- 1 <= word.length <= 25
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 104 calls will be made to addWord and search.

Python

Python

```
class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes
        self.children = {}
        # Flag to denote if a word ends here
        self.is_end = False

class WordDictionary:
    def __init__(self):
        # Initialize the Trie with an empty root node
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        # Start at the root of the Trie
        node = self.root
        for char in word:
            # Traverse or create node if the character is not present
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        # Mark the end of the word
        node.is_end = True

    def search(self, word: str) -> bool:
        # DFS helper function

        def dfs(index: int, node):
            if index == len(word):
                return node.is_end
            char = word[index]
            if char == '.':
                # If wildcard, search all children nodes
                for child in node.children.values():
                    if dfs(index + 1, child):
                        return True
                return False
            # If character not found, word does not exist
            if char not in node.children:
                return False
            return dfs(index + 1, node.children[char])
        return dfs(0, self.root)

# Example usage
wd = WordDictionary()
wd.addWord("bad")
wd.addWord("dad")
wd.addWord("mad")
print(wd.search("pad")) # Output: False
print(wd.search("bad")) # Output: True
print(wd.search(".a")) # Output: True
print(wd.search("b..")) # Output: True
```

Python

```

class TrieNode:
    def __init__(self):
        self.children = dict[str, TrieNode] = {}
        self.isword = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node = TrieNode = self.root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
            node.isword = True

    def search(self, word: str) -> bool:
        return self._dfs(word, 0, self.root)

    def _dfs(self, word: str, i: int, node: TrieNode) -> bool:
        if i == len(word):
            return node.isword
        if word[i] != '.':
            child = TrieNode = node.children.get(word[i], None)
            return self._dfs(word, i + 1, child) if child else False
        return any(self._dfs(word, i + 1, child) for child in node.children.values())

Python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.is_end = True

    def search(self, word: str) -> bool:
        def search(word, node):
            for i in range(len(word)):
                c = word[i]
                idx = ord(c) - ord('a')
                if c != '.' and node.children[idx] is None:
                    return False
                    if c == '.' and node.children[idx] is None:
                        return False
                    for child in node.children:
                        if child is not None and search(word[i + 1:], child):
                            return True
                    return False
                    return True
                    node = node.children[idx]
                    return search(word, node)
        return search(word, self.trie)

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)
# param_2 = obj.search(word)

```

```

return False
if c == '.':
    for child in node.children:
        if child is not None and search(word[i + 1:], child):
            return True
    return False
node = node.children[idx]
return node is not None and search(word, self.trie)

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)
# param_2 = obj.search(word)

Python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
            node.is_end = True

    def search(self, word: str) -> bool:
        def search(word, node):
            for i in range(len(word)):
                c = word[i]
                idx = ord(c) - ord('a')
                if c != '.' and node.children[idx] is None:
                    return False
                    if c == '.' and node.children[idx] is None:
                        return False
                    for child in node.children:
                        if child is not None and search(word[i + 1:], child):
                            return True
                    return False
                    return True
                    node = node.children[idx]
                    return search(word, node)
        return search(word, self.trie)

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)
# param_2 = obj.search(word)

```

```

return False
if c == '.':
    for child in node.children:
        if child is not None and search(word[i + 1:], child):
            return True
    return False
node = node.children[idx]
return node is not None and search(word, self.trie)

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)
# param_2 = obj.search(word)

C++
class Trie {
public:
    vector<Trie*> children;
    bool is_end;

    Trie() {
        children = vector<Trie*>(26, nullptr);
        is_end = false;
    }

    void insert(const string& word) {
        Trie* cur = this;
        for (char c : word) {
            c -= 'a';
            if (cur->children[c] == nullptr) {
                cur->children[c] = new Trie();
            }
            cur = cur->children[c];
        }
        cur->is_end = true;
    }
};

class WordDictionary {
private:

```

```

trie = root;

public:
    WordDictionary() {
        root = new Trie();
    }

    void addWord(string word) {
        root->insert(word);
    }

    bool search(string word) {
        return dfs(word, 0, root);
    }

private:
    bool dfs(const string& word, int i, Trie* cur) {
        if (i == word.size()) {
            return cur->is_end;
        }
        char c = word[i];
        if (c != '.') {
            Trie* child = cur->children[c - 'a'];
            if (child != nullptr && dfs(word, i + 1, child)) {
                return true;
            }
        } else {
            for (Trie* child : cur->children) {
                if (child != nullptr && dfs(word, i + 1, child)) {
                    return true;
                }
            }
        }
        return false;
    }
};

//
// Your WordDictionary object will be instantiated and called as such:
// WordDictionary obj = new WordDictionary();
// obj.addWord(word);
// bool param_2 = obj.search(word);
//

Java
class Trie {
    List<Trie> children;
    boolean isEnd;

    Trie() {
        children = new ArrayList<>();
        isEnd = false;
    }

    void insert(String word) {
        Trie cur = this;
        for (char c : word.toCharArray()) {
            c -= 'a';
            if (cur.children.get(c) == null) {
                cur.children.add(new Trie());
            }
            cur = cur.children.get(c);
        }
        cur.isEnd = true;
    }
}

class WordDictionary {
    private:

```

```

class Trie {
    Trie[] children = new Trie[26];
    boolean isEnd;
}

class WordDictionary {
    private Trie trie;

    // Initialize your data structure here.
    public WordDictionary() {
        trie = new Trie();
    }

    public void addWord(String word) {
        Trie node = trie;
        for (char c : word.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null) {
                node.children[idx] = new Trie();
            }
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        return search(word, trie);
    }

    private boolean search(String word, Trie node) {
        for (int i = 0; i < word.length(); ++i) {
            char c = word.charAt(i);
            int idx = c - 'a';
            if (c != '.' && node.children[idx] == null) {
                return false;
            }
            if (c == '.') {
                for (Trie child : node.children) {
                    if (child != null && search(word.substring(i + 1), child)) {
                        return true;
                    }
                }
            }
            node = node.children[idx];
        }
        return node.isEnd;
    }
}

//
// Your WordDictionary object will be instantiated and called as such:
// WordDictionary obj = new WordDictionary();
// obj.addWord(word);
// boolean param_2 = obj.search(word);
//

Java

```

```

using System.Collections.Generic;
using System.Linq;

class TrieNode {
    public bool IsEnd { get; set; }
    public TrieNode[] Children { get; set; }
    public TrieNode() {
        Children = new TrieNode[26];
    }
}

public class WordDictionary {
    private TrieNode root;

    public WordDictionary() {
        root = new TrieNode();
    }

    public void AddWord(string word) {
        var node = root;
        for (var i = 0; i < word.Length; ++i) {
            TrieNode nextNode;
            var index = word[i] - 'a';
            nextNode = node.Children[index];

            if (nextNode == null) {
                nextNode = new TrieNode();
                node.Children[index] = nextNode;
            }
            node = nextNode;
        }
        node.IsEnd = true;
    }

    public bool Search(string word) {
        var queue = new Queue<TrieNode>();
        queue.Enqueue(root);
        for (var i = 0; i < word.Length; ++i) {
            var count = queue.Count;
            while (count-- > 0) {
                var node = queue.Dequeue();
                if (word[i] == ".") {
                    foreach (var nextNode in node.Children) {
                        if (nextNode != null) {
                            queue.Enqueue(nextNode);
                        }
                    }
                } else {
                    var index = word[i] - 'a';
                    nextNode = node.Children[index];
                    if (nextNode == null) {
                        return false;
                    }
                    queue.Enqueue(nextNode);
                }
            }
        }
        return queue.Count > 0;
    }
}

```

```

        queue.Enqueue(nextNode);
    }
}
else
{
    var nextNode = node.Children[word[i] - 'a'];
    if (nextNode != null)
    {
        queue.Enqueue(nextNode);
    }
}
}
return queue.Any(x => x.IsEnd);
}
}

```

[1] Trie - Pattern Solution (matched from community answers)

Python

```

class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes
        self.children = {}
        # Flag to denote if a word ends here
        self.is_end = False

class WordDictionary:
    def __init__(self):
        # Initialize the Trie with an empty root node
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        # Start at the root of the Trie
        node = self.root
        for char in word:
            # Check if the character is not present
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        # Mark the end of the word
        node.is_end = True

    def search(self, word: str) -> bool:
        # DFS helper function

```

```

def dfs(index, node):
    if index == len(word):
        return node.is_end and
        char == word[index]
    if char == '?':
        # If wildcard, search all children nodes
        for child in node.children.values():
            if dfs(index + 1, child):
                return True
        return False
    else:
        # If character not found, word does not exist
        if char not in node.children:
            return False
        return dfs(index + 1, node.children[char])
    return dfs(0, self.root)

# Example usage:
wt = WordDictionary()
wt.addWord("bar")
wt.addWord("bar")
wt.addWord("bar")
print(wt.search("bar")) # Output: False
print(wt.search("bar")) # Output: True
print(wt.search("b?r")) # Output: True
print(wt.search("b..r")) # Output: True

```

#616

MEDIUM

Add Bold Tag in String

LeetCode - SimplyList - WUCC - LeetCode - Editorial

Array - Hash Table - String - Trie

Like - Dislike 98

Problem:

You are given a string `s` and an array of strings `words`.

You should add a closed pair of bold tag `<b>` and `</b>` to wrap the substrings in `s` that exist in `words`.

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return `s` after adding the bold tags.

Example 1:

```

Input: s = "abcxyz123", words = ["abc","123"]
Output: "<b>abc</b><b>xyz123</b>"
Explanation: The two strings of words are substrings of s as following: "abcxyz123".
We add <b> before each substring and </b> after each substring.

```

Example 2:

```

Input: s = "aaabbb", words = ["aa","b"]
Output: "<b>aaabbb</b>"
Explanation:
"aa" appears as a substring three times: "aaabbb" and "aaabbb".
"b" appears as a substring three times: "aaabbb", "aaabbb", and "aaabbb".
We add <b> before each substring and </b> after each substring:
"<b>aa</b><b>b</b><b>b</b><b>b</b><b>b</b><b>b</b>".
Since the first two <b>'s overlap, we merge them: "<b>aaab</b><b>b</b><b>b</b><b>b</b><b>b</b>".
Since now the four <b>'s are consecutive, we merge them: "<b>aaabbb</b>".

```

- `1 <= s.length <= 1000`
- `0 <= words.length <= 100`
- `1 <= words[i].length <= 1000`
- `s` and `words[i]` consist of English letters and digits.
- All the values of words are unique.

Note: This question is the same as 758. Bold Words in String.

>> Brute Force [Trie] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        # Brute Force (O(n^2)) - Linear scan through all words
        matches = []
        for word in words:
            if s.startswith(word):
                matches.append(word)
        return matches

```

Python

Python

```

# Python Code Solution
def addBoldTag(s, words):
    n = len(s)
    # Mark positions to be bolded
    bold = [False] * n
    # Iterate each word and mark its occurrences
    for word in words:
        start = s.find(word)
        while start != -1:
            for i in range(start, start + len(word)):
                bold[i] = True
            start = s.find(word, start + 1)
    # Build the final result with bold tags
    result = []
    i = 0
    while i < n:
        if not bold[i]:
            result.append(s[i])
            i += 1
        else:
            result.append("<b>")
            while i < n and bold[i]:
                result.append(s[i])
                i += 1
            result.append("</b>")
    return ''.join(result)

# Example usage:
s = "abcxyz123"
words = ["abc", "123"]
print(addBoldTag(s, words)) # "<b>abc</b><b>xyz123</b>"

# Line-by-Line:
# 1. Initialize an array 'bold' of length n to track which characters need bold formatting.
# 2. For each word, use string.find to locate every occurrence and mark the corresponding indices.
# 3. Traverse the string s. If the current character is marked bold and it is the start of a bold segment,
# insert the opening tag before appending character until the end of that bold segment.
# 4. Append the closing tag after the bold segment and continue the process.

```

Python

```

class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        n = len(s)
        ans = []
        # Initialize a Trie if s[i] should be bolded
        bold = [0] * n
        bolded = -1 # s[i:bolded] should be bolded
        for i in range(n):
            for word in words:
                if s[i:].startswith(word):
                    bolded = max(bolded, i + len(word))
                    bold[i] = bolded + 1
        # Construct the with bold tags
        i = 0
        while i < n:
            if bold[i]:
                j = i
                while j < n and bold[j]:
                    j += 1
                # s[i:j] should be bolded
                ans.append("<b>" + s[i:j] + "</b>")
                i = j
            else:
                ans.append(s[i])
                i += 1
        return ''.join(ans)

```

Python

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)
        n = len(s)
        pairs = []
        for i in range(n):
            node = trie
            for j in range(i, n):
                if node.children[ord(s[j]) - ord('a')] is None:
                    break
                node = node.children[ord(s[j]) - ord('a')]
                if node.is_end:
                    pairs.append([i, j])

```

```

idx = ord(s[i])
if node.children[idx] is None:
    break
node = node.children[idx]
if node.is_end:
    pairs.append([i, j])

if not pairs:
    return s
st, ed = pairs[0]
for i, j in pairs[1:]:
    if ed < i <= s:
        t.append(st, ed)
        st, ed = a, b
    else:
        ed = max(ed, b)
t.append(st, ed)

ans = []
i = 1
while i < n:
    if j not in len(t):
        ans.append(s[i:i+1])
        break
    st, ed = t[i]

if i <= st:
    ans.append(s[i:st])
    ans.append("<b>")
    ans.append(s[st:ed + 1])
    ans.append("</b>")
    j += 1
    i = ed + 1
return ''.join(ans)

```

C++

C++


```

class Solution {
public:
    string addBoldTag(string s, vector<string>& words) {
        const int n = s.length();
        string ans;
        // bold[i] == true if s[i] should be bolded
        vector<bool> bold(n);

        int boldEnd = -1; // s[boldEnd] should be bolded
        for (int i = 0; i < n; ++i) {
            for (const string& word : words)
                if (s.substr(i, word.length()) == word) {
                    boldEnd = max(boldEnd, i + static_cast<int>(word.length()));
                    bold[i] = boldEnd > i;
                }

            // Construct the string with the bold tags.
            int i = 0;
            while (i < n) {
                if (bold[i]) {
                    int j = i;
                    while (j < n && bold[j])
                        ++j;
                    // s[i..j-1] should be bolded
                    ans += "<b>" + s.substr(i, j - i) + "</b>";
                    i = j;
                } else {
                    ans += s[i++];
                }
            }
            return ans;
        }
    };
};

```

C++

Trie · LeetCode · By Pattern

— 745 —

LeetCode

```

class Trie {
public:
    vector<Trie*> children;
    bool isEnd;

    Trie() {
        children.resize(26);
        isEnd = false;
    }

    void insert(string word) {
        Trie* node = this;
        for (char c : word) {
            if (!node->children[c]) node->children[c] = new Trie();
            node = node->children[c];
        }
        node->isEnd = true;
    }

    class Solution {
    public:
        string addBoldTag(string s, vector<string>& words) {
            Trie* trie = new Trie();
            for (string w : words) trie->insert(w);

            int n = s.size();
            vector<pair<int, int>> pairs;
            for (int i = 0; i < n; ++i) {
                Trie* node = trie;
                for (int j = i; j < n; ++j) {
                    int id = -1;
                    if (!node->children[id]) break;
                    node = node->children[id];
                    if (node->isEnd) pairs.push_back({i, j});
                }

                if (pairs.empty()) return s;
                vector<pair<int, int>> list;
                int st = pairs[0].first, ed = pairs[0].second;
                for (int i = 1; i < pairs.size(); ++i) {
                    int st2 = pairs[i].first, ed2 = pairs[i].second;
                    if (ed < st2 && st < ed2) {
                        t.push_back({st, ed2});
                        st = st2, ed = ed2;
                    } else {
                        ed = max(ed, ed2);
                    }
                    t.push_back({st, ed});
                    string ans = "";
                    int i = 0, j = 0;
                }
            }
        }
    };
}

```

Trie · LeetCode · By Pattern

— 746 —

LeetCode

```

        while (i < n) {
            if (j == t.size()) {
                ans += s.substr(i);
                break;
            }
            st = t[j].first, ed = t[j].second;
            if (i < st) ans += s.substr(i, st - i);
            ans += "<b>";
            ans += s.substr(st, ed - st + 1);
            i = ed + 1;
            ++j;
        }
        return ans;
    };
};

```

Java

Java

```

class Solution {
public String addBoldTag(String s, String[] words) {
    final int n = s.length();
    Stringbuilder sb = new Stringbuilder();
    // bold[i] == true if s[i] should be bolded
    boolean[] bold = new boolean[n];

    int boldEnd = -1; // s[boldEnd] should be bolded
    for (int i = 0; i < n; ++i) {
        for (final String word : words)
            if (s.substring(i, i + word.length()) == word) {
                boldEnd = Math.max(boldEnd, i + word.length());
                bold[i] = boldEnd > i;
            }

            // Construct the string with the bold tags.
            int i = 0;
            while (i < n) {
                if (bold[i]) {
                    int j = i;
                    while (j < n && bold[j])
                        ++j;
                    // s[i..j-1] should be bolded
                    sb.append("<b>").append(s.substring(i, j)).append("</b>");
                    i = j;
                } else {
                    sb.append(s.charAt(i++));
                }
            }
            return sb.toString();
        }
    }
}

```

Java

Trie · LeetCode · By Pattern

— 747 —

LeetCode

```

class Trie {
    Trie[] children = new Trie[26];
    boolean isEnd;

    public void insert(String word) {
        Trie node = this;
        for (char c : word.toCharArray()) {
            if (node.children[c] == null) {
                node.children[c] = new Trie();
            }
            node = node.children[c];
        }
        node.isEnd = true;
    }

    class Solution {
    public String addBoldTag(String s, String[] words) {
        Trie trie = new Trie();
        for (String w : words) {
            trie.insert(w);
        }

        List<int[]> pairs = new ArrayList<>();
        int n = s.length();
        for (int i = 0; i < n; ++i) {
            Trie node = trie;
            for (int j = i; j < n; ++j) {
                int id = s.charAt(j);
                if (node.children[id] == null) {
                    break;
                }
                node = node.children[id];
                if (node.isEnd) {
                    pairs.add(new int[] {i, j});
                }
            }

            if (pairs.isEmpty()) {
                return s;
            }

            List<int[]> t = new ArrayList<>();
            int st = pairs.get(0)[0], ed = pairs.get(0)[1];
            for (int i = 1; i < pairs.size(); ++i) {
                int st2 = pairs.get(i)[0], ed2 = pairs.get(i)[1];
                if (ed < st2 && st < ed2) {
                    t.add(new int[] {st, ed2});
                    st = st2, ed = ed2;
                } else {
                    ed = Math.max(ed, ed2);
                }
            }
            t.add(new int[] {st, ed});
        }
    }
}

```

Trie · LeetCode · By Pattern

— 748 —

LeetCode

```

        }
        t.add(new int[] {st, ed});
        int i = 0, j = 0;
        Stringbuilder ans = new Stringbuilder();
        while (i < n) {
            if (j == t.size()) {
                ans.append(s.substring(i));
                break;
            }
            st = t[j][0];
            ed = t[j][1];
            if (i < st) {
                ans.append(s.substring(i, st));
            }
            ++j;
            ans.append("<b>");
            ans.append(s.substring(st, ed + 1));
            ans.append("</b>");
            i = ed + 1;
        }
        return ans.toString();
    }
}

```

Go

Go

```

type Trie struct {
    children [26]*Trie
    isEnd bool
}

func newTrie() *Trie {
    return &Trie{}
}

func (this *Trie) insert(word string) {
    node := this
    for _, c := range word {
        if node.children[c] == nil {
            node.children[c] = newTrie()
        }
        node = node.children[c]
    }
    node.isEnd = true
}

func addBoldTag(s string, words []string) string {
    trie := newTrie()
    for _, w := range words {
        trie.insert(w)
    }
}

```

Trie · LeetCode · By Pattern

— 749 —

LeetCode

```

n := len(s)
var pairs [][]int
for i := range s {
    node := trie
    for j := i; j < n; j++ {
        if node.children[s[j]] == nil {
            break
        }
        node = node.children[s[j]]
        if node.isEnd {
            pairs = append(pairs, []int{i, j})
        }
    }

    if len(pairs) == 0 {
        return s
    }

    var t [][]int
    st, ed := pairs[0][0], pairs[0][1]
    for i = 1; i < len(pairs); i++ {
        a, b := pairs[i][0], pairs[i][1]
        if ed < b && st < a {
            t = append(t, []int{st, a})
            st, ed = a, b
        } else {
            ed = max(ed, b)
        }
    }
    t = append(t, []int{st, ed})
}

var ans strings.Builder
i, j := 0, 0
for i < n {
    if j == len(t) {
        ans.WriteString(s[i:])
        break
    }
    st, ed = t[j][0], t[j][1]
    if i < st {
        ans.WriteString(s[i:st])
    }
    ans.WriteString("<b>")
    ans.WriteString(s[st:ed+1])
    ans.WriteString("</b>")
    i = ed + 1
    j++
}
return ans.String()
}

```

[*] Trie — Pattern Solution (matched from community answers)

Python

Trie · LeetCode · By Pattern

— 750 —

LeetCode

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c)
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

class Solution:
    def findAllPrefixes(self, s: str, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)
        n = len(s)
        pairs = []
        for i in range(n):
            node = trie
            for j in range(i, n):
                idx = ord(s[j])
                if node.children[idx] is None:
                    break
                node = node.children[idx]
                if node.is_end:
                    pairs.append((i, j))

            if not pairs:
                return s
            st, ed = min(pairs)
            t = []
            for a, b in pairs[(st,):]:
                if ed + 1 < a:
                    t.append((st, ed))
                    st, ed = a, b
            else:
                ed = max(ed, b)
            t.append((st, ed))

            ans = []
            i = j = 0
            while i < n:
                if j == len(t):
                    ans.append(s[i:j])
                    break
                st, ed = t[j]

```

Trie - LeetCode - By Pattern

- 751 -

LeetCode

```

if i < st:
    ans.append(s[st])
    ans.append(" ")
    ans.append(s[st : ed + 1])
    ans.append("\n")
    j = 1
    i = ed + 1
return "".join(ans)

```

#692

MEDIUM

Top K Frequent Words

LeetCode - Simplified - WubiCC - LeetCode - Editorial

Hash Table - String - Trie - Sorting - Heap

Lists Grid 169

Problem:

Given an array of strings words and an integer k, return the k most frequent strings.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1:

Input: words = ["i","love","leetcode","i","love","coding"], k = 2

Output: ["i","love"]

Explanation: "i" and "love" are the two most frequent words.

Note that "i" comes before "love" due to a lower alphabetical order.

Example 2:

Input: words = ["the","day","is","sunny","the","the","the","sunny","is","is"], k = 4

Output: ["the","is","sunny","day"]

Explanation: "the", "is", "sunny" and "day" are the four most frequent words, with the number of occurrence being 4, 3, 2 and 1 respectively.

Constraints:

- 1 <= words.length <= 500

- 1 <= words[i].length <= 10

- words[i] consists of lowercase English letters.

- k is in the range [1, The number of unique words[i]]

Follow-up: Could you solve it in O(n log(k)) time and O(n) extra space?

>> Brute Force (Trie) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        # Brute Force O(n^2) - linear scan through all words
        matches = {}
        for word in words:
            if word.startswith(prefix):
                matches.append(word)
        return matches

Python
import heapq
from collections import Counter

def topKFrequent(words: List[str], k: int) -> List[str]:
    # Count the frequency of each word
    count = Counter(words)

    # Initialize a min-heap. Python's heapq is a min-heap.
    # The comparator is (frequency, negative word) but not word in reverse order.
    # Since we want words with lower lexicographical order to be prioritized in the result,
    # and in the min-heap if frequency is equal, we want the one with lexicographically larger word to be popped.
    heap = []

    for word, freq in count.items():
        # Push each pair into the heap. The tuple ordering:
        # First element is frequency and second element is the word (with reverse lex order trick)
        heapq.heappush(heap, (-freq, -word))
        # However, using -word[0] is not robust for multi-letter words.
        # Instead, we can use tuple (freq, word) with a custom comparator by ensuring we insert the word
        # order by using word in reverse.
        # For simplicity, we push tuple (freq, word) and then pop if heap size > k with a custom condition.
        # Next heap and use a custom comparator implemented by the tuple (-freq, word) to simulate max heap behavior.
        heap = []

    for word, freq in count.items():
        # We push with frequency as negative so that larger frequencies come first.
        # We push with frequency as negative so that larger frequencies come first.
        heapq.heappush(heap, (-freq, word))

    # Extract k elements from the heap and sort them accordingly.
    result = []
    for _ in range(k):
        freq, word = heapq.heappop(heap)
        result.append(word)
    return result

# Example usage:
print(topKFrequent(["i","love","leetcode","i","love","coding"], 2)) # Output: ["i", "love"]

Python

```

Trie - LeetCode - By Pattern

- 753 -

LeetCode

```

class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        ans = []
        bucket = [[] for _ in range(len(words) + 1)]

        for word, freq in collections.Counter(words).items():
            bucket[freq].append(word)

        for b in reversed(bucket):
            for word in sorted(b):
                ans.append(word)
            if len(ans) == k:
                return ans

Python
from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    word: str
    freq: int

def __lt__(self, other):
    if self.freq == other.freq:
        return self.word > other.word
    return self.freq < other.freq

class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        ans = []
        heap = []

        for word, freq in collections.Counter(words).items():
            heap.append((-freq, word, freq))
            if len(heap) > k:
                heapq.heappop(heap)

        while heap:
            ans.append(heapq.heappop(heap).word)

        return ans[::-1]

Python
class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        return sorted(cnt, key=lambda x: (-cnt[x], x))[:k]

Python

```

Trie - LeetCode - By Pattern

- 754 -

LeetCode

```

...
>>> sorted(student_objects, key=getattr('grade', 'age'))
[('john', 'A', 31), ('dave', 'B', 30), ('jane', 'B', 32)]

ref: https://docs.python.org/3/howto/sorting.html#operator-module-functions

class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        # Multiple comparators: lambda x: (-cnt[x], x)
        # Why the returned sorted() not a tuple (word, count), but only word?
        # It's the property of Counter() class
        # So, also pass 0: return sorted(cnt.keys(), key=lambda x: (-cnt[x], x))[:k]

...
words = ["i","love","leetcode","i","love","coding"]
k = 2
cnt = Counter(words)
cnt
Counter({'i': 2, 'love': 2, 'leetcode': 1, 'coding': 1})

sorted(cnt, key=lambda x: (-cnt[x], x))
['i', 'love', 'coding', 'leetcode']

# Default, only for keys()
>>> sorted(cnt)
['coding', 'i', 'leetcode', 'love']
>>> sorted(cnt.keys())
['coding', 'i', 'leetcode', 'love']
>>> sorted(cnt.values())
[1, 1, 2, 2]
>>> sorted(cnt.items())
[('coding', 1), ('i', 2), ('leetcode', 1), ('love', 2)]

#####
...
words = ["the","day","is","sunny","the","the","the","sunny","is","is"]
cnt = collections.Counter(words)
cnt
Counter({'the': 4, 'is': 2, 'sunny': 2, 'day': 1})

>>> cnt.items()
dict_items([('the', 4), ('is', 2), ('sunny', 2), ('day', 1)])
...

```

Trie - LeetCode - By Pattern

- 755 -

LeetCode

```

...
cap doesn't exist in Python 3. If you really want it, you could define it yourself.

def cmp(a, b):
    return (a > b) - (a < b)

class Solution(object):
    def topKFrequent(self, words, k):
        # type: words: List[str]
        # type: k: int
        # type: List[str]
        count = collections.Counter(words)

        # If you really want, see: https://docs.python.org/3/howto/sorting.html#comparing-functions
        def compare(x, y):
            if x[1] == y[1]:
                return cmp(x[0], y[0])
            else:
                return cmp(x[1], y[1]) if x[1] > y[1] else -1

        return [x[0] for x in sorted(count.items(), key=compare)[:k]]

C++
...
class Solution {
public:
    vector<string> topKFrequent(vector<string>& words, int k) {
        const int n = words.size();
        vector<string> ans;
        vector<vector<string>> buckets(n);
        unordered_map<string, int> count;

        for (const string& word : words)
            ++count[word];

        for (const auto& [word, freq] : count)
            buckets[freq].push_back(word);

        for (int freq = n; freq > 0; --freq) {
            ranges::sort(buckets[freq]);
            for (const string& word : buckets[freq]) {
                ans.push_back(word);
                if (ans.size() == k)
                    return ans;
            }
        }
        throw;
    }
};

C++

```

Trie - LeetCode - By Pattern

- 756 -

LeetCode

```

struct T {
    string word;
    int freq;
};

class Solution {
public:
    vector<string> topKFrequent(vector<string>& words, int k) {
        vector<string> ans;
        unordered_map<string, int> count;
        // Words with higher frequency and lower alphabetical order are in the
        // bottom of the heap because we'll pop words with lower frequency and
        // higher alphabetical order if the heap's size > k.
        auto compare = [](const T& a, const T& b) {
            return a.freq > b.freq ? a.word < b.word : a.freq > b.freq;
        };
        priority_queue<T, vector<T>, decltype(compare)> heap(compare);
        for (const string& word : words)
            count[word]++;
        for (const auto& [word, freq] : count) {
            heap.emplace(word, freq);
            if (heap.size() > k)
                heap.pop();
        }
        while (!heap.empty())
            ans.push_back(heap.top().word); heap.pop();
        reverse(ans.begin(), ans.end());
        return ans;
    }
};

C++

```

[Trie](#) · [LeetCode](#) — By [Pattern](#) — 757 — [LeetCode](#)

```

// O(1) https://leetcode.com/problems/top-k-frequent-words/
// Time: O(NlogK)
// Space: O(N)
class Solution {
public:
    vector<string> topKFrequent(vector<string>& A, int k) {
        unordered_map<string, int> m;
        for (auto& s : A) m[s]++;
        auto cmp = [](auto& a, auto& b) { return a[0] == b[0] ? a[1] > b[1] : a[0] < b[0]; };
        priority_queue<string, vector<string>, decltype(cmp)> pq(cmp);
        for (auto& [s, cnt] : m) {
            pq.push(s);
            if (pq.size() > k) {
                pq.pop();
            }
        }
        vector<string> ans;
        while (pq.size()) {
            ans.push_back(pq.top());
            pq.pop();
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};

Java

```

```

class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        final int n = words.length;
        List<String> ans = new ArrayList<>();
        List<String>[] bucket = new List[n + 1];
        Map<String, Integer> count = new HashMap<>();
        for (final String word : words)
            count.merge(word, 1, Integer::sum);
        for (final String word : count.keySet()) {
            final int freq = count.get(word);
            if (bucket[freq] == null)
                bucket[freq] = new ArrayList<>();
            bucket[freq].add(word);
        }
        for (int freq = n; freq > 0; --freq)
            if (bucket[freq] != null) {
                Collections.sort(bucket[freq]);
                for (final String word : bucket[freq]) {
                    ans.add(word);
                    if (ans.size() == k)
                        return ans;
                }
            }
    }
}

```

[Trie](#) · [LeetCode](#) — By [Pattern](#) — 758 — [LeetCode](#)

```

}
throw new IllegalArgumentException();
}

Java
class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        Map<String, Integer> cnt = new HashMap<>();
        for (String w : words) {
            cnt.merge(w, 1, Integer::sum);
        }
        Arrays.sort(words, (a, b) -> {
            int c1 = cnt.get(a), c2 = cnt.get(b);
            return c1 == c2 ? a.compareTo(b) : c2 - c1;
        });
        List<String> ans = new ArrayList<>();
        for (int i = 0; i < words.length && ans.size() < k; ++i) {
            if (i == 0 || words[i].equals(words[i - 1])) {
                ans.add(words[i]);
            }
        }
        return ans;
    }
}

Java
public class Top_K_Frequent_Words {
    public static void main(String[] args) {
        Top_K_Frequent_Words cnt = new Top_K_Frequent_Words();
        Solution s = cnt.new Solution();
        System.out.println(s.topKFrequent(new String[]{"the", "day", "is", "sunny", "the", "the", "the", "sunny", "is", "is"}, 4));
        System.out.println(s.topKFrequent(new String[]{"i", "love", "leetcode", "i", "love", "coding"}, 2));
    }
}

// ref: https://leetcode.com/articles/top-k-frequent-words/
// using a heap (pq)
// Time Complexity: O(NlogK), where N is the length of words.
// We count the frequency of each word in O(N) time, then we add N words to the heap, each in O(logK) time.
// Finally, we pop from the heap up to K times. So both time is O(NlogK) in total.
// Space Complexity: O(N), the space used to store our count.
class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        Map<String, Integer> count = new HashMap<>();
        for (String word : words) {
            count.put(word, count.getOrDefault(word, 0) + 1);
        }
        PriorityQueue<String> heap = new PriorityQueue<String>((w1, w2) -> count.get(w1).equals(count.get(w2)) ? w1.compareTo(w2) : count.get(w1) - count.get(w2));
        for (String word : count.keySet())
            heap.add(word);
        List<String> ans = new ArrayList<>();
        while (!heap.isEmpty()) {
            ans.add(heap.poll());
        }
        return ans;
    }
}

```

[Trie](#) · [LeetCode](#) — By [Pattern](#) — 759 — [LeetCode](#)

```

})
for (String word: count.keySet()) {
    heap.offer(word);
    if (heap.size() > k) heap.poll();
}

List<String> result = new ArrayList<>();
while (!heap.isEmpty()) result.add(heap.poll());
Collections.reverse(result);
return result;

// below also working
// List<String> result = new ArrayList<>();
// for (int i = k - 1; i <= 0; i--) {
//     result.add(i, heap.poll());
// }
// return result;

}

}

#####

class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        Map<String, Integer> cnt = new HashMap<>();
        for (String v : words) {
            cnt.put(v, cnt.getOrDefault(v, 0) + 1);
        }
        PriorityQueue<String> q = new PriorityQueue<String>((a, b) -> {
            int d = cnt.get(a) - cnt.get(b);
            return d == 0 ? b.compareTo(a) : d;
        });
        for (String v : cnt.keySet()) {
            q.offer(v);
            if (q.size() > k) {
                q.poll();
            }
        }
        List<String> ans = new List<>();
        while (!q.isEmpty()) {
            ans.add(q.poll());
        }
        return ans;
    }
}

TypeScript
TypeScript

```

[Trie](#) · [LeetCode](#) — By [Pattern](#) — 760 — [LeetCode](#)

```

function topKFrequent(words: string[], k: number): string[] {
    const cnt: Map<string, number> = new Map();
    for (const w of words) {
        cnt.set(w, (cnt.get(w) || 0) + 1);
    }
    const ans: string[] = Array.from(cnt.keys());
    ans.sort((a, b) => {
        return cnt.get(a) === cnt.get(b) ? a.localeCompare(b) : cnt.get(b) - cnt.get(a);
    });
    return ans.slice(0, k);
}

TypeScript
function topKFrequent(words: string[], k: number): string[] {
    const cnt: Map<string, number> = new Map();
    for (const w of words) {
        cnt.set(w, (cnt.get(w) || 0) + 1);
    }
    const ans: string[] = Array.from(cnt.keys());
    ans.sort((a, b) => {
        return cnt.get(a) === cnt.get(b) ? a.localeCompare(b) : cnt.get(b) - cnt.get(a);
    });
    return ans.slice(0, k);
}

Go
func topKFrequent(words []string, k int) []string {
    cnt := map[string]int{}
    for _, w := range words {
        cnt[w]++
    }
    for w := range cnt {
        ans = append(ans, w)
    }
    sort.Slice(ans, func(i, j int) bool {
        a, b := ans[i], ans[j]
        return cnt[a] > cnt[b] || cnt[a] == cnt[b] && a < b
    })
    return ans[:k]
}

Go
func topKFrequent(words []string, k int) []string {
    cnt := map[string]int{}
    for _, w := range words {
        cnt[w]++
    }
    ans := []string{}
    for w := range cnt {
        ans = append(ans, w)
    }
    sort.Slice(ans, func(i, j int) bool {
        a, b := ans[i], ans[j]
        return cnt[a] > cnt[b] || cnt[a] == cnt[b] && a < b
    })
    return ans[:k]
}

```

[Trie](#) · [LeetCode](#) — By [Pattern](#) — 761 — [LeetCode](#)

```

[ ] Trie — Pattern Solution (stored optimal)
Python
import heapq
from collections import Counter

def topKFrequent(words, k):
    # Count the frequency of each word
    count = Counter(words)

    # Initialize a min-heap; Python's heapq is a min-heap.
    # The comparator is (frequency, negative word) but we use word in reverse order.
    # Since we want words with lower alphabetical order to be prioritized in the result,
    # we put in the min-heap if frequency equal, we want the one with lexicographically larger word to be
    # popped.
    heap = []

    for word, freq in count.items():
        # First, we push into the heap. The tuple ordering:
        # First element is frequency and second element is the word (with reverse lex order trick)
        # However, using word[::-1] is not robust for multi-letter words.
        # Instead, we can use tuple (freq, word) with a custom comparator by ensuring we invert the word
        # order by using word in reverse.
        # For simplicity, we push tuple (freq, word) and then pop if heap size > k with a custom condition.
        # Next heap and use a custom comparator implemented by the tuple (-freq, word) to simulate max heap
        # behavior.
        heap = []
        for word, freq in count.items():
            # We push with frequency as negative so that larger frequencies come first;
            # Since we want words with lower alphabetical order when frequencies are equal,
            # we push heapq(heap, (-freq, word))

            # Extract k elements from the heap and sort them externally.
            result = []
            for _ in range(k):
                (freq, word) = heapq.heappop(heap)
                result.append(word)
            return result

# Example usage
print(topKFrequent(["i", "love", "leetcode", "i", "love", "coding"], 2)) # Output: ["i", "love"]

C++

```

[Trie](#) · [LeetCode](#) — By [Pattern](#) — 762 — [LeetCode](#)

```

#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <memory>
#include <functional>
#include <algorithm>
using namespace std;

vector<string> topKFrequent(vector<string>& words, int k) {
    // Count frequency of each word
    unordered_map<string, int> count;
    for (const auto& word : words)
        count[word]++;

    // Define a custom comparator for the min-heap
    auto cmp = [](const pair<int, string>& a, const pair<int, string>& b) {
        // If frequency of both words is same, lexicographically smaller word
        if (a.first == b.first)
            return a.second > b.second;
        return a.first > b.first;
    };

    // Use a min-heap to keep top k elements
    priority_queue<pair<int, string>, vector<pair<int, string>, decltype(cmp)>, heap(cmp)>;

    for (auto& entry : count) {
        heap.push(entry.second, entry.first);
        if (heap.size() > (size_t) k)
            heap.pop();
    }

    // Extract words from the heap, they are in the reverse order.
    vector<string> result;
    while (!heap.empty()) {
        result.push_back(heap.top().second);
        heap.pop();
    }

    // Reverse the result list to have highest frequency first.
    reverse(result.begin(), result.end());
    return result;
}

// Example usage:
// int main() {
//     vector<string> words = {"i","love","leetcode","i","love","coding"};
//     int k = 2;
//     vector<string> ans = topKFrequent(words, k);
//     for (const string& word : ans)
//         cout << word << " ";
// }

// return 0;
// }

```

#720 MEDIUM Longest Word in Dictionary

Trie · LeetCode · By Pattern

— 763 —

LeetCode

```

# Python solution for Longest Word in Dictionary

def longestWord(words):
    # Sort words by length and lexicographical order
    words.sort(key=lambda x: (len(x), x))

    valid_words = set()
    result = ""

    # Process each word
    for word in words:
        # If word is longer than the current word, they are valid if they exist in the list
        if len(word) == len(result) + 1 and word[:len(result)] in valid_words:
            valid_words.add(word)
            # Update result if the current word is longer or if equal length but lexicographically smaller
            if len(word) > len(result) or (len(word) == len(result) and word < result):
                result = word

    return result

# Example usage:
# print(longestWord(["i","love","leetcode","i","love","coding"]))

```

```

Python

class Solution:
    def longestWord(self, words: List[str]) -> str:
        root = {}

        for word in words:
            node = root
            for c in word:
                if c not in node:
                    node[c] = {}
                node = node[c]
            node[word] = word

        def dfs(node: dict) -> str:
            ans = ""
            for child in node:
                if child == word:
                    child_node = dfs(node[child])
                    if len(child_node) > len(ans) or (len(child_node) == len(ans) and child_node < ans):
                        ans = child_node
            return ans

        return dfs(root)

```

Python

Trie · LeetCode · By Pattern

— 763 —

LeetCode

```

class Solution {
public:
    string longestWord(vector<string>& words) {
        unordered_set<string> s(words.begin(), words.end());
        int cnt = 0;
        string ans = "";
        for (auto w : s) {
            int n = w.size();
            if (check(w, s)) {
                if (cnt < n) {
                    cnt = n;
                    ans = w;
                } else if (cnt == n && w < ans)
                    ans = w;
            }
        }
        return ans;
    }

    bool check(string word, unordered_set<string>& s) {
        for (int i = 1; i < word.size(); i++) {
            if (s.count(word.substr(0, i)))
                return true;
        }
        return false;
    }
};

```

Java

Java

```

class Solution {
    private Set<String> s;

    public String longestWord(String[] words) {
        s = new HashSet<>(Arrays.asList(words));
        int cnt = 0;
        String ans = "";
        for (String w : s) {
            int n = w.length();
            if (check(w)) {
                if (cnt < n) {
                    cnt = n;
                    ans = w;
                } else if (cnt == n && w.compareTo(ans) < 0) {
                    ans = w;
                }
            }
        }
        return ans;
    }

    private boolean check(String word) {
        for (int i = 1; i < word.length(); i++) {
            if (s.contains(word.substring(0, i)))
                return true;
        }
        return false;
    }
}

```

Trie · LeetCode · By Pattern

— 767 —

LeetCode

LeetCode · SimplyTest · WalkCC · LeetCode · Editorial

Array · Hash Table · String · Trie · Sorting

Like · Donny Zhang

Problem:

Given an array of strings words representing an English Dictionary, return the longest word in words that can be built one character at a time by other words in words.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

Input: words = ["w","wo","wor","worl","world"]
Output: "world"
Explanation: The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".

Example 2:

Input: words = ["a","banana","app","appl","ap","apply","apple"]
Output: "apple"
Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically smaller than "apply".

Constraints:

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 30
- words[i] consists of lowercase English letters.

>> Brute Force [Trie] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def longestWord(self, words):
        # Simple Brute Force - Linear scan through all words
        matches = []
        for word in words:
            if word.startswith(prefix):
                matches.append(word)
        return matches

```

Python

Python

Trie · LeetCode · By Pattern

— 764 —

LeetCode

```

class Trie:
    def __init__(self):
        self.children = List[Optional[Trie]] = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, w: str) -> bool:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return False
            node = node.children[idx]
        if not node.is_end:
            return False
        return True

```

```

class Solution:
    def longestWord(self, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)
        ans = ""
        for w in words:
            if trie.search(w) and (len(ans) < len(w) or (len(ans) == len(w) and ans > w)):
                ans = w
        return ans

```

Python

```

class Solution:
    def longestWord(self, words: List[str]) -> str:
        cnt = 0
        s = set(words)
        for w in s:
            n = len(w)
            if all(w[i-1] in s for i in range(1, n)):
                cnt = max(cnt, n)
                ans = w
        return ans

```

C++

C++

Trie · LeetCode · By Pattern

— 766 —

LeetCode

```

}
return true;
}
}

```

JavaScript

```

/**
 * @param {string[]} words
 * @return {string}
 */
var longestWord = function(words) {
    const trie = new Trie();
    for (const w of words) {
        trie.insert(w);
    }

    let ans = "";
    for (const w of words) {
        if (trie.search(w) && (ans.length < w.length || (ans.length === w.length && w < ans))) {
            ans = w;
        }
    }
    return ans;
};

```

class Trie {

```

    constructor() {
        this.children = Array(26).fill(null);
        this.isEnd = false;
    }

    insert(w) {
        let node = this;
        for (let i = 0; i < w.length; i++) {
            const idx = w.charCodeAt(i) - 'a'.charCodeAt(0);
            if (node.children[idx] === null) {
                node.children[idx] = new Trie();
            }
            node = node.children[idx];
        }
        node.isEnd = true;
    }

    search(w) {
        let node = this;
        for (let i = 0; i < w.length; i++) {
            const idx = w.charCodeAt(i) - 'a'.charCodeAt(0);
            if (node.children[idx] === null || !node.children[idx].isEnd) {
                return false;
            }
            node = node.children[idx];
        }
        return true;
    }
}

```

Trie · LeetCode · By Pattern

— 768 —

LeetCode

Trie · LeetCode — By Pattern — 769 — LeetCode

Trie · LeetCode — By Pattern — 770 — LeetCode

Trie · LeetCode — By Pattern — 771 — LeetCode

Trie · LeetCode — By Pattern — 772 — LeetCode

Python

Python

```

//
//> "A/B/C", split("/")
//> [1] "A" "B" "C"

class Trie {
public:
    Trie() {
        //init...
    }

    //insert
    bool insert(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                node->children[c] = new Trie();
            }
            node = node->children[c];
        }
        node->isLeaf = true;
        return true;
    }

    //search
    bool search(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                return false;
            }
            node = node->children[c];
        }
        return node->isLeaf;
    }
};

//Filesystem
class Filesystem {
public:
    Filesystem() {
        //init...
    }

    //createPath
    bool createPath(string path, int value) {
        return trie.insert(path, value);
    }

    //get
    int get(string path) {
        return trie.search(path);
    }
};

//Your Filesystem object will be instantiated and called as such:
//Filesystem obj = new Filesystem();
// int param_1 = obj.createPath(path,value);
// int param_2 = obj.get(path);

```

```

class Trie {
public:
    unordered_map<char, Trie*> children;
    bool isLeaf;

    Trie() {}

    Trie(char c) {
        this->children[c] = new Trie();
    }

    bool insert(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                node->children[c] = new Trie();
            }
            node = node->children[c];
        }
        node->isLeaf = true;
        return true;
    }

    bool search(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                return false;
            }
            node = node->children[c];
        }
        return node->isLeaf;
    }
};

//Filesystem
class Filesystem {
public:
    Filesystem() {}

    bool createPath(string path, int value) {
        return trie.insert(path, value);
    }

    int get(string path) {
        return trie.search(path);
    }
};

//Your Filesystem object will be instantiated and called as such:
//Filesystem obj = new Filesystem();
// int param_1 = obj.createPath(path,value);
// int param_2 = obj.get(path);

```

```

//
//> "A/B/C", split("/")
//> [1] "A" "B" "C"

class Trie {
public:
    Trie() {
        //init...
    }

    //insert
    bool insert(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                node->children[c] = new Trie();
            }
            node = node->children[c];
        }
        node->isLeaf = true;
        return true;
    }

    //search
    bool search(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                return false;
            }
            node = node->children[c];
        }
        return node->isLeaf;
    }
};

//Filesystem
class Filesystem {
public:
    Filesystem() {
        //init...
    }

    //createPath
    bool createPath(string path, int value) {
        return trie.insert(path, value);
    }

    //get
    int get(string path) {
        return trie.search(path);
    }
};

//Your Filesystem object will be instantiated and called as such:
//Filesystem obj = new Filesystem();
// int param_1 = obj.createPath(path,value);
// int param_2 = obj.get(path);

```

```

//
//> "A/B/C", split("/")
//> [1] "A" "B" "C"

class Trie {
public:
    Trie() {
        //init...
    }

    //insert
    bool insert(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                node->children[c] = new Trie();
            }
            node = node->children[c];
        }
        node->isLeaf = true;
        return true;
    }

    //search
    bool search(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                return false;
            }
            node = node->children[c];
        }
        return node->isLeaf;
    }
};

//Filesystem
class Filesystem {
public:
    Filesystem() {
        //init...
    }

    //createPath
    bool createPath(string path, int value) {
        return trie.insert(path, value);
    }

    //get
    int get(string path) {
        return trie.search(path);
    }
};

//Your Filesystem object will be instantiated and called as such:
//Filesystem obj = new Filesystem();
// int param_1 = obj.createPath(path,value);
// int param_2 = obj.get(path);

```

```

//
//> "A/B/C", split("/")
//> [1] "A" "B" "C"

class Trie {
public:
    Trie() {
        //init...
    }

    //insert
    bool insert(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                node->children[c] = new Trie();
            }
            node = node->children[c];
        }
        node->isLeaf = true;
        return true;
    }

    //search
    bool search(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                return false;
            }
            node = node->children[c];
        }
        return node->isLeaf;
    }
};

//Filesystem
class Filesystem {
public:
    Filesystem() {
        //init...
    }

    //createPath
    bool createPath(string path, int value) {
        return trie.insert(path, value);
    }

    //get
    int get(string path) {
        return trie.search(path);
    }
};

//Your Filesystem object will be instantiated and called as such:
//Filesystem obj = new Filesystem();
// int param_1 = obj.createPath(path,value);
// int param_2 = obj.get(path);

```

```

//
//> "A/B/C", split("/")
//> [1] "A" "B" "C"

class Trie {
public:
    Trie() {
        //init...
    }

    //insert
    bool insert(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                node->children[c] = new Trie();
            }
            node = node->children[c];
        }
        node->isLeaf = true;
        return true;
    }

    //search
    bool search(string w) {
        Trie* node = this;
        for (int i = 0; i < w.size(); ++i) {
            char c = w[i];
            if (!node->children[c]) {
                return false;
            }
            node = node->children[c];
        }
        return node->isLeaf;
    }
};

//Filesystem
class Filesystem {
public:
    Filesystem() {
        //init...
    }

    //createPath
    bool createPath(string path, int value) {
        return trie.insert(path, value);
    }

    //get
    int get(string path) {
        return trie.search(path);
    }
};

//Your Filesystem object will be instantiated and called as such:
//Filesystem obj = new Filesystem();
// int param_1 = obj.createPath(path,value);
// int param_2 = obj.get(path);

```

```

func (t *trie) search(string) int {
    node := t
    ps := strings.Split(s, "")
    for _, p := range ps[1:] {
        if _, ok := node.children[p]; !ok {
            return -1
        }
        node = node.children[p]
    }
    return node.v
}

type FileSystem struct {
    trie *trie
}

func Constructor() FileSystem {
    return FileSystem{trie: new(trie)}
}

func (this *FileSystem) CreatePath(path string, value int) bool {
    return this.trie.insert(path, value)
}

func (this *FileSystem) Get(path string) int {
    return this.trie.search(path)
}

/*
 * Your FileSystem object will be instantiated and called as such:
 * obj := Constructor();
 * param_1 := obj.CreatePath(path,value);
 * param_2 := obj.Get(path);
 */

```

[*] Trie - Pattern Solution (matched from community answers)

Python

Trie - LeetCode - By Pattern

— 781 —

LeetCode

```

class Trie:
    def __init__(self, v: int = -1):
        self.children = {}
        self.v = v

    def insert(self, w: str, v: int) -> bool:
        node = self
        ps = w.split("")
        for p in ps[1:]:
            if p not in node.children:
                return False
            node = node.children[p]
            if p[-1] in node.children:
                return False
            node.children[p[-1]] = Trie(v)
        return True

    def search(self, w: str) -> int:
        node = self
        ps = w.split("")
        for p in ps[1:]:
            if p not in node.children:
                return -1
            node = node.children[p]
        return node.v

class FileSystem:
    def __init__(self):
        self.trie = Trie()

    def createPath(self, path: str, value: int) -> bool:
        return self.trie.insert(path, value)

    def get(self, path: str) -> int:
        return self.trie.search(path)

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem();
# param_1 := obj.createPath(path,value);
# param_2 := obj.get(path);

```

#212 HARD

Word Search II

LeetCode - Simplify - WalkCC - LeetCode - Editorial

Array - String - Backtracking - Trie - Matrix

Lists Grid 169

Problem:

Given an $m \times n$ board of characters and a list of strings words, return all words on the board.

Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Trie - LeetCode - By Pattern

— 782 —

LeetCode

Example 1:

o	a	a	n
e	t	a	e
i	h	k	r
i	f	l	v

Input: board = [[["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words = ["oath","pea","eat","rain"]
Output: ["eat","oath"]

Example 2:

a	b
c	d

Input: board = [[["a","b"],["c","d"]], words = ["abcd"]
Output: []

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 12$
- $\text{board}[i][j]$ is a lowercase English letter.

Trie - LeetCode - By Pattern

— 783 —

LeetCode

- $1 \leq \text{words.length} \leq 3 \times 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$
- $\text{words}[i]$ consists of lowercase English letters.
- All the strings of words are unique.

>> Brute Force [Trie] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        # Brute Force O(N * M * L) - run word-search for each word independently
        def exists(board, word):
            m, n = len(board), len(board[0])
            def dfs(i, j, k, seen):
                if k == len(word): return True
                if i < 0 or i > m or j < 0 or j > n: return False
                if board[i][j] == word[k]: return False
                seen.add((i, j))
                ok = any(dfs(i+dr, j+dc, k+1, seen) for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)))
                return ok
            return any(dfs(i, j, 0, set())) for i in range(m) for j in range(n)
        return [w for w in words if exists(board, w)]

```

Python

```

# Define a TrieNode for building the Trie.
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None # Stores complete word at the end node

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        # Build the Trie from the list of words.
        root = TrieNode()
        for word in words:
            node = root
            for char in word:
                if char not in node.children:
                    node.children[char] = TrieNode()
                node = node.children[char]
            node.word = word # Mark the end of a word

        result = []
        m, n = len(board), len(board[0])
        # Perform the DFS helper function.
        def dfs(i, j, node):
            char = board[i][j]
            if char not in node.children:
                return
            node = node.children[char]
            if node.word:
                result.append(node.word)
                node.word = None
            dfs(i+1, j, node)
            dfs(i, j+1, node)
            dfs(i-1, j, node)
            dfs(i, j-1, node)

```

Trie - LeetCode - By Pattern

— 784 —

LeetCode

```

        return
        next_node = node.children[char]
        if next_node.word:
            result.append(next_node.word)
            next_node.word = None # Avoid duplicate entries

        board[i][j] = '#' # Mark the current cell as visited
        # Explore all four adjacent directions
        for dx, dy in ((0, 1), (1, 0), (0, -1), (-1, 0)):
            nx, ny = i + dx, j + dy
            if 0 <= nx < m and 0 <= ny < n and board[nx][ny] != '#':
                dfs(nx, ny, next_node)
        board[i][j] = char # Restore the original character

# Store DFS results from each cell.
for i in range(m):
    for j in range(n):
        dfs(i, j, root)
return result

Python
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        m = len(board)
        n = len(board[0])
        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        for word in words:
            insert(word)

        def dfs(i: int, j: int, node: TrieNode) -> None:
            if i < 0 or i > m or j < 0 or j > n:
                return
            char = board[i][j]
            if char not in node.children:
                return
            node = node.children[char]
            if node.word:
                ans.append(node.word)
                node.word = None
            dfs(i+1, j, node)
            dfs(i-1, j, node)
            dfs(i, j+1, node)
            dfs(i, j-1, node)

```

Trie - LeetCode - By Pattern

— 785 —

LeetCode

```

        if board[i][j] == 'a':
            return

        <= board[i][j]
        if a not in node.children:
            return

        child = node.children[a]
        if child.word:
            ans.append(child.word)
            child.word = None

        board[i][j] = 'a'
        dfs(i+1, j, child)
        dfs(i-1, j, child)
        dfs(i, j+1, child)
        dfs(i, j-1, child)
        board[i][j] = <

        for i in range(m):
            for j in range(n):
                dfs(i, j, root)
        return ans

Python
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.word = ""

        # Since 'start' not a lowercase 'a', but the whole word
        # so it's easier for dfs to save the word

    def insert(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.w = w

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        def dfs(i, j):
            idx = ord(board[i][j]) - ord('a')
            if node.children[idx] is None:
                return
            node = node.children[idx]
            if node.w:
                ans.append(node.w)
                node.w = None
            dfs(i+1, j)
            dfs(i-1, j)
            dfs(i, j+1)
            dfs(i, j-1)

```

Trie - LeetCode - By Pattern

— 786 —

LeetCode

```

        ans.add(node.u)
        c = board[i][j]
        board[i][j] = '0' // 0 for visited already
        for a, b in [(0, -1), (0, 1), (1, 0), (-1, 0)]:
            x = i + a, y = j + b
            if 0 <= x < n and 0 <= y < n and board[x][y] != '0':
                dfs(node, x, y)
        board[i][j] = c

        trie = Trie()
        for w in words:
            trie.insert(w)
        ans = set()
        n = len(board), len(board[0])
        for i in range(n):
            for j in range(n):
                dfs(trie, i, j)
        return list(ans)

#####

class Trie:
    def __init__(self):
        self.children: List[Trie] = [None] * 26
        self.ref: int = -1

    def insert(self, w: str, ref: int):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.ref = ref

    class Solution:
        def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
            def dfs(node: Trie, i: int, j: int):
                idx = ord(board[i][j]) - ord('a')
                if node.children[idx] is None:
                    return
                node = node.children[idx]
                if node.ref == 0:
                    ans.append(words[node.ref])
                node.ref -= 1
                c = board[i][j]
                board[i][j] = '#'

                for a, b in pairdir((-1, 0), (0, 1), (0, -1)):
                    x, y = i + a, j + b
                    if 0 <= x < n and 0 <= y < n and board[x][y] != '0':
                        dfs(node, x, y)
                board[i][j] = c

```

```

C++
C++

struct TrieNode {
    vector<char*> ptrTrieNodes; children;
    const string word = nullptr;
    TrieNode() : children(26) {}
};

class Solution {
public:
    vector<string> findWords(vector<vector<char*>& board, vector<string>& words) {
        vector<string> ans;

        for (const string& word : words)
            insert(word);

        for (int i = 0; i < board.size(); ++i)
            for (int j = 0; j < board[0].size(); ++j)
                dfs(board, i, j, root, ans);

        return ans;
    }

private:
    shared_ptr<TrieNode> root = make_shared<TrieNode>();

    void insert(const string& word) {
        shared_ptr<TrieNode> node = root;
        for (const char c : word) {
            const int i = c - 'a';
            if (node->children[i] == nullptr)
                node->children[i] = make_shared<TrieNode>();
            node = node->children[i];
        }
        node->word = word;
    }

    void dfs(vector<vector<char*>& board, int i, int j, shared_ptr<TrieNode> node,
            vector<string>& ans) {
        if (i < 0 || i == board.size() || j < 0 || j == board[0].size())
            return;
        if (board[i][j] == 'a')
            return;
        const char c = board[i][j];
        shared_ptr<TrieNode> child = node->children[c - 'a'];
        if (child == nullptr)
            ans.push_back(word);
        child->word = nullptr;
    }
}

```

```

board[i][j] = 'a';
dfs(board, i + 1, j, child, ans);
dfs(board, i - 1, j, child, ans);
dfs(board, i, j + 1, child, ans);
dfs(board, i, j - 1, child, ans);
board[i][j] = c;
}
}

C++
class Trie {
public:
    vector<Trie*> children;
    int ref;

    Trie()
        : children(26, nullptr)
        , ref(-1) {}

    void insert(const string& w, int ref) {
        Trie* node = this;
        for (char c : w) {
            c -= 'a';
            if (node->children[c] == nullptr)
                node->children[c] = new Trie();
            node = node->children[c];
        }
        node->ref = ref;
    }
};

class Solution {
public:
    vector<string> findWords(vector<vector<char*>& board, vector<string>& words) {

```

```

Trie* tree = new Trie();
for (int i = 0; i < words.size(); ++i) {
    tree->insert(words[i], i);
}

vector<string> ans;
int n = board.size(), m = board[0].size();

function<void(Trie*, int, int)> dfs = [&](Trie* node, int i, int j) {
    int idx = board[i][j] - 'a';
    if (node->children[idx] == nullptr)
        return;
    node = node->children[idx];
    if (node->ref != -1) {
        ans.emplace_back(words[node->ref]);
        node->ref = -1;
    }

    int dir[4] = {-1, 0, 1, 0, -1};
    char c = board[i][j];
    board[i][j] = '#';
    for (int k = 0; k < 4; ++k) {
        int x = i + dir[k], y = j + dir[k] + 1;
        if (x >= 0 && x < n && y >= 0 && y < n && board[x][y] != '0') {
            dfs(node, x, y);
        }
    }

    board[i][j] = c;
}

for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        dfs(tree, i, j);
    }
}

return ans;
}

Java
Java

```

```

class TrieNode {
    public TrieNode[] children = new TrieNode[26];
    public String word;
}

class Solution {
    public List<String> findWords(char[][] board, String[] words) {
        for (final String word : words)
            insert(word);

        List<String> ans = new ArrayList<>();

        for (int i = 0; i < board.length; ++i)
            for (int j = 0; j < board[0].length; ++j)
                dfs(board, i, j, root, ans);

        return ans;
    }

    private TrieNode root = new TrieNode();

    private void insert(final String word) {
        TrieNode node = root;
        for (final char c : word.toCharArray()) {
            final int i = c - 'a';

            if (node.children[i] == null)
                node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.word = word;
    }

    private void dfs(char[][] board, int i, int j, TrieNode node, List<String> ans) {
        if (i < 0 || i == board.length || j < 0 || j == board[0].length)
            return;
        if (board[i][j] == '0')
            return;
        final char c = board[i][j];
        TrieNode child = node.children[c - 'a'];
        if (child == null)
            return;
        if (child.word != null) {
            ans.add(child.word);
            ans.add(child.word);
            child.word = null;
        }

        board[i][j] = '0';
        dfs(board, i + 1, j, child, ans);
        dfs(board, i - 1, j, child, ans);
        dfs(board, i, j + 1, child, ans);
        dfs(board, i, j - 1, child, ans);
        board[i][j] = c;
    }
}

Java

```

```

class Trie {
    Trie[] children = new Trie[26];
    int ref = -1;

    public void insert(String w, int ref) {
        Trie node = this;
        for (int i = 0; i < w.length(); ++i) {
            int j = w.charAt(i) - 'a';
            if (node.children[j] == null)
                node.children[j] = new Trie();
            node = node.children[j];
        }
        node.ref = ref;
    }
}

class Solution {
    private char[][] board;
    private String[] words;
    private List<String> ans = new ArrayList<>();

    public List<String> findWords(char[][] board, String[] words) {
        this.board = board;
        this.words = words;

        Trie tree = new Trie();
        for (int i = 0; i < words.length; ++i) {
            tree.insert(words[i], i);
        }

        int n = board.length, m = board[0].length;
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                dfs(tree, i, j);
            }
        }

        return ans;
    }

    private void dfs(Trie node, int i, int j) {
        int idx = board[i][j] - 'a';
        if (node.children[idx] == null)
            return;
        node = node.children[idx];
        if (node.ref != -1) {
            ans.add(words[node.ref]);
            node.ref = -1;
        }

        char c = board[i][j];
        board[i][j] = '#';
    }
}

```



```

        let[i] dirx = [-1, 0, 1, 0, -1];
        for (let k = 0; k < 4; ++k) {
            let x = i + dirx[k], y = j + dirrk[k + 1];
            if (x == 0 && x == board.length && y == 0 && y == board[0].length && board[x][y] != 'P') {
                dfs(node, x, y);
            }
            board[i][j] = c;
        }
    }

TypeScript
class Trie {
    children: Trie[];
    ref: number;

    constructor() {
        this.children = new Array(26);
        this.ref = -1;
    }

    insert(w: string, ref: number): void {
        let node: Trie = this;
        for (let i = 0; i < w.length; ++i) {
            const c = w.charCodeAt(i) - 97;
            if (node.children[c] == null) {
                node.children[c] = new Trie();
            }
            node = node.children[c];
        }
        node.ref = ref;
    }

    function findWords(board: string[][], words: string[]): string[] {
        const tree = new Trie();
        for (let i = 0; i < words.length; ++i) {

```

```

        tree.insert(words[i], i);
    }

    const n = board.length;
    const m = board[0].length;
    const ans: string[] = [];
    const dirs: number[] = [-1, 0, 1, 0, -1];
    const dfs = (node: Trie, i: number, j: number) => {
        const idx = board[i][j].charCodeAt() - 97;
        if (node.children[idx] == null) {
            return;
        }
        node = node.children[idx];
        if (node.ref != -1) {
            ans.push(words[node.ref]);
            node.ref = -1;
        }
        const c = board[i][j];
        board[i][j] = 'P';
        for (let k = 0; k < 4; ++k) {
            const x = i + dirx[k];
            const y = j + dirrk[k + 1];
            if (x == 0 && x == m && y == 0 && y == n && board[x][y] != 'P') {
                dfs(node, x, y);
            }
        }
        board[i][j] = c;
    };
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            dfs(tree, i, j);
        }
    }
    return ans;
}

Go
Go

```

```

type Trie struct {
    children [26]*Trie
    ref int
}

func newTrie() *Trie {
    return &Trie{ref: -1}
}

func (this *Trie) insert(w string, ref int) {
    node := this
    for i, c := range w {
        c -= 'a'
        if node.children[c] == nil {
            node.children[c] = newTrie()
        }
        node = node.children[c]
    }
    node.ref = ref
}

func findWords(board [][]byte, words []string) ans []string {
    trie := newTrie()
    for i, w := range words {
        trie.insert(w, i)
    }

    m, n := len(board), len(board[0])
    var dfs func(i, j, int)
    dfs = func(i, j, int) {
        if i < 0 || i > m-1 || j < 0 || j > n-1 {
            return
        }
        node = node.children[board[i][j] - 'a']
        if node.ref != -1 {
            ans = append(ans, words[node.ref])
            node.ref = -1
        }
        c := board[i][j]
        board[i][j] = '#'
        dirs := []int{-1, 0, 1, 0, -1}
        for k = 0; k < 4; ++k {
            x, y = i+dirs[k], j+dirs[k+1]
            if x < 0 && x > m && y < 0 && y > n && board[x][y] != 'P' {
                dfs(node, x, y)
            }
        }
        board[i][j] = c
    }
    for i := 0; i < m; ++i {
        for j := 0; j < n; ++j {
            dfs(trie, i, j)
        }
    }
    return ans
}

Python

```

```

[?] Trie - Pattern Solution (matched from community answers)

Python
# Define a TrieNode for building the Trie.
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None # Stores complete word at the end node

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        # Build the Trie from the list of words.
        root = TrieNode()
        for word in words:
            node = root
            for char in word:
                if char not in node.children:
                    node.children[char] = TrieNode()
                node = node.children[char]
            node.word = word # Mark the end of a word

        result = []
        m, n = len(board), len(board[0])

        # Define the DFS helper function.
        def dfs(i, j, node):
            char = board[i][j]
            if char not in node.children:
                return

            next_node = node.children[char]
            if next_node.word:
                result.append(next_node.word)
                next_node.word = None # Avoid duplicate entries

            board[i][j] = '#' # Mark the current cell as visited
            # Explore all four adjacent directions.
            for dx, dy in ((0, 1), (1, 0), (0, -1), (-1, 0)):
                ni, nj = i + dx, j + dy
                if 0 <= ni < m and 0 <= nj < n and board[ni][nj] != '#':
                    dfs(ni, nj, next_node)
            board[i][j] = char # Restore the original character

        # Start DFS from each cell.
        for i in range(m):
            for j in range(n):
                dfs(i, j, root)

        return result

#336 HARD
Palindromes Pairs
LeetCode - Solution - LeetCode - Editorial
Array - Hash Table - String - Trie
Like Grind 169

Problem:

```

You are given a 0-indexed array of unique strings words.

A palindrome pair is a pair of integers (i, j) such that:

- 0 <= i, j < words.length,
- i != j, and
- words[i] + words[j] (the concatenation of the two strings) is a palindrome.

Return an array of all the palindrome pairs of words.

You must write an algorithm with O(sum of words[i].length) runtime complexity.

Example 1:

Input: words = ["abcd", "dcba", "ll", "x", "yzzl"]
Output: [[0,1],[1,0],[2,2],[2,4]]
Explanation: The palindromes are "abcdcba", "dcbabcd", "lls", "lssll"

Example 2:

Input: words = ["bat", "tab", "cat"]
Output: [[0,1],[1,0]]
Explanation: The palindromes are "battab", "tabbat"

Example 3:

Input: words = ["a", ""]
Output: [[0,1],[1,0]]
Explanation: The palindromes are "a", "a"

Constraints:

- 1 <= words.length <= 5000
- 0 <= words[i].length <= 300
- words[i] consists of lowercase English letters.

>> Brute Force [Trie] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def is_palindrome(self, str):
        # Brute Force O(n^2) - Linear scan through all words
        matches = []
        for word in words:
            if word.startswith(str):
                matches.append(word)
        return matches

Python

```

```

# Python Implementation for Palindrome Pairs

def is_palindrome(s):
    # Helper function to check if a string is a palindrome
    return s == s[::-1]

def palindromePairs(words):
    result = []
    # Create a dictionary mapping each word to its index
    word_dict = {word: i for i, word in enumerate(words)}

    # Iterate over each word and each index
    for i, word in enumerate(words):
        # For each possible split position (including empty string splits)
        for j in range(len(word) + 1):
            prefix = word[:j]
            suffix = word[j:]

            # If prefix is palindrome, check for reversed suffix in dictionary
            if is_palindrome(prefix):
                reversed_suffix = suffix[::-1]
                # Verify that the reversed suffix exists and is not the same word
                if reversed_suffix in word_dict and word_dict[reversed_suffix] != i:
                    result.append([word_dict[reversed_suffix], i])

            # Check the suffix, avoid duplicate pairs when j equals len(word) because
            # the case where suffix is empty was handled in the first condition
            if j != len(word) and is_palindrome(suffix):
                reversed_prefix = prefix[::-1]
                if reversed_prefix in word_dict and word_dict[reversed_prefix] != i:
                    result.append([i, word_dict[reversed_prefix]])

    return result

# Example Usage
words = ["abcd", "dcba", "ll", "x", "yzzl"]
print(palindromePairs(words)) # Expected Output: [[0,1],[1,0],[2,2],[2,4]]

Python

class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        ans = []
        dict = {word[i]: i for i, word in enumerate(words)}

        for i, word in enumerate(words):
            if "" in dict and dict[""] != i and word == word[::-1]:
                ans.append([i, dict[""]])

            for j in range(1, len(word) + 1):
                l = word[:j]
                r = word[j:]
                if l in dict and dict[l] != i and r == r[::-1]:
                    ans.append([i, dict[l]])
                if r in dict and dict[r] != i and l == l[::-1]:
                    ans.append([dict[r], i])

        return ans

```

```

Python
class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {}
        for i, w in enumerate(words):
            d[w] = i
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])
                if j and rb in d and d[rb] != i and a == ra:
                    ans.append([d[rb], i])
        return ans

Python
class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {}
        for i, w in enumerate(words):
            d[w] = i
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])
                if j and rb in d and d[rb] != i and a == ra:
                    ans.append([d[rb], i])
        return ans

Java
class Solution {
    private static final int BASE = 131;
    private static final long[] MUL = new long[26];
    private static final int MOD = (int) 1e9 + 7;
    static {
        MUL[0] = 1;
        for (int i = 1; i < MUL.length; ++i) {
            MUL[i] = (MUL[i - 1] * BASE) % MOD;
        }
    }
    public List<List<Integer>> palindromePairs(String[] words) {
        int n = words.length;
        long[] prefix = new long[n];
        long[] suffix = new long[n];
        for (int i = 0; i < n; ++i) {
            String word = words[i];
            int m = word.length();
            for (int j = 0; j < m; ++j) {
                int t = word.charAt(j) - 'a' + 1;
                int s = word.charAt(j - 1) - 'a' + 1;
                prefix[i] = (prefix[i] * BASE + t) % MOD + 1;
                suffix[i] = (suffix[i] * BASE + s) % MOD + s;
            }
        }
        List<List<Integer>> ans = new ArrayList();
    }
}

```

```

for (int i = 0; i < n; ++i) {
    for (int j = 1; j < n; ++j) {
        if (check(i, j, word[i].length(), word[j].length(), prefix, suffix)) {
            ans.add(Arrays.asList(i, j));
        }
        if (check(j, i, word[j].length(), word[i].length(), prefix, suffix)) {
            ans.add(Arrays.asList(j, i));
        }
    }
}
return ans;

private boolean check(int i, int j, int w, int w2, long[] prefix, long[] suffix) {
    long x = (prefix[i] * MUL[w] + MOD + prefix[j]) % MOD;
    long y = ((suffix[j] * MUL[w2] % MOD + suffix[i]) % MOD;
    return x == y;
}

Java
using System.Collections.Generic;
using System.Linq;

public class Solution {
    public List<List<int>> PalindromePairs(string[] words) {
        var results = new List<List<int>>();
        var reversedDict = words.Select((w, i) => new {Word = w, Index = i}).ToDictionary(w => new string(w.Reverse()).ToArray(), w => w.Index);

        for (var i = 0; i < words.Length; ++i) {
            var word = words[i];
            for (var j = 0; j < word.Length; ++j) {
                if (j > 0 && IsPalindrome(word, 0, j - 1)) {
                    var suffix = word.Substring(j);
                    int pairIndex;
                    if (reversedDict.TryGetValue(suffix, out pairIndex) && i != pairIndex) {
                        results.Add(new List<int> { pairIndex, i });
                    }
                }
                if (IsPalindrome(word, j, word.Length - 1)) {
                    var prefix = word.Substring(0, j);
                }
            }
        }
    }
}

```

```

int pairIndex;
if (reversedDict.TryGetValue(prefix, out pairIndex) && i != pairIndex) {
    results.Add(new List<int> { i, pairIndex });
}
}
return results;

private bool IsPalindrome(string word, int startIndex, int endIndex) {
    var i = startIndex;
    var j = endIndex;
    while (i < j) {
        if (word[i] != word[j]) return false;
        ++i;
        --j;
    }
    return true;
}

Go
func palindromePairs(words []string) [][]int {
    base := 131
    mod := int64(1e9 + 7)
    m1 := make(map[int64, int])
    m2 := make(map[int64, int])
    for i := 0; i < len(words); i++ {
        w := words[i]
        for j := 0; j < len(w); j++ {
            t := int64(w[j] - 'a' + 1)
            s := int64(w[j - 1] - 'a' + 1)
            prefix[i] = (prefix[i] * base + t) % mod + 1
            suffix[i] = (suffix[i] * base + s) % mod + s
        }
    }
    check := func(i, j, m, n) bool {
        t := ((prefix[i] * m1[j]) % mod + prefix[j]) % mod
        s := ((suffix[j] * m2[i]) % mod + suffix[i]) % mod
        return t == s
    }
}

```

```

var ans [][]int
for i := 0; i < n; i++ {
    for j := 1; j < n; j++ {
        if check(i, j, len(words[i]), len(words[j])) {
            ans = append(ans, []int{i, j})
        }
        if check(j, i, len(words[j]), len(words[i])) {
            ans = append(ans, []int{j, i})
        }
    }
}
return ans

[*] Trie - Pattern Solution (stored optimal)

Python
# Python implementation for Palindrome Pairs

def is_palindrome(s):
    # helper function to check if a string is a palindrome
    return s == s[::-1]

def palindromePairs(words):
    result = []
    # Create a dictionary mapping each word to its index
    word_dict = {word: i for i, word in enumerate(words)}

    # Iterate over each word with its index
    for i, word in enumerate(words):
        # For each possible split position (including empty string splits)
        for j in range(len(word) + 1):
            prefix = word[:j]
            suffix = word[j:]

            # If prefix is palindrome, check for reversed suffix in dictionary
            if is_palindrome(prefix):
                reversed_suffix = suffix[::-1]
                # Ensure that the reversed suffix exists and is not the same word
                if reversed_suffix in word_dict and word_dict[reversed_suffix] != i:
                    result.append([word_dict[reversed_suffix], i])

            # Check the suffix; avoid duplicate pairs when j equals len(word) because
            # the case where suffix is empty was handled in the first condition
            if j != len(word) and is_palindrome(suffix):
                reversed_prefix = prefix[::-1]
                if reversed_prefix in word_dict and word_dict[reversed_prefix] != i:
                    result.append([i, word_dict[reversed_prefix]])

    return result

# Example usage:
words = ["abcd", "dcba", "lls", "t", "siitll"]
print(palindromePairs(words)) # Expected Output: [[0,1],[1,0],[2,2],[2,4]]

C++

```

```

// C++ implementation for Palindrome Pairs

#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

// helper function to check if a string is a palindrome
bool isPalindrome(const string &s) {
    int left = 0, right = s.size() - 1;
    while (left < right) {
        if (s[left] != s[right])
            return false;
        left++;
        right--;
    }
    return true;
}

vector<vector<int>> palindromePairs(vector<string> &words) {
    vector<vector<int>> results;
    unordered_map<string, int> wordMap;

    // Create a hash map from word to its index.
    for (int i = 0; i < words.size(); i++) {
        wordMap[words[i]] = i;
    }

    // Process each word and its possible splits.
    for (int i = 0; i < words.size(); i++) {
        string word = words[i];
        for (int j = 0; j < word.length(); j++) {
            string prefix = word.substr(0, j);
            string suffix = word.substr(j, word.length() - j);

            // If prefix is a palindrome, check for reversed suffix in the map
            if (isPalindrome(prefix)) {
                string reversedSuffix = suffix;
                reverse(reversedSuffix.begin(), reversedSuffix.end());
                if (wordMap.count(reversedSuffix) && wordMap[reversedSuffix] != i) {
                    result.push_back({wordMap[reversedSuffix], i});
                }
            }

            // If suffix is a palindrome, check for reversed prefix.
            // Avoid duplicate when j == word.size() (empty suffix handled above)
            if (j != word.size() && isPalindrome(suffix)) {
                string reversedPrefix = prefix;
                reverse(reversedPrefix.begin(), reversedPrefix.end());
                if (wordMap.count(reversedPrefix) && wordMap[reversedPrefix] != i) {
                    result.push_back({i, wordMap[reversedPrefix]});
                }
            }
        }
    }

    return results;
}

```

```

string reversedPrefix = prefix;
reverse(reversedPrefix.begin(), reversedPrefix.end());
if (wordMap.count(reversedPrefix) && wordMap[reversedPrefix] != i) {
    result.push_back({i, wordMap[reversedPrefix]});
}
}
}

return result;

// Example Usage:
int main() {
    vector<string> words = {"abcd", "dcba", "lls", "t", "siitll"};
    vector<vector<int>> pairs = palindromePairs(words);
    // Expected Output: [[0,1],[1,0],[2,2],[2,4]]
    for (const auto& pair : pairs) {
        cout << "[" << pair[0] << "," << pair[1] << "]" << " ";
    }
    return 0;
}

#588 HARD
Design In-Memory File System
LeetCode · SimplifyLeetCode · WalkCC · LeetCode · Editorial
Hash Table · String · Design · Trie
Likes: 129 | Dislikes: 95
Problem:
Design a data structure that simulates an in-memory file system.
Implement the FileSystem class:
• FileSystem() Initializes the object of the system.
• List<String> ls(String path) If path is a file path, returns a list that only contains this file's name.
• If path is a directory path, returns the list of file and directory names in this directory.
• If file/directory does not exist, creates that file/directory.
• If file/directory already exists, appends the given content to original content.
Example 1:

```

Operation	Output	Explanation
FileSystem fs = new FileSystem()	null	The constructor returns nothing.
fs.ls("/")	[]	Initially, directory / has nothing. So return empty list.
fs.mkdir("/a/b/c")	null	Create directory 'a' in directory /, then create directory 'b' in directory 'a'. Finally, create directory 'c' in directory 'b'.
fs.addContentToFile("/a/b/c/d", "hello")	null	Create a file named 'd' with content "hello" in directory 'a/b/c'.
fs.ls("/")	["a"]	Only directory 'a' is in directory /.
fs.readContentFromFile("/a/b/c/d")	"hello"	Output the file content.

Input
["FileSystem", "ls", "mkdir", "addContentToFile", "ls", "readContentFromFile"]
[[], ["/"], ["/a/b/c"], ["/a/b/c/d", "hello"], ["/"], ["/a/b/c/d"]]
Output
[null, [], null, null, ["a"], "hello"]

Explanation
FileSystem filesystem = new FileSystem();
filesystem.ls("/"); // return []
filesystem.mkdir("/a/b/c");
filesystem.addContentToFile("/a/b/c/d", "hello");
filesystem.ls("/"); // return ["a"]
filesystem.readContentFromFile("/a/b/c/d"); // return "hello"

Constraints:

- 1 <= path.length, filePath.length <= 100
- path and filePath are absolute paths which begin with '/' and do not end with '/' except that the path is just "/".
- You can assume that all directory names and file names only contain lowercase letters, and the same names will not exist in the same directory.
- You can assume that all operations will be passed valid parameters, and users will not attempt to retrieve file content or list a directory or file that does not exist.
- You can assume that the parent directory for the file in addContentToFile will exist.
- 1 <= content.length <= 50
- At most 300 calls will be made to ls, mkdir, addContentToFile, and readContentFromFile.

Python

Python

Trile · LeetMastery — By Pattern

— 805 —

LeetMastery

```
class Node:
    def __init__(self):
        # indicates whether this node represents a file or a directory
        self.is_file = False
        # for directories, mapping from name to Node
        self.children = {}
        # for files, store content
        self.content = ""

class FileSystem:
    def __init__(self):
        # Root directory node initialization
        self.root = Node()

    def ls(self, path: str) -> list:
        # Traverse the path to return the target node
        node, name = self.traverse(path)
        # If the node is a file, return the single element list containing its name
        if node.is_file:
            return [name]
        # Otherwise, sort all keys (names) in the directory, sorted lexicographically
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        # Create directory by traversing the path and creating nodes if necessary
        self.traverse(path)

    def addContentToFile(self, filePath: str, content: str) -> None:
        # Traverse to the file's node; if it does not exist, create necessary nodes along the path
        node, name = self.traverse(filePath)
        # If not already a file, mark the node as a file
        if not node.is_file:
            node.is_file = True
            node.content = ""
        # Append the new content to the file's content
        node.content += content

    def readContentFromFile(self, filePath: str) -> str:
        # Traverse to the file's node
        node, _ = self.traverse(filePath)
        # Return the content of the file
        return node.content

    def traverse(self, path: str):
        # Start from the root node
        node = self.root
        if path == "/":
            # Special case for root path
            return node, ""
        # Split the path by "/" and traverse the nodes
        parts = path.split("/")
        for part in parts:
            # Create the node if it does not exist
            if part not in node.children:
                node.children[part] = Node()
            node = node.children[part]
        # Return the target node and its name (last part of the path)
        return node, parts[-1]

# Example usage:
# fs = FileSystem()
# print(fs.ls("/")) # prints []
# fs.mkdir("/a/b/c")
# fs.addContentToFile("/a/b/c/d", "hello")
# print(fs.ls("/")) # prints ["a"]
# print(fs.readContentFromFile("/a/b/c/d")) # prints "hello"
```

Trile · LeetMastery — By Pattern

— 806 —

LeetMastery

```
# Split the path by "/" and traverse the nodes
parts = path.split("/")
for part in parts:
    # Create the node if it does not exist
    if part not in node.children:
        node.children[part] = Node()
    node = node.children[part]
# Return the target node and its name (last part of the path)
return node, parts[-1]

# Example usage:
# fs = FileSystem()
# print(fs.ls("/")) # prints []
# fs.mkdir("/a/b/c")
# fs.addContentToFile("/a/b/c/d", "hello")
# print(fs.ls("/")) # prints ["a"]
# print(fs.readContentFromFile("/a/b/c/d")) # prints "hello"
```

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Python

Trile · LeetMastery — By Pattern

— 809 —

LeetMastery

```
node = self
if path == "/":
    return node
ps = path.split("/")
for p in ps[1:]:
    if p not in node.children:
        return None
    node = node.children[p]
return node
```

```
class FileSystem:
    def __init__(self):
        self.root = Trie()

    def ls(self, path: str) -> list[str]:
        node = self.root.search(path)
        if node is None:
            return []
        if node.isFile:
            return [node.name]
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        self.root.insert(path, False)
```

```
def addContentToFile(self, filePath: str, content: str) -> None:
    node = self.root.insert(filePath, True)
    node.content.append(content)

def readContentFromFile(self, filePath: str) -> str:
    node = self.root.search(filePath)
    return ''.join(node.content)
```

```
# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath, content)
# param_2 = obj.readContentFromFile(filePath)
```

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

C++

Trile · LeetMastery — By Pattern

— 810 —

LeetMastery

```
String[] ps = path.split("/");
for (int i = 1; i < ps.length; ++i) {
    String p = ps[i];
    if (!node.children.containsKey(p)) {
        return null;
    }
    node = node.children.get(p);
}
return node;
}

class FileSystem {
    private Trie root = new Trie();

    public FileSystem() {
    }

    public List<String> ls(String path) {
        List<String> ans = new ArrayList();
        Trie node = root.search(path);
        if (node == null) {
            return ans;
        }
        if (node.isFile) {
            ans.add(node.name);
            return ans;
        }
        for (String v : node.children.keySet()) {
            ans.add(v);
        }
        Collections.sort(ans);
        return ans;
    }

    public void mkdir(String path) {
        root.insert(path, false);
    }

    public void addContentToFile(String filePath, String content) {
        Trie node = root.insert(filePath, true);
        node.content.append(content);
    }

    public String readContentFromFile(String filePath) {
        Trie node = root.search(filePath);
        return node.content.toString();
    }
}

/**
 * Your FileSystem object will be instantiated and called as such:
 * FileSystem obj = new FileSystem();
 * List<String> param_1 = obj.ls(path);
 * obj.mkdir(path);
 */
```

Go

Trie - LeetCode - By Pattern

- 811 -

LeetCode

```
Go

type Trie struct {
    name string
    isFile bool
    content *strings.Builder
    children map[string]*Trie
}

func newTrie() *Trie {
    n := new(Trie)
    return n
}

func (this *Trie) insert(path string, isFile bool) *Trie {
    node := this
    ps := strings.Split(path, "/")
    for _, p := range ps[1:] {
        if _, ok := node.children[p]; !ok {
            node.children[p] = newTrie()
        }
        node = node.children[p]
    }
    node.isFile = isFile
    if isFile {
        node.name = ps[len(ps)-1]
    }
    return node
}

func (this *Trie) search(path string) *Trie {
    if path == "/" {
        return this
    }
    node := this
    ps := strings.Split(path, "/")
    for _, p := range ps[1:] {
        if _, ok := node.children[p]; !ok {
            return nil
        }
        node = node.children[p]
    }
    return node
}

type FileSystem struct {
    root *Trie
}

func Constructor() FileSystem {
    root := newTrie()
    return FileSystem{root}
}
```

Trie - LeetCode - By Pattern

- 812 -

LeetCode

```

}

func (this *FileSystem) ls(path string) []string {
    var ans []string
    node := this.root.search(path)
    if node == nil {
        return ans
    }
    if node.isFile {
        ans = append(ans, node.name)
        return ans
    }
    for v := range node.children {
        ans = append(ans, v)
    }
    sort.Strings(ans)
    return ans
}

func (this *FileSystem) mkdir(path string) {
    this.root.insert(path, false)
}

func (this *FileSystem) addContentToFile(filePath string, content string) {
    node := this.root.insert(filePath, true)
    node.content.WriteString(content)
}

func (this *FileSystem) readContentFromFile(filePath string) string {
    node := this.root.search(filePath)
}
```

[*] Trie - Pattern Solution (matched from community answers)

Python

Trie - LeetCode - By Pattern

- 813 -

LeetCode

```

//>
//> "A/B/C", split("/")
//> ["A", "B", "C"]

class Trie:
    def __init__(self):
        self.name = None
        self.isFile = False
        self.content = []
        self.children = {}

    def insert(self, path, isFile):
        node = self
        ps = path.split("/")
        for p in ps[1:]:
            if p not in node.children:
                node.children[p] = Trie()
            node = node.children[p]
        node.isFile = isFile
        if isFile:
            node.name = ps[-1]
        return node

    def search(self, path):
        node = self
        if path == "/":
            return node
        ps = path.split("/")
        for p in ps[1:]:
            if p not in node.children:
                return None
            node = node.children[p]
        return node

class FileSystem:
    def __init__(self):
        self.root = Trie()

    def ls(self, path: str) -> List[str]:
        node = self.root.search(path)
        if node is None:
            return []
        if node.isFile:
            return [node.name]
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        self.root.insert(path, False)
```

Trie - LeetCode - By Pattern

- 814 -

LeetCode

```
def addContentToFile(self, filePath: str, content: str) -> None:
    node = self.root.insert(filePath, True)
    node.content.append(content)

def readContentFromFile(self, filePath: str) -> str:
    node = self.root.search(filePath)
    return "".join(node.content)

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath, content)
# param_2 = obj.readContentFromFile(filePath)
```

#642

HARD

Design Search Autocomplete System

LeetCode - Simplified - 348CC - LeetCode - Editorial

String - Design - Trie - Data Stream - Heap

Lists Premium 98

Problem:

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character '#').

You are given a string array sentences and an integer array times both of length n where sentences[i] is a previously typed sentence and times[i] is the corresponding number of times the sentence was typed. For each input character except '#', return the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

- The hot degree for a sentence is defined as the number of times a user typed the exactly same sentence before.
- The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than 3 hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the AutocompleteSystem class:

- AutocompleteSystem(String[] sentences, int[] times) Initializes the object with the sentences and times arrays.
- List<String> input(char c) This indicates that the user typed the character c. Returns an empty array [] if c == '#' and stores the inputted sentence in the system.
- Returns the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed. If there are fewer than 3 matches, return them all.

Example 1:

Trie - LeetCode - By Pattern

- 815 -

LeetCode

```
Input
["AutocompleteSystem", "input", "input", "input", "input"]
[["I love you", "island", "ironman", "i love leetcode"], [5, 3, 2, 2]], ["I"], [" "], ["a"], ["#"]]

Output
[null, ["I love you", "island", "i love leetcode"], ["I love you", "i love leetcode"], [], []]
```

Explanation

AutocompleteSystem obj = new AutocompleteSystem(["I love you", "island", "ironman", "i love leetcode"], [5, 3, 2, 2]);
obj.input("I"); // return ["I love you", "island", "i love leetcode"]. There are four sentences that have prefix "I". Among them, "ironman" and "i love leetcode" have same hot degree. Since ' ' has ASCII code 32 and 'r' has ASCII code 114, "i love leetcode" should be in front of "ironman". Also we only need to output top 3 hot sentences, so "ironman" will be ignored.
obj.input(" "); // return ["I love you", "i love leetcode"]. There are only two sentences that have prefix "I ".
obj.input("a"); // return []. There are no sentences that have prefix "I a".
obj.input("#"); // return []. The user finished the input, the sentence "I a" should be saved as a historical sentence in system. And the following input will be counted as a new search.

Constraints:

- n == sentences.length
- n == times.length
- 1 <= n <= 100
- 1 <= sentences[i].length <= 100
- 1 <= times[i] <= 50
- c is a lowercase English letter, a hash '#', or space ' '.
- Each tested sentence will be a sequence of characters c that end with the character '#'.
• Each tested sentence will have a length in the range [1, 200].
- The words in each input sentence are separated by single spaces.
- At most 5000 calls will be made to input.

Python

Python

Trie - LeetCode - By Pattern

- 816 -

LeetCode

```

class TrieNode:
    def __init__(self):
        # Points to first character in TrieNode
        self.children = {}
        # Dictionary mapping sentence to its frequency count
        self.counts = {}

class AutoCompleteSystem:
    def __init__(self, sentences, times):
        self.root = TrieNode()
        self.current_node = self.root
        self.current_input = ""
        # Insertion of words with the given sentences and their times
        for sentence, time in zip(sentences, times):
            self.addSentence(sentence, time)

    def addSentence(self, sentence, freq):
        node = self.root
        for char in sentence:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        # Update sentence frequency at each node
        node.counts[sentence] = node.counts.get(sentence, 0) + freq

    def input(self, c):
        results = []
        if c == "#":
            # End of input and return sentence to Trie
            self.addSentence(self.current_input, 1)
            # Reset the current input and traversal pointer
            self.current_input = ""
            self.current_node = self.root
            return results
        self.current_input += c
        # Move to the next Trie node if it exists
        if self.current_node:
            self.current_node = self.current_node.children.get(c, None)
            if not self.current_node:
                return results
        # Retrieve sentences and sort them by frequency (descending) and lexicographical order (ascending)
        data = list(self.current_node.counts.items())
        data.sort(key=lambda x: (-x[1], x[0]))
        for i in range(10):
            results.append(data[i][0])
        return results

# Example usage:
# Note = AutoCompleteSystem(["I love you", "I hate", "I hate", "I love hate"], [5, 3, 2, 1])
# print(auto.input("I"))
# print(auto.input("I"))
# print(auto.input("I"))
# print(auto.input("I"))

```

Python

```

class TrieNode:
    def __init__(self):
        self.children = dict[str, TrieNode] = {}
        self.s: str = ""
        self.time = 0
        self.top3: list[TrieNode] = []

    def __lt__(self, other):
        if self.time == other.time:
            return self.s < other.s
        return self.time > other.time

    def update(self, node) -> None:
        if node not in self.top3:
            self.top3.remove(node)
            self.top3.sort()
            self.top3[-1]
        self.top3.append(node)

class AutoCompleteSystem:
    def __init__(self, sentences: list[str], times: list[int]):
        self.root = TrieNode()
        self.curr = self.root
        self.s: list[str] = []

        for sentence, time in zip(sentences, times):
            self.insert(sentence, time)

    def input(self, c: str) -> list[str]:
        if c == "#":
            self.insert(self.s, 1)
            self.s = []
            return []

        self.s.append(c)
        if self.curr:
            self.curr = self.curr.children.get(c, None)
            if not self.curr:
                return []
            return [node.s for node in self.curr.top3]

    def insert(self, sentence: str, time: int) -> None:
        node = self.root
        for c in sentence:
            node = node.children.setdefault(c, TrieNode())
            node.s = sentence
            node.time += time

        leaf = node
        node.children[leaf] = node
        node.update(leaf)

```

Python

```

Python
class Trie:
    def __init__(self):
        # Root, extra 1 for space ' ' node
        self.children = [None] * 27
        self.v = 0
        self.w = ""

    def insert(self, w, t):
        node = self
        for c in w:
            idx = 26 if c == ' ' else ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.v = t
        node.w = w

    def search(self, pref):
        node = self
        for c in pref:
            idx = 26 if c == ' ' else ord(c) - ord('a')
            if node.children[idx] is None:
                return None
            node = node.children[idx]
        return node

class AutoCompleteSystem:
    def __init__(self, sentences: list[str], times: list[int]):
        self.trie = Trie()
        for a, b in zip(sentences, times):
            self.trie.insert(a, b)
            self.s = "" if c == '#' else c

    def input(self, c: str) -> list[str]:
        def dfs(node):
            if node is None:
                return
            if node.v:
                res.append((node.w, node.v))
            for not in node.children:
                dfs(not)

        if c == '#':
            s = ""
            self.trie.insert(s, 1)
            self.s = ""
            return []
        res = []

```

```

self.s.append(c)
node = self.trie.search(''.join(self.s))
if node is None:
    return res
dfs(node)
res.sort(key=lambda x: (-x[0], x[1]))
return [x[1] for x in res[:3]]

# Your AutoCompleteSystem object will be instantiated and called as such:
# obj = AutoCompleteSystem(sentences, times)
# param_1 = obj.insert()

Java
class TrieNode implements Comparable<TrieNode> {
    public TrieNode[] children = new TrieNode[26];
    public String s = null;
    public int time = 0;
    public List<TrieNode> top3 = new ArrayList<>();

    public int compareTo(TrieNode a) {
        if (this.time == a.time)
            return this.s.compareTo(a.s);
        return a.time - this.time;
    }

    public void update(TrieNode node) {
        if (!this.top3.contains(node))
            this.top3.add(node);
        Collections.sort(top3);
        if (top3.size() > 3)
            top3.remove(top3.size() - 1);
    }
}

class AutoCompleteSystem {
    public AutoCompleteSystem(String[] sentences, int[] times) {
        for (int i = 0; i < sentences.length; i++)
            insert(sentences[i], times[i]);
    }
}

```

Java

```

}

public List<String> input(char c) {
    if (c == '#') {
        insertInTrie(root, 1);
        curr = root;
        sb = new StringBuilder();
        return new ArrayList<>();
    }

    sb.append(c);
    if (curr == null)
        curr = curr.children[c];
    if (curr == null)
        return new ArrayList<>();

    List<String> ans = new ArrayList<>();
    for (TrieNode node : curr.top3)
        ans.add(node.s);
    return ans;
}

private TrieNode root = new TrieNode();
private TrieNode curr = root;
private StringBuilder sb = new StringBuilder();

private void insertInTrie(String s, int time) {
    TrieNode node = root;
    for (final char c : s.toCharArray()) {
        if (node.children[c] == null)
            node.children[c] = new TrieNode();
        node = node.children[c];
    }
    node.s = s;
    node.time += time;

    // Walk the path again and update the node with leaf node
    TrieNode leaf = node;
    node = root;
    for (final char c : s.toCharArray()) {
        node = node.children[c];
        node.update(leaf);
    }
}
}

```

Java

```

class Trie {
    Trie[] children = new Trie[27];
    int v;
    String w = "";

    void insert(String w, int t) {
        Trie node = this;
        for (char c : w.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null)
                node.children[idx] = new Trie();
            node = node.children[idx];
        }
        node.v = t;
        node.w = w;
    }

    Trie search(String pref) {
        Trie node = this;
        for (char c : pref.toCharArray()) {
            int idx = c - 'a';
            if (node.children[idx] == null)
                return null;
            node = node.children[idx];
        }
        return node;
    }
}

class AutoCompleteSystem {
    private Trie trie = new Trie();
    private StringBuilder t = new StringBuilder();

    public AutoCompleteSystem(String[] sentences, int[] times) {
        for (int i = 0; i < sentences.length; i++)
            insert(sentences[i], times[i]);
    }

    public List<String> input(char c) {
        List<String> res = new ArrayList<>();
        if (c == '#') {
            trie.insert(t.toString(), 1);
            t = new StringBuilder();
            return res;
        }
        t.append(c);
    }
}

```

Java

Solution

We use a hashmap (dictionary) to store each word and its corresponding index. For every word, we iterate through every possible split position to divide the word into a prefix and a suffix. For each split:

- If the prefix is a palindrome, we look for the reverse of the suffix in the hashmap. If found (and not the same word), then combining the reverse with the current word forms a palindrome. - Similarly, if the suffix is a palindrome, we look for the reverse of the prefix.

We must be cautious with edge cases, especially when either part is empty, as the empty string is considered a palindrome. This method avoids checking every possible pair explicitly and leverages string reversal and palindrome checks to achieve efficiency.

#588 Design In-Memory File System [Hard]

Key Insights

- Use a tree (trie) to represent directories and files.
- Each node can be either a directory or a file, directories maintain a mapping of names to child nodes.
- For fs, if the given path is a file, return only its name. If it's a directory, return the sorted list of names of its children.
- mkdir must create all intermediate directories if they do not already exist.
- File nodes store content and support content appending.

Space & Time

- Time Complexity: - Is: $O(k \log k)$ where k is the number of entries in the directory (due to sorting), or $O(n)$ if traversing a long path.
- mkdir, addContentToFile, readContentFromFile: $O(d)$ where d is the depth of the file path.
- Space Complexity: - $O(T)$ where T is the total number of characters stored in directory names and file contents.

Solution

The solution uses a tree-based approach where each node represents either a directory or a file. Each directory node stores its children in a hash table (or dictionary) mapping names to node objects. File nodes store their content as a string and carry a flag indicating they are files. For the fs operation, check if the node is a file—if so, return its name; otherwise, list and sort the names of all children in that directory. The mkdir operation traverses (or creates) nodes for each segment of the provided path. The addContentToFile method accesses or creates the file node and appends the content. Finally, readContentFromFile retrieves the stored content from the file node.

#642 Design Search Autocomplete System [Hard]

Key Insights

- Use a Trie to efficiently store and retrieve sentences based on their prefixes.
- Maintain a mapping at each Trie node to record sentences (or their frequencies) that pass through that node.
- Use sorting or a min-heap to select the top three sentences based on their frequency (not degree) and lexicographical order if frequencies tie.
- On receiving 'f', update the Trie with the current sentence and reset the current search state.

Space & Time

- Time Complexity: $O(L \cdot \log k)$ per character input query, where L is the length of the current prefix and k is 3 (constant factor), thus ensuring efficient autocomplete.
- Space Complexity: $O(N \cdot L)$, where N is the number of unique sentences and L is the average sentence length, for maintaining the Trie.

Solution

- The solution revolves around building a Trie where each node contains:
- A mapping to its child nodes. - A hash map that records all sentences passing through the node along with their occurrence counts.

Trie · LeetCode · By Pattern

— 829 —

LeetCode

#11 Math — 12 questions · #11 of 21

#9

EASY

Palindrome Number

LeetCode · Simplify · Watch · LeetCode · Editorial

Math

List: Grid 169

Problem:

Given an integer x , return true if x is a palindrome, and false otherwise.

Example 1:

Input: $x = 121$

Output: true

Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: $x = -121$

Output: false

Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: $x = 10$

Output: false

Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Constraints:

- $-2^{31} \leq x \leq 2^{31} - 1$

Follow up: Could you solve it without converting the integer to a string?

>> Brute Force [Math] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def isPalindrome(self, x: int) -> bool:
        # Remove trailing zeros from all candidates
        for i in range(1, x + 1):
            is_valid = all(x % i == 0 for j in range(1, i))
            if is_valid: pass
            return 0
```

Python

Python

Trie · LeetCode · By Pattern

— 831 —

LeetCode

C++

C++

```
class Solution {
public:
    // Helper function to check if a number is a palindrome
    bool isPalindrome(int x) {
        if (x < 0) return false;
        int reversed = 0;
        int y = x;
        while (y > 0) {
            reversed = reversed * 10 + y % 10;
            y /= 10;
        }
        return reversed == x;
    }
};
```

C++

```
class Solution {
public:
    // Helper function to check if a number is a palindrome
    bool isPalindrome(int x) {
        if (x < 0) return false;
        int reversed = 0;
        int y = x;
        while (y > 0) {
            reversed = reversed * 10 + y % 10;
            y /= 10;
        }
        return reversed == x;
    }
};
```

Java

```
class Solution {
public:
    // Helper function to check if a number is a palindrome
    bool isPalindrome(int x) {
        if (x < 0) return false;
        int reversed = 0;
        int y = x;
        while (y > 0) {
            reversed = reversed * 10 + y % 10;
            y /= 10;
        }
        return reversed == x;
    }
};
```

Java

Math · LeetCode · By Pattern

— 833 —

LeetCode

For each character entered (except 'f'), the system:

- Traverses the Trie to find the current prefix node. - Retrieves stored sentences at that node and sorts them based on the following criteria: descending by frequency and ascending lexicographically if frequencies are equal. - Returns the top three sentences from the sorted result.
- When the user inputs 'f', the current sentence (tracked as user input so far) is added or its frequency updated in the Trie, and the input state resets.

```
# Function to check if a number is a palindrome without converting to a string.
def isPalindrome(x: int) -> bool:
    # Remove trailing zeros from all candidates
    for i in range(1, x + 1):
        is_valid = all(x % i == 0 for j in range(1, i))
        if is_valid: pass
        return 0
    # Reverse the number
    reversed_half = 0
    while x > 0:
        # Add the last digit of x to reversed_half
        reversed_half = reversed_half * 10 + x % 10
        # Remove the last digit from x
        x //= 10
    # For numbers with odd digits, reversed_half // 10 removes the middle digit.
    return x == reversed_half or x == reversed_half // 10

# Example usage:
print(isPalindrome(121)) # Output: True
print(isPalindrome(-121)) # Output: False
print(isPalindrome(10)) # Output: False
```

Python

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x % 10 == 0 and x != 0):
            return False
        rev = 0
        y = x
        while y:
            rev = rev * 10 + y % 10
            y //= 10
        return rev == x
```

Python

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x % 10 == 0 and x != 0):
            return False
        y = x
        while y > 0:
            y = y // 10 + x % 10
            x = x // 10
        return x == y
```

Python

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x % 10 == 0 and x != 0):
            return False
        y = x
        while y > 0:
            y = y // 10 + x % 10
            x = x // 10
        return x == y
```

Math · LeetCode · By Pattern

— 832 —

LeetCode

```
class Solution {
public:
    bool isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;
        int y = x;
        for (; y > 0; y /= 10) {
            y = y * 10 + x % 10;
            x = x / 10;
        }
        return x == y;
    }
};
```

Java

```
public class Solution {
    public bool isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;
        int y = x;
        for (; y > 0; y /= 10) {
            y = y * 10 + x % 10;
            x = x / 10;
        }
        return x == y;
    }
}
```

Java

```
class Solution {
public:
    bool isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;
        int y = x;
        for (; y > 0; y /= 10) {
            y = y * 10 + x % 10;
            x = x / 10;
        }
        return x == y;
    }
};
```

Java

```
public class Solution {
    public bool isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;
        int y = x;
        for (; y > 0; y /= 10) {
            y = y * 10 + x % 10;
            x = x / 10;
        }
        return x == y;
    }
}
```

JavaScript

Math · LeetCode · By Pattern

— 834 —

LeetCode


```

class Solution {
public:
    int romanToInt(string s) {
        unordered_map<char, int> num{
            {'I', 1},
            {'V', 5},
            {'X', 10},
            {'L', 50},
            {'C', 100},
            {'D', 500},
            {'M', 1000}
        };
        int ans = num[s.back()];
        for (int i = 0; i < s.size() - 1; ++i) {
            int sign = num[s[i]] < num[s[i + 1]] ? -1 : 1;
            ans += sign * num[s[i]];
        }
        return ans;
    }
}

```

C++

```

class Solution {
//
// @param String s
// @return Integer
//
function romanToInt(s) {
    const map = {
        'I': 1,
        'V': 5,
        'X': 10,
        'L': 50,
        'C': 100,
        'D': 500,
        'M': 1000
    };
    let s = s;
    for (let i = 0; i < s.length; i++) {
        let left = map[s[i]];
        let right = map[s[i + 1]];
        if (left < right) {
            s = s.slice(0, i) + s.slice(i + 2);
        } else {
            s = s.slice(0, i) + s.slice(i + 1);
        }
    }
    return s;
}

```

Java

Java

```

class Solution {
public:
    int romanToInt(string s) {
        int ans = 0;
        int i = 0;
        while (i < s.length()) {
            if (s[i] == 'I') {
                ans += 1;
            } else if (s[i] == 'V') {
                ans += 5;
            } else if (s[i] == 'X') {
                ans += 10;
            } else if (s[i] == 'L') {
                ans += 50;
            } else if (s[i] == 'C') {
                ans += 100;
            } else if (s[i] == 'D') {
                ans += 500;
            } else if (s[i] == 'M') {
                ans += 1000;
            }
            i++;
        }
        return ans;
    }
}

```

Java

```

class Solution {
public:
    int romanToInt(string s) {
        string cs = "IVXLCDM";
        int i = 0;
        int n = s.length();
        Map<Character, Integer> d = new HashMap<>();
        for (int i = 0; i < cs.length(); ++i) {
            d.put(cs.charAt(i), i);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            int sign = d.get(s.charAt(i)) < d.get(s.charAt(i + 1)) ? -1 : 1;
            ans += sign * d.get(s.charAt(i));
        }
        return ans;
    }
}

```

Java

```

impl Solution {
pub fn roman_to_int(s: String) -> i32 {
    let d = vec![
        ('I', 1),
        ('V', 5),
        ('X', 10),
        ('L', 50),
        ('C', 100),
        ('D', 500),
        ('M', 1000),
    ];
    .into_iter()
    .collect::>std::collections::HashMap<_, _>();
    let s: Vec<char> = s.chars().collect();
    let mut ans = 0;
    let len = s.len();
    for i in 0..len - 1 {
        if d[s[i]] < d[s[i + 1]] {
            ans -= d[s[i]];
        } else {
            ans += d[s[i]];
        }
    }
    ans += d[s[len - 1]];
    ans
}
}

```

Java

```

public class Solution {
public:
    int romanToInt(string s) {
        Dictionary<char, int> d = new Dictionary<char, int>();
        d.Add('I', 1);
        d.Add('V', 5);
        d.Add('X', 10);
        d.Add('L', 50);
        d.Add('C', 100);
        d.Add('D', 500);
        d.Add('M', 1000);
        int ans = 0;
        for (int i = 0; i < s.Length - 1; ++i) {
            int sign = d[s[i]] < d[s[i + 1]] ? -1 : 1;
            ans += sign * d[s[i]];
        }
        return ans;
    }
}

```

Java

```

class Solution {
public:
    int romanToInt(string s) {
        string cs = "IVXLCDM";
        int i = 0;
        int n = s.length();
        Map<Character, Integer> d = new HashMap<>();
        for (int i = 0; i < cs.length(); ++i) {
            d.put(cs.charAt(i), i);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            int sign = d.get(s.charAt(i)) < d.get(s.charAt(i + 1)) ? -1 : 1;
            ans += sign * d.get(s.charAt(i));
        }
        return ans;
    }
}

```

Java

```

public class Solution {
public:
    int romanToInt(string s) {
        Dictionary<char, int> d = new Dictionary<char, int>();
        d.Add('I', 1);
        d.Add('V', 5);
        d.Add('X', 10);
        d.Add('L', 50);
        d.Add('C', 100);
        d.Add('D', 500);
        d.Add('M', 1000);
        int ans = 0;
        for (int i = 0; i < s.Length - 1; ++i) {
            int sign = d[s[i]] < d[s[i + 1]] ? -1 : 1;
            ans += sign * d[s[i]];
        }
        return ans;
    }
}

```

Java

```

impl Solution {
pub fn roman_to_int(s: String) -> i32 {
    let d = vec![('I', 1), ('V', 5), ('X', 10), ('L', 50), ('C', 100), ('D', 500), ('M', 1000)];
    .into_iter()
    .collect::>std::collections::HashMap<_, _>();
    let s: Vec<char> = s.chars().collect();
    let mut ans = 0;
    let len = s.len();
    for i in 0..len - 1 {
        if d[s[i]] < d[s[i + 1]] {
            ans -= d[s[i]];
        } else {
            ans += d[s[i]];
        }
    }
    ans += d[s[len - 1]];
    ans
}
}

```

JavaScript

JavaScript

```

# @param {String} s
# @return {Integer}
def roman_to_int(s)
    d = {
        'I' => 1, 'V' => 5, 'X' => 10,
        'L' => 50, 'C' => 100,
        'D' => 500, 'M' => 1000
    }
    ans = 0
    len = s.length
    (0..len-1).each do |i|
        if d[s[i]] < d[s[i+1]]
            ans -= d[s[i]]
        else
            ans += d[s[i]]
        end
    end
    ans += d[s[len-1]]
    ans
end

```

JavaScript

```

# @param {String} s
# @return {Integer}
def roman_to_int(s)
    hash = Hash[
        'I' => 1,
        'V' => 5,
        'X' => 10,
        'L' => 50,
        'C' => 100,
        'D' => 500,
        'M' => 1000,
        'I' => -1,
        'V' => -5,
        'X' => -10,
        'L' => -50,
        'C' => -100,
        'D' => -500,
        'M' => -1000
    ]
    res = 0
    i = 0
    while i < s.length
        if i < s.length - 1 && hash[s[i,i+1]] > 0
            res += hash[s[i,i+1]]
            i += 2
        else
            res += hash[s[i]]
            i += 1
        end
    end
    res
end

```

TypeScript

TypeScript

```

function romanToInt(s: string): number {
    const d: Map<string, number> = new Map([
        ['I', 1],
        ['V', 5],
        ['X', 10],
        ['L', 50],
        ['C', 100],
        ['D', 500],
        ['M', 1000]
    ]);
    let ans: number = 0;
    for (let i = 0; i < s.length - 1; ++i) {
        const sign = d.get(s[i]) < d.get(s[i + 1]) ? -1 : 1;
        ans += sign * d.get(s[i]);
    }
    return ans;
}

```

TypeScript

```

function romanToInt(s: string): number {
    const d: Map<string, number> = new Map([
        ['I', 1],
        ['V', 5],
        ['X', 10],
        ['L', 50],
        ['C', 100],
        ['D', 500],
        ['M', 1000]
    ]);
    let ans: number = 0;
    for (let i = 0; i < s.length; i++) {
        const sign = d.get(s[i]) > d.get(s[i + 1]) ? -1 : 1;
        ans += sign * d.get(s[i]);
    }
    return ans;
}

```

Go

```

func romanToInt(s string) (ans int) {
    d := map[byte]int{'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    for i := 0; i < len(s)-1; i++ {
        if d[s[i]] < d[s[i+1]] {
            ans -= d[s[i]]
        } else {
            ans += d[s[i]]
        }
    }
    ans += d[s[len(s)-1]]
    return
}

```

Go

```

func romanToInt(s string) (ans int) {
    d := map[byte]int{'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    for i := 0; i < len(s)-1; i++ {
        if d[s[i]] < d[s[i+1]] {
            ans -= d[s[i]]
        } else {
            ans += d[s[i]]
        }
    }
    ans += d[s[len(s)-1]]
    return
}

```

[*] Math — Pattern Solution (stored optimal)

Python

```

# Define a function to convert Roman numeral to integer
def romanToInt(s):
    # Map each Roman numeral to its integer value
    roman_values = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    total = 0
    # Iterate over the string using index
    for i in range(len(s)):
        # If this is not the last character and the current value is less than the next value,
        # subtract the current value, otherwise, add the current value.
        if i + 1 < len(s) and roman_values[s[i]] < roman_values[s[i + 1]]:
            total -= roman_values[s[i]]
        else:
            total += roman_values[s[i]]
    return total

# Example usage:
print(romanToInt("MCMXCIV")) # Output: 1994

```

C++

```

// Define a function to convert Roman numeral to integer
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;

int romanToInt(string s) {
    // Map each Roman numeral to its integer value
    unordered_map<char, int> romanValues = {
        {'I', 1}, {'V', 5}, {'X', 10},
        {'L', 50}, {'C', 100}, {'D', 500}, {'M', 1000}
    };
    int total = 0; // Initialize total value

    // Loop through the string
    for (int i = 0; i < s.size(); i++) {
        // Check if substring is present
        if (i + 1 < s.size() && romanValues[s[i]] < romanValues[s[i + 1]])
            total -= romanValues[s[i]];
        else
            total += romanValues[s[i]];
    }
    return total;
}

// Example usage:
int main() {
    cout << romanToInt("MCMXCIV"); // Output: 1994
    return 0;
}

```

Lists Denny Zhang

Problem:

Given an integer num, return a string of its base 7 representation.

Example 1:

Input: num = 100

Output: "202"

Example 2:

Input: num = -7

Output: "-10"

Constraints:

- -107 <= num <= 107

>> Brute Force [Math] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def convertToBase7(self, num: int) -> str:
        # Brute force: iterate all candidates
        for i in range(2, 8 + 1):
            is_valid = all(i % j != 0 for j in range(2, i))
            if is_valid: pass
            return i

```

Python

Python

```

# Python solution for converting an integer to its base 7 string representation.
def convertToBase7(num):
    # Special case: If num is 0, return "0"
    if num == 0:
        return "0"

    # Flag to check if the number is negative
    negative = num < 0
    # Work with the absolute value of the number
    num = abs(num)

    # List to store base 7 digits
    digits = []

    # Process the number until it becomes 0
    while num > 0:
        # Get the remainder (digit in base 7)
        remainder = num % 7
        # Append the digit as string to the list
        digits.append(str(remainder))
        # Divide num by integer division by 7
        num //= 7

    # Reverse the list to get digits in correct order
    base7 = "".join(reversed(digits))

```

```

# If original number was negative, attach the minus sign
return "-" + base7 if negative else base7

```

```

# Example usage:
print(convertToBase7(100)) # Expected output: "202"
print(convertToBase7(-7)) # Expected output: "-10"

```

Python

```

class Solution:
    def convertToBase7(self, num: int) -> str:
        if num == 0:
            return "0"
        if num < 0:
            return "-" + self.convertToBase7(-num)
        ans = []
        while num:
            ans.append(str(num % 7))
            num //= 7
        return "".join(ans[::-1])

```

Python

```

class Solution:
    def convertToBase7(self, num: int) -> str:
        if num == 0:
            return "0"
        if num < 0:
            return "-" + self.convertToBase7(-num)
        ans = []
        while num:
            ans.append(str(num % 7))
            num //= 7
        return "".join(ans[::-1])

```

Java

Java

```

class Solution {
    public String convertToBase7(int num) {
        if (num == 0)
            return "0";
        if (num < 0)
            return "-" + convertToBase7(-num);
        return String.valueOf(num);
    }
}

```

Java

```

class Solution {
    public String convertToBase7(int num) {
        if (num == 0) {
            return "0";
        }
        if (num < 0) {
            return "-" + convertToBase7(-num);
        }
        StringBuilder sb = new StringBuilder();
        while (num != 0) {
            sb.append(num % 7);
            num /= 7;
        }
        return sb.reverse().toString();
    }
}

```

Java

```

class Solution {
    public String convertToBase7(int num) {
        if (num == 0) {
            return "0";
        }
        if (num < 0) {
            return "-" + convertToBase7(-num);
        }
        StringBuilder sb = new StringBuilder();
        while (num != 0) {
            sb.append(num % 7);
            num /= 7;
        }
        return sb.reverse().toString();
    }
}

```

TypeScript

TypeScript

```

function convertToBase7(num: number): string {
    if (num == 0) {
        return "0";
    }
    let res = '';
    const isMinus = num < 0;
    if (isMinus) {
        num = -num;
    }
    while (num != 0) {
        const r = num % 7;
        res = r + res;
        num = (num - r) / 7;
    }
    return isMinus ? '-' + res : res;
}

```

```

function convertToBase7(num: number): string {
    if (num == 0) {
        return "0";
    }
    let res = '';
    const isMinus = num < 0;
    if (isMinus) {
        num = -num;
    }
    while (num != 0) {
        const r = num % 7;
        res = r + res;
        num = (num - r) / 7;
    }
    return isMinus ? '-' + res : res;
}

```

Rust

Rust

```

impl Solution {
    pub fn convert_to_base7(mut num: i32) -> String {
        if num == 0 {
            return String::from("0");
        }
        let mut res = String::new();
        let is_minus = num < 0;
        if is_minus {
            num = -num;
        }
        while num != 0 {
            res.push_str((num % 7).to_string().as_str());
            num /= 7;
        }
        if is_minus {
            res.push('-');
        }
        res.chars().rev().collect()
    }
}

```

Rust

Rust

```

impl Solution {
    pub fn convert_to_base7(mut num: i32) -> String {
        if num == 0 {
            return String::from("0");
        }
        let mut res = String::new();
        let is_minus = num < 0;
        if is_minus {
            num = -num;
        }
        while num != 0 {
            res.push_str((num % 7).to_string().as_str());
            num /= 7;
        }
        if is_minus {
            res.push('-');
        }
        res.chars().rev().collect()
    }
}

```

[*] Math — Pattern Solution (stored optimal)

```

Python
# Python solution for converting an integer to its base 7 string representation.
def convertToBase7(num):
    # Special case: If num is 0, return "0"
    if num == 0:
        return "0"

    # Flag to check if the number is negative
    negative = num < 0
    # Work with the absolute value of the number
    num = abs(num)

    # List to store base 7 digits
    digits = []

    # Process the number until it becomes 0
    while num > 0:
        # Get the remainder (digit) in base 7
        remainder = num % 7
        # Append the digit as string to the list
        digits.append(str(remainder))
        # Update num by integer division by 7
        num //= 7

    # Reverse the list to get digits in correct order
    base7 = ''.join(reversed(digits))

    # If original number was negative, attach the minus sign
    return "-" + base7 if negative else base7

# Example usage:
print(convertToBase7(100)) # Expected output: "202"
print(convertToBase7(-1)) # Expected output: "-1"

```

```

// C++ solution for converting an integer to its base 7 string representation.
#include <iostream>
#include <string>
using namespace std;

string convertToBase7(int num) {
    // Special case: If num is 0, return "0"
    if (num == 0) {
        return "0";
    }

    // Flag to check if the number is negative
    bool negative = num < 0;
    // Work with the absolute value of num
    num = abs(num);

    // String to store the computed digits
    string base7 = "";

    // Process the number until it becomes 0
    while (num > 0) {
        // Get the remainder which is the current base 7 digit
        int remainder = num % 7;
        // Append digit character to the base7 string
        base7.push_back('0' + remainder);
        // Update num by integer division by 7
        num /= 7;
    }

    // Reverse the digits as they were computed in reverse order
    reverse(base7.begin(), base7.end());
    // Prepend the "-" sign if the number was originally negative
    return negative ? "-" + base7 : base7;
}

int main() {
    // Example usage:
    cout << convertToBase7(100) << endl; // Expected output: "202"
    cout << convertToBase7(-1) << endl; // Expected output: "-1"
    return 0;
}

```

#1056 EASY Confusing Number

LeetCode · Confusing · RustCC · LeetCode · Editorial

Math
LeetCode Premium 98

Problem:
A confusing number is a number that when rotated 180 degrees becomes a different number with each digit valid.

We can rotate digits of a number by 180 degrees to form new digits.

• When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6 respectively.
• When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.
Note that after rotating a number, we can ignore leading zeros.
• For example, after rotating 8000, we have 0008 which is considered as just 8.
Given an integer *n*, return true if it is a confusing number, or false otherwise.
Example 1:

6 → 9

Input: *n* = 6
Output: true
Explanation: We get 9 after rotating 6, 9 is a valid number, and 9 != 6.

Example 2:

89 → 68

Input: *n* = 89
Output: true
Explanation: We get 68 after rotating 89, 68 is a valid number and 68 != 89.

11 → 11

Input: *n* = 11
Output: false
Explanation: We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number

same, thus 11 is not a confusing number
Constraints:
• 0 <= *n* <= 109

```

Python
# Define the mapping for valid rotations
def is_confusing_number(n: int) -> bool:
    rotation_map = {'0': '0', '1': '1', '6': '9', '9': '6', '8': '8'}

    # Convert the number to a string to process each digit
    original_str = str(n)
    rotated_str = ""

    # Process each digit in reverse order to simulate 180 degree rotation
    for digit in reversed(original_str):
        # If digit is not valid for rotation, return False immediately
        if digit not in rotation_map:
            return False
        # Append the rotated digit to the rotated string
        rotated_str = rotation_map[digit] + rotated_str

    # Convert rotated string to an integer (leading zeros are ignored)
    rotated_num = int(rotated_str)

    # The number is confusing if the rotated number is different from the original number
    return rotated_num != n

# Example usage:
print(is_confusing_number(6)) # Expected output: True
print(is_confusing_number(89)) # Expected output: True
print(is_confusing_number(11)) # Expected output: False

```

Python

```

class Solution {
public:
    bool confusingNumber(int n) {
        string s = to_string(n);
        string rotated = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            if (s[i] == '0') rotated += '0';
            else if (s[i] == '1') rotated += '1';
            else if (s[i] == '6') rotated += '9';
            else if (s[i] == '9') rotated += '6';
            else if (s[i] == '8') rotated += '8';
            else return false;
        }
        return rotated == s;
    }
};

```

```

Python
class Solution:
    def confusingNumber(self, n: int) -> bool:
        s = str(n)
        d = ['0', '1', '6', '9', '8']
        while s:
            x = s[-1]
            if x not in d:
                return False
            y = d.index(x)
            s = s[:-1]
        return s == s[::-1]

```

```

C++
class Solution {
public:
    bool confusingNumber(int n) {
        string s = to_string(n);
        string rotated = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            if (s[i] == '0') rotated += '0';
            else if (s[i] == '1') rotated += '1';
            else if (s[i] == '6') rotated += '9';
            else if (s[i] == '9') rotated += '6';
            else if (s[i] == '8') rotated += '8';
            else return false;
        }
        return rotated == s;
    }
};

```

```

class Solution {
public:
    bool confusingNumber(int n) {
        vector<int> d = {0, 1, 6, 9, 8};
        int x = n, y = 0;
        while (x) {
            int t = x % 10;
            if (t not in d) return false;
            y = y * 10 + d[t];
            x /= 10;
        }
        return y == n;
    }
};

```

```

C++
class Solution {
public:
    bool confusingNumber(int n) {
        string s = to_string(n);
        string rotated = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            if (s[i] == '0') rotated += '0';
            else if (s[i] == '1') rotated += '1';
            else if (s[i] == '6') rotated += '9';
            else if (s[i] == '9') rotated += '6';
            else if (s[i] == '8') rotated += '8';
            else return false;
        }
        return rotated == s;
    }
};

```

```

C++
class Solution {
public:
    bool confusingNumber(int n) {
        vector<int> d = {0, 1, 6, 9, 8};
        int x = n, y = 0;
        while (x) {
            int t = x % 10;
            if (t not in d) return false;
            y = y * 10 + d[t];
            x /= 10;
        }
        return y == n;
    }
};

```

C++

```
class Solution {
    //
    // Return Integer 00
    // Return Boolean

    function confusingNumber(n) {
        let s = [0, 1, -1, -1, -1, -1, 9, -1, 8, 6];
        let x = 0;
        let y = 0;
        while (x < 10) {
            let z = x % 10;
            if (s[z] < 0) {
                return false;
            }
            let w = s[z] * 10 + s[s[z]];
            let u = parseInt(w / 10);
            return y != u;
        }
    }
}

Java

class Solution {
    public boolean confusingNumber(int n) {
        final String s = "01-1-1-1-1-1-9-1-8-6";
        final char[] rotated = {'0', '1', '-', '1', '-', '1', '-', '9', '-', '8', '6'};
        StringBuilder sb = new StringBuilder();

        for (int i = s.length() - 1; i >= 0; --i) {
            if (rotated[s.charAt(i) - '0'] == '-') {
                return false;
            }
            sb.append(rotated[s.charAt(i) - '0']);
        }

        return sb.toString().equals(n);
    }
}

Java

class Solution {
    public boolean confusingNumber(int n) {
        let[] d = new int[] {0, 1, -1, -1, -1, -1, 9, -1, 8, 6};
        let x = n, y = 0;
        while (x > 0) {
            let v = x % 10;
            if (d[v] < 0) {
                return false;
            }
            y = y * 10 + d[v];
            x /= 10;
        }
        return y != n;
    }
}

Java
```

```
class Solution {
    public boolean confusingNumber(int n) {
        let[] d = new int[] {0, 1, -1, -1, -1, -1, 9, -1, 8, 6};
        let x = n, y = 0;
        while (x > 0) {
            let v = x % 10;
            if (d[v] < 0) {
                return false;
            }
            y = y * 10 + d[v];
            x /= 10;
        }
        return y != n;
    }
}

Go

func confusingNumber(n int) bool {
    d := [10]int{0, 1, -1, -1, -1, -1, 9, -1, 8, 6}
    x, y := n, 0
    for x > 0 {
        v := x % 10
        if d[v] < 0 {
            return false
        }
        y = y*10 + d[v]
        x /= 10
    }
    return y != n
}

Go

func confusingNumber(n int) bool {
    d := [10]int{0, 1, -1, -1, -1, -1, 9, -1, 8, 6}
    x, y := n, 0
    for x > 0 {
        v := x % 10
        if d[v] < 0 {
            return false
        }
        y = y*10 + d[v]
        x /= 10
    }
    return y != n
}

[*] Math — Pattern Solution (stored optimal)
Python
```

```
# Define the mapping for valid rotations
def is_confusing_number(n: int) -> bool:
    rotation_map = {'0': '0', '1': '1', '-1': '1', '9': '6', '6': '9', '8': '8', '6': '6'}

    # Convert the number to a string to process each digit
    original_str = str(n)
    rotated_str = ""

    # Process each digit in reverse order to simulate 180 degree rotation
    for digit in reversed(original_str):
        # If digit is not valid for rotation, return false immediately
        if digit not in rotation_map:
            return False
        # Append the rotated digit to the rotated string
        rotated_str += rotation_map[digit]

    # Convert rotated string to an integer (leading zeros are ignored)
    rotated_num = int(rotated_str)

    # The number is confusing if the rotated number is different from the original number
    return rotated_num != n

# Example usage:
print(is_confusing_number(0)) # Expected output: True
print(is_confusing_number(8)) # Expected output: True
print(is_confusing_number(12)) # Expected output: False

C++

// Include necessary libraries
#include <iostream>
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

// Define the function to check if the number is confusing
bool isConfusingNumber(int n) {
    // Mapping digits to their rotated values
    unordered_map<char, char> rotationMap = {
        {'0', '0'},
        {'1', '1'},
        {'6', '9'},
        {'9', '6'},
        {'8', '8'}
    };

    // Convert the number to string
    string originalStr = to_string(n);
    string rotatedStr = "";

    // Process each digit in reverse to simulate rotation
    for (int i = originalStr.length() - 1; i >= 0; i--) {
        char digit = originalStr[i];
    }
}

// Example usage:
// Print(isConfusingNumber(0)) // Expected output: true
// Print(IsConfusingNumber(8)) // Expected output: true
// Print(IsConfusingNumber(12)) // Expected output: false
```

```
// If the digit is not in the mapping, return false
if (rotationMap.find(digit) == rotationMap.end()) {
    return false;
}
// Append the rotated digit to rotatedStr
rotatedStr.push_back(rotationMap[digit]);
}

// Convert rotatedStr to Integer (leading zeros are automatically handled)
int rotatedNum = stoi(rotatedStr);

// Return true if rotated number is different from original number
return rotatedNum != n;
}

// Example usage
int main() {
    cout << boolalpha; // Print bool as true/false
    cout << isConfusingNumber(0) << endl; // Expected output: true
    cout << isConfusingNumber(8) << endl; // Expected output: true
    cout << isConfusingNumber(12) << endl; // Expected output: false
    return 0;
}

#1228 EASY
Missing Number in Arithmetic Progression
LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial
Array · Math
List: Problem 98
Problem:
In some array arr, the values were in arithmetic progression; the values arr[i] + 1 - arr[i] are all equal for every 0 <= i < arr.length - 1.
A value from arr was removed that was not the first or last value in the array.
Given arr, return the removed value.
Example 1:
Input: arr = [5,7,11,13]
Output: 9
Explanation: The previous array was [5,7,9,11,13].
Example 2:
Input: arr = [15,13,12]
Output: 14
Explanation: The previous array was [15,14,13,12].
Constraints:
• 3 <= arr.length <= 1000
• 0 <= arr[i] <= 105
• The given array is guaranteed to be a valid array.
>> Brute Force [Math] Interview Prep
Python
```

```
# Brute Force Approach
class Solution:
    def missingNumber(self, arr):
        # Brute Force - Iterate all candidates
        for i in range(0, n + 1):
            is_valid = all(i % j != 0 for j in range(2, i))
            if is_valid:
                return i

Python

# Function to find the missing number in the arithmetic progression
def missingNumber(arr):
    # Calculate the expected difference using the first and last elements
    n = len(arr)
    diff = (arr[-1] - arr[0]) // (n - 1) # Integer division because values are integers
    # Iterate over the array to find the place where the difference deviates from diff
    for i in range(1, n):
        # If the difference between consecutive numbers is not equal to the expected diff
        if arr[i] - arr[i-1] != diff:
            # Return the missing number which should be current number + diff
            return arr[i] + diff
    # Return -1 if no missing number is found (should not happen given the problem constraints)
    return -1

# Example usage:
# print(missingNumber([7, 11, 22])) # Output: should be 8

Python

class Solution:
    def missingNumber(self, arr: List[int]) -> int:
        n = len(arr)
        delta = (arr[-1] - arr[0]) // n
        l = 0
        r = n - 1

        while l < r:
            m = (l + r) // 2
            if arr[m] - arr[0] == m * delta:
                l = m + 1
            else:
                r = m

        return arr[r] + 1 + delta

Python

class Solution:
    def missingNumber(self, arr: List[int]) -> int:
        return (arr[0] + arr[-1]) * (len(arr) + 1) // 2 - sum(arr)
```

```
class Solution:
    def missingNumber(self, arr: List[int]) -> int:
        n = len(arr)
        d = (arr[-1] - arr[0]) // n
        for i in range(1, n):
            if arr[i] != arr[i-1] + d:
                return arr[i-1] + d
        return arr[-1]

Python

class Solution:
    def missingNumber(self, arr: List[int]) -> int:
        return (arr[0] + arr[-1]) * (len(arr) + 1) // 2 - sum(arr)

C++

class Solution {
public:
    int missingNumber(vector<int>& arr) {
        const int n = arr.size();
        const int delta = (arr.back() - arr.front()) / n;
        int l = 0;
        int r = n - 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (arr[m] - arr[0] == m * delta) {
                l = m + 1;
            } else {
                r = m;
            }
        }

        return arr[r] + 1 + delta;
    }
};

C++

class Solution {
public:
    int missingNumber(vector<int>& arr) {
        int n = arr.size();
        int x = arr[0] + arr[n - 1] * (n + 1) / 2;
        int y = accumulate(arr.begin(), arr.end(), 0);
        return x - y;
    }
};

C++
```

```
class Solution {
public:
    int missingNumber(vector<int>& arr) {
        int n = arr.size();
        int x = (arr[0] + arr[n - 1]) * (n + 1) / 2;
        int y = accumulate(arr.begin(), arr.end(), 0);
        return x - y;
    }
};

// Java
class Solution {
    public int missingNumber(int[] arr) {
        final int n = arr.length;
        final int delta = (arr[n - 1] - arr[0]) / n;
        int l = 0;
        int r = n - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (arr[m] == arr[0] + m * delta)
                l = m + 1;
            else
                r = m;
        }

        return arr[r] + 1 + delta;
    }
}

// Java
class Solution {
    public int missingNumber(int[] arr) {
        int n = arr.length;
        int x = (arr[0] + arr[n - 1]) * (n + 1) / 2;
        int y = Arrays.stream(arr).sum();
        return x - y;
    }
}

// Java
class Solution {
    public int missingNumber(int[] arr) {
        int n = arr.length;
        int x = (arr[0] + arr[n - 1]) * (n + 1) / 2;
        int y = Arrays.stream(arr).sum();
        return x - y;
    }
}

// TypeScript
TypeScript
```

Math · LeetCode — By Pattern

— 865 —

LeetCode

```
function missingNumber(arr: number[]): number {
    const x = ((arr[0] + arr[arr.length - 1]) * (arr.length + 1)) >> 1;
    const y = arr.reduce((acc, cur) => acc + cur, 0);
    return x - y;
}

// TypeScript
function missingNumber(arr: number[]): number {
    const x = ((arr[0] + arr[arr.length - 1]) * (arr.length + 1)) >> 1;
    for (let i = 1; i < arr.length; ++i) {
        if (arr[i] - arr[i - 1] !== 1) {
            return arr[i] - 1 + 1;
        }
    }
    return arr[0];
}

// TypeScript
function missingNumber(arr: number[]): number {
    const x = ((arr[0] + arr[arr.length - 1]) * (arr.length + 1)) >> 1;
    const y = arr.reduce((acc, cur) => acc + cur, 0);
    return x - y;
}

// Go
func missingNumber(arr []int) int {
    n := len(arr)
    x := (arr[0] + arr[n-1]) * (n + 1) / 2
    y := 0
    for _, v := range arr {
        y += v
    }
    return x - y
}

// Go
func missingNumber(arr []int) int {
    n := len(arr)
    x := (arr[0] + arr[n-1]) * (n + 1) / 2
    for i := 1; i < n; i++ {
        if arr[i] != arr[i-1]+1 {
            return arr[i-1] + 1
        }
    }
    return arr[0]
}

// Go
```

Math · LeetCode — By Pattern

— 866 —

LeetCode

```
func missingNumber(arr []int) int {
    n := len(arr)
    d := (arr[n-1] - arr[0]) / n
    for i := 1; i < n; i++ {
        if arr[i] != arr[i-1]+d {
            return arr[i-1] + d
        }
    }
    return arr[0]
}

// [7] Math — Pattern Solution (stored optimal)
Python
# Function to find the missing number in the arithmetic progression
def missingNumber(arr):
    # Calculate the expected difference using the first and last elements
    n = len(arr)
    diff = (arr[n-1] - arr[0]) // n # Integer division because values are integers
    # Iterate over the array to find the place where the difference deviates from diff
    for i in range(1, n-1):
        # If the difference between consecutive numbers is not equal to the expected diff
        if arr[i] - arr[i-1] != diff:
            # Return the missing number which should be current number + diff
            return arr[i] + diff
    # Return -1 if no missing number is found (should not happen given the problem constraints)
    return -1

# Example usage
# Print missingNumber([5,7,11,13]) // Output should be 9
C++
// Include necessary headers
#include <iostream>
using namespace std;

// Function to find the missing number in the arithmetic progression
int missingNumber(vector<int>& arr) {
    int n = arr.size();
    // Calculate the expected difference for the complete progression
    int diff = (arr.back() - arr.front()) / n;
    // Iterate over the array to find where the difference deviates
    for (int i = 0; i < n - 1; i++) {
        if (arr[i+1] - arr[i] != diff) {
            // Return the missing number, which is previous element plus diff
            return arr[i] + diff;
        }
    }
    // Return -1 as a fallback (should not happen based on the problem statement)
    return -1;
}

// Example usage
// int main() {
//     vector<int> arr = {5, 7, 11, 13};
//     int result = missingNumber(arr);
// }
```

Math · LeetCode — By Pattern

— 867 —

LeetCode

```
// // Expected output: 9
// // return 9;
// }
```

#1427 EASY

Perform String Shifts

LeetCode · SimplifyC++ · NumACCC · LeetCode · Editorial

Array · Math · String

List Premium 98

Problem:

You are given a string `s` containing lowercase English letters, and a matrix `shift`, where `shift[i] = [directioni, amounti]`:

- `directioni` can be 0 (for left shift) or 1 (for right shift).
- `amounti` is the amount by which string `s` is to be shifted.

A left shift by 1 means remove the first character of `s` and append it to the end.

Similarly, a right shift by 1 means remove the last character of `s` and add it to the beginning.

Return the final string after all operations.

Example 1:

Input: `s = "abc"`, `shift = [[0,1],[1,2]]`
Output: `"cab"`
Explanation:
[0,1] means shift to left by 1. "abc" -> "bca"
[1,2] means shift to right by 2. "bca" -> "cab"

Example 2:

Input: `s = "abcdefg"`, `shift = [[1,1],[1,1],[0,2],[1,3]]`
Output: `"efgabcd"`
Explanation:
[1,1] means shift to right by 1. "abcdefg" -> "gabcdef"
[1,1] means shift to right by 1. "gabcdef" -> "fgabcde"
[0,2] means shift to left by 2. "fgabcde" -> "abcedfg"
[1,3] means shift to right by 3. "abcedfg" -> "efgabcd"

Constraints:

- `1 <= s.length <= 100`
- `s` only contains lower case English letters.
- `1 <= shift.length <= 100`
- `shift[i].length == 2`
- `directioni` is either 0 or 1.
- `0 <= amounti <= 100`

>> Brute Force (Math) Interview Prep

Python

Math · LeetCode — By Pattern

— 868 —

LeetCode

```
# Brute Force Approach
class Solution:
    def stringShift(self, s: str, shift: List[List[int]]) -> str:
        # Brute Force - Iterate all candidates
        for i in range(1, n + 1):
            is_valid = all(i % j != 0 for j in range(2, i))
            if is_valid: pass
            return s

// Python
Python
# Python solution
def stringShift(s, shift):
    # Initialize the net shift counter
    net_shift = 0
    for direction, amount in shift:
        if direction == 0:
            net_shift -= amount # Left shift reduces index
        else:
            net_shift += amount # Right shift increases index
    # Normalize shift to be within bounds of the string length
    net_shift %= len(s)
    # A right shift can be simulated by taking the last net_shift characters
    # and concatenating them with the beginning of the string.
    return s[-net_shift:] + s[:-net_shift]

# Example usage
if __name__ == "__main__":
    s = "abcdefg"
    shift_operations = [[1,1],[1,1],[0,2],[1,3]]
    print(stringShift(s, shift_operations)) # Output: "efgabcd"

Python
class Solution:
    def stringShift(self, s: str, shift: List[List[int]]) -> str:
        move = 0
        for direction, amount in shift:
            if direction == 0:
                move -= amount
            else:
                move += amount
        move %= len(s)
        return s[-move:] + s[:-move]

Python
class Solution:
    def stringShift(self, s: str, shift: List[List[int]]) -> str:
        x = sum(i if a else -b) for a, b in shift)
        s = s[-move:] + s[:-move]
        return s[-move:] + s[:-move]
```

Math · LeetCode — By Pattern

— 869 —

LeetCode

```
Python
class Solution:
    def stringShift(self, s: str, shift: List[List[int]]) -> str:
        x = sum(i if a else -b) for a, b in shift)
        s = s[-move:] + s[:-move]
        return s[-move:] + s[:-move]

C++
class Solution {
public:
    string stringShift(string s, vector<vector<int>>& shift) {
        const int n = s.length();
        int move = 0;
        for (const vector<int>& pair : shift) {
            const int direction = pair[0];
            const int amount = pair[1];
            if (direction == 0)
                move -= amount;
            else
                move += amount;
        }
        move = (move % n + n) % n;
        return s.substr(0, n - move) + s.substr(n - move, move);
    }
};

C++
class Solution {
public:
    string stringShift(string s, vector<vector<int>>& shift) {
        int n = s.length();
        for (const auto & shift) {
            if (shift[0] == 0) {
                n -= shift[1];
            }
            else {
                n += shift[1];
            }
        }
        int s = s.size();
        s = (s % n + n) % n;
        return s.substr(0, n - s) + s.substr(n - s, s);
    }
};

C++
```

Math · LeetCode — By Pattern

— 870 —

LeetCode

```

class Solution {
public:
    string stringShift(string s, vector<vector<int>>& shifts) {
        int x = 0;
        for (auto& a : shifts) {
            if (a[0] == 0) {
                a[1] = -a[1];
            }
            x += a[1];
        }
        int n = s.size();
        x = (x % n + n) % n;
        return s.substr(-x, x) + s.substr(0, n - x);
    }
};

```

Java

```

class Solution {
    public String stringShift(String s, int[][] shifts) {
        final int n = s.length();
        int move = 0;

        for (int[] pair : shifts) {
            final int direction = pair[0];
            final int amount = pair[1];
            if (direction == 0)
                move += amount;
            else
                move -= amount;
        }

        move = (move % n + n) % n;
        return s.substring(n - move) + s.substring(0, n - move);
    }
}

```

Java

```

class Solution {
    public String stringShift(String s, int[][] shifts) {
        int x = 0;
        for (var a : shifts) {
            if (a[0] == 0) {
                a[1] *= -1;
            }
            x += a[1];
        }
        int n = s.length();
        x = (x % n + n) % n;
        return s.substring(-x) + s.substring(0, n - x);
    }
}

```

Java

Math · LeetCode · By Pattern

— 871 —

LeetCode

```

class Solution {
    public String stringShift(String s, int[][] shifts) {
        int x = 0;
        for (var a : shifts) {
            if (a[0] == 0) {
                a[1] *= -1;
            }
            x += a[1];
        }
        int n = s.length();
        x = (x % n + n) % n;
        return s.substring(n - x) + s.substring(0, n - x);
    }
}

```

TypeScript

```

function stringShift(s: string, shifts: number[][]): string {
    let x = 0;
    for (const [a, b] of shifts) {
        x += a === 0 ? -b : b;
    }
    x %= s.length;
    if (x < 0) {
        x += s.length;
    }
    return s.slice(-x) + s.slice(0, -x);
}

```

TypeScript

```

function stringShift(s: string, shifts: number[][]): string {
    let x = 0;
    for (const [a, b] of shifts) {
        x += a === 0 ? -b : b;
    }
    x %= s.length;
    if (x < 0) {
        x += s.length;
    }
    return s.slice(-x) + s.slice(0, -x);
}

```

Go

Go

Math · LeetCode · By Pattern

— 872 —

LeetCode

```

func stringShift(s string, shifts [][]int) string {
    x := 0
    for _, a := range shifts {
        if a[0] == 0 {
            a[1] = -a[1]
        }
        x += a[1]
    }
    n := len(s)
    x = (x+n) % n
    return s[n-x:] + s[:n-x]
}

```

Go

```

func stringShift(s string, shifts [][]int) string {
    x := 0
    for _, a := range shifts {
        if a[0] == 0 {
            a[1] = -a[1]
        }
        x += a[1]
    }
    n := len(s)
    x = (x+n) % n
    return s[n-x:] + s[:n-x]
}

```

[*] Math · Pattern Solution (stored optimal)

Python

```

# Python solution
def stringShift(s, shifts):
    # Initialize the net shift counter
    net_shift = 0
    for direction, amount in shifts:
        if direction == 0:
            net_shift -= amount # Left shift reduces index
        else:
            net_shift += amount # Right shift increases index

    # Normalize shift to be within bounds of the string length
    net_shift %= len(s)

    # A right shift can be simulated by taking the last net_shift characters
    # and concatenating them with the beginning of the string.
    return s[-net_shift:] + s[:-net_shift]

# Example usage
if __name__ == "__main__":
    s = "abcdefg"
    shift_operations = [[1,1],[1,2],[0,2],[1,3]]
    print(stringShift(s, shift_operations)) # Output: "efgabcd"

```

C++

Math · LeetCode · By Pattern

— 873 —

LeetCode

```

// C++ solution
#include <iostream>
#include <vector>
using namespace std;

string stringShift(string s, vector<vector<int>>& shifts) {
    int x = 0;
    // Compute net shift
    for(auto op : shifts) {
        int direction = op[0], amount = op[1];
        if (direction == 0)
            netShift -= amount; // Left shift reduces index
        else
            netShift += amount; // Right shift increases index
    }

    // Normalize the shift to be within string bounds
    netShift = (netShift % s.size() + s.size()) % s.size();

    // Construct shifted string using substring operations
    return s.substr(-netShift) + s.substr(0, s.size() - netShift);
}

int main() {
    string s = "abcdefg";
    vector<vector<int>> shifts = {{1,1},{1,2},{0,2},{1,3}};
    cout << stringShift(s, shifts) << endl; // Output: "efgabcd"
    return 0;
}

```

#7 MEDIUM

Reverse Integer

LeetCode · Simplest · [WuJACE](#) · [LeetCode](#) · [Editorial](#)

Math

List Grind 189

Problem:

Given a signed 32-bit integer x, return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range [−2³¹, 2³¹ − 1], then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: x = 123

Output: 321

Example 2:

Input: x = -123

Output: -321

Example 3:

Math · LeetCode · By Pattern

— 874 —

LeetCode

```

class Solution {
    def reverse(self, x: int) -> int:
        ans = 0
        while x:
            if ans <= 2147483647 // 10 or ans >= -2147483648 // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y = -y
            ans = ans * 10 + y
            x = (x - y) // 10
        return ans

```

Python

```

class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        while x:
            if ans <= 2147483647 // 10 or ans >= -2147483648 // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y = -y
            ans = ans * 10 + y
            x = (x - y) // 10
        return ans

```

C++

C++

```

class Solution {
//
// Given int x,
// return int
//
    function reverse(x) {
        let negative = x < 0;
        x = abs(x);

        reversed = 0;

        while (x > 0) {
            reversed = reversed * 10 + (x % 10);
            x = Math.floor(x / 10);
        }

        if (isNegative) {
            reversed *= -1;
        }

        if (reversed < -pow(2, 31) || reversed > pow(2, 31) - 1) {
            return 0;
        }

        return reversed;
    }
}

```

Math · LeetCode · By Pattern

— 876 —

LeetCode

Input: x = 120

Output: 21

Constraints:

−2³¹ ≤ x ≤ 2³¹ − 1

Python

```

# Solution for reverse of Integer
def reverse(x: int) -> int:
    # Define the maximum and minimum limits for a 32-bit signed integer
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31

    reversed_num = 0 # This will store the reversed number

    # Process each digit until x becomes 0
    while x != 0:
        # Pop the last digit from x
        # Use modulo 10 to get the last digit
        # For negative numbers, modulo may yield negative remainder
        pop = x % 10 if x >= 0 else x % -10
        # Remove the last digit from x by integer division
        x = x // 10

        # Check if appending the digit would cause overflow
        if reversed_num > INT_MAX // 10 or (reversed_num == INT_MAX // 10 and pop > 7):
            return 0
        if reversed_num < INT_MIN // 10 or (reversed_num == INT_MIN // 10 and pop < -8):
            return 0

        # Append the digit to the reversed number
        reversed_num = reversed_num * 10 + pop

    return reversed_num

# Example usage
print(reverse(123)) # Output: 321
print(reverse(-123)) # Output: -321
print(reverse(120)) # Output: 21

```

Python

```

class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        sign = -1 if x < 0 else 1
        x = sign * x

        while x:
            ans = ans * 10 + x % 10
            x //= 10

        return 0 if ans > 2**31 or ans < -2**31 else sign * ans

```

Python

Math · LeetCode · By Pattern

— 875 —

LeetCode

```

    }
}

C++
class Solution {
//
// Reverses int &x
// Returns int
//
function reverse(x) {
    if(Negative &x < 0;
        &x = abs(&x);

        &reversed = 0;

        while (&x > 0) {
            &reversed = &reversed * 10 + (&x % 10);
            &x = (&x) / 10;
        }

        if (&isNegative) {
            &reversed = -&reversed;
        }

        if (&reversed < -pow(2, 31) || &reversed > pow(2, 31) - 1) {
            return 0;
        }

        return &reversed;
    }
}

Java
class Solution {
    public int reverse(int x) {
        long ans = 0;

        while (x != 0) {
            ans = ans * 10 + x % 10;
            x /= 10;
        }

        return (ans < Integer.MIN_VALUE || ans > Integer.MAX_VALUE ? 0 : (int) ans);
    }
}

Java

```

```

class Solution {
    public int reverse(int x) {
        int ans = 0;
        for (; x != 0; x /= 10) {
            if (ans > Integer.MAX_VALUE / 10 || ans < Integer.MIN_VALUE / 10) {
                return 0;
            }
            ans = ans * 10 + x % 10;
        }
        return ans;
    }
}

Java
public class Solution {
    public int Reverse(int x) {
        int ans = 0;
        for (; x != 0; x /= 10) {
            if (ans > Integer.MAX_VALUE / 10 || ans < Integer.MIN_VALUE / 10) {
                return 0;
            }
            ans = ans * 10 + x % 10;
        }
        return ans;
    }
}

Java
class Solution {
    public int reverse(int x) {
        int ans = 0;
        for (; x != 0; x /= 10) {
            if (ans > Integer.MAX_VALUE / 10 || ans < Integer.MIN_VALUE / 10) {
                return 0;
            }
            ans = ans * 10 + x % 10;
        }
        return ans;
    }
}

Java
public class Solution {
    public int Reverse(int x) {
        int ans = 0;
        for (; x != 0; x /= 10) {
            if (ans > Integer.MAX_VALUE / 10 || ans < Integer.MIN_VALUE / 10) {
                return 0;
            }
            ans = ans * 10 + x % 10;
        }
        return ans;
    }
}

JavaScript

```

```

JavaScript
//
// Reverses (number) x
// Returns (number)
//
var reverse = function(x) {
    const n1 = -12 == 31;
    const n2 = 2 == 31 - 1;
    let ans = 0;
    for (; x != 0; x = -(x % 10)) {
        if (ans > -(n1 / 10) || ans < -(n2 / 10)) {
            return 0;
        }
        ans = ans * 10 + (x % 10);
    }
    return ans;
};

JavaScript
//
// Reverses (number) x
// Returns (number)
//
var reverse = function(x) {
    const n1 = -12 == 31;
    const n2 = 2 == 31 - 1;
    let ans = 0;
    for (; x != 0; x = -(x % 10)) {
        if (ans > -(n1 / 10) || ans < -(n2 / 10)) {
            return 0;
        }
        ans = ans * 10 + (x % 10);
    }
    return ans;
};

Rust
impl Solution {
    pub fn reverse(x: i32) -> i32 {
        let is_minus = x < 0;
        match x {
            -abs(x)
            .to_string()
            .chars()
            .rev()
            .collect::<String>()
            .parse::<i32>()
        {
            Ok(x) => x * (if is_minus { -1 } else { 1 });
            Err(_) => 0;
        }
    }
}

Rust

```

```

impl Solution {
    pub fn reverse(x: i32) -> i32 {
        let is_minus = x < 0;
        match x.abs().to_string().chars().rev().collect::<String>().parse::<i32>() {
            Ok(x) => x * (if is_minus { -1 } else { 1 });
            Err(_) => 0;
        }
    }
}

Python
""" Math — Pattern Solution (stored optimal) """

# Solution to reverse an integer
def reverse(x: int) -> int:
    # Define the maximum and minimum limits for a 32-bit signed integer
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31

    reversed_num = 0 # This will store the reversed number

    # Process each digit until it becomes 0
    while x != 0:
        # Pop the last digit from x
        # Use modulo 10 to get the last digit
        # For negative numbers, modulo may yield negative remainder
        pop = x % 10 if x > 0 else x % -10
        # Remove the last digit from x by integer division
        x = x // 10

        # Check if appending the digit would cause overflow
        if reversed_num > INT_MAX // 10 or (reversed_num == INT_MAX // 10 and pop > 7):
            return 0
        if reversed_num < INT_MIN // 10 or (reversed_num == INT_MIN // 10 and pop < -8):
            return 0

        # Append the digit to the reversed number
        reversed_num = reversed_num * 10 + pop

    return reversed_num

# Example usage
print(reverse(123)) # Output: 321
print(reverse(-321)) # Output: -123
print(reverse(0)) # Output: 0

C++

```

```

// Function to reverse an integer
#include <iostream>
using namespace std;

int reverse(int x) {
    int reversedNum = 0; // This will hold the reversed number

    // Process each digit until it becomes 0
    while (x != 0) {
        int pop = x % 10; // Get the last digit
        x /= 10; // Remove the last digit from x

        // Check for overflow: if reversedNum will go beyond INT_MAX or INT_MIN
        if (reversedNum > INT_MAX/10 || (reversedNum == INT_MAX/10 && pop > 7)) return 0;
        if (reversedNum < INT_MIN/10 || (reversedNum == INT_MIN/10 && pop < -8)) return 0;

        // Append the digit to reversedNum
        reversedNum = reversedNum * 10 + pop;
    }

    return reversedNum;
}

int main() {
    cout << reverse(123) << endl; // Output: 321
    cout << reverse(-321) << endl; // Output: -123
    cout << reverse(0) << endl; // Output: 0
    return 0;
}

#50 MEDIUM
Pow(x, n)
LeetCode · Simple · WalkCC · LeetCode · Editorial
Math · Recursion
List: Grid 169
Problem:
Implement pow(x, n), which calculates x raised to the power n (i.e., x^n).
Example 1:
Input: x = 2.00000, n = 10
Output: 1024.00000
Example 2:
Input: x = 2.10000, n = 3
Output: 9.26100
Example 3:
Input: x = 2.00000, n = -2
Output: 0.25000
Explanation: 2^-2 = 1/2^2 = 1/4 = 0.25

```

```

Constraints:
• -100.0 < x < 100.0
• -231 <= n <= 231-1
• n is an integer.
• Either x is not zero or n > 0.
• -104 <= xn <= 104

>> Brute Force [Math] Interview Prep

Python
# Brute Force Approach
class Solution:
    def myPow(self, x: float, n: int) -> float:
        # Base Cases
        if n == 0:
            return 1.0
        if n < 0:
            return 1 / self.myPow(x, -n)
        # Recursive Case
        result = 1.0
        while n:
            if n % 2 == 1:
                result *= x
            x *= x
            n //= 2
        return result

# Example usage
print(myPow(2.0, 10)) # Output: 1024.0
print(myPow(2.1, 3)) # Output: 9.261000000000001
print(myPow(2.0, -2)) # Output: 0.25

Python
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n == 0:
            return 1
        if n < 0:
            return 1 / self.myPow(x, -n)
        if n % 2 == 1:
            return x * self.myPow(x, n - 1)
        return self.myPow(x * x, n // 2)

Python

```

```

class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpow(x: float, a: int) -> float:
            ans = 1
            while a:
                if a & 1:
                    ans *= x
                a //= 2
            return ans
        return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)

```

```

Python
class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpow(x: float, a: int) -> float:
            ans = 1
            while a:
                if a & 1:
                    ans *= x
                a //= 2
            return ans
        return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)

```

Java

```

Java
class Solution {
    public double myPow(double x, long n) {
        if (n == 0)
            return 1;
        if (n < 0)
            return 1 / myPow(x, -n);
        if (n % 2 == 1)
            return x * myPow(x, n - 1);
        return myPow(x, n / 2);
    }
}

```

Java

```

class Solution {
    public double myPow(double x, int n) {
        return n >= 0 ? qpow(x, n) : 1 / qpow(x, -(long) n);
    }

    private double qpow(double x, long n) {
        double ans = 1;
        for (; n > 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    public double myPow(double x, int n) {
        return n >= 0 ? qpow(x, n) : 1.0 / qpow(x, -(long)n);
    }

    private double qpow(double x, long n) {
        double ans = 1;
        for (; n > 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public double myPow(double x, int n) {
        return n >= 0 ? qpow(x, n) : 1 / qpow(x, -(long) n);
    }

    private double qpow(double x, long n) {
        double ans = 1;
        for (; n > 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    public double myPow(double x, int n) {
        return n >= 0 ? qpow(x, n) : 1.0 / qpow(x, -(long)n);
    }

    private double qpow(double x, long n) {
        double ans = 1;
        for (; n > 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    }
}

```

JavaScript

JavaScript

```

//
// @param {number} x
// @param {number} n
// @return {number}
//
var myPow = function(x, n) {
    const qpow = (x, n) => {
        let ans = 1;
        for (; n >= 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    };
    return n >= 0 ? qpow(x, n) : 1 / qpow(x, -n);
};

```

JavaScript

```

//
// @param {number} x
// @param {number} n
// @return {number}
//
var myPow = function(x, n) {
    const qpow = (x, n) => {
        let ans = 1;
        for (; n >= 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    };
    return n >= 0 ? qpow(x, n) : 1 / qpow(x, -n);
};

```

```

TypeScript
TypeScript
function myPow(x: number, n: number): number {
    const qpow = (x: number, n: number): number => {
        let ans = 1;
        for (; n >= 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    };
    return n >= 0 ? qpow(x, n) : 1 / qpow(x, -n);
}

```

```

TypeScript
function myPow(x: number, n: number): number {
    const qpow = (x: number, n: number): number => {
        let ans = 1;
        for (; n >= 0; n /= 2) {
            if (n % 2 == 1) {
                ans *= x;
            }
            x *= x;
        }
        return ans;
    };
    return n >= 0 ? qpow(x, n) : 1 / qpow(x, -n);
}

```

Rust

Rust

```

impl Solution {
    #[allow(dead_code)]
    pub fn my_pow(x: f64, n: i32) -> f64 {
        let mut x = x;
        let mut n = n as i64;
        if n == 0 {
            return 1.0;
        } else {
            1.0 / Self::quick_pow(x, -n)
        }
    }

    #[allow(dead_code)]
    fn quick_pow(x: f64, mut n: i64) -> f64 {
        // 'x' should greater or equal to zero
        let mut ret = 1.0;
        while n > 0 {
            if (n & 1) == 1 {
                ret *= x;
            }
            x *= x;
            n /= 2;
        }
        ret
    }
}

```

Rust

```

impl Solution {
    #[allow(dead_code)]
    pub fn my_pow(x: f64, n: i32) -> f64 {
        let mut x = x;
        let mut n = n as i64;
        if n == 0 {
            return 1.0;
        } else {
            1.0 / Self::quick_pow(x, -n)
        }
    }

    #[allow(dead_code)]
    fn quick_pow(x: f64, mut n: i64) -> f64 {
        // 'x' should greater or equal to zero
        let mut ret = 1.0;
        while n > 0 {
            if (n & 1) == 1 {
                ret *= x;
            }
            x *= x;
            n /= 2;
        }
        ret
    }
}

```

[*] Math — Pattern Solution (matched from community answers)

Python

```

# Python implementation of pow(x, n) using exponentiation by squaring
def myPow(x: float, n: int) -> float:
    # If exponent is negative, invert x and make n positive
    if n < 0:
        x = 1 / x
        n = -n

    result = 1.0 # Initialize result
    while n:
        # If the current exponent is odd, multiply result by the base
        if n % 2:
            result *= x
        x *= x # Square the base for the next iteration
        n //= 2 # Divide the exponent by 2
    return result

# Example usage:
print(myPow(2.0, 10)) # Output: 1024.0
print(myPow(2.0, 20)) # Output: 1048576.0
print(myPow(2.0, -20)) # Output: 9.53674316e-07

```

#380

MEDIUM

Insert Delete GetRandom O(1)

LeetCode · Companies · SQL · LeetCode · Editorial

Array · Hash Table · Math · Design

List: Grind 169

Problem:

Implement the RandomizedSet class:

- RandomizedSet() Initializes the RandomizedSet object.
- bool insert(int val) Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- bool remove(int val) Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- int getRandom() Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the same probability of being returned.

You must implement the functions of the class such that each function works in average O(1) time complexity.

Example 1:

Input

```

["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove", "insert", "getRandom"]
[[], [1], [2], [2], [1], [1], [2], [1]]

```

Output

```

[null, true, false, true, 2, true, false, 2]

```


- $-231 \leq \text{val} \leq 231 - 1$
- At most $2 \cdot 10^5$ calls will be made to insert, remove, and getRandom.
- There will be at least one element in the data structure when getRandom is called.

```

class RandomIndexSet:
    def __init__(self, n):
        self.vals = []
        self.valsIndex = collections.defaultdict(list) # (val, index in vals)

    def insert(self, val, idx) -> bool:
        if val in self.valsIndex:
            return False
        self.valsIndex[val] = list(self.vals)
        self.vals.append(val)
        return True

    def remove(self, val, idx) -> bool:
        if val not in self.valsIndex:
            return False
        index = self.valsIndex[val]
        # The order of the following two lines is important when vals.size() == 1.
        self.valsIndex[self.vals[-1]] = index
        del self.valsIndex[val]
        self.vals[index] = self.vals[-1]
        self.vals.pop()
        return True

    def getRandom(self) -> int:
        index = random.randint(0, len(self.vals) - 1)
        return self.vals[index]

```

Math · LeetCode · By Pattern — 892 — LeetCode

```
class RandomizedSet {
public:
    RandomizedSet() {}

    bool insert(int val) {
        if (d.contains(val)) {
            return false;
        }
        d[val] = q.size();
        q.push_back(val);
        return true;
    }

    bool remove(int val) {
        if (d.contains(val)) {
            return false;
        }
        int i = d[val];
        d[q.back()] = i;
        q[i] = q.back();
        q.pop_back();
        d.erase(val);
        return true;
    }

    int getRandom() {
        return q[rand() % q.size()];
    }
private:
    unordered_map<int, int> d;
    vector<int> q;
};

/**
 * Your RandomizedSet object will be instantiated and called as such:
 * RandomizedSet obj = new RandomizedSet();
 * bool param_1 = obj.insert(val);
 * bool param_2 = obj.remove(val);
 * int param_3 = obj.getRandom();
 */
}

Java
Java
```

```
class RandomizedSet {
public:
    bool insert(int val) {
        if (valToIndex.contains(val)) {
            return false;
        }
        valToIndex.put(val, val.size());
        val.add(val);
        return true;
    }

    public boolean remove(int val) {
        if (valToIndex.containsKey(val)) {
            return false;
        }
        final int index = valToIndex.get(val);
        // The value of the following two lines is important when val.size() == 1.
        valToIndex.put(index, index);
        valToIndex.remove(val);
        val.set(index, lastElement);
        val.remove(val.size() - 1);
        return true;
    }

    public int getRandom() {
        final int index = rand.nextInt(val.size());
        return val.get(index);
    }

    // [val, index in val]
    private Map<Integer, Integer> valToIndex = new HashMap<>();
    private List<Integer> val = new ArrayList<>();
    private Random rand = new Random();

    private int last(List<Integer> val) {
        return val.get(val.size() - 1);
    }
}

Java
```

```
class RandomizedSet {
private Map<Integer, Integer> d = new HashMap<>();
private List<Integer> q = new ArrayList<>();
private Random rand = new Random();

public RandomizedSet() {}

public boolean insert(int val) {
    if (d.containsKey(val)) {
        return false;
    }
    d.put(val, q.size());
    q.add(val);
    return true;
}

public boolean remove(int val) {
    if (!d.containsKey(val)) {
        return false;
    }
    int i = d.get(val);
    d.put(q.get(q.size() - 1), i);
    q.set(i, q.get(q.size() - 1));
    q.remove(q.size() - 1);

    d.remove(val);
    return true;
}

public int getRandom() {
    return q.get(rand.nextInt(q.size()));
}
}

/**
 * Your RandomizedSet object will be instantiated and called as such:
 * RandomizedSet obj = new RandomizedSet();
 * boolean param_1 = obj.insert(val);
 * boolean param_2 = obj.remove(val);
 * int param_3 = obj.getRandom();
 */
}

Java
```

```
use rand:Rng;
use std::collections::HashMap;

struct RandomizedSet {
    list: HashMap<i32>,
}

/**
 * Self means the method takes an unstable reference.
 * If you need a mutable reference, change it to &mut self instead.
 */
impl RandomizedSet {
    fn new() -> Self {
        Self {
            list: HashMap::new(),
        }
    }

    fn insert(&mut self, val: i32) -> bool {
        !self.list.contains(&val)
    }

    fn remove(&mut self, val: i32) -> bool {
        self.list.remove(&val)
    }

    fn get_random(&self) -> i32 {
        let i = rand::thread_rng().gen_range(0, self.list.len());
        *self.list.iter().collect::
```

```
public class RandomizedSet {
private Dictionary<int, int> d = new Dictionary<int, int>();
private List<int> q = new List<int>();

public RandomizedSet() {}

public bool Insert(int val) {
    if (d.ContainsKey(val)) {
        return false;
    }
    d.Add(val, q.Count);
    q.Add(val);
    return true;
}

public bool Remove(int val) {
    if (!d.ContainsKey(val)) {
        return false;
    }
    int i = d[val];
    d[q[q.Count - 1]] = i;
    q[i] = q[q.Count - 1];
    q.RemoveAt(q.Count - 1);

    d.Remove(val);
    return true;
}

public int GetRandom() {
    return q[(int) Random.Next(0, q.Count)];
}
}

/**
 * Your RandomizedSet object will be instantiated and called as such:
 * RandomizedSet obj = new RandomizedSet();
 * bool param_1 = obj.Insert(val);
 * bool param_2 = obj.Remove(val);
 * int param_3 = obj.GetRandom();
 */
}

Java
```

```
class RandomizedSet {
private Map<Integer, Integer> d = new HashMap<>();
private List<Integer> q = new ArrayList<>();
private Random rand = new Random();

public RandomizedSet() {}

public boolean insert(int val) {
    if (d.containsKey(val)) {
        return false;
    }
    d.put(val, q.size());
    q.add(val);
    return true;
}

public boolean remove(int val) {
    if (!d.containsKey(val)) {
        return false;
    }
    int i = d.get(val);
    d.put(q.get(q.size() - 1), i);
    q.set(i, q.get(q.size() - 1));
    q.remove(q.size() - 1);

    d.remove(val);
    return true;
}

public int getRandom() {
    return q.get(rand.nextInt(q.size()));
}
}

/**
 * Your RandomizedSet object will be instantiated and called as such:
 * RandomizedSet obj = new RandomizedSet();
 * boolean param_1 = obj.insert(val);
 * boolean param_2 = obj.remove(val);
 * int param_3 = obj.getRandom();
 */
}

//////////////////////////////////////////////////
public class Insert_Delete_GetRandom_0_1 {

    public static void main(String[] args) {
        Insert_Delete_GetRandom_0_1 out = new Insert_Delete_GetRandom_0_1();
    }
}
```

```

// this is empty set
RandomizedSet randomized = new RandomizedSet();

System.out.println(randomSet.remove(0));
System.out.println(randomSet.remove(0));
System.out.println(randomSet.insert(0));
System.out.println(randomSet.insert(0));
System.out.println(randomSet.insert(0));

System.out.println(randomSet.getRandom());

System.out.println(randomSet.remove(0));
System.out.println(randomSet.insert(0));

```

```

class RandomizedSet {
    List<Integer> list;
    Map<Integer, Integer> map; // val => its index in list
    Random rand;

    // we initialize your data structure here.
    public RandomizedSet() {
        list = new ArrayList();
    }

    // we use HashSet<Integer>
    random = new Random();

    // Inserts a value to the set. Returns true if the set did not already contain the specified
    // element.
}

```

Java

```

public class RandomizedSet {
    private Dictionary<int, int> d = new Dictionary<int, int>();
    private List<int> q = new List<int>();

    public RandomizedSet() {
    }

    public bool Insert(int val) {
        if (d.ContainsKey(val)) {
            return false;
        }
        d.Add(val, q.Count);
        q.Add(val);
        return true;
    }

    public bool Remove(int val) {
        if (!d.ContainsKey(val)) {
            return false;
        }
        int i = d[val];
        d[q[q.Count - 1]] = i;
        q[i] = q[q.Count - 1];
        d.Remove(i, Count - 1);

        d.Remove(val);
        return true;
    }

    public int GetRandom() {
        return q[low Random().Next(0, q.Count)];
    }
}

```

```

// =
// Your RandomizedSet object will be instantiated and called as such:
// RandomizedSet obj = new RandomizedSet();
// bool param_1 = obj.Insert(val);
// bool param_2 = obj.Remove(val);
// int param_3 = obj.GetRandom();
//

```

Java

```

use std::collections::HashSet;
use rand::Rng;

struct RandomizedSet {
    list: HashSet<i32>,
}

// =
// 'self' means the method takes an immutable reference.
// If you need a mutable reference, change it to 'mut self' instead.
//
impl RandomizedSet {
    fn new() -> Self {
        Self {
            list: HashSet::new(),
        }
    }

    fn insert(&mut self, val: i32) -> bool {
        self.list.insert(val)
    }

    fn remove(&mut self, val: i32) -> bool {
        self.list.remove(val)
    }

    fn get_random(&self) -> i32 {
        let i = rand::thread_rng().gen_range(0, self.list.len());
        *self.list.iter().collect::

```

TypeScript

TypeScript

```

class RandomizedSet {
    private d: Map<number, number> = new Map();
    private q: number[] = [];

    constructor() {}

    insert(val: number): boolean {
        if (this.d.has(val)) {
            return false;
        }
        this.d.set(val, this.q.length);
        this.q.push(val);
        return true;
    }

    remove(val: number): boolean {
        if (!this.d.has(val)) {
            return false;
        }
        const i = this.d.get(val);
        this.d.set(this.q[this.q.length - 1], i);
        this.q[i] = this.q[this.q.length - 1];
        this.q.pop();
        this.d.delete(val);
        return true;
    }

    getRandom(): number {
        return this.q[Math.floor(Math.random() * this.q.length)];
    }
}
// =
// Your RandomizedSet object will be instantiated and called as such:
// var obj = new RandomizedSet();
// var param_1 = obj.insert(val);
// var param_2 = obj.remove(val);
// var param_3 = obj.getRandom();
//

```

TypeScript

```

class RandomizedSet {
    private d: Map<number, number> = new Map();
    private q: number[] = [];

    constructor() {}

    insert(val: number): boolean {
        if (this.d.has(val)) {
            return false;
        }
        this.d.set(val, this.q.length);
        this.q.push(val);
        return true;
    }

    remove(val: number): boolean {
        if (!this.d.has(val)) {
            return false;
        }
        const i = this.d.get(val);
        this.d.set(this.q[this.q.length - 1], i);
        this.q[i] = this.q[this.q.length - 1];
        this.q.pop();
        this.d.delete(val);
        return true;
    }

    getRandom(): number {
        return this.q[Math.floor(Math.random() * this.q.length)];
    }
}
// =
// Your RandomizedSet object will be instantiated and called as such:
// var obj = new RandomizedSet();
// var param_1 = obj.insert(val);
// var param_2 = obj.remove(val);
// var param_3 = obj.getRandom();
//

```

Go

Go

```

type RandomizedSet struct {
    d map[int]int
    q []int
}

func Constructor() RandomizedSet {
    return RandomizedSet{map[int]int{}, []int{}}
}

func (this *RandomizedSet) Insert(val int) bool {
    if _, ok := this.d[val]; ok {
        return false
    }
    this.d[val] = len(this.q)
    this.q = append(this.q, val)
    return true
}

func (this *RandomizedSet) Remove(val int) bool {
    if _, ok := this.d[val]; !ok {
        return false
    }
    i := this.d[val]
    this.d[this.q[len(this.q)-1]] = i
    this.q[i] = this.q[len(this.q)-1]
    this.q = this.q[:len(this.q)-1]
    delete(this.d, val)
    return true
}

func (this *RandomizedSet) GetRandom() int {
    return this.q[rand.Intn(len(this.q))]
}

```

```

// =
// Your RandomizedSet object will be instantiated and called as such:
// obj := Constructor();
// param_1 := obj.Insert(val);
// param_2 := obj.Remove(val);
// param_3 := obj.GetRandom();
//

```

[*] Math — Pattern Solution (stored optimal)

Python

```

import random

class RandomizedSet():
    def __init__(self):
        # List to store the values
        self.values = []
        # Dictionary to map a value to its index in the list
        self.val_to_index = {}

    def insert(self, val: int) -> bool:
        # If the value is already present, do not insert.
        if val in self.val_to_index:
            return False
        # Append the value and store its index.
        self.val_to_index[val] = len(self.values)
        self.values.append(val)
        return True

    def remove(self, val: int) -> bool:
        # If the value is not present, return False.
        if val not in self.val_to_index:
            return False
        # Index of element to remove.
        index = self.val_to_index[val]
        # Get the last element
        last_element = self.values[-1]
        # Place the last element at the index of the element to remove.
        self.values[index] = last_element
        self.val_to_index[last_element] = index
        # Remove the last element from the list.
        self.values.pop()
        # Delete the mapping for the removed element.
        del self.val_to_index[val]
        return True

    def getRandom(self) -> int:
        # Return a random element from the list.
        return random.choice(self.values)

```

C++

```

#include <vector>
#include <unordered_map>
#include <cstdlib>
using namespace std;

class RandomizedSet {
    // Vector to store values.
    vector<int> values;
    // Unordered map to store value to index mapping.
    unordered_map<int, int> valToIndex;
public:
    RandomizedSet() {
        // Constructor initializes empty vector and map.
    }

    bool insert(int val) {
        // Check if value already exists.
        if (valToIndex.find(val) != valToIndex.end()) {
            return false;
        }
        // Push value to vector and record its index.
        values.push_back(val);
        valToIndex[val] = values.size() - 1;
        return true;
    }

    bool remove(int val) {
        // If vector is empty, return false.
        if (valToIndex.find(val) == valToIndex.end()) {
            return false;
        }
        // Get index of element to remove.
        int idx = valToIndex[val];
        // Get last element in the vector.
        int lastElement = values.back();
        // Swap the last element with the element at idx.
        values[idx] = lastElement;
        valToIndex[lastElement] = idx;
        // Remove the last element.
        values.pop_back();
        // Erase val from the map.
        valToIndex.erase(val);
        return true;
    }

    int getRandom() {
        // Generate a random index and return the value.
        int randomIndex = rand() % values.size();
        return values[randomIndex];
    }
};

```

#1017 MEDIUM Convert to Base -2

LeetCode · SimplifyInt · WalkCC · LeetCode · Editorial

Math

Liya Cheny Zhang

Problem:

Given an integer n , return a binary string representing its representation in base -2.

Note that the returned string should not have leading zeros unless the string is "0".

Example 1:

Input: $n = 2$
Output: "110"
Explanation: $(-2)2 + (-2)1 = 2$

Example 2:

Input: $n = 3$
Output: "111"
Explanation: $(-2)2 + (-2)1 + (-2)0 = 3$

Example 3:

Input: $n = 4$
Output: "100"
Explanation: $(-2)2 = 4$

Constraints:

$-10 \leq n \leq 100$

Python

```

Python
# Python Solution with Detailed Comments
def baseNeg2(n: int) -> str:
    # Special case for zero as input.
    if n == 0:
        return "0"

    digits = [] # List to store digits in base -2.

    while n != 0:
        n, remainder = divmod(n, -2) # Get quotient and remainder when dividing by -2.
        if remainder == 0:
            # Adjust the remainder and increment the quotient.
            remainder += 2
            n += 1
        # Append the remainder (digit) to the list.
        digits.append(str(remainder))

    # Reverse the digits to form the proper binary string and return.
    return "".join(reversed(digits))

# Example usage:
print(baseNeg2(2)) # Output: "110"
print(baseNeg2(3)) # Output: "111"
print(baseNeg2(4)) # Output: "100"

```

Python

```

class Solution {
    def baseNeg2(self, n: int) -> str:
        ans = ""
        while n != 0:
            ans.append(str(n % 2))
            n = -(n // 2)
        return ''.join(reversed(ans)) if ans else '0'
}

Python
class Solution:
    def baseNeg2(self, n: int) -> str:
        k = 1
        ans = ""
        while n:
            if n % 2:
                ans.append('1')
                n = -(n // 2)
            else:
                ans.append('0')
                n //= 2
            k += 1
        return ''.join(ans[::-1]) or '0'

Python
class Solution:
    def baseNeg2(self, n: int) -> str:
        k = 1
        ans = ""
        while n:
            if n % 2:
                ans.append('1')
                n = -(n // 2)
            else:
                ans.append('0')
                n //= 2
            k += 1
        return ''.join(ans[::-1]) or '0'

Java
class Solution {
    public String baseNeg2(int n) {
        StringBuilder sb = new StringBuilder();

        while (n != 0) {
            sb.append(n % 2);
            n = -(n // 2);
        }

        return sb.length() > 0 ? sb.reverse().toString() : "0";
    }
}

```

Java

```

class Solution {
    public String baseNeg2(int n) {
        if (n == 0) {
            return "0";
        }
        int k = 1;
        StringBuilder ans = new StringBuilder();
        while (n != 0) {
            if (n % 2 != 0) {
                ans.append(1);
                n = -(n // 2);
            } else {
                ans.append(0);
                n = -(n // 2);
            }
            k++;
        }
        return ans.reverse().toString();
    }
}

Java
public class Solution {
    public String baseNeg2(int n) {
        if (n == 0) {
            return "0";
        }
        int k = 1;
        StringBuilder ans = new StringBuilder();
        while (n != 0) {
            if (n % 2 != 0) {
                ans.append(1);
                n = -(n // 2);
            } else {
                ans.append(0);
                n = -(n // 2);
            }
            k++;
        }
        char[] cs = ans.toString().toCharArray();
        Array.Reverse(cs);
        return new String(cs);
    }
}

```

Java

```

class Solution {
    public String baseNeg2(int n) {
        if (n == 0) {
            return "0";
        }
        int k = 1;
        StringBuilder ans = new StringBuilder();
        while (n != 0) {
            if (n % 2 != 0) {
                ans.append(1);
                n = -(n // 2);
            } else {
                ans.append(0);
                n = -(n // 2);
            }
            k++;
        }
        return ans.reverse().toString();
    }
}

Java
public class Solution {
    public String baseNeg2(int n) {
        if (n == 0) {
            return "0";
        }
        int k = 1;
        StringBuilder ans = new StringBuilder();
        while (n != 0) {
            if (n % 2 != 0) {
                ans.append(1);
                n = -(n // 2);
            } else {
                ans.append(0);
                n = -(n // 2);
            }
            k++;
        }
        char[] cs = ans.toString().toCharArray();
        Array.Reverse(cs);
        return new String(cs);
    }
}

TypeScript
TypeScript

```

```
function hasSingle(a: number): string {
  if (a === 0) {
    return '0';
  }
  let k = 1;
  const ans: string[] = [];
  while (a) {
    if (a % 2) {
      ans.push('1');
      a = (a - 1) / 2;
    } else {
      ans.push('0');
      a = a / 2;
    }
  }
  return ans.reverse().join('');
}
```

TypeScript

```
function hasSingle(a: number): string {
  if (a === 0) {
    return '0';
  }
  let k = 1;
  const ans: string[] = [];
  while (a) {
    if (a % 2) {
      ans.push('1');
      a = (a - 1) / 2;
    } else {
      ans.push('0');
      a = a / 2;
    }
  }
  return ans.reverse().join('');
}
```

Rust

Rust

```
impl Solution {
  pub fn hasSingle(a: i32) -> String {
    if a == 0 {
      return "0".to_string();
    }
    let mut k = 1;
    let mut ans = String::new();
    let mut num = a;
    while num != 0 {
      if num % 2 != 0 {
        ans.push('1');
        num = (num - 1) / 2;
      } else {
        ans.push('0');
        num /= 2;
      }
    }
    ans.chars().rev().collect::<String>()
  }
}
```

Rust

```
impl Solution {
  pub fn hasSingle(a: i32) -> String {
    if a == 0 {
      return "0".to_string();
    }
    let mut k = 1;
    let mut ans = String::new();
    let mut num = a;
    while num != 0 {
      if num % 2 != 0 {
        ans.push('1');
        num = (num - 1) / 2;
      } else {
        ans.push('0');
        num /= 2;
      }
    }
    ans.chars().rev().collect::<String>()
  }
}
```

[*] Math — Pattern Solution (matched from community answers)

Java

```
class Solution {
  public String hasSingle(int a) {
    StringBuilder sb = new StringBuilder();
    while (a != 0) {
      sb.append(a % 2);
      a = (a - 1) / 2;
    }
    return sb.length() > 0 ? sb.reverse().toString() : "0";
  }
}
```

#3160

MEDIUM

Find the Number of Distinct Colors Among the Balls

LeetCode · SimplifyLast · WalkCC · LeetCode · Editorial

Array · Hash Table · Simulation

Like · CodeSignal

Problem:

You are given an integer limit and a 2D array queries of size n x 2.

There are limit + 1 balls with distinct labels in the range [0, limit]. Initially, all balls are uncolored.

For every query in queries that is of the form [x, y], you mark ball x with the color y. After each query, you need to find the number of colors among the balls.

Return an array result of length n, where result[i] denotes the number of colors after its query.

Note that when answering a query, lack of a color will not be considered as a color.

Example 1:

Input: limit = 4, queries = [[1,4],[2,5],[1,3],[3,4]]

Output: [1,2,2,3]

Explanation:



- After query 0, ball 1 has color 4.
- After query 1, ball 1 has color 4, and ball 2 has color 5.
- After query 2, ball 1 has color 3, and ball 2 has color 5.
- After query 3, ball 1 has color 3, ball 2 has color 5, and ball 3 has color 4.

Example 2:

Input: limit = 4, queries = [[0,1],[1,2],[2,3],[3,4],[4,5]]

Output: [1,2,2,3,4]

Explanation:



- After query 0, ball 0 has color 1.
- After query 1, ball 0 has color 1, and ball 1 has color 2.
- After query 2, ball 0 has color 1, and balls 1 and 2 have color 2.
- After query 3, ball 0 has color 1, balls 1 and 2 have color 2, and ball 3 has color 4.
- After query 4, ball 0 has color 1, balls 1 and 2 have color 2, ball 3 has color 4, and ball 4 has color 5.

Constraints:

- 1 <= limit <= 109
- 1 <= n = queries.length <= 105
- queries[i].length == 2
- 0 <= queries[i][0] <= limit
- 1 <= queries[i][1] <= 109

Python

Python

```
# Python solution with detailed comments

def distinctColors(limit, queries):
    # Dictionary to keep track of the current color of each ball
    ball_color = {}
    # Dictionary to count the frequency of each color applied
    color_count = {}
    # List to store the results after each query
    results = []

    # Process each query one by one
    for ball, color in queries:
        # If the ball has been colored before, update the old color frequency
        if ball in ball_color:
            old_color = ball_color[ball]
            color_count[old_color] -= 1
            # If the frequency becomes zero, remove the color from the dictionary
            if color_count[old_color] == 0:
                del color_count[old_color]
            # Update ball's color to the new color
            ball_color[ball] = color
            # Increase the frequency count of the new color
            color_count[color] = color_count.get(color, 0) + 1

    # The number of distinct colors is the number of keys in color_count
    results.append(len(color_count))

    return results

# Example usage:
# limit = 4
# queries = [[1,4],[2,5],[1,3],[3,4]]
# print(distinctColors(limit, queries)) # Output: [1,2,2,3]
```

Python

```
class Solution:
  def queryResults(self, limit: int, queries: List[List[int]]) -> List[int]:
    ans = []
    ball_color = {}
    color_count = collections.Counter()

    for ball, color in queries:
      if ball in ball_color:
        prev_color = ball_color[ball]
        color_count[prev_color] -= 1
        if color_count[prev_color] == 0:
          del color_count[prev_color]
        ball_color[ball] = color
        color_count[color] += 1
      ans.append(len(color_count))

    return ans
```

Python

```
class Solution:
  def queryResults(self, limit: int, queries: List[List[int]]) -> List[int]:
    ans = []
    cnt = Counter()
    ans = []
    for x, y in queries:
      cnt[y] += 1
      if x in cnt:
        cnt[x] -= 1
        if cnt[x] == 0:
          del cnt[x]
      ans.append(len(cnt))
    return ans
```

Python

```
class Solution:
  def queryResults(self, limit: int, queries: List[List[int]]) -> List[int]:
    ans = []
    cnt = Counter()
    ans = []
    for x, y in queries:
      cnt[y] += 1
      if x in cnt:
        cnt[x] -= 1
        if cnt[x] == 0:
          del cnt[x]
      ans.append(len(cnt))
    return ans
```

C++

C++

```
class Solution {
public:
  vector<int> queryResults(int limit, vector<vector<int>>& queries) {
    vector<int> ans;
    unordered_map<int, int> ballToColor;
    unordered_map<int, int> colorCount;

    for (const vector<int>& query : queries) {
      const int ball = query[0];
      const int color = query[1];
      if (const auto it = ballToColor.find(ball); it != ballToColor.cend()) {
        const int prevColor = it->second;
        if (--colorCount[prevColor] == 0)
          colorCount.erase(prevColor);
        ballToColor[ball] = color;
        ++colorCount[color];
        ans.push_back(colorCount.size());
      }
    }
    return ans;
  }
};
```

```

C++
class Solution {
public:
    vector<int> queryResult(int limit, vector<vector<int>>& queries) {
        unordered_map<int, int> g;
        unordered_map<int, int> cnt;
        vector<int> ans;
        for (auto& q : queries) {
            int x = q[0], y = q[1];
            cnt[y]++;
            if (g.contains(x) && --cnt[g[x]] == 0) {
                cnt.erase(g[x]);
            }
            g[x] = y;
            ans.push_back(cnt.size());
        }
        return ans;
    }
};

C++
class Solution {
public:
    vector<int> queryResult(int limit, vector<vector<int>>& queries) {
        unordered_map<int, int> g;
        unordered_map<int, int> cnt;
        vector<int> ans;
        for (auto& q : queries) {
            int x = q[0], y = q[1];
            cnt[y]++;
            if (g.contains(x) && --cnt[g[x]] == 0) {
                cnt.erase(g[x]);
            }
            g[x] = y;
            ans.push_back(cnt.size());
        }
        return ans;
    }
};

Java
Java

```

```

class Solution {
    public int[] queryResult(int limit, int[][] queries) {
        int[] ans = new int[queries.length];
        Map<Integer, Integer> ballToColor = new HashMap<>();
        Map<Integer, Integer> colorCount = new HashMap<>();
        for (int i = 0; i < queries.length; ++i) {
            final int ball = queries[i][0];
            final int color = queries[i][1];
            if (ballToColor.containsKey(ball)) {
                final int prevColor = ballToColor.get(ball);
                if (colorCount.merge(prevColor, -1, Integer::sum) == 0) {
                    colorCount.remove(prevColor);
                }
                ballToColor.put(ball, color);
                colorCount.merge(color, 1, Integer::sum);
            }
            ans[i] = colorCount.size();
        }
        return ans;
    }
}

Java
class Solution {
    public int[] queryResult(int limit, int[][] queries) {
        Map<Integer, Integer> g = new HashMap<>();
        Map<Integer, Integer> cnt = new HashMap<>();
        int n = queries.length;
        int[] ans = new int[n];
        for (int i = 0; i < n; ++i) {
            int x = queries[i][0], y = queries[i][1];
            cnt.merge(y, 1, Integer::sum);
            if (g.containsKey(x) && cnt.merge(g.get(x), -1, Integer::sum) == 0) {
                cnt.remove(g.get(x));
            }
            g.put(x, y);
            ans[i] = cnt.size();
        }
        return ans;
    }
}

Java

```

```

class Solution {
    public int[] queryResult(int limit, int[][] queries) {
        Map<Integer, Integer> g = new HashMap<>();
        Map<Integer, Integer> cnt = new HashMap<>();
        int n = queries.length;
        int[] ans = new int[n];
        for (int i = 0; i < n; ++i) {
            int x = queries[i][0], y = queries[i][1];
            cnt.merge(y, 1, Integer::sum);
            if (g.containsKey(x) && cnt.merge(g.get(x), -1, Integer::sum) == 0) {
                cnt.remove(g.get(x));
            }
            g.put(x, y);
            ans[i] = cnt.size();
        }
        return ans;
    }
}

TypeScript
function queryResult(limit: number, queries: number[][]): number[] {
    const g = new Map<number, number>();
    const cnt = new Map<number, number>();
    const ans: number[] = [];
    for (const [x, y] of queries) {
        cnt.set(y, cnt.get(y) + 1);
        if (g.has(x)) {
            const v = g.get(x);
            cnt.set(v, cnt.get(v) - 1);
            if (cnt.get(v) === 0) {
                cnt.delete(v);
            }
        }
        g.set(x, y);
        ans.push(cnt.size);
    }
    return ans;
}

TypeScript

```

```

function queryResult(limit: number, queries: number[][]): number[] {
    const g = new Map<number, number>();
    const cnt = new Map<number, number>();
    const ans: number[] = [];
    for (const [x, y] of queries) {
        cnt.set(y, cnt.get(y) + 1);
        if (g.has(x)) {
            const v = g.get(x);
            cnt.set(v, cnt.get(v) - 1);
            if (cnt.get(v) === 0) {
                cnt.delete(v);
            }
        }
        g.set(x, y);
        ans.push(cnt.size);
    }
    return ans;
}

Go
func queryResult(limit int, queries [][]int) (ans []int) {
    g := map[int]int{}
    cnt := map[int]int{}
    for _, q := range queries {
        x, y := q[0], q[1]
        cnt[y]++
        if v, ok := g[x]; ok {
            cnt[v]--
            if cnt[v] == 0 {
                delete(cnt, v)
            }
        }
        g[x] = y
        ans = append(ans, len(cnt))
    }
    return ans
}

Go
func queryResult(limit int, queries [][]int) (ans []int) {
    g := map[int]int{}
    cnt := map[int]int{}
    for _, q := range queries {
        x, y := q[0], q[1]
        cnt[y]++
        if v, ok := g[x]; ok {
            cnt[v]--
            if cnt[v] == 0 {
                delete(cnt, v)
            }
        }
        g[x] = y
        ans = append(ans, len(cnt))
    }
    return ans
}

```

Math · Pattern Solution (stored optimal)

```

Python
class Solution:
    def queryResult(self, n: int, queries: List[List[int]]) -> List[int]:
        node_color = {}
        color_count = {}
        ans = []
        for node, color in queries:
            if node in node_color:
                old = node_color[node]
                color_count[old] -= 1
                if color_count[old] == 0:
                    del color_count[old]
            node_color[node] = color
            color_count[color] = color_count.get(color, 0) + 1
            ans.append(len(color_count))
        return ans

C++
class Solution {
public:
    vector<int> queryResult(int n, vector<vector<int>>& queries) {
        unordered_map<int, int> nodeColor, colorCount;
        vector<int> ans;
        for (auto& q : queries) {
            int node = q[0], color = q[1];
            if (nodeColor.count(node)) {
                int old = nodeColor[node];
                if (--colorCount[old] == 0) colorCount.erase(old);
            }
            nodeColor[node] = color;
            colorCount[color]++;
            ans.push_back(colorCount.size());
        }
        return ans;
    }
};

#68 HARD
Text Justification
LeetCode · Simple · Math · LeetCode · Editorial
Array · String · Simulation
List · CodeSignal
Problem:
Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified.
You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces " " when necessary so that each line has exactly maxWidth characters.

```

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Notes:

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than 0 and not exceed maxWidth.
- The input array words contains at least one word.

Example 1:

Input: words = ["This", "is", "an", "example", "of", "text", "justification."], maxWidth = 16

Output:

```

["This is an",
"example of text",
"justification. "]

```

Example 2:

Input: words = ["What", "must", "be", "acknowledgment", "shall", "be"], maxWidth = 16

Output:

```

["What must be",
"acknowledgment ",
"shall be "]

```

Explanation: Note that the last line is "shall be " instead of "shall be", because the last line must be left-justified instead of fully-justified. Note that the second line is also left-justified because it contains only one word.

Example 3:

Input: words = ["Science", "is", "what", "we", "understand", "well", "enough", "to", "explain", "to", "a", "computer.", "Art", "is", "everything", "else", "we", "do"], maxWidth = 20

Output:

```

["Science is what we",
"understand well",
"enough to explain to",
"a computer. Art is",
"everything else we",
"do "]

```

Constraints:

- 1 <= words.length <= 300
- 1 <= words[i].length <= 20
- words[i] consists of only English letters and symbols.
- 1 <= maxWidth <= 100
- words[i].length <= maxWidth

Python

```
# Brute Force Approach
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        # Brute Force - Iterate all candidates
        for i in range(2, n + 1):
            is_valid = all(i % j == 0 for j in range(2, i))
            if is_valid: pass
            return 0

Python

def fullJustify(words, maxWidth):
    result = [] # final list of justified text lines.
    current_words = [] # current line's words.
    current_len = 0 # Total length of current line's words (including spaces).

    # Process all words.
    for word in words:
        # Check if adding the next word would exceed the maxWidth when including minimum one space between words.
        if current_words and (current_len + len(word) + len(current_words)) > maxWidth:
            total_spaces = maxWidth - current_len # Total spaces to distribute.
            gaps = len(current_words) - 1 # Number of gaps between words.

            # If there is only one word, left justify.
            if gaps == 0:
                line = current_words[0] + ' ' * total_spaces
            else:
                # Calculate even spaces and extras to add from the left.
                space_per_gap, extra = divmod(total_spaces, gaps)
                line = ''
                for i in range(gaps):
                    # Distribute as extra spaces to the leftmost gaps.
                    spaces = space_per_gap + (1 if i < extra else 0)
                    line = current_words[i] + ' ' * spaces
                result.append(line) # Append the last word.
                result.append(line)
```

```
# Reset the current line to start with the current word.
current_words = []
current_len = 0

# Add the word to the current line.
current_words.append(word)
current_len += len(word)

# Process the last line, which is left-justified.
line = ' '.join(current_words)
line += ' ' * (maxWidth - len(line))
result.append(line)
return result

# Example usage:
words = ["This", "is", "an", "example", "of", "text", "justification."]
maxWidth = 10
print(fullJustify(words, maxWidth))
```

Python

```
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        ans = []
        row = []
        rowLetters = 0

        for word in words:
            # If we place the word in this row, it will exceed the maximum width.
            # Therefore, we cannot put the word in this row and have to add spaces.
            # For gaps more than this row:
            if rowLetters + len(word) + len(row) > maxWidth:
                for i in range(maxWidth - rowLetters):
                    row[i] = (len(row) - 1 or 1) * ' '
                    ans.append(' '.join(row))
                row = []
                rowLetters = 0
                row.append(word)
                rowLetters += len(word)
            else:
                row.append(word)
                rowLetters += len(word)

        return ans + [' '.join(row).ljust(maxWidth)]
```

Python

```
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        ans = []
        i, n = 0, len(words)
        while i < n:
            t = []
            cur = len(words[i])
            t.append(words[i])
            i += 1
            while i < n and cur + 1 + len(words[i]) == maxWidth:
                cur += 1 + len(words[i])
                t.append(words[i])
                i += 1
            if i == n or len(t) == 1:
                left = ' '.join(t)
                right = ' ' * (maxWidth - len(left))
                ans.append(left + right)
                continue
            space_width = maxWidth - (len(t) + 1)
            w, n = divmod(space_width, len(t) - 1)
            row = []
            for j, s in enumerate(t[1:-1]):
                row.append(s)
                row.append(' ' * (w + (1 if j < n else 0)))
            row.append(t[-1])
            ans.append(' '.join(row))
            return ans
```

Python

```
...
Explanation:
- The function 'fullJustify' takes a list of words and a maximum width 'maxWidth' as inputs.
- It iterates over each word, deciding whether to add the current word to the current line ('cur') or to start a new line.
- If adding the current word to the line would exceed 'maxWidth', it justifies the current line by adding extra spaces between words as needed, then starts a new line.
- Once all words are processed, it left-justifies the last line by joining the remaining words in 'cur' with a single space and then using '.ljust(maxWidth)' to ensure the line is of maximum width.
- The result is a list of strings, where each string represents a justified line of text.

This solution carefully handles edge cases, such as when there's only one word in the line (avoiding division by zero) and ensuring the last line is left-justified instead of fully justified.

...
cur = []
len(cur) - 1
0
len(cur) - 1 or 1
1
...
# api: string.ljust(width, fillchar)
>>> "Hello".ljust(10)
'Hello '
>>> "Hello".ljust(10, '-')
'Hello-----'
>>> "Hello".ljust(10)
'Hello'
>>> "Hello".ljust(10, '-')
'Hello-----'
>>> "Hello World".ljust(20)
'Hello World'
>>> "Hello World".ljust(20, '-')
'Hello World-----'

# api: string.rjust(width, fillchar)
>>> "Hello".rjust(10)
'     Hello'
>>> "Hello".rjust(10, '-')
'-----Hello'
>>> "Hello".rjust(10)
'Hello'
>>> "Hello World".rjust(20)
'Hello World'
>>> "Hello World".rjust(20, '-')
'-----Hello World'

...

```

```
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        res, cur, num_of_letters = [], [], 0

        for w in words:
            # 'len(cur)' is the space count. First line (empty), right one less after adding " ".
            if num_of_letters + len(cur) + len(w) > maxWidth:
                for i in range(maxWidth - num_of_letters):
                    # The row is full so far, starting with the edge case "len(cur) == 1".
                    cur[i % (len(cur) - 1 or 1)] += ' '
                res.append(' '.join(cur))
                cur, num_of_letters = [], 0
                cur += [w]
                num_of_letters += len(w)

        return res + [' '.join(cur).ljust(maxWidth)]

#####
...
>>> 17 % 3
2
>>> divmod(17, 3)
(5, 2)
...
>>> ["This", "is", "an"]

C++
C++
```

```
class Solution {
public:
    vector<string> fullJustify(vector<string>& words, int maxWidth) {
        vector<string> ans;
        for (int i = 0; i < words.size(); i++) {
            vector<string> t = {words[i]};
            int cur = words[i].size();
            while (i < n && cur + 1 + words[i+1].size() <= maxWidth) {
                cur += 1 + words[i+1].size();
                t.emplace_back(words[i+1]);
            }
            if (i == n || t.size() == 1) {
                string left = t[0];
                for (int j = 1; j < t.size(); ++j) {
                    left += " " + t[j];
                }
                string right = string(maxWidth - left.size(), ' ');
                ans.emplace_back(left + right);
                continue;
            }
            int spaceWidth = maxWidth - (len(t) + 1);
            int w = spaceWidth / (t.size() - 1);
            int n = spaceWidth % (t.size() - 1);
            string row;
            for (int j = 0; j < t.size(); ++j) {
                row += t[j] + string(w + (j < n ? 1 : 0), ' ');
            }
            row = t.back();
            ans.emplace_back(row);
        }
        return ans;
    }
};

Java
Java
```

```

class Solution {
public List<String> fullJustify(String[] words, int maxWidth) {
    List<String> ans = new ArrayList<>();
    for (int i = 0, n = words.length; i < n; ) {
        List<String> w = new ArrayList<>();
        t.add(words[i]);
        int cut = words[i].length();
        while (i < n && cut + 1 + words[i+1].length() <= maxWidth) {
            cut += 1 + words[i+1].length();
            t.add(words[i++]);
        }
        if (t == n || t.size() == 1) {
            String left = String.join(" ", t);
            String right = " ".repeat(maxWidth - left.length());
            ans.add(left + right);
            continue;
        }
        int spaceWidth = maxWidth - (cut - t.size() + 1);
        int s = spaceWidth / (t.size() - 1);
        int m = spaceWidth % (t.size() - 1);
        Stringbuilder row = new StringBuilder();
        for (int j = 0, j = t.size() - 1; j++) {
            row.append(t.get(j));
            row.append(" ".repeat(s + (j < m ? 1 : 0)));
        }
        row.append(t.get(t.size() - 1));
        ans.add(row.toString());
    }
    return ans;
}
}

```

Java

```

public class Solution {
public List<String> fullJustify(String[] words, int maxWidth) {
    var ans = new List<String>();
    for (int i = 0, n = words.length; i < n; ) {
        var t = new List<String>();
        t.add(words[i]);
        int cut = words[i].length;
        while (i < n && cut + 1 + words[i+1].length <= maxWidth) {
            t.add(words[i++]);
            cut += 1 + words[i].length;
        }
        if (t == n || t.Count == 1) {
            string left = string.Join(" ", t);
            string right = new string(" ", maxWidth - left.Length);
            ans.Add(left + right);
            continue;
        }
        int spaceWidth = maxWidth - (cut - t.Count + 1);
        int s = spaceWidth / (t.Count - 1);
        int m = spaceWidth % (t.Count - 1);
        var row = new StringBuilder();
        for (int j = 0; j < t.Count - 1; j++) {
            row.Append(t[j]);
            row.Append(new string(" ", s + (j < m ? 1 : 0)));
        }
        row.Append(t[t.Count - 1]);
        ans.Add(row.ToString());
    }
    return ans;
}
}

```

TypeScript
TypeScript

```

function fullJustify(words, string[], maxWidth: number): string[] {
    const ans: string[] = [];
    for (let i = 0, n = words.length; i < n; ) {
        const t: string[] = [words[i]];
        let cut = words[i].length;
        while (i < n && cut + 1 + words[i+1].length <= maxWidth) {
            t.push(words[i++]);
            cut += 1 + words[i].length;
        }
        if (t == n || t.length == 1) {
            const left: string = t.join(" ");
            const right: string = " ".repeat(maxWidth - left.length);
            ans.push(left + right);
            continue;
        }
        const spaceWidth: number = maxWidth - (cut - t.length + 1);
        const s: number = Math.floor(spaceWidth / (t.length - 1));
        const m: number = spaceWidth % (t.length - 1);
        const row: string[] = [];
        for (let j = 0, j = t.length - 1; j++) {
            row.push(t[j]);
            row.push(" ".repeat(s + (j < m ? 1 : 0)));
        }
        row.push(t[t.length - 1]);
        ans.push(row.join(" "));
    }
    return ans;
}

```

Go

```

func fullJustify(words []string, maxWidth int) []string {
    for i, n := 0, len(words); i < n; {
        t := []string{}
        cut := len(words[i])
        for i = n && cut + 1 + len(words[i+1]) <= maxWidth {
            t = append(t, words[i])
            cut += 1 + len(words[i])
            i++
        }
        if i == n || len(t) == 1 {
            left := strings.Join(t, " ")
            right := strings.Repeat(" ", maxWidth-len(left))
            ans = append(ans, left+right)
            continue
        }
        spaceWidth := maxWidth - (len(t) - 1)
        s := spaceWidth / (len(t) - 1)
        m := spaceWidth % (len(t) - 1)
        row := strings.Builder{}
        for j, n := range t; j < len(t)-1; {
            row.WriteString(t[j])
            row.WriteString(strings.Repeat(" ", s + (j < m ? 1 : 0)))
            if j < n {
                row.WriteString(" ")
            }
        }
    }
}

```

```

}
row.WriteString(t[len(t)-1])
ans = append(ans, row.String())
}
return
}

```

[I] Math · Pattern Solution (stored optimal)

Python

```

def fullJustify(words, maxWidth):
    result = [] # Final list of justified text lines.
    current_words = [] # Current line's words.
    current_len = 0 # Total length of current line's words (excluding spaces).

    # Process all words.
    for word in words:
        # Check if adding the next word would exceed the maxWidth when including minimum one space between words.
        if current_words and (current_len + len(word) + len(current_words)) > maxWidth:
            total_spaces = maxWidth - current_len # Total spaces to distribute.
            gaps = len(current_words) - 1 # Number of gaps between words.

            # If there is only one word, left justify.
            if gaps == 0:
                line = current_words[0] + ' ' * total_spaces
            else:
                # Calculate how many spaces and extras to add from the left.
                space_per_gap, extra = divmod(total_spaces, gaps)
                line = ''
                for i in range(gaps):
                    # Distribute even extra spaces to the leftmost gaps.
                    spaces = space_per_gap + (1 if i < extra else 0)
                    line += current_words[i] + ' ' * spaces
                line += current_words[-1] # Append the last word.
            result.append(line)

            # Reset the current line to start with the current word.
            current_words = [word]
            current_len = len(word)

        # Add the word to the current line.
        current_words.append(word)
        current_len += len(word)

    # Process the last line, which is left-justified.
    line = ' '.join(current_words)
    line += ' ' * (maxWidth - len(line))
    result.append(line)
    return result

# Example usage:
words = ["This", "is", "an", "example", "of", "text", "justification."]
maxWidth = 16
print(fullJustify(words, maxWidth))

```

C++

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

vector<string> fullJustify(vector<string>& words, int maxWidth) {
    vector<string> result; // Final array of justified lines.
    vector<string> currentWords; // Words in the current line.
    int currentLen = 0; // Total length of words in current line.

    for (const string &word : words) {
        // Check if adding next word exceeds the width (consider one space between words).
        if (currentWords.empty() && currentLen + word.size() < currentWords.size() > maxWidth) {
            int totalSpaces = maxWidth - currentLen; // Total spaces to distribute.
            int gaps = currentWords.size() - 1; // Number of gaps between words.
            string line;

            if (gaps == 0) {
                // Only one word: left justify.
                line = currentWords[0] + string(totalSpaces, ' ');
            } else {
                int spacePerGap = totalSpaces / gaps; // Base space per gap.
                int extra = totalSpaces % gaps; // Remainder extra spaces.
                for (int i = 0; i < gaps; i++) {
                    int spaces = spacePerGap + (i < extra ? 1 : 0);
                    line += currentWords[i] + string(spaces, ' ');
                }
                line += currentWords.back(); // Append the last word.
            }
            result.push_back(line);
            currentWords.clear();
            currentLen = 0;
        }
        // Add current word to the line.
        currentWords.push_back(word);
        currentLen += word.size();
    }

    // Process the last line (left-justified).
    string lastLine;
    for (int i = 0; i < currentWords.size(); i++) {
        if (i < 0) lastLine += " ";
        lastLine += currentWords[i];
    }
    lastLine += string(maxWidth - lastLine.size(), ' ');
    result.push_back(lastLine);

    return result;
}

int main() {

```

```

vector<string> words = {"This", "is", "an", "example", "of", "text", "justification."};
int maxWidth = 16;
vector<string> justifiedText = fullJustify(words, maxWidth);
for (auto &line : justifiedText) {
    cout << line << endl;
}
return 0;
}

```


#9 Palindrome Number [Easy]

Key Insights

- Negative numbers are not palindromic.
- A number ending in 0 (except 0 itself) cannot be a palindrome.
- Instead of reversing the entire number (risking overflow), reverse only half of the number and compare with the other half.
- For numbers with an odd number of digits, the middle digit can be disregarded.

Space & Time

Time Complexity: $O(\log_{10}(n))$ - Each iteration reduces the number by a factor of 10.Space Complexity: $O(1)$ - Only constant extra space is used.

Solution

The solution checks for obvious non-palindrome cases first (negative numbers and numbers ending with 0 that are not 0). Then, it reverses half of the digits of the number while the original number is larger than the reversed half. Finally, it compares the remaining part of the original number with the reversed half. In the case of an odd number of digits, the middle digit is removed by dividing the reversed half by 10 before the final comparison.

#13 Roman to Integer [Easy]

Key Insights

- Map each Roman numeral character to its integer value using a hash table or dictionary.
- Loop through the string and check if the current numeral is less than the next numeral; if so, subtract its value.
- Otherwise, add its value to the total.
- This approach takes advantage of the subtractive notation used in Roman numerals.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.Space Complexity: $O(1)$, since the mapping and additional variables use constant space.

Solution

We use a dictionary (or hash map) to store the values of each Roman numeral. As we iterate over the string, we compare the current numeral's value with the next numeral's value. When a numeral of smaller value appears before a numeral of larger value, we subtract the current numeral's value; otherwise, we add it. This technique efficiently handles both the standard and subtractive cases in one pass through the string.

#504 Base 7 [Easy]

Key Insights

- Handle the edge case where $\text{num} = 0$.
- Use division and modulo operations to extract base 7 digits.
- Manage negative numbers by converting to positive and reattaching the minus sign.
- Build the base 7 digits in reverse order during iteration.

Space & Time

Time Complexity: $O(\log_7 |n|)$ where n is the absolute value of the number.Space Complexity: $O(\log_7 |n|)$ to store the digits of the number.

Solution

The algorithm follows these steps:

• Check if num equals 0 and return "0". - Determine if the number is negative. If so, work with its absolute value but remember the negative flag. - Iteratively compute the remainder when divided by 7 (using modulo) and update the number by integer division by 7. - Append each remainder (converted to character) to a list representing the digits. - The digits are computed in reverse order; reverse the list for the correct order. - If the original number was negative, append the "-" sign to the front. - Join the digits to form the resulting base 7 string. This approach uses basic arithmetic (division and modulo) and a list (or equivalent structure) to accumulate the computed digits.

#1056 Confusing Number [Easy]

Key Insights

- Use a mapping dictionary for valid rotations: {0:0, 1:1, 6:9, 8:8, 9:6}.
- Process the digits of n from right to left to form the rotated number.
- If any digit is not present in the mapping, the number is instantly not confusing.
- Compare the rotated number with the original number to check if they are different.

Space & Time

Time Complexity: $O(d)$, where d is the number of digits in n (approximately $O(\log n)$).Space Complexity: $O(d)$, needed to store the rotated number string.

Solution

The approach is to convert the number to a string and iterate through it in reverse. For each digit, check if it is valid using the predefined mapping. If it is, append its rotated counterpart to a new string. After processing, convert the rotated string to an integer (ignoring any leading zeros). Finally, compare the rotated number to the original number. If they are different, the number is confusing.

#1228 Missing Number in Arithmetic Progression [Easy]

Key Insights

- The complete arithmetic progression has equal differences between consecutive numbers.
- The missing element can be found by calculating the expected common difference using the first and last elements.
- Since exactly one number is missing, iterate through the provided array to detect where the difference deviates from the expected common difference.

Space & Time

Time Complexity: $O(n)$ Space Complexity: $O(1)$

Solution

We first compute the expected common difference (diff) for the original arithmetic progression. Since the actual progression had one more element than the current array, the formula used is: $\text{diff} = (\text{last element} - \text{first element}) / (n)$ where n is the length of the current array. Next, iterate over the array and check the difference between each pair of consecutive elements. When the actual difference deviates from diff, it indicates that the missing number is the previous element plus diff. This approach uses a simple loop and constant extra space.

#1427 Perform String Shifts [Easy]

Key Insights

- Consolidate all shifts by calculating a net shift: subtract for left shifts and add for right shifts.
- Use modulo arithmetic with the string length to avoid unnecessary full rotations.
- Slice the string based on the effective shift to construct the final string.

Space & Time

Time Complexity: $O(n + m)$, where n is the length of the string and m is the number of shift operations.Space Complexity: $O(n)$ for the output string.

Solution

The solution involves computing the net shift value by iterating through the shift operations. Left shifts subtract from the net value, and right shifts add to the net value. After obtaining the cumulative shift, use modulo with the string length to normalize the shift, ensuring it lies within bounds. The final string is constructed by slicing the string into two parts and concatenating them appropriately. This approach leverages string slicing as the primary operation for shifting.

#7 Reverse Integer [Medium]

Key Insights

- The problem requires reversing the digits while preserving the sign.
- Watch out for integer overflow due to the reversed number exceeding the 32-bit signed integer limits.
- The approach involves repeatedly extracting the last digit from the number and appending it to a new reversed integer.
- Overflow checks are done during each iteration before actually appending the digit.

Space & Time

Time Complexity: $O(n)$, where n is the number of digits in the integer.Space Complexity: $O(1)$, only constant extra space is used.

Solution

The solution uses a while loop to extract the last digit from the original number using modulo operation ($x \% 10$) and then removes this digit using integer division ($x // 10$). A reversed number is constructed by multiplying the current reversed number by 10 and adding the extracted digit. Before appending each digit, the algorithm checks if the new reversed number will overflow a 32-bit signed integer by comparing it with the thresholds derived from INT_MAX and INT_MIN. The algorithm handles negative values by working with x directly since the modulo and division operations work properly with negatives in most languages.

#50 Pow(x, n) [Medium]

Key Insights

- Use exponentiation by squaring to reduce the number of multiplications.
- For a negative exponent, compute the power for $|n|$ and then take the reciprocal.
- Handle the edge case when n is 0 (return 1).
- The algorithm works in $O(\log |n|)$ time complexity.

Space & Time

Time Complexity: $O(\log |n|)$ Space Complexity: $O(1)$ for the iterative solution, or $O(\log |n|)$ if implemented recursively

Solution

We use exponentiation by squaring, an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how it works: - If n is negative, switch x to $1/x$ and n to $-n$. - Initialize the result as 1. - Loop while n is greater than 0: If the current n is odd, multiply the result by x . Square the base x . Divide n by 2 (using integer division). This iterative approach efficiently computes the power in logarithmic steps. The same approach can be implemented recursively.

#380 Insert Delete GetRandom O(1) [Medium]

Key Insights

- Use a dynamic array (or list) to store the elements to allow $O(1)$ access by index.
- Use a hash table (or dictionary/map) to store the mapping from value to its index in the array.
- For removal, swap the element to be removed with the last element in the array, then delete the last element. This avoids expensive shifting of elements.
- getRandom can be implemented by generating a random index into the array.

Space & Time

Time Complexity: $O(1)$ on average for insert, remove, and getRandom operations.Space Complexity: $O(n)$ where n is the number of elements stored in the data structure.

Solution

The solution involves maintaining a list to store the values and a hash map to quickly check for the presence of a value and to know its index in the list. When an element is removed, swap it with the last element of the list and update the map accordingly; then, remove the last element in $O(1)$ time. This approach ensures that insert, remove, and getRandom can all be achieved in constant time on average.

#1017 Convert to Base -2 [Medium]

Key Insights

- Use division and remainder operations to simulate converting a number to a different base.
- When working with a negative base (-2), the remainder can become negative; adjust the remainder (by adding the base's absolute value) and update the quotient accordingly.
- For $n = 0$, simply return "0" since there is nothing to convert.
- Build the answer by collecting digits (remainders) and then reversing the order since the conversion provides digits from least significant to most significant.

Space & Time

Time Complexity: $O(\log |n|)$ - the loop runs for about the number of digits in the base -2 representation.Space Complexity: $O(\log |n|)$ - extra space for storing the computed digits.

Solution

To solve the problem, we simulate the conversion process similar to how you would convert an integer to a positive base. However, since the base here is -2, special handling of negative remainders is needed. In each iteration, we: - Divide the current number by -2. - Calculate the remainder. If the remainder is negative, adjust it by adding 2, and increment the quotient to compensate. - Append the computed remainder to a list (which represents the binary digits). - Continue until the number reduces to zero. Finally, reverse the list to form the correct string representation from the most significant to the least significant digit.

#3160 Find the Number of Distinct Colors Among the Balls [Medium]

Key Insights

- Use a hash map to store the current color of each ball that has been updated.
- Use another hash map to store how many balls currently use each color.
- When a ball is recolored, decrement the count of its old color first.
- If an old color count becomes zero, remove it from the color-frequency map.
- Add the new color to the ball and increment its frequency.
- After each query, the number of distinct colors is the number of keys in the color-frequency map.

Space & Time

Time Complexity: $O(q)$ average for q queries, since each query does only $O(1)$ average-time hash map work.Space Complexity: $O(\min(n, q))$ for colored balls, plus $O(q)$ in the worst case for distinct active colors.

Solution

We solve the problem using two hash maps. The first map stores the current color of each ball, and the second map stores the frequency of each active color. For every query, if the ball already has a color, we decrement the old color's count and remove that color from the frequency map if its count reaches zero. Then we update the ball with the new color and increment the new color's frequency. After processing the query, the number of distinct colors is simply the number of keys remaining in the frequency map.

#68 Text Justification [Hard]

Key Insights

• Use a greedy approach to fill each line with as many words as possible without exceeding maxWidth. • Calculate total space to distribute by subtracting the combined words length from maxWidth. • If the line has more than one word, distribute extra spaces evenly; if not, pad the line with spaces at the end. • Handle the last line separately by left-justifying the text (i.e., adding a single space between words and padding the end).

- Use a greedy approach to fill each line with as many words as possible without exceeding maxWidth. • Calculate total space to distribute by subtracting the combined words length from maxWidth. • If the line has more than one word, distribute extra spaces evenly; if not, pad the line with spaces at the end. • Handle the last line separately by left-justifying the text (i.e., adding a single space between words and padding the end).

Space & Time

Time Complexity: $O(n)$ where n is the number of words, since each word is processed once.Space Complexity: $O(n)$ for storing the result and temporary buffers.

Solution

The solution iterates over the list of words and groups them into lines such that the combined length of the words and minimum required spaces does not exceed maxWidth. For each completed group (line): - If it's not the last line, calculate extra spaces required. - If the line has multiple words, distribute the spaces evenly, adding any extra spaces from left to right. - If the line contains a single word, append spaces to the right. - For the last line, join words with a single space and pad the remaining width with spaces. Data structures used include lists (or arrays) for current words accumulation and the final result. The logic is implemented using a simulation approach to mimic text formatting.

#12 Arrays & Hashing — 18 questions · #12 of 21

#1 EASY

Two Sum

LeetCode · SimplyLeet · WalkCC · LeetCode · Editorial

Array · Hash Table

Lists · Grid 169

Problem:

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].

Example 2:

Input: nums = [3,2,4], target = 6
Output: [1,2]

Example 3:

Input: nums = [3,3], target = 6
Output: [0,1]

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$
- Only one valid answer exists.

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

>> Brute Force [Arrays & Hashing] Interview Prep

Python

Brute Force Approach

class Solution:

```
def two_sum(self, nums, target):  
    """  
    Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.  
    """  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            if nums[i] + nums[j] == target:  
                return [i, j]
```

```

Python
Python
def two_sum(nums, target):
    # Create a dictionary to store the number and its index
    num_to_index = {}
    # Iterate over the list with index
    for index, num in enumerate(nums):
        # Calculate the complement needed to reach the target
        complement = target - num
        # Check if the complement is in the dictionary
        if complement in num_to_index:
            # Return the index of the complement and the current index
            return [num_to_index[complement], index]
        # Store the number with its index
        num_to_index[num] = index
    # Since the problem guarantees one solution, we don't need a return statement here.

# Example usage:
# print(two_sum([2, 7, 11, 15, 3], 9)) # Output: 0, 1

Python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        numToIndex = {}
        for i, num in enumerate(nums):
            if target - num in numToIndex:
                return [numToIndex[target - num], i]
            numToIndex[num] = i

Python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        d = {}
        for i, x in enumerate(nums):
            if (x in target - x) && x in d:
                return [d[x], i]
            d[x] = i

Python
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        n = {}
        for i, x in enumerate(nums):
            y = target - x
            if y in n:
                return [n[y], i]
            n[x] = i

C++
C++

```

```

class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> numToIndex;

        for (int i = 0; i < nums.size(); ++i) {
            if (count(nums.begin() + i, nums.end(), target - nums[i]) > 0) {
                return {i, numToIndex[target - nums[i]]};
            }
            numToIndex[nums[i]] = i;
        }

        throw;
    }
};

C++
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> d;
        for (int i = 0; i < nums.size(); ++i) {
            int x = nums[i];
            int y = target - x;
            if (d.contains(y)) {
                return {d[y], i};
            }
            d[x] = i;
        }
    }
};

C++
class Solution {
//
// @param Integer[] nums
// @param Integer target
// @return Integer[]
//
    function twoSum(nums, target) {
        let n = {};
        for (let i = 0; i < nums.length; ++i) {
            let y = target - nums[i];
            if (n.contains(y)) {
                return [d[y], i];
            }
            n[nums[i]] = i;
        }
    }
};

C++

```

```

class Solution {
    func twoSum(_ nums: [Int], _ target: Int) -> [Int] {
        var d = [Int: Int]()
        for (i, x) in nums.enumerated() {
            let y = target - x
            if let j = d[y] {
                return [j, i]
            }
            d[x] = i
        }
        return []
    }
}

C++
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> m;
        for (int i = 0; i < nums.size(); ++i) {
            int x = nums[i];
            int y = target - x;
            if (m.contains(y)) {
                return {m[y], i};
            }
            m[x] = i;
        }
    }
};

C++
class Solution {
//
// @param Integer[] nums
// @param Integer target
// @return Integer[]
//
    function twoSum(nums, target) {
        for (let i = 0; i < nums.length; ++i) {
            let y = target - nums[i];
            if (m.contains(y)) {
                return [m[y], i];
            }
            m[nums[i]] = i;
        }
    }
}

C++

```

```

class Solution {
    func twoSum(_ nums: [Int], _ target: Int) -> [Int] {
        var m = [Int: Int]()
        var i = 0
        while true {
            let x = nums[i]
            let y = target - x
            if let j = m[y] {
                return [j, i]
            }
            m[nums[i]] = i
            i += 1
        }
    }
}

Java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> numToIndex = new HashMap<>();

        for (int i = 0; i < nums.length; ++i) {
            if (numToIndex.containsKey(target - nums[i])) {
                return new int[] {numToIndex.get(target - nums[i]), i};
            }
            numToIndex.put(nums[i], i);
        }

        throw new IllegalArgumentException();
    }
}

Java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> d = new HashMap<>();
        for (int i = 0; i < nums.length; ++i) {
            int x = nums[i];
            int y = target - x;
            if (d.containsKey(y)) {
                return new int[] {d.get(y), i};
            }
            d.put(x, i);
        }
    }
}

Java

```

```

use std::collections::HashMap;

impl Solution {
    pub fn two_sum(nums: Vec<i32>, target: i32) -> Vec<i32> {
        let mut d = HashMap::new();
        for (i, &x) in nums.iter().enumerate() {
            let y = target - x;
            if let Some(j) = d.get(&y) {
                return vec![j as i32, i as i32];
            }
            d.insert(x, i);
        }
        vec![]
    }
}

Java
public class Solution {
    public int[] twoSum(int[] nums, int target) {
        var d = new Dictionary<int, int>();
        for (int i = 0; i < nums.length; ++i) {
            int x = nums[i];
            int y = target - x;
            if (d.TryGetValue(y, out j)) {
                return new int[] {j, i};
            }
            if (!d.ContainsKey(x)) {
                d.Add(x, i);
            }
        }
    }
}

Java
class Solution {
    func twoSum(nums Array<int64>, target: int64) Array<int64> {
        let d = HashMap<int64, int64>()
        for (i in 0..nums.size) {
            if (d.containsKey(target - nums[i])) {
                return [d[target - nums[i]], i]
            }
            d[nums[i]] = i
        }
        []
    }
}

Java

```

```

class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> m = new HashMap<>();
        for (int i = 0; i < nums.length; ++i) {
            int x = nums[i];
            int y = target - x;
            if (m.containsKey(y)) {
                return new int[] {m.get(y), i};
            }
            m.put(x, i);
        }
    }
}

Java
import scala.collection.mutable

object Solution {
    def twoSum(nums: Array<Int>, target: Int): Array<Int> = {
        var map = new mutable.HashMap<Int, Int>()
        for (i <- 0 to nums.length - 1) {
            if (map.contains(target - nums(i))) {
                return Array(map(target - nums(i)), i)
            }
            map += (nums(i) -> i)
        }
        Array(0, 0)
    }
}

Java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        var m = new Dictionary<int, int>();
        for (int i = 0; i < nums.length; ++i) {
            int x = nums[i];
            int y = target - x;
            if (m.TryGetValue(y, out j)) {
                return new int[] {j, i};
            }
            if (!m.ContainsKey(x)) {
                m.Add(x, i);
            }
        }
    }
}

Java

```

```
use std::collections::HashMap;

impl Solution {
    pub fn two_sum(nums: Vec<i32>, target: i32) -> Vec<i32> {
        let mut map = HashMap::new();
        for (i, item) in nums.iter().enumerate() {
            if map.contains_key(item) {
                return vec![i as i32, map[item]];
            } else {
                let x = target - nums[i];
                map.insert(x, i as i32);
            }
        }
        unreachable!()
    }
}
```

JavaScript

JavaScript

```
//
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
var twoSum = function(nums, target) {
    const d = new Map();
    for (let i = 0; i < nums.length; i++) {
        const x = nums[i];
        const y = target - x;
        if (d.has(y)) {
            return [d.get(y), i];
        }
        d.set(x, i);
    }
};
```

JavaScript

```
# @param {Integer[]} nums
# @param {Integer} target
# @return {Integer[]}
def two_sum(nums, target):
    d = {}
    num_to_index = {}
    for i, num in enumerate(nums):
        y = target - num
        if d.get(y) is not None:
            return [d.get(y), i]
        d[num] = i
    return []
```

JavaScript

```
//
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
var twoSum = function(nums, target) {
    const d = new Map();
    for (let i = 0; i < nums.length; i++) {
        const x = nums[i];
        const y = target - x;
        if (d.has(y)) {
            return [d.get(y), i];
        }
        d.set(x, i);
    }
};
```

JavaScript

```
# @param {Integer[]} nums
# @param {Integer} target
# @return {Integer[]}
def two_sum(nums, target):
    num_to_index = {}
    for i, num in enumerate(nums):
        y = target - num
        if d.get(y) is not None:
            return [d.get(y), i]
        d[num] = i
    return []
```

TypeScript

```
function twoSum(nums: number[], target: number): number[] {
    const d = new Map<number, number>();
    for (let i = 0; i < nums.length; i++) {
        const x = nums[i];
        const y = target - x;
        if (d.has(y)) {
            return [d.get(y), i];
        }
        d.set(x, i);
    }
}
```

TypeScript

```
function twoSum(nums: number[], target: number): number[] {
    const map: Map<number, number> = new Map();

    for (let i = 0; i < nums.length; i++) {
        const x = nums[i];
        const y = target - x;
        if (map.has(y)) {
            return [map.get(y), i];
        }
        map.set(x, i);
    }
}
```

Go

Go

```
func twoSum(nums []int, target int) []int {
    d := map[int]int{}
    for i := 0; i < len(nums); i++ {
        x := nums[i]
        y := target - x
        if j, ok := d[x]; ok {
            return []int{i, j}
        }
        d[x] = i
    }
}
```

Go

```
func twoSum(nums []int, target int) []int {
    m := map[int]int{}
    for i := 0; i < len(nums); i++ {
        x := nums[i]
        y := target - x
        if j, ok := m[x]; ok {
            return []int{i, j}
        }
        m[x] = i
    }
}
```

Kotlin

Kotlin

```
class Solution {
    fun twoSum(nums: IntArray, target: Int): IntArray {
        val m = mutableMapOf<Int, Int>()
        for (i in 0 until nums.size) {
            val x = nums[i]
            val y = target - x
            if (m.containsKey(y)) {
                return intArrayOf(i, m[y])
            }
            m[x] = i
        }
        return intArrayOf()
    }
}
```

Scala

Scala

```
import scala.collection.mutable

object Solution {
    def twoSum(nums: Array[Int], target: Int): Array[Int] = {
        val d = mutable.Map[Int, Int]()
        for (i <- nums.indices) {
            val x = nums[i]
            val y = target - x
            if (d.contains(y)) {
                return Array(d(y), i)
            } else {
                d[x] = i
            }
        }
        Array()
    }
}
```

[*] Arrays & Hashing - Pattern Solution (stored optimal)

Python

```
def two_sum(nums, target):
    # Create a dictionary to store the number and its index
    num_to_index = {}
    # Iterate over the list and find the complement
    for index, num in enumerate(nums):
        # Calculate the complement needed to reach the target
        complement = target - num
        # Check if the complement is in the dictionary
        if complement in num_to_index:
            # Return the index of the complement and the current index
            return [num_to_index[complement], index]
    # Since the problem guarantees one solution, we don't need a return statement here.

# Example usage
# print(two_sum([2, 7, 11, 15], 9)) # Output: [0, 1]
```

```
class Solution {
    // Create a dictionary to store the number and its index
    unordered_map<int, int> num_to_index;
    // Iterate over the vector and find the complement
    for (int i = 0; i < nums.size(); i++) {
        // Calculate the complement needed to reach the target
        int complement = target - nums[i];
        // Check if the complement is in the map
        if (num_to_index.find(complement) != num_to_index.end()) {
            // Return the index of the complement and the current index
            return {num_to_index[complement], i};
        }
        // Store the number with its index
        num_to_index[nums[i]] = i;
    }
    // Since the problem guarantees one solution, no return is needed here.
}
```

#163

EASY

Missing Ranges

LeetCode - SimplifyLeetCode - LeetCode - Editorial

Array

Like Premium 98

Problem:

You are given an inclusive range [lower, upper] and a sorted unique integer array nums, where all elements are within the inclusive range.

A number x is considered missing if x is in the range [lower, upper] and x is not in nums.

Return the shortest sorted list of ranges that exactly covers all the missing numbers. That is, no element of nums is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1:

Input: nums = [0,1,3,5,7,9], lower = 0, upper = 9

Output: [[2,2],[4,4],[6,6],[8,8]]

Explanation: The ranges are:

[2,2]

[4,4]

[6,6]

[8,8]

Example 2:

Input: nums = [-1], lower = -1, upper = -1

Output: []

Explanation: There are no missing ranges since there are no missing numbers.

Constraints:

• -109 <= lower <= upper <= 109

• 0 <= nums.length <= 100

• lower <= nums[i] <= upper

• All the values of nums are unique.

>> Brute Force [Arrays & Hashing] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def findMissingRanges(self, nums, lower, upper):
        # Create a set of all numbers in the range
        n = len(nums)
        for i in range(lower, upper + 1):
            # Check if the number is in the array
            if i not in nums:
                # Add the missing range
                return [i, i]
        return []
```

Python

Python

```
# Python solution for finding missing ranges
def findMissingRanges(lower, upper):
    missing_ranges = []
    # Create a set of all numbers in the range
    prev = lower - 1
    # Iterate over the range and find the missing ranges
    for i in range(lower, upper + 1):
        # Check if the number is in the array
        if i not in nums:
            # Add the missing range
            missing_ranges.append([prev + 1, i])
        prev = i
    return missing_ranges

# Example usage
print(findMissingRanges(0, 10, 10))
```

Python

```

class Solution {
    def findMissingRanges(
        self, num: List<Int>, lower: Int, upper: Int
    ) -> List<List<Int>> {
        def getRange(lo: Int, hi: Int) -> List<Int>:
            if lo == hi
                return [lo, hi]
            return [lo, hi]

        if not num:
            return [getRange(lower, upper)]

        ans = []

        if num[0] > lower:
            ans.append(getRange(lower, num[0] - 1))

        for prev, curr in zip(num, num[1:]):
            if curr - prev > 2:
                ans.append(getRange(prev + 1, curr - 1))

        if num[-1] < upper:
            ans.append(getRange(num[-1] + 1, upper))

        return ans
    }
}

```

```

Python
class Solution:
    def findMissingRanges(
        self, num: List<int>, lower: int, upper: int
    ) -> List[List<int>]:
        n = len(num)
        if n == 0:
            return [[lower, upper]]
        ans = []
        if num[0] > lower:
            ans.append([lower, num[0] - 1])
        for a, b in pairwise(num):
            if b - a > 2:
                ans.append([a + 1, b - 1])
        if num[-1] < upper:
            ans.append([num[-1] + 1, upper])
        return ans

```

Python

```

class Solution {
    def findMissingRanges(
        self, num: List<int>, lower: int, upper: int
    ) -> List<List<int>> {
        n = len(num)
        if n == 0:
            return [[lower, upper]]
        ans = []
        if num[0] > lower:
            ans.append([lower, num[0] - 1])
        for a, b in pairwise(num):
            if b - a > 2:
                ans.append([a + 1, b - 1])
        if num[-1] < upper:
            ans.append([num[-1] + 1, upper])
        return ans
    }
}

```

C++

C++

```

class Solution {
public:
    vector<vector<int>> findMissingRanges(vector<int>& num, int lower,
                                        int upper) {
        if (num.empty())
            return {getRange(lower, upper)};

        vector<vector<int>> ans;

        if (num.front() > lower)
            ans.push_back(getRange(lower, num.front() - 1));

        for (int i = 1; i < num.size(); ++i)
            if (num[i] > num[i - 1] + 1)
                ans.push_back(getRange(num[i - 1] + 1, num[i] - 1));

        if (num.back() < upper)
            ans.push_back(getRange(num.back() + 1, upper));

        return ans;
    }

private:
    vector<int> getRange(int lo, int hi) {
        if (lo == hi)
            return {lo, hi};
        return {lo, hi};
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> findMissingRanges(vector<int>& num, int lower, int upper) {
        int n = num.size();
        if (n == 0) {
            return {{lower, upper}};
        }
        vector<vector<int>> ans;
        if (num[0] > lower) {
            ans.push_back({lower, num[0] - 1});
        }
        for (int i = 1; i < num.size(); ++i) {
            if (num[i] - num[i - 1] > 2) {
                ans.push_back({num[i - 1] + 1, num[i] - 1});
            }
        }
        if (num[n - 1] < upper) {
            ans.push_back({num[n - 1] + 1, upper});
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> findMissingRanges(vector<int>& num, int lower, int upper) {
        int n = num.size();
        if (n == 0) {
            return {{lower, upper}};
        }
        vector<vector<int>> ans;
        if (num[0] > lower) {
            ans.push_back({lower, num[0] - 1});
        }
        for (int i = 1; i < num.size(); ++i) {
            if (num[i] - num[i - 1] > 2) {
                ans.push_back({num[i - 1] + 1, num[i] - 1});
            }
        }
        if (num[n - 1] < upper) {
            ans.push_back({num[n - 1] + 1, upper});
        }
        return ans;
    }
};

```

Java

Java

```

class Solution {
    public List<List<Integer>> findMissingRanges(int[] num, int lower, int upper) {
        if (num.length == 0)
            return List.of(getRange(lower, upper));

        List<List<Integer>> ans = new ArrayList<>();

        if (num[0] > lower)
            ans.add(getRange(lower, num[0] - 1));

        for (int i = 1; i < num.length; ++i)
            if (num[i] > num[i - 1] + 1)
                ans.add(getRange(num[i - 1] + 1, num[i] - 1));

        if (num[num.length - 1] < upper)
            ans.add(getRange(num[num.length - 1] + 1, upper));

        return ans;
    }

    private List<Integer> getRange(int lo, int hi) {
        if (lo == hi)
            return List.of(lo, hi);
        return List.of(lo, hi);
    }
}

```

Java

```

class Solution {
    public List<List<Integer>> findMissingRanges(int[] num, int lower, int upper) {
        int n = num.length;
        if (n == 0) {
            return List.of(List.of(lower, upper));
        }
        List<List<Integer>> ans = new ArrayList<>();
        if (num[0] > lower) {
            ans.add(List.of(lower, num[0] - 1));
        }
        for (int i = 1; i < n; ++i) {
            if (num[i] - num[i - 1] > 2) {
                ans.add(List.of(num[i - 1] + 1, num[i] - 1));
            }
        }
        if (num[n - 1] < upper) {
            ans.add(List.of(num[n - 1] + 1, upper));
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public List<List<Integer>> findMissingRanges(int[] num, int lower, int upper) {
        int n = num.length;
        if (n == 0) {
            return List.of(List.of(lower, upper));
        }
        List<List<Integer>> ans = new ArrayList<>();
        if (num[0] > lower) {
            ans.add(List.of(lower, num[0] - 1));
        }
        for (int i = 1; i < n; ++i) {
            if (num[i] - num[i - 1] > 2) {
                ans.add(List.of(num[i - 1] + 1, num[i] - 1));
            }
        }
        if (num[n - 1] < upper) {
            ans.add(List.of(num[n - 1] + 1, upper));
        }
        return ans;
    }
}

```

TypeScript

TypeScript

```

function findMissingRanges(num: number[], lower: number, upper: number): number[][] {
    const n = num.length;
    if (n === 0) {
        return [[lower, upper]];
    }
    const ans: number[][] = [];
    if (num[0] > lower) {
        ans.push([lower, num[0] - 1]);
    }
    for (let i = 1; i < n; ++i) {
        if (num[i] - num[i - 1] > 2) {
            ans.push([num[i - 1] + 1, num[i] - 1]);
        }
    }
    if (num[n - 1] < upper) {
        ans.push([num[n - 1] + 1, upper]);
    }
    return ans;
}

```

TypeScript

```

function findMissingRanges(num: number[], lower: number, upper: number): number[][] {
    const n = num.length;
    if (n === 0) {
        return [[lower, upper]];
    }
    const ans: number[][] = [];
    if (num[0] > lower) {
        ans.push([lower, num[0] - 1]);
    }
    for (let i = 1; i < n; ++i) {
        if (num[i] - num[i - 1] > 2) {
            ans.push([num[i - 1] + 1, num[i] - 1]);
        }
    }
    if (num[n - 1] < upper) {
        ans.push([num[n - 1] + 1, upper]);
    }
    return ans;
}

```

Go

Go

```

func findMissingRanges(num []int, lower int, upper int) ans [][]int {
    n := len(num)
    if n == 0 {
        return [][]int{{lower, upper}}
    }
    if num[0] > lower {
        ans = append(ans, []int{lower, num[0] - 1})
    }
    for i, b := range num[1:] {
        if a := num[i-1]; b-a > 2 {
            ans = append(ans, []int{a + 1, b - 1})
        }
    }
    if num[n-1] < upper {
        ans = append(ans, []int{num[n-1] + 1, upper})
    }
    return ans
}

```

Go

```
func findMissingRanges(nums [int, lower int, upper int] ans []int) {
    n := len(nums)
    if n == 0 {
        return [][]int{ {lower, upper} }
    }
    if nums[0] > lower {
        ans = append(ans, []int{lower, nums[0] - 1})
    }
    for i, b := range nums[1:] {
        if a := nums[i]; b-a > 1 {
            ans = append(ans, []int{a + 1, b - 1})
        }
    }
    if nums[len-1] < upper {
        ans = append(ans, []int{nums[len-1] + 1, upper})
    }
    return
}
```

[*] Arrays & Hashing — Pattern Solution (stored optimal)

Python

```
# Python solution for Missing Ranges problem

def findMissingRanges(nums, lower, upper):
    missing_ranges = [] # List to store the missing range intervals
    prev = lower - 1 # Initialize previous value to one less than lower

    # Append a sentinel at the end for easier gap handling
    nums.append(upper + 1)

    # Loop through each number, including the sentinel
    for num in nums:
        # If there is a gap between previous and current number
        if num - prev == 2:
            # If num is not single number, add that number as a range
            if num - prev == 2:
                missing_ranges.append([prev + 1, prev + 1])
            else:
                missing_ranges.append([prev + 1, num - 1])
            prev = num # Update previous to current
    return missing_ranges

# Example usage:
print(findMissingRanges([0,1,3,50,75], 0, 99))
```

C++

```
// C++ solution for Missing Ranges problem using STL vector.
// Compile using: g++ solution.cpp -std=c++11
```

#include <vector>
using namespace std;

```
vector<vector<int>> findMissingRanges(vector<int>& nums, int lower, int upper) {
    vector<vector<int>> missingRanges; // To store the result missing ranges
    int prev = lower - 1; // Initialize previous value to one less than lower

    // Iterate through nums with extra iteration for the upper bound sentinel
    for (int i = 0; i <= nums.size(); i++) {
        int curr = (i < nums.size()) ? nums[i] : upper + 1;
        if (curr - prev == 2) {
            int start = prev + 1;
            int end = curr - 1;
            // If both are single number, both start and end are same
            missingRanges.push_back({start, end});
        }
        prev = curr; // Update previous value
    }
    return missingRanges;
}
```

```
int main() {
    vector<int> nums = {0, 1, 3, 50, 75};
    int lower = 0, upper = 99;
    vector<vector<int>> result = findMissingRanges(nums, lower, upper);
    // Print result ranges
    for (auto range : result) {
        cout << "[" << range[0] << ", " << range[1] << " ] ";
    }
    return 0;
}
```

#346

EASY

Moving Average from Data Stream

LeetCode — Simple — Medium — LeetCode — Editorial

Array — Design — Queue — Data Stream

LeetCode Premium 98

Problem:

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the `MovingAverage` class:

- `MovingAverage(int size)` Initializes the object with the size of the window size.

- `double next(int val)` Returns the moving average of the last size values of the stream.

Example 1:

```
Input
["MovingAverage", "next", "next", "next", "next"]
[[3], [1], [10], [3], [5]]

Output
[null, 1.0, 5.5, 4.66667, 6.0]
```

Explanation

```
MovingAverage movingAverage = new MovingAverage(3);
movingAverage.next(1); // return 1.0 = 1 / 3
movingAverage.next(10); // return 5.5 = (1 + 10) / 2
movingAverage.next(3); // return 4.66667 = (1 + 10 + 3) / 3
movingAverage.next(5); // return 6.0 = (10 + 3 + 5) / 3
```

Constraints:

- $1 \leq \text{size} \leq 1000$
- $-105 \leq \text{val} \leq 105$
- At most 104 calls will be made to `next`.

Python

Python

```
# Python solution for Moving Average from Data Stream

from collections import deque # Import deque for efficient queue operations

class MovingAverage:
    def __init__(self, size):
        # Initialize with the window size
        self.size = size # Window number of elements in the window
        self.queue = deque() # Queue to store current window values
        self.current_sum = 0 # Running sum of the current window

    def next(self, val):
        # Add new value to the window
        if len(self.queue) == self.size:
            # If the window is full, remove the oldest element and update the sum
            removed_val = self.queue.popleft() # Remove the front element
            self.current_sum -= removed_val # Subtract its value from the running sum
        # Append the new value to the queue and update the sum
        self.queue.append(val)
        self.current_sum += val

        # Calculate and return the moving average by dividing the running sum by the current window size
        return self.current_sum / len(self.queue)
```

Python

```
class MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.sum = 0
        self.q = collections.deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.size:
            self.sum -= self.q.popleft()
            self.sum += val
            self.q.append(val)
        return self.sum / len(self.q)
```

Python

```
class MovingAverage:
    def __init__(self, size: int):
        self.s = 0
        self.data = [] # size
        self.cnt = 0

    def next(self, val: int) -> float:
        l = self.cnt % len(self.data)
        self.s += val - self.data[l]
        self.data[l] = val
        self.cnt += 1
        return self.s / min(self.cnt, len(self.data))
```

Your MovingAverage object will be instantiated and called as such:

obj = MovingAverage(size)

param_1 = obj.next(val)

Python

```
class MovingAverage:
    def __init__(self, size: int):
        self.s = 0
        self.size = size
        self.q = deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.size:
            self.s -= self.q.popleft()
            self.s += val
            self.q.append(val)
        return self.s / len(self.q)
```

Your MovingAverage object will be instantiated and called as such:

obj = MovingAverage(size)

param_1 = obj.next(val)

Python

```
class MovingAverage:
    def __init__(self, size: int):
        self.cnt = 0 # size
        self.s = 0
        self.cnt = 0

    def next(self, val: int) -> float:
        l = self.cnt % len(self.arr) # circular array
        self.s += val - self.arr[l]
        self.arr[l] = val
        self.cnt += 1
        return self.s / min(self.cnt, len(self.arr))
```

Your MovingAverage object will be instantiated and called as such:

obj = MovingAverage(size)

param_1 = obj.next(val)

from collections import deque

With no sum variable, calculate on the fly

class MovingAverage:

def __init__(self, size: int):

```
self.queue = deque()
self.size = size
self.avg = 0
```

```
def next(self, val: int) -> float:
    if len(self.queue) == self.size:
        self.queue.append(val)
        self.avg = sum(self.queue) / len(self.queue)
        return self.avg
    else:
```

```
        head = self.queue.popleft()
        self.queue.append(val)
        minus = head * self.size
        add = val * self.size
        self.avg = self.avg + add - minus
        return self.avg
```

from collections import deque

class MovingAverage(object):

def __init__(self, size):

self.queue = deque()

self.size = size

self.avg = 0

def next(self, val: int) -> float:

if len(self.queue) == self.size:

self.queue.append(val)

self.avg = sum(self.queue) / len(self.queue)

return self.avg

else:

head = self.queue.popleft()

self.queue.append(val)

minus = head * self.size

add = val * self.size

self.avg = self.avg + add - minus

return self.avg

```
Initialize your data structure here.
type size: int

self.windowSize = size
self.windowSum = 0.0
self.data = deque()
```

def next(self, val: int) -> float:

type val: int

type res: float

self.windowSum += val

data = self.data

leftTop = 0

if len(data) == self.windowSize:

leftTop = data.popleft()

data.append(val)

self.windowSum -= leftTop

if len(data) < self.windowSize:

return (self.windowSum / len(data))

return self.windowSum / self.windowSize

Your MovingAverage object will be instantiated and called as such:

obj = MovingAverage(size)

param_1 = obj.next(val)

C++

C++

```
class MovingAverage {
public:
    MovingAverage(int size) {
        data.resize(size);
    }

    double next(int val) {
        int i = cnt % data.size();
        s += val - data[i];
        data[i] = val;
        cnt++;
        return s * 1.0 / min(cnt, (int) data.size());
    }

private:
    int s = 0;
    int cnt = 0;
    vector<int> data;
};
```

/*
 * Your MovingAverage object will be instantiated and called as such:
 * MovingAverage obj = new MovingAverage(size);
 * double param_1 = obj.next(val);
 */

C++

```

class MovingAverage {
public:
    MovingAverage(int size) {
        arr.resize(size);
    }

    double next(int val) {
        int idx = cnt % arr.size();
        s += val - arr[idx];
        arr[idx] = val;
        ++cnt;
        return (double) s / min(cnt, (int) arr.size());
    }
private:
    vector<int> arr;
    int cnt = 0;
    int s = 0;
};

/*
 * Your MovingAverage object will be instantiated and called as such:
 * MovingAverage obj = new MovingAverage(size);
 * double param_1 = obj.next(val);
 */

```

Java

```

class MovingAverage {
public:
    MovingAverage(int size) {
        this.size = size;
    }

    public double next(int val) {
        if (q.size() == size)
            sum -= q.poll();
        sum += val;
        q.offer(val);
        return sum / q.size();
    }

private:
    int size = 0;
    private double sum = 0;
    private Queue<Integer> q = new ArrayDeque<>();
}

```

Java

```

class MovingAverage {
private int s;
private int cnt;
private int[] data;

public MovingAverage(int size) {
    data = new int[size];
}

public double next(int val) {
    int i = cnt % data.length;
    s += val - data[i];
    data[i] = val;
    ++cnt;
    return s * 1.0 / Math.min(cnt, data.length);
}
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * MovingAverage obj = new MovingAverage(size);
 * double param_1 = obj.next(val);
 */

```

Java

```

class MovingAverage {
private Deque<Integer> q = new ArrayDeque<>();
private int s;
private int i;

public MovingAverage(int size) {
    n = size;
}

public double next(int val) {
    if (q.size() == n) {
        s -= q.pollFirst();
    }
    q.offer(val);
    s += val;
    return s * 1.0 / q.size();
}
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * MovingAverage obj = new MovingAverage(size);
 * double param_1 = obj.next(val);
 */

```

Java

```

class MovingAverage {
private int[] arr;
private int s;
private int cnt;

public MovingAverage(int size) {
    arr = new int[size];
}

public double next(int val) {
    int idx = cnt % arr.length;
    s += val - arr[idx];
    arr[idx] = val;
    ++cnt;
    return s * 1.0 / Math.min(cnt, arr.length);
}
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * MovingAverage obj = new MovingAverage(size);
 * double param_1 = obj.next(val);
 */

```

TypeScript

```

class MovingAverage {
private s: number = 0;
private cnt: number = 0;
private data: number[];

constructor(size: number) {
    this.data = Array(size).fill(0);
}

next(val: number): number {
    const i = this.cnt % this.data.length;
    this.s += val - this.data[i];
    this.data[i] = val;
    ++this.cnt;
    return this.s / Math.min(this.cnt, this.data.length);
}
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * var obj = new MovingAverage(size);
 * var param_1 = obj.next(val);
 */

```

TypeScript

```

class MovingAverage {
private q: number[] = [];
private s: number = 0;
private n: number;

constructor(size: number) {
    this.n = size;
}

next(val: number): number {
    if (this.q.length === this.n) {
        this.s -= this.q.shift();
    }
    this.q.push(val);
    this.s += val;
    return this.s / this.q.length;
}
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * var obj = new MovingAverage(size);
 * var param_1 = obj.next(val);
 */

```

TypeScript

```

class MovingAverage {
private s: number = 0;
private cnt: number = 0;
private data: number[];

constructor(size: number) {
    this.data = Array(size).fill(0);
}

next(val: number): number {
    const i = this.cnt % this.data.length;
    this.s += val - this.data[i];
    this.data[i] = val;
    ++this.cnt;
    return this.s / Math.min(this.cnt, this.data.length);
}
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * var obj = new MovingAverage(size);
 * var param_1 = obj.next(val);
 */

```

Go

Go

```

type MovingAverage struct {
    s int
    cnt int
    data []int
}

func Constructor(size int) MovingAverage {
    return MovingAverage{
        data: make([]int, size),
    }
}

func (this *MovingAverage) Next(val int) float64 {
    i := this.cnt % len(this.data)
    this.s += val - this.data[i]
    this.data[i] = val
    ++this.cnt
    return float64(this.s) / float64(min(this.cnt, len(this.data)))
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * obj := Constructor(size);
 * param_1 := obj.Next(val);
 */

```

Go

```

type MovingAverage struct {
    q []int
    s int
    n int
}

func Constructor(size int) MovingAverage {
    return MovingAverage{size}
}

func (this *MovingAverage) Next(val int) float64 {
    if len(this.q) == this.n {
        this.s -= this.q[0]
        this.s = this.q[len]
    }
    this.q = append(this.q, val)
    this.s += val
    return float64(this.s) / float64(len(this.q))
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * obj := Constructor(size);
 * param_1 := obj.Next(val);
 */

```

Go

```

type MovingAverage struct {
    arr []int
    cnt int
    s int
}

func Constructor(size int) MovingAverage {
    arr := make([]int, size)
    return MovingAverage{arr, 0, 0}
}

func (this *MovingAverage) Next(val int) float64 {
    size := this.cnt % len(this.arr)
    this.s += val - this.arr[size]
    this.arr[size] = val
    ++this.cnt
    return float64(this.s) / float64(min(this.cnt, len(this.arr)))
}

/*
 * Your MovingAverage object will be instantiated and called as such:
 * obj := Constructor(size);
 * param_1 := obj.Next(val);
 */

```

[I] Arrays & Hashing - Pattern Solution (stored optimal)

Python

Python solution for Moving Average from Data Stream

from collections import deque # Import deque for efficient queue operations

```

class MovingAverage:
    # Initialization with the window size
    self.size = size # Store the number of elements in the window
    self.queue = deque() # Queue to store current window values
    self.current_sum = 0 # Running sum of the current window

    def next(self, val):
        # Add new value to the window
        if len(self.queue) == self.size:
            # If the queue is full, remove the oldest element and update the sum
            removed_val = self.queue.popleft() # Remove the front element
            self.current_sum -= removed_val # Subtract its value from the running sum
        # Append the new value to the queue and update the sum
        self.queue.append(val)
        self.current_sum += val

        # Calculate and return the moving average by dividing the running sum by the current window size
        return self.current_sum / len(self.queue)

```

C++

```
// C++ solution for Moving Average from Data Stream
#include <iostream>
using namespace std;

class MovingAverage {
private:
    int windowSize; // Maximum number of elements in the window
    double sum; // Running sum of the current window
    queue<int> window; // Queue to store current window elements

public:
    // Constructor initializes the moving average with a specified window size
    MovingAverage(int size) : windowSize(size), sum(0) {}

    // Function to add a new value and return the moving average
    double next(int val) {
        // If the window is full, remove the oldest element and adjust the sum
        if (window.size() == windowSize) {
            sum -= window.front(); // Subtract the value of the oldest element
            window.pop(); // Remove the oldest element from the queue
        }

        // Add the new value to the queue and update the running sum
        window.push(val);
        sum += val;

        // Return the computed moving average
        return sum / window.size();
    }
};
```

#760

EASY

Find Anagram Mappings

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Hash Table

Lists Premium 98

Problem:

You are given two integer arrays `nums1` and `nums2` where `nums2` is an anagram of `nums1`. Both arrays may contain duplicates.

Return an index mapping array `mapping` from `nums1` to `nums2` where `mapping[i] = j` means the `i`th element in `nums1` appears in `nums2` at index `j`. If there are multiple answers, return any of them.

An array `a` is an anagram of an array `b` means `b` is made by randomizing the order of the elements in `a`.

Example 1:

Input: `nums1 = [12,28,46,32,58]`, `nums2 = [58,12,32,46,28]`Output: `[1,4,3,2,0]`

Explanation: As `mapping[0] = 1` because the 0th element of `nums1` appears at `nums2[1]`, and `mapping[1] = 4` because the 1st element of `nums1` appears at `nums2[4]`, and so on.

Arrays & Hashing · LeetCode · By Pattern

— 973 —

LeetCode

```
class Solution {
public:
    def anagramMappings(int[] nums1, List<int> nums2) -> List<int> {
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums2):
            numToIndices[num].append(i)

        return [numToIndices[num].pop() for num in nums1]
    }
}
```

Python

```
class Solution:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        d = {}
        for i, x in enumerate(nums2):
            d[x] = i

        return [d[x] for x in nums1]
```

Python

```
class Solution:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        mapper = defaultdict(int)
        for i, num in enumerate(nums2):
            mapper[num].add(i)

        return [mapper[num].pop() for num in nums1]
```

C++

C++

```
class Solution {
public:
    vector<int> anagramMappings(vector<int>& nums1, vector<int>& nums2) {
        vector<int> ans;
        unordered_map<int, stack<int>> numToIndices;

        for (int i = 0; i < nums2.size(); ++i)
            numToIndices[nums2[i]].push(i);

        for (const int num : nums1)
            ans.push_back(numToIndices[num].top());
        numToIndices[num].pop();

        return ans;
    }
};
```

C++

Arrays & Hashing · LeetCode · By Pattern

— 975 —

LeetCode

```
class Solution {
public:
    List<int> anagramMappings(List<int> nums1, List<int> nums2) {
        int n = nums1.length;
        Map<Integer, Integer> d = new HashMap<>();
        for (int i = 0; i < n; ++i) {
            d.put(nums2[i], i);
        }
        List<int> ans = new List<>();
        for (int i = 0; i < n; ++i) {
            ans.add(d.get(nums1[i]));
        }
        return ans;
    }
}
```

Java

```
class Solution {
public:
    List<int> anagramMappings(List<int> nums1, List<int> nums2) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums2.length; ++i) {
            map.computeIfAbsent(nums2[i], k -> new HashSet<>()).add(i);
        }
        List<Integer> res = new ArrayList<>();
        for (int i = 0; i < nums1.length; ++i) {
            int idx = map.get(nums1[i]).iterator().next();
            res.add(idx);
            map.get(nums1[i]).remove(idx);
        }
        return res;
    }
}
```

TypeScript

TypeScript

```
function anagramMappings(nums1: number[], nums2: number[]): number[] {
    const d: Map<number, number> = new Map();
    for (let i = 0; i < nums2.length; ++i) {
        d.set(nums2[i], i);
    }
    return nums1.map(num => d.get(num));
}
```

TypeScript

```
function anagramMappings(nums1: number[], nums2: number[]): number[] {
    const d: Map<number, number> = new Map();
    for (let i = 0; i < nums2.length; ++i) {
        d.set(nums2[i], i);
    }
    return nums1.map(num => d.get(num));
}
```

Go

Go

Arrays & Hashing · LeetCode · By Pattern

— 977 —

LeetCode

Example 2:

Input: `nums1 = [84,46]`, `nums2 = [84,46]`Output: `[0,1]`

Constraints:

- `1 <= nums1.length <= 100`
- `nums2.length == nums1.length`
- `0 <= nums1[i], nums2[i] <= 105`
- `nums2` is an anagram of `nums1`.

>> Brute Force [Arrays & Hashing] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def anagramMappings(self, nums1, nums2):
        # Brute Force O(n^2) - check all pairs without a hash map
        n = len(nums1)
        for i in range(n):
            for j in range(i + 1, n):
                # Check if pair (nums1[i], nums1[j])
                pass
            return i # return value
```

Python

Python

```
# Build a dictionary mapping each number in nums2 to a list of its indices.
def anagramMappings(nums1, nums2):
    # Dictionary to store number - list of indices
    index_dict = {}
    for index, num in enumerate(nums2):
        if num in index_dict:
            index_dict[num].append(index)
        else:
            index_dict[num] = [index]

    mapping = []
    # Iterate the mapping for each element in nums1.
    for num in nums1:
        # For the first duplicate value for num and add it to mapping.
        mapping.append(index_dict[num].pop(0))
    return mapping

# Example output:
nums1 = [12,28,46,32,58]
nums2 = [58,12,32,46,28]
print(anagramMappings(nums1, nums2)) # Output: [1,4,3,2,0] or any valid mapping
```

Python

Arrays & Hashing · LeetCode · By Pattern

— 974 —

LeetCode

```
class Solution {
public:
    vector<int> anagramMappings(vector<int>& nums1, vector<int>& nums2) {
        int n = nums1.size();
        unordered_map<int, int> d;
        for (int i = 0; i < n; ++i) {
            d[nums2[i]] = i;
        }
        vector<int> ans;
        for (int i = 0; i < n; ++i) {
            ans.push_back(d[nums1[i]]);
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    vector<int> anagramMappings(vector<int>& nums1, vector<int>& nums2) {
        int n = nums1.size();
        unordered_map<int, int> d;
        for (int i = 0; i < n; ++i) {
            d[nums2[i]] = i;
        }
        vector<int> ans;
        for (int i = 0; i < n; ++i) {
            ans.push_back(d[nums1[i]]);
        }
        return ans;
    }
};
```

Java

Java

```
class Solution {
    public List<Integer> anagramMappings(List<Integer> nums1, List<Integer> nums2) {
        List<Integer> ans = new ArrayList<>();
        Map<Integer, Queue<Integer>> numToIndices = new HashMap<>();

        for (int i = 0; i < nums2.length; ++i) {
            numToIndices.putIfAbsent(nums2[i], new ArrayDeque<>());
            numToIndices.get(nums2[i]).push(i);
        }

        for (int i = 0; i < nums1.length; ++i) {
            ans.add(numToIndices.get(nums1[i]).pop());
        }
        return ans;
    }
}
```

Java

Arrays & Hashing · LeetCode · By Pattern

— 976 —

LeetCode

```
class Solution {
public:
    List<int> anagramMappings(List<int> nums1, List<int> nums2) {
        int n = nums1.length;
        Map<Integer, Integer> d = new HashMap<>();
        for (int i = 0; i < n; ++i) {
            d.put(nums2[i], i);
        }
        List<int> ans = new List<>();
        for (int i = 0; i < n; ++i) {
            ans.add(d.get(nums1[i]));
        }
        return ans;
    }
}
```

Java

```
class Solution {
public:
    List<int> anagramMappings(List<int> nums1, List<int> nums2) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums2.length; ++i) {
            map.computeIfAbsent(nums2[i], k -> new HashSet<>()).add(i);
        }
        List<Integer> res = new ArrayList<>();
        for (int i = 0; i < nums1.length; ++i) {
            int idx = map.get(nums1[i]).iterator().next();
            res.add(idx);
            map.get(nums1[i]).remove(idx);
        }
        return res;
    }
}
```

TypeScript

TypeScript

```
function anagramMappings(nums1: number[], nums2: number[]): number[] {
    const d: Map<number, number> = new Map();
    for (let i = 0; i < nums2.length; ++i) {
        d.set(nums2[i], i);
    }
    return nums1.map(num => d.get(num));
}
```

TypeScript

```
function anagramMappings(nums1: number[], nums2: number[]): number[] {
    const d: Map<number, number> = new Map();
    for (let i = 0; i < nums2.length; ++i) {
        d.set(nums2[i], i);
    }
    return nums1.map(num => d.get(num));
}
```

Go

Go

Arrays & Hashing · LeetCode · By Pattern

— 977 —

LeetCode

```
func anagramMappings(nums1 []int, nums2 []int) []int {
    d := map[int]int{}
    for i, x := range nums2 {
        d[x] = i
    }
    ans := make([]int, len(nums1))
    for i, x := range nums1 {
        ans[i] = d[x]
    }
    return ans
}
```

Go

```
func anagramMappings(nums1 []int, nums2 []int) []int {
    d := map[int]int{}
    for i, x := range nums2 {
        d[x] = i
    }
    ans := make([]int, len(nums1))
    for i, x := range nums1 {
        ans[i] = d[x]
    }
    return ans
}
```

[*] Arrays & Hashing — Pattern Solution (matched from community answers)

Python

```
class Solution:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums2):
            numToIndices[num].append(i)

        return [numToIndices[num].pop() for num in nums1]
```

#1426

EASY

Counting Elements

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Hash Table

Lists Premium 98

Problem:

Given an integer array `arr`, count how many elements `x` there are, such that `x + 1` is also in `arr`. If there are duplicates in `arr`, count them separately.

Example 1:

Input: `arr = [1,2,3]`

Output: 2

Explanation: 1 and 2 are counted cause 2 and 3 are in `arr`.

Example 2:

Arrays & Hashing · LeetCode · By Pattern

— 978 —

LeetCode

```
Input: arr = [1,1,3,3,5,5,7,7]
Output: 0
Explanation: No numbers are counted, cause there is no 2, 4, 6, or 8 in arr.

Constraints:
• 1 <= arr.length <= 1000
• 0 <= arr[i] <= 1000
```

>> Brute Force [Arrays & Hashing] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def countElements(self, arr):
        # Brute Force O(n^2) - check all pairs without a hash map
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                # if arr[i] == arr[j], arr[i]
                pass
        return 0 # return value
```

Python

```
# Function to count elements where (x + 1) is in the array
def countElements(arr):
    # Create a set of unique numbers for O(1) lookups
    unique_numbers = set(arr)
    count = 0
    # Iterate through each number in the array
    for number in arr:
        # If (number + 1) exists in the unique set, increment count
        if number + 1 in unique_numbers:
            count += 1
    return count

# Example usage:
# arr = [1, 2, 3]
# print(countElements(arr)) # Expected output: 2
```

Python

```
class Solution:
    def countElements(self, arr: List[int]) -> int:
        count = collections.Counter(arr)
        return sum(freq
                    for a, freq in count.items()
                    if count[a + 1] > 0)
```

Python

```
class Solution:
    def countElements(self, arr: List[int]) -> int:
        cnt = Counter(arr)
        return sum(1 for x, v in cnt.items() if cnt[x + 1])
```

Python

```
class Solution {
    def countElements(self, arr: List[int]) -> int:
        cnt = Counter(arr)
        return sum(1 for x, v in cnt.items() if cnt[x + 1])
}
```

C++

C++

```
class Solution {
public:
    int countElements(vector<int>& arr) {
        unordered_set<int> arrSet(arr.begin(), arr.end());
        return ranges::count_if(
            arr, [arrSet](int x) { return arrSet.contains(x + 1); });
    }
};
```

C++

```
class Solution {
public:
    int countElements(vector<int>& arr) {
        int cnt[1001];
        for (int x : arr) {
            ++cnt[x];
        }
        int ans = 0;
        for (int x = 0; x < 1000; ++x) {
            if (cnt[x + 1] > 0) {
                ans += cnt[x];
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int countElements(vector<int>& arr) {
        unordered_set<int> arrSet(arr.begin(), arr.end());
        return ranges::count_if(
            arr, [arrSet](int x) { return arrSet.contains(x + 1); });
    }
};
```

C++

```
class Solution {
public:
    int countElements(vector<int>& arr) {
        int cnt[1001];
        for (int x : arr) {
            ++cnt[x];
        }
        int ans = 0;
        for (int x = 0; x < 1000; ++x) {
            if (cnt[x + 1] > 0) {
                ans += cnt[x];
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int countElements(vector<int>& arr) {
        unordered_set<int> arrSet(arr.begin(), arr.end());
        return ranges::count_if(
            arr, [arrSet](int x) { return arrSet.contains(x + 1); });
    }
};
```

Java

```
class Solution {
    public int countElements(int[] arr) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();
        for (final int a : arr)
            count.merge(a, 1, Integer::sum);
        for (Map.Entry<Integer, Integer> entry : count.entrySet())
            if (count.containsKey(entry.getKey() + 1))
                ans += entry.getValue();
        return ans;
    }
}
```

Java

```
class Solution {
    public int countElements(int[] arr) {
        int[] cnt = new int[1001];
        for (int x : arr) {
            ++cnt[x];
        }
        int ans = 0;
        for (int x = 0; x < 1000; ++x) {
            if (cnt[x + 1] > 0) {
                ans += cnt[x];
            }
        }
        return ans;
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn countElements(arr: Vec<i32>) -> i32 {
        let mut cnt = HashMap::new();
        for &num in arr {
            *cnt.entry(num).or_insert(0) += 1;
        }
        cnt.iter()
            .filter(|(&_, &v)| cnt.contains_key(&(v + 1)))
            .map(|(&_, &v)| *v)
            .sum()
    }
}
```

Java

```
class Solution {
    public int countElements(int[] arr) {
        int[] cnt = new int[1001];
        for (int x : arr) {
            ++cnt[x];
        }
        int ans = 0;
        for (int x = 0; x < 1000; ++x) {
            if (cnt[x + 1] > 0) {
                ans += cnt[x];
            }
        }
        return ans;
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn countElements(arr: Vec<i32>) -> i32 {
        let mut cnt = HashMap::new();
        for &num in arr {
            *cnt.entry(num).or_insert(0) += 1;
        }
        cnt.iter()
            .filter(|(&_, &v)| cnt.contains_key(&(v + 1)))
            .map(|(&_, &v)| *v)
            .sum()
    }
}
```

JavaScript

JavaScript

```
// @param {number[]} arr
// @return {number}
var countElements = function(arr) {
    const m = Math.max(...arr);
    const cnt = Array(m + 1).fill(0);
    for (const x of arr) {
        ++cnt[x];
    }
    let ans = 0;
    for (let i = 0; i < m; ++i) {
        if (cnt[i + 1] > 0) {
            ans += cnt[i];
        }
    }
    return ans;
};
```

JavaScript

```
// @param {number[]} arr
// @return {number}
var countElements = function(arr) {
    const m = Math.max(...arr);
    const cnt = Array(m + 1).fill(0);
    for (const x of arr) {
        ++cnt[x];
    }
    let ans = 0;
    for (let i = 0; i < m; ++i) {
        if (cnt[i + 1] > 0) {
            ans += cnt[i];
        }
    }
    return ans;
};
```

```
TypeScript
TypeScript
function countElements(arr: number[]): number {
    const m = Math.max(...arr);
    const cnt = Array(m + 1).fill(0);
    for (const x of arr) {
        ++cnt[x];
    }
    let ans = 0;
    for (let i = 0; i < m; ++i) {
        if (cnt[i + 1] > 0) {
            ans += cnt[i];
        }
    }
    return ans;
}
```

TypeScript

```
function countElements(arr: number[]): number {
    const m = Math.max(...arr);
    const cnt = Array(m + 1).fill(0);
    for (const x of arr) {
        ++cnt[x];
    }
    let ans = 0;
    for (let i = 0; i < m; ++i) {
        if (cnt[i + 1] > 0) {
            ans += cnt[i];
        }
    }
    return ans;
}
```

Go

Go

```
func countElements(arr []int) (ans int) {
    m := slices.Max(arr)
    cnt := make([]int, m+1)
    for _, x := range arr {
        cnt[x]++
    }
    for i := 0; i < m; i++ {
        if cnt[i+1] > 0 {
            ans += cnt[i]
        }
    }
    return
}
```

Go


```
func countTriplets(arr: [Int]) Int {
    m = ArrayList()
    cnt = HashMap()
    for i in range arr {
        cnt[i] = 0
    }
    for a in arr {
        for b in arr {
            if cnt[a] > 0 {
                cnt[b] += 1
            }
        }
    }
    return cnt
}
```

[*] Arrays & Hashing — Pattern Solution (matched from community answers)

```
Python
class Solution:
    def countTriplets(self, arr: List[int]) -> int:
        count = collections.Counter(arr)
        return sum(
            for a, freq in count.items()
            if count[a + 1] > 0
        )
```

#1534

EASY

Count Good Triplets

LeetCode · SimplifyLeetCode · Hash · LeetCode · Editorial

Array · Enumeration

List · CodeSignal

Problem: Given an array of integers arr, and three integers a, b, and c. You need to find the number of good triplets.

A triplet (arr[i], arr[j], arr[k]) is good if the following conditions are true:

- $0 \leq i < j < k < \text{arr.length}$
- $|arr[i] - arr[j]| \leq a$
- $|arr[j] - arr[k]| \leq b$
- $|arr[i] - arr[k]| \leq c$

Where $|x|$ denotes the absolute value of x.

Return the number of good triplets.

Example 1:

Input: arr = [3,0,1,1,9,7], a = 7, b = 2, c = 3

Output: 4

Explanation: There are 4 good triplets: [(3,0,1), (3,0,1), (3,1,1), (0,1,1)].

Example 2:

Arrays & Hashing · LeetCode · By Pattern — 985 — LeetCode

```
Python
class Solution:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        ans, n = 0, len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if abs(arr[i] - arr[j]) <= a and abs(arr[j] - arr[k]) <= b and abs(arr[i] - arr[k]) <= c:
                        ans += 1
        return ans
```

```
Python
class Solution:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        ans, n = 0, len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if abs(arr[i] - arr[j]) <= a and abs(arr[j] - arr[k]) <= b and abs(arr[i] - arr[k]) <= c:
                        ans += 1
        return ans
```

```
C++
class Solution {
public:
    int countGoodTriplets(vector<int>& arr, int a, int b, int c) {
        int ans = 0;
        for (int i = 0; i < arr.size(); ++i)
            for (int j = i + 1; j < arr.size(); ++j)
                for (int k = j + 1; k < arr.size(); ++k)
                    if (abs(arr[i] - arr[j]) <= a && abs(arr[j] - arr[k]) <= b && abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
};
```

C++

Arrays & Hashing · LeetCode · By Pattern — 987 — LeetCode

```
class Solution {
    public int countGoodTriplets(int[] arr, int a, int b, int c) {
        int n = arr.length;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                for (int k = j + 1; k < n; ++k)
                    if (Math.abs(arr[i] - arr[j]) <= a && Math.abs(arr[j] - arr[k]) <= b && Math.abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
}
```

```
Java
public class Solution {
    public int CountGoodTriplets(int[] arr, int a, int b, int c) {
        int n = arr.length;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                for (int k = j + 1; k < n; ++k)
                    if (Math.abs(arr[i] - arr[j]) <= a && Math.abs(arr[j] - arr[k]) <= b && Math.abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
}
```

```
Java
class Solution {
    public int countGoodTriplets(int[] arr, int a, int b, int c) {
        int n = arr.length;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                for (int k = j + 1; k < n; ++k)
                    if (Math.abs(arr[i] - arr[j]) <= a && Math.abs(arr[j] - arr[k]) <= b && Math.abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
}
```

Arrays & Hashing · LeetCode · By Pattern — 989 — LeetCode

Input: arr = [1,1,2,2,3], a = 0, b = 0, c = 1

Output: 0

Explanation: No triplet satisfies all conditions.

Constraints:

- $3 \leq \text{arr.length} \leq 100$
- $0 \leq \text{arr}[i] \leq 1000$
- $0 \leq a, b, c \leq 1000$

>> Brute Force [Arrays & Hashing] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        # Brute Force Approach: Check all triplets without a hash map
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if abs(arr[i] - arr[j]) <= a and abs(arr[j] - arr[k]) <= b and abs(arr[i] - arr[k]) <= c:
                        ans += 1
        return ans
```

```
Python
# Python solution for Count Good Triplets
def countGoodTriplets(arr, a, b, c):
    # Initialize count of good triplets
    good_triplets_count = 0
    n = len(arr)
    # Iterate over all triplet combinations with indices i, j, k where i < j < k
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                # Check if the triplet satisfies the conditions
                if abs(arr[i] - arr[j]) <= a and abs(arr[j] - arr[k]) <= b and abs(arr[i] - arr[k]) <= c:
                    good_triplets_count += 1
    # Return the count of good triplets
    return good_triplets_count
```

```
# Example usage:
arr = [1, 1, 2, 2, 3]
a, b, c = 0, 0, 1
print(countGoodTriplets(arr, a, b, c)) # Expected output: 4
```

Arrays & Hashing · LeetCode · By Pattern — 986 — LeetCode

```
class Solution {
public:
    int countGoodTriplets(vector<int>& arr, int a, int b, int c) {
        int n = arr.size();
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                for (int k = j + 1; k < n; ++k)
                    if (abs(arr[i] - arr[j]) <= a && abs(arr[j] - arr[k]) <= b && abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
};
```

```
C++
class Solution {
public:
    int countGoodTriplets(vector<int>& arr, int a, int b, int c) {
        int n = arr.size();
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                for (int k = j + 1; k < n; ++k)
                    if (abs(arr[i] - arr[j]) <= a && abs(arr[j] - arr[k]) <= b && abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
};
```

```
Java
class Solution {
    public int countGoodTriplets(int[] arr, int a, int b, int c) {
        int n = arr.length;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                for (int k = j + 1; k < n; ++k)
                    if (Math.abs(arr[i] - arr[j]) <= a && Math.abs(arr[j] - arr[k]) <= b && Math.abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
}
```

Java

Arrays & Hashing · LeetCode · By Pattern — 988 — LeetCode

```
Java
public class Solution {
    public int CountGoodTriplets(int[] arr, int a, int b, int c) {
        int n = arr.length;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j)
                for (int k = j + 1; k < n; ++k)
                    if (Math.abs(arr[i] - arr[j]) <= a && Math.abs(arr[j] - arr[k]) <= b && Math.abs(arr[i] - arr[k]) <= c)
                        ++ans;
        return ans;
    }
}
```

```
TypeScript
function countGoodTriplets(arr: number[], a: number, b: number, c: number): number {
    let n = arr.length;
    let ans = 0;
    for (let i = 0; i < n; ++i)
        for (let j = i + 1; j < n; ++j)
            for (let k = j + 1; k < n; ++k)
                if (Math.abs(arr[i] - arr[j]) <= a && Math.abs(arr[j] - arr[k]) <= b && Math.abs(arr[i] - arr[k]) <= c)
                    ++ans;
    return ans;
}
```

TypeScript

Arrays & Hashing · LeetCode · By Pattern — 990 — LeetCode

```

function countGoodTriplets(arr: number[], a: number, b: number, c: number): number {
    let n = arr.length;
    let ans = 0;
    for (let i = 0; i < n; ++i) {
        for (let j = i + 1; j < n; ++j) {
            for (let k = j + 1; k < n; ++k) {
                if (
                    Math.abs(arr[i] - arr[j]) <= a &&
                    Math.abs(arr[j] - arr[k]) <= b &&
                    Math.abs(arr[i] - arr[k]) <= c
                ) {
                    ++ans;
                }
            }
        }
    }
    return ans;
}

```

Go

```

func countGoodTriplets(arr []int, a int, b int, c int) (ans int) {
    n := len(arr)
    for i := 0; i < n; i++ {
        for j := i + 1; j < n; j++ {
            for k := j + 1; k < n; k++ {
                if abs(arr[i]-arr[j]) <= a && abs(arr[j]-arr[k]) <= b && abs(arr[i]-arr[k]) <= c {
                    ans++
                }
            }
        }
    }
    return
}

func abs(a int) int {
    if a < 0 {
        return -a
    }
    return a
}

```

Go

```

func countGoodTriplets(arr []int, a int, b int, c int) (ans int) {
    n := len(arr)
    for i := 0; i < n; i++ {
        for j := i + 1; j < n; j++ {
            for k := j + 1; k < n; k++ {
                if abs(arr[i]-arr[j]) <= a && abs(arr[j]-arr[k]) <= b && abs(arr[i]-arr[k]) <= c {
                    ans++
                }
            }
        }
    }
    return
}

func abs(a int) int {
    if a < 0 {
        return -a
    }
    return a
}

```

Rust

```

impl Solution {
    pub fn count_good_triplets(arr: Vec<i32>, a: i32, b: i32, c: i32) -> i32 {
        let n = arr.len();
        let mut ans = 0;

        for i in 0..n {
            for j in i + 1..n {
                for k in j + 1..n {
                    if (arr[i] - arr[j]).abs() <= a && (arr[j] - arr[k]).abs() <= b && (arr[i] - arr[k]).abs() <= c {
                        ans += 1;
                    }
                }
            }
        }
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn count_good_triplets(arr: Vec<i32>, a: i32, b: i32, c: i32) -> i32 {
        let n = arr.len();
        let mut ans = 0;

        for i in 0..n {
            for j in i + 1..n {
                for k in j + 1..n {
                    if (arr[i] - arr[j]).abs() <= a
                        && (arr[j] - arr[k]).abs() <= b
                        && (arr[i] - arr[k]).abs() <= c
                    {
                        ans += 1;
                    }
                }
            }
        }
        ans
    }
}

```

[*] Arrays & Hashing — Pattern Solution (stored optimal)

Python

```

# Python solution for Count Good Triplets
def countGoodTriplets(arr, a, b, c):
    # Initialize counter for good triplets
    good_triplets_count = 0
    n = len(arr)
    # Iterate over all triplet combinations with indices i, j, k where i < j < k
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                # Check if absolute differences satisfy the conditions
                if abs(arr[i] - arr[j]) <= a and abs(arr[j] - arr[k]) <= b and abs(arr[i] - arr[k]) <= c:
                    good_triplets_count += 1
    # Return the count of good triplets
    return good_triplets_count

# Example usage:
arr = [3, 0, 1, 1, 9, 7]
a, b, c = 7, 2, 2
print(countGoodTriplets(arr, a, b, c)) # Expected output: 4

```

C++

```

// C++ solution for Count Good Triplets
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

int countGoodTriplets(const vector<int>& arr, int a, int b, int c) {
    int goodTripletsCount = 0;
    int n = arr.size();
    // Iterate over all triplet combinations with indices i, j, k where i < j < k
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            for (int k = j + 1; k < n; k++) {
                // Check if absolute differences satisfy the conditions
                if (abs(arr[i] - arr[j]) <= a && abs(arr[j] - arr[k]) <= b && abs(arr[i] - arr[k]) <= c) {
                    goodTripletsCount++;
                }
            }
        }
    }
    // Return the count of good triplets
    return goodTripletsCount;
}

int main() {
    vector<int> arr = {3, 0, 1, 1, 9, 7};
    int a = 7, b = 2, c = 2;
    // Print the count of good triplets
    cout << countGoodTriplets(arr, a, b, c) << endl; // Expected output: 4
    return 0;
}

```

#57

MEDIUM

Insert Interval

LeetCode · SimplifyLeet · WubCC · LeetCode · Editorial

Array

Lists Grid 169

Problem:

You are given an array of non-overlapping intervals where intervals[i] = [starti, endi] represent the start and the end of the ith interval and intervals is sorted in ascending order by starti. You are also given an interval newInterval = [start, end] that represents the start and end of another interval.

Insert newInterval into intervals such that intervals is still sorted in ascending order by starti and intervals still does not have any overlapping intervals (merge overlapping intervals if necessary). Return intervals after the insertion.

Note that you don't need to modify intervals in-place. You can make a new array and return it. Example 1:

Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
Output: [[1,3],[2,5],[6,9]]

Example 2:

Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
Output: [[1,2],[3,10],[12,16]]
Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].

Constraints:

- 0 <= intervals.length <= 104
- intervals[i].length == 2
- 0 <= starti <= endi <= 105
- intervals is sorted by starti in ascending order.
- newInterval.length == 2
- 0 <= start <= end <= 105

>> Brute Force (Arrays & Hashing) Interview Prep

Python

```

# Brute Force Interview
class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[List[int]]) -> List[List[int]]:
        # Brute Force: Use log n = insert, sort, then merge like merge-intervals
        intervals.append(newInterval)
        intervals.sort(key=lambda x: x[0])
        result = [intervals[0]]
        for i in range(1, len(intervals)):
            if intervals[i][0] <= result[-1][1]:
                result[-1][1] = max(result[-1][1], intervals[i][1])
            else:
                result.append(intervals[i])
        return result

```

Python

```

# Python solution for Insert Interval
class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[List[int]]) -> List[List[int]]:
        # Step 1: Sort the newInterval
        merged_intervals = []
        i = 0
        n = len(intervals)

        # Add all intervals that end before newInterval starts
        while i < n and intervals[i][1] < newInterval[0]:
            merged_intervals.append(intervals[i])
            i += 1

        # Merge overlapping intervals with newInterval
        while i < n and intervals[i][0] <= newInterval[1]:
            # Update the start and end of newInterval to the max of overlapping intervals
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1

        # Add the merged newInterval
        merged_intervals.append(newInterval)

        # Add any remaining intervals that come after newInterval
        while i < n:
            merged_intervals.append(intervals[i])
            i += 1

        return merged_intervals

```

Python

```

class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[List[int]]) -> List[List[int]]:
        n = len(intervals)
        ans = []
        i = 0

        while i < n and intervals[i][1] < newInterval[0]:
            ans.append(intervals[i])
            i += 1

        # Merge overlapping intervals
        while i < n and intervals[i][0] <= newInterval[1]:
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1

        ans.append(newInterval)

        while i < n:
            ans.append(intervals[i])
            i += 1

        return ans

```

Python

```

class Solution {
    def insert(
        self, intervals: List[List[int]], newInterval: List[int]
    ) -> List[List[int]]:
        def mergeIntervals: List[List[int]] -> List[List[int]]:
            intervals.sort()
            ans = [intervals[0]]
            for s, e in intervals[1:]:
                if ans[-1][1] < s:
                    ans.append([s, e])
                else:
                    ans[-1][1] = max(ans[-1][1], e)
            return ans

        intervals.append(newInterval)
        return mergeIntervals()
}

Python
"""
a = [[1,2], [3,5]]
-> [[1]]
2
-> a[-1]
4
"""
class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        start = newInterval[0]
        end = newInterval[1]

        left = list(filter(lambda x: x[1] < start, intervals)) # cut via list()
        right = list(filter(lambda x: x[0] > end, intervals))

        if left + right != intervals:
            start = min(start, intervals[len(left)][0]) # note, left not -1, because index starts at 0
            end = max(end, intervals[len(right)-1][1]) # same, starting at right with len-1
            return left + [[start, end]] + right

        #####
        class Solution: # remove method's name conflict
            def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:

```

```

        intervals.append(newInterval)
        return self.merge(intervals)

    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ans = []
        for interval in sorted(intervals, key=lambda x: x[0]):
            if ans and ans[-1][1] >= interval[0]:
                ans[-1][1] = max(ans[-1][1], interval[1])
            else:
                ans.append(interval)
        return ans

#####

class Solution:
    def insert(
        self, intervals: List[List[int]], newInterval: List[int]
    ) -> List[List[int]]:
        def mergeIntervals: List[List[int]] -> List[List[int]]:
            intervals.sort()
            ans = [intervals[0]]
            for s, e in intervals[1:]:
                if ans[-1][1] < s:
                    ans.append([s, e])
                else:
                    ans[-1][1] = max(ans[-1][1], e)
            return ans

        intervals.append(newInterval)
        return mergeIntervals()
}

C++
C++

```

```

class Solution {
public:
    vector<vector<int>> insert(vector<vector<int>>& intervals,
        vector<int>& newInterval) {
        const int n = intervals.size();
        vector<vector<int>> ans;
        int i = 0;

        while (i < n && intervals[i][1] < newInterval[0])
            ans.push_back(intervals[i++]);

        // Merge overlapping intervals
        while (i < n && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = min(newInterval[0], intervals[i][0]);
            newInterval[1] = max(newInterval[1], intervals[i][1]);
            ++i;
        }
        ans.push_back(newInterval);

        while (i < n)
            ans.push_back(intervals[i++]);

        return ans;
    }
};

C++
class Solution {
public:
    vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>& newInterval) {
        intervals.emplace_back(newInterval);
        return merge(intervals);
    }

    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> ans;
        ans.emplace_back(intervals[0]);
        for (int i = 1; i < intervals.size(); ++i) {
            if (ans.back()[1] < intervals[i][0]) {
                ans.emplace_back(intervals[i]);
            } else {
                ans.back()[1] = max(ans.back()[1], intervals[i][1]);
            }
        }
        return ans;
    }
};

C++

```

```

class Solution {
public:
    vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>& newInterval) {
        intervals.emplace_back(newInterval);
        return merge(intervals);
    }

    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> ans;
        ans.emplace_back(intervals[0]);
        for (int i = 1; i < intervals.size(); ++i) {
            if (ans.back()[1] < intervals[i][0]) {
                ans.emplace_back(intervals[i]);
            } else {
                ans.back()[1] = max(ans.back()[1], intervals[i][1]);
            }
        }
        return ans;
    }
};

Java
Java
class Solution {
    public List<int[]> insert(int[][] intervals, int[] newInterval) {
        if (int n = intervals.length;
            List<int[]> ans = new ArrayList<>();
            int i = 0;

            while (i < n && intervals[i][1] < newInterval[0])
                ans.add(intervals[i++]);

            // Merge overlapping intervals
            while (i < n && intervals[i][0] <= newInterval[1]) {
                newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
                newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
                ++i;
            }
            ans.add(newInterval);

            while (i < n)
                ans.add(intervals[i++]);

            return ans.toArray(List<int[]>::new);
        }
    }
}

Java

```

```

public class Solution {
    public List<int[]> insert(int[][] intervals, int[] newInterval) {
        List<int[]> ans = new ArrayList<>();
        for (int i = 0; i < intervals.length; ++i) {
            newInterval[i] = intervals[i];
        }
        newInterval[intervals.length] = newInterval;
        return merge(newInterval);
    }

    public List<int[]> Merge(int[][] intervals) {
        intervals = intervals.OrderBy(a -> a[0]).ToArray();
        var ans = new List<int[]>();
        ans.Add(intervals[0]);
        for (int i = 1; i < intervals.Length; ++i) {
            if (ans[ans.Count - 1][1] < intervals[i][0]) {
                ans.Add(intervals[i]);
            } else {
                ans[ans.Count - 1][1] = Math.Max(ans[ans.Count - 1][1], intervals[i][1]);
            }
        }
        return ans.ToArray();
    }
}

Java
class Solution {
    public List<int[]> insert(int[][] intervals, int[] newInterval) {
        List<int[]> ans = new ArrayList<>();
        int st = newInterval[0], ed = newInterval[1];
        boolean insert = false;
        for (int[] interval : intervals) {
            int s = interval[0], e = interval[1];
            if (ed < s) {
                if (!insert) {
                    ans.add(new int[] {st, ed});
                    insert = true;
                }
            } else if (s <= ed) {
                ans.add(interval);
            } else {
                st = Math.min(st, s);
                ed = Math.max(ed, e);
            }
        }
        if (!insert) {
            ans.add(new int[] {st, ed});
        }
        return ans.toArray(new int[ans.size()][]);
    }
}

Java

```

```

class Solution {
    public List<int[]> insert(int[][] intervals, int[] newInterval) {
        List<int[]> ans = new ArrayList<>();
        for (int i = 0; i < intervals.length; ++i) {
            newInterval[i] = intervals[i];
        }
        newInterval[intervals.length] = newInterval;
        return merge(newInterval);
    }

    private List<int[]> Merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> ans = new ArrayList<>();
        ans.add(intervals[0]);
        for (int i = 1; i < intervals.length; ++i) {
            int s = intervals[i][0], e = intervals[i][1];
            if (ans.get(ans.size() - 1)[1] < s) {
                ans.add(intervals[i]);
            } else {
                ans.get(ans.size() - 1)[1] = Math.max(ans.get(ans.size() - 1)[1], e);
            }
        }
        return ans.toArray(new int[ans.size()][]);
    }
}

Java
class Solution {
    public List<int[]> insert(int[][] intervals, int[] newInterval) {
        var ans = new List<int[]>();
        int st = newInterval[0], ed = newInterval[1];
        boolean insert = false;
        for (int[] interval : intervals) {
            int s = interval[0], e = interval[1];
            if (ed < s) {
                if (!insert) {
                    ans.Add(new int[] {st, ed});
                    insert = true;
                }
            } else if (s <= ed) {
                ans.Add(interval);
            } else {
                st = Math.Min(st, s);
                ed = Math.Max(ed, e);
            }
        }
        if (!insert) {
            ans.Add(new int[] {st, ed});
        }
        return ans.ToArray();
    }
}

TypeScript
TypeScript

```

```
function insert(intervals: number[][], newInterval: number[]): number[][] {
    const merge = (intervals: number[][], number[][]) => {
        intervals.sort((a, b) => a[0] > b[0]);
        const ans: number[][] = [intervals[0]];
        for (let i = 1; i < intervals.length; i++) {
            if (ans.at(-1)[1] < intervals[i][0]) {
                ans.push(intervals[i]);
            } else {
                ans.at(-1)[1] = Math.max(ans.at(-1)[1], intervals[i][1]);
            }
        }
        return ans;
    };
    intervals.push(newInterval);
    return merge(intervals);
}
```

TypeScript

```
function insert(intervals: number[][], newInterval: number[]): number[][] {
    let [st, ed] = newInterval;
    const ans: number[][] = [];
    let insert = false;
    for (const [s, e] of intervals) {
        if (ed < s) {
            if (!insert) {
                ans.push([st, ed]);
                insert = true;
            }
            ans.push([s, e]);
        } else if (s <= ed) {
            ans.push([s, e]);
        } else {
            st = Math.min(st, s);
            ed = Math.max(ed, e);
        }
    }
    if (!insert) {
        ans.push([st, ed]);
    }
    return ans;
}
```

Go

Go

Arrays & Hashing · LeetCode · By Pattern

— 1003 —

LeetCode

```
func insert(intervals [][]int, newInterval []int) [][]int {
    merge := func(intervals [][]int) (ans [][]int) {
        sort.Slice(intervals, func(i, j) bool {
            return intervals[i][0] < intervals[j][0]
        })
        ans = append(ans, intervals[0])
        for i := 1; i < len(intervals); i++ {
            if ans[len(ans)-1][1] < intervals[i][0] {
                ans = append(ans, intervals[i])
            } else {
                ans[len(ans)-1][1] = max(ans[len(ans)-1][1], intervals[i][1])
            }
        }
        return ans
    }
    intervals = append(intervals, newInterval)
    return merge(intervals)
}
```

Go

```
func insert(intervals [][]int, newInterval []int) (ans [][]int) {
    st, ed := newInterval[0], newInterval[1]
    insert := false
    for i, interval := range intervals {
        s, e := interval[0], interval[1]
        if s < st {
            if !insert {
                ans = append(ans, [][2]int{st, ed})
                insert = true
            }
            ans = append(ans, interval)
        } else if s <= st {
            ans = append(ans, interval)
        } else {
            st = min(st, s)
            ed = max(ed, e)
        }
    }
    if !insert {
        ans = append(ans, [][2]int{st, ed})
    }
    return ans
}
```

Go

Arrays & Hashing · LeetCode · By Pattern

— 1004 —

LeetCode

```
func insert(intervals [][]int, newInterval []int) [][]int {
    merge := func(intervals [][]int) (ans [][]int) {
        sort.Slice(intervals, func(i, j) bool {
            return intervals[i][0] < intervals[j][0]
        })
        ans = append(ans, intervals[0])
        for i := 1; i < len(intervals); i++ {
            if ans[len(ans)-1][1] < intervals[i][0] {
                ans = append(ans, intervals[i])
            } else {
                ans[len(ans)-1][1] = max(ans[len(ans)-1][1], intervals[i][1])
            }
        }
        return ans
    }
    intervals = append(intervals, newInterval)
    return merge(intervals)
}
```

Rust

Rust

```
impl Solution {
    pub fn insert(intervals: Vec<Vec<i32>>, new_interval: Vec<i32>) -> Vec<Vec<i32>> {
        let mut merged_intervals = intervals.clone();
        merged_intervals.push(intervals[0], new_interval[0], new_interval[1]);
        // sort by start
        merged_intervals.sort_by_key(|x| x[0]);
        // merge interval
        let mut result = vec![];
        for interval in merged_intervals {
            if result.is_empty() {
                result.push(interval);
                continue;
            }
            let last_elem = result.last_mut().unwrap();
            if interval[0] > last_elem[1] {
                result.push(interval);
            } else {
                last_elem[1] = last_elem[1].max(interval[1]);
            }
        }
        result
    }
}
```

Rust

Arrays & Hashing · LeetCode · By Pattern

— 1005 —

LeetCode

```
impl Solution {
    pub fn insert(intervals: Vec<Vec<i32>>, new_interval: Vec<i32>) -> Vec<Vec<i32>> {
        let mut intervals = Vec::new();
        let mut result = vec![];

        let (mut start, mut end) = (new_interval[0], new_interval[1]);
        for iter in intervals.iter() {
            let (cur_st, cur_ed) = (iter[0], iter[1]);
            if cur_ed < start {
                result.push(vec![cur_st, cur_ed]);
            } else if cur_st > end {
                if !inserted {
                    inserted = true;
                    result.push(vec![start, end]);
                }
                result.push(vec![cur_st, cur_ed]);
            } else {
                start = std::cmp::min(start, cur_st);
                end = std::cmp::max(end, cur_ed);
            }
        }
        if !inserted {
            result.push(vec![start, end]);
        }
        result
    }
}
```

[1] Arrays & Hashing — Pattern Solution (stored optimal)

Python

Python solution for Insert Interval

```
class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        # Step 1: Store the resulting intervals
        merged_intervals = []
        i = 0
        n = len(intervals)

        # Add all intervals that end before newInterval starts
        while i < n and intervals[i][1] < newInterval[0]:
            merged_intervals.append(intervals[i])
            i += 1

        # Merge overlapping intervals with newInterval
        while i < n and intervals[i][0] <= newInterval[1]:
            # Update the start and end of newInterval by merging with overlapping intervals
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1

        # Add the merged newInterval
        merged_intervals.append(newInterval)

        # Add any remaining intervals that come after newInterval
        while i < n:
```

Arrays & Hashing · LeetCode · By Pattern

— 1006 —

LeetCode

```
merged_intervals.append(intervals[i])
i += 1
return merged_intervals
```

C++

// C++ solution for Insert Interval

#include <vector>

#include <algorithm>

using namespace std;

class Solution {

public:

vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>& newInterval) {

vector<vector<int>> merged_intervals;

int i = 0, n = intervals.size();

// Add intervals that come before the new interval

while (i < n && intervals[i][1] < newInterval[0]) {

merged_intervals.push_back(intervals[i]);

i++;

}

// Merge all overlapping intervals with newInterval

while (i < n && intervals[i][0] <= newInterval[1]) {

newInterval[0] = min(newInterval[0], intervals[i][0]);

newInterval[1] = max(newInterval[1], intervals[i][1]);

i++;

}

// Append the merged newInterval

merged_intervals.push_back(newInterval);

// Append remaining intervals

while (i < n) {

merged_intervals.push_back(intervals[i]);

i++;

}

return merged_intervals;

}

#238

MEDIUM

Product of Array Except Self

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Prefix Sum

Units: Grid 100

Problem:

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

Arrays & Hashing · LeetCode · By Pattern

— 1007 —

LeetCode

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`

Output: `[0,8,9,0,8]`

Constraints:

$2 \leq \text{nums.length} \leq 10^5$

$-30 \leq \text{nums}[i] \leq 30$

• The input is generated such that `answer[i]` is guaranteed to fit in a 32-bit integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array does not count as extra space for space complexity analysis.)

>> Brute Force [Arrays & Hashing] Interview Prep

Python

Brute Force Approach

class Solution:

def productExceptSelf(self, nums: List[int]) -> List[int]:

Brute Force O(n^2) -> for each index multiply all other elements

n = len(nums)

result = []

for i in range(n):

prod = 1

for j in range(n):

if i != j:

prod *= nums[j]

result.append(prod)

return result

Python

Python

Arrays & Hashing · LeetCode · By Pattern

— 1008 —

LeetCode

```
# Function to compute product of array except self
def productExceptSelf(nums):
    n = len(nums)
    # Initialize the output array with 1 for prefix multiplication
    answer = [1] * n

    # Compute prefix products: for each element answer[i] holds the product of all numbers left of i
    prefix_product = 1
    for i in range(n):
        answer[i] = prefix_product # store prefix product so far
        prefix_product = nums[i] # update the prefix product

    # Compute suffix products and multiply with prefix products in answer array
    suffix_product = 1
    for i in range(n-1, -1, -1):
        answer[i] = suffix_product # multiply current answer with suffix product
        suffix_product = nums[i] # update the suffix product

    return answer

# Example usage
nums = [1, 2, 3, 4]
print(productExceptSelf(nums)) # Output: [24, 12, 8, 6]
```

Python

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        prefix = [1] * n # prefix product
        suffix = [1] * n # suffix product

        for i in range(1, n):
            prefix[i] = prefix[i-1] * nums[i-1]

        for i in reversed(range(n-1)):
            suffix[i] = suffix[i+1] * nums[i+1]

        return [prefix[i] * suffix[i] for i in range(n)]
```

Python

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, v in enumerate(nums):
            ans[i] = left
            left *= v
        for i in range(n-1, -1, -1):
            ans[i] *= right
            right *= nums[i]
        return ans
```

Python

```
class Solution: # Extra arrays
    def productExceptSelf(self, nums):
        if not nums or len(nums) == 0:
            return nums

        # Product from index=0 to index=i-1
        from_left = [1] * len(nums)
        for i in range(1, len(nums)):
            from_left[i] = nums[i-1] * from_left[i-1]

        # Product from index=i+1 to current position
        from_right = [1] * len(nums)
        for i in range(len(nums)-2, -1, -1):
            from_right[i] = nums[i+1] * from_right[i+1]

        # Calculate result
        result = [0] * len(nums)
        for i in range(len(nums)):
            result[i] = from_left[i] * from_right[i]

        return result
```

LeetCode

class Solution: # Follow up, no extra space

```
def productExceptSelf(self, nums: List[int]) -> List[int]:
    n = len(nums)
    ans = [0] * n
    left = right = 1
    for i, v in enumerate(nums):
        ans[i] = left
        left *= v
    for i in range(n-1, -1, -1):
        ans[i] = right
        right *= nums[i]
    return ans
```

LeetCode

```
class Solution(object):
    def productExceptSelf(self, nums):
        type nums: List[int]
        type: List[int]
        --
        dp = [1] * len(nums)
        for i in range(1, len(nums)):
            dp[i] = dp[i-1] * nums[i-1]

        prod = 1
        for i in reversed(range(0, len(nums))):
            dp[i] = dp[i] * prod
            prod = nums[i]
        return dp
```

```
C++
C++

class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans(n); // Can also use 'nums' as the ans array.
        vector<int> prefix(n, 1); // prefix product
        vector<int> suffix(n, 1); // suffix product

        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i-1] * nums[i-1];

        for (int i = n-2; i >= 0; --i)
            suffix[i] = suffix[i+1] * nums[i+1];

        for (int i = 0; i < n; ++i)
            ans[i] = prefix[i] * suffix[i];

        return ans;
    }
};
```

C++

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(n);
        for (int i = 0; left = 1; i < n; ++i) {
            ans[i] = left;
            left *= nums[i];
        }
        for (int i = n-1; right = 1; i >= 0; --i) {
            ans[i] *= right;
            right *= nums[i];
        }
        return ans;
    }
};
```

C++

```
class Solution {
    //
    // @param Integer[] nums
    // @return Integer[]
    //
    function productExceptSelf($nums) {
        $n = count($nums);
        $ans = [];
        for ($i = 0, $left = 1; $i < $n; ++$i) {
            $ans[i] = $left;
            $left *= $nums[i];
        }
        for ($i = $n-1, $right = 1; $i >= 0; --$i) {
            $ans[i] *= $right;
            $right *= $nums[i];
        }
        return $ans;
    }
}
```

C++

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(n);
        for (int i = 0; left = 1; i < n; ++i) {
            ans[i] = left;
            left *= nums[i];
        }
        for (int i = n-1; right = 1; i >= 0; --i) {
            ans[i] *= right;
            right *= nums[i];
        }
        return ans;
    }
};
```

C++

```
class Solution {
    //
    // @param Integer[] nums
    // @return Integer[]
    //
    function productExceptSelf($nums) {
        $n = count($nums);
        $ans = [];
        for ($i = 0, $left = 1; $i < $n; ++$i) {
            $ans[i] = $left;
            $left *= $nums[i];
        }
        for ($i = $n-1, $right = 1; $i >= 0; --$i) {
            $ans[i] *= $right;
            $right *= $nums[i];
        }
        return $ans;
    }
}
```

```
Java
Java

class Solution {
    public int[] productExceptSelf(int[] nums) {
        final int n = nums.length;
        int[] ans = new int[n]; // Can also use 'nums' as the ans array.
        int[] prefix = new int[n]; // prefix product
        int[] suffix = new int[n]; // suffix product

        prefix[0] = 1;
        for (int i = 1; i < n; ++i)
            prefix[i] = prefix[i-1] * nums[i-1];

        suffix[n-1] = 1;
        for (int i = n-2; i >= 0; --i)
            suffix[i] = suffix[i+1] * nums[i+1];

        for (int i = 0; i < n; ++i)
            ans[i] = prefix[i] * suffix[i];

        return ans;
    }
}
```

Java

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        for (int i = 0; left = 1; i < n; ++i) {
            ans[i] = left;
            left *= nums[i];
        }
        for (int i = n-1; right = 1; i >= 0; --i) {
            ans[i] *= right;
            right *= nums[i];
        }
        return ans;
    }
}
```

Java

```
public class Solution {
    public int[] ProductExceptSelf(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        for (int i = 0; left = 1; i < n; ++i) {
            ans[i] = left;
            left *= nums[i];
        }
        for (int i = n-1; right = 1; i >= 0; --i) {
            ans[i] *= right;
            right *= nums[i];
        }
        return ans;
    }
}
```

Java

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        for (int i = 0; left = 1; i < n; ++i) {
            ans[i] = left;
            left *= nums[i];
        }
        for (int i = n-1; right = 1; i >= 0; --i) {
            ans[i] *= right;
            right *= nums[i];
        }
        return ans;
    }
}
```

JavaScript

```
public class Solution {
    public int[] ProductExceptSelf(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        for (int i = 0; left = 1; i < n; ++i) {
            ans[i] = left;
            left *= nums[i];
        }
        for (int i = n-1; right = 1; i >= 0; --i) {
            ans[i] *= right;
            right *= nums[i];
        }
        return ans;
    }
}
```

JavaScript

```

/*
 * Given a number[] num
 * Return number[]
 */
var productExceptSelf = function (nums) {
    const n = nums.length;
    const ans = new Array(n);
    for (let i = 0, left = 1; i < n; ++i) {
        ans[i] = left;
        left = nums[i];
    }
    for (let i = n - 1, right = 1; i >= 0; --i) {
        ans[i] *= right;
        right = nums[i];
    }
    return ans;
};

```

JavaScript

```

/*
 * Given a number[] num
 * Return number[]
 */
var productExceptSelf = function (nums) {
    const n = nums.length;
    const ans = new Array(n);
    for (let i = 0, left = 1; i < n; ++i) {
        ans[i] = left;
        left = nums[i];
    }
    for (let i = n - 1, right = 1; i >= 0; --i) {
        ans[i] *= right;
        right = nums[i];
    }
    return ans;
};

```

TypeScript

```

typeScript
function productExceptSelf(nums: number[]): number[] {
    const n = nums.length;
    const ans: number[] = new Array(n);
    for (let i = 0, left = 1; i < n; ++i) {
        ans[i] = left;
        left = nums[i];
    }
    for (let i = n - 1, right = 1; i >= 0; --i) {
        ans[i] *= right;
        right = nums[i];
    }
    return ans;
}

```

TypeScript

Arrays & Hashing · LeetCode — By Pattern

— 1015 —

LeetCode

```

function productExceptSelf(nums: number[]): number[] {
    const n = nums.length;
    const ans: number[] = new Array(n);
    for (let i = 0, left = 1; i < n; ++i) {
        ans[i] = left;
        left = nums[i];
    }
    for (let i = n - 1, right = 1; i >= 0; --i) {
        ans[i] *= right;
        right = nums[i];
    }
    return ans;
}

```

Go

Go

```

func productExceptSelf(nums []int) []int {
    n := len(nums)
    ans := make([]int, n)
    left, right := 1, 1
    for i, v := range nums {
        ans[i] = left
        left = v
    }
    for i := n - 1; i >= 0; i-- {
        ans[i] *= right
        right = nums[i]
    }
    return ans
}

```

Go

```

func productExceptSelf(nums []int) []int {
    n := len(nums)
    ans := make([]int, n)
    left, right := 1, 1
    for i, v := range nums {
        ans[i] = left
        left = v
    }
    for i := n - 1; i >= 0; i-- {
        ans[i] *= right
        right = nums[i]
    }
    return ans
}

```

Rust

Rust

Arrays & Hashing · LeetCode — By Pattern

— 1016 —

LeetCode

```

impl Solution {
    pub fn product_except_self(nums: Vec<i32>) -> Vec<i32> {
        let n = nums.len();
        let mut ans = vec![0; n];
        for i in 0..n {
            ans[i] = ans[i - 1] * nums[i - 1];
        }
        let mut r = 1;
        for i in (0..n).rev() {
            ans[i] *= r;
            r = nums[i];
        }
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn product_except_self(nums: Vec<i32>) -> Vec<i32> {
        let n = nums.len();
        let mut ans = vec![0; n];
        for i in 0..n {
            ans[i] = ans[i - 1] * nums[i - 1];
        }
        let mut r = 1;
        for i in (0..n).rev() {
            ans[i] *= r;
            r = nums[i];
        }
        ans
    }
}

```

[*] Arrays & Hashing — Pattern Solution (stored optimal)

Python

Arrays & Hashing · LeetCode — By Pattern

— 1017 —

LeetCode

```

# Function to compute product of array except self
def productExceptSelf(nums):
    n = len(nums)
    # Initialize the output array with 1 for prefix multiplication
    answer = [1] * n

    # Compute prefix products: for each element answer[i] holds the product of all numbers left of i
    prefix_product = 1
    for i in range(n):
        answer[i] = prefix_product # store prefix product so far
        prefix_product *= nums[i] # update the prefix product

    # Compute suffix products and multiply with prefix products in answer array
    suffix_product = 1
    for i in range(n-1, -1, -1):
        answer[i] *= suffix_product # multiply current answer with suffix product
        suffix_product *= nums[i] # update the suffix product

    return answer

# Example usage
nums = [1, 2, 3, 4]
print(productExceptSelf(nums)) # Output: [24, 12, 8, 6]

```

C++

```

// Function to compute product of array except self
#include <vector>
using namespace std;

vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> answer(n, 1);

    // Compute prefix products and store in answer array
    int prefixProduct = 1;
    for (int i = 0; i < n; ++i) {
        answer[i] = prefixProduct; // assigns product of all elements to the left of i
        prefixProduct *= nums[i]; // update prefix product for next iteration
    }

    // Compute suffix products and multiply into answer array
    int suffixProduct = 1;
    for (int i = n - 1; i >= 0; i--) {
        answer[i] *= suffixProduct; // update answer[i] with product of suffix elements
        suffixProduct *= nums[i]; // update suffix product
    }

    return answer;
}

// Example usage
// int main() {
//     vector<int> nums = {1, 2, 3, 4};
//     vector<int> result = productExceptSelf(nums);
//     // result should be {24, 12, 8, 6}
// }

```

Arrays & Hashing · LeetCode — By Pattern

— 1018 —

LeetCode

#281

MEDIUM

Zigzag Iterator

LeetCode · Simplify · WalkCC · LeetCode · Editorial

Array · Design · Queue · Iterator

Uttis Denny Zhang

Problem:

Given two vectors of integers v1 and v2, implement an iterator to return their elements alternately.

Implement the ZigzagIterator class:

- ZigzagIterator(List<int> v1, List<int> v2) initializes the object with the two vectors v1 and v2.
- boolean hasNext() returns true if the iterator still has elements, and false otherwise.
- int next() returns the current element of the iterator and moves the iterator to the next element.

Example 1:

Input: v1 = [1,2], v2 = [3,4,5,6]

Output: [1,3,2,4,5,6]

Explanation: By calling next repeatedly until hasNext returns false, the order of elements returned by next should be: [1,3,2,4,5,6].

Example 2:

Input: v1 = [1], v2 = []

Output: [1]

Example 3:

Input: v1 = [], v2 = [1]

Output: [1]

Constraints:

- 0 <= v1.length, v2.length <= 1000
- 1 <= v1.length + v2.length <= 2000
- -231 <= v1[i], v2[i] <= 231 - 1

Follow up: What if you are given k vectors? How well can your code be extended to such cases?

Clarification for the follow-up question:

The "Zigzag" order is not clearly defined and is ambiguous for k > 2 cases. If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic".

Follow-up Example:

Input: v1 = [1,2,3], v2 = [4,5,6,7], v3 = [8,9]

Output: [1,4,8,2,5,9,3,6,7]

Python

Python

Arrays & Hashing · LeetCode — By Pattern

— 1019 —

LeetCode

```

class ZigzagIterator:
    def __init__(self, v1, v2):
        # Initialize the queue with non-empty vectors paired with starting index 0
        from collections import deque
        self.queue = deque()
        if v1:
            self.queue.append((v1, 0))
        if v2:
            self.queue.append((v2, 0))

    def next(self):
        # Pop from the list
        vec, idx = self.queue.popleft()
        result = vec[idx]
        # If there are more elements, put it back with the next index
        if idx + 1 < len(vec):
            self.queue.append((vec, idx + 1))
        return result

    def hasNext(self):
        return len(self.queue) > 0

# Example usage
# v1 = [1,2], v2 = [3,4,5,6]
# iterator = ZigzagIterator(v1, v2)
# while iterator.hasNext():
#     print(iterator.next(), end=" ")

```

Python

```

class ZigzagIterator:
    def __init__(self, v1: list[int], v2: list[int]):
        self.v1 = v1
        self.v2 = v2
        self.v1_idx = 0
        self.v2_idx = 0
        self.v1_len = len(v1)
        self.v2_len = len(v2)

    def next(self):
        self.v1_idx += 1
        return self.v1[self.v1_idx - 1]

    def hasNext(self):
        return self.v1_idx < self.v1_len or self.v2_idx < self.v2_len

```

Python

Arrays & Hashing · LeetCode — By Pattern

— 1020 —

LeetCode

```

class ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.cur = 0
        self.size = 2
        self.indexes = [0] * self.size
        self.vectors = [v1, v2]

    def next(self) -> int:
        vector = self.vectors[self.cur]
        index = self.indexes[self.cur]
        res = vector[index]
        self.indexes[self.cur] = index + 1
        self.cur = (self.cur + 1) % self.size
        return res

    def hasNext(self) -> bool:
        start = self.cur
        while self.indexes[self.cur] == len(self.vectors[self.cur]):
            self.cur = (self.cur + 1) % self.size
            if self.cur == start:
                return False
            return True
        return True

# Your ZigzagIterator object will be instantiated and called as such:
# i, v = ZigzagIterator(v1, v2)
# while i.hasNext(): v.append(i.next())

```

Python

```

class ZigzagIterator:
    # list[] is index of vector of vectors list, list[] is start of a vector, list[] is end
    # = []
    # no map needed, if some 'vectors' a class variable
    vectors = []

    def __init__(self, vectors):
        self.vectors = vectors
        for i in range(len(vectors)):
            self.q.append([i, 0, len(vectors[i])])

    def next(self):
        current = self.q.pop(0)
        i = current[0]
        start = current[1]
        end = current[2]

        if start + 1 == end:
            self.q.append([i, start + 1, end])
        return self.vectors[i][start]

    private:
        queue-pair<vector-idx::iterator, vector-idx::iterator> q;

```

Arrays & Hashing · LeetCode · By Pattern

— 1021 —

LeetCode

```

def hasNext(self):
    return len(self.q) != 0

#####

class ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.cur = 0
        self.size = 2
        self.indexes = [0] * self.size
        self.vectors = [v1, v2]

    def next(self) -> int:
        vector = self.vectors[self.cur]
        index = self.indexes[self.cur]
        res = vector[index]
        self.indexes[self.cur] = index + 1
        self.cur = (self.cur + 1) % self.size
        return res

    def hasNext(self) -> bool:
        start = self.cur
        while self.indexes[self.cur] == len(self.vectors[self.cur]):
            self.cur = (self.cur + 1) % self.size
            if self.cur == start:
                return False
            return True
        return True

# Your ZigzagIterator object will be instantiated and called as such:
# i, v = ZigzagIterator(v1, v2)
# while i.hasNext(): v.append(i.next())
#####

iter() - builtin function, return an iterator object
https://docs.python.org/3/library/functions.html#iter

>>> a = next(iter, [1,2],[3,4])
>>>
>>> b = next(a)
>>> b
<list_iterator object at 0x01302020>
>>> next(a)
1
>>> next(a)
2
>>> next(a)
3

Traceback (most recent call last):
File "<stdin>", line 1, in <module>
StopIteration

```

Python

```

...

iter() - builtin function, return an iterator object
https://docs.python.org/3/library/functions.html#iter

>>> a = next(iter, [1,2],[3,4])
>>>
>>> b = next(a)
>>> b
<list_iterator object at 0x01302020>
>>> next(a)
1
>>> next(a)
2
>>> next(a)
3

Traceback (most recent call last):
File "<stdin>", line 1, in <module>
StopIteration

```

C++

C++

Arrays & Hashing · LeetCode · By Pattern

— 1022 —

LeetCode

```

class ZigzagIterator {
public:
    ZigzagIterator(vector<int>& v1, vector<int>& v2) {
        if (!v1.empty())
            q.emplace(v1.begin(), v1.end());
        if (!v2.empty())
            q.emplace(v2.begin(), v2.end());
    }

    int next() {
        const auto [it, endIt] = q.front();
        q.pop();
        if (it + 1 != endIt)
            q.emplace(it + 1, endIt);
        return *it;
    }

    bool hasNext() {
        return !q.empty();
    }

private:
    queue<pair<vector<int>::iterator, vector<int>::iterator>> q;
};

```

C++

```

// 02: https://leetcode.com/problems/zigzag-iterator/
// Time:
// ZigzagIterator: O(N)
// next: O(1)
// hasNext: O(1)
// Space: O(N), where N is the number of input vectors
class ZigzagIterator {
private:
    queue<pair<vector<int>::iterator, vector<int>::iterator>> q;

public:
    ZigzagIterator(vector<int>& v1, vector<int>& v2) {
        if (v1.begin() != v1.end()) q.push(make_pair(v1.begin(), v1.end()));
        if (v2.begin() != v2.end()) q.push(make_pair(v2.begin(), v2.end()));
    }

    int next() {
        auto p = q.front();
        q.pop();
        int val = *p.first;
        p.first++;
        if (p.first != p.second) q.push(p);
        return val;
    }

    bool hasNext() {
        return !q.empty();
    }
};

```

Java

Java

Arrays & Hashing · LeetCode · By Pattern

— 1023 —

LeetCode

```

public class ZigzagIterator {
    public ZigzagIterator(List<Integer> v1, List<Integer> v2) {
        if (!v1.isEmpty())
            q.offer(v1.iterator());
        if (!v2.isEmpty())
            q.offer(v2.iterator());
    }

    public int next() {
        final Iterator it = q.poll();
        final int next = (it != null) ? it.next() : -1;
        if (it.hasNext())
            q.offer(it);
        return next;
    }

    public boolean hasNext() {
        return !q.isEmpty();
    }

    private Queue<Iterator> q = new ArrayDeque<>();
}

```

Java

```

public class ZigzagIterator {
    public static void main(String[] args) {
        List<Integer> v1 = Arrays.asList(1,2);
        List<Integer> v2 = Arrays.asList(3, 4, 5, 6);
        ZigzagIterator int = new ZigzagIterator(v1, v2);
        ZigzagIterator z = new ZigzagIterator(v1, v2);
        while (z.hasNext()) {
            System.out.println(z.next());
        }

        List<Integer> v3 = Arrays.asList(1,2,3);
        List<Integer> v4 = Arrays.asList(4,5,6,7);
        List<Integer> v5 = Arrays.asList(8,9);

        List<List<Integer>> vectors = Arrays.asList(v3, v4, v5);
        ZigzagIterator z6 = new ZigzagIterator(vectors);
        while (z6.hasNext()) {
            System.out.println(z6.next());
        }
    }
}

```

Arrays & Hashing · LeetCode · By Pattern

— 1024 —

LeetCode

```

}

public class ZigzagIterator {
    // list[] is index of vector of vectors list, list[] is start of a vector, list[] is end
    Queue<int> q = new LinkedList<>();

    // Map<Integer, List<Integer>> vectorsIndexMap = new HashMap<>(); // no map needed, if some 'vectors' a
    // class variable
    List<List<Integer>> vectors;

    public ZigzagIterator(List<List<Integer>> vectors) {
        this.vectors = vectors;
        for (int i = 0; i < vectors.size(); i++) {
            q.offer(new int[]{i, 0, vectors.get(i).size()});
        }
        // vectorsIndexMap.put(i, vectors.get(i));
    }

    public int next() {
        int[] current = q.poll();

        int i = current[0];
        int start = current[1];
        int end = current[2];

        if (start + 1 == end) {
            q.offer(new int[]{i, start + 1, end});
        }

        // return vectorsIndexMap.get(i).get(start);
        return vectors.get(i).get(start);
    }

    public boolean hasNext() {
        return !q.isEmpty();
    }

    // ZigzagIterator KUDU
    public class ZigzagIteratorBuiltinIterator {
        List<List<Integer>> iters = new ArrayList<List<Integer>>();

        int count = 0;

        public ZigzagIteratorBuiltinIterator(List<Integer> v1, List<Integer> v2) {
            if (!v1.isEmpty()) {
                iters.add(v1.iterator());
            }
            if (!v2.isEmpty()) {
                iters.add(v2.iterator());
            }
        }

        public int next() {
            int s = iters.get(count).next();
        }
    }
}

```

Arrays & Hashing · LeetCode · By Pattern

— 1025 —

LeetCode

```

Go

type ZigzagIterator struct {
    cur int
    size int
    indexes []int
    vectors [][]int
}

func Constructor(v1, v2 []int) *ZigzagIterator {
    return &ZigzagIterator{
        cur: 0,
        size: 2,
        indexes: []int{0, 0},
        vectors: [][]int{v1, v2},
    }
}

func (this *ZigzagIterator) next() int {
    vector := this.vectors[this.cur]
    index := this.indexes[this.cur]
    res := vector[index]
    this.indexes[this.cur] = index + 1
    this.cur = (this.cur + 1) % this.size
    return res
}

func (this *ZigzagIterator) hasNext() bool {
    start := this.cur
    for this.indexes[this.cur] == len(this.vectors[this.cur]) {
        this.cur = (this.cur + 1) % this.size
        if start == this.cur {
            return false
        }
    }
    return true
}

//
# Your ZigzagIterator object will be instantiated and called as such:
# obj := Constructor(v1, v2)
# for obj.hasNext() {
#     ans = append(ans, obj.next())
# }

```

Go

Arrays & Hashing · LeetCode · By Pattern

— 1026 —

LeetCode

```
type ZigzagIterator struct {
    cur int
    size int
    indexes []int
    vectors [][]int
}

func Constructor(v1, v2 []int) ZigzagIterator {
    return ZigzagIterator{
        cur: 0,
        size: 2,
        indexes: []int{0, 0},
        vectors: [][]int{v1, v2},
    }
}

func (this *ZigzagIterator) next() int {
    vector := this.vectors[this.cur]
    index := this.indexes[this.cur]
    res := vector[index]
    this.indexes[this.cur]++
    this.cur = (this.cur + 1) % this.size
    return res
}

func (this *ZigzagIterator) hasNext() bool {
    start := this.cur
    for this.indexes[this.cur] == len(this.vectors[this.cur]) {
        this.cur = (this.cur + 1) % this.size
        if start == this.cur {
            return false
        }
    }
    return true
}

/**
 * Your ZigzagIterator object will be instantiated and called as such:
 * int i;
 * ZigzagIterator zigzag(Constructor(param_1, param_2));
 * for (i; zigzag.hasNext(); i++) {
 *     int val = zigzag.next();
 * }
 */
}

// Rust
// Rust
```

```
struct ZigzagIterator {
    v1: Vec<i32>,
    v2: Vec<i32>,
    // false represents 'v1', true represents 'v2'
    flag: bool,
}

impl ZigzagIterator {
    fn new(v1: Vec<i32>, v2: Vec<i32>) -> Self {
        Self {
            v1,
            v2,
            // Initially beginning with 'v1'
            flag: false,
        }
    }

    fn next(&mut self) -> i32 {
        if !self.flag {
            // v1
            if self.v1.is_empty() && !self.v2.is_empty() {
                self.flag = true;
                let res = self.v2.remove(0);
                return res;
            }

            if !self.v2.is_empty() {
                let res = self.v2.remove(0);
                return res;
            }

            if self.v1.is_empty() {
                let res = self.v1.remove(0);
                return res;
            }

            if self.v1.is_empty() && !self.v2.is_empty() {
                self.flag = false;
                let res = self.v1.remove(0);
                return res;
            }

            if self.v1.is_empty() {
                let res = self.v2.remove(0);
                return res;
            }

            let res = self.v2.remove(0);
            self.flag = false;
            return res;
        }
    }

    fn has_next(&self) -> bool {
        !self.v1.is_empty() || !self.v2.is_empty()
    }
}

// Python
```

```
class ZigzagIterator:
    def __init__(self, v1, v2):
        # Initialize the queue with non-empty vectors paired with starting index 0
        from collections import deque
        self.queue = deque()
        if v1:
            self.queue.append((v1, 0))
        if v2:
            self.queue.append((v2, 0))

    def next(self):
        # Pop from the left
        vec, idx = self.queue.popleft()
        result = vec[idx]
        # If there are more elements, put it back with the next index
        if idx + 1 < len(vec):
            self.queue.append((vec, idx + 1))
        return result

    def hasNext(self):
        return len(self.queue) > 0

# Example usage:
# v1 = [1,2], v2 = [3,4,5,6]
# iterator = ZigzagIterator(v1, v2)
# while iterator.hasNext():
#     print(iterator.next(), end=" ")

// C++
// C++
```

```
#include <vector>
#include <queue>
using namespace std;

class ZigzagIterator {
private:
    // The queue holds pairs, the vector and the current index
    queue<pair<const vector<int>*, int>> iterQueue;
public:
    ZigzagIterator(vector<int>* v1, vector<int>* v2) {
        if (!v1.empty())
            iterQueue.push({v1, 0});
        if (!v2.empty()) {
            iterQueue.push({v2, 0});
        }
    }

    int next() {
        // Get the front pair from the queue
        auto current = iterQueue.front();
        iterQueue.pop();
        const vector<int>* vecPtr = current.first;
        int idx = current.second;
        int result = (*vecPtr)[idx];
        if (idx + 1 < vecPtr->size())
            iterQueue.push({vecPtr, idx + 1});
    }
};

// Python
```

```
// If there are more elements in the current vector, push back with next index
if (idx + 1 < vecPtr->size()) {
    iterQueue.push({vecPtr, idx + 1});
}
return result;
}

bool hasNext() {
    return !iterQueue.empty();
}

// Example usage:
// vector<int> v1 = {1,2};
// vector<int> v2 = {3,4,5,6};
// ZigzagIterator zigzag(v1, v2);
// while(zigzag.hasNext()) {
//     cout << zigzag.next() << " ";
// }
```

#307 MEDIUM Range Sum Query - Mutable

LeetCode - SimplyLess - NumArray - LeetCode - Editorial

Array - Design - Binary Indexed Tree - Segment Tree

Lists Denny Zhang

Problem:
Given an integer array nums, handle multiple queries of the following types:

- Update the value of an element in nums.
- Calculate the sum of the elements of nums between indices left and right inclusive where left <= right.

Implement the NumArray class:

- NumArray(int[] nums) Initializes the object with the integer array nums.
- void update(int index, int val) Updates the value of nums[index] to be val.
- int sumRange(int left, int right) Returns the sum of the elements of nums between indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).

Example 1:

Input

```
["NumArray", "sumRange", "update", "sumRange"]
[[1, 3, 5], [0, 2], [1, 2], [0, 2]]
```

Output

```
[null, 9, null, 8]
```

Explanation

```
NumArray numArray = new NumArray([1, 3, 5]);
numArray.sumRange(0, 2); // return 1 + 3 + 5 = 9
numArray.update(1, 2); // nums = [1, 2, 5]
numArray.sumRange(0, 2); // return 1 + 2 + 5 = 8
```

Constraints:

- 1 <= nums.length <= 3 * 10⁴
- 100 <= nums[i] <= 100
- 0 <= index < nums.length
- 100 <= val <= 100
- 0 <= left <= right < nums.length
- At most 3 * 10⁴ calls will be made to update and sumRange.

Python

```
Python
# Python implementation using a Binary Indexed Tree (Fenwick Tree)
class NumArray:
    def __init__(self, nums):
        # Store the original array
        self.nums = nums
        self.n = len(nums)
        # Initialize the BIT with 0s, size is n+1 because BIT is 1-indexed
        self.tree = [0] * (self.n + 1)
        # Build the BIT: update time for each value in the nums array
        for i, val in enumerate(nums):
            self._update_tree(i, val)

    def _update_tree(self, index, delta):
        # Update the BIT for index with delta change
        i = index + 1 # Convert to 1-indexed for BIT operations
        while i <= self.n:
            self.tree[i] += delta # Add delta to current tree node
            i += i & -i # Move to the next responsible node

    def _update(self, index, val):
        # Calculate the difference between new value and current value
        delta = val - self.nums[index]
        self.nums[index] = val # Update the original array
        self._update_tree(index, delta) # Update the BIT to reflect this change

    def _prefix_sum(self, index):
        # Compute the prefix sum from start up to and including index
        result = 0
        i = index + 1 # Convert to 1-indexed
        while i:
            result += self.tree[i]
            i -= i & -i # Move to parent node
        return result

    def sumRange(self, left, right):
        # Return the sum of the range [left, right]
        return self._prefix_sum(right) - self._prefix_sum(left - 1) if left > 0 else self._prefix_sum(right)

# Example usage:
# numArray = NumArray([1, 3, 5])
# print(numArray.sumRange(0, 2))
# numArray.update(1, 2)
# print(numArray.sumRange(0, 2))

// Python
```

```
class FenwickTree:
    def __init__(self, n: int):
        self.size = n
        self.tree = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i <= self.size:
            self.tree[i] += delta
            i = FenwickTree._lsh(i)

    def get(self, i: int) -> int:
        sum = 0
        while i > 0:
            sum += self.tree[i]
            i = FenwickTree._lsh(i)
        return sum

    @staticmethod
    def _lsh(i: int) -> int:
        return i & -i

class NumArray:
    def __init__(self, nums: List[int]):
        self.nums = nums
        self.tree = FenwickTree(len(nums))

        for i, num in enumerate(nums):
            self.tree.add(i + 1, num)

    def update(self, index: int, val: int) -> None:
        self.tree.add(index + 1, val - self.nums[index])
        self.nums[index] = val

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.get(right + 1) - self.tree.get(left)
```

Python


```
class BinaryIndexedTree:
    __slots__ = ["n", "t"]

    def __init__(self, n):
        self.n = n
        self.t = [0] * (n + 1)

    def update(self, x: int, delta: int):
        while x <= self.n:
            self.t[x] += delta
            x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x > 0:
            s += self.t[x]
            x -= x & -x
        return s

class NumArray:
    __slots__ = ["tree"]

    def __init__(self, nums: list[int]):
        self.tree = BinaryIndexedTree(len(nums))

        for i, v in enumerate(nums, 1):
            self.tree.update(i, v)

    def update(self, index: int, val: int) -> None:
        prev = self.sumRange(index, index)
        self.tree.update(index + 1, val - prev)

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(right + 1) - self.tree.query(left)

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index, val)
# param_2 = obj.sumRange(left, right)
```

Python

```
class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.t = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.t[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.t[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

class NumArray:
    def __init__(self, nums: list[int]):
        self.tree = BinaryIndexedTree(len(nums))

        for i, v in enumerate(nums, 1):
            self.tree.update(i, v)

    def update(self, index: int, val: int) -> None:
        prev = self.sumRange(index, index)
        self.tree.update(index + 1, val - prev)

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(right + 1) - self.tree.query(left)

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index, val)
# param_2 = obj.sumRange(left, right)
```

Python

```
self.right = None

class SegmentTree(object):
    def __init__(self, nums, start, end):
        self.root = self.buildTree(nums, start, end)

    def buildTree(self, nums, start, end):
        if start == end:
            return None

        if start == end:
            node = STNode(start, end)
            node.total = nums[start]
            return node

        mid = start + (end - start) // 2

        root = STNode(start, end)
        root.left = self.buildTree(nums, start, mid)
        root.right = self.buildTree(nums, mid + 1, end)
        root.total = root.left.total + root.right.total
        return root

    def update(self, i, val):
        def updateNode(root, i, val):
            if root.start == root.end:
                root.total = val
                return val
            mid = root.start + (root.end - root.start) // 2
```

C++

C++

```
class BinaryIndexedTree {
public:
    int n;
    vector<int> t;

    BinaryIndexedTree(int _n) :
        n(_n),
        t(1 <= i <= n) {}

    void update(int x, int delta) {
        while (x <= n) {
            t[x] += delta;
            x += x & -x;
        }
    }

    int query(int x) {
        int s = 0;
        while (x > 0) {
            s += t[x];
            x -= x & -x;
        }
        return s;
    }
};

class NumArray {
public:
    BinaryIndexedTree tree;

    NumArray(vector<int>& nums) {
        int n = nums.size();
        tree = new BinaryIndexedTree(n);
        for (int i = 0; i < n; ++i) tree.update(i + 1, nums[i]);
    }

    void update(int index, int val) {
        int prev = sumRange(index, index);
        tree.update(index + 1, val - prev);
    }

    int sumRange(int left, int right) {
        return tree.query(right + 1) - tree.query(left);
    }
};
```

```
/*
 * Your NumArray object will be instantiated and called as such:
 * NumArray obj = new NumArray(nums);
 * obj.update(index, val);
 * int param_2 = obj.sumRange(left, right);
 */
```

Java

Java

```
class BinaryIndexedTree {
    private int n;
    private int[] t;

    public BinaryIndexedTree(int n) {
        this.n = n;
        t = new int[n + 1];
    }

    public void update(int x, int delta) {
        while (x <= n) {
            t[x] += delta;
            x += x & -x;
        }
    }

    public int query(int x) {
        int s = 0;
        while (x > 0) {
            s += t[x];
            x -= x & -x;
        }
        return s;
    }
}

class NumArray {
    private BinaryIndexedTree tree;

    public NumArray(int[] nums) {
        int n = nums.length;
        tree = new BinaryIndexedTree(n);
        for (int i = 0; i < n; ++i) {
            tree.update(i + 1, nums[i]);
        }
    }

    public void update(int index, int val) {
        int prev = sumRange(index, index);
        tree.update(index + 1, val - prev);
    }

    public int sumRange(int left, int right) {
        return tree.query(right + 1) - tree.query(left);
    }
}

/*
 * Your NumArray object will be instantiated and called as such:
 * NumArray obj = new NumArray(nums);
 * obj.update(index, val);
 * int param_2 = obj.sumRange(left, right);
 */
```

TypeScript

TypeScript

```
class BinaryIndexedTree {
    private int number;
    private int[] number[];

    constructor(n: number) {
        this.n = n;
        this.t = Array(n + 1).fill(0);
    }

    update(x: number, delta: number): void {
        while (x <= this.n) {
            this.t[x] += delta;
            x += x & -x;
        }
    }

    query(x: number): number {
        let s = 0;
        while (x > 0) {
            s += this.t[x];
            x -= x & -x;
        }
        return s;
    }
}

class NumArray {
    private tree: BinaryIndexedTree;

    constructor(nums: number[]) {
        const n = nums.length;
        this.tree = new BinaryIndexedTree(n);
        for (let i = 0; i < n; ++i) {
            this.tree.update(i + 1, nums[i]);
        }
    }

    update(index: number, val: number): void {
        const prev = this.sumRange(index, index);
        this.tree.update(index + 1, val - prev);
    }

    sumRange(left: number, right: number): number {
        return this.tree.query(right + 1) - this.tree.query(left);
    }
}

/*
 * Your NumArray object will be instantiated and called as such:
 * var obj = new NumArray(nums);
 * obj.update(index, val);
 * var param_2 = obj.sumRange(left, right);
 */
```

Go

```

type BinaryIndexedTree struct {
    n int
    t []int
}

func newBinaryIndexedTree(n int) *BinaryIndexedTree {
    t := make([]int, n+1)
    return &BinaryIndexedTree{n, t}
}

func (t *BinaryIndexedTree) update(i, delta int) {
    for ; i <= t.n; i += i & -i {
        t.t[i] += delta
    }
}

func (t *BinaryIndexedTree) query(i int) int {
    s := 0
    for ; i > 0; i -= i & -i {
        s += t.t[i]
    }
    return s
}

type NumArray struct {
    tree *BinaryIndexedTree
}

func Constructor(nums []int) NumArray {
    tree := newBinaryIndexedTree(len(nums))
    for i, v := range nums {
        tree.update(i+1, v)
    }
    return NumArray{tree}
}

func (t *NumArray) Update(index int, val int) {
    prev := t.sumRange(index, index)
    t.tree.update(index+1, val-prev)
}

func (t *NumArray) SumRange(left int, right int) int {
    return t.tree.query(right+1) - t.tree.query(left)
}

/*
 * Your NumArray object will be instantiated and called as such:
 * obj := Constructor(nums);
 * obj.Update(index, val);
 * param_1 := obj.SumRange(left, right);
 */

```

[7] Arrays & Hashing — Pattern Solution (stored optimal)

Python

Arrays & Hashing · LeetCode · By Pattern

— 1039 —

LeetCode

```

# Python Implementation using a Binary Indexed Tree (Fenwick Tree)
class NumArray:
    def __init__(self, nums):
        # Store the original array
        self.nums = nums
        self.n = len(nums)
        # Initialize the BIT with size n+1 (n+1 because BIT is 1-indexed)
        self.tree = [0] * (self.n + 1)
        # Build the BIT: update tree for each value in the nums array
        for i, num in enumerate(self.nums):
            self.update_tree(i, num)

    def update_tree(self, index, delta):
        # Update the BIT for index with delta change
        i = index + 1 # Convert to 1-indexed for BIT operations
        while i <= self.n:
            self.tree[i] += delta # Add delta to current tree node
            i += i & -i # Move to the next responsible node

    def update(self, index, val):
        # Calculate the difference between new value and current value
        delta = val - self.nums[index]
        self.nums[index] = val # Update the original array
        self.update_tree(index, delta) # Update the BIT to reflect this change

    def prefix_sum(self, index):
        # Compute the prefix sum from start up to and including index
        result = 0
        i = index + 1 # Convert to 1-indexed
        while i:
            result += self.tree[i]
            i -= i & -i # Move to parent node
        return result

    def sumRange(self, left, right):
        # Return the sum of the range [left, right]
        return self.prefix_sum(right) - self.prefix_sum(left - 1) if left > 0 else self.prefix_sum(right)

# Example usage:
# nums = NumArray([1, 2, 3])
# print(nums.sumRange(0, 2))
# nums.update(1, 2)
# print(nums.sumRange(0, 2))

```

C++

Arrays & Hashing · LeetCode · By Pattern

— 1040 —

LeetCode

```

// C++ Implementation using a Binary Indexed Tree (Fenwick Tree)
// Solution: Fenwick Tree
using namespace std;

class NumArray {
private:
    vector<int> nums; // Store the original array
    vector<int> tree; // Binary Indexed Tree array
    int n; // Size of the original array

    // Helper to update the BIT at position index with a delta
    void updateTree(int index, int delta) {
        for (int i = index + 1; i <= n; i += i & -i) {
            tree[i] += delta;
        }
    }

    // Helper to compute prefix sum up to and including index
    int prefixSum(int index) {
        int sum = 0;
        for (int i = index + 1; i > 0; i -= i & -i) {
            sum += tree[i];
        }
        return sum;
    }

public:
    // Constructor: Initialize the data structure
    NumArray(vector<int>& nums) : nums(nums) {
        n = nums.size();
        tree.resize(n + 1, 0);
        // Build the BIT by updating tree with each num value
        for (int i = 0; i < n; i++) {
            updateTree(i, nums[i]);
        }
    }

    // Update value at index i by val
    void update(int index, int val) {
        int delta = val - nums[index]; // Calculate change needed
        nums[index] = val; // Set new value in original array
        updateTree(index, delta); // Update the BIT accordingly
    }

    // Compute the sum of elements in the range [left, right]
    int sumRange(int left, int right) {
        return left > 0 ? prefixSum(right) - prefixSum(left - 1) : prefixSum(right);
    }
};

```

#454

MEDIUM

4Sum II

LeetCode · SimplifyC++ · WubCC · LeetCode · Editorial

Array · Hash Table

Unit CodeSignal

Problem:

Arrays & Hashing · LeetCode · By Pattern

— 1041 —

LeetCode

Given four integer arrays `nums1`, `nums2`, `nums3`, and `nums4` all of length `n`, return the number of tuples (i, j, k, l) such that:

• $0 \leq i, j, k, l < n$

• $nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0$

Example 1:

Input: `nums1 = [1,2], nums2 = [-2,-1], nums3 = [-1,2], nums4 = [0,2]`

Output: 2

Explanation:

The two tuples are:

1. $(0, 0, 0, 1) \Rightarrow nums1[0] + nums2[0] + nums3[0] + nums4[1] = 1 + (-2) + (-1) + 2 = 0$

2. $(1, 1, 0, 0) \Rightarrow nums1[1] + nums2[1] + nums3[0] + nums4[0] = 2 + (-1) + (-1) + 0 = 0$

Example 2:

Input: `nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]`

Output: 1

Constraints:

• $n == nums1.length$

• $n == nums2.length$

• $n == nums3.length$

• $n == nums4.length$

• $1 \leq n \leq 200$

• $-228 \leq nums1[i], nums2[j], nums3[k], nums4[l] \leq 228$

>> Brute Force [Arrays & Hashing] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def fourSumCount(self, nums1, nums2, nums3, nums4):
        # Brute Force O(n^4) - check all pairs without a hash map
        n = len(nums1)
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    for l in range(n):
                        # Check if the sum of the four elements is zero
                        if nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0:
                            count += 1
        return count

```

Python

Python

Arrays & Hashing · LeetCode · By Pattern

— 1042 —

LeetCode

```

# Function to count quadruplets that sum to zero using a hash map approach
def fourSumCount(nums1, nums2, nums3, nums4):
    # Dictionary to store the frequency of sums of elements from nums1 and nums2
    sum_count = {}
    for num1 in nums1:
        for num2 in nums2:
            current_sum = num1 + num2
            sum_count[current_sum] = sum_count.get(current_sum, 0) + 1

    count = 0
    # For each pair from nums3 and nums4, find the complement that makes the total sum zero
    for num3 in nums3:
        for num4 in nums4:
            complement = -(num3 + num4)
            count += sum_count.get(complement, 0)
    return count

# Example usage
nums1 = [1,2]
nums2 = [-2,-1]
nums3 = [-1,2]
nums4 = [0,2]
print(fourSumCount(nums1, nums2, nums3, nums4)) # Expected Output: 2

```

Python

```

class Solution:
    def fourSumCount(self, nums1: List[int], nums2: List[int],
                     nums3: List[int], nums4: List[int]) -> int:
        count = collections.Counter(a + b for a in nums1 for b in nums2)
        return sum(count[-c - d] for c in nums3 for d in nums4)

```

Python

```

class Solution:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: List[int]) -> int:
        count = Counter(a + b for a in nums1 for b in nums2)
        return sum(count[-c - d] for c in nums3 for d in nums4)

```

Python

Arrays & Hashing · LeetCode · By Pattern

— 1043 —

LeetCode

```

from collections import Counter

class Solution:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: List[int]) -> int:
        count = Counter(a + b for a in nums1 for b in nums2)
        return sum(count[-c - d] for c in nums3 for d in nums4)

```

Python

```

class Solution:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: List[int]) -> int:
        counter = Counter()
        for a in nums1:
            for b in nums2:
                counter[a + b] += 1
        ans = 0
        for c in nums3:
            for d in nums4:
                # adding for every time -c-d
                ans += counter[-c - d]
        return ans

```

Python

```

class Solution(object):
    def fourSumCount(self, A, B, C, D):

```

```

        (type A: List[int])
        (type B: List[int])
        (type C: List[int])
        (type D: List[int])
        (type int: int)

        ans = 0
        abDict = {}
        for i in range(len(A)):
            for j in range(len(B)):
                if A[i] + B[j] not in abDict:
                    abDict[A[i] + B[j]] = 1
                else:
                    abDict[A[i] + B[j]] += 1

        for i in range(len(C)):
            for j in range(len(D)):
                if -C[i] - D[j] in abDict:
                    ans += abDict[-C[i] - D[j]]

        return ans

```

C++

C++

Arrays & Hashing · LeetCode · By Pattern

— 1044 —

LeetCode

```

class Solution {
public:
    int fourSumCount(vector<int>& nums1, vector<int>& nums2, vector<int>& nums3,
                    vector<int>& nums4) {
        int ans = 0;
        unordered_map<int, int> count;
        for (const int a : nums1)
            for (const int b : nums2)
                ++count[a + b];
        for (const int c : nums3)
            for (const int d : nums4)
                if (count.find(-c - d) != count.end())
                    ans += count[-c - d];
        return ans;
    }
};

C++
class Solution {
public:
    int fourSumCount(vector<int>& nums1, vector<int>& nums2, vector<int>& nums3, vector<int>& nums4) {
        unordered_map<int, int> cnt;
        for (int a : nums1) {
            for (int b : nums2) {
                ++cnt[a + b];
            }
        }
        int ans = 0;
        for (int c : nums3) {
            for (int d : nums4) {
                ans += cnt[-(c + d)];
            }
        }
        return ans;
    }
};

C++

```

Arrays & Hashing · LeetCode · By Pattern

— 1045 —

LeetCode

```

// OJ: https://leetcode.com/problems/four-sum-ii
// Time: O(N^2 * logN)
// Space: O(N^2)
class Solution {
private:
    void sum(vector<int>& A, vector<int>& B, map<int, int>& C) {
        for (auto a : A) {
            for (auto b : B) {
                C[a + b]++;
            }
        }
    };
public:
    int fourSumCount(vector<int>& A, vector<int>& B, vector<int>& C, vector<int>& D) {
        map<int, int> A, B;
        sum(A, B, A);
        sum(C, D, B);
        auto i = A.begin();
        int ans = 0;
        while (i != A.end()) {
            while (i != A.end() && j != B.end()) {
                if (i->first + j->first == 0) {
                    ans += i->second * j->second;
                    ++i;
                }
            }
            ++i;
        }
        return ans;
    }
};

Java
class Solution {
    public int fourSumCount(int[] nums1, int[] nums2, int[] nums3, int[] nums4) {
        int ans = 0;
        Map<Integer, Integer> count = new HashMap<>();
        for (final int a : nums1)
            for (final int b : nums2)
                count.merge(a + b, 1, Integer::sum);
        for (final int c : nums3)
            for (final int d : nums4)
                if (count.containsKey(-c - d))
                    ans += count.get(-c - d);
        return ans;
    }
};

Java

```

Arrays & Hashing · LeetCode · By Pattern

— 1046 —

LeetCode

```

class Solution {
public:
    int fourSumCount(vector<int>& nums1, vector<int>& nums2, vector<int>& nums3, vector<int>& nums4) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int a : nums1) {
            for (int b : nums2) {
                count.merge(a + b, 1, Integer::sum);
            }
        }
        int ans = 0;
        for (int c : nums3) {
            for (int d : nums4) {
                ans += count.getOrDefault(-c - d, 0);
            }
        }
        return ans;
    }
};

Java
use std::collections::HashMap;

impl Solution {
    pub fn four_sum_count(
        nums1: Vec<i32>,
        nums2: Vec<i32>,
        nums3: Vec<i32>,
        nums4: Vec<i32>,
    ) -> i32 {
        let mut cnt = HashMap::new();
        for &a in &nums1 {
            for &b in &nums2 {
                *cnt.entry(a + b).or_insert(0) += 1;
            }
        }
        let mut ans = 0;
        for &c in &nums3 {
            for &d in &nums4 {
                if let Some(&count) = cnt.get(&(-(c + d))) {
                    ans += count;
                }
            }
        }
        ans
    }
}

Java

```

Arrays & Hashing · LeetCode · By Pattern

— 1047 —

LeetCode

```

class Solution {
    public int fourSumCount(int[] A, int[] B, int[] C, int[] D) {
        int count = 0;

        // Use an unordered
        Map<Integer, Integer> sumCountMap = new HashMap<Integer, Integer>();
        int length = A.length;
        for (int i = 0; i < length; i++) {
            for (int j = 0; j < length; j++) {
                int sumAB = A[i] + B[j];
                int countAB = sumCountMap.getOrDefault(sumAB, 0);
                countAB++;
                sumCountMap.put(sumAB, countAB);
            }
        }
        for (int i = 0; i < length; i++) {
            for (int j = 0; j < length; j++) {
                int sumCD = C[i] + D[j];
                int curCount = sumCountMap.getOrDefault(-sumCD, 0);
                count += curCount;
            }
        }
        return count;
    }
}

class Solution {
    public int fourSumCount(int[] nums1, int[] nums2, int[] nums3, int[] nums4) {
        Map<Integer, Integer> counter = new HashMap<>();
        for (int a : nums1) {
            for (int b : nums2) {
                counter.merge(a + b, counter.getOrDefault(a + b, 0) + 1);
            }
        }
        int ans = 0;
        for (int a : nums3) {
            for (int d : nums4) {
                ans += counter.getOrDefault(-(a + d), 0);
            }
        }
        return ans;
    }
};

Java

```

Arrays & Hashing · LeetCode · By Pattern

— 1048 —

LeetCode

```

use std::collections::HashMap;

impl Solution {
    pub fn four_sum_count(
        nums1: Vec<i32>,
        nums2: Vec<i32>,
        nums3: Vec<i32>,
        nums4: Vec<i32>,
    ) -> i32 {
        let mut cnt = HashMap::new();
        for &a in &nums1 {
            for &b in &nums2 {
                *cnt.entry(a + b).or_insert(0) += 1;
            }
        }
        let mut ans = 0;
        for &c in &nums3 {
            for &d in &nums4 {
                if let Some(&count) = cnt.get(&(-(c + d))) {
                    ans += count;
                }
            }
        }
        ans
    }
}

TypeScript
function fourSumCount(nums1: number[], nums2: number[], nums3: number[], nums4: number[]): number {
    const cnt: Map<number, number> = new Map();
    for (const a of nums1) {
        for (const b of nums2) {
            const x = a + b;
            cnt[x] = (cnt[x] || 0) + 1;
        }
    }
    let ans = 0;
    for (const c of nums3) {
        for (const d of nums4) {
            const x = -c - d;
            ans += cnt[x] || 0;
        }
    }
    return ans;
}

TypeScript

```

Arrays & Hashing · LeetCode · By Pattern

— 1049 —

LeetCode

```

function fourSumCount(
    nums1: number[],
    nums2: number[],
    nums3: number[],
    nums4: number[]
): number {
    const cnt: Map<number, number> = new Map();
    for (const a of nums1) {
        for (const b of nums2) {
            const x = a + b;
            cnt[x] = (cnt[x] || 0) + 1;
        }
    }
    let ans = 0;
    for (const c of nums3) {
        for (const d of nums4) {
            const x = -c - d;
            ans += cnt[x] || 0;
        }
    }
    return ans;
}

Go
func fourSumCount(nums1 []int, nums2 []int, nums3 []int, nums4 []int) int {
    cnt := map[int]int{}
    for _, a := range nums1 {
        for _, b := range nums2 {
            cnt[a+b]++
        }
    }
    for _, c := range nums3 {
        for _, d := range nums4 {
            ans += cnt[-(c+d)]
        }
    }
    return ans
}

Go
func fourSumCount(nums1 []int, nums2 []int, nums3 []int, nums4 []int) int {
    counter := make(map[int]int)
    for _, a := range nums1 {
        for _, b := range nums2 {
            counter[a+b]++
        }
    }
    ans = 0
    for _, c := range nums3 {
        for _, d := range nums4 {
            ans += counter[-(c+d)]
        }
    }
    return ans
}

```

Arrays & Hashing · LeetCode · By Pattern

— 1050 —

LeetCode

[*] Arrays & Hashing — Pattern Solution (matched from community answers)

Python

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        n = len(nums)
        count = collections.Counter(a = b for a in nums1 for b in nums2)
        return sum(count[x - d] for x in nums1 for d in nums1)
```

#525

MEDIUM

Contiguous Array

LeetCode · Simplest · BrackCC · LeetCode · Editorial

Array · Hash Table · Prefix Sum

List · Grid · 169

Problem:

Given a binary array nums, return the maximum length of a contiguous subarray with an equal number of 0 and 1.

Example 1:

Input: nums = [0,1]

Output: 2

Explanation: [0, 1] is the longest contiguous subarray with an equal number of 0 and 1.

Example 2:

Input: nums = [0,1,0]

Output: 2

Explanation: [0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0 and 1.

Example 3:

Input: nums = [0,1,1,1,1,0,0,0]

Output: 6

Explanation: [1,1,0,0,0,0] is the longest contiguous subarray with equal number of 0 and 1.

Constraints:

* 1 <= nums.length <= 105

* nums[i] is either 0 or 1.

>> Brute Force [Arrays & Hashing] Interview Prep

Python

Brute Force Approach

```
class Solution:
    def findMaxLength(self, nums):
        # Brute Force O(n^2) - check all pairs without a hash map
        n = len(nums)
        for i in range(n):
            for j in range(i + 1, n):
                # Check if subarray [i, j] has equal 0s and 1s
                return (j - i + 1) if nums[i:j+1].count('0') == nums[i:j+1].count('1') else 0
```

Python

Python solution for Contiguous Array

```
def findMaxLength(nums):
    # Dictionary to store the earliest occurrence of each prefix sum
    prefix_indices = {}
    prefix_sum = 0
    max_length = 0

    # Iterate over the array
    for index, num in enumerate(nums):
        # Convert 0 to -1, keep 1 as is
        if num == 0:
            prefix_sum -= 1
        else:
            prefix_sum += 1

        # If the prefix sum has been seen before, a balanced subarray exists
        if prefix_sum in prefix_indices:
            # Calculate the length of the subarray
            current_length = index - prefix_indices[prefix_sum]
            max_length = max(max_length, current_length)
        else:
            # Store the index for this prefix sum if not already seen
            prefix_indices[prefix_sum] = index

    # Return the maximum length of balanced subarray
    return max_length
```

Example usage:
print(findMaxLength([0,1,0])) # Expected output: 2

Python

class Solution {

def findMaxLength(self, nums: List[int]) -> int:

ans = 0

prefix = 0

prefixIndices = {}

for i, num in enumerate(nums):

prefix = 1 if num == 1 else -1

ans = max(ans, 1 - prefixIndices.get(prefix, i))

return ans

}

Python

class Solution:

def findMaxLength(self, nums: List[int]) -> int:

d = {}

ans = 0

for i, v in enumerate(nums):

s = 1 if v == 1 else -1

if s in d:

ans = max(ans, i - d[s])

else:

d[s] = i

return ans

Python

class Solution:

def findMaxLength(self, nums: List[int]) -> int:

s = ans = 0

mp = {}

for i, v in enumerate(nums):

s = 1 if v == 1 else -1

if s in mp:

ans = max(ans, i - mp[s])

else:

mp[s] = i

return ans

C++

C++

class Solution {

public:

int findMaxLength(vector<int>& nums) {

int ans = 0;

int prefix = 0;

unordered_map<int, int> prefixIndices{{0, -1}};

for (int i = 0; i < nums.size(); ++i) {

prefix = nums[i] ? 1 : -1;

if (count auto it = prefixIndices.find(prefix);

it != prefixIndices.end())

ans = max(ans, i - it->second);

else

prefixIndices[prefix] = i;

}

return ans;

}

C++

class Solution {

public:

int findMaxLength(vector<int>& nums) {

unordered_map<int, int> mp;

int s = 0, ans = 0;

mp[0] = -1;

for (int i = 0; i < nums.size(); ++i) {

s += nums[i] ? 1 : -1;

if (d.contains(s)) {

ans = max(ans, i - d[s]);

} else {

d[s] = i;

}

}

return ans;

}

C++

class Solution {

public:

int findMaxLength(vector<int>& nums) {

unordered_map<int, int> mp;

int s = 0, ans = 0;

mp[0] = -1;

for (int i = 0; i < nums.size(); ++i) {

s += nums[i] ? 1 : -1;

if (mp.contains(s))

ans = Math.max(ans, i - mp[s]);

else

mp[s] = i;

}

return ans;

}

Java

Java

```
class Solution {
    public int findMaxLength(int[] nums) {
        int ans = 0;
        Map<Integer, Integer> prefixIndices = new HashMap<>();
        prefixIndices.put(0, -1);
        for (int i = 0; i < nums.length; ++i) {
            prefix = nums[i] == 1 ? 1 : -1;
            if (prefixIndices.containsKey(prefix))
                ans = Math.max(ans, i - prefixIndices.get(prefix));
            else
                prefixIndices.put(prefix, i);
        }
        return ans;
    }
}
```

Java

class Solution {

public int findMaxLength(int[] nums) {

Map<Integer, Integer> mp = new HashMap<>();

d.put(0, -1);

int ans = 0, s = 0;

for (int i = 0; i < nums.length; ++i) {

s += nums[i] == 1 ? 1 : -1;

if (d.containsKey(s)) {

ans = Math.max(ans, i - d.get(s));

} else {

d.put(s, i);

}

}

return ans;

}

Java

class Solution {

public int findMaxLength(int[] nums) {

Map<Integer, Integer> mp = new HashMap<>();

mp.put(0, -1);

int s = 0, ans = 0;

for (int i = 0; i < nums.length; ++i) {

s += nums[i] == 1 ? 1 : -1;

if (mp.containsKey(s)) {

ans = Math.max(ans, i - mp.get(s));

} else {

mp.put(s, i);

}

}

return ans;

}

JavaScript

JavaScript

```
// @param {number[]} nums
// @return {number}
var findMaxLength = function(nums) {
    const d = { 0: -1 };
    let ans = 0;
    let s = 0;
    for (let i = 0; i < nums.length; ++i) {
        s += nums[i] ? 1 : -1;
        if (d.hasOwnProperty(s)) {
            ans = Math.max(ans, i - d[s]);
        } else {
            d[s] = i;
        }
    }
    return ans;
};
```

JavaScript

// @param {number[]} nums
// @return {number}
var findMaxLength = function(nums) {
 const mp = new Map();
 mp.set(0, -1);
 let s = 0;
 let ans = 0;
 for (let i = 0; i < nums.length; ++i) {
 s += nums[i] == 1 ? 1 : -1;
 if (mp.has(s)) ans = Math.max(ans, i - mp.get(s));
 else mp.set(s, i);
 }
 return ans;
};

TypeScript

function findMaxLength(nums: number[]): number {
 const d: Record<number, number> = { 0: -1 };
 let ans = 0;
 let s = 0;
 for (let i = 0; i < nums.length; ++i) {
 s += nums[i] ? 1 : -1;
 if (d.hasOwnProperty(s)) {
 ans = Math.max(ans, i - d[s]);
 } else {
 d[s] = i;
 }
 }
 return ans;
}

TypeScript

```

function findMaxLength(nums: number[]): number {
    const d: Record<number, number> = { 0: -1 };
    let ans = 0;
    let s = 0;
    for (let i = 0; i < nums.length; i++) {
        s += nums[i] > 0 ? 1 : -1;
        if (d.hasOwnProperty(s)) {
            ans = Math.max(ans, i - d[s]);
        } else {
            d[s] = i;
        }
    }
    return ans;
}

```

Go

```

func findMaxLength(nums []int) int {
    d := map[int]int{-1: -1}
    ans, s := 0, 0
    for i, v := range nums {
        if v == 0 {
            s += -1
        }
        s += v
        if j, ok := d[s]; ok {
            ans = max(ans, i-j)
        } else {
            d[s] = i
        }
    }
    return ans
}

```

Go

```

func findMaxLength(nums []int) int {
    mp := map[int]int{-1: -1}
    ans, s := 0, 0
    for i, v := range nums {
        if v == 0 {
            s += -1
        }
        s += v
        if j, ok := mp[s]; ok {
            ans = max(ans, i-j)
        } else {
            mp[s] = i
        }
    }
    return ans
}

```

[*] Arrays & Hashing — Pattern Solution (stored optimal)

Python

Arrays & Hashing · LeetCode — By Pattern — 1057 — LeetCode

```

# Python solution for Contiguous Array
def findMaxLength(nums):
    # Dictionary to store the earliest occurrence of each prefix sum
    prefix_indices = {0: -1}
    prefix_sum = 0
    max_length = 0

    # Iterate over the array
    for index, num in enumerate(nums):
        # Convert 0 to -1, keep 1 as is
        if num == 0:
            prefix_sum -= 1
        else:
            prefix_sum += 1

        # If the prefix sum has been seen before, a balanced subarray exists
        if prefix_sum in prefix_indices:
            # Calculate the length of the subarray
            current_length = index - prefix_indices[prefix_sum]
            max_length = max(max_length, current_length)
        else:
            # Store the index for this prefix sum if not already seen
            prefix_indices[prefix_sum] = index

    # Return the maximum length of balanced subarray
    return max_length

# Example usage:
# print(findMaxLength([0,1,0])) # Expected output: 2

```

C++

```

// C++ solution for Contiguous Array
#include <vector>
#include <unordered_map>
#include <algorithm>
using namespace std;

class Solution {
public:
    int findMaxLength(vector<int>& nums) {
        // Map to store the first occurrence index of each prefix sum
        unordered_map<int, int> prefixIndices;
        // Initialize with prefix sum zero at index -1
        prefixIndices[0] = -1;
        int prefixSum = 0;
        int maxLength = 0;

        // Iterate over the array
        for (int i = 0; i < nums.size(); i++) {
            // Convert 0 to -1, keep 1 as is
            prefixSum += (nums[i] == 0 ? -1 : 1);

            // If prefix sum has been seen before, update maximum subarray length
            if (prefixIndices.find(prefixSum) != prefixIndices.end()) {
                maxLength = max(maxLength, i - prefixIndices[prefixSum]);
            }
        }

        return maxLength;
    }
};

```

Arrays & Hashing · LeetCode — By Pattern — 1058 — LeetCode

```

        } else {
            // Store the index of the first occurrence of this prefix sum
            prefixIndices[prefixSum] = i;
        }
    }
    return maxLength;
}

```

#560 MEDIUM

Subarray Sum Equals K

LeetCode · Simplest · BruteCC · LeetCode · Editorial

Array · Hash Table · Prefix Sum

Lists · Grid · 169

Problem:
Given an array of integers `nums` and an integer `k`, return the total number of subarrays whose sum equals to `k`.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:
Input: `nums = [1,1,1]`, `k = 2`
Output: 2

Example 2:
Input: `nums = [1,2,3]`, `k = 3`
Output: 2

Constraints:

- `1 <= nums.length <= 2 * 104`
- `-1000 <= nums[i] <= 1000`
- `-107 <= k <= 107`

>> Brute Force [Arrays & Hashing] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        # Brute Force O(n^2) - compute sum of every contiguous subarray
        count = 0
        for i in range(len(nums)):
            total = 0
            for j in range(i, len(nums)):
                total += nums[j]
                if total == k:
                    count += 1
            return count

```

Python

Arrays & Hashing · LeetCode — By Pattern — 1059 — LeetCode

```

# Function to calculate the number of subarrays that sum to k
def subarraySum(nums, k):
    # Dictionary to store frequency of prefix sums
    prefix_counts = {}
    # Initialize with prefix sum 0
    current_sum = 0
    # Counting subarray frequency
    count = 0
    # Iterate over the array
    for num in nums:
        current_sum += num
        # Update the running (prefix) sum
        # Check if there is a prefix sum that differs from current_sum by k
        count += prefix_counts.get(current_sum - k, 0)
        # Update the frequency count for the current sum
        prefix_counts[current_sum] = prefix_counts.get(current_sum, 0) + 1
    return count

# Example usage:
print(subarraySum([1,1,1], 2)) # Expected output: 2

```

Python

```

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = collections.Counter({0: 1})

        for num in nums:
            prefix += num
            ans += count[prefix - k]
            count[prefix] += 1

        return ans

```

Python

```

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = 0
        for x in nums:
            s += x
            ans += cnt[s - k]
            cnt[s] += 1
        return ans

```

Python

Arrays & Hashing · LeetCode — By Pattern — 1060 — LeetCode

```

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = 0
        for num in nums:
            s += num
            ans += cnt[s - k]
            cnt[s] += 1
        return ans

# Brute Force O(n^2) - compute sum of every contiguous subarray
count = 0
for i in range(len(nums)):
    total = 0
    for j in range(i, len(nums)):
        total += nums[j]
        if total == k:
            count += 1
    return count

```

Python

```

# Brute Force O(n^2) - compute sum of every contiguous subarray
count = 0
for i in range(len(nums)):
    total = 0
    for j in range(i, len(nums)):
        total += nums[j]
        if total == k:
            count += 1
    return count

```

C++

```

class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        int ans = 0;
        int prefix = 0;
        unordered_map<int, int> prefix_counts{{0, 1}};

        for (int i = 0; i < nums.size(); i++) {
            prefix += nums[i];
            ans += prefix_counts[prefix - k];
            prefix_counts[prefix]++;
        }

        return ans;
    }
};

```

C++

Arrays & Hashing · LeetCode — By Pattern — 1061 — LeetCode

```

class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        int ans = 0;
        int prefix = 0;
        unordered_map<int, int> prefix_counts{{0, 1}};

        for (int i = 0; i < nums.size(); i++) {
            prefix += nums[i];
            ans += prefix_counts[prefix - k];
            prefix_counts[prefix]++;
        }

        return ans;
    }
};

```

C++

```

class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        int ans = 0;
        int prefix = 0;
        unordered_map<int, int> prefix_counts{{0, 1}};

        for (int i = 0; i < nums.size(); i++) {
            prefix += nums[i];
            ans += prefix_counts[prefix - k];
            prefix_counts[prefix]++;
        }

        return ans;
    }
};

```

Java

```

class Solution {
    public int subarraySum(int[] nums, int k) {
        int ans = 0;
        int prefix = 0;
        Map<Integer, Integer> count = new HashMap<>();
        count.put(0, 1);

        for (int i = 0; i < nums.length; i++) {
            prefix += nums[i];
            ans += count.getOrDefault(prefix - k, 0);
            count.put(prefix, count.getOrDefault(prefix, 0) + 1);
        }

        return ans;
    }
}

```

Java

Arrays & Hashing · LeetCode — By Pattern — 1062 — LeetCode

```

class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> cnt = new HashMap<>();
        cnt.put(0, 1);
        int ans = 0, s = 0;
        for (int x : nums) {
            s += x;
            ans += cnt.getOrDefault(s - k, 0);
            cnt.merge(s, 1, Integer::sum);
        }
        return ans;
    }
}

```

Java

```

use std::collections::HashMap;

impl Solution {
    pub fn subarray_sum(nums: Vec<i32>, k: i32) -> i32 {
        let mut cnt = HashMap::new();
        cnt.insert(0, 1);
        let mut ans = 0;
        let mut s = 0;
        for x in nums {
            s += x;
            if let Some(v) = cnt.get(&s - k) {
                ans += *v;
            }
            cnt.entry(s).or_insert(0) += 1;
        }
        ans
    }
}

```

Java

```

class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> counter = new HashMap<>();
        counter.put(0, 1);
        int ans = 0, s = 0;
        for (int num : nums) {
            s += num;
            ans += counter.getOrDefault(s - k, 0);
            counter.put(s, counter.getOrDefault(s, 0) + 1);
        }
        return ans;
    }
}

```

Java

```

use std::collections::HashMap;

impl Solution {
    pub fn subarray_sum(nums: Vec<i32>, k: i32) -> i32 {
        let mut res = 0;
        let mut s = 0;
        let mut map = HashMap::new();
        map.insert(0, 1);
        num.iter().for_each(|num| {
            s += num;
            res += map.get(&(s - k)).unwrap_or(0);
            map.insert(s, map.get(s).unwrap_or(0) + 1);
        });
        res
    }
}

```

TypeScript

TypeScript

```

function subarraySum(nums: number[], k: number): number {
    const cnt: Map<number, number> = new Map();
    cnt.set(0, 1);
    let [ans, s] = [0, 0];
    for (const x of nums) {
        s += x;
        ans += cnt.get(s - k) || 0;
        cnt.set(s, (cnt.get(s) || 0) + 1);
    }
    return ans;
}

```

TypeScript

```

function subarraySum(nums: number[], k: number): number {
    let ans = 0;
    s = 0;
    const counter = new Map();
    counter.set(0, 1);
    for (const num of nums) {
        s += num;
        ans += counter.get(s - k) || 0;
        counter.set(s, (counter.get(s) || 0) + 1);
    }
    return ans;
}

```

Go

Go

```

func subarraySum(nums []int, k int) ans int {
    cnt := map[int]int{0: 1}
    s := 0
    for i, v := range nums {
        s += v
        ans += cnt[s-k]
        cnt[s]++
    }
    return ans
}

```

Go

```

func subarraySum(nums []int, k int) int {
    counter := map[int]int{0: 1}
    ans, s := 0, 0
    for i, num := range nums {
        s += num
        ans += counter[s-k]
        counter[s]++
    }
    return ans
}

```

[*] Arrays & Hashing — Pattern Solution (matched from community answers)

```

Python
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = collections.Counter([0])

        for num in nums:
            prefix += num
            ans += count[prefix - k]
            count[prefix] += 1

        return ans

```

#1010 MEDIUM

Pairs of Songs With Total Durations Divisible by 60

LeetCode · Sum of Subarray Minimums · LeetCode · Editorial

Array · Hash Table · Counting

List: Code Snippet

Problem:

You are given a list of songs where the i th song has a duration of $time[i]$ seconds.

Return the number of pairs of songs for which their total duration in seconds is divisible by 60.

Formally, we want the number of indices i, j such that $i < j$ with $(time[i] + time[j]) \% 60 == 0$.

Example 1:

Input: time = [30,20,150,100,40]
Output: 3
Explanation: Three pairs have a total duration divisible by 60:
(time[0] = 30, time[2] = 150): total duration 180
(time[1] = 20, time[3] = 100): total duration 120
(time[1] = 20, time[4] = 40): total duration 60

Example 2:

Input: time = [60,60,60]
Output: 3
Explanation: All three pairs have a total duration of 120, which is divisible by 60.

Constraints:

- $1 \leq \text{time.length} \leq 6 \cdot 10^4$
- $1 \leq \text{time}[i] \leq 500$

>> Brute Force [Arrays & Hashing] Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        # Brute Force Approach - Check all pairs without a hash map
        n = len(time)
        for i in range(n):
            for j in range(i + 1, n):
                # Check if pair (i, j) is divisible by 60
                if (time[i] + time[j]) % 60 == 0:
                    pair_count += 1
        return pair_count

```

Python

Python

```

# Python solution for counting pairs of songs whose duration sum are divisible by 60
def numPairsDivisibleBy60(time):
    # Frequency array for remainder 0-59
    remainder_freq = [0] * 60
    pair_count = 0

    # Loop over each song duration
    for t in time:
        # Calculate the remainder when dividing by 60
        remainder = t % 60
        # Calculate the complementary remainder needed for the sum to be divisible by 60
        complement = (60 - remainder) % 60
        # Increase pair count by frequency of complement sum so far
        pair_count += remainder_freq[complement]
        # Update the frequency counter for the current remainder
        remainder_freq[remainder] += 1

    return pair_count

# Example usage
# print(numPairsDivisibleBy60([30,20,150,100,40])) # Expected output: 3

```

Python

```

class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        ans = 0
        count = [0] * 60

        for t in time:
            t %= 60
            ans += count[(60 - t) % 60]
            count[t] += 1

        return ans

```

Python

```

class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        cnt = Counter()
        ans = 0
        for x in time:
            x %= 60
            y = (60 - x) % 60
            ans += cnt[y]
            cnt[x] += 1
        return ans

```

Python

```

class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        cnt = Counter([0] * 60)
        ans = 0
        for x in time:
            x %= 60
            ans += cnt[(60 - x) % 60]
            cnt[x] += 1
        return ans

```

C++

C++

```

class Solution {
public:
    int numPairsDivisibleBy60(vector<int>& time) {
        int ans = 0;
        vector<int> count(60);

        for (int t : time) {
            t %= 60;
            ans += count[(60 - t) % 60];
            ++count[t];
        }

        return ans;
    }
};

```

C++

```

class Solution {
public:
    int numPairsDivisibleBy60(vector<int>& time) {
        int cnt(0);
        int ans = 0;
        for (int x : time) {
            x %= 60;
            int y = (60 - x) % 60;
            ans += cnt[y];
            ++cnt[x];
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int numPairsDivisibleBy60(vector<int>& time) {
        int cnt(0);
        for (int x : time) {
            ++cnt[x % 60];
        }
        int ans = 0;
        for (int i = 1; i < 60; ++i) {
            ans += cnt[i] * cnt[60 - i];
        }
        ans += cnt[0] * cnt[0] / 2;
        return ans;
    }
};

```

Java

Java

```

class Solution {
    public int numPairsDivisibleBy60(int[] time) {
        int ans = 0;
        int[] count = new int[60];

        for (int t : time) {
            t %= 60;
            ans += count[(60 - t) % 60];
            ++count[t];
        }

        return ans;
    }
}

```

Java

```

class Solution {
public: int numPairsDivisibleBy60(int[] time) {
    int[] cnt = new int[60];
    int ans = 0;
    for (int i = 0; i < time.length; i++) {
        int x = time[i] % 60;
        ans += cnt[(60 - x) % 60];
        cnt[x]++;
    }
    return ans;
}
}

```

Java

```

class Solution {
public: int numPairsDivisibleBy60(int[] time) {
    int[] cnt = new int[60];
    for (int i = 0; i < time.length; i++) {
        int x = time[i] % 60;
        ans += cnt[(60 - x) % 60];
        cnt[x]++;
    }
    return ans;
}
}

```

TypeScript

```

TypeScript
function numPairsDivisibleBy60(time: number[]): number {
    const cnt: number[] = new Array(60).fill(0);
    let ans: number = 0;
    for (let i of time) {
        i %= 60;
        const y = (60 - i) % 60;
        ans += cnt[y];
        cnt[i]++;
    }
    return ans;
}

```

TypeScript

```

function numPairsDivisibleBy60(time: number[]): number {
    const cnt: number[] = new Array(60).fill(0);
    for (const i of time) {
        i %= 60;
    }
    let ans = 0;
    for (let i = 1; i < 30; i++) {
        ans += cnt[i] * cnt[60 - i];
    }
    ans += (cnt[0] + (cnt[30] - 1)) / 2;
    ans += (cnt[30] + (cnt[30] - 1)) / 2;
    return ans;
}

```

Go

```

func numPairsDivisibleBy60(time []int) (ans int) {
    cnt := [60]int{}
    for _, i := range time {
        i %= 60
        y := (60 - i) % 60
        ans += cnt[y]
        cnt[i]++
    }
    return
}

```

Go

```

func numPairsDivisibleBy60(time []int) (ans int) {
    cnt := [60]int{}
    for _, i := range time {
        cnt[i%60]++
    }
    for x := 1; x < 30; x++ {
        ans += cnt[x] * cnt[60-x]
    }
    ans += cnt[0] * (cnt[0] - 1) / 2
    ans += cnt[30] * (cnt[30] - 1) / 2
    return
}

```

Rust

Rust

```

impl Solution {
    pub fn num_pairs_divisible_by60(time: Vec<i32>) -> i32 {
        let mut cnt = [0; 60];
        let mut ans: i32 = 0;
        for mut i in time {
            i %= 60;
            let y = (60 - i) % 60;
            ans += cnt[y as usize];
            cnt[i as usize] += 1;
        }
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn num_pairs_divisible_by60(time: Vec<i32>) -> i32 {
        let mut cnt = [0; 60];
        let mut ans: i32 = 0;
        for mut i in time {
            i %= 60;
            let y = (60 - i) % 60;
            ans += cnt[y as usize];
            cnt[i as usize] += 1;
        }
        ans
    }
}

```

[1] Arrays & Hashing - Pattern Solution (matched from community answers)

Python

```

class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        cnt = Counter()
        ans = 0
        for i in time:
            i %= 60
            y = (60 - i) % 60
            ans += cnt[y]
            cnt[i] += 1
        return ans

```

#1272

MEDIUM

Remove Interval

LeetCode - SimplifyLeetCode - WalkCC - LeetCode - Editorial

Array

LeetCode Premium 98

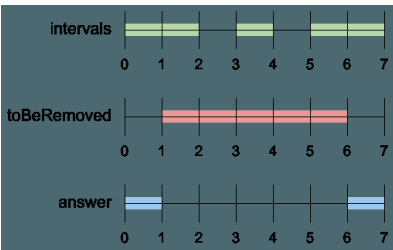
Problem:

A set of real numbers can be represented as the union of several disjoint intervals, where each interval is in the form [a, b]. A real number x is in the set if one of its intervals [a, b] contains x (i.e. a ≤ x ≤ b).

You are given a sorted list of disjoint intervals representing a set of real numbers as described above, where intervals[i] = [a, b] represents the interval [a, b]. You are also given another interval toBeRemoved.

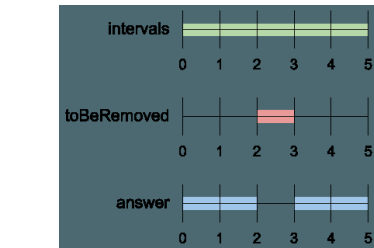
Return the set of real numbers with the interval toBeRemoved removed from intervals. In other words, return the set of real numbers such that every x in the set is in intervals but not in toBeRemoved. Your answer should be a sorted list of disjoint intervals as described above.

Example 1:



Input: intervals = [[0,2],[3,4],[5,7]], toBeRemoved = [1,6]
Output: [[0,1],[6,7]]

Example 2:



Input: intervals = [[0,5]], toBeRemoved = [2,3]
Output: [[0,2],[3,5]]

Example 3:

Input: intervals = [[-5,-4],[-3,-2],[1,2],[3,5],[8,9]], toBeRemoved = [-1,4]
Output: [[-5,-4],[-3,-2],[4,5],[8,9]]

Constraints:

- 1 ≤ intervals.length ≤ 104
- 109 ≤ ai < bi ≤ 109

>> Brute Force [Arrays & Hashing] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) -> List[List[int]]:
        # Brute Force O(N^2) - check all pairs without a hash map
        n = len(intervals)
        for i in range(n):
            for j in range(i + 1, n):
                # remove pair intervals[i], intervals[j]
                pass
        return [] # adjust return

```

Python

Python

```

def removeInterval(intervals, toBeRemoved):
    # Remove the 'remove' interval boundaries.
    remove_start, remove_end = toBeRemoved
    result = []

    # Iterate over each interval.
    for interval in intervals:
        start, end = interval

        # If the interval is completely outside toBeRemoved, add as is.
        if end <= remove_start or start >= remove_end:
            result.append([start, end])
        else:
            # If there is a non-overlapping left part, add it.
            if start < remove_start:
                result.append([start, remove_start])
            # If there is a non-overlapping right part, add it.
            if end > remove_end:
                result.append([remove_end, end])

    return result

# Example usage:
intervals = [[0,2],[3,4],[5,7]]
toBeRemoved = [1,6]

print(removeInterval(intervals, toBeRemoved)) # Expected output: [[0,1],[6,7]]

```

Python

```

class Solution:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) -> List[List[int]]:
        ans = []
        for a, b in intervals:
            if a < toBeRemoved[0] and b <= toBeRemoved[0]:
                ans.append([a, b])
            elif a <= toBeRemoved[0] and b < toBeRemoved[1]:
                ans.append([a, toBeRemoved[0]])
            elif a < toBeRemoved[1] and b > toBeRemoved[1]:
                ans.append([toBeRemoved[1], b])
            else:
                continue
        return ans

```

Python

```

class Solution:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) -> List[List[int]]:
        ans = []
        for a, b in intervals:
            if a == y or b == x:
                ans.append([a, b])
            else:
                if a < x:
                    ans.append([a, x])
                if b > y:
                    ans.append([y, b])
        return ans

```

Python

```

//
// 1, 2, 3, 4, 5]
//
// x = 2, y = 4
//
// Invalid: must repeat add logic!
// File "stdout", line 1, in <module>
// ValueError: too many values to unpack
// x = 2, y = 4, b = 4
//
// x = 2
//
// y = 4
//
// x = 2
//
// y = 4
//
//

```

```

class Solution:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) -> List[List[int]]:
        ans = []
        for a, b in intervals:
            if a == y or b == x:
                ans.append([a, b])

```

```

        else:
            if a == x:
                ans.append([a, x]) # if in this else block, meaning b < x, so no need to append
            if b == y:
                ans.append([y, b])
            # if x < a < b < y, then delete [a,b], do nothing and skip here
            return ans

```

C++

C++

```

class Solution {
public:
    vector<vector<int>> removeInterval(vector<vector<int>>& intervals, vector<int>& toBeRemoved) {
        vector<vector<int>> ans;
        for (const vector<int>& interval : intervals) {
            const int a = interval[0];
            const int b = interval[1];
            if (a == toBeRemoved[0] || b == toBeRemoved[1]) {
                ans.push_back(interval);
            } else if (a < toBeRemoved[0] && b > toBeRemoved[1]) {
                if (a < toBeRemoved[0]) {
                    ans.push_back({a, toBeRemoved[0]});
                }
                if (b > toBeRemoved[1]) {
                    ans.push_back({toBeRemoved[1], b});
                }
            }
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> removeInterval(vector<vector<int>>& intervals, vector<int>& toBeRemoved) {
        int x = toBeRemoved[0], y = toBeRemoved[1];
        vector<vector<int>> ans;
        for (auto& e : intervals) {
            int a = e[0], b = e[1];
            if (a == y || b == x) {
                ans.push_back(e);
            } else {
                if (a < x) {
                    ans.push_back({a, x});
                }
                if (b > y) {
                    ans.push_back({y, b});
                }
            }
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> removeInterval(vector<vector<int>>& intervals, vector<int>& toBeRemoved) {
        int x = toBeRemoved[0], y = toBeRemoved[1];
        vector<vector<int>> ans;
        for (auto& e : intervals) {
            int a = e[0], b = e[1];
            if (a == y || b == x) {
                ans.push_back(e);
            } else {
                if (a < x) {
                    ans.push_back({a, x});
                }
                if (b > y) {
                    ans.push_back({y, b});
                }
            }
        }
        return ans;
    }
};

```

Java

Java

```

class Solution {
    public List<List<Integer>> removeInterval(int[][] intervals, int[] toBeRemoved) {
        List<List<Integer>> ans = new ArrayList<>();
        for (int[] interval : intervals) {
            final int a = interval[0];
            final int b = interval[1];
            if (a == toBeRemoved[0] || b == toBeRemoved[1]) {
                ans.add(Arrays.asList(a, b));
            } else if (a < toBeRemoved[0] && b > toBeRemoved[1]) {
                ans.add(Arrays.asList(a, toBeRemoved[0]));
                ans.add(Arrays.asList(toBeRemoved[1], b));
            }
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public List<List<Integer>> removeInterval(int[][] intervals, int[] toBeRemoved) {
        List<List<Integer>> ans = new ArrayList<>();
        for (int[] interval : intervals) {
            int a = interval[0], b = interval[1];
            if (a == y || b == x) {
                ans.add(Arrays.asList(a, b));
            } else {
                if (a < x) {
                    ans.add(Arrays.asList(a, x));
                }
                if (b > y) {
                    ans.add(Arrays.asList(y, b));
                }
            }
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public List<List<Integer>> removeInterval(int[][] intervals, int[] toBeRemoved) {
        int x = toBeRemoved[0], y = toBeRemoved[1];
        List<List<Integer>> ans = new ArrayList<>();
        for (int[] e : intervals) {
            int a = e[0], b = e[1];
            if (a == y || b == x) {
                ans.add(Arrays.asList(a, b));
            } else {
                if (a < x) {
                    ans.add(Arrays.asList(a, x));
                }
                if (b > y) {
                    ans.add(Arrays.asList(y, b));
                }
            }
        }
        return ans;
    }
}

```

Go

Go

```

func removeInterval(intervals [][]int, toBeRemoved []int) ans [][]int {
    x, y := toBeRemoved[0], toBeRemoved[1]
    for _, e := range intervals {
        a, b := e[0], e[1]
        if a == y || b == x {
            ans = append(ans, e)
        } else {
            if a < x {
                ans = append(ans, []int{a, x})
            }
            if b > y {
                ans = append(ans, []int{y, b})
            }
        }
    }
    return ans
}

```

Go

```

func removeInterval(intervals [][]int, toBeRemoved []int) ans [][]int {
    x, y := toBeRemoved[0], toBeRemoved[1]
    for _, e := range intervals {
        a, b := e[0], e[1]
        if a == y || b == x {
            ans = append(ans, e)
        } else {
            if a < x {
                ans = append(ans, []int{a, x})
            }
            if b > y {
                ans = append(ans, []int{y, b})
            }
        }
    }
    return ans
}

```

[7] Arrays & Hashing — Pattern Solution (stored optimal)

Python

```

def removeInterval(intervals, toBeRemoved):
    # Remove the remove interval boundaries.
    remove_start, remove_end = toBeRemoved
    result = []

    # Iterate over each interval.
    for interval in intervals:
        start, end = interval

        # If the interval is completely outside toBeRemoved, add it to ans.
        if end < remove_start or start > remove_end:
            result.append([start, end])
        else:
            # If there is a non-overlapping left part, add it.
            if start < remove_start:
                result.append([start, remove_start])
            # If there is a non-overlapping right part, add it.
            if end > remove_end:
                result.append([remove_end, end])

    return result

# Example usage:
intervals = [[0,2],[3,4],[5,7]]
toBeRemoved = [1,5]

print(removeInterval(intervals, toBeRemoved)) # Expected output: [[0,1],[6,7]]

```

C++


```
// C++ solution to remove the specified interval from a list of intervals.
// Input:
// intervals = [[1,2],[3,4],[5,6],[7,8]]
// removeInterval = [2,5]
// Output:
// [[1,2],[3,4],[7,8]]

class Solution {
public:
    vector<vector<int>> removeInterval(vector<vector<int>>& intervals, vector<int>& toRemove) {
        int removedStart = toRemove[0]; removedEnd = toRemove[1];
        vector<vector<int>> result;

        // Iterate over each interval.
        for (auto interval : intervals) {
            int start = interval[0]; end = interval[1];

            // If interval is completely outside of toRemove, add it as is.
            if (end <= removedStart || start >= removedEnd) {
                result.push_back(interval);
            } else {
                // If there is a left portion not affected by removal.
                if (start < removedStart) {
                    result.push_back({start, removedStart});
                }
                // If there is a right portion not affected by removal.
                if (end > removedEnd) {
                    result.push_back({removedEnd, end});
                }
            }
        }
        return result;
    }
};

// Example usage:
// int main() {
//     Solution sol;
//     vector<vector<int>> intervals = {{0,2},{3,4},{5,7}};
//     vector<int> toRemove = {1,4};
//     vector<vector<int>> res = sol.removeInterval(intervals, toRemove);
//     // Expected output: {{0,1},{5,7}}
//     return 0;
// }
```

#1429 MEDIUM

First Unique Number

LeetCode · SimplifyLeetCode · WalkCC · LeetCode · Editorial

Array · Hash Table · Design · Queue · Data Stream

LeetCode Premium 98

Problem:

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the FirstUnique class:

- FirstUnique(int[] nums) Initializes the object with the numbers in the queue.

Arrays & Hashing · LeetCodeMastery — By Pattern

— 1081 —

LeetCodeMastery

- int showFirstUnique() returns the value of the first unique integer of the queue, and returns -1 if there is no such integer.
- void add(int value) insert value to the queue.

Example 1:

Input:
["FirstUnique", "showFirstUnique", "add", "showFirstUnique", "add", "showFirstUnique", "add", "showFirstUnique"]
[[1, 2, 3, 1], [1], [5], [1], [2], [1], [3], [1]]
Output:
[null, 2, null, 2, null, 3, null, -1]
Explanation:
FirstUnique firstUnique = new FirstUnique([2,3,5]);
firstUnique.showFirstUnique(); // return 2
firstUnique.add(5); // the queue is now [2,3,5,5]
firstUnique.showFirstUnique(); // return 2
firstUnique.add(2); // the queue is now [2,3,5,5,2]
firstUnique.showFirstUnique(); // return 3
firstUnique.add(3); // the queue is now [2,3,5,5,2,3]
firstUnique.showFirstUnique(); // return -1

Example 2:

Input:
["FirstUnique", "showFirstUnique", "add", "add", "add", "add", "add", "showFirstUnique"]
[[7, 7, 7, 7, 7, 1], [1], [7], [2], [3], [3], [7], [2], [1]]
Output:
[null, -1, null, null, null, null, null, 17]
Explanation:
FirstUnique firstUnique = new FirstUnique([7,7,7,7,7,7]);
firstUnique.showFirstUnique(); // return -1
firstUnique.add(7); // the queue is now [7,7,7,7,7,7,7]
firstUnique.add(3); // the queue is now [7,7,7,7,7,7,7,3]
firstUnique.add(3); // the queue is now [7,7,7,7,7,7,7,3,3]
firstUnique.add(7); // the queue is now [7,7,7,7,7,7,7,3,3,7]
firstUnique.add(17); // the queue is now [7,7,7,7,7,7,7,3,3,7,17]
firstUnique.showFirstUnique(); // return 17

Example 3:

Input:
["FirstUnique", "showFirstUnique", "add", "showFirstUnique"]
[[[809], [1], [809], [1]]]
Output:
[null, 809, null, -1]
Explanation:
FirstUnique firstUnique = new FirstUnique([809]);
firstUnique.showFirstUnique(); // return 809
firstUnique.add(809); // the queue is now [809,809]
firstUnique.showFirstUnique(); // return -1

Constraints:

- 1 <= nums.length <= 10⁵
- 1 <= nums[i] <= 10⁴

Arrays & Hashing · LeetCodeMastery — By Pattern

— 1082 —

LeetCodeMastery

- 1 <= value <= 10⁴
- At most 50000 calls will be made to showFirstUnique and add.

Python

Python

```
import collections

class FirstUnique:
    # Initialize the object with the numbers in the queue.
    def __init__(self, nums):
        # Using defaultdict for efficient peek/pop operations.
        self.queue = collections.deque()
        # Dictionary to store frequency count.
        self.count = collections.defaultdict(int)
        # Process the initial list of numbers.
        for num in nums:
            self.add(num)

    # Returns the value of the first unique integer of the queue.
    def showFirstUnique(self):
        # Remove numbers from the front if they are no longer unique.
        while self.queue and self.count[self.queue[0]] > 1:
            self.queue.popleft()
        # Return the first unique number if available.
        return self.queue[0] if self.queue else -1

    # Inserts value to the queue.
    def add(self, value):
        self.count[value] += 1
        # If it is the first occurrence, append it to the queue.
        if self.count[value] == 1:
            self.queue.append(value)
```

Python

```
class FirstUnique:
    def __init__(self, nums: List[int]):
        self.queue = deque()
        self.unique = {}
        for num in nums:
            self.add(num)

    def showFirstUnique(self) -> int:
        return next(iter(self.unique), -1)

    def add(self, value: int) -> None:
        if value not in self.unique:
            self.queue.append(value)
            self.unique[value] = 1
        elif value in self.queue:
            # We have added this value before, and this is the second time we're
            # adding it. So, remove the value from 'unique' and mark it as erased.
            self.unique.pop(value)
```

Python

Arrays & Hashing · LeetCodeMastery — By Pattern

— 1083 —

LeetCodeMastery

```
class FirstUnique {
    def __init__(self, nums: List[int]):
        self.queue = collections.deque()
        self.unique = defaultdict(int)
        for num in nums:
            self.add(num)

    def showFirstUnique(self) -> int:
        return next(iter(self.unique), -1)

    def add(self, value: int) -> None:
        self.queue.append(value)
        if self.queue[0] in self.unique:
            self.unique[self.queue[0]] += 1
            if self.unique[self.queue[0]] > 1:
                self.queue.popleft()
        else:
            self.unique[self.queue[0]] = 1
```

Your FirstUnique object will be instantiated and called as such:

```
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)
```

Python

```
class FirstUnique:
    def __init__(self, nums: List[int]):
        self.queue = collections.deque()
        self.unique = defaultdict(int)
        for num in nums:
            self.add(num)

    def showFirstUnique(self) -> int:
        return next(iter(self.unique), -1)

    def add(self, value: int) -> None:
        self.queue.append(value)
        if self.queue[0] in self.unique:
            self.unique[self.queue[0]] += 1
            if self.unique[self.queue[0]] > 1:
                self.queue.popleft()
        else:
            self.unique[self.queue[0]] = 1
```

Your FirstUnique object will be instantiated and called as such:

```
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)
```

C++

C++

Arrays & Hashing · LeetCodeMastery — By Pattern

— 1084 —

LeetCodeMastery

```
class FirstUnique {
public:
    FirstUnique(vector<int>& nums) {
        for (const int num : nums)
            add(num);
    }

    int showFirstUnique() {
        return unique.empty() ? -1 : unique.front();
    }

    void add(int value) {
        const auto it = valueToIterator.find(value);
        if (it == valueToIterator.end()) {
            unique.push_back(value);
            valueToIterator[value] = prev(unique.end());
        } else if (it->second != unique.end()) {
            // We have added this value before, and this is the second time we're
            // adding it. So, remove the value from 'unique' and mark it as erased.
            unique.erase(it->second);
            valueToIterator[value] = unique.end();
        }
    }

private:
    list<int> unique;
    unordered_map<int, list<int>::iterator> valueToIterator;
};
```

C++

```
class FirstUnique {
public:
    FirstUnique(vector<int>& nums) {
        for (int v : nums) {
            ++cnt[v];
            q.push_back(v);
        }

        int showFirstUnique() {
            while (q.size() && cnt[q.front()] > 1) q.pop_front();
            return q.size() ? q.front() : -1;
        }

        void add(int value) {
            ++cnt[value];
            q.push_back(value);
        }

private:
    unordered_map<int, int> cnt;
    deque<int> q;
};
```

Arrays & Hashing · LeetCodeMastery — By Pattern

— 1085 —

LeetCodeMastery

```
# Your FirstUnique object will be instantiated and called as such:
# FirstUnique obj = new FirstUnique(nums);
# int param_1 = obj.showFirstUnique();
# obj.add(value);
//
```

C++

```
class FirstUnique {
public:
    FirstUnique(vector<int>& nums) {
        for (int v : nums) {
            ++cnt[v];
            q.push_back(v);
        }

        int showFirstUnique() {
            while (q.size() && cnt[q.front()] > 1) q.pop_front();
            return q.size() ? q.front() : -1;
        }

        void add(int value) {
            ++cnt[value];
            q.push_back(value);
        }

private:
    unordered_map<int, int> cnt;
    deque<int> q;
};
```

Your FirstUnique object will be instantiated and called as such:

```
# FirstUnique obj = new FirstUnique(nums);
# int param_1 = obj.showFirstUnique();
# obj.add(value);
//
```

Java

Java

Arrays & Hashing · LeetCodeMastery — By Pattern

— 1086 —

LeetCodeMastery

```

class FirstUnique {
public:
    FirstUnique(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            add(nums[i]);
        }
    }

    public int showFirstUnique() {
        return unique.isEmpty() ? -1 : unique.iterator().next();
    }

    public void add(int value) {
        if (!seen.contains(value)) {
            unique.add(value);
            seen.add(value);
        } else if (unique.contains(value)) {
            // We have added this value before, and this is the second time we're
            // adding it, so we remove the value from unique.
            unique.remove(value);
        }
    }

    private Set<Integer> seen = new HashSet<>();
    private Set<Integer> unique = new LinkedHashSet<>();
}

```

```

// Java
class FirstUnique {
    private Map<Integer, Integer> cnt = new HashMap<>();
    private Set<Integer> unique = new LinkedHashSet<>();

    public FirstUnique(int[] nums) {
        for (int v : nums) {
            cnt.put(v, cnt.getOrDefault(v, 0) + 1);
        }
        for (int v : nums) {
            if (cnt.get(v) == 1) {
                unique.add(v);
            }
        }
    }

    public int showFirstUnique() {
        return unique.isEmpty() ? -1 : unique.iterator().next();
    }

    public void add(int value) {
        cnt.put(value, cnt.getOrDefault(value, 0) + 1);
        if (cnt.get(value) == 1) {
            unique.add(value);
        } else {
            unique.remove(value);
        }
    }
}

```

```

    }
}

/**
 * Your FirstUnique object will be instantiated and called as such:
 * FirstUnique obj = new FirstUnique(nums);
 * int param_1 = obj.showFirstUnique();
 * obj.add(value);
 */

```

```

// Java
class FirstUnique {
    private Map<Integer, Integer> cnt = new HashMap<>();
    private Set<Integer> unique = new LinkedHashSet<>();

    public FirstUnique(int[] nums) {
        for (int v : nums) {
            cnt.put(v, cnt.getOrDefault(v, 0) + 1);
        }
        for (int v : nums) {
            if (cnt.get(v) == 1) {
                unique.add(v);
            }
        }
    }

    public int showFirstUnique() {
        return unique.isEmpty() ? -1 : unique.iterator().next();
    }

    public void add(int value) {
        cnt.put(value, cnt.getOrDefault(value, 0) + 1);
        if (cnt.get(value) == 1) {
            unique.add(value);
        } else {
            unique.remove(value);
        }
    }
}

```

```

/**
 * Your FirstUnique object will be instantiated and called as such:
 * FirstUnique obj = new FirstUnique(nums);
 * int param_1 = obj.showFirstUnique();
 * obj.add(value);
 */

```

```

type FirstUnique struct {
    cnt map[int]int
    q []int
}

func Constructor(nums []int) FirstUnique {
    cnt := map[int]int{}
    for _, v := range nums {
        cnt[v]++
    }
    return FirstUnique{cnt, nums}
}

func (this *FirstUnique) ShowFirstUnique() int {
    for len(this.q) > 0 && this.cnt[this.q[0]] > 1 {
        this.q = this.q[1:]
    }
    if len(this.q) > 0 {
        return this.q[0]
    }
    return -1
}

func (this *FirstUnique) AddValue(int value int) {
    this.cnt[value]++
    this.q = append(this.q, value)
}

/**
 * Your FirstUnique object will be instantiated and called as such:
 * obj := Constructor(nums);
 * param_1 := obj.ShowFirstUnique();
 * obj.Add(value);
 */

```

Go

```

type FirstUnique struct {
    cnt map[int]int
    q []int
}

func Constructor(nums []int) FirstUnique {
    cnt := map[int]int{}
    for _, v := range nums {
        cnt[v]++
    }
    return FirstUnique{cnt, nums}
}

func (this *FirstUnique) ShowFirstUnique() int {
    for len(this.q) > 0 && this.cnt[this.q[0]] > 1 {
        this.q = this.q[1:]
    }
    if len(this.q) > 0 {
        return this.q[0]
    }
    return -1
}

func (this *FirstUnique) AddValue(int value int) {
    this.cnt[value]++
    this.q = append(this.q, value)
}

/**
 * Your FirstUnique object will be instantiated and called as such:
 * obj := Constructor(nums);
 * param_1 := obj.ShowFirstUnique();
 * obj.Add(value);
 */

```

[*] Arrays & Hashing - Pattern Solution (matched from community answers)

Python

```

import collections

class FirstUnique:
    # Initialize the object with the numbers in the queue.
    def __init__(self, nums):
        # Using deque for efficient popLeft operations.
        self.queue = collections.deque()
        # Dictionary to store frequency count.
        self.count = collections.defaultdict(int)
        # From the initial list of numbers.
        for num in nums:
            self.add(num)

    # Return the value of the first unique integer of the queue.
    def showFirstUnique(self):
        # Remove numbers from the front if they are no longer unique.
        while self.queue and self.count[self.queue[0]] > 1:
            self.queue.popleft()
        # Return the first unique number if available.
        return self.queue[0] if self.queue else -1

    # Insert value to the queue.
    def add(self, value):
        self.count[value] += 1
        # If it is the first occurrence, append it to the queue.
        if self.count[value] == 1:
            self.queue.append(value)

```

#3159 **MEDIUM**
Find Occurrences of an Element in an Array
 LeetCode - Simplest - **NaiveCC** - **LeetCode** - Editorial
 Array - Hash Table
 List: CodeSignal

Problem:
 You are given an integer array `nums`, an integer array `queries`, and an integer `x`.
 For each `queries[i]`, you need to find the index of the `queries[i]`th occurrence of `x` in the `nums` array.
 If there are fewer than `queries[i]` occurrences of `x`, the answer should be `-1` for that query.
 Return an integer array answer containing the answers to all queries.

Example 1:
 Input: `nums = [1,3,1,7]`, `queries = [1,3,2,4]`, `x = 1`
 Output: `[0,-1,2,-1]`

Explanation:

- For the 1st query, the first occurrence of 1 is at index 0.
- For the 2nd query, there are only two occurrences of 1 in `nums`, so the answer is `-1`.
- For the 3rd query, the second occurrence of 1 is at index 2.
- For the 4th query, there are only two occurrences of 1 in `nums`, so the answer is `-1`.

Example 2:
 Input: `nums = [1,2,3]`, `queries = [10]`, `x = 5`
 Output: `[-1]`
Explanation:

- For the 1st query, 5 doesn't exist in `nums`, so the answer is `-1`.

Constraints:

- `1 <= nums.length, queries.length <= 105`
- `1 <= queries[i] <= 105`
- `1 <= nums[i], x <= 104`

```

Python
# Problem: To find occurrences of 'x' at specified query indices
def find_occurrences(nums, queries, x):
    # Precompute the indices where 'x' occurs in 'nums'
    positions = []
    for index, num in enumerate(nums):
        if num == x:
            positions.append(index)

    # Process each query
    results = []
    for occurrence in queries:
        # Check if the requested occurrence exists
        if occurrence > len(positions):
            results.append(position[occurrence - 1]) # Convert to zero-based index
        else:
            results.append(-1)

    return results

# Example usage:
nums = [1, 3, 1, 7]
queries = [1, 3, 2, 4]
x = 1
print(find_occurrences(nums, queries, x)) # Output: [0, -1, 2, -1]

```

```

Python
class Solution:
    def occurrencesOfElement(
        self,
        nums: List[int],
        queries: List[int],
        x: int,
    ) -> List[int]:
        indices = []
        for i, num in enumerate(nums):
            if num == x:
                indices.append(i)
        for query in queries:
            if query > len(indices):
                return -1
            else:
                return indices[query - 1]

```

Python

```

class Solution:
    def occurrenceOfElement(self, nums: List[int], queries: List[int], x: int) -> List[int]:
        ans = []
        for i, v in enumerate(nums):
            if v == x:
                return [ans[i - 1] if i > 0 else -1 for i in queries]

```

Python

```

class Solution:
    def occurrenceOfElement(self, nums: List[int], queries: List[int], x: int) -> List[int]:
        ans = []
        for i, v in enumerate(nums):
            if v == x:
                return [ans[i - 1] if i > 0 else -1 for i in queries]

```

C++

```

class Solution {
public:
    vector<int> occurrenceOfElement(vector<int>& nums, vector<int>& queries, int x) {
        const vector<int> indices = getIndices(nums, x);
        vector<int> ans;
        for (const int query : queries)
            ans.push_back(query > indices.size() ? indices[query - 1] : -1);
        return ans;
    }
private:
    vector<int> getIndices(const vector<int>& nums, int x) {
        vector<int> indices;
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == x)
                indices.push_back(i);
        return indices;
    }
};

```

C++

Arrays & Hashing - LeetCode - By Pattern

— 1093 —

LeetCode

```

class Solution {
public:
    vector<int> occurrenceOfElement(vector<int>& nums, vector<int>& queries, int x) {
        vector<int> ans;
        for (int i = 0; i < nums.size(); ++i) {
            if (nums[i] == x) {
                ans.push_back(i);
            }
        }
        vector<int> ans;
        for (int i : queries) {
            ans.push_back(i > ans.size() ? ans[i - 1] : -1);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<int> occurrenceOfElement(vector<int>& nums, vector<int>& queries, int x) {
        vector<int> ans;
        for (int i = 0; i < nums.size(); ++i) {
            if (nums[i] == x) {
                ans.push_back(i);
            }
        }
        vector<int> ans;
        for (int i : queries) {
            ans.push_back(i > ans.size() ? ans[i - 1] : -1);
        }
        return ans;
    }
};

```

Java

```

class Solution {
    public List<Integer> occurrenceOfElement(List<Integer> nums, List<Integer> queries, int x) {
        List<Integer> indices = new ArrayList<>();
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == x)
                indices.add(i);
        List<Integer> ans = new ArrayList<>();
        for (int i : queries)
            ans.add(i > indices.size() ? indices.get(i - 1) : -1);
        return ans;
    }
private List<Integer> getIndices(List<Integer> nums, int x) {
    List<Integer> indices = new ArrayList<>();
    for (int i = 0; i < nums.size(); ++i)
        if (nums[i] == x)
            indices.add(i);
    return indices;
}
}

```

Arrays & Hashing - LeetCode - By Pattern

— 1094 —

LeetCode

```

class Solution {
    public List<Integer> occurrenceOfElement(List<Integer> nums, List<Integer> queries, int x) {
        List<Integer> ans = new ArrayList<>();
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == x)
                ans.add(i);
        List<Integer> ans = new ArrayList<>();
        for (int i : queries)
            ans.add(i > ans.size() ? ans.get(i - 1) : -1);
        return ans;
    }
}

```

Java

```

class Solution {
    public List<Integer> occurrenceOfElement(List<Integer> nums, List<Integer> queries, int x) {
        List<Integer> ans = new ArrayList<>();
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == x)
                ans.add(i);
        List<Integer> ans = new ArrayList<>();
        for (int i : queries)
            ans.add(i > ans.size() ? ans.get(i - 1) : -1);
        return ans;
    }
}

```

TypeScript

```

function occurrenceOfElement(nums: number[], queries: number[], x: number): number[] {
    const ans: number[] = [];
    for (let i = 0; i < nums.length; ++i)
        if (nums[i] == x)
            ans.push(i);
    for (let i : queries)
        ans.push(i > ans.length ? ans[i - 1] : -1);
    return ans;
}

```

TypeScript

```

function occurrenceOfElement(nums: number[], queries: number[], x: number): number[] {
    const ans: number[] = [];
    for (let i = 0; i < nums.length; ++i)
        if (nums[i] == x)
            ans.push(i);
    for (let i : queries)
        ans.push(i > ans.length ? ans[i - 1] : -1);
    return ans;
}

```

Go

Go

Arrays & Hashing - LeetCode - By Pattern

— 1095 —

LeetCode

```

func occurrenceOfElement(nums []int, queries []int, x int) []int {
    ans := []int{}
    for i, v := range nums {
        if v == x {
            ans = append(ans, i)
        }
    }
    for _, i := range queries {
        if i < len(ans) {
            ans = append(ans, ans[i-1])
        } else {
            ans = append(ans, -1)
        }
    }
    return ans
}

```

Go

```

func occurrenceOfElement(nums []int, queries []int, x int) []int {
    ans := []int{}
    for i, v := range nums {
        if v == x {
            ans = append(ans, i)
        }
    }
    for _, i := range queries {
        if i < len(ans) {
            ans = append(ans, ans[i-1])
        } else {
            ans = append(ans, -1)
        }
    }
    return ans
}

```

[*] Arrays & Hashing - Pattern Solution (stored optimal)

Python

```

class Solution:
    def occurrenceOfElement(self, nums: List[int], queries: List[int], x: int) -> List[int]:
        indices = []
        for i, v in enumerate(nums):
            if v == x:
                indices.append(i)
        for i in queries:
            if i > len(indices):
                return [indices[i - 1] if i > 0 else -1 for i in queries]

```

C++

```

class Solution {
public:
    vector<int> occurrenceOfElement(vector<int>& nums, vector<int>& queries, int x) {
        vector<int> indices;
        for (int i = 0; i < nums.size(); ++i)
            if (nums[i] == x)
                indices.push_back(i);
        vector<int> ans;
        for (int i : queries)
            ans.push_back(i > indices.size() ? indices[i - 1] : -1);
        return ans;
    }
};

```

Arrays & Hashing - LeetCode - By Pattern

— 1096 —

LeetCode

#41 **HARD**
First Missing Positive
 LeetCode - Simplify - WalkCC - LeetCode - Editorial
 Array - Hash Table
 Likes: 100

Problem:
 Given an unsorted integer array nums. Return the smallest positive integer that is not present in nums.

You must implement an algorithm that runs in O(n) time and uses O(1) auxiliary space.

Example 1:

Input: nums = [1,2,0]
 Output: 3
 Explanation: The numbers in the range [1,2] are all in the array.

Example 2:

Input: nums = [3,4,-1,1]
 Output: 2
 Explanation: 1 is in the array but 2 is missing.

Example 3:

Input: nums = [7,8,9,11,12]
 Output: 1
 Explanation: The smallest positive integer 1 is missing.

Constraints:

- 1 <= nums.length <= 105
- -231 <= nums[i] <= 231 - 1

>> Brute Force [Arrays & Hashing] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        # Brute Force O(n^2) - check 1,2,3,... until one is missing
        num_set = set(nums)
        i = 1
        while i in num_set:
            i += 1
        return i

```

Python

Python

Arrays & Hashing - LeetCode - By Pattern

— 1097 —

LeetCode

```

# Python Implementation of First Missing Positive
def firstMissingPositive(nums):
    n = len(nums)
    # Initialize an array to store the correct positions
    for i in range(n):
        while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
            current_index = nums[i] - 1
            nums[i], nums[current_index] = nums[current_index], nums[i]
    # Identify the first missing positive integer
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1

```

Python

```

class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        # Correct slot:
        # nums[i] - 1 <= i <= n
        # nums[i] - 1 <= i
        # nums[nums[i] - 1] <= nums[i]
        while nums[i] > 0 and nums[nums[i] - 1] != nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]
        for i, num in enumerate(nums):
            if num != i + 1:
                return i + 1
        return n + 1

```

Python

```

class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                j = nums[i] - 1
                nums[i], nums[j] = nums[j], nums[i]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1

```

Python

Arrays & Hashing - LeetCode - By Pattern

— 1098 —

LeetCode

```

class Solution {
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while i != nums[i] and nums[i] != n + 1:
                swap(i, nums[i] - 1)
            if i + 1 != nums[i]:
                return i + 1
        return n + 1
}

```

C++

```

class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        const int n = nums.size();
        // Check if 1 is present
        // nums[i] - 1 = i
        // nums[i] - 1 = i
        // nums[i] - 1 = i
        for (int i = 0; i < n; ++i)
            while (nums[i] != i + 1 && nums[i] != n + 1)
                swap(nums[i], nums[nums[i] - 1]);
        for (int i = 0; i < n; ++i)
            if (nums[i] != i + 1)
                return i + 1;
        return n + 1;
    }
};

```

C++

```

class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        int n = nums.size();
        for (int i = 0; i < n; ++i) {
            while (nums[i] != i + 1 && nums[i] != n + 1) {
                swap(nums[i], nums[nums[i] - 1]);
            }
        }
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }
};

```

C++

```

class Solution {
    // O(n) time, O(1) space
    // O(n) space
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; ++i) {
            while (nums[i] != i + 1 && nums[i] != n + 1) {
                swap(nums[i], nums[nums[i] - 1]);
            }
        }
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }
}

```

C++

```

class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        int n = nums.size();
        for (int i = 0; i < n; ++i) {
            while (nums[i] != i + 1 && nums[i] != n + 1) {
                swap(nums[i], nums[nums[i] - 1]);
            }
        }
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }
};

```

C++

```

class Solution {
    // O(n) time, O(1) space
    // O(n) space
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1 && nums[i] != n + 1) {
                swap(nums[i], nums[nums[i] - 1]);
            }
        }
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }
}

```

Java

Java

```

class Solution {
    public int firstMissingPositive(int[] nums) {
        final int n = nums.length;
        // Check if 1 is present
        // nums[i] - 1 = i
        // nums[i] - 1 = i
        // nums[i] - 1 = i
        for (int i = 0; i < n; ++i)
            while (nums[i] != i + 1 && nums[i] != n + 1)
                swap(nums[i], nums[nums[i] - 1]);
        for (int i = 0; i < n; ++i)
            if (nums[i] != i + 1)
                return i + 1;
        return n + 1;
    }

    private void swap(int[] nums, int i, int j) {
        final int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}

```

Java

```

class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; ++i) {
            while (nums[i] != i + 1 && nums[i] != n + 1) {
                swap(nums, i, nums[i] - 1);
            }
        }
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }

    private void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

Java

```

public class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; ++i) {
            while (nums[i] != i + 1 && nums[i] != n + 1) {
                swap(nums, i, nums[i] - 1);
            }
        }
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }

    private void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

Java

```

class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; ++i) {
            while (nums[i] != i + 1 && nums[i] != n + 1) {
                swap(nums, i, nums[i] - 1);
            }
        }
        for (int i = 0; i < n; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }

    private void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

Java

```

public class Solution {
    public int firstMissingPositive(int[] nums) {
        var i = 0;
        while (i < nums.length) {
            if (nums[i] > 0 && nums[i] <= nums.length) {
                var index = nums[i] - 1;
                if (index < i && nums[index] != nums[i]) {
                    var temp = nums[i];
                    nums[i] = nums[index];
                    nums[index] = temp;
                }
            }
            ++i;
        }
        for (i = 0; i < nums.length; ++i) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return nums.length + 1;
    }
}

```

TypeScript

TypeScript

```

function firstMissingPositive(nums: number[]): number {
    const n = nums.length;
    for (let i = 0; i < n; i++) {
        while (nums[i] != i + 1 && nums[i] != n + 1) {
            const j = nums[i] - 1;
            [nums[i], nums[j]] = [nums[j], nums[i]];
        }
    }
    for (let i = 0; i < n; i++) {
        if (nums[i] != i + 1) {
            return i + 1;
        }
    }
    return n + 1;
}

```

TypeScript

```
function firstMissingPositive(nums: number[]): number {
    const n = nums.length;
    let i = 0;
    while (i < n) {
        const j = nums[i] - 1;
        if (j >= 0 && j < n && !nums[j] && nums[i] !== nums[j]) {
            [nums[i], nums[j]] = [nums[j], nums[i]];
        }
    }
    const res = nums.findIndex((v, i) => v !== i + 1);
    return (res === -1 ? 1 : res) + 1;
}

Go

func firstMissingPositive(nums []int) int {
    n := len(nums)
    for i := range nums {
        for k := nums[i]; k <= n && nums[k] != k; k = nums[k] + 1 {
            nums[k], nums[nums[i]-1] = nums[nums[i]-1], nums[i]
        }
    }
    for i, v := range nums {
        if i+1 != v {
            return i + 1
        }
    }
    return n + 1
}

Go

func firstMissingPositive(nums []int) int {
    n := len(nums)
    for i := range nums {
        for k := nums[i]; k <= n && nums[k] != k; k = nums[k] + 1 {
            nums[k], nums[nums[i]-1] = nums[nums[i]-1], nums[i]
        }
    }
    for i, v := range nums {
        if i+1 != v {
            return i + 1
        }
    }
    return n + 1
}

Rust
Rust
```

Arrays & Hashing · LeetCode · By Pattern — 105 — LeetCode

```
impl Solution {
    pub fn first_missing_positive(nums: Vec<i32>) -> i32 {
        let n = nums.len();
        for i in 0..n {
            while nums[i] > 0 && nums[i] <= n as i32 && nums[i] != nums[nums[i] as usize - 1] {
                let j = nums[i] as usize - 1;
                nums.swap(i, j);
            }
        }
        for i in 0..n {
            if nums[i] != (i + 1) as i32 {
                return (i + 1) as i32;
            }
        }
        return (n + 1) as i32;
    }
}

Rust

impl Solution {
    pub fn first_missing_positive(nums: Vec<i32>) -> i32 {
        let n = nums.len();
        let mut i = 0;
        while i < n {
            let j = nums[i] - 1;
            if (i as i32) == j && j < n as i32 && !nums[j] && nums[i] != nums[j as usize] {
                i += 1;
            } else {
                nums.swap(i, j as usize);
            }
        }
        for i in 0..n {
            if nums[i] != (i + 1) as i32 {
                return (i + 1) as i32;
            }
        }
        return (n + 1) as i32;
    }
}

Python
```

[*] Arrays & Hashing — Pattern Solution (stored optimal)

Python

Arrays & Hashing · LeetCode · By Pattern — 106 — LeetCode

```
# Python Implementation of First Missing Positive
def firstMissingPositive(nums):
    n = len(nums)
    # Move numbers to their correct positions
    for i in range(n):
        while i != nums[i] and 0 <= nums[i] <= n:
            correct_index = nums[i] - 1
            nums[i], nums[correct_index] = nums[correct_index], nums[i]
        i += 1
    # Identify the first missing positive integer
    for i in range(n):
        if i + 1 != nums[i]:
            return i + 1
    return n + 1

# Example
print(firstMissingPositive([3, 4, -1, 1])) # Output should be 2
```

```
// C++ Implementation of First Missing Positive
#include <vector>
using namespace std;

class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        int n = nums.size();
        // Move numbers to their correct positions
        for (int i = 0; i < n; i++) {
            while (nums[i] >= 1 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
                int correct_index = nums[i] - 1;
                swap(nums[i], nums[correct_index]);
            }
        }
        // Identify the first missing positive integer
        for (int i = 0; i < n; i++) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        return n + 1;
    }
};
```

Arrays & Hashing · LeetCode · By Pattern — 107 — LeetCode

Quick Review — Arrays & Hashing 16 questions · insights · complexity · solution

#1 Two Sum [Easy]

Key Insights

- Use a hash table to store the elements and their indices for fast lookup.
- For each element, calculate its complement: target minus the current element.
- Check if the complement exists in the hash table.
- If found, return the indices of the current element and its complement.
- This approach only requires a single pass through the array.

Space & Time

Time Complexity: $O(n)$ - We traverse the list only once.

Space Complexity: $O(n)$ - In the worst-case, we store all elements in the hash table.

Solution

We use a hash table (dictionary) to map each array element to its index as we iterate. For each number, we compute the complement (target - number) and check if it already exists in our hash table. If the complement is found, we return the indices. This one-pass solution ensures linear time complexity and avoids the need for nested loops.

#163 Missing Ranges [Easy]

Key Insights

- Use a pointer (or iterate index) to traverse the nums array while keeping track of the previous number considered.
- Check for missing numbers before the first element, between consecutive elements, and after the last element.
- When a gap is found, convert it to a range string that is either a single number (if the gap is exactly one number) or an interval.
- Handle edge cases when the nums array is empty.

Space & Time

Time Complexity: $O(n)$, where n is the length of the nums array, since we iterate through the array once.

Space Complexity: $O(1)$ additional space (ignoring the output list), as we only use a few extra variables.

Solution

The solution leverages a single pass through the input array and calculates the missing ranges by comparing the expected number with the current number in nums. First, initialize a variable "prev" to one less than the lower bound so that missing numbers from the very start of the range can be handled uniformly. For each number in the array, if there is a gap between prev and the current number (i.e., current number is at least 2 more than prev), then add the missing range (prev+1 to current number-1) to the output list. Update "prev" to the current number and finally handle any gap between the last number and the upper bound. The primary data structures used are the list for the output; the approach is iterative.

#346 Moving Average from Data Stream [Easy]

Key Insights

- Use a queue (or circular buffer) to keep track of the current window elements.
- Maintain a running sum to quickly update the window's total when a new element comes in.
- When the window is full, subtract the value of the element that falls out as a new one is added.
- Each call to next() is executed in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per call to next()

Space Complexity: $O(\text{window size})$

Arrays & Hashing · LeetCode · By Pattern — 108 — LeetCode

Solution

The solution leverages a queue data structure to store the elements of the sliding window. A running sum variable is maintained for efficient calculation of the moving average. When a new integer is added:

- If the size of the queue is equal to the maximum window size, remove the oldest element and subtract its value from the running sum.
- Add the new element to both the queue and the running sum.
- Compute and return the moving average as the running sum divided by the number of elements in the queue.

This approach ensures that each update is done in constant time without having to sum all elements in the window repeatedly.

#760 Find Anagram Mappings [Easy]

Key Insights

- Since nums2 is an anagram of nums1, every element in nums2 has a corresponding position in nums1.
- A dictionary (or hash table) can be used to map each number in nums2 to its indices.
- When iterating through nums1, we can assign the next available index from the dictionary for that number.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the arrays (single pass to build the dictionary and another pass to form the mapping).

Space Complexity: $O(n)$ for storing the dictionary that holds indices of elements in nums2.

Solution

The approach consists of two primary steps:

- Build a dictionary to record all indices where each number appears in nums2. This dictionary maps a number to a list of indices.
- Iterate over nums1 and for each element, remove (or pop) one index from the corresponding list in the dictionary, and add that index to the final mapping array. This guarantees that each number from nums1 is correctly mapped to one of its positions in nums2 while efficiently handling duplicates.

#1426 Counting Elements [Easy]

Key Insights

- Use a hash table or set to store unique elements from arr for constant time lookups.
- Iterate through each element in arr and check if $x + 1$ exists in the set.
- Count each valid occurrence, including duplicates.
- This approach efficiently handles the problem without needing nested loops.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in arr (set lookups are $O(1)$ on average).

Space Complexity: $O(n)$, due to the use of an auxiliary set to store unique elements.

Solution

The solution leverages a set to achieve constant time membership checks. First, we convert arr to a set of unique numbers. Then, iterate through arr, and for each element, check if (element + 1) exists in the set. If it does, increment the count. This approach correctly handles duplicates by counting each occurrence during iteration.

#1534 Count Good Triplets [Easy]

Key Insights

- Brute force approach using three nested loops is acceptable since the maximum array length is 100.
- Check all possible triplets and count only those meeting the specified conditions.
- Absolute difference comparisons are the key condition validations.

Space & Time

Time Complexity: $O(n^3)$ where n is the length of the array.

Space Complexity: $O(1)$ extra space (ignoring input space).

Solution

Arrays & Hashing · LeetCode · By Pattern — 109 — LeetCode

We use a brute force algorithm with three nested loops, iterating over all combinations of triplets (i, j, k) ensuring $i < j < k$. For each triplet, we calculate the absolute differences:

$$= |arr[i] - arr[j]| - |arr[j] - arr[k]| - |arr[i] - arr[k]|$$

The triplet is counted as good if all absolute differences satisfy the conditions $(<= a, <= b, \text{ and } <= c)$ respectively. This simple approach leverages the small input size, eliminating the need for more complex data structures or optimizations.

#57 Insert Interval [Medium]

Key Insights

- The intervals array is sorted by start times.
- Three main steps are used:
- Add intervals that come completely before the new interval.
- Merge overlapping intervals with the new interval.
- Append intervals that start after the new (merged) interval.
- Iterating through the list once allows an $O(n)$ solution.

Space & Time

Time Complexity: $O(n)$ since we scan the list once.

Space Complexity: $O(n)$ to store the resulting list of intervals.

Solution

Traverse the intervals list and perform the following:

- Append all intervals that end before the new interval starts.
- For intervals that overlap with newInterval, update newInterval to merge them (using min for start and max for end).
- Append the new merged interval.
- Append all remaining intervals. This ensures that all intervals remain non-overlapping and sorted with a single pass.

#238 Product of Array Except Self [Medium]

Key Insights

- The product for each index can be computed by multiplying the product of all numbers to the left and the product of all numbers to the right of that index.
- Division is not allowed, so we must compute the cumulative products explicitly.
- Create two passes: one forward pass for prefix products and one backward pass for suffix products.
- Use the output array to store the cumulative product values, achieving $O(1)$ extra space by not using additional arrays.

Space & Time

Time Complexity: $O(n)$ because we iterate through the array twice.

Space Complexity: $O(1)$ extra space (excluding the output array).

Solution

The approach uses two passes over the input array:

- Initialize the answer array and fill it with prefix products. For each index i , answer[i] contains the product of all elements before i .
- Traverse the array from the end using a running suffix product. Update the answer array by multiplying the current suffix product with the prefix product already stored. This method leverages an output array to store intermediate results and uses a temporary variable for the suffix product, ensuring constant extra space.

#261 Zigzag Iterator [Medium]

Key Insights

- Use a FIFO (first-in-first-out) structure to maintain the order of vectors which still have remaining elements.
- For each call to next(), select the next element from the current vector and then move that vector to the end of the queue if it still contains elements.
- The approach ensures proper zigzag (or cyclic) ordering among the vectors.
- When extending to k vectors, the same approach works: initialize the queue with each non-empty vector.

Arrays & Hashing · LeetCode · By Pattern — 110 — LeetCode

Space & Time

Time Complexity: $O(1)$ per next() call on average, with an overall $O(n)$ for n total elements.
Space Complexity: $O(k)$ for the queue structure (where k is the number of vectors), plus the storage for the input vectors.

Solution

We can solve the Zigzag Iterator problem by using a queue (or deque) to hold pairs of the vector's current index and the vector itself. For each call to next(), we remove the front element from the queue, return the current item from its associated vector, and if the vector has more elements, reinsert the pair (with an updated index) into the back of the queue. This cyclic process naturally interleaves the elements from the provided vectors.

#307 Range Sum Query - Mutable [Medium]

Key Insights

- To efficiently support both point updates and range sum queries, a specialized data structure like a Binary Indexed Tree (Fenwick Tree) or a Segment Tree is ideal.
- Both data structures provide update and sum query operations in $O(\log n)$ time.
- The challenge is to maintain a mutable array while enabling fast sum computation over any specified range.

Space & Time

Time Complexity: $O(\log n)$ for both update and sum/range operations

Space Complexity: $O(n)$ to store the additional tree structure

Solution

We can use a Binary Indexed Tree (Fenwick Tree) to solve this problem. The Fenwick Tree allows for efficient prefix sum calculations and point updates, both in $O(\log n)$ time. The approach involves:

- Building the Fenwick Tree using the initial array.
- On an update, computing the difference between the new value and the current value, then propagating the change appropriately in the tree.
- For the sum range query, computing the prefix sum up to right and subtracting the prefix sum up to left-1.

#454 4Sum II [Medium]

Key Insights

- Use a hash table to store the frequency of all possible sums from $nums1$ and $nums2$.
- For each pair from $nums3$ and $nums4$, compute the complement (negative sum) and look it up in the hash table.
- This reduces the time complexity from $O(n^4)$ to $O(n^2)$.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of each array.

Space Complexity: $O(n^2)$ due to the storage of pair sums in the hash table.

Solution

The solution employs a hash table to precompute and store the frequency of sums of all pairs from the first two arrays. Then, by iterating over all pairs from the third and fourth arrays, we calculate the required complement that would result in a total sum of zero. This complement is checked in the hash table, and the frequency (number of valid pairs) is added to the result. This method efficiently finds the number of valid quadruplets without resorting to a brute-force $O(n^4)$ approach.

#525 Contiguous Array [Medium]

Key Insights

- Convert 0s to -1 so that a balanced subarray (equal number of 0s and 1s) sums to zero.
- Use a prefix sum to track the cumulative sum as you iterate through the array.
- Use a hash table (or dictionary) to store the first occurrence of each prefix sum.
- When the same prefix sum is encountered again, it indicates that the subarray between these indices is balanced.

Space & Time

Arrays & Hashing · LeetCode · By Pattern — 1111 — LeetCode

Time Complexity: $O(n)$ - We iterate through the array once.

Space Complexity: $O(n)$ - In the worst case, we store an entry for each prefix sum.

Solution

We use a technique where we convert each 0 in the array to -1. Then, we compute a running prefix sum as we iterate through the array. A prefix sum of 0 at any point indicates that the subarray from the beginning to the current index is balanced. For each prefix sum, if we have seen it before, the subarray between the previous index (where the prefix sum was first seen) and the current index sums to 0. We update the maximum length accordingly. We use a hash table to store the first index where each prefix sum occurs.

#560 Subarray Sum Equals K [Medium]

Key Insights

- Utilize the prefix sum technique to simplify sum calculation for subarrays.
- Use a hash table for dictionary/map to store the frequency of prefix sums encountered.
- For each element, check if (current prefix sum - k) exists in the hash table; if it does, it indicates a valid subarray.
- Initialize the hash table with (0: 1) to manage cases where a subarray's sum is exactly k.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We solve the problem by maintaining a running sum (prefix sum) and using a hash table to record the number of times each sum has occurred. As we iterate through the array, we update the running sum. For each new sum, we check if (current sum - k) exists in our hash table. If it does, the value associated with (current sum - k) gives the number of subarrays ending at the current index that sum to k. This method efficiently computes the result in a single pass through the array.

#1010 Pairs of Songs With Total Durations Divisible by 60 [Medium]

Key Insights

- Each song duration's effect is determined by its remainder when divided by 60.
- For any given module value r (where $0 \leq r < 60$), the complementary module needed to complete 60 is $(60 - r) \% 60$.
- Special handling is needed when r is 0 (or 30, since $30 + 30 = 60$) to avoid double-counting.
- A frequency counter for remainders can be efficiently used to count valid pairs in one pass through the list.

Space & Time

Time Complexity: $O(n)$, where n is the number of songs.

Space Complexity: $O(60)$ - $O(1)$, since we only need to keep track of 60 possible remainders.

Solution

We iterate over the list of song durations and calculate the remainder of each duration modulo 60. A frequency array (or hash table) is used to record how many times each remainder has been seen so far. For each song, we look up the frequency of its complementary remainder (i.e., $60 - \text{current_remainder} \% 60$) already encountered. This frequency indicates how many valid pairs can be formed with the current song. After counting pairs with the current song, we update the frequency of its remainder for future pairings.

#1272 Remove Interval [Medium]

Key Insights

- Iterate over each interval and check if it overlaps with the toBeRemoved interval.
- If the current interval is completely outside of toBeRemoved, add it to the result as is.
- For overlapping intervals, compute the non-overlapping portions:
- If the left side of the interval lies before toBeRemoved, include [start, min(end, toBeRemoved.start)].

Arrays & Hashing · LeetCode · By Pattern — 1112 — LeetCode

- If the right side of the interval lies after toBeRemoved, include [max(start, toBeRemoved.end), end].

Space & Time

Time Complexity: $O(n)$, where n is the number of intervals.

Space Complexity: $O(n)$, for storing the resulting intervals.

Solution

The approach involves iterating through the list of sorted disjoint intervals and checking for overlap with the toBeRemoved interval. For each interval:

- If it is completely before or after the removal interval, add it directly to the result.
- If it overlaps with toBeRemoved, determine the remaining portions: The left side of the interval that does not intersect (if any). The right side of the interval that does not intersect (if any). Use simple conditional checks and list manipulation to form the final set of intervals.

#1429 First Unique Number [Medium]

Key Insights

- Use a queue to maintain the order of potential unique numbers.
- Use a hash table (or map) to count the occurrences of each number.
- When showing the first unique number, remove numbers from the front of the queue if their frequency is more than one.
- All operations (add and show/firstUnique) can be done in $O(1)$ amortized time.

Space & Time

Time Complexity: $O(1)$ amortized for each add and showFirstUnique call.

Space Complexity: $O(n)$, where n is the number of distinct integers encountered.

Solution

We design the FirstUnique class using two main data structures: a queue and a hash map (or dictionary). The queue preserves the order of insertion for unique candidates, while the hash map holds the frequency count for each integer.

When a new number is added:

- Increment its count in the hash map.
- If the count is one, append it to the queue.

To retrieve the first unique number:

- While the queue is not empty and the count for the element at the front is greater than one, remove it from the queue.
- Return the front element of the queue if it exists, otherwise return -1.

This approach ensures that we are always up-to-date with the current first unique number.

#1519 Find Occurrences of an Element in an Array [Medium]

Key Insights

- Scan the array once and record every index where 'nums[i] == x'.
- Store those indices in order inside a positions array.
- For each query 'k', check whether the 'k'-th occurrence exists.
- If it exists, return 'positions[k] - 1'.
- Otherwise, return '-1'.
- This lets every query be answered in constant time after the preprocessing pass.

Space & Time

Time Complexity: $O(n + q)$, where n is the length of 'nums' and q is the number of queries.

Space Complexity: $O(n)$, where m is the number of occurrences of 'x'.

Solution

We solve the problem by first precomputing all occurrences of 'x'. As we iterate through 'nums', every time we see 'x', we append its index to an array called 'positions'. Then, for each query value 'k', we check whether 'k' is

Arrays & Hashing · LeetCode · By Pattern — 1113 — LeetCode

within the bounds of 'positions'. If it is, the answer is 'positions[k] - 1'; otherwise, the required occurrence does not exist, so the answer is '-1'. This avoids rescanning the array for every query.

#41 First Missing Positive [Hard]

Key Insights

- The missing positive number must be in the range $[1, n+1]$ where n is the length of the array.
- Use the array indices to place numbers in their correct positions (i.e., number i should be at index $i-1$) with in-place swapping.
- After rearrangement, scan the array: the first index where the number is not equal to index+1 indicates the missing positive integer.

Space & Time

Time Complexity: $O(n)$ as each number is swapped at most once.

Space Complexity: $O(1)$ since the rearrangement occurs in-place.

Solution

The approach involves reordering the array so that each positive integer in the range $[1, n]$ is placed at the index corresponding to its value minus one. This means for any valid number i , $nums[i-1]$ should be i . We iterate through the array and perform in-place swaps to achieve this order. After this process, another pass through the array reveals the first position where the number does not match the expected value ($i+1$), which is the smallest missing positive. If every position is correct, then the missing value is $n+1$.

Arrays & Hashing · LeetCode · By Pattern — 1114 — LeetCode

#13 Two Pointers — 19 questions · #13 of 21

#88 EASY

Merge Sorted Array

LeetCode · SimpleList · WalkCC · LeetCode · Editorial

Array · Two Pointers · Sorting

List: Denny Zhang

Problem:

You are given two integer arrays $nums1$ and $nums2$, sorted in non-decreasing order, and two integers m and n , representing the number of elements in $nums1$ and $nums2$ respectively.

Merge $nums1$ and $nums2$ into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array $nums1$. To accommodate this, $nums1$ has a length of $m + n$, where the first m elements denote the elements that should be merged, and the last n elements are set to 0 and should be ignored. $nums2$ has a length of n .

Example 1:

Input: $nums1 = [1,2,3,0,0,0]$, $m = 3$, $nums2 = [2,5,6]$, $n = 3$
Output: $[1,2,2,3,5,6]$
Explanation: The arrays we are merging are $[1,2,3]$ and $[2,5,6]$.
The result of the merge is $[1,2,2,3,5,6]$ with the underlined elements coming from $nums1$.

Example 2:

Input: $nums1 = [1]$, $m = 1$, $nums2 = [1]$, $n = 0$
Output: $[1]$
Explanation: The arrays we are merging are $[1]$ and $[]$.
The result of the merge is $[1]$.

Example 3:

Input: $nums1 = [0]$, $m = 0$, $nums2 = [1]$, $n = 1$
Output: $[1]$
Explanation: The arrays we are merging are $[]$ and $[1]$.
The result of the merge is $[1]$.
Note that because $n = 0$, there are no elements in $nums2$. The 0 is only there to ensure the merge result can fit in $nums1$.

Constraints:

- $nums1.length == m + n$
 - $nums2.length == n$
 - $0 \leq m, n \leq 200$
 - $1 \leq m + n \leq 200$
 - $-109 \leq nums1[i], nums2[j] \leq 109$
- Follow up: Can you come up with an algorithm that runs in $O(m + n)$ time?

Arrays & Hashing · LeetCode · By Pattern — 1115 — LeetCode

>> Brute Force [Two Pointers] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def merge(self, nums1, m, nums2, n):
        # Brute Force O(n^2) - nested loops instead of two pointers
        for i in range(m):
            for j in range(n):
                if nums1[i] > nums2[j]:
                    pass
```

Python

```
def merge(nums1, m, nums2, n):
    # Initialize pointers for nums1, nums2 and the tail index
    p1 = m - 1
    p2 = n - 1
    tail = m + n - 1

    # Merge elements from the end by choosing the larger element
    while p1 >= 0 and p2 >= 0:
        if nums1[p1] > nums2[p2]:
            nums1[tail] = nums1[p1]
            p1 -= 1 # Move pointer in nums1
        else:
            nums1[tail] = nums2[p2]
            p2 -= 1 # Move pointer in nums2
        tail -= 1 # Move tail pointer

    # Copy any remaining elements from nums2 into nums1
    while p2 >= 0:
        nums1[tail] = nums2[p2]
        p2 -= 1
        tail -= 1
```

Python

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        # m = 1 if nums1 is empty, the actual m is
        j = m - 1 if nums1 < nums2 else m
        k = m + n - 1 if nums1 < nums2 else m + n - 1
        while j >= 0 and k >= 0:
            if j >= 0 and nums1[j] > nums2[k]:
                nums1[k] = nums1[j]
                k -= 1
                j -= 1
            else:
                nums1[k] = nums2[k]
                k -= 1
                j -= 1
```

Python

Two Pointers · LeetCode · By Pattern — 1116 — LeetCode

```

class Solution {
    def merge(int[] nums1, int m, int[] nums2, int n) -> None:
        k = m + n - 1
        i = m - 1, n = n - 1
        while j >= 0:
            if i <= 0 and n >= 0:
                nums1[k] = nums2[j]
                k -= 1
            else:
                nums1[k] = max(nums1[i], nums2[j])
                i -= 1
                j -= 1
                k -= 1

```

```

Python
class Solution:
    def merge(nums1, m, nums2, n) -> None:
        # Do not return anything, modify nums1 in-place instead.
        i, j, k = m - 1, n - 1, m + n - 1
        # Since if just always put 0 as last e.g. arr=[1,2,0], arr[i]
        # ==> then no more num needed, rest of arr already sorted, so this while is good enough
        while k >= 0:
            if i <= 0 and n >= 0:
                nums1[k] = nums2[j]
                j -= 1
            else:
                nums1[k] = max(nums1[i], nums2[j])
                i -= 1
                j -= 1
                k -= 1

```

```

C++
class Solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        int i = m - 1; // nums1's index (the actual nums)
        int j = n - 1; // nums2's index
        int k = m + n - 1; // nums1's index (the next filled position)

        while (j >= 0) {
            if (i <= 0 && n >= 0) {
                nums1[k--] = nums2[j--];
            } else {
                nums1[k--] = max(nums1[i], nums2[j]);
                i--;
                j--;
                k--;
            }
        }
    }
};

```

```

class Solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        for (int i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
            nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : max(nums1[i--], nums2[j--]);
        }
    }
};

```

```

C++
class Solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;
        while (j >= 0) {
            if (i <= 0 && n >= 0) {
                nums1[k] = nums2[j--];
            } else {
                nums1[k] = max(nums1[i], nums2[j]);
                i--;
                j--;
                k--;
            }
        }
    }
};

```

```

C++
class Solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        for (int i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
            nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : max(nums1[i--], nums2[j--]);
        }
    }
};

```

C++

```

class Solution {
    //
    // @param Integer[] nums1
    // @param Integer m
    // @param Integer[] nums2
    // @param Integer n
    // @return None
    //
    function merge($nums1, $m, $nums2, $n) {
        while (count($nums1) < $m + $n) {
            array_push($nums1, 0);
        }
        for ($i = 0; $i < $m; $i++) {
            array_push($nums1, $nums2[$i]);
        }
        sort($nums1);
    }
}

```

```

Java
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1; // nums1's index (the actual nums)
        int j = n - 1; // nums2's index
        int k = m + n - 1; // nums1's index (the next filled position)

        while (j >= 0) {
            if (i <= 0 && n >= 0) {
                nums1[k--] = nums2[j--];
            } else {
                nums1[k--] = Math.max(nums1[i], nums2[j]);
                i--;
                j--;
                k--;
            }
        }
    }
}

```

```

Java
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        for (int i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
            nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : Math.max(nums1[i--], nums2[j--]);
        }
    }
}

```

```

Java
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        for (int i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
            nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : Math.max(nums1[i--], nums2[j--]);
        }
    }
}

```

JavaScript
JavaScript

```

//
// @param {number[]} nums1
// @param {number} m
// @param {number[]} nums2
// @param {number} n
// @return {void} Do not return anything, modify nums1 in-place instead.
//
var merge = function(nums1, m, nums2, n) {
    for (let i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
        nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : Math.max(nums1[i--], nums2[j--]);
    }
};

```

```

JavaScript
//
// @param {number[]} nums1
// @param {number} m
// @param {number[]} nums2
// @param {number} n
// @return {void} Do not return anything, modify nums1 in-place instead.
//
var merge = function(nums1, m, nums2, n) {
    for (let i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
        nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : Math.max(nums1[i--], nums2[j--]);
    }
};

```

```

TypeScript
TypeScript
//
// Do not return anything, modify nums1 in-place instead.
//
function merge(nums1: number[], m: number, nums2: number[], n: number): void {
    for (let i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
        nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : Math.max(nums1[i--], nums2[j--]);
    }
}

```

```

TypeScript
//
// Do not return anything, modify nums1 in-place instead.
//
function merge(nums1: number[], m: number, nums2: number[], n: number): void {
    for (let i = m - 1, j = n - 1, k = m + n - 1; j >= 0; --k) {
        nums1[k] = i <= 0 && n >= 0 ? nums2[j--] : Math.max(nums1[i--], nums2[j--]);
    }
}

```

Go
Go

```

func merge(nums1 []int, m int, nums2 []int, n int) {
    for i, j, k := m-1, n-1, m+n-1; j >= 0; k-- {
        if i <= 0 && n >= 0 {
            nums1[k] = nums2[j]
            j--
        } else {
            nums1[k] = max(nums1[i], nums2[j])
            i--
            j--
            k--
        }
    }
}

```

Go

```

func merge(nums1 []int, m int, nums2 []int, n int) {
    for i, j, k := m-1, n-1, m+n-1; j >= 0; k-- {
        if i <= 0 && n >= 0 {
            nums1[k] = nums2[j]
            j--
        } else {
            nums1[k] = max(nums1[i], nums2[j])
            i--
            j--
            k--
        }
    }
}

```

Rust

```

impl Solution {
    pub fn merge(nums1: &mut Vec<i32>, m: i32, nums2: &mut Vec<i32>, n: i32) {
        let mut k = (m + n - 1) as usize;
        let mut i = (m - 1) as isize;
        let mut j = (n - 1) as isize;

        while j >= 0 {
            if i <= 0 && n >= 0 as usize {
                nums1[k] = nums2[j as usize];
                j -= 1;
            } else {
                nums1[k] = nums2[j as usize] if nums1[i as usize] < nums2[j as usize] else {
                    nums1[i as usize];
                    i -= 1;
                };
                j -= 1;
                k -= 1;
            }
        }
    }
}

```

Rust

```

impl Solution {
    pub fn merge(nums1: &mut Vec<i32>, m: i32, nums2: &mut Vec<i32>, n: i32) {
        let mut i = m - 1, j = n - 1, k = m + n - 1;
        for i in (0..m).rev() {
            nums1[i] = match (i <= 0, n >= 0) {
                (true, false) => {
                    n -= 1;
                    nums2[j]
                }
                (false, true) => {
                    m -= 1;
                    nums1[i]
                }
                (_, _) => {
                    if nums1[i] < nums2[j] {
                        n -= 1;
                        nums2[j]
                    } else {
                        m -= 1;
                        nums1[i]
                    }
                }
            };
            i -= 1;
        }
    }
}

```

[1] Two Pointers — Pattern Solution (stored optimal)

Python

```

def merge(nums1, m, nums2, n):
    # Initialize pointers for nums1, nums2 and the tail index
    p1 = m - 1
    p2 = n - 1
    tail = m + n - 1

    # Merge arrays from the end by choosing the larger element
    while p1 >= 0 and p2 >= 0:
        if nums1[p1] >= nums2[p2]:
            nums1[tail] = nums1[p1]
            p1 -= 1 # Move pointer in nums1
        else:
            nums1[tail] = nums2[p2]
            p2 -= 1 # Move pointer in nums2
            tail -= 1 # Move tail pointer

    # Copy any remaining elements from nums2 into nums1
    while p2 >= 0:
        nums1[tail] = nums2[p2]
        p2 -= 1
        tail -= 1

```

C++

```
#include <vector>
using namespace std;

void merge(vector<int>& num1, int n, vector<int>& num2, int n) {
    // Initializing pointers for num1, num2 and the tail index of num1
    int p1 = n - 1, p2 = n - 1, tail = n + 1;

    // Merge arrays starting from the end
    while (p1 >= 0 && p2 >= 0) {
        if (num1[p1] > num2[p2]) {
            num1[tail] = num1[p1];
            p1--; // Move pointer in num1
        } else {
            num1[tail] = num2[p2];
            p2--; // Move pointer in num2
        }
        tail--; // Move the tail pointer
    }

    // Copy any remaining elements from num2 into num1
    while (p2 >= 0) {
        num1[tail] = num2[p2];
        p2--;
        tail--;
    }
}
```

#125

EASY

Valid Palindrome

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Two Pointers · String

Like · Cloned 109

Problem:

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string *s*, return true if it is a palindrome, or false otherwise.

Example 1:

Input: *s* = "A man, a plan, a canal: Panama"

Output: true

Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: *s* = "race a car"

Output: false

Explanation: "racecar" is not a palindrome.

Example 3:

Two Pointers · LeetCode · By Pattern

— 123 —

LeetCode

Input: *s* = ""

Output: true

Explanation: *s* is an empty string "" after removing non-alphanumeric characters. Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- 1 ≤ *s*.length ≤ 2 * 10⁵
- *s* consists only of printable ASCII characters.

Python

Python

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Initialize two pointers at the beginning and end of the string
        left, right = 0, len(s) - 1
        while left < right:
            # Skip non-alphanumeric characters from the left
            while left < right and not s[left].isalnum():
                left += 1
            # Skip non-alphanumeric characters from the right
            while left < right and not s[right].isalnum():
                right -= 1
            # Compare the characters in lowercase
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True
```

Python

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l = 0
        r = len(s) - 1
        while l < r:
            while l < r and not s[l].isalnum():
                l += 1
            while l < r and not s[r].isalnum():
                r -= 1
            if s[l].lower() != s[r].lower():
                return False
            l += 1
            r -= 1
        return True
```

Python

Two Pointers · LeetCode · By Pattern

— 124 —

LeetCode

```
class Solution {
public:
    bool isPalindrome(string s) {
        int l = 0, r = s.length() - 1;
        while (l < r) {
            if (!s[l].isalnum())
                l++;
            if (!s[r].isalnum())
                r--;
            if (s[l].lower() != s[r].lower())
                return false;
            l++;
            r--;
        }
        return true;
    }
};
```

Python

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l, r = 0, len(s) - 1
        while l < r:
            if not s[l].isalnum():
                l += 1
            elif not s[r].isalnum():
                r -= 1
            elif s[l].lower() != s[r].lower():
                return False
            else:
                l += 1
                r -= 1
        return True
```

C++

C++

```
class Solution {
//
// @param String s
// @return Boolean
//
function isPalindrome(s) {
    s = s.toLowerCase();
    s = s.replace(/ /g, '');
    let l = 0, r = s.length - 1;
    while (l < r) {
        if (s[l].toLowerCase() != s[r].toLowerCase())
            return false;
        l++;
        r--;
    }
    return true;
}
};
```

C++

Two Pointers · LeetCode · By Pattern

— 125 —

LeetCode

```
class Solution {
//
// @param String s
// @return Boolean
//
function isPalindrome(s) {
    s = s.toLowerCase();
    s = s.replace(/ /g, '');
    let l = 0, r = s.length - 1;
    while (l < r) {
        if (s[l].toLowerCase() != s[r].toLowerCase())
            return false;
        l++;
        r--;
    }
    return true;
}
};
```

Java

Java

```
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0;
        int r = s.length() - 1;
        while (l < r) {
            while (l < r && !Character.isLetterOrDigit(s.charAt(l)))
                l++;
            while (l < r && !Character.isLetterOrDigit(s.charAt(r)))
                r--;
            if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
                return false;
            l++;
            r--;
        }
        return true;
    }
}
```

Java

Two Pointers · LeetCode · By Pattern

— 126 —

LeetCode

```
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0, j = s.length() - 1;
        while (l < j) {
            if (!Character.isLetterOrDigit(s.charAt(l)))
                l++;
            } else if (!Character.isLetterOrDigit(s.charAt(j)))
                j--;
            } else if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(j)))
                return false;
            l++;
            j--;
        }
        return true;
    }
}
```

Java

```
public class Solution {
    public boolean isPalindrome(String s) {
        int l = 0, j = s.length() - 1;
        while (l < j) {
            if (!Character.isLetterOrDigit(s.charAt(l)))
                l++;
            } else if (!Character.isLetterOrDigit(s.charAt(j)))
                j--;
            } else if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(j)))
                return false;
            l++;
            j--;
        }
        return true;
    }
}
```

Java

```
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0, j = s.length() - 1;
        while (l < j) {
            if (!Character.isLetterOrDigit(s.charAt(l)))
                l++;
            } else if (!Character.isLetterOrDigit(s.charAt(j)))
                j--;
            } else if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(j)))
                return false;
            l++;
            j--;
        }
        return true;
    }
}
```

Java

Two Pointers · LeetCode · By Pattern

— 127 —

LeetCode

```
public class Solution {
    public boolean isPalindrome(String s) {
        int l = 0, j = s.length() - 1;
        while (l < j) {
            if (!Character.isLetterOrDigit(s.charAt(l)))
                l++;
            } else if (!Character.isLetterOrDigit(s.charAt(j)))
                j--;
            } else if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(j)))
                return false;
            l++;
            j--;
        }
        return true;
    }
}
```

JavaScript

JavaScript

```
//
// @param (string) s
// @return (boolean)
//
var isPalindrome = function (s) {
    let l = 0;
    let j = s.length - 1;
    while (l < j) {
        if (!/[a-zA-Z0-9]/.test(s.charAt(l)))
            l++;
        } else if (!/[a-zA-Z0-9]/.test(s.charAt(j)))
            j--;
        } else if (s[l].toLowerCase() != s[j].toLowerCase())
            return false;
        l++;
        j--;
    }
    return true;
};
```

JavaScript

Two Pointers · LeetCode · By Pattern

— 128 —

LeetCode


```

//
 * @param {string} s
 * @return {boolean}
 */
var isPalindrome = function (s) {
    let l = 0;
    let j = s.length - 1;
    while (l < j) {
        if (/[a-z0-9]/.test(s[l])) {
            ++l;
        } else if (/[a-z0-9]/.test(s[j])) {
            --j;
        } else if (s[l].toLowerCase() !== s[j].toLowerCase()) {
            return false;
        } else {
            ++l;
            --j;
        }
    }
    return true;
};

```

TypeScript

```

function isPalindrome(s: string): boolean {
    let l = 0;
    let j = s.length - 1;
    while (l < j) {
        if (/[a-z0-9]/.test(s[l])) {
            ++l;
        } else if (/[a-z0-9]/.test(s[j])) {
            --j;
        } else if (s[l].toLowerCase() !== s[j].toLowerCase()) {
            return false;
        } else {
            ++l;
            --j;
        }
    }
    return true;
}

```

TypeScript

Two Pointers · LeetCode · By Pattern

— 1129 —

LeetCode

```

function isPalindrome(s: string): boolean {
    let l = 0;
    let j = s.length - 1;
    while (l < j) {
        if (/[a-z0-9]/.test(s[l])) {
            ++l;
        } else if (/[a-z0-9]/.test(s[j])) {
            --j;
        } else if (s[l].toLowerCase() !== s[j].toLowerCase()) {
            return false;
        } else {
            ++l;
            --j;
        }
    }
    return true;
}

```

Rust

```

impl Solution {
    pub fn is_palindrome(s: String) -> bool {
        let s = s.to_lowercase();
        let s = s.as_bytes();
        let n = s.len();
        let (mut l, mut r) = (0, n - 1);
        while l < r {
            while l < r && !s[l].is_ascii_alphanumeric() {
                l += 1;
            }
            while l < r && !s[r].is_ascii_alphanumeric() {
                r -= 1;
            }
            if s[l] != s[r] {
                return false;
            }
            l += 1;
            if r != 0 {
                r -= 1;
            }
        }
        true
    }
}

```

Rust

Two Pointers · LeetCode · By Pattern

— 1130 —

LeetCode

```

impl Solution {
    pub fn is_palindrome(s: String) -> bool {
        let s = s.to_lowercase();
        let s = s.as_bytes();
        let n = s.len();
        let (mut l, mut r) = (0, n - 1);
        while l < r {
            while l < r && !s[l].is_ascii_alphanumeric() {
                l += 1;
            }
            while l < r && !s[r].is_ascii_alphanumeric() {
                r -= 1;
            }
            if s[l] != s[r] {
                return false;
            }
            l += 1;
            if r != 0 {
                r -= 1;
            }
        }
        true
    }
}

```

[1] Two Pointers — Pattern Solution (matched from community answers)

```

Python
class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Initialize two pointers at the beginning and end of the string
        left, right = 0, len(s) - 1
        while left < right:
            # Skip non-alphanumeric characters from the left
            while left < right and not s[left].isalnum():
                left += 1
            # Skip non-alphanumeric characters from the right
            while left < right and not s[right].isalnum():
                right -= 1
            # Compare the characters in lowercase
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True

```

#283

EASY

Move Zeros

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Two Pointers

Lists Grid 189

Problem:

Two Pointers · LeetCode · By Pattern

— 1131 —

LeetCode

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: nums = [0,1,0,3,12]

Output: [1,2,12,0,0]

Example 2:

Input: nums = [0]

Output: [0]

Constraints:

• 1 <= nums.length <= 104

• -231 <= nums[i] <= 231 - 1

Follow up: Could you minimize the total number of operations done?

>> Brute Force [Two Pointers] Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def moveZeroes(self, nums):
        # Brute Force O(n^2) - nested loops instead of two pointers
        n = len(nums)
        for i in range(n):
            for j in range(i + 1, n):
                # Check nums[i] and nums[j]
                pass

Python
# Moves all zeros in the list 'nums' to the end and in-place.
# j is the pointer where the next non-zero element should be placed.
# i is the current element being processed.
# Example usage:
nums = [0, 1, 0, 3, 12]
moveZeroes(nums)
print(nums) # Expected output: [1, 3, 12, 0, 0]

```

Python

Two Pointers · LeetCode · By Pattern

— 1132 —

LeetCode

```

class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        j = 0
        for num in nums:
            if num != 0:
                nums[j] = num
                j += 1
        for i in range(j, len(nums)):
            nums[i] = 0

```

Python

```

class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        k = 0
        for i, x in enumerate(nums):
            if x:
                nums[k], nums[i] = nums[i], nums[k]
                k += 1

```

Python

```

class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        l = 0
        for j, x in enumerate(nums):
            if x:
                l += 1
                nums[l], nums[j] = nums[j], nums[l]

```

C++

```

C++
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int l = 0;
        for (const int num : nums) {
            if (num != 0) {
                nums[l++] = num;
            }
        }
        while (l < nums.size()) {
            nums[l++] = 0;
        }
    }
};

```

C++

Two Pointers · LeetCode · By Pattern

— 1133 —

LeetCode

```

class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int k = 0, n = nums.size();
        for (int i = 0; i < n; ++i) {
            if (nums[i] != 0) {
                swap(nums[i], nums[k++]);
            }
        }
    }
};

```

C++

```

class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int l = 0, n = nums.size();
        for (int j = 0; j < n; ++j) {
            if (nums[j] != 0) {
                swap(nums[l++], nums[j]);
            }
        }
    }
};

```

Java

```

Java
class Solution {
    public void moveZeroes(int[] nums) {
        int l = 0;
        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] != 0) {
                swap(nums[l++], nums[i]);
            }
        }
    }
}

```

Java

```

class Solution {
    public void moveZeroes(int[] nums) {
        int k = 0, n = nums.length;
        for (int i = 0; i < n; ++i) {
            if (nums[i] != 0) {
                int t = nums[i];
                nums[i] = nums[k];
                nums[k++] = t;
            }
        }
    }
}

```

Java

Two Pointers · LeetCode · By Pattern

— 1134 —

LeetCode

```

class Solution {
public: void moveZeros(int[] nums) {
    int i = 0; int n = nums.length;
    for (int j = 0; j < n; ++j) {
        if (nums[j] != 0) {
            int t = nums[i];
            nums[i] = nums[j];
            nums[j] = t;
        }
    }
}
}

JavaScript
JavaScript
// @param {number[]} nums
// @return {void} Do not return anything, modify nums in-place instead.
//
var moveZeros = function(nums) {
    let k = 0;
    for (let i = 0; i < nums.length; ++i) {
        if (nums[i]) {
            [nums[i], nums[k]] = [nums[k], nums[i]];
            ++k;
        }
    }
};

JavaScript
// @param {number[]} nums
// @return {void} Do not return anything, modify nums in-place instead.
//
var moveZeros = function(nums) {
    let i = 0;
    for (let j = 0; j < nums.length; ++j) {
        if (nums[j]) {
            const t = nums[i];
            nums[i] = nums[j];
            nums[j] = t;
        }
    }
};

TypeScript
TypeScript

```

```

// @param {number[]} nums
// @return {void} Do not return anything, modify nums in-place instead.
//
function moveZeros(nums: number[]): void {
    let n = 0;
    for (let i = 0; i < nums.length; ++i) {
        if (nums[i]) {
            [nums[i], nums[n]] = [nums[n], nums[i]];
            ++n;
        }
    }
}

TypeScript
TypeScript
// @param {number[]} nums
// @return {void} Do not return anything, modify nums in-place instead.
//
function moveZeros(nums: number[]): void {
    const n = nums.length;
    let i = 0;
    for (let j = 0; j < n; ++j) {
        if (nums[j]) {
            if (i < j) {
                [nums[i], nums[j]] = [nums[j], 0];
            }
            ++i;
        }
    }
}

Go
Go
func moveZeros(nums []int) {
    n := 0
    for i, v := range nums {
        if v != 0 {
            nums[i], nums[n] = nums[n], nums[i]
            n++
        }
    }
}

C#
C#
func MoveZeros(nums []int) {
    i := 0
    for j, v := range nums {
        if v != 0 {
            nums[i], nums[j] = nums[j], nums[i]
        }
    }
}

Rust
Rust

```

```

impl Solution {
    pub fn move_zeros(nums: &mut Vec<i32>) {
        let mut i = 0;
        let n = nums.len();
        for i in 0..n {
            if nums[i] != 0 {
                nums.swap(i, i);
                i += 1;
            }
        }
    }
}

Rust
Rust
impl Solution {
    pub fn move_zeros(nums: &mut Vec<i32>) {
        let mut i = 0;
        for j in 0..nums.len() {
            if nums[j] != 0 {
                if j > i {
                    nums[i] = nums[j];
                    nums[j] = 0;
                }
                i += 1;
            }
        }
    }
}

[+] Two Pointers — Pattern Solution (stored optimal)
Python
Python
# Moves all zeros in the list 'nums' to the end and in-place.
def moveZeros(nums):
    # i is the pointer where the next non-zero element should be placed.
    i = 0
    # Iterate through all the elements with index 'i'.
    for i in range(len(nums)):
        # Check if the current element is non-zero.
        if nums[i] != 0:
            # Swap only if i is not equal to j to avoid unnecessary operation.
            nums[i], nums[i] = nums[i], nums[i]
            # Increment i to move to the next insertion index.
            i += 1

    # Example usage
    nums = [0, 1, 0, 3, 12]
    moveZeros(nums)
    print(nums) # Expected output: [1, 3, 12, 0, 0]

C++
C++

```

```

// C++ solution to move zeros to the end of the vector in-place.
#include <vector>
#include <iostream>
using namespace std;

void moveZeros(vector<int>& nums) {
    // i is used as the index for placing the next non-zero element.
    int i = 0;
    // Iterate through the vector using index 'i'.
    for (int i = 0; i < nums.size(); ++i) {
        // If the element is non-zero, swap if necessary.
        if (nums[i] != 0) {
            swap(nums[i], nums[i]);
            ++i; // Increment i after placing a non-zero element.
        }
    }
}

int main() {
    vector<int> nums = {0, 1, 0, 3, 1, 12};
    moveZeros(nums);
    // Print the modified vector.
    for (int num : nums) {
        cout << num << " ";
    }

    // Expected output: 1 3 12 0 0
    return 0;
}

#A08 EASY
Valid Word Abbreviation
LeetCode · SimplifyLife · WalleCC · LeetCode · Editorial
Two Pointers · String
List Premium 98
Problem:
A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings with their lengths. The lengths should not have leading zeros.
For example, a string such as "substitution" could be abbreviated as (but not limited to):
• "s10n" ("s ubstitut ion")
• "sub4u4" ("sub stit u tion")
• "12n" ("substitution")
• "su3i1u2on" ("su bst it u ti on")
• "substitution" (no substrings replaced)
The following are not valid abbreviations:
• "5Srt" ("s ubst it u ti on", the replaced substrings are adjacent)
• "a010n" (has leading zeros)
• "substitution" (replaces an empty substring)

```

Given a string word and an abbreviation abbr, return whether the string matches the given abbreviation.

A substring is a contiguous non-empty sequence of characters within a string.

Example 1:

Input: word = "internationalization", abbr = "i12iz4n"

Output: true

Explanation: The word "internationalization" can be abbreviated as "i12iz4n" ("i nternatio n al iz atio n").

Example 2:

Input: word = "apple", abbr = "a2e"

Output: false

Explanation: The word "apple" cannot be abbreviated as "a2e".

Constraints:

- 1 <= word.length <= 20
- word consists of only lowercase English letters.
- 1 <= abbr.length <= 10
- abbr consists of lowercase English letters and digits.
- All the integers in abbr will fit in a 32-bit integer.

Python

```

Python
# Python solution for valid word abbreviation
def validWordAbbreviation(word: str, abbr: str) -> bool:
    word_index = 0 # pointer for the word
    abbr_index = 0 # pointer for the abbreviation

    while abbr_index < len(abbr):
        # If current character in word is a digit
        if abbr[abbr_index].isdigit():
            # Leading zero check: if the digit is '0', it is invalid
            if abbr[abbr_index] == '0':
                return False
            # Build the entire number
            num = 0
            while abbr_index < len(abbr) and abbr[abbr_index].isdigit():
                num = num * 10 + int(abbr[abbr_index])
                abbr_index += 1
            # Skip num characters in word
            word_index += num
        else:
            # If letter doesn't match or word index is out of bounds, invalid
            if word_index >= len(word) or word[word_index] != abbr[abbr_index]:
                return False
            word_index += 1
            abbr_index += 1
    return word_index == len(word) and abbr_index == len(abbr)

```

```

# Valid if and only if the entire word was covered
return word_index == len(word)

# Example usage:
print(validWordAbbreviation("internationalization", "i12iz4n")) # Expected output: True
print(validWordAbbreviation("apple", "a2e")) # Expected output: False

Python
Python
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        i = 0 # word's index
        j = 0 # abbr's index

        while i < len(word) and j < len(abbr):
            if word[i] == abbr[j]:
                i += 1
                j += 1
                continue
            if not abbr[j].isdigit() or abbr[j] == '0':
                return False
            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1
            i += num
            return i == len(word) and j == len(abbr)

Python
Python
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        m, n = len(word), len(abbr)
        i = j = 0
        while i < m and j < n:
            if word[i].isdigit():
                if abbr[j] == '0' and x == 0:
                    return False
                x = x * 10 + int(abbr[j])
            else:
                i += 1
                x = 0
                if i == m or word[i] != abbr[j]:
                    return False
                i += 1
            j += 1
            return i == m and j == n

```

```

class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        n, m = len(word), len(abbr)
        i = j = 0
        while i < m and j < n:
            if abbr[i] == '0' and x == 0:
                return False
            return False
            x = x * 10 + int(abbr[i])
        else:
            i += x
            if i == m or word[j] != abbr[j]:
                return False
            i += 1
            j += 1
        return i + x == m and j == n

```

Java

```

class Solution {
    public boolean validWordAbbreviation(String word, String abbr) {
        int i = 0; // word's index
        int j = 0; // abbr's index
        while (i < word.length() && j < abbr.length()) {
            if (word.charAt(i) == abbr.charAt(j)) {
                ++i;
                ++j;
            } else {
                if (abbr.charAt(j) == '0' || (abbr.charAt(j) > '9'))
                    return false;
                int num = 0;
                while (i < word.length() && Character.isDigit(abbr.charAt(j))) {
                    num = num * 10 + abbr.charAt(j) - '0';
                    ++j;
                }
                i += num;
            }
        }
        return i == word.length() && j == abbr.length();
    }
}

```

Java

```

class Solution {
    public boolean validWordAbbreviation(String word, String abbr) {
        int n = word.length(), m = abbr.length();
        int i = 0, j = 0, x = 0;
        for (i = 0; j < m; ++j) {
            char c = abbr.charAt(j);
            if (Character.isDigit(c)) {
                if (c == '0' && x == 0)
                    return false;
                x = x * 10 + (c - '0');
            } else {
                i += x;
                x = 0;
                if (i == n || word.charAt(i) != c)
                    return false;
                ++i;
            }
        }
        return i + x == m && j == m;
    }
}

```

Java

```

class Solution {
    public boolean validWordAbbreviation(String word, String abbr) {
        int n = word.length(), m = abbr.length();
        int i = 0, j = 0, x = 0;
        for (i = 0; j < m; ++j) {
            char c = abbr.charAt(j);
            if (Character.isDigit(c)) {
                if (c == '0' && x == 0)
                    return false;
                x = x * 10 + (c - '0');
            } else {
                i += x;
                x = 0;
                if (i == n || word.charAt(i) != c)
                    return false;
                ++i;
            }
        }
        return i + x == m && j == m;
    }
}

```

TypeScript

TypeScript

```

function validWordAbbreviation(word: string, abbr: string): boolean {
    const [n, m] = [word.length, abbr.length];
    let [i, j, x] = [0, 0, 0];
    for (; i < m && j < n; ++j) {
        if (abbr[j] == '0' && x == 0) {
            return false;
        }
        x = x * 10 + Number(abbr[j]);
    } else {
        i += x;
        x = 0;
        if (i == m || word[i] != abbr[j]) {
            return false;
        }
        ++i;
    }
    return i + x == m && j == n;
}

```

TypeScript

```

function validWordAbbreviation(word: string, abbr: string): boolean {
    const [n, m] = [word.length, abbr.length];
    let [i, j, x] = [0, 0, 0];
    for (; i < m && j < n; ++j) {
        if (abbr[j] == '0' && x == 0) {
            return false;
        }
        x = x * 10 + Number(abbr[j]);
    } else {
        i += x;
        x = 0;
        if (i == m || word[i] != abbr[j]) {
            return false;
        }
        ++i;
    }
    return i + x == m && j == n;
}

```

[7] Two Pointers — Pattern Solution (stored optimal)

Python

```

# Python solution for valid word abbreviation
def validWordAbbreviation(word: str, abbr: str) -> bool:
    word_index = 0 # pointer for the word
    abbr_index = 0 # pointer for the abbreviation
    while abbr_index < len(abbr):
        # if current abbrev char is a digit
        if abbr[abbr_index].isdigit():
            # looking zero check: if the digit is '0', it is invalid
            if abbr[abbr_index] == '0':
                return False
            # build the numeric number
            num = 0
            while abbr_index < len(abbr) and abbr[abbr_index].isdigit():
                num = num * 10 + int(abbr[abbr_index])
                abbr_index += 1
            # skip num characters in word
            word_index += num
        else:
            # if letter doesn't match or word index is out of bounds, invalid
            if word_index == len(word) or word[word_index] != abbr[abbr_index]:
                return False
            word_index += 1
            abbr_index += 1
    # valid if and only if the entire word was covered
    return word_index == len(word)

# Example usage:
print(validWordAbbreviation("internationalization", "i22na9n")) # Expected output: True
print(validWordAbbreviation("apple", "a2e")) # Expected output: False

```

C++

```

// C++ solution for valid word abbreviation
#include <string>
using namespace std;

class Solution {
public:
    bool validWordAbbreviation(string word, string abbr) {
        int wordIndex = 0; // pointer for word
        int abbrIndex = 0; // pointer for abbreviation
        int wordLen = word.size();
        int abbrLen = abbr.size();
        while (abbrIndex < abbrLen) {
            if (isdigit(abbr[abbrIndex])) {
                // looking zero check
                if (abbr[abbrIndex] == '0') return false;
                int num = 0;
                // build the full number
                while (abbrIndex < abbrLen && isdigit(abbr[abbrIndex])) {
                    num = num * 10 + (abbr[abbrIndex] - '0');
                }
                wordIndex += num; // skip num characters in word
            } else {
                // check for letter matching as if wordIndex out of bounds
                if (wordIndex == wordLen || word[wordIndex] != abbr[abbrIndex])
                    return false;
                wordIndex++;
                abbrIndex++;
            }
        }
        return wordIndex == wordLen;
    }
};

// Example usage:
// int main() {
//     Solution sol;
//     bool res1 = sol.validWordAbbreviation("internationalization", "i22na9n"); // true
//     bool res2 = sol.validWordAbbreviation("apple", "a2e"); // false
//     return 0;
// }

```

#977

EASY

Squares of a Sorted Array

LeetCode · SimplyLeetCode · WalkCC · LeetCode · Editorial

Array · Two Pointers · Sorting

Like · 610 · 100

Problem:

Given an integer array `nums` sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1:

Input: `nums = [-4,-1,0,3,10]`
Output: `[0,1,9,16,100]`
Explanation: After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`
Output: `[4,9,49,121]`

Constraints:

- `1 <= nums.length <= 104`
- `-104 <= nums[i] <= 104`
- `nums` is sorted in non-decreasing order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

>> Brute Force (Two Pointers) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def sortedSquares(self, nums):
        # Brute Force O(n^2) - nested loops instead of two pointers
        n = len(nums)
        for i in range(n):
            for j in range(i + 1, n):
                # Check nums[i] and nums[j]
                pass

```

Python

Python

```
# Function to return squares of a sorted array in non-decreasing order.
def sortedSquares(nums):
    # Initialize two pointers at the start and end of the array.
    left, right = 0, len(nums) - 1
    # Create result array of the same length as nums.
    result = [0] * len(nums)
    # Loop to insert the largest square at the current position.
    position = len(nums) - 1
    # Loop until the left and right pointers cross.
    while left <= right:
        # Compare absolute values at the left and right pointers.
        if abs(nums[left]) > abs(nums[right]):
            result[position] = nums[left] * nums[left]
            left += 1 # Move left pointer rightward.
        else:
            result[position] = nums[right] * nums[right]
            right -= 1 # Move right pointer leftward.
        position -= 1 # Move to the next insertion position.
    return result

# Example usage
print(sortedSquares([-4, -1, 0, 3, 10])) # Output: [0, 1, 9, 16, 100]
```

Python

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        l = 0
        r = n - 1
        ans = [0] * n
        while l <= r:
            a = abs(nums[l])
            b = abs(nums[r])
            if a > b:
                ans[l--] = a * a
            else:
                ans[r--] = b * b
        return ans
```

Python

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        ans = []
        l, j = 0, len(nums) - 1
        while l <= j:
            a = nums[l] * nums[l]
            b = nums[j] * nums[j]
            if a > b:
                ans.append(a)
                l += 1
            else:
                ans.append(b)
                j -= 1
        return ans[::-1]
```

Two Pointers · LeetCode · By Pattern

— 1147 —

LeetCode

Python

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        res = [0] * n
        l, j, k = 0, n - 1, n - 1
        while l <= j:
            if nums[l] * nums[l] > nums[j] * nums[j]:
                res[k] = nums[l] * nums[l]
                l += 1
            else:
                res[k] = nums[j] * nums[j]
                j -= 1
            k -= 1
        return res
```

C++

C++

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& nums) {
        const int n = nums.size();
        vector<int> ans(n);
        int l = 0, r = n - 1;

        for (int i = 0, r = n - 1; i <= r; i++) {
            if (abs(nums[l]) > abs(nums[r]))
                ans[i--] = nums[l] * nums[l++];
            else
                ans[i--] = nums[r] * nums[r--];
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(n);
        for (int l = 0, j = n - 1, k = n - 1; l <= j; --k) {
            int a = nums[l] * nums[l];
            int b = nums[j] * nums[j];
            if (a > b) {
                ans[k] = a;
                l++;
            } else {
                ans[k] = b;
                j--;
            }
        }
        return ans;
    }
};
```

C++

Two Pointers · LeetCode · By Pattern

— 1148 —

LeetCode

```
class Solution {
    // @param Integer[] nums
    // @return Integer[]
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        for (int i = 0, j = n - 1, k = n - 1; i <= j; --k) {
            int a = nums[i] * nums[i];
            int b = nums[j] * nums[j];
            if (a > b) {
                ans[k] = a;
                i++;
            } else {
                ans[k] = b;
                j--;
            }
        }
        return ans;
    }
}
```

C++

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& nums) {
        int n = nums.size();
        vector<int> res(n);
        for (int l = 0, j = n - 1, k = n - 1; l <= j; --k) {
            if (nums[l] * nums[l] > nums[j] * nums[j]) {
                res[k--] = nums[l] * nums[l];
                l++;
            } else {
                res[k--] = nums[j] * nums[j];
                j--;
            }
        }
        return res;
    }
};
```

C++

Two Pointers · LeetCode · By Pattern

— 1149 —

LeetCode

```
class Solution {
    // @param Integer[] nums
    // @return Integer[]
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        for (int l = 0, j = n - 1, k = n - 1; l <= j; --k) {
            int a = nums[l] * nums[l];
            int b = nums[j] * nums[j];
            if (a > b) {
                ans[k] = a;
                l++;
            } else {
                ans[k] = b;
                j--;
            }
        }
        return ans;
    }
}
```

Java

Java

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        int l = 0, r = n - 1;
        for (int i = 0, r = n - 1; i <= r; i++) {
            if (Math.abs(nums[l]) > Math.abs(nums[r]))
                ans[i--] = nums[l] * nums[l++];
            else
                ans[i--] = nums[r] * nums[r--];
        }
        return ans;
    }
}
```

Java

Two Pointers · LeetCode · By Pattern

— 1150 —

LeetCode

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        for (int l = 0, j = n - 1, k = n - 1; l <= j; --k) {
            int a = nums[l] * nums[l];
            int b = nums[j] * nums[j];
            if (a > b) {
                ans[k] = a;
                l++;
            } else {
                ans[k] = b;
                j--;
            }
        }
        return ans;
    }
}
```

Java

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        for (int l = 0, j = n - 1, k = n - 1; l <= j; --k) {
            if (nums[l] * nums[l] > nums[j] * nums[j]) {
                res[k--] = nums[l] * nums[l];
                l++;
            } else {
                res[k--] = nums[j] * nums[j];
                j--;
            }
        }
        return res;
    }
}
```

JavaScript

JavaScript

Two Pointers · LeetCode · By Pattern

— 1151 —

LeetCode

```
// @param {number[]} nums
// @return {number[]}
var sortedSquares = function(nums) {
    const n = nums.length;
    const ans = Array(n).fill(0);
    for (let l = 0, j = n - 1, k = n - 1; l <= j; --k) {
        const [a, b] = [nums[l] * nums[l], nums[j] * nums[j]];
        if (a > b) {
            ans[k] = a;
            l++;
        } else {
            ans[k] = b;
            j--;
        }
    }
    return ans;
};
```

JavaScript

```
// @param {number[]} nums
// @return {number[]}
var sortedSquares = function(nums) {
    const n = nums.length;
    const res = new Array(n);
    for (let l = 0, j = n - 1, k = n - 1; l <= j; --k) {
        if (nums[l] * nums[l] > nums[j] * nums[j]) {
            res[k--] = nums[l] * nums[l];
            l++;
        } else {
            res[k--] = nums[j] * nums[j];
            j--;
        }
    }
    return res;
};
```

Go

Go

Two Pointers · LeetCode · By Pattern

— 1152 —

LeetCode

```
func sortedSquares(nums []int) []int {
    n := len(nums)
    ans := make([]int, n)
    for i, j, k := 0, n-1, n-1; i <= j; k-- {
        a := nums[i] * nums[i]
        b := nums[j] * nums[j]
        if a > b {
            ans[k] = a
            i++
        } else {
            ans[k] = b
            j--
        }
    }
    return ans
}
```

Go

```
func sortedSquares(nums []int) []int {
    n := len(nums)
    res := make([]int, n)
    for i, j, k := 0, n-1, n-1; i <= j; {
        if nums[i]*nums[i] > nums[j]*nums[j] {
            res[k] = nums[i] * nums[i]
        } else {
            res[k] = nums[j] * nums[j]
        }
        k--
        if i < j {
            i++
        } else {
            j--
        }
    }
    return res
}
```

Rust

```
impl Solution {
    pub fn sorted_squares(nums: Vec<i32>) -> Vec<i32> {
        let n = nums.len();
        let mut ans = vec![0; n];
        let mut i, mut j := (0, n - 1);
        for k in (0..n).rev() {
            let a = nums[i] * nums[i];
            let b = nums[j] * nums[j];
            if a > b {
                ans[k] = a;
                i += 1;
            } else {
                ans[k] = b;
                j -= 1;
            }
        }
        ans
    }
}
```

Rust

Two Pointers · LeetCode · By Pattern

— 1153 —

LeetCode

```
impl Solution {
    pub fn sorted_squares(nums: Vec<i32>) -> Vec<i32> {
        let n = nums.len();
        let mut l = 0;
        let mut r = n - 1;
        let mut res = vec![0; n];
        for i in (0..n).rev() {
            let a = nums[l] * nums[l];
            let b = nums[r] * nums[r];
            if a > b {
                res[i] = a;
                l += 1;
            } else {
                res[i] = b;
                r -= 1;
            }
        }
        res
    }
}
```

[*] Two Pointers — Pattern Solution (matched from community answers)

Python

```
# Function to return squares of a sorted array in non-decreasing order.
def sortedSquares(nums):
    # Initialize two pointers at the start and end of the array.
    left, right = 0, len(nums) - 1
    # Create a result array of the same length as nums.
    result = [0] * len(nums)
    # Loop to insert the largest square at the current position.
    position = len(nums) - 1
    # Loop until the left and right pointers cross.
    while left <= right:
        # Compare squares of values at the left and right pointers.
        if abs(nums[left]) > abs(nums[right]):
            result[position] = nums[left] * nums[left]
            left += 1 # Move left pointer rightward.
        else:
            result[position] = nums[right] * nums[right]
            right -= 1 # Move right pointer leftward.
        position -= 1 # Move to the next insertion position.
    return result

# Example usage:
print(sortedSquares([-4, -1, 0, 3, 10])) # Output: [0, 1, 9, 16, 100]
```

#11 MEDIUM

Container With Most Water

LeetCode · Simplify · WAACCC · LeetCode · Editorial

Array · Two Pointers · Greedy

LeetCode 11

Problem:

Two Pointers · LeetCode · By Pattern

— 1154 —

LeetCode

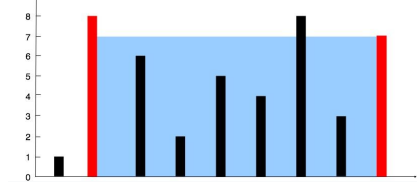
You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the i-th line are (i, 0) and (i, height[i]).

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]

Output: 1

Constraints:

n == height.length

2 <= n <= 10⁵

0 <= height[i] <= 10⁴

>> Brute Force [Two Pointers] Interview Prep

Python

Two Pointers · LeetCode · By Pattern

— 1155 —

LeetCode

```
# Brute Force Approach
class Solution:
    def maxArea(self, height: List[int]) -> int:
        # Brute Force O(n^2) - try every pair of lines
        res = 0
        for i in range(len(height)):
            for j in range(i + 1, len(height)):
                water = min(height[i], height[j]) * (j - i)
                res = max(res, water)
        return res
```

Python

```
# Function to calculate the maximum water container area
def maxArea(height):
    left, right = 0, len(height) - 1 # Initialize two pointers
    max_water = 0 # Variable to store the maximum area
    # Loop until pointers meet
    while left < right:
        # Calculate current area based on smaller height and distance between pointers
        current_area = min(height[left], height[right]) * (right - left)
        max_water = max(max_water, current_area) # Update max area if current is larger
        # Move the pointer corresponding to the smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return max_water

# Test the function with an example
if __name__ == "__main__":
    input_heights = [1,8,6,2,5,4,8,3,7]
    print(maxArea(input_heights)) # Expected output: 49
```

Python

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        n = len(height)
        l = 0
        r = len(height) - 1
        while l < r:
            minHeight = min(height[l], height[r])
            area = max(area, minHeight * (r - l))
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return area
```

Python

Two Pointers · LeetCode · By Pattern

— 1156 —

LeetCode

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r = 0, len(height) - 1
        ans = 0
        while l < r:
            t = min(height[l], height[r]) * (r - l)
            ans = max(ans, t)
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return ans
```

Python

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r = 0, len(height) - 1
        ans = 0
        while l < r:
            t = (r - l) * min(height[l], height[r])
            ans = max(ans, t)
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return ans
```

C++

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int ans = 0;
        int l = 0;
        int r = height.size() - 1;
        while (l < r) {
            const int minHeight = min(height[l], height[r]);
            ans = max(ans, minHeight * (r - l));
            if (height[l] < height[r])
                ++l;
            else
                --r;
        }
        return ans;
    }
};
```

C++

Two Pointers · LeetCode · By Pattern

— 1157 —

LeetCode

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int l = 0, r = height.size() - 1;
        int ans = 0;
        while (l < r) {
            int t = min(height[l], height[r]) * (r - l);
            ans = max(ans, t);
            if (height[l] < height[r]) {
                ++l;
            } else {
                --r;
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
//
// @param Integer[] height
// @return Integer
//
function maxArea(height) {
    let l = 0;
    let r = height.length - 1;
    let ans = 0;
    while (l < r) {
        let t = min(height[l], height[r]) * (r - l);
        ans = Math.max(ans, t);
        if (height[l] < height[r]) {
            ++l;
        } else {
            --r;
        }
    }
    return ans;
}
}
```

C++

Two Pointers · LeetCode · By Pattern

— 1158 —

LeetCode

```

class Solution {
public:
    int maxArea(vector<int>& height) {
        int l = 0, j = height.size() - 1;
        int ans = 0;
        while (l < j) {
            int t = min(height[l], height[j]) * (j - l);
            ans = max(ans, t);
            if (height[l] < height[j]) {
                ++l;
            } else {
                --j;
            }
        }
        return ans;
    }
};

```

C++

```

class Solution {
//
// @param int[] height
// @return int
//
function maxArea(height) {
    left = 0;
    right = height.length - 1;
    maxArea = 0;

    while (left < right) {
        area = min(height[left], height[right]) * (right - left);
        maxArea = max(maxArea, area);

        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    return maxArea;
}
}

```

Java

Java

Two Pointers · LeetCode — By Pattern

— 1159 —

LeetCode

```

class Solution {
public int maxArea(int[] height) {
    int ans = 0;
    int l = 0;
    int r = height.length - 1;

    while (l < r) {
        final int minHeight = Math.min(height[l], height[r]);
        ans = Math.max(ans, minHeight * (r - l));
        if (height[l] < height[r]) {
            ++l;
        } else {
            --r;
        }
    }

    return ans;
}
}

```

Java

```

class Solution {
public int maxArea(int[] height) {
    int l = 0, r = height.length - 1;
    int ans = 0;
    while (l < r) {
        int t = Math.min(height[l], height[r]) * (r - l);
        ans = Math.max(ans, t);
        if (height[l] < height[r]) {
            ++l;
        } else {
            --r;
        }
    }

    return ans;
}
}

```

Java

```

public class Solution {
    public int maxArea(int[] height) {
        int l = 0, r = height.length - 1;
        int ans = 0;
        while (l < r) {
            int t = Math.min(height[l], height[r]) * (r - l);
            ans = Math.max(ans, t);
            if (height[l] < height[r]) {
                ++l;
            } else {
                --r;
            }
        }

        return ans;
    }
}

```

Java

Two Pointers · LeetCode — By Pattern

— 1160 —

LeetCode

```

class Solution {
public int maxArea(int[] height) {
    int l = 0, j = height.length - 1;
    int ans = 0;
    while (l < j) {
        int t = Math.min(height[l], height[j]) * (j - l);
        ans = Math.max(ans, t);
        if (height[l] < height[j]) {
            ++l;
        } else {
            --j;
        }
    }
    return ans;
}
}

```

Java

```

public class Solution {
    public int maxArea(int[] height) {
        int l = 0, j = height.length - 1;
        int ans = 0;
        while (l < j) {
            int t = Math.min(height[l], height[j]) * (j - l);
            ans = Math.max(ans, t);
            if (height[l] < height[j]) {
                ++l;
            } else {
                --j;
            }
        }
        return ans;
    }
}

```

JavaScript

JavaScript

```

//
// @param {number[]} height
// @return {number}
//
var maxArea = function (height) {
    let l, r = 0, height.length - 1;
    let ans = 0;
    while (l < r) {
        const t = Math.min(height[l], height[r]) * (r - l);
        ans = Math.max(ans, t);
        if (height[l] < height[r]) {
            ++l;
        } else {
            --r;
        }
    }
    return ans;
};

```

JavaScript

Two Pointers · LeetCode — By Pattern

— 1161 —

LeetCode

```

//
// @param {number[]} height
// @return {number}
//
var maxArea = function (height) {
    let l = 0;
    let j = height.length - 1;
    let ans = 0;
    while (l < j) {
        const t = Math.min(height[l], height[j]) * (j - l);
        ans = Math.max(ans, t);
        if (height[l] < height[j]) {
            ++l;
        } else {
            --j;
        }
    }
    return ans;
};

```

TypeScript

TypeScript

```

function maxArea(height: number[]): number {
    let l, r = 0, height.length - 1;
    let ans = 0;
    while (l < r) {
        const t = Math.min(height[l], height[r]) * (r - l);
        ans = Math.max(ans, t);
        if (height[l] < height[r]) {
            ++l;
        } else {
            --r;
        }
    }
    return ans;
}

```

TypeScript

```

function maxArea(height: number[]): number {
    let l = 0;
    let j = height.length - 1;
    let ans = 0;
    while (l < j) {
        const t = Math.min(height[l], height[j]) * (j - l);
        ans = Math.max(ans, t);
        if (height[l] < height[j]) {
            ++l;
        } else {
            --j;
        }
    }
    return ans;
}

```

Go

Go

Two Pointers · LeetCode — By Pattern

— 1162 —

LeetCode

```

func maxArea(height []int) (ans int) {
    l, r := 0, len(height)-1
    for l < r {
        t := min(height[l], height[r]) * (r - l)
        ans = max(ans, t)
        if height[l] < height[r] {
            l++
        } else {
            r--
        }
    }
    return
}

```

Go

```

func maxArea(height []int) (ans int) {
    l, r := 0, len(height)-1
    for l < r {
        t := min(height[l], height[r]) * (j - l)
        ans = max(ans, t)
        if height[l] < height[j] {
            ++l
        } else {
            --j
        }
    }
    return
}

```

Rust

Rust

```

impl Solution {
    pub fn max_area(height: Vec<i32>) -> i32 {
        let mut l = 0;
        let mut r = height.len() - 1;
        let mut ans = 0;
        while l < r {
            let t = min(height[l], height[r]) * (r - l) as i32;
            ans = ans.max(t);
            if height[l] < height[r] {
                l += 1;
            } else {
                r -= 1;
            }
        }
        ans
    }
}

```

Rust

Two Pointers · LeetCode — By Pattern

— 1163 —

LeetCode

```

impl Solution {
    pub fn max_area(height: Vec<i32>) -> i32 {
        let mut l = 0;
        let mut j = height.len() - 1;
        let mut res = 0;
        while l < j {
            res = res.max(height[l].min(height[j]) * ((j - l) as i32));
            if height[l] < height[j] {
                l += 1;
            } else {
                j -= 1;
            }
        }
        res
    }
}

```

[*] Two Pointers — Pattern Solution (matched from community answers)

Python

```

# Returns the maximum area of water container
def maxArea(height):
    left, right = 0, len(height) - 1 # Initialize two pointers
    max_water = 0 # Variable to store the maximum area

    # Loop until pointers meet
    while left < right:
        # Calculate current area based on pointer position and distance between pointers
        current_area = min(height[left], height[right]) * (right - left)
        max_water = max(max_water, current_area) # Update maximum area if current is larger

    # Move the pointer corresponding to the smaller height
    if height[left] < height[right]:
        left += 1
    else:
        right -= 1

    return max_water

# Test the function with an example
if __name__ == '__main__':
    input_heights = [1,8,6,2,5,4,4,3,7]
    print(maxArea(input_heights)) # Expected output: 49

```

#15

MEDIUM

3Sum

LeetCode · SimplyLearn · WalkCC · LeetCode · Editorial

Array · Two Pointers · Sorting

Lists · Grid 169

Problem:

Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such that i < j, i < k, and j < k, and nums[i] + nums[j] + nums[k] == 0. Notice that the solution set must not contain duplicate triplets.

Two Pointers · LeetCode — By Pattern

— 1164 —

LeetCode

Example 1:

Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: nums = [0,1,1]
Output: []
Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: nums = [0,0,0]
Output: [[0,0,0]]
Explanation: The only possible triplet sums up to 0.

Constraints:

- 3 <= nums.length <= 3000
- -105 <= nums[i] <= 105

>> Brute Force (Two Pointers) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        # Brute Force O(n^3) - check all triples, deduplicate via set
        nums.sort()
        res = set()
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                for k in range(j + 1, len(nums)):
                    if nums[i] + nums[j] + nums[k] == 0:
                        res.add(tuple(sorted([nums[i], nums[j], nums[k]])))
        return [list(t) for t in res]
```

Python

Python

```
# Function to find all unique triplets that sum to zero.
def threeSum(nums):
    # Sort the array to facilitate finding duplicates and using two pointers.
    nums.sort()
    n = len(nums)

    # Iterate through the array, fixing one element at a time.
    for i in range(n - 2):
        # Skip duplicate elements for the first element.
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1
        while left < right:
            current_sum = nums[i] + nums[left] + nums[right]
            # Check if the triplet sums up to zero.
            if current_sum == 0:
                result.append([nums[i], nums[left], nums[right]])
                # Skip duplicates for the second element.
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                # Skip duplicates for the third element.
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1
            elif current_sum < 0:
                left += 1
            else:
                right -= 1

    return result

# Example usage:
nums = [-1, 0, 1, 2, -1, -4]
result = threeSum(nums)
```

Python

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        if len(nums) < 3:
            return []

        ans = []
        nums.sort()

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the
            # remaining numbers in [i + 1, n - 1].
            l = i + 1
            r = len(nums) - 1
            while l < r:
                sum = nums[i] + nums[l] + nums[r]
                if sum == 0:
                    ans.append([nums[i], nums[l], nums[r]])
                    l += 1
                    r -= 1
                    while nums[l] == nums[l - 1] and l < r:
                        l += 1
                    while nums[r] == nums[r + 1] and l < r:
                        r -= 1
                elif sum < 0:
                    l += 1
                else:
                    r -= 1

        return ans
```

Python

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue
            j, k = i + 1, n - 1
            while j < k:
                s = nums[i] + nums[j] + nums[k]
                if s == 0:
                    ans.append([nums[i], nums[j], nums[k]])
                    j += 1
                    k -= 1
                    while j < k and nums[j] == nums[j - 1]:
                        j += 1
                    while j < k and nums[k] == nums[k + 1]:
                        k -= 1
                elif s < 0:
                    j += 1
                else:
                    k -= 1
```

Python

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue
            j, k = i + 1, n - 1
            while j < k:
                s = nums[i] + nums[j] + nums[k]
                if s == 0:
                    ans.append([nums[i], nums[j], nums[k]])
                    j += 1
                    k -= 1
                    while j < k and nums[j] == nums[j - 1]:
                        j += 1
                    while j < k and nums[k] == nums[k + 1]:
                        k -= 1
                elif s < 0:
                    j += 1
                else:
                    k -= 1
```

C++

C++

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        int n = nums.size();
        for (int i = 0; i < n - 2; ++i) {
            if (i > 0 and nums[i] == nums[i - 1]) continue;
            int left = i + 1, right = n - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    ans.push_back({nums[i], nums[left], nums[right]});
                    while (left < right and nums[left] == nums[left + 1]) ++left;
                    while (left < right and nums[right] == nums[right - 1]) --right;
                } else if (sum < 0) ++left;
                else --right;
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        sort(nums.begin(), nums.end());
        int n = nums.size();
        for (int i = 0; i < n - 2; ++i) {
            if (i > 0 and nums[i] == nums[i - 1]) continue;
            int left = i + 1, right = n - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    ans.push_back({nums[i], nums[left], nums[right]});
                    while (left < right and nums[left] == nums[left + 1]) ++left;
                    while (left < right and nums[right] == nums[right - 1]) --right;
                } else if (sum < 0) ++left;
                else --right;
            }
        }
        return ans;
    }
};
```

C++

```

class Solution {
    //
    // @param int[] nums
    // @return int[][]
    //
    function threeSum(nums) {
        result = [];
        n = nums.length;

        sort(nums);
        for (let i = 0; i < n - 2; i++) {
            if (i > 0 && nums[i] === nums[i - 1]) {
                continue;
            }

            let left = i + 1;
            let right = n - 1;

            while (left < right) {
                sum = nums[i] + nums[left] + nums[right];

                if (sum === 0) {
                    triplet = [nums[i], nums[left], nums[right]];

                    result.push(triplet);

                    while (left < right && nums[left] === nums[left + 1]) {
                        left++;
                    }

                    while (left < right && nums[right] === nums[right - 1]) {
                        right--;
                    }

                    left++;
                    right--;
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }

        return result;
    }
}

```

```

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> ans = new ArrayList<>();
        let n = nums.length;
        for (let i = 0; i < n - 2 && nums[i] <= 0; i++) {
            if (i > 0 && nums[i] === nums[i - 1]) {
                continue;
            }
            let j = i + 1, k = n - 1;
            while (j < k) {
                let x = nums[i] + nums[j] + nums[k];
                if (x === 0) {
                    ans.add([nums[i], nums[j], nums[k]]);
                    while (j < k && nums[j] === nums[j + 1]) {
                        j++;
                    }
                    while (j < k && nums[k] === nums[k - 1]) {
                        k--;
                    }
                } else if (x > 0) {
                    k--;
                } else {
                    j++;
                }
            }
        }
        return ans;
    }
}

```

```

}
return ans;
}

JavaScript
JavaScript
# @param {Integer[]} nums
# @return {Integer[][]}
def threeSum(nums)
    res = []
    nums.sort()
    for i in 0..(nums.length - 3)
        next if i > 0 && nums[i] == nums[i - 1]
        j = i + 1
        k = nums.length - 1
        while j < k do
            sum = nums[i] + nums[j] + nums[k]
            if sum == 0
                j++
            elsif sum > 0
                k--
            else
                res += [[nums[i], nums[j], nums[k]]]
                j++
                k--
                j++ while nums[j] == nums[j - 1]
                k-- while nums[k] == nums[k + 1]
            end
        end
    end
    res
end

```

```

//
// @param {number[]} nums
// @return {number[][]}
//
var threeSum = function(nums) {
    const n = nums.length;
    nums.sort((a, b) => a - b);
    const ans = [];
    for (let i = 0; i < n - 2 && nums[i] <= 0; i++) {
        if (i > 0 && nums[i] === nums[i - 1]) {
            continue;
        }
        let j = i + 1;
        let k = n - 1;
        while (j < k) {
            const x = nums[i] + nums[j] + nums[k];
            if (x === 0) {
                ans.push([nums[i], nums[j], nums[k]]);
                while (j < k && nums[j] === nums[j + 1]) {
                    j++;
                }
                while (j < k && nums[k] === nums[k - 1]) {
                    k--;
                }
            } else if (x > 0) {
                k--;
            } else {
                j++;
            }
        }
    }
    return ans;
};

```

```

# @param {Integer[]} nums
# @return {Integer[][]}
def threeSum(nums)
    res = []
    nums.sort()
    for i in 0..(nums.length - 3)
        next if i > 0 && nums[i] == nums[i - 1]
        j = i + 1
        k = nums.length - 1
        while j < k do
            sum = nums[i] + nums[j] + nums[k]
            if sum == 0
                j++
            elsif sum > 0
                k--
            else
                res += [[nums[i], nums[j], nums[k]]]
                j++
                k--
                j++ while nums[j] == nums[j - 1]
                k-- while nums[k] == nums[k + 1]
            end
        end
    end
    res
end

```

```

}
return ans;
}

Go
Go
func threeSum(nums []int) [][]int {
    sort.Ints(nums)
    n := len(nums)
    for i := 0; i < n-2 && nums[i] <= 0; i++ {
        if i > 0 && nums[i] == nums[i-1] {
            continue
        }
        j, k := i+1, n-1
        for j < k {
            x := nums[i] + nums[j] + nums[k]
            if x == 0 {
                res = append(res, []int{nums[i], nums[j], nums[k]})
                j, k = j+1, k-1
                for j < k && nums[j] == nums[j+1] {
                    j++
                }
                for j < k && nums[k] == nums[k-1] {
                    k--
                }
            } else if x > 0 {
                k--
            } else {
                j++
            }
        }
    }
    return res
}

```



```
func threeSum(nums: [Int]) List<[[Int]]> {
    sort(nums)
    n = nums.size
    for i in 0..n-2 {
        if i > 0 && nums[i] == nums[i-1] {
            continue
        }
        j, k := i+1, n-1
        for j < k {
            s = nums[i] + nums[j] + nums[k]
            if s == 0 {
                j++
            } else if s > 0 {
                k--
            } else {
                ans = append(ans, [i, nums[i], nums[j], nums[k]])
                j, k = j+1, k-1
                for j < k && nums[j] == nums[j+1] {
                    j++
                }
                for j < k && nums[k] == nums[k-1] {
                    k--
                }
            }
        }
    }
    return ans
}
```

Rust

Rust

use std::cmp::Ordering;

```
impl Solution {
    pub fn three_sum_closest(nums: Vec<i32>) -> Vec<i32> {
        nums.sort();
        let n = nums.len();
        let mut res = vec![0];
        let mut i = 0;
        while i < n - 2 && nums[i] <= 0 {
            let mut l = i + 1;
            let mut r = n - 1;
            while l < r {
                match (nums[i] + nums[l] + nums[r]).cmp(&0) {
                    Ordering::Less => {
                        l += 1;
                    }
                    Ordering::Greater => {
                        r -= 1;
                    }
                    Ordering::Equal => {
                        res.push(vec![nums[i], nums[l], nums[r]]);
                        l += 1;
                        r -= 1;
                        while l < n && nums[l] == nums[l + 1] {
                            l += 1;
                        }
                    }
                }
            }
        }
        res.push(vec![nums[i], nums[l], nums[r]]);
        l += 1;
        r -= 1;
        while l < n && nums[l] == nums[l + 1] {
            l += 1;
        }
    }
}
```

Two Pointers · LeetCode · By Pattern

— 177 —

LeetCode

```
} while r > 0 && nums[r] == nums[r + 1] {
    r -= 1;
}
}
}
}
}
}
}
}
}
}
}
```

[*] Two Pointers — Pattern Solution (matched from community answers)

Python

```
# Function to find all unique triplets that sum to zero.
def threeSum(nums):
    # Sort the array to facilitate finding duplicates and using two pointers.
    nums.sort()
    n = len(nums)

    # Iterate through the array, fixing one element at a time.
    for i in range(n - 2):
        # Skip duplicate elements for the first element.
        if i > 0 && nums[i] == nums[i - 1]:
            continue

        left, right = i + 1, n - 1
        while left < right:
            current_sum = nums[i] + nums[left] + nums[right]
            # Check if the triplet sums up to zero.
            if current_sum == 0:
                result.append([nums[i], nums[left], nums[right]])
                # Skip duplicates for the second element.
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                # Skip duplicates for the third element.
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1
            elif current_sum < 0:
                left += 1
            else:
                right -= 1

    return result

# Example usage:
nums = [-1, 0, 1, 2, -1, -4]
print(threeSum(nums))
```

Two Pointers · LeetCode · By Pattern

— 178 —

LeetCode

#16

MEDIUM

3Sum Closest

LeetCode · SimplifyLeet · WubCC · LeetCode · Editorial

Array · Two Pointers · Sorting

LeetCode Grind 169

Problem:

Given an integer array `nums` of length `n` and an integer `target`, find three integers at distinct indices in `nums` such that the sum is closest to `target`.

Return the sum of the three integers.

You may assume that each input would have exactly one solution.

Example 1:

Input: `nums = [-1,2,1,-4]`, `target = 1`

Output: `2`

Explanation: The sum that is closest to the target is 2. $(-1 + 2 + 1 = 2)$.

Example 2:

Input: `nums = [0,0,0]`, `target = 1`

Output: `0`

Explanation: The sum that is closest to the target is 0. $(0 + 0 + 0 = 0)$.

Constraints:

$3 \leq \text{nums.length} \leq 500$

$-1000 \leq \text{nums}[i] \leq 1000$

$-104 \leq \text{target} \leq 104$

>> Brute Force (Two Pointers) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        # Sort the array to facilitate finding duplicates and using two pointers.
        n = len(nums)
        ans = inf
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    # Check if the triplet sums up to zero.
                    s = nums[i] + nums[j] + nums[k]
                    if abs(s - target) < abs(ans - target):
                        ans = s
        return ans
```

Python

Python

Two Pointers · LeetCode · By Pattern

— 179 —

LeetCode

```
# Python solution for 3Sum Closest
def threeSumClosest(nums, target):
    # Sort the array to facilitate finding duplicates and using two pointers.
    nums.sort()
    n = len(nums)
    best_sum = sum(nums[:3])

    # Iterate through each number as the first element
    for i in range(n - 2):
        left, right = i + 1, len(nums) - 1
        # Use two pointers to check pairs
        while left < right:
            current_sum = nums[i] + nums[left] + nums[right]
            # Update best_sum if current_sum is closer to target.
            if abs(target - current_sum) < abs(target - best_sum):
                best_sum = current_sum
            # Move left pointer if current_sum is less than target to increase sum
            if current_sum < target:
                left += 1
            # Move right pointer if current_sum is greater than target to decrease sum
            elif current_sum > target:
                right -= 1
            # If the current_sum exactly equals target, return the target sum
            else:
                return current_sum

    # Return the closest sum found
    return best_sum

# Example usage:
nums = [-1, 2, 1, -4]
target = 1
print(threeSumClosest(nums, target))
```

Python

```
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        ans = nums[0] + nums[1] + nums[2]
        nums.sort()

        for i in range(len(nums) - 2):
            if i > 0 && nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the
            # remaining numbers in [i + 1, n - 1].
            l, r = i + 1, len(nums) - 1
            while l < r:
                s = nums[i] + nums[l] + nums[r]
                if s == target:
                    return s
                if abs(s - target) < abs(ans - target):
                    ans = s
                if s < target:
                    l += 1
                else:
                    r -= 1

        return ans
```

Two Pointers · LeetCode · By Pattern

— 180 —

LeetCode

```
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        nums.sort()
        n = len(nums)
        ans = inf
        for i, v in enumerate(nums):
            j, k = i + 1, n - 1
            while j < k:
                t = v + nums[j] + nums[k]
                if t == target:
                    return t
                if abs(t - target) < abs(ans - target):
                    ans = t
                if t < target:
                    j += 1
                else:
                    k -= 1
        return ans
```

Python

```
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        nums.sort()
        n = len(nums)
        ans = inf
        for i, v in enumerate(nums):
            j, k = i + 1, n - 1
            while j < k:
                t = v + nums[j] + nums[k]
                if t == target:
                    return t
                if abs(t - target) < abs(ans - target):
                    ans = t
                if t < target:
                    j += 1
                else:
                    k -= 1
        return ans
```

C++

C++

Two Pointers · LeetCode · By Pattern

— 181 —

LeetCode

```
class Solution {
public:
    int threeSumClosest(vector<int>& nums, int target) {
        int ans = nums[0] + nums[1] + nums[2];
        ranges::sort(nums);

        for (int i = 0; i < nums.size(); ++i) {
            if (i > 0 && nums[i] == nums[i - 1])
                continue;
            // Choose nums[i] as the first number in the triplet, then search the
            // remaining numbers in [i + 1, n - 1].
            int l = i + 1;
            int r = nums.size() - 1;
            while (l < r) {
                const int sum = nums[i] + nums[l] + nums[r];
                if (sum == target)
                    return sum;
                if (abs(sum - target) < abs(ans - target))
                    ans = sum;
                if (sum < target)
                    ++l;
                else
                    --r;
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int threeSumClosest(vector<int>& nums, int target) {
        sort(nums.begin(), nums.end());
        int ans = 2 * 1000000000;
        int n = nums.size();
        for (int i = 0; i < n; ++i) {
            int l = i + 1, r = n - 1;
            while (l < r) {
                int t = nums[i] + nums[l] + nums[r];
                if (t == target) return t;
                if (abs(t - target) < abs(ans - target)) ans = t;
                if (t < target)
                    ++l;
                else
                    --r;
            }
        }
        return ans;
    }
};
```

C++

Two Pointers · LeetCode · By Pattern

— 182 —

LeetCode

```

class Solution {
    //
    // @param int[] nums
    // @param int target
    // @return int
    //
    function threeSumClosest(nums, target) {
        let n = nums.length;
        let ans = 0;
        let diff = Math.abs(nums[0] + nums[1] - target);
        sort(nums);

        for (let i = 0; i < n - 2; i++) {
            let left = i + 1;
            let right = n - 1;

            while (left < right) {
                let sum = nums[i] + nums[left] + nums[right];
                let diff = Math.abs(sum - target);

                if (diff < ans) {
                    ans = diff;
                    let closestSum = sum;
                }

                if (sum < target) {
                    left++;
                } else if (sum > target) {
                    right--;
                } else {
                    return sum;
                }
            }
        }

        return closestSum;
    }
}

```

C++

```

class Solution {
    public:
        int threeSumClosest(vector<int>& nums, int target) {
            sort(nums.begin(), nums.end());
            int ans = 0;
            int n = nums.size();
            for (int i = 0; i < n; i++) {
                int j = i + 1;
                int k = n - 1;
                while (j < k) {
                    int t = nums[i] + nums[j] + nums[k];
                    if (t == target) return t;
                    if (abs(t - target) < abs(ans - target)) ans = t;
                    if (t > target) k--;
                    else j++;
                }
            }
            return ans;
        }
};

```

C++

```

class Solution {
    //
    // @param int[] nums
    // @param int target
    // @return int
    //
    function threeSumClosest(nums, target) {
        let n = nums.length;
        let ans = 0;
        let diff = Math.abs(nums[0] + nums[1] - target);
        sort(nums);

        for (let i = 0; i < n - 2; i++) {
            let left = i + 1;
            let right = n - 1;

            while (left < right) {
                let sum = nums[i] + nums[left] + nums[right];
                let diff = Math.abs(sum - target);

                if (diff < ans) {
                    ans = sum;
                }

                if (sum < target) {
                    left++;
                } else if (sum > target) {
                    right--;
                } else {
                    return sum;
                }
            }
        }

        return ans;
    }
}

```

C++

```

class Solution {
    //
    // @param int[] nums
    // @param int target
    // @return int
    //
    function threeSumClosest(nums, target) {
        let n = nums.length;
        let ans = 0;
        let diff = Math.abs(nums[0] + nums[1] - target);
        sort(nums);

        for (let i = 0; i < n - 2; i++) {
            let left = i + 1;
            let right = n - 1;

            while (left < right) {
                let sum = nums[i] + nums[left] + nums[right];
                let diff = Math.abs(sum - target);

                if (diff < ans) {
                    ans = sum;
                }

                if (sum < target) {
                    left++;
                } else if (sum > target) {
                    right--;
                } else {
                    return sum;
                }
            }
        }

        return ans;
    }
}

```

Java

```

class Solution {
    public int threeSumClosest(int[] nums, int target) {
        int ans = nums[0] + nums[1] + nums[2];
        Arrays.sort(nums);

        for (let i = 0; i < n - 2; i++) {
            let left = i + 1;
            let right = n - 1;

            while (left < right) {
                let sum = nums[i] + nums[left] + nums[right];
                let diff = Math.abs(sum - target);

                if (diff < ans) {
                    ans = sum;
                }

                if (sum < target) {
                    left++;
                } else if (sum > target) {
                    right--;
                } else {
                    return sum;
                }
            }
        }

        return ans;
    }
}

```

Java

```

class Solution {
    public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int ans = 0;
        int n = nums.length;
        for (let i = 0; i < n; i++) {
            int j = i + 1;
            int k = n - 1;
            while (j < k) {
                int t = nums[i] + nums[j] + nums[k];
                if (t == target) return t;
                if (Math.abs(t - target) < Math.abs(ans - target)) {
                    ans = t;
                }
                if (t > target) {
                    k--;
                } else {
                    j++;
                }
            }
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int ans = 0;
        int n = nums.length;
        for (let i = 0; i < n; i++) {
            int j = i + 1;
            int k = n - 1;
            while (j < k) {
                int t = nums[i] + nums[j] + nums[k];
                if (t == target) return t;
                if (Math.abs(t - target) < Math.abs(ans - target)) {
                    ans = t;
                }
                if (t > target) {
                    k--;
                } else {
                    j++;
                }
            }
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int ans = 0;
        int n = nums.length;
        for (let i = 0; i < n; i++) {
            int j = i + 1;
            int k = n - 1;
            while (j < k) {
                int t = nums[i] + nums[j] + nums[k];
                if (t == target) return t;
                if (Math.abs(t - target) < Math.abs(ans - target)) {
                    ans = t;
                }
                if (t > target) {
                    k--;
                } else {
                    j++;
                }
            }
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int ans = 0;
        int n = nums.length;
        for (let i = 0; i < n; i++) {
            int j = i + 1;
            int k = n - 1;
            while (j < k) {
                int t = nums[i] + nums[j] + nums[k];
                if (t == target) return t;
                if (Math.abs(t - target) < Math.abs(ans - target)) {
                    ans = t;
                }
                if (t > target) {
                    k--;
                } else {
                    j++;
                }
            }
        }
        return ans;
    }
}

```

JavaScript

JavaScript

```

//
// @param number[] nums
// @param number target
// @return number
//
var threeSumClosest = function(nums, target) {
    nums.sort((a, b) => a - b);
    let ans = 0;
    let n = nums.length;
    for (let i = 0; i < n; i++) {
        let j = i + 1;
        let k = n - 1;
        while (j < k) {
            let t = nums[i] + nums[j] + nums[k];
            if (t == target) return t;
            if (Math.abs(t - target) < Math.abs(ans - target)) {
                ans = t;
            }
            if (t > target) {
                k--;
            } else {
                j++;
            }
        }
    }
    return ans;
};

```

JavaScript

```

//
// @param number[] nums
// @param number target
// @return number
//
var threeSumClosest = function(nums, target) {
    nums.sort((a, b) => a - b);
    let ans = 0;
    let n = nums.length;
    for (let i = 0; i < n; i++) {
        let j = i + 1;
        let k = n - 1;
        while (j < k) {
            let t = nums[i] + nums[j] + nums[k];
            if (t == target) return t;
            if (Math.abs(t - target) < Math.abs(ans - target)) {
                ans = t;
            }
            if (t > target) {
                k--;
            } else {
                j++;
            }
        }
    }
    return ans;
};

```



```
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        n = len(nums)
        i = next((j for j in range(n - 2, -1, -1) if nums[j] < nums[j + 1]), -1)
        if i:
            j = next((j for j in range(n - 1, 1, -1) if nums[j] > nums[i]))
            nums[i], nums[j] = nums[j], nums[i]
        nums[i + 1:] = nums[i + 1:][::-1]
```

Python

```

>>> i = 3
>>> ~i
~4
>>> bool(~i)
True
#####

>>> j = ~i
>>> ~j
4
>>> bool(~j)
False
#####

>>> a = (i for i in range (10, -1, -1) if i < 0)
>>> a
<generator object <genspr> at 0x1a17a0eb>
>>> next(a)
5
>>>
>>>
>>> b = (i for i in range (10, -1, -1) if i < 0)
>>> next(b)
Traceback (most recent call last):

```

```
File "main.py" line 1, in module
    Steploration
      ^^^^^^^
-> main((), [])
-1
>>>
>>>
```

Class Solution:

```
def maxPermutation(self, nums: List[int]) -> None:
    """
        Do not return anything, modify nums in-place instead.
    """
```

n = len(nums)

```
nums.sort() # default option -1
i = max(0) for i in range(n - 2, -1, -1) if nums[i] + nums[i+1] == -1
if i != -1:
    j = max(i) for j in range(n - 1, i, -1) if nums[j] > nums[i+1]
    nums[i], nums[j] = nums[j], nums[i]
nums[i+1] += nums[i+1] // (-1)
return nums
```

XXXXXXXXXXXX

```
>>> a=[1,2,3]
>>> reversed(a)

<list_reverseiterator object at 0x108458be0>
>>> list(reversed(a))
[3, 2, 1]
```

```
# but, use reversed(a) to directly assign values is ok
>>> b=[4,5,6,7,8]
>>> b[:3] = reversed(a)
>>> b
```

[illegible]

```
nums.reverse() # for[] returns[]
```

C++

```

class Solution {
public:
    void swapPermutation(vector<int>& nums) {
        const int n = nums.size();

        // From back to front, find the first number + nums[i] + 1.
        for (int i = n - 2; i >= 0; --i) {
            if (nums[i] + nums[i + 1])
                break;

            // From back to front, find the first number + nums[i], swap it with
            // the first number + nums[i] + 1.
            for (int j = i + 1; j < n; ++j) {
                if (nums[j] + nums[i]) {
                    swap(nums[i], nums[j]);
                    break;
                }
            }

            // Reverse nums[i + 1 .. n - 1].
            reverse(nums, i + 1, n - 1);
        }
    }

private:
    void reverse(vector<int>& nums, int l, int r) {
        while (l < r)
            swap(nums[l++], nums[r--]);
    }
};

```

```
C++
class Solution {
public:
    void nextPermutation(vector<int>& nums) {
        int n = nums.size();
        int i = n - 2;
        while (<i && nums[i] >= nums[i + 1]) {
            --i;
        }
        if (<i < 0)
            for (int j = n - 1; j >= 1; --j) {
                if (nums[j] > nums[j - 1]) {
                    swap(nums[i], nums[j]);
                    break;
                }
            }
        reverse(nums.begin() + i + 1, nums.end());
    }
};
```

C++

[illegible]

```
C++
class Solution {
public:
    void nextPermutation(vector<int>& nums) {
        int n = nums.size();
        int i = n - 2;
        while (-1 < i && nums[i] >= nums[i + 1]) {
            --i;
        }
        if (-1 < i) {
            for (int j = n - 1; j > i; --j) {
                if (nums[j] > nums[i]) {
                    swap(nums[i], nums[j]);
                    break;
                }
            }
        }
        reverse(nums.begin() + i + 1, nums.end());
    }
};
```

C++

```

// test solution {
//
// @param Integer[] nums
// @return void
//}

function mergeSort(nums) {
    let count = 0;
    let n = nums.length;
    while (n > 1) {
        let mid = Math.floor(n / 2);
        mergeSort(nums.slice(0, mid));
        mergeSort(nums.slice(mid, n));
        merge(nums, 0, mid, n);
        count += mid * (n - mid);
        n = mid;
    }
    return count;
}

function merge(nums, start, mid, end) {
    let temp = new Array(end - start + 1).fill(0);
    let i = start, j = mid, k = start;
    while (i < mid && j < end) {
        if (nums[i] <= nums[j]) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }
    while (i < mid) temp[k++] = nums[i++];
    while (j < end) temp[k++] = nums[j++];
    for (let i = start; i < end; i++) {
        nums[i] = temp[i - start];
    }
}
}

```

Java

```

class Solution {
public: void nextPermutation(int[] nums) {
    final int n = nums.length;

    // From back to front, find the first number + nums[i + 1].
    for (int i = n - 2; i >= 0; --i) {
        if (nums[i] < nums[i + 1])
            break;
    }

    // From back to front, find the first number > nums[i], swap it with
    // it (i = 0)
    for (int j = n - 1; j > i; --j) {
        if (nums[j] > nums[i]) {
            swap(nums, i, j);
            break;
        }
    }

    // Reverse nums[i + 1, n - 1].
    reverse(nums, i + 1, n - 1);
}

private void reverse(int[] nums, int l, int r) {
    while (l < r)
        swap(nums, l++, r--);
}

private void swap(int[] nums, int l, int r) {
    final int temp = nums[l];
    nums[l] = nums[r];
    nums[r] = temp;
}
}

```

Java

```

class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n - 2;
        for (; i >= 0; --i) {
            if (nums[i] < nums[i + 1]) {
                break;
            }
        }
        if (i >= 0) {
            for (int j = n - 1; j > i; --j) {
                if (nums[j] > nums[i]) {
                    swap(nums, i, j);
                    break;
                }
            }
        }
        for (int j = i + 1, k = n - 1; j < k; ++j, --k) {
            swap(nums, j, k);
        }
    }

    private void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

```

Java
public class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n - 2;
        while (i >= 0 && nums[i] >= nums[i + 1]) {
            --i;
        }
        if (i >= 0) {
            for (int j = n - 1; j > i; --j) {
                if (nums[j] > nums[i]) {
                    swap(nums, i, j);
                    break;
                }
            }
        }
        for (int j = i + 1, k = n - 1; j < k; ++j, --k) {
            swap(nums, j, k);
        }
    }

    private void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

Two Pointers · NextPermutation — By Pattern

— 1201 —

NextPermutation

```

Java
import java.util.Arrays;

public class NextPermutation {
    // time: O(N^2)
    // space: O(1)
    public class Solution {
        public void nextPermutation(int[] nums) {
            if (nums == null || nums.length == 0) {
                return;
            }
            // [1,2,3,4,5,6,7,8,9,10] -> next
            for (int i = nums.length - 2; i >= 0; --i) { // 3, 4, 5, 2, 1 // 10, 1 + 10 - 1, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
                if (nums[i] < nums[i + 1]) { // 11, 12, 13, 14, 15, 16, 17, 18, 19, 20
                    for (int j = nums.length - 1; j > i; --j) {
                        if (nums[j] > nums[i]) {
                            // swap nums[i] <= 10
                            swap(nums, i, j); // 3, 5, 4, 2, 1
                        }
                    }
                    // reverse nums[i+1.....] to next position
                    reverse(nums, i + 1, nums.length - 1); // [4,2,1] reverse to [1,2,4] => 3, 5, 1, 2, 4
                }
            }
            return;
        }

        private void swap(int[] nums, int i, int j) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }

        private void reverse(int[] nums, int i, int j) {
            while (i < j) {
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
            }
        }

        private void reverse(int[] nums, int i, int j) {
            while (i < j) {
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
            }
        }
    }
}

```

Two Pointers · NextPermutation — By Pattern

— 1202 —

NextPermutation

```

Java
class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n - 2;
        for (; i >= 0; --i) {
            if (nums[i] < nums[i + 1]) {
                break;
            }
        }
        if (i >= 0) {
            for (int j = n - 1; j > i; --j) {
                if (nums[j] > nums[i]) {
                    swap(nums, i, j);
                    break;
                }
            }
        }
        for (int j = i + 1, k = n - 1; j < k; ++j, --k) {
            swap(nums, j, k);
        }
    }

    private void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

```

Java
public class Solution {
    public void nextPermutation(int[] nums) {
        int n = nums.length;
        int i = n - 2;
        while (i >= 0 && nums[i] >= nums[i + 1]) {
            --i;
        }
        if (i >= 0) {
            for (int j = n - 1; j > i; --j) {
                if (nums[j] > nums[i]) {
                    swap(nums, i, j);
                    break;
                }
            }
        }
        for (int j = i + 1, k = n - 1; j < k; ++j, --k) {
            swap(nums, j, k);
        }
    }

    private void swap(int[] nums, int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

Two Pointers · NextPermutation — By Pattern

— 1203 —

NextPermutation

```

JavaScript
//
// @param {number[]} nums
// @return {void} Do not return anything, modify nums in-place instead.
//
var nextPermutation = function(nums) {
    const n = nums.length;
    let i = n - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) {
        --i;
    }
    if (i >= 0) {
        let j = n - 1;
        while (j > i && nums[j] >= nums[i]) {
            --j;
        }
        [nums[i], nums[j]] = [nums[j], nums[i]];
    }
    for (let t = i + 1, j = n - 1; t < j; ++t, --j) {
        [nums[t], nums[j]] = [nums[j], nums[t]];
    }
};

```

```

JavaScript
//
// @param {number[]} nums
// @return {void} Do not return anything, modify nums in-place instead.
//
var nextPermutation = function(nums) {
    const n = nums.length;
    let i = n - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) {
        --i;
    }
    if (i >= 0) {
        let j = n - 1;
        while (j > i && nums[j] >= nums[i]) {
            --j;
        }
        [nums[i], nums[j]] = [nums[j], nums[i]];
    }
    for (let t = i + 1, j = n - 1; t < j; ++t, --j) {
        [nums[t], nums[j]] = [nums[j], nums[t]];
    }
};

```

Two Pointers · NextPermutation — By Pattern

— 1204 —

NextPermutation

```

function nextPermutation(nums: number[]): void {
    const n = nums.length;
    let i = n - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) {
        --i;
    }
    if (i >= 0) {
        for (let j = n - 1; j > i; --j) {
            if (nums[j] > nums[i]) {
                [nums[i], nums[j]] = [nums[j], nums[i]];
                break;
            }
        }
    }
    for (let j = i + 1, k = n - 1; j < k; ++j, --k) {
        [nums[j], nums[k]] = [nums[k], nums[j]];
    }
}

```

```

TypeScript
function nextPermutation(nums: number[]): void {
    const n = nums.length;
    let i = n - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) {
        --i;
    }
    if (i >= 0) {
        for (let j = n - 1; j > i; --j) {
            if (nums[j] > nums[i]) {
                [nums[i], nums[j]] = [nums[j], nums[i]];
                break;
            }
        }
    }
    for (let j = i + 1, k = n - 1; j < k; ++j, --k) {
        [nums[j], nums[k]] = [nums[k], nums[j]];
    }
}

```

Go

Two Pointers · NextPermutation — By Pattern

— 1205 —

NextPermutation

```

func nextPermutation(nums []int) {
    n := len(nums)
    i := n - 2
    for ; i >= 0 && nums[i] >= nums[i + 1]; i-- {
    }
    if i >= 0 {
        for j := n - 1; j > i; j-- {
            if nums[j] > nums[i] {
                nums[i], nums[j] = nums[j], nums[i]
                break
            }
        }
    }
    for j, k := i + 1, n - 1; j < k; j, k = j + 1, k - 1 {
        nums[j], nums[k] = nums[k], nums[j]
    }
}

```

```

Go
func nextPermutation(nums []int) {
    n := len(nums)
    i := n - 2
    for ; i >= 0 && nums[i] >= nums[i + 1]; i-- {
    }
    if i >= 0 {
        for j := n - 1; j > i; j-- {
            if nums[j] > nums[i] {
                nums[i], nums[j] = nums[j], nums[i]
                break
            }
        }
    }
    for j, k := i + 1, n - 1; j < k; j, k = j + 1, k - 1 {
        nums[j], nums[k] = nums[k], nums[j]
    }
}

```

[*] Two Pointers — Pattern Solution (matched from community answers)

Python

Two Pointers · NextPermutation — By Pattern

— 1206 —

NextPermutation

```
def nextPermutation(nums):
    # length of the array
    n = len(nums)
    # Find the first index from the right where the current number is less than the next number
    i = n - 2
    while i >= 0 and nums[i] >= nums[i + 1]:
        i -= 1
    # If such an index is found, find the number just larger than nums[i] from the right side to swap
    if i >= 0:
        j = n - 1
        while nums[j] <= nums[i]:
            j -= 1
        nums[i], nums[j] = nums[j], nums[i] # Swap the two numbers
    # Reverse the subarray from i+1 to the end to get the smallest sequence
    left, right = i + 1, n - 1
    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1
    # Example usage:
    nums = [1, 2, 3]
    nextPermutation(nums)
    print(nums) # Output: [1, 3, 2]
```

#75

MEDIUM

Sort Colors

LeetCode · [SimplyLeet](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

Array · Two Pointers · Sorting

Like · Grind 169

Problem:

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: `nums = [2,0,2,1,1,0]`
Output: `[0,0,1,1,2,2]`

Example 2:

Input: `nums = [2,0,1]`
Output: `[0,1,2]`

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

Two Pointers · [LeetCode](#) · [By Pattern](#)

— 1207 —

LeetCode

>> Brute Force [Two Pointers] Interview Prep

Python

Brute Force Approach

```
class Solution:
    def sortColors(self, nums):
        # Brute Force O(n log n) - just sort; Dutch-flag is the optimal O(n) approach
        nums.sort()
```

Python

Python

Function to sort the colors using Dutch National Flag algorithm

```
def sortColors(nums):
    # Initialize pointers for the start, current, and end positions
    low, mid, high = 0, 0, len(nums) - 1

    # Process elements until mid pointer exceeds high pointer
    while mid <= high:
        # If the current element is 0 (red)
        if nums[mid] == 0:
            # Swap current element with the element at low pointer
            nums[low], nums[mid] = nums[mid], nums[low]
            # Increment both low and mid pointers
            low += 1
            mid += 1
        # If the current element is 1 (white)
        elif nums[mid] == 1:
            # Move mid pointer forward since it's already in the correct place
            mid += 1
        # If the current element is 2 (blue)
        else:
            # Swap current element with the element at high pointer
            nums[mid], nums[high] = nums[high], nums[mid]
            # Decrement high pointer
            high -= 1
```

```
# Example usage:
nums = [2,0,2,1,1,0]
sortColors(nums)
print(nums) # Output: [0,0,1,1,2,2]
```

Python

Two Pointers · [LeetCode](#) · [By Pattern](#)

— 1208 —

LeetCode

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        zero = -1
        one = -1
        two = -1

        for num in nums:
            if num == 0:
                two += 1
            elif num == 1:
                one += 1
            else:
                zero += 1

        # Now we have counts for each color
        # Place them back in the array
        zero_idx = 0
        one_idx = 0
        two_idx = 0

        for num in nums:
            if num == 0:
                nums[zero_idx] = 0
                zero_idx += 1
            elif num == 1:
                nums[one_idx] = 1
                one_idx += 1
            else:
                nums[two_idx] = 2
                two_idx += 1
```

Python

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        l = 0 # The next 0 should be placed in l
        r = len(nums) - 1 # The next 2 should be placed in r

        i = 0
        while i <= r:
            if nums[i] == 0:
                nums[i], nums[l] = nums[l], nums[i]
                i += 1
            elif nums[i] == 1:
                i += 1
            else:
                # We may swap 0 to index i, but we're still not sure whether this 0
                # is placed in the correct index, so we can't move pointer i.
                nums[i], nums[r] = nums[r], nums[i]
                r -= 1
```

Python

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        l, j, k = -1, len(nums), 0
        while k < j:
            if nums[k] == 0:
                nums[k], nums[l] = nums[l], nums[k]
                l += 1
            elif nums[k] == 2:
                j -= 1
                nums[k], nums[j] = nums[j], nums[k]
            else:
                k += 1
```

Two Pointers · [LeetCode](#) · [By Pattern](#)

— 1209 —

LeetCode

Python

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        l, j, k = -1, len(nums), 0
        while k < j:
            if nums[k] == 0:
                l += 1
                nums[k], nums[l] = nums[l], nums[k]
                k += 1
            elif nums[k] == 2:
                j -= 1
                nums[k], nums[j] = nums[j], nums[k]
            else:
                k += 1
```

C++

C++

```
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int zero = -1;
        int one = -1;
        int two = -1;

        for (const int num : nums) {
            if (num == 0) {
                nums[++two] = 0;
                nums[zero] = 0;
                nums[zero] = 0;
            } else if (num == 1) {
                nums[++one] = 1;
                nums[one] = 1;
            } else {
                nums[zero] = 2;
            }
        }
    }
};
```

C++

```
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int l = 0; // The next 0 should be placed in l
        int r = nums.size() - 1; // The next 2 should be placed in r

        for (int i = 0; i <= r; i++) {
            if (nums[i] == 0) {
                swap(nums[i], nums[l]);
                l++;
            } else if (nums[i] == 2) {
                swap(nums[i], nums[r]);
                r--;
            }
        }
    }
};
```

Two Pointers · [LeetCode](#) · [By Pattern](#)

— 1210 —

LeetCode

C++

```
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int l = -1, j = nums.size(), k = 0;
        while (k < j) {
            if (nums[k] == 0) {
                swap(nums[++l], nums[k]);
            } else if (nums[k] == 2) {
                swap(nums[--j], nums[k]);
            } else {
                ++k;
            }
        }
    }
};
```

C++

```
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int l = -1, j = nums.size(), k = 0;
        while (k < j) {
            if (nums[k] == 0) {
                swap(nums[++l], nums[k]);
            } else if (nums[k] == 2) {
                swap(nums[--j], nums[k]);
            } else {
                ++k;
            }
        }
    }
};
```

Java

Java

```
class Solution {
    public void sortColors(int[] nums) {
        int zero = -1;
        int one = -1;
        int two = -1;

        for (final int num : nums) {
            if (num == 0) {
                nums[++two] = 0;
                nums[zero] = 0;
                zero++;
            } else if (num == 1) {
                nums[++one] = 1;
                nums[one] = 1;
            } else {
                nums[zero] = 2;
            }
        }
    }
}
```

Java

Two Pointers · [LeetCode](#) · [By Pattern](#)

— 1211 —

LeetCode

```
class Solution {
    public void sortColors(int[] nums) {
        int l = 0; // The next 0 should be placed in l
        int r = nums.length - 1; // The next 2 should be placed in r

        for (int i = 0; i <= r; i++) {
            if (nums[i] == 0) {
                swap(nums, i, l);
                l++;
            } else if (nums[i] == 2) {
                swap(nums, i, r);
                r--;
            }
        }
    }

    private void swap(int[] nums, int l, int r) {
        final int temp = nums[l];
        nums[l] = nums[r];
        nums[r] = temp;
    }
}
```

Java

```
class Solution {
    public void sortColors(int[] nums) {
        int l = -1, j = nums.length, k = 0;
        while (k < j) {
            if (nums[k] == 0) {
                swap(nums, ++l, k);
            } else if (nums[k] == 2) {
                swap(nums, --j, k);
            } else {
                ++k;
            }
        }
    }

    private void swap(int[] nums, int l, int j) {
        int t = nums[l];
        nums[l] = nums[j];
        nums[j] = t;
    }
}
```

Java

Two Pointers · [LeetCode](#) · [By Pattern](#)

— 1212 —

LeetCode

```
public class Solution {
    public void sortColors(int[] nums) {
        let i = -1, j = nums.length, k = 0;
        while (k < j) {
            if (nums[k] == 0) {
                swap(nums, ++i, k);
            } else if (nums[k] == 2) {
                swap(nums, --j, k);
            } else {
                ++k;
            }
        }
    }

    private void swap(int[] nums, let i, let j) {
        let t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

Java

class Solution {
    public void sortColors(int[] nums) {
        let i = -1, j = nums.length, k = 0;
        while (k < j) {
            if (nums[k] == 0) {
                swap(nums, ++i, k);
            } else if (nums[k] == 2) {
                swap(nums, --j, k);
            } else {
                ++k;
            }
        }
    }

    private void swap(int[] nums, let i, let j) {
        let t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

Java
```

```
public class Solution {
    public void sortColors(int[] nums) {
        let i = -1, j = nums.length, k = 0;
        while (k < j) {
            if (nums[k] == 0) {
                swap(nums, ++i, k);
            } else if (nums[k] == 2) {
                swap(nums, --j, k);
            } else {
                ++k;
            }
        }
    }

    private void swap(int[] nums, let i, let j) {
        let t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

TypeScript

TypeScript

// Do not return anything, modify nums in-place instead.
function sortColors(nums: number[]): void {
    let i = -1;
    let j = nums.length;
    let k = 0;
    while (k < j) {
        if (nums[k] === 0) {
            ++i;
            [nums[i], nums[k]] = [nums[k], nums[i]];
            ++k;
        } else if (nums[k] === 2) {
            --j;
            [nums[j], nums[k]] = [nums[k], nums[j]];
        } else {
            ++k;
        }
    }
}

TypeScript
```

```
// Do not return anything, modify nums in-place instead.
function sortColors(nums: number[]): void {
    let i = -1;
    let j = nums.length;
    let k = 0;
    while (k < j) {
        if (nums[k] === 0) {
            ++i;
            [nums[i], nums[k]] = [nums[k], nums[i]];
            ++k;
        } else if (nums[k] === 2) {
            --j;
            [nums[j], nums[k]] = [nums[k], nums[j]];
        } else {
            ++k;
        }
    }
}

Go

func sortColors(nums []int) {
    i, j, k := -1, len(nums), 0
    for k < j {
        if nums[k] == 0 {
            i++
            nums[i], nums[k] = nums[k], nums[i]
            k++
        } else if nums[k] == 2 {
            j--
            nums[j], nums[k] = nums[k], nums[j]
        } else {
            k++
        }
    }
}

Go

func sortColors(nums []int) {
    i, j, k := -1, len(nums), 0
    for k < j {
        if nums[k] == 0 {
            i++
            nums[i], nums[k] = nums[k], nums[i]
            k++
        } else if nums[k] == 2 {
            j--
            nums[j], nums[k] = nums[k], nums[j]
        } else {
            k++
        }
    }
}

Go
```

```
Rust

impl Solution {
    pub fn sort_colors(nums: &mut Vec<i32>) {
        let mut i = -1;
        let mut j = nums.len();
        let mut k = 0;
        while k < j {
            if nums[k] == 0 {
                i += 1;
                nums.swap(i as usize, k as usize);
                k += 1;
            } else if nums[k] == 2 {
                j -= 1;
                nums.swap(j, k);
            } else {
                k += 1;
            }
        }
    }
}

Rust

impl Solution {
    pub fn sort_colors(nums: &mut Vec<i32>) {
        let mut i = -1;
        let mut j = nums.len();
        let mut k = 0;
        while k < j {
            if nums[k] == 0 {
                i += 1;
                nums.swap(i as usize, k as usize);
                k += 1;
            } else if nums[k] == 2 {
                j -= 1;
                nums.swap(j, k);
            } else {
                k += 1;
            }
        }
    }
}

[*] Two Pointers — Pattern Solution (stored optimal)

Python
```

```
# Function to sort the colors using Dutch National Flag algorithm
def sortColors(nums):
    # Initialize pointers for the start, current, and end pointers
    low, mid, high = 0, 0, len(nums) - 1

    # Process elements until mid pointer exceeds high pointer
    while mid <= high:
        # If the current element is 0 (red)
        if nums[mid] == 0:
            # Swap current element with the element at low pointer
            nums[low], nums[mid] = nums[mid], nums[low]
            # Increment both low and mid pointers
            low += 1
            mid += 1
        # If the current element is 1 (white)
        elif nums[mid] == 1:
            # Swap mid pointer forward since it's already in the correct place
            mid += 1
        # If the current element is 2 (blue)
        else:
            # Swap current element with the element at high pointer
            nums[mid], nums[high] = nums[high], nums[mid]
            # Decrement high pointer
            high -= 1

# Example usage
nums = [2,0,2,1,0]
sortColors(nums)
print(nums) # Output: [0,0,1,1,2,2]

C++

// Function to sort the colors using Dutch National Flag algorithm
#include <vector>
#include <iostream>
using namespace std;

void sortColors(vector<int>& nums) {
    // Initialize pointers for low, mid, and high
    int low = 0, mid = 0;
    int high = nums.size() - 1;

    // Iterate until mid pointer exceeds high pointer
    while (mid <= high) {
        if (nums[mid] == 0) {
            // Swap element at mid with element at low pointer
            swap(nums[low], nums[mid]);
            low++;
            mid++;
        } else if (nums[mid] == 1) {
            // Swap mid pointer forward if element is 1
            mid++;
        } else { // nums[mid] == 2
            // Swap element at mid with element at high pointer
            swap(nums[mid], nums[high]);
            high--;
        }
    }
}
```

```
}
}

// Example usage
int main() {
    vector<int> nums = {2,0,2,1,1,0};
    sortColors(nums);
    // Output the sorted array
    for (int num : nums)
        cout << num << " ";
    // Expected Output: 0 0 1 1 2 2
    return 0;
}

#161 MEDIUM
One Edit Distance
LeetCode's Simplest · Kotlin · LeetCode · Editorial
Two Pointers · String
List: Premium 98

Problem:
Given two strings s and t, return true if they are both one edit distance apart, otherwise return false.
A string s is said to be one distance apart from a string t if you can:
    • Insert exactly one character into s to get t.
    • Delete exactly one character from s to get t.
    • Replace exactly one character of s with a different character to get t.

Example 1:
Input: s = "ab", t = "acb"
Output: true
Explanation: We can insert 'c' into s to get t.

Example 2:
Input: s = "", t = ""
Output: false
Explanation: We cannot get t from s by only one step.

Constraints:
    • 0 <= s.length, t.length <= 104
    • s and t consist of lowercase letters, uppercase letters, and digits.

Python
Python
```

```
# Function to check if two strings are one edit distance apart
def isOneEditDistance(s1: str, t1: str) -> bool:
    # Get lengths of both strings
    m, n = len(s1), len(t1)

    # The difference in length must not be greater than 1
    if abs(m - n) > 1:
        return False

    # Ensure s is the shorter string, or they are equal in length
    if m > n:
        return isOneEditDistance(t, s)

    # Initialize variables: found difference flag and indices for both strings
    found_difference = False
    i = j = 0

    while i < m and j < n:
        if s[i] != t[j]:
            # If a difference has already been found, return False
            if found_difference:
                return False
            found_difference = True
            # If lengths are equal, move both pointers
            if m == n:
                i += 1
            # If characters are the same, move the pointer in the shorter string
            i += 1
            # Always move pointer for longer string
            j += 1

    # If no difference was found during iteration, then one extra character at the end is the edit
    return found_difference or (j == n - 1)

# Example usage:
print(isOneEditDistance("abc", "abd")) # Output: True
```

Python

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        m = len(s)
        n = len(t)
        if m == n:
            return self.isOneEditDistance(s, t)
        for i in range(m):
            if s[i] != t[i]:
                if m == n:
                    return s[i + 1] == t[i + 1] # Replace s[i] with t[i].
                return s[i + 1] == t[i + 1] # Delete t[i].
        return m + 1 == n # Delete t[-1].
```

Python

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        if len(s) < len(t):
            return self.isOneEditDistance(t, s)
        m = len(s), len(t)
        if m == n:
            # case: abc, abcd, return False
            return False
        for i in range(m):
            if s[i] != t[i]:
                return s[i + 1] == t[i + 1] if m == n else s[i + 1] == t[i]
        # case: abc, abcd, return True
        # case: abc, abcd, the same 2 strings should return False
        # case: abc, abcd, return False
        return m == n + 1

#####
class Solution(object):
    def isOneEditDistance(self, s, t):
        :type s: str
        :type t: str
        :rtype: bool

        if len(s) == len(t):
            i = 0
            while i < len(s) and s[i] == t[i]:
                i += 1
            return s[i + 1] == t[i + 1] if len(s) == len(t) else s[i + 1] == t[i]
        else:
            return self.isOneEditDistance(s, t)
```

C++

C++

```
// 01: https://leetcode.com/problems/one-edit-distance/
// Time: O(NM)
// Space: O(1) (O(N))
class Solution {
private:
    int editDistance(string &s, string &t) {
        if (&s == &t) return 0;
        int m = s.length(), n = t.length();
        vector<int> dp(m + 1, 0);
        for (int i = 1; i <= m; ++i) dp[i] = i;
        for (int i = 1; i <= n; ++i) {
            int pre = dp[i];
            dp[i] = 1;
            for (int j = 1; j <= m; ++j) {
                if (s[i - 1] == t[j - 1]) dp[j] = pre;
                else dp[j] = min(pre, min(dp[i - 1], dp[j])) + 1;
                pre = dp[j];
            }
            return dp[n];
        }
    }
public:
    bool isOneEditDistance(string s, string t) {
        return editDistance(s, t) == 1;
    }
};
```

Java

```
class Solution {
    public boolean isOneEditDistance(String s, String t) {
        final int m = s.length();
        final int n = t.length();
        if (m > n) // Make sure that s is longer
            return isOneEditDistance(t, s);
        for (int i = 0; i < m; ++i) {
            if (s.charAt(i) != t.charAt(i)) {
                if (m == n)
                    return s.substring(i + 1).equals(t.substring(i + 1));
                return s.substring(i).equals(t.substring(i + 1)); // Delete t[i].
            }
        }
        return m + 1 == n; // Delete t[-1].
    }
}
```

Java

```
class Solution {
    public boolean isOneEditDistance(String s, String t) {
        int m = s.length(), n = t.length();
        if (m == n) {
            return isOneEditDistance(s, t);
        }
        if (m - n > 1) {
            return false;
        }
        for (int i = 0; i < m; ++i) {
            if (s.charAt(i) != t.charAt(i)) {
                if (m == n) {
                    return s.substring(i + 1).equals(t.substring(i + 1));
                }
                return s.substring(i + 1).equals(t.substring(i));
            }
        }
        return m == n + 1;
    }
}
```

Java

```
import java.util.Objects;

public class One_Edit_Distance {

    // best solution has
    public boolean isOneEditDistance(String s, String t) {
        for (int i = 0; i < Math.min(s.length(), t.length()); ++i) {
            if (s.charAt(i) != t.charAt(i)) {
                if (s.length() == t.length()) {
                    return Objects.equals(s.substring(i + 1), t.substring(i + 1));
                }
                if (s.length() < t.length()) {
                    return Objects.equals(s.substring(i), t.substring(i + 1));
                }
                return Objects.equals(s.substring(i + 1), t.substring(i));
            }
        }
        // case: abc, abcd,
        return Math.abs(s.length() - t.length()) == 1;
    }

    class Solution_true_dp {
```

```
public boolean isOneEditDistance(String s, String t) {
    int m = s.length(), n = t.length(), i, j;
    int diff = m - n;
    for (i = 0; i <= m; ++i) {
        dp[i][0] = i;
        for (j = 0; j <= n; ++j) {
            dp[i][j] = j;
            for (k = 0; k <= m; ++k) {
                for (l = 0; l <= n; ++l) {
                    if (s.charAt(k) == t.charAt(l)) {
                        dp[k + 1][l + 1] = dp[k][l];
                    }
                    else {
                        dp[k + 1][l + 1] = Math.min(dp[k + 1][l], Math.min(dp[k][l + 1], dp[k][l])) + 1;
                    }
                }
            }
            return dp[m][n] == 1;
        }
    }
}
```

Java

```
public class Solution_dp {
    public boolean isOneEditDistance(String s, String t) {
        if (s.length() < t.length()) return isOneEditDistance(t, s); // so 't' is always the shorter
        one in below logic
        int m = s.length(), n = t.length(), diff = m - n;
        if (diff == 2) return false;

        else if (diff == 1) {
            for (int i = 0; i < m; ++i) {
                if (s.charAt(i) != t.charAt(i)) {
                    return s.substring(i + 1).equals(t.substring(i));
                }
            }
            return true; // missing, diff is happening at the final char
        }
        else {
            int cnt = 0;
            for (int i = 0; i < m; ++i) {
                if (s.charAt(i) != t.charAt(i)) ++cnt;
            }
            return cnt == 1;
        }
    }
}
```

Java

```
class Solution {
    public boolean isOneEditDistance(String s, String t) {
        int m = s.length(), n = t.length();
        if (m == n) {
            return isOneEditDistance(s, t);
        }
        if (m - n > 1) {
            return false;
        }
        for (int i = 0; i < m; ++i) {
```

```
function isOneEditDistance(s: string, t: string): boolean {
    const [m, n] = [s.length, t.length];
    if (m == n) {
        return isOneEditDistance(s, t);
    }
    if (m - n > 1) {
        return false;
    }
    for (let i = 0; i < m; ++i) {
        if (s[i] != t[i]) {
            return s.slice(i + 1) === t.slice(i + 1) || s.slice(i) === t.slice(i + 1);
        }
    }
    return m === n + 1;
}
```

JavaScript

```
function isOneEditDistance(s: string, t: string): boolean {
    const [m, n] = [s.length, t.length];
    if (m == n) {
        return isOneEditDistance(s, t);
    }
    if (m - n > 1) {
        return false;
    }
    for (let i = 0; i < m; ++i) {
        if (s[i] != t[i]) {
            return s.slice(i + 1) === t.slice(i + 1) || s.slice(i) === t.slice(i + 1);
        }
    }
    return m === n + 1;
}
```

JavaScript


```
# Function to check if two strings are one edit distance apart
def isOneEditDistance(s1: str, t1: str) -> bool:
    # Get lengths of both strings
    m, n = len(s1), len(t1)

    # The difference in length must not be greater than 1
    if abs(m - n) > 1:
        return False

    # Ensure s is the shorter string, or they are equal in length
    if m > n:
        return isOneEditDistance(t1, s1)

    # Initialize variables: found difference flag and indices for both strings
    found_difference = False
    i = j = 0

    while i < m and j < n:
        if s[i] != t[j]:
            # If a difference has already been found, return False
            if found_difference:
                return False
            found_difference = True
            # If lengths are equal, move both pointers
            if m == n:
                i += 1
            else:
                # If characters are the same, move the pointer in the shorter string
                i += 1
            # Always move pointer for longer string
            j += 1

    # If no difference was found during iteration, then one extra character at the end is the edit
    return found_difference or (j == i == 1)

# Example usage:
print(isOneEditDistance("ab", "acb")) # Output: True
```

C++

```
// Function to check if two strings are one edit distance apart
#include <string>
#include <vector>
using namespace std;

class Solution {
public:
    bool isOneEditDistance(string s, string t) {
        int m = s.size(), n = t.size();

        // Difference in length should not exceed 1
        if (abs(m - n) > 1) return false;

        // Ensure s is the shorter string or equal in length
        if (m > n) return isOneEditDistance(t, s);

        bool foundDifference = false;
        int i = 0, j = 0;

        while (i < m && j < n) {
            if (s[i] != t[j]) {
                // If difference has been found already, return false
                if (foundDifference) return false;
                foundDifference = true;
                // If same length, increment i
                if (m == n) {
                    i++;
                }
            } else {
                // If characters match, always move i pointer
                i++;
            }
            // Move pointer j for the longer string
            j++;
        }

        // If no difference was found, check if extra missing character at the end qualifies
        return foundDifference || (j == i == 1);
    }
};

// Example usage:
// int main() {
//     Solution sol;
//     bool result = sol.isOneEditDistance("ab", "acb"); // Should return true
//     return 0;
// }
```

#167

MEDIUM

Two Sum II - Input Array Is Sorted

LeetCode · SimplifyLeetCode · WalkCC · LeetCode · Editorial

Array · Two Pointers · Binary Search

Lists Denny Zhang

Problem:

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where 1 <= index1 < index2 <= numbers.length.

Return the indices of the two numbers index1 and index2, each incremented by one, as an integer array [index1, index2] of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].

Example 2:

Input: numbers = [2,3,4], target = 6

Output: [1,3]

Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].

Example 3:

Input: numbers = [-1,0], target = -1

Output: [1,2]

Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].

Constraints:

- 2 <= numbers.length <= 3 * 10⁴
- 1000 <= numbers[i] <= 1000
- numbers is sorted in non-decreasing order.
- 1000 <= target <= 1000
- The tests are generated such that there is exactly one solution.

>> Brute Force [Two Pointers] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        # Brute Force O(n^2) - check every pair (ignore sorted property)
        for i in range(len(numbers)):
            for j in range(i + 1, len(numbers)):
                if numbers[i] + numbers[j] == target:
                    return [i + 1, j + 1]
```

Python

Python

```
# Python solution for Two Sum II - Input Array Is Sorted
def twoSum(numbers, target):
    # Initialize two pointers at the beginning and end of the array
    left = 0
    right = len(numbers) - 1

    # Continue until the two pointers meet
    while left < right:
        # Calculate the sum of the two pointer values
        current_sum = numbers[left] + numbers[right]
        # If the sum matches the target, return the 1-indexed results
        if current_sum == target:
            return [left + 1, right + 1]
        # If the current sum is less than the target, move the left pointer to the right
        elif current_sum < target:
            left += 1
        # Otherwise, move the right pointer to the left since the current sum is too high
        else:
            right -= 1

# Example call
print(twoSum([2, 7, 11, 15], 9)) # Output: [1, 2]
```

Python

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        l = 0
        r = len(numbers) - 1

        while l < r:
            sum = numbers[l] + numbers[r]
            if sum == target:
                return [l + 1, r + 1]
            elif sum < target:
                l += 1
            else:
                r -= 1
```

Python

```
class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        n = len(numbers)
        for i in range(n - 1):
            x = target - numbers[i]
            j = bisect_left(numbers, x, i + 1)
            if j < n and numbers[j] == x:
                return [i + 1, j + 1]
```

Python

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        int l = 0, r = numbers.size() - 1;

        while (l < r) {
            int sum = numbers[l] + numbers[r];
            if (sum == target) {
                return {l + 1, r + 1};
            } else if (sum < target) {
                l++;
            } else {
                r--;
            }
        }

        return {};
    }
};
```

C++

C++

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        for (int l = 0, r = numbers.size() - 1; l < r; ++l) {
            int s = target - numbers[l];
            int j = lower_bound(numbers.begin() + l + 1, numbers.end(), s) - numbers.begin();
            if (j < numbers.size() && numbers[j] == s) {
                return {l + 1, j + 1};
            }
        }
    }
};
```

C++

```
class Solution {
public:
    vector<int> twoSum(vector<int>& numbers, int target) {
        for (int l = 0, r = numbers.size() - 1; l < r; ++l) {
            int s = target - numbers[l];
            if (s == target) {
                return {l + 1, r + 1};
            } else if (s < target) {
                l++;
            } else {
                r--;
            }
        }

        return {};
    }
};
```

Java

Java

```
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        for (int l = 0, r = numbers.length - 1; l < r; ++l) {
            int s = target - numbers[l];
            while (l < r) {
                int mid = (l + r) / 2;
                if (numbers[mid] == s) {
                    r = mid;
                } else {
                    l = mid + 1;
                }
            }
            if (numbers[r] == s) {
                return new int[] {l + 1, r + 1};
            }
        }
    }
}
```

Java

Java

```
class Solution {
public:
    int twoSum(int[] numbers, int target) {
        for (int i = 0, j = numbers.length - 1; ) {
            int s = numbers[i] + numbers[j];
            if (s == target) {
                return new int[] {i + 1, j + 1};
            }
            if (s < target) {
                ++i;
            } else {
                --j;
            }
        }
    }
}

JavaScript

//
* @param {number[]} numbers
* @param {number} target
* @return {number[]}
*/
var twoSum = function (numbers, target) {
    const n = numbers.length;
    for (let i = 0; ; ++i) {
        const s = target - numbers[i];
        let l = i + 1;
        let r = n - 1;
        while (l < r) {
            const mid = (l + r) >> 1;
            if (numbers[mid] == s) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        if (numbers[i] == s) {
            return [i + 1, l + 1];
        }
    }
};

JavaScript
```

```
//
* @param {number[]} numbers
* @param {number} target
* @return {number[]}
*/
var twoSum = function (numbers, target) {
    for (let i = 0, j = numbers.length - 1; ; ) {
        const s = numbers[i] + numbers[j];
        if (s == target) {
            return [i + 1, j + 1];
        }
        if (s < target) {
            ++i;
        } else {
            --j;
        }
    }
};

TypeScript

function twoSum(numbers: number[], target: number): number[] {
    const n = numbers.length;
    for (let i = 0; ; ++i) {
        const s = target - numbers[i];
        let l = i + 1;
        let r = n - 1;
        while (l < r) {
            const mid = (l + r) >> 1;
            if (numbers[mid] == s) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        if (numbers[i] == s) {
            return [i + 1, l + 1];
        }
    }
};

TypeScript

function twoSum(numbers: number[], target: number): number[] {
    for (let i = 0, j = numbers.length - 1; ; ) {
        const s = numbers[i] + numbers[j];
        if (s == target) {
            return [i + 1, j + 1];
        }
        if (s < target) {
            ++i;
        } else {
            --j;
        }
    }
};

TypeScript
```

```
function twoSum(numbers: number[], target: number): number[] {
    for (let i = 0, j = numbers.length - 1; ; ) {
        const s = numbers[i] + numbers[j];
        if (s == target) {
            return [i + 1, j + 1];
        }
        if (s < target) {
            ++i;
        } else {
            --j;
        }
    }
};

Go

func twoSum(numbers []int, target int) []int {
    for i, n := 0, len(numbers)-1; ; i++ {
        s := target - numbers[i]
        j := sort.Search(len(numbers[i:]), s) + i + 1
        if j <= n && numbers[j] == s {
            return []int{i + 1, j + 1}
        }
    }
};

Go

func twoSum(numbers []int, target int) []int {
    for i, j := 0, len(numbers)-1; ; i++ {
        s := numbers[i] + numbers[j]
        if s == target {
            return []int{i + 1, j + 1}
        }
        if s < target {
            ++i
        } else {
            --j
        }
    }
};

Go

func twoSum(numbers []int, target int) []int {
    for i, j := 0, len(numbers)-1; ; i++ {
        s := numbers[i] + numbers[j]
        if s == target {
            return []int{i + 1, j + 1}
        }
        if s < target {
            ++i
        } else {
            --j
        }
    }
};

Two Pointers · LeetCode — By Patterns
```

```
Rust

use std::cmp::Ordering;

impl Solution {
    pub fn two_sum(numbers: Vec<i32>, target: i32) -> Vec<i32> {
        let n = numbers.len();
        let mut l = 0;
        let mut r = n - 1;
        loop {
            match (numbers[l] + numbers[r]).cmp(&target) {
                Ordering::Less => {
                    l += 1;
                }
                Ordering::Greater => {
                    r -= 1;
                }
                Ordering::Equal => {
                    break;
                }
            }
        }
        vec![l as i32 + 1, r as i32 + 1]
    }
};

Rust

use std::cmp::Ordering;

impl Solution {
    pub fn two_sum(numbers: Vec<i32>, target: i32) -> Vec<i32> {
        let n = numbers.len();
        let mut l = 0;
        let mut r = n - 1;
        loop {
            match (numbers[l] + numbers[r]).cmp(&target) {
                Ordering::Less => {
                    l += 1;
                }
                Ordering::Greater => {
                    r -= 1;
                }
                Ordering::Equal => {
                    break;
                }
            }
        }
        vec![l as i32 + 1, r as i32 + 1]
    }
};

[*] Two Pointers — Pattern Solution (matched from community answers)

Python
```

```
# Python solution for Two Sum II - Input Array Is Sorted
def twoSum(numbers, target):
    # Initialize two pointers at the beginning and end of the array
    left = 0
    right = len(numbers) - 1

    # Continue until the two pointers meet
    while left < right:
        # Calculate the sum of the two pointer values
        current_sum = numbers[left] + numbers[right]
        # If the sum matches the target, return the 1-indexed results
        if current_sum == target:
            return [left + 1, right + 1]
        # If the current sum is less than the target, move the left pointer to the right
        elif current_sum < target:
            left += 1
        # Otherwise, move the right pointer to the left since the current sum is too high
        else:
            right -= 1

    # Example call
    print(twoSum([2, 7, 11, 15], 9)) # Output: [1, 2]
```

#186 MEDIUM

Reverse Words in a String II

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Two Pointers · String

Like Premium 50

Problem:

Given a character array `s`, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in `s` will be separated by a single space.

Your code must solve the problem in-place, i.e. without allocating extra space.

Example 1:

Input: `s = ["t","h","e"," ","b","o","o","k"," ","o","f"," ","t","h","e"," ","a","m","e","i","c","a","n"," ","r","e","v","o","l","u","t","i","o","n","a","r","y"," ","w","a","r"]`

Output: `["b","o","o","k"," ","o","f"," ","t","h","e"," ","a","m","e","i","c","a","n"," ","r","e","v","o","l","u","t","i","o","n","a","r","y"," ","w","a","r"]`

Example 2:

Input: `s = ["a"]`

Output: `["a"]`

Constraints:

- `1 <= s.length <= 105`
- `s[i]` is an English letter (uppercase or lowercase), digit, or space `' '`.
- There is at least one word in `s`.
- `s` does not contain leading or trailing spaces.
- All the words in `s` are guaranteed to be separated by a single space.

```
>> Brute Force [Two Pointers] Interview Prep

Python

# Brute Force Approach
def reverse_range(left, right):
    # Brute Force O(n^2) - nested loops instead of two pointers
    n = len(s)
    for i in range(left, right):
        for j in range(i + 1, n):
            s[i], s[j] = s[j], s[i]

    pass

Python

def reverse_range(left, right):
    # Reverse the elements in s from index left to right.
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

def reverseWords(s):
    # Step 1: Reverse the entire array.
    reverse_range(0, len(s) - 1)

    n = len(s)
    start = 0

    # Step 2: Reverse each word to correct their order.
    for i in range(1, n):
        if i == n or s[i] == " ":
            reverse_range(start, i - 1)
            start = i + 1

Python

class Solution:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(i: int, j: int) -> None:
            while i < j:
                s[i], s[j] = s[j], s[i]
                i += 1
                j -= 1

        def reverseWords(i: int) -> None:
            -> None
            j = 0
            while i < n:
                while i < j or (i < n and s[i] == " "): # Skip the spaces.
                    i += 1
                while j < i or (j < n and s[j] != " "): # Skip the spaces.
                    j += 1
                reverse(i, j - 1) # Reverse the word.

        reverse(0, len(s) - 1) # Reverse the whole string.
        reverseWords(len(s)) # Reverse each word.
```

```

Python
class Solution:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(i: int, j: int):
            while i < j:
                s[i], s[j] = s[j], s[i]
                i, j = i + 1, j - 1

            i, n = 0, len(s)
            for j, c in enumerate(s):
                if c == " ":
                    reverse(i, j - 1)
                    i = j + 1
                elif j == n - 1:
                    reverse(i, j)
            reverse(0, n - 1)

C++
C++

```

```

class Solution {
public:
    void reverseWords(vector<char*> s) {
        ranges::reverse(s); // Reverse the whole string.
        reverseWords(s, s.size()); // Reverse each word.
    }

private:
    void reverseWords(vector<char*> s, int n) {
        int i = 0;
        int j = 0;

        while (i < n) {
            while (i < j || i < n && s[i] == ' ') // Skip the spaces.
                i++;
            while (j < i || j < n && s[j] != ' ') // Skip the spaces.
                j++;
            reverse(s.begin() + i, s.begin() + j); // Reverse the word.
            i = j;
        }
    }
};

C++
class Solution {
public:
    void reverseWords(vector<char*> s) {
        auto reverse = [&](int i, int j) {
            for (; i < j; ++i, --j) {
                swap(s[i], s[j]);
            }
        };
        int n = s.size();
        for (int i = 0, j = 0; j < n; ++j) {
            if (s[j] == ' ') {
                reverse(i, j - 1);
                i = j + 1;
            } else if (j == n - 1) {
                reverse(i, j);
            }
        }
        reverse(0, n - 1);
    }
};

C++

```

```

class Solution {
public:
    void reverseWords(vector<char*> s) {
        int n = s.size();
        for (int i = 0, j = 0; j < n; ++j) {
            if (s[j] == ' ') {
                reverse(i, j - 1);
                i = j + 1;
            } else if (j == n - 1) {
                reverse(i, j);
            }
        }
        reverse(0, n - 1);
    }

    void reverse(vector<char*> s, int i, int j) {
        for (; i < j; ++i, --j) {
            swap(s[i], s[j]);
        }
    }
};

Java
Java
class Solution {
    public void reverseWords(char[] s) {
        reverse(s, 0, s.length - 1); // Reverse the whole string.
        reverseWords(s, s.length); // Reverse each word.
    }

    private void reverse(char[] s, int l, int r) {
        while (l < r) {
            final char c = s[l];
            s[l] = s[r];
            s[r] = c;
        }
    }

    private void reverseWords(char[] s, int n) {
        int i = 0;
        int j = 0;

        while (i < n) {
            while (i < j || i < n && s[i] == ' ') // Skip the spaces.
                i++;
            while (j < i || j < n && s[j] != ' ') // Skip the spaces.
                j++;
            reverse(s, i, j - 1); // Reverse the word.
            i = j;
        }
    }
};

Java

```

```

class Solution {
    public void reverseWords(char[] s) {
        int n = s.length;
        for (int i = 0, j = 0; j < n; ++j) {
            if (s[j] == ' ') {
                reverse(s, i, j - 1);
                i = j + 1;
            } else if (j == n - 1) {
                reverse(s, i, j);
            }
        }
        reverse(s, 0, n - 1);
    }

    private void reverse(char[] s, int i, int j) {
        for (; i < j; ++i, --j) {
            char t = s[i];
            s[i] = s[j];
            s[j] = t;
        }
    }
};

Java
class Solution {
    public void reverseWords(char[] s) {
        int n = s.length;
        for (int i = 0, j = 0; j < n; ++j) {
            if (s[j] == ' ') {
                reverse(s, i, j - 1);
                i = j + 1;
            } else if (j == n - 1) {
                reverse(s, i, j);
            }
        }
        reverse(s, 0, n - 1);
    }

    private void reverse(char[] s, int i, int j) {
        for (; i < j; ++i, --j) {
            char t = s[i];
            s[i] = s[j];
            s[j] = t;
        }
    }
};

TypeScript
TypeScript

```

```

// Do not return anything, modify s in-place instead.
//
function reverseWords(s: string[]): void {
    const n = s.length;
    const reverse = (l: number, r: number): void => {
        for (; l < r; ++l, --r) {
            [s[l], s[r]] = [s[r], s[l]];
        }
    };
    for (let i = 0, j = 0; j < n; ++j) {
        if (s[j] === ' ') {
            reverse(i, j - 1);
            i = j + 1;
        } else if (j === n - 1) {
            reverse(i, j);
        }
    }
    reverse(0, n - 1);
}

TypeScript
// Do not return anything, modify s in-place instead.
//
function reverseWords(s: string[]): void {
    const n = s.length;
    const reverse = (l: number, r: number): void => {
        for (; l < r; ++l, --r) {
            [s[l], s[r]] = [s[r], s[l]];
        }
    };
    for (let i = 0, j = 0; j < n; ++j) {
        if (s[j] === ' ') {
            reverse(i, j - 1);
            i = j + 1;
        } else if (j === n - 1) {
            reverse(i, j);
        }
    }
    reverse(0, n - 1);
}

[1] Two Pointers — Pattern Solution (matched from community answers)
Python

```

```

def reverse_ranges(left, right):
    # Reverse the elements in s from index left to right.
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

def reverseWords(s):
    # Step 1: Reverse the entire array.
    reverse_ranges(0, len(s) - 1)

    n = len(s)
    start = 0
    # Step 2: Reverse each word to correct their order.
    for i in range(n - 1):
        if i == n or s[i] == ' ':
            reverse_ranges(start, i - 1)
            start = i + 1

#189 MEDIUM
Rotate Array
LeetCode · Simplest · WUJICE · LeetCode · Editorial
Array · Math · Two Pointers
Lists · Grind 189
Problem:
Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.
Example 1:
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
Example 2:
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
Constraints:
• 1 <= nums.length <= 105
• -231 <= nums[i] <= 231 - 1
• 0 <= k <= 105
Follow up:
• Try to come up with as many solutions as you can. There are at least three different ways to solve this problem.

```

• Could you do it in-place with O(1) extra space?

>> Brute Force [Two Pointers] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def rotate(self, nums, k):
        # Brute Force O(n^2) - nested loops instead of two pointers
        n = len(nums)
        for i in range(n):
            for j in range(i + 1, n):
                # Swap nums[i] and nums[j]
                pass
```

Python

```
Python
# Python solution using the reverse method

def rotate(nums, k):
    # Calculate the effective rotations needed
    n = len(nums)
    k = k % n # In case k is greater than the length of the array

    # Helper function to reverse a portion of the list in place
    def reverse(left, right):
        while left < right:
            # Swap the elements at indices left and right
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1

    # Reverse the entire array
    reverse(0, n - 1)
    # Reverse the first k elements to restore their order
    reverse(0, k - 1)
    # Reverse the remaining n - k elements
    reverse(k, n - 1)

# Example usage:
# nums = [1,2,3,4,5,6,7]
# rotate(nums, 3)

# print(nums) # Output: [4,5,6,7,1,2,3]
```

Python

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        n = len(nums)
        self.reverse(nums, 0, len(nums) - 1)
        self.reverse(nums, 0, k - 1)
        self.reverse(nums, k, len(nums) - 1)

    def reverse(self, nums: List[int], l: int, r: int) -> None:
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l += 1
            r -= 1
```

Python

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        def reverse(i: int, j: int):
            while i < j:
                nums[i], nums[j] = nums[j], nums[i]
                i, j = i + 1, j - 1

        n = len(nums)
        reverse(0, n - 1)
        reverse(0, k - 1)
        reverse(k, n - 1)
```

Python

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        n = len(nums)
        nums[:] = nums[-k:] + nums[:-k]

#####

class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        n = len(nums)
        k %= n
        if n == 2 or k == 0:
            return
        nums[:] = nums[::-1]
        nums[k:] = nums[k:][::-1]
        nums[:k] = nums[:k][::-1]

#####

class Solution(object):
    def rotate(self, nums, k):
        """
```

```
/*Type: nums: List<int>
Type: k: int
Return: void Do not return anything, modify nums in-place instead.
*/
if (len(nums) == 0 || k == 0)
    return

def reverse(start, end, s):
    while start < end:
        s[start], s[end] = s[end], s[start]
        start += 1
        end -= 1

n = len(nums) - 1
k = k % len(nums)
reverse(0, n - k, nums)
reverse(k, n - 1, nums)
reverse(0, k, nums)
```

C++

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k %= nums.size();
        reverse(nums, 0, nums.size() - 1);
        reverse(nums, 0, k);
        reverse(nums, k, nums.size() - 1);
    }

private:
    void reverse(vector<int>& nums, int l, int r) {
        while (l < r)
            swap(nums[l++], nums[r--]);
    }
};
```

C++

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        int n = nums.size();
        k %= n;
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin() + k);
        reverse(nums.begin() + k, nums.end());
    }
};
```

C++

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        int n = nums.size();
        k %= n;
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin() + k);
        reverse(nums.begin() + k, nums.end());
    }
};
```

Java

```
class Solution {
    public void rotate(int[] nums, int k) {
        k %= nums.length;
        reverse(nums, 0, nums.length - 1);
        reverse(nums, 0, k);
        reverse(nums, k, nums.length - 1);
    }

    private void reverse(int[] nums, int l, int r) {
        while (l < r)
            swap(nums, l++, r--);
    }

    private void swap(int[] nums, int l, int r) {
        int temp = nums[l];
        nums[l] = nums[r];
        nums[r] = temp;
    }
}
```

Java

```
class Solution {
    private int[] nums;

    public void rotate(int[] nums, int k) {
        this.nums = nums;
        int n = nums.length;
        k %= n;
        reverse(0, n - 1);
        reverse(0, k);
        reverse(k, n - 1);
    }

    private void reverse(int l, int j) {
        for (; l < j; ++l, --j) {
            int t = nums[l];
            nums[l] = nums[j];
            nums[j] = t;
        }
    }
}
```

Java

Java

Java

```
public class Solution {
    private int[] nums;

    public void rotate(int[] nums, int k) {
        this.nums = nums;
        int n = nums.length;
        k %= n;
        reverse(0, n - 1);
        reverse(0, k);
        reverse(k, n - 1);
    }

    private void reverse(int l, int j) {
        for (; l < j; ++l, --j) {
            int t = nums[l];
            nums[l] = nums[j];
            nums[j] = t;
        }
    }
}
```

Java

```
class Solution {
    private int[] nums;

    public void rotate(int[] nums, int k) {
        this.nums = nums;
        int n = nums.length;
        k %= n;
        reverse(0, n - 1);
        reverse(0, k);
        reverse(k, n - 1);
    }

    private void reverse(int l, int j) {
        for (; l < j; ++l, --j) {
            int t = nums[l];
            nums[l] = nums[j];
            nums[j] = t;
        }
    }
}
```

Java

```
public class Solution {
    private int[] nums;

    public void rotate(int[] nums, int k) {
        this.nums = nums;
        int n = nums.length;
        k %= n;
        reverse(0, n - 1);
        reverse(0, k);
        reverse(k, n - 1);
    }

    private void reverse(int l, int j) {
        for (; l < j; ++l, --j) {
            int t = nums[l];
            nums[l] = nums[j];
            nums[j] = t;
        }
    }
}
```

JavaScript

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {void} Do not return anything, modify nums in-place instead.
 */
var rotate = function(nums, k) {
    const n = nums.length;
    k %= n;
    const reverse = (l, j) => {
        for (; l < j; ++l, --j) {
            [nums[l], nums[j]] = [nums[j], nums[l]];
        }
    };
    reverse(0, n - 1);
    reverse(0, k);
    reverse(k, n - 1);
};
```

JavaScript

```

//
* Given number[] nums
* Return number k
* Return void if not return anything, modify nums in-place instead.
//
var rotate = function (nums, k) {
    const n = nums.length;
    k = k % n;
    const reverse = (l, r) => {
        for (l <= r; l++, r--) {
            [nums[l], nums[r]] = [nums[r], nums[l]];
        }
    };
    reverse(0, n - 1);
    reverse(0, k - 1);
    reverse(k, n - 1);
};

```

TypeScript

TypeScript

```

//
// Do not return anything, modify nums in-place instead.
//
function rotate(nums: number[], k: number): void {
    const n: number = nums.length;
    k = k % n;
    const reverse = (l: number, r: number): void => {
        for (l <= r; l++, r--) {
            const t: number = nums[l];
            nums[l] = nums[r];
            nums[r] = t;
        }
    };
    reverse(0, n - 1);
    reverse(0, k - 1);
    reverse(k, n - 1);
}

```

TypeScript

```

//
// Do not return anything, modify nums in-place instead.
//
function rotate(nums: number[], k: number): void {
    const n: number = nums.length;
    k = k % n;
    const reverse = (l: number, r: number): void => {
        for (l <= r; l++, r--) {
            const t: number = nums[l];
            nums[l] = nums[r];
            nums[r] = t;
        }
    };
    reverse(0, n - 1);
    reverse(0, k - 1);
    reverse(k, n - 1);
}

```

Go

Go

```

func rotate(nums []int, k int) {
    n := len(nums)
    k = k % n
    reverse := func(i, j int) {
        for i <= j; i, j = i+1, j-1 {
            nums[i], nums[j] = nums[j], nums[i]
        }
    }
    reverse(0, n-1)
    reverse(0, k-1)
    reverse(k, n-1)
}

```

Go

```

func rotate(nums []int, k int) {
    n := len(nums)
    k = k % n
    reverse := func(i, j int) {
        for i <= j; i, j = i+1, j-1 {
            nums[i], nums[j] = nums[j], nums[i]
        }
    }
    reverse(0, n-1)
    reverse(0, k-1)
    reverse(k, n-1)
}

```

Rust

Rust

```

impl Solution {
    pub fn rotate(nums: &mut Vec<i32>, k: i32) {
        let n = nums.len();
        let k = (k as usize) % n;
        nums.reverse();
        nums[0..k].reverse();
        nums[k..].reverse();
    }
}

```

Rust

```

impl Solution {
    pub fn rotate(nums: &mut Vec<i32>, k: i32) {
        let n = nums.len();
        let k = (k as usize) % n;
        nums.reverse();
        nums[0..k].reverse();
        nums[k..].reverse();
    }
}

```

[*] Two Pointers — Pattern Solution (matched from community answers)

Python

```

# Python solution using the reverse method
def rotate(nums, k):
    # Calculate the effective rotations needed
    n = len(nums)
    k = k % n # in case k is greater than the length of the array

    # Helper function to reverse a portion of the list in place
    def reverse(left, right):
        while left < right:
            # Swap the elements at indices left and right
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1

    # Reverse the entire array
    reverse(0, n - 1)
    # Reverse the first k elements to restore their order
    reverse(0, k - 1)
    # Reverse the remaining n - k elements
    reverse(k, n - 1)

# Example usage:
# nums = [1,2,3,4,5,6,7]
# rotate(nums, 3)

# print(nums) # Output: [4,5,6,1,2,3,7]

```

#532

MEDIUM

K-diff Pairs in an Array

LeetCode · SimplyList · WalkCC · LeetCode · Editorial
 Array · Hash Table · Two Pointers · Binary Search · Sorting
 Lists · Collections

Problem:

Given an array of integers nums and an integer k, return the number of unique k-diff pairs in the array.

A k-diff pair is an integer pair (nums[i], nums[j]), where the following are true:

- 0 <= i, j < nums.length
- i != j
- |nums[i] - nums[j]| == k

Notice that |val| denotes the absolute value of val.

Example 1:

Input: nums = [3,1,4,1,5], k = 2

Output: 2

Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5).

Although we have two 1s in the input, we should only return the number of unique pairs.

Example 2:

```

# Example usage:
# print(findPairs([1,4,1,3], 2)) # Expected output: 2

```

Python

```

class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        ans = 0
        numToIndex = {num: i for i, num in enumerate(nums)}

        for i, num in enumerate(nums):
            target = num + k
            if target in numToIndex and numToIndex[target] != i:
                ans += 1
                del numToIndex[target]

        return ans

```

Python

```

class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        ans = set()
        vis = set()
        for x in nums:
            if x - k in vis:
                ans.add(x - k)
            if x + k in vis:
                ans.add(x)
            vis.add(x)
        return len(ans)

```

Python

```

class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        vis, ans = set(), set()
        for x in nums:
            if x - k in vis:
                ans.add(x - k)
            if x + k in vis:
                ans.add(x)
            vis.add(x)
        return len(ans)

```

C++

C++

```

class Solution {
public:
    int findPairs(vector<int>& nums, int k) {
        int ans = 0;
        unordered_map<int, int> numToIndex;
        for (int i = 0; i < nums.size(); ++i)
            numToIndex[nums[i]] = i;

        for (int i = 0; i < nums.size(); ++i) {
            const int target = nums[i] + k;
            if (count auto it = numToIndex.find(target);
                it != numToIndex.end()) {
                ++ans;
                numToIndex.erase(target);
            }
        }
        return ans;
    }
}

```

C++

```

class Solution {
public:
    int findPairs(vector<int>& nums, int k) {
        unordered_set<int> ans;
        for (int i = 0; i < nums.size(); ++i) {
            if (vis.count(x - k)) {
                ans.insert(x - k);
            }
            if (vis.count(x + k)) {
                ans.insert(x);
            }
            vis.insert(x);
        }
        return ans.size();
    }
}

```

C++

```

class Solution {
public:
    int findPairs(vector<int>& nums, int k) {
        unordered_set<int> vis;
        unordered_set<int> ans;
        for (int i = 0; i < nums.size(); ++i) {
            if (vis.count(x - k)) ans.insert(x - k);
            if (vis.count(x + k)) ans.insert(x);
            vis.insert(x);
        }
        return ans.size();
    }
}

```

Java

Java

```

class Solution {
public: int findPairs(int[] nums, int k) {
    Set ans = {};
    Map<Integer, Integer> numToIndex = new HashMap<>();
    for (int i = 0; i < nums.length; ++i)
        numToIndex.put(nums[i], i);
    for (int i = 0; i < nums.length; ++i) {
        final int target = nums[i] + k;
        if (numToIndex.containsKey(target) && numToIndex.get(target) != i) {
            ans.add(target);
            numToIndex.remove(target);
        }
    }
    return ans.size();
}
}

Java
class Solution {
public: int findPairs(int[] nums, int k) {
    Set<Integer> ans = new HashSet<>();
    Set<Integer> vis = new HashSet<>();
    for (int x : nums) {
        if (vis.contains(x - k)) {
            ans.add(x - k);
        }
        if (vis.contains(x + k)) {
            ans.add(x);
        }
        vis.add(x);
    }
    return ans.size();
}
}

Java
use std::collections::HashSet;

impl Solution {
pub fn find_pairs(nums: Vec<i32>, k: i32) -> i32 {
    let mut ans = HashSet::new();
    let mut vis = HashSet::new();

    for &x in &nums {
        if vis.contains(&x - k) {
            ans.insert(x - k);
        }
        if vis.contains(&x + k) {
            ans.insert(x);
        }
        vis.insert(x);
    }
    ans.len() as i32
}
}

```

Two Pointers · LeetCode · By Pattern

— 1255 —

LeetCode

```

Java
class Solution {
public: int findPairs(int[] nums, int k) {
    Set<Integer> vis = new HashSet<>();
    Set<Integer> ans = new HashSet<>();
    for (int v : nums) {
        if (vis.contains(v - k)) {
            ans.add(v - k);
        }
        if (vis.contains(v + k)) {
            ans.add(v);
        }
        vis.add(v);
    }
    return ans.size();
}
}

TypeScript
typeScript
function findPairs(nums: number[], k: number): number {
    const ans = new Set<number>();
    const vis = new Set<number>();
    for (const x of nums) {
        if (vis.has(x - k)) {
            ans.add(x - k);
        }
        if (vis.has(x + k)) {
            ans.add(x);
        }
        vis.add(x);
    }
    return ans.size;
}

TypeScript
function findPairs(nums: number[], k: number): number {
    const ans = new Set<number>();
    const vis = new Set<number>();
    for (const x of nums) {
        if (vis.has(x - k)) {
            ans.add(x - k);
        }
        if (vis.has(x + k)) {
            ans.add(x);
        }
        vis.add(x);
    }
    return ans.size;
}

Go
Go

```

Two Pointers · LeetCode · By Pattern

— 1256 —

LeetCode

```

func findPairs(nums []int, k int) int {
    ans := make(map[int]struct{})
    vis := make(map[int]struct{})

    for _, v := range nums {
        if _, ok := vis[v-k]; ok {
            ans[v-k] = struct{}{}
        }
        if _, ok := vis[v+k]; ok {
            ans[v+k] = struct{}{}
        }
        vis[v] = struct{}{}
    }
    return len(ans)
}

Go
func findPairs(nums []int, k int) int {
    vis := map[int]bool{}
    ans := map[int]bool{}
    for _, v := range nums {
        if vis[v-k] {
            ans[v-k] = true
        }
        if vis[v+k] {
            ans[v+k] = true
        }
        vis[v] = true
    }
    return len(ans)
}

Rust
Rust

```

Two Pointers · LeetCode · By Pattern

— 1257 —

LeetCode

```

impl Solution {
pub fn find_pairs(mut nums: Vec<i32>, k: i32) -> i32 {
    nums.sort();
    let n = nums.len();
    let mut res = 0;
    let mut left = 0;
    let mut right = 1;
    while right < n {
        let num = i32::abs(nums[left] - nums[right]);
        if num == k {
            res += 1;
        }
        if num < k {
            right += 1;
        }
        while right < n && nums[right - 1] == nums[right] {
            right += 1;
        }
    }
    while left < right && nums[left - 1] == nums[left] {
        left -= 1;
    }
    if left == right {
        right += 1;
    }
}
res
}
}

[*] Two Pointers — Pattern Solution (stored optimal)
Python

```

Two Pointers · LeetCode · By Pattern

— 1258 —

LeetCode

```

# Define a function to count k-diff pairs in the array.
def find_pairs(nums, k):
    # If k is negative, return 0 as absolute difference can't be negative.
    if k < 0:
        return 0

    count = 0

    # k == 0, need to find numbers that appear more than once.
    if k == 0:
        # Create a frequency map for nums.
        freq = {}
        for num in nums:
            freq[num] = freq.get(num, 0) + 1
        # Count numbers with frequency greater than 1.
        for val in freq.values():
            if val > 1:
                count += 1
    else:
        # k > 0, use a set to check for the existence of each number's complement.
        unique_nums = set(nums)
        for num in unique_nums:
            if num + k in unique_nums:
                count += 1
    return count

# Example usage:
# print(find_pairs([3,1,4,1,5], 2)) # Expected output: 2

C++
// Include necessary libraries.
#include <vector>
#include <unordered_map>
#include <unordered_set>
using namespace std;

// Define a function that returns the count of unique k-diff pairs.
int findPairs(vector<int>& nums, int k) {
    if (k < 0) return 0; // absolute difference can't be negative

    int count = 0;
    if (k == 0) {
        // Frequency map to count occurrences.
        unordered_map<int, int> freq;
        for (int num : nums) {
            freq[num]++;
        }
        // Count keys with frequency greater than 1.
        for (auto& pair : freq) {
            if (pair.second > 1) count++;
        }
    } else {
        // Using unordered_set for efficient lookup of unique numbers.
        unordered_set<int> uniqueNums(nums.begin(), nums.end());
        for (int num : uniqueNums) {

```

Two Pointers · LeetCode · By Pattern

— 1259 —

LeetCode

```

        if (uniqueNums.count(num + k)) count++;
    }
    return count;
}

// Example usage:
// int main() {
//     vector<int> nums = {3,1,4,1,5};
//     int k = 2;
//     int result = findPairs(nums, k); // Expected output: 2
//     return 0;
// }

#923 MEDIUM
3Sum With Multiplicity
LeetCode · Simplified · BackC · LeetCode · Editorial
Array · Hash Table · Two Pointers · Sorting · Counting
ListC CodeSignal

Problem:
Given an integer array arr, and an integer target, return the number of tuples i, j, k such that i < j < k and arr[i] + arr[j] + arr[k] == target.
As the answer can be very large, return it modulo 109 + 7.

Example 1:
Input: arr = [1,1,2,2,3,4,4,5,5], target = 8
Output: 10
Explanation:
Enumerating by the values (arr[i], arr[j], arr[k]):
(1, 1, 5) occurs 8 times;
(1, 1, 4) occurs 8 times;
(2, 1, 4) occurs 2 times;
(2, 1, 3) occurs 2 times.

Example 2:
Input: arr = [1,1,2,2,2,2], target = 5
Output: 12
Explanation:
arr[i] = 1, arr[j] = arr[k] = 2 occurs 12 times:
We choose one 1 from [1,1] in 2 ways,
and two 2s from [2,2,2,2] in 6 ways.

Example 3:
Input: arr = [2,1,3], target = 6
Output: 1
Explanation: (1, 2, 3) occurred one time in the array so we return 1.

Constraints:
• 3 <= arr.length <= 3000

```

Two Pointers · LeetCode · By Pattern

— 1260 —

LeetCode

```
• 0 <= arr[i] <= 100
• 0 <= target <= 300
```

>> Brute Force [Two Pointers] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def threeSumMulti(self, arr, target):
        # Brute Force O(n^2) - nested loops instead of two pointers
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                # check arr[i] and arr[j]
                pass
```

Python

```
MOD = 10**9 + 7
```

```
def threeSumMulti(arr, target):
    # Create a frequency array for numbers 0 to 100
    freq = [0] * 101
    for num in arr:
        freq[num] += 1

    count = 0
    # Iterate through all possible values for x, y, and implicitly z
    for x in range(101):
        if freq[x] == 0:
            continue
        for y in range(x, 101):
            if freq[y] == 0:
                continue
            z = target - x - y
            if x < 0 or x > 100 or freq[z] == 0:
                continue
            if x == y and y == z:
                # All numbers are the same, nC3 = n*(n-1)*(n-2)/6
                count += freq[x] * (freq[x] - 1) * (freq[x] - 2) // 6
            elif x == y and y != z:
                # x and y are the same, z is different, nC2 for x multiplied by count[z]
                count += (freq[x] * (freq[x] - 1) // 2) * freq[z]

            elif x < y and y == z:
                # All distinct, product of counts
                count += freq[x] * freq[y] * freq[z]
            elif x < y and y < z:
                # x, y, and z are all different, each multiplied by nC2 for y
                count += freq[x] * (freq[y] - 1) * (freq[z] - 1) // 2

    count %= MOD
    return count

# Example usage
print(threeSumMulti([1,1,1,1,2,2,3,3,4,4,5,5], 8))
```

Python

```
class Solution:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        MOD = 10**9 + 7
        ans = 0
        count = collections.Counter(arr)

        for i, x in count.items():
            for j, y in count.items():
                k = target - x - y
                if k not in count:
                    continue
                if i == j and j == k:
                    ans += ans * x * (x - 1) * (x - 2) // 6 % MOD
                elif i == j and j != k:
                    ans += ans * x * (x - 1) // 2 * count[k] % MOD
                elif i < j and j < k:
                    ans += (ans + x * y * count[k]) % MOD

        return ans % MOD
```

Python

```
class Solution:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        MOD = 10**9 + 7
        cnt = Counter(arr)
        ans = 0
        for j, b in enumerate(arr):
            cnt[j] -= 1
            for a in arr[j:]:
                c = target - a - b
                ans = (ans + cnt[c]) % MOD
            return ans
```

Python

```
class Solution:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        cnt = Counter(arr)
        ans = 0
        MOD = 10**9 + 7
        for j, b in enumerate(arr):
            cnt[j] -= 1
            for i in range(j):
                a = arr[i]
                c = target - a - b
                ans = (ans + cnt[c]) % MOD
            return ans
```

C++

C++

```
class Solution {
public:
    int threeSumMulti(vector<int>& arr, int target) {
        const int mod = 1000000007;
        int cnt[101] = {0};
        for (int x : arr) {
            ++cnt[x];
        }
        int n = arr.size();
        int ans = 0;
        for (int j = 0; j < n; ++j) {
            --cnt[j];
            for (int i = 0; i < j; ++i) {
                int c = target - arr[i] - arr[j];
                if (c <= 0 && c <= 100) {
                    ans = (ans + cnt[c]) % mod;
                }
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    const int mod = 1000000007;

    int threeSumMulti(vector<int>& arr, int target) {
        int cnt[101] = {0};
        for (int x : arr) {
            ++cnt[x];
        }
        long ans = 0;
        for (int j = 0; j < arr.size(); ++j) {
            int b = arr[j];
            --cnt[j];
            for (int i = 0; i < j; ++i) {
                int a = arr[i];
                int c = target - a - b;
                if (c <= 0 && c <= 100) {
                    ans += cnt[c];
                    ans %= mod;
                }
            }
        }
        return ans;
    }
};
```

Java

Java

```
class Solution {
    public int threeSumMulti(int[] arr, int target) {
        final int mod = (int) 1e9 + 7;
        int[] cnt = new int[101];
        for (int a : arr) {
            ++cnt[a];
        }
        int n = arr.length;
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            --cnt[i];
            for (int j = 0; j < i; ++j) {
                int c = target - arr[i] - arr[j];
                if (c <= 0 && c <= 100) {
                    ans = (ans + cnt[c]) % mod;
                }
            }
        }
        return ans;
    }
}
```

Java

```
class Solution {
    private static final int MOD = (int) 1e9 + 7;

    public int threeSumMulti(int[] arr, int target) {
        int[] cnt = new int[101];
        for (int v : arr) {
            ++cnt[v];
        }
        long ans = 0;
        for (int i = 0; i < arr.length; ++i) {
            int b = arr[i];
            --cnt[i];
            for (int j = 0; j < i; ++j) {
                int a = arr[j];
                int c = target - a - b;
                if (c <= 0 && c <= 100) {
                    ans = (ans + cnt[c]) % MOD;
                }
            }
        }
        return (int) ans;
    }
}
```

TypeScript

TypeScript

```
function threeSumMulti(arr: number[], target: number): number {
    const mod = 10**9 + 7;
    const cnt: number[] = Array(101).fill(0);
    for (const x of arr) {
        ++cnt[x];
    }
    let ans = 0;
    const n = arr.length;
    for (let j = 0; j < n; ++j) {
        --cnt[j];
        for (let i = 0; i < j; ++i) {
            const c = target - arr[i] - arr[j];
            if (c <= 0 && c <= 100) {
                ans = (ans + cnt[c]) % mod;
            }
        }
    }
    return ans;
}
```

TypeScript

```
function threeSumMulti(arr: number[], target: number): number {
    const mod = 10**9 + 7;
    const cnt: number[] = Array(101).fill(0);
    for (const x of arr) {
        ++cnt[x];
    }
    let ans = 0;
    const n = arr.length;
    for (let j = 0; j < n; ++j) {
        --cnt[j];
        for (let i = 0; i < j; ++i) {
            const c = target - arr[i] - arr[j];
            if (c <= 0 && c <= 100) {
                ans = (ans + cnt[c]) % mod;
            }
        }
    }
    return ans;
}
```

Go

Go

```
func threeSumMulti(arr []int, target int) (ans int) {
    const mod int = 1000000007
    cnt := [101]int{}
    for _, x := range arr {
        cnt[x]++
    }
    for j, b := range arr {
        cnt[j]--
        for _, a := range arr[j:] {
            if c := target - a - b; c <= 0 && c <= 100 {
                ans = (ans + cnt[c]) % mod
            }
        }
    }
    return ans
}
```

Go

```
func threeSumMulti(arr []int, target int) int {
    const mod int = 1000000007
    cnt := [101]int{}
    for _, v := range arr {
        cnt[v]++
    }
    ans := 0
    for j, b := range arr {
        cnt[j]--
        for i := 0; i < j; i++ {
            a := arr[i]
            c := target - a - b
            if c <= 0 && c <= 100 {
                ans += cnt[c]
                ans %= mod
            }
        }
    }
    return ans
}
```

[*] Two Pointers — Pattern Solution (stored optimal)

Python

Two Pointers · LeetMastery — By Pattern — 1267 — LeetMastery

#986

MEDIUM

Interval List Intersections

[LeetCode](#) •
 [SimplyList](#) •
 [WalkCC](#) •
 [LeetCode](#) •
 [Editorial](#)

Array • Two Pointers

Listo: Denny Zhang

Problem:

You are given two lists of closed intervals, `firstList` and `secondList`, where `firstList[i] = [starti, endi]` and `secondList[j] = [startj, endj]`. Each list of intervals is pairwise disjoint and in sorted order.

Two Pointers • LeetCodeMastery — By Pattern

— 1268 —

LeetCodeMastery

```

class Solution {
public:
    vector<vector<int>> intervalIntersection(vector<vector<int>>& firstList, vector<vector<int>>& secondList) {
        vector<vector<int>> ans;
        int n = firstList.size(), m = secondList.size();
        for (int i = 0, j = 0; i < n && j < m; ) {
            int l1 = max(firstList[i][0], secondList[j][0]);
            int r = min(firstList[i][1], secondList[j][1]);
            if (l1 <= r) ans.push_back({l1, r});
            if (firstList[i][1] < secondList[j][1])
                ++i;
            else
                ++j;
        }
        return ans;
    }
};

```

Java

```

class Solution {
public List<List<Integer>> intervalIntersection(List<List<Integer>> firstList, List<List<Integer>> secondList) {
    short i = 0;
    short j = 0;

    while (i < firstList.length && j < secondList.length) {
        // l1 is the start of the intersection
        // r1 is the end of the intersection
        final int l1 = Math.max(firstList[i].get(0), secondList[j].get(0));
        final int r1 = Math.min(firstList[i].get(1), secondList[j].get(1));
        if (l1 <= r1) {
            ans.add(new List<Integer>() {
                if (firstList[i].get(1) < secondList[j].get(1))
                    ++i;
                else
                    ++j;
            });
        }
        return ans.toArray(new List[ans.size()]);
    }
}

```

Java

```

class Solution {
public List<List<Integer>> intervalIntersection(List<List<Integer>> firstList, List<List<Integer>> secondList) {
    List<List<Integer>> ans = new ArrayList<>();
    int n = firstList.length, m = secondList.length;
    for (int i = 0, j = 0; i < n && j < m; ) {
        int l1 = Math.max(firstList[i].get(0), secondList[j].get(0));
        int r = min(firstList[i].get(1), secondList[j].get(1));
        if (l1 <= r) {
            ans.add(new List<Integer>() {
                if (firstList[i].get(1) < secondList[j].get(1))
                    ++i;
                else
                    ++j;
            });
        }
        return ans.toArray(new List[ans.size()]);
    }
}

```

Java

```

class Solution {
public List<List<Integer>> intervalIntersection(List<List<Integer>> firstList, List<List<Integer>> secondList) {
    List<List<Integer>> ans = new ArrayList<>();
    int n = firstList.length, m = secondList.length;
    for (int i = 0, j = 0; i < n && j < m; ) {
        int l1 = Math.max(firstList[i].get(0), secondList[j].get(0));
        int r = min(firstList[i].get(1), secondList[j].get(1));
        if (l1 <= r) {
            ans.add(new List<Integer>() {
                if (firstList[i].get(1) < secondList[j].get(1))
                    ++i;
                else
                    ++j;
            });
        }
        return ans.toArray(new List[ans.size()]);
    }
}

```

TypeScript

TypeScript

```

function intervalIntersection(firstList: number[][], secondList: number[][]): number[][] {
    const n = firstList.length;
    const m = secondList.length;
    const res = [];
    let i = 0;
    let j = 0;
    while (i < n && j < m) {
        const start = Math.max(firstList[i][0], secondList[j][0]);
        const end = Math.min(firstList[i][1], secondList[j][1]);
        if (start <= end) {
            res.push([start, end]);
        }
        if (firstList[i][1] < secondList[j][1]) {
            i++;
        } else {
            j++;
        }
    }
    return res;
}

```

TypeScript

```

function intervalIntersection(firstList: number[][], secondList: number[][]): number[][] {
    const n = firstList.length;
    const m = secondList.length;
    const res = [];
    let i = 0;
    let j = 0;
    while (i < n && j < m) {
        const start = Math.max(firstList[i][0], secondList[j][0]);
        const end = Math.min(firstList[i][1], secondList[j][1]);
        if (start <= end) {
            res.push([start, end]);
        }
        if (firstList[i][1] < secondList[j][1]) {
            i++;
        } else {
            j++;
        }
    }
    return res;
}

```

Go

Go

```

func intervalIntersection(firstList [][]int, secondList [][]int) [][]int {
    n, m := len(firstList), len(secondList)
    var ans [][]int
    for i, j := 0, 0; i < n && j < m; i++ {
        l1 := max(firstList[i][0], secondList[j][0])
        r := min(firstList[i][1], secondList[j][1])
        if l1 <= r {
            ans = append(ans, []int{l1, r})
        }
        if firstList[i][1] < secondList[j][1] {
            j++
        } else {
            i++
        }
    }
    return ans
}

```

Go

```

func intervalIntersection(firstList [][]int, secondList [][]int) [][]int {
    n, m := len(firstList), len(secondList)
    var ans [][]int
    for i, j := 0, 0; i < n && j < m; i++ {
        l1 := max(firstList[i][0], secondList[j][0])
        r := min(firstList[i][1], secondList[j][1])
        if l1 <= r {
            ans = append(ans, []int{l1, r})
        }
        if firstList[i][1] < secondList[j][1] {
            j++
        } else {
            i++
        }
    }
    return ans
}

```

Rust

Rust

```

impl Solution {
    pub fn interval_intersection(
        first_list: Vec<Vec<i32>>,
        second_list: Vec<Vec<i32>>,
    ) -> Vec<Vec<i32>> {
        let n = first_list.len();
        let m = second_list.len();
        let mut res = Vec::new();
        let mut i = 0;
        let mut j = 0;
        while i < n && j < m {
            let start = first_list[i][0].max(second_list[j][0]);
            let end = first_list[i][1].min(second_list[j][1]);
            if start <= end {
                res.push(vec![start, end]);
            }
            if first_list[i][1] < second_list[j][1] {
                i += 1;
            } else {
                j += 1;
            }
        }
        res
    }
}

```

Rust

```

impl Solution {
    pub fn interval_intersection(
        first_list: Vec<Vec<i32>>,
        second_list: Vec<Vec<i32>>,
    ) -> Vec<Vec<i32>> {
        let n = first_list.len();
        let m = second_list.len();
        let mut res = Vec::new();
        let mut i = 0;
        let mut j = 0;
        while i < n && j < m {
            let start = first_list[i][0].max(second_list[j][0]);
            let end = first_list[i][1].min(second_list[j][1]);
            if start <= end {
                res.push(vec![start, end]);
            }
            if first_list[i][1] < second_list[j][1] {
                i += 1;
            } else {
                j += 1;
            }
        }
        res
    }
}

```

[*] Two Pointers — Pattern Solution (stored optimal)

Python

```

# Function to compute the interval intersections using two pointers
def intervalIntersections(firstList, secondList):
    interactions = [] # List to store the resulting interactions
    i, j = 0, 0 # Pointers for firstList and secondList

    # Loop until either list is fully traversed
    while i < len(firstList) and j < len(secondList):
        start1, end1 = firstList[i]
        start2, end2 = secondList[j]

        # Find the intersection boundaries
        start_overlap = max(start1, start2)
        end_overlap = min(end1, end2)

        # Check if there is an intersection
        if start_overlap <= end_overlap:
            interactions.append([start_overlap, end_overlap])

        # Move the pointer that has the smaller interval ending
        if end1 < end2:
            i += 1
        else:
            j += 1

    return interactions

# Example usage:
firstList = [[0,2],[5,8],[13,23],[24,25]]
secondList = [[1,3],[8,12],[15,24],[25,30]]
print(intervalIntersections(firstList, secondList)) # Output: [[1,2],[5,8],[13,23],[24,24],[25,25]]

```

C++

```
// C++ solution using the two pointers technique
#include <vector>
#include <algorithm>
using namespace std;

class Solution {
public:
    vector<vector<int>> intervalIntersection(vector<vector<int>>& firstList, vector<vector<int>>& secondList) {
        vector<vector<int>> intersections;
        int i = 0, j = 0;

        // Iterate until one list is fully traversed
        while (i < firstList.size() && j < secondList.size()) {
            int start1 = firstList[i][0], end1 = firstList[i][1];
            int start2 = secondList[j][0], end2 = secondList[j][1];

            // Find the overlapping interval boundaries
            int startOverlap = max(start1, start2);
            int endOverlap = min(end1, end2);

            // If the intervals intersect, add the intersection to the result
            if (startOverlap <= endOverlap) {
                intersections.push_back({startOverlap, endOverlap});
            }

            // Move the pointer for the interval that ends first
            if (end1 < end2) {
                i++;
            } else {
                j++;
            }
        }

        return intersections;
    }
};

// Example usage:
// int main() {
//     // Solution one:
//     vector<vector<int>> firstList = {{0,2},{5,10},{12,22},{24,25}};
//     vector<vector<int>> secondList = {{1,5},{6,12},{15,24},{26,28}};
//     vector<vector<int>> result = sol.intervalIntersection(firstList, secondList);
//     // The result should be {{1,2},{5,6},{6,10},{12,22},{24,24},{25,25}}
//     return 0;
// }
```

#1055 **MEDIUM**
Shortest Way to Form String
[LeetCode](#) · [Simple Solution](#) · [HindiCC](#) · [LeetCode](#) · [Editorial](#)
 Two Pointers · String · Binary Search
 List's Premium 98

Problem:

Two Pointers · LeetCodeMastery — By Pattern

— 1279 —

LeetCodeMastery

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Given two strings source and target, return the minimum number of subsequences of source such that their concatenation equals target. If the task is impossible, return -1.

Example 1:

Input: source = "abc", target = "abcbc"
 Output: 2
 Explanation: The target "abcbc" can be formed by "abc" and "bc", which are subsequences of source "abc".

Example 2:

Input: source = "abc", target = "acdbc"
 Output: -1
 Explanation: The target string cannot be constructed from the subsequences of source string due to the character 'd' in target string.

Example 3:

Input: source = "xyz", target = "xyxyz"
 Output: 3
 Explanation: The target string can be constructed as follows "xy" + "y" + "xyz".

Constraints:

- 1 <= source.length, target.length <= 1000
- source and target consist of lowercase English letters.

>> **Brute Force [Two Pointers]** Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def shortestWay(self, source, target):
        # Brute Force (O(N^2)) - nested loops instead of two pointers
        n = len(source)
        for i in range(1, n):
            for j in range(i + 1, n):
                # Check source[i:j] and source[j:]
                pass
```

Python

Python

Two Pointers · LeetCodeMastery — By Pattern

— 1280 —

LeetCodeMastery

```
# Python solution using greedy two-pointer approach
def shortestWay(source, target):
    count = 0 # Count of subsequences used
    t_index = 0
    t_len = len(target)

    # Pre-check: If any character in target is not in source, return -1
    if set(target) - set(source):
        return -1

    # While there are still characters in target to match
    while t_index < t_len:
        s_index = 0 # Pointer for source
        progress = False # Flag to check if we made progress this pass

        # Iterate through source and try to match target characters
        while s_index < len(source) and t_index < t_len:
            if source[s_index] == target[t_index]:
                # Move to next character in target
                progress = True
                s_index += 1
                t_index += 1

        count += 1 # One subsequence used for this pass

        # If no progress was made, then it is impossible to form the target
        if not progress:
            return -1

    return count
```

Example usage:
 # print(shortestWay("abc", "abdcba")) # Output: 2

Python

```
class Solution:
    def shortestWay(self, source: str, target: str) -> int:
        def f(i, j):
            while i < n and j < m:
                if source[i] == target[j]:
                    j += 1
                    i += 1
                return j

            m, n = len(source), len(target)
            ans = j = 0
            while j < m:
                k = f(i, j)
                if k == j:
                    return -1
                j = k
                ans += 1
            return ans
```

Python

Two Pointers · LeetCodeMastery — By Pattern

— 1281 —

LeetCodeMastery

```
class Solution:
    def shortestWay(self, source: str, target: str) -> int:
        def f(i, j):
            while i < n and j < m:
                if source[i] == target[j]:
                    j += 1
                    i += 1
                return j

            m, n = len(source), len(target)
            ans = j = 0
            while j < m:
                k = f(i, j)
                if k == j:
                    return -1
                j = k
                ans += 1
            return ans
```

Java

Java

```
class Solution {
    public int shortestWay(String source, String target) {
        int ans = 0;

        for (int i = 0; i < target.length(); i++) {
            final int prevIndex = i;
            for (int j = 0; j < source.length(); j++) {
                if (i == target.length() && source.charAt(j) == target.charAt(i))
                    ++i;
                // All chars in source didn't match target[i].
                if (i == prevIndex)
                    return -1;
            }
            ++ans;
        }
        return ans;
    }
}
```

Java

Two Pointers · LeetCodeMastery — By Pattern

— 1282 —

LeetCodeMastery

```
class Solution {
    public int shortestWay(String source, String target) {
        int n = source.length(), m = target.length();
        int ans = 0, j = 0;
        while (j < m) {
            int i = 0;
            boolean ok = false;
            while (i < n && j < m) {
                if (source.charAt(i) == target.charAt(j)) {
                    ok = true;
                    ++j;
                }
                ++i;
            }
            if (!ok) {
                return -1;
            }
            ++ans;
        }
        return ans;
    }
}
```

Java

```
class Solution {
    public int shortestWay(String source, String target) {
        int n = source.length(), m = target.length();
        int ans = 0, j = 0;
        while (j < m) {
            int i = 0;
            boolean ok = false;
            while (i < n && j < m) {
                if (source.charAt(i) == target.charAt(j)) {
                    ok = true;
                    ++j;
                }
                ++i;
            }
            if (!ok) {
                return -1;
            }
            ++ans;
        }
        return ans;
    }
}
```

[*] **Two Pointers — Pattern Solution (stored optimal)**

Python

Two Pointers · LeetCodeMastery — By Pattern

— 1283 —

LeetCodeMastery

```
# Python solution using greedy two-pointer approach
def shortestWay(source, target):
    count = 0 # Count of subsequences used
    t_index = 0
    t_len = len(target)

    # Pre-check: If any character in target is not in source, return -1
    if set(target) - set(source):
        return -1

    # While there are still characters in target to match
    while t_index < t_len:
        s_index = 0 # Pointer for source
        progress = False # Flag to check if we made progress this pass

        # Iterate through source and try to match target characters
        while s_index < len(source) and t_index < t_len:
            if source[s_index] == target[t_index]:
                # Move to next character in target
                progress = True
                s_index += 1
                t_index += 1

        count += 1 # One subsequence used for this pass

        # If no progress was made, then it is impossible to form the target
        if not progress:
            return -1

    return count
```

Example usage:
 # print(shortestWay("abc", "abdcba")) # Output: 2

C++

Two Pointers · LeetCodeMastery — By Pattern

— 1284 —

LeetCodeMastery

```
// C++ solution using greedy two-pointer approach

#include <iostream>
#include <string>
#include <unordered_set>
using namespace std;

int shortestWay(string source, string target) {
    int count = 0;
    int sIndex = 0;
    int tIndex = 0;

    // Pre-check: Ensure every character in target exists in source
    unordered_set<char> sourceSet(source.begin(), source.end());
    for (char c : target) {
        if (sourceSet.find(c) == sourceSet.end()) {
            return -1;
        }
    }

    // Process the target by scanning the source repeatedly
    while (tIndex < target.size()) {
        int sIndex = 0; // Pointer for source
        bool progress = false; // Check if any matching occurs in this pass

        while (sIndex < source.size()) {
            if (source[sIndex] == target[tIndex]) {
                tIndex++; // Move to the next character in target when a match is found
                progress = true;
            }
            sIndex++;
        }

        count++; // One subsequence has been used

        if (!progress)
            return -1; // If no progress is made, forming target is impossible
    }

    return count;
}

// Example usage:
// int main() {
//     cout << shortestWay("abc", "abcb") << endl; // Output: 2
//     return 0;
// }
```

#2563 MEDIUM
[LeetCode](#) · [Simplest](#) · [Naive](#) · [LeetCode](#) · [Editorial](#)

Array · Two Pointers · Binary Search · Sorting
[LeetCodeSignal](#)

Problem:

Two Pointers · [LeetCode](#) · By Pattern

— 1285 —

[LeetCode](#)

Given a 0-indexed integer array `nums` of size `n` and two integers `lower` and `upper`, return the number of fair pairs.

A pair (i, j) is fair if:

- $0 \leq i < j < n$, and
- $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$

Example 1:

Input: `nums = [0,1,7,4,5]`, `lower = 3`, `upper = 6`

Output: 6

Explanation: There are 6 fair pairs: (0,3), (0,4), (0,5), (1,3), (1,4), and (1,5).

Example 2:

Input: `nums = [1,7,9,2,5]`, `lower = 11`, `upper = 11`

Output: 1

Explanation: There is a single fair pair: (2,3).

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums.length} == n$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{lower} \leq \text{upper} \leq 109$

>> Brute Force [Two Pointers] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        # Brute Force O(N^2) - nested loops instead of two pointers
        n = len(nums)
        for i in range(n):
            for j in range(i + 1, n):
                # Check nums[i] and nums[j]
                pass
```

Python

Python

Two Pointers · [LeetCode](#) · By Pattern

— 1286 —

[LeetCode](#)

```
import bisect

def countFairPairs(nums: List[int], lower: int, upper: int) -> int:
    # Sort the array to allow binary search
    nums.sort()
    fair_pairs = 0
    n = len(nums)

    for i in range(n):
        # Find lower bound index for the required complement so that nums[i] + nums[j] == lower
        left = bisect.bisect_left(nums, lower - nums[i], i + 1, n)
        # Find upper bound index for the required complement so that nums[i] + nums[j] == upper
        right = bisect.bisect_right(nums, upper - nums[i], i + 1, n)
        fair_pairs += (right - left)

    return fair_pairs

# Example usage:
print(countFairPairs([0,1,7,4,5], 3, 6)) # Expected output: 6
```

Python

```
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        # nums[i] + nums[j] == nums[i] + nums[j], so the condition that i < j
        # depends on i >= j and we can sort the array.
        nums.sort()

        def countLess(nums: List[int]) -> int:
            res = 0
            l = 0
            j = len(nums) - 1
            while l < j:
                while l < j and nums[l] + nums[j] > upper:
                    j -= 1
                res += j - l
                l += 1
            return res

        return countLess(upper) - countLess(lower - 1)
```

Python

```
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        nums.sort()
        ans = 0
        for i, x in enumerate(nums):
            j = bisect_left(nums, lower - x, lo=i + 1)
            k = bisect_right(nums, upper - x + 1, lo=i + 1)
            ans += k - j
        return ans
```

Python

Two Pointers · [LeetCode](#) · By Pattern

— 1287 —

[LeetCode](#)

```
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        ans = 0
        n = len(nums)
        for i, x in enumerate(nums):
            j = bisect_left(nums, lower - x, lo=i + 1)
            k = bisect_right(nums, upper - x + 1, lo=i + 1)
            ans += k - j
        return ans
```

C++

C++

```
class Solution {
public:
    long long countFairPairs(vector<int>& nums, int lower, int upper) {
        // nums[i] + nums[j] == nums[i] + nums[j], so the condition that i < j
        // depends on i >= j and we can sort the array.
        ranges::sort(nums);
        return countLess(nums, upper) - countLess(nums, lower - 1);
    }

    long long countLess(vector<int>& nums, int sum) {
        long res = 0;
        for (int i = 0; i < nums.size(); i++) {
            while (i < j && nums[i] + nums[j] > sum)
                j--;
            res += j - i;
        }
        return res;
    }
};
```

C++

```
class Solution {
public:
    long long countFairPairs(vector<int>& nums, int lower, int upper) {
        long long ans = 0;
        sort(nums.begin(), nums.end());
        for (int i = 0; i < nums.size(); i++) {
            auto l = lower_bound(nums.begin() + i + 1, nums.end(), lower - nums[i]);
            auto h = lower_bound(nums.begin() + i + 1, nums.end(), upper - nums[i] + 1);
            ans += h - l;
        }
        return ans;
    }
};
```

C++

Two Pointers · [LeetCode](#) · By Pattern

— 1288 —

[LeetCode](#)

```
class Solution {
public:
    long long countFairPairs(vector<int>& nums, int lower, int upper) {
        long long ans = 0;
        sort(nums.begin(), nums.end());
        for (int i = 0; i < nums.size(); i++) {
            auto l = lower_bound(nums.begin() + i + 1, nums.end(), lower - nums[i]);
            auto h = lower_bound(nums.begin() + i + 1, nums.end(), upper - nums[i] + 1);
            ans += h - l;
        }
        return ans;
    }
};
```

Java

```
class Solution {
    public long countFairPairs(int[] nums, int lower, int upper) {
        // nums[i] + nums[j] == nums[i] + nums[j], so the condition that i < j
        // depends on i >= j and we can sort the array.
        Arrays.sort(nums);
        return countLess(nums, upper) - countLess(nums, lower - 1);
    }

    private long countLess(int[] nums, int sum) {
        long res = 0;
        for (int i = 0; i < nums.length; i++) {
            while (i < j && nums[i] + nums[j] > sum)
                j--;
            res += j - i;
        }
        return res;
    }
}
```

Java

Two Pointers · [LeetCode](#) · By Pattern

— 1289 —

[LeetCode](#)

```
class Solution {
    public long countFairPairs(int[] nums, int lower, int upper) {
        Arrays.sort(nums);
        long ans = 0;
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            int j = search(nums, lower - nums[i], i + 1);
            int k = search(nums, upper - nums[i] + 1, i + 1);
            ans += k - j;
        }
        return ans;
    }

    private int search(int[] nums, int x, int left) {
        int right = nums.length;
        while (left < right) {
            int mid = (left + right) / 2;
            if (nums[mid] == x) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
```

Java

```
class Solution {
    public long countFairPairs(int[] nums, int lower, int upper) {
        Arrays.sort(nums);
        long ans = 0;
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            int j = search(nums, lower - nums[i], i + 1);
            int k = search(nums, upper - nums[i] + 1, i + 1);
            ans += k - j;
        }
        return ans;
    }

    private int search(int[] nums, int x, int left) {
        int right = nums.length;
        while (left < right) {
            int mid = (left + right) / 2;
            if (nums[mid] == x) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
```

TypeScript

Two Pointers · [LeetCode](#) · By Pattern

— 1290 —

[LeetCode](#)

```
TypeScript
function countFairPairs(nums: number[], lower: number, upper: number): number {
    const search = (x: number, l: number): number => {
        let r = nums.length;
        while (l < r) {
            const mid = (l + r) >> 1;
            if (nums[mid] == x) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    };

    nums.sort((a, b) => a - b);
    let ans = 0;
    for (let i = 0; i < nums.length; ++i) {
        const j = search(lower - nums[i], i + 1);
        const k = search(upper - nums[i] + 1, i + 1);
        ans += k - j;
    }
    return ans;
}

TypeScript
function countFairPairs(nums: number[], lower: number, upper: number): number {
    const search = (x: number, l: number): number => {
        let r = nums.length;
        while (l < r) {
            const mid = (l + r) >> 1;
            if (nums[mid] == x) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    };

    nums.sort((a, b) => a - b);
    let ans = 0;
    for (let i = 0; i < nums.length; ++i) {
        const j = search(lower - nums[i], i + 1);
        const k = search(upper - nums[i] + 1, i + 1);
        ans += k - j;
    }
    return ans;
}

Go
Go
```

```
func countFairPairs(nums []int, lower int, upper int) (ans int64) {
    sort.Ints(nums)
    for i, n := range nums {
        j := sort.Search(len(nums), func(h int) bool { return h >= 1 && nums[h] == lower-n })
        k := sort.Search(len(nums), func(h int) bool { return h >= 1 && nums[h] == upper-n+1 })
        ans += int64(k - j)
    }
    return
}

Go
func countFairPairs(nums []int, lower int, upper int) (ans int64) {
    sort.Ints(nums)
    for i, n := range nums {
        j := sort.Search(len(nums), func(h int) bool { return h >= 1 && nums[h] == lower-n })
        k := sort.Search(len(nums), func(h int) bool { return h >= 1 && nums[h] == upper-n+1 })
        ans += int64(k - j)
    }
    return
}

Two Pointers — Pattern Solution (stored optimal)
Python
import bisect
def countFairPairs(nums, lower, upper):
    # Sort the array to allow binary search
    nums.sort()
    fair_pairs = 0
    n = len(nums)

    for i in range(n):
        # Find lower bound index for the required complement so that nums[i] + nums[j] == lower
        left = bisect.bisect_left(nums, lower - nums[i], i + 1, n)
        # Find upper bound index for the required complement so that nums[i] + nums[j] == upper
        right = bisect.bisect_right(nums, upper - nums[i], i + 1, n)
        fair_pairs += (right - left)

    return fair_pairs

# Example usage
print(countFairPairs([8,1,7,4,4,5], 3, 6)) # Expected output: 6

C++
```

```
#include <iostream>
#include <vector>
#include <algorithm> // For sort, lower_bound, and upper_bound

using namespace std;

long Long countFairPairs(vector<int>& nums, int lower, int upper) {
    // Sort the array to allow binary search.
    sort(nums.begin(), nums.end());
    long long fairPairs = 0;
    int n = nums.size();

    // Iterate over each element.
    for (int i = 0; i < n; ++i) {
        // Find the lower bound index for the required complement:
        // nums[i] + nums[j] == lower ==> nums[j] == lower - nums[i]
        auto leftIt = lower_bound(nums.begin() + i + 1, nums.end(), lower - nums[i]);
        // Find the upper bound index:
        // nums[i] + nums[j] == upper ==> nums[j] == upper - nums[i]
        auto rightIt = upper_bound(nums.begin() + i + 1, nums.end(), upper - nums[i]);
        // Add the count of valid pairs for index i.
        fairPairs += (rightIt - leftIt);
    }

    return fairPairs;
}

int main() {
    vector<int> nums = {8,1,7,4,4,5};
    int lower = 3, upper = 6;
    cout << countFairPairs(nums, lower, upper) << endl; // Expected output: 6
    return 0;
}
```

Quick Review — Two Pointers 19 questions · insights · complexity · solution

#88 Merge Sorted Array [Easy]Key Insights

- Both arrays are already sorted in non-decreasing order.
- The merge should be done in-place into `nums1`.
- Start merging from the end to avoid overwriting elements in `nums1`.
- Use two pointers, one for `nums1` and one for `nums2`, and a tail pointer for placement.

Space & TimeTime Complexity: $O(m + n)$ Space Complexity: $O(1)$ SolutionWe use a two-pointer approach starting from the end of both arrays. Initialize three pointers:

- `p1` pointing to the last valid element in `nums1`.
- `p2` pointing to the last element in `nums2`.
- `tail` pointing to the last index of `nums1`.

While both pointers are valid, compare the elements pointed by `p1` and `p2`, placing the larger element at the tail position. Decrement the corresponding pointer and the tail pointer. Finally, if any elements remain in `nums2` (i.e., `p2 >= 0`), copy them over into `nums1`. This solution leverages the extra space at the end of `nums1` and ensures the merge is done in-place in linear time.

#125 Valid Palindrome [Easy]Key Insights

- Use two pointers (one starting at the beginning and one at the end) to compare characters.
- Skip characters that are not alphanumeric.
- Compare characters in a case-insensitive manner.
- Continue until the pointers meet or cross.

Space & TimeTime Complexity: $O(n)$, where n is the length of the string, as each character is processed at most once. Space Complexity: $O(1)$, using only a few extra variables for pointers and temporary comparisons.SolutionThe solution employs a two-pointer approach:

- Initialize two pointers: one at the start (left) and one at the end (right) of the string.
- Skip any non-alphanumeric characters using these pointers.
- Convert the remaining characters at each pointer to lowercase and compare them.
- If any mismatch is found, return false; otherwise, continue until the pointers cross.
- Return true if all corresponding characters match. This method efficiently checks for a palindrome without needing additional space for a filtered string.

#283 Move Zeroes [Easy]Key Insights

- Use a two-pointer technique: one pointer to iterate through the array and another pointer to track the position for the next non-zero element.
- For each non-zero element encountered, swap it with the element at the tracking pointer.
- This strategy minimizes operations by ensuring each non-zero element is moved at most once.
- After repositioning all non-zero elements, zeros will naturally occupy the remaining positions.
- The in-place requirement ensures space complexity remains $O(1)$.

Two Pointers · LeetCodeMastery — By Patterns — 1294 — LeetCodeMastery

Space & TimeTime Complexity: $O(n)$, where n is the number of elements in the array, since only one pass (or at most two simple passes) is required. Space Complexity: $O(1)$ extra space, as the reordering is accomplished in-place.SolutionThe solution leverages a two-pointer approach. The left pointer (`i`) keeps track of the index where the next non-zero element should be placed, while the right pointer (`j`) iterates through the array. Whenever a non-zero element is encountered, it is swapped with the element at index `i`, and `j` is then incremented. This effectively gathers all non-zero elements at the beginning while moving zeros to the end. A key optimization is to perform the swap only when necessary (i.e., when `i` and `j` are not the same) to reduce the number of write operations.

#408 Valid Word Abbreviation [Easy]Key Insights

- Use two pointers: one for looping through the word and one for the abbreviation.
- When encountering a digit in `abbr`, accumulate the number and skip that many characters in the word.
- Check for invalid cases such as numbers with leading zeros or consecutive digit segments that might imply adjacent skipped substrings.
- Compare letters directly when they are not digits.

Space & TimeTime Complexity: $O(n \cdot m)$ where n is the length of the word and m is the length of the abbreviation. Space Complexity: $O(1)$ (constant extra space).SolutionThe approach uses two pointers to iterate over the word and abbreviation. For each character in the abbreviation:

- If it is a letter, compare it with the current character in the word.
- If it is a digit, first ensure that it does not start with '0'. Then, convert the sequence of digits into a number which represents how many characters in the word to skip. After processing, both pointers should have reached the end of their respective strings for the abbreviation to be valid. This approach is efficient due to its linear pass with constant extra space.

#977 Squares of a Sorted Array [Easy]Key Insights

- Squaring numbers can disrupt the original order since negative numbers become positive.
- A two-pointer approach can efficiently retrieve the largest square from either end of the array.
- By comparing the absolute values at the two pointers, the largest square is inserted from the back of the result array.
- This method achieves an $O(n)$ time complexity without needing to sort the squared values afterward.

Space & TimeTime Complexity: $O(n)$ - We traverse the array only once. Space Complexity: $O(n)$ - We use an extra array to store the output.SolutionWe use a two-pointer technique to traverse the array from both ends. Initialize pointers (left and right) at the beginning and end of the array, respectively, and an index pointer starting at the end of a new results array. At each step, compare the absolute values of the elements at the left and right pointers. The larger absolute value, when squared, is placed in the result array at the current index, and the corresponding pointer is adjusted. This continues until the left pointer exceeds the right pointer. Filling the result array from the back ensures that the array remains sorted in non-decreasing order.

#11 Container With Most Water [Medium]Key Insights

Two Pointers · LeetCodeMastery — By Patterns — 1295 — LeetCodeMastery

• The container's area is determined by the distance between two lines multiplied by the shorter line's height.

• A brute-force approach checking all possible pairs would result in $O(n^2)$ time complexity, which is inefficient for large inputs.

• A two-pointer approach allows us to solve the problem in $O(n)$ time by initializing pointers at both ends and gradually moving them inward.

• By always moving the pointer corresponding to the shorter line, we are more likely to find a taller line that could potentially form a container with a larger capacity.

Space & TimeTime Complexity: $O(n)$ - The algorithm makes a single pass through the array with two pointers. Space Complexity: $O(1)$ - Only a few extra variables are used, regardless of the input size.SolutionThe optimal approach uses a two-pointer strategy:

- Start with two pointers at each end of the array.
- Calculate the area formed between the two lines at these pointers.
- Update the maximum area if the calculated value is larger.
- Move the pointer pointing to the shorter line inward, as this is the only possible way to potentially increase the height of the shorter line and thus the area.
- Continue until the two pointers meet.

This method effectively reduces the number of comparisons and ensures that every possible pair that can yield a larger area is considered.

#15 3Sum [Medium]Key Insights

- Sort the array to efficiently skip duplicates and facilitate the two pointers approach.
- Use a fixed pointer for the first element, and then use two additional pointers (left and right) to find pairs that sum to the negation of the fixed element.
- Skip duplicate elements to ensure unique triplets.

Space & TimeTime Complexity: $O(n^2)$ where n is the number of elements in the array. Space Complexity: $O(1)$ extra space (ignoring the space required for the output).SolutionWe first sort the array to bring duplicates together and allow the two-pointer technique to work efficiently. Then, iterate over the array and for each element, use two pointers (one at the beginning right after the current element and one at the end of the array) to find a pair that sums with the current element to zero. Skip duplicates for the current element and within the inner loop to avoid duplicate triplets. This ensures that each valid triplet is unique and the overall runtime remains efficient.

#16 3Sum Closest [Medium]Key Insights

- Sort the array to make it easier to use the two pointers technique.
- Fix one element and then use two pointers (left and right) to find the best pair that, along with the fixed element, produces a sum closest to the target.
- Update the best sum whenever a closer sum is found.
- The sorted order helps eliminate unnecessary iterations and allows efficient movement of pointers.

Space & TimeTime Complexity: $O(n^2)$ Space Complexity: $O(\log n)$ to $O(n)$ depending on the sorting algorithm used (typically in-place sort, so extra space might be minimal)Solution

Two Pointers · LeetCodeMastery — By Patterns — 1296 — LeetCodeMastery

The solution starts by sorting the array. For each index in the array (considered as the first element of the triple), we use two pointers: one starting just after the fixed element and the other at the end of the array. We calculate the sum of the fixed element and the two pointers. If this sum is closer to the target than the current best, we update our best sum. Depending on whether the current sum is less than or greater than the target, we adjust the left or right pointer to try and get closer to the target. This two-pointer method leverages the sorted order of the array for efficient summing.

#31 Next Permutation [Medium]

Key Insights

- The problem is solved by identifying the first pair from the right where the left element is less than the right element.
- Once this pivot is found, find the rightmost element that is greater than this pivot.
- Swap these two elements.
- Reverse the subarray that comes after the pivot to ensure the next permutation is the next lexicographically larger sequence.
- If no such pivot exists, the array is in descending order and should be reversed to get the smallest permutation.

Space & Time
Time Complexity: O(n)
Space Complexity: O(1)

Solution

We start by scanning the array from the right to find the first index i such that $nums[i] < \text{less than } nums[i+1]$. This index i represents the pivot point where the next permutation can be formed. If we find such an index, we then scan from the right again to locate the first number that is greater than $nums[i]$ (let this index be j), and swap these two numbers. Finally, we reverse the subarray starting from index $i+1$ to the end of the array, which gives us the next permutation in lexicographical order. This approach leverages in-place modifications using basic operations like swapping and reversing, thus meeting the constant space requirement.

#75 Sort Colors [Medium]

Key Insights

- Use the Dutch National Flag algorithm to partition the array into three sections.
- Maintain three pointers: one for the next position for 0 (low), one for the next position for 2 (high), and one to traverse the array (mid).
- Swap elements appropriately to ensure that all 0's are at the beginning, all 2's are at the end, and 1's remain in the middle.

Space & Time
Time Complexity: O(n)
Space Complexity: O(1)

Solution

The solution uses a three-pointer approach (Dutch National Flag algorithm). Initialize three pointers: low, mid, and high. As you iterate through the array:

- If the element is 0, swap it with the element at the low pointer and increment both low and mid.
- If the element is 1, simply move mid forward.
- If the element is 2, swap it with the element at the high pointer and decrement high.

This single pass method ensures that the array is sorted in-place with constant extra space. Key details include correctly handling swaps without losing any data and ensuring that the pointers are updated in the proper order.

#161 One Edit Distance [Medium]

Key Insights

- The difference in length between s and t must be at most 1.

Two Pointers · LeetCode · By Patterns

— 1297 —

LeetCode

the array and locate each word (using the space character as a delimiter) and then reverse each word individually to correct its letter order. This process is done in-place without any additional data structures, conserving space.

#189 Rotate Array [Medium]

Key Insights

- The rotation amount k might be greater than the length of the array, so use $k \% n$ (where n is the array length) to compute the effective rotation.
- A smart in-place solution involves reversing sections of the array:
- Reverse the entire array.
- Reverse the first k elements.
- Reverse the remaining n-k elements.
- Alternative approaches include using extra space or performing k single-step rotations (which is less optimal).

Space & Time
Time Complexity: O(n) where n is the number of elements in the array.
Space Complexity: O(1) if done in-place.

Solution

We solve the problem using the "reverse" technique to perform the rotation in-place. First, we adjust k by taking k modulo the length of the array to handle cases where k exceeds the array length. Then, we reverse the entire array. Next, we reverse the first k elements to restore their correct order. Finally, we reverse the remaining n-k elements. This sequence yields the array rotated to the right by k positions while strictly using O(1) extra space.

#532 K-diff Pairs in an Array [Medium]

Key Insights

- For $k < 0$, the answer is always 0 because the absolute difference cannot be negative.
- When $k = 0$, you are looking for numbers that appear at least twice.
- For $k > 0$, it is efficient to use a hash set or a hash map to check if each number's complement (number + k) exists.
- Uniqueness is ensured by processing each number only once from the set of unique numbers.

Space & Time
Time Complexity: O(n), where n is the length of the array since we iterate over the array and then over the unique numbers.
Space Complexity: O(n), for storing the frequency count or the unique elements in a hash map or hash set.

Solution

The solution first checks if k is negative, returning 0 immediately if true. For k equals 0, we count numbers that appear more than once using a frequency map. For k greater than 0, a set of unique numbers is created and for each number, we check if (number + k) is also present in the set. Each valid case contributes to the result count, ensuring that only unique pairs are considered.

#923 3Sum With Multiplicity [Medium]

Key Insights

- Use a frequency counter for elements since arr[i] is in the range [0, 100].
- Iterate through all possible combinations of numbers x, y, and z with $x < y < z$.
- For each valid triplet (x, y, z) that sums to target, count the number of ways using combinatorial formulas:
- All distinct: $\text{count}[x] * \text{count}[y] * \text{count}[z]$.
- Two numbers the same: use nc2 for the repeated element.
- All three the same: use nc3 .
- Use modulo arithmetic $(\text{MOD} = 10^9 + 7)$ at every step to avoid overflow.

Two Pointers · LeetCode · By Patterns

— 1299 —

LeetCode

Key Insights

- Sorting the array simplifies searching for pairs with required sum limits.
- For each element, binary search is used to locate:
- The first index where the complement makes the sum $\geq$ lower.
- The last index where the complement makes the sum $\leq$ upper.
- This avoids the need for a nested loop through the entire array, yielding a more efficient solution.

Space & Time
Time Complexity: O(n log n), primarily due to sorting and (for each element) binary search operations.
Space Complexity: O(n) if sorting creates a copy; O(1) additional space if sorting in-place.

Solution

The solution starts by sorting the input array. For each element at index i in the sorted array, we search for indices j (where $j > i$) such that the sum of $nums[j]$ and $nums[i]$ falls between lower and upper. Two binary search operations are performed:

- To find the smallest index j where $nums[i] + nums[j] \geq \text{lower}$.
- To find the largest index j where $nums[i] + nums[j] \leq \text{upper}$. Subtracting the two positions (and summing the counts for each i) gives the total number of fair pairs. Edge cases (like no valid pair) are automatically handled by the binary search approach.

Two Pointers · LeetCode · By Patterns

— 1301 —

LeetCode

- When lengths are equal, the only possibility is that exactly one character is different (replacement).
- When one string is longer by one character, the difference can be fixed by one insertion (or deletion in the other direction).
- Use two pointers to compare corresponding characters and carefully handle the first difference.

Space & Time

Time Complexity: O(n), where n is the length of the shorter string, as we traverse both strings at most once.
Space Complexity: O(1), using only constant extra space.

Solution

The solution uses a two-pointer approach. First, we check if the length difference is greater than 1, in which case they cannot be one edit apart. For the two primary cases:

- When strings are of equal length, iterate through both strings and count the number of differing characters. There should be exactly one difference.
- When lengths differ by one, use pointers for both strings. Upon encountering a difference, increment the pointer for the longer string only and ensure that no previous difference has been encountered. If a second discrepancy is found, return false. The key is to handle the edge case where the difference is at the end of the string.

#167 Two Sum II - Input Array Is Sorted [Medium]

Key Insights

- The array is sorted in non-decreasing order, which makes the two pointers technique very effective.
- Using two pointers (one at the start and one at the end) allows checking pairs and adjusting the pointers based on the sum.
- Since the problem guarantees exactly one solution and requires constant space, using two pointers satisfies both constraints.

Space & Time

Time Complexity: O(n), where n is the number of elements in the array, because in the worst-case scenario each element is visited at most once.
Space Complexity: O(1), as no additional data structure is used that scales with the input size.

Solution

We use the two pointers approach:

- Initialize two pointers, left at the beginning (index 0) and right at the end (last index) of the array.
- While left is less than right, compute the sum of the elements at the two pointers.
- If the sum equals the target, return the indices as [left+1, right+1] (to adjust for the 1-indexed array).
- If the sum is less than the target, move the left pointer to the right to increase the sum.
- If the sum is greater than the target, move the right pointer to the left to decrease the sum.

This approach leverages the sorted property and uses constant extra space.

#186 Reverse Words in a String II [Medium]

Key Insights

- Reverse the entire array first to obtain the reversed order of characters.
- Then, reverse each individual word to restore the correct letter order.
- Doing so leverages the two-pointer technique for in-place reversals.
- This method accomplishes the goal in O(n) time while using O(1) extra space.

Space & Time

Time Complexity: O(n)
Space Complexity: O(1)

Solution

The solution employs a two-step reversal process using the two-pointer technique. First, reverse the entire character array to reverse the overall order of characters. Although this reverses both the words and the letters in each word, it sets up the scenario where each word's letters are in reverse order. The next step is to iterate through

Two Pointers · LeetCode · By Patterns

— 1298 —

LeetCode

Space & Time

Time Complexity: O(M*2), where M = 101 (the range of possible numbers), which is efficient given the constraints.
Space Complexity: O(M) for the frequency array.

Solution

The solution first counts the frequency of each number in the array. Then it iterates through all valid triplets (x, y, z) with $x < y < z$ and checks if they sum up to the target. Depending on whether x, y, and z are distinct or have duplicates, the combination count is computed accordingly:

- If x, y, and z are all the same, compute the combination as nc3 .
- If only two are the same, compute as nc2 multiplied by the count of the third number.
- If all are distinct, use the direct product of the counts. This covers all cases while applying the modulo operation to the result.

#986 Interval List Intersections [Medium]

Key Insights

- Use two pointers to traverse both lists simultaneously.
- Determine the overlapping interval by taking the maximum of the start points and the minimum of the end points.
- If the overlapping condition (start $<=$ end) is met, add the intersection to the result.
- Move the pointer that has the smaller end value to explore further potential intersections efficiently.
- Achieve an overall time complexity of O(m+n), where m and n are the lengths of the two interval lists.

Space & Time

Time Complexity: O(m + n) where m and n are the lengths of firstList and secondList respectively.
Space Complexity: O(1) extra space (excluding the space used for the output).

Solution

The solution uses two pointers approach. Initialize two indices, one for each list, and iterate through both lists. For each pair of intervals, the potential intersection is computed by:

- Calculating the maximum of the two start values.
- Calculating the minimum of the two end values.

If the maximum start is less than or equal to the minimum end, there is an intersection, and it is added to the result list. The pointer corresponding to the interval with the smaller end value is then incremented since that interval cannot intersect with any further intervals from the other list. This process repeats until one of the lists is fully traversed.

#1055 Shortest Way to Form String [Medium]

Key Insights

- Check if every character in target exists in source; if not, the answer is -1.
- Use a greedy two-pointer strategy: iterate over target while scanning source for matching characters.
- During each pass over source, match as many characters as possible from target.
- Count the number of passes (subsequences) required until the target is completely matched.
- An optimized approach may precompute indices for binary search; however, the basic greedy simulation is efficient for the given constraints.

Space & Time

Time Complexity: O(n * m) in the worst-case, where n = length of source and m = length of target.
Space Complexity: O(1) for the greedy simulation (O(n) if additional data structures for binary search are used).

Solution

The solution uses a greedy two-pointer approach. We simulate the process of reading the target by repeatedly scanning through the source string. Each scan through source is treated as one subsequence concatenated to form the target. A pointer in the target moves forward whenever a matching character is found in source. If a complete pass through source does not advance the target pointer, then it is impossible to form the target and we return -1. The algorithm counts the number of passes required until the target is fully matched.

#2563 Count the Number of Fair Pairs [Medium]

Two Pointers · LeetCode · By Patterns

— 1300 —

LeetCode

#14 Heap — 20 questions · #14 of 21

#215 MEDIUM

Kth Largest Element in an Array

LeetCode · Simplify · Walkthrough · LeetCode · Editorial

Array · Divide and Conquer · Sorting · Heap · Quickselect

Lists · Grid 169

Problem:

Given an integer array nums and an integer k, return the kth largest element in the array.

Note that it is the kth largest element in the sorted order, not the kth distinct element.

Can you solve it without sorting?

Example 1:

Input: nums = [3,2,1,5,6,4], k = 2

Output: 5

Example 2:

Input: nums = [3,2,3,1,2,4,5,5,6], k = 4

Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

>> Brute Force [Heap] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def findKthLargest(self, nums, k):
        # Brute Force: Sort array in descending order and return the kth element
        nums.sort(reverse=True)
        return nums[k - 1]
```

Python

Python

Two Pointers · LeetCode · By Patterns

— 1302 —

LeetCode

```

import heapq

def findKthLargest(nums, k):
    # Create a min-heap with the first k elements
    min_heap = nums[:k]
    heapq.heapify(min_heap) # O(k) time to build the initial heap

    # Process the remaining elements
    for num in nums[k:]:
        # If the current element is larger than the smallest in the heap, replace it
        if num > min_heap[0]:
            heapq.heapreplace(min_heap, num) # O(log k)
    # The smallest element in the heap is the k-th largest overall
    return min_heap[0]

# Example usage:
nums = [3, 2, 1, 5, 6, 4]
k = 2
print(findKthLargest(nums, k)) # Expected output: 5

```

```

Python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        minheap = []

        for num in nums:
            heapq.heappush(minheap, num)
            if len(minheap) > k:
                heapq.heappop(minheap)

        return minheap[0]

```

```

Python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(l: int, r: int) -> int:
            if l >= r:
                return nums[l]
            i, j = l, r
            x = nums[(l + r) // 2]
            while i < j:
                while i < j:
                    if nums[i] >= x:
                        break
                while j > i:
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]
            if i < j:
                return quick_sort(i + 1, r)
            return quick_sort(l, j)

        n = len(nums)
        k = n - k
        return quick_sort(0, n - 1)

```

Heap · LeetCode - By Pattern

— 1303 —

LeetCode

```

Python
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        lo, hi = 0, len(nums) - 1

        while lo < hi:
            pos = self.partition(nums, lo, hi)
            if pos == k - 1:
                return nums[pos] # partially sorted, done
            elif pos < k - 1:
                lo = pos + 1
            else: # pos > k - 1
                hi = pos - 1

        return -1 # no value description

    def partition(self, nums: List[int], lo: int, hi: int) -> int:
        pivot, l, r = nums[lo], lo + 1, hi
        while l <= r:
            if nums[l] < pivot and nums[r] > pivot:
                # swap: num < left of pivot, num > r to count for both largest
                nums[l], nums[r] = nums[r], nums[l]
                l += 1
                r -= 1
            if nums[l] >= pivot:
                l += 1
            if nums[r] <= pivot:
                r -= 1

        if nums[l] == pivot:
            r = l

        # now nums[l] will have to influence looping
        nums[lo], nums[r] = nums[r], nums[lo]
        # possible there is duplicated num, but will be covered here
        return r

#####
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(left, right, k):
            if left == right:
                return nums[left]
            i, j = left - 1, right + 1
            x = nums[(left + right) // 2]
            while i < j:
                while i < j:
                    if nums[i] >= x:
                        break
                while j > i:
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]
            if i < j:
                return quick_sort(i + 1, r)
            return quick_sort(l, j)

        n = len(nums)
        k = n - k
        return quick_sort(0, n - 1)

```

Heap · LeetCode - By Pattern

— 1304 —

LeetCode

```

        if i < j:
            nums[i], nums[j] = nums[j], nums[i]
        if j > k:
            return quick_sort(j + 1, right, k)
        return quick_sort(left, j, k)

        n = len(nums)
        return quick_sort(0, n - 1, n - k)

#####
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(left, right, k):
            if left == right:
                return nums[left]
            i, j = left - 1, right + 1
            x = nums[(left + right) // 2]
            while i < j:
                while i < j:
                    if nums[i] >= x:
                        break
                while j > i:
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]
            if i < j:
                return quick_sort(i + 1, r)
            return quick_sort(l, j)

        n = len(nums)
        k = n - k
        return quick_sort(0, n - 1)

```

```

C++
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<>> minHeap;

        for (const int num : nums) {
            minHeap.push(num);
            if (minHeap.size() > k)
                minHeap.pop();
        }

        return minHeap.top();
    }
};

```

Heap · LeetCode - By Pattern

— 1305 —

LeetCode

```

class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        return quickSelect(nums, 0, nums.size() - 1, k);
    }

private:
    int quickSelect(vector<int>& nums, int l, int r, int k) {
        const int pivot = nums[r];

        int nextSwapped = l;
        for (int i = l; i < r; ++i)
            if (nums[i] > pivot)
                swap(nums[nextSwapped++], nums[i]);
        swap(nums[nextSwapped], nums[r]);

        const int count = nextSwapped - l + 1; // the number of 'num' == pivot
        if (count == k)
            return nums[nextSwapped];
        if (count < k)
            return quickSelect(nums, l, nextSwapped - 1, k);
        return quickSelect(nums, nextSwapped + 1, r, k - count);
    }
};

```

```

C++
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        int n = nums.size();
        k = n - k;

        auto quickSort = [&](int& left, int l, int r) -> int {
            if (l == r) {
                return nums[l];
            }

            int i = l - 1, j = r + 1;
            int x = nums[(l + r) // 2];
            while (i < j) {
                while (nums[i] < x) {
                    i++;
                }
                while (nums[j] > x) {
                    j--;
                }
                if (i < j) {
                    swap(nums[i], nums[j]);
                }
            }
            if (i < k) {
                return quickSort(quickSort, j + 1, r);
            }
            return quickSort(quickSort, l, j);
        };

        return quickSort(quickSort, 0, n - 1);
    }
};

```

Heap · LeetCode - By Pattern

— 1306 —

LeetCode

```

class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        int n = nums.size();
        return quickSort(nums, 0, n - 1, n - k);
    }

    int quickSort(vector<int>& nums, int left, int right, int k) {
        if (left == right) return nums[left];
        int l = left - 1, j = right + 1;
        int x = nums[(left + right) // 2];
        while (l < j) {
            while (nums[l] < x) {
                l++;
            }
            while (nums[j] > x) {
                j--;
            }
            if (l < j) swap(nums[l], nums[j]);
        }
        return j < k ? quickSort(nums, j + 1, right, k) : quickSort(nums, left, j, k);
    }
};

```

```

Java
class Solution {
    public int findKthLargest(int[] nums, int k) {
        Queue<Integer> minHeap = new PriorityQueue<>();

        for (final int num : nums) {
            minHeap.offer(num);
            while (minHeap.size() > k)
                minHeap.poll();
        }

        return minHeap.peek();
    }
}

```

Heap · LeetCode - By Pattern

— 1307 —

LeetCode

```

class Solution {
    private int[] nums;
    private int k;

    public int findKthLargest(int[] nums, int k) {
        this.nums = nums;
        this.k = nums.length - k;
        return quickSort(0, nums.length - 1);
    }

    private int quickSort(int l, int r) {
        if (l == r) {
            return nums[l];
        }

        int i = l - 1, j = r + 1;
        int x = nums[(l + r) // 2];
        while (l < j) {
            while (nums[i] < x) {
                i++;
            }
            while (nums[j] > x) {
                j--;
            }
            if (i < j) {
                swap(nums[i], nums[j]);
            }
        }
        if (i < k) {
            return quickSort(i + 1, r);
        }
        return quickSort(l, j);
    }

    private void swap(int i, int j) {
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}

```

```

Java
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        for (int num : nums) {
            minHeap.offer(num);
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }
        return minHeap.peek();
    }
}

```

Heap · LeetCode - By Pattern

— 1308 —

LeetCode

```

class Solution {
public: int findnthlargest(int[] nums, int k) {
    Map<Integer, Integer> cnt = new HashMap<>(nums.length);
    int n = Integer.MAX_VALUE;
    for (int i : nums) {
        n = Math.min(n, i);
        cnt.merge(n, 1, Integer::sum);
    }
    for (int i = n; --i > 0) {
        k = cnt.getOrDefault(i, 0);
        if (k == 0) {
            return i;
        }
    }
}
}

```

Java

```

use std::collections::HashMap;

impl Solution {
    pub fn find_nth_largest(nums: Vec<i32>, k: i32) -> i32 {
        const cnt: HashMap<isize, isize> = HashMap::new();
        let mut n = i32::MAX;

        for &v in &nums {
            cnt.entry(v).or_insert(0) += 1;
            if v < n {
                n = v;
            }
        }

        let mut k = k;
        for i in (i32::MAX..=n).rev() {
            if let Some(count) = cnt.get(&i) {
                k -= count;
                if k == 0 {
                    return i;
                }
            }
        }

        unreachable!();
    }
}

```

Java

```

class Solution {
public: int findnthlargest(int[] nums, int k) {
    int n = nums.length;
    return quickSort(nums, 0, n - 1, n - k);
}

private: int quickSort(int[] nums, int left, int right, int k) {
    if (left == right) {
        return nums[left];
    }
    int i = left - 1, j = right + 1;
    int a = nums[(left + right) >> 1];
    while (i < j) {
        while (nums[i] < a) {
            i++;
        }
        while (nums[j] > a) {
            j--;
        }
        if (i < j) {
            swap(nums[i], nums[j]);
        }
    }
    if (i < k) {
        return quickSort(nums, j + 1, right, k);
    }
    return quickSort(nums, left, j, k);
}
}

```

TypeScript

```

function findnthlargest(nums: number[], k: number): number {
    const n = nums.length;
    k = n - k;
    const quickSort = (l: number, r: number): number => {
        if (l === r) {
            return nums[l];
        }
        let [i, j] = [l - 1, r + 1];
        const t = nums[(l + r) >> 1];
        while (i < j) {
            while (nums[i] < t) {
                i++;
            }
            while (nums[j] > t) {
                j--;
            }
            if (i < j) {
                [nums[i], nums[j]] = [nums[j], nums[i]];
            }
        }
        if (j < k) {
            return quickSort(j + 1, r);
        }
        return quickSort(l, j);
    };
    return quickSort(0, n - 1);
}

```

TypeScript

```

function findnthlargest(nums: number[], k: number): number {
    const minQ = new MinPriorityQueue<number>();
    for (const x of nums) {
        minQ.enqueue(x);
        if (minQ.size() > k) {
            minQ.dequeue();
        }
    }
    return minQ.front();
}

```

TypeScript

```

function findnthlargest(nums: number[], k: number): number {
    const cnt: Record<number, number> = {};
    for (const x of nums) {
        cnt[x] = (cnt[x] || 0) + 1;
    }
    const n = Math.max(...nums);
    for (let i = n; --i > 0) {
        k -= cnt[i] || 0;
        if (k == 0) {
            return i;
        }
    }
}

```

TypeScript

```

function findnthlargest(nums: number[], k: number): number {
    const n = nums.length;
    const swap = [i: number, j: number] => {
        [nums[i], nums[j]] = [nums[j], nums[i]];
    };
    const sort = (l: number, r: number) => {
        if (l >= r) {
            return;
        }
        swap(l, l + Math.floor(Math.random() * (r - l + 1)));
        const num = nums[l];
        let mark = l;
        for (let i = l + 1; i <= r; i++) {
            if (nums[i] > num) {
                swap(i, mark);
                mark = i;
            }
        }
        swap(l, mark);
        sort(l, mark);
        sort(mark + 1, r);
    };
    sort(0, n);
    return nums[k - 1];
}

```

Go

```

func findnthlargest(nums []int, k int) int {
    k = len(nums) - k
    var quickSort func(l, r int) int
    quickSort = func(l, r int) int {
        if l == r {
            return nums[l]
        }
        i, j := l-1, r+1
        x := nums[(l+r)>>1]
        for i < j {
            for {
                if nums[i] <= x {
                    break
                }
            }
            for {
                if nums[j] >= x {
                    break
                }
            }
            if i < j {
                nums[i], nums[j] = nums[j], nums[i]
            }
        }
        if j < k {
            return quickSort(j+1, r)
        }
        return quickSort(l, j)
    }
    return quickSort(0, len(nums)-1)
}

```

Go

```

func findnthlargest(nums []int, k int) int {
    minQ := &[]{}
    for _, v := range nums {
        heap.Push(minQ, v)
        if minQ.Len() > k {
            heap.Pop(minQ)
        }
    }
    return minQ.IntSlice[0]
}

type hp struct{ sort.IntSlice }

func (h hp) Less(i, j int) bool { return h.IntSlice[i] < h.IntSlice[j] }
func (h hp) Push(v any) { h.IntSlice = append(h.IntSlice, v.(int)) }
func (h hp) Pop() any {
    v := h.IntSlice
    v = v[:len(v)-1]
    h.IntSlice = v[:len(v)-1]
    return v
}

```

Go

```

func findnthlargest(nums []int, k int) int {
    cnt := map[int]int{}
    n := len(nums)
    for _, v := range nums {
        cnt[v]++
        n = min(n, v)
    }
    for i := n; i-- > 0 {
        k -= cnt[i]
        if k == 0 {
            return i
        }
    }
}

```

Go

```

func findnthlargest(nums []int, k int) int {
    n := len(nums)
    return quickSort(nums, 0, n-1, n-k)
}

func quickSort(nums []int, left, right, k int) int {
    if left == right {
        return nums[left]
    }
    i, j := left-1, right+1
    x := nums[(left+right)>>1]
    for i < j {
        for {
            if nums[i] <= x {
                break
            }
        }
        for {
            if nums[j] >= x {
                break
            }
        }
        if i < j {
            nums[i], nums[j] = nums[j], nums[i]
        }
    }
    if j < k {
        return quickSort(nums, j+1, right, k)
    }
    return quickSort(nums, left, j, k)
}

```

Rust

Rust

```

use std::collections::BinaryHeap;

impl Solution {
    pub fn find_nth_largest(nums: Vec<i32>, k: i32) -> i32 {
        let mut minQ = BinaryHeap::new();
        for &v in &nums.iter() {
            minQ.push(-v);
            if minQ.len() > k as usize {
                minQ.pop();
            }
        }
        -minQ.peek().unwrap()
    }
}

```

Rust

```

use rand::Rng;

impl Solution {
    pub fn find_nth_largest(nums: Vec<i32>, k: i32) -> i32 {
        let k = k as usize;
        let n = nums.len();
        let mut l = 0;
        let mut r = n;
        while l < k - 1 && l < r {
            let mut i, rand = rand::rng().gen_range(l, r);
            let num = nums[i];
            let mut mark = k;
            for i in l..r {
                if nums[i] > num {
                    mark -= 1;
                    num.swap(i, mark);
                }
            }
            num.swap(l, mark);
            if mark > l == k {
                l = mark + 1;
            } else {
                r = mark;
            }
        }
        nums[k - 1]
    }
}

```

[1] Heap — Pattern Solution (matched from community answers)

Python

```

class Solution:
    def findMinRooms(self, nums: List[int], k: int) -> int:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, num)
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return len(minHeap[0])

```

#253 **MEDIUM**
Meeting Rooms II
LeetCode · SimplifyLast · WalaCC · LeetCode · Editorial
Array · Two Pointers · Greedy · Sorting · Heap
LeetCode Grind 100 | Premium 98

Problem:
Given an array of meeting time intervals intervals where intervals[i] = [starti, endi], return the minimum number of conference rooms required.

Example 1:

Input: intervals = [[0,30],[5,10],[15,20]]

Output: 2

Example 2:

Input: intervals = [[7,10],[2,4]]

Output: 1

Constraints:

- 1 <= intervals.length <= 104
- 0 <= starti < endi <= 106

>> **Brute Force (Heap) Interview Prep**

```

Python
# Brute Force (Heap)
class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        # Brute Force O(n^2) - calculate, assigns each meeting to first free room
        intervals.sort()
        rooms = [] # end times of each room
        for start, end in intervals:
            # Find and assign meeting to free room
            placed = False
            for i in range(len(rooms)):
                if rooms[i] <= start:
                    rooms[i] = end
                    placed = True
                    break
            # If not placed,
            # rooms.append(end)
            return len(rooms)

```

Heap · LeetCode · By Pattern

— 1315 —

LeetCode

Python

```

Python
import heapq

def minMeetingRooms(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])

    # Initialize a min-heap to store end times of meetings that are currently using rooms
    min_heap = []

    # Process each meeting interval
    for interval in intervals:
        start, end = interval

        # If the heap is not empty and the current meeting starts after the earliest ended meeting
        if min_heap and start >= min_heap[0]:
            # Pop the earliest ended meeting
            heapq.heappop(min_heap)

        # Allocate the current meeting and time into the heap (new or reused room)
        heapq.heappush(min_heap, end)

    # The number of rooms is the number of simultaneous meetings
    return len(min_heap)

# Example usage
intervals = [[0,30],[5,10],[15,20]]
print(minMeetingRooms(intervals)) # Output: 2

```

Python

```

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        minHeap = [] # Store the end times of each room

        for start, end in sorted(intervals):
            # There's no overlap, so we can reuse the same room.
            if minHeap and start >= minHeap[0]:
                heapq.heappop(minHeap)
            heapq.heappush(minHeap, end)

        return len(minHeap)

```

Python

```

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        n = len(intervals)
        d = [0] * (n + 1)
        for i in intervals:
            d[i[0]] += 1
            d[i[1]] -= 1
        ans = 0
        for i in d:
            ans += max(ans, 0)
        return ans

```

Heap · LeetCode · By Pattern

— 1316 —

LeetCode

```

Python
...
data = [1, 4, 4, 2, 1, 9, 4, 7, 5, 8]

list(accumulate(data, operator.mul)) # running product
[1, 12, 72, 288, 144, 1296, 9, 4, 48]

list(accumulate(data, max)) # running max/min
[1, 4, 5, 5, 5, 5, 9, 9, 9, 9]

>>> from itertools import accumulate
>>> data = [1, 4, 4, 2, 1, 9, 4, 7, 5, 8]
>>> accumulate(data)
<itertools.accumulate object at 802670704>
>>> list(accumulate(data))
[1, 7, 13, 15, 16, 25, 29, 32, 37, 45]
>>> max(accumulate(data))
45

https://docs.python.org/3/library/itertools.html#itertools.accumulate

```

```

# Analyze a 50 lines of 1000 with 4 annual payments of 50
cashflow = [1000, -500, -500, -500, -500]
list(accumulate(cashflow, lambda bal, pct: bal*(1.05 + pct)))

[1000, 505.0, 525.5, 552.8, 577.9000000000001, 627.5500000000001]

```

```

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        delta = [0] * 10000010
        for start, end in intervals:
            delta[start] += 1
            delta[end] -= 1
        return max(accumulate(delta))
# why not delta.sort()?
# Because accumulate will go by order from index 0 to index float
# just like from sortedcontainers import SortedDict

```

Heap · LeetCode · By Pattern

— 1317 —

LeetCode

```

meetings.sort()
ans = 0
count = 0
for meeting in meetings:
    if meeting[0] <= count:
        count += 1
    else:
        count -= 1
    ans = max(ans, count)
return ans

```

C++

```

C++
class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        // There is no meeting, so we can reuse the same room.
        priority_queue<int, vector<int>, greater<>> minHeap;
        ranges::sort(intervals);
        for (const vector<int>& interval : intervals) {
            // There's no overlap, so we can reuse the same room.
            if (minHeap.empty() && interval[0] >= minHeap.top())
                minHeap.pop();
            minHeap.push(interval[1]);
        }
        return minHeap.size();
    }
};

```

C++

```

class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        int n = 0;
        for (const auto& e : intervals) {
            n = max(n, e[1]);
        }
        vector<int> d(n + 1);
        for (const auto& e : intervals) {
            d[e[0]]++;
            d[e[1]]--;
        }
        int ans = 0, s = 0;
        for (int v : d) {
            s += v;
            ans = max(ans, s);
        }
        return ans;
    }
};

```

C++

Heap · LeetCode · By Pattern

— 1318 —

LeetCode

```

class Solution {
public:
    int minMeetingRooms(vector<vector<int>>& intervals) {
        int n = 10000010;
        vector<int> delta(n);
        for (auto e : intervals) {
            delta[e[0]]++;
            delta[e[1]]--;
        }
        for (int i = 0; i < n - 1; ++i) {
            delta[i + 1] += delta[i];
        }
        return *max_element(delta.begin(), delta.end());
    }
};

```

Java

```

Java
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        // Store the end times of each room.
        Queue<Integer> minHeap = new PriorityQueue<>();

        Arrays.sort(intervals, Comparator.comparingInt(interval -> interval[0]));

        for (int[] interval : intervals) {
            // There's no overlap, so we can reuse the same room.
            if (minHeap.isEmpty() && interval[0] >= minHeap.peek())
                minHeap.poll();
            minHeap.offer(interval[1]);
        }

        return minHeap.size();
    }
}

```

Java

Heap · LeetCode · By Pattern

— 1319 —

LeetCode

```

class Solution {
    public int minMeetingRooms(int[][] intervals) {
        final int n = intervals.length;
        int ans = 0;
        int[] starts = new int[n];
        int[] ends = new int[n];

        for (int i = 0; i < n; ++i) {
            starts[i] = intervals[i][0];
            ends[i] = intervals[i][1];
        }

        Arrays.sort(starts);
        Arrays.sort(ends);

        // 3 points to ends
        for (int i = 0; i < n; ++i) {
            if (starts[i] < ends[i])
                ++ans;
            else
                ++i;
        }

        return ans;
    }
}

```

Java

```

class Solution {
    public int minMeetingRooms(int[][] intervals) {
        int n = 0;
        for (int e : intervals) {
            n = Math.max(n, e[1]);
        }
        int[] d = new int[n + 1];
        for (var e : intervals) {
            d[e[0]]++;
            d[e[1]]--;
        }
        int ans = 0, s = 0;
        for (int v : d) {
            s += v;
            ans = Math.max(ans, s);
        }
        return ans;
    }
}

```

Java

Heap · LeetCode · By Pattern

— 1320 —

LeetCode


```
class Solution {
public:
    int minMeetingRooms(vector<int>& intervals) {
        map<int,int> delta;
        for (var i : intervals) {
            delta[i[0], 1, 1];
            delta[i[1], -1, 1];
        }
        int ans = 0, s = 0;
        for (var i : delta.keys()) {
            s += delta[i];
            ans = Math.max(ans, s);
        }
        return ans;
    }
}
```

Java

```
use std::collections::HashMap;

impl Solution {
    pub fn min_meeting_rooms(intervals: Vec<Vec<i32>>) -> i32 {
        let mut d: HashMap<i32, i32> = HashMap::new();
        for interval in intervals {
            let (l, r) = (interval[0], interval[1]);
            *d.entry(l).or_insert(0) += 1;
            *d.entry(r).or_insert(0) -= 1;
        }
        let mut times: Vec<i32> = d.keys().cloned().collect();
        times.sort();
        let mut ans = 0;
        let mut s = 0;
        for time in times {
            s += d[time];
            ans = ans.max(s);
        }
        ans
    }
}
```

Java

```
class Solution {
public:
    int minMeetingRooms(vector<int>& intervals) {
        int n = intervals.size();
        int delta = new int[n];
        for (int i = 0; i < intervals.size(); i++) {
            delta[i]++;
            delta[i+1]--;
        }
        int res = delta[0];
        for (int i = 1; i < n; i++) {
            delta[i] += delta[i-1];
            res = Math.max(res, delta[i]);
        }
        return res;
    }
}
```

TypeScript

TypeScript

```
function minMeetingRooms(intervals: number[][]): number {
    const n = Math.max(...intervals.map(([, r]) => r));
    const d: number[] = Array(n + 1).fill(0);
    for (const [l, r] of intervals) {
        d[l]++;
        d[r]--;
    }
    let ans, s = 0, i = 0;
    for (const v of d) {
        s += v;
        ans = Math.max(ans, s);
    }
    return ans;
}
```

TypeScript

```
function minMeetingRooms(intervals: number[][]): number {
    const d: { [key: number]: number } = {};
    for (const [l, r] of intervals) {
        d[l] = (d[l] || 0) + 1;
        d[r] = (d[r] || 0) - 1;
    }
    let [ans, s] = [0, 0];
    const keys = Object.keys(d);
    .map(Number)
    .sort((a, b) => a - b);
    for (const k of keys) {
        s += d[k];
        ans = Math.max(ans, s);
    }
    return ans;
}
```

TypeScript

```
function minMeetingRooms(intervals: number[][]): number {
    const n = Math.max(...intervals.map(([, r]) => r));
    const d: number[] = Array(n + 1).fill(0);
    for (const [l, r] of intervals) {
        d[l]++;
        d[r]--;
    }
    let [ans, s] = [0, 0];
    for (const v of d) {
        s += v;
        ans = Math.max(ans, s);
    }
    return ans;
}
```

Go

Go

```
func minMeetingRooms(intervals [][]int) int {
    n := 0
    for _, r := range intervals {
        n = max(n, r[1])
    }
    d := make([]int, n+1)
    for _, r := range intervals {
        d[r[0]]++
        d[r[1]]--
    }
    s := 0
    for _, v := range d {
        s += v
        ans = max(ans, s)
    }
    return ans
}
```

Go

```
func minMeetingRooms(intervals [][]int) int {
    d := make(map[int]int)
    for _, r := range intervals {
        d[r[0]]++
        d[r[1]]--
    }
    keys := make([]int, 0, len(d))
    for k := range d {
        keys = append(keys, k)
    }
    sort.Ints(keys)
    s := 0
    for _, k := range keys {
        s += d[k]
        ans = max(ans, s)
    }
    return ans
}
```

Go

```
func minMeetingRooms(intervals [][]int) int {
    n := 100000
    delta := make([]int, n)
    for _, r := range intervals {
        delta[r[0]]++
        delta[r[1]]--
    }
    for i := 1; i < n; i++ {
        delta[i] += delta[i-1]
    }
    return slices.Max(delta)
}
```

Rust

Rust

```
impl Solution {
    pub fn min_meeting_rooms(intervals: Vec<Vec<i32>>) -> i32 {
        let mut m = 0;
        for e in intervals {
            m = m.max(e[1]);
        }
        let mut d = vec![0; (m + 1) as usize];
        for e in intervals {
            d[e[0] as usize] += 1;
            d[e[1] as usize] -= 1;
        }
        let mut ans = 0;
        let mut s = 0;
        for v in d {
            s += v;
            ans = ans.max(s);
        }
        ans
    }
}
```

Rust

```
use std::collections::BinaryHeap;

impl Solution {
    pub fn min_meeting_rooms(intervals: Vec<Vec<i32>>) -> i32 {
        // The min heap that stores the earliest ending time among all meeting rooms
        let mut pq = BinaryHeap::new();
        let mut intervals = intervals;
        let n = intervals.len();
        // Let's first sort the intervals vector
        intervals.sort_by(|(l1, r1), (l2, r2)| r1.cmp(&r2));
        // Push the first and time to the heap
        pq.push(intervals[0][1]);
        // Traverse the intervals vector
        for i in 1..n {
            // Get the current top element from the heap
            if let Some(earliest_end_time) = pq.pop() {
                if end_time < intervals[i][0] {
                    // If the end time is earlier than the current begin time
                    let new_end_time = intervals[i][1];
                    pq.push(Reverse(new_end_time));
                } else {
                    // If the end time is later than the current begin time
                    pq.push(Reverse(intervals[i][1]));
                }
            }
        }
    }
}
```

```
// Otherwise, push the end time back and we also need a new room
pq.push(Reverse(end_time));
pq.push(Reverse(intervals[i][1]));
}
}
pq.len() as i32
}
```

[1] Heap — Pattern Solution (matched from community answers)

Python

```
import heapq

def minMeetingRooms(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])
    # Initialize a min-heap to store end times of meetings that are currently using rooms
    min_heap = []
    # Process each meeting interval
    for interval in intervals:
        start, end = interval
        # If the heap is not empty and the current meeting starts after the earliest ended meeting
        if min_heap and start > min_heap[0]:
            heapq.heappop(min_heap) # Reuse the room
        # Allocate the current meeting and time into the heap (new or reused room)
        heapq.heappush(min_heap, end)
    # The number of rooms is the number of simultaneous meetings
    return len(min_heap)

# Example usage
intervals = [[10, 30], [15, 20], [15, 30]]
print(minMeetingRooms(intervals)) # Output: 3
```

#347

MEDIUM

Top K Frequent Elements

LeetCode · Simplest · WalkCC · LeetCode · Editorial

Array · Hash Table · Divide and Conquer · Sorting · Heap · Bucket Sort · Counting · Quickselect
Like Donny Zhang

Problem:

Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]
Example 2:
Input: nums = [1], k = 1
Output: [1]
Example 3:
Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2
Output: [1,2]
Constraints:

- 1 <= nums.length <= 105
- -104 <= nums[i] <= 104
- k is in the range [1, the number of unique elements in the array].
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size.

>> Brute Force [Heap] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def topKFrequent(self, nums, k):
        # Count the frequency of each element in nums
        count = {}
        for num in nums:
            count[num] = count.get(num, 0) + 1
        return sorted(count, key=lambda x: -count[x])[:k]
```

Python

Python

```
# Python solution using bucket sort
def topKFrequent(nums, k):
    # Count the frequency of each element in nums
    frequency = {}
    for num in nums:
        frequency[num] = frequency.get(num, 0) + 1

    # Create buckets where index represents frequency
    buckets = [[] for _ in range(n + 1)]
    for num, count in frequency.items():
        buckets[count].append(num)

    # Gather the k most frequent elements
    result = []
    # Iterate over buckets in reverse order (highest frequency first)
    for freq in range(n, 0, -1):
        for num in buckets[freq]:
            result.append(num)
            if len(result) == k:
                return result

# Example usage
# print(topKFrequent([1,1,1,2,2,3], 2)) == Output: [1, 2]
```

Python

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        ans = []
        bucket = [[] for _ in range(len(nums) + 1)]
        for num, freq in collections.Counter(nums).items():
            bucket[freq].append(num)
        for k in reversed(bucket):
            ans += k
            if len(ans) == k:
                return ans
```

Python

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        return [x for x, _ in cnt.most_common(k)]
```

Python

```
...
nums = ["a", "a", "a", "a", "a"]
cnt = Counter(nums)
cnt
Counter({'a': 5, 'c': 2, 'b': 1})

# Default to keys
sorted_freq = sorted(cnt, key=lambda x: (-cnt[x], x))
sorted_freq
[('a', 5), ('c', 2), ('b', 1)]

from collections import Counter

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # Count the frequency of each element in the list
        cnt = Counter(nums)

        # Sort the elements by frequency in decreasing order
        # diff from below solution: sorted(cnt), not sorted(cnt.items())
        # since in below solution, we sort by (freq, num) not (num, freq)
        sorted_freq = sorted(cnt, key=lambda x: (-cnt[x], x))

        return sorted_freq[:k]

#####

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # Count the frequency of each element in the list
        freqs = Counter(nums)

        # Sort the elements by frequency in decreasing order
        sorted_freq = sorted(freqs.items(), key=lambda x: x[1], reverse=True)
        # since sorting by freq, we need to sort by (freq, num) not (num, freq)
        # since the key is (num, freq)
        top_k = [num for num, freq in sorted_freq[:k]]

        return top_k

#####

...
from heapq import heappush
h = []
heappush(h, (2, 2))
heappush(h, (1, 1))
heappush(h, (2, 1))
heappush(h, (2, 1))
```

```
... h
[(1, 1), (1, 1), (2, 1)]

from heapq import heappush
h = []
heappush(h, (1, 1))
heappush(h, (2, 1))
heappush(h, (2, 1))

from collections import Counter

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        h = []
        for num, freq in cnt.items():
            heappush(h, (freq, num)) # freq first, default sort by 1st element
            if len(h) > k:
                heappop(h)
        return [x[1] for x in h]

#####

from typing import List
import random
# (1, 1) is the average case
# O(n^2) worst case

C++
C++
```

```
struct T {
    int num;
    int freq;
};

class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        const int n = nums.size();
        vector<int> ans;
        unordered_map<int, int> count;
        auto compare = [](const T& a, const T& b) { return a.freq > b.freq; };
        priority_queue<T, vector<T>, decltype(compare)> minHeap(compare);

        for (const int num : nums)
            ++count[num];

        for (const auto& [num, freq] : count) {
            minHeap.emplace(num, freq);
            if (minHeap.size() > k)
                minHeap.pop();
        }

        while (!minHeap.empty())
            ans.push_back(minHeap.top().num), minHeap.pop();

        return ans;
    }
};
```

C++

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        const int n = nums.size();
        vector<int> ans;
        vector<vector<int>> buckets(n + 1);
        unordered_map<int, int> count;

        for (const int num : nums)
            ++count[num];

        for (const auto& [num, freq] : count)
            buckets[freq].push_back(num);

        for (int freq = n; freq > 0; --freq) {
            for (const int num : buckets[freq])
                ans.push_back(num);
            if (ans.size() == k)
                return ans;
        }

        throw;
    }
};
```

C++

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int, int> cnt;
        using pii = pair<int, int>;
        for (int x : nums) {
            ++cnt[x];
        }
        priority_queue<pii, vector<pii>, greater<pii>> pq;
        for (auto [x, c] : cnt) {
            pq.push({c, x});
            if (pq.size() > k) {
                pq.pop();
            }
        }
        vector<int> ans;
        while (!pq.empty()) {
            auto [freq, num] = pq.top().second;
            pq.pop();
            ans.push_back(num);
        }
    }
};
```

C++

```
// O(N * log(N * log(N))) time complexity for top-k frequent elements.
// Time: O(N * log N) on average, O(N * log^2 N) in the worst case
// Space: O(N)
class Solution {
public:
    vector<int> topKFrequent(vector<int>& A, int k) {
        if (A.size() == k) return A;
        unordered_map<int, int> cnt;
        for (int x : A) cnt[x]++;
        vector<int> ans;
        for (auto [x, c] : cnt) ans.push_back(x);
        if (ans.size() == k) return ans;
        auto partition = [&](int l, int R) {
            int i = l, j = R, pivotIndex = l + rand() * (R - l + 1), pivot = cnt[ans[pivotIndex]];
            swap(ans[l], ans[R]);
            for (int i = l + 1; i <= R; ++i) {
                if (cnt[ans[i]] > pivot) swap(ans[i], ans[i+1]);
            }
            swap(ans[l], ans[R]);
            return j;
        };
        auto quickSelect = [&](int l, int R) {
            int i = R - 1, j = ans.size() - 1;
            while (l < R) {
                int p = partition(l, R);
                int i = p - 1;
            }
        };
    }
};
```

```

        if (M + 1 == k) break;
        if (M + 1 < k) M = M - 1;
        else L = M + 1;
    }
    quickSelect(k);
    ans.println(k);
    return ans;
}
}

Java
class Solution {
    public int[] topK frequent(int[] nums, int k) {
        record V(int num, int freq) {}
        final int n = nums.length;
        int[] ans = new int[k];
        Map<Integer, Integer> count = new HashMap<>();
        Queue<V> pq = new PriorityQueue<>((Comparator.comparingInt((V f) -> f.freq));
        for (final int num : nums)
            count.merge(num, 1, Integer::sum);
        for (Map.Entry<Integer, Integer> entry : count.entrySet()) {
            final int num = entry.getKey();
            final int freq = entry.getValue();
            minHeap.offer(new V(num, freq));
            if (minHeap.size() > k)
                minHeap.poll();
        }
        for (int i = 0; i < k; i++)
            ans[i] = minHeap.poll().num;
        return ans;
    }
}

Java

```

```

class Solution {
    public int[] topK frequent(int[] nums, int k) {
        final int n = nums.length;
        List<Integer> ans = new ArrayList<>();
        List<Integer> bucket = new ArrayList<>();
        Map<Integer, Integer> count = new HashMap<>();
        for (final int num : nums)
            count.merge(num, 1, Integer::sum);
        for (final int num : count.keySet()) {
            final int freq = count.get(num);
            if (bucket.size() == 0)
                bucket.add(num);
            else if (freq > bucket.get(bucket.size() - 1))
                bucket.add(num);
        }
        for (int freq = n; freq > 0; --freq) {
            if (bucket.size() > 0)
                ans.add(bucket.get(0));
            if (ans.size() == k)
                return ans.stream().mapToInt(Integer::intValue).toArray();
        }
        throw new IllegalArgumentException();
    }
}

Java
class Solution {
    public int[] topK frequent(int[] nums, int k) {
        Map<Integer, Integer> cnt = new HashMap<>();
        for (int a : nums) {
            cnt.merge(a, 1, Integer::sum);
        }
        PriorityQueue<Map.Entry<Integer, Integer>> pq
            = new PriorityQueue<>((Comparator.comparingInt((Map.Entry e) -> e.getValue()));
        for (var e : cnt.entrySet()) {
            pq.offer(e);
            if (pq.size() > k) {
                pq.poll();
            }
        }
        return pq.stream().mapToInt(Map.Entry::getValue).toArray();
    }
}

Java

```

```

function topKFrequent(nums: number[], k: number): number[] {
    const cnt = new Map<number, number>();
    for (const n of nums) {
        cnt.set(n, (cnt.get(n) ?? 0) + 1);
    }
    const pq = new PriorityQueue<number>[(a, b) => a[1] - b[1]];
    for (const [n, c] of cnt) {
        pq.enqueue([n, c]);
        if (pq.size() > k) {
            pq.dequeue();
        }
    }
    return pq.toArray().map(x => x[0]);
}

Java
use std::cmp::Reverse;
use std::collections::BTreeMap, HashMap;

impl Solution {
    pub fn top_k_frequent(nums: Vec<i32>, k: i32) -> Vec<i32> {
        let mut cnt = HashMap::new();
        for x in nums {
            *cnt.entry(x).or_insert(0) += 1;
        }
        let mut pq = BinaryHeap::with_capacity(k as usize);
        for (x, &c) in cnt.iter() {
            pq.push(Reverse((x, *c)));
            if pq.len() > k as usize {
                pq.pop();
            }
        }
        pq.into_iter().map(|Reverse((_, x))| x).collect()
    }
}

Java

```

```

import java.util.*;

public class Top_K_Frequent_Elements {
    // ref: https://leetcode.com/problems/top-k-frequent-elements/solution/

    class Solution_official_01 {
        int[] ans;
        Map<Integer, Integer> count;

        public void swap(int a, int b) {
            int tmp = ans[a];
            ans[a] = ans[b];
            ans[b] = tmp;
        }

        public int partition(int left, int right, int pivot_index) {
            int pivot_frequency = count.get(ans[pivot_index]);
            // 1. move pivot to end
            swap(pivot_index, right);
            int store_index = left;
            // 2. move all less frequent elements to the left
            for (int i = left; i < right; i++) {
                if (count.get(ans[i]) < pivot_frequency) {
                    swap(store_index, i);
                    store_index++;
                }
            }
            // 3. move pivot to its final place
            swap(store_index, right);
            return store_index;
        }

        public void quickSelect(int left, int right, int k_smallest) {
            // base case: the list contains only one element
            if (left == right) return;
            // select a random pivot index
            Random random_num = new Random();
            int pivot_index = left + random_num.nextInt(right - left);
            // find the pivot position in a sorted list
        }
    }
}

```

```

        pivot_index = partition(left, right, pivot_index);
        // if the pivot is in its final sorted position
        if (k_smallest == pivot_index) {
            return;
        }
        // else if (k_smallest < pivot_index) {
        //     // go left
        //     quickSelect(left, pivot_index - 1, k_smallest);
        // } else {
        //     // go right
        //     quickSelect(pivot_index + 1, right, k_smallest);
        // }

        public int[] topKFrequent(int[] nums, int k) {
            // build hash map: character and how often it appears
            count = new HashMap<>();
            for (int num: nums) {
                count.put(num, count.getOrDefault(num, 0) + 1);
            }
            // array of unique elements
            int n = count.size();
            int[] arr = new int[n];
            for (int i = 0; i < n; i++)
                arr[i] = count.get(i);
            // sort by value
            Arrays.sort(arr);
            int[] ans = new int[k];
            for (int i = 0; i < k; i++)
                ans[i] = arr[n - i - 1];
            return ans;
        }
    }
}

Java
use std::collections::HashMap;

impl Solution {
    pub fn top_k_frequent(nums: Vec<i32>, k: i32) -> Vec<i32> {
        let mut map = HashMap::new();
        let mut max_count = 0;
        for num in nums.iter() {
            let val = map.get(&num).or_insert(0) + 1;
            map.insert(&num, val);
            max_count = max_count.max(val);
        }
        let mut h = BTreeMap::new();
        let mut res = Vec::new();
        while k > 0 {
            let mut max = 0;
            for key in map.keys() {
                let val = map.get(&key).or_insert(0);
                if val > max {
                    max = val;
                    h.insert(key, val);
                }
            }
            res.push(*h.keys().next().unwrap());
            h.clear();
            k -= 1;
        }
        res
    }
}

Java

```

```

}
res
}

TypeScript
function topKFrequent(nums: number[], k: number): number[] {
    let hashMap = new Map();
    for (let num of nums) {
        hashMap.set(num, (hashMap.get(num) ?? 0) + 1);
    }
    let list = [...hashMap.values()];
    list.sort((a, b) => b[1] - a[1]);
    let ans = [];
    for (let i = 0; i < k; i++) {
        ans.push(list[i][0]);
    }
    return ans;
}

Go
func topKFrequent(nums []int, k int) []int {
    cnt := map[int]int{}
    for _, v := range nums {
        cnt[v]++
    }
    h := hp{}
    for v, freq := range cnt {
        h.push(freq, v)
        if len(h) > k {
            h.pop()
        }
    }
    ans := make([]int, k)
    for i := range ans {
        ans[i] = h.pop().v
    }
    return ans
}

type pair struct { v, cnt int }
type hp []pair

func (h hp) Len() int { return len(h) }
func (h hp) Less(i, j int) bool { return h[j].cnt < h[i].cnt }
func (h hp) Swap(i, j int) { h[i], h[j] = h[j], h[i] }
func (h hp) Push(v interface{}) { h = append(h, v.(pair)) }
func (h hp) Pop() interface{} { h = h[:len(h)-1]; return h[len(h)-1].v }

Python

```

```

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # Count the frequency of each element in the list
        cnt = Counter(nums)

        # Sort the elements by frequency in decreasing order
        # diff from below solution: sorted(cnt), not sorted(cnt.items())
        # since in below solution, we call O(2^32-2^31) for O(2^32) for O(2^32) for O(2^32)
        sorted_freqs = sorted(cnt.items(), key=lambda x: (-cnt[x], x))

        return sorted_freqs[:k]

# Heap - LeetCode - By Pattern
# 1339 -
# LeetCode

```

```

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        # Sort the elements by frequency in decreasing order
        # diff from below solution: sorted(cnt), not sorted(cnt.items())
        # since in below solution, we call O(2^32-2^31) for O(2^32) for O(2^32) for O(2^32)
        sorted_freqs = sorted(cnt.items(), key=lambda x: (-cnt[x], x))

        return sorted_freqs[:k]

# Heap - LeetCode - By Pattern
# 1339 -
# LeetCode

```

#505 MEDIUM

The Maze II

LeetCode - Interview - WalkCC - LeetCode - Editorial

Array - DFS - BFS - Graph - Matrix - Heap - Shortest Path

List: Premium 98

Problem:

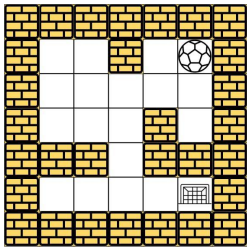
There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the m x n maze, the ball's start position and the destination, where start = [startrow, startcol] and destination = [destinationrow, destinationcol], return the shortest distance for the ball to stop at the destination. If the ball cannot stop at destination, return -1.

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1:

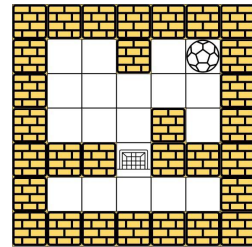


Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [4,4]

Output: 12

Explanation: One possible way is: left -> down -> left -> down -> right -> down -> right. The length of the path is 1 + 1 + 3 + 1 + 2 + 2 + 2 = 12.

Example 2:



Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [2,2]

Output: -1

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], start = [4,3], destination = [0,1]

Output: -1

- Constraints:**
- m == maze.length
 - n == maze[0].length
 - 1 <= m, n <= 100
 - maze[i][j] is 0 or 1.
 - start.length == 2
 - destination.length == 2
 - 0 <= startrow, destinationrow < m
 - 0 <= startcol, destinationcol < n
 - Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
 - The maze contains at least 2 empty spaces.

>> Brute Force (Heap) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def shortestDistance(self, maze, start, destination):
        # Brute Force: Do log n - sort repeatedly instead of heap
        data = list(maze)
        result = []
        for _ in range(k):
            data.sort()
            result.append(data.pop(0))
        return result

Python
Python
import heapq

def shortestDistance(maze, start, destination):
    m, n = len(maze), len(maze[0])
    # Initialize distance matrix with infinity
    dist = [[float('inf')] * n for _ in range(m)]
    dist[start[0]][start[1]] = 0
    # Priority queue (min-heap) storing (distance, row, col)
    heap = [(0, start[0], start[1])]
    # Directions: up, down, left, right
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while heap:
        d, row, col = heapq.heappop(heap)
        # If we have reached the destination, return the corresponding distance
        if (row, col) == destination:
            return d
        # Skip if we already found a better path to (row, col)
        if d > dist[row][col]:
            continue
        for dr, dc in directions:
            next_row, next_col = row + dr, col + dc
            # Ball the ball until it hits a wall
            while 0 <= next_row < m and 0 <= next_col < n and maze[next_row][next_col] == 0:
                next_row += dr
                next_col += dc
            # Check if the new distance is shorter
            if d + count < dist[next_row][next_col]:
                dist[next_row][next_col] = d + count
                heapq.heappush(heap, (dist[next_row][next_col], next_row, next_col))
    return -1

Python

```

```

class Solution:
    def shortestDistance(self, maze, start, destination):
        # Brute Force: Do log n - sort repeatedly instead of heap
        data = list(maze)
        result = []
        for _ in range(k):
            data.sort()
            result.append(data.pop(0))
        return result

Python
Python
import heapq

def shortestDistance(maze, start, destination):
    m, n = len(maze), len(maze[0])
    # Initialize distance matrix with infinity
    dist = [[float('inf')] * n for _ in range(m)]
    dist[start[0]][start[1]] = 0
    # Priority queue (min-heap) storing (distance, row, col)
    heap = [(0, start[0], start[1])]
    # Directions: up, down, left, right
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while heap:
        d, row, col = heapq.heappop(heap)
        # If we have reached the destination, return the corresponding distance
        if (row, col) == destination:
            return d
        # Skip if we already found a better path to (row, col)
        if d > dist[row][col]:
            continue
        for dr, dc in directions:
            next_row, next_col = row + dr, col + dc
            # Ball the ball until it hits a wall
            while 0 <= next_row < m and 0 <= next_col < n and maze[next_row][next_col] == 0:
                next_row += dr
                next_col += dc
            # Check if the new distance is shorter
            if d + count < dist[next_row][next_col]:
                dist[next_row][next_col] = d + count
                heapq.heappush(heap, (dist[next_row][next_col], next_row, next_col))
    return -1

Python

```

```

class Solution {
public:
    int shortestDistance(vector<vector<int>>& maze, vector<int>& start,
                        vector<int>& destination) {
        constint dir = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        const int n = maze.size();
        const int m = maze[0].size();
        queue<pair<int, int>> q{{start[0], start[1]}};
        vector<vector<int>> dist(n, vector<int>(n, INT_MAX));
        dist[start[0]][start[1]] = 0;

        while (!q.empty()) {
            const auto [x, y] = q.front();
            q.pop();
            for (const auto& [dx, dy] : dir) {
                int nx = x + dx;
                int ny = y + dy;
                while (isvalid(maze, nx == dx, y == dy)) {
                    x = nx;
                    y = ny;
                    ++step;
                }
                if (step < dist[x][y]) {
                    dist[x][y] = step;
                }
                if (x == destination[0] && y == destination[1])
                    continue;
                q.emplace(x, y);
            }
        }

        return dist[destination[0]][destination[1]] == INT_MAX
            ? -1
            : dist[destination[0]][destination[1]];
    }

private:
    bool isvalid(const vector<vector<int>>& maze, int x, int y) {
        return x == 0 && x == maze.size() && y == 0 && y == maze[0].size() &&
            maze[x][y] == 0;
    }
};

```

```

class Solution {
public:
    int shortestDistance(vector<vector<vector<int>>& maze, vector<int>& start, vector<int>& destination) {
        int n = maze.size(), m = maze[0].size();
        int dist[n][m];
        memset(dist, 0x3f, sizeof(dist));
        int xi = start[0], xj = start[1];
        int di = destination[0], dj = destination[1];
        dist[xi][xj] = 0;
        queue<pair<int, int>> q;
        q.emplace(xi, xj);
        int dirs[4] = {-1, 0, 1, 0, -1};
        while (!q.empty()) {
            auto [i, j] = q.front();
            q.pop();
            for (int d = 0; d < 4; ++d) {
                int x = i + j, y = j + d - dist[i][j];
                int a = dir[d][0], b = dir[d][1];
                while (x == 0 && x == m && y + b == 0 && y + b == m && maze[x + a][y + b] == 0) {
                    x += a;
                    y += b;
                    ++k;
                }
                if (k < dist[x][y]) {
                    dist[x][y] = k;
                    q.emplace(x, y);
                }
            }
        }
        return dist[di][dj] == 0x3f3f3f3f ? -1 : dist[di][dj];
    }
};

```

```

class Solution {
public:
    int shortestDistance(int[][] maze, int[] start, int[] destination) {
        final int[][] DMS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        final int n = maze.length;
        final int m = maze[0].length;
        Queue<Pair<Integer, Integer>> q = new ArrayDeque<>();
        List<int> dist = new List<Integer>(n * m);
        Arrays.stream(dist).forEach(v -> Arrays.fill(v, Integer.MAX_VALUE));
        dist[start[0]][start[1]] = 0;

        while (!q.isEmpty()) {
            final int i = q.poll().getKey();
            final int j = q.poll().getValue();
            for (int[] dir : DMS) {
                int nx = i + dir[0];
                int ny = j + dir[1];
                while (isvalid(maze, nx == dir[0], y == dir[1])) {
                    x = nx;
                    y = ny;
                    ++step;
                }
                if (step < dist[x][y]) {
                    dist[x][y] = step;
                    q.offer(new Pair<Integer, Integer>());
                }
            }
        }

        return dist[destination[0]][destination[1]] == Integer.MAX_VALUE
            ? -1
            : dist[destination[0]][destination[1]];
    }

private boolean isvalid(int[][] maze, int x, int y) {
    return x == 0 && x == maze.length && y == 0 && y == maze[0].length && maze[x][y] == 0;
}
}

```

```

class Solution {
public:
    int shortestDistance(int[][] maze, int[] start, int[] destination) {
        int n = maze.length, m = maze[0].length;
        final int inf = 1 << 30;
        final int[][] dist = new int[n][m];
        for (var row : dist)
            Arrays.fill(row, inf);
        int xi = start[0], xj = start[1];
        int di = destination[0], dj = destination[1];
        dist[xi][xj] = 0;
        Deque<int> q = new ArrayDeque<>();
        q.offer(new int[] {xi, xj});
        int[] dirs = {-1, 0, 1, 0, -1};
        while (!q.isEmpty()) {
            var p = q.poll();
            int i = p[0], j = p[1];
            for (int d = 0; d < 4; ++d) {
                int x = i + j, y = j + d - dist[i][j];
                int a = dir[d][0], b = dir[d][1];
                while (x == 0 && x == m && y + b == 0 && y + b == m && maze[x + a][y + b] == 0) {
                    x += a;
                    y += b;
                    ++k;
                }
                if (k < dist[x][y]) {
                    dist[x][y] = k;
                    q.offer(new int[] {x, y});
                }
            }
        }
        return dist[di][dj] == inf ? -1 : dist[di][dj];
    }
}

```

```

function shortestDistance(maze: number[][], start: number[], destination: number[]): number {
    const n = maze.length;
    const m = maze[0].length;
    const dist: number[][] = Array.from({ length: n }, () =>
        Array.from({ length: m }, () => Infinity));
    const [sx, sy] = start;
    const [dx, dy] = destination;
    dist[sx][sy] = 0;
    const q: number[][] = [[sx, sy]];
    const dirs = [-1, 0, 1, 0, -1];
    while (q.length) {
        const [i, j] = q.shift();
        for (let d = 0; d < 4; ++d) {
            let x = i + j, y = j + d - dist[i][j];
            const [a, b] = [dirs[d][0], dirs[d][1]];
            while (x == 0 && x == m && y + b == 0 && y + b == m && maze[x + a][y + b] == 0) {
                x += a;
                y += b;
                ++k;
            }
            if (k < dist[x][y]) {
                dist[x][y] = k;
                q.push([x, y]);
            }
        }
    }
    return dist[dx][dy] === Infinity ? -1 : dist[dx][dy];
}

```

```

func shortestDistance(maze [][]int, start []int, destination []int) int {
    n, m := len(maze), len(maze[0])
    dist = make([][]int, n)
    const inf = 1 << 30
    for i := range dist {
        dist[i] = make([]int, m)
        dist[i][0] = inf
    }
    dist[start[0]][start[1]] = 0
    dirs := [...]int{-1, 0, 1, 0, -1}
    for len(q) > 0 {
        p = q[0]
        q = q[1:]
        i, j := p[0], p[1]
        for d := 0; d < 4; d++ {
            x, y, k := i, j, dist[i][j]
            a, b := dirs[d][0], dirs[d][1]
            for x += a && x == m && y += b && y == m && maze[x][y] == 0 {
                x, y, k = x+a, y+b, k+1
            }
            if k < dist[x][y] {
                dist[x][y] = k
                q = append(q, []int{x, y})
            }
        }
    }
    di, dj := destination[0], destination[1]
    if dist[di][dj] == inf {
        return -1
    }
    return dist[di][dj]
}

```

```

import heapq

def shortestDistance(maze, start, destination):
    n, m = len(maze), len(maze[0])
    # Initialize priority queue with start location
    dist = [[float('inf')] * m for _ in range(n)]
    dist[start[0]][start[1]] = 0
    # Priority queue with (sum of steps, starting tuple (distance, row, col))
    heap = [(0, start[0], start[1])]
    # Directional vectors: down, up, right, left
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    while heap:
        d, row, col = heapq.heappop(heap)
        # If we have reached the destination, return the corresponding distance
        if (row, col) == destination:
            return d
        # Skip if we already found a better path to (row, col)
        if d > dist[row][col]:
            continue
        for dr, dc in directions:
            next_row, next_col, count = row, col, 0
            # Skip the cell, until it hits a wall
            while 0 <= next_row < dr < n and 0 <= next_col < dc < m and maze[next_row < dr][next_col < dc] != 0:
                next_row <= dr
                next_col <= dc
                count <= 1
            # Check if the new distance is shorter
            if d + count < dist[next_row][next_col]:
                dist[next_row][next_col] = d + count
                heapq.heappush(heap, (dist[next_row][next_col], next_row, next_col))

    return -1

```

#621

MEDIUM

Task Scheduler

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Hash Table · Greedy · Heap · Counting

Lists Grid 189

Problem:

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

Heap · LeetCode · By Pattern

— 1351 —

LeetCode

```

# Python solution for Task Scheduler
from collections import Counter

def leastInterval(tasks, n):
    # Count the frequency of each task
    task_counts = Counter(tasks)
    # Find the maximum frequency among tasks
    max_freq = max(task_counts.values())
    # Count how many tasks have the maximum frequency
    max_count = sum(1 for count in task_counts.values() if count == max_freq)

    # Calculate the minimum intervals required based on the most frequent task
    part_count = max_freq - 1 # Number of parts between the most frequent task
    part_length = n - (max_count - 1) # Effective empty slots per part after placing max frequency tasks
    empty_slots = part_count * part_length # Total empty slots to fill with idle or other tasks
    available_tasks = len(tasks) - max_freq * max_count # Remaining tasks to fill the empty slots
    idles = max(0, empty_slots - available_tasks) # Remaining idle slots if available tasks are not enough
    # The total intervals is the sum of all tasks and idle slots inserted
    return len(tasks) + idles

# Example usage
if __name__ == "__main__":
    tasks = ["A","B","A","B","B","B"]
    n = 2
    print(leastInterval(tasks, n)) # Output: 8

```

Python

```

class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        count = collections.Counter(tasks)
        max_freq = max(count.values())
        # Put the most frequent task in the slot first
        max_freq_task_copy = [max_freq - 1] * (n + 1)
        # Get the number of tasks with same frequency as max_freq, we'll append them after the
        # max_freq_task_copy
        max_freq = sum(value == max_freq for value in count.values())
        # max
        # The most frequent task is frequent enough to force some idle slots,
        # the most frequent task is not frequent enough to force idle slots
        #
        return max(max_freq_task_copy + [max_freq, len(tasks)])

```

Python

```

class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)

```

Python

Heap · LeetCode · By Pattern

— 1351 —

LeetCode

```

class Solution {
public:
    int leastInterval(vector<char>& tasks, int n) {
        vector<int> cnt(26);
        int x = 0;
        for (char c : tasks) {
            c -= 'A';
            ++cnt[c];
            x = max(x, cnt[c]);
        }
        int s = 0;
        for (int v : cnt) {
            s += v == x;
        }
        return max((int) tasks.size(), (x - 1) * (n + 1) + s);
    }
};

```

Java

```

class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] count = new int[26];
        for (final char task : tasks)
            ++count[task - 'A'];

        final int maxFreq = Arrays.stream(count).max().getAsInt();
        final int maxFreqTaskCopy = [maxFreq - 1] * (n + 1);
        // Get the number of tasks with the same frequency as maxFreq, we'll
        // append them after maxFreqTaskCopy
        final int sMaxFreq = (int) Arrays.stream(count).filter(c -> c == maxFreq).count();
        // max
        // the most frequent task is frequent enough to force some idle slots,
        // the most frequent task is not frequent enough to force idle slots
        //
        return Math.max(maxFreqTaskCopy + sMaxFreq, tasks.length);
    }
}

```

Java

Heap · LeetCode · By Pattern

— 1351 —

LeetCode

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","C","A","B","B","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A","A","B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- 1 <= tasks.length <= 104
- tasks[i] is an uppercase English letter.
- 0 <= n <= 100

>> Brute Force (Heap) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        # Brute Force (Heap) Approach - simulate the CPU schedule tick by tick
        from collections import Counter
        counts = Counter(tasks)
        time = 0
        while counts:
            # Each cycle, pick up to (n+1) most frequent tasks
            cycle = sorted(counts.keys(), key=lambda t: -counts[t])[:n+1]
            for i in cycle:
                counts[i] -= 1
            if counts[i] == 0:
                del counts[i]
            time += 1
            # If counts <= 0, then the task of the cycle
            time -= min(0, n + 1 - len(cycle))
        return time

```

Python

Python

Heap · LeetCode · By Pattern

— 1352 —

LeetCode

```

class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)

```

C++

C++

```

class Solution {
public:
    int leastInterval(vector<char>& tasks, int n) {
        if (n == 0)
            return tasks.size();

        vector<int> cnt(26);
        for (const char task : tasks)
            ++cnt[task - 'A'];

        count int maxFreq = ranges::max(cnt);
        // Put the most frequent task in the slot first
        count int maxFreqTaskCopy = (maxFreq - 1) * (n + 1);
        // Get the number of tasks with the same frequency as maxFreq, we'll
        // append them after maxFreqTaskCopy
        count int sMaxFreq = ranges::count(cnt, maxFreq);
        // max
        // the most frequent task is frequent enough to force some idle slots,
        // the most frequent task is not frequent enough to force idle slots
        //
        return max(maxFreqTaskCopy + sMaxFreq, static_cast<int>(tasks.size()));
    }
};

```

C++

```

class Solution {
public:
    int leastInterval(vector<char>& tasks, int n) {
        int x = 0;
        vector<int> cnt(26);
        for (char c : tasks) {
            c -= 'A';
            ++cnt[c];
            x = max(x, cnt[c]);
        }
        int s = 0;
        for (int v : cnt) {
            s += v == x;
        }
        return max((int) tasks.size(), (x - 1) * (n + 1) + s);
    }
};

```

C++

Heap · LeetCode · By Pattern

— 1354 —

LeetCode

```

class Solution {
public:
    int leastInterval(char[] tasks, int n) {
        int[] cnt = new int[26];
        int x = 0;
        for (char c : tasks) {
            c -= 'A';
            ++cnt[c];
            x = Math.max(x, cnt[c]);
        }
        int s = 0;
        for (int v : cnt) {
            s += v == x;
        }
        return Math.max(tasks.length, (x - 1) * (n + 1) + s);
    }
}

```

Java

```

public class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] cnt = new int[26];
        int x = 0;
        for (char c : tasks) {
            cnt[c - 'A']++;
            x = Math.Max(x, cnt[c - 'A']);
        }
        int s = 0;
        for (int v : cnt) {
            s += v == x ? 1 : 0;
        }
        return Math.Max(tasks.Length, (x - 1) * (n + 1) + s);
    }
}

```

Java

```

class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] cnt = new int[26];
        int x = 0;
        for (char c : tasks) {
            c -= 'A';
            ++cnt[c];
            x = Math.max(x, cnt[c]);
        }
        int s = 0;
        for (int v : cnt) {
            s += v == x;
        }
        return Math.max(tasks.length, (x - 1) * (n + 1) + s);
    }
}

```

Java

Heap · LeetCode · By Pattern

— 1356 —

LeetCode

```

public class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] cnt = new int[26];
        int x = 0;
        for(int i=0; i < tasks.length; i++) {
            cnt[i - 'A']++;
            x = Math.Max(x, cnt[i - 'A']);
        }
        int s = 0;
        for(int i=0; i < cnt.length; i++) {
            s = s >= 2 * n - 1 ? i : s;
        }
        return Math.Max(tasks.length, (s - 1) * (n + 1) + x);
    }
}

```

Go

```

func leastInterval(tasks []byte, n int) int {
    cnt := make([]int, 26)
    x := 0
    for _, c := range tasks {
        cnt[c-'A']++
        x = max(x, cnt[c-'A'])
    }
    s := 0
    for _, v := range cnt {
        if v == x {
            s++
        }
    }
    return max(len(tasks), (s-1)*(n+1)+x)
}

```

Go

```

func leastInterval(tasks []byte, n int) int {
    cnt := make([]int, 26)
    x := 0
    for _, c := range tasks {
        cnt[c-'A']++
        x = max(x, cnt[c-'A'])
    }
    s := 0
    for _, v := range cnt {
        if v == x {
            s++
        }
    }
    return max(len(tasks), (s-1)*(n+1)+x)
}

```

[7] Heap -- Pattern Solution (stored optimal)

Python

Heap · LeetCode · By Pattern

— 1357 —

LeetCode

```

# Python solution for Task Scheduler
from collections import Counter

def leastInterval(tasks, n):
    # Count the frequency of each task
    task_counts = Counter(tasks)
    # Find the maximum frequency among tasks
    max_freq = max(task_counts.values())
    # Count how many tasks have the maximum frequency
    max_count = sum(1 for count in task_counts.values() if count == max_freq)

    # Calculate the minimum intervals required based on the most frequent tasks
    part_count = max_freq - 1 # Number of parts between the most frequent task
    part_length = n - (max_count - 1) # Effective empty slots per part after placing max frequency tasks
    empty_slots = part_count * part_length # Total empty slots to fill with idle or other tasks
    available_tasks = len(tasks) - max_freq * max_count # Remaining tasks to fill the empty slots
    idles = max(0, empty_slots - available_tasks) # Remaining idle slots if available tasks are not enough
    # The total intervals is the sum of all tasks and idle slots inserted
    return len(tasks) + idles

# Example usage:
if __name__ == "__main__":
    tasks = ["A","A","A","B","B","B"]
    n = 2
    print(leastInterval(tasks, n)) # Output: 8

```

C++

```

// C++ solution for Task Scheduler
#include <vector>
#include <unordered_map>
#include <algorithm>
using namespace std;

int leastInterval(vector<char>& tasks, int n) {
    // Count frequency of each task using an unordered map
    unordered_map<char, int> freq;
    for (char task : tasks) {
        freq[task]++;
    }

    // Determine the maximum frequency among the tasks
    int maxFreq = 0;
    for (auto& p : freq) {
        maxFreq = max(maxFreq, p.second);
    }

    // Count how many tasks have the maximum frequency
    int maxCount = 0;
    for (auto& p : freq) {
        if (p.second == maxFreq) {
            maxCount++;
        }
    }
}

```

Heap · LeetCode · By Pattern

— 1358 —

LeetCode

```

    }

    // Calculate number of parts (slots between the most frequent tasks)
    int partCount = maxFreq - 1;
    int partLength = n - (maxCount - 1);
    int emptySlots = partCount * partLength;
    int availableTasks = tasks.size() - maxFreq * maxCount;
    int idles = max(0, emptySlots - availableTasks);

    // Total intervals is the sum of tasks and idles
    return tasks.size() + idles;
}

int main() {
    vector<char> tasks = {"A","A","A","B","B","B"};
    int n = 2;
    cout << leastInterval(tasks, n) << endl; // Output: 8
    return 0;
}

```

#658

MEDIUM

Find K Closest Elements

LeetCode · Simplest & Most Efficient Solution · Editorial

Array · Binary Search · Sorting · Heap · Two Pointers

List · Grind 169

Problem:

Given a sorted integer array arr, two integers k and x, return the k closest integers to x in the array. The result should also be sorted in ascending order.

An integer a is closer to x than an integer b if:

- $|a - x| < |b - x|$, or
- $|a - x| == |b - x|$ and $a < b$

Example 1:

Input: arr = [1,2,3,4,5], k = 4, x = 3

Output: [1,2,3,4]

Example 2:

Input: arr = [1,1,2,3,4,5], k = 4, x = -1

Output: [1,1,2,3]

Constraints:

- $1 \leq k \leq arr.length$
- $1 \leq arr.length \leq 10^4$
- arr is sorted in ascending order.
- $-10^4 \leq arr[i], x \leq 10^4$

>> Brute Force (Heap) Interview Prep

Heap · LeetCode · By Pattern

— 1359 —

LeetCode

```

Python
# Brute Force Approach
class Solution:
    def findClosestElements(self, arr, k, x):
        # Simple Brute Force: log n - sort repeatedly instead of heap
        data = list(arr)
        result = []
        for _ in range(k):
            data.sort()
            result.append(data.pop(0))
        return result

```

Python

```

# Python: To find k closest elements to x in a sorted array
def findClosestElements(arr, k, x):
    n = len(arr)
    # Initialize the binary search bounds for the starting index of the window
    left, right = 0, n - k
    # Binary search to find the best starting index
    while left < right:
        mid = (left + right) // 2
        # Compare the distance from x to arr[mid] and arr[mid+k]
        # If arr[mid] is further from x than arr[mid+k], move window to the right
        if x - arr[mid] > arr[mid + k] - x:
            left = mid + 1
        else:
            # Otherwise, continue searching towards the left
            right = mid
    # Return the k elements starting from index 'left'
    return arr[left:left + k]

# Example usage:
arr = [1,2,3,4,5], k = 4, x = 3
print(findClosestElements(arr, k, x)) # Output: [1,2,3,4]

```

Python

```

class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        l = 0
        r = len(arr) - k
        while l < r:
            m = (l + r) // 2
            if x - arr[m] > arr[m + k] - x:
                l = m + 1
            else:
                r = m
        return arr[l:l + k]

```

Python

```

class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        arr.sort(key=lambda v: abs(v - x))
        return sorted(arr[:k])

```

Heap · LeetCode · By Pattern

— 1360 —

LeetCode

```

Python
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        left, right = 0, len(arr) - k
        while left < right:
            mid = (left + right) // 2
            if x - arr[mid] > arr[mid + k] - x:
                right = mid
            else:
                left = mid + 1
        return arr[left : left + k]

```

C++

C++

```

class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {
        int l = 0;
        int r = arr.size() - k;
        while (l < r) {
            const int m = (l + r) / 2;
            if (x - arr[m] > arr[m + k] - x)
                l = m + 1;
            else
                r = m;
        }
        return {arr.begin() + l, arr.begin() + l + k};
    }
};

```

C++

```

int target;

class Solution {
public:
    static bool comp(int a, int b) {
        int x = abs(a - target) - abs(b - target);
        return x > 0 ? a < b : x < 0;
    }

    vector<int> findClosestElements(vector<int>& arr, int k, int x) {
        target = x;
        sort(arr.begin(), arr.end(), comp);
        vector<int> ans(arr.begin(), arr.begin() + k);
        sort(ans.begin(), ans.end());
        return ans;
    }
};

```

C++

Heap · LeetCode · By Pattern

— 1361 —

LeetCode

```

class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {
        int l = 0, r = arr.size();
        while (r - l > k) {
            if (x - arr[l] > arr[r - 1] - x) {
                ++l;
            } else {
                ++r;
            }
        }
        return vector<int>(arr.begin() + l, arr.begin() + r);
    }
};

```

C++

```

class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {
        int left = 0, right = arr.size() - k;
        while (left < right) {
            int mid = (left + right) / 2;
            if (x - arr[mid] > arr[mid + k] - x)
                left = mid + 1;
            else
                right = mid;
        }
        return vector<int>(arr.begin() + left, arr.begin() + left + k);
    }
};

```

Java

Java

```

class Solution {
    public List<Integer> findClosestElements(int[] arr, int k, int x) {
        int l = 0;
        int r = arr.length - k;
        while (l < r) {
            final int m = (l + r) / 2;
            if (x - arr[m] > arr[m + k] - x)
                l = m + 1;
            else
                r = m;
        }
        return Arrays.stream(arr, l, l + k).boxed().collect(Collectors.toList());
    }
}

```

Java

Heap · LeetCode · By Pattern

— 1362 —

LeetCode

```
class Solution {
public: List<Integer> findClosestElements(int[] arr, int k, int x) {
    List<Integer> ans = ArrayList<Integer>()
    .addAll()
    .sorted((a, b) => {
        int v = Math.abs(a - x) - Math.abs(b - x);
        return v == 0 ? a - b : v;
    })
    .subList(0, k);
    Collections.sort(ans);
    return ans;
}
}

Java

class Solution {
public: List<Integer> findClosestElements(int[] arr, int k, int x) {
    int left = 0;
    int right = arr.length - k;
    while (left < right) {
        int mid = (left + right) / 2;
        if (x - arr[mid] < arr[mid + k] - x) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    List<Integer> ans = new ArrayList<>();
    for (int i = left; i < left + k; ++i) {
        ans.add(arr[i]);
    }
    return ans;
}
}

TypeScript

TypeScript

function findClosestElements(arr: number[], k: number, x: number): number[] {
    let l = 0;
    let r = arr.length;
    while (r - l > k) {
        let m = Math.floor((r + l) / 2);
        if (x - arr[m] < arr[m + k] - x) {
            r = m;
        } else {
            l = m + 1;
        }
    }
    return arr.slice(l, r);
}

TypeScript
```

```
function findClosestElements(arr: number[], k: number, x: number): number[] {
    let left = 0;
    let right = arr.length - k;
    while (left < right) {
        const mid = (left + right) / 2;
        if (x - arr[mid] < arr[mid + k] - x) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return arr.slice(left, left + k);
}

Go

Go

func findClosestElements(arr []int, k int, x int) []int {
    sort.Slice(arr, func(i, j int) bool {
        v := abs(arr[i]-x) - abs(arr[j]-x)
        if v == 0 {
            return arr[i] < arr[j]
        }
        return v < 0
    })
    ans := arr[k:]
    sort.Ints(ans)
    return ans
}

func abs(x int) int {
    if x <= 0 {
        return -x
    }
    return x
}

Go

Go

func findClosestElements(arr []int, k int, x int) []int {
    l, r := 0, len(arr)
    for r-l > k {
        if arr[(l+r)/2] - arr[r-k] < x {
            l = r-k
        } else {
            r = (l+r)/2
        }
    }
    return arr[l:r]
}

Go
```

```
func findClosestElements(arr []int, k int, x int) []int {
    left, right = 0, len(arr)-k
    for left < right {
        mid := (left + right) / 2
        if arr[mid] - arr[mid+k] < x - arr[mid+k] {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return arr[left : left+k]
}

Go

func findClosestElements(arr []int, k int, x int) []int {
    left, right = 0, len(arr)-k
    for left < right {
        mid := (left + right) / 2
        if arr[mid] - arr[mid+k] < x - arr[mid+k] {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return arr[left : left+k]
}

Rust

Rust

impl Solution {
    pub fn find_closest_elements(arr: Vec<i32>, k: i32, x: i32) -> Vec<i32> {
        let n = arr.len();
        let mut l = 0;
        let mut r = n;
        while r - l > k {
            let m = (l + r) / 2;
            if x - arr[m] < arr[m+k] - x {
                r = m;
            } else {
                l = m + 1;
            }
        }
        arr[l..r].to_vec()
    }
}

Rust
```

```
impl Solution {
    pub fn find_closest_elements(arr: Vec<i32>, k: i32, x: i32) -> Vec<i32> {
        let n = arr.len();
        let mut l = 0;
        let mut r = n;
        while left < right {
            let mid = (left + right) / 2;
            if x - arr[mid] < arr[mid + k] - x {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        arr[left..left + k].to_vec()
    }
}

[i] Heap — Pattern Solution (stored optimal)

Python

# Function to find k closest elements to x in a sorted array
def findClosestElements(arr, k, x):
    n = len(arr)
    left, right = 0, n - k
    # Binary search to find the best starting index
    while left < right:
        mid = (left + right) // 2
        # Compare the distance from x to arr[mid] and arr[mid+k]
        # If arr[mid] is further from x than arr[mid+k], move window to the right
        if x - arr[mid] > arr[mid + k] - x:
            left = mid + 1
        else:
            # Otherwise, continue searching towards the left
            right = mid
    # Return the k elements starting from index 'left'
    return arr[left:left + k]

# Example usage:
# arr = [1,2,3,4,5], k = 4, x = 3
# print(findClosestElements(arr, 4, 3)) # Output: [1,2,3,4]

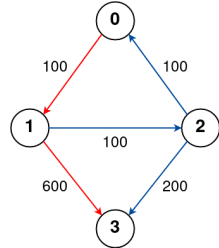
C++
```

```
// C++ solution using binary search with comments
#include <vector>
#include <algorithm>
using namespace std;

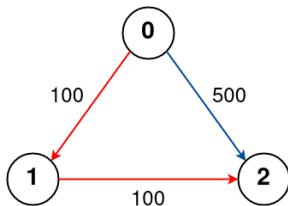
class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {
        int n = arr.size();
        // Initialize binary search bounds for the starting index of window
        int left = 0, right = n - k;
        while (left < right) {
            int mid = (left + right) / 2;
            // Compare distances: if element at mid is less favorable, move window right
            if (x - arr[mid] > arr[mid + k] - x) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        // Extract the subarray starting from the found index
        return vector<int>(arr.begin() + left, arr.begin() + left + k);
    }
};

#787 MEDIUM
Cheapest Flights Within K Stops
LeetCode · SimplifyLaw · WalkCC · LeetCode · Editorial
Dynamic Programming · DFS · BFS · Graph · Heap
Like · 619 · 159

Problem:
There are n cities connected by some number of flights. You are given an array flights where
flights[i] = [from, to, price] indicates that there is a flight from city from to city to with cost
price.
You are also given three integers src, dst, and k, return the cheapest price from src to dst with at
most k stops. If there is no such route, return -1.
Example 1:
```

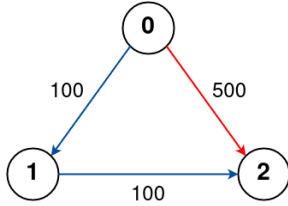


Input: n = 4, flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src = 0, dst = 3, k = 1
Output: 700
Explanation:
The graph is shown above.
The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost 100 + 600 = 700.
Note that the path through cities [0,1,2,3] is cheaper but is invalid because it uses 2 stops.
Example 2:



Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 1
Output: 200
Explanation:
The graph is shown above.
The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost 100 + 100 = 200.

Example 3:



Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 0
Output: 500
Explanation:
The graph is shown above.
The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

- Constraints:
- 2 <= n <= 100
 - 0 <= flights.length <= (n * (n - 1) / 2)
 - flights[i].length == 3
 - 0 <= fromi, toi < n
 - fromi != toi
 - 1 <= pricei <= 104
 - There will not be any multiple flights between two cities.
 - 0 <= src, dst, k < n
 - src != dst

>> Brute Force [Heap] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def findCheapestPrice(self, n, flights, src, dst, k):
        # Brute Force DFS (Rec) - not trying all paths with at most k stops
        from collections import defaultdict
        graph = defaultdict(list)
        for u, v, w in flights:
            graph[u].append((v, w))
        self.res = float('inf')
        def dfs(node, steps, cost):
            if node == dst:
                self.res = min(self.res, cost); return
            if steps == k: return
            for nei, price in graph[node]:
                if cost + price < self.res: # prune only on cost
                    dfs(nei, steps+1, cost+price)
        dfs(src, 0, 0)
        return self.res if self.res < float('inf') else -1
```

Python

Python

```
import heapq

def findCheapestPrice(self, flights, src, dst, k):
    # Build graph: each city maps to a list of (neighboring city, flight price)
    graph = {}
    for i in range(n):
        graph[i] = []
    for from_city, to_city, price in flights:
        graph[from_city].append((to_city, price))

    # Priority queue: (cost, current_city, stops so far)
    heap = [(0, src, 0)]

    # Dictionary to store the best cost to reach a city with a given stop count
    best = {}

    while heap:
        cost, city, steps = heapq.heappop(heap)
        # Success case: if destination is reached
        if city == dst:
            return cost

        # If the current number of stops exceeds k, skip further processing
        if steps > k:
            continue

        # Expand the neighboring flights
        for neighbor, price in graph[city]:
            new_cost = cost + price
            # Only proceed if this route is cheaper or not yet visited for this many stops
            if (neighbor, steps + 1) not in best or new_cost < best[neighbor, steps + 1]:
                best[(neighbor, steps + 1)] = new_cost
                heapq.heappush(heap, (new_cost, neighbor, steps + 1))

    return -1

# Example usage:
print(findCheapestPrice(4, [[0,1,100],[1,2,100],[2,0,100],[1,3,400],[2,3,200]], 0, 3, 1))
```

Python

```
class Solution:
    def findCheapestPrice(self, n, flights, src, dst, k):
        self.n = n
        self.flights = flights
        self.src = src
        self.dst = dst
        self.k = k

        graph = [[] for _ in range(n)]
        for u, v, w in flights:
            graph[u].append((v, w))

        return self.dijkstra(graph, src, dst, k)

    def dijkstra(self):
        graph = list(list(zip(self.src, self.dst)))
        self.src = self.dst
        self.k = k
        dist = [math.inf] * (n + 1)
        for _ in range(self.k + 1):
            dist[self.src] = 0
            minHeap = [(dist[self.src], self.src, 0)]
            while minHeap:
                d, u, steps = heapq.heappop(minHeap)
                if u == self.dst:
                    return d
                if steps == 0 or d < dist[u]:
                    continue
                for v, w in graph[u]:
                    if d + w < dist[v]:
                        dist[v] = d + w
                        heapq.heappush(minHeap, (dist[v], v, steps + 1))
            return -1
```

Python

```
class Solution:
    def findCheapestPrice(self, n, flights, src, dst, k):
        self.n = n
        self.flights = flights
        self.src = src
        self.dst = dst
        self.k = k

        INF = float('inf')
        dist = [INF] * n
        dist[src] = 0

        for _ in range(k + 1):
            backup = dist.copy()
            for i, p in flights:
                if dist[i] < backup[i]:
                    dist[i] = min(dist[i], backup[i] + p)
            return -1 if dist[dst] == INF else dist[dst]
```

Python

```
class Solution:
    def findCheapestPrice(self, n, flights, src, dst, k):
        self.n = n
        self.flights = flights
        self.src = src
        self.dst = dst
        self.k = k

        INF = float('inf')
        dist = [INF] * n
        dist[src] = 0

        for _ in range(k + 1):
            backup = dist.copy()
            for i, p in flights:
                if dist[i] < backup[i]:
                    dist[i] = min(dist[i], backup[i] + p)
            return -1 if dist[dst] == INF else dist[dst]
```

C++

C++

```
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
        const int INF = 0x3f3f3f3f;
        vector<int> dist(n, INF);
        vector<int> backup;
        dist[src] = 0;
        for (int i = 0; i < k + 1; ++i) {
            backup = dist;
            for (auto& f : flights) {
                int f = f[0], t = f[1], p = f[2];
                dist[t] = min(dist[t], backup[f] + p);
            }
            return dist[dst] == INF ? -1 : dist[dst];
        }
    }
};
```

C++

```
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
        const int INF = 0x3f3f3f3f;
        vector<int> dist(n, INF);
        vector<int> backup;
        dist[src] = 0;
        for (int i = 0; i < k + 1; ++i) {
            backup = dist;
            for (auto& f : flights) {
                int f = f[0], t = f[1], p = f[2];
                dist[t] = min(dist[t], backup[f] + p);
            }
            return dist[dst] == INF ? -1 : dist[dst];
        }
    }
};
```

Java

Java

```
class Solution {
    private static final int INF = 0x3f3f3f3f;

    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        int[] dist = new int[n];
        int[] backup = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int i = 0; i < k + 1; ++i) {
            System.arraycopy(dist, 0, backup, 0, n);
            for (int[] f : flights) {
                int f = f[0], t = f[1], p = f[2];
                dist[t] = Math.min(dist[t], backup[f] + p);
            }
            return dist[dst] == INF ? -1 : dist[dst];
        }
    }
}
```

Java

```
class Solution {
    private static final int INF = 0x3f3f3f3f;

    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        int[] dist = new int[n];
        int[] backup = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int i = 0; i < k + 1; ++i) {
            System.arraycopy(dist, 0, backup, 0, n);
            for (int[] f : flights) {
                int f = f[0], t = f[1], p = f[2];
                dist[t] = Math.min(dist[t], backup[f] + p);
            }
            return dist[dst] == INF ? -1 : dist[dst];
        }
    }
}
```

Go

Go

```
func findCheapestPrice(int, flights [][]int, src int, dst int, k int) int {
    const inf = 0x3f3f3f3f
    dist := make([]int, n)
    backup := make([]int, n)
    for i := range dist {
        dist[i] = inf
    }
    dist[src] = 0
    for i := 0; i < k+1; i++ {
        copy(backup, dist)
        for u, v := range flights {
            f, t, p := u[0], u[1], u[2]
            dist[t] = min(dist[t], backup[u]+p)
        }
    }
    if dist[dst] == inf {
        return -1
    }
    return dist[dst]
}
```

Go

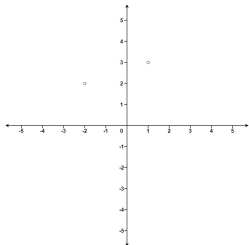
```
func findCheapestPrice(int, flights [][]int, src int, dst int, k int) int {
    const inf = 0x3f3f3f3f
    dist := make([]int, n)
    backup := make([]int, n)
    for i := range dist {
        dist[i] = inf
    }
    dist[src] = 0
    for i := 0; i < k+1; i++ {
        copy(backup, dist)
        for u, v := range flights {
            f, t, p := u[0], u[1], u[2]
            dist[t] = min(dist[t], backup[u]+p)
        }
    }
    if dist[dst] == inf {
        return -1
    }
    return dist[dst]
}
```

[7] Heap — Pattern Solution (matched from community answers)
Python

Heap · LeetCode — By Pattern

— 1375 —

LeetCode



Input: points = [[1,3],[-2,2]], k = 1
Output: [[-2,2]]
Explanation:
The distance between (1, 3) and the origin is sqrt(10).
The distance between (-2, 2) and the origin is sqrt(8).
Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin.
We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].
Example 2:
Input: points = [[3,3],[5,-1],[-2,4]], k = 2
Output: [[3,3],[-2,4]]
Explanation: The answer [[-2,4],[3,3]] would also be accepted.

Constraints:
• 1 <= k <= points.length <= 104
• -104 <= xi, yi <= 104

>> Brute Force [Heap] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        # Brute Force: Sort points by their Euclidean distance
        points.sort(key=lambda p: p[0]**2 + p[1]**2)
        return points[:k]
```

Python

Python

Heap · LeetCode — By Pattern

— 1377 —

LeetCode

```
class Solution {
public:
    vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
        vector<vector<int>> ans;
        auto compare = [&](const vector<int>& a, const vector<int>& b) {
            return squaredist(a) < squaredist(b);
        };
        priority_queue<vector<int>, vector<vector<int>>, decltype(compare)> maxHeap(
            compare);
        for (const vector<int>& point : points) {
            maxHeap.push(point);
            if (maxHeap.size() > k)
                maxHeap.pop();
        }
        while (!maxHeap.empty())
            ans.push_back(maxHeap.top());
            maxHeap.pop();
        return ans;
    };
private:
    int squaredist(const vector<int>& p) {
        return p[0] * p[0] + p[1] * p[1];
    }
};
```

C++

```
class Solution {
public:
    vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
        sort(points.begin(), points.end());
        for (const vector<int>& p: points) {
            return hypot(p[0], p[1]) < hypot(p[0], p[1]);
        }
        return vector<vector<int>>(points.begin(), points.end() + k);
    };
};
```

C++

```
class Solution {
public:
    vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
        sort(points.begin(), points.end());
        for (const vector<int>& a, const vector<int>& b) {
            return a[0] * a[0] + a[1] * a[1] < b[0] * b[0] + b[1] * b[1];
        }
        return vector<vector<int>>(points.begin(), points.end() + k);
    };
};
```

Java

Java

Heap · LeetCode — By Pattern

— 1379 —

LeetCode

```
import heapq

def findCheapestPrice(int, flights, src, dst, k):
    # Build graph: each city maps to a list of (neighboring city, flight price)
    graph = {}
    for i in range(n):
        for j in range(n):
            if flights[i][j] != 0:
                graph[i].append((j, flights[i][j]))

    # Priority queue: (total cost, current city, stops so far)
    heap = [(0, src, 0)]

    # Dictionary to store the best cost to reach a city with a given stop count
    best = {}

    while heap:
        cost, city, stops = heapq.heappop(heap)
        # Return cost if destination is reached
        if city == dst:
            return cost
        # If the current number of stops exceeds k, skip further processing
        if stops > k:
            continue
        # Expand the neighboring flights
        for neighbor, price in graph[city]:
            new_cost = cost + price
            # Only process if this route is cheaper or not yet visited for this many stops
            if (neighbor, stops + 1) not in best or new_cost < best[neighbor, stops + 1]:
                best[neighbor, stops + 1] = new_cost
                heapq.heappush(heap, (new_cost, neighbor, stops + 1))

    return -1

# Example usage:
print(findCheapestPrice(4, [[0,1,100],[1,2,100],[2,0,100],[1,3,400],[2,3,200]], 0, 3, 1))
```

#973

MEDIUM

K Closest Points to Origin

LeetCode · SimpleHash · WUJIE · LeetCode · Editorial

Array · Math · Divide and Conquer · Sorting · Heap

List · Grid · 169

Problem:

Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{x_1^2 + y_1^2 - x_2^2 - y_2^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:

Heap · LeetCode — By Pattern

— 1376 —

LeetCode

```
# Python Solution
def kClosest(points, k):
    # Sort the points based on their squared Euclidean distance from the origin.
    points.sort(key=lambda point: point[0]**2 + point[1]**2)
    # Return the first k points from the sorted list.
    return points[:k]

# Example usage:
points = [[1,3], [-2,2]]
k = 1
print(kClosest(points, k))
```

Python

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        maxHeap = []
        for x, y in points:
            heapq.heappush(maxHeap, (-x**2 - y**2, [x, y]))
            if len(maxHeap) > k:
                heapq.heappop(maxHeap)
        return [pair[1] for pair in maxHeap]
```

Python

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: hypot(p[0], p[1]))
        return points[:k]
```

Python

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: p[0]**2 + p[1]**2)
        return points[:k]
```

C++

C++

Heap · LeetCode — By Pattern

— 1378 —

LeetCode

```
class Solution {
public:
    int[][] kClosest(int[][] points, int k) {
        int[] ans = new int[k][2];
        Queue<int> maxHeap = new PriorityQueue<>((Comparator.comparingInt(a) -> -squaredist(a)));
        for (int[] point : points) {
            maxHeap.offer(point);
            if (maxHeap.size() > k)
                maxHeap.poll();
        }
        int i = 0;
        while (!maxHeap.isEmpty())
            ans[i++] = maxHeap.poll();
        return ans;
    };
private:
    int squaredist(int[] p) {
        return p[0] * p[0] + p[1] * p[1];
    }
};
```

Java

```
class Solution {
    public int[][] kClosest(int[][] points, int k) {
        Arrays.sort(
            points, (a, b) -> Math.hypot(a[0], a[1]) - Math.hypot(b[0], b[1]) > 0 ? 1 : -1);
        return Arrays.copyOfRange(points, 0, k);
    }
}
```

Java

```
class Solution {
    public int[][] kClosest(int[][] points, int k) {
        PriorityQueue<int> maxHeap = new PriorityQueue<>((a, b) -> b[0]**2 + b[1]**2 - a[0]**2 - a[1]**2);
        for (int i = 0; i < points.length; i++) {
            int x = points[i][0], y = points[i][1];
            maxHeap.offer(new int[] {x**2 + y**2, i});
            if (maxHeap.size() > k) {
                maxHeap.poll();
            }
        }
        int[] ans = new int[k][2];
        for (int i = 0; i < k; i++) {
            ans[i] = points[maxHeap.poll()][1];
        }
        return ans;
    }
}
```

Java

Heap · LeetCode — By Pattern

— 1380 —

LeetCode

```

class Solution {
public: int[][] kClosest(int[][] points, int k) {
    Arrays.sort(points, (a, b) -> {
        int d1 = a[0] * a[0] + a[1] * a[1];
        int d2 = b[0] * b[0] + b[1] * b[1];
        return d1 - d2;
    });
    return Arrays.copyOfRange(points, 0, k);
}
}

TypeScript
TypeScript
function kClosest(points: number[][][], k: number): number[][] {
    points.sort((a, b) => Math.hypot(a[0], a[1]) - Math.hypot(b[0], b[1]));
    return points.slice(0, k);
}

TypeScript
function kClosest(points: number[][][], k: number): number[][] {
    const map = new Map<number, number>();
    for (const [x, y] of points) {
        const dist = x * x + y * y;
        const current = map.get(x * y * dist);
        if (current > k) {
            map.delete(x * y * dist);
        }
        map.set(x * y * dist, 1);
    }
    return map.keys().map(entry => entry.point);
}

TypeScript
function kClosest(points: number[][][], k: number): number[][] {
    const dist = points.map((x, y) => x * x + y * y);
    let [i, j] = [0, Math.max(...dist)];
    while (i < j) {
        const mid = (i + j) >> 1;
        let cnt = 0;
        for (const d of dist) {
            if (d <= mid) {
                ++cnt;
            }
        }
        if (cnt == k) {
            i = mid;
        } else {
            j = mid + 1;
        }
    }
    return points.filter((_, i) => dist[i] <= i);
}

```

Heap · LeetCode · By Pattern — 1381 — LeetCode

```

function kClosest(points: number[][][], k: number): number[][] {
    return points.sort((a, b) => a[0] * a[0] + a[1] * a[1] - (b[0] * b[0] + b[1] * b[1])).slice(0, k);
}

Go
Go
func kClosest(points [][]int, k int) [][]int {
    sort.Slice(points, func(i, j) bool {
        return math.Hypot(float64(points[i][0]), float64(points[i][1])) <
            math.Hypot(float64(points[j][0]), float64(points[j][1]))
    })
    return points[:k]
}

Go
func kClosest(points [][]int, k int) (ans [][]int) {
    n := len(points)
    dist := make([]int, n)
    for i, p := range points {
        dist[i] = p[0]*p[0] + p[1]*p[1]
        r := math.Sqrt(dist[i])
    }
    for i, r := range dist {
        mid := (1 + r) >> 1
        cnt := 0
        for j, d := range dist {
            if d <= mid {
                cnt++
            }
        }
        if cnt == k {
            r = mid
        } else {
            r = mid + 1
        }
    }
    for i, p := range points {
        if dist[i] <= r {
            ans = append(ans, p)
        }
    }
    return
}

Go
func kClosest(points [][]int, k int) [][]int {
    sort.Slice(points, func(i, j) bool {
        a, b := points[i], points[j]
        return a[0]*a[0]+a[1]*a[1] < b[0]*b[0]+b[1]*b[1]
    })
    return points[:k]
}

```

Heap · LeetCode · By Pattern — 1382 — LeetCode

```

// Rust
impl Solution {
    pub fn k_closest(points: Vec<Vec<i32>>, k: i32) -> Vec<Vec<i32>> {
        points.sort_by(|a, b| {
            let dist_a = f64::hypot(a[0] as f64, a[1] as f64);
            let dist_b = f64::hypot(b[0] as f64, b[1] as f64);
            dist_a.partial_cmp(&dist_b).unwrap()
        });
        points.into_iter().take(k as usize).collect()
    }
}

// Rust
impl Solution {
    pub fn k_closest(points: Vec<Vec<i32>>, k: i32) -> Vec<Vec<i32>> {
        points.sort_unstable_by(|a, b| {
            (a[0].pow(2) + a[1].pow(2)).cmp(&(b[0].pow(2) + b[1].pow(2)))
        });
        points[0..k as usize].to_vec()
    }
}

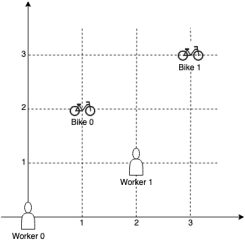
// [7] Heap — Pattern Solution (matched from community answers)
Python
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        minHeap = []
        for x, y in points:
            heap.heappush(minHeap, (-x*x - y*y, [x, y]))
            if len(minHeap) > k:
                heap.heappop(minHeap)
        return [pair[1] for pair in minHeap]

```

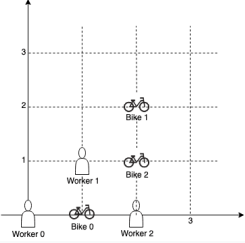
#1057 MEDIUM
Campus Bikes
LeetCode · SimplyLeetCode · WalkCC · LeetCode · Editorial
Array · Greedy · Sorting · Heap
Like · Premium 25

Problem:
On a campus represented on the X-Y plane, there are n workers and m bikes, with n <= m.
You are given an array workers of length n where workers[i] = [xi, yi] is the position of the ith worker. You are also given an array bikes of length m where bikes[j] = [xj, yj] is the position of the jth bike. All the given positions are unique.
Assign a bike to each worker. Among the available bikes and workers, we choose the (workeri, bikej) pair with the shortest Manhattan distance between each other and assign the bike to that

Heap · LeetCode · By Pattern — 1383 — LeetCode

worker.
If there are multiple (workeri, bikej) pairs with the same shortest Manhattan distance, we choose the pair with the smallest worker index. If there are multiple ways to do that, we choose the pair with the smallest bike index. Repeat this process until there are no available workers.
Return an array answer of length n, where answer[i] is the index (0-indexed) of the bike that the ith worker is assigned to.
The Manhattan distance between two points p1 and p2 is Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|.
Example 1:

Input: workers = [[0,0],[2,1]], bikes = [[1,2],[3,3]]
Output: [1,0]
Explanation: Worker 1 grabs Bike 0 as they are closest (without ties), and Worker 0 is assigned Bike 1. So the output is [1, 0].
Example 2:

Heap · LeetCode · By Pattern — 1384 — LeetCode


Input: workers = [[0,0],[1,1],[2,1]], bikes = [[1,2],[2,2]]
Output: [0,2,1]
Explanation: Worker 0 grabs Bike 0 at first. Worker 1 and Worker 2 share the same distance to Bike 2, thus Worker 1 is assigned to Bike 2, and Worker 2 will take Bike 1. So the output is [0,2,1].
Constraints:
• n == workers.length
• m == bikes.length
• 1 <= n <= m <= 1000
• workers[i].length == bikes[j].length == 2
• 0 <= xi, yi < 1000
• 0 <= xj, yj < 1000
• All worker and bike locations are unique.

>> Brute Force [Heap] Interview Prep
Python
Brute Force Approach
class Solution:
 def assign_bikes(self, workers, bikes):
 # From Task 01, we know that we need to repeatedly instead of heap
 data = list(workers)
 result = []
 for i in range(n):
 data.sort()
 result.append(data.pop(0))
 return result

Heap · LeetCode · By Pattern — 1385 — LeetCode

```

Python
Python
# Python solution
def assign_bikes(workers, bikes):
    # list to hold all possible (distance, worker, bike) tuples
    pairs = []
    for worker_index, (wx, wy) in enumerate(workers):
        for bike_index, (bx, by) in enumerate(bikes):
            # Calculate Manhattan distance between worker and bike
            distance = abs(wx - bx) + abs(wy - by)
            pairs.append((distance, worker_index, bike_index))
    # Sort pairs based on distance, then worker index, then bike index
    pairs.sort()
    n = len(workers)
    assigned_workers = [False] * n
    assigned_bikes = [False] * len(bikes)
    result = [-1] * n
    # Iterate over the sorted pairs and assign bikes
    for distance, worker_index, bike_index in pairs:
        if not assigned_workers[worker_index] and not assigned_bikes[bike_index]:
            result[worker_index] = bike_index
            assigned_workers[worker_index] = True
            assigned_bikes[bike_index] = True
    return result
# Example usage
workers = [[0,0],[2,1]]
bikes = [[1,2],[3,3]]
print(assign_bikes(workers, bikes)) # Output: [1, 0]

```

Python

Heap · LeetCode · By Pattern — 1386 — LeetCode

```

class Solution:
    def assignBikes(self):
        self, workers: List[List[int]], bikes: List[List[int]]
        ) -> List[int]:
            ans = [-1] * len(workers)
            usedBikes = [False] * len(bikes)
            # buckets[i][j] = [x, y], where x is dist(workers[i], bikes[j])
            buckets = [[[] for _ in range(2001)]

            def dist(x1: int, y1: int, x2: int, y2: int) -> int:
                return abs(x1 - x2) + abs(y1 - y2)

            for i, worker in enumerate(workers):
                for j, bike in enumerate(bikes):
                    buckets[dist(worker, bike)].append((i, j))

            for k in range(2001):
                for i, j in buckets[k]:
                    if ans[i] == -1 and not usedBikes[j]:
                        ans[i] = j
                        usedBikes[j] = True
            return ans

```

Python

```

class Solution:
    def assignBikes(self):
        self, workers: List[List[int]], bikes: List[List[int]]
        ) -> List[int]:
            n, m = len(workers), len(bikes)
            arr = []
            for i, j in product(range(n), range(m)):
                dist = abs(workers[i][0] - bikes[j][0]) + abs(workers[i][1] - bikes[j][1])
                arr.append((dist, i, j))
            arr.sort()
            vis1 = [False] * n
            vis2 = [False] * m
            ans = [0] * n
            for i, j in arr:
                if not vis1[i] and not vis2[j]:
                    vis1[i] = vis2[j] = True
                    ans[i] = j
            return ans

```

Python

Heap · LeetCode — By Pattern

— 1387 —

LeetCode

```

class Solution:
    def assignBikes(self):
        self, workers: List[List[int]], bikes: List[List[int]]
        ) -> List[int]:
            n, m = len(workers), len(bikes)
            arr = []
            for i, j in product(range(n), range(m)):
                dist = abs(workers[i][0] - bikes[j][0]) + abs(workers[i][1] - bikes[j][1])
                arr.append((dist, i, j))
            arr.sort()
            vis1 = [False] * n
            vis2 = [False] * m
            ans = [0] * n
            for i, j in arr:
                if not vis1[i] and not vis2[j]:
                    vis1[i] = vis2[j] = True
                    ans[i] = j
            return ans

```

C++

```

class Solution {
public:
    vector<int> assignBikes(vector<vector<int>>& workers, vector<vector<int>>& bikes) {
        int n = workers.size(), m = bikes.size();
        vector<tuple<int, int, int>> arr(n * m);
        for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) {
            int dist = abs(workers[i][0] - bikes[j][0]) + abs(workers[i][1] - bikes[j][1]);
            arr[i * m + j] = {dist, i, j};
        }
        sort(arr.begin(), arr.end());
        vector<bool> vis1(n), vis2(m);
        vector<int> ans(n);
        for (auto [d, i, j] : arr) {
            if (!vis1[i] && !vis2[j]) {
                vis1[i] = true;
                vis2[j] = true;
                ans[i] = j;
            }
        }
        return ans;
    }
};

```

Java

Java

Heap · LeetCode — By Pattern

— 1388 —

LeetCode

```

class Solution {
    public int[] assignBikes(int[][] workers, int[][] bikes) {
        int n = workers.length, m = bikes.length;
        int[] arr = new int[n * m];
        for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) {
            int dist = Math.abs(workers[i][0] - bikes[j][0]) + Math.abs(workers[i][1] - bikes[j][1]);
            arr[i * m + j] = dist, i, j;
        }
        Arrays.sort(arr, (a, b) -> {
            if (a[0] < b[0]) {
                return a[0] - b[0];
            }
            if (a[1] < b[1]) {
                return a[1] - b[1];
            }
            return a[2] - b[2];
        });
        boolean[] vis1 = new boolean[n];
        boolean[] vis2 = new boolean[m];
        int[] ans = new int[n];
        for (var a : arr) {
            int i = a[1], j = a[2];
            if (!vis1[i] && !vis2[j]) {
                vis1[i] = true;
                vis2[j] = true;
                ans[i] = j;
            }
        }
        return ans;
    }
}

```

Go

Go

Heap · LeetCode — By Pattern

— 1389 —

LeetCode

```

func assignBikes(workers [][]int, bikes [][]int) []int {
    n, m := len(workers), len(bikes)
    type tuple struct {
        d, i, j int
    }
    arr := make([]tuple, n*m)
    for i, k := 0; k < n; k++ {
        for j := 0; j < m; j++ {
            d := abs(workers[i][0]-bikes[j][0]) + abs(workers[i][1]-bikes[j][1])
            arr[i*m+k] = tuple{d, i, j}
        }
    }
    sort.Slice(arr, func(i, j int) bool {
        if arr[i].d < arr[j].d {
            return arr[i].d < arr[j].d
        }
        if arr[i].i < arr[j].i {
            return arr[i].i < arr[j].i
        }
        return arr[i].j < arr[j].j
    })
    vis1, vis2 := make([]bool, n), make([]bool, m)
    ans := make([]int, n)
    for i, m := range arr {
        i, j := m.i, m.j
        if !vis1[i] && !vis2[j] {
            vis1[i], vis2[j] = true, true
            ans[i] = j
        }
    }
    return ans
}

func abs(x int) int {
    if x < 0 {
        return -x
    }
    return x
}

```

[7] Heap — Pattern Solution (stored optimal)

Python

Heap · LeetCode — By Pattern

— 1390 —

LeetCode

```

# Python solution
def assign_bikes(workers, bikes):
    # List to hold all possible (distance, worker, bike) tuples
    pairs = []
    for worker_index, (w_x, w_y) in enumerate(workers):
        for bike_index, (b_x, b_y) in enumerate(bikes):
            # Calculate Manhattan distance between worker and bike
            distance = abs(w_x - b_x) + abs(w_y - b_y)
            pairs.append((distance, worker_index, bike_index))

    # Sort pairs based on distance, then worker index, then bike index
    pairs.sort()

    n = len(workers) # Number of workers
    assigned_workers = [False] * n # Track if worker is already assigned
    assigned_bikes = [False] * len(bikes) # Track if bike is already assigned
    result = [-1] * n # Result list to store bike assignment for each worker

    # Iterate over the sorted pairs and assign bikes
    for distance, worker_index, bike_index in pairs:
        if not assigned_workers[worker_index] and not assigned_bikes[bike_index]:
            result[worker_index] = bike_index # Assign bike to worker
            assigned_workers[worker_index] = True # Mark worker as assigned
            assigned_bikes[bike_index] = True # Mark bike as assigned

    return result

# Example usage
workers = [[0,0],[2,1]]
bikes = [[1,2],[3,3]]
print(assign_bikes(workers, bikes)) # Returns: [1, 0]

```

C++

```

// C++ solution
#include <vector>
#include <utility>
#include <algorithm>
using namespace std;

vector<int> assignBikes(vector<vector<int>>& workers, vector<vector<int>>& bikes) {
    int n = workers.size();
    int m = bikes.size();
    // Define a tuple to store (distance, worker index, bike index)
    vector<tuple<int, int, int>> pairs;
    // Generate all possible pairs with their Manhattan distances
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) {
        int dist = abs(workers[i][0] - bikes[j][0]) + abs(workers[i][1] - bikes[j][1]);
        pairs.push_back(make_tuple(dist, i, j));
    }
    // Sort pairs by distance, then by worker index, then by bike index
    sort(pairs.begin(), pairs.end());
    vector<int> result(n, -1); // To store bike assignment for each worker
    // Iterate through sorted pairs and assign bikes
    for (auto& pair : pairs) {
        int distance, worker, bike;
        tie(distance, worker, bike) = pair;
        if (!workerAssigned[worker] && !bikeAssigned[bike]) {
            result[worker] = bike; // Assign bike to worker
            workerAssigned[worker] = true; // Mark worker as assigned
            bikeAssigned[bike] = true; // Mark bike as assigned
        }
    }
    return result;
}

// Example usage:
// int main() {
//     vector<vector<int>> workers = {{0, 0}, {2, 1}};
//     vector<vector<int>> bikes = {{1, 2}, {3, 3}};
//     vector<int> result = assignBikes(workers, bikes);
//     // Expected output: {1, 0}
//     return 0;
// }

```

#1167 MEDIUM

Minimum Cost to Connect Sticks

LeetCode · Difficulty: Medium · LeetCode · Editorial

Array · Greedy · Heap

List: Premium 98

Heap · LeetCode — By Pattern

— 1392 —

LeetCode

Heap · LeetCode — By Pattern

— 1391 —

LeetCode

Problem:

You have some number of sticks with positive integer lengths. These lengths are given as an array sticks, where sticks[i] is the length of the i-th stick.

You can connect any two sticks of lengths x and y into one stick by paying a cost of x + y. You must connect all the sticks until there is only one stick remaining.

Return the minimum cost of connecting all the given sticks into one stick in this way.

Example 1:

Input: sticks = [2,4,3]
Output: 14
Explanation: You start with sticks = [2,4,3].
1. Combine sticks 2 and 3 for a cost of 2 + 3 = 5. Now you have sticks = [5,4].
2. Combine sticks 5 and 4 for a cost of 5 + 4 = 9. Now you have sticks = [9].
There is only one stick left, so you are done. The total cost is 5 + 9 = 14.

Example 2:

Input: sticks = [1,8,3,5]
Output: 30
Explanation: You start with sticks = [1,8,3,5].
1. Combine sticks 1 and 3 for a cost of 1 + 3 = 4. Now you have sticks = [4,8,5].
2. Combine sticks 4 and 5 for a cost of 4 + 5 = 9. Now you have sticks = [9,8].
3. Combine sticks 9 and 8 for a cost of 9 + 8 = 17. Now you have sticks = [17].
There is only one stick left, so you are done. The total cost is 4 + 9 + 17 = 30.

Example 3:

Input: sticks = [5]
Output: 0
Explanation: There is only one stick, so you don't need to do anything. The total cost is 0.

Constraints:

- 1 <= sticks.length <= 104
- 1 <= sticks[i] <= 104

>> Brute Force (Heap) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        # Brute Force (naive) - sort repeatedly instead of heap
        data = sorted(sticks)
        result = 0
        for i in range(1, len(data)):
            result.append(data.pop(0))
        return result
```

Python

Python

```
import heapq

def connectSticks(sticks):
    # Initialize the heap with the provided stick lengths
    heapq.heapify(sticks)

    # Initialize total cost to 0
    total_cost = 0

    # Continue connecting sticks until there is only one left
    while len(sticks) > 1:
        # Extract the two smallest sticks
        stick1 = heapq.heappop(sticks)
        stick2 = heapq.heappop(sticks)
        # Calculate the cost to connect these two sticks
        cost = stick1 + stick2
        # Add the cost to the total cost
        total_cost += cost
        # Push the combined stick back into the heap
        heapq.heappush(sticks, cost)

    return total_cost

# Example usage:
# print(connectSticks([2, 4, 3])) # Output: 14
```

Python

```
class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        ans = 0
        heapq.heapify(sticks)
        while len(sticks) > 1:
            x = heapq.heappop(sticks)
            y = heapq.heappop(sticks)
            ans += x + y
            heapq.heappush(sticks, x + y)
        return ans
```

Python

```
class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        heapify(sticks)
        ans = 0
        while len(sticks) > 1:
            z = heappop(sticks) + heappop(sticks)
            ans += z
            heappush(sticks, z)
        return ans
```

Python

```
class Solution {
public:
    int connectSticks(vector<int>& sticks) {
        heapify(sticks);
        ans = 0;
        while (len(sticks) > 1) {
            x = heappop(sticks);
            y = heappop(sticks);
            ans += x + y;
            heappush(sticks, x + y);
        }
        return ans;
    }
};
```

C++

C++

```
class Solution {
public:
    int connectSticks(vector<int>& sticks) {
        int ans = 0;
        priority_queue<int, vector<int>, greater<>> minHeap;
        for (const int stick : sticks)
            minHeap.push(stick);
        while (minHeap.size() > 1) {
            const int x = minHeap.top();
            minHeap.pop();
            const int y = minHeap.top();
            minHeap.pop();
            ans += x + y;
            minHeap.push(x + y);
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int connectSticks(vector<int>& sticks) {
        priority_queue<int, vector<int>, greater<>> pq;
        for (const int stick : sticks) {
            pq.push(stick);
        }
        int ans = 0;
        while (pq.size() > 1) {
            int x = pq.top();
            pq.pop();
            int y = pq.top();
            pq.pop();
            int z = x + y;
            ans += z;
            pq.push(z);
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int connectSticks(vector<int>& sticks) {
        priority_queue<int, vector<int>, greater<>> pq;
        for (const int stick : sticks) {
            pq.push(stick);
        }
        int ans = 0;
        while (pq.size() > 1) {
            int x = pq.top();
            pq.pop();
            int y = pq.top();
            pq.pop();
            int z = x + y;
            ans += z;
            pq.push(z);
        }
        return ans;
    }
};
```

Java

Java

```
class Solution {
    public int connectSticks(int[] sticks) {
        int ans = 0;
        Queue<Integer> minHeap = new PriorityQueue<>();
        for (final int stick : sticks)
            minHeap.offer(stick);
        while (minHeap.size() > 1) {
            final int x = minHeap.poll();
            final int y = minHeap.poll();
            ans += x + y;
            minHeap.offer(x + y);
        }
        return ans;
    }
}
```

Java

```
class Solution {
    public int connectSticks(int[] sticks) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int x : sticks) {
            pq.offer(x);
        }
        int ans = 0;
        while (pq.size() > 1) {
            int z = pq.poll() + pq.poll();
            ans += z;
            pq.offer(z);
        }
        return ans;
    }
}
```

Java

```
class Solution {
    public int connectSticks(int[] sticks) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int x : sticks) {
            pq.offer(x);
        }
        int ans = 0;
        while (pq.size() > 1) {
            int z = pq.poll() + pq.poll();
            ans += z;
            pq.offer(z);
        }
        return ans;
    }
}
```

TypeScript

TypeScript

```
function connectSticks(sticks: number[]): number {
    const pq = new Heap(sticks);
    let ans = 0;
    while (pq.size() > 1) {
        const x = pq.pop();
        const y = pq.pop();
        ans += x + y;
        pq.push(x + y);
    }
    return ans;
}

type CompareFn = (this: T, rhs: T) => number;

class Heap<T> {
    data: Array<T> | null;
    constructor() {
        this.data = null;
    }
    constructor(data: T[]): void {
        this.data = data;
    }
    constructor(compare: CompareFn): void {
        this.data = null;
    }
    constructor(data: T[], compare: CompareFn): void {
        this.data = data;
    }
    constructor(data: T[], compareFn: CompareFn): void {
        this.data = data;
    }
    constructor(data: T[], compareFn: CompareFn, compare: CompareFn): void {
        this.data = data;
    }
}

if (typeof data === 'function') {
    compare = data;
    data = [];
}

this.data = [...data];
this.lt = (i, j) => compare(this.data[i], this.data[j]) < 0;
for (let i = this.size(); i > 0; i--) this.heapify(i);

size(): number {
    return this.data.length - 1;
}

push(v: T): void {
    this.data.push(v);
    let i = this.size();
    while (i > 1 && this.lt(i, i / 2)) this.swap(i, i / 2);
}

pop(): T {
    this.swap(1, this.size());
    const top = this.data.pop();
    this.heapify(1);
    return top;
}
```

```

    }
    top[i] = T {
        return this.data[i];
    }
}
heapify(i: number): void {
    while (true) {
        let min = i;
        const {l, r, n} = [i + 2, i + 2 + 1, this.data.length];
        if (l < n && this.lt(l, min)) min = l;
        if (r < n && this.lt(r, min)) min = r;
        if (min == i) {
            this.swap(i, min);
            i = min;
        } else break;
    }
}
clear(): void {
    this.data = [null];
}
}
private swap(i: number, j: number): void {
    const n = this.data;
    [n[i], n[j]] = [n[j], n[i]];
}
}
}

```

Go

```

func connectSticks(sticks [][]int) (ans int) {
    hp := &hp(sticks)
    heap.Init(hp)
    for hp.Len() > 1 {
        x, y := heap.Pop(hp).(int), heap.Pop(hp).(int)
        ans += x + y
        heap.Push(hp, x+y)
    }
    return
}

type hp struct{ sort.IntSlice }

func (h hp) Less(i, j int) bool { return h.IntSlice[i] < h.IntSlice[j] }
func (h hp) Push(x any) { h.IntSlice = append(h.IntSlice, x.(int)) }
func (h hp) Pop() any {
    a := h.IntSlice
    v := a[len(a)-1]
    h.IntSlice = a[:len(a)-1]
    return v
}

```

Go

```

func connectSticks(sticks [][]int) (ans int) {
    hp := &hp(sticks)
    heap.Init(hp)
    for hp.Len() > 1 {
        x, y := heap.Pop(hp).(int), heap.Pop(hp).(int)
        ans += x + y
        heap.Push(hp, x+y)
    }
    return
}

type hp struct{ sort.IntSlice }

func (h hp) Less(i, j int) bool { return h.IntSlice[i] < h.IntSlice[j] }
func (h hp) Push(x any) { h.IntSlice = append(h.IntSlice, x.(int)) }
func (h hp) Pop() any {
    a := h.IntSlice
    v := a[len(a)-1]
    h.IntSlice = a[:len(a)-1]
    return v
}

```

[*] Heap — Pattern Solution (matched from community answers)

Python

```

import heapq

def connectSticks(sticks):
    # Initialize the heap with the provided stick lengths
    heapq.heapify(sticks)

    # Initialize total cost to 0
    total_cost = 0

    # Continue connecting sticks until there is only one left
    while len(sticks) > 1:
        # Extract the two smallest sticks
        stick1 = heapq.heappop(sticks)
        stick2 = heapq.heappop(sticks)
        # Calculate the cost to connect these two sticks
        cost = stick1 + stick2
        # Add the cost to the total cost
        total_cost += cost
        # Push the combined stick back into the heap
        heapq.heappush(sticks, cost)

    return total_cost

# Example usage:
# print(connectSticks([2, 4, 3])) # Output: 14

```

#1424 MEDIUM

Diagonal Traverse II

LeetCode · SimplyList · WalkCC · LeetCode · Editorial

Array · Sorting · Heap

Lists · CodeSignal

Problem:

Given a 2D integer array `nums`, return all elements of `nums` in diagonal order as shown in the below images.

Example 1:



Input: `nums = [[1,2,3],[4,5,6],[7,8,9]]`

Output: `[1,4,2,7,5,3,6,8,9]`

Example 2:



Input: `nums = [[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]]`

Output: `[1,6,2,7,3,8,4,9,5,10,11,12,13,14,15,16]`

Constraints:

- `1 <= nums.length <= 105`
- `1 <= nums[i].length <= 105`
- `1 <= sum(nums[i].length) <= 105`
- `1 <= nums[i][j] <= 105`

>> Brute Force [Heap] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def findDiagonalOrder(self, nums):
        # Brute Force O(n^2) - sort repeatedly instead of heap
        data = list(nums)
        result = []
        for i, row in enumerate(nums):
            for j, value in enumerate(row):
                # Collect on the diagonal to achieve the desired order for the diagonal
                diagonal = i + j, insert(0, value)

        result = []
        # Process the diagonals in order of increasing key
        for key in sorted(diagonal.keys()):
            result.extend(diagonal[key])
        return result

# Example usage:
# nums = [[1,2,3],[4,5,6],[7,8,9]]
# print(findDiagonalOrder(nums)) # Output: [1,4,2,7,5,3,6,8,9]

```

Python

```

def findDiagonalOrder(nums):
    # Use a dictionary to collect elements by their diagonal key (i + j)
    from collections import defaultdict
    diagonals = defaultdict(list)

    # Iterate over each row and its elements
    for i, row in enumerate(nums):
        for j, value in enumerate(row):
            # Collect on the diagonal to achieve the desired order for the diagonal
            diagonals[i + j].append(value)

    result = []
    # Process the diagonals in order of increasing key
    for key in sorted(diagonals.keys()):
        result.extend(diagonals[key])
    return result

# Example usage:
# nums = [[1,2,3],[4,5,6],[7,8,9]]
# print(findDiagonalOrder(nums)) # Output: [1,4,2,7,5,3,6,8,9]

```

Python

```

class Solution:
    def findDiagonalOrder(self, nums: List[List[int]]) -> List[int]:
        arr = []
        for i, row in enumerate(nums):
            for j, v in enumerate(row):
                arr.append(i * 2 + j, v)
        arr.sort()
        return [v[2] for v in arr]

```

Python

```

class Solution:
    def findDiagonalOrder(self, nums: List[List[int]]) -> List[int]:
        arr = []
        for i, row in enumerate(nums):
            for j, v in enumerate(row):
                arr.append(i * 2 + j, v)
        arr.sort()
        return [v[2] for v in arr]

```

C++

```

C++
class Solution {
public:
    vector<int> findDiagonalOrder(vector<vector<int>>& nums) {
        vector<int> ans;
        unordered_map<int, vector<int>> keyValues; // key := row + column
        int maxkey = 0;

        for (int i = 0; i < nums.size(); ++i) {
            for (int j = 0; j < nums[i].size(); ++j) {
                const int key = i + j;
                keyValues[key].push_back(nums[i][j]);
                maxkey = max(maxkey, key);
            }
        }

        for (int i = 0; i <= maxkey; ++i) {
            for (auto it = keyValues[i].begin(); it != keyValues[i].end(); ++it) {
                ans.push_back(*it);
            }
        }

        return ans;
    }
};

```

```

C++
class Solution {
public:
    vector<int> findDiagonalOrder(vector<vector<int>>& nums) {
        vector<int> ans;
        for (int i = 0; i < nums.size(); ++i) {
            for (int j = 0; j < nums[i].size(); ++j) {
                arr.push_back(i * 2 + j, nums[i][j]);
            }
        }
        sort(arr.begin(), arr.end());
        vector<int> ans;
        for (auto it = arr.begin(); it != arr.end(); ++it) {
            ans.push_back(it->2);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<int> findDiagonalOrder(vector<vector<int>>& nums) {
        vector<int> ans;
        for (int i = 0; i < nums.size(); ++i) {
            for (int j = 0; j < nums[i].size(); ++j) {
                arr.push_back(i * 2 + j, nums[i][j]);
            }
        }
        sort(arr.begin(), arr.end());
        vector<int> ans;
        for (auto it = arr.begin(); it != arr.end(); ++it) {
            ans.push_back(it->2);
        }
        return ans;
    }
};

```

Java

```

class Solution {
public:
    List<Integer> findDiagonalOrder(List<List<Integer>> nums) {
        List<Integer> ans = new ArrayList<>();
        Map<Integer, List<Integer>> keyValues = new HashMap<>(); // key := row + column
        int maxkey = 0;

        for (int i = 0; i < nums.size(); ++i) {
            for (int j = 0; j < nums.get(i).size(); ++j) {
                final int key = i + j;
                keyValues.putIfAbsent(key, new ArrayList<>());
                keyValues.get(key).add(nums.get(i).get(j));
                maxkey = Math.max(maxkey, key);
            }
        }

        for (int i = 0; i <= maxkey; ++i) {
            for (int j = keyValues.get(i).size() - 1; j >= 0; --j) {
                ans.add(keyValues.get(i).get(j));
            }
        }

        return ans.stream().mapToInt(Integer::intValue).toArray();
    }
}

```

Java

```

class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        List<int> arr = new ArrayList<>();
        for (int i = 0; i < nums.size(); ++i) {
            for (int j = 0; j < nums.get(i).size(); ++j) {
                arr.add(nums.get(i + j, j, nums.get(i).get(j)));
            }
        }
        arr.sort((a, b) -> a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        int[] ans = new int[arr.size()];
        for (int i = 0; i < arr.size(); ++i) {
            ans[i] = arr.get(i)[2];
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    public int[] findDiagonalOrder(List<List<int>> nums) {
        List<int> arr = new ArrayList<>();
        for (int i = 0; i < nums.Count; ++i) {
            for (int j = 0; j < nums[i].Count; ++j) {
                arr.Add(nums[i + j, j, nums[i][j]]);
            }
        }
        arr.Sort((a, b) => a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        int[] ans = new int[arr.Count];
        for (int i = 0; i < arr.Count; ++i) {
            ans[i] = arr[i][2];
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int[] findDiagonalOrder(List<List<Integer>> nums) {
        List<int> arr = new ArrayList<>();
        for (int i = 0; i < nums.size(); ++i) {
            for (int j = 0; j < nums.get(i).size(); ++j) {
                arr.add(nums.get(i + j, j, nums.get(i).get(j)));
            }
        }
        arr.sort((a, b) -> a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        int[] ans = new int[arr.size()];
        for (int i = 0; i < arr.size(); ++i) {
            ans[i] = arr.get(i)[2];
        }
        return ans;
    }
}

```

Java

```

public class Solution {
    public int[] findDiagonalOrder(List<List<int>> nums) {
        List<int> arr = new ArrayList<>();
        for (int i = 0; i < nums.Count; ++i) {
            for (int j = 0; j < nums[i].Count; ++j) {
                arr.Add(nums[i + j, j, nums[i][j]]);
            }
        }
        arr.Sort((a, b) -> a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]);
        int[] ans = new int[arr.Count];
        for (int i = 0; i < arr.Count; ++i) {
            ans[i] = arr[i][2];
        }
        return ans;
    }
}

```

TypeScript

```

function findDiagonalOrder(nums: number[][]): number[] {
    const arr: number[][] = [];
    for (let i = 0; i < nums.length; ++i) {
        for (let j = 0; j < nums[i].length; ++j) {
            arr.push([i + j, j, nums[i][j]]);
        }
    }
    arr.sort((a, b) => (a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]));
    return arr.map(a => a[2]);
}

```

TypeScript

```

function findDiagonalOrder(nums: number[][]): number[] {
    const arr: number[][] = [];
    for (let i = 0; i < nums.length; ++i) {
        for (let j = 0; j < nums[i].length; ++j) {
            arr.push([i + j, j, nums[i][j]]);
        }
    }
    arr.sort((a, b) => (a[0] == b[0] ? a[1] - b[1] : a[0] - b[0]));
    return arr.map(a => a[2]);
}

```

Go

Go

```

func findDiagonalOrder(nums [][]int) []int {
    arr := [][]int{}
    for i, row := range nums {
        for j, v := range row {
            arr = append(arr, []int{i + j, j, v})
        }
    }
    sort.Slice(arr, func(i, j int) bool {
        if arr[i][0] == arr[j][0] {
            return arr[i][1] < arr[j][1]
        }
        return arr[i][0] < arr[j][0]
    })
    ans := []int{}
    for v := range arr {
        ans = append(ans, arr[v][2])
    }
    return ans
}

```

Go

```

func findDiagonalOrder(nums [][]int) []int {
    arr := [][]int{}
    for i, row := range nums {
        for j, v := range row {
            arr = append(arr, []int{i + j, j, v})
        }
    }
    sort.Slice(arr, func(i, j int) bool {
        if arr[i][0] == arr[j][0] {
            return arr[i][1] < arr[j][1]
        }
        return arr[i][0] < arr[j][0]
    })
    ans := []int{}
    for v := range arr {
        ans = append(ans, arr[v][2])
    }
    return ans
}

```

[?] Heap — Pattern Solution (stored optimal)

Python

```

def findDiagonalOrder(nums):
    # Use a dictionary to collect elements by their diagonal key (i + j)
    from collections import defaultdict
    diagonals = defaultdict(list)

    # Iterate over each row and its elements
    for i, row in enumerate(nums):
        for j, value in enumerate(row):
            # Insert at the beginning to achieve the desired order for the diagonal
            diagonals[i + j].insert(0, value)

    result = []
    # Process the diagonals in order of increasing key
    for key in sorted(diagonals.keys()):
        result.extend(diagonals[key])
    return result

# Example usage:
# nums = [[1,2,3],[4,5,6],[7,8,9]]
# print(findDiagonalOrder(nums)) # Output: [1,4,2,7,5,3,8,6,9]

```

C++

```

// Include headers:
// include <vector>
// include <unordered_map>
// include <algorithm>
// using namespace std;

vector<int> findDiagonalOrder(vector<vector<int>>& nums) {
    // Map from diagonal key to vector of integers
    unordered_map<int, vector<int>> diagonals;
    int maxKey = 0;

    // Iterate over each element in the 2D vector
    for (int i = 0; i < nums.size(); ++i) {
        for (int j = 0; j < nums[i].size(); ++j) {
            int key = i + j;
            maxKey = max(maxKey, key);
            diagonals[key].insert(diagonals[key].begin(), nums[i][j]);
        }
    }

    vector<int> result;
    // Iterate through keys from 0 to maxKey
    for (int key = 0; key <= maxKey; ++key) {
        if (diagonals.count(key)) {
            for (int value : diagonals[key]) {

```

```

        result.push_back(value);
    }
}
return result;
}

```

```

// Example usage:
// int main() {
//     vector<vector<int>> nums = {{1,2,3},{4,5,6},{7,8,9}};
//     vector<int> result = findDiagonalOrder(nums);
//     // result should be: [1,4,2,7,5,3,8,6,9]
//     return 0;
// }

```

#23

HARD

Merge k Sorted Lists

LeetCode · SimplifyList · WalkCC · LeetCode · Editorial

Linked List · Divide and Conquer · Heap · Merge Sort

Lists Grind 169

Problem:

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

Input: lists = [[1,4,5],[1,3,4],[2,6]]

Output: [1,1,2,3,4,4,5,6]

Explanation: The linked-lists are:

1->4->5,

1->3->4,

2->6

Merging them into one sorted linked list:

1->1->2->3->4->4->5->6

Example 2:

Input: lists = []

Output: []

Example 3:

Input: lists = [[]]

Output: []

Constraints:

- k == lists.length
- 0 <= k <= 104
- 0 <= lists[i].length <= 500
- 104 <= lists[i][j] <= 104

- lists[i] is sorted in ascending order.
- The sum of lists[i].length will not exceed 104.

>> Brute Force [Heap] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def mergeLists(self, lists):
        # Using force O(N log N) - collect all values, sort, rebuild
        vals = []
        for node in lists:
            while node:
                vals.append(node.val)
                node = node.next
        vals.sort()
        dummy = ListNode(0)
        curr = dummy
        for v in vals:
            curr.next = ListNode(v)
            curr = curr.next
        return dummy.next

```

Python

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

import heapq

class Solution:
    def mergeLists(self, lists):
        # Create a min-heap
        min_heap = []
        # Push every (node value, unique id, node) to handle duplicate values
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(min_heap, (node.val, i, node))

        # Dummy node to form the start of merged list
        dummy = ListNode(0)
        current = dummy

        # While there are elements in the heap
        while min_heap:
            # Pop the smallest node from the heap
            val, id, node = heapq.heappop(min_heap)

```

```

# Add the nextlast node to the merged list
current.next = node
current = current.next
# If there is a next node in the list, push it into the heap
if node.next:
    heapq.heappush(in_heap, (node.next.val, idx, node.next))

return dummy.next

```

```

Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        nodelist(ListNode, "_it_", lambda a, b: a.val < b.val)
        pq = Heapq for head in lists if head
        heapq.heappush(pq, (0, None))
        dummy = cur = ListNode()
        while pq:
            node = heapq.heappop(pq)
            if node.next:
                heapq.heappush(pq, node.next)
            cur.next = node
            cur = cur.next
        return dummy.next

```

```

Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        nodelist(ListNode, "_it_", lambda a, b: a.val < b.val)
        pq = Heapq for head in lists if head
        heapq.heappush(pq, (0, None))
        dummy = cur = ListNode()
        while pq:
            node = heapq.heappop(pq)
            if node.next:
                heapq.heappush(pq, node.next)
            cur.next = node
            cur = cur.next
        return dummy.next

```

```

#####
import heapq

...
...it(self, other) for =
...it(self, other) for =
...it(self, other) for =
...it(self, other) for =

...
# or create a wrapper class, without modifying existing node class
class NodeWrapper:
    def __init__(self, node):
        self.node = node

    def __lt__(self, other):
        return self.node.val < other.node.val

```

```

# override the comparison function, so the node can be comparable
# or else, error, TypeError: < not supported between instances of 'ListNode' and 'ListNode'
# define 'gt' will also work, as either lt or gt will do the compare job for ListNode
# ListNode.__lt__ = lambda x, y: (x.val < y.val)

ListNode.__lt__ = lambda x, y: (x.val < y.val)

```

```

class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        dummy = current = ListNode()
        heap = []
        for i, node in enumerate(lists):
            if node:
                # need to override
                # or else error: TypeError: < not supported between instances of 'ListNode' and 'ListNode'
                heapq.heappush(heap, node)

        while heap:
            node = heapq.heappop(heap)
            current.next = node
            current = current.next

            if node.next:
                heapq.heappush(heap, node.next)

        return dummy.next

```

```

C++
C++

```

```

class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*> lists) {
        ListNode dummy(0);
        ListNode curr = dummy;
        auto compare = [&](ListNode* a, ListNode* b) { return a->val > b->val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(compare)> minHeap(
            compare);
        for (ListNode* list : lists)
            if (list != nullptr)
                minHeap.push(list);
        while (!minHeap.empty()) {
            ListNode* minNode = minHeap.top();
            minHeap.pop();
            if (minNode->next)
                minHeap.push(minNode->next);
            curr->next = minNode;
            curr = curr->next;
        }
        return dummy.next;
    }
};

```

```

C++
// =
// Definition for singly-linked list.
// struct ListNode {
//     int val;
//     ListNode* next;
//     ListNode() : next(nullptr) {}
//     ListNode(int x) : val(x), next(nullptr) {}
//     ListNode(int x, ListNode* next) : val(x), next(next) {}
// };
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*> lists) {
        auto cmp = [&](ListNode* a, ListNode* b) { return a->val > b->val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq;
        for (auto head : lists) {
            if (head) {
                pq.push(head);
            }
        }
        ListNode* dummy = new ListNode();
        ListNode* cur = dummy;
        while (!pq.empty()) {
            ListNode* node = pq.top();
            pq.pop();
            cur->next = node;
            cur = cur->next;
        }
        return dummy->next;
    }
};

```

```

if (node->next) {
    pq.push(node->next);
}
cur->next = node;
cur = cur->next;
return dummy->next;
}
};

```

```

C++
// Q: https://leetcode.com/problems/merge-k-sorted-lists/
// Time: O(NlogK)
// Space: O(K)
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*> lists) {
        ListNode dummy, *tail = &dummy;
        auto cmp = [&](ListNode* a, ListNode* b) { return a->val > b->val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
        for (auto list : lists) {
            if (list) pq.push(list); // avoid pushing NULL list
        }
        while (pq.size()) {
            auto node = pq.top();
            pq.pop();
            if (node->next) pq.push(node->next);
            tail->next = node;
            tail = node;
        }
        return dummy.next;
    }
};

```

```

Java
Java
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        Queue<ListNode> minHeap = new PriorityQueue<Comparator<ListNode>>((a, b) -> a.val < b.val));
        for (final ListNode list : lists)
            if (list != null)
                minHeap.offer(list);
        while (!minHeap.isEmpty()) {
            ListNode minNode = minHeap.poll();
            if (minNode.next != null)
                minHeap.offer(minNode.next);
            curr.next = minNode;
            curr = curr.next;
        }
        return dummy.next;
    }
}

```

```

Java
from queue import PriorityQueue

class Solution:
    def mergeKLists(self, lists: List[ListNode]) -> ListNode:
        dummy = ListNode(0)
        cur = dummy
        pq = PriorityQueue()

        for i, list in enumerate(lists):
            if list:
                pq.put((list.val, i, list))

        while not pq.empty():
            _, i, minNode = pq.get()
            if minNode.next:
                pq.put((minNode.next.val, i, minNode.next))
            cur.next = minNode
            cur = cur.next

        return dummy.next

```

```

Java
// =
// Definition for singly-linked list.
// public class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val < b.val);
        for (ListNode head : lists) {
            if (head != null) {
                pq.offer(head);
            }
        }
        ListNode dummy = new ListNode();
        ListNode cur = dummy;
        while (!pq.isEmpty()) {
            while (pq.isEmpty()) {
                ListNode node = pq.poll();
                if (node.next != null) {
                    pq.offer(node.next);
                }
            }
            cur.next = node;
            cur = cur.next;
        }
        return dummy.next;
    }
}

```

```

// =
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }
function mergeKLists(lists: Array<ListNode> | null[]): ListNode | null {
    const pq = new PriorityQueue<((a, b) => a.val < b.val);
    lists.filter(head => head).forEach(head => pq.enqueue(head));
    let cur = dummy;
    while (pq.isEmpty()) {
        const node = pq.dequeue();
        cur.next = node;
        cur = cur.next;
        if (node.next) {
            pq.enqueue(node.next);
        }
    }
    return dummy.next;
}

```

```

Java
// =
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }
var mergeKLists = function(lists) {
    const pq = new PriorityQueue<((a, b) => a.val < b.val);
    lists.filter(head => head).forEach(head => pq.enqueue(head));
    let cur = dummy;
    while (pq.isEmpty()) {
        const node = pq.dequeue();
        cur.next = node;
        cur = cur.next;
        if (node.next) {
            pq.enqueue(node.next);
        }
    }
    return dummy.next;
}

```



```

Java
/*
 * Definition for singly-linked list.
 */
public class ListNode {
    int val;
    ListNode next;
    ListNode(int val) {
        this.val = val;
        this.next = null;
    }
}

public class Solution {
    public ListNode MergeLists(ListNode[] lists) {
        PriorityQueue<ListNode, Integer> pq = new PriorityQueue<ListNode, Integer>();
        for (var head : lists) {
            if (head != null) {
                pq.offer(head, head.val);
            }
        }
        var dummy = new ListNode();
        var cur = dummy;
        while (pq.size() > 0) {
            var node = pq.poll();
            cur.next = node;
            cur = cur.next;
        }
        if (node.next != null) {
            pq.offer(node.next, node.next.val);
        }
        return dummy.next;
    }
}

```

Java

```

public class MergeKSortedLists {
    public static void main(String[] args) {
        MergeKSortedLists sol = new MergeKSortedLists();
        Solution s = sol.new Solution();

        ListNode l1 = null;
        ListNode l2 = new ListNode(1);
        s.mergeLists(new ListNode[] {l1, l2});
    }

    public class Solution {
        public ListNode mergeKLists(ListNode[] lists) {
            if (lists == null || lists.length == 0) {
                return null;
            }

            // base case: merge two arrays
            return merge(lists, 0, lists.length - 1);
        }

        public ListNode merge(ListNode[] lists, int start, int end) {
            // single list
            if (start == end) {
                return lists[start];
            }

            int mid = (end - start) / 2 + start;
            ListNode leftHalf = merge(lists, start, mid);
            ListNode rightHalf = merge(lists, mid + 1, end);
            return mergeTwoLists(leftHalf, rightHalf);
        }

        // from previous question: 21. Merge Two Sorted Lists
        public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
            ListNode dummy = new ListNode(0);
            ListNode current = dummy;

            while (l1 != null || l2 != null) {
                int v1 = l1 == null ? Integer.MAX_VALUE : l1.val;
                int v2 = l2 == null ? Integer.MAX_VALUE : l2.val;
                if (v1 < v2) {
                    current.next = l1;
                    l1 = l1.next;
                } else {
                    current.next = l2;
                    l2 = l2.next;
                }
                current = current.next;
            }
            return dummy.next;
        }
    }
}

```

```

current.next = l1;
l1 = l1.next;
} else {
    current.next = l2;
    l2 = l2.next;
}
current = current.next; // new current is the new node, but still pointing to next node
current.next = null; // quote: key, cut this node from l1 or l2
}
return dummy.next;
}
}

/////

class Solution_Heap {
    public ListNode mergeKLists(ListNode[] lists) {
        if (lists == null || lists.length == 0) {
            return null;
        }

        ListNode dummy = new ListNode(0);
        ListNode current = dummy;

        // put list of each list to heap
    }
}

```

Java

```

/*
 * Definition for singly-linked list.
 */
public class ListNode {
    int val;
    ListNode next;
    ListNode(int val) {
        this.val = val;
        this.next = null;
    }
}

//
public class Solution {
    public ListNode MergeKLists(ListNode[] lists) {
        int n = lists.length;
        if (n == 0) {
            return null;
        }
        for (int i = 1; i < n; ++i) {
            lists[i] = MergeTwoLists(lists[i - 1], lists[i]);
        }
        return lists[n - 1];
    }

    private ListNode MergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
    }
}

```

```

ListNode cur = dummy;
while (l1 != null || l2 != null) {
    if (l1.val < l2.val) {
        cur.next = l1;
        l1 = l1.next;
    } else {
        cur.next = l2;
        l2 = l2.next;
    }
    cur = cur.next;
}
cur.next = l1 == null ? l2 : l1;
return dummy.next;
}

```

JavaScript

```

/*
 * Definition for singly-linked list.
 */
function ListNode(val, next) {
    this.val = (val === undefined ? 0 : val);
    this.next = (next === undefined ? null : next);
}

//
// @param {ListNode[]} lists
// @return {ListNode}
var mergeKLists = function (lists) {
    const n = lists.length;
    if (n == 0) {
        return null;
    }
    for (let i = 1; i < n; ++i) {
        lists[i] = mergeTwoLists(lists[i - 1], lists[i]);
    }
    return lists[n - 1];
};

function mergeTwoLists(l1, l2) {
    const dummy = new ListNode();
    let cur = dummy;

    while (l1 || l2) {
        if (l1.val < l2.val) {
            cur.next = l1;
            l1 = l1.next;
        } else {
            cur.next = l2;
            l2 = l2.next;
        }
        cur = cur.next;
    }
    cur.next = l1 || l2;
    return dummy.next;
}

```

```

//
// Definition for singly-linked list.
// class ListNode {
//     attr: number;
//     def: init(attr: number, next: null | ListNode) {
//         this.val = attr;
//         this.next = next;
//     }
// }
// @param {ListNode[]} lists
// @return {ListNode}
def merge_k_lists(lists: List[ListNode]) -> ListNode {
    n = lists.length
    while n > 1 {
        lists[i] = merge_two_lists(lists[i - 1], lists[i])
        i += 1
    }
    return lists[0]
}

def merge_two_lists(l1: ListNode, l2: ListNode) -> ListNode {
    dummy = ListNode(0)
    cur = dummy
    while l1 || l2 {
        if l1.val < l2.val {
            cur.next = l1
            l1 = l1.next
        } else {
            cur.next = l2
            l2 = l2.next
        }
        cur = cur.next
    }
    cur.next = l1 || l2
    return dummy.next
}

```

TypeScript

TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val === undefined ? 0 : val);
//         this.next = (next === undefined ? null : next);
//     }
// }
//
function mergeKLists(lists: Array<ListNode | null>): ListNode | null {
    const n = lists.length;
    const dfs = (start: number, end: number) => {
        if (end - start == 1) {
            return lists[start] ?? null;
        }
        const mid = (start + end) >> 1;
        let left = dfs(start, mid);
        let right = dfs(mid, end);
        const dummy = new ListNode();
        let cur = dummy;
        while (left || right) {
            let next = ListNode();
            if (
                (left ?? (val: number, end: number)) <
                (right ?? (val: number, end: number))
            ) {
                next = left;
                left = left.next;
            } else {
                next = right;
                right = right.next;
            }
            cur.next = next;
            cur = next;
        }
        return dfs(0, n);
    };
    return dfs(0, n);
}

```

Rust

Rust

```

// Definition for singly-linked list.
// #define Prettily, eg. class, struct
// pub struct ListNode {
//     pub val: i32,
//     pub next: Option

```

Heap · LeetCode - By Pattern

— 1423 —

LeetCode

[*] Heap — Pattern Solution (matched from community answers)

Python

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

import heapq

class Solution:
    def mergeKLists(self, lists):
        # Create a min heap
        min_heap = []
        # Push all nodes into the heap
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(min_heap, (node.val, i, node))

        # Dummy node to form the start of merged list
        dummy = ListNode(0)
        current = dummy

        # While there are elements in the heap
        while min_heap:
            # Pop the smallest node from the heap
            val, idx, node = heapq.heappop(min_heap)

            # Add the smallest node to the merged list
            current.next = node
            current = current.next

            # If there is a next node in the list, push it into the heap
            if node.next:
                heapq.heappush(min_heap, (node.next.val, idx, node.next))

        return dummy.next

```

#218

HARD

The Skyline Problem

LeetCode · Companies · BuildCC · LeetCode · Editorial

Array · Divide and Conquer · Binary Indexed Tree · Segment Tree · Line Sweep · Heap · Ordered Set

Little Denny Zhang

Problem:

A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively.

The geometric information of each building is given in the array buildings where buildings[i] = [lefti, righti, heighti].

- lefti is the x coordinate of the left edge of the ith building.

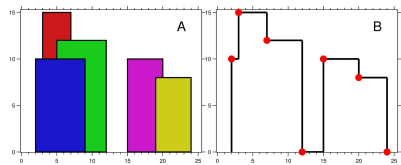
- righti is the x coordinate of the right edge of the ith building.
- heighti is the height of the ith building.

You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0.

The skyline should be represented as a list of "key points" sorted by their x-coordinate in the form [[x1,y1],[x2,y2]...]. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate 0 and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, [...[2 3], [4 5], [12 7], ...] is not acceptable; the three lines of height 5 should be merged into one in the final output as such: [...[2 3], [4 5], [12 7], ...]

Example 1:



Input: buildings = [[1,9,10],[3,7,15],[12,12,12],[15,20,8],[19,24,4]]

Output: [[1,9],[3,15],[7,12],[12,10],[15,8],[20,0]]

Explanation:

Figure A shows the buildings of the input.

Figure B shows the skyline formed by those buildings. The red points in figure B represent the key points in the output list.

Example 2:

Input: buildings = [[0,2,3],[2,5,3]]

Output: [[0,3],[5,0]]

Constraints:

- 1 <= buildings.length <= 104
- 0 <= lefti < righti <= 231 - 1
- 1 <= heighti <= 231 - 1
- buildings is sorted by lefti in non-decreasing order.

>> Brute Force [Heap] Interview Prep

Python

Heap · LeetCode - By Pattern

— 1425 —

LeetCode

Python

Python

```

import heapq

def getSkyline(buildings):
    # List to store all events (x, height, right)
    events = []
    # Create events for all buildings: left edge (start) and right edge (end)
    for left, right, height in buildings:
        # For the start point, height is negative for proper sorting (start events come before end events)
        events.append((-height, left, right))
        # For the end event, height is 0 to indicate removal of building from active heap
        events.append((0, right, 0))

    # Sort events by x coordinate
    events.sort()

    # Use heap to store active buildings (using negative heights)
    result = []
    # Initialize the heap with a dummy building of height 0 that extends infinitely
    heap = [(0, float('inf'))]

    for x, neg_height, right in events:
        # Remove buildings from the heap that have ended by current x
        while heap and heap[0][1] == x:
            heapq.heappop(heap)

        # If event is a start event (neg_height != 0), push it into the heap
        if neg_height != 0:
            heapq.heappush(heap, (neg_height, right))

        # Current highest building height is at the top of the heap
        current_height = -heap[0][0]
        # If the result is empty or current height differs from last recorded height, add a new key point
        if not result or current_height != result[-1][1]:
            result.append([x, current_height])

    return result

# Example usage
buildings = [[1,9,10],[3,7,15],[12,12,12],[15,20,8],[19,24,4]]
print(getSkyline(buildings)) # Expected output: [[1,9],[3,15],[7,12],[12,10],[15,8],[20,0]]

```

Python

Heap · LeetCode - By Pattern

— 1426 —

LeetCode

```

class Solution {
public:
    vector<vector<int>> getSkyline(vector<vector<int>>& buildings) {
        if (buildings.empty()) return {};
        int n = buildings.size();
        if (n == 1) return {{0, 0}};
        vector<vector<int>> lefts, rights;
        for (int i = 0; i < n; i++) {
            lefts.push_back({buildings[i][0], buildings[i][2]});
            rights.push_back({buildings[i][1], buildings[i][2]});
        }
        sort(lefts.begin(), lefts.end());
        sort(rights.begin(), rights.end());
        int i = 0, j = 0;
        while (i < lefts.size() && j < rights.size()) {
            if (lefts[i][0] <= rights[j][1]) {
                lefts[i][2] = max(lefts[i][2], rights[j][2]);
                i++;
            } else {
                rights[j][2] = max(rights[j][2], lefts[i][2]);
                j++;
            }
        }
        vector<vector<int>> ans;
        int curHeight = 0;
        for (int i = 0; i < lefts.size(); i++) {
            if (lefts[i][2] > curHeight) {
                ans.push_back({lefts[i][0], lefts[i][2]});
                curHeight = lefts[i][2];
            }
        }
        return ans;
    }
};

```

C++

C++

Heap · LeetCode - By Pattern

— 1427 —

LeetCode

```

class Solution {
public:
    vector<vector<int>> getSkyline(vector<vector<int>>& buildings) {
        const int n = buildings.size();
        if (n == 0) return {};
        if (n == 1) {
            const int left = buildings[0][0];
            const int right = buildings[0][1];
            const int height = buildings[0][2];
            return {{left, height}, {right, 0}};
        }
        const vector<vector<int>> lefts =
            getSkyline(buildings.begin(), buildings.begin() + n / 2);
        const vector<vector<int>> rights =
            getSkyline(buildings.begin() + n / 2, buildings.end());
        return merge(lefts, rights);
    }
private:
    vector<vector<int>> merge(const vector<vector<int>>& lefts,
                           const vector<vector<int>>& rights) {
        int i = 0, j = 0;
        int left = 0;
        int right = 0;
        while (i < lefts.size() && j < rights.size()) {
            if (lefts[i][0] <= rights[j][1]) {
                left = lefts[i][2];
                addPoints(lefts[i][0], max(lefts[i][2], rights[j][2]));
            } else {
                right = rights[j][2];
                addPoints(rights[j][0], max(rights[j][2], lefts[i][2]));
            }
        }
        while (i < lefts.size()) {
            addPoints(lefts[i][0], lefts[i][2]);
        }
        while (j < rights.size()) {
            addPoints(rights[j][0], rights[j][2]);
        }
        return ans;
    }
    void addPoint(vector<vector<int>>& ans, int x, int y) {
        if (ans.empty() || ans.back()[0] == x) {
            ans.back()[1] = max(ans.back()[1], y);
        } else {
            ans.push_back({x, y});
        }
    }
};

```

Heap · LeetCode - By Pattern

— 1428 —

LeetCode

```

ans.back()[i] = y;
return;
}
if (ans.empty() && ans.back()[i] == y)
return;
ans.push_back(x, y);
};

C++
class Solution {
public:
vector<vector<int>> getSkyline(vector<vector<int>>& buildings) {
vector<vector<int>> ans;
vector<vector<int>> events; // {x1, h1} | {x2, -h2}
for (const vector<int>& b : buildings) {
events.push_back({b[0], b[1]}); // Minus means leaving.
}
ranges::sort(events, ranges::less(), [](const vector<int>& event) {
return pair{event[0], -event[1]};
});
for (const vector<int>& event : events) {
const int x = event[0];
const int h = abs(event[1]);
const int isEntering = event[1] > 0;
if (isEntering) {
if (h > maxHeight())
ans.push_back({x, h});
set.insert(h);
} else {
set.erase(set.equal_range(h).first);
}
if (h > maxHeight())
ans.push_back({x, maxHeight()});
}
return ans;
};
private:
multiset<int> set;
int maxHeight() const {
return set.empty() ? 0 : *set.rbegin();
};
};

```

```

class Solution {
public List<List<Integer>> getSkyline(int[][] buildings) {
final int n = buildings.length;
if (n == 0)
return new ArrayList<>();
if (n == 1) {
final int left = buildings[0][0];
final int right = buildings[0][1];
final int height = buildings[0][2];
List<List<Integer>> ans = new ArrayList<>();
ans.add(new ArrayList<>{left, right, height});
return ans;
}
List<List<Integer>> leftSkyline = getSkyline(Arrays.copyOfRange(buildings, 0, n / 2));
List<List<Integer>> rightSkyline = getSkyline(Arrays.copyOfRange(buildings, n / 2, n));
return merge(leftSkyline, rightSkyline);
}
private List<List<Integer>> merge(List<List<Integer>> left, List<List<Integer>> right) {
List<List<Integer>> ans = new ArrayList<>();
int i = 0; // left's index
int j = 0; // right's index
int leftH = 0;
int rightH = 0;
while (i < left.size() && j < right.size()) {
// Choose the point with the smaller x
if (left.get(i).get(0) < right.get(j).get(0)) {
leftH = left.get(i).get(2); // Store the ongoing 'leftH'
addPoint(ans, left.get(i).get(0), Math.max(left.get(i+1).get(2), rightH));
} else {
rightH = right.get(j).get(2); // Update the ongoing 'rightH'
addPoint(ans, right.get(j).get(0), Math.max(right.get(j+1).get(2), leftH));
}
}
while (i < left.size())
addPoint(ans, left.get(i).get(0), left.get(i+1).get(2));
while (j < right.size())
addPoint(ans, right.get(j).get(0), right.get(j+1).get(2));
return ans;
}
private void addPoint(List<List<Integer>> ans, int x, int y) {
if (ans.isEmpty() && ans.get(ans.size() - 1).get(0) == x)
ans.get(ans.size() - 1).set(2, y);
return;
ans.add(new ArrayList<>{x, y});
}
}

```

```

Java
from queue import PriorityQueue

class Solution:
def getSkyline(self, buildings: List[List[int]]) -> List[List[int]]:
sky, time, pq = [], [], PriorityQueue()
for b in buildings:
time.append(b[0]), time.append(b[1])
time.sort()
city, n = 0, len(buildings)
for line in time:
while city < n and buildings[city][0] == line:
pq.put((-buildings[city][2], buildings[city][1]), buildings[city][1])
city += 1
while not pq.empty() and pq.queue[0][2] == line:
pq.get()
high = 0
if not pq.empty():
high = pq.queue[0][2]
if len(sky) > 0 and sky[-1][1] == high:
continue
sky.append(line, high)
return sky

plaintext
PLAINTEXT
[ [2,10], [3,15], [7,12], [12,9], [15,8], [20,8], [24,6] ]

[*] Heap - Pattern Solution (matched from community answers)
Python

```

```

import heapq

def getSkyline(buildings):
# List to store all events: (x, height, right)
events = []
# Create events for all buildings: left edge (start) and right edge (end)
for left, right, height in buildings:
# For the first event, height is negative for proper sorting (start events come before and ends)
events.append((left, -height, right))
# For the last event, height is 0 to indicate removal of building from active heap
events.append((right, 0, 0))

# Sort events by x coordinates
events.sort()

# Max heap to store active buildings (using negative heights)
result = []
# Initialize the heap with a dummy building of height 0 that extends infinitely
heap = [(0, float('inf'))]

for x, neg_height, right in events:
# Remove buildings from the heap that have ended by current x
while heap and heap[0][1] == x:
heapq.heappop(heap)

# If event is a start event (neg_height != 0), push it into the heap.
if neg_height != 0:
heapq.heappush(heap, (neg_height, right))

# Current highest building height is at the top of the heap
current_height = -heap[0][1]
# If the result is empty or current height differs from last recorded height, add a new key point.
if not result or current_height != result[-1][1]:
result.append((x, current_height))

return result

# Expected output: [[2,10],[3,15],[7,12],[12,9],[15,8],[20,8],[24,6]]
print(getSkyline(buildings))

```

#272 **HARD**

Closest Binary Search Tree Value II

LeetCode · [Simplify](#) · [Watch](#) · [LeetCode](#) · [Editorial](#)

Two Pointers · Stack · Tree · DFS · BST · Heap

LeetCode Premium 98

Problem:

Given the root of a binary search tree, a target value, and an integer k, return the k values in the BST that are closest to the target. You may return the answer in any order.

You are guaranteed to have only one unique set of k values in the BST that are closest to the target.

Example 1:

```

graph TD
    4((4)) --- 2((2))
    4 --- 5((5))
    2 --- 1((1))
    2 --- 3((3))

```

Input: root = [4,2,5,1,3], target = 3.714286, k = 2
Output: [4,3]

Example 2:
Input: root = [1], target = 0.000000, k = 1
Output: [1]

Constraints:

- The number of nodes in the tree is n.
- 1 <= k <= n <= 104.
- 0 <= Node.val <= 109.
- 109 <= target <= 109

Follow up: Assume that the BST is balanced. Could you solve it in less than O(n) runtime (where n = total nodes)?

>> Brute Force (Heap) Interview Prep

Python

```

# Brute Force Approach
class Solution:
def closestK(self, root: TreeNode, target: float, k: int):
# Brute Force: Use long to sort repeatedly instead of heap
data = list(root)
result = []
for i in range(k):
data.sort()
result.append(data.pop(0))
return result

```

```

Python
Python
# Python solution for Closest Binary Search Tree Value II

# Definition for a binary tree node.
class TreeNode:
def __init__(self, val=0, left=None, right=None):
self.val = val
self.left = left
self.right = right

class Solution:
def closestKValues(self, root: TreeNode, target: float, k: int):
# Helper function to initialize predecessor's stack
def initPredecessor(node):
while node:
# If node value is less or equal, push to pre and go right
if node.val <= target:
pre.append(node)
node = node.right
else:
node = node.left

# Helper function to initialize successor's stack
def initSuccessor(node):
while node:
# If node value is greater than target, push to succ and go left
if node.val > target:
succ.append(node)
node = node.left
else:
node = node.right

# Helper function to get next predecessor value
def getPredecessor():
if not pre: return None
node = pre.pop()
val = node.val
# Traverse the left subtree: push nodes from left child then rightmost branch
node = node.left
while node:
pre.append(node)
node = node.right
return val

# Helper function to get next successor value
def getSuccessor():
if not succ: return None
node = succ.pop()
val = node.val
# Traverse the right subtree: push nodes from right child then leftmost branch
node = node.left

```

```

        while node:
            succs.append(node)
            node = node.left
            return val

        preds, succs = [], []
        # Initialize the deque with correct boundary values
        initPredecessor(root)
        initSuccessor(root)

        result = []
        # If the closest value from predecessor and successor are same, skip one to avoid duplicates.
        if preds and succs and preds[-1].val == succs[-1].val:
            getPredecessor()

        # Merge k closest values
        for _ in range(k):
            if not preds:
                # Only successor left
                result.append(getSuccessor())
            elif not succs:
                # Only predecessor left
                result.append(getPredecessor())
            else:
                # Compare which candidate is closer to the target

                if abs(preds[-1].val - target) <= abs(succs[-1].val - target):
                    result.append(getPredecessor())
                else:
                    result.append(getSuccessor())

        return result

```

```

Python
class Solution:
    def closestKValues(
        self,
        root: TreeNode | None,
        target: float,
        k: int,
    ) -> List[int]:
        dq = collections.deque()

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return
            inorder(root.left)
            dq.append(root.val)
            inorder(root.right)

        inorder(root)

        while len(dq) > k:
            if abs(dq[0] - target) > abs(dq[-1] - target):
                dq.popleft()
            else:
                dq.pop()

```

```

        return List(dq)

```

```

Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestKValues(self, root: TreeNode, target: float, k: int) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            if len(q) < k:
                q.append(root.val)
            else:
                if abs(root.val - target) <= abs(q[0] - target):
                    return
                q.popleft()
                q.append(root.val)
            dfs(root.right)

        q = deque()
        dfs(root)
        return List(q)

```

```

// @ts-ignore
from collections import deque

# Iterative
class Solution:
    def closestKValues(self, root: TreeNode, target: float, k: int) -> List[int]:
        stack = []
        q = deque()

        # Iterative in-order traversal
        while stack or root:
            while root:
                stack.append(root)
                root = root.left

            root = stack.pop()

            # Process current node
            if len(q) < k:
                q.append(root.val)
            else:
                if abs(root.val - target) < abs(q[0] - target):
                    q.popleft()
                    q.append(root.val)

            else:
                break
            # Early stop if the current value is farther than the first value in queue

            root = root.right

        return List(q)

```

C++

```

class Solution {
public:
    vector<int> closestKValues(TreeNode* root, double target, int k) {
        deque<int> dq;

        inorder(root, dq);

        while (dq.size() > k)
            if (abs(dq.front() - target) > abs(dq.back() - target))
                dq.pop_front();
            else
                dq.pop_back();

        return {dq.begin(), dq.end()};
    }

    void inorder(TreeNode* root, deque<int>& dq) {
        if (root == nullptr)
            return;

        inorder(root->left, dq);
        dq.push_back(root->val);
        inorder(root->right, dq);
    }
};

```

```

C++
// @
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };

class Solution {
public:
    vector<int> q;
    double target;
    int k;

    vector<int> closestKValues(TreeNode* root, double target, int k) {
        this->target = target;
        this->k = k;
        dfs(root);
        while (!q.empty()) {
            int popBack = q.back();
            q.pop();
        }
    }
};

```

```

        }
        return ans;
    }

    void dfs(TreeNode* root) {
        if (!root) return;
        dfs(root->left);
        if (q.size() > k)
            q.push(root->val);
        else {
            if (abs(root->val - target) <= abs(q.front() - target)) return;
            q.pop();
            q.push(root->val);
        }
        dfs(root->right);
    }
};

```

```

C++
// @
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };

class Solution {
public:
    vector<int> q;
    double target;
    int k;

    vector<int> closestKValues(TreeNode* root, double target, int k) {
        this->target = target;
        this->k = k;
        dfs(root);
        while (!q.empty()) {
            int popBack = q.back();
            q.pop();
        }
    }
};

```

```

        }
        return ans;
    }

    void dfs(TreeNode* root) {
        if (!root) return;
        dfs(root->left);
        if (q.size() > k)
            q.push(root->val);
        else {
            if (abs(root->val - target) <= abs(q.front() - target)) return;
            q.pop();
            q.push(root->val);
        }
        dfs(root->right);
    }
};

```

```

Java
// @
// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    public List<Integer> closestKValues(TreeNode root, double target, int k) {
        Deque<Integer> dq = new ArrayDeque<>();

        inorder(root, dq);

        while (dq.size() > k)
            if (Math.abs(dq.peekFirst() - target) > Math.abs(dq.peekLast() - target))
                dq.pollFirst();
            else
                dq.pollLast();

        return new ArrayList<>(dq);
    }

    private void inorder(TreeNode root, Deque<Integer> dq) {
        if (root == null)
            return;

        inorder(root.left, dq);
        dq.offerLast(root.val);
        inorder(root.right, dq);
    }
}

```

```

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    private List<Integer> ans;
    private double target;
    private int k;

    public List<Integer> closestValues(TreeNode root, double target, int k) {
        ans = new LinkedList<>();
        this.target = target;
        this.k = k;

        dfs(root);
        return ans;
    }

    private void dfs(TreeNode root) {
        if (root == null) {
            return;
        }
        dfs(root.left);
        if (ans.size() < k) {
            ans.add(root.val);
        } else {
            if (Math.abs(root.val - target) <= Math.abs(ans.get(0) - target)) {
                return;
            }
            ans.remove(0);
            ans.add(root.val);
        }
        dfs(root.right);
    }
}

```

Java

```

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    private List<Integer> ans;
    private double target;
    private int k;

    public List<Integer> closestValues(TreeNode root, double target, int k) {
        ans = new LinkedList<>();
        this.target = target;
        this.k = k;
        dfs(root);

        return ans;
    }

    private void dfs(TreeNode root) {
        if (root == null) {
            return;
        }
        dfs(root.left);
        if (ans.size() < k) {
            ans.add(root.val);
        } else {
            if (Math.abs(root.val - target) <= Math.abs(ans.get(0) - target)) {
                return;
            }
            ans.remove(0);
            ans.add(root.val);
        }
        dfs(root.right);
    }
}

```

Go

Go

```

//
// Definition for a binary tree node.
//
type TreeNode struct {
    Val int
    Left *TreeNode
    Right *TreeNode
}

func closestValues(root *TreeNode, target float64, k int) []int {
    var ans []int
    var dfs func(root *TreeNode)
    dfs = func(root *TreeNode) {
        if root == nil {
            return
        }
        dfs(root.Left)
        if len(ans) < k {
            ans = append(ans, root.Val)
        } else {
            if math.Abs(float64(root.Val)-target) <= math.Abs(float64(ans[0])-target) {
                return
            }
            ans = ans[1:]
            ans = append(ans, root.Val)
        }
        dfs(root.Right)
    }
    dfs(root)
    return ans
}

```

Go

```

//
// Definition for a binary tree node.
//
type TreeNode struct {
    Val int
    Left *TreeNode
    Right *TreeNode
}

func closestValues(root *TreeNode, target float64, k int) []int {
    var ans []int
    var dfs func(root *TreeNode)
    dfs = func(root *TreeNode) {
        if root == nil {
            return
        }
        dfs(root.Left)
        if len(ans) < k {
            ans = append(ans, root.Val)
        } else {
            if math.Abs(float64(root.Val)-target) <= math.Abs(float64(ans[0])-target) {
                return
            }
            ans = ans[1:]
            ans = append(ans, root.Val)
        }
        dfs(root.Right)
    }
    dfs(root)
    return ans
}

```

```

    dfs(root.Right)
}
dfs(root)
return ans
}

```

[I] Heap — Pattern Solution (stored optimal)

Python

Python solution for Closest Binary Search Tree Value II

Definition for a binary tree node.

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

```

class Solution:

```

    def closestValues(self, root: TreeNode, target: float, k: int):
        # Helper function to initialize predecessor stack
        def initPredecessor(node):
            while node:
                # If node value is less or equal, push to prede and go right
                if node.val <= target:
                    prede.append(node)
                    node = node.right
                else:
                    node = node.left

        # Helper function to initialize successor stack
        def initSuccessor(node):
            while node:
                # If node value is greater than target, push to succ and go left

```

```

        if node.val > target:
            succ.append(node)
            node = node.left
        else:
            node = node.right

# Helper function to get next predecessor value
def getPredecessor():
    if not prede: return None
    node = prede.pop()
    val = node.val
    # Traverse the left subtree: push nodes from left child then rightmost branch
    node = node.left
    while node:
        prede.append(node)
        node = node.right
    return val

# Helper function to get next successor value
def getSuccessor():
    if not succ: return None
    node = succ.pop()
    val = node.val
    # Traverse the right subtree: push nodes from right child then leftmost branch
    node = node.right
    while node:
        succ.append(node)
        node = node.left
    return val

prede, succ = [], []
# Initialize the stacks with correct boundary values
initPredecessor(root)
initSuccessor(root)

result = []
# If the closest value from predecessor and successor are same, skip one to avoid duplicates.
if prede and succ and prede[-1].val == succ[-1].val:
    getPredecessor()

# Return k closest values
for i in range(k):
    if not prede:
        # Only successors left
        result.append(getSuccessor())
    elif not succ:
        # Only predecessors left
        result.append(getPredecessor())
    else:
        # Compare which candidate is closer to the target
        if abs(prede[-1].val - target) <= abs(succ[-1].val - target):
            result.append(getPredecessor())
        else:
            result.append(getSuccessor())
    return result

```

C++

```

// C++ solution for Closest Binary Search Tree Value II

#include <vector>
#include <stack>
using namespace std;

// Definition for a binary tree node.
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    vector<int> closestKvalues(TreeNode* root, double target, int k) {
        vector<TreeNode*> prede, succ;

        // Initialize predecessor stack
        auto initPredecessor = [&]()>TreeNode* node) {
            while (node) {
                if (node->val <= target) {
                    prede.push_back(node);
                    node = node->right;
                }
            }
        };

        // Initialize successor stack
        auto initSuccessor = [&]()>TreeNode* node) {
            while (node) {
                if (node->val > target) {
                    succ.push_back(node);
                    node = node->left;
                }
            }
        };

        initPredecessor(root);
        initSuccessor(root);

        // Helper lambda to get next predecessor and successor
        auto getPredecessor = [&]()>TreeNode* node) {
            if (prede.empty()) return nullptr;
            return prede.back();
        };
        auto getSuccessor = [&]()>TreeNode* node) {
            if (succ.empty()) return nullptr;
            return succ.back();
        };
    };
};

```

Heap · LeetCode — By Pattern — 1447 — LeetCode

Heap · LeetCode — By Pattern — 1449 — LeetCode

Heap · LeetCode — By Pattern — 1448 — LeetCode

Heap · LeetCode — By Pattern — 1450 — LeetCode

Heap · LeetMastery — By Pattern — 1451 — LeetMastery

Heap · LeetCode — By Pattern — 1452 — LeetCode

```

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
}

//
class MedianFinder {
public:
    // Initialize your data structure here.
    MedianFinder() {}

    void addNum(int num) {
        q1.push(num);
        q2.push(q1.top());
        q1.pop();
        if (q2.size() < q1.size() > 1) {
            q1.push(q2.top());
            q2.pop();
        }
    }

    double findMedian() {
        if (q1.size() > q2.size()) {
            return q1.top();
        }
        return (double) (q1.top() + q2.top()) / 2;
    }

private:
    priority_queue<int>, vector<int>, greater<int>> q1;
    priority_queue<int>, vector<int>, less<int>> q2;
};

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
}

```

Java

```

class MedianFinder {
public:
    void addNum(int num) {
        if (minHeap.empty()) minHeap.push(num);
        else {
            minHeap.offer(num);
            // Balance the two heaps
            if (minHeap.size() > maxHeap.size()) {
                minHeap.offer(maxHeap.poll());
            } else if (maxHeap.size() > minHeap.size() + 1) {
                maxHeap.offer(minHeap.poll());
            }
        }
    }

    double findMedian() {
        if (minHeap.size() == maxHeap.size()) {
            return (double) (minHeap.peek() + maxHeap.peek()) / 2.0;
        } else {
            return (double) minHeap.peek();
        }
    }

private:
    Queue<Integer> minHeap = new PriorityQueue<>();
    Queue<Integer> maxHeap = new PriorityQueue<>();
}

```

Java

```

public class MedianFinder {
    private PriorityQueue<int> minQ = new PriorityQueue<>();
    private PriorityQueue<int> maxQ = new PriorityQueue<>();
    private int count = 0;

    public MedianFinder() {}

    public void addNum(int num) {
        minQ.offer(num);
        maxQ.offer(num);
        if (minQ.size() > maxQ.size()) {
            minQ.offer(maxQ.poll());
        } else if (maxQ.size() > minQ.size()) {
            maxQ.offer(minQ.poll());
        }
    }

    public double findMedian() {
        return minQ.size() > maxQ.size() ? (minQ.peek() + maxQ.peek()) / 2.0 : minQ.peek();
    }
}

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
}

```

```

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
}

//
class MedianFinder {
    private List<Integer> nums;
    private int curIndex;

    // Initialize your data structure here.
    public MedianFinder() {
        nums = new ArrayList<>();
    }

    private int findIndex(int val) {
        int left = 0;
        int right = nums.size() - 1;
        while (left < right) {
            int mid = (left + right) / 2;
            if (nums.get(mid) < val) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return left;
    }

    public void addNum(int num) {
        if (nums.size() == 0) {
            nums.add(num);
            curIndex = 0;
        } else {
            curIndex = findIndex(num);
            if (curIndex == nums.size()) {
                nums.add(num);
            } else {
                nums.add(curIndex, num);
            }
        }
    }

    public double findMedian() {
        if (nums.size() % 2 == 1) {
            return (double) nums.get(nums.size() / 2);
        } else {
            if (nums.size() == 0) {
                return 0;
            }
            return (double) (nums.get(nums.size() / 2 - 1) + nums.get(nums.size() / 2)) / 2;
        }
    }
}

```

```

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
}

//
class MedianFinder {
public:
    // Initialize your data structure here.
    MedianFinder() {}

    void addNum(int num) {
        minQ.push(num);
        maxQ.push(minQ.top());
        minQ.pop();
        if (minQ.size() > maxQ.size()) {
            maxQ.push(minQ.top());
            minQ.pop();
        }
    }

    double findMedian() {
        if (minQ.size() > maxQ.size()) {
            return (double) minQ.top();
        } else {
            return (double) (minQ.top() + maxQ.top()) / 2;
        }
    }
};

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
}

```

JavaScript

```

/**
 * Initialize your data structure here.
 */
var MedianFinder = function() {
    this.min = [];
    this.max = [];
};

/**
 * @param {number} num
 * @return {void}
 */
MedianFinder.prototype.addNum = function(num) {
    let left = 0;
    let right = this.min.length;
    while (left < right) {
        let mid = (left + right) / 2;
        if (num < this.min[mid]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    this.min.splice(left, 0, num);
};

/**
 * @return {number}
 */
MedianFinder.prototype.findMedian = function() {
    let mid = Math.floor(this.min.length / 2);
    return (this.min[mid] + this.min[this.min.length - 1 - mid]) / 2;
};

```

TypeScript

TypeScript

```

class MedianFinder {
public:
    MedianFinder() {
        minQ = new MinPriorityQueue<number>();
        maxQ = new MaxPriorityQueue<number>();
    }

    addNum(number num) {
        const [minQ, maxQ] = [this.minQ, this.maxQ];
        minQ.enqueue(num);
        minQ.enqueue().dequeue();
        if (minQ.size() - maxQ.size() > 1) {
            maxQ.enqueue(minQ.dequeue());
        }
    }

    findMedian() {
        const [minQ, maxQ] = [this.minQ, this.maxQ];
        if (minQ.size() == maxQ.size()) {
            return (minQ.front() + maxQ.front()) / 2;
        }
        return minQ.front();
    }
};

// Your MedianFinder object will be instantiated and called as such:
// var obj = new MedianFinder();

// obj.addNum(num);
// var param_2 = obj.findMedian();
//

```

```

TypeScript
class MedianFinder {
    private nums: number[];

    constructor() {
        this.nums = [];
    }

    addNum(num: number): void {
        const { nums } = this;
        let l = 0;
        let r = nums.length;
        while (l < r) {
            const mid = (l + r) >> 1;
            if (nums[mid] < num) {
                l = mid + 1;
            } else {
                r = mid;
            }
        }
        nums.splice(l, 0, num);
    }

    findMedian(): number {
        const { nums } = this;
        const n = nums.length;
    }
}

```

```

if ((n & 1) == 1) {
    return nums[n >> 1];
}
return (nums[n >> 1] + nums[(n >> 1) - 1]) / 2;
}

//
// Your MedianFinder object will be instantiated and called as such:
// var obj = new MedianFinder();
// obj.addNum(num);
// var param_2 = obj.findMedian();
//

```

```

Rust
use std::cmp::Reverse;
use std::collections::BinaryHeap;

struct MedianFinder {
    minQ: BinaryHeap<Reverse<i32>>,
    maxQ: BinaryHeap<i32>,
}

impl MedianFinder {
    fn new() -> Self {
        MedianFinder {
            minQ: BinaryHeap::new(),
            maxQ: BinaryHeap::new(),
        }
    }

    fn add_num(&mut self, num: i32) {
        self.max.push(num);
        self.min.push(Reverse(self.max.pop().unwrap()));

        if self.min.len() > self.max.len() + 1 {
            self.max.push(self.min.pop().unwrap().0);
        }
    }

    fn find_median(&self) -> f64 {
        if self.min.len() == self.max.len() {
            let min_top = self.min.peek().unwrap().0;
            let max_top = self.max.peek().unwrap();
            (min_top + max_top) as f64 / 2.0
        } else {
            self.min.peek().unwrap().0 as f64
        }
    }
}

```

```

struct MedianFinder {
    nums: Vec<i32>,
}

//
// Self means the method takes an immutable reference.
// If you need a mutable reference, change it to &mut self instead.
//
impl MedianFinder {
    // initialize your data structure here.
    fn new() -> Self {
        Self { nums: Vec::new() }
    }

    fn add_num(&mut self, num: i32) {
        let mut l = 0;
        let mut r = self.nums.len();
        while l < r {
            let mid = (l + r) >> 1;
            if self.nums[mid] < num {
                l = mid + 1;
            } else {
                r = mid;
            }
        }
        self.nums.insert(l, num);
    }

    fn find_median(&self) -> f64 {
        let n = self.nums.len();
        if (n & 1) == 1 {
            return f64::from(self.nums[n >> 1]);
        }
        f64::from(self.nums[n >> 1] + self.nums[(n >> 1) - 1]) / 2.0
    }
}

// Your MedianFinder object will be instantiated and called as such:
// let obj = MedianFinder::new();
// obj.add_num(num);
// let res_2: f64 = obj.find_median();
//

```

[1] Heap — Pattern Solution (matched from community answers)
Python

```

import heapq

class MedianFinder:
    def __init__(self):
        # min-heap for the lower half (store negatives to simulate max-heap)
        self.lowerHalf = []
        # min-heap for the upper half
        self.upperHalf = []

    def addNum(self, num: int) -> None:
        # Push to min-heap (simulate by pushing negative)
        heapq.heappush(self.lowerHalf, -num)
        # Ensure every element in lowerHalf is >= every element in upperHalf
        # Move maximum of lowerHalf to upperHalf
        heapq.heappush(self.upperHalf, -heapq.heappop(self.lowerHalf))

        # Balance the number of elements between lowerHalf and size
        if len(self.upperHalf) > len(self.lowerHalf):
            heapq.heappush(self.lowerHalf, -heapq.heappop(self.upperHalf))

    def findMedian(self) -> float:
        # If both heaps are balanced, median is average of both heaps' top elements
        if len(self.lowerHalf) == len(self.upperHalf):
            return (-self.lowerHalf[0] + self.upperHalf[0]) / 2.0
        # Otherwise, median is the top element of the larger heap
        return -self.lowerHalf[0]

# Example usage:
# medianFinder = MedianFinder()
# medianFinder.addNum(1)
# medianFinder.addNum(2)
# print(medianFinder.findMedian()) # Outputs 1.5
# medianFinder.addNum(3)
# print(medianFinder.findMedian()) # Outputs 2.0

```

#358 **HARD**
Rearrange String k Distance Apart
LeetCode · Simplify · Stack · LeetCode · Editorial
Hash Table · String · Greedy · Sorting · Heap · Counting
List: Premium 98

Problem:
Given a string s and an integer k, rearrange s such that the same characters are at least distance k from each other. If it is not possible to rearrange the string, return an empty string "".

Example 1:
Input: s = "aabbcc", k = 3
Output: "abcabc"
Explanation: The same letters are at least a distance of 3 from each other.

Example 2:
Input: s = "aaabcc", k = 3
Output: ""

Explanation: It is not possible to rearrange the string.

Example 3:
Input: s = "aaabbbcc", k = 2
Output: "ababcbcc"
Explanation: The same letters are at least a distance of 2 from each other.

Constraints:

- 1 <= s.length <= 3 * 10⁵
- s consists of only lowercase English letters.
- 0 <= k <= s.length

```

Python
from collections import Counter, deque
import heapq

def rearrangeString(s: str, k: int) -> str:
    if k == 1:
        return s # No rearrangement needed if k is 1

    # Count the frequency of each character
    char_count = Counter(s)

    # Build a max-heap using negative frequencies
    max_heap = [(-freq, ch) for ch, freq in char_count.items()]
    heapq.heapify(max_heap)

    # Queue to keep track of characters waiting for k distance
    wait_queue = deque()
    result = []

    # Process the heap until it's empty
    while max_heap:
        freq, ch = heapq.heappop(max_heap)
        result.append(ch)

        # Decrease frequency (remember freq is negative)
        freq += 1

        # Add current char to the wait queue
        wait_queue.append((freq, ch))

        # If we've placed k characters, we can re-insert the oldest one if needed
        if len(wait_queue) < k:
            continue

        freq, front_ch, front = wait_queue.popleft()
        if front_ch != ch:
            heapq.heappush(max_heap, (freq, front_ch))

    rearranged = ''.join(result)
    return rearranged if len(rearranged) == len(s) else ""

```

```

class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        n = len(s)
        ans = []
        count = collections.Counter(s)
        # valid[i] is the leftmost index i can appear
        valid = collections.Counter()

        def getBestLetter(index: int) -> str:
            """Returns the valid letter that has the most count."""
            maxCount = -1
            bestLetter = ""
            for c in string.ascii_lowercase:
                if count[c] > 0 and count[c] > maxCount and index >= valid[c]:
                    bestLetter = c
                    maxCount = count[c]
            return bestLetter

        for i in range(n):
            c = getBestLetter(i)
            if c == '':
                return ""
            ans.append(c)
            count[c] -= 1
            valid[c] = i + k
            return ''.join(ans)

```

```

Python
class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        cnt = Counter(s)
        pq = [(v, c) for c, v in cnt.items()]
        heapq.heapify(pq)
        n = len(s)
        ans = []
        while pq:
            v, c = heapq.heappop(pq)
            ans.append(c)
            q.append((v - 1, c))
            if len(s) == k:
                v = q.popleft()
                if v > 0:
                    heapq.heappush(pq, v)
            return ''.join(ans) if len(s) == len(s) else ""

```



```

from heapq import heappify

class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        n = len(s)
        for c, v in Counter(s).items():
            heappify(s)
            q = deque()
            ans = []
            while k:
                v, c = heappop(s)
                v -= 1
                ans.append(c)
                k.append((v - 1, c)) # append even if 'v-1=0'
                if not v:
                    # our example had, then q size is 2, then q can pop one out
                    if len(q) > k: # guard: this is avoid you pick up same char in the same k-segment.
                        v, c = q.popleft()
                        if v, c == v, c:
                            heappush(s, (v, c))
            return "" if len(ans) > len(s) else "".join(ans)

```

```

#####
import collections
import heapq

class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        if k == 0:
            return s

        counts = collections.Counter(s)

        pq = []
        for ch, count in counts.items():
            heapq.heappush(pq, (-count, ch))

        ans = []
        while pq:
            top = []
            d = heapq.heappop(pq)
            for i in range(d[0]):
                if not pq:
                    return ""
                count, ch = heapq.heappop(pq)
                ch, count = ch, count + 1
                if count > 0: # pushed in as negated, so it's: if actual-position-count > 1
                    top.append((count, ch))
                s = s[1:]

            for count, ch in top:
                heapq.heappush(pq, (count, ch))

            ans += ch

        return "".join(ans)

```

C++

C++

Heap · LeetCode · By Pattern

— 1465 —

LeetCode

```

class Solution {
public:
    string rearrangeString(string s, int k) {
        vector<int> cnt(26, 0);
        for (char c : s) {
            cnt[c - 'a']++;
        }

        priority_queue<pair<int, int>, pair<int, int>> pq;
        for (int i = 0; i < 26; ++i) {
            if (cnt[i] > 0) {
                pq.emplace(cnt[i], i);
            }
        }

        queue<pair<int, int>> q;
        string ans;
        while (pq.empty()) {
            auto p = pq.top();
            pq.pop();
            p.first--;
            ans.push_back('a' + p.second);
            q.push(p);
            if (q.size() == k) {
                p = q.front();
                q.pop();
                if (p.first > 0) {
                    pq.push(p);
                }
            }
            return ans.size() < s.size() ? "" : ans;
        }
    };
}

```

C++

Heap · LeetCode · By Pattern

— 1466 —

LeetCode

```

class Solution {
public:
    string rearrangeString(string s, int k) {
        unordered_map<char, int> cnt;
        for (char c : s) ++cnt[c];
        priority_queue<pair<int, char>> pq;
        for (auto [c, v] : cnt) pq.push({v, c});
        queue<pair<int, char>> q;
        string ans;
        while (!pq.empty()) {
            auto [v, c] = pq.top();
            pq.pop();
            ans += c;
            q.push({v - 1, c});
            if (q.size() == k) {
                auto p = q.front();
                q.pop();
                if (p.first) {
                    pq.push(p);
                }
            }
        }
        return ans.size() == s.size() ? ans : "";
    };
}

```

Java

```

class Solution {
    public String rearrangeString(String s, int k) {
        int n = s.length();
        int[] cnt = new int[26];
        for (char c : s.toCharArray()) {
            ++cnt[c - 'a'];
        }
        PriorityQueue<int[]> pq = new PriorityQueue<>(a, b) -> b[0] - a[0]);
        for (int i = 0; i < 26; ++i) {
            if (cnt[i] > 0) {
                pq.offer(new int[] { cnt[i], i });
            }
        }
        Deque<int[]> q = new ArrayDeque<>();
        StringBuilder ans = new StringBuilder();
        while (pq.isEmpty()) {
            var p = pq.poll();
            int v = p[0], c = p[1];
            ans.append(char { 'a' + c });
            q.offer(new int[] { v - 1, c });
            if (q.size() == k) {
                p = q.pollFirst();
                if (p[0] > 0) {
                    pq.offer(p);
                }
            }
        }
    }
}

```

Java

Heap · LeetCode · By Pattern

— 1467 —

LeetCode

```

    }
    return ans.length() == n ? ans.toString() : "";
}

}

```

Java

```

export function rearrangeString(s: string, k: number): string {
    const cnt: number[] = Array(26).fill(0);
    for (const c of s) {
        cnt[c.charCodeAt(0) - 'a'.charCodeAt(0)]++;
    }

    const pq = new PriorityQueue<number, number>((a, b) => b[0] - a[0]);
    for (let i = 0; i < 26; i++) {
        if (cnt[i] > 0) {
            pq.enqueue([cnt[i], i]);
        }
    }

    const q: [number, number][] = [];
    const ans: string[] = [];
    while (pq.length) {
        const [count, id] = pq.dequeue();
        const newCount = count - 1;
        ans.push(String.fromCharCode('a'.charCodeAt(0) + id));
        q.push([newCount, id]);
        if (q.length == k) {
            const [frontCount, frontId] = q.shift();
            if (frontCount > 0) {
                pq.enqueue([frontCount, frontId]);
            }
        }
    }
    return ans.length < s.length ? "" : ans.join("");
}

```

Go

Go

Heap · LeetCode · By Pattern

— 1468 —

LeetCode

```

func rearrangeString(s string, k int) string {
    cnt := map[byte]int{}
    for i := range s {
        cnt[s[i]]++
    }
    pq := hp{}
    for c, v := range cnt {
        heap.Push(&pq, pair{v, c})
    }
    ans := []byte{}
    q := []pair{}
    for len(pq) > 0 {
        p := heap.Pop(&pq).(pair)
        v, c := p.v, p.c
        ans = append(ans, c)
        q = append(q, pair{v-1, c})
        if len(q) == k {
            p = q[0]
            q = q[1:]
            if p.v > 0 {
                heap.Push(&pq, p)
            }
        }
    }
    if len(ans) == len(s) {
        return string(ans)
    }
    return ""
}

type pair struct {
    v int
    c byte
}

type hp []pair

func (h hp) Len() int { return len(h) }
func (h hp) Less(i, j int) bool {
    a, b := h[i].v, h[j].v
    return a > b
}
func (h hp) Swap(i, j int) { h[i].v, h[j].v = h[j].v, h[i].v }
func (h hp) Push(p pair) { h = append(h, p).(pair) }
func (h hp) Pop() pair { a := h[0]; h = h[1:]; return a }

```

[7] Heap — Pattern Solution (matched from community answers)

Python

Heap · LeetCode · By Pattern

— 1469 —

LeetCode

```

from collections import Counter, deque
import heapq

def rearrangeString(s: str, k: int) -> str:
    if k == 0:
        return s # No rearrangement needed if k is 0

    # Count the frequency of each character
    char_count = Counter(s)

    # Build a max-heap using negative frequencies
    max_heap = [(-freq, ch) for ch, freq in char_count.items()]
    heapq.heapify(max_heap)

    # Buffer to keep track of characters waiting for k distance
    wait_queue = deque()

    # Process the heap until it's empty
    while max_heap:
        freq, ch = heapq.heappop(max_heap)
        result.append(ch)

        # Decrease frequency (remember freq is negative)
        freq += 1

        # Add current char to the wait queue
        wait_queue.append((freq, ch))

        # If we've placed k characters, we can re-insert the oldest one if needed
        if len(wait_queue) == k:
            continue

        freq, front, ch_front = wait_queue.popleft()
        if freq > 0:
            heapq.heappush(max_heap, (freq, front, ch_front))

    rearranged = "".join(result)
    return rearranged if len(rearranged) == len(s) else ""

```

#499

HARD

The Maze III

LeetCode · SimpleList · WdNcE · LeetCode · Editorial

Array · DFS · BFS · Graph · Matrix · Heap · Shortest Path

List: Premium 98

Problem:

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction (must be different from last chosen direction). There is also a hole in this maze. The ball will drop into the hole if it rolls onto the hole.

Heap · LeetCode · By Pattern

— 1470 —

LeetCode

Java
Java

Heap · LeetCode — By Pattern

— 1477 —

LeetMastery

java

Heap · LeetCode — By Pattern

— 1478 —

LeetMastery

Go

Heap · LeetCode — By Pattern

— 1479 —

LeetMastery

[6] Heap — Pattern Solution (matched from community answers)

Heap · LeetCode — By Pattern

— 1480 —

LeetMastery

Heap · LeetCode — By Pattern

— 1481 —

LeetMastery

#632 **HARD**

Smallest Range Covering Elements from K Lists

[LeetCode](#) · [SimpleList](#) · [WalkCF](#) · [LeetCode](#) · [Editorial](#)

Array - Hash Table - Greedy - Heap - Sliding Window - Sorting

Heap: 350

You have k lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the k lists.

We define the range $[a, b]$ is smaller than range $[c, d]$ if $b - a < d - c$ or $a < c$ if $b - a == d - c$.

Example 1:

```
Input: nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
Output: [20,24]
```

Explanation:
List 1: [4, 10, 15, 24, 26], 24 is in range [20, 24].

```
List 2: [0, 9, 12, 20], 20 is in range [20,24].
List 3: [5, 18, 22, 30], 22 is in range [20,24]
```

Example 2:

Input: nums = [[1,2,3],[1,2,3]]
Output: [1,1]

Constraints:

- `nums.length == k`

- $1 \leq k \leq 3500$

- `1 <= nums[i].length <= 50`
- `-105 <= nums[i][j] <= 105`

- `nums[i]` is sorted in non-decreasing order

>> Brute Force [Heap] Interview

```
# Brute Force Approach
class Solution:
```

```
def smallestRange(self, nums):
    # Brute Force  $O(n \log n)$ 
```

```
data = list(nums)
result = []
for i in range(k):
```

```
data.sort()
result.append(data.po
```

```
return result
```

Python

Heap · LeetCode — By Pattern

— 1482 —

LeetMastery

```

import heapq

def smallestRange(nums):
    # Initialize the min-heap and current maximum value
    heap = []
    current_max = float('-inf')

    # Add the first element of each list to the heap
    for i, list in enumerate(nums):
        value = list[0]
        heapq.heappush(heap, (value, i, 0))
        current_max = max(current_max, value)

    best_range = [float('-inf'), float('inf')]

    # Process until one list is exhausted
    while heap:
        current_min, list_idx, elem_idx = heapq.heappop(heap)

        # Update the best range if the current range is smaller
        if current_max - current_min < best_range[1] - best_range[0] or \
            (current_max - current_min == best_range[1] - best_range[0] and current_min < best_range[0]):
            best_range = [current_min, current_max]

        # If there is no next element in the same list, break out
        if elem_idx + 1 == len(nums[list_idx]):
            break
        next_value = nums[list_idx][elem_idx + 1]
        heapq.heappush(heap, (next_value, list_idx, elem_idx + 1))
        current_max = max(current_max, next_value)

    return best_range

# Example Usage:
nums = [[4,10,15,16,20], [9,12,20], [5,18,22,30]]
print(smallestRange(nums))

```

Python

```

class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        minheap = [(row[0], i, 0) for i, row in enumerate(nums)]
        heapq.heapify(minheap)

        maxheap = max(row[0] for row in nums)
        minheap = heapq.nsmallest(1, minheap)[0][0]
        ans = [minheap, maxheap]

        while len(minheap) == len(nums):
            min, r, c = heapq.heappop(minheap)
            if c + 1 < len(nums[r]):
                heapq.heappush(minheap, (nums[r][c + 1], r, c + 1))
                minheap = ans[0], ans[1][c + 1]
                minheap = heapq.nsmallest(1, minheap)[0][0]
            if minheap < minheap < ans[1] - ans[0]:
                ans[0], ans[1] = minheap, maxheap

        return ans

```

Heap · LeetCode — By Pattern

— 1483 —

LeetCode

Python

```

class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, x in enumerate(nums)]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for i, (x, v) in enumerate(t):
            cnt[i] += 1
            while len(cnt) == len(nums):
                a = t[i][0]
                x = b = a + (ans[1] - ans[0])
                if x < 0 or (x == 0 and a < ans[0]):
                    ans = [a, b]
                w = t[i][1]
                cnt[w] -= 1
                if cnt[w] == 0:
                    cnt.pop(w)
                    j += 1
            return ans

```

Python

```

class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, x in enumerate(nums)]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for k, v in t:
            cnt[k] += 1
            while len(cnt) == len(nums):
                a = t[j][0]
                x = b = a + (ans[1] - ans[0])
                if x < 0 or (x == 0 and a < ans[0]):
                    ans = [a, b]
                w = t[j][1]
                cnt[w] -= 1
                if cnt[w] == 0:
                    cnt.pop(w)
                    j += 1
            return ans

```

C++

C++

Heap · LeetCode — By Pattern

— 1484 —

LeetCode

```

class Solution {
public:
    vector<int> smallestRange(vector<vector<int>>& nums) {
        int n = 0;
        for (const v : nums) n = v.size();
        vector<pair<int, int>> t(n);
        int k = nums.size();
        for (int i = 0; i < n; i += 1) {
            for (int v : nums[i]) {
                t[i] += (v, i);
            }
        }
        sort(t.begin(), t.end());
        int j = 0;
        unordered_map<int, int> cnt;
        vector<int> ans = {-100000, 100000};
        for (int i = 0; i < n; ++i) {
            int b = t[i].first;
            int v = t[i].second;
            cnt[v]++;
            while (cnt.size() == k) {
                int a = t[j].first;
                int w = t[j].second;
                int x = b - a + (ans[1] - ans[0]);
                if (x < 0 || (x == 0 && a < ans[0])) {
                    ans[0] = a;
                    ans[1] = b;
                }
                if (--cnt[w] == 0) {
                    cnt.erase(w);
                }
                ++j;
            }
        }
        return ans;
    }
};

```

Java

Java

Heap · LeetCode — By Pattern

— 1485 —

LeetCode

```

const smallestRange = nums => {
    const pq = new PriorityQueue((a, b) => a[0] - b[0]);
    const n = nums.length;
    let [l, r, max] = [0, Number.POSITIVE_INFINITY, Number.NEGATIVE_INFINITY];

    for (let j = 0; j < n; j++) {
        const x = nums[j][0];
        pq.enqueue([x, j, 0]);
        max = Math.max(max, x);
    }

    while (pq.size() === n) {
        const [idx, j, i] = pq.dequeue();
        const [a, b, c] = [l, r, max];

        if (max - min < r - l) {
            [l, r] = [a, max];
        }

        const [next, i] = t;
        if (i === n) {
            const next = nums[j][1];
            pq.enqueue([next, j, next]);
            max = Math.max(max, next);
        }
    }

    return [l, r];
};

```

Java

```

class Solution {
    public List<int> smallestRange(List<List<Integer>> nums) {
        int n = 0;
        for (var v : nums) {
            n = v.size();
        }
        List<int> t = new List<int>[n];
        int k = nums.size();
        for (int i = 0; i < n; i += 1) {
            for (int v : nums.get(i)) {
                t[i] += new List<int>((v, i));
            }
        }
        Arrays.sort(t, (a, b) -> a[0] - b[0]);
        int j = 0;
        Map<Integer, Integer> cnt = new HashMap<>();
        int[] ans = new int[] { -100000, 100000 };
        for (int i = 0; i < n; i += 1) {
            int b = t[i][0];
            int v = t[i][1];
            cnt.put(v, cnt.getOrDefault(v, 0) + 1);
            while (cnt.size() == k) {
                int a = t[j][0];
                int w = t[j][1];
                int x = b - a + (ans[1] - ans[0]);
                if (x < 0 || (x == 0 && a < ans[0])) {
                    ans[0] = a;
                    ans[1] = b;
                }
                if (--cnt[w] == 0) {
                    cnt.remove(w);
                }
                ++j;
            }
        }
        return ans;
    }
}

```

Heap · LeetCode — By Pattern

— 1486 —

LeetCode

```

        if (x < 0 || (x == 0 && a < ans[0])) {
            ans[0] = a;
            ans[1] = b;
        }
        cnt.put(v, cnt.get(v) + 1);
        if (cnt.get(v) == 0) {
            cnt.remove(v);
        }
        ++j;
    }
    return ans;
}

```

Java

```

impl Solution {
    pub fn smallestRange(nums: Vec<Vec<i32>>) -> Vec<i32> {
        let mut t = vec![];
        for (i, v) in nums.iter().enumerate() {
            for &v in v {
                t.push((v, i));
            }
        }
        t.sort_unstable();
        let (mut ans, 0) = (vec![-100000, 100000], nums.len());
        let mut j = 0;
        let mut cnt = std::collections::HashMap::new();

        for (b, v) in t.iter() {
            let (a, w) = (t[j].0, t[j].1);
            if let Some(x) = cnt.get_mut(w) {
                *x += 1;
            } else {
                cnt.insert(w, 1);
            }
            while cnt.len() == n {
                let (a, w) = t[j];
                let x = b - a + (ans[1] - ans[0]);
                if x < 0 || (x == 0 && a < ans[0]) {
                    ans = vec![a, b];
                }
                if let Some(x) = cnt.get_mut(w) {
                    *x -= 1;
                }
                if cnt[w] == 0 {
                    cnt.remove(w);
                }
                j += 1;
            }
        }
        ans
    }
}

```

Go

Heap · LeetCode — By Pattern

— 1487 —

LeetCode

```

func smallestRange(nums [][]int) []int {
    t := [][]int{}
    for i, n := range nums {
        for _, v := range n {
            t = append(t, []int{v, i})
        }
    }
    sort.Slice(t, func(i, j int) bool { return t[i][0] < t[j][0] })
    ans := []int{-100000, 100000}
    j := 0
    cnt := map[int]int{}
    for _, v := range t {
        b, v = v[0], v[1]
        cnt[v]++
        for len(cnt) == len(nums) {
            a, w = t[j][0], t[j][1]
            x = b - a + (ans[1] - ans[0])
            if x < 0 || (x == 0 && a < ans[0]) {
                ans[0], ans[1] = a, b
            }
            cnt[w]--
            if cnt[w] == 0 {
                delete(cnt, w)
            }
            j++
        }
    }
    return ans
}

```

[*] Heap — Pattern Solution (matched from community answers)

Python

Heap · LeetCode — By Pattern

— 1488 —

LeetCode

```

import heapq

def smallestRange(nums):
    # Initialize the min-heap and current maximum value
    heap = []
    current_max = float('-inf')

    # Add the first element of each list to the heap
    for i, list in enumerate(nums):
        value = list[0]
        heapq.heappush(heap, (value, i, 0))
        current_max = max(current_max, value)

    best_range = [(float('-inf'), float('-inf'))]

    # Process until one list is exhausted
    while heap:
        current_min, list_idx, elem_idx = heapq.heappop(heap)

        # Update the best range if the current range is smaller
        if current_max - current_min < best_range[1] - best_range[0] or \
            (current_max - current_min == best_range[1] - best_range[0] and current_min < best_range[0]):
            best_range = [current_min, current_max]

        # If there is no next element in the same list, break out
        if elem_idx + 1 == len(nums[list_idx]):
            break
        next_value = nums[list_idx][elem_idx + 1]
        heapq.heappush(heap, (next_value, list_idx, elem_idx + 1))
        current_max = max(current_max, next_value)

    return best_range

# Example usage:
nums = [[4,10,15,16,20], [9,12,20], [5,18,22,30]]
print(smallestRange(nums))

```

#759

HARD

Employee Free Time

LeetCode · SimplifyTag · WalkCC · LeetCode · Editorial

Array · Sorting · Heap
Views Grid 169

Problem:

We are given a list schedule of employees, which represents the working time for each employee.

Each employee has a list of non-overlapping Intervals, and these intervals are in sorted order.

Return the list of finite intervals representing common, positive-length free time for all employees, also in sorted order.

(Even though we are representing Intervals in the form [x, y], the objects inside are Intervals, not lists or arrays. For example, schedule[0][0].start = 1, schedule[0][0].end = 2, and schedule[0][0][0] is not defined). Also, we wouldn't include intervals like [5, 5] in our answer, as they have zero length.

Heap · LeetCode · By Pattern

— 1489 —

LeetCode

```

prev = interval
merged.append(prev)

# Collect free times from the gaps between merged intervals
freeTimes = []
for i in range(1, len(merged)):
    # Only intervals with positive length are added
    if merged[i].start > merged[i-1].end:
        freeTimes.append(Interval(merged[i-1].end, merged[i].start))

return freeTimes

# Example usage:
# schedule = [
#   [Interval(1,2), Interval(5,6)],
#   [Interval(1,3)],
#   [Interval(4,8)]
# ]
# freeTimes = employeeFreeTime(schedule)
# for interval in freeTimes:
#   print(f"[{interval.start}, {interval.end}]")

```

Python

```

class Solution:
    def employeeFreeTime(self, schedule: "[Interval]") -> "[Interval]":
        ans = []
        intervals = []

        for s in schedule:
            intervals.extend(s)
            intervals.sort(key=lambda x: x.start)

        prevEnd = intervals[0].end

        for interval in intervals:
            if interval.start > prevEnd:
                ans.append(Interval(prevEnd, interval.start))
                prevEnd = max(prevEnd, interval.end)

        return ans

```

Python

Heap · LeetCode · By Pattern

— 1491 —

LeetCode

```

class Solution {
public:
    vector<Interval> employeeFreeTime(vector<vector<Interval>>& schedule) {
        vector<Interval> ans;
        vector<Interval> intervals;

        for (const vector<Interval>& s : schedule)
            intervals.insert(intervals.end(), s.begin(), s.end());

        ranges::sort(intervals, ranges::less{
            [](const Interval& interval) { return interval.start; }
        });

        int prevEnd = intervals[0].end;

        for (const auto& [start, end] : intervals) {
            if (start > prevEnd)
                ans.emplace_back(prevEnd, start);
            prevEnd = max(prevEnd, end);
        }

        return ans;
    }
};

```

C++

```

// O( N log N ) solution, complexity is O(N log N)
// Time: O(N log N + T) where N is the total number of intervals, and T is the total number of unique times.
// Space: O(1)
class Solution {
public:
    vector<Interval> employeeFreeTime(vector<vector<Interval>>& A) {
        map<int, int> m;
        for (auto & s : A) {
            for (auto & it : s) {
                m[it.start]++;
                m[it.end]--;
            }
        }
        vector<Interval> ans;
        int cur = 0;
        for (auto it = m.begin(); it != m.end(); ++it) {
            cur += it->second;
            if (cur > 0) continue;
            int start = it->first;
            while (it != m.end()) {
                if (it->second == 0) break;
                cur += it->second;
                ans.emplace_back(start, it->first);
            }
            return ans;
        }
    }
};

```

Java

Java

Heap · LeetCode · By Pattern

— 1493 —

LeetCode

Example 1:

Input: schedule = [[[1,2],[5,6]],[[1,3]],[[4,10]]]

Output: [[3,4]]

Explanation: There are a total of three employees, and all common free time intervals would be [1-inf, 1], [3, 4], [10, inf]. We discard any intervals that contain inf as they aren't finite.

Example 2:

Input: schedule = [[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]

Output: [[5,6],[7,9]]

Constraints:

- 1 <= schedule.length, schedule[i].length <= 50
- 0 <= schedule[i].start < schedule[i].end <= 10^8

Python

Python

```

# Class definition for an Interval
class Interval:
    def __init__(self, start, end):
        self.start = start # Start time of the interval
        self.end = end # End time of the interval

def employeeFreeTime(schedule):
    # Flatten the schedule, extract all intervals from each employee
    allIntervals = []
    for employee in schedule:
        for interval in employee:
            allIntervals.append(interval)

    # Sort intervals by start time
    allIntervals.sort(key=lambda interval: interval.start)

    # Merge overlapping intervals
    merged = []
    prev = allIntervals[0]
    for interval in allIntervals[1:]:
        if interval.start <= prev.end:
            # Extend the previous interval if there is an overlap
            prev.end = max(prev.end, interval.end)
        else:
            merged.append(prev)
            prev = interval
    merged.append(prev)

```

Heap · LeetCode · By Pattern

— 1490 —

LeetCode

```

// Definition for an Interval.
class Interval {
public:
    Interval(int start, int end, int = 0) : start(start), end(end) {}
};

// Solution
class Solution {
public:
    vector<Interval> employeeFreeTime(vector<vector<Interval>>& schedule) {
        // Flatten the schedule
        vector<Interval> allIntervals;
        for (auto& s : schedule)
            allIntervals.insert(allIntervals.end(), s.begin(), s.end());

        // Sort intervals by start time
        sort(allIntervals.begin(), allIntervals.end(), [](Interval& a, Interval& b) {
            return a.start < b.start;
        });

        // Merge overlapping intervals
        vector<Interval> merged;
        if (!allIntervals.empty())
            merged.push_back(allIntervals[0]);

        for (int i = 1; i < allIntervals.size(); ++i) {
            Interval& prev = merged.back();
            Interval& curr = allIntervals[i];
            if (curr.start <= prev.end)
                prev.end = max(prev.end, curr.end);
            else
                merged.push_back(curr);
        }

        // Extract free time intervals
        vector<Interval> ans;
        for (int i = 1; i < merged.size(); ++i) {
            ans.push_back(Interval(merged[i-1].end, merged[i].start));
        }

        return ans;
    }
};

```

Python

```

// Definition for an Interval.
class Interval {
public:
    Interval(int start, int end, int = 0) : start(start), end(end) {}
};

// Solution
class Solution {
public:
    vector<Interval> employeeFreeTime(vector<vector<Interval>>& schedule) {
        // Flatten the schedule
        vector<Interval> allIntervals;
        for (auto& s : schedule)
            allIntervals.insert(allIntervals.end(), s.begin(), s.end());

        // Sort intervals by start time
        sort(allIntervals.begin(), allIntervals.end(), [](Interval& a, Interval& b) {
            return a.start < b.start;
        });

        // Merge overlapping intervals
        vector<Interval> merged;
        if (!allIntervals.empty())
            merged.push_back(allIntervals[0]);

        for (int i = 1; i < allIntervals.size(); ++i) {
            Interval& prev = merged.back();
            Interval& curr = allIntervals[i];
            if (curr.start <= prev.end)
                prev.end = max(prev.end, curr.end);
            else
                merged.push_back(curr);
        }

        // Extract free time intervals
        vector<Interval> ans;
        for (int i = 1; i < merged.size(); ++i) {
            ans.push_back(Interval(merged[i-1].end, merged[i].start));
        }

        return ans;
    }
};

```

C++

C++

Heap · LeetCode · By Pattern

— 1492 —

LeetCode

```

class Solution {
public:
    List<Interval> employeeFreeTime(List<List<Interval>>& schedule) {
        List<Interval> ans = new ArrayList();
        List<Interval> intervals = new ArrayList();

        schedule.forEach(s -> intervals.addAll(s));
        Collections.sort(intervals, Comparator.comparingInt(Interval -> Interval.start));

        int prevEnd = intervals.get(0).end;

        for (Interval interval : intervals) {
            if (interval.start > prevEnd)
                ans.add(new Interval(prevEnd, interval.start));
            prevEnd = Math.max(prevEnd, interval.end);
        }

        return ans;
    }
}

```

Java

```

// Definition for an Interval.
class Interval {
public:
    Interval(int start, int end, int = 0) : start(start), end(end) {}
};

// Solution
class Solution {
public:
    List<Interval> employeeFreeTime(List<List<Interval>>& schedule) {
        List<Interval> allIntervals = new ArrayList<Interval>();
        for (List<Interval> list : schedule)
            allIntervals.addAll(list);

        Collections.sort(allIntervals, new Comparator<Interval>() {
            public int compare(Interval interval1, Interval interval2) {
                if (interval1.start < interval2.start)
                    return interval1.start - interval2.start;
                else
                    return interval1.end - interval2.end;
            }
        });
    }
}

```

Heap · LeetCode · By Pattern

— 1494 —

LeetCode

```

    }
    ListInterval sorted = new ArrayList<Interval>();
    Interval interval0 = allSchedules.get(0);
    int curStart = interval0.start, curEnd = interval0.end;
    int size = allSchedules.size();
    for (int i = 1; i < size; i++) {
        Interval interval = allSchedules.get(i);
        if (interval.start == curEnd)
            curEnd = Math.max(curEnd, interval.end);
        else {
            sorted.add(new Interval(curStart, curEnd));
            curStart = interval.start;
            curEnd = interval.end;
        }
    }
    sorted.add(new Interval(curStart, curEnd));
    ListInterval freeTimeList = new ArrayList<Interval>();
    int curTimeLine = sorted.size();
    for (int i = 1; i < sorted.size(); i++)
        freeTimeList.add(new Interval(sorted.get(i - 1).end, sorted.get(i).start));
    return freeTimeList;
}

```

TypeScript

TypeScript

```

// Definition for an Interval.
class Interval {
    + start: number;
    + end: number;
    constructor(start: number, end: number) {
        this.start = start;
        this.end = end;
    }
}

function employeeFreeTime(schedules: Interval[][]): Interval[] {
    const intervals: Interval[] = [];
    for (const s of schedules) {
        intervals.push(...s);
    }
    intervals.sort((a, b) => (a.start === b.start ? a.end - b.end : a.start - b.start));

    const merged: Interval[] = [intervals[0]];
    for (let i = 1; i < intervals.length; i++) {
        const last = merged[merged.length - 1];
        const cur = intervals[i];
        if (last.end <= cur.start) {
            merged.push(cur);
        }
    }
}

```

```

    } else {
        last.end = Math.max(last.end, cur.end);
    }
}

const ans: Interval[] = [];
for (let i = 1; i < merged.length; i++) {
    const a = merged[i - 1];
    const b = merged[i];
    ans.push(new Interval(a.end, b.start));
}

return ans;
}

```

[*] Heap — Pattern Solution (stored optimal)

Python

```

# Class Definition for Interval.
class Interval:
    def __init__(self, start, end):
        self.start = start # Start time of the interval
        self.end = end # End time of the interval

def employeeFreeTime(schedules):
    # Extract the schedule, extract all intervals from each employee
    allIntervals = []
    for employee in schedules:
        for interval in employee:
            allIntervals.append(interval)

    # Sort intervals by start time
    allIntervals.sort(key=lambda interval: interval.start)

    # Merge overlapping intervals
    merged = []
    prev = allIntervals[0]
    for interval in allIntervals[1:]:
        if interval.start == prev.end:
            # Extend the previous interval, if there is an overlap
            prev.end = max(prev.end, interval.end)
        else:
            merged.append(prev)

```

```

prev = interval
merged.append(prev)

# Collect free times from the gaps between merged intervals
freeTimes = []
for i in range(1, len(merged)):
    # Only intervals with non-zero length are added
    if merged[i].start > merged[i-1].end:
        freeTimes.append(Interval(merged[i-1].end, merged[i].start))

return freeTimes

# Example usage:
# schedule = [
#     [Interval(1,2), Interval(5,6)],
#     [Interval(1,3)],
#     [Interval(4,10)]
# ]
# freeTimes = employeeFreeTime(schedule)
# for interval in freeTimes:
#     print(f"[{interval.start}, {interval.end}]")

```

C++

```

// Include necessary libraries
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Definition for an interval.
struct Interval {
    int start;
    int end;
    Interval(int s, int e) : start(s), end(e) {}
};

vector<Interval> employeeFreeTime(vector<vector<Interval>>& schedules) {
    // Extract the schedule into a single vector
    vector<Interval> allIntervals;
    for (const auto& employee : schedules) {
        for (const auto& interval : employee) {
            allIntervals.push_back(interval);
        }
    }

    // Sort intervals based on start time
    sort(allIntervals.begin(), allIntervals.end());
    return {Interval(s, t) : s < t};
}

```

```

});

// Merge overlapping intervals
vector<Interval> merged;
Interval prev = allIntervals[0];
for (int i = 1; i < allIntervals.size(); i++) {
    if (allIntervals[i].start == prev.end) {
        // Merge intervals by updating the end time
        prev.end = max(prev.end, allIntervals[i].end);
    } else {
        merged.push_back(prev);
        prev = allIntervals[i];
    }
}
merged.push_back(prev);

// Find the free intervals as gaps between merged intervals
vector<Interval> freeTimes;
for (int i = 1; i < merged.size(); i++) {
    if (merged[i].start > merged[i-1].end) {
        freeTimes.push_back(Interval(merged[i-1].end, merged[i].start));
    }
}

return freeTimes;
}

```

```

// Example usage:
// List data[] :
// vector<vector<Interval>> schedule = {
//     { Interval(1,2), Interval(5,6) },
//     { Interval(1,3) },
//     { Interval(4,10) }
// };
// vector<Interval> freeTimes = employeeFreeTime(schedule);
// for (const auto& interval : freeTimes) {
//     cout << "[" << interval.start << ", " << interval.end << "]" << endl;
// }
// Return 0;
}

```

#1425

HARD

Constrained Subsequence Sum

LeetCode · Simplex · Waffle · LeetCode · Editorial

Array · Dynamic Programming · Queue · Sliding Window · Heap · Monotonic Queue

List Denny Zhang

Problem:

Given an integer array `nums` and an integer `k`, return the maximum sum of a non-empty subsequence of that array such that for every two consecutive integers in the subsequence, `nums[i]` and `nums[j]`, where $i < j$, the condition $j - i \leq k$ is satisfied.

A subsequence of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Example 1:

Input: `nums = [10,2,-10,5,20]`, `k = 2`

Output: 37

Explanation: The subsequence is [10, 2, 5, 20].

Example 2:

Input: `nums = [-1,-2,-3]`, `k = 1`

Output: -1

Explanation: The subsequence must be non-empty, so we choose the largest number.

Example 3:

Input: `nums = [10,-2,-10,-5,20]`, `k = 2`

Output: 23

Explanation: The subsequence is [10, -2, -5, 20].

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

>> Brute Force [Heap] Interview Prep

Python

Python

Python

```

# Brute Force Approach
class Solution:
    def constrainedSubsequenceSum(self, nums, k):
        # Brute Force (log n) - sort repeatedly instead of heap
        data = list(nums)
        result = 0
        for i in range(k):
            data.sort()
            result += data.pop(0)
        return result

```

Python

Python

```

# Python solution using a monotonic deque for efficient window maximum computation.
from collections import deque

def constrainedSubsequenceSum(nums, k):
    n = len(nums)
    dp = [0] * n # dp[i] stores the maximum subsequence sum ending at index i
    dp[0] = nums[0]
    result = dp[0]

    # Deque to store indices in a decreasing order of dp values
    dq = deque([0])

    for i in range(1, n):
        # Remove indices from the front which are out of the window
        while dq and i - dq[0] > k:
            dq.popleft()

        # The maximum dp in the window is at the front of the deque
        max_in_window = dp[dq[0]] if dq else 0

        # Calculate dp[i] by either starting a new subsequence or extending the best one from the window
        dp[i] = max(dp[i], max_in_window + nums[i])

        # Update result (max)
        result = max(result, dp[i])

    # Maintain the deque monotonically so that dp indices are in decreasing order
    while dq and dp[i] >= dp[dq[-1]]:
        dq.pop()
    dq.append(i)

    return result

# Example usage:
# print(constrainedSubsequenceSum([10,2,-10,5,20], 2))

```

Python

```

class Solution:
    def constrainedSubsequenceSum(self, nums: List[int], k: int) -> int:
        dp = [0] * len(nums)
        # dp stores dp[i-1], dp[i-2], ..., dp[i-k] whose values are > 0
        # in decreasing order
        dq = collections.deque()

        for i, num in enumerate(nums):
            if dq:
                dp[i] = max(dp[i], 0) + num
            else:
                dp[i] = num
            while dq and dp[i-1] < dp[i]:
                dq.pop()
            dq.append(dp[i])
            if i < k and dp[i-k] >= dp[i]:
                dq.popleft()

        return max(dp)

```

```

Python
class Solution:
    def constrainedSubsetSum(self, nums: List[int], k: int) -> int:
        q = deque([0])
        n = len(nums)
        f = [0] * n
        ans = -inf
        for i, v in enumerate(nums):
            while i - q[0] > k:
                q.popleft()
            f[i] = max(0, f[q[0]]) + v
            ans = max(ans, f[i])
            while q and f[q[-1]] <= f[i]:
                q.pop()
            q.append(i)
        return ans

Python
class Solution:
    def constrainedSubsetSum(self, nums: List[int], k: int) -> int:
        n = len(nums)
        dp = [0] * n
        ans = -inf
        q = deque()
        for i, v in enumerate(nums):
            if q and i - q[0] > k:
                q.popleft()
            dp[i] = max(0, dp[q[0]] if q else dp[q[0]] + v)
            while q and dp[q[-1]] <= dp[i]:
                q.pop()
            q.append(i)
            ans = max(ans, dp[i])
        return ans

C++
C++

```

Heap · LeetCode - By Pattern — 1501 — LeetCode

```

class Solution {
public:
    int constrainedSubsetSum(vector<int>& nums, int k) {
        // dp[i] = the maximum the sum of non-empty subsequences in nums[0...i]
        vector<int> dp(nums.size());
        // q stores dp[0], dp[1], ..., dp[k-1] whose values are > 0
        // in decreasing order.
        deque<int> dq;
        for (int i = 0; i < nums.size(); ++i) {
            if (dq.empty())
                dp[i] = nums[i];
            else
                dp[i] = max(dq.front(), 0) + nums[i];
            while (!dq.empty() && dq.back() <= dp[i])
                dq.pop_back();
            dq.push_back(dp[i]);
            if (i > k && dp[i - k] <= dq.front())
                dq.pop_front();
        }
        return ranges::max(dp);
    }
};

C++
C++

```

Heap · LeetCode - By Pattern — 1502 — LeetCode

```

class Solution {
public:
    int constrainedSubsetSum(vector<int>& nums, int k) {
        int n = nums.size();
        vector<int> dp(n);
        int ans = INT_MIN;
        deque<int> q;
        for (int i = 0; i < n; ++i) {
            if (!q.empty() && i - q.front() > k) q.pop_front();
            dp[i] = max(0, q.empty() ? 0 : dp[q.front()]) + nums[i];
            ans = max(ans, dp[i]);
            while (!q.empty() && dp[q.back()] <= dp[i]) q.pop_back();
            q.push_back(i);
        }
        return ans;
    }
};

Java
Java
class Solution {
    public int constrainedSubsetSum(int[] nums, int k) {
        // dp[i] = the maximum the sum of non-empty subsequences in nums[0...i]
        int[] dp = new int[nums.length];
        // q stores dp[0], dp[1], ..., dp[k-1] whose values are > 0
        // in decreasing order.
        Deque<Integer> dq = new ArrayDeque<>();
        for (int i = 0; i < nums.length; ++i) {
            if (dq.isEmpty())
                dp[i] = nums[i];
            else
                dp[i] = Math.max(dq.peekFirst(), 0) + nums[i];
            while (!dq.isEmpty() && dp[dq.peekLast()] <= dp[i])
                dq.pollLast();
            dq.offerLast(dp[i]);
            if (i > k && dp[i - k] <= dq.peekFirst())
                dq.pollFirst();
        }
        return Arrays.stream(dp).max().getAsInt();
    }
}

Java

```

Heap · LeetCode - By Pattern — 1503 — LeetCode

```

class Solution {
    public int constrainedSubsetSum(int[] nums, int k) {
        Deque<Integer> q = new ArrayDeque<>();
        q.offer(0);
        int n = nums.length;
        int[] f = new int[n];
        int ans = -1;
        for (int i = 0; i < n; ++i) {
            while (!q.isEmpty() && i - q.peekFirst() > k)
                q.pollFirst();
            f[i] = Math.max(0, f[q.peekFirst()]) + nums[i];
            ans = Math.max(ans, f[i]);
            while (!q.isEmpty() && f[q.peekLast()] <= f[i])
                q.pollLast();
            q.offerLast(f[i]);
        }
        return ans;
    }
}

Java
class Solution {
    public int constrainedSubsetSum(int[] nums, int k) {
        int n = nums.length;
        int[] dp = new int[n];
        int ans = Integer.MIN_VALUE;
        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; ++i) {
            if (!q.isEmpty() && i - q.peek() > k)
                q.poll();
            dp[i] = Math.max(0, q.isEmpty() ? 0 : dp[q.peek()]) + nums[i];
            while (!q.isEmpty() && dp[q.peekLast()] <= dp[i])
                q.pollLast();
            q.offerLast(i);
            ans = Math.max(ans, dp[i]);
        }
        return ans;
    }
}

TypeScript
TypeScript

```

Heap · LeetCode - By Pattern — 1504 — LeetCode

```

function constrainedSubsetSum(nums: number[], k: number): number {
    const q = new Deque<number>();
    const n = nums.length;
    q.pushBack(0);
    let ans = nums[0];
    const f: number[] = Array(n).fill(0);
    for (let i = 0; i < n; ++i) {
        while (i - q.frontValue() > k) {
            q.popFront();
        }
        f[i] = Math.max(0, f[q.frontValue()]) + nums[i];
        ans = Math.max(ans, f[i]);
        while (!q.isEmpty() && f[q.backValue()]) <= f[i]) {
            q.popBack();
        }
        q.pushBack(i);
    }
    return ans;
}

class Node<T> {
    value: T;
    next: Node<T> | null;
    prev: Node<T> | null;
}

constructor(value: T) {
    this.value = value;
    this.next = null;
    this.prev = null;
}

class Deque<T> {
    private front: Node<T> | null;
    private back: Node<T> | null;
    private size: number;

    constructor() {
        this.front = null;
        this.back = null;
        this.size = 0;
    }

    pushFront(val: T): void {
        const node = new Node(val);
        if (this.isEmpty()) {
            this.front = node;
            this.back = node;
        } else {
            node.prev = this.back;
            this.back.next = node;
            this.back = node;
        }
        this.size++;
    }

    popFront(): T | undefined {
        if (this.isEmpty()) {
            return undefined;
        }
        const value = this.front.value;
        this.front = this.front.next;
        this.size--;

        if (this.front === null) {
            this.back = null;
        }
    }
}

function constrainedSubsetSum(nums: [int, k] int) {
    n = len(nums)
    dp = nums[0]
    ans = math.MIN_VALUE
    q = [0]
    for i, v in range nums:
        if len(q) > 0 && i - q[0] > k:
            q = q[1:]
        dp[i] = v
        if len(q) > 0 && dp[q[0]] > 0:
            dp[i] += dp[q[0]]
        for len(q) > 0 && dp[q[-1]] <= dp[i]:
            q = q[-1:]
        q = append(q, i)
        ans = max(ans, dp[i])
    return ans
}

```

Heap · LeetCode - By Pattern — 1505 — LeetCode

```

this.front.prev = nullNode;
this.front = nullNode;
}
this.size++;
}

pushBack(val: T): void {
    const node = new Node(val);
    if (this.isEmpty()) {
        this.front = node;
        this.back = node;
    } else {
        node.prev = this.back;
        this.back.next = node;
        this.back = node;
    }
    this.size++;
}

popFront(): T | undefined {
    if (this.isEmpty()) {
        return undefined;
    }
    const value = this.front.value;
    this.front = this.front.next;
    this.size--;

    if (this.front === null) {
        this.back = null;
    }
}

Go
Go
func constrainedSubsetSum(nums []int, k int) int {
    n := len(nums)
    dp := make([]int, n)
    ans := math.MinInt32
    q := []int{}
    for i, v := range nums {
        if len(q) > 0 && i - q[0] > k {
            q = q[1:]
        }
        dp[i] = v
        if len(q) > 0 && dp[q[0]] > 0 {
            dp[i] += dp[q[0]]
        }
        for len(q) > 0 && dp[q[-1]] <= dp[i] {
            q = q[:len(q)-1]
        }
        q = append(q, i)
        ans = max(ans, dp[i])
    }
    return ans
}

```

Heap · LeetCode - By Pattern — 1506 — LeetCode

```
[*] Heap -- Pattern Solution (stored optimal)

Python

# Python solution using a monotonic deque for efficient window maximum computation.
from collections import deque

def constrained_subsequence_sum(nums, k):
    n = len(nums)
    dp = [0] * n # dp[i] stores the maximum subsequence sum ending at index i
    dp[0] = nums[0]
    result = dp[0]

    # Deque to store indices in a decreasing order of dp values
    dq = deque([0])

    for i in range(1, n):
        # Remove indices from the front which are out of the window
        while dq and i - dq[0] > k:
            dq.popleft()

        # The maximum dp in the window is at the front of the deque
        max_in_window = dp[dq[0]] if dp[dq[0]] > 0 else 0

        # Calculate dp[i] by either starting a new subsequence or extending the best one from the window
        dp[i] = max(1 + max_in_window, nums[i])

        # Update overall answer
        result = max(result, dp[i])

    # Maintain the deque monotonically so that dp indices are in decreasing order
    while dq and dp[i] >= dp[dq[-1]]:
        dq.pop()
    dq.append(i)

    return result

# Example usage:
# print(constrained_subsequence_sum([10,2,-10,3,-8], 2))

C++
```

```
// C++ solution using deque from STL for maintaining sliding window maximum.
#include <vector>
#include <deque>
#include <algorithm>
using namespace std;

class Solution {
public:
    int constrainedSubsequenceSum(vector<int>& nums, int k) {
        int n = nums.size();
        vector<int> dp(n, 0); // dp[i] stores the maximum subsequence sum ending at i
        dp[0] = nums[0];
        int result = dp[0];
        deque<int> dq;
        dq.push_back(0);

        for (int i = 1; i < n; i++) {
            // Remove indices from the front that are out of the window
            while (!dq.empty() && i - dq.front() > k) {
                dq.pop_front();
            }

            int maxInWindow = dq.empty() ? 0 : *dq.front(); // 0 if dq is empty
            dp[i] = max(1 + maxInWindow, nums[i]);
            result = max(result, dp[i]);

            // Maintain deque in decreasing order of dp values
            while (!dq.empty() && dp[i] >= dp[dq.back()]) {
                dq.pop_back();
            }
            dq.push_back(i);
        }

        return result;
    }
};

// Example usage:
// int main() {
//     // Solution obj
//     vector<int> nums = {10, 2, -10, 3, -8};
//     int k = 2;
//     int result = sol.constrainedSubsequenceSum(nums, k);
//     return 0;
// }
```

Quick Review — Heap 20 questions · insights · complexity · solution

#215 Kth Largest Element in an Array [Medium]

Key Insights

- kth largest element can be converted to finding the element at index n-k if the array were sorted.
- A min-heap of size k is a practical method to maintain the k largest elements seen so far.
- The Quickselect algorithm can find the kth largest element in average O(n) time with in-place partitioning.
- Choose an appropriate method based on performance needs and worst-case scenarios.

Space & Time

Time Complexity:

- Min-Heap: O(n log k) by maintaining a heap of size k.
- Quickselect: Average O(n) but worst-case O(n²) with poor pivot selection.

Space Complexity:

- Min-Heap: O(k)
- Quickselect: O(1) additional space with in-place partitioning.

Solution

We can solve the problem using either a min-heap or the Quickselect algorithm.

- **Min-Heap Approach:** Initialize a min-heap with the first k elements. Iterate through the remaining elements; if an element is greater than the heap's root (the smallest in the heap), replace the root. After processing all elements, the root of the min-heap is the kth largest element. - **Quickselect Approach:** Choose a pivot and partition the array so that elements larger than the pivot come before it. Determine the position of the pivot; if it matches the index corresponding to the kth largest element, return it. Recurse into the appropriate part of the array.

Key points to consider include maintaining heap size for the first approach and proper pivot selection for the Quickselect approach.

#253 Meeting Rooms II [Medium]

Key Insights

- Sort the intervals based on start times to process meetings in order.
- Use a min-heap (priority queue) to keep track of meeting end times.
- When processing a new meeting, check if its start time is greater than or equal to the smallest end time from the heap.
- If so, it means a meeting room has been freed (the meeting with the smallest end time has ended) and can be reused.
- If not, allocate a new room by pushing the current meeting's end time into the heap.
- The size of the heap at any moment represents the number of rooms currently in use.

Space & Time

Time Complexity: O(n log n) due to sorting and each heap operation (push/pop) taking O(log n).

Space Complexity: O(n) in the worst-case scenario when all meetings overlap, requiring all end times in the heap.

Solution

The solution involves first sorting the meeting intervals by their start times. Then, we utilize a min-heap (priority queue) to keep track of the end times of ongoing meetings. For each meeting, we compare its start time with the smallest end time in the heap:

- If the meeting can start after the meeting at the top of the heap has ended (i.e., its start time is greater than or equal to the smallest end), we remove that end time from the heap (freeing up a room). - Then, we add the current

meeting's end time to the heap. By the end of processing all meetings, the size of the heap represents the minimum number of conference rooms required.

#347 Top K Frequent Elements [Medium]

Key Insights

- Count the frequency of each element using a hash table.
- Use bucket sort to achieve linear time complexity by grouping numbers by their frequency.
- Traverse the frequency buckets from highest to lowest to collect the k most frequent elements.
- Alternative approaches include using heaps or Quickselect, but bucket sort meets the follow-up requirement of better than O(n log n) time complexity.

Space & Time

Time Complexity: O(n) where n is the length of the array (bucket sort and frequency counting are linear).

Space Complexity: O(n) for storing the counts and buckets.

Solution

The solution starts by counting the frequency of each element with a hash table (or dictionary). Then, we create an array of buckets where the index represents the frequency. Each bucket holds the numbers that appear with that frequency. Finally, we iterate over the buckets in reverse order (from high frequency to low) and collect elements until we have k of them. This bucket sort approach avoids the higher complexity of sorting a list of frequencies and guarantees linear time performance.

#505 The Maze II [Medium]

Key Insights

- The ball rolls until it hits a wall, so each move is not one step but a continuous travel in a direction.
- The problem can be modeled as finding the shortest path in a weighted graph where nodes are positions in the maze and edge weights are the distance traveled in that roll.
- A priority queue (min-heap) based algorithm, such as Dijkstra's algorithm, is ideal since each roll covers variable distances.
- Always check if the new computed distance for a cell is less than a previously recorded one before pushing it to the heap.

Space & Time

Time Complexity: O(m * n * log(m * n)), where m and n are the maze dimensions (each cell may get processed and queued with a logarithmic cost).

Space Complexity: O(m * n) for maintaining the distance matrix and the priority queue.

Solution

We solve the problem using a modified version of Dijkstra's algorithm. The maze grid is treated as a graph where each cell is a node and an edge exists between the current cell and the cell where the ball stops after rolling in one direction. For every cell popped from the priority queue, we roll the ball in all four directions until it hits a wall, counting the number of steps taken. If the distance calculated from this roll is shorter than a previously stored distance for the stopping cell, we update the distance and add it to the priority queue. This process continues until we either reach the destination (and return the distance) or exhaust all possibilities (returning -1).

#621 Task Scheduler [Medium]

Key Insights

- Count the frequency of each task.
- The task with the highest frequency determines the base structure.
- Identify how many tasks share the maximum frequency.
- Calculate the total time based on the formula: max(task counts, (max frequency - 1) * (n + 1) + count of tasks with maximum frequency).

- This approach avoids simulating the entire scheduling by using a greedy strategy.

Space & Time

Time Complexity: O(N) where N is the number of tasks (counting frequency, then iterating over constant 26 letters).

Space Complexity: O(1) since the frequency count uses an array or hash map with fixed size (26 letters).

Solution

We first count the frequency of each task. The task(s) with the maximum frequency establish the minimum structure required. Compute the ideal idle slots based on the count of these maximum tasks and the cooling period n. Finally, compare this computed length with the total number of tasks to get the final answer. This greedy strategy avoids explicit simulation and leverages mathematical formulation.

#658 Find K Closest Elements [Medium]

Key Insights

- The input array is already sorted.
- A binary search can be used to efficiently narrow down the potential starting position.
- A two-pointers or sliding window approach is effective for selecting k consecutive elements.
- The comparison of absolute differences (with a tie-breaker on values) is crucial.
- Maintaining ascending order simplifies the final output.

Space & Time

Time Complexity: O(log n + k), where O(log n) for binary search and O(k) for collecting results.

Space Complexity: O(k), for storing the output.

Solution

We can solve this problem by first using binary search to identify the left-most window where the k closest elements might start. The binary search operates over the possible starting indices for windows of size k (from 0 to n-k). For each middle index mid, we compare the distances between x and arr[mid] and arr[mid + k] to decide whether to move the window to the right or left. Once the correct starting position is found, return the k elements starting from that index. This approach leverages the sorted property of the array to efficiently converge on the optimal window.

#787 Cheapest Flights Within K Stops [Medium]

Key Insights

- Build the graph as an adjacency list of flights and prices.
- Use a min-heap so the cheapest partial route is always processed first.
- Track states as 'city, flightsUsed' instead of only 'city', because the stop count matters.
- For each popped state, try all outgoing flights and push the updated cost and flight count.
- Do not expand a state once it has already used more than 'k + 1' flights.
- Keep the best known cost for each 'city, flightsUsed' state so worse duplicate states can be skipped.

Space & Time

Time Complexity: O((V + E) * log(V * (k + 1))) for the heap-based search over 'city, stopsUsed' states.

Space Complexity: O(E + V * (k + 1)) for the graph plus the best-cost tracking table.

Solution

We solve the problem using a Dijkstra-style min-heap search, but the state must include both the city and how many flights have been used so far. We start from ('0', src, 0) and always pop the cheapest state from the heap. For each state, if we reach 'dst', that cost is the answer. Otherwise, if we still have flights available within the 'k + 1' limit, we explore all outgoing edges and push the next states into the heap. To avoid useless work, we store the best cost seen for each 'city, flightsUsed' pair and only continue when the new state improves that record. This guarantees the cheapest valid route under the stop constraint.

#973 K Closest Points to Origin [Medium]

Key Insights

- Calculate the squared Euclidean distance (x² + y²) to avoid the overhead of square root computation.
- Sorting the points based on their distance gives an O(n log n) solution.
- Alternatively, using a max-heap to keep track of the k closest points results in O(n log k) time, which is effective if k is much smaller than n.
- Using Quickselect can achieve an average time complexity of O(n), though the worst-case is O(n³).
- The problem does not require the output to be sorted in any specific order.

Space & Time

Time Complexity: O(n log n) with a sorting approach, O(n log k) with a heap approach, or average O(n) with a Quickselect approach.

Space Complexity: O(n) if extra space is used for sorting, or O(k) for the heap approach.

Solution

We can solve this problem by first computing the squared Euclidean distance for each point to avoid unnecessary square root operations. Then, we can sort the points based on their computed distance and select the first k points. Alternatively, a max-heap or Quickselect algorithm can be used.

- **Max-Heap:** Maintain a heap of size k while iterating through the points, ensuring that only the k points with the smallest distances are kept.
- **Quickselect:** Partition the array like in QuickSort to position the kth closest point in its correct position. Once partitioned, all points to the left of that position will be the k closest. The sorting solution is straightforward and efficient enough given the constraints.

#1057 Campus Bikes [Medium]

Key Insights

- Compute the Manhattan distance between every worker and bike pair.
- To achieve the tie-breaking rules, sort all pairs by distance, then by worker index and bike index.
- Once a worker is assigned a bike, skip any other pairs involving that worker or the already assigned bike.
- The problem can also be approached using bucket sort because the Manhattan distance range is limited, but sorting is intuitive given the constraints.

Space & Time

Time Complexity: O(n * m * log(n * m)) where n is the number of workers and m is the number of bikes.

Space Complexity: O(n * m) for storing all worker-bike pairs.

Solution

Create all possible (worker, bike) pairs along with their Manhattan distances. Sort this list of pairs by:

- Manhattan distance, - Worker index (if distances are equal), - Bike index (if both distance and worker index are equal).

Then, iterate over the sorted list and assign bikes to workers if neither the worker nor the bike has already been assigned. This guarantees that at each step, the optimal available pairing (according to the problem's requirements) is made. This method naturally enforces the tie-breaking rules specified in the problem.

#1167 Minimum Cost to Connect Sticks [Medium]

Key Insights

- Always connect the two smallest sticks available to minimize the incremental cost.
- A min-heap (or priority queue) is an ideal data structure to efficiently retrieve the smallest sticks.
- The process continues until only one stick remains.

Space & Time

Time Complexity: O(n log n), where n is the number of sticks (due to heap insertion and removal operations).

Space Complexity: O(n), for storing the sticks in the heap.

Solution

The solution uses a greedy algorithm with a min-heap:

→ Insert all stick lengths into a min-heap. → While there are at least two sticks in the heap: Extract the two smallest sticks. Compute the cost of connecting them (sum the two lengths). Add this cost to the total result. Push the combined stick (with length equal to their sum) back into the heap. → Once the heap is reduced to one element, return the total cost.
This approach ensures that at every connection, the smallest possible sticks are merged, which leads to the minimum overall cost.

#1424 Diagonal Traverse II [Medium]

Key Insights

- The diagonal key for each element can be defined as $(row_index + column_index)$.
- Elements in the same diagonal (with equal diagonal key) must be output in an order such that the element from the lower row comes before the one from a higher row.
- Because the input grid may be jagged (rows with varying lengths), you must carefully iterate only over valid indices.
- A hash table (or map) can be used to collect elements grouped by their diagonal key.
- Inserting each element at the beginning of the list for that diagonal builds the list in the desired order without needing an extra reverse step later.
- Finally, traverse the keys in increasing order (from 0 to maximum key) to produce the final result.

Space & Time

Time Complexity: $O(N)$, where N is the total number of elements (each element is processed once).
Space Complexity: $O(N)$, for storing the elements in the map and the output array.

Solution

The solution iterates over every element in nums while computing its diagonal key $i + j$. For each element, the value is inserted at the beginning of the list corresponding to that diagonal key. This ensures that when the diagonal groups are later concatenated in order of increasing key, the internal ordering within each diagonal is correct (i.e., elements appear from the bottom of the diagonal to the top). The final result is built by iterating over the keys in sorted order and appending each list of diagonal elements.

#23 Merge k Sorted Lists [Hard]

Key Insights

- Use a min-heap (priority queue) to efficiently extract the smallest value among the current heads of the lists.
- By pushing the head of each non-empty list into the heap, we can pull the smallest node, then push that node's next element into the heap.
- Alternatively, a divide and conquer approach can be used by recursively merging pairs of lists.
- Careful management of pointers (or references) is key to constructing the new linked list.

Space & Time

Time Complexity: $O(N \log k)$ where N is the total number of nodes across all lists and k is the number of lists.
Space Complexity: $O(k)$ due to the heap containing at most one node from each list.

Solution

We use a min-heap to always retrieve the smallest node among the current nodes in the linked lists. First, initialize the heap with the head of each list. Then, continuously extract the smallest node from the heap, attach it to the merged list, and if the extracted node has a next node, insert that into the heap. The process repeats until the heap is empty.

#218 The Skyline Problem [Hard]

Key Insights

- Use a line sweep approach to process building start and end events (which remove from the skyline).
- Differentiate between start events (which add to the skyline) and end events (which remove from the skyline).

Heap · LeetCode · By Pattern

— 1513 —

LeetCode

- addNum: $O(\log n)$ per insertion (heap operations).
- findMedian: $O(1)$ (accessing the top elements).

Space Complexity

$O(n)$, where n is the number of elements added in the data stream, as they are stored in the two heaps.

Solution

We maintain two heaps:

- A max-heap (lowerHalf) for the smaller half of the numbers. A min-heap (upperHalf) for the larger half of the numbers. On inserting a new number, decide the heap membership based on the current median. After each insertion, ensure that the heaps have balanced sizes (the size difference is at most one). When computing the median — if both heaps have the same number of elements, the median is the average of the max of lowerHalf and the min of upperHalf. If one heap has more elements, the median is the top element of that heap.
- This method makes it efficient to dynamically update and retrieve the median as new numbers are inserted.

#358 Rearrange String k Distance Apart [Hard]

Key Insights

- The frequency of each character dictates the feasibility and placement within the rearrangement.
- A greedy approach picks the most frequent available character to maximize the chance of successful placement.
- A max-heap (priority queue) efficiently selects the highest frequency character each time.
- A waiting queue ensures that once a character is used, it can't be reused until k positions later.
- Special case: When k is 0, no rearrangement is required because all orders satisfy the condition.

Space & Time

Time Complexity: $O(n \log C)$ where n is the length of s and C is the number of unique characters.

Space Complexity: $O(n)$ due to the storage used for the heap and wait queue.

Solution

We use a greedy algorithm with a max-heap to always select the character with the highest remaining frequency that is currently available. The algorithm first counts the frequency of each character and pushes these into a max-heap (using negative frequencies for languages like Python where heaps are a min-heap). Each time a character is selected, it is appended to the result string and its frequency is decreased. This character is then placed in a waiting queue to ensure it isn't reused until k positions later. Once the waiting period completes, if the character still has remaining occurrences, it is reinserted into the heap. If at any point the heap is empty and the waiting queue still contains characters that haven't been reinserted while the result string is incomplete, we conclude that a valid rearrangement is impossible.

#499 The Maze III [Hard]

Key Insights

- The ball rolls continuously until hitting a wall or falling into the hole.
- A shortest path search is needed based on rolling distance (each step counts when passing an empty cell).
- Dijkstra's algorithm (or a modified BFS with a priority queue) is appropriate since we need to consider distance and lexicographic order.
- When expanding moves, simulate the roll until the ball stops — if a hole is passed en route, use that stopping point.
- Track the best (distance, path) for each cell to avoid redundant work.

Space & Time

Time Complexity: $O(mn \log(mn))$ where m and n are the maze dimensions

Space Complexity: $O(mn)$

Solution

We use a Dijkstra-like approach with a priority queue containing tuples of (distance, path, row, col). The priority queue is ordered by distance first and then by the instruction string lexicographically. For each cell, simulate rolling

Heap · LeetCode · By Pattern

— 1515 —

LeetCode

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ ($O(k)$ extra space for the deque, with dp array using $O(n)$)

Solution

We solve the problem by defining a dp array where $dp[i] = \text{nums}[i] + \text{max}(0, \text{maximum dp value in the window } [i-k, i-1])$. The trick is to include $\text{max}(0, \dots)$ to allow starting a new subsequence when previous sums are negative. To access the maximum dp in the window in constant time, a monotonic deque is maintained that stores indices with their dp values in decreasing order. At each index, we:

- Remove elements from the front of the deque if they are out of the current window ($i - \text{index} > k$).
- Use the front of the deque (if non-empty) as the maximum dp value from the previous k indices.
- Compute $dp[i]$ and update the global maximum answer.
- Remove from the back of the deque all indices with dp values less than the current $dp[i]$ since they are not useful.
- Insert the current index into the deque.

Heap · LeetCode · By Pattern

— 1517 —

LeetCode

- Employ a max heap (or a priority queue) to efficiently track the current highest building for each sweep line position.
- When the current maximum height changes, record the corresponding x -coordinate and new height as a key point.
- Take care to remove expired building heights from the heap as the sweep line moves past the building's end.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting events and heap operations for each building event.

Space Complexity: $O(n)$ for the event list and the heap used for processing active buildings.

Solution

The solution uses the line sweep algorithm with a max heap (or priority queue) to maintain the heights of active buildings. Each building contributes two events: one when the building starts (adding its height) and one when it ends (removing its height). These events are sorted primarily by their x -coordinate. For start events, we use negative height values to ensure they are processed before end events at the same x -coordinate. During the sweep, the heap is updated by adding new building heights and removing buildings that have ended. Whenever there is a change in the top of the heap (i.e., the current maximum height), a key point is added to the result. This key point signifies a vertical change in the skyline.

#272 Closest Binary Search Tree Value II [Hard]

Key Insights

- The BST property allows efficient narrowing down of candidate nodes.
- Using two stacks to track predecessors (values $\leq \text{target}$) and successors (values $> \text{target}$) can efficiently yield the closest values.
- A modified in-order traversal helps in retrieving the next closest candidate from either side.
- Merging two sorted lists of candidate values based on absolute difference with the target is the key step.

Space & Time

Time Complexity: $O(k + \log n + O(\log n))$ to initialize the two stacks and $O(k)$ to collect k elements.

Space Complexity: $O(\log n)$ — due to the stack space in a balanced BST.

Solution

We can solve the problem by splitting it into two parts:

- Build two stacks: The predecessor stack for nodes with values less than or equal to target. The successor stack for nodes with values greater than target. To build these stacks, traverse the BST starting from the root and push nodes along the path: if the node's value is less than or equal to target, push it to the predecessor stack and go right; if greater, push it to the successor stack and go left. Merge the two stacks: At each step, compare the top element of the predecessor stack (if available) and the successor stack (if available) based on their distance from the target. Pop from the stack with the closer candidate and add that value to the result list. For the popped node, update the given stack by moving to the next appropriate node in the in-order traversal (for predecessor, go to left child then rightmost descendant; for successor, go to right child then leftmost descendant). Repeat until k values have been collected.

Key Data Structures and Techniques:

- Two stacks to simulate reverse and forward in-order traversals.
- Merge technique to choose the closest candidate at each iteration.

#295 Find Median from Data Stream [Hard]

Key Insights

- Use two heaps: a max-heap for the lower half and a min-heap for the upper half of numbers.
- Balancing the heaps ensures the median can be computed in $O(1)$ time once numbers are added.
- Rebalance the heaps after each insertion to ensure size properties hold (difference in sizes is at most 1).

Space & Time

Time Complexity:

Heap · LeetCode · By Pattern

— 1514 —

LeetCode

in each possible direction (up, down, left, right); iterated in alphabetical order so that the lexicographical priority is maintained until the ball stops. If the ball encounters the hole during rolling, treat that as a valid stop immediately. For every new stopping cell, if a shorter distance or lexicographically smaller path is found, update the record and add the new state to the priority queue. If you reach the hole, return the corresponding path string. If no solution is found when the queue is empty, return "impossible".

#632 Smallest Range Covering Elements from K Lists [Hard]

Key Insights

- Use a min heap to maintain the smallest current element among the selected elements from each list.
- Track the current maximum among the chosen elements to easily compute the range.
- Iteratively replace the minimum element with the next element from the same list and update the current maximum.
- Stop when one of the lists is exhausted, as the range must include at least one element from each list.

Space & Time

Time Complexity: $O(N \log k)$, where N is the total number of elements across all lists.

Space Complexity: $O(k)$ for the heap, plus additional space for storing the range.

Solution

The solution uses a min heap (priority queue) that stores a tuple containing the current element, the index of the list it comes from, and its index in that list. Initially, the first element from each list is inserted into the heap and the current maximum among them is tracked. In each iteration, the smallest element is removed (heap root) which provides the current minimum, and the range (current min, current max) is assessed. If the list from which the minimum element was extracted has a next element, that element is inserted into the heap and the current maximum is updated if necessary. This process continues until one of the lists is exhausted.

#759 Employee Free Time [Hard]

Key Insights

- Flatten the schedule to combine all employees' intervals.
- Sort the flattened intervals by start time.
- Merge overlapping intervals to form a consolidated view of busy times.
- Identify gaps between merged intervals as the common free time.
- Ensure free intervals have strictly positive length.

Space & Time

Time Complexity: $O(N \log N)$, where N is the total number of intervals (due to sorting).

Space Complexity: $O(N)$, used for the flattened list and merged intervals.

Solution

The solution leverages interval merging. First, all intervals from every employee are collected and sorted by start time. By iterating through the sorted intervals, overlapping intervals are merged into one consolidated busy period. Finally, the gaps between consecutive merged intervals are identified as free periods available to all employees. This approach efficiently handles the intervals with a simple sort and a linear scan.

#1425 Constrained Subsequence Sum [Hard]

Key Insights

- Use dynamic programming where $dp[i]$ represents the maximum sum of a valid subsequence ending at index i .
- For each index i , consider taking $\text{nums}[i]$ alone or appending it to a valid subsequence ending at an index j (where $i - j \leq k$) that maximizes the sum.
- To efficiently obtain the maximum $dp[j]$ in a sliding window of the last k indices, use a monotonic (decreasing) deque.
- The idea is to maintain the window maximum and update it as we proceed through the array.

Heap · LeetCode · By Pattern

— 1516 —

LeetCode

#15 Matrix — 20 questions · #15 of 21

#422 EASY

Valid Word Square

LeetCode · Simplify · WAAACE · LeetCode · Editorial

Array · Matrix

List: Premium 98

Problem:

Given an array of strings words, return true if it forms a valid word square.

A sequence of strings forms a valid word square if the k th row and column read the same string, where $0 \leq k < \max(\text{numRows}, \text{numColumns})$.

Example 1:

a	b	c	d
b	n	r	t
c	r	m	y
d	t	y	e

Input: words = ["abcd","birt","crry","dtye"]

Output: true

Explanation:

The 1st row and 1st column both read "abcd".

The 2nd row and 2nd column both read "birt".

The 3rd row and 3rd column both read "crry".

The 4th row and 4th column both read "dtye".

Therefore, it is a valid word square.

Example 2:

Heap · LeetCode · By Pattern

— 1518 —

LeetCode

a	b	c	d
b	n	r	t
c	r	m	
d	t		

Input: words = ["abcd","burt","cr","dt"]
Output: true
Explanation:
The 1st row and 1st column both read "abcd".
The 2nd row and 2nd column both read "burt".
The 3rd row and 3rd column both read "cr".
The 4th row and 4th column both read "dt".
Therefore, it is a valid word square.

Example 3:

b	a	l	l
a	r	e	a
r	e	a	d
l	a	d	y

Input: words = ["ball","area","read","lady"]
Output: false
Explanation:
The 3rd row reads "read" while the 3rd column reads "lead".
Therefore, it is NOT a valid word square.

Constraints:

- 1 <= words.length <= 500
- 1 <= words[i].length <= 500
- words[i] consists of only lowercase English letters.

>> Brute Force (Matrix) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:
        # Brute Force O(n^2 * L) - Flood fill from every cell
        n = len(words)
        result = 0
        for r in range(n):
            for c in range(n):
                visited = set()
                def flood(r, c):
                    if (r, c) in visited or not (0 <= r < n and 0 <= c < n): return
                    visited.add((r, c))
                    for dr, dc in [(1, 1), (0, -1), (1, 0), (-1, 0)]: flood(r+dr, c+dc)
                flood(r, c)
                result = max(result, len(visited))
        return result
```

```
Python
# Function to check if the provided words form a valid word square
def validWordSquare(words):
    # Iterate over each row with its index
    for i in range(len(words)):
        # Iterate over each character in the current row
        for j in range(len(words[i])):
            # Check if the corresponding column exists
            if j == len(words) or i == len(words[j]) or words[i][j] != words[j][i]:
                return False
    return True

# Example test case
words = ["abcd","burt","cr","dt"]
print(validWordSquare(words)) # Expected output: True
```

```
Python
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:
        for i, word in enumerate(words):
            for j, c in enumerate(word):
                if len(words) == j or len(words[j]) == i: # out-of-bounds
                    return False
                if c != words[j][i]:
                    return False
        return True
```

```
Python
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:
        n = len(words)
        for i in range(n):
            for j in range(n):
                if j == n or i == len(words[j]) or words[i][j] != words[j][i]:
                    return False
        return True
```

```
Python
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:
        n = len(words)
        for i in range(n):
            for j in range(n):
                if j == n or i == len(words[j]) or words[i][j] != words[j][i]:
                    return False
        return True
```

C++

C++

```
class Solution {
public:
    bool validWordSquare(vector<string>& words) {
        for (int i = 0; i < words.size(); ++i)
            for (int j = 0; j < words[i].size(); ++j) {
                if (words.size() == j || words[j].size() == i) // out-of-bounds
                    return false;
                if (words[i][j] != words[j][i])
                    return false;
            }
        return true;
    }
};
```

C++

```
class Solution {
public:
    bool validWordSquare(vector<string>& words) {
        int n = words.size();
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == n || j < words[j].size() || words[i][j] != words[j][i]) {
                    return false;
                }
            }
        }
        return true;
    }
};
```

C++

```
class Solution {
public:
    bool validWordSquare(vector<string>& words) {
        int n = words.size();
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == n || j < words[j].size() || words[i][j] != words[j][i]) {
                    return false;
                }
            }
        }
        return true;
    }
};
```

Java

Java

```
class Solution {
    public boolean validWordSquare(List<String> words) {
        for (int i = 0; i < words.size(); ++i)
            for (int j = 0; j < words.get(i).length(); ++j) {
                if (words.size() == j || words.get(j).length() == i) // out-of-bounds
                    return false;
                if (words.get(i).charAt(j) != words.get(j).charAt(i))
                    return false;
            }
        return true;
    }
}
```

Java

```
class Solution {
    public boolean validWordSquare(List<String> words) {
        int n = words.size();
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == n || j < words.get(j).length()) {
                    return false;
                }
                if (words.get(i).charAt(j) != words.get(j).charAt(i)) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

Java

```
class Solution {
    public boolean validWordSquare(List<String> words) {
        int n = words.size();
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == n || j < words.get(j).length()) {
                    return false;
                }
                if (words.get(i).charAt(j) != words.get(j).charAt(i)) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

TypeScript

TypeScript

```
function validWordSquare(words: string[]): boolean {
    const n = words.length;
    for (let i = 0; i < n; ++i) {
        const m = words[i].length;
        for (let j = 0; j < m; ++j) {
            if (i == n || j < words[j].length || words[i][j] != words[j][i]) {
                return false;
            }
        }
    }
    return true;
}
```

TypeScript

```
function validWordSquare(words: string[]): boolean {
    const n = words.length;
    for (let i = 0; i < n; ++i) {
        const m = words[i].length;
        for (let j = 0; j < m; ++j) {
            if (i == n || j < words[j].length || words[i][j] != words[j][i]) {
                return false;
            }
        }
    }
    return true;
}
```

Go

Go

```
func validWordSquare(words []string) bool {
    n := len(words)
    for i, w := range words {
        for j := range w {
            if j == n || i == len(words[j]) || w[j] != words[j][i] {
                return false
            }
        }
    }
    return true
}
```

Go

```
func validWordSquare(words []string) bool {
    n := len(words)
    for i, w := range words {
        for j := range w {
            if j == n || i == len(words[j]) || w[j] != words[j][i] {
                return false
            }
        }
    }
    return true
}
```

[1] Matrix — Pattern Solution (stored optimal)

```

Python
# Function to check if the provided words form a valid word square
def validWordSquare(words):
    # Iterate over each row with its index
    for i in range(len(words)):
        # Iterate over each character in the current row
        for j in range(len(words[i])):
            # Check if the corresponding column exists
            if j == len(words) or i == len(words[i]) or words[i][j] != words[j][i]:
                return False
    return True

# Example test case
words = ["abcd", "bcrt", "cray", "dtye"]
print(validWordSquare(words)) # Expected output: True

C++
// Include iostream and vector for input/output and storage
#include <iostream>
#include <vector>
using namespace std;

// Function to check if the provided words form a valid word square
bool validWordSquare(vector<string>& words) {
    // Iterate over each row (start with index 1)
    for (int i = 0; i < words.size(); i++) {
        // Iterate over each character in the current word
        for (int j = 0; j < words[i].size(); j++) {
            // Check if the corresponding column index exists and if the row at index j has enough length
            if (j == words.size() || i == words[j].size() || words[i][j] != words[j][i]) {
                return false;
            }
        }
    }
    return true;
}

// Main function to test the implementation
int main() {
    vector<string> words = {"abcd", "bcrt", "cray", "dtye"};
    cout << (validWordSquare(words) ? "true" : "false") << endl; // Expected output: true
    return 0;
}

```

#566 EASY
Reshape the Matrix
[LeetCode](#) · [SimplyEazy](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)
[Array](#) · [Matrix](#) · [Simulation](#)
[List](#) · [CodeSignal](#)
Problem:

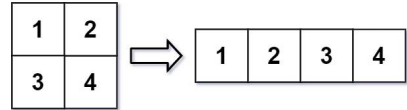
Matrix · [LeetCode](#) · [By Pattern](#) — 1525 — [LeetCode](#)

In MATLAB, there is a handy function called reshape which can reshape an m x n matrix into a new one with a different size r x c keeping its original data.

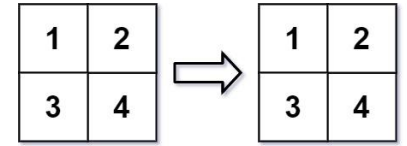
You are given an m x n matrix mat and two integers r and c representing the number of rows and the number of columns of the wanted reshaped matrix.
 The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

If the reshape operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

Example 1:



Example 2:



Constraints:

- m == mat.length
- n == mat[i].length
- 1 <= m, n <= 100
- 1 <= r, c <= 300

Matrix · [LeetCode](#) · [By Pattern](#) — 1526 — [LeetCode](#)

```

Python
# Brute Force Approach
class Solution:
    def matrixReshape(self, mat, r, c):
        # Brute Force O(N^2) - Flood fill from every cell
        m = len(mat), len(mat[0])
        result = []
        for r in range(m):
            for c in range(c):
                visited = set()
                def flood(r, c):
                    if (r, c) in visited or not (0 <= r < m and 0 <= c < m): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: flood(r+dr, c+dc)
                flood(r, c)
                result.append(mat[r][c])
        return result

Python
# Function to reshape the matrix
def matrixReshape(mat, r, c):
    # Calculate the dimensions of the original matrix
    m = len(mat)
    n = len(mat[0])

    # Check if reshape is possible
    if m * n != r * c:
        return mat

    # Flatten the original matrix into a list
    flat_list = []
    for row in mat:
        for num in row:
            flat_list.append(num)

    # Create the new matrix by slicing the flat list
    new_matrix = []
    for i in range(0, len(flat_list), c):
        new_matrix.append(flat_list[i:i+c])

    return new_matrix

# Example call:
# print(matrixReshape([[1,2],[3,4]], 1, 4))

```

Matrix · [LeetCode](#) · [By Pattern](#) — 1527 — [LeetCode](#)

```

class Solution {
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        ans = []
        for i in range(r):
            row = []
            for j in range(c):
                ans.append(mat[i][j])
            row.append(ans)
            ans = []
        return ans

Python
class Solution:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        ans = []
        for i in range(r):
            row = []
            for j in range(c):
                ans.append(mat[i][j])
            row.append(ans)
            ans = []
        return ans

C++
C++

```

Matrix · [LeetCode](#) · [By Pattern](#) — 1528 — [LeetCode](#)

```

class Solution {
public:
    vector<vector<int>> matrixReshape(vector<vector<int>>& mat, int r, int c) {
        if (mat.empty() || r <= 0 || c <= 0 || mat.size() != r || mat[0].size() != c)
            return mat;

        vector<vector<int>> ans(r, vector<int>(c));
        int k = 0;

        for (const vector<int>& row : mat) {
            for (const int& num : row) {
                ans[k / c][k % c] = num;
                ++k;
            }
        }

        return ans;
    }
};

C++
class Solution {
public:
    vector<vector<int>> matrixReshape(vector<vector<int>>& mat, int r, int c) {
        int m = mat.size(), n = mat[0].size();
        if (m * n != r * c)
            return mat;

        vector<vector<int>> ans(r, vector<int>(c));
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                ans[i / c][i % c] = mat[i][j];
            }
        }

        return ans;
    }
};

Java
Java

```

Matrix · [LeetCode](#) · [By Pattern](#) — 1529 — [LeetCode](#)

```

class Solution {
    public int[][] matrixReshape(int[][] mat, int r, int c) {
        if (mat.length == 0 || r <= 0 || c <= 0 || mat.length != r || mat[0].length != c)
            return mat;

        int[][] ans = new int[r][c];
        int k = 0;

        for (int[] row : mat) {
            for (final int num : row) {
                ans[k / c][k % c] = num;
                ++k;
            }
        }

        return ans;
    }
}

Java
class Solution {
    public int[][] matrixReshape(int[][] mat, int r, int c) {
        int m = mat.length, n = mat[0].length;
        if (m * n != r * c)
            return mat;

        int[][] ans = new int[r][c];
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                ans[i / c][i % c] = mat[i][j];
            }
        }

        return ans;
    }
}

TypeScript
TypeScript

```

Matrix · [LeetCode](#) · [By Pattern](#) — 1530 — [LeetCode](#)

```
function matrixReshape(num: number[][], r: number, c: number): number[][] {
    let n = mat.length;
    m = mat[0].length;
    if (n * m != r * c) return mat;
    let ans = Array.from(<length: r>(), v => new Array(c).fill(0));
    let k = 0;
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            ans[Math.floor(k / c)][k % c] = mat[i][j];
            k++;
        }
    }
    return ans;
}
```

TypeScript

```
function matrixReshape(num: number[][], r: number, c: number): number[][] {
    let n = mat.length;
    m = mat[0].length;
    if (n * m != r * c) return mat;
    let ans = Array.from(<length: r>(), v => new Array(c).fill(0));
    let k = 0;
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            ans[Math.floor(k / c)][k % c] = mat[i][j];
            k++;
        }
    }
    return ans;
}
```

Go

```
func matrixReshape(num [][]int, r int, c int) [][]int {
    n, m := len(num), len(num[0])
    if m*n != r*c {
        return num
    }
    ans := make([][]int, r)
    for i := range ans {
        ans[i] = make([]int, c)
    }
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            ans[i/c][i%c] = mat[i][j]
        }
    }
    return ans
}
```

Go

```
func matrixReshape(num [][]int, r int, c int) [][]int {
    n, m := len(num), len(num[0])
    if m*n != r*c {
        return num
    }
    ans := make([][]int, r)
    for i := range ans {
        ans[i] = make([]int, c)
    }
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            ans[i/c][i%c] = mat[i][j]
        }
    }
    return ans
}
```

Rust

```
impl Solution {
    pub fn matrix_reshape(mat: Vec<Vec<i32>>, r: i32, c: i32) -> Vec<Vec<i32>> {
        let r = r as usize;
        let c = c as usize;
        let n = mat.len();
        let m = mat[0].len();
        if m * n != r * c {
            return mat;
        }
        let mut i = 0;
        let mut j = 0;
        (0..r).for_each(|i| {
            (0..c).for_each(|j| {
                let mut res = mat[i][j];
                j += 1;
                if j == m {
                    j = 0;
                    i += 1;
                }
            })
        })
        .collect()
    }
}
```

Rust

```
impl Solution {
    pub fn matrix_reshape(mat: Vec<Vec<i32>>, r: i32, c: i32) -> Vec<Vec<i32>> {
        let r = r as usize;
        let c = c as usize;
        let n = mat.len();
        let m = mat[0].len();
        if m * n != r * c {
            return mat;
        }
        let mut i = 0;
        let mut j = 0;
        (0..r).for_each(|i| {
            (0..c).for_each(|j| {
                let mut res = mat[i][j];
                j += 1;
                if j == m {
                    j = 0;
                    i += 1;
                }
            })
        })
        .collect()
    }
}
```

[*] Matrix — Pattern Solution (stored optimal)

Python

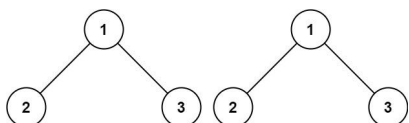
```
# Function to reshape the matrix
def matrixReshape(mat, r, c):
    # Calculating the dimensions of the original matrix
    n = len(mat)
    m = len(mat[0])
    # Check if reshape is possible
    if m * n != r * c:
        return mat
    # Flatten the original matrix into a list
    flat_list = []
    for row in mat:
        for num in row:
            flat_list.append(num)
    # Create the new matrix by slicing the flat list
    new_matrix = []
    for i in range(0, len(flat_list), c):
        new_matrix.append(flat_list[i:i+c])
    return new_matrix
# Example call:
# print(matrixReshape([[1,2],[2,3]], 1, 4))

// Function to reshape the matrix using C++
#include <vector>
using namespace std;
class Solution {
public:
    vector<vector<int>> matrixReshape(vector<vector<int>>& mat, int r, int c) {
        int n = mat.size();
        int m = mat[0].size();
        // Check if reshaping is possible
        if (m * n != r * c) {
            return mat;
        }
        // Flatten the matrix
        vector<int> flat;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                flat.push_back(mat[j][i]);
            }
        }
        // Build the new reshaped matrix
        vector<vector<int>> reshaped;
        for (int i = 0; i < flat.size(); i += c) {
            vector<int> row;
            for (int j = 0; j < c; j++) {
                row.push_back(flat[i+j]);
            }
            reshaped.push_back(row);
        }
        return reshaped;
    }
};
```

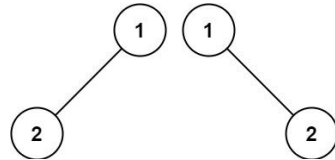
```
vector<int> row(flat.begin() + i, flat.begin() + i + c);
reshaped.push_back(row);
return reshaped;
}
```

#766 EASY
Toeplitz Matrix
 LeetCode · Simple · Math · LeetCode · Editorial
 Array · Matrix
 List · CodeSignal

Problem:
 Given an m x n matrix, return true if the matrix is Toeplitz. Otherwise, return false.
 A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.
 Example 1:



Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
 Output: true
 Explanation:
 In the above grid, the diagonals are:
 "[9]", "[5, 1]", "[1, 2, 3]", "[2, 2, 2]", "[3, 3]", "[4]".
 In each diagonal all elements are the same, so the answer is True.
 Example 2:



Input: matrix = [[1,2],[2,2]]
 Output: false
 Explanation:
 The diagonal "[1, 2]" has different elements.
 Constraints:
 • m == matrix.length
 • n == matrix[0].length
 • 1 <= m, n <= 20
 • 0 <= matrix[i][j] <= 99
 Follow up:
 • What if the matrix is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?
 • What if the matrix is so large that you can only load up a partial row into the memory at once?

>> Brute Force [Matrix Interview Prep]
 Python

```
# Brute Force Approach
class Solution:
    def isToeplitzMatrix(self, matrix):
        # Brute Force O(n^2 * n) - Flood fill from every cell
        n, m = len(matrix), len(matrix[0])
        result = 0
        for r in range(n):
            for c in range(m):
                visited = set()
                def floodfill(r, c):
                    if (r, c) in visited or not (0 <= r < n and 0 <= c < m): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: floodfill(r+dr, c+dc)
                floodfill(r, c)
                result = matrix[r][c]
            return result
```

Python

```
# Function to determine if the matrix is Toeplitz
def isToeplitzMatrix(matrix):
    # Get the number of rows and columns in the matrix
    rows = len(matrix)
    cols = len(matrix[0])

    # Iterate starting from row 1 and column 1 because first row and column are trivially Toeplitz
    for i in range(1, rows):
        for j in range(1, cols):
            # Compare current element with its top-left neighbor
            if matrix[i][j] != matrix[i-1][j-1]:
                return False
    return True

# Example usage:
matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
print(isToeplitzMatrix(matrix)) # Expected output: True
```

```
Python
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        for i in range(len(matrix) - 1):
            for j in range(len(matrix[0]) - 1):
                if matrix[i][j] != matrix[i + 1][j + 1]:
                    return False
        return True
```

```
Python
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        m, n = len(matrix), len(matrix[0])
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] != matrix[i - 1][j - 1]:
                    return False
        return True
```

```
Python
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        m, n = len(matrix), len(matrix[0])
        return all(
            matrix[i][j] == matrix[i - 1][j - 1]
            for i in range(1, m)
            for j in range(1, n)
        )
```

C++

C++

Matrix - LeetCode - By Pattern

— 1537 —

LeetCode

```
class Solution {
public:
    bool isToeplitzMatrix(vector<vector<int>>& matrix) {
        for (int i = 0; i < matrix.size(); ++i)
            for (int j = 0; j < matrix[i].size(); ++j)
                if (matrix[i][j] != matrix[i - 1][j + 1])
                    return false;
        return true;
    }
};
```

C++

```
class Solution {
public:
    bool isToeplitzMatrix(vector<vector<int>>& matrix) {
        int n = matrix.size(), m = matrix[0].size();
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j) {
                if (matrix[i][j] != matrix[i - 1][j - 1]) {
                    return false;
                }
            }
        return true;
    }
};
```

C++

```
class Solution {
public:
    bool isToeplitzMatrix(vector<vector<int>>& matrix) {
        int n = matrix.size(), m = matrix[0].size();
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j) {
                if (matrix[i][j] != matrix[i - 1][j - 1]) {
                    return false;
                }
            }
        return true;
    }
};
```

Java

Java

```
class Solution {
    public boolean isToeplitzMatrix(int[][] matrix) {
        for (int i = 0; i < 1 < matrix.length; ++i)
            for (int j = 0; j < 1 < matrix[0].length; ++j)
                if (matrix[i][j] != matrix[i + 1][j + 1])
                    return false;
        return true;
    }
}
```

Java

Matrix - LeetCode - By Pattern

— 1538 —

LeetCode

```
class Solution {
    public boolean isToeplitzMatrix(int[][] matrix) {
        if (matrix.length == 0)
            return true;

        final int m = matrix.length;
        final int n = matrix[0].length;
        int[] buffer = new int[n];

        // Load the row(0) to the buffer.
        for (int j = 0; j < n; ++j)
            buffer[j] = matrix[0][j];

        // Roll the array.
        for (int i = 1; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (buffer[j] != matrix[i][j + 1])
                    return false;
                buffer = matrix[i];
            }
        }
        return true;
    }
}
```

Java

```
class Solution {
    public boolean isToeplitzMatrix(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        for (int i = 1; i < m; ++i) {
            for (int j = 1; j < n; ++j) {
                if (matrix[i][j] != matrix[i - 1][j - 1]) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

Java

```
class Solution {
    public boolean isToeplitzMatrix(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        for (int i = 1; i < m; ++i) {
            for (int j = 1; j < n; ++j) {
                if (matrix[i][j] != matrix[i - 1][j - 1]) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

JavaScript

JavaScript

Matrix - LeetCode - By Pattern

— 1539 —

LeetCode

```
// @param {number[][]} matrix
// @return {boolean}
//
var isToeplitzMatrix = function (matrix) {
    const [m, n] = [matrix.length, matrix[0].length];
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            if (matrix[i][j] !== matrix[i - 1][j - 1]) {
                return false;
            }
        }
    }
    return true;
};
```

JavaScript

```
// @param {number[][]} matrix
// @return {boolean}
//
var isToeplitzMatrix = function (matrix) {
    const m = matrix.length;
    const n = matrix[0].length;
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            if (matrix[i][j] != matrix[i - 1][j - 1]) {
                return false;
            }
        }
    }
    return true;
};
```

TypeScript

```
TypeScript
function isToeplitzMatrix(matrix: number[][]): boolean {
    const [m, n] = [matrix.length, matrix[0].length];
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            if (matrix[i][j] !== matrix[i - 1][j - 1]) {
                return false;
            }
        }
    }
    return true;
}

TypeScript
```

Matrix - LeetCode - By Pattern

— 1540 —

LeetCode

```
function isToeplitzMatrix(matrix: number[][]): boolean {
    const [m, n] = [matrix.length, matrix[0].length];
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            if (matrix[i][j] !== matrix[i - 1][j - 1]) {
                return false;
            }
        }
    }
    return true;
}
```

Go

Go

```
func isToeplitzMatrix(matrix [][]int) bool {
    m, n := len(matrix), len(matrix[0])
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            if matrix[i][j] != matrix[i-1][j-1] {
                return false
            }
        }
    }
    return true
}
```

Go

```
func isToeplitzMatrix(matrix [][]int) bool {
    m, n := len(matrix), len(matrix[0])
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            if matrix[i][j] != matrix[i-1][j-1] {
                return false
            }
        }
    }
    return true
}
```

Rust

Rust

```
impl Solution {
    pub fn isToeplitz_matrix(matrix: Vec<Vec<i32>>)-> bool {
        let (m, n) = (matrix.len(), matrix[0].len());
        for i in 1..m {
            for j in 1..n {
                if matrix[i][j] != matrix[i - 1][j - 1] {
                    return false;
                }
            }
        }
        true
    }
}
```

Matrix - LeetCode - By Pattern

— 1541 —

LeetCode

Rust

```
impl Solution {
    pub fn isToeplitz_matrix(matrix: Vec<Vec<i32>>)-> bool {
        let (m, n) = (matrix.len(), matrix[0].len());
        for i in 1..m {
            for j in 1..n {
                if matrix[i][j] != matrix[i - 1][j - 1] {
                    return false;
                }
            }
        }
        true
    }
}
```

[*] Matrix — Pattern Solution (stored optimal)

Python

```
# Function to determine if the matrix is Toeplitz
def isToeplitzMatrix(matrix):
    # Get the number of rows and columns in the matrix
    rows = len(matrix)
    cols = len(matrix[0])

    # Iterate starting from row 1 and column 1 because first row and column are trivially Toeplitz
    for i in range(1, rows):
        for j in range(1, cols):
            # Compare current element with its top-left neighbor
            if matrix[i][j] != matrix[i-1][j-1]:
                return False
    return True

# Example usage:
matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
print(isToeplitzMatrix(matrix)) # Expected output: True
```

C++

Matrix - LeetCode - By Pattern

— 1542 —

LeetCode

```
// Function to determine if the matrix is Toeplitz using C++
#include <iostream>
using namespace std;

bool isToeplitzMatrix(const vector<vector<int>>& matrix) {
    // Get the number of rows and columns
    int rows = matrix.size();
    int cols = matrix[0].size();

    // Iterate starting from the second row and second column
    for (int i = 1; i < rows; i++) {
        for (int j = 1; j < cols; j++) {
            // Compare current element with its top-left neighbor
            if (matrix[i][j] != matrix[i-1][j-1]) {
                return false;
            }
        }
    }
    return true;
}

int main() {
    // Example usage:
    vector<vector<int>> matrix = {{1,2,3,4}, {5,1,2,3}, {9,5,1,2}};

    cout << (isToeplitzMatrix(matrix) ? "true" : "false") << endl; // Expected output: true
    return 0;
}
```

#867

EASY

Transpose Matrix

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Matrix · Simulation

List CodeSignal

Problem:

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.

Matrix · LeetCode · By Pattern

— 1543 —

LeetCode

2	4	-1
-10	5	11
18	-7	6



2	-10	-18
4	5	-7
-1	11	6

Example 1:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [[1,4,7],[2,5,8],[3,6,9]]

Example 2:

Input: matrix = [[1,2,3],[4,5,6]]

Output: [[1,4],[2,5],[3,6]]

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 1000
- 1 <= m * n <= 105
- 109 <= matrix[i][j] <= 109

>> Brute Force | Matrix Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def transpose(self, matrix):
        # Brute Force O(n^2 * 2) - Flood fill from every cell
        m, n = len(matrix), len(matrix[0])
        result = []
        for r in range(m):
            for c in range(n):
                visited = set()
                def flood(r, c):
                    if (r, c) in visited or not (0 <= r < m and 0 <= c < n): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: flood(r+dr, c+dc)
                flood(r, c)
            result = ans(result, len(visited))
        return result
```

Python

Python

Matrix · LeetCode · By Pattern

— 1544 —

LeetCode

```
# Python solution with clear comments
def transpose(matrix):
    # Number of rows in the original matrix
    num_rows = len(matrix)
    # Number of columns in the original matrix
    num_cols = len(matrix[0])
    # Create a new matrix with dimensions swapped: num_cols x num_rows
    transposed = [[0] * num_cols for _ in range(num_rows)]
    # Iterate through each element in the original matrix
    for i in range(num_rows):
        for j in range(num_cols):
            # Store each element at the swapped index in the transposed matrix
            transposed[j][i] = matrix[i][j]
    return transposed

# Example usage:
matrix = [[1, 2, 3], [4, 5, 6]]
print(transpose(matrix))
```

```
Python
class Solution:
    def transpose(self, A: List[List[int]]) -> List[List[int]]:
        ans = [[0] * len(A) for _ in range(len(A[0]))]
        for i in range(len(A)):
            for j in range(len(A[0])):
                ans[j][i] = A[i][j]
        return ans
```

```
Python
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        return list(zip(*matrix))
```

```
Python
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        return list(zip(*matrix))
```

```
C++
C++
```

Matrix · LeetCode · By Pattern

— 1545 —

LeetCode

```
class Solution {
public:
    vector<vector<int>> transpose(vector<vector<int>>& A) {
        vector<vector<int>> ans(A[0].size(), vector<int>(A.size()));
        for (int i = 0; i < A.size(); ++i)
            for (int j = 0; j < A[0].size(); ++j)
                ans[j][i] = A[i][j];
        return ans;
    }
};
```

```
C++
class Solution {
public:
    vector<vector<int>> transpose(vector<vector<int>>& matrix) {
        int m = matrix.size(), n = matrix[0].size();
        vector<vector<int>> ans(n, vector<int>(m));
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                ans[j][i] = matrix[i][j];
        return ans;
    }
};
```

```
C++
class Solution {
public:
    vector<vector<int>> transpose(vector<vector<int>>& matrix) {
        int m = matrix.size(), n = matrix[0].size();
        vector<vector<int>> ans(n, vector<int>(m));
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                ans[j][i] = matrix[i][j];
        return ans;
    }
};
```

```
Java
class Solution {
    public List<List<Integer>> transpose(List<List<Integer>> A) {
        List<List<Integer>> ans = new List<List<Integer>>(A.size());
        for (int i = 0; i < A.size(); ++i)
            for (int j = 0; j < A.get(i).size(); ++j)
                ans[j].add(A.get(i).get(j));
        return ans;
    }
}
```

Java

Matrix · LeetCode · By Pattern

— 1546 —

LeetCode

```
class Solution {
    public List<List<Integer>> transpose(List<List<Integer>> matrix) {
        int m = matrix.length, n = matrix[0].length;
        List<List<Integer>> ans = new List<List<Integer>>(n);
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                ans[i].add(matrix[j].get(j));
        return ans;
    }
}
```

```
Java
class Solution {
    public List<List<Integer>> transpose(List<List<Integer>> matrix) {
        int m = matrix.length, n = matrix[0].length;
        List<List<Integer>> ans = new List<List<Integer>>(n);
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                ans[i].add(matrix[j].get(j));
        return ans;
    }
}
```

```
JavaScript
JavaScript
// @param {number[][]} matrix
// @return {number[][]}
var transpose = function (matrix) {
    const m = matrix.length;
    const ans = new Array(m).fill(0).map(() => new Array(n).fill(0));
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            ans[j][i] = matrix[i][j];
    return ans;
};
```

JavaScript

Matrix · LeetCode · By Pattern

— 1547 —

LeetCode

```
// @param {number[][]} matrix
// @return {number[][]}
var transpose = function (matrix) {
    const m = matrix.length;
    const ans = new Array(m).fill(0).map(() => new Array(n).fill(0));
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            ans[j][i] = matrix[i][j];
    return ans;
};
```

```
TypeScript
TypeScript
function transpose(matrix: number[][]): number[][] {
    const [m, n] = [matrix.length, matrix[0].length];
    const ans: number[][] = Array.from({ length: n }, () => Array(n).fill(0));
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            ans[j][i] = matrix[i][j];
    return ans;
}
```

```
TypeScript
function transpose(matrix: number[][]): number[][] {
    const [m, n] = [matrix.length, matrix[0].length];
    const ans: number[][] = Array.from({ length: n }, () => Array(n).fill(0));
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            ans[j][i] = matrix[i][j];
    return ans;
}
```

```
Go
Go
func transpose(matrix [][]int) [][]int {
    m, n := len(matrix), len(matrix[0])
    ans := make([][]int, n)
    for i := range ans {
        ans[i] = make([]int, m)
        for j := range ans[i] {
            ans[i][j] = matrix[j][i]
        }
    }
    return ans
}
```

Matrix · LeetCode · By Pattern

— 1548 —

LeetCode


```

class Solution {
public: int[][] largestLocal(int[][] grid) {
    int n = grid.length;
    int[][] ans = new int[n - 2][n - 2];
    for (int i = 0; i < n - 2; ++i) {
        for (int j = 0; j < n - 2; ++j) {
            for (int a = i; a <= i + 2; ++a) {
                for (int b = j; b <= j + 2; ++b) {
                    ans[i][j] = Math.max(ans[i][j], grid[a][b]);
                }
            }
        }
    }
    return ans;
}
}

```

Java

```

class Solution {
public: int[][] largestLocal(int[][] grid) {
    int n = grid.length;
    int[][] ans = new int[n - 2][n - 2];
    for (int i = 0; i < n - 2; ++i) {
        for (int j = 0; j < n - 2; ++j) {
            for (int a = i; a <= i + 2; ++a) {
                for (int b = j; b <= j + 2; ++b) {
                    ans[i][j] = Math.max(ans[i][j], grid[a][b]);
                }
            }
        }
    }
    return ans;
}
}

```

TypeScript

```

function largestLocal(grid: number[][]): number[][] {
    const n = grid.length;
    const res = Array.from({ length: n - 2 }, () => new Array(n - 2).fill(0));
    for (let i = 0; i < n - 2; i++) {
        for (let j = 0; j < n - 2; j++) {
            let max = 0;
            for (let k = i; k <= i + 2; k++) {
                for (let l = j; l <= j + 2; l++) {
                    max = Math.max(max, grid[k][l]);
                }
            }
            res[i][j] = max;
        }
    }
    return res;
}

```

TypeScript

Matrix · LeetCode · By Pattern — 1555 — LeetCode

```

function largestLocal(grid: number[][]): number[][] {
    const n = grid.length;
    const res = Array.from({ length: n - 2 }, () => new Array(n - 2).fill(0));
    for (let i = 0; i < n - 2; i++) {
        for (let j = 0; j < n - 2; j++) {
            let max = 0;
            for (let k = i; k <= i + 2; k++) {
                for (let l = j; l <= j + 2; l++) {
                    max = Math.max(max, grid[k][l]);
                }
            }
            res[i][j] = max;
        }
    }
    return res;
}

```

Go

```

func largestLocal(grid [][]int) [][]int {
    n := len(grid)
    ans := make([][]int, n-2)
    for i := range ans {
        ans[i] = make([]int, n-2)
        for j := 0; j < n-2; j++ {
            for k := i; k <= i+2; k++ {
                for y := j; y <= j+2; y++ {
                    ans[i][j] = Max(ans[i][j], grid[k][y])
                }
            }
        }
    }
    return ans
}

```

Go

```

func largestLocal(grid [][]int) [][]int {
    n := len(grid)
    ans := make([][]int, n-2)
    for i := range ans {
        ans[i] = make([]int, n-2)
        for j := 0; j < n-2; j++ {
            for k := i; k <= i+2; k++ {
                for y := j; y <= j+2; y++ {
                    ans[i][j] = max(ans[i][j], grid[k][y])
                }
            }
        }
    }
    return ans
}

```

Go

[*] Matrix — Pattern Solution (stored optimal)

Python

Matrix · LeetCode · By Pattern — 1556 — LeetCode

```

# Python solution for finding the largest local values in a matrix
def largestLocal(grid):
    n = len(grid) # dimension of the grid
    # Initialize the result matrix with zeros, of size (n-2) x (n-2)
    result = [[0] * (n - 2) for _ in range(n - 2)]

    # Loop through each possible top-left index of a 3x3 submatrix
    for i in range(n - 2):
        for j in range(n - 2):
            # Compute the maximum in the current 3x3 submatrix
            local_max = 0
            # Since grid elements are >= 1, starting with 0 is safe
            for k in range(3):
                for l in range(3):
                    local_max = max(local_max, grid[i + k][j + l])
            # Store the local maximum in the result matrix
            result[i][j] = local_max

    return result

# Example usage:
grid = [[9,9,8,1],[5,6,2,4],[8,2,6,4],[6,3,2,2]]
print(largestLocal(grid))

```

C++

```

// C++ solution for finding the largest local values in a matrix
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<vector<int>> largestLocal(vector<vector<int>>& grid) {
    int n = grid.size(); // dimension of the grid
    // Initialize the result matrix with dimensions (n-2) x (n-2)
    vector<vector<int>> result(n - 2, vector<int>(n - 2, 0));

    // Loop through each possible top-left index of a 3x3 submatrix
    for (int i = 0; i < n - 2; ++i) {
        for (int j = 0; j < n - 2; ++j) {
            int localMax = 0; // Since grid elements are >= 1, 0 is safe as initial value
            // Check each element in the 3x3 submatrix
            for (int k = i; k <= i + 2; ++k) {
                for (int l = j; l <= j + 2; ++l) {
                    localMax = max(localMax, grid[k][l]);
                }
            }
            // Store the computed maximum in the result matrix
            result[i][j] = localMax;
        }
    }

    return result;
}

```

Matrix · LeetCode · By Pattern — 1557 — LeetCode

5	3			7			
6			1	9	5		
	9	8					6
8				6			3
4		8		3			1
7				2			6
	6						
				4	1	9	
				8			7
							9

Input: board =
[[["5","3",".", ".", ".", "7",".", ".", "."],
["6",".", ".", "1","9","5",".", ".", "."],
[".", "9","8",".", ".", ".", ".", "6"],
["8",".", ".", ".", "6",".", ".", "3"],
["4",".", ".", "8",".", "3",".", ".", "1"],
["7",".", ".", ".", "2",".", ".", ".", "6"],
[".", "6",".", ".", ".", ".", "4","1","9"],
[".", ".", ".", "4","1","9",".", ".", "5"],
[".", ".", ".", "8",".", ".", "7","9"]]]
Output: true

Example 2:

Input: board =
[[["8","3",".", ".", ".", "7",".", ".", "."],
["6",".", ".", "1","9","5",".", ".", "."],
[".", "9","8",".", ".", ".", ".", "6"],
["8",".", ".", ".", "6",".", ".", "3"],
["4",".", ".", "8",".", "3",".", ".", "1"],
["7",".", ".", ".", "2",".", ".", ".", "6"],
[".", "6",".", ".", ".", ".", "4","1","9"],
[".", ".", ".", "4","1","9",".", ".", "5"],
[".", ".", ".", "8",".", ".", "7","9"]]]
Output: false

Explanation: Same as Example 1, except with the 5 in the top left corner being modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.

Constraints:

- board.length == 9
- board[i].length == 9
- board[i][j] is a digit 1-9 or '.'.

Matrix · LeetCode · By Pattern — 1559 — LeetCode

```

}
return result;
}

int main() {
    vector<vector<int>> grid = {
        {5,9,8,1},
        {5,6,2,4},
        {8,2,6,4},
        {6,3,2,2}
    };

    vector<vector<int>> result = largestLocal(grid);

    // Print the result matrix
    for (auto& row : result) {
        for (int num : row) {
            cout << num << " ";
        }
        cout << "\n";
    }

    return 0;
}

```

#36 MEDIUM

Valid Sudoku

LeetCode · SimplyLeet · WalkCC · LeetCode · Editorial

Array · Hash Table · Matrix

Lines: 6/10/10

Problem:

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

- Each row must contain the digits 1-9 without repetition.
- Each column must contain the digits 1-9 without repetition.
- Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:

Matrix · LeetCode · By Pattern — 1558 — LeetCode

```

>> Brute Force [Matrix] Interview Prep
Python
# Brute Force Approach
def isValidSudoku(self, board):
    # Brute Force will check each row, col, box for duplicates
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    for r in range(9):
        for c in range(9):
            val = board[r][c]
            if val == ".": continue
            box_idx = (r // 3) * 3 + (c // 3)
            if val in rows[r] or val in cols[c] or val in boxes[box_idx]:
                return False
            rows[r].add(val); cols[c].add(val); boxes[box_idx].add(val)
    return True

```

Python

```

# Python solution with inline comments
def isValidSudoku(board):
    # Initialize sets for rows, columns, and boxes
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]

    # Iterate over each cell in the board
    for r in range(9):
        for c in range(9):
            num = board[r][c]
            # Skip empty cells
            if num == ".":
                continue
            # Compute box index using integer division
            box_index = (r // 3) * 3 + (c // 3)
            # Check if the number is already present in the row, column, or box
            if num in rows[r] or num in cols[c] or num in boxes[box_index]:
                return False
            # Add number to row, column, and box sets
            rows[r].add(num)
            cols[c].add(num)
            boxes[box_index].add(num)
    return True

```

Matrix · LeetCode · By Pattern — 1560 — LeetCode

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

```

public class Solution {
    public bool isValidSudoku(char[][] board) {
        bool[] row = new bool[9, 0];
        bool[] col = new bool[9, 0];
        bool[] sub = new bool[9, 0];

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                char c = board[i][j];
                if (c == '.') {
                    continue;
                }
                int num = c - '0' - 1;
                int k = (i / 3) * 3 + (j / 3);
                if (row[i][num] || col[j][num] || sub[k, num]) {
                    return false;
                }
                row[i][num] = true;
                col[j][num] = true;
                sub[k, num] = true;
            }
        }
        return true;
    }
}

```

JavaScript

```

// =
function isValidSudoku(board) {
    // Return (character[][] board)
    // Return (boolean)
    // =
    var isValidSudoku = function (board) {
        const row = [...Array(9)].map() => Array(9).fill(false);
        const col = [...Array(9)].map() => Array(9).fill(false);
        const sub = [...Array(9)].map() => Array(9).fill(false);
        for (let i = 0; i < 9; i++) {
            for (let j = 0; j < 9; j++) {
                const num = board[i][j].charCodeAt() - '1'.charCodeAt();
                if (num > 9 || num < 0) {
                    continue;
                }
                const k = Math.floor(i / 3) + 3 * Math.floor(j / 3);
                if (row[i][num] || col[j][num] || sub[k][num]) {
                    return false;
                }
                row[i][num] = true;
                col[j][num] = true;
                sub[k][num] = true;
            }
        }
        return true;
    };
}

```

TypeScript

TypeScript

Matrix · LeetCode · By Pattern

— 1567 —

LeetCode

```

function isValidSudoku(board: string[][]): boolean {
    const row: boolean[][] = Array.from({ length: 9 }, () =>
        Array.from({ length: 9 }, () => false));
    };
    const col: boolean[][] = Array.from({ length: 9 }, () =>
        Array.from({ length: 9 }, () => false));
    };
    const sub: boolean[][] = Array.from({ length: 9 }, () =>
        Array.from({ length: 9 }, () => false));
    };
    for (let i = 0; i < 9; i++) {
        for (let j = 0; j < 9; j++) {
            const num = board[i][j].charCodeAt() - '1'.charCodeAt();
            if (num < 0 || num > 9) {
                continue;
            }
            const k = Math.floor(i / 3) + 3 * Math.floor(j / 3);
            if (row[i][num] || col[j][num] || sub[k][num]) {
                return false;
            }
            row[i][num] = true;
            col[j][num] = true;
            sub[k][num] = true;
        }
    }
    return true;
}

```

Rust

Rust

```

impl Solution {
    pub fn is_valid_sudoku(board: Vec<Vec<char>>) -> bool {
        let mut row = vec![vec![false; 9]; 9];
        let mut col = vec![vec![false; 9]; 9];
        let mut sub = vec![vec![false; 9]; 9];

        for i in 0..9 {
            for j in 0..9 {
                let c = board[i][j];
                if c == '.' {
                    continue;
                }
                let num = (c as u8 - b'0' - 1) as usize;
                let k = i / 3 + 3 * j / 3;
                if row[i][num] || col[j][num] || sub[k][num] {
                    return false;
                }
                row[i][num] = true;
                col[j][num] = true;
                sub[k][num] = true;
            }
        }
        true
    }
}

```

Rust

Matrix · LeetCode · By Pattern

— 1568 —

LeetCode

```

impl Solution {
    pub fn is_valid_sudoku(board: Vec<Vec<char>>) -> bool {
        let mut row = vec![vec![false; 9]; 9];
        let mut col = vec![vec![false; 9]; 9];
        let mut sub = vec![vec![false; 9]; 9];

        for i in 0..9 {
            for j in 0..9 {
                let c = board[i][j];
                if c == '.' {
                    continue;
                }
                let num = (c as u8 - b'0' - 1) as usize;
                let k = i / 3 + 3 * j / 3;
                if row[i][num] || col[j][num] || sub[k][num] {
                    return false;
                }
                row[i][num] = true;
                col[j][num] = true;
                sub[k][num] = true;
            }
        }
        true
    }
}

```

[*] Matrix — Pattern Solution (stored optimal)

Python

```

# Python solution with inline comments
def isValidSudoku(board):
    # Initialize sets for row, column, and boxes
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]

    # Iterate over each cell in the board
    for r in range(9):
        for c in range(9):
            num = board[r][c]
            # Skip empty cells
            if num == '.':
                continue
            # Compute box index using integer division
            box_index = (r // 3) * 3 + (c // 3)
            # Check if the number is already present in the row, column, or box
            if num in rows[r] or num in cols[c] or num in boxes[box_index]:
                return False
            # Add number to row, column, and box sets
            rows[r].add(num)
            cols[c].add(num)
            boxes[box_index].add(num)

    return True

```

Matrix · LeetCode · By Pattern

— 1569 —

LeetCode

```

# Example usage:
if __name__ == "__main__":
    board = [
        ["5","3",".", ".", ".", ".", ".", ".", ".", "."],
        ["6",".", ".", "4",".", ".", "8",".", ".", "."],
        [".","9","8",".", ".", ".", ".", ".", ".", "."],
        ["8",".", ".", "3",".", ".", ".", ".", ".", "."],
        [".","4",".", ".", ".", ".", "9",".", ".", "."],
        [".","9",".", ".", "5",".", ".", ".", ".", "."],
        [".","8",".", ".", ".", ".", ".", ".", ".", "."],
        [".",".", "6",".", ".", "3",".", ".", ".", "."],
        [".",".", "7",".", ".", ".", ".", ".", ".", "."]
    ]
    print(isValidSudoku(board)) # Expected output: True

```

C++

```

// C++ solution with inline comments
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {
        // Vectors of unordered sets for row, column, and boxes
        vector<unordered_set<char>> rows(9), cols(9), boxes(9);

        // Iterate through the board
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                char num = board[r][c];
                // Skip if cell is empty
                if (num == '.') continue;
                // Calculate box index
                int boxIndex = (r / 3) * 3 + (c / 3);
                // Check if the number is already present in the row, column or box
                if (rows[r].count(num) || cols[c].count(num) || boxes[boxIndex].count(num)) {
                    return false;
                }
            }
        }
        // Insert the number into the respective sets
    }
}

```

Matrix · LeetCode · By Pattern

— 1570 —

LeetCode

```

        row[r].insert(num);
        col[c].insert(num);
        boxes[boxIndex].insert(num);
    }
}
return true;
};

```

int main()

```

// Sample board input
vector<vector<char>> board = {
    {"5","3",".", ".", ".", ".", ".", ".", ".", "."},
    {"6",".", ".", "4",".", ".", "8",".", ".", "."},
    [".","9","8",".", ".", ".", ".", ".", ".", "."],
    {"8",".", ".", "3",".", ".", ".", ".", ".", "."},
    [".","4",".", ".", ".", ".", "9",".", ".", "."],
    [".","9",".", ".", "5",".", ".", ".", ".", "."],
    [".","8",".", ".", ".", ".", ".", ".", ".", "."],
    [".",".", "6",".", ".", "3",".", ".", ".", "."],
    [".",".", "7",".", ".", ".", ".", ".", ".", "."]
};

// Solution call
bool result = isValidSudoku(board); // true; // Expected output: true

return 0;
}

```

#48

MEDIUM

Rotate Image

LeetCode · Simple · C++ · WUBCC · LeetCode · Editorial

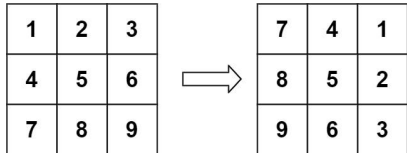
Array · Math · Matrix

List · Grid · 169 | CodeSignal

Problem:

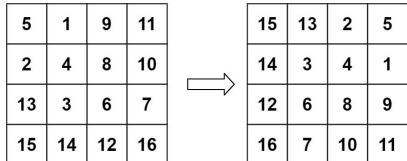
You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise). You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:



Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

>> Brute Force (Matrix) Interview Prep

Python

Matrix · LeetCode · By Pattern

— 1571 —

LeetCode

Matrix · LeetCode · By Pattern

— 1572 —

LeetCode

```
# Brute Force Approach
class Solution:
    def rotate(self, matrix):
        # Brute Force O(N^2) space - copy to new matrix with rotated indices
        n = len(matrix)
        copy = [row[:] for row in matrix]
        for i in range(n):
            for j in range(n):
                matrix[i][n-1-j] = copy[j][i]
```

Python

Python

```
def rotate(matrix):
    n = len(matrix)
    # Transpose the matrix by swapping elements across the diagonal
    for i in range(n):
        for j in range(n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each row to complete the clockwise rotation
    for i in range(n):
        matrix[i].reverse()

# Example usage:
matrix = [[1,2,3],[4,5,6],[7,8,9]]
rotate(matrix)
print(matrix) # Output: [[7,4,1],[6,5,2],[9,6,3]]
```

Python

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        matrix.reverse()
        for i, j in itertools.combinations(range(len(matrix)), 2):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

Python

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        for m in range(len(matrix) // 2):
            m1, m2 = m, len(matrix) - m - 1
            for i in range(m, m2):
                offset = i - m
                temp = matrix[m1][i]
                matrix[m1][i] = matrix[m2 - offset][m1]
                matrix[m2 - offset][m1] = matrix[m2][m1 - offset]
                matrix[m2][m1 - offset] = matrix[i][m2]
                matrix[i][m2] = temp
```

Python

Matrix · LeetCode · By Pattern

— 1573 —

LeetCode

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n - 1):
            for j in range(n):
                matrix[i][j], matrix[n - 1 - j][i] = matrix[n - 1 - j][i], matrix[i][j]
            for i in range(n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

Python

```
class Solution(object):
    def rotate(self, matrix):
        (type matrix, List[List[int]])
        (type void do not return anything, modify matrix in-place instead.)
        if len(matrix) == 0:
            return
        n = len(matrix)
        w = len(matrix[0])
        for i in range(n, w // 2):
            matrix[i][w - j - 1] = matrix[i][w - j - 1], matrix[i][j]
        for i in range(n, w // 2):
            for j in range(n, w - 1 - i):
                matrix[i][j], matrix[w - 1 - j][n - 1 - i] = matrix[w - 1 - j][n - 1 - i], matrix[i][j]
```

#####

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n):
            for j in range(n):
                matrix[i][j], matrix[n - 1 - j][i] = matrix[n - 1 - j][i], matrix[i][j]
            for i in range(n):
                for j in range(n):
                    matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

C++

C++

```
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        reverse(matrix);
        for (int i = 0; i < matrix.size(); ++i)
            for (int j = 0; j < matrix.size(); ++j)
                swap(matrix[i][j], matrix[j][i]);
    }
};
```

C++

Matrix · LeetCode · By Pattern

— 1574 —

LeetCode

```
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        for (int m = 0; m < matrix.size() / 2; ++m) {
            const int m1 = matrix.size() - m - 1;
            for (int i = m; i < m1; ++i) {
                const int offset = i - m;
                const int top = matrix[m1][i];
                matrix[m1][i] = matrix[m1 - offset][m1];
                matrix[m1 - offset][m1] = matrix[m1][m1 - offset];
                matrix[m1][m1 - offset] = matrix[i][m1];
                matrix[i][m1] = top;
            }
        }
    }
};
```

C++

```
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n = matrix.size();
        for (int i = 0; i < n - 1; ++i)
            for (int j = 0; j < n; ++j) {
                swap(matrix[i][j], matrix[n - 1 - j][i]);
            }
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                swap(matrix[i][j], matrix[j][i]);
            }
    }
};
```

C++

```
// O(1) Space, O(N^2) Time
// Time: O(N^2)
// Space: O(1)
class Solution {
public:
    void rotate(vector<vector<int>>& A) {
        int N = A.size();
        for (int i = 0; i < N / 2; ++i) {
            for (int j = i; j < N - 1 - i; ++j) {
                int temp = A[i][j];
                A[i][j] = A[N - j - 1][i];
                A[N - j - 1][i] = A[N - j - 1][N - j - 1];
                A[N - j - 1][N - j - 1] = A[j][N - 1 - i];
                A[j][N - 1 - i] = temp;
            }
        }
    }
};
```

Java

Java

Matrix · LeetCode · By Pattern

— 1575 —

LeetCode

```
class Solution {
    public void rotate(int[][] matrix) {
        for (int i = 0; i < matrix.length - 1; i++) {
            int[] temp = matrix[i];
            matrix[i] = matrix[i+1];
            matrix[i+1] = temp;
        }
        for (int i = 0; i < matrix.length; i++)
            for (int j = 0; j < matrix.length; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
    }
}
```

Java

```
class Solution {
    public void rotate(int[][] matrix) {
        for (int m = 0; m < matrix.length / 2; ++m) {
            final int m1 = matrix.length - m - 1;
            for (int i = m; i < m1; ++i) {
                final int offset = i - m;
                final int top = matrix[m1][i];
                matrix[m1][i] = matrix[m1 - offset][m1];
                matrix[m1 - offset][m1] = matrix[m1][m1 - offset];
                matrix[m1][m1 - offset] = matrix[i][m1];
                matrix[i][m1] = top;
            }
        }
    }
}
```

Java

```
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        for (int i = 0; i < n - 1; ++i) {
            for (int j = 0; j < n; ++j) {
                int t = matrix[i][j];
                matrix[i][j] = matrix[n - 1 - j][i];
                matrix[n - 1 - j][i] = t;
            }
        }
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                int t = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = t;
            }
        }
    }
}
```

Java

Matrix · LeetCode · By Pattern

— 1576 —

LeetCode

```
public class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        for (int i = 0; i < n - 1; ++i) {
            for (int j = 0; j < n; ++j) {
                int t = matrix[i][j];
                matrix[i][j] = matrix[n - 1 - j][i];
                matrix[n - 1 - j][i] = t;
            }
        }
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                int t = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = t;
            }
        }
    }
}
```

Java

```
public class Rotate_Image {
    public class Solution {
        // n x m
        1 2 3 4 5
        6 7 8 9 10
        11 12 13 14 15
        16 17 18 19 20
        21 22 23 24 25
        //
        public void rotate(int[][] m) { // n for matrix
            if (m == null || m.length == 0) {
                return;
            }
            // for each circle, start position is (1,1), length is rectangle size
            // eg. above: (8,8), length=8, (1,1), length=1
            int i = 0;
            int length = m.length;
            while (i < m.length / 2) {
```

Matrix · LeetCode · By Pattern

— 1577 —

LeetCode

```
int count = 0;
while (count < length - 1) { // (note: extra attention here "-1")
    // while count < length-1
    int temp = m[i][i + count];
    // (length-1): last index of this rectangle
    m[i][i + count] = m[i + length - 1 - count][i];
    m[i + length - 1 - count][i] = m[i + length - 1][i + length - 1 - count];
    m[i + length - 1][i + length - 1 - count] = m[i + count][i + length - 1];
    m[i + count][i + length - 1] = temp;
    count++;
}
```

```
length -= 2; // (note: shrink each edge length by 2
keep; // start point moving along diagonal
```

Java

```
// An Inplace function to rotate a N x N matrix by 90 degrees in anti-clockwise direction
static void rotateMatrix_anticlockwise(int N, int mat[][]) {
```

```
{
    // Consider all squares one by one
    for (int x = 0; x < N / 2; ++x) {
        // Consider elements in group of 4 in
        // current square
        for (int y = x; y < N-x-1; ++y) {
            // Store current cell in temp variable
            int temp = mat[x][y];
            // Move values from right to top
            mat[x][y] = mat[y][N-x-1];
            // Move values from bottom to right
            mat[y][N-x-1] = mat[N-1-x][N-1-y];
            // Move values from left to bottom
            mat[N-1-x][N-1-y] = mat[N-1-y][x];
            // Assign temp to left
            mat[N-1-y][x] = temp;
        }
    }
}
```

```
static void rotateMatrix_clockwise(int N, int mat[][]) {
    // Consider all squares one by one
    for (int x = 0; x < N / 2; ++x) {
        // Consider elements in group of 4 in
        // current square
        for (int y = x; y < N-x-1; ++y) {
```

Java

Matrix · LeetCode · By Pattern

— 1578 —

LeetCode

```

public class Solution {
    public void rotate(int[][] matrix) {
        let n = matrix.length;
        for (let i = 0; i < n; ++i) {
            for (let j = 0; j < n; ++j) {
                let t = matrix[i][j];
                matrix[i][j] = matrix[n - 1 - j][i];
                matrix[n - 1 - j][i] = t;
            }
        }
    }
}

JavaScript
JavaScript
// @param {number[][]} matrix
// return {void} Do not return anything, modify matrix in-place instead.
//
var rotate = function (matrix) {
    matrix.reverse();
    for (let i = 0; i < matrix.length; ++i) {
        for (let j = 0; j < i; ++j) {
            [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];
        }
    }
};

// @param {number[][]} matrix
// return {void} Do not return anything, modify matrix in-place instead.
//
var rotate = function (matrix) {
    matrix.reverse();
    for (let i = 0; i < matrix.length; ++i) {
        for (let j = 0; j < i; ++j) {
            [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];
        }
    }
};

TypeScript
TypeScript

```

Matrix · LeetCode — By Pattern — 1579 — LeetCode

```

// Do not return anything, modify matrix in-place instead.
//
function rotate(matrix: number[][]): void {
    matrix.reverse();
    for (let i = 0; i < matrix.length; ++i) {
        for (let j = 0; j < i; ++j) {
            const t = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = t;
        }
    }
}

TypeScript
TypeScript
// Do not return anything, modify matrix in-place instead.
//
function rotate(matrix: number[][]): void {
    let n = matrix[0].length;
    for (let i = 0; i < Math.floor(n / 2); ++i) {
        for (let j = 0; j < Math.floor(n - 1 - 2 * i); ++j) {
            let tmp = matrix[i][j];
            matrix[i][j] = matrix[n - 1 - j][i];
            matrix[n - 1 - j][i] = matrix[n - 1 - j][n - 1 - j];
            matrix[n - 1 - j][n - 1 - j] = matrix[i][j];
            matrix[i][j] = tmp;
        }
    }
}

Go
Go
func rotate(matrix [][]int) {
    n := len(matrix)
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
            matrix[i][j], matrix[n-1-j][i] = matrix[n-1-j][i], matrix[i][j]
        }
    }
    for i := 0; i < n; i++ {
        for j := 0; j < i; j++ {
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
        }
    }
}

Go
Go

```

Matrix · LeetCode — By Pattern — 1580 — LeetCode

```

func rotate(matrix [][]int) {
    n := len(matrix)
    // reverse the matrix
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
            matrix[i][j], matrix[n-1-j][i] = matrix[j][i], matrix[i][j]
        }
    }
    // reverse each matrix[i]
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
            matrix[i][j], matrix[n-1-j][i] = matrix[j][i], matrix[i][j]
        }
    }
}

Rust
Rust
impl Solution {
    pub fn rotate(matrix: &mut Vec<Vec<i32>>()) {
        let n = matrix.len();
        for i in 0..n {
            for j in 0..n {
                let t = matrix[i][j];
                matrix[i][j] = matrix[n - 1 - j][i];
                matrix[n - 1 - j][i] = t;
            }
        }
        for i in 0..n {
            for j in 0..i {
                let t = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = t;
            }
        }
    }
}

Rust
impl Solution {
    pub fn rotate(matrix: &mut Vec<Vec<i32>>()) {
        let n = matrix.len();
        for i in 0..n {
            for j in 0..n {
                let t = matrix[i][j];
                matrix[i][j] = matrix[n - 1 - j][i];
                matrix[n - 1 - j][i] = t;
            }
        }
        for i in 0..n {
            for j in 0..i {
                let t = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = t;
            }
        }
    }
}

[*] Matrix — Pattern Solution (stored optimal)
Python

```

Matrix · LeetCode — By Pattern — 1581 — LeetCode

```

def rotate(matrix):
    n = len(matrix)
    # Transpose the matrix by swapping elements across the diagonal
    for i in range(n):
        for j in range(i, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each row to complete the clockwise rotation
    for i in range(n):
        matrix[i].reverse()

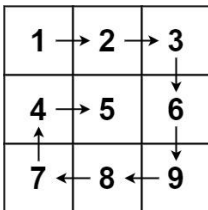
# Example usage:
matrix = [[1,2,3],[4,5,6],[7,8,9]]
rotate(matrix)
print(matrix) # Output: [[7,4,1],[9,5,2],[8,6,3]]

C++
#include <vector>
using namespace std;

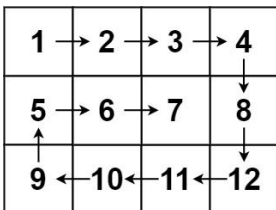
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n = matrix.size();
        // Transpose the matrix by swapping elements across the diagonal
        for (int i = 0; i < n; ++i) {
            for (int j = i; j < n; ++j) {
                swap(matrix[i][j], matrix[j][i]);
            }
        }
        // Reverse each row to complete the rotation
        for (int i = 0; i < n; ++i) {
            reverse(matrix[i].begin(), matrix[i].end());
        }
    }
};

```

#54 MEDIUM
Spiral Matrix
 LeetCode · SimplifyC++ · WubCC · LeetCode · Editorial
 Array · Matrix · Simulation
 Lists: Grind 169 | CodeSignal
Problem:
 Given an m x n matrix, return all elements of the matrix in spiral order.
 Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
 Output: [1,2,3,6,9,8,7,4,5]
 Example 2:



Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
 Output: [1,2,3,4,8,11,10,9,5,6,7]
 Constraints:
 • m == matrix.length
 • n == matrix[i].length
 • 1 <= m, n <= 10
 • -100 <= matrix[i][j] <= 100

Matrix · LeetCode — By Pattern — 1583 — LeetCode

```

>> Brute Force [Matrix] Interview Prep
Python
class Solution:
    def spiralOrder(self, matrix):
        # Brute Force Approach - track visited cells, simulate turning
        if not matrix: return []
        m, n = len(matrix), len(matrix[0])
        directions = [(0,1),(0,-1),(1,0),(-1,0)] # right, down, left, up
        visited = [[False] * n for _ in range(m)]
        r,c,d = 0,0,0
        for _ in range(m*n):
            result.append(matrix[r][c])
            visited[r][c] = True
            dr,dc = directions[d]
            nr,nc = r+dr,c+dc
            if 0 <= nr < m and 0 <= nc < n and not visited[nr][nc]:
                r, c, d = nr, nc, d
            else:
                d = (d+1)%4; dr,dc = directions[d]; r,c = r+dr,c+dc
        return result

Python
Python
def spiralOrder(matrix):
    # Initialize the boundaries of the matrix
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    result = []
    # Traverse the matrix, store the spiral order
    # Loop until the pointers cross each other
    while top <= bottom and left <= right:
        # Traverse from left to right on the current top row
        for col in range(left, right + 1):
            result.append(matrix[top][col])
        top += 1 # Move the top boundary down
        # Traverse downwards on the current right column
        for row in range(top, bottom + 1):
            result.append(matrix[row][right])
        right -= 1 # Move the right boundary to the left
        # Traverse from right to left on the current bottom row, if valid
        if top <= bottom:
            for col in range(right, left - 1, -1):
                result.append(matrix[bottom][col])
            bottom += 1 # Move the bottom boundary up
        # Traverse upwards on the current left column, if valid

```

Matrix · LeetCode — By Pattern — 1584 — LeetCode

```

        if left == right:
            for row in range(bottom, top - 1, -1):
                result.append(matrix[row][left])
            left += 1 # Move the left boundary right

    return result

# Example usage:
# print(spiralOrder([[1,2,3],[4,5,6],[7,8,9]]))

```

Python

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        if not matrix:
            return []

        n = len(matrix)
        m = len(matrix[0])
        ans = []
        r1 = 0
        c1 = 0
        r2 = m - 1
        c2 = n - 1

        # Traversely add matrix[r1...r2][c1...c2] to 'ans'.
        while len(ans) < n * m:
            j = c1
            while j <= c2 and len(ans) < n * m:
                ans.append(matrix[r1][j])
                j += 1
            c1 += 1
            while i <= r2 - 1 and len(ans) < n * m:
                ans.append(matrix[i][c2])
                i += 1
            r2 -= 1
            while j >= c1 and len(ans) < n * m:
                ans.append(matrix[r2][j])
                j -= 1
            r1 += 1
            while i >= r1 + 1 and len(ans) < n * m:
                ans.append(matrix[i][c1])
                i -= 1
            c2 += 1
            r1 += 1
            c1 += 1
            c2 -= 1
            r2 -= 1

        return ans

```

Python

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        vis = [[False] * n for _ in range(m)]
        i = j = k = 0
        ans = []
        for _ in range(n * m):
            ans.append(matrix[i][j])
            vis[i][j] = True
            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x == n or y < 0 or y == m or vis[x][y]:
                k = (k + 1) % 4
                i = dirs[k]
                j = dirs[k + 1]
            return ans

```

Python

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        ans = []
        for _ in range(n * m):
            ans.append(matrix[i][j])
            vis[i][j] = True
            x, y = i + dirs[k], j + dirs[k + 1]
            if not 0 <= x < n or not 0 <= y < m or (x, y) in vis:
                k = (k + 1) % 4
                i = dirs[k]
                j = dirs[k + 1]
            return ans

```

C++

C++

```

class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        int n = matrix.size(), m = matrix[0].size();
        int dirx[] = {0, 1, 0, -1, 0};
        int i = 0, j = 0, k = 0;
        vector<int> ans;
        bool vis[n][m];
        memset(vis, false, sizeof(vis));
        for (int h = n * m; h >= 0; --h) {
            ans.push_back(matrix[i][j]);
            vis[i][j] = true;
            int x = i + dirx[k], y = j + dirx[k + 1];
            if (x < 0 || x == n || y < 0 || y == m || vis[x][y]) {
                k = (k + 1) % 4;
                i = dirs[k];
                j = dirs[k + 1];
            }
            return ans;
        }
    }
};

```

C++

```

class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        int n = matrix.size(), m = matrix[0].size();
        int dirx[] = {0, 1, 0, -1, 0};
        int i = 0, j = 0, k = 0;
        vector<int> ans;
        bool vis[n][m];
        memset(vis, false, sizeof(vis));
        for (int h = n * m; h >= 0; --h) {
            ans.push_back(matrix[i][j]);
            vis[i][j] = true;
            int x = i + dirx[k], y = j + dirx[k + 1];
            if (x < 0 || x == n || y < 0 || y == m || vis[x][y]) {
                k = (k + 1) % 4;
                i = dirs[k];
                j = dirs[k + 1];
            }
            return ans;
        }
    }
};

```

Java

Java

```

class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        int n = matrix.length, m = matrix[0].length;
        int[] dirx = {0, 1, 0, -1, 0};
        int i = 0, j = 0, k = 0;
        List<Integer> ans = new ArrayList<>();
        boolean[][] vis = new boolean[n][m];
        for (int h = n * m; h >= 0; --h) {
            ans.add(matrix[i][j]);
            vis[i][j] = true;
            int x = i + dirx[k], y = j + dirx[k + 1];
            if (x < 0 || x == m || y < 0 || y == n || vis[x][y]) {
                k = (k + 1) % 4;
                i = dirs[k];
                j = dirs[k + 1];
            }
            return ans;
        }
    }
}

```

Java

```

public class Solution {
    public List<int> SpiralOrder(int[][] matrix) {
        int n = matrix.length, m = matrix[0].length;
        int[] dirx = {0, 1, 0, -1, 0};
        int i = 0, j = 0, k = 0;
        List<int> ans = new List<int>();
        boolean[] vis = new boolean[n];
        for (int h = n * m; h >= 0; --h) {
            ans.add(matrix[i][j]);
            vis[i] = true;
            int x = i + dirx[k], y = j + dirx[k + 1];
            if (x < 0 || x == m || y < 0 || y == n || vis[x], y)) {
                k = (k + 1) % 4;
                i = dirs[k];
                j = dirs[k + 1];
            }
            return ans;
        }
    }
}

```

Java

```

class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        int n = matrix.length, m = matrix[0].length;
        int[] dirx = {0, 1, 0, -1, 0};
        int i = 0, j = 0, k = 0;
        List<Integer> ans = new ArrayList<>();
        boolean[][] vis = new boolean[n][m];
        for (int h = n * m; h >= 0; --h) {
            ans.add(matrix[i][j]);
            vis[i][j] = true;
            int x = i + dirx[k], y = j + dirx[k + 1];
            if (x < 0 || x == m || y < 0 || y == n || vis[x][y]) {
                k = (k + 1) % 4;
                i = dirs[k];
                j = dirs[k + 1];
            }
            return ans;
        }
    }
}

```

Java

```

public class Solution {
    public List<int> SpiralOrder(int[][] matrix) {
        int n = matrix.length, m = matrix[0].length;
        int[] dirx = new int[] {0, 1, 0, -1, 0};
        List<int> ans = new List<int>();
        boolean[] visied = new boolean[n];
        for (int h = n * m, i = 0, j = 0, k = 0; h >= 0; --h) {
            ans.add(matrix[i][j]);
            visied[i] = true;
            int x = i + dirx[k], y = j + dirx[k + 1];
            if (x < 0 || x == m || y < 0 || y == n || visied[x], y)) {
                k = (k + 1) % 4;
                i = dirs[k];
                j = dirs[k + 1];
            }
            return ans;
        }
    }
}

```

JavaScript

JavaScript

```

/**
 * @param {number[][]} matrix
 * @return {number[]}
 */
var spiralOrder = function(matrix) {
    const n = matrix.length;
    const m = matrix[0].length;
    const ans = [];
    const vis = Array.from({ length: n }, () => Array(m).fill(false));
    const dirx = [0, 1, 0, -1, 0];
    for (let h = n * m, i = 0, j = 0, k = 0; h >= 0; --h) {
        ans.push(matrix[i][j]);
        vis[i][j] = true;
        const x = i + dirx[k];
        const y = j + dirx[k + 1];
        if (x < 0 || x == m || y < 0 || y == n || vis[x][y]) {
            k = (k + 1) % 4;
            i = dirs[k];
            j = dirs[k + 1];
        }
        return ans;
    }
};

```

JavaScript

```

/**
 * @param {number[][]} matrix
 * @return {number[]}
 */
var spiralOrder = function(matrix) {
    const n = matrix.length;
    const m = matrix[0].length;
    const vis = new Array(n).fill(0).map(() => new Array(m).fill(false));
    const dirx = [0, 1, 0, -1, 0];
    for (let h = n * m, i = 0, j = 0, k = 0; h >= 0; --h) {
        ans.push(matrix[i][j]);
        vis[i][j] = true;
        const x = i + dirx[k];
        const y = j + dirx[k + 1];
        if (x < 0 || x == m || y < 0 || y == n || vis[x][y]) {
            k = (k + 1) % 4;
            i = dirs[k];
            j = dirs[k + 1];
        }
        return ans;
    }
};

```

TypeScript

TypeScript

```
function spiralOrder(matrix: number[][]): number[] {
    const n = matrix.length;
    const m = matrix[0].length;
    const ans: number[] = [];
    const vis: boolean[][] = Array.from({ length: n }, () => Array(m).fill(false));
    const dirs = [[0, 1, 0, -1, 0]];
    for (let h = 0, w = 1, d = 0, j = 0, k = 0; h > 0; --h) {
        ans.push(matrix[i][j]);
        vis[i][j] = true;
        const x = i + dirs[k];
        const y = j + dirs[k + 1];
        if (x < 0 || x == n || y < 0 || y == m || vis[x][y]) {
            k = (k + 1) % 4;
        }
        i += dirs[k];
        j += dirs[k + 1];
    }
    return ans;
}

TypeScript
function spiralOrder(matrix: number[][]): number[] {
    const n = matrix.length;
    const m = matrix[0].length;
    const ans: number[] = [];
    const vis = new Array(n).fill(0).map(() => new Array(m).fill(false));
    const dirs = [[0, 1, 0, -1, 0]];
    for (let h = 0, w = 1, d = 0, j = 0, k = 0; h > 0; --h) {
        ans.push(matrix[i][j]);
        vis[i][j] = true;
        const x = i + dirs[k];
        const y = j + dirs[k + 1];
        if (x < 0 || x == n || y < 0 || y == m || vis[x][y]) {
            k = (k + 1) % 4;
        }
        i += dirs[k];
        j += dirs[k + 1];
    }
    return ans;
}

Go
Go
```

Matrix · LeetCode · By Pattern — 1991 — LeetCode

```
func spiralOrder(matrix [][]int) (ans []int) {
    m, n := len(matrix), len(matrix[0])
    vis := make([]bool, m)
    for i := range vis {
        vis[i] = make([]bool, n)
    }
    dirs := [][]int{0, 1, 0, -1, 0}
    i, j, k := 0, 0, 0
    for h := m; h > 0; h-- {
        ans = append(ans, matrix[i][j])
        vis[i][j] = true
        x, y := i+dirs[k], j+dirs[k+1]
        if x < 0 || x == m || y < 0 || y == n || vis[x][y] {
            k = (k + 1) % 4
        }
        i, j = i+dirs[k], j+dirs[k+1]
    }
    return
}

Go
func spiralOrder(matrix [][]int) (ans []int) {
    m, n := len(matrix), len(matrix[0])
    vis := make([]bool, m)
    for i := range vis {
        vis[i] = make([]bool, n)
    }
    dirs := [][]int{0, 1, 0, -1, 0}
    i, j, k := 0, 0, 0
    for h := m; h > 0; h-- {
        ans = append(ans, matrix[i][j])
        vis[i][j] = true
        x, y := i+dirs[k], j+dirs[k+1]
        if x < 0 || x == m || y < 0 || y == n || vis[x][y] {
            k = (k + 1) % 4
        }
        i, j = i+dirs[k], j+dirs[k+1]
    }
    return
}

Rust
Rust
```

Matrix · LeetCode · By Pattern — 1992 — LeetCode

```
impl Solution {
    pub fn spiral_order(matrix: Vec<Vec<i32>>) -> Vec<i32> {
        let mut x1 = 0;
        let mut y1 = 0;
        let mut x2 = matrix.len() - 1;
        let mut y2 = matrix[0].len() - 1;
        let mut result = vec![];
        while x1 <= x2 && y1 <= y2 {
            for j in x1..=x2 {
                result.push(matrix[x1][j]);
            }
            for i in x1 + 1..=x2 {
                result.push(matrix[i][y2]);
            }
            if x1 < x2 && y1 < y2 {
                for j in y1..=y2 {
                    result.push(matrix[x2][j]);
                }
                for i in x1 + 1..=x2 {
                    result.push(matrix[i][y1]);
                }
            }
            x1 += 1;
            y1 += 1;
        }
        if x2 <= 0 {
            x2 += 1;
        }
        if y2 <= 0 {
            y2 += 1;
        }
        return result;
    }
}

[*] Matrix — Pattern Solution (stored optimal)
Python
```

Matrix · LeetCode · By Pattern — 1993 — LeetCode

```
def spiralOrder(matrix):
    # Initialize the boundaries of the matrix
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    result = [] # This will store the spiral order

    # Loop until the pointers cross each other
    while top <= bottom and left <= right:
        # Traverse from left to right on the current top row
        for col in range(left, right + 1):
            result.append(matrix[top][col])
        top += 1 # Move the top boundary down

        # Traverse upwards on the current right column
        for row in range(top, bottom + 1):
            result.append(matrix[row][right])
        right -= 1 # Move the right boundary to the left

        # Traverse from right to left on the current bottom row, if valid
        if top <= bottom:
            for col in range(right, left - 1, -1):
                result.append(matrix[bottom][col])
            bottom -= 1 # Move the bottom boundary up

        # Traverse upwards on the current left column, if valid
        if left <= right:
            for row in range(bottom, top - 1, -1):
                result.append(matrix[row][left])
            left += 1 # Move the left boundary right

    return result

# Example usage:
# print(spiralOrder([[1,2,3],[4,5,6],[7,8,9]]))

C++
```

Matrix · LeetCode · By Pattern — 1994 — LeetCode

```
#include <vector>
using namespace std;

class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        vector<int> result;
        if(matrix.empty()) return result;
        int top = 0, bottom = matrix.size() - 1;
        int left = 0, right = matrix[0].size() - 1;

        // Loop until all boundaries meet or cross
        while(top <= bottom && left <= right) {
            // Traverse from left to right on the top row
            for (int col = left; col <= right; col++) {
                result.push_back(matrix[top][col]);
            }
            top++; // Move the top boundary down

            // Traverse downwards on the right column
            for (int row = top; row <= bottom; row++) {
                result.push_back(matrix[row][right]);
            }
            right--; // Move the right boundary left

            // Traverse from right to left on the bottom row, if still valid
            if (top <= bottom) {
                for (int col = right; col >= left; col--) {
                    result.push_back(matrix[bottom][col]);
                }
                bottom--; // Move the bottom boundary up

            // Traverse from bottom to top on the left column, if still valid
            if (left <= right) {
                for (int row = bottom; row >= top; row--) {
                    result.push_back(matrix[row][left]);
                }
                left++; // Move the left boundary right
            }
        }
        return result;
    }
};

// Example usage:
// int main() {
//     vector<vector<int>> matrix = {{1,2,3}, {4,5,6}, {7,8,9}};
//     Solution sol;
//     vector<int> result = sol.spiralOrder(matrix);
//     return 0;
// }
```

#59 MEDIUM Matrix · LeetCode · By Pattern — 1995 — LeetCode

Spiral Matrix II
LeetCode · SimplifyLeet · WalkCCC · LeetCode · Editorial
Array · Matrix · Simulation
LeetCode Signal

Problem:
Given a positive integer n , generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order.

Example 1:

1	→ 2	→ 3
8	→ 9	↓ 4
↑ 7	← 6	← 5

Input: $n = 3$
Output: $[[1,2,3],[8,9,4],[7,6,5]]$

Example 2:

Input: $n = 1$
Output: $[[1]]$

Constraints:

- $1 \leq n \leq 20$

>> Brute Force [Matrix] Interview Prep
Python

Matrix · LeetCode · By Pattern — 1996 — LeetCode

```
# Brute Force Approach
class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        # Brute Force O(n^2 * 4) = Flood fill from every cell
        m, b = limits, limits[0]
        result = []
        for r in range(m):
            for c in range(m):
                visited = set()
                def floodfill(r, c):
                    if (r, c) in visited or not (0 <= r < m and 0 <= c < m): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: floodfill(r+dr, c+dc)
                floodfill(r, c)
                result.append(result, len(visited))
            return result
```

```
Python
# Solution to generate spiral matrix of size n x n
def generateMatrix(n):
    # Initialize ans = a matrix with zeros
    matrix = [[0] * n for _ in range(n)]
    # Initialize the starting number to fill
    num = 1
    # Set the initial boundaries for rows and columns
    top, bottom = 0, n - 1
    left, right = 0, n - 1
    # Continue the loop until the boundaries are valid
    while top <= bottom and left <= right:
        # Traverse from left to right along the top row
        for j in range(left, right + 1):
            matrix[top][j] = num
            num += 1
        top += 1 # Move the top boundary down
        # Traverse from top to bottom along the right column
        for i in range(top, bottom + 1):
            matrix[i][right] = num
            num += 1
        right -= 1 # Move the right boundary left
        # Traverse from right to left along the bottom row if still in valid bounds
```

```
if top == bottom:
    for j in range(right, left - 1, -1):
        matrix[bottom][j] = num
        num += 1
    bottom -= 1 # Move the bottom boundary up
    # Traverse from bottom to top along the left column if still in valid bounds
    if left == right:
        for i in range(bottom, top - 1, -1):
            matrix[i][left] = num
            num += 1
        left += 1 # Move the left boundary right
    return matrix
# Example usage:
print(generateMatrix(10))
```

```
Python
class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        count = 1
        for m in range(n // 2):
            m += m + 1
            for i in range(m, n):
                ans[i][m] = count
                count += 1
            for i in range(m, n-m-1):
                ans[m][i] = count
                count += 1
            for i in range(m, n-m-1):
                ans[i][m] = count
                count += 1
            if n % 2 == 1:
                ans[n // 2][n // 2] = count
        return ans
```

```
Python
class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        dirs = [(0, 1), (0, -1), (1, 0), (-1, 0)]
        i = j = k = 0
        for x in range(1, n + n + 1):
            ans[i][j] = x
            x, y = i + dirs[k][0], j + dirs[k][1]
            if x < 0 or x >= n or y < 0 or y >= n or ans[i][y]:
                k = (k + 1) % 4
                i, j = i + dirs[k][0], j + dirs[k][1]
            else:
                i, j = x, y
        return ans
```

```
Python
class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        dirs = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        i = j = k = 0
        for x in range(1, n + n + 1):
            ans[i][j] = x
            x, y = i + dirs[k][0], j + dirs[k][1]
            if x < 0 or x >= n or y < 0 or y >= n or ans[i][y]:
                k = (k + 1) % 4
                i, j = i + dirs[k][0], j + dirs[k][1]
            else:
                i, j = x, y
        return ans
```

```
C++
class Solution {
public:
    vector<vector<int>> generateMatrix(int n) {
        vector<vector<int>> ans(n, vector<int>(n));
        int count = 1;
        for (int m = 0; m < n / 2; ++m) {
            count += m + 1;
            for (int i = m; i < n; ++i) {
                ans[i][m] = count++;
            }
            for (int i = m; i < n; ++i) {
                ans[m][i] = count++;
            }
            for (int i = m; i < n; ++i) {
                ans[i][n-1-i] = count++;
            }
            for (int i = m; i < n; ++i) {
                ans[n-1-i][m] = count++;
            }
        }
        if (n % 2 == 1) {
            ans[n / 2][n / 2] = count;
        }
        return ans;
    }
};
```

```
class Solution {
public:
    vector<vector<int>> generateMatrix(int n) {
        vector<vector<int>> ans(n, vector<int>(n, 0));
        const int dirs[] = {0, 1, 0, -1, -1, 0};
        int i = 0, j = 0, k = 0;
        for (int v = 1; v <= n * n; ++v) {
            ans[i][j] = v;
            int x = i + dirs[k], y = j + dirs[k + 1];
            if (x < 0 || x >= n || y < 0 || y >= n || ans[x][y] != 0) {
                k = (k + 1) % 4;
            }
            i += dirs[k];
            j += dirs[k + 1];
        }
        return ans;
    }
};
```

```
C++
class Solution {
public:
    const int dirs[4][2] = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
    vector<vector<int>> generateMatrix(int n) {
        vector<vector<int>> ans(n, vector<int>(n));
        int i = 0, j = 0, k = 0;
        for (int v = 1; v <= n * n; ++v) {
            ans[i][j] = v;
            int x = i + dirs[k][0], y = j + dirs[k][1];
            if (x < 0 || x >= n || y < 0 || y >= n || ans[x][y] != 0) {
                k = (k + 1) % 4;
                x = i + dirs[k][0], y = j + dirs[k][1];
            }
            i = x, j = y;
        }
        return ans;
    }
};
```

Java

Java

```
class Solution {
    public int[][] generateMatrix(int n) {
        int[][] ans = new int[n][n];
        int count = 1;
        for (int m = 0; m < n / 2; ++m) {
            final int m1 = m + 1;
            for (int i = m; i < n; ++i) {
                ans[i][m] = count++;
            }
            for (int i = m; i < n; ++i) {
                ans[m][i] = count++;
            }
            for (int i = m; i < n; ++i) {
                ans[i][n-1-i] = count++;
            }
            for (int i = m; i < n; ++i) {
                ans[n-1-i][m] = count++;
            }
        }
        if (n % 2 == 1) {
            ans[n / 2][n / 2] = count;
        }
        return ans;
    }
}
```

```
Java
class Solution {
    public int[][] generateMatrix(int n) {
        int[][] ans = new int[n][n];
        final int[] dirs = {0, 1, 0, -1, -1, 0};
        int i = 0, j = 0, k = 0;
        for (int v = 1; v <= n * n; ++v) {
            ans[i][j] = v;
            int x = i + dirs[k], y = j + dirs[k + 1];
            if (x < 0 || x >= n || y < 0 || y >= n || ans[x][y] != 0) {
                k = (k + 1) % 4;
                x = i + dirs[k], y = j + dirs[k + 1];
            }
            i = x, j = y;
        }
        return ans;
    }
}
```

Java

```
class Solution {
    public int[][] generateMatrix(int n) {
        int[][] ans = new int[n][n];
        int i = 0, j = 0, k = 0;
        int[] dirs = {0, 1, 0, -1, -1, 0};
        for (int v = 1; v <= n * n; ++v) {
            ans[i][j] = v;
            int x = i + dirs[k], y = j + dirs[k + 1];
            if (x < 0 || x >= n || y < 0 || y >= n || ans[x][y] != 0) {
                k = (k + 1) % 4;
                x = i + dirs[k], y = j + dirs[k + 1];
            }
            i = x, j = y;
        }
        return ans;
    }
}
```

```
JavaScript
var generateMatrix = function(n) {
    const ans = Array.from({length: n}, () => Array(n).fill(0));
    let i = 0, j = 0, k = 0;
    let [x, y] = [0, 0];
    for (let v = 1; v <= n * n; ++v) {
        ans[i][j] = v;
        const [x, y] = [i + dirs[k], j + dirs[k + 1]];
        if (x < 0 || x >= n || y < 0 || y >= n || ans[x][y] != 0) {
            k = (k + 1) % 4;
            [x, y] = [i + dirs[k], j + dirs[k + 1]];
        }
        i = x, j = y;
    }
    return ans;
};
```

JavaScript

```

//
+ Generate (number) n
+ Return (number[i][j])
+
var generateMatrix = function (n) {
    const ans = new Array(n).fill(0).map(() => new Array(n).fill(0));
    let [i, j, k] = [0, 0, 0];
    const dirs = [
        [0, 1],
        [1, 0],
        [0, -1],
        [-1, 0],
    ];
    for (let v = 1; v <= n * n; ++v) {
        ans[i][j] = v;
        let [x, y] = [i + dirs[k][0], j + dirs[k][1]];
        if (x < 0 || y < 0 || x == n || y == n || ans[x][y] > 0) {
            k = (k + 1) % 4;
            [x, y] = [i + dirs[k][0], j + dirs[k][1]];
        }
        [i, j] = [x, y];
    }
    return ans;
};

```

TypeScript

```

TypeScript
function generateMatrix(n: number): number[][] {
    const ans: number[][] = Array.from({ length: n }, () => Array(n).fill(0));
    const dirs = [[0, 1], [1, 0], [-1, 0], [0, -1]];
    let [i, j, k] = [0, 0, 0];
    for (let v = 1; v <= n * n; ++v) {
        ans[i][j] = v;
        const [x, y] = [i + dirs[k][0], j + dirs[k][1]];
        if (x < 0 || y < 0 || x == n || y == n || ans[x][y] != 0) {
            k = (k + 1) % 4;
            [x, y] = [i + dirs[k][0], j + dirs[k][1]];
        }
        i += dirs[k][0];
        j += dirs[k][1];
    }
    return ans;
}

```

TypeScript

```

function generateMatrix(n: number): number[][] {
    let ans = Array.from({ length: n }, () => new Array(n));
    let dir = [
        [0, 1],
        [1, 0],
        [0, -1],
        [-1, 0],
    ];
    let i = 0,
        j = 0;
    for (let cnt = 1, k = 0; cnt <= n * n; cnt++) {
        ans[i][j] = cnt;
        let x = i + dir[k][0],
            y = j + dir[k][1];
        if (x < 0 || x == n || y < 0 || y == n || ans[x][y]) {
            k = (k + 1) % 4;
            [x, y] = [i + dir[k][0], j + dir[k][1]];
        }
        [i, j] = [x, y];
    }
    return ans;
}

```

Go

```

Go
func generateMatrix(n int) [][]int {
    ans := make([][]int, 0)
    for i := range ans {
        ans[i] = make([]int, n)
    }
    i, j, k := 0, 0, 0
    for v := 1; v <= ans; v++ {
        ans[i][j] = v
        x, y := i+dirs[k], j+dirs[k+1]
        if x < 0 || x == n || y < 0 || y == n || ans[x][y] != 0 {
            k = (k + 1) % 4
        }
        i += dirs[k][0]
        j += dirs[k][1]
    }
    return ans
}

```

Go

```

func generateMatrix(n int) [][]int {
    ans := make([][]int, 0)
    for i := range ans {
        ans[i] = make([]int, n)
    }
    dirs := [][]int{{0, 1}, {1, 0}, {0, -1}, {-1, 0}}
    var i, j, k int
    for v := 1; v <= ans; v++ {
        ans[i][j] = v
        x, y := i+dirs[k][0], j+dirs[k][1]
        if x < 0 || y < 0 || x == n || y == n || ans[x][y] > 0 {
            k = (k + 1) % 4
            x, y = i+dirs[k][0], j+dirs[k][1]
        }
        i, j = x, y
    }
    return ans
}

```

Rust

```

Rust
impl Solution {
    pub fn generate_matrix(n: i32) -> Vec<Vec<i32>> {
        let mut ans = vec![vec![0; n as usize]; n as usize];
        let dirs = [[0, 1], [1, 0], [0, -1], [-1, 0]];
        let (mut i, mut j, mut k) = (0, 0, 0);
        for v in 1..=n as i32 {
            ans[i as usize][j as usize] = v;
            let (x, y) = (i + dirs[k][0], j + dirs[k][1]);
            if x < 0 || x == n || y < 0 || y == n || ans[x as usize][y as usize] != 0 {
                k = (k + 1) % 4;
            }
            i += dirs[k][0];
            j += dirs[k][1];
        }
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn generate_matrix(n: i32) -> Vec<Vec<i32>> {
        let mut ans = vec![vec![0; n as usize]; n as usize];
        let mut row = vec![vec![0; n]; n];
        let mut num = 1;
        for i in 0..n / 2 {
            for j in 0..n / 2 {
                num += 1;
            }
            for j in 0..n - 1 - i {
                row[i][n - 1 - i] = num;
                num += 1;
            }
            for j in 0..n - 1 - i {
                row[i + 1][n - j - 1] = num;
                num += 1;
            }
            for j in 0..n - 1 - i {
                row[n - j - 1][i] = num;
                num += 1;
            }
            for j in 0..n - 1 - i {
                row[n - j - 1][n - i] = num;
                num += 1;
            }
        }
        if n % 2 == 1 {
            row[n / 2][n / 2] = num;
        }
    }
}

```

[*] Matrix — Pattern Solution (stored optimal)

Python

```

# Function to generate spiral matrix of size n x n
def generateMatrix(n):
    # Initialize an n x n matrix with zeros
    matrix = [[0] * n for _ in range(n)]
    # Initialize the starting number to fill
    num = 1
    # Set the initial boundaries for rows and columns
    top, bottom = 0, n - 1
    left, right = 0, n - 1

    # Continue the loop until the boundaries are valid
    while top <= bottom and left <= right:
        # Traverse from left to right along the top row
        for j in range(left, right + 1):
            matrix[top][j] = num
            num += 1
        top += 1 # Move the top boundary down

        # Traverse from top to bottom along the right column
        for i in range(top, bottom + 1):
            matrix[i][right] = num
            num += 1
        right -= 1 # Move the right boundary left

        # Traverse from right to left along the bottom row if still in valid bounds
        if top <= bottom:
            for j in range(right, left - 1, -1):
                matrix[bottom][j] = num
                num += 1
            bottom -= 1 # Move the bottom boundary up

        # Traverse from bottom to top along the left column if still in valid bounds
        if left <= right:
            for i in range(bottom, top - 1, -1):
                matrix[i][left] = num
                num += 1
            left += 1 # Move the left boundary right

    return matrix

# Example usage:
print(generateMatrix(5))

```

C++

```

// Function to generate spiral matrix of size n x n
// direction vector
using namespace std;

vector<vector<int>> generateMatrix(int n) {
    // Initialize an n x n matrix filled with zeros
    vector<vector<int>> matrix(n, vector<int>(n, 0));
    int num = 1; // Start filling with 1
    int top = 0, bottom = n - 1;
    int left = 0, right = n - 1;

    // Continue until the matrix is filled the entire matrix
    while (top <= bottom && left <= right) {
        // Traverse from left to right along the top row
        for (int i = left; i <= right; i++) {
            matrix[top][i] = num++;
        }
        top++; // Move the top boundary down

        // Traverse from top to bottom along the right column
        for (int i = top; i <= bottom; i++) {
            matrix[i][right] = num++;
        }
        right--; // Move the right boundary left

        // Traverse from right to left along the bottom row if valid
        if (top <= bottom) {
            for (int i = right; i >= left; i--) {
                matrix[bottom][i] = num++;
            }
            bottom--; // Move the bottom boundary up

        // Traverse from bottom to top along the left column if valid
        if (left <= right) {
            for (int i = bottom; i >= top; i--) {
                matrix[i][left] = num++;
            }
            left++; // Move the left boundary right
        }
    }

    return matrix;
}

// Example usage:
// int n = 5;
// vector<vector<int>> result = generateMatrix(n);
// return result;
// }

```


Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:

1	3	1
1	5	1
4	2	1

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Explanation: Because the path 1 -> 3 -> 1 -> 1 -> 1 minimizes the sum.

Example 2:

Input: grid = [[1,2,3],[4,5,6]]

Output: 12

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[0].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{grid}[i][j] \leq 200$

>> Brute Force (Matrix Interview Prep)

Python

```
# Brute Force Approach
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        # Brute Force (O(m*n)^2) - Flood fill from every cell
        m, n = len(grid), len(grid[0])
        result = 0
        for i in range(m):
            for j in range(n):
                visited = set()
                def flood(r, c):
                    if (r, c) in visited or not (0 <= r < m and 0 <= c < n): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: flood(r+dr, c+dc)
                flood(i, j)
                result = min(result, len(visited))
        return result
```

Python

```
# Python Solution using dynamic programming
def minPathSum(grid):
    # Number of rows and columns in the grid
    rows = len(grid)
    cols = len(grid[0])

    # Initialize the DP table directly in grid to save space
    # Fill the first row by accumulating the sum from the left
    for j in range(1, cols):
        grid[0][j] += grid[0][j-1]

    # Fill the first column by accumulating the sum above
    for i in range(1, rows):
        grid[i][0] += grid[i-1][0]

    # Compute the minimum path sum for the rest of the cells
    for i in range(1, rows):
        for j in range(1, cols):
            # The cell value is added to the minimum of the top or left cell
            grid[i][j] += min(grid[i-1][j], grid[i][j-1])

    # The bottom-right cell contains the answer
    return grid[rows-1][cols-1]

# Example usage
example_grid = [[1,3,1],[1,5,1],[4,2,1]]
print(minPathSum(example_grid)) # Output should be 7
```

Python

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid)
        n = len(grid[0])

        for i in range(m):
            for j in range(n):
                if i > 0 and j > 0:
                    grid[i][j] = min(grid[i-1][j], grid[i][j-1]) + grid[i][j]
                elif i > 0:
                    grid[i][j] = grid[i-1][j] + grid[i][j]
                elif j > 0:
                    grid[i][j] = grid[i][j-1] + grid[i][j]
                else:
                    continue
        return grid[m-1][n-1]
```

Python

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]

        for i in range(1, m):
            f[i][0] = f[i-1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j-1] + grid[0][j]

        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j]
        return f[m-1][n-1]
```

Python

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]

        for i in range(1, m):
            f[i][0] = f[i-1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j-1] + grid[0][j]

        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j]
        return f[m-1][n-1]
```

C++

C++

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        const int m = grid.size();
        const int n = grid[0].size();

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i > 0 && j > 0)
                    grid[i][j] = min(grid[i-1][j], grid[i][j-1]) + grid[i][j];
                else if (i > 0)
                    grid[i][j] = grid[i-1][j] + grid[i][j];
                else if (j > 0)
                    grid[i][j] = grid[i][j-1] + grid[i][j];
                else
                    continue;

        return grid[m-1][n-1];
    }
};
```

C++

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int m = grid.size(), n = grid[0].size();
        int f[m][n];
        f[0][0] = grid[0][0];
        for (int i = 1; i < m; ++i)
            f[i][0] = f[i-1][0] + grid[i][0];
        for (int j = 1; j < n; ++j)
            f[0][j] = f[0][j-1] + grid[0][j];
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j];
        return f[m-1][n-1];
    }
};
```

C++

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int m = grid.size(), n = grid[0].size();
        int f[m][n];
        f[0][0] = grid[0][0];
        for (int i = 1; i < m; ++i)
            f[i][0] = f[i-1][0] + grid[i][0];
        for (int j = 1; j < n; ++j)
            f[0][j] = f[0][j-1] + grid[0][j];
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j];
        return f[m-1][n-1];
    }
};
```

Java

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        final int m = grid.length;
        final int n = grid[0].length;

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (i > 0 && j > 0)
                    grid[i][j] = Math.min(grid[i-1][j], grid[i][j-1]) + grid[i][j];
                else if (i > 0)
                    grid[i][j] = grid[i-1][j] + grid[i][j];
                else if (j > 0)
                    grid[i][j] = grid[i][j-1] + grid[i][j];
                else
                    continue;

        return grid[m-1][n-1];
    }
}
```

Java

```
class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int m = grid.length, n = grid[0].length;
        int f[m][n];
        f[0][0] = grid[0][0];
        for (int i = 1; i < m; ++i)
            f[i][0] = f[i-1][0] + grid[i][0];
        for (int j = 1; j < n; ++j)
            f[0][j] = f[0][j-1] + grid[0][j];
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[i][j] = Math.min(f[i-1][j], f[i][j-1]) + grid[i][j];
        return f[m-1][n-1];
    }
};
```

Java

```
public class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int f = new int[m][n];
        f[0][0] = grid[0][0];
        for (int i = 1; i < m; ++i)
            f[i][0] = f[i-1][0] + grid[i][0];
        for (int j = 1; j < n; ++j)
            f[0][j] = f[0][j-1] + grid[0][j];
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[i][j] = Math.min(f[i-1][j], f[i][j-1]) + grid[i][j];
        return f[m-1][n-1];
    }
}
```

Java

```

class Solution {
public: int minPathSum(vector<vector<int>>& grid) {
    int m = grid.length, n = grid[0].length;
    int[][] f = new int[m][n];
    f[0][0] = grid[0][0];
    for (int i = 1; i < m; ++i) {
        f[i][0] = f[i-1][0] + grid[i][0];
    }
    for (int j = 1; j < n; ++j) {
        f[0][j] = f[0][j-1] + grid[0][j];
    }
    for (int i = 1; i < m; ++i) {
        for (int j = 1; j < n; ++j) {
            f[i][j] = Math.min(f[i-1][j], f[i][j-1]) + grid[i][j];
        }
    }
    return f[m-1][n-1];
}
}

```

```

java
public class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int[] f = new int[m, n];
        f[0, 0] = grid[0][0];
        for (int i = 1; i < m; ++i) {
            f[i, 0] = f[i-1, 0] + grid[i][0];
        }
        for (int j = 1; j < n; ++j) {
            f[0, j] = f[0, j-1] + grid[0][j];
        }
        for (int i = 1; i < m; ++i) {
            for (int j = 1; j < n; ++j) {
                f[i, j] = Math.min(f[i-1, j], f[i, j-1]) + grid[i][j];
            }
        }
        return f[m-1, n-1];
    }
}

```

JavaScript
JavaScript

Matrix · LeetCode - By Pattern

— 1615 —

LeetCode

```

// @param {number[][]} grid
// @return {number}
var minPathSum = function(grid) {
    const m = grid.length;
    const n = grid[0].length;
    const f = Array(m).fill(0);
    f[0][0] = grid[0][0];
    for (let i = 1; i < m; ++i) {
        f[i][0] = f[i-1][0] + grid[i][0];
    }
    for (let j = 1; j < n; ++j) {
        f[0][j] = f[0][j-1] + grid[0][j];
    }
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            f[i][j] = Math.min(f[i-1][j], f[i][j-1]) + grid[i][j];
        }
    }
    return f[m-1][n-1];
};

```

TypeScript

```

TypeScript
function minPathSum(grid: number[][]): number {
    const m = grid.length;
    const n = grid[0].length;
    const f: number[][] = Array(m).fill(0);
    f[0][0] = grid[0][0];
    for (let i = 1; i < m; ++i) {
        f[i][0] = f[i-1][0] + grid[i][0];
    }
    for (let j = 1; j < n; ++j) {
        f[0][j] = f[0][j-1] + grid[0][j];
    }
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            f[i][j] = Math.min(f[i-1][j], f[i][j-1]) + grid[i][j];
        }
    }
    return f[m-1][n-1];
}

```

Go

Go

Matrix · LeetCode - By Pattern

— 1616 —

LeetCode

```

func minPathSum(grid [][]int) int {
    m, n := len(grid), len(grid[0])
    f := make([][]int, m)
    for i := range f {
        f[i] = make([]int, n)
    }
    f[0][0] = grid[0][0]
    for i := 1; i < m; i++ {
        f[i][0] = f[i-1][0] + grid[i][0]
    }
    for j := 1; j < n; j++ {
        f[0][j] = f[0][j-1] + grid[0][j]
    }
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j]
        }
    }
    return f[m-1][n-1]
}

```

Go

```

func minPathSum(grid [][]int) int {
    m, n := len(grid), len(grid[0])
    f := make([][]int, m)
    for i := range f {
        f[i] = make([]int, n)
    }
    f[0][0] = grid[0][0]
    for i := 1; i < m; i++ {
        f[i][0] = f[i-1][0] + grid[i][0]
    }
    for j := 1; j < n; j++ {
        f[0][j] = f[0][j-1] + grid[0][j]
    }
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            f[i][j] = min(f[i-1][j], f[i][j-1]) + grid[i][j]
        }
    }
    return f[m-1][n-1]
}

```

Rust

Rust

Matrix · LeetCode - By Pattern

— 1617 —

LeetCode

```

impl Solution {
    pub fn min_path_sum(grid: Vec<Vec<i32>>)) -> i32 {
        let m = grid.len();
        let n = grid[0].len();
        for i in 0..m {
            grid[i][0] += grid[i-1][0];
        }
        for i in 1..m {
            grid[0][i] += grid[0][i-1];
        }
        for i in 1..m {
            for j in 1..n {
                grid[i][j] += grid[i-1][j].min(grid[i][j-1]);
            }
        }
        grid[m-1][n-1]
    }
}

```

Rust

```

impl Solution {
    pub fn min_path_sum(grid: Vec<Vec<i32>>)) -> i32 {
        let m = grid.len();
        let n = grid[0].len();
        for i in 0..m {
            grid[i][0] += grid[i-1][0];
        }
        for i in 1..m {
            for j in 1..n {
                grid[i][j] += grid[i-1][j].min(grid[i][j-1]);
            }
        }
        grid[m-1][n-1]
    }
}

```

[*] Matrix — Pattern Solution (stored optimal)

Python

Matrix · LeetCode - By Pattern

— 1618 —

LeetCode

```

# Python solution using dynamic programming
def minPathSum(grid):
    # Number of rows and columns in the grid
    rows = len(grid)
    cols = len(grid[0])

    # Initialize the dp table directly in grid to save space
    # Fill the first row by accumulating the sum from the left
    for j in range(1, cols):
        grid[0][j] += grid[0][j-1]

    # Fill the first column by accumulating the sum above
    for i in range(1, rows):
        grid[i][0] += grid[i-1][0]

    # Compute the minimum path sum for the rest of the cells
    for i in range(1, rows):
        for j in range(1, cols):
            # The min value is either to the min of the top or left cell
            grid[i][j] += min(grid[i-1][j], grid[i][j-1])

    # The bottom-right cell contains the answer
    return grid[rows-1][cols-1]

# Example usage
example_grid = [[1,3,1],[1,5,1],[4,2,1]]
print(minPathSum(example_grid)) # Should be 7

```

C++

```

// C++ solution using dynamic programming
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int minPathSum(vector<vector<int>>& grid) {
    int rows = grid.size();
    int cols = grid[0].size();

    // Fill the first row by accumulating the sum from left
    for (int j = 1; j < cols; ++j) {
        grid[0][j] += grid[0][j-1];
    }

    // Fill the first column by accumulating the sum from above
    for (int i = 1; i < rows; ++i) {
        grid[i][0] += grid[i-1][0];
    }

    // Fill the rest of the grid with the minimum path sum for each cell
    for (int i = 1; i < rows; ++i) {
        for (int j = 1; j < cols; ++j) {
            grid[i][j] += min(grid[i-1][j], grid[i][j-1]);
        }
    }
}

```

Matrix · LeetCode - By Pattern

— 1619 —

LeetCode

```

}

// The bottom-right cell contains the answer
return grid[rows-1][cols-1];
}

int main() {
    // Test example
    vector<vector<int>> grid = {{1,3,1}, {1,5,1}, {4,2,1}};
    cout << minPathSum(grid) << endl; // Should be 7
    return 0;
}

```

#73

MEDIUM

Set Matrix Zeroes

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Hash Table · Matrix

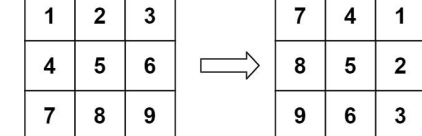
Like Grid 169

Problem:

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it in place.

Example 1:



Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
Output: `[[7,4,1],[8,5,2],[9,6,3]]`

Example 2:

Matrix · LeetCode - By Pattern

— 1620 —

LeetCode

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16

→

15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]

Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

Constraints:

- m == matrix.length
- n == matrix[0].length
- 1 <= m, n <= 200
- 231 <= matrix[i][j] <= 231 - 1

Follow up:

- A straightforward solution using O(mn) space is probably a bad idea.
- A simple improvement uses O(m + n) space, but still not the best solution.
- Could you devise a constant space solution?

>> Brute Force (Matrix Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def setZeroes(self, matrix):
        # Brute Force Way: (row) - record zero positions, then zero out rows/cols
        zeros = []
        for r in range(len(matrix)):
            if matrix[r][0] == 0:
                for c in range(len(matrix[0])):
                    matrix[r][c] = 0
        for row in range(len(matrix)):
            if matrix[row][0] == 0:
                for c in range(len(matrix[0])):
                    matrix[row][c] = 0
```

Python

Python

Matrix · LeetCode · By Pattern

— 1621 —

LeetCode

Python solution for the Set Matrix Zeroes problem

```
def setZeroes(matrix):
    # Step 1: Determine the dimensions of matrix
    m = len(matrix)
    n = len(matrix[0])

    # Determine if first row or first column should be zeroed
    first_row_zero = any(matrix[0][j] == 0 for j in range(n))
    first_col_zero = any(matrix[i][0] == 0 for i in range(m))

    # Step 2: Use first row and column as markers
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][j] == 0:
                matrix[i][0] = 0
                matrix[0][j] = 0

    # Step 3: Zero out cells based on the markers in first row and column
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][0] == 0 or matrix[0][j] == 0:
                matrix[i][j] = 0

    # Step 4: Zero out the first row if needed
    if first_row_zero:
        for j in range(n):
            matrix[0][j] = 0

    # Step 5: Zero out the first column if needed
    if first_col_zero:
        for i in range(m):
            matrix[i][0] = 0

    # Example usage:
    # matrix = [[1,1,1],[1,0,1],[1,1,1]]
    # setZeroes(matrix)
    # print(matrix)
```

Python

Matrix · LeetCode · By Pattern

— 1622 —

LeetCode

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m = len(matrix)
        n = len(matrix[0])
        shouldFillFirstRow = 0 in matrix[0]
        shouldFillFirstCol = 0 in list(zip(*matrix))[0]

        # Store the information in the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        # Fill 0s for the matrix except the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

        # Fill 0s for the first row if needed.
        if shouldFillFirstRow:
            matrix[0] = [0] * n

        # Fill 0s for the first column if needed.
```

Python

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        row = [False] * m
        col = [False] * n
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 0:
                    row[i] = col[j] = True
        for i in range(m):
            for j in range(n):
                if row[i] or col[j]:
                    matrix[i][j] = 0
```

Python

Matrix · LeetCode · By Pattern

— 1623 —

LeetCode

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        is_row = any(v == 0 for v in matrix[0])
        is_col = any(matrix[i][0] == 0 for i in range(m))
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = matrix[0][j] = 0
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0
        if is_row:
            for j in range(n):
                matrix[0][j] = 0
        if is_col:
            for i in range(m):
                matrix[i][0] = 0
```

C++

C++

```
class Solution {
public:
    void setZeroes(vector<vector<int>&& matrix) {
        int m = matrix.size(), n = matrix[0].size();
        vector<bool> row(m);
        vector<bool> col(n);
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == 0) {
                    row[i] = col[j] = true;
                }
            }
        }
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (row[i] || col[j]) {
                    matrix[i][j] = 0;
                }
            }
        }
    }
};
```

C++

Matrix · LeetCode · By Pattern

— 1624 —

LeetCode

```
class Solution {
public:
    void setZeroes(vector<vector<int>&& matrix) {
        int m = matrix.size(), n = matrix[0].size();
        bool isRow = false, isCol = false;
        for (int j = 0; j < n; ++j) {
            if (matrix[0][j] == 0) {
                isCol = true;
                break;
            }
        }
        for (int i = 0; i < m; ++i) {
            if (matrix[i][0] == 0) {
                isRow = true;
                break;
            }
        }
        for (int i = 1; i < m; ++i) {
            for (int j = 1; j < n; ++j) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }
        for (int i = 1; i < m; ++i) {
            for (int j = 1; j < n; ++j) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }
        if (isRow) {
            for (int j = 0; j < n; ++j) {
                matrix[0][j] = 0;
            }
        }
        if (isCol) {
            for (int i = 0; i < m; ++i) {
                matrix[i][0] = 0;
            }
        }
    }
};
```

Java

Java

Matrix · LeetCode · By Pattern

— 1625 —

LeetCode

```
class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        boolean[] row = new boolean[m];
        boolean[] col = new boolean[n];
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == 0) {
                    row[i] = col[j] = true;
                }
            }
        }
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (row[i] || col[j]) {
                    matrix[i][j] = 0;
                }
            }
        }
    }
}
```

Java

```
public class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        boolean[] row = new boolean[m], col = new boolean[n];
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == 0) {
                    row[i] = true;
                    col[j] = true;
                }
            }
        }
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (row[i] || col[j]) {
                    matrix[i][j] = 0;
                }
            }
        }
    }
}
```

Java

Matrix · LeetCode · By Pattern

— 1626 —

LeetCode

```

class Solution {
public: void setZeroes(vector<vector<int>>& matrix) {
    int n = matrix.length, m = matrix[0].length;
    bool row = false, col = false;
    for (int i = 0; i < n; ++i) {
        if (matrix[i][0] == 0) {
            col = true;
            break;
        }
    }
    for (int i = 0; i < m; ++i) {
        if (matrix[0][i] == 0) {
            row = true;
            break;
        }
    }
    for (int i = 1; i < n; ++i) {
        for (int j = 1; j < m; ++j) {
            if (matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }
    for (int i = 1; i < n; ++i) {
        for (int j = 1; j < m; ++j) {
            if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
        }
    }
    if (row) {
        for (int j = 0; j < m; ++j) {
            matrix[0][j] = 0;
        }
    }
    if (col) {
        for (int i = 0; i < n; ++i) {
            matrix[i][0] = 0;
        }
    }
    if (!row) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                matrix[i][j] = 0;
            }
        }
    }
}
}

```

Java

```

public class Solution {
    public void setZeroes(int[][] matrix) {
        int n = matrix.length, m = matrix[0].length;
        bool row = matrix[0].Contains(0), col = false;
        for (int i = 0; i < n; ++i) {
            if (matrix[i][0] == 0) {
                col = true;
                break;
            }
        }
        for (int i = 1; i < n; ++i) {
            for (int j = 1; j < m; ++j) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }
        for (int i = 1; i < n; ++i) {
            for (int j = 1; j < m; ++j) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }
        if (row) {
            for (int j = 0; j < m; ++j) {
                matrix[0][j] = 0;
            }
        }
        if (col) {
            for (int i = 0; i < n; ++i) {
                matrix[i][0] = 0;
            }
        }
    }
}

```

JavaScript

JavaScript

```

/**
 * Given a number[][] matrix
 * returns void[] Do not return anything, modify matrix in-place instead.
 */
var setZeroes = function(matrix) {
    const n = matrix.length;
    const row = Array(n).fill(false);
    const col = Array(n).fill(false);
    for (let i = 0; i < n; ++i) {
        for (let j = 0; j < n; ++j) {
            if (matrix[i][j] == 0) {
                row[i] = col[j] = true;
            }
        }
    }
    for (let i = 0; i < n; ++i) {
        for (let j = 0; j < n; ++j) {
            if (row[i] || col[j]) {
                matrix[i][j] = 0;
            }
        }
    }
};

```

JavaScript

```

/**
 * Given a number[][] matrix
 * returns void[] Do not return anything, modify matrix in-place instead.
 */
var setZeroes = function(matrix) {
    const n = matrix.length;
    const m = matrix[0].length;
    let row = matrix[0].some(v => v == 0);
    let col = false;
    for (let i = 0; i < n; ++i) {
        if (matrix[i][0] == 0) {
            col = true;
            break;
        }
    }
    for (let i = 1; i < n; ++i) {
        for (let j = 1; j < m; ++j) {
            if (matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }
    for (let i = 1; i < n; ++i) {
        for (let j = 1; j < m; ++j) {
            if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
        }
    }
    if (row) {
        for (let i = 0; i < n; ++i) {
            matrix[i][0] = 0;
        }
    }
    if (col) {
        for (let j = 0; j < m; ++j) {
            matrix[0][j] = 0;
        }
    }
}

```

```

        if (matrix[i][0] == 0 || matrix[0][j] == 0) {
            matrix[i][j] = 0;
        }
    }
    if (row) {
        for (let j = 0; j < m; ++j) {
            matrix[0][j] = 0;
        }
    }
    if (col) {
        for (let i = 0; i < n; ++i) {
            matrix[i][0] = 0;
        }
    }
}

```

TypeScript

TypeScript

```

/**
 * Do not return anything, modify matrix in-place instead.
 */
function setZeroes(matrix: number[][]): void {
    const n = matrix.length;
    const m = matrix[0].length;
    const row = Array(n).fill(false);
    const col = Array(m).fill(false);
    for (let i = 0; i < n; ++i) {
        for (let j = 0; j < m; ++j) {
            if (matrix[i][j] == 0) {
                row[i] = col[j] = true;
            }
        }
    }
    for (let i = 0; i < n; ++i) {
        for (let j = 0; j < m; ++j) {
            if (row[i] || col[j]) {
                matrix[i][j] = 0;
            }
        }
    }
}

```

TypeScript

```

/**
 * Do not return anything, modify matrix in-place instead.
 */
function setZeroes(matrix: number[][]): void {
    const n = matrix.length;
    const m = matrix[0].length;
    const row = matrix[0].includes(0);
    const col = matrix.some(r => r.includes(0));
    for (let i = 1; i < n; ++i) {
        for (let j = 1; j < m; ++j) {
            if (matrix[i][j] == 0) {
                matrix[i][0] = 0;
                matrix[0][j] = 0;
            }
        }
    }
    for (let i = 1; i < n; ++i) {
        for (let j = 1; j < m; ++j) {
            if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                matrix[i][j] = 0;
            }
        }
    }
    if (row) {
        for (let i = 0; i < n; ++i) {
            matrix[i][0] = 0;
        }
    }
    if (col) {
        for (let j = 0; j < m; ++j) {
            matrix[0][j] = 0;
        }
    }
}

```

Go

Go

```

func setZeroes(matrix [][]int) {
    row := make([]bool, len(matrix))
    col := make([]bool, len(matrix[0]))
    for i := range matrix {
        for j := range matrix[i] {
            if matrix[i][j] == 0 {
                row[i] = true
                col[j] = true
            }
        }
    }
    for i := range matrix {
        for j := range matrix[i] {
            if row[i] || col[j] {
                matrix[i][j] = 0
            }
        }
    }
}

```

Go

```

func setZeroes(matrix [][]int) {
    m, n := len(matrix), len(matrix[0])
    row, col := false, false
    for i := 0; i < m; i++ {
        if matrix[i][0] == 0 {
            row = true
            break
        }
    }
    for i := 0; i < n; i++ {
        for j := 0; j < m; j++ {
            if matrix[i][j] == 0 {
                matrix[i][0], matrix[0][j] = 0, 0
            }
        }
    }
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            if matrix[i][0] == 0 || matrix[0][j] == 0 {
                matrix[i][j] = 0
            }
        }
    }
    if row {
        for j := 0; j < n; j++ {
            matrix[0][j] = 0
        }
    }
    if col {
        for i := 0; i < m; i++ {
            matrix[i][0] = 0
        }
    }
}

```

Rust

Rust

```
1 def setZeroes(matrix: List[List[Int]]): Unit = {
2   let n = matrix.length
3   let m = matrix(0).length
4   let row = new Array[Boolean](n)
5   let col = new Array[Boolean](m)
6   for i in 0..n-1 {
7     for j in 0..m-1 {
8       if matrix[i][j] == 0 {
9         row[i] = true
10        col[j] = true
11      }
12    }
13  }
14  for i in 0..n-1 {
15    for j in 0..m-1 {
16      if row[i] || col[j] {
17        matrix[i][j] = 0
18      }
19    }
20  }
21 }
22
23 // Rust
24
25 impl Solution {
26     pub fn setZeroes(matrix: &mut Vec<Vec<i32>>()) {
27         let n = matrix.len();
28         let m = matrix[0].len();
29         let mut row = vec![false; n];
30         let mut col = vec![false; m];
31         for i in 0..n {
32             for j in 0..m {
33                 if matrix[i][j] == 0 {
34                     row[i] = true;
35                     col[j] = true;
36                 }
37             }
38         }
39         for i in 0..n {
40             for j in 0..m {
41                 if row[i] || col[j] {
42                     matrix[i][j] = 0;
43                 }
44             }
45         }
46     }
47 }
48
49 [*] Matrix — Pattern Solution (stored optimal)
50 Python
```

```
1 # Python solution for the Set Matrix Zeroes problem
2
3 def setZeroes(matrix):
4     # Step 1: Determine the dimensions of matrix
5     n = len(matrix)
6     m = len(matrix[0])
7
8     # Determine if first row or first column should be zeroed
9     first_row_zero = any(matrix[0][j] == 0 for j in range(m))
10    first_col_zero = any(matrix[i][0] == 0 for i in range(n))
11
12    # Step 2: Use first row and column as markers
13    for i in range(1, n):
14        for j in range(1, m):
15            if matrix[i][j] == 0:
16                matrix[i][0] = 0 # mark the first element of the row
17                matrix[0][j] = 0 # mark the first element of the column
18
19    # Step 3: Zero out cells based on the markers in first row and column
20    for i in range(1, n):
21        for j in range(1, m):
22            if matrix[i][0] == 0 or matrix[0][j] == 0:
23                matrix[i][j] = 0
24
25    # Step 4: Zero out the first row if needed
26    if first_row_zero:
27        for j in range(m):
28            matrix[0][j] = 0
29
30    # Step 5: Zero out the first column if needed
31    if first_col_zero:
32        for i in range(n):
33            matrix[i][0] = 0
34
35    # Example usage:
36    # matrix = [[1,1,1],[1,0,1],[1,1,1]]
37    # setZeroes(matrix)
38    # print(matrix)
```

```
1 // C++ solution for the Set Matrix Zeroes problem
2
3 #include <vector>
4 using namespace std;
5
6 class Solution {
7 public:
8     void setZeroes(vector<vector<int>>& matrix) {
9         int n = matrix.size();
10        int m = matrix[0].size();
11
12        // Check if first row or first column should be zeroed.
13        bool firstRowZero = false, firstColZero = false;
14        for (int j = 0; j < m; j++) {
15            if (matrix[0][j] == 0) {
16                firstRowZero = true;
17                break;
18            }
19        }
20        for (int i = 0; i < n; i++) {
21            if (matrix[i][0] == 0) {
22                firstColZero = true;
23                break;
24            }
25        }
26
27        // Use first row and column as markers.
28        for (int i = 1; i < m; i++) {
29            for (int j = 1; j < n; j++) {
30                if (matrix[i][j] == 0) {
31                    matrix[i][0] = 0; // mark the entire row
32                    matrix[0][j] = 0; // mark the entire column
33                }
34            }
35        }
36
37        // Zero out cells based on markers.
38        for (int i = 1; i < m; i++) {
39            for (int j = 1; j < n; j++) {
40                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
41                    matrix[i][j] = 0;
42                }
43            }
44        }
45
46        // Zero out the first row if needed.
47        if (firstRowZero) {
48            for (int j = 0; j < m; j++) {
49                matrix[0][j] = 0;
50            }
51        }
52    }
53 }
```

```
1 }
2
3 // Zero out the first column if needed.
4 if (firstColZero) {
5     for (int i = 0; i < n; i++) {
6         matrix[i][0] = 0;
7     }
8 }
9
10 }
```

#74 MEDIUM

Search a 2D Matrix

LeetCode · SimplifyLeetCode · WalaCC · LeetCode · Editorial

Array · Binary Search · Matrix

Utsav Grind 169

Problem:

You are given an $m \times n$ integer matrix with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer target, return true if target is in matrix or false otherwise.

You must write a solution in $O(\log(m \cdot n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16

15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-104 \leq \text{matrix}[i][j], \text{target} \leq 104$

>> Brute Force [Matrix] Interview Prep

Python

```
1 # Brute Force Approach
2 class Solution:
3     def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
4         # Brute Force (O(m * n)) - row every cell
5         for row in matrix:
6             for col in row:
7                 if col == target:
8                     return True
9         return False
```

```
1 def searchMatrix(matrix: List[List[int]], target: int) -> bool:
2     # Set number of rows and columns
3     if not matrix or not matrix[0]:
4         return False
5
6     rows, cols = len(matrix), len(matrix[0])
7     left, right = 0, rows * cols - 1
8
9     # Perform binary search over the virtual flattened array
10    while left <= right:
11        # Find the middle index
12        mid = (left + right) // 2
13        # Map the middle index to row and column in the matrix
14        row = mid // cols
15        col = mid % cols
16        mid_value = matrix[row][col]
17
18        # Check if the mid_value equals the target
19        if mid_value == target:
20            return True
21        elif mid_value < target:
22            left = mid + 1 # Move to the right subarray
23        else:
24            right = mid - 1 # Move to the left subarray
25
26    # Target not found
27    return False
28
29 # Example usage:
30 # matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]
31 # target = 13
32 # print(searchMatrix(matrix, target))
33
34 Python
35
36 class Solution:
37     def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
38         if not matrix:
39             return False
40
41         n = len(matrix)
42         m = len(matrix[0])
43         l = 0
44         r = m * n
45
46         while l < r:
47             mid = (l + r) // 2
48             i = mid // m
49             j = mid % m
50             if matrix[i][j] == target:
51                 return True
52             elif matrix[i][j] < target:
53                 l = mid + 1
54             else:
55                 r = mid
56         return False
```

```
class Solution {
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m - 1
        while left < right:
            mid = (left + right) // 2
            x, y = divmod(mid, n)
            if matrix[x][y] == target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target

Python
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m - 1
        while left < right:
            mid = (left + right) // 2
            x, y = divmod(mid, n)
            if matrix[x][y] == target:
                right = mid
            elif matrix[x][y] > target:
                right = mid - 1
            else:
                left = mid + 1
        return False

#####
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m - 1
        while left < right:
            mid = (left + right) // 2
            x, y = divmod(mid, n)
            if matrix[x][y] == target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target

C++
C++
```

```
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        if (matrix.empty())
            return false;

        const int m = matrix.size();
        const int n = matrix[0].size();
        int l = 0;
        int r = m - 1;

        while (l < r) {
            const int mid = (l + r) / 2;
            const int i = mid / n;
            const int j = mid % n;
            if (matrix[i][j] == target)
                return true;
            if (matrix[i][j] < target)
                l = mid + 1;
            else
                r = mid;
        }
        return false;
    }
};

C++
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int m = matrix.size(), n = matrix[0].size();
        int left = 0, right = m - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            int x = mid / n, y = mid % n;
            if (matrix[x][y] == target) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return matrix[left / n][left % n] == target;
    }
};

C++
```

```
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int m = matrix.size(), n = matrix[0].size();
        int left = 0, right = m - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            int x = mid / n, y = mid % n;
            if (matrix[x][y] == target) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return matrix[left / n][left % n] == target;
    }
};

Java
Java
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        if (matrix.length == 0)
            return false;

        final int m = matrix.length;
        final int n = matrix[0].length;
        int l = 0;
        int r = m - 1;

        while (l < r) {
            final int mid = (l + r) / 2;
            final int i = mid / n;
            final int j = mid % n;
            if (matrix[i][j] == target)
                return true;
            if (matrix[i][j] < target)
                l = mid + 1;
            else
                r = mid;
        }
        return false;
    }
}

Java
```

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int left = 0, right = m - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            int x = mid / n, y = mid % n;
            if (matrix[x][y] == target) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return matrix[left / n][left % n] == target;
    }
}

JavaScript
JavaScript
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int left = 0, right = m - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            int x = mid / n, y = mid % n;
            if (matrix[x][y] == target) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return matrix[left / n][left % n] == target;
    }
}

JavaScript
JavaScript
```

```
/**
 * @param {number[][]} matrix
 * @param {number} target
 * @return {boolean}
 */
var searchMatrix = function(matrix, target) {
    const m = matrix.length;
    n = matrix[0].length;
    let left = 0;
    right = m - 1;
    while (left < right) {
        const mid = (left + right) // 2;
        const x = Math.floor(mid / n);
        const y = mid % n;
        if (matrix[x][y] == target) {
            left = mid;
        } else {
            right = mid + 1;
        }
    }
    return matrix[Math.floor(left / n)][left % n] == target;
};

JavaScript
JavaScript
/**
 * @param {number[][]} matrix
 * @param {number} target
 * @return {boolean}
 */
var searchMatrix = function(matrix, target) {
    const m = matrix.length;
    n = matrix[0].length;
    let left = 0;
    right = m - 1;
    while (left < right) {
        const mid = (left + right + 1) // 2;
        const x = Math.floor(mid / n);
        const y = mid % n;
        if (matrix[x][y] == target) {
            left = mid;
        } else {
            right = mid - 1;
        }
    }
    return matrix[Math.floor(left / n)][left % n] == target;
};

TypeScript
TypeScript
```

```
function searchMatrix(matrix: number[][], target: number): boolean {
    const m = matrix.length;
    const n = matrix[0].length;
    let left = 0;
    let right = m - 1;
    while (left < right) {
        const mid = (left + right) // 2;
        const i = Math.floor(mid / n);
        const j = mid % n;
        if (matrix[i][j] == target) {
            return true;
        }
        if (matrix[i][j] < target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return false;
}

TypeScript
function searchMatrix(matrix: number[][], target: number): boolean {
    const m = matrix.length;
    const n = matrix[0].length;
    let left = 0;
    let right = m - 1;
    while (left < right) {
        const mid = (left + right) // 2;
        const i = Math.floor(mid / n);
        const j = mid % n;
        if (matrix[i][j] == target) {
            return true;
        }
        if (matrix[i][j] < target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return false;
}

Go
Go
```

```
func searchMatrix(matrix: [[Int, target Int]) bool {
    m, n := len(matrix), len(matrix[0])
    left, right := 0, m-1
    for left < right {
        mid := (left + right) == 1
        x, y := mid*n, mid*n
        if matrix[x][y] == target {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return matrix[left*n][left*n] == target
}
```

Go

```
func searchMatrix(matrix: [[Int, target Int]) bool {
    m, n := len(matrix), len(matrix[0])
    left, right := 0, m-1
    for left < right {
        mid := (left + right) == 1
        x, y := mid*n, mid*n
        if matrix[x][y] == target {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return matrix[left*n][left*n] == target
}
```

Rust

```
use std::cmp::Ordering;

impl Solution {
    pub fn searchMatrix(matrix: Vec<Vec<i32>>, target: i32) -> bool {
        let n = matrix.len();
        let m = matrix[0].len();
        let mut i = 0;
        while i < m {
            let mut j = 0;
            while j < n {
                match matrix[i][j] - target {
                    Ordering::Equal => {
                        return true;
                    }
                    Ordering::Less => {
                        i += 1;
                    }
                    Ordering::Greater => {
                        j += 1;
                    }
                }
            }
        }
        false
    }
}
```

Matrix · LeetCode · By Pattern

— 1645 —

LeetCode

Rust

```
use std::cmp::Ordering;

impl Solution {
    pub fn searchMatrix(matrix: Vec<Vec<i32>>, target: i32) -> bool {
        let n = matrix.len();
        let m = matrix[0].len();
        let mut i = 0;
        while i < m {
            let mut j = 0;
            while j < n {
                match matrix[i][j] - target {
                    Ordering::Equal => {
                        return true;
                    }
                    Ordering::Less => {
                        i += 1;
                    }
                    Ordering::Greater => {
                        j += 1;
                    }
                }
            }
        }
        false
    }
}
```

[1] Matrix — Pattern Solution (matched from community answers)

Python

```
def searchMatrix(matrix, target):
    # Get number of rows and columns
    if not matrix or not matrix[0]:
        return False

    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows + cols - 1

    # Perform binary search over the virtual flattened array
    while left <= right:
        # Find the middle index
        mid = (left + right) // 2
        # Find the middle element in row and column in the matrix
        row = mid // cols
        col = mid % cols
        mid_value = matrix[row][col]

        # Check if the mid value equals the target
        if mid_value == target:
            return True
        elif mid_value < target:
            left = mid + 1 # Move to the right subarray
        else:
            right = mid - 1 # Move to the left subarray
```

Matrix · LeetCode · By Pattern

— 1646 —

LeetCode

```
# Target not found
return False

# Example usage:
# matrix = [[1,1,1,1,1],[1,0,0,0,0],[1,0,0,0,0]]
# target = 1
# print(searchMatrix(matrix, target))
```

#221

MEDIUM

Maximal Square

LeetCode · SimplyEasy · WalkCC · LeetCode · Editorial

Array · Dynamic Programming · Matrix

Lists Grid 189

Problem:

Given an $m \times n$ binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

```
Input: matrix =
[[1,0,1,0,0],
 [1,0,1,1,1],
 [1,1,1,1,1],
 [1,0,0,1,0]]
Output: 4
```

Example 2:

Matrix · LeetCode · By Pattern

— 1647 —

LeetCode

0	1
1	0

```
Input: matrix = [[0,1],[1,0]]
Output: 1
```

Example 3:

```
Input: matrix = [[0]]
Output: 0
```

Constraints:

- m == matrix.length
- n == matrix[0].length
- $1 \leq m, n \leq 300$
- matrix[i][j] is '0' or '1'.

>> Brute Force [Matrix] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def maximalSquare(self, matrix):
        # Brute Force O(m^2 * n^2) - Flattened Fill from every cell
        m, n = len(matrix), len(matrix[0])
        result = 0
        for i in range(m):
            for j in range(n):
                visited = set()
                def flood(r, c):
                    if (r, c) in visited or not (0 <= r < m and 0 <= c < n): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: flood(r+dr, c+dc)
                flood(r, c)
                result = max(result, len(visited))
        return result
```

Python

Python

Matrix · LeetCode · By Pattern

— 1648 —

LeetCode

```
# Python Implementation of the Maximal Square problem
def maximalSquare(matrix):
    # Check if matrix is empty
    if not matrix or not matrix[0]:
        return 0
    rows = len(matrix)
    cols = len(matrix[0])
    # DP table with extra row and col for ease of boundary conditions
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    max_side = 0
    # Iterate through each cell in the matrix
    for i in range(1, rows + 1):
        for j in range(1, cols + 1):
            # If the current cell is '1'
            if matrix[i-1][j-1] == '1':
                # Check the top, left, and top-left neighbors and add 1
                dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
            # The area of the square is the square of the side length
            return max_side * max_side
```

```
# Example usage:
matrix = [
    ["1", "0", "1", "0", "0"],
    ["1", "0", "1", "1", "1"],
    ["1", "1", "1", "1", "1"],
    ["1", "0", "0", "1", "0"]
]
print(maximalSquare(matrix))
```

Python

```
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * n for _ in range(m)]
        max_length = 0
        for i in range(m):
            for j in range(n):
                if i == 0 or j == 0 or matrix[i][j] == '0':
                    dp[i][j] = 1 if matrix[i][j] == '1' else 0
                else:
                    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
                    max_length = max(max_length, dp[i][j])
        return max_length * max_length
```

Python

Matrix · LeetCode · By Pattern

— 1649 —

LeetCode

```
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        max = 0
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == '1':
                    dp[i+1][j+1] = 1 + min(dp[i][j], dp[i+1][j], dp[i][j+1])
                    max = max(max, dp[i+1][j+1])
        return max * max
```

Python

```
eq. a 1x10 square with full of 1s
this square more 1 line down
this square more 1 line right
...
so there needs an extra single 1 at bottom right, to make it a larger full square
```

```
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        max = 0
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == '1':
                    dp[i+1][j+1] = 1 + min(dp[i][j], dp[i+1][j], dp[i][j+1])
                    max = max(max, dp[i+1][j+1])
        return max * max
```

C++

C++

```
class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        const int m = matrix.size();
        const int n = matrix[0].size();
        vector<vector<int>> dp(m, vector<int>(n));
        int maxLength = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                if (i == 0 || j == 0 || matrix[i][j] == '0')
                    dp[i][j] = matrix[i][j] == '1' ? 1 : 0;
                else
                    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1;
                maxLength = max(maxLength, dp[i][j]);
            }
        return maxLength * maxLength;
    }
};
```

C++

Matrix · LeetCode · By Pattern

— 1650 —

LeetCode

```

class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        int m = matrix.size(), n = matrix[0].size();
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
        int mx = 0;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == '1') {
                    dp[i + 1][j + 1] = min(dp[i][j] + 1, dp[i + 1][j], dp[i][j + 1]) + 1;
                    mx = max(mx, dp[i + 1][j + 1]);
                }
            }
        }
        return mx * mx;
    }
};

```

```

C++
class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        int m = matrix.size(), n = matrix[0].size();
        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
        int mx = 0;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == '1') {
                    dp[i + 1][j + 1] = min(dp[i][j] + 1, dp[i + 1][j], dp[i][j + 1]) + 1;
                    mx = max(mx, dp[i + 1][j + 1]);
                }
            }
        }
        return mx * mx;
    }
};

```

Java
Java

Matrix · LeetCode — By Pattern

— 1651 —

LeetCode

```

class Solution {
public int maximalSquare(char[][] matrix) {
    final int m = matrix.length;
    final int n = matrix[0].length;
    int[][] dp = new int[m][n];
    int maxLength = 0;

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) {
            if (i == 0 || j == 0) matrix[i][j] == '0'
                dp[i][j] = matrix[i][j] == '1' ? 1 : 0;
            else
                dp[i][j] = Math.min(dp[i - 1][j] - 1, Math.min(dp[i - 1][j], dp[i][j - 1])) + 1;
            maxLength = Math.max(maxLength, dp[i][j]);
        }

    return maxLength * maxLength;
}
}

```

```

Java
class Solution {
public int maximalSquare(char[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    int[][] dp = new int[m + 1][n + 1];
    int mx = 0;
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            if (matrix[i][j] == '1') {
                dp[i + 1][j + 1] = Math.min(Math.min(dp[i][j] + 1, dp[i + 1][j]), dp[i][j + 1]) + 1;
                mx = Math.max(mx, dp[i + 1][j + 1]);
            }
        }
    }
    return mx * mx;
}
}

```

```

Java
public class Solution {
    public int MaximalSquare(char[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        var dp = new int[m + 1, n + 1];
        int mx = 0;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == '1') {
                    dp[i + 1, j + 1] = Math.Max(Math.Min(dp[i, j] + 1, dp[i + 1, j]), dp[i, j + 1]) + 1;
                    mx = Math.Max(mx, dp[i + 1, j + 1]);
                }
            }
        }
        return mx * mx;
    }
}

```

Matrix · LeetCode — By Pattern

— 1652 —

LeetCode

```

Java
class Solution {
    public int maximalSquare(char[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        int[][] dp = new int[m + 1][n + 1];
        int mx = 0;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == '1') {
                    dp[i + 1][j + 1] = Math.min(Math.min(dp[i][j] + 1, dp[i + 1][j]), dp[i][j + 1]) + 1;
                    mx = Math.max(mx, dp[i + 1][j + 1]);
                }
            }
        }
        return mx * mx;
    }
}

```

```

Java
public class Solution {
    public int MaximalSquare(char[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        var dp = new int[m + 1, n + 1];
        int mx = 0;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (matrix[i][j] == '1') {
                    dp[i + 1, j + 1] = Math.Max(Math.Min(dp[i, j] + 1, dp[i + 1, j]), dp[i, j + 1]) + 1;
                    mx = Math.Max(mx, dp[i + 1, j + 1]);
                }
            }
        }
        return mx * mx;
    }
}

```

Go
Go

Matrix · LeetCode — By Pattern

— 1653 —

LeetCode

```

func maximalSquare(matrix [][]byte) int {
    m, n := len(matrix), len(matrix[0])
    dp := make([][]int, m+1)
    for i := 0; i <= m; i++ {
        dp[i] = make([]int, n+1)
    }
    mx := 0
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if matrix[i][j] == '1' {
                dp[i+1][j+1] = min(min(dp[i][j]+1, dp[i+1][j]), dp[i][j+1]) + 1
                mx = max(mx, dp[i+1][j+1])
            }
        }
    }
    return mx * mx
}

```

```

Go
func maximalSquare(matrix [][]byte) int {
    m, n := len(matrix), len(matrix[0])
    dp := make([][]int, m+1)
    for i := 0; i <= m; i++ {
        dp[i] = make([]int, n+1)
    }
    mx := 0
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if matrix[i][j] == '1' {
                dp[i+1][j+1] = min(min(dp[i][j]+1, dp[i+1][j]), dp[i][j+1]) + 1
                mx = max(mx, dp[i+1][j+1])
            }
        }
    }
    return mx * mx
}

```

[*] Matrix — Pattern Solution (stored optimal)

Python

Matrix · LeetCode — By Pattern

— 1654 —

LeetCode

```

# Python implementation of the Maximal Square problem
def maximalSquare(matrix):
    # Check if matrix is empty
    if not matrix or not matrix[0]:
        return 0
    rows = len(matrix)
    cols = len(matrix[0])
    # DP table with extra row and col for ease of boundary conditions
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    max_side = 0
    # Iterate through each cell in the matrix
    for i in range(1, rows + 1):
        for j in range(1, cols + 1):
            # If the current cell is '1'
            if matrix[i - 1][j - 1] == '1':
                dp[i][j] = min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]) + 1
                max_side = max(max_side, dp[i][j])
    # The area of the square is the square of the side length
    return max_side * max_side

```

```

# Example usage:
matrix = [
    ["1", "0", "1", "0", "0"],
    ["1", "0", "0", "1", "0"],
    ["1", "1", "1", "0", "1"],
    ["1", "1", "1", "1", "1"],
    ["1", "0", "0", "1", "0"]
]
print(maximalSquare(matrix))

```

C++

```

// C++ implementation of the Maximal Square problem
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int maximalSquare(vector<vector<char>>& matrix) {
    if (matrix.empty() || matrix[0].empty()) return 0;
    int rows = matrix.size(), cols = matrix[0].size();
    // Create a DP table with extra row and column
    vector<vector<int>> dp(rows + 1, vector<int>(cols + 1, 0));
    int maxSide = 0;
    // Iterate through each cell
    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= cols; ++j) {
            if (matrix[i - 1][j - 1] == '1') {
                dp[i][j] = min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]) + 1;
                maxSide = max(maxSide, dp[i][j]);
            }
        }
    }
    // Return the area of the largest square
    return maxSide * maxSide;
}

```

Matrix · LeetCode — By Pattern

— 1655 —

LeetCode

```

// Example usage:
int main() {
    vector<vector<char>> matrix = {
        {'1', '0', '1', '0', '0'},
        {'1', '0', '0', '1', '0'},
        {'1', '1', '1', '0', '1'},
        {'1', '1', '1', '1', '1'},
        {'1', '0', '0', '1', '0'}
    };
    cout << maximalSquare(matrix) << endl;
    return 0;
}

```

#311 MEDIUM

Sparse Matrix Multiplication

LeetCode · Solutions · MultiC · LeetCode · Editorial

Array · Hash Table · Matrix

LeetCode Premium 98

Problem:

Given two sparse matrices mat1 of size m x k and mat2 of size k x n, return the result of mat1 x mat2. You may assume that multiplication is always possible.

Example 1:

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 0 & 3 \end{bmatrix} \times \begin{bmatrix} 7 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 0 & 0 \\ -7 & 0 & 3 \end{bmatrix}$$

Input: mat1 = [[1,0,0],[-1,0,3]], mat2 = [[7,0,0],[0,0,0],[0,0,1]]
Output: [[7,0,0],[-7,0,3]]

Example 2:

Input: mat1 = [[0]], mat2 = [[0]]
Output: [[0]]

Constraints:

- m == mat1.length
- k == mat1[0].length == mat2.length
- n == mat2[0].length
- 1 <= m, n, k <= 100
- 100 <= mat1[i][j], mat2[i][j] <= 100

>> Brute Force [Matrix Interview Prep]

Matrix · LeetCode — By Pattern

— 1656 —

LeetCode


```

Python
# Brute Force Approach
class Solution:
    def sparseMatrixMultiplication(self, mat1, mat2):
        # Brute Force Approach: Flood Fill from every cell
        m, n = len(mat1), len(mat1[0])
        result = 0
        for i in range(m):
            for c in range(n):
                visited = set()
                def flood(r, c):
                    if (r,c) in visited or (b==r and b==c): return
                    visited.add((r,c))
                    for dr,dc in [(0,1),(0,-1),(1,0),(-1,0)]: flood(r+dr,c+dc)
                flood(r, c)
            result = max(result, len(visited))
        return result

Python
def sparseMatrixMultiplication(mat1, mat2):
    # Get dimensions of mat1 and mat2
    m, n = len(mat1), len(mat1[0])
    n2 = len(mat2[0])
    # Initialize result matrix with zeros
    result = [[0] * n2 for _ in range(m)]
    # Iterate through each row in mat1
    for i in range(m):
        # For each element in row i of mat1
        for p in range(n):
            if mat1[i][p] != 0: # Only consider non-zero element in mat1
                # Multiply with corresponding elements in the row of mat2
                for j in range(n2):
                    if mat2[p][j] != 0: # Only consider non-zero element in mat2
                        result[i][j] = mat1[i][p] * mat2[p][j]
    return result

# Example usage
print(sparseMatrixMultiplication([[1,0,0],[-1,0,0]], [[7,6,0],[0,6,0],[0,6,1]]))

Python
class Solution:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        m = len(mat1)
        n = len(mat2)
        l = len(mat2[0])
        ans = [[0] * l for _ in range(m)]
        for i in range(m):
            for j in range(l):
                for k in range(n):
                    ans[i][j] += mat1[i][k] * mat2[k][j]
        return ans

Matrix · LeetCode · By Pattern — 1657 — LeetCode

```

```

Python
class Solution:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        m, n = len(mat1), len(mat2[0])
        ans = [[0] * n for _ in range(m)]
        for i in range(m):
            for k in range(len(mat2)):
                for j in range(n):
                    ans[i][j] += mat1[i][k] * mat2[k][j]
        return ans

Python
class Solution:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        q = [[0] for _ in range(len(mat1))]
        for i, row in enumerate(mat1):
            for j, x in enumerate(row):
                if x:
                    q[j].append(i, x)
            return q
        q1 = f(mat1)
        q2 = f(mat2)
        m = len(mat1), len(mat2[0])
        ans = [[0] * n for _ in range(m)]
        for i in range(m):
            for k, x in q1[i]:
                for j, y in q2[k]:
                    ans[i][j] += x * y
        return ans

C++
class Solution {
public:
    vector<vector<int>> multiply(vector<vector<int>>& mat1, vector<vector<int>>& mat2) {
        const int m = mat1.size();
        const int n = mat1[0].size();
        const int l = mat2[0].size();
        vector<vector<int>> ans(m, vector<int>(l));
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < l; ++j)
                for (int k = 0; k < n; ++k)
                    ans[i][j] += mat1[i][k] * mat2[k][j];
        return ans;
    }
};

C++

```

```

class Solution {
public:
    vector<vector<int>> multiply(vector<vector<int>>& mat1, vector<vector<int>>& mat2) {
        int m = mat1.size(), n = mat2[0].size();
        int l = mat2[0].size();
        vector<vector<int>> ans(m, vector<int>(l));
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < l; ++j)
                for (int k = 0; k < n; ++k)
                    ans[i][j] += mat1[i][k] * mat2[k][j];
        return ans;
    }
};

C++
class Solution {
public:
    vector<vector<int>> multiply(vector<vector<int>>& mat1, vector<vector<int>>& mat2) {
        int m = mat1.size(), n = mat2[0].size();
        vector<vector<int>> ans(m, vector<int>(n));
        mat1 = f(mat1), n2 = f(mat2);
        for (int i = 0; i < m; ++i)
            for (int k = 0; k < n; ++k)
                for (int j = 0; j < n2; ++j)
                    ans[i][j] += mat1[i][k] * mat2[k][j];
        return ans;
    }

    vector<vector<int>> f(vector<vector<int>>& mat) {
        int m = mat.size(), n = mat[0].size();
        vector<vector<int>> ans(m, vector<int>(n));
        for (int i = 0; i < m; ++i)
            for (int k = 0; k < n; ++k)
                if (mat[i][k])
                    q[i].emplace_back(k, mat[i][k]);
        return q;
    }
};

Java
Java

```

```

class Solution {
    public List<List<Integer>> multiply(List<List<Integer>> mat1, List<List<Integer>> mat2) {
        int m = mat1.size(), n = mat2[0].size();
        List<List<Integer>> ans = new ArrayList<>();
        for (int i = 0; i < m; ++i)
            for (int k = 0; k < n; ++k)
                if (mat1.get(i).get(k) != 0)
                    ans.add(new ArrayList<>());
        for (int i = 0; i < m; ++i)
            for (int k = 0; k < n; ++k)
                if (mat1.get(i).get(k) != 0)
                    for (int j = 0; j < mat2[0].size(); ++j)
                        ans.get(i).add(mat1.get(i).get(k) * mat2.get(k).get(j));
        return ans;
    }
}

TypeScript
function multiply(mat1: number[][], mat2: number[][]): number[][] {
    const [m, n] = [mat1.length, mat2[0].length];
    const ans: number[][] = Array.from<length: m>(), () => Array.from<length: n>(), () => 0);
    for (let i = 0; i < m; ++i) {
        for (let k = 0; k < n; ++k) {
            if (mat1[i][k] !== 0) {
                const q1 = f(mat1);
                const q2 = f(mat2);
                for (let i = 0; i < m; ++i)
                    for (let k = 0; k < n; ++k)
                        if (mat1[i][k] !== 0)
                            for (let j = 0; j < q2[0].length; ++j)
                                ans[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }
    return ans;
}

TypeScript
function multiply(mat1: number[][], mat2: number[][]): number[][] {
    const [m, n] = [mat1.length, mat2[0].length];
    const ans: number[][] = Array.from<length: m>(), () => Array.from<length: n>(), () => 0);
    const f = (mat: number[][]): number[][] => {
        const [m, n] = [mat.length, mat[0].length];
        const ans: number[][] = Array.from<length: m>(), () => [];
        for (let i = 0; i < m; ++i)
            for (let k = 0; k < n; ++k)
                if (mat[i][k] !== 0)
                    ans[i].push(k, mat[i][k]);
        return ans;
    };
    const q1 = f(mat1);
    const q2 = f(mat2);
    for (let i = 0; i < m; ++i)
        for (let k = 0; k < n; ++k)
            if (mat1[i][k] !== 0)
                for (const [k2, v2] of q2[k])
                    ans[i][k2] += mat1[i][k] * v2;
    return ans;
}

Go
Go

```

```

func multiply(mat1 [][]int, mat2 [][]int) [][]int {
    m, n := len(mat1), len(mat2[0])
    ans := make([][]int, m)
    for i := range ans {
        ans[i] = make([]int, n)
        for k := 0; k < n; ++k {
            for j := 0; j < n; ++j {
                ans[i][j] += mat1[i][k] * mat2[k][j]
            }
        }
    }
    return ans
}

Go
func multiply(mat1 [][]int, mat2 [][]int) [][]int {
    m, n := len(mat1), len(mat2[0])
    ans := make([][]int, m)
    for i := range ans {
        ans[i] = make([]int, n)
        f := func(mat [][]int) [][]int {
            m, n := len(mat), len(mat[0])
            q := make([][]int, m)
            for i := range q {
                q[i] = make([]int, n)
                for j := range mat[i] {
                    if mat[i][j] != 0 {
                        q[i] = append(q[i], (2*int(i), mat[i][j]))
                    }
                }
            }
            return q
        }
        q1, q2 := f(mat1), f(mat2)
        for i := range q1 {
            for j := range q1[i] {
                k, v := q1[i][j]
                for k2 := 0; k2 < n; ++k2 {
                    if q2[k2] != 0 {
                        ans[i][k2] += v * q2[k2]
                    }
                }
            }
        }
    }
    return ans
}

[*] Matrix — Pattern Solution (stored optimal)
Python

```



```

class Solution {
    public int[] findDiagonalOrder(int[][] matrix) {
        final int n = matrix.length;
        final int m = matrix[0].length;
        int[] ans = new int[n * m];
        int d = 1; // left-bottom to right-top
        int row = 0;
        int col = 0;

        for (int i = 0; i < n * m; ++i) {
            ans[i] = matrix[row][col];
            row += d;
            col += d;
            // reached-bottom
            if (row == n) {
                row = m - 1;
                d = -d;
            }
            // reached-right
            if (col == m) {
                col = m - 1;
                d = -d;
            }
            if (row < 0) {
                row = 0;
                d = -d;
            }
            if (col < 0) {
                col = 0;
                d = -d;
            }
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int[] findDiagonalOrder(int[][] mat) {
        int n = mat.length, m = mat[0].length;
        int[] ans = new int[n * m];
        int i = 0;
        List<Integer> t = new ArrayList<>();
        for (int k = 0; k < n + m - 1; ++k) {
            int l = k < m ? 0 : k - m + 1;
            int j = k < m ? k : n - 1;
            while (l < m && j >= 0) {
                t.add(mat[l][j]);
                ++l;
                --j;
            }
            if (k % 2 == 0) {
                Collections.reverse(t);
            }
            for (int v : t) {
                ans[i++] = v;
            }
            t.clear();
        }
        return ans;
    }
}

```

```

class Solution {
    public int[] findDiagonalOrder(int[][] mat) {
        int n = mat.length;
        int m = mat[0].length;
        List<Integer> ans = new List<>();
        for (int k = 0; k < n + m - 1; ++k) {
            List<Integer> t = new List<>();
            int l = k < m ? 0 : k - m + 1;
            int j = k < m ? k : n - 1;
            while (l < m && j >= 0) {
                t.add(mat[l][j]);
                ++l;
                --j;
            }
            if (k % 2 == 0) {
                t.reverse();
            }
            ans.addAll(t);
        }
        return ans.toArray();
    }
}

```

Java

```

class Solution {
    public int[] findDiagonalOrder(int[][] mat) {
        int n = mat.length, m = mat[0].length;
        int[] ans = new int[n * m];
        int i = 0;
        List<Integer> t = new ArrayList<>();
        for (int k = 0; k < n + m - 1; ++k) {
            int l = k < m ? 0 : k - m + 1;
            int j = k < m ? k : n - 1;
            while (l < m && j >= 0) {
                t.add(mat[l][j]);
                ++l;
                --j;
            }
            if (k % 2 == 0) {
                Collections.reverse(t);
            }
            for (int v : t) {
                ans[i++] = v;
            }
            t.clear();
        }
        return ans;
    }
}

```

```

class Solution {
    public int[] findDiagonalOrder(int[][] mat) {
        int n = mat.length;
        int m = mat[0].length;
        List<Integer> ans = new List<>();
        for (int k = 0; k < n + m - 1; ++k) {
            List<Integer> t = new List<>();
            int l = k < m ? 0 : k - m + 1;
            int j = k < m ? k : n - 1;
            while (l < m && j >= 0) {
                t.add(mat[l][j]);
                ++l;
                --j;
            }
            if (k % 2 == 0) {
                t.reverse();
            }
            ans.addAll(t);
        }
        return ans.toArray();
    }
}

```

TypeScript
TypeScript

```

function findDiagonalOrder(mat: number[][]): number[] {
    const n = mat.length;
    const m = mat[0].length;
    const ans: number[] = [];
    for (let k = 0; k < n + m - 1; ++k) {
        const t: number[] = [];
        let l = k < m ? 0 : k - m + 1;
        let j = k < m ? k : n - 1;
        while (l < m && j >= 0) {
            t.push(mat[l][j]);
            ++l;
            --j;
        }
        if (k % 2 === 0) {
            t.reverse();
        }
        ans.push(...t);
    }
    return ans;
}

```

```

function findDiagonalOrder(mat: number[][]): number[] {
    const res = [];
    const n = mat.length;
    const m = mat[0].length;
    let i = 0;
    let mark = true;
    while (res.length !== n * m) {
        if (mark) {
            while (i <= 0 && j < m) {
                res.push(mat[i][j]);
                ++i;
                --j;
            }
            if (i === 0) {
                ++i;
                --j;
            }
        } else {
            while (i < m && j >= 0) {
                res.push(mat[i][j]);
                ++i;
                --j;
            }
            if (i === m) {
                ++i;
                --j;
            }
        }
        mark = !mark;
    }
    return res;
}

```

```

Go
func findDiagonalOrder(mat [][]int) []int {
    n := len(mat)
    m := len(mat[0])
    ans := make([]int, 0, n*m)
    for k := 0; k < n+m-1; k++ {
        t := make([]int, 0)
        if k % 2 == 0 {
            i, j := 0, k
        } else {
            i, j := k-m+1, 0
        }
        for i < n && j >= 0 {
            t = append(t, mat[i][j])
            i++
            j--
        }
        if k % 2 == 0 {
            slices.Reverse(t)
        }
        ans = append(ans, t...)
    }
    return ans
}

```

```

Go
func findDiagonalOrder(mat [][]int) []int {
    n, m := len(mat), len(mat[0])
    var ans []int
    for k := 0; k < n+m-1; k++ {
        var t []int
        i, j := k-m+1, 0
        if k % 2 == 0 {
            i, j := 0, k
        } else {
            i, j := k-m+1, 0
        }
        for i < n && j >= 0 {
            t = append(t, mat[i][j])
            i++
            j--
        }
        if k % 2 == 0 {
            p, q := 0, len(t)-1
            for p < q {
                t[p], t[q] = t[q], t[p]
                p++
                q--
            }
        }
        for _, v := range t {
            ans = append(ans, v)
        }
    }
    return ans
}

```

```

}
return ans
}

Rust
impl Solution {
    pub fn find_diagonal_order(mat: Vec<Vec<i32>>) -> Vec<i32> {
        let n = mat.len();
        let m = mat[0].len();
        let mut ans = Vec::with_capacity(n * m);
        for k in 0..(n + m - 1) {
            let mut t = Vec::new();
            let (i, mut j) = if k % 2 == 0 {
                (0, k)
            } else {
                (k - m + 1, 0)
            };
            while i < n && j >= 0 {
                t.push(mat[i][j]);
                i += 1;
                j -= 1;
            }
            if k % 2 == 0 {
                t.reverse();
            }
            ans.extend(t);
        }
        ans
    }
}

```

Rust

```
impl Solution {
    pub fn find_diagonal_order(mat: Vec<Vec<i32>>) -> Vec<i32> {
        let (m, n) = (mat.len(), mat[0].len());
        let (mut i, mut j) = (0, 0);
        let mut res = Vec::new();
        while i < m || j < n {
            let res = mat[i][j];
            if (i > 0) && j == 0 {
                i -= 1;
                j += 1;
            } else if i == 0 {
                i += 1;
                j += 1;
            } else {
                i += 1;
                j -= 1;
            }
            if i == m - 1 {
                i -= 1;
                j += 1;
            } else if j == n - 1 {
                i += 1;
                j -= 1;
            }
            res.push(res);
        }
    }
}
```

[*] Matrix — Pattern Solution (stored optimal)

Python

```
def findDiagonalOrder(mat):
    if not mat or not mat[0]:
        return []
    m, n = len(mat), len(mat[0])
    result = []
    row, col = 0, 0
    direction = 1 # 1 means upward, -1 means downward
    while len(result) < m * n:
        result.append(mat[row][col])
        if direction == 1: # moving upward
            if col == 0: # hit the left boundary
                row += 1
                direction = -1
            elif row == 0: # hit the top boundary
                col += 1
                direction = -1
            else:
                row -= 1
                col += 1
        else: # moving downward
            if row == m - 1: # hit the bottom boundary
                col += 1
                direction = 1
            elif col == n - 1: # hit the right boundary
                row += 1
                direction = 1
            else:
                row += 1
                col += 1
        direction = -direction
    return result
# Example usage:
print(findDiagonalOrder([[1,2,3],[4,5,6],[7,8,9]]))
```

C++

```
#include <vector>
using namespace std;

class Solution {
public:
    vector<int> findDiagonalOrder(vector<vector<int>>& mat) {
        if(mat.empty() || mat[0].empty())
            return {};
        int n = mat.size(), m = mat[0].size();
        vector<int> result;
        int row = 0, col = 0;
        int direction = 1; // 1 for upward, -1 for downward
        while(result.size() < m * n) {
            result.push_back(mat[row][col]);
            if(direction == 1) { // moving upward
                if(col == m - 1) {
                    row++;
                    direction = -1;
                } else if(row == 0) {
                    col++;
                    direction = -1;
                } else {
                    row--;
                    col++;
                }
            } else { // moving downward
                if(row == m - 1) {
                    col++;
                    direction = 1;
                } else if(col == 0) {
                    row++;
                    direction = 1;
                } else {
                    row++;
                    col--;
                }
            }
        }
        return result;
    }
};

// Example usage is handled in the testing environment.
```

#531 MEDIUM

Lonely Pixel I

LeetCode · Company · [WubCC](#) · [LeetCode](#) · [Editorial](#)

Array · Hash Table · Matrix

Like Premium 98

Problem:
Given an m x n picture consisting of black 'B' and white 'W' pixels, return the number of lonely black pixels.

A black lonely pixel is a character 'B' that located at a specific position where the same row and same column don't have any other black pixels.

Example 1:

W	W	B
W	B	W
B	W	W

Input: picture = [[“W”,“W”,“B”],[“W”,“B”,“W”],[“B”,“W”,“W”]]
Output: 3
Explanation: All the three 'B's are black lonely pixels.

Example 2:

B	B	B
B	B	W
B	B	B

Input: picture = [[“B”,“B”,“B”],[“B”,“B”,“W”],[“B”,“B”,“B”]]
Output: 0
Constraints:

```
• m == picture.length
• n == picture[0].length
• 1 <= m, n <= 500
• picture[i][j] is 'W' or 'B'.

>> Brute Force (Matrix) Interview Prep

Python
# Brute Force Approach
class Solution:
    def findLonelyPixel(self, picture):
        # Count lonely pixels by checking each cell
        m, n = len(picture), len(picture[0])
        result = 0
        for i in range(m):
            for j in range(n):
                visited = set()
                def findLonelyPixel(i, j):
                    if (i, j) in visited or not (0 <= i < m and 0 <= j < n): return
                    visited.add((i, j))
                    for r, c in [(i+1, j), (i-1, j), (i, j+1), (i, j-1)]:
                        findLonelyPixel(r, c)
                result += max(result, len(visited))
        return result

Python
# Function to find the number of lonely black pixels in a picture.
def findLonelyPixel(picture):
    # Get dimensions of the picture
    rows = len(picture)
    cols = len(picture[0]) if rows > 0 else 0
    # Initialize row and column counts with zeros
    rowCount = [0] * rows
    colCount = [0] * cols
    # First pass: Count the number of 'B's in each row and column.
    for i in range(rows):
        for j in range(cols):
            if picture[i][j] == 'B':
                rowCount[i] += 1
                colCount[j] += 1
    # Second pass: Check for lonely black pixels.
    lonelyPixels = 0
    for i in range(rows):
        for j in range(cols):
            if picture[i][j] == 'B' and rowCount[i] == 1 and colCount[j] == 1:
                lonelyPixels += 1
    return lonelyPixels
```

```
# Example usage:
picture = [[“W”,“W”,“B”],[“W”,“B”,“W”],[“B”,“W”,“W”]]
print(findLonelyPixel(picture)) # Output: 3

Python
class Solution:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        m = len(picture)
        n = len(picture[0])
        ans = 0
        rows = [0] * m # rows[i] := the number of B's in row i
        cols = [0] * n # cols[i] := the number of B's in col i
        for i in range(m):
            for j in range(n):
                if picture[i][j] == 'B':
                    rows[i] += 1
                    cols[j] += 1
        for i in range(m):
            if rows[i] == 1: # Only have to examine the rows if rows[i] == 1.
                for j in range(n):
                    # After meeting a 'B' in this row, break and search the next row.
                    if picture[i][j] == 'B':
                        if cols[j] == 1:
                            ans += 1
                            break
        return ans

Python
class Solution:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        rows = [0] * len(picture)
        cols = [0] * len(picture[0])
        for i, row in enumerate(picture):
            for j, x in enumerate(row):
                if x == 'B':
                    rows[i] += 1
                    cols[j] += 1
        ans = 0
        for i, row in enumerate(picture):
            for j, x in enumerate(row):
                if x == 'B' and rows[i] == 1 and cols[j] == 1:
                    ans += 1
        return ans
```

Python

```

class Solution {
    def findMaximalPixel(int[] picture: List<List<Int>[]>): Int {
        m = picture.size(), len(picture[0])
        rows, cols = [0] * m, [0] * n
        for i in range(m):
            for j in range(n):
                if picture[i][j] == 'B':
                    rows[i] += 1
                    cols[j] += 1
        res = 0
        for i in range(m):
            if rows[i] == 1:
                for j in range(n):
                    if picture[i][j] == 'B' and cols[j] == 1:
                        res += 1
                        break
        return res
    }
}

```

C++

```

class Solution {
public:
    int findMaximalPixel(vector<vector<char>>& picture) {
        const int m = picture.size();
        const int n = picture[0].size();
        int ans = 0;
        vector<int> rows(m); // rows[i] := the number of B's in row i
        vector<int> cols(n); // cols[j] := the number of B's in col j

        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B') {
                    ++rows[i];
                    ++cols[j];
                }

        for (int i = 0; i < m; ++i)
            if (rows[i] == 1) // Only have to examine the rows if rows[i] == 1.
                for (int j = 0; j < n; ++j)
                    // After examining j == 0 in this row, break and search the next row.
                    if (picture[i][j] == 'B') {
                        ++ans;
                        break;
                    }

        return ans;
    }
};

```

C++

Matrix · LeetCode · By Pattern — 1681 — LeetCode

```

class Solution {
public:
    int findMaximalPixel(vector<vector<char>>& picture) {
        int m = picture.size(), n = picture[0].size();
        vector<int> rows(m);
        vector<int> cols(n);
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B') {
                    ++rows[i];
                    ++cols[j];
                }
        int ans = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B' && rows[i] == 1 && cols[j] == 1) {
                    ++ans;
                }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int findMaximalPixel(vector<vector<char>>& picture) {
        int m = picture.size(), n = picture[0].size();
        vector<int> rows(m);
        vector<int> cols(n);
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B') {
                    ++rows[i];
                    ++cols[j];
                }
        int res = 0;
        for (int i = 0; i < m; ++i)
            if (rows[i] == 1) {
                for (int j = 0; j < n; ++j)
                    if (picture[i][j] == 'B' && cols[j] == 1) {
                        ++res;
                        break;
                    }
            }
        return res;
    }
};

```

Java

Matrix · LeetCode · By Pattern — 1682 — LeetCode

Java

```

class Solution {
    public int findMaximalPixel(char[][] picture) {
        final int m = picture.length;
        final int n = picture[0].length;
        int[] rows = new int[m];
        int[] cols = new int[n];
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B') {
                    ++rows[i];
                    ++cols[j];
                }

        for (int i = 0; i < m; ++i)
            if (rows[i] == 1) // Only have to examine the rows if rows[i] == 1.
                for (int j = 0; j < n; ++j)
                    // After examining j == 0 in this row, break and search the next row.
                    if (picture[i][j] == 'B') {
                        ++ans;
                        break;
                    }

        return ans;
    }
}

```

Java

```

class Solution {
    public int findMaximalPixel(char[][] picture) {
        int m = picture.length, n = picture[0].length;
        int[] rows = new int[m];
        int[] cols = new int[n];
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B') {
                    ++rows[i];
                    ++cols[j];
                }
        int ans = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B' && rows[i] == 1 && cols[j] == 1) {
                    ++ans;
                }
        return ans;
    }
}

```

Java

Matrix · LeetCode · By Pattern — 1683 — LeetCode

```

class Solution {
    public int findMaximalPixel(char[][] picture) {
        int m = picture.length, n = picture[0].length;
        int[] rows = new int[m];
        int[] cols = new int[n];
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (picture[i][j] == 'B') {
                    ++rows[i];
                    ++cols[j];
                }
        int res = 0;
        for (int i = 0; i < m; ++i)
            if (rows[i] == 1) {
                for (int j = 0; j < n; ++j)
                    if (picture[i][j] == 'B' && cols[j] == 1) {
                        ++res;
                        break;
                    }
            }
        return res;
    }
}

```

TypeScript

```

function findMaximalPixel(picture: string[][]): number {
    const m = picture.length;
    const n = picture[0].length;
    const rows: number[] = Array(m).fill(0);
    const cols: number[] = Array(n).fill(0);
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            if (picture[i][j] === 'B') {
                ++rows[i];
                ++cols[j];
            }
    let ans = 0;
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            if (picture[i][j] === 'B' && rows[i] === 1 && cols[j] === 1) {
                ++ans;
            }
    return ans;
}

```

TypeScript

Matrix · LeetCode · By Pattern — 1684 — LeetCode

```

function findMaximalPixel(picture: string[][]): number {
    const m = picture.length;
    const n = picture[0].length;
    const rows: number[] = Array(m).fill(0);
    const cols: number[] = Array(n).fill(0);
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            if (picture[i][j] === 'B') {
                ++rows[i];
                ++cols[j];
            }
    let ans = 0;
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j)
            if (picture[i][j] === 'B' && rows[i] === 1 && cols[j] === 1) {
                ++ans;
            }
    return ans;
}

```

Go

```

func findMaximalPixel(picture [][]byte) (ans int) {
    rows := make([]int, len(picture))
    cols := make([]int, len(picture[0]))
    for i, row := range picture {
        for j, x := range row {
            if x == 'B' {
                rows[i]++
                cols[j]++
            }
        }
    }
    for i, row := range picture {
        for j, x := range row {
            if x == 'B' && rows[i] == 1 && cols[j] == 1 {
                ans++
            }
        }
    }
    return
}

```

Go

Matrix · LeetCode · By Pattern — 1685 — LeetCode

```

func findMaximalPixel(picture [][]byte) int {
    m, n := len(picture), len(picture[0])
    rows := make([]int, m)
    cols := make([]int, n)
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if picture[i][j] == 'B' {
                rows[i]++
                cols[j]++
            }
        }
    }
    res := 0
    for i := 0; i < m; i++ {
        if rows[i] == 1 {
            for j := 0; j < n; j++ {
                if picture[i][j] == 'B' && cols[j] == 1 {
                    res++
                    break
                }
            }
        }
    }
    return res
}

```

[*] Matrix — Pattern Solution (matched from community answers)

Python

```

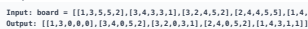
class Solution:
    def findMaximalPixel(self, picture: List<List<str>[]>) -> int:
        m, n = len(picture), len(picture[0])
        rows, cols = [0] * m, [0] * n
        for i in range(m):
            for j in range(n):
                if picture[i][j] == 'B':
                    rows[i] += 1
                    cols[j] += 1
        res = 0
        for i in range(m):
            if rows[i] == 1:
                for j in range(n):
                    if picture[i][j] == 'B' and cols[j] == 1:
                        res += 1
                        break
        return res

```

#723 MEDIUM
Candy Crush
 LeetCode · Simulation · WUJICC · LeetCode · Editorial
 Array · Two Pointers · Matrix · Simulation
 Little Premium 98
Problem:

Matrix · LeetCode · By Pattern — 1686 — LeetCode

Example 1:



```

Python
class Solution:
    def canConstruct(self, board: List[List[str]]) -> List[List[str]]:
        m, n = len(board), len(board[0])
        run = True
        while run:
            run = False
            for i in range(m):
                for j in range(n - 2):
                    if (
                        board[i][j] != 0
                        and abs(board[i][j] + 1)
                        and abs(board[i][j] + 2)
                    ):
                        run = True
                        board[i][j] = board[i][j] + 1
                        board[i][j] = board[i][j] + 2
                        board[i][j]
            for j in range(n):
                for i in range(m - 2):
                    if (
                        board[i][j] != 0
                        and abs(board[i][j] + 1)
                        and abs(board[i] + 2)
                    ):
                        run = True

```

```

class Solution {
public:
    vector<vector<int>> candyCrush(vector<vector<int>>& board) {
        int n = board.size(), m = board[0].size();
        bool run = true;
        while (run) {
            run = false;
            for (int i = 0; i < n; ++i) {
                if (board[i][0] != 0 && abs(board[i][1]) == abs(board[i][2]) && abs(board[i][3]) == abs(board[i][4]) && abs(board[i][5]) > 0) {
                    run = true;
                    board[i][1] = -board[i][1];
                    board[i][2] = -board[i][2];
                    board[i][3] = -board[i][3];
                    board[i][4] = -board[i][4];
                }
            }
            for (int j = 0; j < m; ++j) {
                if (board[0][j] != 0 && abs(board[1][j]) == abs(board[2][j]) && abs(board[3][j]) == abs(board[4][j]) && abs(board[5][j]) > 0) {
                    run = true;
                    board[1][j] = -board[1][j];
                    board[2][j] = -board[2][j];
                    board[3][j] = -board[3][j];
                    board[4][j] = -board[4][j];
                }
            }
            if (run) {
                for (int i = 0; i < n; ++i) {
                    if (board[i][0] != 0 && abs(board[i][1]) == abs(board[i][2]) && abs(board[i][3]) == abs(board[i][4]) && abs(board[i][5]) > 0) {
                        board[i][0] = 0;
                    }
                }
                for (int j = 0; j < m; ++j) {
                    if (board[0][j] != 0 && abs(board[1][j]) == abs(board[2][j]) && abs(board[3][j]) == abs(board[4][j]) && abs(board[5][j]) > 0) {
                        board[0][j] = 0;
                    }
                }
            }
        }
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                if (board[i][j] == 0) {
                    int k = i + 1;
                    while (k < n && board[k][j] == 0) {
                        k++;
                    }
                    if (k < n) {
                        swap(board[i][j], board[k][j]);
                    }
                }
            }
        }
    }
};

```

100

```

class Solution {
public:
    vector<vector<int>> candyCrush(vector<vector<int>>& board) {
        const int n = board.size();
        const int m = board[0].size();
        bool hasCrushes = true;

        while (hasCrushes) {
            hasCrushes = false;

            for (int i = 0; i < n; ++i)
                for (int j = 0; j < m; ++j) {
                    const int val = abs(board[i][j]);
                    if (val == 0)
                        continue;

                    // Check the vertical candies.
                    if (i + 2 < n && abs(board[i][j] + 1) == val &&
                        abs(board[i+1][j] + 1) == val) {
                        board[i] = -val;
                        board[i+1] = -val;
                        board[i+2] = -val;
                    }

                    // Check the horizontal candies.
                    if (i + 2 < n && abs(board[i][j + 1]) == val &&
                        abs(board[i][j+2]) == val) {
                        board[i][j] = -val;
                        board[i][j+1] = -val;
                        board[i][j+2] = -val;
                    }
                }

            hasCrushes = true;
        }

        return board;
    }
};

```

```

    n = board.length;
    final int = board[0].length;
    boolean haveCrushes = true;

    while (haveCrushes) {
        haveCrushes = false;

        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) {
                final int val = Math.abs(board[i][j]);
                if (val < 8)
                    continue;

                // check the vertical column.
                if (i < 2 && val >= Math.abs(board[i+1][j] + 1)) == val && Math.abs(board[i][j+2]) == val() {
                    haveCrushes = true;
                    for (int k = j; k < j + 3; k++)
                        board[i][k] = -val;
                }

                // check the horizontal row.
                if (i < 2 && val >= Math.abs(board[i+1][j+1]) == val && Math.abs(board[i+2][j]) == val() {
                    haveCrushes = true;
                    for (int k = i; k < i + 3; k++)
                        board[k][j] = -val;
                }
            }
    }
}

```

Matrix · LeetMastery — By Pattern — 1693 — LeetMastery

Matrix · LeetCode — By Pattern — 1694 — LeetCode

Matrix · LeetCode — By Pattern — 1695 — LeetCode

Matrix · LeetCode — By Pattern — 1696 — LeetCode

Matrix · LeetCode · By Pattern — 1697 — LeetCode

Matrix · LeetCode — By Pattern — 1698 — LeetCode

Problem:

You are given two images, `img1` and `img2`, represented as binary, square matrices of size `n x n`. A binary matrix has only 0s and 1s as values. We translate one image however we choose by sliding all the 1 bits left, right, up, and/or down any number of units. We then place it on top of the other image. We can then calculate the overlap by counting the number of positions that have a 1 in both images.

Note also that a translation does not include any kind of rotation. Any 1 bits that are translated outside of the matrix borders are erased.

Return the largest possible overlap.

Example 1:

1	1	0
0	1	0
0	1	0

img1

0	0	0
0	1	1
0	0	1

img2

Input: `img1 = [[1,1,0],[0,1,0],[0,0,0]]`, `img2 = [[0,0,0],[0,1,1],[0,0,1]]`
Output: 3
Explanation: We translate `img1` to right by 1 unit and down by 1 unit.

The number of positions that have a 1 in both images is 3 (shown in red).

Example 2:

Input: `img1 = [[1]], img2 = [[1]]`

Output: 1

Example 3:

Input: `img1 = [[0]], img2 = [[0]]`

Output: 0

Constraints:

- `n == img1.length == img1[0].length`
- `n == img2.length == img2[0].length`
- `1 <= n <= 30`

- `img1[i][j]` is either 0 or 1.
- `img2[i][j]` is either 0 or 1.

>> Brute Force (Matrix) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) -> int:
        # Brute force approach: try all possible translations of img1 over img2
        n = len(img1)
        result = 0
        for r1 in range(n):
            for c1 in range(n):
                visited = set()
                for r2 in range(n):
                    for c2 in range(n):
                        if (r1, c1) in visited or not (0 <= r1-r2 and 0 <= c1-c2):
                            continue
                        visited.add((r1, c1))
                        for dr, dc in [(0,1), (0,-1), (1,0), (-1,0)]:
                            r = r1 + dr
                            c = c1 + dc
                            if 0 <= r < n and 0 <= c < n:
                                result = max(result, result + 1)
                return result
```

Python

```
Python
def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) -> int:
    # Get the dimensions of the square images
    n = len(img1)

    # Gather positions (coordinates) where there is a 1 in img1 and img2
    ones_img1 = []
    ones_img2 = []
    for i in range(n):
        for j in range(n):
            if img1[i][j] == 1:
                ones_img1.append((i, j))
            if img2[i][j] == 1:
                ones_img2.append((i, j))

    # Dictionary to count translation shifts that align 1's of img1 with img2
    translation_count = {}
    max_overlap = 0

    # Iterate over every pair of 1's from both images
    for (i1, j1) in ones_img1:
        for (i2, j2) in ones_img2:
            # Calculate the translation vector needed to align img1's 1 with img2's 1
            delta = (i2 - i1, j2 - j1)
            # Update the count for this translation vector
            translation_count[delta] = translation_count.get(delta, 0) + 1
```

```
# Iterate over every pair of 1's from both images
max_overlap = max(translation_count.get(delta, 0) for delta in translation_count)

return max_overlap

# Example usage:
img1 = [[1,1,0],[0,1,0],[0,0,0]]
img2 = [[0,0,0],[0,1,1],[0,0,1]]
print(largestOverlap(img1, img2)) # Expected output: 3
```

Python

```
class Solution:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) -> int:
        n = len(img1)
        ones_img1 = []
        ones_img2 = []
        for i in range(n):
            for j in range(n):
                if img1[i][j] == 1:
                    ones_img1.append((i, j))
                if img2[i][j] == 1:
                    ones_img2.append((i, j))

        # Dictionary to count translation shifts that align 1's of img1 with img2
        translation_count = {}
        max_overlap = 0

        # Iterate over every pair of 1's from both images
        for (i1, j1) in ones_img1:
            for (i2, j2) in ones_img2:
                # Calculate the translation vector needed to align img1's 1 with img2's 1
                delta = (i2 - i1, j2 - j1)
                # Update the count for this translation vector
                translation_count[delta] = translation_count.get(delta, 0) + 1

        return max(translation_count.values()) if translation_count else 0
```

Python

```
class Solution:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) -> int:
        n = len(img1)
        cnt = Counter()
        for i in range(n):
            for j in range(n):
                if img1[i][j] == 1:
                    for k in range(n):
                        if img2[i+k][j-k] == 1:
                            cnt[(i-k, j-k)] += 1
        return max(cnt.values()) if cnt else 0
```

Python

```
class Solution:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) -> int:
        n = len(img1)
        cnt = Counter()
        for i in range(n):
            for j in range(n):
                if img1[i][j] == 1:
                    for k in range(n):
                        if img2[i+k][j-k] == 1:
                            cnt[(i-k, j-k)] += 1
        return max(cnt.values()) if cnt else 0
```

```
C++
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        const int N = img1.size();
        int ans = 0;
        vector<pair<int, int>> ones1;
        vector<pair<int, int>> ones2;
        unordered_map<int, int> offsetCount;

        for (int i = 0; i < N; ++i)
            for (int j = 0; j < N; ++j) {
                if (img1[i][j] == 1)
                    ones1.emplace_back(i, j);
                if (img2[i][j] == 1)
                    ones2.emplace_back(i, j);
            }

        for (const auto& [x1, y1] : ones1)
            for (const auto& [x2, y2] : ones2)
                ++offsetCount[(x2 - x1) * N + (y2 - y1)];

        for (const auto& [_, count] : offsetCount)
            ans = max(ans, count);

        return ans;
    }
};
```

C++

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

Java

Java

```
class Solution {
public:
    int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
        int n = img1.size();
        map<pair<int, int>, int> cnt;
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j) {
                if (img1[i][j] == 1)
                    for (int h = 0; h < n; ++h) {
                        for (int k = 0; k < n; ++k) {
                            if (img2[i+h][j+k] == 1)
                                ans = max(ans, ++cnt[(i-h) * n + (j-k)]);
                        }
                    }
            }
        return ans;
    }
};
```

Java


```
function largestOverlap(img1: number[][], img2: number[][]) : number {
    const n = img1.length;
    const cnt = new number;
    let ans = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            if (img1[i][j] == 1) {
                for (let k = 0; k < n; k++) {
                    for (let h = 0; h < n; h++) {
                        if (img2[h][k] == 1) {
                            const t = (i - h) + 200 + (j - k);
                            cnt.set(t, cnt.get(t) ?? 0 + 1);
                            ans = Math.max(ans, cnt.get(t));
                        }
                    }
                }
            }
        }
    }
    return ans;
}
```

Go

```
func largestOverlap(img1 []][int, img2 []][int) (ans int) {
    type pair struct{ x, y int }
    cnt = map[pair]int{}
    for i, row1 := range img1 {
        for j, x1 := range row1 {
            if x1 == 1 {
                for h, row2 := range img2 {
                    for k, x2 := range row2 {
                        if x2 == 1 {
                            t := pair{i - h, j - k}
                            cnt[t]++
                            ans = max(ans, cnt[t])
                        }
                    }
                }
            }
        }
    }
    return
}
```

Go

Matrix · LeetCode · By Pattern

— 1705 —

LeetCode

```
func largestOverlap(img1 []][int, img2 []][int) (ans int) {
    type pair struct{ x, y int }
    cnt = map[pair]int{}
    for i, row1 := range img1 {
        for j, x1 := range row1 {
            if x1 == 1 {
                for h, row2 := range img2 {
                    for k, x2 := range row2 {
                        if x2 == 1 {
                            t := pair{i - h, j - k}
                            cnt[t]++
                            ans = max(ans, cnt[t])
                        }
                    }
                }
            }
        }
    }
    return
}
```

[*] Matrix — Pattern Solution (stored optimal)

Python

```
def largestOverlap(img1: List[List[int]], img2: List[List[int]]) -> int:
    # Get the dimensions of the square images
    n = len(img1)

    # Gather positions (coordinates) where there is a 1 in img1 and img2
    ones_img1 = []
    ones_img2 = []

    for i in range(n):
        for j in range(n):
            if img1[i][j] == 1:
                ones_img1.append((i, j))
            if img2[i][j] == 1:
                ones_img2.append((i, j))

    # Dictionary to count translation shifts that align 1's of img1 with img2
    translation_count = {}
    max_overlap = 0

    # Iterate over every pair of 1's from both images
    for (x1, y1) in ones_img1:
        for (x2, y2) in ones_img2:
            # Calculate the translation vector needed to align img1's 1 with img2's 1
            delta = (x2 - x1, y2 - y1)
            # Update the count for this translation vector
            translation_count[delta] = translation_count.get(delta, 0) + 1
```

Matrix · LeetCode · By Pattern

— 1706 —

LeetCode

```
# Update max_overlap if we find a higher count
max_overlap = max(max_overlap, translation_count[delta])

return max_overlap

# Example usage:
img1 = [[1,1,0],[0,1,0],[0,0,0]]
img2 = [[0,0,0],[0,1,1],[0,0,1]]
print(largestOverlap(img1, img2)) # Expected output: 3

C++
// C++ implementation of the target overlap problem
#include <vector>
#include <unordered_map>
#include <algorithm>
#include <iostream>
using namespace std;

int largestOverlap(vector<vector<int>>& img1, vector<vector<int>>& img2) {
    int n = img1.size();
    // Vectors to store positions of 1's in each image
    vector<pair<int, int>> onesimg1, onesimg2;

    // Iterate positions of 1's for both images
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (img1[i][j] == 1)
                onesimg1.push_back({i, j});
            if (img2[i][j] == 1)
                onesimg2.push_back({i, j});
        }
    }

    // Use an unordered map to count frequency of translation vectors.
    unordered_map<string, int> translationCount;
```

Matrix · LeetCode · By Pattern

— 1707 —

LeetCode

```
int maxOverlap = 0;

// Check each pair of positions, one from each image
for (auto [x1, y1] : onesimg1) {
    for (auto [x2, y2] : onesimg2) {
        int dx = x2 - x1;
        int dy = y2 - y1;
        // Create a unique key for the translation vector
        string key = to_string(dx) + "," + to_string(dy);
        translationCount[key]++;
        maxOverlap = max(maxOverlap, translationCount[key]);
    }
}

return maxOverlap;

int main() {
    // Example usage:
    vector<vector<int>> img1 = {{1,1,0},{0,1,0},{0,1,0}};
    vector<vector<int>> img2 = {{0,0,0},{0,1,1},{0,0,1}};
    cout << largestOverlap(img1, img2) << endl; // Expected output: 3
    return 0;
}
```

#1198 MEDIUM

Find Smallest Common Element in All Rows

LeetCode · SimplyList · WalkCC · LeetCode · Editorial

Array · Hash Table · Binary Search · Matrix · Counting

Like · Premium 98

Problem:

Given an $n \times n$ matrix `mat` where every row is sorted in strictly increasing order, return the smallest common element in all rows.

If there is no common element, return `-1`.

Example 1:

Input: `mat = [[1,2,3,4,5],[2,4,5,6,10],[3,5,7,9,11],[1,3,5,7,9]]`

Output: 5

Example 2:

Input: `mat = [[1,2,3],[2,3,4],[2,3,5]]`

Output: 2

Constraints:

- $n == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 500$
- $1 \leq \text{mat}[i][j] \leq 104$
- `mat[i]` is sorted in strictly increasing order.

Matrix · LeetCode · By Pattern

— 1710 —

LeetCode

```
>> Brute Force [Matrix] Interview Prep

Python
# Brute Force Approach
class Solution:
    def smallestCommonElement(self, mat):
        # Brute Force O(N^2) - Flood fill from every cell
        m, n = len(mat), len(mat[0])
        result = 0
        for r in range(m):
            for c in range(n):
                visited = set()
                def flood(r, c):
                    if (r, c) in visited or not (0 <= r < m and 0 <= c < n): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: flood(r+dr, c+dc)
                flood(r, c)
                result = max(result, len(visited))
        return result
```

Python

```
Python
def smallestCommonElement(mat):
    # Get number of rows and columns
    m = len(mat)
    n = len(mat[0])

    # Helper function: binary search for target in a row
    def binary_search(row, target):
        left, right = 0, n - 1
        while left <= right:
            mid = (left + right) // 2
            if mat[row][mid] == target:
                return True
            elif mat[row][mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return False

    # Iterate over each candidate in the first row
    for candidate in mat[0]:
        is_common = True
        # Check each row for the candidate using binary search
        for i in range(1, m):
            if not binary_search(mat[i], candidate):
                is_common = False
                break
        if is_common:
            return candidate
    return -1

# Example usage:
print(smallestCommonElement([[1,2,3,4,5],[2,4,5,6,10],[3,5,7,9,11],[1,3,5,7,9]]))

Python
```

Matrix · LeetCode · By Pattern

— 1709 —

LeetCode

```
class Solution:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        m = len(mat)
        count = [0] * (MAX + 1)

        for row in mat:
            for a in row:
                count[a] += 1
            if count[a] == len(mat):
                return a
        return -1
```

Python

```
class Solution:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        cnt = Counter()
        for row in mat:
            for a in row:
                cnt[a] += 1
            if cnt[a] == len(mat):
                return a
        return -1
```

Python

```
class Solution:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        cnt = Counter()
        for row in mat:
            for a in row:
                cnt[a] += 1
            if cnt[a] == len(mat):
                return a
        return -1
```

C++

```
class Solution {
public:
    int smallestCommonElement(vector<vector<int>>& mat) {
        const int MAX = 10000;
        vector<int> count(MAX + 1);

        for (const vector<int>& row : mat)
            for (const int a : row)
                if (++count[a] == mat.size())
                    return a;

        return -1;
    }
};
```

C++

Matrix · LeetCode · By Pattern

— 1710 —

LeetCode

```

class Solution {
public:
    int smallestCommonElement(vector<vector<int>>& mat) {
        int col[10001]{};
        for (const row : mat) {
            for (int x : row) {
                if (++col[x] == mat.size()) {
                    return x;
                }
            }
        }
        return -1;
    }
};

```

C++

```

class Solution {
public:
    int smallestCommonElement(vector<vector<int>>& mat) {
        int col[10001]{};
        for (const row : mat) {
            for (int x : row) {
                if (++col[x] == mat.size()) {
                    return x;
                }
            }
        }
        return -1;
    }
};

```

Java

```

class Solution {
    public int smallestCommonElement(int[][] mat) {
        final int MAX = 3000;
        int[] count = new int[MAX + 1];

        for (int[] row : mat)
            for (final int x : row)
                if (++count[x] == mat.length)
                    return x;

        return -1;
    }
}

```

Java

Matrix · LeetCode · By Pattern

— 1711 —

LeetCode

```

class Solution {
    public int smallestCommonElement(int[][] mat) {
        int[] cnt = new int[10001];
        for (var row : mat) {
            for (int x : row) {
                if (++cnt[x] == mat.length) {
                    return x;
                }
            }
        }
        return -1;
    }
}

```

Java

```

class Solution {
    public int smallestCommonElement(int[][] mat) {
        int[] cnt = new int[10001];
        for (var row : mat) {
            for (int x : row) {
                if (++cnt[x] == mat.length) {
                    return x;
                }
            }
        }
        return -1;
    }
}

```

TypeScript

```

function smallestCommonElement(mat: number[][]): number {
    const cnt: number[] = new Array(10001).fill(0);
    for (const row of mat) {
        for (const x of row) {
            if (++cnt[x] === mat.length) {
                return x;
            }
        }
    }
    return -1;
}

```

TypeScript

```

function smallestCommonElement(mat: number[][]): number {
    const cnt: number[] = new Array(10001).fill(0);
    for (const row of mat) {
        for (const x of row) {
            if (++cnt[x] === mat.length) {
                return x;
            }
        }
    }
    return -1;
}

```

Matrix · LeetCode · By Pattern

— 1712 —

LeetCode

```

Go
func smallestCommonElement(mat [][]int) int {
    col := [10001]int{}
    for _, row := range mat {
        for _, x := range row {
            col[x]++
            if col[x] == len(mat) {
                return x
            }
        }
    }
    return -1
}

```

```

Go
func smallestCommonElement(mat [][]int) int {
    col := [10001]int{}
    for _, row := range mat {
        for _, x := range row {
            col[x]++
            if col[x] == len(mat) {
                return x
            }
        }
    }
    return -1
}

```

[*] Matrix — Pattern Solution (stored optimal)

Python

Matrix · LeetCode · By Pattern

— 1713 —

LeetCode

```

def smallestCommonElement(mat):
    # Get number of rows and columns
    m = len(mat)
    n = len(mat[0])

    # Helper function: binary search for target in a row
    def binary_search(row, target):
        left, right = 0, n - 1
        while left < right:
            mid = (left + right) // 2
            if row[mid] == target:
                return True
            elif row[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return False

    # Iterate over each candidate in the first row
    for candidate in mat[0]:
        is_common = True
        # Check each remaining row for the candidate using binary search
        for i in range(1, m):
            if not binary_search(mat[i], candidate):
                is_common = False
                break
        if is_common:
            return candidate
    return -1

# Example usage:
print(smallestCommonElement([[1,2,3,4,5],[2,4,5,6,10],[3,5,7,9,11],[1,3,5,7,9]]))

```

C++

Matrix · LeetCode · By Pattern

— 1714 —

LeetCode

```

#include <iostream>
#include <vector>
using namespace std;

// Binary search helper function to check if target exists in the row
bool binarySearch(const vector<int>& row, int target) {
    int left = 0, right = row.size() - 1;
    while (left < right) {
        int mid = (left + right) / 2;
        if (row[mid] == target)
            return true;
        else if (row[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return false;
}

int smallestCommonElement(vector<vector<int>>& mat) {
    int n = mat.size();
    int m = mat[0].size();

    // Iterate over each candidate in the first row
    for (int candidate : mat[0]) {
        bool isCommon = true;
        // Use binary search in every other row
        for (int i = 1; i < n; i++) {
            if (!binarySearch(mat[i], candidate)) {
                isCommon = false;
                break;
            }
        }
        if (isCommon)
            return candidate;
    }
    return -1;
}

int main() {
    vector<vector<int>> mat = {
        {1,2,3,4,5},
        {2,4,5,6,10},
        {3,5,7,9,11},
        {1,3,5,7,9}
    };

    cout << smallestCommonElement(mat) << endl;
    return 0;
}

```

#1074

HARD

Number of Submatrices That Sum to Target

LeetCode · SimplifyLess · WalkCC · LeetCode · Editorial

Array · Hash Table · Matrix · Prefix Sum

Matrix · LeetCode · By Pattern

— 1715 —

LeetCode

Lists Denny Zhang

Problem:

Given a matrix and a target, return the number of non-empty submatrices that sum to target.

A submatrix $x1, y1, x2, y2$ is the set of all cells $matrix[x][y]$ with $x1 \leq x \leq x2$ and $y1 \leq y \leq y2$.

Two submatrices $(x1, y1, x2, y2)$ and $(x1', y1', x2', y2')$ are different if they have some coordinate that is different for example, if $x1 \neq x1'$.

Example 1:

0	1	0
1	1	1
0	1	0

Input: matrix = [[0,1,0],[1,1,1],[0,1,0]], target = 0

Output: 4

Explanation: The four 1x1 submatrices that only contain 0.

Example 2:

Input: matrix = [[1,-1],[-1,1]], target = 0

Output: 5

Explanation: The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2 submatrix.

Example 3:

Input: matrix = [[904]], target = 0

Output: 0

Constraints:

- $1 \leq \text{matrix.length} \leq 100$
- $1 \leq \text{matrix}[0].\text{length} \leq 100$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$
- $-10^4 \leq \text{target} \leq 10^4$

>> Brute Force [Matrix] Interview Prep

Python

Matrix · LeetCode · By Pattern

— 1716 —

LeetCode

```
# Brute Force Approach
class Solution:
    def numSubmatrixSumTarget(self, matrix, target):
        # Brute Force O(n^4 * 2) - Flood fill from every cell
        m, n = len(matrix), len(matrix[0])
        result = 0
        for r in range(m):
            for c in range(n):
                visited = set()
                def floodfill(r, c):
                    if (r, c) in visited or not (0 <= r < m and 0 <= c < n): return
                    visited.add((r, c))
                    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]: floodfill(r+dr, c+dc)
                floodfill(r, c)
                result += sum(result, len(visited))
            return result

# Python
Python

# Python solution for counting number of submatrices that sum to target
def numSubmatrixSumTarget(matrix, target):
    # Dimensions of the matrix
    m = len(matrix)
    n = len(matrix[0])
    result = 0

    # Loop over all possible starting rows
    for top in range(m):
        # Initialize a list to store the column sums between row 'top' and 'bottom'
        col_sums = [0] * n

        # Extend the submatrix from row 'top' downwards to 'bottom'
        for bottom in range(top, m):
            # Summation column sums for the current bottom row
            for col in range(n):
                col_sums[col] += matrix[bottom][col]

            # Use a dictionary to count cumulative sum in the 2D array (col_sums)
            sum_count = {}
            curr_sum = 0
            # Iterate over each column in col_sums to find the number of subarrays with sum equal to target
            for value in col_sums:
                curr_sum += value

                if (curr_sum - target) in sum_count:
                    result += sum_count[curr_sum - target]
                # Update the dictionary with the current sum count
                sum_count[curr_sum] = sum_count.get(curr_sum, 0) + 1

            return result

# Example usage:
if __name__ == "__main__":
    matrix = [[0, 1, 0], [1, 1, 1], [0, 1, 0]]
    target = 0
    print(numSubmatrixSumTarget(matrix, target)) # Expected Output: 4
```

Matrix · LeetCode · By Pattern

— 1717 —

LeetCode

```
Python
class Solution:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:
        m = len(matrix)
        n = len(matrix[0])
        ans = 0

        # Transfer each row in the matrix to the prefix sum.
        for row in matrix:
            for i in range(n):
                row[i] += row[i - 1]

        for baseCol in range(n):
            for j in range(baseCol, n):
                prefixCount = collections.Counter([0, 1])
                sums = 0
                for i in range(m):
                    if baseCol > 0:
                        sums += matrix[i][baseCol - 1]
                    sums += matrix[i][j]
                    ans += prefixCount[sums - target]
                prefixCount[sums] += 1

            return ans

Python
class Solution:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:
        def f(nums: List[int]) -> int:
            d = defaultdict(int)
            d[0] = 1
            cnt = s = 0
            for x in nums:
                s += x
                cnt += d[s - target]
                d[s] += 1
            return cnt

        m, n = len(matrix), len(matrix[0])
        ans = 0
        for i in range(m):
            col = [0] * n
            for j in range(m, n):
                for k in range(i):
                    col[k] += matrix[i][j]
                ans += f(col)
            return ans
```

Python

Matrix · LeetCode · By Pattern

— 1718 —

LeetCode

```
class Solution {
public:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:
        def f(nums: List[int]) -> int:
            d = defaultdict(int)
            d[0] = 1
            cnt = s = 0
            for x in nums:
                s += x
                cnt += d[s - target]
                d[s] += 1
            return cnt

        m, n = len(matrix), len(matrix[0])
        ans = 0
        for i in range(m):
            col = [0] * n
            for j in range(m, n):
                for k in range(i):
                    col[k] += matrix[i][j]
                ans += f(col)
            return ans

# C++
C++
class Solution {
public:
    int numSubmatrixSumTarget(vector<vector<int>>& matrix, int target) {
        int m = matrix.size(), n = matrix[0].size();
        int ans = 0;
        for (int i = 0; i < m; ++i) {
            vector<int> col(n);
            for (int j = 0; j < n; ++j) {
                for (int k = 0; k < i; ++k) {
                    col[k] += matrix[i][j];
                }
                ans += f(col, target);
            }
        }
        return ans;

        int f(vector<int>& nums, int target) {
            unordered_map<int, int> d{{0, 1}};
            int cnt = 0, s = 0;
            for (int x : nums) {
                s += x;
                if (d.count(s - target)) {
                    cnt += d[s - target];
                }
            }
            ++d[s];
            return cnt;
        }
    };
};

C++
```

Matrix · LeetCode · By Pattern

— 1719 —

LeetCode

```
class Solution {
public:
    int numSubmatrixSumTarget(vector<vector<int>>& matrix, int target) {
        int m = matrix.size(), n = matrix[0].size();
        int ans = 0;
        for (int i = 0; i < m; ++i) {
            vector<int> col(n);
            for (int j = 0; j < n; ++j) {
                for (int k = 0; k < i; ++k) {
                    col[k] += matrix[i][j];
                }
                ans += f(col, target);
            }
        }
        return ans;

        int f(vector<int>& nums, int target) {
            unordered_map<int, int> d{{0, 1}};
            int cnt = 0, s = 0;
            for (int x : nums) {
                s += x;
                if (d.count(s - target)) {
                    cnt += d[s - target];
                }
            }
            ++d[s];
            return cnt;
        }
    };
};

Java
Java
```

Java

Matrix · LeetCode · By Pattern

— 1720 —

LeetCode

```
class Solution {
public:
    int numSubmatrixSumTarget(vector<vector<int>>& matrix, int target) {
        int m = matrix.size(), n = matrix[0].size();
        int ans = 0;
        for (int i = 0; i < m; ++i) {
            vector<int> col(n);
            for (int j = 0; j < n; ++j) {
                for (int k = 0; k < i; ++k) {
                    col[k] += matrix[i][j];
                }
                ans += f(col, target);
            }
        }
        return ans;

        int f(vector<int>& nums, int target) {
            unordered_map<int, int> d{{0, 1}};
            int cnt = 0, s = 0;
            for (int x : nums) {
                s += x;
                if (d.count(s - target)) {
                    cnt += d[s - target];
                }
            }
            ++d[s];
            return cnt;
        }
    };
};

TypeScript
TypeScript
function numSubmatrixSumTarget(matrix: number[][], target: number): number {
    const m = matrix.length;
    const n = matrix[0].length;
    let ans = 0;
    for (let i = 0; i < m; ++i) {
        const col: number[] = new Array(n).fill(0);
        for (let j = 0; j < n; ++j) {
            for (let k = 0; k < i; ++k) {
                col[k] += matrix[i][j];
            }
            ans += f(col, target);
        }
    }
    return ans;

    function f(nums: number[], target: number): number {
        const d: Map<number, number> = new Map();
        d.set(0, 1);
        let cnt = 0;
        let s = 0;
        for (const v of nums) {
            s += v;
            if (d.has(s - target)) {
                cnt += d.get(s - target);
            }
        }
        return cnt;
    }
}
```

Matrix · LeetCode · By Pattern

— 1721 —

LeetCode

```
}
d.set(s, (d.get(s) || 0) + 1);
return cnt;
}

Go
Go
func numSubmatrixSumTarget(matrix [][]int, target int) (ans int) {
    m, n := len(matrix), len(matrix[0])
    for i := 0; i < m; i++ {
        col := make([]int, n)
        for j := 0; j < n; j++ {
            for k := 0; k < i; k++ {
                col[k] += matrix[i][j]
            }
            ans += f(col, target)
        }
    }
    return

    func f(nums []int, target int) (cnt int) {
        d := map[int]int{0: 1}
        s := 0
        for _, v := range nums {
            s += v
            if v, ok := d[s-target]; ok {
                cnt += v
            }
            d[s]++
        }
        return
    }
}
```

Go

Matrix · LeetCode · By Pattern

— 1722 —

LeetCode

```
func numSubmatrixSumTarget(matrix [][]int, target int) (ans int) {
    m, n := len(matrix), len(matrix[0])
    for i := 0; i < m; i++ {
        col := make([]int, n)
        for j := 0; j < n; j++ {
            for k := 0; k < n; k++ {
                col[k] += matrix[i][k]
            }
            ans += f(col, target)
        }
    }
    return
}

func f(nums []int, target int) (cnt int) {
    d := map[int]int{0: 1}
    s := 0
    for _, v := range nums {
        s += v
        if v, ok := d[s-target]; ok {
            cnt += v
        }
        d[s]++
    }
    return
}
```

[1] Matrix — Pattern Solution (stored optimal)

```
Python
# Python solution for counting number of substrings that sum to target

def numSubmatrixSumTarget(matrix, target):
    # Dimensions of the matrix
    m = len(matrix)
    n = len(matrix[0])
    result = 0

    # Loop over all possible starting rows
    for top in range(0, m):
        # Initialize a list to store the column sums between row 'top' and 'bottom'
        col_sums = [0] * n

        # Extend the submatrix from row 'top' downwards to 'bottom'
        for bottom in range(top, m):
            # Update column sums for the current bottom row
            for col in range(0, n):
                col_sums[col] += matrix[bottom][col]

            # Use a dictionary to count cumulative sums in the 1D array (col_sums)
            sum_count = {0: 1}
            curr_sum = 0
            # Iterate over each column in col_sums to find the number of subarrays with sum equal to target
            for value in col_sums:
                curr_sum += value
```

```
for (let sum = 0; sum < colSum) {
    currSum += sum;
    if (sumCount[currSum - target]) {
        result += sumCount[currSum - target];
    }
    sumCount[currSum]++;
}
}
return result;
}
};

// Example usage:
// let mat = [
//   [0, 1, 0],
//   [1, 1, 1],
//   [0, 1, 0]
// ];
// let res = numSubmatrixSumTarget(mat, 0); // Expected Output: 4
// console.log(res);
```

Solution

The problem is solved by iterating through the matrix starting from the element at position (1, 1). For every element at matrix[i][j], compare it with the element at matrix[i-1][j-1]. If any two elements in this pair do not match, the matrix is not Toeplitz. This check ensures that every diagonal from the top-left to bottom-right has identical elements. In terms of auxiliary space, only constant extra space is used, which makes this approach efficient both in time and space.

#867 Transpose Matrix [Easy]

Key Insights

- The value at position (i, j) in the original matrix moves to (j, i) in the transposed matrix.
- The transposed matrix dimensions are the reverse of the original: if the original is m x n, then the transposed is n x m.
- A simple double loop is sufficient to swap the indices, ensuring O(m*n) time complexity.
- This problem handles both square and rectangular matrices.

Space & Time

Time Complexity: O(m * n)

Space Complexity: O(m * n)

n) for the new matrix holding the transposed values

Solution

The solution involves iterating over each element of the original matrix and assigning it to the corresponding position in the new matrix where the indices are swapped. We create the transposed matrix with dimensions based on the number of columns and rows of the original matrix, respectively. The approach leverages basic array manipulation with nested loops to fill in the new matrix.

#2373 Largest Local Values in a Matrix [Easy]

Key Insights

- The size of the output matrix is (n - 2) x (n - 2).
- For each valid index (i, j) in the output, examine the 3 x 3 block in grid starting at grid[i][j] to grid[i+2][j+2].
- Since each 3 x 3 block contains exactly 9 elements, finding the maximum in a block takes constant time.
- Use nested loops to slide over the grid and efficiently compute each local maximum.

Space & Time

Time Complexity: O(n^2) - We iterate over (n-2) x (n-2) positions, and for each position, we look at 9 elements.

Space Complexity: O(n^2) - The additional space is required for the output matrix.

Solution

The solution involves iterating over each possible 3 x 3 submatrix in grid. For each (i, j) starting index of the submatrix:

- Examine the nine elements from grid[i][j] to grid[i+2][j+2]. Compute the maximum value from these elements.
- Store the maximum in the corresponding position of the result matrix. Data structures used include the result matrix for storing the maximum values. The algorithmic approach involves two nested loops (one for rows and one for columns) to traverse each possible starting position of a 3 x 3 window. The simplicity of the solution comes from the fixed 3x3 block size, which enables constant-time maximum computation for each block.

#36 Valid Sudoku [Medium]

Key Insights

- Validate each row by checking for duplicate numbers while ignoring empty cells.
- Validate each column in the same way.
- Divide the board into nine 3x3 sub-boxes and check each for duplicate numbers.

```
if (curr_sum - target) in sum_count:
    result += sum_count[curr_sum - target]
# Update the dictionary with the current sum count
sum_count[curr_sum] = sum_count.get(curr_sum, 0) + 1

return result

# Example usage:
if __name__ == "__main__":
    matrix = [[0, 1, 0], [1, 1, 1], [0, 1, 0]]
    target = 0
    print(numSubmatrixSumTarget(matrix, target)) # Expected Output: 4
```

```
C++
// C++ solution for counting number of substrings that sum to target
#include <vector>
#include <unordered_map>
using namespace std;

class Solution {
public:
    int numSubmatrixSumTarget(vector<vector<int>> &matrix, int target) {
        int m = matrix.size();
        int n = matrix[0].size();
        int result = 0;

        // Consider every possible pair of rows as boundaries for the submatrix
        for (int top = 0; top < m; ++top) {
            vector<int> colSums(n, 0); // Initialize column sums for each pair
            for (int bottom = top; bottom < m; ++bottom) {
                // Summarize column sums for the current bottom row
                for (int col = 0; col < n; ++col) {
                    colSums[col] += matrix[bottom][col];
                }

                // Use unordered_map to find the number of subarrays in colSums that sum to target
                unordered_map<int, int> sumCount;
                sumCount[0] = 1;
                int currSum = 0;
            }
        }
    }
};
```

Quick Review — Matrix 20 questions · insights · complexity · solution

#422 Valid Word Square [Easy]

Key Insights

- The k-th row should be equal to the k-th column.
- Not every word is the same length; checks must be performed within bounds.
- Iterating over each character and cross-verifying with the corresponding column character is sufficient.

Space & Time

Time Complexity: O(N * L) where N is the number of words and L is the average length of the words.

Space Complexity: O(1) additional space is used for the checking procedure.

Solution

We iterate through each word (row) and each character (column) in the word. For every character at position (i, j), we confirm that there is a corresponding word at index j and that its length is greater than i. Then we check that the character at position (j, i) in that word is the same as the current character. If any of these checks fail, we return false. If all checks pass, the words form a valid word square.

#566 Reshape the Matrix [Easy]

Key Insights

- Check if the reshape operation is possible by comparing the total elements (m * n with r * c).
- Flatten the original matrix while preserving the row-traversing order.
- Use the flattened list to build the new matrix row by row.
- If the total number of elements does not match, simply return the original matrix.

Space & Time

Time Complexity: O(m * n)

n) as we iterate through all elements of the matrix once.

Space Complexity: O(m * n)

n) for the additional list used to store flattened elements (or the new matrix).

Solution

The solution involves first checking whether the reshape is possible by ensuring that the product of the rows and columns of the original matrix equals that of the desired matrix (r * c). If not, the original matrix is returned. Otherwise, the matrix is flattened into a one-dimensional list, preserving the row-order traversal. Finally, the flattened list is segmented by the new column size c to form the reshaped matrix. The primary data structures used include a list (or array) for flattening the matrix and then another list (or vector/array) to hold the final reshaped matrix.

#766 Toeplitz Matrix [Easy]

Key Insights

- Each element (except those in the first row and first column) must be equal to its top-left neighbor.
- You can validate the matrix by iterating from the second row/column and comparing each element with matrix[i-1][j-1].
- This simple check allows you to determine if every diagonal has the same elements without extra space.
- For streaming data (loading one row at a time), maintain the previous row for comparison.

Space & Time

Time Complexity: O(m * n)

Space Complexity: O(1)

- Use hash sets (or equivalent data structures) to keep track of seen digits for rows, columns, and boxes.
- The board size is fixed (3x3), so the overall time complexity is constant with respect to input size.

Space & Time

Time Complexity: O(1) — Since the board size is fixed at 9x9.

Space Complexity: O(1) — The extra space used by the sets is bounded by the fixed board size.

Solution

The solution uses three main data structures: an array of sets for rows, an array of sets for columns, and an array of sets for 3x3 sub-boxes. For each cell, if it is not empty, check if the number is already present in the corresponding row, column, or box. If found, return false immediately. Otherwise, add the number to the respective sets. Compute the index for the 3x3 sub-box using the formula: (row_index // 3) * 3 + (col_index // 3). If no duplicates are found after traversing the entire board, the board is considered valid.

#48 Rotate Image [Medium]

Key Insights

- The rotation can be achieved in two steps: first transpose the matrix and then reverse each row.
- Transposing the matrix swaps matrix[i][j] with matrix[j][i], converting rows into columns.
- Reversing each row post-transposition yields the desired clockwise rotation.
- This approach works in-place with O(1) extra space.

Space & Time

Time Complexity: O(n^2) since every element is processed during the transpose and row reversal.

Space Complexity: O(1) as the operations are performed in-place without extra data structures.

Solution

The solution leverages a two-step in-place transformation:

- Transpose the matrix: Iterate through the matrix and swap each element (i, j) with element (j, i) for j > i.
- Reverse each row: After the transpose, each row is reversed to rotate the matrix 90 degrees clockwise. The key trick is to realize that a transpose followed by a reverse of each row achieves the rotation without needing extra space.

#54 Spiral Matrix [Medium]

Key Insights

- Use four boundaries: top, bottom, left, and right to track the current layer.
- Traverse right across the top row, then down the right column, then left on the bottom row, and finally up the left column.
- After each direction, adjust the corresponding boundary to move inward.
- Continue the traversal until the boundaries cross.

Space & Time

Time Complexity: O(m * n) since each element is visited exactly once.

Space Complexity: O(1) extra space (excluding the space required for the output list).

Solution

The solution uses a simulation approach with boundary pointers (top, bottom, left, right). At each step, the boundaries indicate the current submatrix to traverse. You iterate as follows:

- Traverse from left to right along the top boundary.
- Traverse downward along the right boundary.
- If the top boundary has not passed the bottom, traverse from right to left along the bottom boundary.
- If the left boundary has not passed the right, traverse upward along the left boundary.
- After each traversal, the boundaries are adjusted inward. This continues until all elements have been visited.

#59 Spiral Matrix II [Medium]

Key Insights

- Use four pointers (top, bottom, left, right) to keep track of the matrix boundaries.

- Traverse the matrix in a spiral: move right along the top row, then down the right column, left along the bottom row, and finally up the left column.
- Update the boundary pointers after each complete traversal of one side.
- Continue until all numbers from 1 to n^2 are filled into the matrix.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$ (required for the output matrix)

Solution

We simulate the spiral movement by initializing four pointers: top, bottom, left, and right, which denote the current boundaries of the matrix that have not yet been filled. For each iteration, we:

- Traverse from left to right along the top boundary, then increment the top pointer. - Traverse from top to bottom along the right boundary, then decrement the right pointer. - If there are rows remaining, traverse from right to left along the bottom boundary, then decrement the bottom pointer. - If there are columns remaining, traverse from bottom to top along the left boundary, then increment the left pointer. This algorithm ensures that the matrix is filled in a spiral order until all numbers are placed.

#76 Minimum Path Sum [Medium]

Key Insights

- The problem can be solved using Dynamic Programming.
- The optimal solution for each cell depends on the minimum path sum of the cell above it and the cell to its left.
- Since you can only move right or down, only two adjacent cells affect the current cell's result.
- Edge cases include the first row and first column, which have only one direction of movement.

Space & Time

Time Complexity: $O(m \times n)$, where m and n are the number of rows and columns.

Space Complexity: $O(m \times n)$ for the DP table (this can be optimized to $O(n)$ with in-place computations).

Solution

We use a Dynamic Programming (DP) approach where we create a DP table (or use the grid itself) that stores the minimum path sum to each cell. For each cell (i, j) :

- If $i = 0$ (first row), the only possible path is from the left. - If $j = 0$ (first column), the only possible path is from above. - For other cells, the minimum path sum is the cell value plus the minimum of the two possible previous cells (above and left). This ensures that by the time we reach the bottom-right cell, we have accumulated the minimum sum required to reach that cell.

#73 Set Matrix Zeroes [Medium]

Key Insights

- The challenge is to mark rows and columns that need to be zeroed without using extra space.
- A common trick is to use the first row and first column as markers.
- Special care is needed for the first row and first column, as they are also used as markers.
- The algorithm involves two passes: one for marking and one for updating the matrix based on the markers.

Space & Time

Time Complexity: $O(m \times n)$

Space Complexity: $O(1)$ (in-place, using only constant extra space)

Solution

We can solve this problem by using the first row and first column of the matrix as markers. First, determine if the first row or first column needs to be zeroed, then traverse the rest of the matrix and mark the corresponding first row and column positions if a zero is encountered. Finally, update the matrix backward using the markers, and finally update the first row and column if needed.

Matrix · LeetCode · By Pattern

— 1729 —

LeetCode

Time Complexity: $O(m \times k \times n)$ in the worst-case, but significantly lower when many elements are zero.

Space Complexity: $O(m \times n)$ for the resulting matrix.

Solution

We loop through each row of `mat1` and for every non-zero element, we traverse the corresponding row in `mat2`. By checking if an element in both matrices is non-zero before multiplying, we avoid redundant calculations. This optimization leverages the sparsity of the matrices. We use standard nested loops along with conditional checks to implement this efficient multiplication.

#488 Diagonal Traverse [Medium]

Key Insights

- Traverse the matrix diagonally with alternating directions: upward (top-right) and downward (bottom-left).
- When the traversal reaches a boundary (top, bottom, left, or right), adjust the position and change the direction.
- Use simulation to process each element exactly once while managing the indices based on the current direction.

Space & Time

Time Complexity: $O(m \times n)$ where m is the number of rows and n is the number of columns, as every element is processed once.

Space Complexity: $O(m \times n)$ for the output array (excluding the input matrix), while additional space usage remains constant.

Solution

The solution simulates the diagonal traversal using a direction flag (1 for upward and -1 for downward).

- Begin at the matrix's top-left corner. - Append the current element to the result. - Depending on the current direction: If moving upward, decrease the row and increase the column. If moving downward, increase the row and decrease the column. - Adjust for boundary hits: When moving upward: If at the last column, increment the row and switch direction. If at the first row, increment the column and switch direction. When moving downward: If at the last row, increment the column and switch direction. If at the first column, increment the row and switch direction. This process continues until all elements are added to the result array.

#531 Lonely Pixel I [Medium]

Key Insights

- Count the number of 'W' pixels in each row.
- Count the number of 'W' pixels in each column.
- A pixel is considered lonely if the count in its corresponding row and column is exactly 1.
- Two passes through the matrix can efficiently solve the problem: one for counting and one for checking conditions.

Space & Time

Time Complexity: $O(m \times n)$, where m and n are the dimensions of the picture.

Space Complexity: $O(m \times n)$, used to store the row and column counts.

Solution

The solution works by first scanning the entire matrix to compute the number of black pixels in each row and each column using two arrays/lists. Once these counts are available, a second pass through the matrix determines for each black pixel if it is lonely by checking if its corresponding row and column count equals 1. The approach uses simple array data structures for tallying counts and ensures that each element is processed only a constant number of times.

#723 Candy Crush [Medium]

Key Insights

- Use a simulation approach: repeatedly detect crushable candies and simulate their removal.
- Traverse horizontally and vertically to mark candies that are in groups of three or more.

Matrix · LeetCode · By Pattern

— 1731 —

LeetCode

Time Complexity: $O(m \times n \times \log(n))$ where m is the number of rows and n is the number of columns (binary search in each row for each candidate from the first row).

Space Complexity: $O(1)$ (only a few auxiliary variables are used).

Solution

The solution iterates over each element (candidate) in the first row of the matrix. For each candidate, it checks if the candidate is present in every other row using binary search, taking advantage of the sorted order. If a candidate is found in every row, return it immediately as it will be the smallest such common element because the first row is processed in order. If no candidate is common across all rows, return -1.

#1074 Number of Submatrices That Sum to Target [Hard]

Key Insights

- Transform the 2D submatrix sum problem into multiple 1D subarray sum problems.
- Precompute cumulative sums by fixing two row boundaries and then summing columns between these rows.
- Use a hash table (or dictionary) to count the number of subarrays with a given sum, reducing the problem to the classic "subarray sum equals k" question.
- Iterate over all possible row pairs and for each, treat the column sums as an array where the target sum is sought.

Space & Time

Time Complexity: $O(n^3 \times m)$, where n is the number of rows and m is the number of columns.

Space Complexity: $O(m)$, using the hash table for storing cumulative sums along the columns.

Solution

The idea is to iterate over all pairs of rows (top and bottom) and, for each pair, compute the cumulative column sums between these rows. This converts the problem into finding the number of subarrays in a one-dimensional array (the column sums) that add up to the target. For each such 1D problem, use a hash table to track the frequency of cumulative sums seen so far and count how many times the difference (current sum - target) has been seen. This count gives the number of subarrays ending at the current index which sum to target. Combining counts from all possible row pairs yields the final result.

Matrix · LeetCode · By Pattern

— 1733 —

LeetCode

The algorithm works in three main steps:

- Check if the first row or first column should be zeroed (store this in two variables). - Iterate through the rest of the matrix: for any element that is zero, mark its row and column by setting the corresponding element in the first row and first column to zero. - Iterate through the matrix again (excluding the first row and column) and set elements to zero if their row or column marker is zero. - Finally, zero out the first row and/or first column if needed.

This approach avoids extra space usage apart from the two boolean flags while ensuring that the matrix is updated correctly.

#74 Search a 2D Matrix [Medium]

Key Insights

- The matrix is essentially a flattened sorted array due to its properties.
- Use binary search to efficiently determine if the target exists.
- Map the 1D binary search index to the corresponding 2D matrix coordinates using division and modulus operations.
- Achieve $O(\log(m \times n))$ time complexity by navigating the matrix as if it were a single sorted list.

Space & Time

Time Complexity: $O(\log(m \times n))$

Space Complexity: $O(1)$

Solution

We can treat the matrix as a flattened sorted list without physically copying the elements. Given the total number of elements $= m \times n$, perform binary search on indices ranging from 0 to $(m \times n - 1)$. For any index `mid`, calculate its corresponding row as `mid / n` and column as `mid % n`. Compare the element at `matrix[mid / n][mid % n]` with the target. If it equals target, return true; if it is less than target, search the right half; otherwise, search the left half.

#221 Maximal Square [Medium]

Key Insights

- Use dynamic programming to build a solution where each cell in the DP table represents the side length of the largest square ending at that cell.
- If the current cell is '1', its DP value is the minimum of the top, left, and top-left neighbors plus one.
- Update a global maximum side length during the process to determine the area.
- The square's area is the square of its maximum side length.

Space & Time

Time Complexity: $O(m \times n)$, where m is the number of rows and n is the number of columns.

Space Complexity: $O(m \times n)$ for the DP table (this can be optimized to $O(n)$ with space reduction techniques).

Solution

We use a two-dimensional dynamic programming approach. We define a DP matrix with one extra row and column to simplify boundary conditions. For each cell (i, j) in the input matrix, if the value is '1', then we calculate the DP value as the minimum of its top, left, and top-left neighboring DP values plus one. This value represents the side length of the square ending at (i, j) . We also keep track of the maximum side length found and finally compute the area of the maximal square by squaring the maximum side length.

#311 Sparse Matrix Multiplication [Medium]

Key Insights

- Exploit sparsity by iterating only over non-zero elements.
- For each non-zero element in `mat1`, multiply it with corresponding non-zero elements in `mat2`.
- Accumulate the product into the result matrix.
- This approach reduces unnecessary multiplication operations when many elements are zero.

Space & Time

Matrix · LeetCode · By Pattern

— 1730 —

LeetCode

- Instead of removing candies immediately, mark them for removal and update the board once for the entire pass.
- After crushing, simulate gravity by shifting non-zero candies downward in each column.
- Continue the process until no further candy crushes can be detected.

Space & Time

Time Complexity: $O(m \times n^2)$

in the worst case since each iteration includes scanning the $m \times n$ grid and then applying gravity.

Space Complexity: $O(m \times n)$

for maintaining additional markers or flags (or using the board itself with in-place modifications).

Solution

The solution involves iteratively scanning the board to identify candy groups that should be crushed using two passes:

- Horizontal Check - For each row, check segments of 3 consecutive candies. - Vertical Check - For each column, check segments of 3 consecutive candies. If any candy qualifies, mark its position for crushing. After marking, update the board by turning all marked positions into 0. Then, for each column, drop down non-zero candies such that all empty cells (zeros) are at the top. Repeat this process until no group of 3 or more adjacent candies is found. Key data structures include the board array and a temporary marker (can be implicit by using signs or a separate boolean matrix) for tracking candies to crush. The algorithm ensures a stable board before returning it.

#835 Image Overlap [Medium]

Key Insights

- You only need to consider the positions of 1's in each matrix.
- The overlap count for a given translation can be computed by comparing the relative differences between the positions of 1's in the two images.
- Using a hash map (or dictionary) to count how many times each translation vector appears lets you compute the maximum overlap efficiently.
- The translation operation is equivalent to shifting the positions, so the best possible translation is the one that aligns the most pairs of 1's.

Space & Time

Time Complexity: $O(n^4)$ in the worst-case when using a nested iteration over positions for both matrices, though the average situation is typically better since not all cells are 1.

Space Complexity: $O(n^2 \times 2)$ for storing the positions of 1's and the counts for translation vectors.

Solution

The solution involves the following steps:

- Traverse both images to record the coordinates of all 1's. - For every pair of 1's (one from the first image, one from the second), compute the translation vector (offset) that aligns them. - Use a hash map to count how many times each offset appears. - The maximum count recorded in the hash map is the largest overlap achievable by any translation. This approach avoids simulating every possible slide over the matrix and focuses only on relative alignments of 1's.

#1190 Find Smallest Common Element in All Rows [Medium]

Key Insights

- Since every row is strictly increasing, binary search can be used efficiently to check for the presence of an element in a row.
- Iterating over the first row's elements and verifying their presence in all other rows is an effective approach.
- Early termination is possible if an element is not found in any one row.
- Time complexity is manageable given the constraints, even using binary search for each candidate across rows.

Space & Time

Matrix · LeetCode · By Pattern

— 1732 —

LeetCode

#16 Linked List — 23 questions · #16 of 21

#21

EASY

Merge Two Sorted Lists

LeetCode · SimplifyLife · WikiCNC · LeetCode · Editorial

Linked List · Recursion

Lists · Grid 169

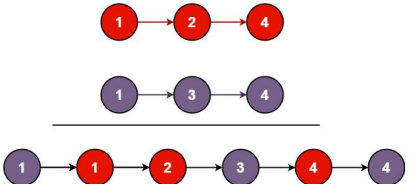
Problem:

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example 1:



Example 2:

Input: list1 = [], list2 = []

Output: []

Example 3:

Input: list1 = [], list2 = [0]

Output: [0]

Constraints:

- The number of nodes in both lists is in the range $[0, 50]$.
- $-100 \leq \text{Node.val} \leq 100$
- Both `list1` and `list2` are sorted in non-decreasing order.

Matrix · LeetCode · By Pattern

— 1734 —

LeetCode

```

Python
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def mergeTwoLists(list1: ListNode, list2: ListNode) -> ListNode:
    # Create a dummy node to serve as the start of the merged list
    dummy = ListNode(-1)
    current = dummy

    # Iterate while both lists have nodes
    while list1 and list2:
        # Compare the values of the current nodes of both lists
        if list1.val <= list2.val:
            current.next = list1 # link the smaller node to the merged list
            list1 = list1.next # Move the pointer in list1
        else:
            current.next = list2
            list2 = list2.next
            current = current.next # Move the pointer in the merged list

    # Once one list is exhausted, attach the remaining nodes from the other list
    if list1:
        current.next = list1
    else:
        current.next = list2

    # The merged list head is at dummy.next
    return dummy.next

Python
class Solution:
    def mergeTwoLists(
        self,
        list1: Optional[ListNode],
        list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode(-1)
        current = dummy
        while list1 and list2:
            if list1.val <= list2.val:
                current.next = list1
                list1 = list1.next
            else:
                current.next = list2
                list2 = list2.next
            current = current.next
        if list1:
            current.next = list1
        else:
            current.next = list2
        return dummy.next

```

Python

Linked List · LeetCode · By Pattern

— 175 —

LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        if list1 is None or list2 is None:
            return list1 or list2
        if list1.val <= list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
            list2.next = self.mergeTwoLists(list1, list2.next)
            return list2

Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
# @param list1 list1: list1
# @param list2 list2: list2
# @return list1
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode(-1)
        current = dummy
        while list1 and list2:
            if list1.val <= list2.val:
                current.next = list1
                list1 = list1.next
            else:
                current.next = list2
                list2 = list2.next
            current = current.next
        if list1:
            current.next = list1
        else:
            current.next = list2
        return dummy.next

```

Python

Linked List · LeetCode · By Pattern

— 176 —

LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode(-1)
        current = dummy
        while list1 and list2:
            if list1.val <= list2.val:
                current.next = list1
                list1 = list1.next
            else:
                current.next = list2
                list2 = list2.next
            current = current.next
        if list1:
            current.next = list1
        else:
            current.next = list2
        return dummy.next

better than:
def mergeTwoLists(list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
    if not list1:
        return list2
    if not list2:
        return list1
    if list1.val <= list2.val:
        list1.next = mergeTwoLists(list1.next, list2)
        return list1
    else:
        list2.next = mergeTwoLists(list1, list2.next)
        return list2

# recursion
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        if not list1:
            return list2
        if not list2:
            return list1
        if list1.val <= list2.val:
            list1.next = mergeTwoLists(list1.next, list2)
            return list1
        else:
            list2.next = mergeTwoLists(list1, list2.next)
            return list2

```

Linked List · LeetCode · By Pattern

— 177 —

LeetCode

```

Java
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

public class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        if (list1 == null) {
            return list2;
        }
        if (list2 == null) {
            return list1;
        }
        if (list1.val <= list2.val) {
            list1.next = mergeTwoLists(list1.next, list2);
            return list1;
        } else {
            list2.next = mergeTwoLists(list1, list2.next);
            return list2;
        }
    }
}

```

Java

Linked List · LeetCode · By Pattern

— 178 —

LeetCode

```

// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

public class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode dummy = new ListNode(-1);
        ListNode cur = dummy;
        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                cur.next = list1;
                list1 = list1.next;
            } else {
                cur.next = list2;
                list2 = list2.next;
            }
            cur = cur.next;
        }
        if (list1 != null) {
            cur.next = list1;
        } else if (list2 != null) {
            cur.next = list2;
        }
        return dummy.next;
    }
}

```

JavaScript

JavaScript

Linked List · LeetCode · By Pattern

— 179 —

LeetCode

```

// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

var mergeTwoLists = function(list1, list2) {
    if (!list1 || !list2) {
        return list1 || list2;
    }
    if (list1.val <= list2.val) {
        list1.next = mergeTwoLists(list1.next, list2);
        return list1;
    } else {
        list2.next = mergeTwoLists(list1, list2.next);
        return list2;
    }
};

```

JavaScript

```

// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }

var mergeTwoLists = function(list1, list2) {
    if (!list1 || !list2) {
        return list1 || list2;
    }
    if (list1.val <= list2.val) {
        list1.next = mergeTwoLists(list1.next, list2);
        return list1;
    } else {
        list2.next = mergeTwoLists(list1, list2.next);
        return list2;
    }
};

```

JavaScript

Linked List · LeetCode · By Pattern

— 180 —

LeetCode


```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val: int, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        # Initiation low pointers, slow and fast, starting from the head.
        slow = head
        fast = head

        # Traverse the list until fast pointer reaches the end.
        while fast and fast.next:
            slow = slow.next # Move slow pointer one step.
            fast = fast.next.next # Move fast pointer two steps.

            # If both pointers meet, there is a cycle.
            if slow == fast:
                return True

        # If loop terminates, there is no cycle.
        return False

```

Python

```

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True
        return False

```

Python

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val: int, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        s = null
        while head:
            if head is s:
                return True
            s.add(head)
            head = head.next
        return False

```

Python

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val: int, next=None):
#         self.val = val
#         self.next = None

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True
        return False

```

Java

Java

```

class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast)
                return true;
        }
        return false;
    }
}

```

Java

```

//
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int x) {
//         val = x;
//         next = null;
//     }
// }

public class Solution {
    public boolean hasCycle(ListNode head) {
        Set<ListNode> s = new HashSet<>();
        for (; head != null; head = head.next) {
            if (s.add(head)) {
                return true;
            }
        }
        return false;
    }
}

```

```

//
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int x) {
//         val = x;
//         next = null;
//     }
// }

public class Solution {
    public boolean hasCycle(ListNode head) {
        var fast = head;
        var slow = head;
        while (fast != null && fast.next != null) {
            fast = fast.next.next;
            slow = slow.next;
            if (fast == slow) {
                return true;
            }
        }
        return false;
    }
}

```

Java

```

//
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int x) {
//         val = x;
//         next = null;
//     }
// }

public class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                return true;
            }
        }
        return false;
    }
}

```

Java

```

//
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int x) {
//         val = x;
//         next = null;
//     }
// }

public class Solution {
    public boolean hasCycle(ListNode head) {
        var fast = head;
        var slow = head;
        while (fast != null && fast.next != null) {
            fast = fast.next.next;
            slow = slow.next;
            if (fast == slow) {
                return true;
            }
        }
        return false;
    }
}

```

JavaScript

JavaScript

```

//
// Definition for singly-linked list.
// function ListNode(val) {
//     this.val = val;
//     this.next = null;
// }

//
// @param {ListNode} head
// @return {boolean}

var hasCycle = function (head) {
    let slow = head;
    let fast = head;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            return true;
        }
    }
    return false;
};

```

JavaScript

```

//
// Definition for singly-linked list.
// function ListNode(val) {
//     this.val = val;
//     this.next = null;
// }

//
// @param {ListNode} head
// @return {boolean}

var hasCycle = function (head) {
    let slow = head;
    let fast = head;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            return true;
        }
    }
    return false;
};

```

TypeScript

TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val);
//         this.next = (next===undefined ? null : next);
//     }
// }

function hasCycle(head: ListNode | null): boolean {
    const s: Set<ListNode> = new Set();
    for (; head; head = head.next) {
        if (s.has(head)) {
            return true;
        }
        s.add(head);
    }
    return false;
}

```

TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val);
//         this.next = (next===undefined ? null : next);
//     }
// }

function hasCycle(head: ListNode | null): boolean {
    let slow = head;
    let fast = head;
    while (fast && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            return true;
        }
    }
    return false;
}

```

Go

Go

```

//
// Definition for singly-linked list.
// type ListNode struct {
//     Val int
//     Next *ListNode
// }

func hasCycle(head *ListNode) bool {
    s := map[*ListNode]struct{}{}
    for ; head != nil; head = head.Next {
        if s[head] {
            return true
        }
        s[head] = struct{}{}
    }
    return false
}

```

[*] Linked List — Pattern Solution (matched from community answers)

Python


```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        # Initialize two pointers, slow and fast, starting from the head.
        slow = head
        fast = head

        # Traverse the list until fast pointer reaches the end.
        while fast and fast.next:
            slow = slow.next # Move slow pointer one step.
            fast = fast.next.next # Move fast pointer two steps.

            # If both pointers meet, there is a cycle.
            if slow == fast:
                return True

        # If loop terminates, there is no cycle.
        return False

```

#206 EASY

Reverse Linked List

LeetCode · Simplest · RustCC · LeetCode · Editorial

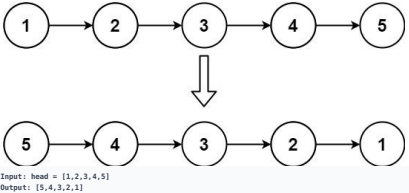
Linked List · Recursion

List: Grind 169

Problem:

Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:

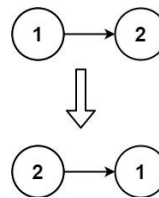


Linked List · LeetCode · By Pattern

— 1753 —

LeetCode

Example 2:



Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].
- 5000 <= Node.val <= 5000

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

>> Brute Force [Linked List] Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def reverseList(self, head):
        # Base case: if head is None, return None
        if not head:
            return None
        # Collect values, rebuild in reverse
        vals = []
        curr = head
        while curr:
            vals.append(curr.val)
            curr = curr.next
        dummy = ListNode(0)
        curr = dummy
        for v in reversed(vals):
            curr.next = ListNode(v)
            curr = curr.next
        return dummy.next

```

Python

Python

Linked List · LeetCode · By Pattern

— 1754 —

LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val # Store the node value
#         self.next = next # Pointer to the next node
# Iterate approach to reverse the linked list.
def reverseList(head):
    prev = None # Initially, there is no previous node
    current = head # Start with the head of the list
    while current:
        next_node = current.next # Temporarily store the next node
        current.next = prev # Reverse the current node's pointer
        prev = current # Move previous pointer one step forward
        current = next_node # Move to the next node in the list
    return prev # The new head of the reversed list

# Recursive approach to reverse the linked list.
def reverseListRecursive(head):
    # Base case: empty list or single node
    if not head or not head.next:
        return head
    new_head = reverseListRecursive(head.next) # Reverse the rest of the list
    head.next.next = head # Reverse the pointer for the current node
    head.next = None # Set the next pointer of the current node to None
    return new_head # Return the new head of the reversed list

```

Python

```

class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        if not head or not head.next:
            return head
        newhead = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return newhead

```

Python

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()
        curr = head
        while curr:
            next = curr.next
            curr.next = dummy.next
            dummy.next = curr
            curr = next
        return dummy.next

```

Python

Linked List · LeetCode · By Pattern

— 1755 —

LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        pre, p = None, head
        while p:
            q = p.next
            p.next = pre
            pre = p
            p = q
        return pre

class Solution(object): # recursive
    def reverseList(self, head):
        if not head or not head.next:
            return head
        rest = self.reverseList(head.next)
        head.next.next = head # head.next is end of the newly reversed list
        head.next = None
        return head

```

Java

Java

Java

Linked List · LeetCode · By Pattern

— 1756 —

LeetCode

```

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int val=0, ListNode next=null) {
 *         this.val = val;
 *         this.next = next;
 *     }
 * }
 */
public class Solution {
    public ListNode ReverseList(ListNode head) {
        ListNode dummy = new ListNode();
        ListNode curr = head;
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = dummy.next;
            dummy.next = curr;
            curr = next;
        }
        return dummy.next;
    }
}

```

Java

```

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int val=0, ListNode next=null) {
 *         this.val = val;
 *         this.next = next;
 *     }
 * }
 */
public class Solution {
    public ListNode ReverseList(ListNode head) {
        ListNode prev = null;
        for (ListNode p = head; p != null;) {
            ListNode t = p.next;
            p.next = prev;
            prev = p;
            p = t;
        }
        return prev;
    }
}

```

JavaScript

JavaScript

Linked List · LeetCode · By Pattern

— 1757 —

LeetCode

```

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var reverseList = function (head) {
    let dummy = new ListNode();
    let curr = head;
    while (curr) {
        let next = curr.next;
        curr.next = dummy.next;
        dummy.next = curr;
        curr = next;
    }
    return dummy.next;
};

```

JavaScript

```

/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var reverseList = function (head) {
    let dummy = new ListNode();
    let curr = head;
    while (curr) {
        let next = curr.next;
        curr.next = dummy.next;
        dummy.next = curr;
        curr = next;
    }
    return dummy.next;
};

```

TypeScript

TypeScript

Linked List · LeetCode · By Pattern

— 1758 —

LeetCode

```
//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function reverseList(head: ListNode | null): ListNode | null {
    if (head == null) {
        return head;
    }
    let pre = null;
    let cur = head;
    while (cur != null) {
        const next = cur.next;
        cur.next = pre;
        [pre, cur] = [cur, next];
    }
    return pre;
}

TypeScript
//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function reverseList(head: ListNode | null): ListNode | null {
    if (head == null) {
        return head;
    }
    let pre = null;
    let cur = head;
    while (cur != null) {
        const next = cur.next;
        cur.next = pre;
        [pre, cur] = [cur, next];
    }
    return pre;
}

Rust
Rust
```

```
// Definition for singly-linked list.
// @derive(PartialEq, Eq, Clone, Debug)
// pub struct ListNode {
//     pub val: i32,
//     pub next: Option>
// }
//
// impl ListNode {
//     #[inline]
//     fn new(val: i32) -> Self {
//         ListNode {
//             next: None,
//             val,
//         }
//     }
// }

impl Solution {
    pub fn reverse_list(head: Option>) -> Option> {
        let mut head = head;
        let mut pre = None;
        while let Some(mut node) = head {
            head = node.next.take();
            node.next = pre.take();
            pre = Some(node);
        }
        pre
    }
}

Rust
// Definition for singly-linked list.
// @derive(PartialEq, Eq, Clone, Debug)
// pub struct ListNode {
//     pub val: i32,
//     pub next: Option>
// }
//
// impl ListNode {
//     #[inline]
//     fn new(val: i32) -> Self {
//         ListNode {
//             next: None,
//             val,
//         }
//     }
// }

impl Solution {
    pub fn reverse_list(head: Option>) -> Option> {
        match head {
            None => None,
            Some(mut head) => {
                let mut cur = head.next.take();
                let mut pre = Some(head);
                while let Some(mut node) = cur {
                    let next = node.next.take();
                }
            }
        }
    }
}
```

```
node.next = pre;
pre = Some(node);
cur = next;
}
pre
}
}

*) Linked List — Pattern Solution (matched from community answers)

Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()
        curr = head
        while curr:
            next = curr.next
            curr.next = dummy.next
            dummy.next = curr
            curr = next
        return dummy.next

#234 EASY
Palindrome Linked List
LeetCode · Simplified · WalkCC · LeetCode · Editorial
Linked List · Two Pointers · Recursion
Lists Grid 189

Problem:
Given the head of a singly linked list, return true if it is a palindrome or false otherwise.
Example 1:
1 -> 2 -> 2 -> 1
Input: head = [1,2,2,1]
Output: true
Example 2:
1 -> 2 -> 3 -> 2 -> 1
Input: head = [1,2,3,2,1]
Output: true
```

1 → 2

Input: head = [1,2]
Output: false

Constraints:

- The number of nodes in the list is in the range [1, 105].
- 0 ≤ Node.val ≤ 9

Follow up: Could you do it in O(n) time and O(1) space?

```
Python
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        # Find the midpoint using slow and fast pointers
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next # Move slow pointer by 1
            fast = fast.next.next # Move fast pointer by 2

        # Reverse the second half of the list
        prev = None
        while slow:
            next_node = slow.next # Store the next node
            slow.next = prev # Reverse the current node's pointer
            prev = slow # Move prev to current node
            slow = next_node # Move to next node

        # Compare the first half and the reversed second half node by node
        left = head
        right = prev
        while right:
            if left.val != right.val:
                return False # Not a palindrome
            left = left.next # Move left pointer
            right = right.next # Move right pointer
        return True # All nodes matched, it's a palindrome

Python
```

```
class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        def reverseList(head: ListNode) -> ListNode:
            pre = None
            curr = head
            while curr:
                next = curr.next
                curr.next = pre
                pre = curr
                curr = next
            return pre

        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        if fast:
            slow = slow.next

        while slow:
            if slow.val != head.val:
                return False
            slow = slow.next
            head = head.next
        return True

Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        pre = None
        while slow:
            t = slow.next
            slow.next = pre
            pre = slow
            slow = t
        while pre:
            if pre.val != head.val:
                return False
            pre = pre.next
            head = head.next
        return True

Python
```

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        pre = None
        while slow:
            t = slow.next
            slow.next = pre
            pre = slow
            slow = t
        while pre:
            if pre.val != head.val:
                return False
            pre = pre.next
            head = head.next
        return True

Java
class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        if (fast != null)
            slow = slow.next;
        slow = reverseList(slow);
        while (slow != null) {
            if (slow.val != head.val)
                return false;
            slow = slow.next;
            head = head.next;
        }
        return true;
    }
    private ListNode reverseList(ListNode head) {
        // ...
    }
}
```

```
ListNode prev = null;
while (head != null) {
    ListNode next = head.next;
    head.next = prev;
    prev = head;
    head = next;
}
return prev;
}
}

Java

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head;
        ListNode fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode cur = slow.next;
        slow.next = null;
        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }
        while (pre != null) {
            if (pre.val != head.val) {
                return false;
            }
            pre = pre.next;
            head = head.next;
        }
        return true;
    }
}

Java
```

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
public class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head;
        ListNode fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode cur = slow.next;
        slow.next = null;
        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }
        while (pre != null) {
            if (pre.val != head.val) {
                return false;
            }
            pre = pre.next;
            head = head.next;
        }
        return true;
    }
}

Java
```

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head;
        ListNode fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode cur = slow.next;
        slow.next = null;
        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }
        while (pre != null) {
            if (pre.val != head.val) {
                return false;
            }
            pre = pre.next;
            head = head.next;
        }
        return true;
    }
}

Java
```

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
public class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head;
        ListNode fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode cur = slow.next;
        slow.next = null;
        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }
        while (pre != null) {
            if (pre.val != head.val) {
                return false;
            }
            pre = pre.next;
            head = head.next;
        }
        return true;
    }
}

JavaScript
JavaScript
```

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {boolean}
 */
var isPalindrome = function(head) {
    let slow = head;
    let fast = head.next;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let cur = slow.next;
    slow.next = null;
    let pre = null;
    while (cur) {
        let t = cur.next;
        cur.next = pre;
        pre = cur;
        cur = t;
    }
    while (pre) {
        if (pre.val != head.val) {
            return false;
        }
        pre = pre.next;
        head = head.next;
    }
    return true;
};

JavaScript
```

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {boolean}
 */
var isPalindrome = function(head) {
    let slow = head;
    let fast = head.next;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let cur = slow.next;
    slow.next = null;
    let pre = null;
    while (cur) {
        let t = cur.next;
        cur.next = pre;
        pre = cur;
        cur = t;
    }
    while (pre) {
        if (pre.val != head.val) {
            return false;
        }
        pre = pre.next;
        head = head.next;
    }
    return true;
};

TypeScript
TypeScript
```

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function isPalindrome(head: ListNode | null): boolean {
    let slow: ListNode = head,
        fast: ListNode = head.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let cur: ListNode = slow.next,
        slow.next = null;
    let prev: ListNode = null;
    while (cur != null) {
        let t: ListNode = cur.next;
        cur.next = prev;
        prev = cur;
        cur = t;
    }
    while (prev != null) {
        if (prev.val != head.val) return false;
        prev = prev.next;
        head = head.next;
    }
    return true;
}

```

TypeScript

Linked List · LeetCode · By Pattern

— 1771 —

LeetCode

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function isPalindrome(head: ListNode | null): boolean {
    let slow: ListNode = head,
        fast: ListNode = head.next;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let cur: ListNode = slow.next,
        slow.next = null;
    let prev: ListNode = null;
    while (cur != null) {
        let t: ListNode = cur.next;
        cur.next = prev;
        prev = cur;
        cur = t;
    }
    while (prev != null) {
        if (prev.val != head.val) return false;
        prev = prev.next;
        head = head.next;
    }
    return true;
}

```

[*] Linked List — Pattern Solution (matched from community answers)

Python

Linked List · LeetCode · By Pattern

— 1772 —

LeetCode

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        # Find the midpoint using slow and fast pointers
        slow, fast = head, head
        while fast and fast.next:
            slow = slow.next # Move slow pointer by 1
            fast = fast.next.next # Move fast pointer by 2

        # Reverse the second half of the list
        prev = None
        while slow:
            next_node = slow.next # Store the next node
            slow.next = prev # Reverse the current node's pointer
            prev = slow # Move prev to current node
            slow = next_node # Move to next node

        # Compare the first half and the reversed second half node by node
        left, right = head, prev
        while right:
            if left.val != right.val:
                return False # Mismatch found, not a palindrome
            left = left.next # Move left pointer
            right = right.next # Move right pointer

        return True # All nodes matched, it's a palindrome

```

#706

EASY

Design HashMap

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Hash Table · Linked List · Design · Hash Function
Like Deemy Zhang

Problem:

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

- MyHashMap() initializes the object with an empty map.
- void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
- void remove(key) removes the key and its corresponding value if the map contains the mapping for the key.

Example 1:

Linked List · LeetCode · By Pattern

— 1773 —

LeetCode

```

# If key exists, update its value
current.value = value
return
if current.next is None:
    break
current = current.next
# Remove if one node at the end of the list if key not found
current.next = ListNode(key, value)

def get(self, key: int) -> int:
    index = self.hash(key)
    current = self.buckets[index]
    while current:
        if current.key == key:
            # key found, return its value
            return current.value
        current = current.next
    # key not found, return -1
    return -1

def remove(self, key: int) -> None:
    index = self.hash(key)
    current = self.buckets[index]
    if current is None:
        return
    if current.key == key:
        # Remove from map if it contains the key
        self.buckets[index] = current.next
    else:
        prev = current
        current = current.next
        while current:
            if current.key == key:
                # Remove the node to remove it from the list
                prev.next = current.next
                return
            prev, current = current, current.next

```

Python

Linked List · LeetCode · By Pattern

— 1775 —

LeetCode

```

class MyHashMap:
    def __init__(self):
        self.data = [-1] * 1000000

    def put(self, key: int, value: int) -> None:
        self.data[key] = value

    def get(self, key: int) -> int:
        return self.data[key]

    def remove(self, key: int) -> None:
        self.data[key] = -1

# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)

```

Python

```

class MyHashMap:
    def __init__(self):
        self.data = [-1] * 1000000

    def put(self, key: int, value: int) -> None:
        self.data[key] = value

    def get(self, key: int) -> int:
        return self.data[key]

    def remove(self, key: int) -> None:
        self.data[key] = -1

# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)

#####

class MyHashMap:
    def __init__(self):

```

Linked List · LeetCode · By Pattern

— 1776 —

LeetCode

```

        Initialize your data structure here.
        self.size = 1000
        self.buckets = [[] for _ in range(self.size)]

    def _hash(self, key: int) -> int:
        """
        Generate a hash for a given key.
        """
        return key % self.size

    def put(self, key: int, value: int) -> None:
        """
        Value will always be non-negative.
        """
        hash_key = self._hash(key)
        for i, (k, v) in enumerate(self.buckets[hash_key]):
            if k == key:
                self.buckets[hash_key][i] = (key, value)
                return
        self.buckets[hash_key].append((key, value))

    def get(self, key: int) -> int:
        """
        Returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
        """
        hash_key = self._hash(key)
        for k, v in self.buckets[hash_key]:
            if k == key:
                return v
        return -1

    def remove(self, key: int) -> None:
        """
        Removes the mapping of the specified value key if this map contains a mapping for the key.
        """
        hash_key = self._hash(key)
        for i, (k, v) in enumerate(self.buckets[hash_key]):
            if k == key:
                del self.buckets[hash_key][i]
                return

#####

class MyHashMap: # open addressing with linear probing
    def __init__(self):
        self.capacity = 1000
        self.data = [None] * self.capacity
        self.DELETED = (None, None) # Marker for deleted entries

    def _hash(self, key):
        return key % self.capacity

    def put(self, key: int, value: int) -> None:
        idx = self._hash(key)

```

Linked List · LeetCode - By Pattern

— 1777 —

LeetCode

```

C++
C++

class MyHashMap {
public:
    MyHashMap() : lists(10000) {}

    void put(int key, int value) {
        auto pairs = lists[key % 10000];
        for (auto& [k, v] : pairs) {
            if (k == key) {
                v = value;
                return;
            }
            pairs.emplace_back(key, value);
        }

        int get(int key) {
            const list<pair<int, int>& pairs = lists[key % 10000];
            for (const auto& [k, v] : pairs) {
                if (k == key) {
                    return v;
                }
            }

            void remove(int key) {
                auto pairs = lists[key % 10000];
                for (auto it = pairs.begin(); it != pairs.end(); ++it) {
                    if (*it--first == key) {
                        pairs.erase(it);
                        return;
                    }
                }

            private:
                static constexpr int kSize = 10000;
                // Each slot stores the (key, value) list.
                vector<list<pair<int, int>> lists;
            };
        }

Java
Java

```

Linked List · LeetCode - By Pattern

— 1778 —

LeetCode

```

class MyHashMap {
    private int[] data = new int[1000001];

    public MyHashMap() {
        Arrays.fill(data, -1);
    }

    public void put(int key, int value) {
        data[key] = value;
    }

    public int get(int key) {
        return data[key];
    }

    public void remove(int key) {
        data[key] = -1;
    }
}

/**
 * Your MyHashMap object will be instantiated and called as such:
 * MyHashMap obj = new MyHashMap();
 * obj.put(key,value);
 * int param_2 = obj.get(key);
 * obj.remove(key);
 */

Java

class MyHashMap {
    private int[] data = new int[1000001];

    public MyHashMap() {
        Arrays.fill(data, -1);
    }

    public void put(int key, int value) {
        data[key] = value;
    }

    public int get(int key) {
        return data[key];
    }

    public void remove(int key) {
        data[key] = -1;
    }
}

/**
 * Your MyHashMap object will be instantiated and called as such:
 * MyHashMap obj = new MyHashMap();
 * obj.put(key,value);
 * int param_2 = obj.get(key);
 * obj.remove(key);
 */

```

Linked List · LeetCode - By Pattern

— 1779 —

LeetCode

```

TypeScript
TypeScript

class MyHashMap {
    data: Array<number>;
    constructor() {
        this.data = new Array(10 < 6 + 1).fill(-1);
    }

    put(key: number, value: number): void {
        this.data[key] = value;
    }

    get(key: number): number {
        return this.data[key];
    }

    remove(key: number): void {
        this.data[key] = -1;
    }
}

/**
 * Your MyHashMap object will be instantiated and called as such:
 * var obj = new MyHashMap();
 * obj.put(key,value);
 * var param_2 = obj.get(key);
 * obj.remove(key);
 */

TypeScript

class MyHashMap {
    data: Array<number>;
    constructor() {
        this.data = new Array(10 < 6 + 1).fill(-1);
    }

    put(key: number, value: number): void {
        this.data[key] = value;
    }

    get(key: number): number {
        return this.data[key];
    }

    remove(key: number): void {
        this.data[key] = -1;
    }
}

/**
 * Your MyHashMap object will be instantiated and called as such:
 * var obj = new MyHashMap();
 * obj.put(key,value);
 * var param_2 = obj.get(key);
 * obj.remove(key);
 */

```

Linked List · LeetCode - By Pattern

— 1780 —

LeetCode

```

/**
 *
 */

Go
Go

type MyHashMap struct {
    data [1000001]
}

func Constructor() MyHashMap {
    data := make([]int, 1000001)
    for i := range data {
        data[i] = -1
    }
    return MyHashMap{data}
}

func (this *MyHashMap) Put(key int, value int) {
    this.data[key] = value
}

func (this *MyHashMap) Get(key int) int {
    return this.data[key]
}

func (this *MyHashMap) Remove(key int) {
    this.data[key] = -1
}

/**
 * Your MyHashMap object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Put(key,value);
 * param_2 := obj.Get(key);
 * obj.Remove(key);
 */

```

Go

Linked List · LeetCode - By Pattern

— 1781 —

LeetCode

```

type MyHashMap struct {
    data [1000001]
}

func Constructor() MyHashMap {
    data := make([]int, 1000001)
    for i := range data {
        data[i] = -1
    }
    return MyHashMap{data}
}

func (this *MyHashMap) Put(key int, value int) {
    this.data[key] = value
}

func (this *MyHashMap) Get(key int) int {
    return this.data[key]
}

func (this *MyHashMap) Remove(key int) {
    this.data[key] = -1
}

/**
 * Your MyHashMap object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Put(key,value);
 * param_2 := obj.Get(key);
 * obj.Remove(key);
 */

Python
Python

```

[1] Linked List - Pattern Solution (matched from community answers)

Python

Linked List · LeetCode - By Pattern

— 1782 —

LeetCode

```
# ListNode class for handling collisions in each bucket
class ListNode:
    def __init__(self, key, value, next=None):
        self.key = key # Store key
        self.value = value # Store value
        self.next = next # Pointer to the next node

class MyHashMap:
    def __init__(self):
        self.size = 1000 # Define the bucket size
        self.buckets = [None] * self.size # Initialize buckets as an array of None

    def hash(self, key):
        # Simple modulo based hash function to map keys to bucket indices
        return key % self.size

    def put(self, key, val, value: int) -> None:
        index = self.hash(key)
        if self.buckets[index] is None:
            # If the bucket is empty, create a new node and place it here
            self.buckets[index] = ListNode(key, value)
        else:
            current = self.buckets[index]
            while True:
                if current.key == key:
                    # If key exists, update its value
                    current.value = value
                    return
                if current.next is None:
                    break
            current.next = ListNode(key, value)
            # Append a new node at the end of the list if key not found
            current.next = ListNode(key, value)

    def get(self, key, key: int) -> int:
        index = self.hash(key)
        current = self.buckets[index]
        while current:
            if current.key == key:
                # Key found, return its value
                return current.value
            current = current.next
        # Key not found, return -1
        return -1

    def remove(self, key: int) -> None:
        index = self.hash(key)
        current = self.buckets[index]
        if current is None:
            return
        # If key exists, update its value
```

Linked List · LeetCode - By Pattern — 1783 —

LeetCode

```
if current.key == key:
    # Remove last node if it contains the key
    self.buckets[index] = current.next
else:
    prev = current
    current = current.next
    while current:
        if current.key == key:
            # Remove the node to remove it from the list
            prev.next = current.next
            return
        prev, current = current, current.next
```

#876

EASY

Middle of the Linked List

LeetCode · SimplifyLeetCode · WubCC · LeetCode · Editorial

Linked List · Two Pointers

Lists - Grid 100

Problem:

Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:

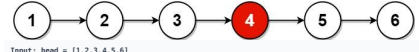


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- 1 <= Node.val <= 100

Python

Python

Linked List · LeetCode - By Pattern — 1784 —

LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        # Initialize slow and fast pointers at the head of the list
        slow = head
        fast = head
        # Loop until fast reaches the end of the list
        while fast and fast.next:
            # Move slow pointer one step
            slow = slow.next
            # Move fast pointer two steps
            fast = fast.next.next
        # When slow reaches the middle node
        return slow

Python

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = head
        fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow

Python

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
        return slow

Python

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
        return slow
```

Linked List · LeetCode - By Pattern — 1785 —

LeetCode

```
TypeScript
TypeScript

// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val);
//         this.next = (next===undefined ? null : next);
//     }
// }

function middleNode(head: ListNode | null): ListNode | null {
    let fast = head,
        slow = head;
    while (fast != null && fast.next != null) {
        fast = fast.next.next;
        slow = slow.next;
    }
    return slow;
}

TypeScript

// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val);
//         this.next = (next===undefined ? null : next);
//     }
// }

function middleNode(head: ListNode | null): ListNode | null {
    let fast = head,
        slow = head;
    while (fast != null && fast.next != null) {
        fast = fast.next.next;
        slow = slow.next;
    }
    return slow;
}

Rust
Rust
```

Linked List · LeetCode - By Pattern — 1786 —

LeetCode

```
// Definition for singly-linked list.
// #define ListNode struct ListNode
// struct ListNode {
//     int val;
//     struct ListNode *next;
// };

impl ListNode {
    #[inline]
    fn new(val: i32) -> Self {
        ListNode {
            val,
            next: None,
        }
    }
}

impl Solution {
    pub fn middleNode(head: Option) -> Option) {
        let mut slow = head;
        let mut fast = head;
        while fast.is_some() && fast.as_ref().unwrap().next.is_some() {
            slow = slow.as_ref().unwrap().next;
            fast = fast.as_ref().unwrap().next.as_ref().unwrap().next;
        }
        slow.clone()
    }
}

Rust

// Definition for singly-linked list.
// #define ListNode struct ListNode
// struct ListNode {
//     int val;
//     struct ListNode *next;
// };

impl ListNode {
    #[inline]
    fn new(val: i32) -> Self {
        ListNode {
            val,
            next: None,
        }
    }
}

impl Solution {
    pub fn middleNode(head: Option) -> OptionListNode) {
        let mut slow = head;
        let mut fast = head;
        while fast.is_some() && fast.as_ref().unwrap().next.is_some() {
            slow = slow.as_ref().unwrap().next;
            fast = fast.as_ref().unwrap().next.as_ref().unwrap().next;
        }
        slow.clone()
    }
}

Rust
```

Linked List · LeetCode - By Pattern — 1787 —

LeetCode

[*] Linked List - Pattern Solution (matched from community answers)

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        # Initialize slow and fast pointers at the head of the list
        slow = head
        fast = head
        # Loop until fast reaches the end of the list
        while fast and fast.next:
            # Move slow pointer one step
            slow = slow.next
            # Move fast pointer two steps
            fast = fast.next.next
        # When slow reaches the middle node
        return slow
```

#1474

EASY

Delete N Nodes After M Nodes of Linked List

LeetCode · SimplifyLeetCode · WubCC · LeetCode · Editorial

Linked List

Lists - Premium 50

Problem:

You are given the head of a linked list and two integers m and n.

Traverse the linked list and remove some nodes in the following way:

- Start with the head as the current node.
- Keep the first m nodes starting with the current node.
- Remove the next n nodes
- Keep repeating steps 2 and 3 until you reach the end of the list.

Return the head of the modified list after removing the mentioned nodes.

Example 1:



Input: head = [1,2,3,4,5,6,7,8,9,10,11,12,13], m = 2, n = 3

Output: [1,2,6,7,11,12]

Explanation: Keep the first (m = 2) nodes starting from the head of the Linked List (1 -> 2) and remove the next (n = 3) nodes.

Linked List · LeetCode - By Pattern — 1788 —

LeetCode

Delete the next ($n = 3$) nodes ($3 \rightarrow 4 \rightarrow 5$) show in red nodes.
Continue with the same procedure until reaching the tail of the Linked List.
Head of the Linked List after removing nodes is returned.

Example 2:



Input: head = [1,2,3,4,5,6,7,8,9,10,11], m = 1, n = 3
Output: [1,5,9]

Explanation: Head of Linked List after removing nodes is returned.

Constraints:

- The number of nodes in the list is in the range [1, 104].
- $1 \leq m \leq 106$
- $1 \leq n \leq 1000$

Follow up: Could you solve this problem by modifying the list in-place?

```
Python
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteNodes(self, head: ListNode, m: int, n: int) -> ListNode:
        # Initialize the current pointer at the head of the list.
        current = head
        while current:
            # Loop m nodes, traverse n-1 next pointers
            for i in range(1, m):
                if current is None: # If list ends, break
                    break
                current = current.next
            # If we have reached the end of the list, break the loop.
            if current is None:
                break
            # Start deletion from the next node after the m-th node.
            temp = current.next
            # Delete next n nodes.
            for j in range(1, n):
                if temp is None:
                    break
                temp = temp.next
            current = temp
        return head
```

Linked List · LeetCode · By Pattern

— 1789 —

LeetCode

```
temp = temp.next
# Connect the next node to the node after the n deleted nodes.
current.next = temp
# Move current pointer to continue from the next kept node.
current = temp
return head
```

```
Python
class Solution:
    def deleteNodes(self, head: ListNode | None, m: int, n: int) -> ListNode | None:
        curr = head
        prev = None # prev.next == curr

        while curr:
            # Set the next node as 'prev'.
            for _ in range(m):
                if not curr:
                    break
                prev = curr
                curr = curr.next
            # Set the next n-1 nodes as 'curr'.
            for _ in range(n):
                if not curr:
                    break
                curr = curr.next
            # Delete the nodes in [1, n-1].
            prev.next = curr
        return head
```

```
Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteNodes(self, head: ListNode, m: int, n: int) -> ListNode:
        pre = head
        while pre:
            for _ in range(m - 1):
                pre = pre.next
            if pre is None:
                return head
            cur = pre
            for _ in range(n):
                if cur:
                    cur = cur.next
            pre.next = None if cur is None else cur.next
            pre = pre.next
        return head
```

Linked List · LeetCode · By Pattern

— 1790 —

LeetCode

```
Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteNodes(self, head: ListNode, m: int, n: int) -> ListNode:
        pre = head
        while pre:
            for _ in range(m - 1):
                if pre:
                    pre = pre.next
            if pre is None:
                return head
            cur = pre
            for _ in range(n):
                if cur:
                    cur = cur.next
            pre.next = None if cur is None else cur.next
            pre = pre.next
        return head
```

```
TypeScript
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function deleteNodes(head: ListNode | null, m: number, n: number): ListNode | null {
    let pre = head;
    while (pre) {
        for (let i = 0; i < m - 1 && pre; i++) {
            pre = pre.next;
        }
        if (!pre) {
            break;
        }
        let cur = pre;
        for (let i = 0; i < n && cur; i++) {
            cur = cur.next;
        }
        pre.next = cur?.next || null;
        pre = pre.next;
    }
    return head;
}
```

Linked List · LeetCode · By Pattern

— 1791 —

LeetCode

```
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function deleteNodes(head: ListNode | null, m: number, n: number): ListNode | null {
    let pre = head;
    while (pre) {
        for (let i = 0; i < m - 1 && pre; i++) {
            pre = pre.next;
        }
        if (!pre) {
            break;
        }
        let cur = pre;
        for (let i = 0; i < n && cur; i++) {
            cur = cur.next;
        }
        pre.next = cur?.next || null;
        pre = pre.next;
    }
    return head;
}
```

[*] Linked List — Pattern Solution (matched from community answers)

Python

```
class Solution:
    def deleteNodes(self, head: ListNode | None, m: int, n: int) -> ListNode | None:
        curr = head
        prev = None # prev.next == curr

        while curr:
            # Set the next node as 'prev'.
            for _ in range(m):
                if not curr:
                    break
                prev = curr
                curr = curr.next
            # Set the next n-1 nodes as 'curr'.
            for _ in range(n):
                if not curr:
                    break
                curr = curr.next
            # Delete the nodes in [1, n-1].
            prev.next = curr
        return head
```

#2 MEDIUM

Add Two Numbers

LeetCode · Simple · Rust · C++ · LeetCode · Editorial

Linked List · Math · Recursion

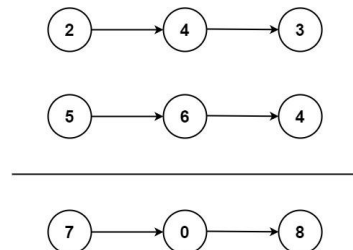
LeetCode 169

Problem:

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,8,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

>> Brute Force [Linked List] Interview Prep

Python

Linked List · LeetCode · By Pattern

— 1793 —

LeetCode

Linked List · LeetCode · By Pattern

— 1794 —

LeetCode

```

# Brute Force Approach
class Solution:
    def addTwoNumbers(self, l1, l2):
        # Brute Force - convert both lists to integers, add, convert back
        def to_int_list(num, mul = 0, l):
            while num:
                num = num.val + mul
                mul *= 10
            return num
        total = to_int(l1) + to_int(l2)
        dummy = cur = ListNode(0)
        if total == 0: return ListNode(0)
        while total:
            curr.next = ListNode(total % 10)
            cur = curr.next
            total //= 10
        return dummy.next

```

Python

Python

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def addTwoNumbers(l1, l2):
    # Initialize a dummy head node to simplify the result list construction.
    dummy = ListNode()
    current = dummy
    carry = 0

    # Traverse through both linked lists until all nodes and any carry are processed.
    while l1 or l2 or carry:
        # Get the current values; if a node is missing, use 0.
        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0

        # Calculate the sum of current digits and carry.
        total = val1 + val2 + carry
        carry = total // 10 # Update carry for next iteration.
        digit = total % 10 # Compute current digit.

        # Append the digit as a new node in the result list.
        current.next = ListNode(digit)
        current = current.next

```

Linked List · LeetCode - By Pattern

— 1795 —

LeetCode

```

current = current.next

# Move to the next nodes if available.
if l1:
    l1 = l1.next
if l2:
    l2 = l2.next

# Return the head of the newly formed linked list.
return dummy.next

```

Python

Python

```

class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        carry = 0

        while carry or l1 or l2:
            if l1:
                carry += l1.val
                l1 = l1.next
            if l2:
                carry += l2.val
                l2 = l2.next
            curr.next = ListNode(carry % 10)
            carry //= 10
            curr = curr.next

        return dummy.next

```

Python

Python

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
        carry, curr = 0, dummy
        while l1 or l2 or carry:
            s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, val = divmod(s, 10)
            curr.next = ListNode(val)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next

```

Python

Python

Linked List · LeetCode - By Pattern — 1796 — LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
        carry, curr = 0, dummy
        while l1 or l2 or carry:
            s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, val = divmod(s, 10)
            curr.next = ListNode(val)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next

```

Java

Java

```

//
// Definition for singly-linked list.
// public class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int val=0, ListNode next=null) {
//         this.val = val;
//         this.next = next;
//     }
// }
public class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode();
        ListNode cur = dummy;
        while (l1 != null || l2 != null || carry != 0) {
            int s = (l1 != null ? l1.val : 0) + (l2 != null ? l2.val : 0) + carry;
            carry = s / 10;
            cur.next = new ListNode(s % 10);
            cur = cur.next;
            l1 = l1 != null ? l1.next : null;
            l2 = l2 != null ? l2.next : null;
        }
        return dummy.next;
    }
}

```

Java

Java

Linked List · LeetCode - By Pattern

— 1797 —

LeetCode

```

//
// Definition for singly-linked list.
// public class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int val=0, ListNode next=null) {
//         this.val = val;
//         this.next = next;
//     }
// }
public class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode();
        ListNode cur = dummy;
        while (l1 != null || l2 != null || carry != 0) {
            int s = (l1 != null ? l1.val : 0) + (l2 != null ? l2.val : 0) + carry;
            carry = s / 10;
            cur.next = new ListNode(s % 10);
            cur = cur.next;
            l1 = l1 != null ? l1.next : null;
            l2 = l2 != null ? l2.next : null;
        }
        return dummy.next;
    }
}

```

JavaScript

JavaScript

```

//
// Definition for singly-linked list.
// function ListNode(val, next) {
//     this.val = (val===undefined ? 0 : val)
//     this.next = (next===undefined ? null : next)
// }
//
// @param {ListNode} l1
// @param {ListNode} l2
// @return {ListNode}
var addTwoNumbers = function(l1, l2) {
    const dummy = new ListNode();
    let cur = dummy;
    while (l1 || l2 || carry) {
        const s = ((l1.val || 0) + (l2.val || 0) + carry);
        carry = Math.floor(s / 10);
        cur.next = new ListNode(s % 10);
        cur = cur.next;
        l1 = l1.next;
        l2 = l2.next;
    }
    return dummy.next;
};

```

Linked List · LeetCode - By Pattern

— 1798 —

LeetCode

```

})

//
// Definition for singly-linked list.
// class ListNode {
//     constructor(val, next) {
//         this.val = val;
//         this.next = next;
//     }
// }
// @param {ListNode} l1
// @param {ListNode} l2
// @return {ListNode}
def add_two_numbers(l1, l2):
    dummy = ListNode(0)
    cur = dummy
    while l1 or l2 or carry:
        s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
        carry, val = divmod(s, 10)
        cur.next = ListNode(val)
        cur = cur.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next

```

JavaScript

JavaScript

```

//
// Definition for singly-linked list.
// function ListNode(val, next) {
//     this.val = (val===undefined ? 0 : val)
//     this.next = (next===undefined ? null : next)
// }
//
// @param {ListNode} l1
// @param {ListNode} l2
// @return {ListNode}
var addTwoNumbers = function(l1, l2) {
    const dummy = new ListNode();
    let cur = dummy;
    while (l1 || l2 || carry) {
        const s = ((l1.val || 0) + (l2.val || 0) + carry);
        carry = Math.floor(s / 10);
        cur.next = new ListNode(s % 10);
        cur = cur.next;
        l1 = l1.next;
        l2 = l2.next;
    }
    return dummy.next;
};

```

Linked List · LeetCode - By Pattern

— 1799 —

LeetCode

```

//
// Definition for singly-linked list.
// class ListNode {
//     constructor(val, next) {
//         this.val = val;
//         this.next = next;
//     }
// }
// @param {ListNode} l1
// @param {ListNode} l2
// @return {ListNode}
def add_two_numbers(l1, l2):
    dummy = ListNode(0)
    cur = dummy
    while l1 or l2 or carry:
        s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
        carry, val = divmod(s, 10)
        cur.next = ListNode(val)
        cur = cur.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None
    return dummy.next

```

TypeScript

TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }
//
function addTwoNumbers(l1: ListNode | null, l2: ListNode | null): ListNode | null {
    const dummy = new ListNode();
    let cur = dummy;
    let sum = 0;
    while (l1 != null || l2 != null || sum !== 0) {
        if (l1 != null) {
            sum += l1.val;
            l1 = l1.next;
        }
        if (l2 != null) {
            sum += l2.val;
            l2 = l2.next;
        }
    }
}

```

Linked List · LeetCode - By Pattern

— 1800 —

LeetCode


```
        cur.next = new ListNode(sum % 10);
        cur = cur.next;
        sum = Math.floor(sum / 10);
    }
    return dummy.next;
}

TypeScript
//
// Definition for singly-linked list.
// class ListNode {
//   val: number;
//   next: ListNode | null;
// }
// constructor(val: number, next?: ListNode | null) {
//   this.val = (val===undefined ? 0 : val);
//   this.next = (next===undefined ? null : next);
// }
//

function addTwoNumbers(l1: ListNode | null, l2: ListNode | null): ListNode | null {
    const dummy = new ListNode();
    let cur = dummy;
    let sum = 0;
    while (sum !== 0 || l1 !== null || l2 !== null) {
        if (l1 !== null) {
            sum += l1.val;
            l1 = l1.next;
        }
        if (l2 !== null) {
            sum += l2.val;
            l2 = l2.next;
        }
        cur.next = new ListNode(sum % 10);
        cur = cur.next;
        sum = Math.floor(sum / 10);
    }
    return dummy.next;
}
```

TypeScript

Linked List · LeetCode · By Pattern

— 1801 —

LeetCode

```
import std::strutils, algorithm

type
  Node[int] = ref object
    value: int
    next: Node[int]

  SinglyLinkedList[T] = object
    head, tail: Node[T]

proc append[T](list: var SinglyLinkedList[T], data: T = nil): void =
  var node = Node[T](value: data)
  if list.head.isNil:
    list.head = node
    list.tail = node
  else:
    list.tail.next = node
    list.tail = node

proc preview[T](list: SinglyLinkedList[T]): string =
  var s: seq[T]
  var n = list.head
  while not n.isNil:
    s.add n.value
    n = n.next

  result = s.join("→ ")

proc addTwoNumbers(l1: var SinglyLinkedList, l2: var SinglyLinkedList): SinglyLinkedList[int] =
  var
    aggregate: SinglyLinkedList
    psum: seq[int]
    temp_l1, temp_l2: seq[int]

  while not l1.head.isNil:
    temp_l1.add(l1.head.value)
    l1.head = l1.head.next
  while not l2.head.isNil:
    temp_l2.add(l2.head.value)
    l2.head = l2.head.next

  psum = reversed(s(reversed(temp_l1).join("").parseInt() + reversed(temp_l2).join("").parseInt()))
  for i in psum: aggregate.append(i.parseInt())

  result = aggregate

var list1: SinglyLinkedList[int]
var list2: SinglyLinkedList[int]

for i in {2, 4, 3}: list1.append(i)
for i in {5, 6, 4}: list2.append(i)

echo(preview(list1))
echo(preview(list2))
echo(preview(addTwoNumbers(list1, list2)))
```

Linked List · LeetCode · By Pattern

— 1802 —

LeetCode

```
Rust
//
// Definition for singly-linked list.
// struct ListNode {
//   val: i32;
//   next: Option,
        mut l2: Option,
    ) -> Option {
        let mut dummy = Some(Box::new(ListNode::new(0)));
        let mut cur = &mut dummy;
        let mut sum = 0;
        while l1.is_some() || l2.is_some() || sum != 0 {
            if let Some(node) = l1 {
                sum += node.val;
                l1 = node.next;
            }
            if let Some(node) = l2 {
                sum += node.val;
                l2 = node.next;
            }
            cur.as_mut().unwrap().next = Some(Box::new(ListNode::new(sum % 10)));
            cur = &mut cur.as_mut().unwrap().next;
            sum /= 10;
        }
        dummy.unwrap().next.take()
    }
}
```

Rust

Linked List · LeetCode · By Pattern

— 1803 —

LeetCode

```
//
// Definition for singly-linked list.
// struct ListNode {
//   val: i32;
//   next: Option,
        mut l2: Option,
    ) -> Option {
        let mut dummy = Some(Box::new(ListNode::new(0)));
        let mut cur = &mut dummy;
        let mut sum = 0;
        while l1.is_some() || l2.is_some() || sum != 0 {
            if let Some(node) = l1 {
                sum += node.val;
                l1 = node.next;
            }
            if let Some(node) = l2 {
                sum += node.val;
                l2 = node.next;
            }
            cur.as_mut().unwrap().next = Some(Box::new(ListNode::new(sum % 10)));
            cur = &mut cur.as_mut().unwrap().next;
            sum /= 10;
        }
        dummy.unwrap().next.take()
    }
}
```

[*] Linked List — Pattern Solution (matched from community answers)

Python

Linked List · LeetCode · By Pattern

— 1804 —

LeetCode

```
#
// Definition for singly-linked list.
// class ListNode {
//   val: number;
//   next: ListNode | null;
// }
// constructor(val: number, next?: ListNode | null) {
//   this.val = (val===undefined ? 0 : val);
//   this.next = (next===undefined ? null : next);
// }
//

def addTwoNumbers(l1: ListNode | null, l2: ListNode | null): ListNode | null {
    // Definition of dummy head node to simplify the result list construction.
    dummy = ListNode()
    current = dummy
    carry = 0

    // Traverse through both linked lists until all nodes and any carry are processed.
    while l1 or l2 or carry:
        // For the current nodes: if a node is missing, use 0.
        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0

        // Calculate the sum of current digits and carry.
        total = val1 + val2 + carry
        carry = total // 10 # Update carry for next iteration.
        digit = total % 10 # Compute current digit.

        # Append the digit as a new node in the result list.
        current.next = ListNode(digit)

        current = current.next

    # Move to the next nodes if available.
    if l1:
        l1 = l1.next
    if l2:
        l2 = l2.next

    # Return the head of the newly formed linked list.
    return dummy.next
```

#19

MEDIUM

Remove Nth Node From End of List

LeetCode · SinglyList · MEDIUM · LeetCode · Editorial

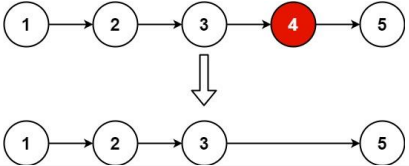
Linked List · Two Pointers

Lists Grid 169

Problem:

Given the head of a linked list, remove the nth node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1

Output: []

Example 3:

Input: head = [1,2], n = 1

Output: [1]

Constraints:

• The number of nodes in the list is sz.

• 1 <= sz <= 30

• 0 <= Node.val <= 100

• 1 <= n <= sz

Follow up: Could you do this in one pass?

Python

Python

Linked List · LeetCode · By Pattern

— 1806 —

LeetCode

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # holds the value of the node
        self.next = next # Points to the next node in the list

class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        # Create a dummy node to simplify edge cases (like removing the head)
        dummy = ListNode(0)
        dummy.next = head

        # Initialize two pointers starting at the dummy node
        first = dummy
        second = dummy

        # Advance slow pointer n-1 steps ahead to maintain a gap of n nodes
        for _ in range(n - 1):
            first = first.next

        # Move both pointers until first reaches the end of the list
        while first:
            first = first.next
            second = second.next

        # Skip the node to remove if it's from the list
        second.next = second.next.next

        # Return the adjusted list starting from dummy.next
        return dummy.next

```

Python

```

class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        slow = head
        fast = head

        for _ in range(n):
            fast = fast.next
            if not fast:
                return head.next

        while fast.next:
            slow = slow.next
            fast = fast.next
            slow.next = slow.next.next

        return head

```

Python

Linked List · LeetCode · By Pattern — 1807 — LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        slow = dummy
        fast = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next

```

Python

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next

```

JavaScript

JavaScript

Linked List · LeetCode · By Pattern — 1808 — LeetCode

```

//
// Definition for singly-linked list.
// function ListNode(val, next) {
//   this.val = (val===undefined ? 0 : val)
//   this.next = (next===undefined ? null : next)
// }
//
// @param {ListNode} head
// @param {number} n
// @return {ListNode}
var removeNthFromEnd = function(head, n) {
    const dummy = new ListNode(0, head);
    let fast = dummy;
    slow = dummy;
    while (true) {
        fast = fast.next;
        while (fast.next) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
};

```

JavaScript

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
# @param {ListNode} head
# @param {number} n
# @return {ListNode}
def removeNthFromEnd(head: Optional[ListNode], n: int) -> Optional[ListNode]:
    dummy = ListNode(0, head)
    fast = slow = dummy
    while n > 0:
        fast = fast.next
        n -= 1
    end = fast.next
    while fast.next:
        slow = slow.next
        fast = fast.next
    end.next = slow.next.next
    return dummy.next

```

JavaScript

Linked List · LeetCode · By Pattern — 1809 — LeetCode

```

//
// Definition for singly-linked list.
// function ListNode(val, next) {
//   this.val = (val===undefined ? 0 : val)
//   this.next = (next===undefined ? null : next)
// }
//
// @param {ListNode} head
// @param {number} n
// @return {ListNode}
var removeNthFromEnd = function(head, n) {
    const dummy = new ListNode(0, head);
    let fast = dummy;
    slow = dummy;
    while (true) {
        fast = fast.next;
        while (fast.next) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
};

```

JavaScript

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
# @param {ListNode} head
# @param {number} n
# @return {ListNode}
def removeNthFromEnd(head: Optional[ListNode], n: int) -> Optional[ListNode]:
    dummy = ListNode(0, head)
    fast = slow = dummy
    while n > 0:
        fast = fast.next
        n -= 1
    end = fast.next
    while fast.next:
        slow = slow.next
        fast = fast.next
    end.next = slow.next.next
    return dummy.next

```

JavaScript

Linked List · LeetCode · By Pattern — 1810 — LeetCode

```

//
// Definition for singly-linked list.
// class ListNode {
//   val: number;
//   next: ListNode | null;
//   constructor(val?: number, next?: ListNode | null) {
//     this.val = (val===undefined ? 0 : val)
//     this.next = (next===undefined ? null : next)
//   }
// }
//
function removeNthFromEnd(head: ListNode | null, n: number): ListNode | null {
    const dummy = new ListNode(0, head);
    let fast = dummy;
    let slow = dummy;
    while (true) {
        fast = fast.next;
        while (fast.next) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}

```

TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//   val: number;
//   next: ListNode | null;
//   constructor(val?: number, next?: ListNode | null) {
//     this.val = (val===undefined ? 0 : val)
//     this.next = (next===undefined ? null : next)
//   }
// }
//
function removeNthFromEnd(head: ListNode | null, n: number): ListNode | null {
    const dummy = new ListNode(0, head);
    let fast = dummy;
    let slow = dummy;
    while (true) {
        fast = fast.next;
        while (fast.next) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}

```

Rust

Linked List · LeetCode · By Pattern — 1811 — LeetCode

```

//
// Definition for singly-linked list.
// struct ListNode {
//   int val;
//   ListNode* next;
//   ListNode(int x) : val(x), next(nullptr) {}
//   ~ListNode() {}
// };
//
// @param {ListNode} head
// @param {number} n
// @return {ListNode}
var removeNthFromEnd = function(head, n) {
    const dummy = new ListNode(0, head);
    let fast = dummy;
    let slow = dummy;
    while (true) {
        fast = fast.next;
        while (fast.next) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}

```

Rust

```

//
// Definition for singly-linked list.
// struct ListNode {
//   int val;
//   ListNode* next;
//   ListNode(int x) : val(x), next(nullptr) {}
//   ~ListNode() {}
// };
//
// @param {ListNode} head
// @param {number} n
// @return {ListNode}
var removeNthFromEnd = function(head, n) {
    const dummy = new ListNode(0, head);
    let fast = dummy;
    let slow = dummy;
    while (true) {
        fast = fast.next;
        while (fast.next) {
            slow = slow.next;
            fast = fast.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}

```

Linked List · LeetCode · By Pattern — 1812 — LeetCode


```

//
// Definition for singly-linked list.
// function ListNode(val, next) {
//   this.val = (val===undefined ? 0 : val)
//   this.next = (next===undefined ? null : next)
// }
//
// @param {ListNode} head
// @return {ListNode}
var swapPairs = function(head) {
  if (!head || !head.next) {
    return head;
  }
  const t = swapPairs(head.next.next);
  const p = head.next;
  p.next = head;
  head.next = t;
  return p;
};

```

JavaScript

```

// Definition for singly-linked list.
// class ListNode {
//   attr: <number> val; next
//   constructor(val = 0, _next = null) {
//     this.val = val;
//     this.next = _next;
//   }
// }
// @param {ListNode} head
// @return {ListNode}
def swapPairs(head):
    dummy = ListNode(0, head)
    pre = dummy
    cur = head
    while (cur.next and cur.next.next):
        t = cur.next
        cur.next = t.next
        t.next = cur
        pre.next = t
        pre = cur
        cur = cur.next
    end
    dummy.next
end

```

JavaScript

Linked List - LeetCode - By Pattern

— 1819 —

LeetCode

```

//
// Definition for singly-linked list.
// function ListNode(val, next) {
//   this.val = (val===undefined ? 0 : val)
//   this.next = (next===undefined ? null : next)
// }
//
// @param {ListNode} head
// @return {ListNode}
var swapPairs = function(head) {
  const dummy = new ListNode(0, head);
  let [pre, cur] = [dummy, head];
  while (cur && cur.next) {
    const t = cur.next;
    cur.next = t.next;
    t.next = cur;
    pre.next = t;
    [pre, cur] = [cur, cur.next];
  }
  return dummy.next;
};

```

JavaScript

```

// Definition for singly-linked list.
// class ListNode {
//   attr: <number> val; next
//   constructor(val = 0, _next = null) {
//     this.val = val;
//     this.next = _next;
//   }
// }
// @param {ListNode} head
// @return {ListNode}
def swapPairs(head):
    dummy = ListNode(0, head)
    pre = dummy
    cur = head
    while (cur.next and cur.next.next):
        t = cur.next
        cur.next = t.next
        t.next = cur
        pre.next = t
        pre = cur
        cur = cur.next
    end
    dummy.next
end

```

TypeScript

TypeScript

Linked List - LeetCode - By Pattern

— 1820 —

LeetCode

```

//
// Definition for singly-linked list.
// class ListNode {
//   val: number
//   next: ListNode | null
//   constructor(val?: number, next?: ListNode | null) {
//     this.val = (val===undefined ? 0 : val)
//     this.next = (next===undefined ? null : next)
//   }
// }
//
function swapPairs(head: ListNode | null): ListNode | null {
  if (!head || !head.next) {
    return head;
  }
  const t = swapPairs(head.next.next);
  const p = head.next;
  p.next = head;
  head.next = t;
  return p;
}

```

TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//   val: number
//   next: ListNode | null
//   constructor(val?: number, next?: ListNode | null) {
//     this.val = (val===undefined ? 0 : val)
//     this.next = (next===undefined ? null : next)
//   }
// }
//
function swapPairs(head: ListNode | null): ListNode | null {
  const dummy = new ListNode(0, head);
  let [pre, cur] = [dummy, head];
  while (cur && cur.next) {
    const t = cur.next;
    cur.next = t.next;
    t.next = cur;
    pre.next = t;
    [pre, cur] = [cur, cur.next];
  }
  return dummy.next;
}

```

Rust

Rust

Linked List - LeetCode - By Pattern

— 1821 —

LeetCode

```

// Definition for singly-linked list.
// #[derive(PartialEq, Eq, Clone, Debug)]
// pub struct ListNode {
//   pub val: i32,
//   pub next: Option

```

Rust

Linked List - LeetCode - By Pattern

— 1822 —

LeetCode

```

// Definition for singly-linked list.
// #[derive(PartialEq, Eq, Clone, Debug)]
// pub struct ListNode {
//   pub val: i32,
//   pub next: Option

```

[9] Linked List - Pattern Solution (matched from community answers)

Python

Linked List - LeetCode - By Pattern

— 1823 —

LeetCode

```

// Definition for singly-linked list.
// class ListNode {
//   attr: <number> val; next: ListNode | null
//   constructor(val = 0, next = null) {
//     this.val = val;
//     this.next = next;
//   }
// }
//
// class Solution {
//   def swapPairs(self, head: ListNode) -> ListNode:
//     # Create a dummy node and point it to the head of the list.
//     dummy = ListNode(0)
//     dummy.next = head
//     # Initialize the pointer 'prev' to dummy.
//     prev = dummy
//
//     # Traverse the list while at least two nodes remain.
//     while head and head.next:
//       # Swap the two nodes to swap.
//       first_node = head
//       second_node = head.next
//
//       # Swapping the nodes by reassigning pointers.
//       prev.next = second_node # Link previous node to second node.
//       first_node.next = second_node.next # Link first node to node after second.
//       second_node.next = first_node # Link second node to first to complete swap.
//
//       # Update the pointers for next pair processing.
//       prev = first_node # Now 'prev' pointer to the end of the swapped pair.
//       head = first_node.next # Now 'head' pointer to the next pair.
//
//     # Return the new head of the list.
//     return dummy.next
// }

```

#61

MEDIUM

Rotate List

LeetCode - LeetCode - LeetCode - LeetCode - LeetCode

Linked List - Two Pointers

List: Grid 169

Problem:

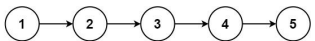
Given the head of a linked list, rotate the list to the right by k places.

Example 1:

Linked List - LeetCode - By Pattern

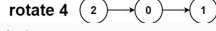
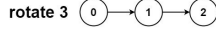
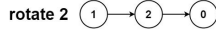
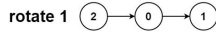
— 1824 —

LeetCode



Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]

Example 2:



Input: head = [0,1,2], k = 4
Output: [2,0,1]

Constraints:

- The number of nodes in the list is in the range [0, 500].
- $-100 \leq \text{Node.val} \leq 100$
- $0 \leq k \leq 2 \cdot 10^9$

```
Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0: # Check if rotation is necessary
            return head

        # Compute the length of the list and locate the tail node
        length = 1
        tail = head
        while tail.next:
            tail = tail.next
            length += 1

        # Calculate the effective number of rotations
        k = k % length
        if k == 0:
            return head

        # Find the new tail: (length - k - 1) steps from the head
        stepsToNewTail = length - k - 1

        newTail = head
        for i in range(stepsToNewTail):
            newTail = newTail.next

        # Set the new head and break the list
        newHead = newTail.next
        newTail.next = None

        # Connect the old tail to the original head
        tail.next = head

        return newHead
```

Python

```
class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0:
            return head

        tail = head
        length = 1
        while tail.next:
            tail = tail.next
            length += 1
        tail.next = head # Circle the list.

        t = length - k % length
        for i in range(t):
            tail = tail.next
            newHead = tail.next
            tail.next = None
        return newHead
```

```
Python
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        cur = head
        while cur:
            k -= 1
            cur = cur.next
            k %= k
        if k == 0:
            return head
        fast = slow = head
        for i in range(k):
            fast = fast.next
        while fast.next:
            fast, slow = fast.next, slow.next
        ans = slow.next
        slow.next = None
        fast.next = head
        return ans
```

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = None

class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        cur = head
        while cur:
            k -= 1
            cur = cur.next
            k %= k
        if k == 0:
            return head
        fast = slow = head
        for i in range(k):
            fast = fast.next
        while fast.next:
            fast, slow = fast.next, slow.next
        ans = slow.next
        slow.next = None
        fast.next = head
        return ans
```

```
Java
/*
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */

public class Solution {
    public ListNode rotateRight(ListNode head, int k) {
        if (head == null || head.next == null) {
            return head;
        }
        var cur = head;
        int n = 0;
        while (cur != null) {
            cur = cur.next;
            ++n;
        }
        k %= n;
        if (k == 0) {
            return head;
        }
    }
}
```

```
}
var fast = head;
var slow = head;
while (k-- > 0) {
    fast = fast.next;
}
while (fast.next != null) {
    fast = fast.next;
    slow = slow.next;
}
var ans = slow.next;
slow.next = null;
fast.next = head;
return ans;
}
```

```
Java
/*
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */

public class Solution {
    public ListNode rotateRight(ListNode head, int k) {
        if (head == null || head.next == null) {
            return head;
        }
        var cur = head;
        int n = 0;
        while (cur != null) {
            cur = cur.next;
            ++n;
        }
        k %= n;
        if (k == 0) {
            return head;
        }
    }
}
```

```
}
var fast = head;
var slow = head;
while (k-- > 0) {
    fast = fast.next;
}
while (fast.next != null) {
    fast = fast.next;
    slow = slow.next;
}
var ans = slow.next;
slow.next = null;
fast.next = head;
return ans;
}
```

```
TypeScript
/*
 * Definition for singly-linked list.
 * class ListNode {
 *     val: number;
 *     next: ListNode | null;
 *     constructor(val?: number, next?: ListNode | null) {
 *         this.val = (val === undefined ? 0 : val);
 *         this.next = (next === undefined ? null : next);
 *     }
 * }
 */

function rotateRight(head: ListNode | null, k: number): ListNode | null {
    if (!head || !head.next) {
        return head;
    }
    let cur = head;
    let n = 0;
    while (cur) {
        cur = cur.next;
        ++n;
    }
    k %= n;
    if (k === 0) {
        return head;
    }
}
```

```

    }
    let fast = head;
    let slow = head;
    while (k--) {
        fast = fast.next;
    }
    while (fast.next) {
        fast = fast.next;
        slow = slow.next;
    }
    const ans = slow.next;
    slow.next = null;
    fast.next = head;
    return ans;
}

TypeScript
//
// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
// }
// function rotateRight(head: ListNode | null, k: number): ListNode | null {
//     if (head === null || head.next === null) {
//         return head;
//     }
//     let cur = head;
//     let n = 0;
//     while (cur) {
//         cur = cur.next;
//         n++;
//     }
//     if (k === 0) {
//         return head;
//     }
//     let fast = head;
//     let slow = head;
//     while (k--) {
//         fast = fast.next;
//     }
//     while (fast.next) {
//         fast = fast.next;
//         slow = slow.next;
//     }
//     const ans = slow.next;
//     slow.next = null;
//     fast.next = head;
//     return ans;
// }

function rotateRight(head: ListNode | null, k: number): ListNode | null {
    if (head === null || head.next === null) {
        return head;
    }
    let cur = head;
    let n = 0;
    while (cur) {
        cur = cur.next;
        n++;
    }
    if (k === 0) {
        return head;
    }
    let fast = head;
    let slow = head;
    while (k--) {
        fast = fast.next;
    }
    while (fast.next) {
        fast = fast.next;
        slow = slow.next;
    }
    const ans = slow.next;
    slow.next = null;
    fast.next = head;
    return ans;
}

```

Linked List · LeetCode · By Pattern

— 1831 —

LeetCode

```

Rust
Rust
// Definition for singly-linked list.
// struct ListNode {
//     val: i32;
//     next: Option
```

Linked List · LeetCode · By Pattern

— 1832 —

LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val):
#         self.val = val
#         self.next = None
class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0: # Check if rotation is necessary
            return head
        # Compute the length of the list and locate the tail node
        length = 1
        tail = head
        while tail.next:
            tail = tail.next
            length += 1
        # Calculate the effective number of rotations
        k = k % length
        if k == 0:
            return head
        # Find the new tail: (length - k - 1) steps from the head
        stepsToNewTail = length - k - 1
        newTail = head
        for _ in range(stepsToNewTail):
            newTail = newTail.next
        # Set the new head and break the list
        newHead = newTail.next
        newTail.next = None
        # Connect the old tail to the original head
        tail.next = head
        return newHead

```

#146

MEDIUM

LRU Cache

LeetCode · Interview · Wiki · LeetCode · Editorial

Hash Table · Linked List · Design

List · Grind 169

Problem:

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

- LRUCache(int capacity) Initialize the LRU cache with positive size capacity.
- get(int key) Return the value of the key if the key exists, otherwise return -1.
- void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict

Linked List · LeetCode · By Pattern

— 1833 —

LeetCode

```

# Define a node for the doubly linked list.
class Node:
    def __init__(self, key, value):
        self.key = key # key of the node
        self.value = value # value of the node
        self.prev = None # pointer to previous node
        self.next = None # pointer to next node

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity # maximum capacity of the cache
        self.cache = {} # dictionary mapping key -> node
        # Create dummy head and tail nodes
        self.head = Node(0, 0)
        self.tail = Node(0, 0)
        self.head.next = self.tail
        self.tail.prev = self.head

    # Helper function to remove a node from the linked list.
    def _remove(self, node):
        prev_node = node.prev
        next_node = node.next
        prev_node.next = next_node
        next_node.prev = prev_node

    # Helper function to add a node right after the head.
    def _add(self, node):
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    # Returns the value of the key if it exists; otherwise, returns -1.
    def get(self, key):
        if key in self.cache:
            node = self.cache[key]
            # Move the accessed node to the head (marking it as recently used)
            self._remove(node)
            self._add(node)
            return node.value
        return -1

    # Updates the value of the key if it exists; otherwise, adds the key-value pair.
    # Removes the least recently used key if the cache exceeds its capacity.
    def put(self, key, value):
        if key in self.cache:
            # Remove the old node
            self._remove(self.cache[key])
            node = Node(key, value)
            self._add(node)
        else:
            self.cache[key] = node
            if len(self.cache) > self.capacity:
                # Remove the least recently used element.
                lru = self.tail.prev
                self._remove(lru)
                del self.cache[lru.key]

```

Python

Linked List · LeetCode · By Pattern

— 1835 —

LeetCode

```

class Node:
    def __init__(self, key, int, value: int):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.keyToNode = {}
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.join(self.head, self.tail)

    def get(self, key: int) -> int:
        if key not in self.keyToNode:
            return -1
        node = self.keyToNode[key]
        self.remove(node)
        self.moveToHead(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.keyToNode:
            node = self.keyToNode[key]
            node.value = value
            self.remove(node)
            self.moveToHead(node)
            return
        if len(self.keyToNode) == self.capacity:
            lastNode = self.tail.prev
            del self.keyToNode[lastNode.key]
            self.remove(lastNode)
        self.moveToHead(Node(key, value))
        self.keyToNode[key] = self.head.next
        self.remove(self.head)

    def join(self, node1: Node, node2: Node):
        node1.next = node2
        node2.prev = node1

    def moveToHead(self, node: Node):
        self.join(node, self.head.next)
        self.join(self.head, node)

    def remove(self, node: Node):
        self.join(node.prev, node.next)

```

Python

Linked List · LeetCode · By Pattern

— 1836 —

LeetCode

```

class Node:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.cache = {} # key -> Node(val)
        self.head = Node() # dummy node
        self.tail = Node() # dummy node
        self.capacity = capacity
        self.size = 0
        self.head.next = self.tail # note: key setup
        self.tail.prev = self.head

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self.move_to_head(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            node = self.cache[key]
            node.val = value
            self.move_to_head(node)
        else:
            node = Node(key, value)
            self.cache[key] = node
            self.add_to_head(node)
            self.size += 1
            if self.size > self.capacity:
                tail = self.remove_tail()
                self.cache.pop(tail.key)
                self.size -= 1

    def move_to_head(self, node):
        self.remove_node(node)
        self.add_to_head(node)

    def remove_node(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def add_to_head(self, node):
        node.next = self.head.next
        node.prev = self.head

```

```

        node.prev = self.head
        self.head.next = node
        node.next.prev = node

    def remove_tail(self):
        node = self.tail.prev
        self.remove_node(node)
        return node

# Your LRUCache object will be instantiated and called as such:
# obj = LRUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key, value)

#####
...
example:
>>> obj = collections.OrderedDict()
>>> obj = LRUCache(capacity)
>>> obj.put(1,1)
>>> obj.put(2,2)
>>> obj.get(1)
1
>>> obj.get(2)
2
>>> obj.remove_lru()
OrderedDict([(1, 1), (2, 2), (3, 3)])
>>> obj.remove_lru()
>>> obj

```

C++
C++

```

struct Node {
    int key;
    int value;
    shared_ptr<Node> prev;
    shared_ptr<Node> next;
};

class LRUCache {
public:
    LRUCache(int capacity) : capacity(capacity) {
        join(head, tail);
    }

    int get(int key) {
        const auto it = keyToNode.find(key);
        if (it == keyToNode.cend())
            return -1;
        shared_ptr<Node> node = it->second;
        remove(node);
        moveToHead(node);
        return node->value;
    }

    void put(int key, int value) {
        if (const auto it = keyToNode.find(key); it != keyToNode.cend()) {
            shared_ptr<Node> node = it->second;
            node->value = value;
            remove(node);
            moveToHead(node);
        }

        if (keyToNode.size() == capacity) {
            shared_ptr<Node> lastNode = tail->prev;
            keyToNode.erase(lastNode->key);
            remove(lastNode);
        }

        moveToHead(make_shared<Node>(key, value));
        keyToNode[key] = head->next;
    }

private:
    const int capacity;
    unordered_map<int, shared_ptr<Node>> keyToNode;
    shared_ptr<Node> head = make_shared<Node>(-1, -1);
    shared_ptr<Node> tail = make_shared<Node>(-1, -1);

    void join(shared_ptr<Node> node1, shared_ptr<Node> node2) {

```

```

        node1->next = node2;
        node2->prev = node1;
    }

    void moveToHead(shared_ptr<Node> node) {
        join(node, head->next);
        join(head, node);
    }

    void remove(shared_ptr<Node> node) {
        join(node->prev, node->next);
    }
};

```

C++

```

struct Node {
    int key;
    int value;
};

class LRUCache {
public:
    LRUCache(int capacity) : capacity(capacity) {}

    int get(int key) {
        const auto it = keyToNode.find(key);
        if (it == keyToNode.cend())
            return -1;
        const auto& listIt = it->second;
        // Move it to the front
        cache.splice(cache.begin(), cache, listIt);
        return listIt->value;
    }

    void put(int key, int value) {
        // There's no capacity limit, so just remove the value.
        if (const auto it = keyToNode.find(key); it != keyToNode.cend()) {
            const auto& listIt = it->second;
            // Move it to the front

```

```

        cache.splice(cache.begin(), cache, listIt);
        listIt->value = value;
        return;
    }

    // Check the capacity
    if (cache.size() == capacity) {
        const Node& lastNode = cache.back();
        // That's why we store key in Node
        keyToNode.erase(lastNode.key);
        cache.pop_back();
    }

    cache.emplace_front(key, value);
    keyToNode[key] = cache.begin();
}

private:
    const int capacity;
    list<Node> cache;
    unordered_map<int, list<Node>::iterator> keyToNode;
};

```

C++

```

struct Node {
    int k;
    int v;
    Node* prev;
    Node* next;

    Node() : k(0), v(0) {}
    Node(int k, int v) : k(k), v(v) {}
    Node(int k, int v, Node* prev, Node* next) : k(k), v(v), prev(prev), next(next) {}
};

class LRUCache {
public:
    LRUCache(int capacity) : cap(capacity) {}
    int get(int key) {
        head = new Node();
        tail = new Node();
    }

```

```

        head->next = tail;
        tail->prev = head;
    }

    int get(int key) {
        if (cache.count(key)) return -1;
        Node* node = cache[key];
        moveToHead(node);
        return node->v;
    }

    void put(int key, int value) {
        if (cache.count(key)) {
            Node* node = cache[key];
            node->v = value;
            moveToHead(node);
        } else {
            Node* node = new Node(key, value);
            cache[key] = node;
            addToHead(node);
        }

        if (size == cap) {
            Node* removeTail();
            cache.erase(removeTail());
            --size;
        }
    }

private:
    unordered_map<int, Node*> cache;
    Node* head;
    Node* tail;
    int cap;
    int size;

    void moveToHead(Node* node) {
        removeNode(node);
        addToHead(node);
    }

    void removeNode(Node* node) {
        node->prev->next = node->next;
        node->next->prev = node->prev;
    }

    void addToHead(Node* node) {
        node->next = head->next;
        node->prev = head;
        head->next = node;
    }

    Node* removeTail() {
        Node* node = tail->prev;

```

```

java
class Node {
    public int key;
    public int value;
    public Node(int key, int value) {
        this.key = key;
        this.value = value;
    }
}

class LRUCache {
    public LRUCache(int capacity) {
        this.capacity = capacity;
    }

    public int get(int key) {
        if (!keyToNode.containsKey(key))
            return -1;

        Node node = keyToNode.get(key);
        cache.remove(node);
        cache.add(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (!keyToNode.containsKey(key)) {
            keyToNode.put(key, value);
            get(key);
            return;
        }

        if (cache.size() == capacity) {
            Node lastNode = cache.iterator().next();
            cache.remove(lastNode);
            keyToNode.remove(lastNode.key);
        }

        Node node = new Node(key, value);
        cache.add(node);
        keyToNode.put(key, node);
    }

    private int capacity;
    private Set<Node> cache = new LinkedHashSet<>();
    private Map<Integer, Node> keyToNode = new HashMap<>();
}

```

Java

```

class Node {
    int key;
    int val;
    Node prev;
    Node next;

    Node() {}

    Node(int key, int val) {
        this.key = key;
        this.val = val;
    }
}

class LRUCache {
    private Map<Integer, Node> cache = new HashMap<>();
    private Node head = new Node();
    private Node tail = new Node();
    private int capacity;
    private int size;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!cache.containsKey(key)) {
            return -1;
        }

        Node node = cache.get(key);
        moveToHead(node);
        return node.val;
    }

    public void put(int key, int value) {
        if (!cache.containsKey(key)) {
            Node node = new Node(key, value);
            cache.put(key, node);
            moveToHead(node);
            ++size;
            if (size > capacity) {
                node = removeTail();
                cache.remove(node.key);
            }
        } else {
            Node node = new Node(key, value);
            cache.put(key, node);
            moveToHead(node);
            ++size;
            if (size > capacity) {
                node = removeTail();
                cache.remove(node.key);
            }
        }
    }
}

```

Java

```

        --size;
    }
}

private void moveToHead(Node node) {
    removeNode(node);
    addNode(node);
}

private void removeNode(Node node) {
    Node prev = node.prev;
    Node next = node.next;
    prev.next = next;
}

private void addNode(Node node) {
    Node next = head.next;
    node.prev = head;
    head.next = node;
    node.next = next;
}

private Node removeTail() {
    Node node = tail.prev;
    removeNode(node);
    return node;
}

}
}

```

Java

```

public class LRUCache {
    class Node {
        public Node Prev;
        public Node Next;
        public int Key;
        public int Val;
    }

    private Node head = new Node();
    private Node tail = new Node();
    private Dictionary<int, Node> cache = new Dictionary<int, Node>();
    private int capacity;
    private int size;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.Next = tail;
        tail.Prev = head;
    }

    public int Get(int key) {
        Node node;
        if (cache.TryGetValue(key, out node)) {
            moveToHead(node);
            return node.Val;
        }
    }

    public void Put(int key, int value) {
        Node node;
        if (cache.TryGetValue(key, out node)) {
            moveToHead(node);
            node.Val = value;
        } else {
            Node node = new Node();
            node.Key = key;
            node.Val = value;
            node.Prev = head;
            head.Next = node;
            node.Next = tail;
            tail.Prev = node;
            cache.Add(key, node);
            ++size;
            if (size > capacity) {
                Node node = removeTail();
                cache.Remove(node.Key);
            }
        }
    }

    private Node removeTail() {
        Node node = tail.Prev;
        removeNode(node);
        return node;
    }

    private void removeNode(Node node) {
        Node prev = node.Prev;
        Node next = node.Next;
        prev.Next = next;
    }

    private void moveToHead(Node node) {
        removeNode(node);
        addNode(node);
    }

    private void addNode(Node node) {
        Node next = head.Next;
        node.Prev = head;
        head.Next = node;
        node.Next = next;
    }
}

```

Java

```

    }
    return -1;
}

public void put(int key, int val) {
    Node node;
    if (cache.containsKey(key, out node)) {
        moveToHead(node);
        node.val = val;
    } else {
        node = new Node() { Key = key, Val = val };
        cache.Add(key, node);
        moveToHead(node);
        if (++size > capacity) {
            node = removeTail();
            cache.Remove(node.Key);
            --size;
        }
    }
}

private void moveToHead(Node node) {
    removeNode(node);
    addNode(node);
}

private void removeNode(Node node) {
    Node prev = node.Prev;
    Node next = node.Next;
    prev.Next = next;
}

private void addNode(Node node) {
    Node next = head.Next;
    node.Prev = head;
    head.Next = node;
    node.Next = next;
}

private Node removeTail() {
    Node node = tail.Prev;
    removeNode(node);
    return node;
}

}
}

```

Java

```

use std::cell::RefCell;
use std::collections::HashMap;
use std::rc::Rc;

struct Node {
    key: i32,
    value: i32,
    prev: Option<Rc<RefCell<Node>>>,
    next: Option<Rc<RefCell<Node>>>,
}

impl Node {
    #[inline]
    fn new(key: i32, value: i32) -> Self {
        Self {
            key,
            value,
            prev: None,
            next: None,
        }
    }
}

struct LRUCache {
    capacity: usize,
    cache: HashMap<i32, Rc<RefCell<Node>>>,
    head: Option<Rc<RefCell<Node>>>,
    tail: Option<Rc<RefCell<Node>>>,
}

/*
 * Self means the method takes an immutable reference.
 * If you need a mutable reference, change it to 'let self' instead.
 */

impl LRUCache {
    fn new(capacity: i32) -> Self {
        Self {
            capacity: capacity as usize,
            cache: HashMap::new(),
            head: None,
            tail: None,
        }
    }

    fn get(&mut self, key: i32) -> i32 {
        match self.cache.get(&key) {
            Some(node) => {
                let node = Rc::clone(node);
                self.remove(&node);
                self.push_front(&node);
            }
        }
    }

    fn put(&mut self, key: i32, value: i32) {
        match self.cache.get(&key) {
            Some(node) => {
                let node = Rc::clone(node);
                self.remove(&node);
                self.push_front(&node);
            }
        }
    }
}

```

JavaScript

```

    let value = node.borrow().value;
    value;
}

}

}

fn put(&mut self, key: i32, value: i32) {
    match self.cache.get(&key) {
        Some(node) => {
            let node = Rc::clone(node);
            node.borrow_mut().value = value;
            self.remove(&node);
            self.push_front(&node);
        }
    }

    let node = Rc::new(RefCell::new(Node::new(key, value)));
    self.cache.insert(key, Rc::clone(&node));
    self.push_front(&node);
    if self.cache.len() > self.capacity {
        let back_key = self.pop_back().unwrap().borrow().key;
        self.cache.remove(&back_key);
    }
}

}

fn push_front(&mut self, node: Rc<RefCell<Node>>) {
    match self.head.take() {
        Some(head) => {
            self.cache.insert(key, Rc::clone(&node));
            self.push_front(&node);
        }
    }
}

```

JavaScript


```

/**
 * @param {number} capacity
 */
var LRUCache = function(capacity) {
    this.size = 0;
    this.capacity = capacity;
    this.cache = new Map();
    this.head = new Node(0, 0);
    this.tail = new Node(0, 0);
    this.head.next = this.tail;
    this.tail.prev = this.head;
};

/**
 * @param {number} key
 * @return {number}
 */
LRUCache.prototype.get = function(key) {
    if (!this.cache.has(key)) {
        return -1;
    }
    const node = this.cache.get(key);
    this.removeNode(node);
    this.addToHead(node);
    return node.val;
};

/**
 * @param {number} key
 * @param {number} value
 * @return {void}
 */
LRUCache.prototype.put = function(key, value) {
    if (!this.cache.has(key)) {
        const node = this.cache.get(key);
        this.removeNode(node);
        node.val = value;
        this.addToHead(node);
    } else {
        const node = new Node(key, value);
        this.cache.set(key, node);
        this.addToHead(node);
        if (this.size === this.capacity) {
            const nodeToRemove = this.tail.prev;
            this.cache.delete(nodeToRemove.key);
            this.removeNode(nodeToRemove);
            --this.size;
        }
    }
};

```

Linked List · LeetCode - By Pattern

— 1849 —

LeetCode

```

LRUCache.prototype.removeNode = function(node) {
    if (!node) return;
    node.prev.next = node.next;
    node.next.prev = node.prev;
};

LRUCache.prototype.addToHead = function(node) {
    node.next = this.head.next;
    node.prev = this.head;
    this.head.next.prev = node;
    this.head.next = node;
};

/**
 * @constructor
 * @param {number} key
 * @param {number} val
 */
function Node(key, val) {
    this.key = key;
    this.val = val;
    this.prev = null;
    this.next = null;
}

/**
 * Your LRUCache object will be instantiated and called as such:
 * var obj = new LRUCache(capacity);
 * var param_1 = obj.get(key);
 */

```

TypeScript
TypeScript

Linked List · LeetCode - By Pattern

— 1850 —

LeetCode

```

class LRUCache {
    capacity: number;
    map: Map<number, number>;
    constructor(capacity: number) {
        this.capacity = capacity;
        this.map = new Map();
    }

    get(key: number): number {
        if (!this.map.has(key)) {
            const val = this.map.get(key);
            this.map.delete(key);
            this.map.set(key, val);
            return val;
        }
        return -1;
    }

    put(key: number, value: number): void {
        this.map.delete(key);
        this.map.set(key, value);
        if (this.map.size === this.capacity) {
            this.map.delete(this.map.keys().next().value);
        }
    }
}

/**
 * Your LRUCache object will be instantiated and called as such:
 * var obj = new LRUCache(capacity);
 * var param_1 = obj.get(key);
 * obj.put(key, value);
 */

```

Go
Go

Linked List · LeetCode - By Pattern

— 1851 —

LeetCode

```

type Node struct {
    key, val int
    prev, next *Node
}

type LRUCache struct {
    capacity int
    cache map[int]*Node
    head, tail *Node
}

func Constructor(capacity int) LRUCache {
    head := new(Node)
    tail := new(Node)
    head.next = tail
    tail.prev = head
    return LRUCache{
        capacity: capacity,
        cache: map[int]*Node{},
        head: head,
        tail: tail,
    }
}

func (this *LRUCache) Get(key int) int {
    n, ok := this.cache[key]
    if !ok {
        return -1
    }
    this.moveToFront(n)
    return n.val
}

func (this *LRUCache) Put(key int, value int) {
    n, ok := this.cache[key]
    if ok {
        n.val = value
        this.moveToFront(n)
        return
    }
    if len(this.cache) == this.capacity {
        back := this.tail.prev
        this.removeNode(back)
        delete(this.cache, back.key)
    }
    n = &Node{key: key, val: value}
    this.pushFront(n)
    this.cache[key] = n
}

```

Linked List · LeetCode - By Pattern

— 1852 —

LeetCode

```

func (this *LRUCache) moveToFront(n *Node) {
    this.remove(n)
    this.pushFront(n)
}

func (this *LRUCache) remove(n *Node) {
    n.prev.next = n.next
    n.next.prev = n.prev
    n.prev = nil
    n.next = nil
}

func (this *LRUCache) pushFront(n *Node) {
    n.prev = this.head
    n.next = this.head.next
    this.head.next.prev = n
    this.head.next = n
}

```

[*] Linked List - Pattern Solution (matched from community answers)

Python

```

class Node:
    def __init__(self, key: int, value: int):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.keyToNode = {}
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.join(self.head, self.tail)

    def get(self, key: int) -> int:
        if key not in self.keyToNode:
            return -1
        node = self.keyToNode[key]
        self.remove(node)
        self.moveToHead(node)
        return node.value

```

Linked List · LeetCode - By Pattern

— 1853 —

LeetCode

```

def put(self, key: int, value: int) -> None:
    if key in self.keyToNode:
        node = self.keyToNode[key]
        node.value = value
        self.remove(node)
        self.moveToHead(node)
        return
    if len(self.keyToNode) == self.capacity:
        lastNode = self.tail.prev
        del self.keyToNode[lastNode.key]
        self.remove(lastNode)
    self.moveToHead(key, value)
    self.keyToNode[key] = self.head.next
    self.join(self.head, node)
    node1.next = node2
    node2.prev = node1

def moveToHead(self, node: Node):
    self.join(node, self.head.next)
    self.join(self.head, node)

def remove(self, node: Node):
    self.join(node.prev, node.next)

```

#148

MEDIUM

Sort List

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial
Linked List · Two Pointers · Divide and Conquer · Sorting · Merge Sort
Like · 100

Problem:

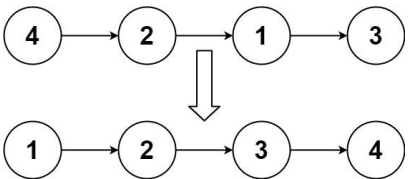
Given the head of a linked list, return the list after sorting it in ascending order.

Example 1:

Linked List · LeetCode - By Pattern

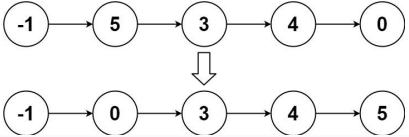
— 1854 —

LeetCode



Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Example 3:

Input: head = []
Output: []

Constraints:

• The number of nodes in the list is in the range [0, 5 * 10⁴].

• -105 <= Node.val <= 105

Follow up: Can you sort the linked list in O(n log n) time and O(1) memory (i.e. constant space)?

Python

Python

Linked List · LeetCode · By Pattern

— 1855 —

LeetCode

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        # Base case: if list is empty or contains a single element.
        if not head or not head.next:
            return head

        # Function to split the list into two halves
        def getMid(head):
            slow, fast = head, head.next
            # Move fast twice as fast as slow to locate middle
            while fast and fast.next:
                slow = slow.next
                fast = fast.next.next
            return slow

        # Split the linked list into two halves
        mid = getMid(head)
        right_head = mid.next
        mid.next = None # cut the list into two parts

        # Recursive calls to sort each half
        left = self.sortList(head)
        right = self.sortList(right_head)

        # Merge two sorted lists
        return merge(left, right)

    def merge(self, l1, l2):
        dummy = ListNode(0)
        tail = dummy
        # Merge the lists by comparing nodes
        while l1 and l2:
            if l1.val < l2.val:
                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next
        # Append the remaining nodes
        tail.next = l1 if l1 else l2
        return dummy.next
```

Python

Linked List · LeetCode · By Pattern

— 1856 —

LeetCode

```
class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        def split(head: ListNode, k: int) -> ListNode:
            while k > 1 and head:
                head = head.next
                k -= 1
            rest = head.next if head else None
            if head:
                head.next = None
            return rest

        def merge(l1: ListNode, l2: ListNode) -> tuple:
            dummy = ListNode(0)
            tail = dummy

            while l1 and l2:
                if l1.val < l2.val:
                    l1, l2 = l2, l1
                    tail.next = l1
                    l1 = l1.next
                else:
                    tail.next = l2
                    l2 = l2.next
                tail = tail.next

            return dummy.next, tail

        length = 0
        curr = head
        while curr:
            length += 1
            curr = curr.next

        dummy = ListNode(0, head)
        k = 1
        while k < length:
            curr = dummy.next
            while curr:
                l = curr
                r = split(l, k)
                curr = split(r, k)
                merge(l, r)
                tail.next = merge(l, r)
                l1, l2 = split(tail, k)
                tail = merge(l1, l2)
                k *= 2
            return dummy.next
```

Python

Linked List · LeetCode · By Pattern

— 1857 —

LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        slow, fast = head, head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        l1, l2 = head, slow.next
        slow.next = None
        l1, l2 = self.sortList(l1), self.sortList(l2)
        dummy = ListNode(0)
        tail = dummy
        while l1 and l2:
            if l1.val < l2.val:
                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next
        tail.next = l1 or l2
        return dummy.next
```

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        if head is None or head.next is None:
            return head
        slow, fast = head, head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        t = slow.next
        slow.next = None
        l1, l2 = self.sortList(head), self.sortList(t)
        dummy = ListNode(0)
        cur = dummy
        while l1 and l2:
            if l1.val < l2.val:
                cur.next = l1
                l1 = l1.next
            else:
                cur.next = l2
                l2 = l2.next
            cur = cur.next
```

Linked List · LeetCode · By Pattern

— 1858 —

LeetCode

```
cur.next = l1 or l2 # and the rest
return dummy.next
```

Java

Java

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     public int val;
 *     public ListNode next;
 *     public ListNode(int val=0, ListNode next=null) {
 *         this.val = val;
 *         this.next = next;
 *     }
 * }
 */
public class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null)
            return head;

        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode t = slow.next;
        slow.next = null;

        ListNode l1 = sortList(head);
        ListNode l2 = sortList(t);
        ListNode dummy = new ListNode(0);
        ListNode cur = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val < l2.val) {
                cur.next = l1;
                l1 = l1.next;
            } else {
                cur.next = l2;
                l2 = l2.next;
            }
            cur = cur.next;
        }
        cur.next = l1 == null ? l2 : l1;
        return dummy.next;
    }
}
```

JavaScript

JavaScript

Linked List · LeetCode · By Pattern

— 1859 —

LeetCode

```
/**
 * Definition for singly-linked list.
 * function ListNode(val, next) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.next = (next===undefined ? null : next)
 * }
 */
/**
 * @param {ListNode} head
 * @return {ListNode}
 */
var sortList = function(head) {
    if (head == null || head.next == null) {
        return head;
    }
    let [slow, fast] = [head, head.next];
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let [l1, l2] = [head, slow.next];
    slow.next = null;
    l1 = sortList(l1);
    l2 = sortList(l2);
    const dummy = new ListNode(0);

    let tail = dummy;
    while (l1 != null && l2 != null) {
        if (l1.val < l2.val) {
            tail.next = l1;
            l1 = l1.next;
        } else {
            tail.next = l2;
            l2 = l2.next;
        }
        tail = tail.next;
    }
    tail.next = l1 ?? l2;
    return dummy.next;
};
```

JavaScript

Linked List · LeetCode · By Pattern

— 1860 —

LeetCode

```

//
// Definition for singly-linked list.
// struct ListNode {
//     int val;
//     ListNode *next;
// };
//
// param [ListNode] head
// return [ListNode]
var sortList = function(head) {
    if (head === null || head.next === null) {
        return head;
    }
    let slow = head;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let f = slow.next;
    slow.next = null;
    let l1 = sortList(head);
    let l2 = sortList(f);

    const dummy = new ListNode();
    let cur = dummy;
    while (l1 && l2) {
        if (l1.val <= l2.val) {
            cur.next = l1;
            l1 = l1.next;
        } else {
            cur.next = l2;
            l2 = l2.next;
        }
        cur = cur.next;
    }
    cur.next = l1 || l2;
    return dummy.next;
};

```

TypeScript
TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//     int val;
//     ListNode next;
//     ListNode(int x) {
//         this.val = x;
//         this.next = null;
//     }
// }
//
function sortList(head: ListNode | null): ListNode | null {
    if (head === null || head.next === null) return head;

    let slow: ListNode = head;
    let fast: ListNode = head.next;
    while (fast && fast.next !== null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    let mid: ListNode = slow.next;
    slow.next = null;
    let l1: ListNode = sortList(head);

    let l2: ListNode = sortList(mid);
    let dummy: ListNode = new ListNode();
    let cur: ListNode = dummy;
    while (l1 && l2) {
        if (l1.val <= l2.val) {
            cur.next = l1;
            l1 = l1.next;
        } else {
            cur.next = l2;
            l2 = l2.next;
        }
        cur = cur.next;
    }
    cur.next = l1 || l2 || null;
    return dummy.next;
}

```

Rust
Rust

```

// Definition for singly-linked list.
// struct ListNode {
//     int val;
//     ListNode *next;
// };
//
// param [ListNode] head
// return [ListNode]
var sortList = function(head) {
    if (head === null || head.next === null) {
        return head;
    }
    let slow = head;
    let fast = head.next;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let mid = slow.next;
    slow.next = null;
    let l1 = sortList(head);
    let l2 = sortList(mid);

    const dummy = new ListNode();
    let cur = dummy;
    while (l1 && l2) {
        if (l1.val <= l2.val) {
            cur.next = l1;
            l1 = l1.next;
        } else {
            cur.next = l2;
            l2 = l2.next;
        }
        cur = cur.next;
    }
    cur.next = l1 || l2;
    return dummy.next;
};

```

TypeScript
TypeScript

```

// Definition for singly-linked list.
// struct ListNode {
//     int val;
//     ListNode *next;
// };
//
// param [ListNode] head
// return [ListNode]
var sortList = function(head) {
    if (head === null || head.next === null) {
        return head;
    }
    let slow = head;
    let fast = head.next;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let mid = slow.next;
    slow.next = null;
    let l1 = sortList(head);
    let l2 = sortList(mid);

    const dummy = new ListNode();
    let cur = dummy;
    while (l1 && l2) {
        if (l1.val <= l2.val) {
            cur.next = l1;
            l1 = l1.next;
        } else {
            cur.next = l2;
            l2 = l2.next;
        }
        cur = cur.next;
    }
    cur.next = l1 || l2;
    return dummy.next;
};

```

TypeScript
TypeScript

Linked List
LeetCode 147

Problem:
Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.
The first node is considered odd, and the second node is even, and so on.
Note that the relative order inside both the even and odd groups should remain as it was in the input.
You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

Example 1:

Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]

Example 2:

Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range [0, 104].
- $-106 \leq \text{Node.val} \leq 106$

```

// Definition for singly-linked list.
// struct ListNode {
//     int val;
//     ListNode *next;
// };
//
// param [ListNode] head
// return [ListNode]
var sortList = function(head) {
    if (head === null || head.next === null) {
        return head;
    }
    let slow = head;
    let fast = head.next;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }
    let mid = slow.next;
    slow.next = null;
    let l1 = sortList(head);
    let l2 = sortList(mid);

    const dummy = new ListNode();
    let cur = dummy;
    while (l1 && l2) {
        if (l1.val <= l2.val) {
            cur.next = l1;
            l1 = l1.next;
        } else {
            cur.next = l2;
            l2 = l2.next;
        }
        cur = cur.next;
    }
    cur.next = l1 || l2;
    return dummy.next;
};

```

TypeScript
TypeScript

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        a = head
        b = c = head.next
        while b and b.next:
            a.next = b.next
            a = a.next
            b.next = a.next
            b = b.next
        a.next = c
        return head
```

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        a = head
        b = c = head.next
        while b and b.next:
            a.next = b.next
            a = a.next
            b.next = a.next
            b = b.next
        a.next = c
        return head
```

TypeScript

TypeScript

```
//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }
//
function oddEvenList(head: ListNode | null): ListNode | null {
    if (!head) {
        return null;
    }
    let [a, b, c] = [head, head.next, head.next?.next];
    while (b && b.next) {
        a.next = b.next;
        a = a.next;
        b.next = a.next;
        b = b.next;
        c = c?.next;
    }
    a.next = c;
    return head;
}
```

TypeScript

```
//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }
//
function oddEvenList(head: ListNode | null): ListNode | null {
    if (!head) {
        return null;
    }
    let [a, b, c] = [head, head.next, head.next?.next];
    while (b && b.next) {
        a.next = b.next;
        a = a.next;
        b.next = a.next;
        b = b.next;
        c = c?.next;
    }
    a.next = c;
    return head;
}
```

[+] Linked List — Pattern Solution (matched from community answers)

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: ListNode) -> ListNode:
        # Check for empty list or list with one node.
        if not head or not head.next:
            return head

        odd = head # Point to the first (odd indexed) node.
        even = head.next # Point to the second (even indexed) node.
        even_head = even # Save the head of the even list.

        # Traverse the list by interleaving next pointers.
        while even and even.next:
            odd.next = even.next # Connect current odd to next odd.
            odd = odd.next # Move odd pointer.
            even.next = odd.next # Connect current even to next even.
            even = even.next # Move even pointer.

        # Append the even list after the odd list.
        odd.next = even_head
        return head
```

#369

MEDIUM

Plus One Linked List

LeetCode - LeetCode - WALTER - LeetCode - Editorial

Linked List - Math - Recursion

LeetCode Premium 98

Problem:

Given a non-negative integer represented as a linked list of digits, plus one to the integer.

The digits are stored such that the most significant digit is at the head of the list.

Example 1:

Input: head = [1,2,3]

Output: [1,2,4]

Example 2:

Input: head = [0]

Output: [1]

Constraints:

• The number of nodes in the linked list is in the range [1, 100].

• 0 <= Node.val <= 9

• The number represented by the linked list does not contain leading zeros except for the zero itself.

>> Brute Force [Linked List] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def reverseList(self, head):
        # Brute Force - extract into list, manipulate, rebuild
        vals = []
        curr = head
        while curr:
            vals.append(curr.val)
            curr = curr.next
        dummy = ListNode(0)
        curr = dummy
        for v in vals:
            curr.next = ListNode(v)
            curr = curr.next
        return dummy.next
```

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
#
# Helper function to reverse the linked list
def reverseList(head):
    prev = None
    curr = head
    while curr:
        next = curr.next # store next node
        curr.next = prev # reverse pointer
        prev = curr # move prev forward
        curr = next # move to next node
    return prev

def plusOne(head: ListNode) -> ListNode:
    # Reverse the linked list to start addition from the least significant digit
    head = reverseList(head)

    current = head
    carry = 1 # initializing carry as the plus one
    while current and carry:
        current.val += carry # add carry to the current node
```

```
carry = current.val // 10 # calculate new carry
current.val = 10 # update the node's value
# If we're at the end and there's still a carry, create a new node.
if not current.next and carry:
    current.next = ListNode(0)
current = current.next

# Reverse the list back to its original order before returning.
return reverseList(head)
```

Python

```
class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        if not head:
            return ListNode(1)
        if not head.next:
            return ListNode(1, head)
        return head

    def addOne(self, node: ListNode) -> int:
        carry = self.addOne(node.next) if node.next else 1
        node.val += carry
        node.val = node.val % 10
        return node.val // 10
```

Python

```
class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        dummy.next = head

        while head:
            if head.val < 9:
                curr = head
                head = head.next
            # If head.val is 9, we need to carry to the rightmost non-9 node.
            curr.val += 1
            while curr.next:
                curr.next.val += 0
                curr = curr.next
            return dummy.next if dummy.val == 0 else dummy
```

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def plusOne(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0, head)
        while head:
            if head.val != 9:
                target = head
                target.val += 1
                target = target.next
            else:
                target.val = 0
                target = target.next
        return dummy if dummy.val else dummy.next
```

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        target = dummy
        while head:
            if head.val != 9:
                target = head
                target.val += 1
                target = target.next
            else:
                target.val = 0
                target = target.next
        return dummy if dummy.val else dummy.next
```

TypeScript

TypeScript

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function plusOne(head: ListNode | null): ListNode | null {
    const dummy = new ListNode(0, head);
    let target = dummy;
    while (head) {
        if (head.val === 9) {
            target = head;
            head = head.next;
        }
        target.val++;
        for (target = target.next; target; target = target.next) {
            target.val = 0;
        }
    }
    return dummy.val ? dummy : dummy.next;
}

```

```

// TypeScript
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function plusOne(head: ListNode | null): ListNode | null {
    const dummy = new ListNode(0, head);
    let target = dummy;
    while (head) {
        if (head.val === 9) {
            target = head;
            head = head.next;
        }
        target.val++;
        for (target = target.next; target; target = target.next) {
            target.val = 0;
        }
    }
    return dummy.val ? dummy : dummy.next;
}

```

[+] Linked List - Pattern Solution (matched from community answers)

Python

```

class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        dummy.next = head

        while head:
            if head.val != 9:
                curr = head
                head = head.next
            # curr now points to the rightmost non-9 node.
            curr.val += 1
            while curr.next:
                curr.next.val += 0
            curr = curr.next

        return dummy.next if dummy.val == 0 else dummy

```

#430

MEDIUM

Flatten a Multilevel Doubly Linked List

LeetCode · Simplify Interview · Stack · LeetCode - Editorial

Linked List · DFS · Doubly Linked List

LeetCode Premium 98

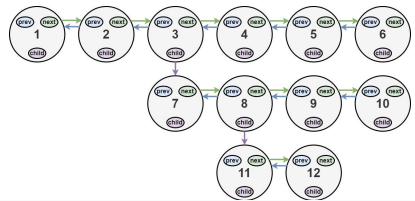
Problem:

You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let curr be a node with a child list. The nodes in the child list should appear after curr and before curr.next in the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1:



Input: head = [1,2,3,4,5,6,null,null,7,8,9,10,null,null,11,12]
Output: [1,2,3,7,8,11,12,9,10,4,5,6]
Explanation: The multilevel linked list in the input is shown. After flattening the multilevel linked list it becomes:

Example 2:
Input: head = [1,2,null,3]
Output: [1,3,2]
Explanation: The multilevel linked list in the input is shown. After flattening the multilevel linked list it becomes:

Example 3:
Input: head = []
Output: []
Explanation: There could be empty list in the input.

Constraints:

- The number of Nodes will not exceed 1000.

- 1 ≤ Node.val ≤ 10⁵

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from Example 1 above:

```

1---2---3---4---5---6---NULL
|
7---8---9---10---NULL
|
11---12---NULL

```

The serialization of each level is as follows:

```

[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]

```

```

// Definition for a Node.
// class Node {
//     val: number
//     prev: Node | null
//     next: Node | null
//     child: Node | null
//     constructor(val?: number, prev?: Node | null, next?: Node | null, child?: Node | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.prev = (prev===undefined ? null : prev)
//         this.next = (next===undefined ? null : next)
//         this.child = (child===undefined ? null : child)
//     }
// }

def flatten(head: Node | null): Node | null {
    // If head is null, simply return head.
    if not head:
        return head

    // Helper function to perform DFS and return the tail of the flattened list.
    def dfs(node: Node): Node | null {
        current = node
        last = None
        while current:
            next_node = current.next
            # Store current.next before modification.
            # If current node has a child, we flatten the child list recursively.
            if current.child:
                # Recursively flatten the child list.
                child_tail = dfs(current.child)
                # Connect current.next with the start of the child list.
                current.next = current.child
                current.child = None
            else:
                current = current.next
            # If there is a next node, attach it after the child list.
            if next_node:
                child_tail.next = next_node
                next_node.prev = child_tail
            # Update the next pointer.
            last = child_tail
        # Head to the next node (which might have been updated after flattening a child).
        current = next_node
        return last

    dfs(head) # Begin DFS traversal from the head.
    return head

```

Python

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def flatten(head: 'Node', next: 'Node') -> 'Node':
            if not head:
                return next
            head.next = flatten(head.child, flatten(head.next, next))
            if head.next:
                head.next.prev = head
            head.child = None
            return head

        return flatten(head, None)

```

Python

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        curr = head
        while curr:
            if curr.child:
                cachedNext = curr.next
                curr.next = curr.child
                curr.child.prev = curr
                curr.child = None
                tail = curr.next
                while tail.next:
                    tail = tail.next
                tail.next = cachedNext
                if cachedNext:
                    cachedNext.prev = tail
                curr = curr.next
            else:
                curr = curr.next
        return head

```

Python

```

class Node:
    def __init__(self, val, prev, next, child):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def preorder(pre, cur):
            if cur is None:
                return pre
            cur.prev = pre
            pre.next = cur
            t = cur.next
            tail = preorder(cur, cur.child)
            cur.child = None
            return preorder(tail, t)

        if head is None:
            return None
        dummy = Node(0, None, head, None)
        preorder(dummy, head)
        dummy.next.prev = None
        return dummy.next

```

Python

```

class Node:
    def __init__(self, val, prev, next, child):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def preorder(pre, cur):
            if cur is None:
                return pre
            cur.prev = pre
            pre.next = cur
            t = cur.next
            tail = preorder(cur, cur.child)
            cur.child = None
            return preorder(tail, t)

        if head is None:
            return None
        dummy = Node(0, None, head, None)
        preorder(dummy, head)
        dummy.next.prev = None
        return dummy.next

```

Linked List · LeetCode - By Pattern

— 1879 —

LeetCode

```

return None
dummy = Node(0, None, head, None)
preorder(dummy, head)
dummy.next.prev = None
return dummy.next

```

Java

```

// Definition for a Node.
class Node {
    public int val;
    public Node prev;
    public Node next;
    public Node child;
};

class Solution {
    public Node flatten(Node head) {
        if (head == null) {
            return null;
        }
        Node dummy = new Node();
        dummy.next = head;
        preorder(dummy, head);
        dummy.next.prev = null;
        return dummy.next;
    }

    private Node preorder(Node pre, Node cur) {
        if (cur == null) {
            return pre;
        }
        cur.prev = pre;
        pre.next = cur;
        Node t = cur.next;
        Node tail = preorder(cur, cur.child);
        cur.child = null;
        return preorder(tail, t);
    }
}

```

Java

Linked List · LeetCode - By Pattern

— 1880 —

LeetCode

```

// Definition for a Node.
class Node {
    public int val;
    public Node prev;
    public Node next;
    public Node child;
};

class Solution {
    public Node flatten(Node head) {
        if (head == null) {
            return null;
        }
        Node dummy = new Node();
        dummy.next = head;
        preorder(dummy, head);
        dummy.next.prev = null;
        return dummy.next;
    }

    private Node preorder(Node pre, Node cur) {
        if (cur == null) {
            return pre;
        }
        cur.prev = pre;
        pre.next = cur;
        Node t = cur.next;
        Node tail = preorder(cur, cur.child);
        cur.child = null;
        return preorder(tail, t);
    }
}

```

[*] Linked List — Pattern Solution (matched from community answers)

Python

Linked List · LeetCode - By Pattern

— 1881 —

LeetCode

```

// Definition for a Node.
class Node {
    def __init__(self, val, prev=None, next=None, child=None):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child

def flatten(head):
    # If the list is empty, simply return None.
    if not head:
        return head

    # Helper function to perform DFS and return the tail of the flattened list.
    def dfs(node):
        current = node
        while current:
            next_node = current.next # Store current.next before modification
            # If current node has a child, we flatten the child list recursively.
            if current.child:
                # Recursively flatten the child list.
                child_tail = dfs(current.child)
                # Connect current node with the start of the child list.
                current.next = current.child

                current.child.prev = current
                # If there was a next node, attach it after the child list.
                if next_node:
                    child_tail.next = next_node
                    next_node.prev = child_tail
            # Set child pointer to None as required.
            current.child = None
            # Update the next pointer.
            last = child_tail

            # Move to the next node (which might have been updated after flattening a child).
            current = next_node
            return last

    dfs(head) # Begin DFS traversal from the head.
    return head

# Example to illustrate usage:
# head = Node(1)
# head.next = Node(2)
# head.child = Node(3)
# flatten(head)

```

#622

MEDIUM

Design Circular Queue

LeetCode · Simplilearn · WUOLCC · LeetCode · Editorial

Array · Linked List · Design · Queue

Like Denny Zhang

Linked List · LeetCode - By Pattern

— 1882 —

LeetCode

Problem:

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

- MyCircularQueue(k) Initializes the object with the size of the queue to be k.
- int Front() Gets the front item from the queue. If the queue is empty, return -1.
- int Rear() Gets the last item from the queue. If the queue is empty, return -1.
- boolean enQueue(int value) Inserts an element into the circular queue. Return true if the operation is successful.
- boolean deQueue() Deletes an element from the circular queue. Return true if the operation is successful.
- boolean isEmpty() Checks whether the circular queue is empty or not.
- boolean isFull() Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

```

Input
["MyCircularQueue", "enQueue", "enQueue", "enQueue", "enQueue", "Rear", "isFull", "deQueue", "enQueue", "Rear"]
[[3], [1], [2], [3], [4], [1], [1], [1], [4], [1]]
Output
[null, true, true, true, false, 3, true, true, true, 4]

```

Explanation
MyCircularQueue myCircularQueue = new MyCircularQueue(3);
myCircularQueue.enQueue(1); // return True
myCircularQueue.enQueue(2); // return True
myCircularQueue.enQueue(3); // return True
myCircularQueue.enQueue(4); // return False
myCircularQueue.Rear(); // return 3
myCircularQueue.isFull(); // return True
myCircularQueue.deQueue(); // return True
myCircularQueue.enQueue(4); // return True
myCircularQueue.Rear(); // return 4

Constraints:

- 1 <= k <= 1000
- 0 <= value <= 1000
- At most 3000 calls will be made to enQueue, deQueue, Front, Rear, isEmpty, and isFull.

Python

Linked List · LeetCode - By Pattern

— 1883 —

LeetCode

```

# Python Implementation of MyCircularQueue

class MyCircularQueue:
    def __init__(self, k):
        # Initialize the queue with capacity k
        self.capacity = k
        self.queue = [0] * k # Fixed-size array
        self.front = 0 # pointer to the front element
        self.rear = 0 # pointer to the next insertion position
        self.count = 0 # Current number of elements in the queue

    def enQueue(self, value):
        # Check if the queue is full
        if self.isFull():
            return False
        self.queue[self.rear] = value
        self.rear = (self.rear + 1) % self.capacity
        self.count += 1
        return True

    def deQueue(self):
        # Check if the queue is empty
        if self.isEmpty():
            return False

        # Move front pointer to the next element using modulo for circularity
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return True

    def Front(self):
        # Return -1 if the queue is empty, else return the front element
        if self.isEmpty():
            return -1
        return self.queue[self.front]

    def Rear(self):
        # Return -1 if the queue is empty
        if self.isEmpty():
            return -1
        # Since rear points to the next insertion position, get the last element index
        return self.queue[(self.rear - 1) % self.capacity] % self.capacity

    def isEmpty(self):
        # Check if count is 0
        return self.count == 0

    def isFull(self):
        # Check if the queue has reached its capacity
        return self.count == self.capacity

```

Python

Linked List · LeetCode - By Pattern

— 1884 —

LeetCode

```

class MyCircularQueue:

    def __init__(self, k: int):
        self.k = (k) + k
        self.size = 0
        self.capacity = k
        self.front = 0

    def enqueue(self, value: int) -> bool:
        if self.isFull():
            return False
        self.q[(self.front + self.size) % self.capacity] = value
        self.size += 1
        return True

    def dequeue(self) -> bool:
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return True

    def front(self) -> int:
        return -1 if self.isEmpty() else self.q[self.front]

    def rear(self) -> int:
        if self.isEmpty():
            return -1
        return self.q[(self.front + self.size - 1) % self.capacity]

    def isEmpty(self) -> bool:
        return self.size == 0

    def isFull(self) -> bool:
        return self.size == self.capacity

```

```

# Your MyCircularQueue object will be instantiated and called as such:
# obj = MyCircularQueue(k)
# param_1 = obj.enqueue(value)
# param_2 = obj.dequeue()
# param_3 = obj.front()
# param_4 = obj.rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()

```

Python

Linked List · LeetCode - By Pattern

— 1885 —

LeetCode

```

class MyCircularQueue:

    def __init__(self, k: int):
        self.k = (k) + k
        self.front = 0
        self.size = 0
        self.capacity = k

    def enqueue(self, value: int) -> bool:
        if self.isFull():
            return False
        idx = (self.front + self.size) % self.capacity
        self.q[idx] = value
        self.size += 1
        return True

    def dequeue(self) -> bool:
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return True

    def front(self) -> int:
        return -1 if self.isEmpty() else self.q[self.front]

    def rear(self) -> int:
        if self.isEmpty():
            return -1
        idx = (self.front + self.size - 1) % self.capacity
        return self.q[idx]

    def isEmpty(self) -> bool:
        return self.size == 0

    def isFull(self) -> bool:
        return self.size == self.capacity

```

```

# Your MyCircularQueue object will be instantiated and called as such:
# obj = MyCircularQueue(k)
# param_1 = obj.enqueue(value)
# param_2 = obj.dequeue()
# param_3 = obj.front()
# param_4 = obj.rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()

```

Linked List · LeetCode - By Pattern

— 1886 —

LeetCode

```

def __init__(self, k):
    """
    Initialize your data structure here. Set the size of the queue to be k.
    :type k: int
    """
    self.queue = []
    self.size = k
    self.front = 0
    self.rear = 0

    def enqueue(self, value):
        """
        Insert an element into the circular queue. Return true if the operation is successful.
        :type value: int
        :rtype: bool

        if self.rear == self.front == self.size:
            self.queue.append(value)
            self.rear += 1
            return True
        else:
            return False

    def dequeue(self):
        """
        Delete an element from the circular queue. Return true if the operation is successful.
        :rtype: bool

        if self.rear > self.front + 0:

```

C++

Linked List · LeetCode - By Pattern

— 1887 —

LeetCode

```

class MyCircularQueue {
public:
    // Initialize your data structure here. Set the size of the queue to be k.
    MyCircularQueue(int k) : k(k), q(k), rear(k - 1) {}

    // Insert an element into the circular queue. Return true if the operation is successful.
    bool enqueue(int value) {
        if (isFull())
            return false;
        rear = ++rear % k;
        q[rear] = value;
        ++size;
        return true;
    }

    // Delete an element from the circular queue. Return true if the operation is successful.
    bool dequeue() {
        if (isEmpty())
            return false;
        front = ++front % k;
        --size;
        return true;
    }

    // Get the front item from the queue.
    int front() {
        return isEmpty() ? -1 : q[front];
    }

    // Get the last item from the queue.
    int rear() {
        return isEmpty() ? -1 : q[rear];
    }

    // Checks whether the circular queue is empty or not.
    bool isEmpty() {
        return size == 0;
    }

    // Checks whether the circular queue is full or not.
    bool isFull() {
        return size == k;
    }

private:
    const int k;

    vector<int> q;
    int size = 0;
    int front = 0;
    int rear;
};

```

C++

Linked List · LeetCode - By Pattern

— 1888 —

LeetCode

```

// 01: https://leetcode.com/problems/design-circular-queue/
// Time: O(1) for all
// Space: O(k)
class MyCircularQueue {
private:
    vector<int> v;
    int start = 0, len = 0;
public:
    MyCircularQueue(int k): v(k) {}
    bool enqueue(int value) {
        if (isFull()) return false;
        v[(start + len) % v.size()] = value;
        return true;
    }
    bool dequeue() {
        if (isEmpty()) return false;
        start = (start + 1) % v.size();
        --len;
        return true;
    }
    int front() {
        if (isEmpty()) return -1;
        return v[start];
    }
    int rear() {
        if (isEmpty()) return -1;
        return v[(start + len - 1) % v.size()];
    }
    bool isEmpty() {
        return len == 0;
    }
    bool isFull() {
        return len == v.size();
    }
};

```

Java

Linked List · LeetCode - By Pattern

— 1889 —

LeetCode

```

public class Design_Circular_Queue {
    class MyCircularQueue {
        int[] arr;
        int size;
        int capacity;
        int front;
        int back;

        // Initialize your data structure here. Set the size of the queue to be k.
        public MyCircularQueue(int k) {
            arr = new int[k];
            capacity = k;
            size = 0;
            front = 0;
            back = -1;
        }

        // Insert an element into the circular queue. Return true if the operation is successful.
        //
        public boolean enqueue(int value) {
            if (size == capacity) {
                return false;
            }
            --back;
            arr[(back + arr.length) % arr.length] = value;
            ++size;
            return true;
        }

        // Delete an element from the circular queue. Return true if the operation is successful.
        //
        public boolean dequeue() {
            if (size == 0) return false;
            ++front;
            --size;
            return true;
        }

        // Get the front item from the queue.
        //
        public int front() {
            if (size == 0) return -1;
            return arr[front % arr.length];
        }
    }
}

```

Linked List · LeetCode - By Pattern

— 1890 —

LeetCode

```

//
// Get the last item from the queue.
//
public int Rear() {
    if (size == 0) return -1;
    return arr[back % arr.length];
}

//
// Checks whether the circular queue is empty or not.
//
public boolean isEmpty() {
    return size == 0;
}

//
// Checks whether the circular queue is full or not.
//
public boolean isFull() {
    return size == capacity;
}
}

```

```

#####
class MyCircularQueue {

```

TypeScript
TypeScript

Linked List · LeetCode - By Pattern

— 1891 —

LeetCode

```

class MyCircularQueue {
    private queue: number[];
    private left: number;
    private right: number;
    private capacity: number;

    constructor(k: number) {
        this.queue = new Array(k);
        this.left = 0;
        this.right = 0;
        this.capacity = k;
    }

    enqueue(value: number): boolean {
        if (this.isFull()) {
            return false;
        }
        this.queue[this.right % this.capacity] = value;
        this.right++;
        return true;
    }

    dequeue(): boolean {
        if (this.isEmpty()) {
            return false;
        }

        this.left++;
        return true;
    }

    front(): number {
        if (this.isEmpty()) {
            return -1;
        }
        return this.queue[(this.left % this.capacity)];
    }

    rear(): number {
        if (this.isEmpty()) {
            return -1;
        }
        return this.queue[(this.right - 1) % this.capacity];
    }

    isEmpty(): boolean {
        return this.right - this.left === 0;
    }

    isFull(): boolean {
        return this.right - this.left === this.capacity;
    }
}

```

Linked List · LeetCode - By Pattern

— 1892 —

LeetCode

```

}
}

//
// Your MyCircularQueue object will be instantiated and called as such:
// var obj = new MyCircularQueue(k)
// var param_1 = obj.enqueue(value)
// var param_2 = obj.dequeue()
// var param_3 = obj.front()
// var param_4 = obj.rear()
// var param_5 = obj.isEmpty()
// var param_6 = obj.isFull()
//

```

Go

Go

```

type MyCircularQueue struct {
    front int
    size int
    capacity int
    q []int
}

func Constructor(k int) MyCircularQueue {
    q := make([]int, k)
    return MyCircularQueue{0, 0, k, q}
}

func (this *MyCircularQueue) Enqueue(value int) bool {
    if this.isFull() {
        return false
    }
    idx := (this.front + this.size) % this.capacity
    this.q[idx] = value
    this.size++
    return true
}

func (this *MyCircularQueue) Dequeue() bool {
    if this.isEmpty() {
        return false
    }
}

```

Linked List · LeetCode - By Pattern

— 1893 —

LeetCode

```

}
this.front = (this.front + 1) % this.capacity
this.size--
return true
}

func (this *MyCircularQueue) Front() int {
    if this.isEmpty() {
        return -1
    }
    return this.q[this.front]
}

func (this *MyCircularQueue) Rear() int {
    if this.isEmpty() {
        return -1
    }
    idx := (this.front + this.size - 1) % this.capacity
    return this.q[idx]
}

func (this *MyCircularQueue) IsEmpty() bool {
    return this.size == 0
}

func (this *MyCircularQueue) IsFull() bool {
    return this.size == this.capacity
}

//
// Your MyCircularQueue object will be instantiated and called as such:
// obj := Constructor(k)
// param_1 := obj.Enqueue(value)
// param_2 := obj.Dequeue()
// param_3 := obj.Front()
// param_4 := obj.Rear()
// param_5 := obj.IsEmpty()
// param_6 := obj.IsFull()
//

```

Rust

Rust

Linked List · LeetCode - By Pattern

— 1894 —

LeetCode

```

struct MyCircularQueue {
    queue: Vec<int>,
    left: usize,
    right: usize,
    capacity: usize,
}

//
// 'self' means the method takes an unstable reference.
// If you need a mutable reference, change it to 'mut self' instead.
//
impl MyCircularQueue {
    fn new(k: int) -> Self {
        let k = k as usize;
        Self {
            queue: vec![0; k],
            left: 0,
            right: 0,
            capacity: k,
        }
    }

    fn en_queue(mut self, value: int) -> bool {
        if self.is_full() {
            return false;
        }

        self.queue[self.right % self.capacity] = value;
        self.right += 1;
        true
    }

    fn de_queue(mut self) -> bool {
        if self.is_empty() {
            return false;
        }
        self.left += 1;
        true
    }

    fn front(self) -> int {
        if self.is_empty() {
            return -1;
        }
        self.queue[self.left % self.capacity]
    }

    fn rear(self) -> int {
        if self.is_empty() {
            return -1;
        }
    }
}

```

Linked List · LeetCode - By Pattern

— 1895 —

LeetCode

```

        self.queue[self.right % self.capacity]

    fn is_empty(self) -> bool {
        self.right - self.left == 0
    }

    fn is_full(self) -> bool {
        self.right - self.left == self.capacity
    }
}

//
// Your MyCircularQueue object will be instantiated and called as such:
// let obj = MyCircularQueue.new(k);
// let ret_1: bool = obj.en_queue(value);
// let ret_2: bool = obj.de_queue();
// let ret_3: int = obj.front();
// let ret_4: int = obj.rear();
// let ret_5: bool = obj.is_empty();
// let ret_6: bool = obj.is_full();
//

```

[*] Linked List - Pattern Solution (stored optimal)

Python

```

# Python Implementation of MyCircularQueue

class MyCircularQueue:
    def __init__(self, k):
        # Initialize the queue with capacity k
        self.capacity = k
        self.queue = [0] * k # fixed-size array
        self.front = 0 # pointer to the front element
        self.rear = 0 # pointer to the next insertion position
        self.count = 0 # current number of elements in the queue

    def enqueue(self, value):
        # Check if the queue is full
        if self.is_full():
            return False
        # Insert element at rear pointer and update pointer using modulo for circularity
        self.queue[self.rear] = value
        self.rear = (self.rear + 1) % self.capacity
        self.count += 1
        return True

    def dequeue(self):
        # Check if the queue is empty
        if self.is_empty():
            return False

```

Linked List · LeetCode - By Pattern

— 1896 —

LeetCode


```

# Move front pointer to the next element using modulo for circularity
self.front = (self.front + 1) % self.capacity
self.count += 1
return True

def front(self):
# Return -1 if the queue is empty, else return the front element
if self.isEmpty():
    return -1
return self.queue[self.front]

def rear(self):
# Return -1 if the queue is empty
if self.isEmpty():
    return -1
# Since rear points to the next insertion position, get the last element index
return self.queue[self.rear - 1 % self.capacity % self.capacity]

def isEmpty(self):
# Check if count is 0
return self.count == 0

def isFull(self):
# Check if the queue has reached its capacity
return self.count == self.capacity

```

C++

```

// C++ Implementation of MyCircularQueue
class MyCircularQueue {
private:
    int capacity; // Maximum capacity of the queue
    int count; // Current number of elements in the queue
    int front; // Index of the front element
    int rear; // Index of the next insertion position
    vector<int> queue; // Fixed-size array to store elements
public:
    // Constructor to initialize the queue with capacity k
    MyCircularQueue(int k) {
        capacity = k;
        count = 0;
        front = 0;
        rear = 0;
        queue.resize(k); // Resize the vector to have k elements
    }

    // Inserts an element into the circular queue
    bool enqueue(int value) {
        // If the queue is full, cannot enqueue
        if (isFull()) return false;
        queue[rear] = value; // Place the value at the rear
    }

```

Linked List · LeetCode · By Pattern — 1897 — LeetCode

```

rear = (rear + 1) % capacity; // Update rear using modulo for circularity
count++; // Increase count
return true;
}

// Delete an element from the circular queue
bool dequeue() {
    // If the queue is empty, cannot dequeue
    if (isEmpty()) return false;
    front = (front + 1) % capacity; // Update front pointer to next element
    count--; // Decrease count
    return true;
}

// Get the front item from the queue
int front() {
    // Return -1 if queue is empty
    if (isEmpty()) return -1;
    return queue[front];
}

// Get the last item from the queue
int rear() {
    // Return -1 if queue is empty
    if (isEmpty()) return -1;
}

// Now moving to the next insertion position, so compute last index
int lastIndex = (rear - 1 + capacity) % capacity;
return queue[lastIndex];
}

// Check if the circular queue is empty
bool isEmpty() {
    return count == 0;
}

// Check if the circular queue is full
bool isFull() {
    return count == capacity;
}
};

```

#707 MEDIUM
Design Linked List
[LeetCode](#) · [SimpleHash](#) · [WuJCC](#) · [LeetCode](#) · [Editorial](#)
 Linked List · Design
 Lists Denny Zhang

Problem:
 Design your implementation of the linked list. You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: val and next, val is the value of the current node, and next is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute prev to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.

Linked List · LeetCode · By Pattern — 1898 — LeetCode

Implement the MyLinkedList class:

- MyLinkedList() Initializes the MyLinkedList object.
- int get(int index) Get the value of the indexth node in the linked list. If the index is invalid, return -1.
- void addAtHead(int val) Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- void addAtTail(int val) Append a node of value val as the last element of the linked list.
- void addAtIndex(int index, int val) Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.
- void deleteAtIndex(int index) Delete the indexth node in the linked list, if the index is valid.

Example 1:

Input
 ["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get", "deleteAtIndex", "get"]
 [[], [1], [3], [1, 2], [1], [1], [1]]
 Output
 [null, null, null, null, 2, null, 3]

Explanation
 MyLinkedList myLinkedList = new MyLinkedList();
 myLinkedList.addAtHead(1);
 myLinkedList.addAtTail(3);
 myLinkedList.addAtIndex(1, 2); // linked list becomes 1->2->3
 myLinkedList.get(1); // returns 2
 myLinkedList.deleteAtIndex(1); // now the linked list is 1->3
 myLinkedList.get(1); // returns 3

Constraints:

- 0 <= index, val <= 1000
- Please do not use the built-in LinkedList library.
- At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

Python
 Python

Linked List · LeetCode · By Pattern — 1899 — LeetCode

```

# Define a node for the singly linked list
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # The value of the node
        self.next = next # Pointer to the next node

class MyLinkedList:
    def __init__(self):
        # Initialize size and a dummy head node to simplify operations
        self.size = 0
        self.head = ListNode(0) # dummy head

    def get(self, index: int) -> int:
        # If index is out of bounds, return -1
        if index < 0 or index >= self.size:
            return -1
        current = self.head.next # start from the actual head
        for _ in range(index):
            current = current.next
        return current.val

    def addAtHead(self, val: int) -> None:
        self.addAtIndex(0, val) # add at index 0

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.size, val) # add at the end

    def addAtIndex(self, index: int, val: int) -> None:
        # Handle edge case where index is 0 (insert at head)
        if index < 0:
            index = 0
        # Only insert if index is valid (no size)
        if index > self.size:
            return
        self.size += 1
        # Start from the dummy head node
        prev = self.head
        for _ in range(index):
            prev = prev.next
        # Insert new node
        new_node = ListNode(val)
        new_node.next = prev.next
        prev.next = new_node

    def deleteAtIndex(self, index: int) -> None:
        # If index is out of bounds, do nothing
        if index < 0 or index >= self.size:
            return
        self.size -= 1
        prev = self.head

        for _ in range(index):
            prev = prev.next
        # Remove the node by skipping it
        prev.next = prev.next.next

```

Python
 Linked List · LeetCode · By Pattern — 1900 — LeetCode

```

from dataclasses import dataclass

@dataclass
class ListNode:
    val: int
    next: ListNode = None = None

class MyLinkedList:
    def __init__(self):
        self.length = 0
        self.dummy = ListNode(0)

    def get(self, index: int) -> int:
        if index < 0 or index >= self.length:
            return -1
        curr = self.dummy.next
        for _ in range(index):
            curr = curr.next
        return curr.val

    def addAtHead(self, val: int) -> None:
        curr = self.dummy.next
        self.dummy.next = ListNode(val)
        self.dummy.next.next = curr
        self.length += 1

    def addAtTail(self, val: int) -> None:
        curr = self.dummy
        while curr.next:
            curr = curr.next
        curr.next = ListNode(val)
        self.length += 1

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.length:
            return
        curr = self.dummy
        for _ in range(index):
            curr = curr.next
        temp = curr.next
        curr.next = temp
        self.length += 1

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.length:
            return
        curr = self.dummy

        for _ in range(index):
            curr = curr.next
            temp = curr.next
            curr.next = temp.next
            self.length -= 1

```

Python
 Linked List · LeetCode · By Pattern — 1901 — LeetCode

```

class MyLinkedList:
    def __init__(self):
        self.dummy = ListNode(0)
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        cur = self.dummy.next
        for _ in range(index):
            cur = cur.next
        return cur.val

    def addAtHead(self, val: int) -> None:
        self.addAtIndex(0, val)

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.cnt:
            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        pre.next = ListNode(val, pre.next)
        self.cnt += 1

    def deleteAtIndex(self, index: int) -> None:
        if index > self.cnt:
            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        t = pre.next
        pre.next = t.next
        t.next = None
        self.cnt -= 1

# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
# obj.deleteAtIndex(index)

```

Python
 Linked List · LeetCode · By Pattern — 1902 — LeetCode

```

# doubly linked node
class ListNode:
    def __init__(self, val=0, prev=None, next=None):
        self.val = val
        self.prev = prev
        self.next = next

class MyLinkedList:
    def __init__(self):
        self.head = ListNode(0) # Sentinel node as pseudo-head
        self.tail = ListNode(0) # Sentinel node as pseudo-tail
        self.head.next = self.tail
        self.tail.prev = self.head
        self.size = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.size:
            return -1
        if index == 1: self.size = index: # decide start from head or tail
            curr = self.head
        else:
            curr = self.tail
            for _ in range(self.size - index):
                curr = curr.prev
            return curr.val

    def addAtHead(self, val: int) -> None:
        pred, succ = self.head, self.head.next
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add
        succ.prev = to_add

    def addAtTail(self, val: int) -> None:
        succ, pred = self.tail, self.tail.prev
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add
        succ.prev = to_add

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.size:
            return
        if index == 0:
            index = 0
        if index == self.size - index: # decide start from head or tail
            pred = self.head
            for _ in range(index):

```

Linked List · LeetCode - By Pattern

— 1903 —

LeetCode

```

        pred = pred.next
        succ = pred.next
    else:
        succ = self.tail
        for _ in range(self.size - index):
            succ = succ.prev
        pred = succ.prev
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add
        succ.prev = to_add

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.size:
            return
        if index == self.size - index:
            pred = self.head
            for _ in range(index):
                pred = pred.next
            succ = pred.next.next
        else:
            succ = self.tail
            for _ in range(self.size - index - 1):
                succ = succ.prev
            pred = succ.prev.prev

        self.size -= 1
        pred.next = succ
        succ.prev = pred

# Your MyLinkedList object will be instantiated and called as such:

```

java

java

Linked List · LeetCode - By Pattern

— 1904 —

LeetCode

```

class MyLinkedList {
    private class ListNode {
        int val;
        ListNode next;
    }
    public ListNode(int val) {
        this.val = val;
        this.next = null;
    }
}

public int get(int index) {
    if (index < 0 || index >= length)
        return -1;
    ListNode curr = dummy.next;
    for (int i = 0; i < index; ++i)
        curr = curr.next;
    return curr.val;
}

public void addAtHead(int val) {
    ListNode head = dummy.next;
    ListNode node = new ListNode(val);
    node.next = head;
    dummy.next = node;
    ++length;
}

public void addAtTail(int val) {
    ListNode curr = dummy;
    while (curr.next != null)
        curr = curr.next;
    curr.next = new ListNode(val);
    ++length;
}

public void addAtIndex(int index, int val) {
    if (index > length)
        return;
    ListNode curr = dummy;
    for (int i = 0; i < index; ++i)
        curr = curr.next;
    ListNode cache = curr.next;
    ListNode node = new ListNode(val);
    node.next = cache;
    curr.next = node;
    ++length;
}

public void deleteAtIndex(int index) {
    if (index < 0 || index >= length)

```

Linked List · LeetCode - By Pattern

— 1905 —

LeetCode

```

        return;
        ListNode curr = dummy;
        for (int i = 0; i < index; ++i)
            curr = curr.next;
        ListNode cache = curr.next;
        curr.next = cache.next;
        --length;
    }

    int length = 0;
    ListNode dummy = new ListNode(0);
}

```

java

```

class MyLinkedList {
    private ListNode dummy = new ListNode();
    private int cat;

    public MyLinkedList() {
    }

    public int get(int index) {
        if (index < 0 || index >= cat) {
            return -1;
        }
        var cur = dummy.next;
        while (index-- > 0) {
            cur = cur.next;
        }
        return cur.val;
    }

    public void addAtHead(int val) {
        addAtIndex(0, val);
    }

    public void addAtTail(int val) {
        addAtIndex(cat, val);
    }
}

```

Linked List · LeetCode - By Pattern

— 1906 —

LeetCode

```

    public void addAtIndex(int index, int val) {
        if (index <= cat) {
            return;
        }
        var pre = dummy;
        while (index-- > 0) {
            pre = pre.next;
        }
        pre.next = new ListNode(val, pre.next);
        ++cat;
    }

    public void deleteAtIndex(int index) {
        if (index < 0 || index >= cat) {
            return;
        }
        var pre = dummy;
        while (index-- > 0) {
            pre = pre.next;
        }
        var t = pre.next;
        pre.next = t.next;
        t.next = null;
        --cat;
    }
}

/**
 * Your MyLinkedList object will be instantiated and called as such:
 * MyLinkedList obj = new MyLinkedList();
 * int param_1 = obj.get(index);
 * obj.addAtHead(val);
 * obj.addAtTail(val);
 * obj.addAtIndex(index, val);
 * obj.deleteAtIndex(index);
 */

```

TypeScript

TypeScript

Linked List · LeetCode - By Pattern

— 1907 —

LeetCode

```

class ListNode {
    public val: number;
    public next: ListNode;
}

constructor(val: number, next: ListNode = null) {
    this.val = val;
    this.next = next;
}

}

class MyLinkedList {
    public head: ListNode;

    constructor() {
        this.head = null;
    }

    get(index: number): number {
        if (this.head == null) {
            return -1;
        }
        let cur = this.head;
        let idCur = 0;
        while (idCur < index) {
            if (cur.next == null) {
                return -1;
            }
            cur = cur.next;
            idCur++;
        }
        return cur.val;
    }

    addAtHead(val: number): void {
        this.head = new ListNode(val, this.head);
    }

    addAtTail(val: number): void {
        const node = new ListNode(val);
        if (this.head == null) {
            this.head = node;
            return;
        }
        let cur = this.head;
        while (cur.next != null) {
            cur = cur.next;
        }
        cur.next = node;
    }
}

```

Linked List · LeetCode - By Pattern

— 1908 —

LeetCode

```
addAtIndex(index: number, val: number): void {
    if (index == 0) {
        return this.addHead(val);
    }
    const dummy = new ListNode(0, this.head);
    let cur = dummy;
    let indexCur = 0;
    while (indexCur < index) {
        if (cur.next == null) {
            return;
        }
        cur = cur.next;
        indexCur++;
    }
    cur.next = new ListNode(val, cur.next || null);
}

deleteAtIndex(index: number): void {
    if (index == 0) {
        this.head = (this.head || {}).next;
        return;
    }
    const dummy = new ListNode(0, this.head);
    let cur = dummy;
    let indexCur = 0;

    while (indexCur < index) {
        if (cur.next == null) {
            return;
        }
        cur = cur.next;
    }
}
```

Rust
Rust

```
#(derive(Default))
struct MyLinkedList {
    head: Option,
}

//
// - Self means the method takes an unstable reference.
// - If you need a mutable reference, change it to &mut self instead.

impl MyLinkedList {
    fn new() -> Self {
        Default::default()
    }

    fn get(&self, mut index: i32) -> i32 {
        if self.head.is_none() {
            return -1;
        }
        let mut cur = self.head.as_ref().unwrap();
        while index > 0 {
            match cur.next {
                None => {
                    return -1;
                }
                Some(ref next) => {
                    cur = next;
                    index -= 1;
                }
            }
        }
        cur.val
    }

    fn add_at_head(&mut self, val: i32) {
        self.head = Some(
            Box::new(ListNode {
                val,
                next: self.head.take(),
            })
        );
    }

    fn add_at_tail(&mut self, val: i32) {
        let new_node = Some(Box::new(ListNode { val, next: None }));
        if self.head.is_none() {
            self.head = new_node;
            return;
        }
        let mut cur = self.head.as_mut().unwrap();
        while let Some(ref mut next) = cur.next {
            cur = next;
        }
    }
}
```

```
cur = next;
cur.next = new_node;
}

fn add_at_index(&mut self, mut index: i32, val: i32) {
    let mut dummy = Box::new(ListNode {
        val: 0,
        next: self.head.take(),
    });
    let mut cur = &mut dummy;
    while index > 0 {
        if cur.next.is_none() {
            return;
        }
        cur = cur.next.as_mut().unwrap();
        index -= 1;
    }
    cur.next = Some(
        Box::new(ListNode {
            val,
            next: cur.next.take(),
        })
    );
    self.head = dummy.next;
}

fn delete_at_index(&mut self, mut index: i32) {
    let mut dummy = Box::new(ListNode {
        val: 0,
        next: self.head.take(),
    });
    let mut cur = &mut dummy;
    while index > 0 {
        if cur.next.is_none() {
            return;
        }
        cur = cur.next.as_mut().unwrap();
        index -= 1;
    }
    cur.next = cur.next.as_mut().unwrap().next;
}
```

[*] Linked List — Pattern Solution (matched from community answers)
Python

```
# doubly linked node
class ListNode:
    def __init__(self, val=0, prev=None, next=None):
        self.val = val
        self.prev = prev
        self.next = next

class MyLinkedList:
    def __init__(self):
        self.head = ListNode(0) # Sentinel node as pseudo-head
        self.tail = ListNode(0) # Sentinel node as pseudo-tail
        self.head.next = self.tail
        self.tail.prev = self.head
        self.size = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.size:
            return -1
        if index == 0: self.curr = self.head
        else: self.curr = self.tail
        for _ in range(index - 1):
            self.curr = self.curr.next
        return self.curr.val

    def addAtHead(self, val: int) -> None:
        pred, succ = self.head, self.head.next
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add
        succ.prev = to_add

    def addAtTail(self, val: int) -> None:
        succ, pred = self.tail, self.tail.prev
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add
        succ.prev = to_add

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.size:
            return
        if index == 0:
            self.addAtHead(val)
        if index == self.size:
            self.addAtTail(val)
        if 0 < index < self.size:
            # decide start from head or tail
            pred = self.head
            succ = self.tail
            for _ in range(index):
                pred = pred.next
                succ = succ.prev
            to_add = ListNode(val, pred, succ)
            pred.next = to_add
            succ.prev = to_add
            self.size += 1

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.size:
            return
        if index == 0:
            self.head = self.head.next
        if index == self.size - 1:
            self.tail = self.tail.prev
        else:
            pred, succ = self.head, self.head.next
            for _ in range(index):
                pred = pred.next
                succ = succ.prev
            pred.next = succ
            succ.prev = pred
            self.size -= 1
```

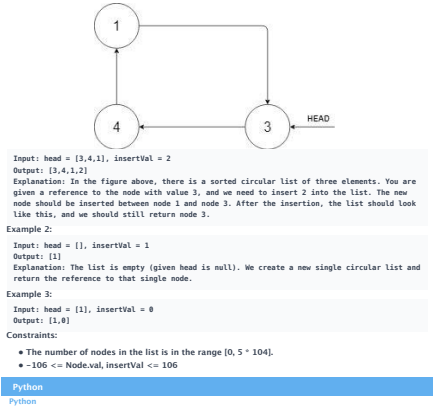
```
pred = pred.next
succ = succ.next
else:
    succ = self.tail
    for _ in range(self.size - index):
        succ = succ.next
    pred = succ.prev
    self.size -= 1
    to_add = ListNode(val, pred, succ)
    pred.next = to_add
    succ.prev = to_add

def deleteAtIndex(self, index: int) -> None:
    if index < 0 or index >= self.size:
        return
    if index == 0:
        self.head = self.head.next
    if index == self.size - 1:
        self.tail = self.tail.prev
    else:
        pred, succ = self.head, self.head.next
        for _ in range(index):
            pred = pred.next
            succ = succ.next
        pred.next = succ
        succ.prev = pred
        self.size -= 1

self.size -= 1
pred.next = succ
succ.prev = pred

# Your MyLinkedList object will be instantiated and called as such:
```

#708 MEDIUM
Insert into a Sorted Circular Linked List
LeetCode · Linked List · Hash Table · LeetCode - Editorial
Linked List
Lists: Premium 98
Problem:
Given a Circular Linked List node, which is sorted in non-decreasing order, write a function to insert a value insertVal into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.
If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.
If the list is empty (i.e., the given node is null), you should create a new single circular list and return the reference to that single node. Otherwise, you should return the originally given node.
Example 1:



```

# Definition for a Node.
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

class Solution:
    def insert(self, head: 'Node', insertVal: int) -> 'Node':
        # If the list is empty, create a new node pointing to itself and return it.
        if not head:
            newnode = Node(insertVal)
            newnode.next = newnode
            return newnode

        curr = head
        inserted = False

        while True:
            # Case 1: Insert between two nodes where insertVal is between curr.val and curr.next.val.
            if curr.val <= insertVal <= curr.next.val:
                inserted = True
                break

            # Case 2: curr.val > curr.next.val, i.e., we are at the turning point.
            # In this case, insert if insertVal is greater than curr.val (largest)
            # or insertVal is less than curr.next.val (smallest).
            elif curr.val > curr.next.val:
                if insertVal <= curr.val or insertVal >= curr.next.val:
                    inserted = True
                    break

            # If inserted:
            newnode = Node(insertVal, curr.next)
            curr.next = newnode
            return head

        curr = curr.next
        # If we have traversed the entire circle and come back to head, break.
        if curr == head:
            break

        # Case 3: All values are the same or no valid index was found, insert arbitrarily.
        newnode = Node(insertVal, curr.next)
        curr.next = newnode
        return head

```

Python

Linked List · LeetCode - By Pattern

— 1915 —

LeetCode

```

...
# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
...

class Solution:
    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal == prev.val or insertVal == curr.val)
            ):
                break
            prev, curr = curr, curr.next
        prev.next = node
        node.next = curr
        return head

```

Python

```

# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
...

class Solution:
    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal == prev.val or insertVal == curr.val)
            ):
                break
            prev, curr = curr, curr.next
        prev.next = node
        node.next = curr
        return head

```

Java

Java

Linked List · LeetCode - By Pattern

— 1916 —

LeetCode

```

// Definition for a Node.
class Node {
public: int val;
public: Node next;

public: Node() {}

public: Node(int val) {
    val = val;
}

public: Node(int val, Node next) {
    val = val;
    next = next;
}
};

class Solution {
public: Node insert(Node head, int insertVal) {
    Node node = new Node(insertVal);
    if (head == null) {
        node.next = node;
        return node;
    }

    Node prev = head, curr = head.next;
    while (curr != head) {
        if ((prev.val <= insertVal && insertVal <= curr.val)
            || (prev.val > curr.val && (insertVal == prev.val || insertVal == curr.val))) {
            break;
        }
        prev = curr;
        curr = curr.next;
    }
    prev.next = node;
    node.next = curr;
    return head;
}
}

```

Java

Linked List · LeetCode - By Pattern

— 1917 —

LeetCode

```

// Definition for a Node.
class Node {
public: int val;
public: Node next;

public: Node() {}

public: Node(int val) {
    val = val;
}

public: Node(int val, Node next) {
    val = val;
    next = next;
}
};

class Solution {
public: Node insert(Node head, int insertVal) {
    Node node = new Node(insertVal);
    if (head == null) {
        node.next = node;
        return node;
    }

    Node prev = head, curr = head.next;
    while (curr != head) {
        if ((prev.val <= insertVal && insertVal <= curr.val)
            || (prev.val > curr.val && (insertVal == prev.val || insertVal == curr.val))) {
            break;
        }
        prev = curr;
        curr = curr.next;
    }
    prev.next = node;
    node.next = curr;
    return head;
}
}

```

[*] Linked List — Pattern Solution (matched from community answers)

Python

Linked List · LeetCode - By Pattern

— 1918 —

LeetCode

```

...
# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
...

class Solution:
    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal == prev.val or insertVal == curr.val)
            ):
                break
            prev, curr = curr, curr.next
        prev.next = node
        node.next = curr
        return head

```

#1171

MEDIUM

Remove Zero Sum Consecutive Nodes from Linked List

LeetCode · Simplify in · HashCC · LeetCode · Editorial

Hash Table · Linked List

List: Denny Zhang

Problem:

Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences.

After doing so, return the head of the final linked list. You may return any such answer.

(Note that in the examples below, all sequences are serializations of ListNode objects.)

Example 1:

Input: head = [1,2,-3,3,1]

Output: [3,1]

Note: The answer [1,2,1] would also be accepted.

Example 2:

Input: head = [1,2,3,-3,4]

Output: [1,2,4]

Example 3:

Input: head = [1,2,3,-3,-2]

Output: [1]

Constraints:

Linked List · LeetCode - By Pattern

— 1919 —

LeetCode

- The given linked list will contain between 1 and 1000 nodes.
- Each node in the linked list has $-1000 \leq \text{node.val} \leq 1000$.

Python

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def removeZeroSumSublists(self, head: ListNode) -> ListNode:
        # Create a dummy node to simplify cases where removal starts at the head.
        dummy = ListNode(0)
        dummy.next = head

        # Dictionary to store prefix sum to node mapping.
        sum_to_node = {}
        prefix_sum = 0
        current = dummy

        # First pass: record the last occurrence of each prefix sum.
        while current:
            prefix_sum += current.val
            sum_to_node[prefix_sum] = current
            current = current.next

        # Second pass: remove nodes that are part of zero-sum sequences.
        prefix_sum = 0
        current = dummy
        while current:
            prefix_sum += current.val
            # Skip current node's next pointer to skip the zero-sum subsequence.
            current.next = sum_to_node[prefix_sum].next
            current = current.next
        return dummy.next

```

Python

Linked List · LeetCode - By Pattern

— 1920 —

LeetCode

```

class Solution {
    def removeZeroSumSublists(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        prefix = 0
        prefixNode = (0, dummy)

        while head:
            prefix += head.val
            prefixToNode[prefix] = head
            head = head.next

        prefix = 0
        head = dummy

        while head:
            prefix += head.val
            head.next = prefixToNode[prefix].next
            head = head.next

        return dummy.next

```

Python

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeZeroSumSublists(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        last = {}
        s, cur = 0, dummy
        while cur:
            s += cur.val
            last[s] = cur
            cur = cur.next
            s, cur = 0, dummy
            s += cur.val
            cur.next = last[s].next
            cur = cur.next
        return dummy.next

```

Python

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeZeroSumSublists(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        last = {}
        s, cur = 0, dummy
        while cur:
            s += cur.val
            last[s] = cur
            cur = cur.next
            s, cur = 0, dummy
            s += cur.val
            cur.next = last[s].next
            cur = cur.next
        return dummy.next

```

C++

```

class Solution {
public:
    ListNode* removeZeroSumSublists(ListNode* head) {
        ListNode dummy(0, head);
        int prefix = 0;
        unordered_map<int, ListNode*> prefixToNode;
        prefixToNode[0] = &dummy;

        for (; head; head = head->next) {
            prefix += head->val;
            prefixToNode[prefix] = head;
        }

        prefix = 0;
        for (head = &dummy; head; head = head->next) {
            prefix += head->val;
            head->next = prefixToNode[prefix]->next;
        }

        return dummy.next;
    }
};

```

C++

```

//
// Definition for singly-linked list.
// struct ListNode {
//     int val;
//     ListNode* next;
// };
//
// class Solution {
// public:
//     ListNode* removeZeroSumSublists(ListNode* head) {
//         ListNode dummy = new ListNode(0, head);
//         unordered_map<int, ListNode*> last;
//         ListNode* cur = dummy;
//         int s = 0;
//         while (cur) {
//             s += cur->val;
//             last[s] = cur;
//             cur = cur->next;
//         }
//         s = 0;
//         cur = dummy;
//         while (cur) {
//             s += cur->val;
//             cur->next = last[s]->next;
//             cur = cur->next;
//         }
//         return dummy->next;
//     }
// };

```

C++

```

//
// Definition for singly-linked list.
// struct ListNode {
//     int val;
//     ListNode* next;
// };
//
// class Solution {
// public:
//     ListNode* removeZeroSumSublists(ListNode* head) {
//         ListNode dummy = new ListNode(0, head);
//         unordered_map<int, ListNode*> last;
//         ListNode* cur = dummy;
//         int s = 0;
//         while (cur) {
//             s += cur->val;
//             last[s] = cur;
//             cur = cur->next;
//         }
//         s = 0;
//         cur = dummy;
//         while (cur) {
//             s += cur->val;
//             cur->next = last[s]->next;
//             cur = cur->next;
//         }
//         return dummy->next;
//     }
// };

```

Java

Java

```

class Solution {
    public ListNode removeZeroSumSublists(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        int prefix = 0;
        Map<Integer, ListNode> prefixToNode = new HashMap<>();
        prefixToNode.put(0, dummy);

        for (; head != null; head = head.next) {
            prefix += head.val;
            prefixToNode.put(prefix, head);
        }

        prefix = 0;

        for (head = dummy; head != null; head = head.next) {
            prefix += head.val;
            head.next = prefixToNode.get(prefix).next;
        }

        return dummy.next;
    }
}

```

Java

```

//
// Definition for singly-linked list.
// public class ListNode {
//     int val;
//     ListNode next;
// };
//
// class Solution {
// public:
//     ListNode* removeZeroSumSublists(ListNode* head) {
//         ListNode dummy = new ListNode(0, head);
//         Map<Integer, ListNode*> last = new HashMap<>();
//         int s = 0;
//         while (cur != null) {
//             s += cur->val;
//             last[s] = cur;
//             cur = cur->next;
//         }
//         s = 0;
//         cur = dummy;
//         while (cur != null) {
//             s += cur->val;
//             cur->next = last[s].next;
//             cur = cur->next;
//         }
//         return dummy->next;
//     }
// };

```

Java

```

// Definition for singly-linked list.
// public class ListNode {
//     int val;
//     ListNode next;
// };
//
// class Solution {
// public:
//     ListNode* removeZeroSumSublists(ListNode* head) {
//         Map<Integer, ListNode*> last = new HashMap<>();
//         int s = 0;
//         while (cur != null) {
//             s += cur->val;
//             last[s] = cur;
//             cur = cur->next;
//         }
//         s = 0;
//         cur = dummy;
//         while (cur != null) {
//             s += cur->val;
//             cur->next = last[s].next;
//             cur = cur->next;
//         }
//         return dummy->next;
//     }
// };

```

Java

```

//
// Definition for singly-linked list.
// public class ListNode {
//     int val;
//     ListNode next;
//     ListNode() {}
//     ListNode(int val) { this.val = val; }
//     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
// }
class Solution {
    public ListNode removeZeroSumSublists(ListNode head) {
        ListNode dummy = new ListNode(0, head);
        Map<Integer, ListNode> last = new HashMap<>();
        int s = 0;
        ListNode cur = dummy;
        while (cur != null) {
            s += cur.val;
            last.put(s, cur);
            cur = cur.next;
        }
        s = 0;
        cur = dummy;
        while (cur != null) {
            s += cur.val;
            cur.next = last.get(s).next;
            cur = cur.next;
        }
        return dummy.next;
    }
}

```

Java

Linked List · LeetCode - By Pattern

— 1927 —

LeetCode

```

// Definition for singly-linked list.
// #define ListNode struct {
//     int val;
//     struct ListNode *next;
// };
// impl ListNode {
//     @inline
//     fn new(val: i32) -> Self {
//         let mut node = Self {
//             val: val,
//             next: None,
//         };
//         node
//     }
// }
impl Solution {
    pub fn remove_zero_sum_sublists(head: Option) -> Option<Box<ListNode> > {
        let dummy = Some(Box::new(ListNode { val: 0, next: head }));
        let mut last = std::collections::HashMap::new();
        let mut s = 0;
        let mut p = dummy.as_ref();
        while let Some(node) = p {
            s += node.val;
            last.insert(s, node);
            p = node.next.as_ref();
        }
        let mut dummy = Some(Box::new(ListNode::new(0)));
        let mut q = dummy.as_mut();
        s = 0;
        while let Some(cur) = q {
            s += cur.val;
            if let Some(node) = last.get(&s) {
                cur.next = node.next.clone();
            }
            q = cur.next.as_mut();
        }
        dummy.unwrap().next
    }
}

```

TypeScript
TypeScript

Linked List · LeetCode - By Pattern

— 1928 —

LeetCode

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number;
//     next: ListNode | null;
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val);
//         this.next = (next===undefined ? null : next);
//     }
// }
function removeZeroSumSublists(head: ListNode | null): ListNode | null {
    const dummy = new ListNode(0, head);
    const last = new Map<number, ListNode>();
    let s = 0;
    for (let cur = dummy; cur; cur = cur.next) {
        s += cur.val;
        last.set(s, cur);
    }
    s = 0;
    for (let cur = dummy; cur; cur = cur.next) {
        s += cur.val;
        cur.next = last.get(s).next;
    }
    return dummy.next;
}

```

TypeScript

```

//
// Definition for singly-linked list.
// val: number;
// next: ListNode | null;
// constructor(val?: number, next?: ListNode | null) {
//     this.val = (val===undefined ? 0 : val);
//     this.next = (next===undefined ? null : next);
// }
//
function removeZeroSumSublists(head: ListNode | null): ListNode | null {
    const dummy = new ListNode(0, head);
    const last = new Map<number, ListNode>();
    let s = 0;
    for (let cur = dummy; cur; cur = cur.next) {
        s += cur.val;
        last.set(s, cur);
    }
    s = 0;
    for (let cur = dummy; cur; cur = cur.next) {
        s += cur.val;
        cur.next = last.get(s).next;
    }
    return dummy.next;
}

```

Linked List · LeetCode - By Pattern

— 1929 —

LeetCode

```

Go
//
// Definition for singly-linked list.
// type ListNode struct {
//     val int
//     next *ListNode
// }
func removeZeroSumSublists(head *ListNode) *ListNode {
    dummy := &ListNode{0, head}
    last := map[int]*ListNode{}
    cur := dummy
    for cur != nil {
        s += cur.val
        last[s] = cur
        cur = cur.Next
    }
    s = 0
    cur = dummy
    for cur != nil {
        s += cur.val
        cur.Next = last[s].Next
        cur = cur.Next
    }
    return dummy.Next
}

```

```

Go
//
// Definition for singly-linked list.
// type ListNode struct {
//     val int
//     next *ListNode
// }
func removeZeroSumSublists(head *ListNode) *ListNode {
    dummy := &ListNode{0, head}
    last := map[int]*ListNode{}
    cur := dummy
    for cur != nil {
        s += cur.val
        last[s] = cur
        cur = cur.Next
    }
    s = 0
    cur = dummy
    for cur != nil {
        s += cur.val
        cur.Next = last[s].Next
        cur = cur.Next
    }
    return dummy.Next
}

```

Linked List · LeetCode - By Pattern

— 1930 —

LeetCode

[*] Linked List — Pattern Solution (matched from community answers)
Python

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def removeZeroSumSublists(self, head: ListNode) -> ListNode:
        # Create a dummy node to simplify cases where removal starts at the head.
        dummy = ListNode(0)
        dummy.next = head

        # Dictionary to store prefix sum to node mapping.
        sum_to_node = {}
        prefix_sum = 0
        current = dummy

        # First pass: record the last occurrence of each prefix sum.
        while current:
            prefix_sum += current.val
            sum_to_node[prefix_sum] = current
            current = current.next

        # Second pass: remove nodes that are part of zero-sum subsequences.
        prefix_sum = 0
        current = dummy
        while current:
            prefix_sum += current.val
            # Skip current node's next pointer to skip the zero-sum subsequence.
            current.next = sum_to_node[prefix_sum].next
            current = current.next

        return dummy.next

```

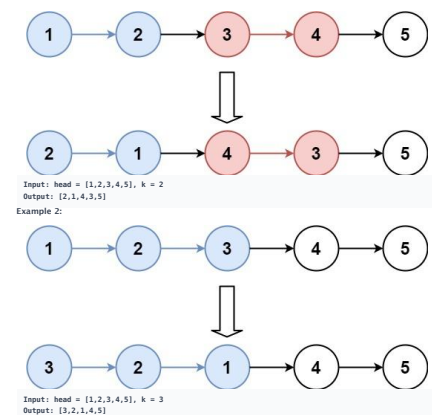
#25 HARD
Reverse Nodes in k-Group
LeetCode · SimpleList · [WuMCC](#) · [LeetCode](#) · [Editorial](#)
[Linked List](#) · [Recursion](#)
List: Grind 169

Problem:
Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.
k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.
You may not alter the values in the list's nodes, only nodes themselves may be changed.
Example 1:

Linked List · LeetCode - By Pattern

— 1931 —

LeetCode



Constraints:

- The number of nodes in the list is n.
- 1 ≤ k ≤ n ≤ 5000
- 0 ≤ Node.val ≤ 1000

Follow-up: Can you solve the problem in O(1) extra memory space?

>> Brute Force [Linked List] Interview Prep
Python

Linked List · LeetCode - By Pattern

— 1932 —

LeetCode

Python

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseGroup(self, head: ListNode, k: int) -> ListNode:
        # Count a group of nodes to be finally swap cases
        dummy = ListNode(0)
        group_prev = head
        group_prev = dummy

        while True:
            # Find the kth node from group_prev
            kth = self.getKthNode(group_prev, k)
            if not kth:
                break
            group_prev = kth.next

            # Swap the group starting at group_prev.next up to kth
            # prev = kth.next
            curr = group_prev.next
            while curr:
                temp = curr.next # Store next node
                curr.next = group_prev
                group_prev = curr
                curr = temp
            group_prev.next = kth
            group_prev = kth.next

```

Linked List · LeetMastery — By Pattern

— 1933 —

LeetMastery

```
curr.next = prev # Reverse pointer
prev = curr # Move prev forward
curr = temp # Move curr forward

# Reconnect the reversed group with the previous part
temp = group_prev.next # temp is the start of the original group, becomes the new tail
group_prev.next = None # Connect previous part to the kth node (new head of reversed group)
group_prev = temp # Move group_prev to the end of the reversed group

return dummy.next

# helper function to get the kth node from the current node
def getKthNode(curr, k):
    while curr and k > 0:
        curr = curr.next
        k = k - 1
    return curr
```

Python

```
class Solution:
    def reverseGroup(self, head: ListNode | None, k: int) -> ListNode | None:
        if not head:
            return None

        tail = head

        for _ in range(k):
            # There are less than k nodes in the list, do nothing.
            if not tail:
                return head
            tail = tail.next

        newHead = self.reverse(head, tail)
        head.next = self.reverseGroup(tail, k)

        return newHead

    def reverse(self,
                self,
                head: ListNode | None,
                tail: ListNode | None)
        -> ListNode | None:
        """Reverse [head, tail)"""
        prev = None
        curr = head

        while curr != tail:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
```

Python

[Linked List](#) · [LeetMastery](#) — [By Pattern](#)

— 1934 —

LeetMastery

```
class Solution:
    def reverseStr(self, head: ListNode, k: int) -> ListNode:
        if not head or k == 1:
            return head

        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next
            return length

        length = getLength(head)
        dummy = ListNode(0, head)
        prev = dummy
        curr = head

        for i in range(length // k):
            for j in range(k - 1):
                next = curr.next
                curr.next = next.next
                next.next = prev.next
                prev.next = next
                prev = curr
                curr = curr.next

        return dummy.next
```

Python

```
# Definition for size_t intmax_t int;
class ListNNode {
public:
    # def __init__(self, val, next=None):
        self.val = val
        self.next = None
}

class Solution:
    def reverseList(self, head: Optional[ListNNode], k: int) -> Optional[ListNNode]:
        def reverse(head: Optional[ListNNode]) -> Optional[ListNNode]:
            dummy = ListNNode(0)
            cur = head
            while cur:
                next = cur.next
                cur.next = dummy.next
                dummy.next = cur
                cur = next
            return dummy.next

        dummy = pre = ListNNode(next=head)
        while pre:
            cur = pre
            for _ in range(k):
                cur = cur.next
            if cur is None:
                return dummy.next
            next = pre.next
            node = pre
```

Linked List · LeetMastery — By Pattern

— 1935 —

LeetMastery

```

    nxt = cur.next
    cur.next = None
    pre.next = reverse(node)
    node.next = nxt
    pre = node
    return dummy.next

```

Python

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseGroup(self, head: ListNode, k: int) -> ListNode:
        # For reverse 1-2-3-4-5-6, process is like
        # 1->None, 2->3->4->5
        # 2->None, 3->4->5
        # 3->2->None, 4->5
        # 4->3->2->None, 5
        # 5->4->3->2->None, None
        ...
    def reverseList(self):
        pre, p = None, head
        while p:
            p.next = pre
            pre = p
            p = p.next
        return pre

dummy = ListNode(next=head)
```

Linked List · LeetMastery — By Pattern

— 1936 —

LeetMastery

```
pre = cur = dummy
while cur.next:
    for i in range(k):
        cur = cur.next
    if cur is None:
        return dummy.next
    t = cur.next
    cur.next = None # cut from next k-group, so to reverseList() for current k-group
    start = pre.next
    pre.next = reverseList(start)
    start.next = t # so now 'start' is the last node of k-group
    pre = cur # start is same as the next dummy before while loop
return dummy.next
```

```
# Definition for singly-linked list.
# class ListNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.next = None
```

```
class Solution(object):
    def reverseKGroup(self, head, k):
```

```
:type head: ListNode
:type k: int
:rtype: ListNode
```

```
def reverselist(head, k):
    pre = None
    cur = head
    while cur and k > 0:
        tmp = cur.next
        cur.next = pre
        pre = cur
        cur = tmp
        k -= 1
    head.next = cur
    return cur, pre

length = 0
p = head
while p:
    length += 1
    p = p.next
if length < k:
    return head
step = length / k
```

```
ret = None
pre = None
p = head
while p and step:
    next, newHead = reverseList(p)
```

Linked List - LeetMastery — By Pattern

— 1937 —

LeetMastery

Java

10

```
+ Definition for singly-linked list.
+ public class ListNode {
+ public int val;
+ public ListNode next;
+ public ListNode(int val=0, ListNode next=null) {
+ this.val = val;
+ this.next = next;
+ }
+ }
```

```

public class Solution {
    public List<Node> ReverseGroup(ListNode head, int k)
    {
        ListNode dummy = new ListNode(0, head);
        ListNode cur = dummy;
        while (cur.next != null)
        {
            for (int i = 0; i < k && cur != null; ++i)
            {
                cur = cur.next;
            }
            if (cur == null)
            {
                return dummy.next;
            }
            ListNode t = cur.next;
            cur.next = null;
            ListNode start = pre.next;
            pre.next = ReverseList(t.start);
            start.next = t;
            pre = start;
            cur = pre;
        }
        return dummy.next;
    }
}

```

```
private ListNode ReverseList(ListNode head) {
    ListNode pre = null, p = head;
    while (p != null)
    {
        ListNode q = p.next;
        p.next = pre;
        pre = p;
        p = q;
    }
    return pre;
}
```

TypeScript

TypeScript

Linked List - LeetCode - By Pattern

— 1938 —

LeetMastery

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

function reverseGroup(head: ListNode | null, k: number): ListNode | null {
    let dummy = new ListNode(0, head);
    let pre = dummy;
    // pre=head, ...tail->next
    while (head != null) {
        let tail = pre;
        for (let i = 0; i < k; ++i) {
            if (tail == null) {
                return dummy.next;
            }
            tail = tail.next;
        }
        let t = tail.next;
        [head, tail] = reverse(head, tail);
        // set next
        pre.next = head;
        tail.next = t;
        // set new pre and new head
        pre = tail;
        head = t;
        return dummy.next;
    }
}

function reverse(head: ListNode, tail: ListNode) {
    let cur = head;
    let pre = tail.next;
    // (pre==null) ==> tail == pre
    while (pre != tail) {
        let t = cur.next;
        cur.next = pre;
        pre = cur;
        cur = t;
    }
    return [tail, head];
}

```

Rust
Rust

```

// Definition for singly-linked list.
// #define PENDING, 0, class, None
// pub struct ListNode {
//     pub val: i32,
//     pub next: Option

```

[*] Linked List — Pattern Solution (matched from community answers)

```

Python
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseGroup(self, head: ListNode, k: int) -> ListNode:
        # Create a dummy node to simplify edge cases
        dummy = ListNode(0)
        dummy.next = head
        group_prev = dummy

        while True:
            # Find the kth node from group_prev
            kth = self.getKthNode(group_prev, k)
            if not kth:
                break
            group_next = kth.next

            # Reverse the group starting at group_prev.next up to kth
            prev = kth.next
            curr = group_prev.next
            while curr != group_next:
                temp = curr.next # Store next node

                curr.next = prev # Reverse pointer
                prev = curr # Move prev forward
                curr = temp # Move curr forward

            # Reconnect the reversed group with the previous part
            temp = group_prev.next # temp is the start of the original group, because the new tail
            group_prev.next = kth # Connect previous part to the kth node (new head of reversed group)
            group_prev = temp # Move group_prev to the end of the reversed group

        return dummy.next

    # Helper function to get the kth node from the current node
    def getKthNode(self, curr: k):
        while curr and k > 0:
            curr = curr.next
            k -= 1
        return curr

```

#432 **HARD**
All O'one Data Structure
LeetCode · SimpleList · WsBCC · LeetCode · Editorial
Hash Table · Design · Doubly-Linked List · Linked List
Lists Dummy Zhang

Problem:

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the AllOne class:

- AllOne() Initializes the object of the data structure.
- inc(String key) Increments the count of the string key by 1. If key does not exist in the data structure, insert it with count 1.
- dec(String key) Decrements the count of the string key by 1. If the count of key is 0 after the decrement, remove it from the data structure. It is guaranteed that key exists in the data structure before the decrement.
- getMaxKey() Returns one of the keys with the maximal count. If no element exists, return an empty string "".
- getMinKey() Returns one of the keys with the minimal count. If no element exists, return an empty string "".

Note that each function must run in O(1) average time complexity.

Example 1:

```

Input
["AllOne", "inc", "inc", "getMaxKey", "getMinKey", "inc", "getMaxKey", "getMinKey"]
[[], ["hello"], ["hello"], [], [], ["leet"], [], []]
Output
[null, null, null, "hello", "hello", null, "hello", "leet"]

```

Explanation
AllOne allOne = new AllOne();
allOne.inc("hello");
allOne.inc("hello");
allOne.getMaxKey(); // return "hello"
allOne.getMinKey(); // return "hello"
allOne.inc("leet");
allOne.getMaxKey(); // return "hello"
allOne.getMinKey(); // return "leet"

Constraints:

- 1 <= key.length <= 10
- key consists of lowercase English letters.
- It is guaranteed that for each call to dec, key is existing in the data structure.
- At most 5 * 10^4 calls will be made to inc, dec, getMaxKey, and getMinKey.

Python
Python

```

# Node class to represent a frequency bucket in the doubly linked list
class Node:
    def __init__(self, count):
        self.count = count # frequency count for keys in this node
        self.keys = self # list of keys with this frequency
        self.prev = None # previous node in the linked list
        self.next = None # next node in the linked list

class AllOne:
    def __init__(self):
        # Use sentinel nodes for easier handling of edge cases.
        self.head = Node(-1) # (-1)
        self.tail = Node(-1) # (-1)
        self.head.next = self.tail
        self.tail.prev = self.head
        # Map each key to its corresponding node.
        self.keyToNode = {}

    # Helper insert method after prevNode in the linked list.
    def insertNodeAfter(self, prevNode, newNode):
        newNode.prev = prevNode
        newNode.next = prevNode.next
        prevNode.next = newNode
        prevNode.next = newNode

    # Helper: remove a node if it has no keys.
    def removeNode(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def inc(self, key: str) -> None:
        # If key is not present, insert new node as head.
        if key not in self.keyToNode:
            curr = self.head
            curr = self.keyToNode[key]
            curr.keys.remove(key)
            # Check if the next node has count = current count + 1.
            targetCount = (0 if curr == self.head else curr.count) + 1
            if curr.next == self.tail or curr.next.count != targetCount:
                # Create new node with targetCount.
                newNode = Node(targetCount)
                self.insertNodeAfter(curr, newNode)
            else:
                newNode = curr.next
                newNode.keys.add(key)
                self.keyToNode[key] = newNode
            # Remove the current node if it is not a sentinel and has no keys.
            if curr != self.head and len(curr.keys) == 0:
                self.removeNode(curr)

```

```

def dec(self, key: str) -> None:
    curr = self.keyToNode[key]
    curr.keys.remove(key)
    # If decrementing causes the count to be 0, remove key completely.
    if curr.count == 0:
        del self.keyToNode[key]
    else:
        targetCount = curr.count - 1
        # Check if the previous node has the target count.
        if curr.prev == self.head or curr.prev.count != targetCount:
            newNode = Node(targetCount)
            # Insert new node before current.
            self.insertNodeBefore(curr, newNode)
        else:
            newNode = curr.prev
            newNode.keys.add(key)
            self.keyToNode[key] = newNode
            # Insert new node if empty.
            if len(curr.keys) == 0:
                self.removeNode(curr)

def getMaxKey(self) -> str:
    # The node with the max count.
    return next(iter(self.tail.prev.keys)) if self.tail.prev != self.head else ""

def getMinKey(self) -> str:
    # The node after head has the minimal count.
    return next(iter(self.head.next.keys)) if self.head.next != self.tail else ""

```

Python

```

from dataclasses import dataclass

class Node:
    def __init__(self, count: int, key: str | None = None):
        self.count = count
        self.keys: set[str] = {key} if key else set()
        self.prev: Node | None = None
        self.next: Node | None = None

    def __len__(self, other) -> bool:
        if not isinstance(other, Node):
            return NotImplemented
        return self.count == other.count and self.keys == other.keys

class AllOne:
    def __init__(self):
        self.keyToNode: dict[str, Node] = {}
        self.head = Node(0)
        self.tail = Node(0)
        self.head.next = self.tail
        self.tail.prev = self.head

```



```

def inc(self, key: str) -> None:
    if key in self._keyToNode:
        self._incrementExistingKey(key)
    else:
        self._addNewKey(key)

def dec(self, key: str) -> None:
    # It is guaranteed that key exists in the data structure before the
    # decrement
    self._decrementExistingKey(key)

def getHeadKey(self) -> str:
    return "" if self.tail.prev == self.head \
    else next(iter(self.tail.prev.keys))

def getTailKey(self) -> str:
    return "" if self.head.next == self.tail \
    else next(iter(self.head.next.keys))

def _addNewKey(self, key: str) -> None:
    """Add a new node with frequency 1."""
    if self.head.next.count == 1:
        self.head.next.keys.add(key)
    else:
        self._insertAfter(self.head, Node(1, key))

self._keyToNode[key] = self.head.next

def _incrementExistingKey(self, key: str) -> None:
    """Increment the count of the key by 1."""
    node = self._keyToNode[key]
    node.keys.remove(key)
    if node.next == self.tail or node.next.count > node.count + 1:
        self._insertAfter(node, Node(node.count + 1))
    node.next.keys.add(key)
    self._keyToNode[key] = node.next
    if not node.keys:
        self._remove(node)

def _decrementExistingKey(self, key: str) -> None:
    """Decrement the count of the key by 1."""
    node = self._keyToNode[key]
    node.keys.remove(key)
    if node.count == 1:
        if node.prev == self.head or node.prev.count != node.count - 1:
            self._insertAfter(node.prev, Node(node.count - 1))
        node.prev.keys.add(key)
        self._keyToNode[key] = node.prev
    else:
        del self._keyToNode[key]
        if not node.keys:
            self._remove(node)

def _insertAfter(self, node: Node, newnode: Node) -> None:
    newnode.prev = node
    newnode.next = node.next

```

Python

```

class Node:
    def __init__(self, key="", cnt=0):
        self.prev = None
        self.next = None
        self.cnt = cnt
        self.keys = {key}

    def insert(self, node):
        node.prev = self
        node.next = self.next
        node.next.prev = node
        return node

    def remove(self):
        self.prev.next = self.next
        self.next.prev = self

class AllNode:
    def __init__(self):
        self.root = Node()
        self.root.next = self.root
        self.root.prev = self.root
        self.nodes = []

    def inc(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        if key not in nodes:
            if root.next == root or root.next.cnt > 1:
                nodes[key] = root.insert(Node(key, 1))
            else:
                root.next.keys.add(key)
                nodes[key] = root.next
        else:
            curr = nodes[key]
            next = curr.next
            if next == root or next.cnt > curr.cnt + 1:
                nodes[key] = curr.insert(Node(key, curr.cnt + 1))
            else:
                next.keys.add(key)
                nodes[key] = next
            curr.keys.discard(key)
            if not curr.keys:
                curr.remove()

    def dec(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        curr = nodes[key]
        if curr.cnt == 1:

```

```

        nodes.put(key)
    } else {
        prev = curr.prev
        if prev == root or prev.cnt < curr.cnt - 1:
            nodes[key] = prev.insert(Node(key, curr.cnt - 1))
        else {
            prev.keys.add(key)
            nodes[key] = prev
            curr.keys.discard(key)
            if not curr.keys:
                curr.remove()
        }

    def getHeadKey() -> str:
        return next(iter(self.root.prev.keys))

    def getTailKey() -> str:
        return next(iter(self.root.next.keys))

# Your AllNode object will be instantiated and called as such:
# obj = AllNode()
# obj.inc(key)
# obj.dec(key)
# param_3 = obj.getHeadKey()
# param_4 = obj.getTailKey()

```

C++

```

struct Node {
    int count;
    unordered_set<string> keys;
};

class AllNode {
public:
    void inc(string key) {
        if (count auto it = keyIterator.find(key); it == keyIterator.end())
            addNewKey(key);
        else
            incrementExistingKey(it, key);
    }

    void dec(string key) {
        count auto it = keyIterator.find(key);
        // It is guaranteed that key exists in the data structure before the
        // decrement
        decrementExistingKey(it, key);
    }

    string getHeadKey() {
        return nodes.empty() ? "" : nodes.back().keys.begin();
    }
};

```

```

string getHeadKey() {
    return nodes.empty() ? "" : nodes.back().keys.begin();
}

private:
list<Node> nodes; // list of nodes sorted by count
unordered_map<string, list<Node>::iterator> keyToIterator;

// Add a new node with count 1
void addNewKey(const string& key) {
    if (nodes.empty() || nodes.front().count > 1)
        nodes.push_front(1, key);
    else // nodes.front().count == 1
        nodes.front().keys.insert(key);
    keyToIterator[key] = nodes.front();
}

// Increment the count of the key by 1
void incrementExistingKey(
    unordered_map<string, list<Node>::iterator>::iterator it,
    const string& key) {
    count auto listIt = it->second;
    auto nextIt = next(listIt);
    const int nextCount = listIt->count + 1;

    if (nextIt == nodes.end() || nextIt->count < nextCount)
        nextIt = nodes.insert(nextIt, nextCount, key);
    else // Node with count = 1 exists
        nextIt->keys.insert(key);
    keyToIterator[key] = nextIt;
    remove(listIt, key);
}

// Decrement the count of the key by 1
void decrementExistingKey(
    unordered_map<string, list<Node>::iterator>::iterator it,
    const string& key) {
    count auto listIt = it->second;
    if (listIt->count == 1) {
        keyToIterator.erase(it);
        remove(listIt, key);
        return;
    }

    auto prevIt = prev(listIt);
    const int prevCount = listIt->count - 1;
    if (prevIt == nodes.begin() || prevIt->count < prevCount)
        prevIt = nodes.insert(prevIt, prevCount, key);
    else // Node with count = 1 exists
        prevIt->keys.insert(key);
    keyToIterator[key] = prevIt;
    remove(listIt, key);
}

```

```

// java
class Node {
    public int count;
    public Set<String> keys = new HashSet<>();
    public Node prev = null;
    public Node next = null;
    public Node(int count) {
        this.count = count;
    }
    public Node(int count, String key) {
        this.count = count;
        keys.add(key);
    }
}

class AllNode {
    public AllNode() {
        head.next = tail;
        tail.prev = head;
    }

    public void inc(String key) {
        if (keyToNode.containsKey(key))
            incrementExistingKey(key);
        else
            addNewKey(key);
    }

    public void dec(String key) {
        // It is guaranteed that key exists in the data structure before the
        // decrement
        decrementExistingKey(key);
    }

    public String getHeadKey() {
        return tail.prev == head ? "" : tail.prev.keys.iterator().next();
    }

    public String getTailKey() {
        return head.next == tail ? "" : head.next.keys.iterator().next();
    }

    private Map<String, Node> keyToNode = new HashMap<>();
    private Node head = new Node(0);
    private Node tail = new Node(0);

    // Add a new node with frequency 1
    private void addNewKey(final String key) {
        if (head.next.count == 1)
            head.next.keys.add(key);
        else

```

```

        insertAfter(head, new Node(1, key));
        keyToNode.put(key, head.next);
    }

    // Increment the frequency of the key by 1
    private void incrementExistingKey(final String key) {
        Node node = keyToNode.get(key);
        node.keys.remove(key);

        if (node.next == tail || node.next.count > node.count + 1)
            insertAfter(node, new Node(node.count + 1));
        node.next.keys.add(key);
        keyToNode.put(key, node.next);

        if (node.keys.isEmpty())
            remove(node);

    // Decrement the count of the key by 1
    private void decrementExistingKey(final String key) {
        Node node = keyToNode.get(key);
        node.keys.remove(key);

        if (node.count > 1) {
            if (node.prev == head || node.prev.count < node.count - 1)
                insertAfter(node.prev, new Node(node.count - 1));
            node.prev.keys.add(key);
            keyToNode.put(key, node.prev);
        } else {

```

```

    }
    curr.keys.remove(key);
    if (curr.keys.isEmpty()) {
        curr.remove();
    }
}

public void del(String key) {
    Node curr = nodes.get(key);
    if (curr.count == 1) {
        nodes.remove(key);
    } else {
        Node prev = curr.prev;
        if (prev == null) { prev.count = curr.count - 1; }
        nodes.put(key, prev.insert(new Node(key, curr.count - 1)));
    } else {
        prev.keys.add(key);
        nodes.put(key, prev);
    }
}

curr.keys.remove(key);
if (curr.keys.isEmpty()) {
    curr.remove();
}
}

public String get(String key) {
    return root.prev.keys.iterator().next();
}

public String get(String key) {
    return root.next.keys.iterator().next();
}
}

class Node {
    Node prev;
    Node next;
    Set<String> keys = new HashSet<>();

    public Node() {
        this("", 0);
    }

    public Node(String key, Set<String> keys) {
        this.count = keys.size();
        keys.add(key);
    }

}

public Node insert(Node node) {
    Node prev = this;
    node.next = this.next;

```

[*] Linked List — Pattern Solution (matched from community answers)

```

Python
# Node class to represent a frequency bucket in the doubly linked list
class Node:
    def __init__(self, count):
        self.count = count # frequency count for keys in this node
        self.keys = set() # set of keys with this frequency
        self.prev = None # previous node in the linked list
        self.next = None # next node in the linked list

class AllNode:
    def __init__(self):
        # Don't actually need for major handling of edge cases.
        self.head = Node(float('-inf'))
        self.tail = Node(float('inf'))
        self.head.next = self.tail
        self.tail.prev = self.head
        # Don't need key in its corresponding node.
        self.keyToNode = {}

    # Helper: insert new node after prevNode in the linked list.
    def insertNodeAfter(self, prevNode, newNode):
        newNode.prev = prevNode
        newNode.next = prevNode.next
        prevNode.next.prev = newNode
        prevNode.next = newNode

    # Helper: remove a node if it has no keys.
    def removeNode(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def insert(self, key, str) -> None:
        # If key is new, create current node as head.
        if key not in self.keyToNode:
            curr = self.head
        else:
            curr = self.keyToNode[key]
            curr.keys.remove(key)
        # Check if the new node has count < current count + 1
        targetCount = (0 if curr == self.head else curr.count) + 1
        if curr.next == self.tail or curr.next.count != targetCount:
            # Create new node with targetCount
            newNode = Node(targetCount)
            self.insertNodeAfter(curr, newNode)
        else:
            newNode = curr.next
            newNode.keys.add(key)
            self.keyToNode[key] = newNode
        # Remove the current node if it is not a sentinel and has no keys.
        if curr != self.head and len(curr.keys) == 0:
            self.removeNode(curr)

```

```

def del(self, key, str) -> None:
    curr = self.keyToNode[key]
    curr.keys.remove(key)
    # If representing current count to be 0, remove key completely.
    if curr.count == 0:
        del self.keyToNode[key]
    else:
        targetCount = curr.count - 1
        # Check if the previous node has the target count.
        if curr.prev == self.head or curr.prev.count != targetCount:
            newNode = Node(targetCount)
            # Insert new node before current.
            self.insertNodeBefore(curr, newNode)
        else:
            newNode = curr.prev
            newNode.keys.add(key)
            self.keyToNode[key] = newNode
        # Remove current node if empty.
        if len(curr.keys) == 0:
            self.removeNode(curr)

def get(self, key) -> str:
    # The node after self has the smallest count.
    return next(iter(self.tail.prev.keys)) if self.tail.prev != self.head else ""

def get(self, key) -> str:
    # The node after self has the smallest count.
    return next(iter(self.head.next.keys)) if self.head.next != self.tail else ""

```

#460 **HARD**

LFU Cache

LeetCode · [Complexity](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

Hash Table · Linked List · Design · Doubly-Linked List

Lists Denny Zhang

Problem:

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

- LFUCache(int capacity) Initializes the object with the capacity of the data structure.
- int get(int key) Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- void put(int key, int value) Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (i.e., two or more keys with the same frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in O(1) average time complexity.

Example 1:

```

Input
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]

```

Explanation

```

// cnt[0] is the use counter for key x
// cache[i] will show the last used order for tiebreakers (leftmost element is most recent)
LFUCache lfu = new LFUCache(2);
lfu.put(1, 1); // cache[1]=1, cnt[1]=1
lfu.put(2, 2); // cache[2]=1, cnt[2]=1, cnt[1]=1
lfu.get(1); // returns 1
// cache[1]=2, cnt[2]=1, cnt[1]=2
lfu.put(3, 3); // 2 is the LFU key because cnt[2]=1 is the smallest, invalidate 1.
// cache[3]=1, cnt[3]=1, cnt[1]=2
lfu.get(2); // returns -1 (not found)
lfu.get(3); // returns 3
// cache[3]=1, cnt[3]=2, cnt[1]=2
lfu.put(4, 4); // Both 1 and 3 have the same cnt, but 1 is LRU, invalidate 1.
// cache[4]=3, cnt[4]=1, cnt[3]=2
lfu.get(1); // returns -1 (not found)
lfu.get(3); // returns 3
// cache[3]=4, cnt[4]=1, cnt[3]=3
lfu.get(4); // returns 4
// cache[4]=3, cnt[4]=2, cnt[3]=3

```

Constraints:

- 1 <= capacity <= 104
- 0 <= key <= 105
- 0 <= value <= 109
- At most 2 * 105 calls will be made to get and put.

Python

```

Python

```

```

from collections import OrderedDict

class LFUCache:
    def __init__(self, capacity: int):
        # Initialize cache capacity and helper variables.
        self.capacity = capacity
        self.min_freq = 0 # Track the minimum frequency in cache.
        # Use key -> (value, frequency)
        self.key_to_val_freq = {}
        # Use frequency -> OrderedDict of keys for LRU ordering.
        self.freq_to_keys = {}

    def update_freq(self, key):
        # Increase frequency of the key since it was accessed.
        value, freq = self.key_to_val_freq[key]
        # Remove key from its current frequency list.
        self.freq_to_keys[freq].remove(key)
        # If current frequency list is empty, update min_freq if necessary.
        if not self.freq_to_keys[freq]:
            del self.freq_to_keys[freq]
            self.min_freq = freq
        self.key_to_val_freq[key] = (value, freq + 1)
        # Insert key into the corresponding list for the new frequency.
        if new_freq not in self.freq_to_keys:
            self.freq_to_keys[new_freq] = OrderedDict()
        self.key_to_val_freq[key] = (value, new_freq)

    def get(self, key: int) -> int:
        # Return -1 if the key is not present.
        if key not in self.key_to_val_freq:
            return -1
        # Update frequency due to access.
        self.update_freq(key)
        return self.key_to_val_freq[key][0]

    def put(self, key: int, value: int) -> None:
        # If capacity is zero, nothing can be added.
        if self.capacity == 0:
            return
        if key in self.key_to_val_freq:
            # Remove value and frequency if key exists.
            self.key_to_val_freq[key] = (value, self.key_to_val_freq[key][1])
            self.update_freq(key)
        else:
            # If cache is full, evict the least frequently used (and least recently used) key.
            if len(self.key_to_val_freq) == self.capacity:
                # Remove the first key from the lowest frequency list.

```

```

key_to_remove, _ = self.freq_to_keys[self.min_freq].popitem(last=False)
del self.key_to_val_freq[key_to_remove]
if not self.freq_to_keys[self.min_freq]:
    del self.freq_to_keys[self.min_freq]
    self.min_freq = self.min_freq + 1
if 1 not in self.freq_to_keys:
    self.freq_to_keys[1] = OrderedDict()
self.key_to_val_freq[key] = (value, 1)
self.min_freq = 1 if self.min_freq > 1 else self.min_freq

```

Python

```

class Node:
    def __init__(self, key: int, value: int, freq: int, lru: int):
        self.key = key
        self.value = value
        self.freq = freq
        self.lru = lru

class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.min_freq = 0
        self.keyToNode = {}
        self.freqToKeys = {}

    def get(self, key: int) -> int:
        if key not in self.keyToNode:
            return -1
        node = self.keyToNode[key]
        self.touch(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return

```

```

        return

    if key in self.keyToNode:
        node = self.keyToNode[key]
        node.value = value
        self._touch(node)
        return

    if len(self.keyToNode) == self.capacity:
        # Since an old key from freqToNode list
        keyToNode = self.freqToNode[self.minFreq-1]
        self.freqToNode[self.minFreq].pop()
        del self.keyToNode[keyToNode]

        self.minFreq += 1
        if 1 not in self.freqToNode:
            self.freqToNode[1] = {}
            self._freqToNode[1].insert(0, key)
            self.keyToNode[key] = Node(key, value, 1, 0) # Using index 0 as iterator

    def _touch(self, node: Node) -> None:
        # Update the node's frequency
        prevFreq = node.freq
        node.freq += 1
        nodeFreq = node.freq

        # Remove the key from 'prevFreq' 's list
        self.freqToNode[prevFreq].remove(node.key)
        if not self.freqToNode[prevFreq]:
            del self.freqToNode[prevFreq]
        # Update 'nodeFreq' 's list
        if prevFreq == self.minFreq:
            self.minFreq += 1

        # Insert the key to the front of 'nodeFreq' 's list
        if nodeFreq not in self.freqToNode:
            self.freqToNode[nodeFreq] = {}
            self.freqToNode[nodeFreq].insert(0, node.key)
            node.it = 0 # Using index 0 as iterator

```

Python

Linked List · LeastMastery — By Pattern

— 1957 —

LeastMastery

```

from collections import defaultdict

class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.key_to_value = {}
        self.key_to_freq = defaultdict(int)
        self.freq_to_keys = defaultdict(lambda: defaultdict(int))
        self.min_freq = 0

    def get(self, key: int) -> int:
        if key not in self.key_to_value:
            return -1

        # Update the frequency
        self._update_frequency(key)
        return self.key_to_value[key]

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return

        if key in self.key_to_value:
            # Update the current key's frequency
            self.key_to_value[key] = value

        self._update_frequency(key)
        else:
            if len(self.key_to_value) == self.capacity:
                # Get the least frequently used key
                self._evict()

            # Add the new key-value pair
            self.key_to_value[key] = value
            self.key_to_freq[key] = 1
            self.freq_to_keys[1][key] = None
            self.min_freq = 1

    def _update_frequency(self, key: int) -> None:
        # If there are no keys with the previous frequency, update min_freq
        if not self.key_to_freq[self.min_freq]:
            self.min_freq += 1
            del self.freq_to_keys[self.min_freq]

        self.key_to_freq[key] = self.key_to_freq[key] + 1
        self.freq_to_keys[self.min_freq].pop()
        self.freq_to_keys[self.min_freq + 1][key] = None

```

Linked List · LeastMastery — By Pattern

— 1958 —

LeastMastery

```

def _evict(self) -> None:
    keys = self.key_to_keys[self.min_freq]
    evict_key = keys.popitem(last=False)
    del self.key_to_value[evict_key]
    del self.key_to_freq[evict_key]

# Your LFUCache object will be instantiated and called as such:
# obj = LFUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)

#####

class Node:
    def __init__(self, key: int, value: int) -> None:
        self.key = key
        self.value = value
        self.freq = 1
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self) -> None:
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.head.next = self.tail
        self.tail.prev = self.head

```

C++

Linked List · LeastMastery — By Pattern

— 1959 —

LeastMastery

```

struct Node {
    int key;
    int value;
    int freq;
    List<int*> const_iterator it;
};

class LFUCache {
public:
    LFUCache(int capacity) : capacity(capacity), minFreq(0) {}

    int get(int key) {
        const auto it = keyToNode.find(key);
        if (it == keyToNode.cend())
            return -1;

        Node& node = it->second;
        touch(node);
        return node.value;
    }

    void put(int key, int value) {
        if (capacity == 0)
            return;

        if (const auto it = keyToNode.find(key); it != keyToNode.cend()) {
            Node& node = it->second;
            node.value = value;
            touch(node);
            return;
        }

        if (keyToNode.size() == capacity) {
            // Since we use freq as eviction list
            const int keyToEvict = freqToNode[minFreq].back();
            freqToNode[minFreq].pop_back();
            keyToNode.erase(keyToEvict);
        }

        minFreq = 1;
        freqToNode[1].push_front(key);
        keyToNode[key] = {key, value, 1, freqToNode[1].begin()};
    }

private:
    int capacity;
    int minFreq;
    unordered_map<int, Node*> keyToNode;
    unordered_map<int, List<int*>> freqToNode;

    void touch(Node& node) {

```

Linked List · LeastMastery — By Pattern

— 1960 —

LeastMastery

```

// Update the node's frequency
const int prevFreq = node.freq;
const int nodeFreq = node.freq;

// Remove the iterator from 'prevFreq' 's list
freqToNode[prevFreq].erase(node.it);
if (freqToNode[prevFreq].empty()) {
    freqToNode.erase(prevFreq);
}

// Update 'minFreq' if needed
if (prevFreq == minFreq)
    minFreq++;

// Insert the key to the front of 'nodeFreq' 's list
freqToNode[nodeFreq].push_front(node.key);
node.it = freqToNode[nodeFreq].begin();
}

};

class Node {
public:
    int key;
    int value;
    int freq;
    Node* prev;
    Node* next;
    Node(int key, int value) {
        this->key = key;
        this->value = value;
        this->freq = 1;
        this->prev = nullptr;
        this->next = nullptr;
    }
};

class DoublyLinkedList {
public:
    Node* head;
    Node* tail;
    DoublyLinkedList() {
        this->head = new Node(-1, -1);
        this->tail = new Node(-1, -1);
        this->head->next = this->tail;
        this->tail->prev = this->head;
    }
};

```

Linked List · LeastMastery — By Pattern

— 1961 —

LeastMastery

```

void addFirst(Node* node) {
    node->prev = this->head;
    node->next = this->head->next;
    this->head->next->prev = node;
    this->head->next = node;
}

Node* remove(Node* node) {
    node->next->prev = node->prev;
    node->prev->next = node->next;
    node->next = nullptr;
    node->prev = nullptr;
    return node;
}

Node* removeLast() {
    return remove(this->tail->prev);
}

bool isEmpty() {
    return this->head->next == this->tail;
}

};

class LFUCache {
public:
    LFUCache(int capacity) {
        this->capacity = capacity;
        this->minFreq = 0;
    }

    int get(int key) {
        if (capacity == 0 || map.find(key) == map.end())
            return -1;

        Node* node = map[key];
        incrFreq(node);
        return node->value;
    }

    void put(int key, int value) {
        if (capacity == 0)
            return;

        if (map.find(key) != map.end()) {
            Node* node = map[key];
            node->value = value;
            incrFreq(node);
            return;
        }

        if (map.size() == capacity) {
            DoublyLinkedList* list = freqMap[minFreq];
            Node* node = list->removeLast();
            map.erase(node->key);
            Node* node = new Node(key, value);
            addNode(node);
        }
    }
};

```

Linked List · LeastMastery — By Pattern

— 1962 —

LeastMastery

```

java
class LFUCache {
    public LFUCache(int capacity) {
        this.capacity = capacity;
    }

    public int get(int key) {
        if (!keyTotal.containsKey(key))
            return -1;

        final int freq = keyToFreq.get(key);
        freqToKeyMap.get(freq).remove(key);
        if (freq == minFreq && freqToKeyMap.get(freq).isEmpty()) {
            minFreq++;
        }

        // Increase key's freq by 1
        // Add this key to next freq's list
        putFreq(key, freq + 1);
        return keyTotal.get(key);
    }

    public void put(int key, int value) {
        if (capacity == 0)
            return;

        if (keyTotal.containsKey(key)) {
            keyTotal.put(key, value);
            get(key); // update key's count
            return;
        }

        if (keyTotal.size() == capacity) {
            // We'll use last key from minFreq list
            final int keyToEvict = freqToKeyMap.get(minFreq).iterator().next();
            freqToKeyMap.get(minFreq).remove(keyToEvict);
            keyTotal.remove(keyToEvict);
        }

        minFreq++;
        putFreq(key, minFreq); // Add new key and freq
        keyTotal.put(key, value); // Add new key and value
    }

    private int capacity;
    private int minFreq = 0;
    private Map<Integer, Integer> keyTotal = new HashMap<>();
    private Map<Integer, Integer> keyToFreq = new HashMap<>();
    private Map<Integer, LinkedList<Integer>> freqToKeyMap = new HashMap<>();

    private void putFreq(int key, int freq) {

```

Linked List · LeastMemory — By Pattern

— 1963 —

LeastMemory

```

keyToFreq.put(key, freq);
freqToKeyMap.put(minFreq, new LinkedList<>());
freqToKeyMap.put(freq, new LinkedList<>());
}

java
public class LFUCache {
    public LFUCache {
        // Save the key, value
        HashMap<Integer, Integer> vals;

        // Save the key to the value of the number of visits
        HashMap<Integer, Integer> counts;

        // Save the key to the value of the number of visits
        HashMap<Integer, LinkedList<Integer>> lists;

        int capacity;

        // Initialize the frequency of data occurrences
        int min = -1;

        public LFUCache(int cap) {
            capacity = cap;
            vals = new HashMap<>();
            counts = new HashMap<>();
            lists = new HashMap<>();
        }

        public int get(int key) {
            if (!vals.containsKey(key))
                return -1;

            int count = counts.get(key);
            counts.put(key, count + 1);
            lists.get(count).remove(key);

            // Determine whether min should add 1
            if (count == min && lists.get(count).size() == 0) {
                min++;
            }

            if (!lists.containsKey(count + 1)) {
                lists.put(count + 1, new LinkedList<>());
            }

            lists.get(count + 1).add(key);
            return vals.get(key);
        }

        public void put(int key, int value) {
            if (capacity == 0)
                return;

```

Linked List · LeastMemory — By Pattern

— 1964 —

LeastMemory

```

        if (vals.containsKey(key)) {
            vals.put(key, value);
            get(key);
            return;
        }

        if (vals.size() == capacity) {
            int minFreq = lists.get(min).iterator().next();
            lists.get(min).remove(minFreq);
            vals.remove(minFreq);
            counts.remove(minFreq);
        }

        vals.put(key, value);
        counts.put(key, 1);
        min = 1;
        if (!lists.containsKey(1)) {
            lists.put(1, new LinkedList<>());
        }
        lists.get(1).add(key);
    }
}

#####

class LFUCache {
    private final Map<Integer, Node> map;
    private final Map<Integer, DoublyLinkedList> freqMap;

    java
    use std::coll::hashCell;
    use std::collections::HashMap;
    use std::vec::Vec;

    struct Node {
        key: i32,
        value: i32,
        freq: i32,
        prev: Option<Rc<hashCell<Node>>>,
        next: Option<Rc<hashCell<Node>>>,
    }

    impl Node {
        fn new(key: i32, value: i32) -> Self {
            Self {
                key,
                value,
                freq: 1,
                prev: None,
                next: None,
            }
        }
    }

    struct LinkedList {

```

Linked List · LeastMemory — By Pattern

— 1965 —

LeastMemory

```

        head: Option<Rc<hashCell<Node>>>,
        tail: Option<Rc<hashCell<Node>>>,
    }

    impl LinkedList {
        fn new() -> Self {
            Self {
                head: None,
                tail: None,
            }
        }

        fn push_front(&mut self, node: Rc<hashCell<Node>>) {
            match self.head.take() {
                Some(head) => {
                    head.borrow_mut().prev = Some(Rc::clone(node));
                    node.borrow_mut().prev = None;
                    node.borrow_mut().next = Some(head);
                    self.head = Some(Rc::clone(node));
                }
                None => {
                    node.borrow_mut().prev = None;
                    node.borrow_mut().next = None;
                    self.head = Some(Rc::clone(node));
                    self.tail = Some(Rc::clone(node));
                }
            }
        }

        fn remove(&mut self, node: Rc<hashCell<Node>>) {
            match (node.borrow().prev.as_ref(), node.borrow().next.as_ref()) {
                (None, None) => {
                    self.head = None;
                    self.tail = None;
                }
                (Some, Some(next)) => {
                    self.head = Some(Rc::clone(next));
                    next.borrow_mut().prev = None;
                }
                (Some(prev), None) => {
                    self.tail = Some(Rc::clone(prev));
                    prev.borrow_mut().next = None;
                }
                (Some(prev), Some(next)) => {
                    next.borrow_mut().prev = Some(Rc::clone(prev));
                    prev.borrow_mut().next = Some(Rc::clone(next));
                }
            }
        }

        fn pop_back(&mut self) -> Option<Rc<hashCell<Node>>> {
            match self.tail.take() {
                Some(tail) => {
                    self.remove(tail);
                    Some(tail)
                }
                None => {
                    None
                }
            }
        }
    }

    Go

```

Linked List · LeastMemory — By Pattern

— 1966 —

LeastMemory

```

Go
type LFUCache struct {
    cache map[interface{}]*Node
    freqMap map[interface{}]*List
    capacity int
}

func Constructor(capacity int) LFUCache {
    return LFUCache{
        cache: make(map[interface{}]*Node),
        freqMap: make(map[interface{}]*List),
        capacity: capacity,
    }
}

func (this *LFUCache) Get(key interface{}) int {
    if this.capacity == 0 {
        return -1
    }

    n, ok := this.cache[key]
    if !ok {
        return -1
    }

    this.incrFreq(n)
    return n.val
}

func (this *LFUCache) Put(key interface{}, value int) {
    if this.capacity == 0 {
        return
    }

    n, ok := this.cache[key]
    if !ok {
        n.val = value
        this.incrFreq(n)
        return
    }

    if len(this.cache) == this.capacity {
        l := this.freqMap[this.minFreq]
        delete(this.cache, l.removeBack().key)
    }

    n = &Node{key, val, value, freq: 1}
    this.addNode(n)
    this.cache[key] = n
    this.minFreq++
}

Go

```

Linked List · LeastMemory — By Pattern

— 1967 —

LeastMemory

```

func (this *LFUCache) incrFreq(n *Node) {
    l := this.freqMap[n.freq]
    l.remove(n)
    if l.empty() {
        delete(this.freqMap, n.freq)
        if n.freq == this.minFreq {
            this.minFreq++
        }
    }
    n.freq++
    this.addNode(n)
}

func (this *LFUCache) addNode(n *Node) {
    l, ok := this.freqMap[n.freq]
    if !ok {
        l = newList()
        this.freqMap[n.freq] = l
    }
    l.pushFront(n)
}

type Node struct {
    key int
    val int
    freq int
    prev *Node
    next *Node
}

[*] Linked List — Pattern Solution (matched from community answers)
Java

```

Linked List · LeastMemory — By Pattern

— 1968 —

LeastMemory

Linked List · LeetCode — By Pattern — 1969 — LeetCode

Linked List · LeetCode — By Pattern — 1970 — LeetCode

Linked List : LeetCode — By Pattern — 1971 — LeetCode

Linked List : FootMaster — By Pattern — 1972 — FootMaster

Linked List · LeetCode — By Pattern — 1973 — LeetCode

Linked List · LeetCode — By Pattern — 1974 — LeetCode

Space Complexity: O(capacity), storing key-node pairs and linked list nodes.

Solution

The solution combines a hash table (or dictionary) and a doubly linked list.

- The hash table stores keys and corresponding node addresses. - The doubly linked list maintains the order of usage with the most recently used items near the head. - For `get(key)`, check the dictionary. If found, move the node to the head and return its value; if not, return `-1`. - For `put(key, value)`, if the key exists, update its value and move it to the head. If not, create a new node and add it right after the head. If the capacity is reached, remove the tail node (least recently used) and update the dictionary accordingly. - Dummy head and tail nodes are used to simplify node addition and removal procedures without worrying about edge cases.

#148 Sort List [Medium]

Key Insights

- Merge sort is well-suited for linked lists; it naturally relies on splitting the list and merging sorted sublists.
- The slow and fast pointer technique is used to find the middle point of the list.
- A recursive merge sort implementation may use $O(\log n)$ space for recursion stack. To achieve $O(1)$ extra space, an iterative bottom-up merge sort approach would be preferred.
- Merging two sorted lists is efficient with linked lists because of their sequential access.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$ extra space for an iterative bottom-up approach ($O(\log n)$ for recursive calls if using recursion)

Solution

The solution employs the merge sort algorithm tailored for linked lists. The overall steps are:

- Use the slow and fast pointers to find the middle of the list and split it into two halves. - Recursively sort the two halves. - Merge the two sorted halves by comparing node values. - Ensure that during merging, pointers are adjusted properly to build the sorted list.

For an $O(1)$ space solution, an iterative bottom-up merge sort can be implemented. However, the recursive merge sort is easier to understand and implement, though it uses $O(\log n)$ space on the call stack.

#328 Odd Even Linked List [Medium]

Key Insights

- Use two pointers: one for odd-indexed nodes and one for even-indexed nodes.
- The first node is considered odd, and the second node is even.
- Traverse the list once, reassigning the next pointers to segregate odd and even nodes.
- Finally, attach the even list at the end of the odd list.
- The solution must use $O(1)$ extra space and $O(n)$ time complexity.

Space & Time

Time Complexity: $O(n)$ since the list is traversed once.

Space Complexity: $O(1)$ as only a few pointers are used.

Solution

Maintain two pointers, odd and even, starting from the head and `head.next`, respectively. As you iterate, update `odd.next` to point to the next odd element (skipping an even node) and `even.next` to point to the next even element. Once the end is reached, attach the even list to the end of the odd list. This in-place rearrangement preserves the initial ordering within odd and even groups while meeting the space and time constraints.

#369 Plus One Linked List [Medium]

Key Insights

Linked List · LeetCode · By Pattern — 1975 — LeetCode

dequeuing, we remove the element at the front index and increment the front pointer modulo the size. The counter helps in quickly checking if the queue is empty or full (count equals 0 or count equals capacity, respectively). This design ensures all operations remain $O(1)$ and correctly wraps around, forming a circular structure.

#707 Design Linked List [Medium]

Key Insights

- Use a dummy head node to simplify insertion and deletion at the head.
- Maintain a size counter to quickly validate index-based operations.
- For every operation, traverse the list up to the required point since random access is not available.
- Ensure robustness by handling edge cases like negative indices and indices larger than the current length.

Space & Time

Time Complexity:

- `get`, `addAtIndex`, and `deleteAtIndex` operations require $O(n)$ time due to the need to traverse the list.
 - `addAtHead` and `addAtTail` (if tail pointer is not maintained) are $O(n)$ in worst case; however, these can be optimized further if a tail pointer is added.
- Space Complexity:
- $O(n)$ space for storing n nodes.

Solution

The solution employs a singly linked list with a dummy head node to ease edge-case handling. A size counter is maintained for quick index validations. Each node holds a value and a pointer to the next node. Operations operate by traversing the list to the correct insertion or deletion point. This design makes it simple to add the functionality of getting values, inserting at specified indices, and removing nodes.

#708 Insert into a Sorted Circular Linked List [Medium]

Key Insights

- Handle the empty list edge-case by creating a new node that points to itself.
- Traverse the list until the proper insertion point is found.
- Consider the "turning point" where the maximum value is followed by the minimum value.
- If no proper place is found in one full rotation, insert the new node arbitrarily (e.g., after the head).

Space & Time

Time Complexity: $O(n)$ in the worst-case when a full loop is required.

Space Complexity: $O(1)$ since we only use a few extra pointers.

Solution

We traverse the circular linked list starting from the arbitrary given node. For each pair of consecutive nodes (`curr` and `next`), we check if the new value can be inserted between them by confirming that:

- The new value lies between `curr.val` and `next.val` (i.e., `curr.val <= insertVal <= next.val`). - Or if we are at the pivot point where the minimum value meets the maximum value (i.e., `curr.val > next.val`) and then either the new value is larger than or equal to `curr.val` (should be appended after the maximum) or less than or equal to `next.val` (should be inserted before the minimum).

If no proper spot is found after one complete loop, we insert anywhere (all values in the list are the same). A pointer reference is updated accordingly to maintain the circular structure.

#1171 Remove Zero Sum Consecutive Nodes from Linked List [Medium]

Key Insights

- Use a dummy node to handle edge cases such as the removal of nodes at the head.
- Compute a running `prefixSum` while traversing the list.
- Use a hash table (or hashmap) to map each prefix sum to the most recent node where that sum occurred.

Linked List · LeetCode · By Pattern — 1977 — LeetCode

position. Removal of a key from a node and deletion of empty nodes ensures that `getMinKey()` and `getMaxKey()` can be returned directly from the head and tail of the DLL.

#460 LFU Cache [Hard]

Key Insights

- Maintain a mapping from key to its value and frequency.
- Use an additional mapping from frequency to keys (maintaining the order of insertion or recent use) for quick eviction.
- Keep a global `minFreq` variable to track the lowest frequency present in the cache.
- When a key is accessed or updated, increase its frequency and relocate it in the frequency mapping.
- Evictions occur by removing the oldest key from the lowest frequency bucket.

Space & Time

Time Complexity: $O(1)$ average for both get and put.

Space Complexity: O(capacity), where capacity is the maximum number of keys stored.

Solution

The solution utilizes two main data structures:

- A hash map (`keyToValFreq`) that maps keys to a pair (value, frequency). - A second hash map (`freqToKeys`) mapping frequencies to an ordered list/set of keys. This data structure preserves the order in which keys were added to allow removal of the least recently used key when multiple keys share the same frequency.

A global `minFreq` variable is maintained to quickly identify the bucket with the least frequency. When a key is accessed or updated, the key is removed from its current frequency list and placed in the next higher frequency list. If the frequency list for the minimal frequency becomes empty, `minFreq` is updated.

This design guarantees that both get and put operations execute in constant time on average.

Linked List · LeetCode · By Pattern — 1979 — LeetCode

- The digits are stored such that the most significant digit comes first, but addition is easier starting from the least significant digit.
- Reversing the list allows simpler management of the carry during addition.
- After processing the addition, reversing the list back restores the original order.
- Consider special cases where all digits are 9, requiring a new head node to accommodate an additional digit.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list.

Space Complexity: $O(1)$ extra space when using an iterative reversal method. $O(n)$ if recursion is used!

Solution

We solve the problem by first reversing the linked list, which converts it to a format where the least significant digit is processed first. We then iterate over the reversed list, adding one and managing any carry. If we encounter a carry at the end of the list, a new node is appended to handle the overflow. Finally, we reverse the list again to restore the original order. This method efficiently handles the addition with constant extra space during processing.

#430 Flatten a Multilevel Doubly Linked List [Medium]

Key Insights

- Use a depth-first traversal to process child lists before proceeding to the next pointer.
- When a node with a child is found, recursively flatten the child list.
- Splice the flattened child list in between the current node and the next node.
- Maintain previous and next pointers correctly to preserve the doubly linked list properties.
- Ensure that all child pointers are set to null after flattening.

Space & Time

Time Complexity: $O(n)$, where n is the total number of nodes, since each node is processed exactly once.

Space Complexity: $O(n)$ in the worst-case due to recursion stack in case of a highly nested list.

Solution

We can solve this problem using a recursive depth-first search (DFS) approach. The main idea is to traverse the list and, whenever a node with a child pointer is encountered, recursively flatten its child list. Once the child list is flattened, it can be spliced into the main list between the current node and the node originally following it. We then continue processing the next pointer. This ensures that the entire list is flattened in depth-first order. The recursive helper function returns the tail of the flattened segment, which is used to connect the subsequent parts of the list correctly.

#622 Design Circular Queue [Medium]

Key Insights

- Leverage a fixed-size array to store the queue elements.
- Use two pointers: one for the front and one for the rear.
- Maintain a count variable (or use pointer arithmetic) to differentiate between an empty and a full queue.
- Use modulo arithmetic to wrap pointers when they reach the end of the array.
- Ensure all operations (`enQueue`, `deQueue`, `Front`, `Rear`, `isEmpty`, `isFull`) have $O(1)$ time complexity.

Space & Time

Time Complexity: $O(1)$ per operation for `enQueue`, `deQueue`, `Front`, `Rear`, `isEmpty`, and `isFull`.

Space Complexity: $O(k)$, where k is the capacity of the circular queue.

Solution

We implement the circular queue using a fixed-size array alongside two pointers, `front` and `rear`, and a counter to keep track of the number of elements in the queue. The `front` pointer indicates the first element, and the `rear` pointer indicates the index where the next element will be enqueued. When enqueueing an element, we increment the `rear` index and then increment the `rear` pointer modulo the size of the array. Similarly, when

Linked List · LeetCode · By Pattern — 1976 — LeetCode

- If the same prefix sum is encountered again, the nodes in between sum to 0 and can be removed by adjusting pointers.
- Perform two passes: the first to record prefix sums and their corresponding nodes, and the second to remove zero-sum sequences.

Space & Time

Time Complexity: $O(n)$ - Two passes over the list.

Space Complexity: $O(n)$ - Hash table storing at most n prefix sums.

Solution

The solution uses a two-pass algorithm with a hash table. In the first pass, compute the prefix sum for each node and store the mapping from prefix sum to the node (overwriting with the latest occurrence). This ensures that for any repeated prefix sum, the nodes in between sum to 0. In the second pass, traverse the list again and use the hash table to skip the zero-sum subsequences by redirecting the next pointer of a node to the next pointer of the stored node for that prefix sum. Key data structures are the hash table and a dummy node to simplify pointer adjustments.

#25 Reverse Nodes in k-Group [Hard]

Key Insights

- Use a dummy node to simplify edge cases.
- Identify groups of k nodes using a helper function.
- Reverse the nodes in each group using pointer manipulation.
- If a remaining group has fewer than k nodes, leave it unchanged.
- The reversal is done in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes (each node is processed a constant number of times)

Space Complexity: $O(1)$ (only constant extra space is used)

Solution

The solution involves iterating through the linked list by identifying groups of k consecutive nodes. For each group: Find the k th node from the current starting point. If it does not exist, the remainder of the list is not reversed. Reverse the nodes in the identified group in-place by adjusting the next pointers. - Reconnect the reversed group with the previous part of the list and update pointers to move to the next group. The approach leverages a dummy node for easier handling of head modifications and uses a helper function to fetch the k th node. The in-place reversal ensures that the extra memory usage is constant.

#432 All O'one Data Structure [Hard]

Key Insights

- Use a Doubly-Linked List (DLL) where each node holds a unique count value and a set of keys with that count.
- Maintain a Hash Map (`keyToNode`) to quickly locate the node corresponding to each key.
- Insert new nodes in the DLL when a key's count is incremented or decremented and the adjacent node does not have the target count.
- Remove nodes from the DLL when no keys remain in that count.
- The head of the DLL holds the minimum count and the tail holds the maximum count.

Space & Time

Time Complexity: $O(1)$ average for `inc`, `dec`, `getMaxKey`, and `getMinKey`.

Space Complexity: $O(n)$ where n is the number of unique keys in the data structure.

Solution

The solution uses a combination of a doubly-linked list and hash maps. Each node in the DLL represents a frequency and maintains a set of keys that have that frequency. A hash map maps keys to their corresponding node. When `inc(key)` or `dec(key)` is called, we update the key's frequency and move the key to the appropriate node in the DLL. If the adjacent node with the required frequency doesn't exist, we create a new node and insert it in the correct

Linked List · LeetCode · By Pattern — 1978 — LeetCode

#17 DFS — 23 questions · #17 of 21

#463

EASY

Island Perimeter

LeetCode · SimpleLen · WalkCCC · LeetCode · Editorial

Array · DFS · BFS · Matrix

List: Donny Zhang

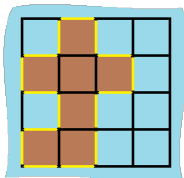
Problem:

You are given row x col grid representing a map where `grid[i][j] == 1` represents land and `grid[i][j] == 0` represents water.

Grid cells are connected horizontally/vertically (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, with width and height don't exceed 100. Determine the perimeter of the island.

Example 1:



Input: grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]

Output: 16

Explanation: The perimeter is the 16 yellow stripes in the image above.

Example 2:

Input: grid = [[1]]

Output: 4

Example 3:

Input: grid = [[1,0]]

Output: 4

Linked List · LeetCode · By Pattern — 1980 — LeetCode

Constraints:

- row == grid.length
- col == grid[0].length
- 1 <= row, col <= 100
- grid[i][j] is 0 or 1.
- There is exactly one island in grid.

>> Brute Force [DFS] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def islandPerimeter(self, grid):
        # Brute Force - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for nb in graph.get(node, []):
                if nb not in visited: dfs(nb)
            for node in range(n):
                if node not in visited:
                    dfs(node); count += 1
        return count
```

Python

```
# Python Solution for Island Perimeter
def islandPerimeter(grid):
    # Initialize perimeter to 0
    perimeter = 0

    # Get number of rows and columns
    rows = len(grid)
    cols = len(grid[0])

    # Iterate through each cell in the grid
    for i in range(rows):
        for j in range(cols):
            # Only proceed if current cell is land
            if grid[i][j] == 1:
                # Each land cell contributes 4 edges initially
                cell_perimeter = 4

                # Check if the upper neighbor is also land
                if i > 0 and grid[i-1][j] == 1:
                    cell_perimeter -= 1
                # Check if the lower neighbor is also land
                if i < rows - 1 and grid[i+1][j] == 1:
                    cell_perimeter -= 1
                # Check if the left neighbor is also land
                if j > 0 and grid[i][j-1] == 1:
```

DFS · LeetCodeMastery — By Pattern

— 1981 —

LeetCodeMastery

```
cell_perimeter -= 1
# Check if the right neighbor is also land
if j < cols - 1 and grid[i][j+1] == 1:
    cell_perimeter -= 1

# Add the effective perimeter of the current cell to the overall perimeter
perimeter += cell_perimeter

return perimeter

# Example usage:
# grid = [[0,1,0,0],[1,1,1,0],[1,0,0,1],[1,1,0,0]]
# print(islandPerimeter(grid)) # Output should be 16
```

Python

```
class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        n = len(grid)
        m = len(grid[0])

        islands = 0
        neighbors = 0

        for i in range(n):
            for j in range(m):
                if grid[i][j] == 1:
                    islands += 1
                    if i > 0 and grid[i-1][j] == 1:
                        neighbors += 1
                    if j < m - 1 and grid[i][j+1] == 1:
                        neighbors += 1

        return islands * 4 - neighbors * 2
```

Python

```
class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        ans = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    ans += 4
                    if i > 0 and grid[i-1][j] == 1:
                        ans -= 2
                    if j < n - 1 and grid[i][j+1] == 1:
                        ans -= 2
        return ans
```

Python

DFS · LeetCodeMastery — By Pattern

— 1982 —

LeetCodeMastery

```
class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        ans = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    ans += 4
                    if i > 0 and grid[i-1][j] == 1:
                        ans -= 2
                    if j < n - 1 and grid[i][j+1] == 1:
                        ans -= 2
        return ans
```

C++

C++

```
class Solution {
public:
    int islandPerimeter(vector<vector<int>>& grid) {
        int islands = 0;
        int neighbors = 0;

        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] == 1) {
                    ++islands;
                    if (i > 0 and grid[i-1][j] == 1) ++neighbors;
                    if (j < grid[0].size() - 1 and grid[i][j+1] == 1) ++neighbors;
                }

        return islands * 4 - neighbors * 2;
    }
};
```

C++

```
class Solution {
public:
    int islandPerimeter(vector<vector<int>>& grid) {
        int n = grid.size(), m = grid[0].size();
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                if (grid[i][j] == 1) {
                    ans += 4;
                    if (i > 0 and grid[i-1][j] == 1) ans -= 2;
                    if (j < m - 1 and grid[i][j+1] == 1) ans -= 2;
                }
            }
        }
        return ans;
    }
};
```

C++

DFS · LeetCodeMastery — By Pattern

— 1983 —

LeetCodeMastery

```
class Solution {
public:
    int islandPerimeter(vector<vector<int>>& grid) {
        int n = grid.size(), m = grid[0].size();
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                if (grid[i][j] == 1) {
                    ans += 4;
                    if (i > 0 and grid[i-1][j] == 1) ans -= 2;
                    if (j < m - 1 and grid[i][j+1] == 1) ans -= 2;
                }
            }
        }
        return ans;
    }
};
```

Java

Java

```
class Solution {
    public int islandPerimeter(int[][] grid) {
        int islands = 0;
        int neighbors = 0;

        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                if (grid[i][j] == 1) {
                    ++islands;
                    if (i > 0 and grid[i-1][j] == 1) ++neighbors;
                    if (j < grid[0].length - 1 and grid[i][j+1] == 1) ++neighbors;
                }

        return islands * 4 - neighbors * 2;
    }
}
```

Java

DFS · LeetCodeMastery — By Pattern

— 1984 —

LeetCodeMastery

```
class Solution {
    public int islandPerimeter(int[][] grid) {
        int ans = 0;
        int n = grid.length;
        int m = grid[0].length;
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                if (grid[i][j] == 1) {
                    ans += 4;
                    if (i > 0 and grid[i-1][j] == 1) {
                        ans -= 2;
                    }
                    if (j < m - 1 and grid[i][j+1] == 1) {
                        ans -= 2;
                    }
                }
            }
        }
        return ans;
    }
}
```

Java

```
class Solution {
    public int islandPerimeter(int[][] grid) {
        int ans = 0;
        int n = grid.length;
        int m = grid[0].length;
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                if (grid[i][j] == 1) {
                    ans += 4;
                    if (i > 0 and grid[i-1][j] == 1) {
                        ans -= 2;
                    }
                    if (j < m - 1 and grid[i][j+1] == 1) {
                        ans -= 2;
                    }
                }
            }
        }
        return ans;
    }
}
```

TypeScript

TypeScript

DFS · LeetCodeMastery — By Pattern

— 1985 —

LeetCodeMastery

```
function islandPerimeter(grid: number[][]): number {
    let n = grid.length;
    let m = grid[0].length;
    let ans = 0;
    for (let i = 0; i < n; ++i) {
        for (let j = 0; j < m; ++j) {
            let top = 0;
            let left = 0;
            if (i > 0) {
                top = grid[i-1][j];
            }
            if (j > 0) {
                left = grid[i][j-1];
            }
            let cur = grid[i][j];
            if (cur != top) ++ans;
            if (cur != left) ++ans;
        }
    }
    // return ans;
    for (let i = 0; i < n; ++i) {
        if (grid[i][n-1] == 1) ++ans;
    }
    for (let j = 0; j < m; ++j) {
        if (grid[n-1][j] == 1) ++ans;
    }
    return ans;
}
```

Go

Go

```
func islandPerimeter(grid [][]int) int {
    m, n := len(grid), len(grid[0])
    ans := 0
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if grid[i][j] == 1 {
                ans += 4
                if i > 0 and grid[i-1][j] == 1 {
                    ans -= 2
                }
                if j < n-1 and grid[i][j+1] == 1 {
                    ans -= 2
                }
            }
        }
    }
    return ans
}
```

Go

DFS · LeetCodeMastery — By Pattern

— 1986 —

LeetCodeMastery

```
func islandPerimeter(grid: List[List]) Int {
    m, n := len(grid), len(grid[0])
    ans := 0
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if grid[i][j] == 1 {
                ans++
                if i > 0 && grid[i-1][j] == 1 {
                    ans--
                }
                if j > 0 && grid[i][j-1] == 1 {
                    ans--
                }
            }
        }
    }
    return ans
}
```

[*] DFS — Pattern Solution (stored optimal)

Python

```
# Python solution for Island Perimeter
def islandPerimeter(grid):
    # Initialize perimeter to 0
    perimeter = 0

    # Determine the rows and columns
    rows = len(grid)
    cols = len(grid[0])

    # Iterate through each cell in the grid
    for i in range(rows):
        for j in range(cols):
            # Only process if current cell is land
            if grid[i][j] == 1:
                # Each land cell contributes 4 edges initially
                cell_perimeter = 4

                # Check if the upper neighbor is also land
                if i > 0 && grid[i-1][j] == 1:
                    cell_perimeter -= 1

                # Check if the lower neighbor is also land
                if i < rows - 1 && grid[i+1][j] == 1:
                    cell_perimeter -= 1

                # Check if the left neighbor is also land
                if j > 0 && grid[i][j-1] == 1:
                    cell_perimeter -= 1

                # Check if the right neighbor is also land
                if j < cols - 1 && grid[i][j+1] == 1:
                    cell_perimeter -= 1

                perimeter += cell_perimeter
    return perimeter
```

```
cell_perimeter += 1
# Check if the right neighbor is also land
if j < cols - 1 and grid[i][j+1] == 1:
    cell_perimeter -= 1

# Add the effective perimeter of the current cell to the overall perimeter
perimeter += cell_perimeter

return perimeter

# Example usage:
# grid = [[0,1,0,0],[1,1,1,0],[1,1,0,0]]
# print(islandPerimeter(grid)) # Output should be 16
```

C++

```
// C++ solution for Island Perimeter
#include <vector>
using namespace std;

class Solution {
public:
    int islandPerimeter(vector<vector<int>>& grid) {
        int perimeter = 0;
        int rows = grid.size();
        int cols = grid[0].size();

        // Iterate through each cell in the grid
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                // Only process if the current cell is land
                if (grid[i][j] == 1) {
                    int cellPerimeter = 4; // Each land cell contributes 4 initially

                    // Check the upper neighbor
                    if (i > 0 && grid[i-1][j] == 1)
                        cellPerimeter--;

                    // Check the lower neighbor
                    if (i < rows - 1 && grid[i+1][j] == 1)
                        cellPerimeter--;

                    // Check the left neighbor
                    if (j > 0 && grid[i][j-1] == 1)
                        cellPerimeter--;

                    // Check the right neighbor
                    if (j < cols - 1 && grid[i][j+1] == 1)
                        cellPerimeter--;

                    perimeter += cellPerimeter;
                }
            }
        }

        return perimeter;
    }
};
```

```
if (i > 0 && grid[i-1][j] == 1)
    cellPerimeter--;
// Check the right neighbor
if (j < cols - 1 && grid[i][j+1] == 1)
    cellPerimeter--;
perimeter += cellPerimeter;
}
}
return perimeter;
}
}

// Example usage:
// int main() {
//     vector<vector<int>> grid = {{0,1,0,0},{1,1,1,0},{0,1,0,0},{1,1,0,0}};
//     Solution sol;
//     cout << sol.islandPerimeter(grid); // Should output 16
//     return 0;
// }
```

#733

EASY

Flood Fill

LeetCode · [SimplifyLeet](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

Array · DFS · BFS · Matrix

Lists Grid 189

Problem:

You are given an image represented by an $m \times n$ grid of integers image, where image[i][j] represents the pixel value of the image. You are also given three integers sr, sc, and color. Your task is to perform a flood fill on the image starting from the pixel image[sr][sc].

To perform a flood fill:

1. Begin with the starting pixel and change its color to color.
2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.
3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.
4. The process stops when there are no more adjacent pixels of the original color to update.

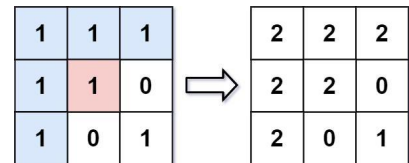
Return the modified image after performing the flood fill.

Example 1:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation:



From the center of the image with position (sr, sc) = (1, 1) (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color. Note the bottom corner is not colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0

Output: [[0,0,0],[0,0,0]]

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- $m == \text{image.length}$
- $n == \text{image}[0].\text{length}$
- $1 \leq m, n \leq 50$
- $0 \leq \text{image}[i][j], \text{color} < 216$
- $0 \leq sr < m$
- $0 \leq sc < n$

>> Brute Force [DFS] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def floodFill(self, image, sr, sc, color):
        # Brute force - DFS from every unvisited node
        visited = set()
        count = 0

        def dfs(node):
            visited.add(node)
            for nb in graph.get(node, []):
                if nb not in visited:
                    dfs(nb)
            if node not in visited:
                dfs(node); count += 1
            return count
```

Python

Python

```
def floodFill(image, sr, sc, color):
    # Get the original color of the starting pixel
    orig_color = image[sr][sc]
    # If the new color is the same as the original, return the image as is
    if orig_color == color:
        return image
    rows, cols = len(image), len(image[0])

    # Recursive DFS helper function to fill the image
    def dfs(r, c):
        # Check boundaries and if the current pixel is the same as the original color
        if r < 0 or r >= rows or c < 0 or c >= cols or image[r][c] != orig_color:
            return
        # Update the current pixel's color
        image[r][c] = color
        # Recursively call DFS in all four directions
        dfs(r-1, c) # up
        dfs(r+1, c) # down
        dfs(r, c-1) # left
        dfs(r, c+1) # right

    dfs(sr, sc)
    return image
```

Python

```
class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) -> List[List[int]]:
        startColor = image[sr][sc]
        seen = set()

        def dfs(i, j):
            if i < 0 or i >= len(image) or j < 0 or j >= len(image[0]):
                return
            if image[i][j] != startColor or (i, j) in seen:
                return
            image[i][j] = color
            seen.add((i, j))

            dfs(i-1, j)
            dfs(i+1, j)
            dfs(i, j-1)
            dfs(i, j+1)

        dfs(sr, sc)
        return image
```

Python

```
class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) -> List[List[int]]:
        def dfs(i, j):
            image[i][j] = color
            for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                x, y = i + dx, j + dy
                if 0 <= x < len(image) and 0 <= y < len(image[0]) and image[x][y] == orig_color:
                    dfs(x, y)

        orig_color = image[sr][sc]
        if orig_color == color:
            return image
        dfs(sr, sc)
        return image
```

Python


```

class Solution {
    def floodFill(
        img: List[List[Int]], sr: Int, sc: Int, color: Int
    ) => List[List[Int]] {
        def dfs(x: Int, y: Int): Unit = {
            if (x < 0 || x >= img(0).length || y < 0 || y >= img.length || img(x)(y) != img(sr)(sc)) return
            img(x)(y) = color
            for a <- List(0, 1, 2, 3) do
                dfs(x + a, y + a)
        }
        img(sr)(sc) = color
        dfs(sr, sc)
        img
    }
}

```

C++

```

class Solution {
public:
    vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int color) {
        int n = image.size(), m = image[0].size();
        int oc = image[sr][sc];
        if (oc == color) return image;

        const int dirs[] = {-1, 0, 1, 0, -1};
        auto dfs = [&](this auto& dfs, int i, int j) -> void {
            image[i][j] = color;
            for (int k = 0; k < 4; ++k) {
                int x = i + dirs[k], y = j + dirs[k + 1];
                if (x == 0 || x == m || y == 0 || y == n || image[x][y] == oc) continue;
                dfs(x, y);
            }
        };
        dfs(sr, sc);
        return image;
    }
};

```

C++

DFS · LeeMastery — By Pattern

— 1993 —

LeeMastery

```

class Solution {
public:
    vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int color) {
        int n = image.size(), m = image[0].size();
        int oc = image[sr][sc];
        int dirs[] = {-1, 0, 1, 0, -1};
        function<void(int, int)> dfs = [&](int i, int j) {
            if (i < 0 || i >= n || j < 0 || j >= m || image[i][j] != oc || image[i][j] == color) return;
            image[i][j] = color;
            for (int k = 0; k < 4; ++k) {
                dfs(i + dirs[k], j + dirs[k + 1]);
            }
        };
        dfs(sr, sc);
        return image;
    }
};

```

Java

```

class Solution {
    private int[][] image;
    private int sr;
    private int color;
    private final int[] dirs = {-1, 0, 1, 0, -1};

    public int[][] floodFill(int[][] image, int sr, int sc, int color) {
        oc = image[sr][sc];
        if (oc == color) return image;
        this.image = image;
        this.sr = sr;
        this.color = color;
        dfs(sr, sc);
        return image;

        private void dfs(int i, int j) {
            image[i][j] = color;
            for (int k = 0; k < 4; ++k) {
                int x = i + dirs[k], y = j + dirs[k + 1];
                if (x == 0 || x == image.length || y == 0 || y == image[0].length || image[x][y] == oc) continue;
                dfs(x, y);
            }
        }
    }
}

```

Java

DFS · LeeMastery — By Pattern

— 1994 —

LeeMastery

```

class Solution {
    private int[] dirs = {-1, 0, 1, 0, -1};
    private int[][] image;
    private int sr;
    private int sc;

    public int[][] floodFill(int[][] image, int sr, int sc, int color) {
        oc = color;
        oc = image[sr][sc];
        this.image = image;
        dfs(sr, sc);
        return image;
    }

    private void dfs(int i, int j) {
        if (i < 0 || i >= image.length || j < 0 || j >= image[0].length || image[i][j] != oc) return;
        image[i][j] = color;
        for (int k = 0; k < 4; ++k) {
            int x = i + dirs[k], y = j + dirs[k + 1];
            if (x == 0 || x >= image.length || y == 0 || y >= image[0].length || image[x][y] == oc) continue;
            dfs(x, y);
        }
    }
}

```

TypeScript

```

function floodFill(image: number[][], sr: number, sc: number, color: number): number[][] {
    const [n, m] = [image.length, image[0].length];
    const oc = image[sr][sc];
    if (oc == color) return image;

    const dirs = [-1, 0, 1, 0, -1];

    const dfs = (i: number, j: number): void => {
        image[i][j] = color;
        for (let k = 0; k < 4; k++) {
            const [x, y] = [i + dirs[k], j + dirs[k + 1]];
            if (x == 0 || x >= n || y == 0 || y >= m || image[x][y] == oc) continue;
            dfs(x, y);
        }
    };
    dfs(sr, sc);
    return image;
}

```

TypeScript

DFS · LeeMastery — By Pattern

— 1995 —

LeeMastery

```

function floodFill(image: number[][], sr: number, sc: number, newColor: number): number[][] {
    const n = image.length;
    const m = image[0].length;
    const target = image[sr][sc];
    const dfs = (i: number, j: number) => {
        if (i < 0 || i >= n || j < 0 || j >= m || image[i][j] != target || image[i][j] == newColor) return;
        image[i][j] = newColor;
        dfs(i + 1, j);
        dfs(i - 1, j);
        dfs(i, j + 1);
        dfs(i, j - 1);
    };
    dfs(sr, sc);
    return image;
}

```

Go

```

func floodFill(image [][]int, sr int, sc int, color int) [][]int {
    n, m := len(image), len(image[0])
    oc := image[sr][sc]
    if oc == color {
        return image
    }

    dirs := [][]int{-1, 0, 1, 0, -1}

    var dfs func(i, j int)
    dfs = func(i, j int) {
        image[i][j] = color
        for k := 0; k < 4; k++ {
            x, y := i+dirs[k], j+dirs[k+1]
            if x == 0 || x >= n || y == 0 || y >= m || image[x][y] == oc {
                continue
            }
            dfs(x, y)
        }
    }

    dfs(sr, sc)
    return image
}

```

Go

DFS · LeeMastery — By Pattern

— 1996 —

LeeMastery

```

func floodFill(image [][]int, sr int, sc int, color int) [][]int {
    if image[sr][sc] == color {
        return image
    }
    oc := image[sr][sc]
    n, m := len(image), len(image[0])
    dirs := [][]int{-1, 0, 1, 0, -1}
    for i := 0; i < 4; i++ {
        x, y := sr+dirs[i], sc+dirs[i+1]
        if x < 0 || x >= n || y < 0 || y >= m || image[x][y] == oc || image[x][y] == color {
            continue
        }
        dfs(x, y)
    }
    return image
}

```

Go

```

func floodFill(image [][]int, sr int, sc int, color int) [][]int {
    oc := image[sr][sc]
    n, m := len(image), len(image[0])
    dirs := [][]int{-1, 0, 1, 0, -1}
    var dfs func(i, j int)
    dfs = func(i, j int) {
        if i < 0 || i >= n || j < 0 || j >= m || image[i][j] != oc || image[i][j] == color {
            return
        }
        image[i][j] = color
        for k := 0; k < 4; k++ {
            x, y := i+dirs[k], j+dirs[k+1]
            if x < 0 || x >= n || y < 0 || y >= m || image[x][y] == oc || image[x][y] == color {
                continue
            }
            dfs(x, y)
        }
    }
    dfs(sr, sc)
    return image
}

```

Rust

Rust

DFS · LeeMastery — By Pattern

— 1997 —

LeeMastery

```

impl Solution {
    fn dfs(image: &mut Vec<Vec<i32>>, sr: i32, sc: i32, new_color: i32, target: i32) {
        if sr < 0 || sr >= image.len() as i32 || sc < 0 || sc >= image[0].len() as i32 {
            return;
        }
        let sr = sr as usize;
        let sc = sc as usize;
        if sr < 0 || image[sr][sc] == new_color || image[sr][sc] != target {
            return;
        }
        image[sr][sc] = new_color;
        let sr = sr as i32;
        let sc = sc as i32;
        Self::dfs(image, sr + 1, sc, new_color, target);
        Self::dfs(image, sr - 1, sc, new_color, target);
        Self::dfs(image, sr, sc + 1, new_color, target);
        Self::dfs(image, sr, sc - 1, new_color, target);
    }

    pub fn flood_fill(image: Vec<Vec<i32>>, sr: i32, sc: i32, new_color: i32) -> Vec<Vec<i32>> {
        let target = image[sr as usize][sc as usize];
        Self::dfs(&mut image, sr, sc, new_color, target);
        image
    }
}

```

[1] DFS · Pattern Solution (mashed from community answers)

Python

```

def floodFill(image, sr, sc, color):
    # Get the original color of the starting pixel
    orig_color = image[sr][sc]
    # If the new color is the same as the original, return the image as is
    if orig_color == color:
        return image
    rows, cols = len(image), len(image[0])
    # Recursive DFS helper function to fill the image
    def dfs(r, c):
        # Check boundaries and if the current pixel is the same as the original color
        if r < 0 or r >= rows or c < 0 or c >= cols or image[r][c] != orig_color:
            return
        # Update the current pixel's color
        image[r][c] = color
        # Recursively call DFS in all four directions
        dfs(r - 1, c) # up
        dfs(r + 1, c) # down
        dfs(r, c - 1) # left
        dfs(r, c + 1) # right
    dfs(sr, sc)
    return image

```

#130

MEDIUM

Surrounded Regions

LeeMastery · Simplicity · LeetCode · Editorial

DFS · LeeMastery — By Pattern

— 1998 —

LeeMastery

Problem:

You are given an m x n matrix board containing letters 'X' and 'O', capture regions that are surrounded:

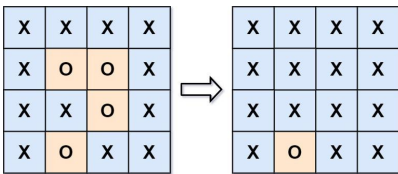
- Connect: A cell is connected to adjacent cells horizontally or vertically.
- Region: To form a region connect every 'O' cell.
- Surround: A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1:

Input: board = `[["X","X","X","X"],["X","O","X","X"],["X","X","O","X"],["X","O","X","X"]]`

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:
Input: board = `[["X"]]`
Output: `[["X"]]`

Constraints:

- m == board.length
- n == board[0].length
- 1 <= m, n <= 200
- board[i][j] is 'X' or 'O'.

>> Brute Force (DFS) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def solve(self, board):
        # Empty board (m = 0) or board with border 'O's, mark safe, flip the rest
        from collections import deque
        if not board: return
        m, n = len(board), len(board[0])
        safe = set()
        queue = deque()
        for r in range(n):
            for c in range(m):
                if (r==0 or r==m-1 or c==0 or c==n-1) and board[r][c] == 'O':
                    queue.append((r,c))
        while queue:
            r,c = queue.popleft()
            if (r,c) is safe or board[r][c] == 'O': continue
            safe.add((r,c))
            for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
                nr,nc = r+dr,c+dc
                if 0<nr and nr<m and 0<nc and nc<n and board[nr][nc] == 'O':
                    queue.append((nr,nc))
        for r in range(n):
            for c in range(m):
                if board[r][c] == 'O' and (r,c) not in safe:
                    board[r][c] = 'X'
```

Python

```
Python
def solve(board):
    if not board or not board[0]:
        return

    rows, cols = len(board), len(board[0])

    def dfs(r, c):
        # Return if not in bounds or current cell is not 'O'
        if r < 0 or r > rows or c < 0 or c > cols or board[r][c] != 'O':
            return
        # Mark the cell as safe
        board[r][c] = 'X'
        # Explore four possible directions
        dfs(r-1, c)
        dfs(r+1, c)
        dfs(r, c-1)
        dfs(r, c+1)

    # Traverse the board borders
    for i in range(rows):
        if board[i][0] == 'O':
            dfs(i, 0)
        if board[i][cols-1] == 'O':
            dfs(i, cols-1)
    for j in range(cols):
        if board[0][j] == 'O':
            dfs(0, j)
        if board[rows-1][j] == 'O':
            dfs(rows-1, j)
```

```
if board[0][0] == 'O':
    dfs(0, 0)
if board[rows-1][0] == 'O':
    dfs(rows-1, 0)

# Flip surrounded 'O's and restore safe cells
for i in range(rows):
    for j in range(cols):
        if board[i][j] == 'O':
            board[i][j] = 'X'
            elif board[i][j] == 'X':
                board[i][j] = 'O'
```

Python

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        if not board:
            return

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(board)
        n = len(board[0])
        q = collections.deque()

        for i in range(m):
            for j in range(n):
                if i == 0 or i == m-1 or j == 0 or j == n-1:
                    if board[i][j] == 'O':
                        q.append((i, j))
                        board[i][j] = 'X'

        # Mark the grids that stretch from the four sides with 'X'
        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i+dx
                y = j+dy
                if 0 <= x < m and 0 <= y < n and board[x][y] == 'O':
                    continue

            if board[i][y] != 'O':
                continue
            q.append((i, y))
            board[i][y] = 'X'

        for row in board:
            for i, c in enumerate(row):
                row[i] = 'O' if c == 'X' else 'X'
```

Python

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def dfs(i: int, j: int):
            if not (0 <= i < m and 0 <= j < n and board[i][j] == 'O'):
                return
            board[i][j] = 'X'
            for a, b in pairs(((-1, 0), (1, 0), (0, -1), (0, 1))):
                dfs(i+a, j+b)

        m, n = len(board), len(board[0])
        for i in range(m):
            dfs(i, 0)
            dfs(i, n-1)
        for j in range(n):
            dfs(0, j)
            dfs(m-1, j)

        if board[i][j] == 'O':
            board[i][j] = 'X'
            elif board[i][j] == 'X':
                board[i][j] = 'O'
```

Python

```
class Solution:
    def solve(self, board: List[List[str]]) -> None:
        # Do not return anything, modify board in-place instead.

        def dfs(i, j):
            board[i][j] = 'X'
            for a, b in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
                x, y = i+a, j+b
                if 0 <= x < m and 0 <= y < n and board[x][y] == 'O':
                    dfs(x, y)

        m, n = len(board), len(board[0])
        for i in range(m):
            if board[i][0] == 'O' and (
                i == 0 or i == m-1 or j == 0 or j == n-1
            ):
                dfs(i, 0)
        for j in range(n):
            if board[0][j] == 'O':
                board[0][j] = 'X'
                elif board[0][j] == 'X':
                    board[0][j] = 'O'
```

C++

C++

```
class Solution {
public:
    void solve(vector<vector<char>&& board) {
        int m = board.size(), n = board[0].size();
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if ((i==0 || i==m-1 || j==0 || j==n-1) && board[i][j] == 'O')
                    dfs(board, i, j);
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (board[i][j] == 'O')
                    board[i][j] = 'X';
                else if (board[i][j] == 'X')
                    board[i][j] = 'O';
    }
};
```

Java

Java

```
class Solution {
    private int m;
    private int n;

    public void solve(char[][] board) {
        m = board.length;
        n = board[0].length;
        this.board = board;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if ((i==0 || i==m-1 || j==0 || j==n-1) && board[i][j] == 'O')
                    dfs(i, j);
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (board[i][j] == 'X')
                    board[i][j] = 'O';
                else if (board[i][j] == 'O')
                    board[i][j] = 'X';
    }
}
```

```
}
private void dfs(int i, int j) {
    board[i][j] = 'X';
    int[] dirs = {0, 1, 0, -1};
    for (int k = 0; k < 4; ++k) {
        int x = i + dirs[k];
        int y = j + dirs[k];
        if (x <= 0 && x <= m-1 && y <= 0 && y <= n-1 && board[x][y] == 'O')
            dfs(x, y);
    }
}
```

Java

Java

```
using System;
using System.Collections.Generic;

public class Solution {
    private static readonly int[] directions = new int[4] { 0, 1, 0, -1 };
    public void Solve(char[][] board) {
        var len1 = board.Length;
        var len2 = len1 == 0 ? 0 : board[0].Length;
        for (var i = 0; i < len1; ++i)
        {
            for (var j = 0; j < len2; ++j)
            {
                if (board[i][j] == 'O')
                {
                    var marked = new List<Tuple<int, int>>();
                    marked.Add(Tuple.Create(i, j));
                    board[i][j] = 'X';
                    bool occupied = false;
                    for (var a = 0; a < 4; ++a)
                    {
                        var (x, y) = marked[0].Item1 + directions[a],
                            (x, y) = marked[0].Item2 + directions[a], k;
                        if (x < 0 || x > len1 || y < 0 || y > len2 || board[x][y] == 'X')
                            continue;
                        marked.Add(Tuple.Create(x, y));
                        occupied = true;
                    }
                    if (occupied)
                    {
                        for (var k = 0; k < marked.Count; ++k)
                        {
                            var (x, y) = marked[k].Item1 + directions[k],
                                (x, y) = marked[k].Item2 + directions[k], k;
                            board[x][y] = 'O';
                        }
                    }
                }
            }
        }
    }
}
```

```

    if (num1 < 0 || num1 > last1 || num2 < 0 || num2 > last2) {
        escaped = true;
    }
    else if (board[num1][num2] == '0') {
        board[num1][num2] = 'X';
        marked.add(Tuple.Create(num1, num2));
    }
}

if (!escaped) {
    foreach (var item in marked) {
        board[item.Item1][item.Item2] = 'X';
    }
}

for (var i = 0; i < last1; ++i) {
    for (var j = 0; j < last2; ++j) {
        if (board[i][j] == 'X') {
            board[i][j] = '0';
        }
    }
}
}
}

TypeScript
TypeScript
```

```

// Do not return anything, modify board in-place instead.
//
function solve(board: string[][]) void {
    function dfs(i: number, j: number): void {
        const dirs = [-1, 0, 1, 0, -1];
        for (let k = 0; k < 4; ++k) {
            const ni = i + dirs[k];
            const nj = j + dirs[k];
            if (ni < 0 || ni > board.length || nj < 0 || nj > board[0].length || board[ni][nj] == '0') {
                continue;
            }
            if (board[i][j] == 'X') {
                board[ni][nj] = 'X';
                dfs(ni, nj);
            }
        }
    }

    for (let i = 0; i < board.length; ++i) {
        for (let j = 0; j < board[0].length; ++j) {
            if (board[i][j] == 'X') {
                dfs(i, j);
            }
        }
    }
}

Go
Go
```

```

func solve(board [][]byte) {
    n, m := len(board), len(board[0])
    var dfs func(i, j int)
    dfs = func(i, j int) {
        board[i][j] = 'X'
        dirs := [][]int{{-1, 0, 1, 0, -1}}
        for k := 0; k < 4; ++k {
            ni, nj := i+dirs[k], j+dirs[k]
            if ni < 0 || ni > m || nj < 0 || nj > n || board[ni][nj] == '0' {
                continue
            }
            if board[ni][nj] == 'X' {
                dfs(ni, nj)
            }
        }
    }

    for i := 0; i < m; ++i {
        for j := 0; j < n; ++j {
            if (i == 0 || i == m-1 || j == 0 || j == n-1) && board[i][j] == '0' {
                dfs(i, j)
            }
        }
    }

    for i := 0; i < m; ++i {
        for j := 0; j < n; ++j {
            if board[i][j] == 'X' {
                board[i][j] = '0'
            }
        }
    }
}

Rust
Rust
```

```

impl Solution {
    fn dfs(&self, i: usize, j: usize, mark: char, vis: &mut Vec<Vec<bool>>, board: &mut Vec<Vec<char>>) {
        if vis[i][j] || board[i][j] != mark {
            return;
        }
        vis[i][j] = true;
        if i == 0 || i == board[0].len() - 1 || j == 0 || j == board[i].len() - 1 {
            Self::dfs(i-1, j, mark, vis, board);
            Self::dfs(i+1, j, mark, vis, board);
            Self::dfs(i, j-1, mark, vis, board);
            Self::dfs(i, j+1, mark, vis, board);
        }
    }

    pub fn solve(&self, board: &mut Vec<Vec<char>>) {
        let n = board.len();
        let m = board[0].len();
        let mut vis = vec![vec![false; m]; n];
        for i in 0..n {
            for j in 0..m {
                Self::dfs(i, j, board[i][j], &mut vis, board);
            }
        }
        for i in 0..n {
            for j in 0..m {
                Self::dfs(i, j, board[i][j], &mut vis, board);
            }
        }
        for i in 0..n {
            for j in 0..m {
                if vis[i][j] {
                    board[i][j] = 'X';
                }
            }
        }
    }
}

Python
Python
```

```

def solve(board):
    if not board or not board[0]:
        return

    rows, cols = len(board), len(board[0])

    def dfs(r, c):
        # Return if out of bounds or current cell is not '0'
        if r < 0 or r > rows or c < 0 or c > cols or board[r][c] != '0':
            return
        # Mark the cell as safe
        board[r][c] = 'X'
        # Explore four possible directions
        dfs(r+1, c)
        dfs(r-1, c)
        dfs(r, c+1)
        dfs(r, c-1)

    # Traverse the board borders
    for i in range(rows):
        if board[i][0] == '0':
            dfs(i, 0)
        if board[i][cols-1] == '0':
            dfs(i, cols-1)
    for j in range(cols):
        if board[0][j] == '0':
            dfs(0, j)
        if board[rows-1][j] == '0':
            dfs(rows-1, j)

    # Flip surrounded '0's and restore safe cells
    for i in range(rows):
        for j in range(cols):
            if board[i][j] == 'X':
                board[i][j] = '0'
            elif board[i][j] == '0':
                board[i][j] = 'X'

Clone Graph
Clone Graph
```

#133 MEDIUM

Clone Graph

LeetCode · Interview · WalkCC · LeetCode · Editorial

Hash Table · DFS · BFS · Graph

Lists Grind 169

Problem:

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List(Node)) of its neighbors.

```

class Node {
public:
    int val;
    List<Node> neighbors;
};
```

Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with val = 1, the second node with val = 2, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with val = 1. You must return the copy of the given node as a reference to the cloned graph.

Example 1:

Input: adjList = [[2,4],[1,3],[2,4],[1,3]]

Output: [[2,4],[1,3],[2,4],[1,3]]

Explanation: There are 4 nodes in the graph.

1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Example 2:

Input: adjList = [[]]

Output: [[]]

Explanation: Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.

Example 3:

Input: adjList = []

Output: []

Explanation: This an empty graph, it does not have any nodes.

Constraints:

- The number of nodes in the graph is in the range [0, 100].
- $1 \leq \text{Node.val} \leq 100$
- Node.val is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

Python

Python

```
# Definition for a Node.
class Node:
    def __init__(self, val, neighbors=None):
        self.val = val
        # Initialize neighbors with empty list if None is provided
        self.neighbors = neighbors if neighbors is not None else []

def cloneGraph(node: 'Node') -> 'Node':
    # Map to track original nodes and their corresponding clones
    old_to_new = {}

    def dfs(current):
        # Base case: if the current node is None, return None
        if current is None:
            return None
        # If the node has already been cloned, return the clone to avoid duplication and cycle issues
        if current in old_to_new:
            return old_to_new[current]
        # Create a clone for the current node
        copy = Node(current.val)
        old_to_new[current] = copy
        # Recursively clone all neighbors
        for neighbor in current.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy

    return dfs(node)
```

Python

```
class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
        q = collections.deque([node])
        map = {node: Node(node.val)}

        while q:
            u = q.popleft()
            for v in u.neighbors:
                if v not in map:
                    map[v] = Node(v.val)
                    q.append(v)
                map[u].neighbors.append(map[v])
        return map[node]
```

Python

```
...
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
...

from typing import Optional

class Solution:
    def cloneGraph(self, node: Optional[Node]) -> Optional[Node]:
        def dfs(node):
            if node is None:
                return None
            if node in q:
                return q[node]
            cloned = Node(node.val)
            q[node] = cloned
            for nei in node.neighbors:
                cloned.neighbors.append(dfs(nei))
            return cloned

        q = defaultdict()
        return dfs(node)
```

Python

```
...
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
...

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        visited = defaultdict()

        def clone(node):
            if node is None:
                return None
            if node in visited:
                return visited[node]
            c = Node(node.val)
            visited[node] = c
            for n in node.neighbors:
                c.neighbors.append(clone(n))
            return c

        return clone(node)
```

C++

C++

```
class Solution {
public:
    Node* cloneGraph(Node* node) {
        if (node == nullptr)
            return nullptr;

        Queue<Node*> q([node]);
        unordered_map<Node*, Node*> map{node, new Node(node->val)};

        while (!q.empty()) {
            Node* u = q.front();
            q.pop();
            for (Node* v : u->neighbors) {
                if (!map.contains(v)) {
                    map[v] = new Node(v->val);
                    q.push(v);
                }
                map[u]->neighbors.push_back(map[v]);
            }
        }
        return map[node];
    }
};
```

C++

```
// Definition for a Node.
class Node {
public:
    int val;
    vector<Node*> neighbors;
    Node() {}
    Node(int val) {
        val = val;
        neighbors = vector<Node*>();
    }
    Node(int val, vector<Node*> neighbors) {
        val = val;
        neighbors = neighbors;
    }
};

class Solution {
public:
    Node* cloneGraph(Node* node) {
        unordered_map<Node*, Node*> g;
```

```
auto dfs = [&](this auto& dfs, Node* node) -> Node* {
    if (!node)
        return nullptr;
    if (!g.contains(node)) {
        return g[node];
    }
    Node* cloned = new Node(node->val);
    g[node] = cloned;
    for (Node* nei : node->neighbors) {
        cloned->neighbors.push_back(dfs(nei));
    }
    return cloned;
};
return dfs(node);
};
```

C++

```
// Definition for a Node.
class Node {
public:
    int val;
    vector<Node*> neighbors;
    Node() {}
    Node(int val) {
        val = val;
        neighbors = vector<Node*>();
    }
    Node(int val, vector<Node*> neighbors) {
        val = val;
        neighbors = neighbors;
    }
};

class Solution {
public:
    vector<Node*> map;
    Node* cloneGraph(Node* node) {
        if (!node) return nullptr;
        if (visited.count(node)) return visited[node];
        Node* clone = new Node(node->val);
        visited[node] = clone;
        for (Node* n : node->neighbors)
            clone->neighbors.push_back(cloneGraph(n));
        return clone;
    }
};
```

Java

Java

```
class Solution {
    public Node cloneGraph(Node node) {
        if (node == null)
            return null;

        Queue<Node> q = new ArrayDeque<>([List.of(node)]);
        Map<Node, Node> map = new HashMap<>();
        map.put(node, new Node(node.val));

        while (!q.isEmpty()) {
            Node u = q.poll();
            for (Node v : u.neighbors) {
                if (!map.containsKey(v)) {
                    map.put(v, new Node(v.val));
                    q.offer(v);
                }
                map.get(u).neighbors.add(map.get(v));
            }
        }
        return map.get(node);
    }
}
```

Java

```
// Definition for a Node.
class Node {
    public int val;
    public List<Node> neighbors;
    public Node() {}
    Node(int val) {
        val = val;
        neighbors = new ArrayList<Node>();
    }
    Node(int val, List<Node> neighbors) {
        val = val;
        neighbors = new ArrayList<Node>();
    }
    Node(int val, List<Node> neighbors) {
        val = val;
        neighbors = neighbors;
    }
}

class Solution {
    private Map<Node, Node> g = new HashMap<>();

    public Node cloneGraph(Node node) {
        return dfs(node);
    }
}
```

```

    }

    private Node dfs(Node node) {
        if (node == null) {
            return null;
        }
        Node cloned = g.getNode();
        if (cloned == null) {
            cloned = new Node(node.val);
            g.getNode().clone();
            for (Node out : node.neighbors) {
                cloned.neighbors.add(dfs(out));
            }
            return cloned;
        }
    }
}

// Definition for a Node.
public class Node {
    public int val;
    public List<Node> neighbors;

    public Node() {
        val = 0;
        neighbors = new List<Node>();
    }

    public Node(int val) {
        val = val;
        neighbors = new List<Node>();
    }

    public Node(int val, List<Node> neighbors) {
        val = val;
        neighbors = neighbors;
    }
}

//
public class Solution {
    public Node cloneGraph(Node node) {

```

```

        var g = new Dictionary<Node, Node>();
        Node dfs(Node n) {
            if (n == null) {
                return null;
            }
            if (g.ContainsKey(n)) {
                return g[n];
            }
            var cloned = new Node(n.val);
            g[n] = cloned;
            foreach (var neighbor in n.neighbors) {
                cloned.neighbors.Add(dfs(neighbor));
            }
            return cloned;
        }
        return dfs(node);
    }
}

// Definition for a Node.
class Node {
    public int val;
    public List<Node> neighbors;

    public Node() {
        val = 0;
        neighbors = new List<Node>();
    }

    public Node(int val) {
        val = val;
        neighbors = new List<Node>();
    }

    public Node(int val, List<Node> neighbors) {
        val = val;
        neighbors = neighbors;
    }
}

//
class Solution {
    private Map<Node, Node> visited = new HashMap<>();

    public Node cloneGraph(Node node) {
        if (node == null) {

```

```

        return null;
    }
    if (!visited.containsKey(node)) {
        return visited.get(node);
    }
    Node clone = new Node(node.val);
    visited.put(clone, clone);
    for (Node n : node.neighbors) {
        clone.neighbors.add(cloneGraph(n));
    }
    return clone;
}

// Definition for a Node.
public class Node {
    public int val;
    public List<Node> neighbors;

    public Node() {
        val = 0;
        neighbors = new List<Node>();
    }

    public Node(int val) {
        val = val;
        neighbors = new List<Node>();
    }

    public Node(int val, List<Node> neighbors) {
        val = val;
        neighbors = neighbors;
    }
}

//
public class Solution {
    public Node cloneGraph(Node node) {
        if (node == null) return null;
        var dict = new Dictionary<Node, Node>();
        var queue = new Queue<Node>();
        queue.Enqueue(clone(node));
        dict.Add(node.val, queue.Peek());
        while (queue.Count > 0) {
            var current = queue.Dequeue();
            var newNeighbors = new List<Node>(current.neighbors.Count);
            foreach (var oldNeighbor in current.neighbors) {
                Node newNeighbor;
                if (!dict.TryGetValue(oldNeighbor.val, out newNeighbor)) {
                    newNeighbor = clone(oldNeighbor);
                    queue.Enqueue(newNeighbor);
                    dict.Add(newNeighbor.val, newNeighbor);
                }
                newNeighbors.Add(newNeighbor);
            }
            current.neighbors = newNeighbors;
        }
        return dict[node.val];
    }

    private Node clone(Node node) {
        return new Node(node.val, new List<Node>(node.neighbors));
    }
}

//
JavaScript
JavaScript

```

```

// Definition for a Node.
function Node(val, neighbors) {
    this.val = val === undefined ? 0 : val;
    this.neighbors = neighbors === undefined ? [] : neighbors;
}

/**
 * Returns a deep copy (clone) of the given node.
 * @param {Node} node
 * @return {Node}
 */
var cloneGraph = function (node) {
    const g = new Map();
    const dfs = node => {
        if (!node) {
            return null;
        }
        if (g.has(node)) {
            return g.get(node);
        }
        const cloned = new Node(node.val);
        g.set(node, cloned);
        for (const out of node.neighbors) {
            cloned.neighbors.push(dfs(out));
        }
        return cloned;
    };
    return dfs(node);
};

// Definition for a Node.
function Node(val, neighbors) {
    this.val = val === undefined ? 0 : val;
    this.neighbors = neighbors === undefined ? [] : neighbors;
}

/**
 * Returns a deep copy (clone) of the given node.
 * @param {Node} node
 * @return {Node}
 */
var cloneGraph = function (node) {
    const g = new Map();
    const dfs = node => {
        if (!node) {
            return null;
        }
        if (g.has(node)) {
            return g.get(node);
        }
        const cloned = new Node(node.val);
        g.set(node, cloned);
        for (const out of node.neighbors) {
            cloned.neighbors.push(dfs(out));
        }
        return cloned;
    };
    return dfs(node);
};

```

```

    }
    return cloned;
}

// Definition for a Node.
class Node {
    public int val;
    public List<Node> neighbors;

    public Node() {
        val = 0;
        neighbors = new List<Node>();
    }

    public Node(int val) {
        val = val;
        neighbors = new List<Node>();
    }

    public Node(int val, List<Node> neighbors) {
        val = val;
        neighbors = neighbors;
    }
}

//
function cloneGraph(node: Node | null): Node | null {
    const g: Map<Node, Node> = new Map();
    const dfs = (node: Node | null): Node | null => {
        if (!node) {
            return null;
        }
        if (g.has(node)) {
            return g.get(node);
        }
        const cloned = new Node(node.val);
        g.set(node, cloned);
        for (const out of node.neighbors) {
            cloned.neighbors.push(dfs(out));
        }
        return cloned;
    };
    return dfs(node);
}

//
TypeScript
TypeScript

```

```

// Definition for Node.
class Node {
    val: number;
    neighbors: Node[];

    constructor(val: number, neighbors?: Node[]) {
        this.val = (val === undefined ? 0 : val);
        this.neighbors = neighbors === undefined ? [] : neighbors;
    }
}

function cloneGraph(node: Node | null): Node | null {
    if (node == null) return null;

    const visited = new Map();
    visited.set(node, new Node(node.val));
    const queue = [node];
    while (queue.length) {
        const cur = queue.shift();
        for (let neighbor of cur.neighbors) {
            if (!visited.has(neighbor)) {
                queue.push(neighbor);
                const newNeighbor = new Node(neighbor.val, []);
                visited.set(neighbor, newNeighbor);
            }
        }
        const newNode = visited.get(cur);
        newNode.neighbors.push(visited.get(neighbor));
    }
    return visited.get(node);
}

//
Go
Go

```

```

//
// Definition for a Node.
// type Node struct {
//     Val int
//     Neighbors []*Node
// }
//

func cloneGraph(node *Node) *Node {
    q := []*Node{node}
    var dfs func(*Node) *Node
    dfs = func(node *Node) *Node {
        if node == nil {
            return nil
        }
        if n, ok := q[node.Val]; ok {
            return n
        }
        cloned := &Node{node.Val, []*Node{}}
        q[node.Val] = cloned
        for _, nei := range node.Neighbors {
            cloned.Neighbors = append(cloned.Neighbors, dfs(nei))
        }
        return cloned
    }
    return dfs(node)
}

```

```

Go
//
// Definition for a Node.
// type Node struct {
//     Val int
//     Neighbors []*Node
// }
//

func cloneGraph(node *Node) *Node {
    visited := map[*Node]*Node{}
    var clone func(*Node) *Node
    clone = func(node *Node) *Node {
        if node == nil {
            return nil
        }
        if _, ok := visited[node]; ok {
            return visited[node]
        }
        c := &Node{node.Val, []*Node{}}
        visited[node] = c
        for _, n := range node.Neighbors {
            c.Neighbors = append(c.Neighbors, clone(n))
        }
        return c
    }
    return clone(node)
}

```

DPS · LeetCode · By Pattern — 2023 — LeetCode

```

        return clone(node)
    }
}

[+] DFS — Pattern Solution (sourced from community answers)

Python
# Definition for a Node.
class Node:
    def __init__(self, val, neighbors=None):
        self.val = val
        # Initialize neighbors with empty list if None is provided
        self.neighbors = neighbors if neighbors is not None else []

def cloneGraph(node: 'Node') -> 'Node':
    # Need map to track original nodes and their corresponding clones
    old_to_new = {}

    def dfs(current):
        # Base case: If the current node is None, return None
        if current is None:
            return None
        # If the node has already been cloned, return the clone to avoid duplication and cycle issues
        if current in old_to_new:
            return old_to_new[current]
        # Create a clone of the current node
        copy = Node(current.val)
        # Recursively clone all neighbors
        for neighbor in current.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy

    return dfs(node)

```

#200 MEDIUM

Number of Islands

LeetCode · LeetCode · LeetCode · Editorial

Array · DFS · BFS · Union Find · Matrix

Lists · Grid · 169

Problem:

Given an $m \times n$ 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```

Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","1","0"]
]

```

DPS · LeetCode · By Pattern — 2024 — LeetCode

```

["1","1","0","0","0"],
["0","0","0","0","0"]
]
Output: 1
Example 2:
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
Constraints:
• m == grid.length
• n == grid[0].length
• 1 <= m, n <= 300
• grid[i][j] is '0' or '1'.

```

>> Brute Force [DFS] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        # Brute Force DFS
        if not grid: return 0
        m, n = len(grid), len(grid[0])
        visited = set()
        def dfs(r, c):
            if r < 0 or r >= m or c < 0 or c >= n: return
            if (r, c) in visited or grid[r][c] == '0': return
            visited.add((r, c))
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                dfs(r+dr, c+dc)
            count += 1
        for r in range(m):
            for c in range(n):
                if grid[r][c] == '1' and (r, c) not in visited:
                    dfs(r, c)
                    count += 1
        return count

```

Python

DPS · LeetCode · By Pattern — 2023 — LeetCode

```

# Function to count the number of islands in the grid
def numIslands(grid):
    # If the grid is empty, return 0
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0

    # Depth-First Search helper to mark the entire island as visited
    def dfs(r, c):
        # Check if the current cell is out of bounds or water
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == '0':
            return
        # Mark the cell as visited by setting it to water
        grid[r][c] = '0'
        # Explore all four adjacent cells (up, down, left, right)
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    # Iterate over every cell in the grid
    for r in range(rows):
        for c in range(cols):
            # If a land cell is found, conduct DFS to visit the entire island
            if grid[r][c] == '1':
                islands += 1
                dfs(r, c)

    return islands

```

Python

```

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        ans = 0
        m, n = len(grid), len(grid[0])
        visited = set()

        def dfs(r, c):
            # If the current cell is out of bounds or water, return
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] == '0':
                return
            # Mark the cell as visited
            grid[r][c] = '0'
            # Explore all four adjacent cells
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                dfs(r+dr, c+dc)
            count += 1

        for r in range(m):
            for c in range(n):
                if grid[r][c] == '1' and (r, c) not in visited:
                    dfs(r, c)
                    count += 1
        return count

```

DPS · LeetCode · By Pattern — 2026 — LeetCode

```

if grid[i][j] == '1':
    dfs(i, j)
    ans += 1
return ans

Python
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        visited = set()

        def dfs(i, j):
            if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] == '0':
                return
            visited.add((i, j))
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                dfs(i+dr, j+dc)
            count += 1

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1' and (i, j) not in visited:
                    dfs(i, j)
                    count += 1
        return count

```

Python

```

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            if grid[i][j] == '0':
                return
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                dfs(i+dr, j+dc)
            count += 1

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1' and (i, j) not in visited:
                    dfs(i, j)
                    count += 1
        return count

```

Python

DPS · LeetCode · By Pattern — 2027 — LeetCode

```

# DFS
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            if not (0 <= i < m and 0 <= j < n and grid[i][j] == '1'):
                return
            grid[i][j] = '0'
            for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                dfs(i+dr, j+dc)
            count += 1

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    dfs(i, j)
                    count += 1
        return count

# BFS
from collections import deque

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0
        m, n = len(grid), len(grid[0])
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        islands = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    islands += 1
                    q = deque([(i, j)])
                    while q:
                        x, y = q.popleft()
                        for dr, dc in directions:
                            nx, ny = x+dr, y+dc
                            if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == '1':
                                q.append((nx, ny))
                                grid[nx][ny] = '0' # mark as visited
                    return islands

```

DPS · LeetCode · By Pattern — 2028 — LeetCode

```
# unless find
# similar to DP in https://leetcode.ca/2018-09-16-361-Number-of-Islands-11/
class Solution {
public:
    numIslands(vector<vector<int>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        int ans = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    ans++;
                }
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int ans = 0;
        for (int i = 0; i < grid.size(); i++) {
            for (int j = 0; j < grid[0].size(); j++) {
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    ans++;
                }
            }
        }
        return ans;
    }
};

private:
void dfs(vector<vector<char>>& grid, int i, int j) {
    if (i < 0 || i >= grid.size() || j < 0 || j >= grid[0].size())
        return;
    if (grid[i][j] != '1')
        return;
    grid[i][j] = '2'; // Mark '1' as visited.
    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
}
```

DPS · LeetCode · By Pattern

— 2029 —

LeetCode

```
dfs(grid, i, j + 1);
dfs(grid, i, j - 1);
}
```

C++

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        int ans = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    ans++;
                }
            }
        }
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        int ans = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    ans++;
                }
            }
        }
        return ans;
    }
};
```

DPS · LeetCode · By Pattern

— 2030 —

LeetCode

```
grid[i][j] = '0';
}
}
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (grid[i][j] == '1') {
            dfs(i, j);
            ans++;
        }
    }
}
return ans;
}
```

JAVA

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int ans = 0;
        for (int i = 0; i < grid.size(); i++) {
            for (int j = 0; j < grid[0].size(); j++) {
                if (grid[i][j] == '1') {
                    dfs(grid, i, j);
                    ans++;
                }
            }
        }
        return ans;
    }
};

private:
void dfs(vector<vector<char>>& grid, int i, int j) {
    if (i < 0 || i >= grid.size() || j < 0 || j >= grid[0].size())
        return;
    if (grid[i][j] != '1')
        return;
    grid[i][j] = '2'; // Mark '1' as visited.
    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
    dfs(grid, i, j + 1);
    dfs(grid, i, j - 1);
}
```

Java

DPS · LeetCode · By Pattern

— 2031 —

LeetCode

```
class Solution {
private:
    char*** grid;
    int m;
    int n;

    public:
    int numIslands(char*** grid) {
        m = grid[0].length;
        n = grid[0].length;
        this->grid = grid;
        int ans = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    dfs(i, j);
                    ans++;
                }
            }
        }
        return ans;
    }
};
```

```
private:
void dfs(int i, int j) {
    grid[i][j] = '0';
    int dx = {-1, 0, 1, 0, -1};
    int dy = {0, 1, 0, -1, 0};
    for (int k = 0; k < 5; k++) {
        int x = i + dx[k];
        int y = j + dy[k];
        if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == '0')
            continue;
        dfs(x, y);
    }
}
```

```
class Solution {
private:
    char*** grid;
    int m;
    int n;

    public:
    int numIslands(char*** grid) {
        m = grid[0].length;
        n = grid[0].length;
        this->grid = grid;
        int ans = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    dfs(i, j);
                    ans++;
                }
            }
        }
        return ans;
    }
};
```

DPS · LeetCode · By Pattern

— 2032 —

LeetCode

```
ans++;
}
}
return ans;
}

private:
void dfs(int i, int j) {
    grid[i][j] = '0';
    Queue<int> q = new ArrayDeque<>();
    q.offer(new int[] {i, j});
    while (!q.isEmpty()) {
        int[] p = q.poll();
        int x = p[0];
        int y = p[1];
        int dx = {-1, 0, 1, 0, -1};
        int dy = {0, 1, 0, -1, 0};
        for (int k = 0; k < 5; k++) {
            int nx = x + dx[k];
            int ny = y + dy[k];
            if (nx < 0 || nx >= m || ny < 0 || ny >= n || grid[nx][ny] == '0')
                continue;
            q.offer(new int[] {nx, ny});
            grid[nx][ny] = '0';
        }
    }
}
```

JAVA

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Solution {
    public int NumIslands(char*** grid) {
        var queue = new Queue<Tuple<int, int>>();
        var land = grid.Length;
        var row = grid[0].Length;
        var path = new int[4] { 0, 1, 0, -1 };
        var result = 0;
        for (var i = 0; i < land; i++) {
            for (var j = 0; j < row; j++) {
                if (grid[i][j] == '1') {
                    ++result;
                    grid[i][j] = '0';
                    queue.Enqueue(Tuple.Create(i, j));
                    while (queue.Any()) {
                        var position = queue.Dequeue();
                        for (var k = 0; k < 4; k++) {
                            var nx = position.Item1 + path[k];
                            var ny = position.Item2 + path[k];
                            if (nx < 0 || nx >= land || ny < 0 || ny >= row || grid[nx][ny] == '0')
                                continue;
                            queue.Enqueue(Tuple.Create(nx, ny));
                            grid[nx][ny] = '0';
                        }
                    }
                }
            }
        }
        return result;
    }
}
```

DPS · LeetCode · By Pattern

— 2033 —

LeetCode

```
var next = Tuple.Create(position.Item1 + path[k], position.Item2 + path[k]);
if (next.Item1 < 0 || next.Item1 >= land || next.Item2 < 0 || next.Item2 >= row || grid[next.Item1][next.Item2] == '0')
    continue;
queue.Enqueue(next);
}
}
return result;
}
```

TypeScript

```
function numIslands(grid: string[][]): number {
    const m = grid.length;
    const n = grid[0].length;
    let ans = 0;
    const dirs = [-1, 0, 1, 0, -1];
    const dfs = (i: number, j: number) => {
        grid[i][j] = '0';
        for (let k = 0; k < 5; k++) {
            const x = i + dirs[k];
            const y = j + dirs[k];
            if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == '0')
                continue;
            dfs(x, y);
        }
    };
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            if (grid[i][j] == '1') {
                dfs(i, j);
                ans++;
            }
        }
    }
    return ans;
}
```

TypeScript

DPS · LeetCode · By Pattern

— 2034 —

LeetCode

```
function numIslands(grid: string[][]): number {
    const n = grid.length;
    const m = grid[0].length;
    let ans = 0;

    function dfs(i: number, j: number): void {
        grid[i][j] = '0';
        const dirs = [-1, 0, 1, 0, -1];
        for (let k = 0; k < 4; k++) {
            const x = i + dirs[k];
            const y = j + dirs[k + 1];
            if (x <= 0 || x >= n || y <= 0 || y >= m || grid[x][y] === '1') {
                dfs(x, y);
            }
        }
    }

    for (let i = 0; i < n; i++) {
        for (let j = 0; j < m; j++) {
            if (grid[i][j] === '1') {
                dfs(i, j);
                ans++;
            }
        }
    }

    return ans;
}
```

Go

```
func numIslands(grid [][]byte) int {
    m, n := len(grid), len(grid[0])
    var dfs func(i, j int)
    dfs = func(i, j int) {
        grid[i][j] = '0'
        dirs := [][]int{{-1, 0, 1, 0, -1}}
        for k := 0; k < 4; k++ {
            x, y := i+dirs[k], j+dirs[k+1]
            if x <= 0 || x >= m || y <= 0 || y >= n || grid[x][y] == '1' {
                dfs(x, y)
            }
        }
    }
    ans := 0
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if grid[i][j] == '1' {
                dfs(i, j)
                ans++
            }
        }
    }
    return ans
}
```

Go

DPS · LeetCode · By Pattern

— 2035 —

LeetCode

```
        if grid[i][j] == '1' {
            dfs(grid, i, j);
            ans++;
        }
    }
}
```

[*] DFS — Pattern Solution (matched from community answers)

Python

```
# DFS
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            if not (0 <= i < n and 0 <= j < m and grid[i][j] == '1'):
                return
            grid[i][j] = '0'
            for dx, dy in dirs:
                x, y = i + dx, j + dy
                dfs(x, y)

        ans = 0
        dirs = [-1, 0, 1, 0, -1]
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    dfs(i, j)
                    ans += 1

        return ans

#####

# bfs
from collections import deque
```

DPS · LeetCode · By Pattern

— 2037 —

LeetCode

Course Schedule

LeetCode · Simplify · WalkCC · LeetCode · Editorial

DPS · BFS · Graph · Topological Sort

Units Grind 189

Problem:

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

• For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `true`

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`

Output: `false`

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- All the pairs `prerequisites[i]` are unique.

>> Brute Force (DFS) Interview Prep

Python

DPS · LeetCode · By Pattern

— 2039 —

LeetCode

```
func numIslands(grid [][]byte) int {
    m, n := len(grid), len(grid[0])
    var dfs func(i, j int)
    dfs = func(i, j int) {
        grid[i][j] = '0'
        dirs := [][]int{{-1, 0, 1, 0, -1}}
        for k := 0; k < 4; k++ {
            x, y := i+dirs[k], j+dirs[k+1]
            if x <= 0 || x >= m || y <= 0 || y >= n || grid[x][y] == '1' {
                dfs(x, y)
            }
        }
    }
    ans := 0
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if grid[i][j] == '1' {
                dfs(i, j)
                ans++
            }
        }
    }
    return ans
}
```

Rust

Rust

```
const DIRS: [(i32, i32)] = [(-1, 0), (1, 0), (0, -1)];

impl Solution {
    pub fn num_islands(grid: Vec<Vec<char>>) -> i32 {
        let dfs = |grid: &mut Vec<Vec<char>>, i: usize, j: usize| {
            grid[i][j] = '0';
            for k in 0..4 {
                let x = i as i32 + DIRS[k].0;
                let y = j as i32 + DIRS[k].1;
                if x >= 0 &&
                    (x as usize) < grid.len() &&
                    y >= 0 &&
                    (y as usize) < grid[0].len() &&
                    grid[x as usize][y as usize] == '1' {
                    dfs(grid, x as usize, y as usize);
                }
            }
        };

        let mut grid = grid;
        let mut ans = 0;
        for i in 0..grid.len() {
            for j in 0..grid[0].len() {
```

DPS · LeetCode · By Pattern

— 2036 —

LeetCode

```
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

        m, n = len(grid), len(grid[0])
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
        islands = 0

        for i in range(m):
            for j in range(n):
                if grid[i][j] == "1":
                    islands += 1
                    q = deque([(i, j)])
                    # no need to reset q, q already drained from previous bfs
                    while q:
                        x, y = q.popleft()
                        for dx, dy in directions:
                            nx, ny = x + dx, y + dy
                            if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == "1":
                                q.append((nx, ny))
                                grid[nx][ny] = "0" # mark as visited

        return islands

#####
```

```
# union find
# visited by ref in https://leetcode.ca/2016-09-20-305-Number-of-Islands-II/
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def check(i, j):
            return 0 <= i < m and 0 <= j < n and grid[i][j] == "1" # Check for "1" instead of 1

        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        m, n = len(grid), len(grid[0])
        p = list(range(m * n))
        cur = 0 # Initialize cur as 0 instead of n
        for i in range(m):
            for j in range(n):
                if grid[i][j] == "1":
                    cur += 1 # Increment cur when encountering "1"
                    for x, y in [(i-1, 0), (i, 1), (i, 0), (0, -1), (0, 1)]:
                        if check(i + x, j + y) and find(i + x) != find(i + x) + n + j + y:
                            p[find(i + x) + n + j + y] = find(i + x) + n + j + y
                    cur -= 1

        return cur

#####
```

#207

MEDIUM

DPS · LeetCode · By Pattern

— 2038 —

LeetCode

```
# Brute Force Approach
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        # Brute Force (DFS) - DFS cycle detection on adjacency list
        from collections import defaultdict
        graph = defaultdict(list)
        for a, b in prerequisites:
            graph[a].append(b)
        # Topological Sort
        state = [0] * numCourses
        def has_cycle(node):
            if state[node] == 1: return True
            if state[node] == 2: return False
            state[node] = 1
            for nei in graph[node]:
                if has_cycle(nei): return True
            state[node] = 2
            return False
        return not any(has_cycle(i) for i in range(numCourses))
```

Python

Python

```
from collections import deque

def canFinish(numCourses: int, prerequisites: List[List[int]]) -> bool:
    # Create a list to hold the number of prerequisites (indegree) for each course.
    indegree = [0] * numCourses
    # Create an adjacency list for each course.
    graph = [[] for i in range(numCourses)]

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1

    # Initialize a queue with courses that have no prerequisites.
    queue = deque([i for i in range(numCourses) if indegree[i] == 0])

    # Remove the courses that can be completed.
    visited = 0

    while queue:
        course = queue.popleft() # Remove course with no remaining prerequisites.
        visited += 1 # Increment count of completed courses.
        # For each neighbor course, reduce its indegree.
        for neighbor in graph[course]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0: # Remove dependency.
```

DPS · LeetCode · By Pattern

— 2040 —

LeetCode


```
public class Solution {
    public bool CanFinish(int numCourses, int[][] prerequisites) {
        var q = new List<int>(numCourses);
        for (int i = 0; i < numCourses; ++i) {
            q[i] = new List<int>();
        }
        var indeg = new int[numCourses];
        foreach (var p in prerequisites) {
            int a = p[0], b = p[1];
            q[b].Add(a);
            ++indeg[a];
        }
        var q = new Queue<int>();
        for (int i = 0; i < numCourses; ++i) {
            if (indeg[i] == 0) {
                q.Enqueue(i);
            }
        }
        var cnt = 0;
        while (q.Count > 0) {
            int i = q.Dequeue();
            ++cnt;
            foreach (int j in q[i]) {
                if (--indeg[j] == 0) {
                    q.Enqueue(j);
                }
            }
        }
        return cnt == numCourses;
    }
}
```

TypeScript

TypeScript

```
function canFinish(numCourses: number, prerequisites: number[][]): boolean {
    const q: number[][] = Array.from({length: numCourses}, () => []);
    const indeg: number[] = Array(numCourses).fill(0);
    for (const [a, b] of prerequisites) {
        q[b].push(a);
        indeg[a]++;
    }
    const q: number[] = [];
    for (let i = 0; i < numCourses; ++i) {
        if (indeg[i] === 0) {
            q.push(i);
        }
    }
    for (const i of q) {
        --numCourses;
        for (const j of q[i]) {
            if (--indeg[j] === 0) {
                q.push(j);
            }
        }
    }
    return numCourses === 0;
}
```

TypeScript

```
function canFinish(numCourses: number, prerequisites: number[][]): boolean {
    const q: number[][] = new Array(numCourses).fill(0).map(() => []);
    const indeg: number[] = new Array(numCourses).fill(0);
    for (const [a, b] of prerequisites) {
        q[b].push(a);
        indeg[a]++;
    }
    const q: number[] = [];
    for (let i = 0; i < numCourses; ++i) {
        if (indeg[i] === 0) {
            q.push(i);
        }
    }
    let cnt = 0;
    while (q.length) {
        const i = q.shift();
        cnt++;
        for (const j of q[i]) {
            if (--indeg[j] === 0) {
                q.push(j);
            }
        }
    }
    return cnt === numCourses;
}
```

Go

Go

```
func canFinish(numCourses int, prerequisites [][]int) bool {
    q := make([][]int, numCourses)
    indeg := make([]int, numCourses)
    for _, p := range prerequisites {
        a, b := p[0], p[1]
        q[b] = append(q[b], a)
        indeg[a]++
    }
    q := []int{}
    for i, x := range indeg {
        if x == 0 {
            q = append(q, i)
        }
    }
    for len(q) > 0 {
        i := q[0]
        q = q[1:]
        numCourses--
        for _, j := range q[i] {
            indeg[j]--
            if indeg[j] == 0 {
                q = append(q, j)
            }
        }
    }
    return numCourses == 0
}
```

Go

```
func canFinish(numCourses int, prerequisites [][]int) bool {
    q := make([][]int, numCourses)
    indeg := make([]int, numCourses)
    for _, p := range prerequisites {
        a, b := p[0], p[1]
        q[b] = append(q[b], a)
        indeg[a]++
    }
    q := []int{}
    for i, x := range indeg {
        if x == 0 {
            q = append(q, i)
        }
    }
    cnt := 0
    for len(q) > 0 {
        i := q[0]
        q = q[1:]
        cnt++
        for _, j := range q[i] {
            indeg[j]--
            if indeg[j] == 0 {
                q = append(q, j)
            }
        }
    }
}
```

```
}
return cnt == numCourses
}
```

Rust

```
use std::collections::VecDeque;

impl Solution {
    pub fn can_finish(num_course: i32, prerequisites: Vec<Vec<i32>>) -> bool {
        let num_course = num_course as usize;
        // The graph representation
        let mut graph: Vec<Vec<i32>> = vec![vec![]; num_course];
        // Build the in-degrees for each node
        let mut in_degree_vec: Vec<i32> = vec![0; num_course];
        let mut q: VecDeque<usize> = VecDeque::new();
        let mut count = 0;

        // Initialize the graph & in-degree vector
        for p in &prerequisites {
            let (from, to) = (&p[0], &p[1]);
            graph[from as usize].push(to);
            in_degree_vec[to as usize] += 1;
        }

        // Choose the first batch of nodes with in-degree 0
        for i in 0..num_course {
            if in_degree_vec[i] == 0 {
                q.push_back(i);
            }
        }

        // Build the traverse & update through the graph
        while !q.is_empty() {
            // Get the current node index
            let index = q.front().clone();
            // This course can be finished
            count += 1;
            q.pop_front();
            for i in &graph[index] {
                // Update the in-degree for the current node
                in_degree_vec[*i as usize] -= 1;
                // See if it can be removed
                if in_degree_vec[*i as usize] == 0 {
                    q.push_back(*i as usize);
                }
            }
        }
        count == num_course
    }
}
```

[*] DPS - Pattern Solution (stored optimal)

Python

```
from collections import deque

def canFinish(numCourses, prerequisites):
    # Create a list to store the number of prerequisites (indegree) for each course.
    indegree = [0] * numCourses
    # Create an adjacency list for each course.
    graph = [[] for i in range(numCourses)]

    # Build the graph and update indegree for each course.
    for course, prereq in prerequisites:
        graph[course].append(prereq)
        indegree[prereq] += 1

    # Initialize a queue with courses that have no prerequisites.
    queue = deque([i for i in range(numCourses) if indegree[i] == 0])

    # Counter for courses that can be completed.
    visited = 0

    while queue:
        course = queue.popleft() # Remove course with no remaining prerequisites.
        visited += 1 # Increment count of completed courses.
        # For each neighbor course, reduce its indegree.
        for neighbor in graph[course]:
            indegree[neighbor] -= 1 # Remove dependency.

        # If no more prerequisites, add to queue.
        if indegree[neighbor] == 0:
            queue.append(neighbor)

    # If we processed all courses, it's possible to finish.
    return visited == numCourses

# Example usage:
print(canFinish(2, [[1,0]])) # Expected output: True
print(canFinish(2, [[1,0],[0,1]])) # Expected output: False
```

C++

```
// C++ solution using DFS (Backtracking Algorithm)
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Solution {
public:
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        // Create an adjacency list for each course.
        vector<int> indegree(numCourses, 0);
        // adjacency list for the graph.
        vector<vector<int>> graph(numCourses);

        // Build the graph.
        for (auto& pre : prerequisites) {
            int course = pre[0], prereq = pre[1];
            graph[prereq].push_back(course);
            indegree[course]++;
        }

        // Queue for courses with no prerequisites.
        queue<int> q;
        for (int i = 0; i < numCourses; i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }

        int visited = 0;
        // Process courses.
        while (!q.empty()) {
            int course = q.front();
            q.pop();
            visited++;
            // Decrease indegree for each neighboring course.
            for (int neighbor : graph[course]) {
                indegree[neighbor]--;
                // If no prerequisites left, add neighbor to queue.
                if (indegree[neighbor] == 0) {
                    q.push(neighbor);
                }
            }
        }

        // If all courses are visited, return true.
        return visited == numCourses;
    }
};
```

```
// Example usage:
int main() {
    Solution sol;
    vector<vector<int>> prerequisites = {{1,0},
    vector<vector<int>> prerequisites = {{1,0}, {0,1}};
    cout << hasCycle(sol.canFinish(2, prerequisites)) << endl; // Expected output: true
    cout << hasCycle(sol.canFinish(2, prerequisites)) << endl; // Expected output: false
    return 0;
}
```

#210

MEDIUM

Course Schedule II

LeetCode · Topical · Medium · LeetCode · LeetCode · Editorial

DFS · BFS · Graph · Topological Sort

Like Grind 189

Problem:

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a, b] indicates that you must take course b first if you want to take course a.

• For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1. Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

Input: numCourses = 2, prerequisites = [[1,0]]

Output: [0,1]

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].

Example 2:

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

Example 3:

Input: numCourses = 1, prerequisites = []

Output: [0]

Constraints:

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= numCourses * (numCourses - 1)
- prerequisites[i].length == 2
- 0 <= a, b < numCourses

DFS · LeetCode · By Pattern

— 2053 —

LeetCode

- a != b
- All the pairs [a, b] are distinct.

>> Brute Force (DFS) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def findOrder(self, numCourses, prerequisites):
        # Build the graph and the in-degree array
        from collections import defaultdict
        graph = defaultdict(list)
        for a, b in prerequisites:
            graph[b].append(a)
        state = [0] * numCourses
        result = []
        def dfs(node):
            if state[node] == 1: return False # cycle
            if state[node] == 2: return True
            state[node] = 1
            for nei in graph[node]:
                if not dfs(nei): return False
            state[node] = 2
            result.append(node)
            return True
        for i in range(numCourses):
            if not dfs(i): return []
        return result[::-1]
```

Python

```
# Python solution using DFS (Node's Algorithm)
from collections import deque

def findOrder(numCourses, prerequisites):
    # Build the graph and the in-degree array
    in_degree = [0] * numCourses
    graph = [[] for i in range(numCourses)]
    for course, pre in prerequisites:
        graph[pre].append(course)
        in_degree[course] += 1

    # Initialize the queue with all courses having zero in-degree
    queue = deque([i for i in range(numCourses) if in_degree[i] == 0])
    order = []

    # Process courses
    while queue:
        course = queue.popleft()
        order.append(course)
        for neighbor in graph[course]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # If the ordering includes all courses, return it; otherwise, return an empty list
    return order if len(order) == numCourses else []
```

DFS · LeetCode · By Pattern

— 2054 —

LeetCode

```
return order if len(order) == numCourses else []

# Example usage:
# print(findOrder(4, [[1,0],[2,0],[3,1],[3,2]]))
```

Python

from enum import Enum

class State(Enum):

INIT = 0

VISITING = 1

VISITED = 2

class Solution:

def findOrder

self,

numCourses: int,

prerequisites: List[List[int]],

) -> List[int]:

ans = []

graph = [[] for i in range(numCourses)]

state = [State.INIT] * numCourses

for v, u in prerequisites:

graph[u].append(v)

def hasCycle(a: int) -> bool:

if state[a] == State.VISITING:

return True

if state[a] == State.VISITED:

return False

state[a] = State.VISITING

if any(hasCycle(a) for a in graph[a]):

return True

state[a] = State.VISITED

ans.append(a)

return False

if any(hasCycle(a) for i in range(numCourses)):

return []

return ans[::-1]

Python

DFS · LeetCode · By Pattern

— 2055 —

LeetCode

```
class Solution:
    def findOrder(
        self,
        numCourses: int,
        prerequisites: List[List[int]]
    ) -> List[int]:
        ans = []
        graph = [[] for i in range(numCourses)]
        indegrees = [0] * numCourses
        q = collections.deque()

        # Build the graph
        for v, u in prerequisites:
            graph[u].append(v)
            indegrees[u] += 1

        # Topological sorting
        q = collections.deque([i for i, d in enumerate(indegrees) if d == 0])

        while q:
            u = q.popleft()
            ans.append(u)
            for v in graph[u]:
                indegrees[v] -= 1
                if indegrees[v] == 0:
                    q.append(v)

        return ans if len(ans) == numCourses else []
```

Python

```
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        # Build the graph and the in-degree array
        in_deg = [0] * numCourses
        for a, b in prerequisites:
            graph[b].append(a)
            in_deg[a] += 1

        ans = []
        q = deque([i for i, x in enumerate(in_deg) if x == 0])

        while q:
            i = q.popleft()
            ans.append(i)
            for j in graph[i]:
                in_deg[j] -= 1
                if in_deg[j] == 0:
                    q.append(j)

        return ans if len(ans) == numCourses else []
```

Python

```
class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        # Build the graph and the in-degree array
        in_deg = [0] * numCourses
        for a, b in prerequisites:
            graph[b].append(a)
            in_deg[a] += 1

        ans = []
        q = deque([i for i, x in enumerate(in_deg) if x == 0])

        while q:
            i = q.popleft()
            ans.append(i)
            # Remove from only one path
            # as in question "You may assume that there are no duplicate edges in the input prerequisites."
            for j in graph[i]:
                in_deg[j] -= 1
                if in_deg[j] == 0:
                    q.append(j)

        return ans if len(ans) == numCourses else []
```

C++

C++

class Solution {

public:

vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {

vector<int> ans;

vector<vector<int>> graph(numCourses);

vector<int> inDegree(numCourses);

queue<int> q;

// Build the graph

for (const vector<int>& prerequisite : prerequisites) {

const int u = prerequisite[1];

const int v = prerequisite[0];

graph[u].push_back(v);

inDegree[v]++;

}

// Perform topological sorting

for (int i = 0; i < numCourses; ++i)

if (inDegree[i] == 0)

q.push(i);

while (!q.empty()) {

const int u = q.front();

q.pop();

ans.push_back(u);

for (const int v : graph[u])

if (--inDegree[v] == 0)

q.push(v);

}

return ans.size() == numCourses ? ans : vector<int>();

}

C++

DFS · LeetCode · By Pattern

— 2057 —

LeetCode

```
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> graph(numCourses);
        vector<int> inDegree(numCourses);
        for (auto& p : prerequisites) {
            int a = p[0], b = p[1];
            graph[b].push_back(a);
            inDegree[a]++;
        }

        queue<int> q;
        for (int i = 0; i < numCourses; ++i) {
            if (inDegree[i] == 0) {
                q.push(i);
            }
        }

        vector<int> ans;
        while (!q.empty()) {
            int i = q.front();
            q.pop();
            ans.push_back(i);
            for (int j : graph[i]) {
                if (--inDegree[j] == 0) {
                    q.push(j);
                }
            }
        }

        return ans.size() == numCourses ? ans : vector<int>();
    }
};
```

C++

```
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> graph(numCourses);
        vector<int> inDegree(numCourses);
        for (auto& p : prerequisites) {
            int a = p[0], b = p[1];
            graph[b].push_back(a);
            inDegree[a]++;
        }

        queue<int> q;
        for (int i = 0; i < numCourses; ++i) {
            if (inDegree[i] == 0) {
                q.push(i);
            }
        }

        vector<int> ans;
        while (!q.empty()) {
            int i = q.front();
            q.pop();
            ans.push_back(i);
            for (int j : graph[i]) {
                if (--inDegree[j] == 0) {
                    q.push(j);
                }
            }
        }

        return ans.size() == numCourses ? ans : vector<int>();
    }
};
```

DFS · LeetCode · By Pattern

— 2058 —

LeetCode

```

    }
    return ans.size() == numCourses ? ans : vector<int>{};
}
}

```

Java

```

class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<Integer> q = new List<numCourses>;
        Arrays.sort(q, b -> new Array<int>{0});
        int[] indeg = new int[numCourses];
        for (var p : prerequisites) {
            int a = p[0], b = p[1];
            q[b].add(a);
            ++indeg[a];
        }
        Deque<Integer> q = new ArrayDeque<int>();
        for (int i = 0; i < numCourses; ++i) {
            if (indeg[i] == 0) {
                q.offer(i);
            }
        }
        int[] ans = new int[numCourses];
        int cnt = 0;
        while (!q.isEmpty()) {
            int i = q.poll();
            ans[cnt++] = i;
            for (int j : q[i]) {
                if (--indeg[j] == 0) {
                    q.offer(j);
                }
            }
        }
        return cnt == numCourses ? ans : new int[0];
    }
}

```

Java

```

public class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        var q = new List<int>(numCourses);
        for (int i = 0; i < numCourses; ++i) {
            q[i] = new List<int>();
        }
        var indeg = new int[numCourses];
        foreach (var p in prerequisites) {
            int a = p[0], b = p[1];
            q[b].Add(a);
            ++indeg[a];
        }
        var q = new Queue<int>();
        for (int i = 0; i < numCourses; ++i) {
            if (indeg[i] == 0) {
                q.Enqueue(i);
            }
        }
        var ans = new List<numCourses>();
        var cnt = 0;
        while (q.Count > 0) {
            int i = q.Dequeue();
            ans.Add(i);
            foreach (int j in q[i]) {
                if (--indeg[j] == 0) {
                    q.Enqueue(j);
                }
            }
        }
        return cnt == numCourses ? ans : new int[0];
    }
}

```

TypeScript

TypeScript

```

function findOrder(numCourses: number, prerequisites: number[][]): number[] {
    const g: number[][] = Array.from<length: numCourses>(), () => [];
    const indeg: number[] = new Array(numCourses).fill(0);
    for (const [a, b] of prerequisites) {
        g[b].push(a);
        indeg[a]++;
    }
    const q: number[] = [];
    for (let i = 0; i < numCourses; ++i) {
        if (indeg[i] === 0) {
            q.push(i);
        }
    }
    const ans: number[] = [];
    while (q.length) {
        const i = q.shift();
        ans.push(i);
        for (const j of g[i]) {
            if (--indeg[j] === 0) {
                q.push(j);
            }
        }
    }
    return ans.length === numCourses ? ans : [];
}

```

TypeScript

```

function findOrder(numCourses: number, prerequisites: number[][]): number[] {
    const g: number[][] = Array.from<length: numCourses>(), () => [];
    const indeg: number[] = new Array(numCourses).fill(0);
    for (const [a, b] of prerequisites) {
        g[b].push(a);
        indeg[a]++;
    }
    const q: number[] = [];
    for (let i = 0; i < numCourses; ++i) {
        if (indeg[i] === 0) {
            q.push(i);
        }
    }
    const ans: number[] = [];
    while (q.length) {
        const i = q.shift();
        ans.push(i);
        for (const j of g[i]) {
            if (--indeg[j] === 0) {
                q.push(j);
            }
        }
    }
    return ans.length === numCourses ? ans : [];
}

```

Go

Go

```

func findOrder(numCourses int, prerequisites [][]int) []int {
    g := make([][]int, numCourses)
    indeg := make([]int, numCourses)
    for _, p := range prerequisites {
        a, b := p[0], p[1]
        g[b] = append(g[b], a)
        indeg[a]++
    }
    q := []int{}
    for i, x := range indeg {
        if x == 0 {
            q = append(q, i)
        }
    }
    ans := []int{}
    for len(q) > 0 {
        i := q[0]
        q = q[1:]
        ans = append(ans, i)
        for _, j := range g[i] {
            indeg[j]--
            if indeg[j] == 0 {
                q = append(q, j)
            }
        }
    }
    if len(ans) == numCourses {
        return ans
    }
    return []int{}
}

```

Go

```

func findOrder(numCourses int, prerequisites [][]int) []int {
    g := make([][]int, numCourses)
    indeg := make([]int, numCourses)
    for _, p := range prerequisites {
        a, b := p[0], p[1]
        g[b] = append(g[b], a)
        indeg[a]++
    }
    q := []int{}
    for i, x := range indeg {
        if x == 0 {
            q = append(q, i)
        }
    }
    ans := []int{}
    for len(q) > 0 {
        i := q[0]
        q = q[1:]
        ans = append(ans, i)
        for _, j := range g[i] {
            indeg[j]--
            if indeg[j] == 0 {
                q = append(q, j)
            }
        }
    }
    if len(ans) == numCourses {
        return ans
    }
    return []int{}
}

```

Rust

Rust

```

impl Solution {
    pub fn find_order(num_courses: i32, prerequisites: Vec<Vec<i32>>) -> Vec<i32> {
        let n = num_courses as usize;
        let mut adjacency = vec![vec![]; n];
        let mut entry = vec![0; n];
        // build
        for iter in prerequisites.iter() {
            let (a, b) = iter[0], iter[1];
            adjacency[b as usize].push(a);
            entry[a as usize] += 1;
        }
        // bfs
        let mut deque = std::collections::VecDeque::new();
        for index in 0..n {
            if entry[index] == 0 {
                deque.push_back(index);
            }
        }
        let mut result = vec![];
        // bfs
        while !deque.is_empty() {
            let head = deque.pop_front().unwrap();
            result.push(head as i32);
            // remove entry of entry
            for dest_entry in adjacency[head].iter() {
                entry[*dest_entry as usize] -= 1;
                if entry[*dest_entry as usize] == 0 {
                    deque.push_back(*dest_entry as usize);
                }
            }
        }
        if result.len() == n {
            result
        } else {
            vec![]
        }
    }
}

```

[1] DFS — Pattern Solution (stored optimal)

Python

```
# Python solution using DFS (Kahn's Algorithm)
from collections import deque

def findOrder(numCourses, prerequisites):
    # Build the graph and the in-degree array
    in_degree = [0] * numCourses
    graph = [[] for i in range(numCourses)]
    for course, pre in prerequisites:
        graph[pre].append(course)
        in_degree[course] += 1

    # Initialize the queue with all courses having zero in-degree
    queue = deque([i for i in range(numCourses) if in_degree[i] == 0])
    order = []

    # Process courses
    while queue:
        course = queue.popleft()
        order.append(course)
        for neighbor in graph[course]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # If the ordering includes all courses, return it; otherwise, return an empty list
    return order if len(order) == numCourses else []

# Example usage:
# print(findOrder(4, [[1,0],[2,0],[3,1],[3,2]]))
```

```
C++
// C++ solution using DFS (Kahn's Algorithm)
#include <vector>
#include <queue>
using namespace std;

class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        // Build graph as an adjacency list and a vector for in-degree counts
        vector<vector<int>> graph(numCourses);
        vector<int> inDegree(numCourses, 0);
        for (auto& pre : prerequisites) {
            int course = pre[1], preReq = pre[0];
            graph[preReq].push_back(course);
            inDegree[course]++;
        }

        // Queue for courses with zero in-degree
        queue<int> zeroInDegree;
        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0)
                zeroInDegree.push(i);
        }

        vector<int> order;
    }
};
```

```
// Process courses
while (!zeroInDegree.empty()) {
    int course = zeroInDegree.front();
    zeroInDegree.pop();
    order.push_back(course);
    for (int neighbor : graph[course]) {
        if (--inDegree[neighbor] == 0) {
            zeroInDegree.push(neighbor);
        }
    }
}

// Return order if all courses are processed; otherwise, return empty vector
return order.size() == numCourses ? order : vector<int>();
}
```

#261

MEDIUM

Graph Valid Tree

LeetCode · SimplyLeet · WalkCC · LeetCode · Editorial

DFS · BFS · Union Find · Graph

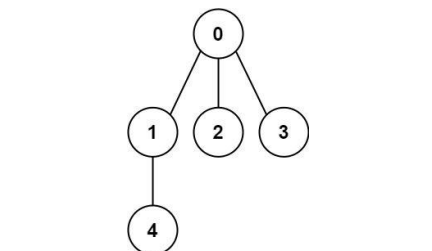
Lint-Grind ID: Premium 50

Problem:

You have a graph of n nodes labeled from 0 to $n - 1$. You are given an integer n and a list of edges where $edges[i] = [a_i, b_i]$ indicates that there is an undirected edge between nodes a_i and b_i in the graph.

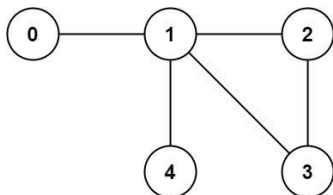
Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example 1:



Input: $n = 5$, $edges = [[0,1],[0,2],[0,3],[1,4]]$
Output: true

Example 2:



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[1,4],[3,4]]$
Output: false

Constraints:

- $1 \leq n \leq 2000$
- $0 \leq edges.length \leq 5000$
- $edges[i].length = 2$

- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- There are no self-loops or repeated edges.

>> Brute Force [DFS] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        # Create parent -> DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for ab in graph.get(node, []):
                if ab not in visited:
                    dfs(ab)
            for node in range(n):
                if node not in visited:
                    dfs(node); count += 1
            return count

        You have to consider that accepts current node and its parent node
        def dfs(node, parent):
            # Add the current node to visited
            visited.add(node)
            # Traverse all the neighboring nodes
            for neighbor in graph[node]:
                # Skip the neighbor if it is the parent to avoid trivial cycle detection
                if neighbor == parent:
                    continue
                # If neighbor is already visited, a cycle is detected
```

```
Python
# Python solution with DFS
def validTree(n, edges):
    # If the number of edges is not exactly n - 1, it cannot be a tree
    if len(edges) != n - 1:
        return False

    # Build the graph as an adjacency list
    graph = [[] for i in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    visited = set()

    # DFS helper function that accepts current node and its parent node
    def dfs(node, parent):
        # Add the current node to visited
        visited.add(node)
        # Traverse all the neighboring nodes
        for neighbor in graph[node]:
            # Skip the neighbor if it is the parent to avoid trivial cycle detection
            if neighbor == parent:
                continue
            # If neighbor is already visited, a cycle is detected
```

```
if neighbor in visited:
    return False
# Return False if cycle is found in recursion, propagate False upward
if not dfs(neighbor, node):
    return False
return True

# Start DFS from node 0, with parent set to -1 (non-existent)
if not dfs(0, -1):
    return False

# Check if all nodes were visited (i.e., the graph is connected)
return len(visited) == n

# Example usage:
print(validTree(5, [[0, 1], [0, 2], [0, 3], [1, 4]])) # Expected output: True
print(validTree(5, [[0, 1], [1, 2], [2, 3], [1, 4], [3, 4]])) # Expected output: False

Python
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

        graph = [[] for i in range(n)]
        q = collections.deque([0])
        seen = {0}

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        while q:
            u = q.popleft()
            for v in graph[u]:
                if v not in seen:
                    q.append(v)
                    seen.add(v)

        return len(seen) == n
```

Python

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return True
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
            self.rank[j] += self.rank[i]
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
            self.rank[i] += self.rank[j]
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
            self.id[j] = j
            self.count -= 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

        uf = UnionFind(n)
        for u, v in edges:
            if not uf.unionByRank(u, v):
                return False

        return uf.count == 1

Python
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            pa, pb = find(a), find(b)
            if pa == pb:
                return False
            p[pa] = pb
            n -= 1
        return n == 1
```

Python

```

class Solution {
    def validTree(int[] n, List<List<int[]>> edges) : bool {
        def find(a):
            if p[a] != a:
                p[a] = find(p[a])
            return p[a]

        p = List<range(n)>()
        for a, b in edges:
            if find(a) == find(b):
                return False
            p[find(a)] = find(b)
            n -= 1
        return n == 1
    }
}

```

C++

C++

```

class Solution {
public:
    bool validTree(int n, vector<vector<int>>& edges) {
        if (n == 0 || edges.size() != n - 1)
            return false;

        vector<vector<int>> graph(n);
        queue<int> q{0};
        unordered_set<int> seen{0};

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            for (const int v : graph[u]) {
                if (seen.contains(v)) {
                    q.push(v);
                    seen.insert(v);
                }
            }
        }

        return seen.size() == n;
    }
};

```

C++

```

class Solution {
public:
    bool validTree(int n, vector<vector<int>>& edges) {
        vector<int> p(n);
        int n1, n2;
        function<int(int)> find = [&](int x) {
            if (p[x] != x) {
                p[x] = find(p[x]);
            }
            return p[x];
        };
        for (auto& e : edges) {
            int pa = find(e[0]), pb = find(e[1]);
            if (pa == pb) {
                return false;
            }
            p[pa] = pb;
            n--;
        }
        return n == 1;
    }
};

```

C++

```

class Solution {
public:
    vector<int> p;

```

```

    bool validTree(int n, vector<vector<int>>& edges) {
        p.resize(n);
        for (int i = 0; i < n; ++i) p[i] = i;
        for (auto& e : edges) {
            int a = e[0], b = e[1];
            if (find(a) == find(b)) return 0;
            p[find(a)] = find(b);
            n--;
        }
        return n == 1;
    }

    int find(int x) {
        if (p[x] != x) p[x] = find(p[x]);
        return p[x];
    }
};

```

C++

```

class Solution {
public:
    vector<int> p;

    bool validTree(int n, vector<vector<int>>& edges) {
        p.resize(n);
        for (int i = 0; i < n; ++i) p[i] = i;
        for (auto& e : edges) {
            int a = e[0], b = e[1];
            if (find(a) == find(b)) return 0;
            p[find(a)] = find(b);
            n--;
        }
        return n == 1;
    }

    int find(int x) {
        if (p[x] != x) p[x] = find(p[x]);
        return p[x];
    }
};

```

Java

Java

```

class Solution {
    public boolean validTree(int n, List<List<Integer>> edges) {
        if (n == 0 || edges.size() != n - 1)
            return false;

        List<Integer>[] graph = new List[n];
        Queue<Integer> q = new ArrayDeque<>();
        Set<Integer> seen = new HashSet<>();
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (List<Integer> edge : edges) {
            final int u = edge.get(0);
            final int v = edge.get(1);
            graph[u].add(v);
            graph[v].add(u);
        }

        while (!q.isEmpty()) {
            final int u = q.poll();
            for (final int v : graph[u]) {
                if (seen.contains(v)) {
                    q.offer(v);
                    seen.add(v);
                }
            }
        }

        return seen.size() == n;
    }
}

```

Java

```

class Solution {
    private List<Integer>[] g;
    private Set<Integer> vis = new HashSet<>();

    public boolean validTree(int n, List<List<Integer>> edges) {
        if (edges.size() != n - 1) {
            return false;
        }
        g = new List[n];
        Arrays.setAll(g, i -> new ArrayList<>());
        for (var e : edges) {
            int a = e.get(0), b = e.get(1);
            g[a].add(b);
            g[b].add(a);
        }
        dfs(0);
        return vis.size() == n;
    }

    private void dfs(int i) {
        vis.add(i);
        for (int j : g[i]) {
            if (vis.contains(j)) {
                dfs(j);
            }
        }
    }
}

```

Java

```

class Solution {
    private List<List<Integer>> g;

    public boolean validTree(int n, List<List<Integer>> edges) {
        p = new List[n];
        for (int i = 0; i < n; ++i) {
            p[i] = new List<>();
        }
        for (List<Integer> e : edges) {
            int a = e.get(0), b = e.get(1);
            if (find(a) == find(b)) {
                return false;
            }
            p[find(a)].add(b);
            p[find(b)].add(a);
            n--;
        }
        return n == 1;
    }

    private int find(int x) {
        if (p[x] != x) p[x] = find(p[x]);
        return p[x];
    }
}

```

JavaScript

JavaScript

```

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {boolean}
 */
var validTree = function(n, edges) {
    const p = Array.from({length: n}, (_, i) => i);
    const find = x => {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    };
    for (const [a, b] of edges) {
        const pa = find(a);
        const pb = find(b);
        if (pa == pb) {
            return false;
        }
        p[pa] = pb;
        n--;
    }
    return n == 1;
};

```

JavaScript

```

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {boolean}
 */
var validTree = function(n, edges) {
    let p = new Array(n);
    for (let i = 0; i < n; ++i) {
        p[i] = i;
    }
    function find(a) {
        if (p[a] != a) {
            p[a] = find(p[a]);
        }
        return p[a];
    }
    for (const [a, b] of edges) {
        if (find(a) == find(b)) {
            return false;
        }
        p[find(a)] = find(b);
        n--;
    }
    return n == 1;
};

```

Go

Go

```

func validTree(n int, edges [][]int) bool {
    p := make([]int, n)
    for i := range p {
        p[i] = i
    }
    var find func(int) int
    find = func(x int) int {
        if p[x] != x {
            p[x] = find(p[x])
        }
        return p[x]
    }
    for _, e := range edges {
        pa, pb := find(e[0]), find(e[1])
        if pa == pb {
            return false
        }
        p[pa] = pb
        n--
    }
    return n == 1
}

```

Go

```

func validTree(n int, edges [][]int) bool {
    if len(edges) != n-1 {
        return false
    }
    q := make([][]int, n)
    vis := make([]bool, n)
    for _, e := range edges {
        a, b := e[0], e[1]
        q[a] = append(q[a], b)
        q[b] = append(q[b], a)
    }
    var dfs func(int)
    dfs = func(i int) {
        vis[i] = true
        n--
        for _, j := range q[i] {
            if !vis[j] {
                dfs(j)
            }
        }
    }
    dfs(0)
    return n == 0
}

```

Go

```

func validTree(n: int, edges: List[List]) bool {
    p := make([]int, n)
    for i := range p {
        p[i] = -1
    }
    var find func(x int) int
    find = func(x int) int {
        if p[x] == -1 {
            p[x] = find(p[x])
        }
        return p[x]
    }
    for _, e := range edges {
        a, b := e[0], e[1]
        if find(a) == find(b) {
            return false
        }
        p[find(a)] = find(b)
    }
    return n == 1
}

```

[1] DFS - Pattern Solution (matched from community answers)

Python

```

# Python solution with DFS
def validTree(n, edges):
    # If the number of edges is not exactly n - 1, it cannot be a tree
    if len(edges) != n - 1:
        return False

    # Build the graph as an adjacency list
    graph = [[] for i in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    visited = set()

    # DFS helper function that accepts current node and its parent node
    def dfs(node, parent):
        # Add the current node to visited
        visited.add(node)
        # Traverse all the neighboring nodes
        for neighbor in graph[node]:
            # Check the neighbor if it is the parent to avoid trivial cycle detection
            if neighbor == parent:
                continue
            # If neighbor is already visited, a cycle is detected
    
```

DFS · LeetCode · By Pattern

— 2077 —

LeetCode

```

    if neighbor in visited:
        return False
    # Recurse down: if cycle is found in recursion, propagate False upward
    if not dfs(neighbor, node):
        return False
    return True

# Start DFS from node 0, with parent set to -1 (non-existent)
if not dfs(0, -1):
    return False

# Check if all nodes were visited (i.e., the graph is connected)
return len(visited) == n

# Example usage:
print(validTree(5, [[0, 1], [0, 2], [0, 3], [1, 4]])) # Expected output: True
print(validTree(5, [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]])) # Expected output: False

```

#310

MEDIUM

Minimum Height Trees

LeetCode · SimplifyLeetCode · WalkCC · LeetCode · Editorial

DFS · BFS · Graph · Topological Sort

Line: 61 of 109

Problem:

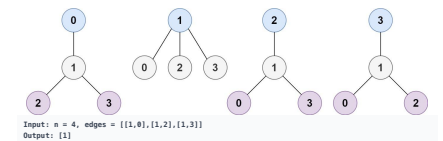
A tree is an undirected graph in which any two vertices are connected by exactly one path. In other words, any connected graph without simple cycles is a tree.

Given a tree of n nodes labelled from 0 to $n - 1$, and an array of $n - 1$ edges where $edges[i] = [a_i, b_i]$ indicates that there is an undirected edge between the two nodes a_i and b_i in the tree, you can choose any node of the tree as the root. When you select a node x as the root, the result tree has height h . Among all possible rooted trees, those with minimum height (i.e. $\min(h)$) are called minimum height trees (MHTs).

Return a list of all MHTs' root labels. You can return the answer in any order.

The height of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:



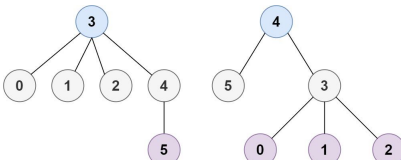
DFS · LeetCode · By Pattern

— 2078 —

LeetCode

Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.

Example 2:



Input: $n = 6$, $edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]$
Output: $[3,4]$

Constraints:

- $1 \leq n \leq 2 \cdot 10^4$
- $edges.length == n - 1$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- All the pairs (a_i, b_i) are distinct.
- The given input is guaranteed to be a tree and there will be no repeated edges.

Python

Python

DFS · LeetCode · By Pattern

— 2079 —

LeetCode

```

# Python solution with line-by-line comments
class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List]) -> List[int]:
        # Special case: if there's only one node, return it as the only MHT root.
        if n == 1:
            return [0]

        # Build the graph using an adjacency list with sets for neighbors.
        graph = [[] for _ in range(n)]
        for a, b in edges:
            graph[a].add(b)
            graph[b].add(a)

        # Initialize the first layer of leaves (nodes with one neighbor).
        leaves = [i for i in range(n) if len(graph[i]) == 1]

        # Remove leaves layer by layer until at most 2 nodes remain.
        remaining_nodes = n
        while remaining_nodes > 2:
            remaining_nodes = len(leaves)
            new_leaves = []
            # Remove current leaves.
            for leaf in leaves:
                # Since it's a leaf, there is only one neighbor.
                neighbor = graph[leaf].pop()

                # Remove the connection from the neighbor to the leaf.
                graph[neighbor].remove(leaf)
                # If the neighbor becomes a leaf, add it to the new leaves list.
                if len(graph[neighbor]) == 1:
                    new_leaves.append(neighbor)
            leaves = new_leaves

        # The remaining nodes are the roots of the MHTs.
        return leaves

```

Python

DFS · LeetCode · By Pattern

— 2080 —

LeetCode

```

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List]) -> List[int]:
        if n == 1:
            return [0]

        ans = []
        graph = collections.defaultdict(set)
        for u, v in edges:
            graph[u].add(v)
            graph[v].add(u)

        for label, children in graph.items():
            if len(children) == 1:
                ans.append(label)

        while n > 2:
            n = len(ans)
            next_leaves = []
            for leaf in ans:
                u = next(iter(graph[leaf]))
                graph[u].remove(leaf)
                if len(graph[u]) == 1:
                    next_leaves.append(u)
            ans = next_leaves

        return ans

```

Python

```

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List]) -> List[int]:
        if n == 1:
            return [0]
        degree = [0] * n
        for a, b in edges:
            degree[a] += 1
            degree[b] += 1
        q = deque()
        for i in range(n):
            if degree[i] == 1:
                q.append(i)
        while q:
            ans = []
            for _ in range(len(q)):
                a = q.popleft()
                ans.append(a)
                for b in q:
                    degree[b] -= 1
                    if degree[b] == 1:
                        q.append(b)
            return ans

```

Python

DFS · LeetCode · By Pattern

— 2081 —

LeetCode

```

from typing import List, Set
from collections import defaultdict

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List]) -> List[int]:
        if n == 1:
            return [0]

        graph = defaultdict(set)
        for a, b in edges:
            graph[a].add(b)
            graph[b].add(a)

        # Just check neighbours' size, not indegree! for each node
        leaves = [i for i in range(n) if len(graph[i]) == 1]

        while n > 2:
            n = len(leaves)
            new_leaves = []
            for leaf in leaves:
                neighbor = graph[leaf].pop()
                graph[neighbor].remove(leaf)
                if len(graph[neighbor]) == 1:
                    new_leaves.append(neighbor)
            leaves = new_leaves

        return leaves

#####
class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List]) -> List[int]:
        if n == 1:
            return [0]
        degree = [0] * n
        for a, b in edges:
            degree[a] += 1
            degree[b] += 1
        q = deque()
        for i in range(n):
            if degree[i] == 1:
                q.append(i)
        while q:
            ans = []
            for _ in range(len(q)):
                a = q.popleft()
                ans.append(a)
                for b in q:
                    degree[b] -= 1
                    if degree[b] == 1:
                        q.append(b)
            return ans

```

DFS · LeetCode · By Pattern

— 2082 —

LeetCode

```

        ans.clear();
        for (int i = 0; i < n; i++) {
            a = g.pushBack(i);
            ans.append(a);
            for (int j = 0; j < d[i]; j++) {
                if (degree[j] == 1) // Final round only 2 left (a,b), then degree[b] here will be 0, and
                    q.append(b);
            }
            return ans;
        }
    }

    C++

class Solution {
public:
    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        if (n == 1) return {0};
        if (n == 2) return {0,1};
        return {0};

        vector<int> ans;
        unordered_map<int, unordered_set<int>> graph;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].insert(v);
            graph[v].insert(u);
        }

        for (const auto& [label, children] : graph) {
            if (children.size() == 1)
                ans.push_back(label);
        }

        while (n > 2) {
            n = ans.size();
            vector<int> nextLeaves;
            for (const int leaf : ans) {
                const int u = *graph[leaf].begin();
                graph[u].erase(leaf);
                if (graph[u].size() == 1)
                    nextLeaves.push_back(u);
            }
            ans = nextLeaves;
        }

        return ans;
    }
};
C++

```

```

class Solution {
public:
    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        if (n == 1) return {0};
        vector<vector<int>> g(n);
        vector<int> degree(n);
        for (const auto& edge : edges) {
            int a = edge[0], b = edge[1];
            g[a].push_back(b);
            g[b].push_back(a);
            ++degree[a];
            ++degree[b];
        }
        queue<int> q;
        for (int i = 0; i < n; ++i)
            if (degree[i] == 1)
                q.push(i);
        vector<int> ans;
        while (!q.empty()) {
            ans.clear();
            for (int i = q.size(); i > 0; --i) {
                int u = q.front();
                q.pop();
                ans.push_back(u);
                for (int b : g[u]) {
                    if (--degree[b] == 1)
                        q.push(b);
                }
            }
            return ans;
        }
    }
};
Java
Java

```

```

class Solution {
    public List<Integer> findMinHeightTrees(int n, List<List<Integer>> edges) {
        if (n == 1) {
            return Collections.singletonList(0);
        }
        List<Integer> g = new ArrayList<>();
        Arrays.setAll(g, () -> new ArrayList<>());
        List<Integer> degree = new ArrayList<>();
        for (List<Integer> e : edges) {
            int a = e.get(0), b = e.get(1);
            g.get(a).add(b);
            g.get(b).add(a);
            degree.add(a);
            degree.add(b);
        }
        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < n; ++i) {
            if (degree.get(i) == 1) {
                q.offer(i);
            }
        }
        List<Integer> ans = new ArrayList<>();
        while (!q.isEmpty()) {
            ans.clear();
            for (int i = q.size(); i > 0; --i) {
                int a = q.poll();
                ans.add(a);
                for (List<Integer> e : g.get(a)) {
                    if (--degree.get(i) == 1) {
                        q.offer(i);
                    }
                }
            }
        }
        return ans;
    }
}
TypeScript
TypeScript

```

```

function findMinHeightTrees(n: number, edges: number[][]): number[] {
    if (n === 1) {
        return [0];
    }
    const g: number[][] = Array.from({ length: n }, () => []);
    const degree: number[] = Array(n).fill(0);
    for (const [a, b] of edges) {
        g[a].push(b);
        g[b].push(a);
        ++degree[a];
        ++degree[b];
    }
    const q: number[] = [];
    for (let i = 0; i < n; ++i) {
        if (degree[i] === 1) {
            q.push(i);
        }
    }
    const ans: number[] = [];
    while (q.length > 0) {
        ans.length = 0;
        const t: number[] = [];
        for (const a of q) {
            ans.push(a);
            for (const b of g[a]) {
                if (--degree[b] === 1) {
                    t.push(b);
                }
            }
        }
        q.splice(0, q.length, ...t);
    }
    return ans;
}
Go
Go

```

```

func findMinHeightTrees(n int, edges [][]int) []int {
    if n == 1 {
        return []int{0}
    }
    g := make([][]int, n)
    degree := make([]int, n)
    for i, j := range edges {
        a, b := j[0], j[1]
        g[a] = append(g[a], b)
        g[b] = append(g[b], a)
        degree[a]++
        degree[b]++
    }
    var q []int
    for i := 0; i < n; i++ {
        if degree[i] == 1 {
            q = append(q, i)
        }
    }
    var ans []int
    for len(q) > 0 {
        ans = []int{}
        for i := len(q); i > 0; i-- {
            a := q[i]
            q = q[:i]
        }
        ans = append(ans, a)
        for i, b := range g[a] {
            degree[b]--
            if degree[b] == 1 {
                q = append(q, b)
            }
        }
    }
    return ans
}
Python
Python

```

```

# Python solution with line-by-line comments
class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        # Special case: if there's only one node, return it as the only MHT root.
        if n == 1:
            return [0]

        # Build the graph using an adjacency list with sets for neighbors.
        graph = [set() for _ in range(n)]
        for a, b in edges:
            graph[a].add(b)
            graph[b].add(a)

        # Initialize the first layer of leaves (nodes with one neighbor).
        leaves = [i for i in range(n) if len(graph[i]) == 1]

        # Remove leaves layer by layer until at most 2 nodes remain.
        remaining_nodes = n
        while remaining_nodes > 2:
            remaining_nodes -= len(leaves)
            new_leaves = []
            # Remove current leaves.
            for leaf in leaves:
                # Since if q is leaf, there is only one neighbor.
                neighbor = graph[leaf].pop()

                # Remove the connection from the neighbor to the leaf.
                graph[neighbor].remove(leaf)
                # If the neighbor becomes a leaf, add it to the new leaves list.
                if len(graph[neighbor]) == 1:
                    new_leaves.append(neighbor)
            leaves = new_leaves

        # The remaining nodes are the roots of the MHTs.
        return leaves
C++
C++

```



```
// C++ solution with line-by-line comments
class Solution {
public:
    vector<int> findRoots(vector<int> a, vector<vector<int>>& edges) {
        // Special case: if there's only one node, return it as the only root.
        if (a == 1)
            return {0};

        // Build the graph using a vector of unordered_set for each node's neighbors.
        vector<unordered_set<int>> graph(a);
        for (auto& edge : edges) {
            int u = edge[0], v = edge[1];
            graph[u].insert(v);
            graph[v].insert(u);
        }

        // Identifying leaves: nodes with a single neighbor.
        vector<int> leaves;
        for (int i = 0; i < a; i++) {
            if (graph[i].size() == 1)
                leaves.push_back(i);
        }

        int remainingNodes = a;
        // Remove leaves iteratively until at most 2 nodes remain.
        while (remainingNodes > 2) {
            remainingNodes -= leaves.size();
            vector<int> nextLeaves;
            for (int leaf : leaves) {
                // Each leaf has only one neighbor.
                int neighbor = *graph[leaf].begin();
                // Remove the leaf from its neighbor's set.
                graph[neighbor].erase(leaf);
                // If the neighbor becomes a leaf, add it to nextLeaves.
                if (graph[neighbor].size() == 1)
                    nextLeaves.push_back(neighbor);
            }
            leaves = nextLeaves;
        }

        // The remaining nodes are the centers (not roots) of the tree.
        return leaves;
    }
};
```

#323 **MEDIUM**
Number of Connected Components in an Undirected Graph
[LeetCode](#) · [SimplyList](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)

DFS · BFS · Union Find · Graph
 Linux Cloud 100 · Premium 50

Problem:
 You have a graph of n nodes. You are given an integer n and an array edges where $edges[i] = [a_i, b_i]$ indicates that there is an edge between a_i and b_i in the graph.

DFS · LeetCodeMastery — By Pattern — 2089 — LeetCodeMastery

- $a_i \neq b_i$
- There are no repeated edges.

>> **Brute Force [DFS] Interview Prep**

```
Python
# Brute Force Approach
class Solution:
    def count_components(self, n, edges):
        # Simple DFS - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for nb in graph.get(node, []):
                if nb not in visited: dfs(nb)
            for node in range(n):
                if node not in visited:
                    dfs(node); count += 1
            return count

        # DFS to build track of visited nodes.
        visited = set()

        # Helper function to perform DFS traversal starting from a node.
        def dfs(node):
            stack = [node]
            while stack:
                current = stack.pop()
                if current not in visited:
                    visited.add(current)
                    for neighbor in graph.get(current, []):
                        if neighbor not in visited:
                            stack.append(neighbor)

        # Count connected components by initiating DFS on unvisited nodes.
        count = 0
        for i in range(n):
            if i not in visited:
                dfs(i)
                count += 1
            return count

# Example usage:
print(count_components(5, [[0,1],[1,2],[3,4]])) # Expected output: 2
```

DFS · LeetCodeMastery — By Pattern — 2091 — LeetCodeMastery

```
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        uf = UnionFind(n)

        for u, v in edges:
            uf.unionByRank(u, v)

        return uf.count

Python
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        def dfs(i: int) -> int:
            if i in vis:
                return 0
            vis.add(i)
            for j in g[i]:
                dfs(j)
            return 1

        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        return sum(dfs(i) for i in range(n))

Python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] == x:
            return self.p[x]
        self.p[x] = self.find(self.p[x])
        return self.p[x]

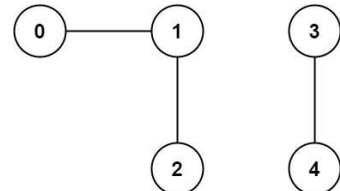
    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] < self.size[pb]:
            self.p[pa] = pb
        else:
            self.size[pb] += self.size[pa]
            self.p[pa] = pb
            self.size[pb] = self.size[pa]
        return True

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
```

DFS · LeetCodeMastery — By Pattern — 2093 — LeetCodeMastery

Return the number of connected components in the graph.

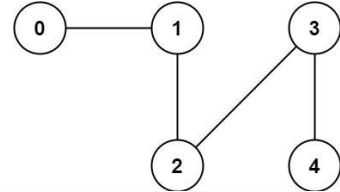
Example 1:



Input: n = 5, edges = [[0,1],[1,2],[3,4]]

Output: 2

Example 2:



Input: n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]

Output: 1

Constraints:

- $1 \leq n \leq 2000$
- $1 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i < b_i < n$

DFS · LeetCodeMastery — By Pattern — 2090 — LeetCodeMastery

```
Python
class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        seen = set()

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def bfs(node: int, seen: set[int]) -> None:
            q = collections.deque([node])
            seen.add(node)

            while q:
                u = q.popleft()
                for v in graph[u]:
                    if v not in seen:
                        q.append(v)
                        seen.add(v)

        for i in range(n):
            if i not in seen:
                bfs(i, seen)
                ans += 1

        return ans

Python
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
            self.count -= 1

    def find(self, u: int) -> int:
        if self.id[u] == u:
            return self.id[u]
        self.id[u] = self.find(self.id[u])
        return self.id[u]
```

DFS · LeetCodeMastery — By Pattern — 2092 — LeetCodeMastery

```
uf = UnionFind(n)
for a, b in edges:
    u = uf.union(a, b)
    return u

Python
...
example2: [[0, 1], [1, 2], [2, 3], [3, 4]]
for loop
[0, 1]
0 parent set to 1
1 parent set to 0
[1, 2]
1 parent set to 2
2 parent set to 1
[2, 3]
2 parent set to 3
3 parent set to 2
final:
0 parent set to 1
1 parent set to 2
2 parent set to 3
3 parent set to 2
====> so filter for "1 == find(i)"
...

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        def find(x):
            if p[x] == x:
                return p[x]
            return find(p[x])

        p = list(range(n))
        for a, b in edges:
            p[find(a)] = find(b)
            return sum(1 for i in range(n) if i == find(i))

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        # Initializing parent array for union-find
        parent = [i for i in range(n)]

        # Perform union-find on each edge
        for u, v in edges:
            self.union(u, v, parent)

        # Count the number of distinct parents (connected components)
        count = len(set(self.find(x, parent) for x in range(n)))
```

DFS · LeetCodeMastery — By Pattern — 2094 — LeetCodeMastery

```

    return count;

    def valin(self, x: int, y: int, parent: List[int]) -> bool:
        rootx = self.find(x, parent)
        rooty = self.find(y, parent)
        if rootx != rooty:
            parent[rootx] = rooty

    def find(self, x: int, parent: List[int]) -> int:
        if parent[x] == -1:
            parent[x] = self.find(parent[x], parent)
        return parent[x]

```

C++

```

class Solution {
public:
    int countComponents(int n, vector<vector<int>>& edges) {
        int ans = 0;
        vector<vector<int>> graph(n);
        unordered_set<int> seen;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            const int v = edge[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        for (int i = 0; i < n; ++i)
            if (seen.insert(i).second) {
                dfs(graph, i, seen);
                ++ans;
            }

        return ans;
    }

private:
    void dfs(const vector<vector<int>>& graph, int u, unordered_set<int>& seen) {
        for (const int v : graph[u])
            if (seen.insert(v).second)
                dfs(graph, v, seen);
    }
};

```

C++

```

class Solution {
public:
    int countComponents(int n, vector<vector<int>>& edges) {
        vector<int> g[n];
        for (const auto & edges) {
            int a = edges[0], b = edges[1];
            g[a].push_back(b);
            g[b].push_back(a);
        }
        vector<bool> vis(n);
        function<int(int)> dfs = [&](int i) {
            if (vis[i]) {
                return 0;
            }
            vis[i] = true;
            for (int j : g[i]) {
                dfs(j);
            }
            return 1;
        };
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans += dfs(i);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int countComponents(int n, vector<vector<int>>& edges) {
        vector<int> g[n];
        for (const auto & edges) {
            int a = edges[0], b = edges[1];
            g[a].push_back(b);
            g[b].push_back(a);
        }
        vector<bool> vis(n);
        function<int(int)> find = [&](int a) -> int {
            if (vis[a] == 1) {
                return 0;
            }
            for (const auto & edges) {
                int a = edges[0], b = edges[1];
                if (find(a) == find(b)) {
                    return 0;
                }
            }
            vis[a] = 1;
            for (int i = 0; i < n; ++i) {
                if (find(i) == find(a)) {
                    return 1;
                }
            }
            return 0;
        };
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans += 1 - find(i);
        }
        return ans;
    }
};

```

Java

```

class Solution {
public:
    int countComponents(int n, int** edges) {
        int ans = 0;
        List<Integer>[] graph = new List[n];
        Set<Integer> seen = new HashSet<>();
        Arrays.setAll(graph, i -> new ArrayList<>());

        for (int[] edge : edges) {
            final int u = edge[0];
            final int v = edge[1];
            graph[u].add(v);
            graph[v].add(u);
        }

        for (int i = 0; i < n; ++i)
            if (seen.add(i)) {
                dfs(graph, i, seen);
                ++ans;
            }

        private void dfs(List<Integer>[] graph, int u, Set<Integer> seen) {
            for (final int v : graph[u])
                if (!seen.contains(v))
                    dfs(graph, v, seen);
        }
    }
}

```

Java

```

class Solution {
    private List<Integer>[] g;
    private boolean[] vis;

    public int countComponents(int n, int[][] edges) {
        g = new List[n];
        vis = new boolean[n];
        Arrays.setAll(g, i -> new ArrayList<>());
        for (int[] e : edges) {
            int a = e[0], b = e[1];
            g[a].add(b);
            g[b].add(a);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans += dfs(i);
        }
        return ans;
    }

    private int dfs(int i) {
        if (vis[i]) {
            return 0;
        }
        vis[i] = true;
        for (int j : g[i]) {
            dfs(j);
        }
        return 1;
    }
}

```

```

        for (int j : g[i]) {
            dfs(j);
        }
        return 1;
    }
}

```

Java

```

class Solution {
    public int countComponents(int n, int[][] edges) {
        List<Integer>[] g = new List[n];
        Arrays.setAll(g, i -> new ArrayList<>());
        for (int[] e : edges) {
            int a = e[0], b = e[1];
            g[a].add(b);
            g[b].add(a);
        }
        int ans = 0;
        boolean[] vis = new boolean[n];
        for (int i = 0; i < n; ++i) {
            if (vis[i]) {
                continue;
            }
            vis[i] = true;
            ++ans;
            Queue<Integer> q = new ArrayDeque<>();
            q.offer(i);
            while (!q.isEmpty()) {
                int u = q.poll();
                for (int v : g[u]) {
                    if (!vis[v]) {
                        vis[v] = true;
                        q.offer(v);
                    }
                }
            }
        }
        return ans;
    }
}

```

Java

```

class Solution {
    private int[] p;

    public int countComponents(int n, int[][] edges) {
        p = new int[n];
        for (int i = 0; i < n; ++i) {
            p[i] = i;
        }
        for (int[] e : edges) {
            int a = e[0], b = e[1];
            p[find(a)] = find(b);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            if (i == find(i)) {
                ++ans;
            }
        }
        return ans;
    }

    private int find(int x) {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
}

```

JavaScript

```

class Solution {
    private int[] p;

    public int countComponents(int n, int[][] edges) {
        p = new int[n];
        for (int i = 0; i < n; ++i) {
            p[i] = i;
        }
        for (int[] e : edges) {
            int a = e[0], b = e[1];
            p[find(a)] = find(b);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            if (i == find(i)) {
                ++ans;
            }
        }
        return ans;
    }

    private int find(int x) {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
}

```

JavaScript

```

class Solution {
    private int[] p;

    public int countComponents(int n, int[][] edges) {
        p = new int[n];
        for (int i = 0; i < n; ++i) {
            p[i] = i;
        }
        for (int[] e : edges) {
            int a = e[0], b = e[1];
            p[find(a)] = find(b);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            if (i == find(i)) {
                ++ans;
            }
        }
        return ans;
    }

    private int find(int x) {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
}

```

JavaScript

```

class Solution {
    private int[] p;

    public int countComponents(int n, int[][] edges) {
        p = new int[n];
        for (int i = 0; i < n; ++i) {
            p[i] = i;
        }
        for (int[] e : edges) {
            int a = e[0], b = e[1];
            p[find(a)] = find(b);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            if (i == find(i)) {
                ++ans;
            }
        }
        return ans;
    }

    private int find(int x) {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
}

```

JavaScript

```

class Solution {
    private int[] p;

    public int countComponents(int n, int[][] edges) {
        p = new int[n];
        for (int i = 0; i < n; ++i) {
            p[i] = i;
        }
        for (int[] e : edges) {
            int a = e[0], b = e[1];
            p[find(a)] = find(b);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            if (i == find(i)) {
                ++ans;
            }
        }
        return ans;
    }

    private int find(int x) {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
}

```

JavaScript

```
function countComponents(n, number, edges: number[][]): number {
    const g: Map<number, number[]> = new Map<number, number[]>();
    for (const [a, b] of edges) {
        g.get(a).push(b);
        g.get(b).push(a);
    }

    const vis = new Set<number>();
    let ans = 0;
    for (const [i, v] of g) {
        if (vis.has(i)) {
            continue;
        }
        const q = [i];
        for (const j of q) {
            if (vis.has(j)) {
                continue;
            }
            vis.add(j);
            q.push(...g.get(j));
        }
        ans++;
    }
    return ans;
}

// runtime
function countComponents(n, number, edges: number[][]): number {
    const g: number[][] = Array.from({length: n}, () => []);
    for (const [a, b] of edges) {
        g[a].push(b);
        g[b].push(a);
    }
    const vis: boolean[] = Array(n).fill(false);
    const dfs = (i: number): number => {
        if (vis[i]) {
            return 0;
        }
        vis[i] = true;
        for (const j of g[i]) {
            dfs(j);
        }
        return 1;
    };
    return g.reduce((acc, _, i) => acc + dfs(i), 0);
}

Go
Go
```

```
func countComponents(n int, edges [][]int) (ans int) {
    g := make([][]int, n)
    for i := range edges {
        a, b := edges[i][0], edges[i][1]
        g[a] = append(g[a], b)
        g[b] = append(g[b], a)
    }
    vis := make([]bool, n)
    var dfs func(int) int
    dfs = func(i int) int {
        if vis[i] {
            return 0
        }
        vis[i] = true
        for j := range g[i] {
            dfs(j)
        }
        return 1
    }
    for i := range g {
        ans += dfs(i)
    }
    return
}

Go
func countComponents(n int, edges [][]int) (ans int) {
    g := make([][]int, n)
    for i := range g {
        g[i] = []
    }
    var find func(int) int
    find = func(i int) int {
        if g[i] != nil {
            p[i] = find(g[i][0])
        }
        return p[i]
    }
    for i := range edges {
        a, b := edges[i][0], edges[i][1]
        p[find(a)] = find(b)
    }
    for i := 0; i < n; i++ {
        if i == find(i) {
            ans++
        }
    }
    return
}

[*] DFS — Pattern Solution (matched from community answers)
Python
```

```
# Function to count the number of connected components in an undirected graph.
def count_components(n, edges):
    # Build the adjacency list for the graph.
    graph = [[] for i in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    # Set to keep track of visited nodes.
    visited = set()

    # DFS function to perform DFS traversal starting from a node.
    def dfs(node):
        stack = [node]
        while stack:
            current = stack.pop()
            if current not in visited:
                visited.add(current)
                for neighbor in graph[current]:
                    if neighbor not in visited:
                        stack.append(neighbor)

    # Count connected components by initiating DFS on unvisited nodes.
    count = 0
    for i in range(n):
        if i not in visited:
            dfs(i)
            count += 1

    return count

# Example usage
print(count_components(5, [[0,1],[1,2],[3,4]])) # Expected output: 2
```

#417 MEDIUM

Pacific Atlantic Water Flow

LeetCode · Simplest · Rust · LeetCode · Editorial

Array · DFS · BFS · Matrix

Like Grind 109

Problem:

There is an $m \times n$ rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix heights where heights[i][j] represents the height above sea level of the cell at coordinate (i, j).

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a 2D list of grid coordinates result where result[i] = [ri, ci] denotes that rain water can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.

Example 1:

Pacific Ocean						
	1	2	2	3	5	
Pacific	3	2	3	4	4	Atlantic
Ocean	2	4	5	3	1	Ocean
	6	7	1	4	5	
	5	1	1	2	4	
Atlantic Ocean						

Input: heights = [[1,2,3,5],[2,3,4,4],[2,4,5,3],[6,7,1,4],[5,1,1,2,4]]

Output: [[0,4],[1,2],[1,4],[2,2],[3,0],[3,1],[4,0]]

Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:

- [0,4] → Atlantic Ocean
- [1,3]: [1,3] → [0,3] → Pacific Ocean
- [1,3] → [1,4] → Atlantic Ocean
- [1,4]: [1,4] → [1,3] → [0,3] → Pacific Ocean
- [1,4] → Atlantic Ocean
- [2,2]: [2,2] → [1,2] → [0,2] → Atlantic Ocean
- [2,2] → [2,3] → [2,4] → Atlantic Ocean
- [3,0]: [3,0] → Pacific Ocean
- [3,0] → [4,0] → Atlantic Ocean
- [3,1]: [3,1] → [3,0] → Pacific Ocean
- [3,1] → [4,1] → Atlantic Ocean
- [4,0]: [4,0] → Pacific Ocean
- [4,0] → Atlantic Ocean

Note that there are other possible paths for these cells to flow to the Pacific and Atlantic oceans.

Example 2:

Input: heights = [[1]]

Output: [[0,0]]

Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Constraints:

- $m == \text{heights.length}$

```
• m == heights[0].length
• 1 <= m, n <= 200
• 0 <= heights[i][j] <= 105

Python
Python

# Python solution for Pacific Atlantic Water Flow

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        if not heights or not heights[0]:
            return []

        rows, cols = len(heights), len(heights[0])
        pacific_reachable = [[False for _ in range(cols)] for _ in range(rows)]
        atlantic_reachable = [[False for _ in range(cols)] for _ in range(rows)]

        # Directions: up, down, left, right
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        # DFS function to mark reachable cells
        def dfs(r, c, reachable):
            reachable[r][c] = True
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                # Check bounds and conditions to flow water
                if (0 <= nr < rows and 0 <= nc < cols and
                    not reachable[nr][nc] and
                    heights[nr][nc] <= heights[r][c]):
                    dfs(nr, nc, reachable)

        # DFS from Pacific ocean borders (top row and left column)
        for i in range(rows):
            dfs(i, 0, pacific_reachable)
        for j in range(cols):
            dfs(0, j, pacific_reachable)

        # DFS from Atlantic ocean borders (bottom row and right column)
        for i in range(rows):
            dfs(i, cols - 1, atlantic_reachable)
        for j in range(cols):
            dfs(rows - 1, j, atlantic_reachable)

        # Collect cells that can reach both oceans
        result = []
        for r in range(rows):
            for c in range(cols):
                if pacific_reachable[r][c] and atlantic_reachable[r][c]:
                    result.append([r, c])

        return result

Python
```

```
class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        m = len(heights)
        n = len(heights[0])
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]

        def dfs(i: int, j: int, h: int, seen: List[List[bool]]) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if seen[i][j] or heights[i][j] < h:
                return
            seen[i][j] = True
            dfs(i - 1, j, heights[i][j], seen)
            dfs(i + 1, j, heights[i][j], seen)
            dfs(i, j - 1, heights[i][j], seen)
            dfs(i, j + 1, heights[i][j], seen)

        for i in range(m):
            dfs(i, 0, 0, seenP)
            dfs(i, n - 1, 0, seenA)

        for j in range(n):
            dfs(0, j, 0, seenP)
            dfs(m - 1, j, 0, seenA)

        return [[i, j]
                for i in range(m)
                for j in range(n)
                if seenP[i][j] and seenA[i][j]]

Python
```

```

class Solution {
    def pacificAtlantic(heights: List[List[Int]]) => List[List[Int]] {
        def bfs(x: Int, y: Int): List[Int] = {
            val q = Queue()
            q.enqueue(x, y)
            while q.nonEmpty {
                val (x, y) = q.dequeue()
                for a in List(-1, 0, 1, 0, 1, -1, 0, 1) {
                    val nx = x + a, ny = y + a
                    if (nx > 0 && nx < heights[0].length && ny > 0 && ny < heights[0].length && heights[nx][ny] > heights[x][y]) {
                        q.enqueue(nx, ny)
                    }
                }
            }
            List(x, y)
        }

        val pacific = bfs(0, 0)
        val atlantic = bfs(heights[0].length - 1, heights[0].length - 1)

        val res = List()
        for i in 0 until heights[0].length {
            for j in 0 until heights[0].length {
                if (pacific.contains(i, j) && atlantic.contains(i, j)) {
                    res += List(i, j)
                }
            }
        }
        res
    }
}

```

C++

C++

DPS · LeetCode · By Pattern

— 2107 —

LeetCode

```

typedef pair<int, int> pii;

class Solution {
public:
    vector<vector<int>>> heights;
    int m;
    int n;

    vector<vector<int>>> pacificAtlantic(vector<vector<int>>> heights) {
        m = heights.size();
        n = heights[0].size();
        this->heights = heights;
        queue<pii> q1;
        queue<pii> q2;
        unordered_set<int> vis1;
        unordered_set<int> vis2;

        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == 0 || j == 0) {
                    vis1.insert(i * n + j);
                    q1.emplace(i, j);
                }
                if (i == m - 1 || j == n - 1) {
                    vis2.insert(i * n + j);
                    q2.emplace(i, j);
                }
            }
        }

        bfs(q1, vis1);
        bfs(q2, vis2);
        vector<vector<int>>> ans;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                int x = i * n + j;
                if (vis1.count(x) && vis2.count(x)) {
                    ans.push_back(i, j);
                }
            }
        }
        return ans;
    }

    void bfs(queue<pii>& q, unordered_set<int>& vis) {
        vector<int> dirs = {-1, 0, 1, 0, -1};
        while (!q.empty()) {
            for (int k = 0; k < dirs.size(); ++k) {
                auto p = q.front();
                q.pop();
                for (int i = 0; i < 4; ++i) {
                    int x = p.first + dirs[i];
                }
            }
        }
    }
}

```

C++

C++

DPS · LeetCode · By Pattern

— 2108 —

LeetCode

```

let y = p.second + dir[i + 1];
if (x == 0 && x < n && y == 0 && y < n && !vis.count(x + n * y) && heights[x][y] > heights[p.first][p.second]) {
    vis.insert(x + n * y);
    q.emplace(x, y);
}
}
}
}
};

```

Java

Java

```

class Solution {
    private List<int> heights;
    private List<int> m;
    private List<int> n;

    public List<List<Integer>> pacificAtlantic(List<int> heights) {
        m = heights.size();
        n = heights[0].size();
        this.heights = heights;
        Deque<int>[] q1 = new LinkedList<>[m];
        Deque<int>[] q2 = new LinkedList<>[m];
        Set<Integer> vis1 = new HashSet<>();
        Set<Integer> vis2 = new HashSet<>();
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                if (i == 0 || j == 0) {
                    vis1.add(i * n + j);
                    q1.offer(new Integer[] { i, j });
                }
                if (i == m - 1 || j == n - 1) {
                    vis2.add(i * n + j);
                    q2.offer(new Integer[] { i, j });
                }
            }
        }

        bfs(q1, vis1);
        bfs(q2, vis2);
        List<List<Integer>> ans = new ArrayList<>();
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                int x = i * n + j;
                if (vis1.contains(x) && vis2.contains(x)) {
                    ans.add(new Integer[] { i, j });
                }
            }
        }
        return ans;
    }

    void bfs(Deque<int[]> q, Set<Integer> vis) {
        int[] dirs = {-1, 0, 1, 0, -1};
        while (!q.isEmpty()) {
            for (int k = 0; k < dirs.length; ++k) {
                int[] p = q.poll();
                for (int i = 0; i < 4; ++i) {
                    int x = p[0] + dirs[i];
                    int y = p[1] + dirs[i + 1];
                    if (x >= 0 && x < n && y >= 0 && y < n && !vis.contains(x + n * y) && heights[x][y] > heights[p[0]][p[1]]) {
                        vis.add(x + n * y);
                        q.offer(new Integer[] { x, y });
                    }
                }
            }
        }
    }
}

```

JavaScript

JavaScript

DPS · LeetCode · By Pattern

— 2109 —

LeetCode

```

bfs(q1, vis1);
bfs(q2, vis2);
List<List<Integer>> ans = new ArrayList<>();
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        int x = i * n + j;
        if (vis1.contains(x) && vis2.contains(x)) {
            ans.add(new Integer[] { i, j });
        }
    }
}
return ans;
}

private void bfs(Deque<int[]> q, Set<Integer> vis) {
    int[] dirs = {-1, 0, 1, 0, -1};
    while (!q.isEmpty()) {
        for (int k = 0; k < dirs.length; ++k) {
            int[] p = q.poll();
            for (int i = 0; i < 4; ++i) {
                int x = p[0] + dirs[i];
                int y = p[1] + dirs[i + 1];
                if (x >= 0 && x < n && y >= 0 && y < n && !vis.contains(x + n * y) && heights[x][y] > heights[p[0]][p[1]]) {
                    vis.add(x + n * y);
                    q.offer(new Integer[] { x, y });
                }
            }
        }
    }
}

```

Python

Python

DPS · LeetCode · By Pattern

— 2110 —

LeetCode

```

function pacificAtlantic(heights: number[][]): number[][] {
    const m = heights.length;
    const n = heights[0].length;
    const dirs = [
        [0, 1],
        [0, -1],
        [1, 0],
        [-1, 0],
    ];

    const grid = new Array(m).fill(0).map(() => new Array(n).fill(0));
    const isVis = new Array(m).fill(0).map(() => new Array(n).fill(false));

    const dfs = (i: number, j: number) => {
        if (isVis[i][j]) {
            return;
        }
        isVis[i][j] = true;
        for (const [x, y] of dirs) {
            if (x >= 0 && x < n && y >= 0 && y < n && !isVis[x][y] && heights[i][j] < heights[x][y]) {
                dfs(x, y);
            }
        }
    };

    for (let i = 0; i < m; ++i) {
        dfs(i, 0);
    }
    for (let i = 0; i < m; ++i) {
        dfs(i, n - 1);
    }
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (isVis[i][j] && !isVis[i][j + 1]) {
                dfs(i, j + 1);
            }
        }
    }
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (isVis[i][j] && !isVis[i - 1][j]) {
                dfs(i - 1, j);
            }
        }
    }
    const res = [];
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (isVis[i][j] && !isVis[i][j + 1]) {
                res.push([i, j]);
            }
        }
    }
    return res;
}

```

Go

Go

DPS · LeetCode · By Pattern

— 2111 —

LeetCode

```

func pacificAtlantic(heights [][]int) [][]int {
    m, n := len(heights), len(heights[0])
    vis1 := make(map[int]bool)
    vis2 := make(map[int]bool)
    var q1, q2 []int
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if i == 0 || j == 0 {
                vis1[j] = true
                q1 = append(q1, i*n+j)
            }
            if i == m-1 || j == n-1 {
                vis2[j] = true
                q2 = append(q2, i*n+j)
            }
        }
    }
    dirs := [][]int{{-1, 0, 1, 0, -1}}
    bfs := func(x, y int) {
        for i := 0; i < 4; i++ {
            x, y = x+dirs[i][0], y+dirs[i][1]
            if x >= 0 && x < m && y >= 0 && y < n && !vis1[x*n+y] && heights[x][y] > heights[x+dirs[i][0]][y+dirs[i][1]] {
                vis1[x*n+y] = true
                q1 = append(q1, x*n+y)
            }
        }
    }
    bfs(q1, vis1)
    bfs(q2, vis2)
    var ans [][]int
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            x, y := i, j
            if vis1[x*n+y] && vis2[x*n+y] {
                ans = append(ans, []int{i, j})
            }
        }
    }
    return ans
}

```

Rust

Rust

DPS · LeetCode · By Pattern

— 2112 —

LeetCode

```

use std::collections::VecDeque;

impl Solution {
    pub fn pacific_atlantic(heights: Vec<Vec<i32>>) -> Vec<Vec<i32>> {
        let (m, n) = (heights.len(), heights[0].len());
        let mut vis1 = vec![vec![false; n]; m];
        let mut vis2 = vec![vec![false; n]; m];
        let mut q1 = VecDeque::new();
        let mut q2 = VecDeque::new();
        let dirs = [-1, 0, 1, 0, -1];

        for i in 0..n {
            q1.push_back((0, i));
            vis1[i][0] = true;
            q2.push_back((m-1, i));
            vis2[i][m-1] = true;
        }

        for j in 0..m {
            q1.push_back((j, 0));
            vis1[j][0] = true;
            q2.push_back((j, m-1));
            vis2[j][m-1] = true;
        }

        let bfs = |q: &mut VecDeque<(usize, usize)>, vis: &mut Vec<Vec<bool>>| {
            while let Some((x, y)) = q.pop_front() {
                for k in 0..4 {
                    let nx = x as i32 + dirs[k];
                    let ny = y as i32 + dirs[k] + 1;
                    if nx == 0 || nx == m || ny == 0 || ny == n || vis[nx as usize][ny as usize] || heights[nx as usize][ny as usize] > heights[x][y] {
                        continue;
                    }
                    vis[nx as usize][ny as usize] = true;
                    q.push_back((nx as usize, ny as usize));
                }
            }
        };

        bfs(&mut q1, &mut vis1);
        bfs(&mut q2, &mut vis2);

        let mut ans = vec![];
        for i in 0..n {
            for j in 0..m {
                if vis1[i][j] && vis2[i][j] {
                    ans.push((vec![i as i32, j as i32]));
                }
            }
        }
        ans
    }
}

```

DPS · LeeMastery — By Pattern

— 2113 —

LeeMastery

[*] DFS — Pattern Solution (matched from community answers)

```

Python
# Python Solution for Pacific Atlantic Water Flow

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        if not heights or not heights[0]:
            return []

        rows, cols = len(heights), len(heights[0])
        pacific_reachable = [[False for _ in range(cols)] for _ in range(rows)]
        atlantic_reachable = [[False for _ in range(cols)] for _ in range(rows)]

        # Directions: up, down, left, right
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

        # DFS function to mark reachable cells
        def dfs(r, c, reachable):
            reachable[r][c] = True
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                # Check bounds and condition for flow water
                if (0 <= nr < rows and 0 <= nc < cols and
                    not reachable[nr][nc] and
                    heights[nr][nc] <= heights[r][c]):
                    dfs(nr, nc, reachable)

        # DFS from Pacific ocean borders (top row and left column)
        for i in range(rows):
            dfs(i, 0, pacific_reachable)
        for j in range(cols):
            dfs(0, j, pacific_reachable)

        # DFS from Atlantic ocean borders (bottom row and right column)
        for i in range(rows):
            dfs(i, cols-1, atlantic_reachable)
        for j in range(cols):
            dfs(rows-1, j, atlantic_reachable)

        # Collect cells that can reach both oceans
        result = []
        for r in range(rows):
            for c in range(cols):
                if pacific_reachable[r][c] and atlantic_reachable[r][c]:
                    result.append([r, c])
        return result

```

#490

MEDIUM

The Maze

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · DFS · BFS · Matrix

LeetCode Premium 98

DPS · LeeMastery — By Pattern

— 2114 —

LeeMastery

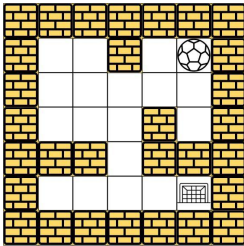
Problem:

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the m x n maze, the ball's start position and the destination, where start = [startrow, startcol] and destination = [destinationrow, destinationcol], return true if the ball can stop at the destination, otherwise return false.

You may assume that the borders of the maze are all walls (see examples).

Example 1:



Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [4,4]

Output: true

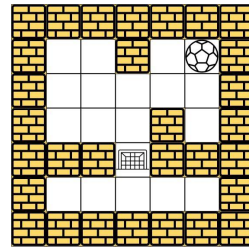
Explanation: One possible way is : left -> down -> left -> down -> right -> down -> right.

Example 2:

DPS · LeeMastery — By Pattern

— 2115 —

LeeMastery



Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [2,2]

Output: false

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], start = [4,3], destination = [0,1]

Output: false

Constraints:

- m == maze.length
- n == maze[0].length
- 1 <= m, n <= 100
- maze[i][j] is 0 or 1.
- start.length == 2
- destination.length == 2
- 0 <= startrow, destinationrow < m
- 0 <= startcol, destinationcol < n
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

>> Brute Force (DFS) Interview Prep

Python

DPS · LeeMastery — By Pattern

— 2116 —

LeeMastery

```

# Brute Force Approach
class Solution:
    def hasPath(self, maze, start, destination):
        # Brute Force - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for nb in graph.get(node, []):
                if nb not in visited:
                    dfs(nb)
            for node in range(n):
                if node not in visited:
                    dfs(node); count += 1
            return count

        # Process nodes in BFS fashion
        while queue:
            current = queue.popleft()
            # Check if the current position is the destination
            if (current[0], current[1]) == destination:
                return True
            # Explore all four possible directions
            for dx, dy in directions:
                x, y = current
                # Add the next cell if it's a cell or boundary
                while 0 <= x + dx < rows and 0 <= y + dy < cols and maze[x + dx][y + dy] == 0:
                    x += dx
                    y += dy

```

Python

Python

Function to determine if there is a valid path in the maze from start to destination.

def hasPath(maze, start, destination):

Create a collection to store the next path of processed cells

rows, cols = len(maze), len(maze[0])

visited = [[False] * cols for _ in range(rows)]

queue = deque([tuple(start)])

visited[start[0]][start[1]] = True

Directions: up, down, left, right

directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

Process nodes in BFS fashion

while queue:

current = queue.popleft()

Check if the current position is the destination

if (current[0], current[1]) == destination:

return True

Explore all four possible directions

for dx, dy in directions:

x, y = current

Add the next cell if it's a cell or boundary

while 0 <= x + dx < rows and 0 <= y + dy < cols and maze[x + dx][y + dy] == 0:

x += dx

y += dy

DPS · LeeMastery — By Pattern

— 2117 —

LeeMastery

```

# If this new step has not been visited, add it to the queue
if not visited[x][y]:
    visited[x][y] = True
    queue.append((x, y))
    return False

# Create a queue
maze = [[0,0,1,0,0],
        [0,0,0,0,0],
        [0,0,0,1,0],
        [1,1,0,1,1],
        [0,0,0,0,0]]
start = [0,4]
destination = [4,4]
print(hasPath(maze, start, destination)) # Expected output: True

Python
class Solution:
    def hasPath(self, maze, start, destination):
        rows, cols = len(maze), len(maze[0])
        visited = [[False] * cols for _ in range(rows)]
        queue = deque([tuple(start)])
        visited[start[0]][start[1]] = True
        # Directions: up, down, left, right
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
        # Process nodes in BFS fashion
        while queue:
            current = queue.popleft()
            # Check if the current position is the destination
            if (current[0], current[1]) == destination:
                return True
            # Explore all four possible directions
            for dx, dy in directions:
                x, y = current
                # Add the next cell if it's a cell or boundary
                while 0 <= x + dx < rows and 0 <= y + dy < cols and maze[x + dx][y + dy] == 0:
                    x += dx
                    y += dy
                if (x, y) == destination:
                    return True
            if (x, y) in seen:
                continue
            queue.append((x, y))
            seen.add((x, y))
        return False

```

Python

DPS · LeeMastery — By Pattern

— 2118 —

LeeMastery

```
class Solution:
    def hasPath(self, maze: List[List[int]], start: List[int], destination: List[int]) -> bool:
        def dfs(i, j):
            if visit[i][j]:
                return
            visit[i][j] = True
            if (i, j) == destination:
                return
            for a, b in [(0, -1), (0, 1), (1, 0), (-1, 0)]:
                x, y = i, j
                while 0 <= x < m and 0 <= y < n and maze[x + a][y + b] == 0:
                    x, y = x + a, y + b
                dfs(x, y)
        m, n = len(maze), len(maze[0])
        visit = [[False] * n for _ in range(m)]
        dfs(start[0], start[1])
        return visit[destination[0]][destination[1]]

Python
class Solution:
    def hasPath(self, maze: List[List[int]], start: List[int], destination: List[int]) -> bool:
        m, n = len(maze), len(maze[0])
        q = deque([start])
        rx, cx = start
        vis = [[rx, cx]]
        while q:
            i, j = q.popleft()
            for a, b in [(0, -1), (0, 1), (-1, 0), (1, 0)]:
                x, y = i, j
                while 0 <= x < m and 0 <= y < n and maze[x + a][y + b] == 0:
                    x, y = x + a, y + b
                if (x, y) == destination:
                    return True
                if (x, y) not in vis:
                    vis.append((x, y))
                    q.append((x, y))
        return False

C++
C++
```

```
class Solution {
public:
    bool hasPath(vector<vector<int>>& maze, vector<int>& start, vector<int>& destination) {
        int m = maze.size();
        int n = maze[0].size();
        queue<vector<int>> q{ start };
        vector<vector<bool>> visit(m, vector<bool>(n));
        visit[start[0]][start[1]] = true;
        vector<int> dirs = {-1, 0, 1, 0, -1};
        while (!q.empty()) {
            auto p = q.front();
            q.pop();
            int i = p[0], j = p[1];
            for (int k = 0; k < 4; ++k) {
                int x = i, y = j;
                int a = dirs[k], b = dirs[k + 1];
                while (x <= 0 && x < m && y <= 0 && y < n && maze[x + a][y + b] == 0) {
                    x += a;
                    y += b;
                }
                if (x == destination[0] && y == destination[1]) return 1;
                if (!visit[i][j]) {
                    visit[i][j] = true;
                    q.push({x, y});
                }
            }
        }
        return 0;
    }
};

Java
Java
```

```
class Solution {
    public boolean hasPath(int[][] maze, int[] start, int[] destination) {
        int m = maze.length;
        int n = maze[0].length;
        boolean[][] vis = new boolean[m][n];
        vis[start[0]][start[1]] = true;
        Deque<int> q = new LinkedList<>();
        q.offer(start);
        int[] dirs = {-1, 0, 1, 0, -1};
        while (!q.isEmpty()) {
            int[] p = q.poll();
            int i = p[0], j = p[1];
            for (int k = 0; k < 4; ++k) {
                int x = i, y = j;
                int a = dirs[k], b = dirs[k + 1];
                while (x <= 0 && x < m && y <= 0 && y < n && maze[x + a][y + b] == 0) {
                    x += a;
                    y += b;
                }
                if (x == destination[0] && y == destination[1]) {
                    return true;
                }
                if (!vis[i][j]) {
                    vis[i][j] = true;
                    q.offer(new int[] {x, y});
                }
            }
        }
        return false;
    }
}

Go
Go
```

```
func hasPath(maze [][]int, start []int, destination []int) bool {
    m, n := len(maze), len(maze[0])
    vis := make([]bool, m)
    for i := range vis {
        vis[i] = make([]bool, n)
    }
    vis[start[0]][start[1]] = true
    q := [][]int{start}
    dirs := [][]int{{-1, 0, 1, 0, -1}}
    for len(q) > 0 {
        i, j := q[0][0], q[0][1]
        q = q[1:]
        for k := 0; k < 4; k++ {
            x, y := i, j
            m, b := dirs[k], dirs[k+1]
            for x += b && x < m && y += b && y < n && maze[x][y] == 0 {
                x += b;
                y += b;
            }
            if x == destination[0] && y == destination[1] {
                return true
            }
            if !vis[i][j] {
                vis[i][j] = true
                q = append(q, []int{x, y})
            }
        }
    }
    return false
}
```

[*] DFS — Pattern Solution (matched from community answers)

```
Python
class Solution:
    def hasPath(self, maze: List[List[int]], start: List[int], destination: List[int]) -> bool:
        def dfs(i, j):
            if visit[i][j]:
                return
            visit[i][j] = True
            if (i, j) == destination:
                return
            for a, b in [(0, -1), (0, 1), (1, 0), (-1, 0)]:
                x, y = i, j
                while 0 <= x < m and 0 <= y < n and maze[x + a][y + b] == 0:
                    x, y = x + a, y + b
                dfs(x, y)
        m, n = len(maze), len(maze[0])
        visit = [[False] * n for _ in range(m)]
        dfs(start[0], start[1])
        return visit[destination[0]][destination[1]]
```

#694 MEDIUM

Number of Distinct Islands

LeetCode · SimplifyLear · WalkCC · LeetCode · Editorial

Hash Table · DFS · BFS · Union Find · Hash Function

Little Premium 98

Problem:

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical). You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

Return the number of distinct islands.

Example 1:

1	1	0	0	0
1	1	0	0	0
0	0	0	1	1
0	0	0	1	1

Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,1,1],[0,0,0,1,1]]

Output: 1

Example 2:

1	1	0	1	1
1	0	0	0	0
0	0	0	0	1
1	1	0	1	1

Input: grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,1,1],[1,1,0,1,1]]

Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- grid[i][j] is either 0 or 1.

>> Brute Force (DFS) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def numDistinctIslands(self, grid):
        # Brute Force - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for nb in grid.get_node(1):
                if nb not in visited:
                    for node in range(n):
                        if node not in visited:
                            dfs(node); count += 1
            return count
```

Python

Python

```

def numDistinctIslands(grid):
    # Get the number of rows and columns
    rows, cols = len(grid), len(grid[0])
    # Set to store unique island shapes
    distinct_shapes = set()

    # DFS function to explore the island and record its relative shape
    def dfs(r, c, orig_r, orig_c, shape):
        # Check boundaries and if the cell is unvisited land
        if r < 0 or c < 0 or r == rows or c == cols or grid[r][c] == 0:
            return
        # Mark the cell as visited by setting it to 0
        grid[r][c] = 0
        # Record the relative position from the origin of this island
        shape.append((r - orig_r, c - orig_c))
        # Explore all 4-directionally adjacent cells
        drs = [-1, 0, 1, 0], [0, 1, 0, -1]
        dfs(r + 1, c, orig_r, orig_c, shape)
        dfs(r - 1, c, orig_r, orig_c, shape)
        dfs(r, c + 1, orig_r, orig_c, shape)
        dfs(r, c - 1, orig_r, orig_c, shape)

    # Iterate over every cell in the grid
    for i in range(rows):
        for j in range(cols):
            if grid[i][j] == 1:
                shape = []
                dfs(i, j, i, j, shape)
                # Convert the shape list to a tuple of sorted coordinates for canonical form
                distinct_shapes.add(tuple(sorted(shape)))

    # The number of distinct islands is the size of the set
    return len(distinct_shapes)

# Example usage:
grid = [[1,1,0,0,1],
        [1,1,0,0,1],
        [0,0,0,1,1],
        [0,0,0,1,1]]
print(numDistinctIslands(grid)) # Output: 1

```

Python

```

class Solution:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        seen = set()

        def dfs(i: int, j: int, dr: int, dc: int):
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return
            if grid[i][j] == 0 or (i, j) in seen:
                return

            seen.add((i, j))
            island.append((i - dr, j - dc))
            dfs(i + 1, j, dr, dc)
            dfs(i - 1, j, dr, dc)
            dfs(i, j + 1, dr, dc)
            dfs(i, j - 1, dr, dc)

        islands = set()
        for i in range(len(grid)):
            for j in range(len(grid[0])):
                island = []
                dfs(i, j, 1, 1)
                if island:
                    islands.add(frozenset(island))

        return len(islands)

```

Python

```

class Solution:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int):
            grid[i][j] = k
            path.append(str(k))
            dirs = (-1, 0, 1, 0, -1)
            for h in range(1, 5):
                x, y = i + dirs[h - 1], j + dirs[h]
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y, h)
            path.append(str(-k))

            paths = set()
            path = []
            m, n = len(grid), len(grid[0])
            for i, row in enumerate(grid):
                for j, v in enumerate(row):
                    if v:
                        dfs(i, j, 0)
                        paths.add(''.join(path))
                        path.clear()

            return len(paths)

```

Python

```

class Solution {
public:
    def dfs(int i, int j, int k, int):
        grid[i][j] = k
        dirs = {-1, 0, 1, 0, -1}
        for h in range(1, 5):
            x, y = i + dirs[h - 1], j + dirs[h]
            if 0 <= x < m and 0 <= y < n and grid[x][y]:
                dfs(x, y, h)
            path.append(str(-k))

        paths = set()
        path = []
        m, n = len(grid), len(grid[0])
        for i, row in enumerate(grid):
            for j, v in enumerate(row):
                if v:
                    dfs(i, j, 0)
                    paths.add(''.join(path))
                    path.clear()

        return len(paths)
}

```

C++

C++

```

class Solution {
public:
    int numDistinctIslands(vector<vector<int>>& grid) {
        unordered_set<string> paths;
        string path;
        int m = grid.size(), n = grid[0].size();
        int dirs[] = {-1, 0, 1, 0, -1};

        function<void(int, int)> dfs = [&](int i, int j, int k) {
            grid[i][j] = k;
            path += to_string(k);
            for (int h = 1; h < 5; ++h) {
                int x = i + dirs[h - 1], y = j + dirs[h];
                if (x == 0 || x == m || y == 0 || y == n || grid[x][y] == 0 ||
                    dfs(x, y, h))
                    path += to_string(k);
            }
            for (int i = 0; i < m; ++i)
                for (int j = 0; j < n; ++j)
                    if (grid[i][j]) {
                        dfs(i, j, 0);
                        paths.insert(path);
                    }
            return paths.size();
        }
    }
};

```

```

        path.clear();
    }
    return paths.size();
};

```

Java

Java

```

class Solution {
    private int m;
    private int n;
    private List<int> grid;
    private StringBuilder path = new StringBuilder();

    public int numDistinctIslands(int[][] grid) {
        m = grid.length;
        n = grid[0].length;
        this.grid = grid;
        Set<String> paths = new HashSet<>();
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j)
                if (grid[i][j] == 1) {
                    dfs(i, j, 0);
                    paths.add(path.toString());
                    path.setLength(0);
                }
        return paths.size();
    }

    private void dfs(int i, int j, int k) {
        grid[i][j] = k;
        path.append(k);
        int[] dirs = {-1, 0, 1, 0, -1};
        for (int h = 1; h < 5; ++h) {
            int x = i + dirs[h - 1];
            int y = j + dirs[h];
            if (x == 0 || x == m || y == 0 || y == n || grid[x][y] == 0 ||
                dfs(x, y, h))
                path.append(k);
        }
        paths.add(path);
    }
}

```

TypeScript

TypeScript

```

function numDistinctIslands(grid: number[][]): number {
    const m = grid.length;
    const n = grid[0].length;
    const paths: Set<string> = new Set();
    const path: number[] = [];
    const dirs: number[] = [-1, 0, 1, 0, -1];
    const dfs = (i: number, j: number, k: number) => {
        grid[i][j] = k;
        path.push(k);
        for (let h = 1; h < 5; ++h) {
            const [x, y] = [i + dirs[h - 1], j + dirs[h]];
            if (x == 0 || x == m || y == 0 || y == n || grid[x][y] == 0 ||
                dfs(x, y, h))
                path.push(k);
        }
        for (let i = 0; i < m; ++i)
            for (let j = 0; j < n; ++j)
                if (grid[i][j]) {
                    dfs(i, j, 0);
                    paths.add(path.join(''));
                    path.length = 0;
                }
        return paths.size;
    }
}

```

Go

Go

```

func numDistinctIslands(grid []][int]) int {
    m, n := len(grid), len(grid[0])
    paths := map[string]bool{}
    path := []byte{}
    dirs := [][]int{{-1, 0, 1, 0, -1}}
    var dfs func(i, j, k int)
    dfs = func(i, j, k int) {
        grid[i][j] = k
        path = append(path, byte(k))
        for h := 1; h < 5; h++ {
            x, y := i+dirs[h-1], j+dirs[h]
            if x == 0 || x == m || y == 0 || y == n || grid[x][y] == 1 ||
                dfs(x, y, k)
                path = append(path, byte(k))
        }
        for i, row := range grid {
            for j, v := range row {
                if v == 1 {
                    dfs(i, j, 0)
                    paths[string(path)] = true
                    path = path[:0]
                }
            }
        }
    }
}

```

```

    return len(paths)
}

```

[*] DPS — Pattern Solution (masched from community answers)

Python

```

def numDistinctIslands(grid):
    # Get the number of rows and columns
    rows, cols = len(grid), len(grid[0])
    # Set to store unique island shapes
    distinct_shapes = set()

    # DFS function to explore the island and record its relative shape
    def dfs(r, c, orig_r, orig_c, shape):
        # Check boundaries and if the cell is unvisited land
        if r < 0 or c < 0 or r == rows or c == cols or grid[r][c] == 0:
            return
        # Mark the cell as visited by setting it to 0
        grid[r][c] = 0
        # Record the relative position from the origin of this island
        shape.append((r - orig_r, c - orig_c))
        # Explore all 4-directionally adjacent cells
        drs = [-1, 0, 1, 0], [0, 1, 0, -1]
        dfs(r + 1, c, orig_r, orig_c, shape)
        dfs(r - 1, c, orig_r, orig_c, shape)
        dfs(r, c + 1, orig_r, orig_c, shape)
        dfs(r, c - 1, orig_r, orig_c, shape)

    # Iterate over every cell in the grid
    for i in range(rows):
        for j in range(cols):
            if grid[i][j] == 1:
                shape = []
                dfs(i, j, i, j, shape)
                # Convert the shape list to a tuple of sorted coordinates for canonical form
                distinct_shapes.add(tuple(sorted(shape)))

    # The number of distinct islands is the size of the set
    return len(distinct_shapes)

# Example usage:
grid = [[1,1,0,0,1],
        [1,1,0,0,1],
        [0,0,0,1,1],
        [0,0,0,1,1]]
print(numDistinctIslands(grid)) # Output: 1

```

#695

MEDIUM

Max Area of Island

LeetCode · Union-find · Math · DFS · LeeCode · Editorial

Array · DFS · BFS · Union Find · Matrix

Little Denny Zhang

Problem:

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical). You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: grid = [[0,0,1,0,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,0,1,1,0,0,0],[0,1,0,0,1,0,0,0,0,0,0,0,0,0],[0,1,0,0,1,0,0,0,0,0,0,0,0,0],[0,0,0,0,0,0,0,0,0,0,0,0,0,0],[0,0,0,0,0,0,0,0,0,0,0,0,0,0],[0,0,0,0,0,0,0,0,0,0,0,0,0,0]]

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: grid = [[0,0,0,0,0,0,0]]

Output: 0

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[0].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

>> Brute Force [DFS] Interview Prep

Python

DFS · LeetCode · By Pattern

— 2131 —

LeetCode

```
# Brute Force Approach
class Solution:
    def maxAreaOfIsland(self, grid):
        # Brute Force O(m^2 * n^2) = O(50^4) from every unvisited land cell, track size
        m, n = len(grid), len(grid[0])
        visited = set()
        def dfs(r, c):
            if r < 0 or r >= m or c < 0 or c >= n: return 0
            if (r, c) in visited or grid[r][c] == 0: return 0
            visited.add((r, c))
            return 1 + dfs(r+1, c) + dfs(r-1, c) + dfs(r, c+1) + dfs(r, c-1)
        return max(dfs(r, c) for r in range(m) for c in range(n), default=0)
```

Python

Python

```
# Function to compute the maximum area of an island in a grid using DFS
def maxAreaOfIsland(grid):
    # Determine the dimensions of the grid
    rows = len(grid)
    cols = len(grid[0]) if rows else 0

    # Helper function for performing DFS on the grid
    def dfs(r, c):
        # Check if the current cell is out-of-bounds or is water
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == 0:
            return 0
        # Mark the current cell as visited by setting it to 0 (water)
        grid[r][c] = 0
        area = 1 # Count the current cell as part of the island
        # Recursively explore all four directions
        area += dfs(r+1, c)
        area += dfs(r-1, c)
        area += dfs(r, c+1)
        area += dfs(r, c-1)
        return area

    max_area = 0
    # Iterate through each cell in the grid
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                # Update max_area with the area obtained from the DFS
                max_area = max(max_area, dfs(r, c))
    return max_area
```

Python

DFS · LeetCode · By Pattern

— 2132 —

LeetCode

```
class Solution {
public:
    int maxAreaOfIsland(vector<vector<int>>& grid) {
        int ans = 0;
        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                if (grid[i][j] == 1)
                    return ans + dfs(i, j);
        return ans;
    }
    int dfs(int i, int j) {
        if (i < 0 || i >= grid.size() || j < 0 || j >= grid[0].size())
            return 0;
        if (grid[i][j] == 0)
            return 0;
        grid[i][j] = 2;
        return 1 + dfs(i+1, j) + dfs(i-1, j) + dfs(i, j+1) + dfs(i, j-1);
    }
};
```

Python

```
class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if grid[i][j] == 0:
                return 0
            ans = 1
            grid[i][j] = 0
            dirs = (-1, 0, 1, 0, -1)
            for a, b in pairs(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n:
                    ans += dfs(x, y)
            return ans
        m, n = len(grid), len(grid[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))
```

Python

```
class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if grid[i][j] == 0:
                return 0
            ans = 1
            grid[i][j] = 0
            dirs = (-1, 0, 1, 0, -1)
            for a, b in pairs(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n:
                    ans += dfs(x, y)
            return ans
        m, n = len(grid), len(grid[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))
```

C++

C++

DFS · LeetCode · By Pattern

— 2133 —

LeetCode

```
class Solution {
public:
    int maxAreaOfIsland(vector<vector<int>>& grid) {
        int ans = 0;
        for (int i = 0; i < grid.size(); ++i)
            for (int j = 0; j < grid[0].size(); ++j)
                ans = max(ans, dfs(i, j));
        return ans;
    }
private:
    int dfs(vector<vector<int>>& grid, int i, int j) {
        if (i < 0 || i >= grid.size() || j < 0 || j >= grid[0].size())
            return 0;
        if (grid[i][j] == 0)
            return 0;
        grid[i][j] = 2;
        return 1 +
            dfs(grid, i+1, j) + dfs(grid, i-1, j) + //
            dfs(grid, i, j+1) + dfs(grid, i, j-1);
    }
};
```

C++

```
class Solution {
public:
    int maxAreaOfIsland(vector<vector<int>>& grid) {
        int m = grid.size(), n = grid[0].size();
        int dirs[] = {-1, 0, 1, 0, -1};
        int ans = 0;
        function<int(int, int)> dfs = [&](int i, int j) {
            if (grid[i][j] == 0) {
                return 0;
            }
            int ans = 1;
            grid[i][j] = 0;
            for (int k = 0; k < 4; ++k) {
                int x = i + dirs[k], y = j + dirs[k + 1];
                if (x <= 0 || x >= m || y <= 0 || y >= n) {
                    ans += dfs(x, y);
                }
            }
            return ans;
        };
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                ans = max(ans, dfs(i, j));
            }
        return ans;
    }
};
```

DFS · LeetCode · By Pattern

— 2134 —

LeetCode

Java

Java

```
class Solution {
    public int maxAreaOfIsland(int[][] grid) {
        int ans = 0;
        for (int i = 0; i < grid.length; ++i)
            for (int j = 0; j < grid[0].length; ++j)
                ans = Math.max(ans, dfs(grid, i, j));
        return ans;
    }
    private int dfs(int[][] grid, int i, int j) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length)
            return 0;
        if (grid[i][j] == 0)
            return 0;
        grid[i][j] = 2;
        return 1 +
            dfs(grid, i+1, j) + dfs(grid, i-1, j) + //
            dfs(grid, i, j+1) + dfs(grid, i, j-1);
    }
}
```

Java

```
class Solution {
    private int m;
    private int n;
    private int[] grid;

    public int maxAreaOfIsland(int[][] grid) {
        m = grid.length;
        n = grid[0].length;
        this.grid = grid;
        int ans = 0;
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) {
                ans = Math.max(ans, dfs(i, j));
            }
        return ans;
    }
    private int dfs(int i, int j) {
        if (grid[i][j] == 0) {
            return 0;
        }
        int ans = 1;
        grid[i][j] = 0;
        int[] dirs = {-1, 0, 1, 0, -1};
    }
```

DFS · LeetCode · By Pattern

— 2135 —

LeetCode

```
for (int k = 0; k < 4; ++k) {
    int x = i + dirs[k], y = j + dirs[k + 1];
    if (x <= 0 || x >= m || y <= 0 || y >= n) {
        ans += dfs(x, y);
    }
}
return ans;
}
```

JavaScript

JavaScript

```
function maxAreaOfIsland(grid: number[][]): number {
    const m = grid.length;
    const n = grid[0].length;
    const dirs = [-1, 0, 1, 0, -1];
    const dfs = (i: number, j: number): number => {
        if (grid[i][j] === 0) {
            return 0;
        }
        let ans = 1;
        grid[i][j] = 0;
        for (let k = 0; k < 4; ++k) {
            const [x, y] = [i + dirs[k], j + dirs[k + 1]];
            if (x <= 0 || x >= m || y <= 0 || y >= n) {
                ans += dfs(x, y);
            }
        }
        return ans;
    };
    let ans = 0;
    for (let i = 0; i < m; ++i)
        for (let j = 0; j < n; ++j) {
            ans = Math.max(ans, dfs(i, j));
        }
    return ans;
}
```

Go

Go

DFS · LeetCode · By Pattern

— 2136 —

LeetCode


```

        for (const auto& [email, index] : emailToIndex)
            indexToEmail[sf.find(index)].insert(email);

        for (const auto& [index, email] : indexToEmail) {
            const String name = accounts[index][0];
            vector<String> row(email);
            row.insert(row.end(), email.begin(), email.end());
            ans.push_back(row);
        }

        return ans;
    };
};

C++

class Solution {
public:
    vector<int> m;

    vector<vector<String>> accountsMerge(vector<vector<String>>& accounts) {
        int n = accounts.size();
        p.resize(n);
        for (int i = 0; i < n; ++i) p[i] = i;
        unordered_map<String, int> emailToIndex;
        for (int i = 0; i < n; ++i) {
            auto account = accounts[i];
            auto name = account[0];
            for (int j = 1; j < account.size(); ++j) {
                String email = account[j];
                if (emailToIndex.count(email))
                    pToEmail[i] = find(emailToIndex[email]);
                else
                    emailToIndex[email] = i;
            }
        }
        unordered_map<int, unordered_set<String>> mp;
        for (int i = 0; i < n; ++i) {
            auto account = accounts[i];
            for (int j = 1; j < account.size(); ++j) {
                String email = account[j];
            }
        }
    }
};

```

```

        mp.find(i).insert(email);
    }

    vector<vector<String>> ans;
    for (int& [i, email] : mp) {
        vector<String> t;
        t.push_back(accounts[i][0]);
        for (String email : email) t.push_back(email);
        sort(t.begin() + 1, t.end());
        ans.push_back(t);
    }

    return ans;
}

int find(int a) {
    if (p[a] != a) {
        p[a] = find(p[a]);
    }
    return p[a];
}

}

Java

class UnionFind {
    public UnionFind(List<List<String>> accounts) {
        for (List<String> account : accounts)
            for (int i = 1; i < account.size(); ++i) {
                final String email = account.get(i);
                id.putIfAbsent(email, email);
            }
    }

    public void union(final String u, final String v) {
        id.putIfAbsent(u, u);
        id.putIfAbsent(v, v);
    }

    public String find(final String u) {
        if (u == id.get(u))
            id.put(u, find(id.get(u)));
        return id.get(u);
    }

    private Map<String, String> id = new HashMap<>();
}

class Solution {
    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        List<List<String>> ans = new ArrayList<>();
    }
}

```

```

Map<String, String> emailToName = new HashMap<>();
Map<String, TreeSet<String>> idEmailToEmails = new HashMap<>();
UnionFind uf = new UnionFind(accounts);

for (final List<String> account : accounts)
    for (int i = 1; i < account.size(); ++i)
        emailToName.putIfAbsent(account.get(i), account.get(0));

for (final List<String> account : accounts)
    for (int i = 2; i < account.size(); ++i)
        uf.union(account.get(i), account.get(1));

for (final List<String> account : accounts)
    for (int i = 1; i < account.size(); ++i) {
        final String id = uf.find(account.get(i));
        idEmailToEmails.putIfAbsent(id, new TreeSet<>());
        idEmailToEmails.get(id).add(account.get(i));
    }

for (final String idEmail : idEmailToEmails.keySet()) {
    List<String> emails = new ArrayList<>(idEmailToEmails.get(idEmail));
    final String name = emailToName.get(idEmail);
    emails.add(0, name);
    ans.add(emails);
}

return ans;
}

Java

class Solution {
    private List<String> p;

    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        int n = accounts.size();
        p = new List<>(n);
        for (int i = 0; i < n; ++i)
            p[i] = i;

        Map<String, Integer> emailToId = new HashMap<>();
        for (int i = 0; i < n; ++i) {
            List<String> account = accounts.get(i);
            String name = account.get(0);
            for (int j = 1; j < account.size(); ++j) {
                String email = account.get(j);
                if (emailToId.containsKey(email)) {
                    pToEmail[i] = find(emailToId.get(email));
                } else {
                    emailToId.put(email, i);
                }
            }
        }

        Map<Integer, Set<String>> mp = new HashMap<>();
        for (int i = 0; i < n; ++i) {
            List<String> account = accounts.get(i);
        }
    }
}

```

```

        for (int j = 1; j < account.size(); ++j) {
            String email = account.get(j);
            mp.computeIfAbsent(find(i), k -> new HashSet<>()).add(email);
        }

        List<List<String>> res = new ArrayList<>();
        for (Map.Entry<Integer, Set<String>> entry : mp.entrySet()) {
            List<String> t = new ArrayList<>();
            t.add(entry.getKey());
            Collections.sort(t);
            t.add(0, accounts.get(entry.getKey()).get(0));
            res.add(t);
        }

        return res;
    }

    private int find(int x) {
        if (p[x] != x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
}

TypeScript

class UnionFind {
    private p: number[];
    private size: number[];

    constructor(n: number) {
        this.p = new Array(n);
        this.size = new Array(n);
        for (let i = 0; i < n; ++i) {
            this.p[i] = i;
            this.size[i] = 1;
        }

        find(x: number): number {
            if (this.p[x] !== x) {
                this.p[x] = this.find(this.p[x]);
            }
            return this.p[x];
        }

        union(a: number, b: number): boolean {
            let pa = this.find(a);
            pb = this.find(b);
            if (pa === pb) {
                return false;
            }
        }
    }
}

```

```

        }
        if (this.size[pa] > this.size[pb]) {
            this.p[pb] = pa;
            this.size[pa] += this.size[pb];
        } else {
            this.p[pa] = pb;
            this.size[pb] += this.size[pa];
        }
        return true;
    }

    function accountsMerge(accounts: string[][]): string[][] {
        const n = accounts.length;
        const uf = new UnionFind(n);
        const d = new Map<string, number>();

        for (let i = 0; i < n; ++i) {
            for (let j = 1; j < accounts[i].length; ++j) {
                const email = accounts[i][j];
                if (d.has(email)) {
                    if (d.get(email) < i)
                        d.set(email, i);
                } else {
                    d.set(email, i);
                }
            }
        }

        const g = new Map<number, Set<string>>();
        for (let i = 0; i < n; ++i) {
            const root = uf.find(i);
            if (!g.has(root)) {
                g.set(root, new Set<string>());
            }
            const emailSet = g.get(root);
            for (let j = 1; j < accounts[i].length; ++j) {
                emailSet.add(accounts[i][j]);
            }
        }

        const ans: string[][] = [];
        for (const [root, emailSet] of g.entries()) {
            const emailList = Array.from(emailSet).sort();
            const mergeCount = [emailList[0], ...emailList];
            ans.push(mergeCount);
        }

        return ans;
    }
}

Go

Go

```

```

type UnionFind struct {
    p, size []int
}

func newUnionFind(n int) UnionFind {
    p := make([]int, n)
    size := make([]int, n)
    for i := range p {
        p[i] = i
        size[i] = 1
    }
    return UnionFind{p, size}
}

func (uf UnionFind) find(x int) int {
    if uf.p[x] != x {
        uf.p[x] = uf.find(uf.p[x])
    }
    return uf.p[x]
}

func (uf UnionFind) union(a, b int) bool {
    pa, pb := uf.find(a), uf.find(b)
    if pa == pb {
        return false
    }

    if uf.size[pa] > uf.size[pb] {
        uf.p[pb] = pa
        uf.size[pa] += uf.size[pb]
    } else {
        uf.p[pa] = pb
        uf.size[pb] += uf.size[pa]
    }
    return true
}

func accountsMerge(accounts [][]string) (ans [][]string) {
    n := len(accounts)
    uf := newUnionFind(n)
    d := make(map[string]int)
    for i := 0; i < n; i++ {
        for j, email := range accounts[i][1:] {
            if j, ok := d[email]; ok {
                uf.union(i, j)
            } else {
                d[email] = i
            }
        }
    }
    g := make(map[int]map[string]struct{})
}

```

```

for i := 0; i < n; i++ {
    root := uf.find(i)
    if ... ok := g[root]; ok {
        g[root] = make(map[string]struct{})
    }
    for _, email := range accounts[i][1:] {
        g[root][email] = struct{}{}
    }
}

for root, s := range g {
    emails := []string{}
    for email := range s {
        emails = append(emails, email)
    }
    sort.Strings(emails)
    account := append([]string{accounts[root][0]}, emails...)
    ans = append(ans, account)
}

return
}

```

[*] DFS — Pattern Solution (stored optimal)

```

Python
# Python solution for Accounts Merge

# Define the UnionFind class for disjoint set operations
class UnionFind:
    def __init__(self):
        self.parent = {}

    # Find with path compression
    def find(self, x):
        if self.parent.setdefault(x, x) != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    # Union two nodes
    def union(self, x, y):
        self.parent[self.find(x)] = self.find(y)

class Solution:
    def accountsMerge(self, accounts):
        uf = UnionFind()
        email_to_name = {}

        # Iterate over each account and union emails
        for account in accounts:
            name = account[0]
            for email in account[1:]:
                uf.union(email, account[0])
                email_to_name[email] = name

        # Group emails by their root parent
        groups = {}
        for email in email_to_name:
            parent = uf.find(email)
            groups.setdefault(parent, []).append(email)

        # Merge the merged account list with sorted emails
        result = []
        for parent, emails in groups.items():
            result.append([email_to_name[parent]] + sorted(emails))
        return result

# Example usage:
# accounts = [{"John", "johnsmith@gmail.com", "john.smith@gmail.com"},
#              {"John", "johnsmith@gmail.com", "johnsmith@gmail.com"},
#              {"Mary", "mary@gmail.com"},
#              {"John", "johnsmith@gmail.com"}]
# solution = Solution()
# print(solution.accountsMerge(accounts))

```

```

first_email = accounts[i]
for email in accounts[i]:
    if email != first_email:
        email_to_name[email] = name

# Group emails by their root parent
groups = {}
for email in email_to_name:
    parent = uf.find(email)
    groups.setdefault(parent, []).append(email)

# Merge the merged account list with sorted emails
result = []
for parent, emails in groups.items():
    result.append([email_to_name[parent]] + sorted(emails))
return result

# Example usage:
# accounts = [{"John", "johnsmith@gmail.com", "john.smith@gmail.com"},
#              {"John", "johnsmith@gmail.com", "johnsmith@gmail.com"},
#              {"Mary", "mary@gmail.com"},
#              {"John", "johnsmith@gmail.com"}]
# solution = Solution()
# print(solution.accountsMerge(accounts))

```

C++

```

// C++ solution for Accounts Merge using Union-Find

#include <vector>
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

class UnionFind {
public:
    unordered_map<string, string> parent;

    // Find with path compression
    string find(string x) {
        if (parent.find(x) == parent.end())
            parent[x] = x;
        if (parent[x] != x)
            parent[x] = find(parent[x]);
        return parent[x];
    }

    // Union two nodes
    void unionSet(string x, string y) {
        parent[find(x)] = find(y);
    }
}

```

```

}

class Solution {
public:
    vector<vector<string>> accountsMerge(vector<vector<string>>& accounts) {
        UnionFind uf;
        unordered_map<string, string> email_to_name;

        // Union emails in the same account
        for (auto& account : accounts) {
            string name = account[0];
            for (int i = 1; i < account.size(); i++) {
                email_to_name[account[i]] = name;
                uf.unionSet(account[i], account[0]);
            }
        }

        // Group emails by their representative
        unordered_map<string, vector<string>> groups;
        for (auto& pair : email_to_name) {
            string email = pair.first;
            string parent = uf.find(email);
            groups[parent].push_back(email);
        }

        // Prepare the final merged accounts
        vector<vector<string>> result;
        for (auto& group : groups) {
            vector<string> emails = group.second;
            sort(emails.begin(), emails.end());
            emails.insert(emails.begin(), email_to_name[group.first]);
            result.push_back(emails);
        }
        return result;
    }
}

// Example usage:
// int main() {
//     vector<vector<string>> accounts = {
//         {"John", "johnsmith@gmail.com", "john.smith@gmail.com"},
//         {"John", "johnsmith@gmail.com", "johnsmith@gmail.com"},
//         {"Mary", "mary@gmail.com"},
//         {"John", "johnsmith@gmail.com"}
//     };
//     Solution sol;
//     vector<vector<string>> merged = sol.accountsMerge(accounts);
//     // Output merged accounts...
// }

```

#785 MEDIUM Is Graph Bipartite?

LeetCode · Simple Interview · Medium · LeetCode · Editorial

DFS · BFS · Union Find · Graph

Like Denny Zhang

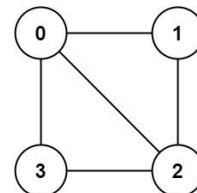
Problem:

There is an undirected graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node u is adjacent to. More formally, for each v in `graph[u]`, there is an undirected edge between node u and node v . The graph has the following properties:

- There are no self-edges (`graph[u]` does not contain u).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If v is in `graph[u]`, then u is in `graph[v]` (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B . Return `true` if and only if it is bipartite.

Example 1:

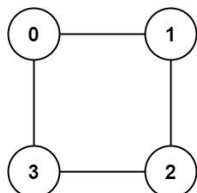


Input: `graph = [[1,2,3],[0,2],[0,1,3],[0,2]]`

Output: `false`

Explanation: There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.

Example 2:



Input: `graph = [[1,3],[0,2],[1,3],[0,2]]`

Output: `true`

Explanation: We can partition the nodes into two sets: $\{0, 2\}$ and $\{1, 3\}$.

Constraints:

- `graph.length == n`
- `1 <= n <= 100`
- `0 <= graph[u].length < n`
- `0 <= graph[u][i] < n - 1`
- `graph[u]` does not contain u .
- All the values of `graph[u]` are unique.
- If `graph[u]` contains v , then `graph[v]` contains u .

>> Brute Force (DFS) Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def isBipartite(self, graph):
        # Union-Find approach: for every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for nb in graph.get(node, []):
                if nb not in visited:
                    if node not in visited:
                        dfs(nb)
                    count += 1
            return count

        # Iterate over each node
        for node in graph:
            if node not in visited:
                count = dfs(node)
                if count % 2 != 0:
                    return False
        return True

```

Python

```

# Using Breadth-First Search (BFS)
from collections import deque

def isBipartite(graph):
    n = len(graph)
    # color array: -1 means uncolored, 0 and 1 are the two colors.
    colors = [-1] * n

    # Iterate through each node to handle disconnected components
    for i in range(n):
        if colors[i] == -1:
            # Start BFS from node i with color 0
            queue = deque([i])
            colors[i] = 0
            while queue:
                node = queue.popleft()
                # Check neighbors of the current node
                for neighbor in graph[node]:
                    if colors[neighbor] == -1:
                        # Assign the opposite color to the neighbor
                        colors[neighbor] = 1 - colors[node]
                        queue.append(neighbor)
                    elif colors[neighbor] == colors[node]:
                        # Found a neighbor with the same color: not bipartite
                        return False
            return True

# Example usage:
if __name__ == '__main__':
    graph_example = [[1,3],[0,2],[1,3],[0,2]]
    print(isBipartite(graph_example)) # Expected output: True

```

Python

```

from enum import Enum

class Color(Enum):
    WHITE = 0
    RED = 1
    GREEN = 2

class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        colors = {Color.WHITE} * len(graph)

        for i in range(len(graph)):
            # This node has been colored, so do nothing.
            if colors[i] != Color.WHITE:
                continue
            # Always point red for a white node.
            colors[i] = Color.RED
            n = i
            q = collections.deque([i])
            while q:
                for _ in range(len(q)):
                    u = q.popleft()
                    for v in graph[u]:
                        if colors[v] == colors[u]:
                            return False
                        if colors[v] == Color.WHITE:
                            colors[v] = Color.GREEN if colors[u] == Color.GREEN else Color.GREEN
                            q.append(v)
            return True

```

Python

```

class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(i: int, c: int) -> bool:
            color[i] = c
            for k in graph[i]:
                if color[k] == c or (color[k] == 0 and not dfs(k, -c)):
                    return False
            return True

        n = len(graph)
        color = [0] * n
        for i in range(n):
            if color[i] == 0 and not dfs(i, 1):
                return False
        return True

```

Python

```

class Solution {
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(i, c):
            color[i] = c
            for v in graph[i]:
                if not color[v]:
                    if not dfs(v, 3 - c):
                        return False
                    elif color[v] == c:
                        return False
            return True

        n = len(graph)
        color = [0] * n
        for i in range(n):
            if not color[i] and not dfs(i, 1):
                return False
        return True

```

C++

```

enum class Color { WHITE, RED, GREEN };

class Solution {
public:
    bool isBipartite(vector<vector<int>>& graph) {
        vector<Color> colors(graph.size(), Color::WHITE);
        for (int i = 0; i < graph.size(); ++i) {
            // This node has been colored, so do nothing.
            if (colors[i] != Color::WHITE)
                continue;
            // Always point red for a white node.
            colors[i] = Color::RED;
            queue<int> q;
            while (!q.empty()) {
                for (int sz = q.size(); sz > 0; --sz) {
                    const int u = q.front();
                    q.pop();
                    for (const int v : graph[u]) {
                        if (colors[v] == colors[u])
                            return false;
                        if (colors[v] == Color::WHITE) {
                            colors[v] = Color::RED if colors[u] == Color::RED : Color::GREEN;
                            q.push(v);
                        }
                    }
                }
            }
            return true;
        }
    };
};

```

C++

```

class Solution {
public:
    bool isBipartite(vector<vector<int>>& graph) {
        int n = graph.size();
        vector<int> color(n);
        for (int i = 0; i < n; ++i)
            if (color[i] && dfs(i, 1, color, graph))
                return false;
        return true;
    }

    bool dfs(int u, int c, vector<int>& color, vector<vector<int>>& g) {
        color[u] = c;
        for (int v : g[u]) {
            if (color[v] == c)
                if (dfs(v, 3 - c, color, g)) return false;
            } else if (color[v] == 0)
                return false;
        }
        return true;
    }
};

```

C++

```

class Solution {
public:
    bool isBipartite(vector<vector<int>>& graph) {
        int n = graph.size();
        vector<int> color(n);
        for (int i = 0; i < n; ++i)
            if (color[i] && dfs(i, 1, color, graph))
                return false;
        return true;
    }

    bool dfs(int u, int c, vector<int>& color, vector<vector<int>>& g) {
        color[u] = c;
        for (int v : g[u]) {
            if (color[v] == c)
                if (dfs(v, 3 - c, color, g)) return false;
            } else if (color[v] == 0)
                return false;
        }
        return true;
    }
};

```

Java

Java

```

enum Color { WHITE, RED, GREEN };

class Solution {
    public boolean isBipartite(int[][] graph) {
        Color[] colors = new Color[graph.length];
        Arrays.fill(colors, Color.WHITE);
        for (int i = 0; i < graph.length; ++i)
            if (colors[i] == Color.WHITE && isValidColor(graph, i, colors, Color.RED))
                return false;
        return true;
    }

    private boolean isValidColor(int[][] graph, int u, Color[] colors, Color color) {
        // The wanted color should be same as 'color'.
        if (colors[u] != Color.WHITE)
            return colors[u] == color;
        colors[u] = color;
        for (final int v : graph[u])
            if (isValidColor(graph, v, colors, color == Color.RED ? Color.GREEN : Color.RED))
                return false;
        return true;
    }
}

```

Java

```

class Solution {
    private int[] color;
    private int[][] g;

    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        color = new int[n];
        g = graph;
        for (int i = 0; i < n; ++i) {
            if (color[i] == 0 && dfs(i, 1)) {
                return false;
            }
        }
        return true;
    }

    private boolean dfs(int u, int c) {
        color[u] = c;
        for (int v : g[u]) {
            if (color[v] == c || (color[v] == 0 && dfs(v, -c))) {
                return false;
            }
        }
        return true;
    }
}

```

Java

Java

```

class Solution {
    private int[] p;

    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        p = new int[n];
        for (int i = 0; i < n; ++i) {
            p[i] = -1;
        }
        for (int a = 0; a < n; ++a) {
            for (int b : graph[a]) {
                int pa = find(a), pb = find(b);
                if (pa == pb) {
                    return false;
                }
                p[pb] = find(graph[a][0]);
            }
        }
        return true;
    }

    private int find(int x) {
        if (p[x] == x) {
            p[x] = find(p[x]);
        }
        return p[x];
    }
}

```

Java

```

class Solution {
    private int[] color;
    private int[][] g;

    public boolean isBipartite(int[][] graph) {
        int n = graph.length;
        color = new int[n];
        g = graph;
        for (int i = 0; i < n; ++i) {
            if (color[i] == 0 && dfs(i, 1)) {
                return false;
            }
        }
        return true;
    }

    private boolean dfs(int u, int c) {
        color[u] = c;
        for (int v : g[u]) {
            if (color[v] == u) {
                if (dfs(v, 3 - c)) {
                    return false;
                }
            } else if (color[v] == c) {
                return false;
            }
        }
        return true;
    }
}

```

TypeScript

TypeScript

```

function isBipartite(graph: number[][]): boolean {
    const n = graph.length;
    const color: number[] = Array(n).fill(0);
    const dfs = (u: number, c: number): boolean => {
        color[u] = c;
        for (const b of graph[u]) {
            if (color[b] === c || (color[b] === 0 && dfs(b, -c))) {
                return false;
            }
        }
        return true;
    };
    for (let i = 0; i < n; i++) {
        if (color[i] === 0 && dfs(i, 1)) {
            return false;
        }
    }
    return true;
}

```

TypeScript

```

function isBipartite(graph: number[][]): boolean {
    const n = graph.length;
    let valid = true;
    let colors = new Array(n).fill(0);
    function dfs(n: number, color: number, graph: number[][]): boolean {
        colors[n] = color;
        const nextColor = 1 - color;
        for (let i of graph[n]) {
            if (colors[i]) {
                if (colors[i] === color) return false;
            } else if (colors[i] !== nextColor) {
                valid = false;
                return;
            }
        }
        for (let i = 0; i < n && valid; i++) {
            if (colors[i]) {
                dfs(i, 1, graph);
            }
        }
        return valid;
    }
}

```

Go

```

func isBipartite(graph [][]int) bool {
    n := len(graph)
    color := make([]int, n)
    var dfs func(c int) bool
    dfs = func(c int) bool {
        color[c] = c
        for _, b := range graph[c] {
            if color[b] == c || (color[b] == 0 && dfs(b, -c)) {
                return false
            }
        }
        return true
    }
    for i := range graph {
        if color[i] == 0 && !dfs(i, 1) {
            return false
        }
    }
    return true
}

```

Go

```

func isBipartite(graph [][]int) bool {
    n := len(graph)
    color := make([]int, n)
    var dfs func(c int) bool
    dfs = func(c int) bool {
        color[c] = c
        for _, b := range graph[c] {
            if color[b] == c {
                return false
            }
            if color[b] == 0 {
                if !dfs(b, -c) {
                    return false
                }
            }
        }
        return true
    }
    for i := range graph {
        if color[i] == 0 && !dfs(i, 1) {
            return false
        }
    }
    return true
}

```

Rust

```

fn is_bipartite(graph: Vec<Vec<i32>>,) -> bool {
    let n = graph.len();
    let mut color = vec![0; n];

    fn dfs(u: usize, c: i32, graph: Vec<Vec<i32>>,) -> bool {
        color[u] = c;
        for &b in graph[u] {
            if color[*b] == c {
                return false;
            }
            if color[*b] == 0 && !dfs(*b, -c, graph, color) {
                return false;
            }
        }
        true
    }

    for i in 0..n {
        if color[i] == 0 && !dfs(i, 1, graph, color) {
            return false;
        }
    }
    true
}

```

Rust

```

impl Solution {
    pub fn is_bipartite(graph: Vec<Vec<i32>>,) -> bool {
        let n = graph.len();
        let mut color = vec![0; n];

        fn dfs(u: usize, c: i32, graph: Vec<Vec<i32>>,) -> bool {
            color[u] = c;
            for &b in graph[u] {
                if color[*b] == c {
                    return false;
                }
                if color[*b] == 0 && !dfs(*b, -c, graph, color) {
                    return false;
                }
            }
            true
        }

        for i in 0..n {
            if color[i] == 0 && !dfs(i, 1, graph, color) {
                return false;
            }
        }
        true
    }
}

```

Rust

```

impl Solution {
    #![allow(dead_code)]
    pub fn is_bipartite(graph: Vec<Vec<i32>>,) -> bool {
        let mut graph = graph;
        let n = graph.len();
        let mut color_vec = vec![0; n];
        for i in 0..n {
            if color_vec[i] == 0 && !dfs(i, 1, &mut color_vec, &mut graph) {
                return false;
            }
        }
        true
    }

    #![allow(dead_code)]
    fn dfs(
        u: usize,
        color_vec: &mut Vec<usize>,
        graph: &mut Vec<Vec<i32>>,)
        -> bool {
        color_vec[u] = color;
        for &b in graph[u].clone() {
            if color_vec[*b] == color {
                return false;
            }
            // This color hasn't been colored
        }
    }
}

```

```

if !dfs(traverse(n as usize, 1 - color, color_vec, graph)) {
    return false;
} else if color_vec[n as usize] == color {
    // The color is the same
    return false;
}
true
}
}

```

[*] DFS — Pattern Solution (matched from community answers)

Python

```

class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(i: int, c: int) -> bool:
            color[i] = c
            for b in graph[i]:
                if color[b] == c or (color[b] == 0 and not dfs(b, -c)):
                    return False
            return True

        n = len(graph)
        color = [0] * n
        for i in range(n):
            if color[i] == 0 and not dfs(i, 1):
                return False
        return True

```

#1236 MEDIUM

Web Crawler

LeetCode · Simple · WalkCC · LeetCode · Editorial

String · DFS · BFS · Interactive

LeetCode Premium 98

Problem:

Given a url startUrl and an interface HtmlParser, implement a web crawler to crawl all links that are under the same hostname as startUrl.

Return all urls obtained by your web crawler in any order.

Your crawler should:

- Start from the page: startUrl
- Call HtmlParser.getUrls(url) to get all urls from a webpage of given url.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as startUrl.

http://example.org:8888/foo/bar#bang

hostname

As shown in the example url above, the hostname is example.org. For simplicity sake, you may assume all urls use http protocol without any port specified. For example, the urls http://leetcode.com/problems and http://leetcode.com/context are under the same hostname, while urls http://example.org/test and http://example.com/abc are not under the same hostname. The HtmlParser interface is defined as such:

```

interface HtmlParser {
    // Return a list of all urls from a webpage of given url.
    public List<String> getUrls(String url);
}

```

Below are two examples explaining the functionality of the problem, for custom testing purposes you'll have three variables urls, edges and startUrl. Notice that you will only have access to startUrl in your code, while urls and edges are not directly accessible to you in code.

Note: Consider the same URL with the trailing slash "/" as a different URL. For example, "http://news.yahoo.com", and "http://news.yahoo.com/" are different urls.

Example 1:

```

startUrl
http://news.yahoo.com/news/topics/
http://news.yahoo.com/
http://news.yahoo.com/us
http://news.google.com
http://news.yahoo.com/news

```

different hostname

different hostname

different hostname

Input:

```

urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com"
]
edges = [[0,1],[1,2],[1,3],[2,3],[3,0]]
startUrl = "http://news.yahoo.com"
Output: ["http://news.yahoo.com", "http://news.yahoo.com/news", "http://news.yahoo.com/news/topics/"]
Explanation: The startUrl links to all other pages that do not share the same hostname.

```

Constraints:

- 1 <= urls.length <= 1000

```

"http://news.google.com",
"http://news.yahoo.com/us"
]
edges = [[0,1],[1,2],[1,3],[2,3],[3,0]]
startUrl = "http://news.yahoo.com/news/topics/"
Output: [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.yahoo.com/us"
]

```

Example 2:

```

startUrl
http://news.google.com
http://news.yahoo.com/news/topics/
http://news.yahoo.com/news
http://news.yahoo.com/

```

different hostname

different hostname

different hostname

Input:

```

urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
startUrl = "http://news.google.com"
Output: ["http://news.google.com"]
Explanation: The startUrl links to all other pages that do not share the same hostname.

```

Constraints:

- 1 <= urls.length <= 1000

- `1 <= url[i].length <= 300`
- `startUrl` is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').
- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in url library.

```
Python
Python
# Python Code Solution
class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> list[str]:
        from collections import deque

        # Return hostname and ordered hostnames from a url string.
        def get_hostname(url: str) -> str:
            # Remove protocol prefix "http://"
            hostname = url.split("://")[1]
            # The hostname is the part before the first "/" (if exists)
            return hostname.split("/")[0]

        # Extract the hostname from the starting URL.
        start_hostname = get_hostname(startUrl)

        # Initialize a set to keep track of visited urls and a queue for BFS.
        visited = set([startUrl])
        queue = deque([startUrl])

        # Perform BFS traversal of the web links.
        while queue:
            current_url = queue.popleft()
            # Extract and store domain of the current URL.
            for url in htmlParser.getUrls(current_url):
                # Check if URL is under the same hostname and has not been visited.
                if url not in visited and get_hostname(url) == start_hostname:
                    visited.add(url)
                    queue.append(url)

        # Return all visited URLs as a list.
        return list(visited)

Python
```

```
# ---
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# ---
# class HtmlParser(object):
#     def getUrls(self, url):
#         """
#         @param url: a url
#         @return list[str]: all urls from url
#         """
# ---

class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> list[str]:
        q = collections.deque([startUrl])
        seen = {startUrl}
        hostname = startUrl.split('/')[2]

        while q:
            curUrl = q.popleft()
            for url in htmlParser.getUrls(curUrl):
                if url in seen:
                    continue
                if hostname in url:
                    q.append(url)
                    seen.add(url)

        return seen
```

```
Python
# ---
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# ---
# class HtmlParser(object):
#     def getUrls(self, url):
#         """
#         @param url: a url
#         @return list[str]: all urls from url
#         """
# ---

class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> list[str]:
        def host(url):
            url = url[7:]
            return url.split('/')[2]

        def dfs(url):
            if url in ans:
                return
            ans.add(url)
            for next in htmlParser.getUrls(url):
                if host(url) == host(next):
                    dfs(next)

        ans = set()
        dfs(startUrl)
        return list(ans)

Python
```

```
# ---
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# ---
# class HtmlParser(object):
#     def getUrls(self, url):
#         """
#         @param url: a url
#         @return list[str]: all urls from url
#         """
# ---

from collections import deque

class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> list[str]:
        visited = set()
        domain = self.get_domain(startUrl)
        queue = deque([startUrl])
        visited.add(startUrl)

        while queue:
            current_url = queue.popleft()
            for url in htmlParser.getUrls(current_url):
                if url not in visited and self.get_domain(url) == domain:
                    visited.add(url)
                    queue.append(url)

        return list(visited)

    def get_domain(self, url: str) -> str:
        return url.split('/')[2]
```

C++

```
// ---
// This is the HtmlParser's API interface.
// You should not implement it, or speculate about its implementation
// ---
// class HtmlParser {
//     public:
//         vector<string> getUrls(string url);
// };
// ---

class Solution {
public:
    vector<string> crawl(string startUrl, HtmlParser htmlParser) {
        unordered_set<string> seen{startUrl};
        const string hostname = get_hostname(startUrl);

        while (!empty()) {
            const string curUrl = q.front();
            q.pop();
            for (const string& url : htmlParser.getUrls(curUrl)) {
                if (seen.contains(url))
                    continue;
                if (url.find(hostname) != string::npos) {
                    q.push(url);
                    seen.insert(url);
                }
            }
        }
        return {seen.begin(), seen.end()};
    }

private:
    string get_hostname(const string& url) {
        const int firstSlash = url.find_first_of('/');
        const int thirdSlash = url.find_first_of('/', firstSlash + 2);
        return url.substr(firstSlash + 2, thirdSlash - firstSlash - 2);
    }
};

C++
```

```
// ---
// This is the HtmlParser's API interface.
// You should not implement it, or speculate about its implementation
// ---
// class HtmlParser {
//     public:
//         vector<string> getUrls(string url);
// };
// ---

class Solution {
public:
    vector<string> ans;
    unordered_set<string> vis;

    vector<string> crawl(string startUrl, HtmlParser htmlParser) {
        dfs(startUrl, htmlParser);
        return ans;
    }

    void dfs(string url, HtmlParser& htmlParser) {
        if (vis.contains(url)) return;
        vis.insert(url);
        ans.push_back(url);
        for (string next : htmlParser.getUrls(url)) {
            if (host(url) == host(next))
                dfs(next, htmlParser);
        }

        string host(string url) {
            int i = 7;
            string res;
            for (; i < url.size(); ++i) {
                if (url[i] == '/') break;
                res += url[i];
            }
            return res;
        }
    };
};

Java
```

```
// ---
// This is the HtmlParser's API interface.
// You should not implement it, or speculate about its implementation
// ---
// interface HtmlParser {
//     public List<String> getUrls(String url);
// }
// ---

class Solution {
    private Set<String> ans;

    public List<String> crawl(String startUrl, HtmlParser htmlParser) {
        ans = new HashSet<>();
        dfs(startUrl, htmlParser);
        return new ArrayList<>(ans);
    }

    private void dfs(String url, HtmlParser htmlParser) {
        if (ans.contains(url)) {
            return;
        }
        ans.add(url);
        for (String next : htmlParser.getUrls(url)) {
            if (host(next).equals(host(url))) {
                dfs(next, htmlParser);
            }
        }
    }

    private String host(String url) {
        url = url.substring(7);
        return url.split("/")[2];
    }
}

Java
```

```

public class WebCrawler {
    class Solution_Dfs {
        Set<String> res = new HashSet<>(); // 结果

        public List<String> crawl(String startUrl, HtmlParser htmlParser) {
            String host = getHost(startUrl); // 主机
            res.add(startUrl); // 加入结果
            dfs(startUrl, host, htmlParser); // dfs
            return new ArrayList<>(res);
        }

        void dfs(String startUrl, String host, HtmlParser htmlParser) {
            List<String> urIs = htmlParser.getUris(startUrl);
            for (String url : urIs) {
                // 如果已经访问过，就跳过
                if (res.contains(url) || !getHost(url).equals(host)) {
                    continue;
                }
                // 加入结果
                res.add(url);
                // dfs
                dfs(url, host, htmlParser);
            }
        }

        private String getHost(String url) {
            String[] args = url.split("/");
            return args[2];
        }
    }
}

class Solution_Bfs {
    Set<String> res = new HashSet<>(); // 结果

    public List<String> crawl(String startUrl, HtmlParser htmlParser) {
        String host = getHost(startUrl); // 主机
        Queue<String> q = new LinkedList<>(); // bfs queue
        res.add(startUrl); // 加入结果
        q.offer(startUrl); // 加入队列
        while (q.size() > 0) {
            String url = q.poll(); // 取出url
            List<String> urIs = htmlParser.getUris(url);
            for (String u : urIs) { // 遍历url
                // 如果已经访问过，就跳过
                if (res.contains(u) || !getHost(u).equals(host)) {
                    continue;
                }
                res.add(u);
                q.offer(u);
            }
        }
        return new ArrayList<>(res);
    }
}

```

```

        if (res.contains(s) || !getHost(s).equals(host)) {
            continue;
        }
        res.add(s); // 加入结果
        q.offer(s); // 加入队列
    }
    return new ArrayList<>(res);
}

private String getHost(String url) {
    String[] args = url.split("/");
    return args[2];
}

interface HtmlParser {
    // Return a list of all urls from a webpage of given url.
    // This is a blocking call, that means it will do HTTP request and return when this request is finished.
    List<String> getUris(String url);
}

```

```

/**
 * This is HtmlParser's API interface.
 * You should not implement it, or speculate about its implementation
 */
interface HtmlParser {
    // Return a list of all urls from a webpage of given url.
    // This is a blocking call, that means it will do HTTP request and return when this request is finished.
    List<String> getUris(String url);
}

func crawl(startUrl string, HtmlParser HtmlParser) []string {
    var ans []string
    vis := make(map[string]bool)
    var dfs func(url string)
    host = getHost(startUrl)
    return strings.Split(crawl(startUrl, host, dfs), "\n")
}

func dfs(url string) {
    if vis[url] {
        return
    }
    vis[url] = true
    ans = append(ans, url)
    for _, next := range HtmlParser.GetUris(url) {
        if !getHost(next).equals(host) {
            continue
        }
        dfs(next)
    }
}

dfs(startUrl)
return ans
}

```

```

/**
 * This is HtmlParser's API interface.
 * You should not implement it, or speculate about its implementation
 */
interface HtmlParser {
    // Return a list of all urls from a webpage of given url.
    // This is a blocking call, that means it will do HTTP request and return when this request is finished.
    List<String> getUris(String url);
}

func crawl(startUrl string, HtmlParser HtmlParser) []string {
    var ans []string
    vis := make(map[string]bool)
    var dfs func(url string)
    host = getHost(startUrl)
    return strings.Split(crawl(startUrl, host, dfs), "\n")
}

func dfs(url string) {
    if vis[url] {
        return
    }
    vis[url] = true
    ans = append(ans, url)
    for _, next := range HtmlParser.GetUris(url) {
        if !getHost(next).equals(host) {
            continue
        }
        dfs(next)
    }
}

dfs(startUrl)
return ans
}

```

```

}
dfs(startUrl)
return ans
}

```

[?] DFS - Pattern Solution (matched from community answers)

```

Python
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# class HtmlParser(object):
#     def getUris(self, url):
#         ...
# type url: str
# return List[str]

class Solution:
    def crawl(self, startUrl: str, HtmlParser: 'HtmlParser') -> List[str]:
        def host(url):
            url = url.split("/")
            return url[2]

        def dfs(url):
            if url in ans:
                return
            ans.add(url)
            for next in HtmlParser.getUris(url):
                if host(next) == host(url):
                    dfs(next)

        ans = set()
        dfs(startUrl)
        return list(ans)

```

#1242 MEDIUM

Web Crawler Multithreaded

LeetCode · Simple Python · Walkthrough · LeetCode · Editorial

DPS · DFS · Concurrency

List: Denny Zhang

Problem:

Given a URL startUrl and an interface HtmlParser, implement a Multi-threaded web crawler to crawl all links that are under the same hostname as startUrl.

Return all URLs obtained by your web crawler in any order.

Your crawler should:

- Start from the page: startUrl

- Call HtmlParser.getUris(url) to get all URLs from a webpage of a given URL.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as startUrl.

http://example.org:8888/foo/bar#bang

hostname

As shown in the example URL above, the hostname is example.org. For simplicity's sake, you may assume all URLs use HTTP protocol without any port specified. For example, the URLs http://leetcode.com/problems and http://leetcode.com/contest are under the same hostname, while URLs http://example.org/test and http://example.com/abc are not under the same hostname.

The HtmlParser interface is defined as such:

```

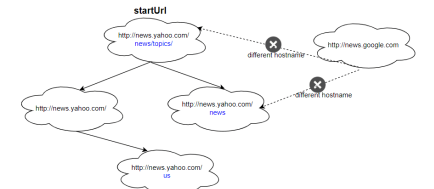
interface HtmlParser {
    // Return a list of all urls from a webpage of given url.
    // This is a blocking call, that means it will do HTTP request and return when this request is finished.
    public List<String> getUris(String url);
}

```

Note that getUris(String url) simulates performing an HTTP request. You can treat it as a blocking function call that waits for an HTTP request to finish. It is guaranteed that getUris(String url) will return the URLs within 15ms. Single-threaded solutions will exceed the time limit so, can your multi-threaded web crawler do better?

Below are two examples explaining the functionality of the problem. For custom testing purposes, you'll have three variables urls, edges and startUrl. Notice that you will only have access to startUrl in your code, while urls and edges are not directly accessible to you in code.

Example 1:



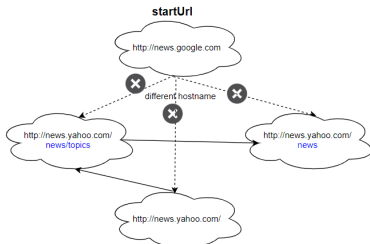
Input:

```

urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com",
    "http://news.yahoo.com/us"
]
edges = [[2,8],[2,1],[3,2],[3,1],[8,4]]
startUrl = "http://news.yahoo.com/news/topics/"
Output: [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.yahoo.com/us"
]

```

Example 2:



Input:
urls = [
 "http://news.yahoo.com",
 "http://news.yahoo.com/news",
 "http://news.yahoo.com/news/topics/",
 "http://news.google.com"
]
edges = [(0,2),(2,1),(3,2),(3,1),(3,0)]
startUrl = "http://news.google.com"
Output: ["http://news.google.com"]
Explanation: The startUrl links to all other pages that do not share the same hostname.

Constraints:

- 1 <= urls.length <= 1000
- 1 <= urls[i].length <= 300
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').
- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in the URL library.

Follow up:

1. Assume we have 10,000 nodes and 1 billion URLs to crawl. We will deploy the same software onto each node. The software can know about all the nodes. We have to minimize communication between machines and make sure each node does equal amount of work. How would your web crawler design change?

2. What if one node fails or does not work?
3. How do you know when the crawler is done?

```
>> Brute Force (DFS) Interview Prep

Python
# Brute Force Approach
class Solution:
    def crawl(self, startUrl, htmlParser):
        # Brute Force - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for url in htmlParser.getUrls(node):
                if url not in visited: dfs(url)
            for node in range(1):
                if node not in visited:
                    dfs(node); count += 1
            return count

        return count

Python
import threading
from queue import Queue
from urllib.parse import urlparse

def crawl(startUrl, htmlParser):
    # Extract hostname of the startUrl
    start_host = urlparse(startUrl).netloc
    # Thread-safe set for visited URLs and a lock for safety
    visited = set()
    visited_lock = threading.Lock()

    # Queue to process URLs, initialized with startUrl
    url_queue = Queue()
    url_queue.put(startUrl)

    def worker():
        while True:
            current_url = url_queue.get(timeout=1) # timeout to avoid blocking indefinitely
            try:
                # Extract hostname of the current url using htmlParser
                host_name = urlparse(current_url).netloc
                for next_url in htmlParser.getUrls(current_url):
                    # Check if next url shares the same hostname as startUrl
                    if urlparse(next_url).netloc == start_host:
                        url_queue.put(next_url)
            except:
                break

    threads = []
    for _ in range(10): # 10 threads as an example
        thread = threading.Thread(target=worker)
        thread.start()
        threads.append(thread)

    # Wait for all threads to finish
    for thread in threads:
        thread.join()

    return list(visited)
```

```
with visited_lock:
    if next_url not in visited:
        visited.add(next_url)
        url_queue.put(next_url)

# Main crawler as visited
with visited_lock:
    visited.add(startUrl)

# Launch worker threads (8 threads used as example)
threads = []
for i in range(8):
    t = threading.Thread(target=worker)
    t.start()
    threads.append(t)

# Wait until all URLs have been processed
url_queue.join()

# Ensure all threads terminate
for t in threads:
    t.join()

return list(visited)

Python
from threading import Thread, Lock
from urllib.parse import urlparse # To parse URLs and extract hostname for comparison
from typing import List

class Solution:
    def crawl(self, startUrl: str, htmlParser: "HtmlParser") -> List[str]:
        hostname = urlparse(startUrl).hostname
        seen = set([startUrl]) # Keep track of seen URLs to avoid re-crawling the same URL
        lock = Lock() # Lock for thread-safe operations on the seen set

        def dfs(url):
            for next_url in htmlParser.getUrls(url):
                if urlparse(next_url).hostname == hostname and next_url not in seen:
                    with lock: # Ensure thread-safe write to the seen set
                        if next_url not in seen:
                            seen.add(next_url)
                            dfs(next_url)

        def worker():
            while True:
                with lock:
                    if not unseen:
                        return
                    url = unseen.pop()
                    dfs(url)

        threads = []
        for _ in range(8):
            thread = Thread(target=worker)
            thread.start()
            threads.append(thread)

        for thread in threads:
            thread.join()

        return list(seen)
```

```
unseen = [startUrl] # Stack of URLs to be crawled
threads = []
num_threads = 10 # For any number you deem appropriate based on the problem constraints

# Start threads
for _ in range(num_threads):
    thread = Thread(target=worker)
    thread.start()
    threads.append(thread)

# Wait for all threads to finish
for thread in threads:
    thread.join()

return list(unseen)

Python
# reference: https://leetcode.com/problems/crawl-for-http-pages-1837/solution/

class Crawler(threading.Thread):
    def __init__(self, base_url, links_to_crawl, have_visited, error_links, url_lock):
        threading.Thread.__init__(self)
        print(f"Web Crawler worker {threading.current_thread()} has Started")
        self.base_url = base_url
        self.links_to_crawl = links_to_crawl
        self.have_visited = have_visited
        self.error_links = error_links
        self.url_lock = url_lock

    def run(self):
        # We create a ssl context so that our script can crawl
        # the https sites with ssl handshake.

        # Create a SSLContext object with default settings.
        my_ssl = ssl.create_default_context()

        # By default when creating a default ssl context and making ssl handshake
        # we verify the hostname with the certificate but our objective is to crawl
        # the webpage so we will not be checking the validity of the certificate.
        my_ssl.check_hostname = False
```

```
# In this case we are not verifying the certificate and my
# certificate is accepted in this mode
my_ssl.verify_mode = ssl.CERT_NONE

# We are defining an infinite while loop so that all the links in our
# queue are processed.

while True:
    # In this part of the code we create a global lock on our queue of
    # links so that all our threads can access the queue at same time
    self.url_lock.acquire()
    print(f"Queue Size: {len(self.links_to_crawl.queue())}")
    link = self.links_to_crawl.get()
    self.url_lock.release()

    # If the link is None the queue is exhausted or the threads are yet
    # to process the links.

    if link is None:
        break

    # If the link is already visited we break the execution.
    if link in self.have_visited:
        break

    # This method constructs a full "absolute" URL by combining the
    # base url with other url. This one component of the base URL,
    # is particularly the addressing scheme, the network
    # location and the port, to provide missing components
    # in the relative URL.

    # In short we reconstruct the relative url if it is broken.
    link = urljoin(self.base_url, link)

    # We use the header parameter to "spoof" the "User-Agent" header
    # value which is used by the browser to identify itself. This is
    # because some servers will allow the connection if it comes
    # from a verified browser. In this case we are using Firefox header.
    req = Request(link, headers={"User-Agent": "Mozilla/5.0"})

    # We are opening the url using a ssl handshake.
    response = urlopen(req, context=my_ssl)

    print(f"The URL {response.geturl()} crawled with \
status {response.getcode()}")

# This returns the html representation of the webpage
soup = BeautifulSoup(response.read(), "html.parser")

# In this case we are finding all the links in the page.
for a_tag in soup.find_all('a'):
    # We are checking if the link is already visited and (network location part) is
```

```
Python
import multiprocessing
from multiprocessing.pool import Pool
from queue import Queue, Empty
from concurrent.futures import ThreadPoolExecutor
from urllib.parse import urljoin, urlparse
import requests

# ThreadPoolExecutor
class MultiThreadCrawler:
    def __init__(self, seed_url):
        self.seed_url = seed_url
        self.root_url = urlparse(seed_url).scheme
        self.pool = ThreadPoolExecutor(max_workers=5)
        self.scraped_pages = set()
        self.crawl_queue = Queue()
        self.crawl_queue.put(self.seed_url)

    def parse_links(self, html):
        soup = BeautifulSoup(html, "html.parser")
        AnchorTags = soup.find_all('a', href=True)
        for link in AnchorTags:
            url = link.get('href')
            if url.startswith('/') or url.startswith(self.root_url):
                url = urljoin(self.root_url, url)
            if url not in self.scraped_pages:
                self.crawl_queue.put(url)

    def scrape_info(self, url):
        soup = BeautifulSoup(url, "html5lib")
        web_page_paragraph_contents = soup.get('p')
        text = ""
        for para in web_page_paragraph_contents:
            if not (isinstance(para, str) or isinstance(para, tuple)):
                text = text + str(para.text) + '\n'
        print(f"Text Present in The webpage is ->{text}, text, '{text}'")
        return text

    def post_scrape_callback(self, res):
        result = res.result()
        if result and result.status_code == 200:
            self.parse_links(result.text)
            self.scrape_info(result.text)

    def scrape_page(self, url):
        try:
            res = requests.get(url, timeout=(3, 30))
            return res
        except requests.RequestException:
```



```

        return

def run_web_crawler(urls):
    while True:
        try:
            print(f"Name of the current executing process: ",
                  multiprocessing.current_process().name, "\n")
            target_url = self.crawl_queue.get(timeout=5)
            if target_url not in self.scraped_pages:
                print(f"Scraping URL: {target_url}")
                self.current_scraping_url = f"{target_url}"
                self.scraped_pages.add(target_url)
                job = self.pool.submit(self.scrape_page, target_url)
                job.add_done_callback(self.post_scrape_callback)

        except Empty:
            return
        except Exception as e:
            print(e)
            continue

def info(urls):
    print(f"Seed URL is: ", self.seed_url, "\n")
    print(f"Scraped pages are: ", self.scraped_pages, "\n")

if __name__ == "__main__":
    cc = MultithreadedCrawler("https://www.lastcode.ca")
    cc.run_web_crawler()
    cc.info()

```

C++
C++

```

//
// This is the HtmlParser's API interface.
// You should not implement it, or speculate about its implementation
//
class HtmlParser {
public:
    vector<string> getUrls(string url);
};

//
class Solution {
public:
    vector<string> crawl(string startUrl, HtmlParser htmlParser) {
        queue<string> q{startUrl};
        unordered_set<string> seen{startUrl};
        const string hostname = getHostname(startUrl);
        const int nThreads = std::thread::hardware_concurrency();
        vector<thread> threads;
        std::mutex mtx;
        std::condition_variable cv;

        auto t = [&]() {
            while (true) {
                unique_lock<mutex> lock(mtx);
                cv.wait_for(lock, 30ms, [&]() { return q.size() > 0; });
                if (q.empty())
                    return;

                auto cur = q.front();
                q.pop();
                lock.unlock();
                const vector<string> urIs = htmlParser.getUrls(cur);
                lock.lock();
                for (const string& url : urIs) {
                    if (seen.contains(url))
                        continue;
                    if (url.find(hostname) != string::npos) {
                        q.push(url);
                        seen.insert(url);
                    }
                }
                lock.unlock();
                cv.notify_all();
            }
        };

        for (int i = 0; i < nThreads; ++i)
            threads.emplace_back(t);

        for (std::thread& t : threads)
            t.join();
    }
};

```

```

        return (seen.begin(), seen.end());
    }

private:
    string getHostname(const string& url) {
        const int firstSlash = url.find_first_of("/");
        const int thirdSlash = url.find_first_of("/", firstSlash + 2);
        return url.substr(firstSlash + 2, thirdSlash - firstSlash - 2);
    }
};

```

C++

```

//
// This is the HtmlParser's API interface.
// You should not implement it, or speculate about its implementation
//
class HtmlParser {
public:
    vector<string> getUrls(string url);
};

//
class Solution {
public:
    vector<string> crawl(string startUrl, HtmlParser htmlParser) {
        queue<string> q{startUrl};
        unordered_set<string> seen{startUrl};
        const string hostname = getHostname(startUrl);

        // Threading
        const int nThreads = std::thread::hardware_concurrency();
        vector<thread> threads;
        std::mutex mtx;
        std::condition_variable cv;

        auto t = [&]() {
            while (true) {
                unique_lock<mutex> lock(mtx);

```

```

        cv.wait_for(lock, 30ms, [&]() { return q.size() > 0; });
        if (q.empty())
            return;

        auto cur = q.front();
        q.pop();
        lock.unlock();
        const vector<string> urIs = htmlParser.getUrls(cur);
        lock.lock();
        for (const string& url : urIs) {
            if (seen.contains(url))
                continue;
            if (url.find(hostname) != string::npos) {
                q.push(url);
                seen.insert(url);
            }
        }
        lock.unlock();
        cv.notify_all();
    }
};

for (int i = 0; i < nThreads; ++i)
    threads.emplace_back(t);

for (std::thread& t : threads)
    t.join();

return (begin(seen), end(seen));
}

```

```

private:
    string getHostname(const string& url) {
        const int firstSlash = url.find_first_of("/");
        const int thirdSlash = url.find_first_of("/", firstSlash + 2);
        return url.substr(firstSlash + 2, thirdSlash - firstSlash - 2);
    }
};

```

Java

```

//
// This is the HtmlParser's API interface.
// You should not implement it, or speculate about its implementation
//
class HtmlParser {
public:
    List<String> getUrls(String url);
};

```

Java

```

import java.net.URI;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentSkipListSet;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class Web_Crawler_Multithreaded {

    class Solution_1_synchronizedList {
        private final Set<String> set = Collections.newSetFromMap(new ConcurrentHashMap<String, Boolean>());
        private final List<String> result = Collections.synchronizedList(new ArrayList<String>());
        private String HOSTNAME = null;

        public List<String> crawl(String startUrl, HtmlParser htmlParser) {
            set.add(startUrl);
            getUrls(startUrl, htmlParser);
            return result;

            private boolean judgeHostname(String url) {
                int idx = url.indexOf("/", 7);
                String hostname = (idx != -1) ? url.substring(0, idx) : url;
                return hostname.equals(HOSTNAME);
            }

            private void initHostname(String url) {
                int idx = url.indexOf("/", 7);
                HOSTNAME = (idx != -1) ? url.substring(0, idx) : url;
            }

            private void getUrls(String startUrl, HtmlParser htmlParser) {
                result.add(startUrl);
                List<String> urIs = htmlParser.getUrls(startUrl);
                List<Thread> threads = new ArrayList<>();
                for (String url : urIs) {
                    if (judgeHostname(url) && !set.contains(url)) {
                        set.add(url);
                        threads.add(new Thread(() -> {
                            getUrls(url, htmlParser);
                        }));
                    }
                }
                for (Thread thread : threads)
                    thread.start();
            }
        }
    }
}

```

```

try {
    for (Thread thread : threads) {
        thread.join(); // waits for this thread to die.
    }
} catch (InterruptedException e) {
    e.printStackTrace();
}

}

}

class Solution_2_ConcurrentSkipListSet {

    public List<String> crawl(String startUrl, HtmlParser htmlParser) {
        // ConcurrentSkipListSet: Constructs a new, empty set that orders its elements
        // according to their (Comparable) natural ordering.
        // https://stackoverflow.com/questions/10443940/when-is-a-concurrentskiplistset-useful
        // ConcurrentSkipListSet and ConcurrentSkipListMap are useful when you need a sorted container
        // that will be accessed by multiple threads.
        // There are additionally two modifiers of Thread and ThreadLocal for concurrent code.
        Set<String> visited = new ConcurrentSkipListSet<>();
        String hostname = getHostname(startUrl);
        visited.add(startUrl);

        return crawlDFS(startUrl, htmlParser, hostname, visited).collect(Collectors.toList());
    }

    private Stream<String> crawlDFS(String startUrl, HtmlParser htmlParser, String hostname,

```

[*] DFS — Pattern Solution (matched from community answers)
Python

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

```

class Solution {
public:
    string allOrder(vector<string& words) {
        unordered_map<char, unordered_set<char>> graph;
        vector<int> indegrees(26);
        buildGraph(graph, words, indegrees);
        return topology(graph, indegrees);
    }

private:
    void buildGraph(unordered_map<char, unordered_set<char>>& graph,
        const vector<string& words, vector<int>& indegrees) {
        // Create a map for each character in each word.
        for (const string word : words)
            for (const char c : word)
                if (graph.contains(c))
                    graph[c].insert(word[i]);
                else
                    graph[c] = unordered_set<char>();

        for (int i = 1; i < words.size(); ++i) {
            const string first = words[i-1];
            const string second = words[i];
            const int length = min(first.length(), second.length());
            for (int j = 0; j < length; ++j) {
                const char w = first[j];
                const char v = second[j];
                if (w == v) continue;
                if (!graph[w].contains(v)) {
                    graph[w].insert(v);
                    ++indegrees[v-'a'];
                }
                break; // The order of characters after this are meaningless.
            }
            // e.g. first = "ab", second = "ac" -> invalid
            if (j == length - 1 && first.length() > second.length()) {
                return;
            }
        }
    }

    string topology(unordered_map<char, unordered_set<char>>& graph,
        vector<int>& indegrees) {
        string s;
        queue<char> q;

        for (const auto& [c, _] : graph)
            if (indegrees[c-'a'] == 0)
                q.push(c);
    }
}

```

```

while (!q.empty()) {
    const char w = q.front();
    q.pop();
    s += w;
    for (const char v : graph[w])
        if (--indegrees[v] == 0)
            q.push(v);
}

// e.g. words = ["a", "b", "c", "d", "e"]
return s.length() == graph.size() ? s : "";
}
}

```

```

// 02. 11/2021 / testcode.com/problems/alias-dictionary/
// Time: O(N)
// Space: O(1)
class Solution {
public:
    string allOrder(vector<string& A) {
        unordered_map<int, unordered_set<int>> G;
        int indegree[26] = {0};
        for (auto& s : A) {
            for (char c : s) G[c-'a'].insert(s[i]);
        }

        for (int i = 0; i < A.size(); ++i) {
            int j = 0;
            for (; j < min(A[i-1].size(), A[i].size()); ++j) {
                if (A[i-1][j] == A[i][j]) continue;
                G[A[i-1][j] - 'a'].insert(A[i][j] - 'a');
                break;
            }
            if (j == A[i].size() && j < A[i-1].size()) return "";
        }

        for (auto& [from, to] : G) {
            for (int to : to) {
                indegree[to]++;
            }
        }

        queue<int> q;
        for (int i = 0; i < 26; ++i)
            if (G.count(i) && indegree[i] == 0) q.push(i);

        string ans;
        while (q.size()) {
            int w = q.front();
            q.pop();
            ans += w;
            for (int v : G[w])
                if (--indegree[v] == 0) q.push(v);
        }

        return ans.size() == G.size() ? ans : "";
    }
};

```

```

java
class Solution {
    public String allOrder(Map<String, Set<Character>> graph, Map<String, Integer> indegree) {
        Map<Character, Set<Character>> g = new HashMap<>();
        List<String> words = new ArrayList<>();
        buildGraph(graph, indegree);
        return topology(graph, indegree);
    }

    private void buildGraph(Map<String, Set<Character>> graph, Map<String, Integer> indegree) {
        // Create a map for each character in each word.
        for (final String word : words)
            for (final char c : word.toCharArray())
                graph.putIfAbsent(c, new HashSet<>());

        for (int i = 1; i < words.size(); ++i) {
            final String first = words[i-1];
            final String second = words[i];
            final int length = Math.min(first.length(), second.length());
            for (int j = 0; j < length; ++j) {
                final char w = first.charAt(j);
                final char v = second.charAt(j);
                if (w == v) continue;
                if (!graph.get(w).contains(v)) {
                    graph.get(w).add(v);
                    ++indegree[v-'a'];
                }
                break; // The order of characters after this are meaningless.
            }
            // e.g. first = "ab", second = "ac" -> invalid
            if (j == length - 1 && first.length() > second.length()) {
                return;
            }
        }
    }

    private String topology(Map<String, Set<Character>> graph, Map<String, Integer> indegree) {
        StringBuilder sb = new StringBuilder();
        Queue<Character> q = new PriorityQueue<>();

        for (final char c : graph.keySet())
            if (indegree[c-'a'] == 0)
                q.offer(c);

        while (!q.isEmpty()) {
            final char w = q.poll();
            sb.append(w);
            for (final char v : graph.get(w))
                if (--indegree[v-'a'] == 0)
                    q.offer(v);
        }

        return sb.toString();
    }
}

```

```

}

// e.g. words = ["a", "b", "c", "d", "e"]
return sb.length() == graph.size() ? sb.toString() : "";
}
}

java
import java.util.*;

public class AliasDictionary {
    public static void main(String[] args) {
        AliasDictionary ad = new AliasDictionary();
        Solution s = new Solution();
        Solution s1 = new Solution();
        Solution s2 = new Solution();

        String[] words = new String[] {
            "act",
            "bat",
            "cat",
            "eat",
            "fat"
        };

        System.out.println(s.allOrder(words));
        System.out.println(s2.allOrder(words));
    }

    // ref: http://bettercraze.blogspot.com/2015/05/testcode-alias-dictionary.html
    public class Solution_s1 {
        // Create a map for each character in each word.
        for (final String word : words)
            for (final char c : word.toCharArray())
                graph.putIfAbsent(c, new HashSet<>());

        for (int i = 1; i < words.size(); ++i) {
            final String first = words[i-1];
            final String second = words[i];
            final int length = Math.min(first.length(), second.length());
            for (int j = 0; j < length; ++j) {
                final char w = first.charAt(j);
                final char v = second.charAt(j);
                if (w == v) continue;
                if (!graph.get(w).contains(v)) {
                    graph.get(w).add(v);
                    ++indegree[v-'a'];
                }
                break; // The order of characters after this are meaningless.
            }
            // e.g. first = "ab", second = "ac" -> invalid
            if (j == length - 1 && first.length() > second.length()) {
                return;
            }
        }

        private String topology(Map<String, Set<Character>> graph, Map<String, Integer> indegree) {
            StringBuilder sb = new StringBuilder();
            Queue<Character> q = new PriorityQueue<>();

            for (final char c : graph.keySet())
                if (indegree[c-'a'] == 0)
                    q.offer(c);

            while (!q.isEmpty()) {
                final char w = q.poll();
                sb.append(w);
                for (final char v : graph.get(w))
                    if (--indegree[v-'a'] == 0)
                        q.offer(v);
            }

            return sb.toString();
        }
    }
}

```

```

for (Character node : adjMap.keySet()) {
    for (Character neighbor : adjMap.get(node)) {
        int indegree = indegreeMap.get(neighbor);
        indegree++;
        indegreeMap.put(neighbor, indegree);
    }
}

// start from indegree=0
Queue<Character> queue = new PriorityQueue<>();
for (Character node : indegreeMap.keySet()) {
    if (indegreeMap.get(node) == 0) {
        queue.offer(node);
    }
}

while (!queue.isEmpty()) {
    char curNode = queue.poll();
    result.append(curNode);

    for (Character neighbor : adjMap.get(curNode)) {
        int indegree = indegreeMap.get(neighbor);
        indegree--;
        indegreeMap.put(neighbor, indegree);
    }
}

// queue: key is Node.
// for A-B, B-C, A-C, C will not be counted until its indegree is 0
if (indegree == 0) {
}

```

Go

```

func allOrder(words []string) string {
    g := [26][26]bool{}
    n := len(words)
    cat := 0
    for i := 0; i < n-1; i++ {
        for j := i+1; j < n; j++ {
            if cat == 0 {
                break
            }
            c := 'a'
            if !g[i][c] {
                cat = true
                g[i][c] = true
            }
        }
        m := false
        for j := 0; j < n; j++ {
            if j == i || j == n-1 {
                continue
            }
            if g[i][j] && !g[j][i] {
                m = true
                break
            }
        }
        if m {
            return ""
        }
        for j := 0; j < n; j++ {
            if g[j][i] {
                g[j][i] = true
                break
            }
        }
    }

    indegree := [26]int{}
    for i, cat := range words {
        for j, v := range cat {
            if v {
                indegree[j]++
            }
        }
    }
}

```

```

    }
    q := []int{}
    for i, in := range indegree {
        if in == 0 && s[i] {
            q = append(q, i)
        }
    }
    ans := ""
    for len(q) > 0 {
        t := q[0]
        q = q[1:]
        ans += string(t) + " "
        for i, v := range g[t] {
            if v {
                indegree[i]--
                if indegree[i] == 0 && s[i] {
                    q = append(q, i)
                }
            }
        }
    }
    if len(ans) < out {
        return ""
    }
    return ans
}

```

[1] DFS – Pattern Solution (matched from community answers)

```

Python
# DFS
def dfs(x, y, z):
    if x < 0 or x > n-1 or y < 0 or y > m-1 or mat[x][y] == z:
        return 0
    mat[x][y] = z
    ans = 0
    for dx, dy in directions:
        ans = max(ans, 1 + dfs(x+dx, y+dy, z))
    return ans

def longestIncreasingPath(mat):
    n, m = len(mat), len(mat[0])
    memo = {}
    for i in range(n):
        for j in range(m):
            memo[i, j] = dfs(i, j, mat[i][j])
    return max(memo.values())

```

DFS · LeeMastery — By Pattern

— 2203 —

LeeMastery

```

# DFS
def dfs(x, y, z):
    if x < 0 or x > n-1 or y < 0 or y > m-1 or mat[x][y] == z:
        return 0
    mat[x][y] = z
    ans = 0
    for dx, dy in directions:
        ans = max(ans, 1 + dfs(x+dx, y+dy, z))
    return ans

def longestIncreasingPath(mat):
    n, m = len(mat), len(mat[0])
    memo = {}
    for i in range(n):
        for j in range(m):
            memo[i, j] = dfs(i, j, mat[i][j])
    return max(memo.values())

```

DFS · LeeMastery — By Pattern

— 2204 —

LeeMastery

#329 **HARD**
Longest Increasing Path in a Matrix
 LeetCode · SimplifyLust · WubCC · LeeCode · Editorial
 Array · DFS · BFS · Graph · Topological Sort · Memoization · Matrix
 Lints Grind 189

Problem:
 Given an $m \times n$ integers matrix, return the length of the longest increasing path in matrix.
 From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

Example 1:

9	9	4
6	6	8
2	1	1

Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]
 Output: 4
 Explanation: The longest increasing path is [1, 2, 6, 9].

Example 2:

3	4	5
3	2	6
2	2	1

Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]
 Output: 4
 Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.
 Example 3:
 Input: matrix = [[1]]
 Output: 1
 Constraints:
 • $m = \text{matrix.length}$
 • $n = \text{matrix[0].length}$
 • $1 \leq m, n \leq 200$
 • $0 \leq \text{matrix[i][j]} \leq 231 - 1$

Python
 Python

DFS · LeeMastery — By Pattern

— 2205 —

LeeMastery

DFS · LeeMastery — By Pattern

— 2206 —

LeeMastery

```

# Python Implementation for Longest Increasing Path in a Matrix
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        # Memoization table
        memo = {}
        # Dimensions of the matrix
        m, n = len(matrix), len(matrix[0])
        # Create a memoization table (same dimensions as the matrix) initialized to 0
        memo = [[0] * n for _ in range(m)]
        # Before the DFS function to explore the matrix
        # If result is already computed, return it
        # If memo[i][j] is 0, it means it's not computed yet
        # Start with a path length of 1 (the cell itself)
        max_len = 1
        # Define directions: up, down, left, right
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        # Explore each direction
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            # If neighbor is within bounds and its value is greater
            if 0 <= nx < m and 0 <= ny < n and matrix[nx][ny] > matrix[x][y]:
                length = 1 + dfs(nx, ny)
                max_len = max(max_len, length)
        # Store computed longest length in memoization table
        memo[i][j] = max_len
        return max_len

# Compute the longest increasing path starting from each cell
longest_path = 0
for i in range(m):
    for j in range(n):
        longest_path = max(longest_path, dfs(i, j))
return longest_path

```

Python

DFS · LeeMastery — By Pattern

— 2207 —

LeeMastery

```

class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        m, n = len(matrix), len(matrix[0])
        @functools.lru_cache(None)
        def dfs(i, j):
            if i < 0 or i >= m or j < 0 or j >= n:
                return 0
            if matrix[i][j] in prev:
                return 0
            curr = matrix[i][j]
            return 1 + max(dfs(i + 1, j, curr),
                          dfs(i - 1, j, curr),
                          dfs(i, j + 1, curr),
                          dfs(i, j - 1, curr))
        return max(dfs(i, j, -1) for i in range(m) for j in range(n))

Python
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        memo = {}
        def dfs(i, j):
            if i < 0 or i >= m or j < 0 or j >= n:
                return 0
            if matrix[i][j] in prev:
                return 0
            curr = matrix[i][j]
            return 1 + max(dfs(i + 1, j, curr),
                          dfs(i - 1, j, curr),
                          dfs(i, j + 1, curr),
                          dfs(i, j - 1, curr))
        return max(dfs(i, j, -1) for i in range(m) for j in range(n))

Python
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        memo = {}
        def dfs(i, j):
            if i < 0 or i >= m or j < 0 or j >= n:
                return 0
            if matrix[i][j] in prev:
                return 0
            curr = matrix[i][j]
            return 1 + max(dfs(i + 1, j, curr),
                          dfs(i - 1, j, curr),
                          dfs(i, j + 1, curr),
                          dfs(i, j - 1, curr))
        return max(dfs(i, j, -1) for i in range(m) for j in range(n))

C++
C++

```

DFS · LeeMastery — By Pattern

— 2208 —

LeeMastery

```

class Solution {
public:
    int longestIncreasingPath(vector<vector<int>>& matrix) {
        int n = matrix.size(), m = matrix[0].size();
        int f[n][m];
        memset(f, 0, sizeof(f));
        int dirs[] = {-1, 0, 1, 0, -1};
        function<int(int, int)> dfs = [&](int i, int j) -> int {
            if (f[i][j]) {
                return f[i][j];
            }
            for (int k = 0; k < 4; ++k) {
                int x = i + dirs[k], y = j + dirs[k + 1];
                if (x >= 0 && x < n && y >= 0 && y < m && matrix[x][y] > matrix[i][j]) {
                    f[i][j] = max(f[i][j], dfs(x, y));
                }
            }
            return ++f[i][j];
        };
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                ans = max(ans, dfs(i, j));
            }
        }
        return ans;
    }
};

```

Java
Java

DPS · LeetCode · By Pattern

— 2209 —

LeetCode

```

class Solution {
    private int m;
    private int n;
    private int[][] matrix;
    private int[][] f;

    public int longestIncreasingPath(int[][] matrix) {
        m = matrix.length;
        n = matrix[0].length;
        f = new int[m][n];
        this.matrix = matrix;
        int ans = 0;
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) {
                ans = Math.max(ans, dfs(i, j));
            }
        }
        return ans;
    }

    private int dfs(int i, int j) {
        if (f[i][j] > 0) {
            return f[i][j];
        }
        int[] dirs = {-1, 0, 1, 0, -1};
        for (int k = 0; k < 4; ++k) {
            int x = i + dirs[k];
            int y = j + dirs[k + 1];
            if (x >= 0 && x < m && y >= 0 && y < n && matrix[x][y] > matrix[i][j]) {
                f[i][j] = Math.max(f[i][j], dfs(x, y));
            }
        }
        return ++f[i][j];
    }
}

```

TypeScript
TypeScript

DPS · LeetCode · By Pattern

— 2210 —

LeetCode

```

function longestIncreasingPath(matrix: number[][]): number {
    const m = matrix.length;
    const n = matrix[0].length;
    const f: number[][] = Array(m).fill(0);
    const dirs = [-1, 0, 1, 0, -1];
    const dfs = (i: number, j: number): number => {
        if (f[i][j] > 0) {
            return f[i][j];
        }
        for (let k = 0; k < 4; ++k) {
            const x = i + dirs[k];
            const y = j + dirs[k + 1];
            if (x >= 0 && x < m && y >= 0 && y < n && matrix[x][y] > matrix[i][j]) {
                f[i][j] = Math.max(f[i][j], dfs(x, y));
            }
        }
        return ++f[i][j];
    };
    let ans = 0;
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            ans = Math.max(ans, dfs(i, j));
        }
    }
    return ans;
}

```

Go
Go

```

func longestIncreasingPath(matrix [][]int) (ans int) {
    m, n := len(matrix), len(matrix[0])
    f := make([][]int, m)
    for i := range f {
        f[i] = make([]int, n)
    }
    dirs := [...]int{-1, 0, 1, 0, -1}
    var dfs func(i, j int) int
    dfs = func(i, j int) int {
        if f[i][j] != 0 {
            return f[i][j]
        }
        for k := 0; k < 4; k++ {
            x, y := i+dirs[k], j+dirs[k+1]
            if 0 <= x && x < m && 0 <= y && y < n && matrix[x][y] > matrix[i][j] {
                f[i][j] = max(f[i][j], dfs(x, y))
            }
        }
        f[i][j]++
        return f[i][j]
    }
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            ans = max(ans, dfs(i, j))
        }
    }
}

```

DPS · LeetCode · By Pattern

— 2211 —

LeetCode

return

[*] DFS — Pattern Solution (masched from community answers)

Python

Python implementation for Longest Increasing Path in a Matrix

```

class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        # Return 0 if matrix is empty
        if not matrix or not matrix[0]:
            return 0

        # Dimensions of the matrix
        m, n = len(matrix), len(matrix[0])
        # Create a memoization table (same dimensions as the matrix) initialized to 0
        memo = [[0] * n for _ in range(m)]

        # Define the dfs function to explore the matrix
        def dfs(i, j):
            # If result is already computed, return it
            if memo[i][j]:
                return memo[i][j]

            # Start with a path length of 1 (the cell itself)
            max_len = 1
            # Define directions: up, down, left, right
            directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

            # Explore each direction
            for dx, dy in directions:
                nx, ny = i + dx, j + dy
                # Skip if out of bounds or if value is greater
                if 0 <= nx < m and 0 <= ny < n and matrix[nx][ny] > matrix[i][j]:
                    length = 1 + dfs(nx, ny)
                    max_len = max(max_len, length)

            # Store computed longest length in memoization table
            memo[i][j] = max_len
            return max_len

        # Compute the longest increasing path starting from each cell
        longest_path = 0
        for i in range(m):
            for j in range(n):
                longest_path = max(longest_path, dfs(i, j))

        return longest_path

```

#685

HARD

Redundant Connection II

LeetCode · SimplifyLab · WalkCC · LeetCode · Editorial

DPS · LeetCode · By Pattern

— 2212 —

LeetCode

DPS · BFS · Union Find · Graph
Little Danny Zhang

Problem:

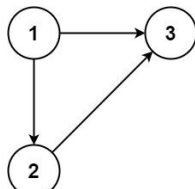
In this problem, a rooted tree is a directed graph such that, there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents.

The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n , and was not an edge that already existed.

The resulting graph is given as a 2D-array of edges. Each element of edges is a pair $[u, v]$ that represents a directed edge connecting nodes u and v , where u is a parent of child v .

Return an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs last in the given 2D-array.

Example 1:



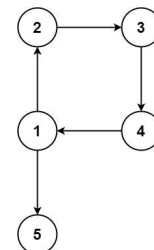
Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:

DPS · LeetCode · By Pattern

— 2213 —

LeetCode



Input: edges = [[1,2],[2,3],[3,4],[4,1],[1,5]]
Output: [4,1]

Constraints:

- $n == \text{edges.length}$
- $3 \leq n \leq 1000$
- $\text{edges}[i].\text{length} == 2$
- $1 \leq u_i, v_i \leq n$
- $u_i \neq v_i$

>> Brute Force [DFS] Interview Prep

Python

```

# Brute-Force Approach
class Solution:
    def findRedundantDirectedConnection(self, edges):
        # Simple DFS - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            for nb in graph.get(node, []):
                if nb not in visited:
                    dfs(nb)
            for node in range(1, n):
                if node not in visited:
                    dfs(node); count += 1
            return count

```

Python
Python

DPS · LeetCode · By Pattern

— 2214 —

LeetCode

```

# Python solution with line-by-line comments
class Solution:
    def findRedundantDirectedConnection(self, self, edges):
        n = len(edges)
        candidate = None
        candidate2 = None
        # Necessary to record the parent for each child node.
        childParent = {}
        # First pass: detect if a node has two parents.
        for u, v in edges:
            if v in childParent:
                candidate = [childParent[v], v] # earlier edge providing parent
                candidate2 = [u, v] # later edge providing parent
            else:
                childParent[v] = u

        # Building undirected structure: each node is its own parent.
        root = [i for i in range(1+n)]

        # Find function with path compression.
        def find(x):
            while root[x] != x:
                root[x] = root[root[x]]
            x = root[x]
            return x

        cycle_edge = None
        # Second pass: union-find to detect cycle, skip candidate2 if conflict exists.
        for u, v in edges:
            # If candidate2 exists and current edge is candidate2, skip it.
            if candidate2 and [u, v] == candidate2:
                continue
            parent_u = find(u)
            parent_v = find(v)
            # If they have the same root, a cycle is detected.
            if parent_u == parent_v:
                cycle_edge = [u, v]
            else:
                root[parent_v] = parent_u

        # Decide which edge to remove based on conflict and cycle detection.
        if candidate:
            return candidate2 if cycle_edge else candidate
        return cycle_edge

```

Python

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return False
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[i] += 1
        return True

    def find(self, u: int) -> int:
        if self.id[u] == u:
            return u
        self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def findRedundantDirectedConnection(self, self, edges: List[List[int]]) -> List[int]:
        id = [i for i in range(1 + len(edges))]
        nodesWithTwoParents = 0

        for u, v in edges:
            id[u] = v
            if id[u] == 2:
                nodesWithTwoParents = v

        def findRedundantDirectedConnection(skippedEdgeIndex: int) -> List[int]:
            uf = UnionFind(len(edges) + 1)

            for i, edge in enumerate(edges):
                if i == skippedEdgeIndex:
                    continue
                if not uf.unionByRank(edge[0], edge[1]):
                    return edge

            return []

        # If there is no edge with two ids, don't skip any edge.
        if nodesWithTwoParents == 0:
            return findRedundantDirectedConnection(-1)

        for i in reversed(range(len(edges))):
            u, v = edges[i]
            if v == nodesWithTwoParents:
                # Try to remove the edge.
                if not findRedundantDirectedConnection(i):
                    return edges[i]

```

```

Python
class Solution:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(edges)
        ind = [0] * n
        for u, v in edges:
            ind[v] += 1
        dup = [i for i, c in enumerate(ind) if ind[i] > 1]
        p = list(range(n))
        if dup:
            for i, (u, v) in enumerate(edges):
                if i == dup[0]:
                    continue
                pu, pv = find(u - 1), find(v - 1)
                if pu == pv:
                    return edges[dup[0]]
                p[pu] = pv
            return edges[dup[1]]
        p[pu] = pv

Python
class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.s = n

    def union(self, a, b):
        if self.find(a) == self.find(b):
            return False
        self.p[self.find(a)] = self.find(b)
        self.s -= 1
        return True

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

class Solution:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
        n = len(edges)
        p = list(range(n + 1))
        uf = UnionFind(n + 1)
        conflict = cycle = None
        for i, (u, v) in enumerate(edges):
            pu, pv = find(u - 1), find(v - 1)
            if pu == pv:
                return edges[i]
            p[pu] = pv

```

Python

```

        if p[u] != v:
            conflict = 1
        else:
            p[u] = u
            if not self.isAncestor(v, u):
                cycle = 1
            if conflict != None:
                return edges[cycle]
            v = edges[conflict][1]
            if cycle is not None:
                return [p[v], v]
            return edges[conflict]

C++
class UnionFind {
public:
    UnionFind(int n) : id(n), rank(n) {
        iota(id.begin(), id.end(), 0);
    }

    bool unionByRank(int u, int v) {
        const int i = find(u);
        const int j = find(v);
        if (i == j)
            return false;
        if (rank[i] < rank[j]) {
            id[i] = j;
        } else if (rank[i] > rank[j]) {
            id[j] = i;
        } else {
            id[i] = j;
            ++rank[j];
        }
        return true;
    }

private:
    vector<int> id;
    vector<int> rank;
}

```

```

int find(int u) {
    return id[u] == u ? u : id[u] = find(id[u]);
}

class Solution {
public:
    vector<int> findRedundantDirectedConnection(vector<vector<int>>& edges) {
        vector<int> id(edges.size() + 1);
        int nodesWithTwoParents = 0;

        for (const vector<int>& edge : edges) {
            const int u = edge[0];
            if (++id[u] == 2) {
                nodesWithTwoParents = u;
                break;
            }
        }

        // If there is no edge with two ids, don't skip any edge.
        if (nodesWithTwoParents == 0)
            return findRedundantDirectedConnection(edges, -1);

        for (int i = edges.size() - 1; i >= 0; --i)
            if (edges[i][1] == nodesWithTwoParents)
                return edges[i];

        throw;
    }

    vector<int> findRedundantDirectedConnection(const vector<vector<int>>& edges,
        int skippedEdgeIndex) {
        UnionFind uf(edges.size() + 1);

        for (int i = 0; i < edges.size(); ++i) {
            if (i == skippedEdgeIndex)
                continue;
            if (uf.unionByRank(edges[i][0], edges[i][1]))
                return edges[i];
        }

        return {};
    }
};

```

C++

```

class UnionFind {
public:
    vector<int> p;
    int n;

    UnionFind(int _n) : n(_n) {
        p.resize(n);
        iota(p.begin(), p.end(), 0);
    }

    bool unionByRank(int a, int b) {
        int pa = find(a), pb = find(b);
        if (pa == pb) return false;
        p[pa] = pb;
        ++pb;
        return true;
    }

    int find(int x) {
        if (p[x] != x) p[x] = find(p[x]);
        return p[x];
    }
};

class Solution {
public:
    vector<int> findRedundantDirectedConnection(vector<vector<int>>& edges) {
        int n = edges.size();
        vector<int> p(n);
        for (int i = 0; i < n; ++i) p[i] = i;
        UnionFind uf(n);
        int conflict = -1, cycle = -1;
        for (int i = 0; i < n; ++i) {
            int u = edges[i][0], v = edges[i][1];
            if (p[u] != v)
                conflict = i;
            else {
                p[u] = u;
                if (uf.isAncestor(v, u)) cycle = i;
            }
        }

        if (conflict == -1) return edges[cycle];
        int v = edges[conflict][1];
        if (cycle != -1) return [v, v];
        return edges[conflict];
    }
};

```

Java

```

class UnionFind {
public UnionFind(int n) {
    id = new int[n];
    rank = new int[n];
    for (int i = 0; i < n; ++i)
        id[i] = i;
}

public boolean unionByRank(int u, int v) {
    find(int i = find(u));
    find(int j = find(v));
    if (i == j)
        return false;
    if (rank[i] < rank[j]) {
        id[j] = i;
    } else if (rank[i] > rank[j]) {
        id[i] = j;
    } else {
        id[i] = j;
        ++rank[j];
    }
    return true;
}

private int[] id;

private int[] rank;

private int find(int u) {
    return id[u] == u ? u : (id[u] = find(id[u]));
}
}

class Solution {
public int[] findRedundantDirectedConnection(int[][] edges) {
    int[] idn = new int[edges.length + 1];
    int nodeWithTwoParents = 0;

    for (int[] edge : edges) {
        find(int u = edge[0]);
        if (idn[edge[1]] == 0) {
            nodeWithTwoParents = u;
            break;
        }
    }

    // If there is no edge with two IDs, don't skip any edge.
    if (nodeWithTwoParents == 0)
        return findRedundantDirectedConnection(edges, -1);

    for (int i = edges.length - 1; i >= 0; --i)

```

```

        if (edges[i][1] == nodeWithTwoParents)
            // try to delete the edge[i]
            if (findRedundantDirectedConnection(edges, i).length == 0)
                return edges[i];
    }

    throw new IllegalArgumentException();
}

private int[] findRedundantDirectedConnection(int[][] edges, int skippedEdgeIndex) {
    UnionFind uf = new UnionFind(edges.length + 1);

    for (int i = 0; i < edges.length; ++i) {
        if (i == skippedEdgeIndex)
            continue;
        if (uf.unionByRank(edges[i][0], edges[i][1]))
            return edges[i];
    }

    return new int[] {};
}
}

Java

class Solution {
public int[] findRedundantDirectedConnection(int[][] edges) {
    int n = edges.length;
    int[] p = new int[n + 1];
    for (int i = 0; i < n; ++i) {
        p[i] = i;
    }

    UnionFind uf = new UnionFind(n + 1);
    int conflict = -1, cycle = -1;
    for (int i = 0; i < n; ++i) {
        int u = edges[i][0], v = edges[i][1];
        if (p[u] == v) {
            conflict = i;
        } else {
            p[v] = u;
            if (uf.union(u, v)) {
                cycle = i;
            }
        }
    }

    if (conflict == -1) {
        return edges[cycle];
    }

    int v = edges[conflict][1];
    if (cycle == -1) {

```

```

        return new int[] {p(v), v};
    }

    return edges[conflict];
}
}

class UnionFind {
public int[] p;
public int n;

public UnionFind(int n) {
    p = new int[n];
    for (int i = 0; i < n; ++i) {
        p[i] = i;
    }
    this.n = n;
}

public boolean union(int a, int b) {
    int pa = find(a);
    int pb = find(b);
    if (pa == pb) {
        return false;
    }
    p[pa] = pb;

    --n;
    return true;
}

public int find(int x) {
    if (p[x] == x) {
        p[x] = find(p[x]);
    }
    return p[x];
}
}

JavaScript
JavaScript

```

```

//n
+ Queryn (number[][] edges
+ returnn (number[]);
//
var findRedundantDirectedConnection = function (edges) {
    const n = edges.length;
    const ind = Array(n).fill(0);
    for (const [u, v] of edges) {
        ++ind[v] - 1;
    }

    const dup = [];
    for (let i = 0; i < n; ++i) {
        if (ind[edges[i][1]] - 1] === 2) {
            dup.push(i);
        }
    }

    const p = Array.from({ length: n }, (_, i) => i);
    const find = n => {
        if (p[n] == n) {
            p[n] = find(p[n]);
        }
        return p[n];
    };

    if (dup.length) {
        for (let i = 0; i < n; ++i) {
            if (i === dup[0]) {
                continue;
            }
            const [pu, pv] = [find(edges[i][0] - 1), find(edges[i][1] - 1)];
            if (pu === pv) {
                return edges[dup[0]];
            }
            p[pu] = pv;
            return edges[dup[0]];
        }
    }

    for (let i = 0; i < n; ++i) {
        const [pu, pv] = [find(edges[i][0] - 1), find(edges[i][1] - 1)];
        if (pu === pv) {
            return edges[i];
        }
        p[pu] = pv;
    }
};

TypeScript
TypeScript

```

```

function findRedundantDirectedConnection(edges: number[][], number[]: number[]) {
    const n = edges.length;
    const ind = number[] = Array(n).fill(0);
    for (const [u, v] of edges) {
        ++ind[v] - 1;
    }

    const dup = number[] = [];
    for (let i = 0; i < n; ++i) {
        if (ind[edges[i][1]] - 1] === 2) {
            dup.push(i);
        }
    }

    const p: number[] = Array.from({ length: n }, (_, i) => i);
    const find = (x: number): number => {
        if (p[x] == x) {
            p[x] = find(p[x]);
        }
        return p[x];
    };

    if (dup.length) {
        for (let i = 0; i < n; ++i) {
            if (i === dup[0]) {
                continue;
            }
            const [pu, pv] = [find(edges[i][0] - 1), find(edges[i][1] - 1)];
            if (pu === pv) {
                return edges[dup[0]];
            }
            p[pu] = pv;
            return edges[dup[0]];
        }
    }

    for (let i = 0; i < n; ++i) {
        const [pu, pv] = [find(edges[i][0] - 1), find(edges[i][1] - 1)];
        if (pu === pv) {
            return edges[i];
        }
        p[pu] = pv;
    }
};

Go
Go

```

```

type unionFind struct {
    p []int
    n int
}

func newUnionFind(n int) unionFind {
    p := make([]int, n)
    for i := range p {
        p[i] = i
    }
    return unionFind{p, n}
}

func (uf unionFind) find(x int) int {
    if uf.p[x] == x {
        uf.p[x] = uf.find(uf.p[x])
    }
    return uf.p[x]
}

func (uf unionFind) union(a, b int) bool {
    if uf.find(a) == uf.find(b) {
        return false
    }
    uf.p[uf.find(a)] = uf.find(b)
}

uf :=
return true
}

func findRedundantDirectedConnection(edges [][]int) []int {
    n := len(edges)
    p := make([]int, n+1)
    for i := range p {
        p[i] = i
    }

    uf := newUnionFind(n)
    conflict, cycle := -1, -1
    for i, w := range edges {
        u, v := w[0], w[1]
        if p[u] == v {
            conflict = i
        } else {
            p[v] = u
            if uf.union(u, v) {
                cycle = i
            }
        }
    }

    if conflict == -1 {
        return edges[cycle]
    }

    v := edges[conflict][1]
    if cycle == -1 {
        return []int{p[v], v}
    }
    return edges[conflict]
}

```

```

[+] DFS -- Pattern Solution (stored optimal)
Python
# Python solution with line-by-line comments
class Solution:
    def findRoots(self, directedConnection: List[List[int]]):
        n = len(edges)
        candidate1 = None
        candidate2 = None
        # Dictionary to record the parent for each child node.
        childParent = {}
        # First pass: detect if a node has two parents.
        for u, v in edges:
            if u in childParent:
                candidate1 = [childParent[u], v] # earlier edge providing parent
            else:
                candidate2 = [u, v] # later edge providing parent
            childParent[u] = v

        # Initialize Union-Find structure: each node is its own parent.
        root = [i for i in range(n+1)]

        # Find function with path compression.
        def find(x):
            while root[x] != x:
                root[x] = root[root[x]]
                x = root[x]
            return x

        cycleEdge = None
        # Second pass: Union-Find to detect cycle, skip candidate1 if conflict exists.
        for u, v in edges:
            # If candidate1 exists and current edge is candidate1, skip it.
            if candidate1 and [u, v] == candidate1:
                continue
            parent_u = find(u)
            parent_v = find(v)
            # If they have the same root, a cycle is detected.
            if parent_u == parent_v:
                cycleEdge = [u, v]
            else:
                root[parent_v] = parent_u

        # Remove extra edge to remove based on conflict and cycle detection.
        if candidate1:
            return candidate1 if cycleEdge else candidate2
        return cycleEdge

```

C++

DFS · LeetCode · By Pattern

— 2227 —

LeetCode

```

// C++ solution with line-by-line comments
class Solution {
public:
    vector<int> findRoots(vector<vector<int>&> edges) {
        int n = edges.size();
        vector<int> parent(n, -1, 0);
        vector<int> candidate1, candidate2;

        // First pass: check if a node has two parents.
        for (auto &edge : edges) {
            int u = edge[0], v = edge[1];
            if (parent[u] != -1) {
                candidate1 = {parent[u], v}; // first parent edge
                candidate2 = {u, v}; // conflicting second edge
                // Note: this edge is to be skipped later.
                edge[0] = -1;
            } else {
                parent[u] = v;
            }
        }

        // Initialize union-find structure.
        vector<int> root(n, -1);
        for (int i = 0; i < n; ++i) {
            root[i] = i;
        }

        // Second pass: perform union-find + root with path compression.
        function<int(int)> findRoot = [&](int x) -> int {
            return root[x] == x ? x : root[x] = findRoot(root[x]);
        };

        vector<int> cycleEdge;
        // Second pass: process each edge using union-find.
        for (auto &edge : edges) {
            if (edge[0] == -1) continue; // Skip candidate1 edge.
            int u = edge[0], v = edge[1];
            int pu = findRoot(u), pv = findRoot(v);
            if (pu == pv) {
                cycleEdge = {u, v}; // Cycle found.
            } else {
                root[pu] = pv;
            }
        }

        // Determine which edge to remove.
        if (!candidate1.empty()) {
            return {cycleEdge.empty() ? candidate1 : candidate2};
        }
        return cycleEdge;
    }
};

```

#1036

HARD

Escape a Large Maze

DFS · LeetCode · By Pattern

— 2228 —

LeetCode

LeetCode · SimplifyTest · WalkCC · LeetCode · Editorial
Array · Hash Table · DFS · BFS

Like · Donny Zhang

Problem:

There is a 1 million by 1 million grid on an XY-plane, and the coordinates of each grid square are (x, y).

We start at the source = [sx, sy] square and want to reach the target = [tx, ty] square. There is also an array of blocked squares, where each blocked[i] = [xi, yi] represents a blocked square with coordinates (xi, yi).

Each move, we can walk one square north, east, south, or west if the square is not in the array of blocked squares. We are also not allowed to walk outside of the grid.

Return true if and only if it is possible to reach the target square from the source square through a sequence of valid moves.

Example 1:

Input: blocked = [[0,1],[1,0]], source = [0,0], target = [0,2]
Output: false

Explanation: The target square is inaccessible starting from the source square because we cannot move.

We cannot move north or east because those squares are blocked.

We cannot move south or west because we cannot go outside of the grid.

Example 2:

Input: blocked = [], source = [0,0], target = [99999,99999]
Output: true

Explanation: Because there are no blocked cells, it is possible to reach the target square.

Constraints:

- 0 <= blocked.length <= 200
- blocked[i].length == 2
- 0 <= xi, yi < 10⁶
- source.length == target.length == 2
- 0 <= sx, sy, tx, ty < 10⁶
- source != target
- It is guaranteed that source and target are not blocked.

>> Brute Force (DFS) Interview Prep

Python

DFS · LeetCode · By Pattern

— 2229 —

LeetCode

```

# Brute Force Approach
class Solution:
    def isEscapePossible(self, blocked, source, target):
        # Brute force - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for nb in graph.get(node, []):
                if nb not in visited:
                    dfs(nb)
            for node in range(n):
                if node not in visited:
                    dfs(node); count += 1
            return count

        # Python
        Python
        from collections import deque

        def isEscapePossible(blocked, source, target):
            blocked_set = {(x, y) for x, y in blocked}
            if n == 2:
                return True
            max_steps = n * n - 1 // 2

            def bfs(start, finish):
                queue = deque([start])
                seen = {tuple(start)}
                direction = [(0, 1), (1, 0), (0, -1), (-1, 0)]
                steps = 0

                while queue:
                    x, y = queue.popleft()
                    if (x, y) == finish:
                        return True
                    for dx, dy in direction:
                        nx, ny = x + dx, y + dy
                        if 0 <= nx < 10**6 and 0 <= ny < 10**6 and (nx, ny) not in blocked_set and (nx, ny) not in seen:
                            queue.append((nx, ny))
                            seen.add((nx, ny))
                            steps += 1

                if steps > max_steps:
                    return True
                return False

            return bfs(source, target) and bfs(target, source)

        # Example output
        print(isEscapePossible([[]], [0,0], [0,0], [0,0])) # Expected output: False
        print(isEscapePossible([[]], [0,0], [99999,99999])) # Expected output: True

        Python

```

DFS · LeetCode · By Pattern

— 2230 —

LeetCode

```

class Solution:
    def isEscapePossible(
        self, blocked: List[List[int]], source: List[int], target: List[int]
    ) -> bool:
        def dfs(i: int, j: int, target: List[int], seen: set) -> bool:
            if i < 0 or i >= 10**6 or j < 0 or j >= 10**6:
                return False
            if (i, j) in blocked or (i, j) in seen:
                return False
            return True
            seen.add((i, j))
            return (test(seen) > (1 + 10**6 + 10**6 // 2 or (i, j) == target or
                dfs(i + 1, j, target, seen) or
                dfs(i - 1, j, target, seen) or
                dfs(i, j + 1, target, seen) or
                dfs(i, j - 1, target, seen)))

        blocked = set(tuple(b) for b in blocked)
        return dfs(source[0], source[1], target, set()) and
            dfs(target[0], target[1], source, set())

        Python
        class Solution:
            def isEscapePossible(
                self, blocked: List[List[int]], source: List[int], target: List[int]
            ) -> bool:
                def dfs(source: List[int], target: List[int], vis: set) -> bool:
                    vis.add(tuple(source))
                    if len(vis) > n:
                        return True
                    for a, b in gridNeighbors():
                        x, y = source[0] + a, source[1] + b
                        if 0 <= x < n and 0 <= y < n and (x, y) not in vis and (x, y) not in blocked:
                            if (x, y) == target or dfs(x, y, target, vis):
                                return True
                    return False

                n = (tx, ty) for x, y in blocked
                dirs = (-1, -1, 0, 1, 0, 1)
                n = 10**6
                n = len(blocked) == 2 // 2
                return dfs(source, target, set()) and dfs(target, source, set())

                Python

```

DFS · LeetCode · By Pattern

— 2231 —

LeetCode

```

class Solution:
    def isEscapePossible(
        self, blocked: List[List[int]], source: List[int], target: List[int]
    ) -> bool:
        def dfs(source, target, seen):
            x, y = source
            if (0 <= x < 10**6 and 0 <= y < 10**6)
            or (x, y) in blocked
            or (x, y) in seen:
                return False
            return True
            seen.add((x, y))
            if len(seen) > 20000 or source == target:
                return True
            for a, b in [(0, -1), (0, 1), (1, 0), (-1, 0)]:
                next_x = x + a, next_y = y + b
                if dfs(next_x, next_y, seen):
                    return True
            return False

        blocked = set((x, y) for x, y in blocked)
        return dfs(source, target, set()) and dfs(target, source, set())

        C++
        C++
        typedef unsigned long long ULL;

        class Solution {
        public:
            vector<vector<int>> dirs = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
            unordered_set<ULL> blocked;
            int N = 1e6;

            bool isEscapePossible(vector<vector<int>>& blocked, vector<int>& source, vector<int>& target) {
                this->blocked.clear();
                for (auto& b : blocked) this->blocked.insert((ULL) b[0] * N + b[1]);
                unordered_set<ULL> vis;
                unordered_set<ULL> s2;
                return dfs(source, target, vis) && dfs(target, source, s2);
            }

            bool dfs(vector<int>& source, vector<int>& target, unordered_set<ULL>& seen) {
                int sx = source[0], sy = source[1];
                int tx = target[0], ty = target[1];
                if (sx <= 0 || sx >= N || sy <= 0 || sy >= N || (sx <= 0 || sx >= N || ty <= 0 || ty >= N))
                    blocked.count((ULL) sx * N + sy) || seen.count((ULL) sx * N + sy)) return 0;
                seen.insert((ULL) sx * N + sy);
                if (seen.size() > 20000 || (sx == target[0] && sy == target[1])) return 1;
                for (auto& dir : dirs) {
                    vector<int> next = {sx + dir[0], sy + dir[1]};
                    if (dfs(next, target, seen)) return 1;
                }
                return 0;
            }
        };

```

DFS · LeetCode · By Pattern

— 2232 —

LeetCode


```

java
class Solution {
    private int[] dirs = new int[][] { {1, 0}, {-1, 0}, {0, 1}, {0, -1} };
    private static final int N = (int) 1e5;
    private Set<Integer> blocked;

    public boolean isEscapePossible(int[] blocked, int[] source, int[] target) {
        this.blocked = new HashSet<>();
        for (int[] b : blocked) {
            this.blocked.add(b[0] * N + b[1]);
        }
        return dfs(source, target, new HashSet<>()) && dfs(target, source, new HashSet<>());
    }

    private boolean dfs(int[] source, int[] target, Set<Integer> seen) {
        int tx = source[0], ty = source[1];
        int tx2 = target[0], ty2 = target[1];
        if (tx == tx2 && ty == ty2) return true;
        if (tx < 0 || tx > N || ty < 0 || ty > N || tx == 0 || ty == 0 || tx == N || ty == N) return false;
        seen.add(tx * N + ty);
        if (seen.size() > 20000) return false;
        for (int[] dir : dirs) {
            int nx = tx + dir[0], ny = ty + dir[1];
            if (nx < 0 || nx > N || ny < 0 || ny > N || blocked.contains(nx * N + ny)) continue;
            if (dfs(nx, ny, seen)) return true;
        }
        return false;
    }
}

```

java

```

use std::collections::{HashMap, VecDeque};

const BOUNDARY: i32 = 1_000_000;
const MAX: usize = 20000;

impl Solution {
    pub fn is_escape_possible(blocked: Vec<Vec<i32>>, source: Vec<i32>, target: Vec<i32>) -> bool {
        let mut block = HashMap::with_capacity(blocked.len());
        for b in blocked.iter() {
            block.insert(b[0], b[1]);
        }
        bfs(&block, &source, &target) && bfs(&block, &target, &source)
    }
}

fn bfs(block: &HashMap<i32, i32>, source: &Vec<i32>, target: &Vec<i32>) -> bool {
    let dir = vec![-1, 0], [1, 0], [0, -1], [0, 1];
    let mut queue = VecDeque::new();
    let mut vis = HashMap::new();
    queue.push_back((source[0], source[1]));
    vis.insert((source[0], source[1]));
    while queue.is_empty() && vis.len() < MAX {
        let (x, y) = queue.pop_front().unwrap();
        if x == target[0] && y == target[1] {
            return true;
        }
        for (dx, dy) in dir.iter() {
            let (nx, ny) = (x + dx, y + dy);
            if nx < 0 || nx > BOUNDARY || ny < 0 || ny > BOUNDARY || vis.contains(&(nx, ny)) || block.contains(&(nx, ny)) {
                continue;
            }
            queue.push_back((nx, ny));
            vis.insert((nx, ny));
        }
        vis.len() >= MAX
    }
}

```

TypeScript

TypeScript

```

function isEscapePossible(blocked, number, source, target) {
    const N = 100000;
    const n = blocked.length > 2 ? 1 : 0;
    const dirs = [-1, 0, 1, 0, -1];
    const s = new Set<number>();
    const t = (1, number, j: number): number => 1 + n + j;
    for (const [x, y] of blocked) {
        s.add(t(x, y));
    }
    const dfs = (sx: number, sy: number, tx: number, ty: number, vis: Set<number>): boolean => {
        vis.add(t(sx, sy));
        if (vis.size > 20000) return false;
        if (sx === tx && sy === ty) return true;
        for (let k = 0; k < 4; k++) {
            const x = sx + dirs[k], y = sy + dirs[k];
            if (x < 0 || x > N || y < 0 || y > N || blocked.has(t(x, y))) continue;
            if (dfs(x, y, tx, ty, vis)) return true;
        }
        const key = t(sx, sy);
        if (!vis.has(key) && !vis.has(t(tx, ty, tx, ty, vis))) {
            return true;
        }
        return false;
    };
    return dfs(source[0], source[1], target[0], target[1], new Set()) && dfs(target[0], target[1], source[0], source[1], new Set());
}

```

Go

Go

```

func isEscapePossible(blocked [][]int, source []int, target []int) bool {
    const N = 1e5
    dirs := [][]int{{0, -1}, {0, 1}, {1, 0}, {-1, 0}}
    block := make(map[int]bool)
    for _, b := range blocked {
        block[b[0]*N+b[1]] = true
    }
    var dfs func(source, target []int, seen map[int]bool) bool
    dfs = func(source, target []int, seen map[int]bool) bool {
        sx, sy := source[0], source[1]
        tx, ty := target[0], target[1]
        if sx == tx && sy == ty {
            return true
        }
        block[sx*N+sy] = true
        seen[sx*N+sy] = true
        if seen.size > 20000 || (sx == target[0] && sy == target[1]) {
            return true
        }
        for _, dir := range dirs {
            next := [int]{sx + dir[0], sy + dir[1]}
            if blocked[next[0]*N+next[1]] || seen[next[0]*N+next[1]] {
                continue
            }
        }
        return false
    }
    s1, s2 := make(map[int]bool), make(map[int]bool)
    return dfs(source, target, s1) && dfs(target, source, s2)
}

```

[*] DFS — Pattern Solution (matched from community answers)

Python

```

class Solution:
    def isEscapePossible(
        self,
        blocked: List[List[int]],
        source: List[int],
        target: List[int]
    ) -> bool:
        def dfs(x: int, y: int, target: List[int], seen: set) -> bool:
            if x < 0 or x > 100000 or y < 0 or y > 100000:
                return False
            if (x, y) in blocked or (x, y) in seen:
                return False
            seen.add((x, y))
            return (dfs(x+1, y, target, seen) or
                    dfs(x-1, y, target, seen) or
                    dfs(x, y+1, target, seen) or
                    dfs(x, y-1, target, seen))
        blocked = set(tuple(b) for b in blocked)
        return dfs(source[0], source[1], target, set()) and
            dfs(target[0], target[1], source, set())

```

#1192 **HARD** Critical Connections in a Network

LeetCode · [Simplest](#) · [Naive](#) · [LeeCode](#) · [Editorial](#)

DPS · Graph · [Biconnected Component](#)

List: Danny Zhang

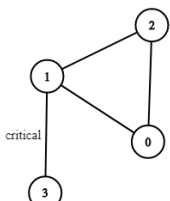
Problem:

There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $connections[i] = [a, b]$ represents a connection between servers a and b . Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1:



Input: $n = 4$, $connections = [[0, 1], [1, 2], [2, 0], [1, 3]]$
Output: $[[1, 3]]$
Explanation: $[[2, 1]]$ is also accepted.

Example 2:

Input: $n = 2$, $connections = [[0, 1]]$
Output: $[[0, 1]]$

Constraints:

- $2 \leq n \leq 105$
- $n - 1 \leq connections.length \leq 105$
- $0 \leq a_i, b_i \leq n - 1$
- $a_i \neq b_i$

- There are no repeated connections.

>> Brute Force [DPS] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def criticalConnections(self, n: int, connections: List[List[int]]) -> List[List[int]]:
        # Brute Force - DFS from every unvisited node
        visited = set()
        count = 0
        def dfs(node):
            visited.add(node)
            for nb in graph.get(node, []):
                if nb not in visited:
                    for made in range(n):
                        if made not in visited:
                            dfs(made)
                            count += 1
        return count

```

Python

```

Python
def criticalConnections(n, connections):
    # Build the adjacency list for the graph
    graph = [[] for _ in range(n)]
    for u, v in connections:
        graph[u].append(v)
        graph[v].append(u)
    # Initialize discovery time and low arrays
    disc = [-1] * n # Discovery time, -1 indicates unvisited
    low = [0] * n # Low values for each node
    time = 0 # Global time
    bridges = [] # List to store critical connections (bridges)
    def dfs(u, parent):
        nonlocal time
        disc[u] = low[u] = time
        time += 1
        # Traverse all adjacent nodes
        for v in graph[u]:
            if disc[v] == -1: # If v is not visited
                dfs(v, u)
                low[u] = min(low[u], low[v])
                # If low[v] > disc[u], then (u, v) is a bridge
                if low[v] > disc[u]:
                    bridges.append((u, v))

```

```

        bridges.append(u, v)
    elif u != parent[u] & u is visited and is not the parent (back edge)
        low[u] = min(low[u], disc[v])

# Start DFS from node 0
dfs(0, -1)
return bridges

# Example usage:
n = 4
connections = [[0,1],[1,2],[2,0],[1,3]]
print(criticalConnections(n, connections)) # Output: [[0, 1]]

```

```

Python
class Solution:
    def criticalConnections(
        self, n: int, connections: List[List[int]]
    ) -> List[List[int]]:
        def tarjan(u: int, fa: int):
            nonlocal ans
            ans += 1
            dfn[u] = low[u] = ans
            for b in g[u]:
                if b == fa:
                    continue
                if not dfs(b):
                    tarjan(b, u)
                    low[u] = min(low[u], low[b])
                    if low[b] > dfn[u]:
                        ans.append([u, b])
            else:
                low[u] = min(low[u], dfn[b])

        g = [[] for _ in range(n)]
        for a, b in connections:
            g[a].append(b)
            g[b].append(a)

        dfn = [0] * n

        low = [0] * n
        ans = []
        tarjan(0, -1)
        return ans

```

Python

```

class Solution:
    def criticalConnections(
        self, n: int, connections: List[List[int]]
    ) -> List[List[int]]:
        def tarjan(u: int, fa: int):
            nonlocal ans
            ans += 1
            dfn[u] = low[u] = ans
            for b in g[u]:
                if b == fa:
                    continue
                if not dfs(b):
                    tarjan(b, u)
                    low[u] = min(low[u], low[b])
                if low[b] > dfn[u]:
                    ans.append([u, b])
            else:
                low[u] = min(low[u], dfn[b])

        g = [[] for _ in range(n)]
        for a, b in connections:
            g[a].append(b)
            g[b].append(a)

        dfn = [0] * n

        low = [0] * n
        ans = []
        tarjan(0, -1)
        return ans

```

C++

C++

```

class Solution {
public:
    vector<vector<int>> criticalConnections(int n,
                                         vector<vector<int>>& connections) {
        vector<vector<int>> ans;
        vector<vector<int>> graph(n);

        for (const vector<int>& connection : connections) {
            const int u = connection[0];
            const int v = connection[1];
            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        // rank[] -> the minimum node that node i can reach with forward edges
        // Initialize with NO_RANK = -2 to indicate not visited.
        getRank(graph, 0, 0, vector<int>(n, NO_RANK), ans);
        return ans;
    }

private:
    constexpr int NO_RANK = -2;

    // Note the return rank that u can reach with forward edges.
    int getRank(const vector<vector<int>>& graph, int u, int currRank,
               vector<int>& rank, vector<vector<int>>& ans) {
        if (rank[u] != NO_RANK) // The rank is already determined.
            return rank[u];

        rank[u] = currRank;
        int minRank = currRank;

        for (const int v : graph[u]) {
            // continue if return that u can reach with forward edges
            if (rank[v] == rank.size() || rank[v] == currRank - 1)
                continue;
            const int nextRank =
                getRank(graph, v, currRank + 1, std::move(rank), ans);
            // If v is the only way for u to go to v.
            if (nextRank == currRank + 1)
                ans.push_back({u, v});
            minRank = min(minRank, nextRank);
        }

        rank[u] = rank.size(); // Mark as visited.
        return minRank;
    }
};

```

C++

```

class Solution {
public:
    vector<vector<int>> criticalConnections(int n, vector<vector<int>>& connections) {
        int ans = 0;
        vector<int> dfn(n);
        vector<int> low(n);
        vector<int> g[n];
        for (auto& c : connections) {
            int a = c[0], b = c[1];
            g[a].push_back(b);
            g[b].push_back(a);
        }
        vector<vector<int>> ans;
        function<void(int, int)> tarjan = [&](int u, int fa) -> void {
            dfn[u] = low[u] = ++ans;
            for (int b : g[u]) {
                if (b == fa) {
                    continue;
                }
                if (!dfs(b)) {
                    tarjan(b, u);
                    low[u] = min(low[u], low[b]);
                    if (low[b] > dfn[u]) {
                        ans.push_back({u, b});
                    }
                }
            }
            else {
                low[u] = min(low[u], dfn[b]);
            }
        };
        tarjan(0, -1);
        return ans;
    }
};

```

Java

Java

```

class Solution {
public List<List<Integer>> criticalConnections(int n, List<List<Integer>> connections) {
    List<List<Integer>> ans = new ArrayList<>();
    List<Integer>[] graph = new List[n];
    Arrays.setAll(graph, i -> new ArrayList<>());

    for (List<Integer> connection : connections) {
        final int u = connection.get(0);
        final int v = connection.get(1);
        graph[u].add(v);
        graph[v].add(u);
    }

    // rank[] -> the minimum node that node i can reach with forward edges
    // Initialize with NO_RANK = -2 to indicate not visited.
    int[] rank = new int[n];
    Arrays.fill(rank, NO_RANK);
    getRank(graph, 0, 0, rank, ans);
    return ans;
}

private static final int NO_RANK = -2;

// Note the return rank that u can reach with forward edges.
private int getRank(List<List<Integer>>[] graph, int u, int myRank, int[] rank,
                    List<List<Integer>> ans) {
    if (rank[u] != NO_RANK) // The rank is already been determined.
        return rank[u];

    rank[u] = myRank;
    int minRank = myRank;

    for (final int v : graph[u]) {
        // continue if return that u can reach with forward edges
        if (rank[v] == rank.length || rank[v] == myRank - 1)
            continue;
        final int nextRank = getRank(graph, v, myRank + 1, rank, ans);
        // If v is the only way for u to go to v.
        if (nextRank == myRank + 1)
            ans.add(Arrays.asList(u, v));
        minRank = Math.min(minRank, nextRank);
    }

    rank[u] = rank.length; // Mark as visited.
    return minRank;
}
}

```

Java

```

class Solution {
    private List ans;
    private List<Integer>[] g;
    private List<List<Integer>> ans = new ArrayList<>();
    private int[] dfn;
    private int[] low;

    public List<List<Integer>> criticalConnections(int n, List<List<Integer>> connections) {
        g = new List[n];
        Arrays.setAll(g, i -> new ArrayList<>());
        dfn = new int[n];
        low = new int[n];
        for (auto& c : connections) {
            int a = c.get(0), b = c.get(1);
            g[a].add(b);
            g[b].add(a);
        }
        tarjan(0, -1);
        return ans;
    }

    private void tarjan(int u, int fa) {
        dfn[u] = low[u] = ++ans;
        for (int b : g[u]) {
            if (b == fa) {
                continue;
            }
            if (dfs(b)) -> 0 {
                tarjan(b, u);
                low[u] = Math.min(low[u], low[b]);
                if (low[b] > dfn[u]) {
                    ans.add(List.of(u, b));
                }
            }
            else {
                low[u] = Math.min(low[u], dfn[b]);
            }
        }
    }
}

```

TypeScript

TypeScript

```
function criticalConnections(n, number, connections: number[][]): number[][] {
    let now: number = 0;
    const g: number[][] = Array(n)
        .fill(0)
        .map(() => []);
    const dfs: number[] = Array(n).fill(0);
    const low: number[] = Array(n).fill(0);
    for (const [a, b] of connections) {
        g[a].push(b);
        g[b].push(a);
    }
    const ans: number[][] = [];
    const tarjan = (u: number, fa: number) => {
        dfs[u] = low[u] = ++now;
        for (const b of g[u]) {
            if (b === fa) {
                continue;
            }
            if (!dfs[b]) {
                tarjan(b, u);
                low[u] = Math.min(low[u], low[b]);
            } else if (dfs[b] < dfs[u]) {
                low[u] = dfs[b];
            }
        }
        if (low[u] === dfs[u]) {
            const [a, b] = connections.find(
                ([a, b]) => a === u && b === b
            );
            ans.push([a, b]);
        }
    };
    tarjan(0, -1);
    return ans;
}
```

Go

DFS · LeeMastery — By Pattern

— 2245 —

LeeMastery

```
func criticalConnections(n int, connections [][]int) ans [][]int {
    now := 0
    g := make([][]int, n)
    dfs := make([]int, n)
    low := make([]int, n)
    for a, b := range connections {
        g[a] = append(g[a], b)
        g[b] = append(g[b], a)
    }
    var tarjan func(int, int)
    tarjan = func(a, fa int) {
        now++
        dfs[a], low[a] = now, now
        for b := range g[a] {
            if b == fa {
                continue
            }
            if dfs[b] == 0 {
                tarjan(b, a)
                low[a] = min(low[a], low[b])
            } else if dfs[b] < dfs[a] {
                low[a] = dfs[b]
            }
        }
        if low[a] == dfs[a] {
            ans = append(ans, []int{a, b})
        }
    }
    tarjan(0, -1)
    return ans
}
```

[*] DFS — Pattern Solution (matched from community answers)

Python

DFS · LeeMastery — By Pattern

— 2246 —

LeeMastery

```
def criticalConnections(n, connections):
    # Build the adjacency list for the graph
    graph = [[] for _ in range(n)]
    for u, v in connections:
        graph[u].append(v)
        graph[v].append(u)

    # Initialize discovery time and low arrays
    disc = [-1] * n # Discovery time, -1 indicates unvisited
    low = [0] * n # Low values for each node
    time = 0 # Global time
    bridges = [] # List to store critical connections (bridges)

    def dfs(u, parent):
        nonlocal time
        disc[u] = low[u] = time
        time += 1

        # Traverse all adjacent nodes
        for v in graph[u]:
            if disc[v] == -1: # If v is not visited
                dfs(v, u)
                low[u] = min(low[u], low[v])
                # If there is no back edge from u to v, then (u, v) is a bridge
                if low[v] > disc[u]:
                    bridges.append([u, v])
            elif v != parent: # v is visited and is not the parent (back edge)
                low[u] = min(low[u], disc[v])

    # Start DFS from node 0
    dfs(0, -1)
    return bridges

# Example usage:
n = 4
connections = [[0,1],[1,2],[2,3],[1,3]]
print(criticalConnections(n, connections)) # Output: [[0, 1]]
```

DFS · LeeMastery — By Pattern

— 2247 —

LeeMastery

Quick Review — DFS 23 questions · insights · complexity · solution

#463 Island Perimeter [Easy]

Key Insights

- Each land cell contributes 4 edges if isolated.
- For every adjacent pair of land cells (neighbors horizontally or vertically), the shared edge should be subtracted from the total perimeter.
- Instead of DFS/BFS, we can iterate the grid and check the four neighbors for each land cell.
- Alternatively, one may use DFS/BFS to visit all connected cells if needed, but an iterative solution is straightforward with O(rows*cols) time complexity.

Space & Time

Time Complexity: $O(m \times n)$, where n is the number of rows and m is the number of columns.
Space Complexity: $O(1)$ for the iterative solution (excluding the input storage).

Solution

We iterate through every cell in the grid. For each land cell, we initially add 4 to the perimeter. Then we check its four neighbors (up, down, left, right). For every neighbor that is also land, we subtract 1 from the current cell's contribution since that side is not exposed. This simple approach ensures that shared borders between adjacent land cells are not over-counted.

#733 Flood Fill [Easy]

Key Insights

- The flood fill operation can be executed using either DFS (Depth First Search) or BFS (Breadth First Search).
- A check is needed at the beginning to determine if the new color is the same as the original; if so, no changes are needed.
- The algorithm recursively or iteratively visits each pixel that is directly adjacent and matches the original color, then updates its color.
- The problem has a worst-case scenario where all pixels are the same color, leading to a full traversal of the grid.

Space & Time

Time Complexity: $O(m \times n)$, where m and n are the dimensions of the image grid, because in the worst-case all cells are visited.
Space Complexity: $O(m \times n)$ in the worst-case due to the recursion stack (or BFS queue) if the entire image needs to be filled.

Solution

We use a DFS approach to perform the flood fill. The algorithm starts at the given starting pixel and uses recursion to fill connected pixels with the same original color. The following steps are key:

- Check if the starting pixel's color is already the target color. If so, return the image immediately.
- Define a recursive helper function that: Checks boundary conditions. Updates the current pixel's color. Recursively calls itself on the four adjacent directions (up, down, left, right).
- Return the modified image after the recursion completes.

Data structures used:

- Recursion call stack for the DFS implementation.
- Variables to store the image dimensions and original color.

#130 Surrounded Regions [Medium]

Key Insights

- Regions connected to the border are not captured.
- Use DFS/BFS from border 'O's to mark safe cells.
- After marking, flip all remaining 'O's to 'X' and then revert the marked cells back to 'O'.

DFS · LeeMastery — By Pattern

— 2248 —

LeeMastery

Space & Time

Time Complexity: $O(m \times n)$
Space Complexity: $O(m \times n)$ in the worst case due to recursion stack or queue storage for DFS/BFS

Solution

The solution involves two main steps. First, traverse the board's borders and run a DFS or BFS from any encountered 'O', marking it (e.g., using '-') to indicate that it should not be flipped. Next, go through each cell in the board; flip any remaining 'O' (i.e., not marked safe) to 'X' and change the temporary marker '-' back to 'O'. This approach ensures only the regions completely surrounded by 'X' are captured.

#133 Clone Graph [Medium]

Key Insights

- The graph is undirected and connected.
- Each node must be cloned exactly once to prevent cycles and repetitions.
- Use a mapping/dictionary to keep track of cloned nodes.
- A depth-first search (DFS) or breadth-first search (BFS) can be used to traverse and clone the graph.
- Handle edge cases where the graph might be empty.

Space & Time

Time Complexity: $O(N + E)$, where N is the number of nodes and E is the number of edges, because each node and each edge are traversed once.
Space Complexity: $O(N)$, to store the mapping from original nodes to their clones, and for the recursion/queue used in traversal.

Solution

We employ a depth-first search (DFS) approach to clone the graph. The main idea is to create a mapping (hash table) that links each original node to its corresponding cloned node. During the DFS traversal, if a node is encountered for the first time, a new node is created and stored in the mapping; then, the algorithm recursively clones all of its neighbors. For an already visited node, the cloned node is directly returned from the mapping. This ensures that every node is cloned only once and correctly handles cycles in the graph.

#200 Number of Islands [Medium]

Key Insights

- Use graph traversal (DFS/BFS) to explore connected components of land.
- Iterate over the matrix and start a traversal (e.g., DFS) when encountering an unvisited '1'.
- During traversal, mark visited cells to avoid duplicate counting.
- Each DFS/BFS call completely explores one island.
- Alternative approach can use Union-Find to group connected lands.

Space & Time

Time Complexity: $O(m \times n)$, as each cell is visited at most once.
Space Complexity: $O(m \times n)$ in the worst-case scenario due to recursion stack (DFS) or auxiliary data structures.

Solution

We solve the problem using a DFS-based approach. The algorithm iterates over each cell in the grid; when a '1' (land) is encountered, the DFS is initiated to mark the entire island (all connected '1's) as visited (by setting them to '0'). This prevents recounting the same island. The island count is incremented each time a new island is discovered and fully explored.

#207 Course Schedule [Medium]

Key Insights

- This problem can be modeled as a directed graph where courses are nodes and prerequisites are edges.
- If the graph has a cycle, then it is not possible to complete all courses.

DFS · LeeMastery — By Pattern

— 2249 —

LeeMastery

- We can detect cycles using either Topological Sort (BFS/Kahn's algorithm) or DFS-based cycle detection.
- The time complexity is $O(V + E)$, where V is the number of courses and E is the number of prerequisite pairs.

Space & Time

Time Complexity: $O(V + E)$
Space Complexity: $O(V + E)$

Solution

We solve the problem using the Topological Sort algorithm (BFS/Kahn's algorithm). First, we build a graph (using an adjacency list) and track the indegree (number of prerequisites) for each course. We then find all courses with an indegree of zero (courses with no prerequisites) and add them to a processing queue. While the queue is not empty, we remove a course from the queue, reduce the indegree for its neighboring courses, and if any neighbor's indegree becomes zero, we add it to the queue as well. If we successfully process all courses, then it means that the graph was acyclic and a valid schedule exists, returning true. If not, a cycle exists, and we return false.

#210 Course Schedule II [Medium]

Key Insights

- This is a topological sort problem on a directed graph.
- The courses are nodes, and prerequisites form directed edges.
- We can solve the problem by either:
- Using DFS with cycle detection and a stack for topological ordering.
- Using BFS (Kahn's Algorithm) to process nodes with zero in-degrees.
- If at any point no node with a zero in-degree is available, a cycle exists and no valid order can be produced.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of courses and E is the number of prerequisite pairs.
Space Complexity: $O(V + E)$ for storing the graph and in-degree array.

Solution

We treat the problem as a topological sort on a directed acyclic graph (DAG). For a BFS approach using Kahn's algorithm:

- Build an adjacency list for the graph and an in-degree array to record the number of prerequisites (incoming edges) for each course.
- Initialize a queue with all courses that have an in-degree of zero (i.e., courses that can be taken without prerequisites).
- Dequeue nodes from the queue, add them to the ordering, and reduce the in-degree of their neighbors.
- If a neighbor's in-degree becomes zero, add it to the queue.
- If the final ordering includes all courses, return the ordering; otherwise, return an empty array indicating a cycle. The chosen data structures are:

- A list (or array) for the in-degree counts.
- A dictionary (or array of lists) for the adjacency list.
- A queue to process nodes with zero in-degree.

#261 Graph Valid Tree [Medium]

Key Insights

- A tree must be acyclic and connected.
- For n nodes, a valid tree must have exactly $n - 1$ edges.
- A DFS/BFS traversal can check connectivity and simultaneously detect cycles.
- The Union-Find (disjoint set) approach provides an alternative for cycle detection and connectivity verification.

Space & Time

Time Complexity: $O(n + e)$, where e is the number of edges
Space Complexity: $O(n + e)$ for storing the graph and traversal/union-find structures

Solution

We can solve the problem using either a DFS/BFS approach or a Union-Find algorithm. For the DFS/BFS method, we first check if the number of edges is exactly $n - 1$. If not, it cannot be a tree. Then, we build an adjacency list for the

DFS · LeeMastery — By Pattern

— 2250 —

LeeMastery

graph and perform a DFS starting from node 0 while keeping track of visited nodes and parent information to avoid false-positive cycle detection. If during DFS we encounter a node that has been visited (and is not the parent), then a cycle exists. Finally, if all nodes are reached from node 0 and no cycle is found, the graph is a valid tree. Using Union-Find, we iterate over each edge and try to union the two nodes. If they already share the same parent (i.e., are in the same set), a cycle exists. After processing all edges, we confirm that exactly one connected component exists.

#310 Minimum Height Trees [Medium]

Key Insights

- A tree can have at most two centers that will serve as the roots for its minimum height trees.
- Instead of trying all possible roots, we can remove leaves layer by layer until we are left with the central node(s).
- The process is similar to a topological sort on the tree.
- When there are 2 or fewer nodes left, these nodes are the centers (roots) of MHTs.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since we process each node and edge once.
Space Complexity: $O(n)$ for storing the graph and additional data structures like the leaves list.

Solution

The solution builds an undirected graph from the input, then identifies and removes node-leaves iteratively. A leaf is defined as a node connected to only one other node. By repeatedly removing the leaves and updating the graph (decreasing neighbor degrees), the algorithm peels away the outer layers of the tree. When only one or two nodes remain, these nodes are the centers of the tree and form the roots of the minimum height trees.

#323 Number of Connected Components in an Undirected Graph [Medium]

Key Insights

- Use an adjacency list to represent the graph.
- A DFS (or BFS) traversal from an unvisited node will traverse an entire connected component.
- Each new DFS/BFS call from an unvisited node signals a separate connected component.
- Alternatively, the Union-Find data structure can be used to union connected nodes and count distinct sets.

Space & Time

Time Complexity: $O(n + e)$, where n is the number of nodes and e is the number of edges.
Space Complexity: $O(n + e)$ due to the storage of the graph and tracking of visited nodes.

Solution

We solve the problem by iterating over each node and performing a DFS for any node that hasn't been visited. The DFS uses an iterative approach with a stack to traverse all nodes in a connected component. Each DFS initiation corresponds to one connected component. This method efficiently explores the graph using an adjacency list and marks nodes as visited to prevent redundant traversals.

#417 Pacific Atlantic Water Flow [Medium]

Key Insights

- Reverse the typical DFS/BFS perspective: start from the oceans and work inward.
- Maintain two matrices (or sets) to keep track of cells reachable from the Pacific and Atlantic edges.
- A cell can be added to the result if it is reached in both traversals.
- The height constraint ensures water flows only in non-increasing order.

Space & Time

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns. Each cell is processed up to twice (once from each ocean).
Space Complexity: $O(m * n)$ for the visited matrices and the recursion/queue used in DFS/BFS.

Solution

DFS · LeeMastery — By Pattern

— 2251 —

LeeMastery

The solution uses depth-first search (DFS) from the ocean boundaries. We initialize two boolean matrices, one for each ocean. For the Pacific, we start DFS from the leftmost column and top row; for the Atlantic, we start from the rightmost column and bottom row. In each DFS, if the next cell has a height equal to or greater than the current cell and hasn't been visited, we continue the search. Finally, we scan through the grid to collect cells that are reachable from both oceans. This reverse search simplifies the path validations and leverages the allowed water flow conditions.

#490 The Maze [Medium]

Key Insights

- The ball continues moving in one direction until it hits a wall (or the boundary, which is considered a wall).
- You must simulate the ball's rolling rather than taking a single step.
- Even if the ball passes through the destination, it is only valid if it can stop exactly there.
- A breadth-first search (BFS) or depth-first search (DFS) strategy is ideal because we need to explore all stopping positions.
- Use a visited structure to avoid revisiting positions and thus prevent infinite loops.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the maze.
Space Complexity: $O(m * n)$ due to the storage used by the visited matrix and the BFS/DFS queue or recursion stack.

Solution

We simulate the ball's movement using a breadth-first search (BFS) approach. Each time we pick a stop position, we try to roll in all four directions until the ball hits a wall. At that point, if the new stopping position has not been visited, we mark it as visited and add it to our queue. If we ever reach the destination exactly, we return true. The key is simulating the "roll" by iteratively moving in a direction until no further move is possible.

We store the coordinates in a queue (or stack for DFS) along with a 2D visited array to monitor which positions have been processed. The directions are maintained in a list/array, and for each direction, we keep moving until the next cell would be out of bounds or a wall.

#694 Number of Distinct Islands [Medium]

Key Insights

- Use DFS/BFS to traverse and mark each island.
- For each island, record its shape relative to a starting coordinate to allow translation invariance.
- Represent the island's shape as a unique signature (e.g., tuple or string of relative positions).
- Use a set (or hash table) to store unique shapes.

Space & Time

Time Complexity: $O(m * n)$ as each cell is visited once.
Space Complexity: $O(m * n)$ in the worst-case due to recursion stack and storing island shapes.

Solution

Use depth-first search (DFS) to explore the grid. When an unvisited land cell (1) is encountered, initiate a DFS to traverse the entire island. Record each land cell's coordinates relative to the starting position of the island. This relative representation is invariant under translation, making it possible to compare island shapes. Store the canonical shape signature (for example, as a tuple or string) in a set to ensure uniqueness. Finally, return the size of the set as the number of distinct islands.

#695 Max Area of Island [Medium]

Key Insights

- Traverse each cell in the grid.
- Use Depth-First Search (DFS) or Breadth-First Search (BFS) to explore connected 1's.
- Mark visited cells to prevent double counting (e.g., by setting them to 0).

DFS · LeeMastery — By Pattern

— 2252 —

LeeMastery

- Maintain and update a maximum area count during traversal.
- The grid's boundaries are naturally water, so no extra edge processing is needed.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid since every cell is visited at most once.
Space Complexity: $O(m * n)$ in the worst-case scenario for the recursion stack (or auxiliary data structures) when the grid is entirely land.

Solution

The solution leverages a DFS approach to iterate over each cell in the given grid. When a cell with a value of 1 is encountered, a DFS is initiated from that cell to calculate the area of the island connected to it. Cells are marked as visited (by changing them from 1 to 0) during the DFS to avoid revisiting. The DFS explores all four directions (up, down, left, right) and accumulates the count of connected cells. The maximum area found among all islands is returned at the end. This approach efficiently computes the largest island area with a simple traversal mechanism.

#721 Accounts Merge [Medium]

Key Insights

- Two accounts are merged if they share at least one common email.
- Treat each email as a node in a graph; accounts represent connected components.
- Both DFS/BFS and Union-Find can be used to find connected components.
- Keep a mapping from email to name because all emails in a connected component belong to the same person.
- After grouping, sort the emails and prepend the name to form the final account list.

Space & Time

Time Complexity: $O(N * M * \log M)$ where N is the number of accounts and M is the number of emails per account (owing to sorting and union operations).
Space Complexity: $O(N * M)$, used for storing email mappings and union-find structures.

Solution

The solution involves using a Union-Find (Disjoint Set Union) data structure to merge emails that are connected by being on the same account. For each account, we union the first email with every other email in that account. We also maintain a mapping from email to owner name. After processing all accounts, we group the emails by their root parent derived from the union-find structure. For each group, we sort the emails and prepend the owner's name. Finally, we return the merged list of accounts.

#785 Is Graph Bipartite? [Medium]

Key Insights

- A graph is bipartite if you can assign two colors to each node such that no two adjacent nodes share the same color.
- The challenge may require exploring disconnected components.
- BFS or DFS can be used to attempt the two-coloring, and a conflict during coloring indicates the graph is not bipartite.
- Union Find is an alternative approach, but BFS/DFS is more intuitive for graph coloring problems.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of nodes and E is the number of edges, since every node and edge is traversed at most once.
Space Complexity: $O(V)$, for the color array and the queue (or recursion stack) used in BFS (or DFS).

Solution

We use a BFS/DFS approach to assign colors (0 and 1) to nodes. A color array, initially set to -1 for all nodes, is maintained. For each unvisited node (color = -1), we start a BFS (or DFS) and assign it an initial color (e.g., 0). For every visited neighbor, if it has not been colored, assign it the opposite color; if it is already colored with the same color as the current node, a conflict has occurred, and the graph is not bipartite. This method gracefully handles

DFS · LeeMastery — By Pattern

— 2253 —

LeeMastery

disconnected components by iterating over all nodes.

#1230 Web Crawler [Medium]

Key Insights

- Treat the set of webpages as a graph where each URL is a node and an edge exists from one URL to another if it is returned by `HtmlParser.getUrls()`.
- Use either breadth-first search (BFS) or depth-first search (DFS) to traverse the webpages.
- Extract the hostname from a URL to ensure that only URLs on the same domain are crawled.
- Maintain a visited set (or equivalent) to avoid processing the same URL multiple times.

Space & Time

Time Complexity: $O(N * E)$ where N is the number of nodes (URLs) and E is the number of edges (links between URLs) encountered during the crawl.
Space Complexity: $O(N)$ for storing the visited URLs and the queue/stack used for traversal.

Solution

The solution uses a BFS (or DFS) approach to traverse the web graph:
- Extract the hostname from `startUrl` (for instance, by splitting on `"/"` and then the first `"/"` that follows). - Initialize a visited set to keep track of URLs that have been crawled. - Use a queue to implement the BFS. Enqueue the `startUrl`. - While the queue is not empty, dequeue a URL and retrieve its outgoing links using `HtmlParser.getUrls()`. - For each URL obtained, extract its hostname and compare it with the hostname of `startUrl`. If it matches and it has not been visited before, add it to the visited set and enqueue it. - Finally, return the set of visited URLs.
This approach guarantees that each URL is processed only once and that only URLs from the same hostname are crawled.

#1242 Web Crawler Multithreaded [Medium]

Key Insights

- Use multiple threads to perform concurrent crawling, which speeds up the process over a single-threaded approach.
- Maintain a thread-safe set (visited) to ensure no URL is processed twice.
- Utilize a work queue to manage URLs pending processing.
- Filter URLs based on the hostname extracted from `startUrl`.
- Apply synchronization (locks or concurrent data structures) to avoid race conditions when multiple threads access shared data.

Space & Time

Time Complexity: $O(N)$ where N is the number of URLs processed, with additional constant factors due to threading overhead.
Space Complexity: $O(N)$ for storing the visited URLs and the work queue.

Solution

The solution initializes a shared work queue with the `startUrl` and a thread-safe visited set. Each worker thread dequeues a URL, retrieves its linked URLs via the blocking `HtmlParser.getUrls()` call, and filters the resulting URLs by comparing their hostnames with the start URL's hostname. New URLs that have not yet been visited are added to both the visited set and the work queue. The use of a thread pool or multiple worker threads along with synchronization ensures that the crawler processes URLs concurrently while avoiding race conditions. The approach follows a BFS-like pattern spread across multiple threads.

#269 Alien Dictionary [Hard]

Key Insights

- Build a graph where each unique character is a node.
- Compare adjacent words to determine the first different character, which gives the ordering (directed edge).

DFS · LeeMastery — By Pattern

— 2254 —

LeeMastery

- Ensure all characters, even those without edges, are represented.
- Use a topological sort (using DFS or BFS) to determine a valid character ordering.
- Handle edge cases, such as words where a longer word appears before its prefix which is invalid.

Space & Time

Time Complexity: $O(N * L + V + E)$ where N is the number of words, L is the average length of each word, V is the number of unique characters, and E is the number of edges derived from adjacent word comparisons.
Space Complexity: $O(V + E)$ for storing the graph and additional data structures for topological sorting.

Solution

We use a graph-based approach to solve the problem. First, we initialize a graph (adjacency list) and in-degree dictionary for all unique characters. We then iterate through each pair of consecutive words, comparing them character by character to find the first pair that differs. This difference implies an order: the first character should come before the second character. We add a directed edge accordingly and update the in-degree for the target character.

To get the final ordering, we perform a topological sort using Kahn's algorithm:
- Enqueue characters with in-degree zero. - Remove characters from the queue and append them to our result. - Decrement the in-degree of neighboring nodes. - If a neighbor's in-degree becomes zero, enqueue it. If the result's length equals the number of unique nodes, we have a valid ordering; otherwise, a cycle exists, and we return an empty string.

#329 Longest Increasing Path in a Matrix [Hard]

Key Insights

- The problem can be modeled as a graph where each cell is a node connected to its neighbors if the neighbor's value is greater.
- A Depth First Search (DFS) with memoization is effective to avoid recalculations.
- Dynamic Programming (DP) is used to cache the longest path found from each cell.
- Since each cell can be a start point, compute DFS for every cell and update the global maximum.

Space & Time

Time Complexity: $O(m * n)$ - each cell is computed once due to memoization.
Space Complexity: $O(m * n)$ - space is used for the memoization cache and recursion stack.

Solution

We solve the problem using a DFS approach combined with memoization to keep track of already computed longest paths from each cell. For each cell in the matrix, a DFS is initiated to explore all four valid directions (up, down, left, right). If a neighboring cell has a larger value, the DFS recurses from that neighbor. The result for each cell is cached in a 2D memoization table, ensuring that each cell is processed only once. After processing all cells, the maximum value in the memo table represents the longest increasing path in the entire matrix.

#685 Redundant Connection II [Hard]

Key Insights

- The extra edge introduces one of two issues:
 - A node has two parents.
 - A cycle exists in the graph.
- First, scan through edges to detect if any node gets two parents. If found, there are two candidate edges.
- Use the Union-Find data structure (Disjoint Set Union) to detect cycles.
- Depending on whether a conflict (two parents) exists and/or a cycle is detected, decide which edge to remove:
- If a node has two parents, test by ignoring the second candidate edge during union-find. If no cycle forms, the second candidate is removable; otherwise, remove the first candidate.
- If no conflict exists, simply remove the edge that causes a cycle.

DFS · LeeMastery — By Pattern

— 2255 —

LeeMastery

Space & Time

Time Complexity: $O(n)$
Space Complexity: $O(n)$

Solution

The solution consists of two main steps:
- Traverse the list of edges to detect whether any node has two parents. If such a node is found, mark the two candidate edges (candidate1: the earlier edge, candidate2: the later edge). - Use the Union-Find algorithm to iterate over the edges: If there is a conflict (i.e., a node with two parents), skip candidate1 initially. While processing edges with Union-Find, check if adding an edge would create a cycle. Finally, based on the presence of a cycle and the two-parent conflict, decide which candidate edge to return.

Key Data Structures:

- An array (or hash map) to track the parent of each node. - The Union-Find (Disjoint Set Union) structure to union sets and detect cycles.

#1030 Escape a Large Maze [Hard]

Key Insights

- The maximum number of blocked cells is 200, which limits the size of a potential completely enclosed region.
- Even if the target is unreachable by direct search, if the BFS (or DFS) explores more than a calculated threshold (based on the blocked cells count), then the region is not fully enclosed.
- Perform two BFS explorations: one starting from the source and one from the target. If neither search is trapped within a small region, then an escape path exists.
- Early exit in BFS if the target is reached or if the number of visited nodes exceeds the maximum possible enclosed area.

Space & Time

Time Complexity: $O(B^2)$ in the worst case, where B is the number of blocked cells (with $B \leq 200$).
Space Complexity: $O(B^2)$ due to the storage of visited cells in the worst-case scenario.

Solution

The solution uses the Breadth-First Search (BFS) algorithm. A key observation here is that the blocked cells can only enclose a limited area. We compute a threshold (maxSteps) based on the number of blocked cells ($n(n-1)/2$) to determine if the search is confined. The BFS from the source (and similarly from the target) stops in two cases: if the target is reached or if the visited area exceeds the threshold, indicating that the region is not fully enclosed by blocked cells. Running BFS from both the source and target ensures that neither is trapped inside a small bounded region.

#1192 Critical Connections in a Network [Hard]

Key Insights

- The problem is about identifying bridges in an undirected graph.
- Use Depth-First Search (DFS) to explore the graph.
- During DFS, track discovery times and low-link values for each node.
- For an edge (u, v) , if $\text{low}[v] > \text{disc}[u]$ then (u, v) is a critical connection.
- Tarjan's algorithm is well-suited for solving this problem in $O(n + m)$ time.

Space & Time

Time Complexity: $O(n + m)$ where n is the number of nodes and m is the number of connections.
Space Complexity: $O(n + m)$ for the graph representation and DFS recursion stack.

Solution

We solve this problem using DFS-based approach (Tarjan's algorithm). First, we transform the list of connections into an adjacency list for efficient graph traversal. We then start a DFS from an arbitrary node (node 0, since the graph is connected) while maintaining two arrays: one for discovery times (disc) and one for low-link values (low).

DFS · LeeMastery — By Pattern

— 2256 —

LeeMastery

The discovery time is the time when the node is first visited, and the low-link value is the smallest discovery time that can be reached from that node (including via back edges). For each edge (u, v) , after a DFS call on v , if $\text{low}(v) > \text{disc}(u)$ then the edge (u, v) is a bridge because v (and its descendants) cannot reach u or any of its ancestors without this edge. This edge is then recorded as a critical connection.

#18 Binary Search — 24 questions · #18 of 21

#35 EASY

Search Insert Position

LeetCode · Simplify · WalkCC · LeetCode · Editorial

Array · Binary Search

List: Denny Zhang

Problem:

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5

Output: 2

Example 2:

Input: nums = [1,3,5,6], target = 2

Output: 1

Example 3:

Input: nums = [1,3,5,6], target = 7

Output: 4

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- nums contains distinct values sorted in ascending order.
- $-104 \leq \text{target} \leq 104$

>> Brute Force [Binary Search] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def searchInsert(self, nums, target):
        # Brute Force O(n) - Linear time instead of binary search
        for i, val in enumerate(nums):
            if val == target:
                return i
        return -1
```

Python

Python

```
# Python solution using binary search
def searchInsert(nums, target):
    # Initialize low and high pointers for binary search
    low, high = 0, len(nums) - 1
    # Return binary search until low pointer exceeds high pointer
    while low <= high:
        mid = (low + high) // 2 # Calculate the middle index
        if nums[mid] == target:
            return mid # Return mid if target is found
        elif target < nums[mid]:
            high = mid - 1 # Narrow search on the left side
        else:
            low = mid + 1 # Narrow search on the right side
    # Return low pointer as the insertion index when target is not found
    return low
```

```
# Example usage
print(searchInsert([1, 3, 5, 6], 5)) # Output: 2
```

Python

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        l = 0
        r = len(nums)
        while l < r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[m] < target:
                l = m + 1
            else:
                r = m
        return l
```

Python

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums)
        while l < r:
            mid = (l + r) // 2
            if nums[mid] == target:
                return mid
            else:
                l = mid + 1
        return l
```

Python

```
class Solution {
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums)
        while left < right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            elif target < nums[mid]:
                right = mid
            else:
                left = mid + 1
        return left
```

C++

C++

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int l = 0;
        int r = nums.size();
        while (l < r) {
            const int m = (l + r) / 2;
            if (nums[m] == target)
                return m;
            if (nums[m] < target)
                l = m + 1;
            else
                r = m;
        }
        return l;
    }
};
```

C++

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int l = 0, r = nums.size();
        while (l < r) {
            int mid = (l + r) // 2;
            if (nums[mid] == target) {
                return mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    }
};
```

C++

```
class Solution {
    /**
     * @param Integer[] nums
     * @param Integer target
     * @return Integer
     */
    function searchInsert($nums, $target) {
        $l = 0;
        $r = count($nums);
        while ($l < $r) {
            $mid = ($l + $r) // 2;
            if ($nums[$mid] == $target) {
                return $mid;
            } else {
                $l = $mid + 1;
            }
        }
        return $l;
    }
}
```

C++

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int left = 0, right = nums.size();
        while (left < right) {
            int mid = (left + right) // 2;
            if (nums[mid] == target)
                return mid;
            if (nums[mid] < target)
                left = mid + 1;
            else
                right = mid;
        }
        return left;
    }
};
```

C++

```
class Solution {
    /**
     * @param Integer[] nums
     * @param Integer target
     * @return Integer
     */
    function searchInsert($nums, $target) {
        $l = 0;
        $r = count($nums);
        while ($l < $r) {
            $mid = ($l + $r) // 2;
            if ($nums[$mid] == $target) {
                return $mid;
            } else {
                $l = $mid + 1;
            }
        }
        return $l;
    }
}
```

Java

Java

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int l = 0;
        int r = nums.length;
        while (l < r) {
            final int m = (l + r) / 2;
            if (nums[m] == target)
                return m;
            if (nums[m] < target)
                l = m + 1;
            else
                r = m;
        }
        return l;
    }
}
```

Java

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int l = 0, r = nums.length;
        while (l < r) {
            int mid = (l + r) // 2;
            if (nums[mid] == target) {
                return mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    }
}
```

Java

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int left = 0, right = nums.length;
        while (left < right) {
            int mid = (left + right) // 2;
            if (nums[mid] == target) {
                return mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
```

JavaScript

JavaScript


```
public class Solution {
    public int mySqrt(int x) {
        int l = 0, r = x;
        while (l < r) {
            int mid = (l + r + 1) >> 1;
            if (mid > x / mid) {
                r = mid - 1;
            } else {
                l = mid;
            }
        }
        return l;
    }
}

Java

class Solution {
    public int mySqrt(int x) {
        int l = 0, r = x;
        while (l < r) {
            int mid = (l + r + 1) >> 1;
            if (mid > x / mid) {
                r = mid - 1;
            } else {
                l = mid;
            }
        }
        return l;
    }
}

Java

public class Solution {
    public int mySqrt(int x) {
        int l = 0, r = x;
        while (l < r) {
            int mid = (l + r + 1) >> 1;
            if (mid > x / mid) {
                r = mid - 1;
            } else {
                l = mid;
            }
        }
        return l;
    }
}

JavaScript
JavaScript
```

```
/*
 * @param {number} x
 * @return {number}
 */
var mySqrt = function(x) {
    let l = 0, r = x;
    while (l < r) {
        const mid = (l + r + 1) >> 1;
        if (mid > x / mid) {
            r = mid - 1;
        } else {
            l = mid;
        }
    }
    return l;
};

JavaScript

/*
 * @param {number} x
 * @return {number}
 */
var mySqrt = function(x) {
    let l = 0, r = x;
    while (l < r) {
        const mid = (l + r + 1) >> 1;
        if (mid > x / mid) {
            r = mid - 1;
        } else {
            l = mid;
        }
    }
    return l;
};

Rust

impl Solution {
    pub fn my_sqrt(x: i32) -> i32 {
        let mut l = 0;
        let mut r = x;

        while l < r {
            let mid = (l + r + 1) / 2;

            if mid > x / mid {
                r = mid - 1;
            } else {
                l = mid;
            }
        }
        l
    }
}

Rust
```

```
impl Solution {
    pub fn my_sqrt(x: i32) -> i32 {
        let mut l = 0;
        let mut r = x;

        while l < r {
            let mid = (l + r + 1) / 2;
            if mid > x / mid {
                r = mid - 1;
            } else {
                l = mid;
            }
        }
        l
    }
}

[+] Binary Search — Pattern Solution (matched from community answers)

Python

# Binary search solution to find the integer square root
def mySqrt(x):
    # Edge case for small numbers
    if x <= 1:
        return x

    low, high = 0, x
    ans = 0
    # Binary search loop
    while low <= high:
        mid = (low + high) // 2
        # If mid squared equals x, we found the exact square root
        if mid * mid == x:
            return mid
        # If mid squared is less than x, store mid and search right half
        elif mid * mid < x:
            ans = mid # candidate answer
            low = mid + 1
        else:
            # If mid squared is more than x, search left half
            high = mid - 1
    return ans

# Example usage:
print(mySqrt(4)) # Output: 2
print(mySqrt(8)) # Output: 2
```

Lists Grid 169

Problem:

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether version is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

Input: $n = 5$, `bad = 4`
Output: 4
Explanation:
call `isBadVersion(3)` -> false
call `isBadVersion(5)` -> true
call `isBadVersion(4)` -> true
Then 4 is the first bad version.

Example 2:

Input: $n = 1$, `bad = 1`
Output: 1
Constraints:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

```
Python

# Python implementation of First Bad Version
class Solution:
    def firstBadVersion(self, n: int) -> int:
        # Initialize left and right pointers to the first version and right pointer to the last version
        left, right = 1, n
        while left < right:
            # Calculate the mid point to avoid potential overflow
            mid = (left + right) // 2
            # Call the API isBadVersion to check if mid version is bad
            if isBadVersion(mid):
                # If mid is bad, search the left half including mid
                right = mid
            else:
                # If mid is not bad, search the right half excluding mid
                left = mid + 1
        # Since left equals right, we found the first bad version
        return left

Python
```

```
class Solution {
    def firstBadVersion(self, n: int) -> int:
        l = 1
        r = n

        while l < r:
            m = (l + r) >> 1
            if isBadVersion(m):
                r = m
            else:
                l = m + 1
        return l
}

Python

# The isBadVersion API is already defined for you.
# def isBadVersion(version: int) -> bool:

class Solution:
    def firstBadVersion(self, n: int) -> int:
        l, r = 1, n
        while l < r:
            mid = (l + r) >> 1
            if isBadVersion(mid):
                r = mid
            else:
                l = mid + 1
        return l

Python

# The isBadVersion API is already defined for you.
# @param version, an integer
# @return an integer
# def isBadVersion(version):

class Solution:
    def firstBadVersion(self, n):
        type n: int
        type int
        left, right = 1, n
        while left < right:
            mid = (left + right) >> 1
            if isBadVersion(mid):
                right = mid
            else:
                left = mid + 1
        return left

Java
Java
```

```
public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int l = 1;
        int r = n;

        while (l < r) {
            final int m = l + (r - l) / 2;
            if (isBadVersion(m)) {
                r = m;
            } else {
                l = m + 1;
            }
        }
        return l;
    }
}

Java

/* The isBadVersion API is defined in the parent class VersionControl.
boolean isBadVersion(int version); */

public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int l = 1, r = n;
        while (l < r) {
            int mid = (l + r) >> 1;
            if (isBadVersion(mid)) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    }
}

JavaScript
JavaScript
```

```
/**
 * Definition for isBadVersion()
 *
 * @param {integer} version number
 * @return {boolean} whether the version is bad
 * isBadVersion = function(version) {
 * ...
 * };
 */

/**
 * @param {function} isBadVersion()
 * @return {function}
 */
var solution = function (isBadVersion) {
    /**
     * @param {integer} n Total versions
     * @return {integer} The first bad version
     */
    return function (n) {
        let l, r = [1, n];
        while (l < r) {
            const mid = (l + r) >> 1;
            if (isBadVersion(mid)) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    };
};
```

JavaScript

```
/**
 * Definition for isBadVersion()
 *
 * @param {integer} version number
 * @return {boolean} whether the version is bad
 * isBadVersion = function(version) {
 * ...
 * };
 */

/**
 * @param {function} isBadVersion()
 * @return {function}
 */
var solution = function (isBadVersion) {
    /**
     * @param {integer} n Total versions
     * @return {integer} The first bad version
     */
    return function (n) {
        let left = 1;
        let right = n;
        while (left < right) {
            const mid = (left + right) >> 1;
            if (isBadVersion(mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    };
};
```

TypeScript

TypeScript

```
/**
 * The knows API is defined in the parent class Relation.
 * isBadVersion(version: number) : boolean {
 * ...
 * };
 */

var solution = function (isBadVersion: any) {
    return function (n: number): number {
        let l, r = [1, n];
        while (l < r) {
            const mid = (l + r) >> 1;
            if (isBadVersion(mid)) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    };
};
```

TypeScript

```
/**
 * The knows API is defined in the parent class Relation.
 * isBadVersion(version: number) : boolean {
 * ...
 * };
 */

var solution = function (isBadVersion: any) {
    return function (n: number): number {
        let l, r = [1, n];
        while (l < r) {
            const mid = (l + r) >> 1;
            if (isBadVersion(mid)) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    };
};
```

Rust

Rust

```
// The API isBadVersion is defined for you.
// isBadVersion(version: int) -> bool;
// To call it use self.isBadVersion(version)

impl Solution {
    pub fn first_bad_version(isBadFn: impl Fn(i32) -> bool) -> i32 {
        let mut l, mut r = (1, n);
        while l < r {
            let mid = l + (r - l) / 2;
            if !isBadFn(isBadFn(mid)) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        l
    }
}
```

Rust

```
// The API isBadVersion is defined for you.
// isBadVersion(version: int) -> bool;
// To call it use self.isBadVersion(version)

impl Solution {
    pub fn first_bad_version(isBadFn: impl Fn(i32) -> bool) -> i32 {
        let mut left = 1;
        let mut right = n;
        while left < right {
            let mid = (right - left) / 2;
            if !isBadFn(isBadFn(left + mid)) {
                right = left + mid;
            } else {
                left = left + mid + 1;
            }
        }
        left
    }
}
```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

```
# Python implementation of First Bad Version
class Solution:
    def firstBadVersion(self, n: int) -> int:
        # Initialize the left pointer to the first version and right pointer to the last version
        left, right = 1, n
        while left < right:
            # Calculate the mid point to avoid potential overflow
            mid = left + (right - left) // 2
            # Call the API isBadVersion to check if mid version is bad
            if isBadVersion(mid):
                # If mid is bad, search the left half including mid
                right = mid
            else:
                # If mid is not bad, search the right half excluding mid
                left = mid + 1
        # When left equals right, we found the first bad version
        return left
```

#374 EASY

Guess Number Higher or Lower

LeetCode · Simplest · WalkCC · LeetCode · Editorial

Math · Binary Search · Interactive

Link: Dong Zhang

Problem:

We are playing the Guess Game. The game is as follows:

I pick a number from 1 to n. You have to guess which number I picked (the number I picked stays the same throughout the game).

Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.

You call a pre-defined API int guess(num), which returns three possible results:

- 1: Your guess is higher than the number I picked (i.e. num > pick).
- 1: Your guess is lower than the number I picked (i.e. num < pick).
- 0: Your guess is equal to the number I picked (i.e. num == pick).

Return the number that I picked.

Example 1:

Input: n = 10, pick = 6

Output: 6

Example 2:

Input: n = 1, pick = 1

Output: 1

Example 3:

Input: n = 2, pick = 1

Output: 1

Constraints:

```
• 1 <= n <= 231 - 1
• 1 <= pick <= n
```

Python

```
Python
class Solution:
    def guessNumber(self, n: int) -> int:
        # Initialize low and high pointers
        low, high = 1, n
        # Continue binary search until low exceeds high
        while low <= high:
            # Calculate mid point safely to prevent overflow
            mid = low + (high - low) // 2
            # Call the guess API with mid
            result = guess(mid)
            # If the guess is correct, return mid
            if result == 0:
                return mid
            # If guess was too high, adjust the high pointer
            elif result == -1:
                high = mid - 1
            # If guess was too low, adjust the low pointer
            else:
                low = mid + 1
        # Return low (ideally should never reach here if pick exists in [1, n])
        return low
```

Python

```
Python
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
#         1 if num is lower than the picked number
#         0 if num is equal to the picked number
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        l = 1
        r = n

        # Find the first guess number that == the target number
        while l < r:
            m = (l + r) // 2
            if guess(m) == 0:
                return m
            elif guess(m) == -1:
                r = m
            else:
                l = m + 1
```

Python


```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        return bisect.bisect(range(1, n + 1), 0, key=lambda x: -guess(x))
```

Python

```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:
```

```
class Solution:
    def guessNumber(self, n: int) -> int:
        return bisect.bisect(range(1, n + 1), 0, key=lambda x: -guess(x))
```

Java

```
/*
 * Forward declaration of guess API.
 * @param num, your guess
 * @return -1 if num is higher than the picked number
 * 1 if num is lower than the picked number
 * otherwise return 0
 * int guess(int num);
 */

public class Solution extends GuessGame {
    public int guessNumber(int n) {
        int left = 1, right = n;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (guess(mid) == 0) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
```

Java

```
/*
 * Forward declaration of guess API.
 * @param num, your guess
 * @return -1 if num is higher than the picked number
 * 1 if num is lower than the picked number
 * otherwise return 0
 * int guess(int num);
 */

public class Solution extends GuessGame {
    public int guessNumber(int n) {
        int left = 1, right = n;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (guess(mid) == 0) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
```

Java

```
/*
 * Forward declaration of guess API.
 * @param num, your guess
 * @return -1 if num is higher than the picked number
 * 1 if num is lower than the picked number
 * otherwise return 0
 * int guess(int num);
 */

public class Solution extends GuessGame {
    public int guessNumber(int n) {
        int left = 1, right = n;
        while (left < right) {
            int mid = left + ((right - left) >> 1);
            if (guess(mid) == 0) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}
```

TypeScript

TypeScript

```
/*
 * Forward declaration of guess API.
 * @param (number): your guess
 * @return -1 if num is higher than the guess number
 * 1 if num is lower than the guess number
 * otherwise return 0
 * var guess = function(num) {}
 */

function guessNumber(n: number): number {
    let l = 1;
    let r = n;
    while (l < r) {
        const mid = (l + r) >> 1;
        if (guess(mid) == 0) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}
```

TypeScript

```
/*
 * Forward declaration of guess API.
 * @param (number): your guess
 * @return -1 if num is higher than the guess number
 * 1 if num is lower than the guess number
 * otherwise return 0
 * var guess = function(num) {}
 */

function guessNumber(n: number): number {
    let l = 1;
    let r = n;
    while (l < r) {
        const mid = (l + r) >> 1;
        if (guess(mid) == 0) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}
```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

```
# Python solution using binary search

class Solution:
    def guessNumber(self, n: int) -> int:
        # Initialize low and high pointers
        low, high = 1, n
        # Perform binary search until low exceeds high
        while low <= high:
            # Calculate midpoint safely to prevent overflow
            mid = low + (high - low) // 2
            # Call the guess API each mid
            result = guess(mid)
            # If the guess is correct, return mid
            if result == 0:
                return mid
            # If guess is too high, adjust the high pointer
            elif result == -1:
                high = mid - 1
            # If guess is too low, adjust the low pointer
            else:
                low = mid + 1
        # Return low (should never reach here if pick exists in [1, n])
        return low
```

#704

EASY

Binary Search

LeetCode - SimplyEasy - WUCC - LeetCode - Editorial

Array - Binary Search

LeetCode 704

Problem:

Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.

You must write an algorithm with O(log n) runtime complexity.

Example 1:

Input: nums = [-1,0,3,5,9,12], target = 9

Output: 4

Explanation: 9 exists in nums and its index is 4

Example 2:

Input: nums = [-1,0,3,5,9,12], target = 2

Output: -1

Explanation: 2 does not exist in nums so return -1

Constraints:

• 1 <= nums.length <= 104

• -104 <= nums[i], target <= 104

• All the integers in nums are unique.

• nums is sorted in ascending order.

>> Brute Force [Binary Search] Interview Prep

```
Python

# Brute-force approach
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        # Iterate over the array, ignoring sorted property
        for i, val in enumerate(nums):
            if val == target:
                return i
        return -1
```

Python

Python code for binary search.

```
def search(nums, target):
    # Set initial boundaries for the search
    left, right = 0, len(nums) - 1
    # Loop until the search space is empty
    while left <= right:
        # Calculate middle index to avoid overflow
        mid = left + (right - left) // 2
        # Check if the mid element matches the target
        if nums[mid] == target:
            return mid
        # If target is smaller than mid element, narrow search to left half
        elif target < nums[mid]:
            right = mid - 1
        # Else, narrow search to right half
        else:
            left = mid + 1
    # Target not found, return -1
    return -1
```

```
# Example usage:
print(search([-1,0,3,5,9,12], 9)) # Expected output: 4
```

Python

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l = bisect.bisect_left(nums, target)
        return -1 if l == len(nums) or nums[l] != target else l
```

Python

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] == target:
                return mid
            else:
                l = mid + 1
        return l if nums[l] == target else -1
```

Python

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] == target:
                return mid
            else:
                left = mid + 1
        return left if nums[left] == target else -1
```

C++

C++

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        const auto it = ranges::lower_bound(nums, target);
        return (it == nums.cend() || *it != target) ? -1
            : distance(nums.begin(), it);
    }
};
```

C++

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        int l = 0, r = nums.size() - 1;
        while (l < r) {
            int mid = (l + r) >> 1;
            if (nums[mid] == target) {
                return mid;
            } else {
                l = mid + 1;
            }
        }
        return nums[l] == target ? l : -1;
    }
};
```

C++

```

class Solution {
public:
    int search(vector<int>& nums, int target) {
        int left = 0, right = nums.size() - 1;
        while (left < right) {
            int mid = (left + right) / 2;
            if (nums[mid] == target)
                right = mid;
            else
                left = mid + 1;
        }
        return nums[left] == target ? left : -1;
    }
};

```

Java

Java

```

class Solution {
    public int search(int[] nums, int target) {
        final int l = Arrays.binarySearch(nums, target);
        return l < 0 ? l + 1 : l;
    }
}

```

Java

```

class Solution {
    public int search(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l < r) {
            int mid = (l + r) / 2;
            if (nums[mid] == target) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return nums[l] == target ? l : -1;
    }
}

```

Java

```

public class Solution {
    public int search(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l < r) {
            int mid = (l + r) / 2;
            if (nums[mid] == target) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return nums[l] == target ? l : -1;
    }
}

```

Binary Search - LeetCode - By Pattern

— 2287 —

LeetCode

Java

```

class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left < right) {
            int mid = (left + right) / 2;
            if (nums[mid] == target) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return nums[left] == target ? left : -1;
    }
}

```

Java

```

public class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left < right) {
            int mid = (left + right) / 2;
            if (nums[mid] == target) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return nums[left] == target ? left : -1;
    }
}

```

JavaScript

JavaScript

```

// @param {number[]} nums
// @param {number} target
// @return {number}

var search = function(nums, target) {
    let l, r = [0, nums.length - 1];
    while (l < r) {
        const mid = (l + r) / 2;
        if (nums[mid] == target) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return nums[l] == target ? l : -1;
};

```

JavaScript

Binary Search - LeetCode - By Pattern

— 2288 —

LeetCode

```

// @param {number[]} nums
// @param {number} target
// @return {number}

var search = function(nums, target) {
    let left = 0;
    let right = nums.length - 1;
    while (left < right) {
        const mid = (left + right) / 2;
        if (nums[mid] == target) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return nums[left] == target ? left : -1;
};

```

TypeScript

TypeScript

```

function search(nums: number[], target: number): number {
    let l, r = [0, nums.length - 1];
    while (l < r) {
        const mid = (l + r) / 2;
        if (nums[mid] == target) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return nums[l] == target ? l : -1;
}

```

TypeScript

```

function search(nums: number[], target: number): number {
    let l, r = [0, nums.length - 1];
    while (l < r) {
        const mid = (l + r) / 2;
        if (nums[mid] == target) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return nums[l] == target ? l : -1;
}

```

Go

Go

Binary Search - LeetCode - By Pattern

— 2289 —

LeetCode

```

func search(nums []int, target int) int {
    l, r := 0, len(nums)-1
    for l < r {
        mid := (l + r) / 2
        if nums[mid] == target {
            r = mid
        } else {
            l = mid + 1
        }
    }
    if nums[l] == target {
        return l
    }
    return -1
}

```

Go

```

func search(nums []int, target int) int {
    left, right := 0, len(nums)-1
    for left < right {
        mid := (left + right) / 2
        if nums[mid] == target {
            right = mid
        } else {
            left = mid + 1
        }
    }
    if nums[left] == target {
        return left
    }
    return -1
}

```

Rust

Rust

```

impl Solution {
    pub fn search(nums: Vec<i32>, target: i32) -> i32 {
        let mut l = 0;
        let mut r = nums.len() - 1;
        while l < r {
            let mid = (l + r) / 2;
            if nums[mid] == target {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        if nums[l] == target {
            l as i32
        } else {
            -1
        }
    }
}

```

Rust

Binary Search - LeetCode - By Pattern

— 2290 —

LeetCode

```

use std::cmp::Ordering;

impl Solution {
    pub fn search(nums: Vec<i32>, target: i32) -> i32 {
        let mut l = 0;
        let mut r = nums.len();
        while l < r {
            let mid = (l + r) / 2;
            match nums[mid].cmp(&target) {
                Ordering::Less => {
                    l = mid + 1;
                }
                Ordering::Greater => {
                    r = mid;
                }
                Ordering::Equal => {
                    return mid as i32;
                }
            }
        }
        -1
    }
}

```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

```

# Python code for binary search

def search(nums, target):
    # Set initial boundaries for the search
    left, right = 0, len(nums) - 1

    # Loop until the search space is empty
    while left <= right:
        # Calculate middle index to avoid overflow
        mid = left + (right - left) // 2

        # Check if the mid element matches the target
        if nums[mid] == target:
            return mid

        # If target is smaller than mid element, narrow search to left half
        elif target < nums[mid]:
            right = mid - 1

        # Else, narrow search to right half
        else:
            left = mid + 1

    # Target not found, return -1
    return -1

# Example usage:
print(search([-1, 0, 3, 5, 9, 12], 9)) # Expected output: 4

```

#33

MEDIUM

Binary Search - LeetCode - By Pattern

— 2291 —

LeetCode

Search in Rotated Sorted Array

LeetCode - SimplifyLeetCode - WubCC - LeetCode - Editorial

Array - Binary Search

Lists Grid 169

Problem:

There is an integer array `nums` sorted in ascending order (with distinct values).

Prior to being passed to your function, `nums` is possibly left rotated at an unknown index `k` ($0 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`

Output: `4`

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`

Output: `-1`

Example 3:

Input: `nums = [1]`, `target = 0`

Output: `-1`

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- All values of `nums` are unique.
- `nums` is an ascending array that is possibly rotated.
- $-104 \leq \text{target} \leq 104$

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def search(self, nums, target):
        # Brute Force O(n) - Linear time
        for i, val in enumerate(nums):
            if val == target:
                return i
        return -1

```

Python

Python

Binary Search - LeetCode - By Pattern

— 2292 —

LeetCode

Python

```
class Selection:
    def search(self, nums: list[int], target: int) ->
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[l] <= nums[m]: # nums[l..m] are sorted.
                if nums[l] <= target <= nums[m]:
                    r = m - 1
            else:
                l = m + 1
            else: # nums[m..r] are sorted.
                if nums[m] <= target <= nums[r]:
                    l = m + 1
            else:
                r = m - 1

        return -1
```

Python

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid
        return left if nums[left] == target else -1
```

Python

```

5 before 2 solutions, diff is while condition: left <= 0 <= -right
6
7 I like this better
8
9 class Solution {
10     def merge(left: num. List[Int], target: Int): Int => Int:
11         n = List.empty
12         left, right = 0, n - 1
13         while left <= right:
14             mid = (left + right) / 2
15             if num[mid] == target:
16                 return mid
17             elif num[mid] < num[mid + 1]: // left half sorted
18                 if num[mid] >= target > num[mid + 1]:
19                     right = mid - 1
20                 else:
21                     left = mid + 1
22             else: // right half sorted
23                 if num[mid] < target < num[mid + 1]:
24                     left = mid + 1
25                 else:
26                     right = mid - 1
27         return -1
28
29 }
30
31 //-----
32
33 class Solution {
34     def search(left: num. List[Int], target: Int): Int => Int:
35         n = List.empty
36         left, right = 0, n - 1
37         while left <= right:
38             mid = (left + right) / 2
39             if num[mid] == target:
40                 return mid
41             elif num[mid] < num[mid + 1]: // left half sorted
42                 if num[mid] >= target > num[mid + 1]:
43                     right = mid
44                 else:
45                     left = mid + 1
46             else: // right half sorted
47                 if num[mid] < target < num[mid + 1]:
48                     left = mid + 1
49                 else:
50                     right = mid
51         return left if num[left] == target else -1
52
53 }

```

C++

```

class Solution {
public:
    int search(vector<int>& nums, int target) {
        int l = 0;
        int r = nums.size() - 1;

        while (l <= r) {
            const int m = (l + r) / 2;
            if (nums[m] == target)
                return m;
            if (nums[l] <= nums[m]) { // [nums[l], m] are sorted
                if (nums[m] > target && target <= nums[l])
                    r = m - 1;
                else
                    l = m + 1;
            } else { // [m+1, nums[m-1]] are sorted.
                if (nums[m] < target && target <= nums[r])
                    l = m + 1;
                else
                    r = m - 1;
            }
        }

        return -1;
    }
};

```

C++

```

class Solution {
public:
    int search(vector<int>& nums, int target) {
        int n = nums.size();
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (nums[mid] == target && nums[mid] < nums[n-1])
                right = mid;
            else
                left = mid + 1;
        }
        if (nums[mid] == target && target <= nums[n - 1])
            left = mid + 1;
        else
            right = mid;
    }
    return nums[left] == target ? left : -1;
};

```

C++

```
class Solution {
    //
    * @param Integer[] nums
    * @param Integer target
    * @return Integer
    */

    function search(nums, target) {
        sfoundKey = -1;
        foreach (nums as key => svalue) {
            if (svalue === target) {
                sfoundKey = key;
            }
        }
        return sfoundKey;
    }
}
```

C++

```
class Solution
public:
    int search(vector<int>& nums, int target) {
        int n = nums.size();
        int left = 0, right = n - 1;
        while (left <= right) {
            int mid = (left + right) >> 1;
            if (nums[mid] == target) return mid;
            else if (nums[mid] < target) left = mid + 1;
            else right = mid;
        }
        return nums[left] == target ? left : -1;
    }
};
```

C++

```
class Solution {
public:
    = {param Integer[]} nums
    = {param Integer} target
    = {return Integer}
};

function search(nums, target) {
    sfoundKey = -1;
    for each (nums as key ==> value) {
        if (value == target) {
            sfoundKey = key;
        }
    }
    return sfoundKey;
}
}
```

Layer

```

class Solution {
public: int search(int[] nums, int target) {
    int l = 0;
    int r = nums.length - 1;

    while (l <= r) {
        final int m = (l + r) / 2;
        if (nums[m] == target)
            return m;
        if (nums[l] <= target && target <= nums[r])
            r = m - 1;
        else
            l = m + 1;
    } else { // nums[m..n-1] are sorted.
        if (nums[m] < target && target <= nums[n-1])
            l = m + 1;
        else
            r = m - 1;
    }
}

return -1;
}
}

```

layers

```

class Solution {
    public int search(int[] nums, int target) {
        int n = nums.length;
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (nums[mid] == nums[mid]) {
                if (nums[mid] == target && target == nums[mid]) {
                    right = mid;
                } else {
                    left = mid + 1;
                }
            } else {
                if (nums[mid] < target && target == nums[n - 1]) {
                    left = mid + 1;
                } else {
                    right = mid;
                }
            }
        }
        return nums[left] == target ? left : -1;
    }
}

```

Java

```

public class Solution {
    public int Search(int[] nums, int target) {
        int n = nums.length;
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (nums[mid] == nums[mid]) {
                if (nums[mid] == target && target == nums[mid]) {
                    right = mid;
                } else {
                    left = mid + 1;
                }
            } else {
                if (nums[mid] < target && target == nums[n - 1]) {
                    left = mid + 1;
                } else {
                    right = mid;
                }
            }
        }
        return nums[left] == target ? left : -1;
    }
}

```

Java

```

class Solution {
    public int search(int[] nums, int target) {
        int n = nums.length;
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (nums[mid] == nums[mid]) {
                if (nums[mid] == target && target == nums[mid]) {
                    right = mid;
                } else {
                    left = mid + 1;
                }
            } else {
                if (nums[mid] < target && target == nums[n - 1]) {
                    left = mid + 1;
                } else {
                    right = mid;
                }
            }
        }
        return nums[left] == target ? left : -1;
    }
}

```

Java

```

public class Solution {
    public int Search(int[] nums, int target) {
        int n = nums.length;
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (nums[mid] == nums[mid]) {
                if (nums[mid] == target && target == nums[mid]) {
                    right = mid;
                } else {
                    left = mid + 1;
                }
            } else {
                if (nums[mid] < target && target == nums[n - 1]) {
                    left = mid + 1;
                } else {
                    right = mid;
                }
            }
        }
        return nums[left] == target ? left : -1;
    }
}

```

JavaScript

JavaScript

```

//
* @param {number[]} nums
* @param {number} target
* @return {number}
*/
var search = function(nums, target) {
    const n = nums.length;
    let left = 0, right = n - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (nums[mid] == nums[mid]) {
            if (nums[mid] == target && target == nums[mid]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target == nums[n - 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
    }
    return nums[left] == target ? left : -1;
};

```

JavaScript

```

//
* @param {number[]} nums
* @param {number} target
* @return {number}
*/
var search = function(nums, target) {
    const n = nums.length;
    let left = 0, right = n - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (nums[mid] == nums[mid]) {
            if (nums[mid] == target && target == nums[mid]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target == nums[n - 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
    }
    return nums[left] == target ? left : -1;
};

```

```

TypeScript
TypeScript
function search(nums: number[], target: number): number {
    const n = nums.length;
    let left = 0, right = n - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (nums[mid] == nums[mid]) {
            if (nums[mid] == target && target == nums[mid]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target == nums[n - 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
    }
    return nums[left] == target ? left : -1;
}

```

```

TypeScript
function search(nums: number[], target: number): number {
    const n = nums.length;
    let left = 0, right = n - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (nums[mid] == nums[mid]) {
            if (nums[mid] == target && target == nums[mid]) {
                right = mid;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target == nums[n - 1]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
    }
    return nums[left] == target ? left : -1;
}

```

Go

Go

```

func search(nums []int, target int) int {
    n := len(nums)
    left, right := 0, n-1
    for left < right {
        mid := (left + right) >> 1
        if nums[mid] == nums[mid] {
            if nums[mid] == target && target == nums[mid] {
                right = mid
            } else {
                left = mid + 1
            }
        } else {
            if nums[mid] < target && target == nums[n-1] {
                left = mid + 1
            } else {
                right = mid
            }
        }
    }
    if nums[left] == target {
        return left
    }
    return -1
}

```

Go

```

func search(nums []int, target int) int {
    n := len(nums)
    left, right := 0, n-1
    for left < right {
        mid := (left + right) >> 1
        if nums[mid] == nums[mid] {
            if nums[mid] == target && target == nums[mid] {
                right = mid
            } else {
                left = mid + 1
            }
        } else {
            if nums[mid] < target && target == nums[n-1] {
                left = mid + 1
            } else {
                right = mid
            }
        }
    }
    if nums[left] == target {
        return left
    }
    return -1
}

```

Rust

Rust

```

impl Solution {
    pub fn search(nums: Vec<i32>, target: i32) -> i32 {
        let mut l = 0;
        let mut r = nums.len() - 1;
        while l < r {
            let mid = (l + r) >> 1;
            if nums[mid] == target {
                return mid as i32;
            }
            if nums[l] == nums[mid] {
                if target < nums[mid] && target == nums[l] {
                    r = mid - 1;
                } else {
                    l = mid + 1;
                }
            } else {
                if target > nums[mid] && target == nums[r] {
                    l = mid + 1;
                } else {
                    r = mid - 1;
                }
            }
        }
        -1
    }
}

```

Rust

```

impl Solution {
    pub fn search(nums: Vec<i32>, target: i32) -> i32 {
        let mut l = 0;
        let mut r = nums.len() - 1;
        while l < r {
            let mid = (l + r) >> 1;
            if nums[mid] == target {
                return mid as i32;
            }
            if nums[l] == nums[mid] {
                if target < nums[mid] && target == nums[l] {
                    r = mid - 1;
                } else {
                    l = mid + 1;
                }
            } else {
                if target > nums[mid] && target == nums[r] {
                    l = mid + 1;
                } else {
                    r = mid - 1;
                }
            }
        }
        -1
    }
}

```

[1] Binary Search — Pattern Solution (matched from community answers)

```
Python
# Define a function to search for the target in a rotated sorted array.
def search(nums, target):
    # Initialize pointers for binary search.
    left, right = 0, len(nums) - 1

    # Loop until the search space is exhausted.
    while left <= right:
        # Compute the middle index.
        mid = (left + right) // 2

        # If the mid element is the target, return its index.
        if nums[mid] == target:
            return mid

        # Check if the left part of the array is sorted.
        if nums[left] <= nums[mid]:
            # If the target is within the sorted left half, adjust the right pointer.
            if nums[left] <= target <= nums[mid]:
                right = mid - 1
            else:
                # Otherwise, search the right half.
                left = mid + 1
        else:
            # The right half must be sorted.
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    # If the target is not found, return -1.
    return -1

# Example usage:
# nums = [4, 5, 6, 7, 8, 9, 1, 2], target = 8
# print(search(nums, target)) # Expected output: 4
```

#34

MEDIUM

Find First and Last Position of Element in Sorted Array

LeetCode • SimplexLive • WubliCC • LeetCode • Editorial

Array • Binary Search
Like Denny Zhang

Problem:

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Binary Search • LeetCode — By Pattern

— 2305 —

LeetCode

```
def searchRange(nums, target):
    # Helper function to find the boundary index (leftmost or rightmost) for target.
    def findBoundary(left):
        left, right = 0, len(nums) - 1
        boundary = -1
        while left <= right:
            mid = (left + right) // 2
            # If the target is found, update boundary and continue to search on one side.
            if nums[mid] == target:
                boundary = mid
                if left == mid:
                    right = mid - 1 # Continue search on left half
                else:
                    left = mid + 1 # Continue search on right half
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return boundary

    # Find the leftmost index
    leftIndex = findBoundary(True)
    if leftIndex == -1:
        return [-1, -1]
    # Find the rightmost index
    rightIndex = findBoundary(False)
    return [leftIndex, rightIndex]

# Example usage:
# nums = [5, 7, 7, 8, 8, 10]
# target = 8
# print(searchRange(nums, target)) # Output: [3, 4]
```

Python

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        l, r = 0, len(nums) - 1
        while l <= r:
            mid = (l + r) // 2
            if nums[mid] == target:
                return [l, r]
            elif nums[mid] < target:
                l = mid + 1
            else:
                r = mid - 1
        return [-1, -1]

# Example usage:
# nums = [5, 7, 7, 8, 8, 10]
# target = 8
# print(searchRange(nums, target)) # Output: [3, 4]
```

Python

Binary Search • LeetCode — By Pattern

— 2307 —

LeetCode

```
else:
    return 0
if n == len(nums) and nums[n] == target:
    # 1. corner not found.
    # Or 2. corner is not found and target is 1, return 0, 0.
    return 0
else:
    return -1
return (findRange(True), findRange(False))
```

C++

C++

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        const int l = ranges::lower_bound(nums, target) - nums.begin();
        if (l == nums.size()) return {-1, -1};
        return {l, -1};
        const int r = ranges::upper_bound(nums, target) - nums.begin() - 1;
        return {l, r};
    }
};
```

C++

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int l = lower_bound(nums.begin(), nums.end(), target) - nums.begin();
        int r = lower_bound(nums.begin(), nums.end(), target + 1) - nums.begin();
        if (l == r) {
            return {-1, -1};
        }
        return {l, r - 1};
    }
};
```

C++

Binary Search • LeetCode — By Pattern

— 2309 —

LeetCode

Input: nums = [5, 7, 7, 8, 8, 10], target = 8

Output: [3, 4]

Example 2:

Input: nums = [5, 7, 7, 8, 8, 10], target = 6

Output: [-1, -1]

Example 3:

Input: nums = [], target = 0

Output: [-1, -1]

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- `nums` is a non-decreasing array.
- $-109 \leq \text{target} \leq 109$

>> Brute Force (Binary Search) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def searchRange(self, nums, target):
        # Brute Force O(n) - Linear Scan instead of binary search
        for i, val in enumerate(nums):
            if val == target:
                return i
        return -1
```

Python

Python

Binary Search • LeetCode — By Pattern

— 2306 —

LeetCode

```
...
>>> bisect.bisect_left([1, 1, 1, 2, 2, 2, 7, 7], 2)
3
>>> bisect.bisect_left([1, 1, 1, 2, 2, 2, 7, 7], 2+1)
6
>>> bisect.bisect_right([1, 1, 1, 2, 2, 2, 7, 7], 2)
6
>>> bisect.bisect_right([1, 1, 1, 2, 2, 2, 7, 7], 2+1)
6
>>> bisect.bisect_left([1, 1, 1, 2, 2, 2, 7, 7], 0)
0
>>> bisect.bisect_left([1, 1, 1, 2, 2, 2, 7, 7], 1)
0
# In low, even single value for 2, after +1 the index will be different
>>> bisect.bisect_left([1, 2, 3, 4, 5], 2)
1
>>> bisect.bisect_left([1, 2, 3, 4, 5], 2+1)
2
...
```

import bisect

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        l = bisect.bisect(nums, target)
        r = bisect.bisect(nums, target + 1)
        return [-1, -1] if l == r else [l, r - 1]
```

#####

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        def findRange(left):
            l, r = 0, len(nums) - 1
            m = 0
            while l <= r:
                mid = (l + r) // 2
                if nums[mid] == target:
                    l = mid + 1
                elif nums[mid] < target:
                    l = mid + 1
                elif nums[mid] > target:
                    r = mid - 1
            return [l, r]
        left = findRange(True)
        right = findRange(False)
        return [-1, -1] if left == right else [left, right - 1]
```

Binary Search • LeetCode — By Pattern

— 2308 —

LeetCode

```
class Solution {
//
// @param Integer[] nums
// @param Integer target
// @return Integer
//
function searchRange(nums, target) {
    search = function (l, r) {
        if (l > r) return -1;
        if (l == r) return l;
        if (l < r) {
            mid = (l + r) // 2;
            if (nums[mid] < target) {
                l = mid + 1;
            } else {
                r = mid;
            }
        }
        return search(l, r);
    };
    l = search(target);
    r = search(target + 1);
    return [l, r - 1];
}
```

C++

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int l = lower_bound(nums.begin(), nums.end(), target) - nums.begin();
        int r = lower_bound(nums.begin(), nums.end(), target + 1) - nums.begin();
        if (l == r) return {-1, -1};
        return {l, r - 1};
    }
};
```

C++

Binary Search • LeetCode — By Pattern

— 2310 —

LeetCode

```

class Solution {
    /**
     * @param Integer[] nums
     * @param Integer target
     * @return Integer
     */
    function searchRange(nums, target) {
        let l = 0, r = nums.length - 1;
        while (l <= r) {
            let mid = (l + r) / 2;
            if (nums[mid] < target) {
                l = mid + 1;
            } else if (nums[mid] > target) {
                r = mid - 1;
            } else {
                return [l, r];
            }
        }
        return [-1, -1];
    }
}

```

Java

```

class Solution {
    public int[] searchRange(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l <= r) {
            int mid = (l + r) / 2;
            if (nums[mid] < target) {
                l = mid + 1;
            } else if (nums[mid] > target) {
                r = mid - 1;
            } else {
                return new int[] {l, r};
            }
        }
        return new int[] {-1, -1};
    }
}

```

Java

```

class Solution {
    public int[] searchRange(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l <= r) {
            int mid = (l + r) / 2;
            if (nums[mid] < target) {
                l = mid + 1;
            } else if (nums[mid] > target) {
                r = mid - 1;
            } else {
                return new int[] {l, r};
            }
        }
        return new int[] {-1, -1};
    }
}

```

Java

```

class Solution {
    public int[] searchRange(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l <= r) {
            int mid = (l + r) / 2;
            if (nums[mid] < target) {
                l = mid + 1;
            } else if (nums[mid] > target) {
                r = mid - 1;
            } else {
                return new int[] {l, r};
            }
        }
        return new int[] {-1, -1};
    }
}

```

Java

```

public class Solution {
    public int[] searchRange(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l <= r) {
            int mid = (l + r) / 2;
            if (nums[mid] < target) {
                l = mid + 1;
            } else if (nums[mid] > target) {
                r = mid - 1;
            } else {
                return new int[] {l, r};
            }
        }
        return new int[] {-1, -1};
    }
}

```

JavaScript

```

var searchRange = function(nums, target) {
    let l = 0, r = nums.length - 1;
    while (l <= r) {
        let mid = (l + r) / 2;
        if (nums[mid] < target) {
            l = mid + 1;
        } else if (nums[mid] > target) {
            r = mid - 1;
        } else {
            return [l, r];
        }
    }
    return [-1, -1];
}

```

JavaScript

```

/**
 * @param {number[]} nums
 * @param {number} target
 * @return {number[]}
 */
var searchRange = function(nums, target) {
    let l = 0, r = nums.length - 1;
    while (l <= r) {
        let mid = (l + r) / 2;
        if (nums[mid] < target) {
            l = mid + 1;
        } else if (nums[mid] > target) {
            r = mid - 1;
        } else {
            return [l, r];
        }
    }
    return [-1, -1];
}

```

TypeScript

```

function searchRange(nums: number[], target: number): number[] {
    let l = 0, r = nums.length - 1;
    while (l <= r) {
        let mid = (l + r) / 2;
        if (nums[mid] < target) {
            l = mid + 1;
        } else if (nums[mid] > target) {
            r = mid - 1;
        } else {
            return [l, r];
        }
    }
    return [-1, -1];
}

```

TypeScript

```

function searchRange(nums: number[], target: number): number[] {
    let l = 0, r = nums.length - 1;
    while (l <= r) {
        let mid = (l + r) / 2;
        if (nums[mid] < target) {
            l = mid + 1;
        } else if (nums[mid] > target) {
            r = mid - 1;
        } else {
            return [l, r];
        }
    }
    return [-1, -1];
}

```

Go

```

func searchRange(nums []int, target int) []int {
    l := 0
    r := len(nums) - 1
    for l <= r {
        mid := (l + r) / 2
        if nums[mid] < target {
            l = mid + 1
        } else if nums[mid] > target {
            r = mid - 1
        } else {
            return []int{l, r}
        }
    }
    return []int{-1, -1}
}

```

Go

```

func searchRange(nums []int, target int) []int {
    l := 0
    r := len(nums) - 1
    for l <= r {
        mid := (l + r) / 2
        if nums[mid] < target {
            l = mid + 1
        } else if nums[mid] > target {
            r = mid - 1
        } else {
            return []int{l, r}
        }
    }
    return []int{-1, -1}
}

```

Rust

```

fn searchRange(nums: Vec<i32>, target: i32) -> Vec<i32> {
    let mut l = 0;
    let mut r = nums.len() - 1;
    while l <= r {
        let mid = (l + r) / 2;
        if nums[mid] < target {
            l = mid + 1;
        } else if nums[mid] > target {
            r = mid - 1;
        } else {
            return vec![l, r];
        }
    }
    return vec![-1, -1];
}

```

Rust

```

impl Solution {
    pub fn search_range(nums: Vec<i32>, target: i32) -> Vec<i32> {
        let n = nums.len();
        let mut l = 0;
        let mut r = n - 1;
        while l <= r {
            let mid = (l + r) / 2;
            if nums[mid] < target {
                l = mid + 1;
            } else if nums[mid] > target {
                r = mid - 1;
            } else {
                return vec![l, r];
            }
        }
        return vec![-1, -1];
    }
}

```

Rust

```

impl Solution {
    pub fn search_range(nums: Vec<i32>, target: i32) -> Vec<i32> {
        let n = nums.len();
        let mut l = 0;
        let mut r = n - 1;
        while l <= r {
            let mid = (l + r) / 2;
            if nums[mid] < target {
                l = mid + 1;
            } else if nums[mid] > target {
                r = mid - 1;
            } else {
                return vec![l, r];
            }
        }
        return vec![-1, -1];
    }
}

```

Rust

```

impl Solution {
    pub fn search_range(nums: Vec<i32>, target: i32) -> Vec<i32> {
        let n = nums.len();
        let mut l = 0;
        let mut r = n - 1;
        while l <= r {
            let mid = (l + r) / 2;
            if nums[mid] < target {
                l = mid + 1;
            } else if nums[mid] > target {
                r = mid - 1;
            } else {
                return vec![l, r];
            }
        }
        return vec![-1, -1];
    }
}

```

Kotlin

```

fun searchRange(nums: List<Int>, target: Int): List<Int> {
    let l = 0
    let r = nums.size - 1
    while (l <= r) {
        let mid = (l + r) / 2
        if (nums[mid] < target) {
            l = mid + 1
        } else if (nums[mid] > target) {
            r = mid - 1
        } else {
            return listOf(l, r)
        }
    }
    return listOf(-1, -1)
}

```

Kotlin

```

class Solution {
    fun searchRange(nums: IntArray, target: Int): IntArray {
        var left = this.search(nums, target)
        var right = this.search(nums, target + 1)
        return if (left == right) IntArrayOff(-1, -1) else IntArrayOff(left, right - 1)
    }

    private fun search(nums: IntArray, target: Int): Int {
        var left = 0
        var right = nums.size
        while (left < right) {
            val mid = (left + right) / 2
            if (nums[mid] == target) {
                left = mid + 1
            } else {
                right = mid
            }
        }
        return left
    }
}

```

[*] Binary Search — Pattern Solution (matched from community answers)

```

Python
def searchRange(nums, target):
    # Helper function to find the boundary index (leftmost or rightmost) for target.
    def findBoundary(left: int, right: int, target: int) -> int:
        left, right = 0, len(nums) - 1
        boundary = -1
        while left <= right:
            mid = (left + right) // 2
            # If the target is found, update boundary and continue to search on one side.
            if nums[mid] == target:
                boundary = mid
                if isLeft:
                    right = mid - 1 # Continue search on left half
                else:
                    left = mid + 1 # Continue search on right half
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return boundary

    # Find the leftmost index
    leftIndex = findBoundary(True)
    if leftIndex == -1:
        return [-1, -1]
    # Find the rightmost index
    rightIndex = findBoundary(False)
    return [leftIndex, rightIndex]

# Example usage:
# nums = [5, 7, 7, 8, 8, 10]
# target = 8
# print(searchRange(nums, target)) # Output: [3, 4]

```

Binary Search · LeetCode — By Pattern

— 2317 —

LeetCode

#153 MEDIUM

Find Minimum in Rotated Sorted Array

LeetCode · Simple · WalkCC · LeetCode · Editorial

Array · Binary Search
Lists Grid 169

Problem:

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array $nums = [0,1,2,4,5,6,7]$ might become:

- $[4,5,6,7,0,1,2]$ if it was rotated 4 times.
- $[0,1,2,4,5,6,7]$ if it was rotated 7 times.

Notice that rotating an array $[a[0], a[1], a[2], \dots, a[n-1]]$ 1 time results in the array $[a[n-1], a[0], a[1], a[2], \dots, a[n-2]]$.

Given the sorted rotated array $nums$ of unique elements, return the minimum element of this array. You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: $nums = [3,4,5,1,2]$
Output: 1
Explanation: The original array was $[1,2,3,4,5]$ rotated 3 times.

Example 2:

Input: $nums = [4,5,6,7,0,1,2]$
Output: 0
Explanation: The original array was $[0,1,2,4,5,6,7]$ and it was rotated 4 times.

Example 3:

Input: $nums = [11,13,15,17]$
Output: 11
Explanation: The original array was $[11,13,15,17]$ and it was rotated 4 times.

Constraints:

- $n == nums.length$
- $1 \leq n \leq 5000$
- $-5000 \leq nums[i] \leq 5000$
- All the integers of $nums$ are unique.
- $nums$ is sorted and rotated between 1 and n times.

>> Brute Force [Binary Search] Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def findMin(self, nums):
        # Brute Force Algo - just return the minimum (ignores sorted structure)
        return min(nums)

```

Binary Search · LeetCode — By Pattern

— 2318 —

LeetCode

```

Python
def findMin(nums):
    # Initialize left and right pointers
    left, right = 0, len(nums) - 1
    # Continue while the search space is valid
    while left < right:
        mid = (left + right) // 2 # Compute middle index
        # If middle element is greater than the rightmost element,
        # the minimum must be in the right half
        if nums[mid] > nums[right]:
            left = mid + 1
        else:
            # Otherwise, the minimum is in the left half (including mid)
            right = mid
    # Once loop ends, left == right, pointing to the minimum element
    return nums[left]

```

```

Python
class Solution:
    def findMin(self, nums: List[int]) -> int:
        l = 0
        r = len(nums) - 1
        while l < r:
            m = (l + r) // 2
            if nums[m] < nums[r]:
                r = m
            else:
                l = m + 1
        return nums[l]

```

```

Python
class Solution:
    def findMin(self, nums: List[int]) -> int:
        if nums[0] <= nums[-1]:
            return nums[0]
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] <= nums[right]:
                left = mid + 1
            else:
                right = mid
        return nums[left]

```

Python

Binary Search · LeetCode — By Pattern

— 2319 —

LeetCode

```

class Solution:
    def findMin(self, nums: List[int]) -> int:
        if nums[0] <= nums[-1]:
            return nums[0]
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] <= nums[right]:
                left = mid + 1
            else:
                right = mid
        return nums[left]

```

C++

C++

```

class Solution {
public:
    int findMin(vector<int>& nums) {
        int l = 0;
        int r = nums.size() - 1;
        while (l < r) {
            const int m = (l + r) / 2;
            if (nums[m] < nums[r])
                r = m;
            else
                l = m + 1;
        }
        return nums[l];
    }
};

```

C++

```

class Solution {
public:
    int findMin(vector<int>& nums) {
        int n = nums.size();
        if (nums[0] <= nums[n - 1]) return nums[0];
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            if (nums[mid] <= nums[right])
                left = mid + 1;
            else
                right = mid;
        }
        return nums[left];
    }
};

```

C++

Binary Search · LeetCode — By Pattern

— 2320 —

LeetCode

```

class Solution {
public:
    int findMin(vector<int>& nums) {
        int n = nums.size();
        if (nums[0] <= nums[n - 1]) return nums[0];
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            if (nums[mid] <= nums[right])
                left = mid + 1;
            else
                right = mid;
        }
        return nums[left];
    }
};

```

Java

Java

```

class Solution {
    public int findMin(int[] nums) {
        int l = 0;
        int r = nums.length - 1;
        while (l < r) {
            final int m = (l + r) / 2;
            if (nums[m] <= nums[r])
                r = m;
            else
                l = m + 1;
        }
        return nums[l];
    }
}

```

Java

```

class Solution {
    public int findMin(int[] nums) {
        int n = nums.length;
        if (nums[0] <= nums[n - 1]) {
            return nums[0];
        }
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            if (nums[mid] <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return nums[left];
    }
}

```

Java

Binary Search · LeetCode — By Pattern

— 2321 —

LeetCode

```

class Solution {
    public int findMin(int[] nums) {
        int n = nums.length;
        if (nums[0] <= nums[n - 1]) {
            return nums[0];
        }
        int left = 0, right = n - 1;
        while (left < right) {
            int mid = (left + right) // 2;
            if (nums[mid] <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return nums[left];
    }
}

```

JavaScript

JavaScript

```

// @param {number[]} nums
// @return {number}
var findMin = function (nums) {
    let l = 0;
    let r = nums.length - 1;
    if (nums[l] <= nums[r]) return nums[l];
    while (l < r) {
        const m = (l + r) // 2;
        if (nums[m] <= nums[r]) l = m + 1;
        else r = m;
    }
    return nums[l];
};

```

JavaScript

Binary Search · LeetCode — By Pattern

— 2322 —

LeetCode

```

//
// @param {number[]} nums
// @return {number}
//
var findMin = function (nums) {
    const n = nums.length;
    if (nums[0] == nums[n - 1]) {
        return nums[0];
    }
    let left = 0,
        right = n - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (nums[0] == nums[mid]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return nums[left];
};

```

TypeScript

TypeScript

```

function findMin(nums: number[]): number {
    let left = 0;
    let right = nums.length - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (nums[mid] > nums[right]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return nums[left];
}

```

TypeScript

```

function findMin(nums: number[]): number {
    let left = 0;
    let right = nums.length - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (nums[mid] > nums[right]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return nums[left];
}

```

Go

Go

```

func findMin(nums []int) int {
    n := len(nums)
    if nums[0] == nums[n-1] {
        return nums[0]
    }
    left, right := 0, n-1
    for left < right {
        mid := (left + right) >> 1
        if nums[mid] > nums[mid] {
            left = mid + 1
        } else {
            right = mid
        }
    }
    return nums[left]
}

```

Go

```

func findMin(nums []int) int {
    n := len(nums)
    if nums[0] == nums[n-1] {
        return nums[0]
    }
    left, right := 0, n-1
    for left < right {
        mid := (left + right) >> 1
        if nums[mid] > nums[mid] {
            left = mid + 1
        } else {
            right = mid
        }
    }
    return nums[left]
}

```

Rust

Rust

```

impl Solution {
    pub fn find_min(nums: Vec<i32>) -> i32 {
        let mut left = 0;
        let mut right = nums.len() - 1;
        while left < right {
            let mid = (left + right) / 2;
            if nums[mid] > nums[right] {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        nums[left]
    }
}

```

Rust

Rust

```

impl Solution {
    pub fn find_min(nums: Vec<i32>) -> i32 {
        let mut left = 0;
        let mut right = nums.len() - 1;
        while left < right {
            let mid = (left + right) / 2;
            if nums[mid] > nums[right] {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        nums[left]
    }
}

```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

```

def findMin(nums):
    # Initialize left and right pointers
    left, right = 0, len(nums) - 1
    # Continue while the search space is valid
    while left < right:
        mid = (left + right) // 2 # Compute middle index
        # If middle element is greater than the rightmost element,
        # the minimum must be in the right half
        if nums[mid] > nums[right]:
            left = mid + 1
        else:
            # Otherwise, the minimum is in the left half (including mid)
            right = mid
    # Once loop ends, left == right, pointing to the minimum element
    return nums[left]

```

#300

MEDIUM

Longest Increasing Subsequence

LeetCode · Simplest · Rust · LeetCode · Editorial

Array · Binary Search · Dynamic Programming

Lists · Grid · 169

Problem:

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Example 1:

Input: `nums = [10,9,2,5,3,7,10,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,10]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7]`

Output: 1

Constraints:

`1 <= nums.length <= 2500`

`-104 <= nums[i] <= 104`

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def longestIncSubseq(self, nums):
        # Brute Force (O(N^2)) - Generate every subsequence, find longest increasing
        def lis(i, prev_val):
            if i == len(nums): return 0
            skip = lis(i + 1, prev_val)
            take = 0
            if nums[i] > prev_val:
                take = 1 + lis(i + 1, nums[i])
            return max(skip, take)
        return lis(0, float('-inf'))

```

Python

Python

Input: `nums = [7,7,7,7,7,7]`

Output: 1

Constraints:

`1 <= nums.length <= 2500`

`-104 <= nums[i] <= 104`

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def longestIncSubseq(self, nums):
        # Brute Force (O(N^2)) - Generate every subsequence, find longest increasing
        def lis(i, prev_val):
            if i == len(nums): return 0
            skip = lis(i + 1, prev_val)
            take = 0
            if nums[i] > prev_val:
                take = 1 + lis(i + 1, nums[i])
            return max(skip, take)
        return lis(0, float('-inf'))

```

Python

Python

Input: `nums = [7,7,7,7,7,7]`

Output: 1

Constraints:

`1 <= nums.length <= 2500`

`-104 <= nums[i] <= 104`

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def longestIncSubseq(self, nums):
        # Brute Force (O(N^2)) - Generate every subsequence, find longest increasing
        def lis(i, prev_val):
            if i == len(nums): return 0
            skip = lis(i + 1, prev_val)
            take = 0
            if nums[i] > prev_val:
                take = 1 + lis(i + 1, nums[i])
            return max(skip, take)
        return lis(0, float('-inf'))

```

Python

Python

Input: `nums = [7,7,7,7,7,7]`

Output: 1

Constraints:

`1 <= nums.length <= 2500`

`-104 <= nums[i] <= 104`

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def longestIncSubseq(self, nums):
        # Brute Force (O(N^2)) - Generate every subsequence, find longest increasing
        def lis(i, prev_val):
            if i == len(nums): return 0
            skip = lis(i + 1, prev_val)
            take = 0
            if nums[i] > prev_val:
                take = 1 + lis(i + 1, nums[i])
            return max(skip, take)
        return lis(0, float('-inf'))

```

Python

Python

Input: `nums = [7,7,7,7,7,7]`

Output: 1

Constraints:

`1 <= nums.length <= 2500`

`-104 <= nums[i] <= 104`

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def longestIncSubseq(self, nums):
        # Brute Force (O(N^2)) - Generate every subsequence, find longest increasing
        def lis(i, prev_val):
            if i == len(nums): return 0
            skip = lis(i + 1, prev_val)
            take = 0
            if nums[i] > prev_val:
                take = 1 + lis(i + 1, nums[i])
            return max(skip, take)
        return lis(0, float('-inf'))

```

```

class Solution:
    def longestIncSubseq(self, nums: List[int]) -> int:
        if not nums:
            return 0

        # dp[i] is the length of LIS ending in nums[i]
        dp = [1] * len(nums)

        for i in range(1, len(nums)):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)

```

Python

```

class Solution:
    def longestIncSubseq(self, nums: List[int]) -> int:
        # tails[i] is the minimum index of all the increasing subsequences having
        # length i + 1
        tails = []

        for num in nums:
            if not tails or num > tails[-1]:
                tails.append(num)
            else:
                tails[bisect.bisect_left(tails, num)] = num

        return len(tails)

```

Python

```

class Solution:
    def longestIncSubseq(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    f[i] = max(f[i], f[j] + 1)
        return max(f)

```

Python


```

if hi is None:
    hi = len(a)

while lo < hi:
    mid = (lo + hi) // 2
    if a[mid] < x:
        lo = mid + 1
    else:
        hi = mid

return lo

#####

class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.t = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.t[x] = min(self.t[x], v)
            x += x & -x

    def query(self, x: int) -> int:
        res = 0
        while x:
            res = max(res, self.t[x])
            x -= x & -x
        return res

class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        tree = BinaryIndexedTree(n)
        for x in nums:
            x = bisect_left(x, x) + 1
            t = tree.query(x - 1) + 1
            tree.update(x, t)
        return tree.query(n)

```

C++
C++

```

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        if (nums.empty())
            return 0;

        // dp[i] = the length of LIS ending in nums[i]
        vector<int> dp(nums.size(), 1);

        for (int i = 1; i < nums.size(); ++i)
            for (int j = 0; j < i; ++j)
                if (nums[i] > nums[j])
                    dp[i] = max(dp[i], dp[j] + 1);

        return ranges::max(dp);
    }
};

C++

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        // tails[i] = the minimum tail of all the increasing subsequences having
        // length i + 1
        vector<int> tails;

        for (const int num : nums)
            if (tails.empty() || num > tails.back())
                tails.push_back(num);
            else
                tails[firstGreaterEqual(tails, num)] = num;

        return tails.size();
    }

private:
    int firstGreaterEqual(const vector<int>& arr, int target) {
        return ranges::lower_bound(arr, target) - arr.begin();
    }
};

C++

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        int n = nums.size();
        vector<int> f(n, 1);
        for (int i = 1; i < n; ++i) {
            for (int j = 0; j < i; ++j) {
                if (nums[i] > nums[j])
                    f[i] = max(f[i], f[j] + 1);
            }
        }
        return *max_element(f.begin(), f.end());
    }
};

```

```

C++

class BinaryIndexedTree {
public:
    BinaryIndexedTree(int _n)
        : n{ _n }
        , t{ 0 } {}

    void update(int x, int v) {
        while (x <= n) {
            t[x] = min(t[x], v);
            x += x & -x;
        }
    }

    int query(int x) {
        int res = 0;
        while (x) {
            res = max(res, t[x]);
            x -= x & -x;
        }
        return res;
    }

private:
    int n;
    vector<int> t;
};

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        vector<int> n = nums;
        sort(n.begin(), n.end());
        n.erase(unique(n.begin(), n.end()), n.end());
        BinaryIndexedTree tree(n.size());
        for (int x : nums) {
            x = lower_bound(n.begin(), n.end(), x) - n.begin() + 1;
            int t = tree.query(x - 1) + 1;
            tree.update(x, t);
        }
        return tree.query(n.size());
    }
};

```

Java
Java

```

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        if (nums.empty())
            return 0;

        // dp[i] = the length of LIS ending in nums[i]
        int n = nums.size();
        vector<int> dp(n, 1);

        for (int i = 1; i < n; ++i)
            for (int j = 0; j < i; ++j)
                if (nums[i] > nums[j])
                    dp[i] = max(dp[i], dp[j] + 1);

        return *max_element(dp.begin(), dp.end());
    }
};

Java

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        // tails[i] = the minimum tail of all the increasing subsequences having
        // length i + 1
        List<Integer> tails = new ArrayList<>();

        for (final int num : nums)
            if (tails.isEmpty() || num > tails.get(tails.size() - 1))
                tails.add(num);
            else
                tails.set(firstGreaterEqual(tails, num), num);

        return tails.size();
    }

private:
    int firstGreaterEqual(List<Integer> arr, int target) {
        final int l = Collections.binarySearch(arr, target);
        return l < 0 ? -l - 1 : l;
    }
};

```

```

Java

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        int n = nums.size();
        int[] f = new int[n];
        Arrays.fill(f, 1);
        for (int i = 1; i < n; ++i)
            for (int j = 0; j < i; ++j)
                if (nums[i] > nums[j])
                    f[i] = Math.max(f[i], f[j] + 1);
        return *max_element(f, f + n);
    }
};

```

```

Java

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        int n = nums.size();
        Arrays.sort(nums);
        int m = 0;
        int s = 0;
        for (int i = 0; i < n; ++i)
            if (i == 0 || s[i] != s[i - 1]) {
                s[m++] = s[i];
            }
        BinaryIndexedTree tree = new BinaryIndexedTree(m);
        for (int x : nums) {
            x = search(s, x);
            int t = tree.query(x - 1) + 1;
            tree.update(x, t);
        }
        return tree.query(m);
    }

private:
    int search(int[] s, int x, int r) {
        int l = 0;
        while (l < r) {
            int mid = (l + r) / 2;
            if (s[mid] == x)
                return mid;
        }
        return l;
    }

private:
    int search(int[] s, int x, int r) {
        int l = 0;
        while (l < r) {
            int mid = (l + r) / 2;
            if (s[mid] == x)
                return mid;
        }
        return l;
    }

private:
    int search(int[] s, int x, int r) {
        int l = 0;
        while (l < r) {
            int mid = (l + r) / 2;
            if (s[mid] == x)
                return mid;
        }
        return l;
    }

private:
    int search(int[] s, int x, int r) {
        int l = 0;
        while (l < r) {
            int mid = (l + r) / 2;
            if (s[mid] == x)
                return mid;
        }
        return l;
    }

private:
    int search(int[] s, int x, int r) {
        int l = 0;
        while (l < r) {
            int mid = (l + r) / 2;
            if (s[mid] == x)
                return mid;
        }
        return l;
    }
};

```

```

public:
    int query(int x) {
        int res = 0;
        while (x <= n) {
            res = max(res, t[x]);
            x -= x & -x;
        }
        return res;
    }
};

TypeScript

function lengthOfLIS(nums: number[]): number {
    const n = nums.length;
    const f: number[] = new Array(n).fill(1);
    for (let i = 1; i < n; ++i)
        for (let j = 0; j < i; ++j)
            if (nums[i] > nums[j])
                f[i] = Math.max(f[i], f[j] + 1);
    return Math.max(...f);
}

```

```

TypeScript

class BinaryIndexedTree {
private n: number;
private t: number[];

constructor(n: number) {
    this.n = n;
    this.t = new Array(n + 1).fill(0);
}

update(x: number, v: number) {
    while (x <= this.n) {
        this.t[x] = Math.max(this.t[x], v);
        x += x & -x;
    }
}

query(x: number): number {
    let res = 0;
    while (x) {
        res = Math.max(res, this.t[x]);
        x -= x & -x;
    }
    return res;
}
}

```

Go

LeetMastery

Python

LeetMastery

LeetMastery

LeetMastery

```

class HitCounter {
    def __init__(self):
        self.timestamps = [] * 300
        self.hits = [0] * 300

    def hit(self, timestamp: int) -> None:
        i = timestamp % 300
        if self.timestamps[i] == timestamp:
            self.hits[i] += 1
        else:
            self.timestamps[i] = timestamp
            self.hits[i] = 1 # Reset the hit count to 1.

    def getHits(self, timestamp: int) -> int:
        return sum(h for t, h in zip(self.timestamps, self.hits)
                    if timestamp - t < 300)
}

Python
class HitCounter:
    def __init__(self):
        self.ts = []

    def hit(self, timestamp: int) -> None:
        self.ts.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        return len(self.ts) - bisect_left(self.ts, timestamp - 300 + 1)

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)

Python

```

```

from collections import deque

class HitCounter:
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.hits = deque()

    def hit(self, timestamp: int) -> None:
        """
        Record a hit.
        @param timestamp - The current timestamp (in seconds granularity).
        """
        self.hits.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        """
        Return the number of hits in the past 5 minutes.
        @param timestamp - The current timestamp (in seconds granularity).
        """
        while self.hits and self.hits[0] <= timestamp - 300:
            self.hits.popleft()
        return len(self.hits)

class HitCounter: # Follow-up
    def __init__(self):
        self.times = [0] * 300
        self.hits = [0] * 300

    def hit(self, timestamp: int) -> None:
        idx = timestamp % 300
        if self.times[idx] != timestamp:
            self.times[idx] = timestamp
            self.hits[idx] += 1
        else:
            self.hits[idx] += 1

    def getHits(self, timestamp: int) -> int:
        res = 0
        for i in range(300):
            if timestamp - self.times[i] < 300:
                res += self.hits[i]
        return res

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)

```

```

# param_2 = obj.getHits(timestamp)

#####

class HitCounter {
public:
    HitCounter() {
        // Initialize your data structure here.
        //
        self.counter = Counter()
    }

    void hit(self, timestamp: int) {
        // Record a hit.
        // @param timestamp - The current timestamp (in seconds granularity).
        //
        self.counter[timestamp] += 1
    }

    int getHits(self, timestamp: int) {
        // Return the number of hits in the past 5 minutes.
        // @param timestamp - The current timestamp (in seconds granularity).
        //
        return sum([v for t, v in self.counter.items() if t < 300 + timestamp])
    }
}

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)

C++
class HitCounter {
public:
    void hit(int timestamp) {
        count[timestamp % 300]++;
        if (timestamp % 300 == timestamp) {
            *hits++;
        } else {
            timestamp[i] = timestamp;
            hits[i] = 1; // Reset the hit count to 1.
        }
    }

    int getHits(int timestamp) {
        int countHits = 0;
        for (int i = 0; i < 300; ++i) {
            if (timestamp - timestamps[i] < 300) {
                countHits += hits[i];
            }
        }
        return countHits;
    }
private:
    vector<int> timestamps = vector<int>(300);
    vector<int> hits = vector<int>(300);
};

```

```

C++
class HitCounter {
public:
    HitCounter() {
    }

    void hit(int timestamp) {
        ts.push_back(timestamp);
    }

    int getHits(int timestamp) {
        return ts.end() - lower_bound(ts.begin(), ts.end(), timestamp - 300 + 1);
    }
private:
    vector<int> ts;
};

//
# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# int param_2 = obj.getHits(timestamp);

Java
class HitCounter {
    public void hit(int timestamp) {
        final int i = timestamp % 300;
        if (timestamps[i] == timestamp) {
            *hits++;
        } else {
            timestamps[i] = timestamp;
            hits[i] = 1; // Reset the hit count to 1.
        }
    }

    public int getHits(int timestamp) {
        int countHits = 0;
        for (int i = 0; i < 300; ++i) {
            if (timestamp - timestamps[i] < 300) {
                countHits += hits[i];
            }
        }
        return countHits;
    }

    private int[] timestamps = new int[300];
    private int[] hits = new int[300];
}

Java

```

```

class HitCounter {
    private List<Integer> ts = new ArrayList<>();

    public HitCounter() {
    }

    public void hit(int timestamp) {
        ts.add(timestamp);
    }

    public int getHits(int timestamp) {
        int l = search(timestamp - 300 + 1);
        return ts.size() - l;
    }

    private int search(int x) {
        int l = 0, r = ts.size();
        while (l < r) {
            int mid = (l + r) / 2;
            if (ts.get(mid) == x) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    }
}

//
# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# int param_2 = obj.getHits(timestamp);

Java

```

```

import java.util.ArrayDeque;

public class Design_Hit_Counter {
    public class HitCounter {
        private ArrayDeque<Integer> queue; // Note: ArrayDeque has no capacity restrictions

        // Why is ArrayDeque better than LinkedList?
        // If you need add/remove of the both ends, ArrayDeque is significantly better than a linked list
        // https://leetcode.com/question/319/why-is-arraydeque-better-than-linkedlist

        // Initialize your data structure here.
        public HitCounter() {
            queue = new ArrayDeque<Integer>();
        }

        // Record a hit.
        // @param timestamp - The current timestamp (in seconds granularity).
        public void hit(int timestamp) {
            queue.offer(timestamp);
        }

        // Return the number of hits in the past 5 minutes.
        // @param timestamp - The current timestamp (in seconds granularity).
        public int getHits(int timestamp) {
            // Time Complexity: O(1)
            while (timestamp - queue.peek() <= 300) {
                queue.poll();
            }
            return queue.size();
        }
    }

    public class HitCounterFollowup {
        private int[] times;
        private int[] hits;

        // Initialize your data structure here.
        public HitCounterFollowup() {
            times = new int[300];
            hits = new int[300];
        }

        // Record a hit.
        // @param timestamp - The current timestamp (in seconds granularity).
        public void hit(int timestamp) {
        }
    }
}

```

```

let idx = timestamp % 300;
if (times[idx] != timestamp) {
    times[idx] = timestamp; // update with most recent timestamp
    hits[idx] = 1;
} else {
    ++hits[idx];
}
}

// Return the number of hits in the past 5 minutes.
// @param timestamp - The current timestamp (in seconds granularity).
// Time Complexity: O(1)
public int getHits(int timestamp) {
    int res = 0;
    for (let i = 0; i < 300; ++i) {
        if (timestamp - times[i] < 300) { // values in times[] not removed at all
            ++res;
        }
    }
    return res;
}
}

//
// Your HitCounter object will be instantiated and called as such:
// HitCounter obj = new HitCounter();
// obj.hit(timestamp);
// int param_2 = obj.getHits(timestamp);
//

```

TypeScript
TypeScript

```

class HitCounter {
    private ts: number[] = [];

    constructor() {}

    hit(timestamp: number): void {
        this.ts.push(timestamp);
    }

    getHits(timestamp: number): number {
        const search = (s: number) => {
            let l = 0, r = 0, this.ts.length;
            while (l < r) {
                const mid = (l + r) >> 1;
                if (this.ts[mid] == s) {
                    r = mid;
                } else {
                    l = mid + 1;
                }
            }
            return l;
        };
        return this.ts.length - search(timestamp - 300 + 1);
    }
}

//
// Your HitCounter object will be instantiated and called as such:
// var obj = new HitCounter();
// obj.hit(timestamp);
// var param_2 = obj.getHits(timestamp);
//

```

TypeScript

```

class HitCounter {
    private ts: number[] = [];

    constructor() {}

    hit(timestamp: number): void {
        this.ts.push(timestamp);
    }

    getHits(timestamp: number): number {
        const search = (s: number) => {
            let l = 0, r = 0, this.ts.length;
            while (l < r) {
                const mid = (l + r) >> 1;
                if (this.ts[mid] == s) {
                    r = mid;
                } else {
                    l = mid + 1;
                }
            }
            return l;
        };
        return this.ts.length - search(timestamp - 300 + 1);
    }
}

//
// Your HitCounter object will be instantiated and called as such:
// var obj = new HitCounter();
// obj.hit(timestamp);
// var param_2 = obj.getHits(timestamp);
//

```

Go
Go

```

type HitCounter struct {
    ts []int
}

func Constructor() HitCounter {
    return HitCounter{}
}

func (this *HitCounter) Hit(timestamp int) {
    this.ts = append(this.ts, timestamp)
}

func (this *HitCounter) GetHits(timestamp int) int {
    return len(this.ts) - sort.SearchInts(this.ts, timestamp-300+1)
}

//
// Your HitCounter object will be instantiated and called as such:
// obj := Constructor();
// obj.Hit(timestamp);
// param_2 := obj.GetHits(timestamp);
//

```

```

Go

type HitCounter struct {
    ts []int
}

func Constructor() HitCounter {
    return HitCounter{}
}

func (this *HitCounter) Hit(timestamp int) {
    this.ts = append(this.ts, timestamp)
}

func (this *HitCounter) GetHits(timestamp int) int {
    return len(this.ts) - sort.SearchInts(this.ts, timestamp-300+1)
}

//
// Your HitCounter object will be instantiated and called as such:
// obj := Constructor();
// obj.Hit(timestamp);
// param_2 := obj.GetHits(timestamp);
//

```

Rust
Rust

```

struct HitCounter {
    ts: Vec<i32>,
}

//
// Self means the method takes an immutable reference.
// If you need a mutable reference, change it to 'mut self' instead.
//
impl HitCounter {
    fn new() -> Self {
        HitCounter { ts: Vec::new() }
    }

    fn hit(&mut self, timestamp: i32) {
        self.ts.push(timestamp);
    }

    fn get_hits(&self, timestamp: i32) -> i32 {
        let l = self.search(timestamp - 300 + 1);
        (self.ts.len() - l) as i32
    }

    fn search(&self, s: i32) -> usize {
        let (mut l, mut r) = (0, self.ts.len());
        while l < r {
            let mid = (l + r) / 2;
            if self.ts[mid] == s {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        l
    }
}

```

Rust

```

use std::collections::BinaryHeap, cmp::Reverse;

struct HitCounter {
    // A min heap
    pq: BinaryHeap<Reverse<i32>>,
}

impl HitCounter {
    fn new() -> Self {
        Self {
            pq: BinaryHeap::new(),
        }
    }

    fn hit(&mut self, timestamp: i32) {
        self.pq.push(Reverse(timestamp));
    }

    fn get_hits(&mut self, timestamp: i32) -> i32 {
        while let Some(Reverse(s)) = self.pq.peek() {
            if *s <= timestamp - 300 {
                self.pq.pop();
            } else {
                break;
            }
        }
        self.pq.len() as i32
    }
}

```

[*] Binary Search — Pattern Solution (stored optimal)

```

Python

# Python implementation of HitCounter
from collections import deque

class HitCounter:
    def __init__(self):
        # Initializing the deque to store (timestamp, count) pairs.
        self.hits_queue = deque()

    def hit(self, timestamp: int) -> None:
        # If the timestamp is less than the timestamp, increment its count.
        # If self.hits_queue and self.hits_queue[-1][0] == timestamp:
            self.hits_queue[-1][1] += 1
        else:
            # Append new (timestamp, count) pair.
            self.hits_queue.append((timestamp, 1))

    def getHits(self, timestamp: int) -> int:
        # Remove outdated hits from the front of the queue.
        while self.hits_queue and self.hits_queue[0][0] <= timestamp - 300:
            self.hits_queue.popleft()
        # Sum counts of the remaining hit entries.
        return sum(count for _, count in self.hits_queue)

```

```

C++
// C++ implementation of HitCounter

#include <iostream>
#include <vector>
using namespace std;

class HitCounter {
private:
    // Queue to store pairs of (timestamp, count)
    queue<pair<int, int>> hitQueue;

public:
    // Constructor to initialize the HitCounter
    HitCounter() {
        // No initialization required
    }

    // Record a hit at the given timestamp
    void hit(int timestamp) {
        if (hitQueue.empty() && hitQueue.back().first == timestamp) {
            // Increase count if last timestamp matches
            hitQueue.back().second++;
        } else {
            // Insertion of a new pair with count 1
            hitQueue.push(timestamp, 1);
        }
    }

    // Return the count of hits in the past 300 seconds
    int getHits(int timestamp) {
        // Remove outdated hits from the front of the queue
        while (!hitQueue.empty() && hitQueue.front().first <= timestamp - 300) {
            hitQueue.pop();
        }

        // Sum the counts in the queue
        int totalHits = 0;
        queue<pair<int, int>> tempQueue = hitQueue; // Temporary copy for iteration
        while (!tempQueue.empty()) {
            totalHits += tempQueue.front().second;
            tempQueue.pop();
        }
        return totalHits;
    }
};

```

#528 MEDIUM
Random Pick with Weight
[LeetCode](#) · [SimplifyLeet](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)
[Math](#) · [Binary Search](#) · [Prefix Sum](#) · [Randomized](#)
[List](#) · [Grid](#) · [100](#)
Problem:

Binary Search · LeetCode · By Pattern — 2353 — LeetCode

You are given a 0-indexed array of positive integers w where $w[i]$ describes the weight of the i th index.

You need to implement the function `pickIndex()`, which randomly picks an index in the range $[0, w.length - 1]$ (inclusive) and returns it. The probability of picking an index i is $w[i] / \text{sum}(w)$.

For example, if $w = [1, 3]$, the probability of picking index 0 is $1 / (1 + 3) = 0.25$ (i.e., 25%), and the probability of picking index 1 is $3 / (1 + 3) = 0.75$ (i.e., 75%).

Example 1:

```

Input
["Solution","pickIndex"]
[[[]],[[]]]
Output
[null,0]

```

Explanation
Solution solution = new Solution([[]]);
solution.pickIndex(); // return 0. The only option is to return 0 since there is only one element in w .

Example 2:

```

Input
["Solution","pickIndex","pickIndex","pickIndex","pickIndex"]
[[[1,3]],[[]],[[]],[[]],[[]]]
Output
[null,1,1,1,0]

```

Explanation
Solution solution = new Solution([1, 3]);
solution.pickIndex(); // return 1. It is returning the second element (index = 1) that has a probability of 3/4.
solution.pickIndex(); // return 1
solution.pickIndex(); // return 1
solution.pickIndex(); // return 0. It is returning the first element (index = 0) that has a probability of 1/4.

Since this is a randomization problem, multiple answers are allowed.

All of the following outputs can be considered correct:

```

[null,1,1,1,0]
[null,1,1,1,1]
[null,1,1,0,0]
[null,1,1,0,1]
[null,1,0,0,0]
.....

```

and so on.

Constraints:

- $1 \leq w.length \leq 104$
- $1 \leq w[i] \leq 105$
- `pickIndex` will be called at most 104 times.

Binary Search · LeetCode · By Pattern — 2354 — LeetCode

```

>> Brute Force (Binary Search) Interview Prep
Python
# Brute Force Approach
class Solution:
    def __init__(self, w):
        # Brute Force O(n) - slower than binary search
        for i, val in enumerate(w):
            if val == target:
                return i
        return -1

Python
import bisect

class Solution:
    def __init__(self, w):
        # Build the prefix sum array.
        self.prefix_sums = []
        current_sum = 0
        for weight in w:
            current_sum += weight
            self.prefix_sums.append(current_sum)
        # Store the total sum of weights.
        self.total_sum = current_sum

    def pickIndex(self):
        # Get a random target in the range [1, total_sum]
        target = random.randint(1, self.total_sum)
        # Use bisect to find the proper index.
        index = bisect.bisect_left(self.prefix_sums, target)
        return index

# Example usage:
# solution = Solution([1, 3])
# print(solution.pickIndex())

Python
class Solution:
    def __init__(self, w: List[int]):
        self.prefix = list(itertools.accumulate(w))

    def pickIndex(self) -> int:
        target = random.randint(1, self.prefix[-1] - 1)
        return bisect.bisect_right(range(len(self.prefix)), target,
                                   key=lambda x: self.prefix[x])

```

Binary Search · LeetCode · By Pattern — 2355 — LeetCode

```

class Solution {
    def __init__(self, w: List[int]):
        self.s = 0
        for c in w:
            self.s.append(self.s[-1] + c)

    def pickIndex(self) -> int:
        s = random.randint(1, self.s[-1])
        left, right = 1, len(self.s) - 1
        while left < right:
            mid = (left + right) >> 1
            if self.s[mid] >= s:
                right = mid
            else:
                left = mid + 1
        return left - 1

# Your Solution object will be instantiated and called as such:
# obj = Solution(w)
# param_1 = obj.pickIndex()

Python
class Solution:
    def __init__(self, w: List[int]):
        self.s = 0
        for c in w:
            self.s.append(self.s[-1] + c)

    def pickIndex(self) -> int:
        s = random.randint(1, self.s[-1])
        left, right = 1, len(self.s) - 1
        while left < right:
            mid = (left + right) >> 1
            if self.s[mid] >= s:
                right = mid
            else:
                left = mid + 1
        return left - 1

# Your Solution object will be instantiated and called as such:
# obj = Solution(w)
# param_1 = obj.pickIndex()

C++
C++

```

Binary Search · LeetCode · By Pattern — 2356 — LeetCode

```

class Solution {
public:
    Solution(vector<int>& w) : prefix(w.size()) {
        partial_sum(w.begin(), w.end(), prefix.begin());
    }

    int pickIndex() {
        const int target = rand() % prefix.back();
        int l = 0;
        int r = prefix.size();
        while (l < r) {
            const int m = (l + r) / 2;
            if (prefix[m] > target)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }
private:
    vector<int> prefix;
};

C++
class Solution {
public:
    vector<int> s;

    Solution(vector<int>& w) {
        int n = w.size();
        s.resize(n + 1);
        for (int i = 0; i < n; ++i) s[i + 1] = s[i] + w[i];
    }

    int pickIndex() {
        int n = s.size();
        int l = 1, r = rand() % n + 1;
        while (left < right) {
            int mid = left + right >> 1;
            if (s[mid] == s)
                right = mid;
            else
                left = mid + 1;
        }
        return left - 1;
    }
};

/*
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(w);
 * int param_1 = obj->pickIndex();
 */

```

Binary Search · LeetCode · By Pattern — 2357 — LeetCode

```

class Solution {
public:
    vector<int> s;

    Solution(vector<int>& w) {
        int n = w.size();
        s.resize(n + 1);
        for (int i = 0; i < n; ++i) s[i + 1] = s[i] + w[i];
    }

    int pickIndex() {
        int n = s.size();
        int s = 1 + rand() % s[n - 1];
        int left = 1, right = n - 1;
        while (left < right) {
            int mid = left + right >> 1;
            if (s[mid] == s)
                right = mid;
            else
                left = mid + 1;
        }
        return left - 1;
    }
};

/*
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(w);
 * int param_1 = obj->pickIndex();
 */

```

Java
Java

Binary Search · LeetCode · By Pattern — 2358 — LeetCode

```

class Solution {
public: Solution(int[] w) {
    prefix = w;
    for (int i = 1; i < prefix.length; ++i)
        prefix[i] += prefix[i - 1];
}

public: int pickIndex() {
    final int target = rand.nextInt(prefix.length - 1);
    int l = 0;
    int r = prefix.length;

    while (l < r) {
        final int m = (l + r) / 2;
        if (prefix[m] > target)
            r = m;
        else
            l = m + 1;
    }

    return l;
}

private int[] prefix;
private Random rand = new Random();
}

```

```

Java
class Solution {
    private int[] w;
    private Random random = new Random();

    public Solution(int[] w) {
        int n = w.length;
        w = new int[n + 1];
        for (int i = 0; i < n; ++i) {
            w[i + 1] = w[i] + w[i];
        }
    }

    public int pickIndex() {
        int s = 1 + random.nextInt(w.length - 1);
        int left = 1, right = s.length - 1;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (w[mid] == s) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left - 1;
    }
}

```

```

}

/**
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(w);
 * int param_1 = obj.pickIndex();
 */

Java
class Solution {
    private int[] w;
    private Random random = new Random();

    public Solution(int[] w) {
        int n = w.length;
        w = new int[n + 1];
        for (int i = 0; i < n; ++i) {
            w[i + 1] = w[i] + w[i];
        }
    }

    public int pickIndex() {
        int s = 1 + random.nextInt(w.length - 1);
        int left = 1, right = s.length - 1;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (w[mid] == s) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left - 1;
    }
}

```

```

/**
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(w);
 * int param_1 = obj.pickIndex();
 */

```

JavaScript

JavaScript

```

/**
 * @param {number[]} w
 */
var Solution = function(w) {
    const n = w.length;
    this.w = new Array(n + 1).fill(0);
    for (let i = 0; i < n; ++i) {
        this.w[i + 1] = this.w[i] + w[i];
    }
};

/**
 * @return {number}
 */
Solution.prototype.pickIndex = function() {
    const n = this.w.length;
    const s = 1 + Math.floor(Math.random() * this.w[n - 1]);
    let left = 1,
        right = n - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (this.w[mid] == s) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left - 1;
};

```

```

/**
 * Your Solution object will be instantiated and called as such:
 * var obj = new Solution(w);
 * int param_1 = obj.pickIndex();
 */

```

JavaScript

```

/**
 * @param {number[]} w
 */
var Solution = function(w) {
    const n = w.length;
    this.w = new Array(n + 1).fill(0);
    for (let i = 0; i < n; ++i) {
        this.w[i + 1] = this.w[i] + w[i];
    }
};

/**
 * @return {number}
 */
Solution.prototype.pickIndex = function() {
    const n = this.w.length;
    const s = 1 + Math.floor(Math.random() * this.w[n - 1]);
    let left = 1,
        right = n - 1;
    while (left < right) {
        const mid = (left + right) >> 1;
        if (this.w[mid] == s) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left - 1;
};

```

```

/**
 * Your Solution object will be instantiated and called as such:
 * var obj = new Solution(w);
 * int param_1 = obj.pickIndex();
 */

```

Go

Go

```

type Solution struct {
    w []int
}

func Constructor(w []int) Solution {
    s := len(w)
    w = make([]int, s+1)
    for i := 0; i < s; i++ {
        w[i+1] = w[i] + w[i]
    }
    return Solution{w}
}

func (this *Solution) PickIndex() int {
    s := len(this.w)
    n := 1 + rand.Intn(s-1)
    left, right := 1, s-1
    for left < right {
        mid := (left + right) >> 1
        if this.w[mid] == n {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return left - 1
}

```

```

/**
 * Your Solution object will be instantiated and called as such:
 * obj := Constructor(w);
 * param_1 := obj.PickIndex();
 */

```

Go

```

type Solution struct {
    w []int
}

func Constructor(w []int) Solution {
    s := len(w)
    w = make([]int, s+1)
    for i := 0; i < s; i++ {
        w[i+1] = w[i] + w[i]
    }
    return Solution{w}
}

func (this *Solution) PickIndex() int {
    s := len(this.w)
    n := 1 + rand.Intn(s-1)
    left, right := 1, s-1
    for left < right {
        mid := (left + right) >> 1
        if this.w[mid] == n {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return left - 1
}

```

```

/**
 * Your Solution object will be instantiated and called as such:
 * obj := Constructor(w);
 * param_1 := obj.PickIndex();
 */

```

Rust

Rust

```
use rand!({ thread_rng, Rng });

struct Solution {
    sum: Vec<i32>,
}

/*
 * 'self' means the method takes an immutable reference.
 * If you need a mutable reference, change it to 'mut self' instead.
 */
impl Solution {
    fn makeVecVec(i32) -> Self {
        let s = Self {
            sum: vec![],
        };
        let mut sum = vec![];
        for i in 1..= i32 {
            sum[i] = sum[i - 1] + i;
        }
        Self { sum }
    }

    fn pick_index(self) -> i32 {
        let s = thread_rng().gen_range(1, self.sum.len().saturating() + 1);
        let (mut left, mut right) = (1, self.sum.len() - 1);
        while left < right {
            let mid = (left + right) / 2;

            if self.sum[mid] < s {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        (left - 1) as i32
    }
}

Rust
```

```
use rand!({ thread_rng, Rng });

struct Solution {
    sum: Vec<i32>,
}

/*
 * 'self' means the method takes an immutable reference.
 * If you need a mutable reference, change it to 'mut self' instead.
 */
impl Solution {
    fn makeVecVec(i32) -> Self {
        let s = Self {
            sum: vec![],
        };
        let mut sum = vec![];
        for i in 1..= i32 {
            sum[i] = sum[i - 1] + i;
        }
        Self { sum }
    }

    fn pick_index(self) -> i32 {
        let s = thread_rng().gen_range(1, self.sum.len().saturating() + 1);
        let (mut left, mut right) = (1, self.sum.len() - 1);
        while left < right {
            let mid = (left + right) / 2;

            if self.sum[mid] < s {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        (left - 1) as i32
    }
}

/*
 * Your Solution object will be instantiated and called as such:
 * let obj = Solution::new();
 * let ret_1 = obj.pick_index();
 */

Python
```

[7] Binary Search — Pattern Solution (matched from community answers)

```
import random
import bisect

class Solution:
    def __init__(self, w):
        # Build the prefix sum array.
        self.prefix_sum = []
        current_sum = 0
        for weight in w:
            current_sum += weight
            self.prefix_sum.append(current_sum)
        # Store the total sum for weights.
        self.total_sum = current_sum

    def pickIndex(self):
        # Get a random target in the range [1, total_sum]
        target = random.randint(1, self.total_sum)
        # Use bisect.bisect to find the answer index.
        index = bisect.bisect_left(self.prefix_sum, target)
        return index

# Example usage:
# solution = Solution([1, 1])
# print(solution.pickIndex())
```

#852 **MEDIUM**
Peak Index in a Mountain Array
LeetCode · SimplyLeet · WalkCC · LeetCode · Editorial

Array · Binary Search
Likes: Denny Zhang

Problem:
You are given an integer mountain array arr of length n where the values increase to a peak element and then decrease.

Return the index of the peak element.
Your task is to solve it in O(log(n)) time complexity.

Example 1:

Input: arr = [0,1,0]

Output: 1

Example 2:

Input: arr = [0,2,1,0]

Output: 1

Example 3:

Input: arr = [0,10,5,2]

Output: 1

Constraints:

• 3 <= arr.length <= 105
• 0 <= arr[i] <= 106
• arr is guaranteed to be a mountain array.

>> Brute Force [Binary Search] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def peakIndexInMountainArray(self, arr):
        # Brute Force: Iterate through the array and find the peak index.
        for i, val in enumerate(arr):
            if val == target:
                return i
        return -1
```

```
Python
def peakIndexInMountainArray(arr):
    # Initialization pointers for the binary search
    left, right = 0, len(arr) - 1
    # Continue while there is more than one element in the search space
    while left < right:
        mid = (left + right) // 2
        # If the element at mid is increasing, move the left pointer to mid + 1
        if arr[mid] < arr[mid + 1]:
            left = mid + 1
        else:
            # Otherwise, the peak is at mid or to the left of mid
            right = mid
    # When left and right converge, we have the peak index
    return left
```

Example usage:
print(peakIndexInMountainArray([0, 1, 0])) # Output: 1

```
Python
class Solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        l = 0
        r = len(arr) - 1
        while l < r:
            m = (l + r) // 2
            if arr[m] < arr[m + 1]:
                l = m + 1
            else:
                r = m
        return l
```

Python

```
class Solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        left, right = 0, len(arr) - 2
        while left < right:
            mid = (left + right) // 2
            if arr[mid] < arr[mid + 1]:
                left = mid + 1
            else:
                right = mid
        return left
```

```
Python
class Solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        left, right = 0, len(arr) - 2
        while left < right:
            mid = (left + right) // 2
            if arr[mid] < arr[mid + 1]:
                left = mid + 1
            else:
                right = mid
        return left
```

C++

```
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {
        int l = 0;
        int r = arr.size() - 1;
        while (l < r) {
            int m = (l + r) / 2;
            if (arr[m] < arr[m + 1])
                l = m + 1;
            else
                r = m;
        }
        return l;
    }
};
```

C++

```
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {
        int left = 0, right = arr.size() - 2;
        while (left < right) {
            int mid = (left + right) // 2;
            if (arr[mid] < arr[mid + 1])
                left = mid + 1;
            else
                right = mid;
        }
        return left;
    }
};
```

C++

```
class Solution {
public:
    int peakIndexInMountainArray(vector<int>& arr) {
        int left = 0, right = arr.size() - 2;
        while (left < right) {
            int mid = (left + right) // 2;
            if (arr[mid] < arr[mid + 1])
                left = mid + 1;
            else
                right = mid;
        }
        return left;
    }
};
```

Java

```
class Solution {
    public int peakIndexInMountainArray(int[] arr) {
        int l = 0;
        int r = arr.length - 1;
        while (l < r) {
            int m = (l + r) / 2;
            if (arr[m] < arr[m + 1])
                l = m + 1;
            else
                r = m;
        }
        return l;
    }
}
```

Java

```

class Solution {
public: int peakIndexInMountainArray(int[] arr) {
    int left = 1, right = arr.length - 2;
    while (left < right) {
        int mid = (left + right) / 2;
        if (arr[mid] > arr[mid + 1]) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
}
}

```

Java

```

class Solution {
public: int peakIndexInMountainArray(int[] arr) {
    int left = 1, right = arr.length - 2;
    while (left < right) {
        int mid = (left + right) / 2;
        if (arr[mid] > arr[mid + 1]) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
}
}

```

JavaScript

JavaScript

```

/**
 * @param {number[]} arr
 * @return {number}
 */
var peakIndexInMountainArray = function(arr) {
    let left = 1;
    let right = arr.length - 2;
    while (left < right) {
        const mid = (left + right) / 2;
        if (arr[mid] > arr[mid + 1]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return left;
};

```

JavaScript

```

/**
 * @param {number[]} arr
 * @return {number}
 */
var peakIndexInMountainArray = function(arr) {
    let left = 1;
    let right = arr.length - 2;
    while (left < right) {
        const mid = (left + right) / 2;
        if (arr[mid] > arr[mid + 1]) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
};

```

TypeScript

TypeScript

```

function peakIndexInMountainArray(arr: number[]): number {
    let left = 1;
    right = arr.length - 2;
    while (left < right) {
        const mid = (left + right) / 2;
        if (arr[mid] > arr[mid + 1]) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
}

```

TypeScript

```

function peakIndexInMountainArray(arr: number[]): number {
    let left = 1;
    right = arr.length - 2;
    while (left < right) {
        const mid = (left + right) / 2;
        if (arr[mid] > arr[mid + 1]) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
}

```

Go

Go

```

func peakIndexInMountainArray(arr []int) int {
    left, right := 1, len(arr)-2
    for left < right {
        mid := (left + right) / 2
        if arr[mid] > arr[mid+1] {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return left
}

```

Go

```

func peakIndexInMountainArray(arr []int) int {
    left, right := 1, len(arr)-2
    for left < right {
        mid := (left + right) / 2
        if arr[mid] > arr[mid+1] {
            right = mid
        } else {
            left = mid + 1
        }
    }
    return left
}

```

Rust

Rust

```

impl Solution {
    pub fn peak_index_in_mountain_array(arr: Vec<i32>) -> i32 {
        let mut left = 1;
        let mut right = arr.len() - 2;
        while left < right {
            let mid = (left + right) / 2;
            if arr[mid] > arr[mid + 1] {
                right = mid;
            } else {
                left = left + 1;
            }
        }
        left as i32
    }
}

```

Rust

• String get(String key, int timestamp) Returns a value such that set was called previously, with timestamp, prev <= timestamp. If there are multiple such values, it returns the value associated with the largest timestamp, prev. If there are no values, it returns "".

Example 1:

```

Input
["TimeMap", "set", "get", "get", "set", "get", "get"]
[[], [{"foo", "bar", 1}], [{"foo", 1}], [{"foo", "bar2", 4}], [{"foo", 4}], [{"foo", 5}]
Output
[null, null, "bar", "bar", null, "bar2", "bar2"]

```

Explanation

```

TimeMap timeMap = new TimeMap();
timeMap.set("foo", "bar", 1); // store the key "foo" and value "bar" along with timestamp = 1.
timeMap.get("foo", 1); // return "bar"
timeMap.get("foo", 3); // return "bar", since there is no value corresponding to foo at timestamp 3 and timestamp 2, then the only value is at timestamp 1 is "bar".
timeMap.set("foo", "bar2", 4); // store the key "foo" and value "bar2" along with timestamp = 4.
timeMap.get("foo", 4); // return "bar2"
timeMap.get("foo", 5); // return "bar2"

```

Constraints:

- 1 <= key.length, value.length <= 100
- key and value consist of lowercase English letters and digits.
- 1 <= timestamp <= 107
- All the timestamps timestamp of set are strictly increasing.
- At most 2 * 105 calls will be made to set and get.

Python

Python

```

impl Solution {
    pub fn peak_index_in_mountain_array(arr: Vec<i32>) -> i32 {
        let mut left = 1;
        let mut right = arr.len() - 2;
        while left < right {
            let mid = (left + right) / 2;
            if arr[mid] > arr[mid + 1] {
                right = mid;
            } else {
                left = left + 1;
            }
        }
        left as i32
    }
}

```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

```

def peakIndexInMountainArray(arr):
    # Initialize pointers of the binary search
    left, right = 0, len(arr) - 1
    # Continue while there is more than one element in the search space
    while left < right:
        mid = (left + right) // 2 # Find the mid index
        # If the sequence is still increasing, move the left pointer to mid+1
        if arr[mid] < arr[mid + 1]:
            left = mid + 1
        else:
            # Otherwise, the peak is at mid or to the left of mid
            right = mid
    # When left and right converge, we have the peak index
    return left

# Example usage:
# print(peakIndexInMountainArray([0, 1, 811])) # Output: 1

```

#981

MEDIUM

Time Based Key-Value Store

LeetCode - Simplilearn - WUOLCC - LeetCode - Editorial

Hash Table - String - Binary Search - Design

Licic Grind 169

Problem:

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

- TimeMap() Initializes the object of the data structure.
- void set(String key, String value, int timestamp) Stores the key key with the value value at the given time timestamp.

```

# TimeMap class in Python with time-by-time comments.
class TimeMap:
    def __init__(self):
        # Initialize the dictionary to hold key and list of (timestamp, value) pairs.
        self.store = {}

    def set(self, key: str, value: str, timestamp: int) -> None:
        # Append the new (timestamp, value) pair for the given key.
        if key not in self.store:
            self.store[key] = []
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        # Return the value for the largest timestamp <= the given timestamp.
        if key not in self.store:
            return ""

        values = self.store[key]
        left, right = 0, len(values) - 1

        # Binary search for the rightmost index with timestamp <= given timestamp.
        while left < right:
            mid = (left + right) // 2
            if values[mid][0] <= timestamp:
                left = mid + 1
            else:
                right = mid - 1

        # If no valid timestamp exists, right will be -1.
        if right == -1:
            return ""
        return values[right][1]

# Example usage:
timeMap = TimeMap()
timeMap.set("foo", "bar", 1)
print(timeMap.get("foo", 1)) # Output: "bar"
print(timeMap.get("foo", 3)) # Output: "bar"
timeMap.set("foo", "bar2", 4)
print(timeMap.get("foo", 4)) # Output: "bar2"
print(timeMap.get("foo", 5)) # Output: "bar2"

```

Python

```

class TimeMap:
    def __init__(self):
        self.values = collections.defaultdict(list)
        self.timestamps = collections.defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.values[key].append(value)
        self.timestamps[key].append(timestamp)

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.timestamps:
            return ""
        i = bisect.bisectleft(self.timestamps[key], timestamp)
        return self.values[key][i - 1] if i > 0 else ""

```



```
Python
class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append(timestamp, value)

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ""
        tv = self.ktv[key]
        i = bisect_right(tv, timestamp, chr(127))
        return tv[i - 1][1] if i else ""

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)

Python
class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append(timestamp, value)

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ""
        tv = self.ktv[key]
        i = bisect_right(tv, timestamp, chr(127))
        return tv[i - 1][1] if i else ""

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)

C++
C++
```

```
struct T {
    string value;
    int timestamp;
};

class TimeMap {
public:
    void set(string key, string value, int timestamp) {
        map[key].emplace_back(value, timestamp);
    }

    string get(string key, int timestamp) {
        if (map.count(key))
            return "";

        const vector<T> &A = map[key];
        int l = 0;
        int r = A.size();

        while (l < r) {
            const int m = (l + r) / 2;
            if (A[m].timestamp > timestamp)
                r = m;
            else
                l = m + 1;
        }

        return l == 0 ? "" : A[l - 1].value;
    }
};

unordered_map<string, vector<T>> map;

C++
```

```
class TimeMap {
public:
    TimeMap() {}

    void set(string key, string value, int timestamp) {
        ktv[key].emplace_back(timestamp, value);
    }

    string get(string key, int timestamp) {
        auto& pairs = ktv[key];
        pair<int, string> p = {timestamp, string(127)};
        auto i = upper_bound(pairs.begin(), pairs.end(), p);
        return i == pairs.begin() ? "" : (i - 1) ->second;
    }

private:
    unordered_map<string, vector<pair<int, string>>> ktv;
};

/*
 * Your TimeMap object will be instantiated and called as such:
 * TimeMap obj = new TimeMap();
 * obj.set(key,value,timestamp);
 * string param_2 = obj.get(key,timestamp);
 */

C++
class TimeMap {
public:
    TimeMap() {}

    void set(string key, string value, int timestamp) {
        ktv[key].emplace_back(timestamp, value);
    }

    string get(string key, int timestamp) {
        auto& pairs = ktv[key];
        pair<int, string> p = {timestamp, string(127)};
        auto i = upper_bound(pairs.begin(), pairs.end(), p);
        return i == pairs.begin() ? "" : (i - 1) ->second;
    }

private:
    unordered_map<string, vector<pair<int, string>>> ktv;
};

/*
 * Your TimeMap object will be instantiated and called as such:
 * TimeMap obj = new TimeMap();
 * obj.set(key,value,timestamp);
 * string param_2 = obj.get(key,timestamp);
 */

Java
```

```
Java
class T {
    public String value;
    public int timestamp;
    public T(String value, int timestamp) {
        this.value = value;
        this.timestamp = timestamp;
    }
}

class TimeMap {
    public void set(String key, String value, int timestamp) {
        map.putIfAbsent(key, new ArrayList<>());
        map.get(key).add(new T(value, timestamp));
    }

    public String get(String key, int timestamp) {
        List<T> A = map.get(key);
        if (A == null)
            return "";

        int l = 0;
        int r = A.size();

        while (l < r) {
            final int m = (l + r) / 2;

            if (A.get(m).timestamp > timestamp)
                r = m;
            else
                l = m + 1;
        }

        return l == 0 ? "" : A.get(l - 1).value;
    }

    private Map<String, List<T>> map = new HashMap<>();
}

Java
```

```
class TimeMap {
    private Map<String, TreeMap<Integer, String>> ktv = new HashMap<>();

    public TimeMap() {}

    public void set(String key, String value, int timestamp) {
        ktv.computeIfAbsent(key, k -> new TreeMap<>()).put(timestamp, value);
    }

    public String get(String key, int timestamp) {
        if (!ktv.containsKey(key))
            return "";

        var tv = ktv.get(key);
        Integer t = tv.floorKey(timestamp);
        return t == null ? "" : tv.get(t);
    }
}

/*
 * Your TimeMap object will be instantiated and called as such:
 * TimeMap obj = new TimeMap();
 * obj.set(key,value,timestamp);
 * String param_2 = obj.get(key,timestamp);
 */

Java
class TimeMap {
    private Map<String, TreeMap<Integer, String>> ktv = new HashMap<>();

    public TimeMap() {}

    public void set(String key, String value, int timestamp) {
        ktv.computeIfAbsent(key, k -> new TreeMap<>()).put(timestamp, value);
    }

    public String get(String key, int timestamp) {
        if (!ktv.containsKey(key))
            return "";

        var tv = ktv.get(key);
        Integer t = tv.floorKey(timestamp);
        return t == null ? "" : tv.get(t);
    }
}

/*
 * Your TimeMap object will be instantiated and called as such:
 * TimeMap obj = new TimeMap();
 * obj.set(key,value,timestamp);
 * String param_2 = obj.get(key,timestamp);
 */

Go
```

```
Go
type TimeMap struct {
    ktv map[string][]pair
}

func Constructor() TimeMap {
    return TimeMap{map[string][]pair{}}
}

func (this *TimeMap) Set(key string, value string, timestamp int) {
    this.ktv[key] = append(this.ktv[key], pair(timestamp, value))
}

func (this *TimeMap) Get(key string, timestamp int) string {
    pairs := this.ktv[key]
    i := sort.Search(len(pairs), func(i int) bool { return pairs[i].t > timestamp })
    if i < 0 {
        return pairs[i-1].v
    }
    return ""
}

type pair struct {
    t int
    v string
}

/*
 * Your TimeMap object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Set(key,value,timestamp);
 * param_2 := obj.Get(key,timestamp);
 */

Go
```

```

type TimeMap struct {
    kv map[string][]pair
}

func Constructor() TimeMap {
    return TimeMap{kv: map[string][]pair{}}
}

func (this *TimeMap) Set(key string, value string, timestamp int) {
    this.kv[key] = append(this.kv[key], pair(timestamp, value))
}

func (this *TimeMap) Get(key string, timestamp int) string {
    pairs := this.kv[key]
    i := sort.Search(len(pairs), func(i int) bool { return pairs[i].t > timestamp })
    if i > 0 {
        return pairs[i-1].v
    }
    return ""
}

type pair struct {
    t int
    v string
}

```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

Binary Search · LeetCode — By Pattern

— 2383 —

LeetCode

```

# TimeMap class in Python with time-by-time comments.
class TimeMap:
    def __init__(self):
        # Initialize the dictionary to hold key and list of (timestamp, value) pairs.
        self.store = {}

    def set(self, key: str, value: str, timestamp: int) -> None:
        # Append the new (timestamp, value) pair for the given key.
        if key not in self.store:
            self.store[key] = []
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        # Return the value for the target timestamp -- the given timestamp.
        # If key not in self.store:
            return ""

        values = self.store[key]
        left, right = 0, len(values) - 1

        # Binary search for the rightmost index with timestamp <= given timestamp.
        while left < right:
            mid = (left + right) // 2
            if values[mid][0] <= timestamp:
                left = mid + 1
            else:
                right = mid - 1

        # If no valid timestamp exists, right will be -1.
        if right == -1:
            return ""
        return values[right][1]

# Example usage:
# timeMap = TimeMap()
# timeMap.set("foo", "bar", 1)
# print(timeMap.get("foo", 1)) # Output: "bar"
# print(timeMap.get("foo", 3)) # Output: "bar"
# timeMap.set("foo", "bar2", 4)
# print(timeMap.get("foo", 4)) # Output: "bar2"
# print(timeMap.get("foo", 5)) # Output: "bar2"

```

#1011 MEDIUM

Capacity to Ship Packages Within D Days

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Binary Search

Licite Denny Zhang

Problem:

A conveyor belt has packages that must be shipped from one port to another within days days.

The ith package on the conveyor belt has a weight of weights[i]. Each day, we load the ship with packages on the conveyor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship.

Binary Search · LeetCode — By Pattern

— 2384 —

LeetCode

Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within days days.

Example 1:

Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5

Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:

1st day: 1, 2, 3, 4, 5
2nd day: 6, 7
3rd day: 8
4th day: 9
5th day: 10

Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.

Example 2:

Input: weights = [3,2,2,4,1,4], days = 3

Output: 6

Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:

1st day: 3, 2
2nd day: 2, 4
3rd day: 1, 4

Example 3:

Input: weights = [1,2,3,1,1], days = 4

Output: 3

Explanation:

1st day: 1
2nd day: 2
3rd day: 3
4th day: 1, 1

Constraints:

• 1 <= days <= weights.length <= 5 * 10⁴
• 1 <= weights[i] <= 500

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        # Brute Force O(n) - Linear scan instead of binary search
        for i, val in enumerate(weights):
            if val == target:
                return i
        return -1

```

Python

Binary Search · LeetCode — By Pattern

— 2385 —

LeetCode

```

Python
def shipWithinDays(weights, days):
    # Assume there is no answer if a given capacity can ship packages within 'days' days.
    def canShip(capacity):
        current_load = 0 # Current total weight on the ship for the current day.
        required_days = 1 # Start with the first day.
        for weight in weights:
            # If adding the current weight exceeds capacity, increment the day counter.
            if current_load + weight > capacity:
                required_days += 1
                current_load = 0
            current_load += weight
        return required_days == days

    # Establish lower and upper bounds for binary search.
    low, high = min(weights), sum(weights)
    while low < high:
        mid = (low + high) // 2
        # If mid capacity can ship all packages within the allowed days, try a lower capacity.
        if canShip(mid):
            high = mid
        else:
            low = mid + 1
    return low

```

Example usage:

```

weights = [1,2,3,4,5,6,7,8,9,10]
days = 5
print(shipWithinDays(weights, days))

```

Python

```

class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def shipDays(shipCapacity: int) -> int:
            shipDays = 1
            capacity = 0
            for weight in weights:
                if capacity + weight > shipCapacity:
                    shipDays += 1
                    capacity = weight
            else:
                capacity += weight
            return shipDays

        l = min(weights)
        r = sum(weights)
        return bisect.bisect_left(range(l, r), True, key=lambda n: shipDays(n) == days) + 1

```

Python

Binary Search · LeetCode — By Pattern

— 2386 —

LeetCode

```

class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def check(n):
            w, cnt = 0, 1
            for w in weights:
                w += w
                if w > n:
                    cnt += 1
                    w = 0
            return cnt == days

        left, right = min(weights), sum(weights) + 1
        return left + bisect_left(range(left, right), True, key=check)

```

Python

```

class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def check(n):
            w, cnt = 0, 1
            for w in weights:
                w += w
                if w > n:
                    cnt += 1
                    w = 0
            return cnt == days

        left, right = min(weights), sum(weights) + 1
        return left + bisect_left(range(left, right), True, key=check)

```

C++

C++

```

class Solution {
public:
    int shipWithinDays(vector<int>& weights, int days) {
        int l = ranges::min(weights);
        int r = accumulate(weights.begin(), weights.end(), 0);
        while (l < r) {
            const int m = (l + r) / 2;
            if (shipDays(weights, m) == days)
                r = m;
            else
                l = m + 1;
        }
        return l;
    }

private:
    int shipDays(const vector<int>& weights, int shipCapacity) {
        int days = 1;
        int capacity = 0;
        for (const int weight : weights)
            if (capacity + weight > shipCapacity) {
                ++days;
                capacity = weight;
            }
        return days;
    }
};

```

Binary Search · LeetCode — By Pattern

— 2387 —

LeetCode

```

    } else {
        capacity == weight;
    }
    return days;
};

```

C++

```

class Solution {
public:
    int shipWithinDays(vector<int>& weights, int days) {
        int left = 0, right = 0;
        for (auto& w : weights) {
            left = max(left, w);
            right += w;
        }
        auto check = [&](int m) {
            int w = 0, cnt = 1;
            for (auto& w : weights) {
                w += w;
                if (w > m) {
                    w = 0;
                    ++cnt;
                }
            }
            return cnt == days;
        };
        while (left < right) {
            int mid = (left + right) / 2;
            if (check(mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
};

```

C++

Binary Search · LeetCode — By Pattern

— 2388 —

LeetCode

```

class Solution {
public:
    int shipWithinDays(vector<int>& weights, int days) {
        int left = 0, right = 0;
        for (const int w : weights) {
            left = max(left, w);
            right += w;
        }
        auto check = [&](int mid) {
            int wt = 0, cnt = 1;
            for (const int w : weights) {
                wt += w;
                if (wt > mid) {
                    wt = w;
                    ++cnt;
                }
            }
            return cnt <= days;
        };
        while (left < right) {
            int mid = (left + right) / 2;
            if (check(mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
};

```

Java

Java

Binary Search - LeetCode - By Pattern

— 2389 —

LeetCode

```

class Solution {
public:
    int shipWithinDays(int[] weights, int days) {
        int l = Arrays.stream(weights).max().getAsInt();
        int r = Arrays.stream(weights).sum();

        while (l < r) {
            final int m = (l + r) / 2;
            if (shipWithinDays(weights, m) <= days) {
                r = m;
            } else {
                l = m + 1;
            }
        }
        return l;
    }

    private int shipWithinDays(int[] weights, int shipCapacity) {
        int days = 1;
        int capacity = 0;
        for (final int weight : weights) {
            if (capacity + weight > shipCapacity) {
                capacity = weight;
                days++;
            } else {
                capacity += weight;
            }
        }
        return days;
    }
}

```

Java

```

class Solution {
public:
    int shipWithinDays(int[] weights, int days) {
        int left = 0, right = 0;
        for (int w : weights) {
            left = Math.max(left, w);
            right += w;
        }
        while (left < right) {
            int mid = (left + right) / 2;
            if (check(mid, weights, days)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }

    private boolean check(int m, int[] weights, int days) {
        int wt = 0, cnt = 1;
        for (int w : weights) {
            wt += w;
            if (wt > m) {
                wt = w;
                ++cnt;
            }
        }
        return cnt <= days;
    }
}

```

Binary Search - LeetCode - By Pattern

— 2390 —

LeetCode

```

    }
    return cnt <= days;
}
}

```

Java

```

class Solution {
public:
    int shipWithinDays(int[] weights, int days) {
        int left = 0, right = 0;
        for (const int w : weights) {
            left = Math.max(left, w);
            right += w;
        }
        while (left < right) {
            int mid = (left + right) / 2;
            if (check(mid, weights, days)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }

    private boolean check(int m, int[] weights, int days) {
        int wt = 0, cnt = 1;
        for (int w : weights) {
            wt += w;
            if (wt > m) {
                wt = w;
                ++cnt;
            }
        }
        return cnt <= days;
    }
}

```

TypeScript

TypeScript

Binary Search - LeetCode - By Pattern

— 2391 —

LeetCode

```

function shipWithinDays(weights: number[], days: number): number {
    let left = 0;
    let right = 0;
    for (const w of weights) {
        left = Math.max(left, w);
        right += w;
    }
    const check = (mx: number) => {
        let wt = 0;
        let cnt = 1;
        for (const w of weights) {
            wt += w;
            if (wt > mx) {
                wt = w;
                ++cnt;
            }
        }
        return cnt <= days;
    };
    while (left < right) {
        const mid = (left + right) / 2;
        if (check(mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
}

```

TypeScript

```

function shipWithinDays(weights: number[], days: number): number {
    let left = 0;
    let right = 0;
    for (const w of weights) {
        left = Math.max(left, w);
        right += w;
    }
    const check = (mx: number) => {
        let wt = 0;
        let cnt = 1;
        for (const w of weights) {
            wt += w;
            if (wt > mx) {
                wt = w;
                ++cnt;
            }
        }
        return cnt <= days;
    };
    while (left < right) {
        const mid = (left + right) / 2;
        if (check(mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
}

```

Binary Search - LeetCode - By Pattern

— 2392 —

LeetCode

```

    }
    return left;
}
}

```

Go

```

func shipWithinDays(weights []int, days int) int {
    var left, right int
    for w := range weights {
        if left + w < left {
            left = w
        }
        right += w
    }
    return left + sort.Search(right, func(int) bool {
        wt := left
        wt, cnt := 0, 1
        for w := range weights {
            wt += w
            if wt > cnt {
                wt = w
                cnt++
            }
        }
        return cnt <= days
    })
}

```

Go

```

func shipWithinDays(weights []int, days int) int {
    var left, right int
    for w := range weights {
        if left + w < left {
            left = w
        }
        right += w
    }
    return left + sort.Search(right, func(int) bool {
        wt := left
        wt, cnt := 0, 1
        for w := range weights {
            wt += w
            if wt > cnt {
                wt = w
                cnt++
            }
        }
        return cnt <= days
    })
}

```

[*] Binary Search - Pattern Solution (matched from community answers)

Python

Binary Search - LeetCode - By Pattern

— 2393 —

LeetCode

```

def shipWithinDays(weights):
    # Helper function to determine if a given capacity can ship packages within 'days' days.
    def canShip(capacity):
        current_load = 0
        required_days = 1
        for weight in weights:
            # If adding the current weight exceeds capacity, increment the day counter.
            if current_load + weight > capacity:
                required_days += 1
                current_load = 0
            current_load += weight
        return required_days <= days

    # Establish lower and upper bounds for binary search.
    low, high = min(weights), sum(weights)
    while low < high:
        mid = (low + high) // 2
        # If mid capacity can ship all packages within the allowed days, try a lower capacity.
        if canShip(mid):
            high = mid
        else:
            low = mid + 1
    return low

# Example usage:
weights = [1,2,3,4,5,6,7,8,9,10]
days = 5
print(shipWithinDays(weights, days))

```

#1060 MEDIUM

Missing Element in Sorted Array

LeetCode - Simplify - WalkCC - LeetCode - Editorial

Array - Binary Search

List Premium 98

Problem:

Given an integer array nums which is sorted in ascending order and all of its elements are unique and given also an integer k, return the kth missing number starting from the leftmost number of the array.

Example 1:

Input: nums = [4,7,9,10], k = 1

Output: 5

Explanation: The first missing number is 5.

Example 2:

Input: nums = [4,7,9,10], k = 3

Output: 8

Explanation: The missing numbers are [5,6,8,...], hence the third missing number is 8.

Example 3:

Binary Search - LeetCode - By Pattern

— 2394 —

LeetCode

Input: nums = [1,2,4], k = 3
Output: 6
Explanation: The missing numbers are [3,5,6,7,...], hence the third missing number is 6.
Constraints:
• 1 <= nums.length <= 5 * 10⁴
• 1 <= nums[i] <= 10⁷
• nums is sorted in ascending order, and all the elements are unique.
• 1 <= k <= 10⁸

Follow up: Can you find a logarithmic time complexity (i.e., O(log(n))) solution?

>> Brute Force (Binary Search) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def missingElement(self, nums, k):
        # Brute Force Sol: Linear time instead of binary search
        for i, val in enumerate(nums):
            if val == target:
                return k
            return -1
```

Python

```
# Python Solution with comprehensive comments
def missingElement(nums, k):
    # Calculate the missing count until index i: missing(i) = nums[i] - nums[0] - 1.
    def missing_count(index):
        return nums[index] - nums[0] - 1

    n = len(nums)
    # If the missing number is beyond the last element.
    if k > missing_count(n - 1):
        return nums[-1] + k - missing_count(n - 1)

    # Perform binary search to find the smallest index where missing_count(index) == k.
    left, right = 0, n - 1
    while left < right:
        mid = (left + right) // 2
        if missing_count(mid) < k:
            left = mid + 1
        else:
            right = mid

    # After binary search, left is the smallest index such that missing_count(left) == k.
    # The kth missing number is after nums[left - 1].
    return nums[left - 1] + k - missing_count(left - 1)

# Example usage:
print(missingElement([4,7,9,10], 3)) # Output: 8
```

Binary Search · LeetCode · By Pattern

— 2395 —

LeetCode

Python

```
class Solution:
    def missingElement(self, nums: List[int], k: int) -> int:
        def missing(i: int) -> int:
            return nums[i] - nums[0] - 1

        n = len(nums)
        if k > missing(n - 1):
            return nums[-1] + k - missing(n - 1)

        l, r = 0, n - 1
        while l < r:
            mid = (l + r) // 2
            if missing(mid) >= k:
                r = mid
            else:
                l = mid + 1
        return nums[l - 1] + k - missing(l - 1)
```

Python

```
class Solution:
    def missingElement(self, nums: List[int], k: int) -> int:
        def missing(i: int) -> int:
            return nums[i] - nums[0] - 1

        n = len(nums)
        if k > missing(n - 1):
            return nums[-1] + k - missing(n - 1)

        l, r = 0, n - 1
        while l < r:
            mid = (l + r) // 2
            if missing(mid) >= k:
                r = mid
            else:
                l = mid + 1
        return nums[l - 1] + k - missing(l - 1)
```

C++

C++

Binary Search · LeetCode · By Pattern

— 2396 —

LeetCode

```
class Solution {
public:
    int missingElement(vector<int>& nums, int k) {
        int l = 0;
        int r = nums.size();

        // The number of missing numbers in [nums[l], nums[r]]
        auto missing = [&](int i) { return nums[i] - nums[0] - 1; };

        // Find the first index l s.t. missing(l) == k
        while (l < r) {
            int m = (l + r) / 2;
            if (missing(m) == k)
                r = m;
            else
                l = m + 1;
        }
        return nums[l - 1] + (k - missing(l - 1));
    }
};
```

C++

```
class Solution {
public:
    int missingElement(vector<int>& nums, int k) {
        auto missing = [&](int i) {
            return nums[i] - nums[0] - 1;
        };
        int n = nums.size();
        if (k > missing(n - 1)) {
            return nums[-1] + k - missing(n - 1);
        }
        int l = 0, r = n - 1;
        while (l < r) {
            int mid = (l + r) // 2;
            if (missing(mid) == k) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return nums[l - 1] + k - missing(l - 1);
    }
};
```

C++

Binary Search · LeetCode · By Pattern

— 2397 —

LeetCode

```
class Solution {
public:
    int missingElement(vector<int>& nums, int k) {
        auto missing = [&](int i) {
            return nums[i] - nums[0] - 1;
        };
        int n = nums.size();
        if (k > missing(n - 1)) {
            return nums[-1] + k - missing(n - 1);
        }
        int l = 0, r = n - 1;
        while (l < r) {
            int mid = (l + r) // 2;
            if (missing(mid) == k) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return nums[l - 1] + k - missing(l - 1);
    }
};
```

C++

```
class Solution {
public:
    int missingElement(vector<int> nums, int k) {
        int l = 0;
        int r = nums.length;

        // Find the first index l s.t. missing(l) == k
        while (l < r) {
            int m = (l + r) / 2;
            if (missing(nums, m) == k)
                r = m;
            else
                l = m + 1;
        }
        return nums[l - 1] + (k - missing(nums, l - 1));
    }

    // the number of missing numbers in [nums[0], nums[i]]
    private int missing(int i, int k) {
        return nums[i] - nums[0] - 1;
    }
};
```

Java

Binary Search · LeetCode · By Pattern

— 2398 —

LeetCode

```
class Solution {
    public int missingElement(int[] nums, int k) {
        int n = nums.length;
        if (k > missing(nums, n - 1)) {
            return nums[n - 1] + k - missing(nums, n - 1);
        }
        int l = 0, r = n - 1;
        while (l < r) {
            int mid = (l + r) // 2;
            if (missing(nums, mid) == k) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return nums[l - 1] + k - missing(nums, l - 1);
    }

    private int missing(int[] nums, int i) {
        return nums[i] - nums[0] - 1;
    }
}
```

Java

```
class Solution {
    public int missingElement(int[] nums, int k) {
        int n = nums.length;
        if (k > missing(nums, n - 1)) {
            return nums[n - 1] + k - missing(nums, n - 1);
        }
        int l = 0, r = n - 1;
        while (l < r) {
            int mid = (l + r) // 2;
            if (missing(nums, mid) == k) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return nums[l - 1] + k - missing(nums, l - 1);
    }

    private int missing(int[] nums, int i) {
        return nums[i] - nums[0] - 1;
    }
}
```

Go

Go

Binary Search · LeetCode · By Pattern

— 2399 —

LeetCode

```
func missingElement(nums []int, k int) int {
    missing := func(i int) int {
        return nums[i] - nums[0] - 1
    }
    n := len(nums)
    if k > missing(n-1) {
        return nums[n-1] + k - missing(n-1)
    }
    l, r := 0, n-1
    for l < r {
        mid := (l + r) // 2
        if missing(mid) == k {
            r = mid
        } else {
            l = mid + 1
        }
    }
    return nums[l-1] + k - missing(l-1)
}
```

Go

```
func missingElement(nums []int, k int) int {
    missing := func(i int) int {
        return nums[i] - nums[0] - 1
    }
    n := len(nums)
    if k > missing(n-1) {
        return nums[n-1] + k - missing(n-1)
    }
    l, r := 0, n-1
    for l < r {
        mid := (l + r) // 2
        if missing(mid) == k {
            r = mid
        } else {
            l = mid + 1
        }
    }
    return nums[l-1] + k - missing(l-1)
}
```

[1] Binary Search — Pattern Solution (matched from community answers)

Python

Binary Search · LeetCode · By Pattern

— 2400 —

LeetCode

```
# Python solution with comprehensive comments
def missingElement(nums, k):
    # Calculate the missing count until index i: missing(i) = nums[i] - nums[0] - i.
    def missing_count(i):
        return nums[i] - nums[0] - i
    n = len(nums)
    # If kth missing number is beyond the last element.
    if k > missing_count(n - 1):
        return nums[-1] + k - missing_count(n - 1)
    # Perform binary search to find the smallest index where missing_count(index) == k.
    left, right = 0, n - 1
    while left < right:
        mid = left + (right - left) // 2
        if missing_count(mid) < k:
            left = mid + 1
        else:
            right = mid
    # After binary search, left is the smallest index such that missing_count(left) == k.
    # The kth missing number is k after nums[left - 1].
    return nums[left - 1] + k - missing_count(left - 1)

# Example usage:
print(missingElement([4,7,8,10], 3)) # Output: 5
```

#1062 MEDIUM

Longest Repeating Substring

LeetCode · Simplexify · [WuJCC](#) · [LeetCode](#) · [Editorial](#)

String · Binary Search · Dynamic Programming · Rolling Hash · Suffix Array · Hash Function

Lists Denny Zhang

Problem:

Given a string *s*, return the length of the longest repeating substrings. If no repeating substring exists, return 0.

Example 1:

Input: *s* = "abcd"
Output: 0
Explanation: There is no repeating substring.

Example 2:

Input: *s* = "abbaba"
Output: 2
Explanation: The longest repeating substrings are "ab" and "ba", each of which occurs twice.

Example 3:

Input: *s* = "aabcaabbaab"
Output: 3

Binary Search · LeetCode · By Pattern

— 2401 —

LeetCode

```
left, right = 1, n
ans = 0
while left <= right:
    mid = left + (right - left) // 2
    if search(mid):
        ans = mid
        left = mid + 1
    else:
        right = mid - 1
return ans

# Example usage:
s = "aabcaabbaab"
print(longestRepeatingSubstring(s)) # Expected output: 3
```

Python

```
class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        ans = 0
        # dp[i][j] = the number of repeating characters of s[0..i] and s[j..]
        dp = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(1, n + 1):
            for j in range(1, n + 1):
                if s[i - 1] == s[j - 1]:
                    dp[i][j] = 1 + dp[i - 1][j - 1]
                ans = max(ans, dp[i][j])
        return ans
```

Python

```
class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        f = [[0] * n for _ in range(n)]
        ans = 0
        for i in range(1, n):
            for j in range(1, n):
                if s[i] == s[j]:
                    f[i][j] = 1 + f[i - 1][j - 1] if j else 0
                ans = max(ans, f[i][j])
        return ans
```

Python

```
class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        ans = 0
        for i in range(1, n):
            for j in range(1, n):
                if s[i] == s[j]:
                    dp[i][j] = dp[i - 1][j - 1] + 1 if i else 1
                ans = max(ans, dp[i][j])
        return ans
```

Binary Search · LeetCode · By Pattern

— 2403 —

LeetCode

```
class Solution {
public: int longestRepeatingSubstring(String s) {
    const int n = s.length();
    int ans = 0;
    // dp[i][j] = the number of repeating characters of s[0..i] and s[j..]
    int dp = new int[n + 1][n + 1];
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j)
            if (s.charAt(i - 1) == s.charAt(j - 1)) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            }
            ans = Math.max(ans, dp[i][j]);
    return ans;
}
```

Java

```
class Solution {
public: int longestRepeatingSubstring(String s) {
    int n = s.length();
    int ans = 0;
    int dp = new int[n][n];
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (s.charAt(i) == s.charAt(j)) {
                dp[i][j] = 1 + (i > 0 & j > 0 ? dp[i - 1][j - 1] : 0);
                ans = Math.max(ans, dp[i][j]);
            }
    return ans;
}
```

Java

```
class Solution {
public: int longestRepeatingSubstring(String s) {
    int n = s.length();
    int ans = 0;
    int dp = new int[n][n];
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (s.charAt(i) == s.charAt(j)) {
                dp[i][j] = 1 + (i > 0 & j > 0 ? dp[i - 1][j - 1] : 1);
                ans = Math.max(ans, dp[i][j]);
            }
    return ans;
}
```

TypeScript

TypeScript

Binary Search · LeetCode · By Pattern

— 2405 —

LeetCode

Explanation: The longest repeating substring is "aab", which occurs 3 times.

Constraints:

- 1 <= s.length <= 2000
- s consists of lowercase English letters.

>> Brute Force [Binary Search] Interview Prep

Python

Python

Python

```
class Solution:
    def longestRepeatingSubstring(self, s):
        # Binary Search Approach: Linear Scan instead of binary search
        for i, val in enumerate(s):
            if val == target:
                return i
        return -1
```

Python

Python

```
def longestRepeatingSubstring(s):
    n = len(s)
    # Convert characters to integers for rolling hash computation.
    nums = [ord(c) - ord('a') for c in s]
    # Base value for rolling hash.
    x = 26
    # Modulus value to avoid overflow.
    mod = 2**32
    # Helper function to check if any substring of given length l repeats.
    def search(l):
        h = 0
        for i in range(l):
            h = (h * x + nums[i]) % mod
            seen.add(h)
        # Process each l's h and for removing the first digit in the rolling hash.
        xl = pow(x, l, mod)
        for start in range(l, n + 1):
            h = (h * x - nums[start - l] + nums[start + l - 1]) % mod
            if h in seen:
                return True
            seen.add(h)
        return False
```

Binary Search · LeetCode · By Pattern

— 2402 —

LeetCode

C++

C++

```
class Solution {
public:
    int longestRepeatingSubstring(string s) {
        const int n = s.length();
        int ans = 0;
        // dp[i][j] = the number of repeating characters of s[0..i] and s[j..]
        vector<vector<int>> dp(n, vector<int>(n + 1));
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (s[i - 1] == s[j - 1]) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                }
                ans = max(ans, dp[i][j]);
        return ans;
    }
};
```

C++

```
class Solution {
public:
    int longestRepeatingSubstring(string s) {
        int n = s.size();
        vector<vector<int>> dp(n, vector<int>(n));
        int ans = 0;
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (s[i] == s[j]) {
                    dp[i][j] = 1 + (i > 0 & j > 0 ? dp[i - 1][j - 1] : 1);
                    ans = max(ans, dp[i][j]);
                }
        return ans;
    }
};
```

Java

Java

Binary Search · LeetCode · By Pattern

— 2404 —

LeetCode

```
function longestRepeatingSubstrings(s: string): number {
    const n = s.length;
    const f: number[][] = Array.from({ length: n }, () => Array(n).fill(0));
    let ans = 0;
    for (let i = 1; i <= n; ++i)
        for (let j = 1; j <= n; ++j)
            if (s[i - 1] == s[j - 1]) {
                f[i][j] = 1 + f[i - 1][j - 1] || 0;
                ans = Math.max(ans, f[i][j]);
            }
    return ans;
}
```

TypeScript

```
function longestRepeatingSubstrings(s: string): number {
    const n = s.length;
    const f: number[][] = Array.from({ length: n }, () => Array(n).fill(0));
    let ans = 0;
    for (let i = 1; i <= n; ++i)
        for (let j = 1; j <= n; ++j)
            if (s[i - 1] == s[j - 1]) {
                f[i][j] = 1 + f[i - 1][j - 1] || 0;
                ans = Math.max(ans, f[i][j]);
            }
    return ans;
}
```

Go

Go

```
func longestRepeatingSubstrings(s string) (ans int) {
    n := len(s)
    f := make([][]int, n)
    for i := range f {
        f[i] = make([]int, n)
        for j := 1; j <= n; j++ {
            for j := 1; j <= n; j++ {
                if s[i] == s[j] {
                    f[i][j] = f[i - 1][j - 1] + 1
                }
                ans = max(ans, f[i][j])
            }
        }
    }
    return
}
```

Go

Binary Search · LeetCode · By Pattern

— 2406 —

LeetCode

```
func longestRepeatingSubstring(s: String) Int {
    n := len(s)
    dp := make([][]Int, n)
    for i := range dp {
        dp[i] = make([]Int, n)
    }
    ans := 0
    for i := 0; i < n; i++ {
        for j := i + 1; j < n; j++ {
            if s[i] == s[j] {
                if i == 0 {
                    dp[i][j] = 1
                } else {
                    dp[i][j] = dp[i-1][j-1] + 1
                }
                ans = max(ans, dp[i][j])
            }
        }
    }
    return ans
}
```

Rust

```
impl Solution {
    pub fn longest_repeating_substring(s: String) -> i32 {
        let n = s.len();
        let mut f = vec![vec![0; n]; n];
        let mut ans = 0;
        let s = s.as_bytes();
        for i in 1..n {
            for j in 0..i {
                if s[i] == s[j] {
                    f[i][j] = if j > 0 { f[i-1][j-1] + 1 } else { 1 };
                    ans = ans.max(f[i][j]);
                }
            }
        }
        ans
    }
}
```

Rust

```
impl Solution {
    pub fn longest_repeating_substring(s: String) -> i32 {
        let n = s.len();
        let mut f = vec![vec![0; n]; n];
        let mut ans = 0;
        let s = s.as_bytes();
        for i in 1..n {
            for j in 0..i {
                if s[i] == s[j] {
                    f[i][j] = if j > 0 { f[i-1][j-1] + 1 } else { 1 };
                    ans = ans.max(f[i][j]);
                }
            }
        }
        ans
    }
}
```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

```
def longestRepeatingSubstring(s):
    n = len(s)
    # Convert characters to integers for rolling hash computation.
    nums = [ord(c) - ord('a') for c in s]
    # Base value for rolling hash.
    B = 26
    # Modulus value to avoid overflow.
    mod = 2**32
    # Helper function to check if any substring of given length l repeats.
    def search(l):
        h = 0
        for i in range(l):
            h = (h * B + nums[i]) % mod
        ans = h
        # Sliding window of size l and for removing the first digit in the rolling hash.
        al = pow(B, l, mod)
        for start in range(1, n - l + 1):
            h = (h * B - nums[start - 1] + al * nums[start + l - 1]) % mod
            if h < 0:
                return True
            ans.add(h)
        return False
```

```
left, right = 1, n
ans = 0
while left <= right:
    mid = left + (right - left) // 2
    if search(mid):
        ans = mid
        left = mid + 1
    else:
        right = mid - 1
return ans
```

Expected output: 3

```
s = "abacabadba"
print(longestRepeatingSubstring(s)) # Expected output: 3
```

#1533 Find the Index of the Large Integer

LeetCode - Simplify - Waffle - LeetCode - Editorial

Array - Binary Search - Interactive

LeetCode Premium 98

Problem:

We have an integer array `arr`, where all the integers in `arr` are equal except for one integer which is larger than the rest of the integers. You will not be given direct access to the array, instead, you will have an API `ArrayReader` which have the following functions:

- `int compareSub(int l, int r, int x, int y):` where $0 \leq l, r, x, y < \text{ArrayReader.length}$, $l < r$ and $x < y$. The function compares the sum of sub-array `arr[l..r]` with the sum of the sub-array `arr[x..y]` and returns: 1 if `arr[l..r] > arr[x..y]`, -1 if `arr[l..r] < arr[x..y]`, and 0 if `arr[l..r] == arr[x..y]`.
- 0 if `arr[l..r] > arr[x..y]`, -1 if `arr[l..r] < arr[x..y]`, and 0 if `arr[l..r] == arr[x..y]`.
- -1 if `arr[l..r] > arr[x..y]`, 1 if `arr[l..r] < arr[x..y]`, and 0 if `arr[l..r] == arr[x..y]`.

You are allowed to call `compareSub` 20 times at most. You can assume both functions work in $O(1)$ time.

Return the index of the array `arr` which has the largest integer.

Example 1:

Input: `arr = [7,7,7,7,10,7,7]`

Output: 4

Explanation: The following calls to the API `reader.compareSub(0, 0, 1, 1)` // returns 0 this is a query comparing the sub-array `(0, 0)` with the sub array `(1, 1)`, (i.e. compares `arr[0]` with `arr[1]`).

Thus we know that `arr[0]` and `arr[1]` doesn't contain the largest element.
`reader.compareSub(2, 2, 3, 3)` // returns 0, we can exclude `arr[2]` and `arr[3]`.
`reader.compareSub(4, 4, 5, 5)` // returns 1, thus for sure `arr[4]` is the largest element in the array.

Notice that we made only 3 calls, so the answer is valid.

Example 2:

Input: `nums = [6,6,12]`

Output: 2

Constraints:

- $2 \leq \text{arr.length} \leq 5 * 10^5$
- $1 \leq \text{arr}[i] \leq 100$
- All elements of `arr` are equal except for one element which is larger than all other elements.

Follow up:

- What if there are two numbers in `arr` that are bigger than all other numbers?
- What if there is one number that is bigger than other numbers and one number that is smaller than other numbers?

>> Brute Force [Binary Search] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def getIndex(self, reader):
        # Brute Force (O(n)) - Linear time instead of binary search
        for i, val in enumerate(reader):
            if val == target:
                return i
            return -1
```

Python

```
# Python Solution with line-by-line comments
class Solution:
    def getIndex(self, reader):
        # Initialize search bounds
        left, right = 0, reader.length() - 1
        # Binary search for the candidate index
        while left < right:
            length = right - left + 1
            if length % 2 == 0:
                # Divide current range into two equal halves.
                mid = left + length // 2
                # Compare left half (left, mid-1) with right half (mid, right)
                comp = reader.compareSub(left, mid - 1, mid, right)
                if comp == 1:
                    # Left half is larger, search in the left half
                    right = mid - 1
                else:
                    # Right half is larger, search in the right half
                    left = mid
            else:
                # Odd length: divide into two halves of equal size, ignoring middle element.
                half = length // 2
                # Compare left half (left, left+half-1) with right half (right-half+1, right)
                comp = reader.compareSub(left, left + half - 1, right - half + 1, right)
```

```
if comp == 1:
    # Left half is larger, search in the left half
    right = left + half - 1
elif comp == -1:
    # Right half is larger, search in the right half
    left = right + half + 1
else:
    # Both halves are equal, so the middle element is the largest one.
    return left + half
return left
```

Python

```
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
class ArrayReader(object):
    # Compares the sum of arr[l..r] with the sum of arr[x..y]
    # Returns 1 if sum(l..r) > sum(x..y), -1 if sum(l..r) < sum(x..y), and 0 if sum(l..r) == sum(x..y)
    def compareSub(self, l: int, r: int, x: int, y: int) -> int:
        pass
    # Returns the length of the array
    def length(self) -> int:
        pass

class Solution:
    def getIndex(self, reader: ArrayReader) -> int:
        l = 0
        r = reader.length() - 1
        while l < r:
            m = (l + r) // 2
            if (r - l) % 2 == 0:
                res = reader.compareSub(l, m - 1, m + 1, r)
            else:
                res = 0
            if res == 1:
                r = m - 1
            elif res == -1:
                l = m + 1
            else:
                res = reader.compareSub(l, m, m + 1, r)
                # res is either 1 or -1
                if res == 1:
                    r = m
                elif res == -1:
                    l = m + 1
            return l
```

Python

```
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
class ArrayReader(object):
    # Compares the sum of arr[l..r] with the sum of arr[x..y]
    # Returns 1 if sum(l..r) > sum(x..y), -1 if sum(l..r) < sum(x..y), and 0 if sum(l..r) == sum(x..y)
    def compareSub(self, l: int, r: int, x: int, y: int) -> int:
        pass
    # Returns the length of the array
    def length(self) -> int:
        pass

class Solution:
    def getIndex(self, reader: ArrayReader) -> int:
        left, right = 0, reader.length() - 1
        while left < right:
            t1, t2, t3 = (
                left,
                left + (right - left) // 3,
                left + ((right - left) // 3) * 2 + 1,
            )
            comp = reader.compareSub(t1, t2, t2 + 1, t3)
            if comp == 0:
                left = t3 + 1
            elif comp == 1:
                right = t2
            else:
                left, right = t2 + 1, t3
            return left
```

Python

```

// This is ArrayHeader's API interface.
// You should not implement it, or speculate about its implementation
//
class ArrayHeader {
public:
    // Computes the sum of arr[l..r] with the sum of arr[x..y]
    // returns 1 if sum(arr[l..r]) > sum(arr[x..y])
    // returns 0 if sum(arr[l..r]) == sum(arr[x..y])
    // returns -1 if sum(arr[l..r]) < sum(arr[x..y])
    // public int compareSub(int l, int r, int x, int y, int z) {}
    //
    // Returns the length of the array
    // public int length() {}
    //
}

```

```

class Solution {
public:
    int getKthLargest(ArrayHeader reader) {
        int left = 0, right = reader.length() - 1;
        while (left < right) {
            int l1 = left, l2 = l1 + 1, l3 = l2 + 1;
            int r1 = right, r2 = r1 - 1, r3 = r1 - 2;
            if (reader.compareSub(l1, l2, l3, r1, r2, r3) > 0) {
                left = l3;
            } else {
                right = r3;
            }
        }
        return left;
    }
};

```

Java

```

// This is ArrayHeader's API interface.
// You should not implement it, or speculate about its implementation
//
interface ArrayHeader {
    // Computes the sum of arr[l..r] with the sum of arr[x..y]
    // returns 1 if sum(arr[l..r]) > sum(arr[x..y])
    // returns 0 if sum(arr[l..r]) == sum(arr[x..y])
    // returns -1 if sum(arr[l..r]) < sum(arr[x..y])
    // public int compareSub(int l, int r, int x, int y) {}
    //
    // Returns the length of the array
    // public int length() {}
    //
}

class Solution {
    public int getKthLargest(ArrayHeader reader) {
        int l = 0;
        int r = reader.length() - 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if ((r - l) < 2) {
                final int res = reader.compareSub(l, m - 1, m + 1, r);
                if (res == 0)
                    return m;
            } else {
                if (res == 1) {
                    l = m + 1;
                } else {
                    r = m;
                }
            }
        }
        return l;
    }
}

```

Java

```

// This is ArrayHeader's API interface.
// You should not implement it, or speculate about its implementation
//
interface ArrayHeader {
    // Computes the sum of arr[l..r] with the sum of arr[x..y]
    // returns 1 if sum(arr[l..r]) > sum(arr[x..y])
    // returns 0 if sum(arr[l..r]) == sum(arr[x..y])
    // returns -1 if sum(arr[l..r]) < sum(arr[x..y])
    // public int compareSub(int l, int r, int x, int y) {}
    //
    // Returns the length of the array
    // public int length() {}
    //
}

class Solution {
    public int getKthLargest(ArrayHeader reader) {
        int left = 0, right = reader.length() - 1;
        while (left < right) {
            int l1 = left, l2 = l1 + 1, l3 = l2 + 1;
            int r1 = right, r2 = r1 - 1, r3 = r1 - 2;
            if (reader.compareSub(l1, l2, l3, r1, r2, r3) > 0) {
                left = l3;
            } else {
                right = r3;
            }
        }
        return left;
    }
}

```

Java

```

// This is ArrayHeader's API interface.
// You should not implement it, or speculate about its implementation
//
interface ArrayHeader {
    // Computes the sum of arr[l..r] with the sum of arr[x..y]
    // returns 1 if sum(arr[l..r]) > sum(arr[x..y])
    // returns 0 if sum(arr[l..r]) == sum(arr[x..y])
    // returns -1 if sum(arr[l..r]) < sum(arr[x..y])
    // public int compareSub(int l, int r, int x, int y) {}
    //
    // Returns the length of the array
    // public int length() {}
    //
}

class Solution {
    public int getKthLargest(ArrayHeader reader) {
        int left = 0, right = reader.length() - 1;
        while (left < right) {
            int l1 = left, l2 = l1 + 1, l3 = l2 + 1;
            int r1 = right, r2 = r1 - 1, r3 = r1 - 2;
            if (reader.compareSub(l1, l2, l3, r1, r2, r3) > 0) {
                left = l3;
            } else {
                right = r3;
            }
        }
        return left;
    }
}

```

Python

Python

```

# Python solution with line-by-line comments
class Solution:
    def getKthLargest(self, reader):
        # Initialize search bounds
        left, right = 0, reader.length() - 1

        # Binary search for the candidate index
        while left < right:
            length = right - left + 1
            if length % 2 == 0:
                # Even length: split into two equal halves.
                mid = left + length // 2
                # Compare left half [left, mid-1] with right half [mid, right]
                comp = reader.compareSub(left, mid - 1, mid, right)
                if comp == 1:
                    # Large integer is in the left half
                    right = mid - 1
                else:
                    # Large integer is in the right half
                    left = mid
            else:
                # Odd length: divide into two halves of equal size, ignoring middle element.
                half = length // 2
                # Compare left half [left, left+half-1] with right half [left+half, right]
                comp = reader.compareSub(left, left + half - 1, left + half, right)
                if comp == 1:
                    # Large integer is in the left half
                    right = left + half - 1
                elif comp == -1:
                    # Large integer is in the right half
                    left = left + half + 1
                else:
                    # Neither one is right, so the middle element is the large one.
                    return left + half
        return left

```

#4

HARD

Median of Two Sorted Arrays

LeetCode - Binary Search - Divide and Conquer

Lists Grid 169

Problem:

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays.

The overall run time complexity should be O(log(m+n)).

Example 1:

Input: nums1 = [1,3], nums2 = [2]

Output: 2.00000

Explanation: merged array = [1,2,3] and median is 2.

Example 2:

Input: nums1 = [1,2], nums2 = [3,4]

Output: 2.50000

Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.

Constraints:

- nums1.length == m
- nums2.length == n
- 0 <= m <= 1000
- 0 <= n <= 1000
- 1 <= m + n <= 2000
- -100 <= nums1[i], nums2[i] <= 100

>> Brute Force (Binary Search) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def findMedianSortedArrays(self, nums1, nums2):
        # Merge both sorted arrays, sort, find median
        merged = sorted(nums1 + nums2)
        n = len(merged)
        if n % 2 == 1:
            return float(merged[n // 2])
        else:
            return (merged[n // 2 - 1] + merged[n // 2]) / 2.0

```

Python

Python

```

# Partition to find the median of two sorted arrays
def findMedianSortedArrays(nums1, nums2):
    # Binary search for the median
    # Return nums1 is the smaller array for faster binary search
    if len(nums1) < len(nums2):
        nums1, nums2 = nums2, nums1
    n = len(nums1), len(nums2)
    low, high = 0, n
    while low <= high:
        # Partition nums1 into left and right
        partition_nums1 = (low + high) // 2
        partition_nums2 = (n + m + 1) // 2 - partition_nums1
        # If partition is 0 then there are no elements on left side, use -infinity for comparison.
        maxLeftNum1 = float('-inf') if partition_nums1 == 0 else nums1[partition_nums1 - 1]
        # If partition is length then there are no elements on right side, use infinity for comparison.
        minRightNum1 = float('inf') if partition_nums1 == n else nums1[partition_nums1]
        # Similarly for nums2
        maxLeftNum2 = float('-inf') if partition_nums2 == 0 else nums2[partition_nums2 - 1]
        minRightNum2 = float('inf') if partition_nums2 == n else nums2[partition_nums2]
        # Check if we have found the correct partition
        if maxLeftNum1 <= minRightNum2 and maxLeftNum2 <= minRightNum1:
            return (maxLeftNum1 + minRightNum1) / 2.0

```

```

# If total number of elements is even
if (n % 2 == 0)
    return (max(leftFind(nums1, nums1Find(nums2) + min(leftRight(nums1, minRight(nums2)) / 2.0
else:
    return float(max(max(leftFind(nums1, nums1Find(nums2))
# If not, select binary search range
elif max(leftFind(nums1, minRight(nums2)
# Move partition to nums1 to left
high = partition_nums1 - 1
else:
    # Move partition to nums1 to right
    low = partition_nums1 + 1

# Example usage
print(findMedianSortedArrays([1,3], [2])) # Output: 2.0
print(findMedianSortedArrays([1,2], [3,4])) # Output: 2.5

Python
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        n1 = len(nums1)
        n2 = len(nums2)
        if n1 == n2:
            return self.findMedianSortedArrays(nums2, nums1)

        l = 0
        r = n1

        while l <= r:
            partition = (l + r) // 2
            partition1 = (n1 + n2 - 1) // 2 - partition
            maxLeft1 = -2**31 if partition1 == 0 else nums1[partition1 - 1]
            maxLeft2 = -2**31 if partition1 == 0 else nums2[partition1 - 1]
            minRight1 = 2**31 - 1 if partition1 == n1 else nums1[partition1]
            minRight2 = 2**31 - 1 if partition1 == n2 else nums2[partition1]
            if maxLeft1 == minRight1 and maxLeft2 == minRight2:
                return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) / 0.5 if (n1 + n2) % 2 == 0 else
            elif maxLeft1 < minRight2:
                # partition1 - 1
            else:
                l = partition1 + 1

Python

```

Binary Search - LeetCode - By Pattern

— 2419 —

LeetCode

```

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        def find(i: int, j: int, k: int) -> float:
            if i == n1:
                return nums2[j + k - 1]
            if j == n2:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])
            p = k // 2
            x = nums1[i + p - 1] if i + p - 1 < n1 else inf
            y = nums2[j + p - 1] if j + p - 1 < n2 else inf
            return find(i + p, j, k - p) if x < y else find(i, j + p, k - p)

        n1 = len(nums1)
        n2 = len(nums2)
        a = find(0, 0, (n1 + n2 + 1) // 2)
        b = find(0, 0, (n1 + n2 + 2) // 2)
        return (a + b) / 2

Python
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        def find(kth: int, i: int, k: int) -> float:
            if i == n1:
                return nums2[k - 1]
            if j == n2:
                return nums1[k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])
            # Division a // b = floor(a/b, b)
            midVal1 = nums1[i + k // 2 - 1] if i + k // 2 - 1 < n1 else math.inf
            midVal2 = nums2[j + k // 2 - 1] if j + k // 2 - 1 < n2 else math.inf
            if midVal1 < midVal2:
                return find(kth(i + k // 2, j, k - k // 2)
            else:
                return find(kth(i, j + k // 2, k - k // 2)

        n = len(nums1)
        m = len(nums2)
        # Division a // b = floor(a/b, b)
        left = (n + m + 1) // 2
        right = (n + m + 2) // 2

Python

```

Binary Search - LeetCode - By Pattern

— 2420 —

LeetCode

```

return (findKth(0, 0, left) + findKth(0, 0, right)) / 2.0

#####
# Iterative version
class Solution(object):
    def findMedianSortedArrays(self, nums1, nums2):
        a, b = sorted(nums1, nums2), key=len
        n = len(a), len(b)
        after = (n + n - 1) / 2
        lo, hi = 0, n
        while lo < hi:
            i = (lo + hi) / 2
            if after - i - 1 < 0 or a[i] > b[after - i - 1]:
                hi = i
            else:
                lo = i + 1
        i = lo
        maxLeft = sorted(a[i + 1 : 2] + b[after - i : after - i + 2])
        return (maxLeft[i] + maxLeft[i + 1]) / 2.0

C++
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        int n1 = nums1.size(), n2 = nums2.size();
        int i = 0, j = 0;
        while (i < n1 && j < n2) {
            if (nums1[i] < nums2[j]) {
                i++;
            } else {
                j++;
            }
        }
        if (i == n1) {
            return nums2[j];
        } else {
            return nums1[i];
        }
    }
};

C++

```

Binary Search - LeetCode - By Pattern

— 2421 —

LeetCode

```

class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        int n1 = nums1.size(), n2 = nums2.size();
        int i = 0, j = 0;
        while (i < n1 && j < n2) {
            if (nums1[i] < nums2[j]) {
                i++;
            } else {
                j++;
            }
        }
        if (i == n1) {
            return nums2[j];
        } else {
            return nums1[i];
        }
    }
};

C++

```

Binary Search - LeetCode - By Pattern

— 2422 —

LeetCode

```

class Solution {
private:
    int m;
    int n;
    int i;
    int j;
    int k;

public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        m = nums1.size();
        n = nums2.size();
        if (m < n) {
            return findMedianSortedArrays(nums2, nums1);
        }
        int a = find(0, 0, (m + n + 1) / 2);
        int b = find(0, 0, (m + n + 2) / 2);
        return (a + b) / 2.0;
    }

private:
    int find(int i, int j, int k) {
        if (i == m) {
            return nums2[k - 1];
        }
        if (j == n) {
            return nums1[k - 1];
        }
        if (k == 1) {
            return min(nums1[i], nums2[j]);
        }
        int p = k / 2;
        int x = i + p - 1 < m ? nums1[i + p - 1] : 1e9;
        int y = j + p - 1 < n ? nums2[j + p - 1] : 1e9;
        return x < y ? find(i + p, j, k - p) : find(i, j + p, k - p);
    }
};

Java

```

Binary Search - LeetCode - By Pattern

— 2423 —

LeetCode

```

public class Solution {
private:
    int m;
    int n;
    int i;
    int j;
    int k;

public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        m = nums1.size();
        n = nums2.size();
        if (m < n) {
            return findMedianSortedArrays(nums2, nums1);
        }
        int a = find(0, 0, (m + n + 1) / 2);
        int b = find(0, 0, (m + n + 2) / 2);
        return (a + b) / 2.0;
    }

private:
    int find(int i, int j, int k) {
        if (i == m) {
            return nums2[k - 1];
        }
        if (j == n) {
            return nums1[k - 1];
        }
        if (k == 1) {
            return min(nums1[i], nums2[j]);
        }
        int p = k / 2;
        int x = i + p - 1 < m ? nums1[i + p - 1] : 1e9;
        int y = j + p - 1 < n ? nums2[j + p - 1] : 1e9;
        return x < y ? find(i + p, j, k - p) : find(i, j + p, k - p);
    }
};

JavaScript

```

Binary Search - LeetCode - By Pattern

— 2424 —

LeetCode


```

//
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number}
 */
var findMedianSortedArrays = function(nums1, nums2) {
    const m = nums1.length;
    const n = nums2.length;
    const f = (l, j, k) => {
        if (l == m) {
            return nums2[j + k - 1];
        }
        if (j == n) {
            return nums1[l + k - 1];
        }
        if (k == 1) {
            return Math.min(nums1[l], nums2[j]);
        }
        const p = Math.floor(k / 2);
        const x = l + p - 1 <= m ? nums1[l + p - 1] : l <= 30;
        const y = j + p - 1 <= n ? nums2[j + p - 1] : j <= 30;
        return x < y ? f(l + p, j, k - p) : f(l, j + p, k - p);
    };
    const a = f(0, 0, Math.floor((m + n + 1) / 2));
    const b = f(0, 0, Math.floor((m + n + 2) / 2));
    return (a + b) / 2;
};

```

TypeScript

```

function findMedianSortedArrays(nums1: number[], nums2: number[]): number {
    const a = nums1.length;
    const n = nums2.length;
    const f = (l: number, j: number, k: number): number => {
        if (l == a) {
            return nums2[j + k - 1];
        }
        if (j == n) {
            return nums1[l + k - 1];
        }
        if (k == 1) {
            return Math.min(nums1[l], nums2[j]);
        }
        const p = Math.floor(k / 2);
        const x = l + p - 1 <= a ? nums1[l + p - 1] : l <= 30;
        const y = j + p - 1 <= n ? nums2[j + p - 1] : j <= 30;
        return x < y ? f(l + p, j, k - p) : f(l, j + p, k - p);
    };
    const a = f(0, 0, Math.floor((m + n + 1) / 2));
    const b = f(0, 0, Math.floor((m + n + 2) / 2));
    return (a + b) / 2;
}

```

Go

Binary Search - LeetCode - By Pattern

— 2425 —

LeetCode

```

func findMedianSortedArrays(nums1 []int, nums2 []int) float64 {
    m, n := len(nums1), len(nums2)
    var f func(i, j, k int) int
    f = func(i, j, k int) int {
        if i == m {
            return nums2[j+k-1]
        }
        if j == n {
            return nums1[i+k-1]
        }
        if k == 1 {
            return min(nums1[i], nums2[j])
        }
        p := k / 2
        x, y := nums1[i+p-1], nums2[j+p-1]
        if x < y {
            i = i + p
        } else {
            j = j + p
        }
        return f(i, j, k-p)
    }
    a, b := f(0, 0, (m+n+1)/2), f(0, 0, (m+n+2)/2)
    return float64(a+b) / 2.0
}

```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

Binary Search - LeetCode - By Pattern

— 2426 —

LeetCode

```

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        def findKth(k: int, j: int, k: int) -> float:
            if k == 0:
                return nums2[j + k - 1]
            if j == n:
                return nums1[k - 1]
            if k == 1:
                return min(nums1[j], nums2[j])
            # binary search
            midVal1 = nums1[k // 2 - 1] if i + k // 2 - 1 <= m else math.inf
            midVal2 = nums2[j + k // 2 - 1] if j + k // 2 - 1 <= n else math.inf
            if midVal1 < midVal2:
                return findKth(k - k // 2, j, k - k // 2)
            else:
                return findKth(i, j + k // 2, k - k // 2)
        m = len(nums1)
        n = len(nums2)
        # binary search
        left = (m + n + 1) // 2
        right = (m + n + 2) // 2
        return (findKth(0, 0, left) + findKth(0, 0, right)) / 2.0

```

#315 **HARD**
Count of Smaller Numbers After Self

LeetCode - Simplify - WalkCC - LeetCode - Editorial
Array - Binary Search - Divide and Conquer - Binary Indexed Tree - Segment Tree - Merge Sort - Ordered Set
Lists Denny Zhang

Problem:

Binary Search - LeetCode - By Pattern

— 2427 —

LeetCode

Given an integer array nums, return an integer array counts where counts[i] is the number of smaller elements to the right of nums[i].

Example 1:

Input: nums = [5,2,6,1]
Output: [2,1,0,0]

Explanation:

To the right of 5 there are 2 smaller elements (2 and 1).

To the right of 2 there is only 1 smaller element (1).

To the right of 6 there is 1 smaller element (1).

To the right of 1 there is 0 smaller element.

Example 2:

Input: nums = [-1]

Output: [0]

Example 3:

Input: nums = [-1,-1]

Output: [0,0]

Constraints:

• 1 <= nums.length <= 105

• -104 <= nums[i] <= 104

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def countSmaller(self, nums):
        # Brute Force O(n^2) - Linear time instead of binary search
        for i, val in enumerate(nums):
            if val == target:
                return i
            return -1

```

Python

Python

Binary Search - LeetCode - By Pattern

— 2430 —

LeetCode

```

# Python solution using merge sort approach
def countSmaller(nums):
    n = len(nums)
    # Initialize counts array for results
    counts = [0] * n
    # Pair each element with its original index
    nums = [(num, i) for i, num in enumerate(nums)]
    # Recursive function for merge sort with counting
    def mergeSort(arr):
        if len(arr) == 1:
            return arr
        # Split the array into two halves
        mid = len(arr) // 2
        left = mergeSort(arr[:mid])
        right = mergeSort(arr[mid:])
        merged = []
        l = r = 0
        # r_count counts number of elements taken from right array
        r_count = 0
        while l < len(left) and r < len(right):
            if left[l][0] <= right[r][0]:
                # r_count counts the original index
                counts[left[l][1]] += r_count
                merged.append(left[l])
                l += 1
            else:
                # right element is smaller, so increment r_count and add it to merged
                merged.append(right[r])
                r_count += 1
            # Append remaining elements from left reduce count by accumulated right elements
            while l < len(left):
                counts[left[l][1]] += r_count
                merged.append(left[l])
                l += 1
            # Append any remaining elements from right side
            while r < len(right):
                merged.append(right[r])
                r += 1
            return merged
    # Store the merge sort process
    mergeSort(nums)
    return counts

```

Python

Binary Search - LeetCode - By Pattern

— 2429 —

LeetCode

```

class FenwickTree:
    def __init__(self, n: int):
        self.size = n
        self.data = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i <= self.size:
            self.data[i] += delta
            i = FenwickTree.index(i)

    def get(self, i: int) -> int:
        sum = 0
        while i >= 0:
            sum += self.data[i]
            i = FenwickTree.index(i)
        return sum

    @staticmethod
    def index(i: int) -> int:
        return i <= 1

class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        ans = []
        ranks = self._getRanks(nums)
        tree = FenwickTree(len(ranks))
        for num in reversed(nums):
            ans.append(tree.getRanks[num] - 1)
            tree.add(ranks[num], 1)
        return ans[::-1]

    def _getRanks(self, nums: List[int]) -> Dict[int, int]:
        ranks = collections.Counter()
        rank = 0
        for num in sorted(nums):
            rank += 1
            ranks[num] = rank
        return ranks

```

Python

Binary Search - LeetCode - By Pattern

— 2430 —

LeetCode

Python

```

insert(): maintaining a list in sorted order without having to sort the list after each insertion.
https://docs.python.org/3/library/bisect.html

bisect.bisect_left()
Locates the insertion point for x in a to maintain sorted order.

bisect.bisect_right() or bisect.bisect()
Similar to bisect_left(), but returns an insertion point which comes after (to the right of) any existing
entries of x in a.

bisect.insort_left(a, x, lo=0, hi=len(a), *, key=None)
Inserts x in a in sorted order.

Keep in mind that the O(log n) search is dominated by the slow O(n) insertion step.

bisect.insort_right(a, x, lo=0, hi=len(a), *, key=None)
bisect.insort(a, x, lo=0, hi=len(a), *, key=None)
Similar to insort_left(), but inserting x in a after any existing entries of x.

insort.bisect
== bisect.bisect
== bisect.bisect_right([1, 2, 3], 2)
2

insort.bisect_right([1, 2, 3], 2)
3

2

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort_left(a, 1.8)
>>> a
[1.8, 1, 1, 1, 2, 3]

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort_right(a, 1.8)
>>> a
[1, 1, 1, 1.8, 2, 3]

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort(a, 1.8)
>>> a
[1, 1, 1, 1.8, 2, 3]

import bisect

class Solution(object):
    def countSmaller(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """

```

```

    ans = []
    bit = 1
    for num in reversed(nums):
        idx = BinaryIndexedTree.lowbit(num)
        ans.append(idx)
        BinaryInsert(bit, num)
    return ans[::-1]

#####

class BinaryIndexedTree:
    def __init__(self, a):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x = BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x = BinaryIndexedTree.lowbit(x)

```

C++

```
// 02. 2D array
// Time: O(N*M)
// Space: O(1)
class Solution {
public:
    vector<vector<int>> temp;
    void solve(vector<int>&A, int begin, int end) {
        if (begin == end) return;
        int mid = (begin + end) / 2, i = begin, j = mid, k = begin;
        solve(A, mid, end);
        solve(A, begin, mid);
        for (i = mid; i < end; ++i)
            while (i < end && A[i] > A[i+1])
                temp[k++] = A[i];
        ++mid[i] = i == mid;
        while (i < end && A[i] < A[i+1])
            temp[k++] = A[i];
        for (i = end; i < end; ++i) temp[k++] = A[i];
        for (int i = begin; i < end; ++i) A[i] = temp[i];
    }
}

public:
    vector<int> convert(vector<vector<int>&& A) {
        int N = A.size();
        vector<int> temp(N);
        temp.assign(N, 0);
        ans.assign(N, 0);
        solve(A, 0, N);
        return ans;
    }
};
```

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Count_of_Smaller_Numbers_After_Self {

    public static void main(String[] args) {
        Count_of_Smaller_Numbers_After_Self out = new Count_of_Smaller_Numbers_After_Self();
        Solution s = out.new Solution();

        System.out.println(s.countSmaller(new int[]{{5,2,1}}));
    }

    class Solution {
        public List<Integer> countSmaller(int[] nums) {
            List<Integer> result = new ArrayList<>();
            List<Integer> sorted = new ArrayList<>();

            if (nums == null || nums.length == 0) {
                return result;
            }

            for (int i = nums.length - 1; i >= 0; i--) {
                // binary search for current pos. reference: Arrays.binarySearch()
                int left = 0;
                int right = sorted.size();
                while (left < right) {
                    int mid = left + (right - left) / 2;
                    if (sorted.get(mid) < nums[i]) {
                        right = mid;
                    } else {
                        left = mid + 1;
                    }
                }

                // now nums[i] should be placed at index left
                sorted.add(left, nums[i]); // insert equal to .insert()
                result.add(i, left); // insert to list now!!!!!! new ArrayList
            }

            return result;
        }
    }
}

#####

class Solution {

```

```
public List<Integer> countMatches(int[] nums) {
    Set<Integer> s = new HashSet<>();
    for (int i : nums) {
        s.add(i);
    }
    List<Integer> all = new ArrayList<>();
    all.add(Comparator.comparingInt(a) -> a);
    int n = all.size();
    Map<Integer, Integer> nMap = new HashMap<>();
    for (int i = 0; i < n; i++) {
        n.put(all.get(i), i + 1);
    }
    BinaryIndexedTree tree = new BinaryIndexedTree(n);
    List<Integer> ans = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        int l = n.get(i) - 1;
        int r = n.get(i);
        tree.update(i, 1);
        ans.add(firstTree.query(r - 1));
    }
    return ans;
}

class BinaryIndexedTree {
    private int n;

    private List<> c;

    public BinaryIndexedTree(int n) {
        this.n = n;
        c = new ArrayList<>();
    }
}
```

```

type BinaryIndexedTree struct {
    n int
    t [int]
}

func newBinaryIndexedTree(n int) *BinaryIndexedTree {
    c := make([]int, n+1)
    return &BinaryIndexedTree{n, c}
}

func (this *BinaryIndexedTree) lowbit(x int) int {
    return x & -x
}

func (this *BinaryIndexedTree) update(x, delta int) {
    for x <= this.n {
        this.t[x] += delta
        x += this.lowbit(x)
    }
}

func (this *BinaryIndexedTree) query(x int) int {
    s := 0
    for x > 0 {
        s += this.t[x]
        x -= this.lowbit(x)
    }
    return s
}

func countSmaller(nums []int) []int {
    n := len(nums)
    for i, v := range nums {
        v += n
    }
    var alls []int
    for v := range v {
        alls = append(alls, v)
    }
    sort.Ints(alls)
    m = make(map[int]int)
    for i, v := range alls {
        m[v] = i + 1
    }
    ans := make([]int, len(nums))
    tree := newBinaryIndexedTree(len(alls))
    for i := len(nums) - 1; i >= 0; i-- {
        x := m[nums[i]]
        tree.update(x, 1)
        ans[i] = tree.query(x - 1)
    }
    return ans
}

```

Python

```

//
// Insert: maintaining a list in sorted order without having to sort the list after each insertion.
// https://docs.python.org/3/library/bisect.html
//
// bisect.bisect_left()
// Locate the insertion point for x in a to maintain sorted order.
//
// bisect.bisect_right() or bisect.bisect()
// Similar to bisect_left(), but returns an insertion point which comes after (to the right of) any existing
// entries of x in a.
//
// bisect.insort_left(a, x, lo=0, hi=len(a), *, key=None)
// Insert x in a in sorted order.
// Keep in mind that the O(log n) search is dominated by the slow O(n) insertion step.
//
// bisect.insort_right(a, x, lo=0, hi=len(a), *, key=None)
// bisect.insort(a, x, lo=0, hi=len(a), *, key=None)
// Similar to insort_left(), but inserting x in a after any existing entries of x.
//
// => Import bisect
// => bisect.bisect_left([1,2,3], 2)
// 1
// => bisect.bisect_right([1,2,3], 2)
// 2
//
// => a = [1, 1, 1, 2, 2, 3]
// => bisect.insort_left(a, 1.8)
// => a
// [1.8, 1, 1, 1, 2, 3]
//
// => a = [1, 1, 1, 2, 3]
// => bisect.insort_right(a, 1.8)
// => a
// [1, 1, 1, 1.8, 2, 3]
//
// => a = [1, 1, 1, 2, 3]
// => bisect.insort(a, 1.8)
// => a
// [1, 1, 1, 1.8, 2, 3]
//
// Import bisect
//
// class Solution(object):
//     def countSmaller(self, nums):
//         :type nums: List[int]
//         :rtype: List[int]
//

```

```

//
// ans = []
// list = []
// for num in reversed(nums):
//     list = bisect.bisect_left(list, num)
//     ans.append(list)
//     bisect.insort(list, num)
//     return ans[::-1]
//
// =====
//
// class BinaryIndexedTree:
//     def __init__(self, n):
//         self.n = n
//         self.t = [0] * (n + 1)
//
//     def update(self, x, delta):
//         while x <= self.n:
//             self.t[x] += delta
//             x += self.lowbit(x)
//
//     def query(self, x):
//         s = 0
//         while x > 0:
//             s += self.t[x]
//             x -= self.lowbit(x)
//         return s
//

```

#410 **HARD**
Split Array Largest Sum
 LeetCode · Simplify · Walk · LeetCode · Editorial
 Array · Binary Search · Dynamic Programming · Greedy · Prefix Sum
 Little Denny Zhang

Problem:
 Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized.
 Return the minimized largest sum of the split.
 A subarray is a contiguous part of the array.
Example 1:
 Input: `nums = [7,2,5,10,8]`, `k = 2`
 Output: 18
 Explanation: There are four ways to split `nums` into two subarrays.
 The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only 18.
Example 2:

```

//
// Input: nums = [1,2,3,4,5], k = 2
// Output: 9
// Explanation: There are four ways to split nums into two subarrays.
// The best way is to split it into [1,2,3] and [4,5], where the largest sum among the two
// subarrays is only 9.
//
// Constraints:
// * 1 <= nums.length <= 1000
// * 0 <= nums[i] <= 106
// * 1 <= k <= min(50, nums.length)
//
// >> Brute Force (Binary Search) Interview Prep
//
// # Brute Force Approach
//
// class Solution:
//     def splitArray(self, nums: List[int], k: int) -> int:
//         # Brute Force O(n^k) - Linear scan instead of binary search
//         for i, val in enumerate(nums):
//             if val == target:
//                 return i
//
// Python
//
// # Python solution for Split Array Largest Sum
//
// def splitArray(nums, k):
//     # Helper function to determine if we can split array into k subarrays with max subarray sum <= mid
//     def canSplit(maxSum):
//         subarrayCount = 1 # start with one subarray
//         currentSum = 0
//         for num in nums:
//             if currentSum + num > maxSum:
//                 subarrayCount += 1 # start a new subarray
//                 currentSum = num # reset the sum with the current number
//             else:
//                 currentSum += num # add num to current subarray
//         return subarrayCount <= k # valid if we used k or fewer subarrays
//
//     low, high = max(nums), sum(nums) # boundaries for binary search
//     while low < high:
//         mid = (low + high) // 2 # candidate maximum subarray sum
//         if canSplit(mid):
//             high = mid # candidate is valid, try to find a smaller one
//         else:
//             low = mid + 1 # candidate too small, increase it
//     return low
//
// # Example usage:
//
// nums = [7,2,5,10,8]
// k = 2
// print(splitArray(nums, k)) # Expected output: 18
//

```

```

//
// Python
//
// class Solution:
//     def splitArray(self, nums: List[int], k: int) -> int:
//         prefix = list(itertools.accumulate(nums, initial=0))
//
//         @functools.lru_cache(None)
//         def dp(i, k):
//             Returns the minimum of the maximum sum to split the first i numbers into k groups.
//             if i == 1:
//                 return prefix[i]
//             return min(dp(i, k-1, prefix[i] - prefix[j])
//                 for j in range(k-1, 1, 1))
//         return dp(len(nums), k)
//
// Python
//
// class Solution:
//     def splitArray(self, nums: List[int], k: int) -> int:
//         n = len(nums)
//         # dp[i][k] := the minimum of the maximum sum to split the first i numbers
//         # into k groups
//         dp = [[math.inf] * (k + 1) for _ in range(n + 1)]
//         prefix = list(itertools.accumulate(nums, initial=0))
//
//         for i in range(1, n + 1):
//             dp[i][1] = prefix[i]
//
//         for i in range(2, n + 1):
//             for k in range(2, k + 1):
//                 dp[i][k] = min(dp[i][k], max(dp[j][k-1], prefix[i] - prefix[j]))
//
//         return dp[n][k]
//
// Python
//

```

```

//
// class Solution:
//     def splitArray(self, nums: List[int], k: int) -> int:
//         l = max(nums)
//         r = sum(nums) + 1
//
//         def numGroups(maxSum: int) -> int:
//             groupCount = 1
//             sumInGroup = 0
//
//             for num in nums:
//                 if sumInGroup + num > maxSum:
//                     sumInGroup = num
//                     groupCount += 1
//                 else:
//                     sumInGroup += num
//
//             return groupCount
//
//         while l < r:
//             m = (l + r) // 2
//             if numGroups(m) > k:
//                 l = m + 1
//             else:
//                 r = m
//
//         return l
//
// Python
//
// class Solution:
//     def splitArray(self, nums: List[int], k: int) -> int:
//         def check(m):
//             s, cnt = 0, 0
//             for x in nums:
//                 s += x
//                 if s > m:
//                     s = x
//                     cnt += 1
//             return cnt <= k
//
//         left, right = max(nums), sum(nums)
//         return left + bisect_left(range(left, right + 1), True, key=check)
//
// Python
//
// class Solution:
//     def splitArray(self, nums: List[int], k: int) -> int:
//         def check(m):
//             s, cnt = 0, 0
//             for x in nums:
//                 s += x
//                 if s > m:
//                     s = x
//                     cnt += 1
//             return cnt <= k
//
//         left, right = max(nums), sum(nums)
//         return left + bisect_left(range(left, right + 1), True, key=check)
//

```

```

C++
class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        const int n = nums.size();
        // dp[i][j] = the minimum of the maximum sum to split the first i numbers
        // into k groups
        vector<vector<long>> dp(n + 1, vector<long>(k + 1, INT_MAX));
        vector<long> prefix(n + 1);

        partial_sum(nums.begin(), nums.end(), prefix.begin() + 1);

        for (int i = 1; i <= n; ++i)
            dp[i][1] = prefix[i];

        for (int i = 2; i <= k; ++i)
            for (int j = 1; j <= n; ++j)
                dp[i][j] = min(dp[i][j], max(dp[j][i - 1], prefix[j] - prefix[i]));

        return dp[n][k];
    }
};

C++
class Solution {
public:
    int splitArray(vector<vector<int>& nums, int k) {
        int l = ranges::min(nums);
        int r = accumulate(nums.begin(), nums.end(), 0) + 1;

        while (l < r) {
            const int m = (l + r) / 2;
            if (canSplit(nums, m) < k)
                l = m + 1;
            else
                r = m;
        }

        return l;
    }

private:
    int canSplit(const vector<vector<int>& nums, int maxInGroup) {
        int groupCount = 1;
        int sumInGroup = 0;

        for (const int num : nums)
            if (sumInGroup + num >= maxInGroup) {
                sumInGroup = num;
            }
    }
};

```

```

    } else {
        ++groupCount;
        sumInGroup = num;
    }

    return groupCount;
};

C++
class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        int left = 0, right = 0;
        for (int i : nums) {
            left = max(left, i);
            right += i;
        }

        auto check = [&](int m) {
            int s = 1, cnt = 0;
            for (int i : nums) {
                s += i;
                if (s > m) {
                    s = i;
                    ++cnt;
                }
            }
            return cnt <= k;
        };

        while (left < right) {
            int mid = (left + right) >> 1;
            if (check(mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
};

```

C++

```

class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        int left = 0, right = 0;
        for (int i : nums) {
            left = max(left, i);
            right += i;
        }

        auto check = [&](int m) {
            int s = 1, cnt = 0;
            for (int i : nums) {
                s += i;
                if (s > m) {
                    s = i;
                    ++cnt;
                }
            }
            return cnt <= k;
        };

        while (left < right) {
            int mid = (left + right) >> 1;
            if (check(mid)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }
};

```

Java

```

class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        int l = Arrays.stream(nums).max().getAsInt();
        int r = Arrays.stream(nums).sum() + 1;

        while (l < r) {
            final int m = (l + r) / 2;
            if (canSplit(nums, m) < k)
                l = m + 1;
            else
                r = m;
        }

        return l;
    }

private:
    int canSplit(int[] nums, int maxInGroup) {
        int groupCount = 1;
        int sumInGroup = 0;

        for (final int num : nums)
            if (sumInGroup + num >= maxInGroup) {
                sumInGroup = num;
            } else {
                ++groupCount;
            }

        return groupCount;
    }
};

```

Java

```

class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        int left = 0, right = 0;
        for (int i : nums) {
            left = Math.max(left, i);
            right += i;
        }

        while (left < right) {
            int mid = (left + right) >> 1;
            if (check(nums, mid, k)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }

private:
    bool check(vector<int>& nums, int m, int k) {
        int s = 1, cnt = 0;
        for (int i : nums) {
            s += i;
            if (s > m) {
                s = i;
                ++cnt;
            }
        }

        return cnt <= k;
    }
};

Java
class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        int left = 0, right = 0;
        for (int i : nums) {
            left = Math.max(left, i);
            right += i;
        }

        while (left < right) {
            int mid = (left + right) >> 1;
            if (check(nums, mid, k)) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }

        return left;
    }

private:
    bool check(vector<int>& nums, int m, int k) {
        int s = 1, cnt = 0;
        for (int i : nums) {
            s += i;
            if (s > m) {
                s = i;
                ++cnt;
            }
        }

        return cnt <= k;
    }
};

```

```

    }
    return cnt <= k;
}

JavaScript
function splitArray(nums, k) {
    let l = Math.max(...nums);
    let r = nums.reduce((a, b) => a + b);

    const check = (m) => {
        let [s, cnt] = [0, 0];
        for (const n of nums) {
            s += n;
            if (s > m) {
                s = n;
                ++cnt;
            }
        }
        return cnt <= k;
    };

    while (l < r) {
        const mid = (l + r) >> 1;
        if (check(mid)) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }

    return l;
}

```

JavaScript

```

//
// @param {number[]} nums
// @param {number} k
// @return {number}
//
var splitArray = function(nums, k) {
    let l = Math.max(...nums);
    let r = Math.min(nums[0], k) * k;

    const check = (m: number) => {
        let [s, cnt] = [0, 0];
        for (const x of nums) {
            s += x;
            if (s > m) {
                s = 0;
                if (++cnt === k) return false;
            }
        }
        return true;
    };

    while (l < r) {
        const mid = (l + r) >> 1;
        if (check(mid)) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
};

```

TypeScript

TypeScript

```

function splitArray(nums: number[], k: number): number {
    let l = Math.max(...nums);
    let r = Math.min(nums[0], k) * k;

    const check = (m: number) => {
        let [s, cnt] = [0, 0];
        for (const x of nums) {
            s += x;
            if (s > m) {
                s = 0;
                if (++cnt === k) return false;
            }
        }
        return true;
    };

    while (l < r) {
        const mid = (l + r) >> 1;
        if (check(mid)) {
            r = mid;
        } else {
            l = mid + 1;
        }
    }
    return l;
}

```

TypeScript

TypeScript

```

    }
    return left;
}

Go
Go
func splitArray(nums []int, k int) int {
    left, right := 0, 0
    for s := range nums {
        left = max(left, s)
        right += s
    }
    return left + sort.Search(right-left, func(x int) bool {
        m := left
        k, cnt := k-1, 0
        for s := range nums {
            k -= s
            if k <= 0 {
                k = 0
                cnt++
            }
        }
        return cnt <= k
    })
}

Go
func splitArray(nums []int, k int) int {
    left, right := 0, 0
    for s := range nums {
        left = max(left, s)
        right += s
    }
    return left + sort.Search(right-left, func(x int) bool {
        m := left
        k, cnt := k-1, 0
        for s := range nums {
            k -= s
            if k <= 0 {
                k = 0
                cnt++
            }
        }
        return cnt <= k
    })
}

```

Binary Search — Pattern Solution (matched from community answers)

Python

```

# Python solution for Split Array Largest Sum
def splitArray(nums, k):
    # Helper function to determine if we can split array into k subarrays with max subarray sum <= mid
    def canSplit(max_sum):
        subarray_count = 1 # start with one subarray
        current_sum = 0
        for num in nums:
            if current_sum + num > max_sum:
                subarray_count += 1 # start a new subarray
                current_sum = num # reset the sum with the current number
            else:
                current_sum += num # add num to current subarray
        return subarray_count <= k # valid if we used k or fewer subarrays

    low, high = max(nums), sum(nums) # boundaries for binary search
    while low < high:
        mid = (low + high) // 2 # candidate maximum subarray sum
        if canSplit(mid):
            high = mid # candidate is valid, try to find a smaller one
        else:
            low = mid + 1 # candidate too small, increase it
    return low

# Example usage:
nums = [7,2,5,10,8]
k = 2
print(splitArray(nums, k)) # Expected output: 18

```

#668 **HARD**

Kth Smallest Number in Multiplication Table

LeetCode - Simplest - [WuJieCC](#) - [LeetCode](#) - [Editorial](#)

Math - Binary Search

Little Denny Zhang

Problem:

Nearly everyone has used the Multiplication Table. The multiplication table of size $m \times n$ is an integer matrix mat where $mat[i][j] = i \times j$ (1-indexed).

Given three integers m , n , and k , return the k th smallest element in the $m \times n$ multiplication table.

Example 1:

1	2	3
2	4	6
3	6	9

1	2	2	3	3	4	6	6	9
---	---	---	---	---	---	---	---	---

Input: m = 3, n = 3, k = 5

Output: 3

Explanation: The 5th smallest number is 3.

Example 2:

1	2	3
2	4	6

1	2	2	3	4	6
---	---	---	---	---	---

Input: m = 2, n = 3, k = 6

Output: 6

Explanation: The 6th smallest number is 6.

Constraints:

- $1 \leq m, n \leq 3 \times 10^4$

```

• 1 <= k <= m * n

Python
Python
# Python solution using binary search

class Solution:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        def countLessEqual(x: int) -> int:
            # Count numbers in the m x n multiplication table <= x
            count = 0
            for i in range(1, m + 1):
                # For each row, count of numbers <= x is minimum of n or x // i
                count += min(n // i, n)
            return count

        # Binary search range from 1 to m * n
        low, high = 1, m * n
        while low < high:
            mid = (low + high) // 2
            if countLessEqual(mid) < k:
                low = mid + 1 # answer must be greater than mid
            else:
                high = mid
        return low

Python
class Solution:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        left, right = 1, m * n
        while left < right:
            mid = (left + right) >> 1
            cnt = 0
            for i in range(1, m + 1):
                cnt += min(mid // i, n)
            if cnt < k:
                left = mid
            else:
                right = mid + 1
        return left

Python
class Solution:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        left, right = 1, m * n
        while left < right:
            mid = (left + right) >> 1
            cnt = 0
            for i in range(1, m + 1):
                cnt += min(mid // i, n)
            if cnt < k:
                left = mid
            else:
                right = mid + 1
        return left

```

```

Java
class Solution {
    public int findKthNumber(int n, int k, int k) {
        int l = 1;
        int r = m * n;

        while (l < r) {
            long mid = (l + r) / 2;
            if (sumOfDigits(n, a, mid) >= k)
                r = mid;
            else
                l = mid + 1;
        }
        return l;
    }

    private int sumOfDigits(int n, int a, int target) {
        int count = 0;
        // For each num <= count the number of numbers >= target.
        for (int i = 1; i <= n; ++i)
            count += Math.min(target / i, a);
        return count;
    }
}

Java
class Solution {
    public int findKthNumber(int n, int k, int k) {
        int left = 1, right = m * n;
        while (left < right) {
            int mid = (left + right) / 2;
            int cnt = 0;
            for (int i = 1; i <= n; ++i) {
                cnt += Math.min(mid / i, a);
            }
            if (cnt == k) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}

```

```

class Solution {
    public int findKthNumber(int n, int k, int k) {
        int left = 1, right = m * n;
        while (left < right) {
            int mid = (left + right) / 2;
            int cnt = 0;
            for (int i = 1; i <= n; ++i) {
                cnt += Math.min(mid / i, a);
            }
            if (cnt == k) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}

```

#710 **HARD**
Random Pick with Blacklist
[LeetCode](#) - [SimplyLeet](#) - [WalkCC](#) - [LeetCode](#) - [Editorial](#)
[Array](#) - [Hash Table](#) - [Math](#) - [Binary Search](#) - [Sorting](#) - [Randomized](#)
[LeetCode](#) - [LeetCode](#) - [LeetCode](#)

Problem:
 You are given an integer n and an array of unique integers blacklist. Design an algorithm to pick a random integer in the range [0, n - 1] that is not in blacklist. Any integer that is in the mentioned range and not in blacklist should be equally likely to be returned.

Optimize your algorithm such that it minimizes the number of calls to the built-in random function of your language.

Implement the Solution class:

- `Solution(int n, int[] blacklist)` Initializes the object with the integer n and the blacklisted integers blacklist.
- `int pick()` Returns a random integer in the range [0, n - 1] and not in blacklist.

Example 1:

Input
 ["Solution", "pick", "pick", "pick", "pick", "pick", "pick", "pick"]
 [[7, [2, 3, 5]], [1], [1], [1], [1], [1], [1], [1]]

Output
 [null, 0, 4, 1, 6, 1, 0, 4]

Explanation
 Solution solution = new Solution(7, [2, 3, 5]);
 solution.pick(); // return 0, any integer from [0,1,4,6] should be ok. Note that for every call of pick,
 // 0, 1, 4, and 6 must be equally likely to be returned (i.e., with probability 1/4).
 solution.pick(); // return 4
 solution.pick(); // return 1
 solution.pick(); // return 6
 solution.pick(); // return 1
 solution.pick(); // return 0
 solution.pick(); // return 4

Constraints:

- 1 <= n <= 109
- 0 <= blacklist.length <= min(105, n - 1)
- 0 <= blacklist[i] < n
- All the values of blacklist are unique.
- At most 2 * 104 calls will be made to pick.

```

>> Brute Force [Binary Search] Interview Prep
Python
# Brute Force Approach
class Solution:
    def __init__(self, n: int, blacklist: List[int]):
        self.n = n
        self.blacklist = blacklist
        self.available = []
        for i in range(n):
            if i not in self.blacklist:
                self.available.append(i)

    def pick(self) -> int:
        return random.choice(self.available)

```

```

import random

class Solution:
    def __init__(self, n: int, blacklist: List[int]):
        self.n = n
        self.blacklist = blacklist
        self.available = []
        for i in range(n):
            if i not in self.blacklist:
                self.available.append(i)

    def pick(self) -> int:
        return random.choice(self.available)

Python
class Solution:
    def __init__(self, n: int, blacklist: List[int]):
        self.n = n
        self.blacklist = blacklist
        self.available = []
        for i in range(n):
            if i not in self.blacklist:
                self.available.append(i)

    def pick(self) -> int:
        return random.choice(self.available)

```

```

class Solution:
    def __init__(self, n: int, blacklist: List[int]):
        self.n = n
        self.blacklist = blacklist
        self.available = []
        for i in range(n):
            if i not in self.blacklist:
                self.available.append(i)

    def pick(self) -> int:
        return random.choice(self.available)

Python
class Solution:
    def __init__(self, n: int, blacklist: List[int]):
        self.n = n
        self.blacklist = blacklist
        self.available = []
        for i in range(n):
            if i not in self.blacklist:
                self.available.append(i)

    def pick(self) -> int:
        return random.choice(self.available)

C++
class Solution {
public:
    Solution(int n, vector<int> &blacklist) : n(n), blacklist(blacklist) {}

    int pick() {
        int i = 0;
        while (i < n) {
            if (i not in blacklist)
                return i;
            i++;
        }
    }
};

```

```

class Solution {
    public:
        Solution(int n, vector<int> &blacklist) : n(n), blacklist(blacklist) {}

        int pick() {
            int i = 0;
            while (i < n) {
                if (i not in blacklist)
                    return i;
                i++;
            }
        }
};

C++
class Solution {
public:
    Solution(int n, vector<int> &blacklist) : n(n), blacklist(blacklist) {}

    int pick() {
        int i = 0;
        while (i < n) {
            if (i not in blacklist)
                return i;
            i++;
        }
    }
};

```

```

+ Solution obj = new Solution(n, blacklist);
+ int param_1 = obj.pick();
+ }
+ }

class Solution {
public:
    unordered_map<int, int> d;
    int n;

    Solution(int n, vector<int>& blacklist) {
        k = n - blacklist.size();
        int i = k;
        unordered_set<int> black(blacklist.begin(), blacklist.end());
        for (int b : blacklist) {
            if (b < k) {
                while (black.count(i) > 0) i++;
                d[b] = i++;
            }
        }

        int pick() {
            int x = rand() % k;
            return d.count(x) ? d[x] : k;
        }
    };
};

/*
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(n, blacklist);
 * int param_1 = obj.pick();
 */

```

Java
Java

```

class Solution {
public:
    Solution(int n, set<int> blacklist) {
        validRange = n - blacklist.length;

        for (first, last : blacklist)
            map.put(b, -1);

        int maskAvailable = n - 1;

        for (first, last : blacklist)
            if (b < validRange) {
                // Find the first valid number's mask and
                while (map.containsKey(maskAvailable))
                    --maskAvailable;
                map.put(b, maskAvailable--);
            }
        }

        public int pick() {
            final int num = rand.nextInt(validRange);
            return map.getOrDefault(num, num);
        }

        private int validRange;
        private Map<Integer, Integer> map = new HashMap<>();

        private Random rand = new Random();
    }
}

```

```

class Solution {
    private Map<Integer, Integer> d = new HashMap<>();
    private Random rand = new Random();
    private int n;

    public Solution(int n, int[] blacklist) {
        k = n - blacklist.length;
        int i = k;
        Set<Integer> black = new HashSet<>();
        for (int b : blacklist) {
            black.add(b);
        }
        for (int b : blacklist) {
            if (b < k) {
                while (black.contains(i)) {
                    i++;
                }
                d.put(b, i++);
            }
        }

        public int pick() {
            int x = rand.nextInt(k);
            return d.getOrDefault(x, k);
        }
    }
}

```

```

    }
}

/*
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(n, blacklist);
 * int param_1 = obj.pick();
 */
+ }
+ }

class Solution {
private Map<Integer, Integer> d = new HashMap<>();
private Random rand = new Random();
private int n;

public Solution(int n, int[] blacklist) {
    k = n - blacklist.length;
    int i = k;
    Set<Integer> black = new HashSet<>();
    for (int b : blacklist) {
        black.add(b);
    }
    for (int b : blacklist) {
        if (b < k) {
            while (black.contains(i)) {
                i++;
            }
            d.put(b, i++);
        }
    }

    public int pick() {
        int x = rand.nextInt(k);
        return d.getOrDefault(x, k);
    }
}

/*
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(n, blacklist);
 * int param_1 = obj.pick();
 */
+ }
+ }

```

Go
Go

```

type Solution struct {
    d map[int]int
    k int
}

func Constructor(n int, blacklist []int) Solution {
    k := n - len(blacklist)
    i := k
    black := map[int]bool{}
    for _, b := range blacklist {
        black[b] = true
    }
    d := map[int]int{}
    for _, b := range blacklist {
        if b < k {
            for black[i] {
                i++
            }
            d[b] = i
            i++
        }
    }
    return Solution{d, k}
}

func (this *Solution) Pick() int {
    x := rand.Intn(this.k)
    if v, ok := this.d[x]; ok {
        return v
    }
    return x
}

/*
 * Your Solution object will be instantiated and called as such:
 * obj := Constructor(n, blacklist);
 * param_1 := obj.Pick();
 */
+ }
+ }

```

Go

```

type Solution struct {
    d map[int]int
    k int
}

func Constructor(n int, blacklist []int) Solution {
    k := n - len(blacklist)
    i := k
    black := map[int]bool{}
    for _, b := range blacklist {
        black[b] = true
    }
    d := map[int]int{}
    for _, b := range blacklist {
        if b < k {
            for black[i] {
                i++
            }
            d[b] = i
            i++
        }
    }
    return Solution{d, k}
}

func (this *Solution) Pick() int {
    x := rand.Intn(this.k)
    if v, ok := this.d[x]; ok {
        return v
    }
    return x
}

/*
 * Your Solution object will be instantiated and called as such:
 * obj := Constructor(n, blacklist);
 * param_1 := obj.Pick();
 */
+ }
+ }

```

[P] Binary Search — Pattern Solution (stored optimal)
Python

```

import random

class Solution:
    def __init__(self, n: int, blacklist: list):
        # First memory setup for picking
        self.W = n - len(blacklist)
        # Mapping for blacklisted numbers in the valid range
        self.mapping = {}
        # Set to hold blacklisted numbers that are == W
        blacklist_set = set(blacklist)

        # Number to find available numbers in [0, n-1]
        available = self.W
        for b in blacklist:
            if b < self.W:
                # Find next available non-blacklisted number in the tail region.
                while available in blacklist_set:
                    available += 1
                self.mapping[b] = available
                available += 1

    def pick(self) -> int:
        # Generate a random number in the range [0, self.W-1]
        idx = random.randint(0, self.W - 1)
        # If the number is mapped, return the mapping; else it is valid.
        return self.mapping.get(idx, idx)

```

```

// C++ Implementation
#include <iostream>
#include <unordered_set>
#include <unordered_map>
#include <vector>

using namespace std;

class Solution {
private:
    int W; // Total count of valid numbers
    unordered_map<int, int> mapping; // Mapping for blacklisted numbers in [0, W-1]
public:
    Solution(int n, vector<int>& blacklist) {
        W = n - blacklist.size();
        unordered_set<int> blacklistSet(blacklist.begin(), blacklist.end());

        int available = W;
        for (int b : blacklist) {
            if (b < W) {
                // Find the next available number that is not in the blacklist
                while (blacklistSet.count(available)) {
                    available++;
                }
            }
        }
    }
}

```

```

        mapping[id] = available;
        available--;
    }
}

int pick() {
    // Generate a random index in [0, n-1]
    int id = rand() % N;
    // Return the second value if exists, else return id
    if (mapping.count(id)) {
        return mapping[id];
    }
    return id;
}
};

```

#774 **HARD** Minimize Max Distance to Gas Station

LeetCode · Simplified · WA/ACC · LeetCode · Editorial

Array · Binary Search

List: Denny Zhang

Problem:

You are given an integer array `stations` that represents the positions of the gas stations on the `x`-axis. You are also given an integer `k`.

You should add `k` new gas stations. You can add the stations anywhere on the `x`-axis, and not necessarily on an integer position.

Let `penalty()` be the maximum distance between adjacent gas stations after adding the `k` new stations.

Return the smallest possible value of `penalty()`. Answers within 10^{-6} of the actual answer will be accepted.

Example 1:

Input: `stations = [1,2,3,4,5,6,7,8,9,10]`, `k = 9`
Output: `0.50000`

Example 2:

Input: `stations = [23,24,36,39,46,56,57,65,84,90]`, `k = 1`
Output: `14.00000`

Constraints:

- $10 \leq \text{stations.length} \leq 2000$
- $0 \leq \text{stations}[i] \leq 10^8$
- `stations` is sorted in a strictly increasing order.
- $1 \leq k \leq 10^6$

>> Brute Force [Binary Search] Interview Prep

Python

Binary Search · LeetCode · By Pattern

— 2467 —

LeetCode

```

# Brute Force Approach
class Solution:
    def minimizeDist(self, stations, k):
        # Brute Force O(N) - Linear time instead of binary search
        for i, val in enumerate(stations):
            if val == target:
                return i
        return -1

```

Python

Python

```

import math

def minimizeDist(stations, k):
    # Initialize the search range between 0 and the max gap between existing stations
    low = 0.0
    high = max(stations[i+1] - stations[i] for i in range(len(stations)-1))

    # Use binary search with a tolerance of 1e-6
    while high - low > 1e-6:
        mid = (low + high) / 2.0
        used = 0
        # Calculate additional stations needed for each gap
        for i in range(1, len(stations)):
            gap = stations[i] - stations[i-1]
            used += math.ceil(gap / mid) - 1
        # If the used stations are less than or equal to k, try a smaller distance
        if used <= k:
            high = mid
        else:
            low = mid
    return high

# Example usage:
if __name__ == '__main__':
    print("%.5f" % minimizeDist([1,2,3,4,5,6,7,8,9,10], 9))
    print("%.5f" % minimizeDist([23,24,36,39,46,56,57,65,84,90], 1))

```

Python

Binary Search · LeetCode · By Pattern

— 2468 —

LeetCode

```

class Solution:
    def minimizeDist(self, stations: List[int], k: int) -> float:
        low = low
        high = high
        r = stations[-1] - stations[0]

        def possible(left: float, m: float) -> bool:
            # Returns True if can use == k gas stations to ensure that each adjacent
            # distance between gas stations == m.
            for a, b in zip(stations, stations[1:]):
                diff = b - a
                if diff > m:
                    k -= math.ceil(diff / m) - 1
                    if k < 0:
                        return False
            return True

        while r - 1 > 1e-6:
            m = (low + r) / 2
            if possible(left, m):
                r = m
            else:
                low = m
        return low

```

Python

```

class Solution:
    def minimizeDist(self, stations: List[int], k: int) -> float:
        def check():
            return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k

        left, right = 0, low
        while right - left > 1e-6:
            mid = (left + right) / 2
            if check():
                right = mid
            else:
                left = mid
        return left

```

Python

Binary Search · LeetCode · By Pattern

— 2469 —

LeetCode

```

class Solution:
    def minimizeDist(self, stations: List[int], k: int) -> float:
        def check():
            return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k

        left, right = 0, low
        while right - left > 1e-6:
            mid = (left + right) / 2
            if check():
                right = mid
            else:
                left = mid
        return left

```

C++

C++

```

class Solution {
public:
    double minimizeDist(vector<int>& stations, int k) {
        const int n = stations.size();
        double l = 0;
        double r = stations[n-1] - stations[0];

        while (r - l > 1e-6) {
            const double m = (l + r) / 2;
            if (check(stations, k, m))
                r = m;
            else
                l = m;
        }
        return l;
    }

private:
    // Returns true if can use == k gas stations to ensure that each adjacent
    // distance between gas stations == m.
    bool check(const vector<int>& stations, int k, double m) {
        for (int i = 1; i < stations.size(); ++i) {
            const int diff = stations[i] - stations[i-1];
            if (diff > m) {
                k -= ceil(diff / m) - 1;
                if (k < 0)
                    return false;
            }
        }
        return true;
    }
};

```

C++

Binary Search · LeetCode · By Pattern

— 2470 —

LeetCode

```

class Solution {
public:
    double minimizeDist(vector<int>& stations, int k) {
        double left = 0, right = low;
        auto check = [&](double x) {
            int s = 0;
            for (int i = 0; i < stations.size(); ++i) {
                s += int((stations[i+1] - stations[i]) / x);
            }
            return s <= k;
        };
        while (right - left > 1e-6) {
            double mid = (left + right) / 2.0;
            if (check(mid))
                right = mid;
            else
                left = mid;
        }
        return left;
    }
};

```

C++

```

class Solution {
public:
    double minimizeDist(vector<int>& stations, int k) {
        double left = 0, right = low;
        auto check = [&](double x) {
            int s = 0;
            for (int i = 0; i < stations.size(); ++i) {
                s += int((stations[i+1] - stations[i]) / x);
            }
            return s <= k;
        };
        while (right - left > 1e-6) {
            double mid = (left + right) / 2.0;
            if (check(mid))
                right = mid;
            else
                left = mid;
        }
        return left;
    }
};

```

Java

Java

Binary Search · LeetCode · By Pattern

— 2471 —

LeetCode

```

class Solution {
public:
    double minimizeDist(vector<int> stations, int k) {
        const int n = stations.size();
        double l = 0;
        double r = stations[n-1] - stations[0];

        while (r - l > 1e-6) {
            const double m = (l + r) / 2;
            if (check(stations, k, m))
                r = m;
            else
                l = m;
        }
        return l;
    }

private:
    // Returns true if can use == k gas stations to ensure that each adjacent
    // distance between gas stations == m.
    bool check(vector<int> stations, int k, double m) {
        for (int i = 1; i < stations.size(); ++i) {
            const int diff = stations[i] - stations[i-1];
            if (diff > m) {
                k -= ceil(diff / m) - 1;
                if (k < 0)
                    return false;
            }
        }
        return true;
    }
};

```

Java

```

class Solution {
public:
    double minimizeDist(vector<int> stations, int k) {
        double left = 0, right = low;
        while (right - left > 1e-6) {
            double mid = (left + right) / 2.0;
            if (check(mid, stations, k))
                right = mid;
            else
                left = mid;
        }
        return left;
    }

private:
    bool check(double x, vector<int> stations, int k) {
        int s = 0;
        for (int i = 0; i < stations.size(); ++i) {
            s += int((stations[i+1] - stations[i]) / x);
        }
        return s <= k;
    }
};

```

Java

Binary Search · LeetCode · By Pattern

— 2472 —

LeetCode


```

class Solution {
public: double minimizeDist(int stations, int k) {
    double left = 0, right = last;
    while (right - left > 1e-6) {
        double mid = (left + right) / 2.0;
        if (checkmid, stations, k) {
            right = mid;
        } else {
            left = mid;
        }
    }
    return left;
}

private: bool check(double x, int stations, int k) {
    int s = 0;
    for (int i = 0; i < stations.length - 1; ++i) {
        k -= (int)(x / (stations[i + 1] - stations[i]));
    }
    return k >= 0;
}
}

```

Go

```

Go
func minimizeDist(stations []int, k int) float64 {
    check := func(x float64) bool {
        s := 0
        for i, v := range stations[1:len(stations)-1] {
            s += int(float64(stations[i+1]-v) / x)
        }
        return s <= k
    }
    var left, right float64 = 0, last
    for right-left > 1e-6 {
        mid := (left + right) / 2.0
        if check(mid) {
            right = mid
        } else {
            left = mid
        }
    }
    return left
}

```

Go

```

func minimizeDist(stations []int, k int) float64 {
    check := func(x float64) bool {
        s := 0
        for i, v := range stations[1:len(stations)-1] {
            s += int(float64(stations[i+1]-v) / x)
        }
        return s <= k
    }
    var left, right float64 = 0, last
    for right-left > 1e-6 {
        mid := (left + right) / 2.0
        if check(mid) {
            right = mid
        } else {
            left = mid
        }
    }
    return left
}

```

[1] Binary Search - Pattern Solution (stored optimal)

Python

```

import math

def minimizeDist(stations, k):
    # Initialize the search range between 0 and the maximum gap between existing stations
    low = 0.0
    high = max(stations[i+1] - stations[i] for i in range(len(stations)-1))

    # Use binary search with a tolerance of 1e-6
    while high - low > 1e-6:
        mid = (low + high) / 2.0
        used = 0
        # Calculate additional stations needed for each gap
        for i in range(1, len(stations)):
            gap = stations[i] - stations[i-1]
            used += math.ceil(gap / mid) - 1
            # If the used stations are less than or equal to k, try a smaller distance
            if used >= k:
                high = mid
            else:
                low = mid
        return high

# Example usage:
if __name__ == '__main__':
    print("%.5f" % minimizeDist([1,2,3,4,5,6,7,8,9,10], 9))
    print("%.5f" % minimizeDist([23,24,36,39,46,56,57,65,84,88], 1))

```

C++

```

// C++ solution with comments
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

class Solution {
public:
    double minimizeDist(vector<int>& stations, int k) {
        double low = 0.0;
        double high = 0.0;
        // Determine the maximum gap between consecutive stations
        for (int i = 1; i < stations.size(); ++i) {
            high = max(high, double(stations[i] - stations[i-1]));
        }

        // Binary search with precision tolerance of 1e-6
        while (high - low > 1e-6) {
            double mid = (low + high) / 2.0;
            long used = 0;
            // Count required additional stations for each gap
            for (int i = 1; i < stations.size(); ++i) {
                double gap = stations[i] - stations[i-1];
                used += ceil(gap / mid) - 1;
            }
            if (used >= k)
                low = mid;
            else
                high = mid;
        }
        return high;
    }
};

int main() {
    Solution sol;
    vector<int> stations1 = {1,2,3,4,5,6,7,8,9,10};
    vector<int> stations2 = {23,24,36,39,46,56,57,65,84,88};
    cout << fixed << setprecision(5);
    cout << sol.minimizeDist(stations1, 9) << endl;
    cout << sol.minimizeDist(stations2, 1) << endl;
    return 0;
}

```

#1235

HARD

Maximum Profit in Job Scheduling

LeetCode - SimplifyLeetCode - WalkCC - LeetCode - Editorial

Array - Binary Search - Dynamic Programming - Sorting

Lists Grid 169

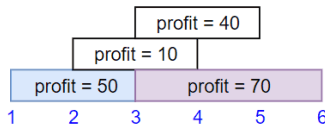
Problem:

We have n jobs, where every job is scheduled to be done from $startTime[i]$ to $endTime[i]$, obtaining a profit of $profit[i]$.

You're given the $startTime$, $endTime$ and $profit$ arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time X you will be able to start another job that starts at time X .

Example 1:



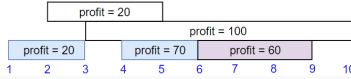
Input: $startTime = [1,2,3,3]$, $endTime = [3,4,5,6]$, $profit = [50,10,40,70]$

Output: 120

Explanation: The subset chosen is the first and fourth job.

Time range $[1-3] \cup [4-6]$. we get profit of $120 = 50 + 70$.

Example 2:



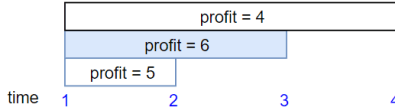
Input: $startTime = [1,2,3,4,6]$, $endTime = [3,5,10,6,9]$, $profit = [20,20,100,70,60]$

Output: 150

Explanation: The subset chosen is the first, fourth and fifth job.

Profit obtained $150 = 20 + 70 + 60$.

Example 3:



Input: $startTime = [1,1,1]$, $endTime = [2,3,4]$, $profit = [5,6,4]$

Output: 6

Constraints:

$1 \leq startTime.length == endTime.length == profit.length \leq 5 \times 10^4$

$1 \leq startTime[i] < endTime[i] \leq 109$

$1 \leq profit[i] \leq 104$

>> Brute Force [Binary Search] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def jobScheduling(self, startTime, endTime, profit):
        # Sort jobs by end time instead of binary search
        for i, val in enumerate(startTime):
            if val == target:
                return i
        return -1

```

Python

```

import bisect

def jobScheduling(startTime, endTime, profit):
    # Create a list of jobs and sort them by end time
    jobs = sorted(zip(startTime, endTime, profit), key=lambda x: x[1])

    # dp and times stores the end times and dp.profit stores the corresponding maximum profit.
    dp_end_times = []
    dp_profits = []

    # Process each job in order.
    for s, e, p in jobs:
        # Find the index of the last job that finishes before or at the start of the current job using
        # binary search.
        index = bisect.bisect_right(dp_end_times, s)
        current_profit = dp_profits[index - 1] + p # Profit if we take current job.

        # If taking current job increases the overall profit, update the dp arrays.
        if current_profit > dp_profits[-1]:
            dp_end_times.append(e)
            dp_profits.append(current_profit)

    return dp_profits[-1]

# Example usage:
print(jobScheduling([1,2,3,3], [3,4,5,6], [50,10,40,70]))

```

Python

```

class Solution:
    def jobScheduling(
        self,
        startTime: List[int],
        endTime: List[int],
        profit: List[int],
    ) -> int:
        jobs = sorted([(s, e, p) for s, e, p in zip(startTime, endTime, profit)])

        # Will use binary search to find the first available startTime
        for i in range(len(startTime)):
            startTime[i] = jobs[i][0]

        @functools.lru_cache(None)
        def dfs(i: int) -> int:
            """Returns the maximum profit to schedule jobs[i..n]."""
            if i == len(startTime):
                return 0
            j = bisect.bisect_left(startTime, jobs[i][0])
            return max(dfs(i+1) + jobs[i][2], dfs(j))

        return dfs(0)

```

Python

```

class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i):
            if i == n:
                return 0
            j = jobs[i]
            j = bisect_left(jobs, s, lo=i + 1, key=lambda x: x[0])
            return max(dfs(i + 1), p + dfs(j))

        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        return dfs(0)

```

Python

```

class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i):
            if i == n:
                return 0
            j = jobs[i]
            j = bisect_left(jobs, s, lo=i + 1, key=lambda x: x[0])
            return max(dfs(i + 1), p + dfs(j))

        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        return dfs(0)

```

```

C++
class Solution {
public:
    int jobScheduling(vector<int>& startTime, vector<int>& endTime, vector<int>& profit) {
        int n = profit.size();
        vector<tuple<int, int, int>> jobs(n);
        for (int i = 0; i < n; ++i) jobs[i] = {startTime[i], endTime[i], profit[i]};
        sort(jobs.begin(), jobs.end());
        vector<int> dp(n + 1);
        for (int i = 0; i < n; ++i) {
            auto [s, e, p] = jobs[i];
            int j = upper_bound(jobs.begin(), jobs.begin() + i, s, [](int x, auto& job) -> bool { return x < get<0>(job); }) - jobs.begin();
            dp[i + 1] = max(dp[i], dp[j] + p);
        }
        return dp[n];
    }
};

C++
class Solution {
public:
    int jobScheduling(vector<int>& startTime, vector<int>& endTime, vector<int>& profit) {
        int n = profit.size();
        vector<tuple<int, int, int>> jobs(n);
        for (int i = 0; i < n; ++i) jobs[i] = {startTime[i], endTime[i], profit[i]};
        sort(jobs.begin(), jobs.end());
        vector<int> f(n);
        function<int(int)> dfs = [&](int i) -> int {
            if (i == n) return 0;
            if (f[i]) return f[i];
            auto [s, e, p] = jobs[i];
            tuple<int, int, int> t(e, 0, 0);
            int j = lower_bound(jobs.begin() + i + 1, jobs.end(), t, [](auto& l, auto& r) -> bool { return get<0>(l) < get<0>(r); }) - jobs.begin();
            int ans = max(dfs(i + 1), p + dfs(j));
            f[i] = ans;
            return ans;
        };
        return dfs(0);
    }
};

C++

```

Binary Search - LeetCode - By Pattern

— 2479 —

LeetCode

```

class Solution {
    func binarySearch(f: Comparable<InputKey: T>, searchItem: T) -> Int? {
        var lowerIndex = 0
        var upperIndex = inputKey.count - 1
        while lowerIndex < upperIndex {
            let currentIndex = (lowerIndex + upperIndex) / 2
            if inputKey[currentIndex] == searchItem {
                lowerIndex = currentIndex + 1
            } else {
                upperIndex = currentIndex
            }
        }
        if inputKey[upperIndex] == searchItem {
            return upperIndex + 1
        }
        return lowerIndex
    }

    func jobScheduling(_ startTime: [Int], _ endTime: [Int], _ profit: [Int]) -> Int {
        let zipList = zip(startTime, endTime, profit)
        var table: [(startTime: Int, endTime: Int, profit: Int, cumsum: Int)] = []

        for (x, y, z) in zipList {
            table.append((x, y, z, 0))
        }
        table.sort(by: { $0.endTime < $1.endTime })
        let sortedEndTime = endTime.sorted()

        var profits: [Int] = []
        for job in table {
            let index: Int? = binarySearch(inputKey: sortedEndTime, searchItem: job.startTime)
            if profits.last == profits[index] + job.profit {
                profits.append(profits[index] + job.profit)
            } else {
                profits.append(profits.last!)
            }
        }
        return profits.last!
    }
}

Java
Java

```

Binary Search - LeetCode - By Pattern

— 2480 —

LeetCode

```

class Solution {
    private List<> jobs;
    private List<> f;
    private List<> n;

    public int jobScheduling(List<> startTime, List<> endTime, List<> profit) {
        n = profit.length;
        jobs = new ArrayList<>();
        for (int i = 0; i < n; ++i) {
            jobs[i] = new List<>() {startTime[i], endTime[i], profit[i]};
        }
        Arrays.sort(jobs, (a, b) -> a[0] - b[0]);
        f = new List<>();
        return dfs(0);
    }

    private int dfs(int i) {
        if (i == n) {
            return 0;
        }
        if (f[i] != 0) {
            return f[i];
        }
        int s = jobs[i][0], p = jobs[i][2];
        int j = search(jobs, s, i + 1);
        int ans = Math.max(dfs(i + 1), p + dfs(j));
        f[i] = ans;
        return ans;
    }

    private int search(List<> jobs, int s, int i) {
        int left = i, right = n;
        while (left < right) {
            int mid = (left + right) >> 1;
            if (jobs[mid][0] == s) {
                right = mid;
            } else {
                left = mid + 1;
            }
        }
        return left;
    }
}

TypeScript
TypeScript

```

Binary Search - LeetCode - By Pattern

— 2481 —

LeetCode

```

function jobScheduling(startTime: number[], endTime: number[], profit: number[]): number {
    const n = startTime.length;
    const f = new Array<number>(n);
    const jobs = new Array<number>().fill(0).map((_, i) => i);
    jobs.sort((i, j) => startTime[i] - startTime[j]);
    const search = (s: number) => {
        let l = 0;
        let r = n;
        while (l < r) {
            const mid = (l + r) >> 1;
            if (startTime[mid] == s) {
                r = mid;
            } else {
                l = mid + 1;
            }
        }
        return l;
    };
    const dfs = (i: number): number => {
        if (i == n) {
            return 0;
        }
        if (f[i] != 0) {
            return f[i];
        }
        const j = search(endTime[i]);
        return f[i] = Math.max(dfs(i + 1), dfs(j) + profit[i]);
    };
    return dfs(0);
}

Go
Go

```

Binary Search - LeetCode - By Pattern

— 2482 —

LeetCode

```

func jobScheduling(startTime []int, endTime []int, profit []int) int {
    n := len(profit)
    type tuple struct { s, e, p int }
    jobs := make([]tuple, n)
    for i, p := range profit {
        jobs[i] = tuple{startTime[i], endTime[i], p}
    }
    sort.Slice(jobs, func(i, j int) bool { return jobs[i].s < jobs[j].s })
    f := make([]int, n)
    var dfs func(int) int
    dfs = func(i int) int {
        if i == n {
            return 0
        }
        if f[i] != 0 {
            return f[i]
        }
        j := sort.SearchN, func(i int) bool { return jobs[j].s < jobs[i].e })
        ans := max(dfs(i+1), jobs[i].profit+f(j))
        f[i] = ans
        return ans
    }
    return dfs(0)
}

Go
func jobScheduling(startTime []int, endTime []int, profit []int) int {
    n := len(profit)
    type tuple struct { s, e, p int }
    jobs := make([]tuple, n)
    for i, p := range profit {
        jobs[i] = tuple{startTime[i], endTime[i], p}
    }
    sort.Slice(jobs, func(i, j int) bool { return jobs[i].s < jobs[j].s })
    f := make([]int, n)
    var dfs func(int) int
    dfs = func(i int) int {
        if i == n {
            return 0
        }
        if f[i] != 0 {
            return f[i]
        }
        j := sort.SearchN, func(i int) bool { return jobs[j].s < jobs[i].e })
        ans := max(dfs(i+1), jobs[i].profit+f(j))
        f[i] = ans
        return ans
    }
    return dfs(0)
}

Python

```

[*] Binary Search — Pattern Solution (matched from community answers)

Python

Binary Search - LeetCode - By Pattern

— 2483 —

LeetCode

```

import bisect

def jobScheduling(startTime, endTime, profit):
    # Combine the jobs and sort them by end time.
    jobs = sorted(zip(startTime, endTime, profit), key=lambda x: x[1])

    # dp_and_times stores the end times and dp_profit stores the corresponding maximum profit.
    dp_and_times = []
    dp_profit = [0]

    # Process each job in order.
    for s, e, p in jobs:
        # Find the index of the last job that finishes before or at the start of the current job using
        # binary search.
        index = bisect.bisect_right(dp_and_times, s)
        current_profit = dp_profit[index - 1] + p # Profit if we take current job.

        # If taking current job increases the overall profit, update the dp arrays.
        if current_profit > dp_profit[-1]:
            dp_and_times.append(e)
            dp_profit.append(current_profit)

    return dp_profit[-1]

# Example usage:
print(jobScheduling([1,2,3,3], [3,4,5,6], [50,10,40,70]))

```

Binary Search - LeetCode - By Pattern

— 2484 —

LeetCode

Quick Review — Binary Search

24 questions · insights ·

#35 Search Insert Position [Easy]

Key Insights

- The array is sorted, enabling the use of binary search.
- Binary search achieves a time complexity of $O(\log n)$ which meets the problem constraints.
- When the target is not found during the search, the final low pointer represents the correct insertion index.

Space & Time

Time Complexity: $O(\log n)$ due to binary search.
Space Complexity: $O(1)$ as no extra space is used.

Solution

We use binary search to find the target in the sorted array. Initialize two pointers, low and high. At each iteration, compute the mid index. If the value at mid matches the target, return mid. If the target is less than the mid value, adjust the high pointer. If greater, adjust the low pointer. When the iterations finish without finding the target, return the low pointer, which represents the correct insertion index.

#69 Sqrt(x) [Easy]

Key Insights

- The problem can be solved efficiently using binary search.
- Instead of iterating from 0 to x , binary search narrows down the candidate square roots quickly.
- The algorithm should handle edge cases like $x = 0$ and $x = 1$.
- By checking mid $\neq$ mid against x , we can determine if we need to search in the lower or upper half.

Space & Time

Time Complexity: $O(\log x)$
Space Complexity: $O(1)$

Solution

We use a binary search approach to find the integer square root. Initialize two pointers, low and high, where low is set to 0 and high is set to x . For each iteration, calculate the mid value. If mid squared equals x , we have found the square root. If mid squared is less than x , record mid as a potential answer and move the low pointer to mid + 1. Otherwise, move the high pointer to mid - 1. Continue the search until low exceeds high, then return the last recorded candidate.

#278 First Bad Version [Easy]

Key Insights

- The API returns true for bad versions and all versions after a bad version.
- The problem is essentially about finding a transition point in a sorted sequence which calls for a binary search approach.
- The aim is to minimize the number of calls to isBadVersion by leveraging the ordered nature of the versions.

Space & Time

Time Complexity: $O(\log n)$
Space Complexity: $O(1)$

Solution

We use binary search to efficiently locate the first bad version. Initialize two pointers, left and right, at 1 and n respectively. At each step, compute the mid point and call isBadVersion(mid). If mid is bad, it indicates that the first bad version may be mid or to its left, so adjust right to mid. Otherwise, adjust left to mid + 1. Continue the process

Binary Search · LeetCode · By Pattern

— 2485 —

LeetCode

until the pointers converge. The converged pointer will be the first bad version. Key structures include simple integer variables for left, right, and mid, with binary search being the fundamental algorithm to reduce the search space logarithmically.

#374 Guess Number Higher or Lower [Easy]

Key Insights

- The problem can be effectively solved using binary search since the search space is sorted by nature (1 to n).
- Each API call helps to halve the search space, leading to an efficient solution.
- Be cautious with potential overflow when calculating the mid-point by using low + (high - low) // 2.

Space & Time

Time Complexity: $O(\log n)$ since we binary search the range.
Space Complexity: $O(1)$ as we use a constant amount of extra space.

Solution

We use a binary search algorithm to efficiently locate the target number. We initialize two pointers: low (1) and high (n). In each iteration, we compute the mid-point, then call the guess API. Based on the response:
– If guess(mid) returns 0, we have found the number.
– If it returns -1, it indicates that our guess is higher than the pick, so we update the high pointer to mid - 1.
– If it returns 1, it indicates that our guess is lower than the pick, so we update the low pointer to mid + 1. This approach guarantees a logarithmic time solution.

#704 Binary Search [Easy]

Key Insights

- Utilize binary search to achieve $O(\log n)$ runtime on a sorted array.
- Use two pointers to maintain the current search region.
- Narrow down the search range by comparing the middle element with the target.
- Return the index when the target is found; otherwise, return -1 if not found.

Space & Time

Time Complexity: $O(\log n)$
Space Complexity: $O(1)$

Solution

We use an iterative binary search algorithm. We initialize two pointers, left and right, to represent the current segment of the array that could contain the target. At each iteration, we compute the middle index and compare the element there with the target. If they match, we return the index. Otherwise, based on whether the target is smaller or larger than the middle element, we adjust our pointers to narrow the search to either the left or right half of the array. This process continues until the target is found or until the pointers cross, indicating that the target is not present. No extra data structures are used, making the space complexity $O(1)$.

#35 Search in Rotated Sorted Array [Medium]

Key Insights

- The array is originally sorted and then rotated at an unknown pivot.
- Use a modified binary search to accommodate the rotation.
- At every step, determine which half of the array is properly sorted.
- Narrow down the search based on the sorted half and the target's value.

Space & Time

Time Complexity: $O(\log n)$
Space Complexity: $O(1)$

Solution

We perform a binary search while accounting for the possibility that the array is rotated. At each iteration, we check if the left-to-mid portion is sorted. If it is, and the target falls within that range, we search the left half; otherwise,

Binary Search · LeetCode · By Pattern

— 2486 —

LeetCode

we search the right half. If the left half is not sorted, then the right half must be sorted, and similarly, we check if the target is in that sorted half. This approach ensures logarithmic time complexity while handling the rotation correctly.

#34 Find First and Last Position of Element in Sorted Array [Medium]

Key Insights

- The array is sorted, allowing the use of binary search.
- Two binary searches can efficiently find the leftmost (first) and rightmost (last) indices of the target.
- Handling edge cases where the target is not present by checking bounds.

Space & Time

Time Complexity: $O(\log n)$
Space Complexity: $O(1)$

Solution

The approach is to use binary search twice:

– The first binary search finds the leftmost occurrence of the target by continuing to search in the left half even after finding the target.
– The second binary search finds the rightmost occurrence by searching in the right half. The algorithm maintains pointers for the current search bounds and narrows the search space using comparisons. If the target is not found during the first binary search, the solution immediately returns [-1, -1]. This method leverages the sorted property of the array, ensuring $O(\log n)$ time complexity with constant extra space.

#153 Find Minimum in Rotated Sorted Array [Medium]

Key Insights

- The rotated sorted array consists of two sorted subarrays.
- A modified binary search can be used to locate the pivot point where the rotation occurs.
- By comparing the middle element to the rightmost element, we can decide which half of the array contains the minimum.

Space & Time

Time Complexity: $O(\log n)$
Space Complexity: $O(1)$

Solution

We use a binary search approach. Initialize two pointers, left and right, at the start and end of the array. While left is less than right, calculate the middle index. If the middle element is greater than the right element, then the minimum is in the right half, so move the left pointer to mid + 1; otherwise, the minimum must be in the left half including mid, so adjust the right pointer to mid. When the loop terminates, left equals right and points to the minimum element. This approach leverages the properties of the rotated sorted array to achieve logarithmic time complexity.

#300 Longest Increasing Subsequence [Medium]

Key Insights

- A dynamic programming solution with $O(n^2)$ time complexity exists, but we focus on an optimized approach.
- Using binary search and a dp array, we can achieve $O(n \log n)$ time.
- The dp array maintains the smallest possible tail for increasing subsequences of various lengths.
- For each number, perform binary search to decide if it can extend or replace a value in dp.

Space & Time

Time Complexity: $O(n \log n)$ where n is the length of nums
Space Complexity: $O(n)$ for storing the dp array

Solution

We use a dynamic programming approach combined with binary search. The idea is to maintain a dp array where dp[i] holds the smallest tail value for any increasing subsequence with length i+1. For each number in the array, we perform a binary search on dp to find its correct insertion position:

Binary Search · LeetCode · By Pattern

— 2487 —

LeetCode

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We use a binary search strategy:

- Initialize two pointers, left and right, at the beginning and end of the array. – In each iteration, calculate mid. If arr[mid] is less than arr[mid + 1], then the peak is in the right half; otherwise, it is in the left half. – When left equals right, we have found the peak index. This approach leverages the mountain array properties and ensures an $O(\log n)$ solution.

#581 Time Based Key-Value Store [Medium]

Key Insights

- Use a dictionary (hash table) to map each key to a list of (timestamp, value) pairs.
- Timestamps are strictly increasing when setting the values, which allows efficient appending.
- For getting the value, perform a binary search on the list of timestamps to find the latest timestamp that is <= the queried timestamp.
- If no valid timestamp exists, return an empty string.

Space & Time

Time Complexity: – set operation: $O(1)$ per call (amortized).
– get operation: $O(\log N)$ per call, where N is the number of timestamps stored for the key.
Space Complexity: $O(\text{total number of set calls})$, to store all (timestamp, value) pairs.

Solution

We use a hash table to store keys mapping to a list of their timestamps and corresponding values. Since the timestamps are inserted in strictly increasing order, the list remains sorted. For the get operation, a binary search is applied to quickly locate the appropriate value for a given timestamp. This approach minimizes the lookup time and efficiently handles the problem constraints.

#1011 Capacity to Ship Packages Within D Days [Medium]

Key Insights

- The minimal possible capacity is the maximum value in weights; no ship can carry a package heavier than its capacity.
- The maximum possible capacity is the sum of all weights; this allows shipping all packages in one day.
- The problem is monotonic: if a capacity C works, any capacity greater than C will also work.
- Use binary search on the capacity range and a greedy approach to simulate shipping for each candidate capacity.

Space & Time

Time Complexity: $O(n * \log(\text{sum(weights)}))$ where n is the number of packages.
Space Complexity: $O(1)$ additional space.

Solution

The solution employs binary search to find the minimum ship capacity. Start by setting the search bounds: the lower bound is the maximum weight (since each package must be shipped individually if needed) and the upper bound is the sum of all weights (shipping everything in one go). For each midpoint capacity in the binary search, use a greedy method to simulate the shipping process—accumulating weights until adding another package would exceed the candidate ship capacity, at which point you increment the day count. If the number of days required exceeds D , the capacity is insufficient and the lower bound is increased; otherwise, adjust the upper bound. Continue until the bounds converge.

#1060 Missing Element in Sorted Array [Medium]

Key Insights

- The array is sorted and every element is unique.

Binary Search · LeetCode · By Pattern

— 2489 —

LeetCode

– if the number is larger than all elements, we append it, effectively extending the current longest subsequence. – Otherwise, we replace the first element in dp that is greater than or equal to the current number to keep the tail as small as possible. This process ensures that dp remains sorted and its length gives the length of the longest increasing subsequence.

#362 Design Hit Counter [Medium]

Key Insights

- Use a sliding window of 300 seconds to count hits.
- Since timestamps are monotonically increasing, outdated hits can be removed from the front.
- A queue (or circular array) is an efficient data structure to maintain and remove expired hit records.
- Grouping consecutive hits at the same timestamp can optimize space.

Space & Time

Time Complexity: $O(1)$ per hit and getHits operation on average, since each hit is enqueued and later dequeued only once.
Space Complexity: $O(W)$ where W is the window size (300 seconds) in the worst case.

Solution

The solution uses a queue (or deque) data structure where each element stores a (timestamp, count) pair. When a hit is recorded, if the last element of the queue shares the same timestamp, then its count is incremented; otherwise, a new element is added. When getHits(timestamp) is called, the algorithm removes elements from the front of the queue whose timestamp is out of the 300-second window relative to the current timestamp, and then sums the counts of the remaining elements to return the result. This ensures that only relevant hits within the past 5 minutes are counted. The design scales well when hit frequency is high because hits at the same timestamp are grouped together instead of inserting individual records.

#528 Random Pick with Weight [Medium]

Key Insights

- Build a prefix sum array from the weights.
- Calculate the total sum of the weights.
- Generate a random number in the range [1, total sum] and use binary search to find the corresponding index in the prefix sum array.
- Using binary search ensures efficient selection in $O(\log n)$ time for each call of pickIndex().

Space & Time

Time Complexity: $O(n)$ for constructing the prefix sum array, and $O(\log n)$ per pickIndex() call.
Space Complexity: $O(n)$ for storing the prefix sum array.

Solution

The solution leverages a prefix sum array where each entry at index i represents the cumulative sum of weights up to i . After computing the total weight, a random integer is generated between 1 and the total sum (inclusive). A binary search is then used to find the first index where the prefix sum is greater than or equal to the random number. This approach ensures that the index is selected with a probability proportional to its weight.

#852 Peak Index in a Mountain Array [Medium]

Key Insights

- The array is strictly increasing up to a peak and then strictly decreasing.
- A binary search technique is applicable because the array exhibits a single transition point (peak) between increasing and decreasing order.
- By comparing mid and mid+1, we can decide which half of the array to continue the search in.

Space & Time

Time Complexity: $O(\log n)$ – The binary search approach reduces the search interval by half at each step.

Binary Search · LeetCode · By Pattern

— 2488 —

LeetCode

- For any index i , the number of missing numbers up to that point can be computed as $\text{nums}[i] - \text{nums}[0] - i$.
- When $\text{miss}(i)$ is less than k , the k th missing element is further to the right.
- Use binary search to find the smallest index where the count of missing numbers is greater than or equal to k .
- If all missing numbers before the last array element are counted and the k th missing still hasn't been reached, calculate the result by continuing past the last element.

Space & Time

Time Complexity: $O(\log n)$
Space Complexity: $O(1)$

Solution

We compute the number of missing elements until an index i with the formula: $\text{miss}(i) = \text{nums}[i] - \text{nums}[0] - i$. We then use binary search over the indices to find the smallest index where $\text{miss}(i) \geq k$. If the binary search index equals the length of the array, it means the k th missing number lies beyond the last element. Otherwise, the k th missing number is derived as: $\text{nums}[\text{index}] - 1 + k - \text{miss}(\text{index} - 1)$. This approach uses binary search for logarithmic time and does not require extra auxiliary data structures.

#1062 Longest Repeating Substring [Medium]

Key Insights

- Use binary search to efficiently check for the existence of a repeating substring of a given length.
- Apply a rolling hash technique to compute hash values for substrings in $O(1)$ time after initial preprocessing.
- Use a hash set to detect duplicate hash values (indicating a repeating substring).
- Adjust binary search bounds based on whether a repeating substring of the current length exists.
- Alternative approaches include dynamic programming and suffix arrays, but the binary search with rolling hash is efficient and conceptually clean.

Space & Time

Time Complexity: $O(n \log n)$ on average, where n is the length of the string. (Each binary search step takes $O(n)$ time for computing rolling hashes.)
Space Complexity: $O(n)$ for storing hash values in the hash set.

Solution

The solution uses binary search to decide on the candidate length L of the repeating substring. For each candidate length L , we use a rolling hash to compute hash values for all substrings of length L and store them in a hash set. If any hash is repeated, it confirms the presence of a repeating substring of length L , and we try a longer length. Otherwise, we decrease the candidate length. The rolling hash is computed using a base (26, for lowercase letters) and a modulus (2^{32} to limit integer overflow) which allows efficient hash recomputation for sliding windows. This approach efficiently narrows down the maximum repeating substring length.

#1533 Find the Index of the Large Integer [Medium]

Key Insights

- Use binary search to efficiently narrow down the position of the unique large value.
- At each step, divide the candidate range into two halves (or almost two halves when the range size is odd) and use compareSub to determine which half contains the large integer.
- Comparing two subarrays of equal length will reveal the region that contains the extra value via the sum difference.
- Handle odd-length ranges carefully by isolating the middle element if needed.

Space & Time

Time Complexity: $O(\log n)$ – because binary search is performed over the range.
Space Complexity: $O(1)$ – only a few pointers are maintained, no additional space is required.

Solution

The solution applies a binary search approach over the array indices. At every iteration:

Binary Search · LeetCode · By Pattern

— 2490 —

LeetCode

• Calculate the current segment length. • If the length is even, split the segment evenly and compare the left half versus the right half using `compareSub`. • If the length is odd, compare two halves of equal size while ignoring the middle element. If the sums are equal, then the middle index holds the large value. • Update the search bounds based on the result of `compareSub`. • Continue until the search space is reduced to one index, which is the answer. This method ensures that you locate the large integer with $O(\log n)$ calls to `compareSub`, well within the limit of 20 calls.

#4 Median of Two Sorted Arrays [Hard]

Key Insights

- The median divides the sorted array into two equal halves.
- We can perform binary search on the smaller array to partition both arrays correctly.
- We need to ensure that every element in the left partition is less than or equal to every element in the right partition.
- Edge cases must be handled when partitions are at the boundaries of either array.

Space & Time

Time Complexity: $O(\log(\min(m, n)))$
Space Complexity: $O(1)$

Solution

The solution uses a binary search technique on the smaller array. We define a partition index i for `nums1` and compute the corresponding index j for `nums2` such that $i + j$ equals half of the total number of elements. The algorithm checks if the largest element on the left side of `nums1` (if any) is less than or equal to the smallest element on the right side of `nums1`, and vice versa for `nums2`. If the condition holds, we have found the correct partition and compute the median. If not, adjust the binary search range until the correct partition is found. This partitioning strategy allows us to find the median in logarithmic time.

#315 Count of Smaller Numbers After Self [Hard]

Key Insights

- The brute force solution takes $O(n^2)$ time, which is too slow for large input arrays.
- An efficient approach leverages the concept of Merge Sort to both sort the array and count the number of smaller elements by tracking the number of elements that "jump over" during the merge step.
- Other efficient methods include Binary Indexed Trees (Fenwick Tree) or Balanced Binary Search Trees.

Space & Time

Time Complexity: $O(n \log n)$ on average due to the merge sort based approach.
Space Complexity: $O(n)$ for auxiliary arrays used during sorting and counting.

Solution

We use a modified Merge Sort to sort the array while counting the number of smaller elements to the right. We keep track of the original indices by pairing each element with its index. During the merge process, when an element from the left subarray is placed into the temporary array, we add the count of elements from the right subarray that have already been placed (since these represent smaller elements that were to its right). This efficient divide and conquer strategy guarantees an overall $O(n \log n)$ time complexity. Care must be taken to update the counts based on indices and during the merge step.

#410 Split Array Largest Sum [Hard]

Key Insights

- Use binary search over the answer: the candidate value is the maximum subarray sum.
- The lower bound of the search is $\max(\text{nums})$ and the upper bound is $\sum(\text{nums})$.
- A greedy approach is used to check if a candidate maximum sum allows splitting `nums` into at most k subarrays.
- Adjust the binary search bounds based on the feasibility check.

Binary Search • LeetCode • By Pattern

— 2493 —

LeetCode

Space & Time

Time Complexity: $O(n * \log(\sum(\text{nums})))$ • n elements are processed for each binary search iteration.
Space Complexity: $O(1)$ • Constant extra space is used.

Solution

The solution employs binary search to determine the smallest maximum subarray sum possible. We set the initial lower bound to the maximum element and the upper bound to the total sum of the array. For each candidate value `mid`, we check if the array can be split into k or fewer subarrays without any subarray sum exceeding `mid` using a greedy approach. If possible, we try to lower the candidate; if not, we increase the candidate. This continues until the optimal solution is found.

#668 Kth Smallest Number in Multiplication Table [Hard]

Key Insights

- The table elements are not fully sorted row-wise or column-wise globally, but there is a pattern that can be exploited.
- For any candidate number x , we can count how many numbers in the table are less than or equal to x by summing over rows: for row i , there are $\text{min}(x // i, n)$ numbers.
- Binary search can be used over the range $[1, m * n]$ to find the smallest x such that the count is at least k .
- The counting function is the key to connecting the candidate value with the order statistic.

Space & Time

Time Complexity: $O(m * \log(m * n))$
Space Complexity: $O(1)$

Solution

We use binary search over the possible values in the multiplication table, which range from 1 to $m * n$. For each midpoint x in the binary search, we compute the number of elements in the table that are less than or equal to x by iterating over each row and summing up $\text{min}(x // i, n)$. This count tells us whether the k th smallest element is to the left or right of x . We then adjust the binary search boundaries accordingly. The approach leverages the multiplicative structure of the table and the monotonicity of the counting function.

#710 Random Pick with Blacklist [Hard]

Key Insights

- The valid output range has a total count of available numbers $W = n - \text{len}(\text{blacklist})$.
- Instead of repeatedly generating random numbers and checking against a set, we can transform the range into a contiguous block $[0, W - 1]$ and map any blacklist number within that block to a valid number in the upper range.
- Mapping is precomputed: for any blacklisted number x in $[0, W - 1]$, assign it a valid number from $[W, n - 1]$ that is not blacklisted.

This approach ensures $O(1)$ pick time after an $O(b)$ precomputation where $b = \text{length of blacklist}$.

Space & Time

Time Complexity: constructor - $O(b)$ for initializing data structures, $\text{pick}()$ - $O(1)$ per call.
Space Complexity: $O(b)$ for storing the blacklist set and mapping.

Solution

We compute the count of valid numbers, $W = n - \text{len}(\text{blacklist})$. For numbers in the blacklist that are less than W , we need to map them to a valid number from the range $[W, n - 1]$. To do this, we first store all blacklist numbers in the higher range $[W, n - 1]$ in a set for quick lookup. Then for each blacklisted value in $[0, W - 1]$, we iterate from W upwards to find a number that is not blacklisted and assign it in a mapping. During the `pick()` function, we generate a random integer r in $[0, W - 1]$. If r is in our mapping, we return its corresponding mapped value; otherwise, we return r as it is already a valid number.

#774 Minimize Max Distance to Gas Station [Hard]

Binary Search • LeetCode • By Pattern

— 2492 —

LeetCode

#19 Dynamic Programming — 24 questions • #19 of 21

#70 EASY

Climbing Stairs

LeetCode • SimplifyIt • WalkCC • LeetCode • Editorial
Math • Dynamic Programming • Memoization

Lists Grid 169

Problem:

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps

Example 2:
Input: $n = 3$
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

>> Brute Force [Dynamic Programming] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def climbStairs(self, n: int) -> int:
        # Brute force (O(2^n)) - pure recursion, recomputes identical subproblems
        def climb(n):
            if n == 0: return 1
            if n == 1: return 1
            if n == 2: return 2
            return climb(n - 1) + climb(n - 2)
        return climb(n)
```

Python

Python

Binary Search • LeetCode • By Pattern

— 2494 —

LeetCode

```
# Define a function to compute the number of ways to climb stairs.
def climbStairs(n):
    # Base case: If there is only one step, return 1.
    if n == 1:
        return 1
    # Initialize the dp array with n+1 elements.
    dp = [0] * (n + 1)
    # There is 1 way to climb 1 step.
    dp[1] = 1
    # There are 2 ways to climb 2 steps.
    dp[2] = 2
    # Compute the number of ways for each step from 3 to n.
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2] # Sum of ways from the previous two steps.
    return dp[n]
```

Example usage:
print(climbStairs(3)) # Output: 3

Python

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # dp[i] is the number of ways to climb to the i-th stair
        dp = [1, 1] + [0] * (n - 1)
        for i in range(2, n + 1):
            dp[i] = dp[i - 1] + dp[i - 2]
        return dp[n]
```

Python

```
class Solution:
    def climbStairs(self, n: int) -> int:
        prev1, prev2 = 1, 0
        for i in range(2, n + 1):
            dp = prev1 + prev2
            prev1 = prev2
            prev2 = dp
        return prev1
```

Python

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

Python

Dynamic Programming • LeetCode • By Pattern

— 2495 —

LeetCode

```
import numpy as np

class Solution:
    def climbStairs(self, n: int) -> int:
        res = np.zeros((1, 1), np.dtype("O"))
        factor = np.zeros((1, 1), np.dtype("O"))
        n = 1
        while n:
            if n < 2:
                res = factor
            else:
                factor = res
                n -= 1
            return res[0, 0]
```

Python

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

C++

C++

```
class Solution {
public:
    int climbStairs(int n) {
        // dp[i] -> the number of ways to climb to the i-th stair
        vector<int> dp(n + 1);
        dp[0] = 1;
        dp[1] = 1;
        for (int i = 2; i <= n; ++i)
            dp[i] = dp[i - 1] + dp[i - 2];
        return dp[n];
    }
};
```

C++

Dynamic Programming • LeetCode • By Pattern

— 2496 —

LeetCode

```

class Solution {
//
// @param Integer n
// @return Integer
//
function climbStairs(n) {
    if (n == 2) {
        return 2;
    }
    dp = [0, 1, 2];
    for (i = 3; i <= n; i++) {
        dp[i] = dp[i - 2] + dp[i - 1];
    }
    return dp[n];
}
}

C++
class Solution {
//
// @param Integer n
// @return Integer
//
function climbStairs(n) {
    if (n == 2) {
        return 2;
    }
    dp = [0, 1, 2];
    for (i = 3; i <= n; i++) {
        dp[i] = dp[i - 2] + dp[i - 1];
    }
    return dp[n];
}
}

Java
//n
class Solution {
    public int climbStairs(int n) {
        // dp[i] = the number of ways to climb to the i-th stair
        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }
}

Java

```

Dynamic Programming · LeetCode - By Pattern — 2497 —

LeetCode

```

class Solution {
    public int climbStairs(int n) {
        int prev2 = 1; // dp[1 - 1]
        int prev1 = 1; // dp[1 - 2]

        for (int i = 2; i <= n; i++) {
            int dp = prev1 + prev2;
            prev2 = prev1;
            prev1 = dp;
        }

        return prev1;
    }
}

Java
class Solution {
    public int climbStairs(int n) {
        int a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
}

Java
class Solution {
    public int climbStairs(int n) {
        int a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            int c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
}

JavaScript
JavaScript

```

Dynamic Programming · LeetCode - By Pattern — 2498 —

LeetCode

```

//n
// @param {number} n
// @return {number}
//
var climbStairs = function(n) {
    let a = 0;
    let b = 1;
    for (let i = 0; i < n; i++) {
        const c = a + b;
        a = b;
        b = c;
    }
    return b;
};

JavaScript
//n
// @param {number} n
// @return {number}
//
var climbStairs = function(n) {
    let a = 0;
    let b = 1;
    for (let i = 0; i < n; i++) {
        const c = a + b;
        a = b;
        b = c;
    }
    return b;
};

TypeScript
TypeScript
function climbStairs(n: number): number {
    let p = 1;
    let q = 1;
    for (let i = 1; i <= n; i++) {
        [p, q] = [q, p + q];
    }
    return q;
}

TypeScript
function climbStairs(n: number): number {
    let p = 1;
    let q = 1;
    for (let i = 1; i <= n; i++) {
        [p, q] = [q, p + q];
    }
    return q;
}

Rust
Rust

```

Dynamic Programming · LeetCode - By Pattern — 2499 —

LeetCode

```

impl Solution {
    pub fn climb_stairs(n: i32) -> i32 {
        let mut dp = vec![0; (n + 1)];
        for i in 1..n {
            let t = p + q;
            p = q;
            q = t;
        }
        q
    }
}

Rust
impl Solution {
    pub fn climb_stairs(n: i32) -> i32 {
        let mut dp = vec![0; (n + 1)];
        for i in 1..n {
            let t = p + q;
            p = q;
            q = t;
        }
        q
    }
}

Python
# Define a function to compute the number of ways to climb stairs.
def climbStairs(n):
    # Base case: If there is only one step, return 1.
    if n == 1:
        return 1
    # Initialize the dp array with n+1 elements.
    dp = [0] * (n + 1)
    # There is 1 way to climb 1 step.
    dp[1] = 1
    # There are 2 ways to climb 2 steps.
    dp[2] = 2
    # Compute the number of ways for each step from 3 to n.
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2] # Sum of ways from the previous two steps.
    return dp[n]

# Example usage:
print(climbStairs(3)) # Output: 3

#121 EASY
Best Time to Buy and Sell Stock
LeetCode · SimpleLeetCode · WAAKCC · LeetCode · Editorial
Array · Dynamic Programming
LeetCode 169
Problem:

```

Dynamic Programming · LeetCode - By Pattern — 2500 —

LeetCode

You are given an array prices where prices[i] is the price of a given stock on the i-th day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- 1 <= prices.length <= 105
- 0 <= prices[i] <= 104

>> Brute Force (Dynamic Programming) Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def maxProfit(self, prices):
        # Buy and sell stock - try every buy/sell pair
        n = len(prices)
        res = 0
        for i in range(n):
            for j in range(i + 1, n):
                res = max(res, prices[j] - prices[i])
        return res

Python
Python

```

Dynamic Programming · LeetCode - By Pattern — 2501 —

LeetCode

```

def maxProfit(prices):
    # Initialize min price as infinity and max profit as 0
    min_price = float('inf')
    max_profit = 0
    # Iterate through each price in the list
    for price in prices:
        # Update the minimum price if current price is lower
        if price < min_price:
            min_price = price
        else:
            # Calculate current profit and update max_profit if it is higher
            max_profit = max(max_profit, price - min_price)
    return max_profit

# Example usage:
# result = maxProfit([7,1,5,3,6,4])
# print(result) # Output: should be 5

Python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        sellOne = 0
        holdOne = -math.inf
        for price in prices:
            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)
        return sellOne

Python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, hold = 0, -inf
        for v in prices:
            ans = max(ans, v - hold)
            hold = min(hold, v)
        return ans

Python

```

Dynamic Programming · LeetCode - By Pattern — 2502 —

LeetCode

```

...
//> import math
//> a = math.inf
//> a
inf
//> v = float("-inf")
//> v < float("inf")
//> print(a < v) # Output: True
True
//>
//> z = -math.inf
//> ans
True
...

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, m1 = 0, inf
        for v in prices:
            m1 = min(m1, v)
            ans = max(ans, v - m1)
        return ans

#####

class Solution(object):
    def maxProfit(self, prices):
        #type prices: List[int]
        #type ans: int
        if not prices:
            return 0
        ans = 0
        pre = prices[0]
        for i in range(1, len(prices)):
            pre = max(pre, prices[i])
            ans = max(prices[i] - pre, ans)
        return ans

```

C++
C++

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int sellDate = 0;
        int holdDate = INT_MIN;
        for (const int price : prices) {
            sellDate = max(sellDate, holdDate + price);
            holdDate = max(holdDate, -price);
        }
        return sellDate;
    }
};

C++

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int ans = 0, m1 = prices[0];
        for (int& v : prices) {
            ans = max(ans, v - m1);
            m1 = min(m1, v);
        }
        return ans;
    }
};

C++

class Solution {
//>
//> @param Integer[] prices
//> @return Integer
//>
    function maxProfit(prices) {
        let ans = 0;
        let m1 = prices[0];
        for each (price in prices) {
            ans = Math.max(ans, v - m1);
            m1 = min(m1, v);
        }
        return ans;
    }
};

C++

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int ans = 0, m1 = prices[0];
        for (int& v : prices) {
            ans = max(ans, v - m1);
            m1 = min(m1, v);
        }
        return ans;
    }
};

```

```

C++

class Solution {
//>
//> @param Integer[] prices
//> @return Integer
//>
    function maxProfit(prices) {
        let ans = 0;
        let m1 = prices[0];
        let len = prices.length;
        for (let i = 1; i < len; i++) {
            let m1 = min(m1, prices[i]);
            let ans = max(ans, prices[i] - m1);
        }
        return ans;
    }
};

Java

class Solution {
    public int maxProfit(int[] prices) {
        int sellDate = 0;
        int holdDate = Integer.MIN_VALUE;
        for (final int price : prices) {
            sellDate = Math.max(sellDate, holdDate + price);
            holdDate = Math.max(holdDate, -price);
        }
        return sellDate;
    }
}

Java

class Solution {
    public int maxProfit(int[] prices) {
        int ans = 0, m1 = prices[0];
        for (int v : prices) {
            ans = Math.max(ans, v - m1);
            m1 = Math.min(m1, v);
        }
        return ans;
    }
}

Java

```

```

public class Solution {
    public int maxProfit(int[] prices) {
        int ans = 0, m1 = prices[0];
        for each (int v in prices) {
            ans = Math.max(ans, v - m1);
            m1 = Math.min(m1, v);
        }
        return ans;
    }
}

Java

class Solution {
    public int maxProfit(int[] prices) {
        int ans = 0, m1 = prices[0];
        for (int v : prices) {
            ans = Math.max(ans, v - m1);
            m1 = Math.min(m1, v);
        }
        return ans;
    }
}

Java

public class Solution {
    public int maxProfit(int[] prices) {
        int ans = 0, m1 = prices[0];
        for each (int v in prices) {
            ans = Math.max(ans, v - m1);
            m1 = Math.min(m1, v);
        }
        return ans;
    }
}

JavaScript

//>
//> @param {number[]} prices
//> @return {number}
//>
var maxProfit = function(prices) {
    let ans = 0;
    let m1 = prices[0];
    for (const v of prices) {
        ans = Math.max(ans, v - m1);
        m1 = Math.min(m1, v);
    }
    return ans;
};

JavaScript

```

```

//>
//> @param {number[]} prices
//> @return {number}
//>
var maxProfit = function(prices) {
    let ans = 0;
    let m1 = prices[0];
    for (const v of prices) {
        ans = Math.max(ans, v - m1);
        m1 = Math.min(m1, v);
    }
    return ans;
};

TypeScript

function maxProfit(prices: number[]): number {
    let ans = 0;
    let m1 = prices[0];
    for (const v of prices) {
        ans = Math.max(ans, v - m1);
        m1 = Math.min(m1, v);
    }
    return ans;
}

TypeScript

function maxProfit(prices: number[]): number {
    let ans = 0;
    let m1 = prices[0];
    for (const v of prices) {
        ans = Math.max(ans, v - m1);
        m1 = Math.min(m1, v);
    }
    return ans;
}

Go

func maxProfit(prices []int) (ans int) {
    m1 := prices[0]
    for _, v := range prices {
        ans = max(ans, v-m1)
        m1 = min(m1, v)
    }
    return
}

Go

```

```

func maxProfit(prices []int) (ans int) {
    m1 := prices[0]
    for _, v := range prices {
        ans = max(ans, v-m1)
        m1 = min(m1, v)
    }
    return
}

Rust

impl Solution {
    pub fn max_profit(prices: Vec<i32>) -> i32 {
        let mut ans = 0;
        let mut m1 = prices[0];
        for &v in prices {
            ans = ans.max(v - m1);
            m1 = m1.min(v);
        }
        ans
    }
}

Rust

impl Solution {
    pub fn max_profit(prices: Vec<i32>) -> i32 {
        let mut res = 0;
        let mut m1 = i32::MAX;
        for price in prices {
            res = res.max(price - m1);
            m1 = m1.min(price);
        }
        res
    }
}

[7] Dynamic Programming - Pattern Solution (stored optimal)

Python

def maxProfit(prices):
    # Initialize max_profit as infinity and max_profit as 0
    m1_price = float('inf')
    max_profit = 0
    # Iterate through each price in the list
    for price in prices:
        # Update the minimum price if current price is lower
        if price < m1_price:
            m1_price = price
        else:
            # Calculate current profit and update max_profit if it is higher
            max_profit = max(max_profit, price - m1_price)
    return max_profit

# Example usage:
# result = maxProfit([7,1,5,3,6,4])
# print(result) # Output should be 5

```

```

C++
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        // Initialize maxPrice to maximum integer and maxProfit to 0
        int maxPrice = INT_MAX;
        int maxProfit = 0;
        // Loop through each price in the vector
        for (int price : prices) {
            // Update maxPrice if the current price is lower
            maxPrice = min(maxPrice, price);
            // Calculate profit for current day and update maxProfit if higher
            maxProfit = max(maxProfit, price - maxPrice);
        }
        return maxProfit;
    }
};

// Example usage:
// int main() {
//     Solution sol;
//     vector<int> prices = {7,1,5,3,6,4};
//     cout << sol.maxProfit(prices); // Output should be 5
//     return 0;
// }

```

#5 MEDIUM Longest Palindromic Substring

LeetCode's Longest Palindromic Substring · WikiCS · LeetCode · Editorial

String · Dynamic Programming

Lists: Grind 169

Problem: Given a string s, return the longest palindromic substring in s.

Example 1:

Input: s = "babab"
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2:

Input: s = "cbbd"
Output: "bb"

Constraints:

- 1 ≤ s.length ≤ 1000
- s consist of only digits and English letters.

Dynamic Programming · LeetCodeMastery — By Pattern — 2509 —

LeetCodeMastery

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ""
        # (start, end) indices of the longest palindrome in s
        indices = (0, 0)
        def extend(i: str, i: int, j: int) -> tuple(int, int):
            Returns the (start, end) indices of the longest palindrome extended from
            the substring s[i:j].
            while i >= 0 and j < len(s):
                if s[i] == s[j]:
                    break
                i -= 1
                j += 1
            return i + 1, j - 1
        for i in range(len(s)):
            l1, r1 = extend(s, i, i)
            if r1 - l1 > indices[1] - indices[0]:
                indices = l1, r1
            if i >= 1 < len(s) and s[i] == s[i - 1]:
                l2, r2 = extend(s, i - 1, i)
                if r2 - l2 > indices[1] - indices[0]:
                    indices = l2, r2
        return s[indices[0]:indices[1] + 1]

```

Python

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        t = '@' + s + '@'
        p = self._manacher(t)
        maxPalindromLength, bestCenter = max(enumerate(p))
        l = (bestCenter - maxPalindromLength) // 2
        r = (bestCenter + maxPalindromLength) // 2
        return s[l:r]
    def _manacher(self, t: str) -> list[int]:
        Returns an array 'p' s.t. p[i] is the length of the longest palindrome
        centered at t[i], where 't' is a string with delimiters and sentinels.
        p = [0] * len(t)
        center = 0
        for i in range(1, len(t) - 1):
            rightBoundary = center + p[center]
            mirrorIndex = center - (i - center)
            if rightBoundary > i:
                p[i] = min(rightBoundary - i, p[mirrorIndex])
            # Try to extend the palindrome centered at i
            while t[i + 1 + p[i]] == t[i - 1 - p[i]]:
                p[i] += 1

```

Dynamic Programming · LeetCodeMastery — By Pattern — 2511 —

LeetCodeMastery

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        nlen = 0
        start = end = 0
        n = len(s)
        dp = [False] * n
        for i in range(n):
            for j in range(i + 1):
                dp[j][i] = (s[i] == s[j] and j - i == 1) or (s[i] == s[j] and dp[i - 1][j - 1])
                if dp[j][i] is True and j - i + 1 > nlen:
                    start = j
                    end = i
                    nlen = j - i + 1
            return s[start: end + 1]
        return s[start: end + 1]

```

C++

```

class Solution {
public:
    string longestPalindrome(string s) {
        int n = s.size();
        vector<vector<bool>> f(n, vector<bool>(n, true));
        int k = 0, mx = 0;
        for (int i = n - 2; i >= 0; i--) {
            f[i][i] = true;
            for (int j = i + 1; j < n; j++) {
                if (s[i] == s[j]) {
                    if (f[i + 1][j - 1]) {
                        if (f[i][j]) && mx < j - i + 1 {
                            mx = j - i + 1;
                            k = i;
                        }
                    }
                }
            }
        }
        return s.substr(k, mx);
    }
};

```

Dynamic Programming · LeetCodeMastery — By Pattern — 2513 —

LeetCodeMastery

>> Brute Force (Dynamic Programming) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def expand_from_center(self, s, left, right):
        # Brute Force O(n^2) - check every substring for palindrome property
        def is_palindrome(sub):
            return sub == sub[::-1]
        res = ""
        for i in range(len(s)):
            for j in range(i + 1, len(s) + 1):
                sub = s[i:j]
                if is_palindrome(sub) and len(sub) > len(res):
                    res = sub
        return res

```

Python

```

# Function to expand from the center and return the palindrome substring
def expand_from_center(s, left, right):
    # Expand while the characters on both sides are the same
    while left >= 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    # Return the palindromic substring found after expansion
    return s[left + 1:right]
def longest_palindromic_substring(s):
    if not s:
        return ""
    longest = ""
    # Iterate over each index in string as potential center
    for i in range(len(s)):
        # Check for odd-length palindromes
        odd_palindrome = expand_from_center(s, i, i)
        if len(odd_palindrome) > len(longest):
            longest = odd_palindrome
        # Check for even-length palindromes
        even_palindrome = expand_from_center(s, i, i + 1)
        if len(even_palindrome) > len(longest):
            longest = even_palindrome
    return longest
# Example usage
print(longest_palindromic_substring("babab"))

```

Python

Dynamic Programming · LeetCodeMastery — By Pattern — 2510 —

LeetCodeMastery

```

# If a palindrome centered at i expands past 'rightBoundary', adjust
# the center based on the expanded palindrome.
if i + p[i] > rightBoundary:
    center = i
    return p

```

Python

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        p = [0] * n
        for i in range(n):
            l, r = i, i
            while l >= 0 and r < n and s[l] == s[r]:
                l -= 1
                r += 1
            p[i] = r - l - 1
        max_i = 0
        for i in range(n):
            start, end = i - p[i], i + p[i]
            if end - start > max_i:
                max_i = end - start
                start_idx = start
                end_idx = end
        return s[start_idx:end_idx]

```

Python

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        def f(l, r):
            while l >= 0 and r < len(s) and s[l] == s[r]:
                l -= 1
                r += 1
            return r - l - 1
        n = len(s)
        start, end = 0, 1
        for i in range(n):
            a = f(i, i)
            b = f(i, i + 1)
            t = max(a, b)
            if t > end - start:
                start = i - (t - 1) // 2
                end = i + (t - 1) // 2
        return s[start: end]

```

Python

Dynamic Programming · LeetCodeMastery — By Pattern — 2512 —

LeetCodeMastery

```

C++
class Solution {
public:
    string longestPalindrome(string s) {
        int n = s.size();
        vector<vector<bool>> f(n, vector<bool>(n, true));
        int k = 0, mx = 0;
        for (int i = n - 2; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s[i] == s[j]) {
                    if (f[i + 1][j - 1]) {
                        if (f[i][j]) && mx < j - i + 1 {
                            mx = j - i + 1;
                            k = i;
                        }
                    }
                }
            }
        }
        return s.substr(k, mx);
    }
};

```

C++

```

class Solution {
public:
    string longestPalindrome(string s) {
        int n = s.size();
        vector<vector<bool>> f(n, vector<bool>(n, true));
        int k = 0, mx = 0;
        for (int i = n - 2; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s[i] == s[j]) {
                    if (f[i + 1][j - 1]) {
                        if (f[i][j]) && mx < j - i + 1 {
                            mx = j - i + 1;
                            k = i;
                        }
                    }
                }
            }
        }
        return s.substr(k, mx);
    }
};

```

Dynamic Programming · LeetCodeMastery — By Pattern — 2514 —

LeetCodeMastery

```

C++
class Solution {
public:
    string longestPalindrome(string s) {
        int start = 0;
        int maxLength = 0;

        for (int i = 0; i < s.length(); i++) {
            int left = i;
            int right = i;

            while (left < s.length() && right < s.length() && s[left] == s[right]) {
                left++;
                right++;
            }

            if (right - left > maxLength) {
                start = left;
                maxLength = right - left;
            }
        }

        return s.substr(start, maxLength);
    }
};

Java

```

Dynamic Programming · LeetCode · By Pattern — 2515 — LeetCode

```

class Solution {
    public String longestPalindrome(String s) {
        int n = s.length();
        boolean[][] f = new boolean[n][n];
        for (int i = 0; i < n; i++) {
            f[i][i] = true;
        }
        int k = 0;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                f[i][j] = false;
                if (s.charAt(i) == s.charAt(j)) {
                    f[i][j] = f[i + 1][j - 1];
                    if (f[i][j] && j - i < 2) {
                        k = j - i + 1;
                    }
                }
            }
        }
        return s.substring(k, k + k);
    }
}

Java
class Solution {
    private String s;
    private int n;

    public String longestPalindrome(String s) {
        this.s = s;
        n = s.length();
        int start = 0;
        int max = 1;
        for (int i = 0; i < n; i++) {
            int a = f(i, i);
            int b = f(i, i + 1);
            int t = Math.max(a, b);
            if (t > max) {
                max = t;
                start = i - (t - 1);
            }
        }
        return s.substring(start, start + max);
    }

    private int f(int l, int r) {
        while (l <= r && s.charAt(l) == s.charAt(r)) {
            l++;
            r++;
        }
        return r - l - 1;
    }
}

Java

```

Dynamic Programming · LeetCode · By Pattern — 2516 — LeetCode

```

public class Solution {
    private String s;
    private int n;

    public String longestPalindrome(String s) {
        this.s = s;
        n = s.length();
        int start = 0;
        int max = 1;
        for (int i = 0; i < n; i++) {
            int a = f(i, i);
            int b = f(i, i + 1);
            int t = Math.max(a, b);
            if (t > max) {
                max = t;
                start = i - (t - 1);
            }
        }
        return s.substring(start, start + max);
    }

    private int f(int l, int r) {
        while (l <= r && s.charAt(l) == s.charAt(r)) {
            l++;
            r++;
        }
        return r - l - 1;
    }
}

Java
class Solution {
    public String longestPalindrome(String s) {
        int n = s.length();
        boolean[][] f = new boolean[n][n];
        for (int i = 0; i < n; i++) {
            f[i][i] = true;
        }
        int k = 0;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                f[i][j] = false;
                if (s.charAt(i) == s.charAt(j)) {
                    f[i][j] = f[i + 1][j - 1];
                    if (f[i][j] && j - i < 2) {
                        k = j - i + 1;
                    }
                }
            }
        }
        return s.substring(k, k + k);
    }
}

Java

```

Dynamic Programming · LeetCode · By Pattern — 2517 — LeetCode

```

public class Solution {
    public String longestPalindrome(String s) {
        int n = s.length();
        boolean[][] f = new boolean[n][n];
        for (int i = 0; i < n; i++) {
            f[i][i] = true;
        }
        int k = 0;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                f[i][j] = false;
                if (s.charAt(i) == s.charAt(j)) {
                    f[i][j] = f[i + 1][j - 1];
                    if (f[i][j] && j - i < 2) {
                        k = j - i + 1;
                    }
                }
            }
        }
        return s.substring(k, k + k);
    }
}

JavaScript
var longestPalindrome = function(s) {
    const n = s.length;
    const f = Array(n).fill(0);
    for (let i = 0; i < n; i++) {
        f[i][i] = true;
    }
    let k = 0;
    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            f[i][j] = false;
            if (s[i] === s[j]) {
                f[i][j] = f[i + 1][j - 1];
                if (f[i][j] && j - i < 2) {
                    k = j - i + 1;
                }
            }
        }
    }
    return s.slice(k, k + k);
};

JavaScript

```

Dynamic Programming · LeetCode · By Pattern — 2518 — LeetCode

```

var longestPalindrome = function(s) {
    const n = s.length;
    const f = Array(n).fill(0);
    for (let i = 0; i < n; i++) {
        f[i][i] = true;
    }
    let k = 0;
    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            f[i][j] = false;
            if (s[i] === s[j]) {
                f[i][j] = f[i + 1][j - 1];
                if (f[i][j] && j - i < 2) {
                    k = j - i + 1;
                }
            }
        }
    }
    return s.slice(k, k + k);
};

TypeScript
function longestPalindrome(s: string): string {
    const n = s.length;
    const f: boolean[][] = Array(n).fill(0);
    for (let i = 0; i < n; i++) {
        f[i][i] = true;
    }
    let k = 0;
    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            f[i][j] = false;
            if (s[i] === s[j]) {
                f[i][j] = f[i + 1][j - 1];
                if (f[i][j] && j - i < 2) {
                    k = j - i + 1;
                }
            }
        }
    }
    return s.slice(k, k + k);
}

TypeScript

```

Dynamic Programming · LeetCode · By Pattern — 2519 — LeetCode

```

import std/sequtils;

proc longestPalindrome(s: string): string =
    let n = s.len()
    var
        dp = newSeqWith(n, newSeqWith(n, false))
        start = 0
        maxLen = 1

    for j in 0 .. n-1:
        for i in 0 .. j:
            dp[i][j] = s[i] == s[j]
            if dp[i][j] == s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] and s[i] == s[j]
            if dp[i][j] and maxLen < j - i + 1:
                start = i
                maxLen = j - i + 1

    result = s[start .. start + maxLen - 1]

TypeScript
function longestPalindrome(s: string): string {
    const n = s.length;
    const f: boolean[][] = Array(n).fill(0);
    for (let i = 0; i < n; i++) {
        f[i][i] = true;
    }
    let k = 0;
    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            f[i][j] = false;
            if (s[i] === s[j]) {
                f[i][j] = f[i + 1][j - 1];
                if (f[i][j] && j - i < 2) {
                    k = j - i + 1;
                }
            }
        }
    }
    return s.slice(k, k + k);
}

TypeScript

```

Dynamic Programming · LeetCode · By Pattern — 2520 — LeetCode


```

import std::string;

proc longestPalindrome(s: string): string =
  let n = s.len()
  var
    dp = newSeqWith(n*(n-1), newSeqWith(n*(n-1), false))
    start, end = 0
    mx = 1
  for j in 0 ..< n:
    for i in 0 .. j:
      if s[i] == s[j]:
        dp[i][j] = s[i] == s[j]
      else:
        dp[i][j] = dp[i+1][j-1] and s[i] == s[j]
      if dp[i][j] and mx < j - i + 1:
        start = i
        end = j - i + 1
  result = s[start .. end]

# Driver Code
# echo longestPalindrome("abcbda")

```

Rust

```

Rust
impl Solution {
    pub fn longest_palindrome(s: String) -> String {
        let (n, mut ans) = (s.len(), &[]: &str);
        let mut dp = vec![vec![false; n]; n];
        let data: Vec<char> = s.chars().collect();
        for end in 1..n {
            for start in 0..=end {
                if data[start] == data[end] {
                    dp[start][end] = end - start < 2 || dp[start + 1][end - 1];
                    if dp[start][end] && end - start + 1 > ans.len() {
                        ans = &data[start..end];
                    }
                }
            }
        }
        ans.to_string()
    }
}

```

Rust

```

impl Solution {
    pub fn longest_palindrome(s: String) -> bool {
        let mut chars = s.chars();
        while let (Some(c1), Some(c2)) = (chars.next(), chars.next_back()) {
            if c1 != c2 {
                return false;
            }
        }
        true
    }

    pub fn longest_palindrome(s: String) -> String {
        let size = s.len();
        let mut ans = &[]: &str;
        for i in 0..size-1 {
            for j in (i+1..size).rev() {
                if ans.len() == j - i + 1 {
                    break;
                }
                if Solution::is_palindrome(&s[i..j]) {
                    ans = &s[i..j];
                }
            }
        }
        ans.to_string();
    }
}

```

Rust

```

impl Solution {
    pub fn longest_palindrome(s: String) -> String {
        let (n, mut ans) = (s.len(), &[]: &str);
        let mut dp = vec![vec![false; n]; n];
        let data: Vec<char> = s.chars().collect();
        for end in 1..n {
            for start in 0..=end {
                if data[start] == data[end] {
                    dp[start][end] = end - start < 2 || dp[start + 1][end - 1];
                    if dp[start][end] && end - start + 1 > ans.len() {
                        ans = &data[start..end];
                    }
                }
            }
        }
        ans.to_string()
    }
}

```

Rust

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        start = end = 0
        dp = [False] * n
        for j in range(n):
            for i in range(0, j):
                if s[i] == s[j] and (j - i == 1 or (j - i == 2 and s[i+1] == s[j-1]) or (j - i > 2 and dp[i+1][j-1])):
                    dp[i][j] = True
                    start = i
                    end = j
        return s[start: end + 1]

```

Python

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        h, m = 0, 1
        for i in range(n-2, -1, -1):
            for j in range(i+1, n):
                if s[i] == s[j]:
                    dp[i][j] = dp[i+1][j-1]
                    if dp[i][j] and m < j - i + 1:
                        h, m = i, j - i + 1
            return s[h: h+m]

```

#53

MEDIUM

Maximum Subarray

LeetCode · Simple · Easy · Medium · Hard · LeetCode · Editorial

Array · Dynamic Programming · Divide and Conquer

Like · Grind 189

Problem:

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: `6`

Explanation: The subarray `[4,-1,2,1]` has the largest sum `6`.

Example 2:

Input: `nums = [1]`

Output: `1`

Explanation: The subarray `[1]` has the largest sum `1`.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: `23`
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum `23`.

Constraints:

- `1 <= nums.length <= 105`
- `-104 <= nums[i] <= 104`

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the divide and conquer approach, which is more subtle.

>> Brute Force [Dynamic Programming] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        # Brute Force O(n^2) - compute sum of every contiguous subarray
        res = nums[0]
        for i in range(len(nums)):
            curr = 0
            for j in range(i, len(nums)):
                curr += nums[j]
            res = max(res, curr)
        return res

```

Python

```

# Python solution using Kadane's Algorithm
def maxSubArray(nums: List[int]) -> int:
    # Initialize max_sum and current to the first element
    max_sum = nums[0]
    current_sum = nums[0]
    # Iterate through the array starting from the second element
    for num in nums[1:]:
        # If current_sum + num is less than num, restart at num
        current_sum = max(num, current_sum + num)
        # Update max_sum if current_sum is greater
        max_sum = max(max_sum, current_sum)
    return max_sum

```

Python

```

# Example usage
if __name__ == "__main__":
    sample = [-2,1,-3,4,-1,2,1,-5,4]
    print(maxSubArray(sample)) # Output: 6

```

Python

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        # dp[i] = the maximum sum subarray ending in i
        dp = [0] * len(nums)
        dp[0] = nums[0]
        for i in range(1, len(nums)):
            dp[i] = max(nums[i], dp[i-1] + nums[i])
        return max(dp)

```

Python

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        ans = -math.inf
        sum = 0
        for num in nums:
            sum = max(sum, num)
            ans = max(ans, sum)
        return ans

```

Python

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Y:
    sum: int
    # The sum of the subarray starting from the first number
    maxSubarrayLeft: int
    # The sum of the subarray ending in the last number
    maxSubarrayRight: int
    maxSubarraySum: int

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def divideAndConquer(l: int, r: int) -> Y:
            if l == r:
                return Y(nums[l], nums[l], nums[l])
            m = (l + r) // 2
            left = divideAndConquer(l, m)
            right = divideAndConquer(m + 1, r)
            maxSubarrayLeft = max(left.maxSubarrayLeft,
                                   left.sum + right.maxSubarraySumLeft)
            maxSubarrayRight = max(right.maxSubarraySumRight,
                                   right.sum + left.maxSubarraySumLeft)
            maxSubarraySum = max(left.maxSubarraySumLeft + right.maxSubarraySumRight,
                                   left.maxSubarraySum, right.maxSubarraySum)
            return Y(sum, maxSubarrayLeft, maxSubarrayRight, maxSubarraySum)
        return divideAndConquer(0, len(nums) - 1).maxSubarraySum

```

Python

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        ans = float('-inf')
        for i in range(1, len(nums)):
            ans = max(ans, nums[i])
        return ans

```

Python

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def crossSubArraySum(left, mid, right):
            lsum = rsum = 0
            for i in range(mid, left - 1, -1):
                lsum += nums[i]
            for i in range(mid + 1, right + 1):
                rsum += nums[i]
            return max(lsum, rsum)
        return max(lsum, rsum)

    def maxSubArray(self, left, right):
        if left == right:
            return nums[left]
        mid = (left + right) // 2
        lsum = maxSubArraySum(left, mid)
        rsum = maxSubArraySum(mid + 1, right)
        crossSubArraySum = crossSubArraySum(left, mid, right)
        return max(lsum, rsum, crossSubArraySum)

    def maxSubArray(self, left, right):
        if left == right:
            return nums[left]
        mid = (left + right) // 2
        lsum = maxSubArraySum(left, mid)
        rsum = maxSubArraySum(mid + 1, right)
        crossSubArraySum = crossSubArraySum(left, mid, right)
        return max(lsum, rsum, crossSubArraySum)

```

Python

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        res = cur_sum = nums[0]
        for num in nums[1:]:
            cur_sum = max(cur_sum, num)
            res = max(res, cur_sum)
        return res

```

C++

```

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        // dp[i] = the maximum sum subarray ending in i
        vector<int> dp(nums.size());
        dp[0] = nums[0];
        for (int i = 1; i < nums.size(); ++i)
            dp[i] = max(nums[i], dp[i-1] + nums[i]);
        return dp[nums.size()-1];
    }
};

```

C++

```

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int ans = INT_MIN;
        int sum = 0;
        for (const int num : nums) {
            sum = max(sum, sum + num);
            ans = max(ans, sum);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int ans = nums[0], f = nums[0];
        for (int i = 1; i <= nums.size(); ++i) {
            f = max(f, 0) + nums[i];
            ans = max(ans, f);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int ans = nums[0], f = nums[0];
        for (int i = 1; i <= nums.size(); ++i) {
            f = max(f, 0) + nums[i];
            ans = max(ans, f);
        }
        return ans;
    }
};

```

Java

```

class Solution {
    public int maxSubArray(int[] nums) {
        int ans = nums[0];
        for (int i = 1; i <= nums.length; ++i) {
            f = Math.max(f, 0) + nums[i];
            ans = Math.max(ans, f);
        }
        return ans;
    }
}

```

Dynamic Programming · LeetCode · By Pattern — 2527 —

LeetCode

```

Java
class Solution {
    public int maxSubArray(int[] nums) {
        int ans = Integer.MIN_VALUE;
        int sum = 0;
        for (final int num : nums) {
            sum = Math.max(sum, sum + num);
            ans = Math.max(ans, sum);
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int maxSubArray(int[] nums) {
        int ans = nums[0];
        for (int i = 1; i <= nums.length; ++i) {
            f = Math.max(f, 0) + nums[i];
            ans = Math.max(ans, f);
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int maxSubArray(int[] nums) {
        int ans = nums[0], f = nums[0];
        for (int i = 1; i <= nums.length; ++i) {
            f = Math.max(f, 0) + nums[i];
            ans = Math.max(ans, f);
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int maxSubArray(int[] nums) {
        int ans = nums[0];
        for (int i = 1; i <= nums.length; ++i) {
            f = Math.max(f, 0) + nums[i];
            ans = Math.max(ans, f);
        }
        return ans;
    }
}

```

Dynamic Programming · LeetCode · By Pattern — 2528 —

LeetCode

```

public class Solution {
    public int maxSubArray(int[] nums) {
        int ans = nums[0], f = nums[0];
        for (int i = 1; i <= nums.length; ++i) {
            f = Math.max(f, 0) + nums[i];
            ans = Math.max(ans, f);
        }
        return ans;
    }
}

```

JavaScript

```

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubArray = function (nums) {
    let [ans, f] = [nums[0], nums[0]];
    for (let i = 1; i <= nums.length; ++i) {
        f = Math.max(f, 0) + nums[i];
        ans = Math.max(ans, f);
    }
    return ans;
};

```

JavaScript

```

/**
 * @param {number[]} nums
 * @return {number}
 */
var maxSubArray = function (nums) {
    let [ans, f] = [nums[0], nums[0]];
    for (let i = 1; i <= nums.length; ++i) {
        f = Math.max(f, 0) + nums[i];
        ans = Math.max(ans, f);
    }
    return ans;
};

```

TypeScript

```

function maxSubArray(nums: number[]): number {
    let [ans, f] = [nums[0], nums[0]];
    for (let i = 1; i <= nums.length; ++i) {
        f = Math.max(f, 0) + nums[i];
        ans = Math.max(ans, f);
    }
    return ans;
}

```

Dynamic Programming · LeetCode · By Pattern — 2529 —

LeetCode

```

function maxSubArray(nums: number[]): number {
    let [ans, f] = [nums[0], nums[0]];
    for (let i = 1; i <= nums.length; ++i) {
        f = Math.max(f, 0) + nums[i];
        ans = Math.max(ans, f);
    }
    return ans;
}

```

Go

```

func maxSubArray(nums []int) int {
    ans, f := nums[0], nums[0]
    for _, x := range nums[1:] {
        f = max(f, 0) + x
        ans = max(ans, f)
    }
    return ans
}

```

Go

```

func maxSubArray(nums []int) int {
    ans, f := nums[0], nums[0]
    for _, x := range nums[1:] {
        f = max(f, 0) + x
        ans = max(ans, f)
    }
    return ans
}

```

Rust

```

impl Solution {
    pub fn max_sub_array(nums: Vec<i32>) -> i32 {
        let n = nums.len();
        let mut ans = nums[0];
        let mut f = nums[0];
        for i in 1..n {
            f = f.max(0) + nums[i];
            ans = ans.max(f);
        }
        ans
    }
}

```

Dynamic Programming · LeetCode · By Pattern — 2530 —

LeetCode

```

impl Solution {
    pub fn max_sub_array(nums: Vec<i32>) -> i32 {
        let n = nums.len();
        let mut ans = nums[0];
        let mut f = nums[0];
        for i in 1..n {
            f = f.max(0) + nums[i];
            ans = ans.max(f);
        }
        ans
    }
}

```

[1] Dynamic Programming — Pattern Solution (matched from community answers)

Python

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        # dp[i] = the maximum sum subarray ending in i
        dp = [0] * len(nums)
        dp[0] = nums[0]
        for i in range(1, len(nums)):
            dp[i] = max(nums[i], dp[i - 1] + nums[i])
        return max(dp)

```

#55 MEDIUM

Jump Game

LeetCode · SimplyLeet · WalkCC · LeetCode · Editorial

Array · Dynamic Programming · Greedy

Like · 600

Problem: You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position. Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

Input: `nums = [2,3,1,1,4]`
Output: `true`
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`
Output: `false`
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

• `1 <= nums.length <= 10^4`

Dynamic Programming · LeetCode · By Pattern — 2531 —

LeetCode

```

• 0 <= nums[i] <= 105

```

>> Brute Force [Dynamic Programming] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def canJump(self, nums):
        # Brute Force O(N^2) - recursion: try every reachable position
        def can_reach(i):
            if i == len(nums) - 1: return True
            for step in range(1, nums[i] + 1):
                if can_reach(i + step): return True
            return False
        return can_reach(0)

```

Python

```

# Function to determine if you can reach the last index of the array
def canJump(nums):
    # Initialize maxReach which is the max index reachable at start (index 0)
    maxReach = 0
    # Iterate through each element and its index
    for i, jump in enumerate(nums):
        # If current index is greater than maxReach, return False
        if i > maxReach:
            return False
        # Update maxReach with the maximum of current maxReach and i + jump length at index i
        maxReach = max(maxReach, i + jump)
        # If current maxReach is already at or beyond the last index, return True
        if maxReach == len(nums) - 1:
            return True
    # Return True if the end is reachable after processing all indices
    return True

```

Example usage:

```

# print(canJump([2,3,1,1,4])) # Expected output: True
# print(canJump([3,2,1,0,4])) # Expected output: False

```

Python

```

class Solution:
    def canJump(self, nums: List[int]) -> bool:
        i = 0
        reach = 0
        while i < len(nums) and i <= reach:
            reach = max(reach, i + nums[i])
            i += 1
        return i == len(nums)

```

Dynamic Programming · LeetCode · By Pattern — 2532 —

LeetCode

```

class Solution {
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
            return True

```

Python

```

class Solution {
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
            return True

```

C++

```

class Solution {
public:
    bool canJump(vector<int>& nums) {
        int i = 0;

        for (int reach = 0; i < nums.size() && i <= reach; ++i)
            reach = max(reach, i + nums[i]);

        return i == nums.size();
    }
};

```

C++

```

class Solution {
public:
    bool canJump(vector<int>& nums) {
        int mx = 0;
        for (int i = 0; i < nums.size(); ++i) {
            if (mx < i) {
                return false;
            }
            mx = max(mx, i + nums[i]);
            return true;
        }
    }
};

```

C++

```

class Solution {
    public:
        bool canJump(vector<int>& nums) {
            int mx = 0;
            for (int i = 0; i < nums.size(); ++i) {
                if (mx < i) {
                    return false;
                }
                mx = max(mx, i + nums[i]);
                return true;
            }
        }
};

```

Java

```

class Solution {
    public boolean canJump(int[] nums) {
        int i = 0;

        for (int reach = 0; i < nums.length && i <= reach; ++i)
            reach = Math.max(reach, i + nums[i]);

        return i == nums.length;
    }
}

```

Java

```

class Solution {
    public boolean canJump(int[] nums) {
        int mx = 0;
        for (int i = 0; i < nums.length; ++i) {
            if (mx < i) {
                return false;
            }
            mx = Math.max(mx, i + nums[i]);
        }
        return true;
    }
}

```

Java

```

public class Solution {
    public boolean canJump(int[] nums) {
        int mx = 0;
        for (int i = 0; i < nums.length; ++i) {
            if (mx < i) {
                return false;
            }
            mx = Math.max(mx, i + nums[i]);
        }
        return true;
    }
}

```

Java

```

class Solution {
    public boolean canJump(int[] nums) {
        int mx = 0;
        for (int i = 0; i < nums.length; ++i) {
            if (mx < i) {
                return false;
            }
            mx = Math.max(mx, i + nums[i]);
        }
        return true;
    }
}

```

Java

```

public class Solution {
    public boolean canJump(int[] nums) {
        int mx = 0;
        for (int i = 0; i < nums.length; ++i) {
            if (mx < i) {
                return false;
            }
            mx = Math.max(mx, i + nums[i]);
        }
        return true;
    }
}

```

JavaScript

```

// @param {number[]} nums
// @return {boolean}
var canJump = function(nums) {
    let mx = 0;
    for (let i = 0; i < nums.length; ++i) {
        if (mx < i) {
            return false;
        }
        mx = Math.max(mx, i + nums[i]);
    }
    return true;
};

```

JavaScript

```

// @param {number[]} nums
// @return {boolean}
var canJump = function(nums) {
    let mx = 0;
    for (let i = 0; i < nums.length; ++i) {
        if (mx < i) {
            return false;
        }
        mx = Math.max(mx, i + nums[i]);
    }
    return true;
};

```

TypeScript

```

function canJump(nums: number[]): boolean {
    let mx: number = 0;
    for (let i = 0; i < nums.length; ++i) {
        if (mx < i) {
            return false;
        }
        mx = Math.max(mx, i + nums[i]);
    }
    return true;
}

```

TypeScript

```

function canJump(nums: number[]): boolean {
    let mx: number = 0;
    for (let i = 0; i < nums.length; ++i) {
        if (mx < i) {
            return false;
        }
        mx = Math.max(mx, i + nums[i]);
    }
    return true;
}

```

Go

Go

```

func canJump(nums []int) bool {
    mx := 0
    for i, x := range nums {
        if mx < i {
            return false
        }
        mx = max(mx, i+x)
    }
    return true
}

```

Go

```

func canJump(nums []int) bool {
    mx := 0
    for i, x := range nums {
        if mx < i {
            return false
        }
        mx = max(mx, i+x)
    }
    return true
}

```

Rust

Rust

```

impl Solution {
    fn canJump(nums: Vec<i32>) -> bool {
        let n = nums.len();
        let mut mx = 0;

        for i in 0..n {
            if mx < i {
                return false;
            }
            mx = std::cmp::max(mx, i + (nums[i] as usize));
        }
        true
    }
}

```

Rust

```

impl Solution {
    fn canJump(nums: Vec<i32>) -> bool {
        let n = nums.len();
        let mut mx = 0;

        for i in 0..n {
            if mx < i {
                return false;
            }
            mx = std::cmp::max(mx, i + (nums[i] as usize));
        }
        true
    }
}

```

[9] Dynamic Programming · Pattern Solution (stored optimal)

Python

```

# Function to determine if you can reach the last index of the array
def canJump(nums):
    # Initializing maxreach which is the max index reachable at start (index 0)
    maxreach = 0
    # Iterate through each element and its index
    for i, jump in enumerate(nums):
        # If current index is greater than maxreach, return False
        if i > maxreach:
            return False
        # Update maxreach with the max of current maxreach and i + jump length at index i
        maxreach = max(maxreach, i + jump)
        # If current maxreach is already at or beyond the last index, return True
        if maxreach >= len(nums) - 1:
            return True
    # Return True if the end is reachable after processing all indices
    return True

# Example usage:
# print(canJump([2,3,1,1])) # Expected output: True
# print(canJump([3,2,1,1,4])) # Expected output: False

```

C++

```

// C++ Implementation to determine if it is possible to jump to the last index in the array
#include <iostream>
using namespace std;

class Solution {
public:
    bool canJump(vector<int>& nums) {
        int maxreach = 0;

        // Iterate through each index in the array
        for (int i = 0; i < nums.size(); i++) {
            // If current index is greater than maxreach, it's unreachable
            if (i > maxreach) {
                return false;
            }

            // Update maxreach with the max of current maxreach and index + jump at index i
            maxreach = max(maxreach, i + nums[i]);
            // If maxreach is greater or equal to the last index, return true
            if (maxreach >= nums.size() - 1) {
                return true;
            }
        }
        return true;
    }
}

```

```

})

// Example usage:
// int main() {
//     Solution sol;
//     vector<int> num1 = {2,3,1,3,4};
//     vector<int> num2 = {3,2,1,4,4};
//     cout << sol.canJump(num1) << endl; // Expected output: 1 (true)
//     cout << sol.canJump(num2) << endl; // Expected output: 0 (false)
//     return 0;
// }

```

#62

MEDIUM

Unique Paths

LeetCode • InterviewBit • WalkCC • LeetCode • Editorial

Math • Dynamic Programming • Combinatorics

Listo Grind 189

Problem:

There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can move either down or right at any point in time.

Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to $2^{31} - 1$.

Example 1:



Input: $m = 3, n = 7$

Output: 28

Example 2:

Input: $m = 3, n = 2$

Output: 3

Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right

Dynamic Programming • LeetCode • By Pattern

— 2539 —

LeetCode

corner:
1. Right → Down → Down
2. Down → Down → Right
3. Down → Right → Down

Constraints:

• $1 \leq m, n \leq 100$

>> Brute Force [Dynamic Programming] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # Brute Force O(2^(m+n)) - recursive without memo; each step right or down
        def path(r, c):
            if r == m - 1 and c == n - 1: return 1
            if r == m or c == n: return 0
            return path(r + 1, c) + path(r, c + 1)
        return path(0, 0)

```

Python

Python

```

# Python solution using Dynamic Programming
def uniquePaths(m, n):
    # Initialize a 2D list with 1's for the first row and first column
    dp = [[1] * n for _ in range(m)]
    # Fill in the rest of the table
    for i in range(1, m):
        for j in range(1, n):
            # The number of ways to reach this cell is the sum of the ways from the cell above and the cell on the left
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    # Return the value in the bottom-right cell
    return dp[m-1][n-1]
# Example usage:
print(uniquePaths(3, 7))

```

Python

```

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[i][j] -> the number of unique paths from (0, 0) to (i, j)
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i-1][j] + dp[i][j-1]
        return dp[m-1][n-1]

```

Python

Dynamic Programming • LeetCode • By Pattern

— 2540 —

LeetCode

```

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] += dp[i-1][j]
        return dp[m-1][n-1]

```

Python

```

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [1] * n
        for i in range(1, m):
            f[0][0] = 1
            for j in range(1, n):
                f[i][j] = f[i-1][j] + f[i][j-1]
            if i == m-1:
                return f[n-1]

```

Python

```

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [1] * n
        for i in range(1, m):
            for j in range(1, n):
                f[j] += f[j-1]
            return f[n-1]

```

Python

```

class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # memoizing dp[i][j] for i from 0 to m-1 and j from 0 to n-1 as initialization
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i-1][j] + dp[i][j-1]
        return dp[m-1][n-1]

```

C++

C++

Dynamic Programming • LeetCode • By Pattern

— 2541 —

LeetCode

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        // dp[i][j] -> the number of unique paths from (0, 0) to (i, j)
        vector<vector<int>> dp(m, vector<int>(n, 1));
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
};

```

C++

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n, 1);
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                dp[j] += dp[j-1];
        return dp[n-1];
    }
};

```

C++

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m, vector<int>(n, 1));
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
};

```

C++

Dynamic Programming • LeetCode • By Pattern

— 2542 —

LeetCode

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> f(n, 1);
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[j] += f[j-1];
        return f[n-1];
    }
};

```

C++

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> f(n, 1);
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[j] += f[j-1];
        return f[n-1];
    }
};

```

Java

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        // dp[i][j] -> the number of unique paths from (0, 0) to (i, j)
        int[][] dp = new int[m][n];
        Arrays.stream(dp).forEach(a -> Arrays.fill(a, 1));
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
}

```

Java

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                dp[j] += dp[j-1];
        return dp[n-1];
    }
}

```

Dynamic Programming • LeetCode • By Pattern

— 2543 —

LeetCode

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        int f = new int[n];
        f[0] = 1;
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[j] += f[j-1];
        return f[n-1];
    }
}

```

Java

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        int f = new int[n];
        Arrays.fill(f, 1);
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[j] += f[j-1];
        return f[n-1];
    }
}

```

Java

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        int f = new int[n];
        Arrays.fill(f, 1);
        for (int i = 1; i < m; ++i)
            for (int j = 1; j < n; ++j)
                f[j] += f[j-1];
        return f[n-1];
    }
}

```

JavaScript

JavaScript

Dynamic Programming • LeetCode • By Pattern

— 2544 —

LeetCode

```

//
* @param {number} n
* @param {number} m
* @param {number} k
*/
var uniquePaths = function(n, m) {
    const f = Array(n)
    f.fill(0)
    f[0][0] = 1;
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (i === 0) {
                f[i][j] == f[i - 1][j];
            }
            if (j === 0) {
                f[i][j] == f[i][j - 1];
            }
        }
    }
    return f[m - 1][n - 1];
};

// TypeScript
//
* @param {number} n
* @param {number} m
* @param {number} k
*/
var uniquePaths = function(n, m) {
    const f = Array(n).fill(1);
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            f[i][j] = f[i][j - 1];
        }
    }
    return f[m - 1];
};

TypeScript
TypeScript
```

```

function uniquePaths(n: number, m: number): number {
    const f: number[][] = Array(n)
    f.fill(0)
    f[0][0] = 1;
    for (let i = 0; i < m; ++i) {
        for (let j = 0; j < n; ++j) {
            if (i === 0) {
                f[i][j] == f[i - 1][j];
            }
            if (j === 0) {
                f[i][j] == f[i][j - 1];
            }
        }
    }
    return f[m - 1][n - 1];
}

TypeScript
TypeScript

function uniquePaths(n: number, m: number): number {
    const f: number[][] = Array(n).fill(1);
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            f[i][j] = f[i][j - 1];
        }
    }
    return f[m - 1];
}

TypeScript
TypeScript

function uniquePaths(n: number, m: number): number {
    const f: number[][] = Array(n).fill(1);
    for (let i = 1; i < m; ++i) {
        for (let j = 1; j < n; ++j) {
            f[i][j] = f[i][j - 1];
        }
    }
    return f[m - 1];
}

Go
Go
```

```

func uniquePaths(m int, n int) int {
    f := make([][]int, m)
    for i := range f {
        f[i] = make([]int, n)
    }
    f[0][0] = 1
    for i := 0; i < m; i++ {
        for j := 0; j < n; j++ {
            if i == 0 {
                f[i][j] == f[i-1][j]
            }
            if j == 0 {
                f[i][j] == f[i][j-1]
            }
        }
    }
    return f[m-1][n-1]
}

Go
func uniquePaths(m int, n int) int {
    f := make([][]int, m)
    for i := range f {
        f[i] = 1
    }
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            f[i][j] = f[i][j-1]
        }
    }
    return f[m-1]
}

Go
func uniquePaths(m int, n int) int {
    f := make([][]int, m)
    for i := range f {
        f[i] = 1
    }
    for i := 1; i < m; i++ {
        for j := 1; j < n; j++ {
            f[i][j] = f[i][j-1]
        }
    }
    return f[m-1]
}

Rust
Rust
```

```

impl Solution {
    pub fn unique_paths(m: i32, n: i32) -> i32 {
        let (m, n) = (m as usize, n as usize);
        let mut f = vec![1; m];
        for i in 1..m {
            for j in 1..n {
                f[j] = f[j - 1];
            }
        }
        f[n - 1]
    }
}

Rust
impl Solution {
    pub fn unique_paths(m: i32, n: i32) -> i32 {
        let (m, n) = (m as usize, n as usize);
        let mut f = vec![1; m];
        for i in 1..m {
            for j in 1..n {
                f[j] = f[j - 1];
            }
        }
        f[n - 1]
    }
}

[" "] Dynamic Programming — Pattern Solution (matched from community answers)
Python
# Python solution using Dynamic Programming

def uniquePaths(m, n):
    # Initialize a 2D list with 1's for the first row and first column
    dp = [[1] * n for _ in range(m)]

    # Fill in the DP table
    for i in range(1, m):
        for j in range(1, n):
            # The number of ways to reach this cell is the sum of the ways from the cell above and the cell on the left
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    # Return the value in the bottom-right cell
    return dp[m-1][n-1]

# Example usage:
print(uniquePaths(3, 7))

#72 MEDIUM
Edit Distance
LeetCode · Implement · WalkCC · LeetCode · Editorial
String · Dynamic Programming
Lints Denny Zhang

Dynamic Programming · LeetCode · By Pattern — 2548 —
LeetCode
```

Problem:
Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

Input: word1 = "horse", word2 = "ros"

Output: 3

Explanation:

horse -> rorse (replace 'h' with 'r')

rorse -> rose (remove 'r')

rose -> ros (remove 'e')

Example 2:

Input: word1 = "intention", word2 = "execution"

Output: 5

Explanation:

intention -> inention (remove 't')

inention -> enention (replace 'i' with 'e')

enention -> exention (replace 'n' with 'x')

exention -> exection (replace 'n' with 'c')

exection -> execution (insert 'u')

Constraints:

- 0 <= word1.length, word2.length <= 500
- word1 and word2 consist of lowercase English letters.

>> Brute Force (Dynamic Programming) Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        # Brute Force O(n^3) - recursion without memo; try insert/delete/replace
        def dp(i, j):
            if i == len(word1): return len(word2) - j
            if j == len(word2): return len(word1) - i
            if word2[i] == word1[j]:
                return dp(i + 1, j + 1)
            return 1 + min(dp(i + 1, j), # delete
                           dp(i, j + 1), # insert
                           dp(i + 1, j + 1)) # replace
        return dp(0, 0)

Python
Python
```

```

# Python solution to compute the Edit Distance
def minDistance(word1: str, word2: str) -> int:
    m = len(word1)
    n = len(word2)

    # Initialize the DP table with size (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases: transform word1[0:i] to empty word2 requires i deletions
    for i in range(m + 1):
        dp[i][0] = i

    # Base cases: transform empty word1 to word2[0:j] requires j insertions
    for j in range(n + 1):
        dp[0][j] = j

    # Fill up the dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If the characters are the same, no new operation is needed.
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                # consider insertion, deletion, and replacement
                dp[i][j] = 1 + min(dp[i][j - 1], # insert
                                   dp[i - 1][j], # delete
                                   dp[i - 1][j - 1]) # replace

    return dp[m][n]

# Example usage:
print(minDistance("horse", "ros"))

Python
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m = len(word1)
        n = len(word2)
        # dp[i][j] = the minimum number of operations to convert word1[0:i] to word2[0:j]
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            dp[i][0] = i

            for j in range(1, n + 1):
                dp[i][j] = 1 + min(
                    dp[i - 1][j], # delete
                    dp[i][j - 1], # insert
                    dp[i - 1][j - 1] if word1[i - 1] == word2[j - 1] else 0, # replace
                )

        return dp[m][n]

Python
```

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        n, m = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, m + 1):
            f[0][j] = j
        for i in enumerate(word1, 1):
            f[i][0] = i
            for j, b in enumerate(word2, 1):
                if a == b:
                    f[i][j] = f[i - 1][j - 1]
                else:
                    f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1
        return f[m][n]

```

Python

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        n, m = len(word1), len(word2)
        dp = [[0] * (n + 1) for i in range(m + 1)]
        for i in range(m + 1):
            dp[i][0] = i
            for j in range(n + 1):
                dp[i][j] = 1
            for i in range(m, n + 1):
                if word1[i - 1] == word2[i - 1]:
                    dp[i][i] = dp[i - 1][i - 1]
                else:
                    dp[i][i] = min(dp[i - 1][i], dp[i - 1][i - 1], dp[i - 1][i - 1] + 1)
        return dp[m][n]

```

#####

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(n + 1)]

        for j in range(1, n + 1):
            f[0][j] = j

        for i in range(1, m + 1):
            f[i][0] = i
            # This loop from above solution, but not good for readability
            if m == n:
                f[i][i] = f[i - 1][i - 1]
            else:
                f[i][i] = min(f[i - 1][i], f[i][i - 1], f[i - 1][i - 1]) + 1

        return f[m][n]

```

C++

C++

Dynamic Programming · LeetMastery — By Pattern — 2551 —

LeetMastery

```

class Solution {
public:
    int minDistance(string word1, string word2) {
        const int n = word1.length();
        const int m = word2.length();
        // dp[i][j] = the minimum number of operations to convert word1[0..i] to word2[0..j]
        vector<vector<int>> dp(n + 1, vector<int>(m + 1));

        for (int i = 1; i <= n; ++i)
            dp[i][0] = i;

        for (int j = 1; j <= m; ++j)
            dp[0][j] = j;

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j)
                dp[i][j] = min(word1[i] == word2[j] ? 1 :
                    dp[i - 1][j] + 1, dp[i - 1][j - 1] + 1) + 1;

        return dp[n][m];
    }
};

```

Java

Java

```

class Solution {
public:
    int minDistance(string word1, string word2) {
        int m = word1.length(), n = word2.length();
        int f[m+1][n+1];
        for (int i = 0; i == m, n; i++) {
            f[i][0] = i;
        }
        for (int i = 1; i <= m; i++) {
            f[i][1] = 0;
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    f[i][j] = f[i-1][j-1];
                } else {
                    f[i][j] = Math.min(f[i-1][j], Math.min(f[i][j-1], f[i-1][j-1]) + 1);
                }
            }
        }
        return f[m][n];
    }
}

```

Java

Dynamic Programming · LeetMastery — By Pattern — 2552 —

LeetMastery

```

class Solution {
public:
    int minDistance(String word1, String word2) {
        int n = word1.length(), m = word2.length();
        int dp[n+1][m+1];
        for (int i = 0; i == n; ++i)
            for (int j = 0; j == m; ++j)
                dp[i][j] = -1;
        return f(0, 0, n, m, dp);
    }
    int f(int i, int j, int n, int m, int dp[]) {
        if (i == n) return m - j;
        if (j == m) return n - i;
        if (dp[i][j] != -1) return dp[i][j];
        int a = 1 + f(i+1, j, n, m, dp);
        int b = 1 + f(i, j+1, n, m, dp);
        int c = 0;
        if (word1[i] == word2[j]) c = f(i+1, j+1, n, m, dp);
        dp[i][j] = min(a, min(b, c));
        return dp[i][j];
    }
};

```

JavaScript

```

n =
    @param {string[]} word1
    @param {string[]} word2
    @return {number}

var minDistance = function(word1, word2) {
    const n = word1.length;
    const m = word2.length;
    const f = Array(n + 1)
        .fill(0);
    for (let i = 0; i <= m; ++i) {
        f[i] = i;
    }
    for (let i = 1; i <= m; ++i) {
        f[i] = 1;
        for (let j = 1; j <= n; ++j) {
            if (word1[j - 1] == word2[i - 1]) {
                f[i][j] = f[i - 1][j - 1];
            } else {
                f[i][j] = Math.min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1;
            }
        }
    }
    return f[m][n];
};

```

JavaScript

Dynamic Programming · LeetMastery — By Pattern — 2553 —

LeetMastery

```

n =
    sparse(1:range(word1)
          1:range(1:range(word2)
          1:range(number)
          )
)

var minDistance = function (word1, word2) {
    n = word1.length;
    const m = word2.length;
    const f = Array(n + 1)
    for (let i = 0; i <= m; i++) {
        f[i] = i;
    }
    for (let i = 1; i <= n; i++) {
        f[i][0] = i;
        for (let j = 1; j <= m; j++) {
            if (word1[i - 1] === word2[j - 1]) {
                f[i][j] = f[i - 1][j - 1];
            } else {
                f[i][j] = Math.min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1;
            }
        }
    }
    return f[n][m];
};

```

TypeScript

```

TypeScript

function midInsertion(words: string, word2: string): number {
    const n = word1.length;
    const m = word2.length;
    const i = word1[0] === word2[0] ? 1 : 0;
    for (let j = 1; j <= m; ++j) {
        f[i][j] = f[i][j-1] + 1;
    }
    for (let k = 1; k <= n; ++k) {
        f[k][0] = k;
        for (let l = 1; l <= m; ++l) {
            if (word1[k-1] === word2[l-1]) {
                f[k][l] = f[k-1][l-1];
            } else {
                f[k][l] = Math.max(f[k-1][l], f[k][l-1], f[k-1][l-1]) + 1;
            }
        }
    }
    return f[n][m];
}

```

TypeScript

Dynamic Programming · LeetMastery — By Pattern — 2554 —

LeetMastery

```
function minDiffPairs(nums2: string[], word2: string): number {
    const n = word2.length;
    const m = word22.length;
    const f: number[][] = Array(n + 1)
        .fill(0)
        .map(() => Array(m + 1).fill(0));
    for (let i = 0; i < n; ++i) {
        f[i][0] = 1;
    }
    for (let i = 1; i <= n; ++i) {
        f[i][0] = 1;
        for (let j = 1; j <= m; ++j) {
            if (word2[i - 1] == word22[j - 1]) {
                f[i][j] = f[i - 1][j - 1];
            } else {
                f[i][j] = Math.min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1;
            }
        }
    }
    return f[n][m];
}
```


Go

```
func minDistance(word1 string, word2 string) int {
    m, n := len(word1), len(word2)
    f := make([][]int, m+1)
    for i := range f {
        f[i] = make([]int, n+1)
    }
    for j := 1; j <= n; j++ {
        f[0][j] = j
    }
    for i := 1; i <= m; i++ {
        f[i][0] = i
        for j := 1; j <= n; j++ {
            if word1[i-1] == word2[j-1] {
                f[i][j] = f[i-1][j-1]
            } else {
                f[i][j] = min(f[i-1][j], f[i][j-1], f[i-1][j-1]) + 1
            }
        }
    }
    return f[m][n]
}
```

Go

Dynamic Programming · LeetCode — By Pattern — 2555 —

LeetMastery

```

func minDistance(word1 string, word2 string) int {
    m, n := len(word1), len(word2)
    f := make([][]int, m+1)
    for i := range f {
        f[i] = make([]int, n+1)
    }
    for j := 1; j <= n; j++ {
        f[0][j] = j
    }
    for i := 1; i <= m; i++ {
        f[i][0] = i
        for j := 1; j <= n; j++ {
            if word1[i-1] == word2[j-1] {
                f[i][j] = f[i-1][j-1]
            } else {
                f[i][j] = min(f[i-1][j], f[i][j-1], f[i-1][j-1]) + 1
            }
        }
    }
    return f[m][n]
}

```

[*] Dynamic Programming — Pattern Solution (matched from community answers)

Python

```
# Python operation to compute the 2DZ Surface
def minDistance(word1, word2):
    n = len(word1)
    m = len(word2)

    # Initialize the DP table with dim (n+1) x (m+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases: transform word[0:i] to empty word requires i deletions
    for i in range(n + 1):
        dp[i][0] = i

    # Base cases: transform empty word to word[0:j] requires j insertions
    for j in range(m + 1):
        dp[0][j] = j

    # Fill up the dp table
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            # If the last character of the same, no new operation is needed.
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                # Consider insertion, deletion, and insert
                dp[i][j] = 1 + min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j - 1])

    return dp[n][m]

# Sample usage
print(minDistance("horse", "ros"))
```

Dynamic Programming · LeetMastery — By Pattern — 2556 —

LeetMastery

Decode Ways

LeetCode · [SimplifyLeet](#) · [WuKCC](#) · [LeetCode](#) · [Editorial](#)
 String · [Dynamic Programming](#)
 Lists · [Grind 189](#)

Problem:

You have intercepted a secret message encoded as a string of numbers. The message is decoded via the following mapping:

"1" -> 'A' "2" -> 'B' ... "25" -> 'Y' "26" -> 'Z'

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)

- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode. Given a string s containing only digits, return the number of ways to decode it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a 32-bit integer.

Example 1:

Input: s = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: s = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: s = "06"

Output: 0

Explanation:

"06" cannot be mapped to "f" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- 1 <= s.length <= 100
- s contains only digits and may contain leading zero(s).

Python

Python

```
# Python solution using dynamic programming
def num_decodings(s: str) -> int:
    # If the string starts with '0', it's not decodable
    if not s or s[0] == '0':
        return 0

    # dp array where dp[i] represents the number of ways to decode s[0:i]
    n = len(s)
    dp = [0] * (n + 1)

    # Base case: an empty string has one decoding
    dp[0] = 1

    for i in range(1, n + 1):
        # Check for single digit decoding (s[i-1]) is valid if not '0'
        if s[i-1] != '0':
            dp[i] = dp[i-1]

        # Check for two-digit decoding (if i>=2, s[i-2:i]) forms a valid number between 10 and 26)
        if i >= 2 and s[i-2:i] != '00':
            two_digit = int(s[i-2:i])
            if 10 <= two_digit <= 26:
                dp[i] += dp[i-2]

    return dp[n]

# Example usage:
print(num_decodings("12")) # Expected output: 2
```

Python

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # dp[i] is the number of ways to decode s[0:i]
        dp = [0] * (n + 1)

        def isValid(s: str, b: int) -> bool:
            if b:
                return s == '0' or s == '2' and b < '7'
            return s != '0'

        if isValid(s[0:1]):
            dp[0] = 1

        for i in reversed(range(n - 1)):
            if isValid(s[i:i+1]):
                dp[i] = dp[i+1]
            if isValid(s[i:i+2]) and s[i:i+2] != '00':
                dp[i] += dp[i+2]

        return dp[0]
```

Python

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        f = [1] + [0] * n
        for i, c in enumerate(s, 1):
            if c != '0':
                f[i] = f[i-1]
            if i >= 2 and s[i-2:i] != '00':
                f[i] += f[i-2]
        return f[n]

Python

class Solution:
    def numDecodings(self, s: str) -> int:
        f, g = 0, 1
        for i, c in enumerate(s, 1):
            h = g if c != '0' else 0
            if i >= 2 and s[i-2:i] != '00' and int(s[i-2:i]) <= 26:
                h += f
            f, g = g, h
        return g

Python

s = "abcdef"
[[i,c] for i,c in enumerate(s, 1)]
[[1,'a'], [2,'b'], [3,'c'], [4,'d'], [5,'e'], [6,'f']]

class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        f, g = 0, 1
        for i, c in enumerate(s, 1):
            h = g if c != '0' else 0
            if i >= 2 and s[i-2:i] != '00' and int(s[i-2:i]) <= 26:
                h += f
            f, g = g, h
        return g

class Solution:
    def numDecodings(self, s: str) -> int:
        if len(s) == 0:
            return 0
        length = len(s)
```

```
dp = [0] * (length + 1) # dp[i] == at index i, its decode ways
dp[length] = 1 # initiator, just make for loop flow working
dp[length - 1] = 0 if s[length - 1] == '0' else 1 # assumption is starting at 1, so starting at 0 is not accurate

# start at 'len-2'
# or else below 'dp[length-2]' will not of index boundary
for i in range(length - 2, -1, -1):
    if s[i] == '0':
        continue
    tem = int(s[i:i+2])
    if tem <= 26:
        dp[i] = dp[i+1]
    else:
        dp[i] = dp[i+1] + dp[i+2]
    return dp[0]
```

```
# optimized from above solution, no dp array
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # a, b, c = 0, 1, 0; c: count for current index i
        for i in range(1, n + 1):
            c = 0
            if s[i-1] != '0':
                c += b
            if i >= 2 and s[i-2:i] != '00' and (int(s[i-2:i]) + 10 + int(s[i-1]) <= 26):
                c += a
            a, b = b, c
        return c

class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        dp = [0] * (n + 1)
        dp[0] = 1
        for i in range(1, n + 1):
            if s[i-1] != '0':
                dp[i] = dp[i-1]
            if i >= 2 and s[i-2:i] != '00' and (int(s[i-2:i]) + 10 + int(s[i-1]) <= 26):
                dp[i] += dp[i-2]
        return dp[n]
```

C++

C++

```
class Solution {
public:
    int numDecodings(string s) {
        const int n = s.length();
        vector<int> dp(n + 1);
        dp[n] = 1; // base case: empty string has 1 way
        for (int i = n - 1; i >= 0; --i) {
            if (isValid(s[i]))
                dp[i] = dp[i + 1];
            if (i >= 1 and isValid(s[i-1:i+1]))
                dp[i] += dp[i + 2];
        }
        return dp[0];
    }

private:
    bool isValid(char c) {
        return c != '0';
    }

    bool isValid(char c1, char c2) {
        return c1 == '1' || c1 == '2' && c2 < '7';
    }
};
```

Java

Java

```
class Solution {
    public int numDecodings(String s) {
        final int n = s.length();
        // dp[i] is the number of ways to decode s[0:i]
        int[] dp = new int[n + 1];
        dp[n] = 1; // base case: empty string has 1 way
        dp[n-1] = isValid(s.charAt(n-1)) ? 1 : 0;
        for (int i = n - 2; i >= 0; --i) {
            if (isValid(s.charAt(i)))
                dp[i] = dp[i + 1];
            if (isValid(s.charAt(i), s.charAt(i + 1)))
                dp[i] += dp[i + 2];
        }
        return dp[0];
    }

    private boolean isValid(char c) {
        return c != '0';
    }

    private boolean isValid(char c1, char c2) {
        return c1 == '1' || c1 == '2' && c2 < '7';
    }
}
```

```
}

Java

class Solution {
    public int numDecodings(String s) {
        int n = s.length();
        int[] f = new int[n + 1];
        f[0] = 1;
        for (int i = 1; i <= n; ++i) {
            if (s.charAt(i-1) != '0') {
                f[i] = f[i-1];
            }
            if (i >= 2 && s.charAt(i-2) != '0' && Integer.valueOf(s.substring(i-2, i)) <= 26) {
                f[i] += f[i-2];
            }
        }
        return f[n];
    }
}

Java

public class Solution {
    public int numDecodings(string s) {
        int n = s.length();
        int[] f = new int[n + 1];
        f[0] = 1;
        for (int i = 1; i <= n; ++i) {
            if (s[i-1] != '0') {
                f[i] = f[i-1];
            }
            if (i >= 2 && s[i-2:i] != "00" && Integer.valueOf(s.substring(i-2, i)) <= 26) {
                f[i] += f[i-2];
            }
        }
        return f[n];
    }
}
```

```
Java

class Solution {
    public int numDecodings(String s) {
        int n = s.length();
        int f = 0, g = 1;
        for (int i = 1; i <= n; ++i) {
            if (i >= 2 && s.charAt(i-2) != '0' && Integer.valueOf(s.substring(i-2, i)) <= 26) {
                h = f;
            }
            f = g;
            g = h;
        }
        return g;
    }
}
```

Java

```

public class Solution {
    public int NumDecodings(string s) {
        int n = s.Length;
        int f = 0, g = 1;
        for (int i = 2; i <= n; i++) {
            int h = s[i-1] < '0' ? 0 : 1;
            int k = s[i-1] < '0' ? 0 : 1;
            if (i > 2 && (s[i-2] == '1' || (s[i-2] == '2' && s[i-1] < '0'))) {
                h += f;
            }
            f = g;
            g = h;
        }
        return g;
    }
}

```

TypeScript

```

function numDecodings(s: string): number {
    const n = s.length;
    const f: number[] = new Array(n + 1).fill(0);
    f[0] = 1;
    for (let i = 1; i <= n; i++) {
        if (s[i-1] < '0') {
            f[i] = f[i-1];
        }
        if (i > 1 && (s[i-2] == '1' || (s[i-2] == '2' && s[i-1] < '0'))) {
            f[i] += f[i-2];
        }
    }
    return f[n];
}

```

TypeScript

```

function numDecodings(s: string): number {
    const n = s.length;
    let [f, g] = [1, 1];
    for (let i = 2; i <= n; i++) {
        let h = s[i-1] < '0' ? 0 : 1;
        let k = s[i-1] < '0' ? 0 : 1;
        if (i > 2 && (s[i-2] == '1' || (s[i-2] == '2' && s[i-1] < '0'))) {
            h += f;
        }
        [f, g] = [g, h];
    }
    return g;
}

```

TypeScript

```

function numDecodings(s: string): number {
    const n = s.length;
    let [f, g] = [1, 1];
    for (let i = 2; i <= n; i++) {
        let h = s[i-1] < '0' ? 0 : 1;
        let k = s[i-1] < '0' ? 0 : 1;
        if (i > 2 && (s[i-2] == '1' || (s[i-2] == '2' && s[i-1] < '0'))) {
            h += f;
        }
        [f, g] = [g, h];
    }
    return g;
}

```

Dynamic Programming - LeetCode - By Pattern

— 2563 —

LeetCode

```

function numDecodings(s: string): number {
    const n = s.length;
    let [f, g] = [1, 1];
    for (let i = 2; i <= n; i++) {
        let h = s[i-1] < '0' ? 0 : 1;
        let k = s[i-1] < '0' ? 0 : 1;
        if (i > 2 && (s[i-2] == '1' || (s[i-2] == '2' && s[i-1] < '0'))) {
            h += f;
        }
        [f, g] = [g, h];
    }
    return g;
}

```

Go

Go

```

func numDecodings(s string) int {
    n := len(s)
    f := make([]int, n+1)
    f[0] = 1
    for i := 1; i <= n; i++ {
        if s[i-1] < '0' {
            f[i] = f[i-1]
        }
        if i > 1 && (s[i-2] == '1' || (s[i-2] == '2' && s[i-1] < '0')) {
            f[i] += f[i-2]
        }
    }
    return f[n]
}

```

[1] Dynamic Programming - Pattern Solution (matched from community answers)

Python

Dynamic Programming - LeetCode - By Pattern

— 2564 —

LeetCode

```

# Python solution using dynamic programming
def num_decodings(s: str) -> int:
    # If the input string starts with '0', it's not decodable
    if not s or s[0] == '0':
        return 0

    # dp array where dp[i] represents the number of ways to decode s[0:i]
    n = len(s)
    dp = [0] * (n + 1)

    # Base case: an empty string has one decoding
    dp[0] = 1

    for i in range(1, n + 1):
        # Check how many digits decoding (s[i-1]) is valid (if not '0')
        if s[i-1] != '0':
            dp[i] = dp[i-1]
        # Check how many digits decoding (if i>2, s[i-2:i]) forms a valid number between 10 and 26
        if i > 2 and s[i-2] != '0':
            two_digits = int(s[i-2:i])
            if 10 <= two_digits <= 26:
                dp[i] += dp[i-2]

    return dp[n]

# Example usage:
print(num_decodings("12")) # Expected output: 2

```

#152

MEDIUM

Maximum Product Subarray

LeetCode - SimplyList - WalleCC - LeetCode - Editorial

Array - Dynamic Programming

Like · 100

Problem:

Given an integer array `nums`, find a subarray that has the largest product, and return the product.

The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

Input: `nums = [2,3,-2,4]`

Output: 6

Explanation: [2,3] has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`

Output: 0

Explanation: The result cannot be 2, because [-2,-1] is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 \times 10^4$
- $-10 \leq \text{nums}[i] \leq 10$

Dynamic Programming - LeetCode - By Pattern

— 2565 —

LeetCode

- The product of any subarray of `nums` is guaranteed to fit in a 32-bit integer.

>> Brute Force (Dynamic Programming) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        # Brute Force O(n^2) - compute product of every contiguous subarray
        res = nums[0]
        for i in range(len(nums)):
            prod = 1
            for j in range(i, len(nums)):
                prod *= nums[j]
                res = max(res, prod)
        return res

```

Python

Python

```

def maxProduct(nums):
    # If the input array is empty, return 0 (though problem guarantees at least one element)
    if not nums:
        return 0
    # Initialize variables: minimum product ending here and the result.
    max_ending_here = nums[0]
    min_ending_here = nums[0]
    global_max = nums[0]

    # Iterate over the array starting from the second element.
    for num in nums[1:]:
        # If the current number is negative, swap max and min.
        if num < 0:
            max_ending_here, min_ending_here = min_ending_here, max_ending_here
        # Update the max product of all the current slices.
        max_ending_here = max(num, max_ending_here * num)
        # Update the min product of all the current slices.
        min_ending_here = min(num, min_ending_here * num)
        # Update the global maximum product.
        global_max = max(global_max, max_ending_here)

    return global_max

```

Python

Dynamic Programming - LeetCode - By Pattern

— 2566 —

LeetCode

```

class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = nums[0]
        dpMin = nums[0] # the minimum so far
        dpMax = nums[0] # the maximum so far

        for i in range(1, len(nums)):
            num = nums[i]
            prevMin = dpMin * dpMax[i-1]
            prevMax = dpMax * dpMax[i-1]
            if num < 0:
                dpMin = min(prevMin, num, num)
                dpMax = max(prevMax, num, num)
            else:
                dpMin = min(prevMin, num, num)
                dpMax = max(prevMax, num, num)

            ans = max(ans, dpMax)

        return ans

```

Python

```

class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = min(f, x, gg * x)
            g = max(g, x, ff * x)
            ans = max(ans, f)
        return ans

```

Python

```

class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = min(f, x, gg * x)
            g = max(g, x, ff * x)
            ans = max(ans, f)
        return ans

```

C++

C++

Dynamic Programming - LeetCode - By Pattern

— 2567 —

LeetCode

```

class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int ans = nums[0];
        int dpMin = nums[0]; // the minimum so far
        int dpMax = nums[0]; // the maximum so far

        for (int i = 1; i < nums.size(); i++) {
            const int num = nums[i];
            const int prevMin = dpMin; // dpMin[i-1]
            const int prevMax = dpMax; // dpMax[i-1]

            if (num < 0) {
                dpMin = min(prevMin * num, num);
                dpMax = max(prevMax * num, num);
            } else {
                dpMin = min(prevMin * num, num);
                dpMax = max(prevMax * num, num);
            }
            ans = max(ans, dpMax);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int f = nums[0], g = nums[0], ans = nums[0];
        for (int i = 1; i < nums.size(); i++) {
            int ff = f, gg = g;
            f = min(nums[i], ff * nums[i], gg * nums[i]);
            g = max(nums[i], ff * nums[i], gg * nums[i]);
            ans = max(ans, f);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int f = nums[0], g = nums[0], ans = nums[0];
        for (int i = 1; i < nums.size(); i++) {
            int ff = f, gg = g;
            f = min(nums[i], ff * nums[i], gg * nums[i]);
            g = max(nums[i], ff * nums[i], gg * nums[i]);
            ans = max(ans, f);
        }
        return ans;
    }
};

```

Java

Java

Dynamic Programming - LeetCode - By Pattern

— 2568 —

LeetCode


```

class Solution {
public: int maxProduct(int[] nums) {
    let ans = nums[0];
    let dpMax = nums[0]; // the maxAns so far
    let dpMin = nums[0]; // the minAns so far

    for (let i = 1; i < nums.length; ++i) {
        final let num = nums[i];
        final let prevMax = dpMax; // dpMax[i-1]
        final let prevMin = dpMin; // dpMin[i-1]
        if (num < 0) {
            dpMin = Math.min(prevMax * num, num);
            dpMax = Math.max(prevMin * num, num);
        } else {
            dpMin = Math.min(prevMin * num, num);
            dpMax = Math.max(prevMax * num, num);
        }
        ans = Math.max(ans, dpMax);
    }

    return ans;
}
}

Java

class Solution {
public: int maxProduct(int[] nums) {
    let f = nums[0]; g = nums[0]; ans = nums[0];
    for (let i = 1; i < nums.length; ++i) {
        let ff = f; gg = g;
        f = Math.max(nums[i], Math.max(ff * nums[i], gg * nums[i]));
        g = Math.min(nums[i], Math.min(ff * nums[i], gg * nums[i]));
        ans = Math.max(ans, f);
    }

    return ans;
}
}

Java

public class Solution {
public: int MaxProduct(int[] nums) {
    let f = nums[0]; g = nums[0]; ans = nums[0];
    for (let i = 1; i < nums.length; ++i) {
        let ff = f; gg = g;
        f = Math.Max(nums[i], Math.Max(ff * nums[i], gg * nums[i]));
        g = Math.Min(nums[i], Math.Min(ff * nums[i], gg * nums[i]));
        ans = Math.Max(ans, f);
    }

    return ans;
}
}

Java

```

```

class Solution {
public: int maxProduct(int[] nums) {
    let f = nums[0]; g = nums[0]; ans = nums[0];
    for (let i = 1; i < nums.length; ++i) {
        let ff = f; gg = g;
        f = Math.max(nums[i], Math.max(ff * nums[i], gg * nums[i]));
        g = Math.min(nums[i], Math.min(ff * nums[i], gg * nums[i]));
        ans = Math.max(ans, f);
    }

    return ans;
}
}

Java

public class Solution {
public: int MaxProduct(int[] nums) {
    let f = nums[0]; g = nums[0]; ans = nums[0];
    for (let i = 1; i < nums.length; ++i) {
        let ff = f; gg = g;
        f = Math.Max(nums[i], Math.Max(ff * nums[i], gg * nums[i]));
        g = Math.Min(nums[i], Math.Min(ff * nums[i], gg * nums[i]));
        ans = Math.Max(ans, f);
    }

    return ans;
}
}

JavaScript

JavaScript

/*
 * @param {number[]} nums
 * @return {number}
 */
var maxProduct = function(nums) {
    let [f, g, ans] = [nums[0], nums[0], nums[0]];
    for (let i = 1; i < nums.length; ++i) {
        let ff = f; gg = g;
        const [ff, gg] = [f, g];
        f = Math.max(nums[i], Math.max(ff * nums[i], gg * nums[i]));
        g = Math.min(nums[i], Math.min(ff * nums[i], gg * nums[i]));
        ans = Math.max(ans, f);
    }

    return ans;
};

```

```

/*
 * @param {number[]} nums
 * @return {number}
 */
var maxProduct = function(nums) {
    let [f, g, ans] = [nums[0], nums[0], nums[0]];
    for (let i = 1; i < nums.length; ++i) {
        const [ff, gg] = [f, g];
        f = Math.max(nums[i], ff * nums[i], gg * nums[i]);
        g = Math.min(nums[i], ff * nums[i], gg * nums[i]);
        ans = Math.max(ans, f);
    }

    return ans;
};

TypeScript

TypeScript

function maxProduct(nums: number[]): number {
    let [f, g, ans] = [nums[0], nums[0], nums[0]];
    for (let i = 1; i < nums.length; ++i) {
        const [ff, gg] = [f, g];
        f = Math.max(nums[i], ff * nums[i], gg * nums[i]);
        g = Math.min(nums[i], ff * nums[i], gg * nums[i]);
        ans = Math.max(ans, f);
    }

    return ans;
}

TypeScript

function maxProduct(nums: number[]): number {
    let [f, g, ans] = [nums[0], nums[0], nums[0]];
    for (let i = 1; i < nums.length; ++i) {
        const [ff, gg] = [f, g];
        f = Math.max(nums[i], ff * nums[i], gg * nums[i]);
        g = Math.min(nums[i], ff * nums[i], gg * nums[i]);
        ans = Math.max(ans, f);
    }

    return ans;
}

Go

Go

func maxProduct(nums []int) int {
    f, g, ans := nums[0], nums[0], nums[0]
    for i, v := range nums[1:] {
        ff, gg := f, g
        f = max(v, max(ff*v, gg*v))
        g = min(v, min(ff*v, gg*v))
        ans = max(ans, f)
    }

    return ans
}

Go

```

```

func maxProduct(nums []int) int {
    f, g, ans := nums[0], nums[0], nums[0]
    for i, v := range nums[1:] {
        ff, gg := f, g
        f = max(v, max(ff*v, gg*v))
        g = min(v, min(ff*v, gg*v))
        ans = max(ans, f)
    }

    return ans
}

Rust

Rust

impl Solution {
    pub fn max_product(nums: Vec<i32>) -> i32 {
        let mut f = nums[0];
        let mut g = nums[0];
        let mut ans = nums[0];
        for &v in nums.iter().skip(1) {
            let [ff, gg] = [f, g];
            f = v.max(v * ff).max(v * gg);
            g = v.min(v * ff).min(v * gg);
            ans = ans.max(f);
        }

        ans
    }
}

Rust

impl Solution {
    pub fn max_product(nums: Vec<i32>) -> i32 {
        let mut f = nums[0];
        let mut g = nums[0];
        let mut ans = nums[0];
        for &v in nums.iter().skip(1) {
            let [ff, gg] = [f, g];
            f = v.max(v * ff).max(v * gg);
            g = v.min(v * ff).min(v * gg);
            ans = ans.max(f);
        }

        ans
    }
}

Python

```

```

def maxProduct(nums):
    # If the input array is empty, return 0 (though problem guarantees at least one element)
    if not nums:
        return 0
    # Initialize maxAns, minimum product ending here and the result.
    max_ending_here = nums[0]
    min_ending_here = nums[0]
    global_max = nums[0]

    # Iterate over the array starting from the second element.
    for num in nums[1:]:
        # If the current number is negative, swap max and min.
        if num < 0:
            max_ending_here, min_ending_here = min_ending_here, max_ending_here
        # Update the max element at the current index.
        max_ending_here = max(num, max_ending_here * num)
        # Update the min element at the current index.
        min_ending_here = min(num, min_ending_here * num)
        # Update the global max element.
        global_max = max(global_max, max_ending_here)

    return global_max

# Example usage:
# print(maxProduct([2, 3, -2, 4])) # Output: 6

C++

#include <iostream>
#include <algorithm>
using namespace std;

int maxProduct(vector<int>& nums) {
    // Initialize the maximum product, minimum product, and global maximum with the first element.
    int maxEndingHere = nums[0];
    int minEndingHere = nums[0];
    int maxSoFar = nums[0];

    // Iterate through the vector starting from the second element.
    for (int i = 1; i < nums.size(); ++i) {
        // If num[i] is negative, swap maxEndingHere and minEndingHere.
        if (nums[i] < 0) {
            swap(maxEndingHere, minEndingHere);
        }
        // Update the maximum product ending here.
        maxEndingHere = max(nums[i], maxEndingHere * nums[i]);
        // Update the minimum product ending here.
        minEndingHere = min(nums[i], minEndingHere * nums[i]);
        // Update the global maximum product.
        maxSoFar = max(maxSoFar, maxEndingHere);
    }

    return maxSoFar;
}

// Example usage:
// vector<int> nums = {2, 3, -2, 4};
// int result = maxProduct(nums); // result should be 6

```

#198 **MEDIUM**
House Robber
[LeetCode](#) · [SolutionSet](#) · [WalkCC](#) · [LeetCode](#) · [Editorial](#)
Array · Dynamic Programming
Lists Grid 169

Problem:

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: `nums = [2,7,9,3,1]`
Output: 12
Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- 1 <= `nums.length` <= 100
- 0 <= `nums[i]` <= 400

Python

Python

```
# Definition of the Solution class
class Solution:
    def rob(self, nums: List[int]) -> int:
        # Edge case: if there are no houses, return 0
        if not nums:
            return 0

        # Initialize two variables to keep track of the max amounts
        prev1, prev2 = 0, 0

        # Iterate over each house's money
        for money in nums:
            # Calculate the maximum money that can be robbed if we consider the current house
            current = max(prev1, prev2 + money) # Either skip current house or rob it
            # Update variables: prev1 becomes previous best, and prev2 becomes current best
            prev1 = prev2
            prev2 = current

        # Return the maximum money that can be robbed
        return prev1
```

Python

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        if not nums:
            return 0
        if len(nums) == 1:
            return nums[0]

        # dp[i] := max money of robbing nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]
        dp[1] = max(nums[0], nums[1])

        for i in range(2, len(nums)):
            dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])

        return dp[-1]
```

Python

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        prev1 = 0 # dp[i - 1]
        prev2 = 0 # dp[i - 2]

        for num in nums:
            dp = max(prev1, prev2 + num)
            prev2 = prev1
            prev1 = dp

        return prev1
```

Python

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        def dfs(i: int) -> int:
            if i == len(nums):
                return 0
            return max(nums[i] + dfs(i + 2), dfs(i + 1))

        return dfs(0)
```

Python

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        f = [0] * (n + 1)
        f[0] = 0
        for i in range(1, n + 1):
            f[i] = max(f[i - 1], f[i - 2] + nums[i - 1])
        return f[n]
```

Python

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        f, g = 0, 0
        for i in nums:
            f, g = max(f, g), f + i
        return max(f, g)
```

Python

```
# Greedy
class Solution:
    def rob(self, nums: List[int]) -> int:
        not_rob, rob = 0, 0, sum(nums)
        for num in nums[1:]:
            # rob and check
            # eg: first robbed 9999, then following ones are just 3,3,3,3,3
            not_rob, rob = rob, max(not_rob, rob)
        return rob
```

#####

```
class Solution(object):
    def rob(self, nums):
        """
        :type nums: List[int]
        :rtype: int

        if len(nums) == 0:
            return 0
        if len(nums) == 1:
            return nums[0]
        dp = [0] * len(nums)
        dp[0] = nums[0]
        dp[1] = max(nums[0], nums[1])
```

```
for i in range(2, len(nums)):
    dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])
return dp[len(nums) - 1]
```

C++

```
class Solution {
public:
    int rob(vector<int>& nums) {
        if (nums.empty())
            return 0;
        if (nums.size() == 1)
            return nums[0];

        // dp[i] := the maximum money of robbing nums[0..i]
        vector<int> dp(nums.size());
        dp[0] = nums[0];
        dp[1] = max(nums[0], nums[1]);

        for (int i = 2; i < nums.size(); ++i)
            dp[i] = max(dp[i - 1], dp[i - 2] + nums[i]);

        return dp.back();
    }
};
```

C++

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int prev1 = 0, prev2 = 0;
        for (const int num : nums) {
            const int dp = max(prev1, prev2 + num);
            prev2 = prev1;
            prev1 = dp;
        }
        return prev1;
    }
};
```

C++

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int n = nums.size();
        int f[0] = 0, g[0] = 0;
        for (int i = 1; i <= n; ++i) {
            f[i] = max(f[i - 1], g[i - 1] + nums[i - 1]);
            g[i] = f[i - 1];
        }
        return f[n];
    }
};
```

C++

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int f = 0, g = 0;
        for (int i = 0; i < nums.size(); ++i) {
            g = f + nums[i];
            f = g;
        }
        return max(f, g);
    }
};
```

C++

```
// rob[i - 1] = max(dp[i - 1], dp[i - 2] + nums[i - 1]) // If we rob at house[i], we must skip house[i - 1]
// skip[i - 1] = max(dp[i - 1], dp[i - 2]) // If we skip house[i], we can pick the maximum from robbing
// skipping house[i - 1]
class Solution {
public:
```

```
int rob(vector<int>& nums) {
    int n = nums.size();
    int a = 0, b = nums[0];
    for (int i = 1; i <= n; ++i) {
        int c = max(nums[i] + a, b);
        a = b;
        b = c;
    }
    return b;
}
```

Java

Java

```
class Solution {
    public int rob(int[] nums) {
        final int n = nums.length;
        if (n == 0)
            return 0;
        if (n == 1)
            return nums[0];

        // dp[i] := the maximum money of robbing nums[0..i]
        int[] dp = new int[n];
        dp[0] = nums[0];
        dp[1] = Math.max(nums[0], nums[1]);

        for (int i = 2; i < n; ++i)
            dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);

        return dp[n - 1];
    }
}
```

Java

```
class Solution {
    public int rob(int[] nums) {
        int prev1 = 0, prev2 = 0;
        for (final int num : nums) {
            final int dp = Math.max(prev1, prev2 + num);
            prev2 = prev1;
            prev1 = dp;
        }
        return prev1;
    }
}
```

Java

```
class Solution {
    private Integer[] f;
    private int[] nums;

    public int rob(int[] nums) {
        this.nums = nums;
        f = new Integer[nums.length];
        return dfs(0);
    }

    private int dfs(int i) {
        if (i == nums.length)
            return 0;
        if (f[i] == null) {
            f[i] = Math.max(nums[i] + dfs(i + 2), dfs(i + 1));
            return f[i];
        }
    }
}
```

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        int f = 0, g = 0;
        for (int i = 1; i <= n; ++i) {
            f[i] = max(f[i - 1], g[i - 1] + nums[i - 1]);
            g[i] = f[i - 1];
        }
        return f[n];
    }
}
```

Java

```
class Solution {
    public int rob(int[] nums) {
        int f = 0, g = 0;
        for (int i = 0; i < nums.length; ++i) {
            g = f + nums[i];
            f = g;
        }
        return Math.max(f, g);
    }
}
```

Java

```
public class HouseRobber {

    class Solution {
        public int rob(int[] nums) {
            if (nums == null || nums.length == 0) {
                return 0;
            }

            // dp[i] means until i, max possible amount
            int[] dp = new int[nums.length + 1];
            dp[0] = 0;
            dp[1] = nums[0];

            for (int i = 2; i <= nums.length; i++) {
                // 2 cases: rob current house, not rob current
                dp[i] = Math.max(nums[i - 1] + dp[i - 2], dp[i - 1]);

                return dp[nums.length];
            }
        }
    }
}
```

```

//[[[[[[
class Solution {
public: int rob(vector<int>& nums) {
    int a = 0, b = nums[0];
    for (int i = 1; i < nums.length; ++i) {
        int c = Math.max(nums[i] + a, b);
        a = b;
        b = c;
    }
    return b;
}
}

TypeScript
TypeScript
function rob(nums: number[]): number {
    const n = nums.length;
    const f: number[] = Array(n).fill(-1);
    const dfs = (i: number): number => {
        if (i == 0) {
            return 0;
        }
        if (f[i] != 0) {
            f[i] = Math.max(nums[i] + dfs(i - 2), dfs(i - 1));
        }
        return f[i];
    };
    return dfs(0);
}

TypeScript
function rob(nums: number[]): number {
    let [f, g] = [0, 0];
    for (const n of nums) {
        [f, g] = [Math.max(f, g), f + n];
    }
    return Math.max(f, g);
}

TypeScript
function rob(nums: number[]): number {
    const dp = [0, 0];
    for (const num of nums) {
        [dp[0], dp[1]] = [dp[1], Math.max(dp[1], dp[0] + num)];
    }
    return dp[1];
}

Go
Go

```

```

func rob(nums []int) int {
    n := len(nums)
    f := make([]int, n)
    for i := range f {
        f[i] = -1
    }
    var dfs func(int) int
    dfs = func(i int) int {
        if i == n {
            return 0
        }
        if f[i] != 0 {
            f[i] = max(nums[i]+dfs(i+2), dfs(i+1))
        }
        return f[i]
    }
    return dfs(0)
}

Go
func rob(nums []int) int {
    n := len(nums)
    f := make([]int, n+1)
    f[0] = 0
    for i := 2; i <= n; i++ {
        f[i] = max(f[i-2], f[i-1]+nums[i-1])
    }
    return f[n]
}

Go
func rob(nums []int) int {
    f, g := 0, 0
    for _, n := range nums {
        f, g = max(f, g), f+n
    }
    return max(f, g)
}

Go
func rob(nums []int) int {
    a, b := 0, 0; num[0], len(nums)
    for i := 1; i <= n; i++ {
        a, b = b, max(nums[i] + a, b)
    }
    return b
}

func max(a, b int) int {
    if a > b {
        return a
    }
    return b
}

}

Rust

```

```

Rust
impl Solution {
    pub fn rob(nums: Vec<i32>) -> i32 {
        let dfs(i: usize, memo: &mut Vec<i32>, f: &mut Vec<i32>) -> i32 {
            if i == memo.len() {
                return 0;
            }
            if f[i] != 0 {
                f[i] = (memo[i] + dfs(i + 2, memo, f)).max(dfs(i + 1, memo, f));
            }
            f[i]
        }

        let n = memo.len();
        let mut f = vec![0; n+1];
        dfs(0, &memo, &mut f)
    }
}

Rust
impl Solution {
    pub fn rob(nums: Vec<i32>) -> i32 {
        let n = memo.len();
        let mut f = vec![0; n + 1];
        f[1] = memo[0];
        for i in 2..=n {
            f[i] = f[i - 1].max(f[i - 2] + memo[i - 1]);
        }
        f[n]
    }
}

Rust
impl Solution {
    pub fn rob(nums: Vec<i32>) -> i32 {
        let mut f = [0, 0];
        for n in memo {
            f = [f[0].max(f[1]), f[0] + n];
        }
        f[0].max(f[1])
    }
}

Rust
impl Solution {
    pub fn rob(nums: Vec<i32>) -> i32 {
        let mut dp = [0, 0];
        for num in memo {
            dp = [dp[1], dp[1].max(dp[0] + num)]
        }
        dp[1]
    }
}

(*) Dynamic Programming -- Pattern Solution (matched from community answers)
Python

```

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        if not nums:
            return 0
        if len(nums) == 1:
            return nums[0]

        # dp[i] := max money of robbing nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]
        dp[1] = max(nums[0], nums[1])

        for i in range(2, len(nums)):
            dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])

        return dp[-1]

#256 MEDIUM
Paint House
LeetCode · SimpleCode · WalkCC · LeetCode · Editorial
Array · Dynamic Programming
List: Premium 98

Problem:
There is a row of n houses, where each house can be painted one of three colors: red, blue, or green. The cost of painting each house with a certain color is different. You have to paint all the houses such that no two adjacent houses have the same color.

The cost of painting each house with a certain color is represented by an n x 3 cost matrix costs.

• For example, costs[0][0] is the cost of painting house 0 with the color red; costs[1][2] is the cost of painting house 1 with color green, and so on..

Return the minimum cost to paint all houses.

Example 1:
Input: costs = [[17,2,17],[16,16,5],[14,3,19]]
Output: 18
Explanation: Paint house 0 into blue, paint house 1 into green, paint house 2 into blue.
Minimum cost: 2 + 5 + 3 = 10.

Example 2:
Input: costs = [[7,6,2]]
Output: 2

Constraints:
• costs.length == n
• costs[i].length == 3
• 1 <= n <= 100
• 1 <= costs[i][j] <= 20

>> Brute Force (Dynamic Programming) Interview Prep

```

```

Python
# Brute Force Approach
class Solution:
    def minCost(self, costs):
        # Brute Force Approach - plain recursion, no memoization
        def recurse(i):
            if i == len(costs): return 0
            take = costs[i] + recurse(i + 1)
            skip = recurse(i + 1)
            return min(take, skip)
        return recurse(0)

Python
# Python Solution for the Paint House problem
def minCost(costs):
    # Get the number of houses
    n = len(costs)
    # Base cases: If there are no houses, return 0 cost
    if n == 0:
        return 0

    # Iterate from the second house to the last house
    for i in range(1, n):
        # Update the cost for painting the current house red
        costs[i][0] = min(costs[i-1][1], costs[i-1][2])
        # Update the cost for painting the current house blue
        costs[i][1] = min(costs[i-1][0], costs[i-1][2])
        # Update the cost for painting the current house green
        costs[i][2] = min(costs[i-1][0], costs[i-1][1])

    # The answer is the minimum cost of painting the last house with any of the colors
    return min(costs[-1])

# Example usage
example_costs = [[17,2,17],[16,16,5],[14,3,19]]
print(minCost(example_costs)) # Output: 18

Python
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        for i in range(1, len(costs)):
            costs[i][0] = min(costs[i-1][1], costs[i-1][2])
            costs[i][1] = min(costs[i-1][0], costs[i-1][2])
            costs[i][2] = min(costs[i-1][0], costs[i-1][1])
        return min(costs[-1])

Python
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(b, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)

```

```

Python
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(a, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)

# Two-dimensional dynamic array dp,
# where dp[i][j] represents the minimum cost of painting to the i-th house with color j,
# and the state transition equation is:
# dp[i][j] = cost[i][j] + min(dp[i-1][0] + 1, dp[i-1][1] + 2, dp[i-1][2] + 3)

class Solution(object):
    def minCost(self, costs):
        (type costs: List[List[int]])
        (type int)
        (type int)
        If not costs:
            return 0

        dp = [0] * len(costs[0])
        dp[0] = costs[0][0]
        for i in range(1, len(costs)):
            dp = [0] * 3
            for j in range(0, 3):
                if j == 0:
                    dp = min(dp[1], dp[2]) + costs[i][0]
                elif j == 1:
                    dp = min(dp[0], dp[2]) + costs[i][1]
                else:
                    dp = min(dp[0], dp[1]) + costs[i][2]
            dp[0], dp[1], dp[2] = dp, dp, dp
        return min(dp)

C++
C++
class Solution {
public:
    int minCost(vector<vector<int>>& costs) {
        for (int i = 1; i < costs.size(); ++i) {
            costs[i][0] = min(costs[i-1][1], costs[i-1][2]) + costs[i][0];
            costs[i][1] = min(costs[i-1][0], costs[i-1][2]) + costs[i][1];
            costs[i][2] = min(costs[i-1][0], costs[i-1][1]) + costs[i][2];
        }
        return min(costs.back());
    }
};

C++

```

```

class Solution {
public:
    int minCost(vector<vector<int>>& costs) {
        int r = 0, g = 0, b = 0;
        for (auto cost : costs) {
            int r_ = r, g_ = g, b_ = b;
            r = min(r_, b_) + cost[0];
            g = min(r_, b_) + cost[1];
            b = min(r_, g_) + cost[2];
        }
        return min(r, min(g, b));
    }
};

C++

class Solution {
public:
    int minCost(vector<vector<int>>& costs) {
        int r = 0, g = 0, b = 0;
        for (auto cost : costs) {
            int r_ = r, g_ = g, b_ = b;
            r = min(r_, b_) + cost[0];
            g = min(r_, b_) + cost[1];
            b = min(r_, g_) + cost[2];
        }
        return min(r, min(g, b));
    }
};

Java

class Solution {
    public int minCost(int[][] costs) {
        final int n = costs.length;

        for (int i = 1; i < n; ++i) {
            costs[i][0] = Math.min(costs[i - 1][1], costs[i - 1][2]);
            costs[i][1] = Math.min(costs[i - 1][0], costs[i - 1][2]);
            costs[i][2] = Math.min(costs[i - 1][0], costs[i - 1][1]);
        }

        return Math.min(costs[n - 1][0], Math.min(costs[n - 1][1], costs[n - 1][2]));
    }
}

```

```

class Solution {
    public int minCost(int[][] costs) {
        int r = 0, g = 0, b = 0;
        for (int i = 1; i < costs.length; ++i) {
            int r_ = r, g_ = g, b_ = b;
            r = Math.min(r_, b_) + costs[i][0];
            g = Math.min(r_, b_) + costs[i][1];
            b = Math.min(r_, g_) + costs[i][2];
        }
        return Math.min(r, Math.min(g, b));
    }
}

JavaScript

/**
 * @param {number[][]} costs
 * @return {number}
 */
var minCost = function(costs) {
    let [a, b, c] = [0, 0, 0];
    for (let [a, b, c] of costs) {
        [a, b, c] = [Math.min(a, c) + ca, Math.min(a, c) + cb, Math.min(a, b) + cc];
    }
    return Math.min(a, b, c);
};

/**
 * @param {number[][]} costs
 * @return {number}
 */
var minCost = function(costs) {
    let [a, b, c] = [0, 0, 0];
    for (let [a, b, c] of costs) {
        [a, b, c] = [Math.min(a, c) + ca, Math.min(a, c) + cb, Math.min(a, b) + cc];
    }
    return Math.min(a, b, c);
};

Go

```

```

Go

func minCost(costs [][]int) int {
    r, g, b := 0, 0, 0
    for _, cost := range costs {
        r_, g_, b_ := r, g, b
        r = min(r_, b_) + cost[0]
        g = min(r_, b_) + cost[1]
        b = min(r_, g_) + cost[2]
    }
    return min(r, min(g, b))
}

Go

func minCost(costs [][]int) int {
    r, g, b := 0, 0, 0
    for _, cost := range costs {
        r_, g_, b_ := r, g, b
        r = min(r_, b_) + cost[0]
        g = min(r_, b_) + cost[1]
        b = min(r_, g_) + cost[2]
    }
    return min(r, min(g, b))
}

[1] Dynamic Programming — Pattern Solution (matched from community answers)

Python

class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(a, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)

#####

"""
Two-dimensional dynamic array dp,
where dp[i][j] represents the minimum cost of breaking to the i-th house with color j,
and the state transition equation is:
dp[i][j] = cost[i][j] + min(dp[i-1][0] + 15N-3], dp[i-1][1] + 21N-3])

class Solution(object):
    def minCost(self, costs):
        # type: List[List[int]]
        # type: int
        # if not costs:
            return 0

```

```

dp = [0] * (len(costs[0]))
dp[0] = costs[0]
for i in range(1, len(costs)):
    d0 = d1 = d2 = 0
    for j in range(3):
        if j == 0:
            d0 = min(dp[1], dp[2]) + costs[i][0]
        elif j == 1:
            d1 = min(dp[0], dp[2]) + costs[i][1]
        else:
            d2 = min(dp[0], dp[1]) + costs[i][2]
    dp[0], dp[1], dp[2] = d0, d1, d2
return min(dp)

```

#309 **MEDIUM**

Best Time to Buy and Sell Stock with Cooldown

LeetCode · Simplify · [MathCE](#) · [LeetCode](#) · Editorial

Array · Dynamic Programming

List: Denny Zhang

Problem:

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: prices = [1,2,3,0,2]

Output: 3

Explanation: transactions = [buy, sell, cooldown, buy, sell]

Example 2:

Input: prices = [1]

Output: 0

Constraints:

- 1 <= prices.length <= 5000
- 0 <= prices[i] <= 1000

>> Brute Force (Dynamic Programming) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def maxProfit(self, prices):
        # Brute Force O(N^2) - recursion: at each day choose buy/sell/rest
        def dfs(i, holding, cooldown):
            if i == len(prices): return 0
            if cooldown: return dfs(i+1, holding, False) # must rest
            if holding:
                sell = prices[i] + dfs(i+1, False, True) # sell today
                rest = dfs(i+1, True, False) # hold
                return max(sell, rest)
            else:
                buy = -prices[i] + dfs(i+1, True, False) # buy today
                rest = dfs(i+1, False, False) # wait
                return max(buy, rest)
        return dfs(0, False, False)

Python

# Python Solution with time-complexity comments
def maxProfit(prices):
    # Handle edge case: if there are no prices, profit is 0
    if not prices:
        return 0

    # Initialization states:
    # s0: Maximum profit when ready to buy
    # s1: Maximum profit when holding a stock (bought)
    # s2: Maximum profit during cooldown after a sale
    s0, s1, s2 = 0, -prices[0], 0

    # Iterate over prices starting from day 1
    for price in prices[1:]:
        # Store previous state values
        prev_s0, prev_s1, prev_s2 = s0, s1, s2
        # s0 is max of either not holding or cooling from a cooldown (s2)
        s0 = max(prev_s0, prev_s2)
        # s1 is max of either buying the stock or buying today with profit from s0
        s1 = max(prev_s1, prev_s0 + price)
        # s2 is achieved by selling the stock that we were holding (prev_s1 + price)
        s2 = prev_s1 + price
    # The final maximum profit is the maximum of not holding a stock (s0 or s2)
    return max(s0, s2)

# Example usage:
print(maxProfit([1,2,3,0,2]))

Python

```

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        sell = 0
        hold = -math.inf
        prev = 0

        for price in prices:
            cache = sell
            sell = max(sell, hold + price)
            hold = max(hold, prev - price)
            prev = cache

        return sell

Python

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i == len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 2, 0))
            else:
                ans = max(ans, -prices[i] + dfs(i + 1, 1))
            return ans

        return dfs(0, 0)

Python

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [0] * 2 for _ in range(n)
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 2][0] - prices[i])
        return f[n - 1][0]

Python

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, fb, f1 = 0, 0, -prices[0]
        for s in prices[1:]:
            f, fb, f1 = fb, max(fb, f1 + s), max(f1, f - s)
        return f

```

```

class Solution {
    def maxProfit(self, prices: List[int]) -> int:
        f1, f2, f3 = -prices[0], 0, 0
        for price in prices[1:]:
            g1, g2, g3 = f1, f2, f3
            f1 = max(g1, g3 - price)
            f2 = max(g2, g1 + price)
            f3 = max(g2, g3) + max(f1, f2)
            return f2

#####

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            g, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
            return f0

#####

class Solution(object):
    def maxProfit(self, prices):
        #type: List[int]
        #type: int
        if len(prices) < 2:
            return 0
        buy = [0] * len(prices)
        sell = [0] * len(prices)
        buy[0] = -prices[0]
        buy[1] = max(prices[1], buy[0])
        sell[0] = 0
        sell[1] = max(prices[1] - prices[0], 0)
        for i in range(2, len(prices)):
            buy[i] = max(sell[i - 2] - prices[i], buy[i - 1])
            sell[i] = max(prices[i] + buy[i - 1], sell[i - 1])
            return max(sell)

```

C++
C++

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int sell = 0;
        int hold = INT_MIN;
        int prev = 0;

        for (const int price : prices) {
            const int cache = sell;
            sell = max(sell, hold + price);
            hold = max(hold, prev - price);
            prev = cache;
        }

        return sell;
    }
};

```

C++

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();
        int f[n+1];
        memset(f, -1, sizeof(f));
        function<int, int> dfs = [&](int i, int j) {
            if (i == n) {
                return 0;
            }
            if (f[i][j] != -1) {
                return f[i][j];
            }
            int ans = dfs(i + 1, j);
            if (i) {
                ans = max(ans, prices[i] + dfs(i + 2, 0));
            } else {
                ans = max(ans, -prices[i] + dfs(i + 1, 1));
            }
            return f[i][j] = ans;
        };
        return dfs(0, 0);
    }
};

```

C++

```

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int f = 0, f0 = 0, f1 = -prices[0];
        for (int i = 1; i < prices.size(); ++i) {
            int g0 = max(f0, f2 + prices[i]);
            f1 = max(f1, f - prices[i]);
            f = f0;
            f0 = g0;
        }
        return f0;
    }
};

```

```

C++
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int f = 0, f0 = 0, f1 = -prices[0];
        for (int i = 1; i < prices.size(); ++i) {
            int g0 = max(f0, f1 + prices[i]);
            f1 = max(f1, f - prices[i]);
            f = f0;
            f0 = g0;
        }
        return f0;
    }
};

```

Java

```

Java
class Solution {
    public int maxProfit(int[] prices) {
        int sell = 0;
        int hold = Integer.MIN_VALUE;
        int prev = 0;

        for (final int price : prices) {
            final int cache = sell;
            sell = Math.max(sell, hold + price);
            hold = Math.max(hold, prev - price);
            prev = cache;
        }

        return sell;
    }
}

```

Java

```

class Solution {
    private int[] prices;
    private Integer[][] f;

    public int maxProfit(int[] prices) {
        this.prices = prices;
        f = new Integer[prices.length][2];
        return dfs(0, 0);
    }

    private int dfs(int i, int j) {
        if (i == prices.length) {
            return 0;
        }
        if (f[i][j] != null) {
            return f[i][j];
        }
        int ans = dfs(i + 1, j);
        if (i < 0) {
            ans = Math.max(ans, prices[i] + dfs(i + 2, 0));
        } else {
            ans = Math.max(ans, -prices[i] + dfs(i + 1, 1));
        }
        return f[i][j] = ans;
    }
}

```

Java

```

class Solution {
    public int maxProfit(int[] prices) {
        int n = prices.length;
        int[] f = new int[n][2];
        f[0][0] = -prices[0];
        for (int i = 1; i < n; ++i) {
            f[i][0] = Math.max(f[i - 1][0], f[i - 1][1] + prices[i]);
            f[i][1] = Math.max(f[i - 1][1], (i > 1 ? f[i - 2][0] : 0) - prices[i]);
        }
        return f[n - 1][0];
    }
}

```

Java

```

class Solution {
    public int maxProfit(int[] prices) {
        int f = 0, f0 = 0, f1 = -prices[0];
        for (int i = 1; i < prices.length; ++i) {
            int g0 = Math.max(f0, f1 + prices[i]);
            f1 = Math.max(f1, f - prices[i]);
            f = f0;
            f0 = g0;
        }
        return f0;
    }
}

```

Java

```

class Solution {
    public int maxProfit(int[] prices) {
        int f = 0, f0 = 0, f1 = -prices[0];
        for (int i = 1; i < prices.length; ++i) {
            int g0 = Math.max(f0, f1 + prices[i]);
            f1 = Math.max(f1, f - prices[i]);
            f = f0;
            f0 = g0;
        }
        return f0;
    }
}

```

TypeScript

```

TypeScript
function maxProfit(prices: number[]): number {
    const n = prices.length;
    const f: number[][] = Array.from({ length: 2 }, () => Array.from({ length: 2 }, () => -1));
    const dfs = (i: number, j: number): number => {
        if (i == n) {
            return 0;
        }
        if (f[i][j] != -1) {
            return f[i][j];
        }
        if (j) {
            let ans = dfs(i + 1, j);
            ans = Math.max(ans, prices[i] + dfs(i + 2, 0));
        } else {
            ans = Math.max(ans, -prices[i] + dfs(i + 1, 1));
        }
        return f[i][j] = ans;
    };
    return dfs(0, 0);
}

```

TypeScript

```

function maxProfit(prices: number[]): number {
    let [f, f0, f1] = [0, 0, -prices[0]];
    for (const x of prices.slice(1)) {
        [f, f0, f1] = [f0, Math.max(f0, f1 + x), Math.max(f1, f - x)];
    }
    return f0;
}

```

TypeScript

```

function maxProfit(prices: number[]): number {
    let [f, f0, f1] = [0, 0, -prices[0]];
    for (const x of prices.slice(1)) {
        [f, f0, f1] = [f0, Math.max(f0, f1 + x), Math.max(f1, f - x)];
    }
    return f0;
}

```

Go

```

Go
func maxProfit(prices []int) int {
    n := len(prices)
    f := make([][]int, n)
    for i := range f {
        f[i] = []int{-1, -1}
    }
    var dfs func(i, j int) int
    dfs = func(i, j int) int {
        if i == n {
            return 0
        }
        if f[i][j] != -1 {
            return f[i][j]
        }
        ans := dfs(i+1, j)
        if j < 0 {
            ans = max(ans, prices[i]-dfs(i+2, 0))
        } else {
            ans = max(ans, -prices[i]+dfs(i+1, 1))
        }
        f[i][j] = ans
        return ans
    }
    return dfs(0, 0)
}

```

Go

```

func maxProfit(prices []int) int {
    f, f0, f1 := 0, 0, -prices[0]
    for _, x := range prices[1:] {
        f, f0, f1 = f0, max(f0, f1+x), max(f1, f-x)
    }
    return f0
}

```

Go

```

func maxProfit(prices []int) int {
    f, f0, f1 := 0, 0, -prices[0]
    for _, x := range prices[1:] {
        f, f0, f1 = f0, max(f0, f1+x), max(f1, f-x)
    }
    return f0
}

```

```

class Solution {
    def maxProfit(self, prices: List[int]) -> int:
        """
        def dfs(i: int, j: int) -> int:
            if i == len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 2, 0))
            else:
                ans = max(ans, -prices[i] + dfs(i + 1, 1))
            return ans
        return dfs(0, 0)

```

#377

MEDIUM

Combination Sum IV

LeetCode · SimplifyTest · WalkCC · LeetCode · Editorial

Array · Dynamic Programming

Lists Grid 169

Problem:

Given an array of distinct integers `nums` and a target integer `target`, return the number of possible combinations that add up to `target`.

The test cases are generated so that the answer can fit in a 32-bit integer.

Example 1:

Input: `nums = [1,2,3]`, `target = 4`

Output: 7

Explanation:

The possible combination ways are:

(1, 1, 1, 1)

(1, 1, 2)

(1, 2, 1)

(1, 3)

(2, 1, 1)

(2, 2)

(3, 1)

Note that different sequences are counted as different combinations.

Example 2:

Input: `nums = [9]`, `target = 3`

Output: 0

Constraints:

• `1 <= nums.length <= 200`

• `1 <= nums[i] <= 1000`

• All the elements of `nums` are unique.

• `1 <= target <= 1000`

Dynamic Programming · LeetCodeMastery — By Pattern

— 2599 —

LeetCodeMastery

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

>> Brute Force [Dynamic Programming] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def combinationSum4(self, nums, target):
        # Brute Force O(2^n) - plain recursion, no memoization
        def recur(idx):
            if i == len(nums): return 0
            table = nums[i] + recur(idx + 1)
            skip = recur(i + 1)
            return max(table, skip)
        return recur(0)

```

Python

Python solution using dynamic programming

```

class Solution:
    def combinationSum4(self, nums, target):
        # Initialize dp array where dp[i] stores the number of ways to reach sum i
        dp = [0] * (target + 1)
        # Base Case: There is one way to form a sum of 0 (use no elements)
        dp[0] = 1

        # Fill the dp table for each sum from 1 to target
        for i in range(1, target + 1):
            for num in nums:
                if i - num >= 0:
                    dp[i] += dp[i - num]
        return dp[target]

```

Python

```

class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        dp = [0] * (target + 1)

        def dfs(target: int) -> int:
            if target < 0:
                return 0
            if dp[target] != -1:
                return dp[target]

            dp[target] = sum(dfs(target - num) for num in nums)
            return dp[target]

        return dfs(target)

```

Python

Dynamic Programming · LeetCodeMastery — By Pattern

— 2600 —

LeetCodeMastery

```

class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        f = [0] * (target + 1)
        for i in range(1, target + 1):
            for x in nums:
                if i >= x:
                    f[i] += f[i - x]
        return f[target]

```

Python

```

class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        f = [0] * (target + 1)
        for i in range(1, target + 1):
            for x in nums:
                if i >= x:
                    f[i] += f[i - x]
        return f[target]

```

C++

C++

```

class Solution {
public:
    int combinationSum4(vector<int>& nums, int target) {
        vector<unsigned long> dp(target + 1);
        dp[0] = 1;

        for (int i = 1; i <= target; ++i)
            for (const int num : nums)
                if (i >= num)
                    dp[i] += dp[i - num];

        return dp[target];
    }
};

```

C++

```

class Solution {
public:
    int combinationSum4(vector<int>& nums, int target) {
        int f[target + 1];
        memset(f, 0, sizeof(f));
        f[0] = 1;
        for (int i = 1; i <= target; ++i) {
            for (int x : nums) {
                if (i >= x && f[i - x] < INT_MAX - f[i]) {
                    f[i] += f[i - x];
                }
            }
        }
        return f[target];
    }
};

```

C++

Dynamic Programming · LeetCodeMastery — By Pattern

— 2601 —

LeetCodeMastery

```

class Solution {
public:
    int combinationSum4(vector<int>& nums, int target) {
        int f[target + 1];
        memset(f, 0, sizeof(f));
        f[0] = 1;
        for (int i = 1; i <= target; ++i) {
            for (int x : nums) {
                if (i >= x && f[i - x] < INT_MAX - f[i]) {
                    f[i] += f[i - x];
                }
            }
        }
        return f[target];
    }
};

```

Java

Java

```

class Solution {
    public int combinationSum4(int[] nums, int target) {
        // dp[i] == the number of combinations that add up to i
        int[] dp = new int[target + 1];
        dp[0] = 1;

        for (int i = 0; i < target; ++i)
            for (final int num : nums)
                if (i >= num)
                    dp[i] += dp[i - num];

        return dp[target];
    }
}

```

Java

```

class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] f = new int[target + 1];
        f[0] = 1;
        for (int i = 1; i <= target; ++i) {
            for (int x : nums) {
                if (i >= x) {
                    f[i] += f[i - x];
                }
            }
        }
        return f[target];
    }
}

```

Java

Dynamic Programming · LeetCodeMastery — By Pattern

— 2602 —

LeetCodeMastery

```

public class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] f = new int[target + 1];
        f[0] = 1;
        for (int i = 1; i <= target; ++i) {
            for (int x : nums) {
                if (i >= x) {
                    f[i] += f[i - x];
                }
            }
        }
        return f[target];
    }
}

```

Java

```

public class Combination_Sum_IV {
    class Solution {
        public int combinationSum4(int[] nums, int target) {
            // dp[i] meaning for value i, how many combination count
            int[] dp = new int[target + 1];
            dp[0] = 1;
            for (int targetValue = 1; targetValue <= target; targetValue++) {
                for (int i = 0; i < nums.length; i++) {
                    if (nums[i] == targetValue) {
                        // dp[nums[i]] = dp[targetValue - nums[i]] + dp[i]
                        // Because both will be added to below line for dp[i] and dp[targetValue - nums[i]]
                        dp[targetValue] += dp[targetValue - nums[i]];
                    }
                }
            }
            return dp[target];
        }
    }
}
}

```

Java

```

class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target + 1];
        dp[0] = 1;
        for (int i = 1; i <= target; ++i) {
            for (int num : nums) {
                if (i >= num) {
                    dp[i] += dp[i - num];
                }
            }
        }
        return dp[target];
    }
}

```

Java

Dynamic Programming · LeetCodeMastery — By Pattern

— 2603 —

LeetCodeMastery

```

public class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] f = new int[target + 1];
        f[0] = 1;
        for (int i = 1; i <= target; ++i) {
            for (int x : nums) {
                if (i >= x) {
                    f[i] += f[i - x];
                }
            }
        }
        return f[target];
    }
}

```

JavaScript

JavaScript

```

// =
// @param {number[]} nums
// @param {number} target
// @return {number}

var combinationSum4 = function (nums, target) {
    const f = Array(target + 1).fill(0);
    f[0] = 1;
    for (let i = 1; i <= target; ++i) {
        for (const x of nums) {
            if (i >= x) {
                f[i] += f[i - x];
            }
        }
    }
    return f[target];
};

```

JavaScript

```

// =
// @param {number[]} nums
// @param {number} target
// @return {number}

var combinationSum4 = function (nums, target) {
    const f = new Array(target + 1).fill(0);
    f[0] = 1;
    for (let i = 1; i <= target; ++i) {
        for (const x of nums) {
            if (i >= x) {
                f[i] += f[i - x];
            }
        }
    }
    return f[target];
};

```

TypeScript

TypeScript

Dynamic Programming · LeetCodeMastery — By Pattern

— 2604 —

LeetCodeMastery

```

function combinationSum(nums: number[], target: number): number {
    const f: number[] = new Array(target + 1).fill(0);
    f[0] = 1;
    for (let i = 1; i <= target; ++i) {
        for (const n of nums) {
            if (i >= n) {
                f[i] += f[i - n];
            }
        }
    }
    return f[target];
}

TypeScript
function combinationSum(nums: number[], target: number): number {
    const f: number[] = new Array(target + 1).fill(0);
    f[0] = 1;
    for (let i = 1; i <= target; ++i) {
        for (const n of nums) {
            if (i >= n) {
                f[i] += f[i - n];
            }
        }
    }
    return f[target];
}

Go
func combinationSum4(nums []int, target int) int {
    f := make([]int, target+1)
    f[0] = 1
    for i := 1; i <= target; i++ {
        for _, x := range nums {
            if i >= x {
                f[i] += f[i-x]
            }
        }
    }
    return f[target]
}

C#
func combinationSum4(nums: IList<int>, target: int) int {
    f := make([]int, target+1)
    f[0] = 1
    for i := 1; i <= target; i++ {
        for _, x := range nums {
            if i >= x {
                f[i] += f[i-x]
            }
        }
    }
    return f[target]
}

```

Dynamic Programming · LeetCode · By Pattern — 2605 —

LeetCode

[*] Dynamic Programming -- Pattern Solution (matched from community answers)

```

Python
# Python solution using dynamic programming

class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        # Initialization: As per the above dp table, the number of ways to reach sum 1
        dp = [0] * (target + 1)
        # Base Case: There is one way to form a sum of 0 (use no elements)
        dp[0] = 1

        # Fill the dp table for each sum from 1 to target
        for i in range(1, target + 1):
            for num in nums:
                if i - num >= 0:
                    dp[i] += dp[i - num]
            return dp[target]

```

#416

MEDIUM

Partition Equal Subset Sum

LeetCode · Simple · Medium · LeetCode · LeetCode · Editorial

Array · Dynamic Programming

Lists · Grind 169

Problem:

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or false otherwise.

Example 1:

Input: `nums = [1,5,11,5]`

Output: `true`

Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`

Output: `false`

Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 100`

>> Brute Force (Dynamic Programming) Interview Prep

Python

Dynamic Programming · LeetCode · By Pattern — 2606 —

LeetCode

```

# Brute Force Approach
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        # Brute Force O(2^n) - try including/excluding each element for target sum
        total = sum(nums)
        if total % 2 != 0: return False
        target = total // 2
        def can_reach(i, sum):
            if sum == 0: return True
            if i <= len(nums) or sum > 0: return False
            return can_reach(i+1, sum - nums[i]) or can_reach(i+1, sum)
        return can_reach(0, target)

Python
# Python solution for Partition Equal Subset Sum

def canPartition(nums: List[int]) -> bool:
    # calculate the total sum of the array
    total_sum = sum(nums)

    # If the total sum is odd, equal partition is not possible
    if total_sum % 2 != 0:
        return False

    # Set the target sum to half of the total sum
    target = total_sum // 2

    # Initialize an array where dp[i] is True if a subset sum i is achievable
    dp = [False] * (target + 1)
    dp[0] = True # Base case: subset sum of 0 is always possible

    # Iterate through each number in the array
    for num in nums:
        # Iterate backwards to avoid overwriting dp values needed for current iteration
        for i in range(target, num - 1, -1):
            # Update dp[i] if dp[i-num] is True
            if dp[i - num]:
                dp[i] = True

    # dp[target] is True if a subset adds up to target sum
    return dp[target]

# Example usage:
# print(canPartition([1, 5, 11, 5]))

Python

```

Dynamic Programming · LeetCode · By Pattern — 2607 —

LeetCode

```

class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        sum = sum(nums)
        if sum % 2 != 0:
            return False
        return self.knapsack(nums, sum // 2)

    def knapsack(self, nums: List[int], subsetSum: int) -> bool:
        n = len(nums)
        # dp[i][j] is True if j can be formed by using elements from 0 to i-1
        dp = [[False] * (subsetSum + 1) for _ in range(n + 1)]
        dp[0][0] = True

        for i in range(1, n + 1):
            num = nums[i-1]
            for j in range(subsetSum + 1):
                if j == num:
                    dp[i][j] = dp[i-1][j]
                else:
                    dp[i][j] = dp[i-1][j] or dp[i-1][j - num]

        return dp[n][subsetSum]

Python
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        n, mod = divmod(sum(nums), 2)
        if mod:
            return False
        n = len(nums)
        f = [False] * (n + 1)
        for i in range(n + 1):
            f[i][0] = True
            for j in range(1, subsetSum + 1):
                f[i][j] = f[i-1][j] or (j >= nums[i-1] and f[i-1][j - nums[i-1]])
        return f[n][subsetSum]

```

Python

Dynamic Programming · LeetCode · By Pattern — 2608 —

LeetCode

```

class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int sum = accumulate(nums.begin(), nums.end(), 0);
        if (sum % 2 != 0)
            return false;
        return knapsack(nums, sum / 2);
    }
private:
    bool knapsack(const vector<int>& nums, int subsetSum) {
        const int n = nums.size();
        // dp[i][j] is True if j can be formed by using elements from 0 to i-1
        vector<vector<bool>> dp(n + 1, vector<bool>(subsetSum + 1));
        dp[0][0] = true;

        for (int i = 1; i <= n; ++i) {
            const int num = nums[i - 1];
            for (int j = 0; j <= subsetSum; ++j) {
                if (j == num)
                    dp[i][j] = dp[i-1][j];
                else
                    dp[i][j] = dp[i-1][j] || dp[i-1][j - num];
            }
        }
        return dp[n][subsetSum];
    }
};

C++
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int s = accumulate(nums.begin(), nums.end(), 0);
        if (s % 2 != 0)
            return false;
        int n = nums.size();
        int s = s / 2;
        bool f[s + 1][s + 1];
        memset(f, false, sizeof(f));
        f[0][0] = true;
        for (int i = 1; i <= n; ++i) {
            int x = nums[i - 1];
            for (int j = 0; j <= s; ++j) {
                f[i][j] = f[i-1][j] || (j >= x && f[i-1][j - x]);
            }
        }
        return f[n][s];
    }
};

C++
C++

```

Dynamic Programming · LeetCode · By Pattern — 2609 —

LeetCode

```

class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int s = accumulate(nums.begin(), nums.end(), 0);
        if (s % 2 != 0)
            return false;
        int n = nums.size();
        int s = s / 2;
        bool f[s + 1][s + 1];
        memset(f, false, sizeof(f));
        f[0][0] = true;
        for (int i = 1; i <= n; ++i) {
            int x = nums[i - 1];
            for (int j = 0; j <= s; ++j) {
                f[i][j] = f[i-1][j] || (j >= x && f[i-1][j - x]);
            }
        }
        return f[n][s];
    }
};

C++
C++

```

Dynamic Programming · LeetCode · By Pattern — 2610 —

LeetCode

```
// 01: https://leetcode.com/problems/partition-equal-subset-sum/
// Time: O(2^N)
// Space: O(2^N)
class Solution {
public:
    bool canPartition(vector<int>& A) {
        int total = accumulate(begin(A), end(A), 0);
        if (total % 2) return false;
        unordered_set<int> s, next;
        for (int a : A) {
            next = s;
            for (int a : s) next.insert(a + a);
            next.insert(a);
            if (next.count(total / 2)) return true;
            swap(s, next);
        }
        return false;
    }
};

Java
Java
class Solution {
public boolean canPartition(int[] nums) {
    final int sum = Arrays.stream(nums).sum();
    if (sum % 2 == 1)
        return false;
    return knapsack(nums, sum / 2);
}

private boolean knapsack(int[] nums, int subsetSum) {
    final int n = nums.length;
    // dp[i][j] := true if j can be formed by nums[0..i]
    boolean[][] dp = new boolean[n + 1][subsetSum + 1];
    dp[0][0] = true;

    for (int i = 1; i <= n; ++i) {
        final int num = nums[i - 1];
        for (int j = 0; j <= subsetSum; ++j) {
            if (j == num)
                dp[i][j] = dp[i - 1][j];
            else
                dp[i][j] = dp[i - 1][j] || dp[i - 1][j - num];
        }
    }
    return dp[n][subsetSum];
}
}
```

Java

```
class Solution {
public boolean canPartition(int[] nums) {
    int n = Arrays.stream(nums).sum();
    int s = 0;
    for (int a : nums) {
        s += a;
    }
    if (s % 2 == 1) {
        return false;
    }
    int n = nums.length;
    int s = s / 2;
    boolean[][] f = new boolean[n + 1][s + 1];
    f[0][0] = true;
    for (int i = 1; i <= n; ++i) {
        int s = nums[i - 1];
        for (int j = 0; j <= s; ++j) {
            f[i][j] = f[i - 1][j] || (j >= s && f[i - 1][j - s]);
        }
    }
    return f[n][s];
}
}

Java
import java.util.Arrays;

public class Partition_Equal_Subset_Sum {
    public static void main(String[] args) {
        Partition_Equal_Subset_Sum sol = new Partition_Equal_Subset_Sum();
        Solution s = sol.new Solution();

        System.out.println(s.canPartition(new int[]{1, 5, 11, 5}));
        System.out.println(s.canPartition(new int[]{1, 2, 3, 5}));
    }

    class Solution {
        public boolean canPartition(int[] nums) {
            if (nums == null || nums.length == 0) {
                return false;
            }

            // Do some preprocessing, should use long
            int sum = sum = Arrays.stream(nums).sum();

            if (sum % 2 != 0) { // Two equal subsets, then sum must be Two
                return false;
            }
        }
    }
}
```

```
int target = sum / 2;

// dp[i][j] := true if j can be formed by nums[0..i]
dp[0][0] = true;

for (int i = 0; i < nums.length; ++i) {
    for (int v = target; v >= nums[i]; v--) {
        dp[v] = dp[v] || dp[v - nums[i]];
    }
}

return dp[target];
}

class Solution_Recursion {
    // bool canPartition(vector<int>& nums) {
    //     return helper(nums, 0);
    // }

    // for (int sum = nums[0]; sum <= 2 * sum; sum++) {
    //     return (sum % 2 == 0) && helper(sum / 2);
    // }

    // only even subsets has 2 elements
    class Solution_TwoSum {
    public boolean canPartition(int[] nums) {
        if (nums == null || nums.length == 0) {
            return false;
        }

        // Use some preprocessing
        long sum = Arrays.stream(nums).reduce(0, (x, y) -> (x + y));
        if (sum % 2 != 0) {
            return false; // Two equal subsets, then sum must be Two
        }

        // then it's 2Sum question to sum/2
        TwoSum twoSum = new TwoSum();
        TwoSum_Solution twoSumSolution = twoSum.new Solution();
        Arrays.sort(nums);

        // possible overflow
        return twoSumSolution.twoSum(nums, (int)sum/2) == null;
    }
}
```

```
JavaScript
JavaScript
/*
 * @param {number[]} nums
 * @return {boolean}
 */
var canPartition = function (nums) {
    const s = nums.reduce((a, b) => a + b, 0);
    if (s % 2 !== 0) {
        return false;
    }
    const n = nums.length;
    const s = s / 2;
    const f = Array.from({ length: n + 1 }, () => Array(n + 1).fill(false));
    f[0][0] = true;
    for (let i = 1; i <= n; ++i) {
        const s = nums[i - 1];
        for (let j = 0; j <= s; ++j) {
            f[i][j] = f[i - 1][j] || (j >= s && f[i - 1][j - s]);
        }
    }
    return f[n][s];
};

JavaScript
/*
 * @param {number[]} nums
 * @return {boolean}
 */
var canPartition = function (nums) {
    let s = 0;
    for (let v of nums) {
        s += v;
    }
    if (s % 2 != 0) {
        return false;
    }
    const n = nums.length;
    const dp = new Array(n + 1).fill(false);
    dp[0] = true;
    for (let i = 1; i <= n; ++i) {
        for (let j = 0; j <= nums[i - 1]; ++j) {
            dp[j] = dp[j] || dp[j - nums[i - 1]];
        }
    }
    return dp[n];
};

TypeScript
TypeScript
```

```
function canPartition(nums: number[]): boolean {
    const s = nums.reduce((a, b) => a + b, 0);
    if (s % 2 !== 0) {
        return false;
    }
    const n = nums.length;
    const s = s / 2;
    const f: boolean[][] = Array.from({ length: n + 1 }, () => Array(n + 1).fill(false));
    f[0][0] = true;
    for (let i = 1; i <= n; ++i) {
        const s = nums[i - 1];
        for (let j = 0; j <= s; ++j) {
            f[i][j] = f[i - 1][j] || (j >= s && f[i - 1][j - s]);
        }
    }
    return f[n][s];
}

TypeScript
function canPartition(nums: number[]): boolean {
    const s = nums.reduce((a, b) => a + b, 0);
    if (s % 2 !== 0) {
        return false;
    }
    const n = s / 2;
    const f: boolean[] = Array(n + 1).fill(false);
    f[0] = true;
    for (const s of nums) {
        for (let j = s; j >= 0; --j) {
            f[j] = f[j] || f[j - s];
        }
    }
    return f[0];
}

Go
Go
```

```
func canPartition(nums []int) bool {
    s := 0
    for _, v := range nums {
        s += v
    }
    if s%2 == 1 {
        return false
    }
    n, m := len(nums), s/2
    f := make([][]bool, n+1)
    for i := range f {
        f[i] = make([]bool, m+1)
    }
    f[0][0] = true
    for i := 1; i <= n; i++ {
        s := nums[i-1]
        for j := 0; j <= m; j++ {
            f[i][j] = f[i-1][j] || (j >= s && f[i-1][j-s])
        }
    }
    return f[n][m]
}

Go
func canPartition(nums []int) bool {
    s := 0
    for _, v := range nums {
        s += v
    }
    if s%2 != 0 {
        return false
    }
    n := s / 2
    dp := make([]bool, n+1)
    dp[0] = true
    for _, v := range nums {
        for j := n; j >= v; j-- {
            dp[j] = dp[j] || dp[j-v]
        }
    }
    return dp[n]
}

Rust
Rust
```



```

impl Solution {
    pub fn can_partition(nums: Vec<i32>) -> bool {
        let n: i32 = nums.iter().sum();
        if n % 2 != 0 {
            return false;
        }
        let n = (n / 2) as usize;
        let n = nums.len();
        let mut f = vec![vec![false; n + 1]; n + 1];
        f[0][0] = true;
        for i in 1..=n {
            let n = nums[i - 1] as usize;
            for j in 0..=n {
                f[i][j] = f[i - 1][j] || (j >= n && f[i - 1][j - n]);
            }
        }
        f[n][n]
    }
}

```

Rust

```

impl Solution {
    // @lc=leetcode.com
    pub fn can_partition(nums: Vec<i32>) -> bool {
        let mut sum = 0;
        for n in nums {
            sum += n;
        }
        if sum % 2 != 0 {
            return false;
        }
        let n = nums.len();
        let n = (sum / 2) as usize;
        let mut dp: Vec<Vec<bool>> = vec![vec![false; n + 1]; n + 1];
        // Initialize the dp vector
        dp[0][0] = true;
        // Begin the actual dp process
        for i in 1..=n {
            for j in 0..=n {
                dp[i][j] = if nums[i - 1] as usize > j {
                    dp[i - 1][j]
                } else {
                    dp[i - 1][j] || dp[i - 1][j - nums[i - 1] as usize]
                }
            }
        }
        dp[n][n]
    }
}

```

Dynamic Programming · LeetCode · By Pattern — 2617 —

LeetCode

[*] Dynamic Programming -- Pattern Solution (matched from community answers)

Python

```

class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        n = len(nums)
        if n < 2:
            return False
        n = n // 2
        dp = [False] * (n + 1)
        dp[0] = True
        for v in nums:
            for target in range(n, v - 1, -1): # including v itself
                dp[target] = dp[target] or dp[target - v]
            return dp[-1]

class Solution: # recursion, but over time limit (n 0)
    def canPartition(self, nums: List[int]) -> bool:
        total_sum = sum(nums)
        if total_sum % 2 != 0:
            return False
        target_sum = total_sum // 2
        memo = {} # key == n, just use cache for dfs()

        def dfs(ids, curr_sum):
            if curr_sum == target_sum:
                return True
            if curr_sum > target_sum or ids == len(nums):
                return False
            if (ids, curr_sum) in memo:
                return memo[(ids, curr_sum)]
            # Explore two options: include the current number or exclude it
            include = dfs(ids + 1, curr_sum + nums[ids])
            exclude = dfs(ids + 1, curr_sum)
            memo[(ids, curr_sum)] = include or exclude
            return memo[(ids, curr_sum)]

        return dfs(0, 0)

#####
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        n, mod = divmod(sum(nums), 2)
        if mod:
            return False
        f = [False] * (n + 1)
        for i in nums:
            for j in range(n, i - 1, -1):
                f[j] = f[j] or f[j - i]
        return f[n]

```

Dynamic Programming · LeetCode · By Pattern — 2618 —

LeetCode

#435 MEDIUM

Non-overlapping Intervals

LeetCode · SimplifyIt · WsAAC · LeetCode · Editorial

Array · Dynamic Programming · Greedy · Sorting

Units Grind 189

Problem:

Given an array of intervals where intervals[i] = [starti, endi], return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, [1, 2] and [2, 3] are non-overlapping.

Example 1:

Input: intervals = [[1,2],[2,3],[3,4],[1,3]]

Output: 1

Explanation: [1,3] can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: intervals = [[1,2],[1,2],[1,2]]

Output: 2

Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping.

Example 3:

Input: intervals = [[1,2],[2,3]]

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- 1 <= intervals.length <= 105
- intervals[i].length == 2
- 5 * 104 <= starti < endi <= 5 * 104

>> Brute Force (Dynamic Programming) Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        # Sort intervals by end time, greedy is O(n log n), brute chosen for intuition
        intervals.sort(key=lambda x: x[1])
        removed = 0
        end = float('-inf')
        for i in intervals:
            if i[0] >= end:
                end = i[1] # keep this interval
            else:
                removed += 1 # drop overlapping interval
        return removed

```

Python

Dynamic Programming · LeetCode · By Pattern — 2619 —

LeetCode

Python

```

# Python Implementation for Non-overlapping Intervals
def eraseOverlapIntervals(intervals):
    # Sort intervals based on the end time
    intervals.sort(key=lambda x: x[1])

    # Initialize the count of intervals in the non-overlapping set
    count = 0

    # Initialize the end of the last added interval to the minimum possible value
    last_included_end = float('-inf')

    # Iterate over every interval in the sorted list
    for interval in intervals:
        # If the current interval does not overlap with the last added interval
        if interval[0] >= last_included_end:
            count += 1 # Include this interval
            last_included_end = interval[1] # Update and for subsequent comparisons
        # The smallest number of removals is total intervals minus non-overlapping intervals count
        return len(intervals) - count

# Example usage
print(eraseOverlapIntervals([[1,2],[2,3],[3,4],[1,3]])) # Expected output: 1

```

Python

```

class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        ans = 0
        currentEnd = -math.inf

        for interval in sorted(intervals, key=lambda x: x[1]):
            if interval[0] >= currentEnd:
                currentEnd = interval[1]
            else:
                ans += 1

        return ans

```

Python

```

class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans = len(intervals)
        pre = -inf
        for i, r in intervals:
            if pre >= i:
                ans -= 1
            pre = r
        return ans

```

Python

Dynamic Programming · LeetCode · By Pattern — 2620 —

LeetCode

```

class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans, i = 0, intervals[0][1]
        for n, e in intervals[1:]:
            if n >= i:
                i = e
            else:
                ans += 1
        return ans

```

C++

```

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.empty())
            return 0;

        ranges::sort(intervals, [](const auto& a, const auto& b) { return a[1] < b[1]; });

        int ans = 0;
        int currentEnd = intervals[0][1];
        for (int i = 1; i < intervals.size(); ++i)
            if (intervals[i][0] >= currentEnd)
                currentEnd = intervals[i][1];
            else
                ++ans;
        return ans;
    }
};

```

C++

```

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        ranges::sort(intervals, [](const vector<int>& a, const vector<int>& b) {
            return a[1] < b[1];
        });
        int ans = intervals.size();
        int pre = INT_MIN;
        for (const auto& v : intervals) {
            int l = v[0], r = v[1];
            if (pre >= l) {
                --ans;
                pre = r;
            }
        }
        return ans;
    }
};

```

C++

Dynamic Programming · LeetCode · By Pattern — 2621 —

LeetCode

```

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        int ans = 0, t = intervals[0][1];
        for (int i = 1; i < intervals.size(); ++i) {
            if (t < intervals[i][0])
                t = intervals[i][1];
            else
                ++ans;
        }
        return ans;
    }
};

```

Java

```

class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        if (intervals.length == 0)
            return 0;

        Arrays.sort(intervals, Comparator.comparingInt(a -> a[1]));

        int ans = 0;
        int currentEnd = intervals[0][1];
        for (int i = 1; i < intervals.length; ++i)
            if (intervals[i][0] >= currentEnd)
                currentEnd = intervals[i][1];
            else
                ++ans;
        return ans;
    }
}

```

Java

```

class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);
        int ans = intervals.length;
        int pre = Integer.MIN_VALUE;
        for (int i = 0; i < intervals.length; ++i) {
            int l = intervals[i][0], r = intervals[i][1];
            if (pre >= l) {
                --ans;
                pre = r;
            }
        }
        return ans;
    }
}

```

Java

Dynamic Programming · LeetCode · By Pattern — 2622 —

LeetCode

```
class Solution {
public: int eraseOverlapIntervals(vector<int>> intervals) {
    Arrays.sort(intervals, Comparator.comparingInt(a -> a[1]));
    int t = intervals[0][1], ans = 0;
    for (int i = 1; i < intervals.length; ++i) {
        if (intervals[i][0] >= t) {
            t = intervals[i][1];
        } else {
            ++ans;
        }
    }
    return ans;
}
}
```

TypeScript

```
function eraseOverlapIntervals(intervals: number[][]): number {
    intervals.sort((a, b) => a[1] - b[1]);
    let [ans, prev] = [intervals.length, -Infinity];
    for (const [l, r] of intervals) {
        if (prev < l) {
            --ans;
            prev = r;
        }
    }
    return ans;
}
```

TypeScript

```
function eraseOverlapIntervals(intervals: number[][]): number {
    intervals.sort((a, b) => a[1] - b[1]);
    let end = intervals[0][1],
        ans = 0;
    for (let i = 1; i < intervals.length; ++i) {
        let cur = intervals[i];
        if (end <= cur[0]) {
            ++ans;
        } else {
            end = cur[1];
        }
    }
    return ans;
}
```

Go

Go

```
func eraseOverlapIntervals(intervals [][]int) int {
    sort.Slice(intervals, func(i, j) bool {
        return intervals[i][1] < intervals[j][1]
    })
    ans := len(intervals)
    pre := math.MinInt32
    for _, v := range intervals {
        if v[0] <= pre {
            pre = v[1]
            ans--
        }
    }
    return ans
}
```

Go

```
func eraseOverlapIntervals(intervals [][]int) int {
    sort.Slice(intervals, func(i, j) bool {
        return intervals[i][1] < intervals[j][1]
    })
    t, ans := intervals[0][1], 0
    for i := 1; i < len(intervals); i++ {
        if intervals[i][0] >= t {
            t = intervals[i][1]
        } else {
            ans++
        }
    }
    return ans
}
```

[*] Dynamic Programming -- Pattern Solution (stored optimal)

Python

```
# Python implementation for Non-overlapping Intervals
def eraseOverlapIntervals(intervals):
    # Sort intervals based on their end time
    intervals.sort(key=lambda x: x[1])

    # Initialize the count of intervals in the non-overlapping set
    count = 0

    # Initialize the end of the last added interval to the minimum possible value
    last_included_end = float("-inf")

    # Iterate over every interval in the sorted list
    for interval in intervals:
        # If the current interval does not overlap with the last added interval
        if interval[0] >= last_included_end:
            count += 1 # We include this interval
            last_included_end = interval[1] # Update and for subsequent comparisons
    # The maximum number of removals is total intervals minus non-overlapping intervals count
    return len(intervals) - count

# Example usage
print(eraseOverlapIntervals([[1,2],[2,3],[3,4],[1,3]])) # Expected output: 1
```

```
C++
// C++ Implementation for Non-overlapping Intervals

#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        // Sort intervals based on their end time
        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
            return a[1] < b[1];
        });

        int count = 0; // Count of non-overlapping intervals
        int lastIncludedEnd = INT_MIN;

        // Iterate over the intervals
        for (const auto& interval : intervals) {
            if (interval[0] >= lastIncludedEnd) {
                ++count; // Include this interval
                lastIncludedEnd = interval[1]; // Update the last included end time
            }
        }

        // The number of removals is the total intervals minus the count of non-overlapping intervals
        return intervals.size() - count;
    }
};

// Example usage:
// int main() {
//     vector<vector<int>> intervals = {{1,2},{2,3},{3,4},{1,3}};
//     Solution sol;
//     int removals = sol.eraseOverlapIntervals(intervals);
//     // Expected output: 1
// }
```

#516 MEDIUM

Longest Palindromic Subsequence

LeetCode · SimplifyLess · WalkCC · LeetCode · Editorial

String · Dynamic Programming

Licite Danny Zhang

Problem:

Given a string *s*, find the longest palindromic subsequence's length in *s*.

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

Input: *s* = "bbbab"

Output: 4

Explanation: One possible longest palindromic subsequence is "bbbb".

Example 2:

Input: *s* = "cbbd"

Output: 2

Explanation: One possible longest palindromic subsequence is "bb".

Constraints:

- 1 <= s.length <= 1000
- s consists only of lowercase English letters.

>> Brute Force [Dynamic Programming] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def longest_palindromic_subseq(self, s):
        # Memoization table to store results of subproblems
        memo = {}

        def recur(i, j):
            # Base case: single character is a palindrome
            if i == j:
                return 1

            # Check if the result is already in the memo
            if (i, j) in memo:
                return memo[(i, j)]

            # If characters at i and j match, add 2 to the result of the inner substring
            if s[i] == s[j]:
                memo[(i, j)] = 2 + recur(i+1, j-1)
            else:
                # If characters don't match, choose the maximum from either excluding left or right char
                memo[(i, j)] = max(recur(i+1, j), recur(i, j-1))

            return memo[(i, j)]

        # Example usage:
        s = "bbbab"
        print(longest_palindromic_subseq(s)) # Expected output: 4
```

```
Python
class Solution:
    def longestPalindromicSubseq(self, s: str) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns the length of s[i:j+1]."""
            if i > j:
                return 0
            if i == j:
                return 1
            if s[i] == s[j]:
                return 2 + dp(i+1, j-1)
            return max(dp(i+1, j), dp(i, j-1))

        return dp(0, len(s) - 1)

Python
class Solution:
    def longestPalindromicSubseq(self, s: str) -> int:
        n = len(s)
        # dp[i][j] = the length of s[i:j+1]
        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for d in range(1, n):
            for i in range(n-d):
                j = i + d
                if s[i] == s[j]:
                    dp[i][j] = dp[i+1][j-1] + 2
                else:
                    dp[i][j] = max(dp[i+1][j], dp[i][j-1])

        return dp[0][n-1]

Python
class Solution:
    def longestPalindromicSubseq(self, s: str) -> int:
        n = len(s)
        f = [[0] * n for _ in range(n)]
        for i in range(n):
            f[i][i] = 1
            for j in range(i+1, n):
                if s[i] == s[j]:
                    f[i][j] = f[i+1][j-1] + 2
                else:
                    f[i][j] = max(f[i+1][j], f[i][j-1])
        return f[0][n-1]
```

```
class Solution:
    def longestPalindromicSubseq(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        for i in range(n):
            dp[i][i] = 1
            for j in range(i+1, n):
                if s[i] == s[j]:
                    dp[i][j] = dp[i+1][j-1] + 2
                else:
                    dp[i][j] = max(dp[i+1][j], dp[i][j-1])
        return dp[0][n-1]

class Solution:
    def longestPalindromicSubseq(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        for i in range(n):
            dp[i][i] = 1
            for j in range(i+1, n):
                if s[i] == s[j]:
                    dp[i][j] = dp[i+1][j-1] + 2
                else:
                    dp[i][j] = max(dp[i+1][j], dp[i][j-1])
        return dp[0][n-1]
```

C++

C++

```
class Solution {
public:
    int longestPalindromicSubseq(string s) {
        const int n = s.length();
        vector<vector<int>> memo(n, vector<int>(n));
        return lps(s, 0, n-1, memo);
    }

private:
    // Returns the length of s[l:r+1].
    int lps(const string& s, int l, int r, vector<vector<int>>& memo) {
        if (l > r)
            return 0;
        if (l == r)
            return 1;
        if (memo[l][r] > 0)
            return memo[l][r];
        return memo[l][r] = 2 + lps(s, l+1, j-1, memo);
    }
};
```

Dynamic Programming · LeetMastery — By Pattern — 2629 — LeetMastery

Dynamic Programming · LeetMastery — By Pattern — 2630 — LeetMastery

Dynamic Programming · LeetCode — By Pattern — 2631 — LeetCode

Dynamic Programming · LeetMastery — By Pattern — 2632 — LeetMastery

Dynamic Programming · LeetCode — By Pattern — 2633 — LeetCode

Dynamic Programming · LeetMastery — By Pattern — 2634 — LeetMastery

```

MOD = 10**9 + 7

def findPaths(m, n, maxMove, startRow, startColumn):
    # Initialize memo dictionary to cache the states
    memo = {}

    def dfs(i, j, movesLeft):
        # If out of grid bounds, it's a valid path
        if i < 0 or i == m or j < 0 or j == n:
            return 0
        # No moves left and still inside the grid means no valid path out
        if movesLeft == 0:
            return 0
        # Check if we already computed this state
        if (i, j, movesLeft) in memo:
            return memo[(i, j, movesLeft)]

        count = 0
        # Four possible directions: up, down, left, right
        count += dfs(i-1, j, movesLeft-1) +
            dfs(i+1, j, movesLeft-1) +
            dfs(i, j-1, movesLeft-1) +
            dfs(i, j+1, movesLeft-1) % MOD

        memo[(i, j, movesLeft)] = count

        return count

    return dfs(startRow, startColumn, maxMove)

# Example usage
if __name__ == "__main__":
    print(findPaths(2, 2, 0, 0)) # Expected output: 6

```

Python

```

class Solution:
    def findPaths(self, m, n, maxMove, startRow, startColumn):
        MOD = 1000000007

        @functools.lru_cache(maxsize=None)
        def dfs(i, j, left, j, left) -> int:
            # Returns the number of paths to move the ball at (i, j) out-of-bounds with k moves

            if i < 0 or i == m or j < 0 or j == n:
                return 0
            if k == 0:
                return 0
            return dfs(i-1, j, left-1, j, left-1) +
                dfs(i+1, j, left-1, j, left-1) +
                dfs(i, j-1, left-1, j, left-1) +
                dfs(i, j+1, left-1, j, left-1) % MOD

        return dfs(maxMove, startRow, startColumn, maxMove)

```

Python

```

class Solution:
    def findPaths(
        self,
        m: int,
        n: int,
        maxMove: int,
        startRow: int,
        startColumn: int,
    ) -> int:
        DIRS = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        MOD = 1_000_000_007

        ans = 0
        # dfs(i, j) -> the number of paths to move the ball (i, j) out-of-bounds
        dp = [[0] * n for _ in range(m)]
        dp[startRow][startColumn] = 1

        for i in range(startRow):
            nextRow = (i+1) % m
            for j in range(n):
                if dp[i][j] > 0:
                    for dx, dy in DIRS:
                        x = i + dx
                        y = j + dy
                        if x < 0 or x == m or y < 0 or y == n:
                            ans = (ans + dp[i][j]) % MOD
                        else:
                            nextDp[x][y] = (nextDp[x][y] + dp[i][j]) % MOD
        dp = nextDp

        return ans

```

Python

```

class Solution:
    def findPaths(
        self, m: int, n: int, maxMove: int, startRow: int, startColumn: int
    ) -> int:
        # cache
        def dfs(i: int, j: int, k: int) -> int:
            if not 0 <= i < m or not 0 <= j < n:
                return 0
            if k == 0:
                return 0
            ans = 0
            for a, b in pairs(DIRS):
                x, y = i + a, j + b
                ans = (ans + dfs(x, y, k-1)) % mod
            return ans

        mod = 10**9 + 7
        dirs = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        return dfs(startRow, startColumn, maxMove)

```

Python

```

class Solution:
    def findPaths(
        self, m: int, n: int, maxMove: int, startRow: int, startColumn: int
    ) -> int:
        # cache
        def dfs(i, j, k):
            if i < 0 or i == m or j < 0 or j == n:
                return 0
            if k == 0:
                return 0
            res = 0
            for a, b in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
                x, y = i + a, j + b
                res += dfs(x, y, k-1)
            res %= mod
            return res

        mod = 10**9 + 7
        return dfs(startRow, startColumn, maxMove)

```

Java

```

class Solution {
    private int m;
    private int n;
    private int[][][] f;
    private static final int[] DIRS = {-1, 0, 1, 0, -1};
    private static final int MOD = (int) 1e9 + 7;

    public int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
        this.m = m;
        this.n = n;
        f = new int[m+1][n+1][maxMove+1];
        for (int a = 0; a < 4; a++)
            for (int b = 0; b < 4; b++)
                Arrays.fill(f[a][b], -1);

        return dfs(startRow, startColumn, maxMove);
    }

    private int dfs(int i, int j, int k) {
        if (i < 0 || i == m || j < 0 || j == n)
            return 0;
        if (f[i][j][k] != -1)
            return f[i][j][k];
    }
}

```

```

}
if (k == 0) {
    return 0;
}

int res = 0;
for (int t = 0; t < 4; t++) {
    int x = i + DIRS[t];
    int y = j + DIRS[t+1];
    res += dfs(x, y, k-1);
    res %= MOD;
}

f[i][j][k] = res;
return res;
}
}

```

TypeScript

```

function findPaths(
    m: number,
    n: number,
    maxMove: number,
    startRow: number,
    startColumn: number,
): number {
    const f = Array.from({ length: m }, () =>
        Array.from({ length: n }, () => Array.from({ length: maxMove + 1, fill: -1 }, () => 0)
    );

    const mod = 1000000007;
    const dirs = [-1, 0, 1, 0, -1];
    const dfs = (i: number, j: number, k: number): number => {
        if (i < 0 || i == m || j < 0 || j == n) {
            return 0;
        }
        if (k == 0) {
            return 0;
        }
        if (f[i][j][k] !== -1) {
            return f[i][j][k];
        }
        let ans = 0;
        for (let d = 0; d < 4; d++) {
            const { x, y } = dirs[d], j = dirid(d, j);
            ans = (ans + dfs(x, y, k-1)) % mod;
        }
        return (f[i][j][k] = ans);
    };
    return dfs(startRow, startColumn, maxMove);
}

```

Go

```

func findPaths(m int, n int, maxMove int, startRow int, startColumn int) int {
    f := make([][][]int, m+1)
    for i := range f {
        f[i] = make([][]int, n+1)
        for j := range f[i] {
            f[i][j] = make([]int, maxMove+1)
            for k := range f[i][j] {
                f[i][j][k] = -1
            }
        }
    }

    var mod int = 1e9 + 7
    dirs := [][]int{{-1, 0, 1, 0, -1}}
    var dfs func(i, j, k int) int
    dfs = func(i, j, k int) int {
        if i < 0 || i == m || j < 0 || j == n {
            return 0
        }
        if f[i][j][k] != -1 {
            return f[i][j][k]
        }

        res := 0
        for t := 0; t < 5; t++ {
            x, y := dirs[t], j+dirs[t+1]
            res += dfs(x, y, k-1)
            res %= mod
        }
        f[i][j][k] = res
        return res
    }

    return dfs(startRow, startColumn, maxMove)
}

```

[*] Dynamic Programming · Pattern Solution (matched from community answers)

Python

```

class Solution:
    def findPaths(
        self,
        m: int,
        n: int,
        maxMove: int,
        startRow: int,
        startColumn: int,
    ) -> int:
        DIRS = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        MOD = 1_000_000_007

        ans = 0
        # dfs(i, j) -> the number of paths to move the ball (i, j) out-of-bounds
        dp = [[0] * n for _ in range(m)]
        dp[startRow][startColumn] = 1

        for i in range(startRow):
            nextRow = (i+1) % m
            for j in range(n):
                if dp[i][j] > 0:
                    for dx, dy in DIRS:
                        x = i + dx
                        y = j + dy
                        if x < 0 or x == m or y < 0 or y == n:
                            ans = (ans + dp[i][j]) % MOD
                        else:
                            nextDp[x][y] = (nextDp[x][y] + dp[i][j]) % MOD
        dp = nextDp

        return ans

```

#1143

MEDIUM

Longest Common Subsequence

LeetCode · SimplyLeetCode · WubACCC · LeetCode · Editorial

String · Dynamic Programming

Little Sheng Zhang

Problem:

Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

• For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

Input: text1 = "abcde", text2 = "ace"
Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:
Input: text1 = "abc", text2 = "abc"
Output: 3
Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:
Input: text1 = "abc", text2 = "def"
Output: 0
Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- 1 <= text1.length, text2.length <= 1000
- text1 and text2 consist of only lowercase English characters.

```
Python
# Python3 code to find the longest common subsequence length
def longestCommonSubsequence(text1: str, text2: str) -> int:
    n, m = len(text1), len(text2)
    # Initialize a 2D DP table filled with zeros
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Build the DP table bottom-up
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            # If characters match, extend the subsequence length by 1
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                # Otherwise, choose the maximum from the left or top cell
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[n][m]

# Example usage
if __name__ == "__main__":
    text1 = "abcde"
    text2 = "ace"
    print(longestCommonSubsequence(text1, text2)) # Output: 3
```

```
Python
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        n = len(text1)
        m = len(text2)
        # dp[i][j] is the length of LCS(text1[0..i], text2[0..j])
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, n + 1):
            for j in range(1, m + 1):
                dp[i + 1][j + 1] = (1 + dp[i][j]) if text1[i] == text2[j]
                else max(dp[i + 1][j], dp[i + 1][j + 1])

        return dp[n][m]
```

```
C++
class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        const int n = text1.length();
        const int m = text2.length();
        // dp[i][j] is the length of LCS(text1[0..i], text2[0..j])
        vector<vector<int>> dp(n + 1, vector<int>(m + 1));

        for (int i = 0; i <= n; ++i)
            for (int j = 0; j <= m; ++j)
                dp[i + 1][j + 1] = text1[i] == text2[j]
                    ? 1 + dp[i][j]
                    : max(dp[i + 1][j], dp[i][j + 1]);

        return dp[n][m];
    }
};
```

```
Java
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int n = text1.length(), m = text2.length();
        int[][] dp = new int[n + 1][m + 1];
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        return dp[n][m];
    }
}
```

```
Java
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        final int n = text1.length();
        final int m = text2.length();
        // dp[i][j] is the length of LCS(text1[0..i], text2[0..j])
        int[][] dp = new int[n + 1][m + 1];

        for (int i = 0; i <= n; ++i)
            for (int j = 0; j <= m; ++j)
                dp[i + 1][j + 1] = text1.charAt(i) == text2.charAt(j)
                    ? 1 + dp[i][j]
                    : Math.max(dp[i + 1][j], dp[i + 1][j + 1]);

        return dp[n][m];
    }
}
```

```
Java
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int n = text1.length(), m = text2.length();
        int[][] f = new int[n + 1][m + 1];
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    f[i][j] = f[i - 1][j - 1] + 1;
                } else {
                    f[i][j] = Math.max(f[i - 1][j], f[i][j - 1]);
                }
            }
        return f[n][m];
    }
}
```

```
Java
public class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int n = text1.length(), m = text2.length();
        int[][] f = new int[n + 1][m + 1];
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    f[i][j] = f[i - 1][j - 1] + 1;
                } else {
                    f[i][j] = Math.max(f[i - 1][j], f[i][j - 1]);
                }
            }
        return f[n][m];
    }
}
```

```
JavaScript
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int n = text1.length(), m = text2.length();
        int[][] dp = new int[n + 1][m + 1];
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        return dp[n][m];
    }
}
```

```
JavaScript
public class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int n = text1.length(), m = text2.length();
        int[][] f = new int[n + 1][m + 1];
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                    f[i][j] = f[i - 1][j - 1] + 1;
                } else {
                    f[i][j] = Math.max(f[i - 1][j], f[i][j - 1]);
                }
            }
        return f[n][m];
    }
}
```

```
JavaScript
function longestCommonSubsequence(text1, text2) {
    const n = text1.length;
    const m = text2.length;
    const f = Array.from({ length: n + 1 }, () => Array(m + 1).fill(0));
    for (let i = 1; i <= n; ++i)
        for (let j = 1; j <= m; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                f[i][j] = f[i - 1][j - 1] + 1;
            } else {
                f[i][j] = Math.max(f[i - 1][j], f[i][j - 1]);
            }
        }
    return f[n][m];
}
```

```
JavaScript
function longestCommonSubsequence(text1, text2) {
    const n = text1.length;
    const m = text2.length;
    const f = Array.from({ length: n + 1 }, () => Array(m + 1).fill(0));
    for (let i = 1; i <= n; ++i)
        for (let j = 1; j <= m; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                f[i][j] = f[i - 1][j - 1] + 1;
            } else {
                f[i][j] = Math.max(f[i - 1][j], f[i][j - 1]);
            }
        }
    return f[n][m];
}
```

```
TypeScript
function longestCommonSubsequence(text1: string, text2: string): number {
    const n = text1.length;
    const m = text2.length;
    const f = Array.from({ length: n + 1 }, () => Array(m + 1).fill(0));
    for (let i = 1; i <= n; ++i)
        for (let j = 1; j <= m; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                f[i][j] = f[i - 1][j - 1] + 1;
            } else {
                f[i][j] = Math.max(f[i - 1][j], f[i][j - 1]);
            }
        }
    return f[n][m];
}
```

```
TypeScript
function longestCommonSubsequence(text1: string, text2: string): number {
    const n = text1.length;
    const m = text2.length;
    const f = Array.from({ length: n + 1 }, () => Array(m + 1).fill(0));
    for (let i = 1; i <= n; ++i)
        for (let j = 1; j <= m; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                f[i][j] = f[i - 1][j - 1] + 1;
            } else {
                f[i][j] = Math.max(f[i - 1][j], f[i][j - 1]);
            }
        }
    return f[n][m];
}
```

```
Go
func longestCommonSubsequence(text1 string, text2 string) int {
    n, m := len(text1), len(text2)
    f := make([][]int, n+1)
    for i := range f {
        f[i] = make([]int, m+1)
    }
    for i := 1; i <= n; i++ {
        for j := 1; j <= m; j++ {
            if text1[i-1] == text2[j-1] {
                f[i][j] = f[i-1][j-1] + 1
            } else {
                f[i][j] = max(f[i-1][j], f[i][j-1])
            }
        }
    }
    return f[n][m]
}
```

```
Go
func longestCommonSubsequence(text1 string, text2 string) int {
    n, m := len(text1), len(text2)
    f := make([][]int, n+1)
    for i := range f {
        f[i] = make([]int, m+1)
    }
    for i := 1; i <= n; i++ {
        for j := 1; j <= m; j++ {
            if text1[i-1] == text2[j-1] {
                f[i][j] = f[i-1][j-1] + 1
            } else {
                f[i][j] = max(f[i-1][j], f[i][j-1])
            }
        }
    }
    return f[n][m]
}
```

```
Rust
fn longestCommonSubsequence(text1: String, text2: String) -> i32 {
    let n = text1.len();
    let m = text2.len();
    let mut f = vec![vec![0; m + 1]; n + 1];
    for i in 1..=n {
        for j in 1..=m {
            if text1[i - 1] == text2[j - 1] {
                f[i][j] = f[i - 1][j - 1] + 1;
            } else {
                f[i][j] = max(f[i - 1][j], f[i][j - 1]);
            }
        }
    }
    f[n][m]
}
```

```
impl Solution {
    pub fn longest_common_subsequence(text1: String, text2: String) -> i32 {
        let (n, m) = (text1.len(), text2.len());
        let (text1, text2) = (text1.as_bytes(), text2.as_bytes());
        let mut f = vec![vec![0; m + 1]; n + 1];
        for i in 1..=n {
            for j in 1..=m {
                f[i][j] = if text1[i - 1] == text2[j - 1] {
                    f[i - 1][j - 1] + 1
                } else {
                    f[i - 1][j].max(f[i][j - 1])
                };
            }
        }
        f[n][m]
    }
}
```

Rust

```
impl Solution {
    pub fn longest_common_subsequence(text1: String, text2: String) -> i32 {
        let (n, m) = (text1.len(), text2.len());
        let (text1, text2) = (text1.as_bytes(), text2.as_bytes());
        let mut f = vec![vec![0; m + 1]; n + 1];
        for i in 1..=n {
            for j in 1..=m {
                f[i][j] = if text1[i - 1] == text2[j - 1] {
                    f[i - 1][j - 1] + 1
                } else {
                    f[i - 1][j].max(f[i][j - 1])
                };
            }
        }
        f[n][m]
    }
}
```

Kotlin

```
class Solution {
    fun longestCommonSubsequence(text1: String, text2: String): Int {
        val n = text1.length
        val m = text2.length
        val f = Array(n + 1) { IntArray(m + 1) }
        for (i in 1..n) {
            for (j in 1..m) {
                if (text1[i - 1] == text2[j - 1]) {
                    f[i][j] = f[i - 1][j - 1] + 1
                } else {
                    f[i][j] = Math.max(f[i - 1][j], f[i][j - 1])
                }
            }
        }
        return f[n][m]
    }
}
```

Input: s = "ab", p = ".*a"
Output: true
Explanation: ".*" means "zero or more (x) of any character (.)".

Constraints:

- 1 <= s.length <= 20
- 1 <= p.length <= 20
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

Python

```
Python
# Python Solution using Recursion with Memoization
def isMatch(s: str, p: str) -> bool:
    memo = {}

    def dp(i: int, j: int) -> bool:
        # If result is already computed, return it.
        if (i, j) in memo:
            return memo[(i, j)]

        # If we have reached the end of the pattern, s must also be at the end.
        if j == len(p):
            ans = i == len(s)
        else:
            # Check if current characters match
            first_match = i < len(s) and (p[j] == s[i] or p[j] == ".")

            # If there is a '*' coming after the current pattern character
            if j + 1 < len(p) and p[j + 1] == '*':
                # Two cases
                # 1. '*' stands for zero occurrence -> skip current pattern char and '*'
                # 2. First match is true, so we try to consume one character in s and remain at same pattern index
                ans = dp(i, j + 2) or (first_match and dp(i + 1, j))
            else:
                # If no '*' comes after, we only check next characters in both s and p.
                ans = first_match and dp(i + 1, j + 1)

        memo[(i, j)] = ans
        return ans

    return dp(0, 0)

# Example test run
print(isMatch("ab", ".*a")) # Expected output: True
```

Python

```
class Solution {
    def isMatch(s: str, p: str) -> bool:
        n, m = len(s), len(p)
        f = [[False] * (m + 1) for _ in range(n + 1)]
        f[0][0] = True
        for i in range(n + 1):
            for j in range(1, m + 1):
                if p[j - 1] == ".":
                    f[i][j] = f[i][j - 1]
                elif s[i - 1] == p[j - 1] or s[i - 1] == p[j - 1] and p[j - 2] == '*':
                    f[i][j] = f[i - 1][j - 1] or f[i][j - 2]
                else:
                    f[i][j] = f[i - 1][j] or f[i][j - 1]
        return f[n][m]
```

Python

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        n, m = len(s), len(p)
        f = [[False] * (m + 1) for _ in range(n + 1)]
        f[0][0] = True
        for i in range(n + 1):
            for j in range(1, m + 1):
                if p[j - 1] == ".":
                    f[i][j] = f[i][j - 1]
                elif s[i - 1] == p[j - 1] or s[i - 1] == p[j - 1] and p[j - 2] == '*':
                    f[i][j] = f[i - 1][j - 1] or f[i][j - 2]
                else:
                    f[i][j] = f[i - 1][j] or f[i][j - 1]
        return f[n][m]
```

C++

C++

[*] Dynamic Programming — Pattern Solution (matched from community answers)

Python

```
# Function to find the longest common subsequence length
def longestCommonSubsequence(text1: str, text2: str) -> int:
    n, m = len(text1), len(text2)
    # Initialize a (n+1) x (m+1) dp table filled with zeros
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    # Build the DP table bottom-up
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            # If characters match, extend the subsequence length by 1
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                # Otherwise, choose the maximum from the left or top cell
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[n][m]

# Example usage
if __name__ == "__main__":
    text1 = "abcde"
    text2 = "ace"
    print(longestCommonSubsequence(text1, text2)) # Output: 3
```

#10 **HARD**

Regular Expression Matching

[LeetCode](#) · [SimpleList](#) · [WahCCE](#) · [LeetCode](#) · [Editorial](#)

String · Dynamic Programming · Recursion

List: Denny Zhang

Problem:

Given an input string s and a pattern p, implement regular expression matching with support for '.' and '*' where:

• '.' Matches any single character.

• '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".

Example 2:

Input: s = "aa", p = "a*"
Output: true
Explanation: 'a' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

```
class Solution {
    def isMatch(s: str, p: str) -> bool:
        n = len(s)
        m = len(p)
        dp = [[False] * (m + 1) for _ in range(n + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return j == 0 and p[j] == '.' or s[i] == p[j]

        for j, c in enumerate(p):
            if c == '.' and dp[0][j - 1]:
                dp[0][j] = True

        for i in range(1, n + 1):
            for j in range(1, m + 1):
                # The minimum index of c in p
                subpattern = dp[i + 1][j - 1]
                subpattern = isMatch(i, j - 1) and dp[i][j - 1]
                dp[i][j] = True if subpattern or subpattern
                elif isMatch(i, j):
                    dp[i + 1][j + 1] = dp[i][j]

        return dp[n][m]
```

Python

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        memo = {}
        def dp(i: int, j: int):
            if j == 0:
                return i == 0
            if j == 1 and p[j] == '.':
                return dp(i, 0)
            if i == 0 and (p[j] == '.' or p[j] == s[i]):
                return dp(i, j + 1)
            if i == 0 and (p[j] == '.' or p[j] == s[i]):
                return dp(i, j + 1)
            return i == 0 and (s[i] == p[j] or p[j] == '.') and dp(i + 1, j + 1)

        n, m = len(s), len(p)
        return dp(0, 0)
```

Python

```
class Solution {
    // s: schar String
    // p: pchar String
    // dp: dp char
    // dp[i][j] = true if s[0..i-1] matches p[0..j-1]

    function isMatch(s, p) {
        n = s.length;
        m = p.length;
        dp = array_fill(0, m + 1, array_fill(0, n + 1, 0));
        dp[0][0] = true;
        for (i = 0; i < n; i++) {
            for (j = 0; j < m; j++) {
                if (s[i] == p[j] || p[j] == '.') {
                    dp[i + 1][j + 1] = dp[i][j];
                } else if (p[j] == '*') {
                    // s[i] == p[j-1] || p[j-1] == '.'
                    dp[i + 1][j + 1] = dp[i][j] || dp[i + 1][j];
                }
            }
        }
        return dp[n][m];
    }
}
```

C++

Java
java

Dynamic Programming · LeetMastery — By Pattern — 2653 — LeetMastery

JavaScript
JavaScript

Dynamic Programming · LeetMastery — By Pattern — 2654 — LeetMaster

```
rust
//rust

impl Solution {
    pub fn isMatch(s: String, p: String) -> bool {
        let n = s.len();
        let m = p.len();
        let s = s.chars().collect::();
        let p = p.chars().collect::();

        let mut dp = vec![vec![false; m + 1]; n + 1];

        // Initialize the dp vector
        dp[0][0] = true;

        for i in 1..=n {
            for j in 1..=m {
                if p[j] == '?' {
                    dp[i][j] = dp[i][j - 1];
                }
            }
        }

        // Update the actual dp process
        for i in 1..=n {
            for j in 1..=m {
                if s[i] == p[j] || p[j] == '?' {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }
    }
}
```

Dynamic Programming · LeetMastery — By Pattern — 2655 — LeetMastery

Dynamic Programming · LeetMastery — By Pattern — 2656 — LeetMaster

Python

C++

```

class Solution {
public:
    int numDistinct(string s, string t) {
        const int n = s.length();
        const int m = t.length();
        vector<vector<unsigned long>> dp(n + 1, vector<unsigned long>(m + 1));

        for (int i = 0; i <= n; ++i)
            dp[i][0] = 1;

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j)
                if (s[i] - 'a' == t[j] - 'a')
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
                else
                    dp[i][j] = dp[i - 1][j];

        return dp[n][m];
    }
};

```

Java

Java

```

class Solution {
    public int numDistinct(String s, String t) {
        final int n = s.length();
        final int m = t.length();
        long[][] dp = new long[n + 1][m + 1];

        for (int i = 0; i <= n; ++i)
            dp[i][0] = 1;

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j)
                if (s.charAt(i - 1) == t.charAt(j - 1))
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
                else
                    dp[i][j] = dp[i - 1][j];

        return (int) dp[n][m];
    }
}

```

Java

```

class Solution {
    public int numDistinct(String s, String t) {
        int n = s.length(), m = t.length();
        int[] f = new int[n + 1];
        for (int i = 0; i <= n; ++i)
            f[i][0] = 1;

        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j)
                if (s.charAt(i - 1) == t.charAt(j - 1))
                    f[i][j] = f[i - 1][j - 1] + f[i - 1][j];
                else
                    f[i][j] = f[i - 1][j];

        return f[n][m];
    }
}

```

Java

```

class Solution {
    public int numDistinct(String s, String t) {
        int n = s.length();
        int[] f = new int[n + 1];
        f[0] = 1;
        for (char a : s.toCharArray())
            for (int j = m; j >= 0; --j)
                if (a == t.charAt(j))
                    f[j + 1] = f[j] + f[j + 1];

        return f[m];
    }
}

```

Java

```

class Solution {
    public int numDistinct(String s, String t) {
        int n = s.length();
        int[] f = new int[n + 1];
        f[0] = 1;
        for (char a : s.toCharArray())
            for (int j = m; j >= 0; --j)
                if (a == t.charAt(j))
                    f[j + 1] = f[j] + f[j + 1];

        return f[m];
    }
}

```

TypeScript

TypeScript

```

function numDistinct(s: string, t: string): number {
    const n = s.length;
    const m = t.length;
    const f: number[][] = new Array(n + 1).fill(0).map(() => new Array(m + 1).fill(0));
    for (let i = 0; i <= n; ++i) {
        f[i][0] = 1;
    }
    for (let i = 1; i <= n; ++i) {
        for (let j = 1; j <= m; ++j) {
            if (s[i] - 'a' == t[j] - 'a') {
                f[i][j] = f[i - 1][j - 1] + f[i - 1][j];
            } else {
                f[i][j] = f[i - 1][j];
            }
        }
    }
    return f[n][m];
}

```

TypeScript

```

function numDistinct(s: string, t: string): number {
    const n = s.length;
    const f: number[] = new Array(n + 1).fill(0);
    f[0] = 1;
    for (const a of s) {
        for (let i = m; i >= 0; --i) {
            const b = t[i] - 'a';
            if (a === b) {
                f[i + 1] = f[i] + f[i + 1];
            }
        }
    }
    return f[m];
}

```

TypeScript

```

function numDistinct(s: string, t: string): number {
    const n = s.length;
    const f: number[] = new Array(n + 1).fill(0);
    f[0] = 1;
    for (const a of s) {
        for (let i = m; i >= 0; --i) {
            const b = t[i] - 'a';
            if (a === b) {
                f[i + 1] = f[i] + f[i + 1];
            }
        }
    }
    return f[m];
}

```

Go

Go

```

func numDistinct(s string, t string) int {
    n, m := len(s), len(t)
    f := make([]int, m+1)
    for i := range f {
        f[i] = make([]int, m+1)
    }
    for i := 0; i <= n; i++ {
        for j := 0; j <= m; j++ {
            f[i][j] = f[i-1][j]
            if i>0 && s[i-1]==t[j-1] {
                f[i][j] += f[i-1][j-1]
            }
        }
    }
    return f[n][m]
}

```

Go

```

func numDistinct(s string, t string) int {
    n := len(s)
    f := make([]int, m+1)
    f[0] = 1
    for i, a := range s {
        for j := m; j >= 0; j-- {
            if b := t[j-1]; bytes.IndexByte(s, b) < i {
                f[j+1] = f[j] + f[j+1]
            }
        }
    }
    return f[m]
}

```

Go

```

func numDistinct(s string, t string) int {
    n := len(s)
    f := make([]int, m+1)
    f[0] = 1
    for i, a := range s {
        for j := m; j >= 0; j-- {
            if b := t[j-1]; bytes.IndexByte(s, b) < i {
                f[j+1] = f[j] + f[j+1]
            }
        }
    }
    return f[m]
}

```

Rust

Rust

```

impl Solution {
    pub fn num_distinct(s: String, t: String) -> i32 {
        let n = s.len();
        let m = t.len();
        let mut dp: Vec<Vec<u64>> = vec![vec![0; n + 1]; m + 1];

        // Initialize the dp vector
        for i in 0..=m {
            dp[i][0] = 1;
        }

        // Begin the actual dp process
        for i in 1..=n {
            for j in 1..=m {
                dp[i][j] = if s.as_bytes()[i - 1] == t.as_bytes()[j - 1] {
                    dp[i - 1][j - 1] + dp[i - 1][j]
                } else {
                    dp[i - 1][j]
                };
            }
        }
        dp[n][m] as i32
    }
}

```

Rust

```

impl Solution {
    pub fn num_distinct(s: String, t: String) -> i32 {
        let n = s.len();
        let m = t.len();
        let mut dp: Vec<Vec<u64>> = vec![vec![0; n + 1]; m + 1];

        // Initialize the dp vector
        for i in 0..=m {
            dp[i][0] = 1;
        }

        // Begin the actual dp process
        for i in 1..=n {
            for j in 1..=m {
                dp[i][j] = if s.as_bytes()[i - 1] == t.as_bytes()[j - 1] {
                    dp[i - 1][j - 1] + dp[i - 1][j]
                } else {
                    dp[i - 1][j]
                };
            }
        }
        dp[n][m] as i32
    }
}

```

[*] Dynamic Programming · Pattern Solution (matched from community answers)

Python

```

# Python solution for Distinct Subsequences
def numDistinct(s: str, t: str) -> int:
    # Get lengths of the strings s and t
    m, n = len(s), len(t)
    # Initialize a dp table with (m+1) rows and (n+1) columns filled with 0
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base case: An empty string t (j = 0) is a subsequence of any prefix of s, so set dp[i][0] = 1
    for i in range(m + 1):
        dp[i][0] = 1

    # Fill the dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If the current characters match, add the count of sequences by either using or skipping s[i-1]
            if s[i-1] == t[j-1]:
                dp[i][j] = dp[i-1][j-1] + dp[i-1][j]
            else:
                # If they do not match, skip s[i-1]
                dp[i][j] = dp[i-1][j]

    # The answer is the one for the full s and t
    return dp[m][n]

# Example usage:
print(numDistinct("rabbbit", "rabbit")) # Output: 3

```

#123

HARD

Best Time to Buy and Sell Stock III

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Array · Dynamic Programming

Like · Denny Zhang

Problem:

You are given an array prices where prices[i] is the price of a given stock on the i-th day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: prices = [3,3,5,0,0,3,1,4]

Output: 6

Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.

Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = 4-1 = 3.

Example 2:

Input: prices = [1,2,3,4,5]

Output: 4

Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.

Note that you cannot buy on day 1, buy on day 2 and sell then later, as you are engaging multiple transactions at the same time. You must sell before buying again.

Example 3:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transaction is done, i.e. max profit = 0.

Constraints:

- 1 <= prices.length <= 105
- 0 <= prices[i] <= 105

>> Brute Force [Dynamic Programming] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def max_profit(self, prices):
        # Brute Force O(N^2) - Basic recursion, no memoization
        def recurse(i):
            if i == len(prices): return 0
            take = prices[i] + recurse(i + 1)
            skip = recurse(i + 1)
            return max(take, skip)
        return recurse(0)
```

Python

Python

```
# Python solution for Best Time to Buy and Sell Stock III
def max_profit(prices):
    # If there are less than 2 days, no transaction can be made.
    if not prices or len(prices) < 2:
        return 0

    n = len(prices)
    # Initialize left and right profit arrays with zeros.
    left_profits = [0] * n
    right_profits = [0] * n

    # Forward pass: compute max profit until day i with one transaction.
    min_price = prices[0]
    for i in range(1, n):
        min_price = min(min_price, prices[i])
        # Maximum profit if sold on day i
        left_profits[i] = max(left_profits[i - 1], prices[i] - min_price)

    # Backward pass: compute max profit from day i to end with one transaction.
    max_price = prices[-1]
    for i in range(n - 2, -1, -1):
        # Update max price seen from the right.
        max_price = max(max_price, prices[i])
        right_profits[i] = max(right_profits[i + 1], max_price - prices[i])
```

```
# Maximum profit possible starting from day i.
right_profits[i] = max(right_profits[i + 1], max_price - prices[i])

# Compute the max profit by finding the maximum total profit.
max_total_profit = 0
for i in range(n):
    max_total_profit = max(max_total_profit, left_profits[i] + right_profits[i])

return max_total_profit

# Example usage:
prices = [3,3,0,0,2,1,4]
print(max_profit(prices)) # Expected output: 6
```

Python

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        sellTwo = 0
        holdTwo = -math.inf
        sellOne = 0
        holdOne = -math.inf

        for price in prices:
            sellTwo = max(sellTwo, holdTwo + price)
            holdTwo = max(holdTwo, sellOne - price)
            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)

        return sellTwo
```

Python

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # Initialize variables
        f1, f2, f3, f4 = -prices[0], 0, -prices[0], 0

        for price in prices[1:]:
            f1 = max(f1, -price)
            f2 = max(f2, f1 + price)
            f3 = max(f3, f2 - price)
            f4 = max(f4, f3 + price)

        return f4
```

Python

```
...
- f1 [11111111111111111111]
- f2 [111111111111111111]
- f3 [111111111111111111]
- f4 [111111111111111111]
...

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # Initialize variables
        f1, f2, f3, f4 = -prices[0], 0, -prices[0], 0
        for price in prices[1:]:
            f1 = max(f1, -price)
            f2 = max(f2, f1 + price)
            f3 = max(f3, f2 - price)
            f4 = max(f4, f3 + price)

        return f4

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        if not prices:
            return 0

        # Find max profit
        left_max = [0] * len(prices)

        temp_min = prices[0]
        for i in range(len(prices)):
            temp_min = min(temp_min, prices[i])
            left_max[i] = max(left_max[i - 1], prices[i] - temp_min) if i > 0 else 0

        # i to end, max profit
        right_max = [0] * len(prices)

        temp_max = prices[-1]
        for i in range(len(prices) - 1, -1, -1):
            temp_max = max(temp_max, prices[i])
            right_max[i] = max(right_max[i + 1], temp_max - prices[i]) if i < len(prices) - 1 else 0

        return max(left_max[i] + right_max[i] for i in range(len(prices)))

#####

class Solution(object):
    def maxProfit(self, prices):
        #type: prices: List[int]
        #type: int
        buy1 = buy2 = float("-inf")
```

```
sell1 = sell2 = 0

for i in range(len(prices)):
    sell1 = max(prices[i] - buy1, sell1)
    buy1 = min(buy1, prices[i])
    sell2 = max(sell1, prices[i] - buy2)
    buy2 = min(buy2, prices[i])
return max(sell1, sell2)
```

C++

C++

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int sellTwo = 0;
        int holdTwo = INT_MIN;
        int sellOne = 0;
        int holdOne = INT_MIN;

        for (const int price : prices) {
            sellTwo = max(sellTwo, holdTwo + price);
            holdTwo = max(holdTwo, sellOne - price);
            sellOne = max(sellOne, holdOne + price);
            holdOne = max(holdOne, -price);
        }

        return sellTwo;
    }
};
```

C++

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int f1 = -prices[0], f2 = 0, f3 = -prices[0], f4 = 0;
        for (int i = 1; i < prices.size(); ++i) {
            f1 = max(f1, -prices[i]);
            f2 = max(f2, f1 + prices[i]);
            f3 = max(f3, f2 - prices[i]);
            f4 = max(f4, f3 + prices[i]);
        }

        return f4;
    }
};
```

C++

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int f1 = -prices[0], f2 = 0, f3 = -prices[0], f4 = 0;
        for (int i = 1; i < prices.size(); ++i) {
            f1 = max(f1, -prices[i]);
            f2 = max(f2, f1 + prices[i]);
            f3 = max(f3, f2 - prices[i]);
            f4 = max(f4, f3 + prices[i]);
        }

        return f4;
    }
};
```

Java

Java

```
class Solution {
    public int maxProfit(int[] prices) {
        int sellTwo = 0;
        int holdTwo = Integer.MIN_VALUE;
        int sellOne = 0;
        int holdOne = Integer.MIN_VALUE;

        for (final int price : prices) {
            sellTwo = Math.max(sellTwo, holdTwo + price);
            holdTwo = Math.max(holdTwo, sellOne - price);
            sellOne = Math.max(sellOne, holdOne + price);
            holdOne = Math.max(holdOne, -price);
        }

        return sellTwo;
    }
}
```

Java

```
class Solution {
    public int maxProfit(int[] prices) {
        // Initialize variables
        int f1 = -prices[0], f2 = 0, f3 = -prices[0], f4 = 0;
        for (int i = 1; i < prices.length; ++i) {
            f1 = Math.max(f1, -prices[i]);
            f2 = Math.max(f2, f1 + prices[i]);
            f3 = Math.max(f3, f2 - prices[i]);
            f4 = Math.max(f4, f3 + prices[i]);
        }

        return f4;
    }
}
```

Java

```
public class Solution {
    public int MaxProfit(int[] prices) {
        int f1 = -prices[0], f2 = 0, f3 = -prices[0], f4 = 0;
        for (int i = 1; i < prices.Length; ++i) {
            f1 = Math.Max(f1, -prices[i]);
            f2 = Math.Max(f2, f1 + prices[i]);
            f3 = Math.Max(f3, f2 - prices[i]);
            f4 = Math.Max(f4, f3 + prices[i]);
        }

        return f4;
    }
}
```

Java

```
public class Solution {
    public int MaxProfit(int[] prices) {
        int f1 = -prices[0], f2 = 0, f3 = -prices[0], f4 = 0;
        for (int i = 1; i < prices.Length; ++i) {
            f1 = Math.Max(f1, -prices[i]);
            f2 = Math.Max(f2, f1 + prices[i]);
            f3 = Math.Max(f3, f2 - prices[i]);
            f4 = Math.Max(f4, f3 + prices[i]);
        }

        return f4;
    }
}
```

TypeScript

TypeScript

```
function maxProfit(prices: number[]): number {
    let f1 = -prices[0], f2 = 0, f3 = -prices[0], f4 = 0;
    for (let i = 1; i < prices.length; ++i) {
        f1 = Math.max(f1, -prices[i]);
        f2 = Math.max(f2, f1 + prices[i]);
        f3 = Math.max(f3, f2 - prices[i]);
        f4 = Math.max(f4, f3 + prices[i]);
    }

    return f4;
}
```

TypeScript

```
function maxProfit(prices: number[]): number {
    let f1 = -prices[0], f2 = 0, f3 = -prices[0], f4 = 0;
    for (let i = 1; i < prices.length; ++i) {
        f1 = Math.max(f1, -prices[i]);
        f2 = Math.max(f2, f1 + prices[i]);
        f3 = Math.max(f3, f2 - prices[i]);
        f4 = Math.max(f4, f3 + prices[i]);
    }

    return f4;
}
```

Go

Go

```

func maxProfit(prices: [Int]) Int {
    f1, f2, f3, f4 := -prices[0], 0, -prices[0], 0
    for i := 1; i < len(prices); i++ {
        f1 = max(f1, -prices[i])
        f2 = max(f2, f1+prices[i])
        f3 = max(f3, f2-prices[i])
        f4 = max(f4, f3+prices[i])
    }
    return f4
}

Go

func maxProfit(prices: [Int]) Int {
    f1, f2, f3, f4 := -prices[0], 0, -prices[0], 0
    for i := 1; i < len(prices); i++ {
        f1 = max(f1, -prices[i])
        f2 = max(f2, f1+prices[i])
        f3 = max(f3, f2-prices[i])
        f4 = max(f4, f3+prices[i])
    }
    return f4
}

Rust
Rust

impl Solution {
    #[allow(dead_code)]
    pub fn maxProfit(prices: Vec<i32>) -> i32 {
        let mut f1 = -prices[0];
        let mut f2 = 0;
        let mut f3 = -prices[0];
        let mut f4 = 0;
        let n = prices.len();

        for i in 1..n {
            f1 = std::cmp::max(f1, -prices[i]);
            f2 = std::cmp::max(f2, f1 + prices[i]);
            f3 = std::cmp::max(f3, f2 - prices[i]);
            f4 = std::cmp::max(f4, f3 + prices[i]);
        }

        f4
    }
}

Rust

```

```

impl Solution {
    #[allow(dead_code)]
    pub fn max_profit(prices: Vec<i32>) -> i32 {
        let mut f1 = -prices[0];
        let mut f2 = 0;
        let mut f3 = -prices[0];
        let mut f4 = 0;
        let n = prices.len();

        for i in 1..n {
            f1 = std::cmp::max(f1, -prices[i]);
            f2 = std::cmp::max(f2, f1 + prices[i]);
            f3 = std::cmp::max(f3, f2 - prices[i]);
            f4 = std::cmp::max(f4, f3 + prices[i]);
        }

        f4
    }
}

Python

# Python solution for Best Time to Buy and Sell Stock 122

def max_profit(prices):
    # If there are less than 2 days, no transaction can be made.
    if not prices or len(prices) < 2:
        return 0

    n = len(prices)
    # Initialize left and right profit arrays with zeros.
    left_profits = [0] * n
    right_profits = [0] * n

    # Forward pass: compute max profit until day i with one transaction.
    min_price = prices[0]
    for i in range(1, n):
        # Update the min price encountered so far.
        min_price = min(min_price, prices[i])
        # Forward profit: i sold on day i.
        left_profits[i] = max(left_profits[i - 1], prices[i] - min_price)

    # Backward pass: compute max profit from day i to end with one transaction.
    max_price = prices[-1]
    for i in range(n - 2, -1, -1):
        # Update max price seen from the right.
        max_price = max(max_price, prices[i])

    # The maximum profit is the maximum of left and right profits.
    return max(left_profits, right_profits)

```

```

# Maximum profit possible starting from day i.
right_profits[i] = max(right_profits[i + 1], max_price - prices[i])

# Combine the two profits to find the maximum total profit.
max_total_profit = 0
for i in range(n):
    max_total_profit = max(max_total_profit, left_profits[i] + right_profits[i])

return max_total_profit

# Example usage:
prices = [3, 5, 0, 0, 3, 1, 4]
print(max_profit(prices)) # Expected output: 6

C++

// C++ solution for Best Time to Buy and Sell Stock 122
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

int maxProfit(vector<int>& prices) {
    if (prices.empty() || prices.size() < 2) return 0;
    int n = prices.size();
    vector<int> leftProfits(n, 0);
    vector<int> rightProfits(n, 0);

    // Forward pass: maximum profit until day i with one transaction
    int minPrice = prices[0];
    for (int i = 1; i < n; i++) {
        // Update the minimum price so far.
        minPrice = min(minPrice, prices[i]);
        leftProfits[i] = max(leftProfits[i - 1], prices[i] - minPrice);
    }

    // Backward pass: maximum profit from day i to end with one transaction
    int maxPrice = prices[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        // Update the maximum price from the right.
        maxPrice = max(maxPrice, prices[i]);
    }

    // Calculate the maximum total profit.
    int maxTotalProfit = 0;
    for (int i = 0; i < n; i++) {
        maxTotalProfit = max(maxTotalProfit, leftProfits[i] + rightProfits[i]);
    }

    return maxTotalProfit;
}

int main() {
    vector<int> prices = {3, 5, 0, 0, 3, 1, 4};
    cout << maxProfit(prices) << endl; // Expected output: 6
    return 0;
}

```

```

rightProfits[i] = max(rightProfits[i + 1], maxPrice - prices[i]);

// Combine results from both passes.
int maxTotalProfit = 0;
for (int i = 0; i < n; i++) {
    maxTotalProfit = max(maxTotalProfit, leftProfits[i] + rightProfits[i]);
}

return maxTotalProfit;

// Example usage:
int main() {
    vector<int> prices = {3, 5, 0, 0, 3, 1, 4};
    cout << maxProfit(prices) << endl; // Expected output: 6
    return 0;
}

```

#132 **HARD**
Palindrome Partitioning II
 LeetCode - [Description](#) - [Solution2](#) - [LeetCode](#) - [Editorial](#)
 String - [Dynamic Programming](#)
 List: Denny Zhang

Problem:
 Given a string s, partition s such that every substring of the partition is a palindrome.
 Return the minimum cuts needed for a palindrome partitioning of s.

Example 1:
 Input: s = "aab"
 Output: 1
 Explanation: The palindrome partitioning ["aa","b"] could be produced using 1 cut.

Example 2:
 Input: s = "a"
 Output: 0

Example 3:
 Input: s = "ab"
 Output: 1

Constraints:
 • 1 <= s.length <= 2000
 • s consists of lowercase English letters only.

Python
 Python

```

def minCut(s: str) -> int:
    n = len(s)
    # Pre-compute palindrome table where isPal[i][j] is True if s[i:j+1] is a palindrome.
    isPal = [[False] * n for _ in range(n)]
    # dp[i] will store the minimum cuts required for substring s[0...i]
    dp = [0] * n
    for i in range(n):
        min_cut = 4 # Maximum cuts (not every character)
        for j in range(0, i + 1):
            # Check if s[j:i+1] is a palindrome.
            if s[j] == s[i] and (i - j < 2 or isPal[j + 1][i - 1]):
                isPal[j][i] = True # Mark substring as palindrome.
            # If the minimum starts at index 0, no cut is needed.
            min_cut = 0 if j == 0 else min(min_cut, dp[j - 1] + 1)
        dp[i] = min_cut # Store minimum cuts needed for s[0...i]
    return dp[-1]

# Example usage:
print(minCut("aab")) # Output: 1

Python

class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        isPalindrome = [[False] * n for _ in range(n)]
        # dp[i] := the minimum cuts needed for a palindrome partitioning of s[0...i]
        dp = [0] * n

        for i in range(2, n + 1):
            for j in range(0, i - 1):
                if isPalindrome[j + 1][i] and isPalindrome[0][j]:
                    dp[i] = min(dp[i], dp[j] + 1)

        return dp[-1]

# Try all the possible partitions.
for j in range(0, i - 1):
    if isPalindrome[j + 1][i]:
        dp[i] = min(dp[i], dp[j] + 1)

return dp[-1]

Python

```

```

class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        g = [[False] * n for _ in range(n)]
        for i in range(1, n):
            for j in range(0, i):
                if s[j] == s[i] and (i - j < 2 or g[j + 1][i - 1]):
                    g[j][i] = True
        for i in range(1, n):
            min_cut = 4 # Maximum cuts (not every character)
            for j in range(0, i):
                if g[j + 1][i] and g[0][j]:
                    min_cut = 0 if j == 0 else min(min_cut, dp[j] + 1)
            dp[i] = min_cut
        return dp[-1]

# Try all the possible partitions.
for j in range(0, i - 1):
    if isPalindrome[j + 1][i]:
        dp[i] = min(dp[i], dp[j] + 1)

return dp[-1]

Python

class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        isPalindrome = [[False] * n for _ in range(n)]
        # dp[i] := the minimum cuts needed for a palindrome partitioning of s[0...i]
        dp = [0] * n

        for i in range(2, n + 1):
            for j in range(0, i - 1):
                if isPalindrome[j + 1][i] and isPalindrome[0][j]:
                    dp[i] = min(dp[i], dp[j] + 1)

        return dp[-1]

# Try all the possible partitions.
for j in range(0, i - 1):
    if isPalindrome[j + 1][i]:
        dp[i] = min(dp[i], dp[j] + 1)

return dp[-1]

Python

```

```

        return ans

        n = len(s)
        q = [[False] * n for _ in range(n)]
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                q[i][j] = s[i] == s[j] and q[i + 1][j - 1]
        ans = float('inf')
        dfs.cache_clear() # nice, clear cache !!!
        return ans

#####
class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        is_pal = [[False] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                is_pal[i][j] = s[i] == s[j] and (j - i < 3 or is_pal[i + 1][j - 1])
        min_cut = float('inf')
        for i in range(n):
            for j in range(i + 1, n):
                if is_pal[i][j]:
                    min_cut = min(min_cut[i], min_cut[j] - 1) + 1 if j > 0 else 0

        return min_cut - 1

```

C++

```

C++
class Solution {
public:
    int minCut(string s) {
        const int n = s.length();
        // isPalindrome[i][j] = true if s[i..j] is a palindrome
        vector<vector<bool>> isPalindrome(n, vector<bool>(n, true));
        // dp[i] = minimum cuts needed for a palindrome partitioning of s[0..i]
        vector<int> dp(n, n);

        for (int i = 2; i <= n; ++i)
            for (int l = 0; j = i - 1; j <= n; ++j)
                isPalindrome[i][j] = s[i] == s[j] && isPalindrome[i + 1][j - 1];

        for (int i = 0; i < n; ++i) {
            if (isPalindrome[0][i]) {
                dp[i] = 0;
                continue;
            }

            // Try all the possible partitions.
            for (int j = 0; j < i; ++j)
                if (isPalindrome[j + 1][i])
                    dp[i] = min(dp[i], dp[j] + 1);
        }
    }
}

```

```

        return dp.back();
    }
};

C++
class Solution {
public:
    int minCut(string s) {
        int n = s.size();
        vector<vector<bool>> dp(n, vector<bool>(n));
        for (int i = 0; i <= n; ++i) {
            for (int j = i; j < n; ++j) {
                dp[i][j] = s[i] == s[j] && (j - i < 3 || dp[i + 1][j - 1]);
            }
        }
        vector<int> dp2(n);
        for (int i = 0; i < n; ++i) {
            if (!dp[i][i]) {
                dp2[i] = n;
                for (int j = i; j <= i; ++j) {
                    if (dp[i + 1][j]) {
                        dp2[i] = min(dp2[i], dp2[j] - 1) + 1;
                    }
                }
            }
        }
        return dp2[n - 1];
    }
};

```

Java

```

Java
class Solution {
    public int minCut(String s) {
        int n = s.length();
        boolean[][] dp1 = new boolean[n][n];
        for (int i = 0; i < n; ++i) {
            for (int j = i; j < n; ++j) {
                dp1[i][j] = s.charAt(i) == s.charAt(j) && (j - i < 3 || dp1[i + 1][j - 1]);
            }
        }
        int[] dp2 = new int[n];
        for (int i = 0; i < n; ++i) {
            if (!dp1[i][i]) {
                dp2[i] = n;
                for (int j = i; j <= i; ++j) {
                    if (dp2[j + 1][i]) {
                        dp2[i] = Math.min(dp2[i], dp2[j] - 1) + 1;
                    }
                }
            }
        }
        return dp2[n - 1];
    }
}
//...

```

```

public class PalindromePartitioning_II {

    public class Solution {
        public int minCut(String s) {

            if (s == null || s.length() == 0) {
                return -1;
            }

            // 'abcba' , when reaching 'c', minCut=2
            // when reaching end 'a', minCut=4
            // when reaching end 'b', minCut=6
            // so, if a substring is palindrome, cut is prev_segment+1

            // set up a fast lookup for if palindrome from i to j
            boolean[][] isPal = new boolean[s.length()][s.length()];
            for (int i = 0; i < s.length(); i++) {
                for (int j = i; j < s.length(); j++) {
                    if (i == j || (s.charAt(i) == s.charAt(j) && (i - j == 1 || isPal[i + 1][j - 1]))) {
                        isPal[i][j] = true; // j is first, then i
                    }
                }
            }

            // set up min count array, worst case, each char is a palindrome
            int[] minCut = new int[s.length()];
            for (int i = 0; i < s.length(); i++) {
                minCut[i] = i;
            }

            // how to count, eg: 'aabcbca', 'aabcbca'
            // quote: if a substring is palindrome, cut is prev_segment+1
            for (int i = 0; i < s.length(); i++) {
                for (int j = i + 1; j < s.length(); j++) {
                    if (isPal[i][j]) {
                        if (i < 2 == 0) { // quote: eg: 'aabcbca'
                            minCut[j] = Math.min(minCut[j], 1 + minCut[i - 1]);
                        } else { // eg: 'aabcbca'
                            minCut[j] = 0;
                        }
                    }
                }
            }

            int min = minCut[s.length() - 1];
            return min;
        }
    }
}

```

Java

```

using System;
using System.Collections.Generic;

public class Solution {
    public int MinCut(string s) {
        if (s.Length == 0) return 0;

        var paths = new List<int>(s.Length);
        for (var i = 0; i < s.Length; ++i)
        {
            int j, k;
            for (j = i, k = i + 1; j <= 0 && k < s.Length; ++j, ++k)
            {
                if (s[j] == s[k])
                {
                    if (paths[k] == null)
                    {
                        paths[k] = new List<int>();
                    }
                    paths[k].Add(j - 1);
                }
                else
                {
                    break;
                }
            }

            for (j = i, k = i - 1; j >= 0 && k < s.Length; --j, ++k)
            {
                if (s[j] == s[k])
                {
                    if (paths[k] == null)
                    {
                        paths[k] = new List<int>();
                    }
                    paths[k].Add(j + 1);
                }
                else
                {
                    break;
                }
            }
        }

        var partCount = new int[s.Length];
        for (var i = 0; i < s.Length; ++i)
        {
            partCount[i] = int.MaxValue;
            if (paths[i] != null)
            {
                foreach (var path in paths[i])
                {

```

```

        {
            if (path == 0)
            {
                partCount[i] = 0;
                break;
            }
            else
            {
                partCount[i] = Math.Min(partCount[i], partCount[path]);
            }
        }
        partCount[i];
    }
    return partCount[s.Length - 1] - 1;
}

```

TypeScript

```

TypeScript
function minCut(s: string): number {
    const n = s.length;
    const g: boolean[][] = Array.from({ length: n }, () => Array(n).fill(true));
    for (let i = n - 1; i >= 0; --i) {
        for (let j = i + 1; j < n; ++j) {
            g[i][j] = s[i] === s[j] && g[i + 1][j - 1];
        }
    }
    const f: number[] = Array.from({ length: n }, (_, i) => i);
    for (let i = 1; i < n; ++i) {
        for (let j = 0; j <= i; ++j) {
            if (g[j][i]) {
                f[i] = Math.min(f[i], j + f[j] - 1 + 0);
            }
        }
    }
    return f[n - 1];
}

```

TypeScript

```

function minCut(s: string): number {
    const n = s.length;
    const g: boolean[][] = Array(n)
        .fill(0)
        .map(() => Array(n).fill(true));
    for (let i = n - 1; i >= 0; --i) {
        for (let j = i + 1; j < n; ++j) {
            g[i][j] = s[i] === s[j] && g[i + 1][j - 1];
        }
    }
    const f: number[] = Array(n)
        .fill(0)
        .map(() => i => i);
    for (let i = 1; i < n; ++i) {
        for (let j = 0; j <= i; ++j) {
            if (g[j][i]) {
                f[i] = Math.min(f[i], j + f[j] - 1 + 0);
            }
        }
    }
    return f[n - 1];
}

```

Go

```

Go
func minCut(s string) int {
    n := len(s)
    dp1 := make([][]bool, n)
    for i := 0; i < n; i++ {
        dp1[i] = make([]bool, n)
    }
    for i := n - 1; i >= 0; i-- {
        for j := i; j < n; j++ {
            dp1[i][j] = s[i] == s[j] && (j - i < 3 || dp1[i + 1][j - 1])
        }
    }
    dp2 := make([]int, n)
    for i := 0; i < n; i++ {
        if !dp1[i][i] {
            dp2[i] = i
            for j := i; j <= i; j++ {
                if dp1[i + 1][j] {
                    dp2[i] = min(dp2[i], dp2[j] - 1) + 1
                }
            }
        }
    }
    return dp2[n - 1]
}

func min(x, y int) int {
    if x < y {
        return x
    }
    return y
}

```

[*] Dynamic Programming -- Pattern Solution (matched from community answers)

Python

```
class Solution:
    def maxCoins(self, s: str) -> int:
        n = len(s)
        is_pal = [[False for j in range(0, len(s)) for i in range(0, len(s) + 1)]
        for i in range(0, len(s)):
            for j in range(0, i + 1):
                if (i == j) or (i + 1 == j) or (s[i] == s[j] and (is_pal[i - 1][j - 1] == True)):
                    is_pal[i][j] = True
                else:
                    is_pal[i][j] = False
        dp = [[0 for i in range(0, len(s) + 1) for j in range(0, len(s) + 1)]
        for i in range(0, len(s)):
            for j in range(i + 1, len(s) + 1):
                if (s[i] == s[j]):
                    dp[i][j] = dp[i + 1][j] + 1
                else:
                    dp[i][j] = 0
        return dp[0][n - 1]
```

Dynamic Programming · LeetCode · By Pattern — 2683 — LeetCode

#312 **HARD**

Burst Balloons

LeetCode · SimplifyLeetCode · WubACE · LeetCode · Editorial

Array · Dynamic Programming

Little Denny Zhang

Problem:

You are given n balloons, indexed from 0 to n - 1. Each balloon is painted with a number on it represented by an array nums. You are asked to burst all the balloons.

If you burst the i-th balloon, you will get nums[i - 1] * nums[i] * nums[i + 1] coins. If i - 1 or i + 1 goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return the maximum coins you can collect by bursting the balloons wisely.

Example 1:

Input: nums = [3,1,5,8]

Output: 167

Explanation:

nums = [3,1,5,8] -> [3,5,8] -> [3,8] -> [8] -> []

coins = 3*1*5 + 3*5*8 + 1*5*8 + 1*8*1 = 167

Example 2:

Input: nums = [1,5]

Output: 10

Constraints:

- n == nums.length
- 1 <= n <= 300
- 0 <= nums[i] <= 100

>> Brute Force [Dynamic Programming] Interview Prep

Python

Dynamic Programming · LeetCode · By Pattern — 2684 — LeetCode

Brute Force Approach

Python

```
class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        # Brute Force Approach: n! - try every order to burst all balloons
        from itertools import permutations
        n = len(nums)
        best = 0
        # Only feasible for tiny n: actual brute is exponential
        def burst(balloons, coins):
            nonlocal best
            if not balloons:
                best = max(best, coins)
                return
            for i in range(len(balloons)):
                left = balloons[:i]
                right = balloons[i+1:]
                gained = left[-1] * balloons[i] * right[0]
                burst(left + [balloons[i]], coins + gained)
            burst(right, coins)
        burst(nums, 0)
        return best
```

Dynamic Programming · LeetCode · By Pattern — 2685 — LeetCode

class Solution:

Python

```
def maxCoins(self, nums: List[int]) -> int:
    n = len(nums)
    arr = [1] * n
    for i in range(1, n - 1):
        for j in range(i + 1, n):
            arr[i][j] = max(arr[i][j], arr[i] * arr[j] * arr[i + 1])
    return arr[0][n - 1]
```

Dynamic Programming · LeetCode · By Pattern — 2686 — LeetCode

class Solution {

C++

```
public:
    int maxCoins(vector<int>& nums) {
        const int n = nums.size();
        vector<vector<int>>> dp(n, vector<vector<int>>>(n, vector<int>(n, 0)));
        for (int i = 0; i < n; ++i) {
            for (int j = i + 1; j < n; ++j) {
                dp[i][j] = max(dp[i][j], dp[i] * dp[j] * dp[i + 1]);
            }
        }
        return dp[0][n - 1];
    }
};
```

Dynamic Programming · LeetCode · By Pattern — 2687 — LeetCode

class Solution {

Java

```
public:
    int maxCoins(vector<int>& nums) {
        const int n = nums.size();
        vector<vector<int>>> dp(n, vector<vector<int>>>(n, vector<int>(n, 0)));
        for (int i = 0; i < n; ++i) {
            for (int j = i + 1; j < n; ++j) {
                dp[i][j] = max(dp[i][j], dp[i] * dp[j] * dp[i + 1]);
            }
        }
        return dp[0][n - 1];
    }
};
```

Dynamic Programming · LeetCode · By Pattern — 2688 — LeetCode

```

class Solution {
public: int maxCoins(int[] nums) {
    let n = nums.length;
    let arr = new int[n + 2];
    arr[0] = 1;
    arr[n + 1] = 1;
    System.Array.Copy(nums, 0, arr, 1, n);
    let f = new int[n + 2][n + 2];
    for (let i = n - 1; i >= 0; i--) {
        for (let j = i + 2; j <= n + 1; j++) {
            for (let k = i + 1; k < j; k++) {
                f[i][j] = Math.max(f[i][j], f[i][k] + f[k][j] + arr[i] + arr[k] + arr[j]);
            }
        }
    }
    return f[0][n + 1];
}
}

```

Java

```

class Solution {
public: int maxCoins(int[] nums) {
    int n = nums.length + 2;
    vector<int> arr(n, 1);
    System.arraycopy(nums, 0, arr, 1, nums.length);
    int f[n][n];
    for (let i = n - 2; i <= 0; i++) {
        for (let j = i + 2; j <= n + 1; j++) {
            for (let k = i + 1; k < j; k++) {
                f[i][j] = Math.max(f[i][j], f[i][k] + f[k][j] + arr[i] + arr[k] + arr[j]);
            }
        }
    }
    return f[0][n + 1];
}
}

```

TypeScript
TypeScript

Dynamic Programming — LeetCode — By Pattern — 2689 —

LeetCode

```

function maxCoins(nums: number[]): number {
    const n = nums.length;
    const arr = Array(n + 2).fill(1);
    for (let i = 0; i < n; i++) {
        arr[i + 1] = nums[i];
    }

    const f: number[][] = Array.from({ length: n + 2 }, () => Array(n + 2).fill(0));
    for (let i = n - 1; i >= 0; i--) {
        for (let j = i + 2; j <= n + 1; j++) {
            for (let k = i + 1; k < j; k++) {
                f[i][j] = Math.max(f[i][j], f[i][k] + f[k][j] + arr[i] + arr[k] + arr[j]);
            }
        }
    }
    return f[0][n + 1];
}

```

TypeScript

```

function maxCoins(nums: number[]): number {
    let n = nums.length;
    let dp = Array.from({ length: n + 1 }, v => new Array(n + 2).fill(0));
    nums.unshift(1);
    nums.push(1);
    for (let i = n - 1; i >= 0; i--) {
        for (let j = i + 2; j <= n + 1; j++) {
            for (let k = i + 1; k < j; k++) {
                dp[i][j] = Math.max(dp[i][j] + nums[k] + dp[i][k] + dp[k][j], dp[i][j]);
            }
        }
    }
    return dp[0][n + 1];
}

```

Go
Go

Dynamic Programming — LeetCode — By Pattern — 2690 —

LeetCode

```

func maxCoins(nums []int) int {
    n := len(nums)
    arr := make([]int, n+2)
    arr[0] = 1
    copy(arr[1:], nums)
    f := make([][]int, n+2)
    for i := range f {
        f[i] = make([]int, n+2)
    }
    for i := n - 1; i >= 0; i-- {
        for j := i + 2; j <= n + 1; j++ {
            for k := i + 1; k < j; k++ {
                f[i][j] = max(f[i][j], f[i][k] + f[k][j] + arr[i] + arr[k] + arr[j])
            }
        }
    }
    return f[0][n+1]
}

```

Go

```

func maxCoins(nums []int) int {
    n := len(nums)
    arr := make([]int, n+2)
    for i := 0; i < len(nums); i++ {
        arr[i+1] = nums[i]
    }
    n = len(arr)
    dp := make([][]int, n)
    for i := 0; i < n; i++ {
        dp[i] = make([]int, n)
    }
    for i := 2; i < n; i++ {
        for j := 0; j < i; j++ {
            for k := j + 1; k < i; k++ {
                dp[i][j] = max(dp[i][j], dp[j][k] + dp[k][i] + arr[j] + arr[k] + arr[i])
            }
        }
    }
    return dp[0][n-1]
}

```

Rust
Rust

Dynamic Programming — LeetCode — By Pattern — 2691 —

LeetCode

```

impl Solution {
    pub fn max_coins(nums: Vec<i32>) -> i32 {
        let n = nums.len();
        let mut arr = vec![1; n + 2];
        for i in 0..n {
            arr[i + 1] = nums[i];
        }
        let mut f = vec![vec![0; n + 2]; n + 2];
        for i in 0..n {
            for j in i + 2..n + 2 {
                for k in i + 1..j {
                    f[i][j] = f[i][j].max(f[i][k] + f[k][j] + arr[i] + arr[k] + arr[j]);
                }
            }
        }
        f[0][n + 1]
    }
}

```

Rust

```

impl Solution {
    pub fn max_coins(nums: Vec<i32>) -> i32 {
        let n = nums.len();
        let mut arr = vec![1; n + 2];
        for i in 0..n {
            arr[i + 1] = nums[i];
        }
        let mut f = vec![vec![0; n + 2]; n + 2];
        for i in 0..n {
            for j in i + 2..n + 2 {
                for k in i + 1..j {
                    f[i][j] = f[i][j].max(f[i][k] + f[k][j] + arr[i] + arr[k] + arr[j]);
                }
            }
        }
        f[0][n + 1]
    }
}

```

[1] Dynamic Programming — Pattern Solution (matched from community answers)
Python

Dynamic Programming — LeetCode — By Pattern — 2692 —

LeetCode

```

# Define the function to compute maximum coins
def maxCoins(stones):
    # Get initial balloons with value 1 at both boundaries
    n = len(stones)
    extended = [1] * (n + 2)
    # Initialize dp table with 0's; dimensions: (n+2) x (n+2)
    dp = [[0] * (n + 2) for _ in range(n + 2)]

    # Calculate dp values for intervals with length 2 to n+1
    for length in range(2, n + 2):
        # Iterate over all possible last burst balloons in the interval (left, right)
        for left in range(2, n + 2 - length + 1):
            right = left + length
            # Iterate through all possible last burst balloons in the interval (left, right)
            for k in range(left + 1, right):
                # Calculate the maximum coins for the interval (left, right)
                # value from bursting: extended[left] + extended[k] + extended[right]
                # plus coins from subsequent [left, k] and [k, right]
                coins = extended[left] + extended[k] + extended[right] + dp[left][k] + dp[k][right]
                dp[left][right] = max(dp[left][right], coins)

    # The final answer is stored in dp[0][n+1] which represents bursting all real balloons.
    return dp[0][n+1]

# Example usage
print(maxCoins([3,3,5,8])) # Expected output: 167
print(maxCoins([1,5])) # Expected output: 10

```

#1000 **HARD**
Minimum Cost to Merge Stones
LeetCode — SimplyLeetCode — LeetCode — Editorial
Array — Dynamic Programming
Like · Donny Zhang

Problem:
There are n piles of stones arranged in a row. The i th pile has $stones[i]$ stones.
A move consists of merging exactly k consecutive piles into one pile, and the cost of this move is equal to the total number of stones in these k piles.
Return the minimum cost to merge all piles of stones into one pile. If it is impossible, return -1 .
Example 1:
Input: $stones = [3,2,4,1]$, $k = 2$
Output: 20
Explanation: We start with $[3, 2, 4, 1]$.
We merge $[3, 2]$ for a cost of 5, and we are left with $[5, 4, 1]$.
We merge $[4, 1]$ for a cost of 5, and we are left with $[5, 5]$.
We merge $[5, 5]$ for a cost of 10, and we are left with $[10]$.
The total cost was 20, and this is the minimum possible.
Example 2:
Input: $stones = [3,2,4,1]$, $k = 3$
Output: -1

Dynamic Programming — LeetCode — By Pattern — 2693 —

LeetCode

Explanation: After any merge operation, there are 2 piles left, and we can't merge anymore.
So the task is impossible.
Example 3:
Input: $stones = [3,5,1,2,6]$, $k = 3$
Output: 25
Explanation: We start with $[3, 5, 1, 2, 6]$.
We merge $[5, 1, 2]$ for a cost of 8, and we are left with $[3, 8, 6]$.
We merge $[3, 8, 6]$ for a cost of 17, and we are left with $[17]$.
The total cost was 25, and this is the minimum possible.

Constraints:
 $n == stones.length$
 $1 \leq n \leq 30$
 $1 \leq stones[i] \leq 100$
 $2 \leq k \leq 30$

Python
Python

```

# Python solution for merging stones with detailed comments.
class Solution:
    def mergeStones(self, stones: List[int], k: int) -> int:
        n = len(stones)
        # Check for merge possibility.
        if (n - 1) % (k - 1) != 0:
            return -1

        # Precompute prefix sum for fast range sum queries.
        prefix = [0] * (n + 1)
        for i in range(n):
            prefix[i + 1] = prefix[i] + stones[i]

        # Initialize dp table. dp[i][j] represents minimal cost to merge stones[i..j].
        dp = [[0] * n for _ in range(n)]

        # Process all possible subranges with lengths >= k.
        for length in range(k, n + 1):
            for i in range(0, n - length + 1):
                j = i + length - 1
                dp[i][j] = float('inf')
                # Try all valid partitions with step size (k-1).
                for mid in range(i + k - 1, j, k - 1):
                    dp[i][j] = min(dp[i][j], dp[i][mid] + dp[mid+1][j])
                # If this segment can be merged into one pile, add cost.
                if (length - 1) % (k - 1) == 0:
                    dp[i][j] = prefix[j+1] - prefix[i]

        return dp[0][n-1]

# Example usage:
# solution = Solution()
# print(solution.mergeStones([1, 2, 4, 1, 1], 2)) # Expected output: 20

```

Python

Dynamic Programming — LeetCode — By Pattern — 2694 —

LeetCode

```

class Solution {
    def mergeStones(stones: List[Int], K: Int) => Int:
        n = stones.length
        if (n == 1) % (K - 1) == 0:
            return 0
        s = List(accumulate(stones, initial=0))
        f = List.fill(n - 1) { _ in range(n - 1) } for _ in range(n - 1)
        for i in range(1, n - 1):
            f[i][i] = 0
            for j in range(i + 1, n):
                for k in range(i + 1, j + 1):
                    f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j] + f[n - 1][j] - 1)
            return f[i][j]

```

Python

```

class Solution {
    def mergeStones(stones: List[Int], K: Int) => Int:
        n = stones.length
        if (n == 1) % (K - 1) == 0:
            return 0
        s = List(accumulate(stones, initial=0))
        f = List.fill(n - 1) { _ in range(n - 1) } for _ in range(n - 1)
        for i in range(1, n - 1):
            f[i][i] = 0
            for j in range(i + 1, n):
                for k in range(i + 1, j + 1):
                    f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j] + f[n - 1][j] - 1)
            return f[i][j]

```

C++
C++

```

class Solution {
    public:
        int mergeStones(vector<int>& stones, int K) {
            const int n = stones.size();
            vector<vector<int>>& dp = mem(
                n, vector<int>(K - 1, 0));
            vector<int> prefix(n + 1);
            partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);
            const int cost = mergeStones(stones, 0, n - 1, K, prefix, dp);
            return cost == KMax ? -1 : cost;
        }
    private:
        static constexpr int KMax = 1'000'000'000;

        // Returns the minimum cost to merge stones[i..j] into k piles.
        int mergeStones(const vector<int>& stones, int i, int j, int k, int K,
            const vector<int>& prefix, vector<vector<int>>& dp) {
            if (i == j)
                return KMax;
            if (K == 1)
                return KMax;
            if (mem[i][j][K] != KMax)
                return mem[i][j][K];
            if (K == 1)
                return mem[i][j][K];
            return mem[i][j][K];
        }
};

```

C++

```

class Solution {
    public:
        int mergeStones(vector<int>& stones, int K) {
            const int n = stones.size();
            if ((n - 1) % (K - 1) != 0)
                return -1;
            vector<vector<int>> dp(n, vector<int>(K - 1, 0));
            vector<int> prefix(n + 1);
            partial_sum(stones.begin(), stones.end(), prefix.begin() + 1);
            const int cost = mergeStones(stones, 0, n - 1, K, prefix, dp);
            return cost == KMax ? -1 : cost;
        }
    private:
        static constexpr int KMax = 1'000'000'000;

        // Returns the minimum cost to merge stones[i..j] into k piles.
        int mergeStones(const vector<int>& stones, int i, int j, int k,
            const vector<int>& prefix, vector<vector<int>>& dp) {
            if (i == j)
                return K;
            if (mem[i][j][K] != KMax)
                return mem[i][j][K];
            return mem[i][j][K];
        }
};

```

C++

```

class Solution {
    public:
        int mergeStones(vector<int>& stones, int K) {
            int n = stones.size();
            if ((n - 1) % (K - 1) != 0)
                return -1;
            int s[n];
            for (int i = 1; i <= n; ++i)
                s[i] = s[i - 1] + stones[i - 1];
            int f[n][n][K];
            memset(f, 0, sizeof(f));
            for (int i = 1; i <= n; ++i)
                f[i][i][1] = 0;
            for (int i = 2; i <= n; ++i)
                for (int j = i; j <= n; ++j)
                    for (int k = 1; k <= K; ++k)
                        f[i][j][k] = min(f[i][j][k], f[i][i][1] + f[i + 1][j] + f[n - 1][j] - 1);
            return f[1][n][K];
        }
};

```

Java

Java

```

class Solution {
    public int mergeStones(int[] stones, int K) {
        final int n = stones.length;
        int[][] mem = new int[n][K + 1];
        int[] prefix = new int[n + 1];
        Arrays.stream(mem).forEach(A -> Arrays.stream(A).forEach(B -> Arrays.fill(B, MAX)));
        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stones[i];
        final int cost = mergeStones(stones, 0, n - 1, 1, K, prefix, mem);
        return cost == MAX ? -1 : cost;
    }
    private static final int MAX = 1'000'000'000;

    // Returns the minimum cost to merge stones[i..j] into k piles.
    private int mergeStones(final int[] stones, int i, int j, int k, int K, int[] prefix,
        int[][] mem) {
        if ((j - i + 1) % (K - 1) != 0)
            return MAX;
        if (i == j)
            return 0;
        if (mem[i][j][K] != MAX)
            return mem[i][j][K];
        if (K == 1)
            return mem[i][j][K];
        return mem[i][j][K];
        for (int m = 1; m <= K - 1; ++m)
            mem[i][j][m] = Math.min(mem[i][j][m], mergeStones(stones, i, m, 1, K, prefix, mem) +
                mergeStones(stones, m + 1, j, K - 1, K, prefix, mem));
        return mem[i][j][K];
    }
}

```

Java

```

class Solution {
    public int mergeStones(int[] stones, int K) {
        final int n = stones.length;
        if ((n - 1) % (K - 1) != 0)
            return -1;
        int[] mem = new int[n];
        int[] prefix = new int[n + 1];
        Arrays.stream(mem).forEach(A -> Arrays.fill(A, MAX));
        for (int i = 0; i < n; ++i)
            prefix[i + 1] = prefix[i] + stones[i];
        final int cost = mergeStones(stones, 0, n - 1, K, prefix, mem);
        return cost == MAX ? -1 : cost;
    }
    private static final int MAX = 1'000'000'000;

    // Returns the minimum cost to merge stones[i..j] into k piles.
    private int mergeStones(final int[] stones, int i, int j, int k, int[] prefix, int[][] mem) {
        if ((j - i + 1) % (K - 1) != 0)
            return MAX;
        if (i == j)
            return 0;
        if (mem[i][j][K] != MAX)
            return mem[i][j][K];
        if (K == 1)
            return mem[i][j][K];
        return mem[i][j][K];
        for (int m = 1; m <= K - 1; ++m)
            mem[i][j][m] = Math.min(mem[i][j][m], mergeStones(stones, i, m, 1, K, prefix, mem) +
                mergeStones(stones, m + 1, j, K - 1, K, prefix, mem));
        return mem[i][j][K];
    }
}

```

Java

```

class Solution {
public: int mergeStones(vector<int>& stones, int K) {
    int n = stones.length;
    if ((n - 1) % (K - 1) != 0) {
        return -1;
    }
    int i = 0;
    while (i < n - 1) {
        for (int j = i + 1; j <= n; j += K - 1) {
            stones[i] += stones[j];
            stones[j] = 0;
        }
        i++;
    }
    return stones[0];
}
};

```

Go

Go

Dynamic Programming · LeetCode · By Pattern

— 2701 —

LeetCode

```

func mergeStones(stones []int, K int) int {
    n := len(stones)
    if (n-1)%(K-1) != 0 {
        return -1
    }
    dp := make([][]int, n+1)
    for i := 0; i < n; i++ {
        dp[i][0] = 0
    }
    for i := 0; i < n; i++ {
        for j := i+1; j <= n; j++ {
            dp[i][j] = dp[i][j-1] + dp[i+1][j]
        }
    }
    for i := 0; i < n; i++ {
        for j := i+1; j <= n; j++ {
            dp[i][j] = dp[i][j-1] + dp[i+1][j]
        }
    }
    return dp[0][n-1]
}

```

[*] Dynamic Programming · Pattern Solution (matched from community answers)
Python

Dynamic Programming · LeetCode · By Pattern

— 2702 —

LeetCode

```

# Python solution for merging stones with detailed comments
class Solution:
    def mergeStones(self, stones: List[int], k: int) -> int:
        n = len(stones)
        # Check for merge feasibility
        if (n - 1) % (k - 1) != 0:
            return -1

        # Precompute prefix sums for fast range sum queries
        prefix = [0] * (n + 1)
        for i in range(n):
            prefix[i + 1] = prefix[i] + stones[i]

        # Initialize dp table, dp[i][j] represents minimal cost to merge stones[i..j]
        dp = [[0] * n for _ in range(n)]

        # Process all possible subarrays with lengths >= k
        for length in range(k, n + 1):
            for i in range(n - length + 1):
                j = i + length - 1
                dp[i][j] = float('inf')
                # Try all possible splits
                for mid in range(i, j, k - 1):
                    dp[i][j] = min(dp[i][j], dp[i][mid] + dp[mid + 1][j])
                # If this subarray can be merged into one, add cost
                if (length - 1) % (k - 1) == 0:
                    dp[i][j] += prefix[j + 1] - prefix[i]

        return dp[0][n - 1]

```

Example usage:
solution = Solution()
print(solution.mergeStones([3, 2, 4, 1, 2])) # Expected output: 28

Dynamic Programming · LeetCode · By Pattern

— 2703 —

LeetCode

Quick Review — Dynamic Programming 24 questions · insights · complexity · solution

#70 Climbing Stairs [Easy]

Key Insights

- The problem can be solved using dynamic programming because the solution for a given step depends on the solutions of the previous two steps.
- The recurrence relation is: $ways[i] = ways[i-1] + ways[i-2]$.
- Base cases: when there is 1 step, there is 1 way; when there are 2 steps, there are 2 ways.
- This is analogous to the Fibonacci sequence, shifting indices accordingly.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (can be optimized to $O(1)$ using constant space)

Solution

We use a dynamic programming approach to build a solution gradually. The idea is that to reach step i , you could have come from step $i-1$ (taking one step) or from step $i-2$ (taking two steps). Therefore, the total number of ways to reach step i is the sum of ways to reach step $i-1$ and $i-2$. We initialize $dp[1] = 1$ and $dp[2] = 2$, and iterate from 3 to n to fill dp . Each code solution below uses this approach with comments to explain the logic.

#121 Best Time to Buy and Sell Stock [Easy]

Key Insights

- Traverse the list of prices once while keeping track of the minimum price encountered so far.
- At each day, calculate the profit if you sold the stock on that day.
- Update the maximum profit when the current profit exceeds the previous maximum.
- The optimal solution makes a single pass through the array with constant space usage.

Space & Time

Time Complexity: $O(n)$ - requires one pass through the prices array.

Space Complexity: $O(1)$ - only a few extra variables are used.

Solution

We solve the problem using a simple linear scan. We maintain two key variables: one for the minimum price seen so far and another for the maximum profit calculated so far. As we iterate, we continuously update the minimum price if a lower price is found, and then compute the potential profit at the current price. If this profit exceeds the current maximum profit, we update the maximum profit. The final maximum profit is returned after iterating through all prices.

#5 Longest Palindromic Substring [Medium]

Key Insights

- Every palindrome is centered around a character (for odd-length) or a pair of characters (for even-length).
- Expand from each center until the substring is no longer a palindrome.
- Update the longest palindrome when a longer one is found during the expansion.
- The solution employs a two-pointer technique without extra storage for dynamic programming tables.

Space & Time

Time Complexity: $O(n^2)$ in the worst case as we expand around each center.

Space Complexity: $O(1)$, excluding the space required for the output substring.

Solution

Space & Time

Time Complexity: $O(m)$

n

Space Complexity: $O(m)$

n - can be optimized to $O(n)$ using a 1D DP array.

Solution

We can solve the problem using a dynamic programming approach. The idea is to create a 2D DP table where each cell (i, j) represents the number of ways to reach that cell. Since the robot can only come from above or from the left, we update the DP table as follows:
- Initialize $dp[0][0] = 1$ for all j (only one way to move right in the first row). - Initialize $dp[i][0] = 1$ for all i (only one way to move down in the first column). - For each other cell, compute $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. The final answer will be at $dp[m-1][n-1]$.
The combinatorial alternative computes $C(m+n-2, m-1)$ or $C(m+n-2, n-1)$ using basic factorial arithmetic, which is efficient for the given constraints.

#72 Edit Distance [Medium]

Key Insights

- This is a classic dynamic programming problem.
- Use a 2D DP table where $dp[i][j]$ represents the minimum edit distance between $word1[0..i-1]$ and $word2[0..j-1]$.
- Base cases: converting an empty string to a string of length j requires j insertions, and vice versa.
- Transition: if characters match then $dp[i][j] = dp[i-1][j-1]$, otherwise take minimum among insertion, deletion, and replacement operations and add 1.

Space & Time

Time Complexity: $O(mn)$

n , where m and n are the lengths of $word1$ and $word2$ respectively.

Space Complexity: $O(m)$

n due to the DP table used for storing computed distances.

Solution

We use a dynamic programming approach by constructing a 2D table dp of size $(m+1) \times (n+1)$, where m and n are the lengths of $word1$ and $word2$. Each $dp[i][j]$ holds the minimum number of operations needed to convert the first i characters of $word1$ to the first j characters of $word2$. The algorithm includes the following steps:
- Initialize the base cases: $dp[i][0] = i$ (converting an empty $word2$ to i characters of $word1$ requires i deletions) and $dp[0][j] = j$ (converting i characters of $word1$ to an empty $word2$ requires i insertions).
- Iterate over each character in $word1$ and $word2$: if the current characters are equal, no additional operation is needed and $dp[i][j] = dp[i-1][j-1]$. Otherwise, choose the best operation among insertion ($dp[i][j-1]$), deletion ($dp[i-1][j]$), or substitution ($dp[i-1][j-1]$) and add 1. - The answer is found in $dp[m][n]$.
This method reliably computes the edit distance using a systematic DP approach, ensuring that all subproblems are solved and reused efficiently.

#91 Decode Ways [Medium]

Key Insights

- A digit '0' cannot be mapped on its own; it must be part of '10' or '20'.
- Use dynamic programming where $dp[i]$ represents the number of ways to decode the substring $s[0..i-1]$.
- For each index i , if the digit at position $i-1$ is non-zero, add $dp[i-1]$ to $dp[i]$.
- If the two-digit number formed by $s[i-2]$ and $s[i-1]$ is between 10 and 26, add $dp[i-2]$ to $dp[i]$.
- Initialize $dp[0]$ as 1 (an empty string has one way to be decoded).

Space & Time

Dynamic Programming · LeetCode · By Pattern

— 2705 —

LeetCode

Dynamic Programming · LeetCode · By Pattern

— 2706 —

LeetCode

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, due to the dp array (which can be optimized to $O(1)$).

Solution

We solve the problem using a dynamic programming approach. We initialize a dp array where $dp[i]$ indicates the number of ways to decode the substring $s[0..i-1]$. We set $dp[0] = 1$ since an empty string is trivially decodable. We loop through the string, and at each step:

- If $s[i-1]$ is not '0', we add $dp[i-1]$ to $dp[i]$ because $s[i-1]$ can be mapped to a valid letter. - If the substring $s[i-2..i]$ (for $i > 2$) forms a valid two-digit number between 10 and 26, we add $dp[i-2]$ to $dp[i]$. This cumulative approach yields the total ways to decode the string, and if the string starts with '0' or contains invalid sequences, $dp[n]$ will eventually be 0.

#152 Maximum Product Subarray [Medium]

Key Insights

- Maintain both the maximum and minimum product at the current index because a negative number can swap the max and min.
- A subarray must be contiguous.
- Use dynamic programming to update both the maximum product and the minimum product ending at each index.
- Resetting the product in case of a zero is crucial since the product becomes 0.

Space & Time

Time Complexity: $O(n)$ - The solution iterates through the array once.

Space Complexity: $O(1)$ - Constant space is used for tracking products.

Solution

The solution uses an iterative dynamic programming approach. At each index, compute the maximum product (maxEndingHere) and minimum product (minEndingHere) ending at that index. If the current element is negative, swap these two values, because multiplying a negative with a minimum product (possibly negative) might yield a larger product. After processing each element, update the global maximum. This method handles edge cases like zeros and negatives efficiently with constant additional space.

#198 House Robber [Medium]

Key Insights

- The problem can be solved using Dynamic Programming.
- For each house, decide whether to rob it or not.
- Recurrence relation: $dp[i] = \max(dp[i-1], dp[i-2] + \text{currentHouseMoney})$
- Use iterative computation to keep track of the optimal solutions without needing extra space for the entire dp array.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$, using only two variables to track the previous decisions.

Solution

We use a dynamic programming approach by iterating through each house in the array. At each step, we consider two scenarios: either we rob the current house (and add its money to the total from two houses ago) or we skip it (keeping the optimal solution up to the previous house). By using two variables to store these intermediate results, we achieve an optimized space complexity of $O(1)$.

#256 Paint House [Medium]

Key Insights

- The choice for painting each house depends on the previous house's color choices.
- Use dynamic programming to accumulate the cost while ensuring adjacent houses do not have the same color.

Dynamic Programming - LeetMastery - By Patterns

— 2707 —

LeetMastery

- The state of each house can be represented by the minimum cost when the house is painted a specific color, computed using the minimal costs of the previous house from the other two colors.
- The problem only requires updating the cost matrix in-place for optimal space usage.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$ extra space if the input matrix is modified in place.

Solution

We use a dynamic programming approach with iterative in-place updating of the cost matrix. For each house starting from the second one, we update the cost for each color by adding the minimal cost of painting the previous house with either of the other two colors. This guarantees we never use the same color for adjacent houses while accumulating the minimal total cost. The final answer is the minimum value for the last house.

#309 Best Time to Buy and Sell Stock with Cooldown [Medium]

Key Insights

- Use Dynamic Programming to solve the problem.
- Maintain three states to represent:
 - $s0$: the state where you are ready to buy (not holding any stock).
 - $s1$: the state where you are holding a stock.
 - $s2$: the cooldown state encountered right after selling a stock.
- Transition between states:
 - Transition into $s0$ can come from staying in $s0$ or coming out from the cooldown state $s2$.
 - To enter $s1$, you either continue holding your stock, or buy a stock using the profit available from $s0$.
 - The only way to reach $s2$ is by selling the stock from $s1$.
- The final answer is determined by the maximum profit when you are not holding a stock (i.e., $\max(s0, s2)$).

Space & Time

Time Complexity: $O(n)$, where n is the number of days.

Space Complexity: $O(1)$, since only a few variables are maintained for the states.

Solution

The solution involves updating three state variables for each day:

- $s0$ (ready to buy) is updated as the maximum of the previous $s0$ and the previous cooldown $s2$.
- $s1$ (holding a stock) is updated as the maximum of the previous $s1$ or buying today with the profit from $s0$ minus the current price.
- $s2$ (cooldown) is updated by selling the stock held in $s1$ (i.e., previous $s1$ plus the current price). At the end of the iteration, the maximum profit will be in state $s0$ (ready to buy) or $s2$ (cooldown), since remaining in $s1$ means you are holding a stock, which does not constitute a realized profit.

#377 Combination Sum IV [Medium]

Key Insights

- Use dynamic programming to count the number of valid combinations.
- The order in which numbers are picked matters, making this akin to permutation counting.
- Initialize a DP table where $dp[i]$ represents the number of ways to form the sum i .
- $dp[0]$ is initialized to 1 since there is one way to form a sum of 0 (by choosing nothing).
- For each sum i from 1 to target, iterate over nums and update $dp[i]$ based on $dp[i - num]$ when $i > \text{num}$.
- For negative numbers, the problem becomes unbounded due to potential infinite combinations; hence a constraint on the combination length is required to make it well-defined.

Space & Time

Time Complexity: $O(\text{target} * n)$, where n is the length of nums.

Space Complexity: $O(\text{target})$, for the dp array of size target + 1.

Dynamic Programming - LeetMastery - By Patterns

— 2708 —

LeetMastery

Solution

The problem is solved using a bottom-up dynamic programming approach. We create a dp array where each $dp[i]$ represents the number of possible combinations to reach the sum i . We iterate from 1 up to target, and for each sum, we check every number in nums. If the number can contribute to the current sum ($i - \text{num} > 0$), we add the number of ways to form the remainder ($i - \text{num}$) to $dp[i]$. This ensures that every possible permutation is counted. When negative numbers are allowed in the input, the potential for an infinite number of combinations arises unless we impose additional constraints, such as limiting the length of the combination.

#416 Partition Equal Subset Sum [Medium]

Key Insights

- The total sum of the array must be even, otherwise, partitioning into two equal subsets is impossible.
- The problem can be transformed into a "subset sum" problem where we need to check if there exists a subset of numbers that sums to half of the total sum.
- A dynamic programming approach can efficiently decide if such a subset exists.

Space & Time

Time Complexity: $O(n * \text{target})$, where target is half of the total sum.

Space Complexity: $O(\text{target})$, using a one-dimensional DP array.

Solution

First, compute the total sum of the array. If the total is odd, return false since an odd number cannot be equally partitioned. Next, set the target sum to half of the total. Use a dynamic programming array $dp[i]$ where $dp[i]$ represents whether a subset with sum i is achievable. Initialize $dp[0]$ as true because a subset sum of 0 is always possible. Then, for each number in the array, iterate through the dp array backwards from target to the number's value and update $dp[i]$ to true if $dp[i - \text{num}]$ is true. Ultimately, if $dp[\text{target}]$ is true, the array can be partitioned into two subsets with equal sum.

#435 Non-overlapping Intervals [Medium]

Key Insights

- Sorting is the key: By sorting intervals by their end times, we can greedily choose the optimal set of non-overlapping intervals.
- Greedy approach: Always select the interval that ends the earliest, as it leaves maximum room for subsequent intervals.
- Counting removals: The answer is the total number of intervals minus the count of intervals selected by this greedy approach.
- Edge observation: Intervals that touch at the boundary are not overlapping, so the check uses ">=" instead of ">".

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ if considering the storage for the sorted list or $O(1)$ if sorting in-place.

Solution

The solution involves first sorting the intervals by their end times. Initialize a variable to track the end of the last chosen interval. Iterate over the sorted intervals: if the current interval's start is greater than or equal to the last chosen interval's end, update the last end and include this interval in the non-overlapping set. Otherwise, count the interval as one that must be removed. The final answer is the removal count, which is computed as the total number of intervals minus the count of non-overlapping intervals selected.

#516 Longest Palindromic Subsequence [Medium]

Key Insights

- The problem can be solved efficiently using dynamic programming.
- A 2D DP table is used where $dp[i][j]$ represents the length of the longest palindromic subsequence in the substring $s[i..j]$.

Dynamic Programming - LeetMastery - By Patterns

— 2709 —

LeetMastery

- If the characters at the two ends of the substring match ($s[i] == s[j]$), they contribute to the palindrome length.
- Otherwise, the solution is the maximum sequence length by either excluding the left or the right character.
- The approach builds the solution from smaller substrings (length 1) to the entire string.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of the string.

Space Complexity: $O(n^2)$ due to the usage of a 2D DP table.

Solution

We solve this problem using a bottom-up dynamic programming approach. We create a 2D table $dp[i][j]$ such that $dp[i][j]$ stores the longest palindromic subsequence length for the substring $s[i..j]$. For any single character, $dp[i][i]$ is 1. For substrings of length greater than 1, if the characters at positions i and j ($s[i]$ and $s[j]$) match, then $dp[i][j] = dp[i+1][j-1] + 2$. If they do not match, we take the maximum of $dp[i+1][j]$ and $dp[i][j-1]$. Finally, the $dp[i][i-1]$ holds the result for the entire string.

#576 Out of Boundary Paths [Medium]

Key Insights

- Use Dynamic Programming (DP) as the state is defined by (current row, current column, moves remaining).
- Transitions are made from a cell to its four adjacent cells.
- If the ball moves out of bounds, it contributes to one valid path.
- States can be memoized to avoid recomputation and optimize the recursive solution.

Space & Time

Time Complexity: $O(\text{rows} * \text{cols} * m^n)$, since we compute each state at most once.

Space Complexity: $O(\text{rows} * \text{cols} * m^n)$ for the memoization table.

Solution

We use a recursive dynamic programming (or memoization) approach. Define a function $dp(i, j, \text{movesLeft})$ that returns the number of ways to move out of bounds starting from cell (i, j) with movesLeft moves remaining. For each move from the current cell, check:

- If the cell is out of bounds, return 1 (base case).
- If no moves are left ($\text{movesLeft} == 0$) and we are still inside, return 0.
- Otherwise, use recursion to explore all four directions and sum the paths. Store results in a memoization table to avoid recomputation, then return the computed value modulo $10^9 + 7$.

#1143 Longest Common Subsequence [Medium]

Key Insights

- The problem is well-suited for a dynamic programming approach.
- Use a 2D table where $dp[i][j]$ stores the length of the longest common subsequence for $\text{text1}[0..i-1]$ and $\text{text2}[0..j-1]$.
- If the characters match at the current position, extend the subsequence by 1 from $dp[i-1][j-1]$; otherwise, take the maximum from $dp[i-1][j]$ or $dp[i][j-1]$.
- Filling the table in a bottom-up manner ensures that all subproblems are solved optimally.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the lengths of text1 and text2 .

Space Complexity: $O(m * n)$ for maintaining the 2D DP table. Space can be optimized to $O(\min(m, n))$ if needed.

Solution

This solution uses a dynamic programming approach with a 2D array dp to record the lengths of common subsequences at each substring pair. We initialize a table of dimensions $(m+1) \times (n+1)$ with zeros, where m and n are the lengths of text1 and text2 . We iterate over both strings, comparing characters. If they match, we set $dp[i][j] = dp[i-1][j-1] + 1$, otherwise we choose the maximum from $dp[i-1][j]$ and $dp[i][j-1]$. The final answer is found at $dp[m][n]$.

Dynamic Programming - LeetMastery - By Patterns

— 2710 —

LeetMastery

#10 Regular Expression Matching [Hard]

Key Insights

- Use recursion or dynamic programming to simulate matching as the '*' operator can affect the current and following characters.
- The '.' operator is a wildcard that matches any single character.
- When encountering a '*' in the pattern, consider both skipping the preceding element or consuming one character from the string if it matches.
- Using memoization helps to save intermediate results and efficiently process overlapping subproblems.

Space & Time

Time Complexity: $O(m)$

n where $m = \text{length}(s)$ and $n = \text{length}(p)$, due to exploring each subproblem combination.

Space Complexity: $O(m)$

n for storing the memoization table in the dynamic programming or recursive approach.

Solution

The solution uses a top-down recursive strategy with memoization to efficiently match the string against the pattern. For each character position i in s and p , we check if they match directly or if the pattern character is '*' to allow any match. When encountering '*' in the pattern, two possibilities are explored: either the '*' stands for zero occurrences of the preceding element, or if there is a valid match, it accounts for one occurrence and we proceed further in the string while maintaining the same pattern index. The memoization aspect prevents repeated computations of the same (i, j) state in the recursion, resulting in an improved overall runtime.

#115 Distinct Subsequences [Hard]

Key Insights

- Use dynamic programming to build the solution by comparing characters of s and t .
- Define a 2D dp table where $dp[i][j]$ represents the number of ways to form the first j characters of t using the first i characters of s .
- The recurrence relation:
 - If $s[i-1]$ equals $t[j-1]$, then include both the cases of matching this character as well as skipping it: $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$.
 - If they do not match, then simply skip the current character of s : $dp[i][j] = dp[i-1][j]$.
- Initialize $dp[0][0] = 1$ for all i since an empty t can always be formed.
- The final answer is $dp[s.length][t.length]$.

Space & Time

Time Complexity: $O(n * m)$, where n is the length of s and m is the length of t .

Space Complexity: $O(n * m)$, which can be optimized to $O(n)$ with careful row reuse.

Solution

We use a dynamic programming approach with a 2D table. The table has dimensions $(\text{len}(s) + 1) \times (\text{len}(t) + 1)$. Each cell $dp[i][j]$ stores the number of distinct subsequences from the first i characters of s that match the first j characters of t .

Steps:

- Initialize $dp[0][0] = 1$ for every i , because an empty t is a subsequence of any prefix of s .
- Iterate over each character s (outer loop) and for each, iterate over t (inner loop).
- For every character in s , if it equals the current character in t , update $dp[i][j]$ as the sum of the ways by using or skipping this character. Otherwise, carry over the previous count.
- Return $dp[\text{len}(s)][\text{len}(t)]$ as the result.

#123 Best Time to Buy and Sell Stock III [Hard]

Key Insights

- The problem requires the optimal selection of at most two buy-sell transactions.

Dynamic Programming - LeetMastery - By Patterns

— 2711 —

LeetMastery

- One effective strategy is to split the problem into two parts:
 - For each day i , calculate the maximum profit obtainable from day 0 to i with one transaction.
 - For each day j , calculate the maximum profit obtainable from day i to the end with one transaction.
- Finally, for each day, the sum of these two profits (left and right) gives a candidate answer.
- Alternatively, a state-based dynamic programming approach can track 4 states corresponding to the two transactions.

Space & Time

Time Complexity: $O(n)$ where n is the number of days

Space Complexity: $O(n)$ due to extra arrays used for forward and backward computations (this can be optimized to $O(1)$ with a state variable approach)

Solution

We use a two-pass dynamic programming approach. In the first pass (forward), we compute an array 'leftProfits' where $\text{leftProfits}[i]$ is the maximum profit from day 0 to day i with one transaction. We do this by keeping track of the minimum price observed so far and calculating the profit if sold on day i . In the second pass (backward), we compute an array 'rightProfits' where $\text{rightProfits}[i]$ is the maximum profit obtainable from day i to the last day with one transaction by tracking the maximum price seen in reverse order. Finally, for every index i , the sum $\text{leftProfits}[i] + \text{rightProfits}[i]$ represents the maximum profit achievable by completing the first transaction on or before day i and the second transaction starting from day i . The answer is the maximum sum over all i .

#132 Palindrome Partitioning II [Hard]

Key Insights

- Precompute a 2D table $isPal$ where $isPal[i][j]$ indicates if the substring $s[i..j]$ is a palindrome.
- Use dynamic programming: $dp[i]$ represents the minimum cuts needed for the substring $s[0..i]$.
- For every index i , iterate over all possible starting indices j . If $s[j..i]$ is a palindrome, update $dp[i]$ using $dp[j-1] + 1$ (or 0 if j is 0).
- Time complexity is dominated by the $O(n^2)$ palindrome checking and state transitions.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n+2)$ (due to the 2D table for palindrome checks, plus $O(n)$ for the dp array)

Solution

The solution relies on dynamic programming with a two-step approach. First, precompute a 2D table $isPal$ where each cell indicates if a substring is a palindrome. This is done in $O(n^2)$ time by leveraging the fact that a substring $s[i..j]$ is a palindrome if the characters at the ends match and the inner substring $s[i+1..j-1]$ is also a palindrome (or if the length is less than 3). Then, use a dp array where $dp[i]$ holds the minimum number of cuts needed for the substring $s[0..i]$. For each i , check every index j ($0 \leq j \leq i$) if $s[j..i]$ is a palindrome, update $dp[i]$ to be the minimum of its current value and $dp[j-1] + 1$ (with a special case when j is 0, meaning no cut is needed). This two-phase approach efficiently breaks down the problem and computes the answer while avoiding redundant palindrome checks.

#132 Burst Balloons [Hard]

Key Insights

- Transform the problem by adding virtual balloons with value 1 at both ends to simplify edge handling.
- Use interval dynamic programming where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all balloons between indices i and j (inclusive).
- Realize that the order of bursting balloons matters; choose the last balloon to burst in each interval to break dependencies into independent subproblems.
- The solution involves iterating over all possible intervals and determining the optimal burst order.

Space & Time

Dynamic Programming - LeetMastery - By Patterns

— 2712 —

LeetMastery

Time Complexity: $O(n^3)$ - Three nested loops: one for the interval length, one for the starting index, and one for the final balloon choice.
Space Complexity: $O(n^2)$ - A 2D dynamic programming table is used to store the maximum coins for each interval.

Solution

The problem is solved using interval dynamic programming. First, extend the input array by adding 1 at both ends to handle edge cases seamlessly. Define a dp table where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all the balloons between i and j (not including i and j , as these are the virtual boundaries). For every interval $[i, j]$, iterate through the possible choices for the last balloon k to burst in that interval. The value for bursting balloon k will be determined by the coins from bursting it last (i.e., $nums[i] * nums[k] * nums[j]$) plus the maximum coins from the two resulting subintervals: $[i, k]$ and $[k, j]$. By filling the dp table in increasing interval lengths, you build up to the solution for the overall interval that covers the entire (extended) array.

#1000 Minimum Cost to Merge Stones [Hard]

Key Insights

- You can only merge stones into one final pile if $(n - 1)$ is divisible by $(k - 1)$. If not, return -1 .
- A dynamic programming approach is used where $dp[i][j]$ represents the minimum cost to merge the subarray $stones[i..j]$ into as few piles as possible.
- A prefix sum array is precomputed to quickly calculate the sum of stones in any subarray.
- When a segment can be merged into one pile (i.e. when $(length - 1) \% (k - 1) == 0$), add the sum of stones in that segment to the dp value.

Space & Time

Time Complexity: $O(n^3)$ due to the triple nested loop over segments and splits.

Space Complexity: $O(n^2)$ from the dp table and $O(n)$ for the prefix sum array.

Solution

The problem is solved using a range DP approach. We first verify that merging all piles into one is feasible by checking if $(n - 1) \% (k - 1) == 0$. Next, a prefix sum array is computed to optimize range sum calculations. A 2D dp table $dp[i][j]$ is then used to store the minimum cost to merge $stones[i..j]$. For each interval, we consider valid split points, updating $dp[i][j]$ by combining sub-problems. If the current segment can form a single merged pile (determined by the condition on the interval length), the cost of merging that segment (the sum of stones) is added to $dp[i][j]$.

```
def isValid(s: str) -> bool:
    # Popping from closing brackets to their corresponding opening brackets.
    paren_map = {'(': ')', '[': ']', '{': '}', '<': '>' }
    # Initialize an empty list to use as a stack.
    stack = []
    # Process each character in the string.
    for char in s:
        # If the character is a closing bracket.
        if char in paren_map:
            # Pop the top element from the stack if available, else use a placeholder.
            top_element = stack.pop() if stack else '#'
            # Check if the opening brace matches with the top element.
            if paren_map[char] != top_element:
                return False
            else:
                # For opening brackets, push onto the stack.
                stack.append(char)
    # Return True if all brackets are matched (i.e., the stack is empty).
    return not stack

# Example usage
print(isValid("(){}[]")) # Expected output: True
```

```
Python
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        for c in s:
            if c == '(':
                stack.append(')')
            elif c == '[':
                stack.append(']')
            elif c == '{':
                stack.append('}')
            elif not stack or stack.pop() != c:
                return False
        return not stack
```

```
Python
class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'(': ')', '[': ']', '{': '}' }
        for c in s:
            if c in '([{':
                stk.append(c)
            elif not stk or stk.pop() != c not in d:
                return False
        return not stk
```

Python

```
class Solution {
//
// @param string s
// @return boolean
//
function isValid(s) {
    stack = [];
    brackets = [
        ']' == '(',
        '}' == '[',
        '}' == '{',
    ];

    for (let i = 0; i < s.length; i++) {
        let char = s[i];
        if (bracketMap[char] && !stack.length) {
            return false;
        }
        if (stack.length && !bracketMap[char] && s[stack.length - 1] !== brackets[char]) {
            return false;
        }
        stack.push(char);
    }
    return stack.length === 0;
}
```

Java

Java

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (final char c : s.toCharArray()) {
            if (c == '(')
                stack.push('(');
            else if (c == '[')
                stack.push('[');
            else if (c == '{')
                stack.push('{');
            else if (stack.isEmpty() || stack.pop() != c)
                return false;
            return stack.isEmpty();
        }
    }
}
```

Java

#20 Stack — 25 questions · #20 of 21

#20 EASY

Valid Parentheses

LeetCode · SimplifyLogic · WalleCC · LeetCode · Editorial

String · Stack

Lists · Grid · 169

Problem:

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: $s = "()"$

Output: true

Example 2:

Input: $s = "([)]"$

Output: true

Example 3:

Input: $s = "(]"$

Output: false

Example 4:

Input: $s = "[]"$

Output: true

Example 5:

Input: $s = "[]"$

Output: false

Constraints:

- $1 \leq s.length \leq 104$
- s consists of parentheses only '()[]{}'.

Python

Python

```
class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'(': ')', '[': ']', '{': '}' }
        for c in s:
            if c in '([{':
                stk.append(c)
            elif not stk or stk.pop() != c not in d:
                return False
        return not stk
```

C++

C++

```
class Solution {
//
// @param string s
// @return boolean
//
function isValid(s) {
    stack = [];
    brackets = [
        ']' == '(',
        '}' == '[',
        '}' == '{',
    ];

    for (let i = 0; i < s.length; i++) {
        let char = s[i];
        if (bracketMap[char] && !stack.length) {
            return false;
        }
        if (stack.length && !bracketMap[char] && s[stack.length - 1] !== brackets[char]) {
            return false;
        }
        stack.push(char);
    }
    return stack.length === 0;
}
```

C++

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stk = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') {
                stk.push(c);
            } else if (stk.isEmpty() || !match(stk.pop(), c)) {
                return false;
            }
        }
        return stk.isEmpty();
    }

    private boolean match(char l, char r) {
        return (l == '(' && r == ')') || (l == '[' && r == ']') || (l == '{' && r == '}');
    }
}
```

Java

use std::collections::HashMap;

```
impl Solution {
    pub fn is_valid(s: String) -> bool {
        let mut map = HashMap::new();
        map.insert('(', ')');
        map.insert('[', ']');
        map.insert('{', '}');
        let mut stack = Vec::new();
        for c in s.chars() {
            if map.contains_key(&c) {
                stack.push(c);
            } else if stack.pop() != map.get(&c).unwrap_or('') {
                return false;
            }
        }
        stack.len() == 0
    }
}
```

Java

```
public class Solution {
    public boolean isValid(String s) {
        Stack<char> stk = new Stack<>();
        for each (char c in s.toCharArray()) {
            if (c == '(') {
                stk.push('(');
            } else if (c == '[') {
                stk.push('[');
            } else if (c == '{') {
                stk.push('{');
            } else if (stk.Count == 0 || stk.Pop() != c) {
                return false;
            }
        }
        return stk.Count == 0;
    }
}
```

```
Java
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stk = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '[' || c == '{' || c == '(') {
                stk.push(c);
            } else if (stk.isEmpty() || !match(stk.pop(), c)) {
                return false;
            }
        }
        return stk.isEmpty();
    }

    private boolean match(char l, char r) {
        return (l == '[' && r == ']') || (l == '{' && r == '}') || (l == '(' && r == ')');
    }
}

Java
use std::collections::HashMap;

impl Solution {
    pub fn is_valid(s: String) -> bool {
        let mut map = HashMap::new();
        map.insert('(', ')');
        map.insert('[', ']');
        map.insert('{', '}');
        let mut stk = Vec::new();
        for c in s.chars() {
            if map.contains_key(&c) {
                stk.push(map[&c]);
            } else if stk.pop().unwrap_or(' ') != c {
                return false;
            }
        }
        stk.len() == 0
    }
}

Java
public class Solution {
    public boolean isValid(String s) {
        Stack<char> stk = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '[') {
                stk.push(']');
            } else if (c == '{') {
                stk.push('}');
            } else if (c == '(') {
                stk.push(')');
            } else if (stk.count == 0 || stk.pop() != c) {
                return false;
            }
        }
        return stk.count == 0;
    }
}
```

Stack · LeetCode — By Pattern — 2719 — LeetCode

```
JavaScript
function isValid(s) {
    const stack = [];
    for (const c of s) {
        if (c === '[' || c === '{' || c === '(') {
            stack.push(c);
        } else if (stack.length === 0 || !match(stack.pop(), c)) {
            return false;
        }
    }
    return stack.length === 0;
}

function match(l, r) {
    return (l === '[' && r === ']') || (l === '{' && r === '}') || (l === '(' && r === ')');
}

JavaScript
# @param {String} s
# @return {Boolean}
def is_valid(s)
    stack = ''
    s.split('').each do |c|
        if ['[', '{', '('].include?(c)
            stack += c
        else
            if c == ']' && stack[stack.length - 1] == '['
                stack = stack[0..stack.length - 2]
            elsif c == '}' && stack[stack.length - 1] == '{'
                stack = stack[0..stack.length - 2]
            elsif c == ')' && stack[stack.length - 1] == '('
                stack = stack[0..stack.length - 2]
            else
                return false
            end
            end
            stack = ''
        end
    end
end

JavaScript
```

Stack · LeetCode — By Pattern — 2720 — LeetCode

```
JavaScript
function isValid(s) {
    const stack = [];
    for (const c of s) {
        if (c === '[' || c === '{' || c === '(') {
            stack.push(c);
        } else if (stack.length === 0 || !match(stack.pop(), c)) {
            return false;
        }
    }
    return stack.length === 0;
}

function match(l, r) {
    return (l === '[' && r === ']') || (l === '{' && r === '}') || (l === '(' && r === ')');
}

JavaScript
# @param {String} s
# @return {Boolean}
def is_valid(s)
    stack = ''
    s.split('').each do |c|
        if ['[', '{', '('].include?(c)
            stack += c
        else
            if c == ']' && stack[stack.length - 1] == '['
                stack = stack[0..stack.length - 2]
            elsif c == '}' && stack[stack.length - 1] == '{'
                stack = stack[0..stack.length - 2]
            elsif c == ')' && stack[stack.length - 1] == '('
                stack = stack[0..stack.length - 2]
            else
                return false
            end
            end
            stack = ''
        end
    end
end

TypeScript
TypeScript
```

Stack · LeetCode — By Pattern — 2721 — LeetCode

```
JavaScript
const map = new Map([
    ['(', ')'],
    ['{', '}'],
    ['[', ']'],
]);

function isValid(s: string): boolean {
    const stack = [];
    for (const c of s) {
        if (map.has(c)) {
            stack.push(map.get(c));
        } else if (stack.pop() !== c) {
            return false;
        }
    }
    return stack.length === 0;
}

TypeScript
const map = new Map([
    ['(', ')'],
    ['{', '}'],
    ['[', ']'],
]);

function isValid(s: string): boolean {
    const stack = [];
    for (const c of s) {
        if (map.has(c)) {
            stack.push(map.get(c));
        } else if (stack.pop() !== c) {
            return false;
        }
    }
    return stack.length === 0;
}

[""] Stack — Pattern Solution (matched from community answers)
Python
```

Stack · LeetCode — By Pattern — 2722 — LeetCode

```
def isValid(s: str) -> bool:
    # Mapping from closing brackets to their corresponding opening brackets.
    paren_map = {'(': ')', '{': '}', '[': ']'}
    # Initialize an empty list to use as a stack.
    stack = []
    # Process each character in the string.
    for char in s:
        # If the character is a closing bracket.
        if char in paren_map:
            # Pop the top element from the stack if available, else use a placeholder.
            top_element = stack.pop() if stack else '#'
            # Check if the element does not match with the top element.
            if paren_map[char] != top_element:
                return False
        else:
            # For opening brackets, push onto the stack.
            stack.append(char)
    # Return True if all brackets are matched (i.e., the stack is empty).
    return not stack

# Example usage
print(isValid("(){}[]")) # Expected output: True
```

#232 EASY
Implement Queue using Stacks
LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial
Stack · Design · Queue
List: Grid 129
Problem:
Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).
Implement the MyQueue class:
• void push(int x) Pushes element x to the back of the queue.
• int pop() Removes the element from the front of the queue and returns it.
• int peek() Returns the element at the front of the queue.
• boolean empty() Returns true if the queue is empty, false otherwise.
Notes:
• You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
• Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.
Example 1:
Input
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[1], [2], [1], [1], [1]]
Output

Stack · LeetCode — By Pattern — 2723 — LeetCode

```
[null, null, null, 1, 1, false]
```

Explanation
MyQueue myQueue = new MyQueue();
myQueue.push(1); // queue is: [1]
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
myQueue.peek(); // return 1
myQueue.pop(); // return 1, queue is [2]
myQueue.empty(); // return false

Constraints:
• 1 <= x <= 9
• At most 100 calls will be made to push, pop, peek, and empty.
• All the calls to pop and peek are valid.
Follow-up: Can you implement the queue such that each operation is amortized O(1) time complexity? In other words, performing n operations will take overall O(n) time even if one of those operations may take longer.

Python
Python
class MyQueue:
 def __init__(self):
 # Stack for enqueue operations.
 self.input_stack = []
 # Stack for dequeue operations.
 self.output_stack = []

 def push(self, x: int) -> None:
 # Always push new elements onto the input stack.
 self.input_stack.append(x)

 def pop(self) -> int:
 # Ensure the output stack has the current queue order.
 if not self.output_stack:
 while self.input_stack:
 # Move all elements from input stack to output stack.
 self.output_stack.append(self.input_stack.pop())
 # The top element of output stack is the front of the queue.
 return self.output_stack.pop()

 def peek(self) -> int:
 # Ensure the output stack has the current queue order.
 if not self.output_stack:
 while self.input_stack:
 # Move all elements from input_stack to output_stack.
 self.output_stack.append(self.input_stack.pop())
 # Peek the top of output stack without popping.
 return self.output_stack[-1]

 def empty(self) -> bool:
 # The queue is empty if both stacks are empty.
 return not self.input_stack and not self.output_stack

Stack · LeetCode — By Pattern — 2724 — LeetCode

```

Python

class MyQueue:
    def __init__(self):
        self.input = []
        self.output = []

    def push(self, x: int) -> None:
        self.input.append(x)

    def pop(self) -> int:
        self.pop()
        return self.output.pop()

    def peek(self) -> int:
        if not self.output:
            while self.input:
                self.output.append(self.input.pop())
        return self.output[-1]

    def empty(self) -> bool:
        return not self.input and not self.output

Python

class MyQueue:
    def __init__(self):
        self.stk1 = []
        self.stk2 = []

    def push(self, x: int) -> None:
        self.stk1.append(x)

    def pop(self) -> int:
        self.move()
        return self.stk2.pop()

    def peek(self) -> int:
        self.move()
        return self.stk2[-1]

    def empty(self) -> bool:
        return not self.stk1 and not self.stk2

    def move(self):
        if not self.stk2:
            while self.stk1:
                self.stk2.append(self.stk1.pop())

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(1)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()

```

Python

```

class MyQueue:
    def __init__(self):
        self.stk1 = []
        self.stk2 = [] # reversed order

    def push(self, x: int) -> None:
        self.stk1.append(x)

    def pop(self) -> int:
        self.move()
        return self.stk2.pop()

    def peek(self) -> int:
        self.move()
        return self.stk2[-1]

    def empty(self) -> bool:
        return not self.stk1 and not self.stk2

    def move(self):
        if not self.stk2: # only when stk2 is empty
            while self.stk1:
                self.stk2.append(self.stk1.pop())

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(1)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()

class MyQueue:
    def __init__(self):
        self.rk = []
        self.rk = [] # reversed

    def push(self, x: int) -> None:
        self.rk.append(x)

    def pop(self) -> int:
        self.pop()
        return self.rk.pop()

    def peek(self) -> int:
        if self.rk:

```

```

        return self.rk[-1]
    else:
        while self.rk:
            self.rk.append(self.rk.pop())
        return self.rk[-1]

    def empty(self) -> bool:
        return not (self.rk or self.rk)

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(1)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()

Java

class MyQueue {
    public void push(int x) {
        input.push(x);
    }

    public int pop() {
        peek();
        return output.pop();
    }

    public int peek() {
        if (output.isEmpty()) {
            while (!input.isEmpty()) {
                output.push(input.pop());
            }
            return output.peek();
        }
        return output.peek();
    }

    public boolean empty() {
        return input.isEmpty() && output.isEmpty();
    }

    private Deque<Integer> input = new ArrayDeque<>();
    private Deque<Integer> output = new ArrayDeque<>();
}

```

Java

```

class MyQueue {
    private Deque<Integer> stk1 = new ArrayDeque<>();
    private Deque<Integer> stk2 = new ArrayDeque<>();

    public MyQueue() {}

    public void push(int x) {
        stk1.push(x);
    }

    public int pop() {
        move();
        return stk2.pop();
    }

    public int peek() {
        move();
        return stk2.peek();
    }

    public boolean empty() {
        return stk1.isEmpty() && stk2.isEmpty();
    }

    private void move() {
        while (!stk2.isEmpty()) {
            while (!stk1.isEmpty()) {
                stk2.push(stk1.pop());
            }
        }
    }
}

/*
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = new MyQueue();
 * obj.push(1);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * boolean param_4 = obj.empty();
 */

```

Java

```

public class Implement_Queue_using_Stacks {
    class MyQueue {
        private Stack<Integer> stack1;
        private Stack<Integer> stack2;

        public MyQueue() {
            this.stack1 = new Stack<Integer>();
            this.stack2 = new Stack<Integer>();
        }

        // Push element x to the back of queue.
        public void push(int x) {
            stack1.push(x);
        }

        // Removes the element from in front of queue.
        public int pop() {
            if (stack2.isEmpty()) {
                return stack2.pop(); // stack is queue-order, queue-head at this-stack-top
            } else {
                while (!stack1.isEmpty()) {
                    stack2.push(stack1.pop());
                }
                return stack2.pop();
            }
        }

        // Get the front element.
        public int peek() {
            int ret = 0;
            if (stack2.isEmpty()) {
                ret = stack2.peek();
            } else {
                while (!stack1.isEmpty()) {
                    stack2.push(stack1.pop());
                }
                ret = stack2.peek();
            }
            return ret;
        }

        // Return whether the queue is empty.
        public boolean empty() {
            return stack1.isEmpty() && stack2.isEmpty();
        }
    }
}

```

```

class MyQueue {
    private Deque<Integer> stk1 = new ArrayDeque<>();
    private Deque<Integer> stk2 = new ArrayDeque<>();

    public MyQueue() {}

    public void push(int x) {
        stk1.push(x);
    }

    public int pop() {
        move();
        return stk2.pop();
    }

    public int peek() {
        move();
        return stk2.peek();
    }

    public boolean empty() {
        return stk1.isEmpty() && stk2.isEmpty();
    }

    private void move() {
        while (!stk2.isEmpty()) {
            while (!stk1.isEmpty()) {
                stk2.push(stk1.pop());
            }
        }
    }
}

```

TypeScript

TypeScript

```
class MyQueue {
  stkl number[];
  stk2 number[];

  constructor() {
    this.stkl = [];
    this.stk2 = [];
  }

  push(x: number): void {
    this.stkl.push(x);
  }

  pop(): number {
    this.move();
    return this.stk2.pop();
  }

  peek(): number {
    this.move();
    return this.stk2.at(-1);
  }

  empty(): boolean {
    return this.stkl.length && this.stk2.length
  }

  move(): void {
    if (this.stk2.length) {
      while (this.stkl.length) {
        this.stk2.push(this.stkl.pop());
      }
    }
  }
}
```

```
/**
 * Your MyQueue object will be instantiated and called as such:
 * var obj = new MyQueue();
 * obj.push(x);
 * var param_2 = obj.pop();
 * var param_3 = obj.peek();
 * var param_4 = obj.empty();
 */
```

TypeScript

Stack · LeetCode — By Pattern

— 2731 —

LeetCode

```
class MyQueue {
  stkl number[];
  stk2 number[];

  constructor() {
    this.stkl = [];
    this.stk2 = [];
  }

  push(x: number): void {
    this.stkl.push(x);
  }

  pop(): number {
    this.move();
    return this.stk2.pop();
  }

  peek(): number {
    this.move();
    return this.stk2[this.stk2.length - 1];
  }

  empty(): boolean {
    return this.stkl.length && this.stk2.length
  }

  move(): void {
    if (this.stk2.length) {
      while (this.stkl.length) {
        this.stk2.push(this.stkl.pop());
      }
    }
  }
}
```

```
/**
 * Your MyQueue object will be instantiated and called as such:
 * var obj = new MyQueue();
 * obj.push(x);
 * var param_2 = obj.pop();
 * var param_3 = obj.peek();
 * var param_4 = obj.empty();
 */
```

Go

Go

Stack · LeetCode — By Pattern

— 2732 —

LeetCode

```
type MyQueue struct {
  stkl []int
  stk2 []int
}

func Constructor() MyQueue {
  return MyQueue{[]int{}, []int{}}
}

func (this *MyQueue) Push(x int) {
  this.stkl = append(this.stkl, x)
}

func (this *MyQueue) Pop() int {
  this.move()
  ans := this.stk2[len(this.stk2)-1]
  this.stk2 = this.stk2[:len(this.stk2)-1]
  return ans
}

func (this *MyQueue) Peek() int {
  this.move()
  return this.stk2[len(this.stk2)-1]
}

func (this *MyQueue) Empty() bool {
  return len(this.stkl) == 0 && len(this.stk2) == 0
}

func (this *MyQueue) Move() {
  if len(this.stk2) == 0 {
    for len(this.stkl) > 0 {
      this.stk2 = append(this.stk2, this.stkl[len(this.stkl)-1])
      this.stkl = this.stkl[:len(this.stkl)-1]
    }
  }
}
```

```
/**
 * Your MyQueue object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Push(x);
 * param_2 := obj.Pop();
 * param_3 := obj.Peek();
 * param_4 := obj.Empty();
 */
```

Rust

Rust

Stack · LeetCode — By Pattern

— 2733 —

LeetCode

```
use std::collections::VecDeque;

struct MyQueue {
  stkl: Vec<i32>,
  stk2: Vec<i32>,
}

impl MyQueue {
  fn new() -> Self {
    MyQueue {
      stkl: Vec::new(),
      stk2: Vec::new(),
    }
  }

  fn push(&mut self, x: i32) {
    self.stkl.push(x);
  }

  fn pop(&mut self) -> i32 {
    self.move_elements();
    self.stk2.pop().unwrap()
  }

  fn peek(&mut self) -> i32 {
    self.move_elements();
    self.stk2.last().unwrap()
  }

  fn empty(&self) -> bool {
    self.stkl.is_empty() && self.stk2.is_empty()
  }

  fn move_elements(&mut self) {
    if self.stk2.is_empty() {
      while let Some(element) = self.stkl.pop() {
        self.stk2.push(element);
      }
    }
  }
}
```

Rust

Stack · LeetCode — By Pattern

— 2734 —

LeetCode

```
struct MyQueue {
  in_stack: Vec<i32>,
  out_stack: Vec<i32>,
}

/**
 * Self means the method takes an immutable reference.
 * If you need a mutable reference, change it to &mut self instead.
 */
impl MyQueue {
  fn new() -> Self {
    Self {
      in_stack: vec![],
      out_stack: vec![],
    }
  }

  fn push(&mut self, x: i32) {
    self.in_stack.push(x);
  }

  fn pop(&mut self) -> i32 {
    if self.out_stack.is_empty() {
      self.fill_out();
    }
    self.out_stack.pop().unwrap()
  }

  fn peek(&mut self) -> i32 {
    if self.out_stack.is_empty() {
      self.fill_out();
    }
    &self.out_stack.last().unwrap()
  }

  fn empty(&self) -> bool {
    self.in_stack.is_empty() && self.out_stack.is_empty()
  }

  fn fill_out(&mut self) {
    let MyQueue { in_stack, out_stack } = self;
    if out_stack.is_empty() {
      while in_stack.is_empty() {
        out_stack.push(in_stack.pop().unwrap());
      }
    }
  }
}
```

Stack · LeetCode — By Pattern

— 2735 —

LeetCode

```
/**
 * Your MyQueue object will be instantiated and called as such:
 * let obj = MyQueue.new();
 * obj.push(x);
 * let ret_2: i32 = obj.pop();
 * let ret_3: i32 = obj.peek();
 * let ret_4: bool = obj.empty();
 */
```

[*] Stack — Pattern Solution (matched from community answers)

Python

```
class MyQueue:
    def __init__(self):
        # Stack for enqueue operations.
        self.input_stack = []
        # Stack for dequeue operations.
        self.output_stack = []

    def push(self, x: int) -> None:
        # Always push new elements onto the input stack.
        self.input_stack.append(x)

    def pop(self) -> int:
        # Ensure the output stack has the current queue order.
        if not self.output_stack:
            while self.input_stack:
                # Move all elements from input stack to output stack.
                self.output_stack.append(self.input_stack.pop())
            # The top element of output stack is the front of the queue.
            return self.output_stack.pop()

    def peek(self) -> int:
        # Ensure the output stack has the current queue order.
        if not self.output_stack:
            while self.input_stack:
                # Move all elements from input_stack to output_stack.
                self.output_stack.append(self.input_stack.pop())
            # Peek the top of output stack without popping.
            return self.output_stack[-1]

    def empty(self) -> bool:
        # The queue is empty if both stacks are empty.
        return not self.input_stack and not self.output_stack
```

#844

EASY

Backspace String Compare

LeetCode · SimplyLeet · WalkCCC · LeetCode · Editorial

Two Pointers · String · Stack · Simulation

Ulid Grid 169

Problems

Stack · LeetCode — By Pattern

— 2736 —

LeetCode

Given two strings s and t, return true if they are equal when both are typed into empty text editors. 'a' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

Input: s = "ab#c", t = "ad#c"
Output: true
Explanation: Both s and t become "ac".

Example 2:

Input: s = "ab##", t = "c#d#"
Output: true
Explanation: Both s and t become "".

Example 3:

Input: s = "a#c", t = "b"
Output: false
Explanation: s becomes "c" while t becomes "b".

Constraints:

- 1 <= s.length, t.length <= 200
- s and t only contain lowercase letters and '#' characters.

Follow up: Can you solve it in O(n) time and O(1) space?

```
Python
# Problem 10. Compare two strings with backspace processing
def backspaceCompare(s: str, t: str) -> bool:
    # Pointers for both strings starting at the end
    i, j = len(s) - 1, len(t) - 1

    # Process both strings until all characters are compared
    while i >= 0 or j >= 0:
        # Process string s: skip backspaced characters
        skip = 0
        while i >= 0:
            if s[i] == '#':
                skip += 1
                i -= 1
            elif skip > 0:
                skip -= 1
                i -= 1
            else:
                break
        # Process string t: skip backspaced characters
        skip = 0
        while j >= 0:
            if t[j] == '#':
                skip += 1
                j -= 1
            elif skip > 0:
                skip -= 1
                j -= 1
            else:
                break
        # Compare characters of s and t
        if i < 0 and j < 0:
            return True
        if i < 0 and j >= 0:
            return False
        if j < 0 and i >= 0:
            return False
        if i >= 0 and j >= 0:
            if s[i] != t[j]:
                return False
            i -= 1
            j -= 1
    return True
```

```
elif skip > 0:
    skip -= 1
    j -= 1
else:
    break

# Compare the current valid characters from s and t
if i < 0 and j >= 0:
    if s[i] != t[j]:
        return False
    elif i < 0 or j < 0:
        return False
    i -= 1
    j -= 1
return True

# Example test cases
print(backspaceCompare("ab#c", "ad#c")) # Expected output: True
print(backspaceCompare("ab##", "c#d#")) # Expected output: True
print(backspaceCompare("a#c", "b")) # Expected output: False
```

```
Python
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        stack = []
        for c in s:
            if c != '#':
                stack.append(c)
            elif stack:
                stack.pop()
        return stack == backspace(t)
```

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            # Delete characters of s if needed
            skip1 = 0
            while i >= 0 and (s[i] == '#' or skip1 > 0):
                skip1 -= 1
                i -= 1
            # Delete characters of t if needed
            skip2 = 0
            while j >= 0 and (t[j] == '#' or skip2 > 0):
                skip2 -= 1
                j -= 1
            if i < 0 and j < 0:
                return True
            if i < 0 and j >= 0:
                return False
            if j < 0 and i >= 0:
                return False
            if i >= 0 and j >= 0:
                if s[i] != t[j]:
                    return False
                i -= 1
                j -= 1
        return True
```

```
Python
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            # Delete characters of s if needed
            skip1 = 0
            while i >= 0 and (s[i] == '#' or skip1 > 0):
                skip1 -= 1
                i -= 1
            # Delete characters of t if needed
            skip2 = 0
            while j >= 0 and (t[j] == '#' or skip2 > 0):
                skip2 -= 1
                j -= 1
            if i < 0 and j < 0:
                return True
            if i < 0 and j >= 0:
                return False
            if j < 0 and i >= 0:
                return False
            if i >= 0 and j >= 0:
                if s[i] != t[j]:
                    return False
                i -= 1
                j -= 1
        return True
```

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            # Delete characters of s if needed
            skip1 = 0
            while i >= 0 and (s[i] == '#' or skip1 > 0):
                skip1 -= 1
                i -= 1
            # Delete characters of t if needed
            skip2 = 0
            while j >= 0 and (t[j] == '#' or skip2 > 0):
                skip2 -= 1
                j -= 1
            if i < 0 and j < 0:
                return True
            if i < 0 and j >= 0:
                return False
            if j < 0 and i >= 0:
                return False
            if i >= 0 and j >= 0:
                if s[i] != t[j]:
                    return False
                i -= 1
                j -= 1
        return True
```

```
Java
class Solution {
    public boolean backspaceCompare(String s, String t) {
        return backspace(s).equals(backspace(t));
    }

    private String backspace(String s) {
        StringBuilder sb = new StringBuilder();
        for (final char c : s.toCharArray()) {
            if (c != '#') {
                sb.append(c);
            } else if (sb.length() > 0) {
                sb.deleteCharAt(sb.length() - 1);
            }
        }
        return sb.toString();
    }
}
```

```
class Solution {
    public boolean backspaceCompare(String s, String t) {
        int i = s.length() - 1;
        int j = t.length() - 1;
        while (true) {
            // Delete characters of s if needed
            int skip1 = 0;
            while (i >= 0 and (s.charAt(i) == '#' || skip1 > 0)) {
                skip1--;
                i--;
            }
            // Delete characters of t if needed
            skip2 = 0;
            while (j >= 0 and (t.charAt(j) == '#' || skip2 > 0)) {
                skip2--;
                j--;
            }
            if (i < 0 and j < 0) {
                return true;
            }
            if (i < 0 and j >= 0) {
                return false;
            }
            if (j < 0 and i >= 0) {
                return false;
            }
            if (i >= 0 and j >= 0) {
                if (s.charAt(i) != t.charAt(j)) {
                    return false;
                }
                i--;
                j--;
            }
        }
    }
}
```

```
Java
class Solution {
    public boolean backspaceCompare(String s, String t) {
        int i = s.length() - 1;
        int j = t.length() - 1;
        while (i >= 0 or j >= 0) {
            // Delete characters of s if needed
            skip1 = 0;
            while (i >= 0 and (s.charAt(i) == '#' || skip1 > 0)) {
                skip1--;
                i--;
            }
            // Delete characters of t if needed
            skip2 = 0;
            while (j >= 0 and (t.charAt(j) == '#' || skip2 > 0)) {
                skip2--;
                j--;
            }
            if (i < 0 and j < 0) {
                return true;
            }
            if (i < 0 and j >= 0) {
                return false;
            }
            if (j < 0 and i >= 0) {
                return false;
            }
            if (i >= 0 and j >= 0) {
                if (s.charAt(i) != t.charAt(j)) {
                    return false;
                }
                i--;
                j--;
            }
        }
    }
}
```

```
class Solution {
    public boolean backspaceCompare(String s, String t) {
        int i = s.length() - 1;
        int j = t.length() - 1;
        while (i >= 0 or j >= 0) {
            // Delete characters of s if needed
            skip1 = 0;
            while (i >= 0 and (s.charAt(i) == '#' || skip1 > 0)) {
                skip1--;
                i--;
            }
            // Delete characters of t if needed
            skip2 = 0;
            while (j >= 0 and (t.charAt(j) == '#' || skip2 > 0)) {
                skip2--;
                j--;
            }
            if (i < 0 and j < 0) {
                return true;
            }
            if (i < 0 and j >= 0) {
                return false;
            }
            if (j < 0 and i >= 0) {
                return false;
            }
            if (i >= 0 and j >= 0) {
                if (s.charAt(i) != t.charAt(j)) {
                    return false;
                }
                i--;
                j--;
            }
        }
    }
}
```

```
TypeScript
function backspaceCompare(s: string, t: string): boolean {
    let i = s.length - 1;
    let j = t.length - 1;
    while (i >= 0 || j >= 0) {
        // Delete characters of s if needed
        let skip1 = 0;
        while (i >= 0 and (s[i] === '#' || skip1 > 0)) {
            skip1--;
            i--;
        }
        // Delete characters of t if needed
        let skip2 = 0;
        while (j >= 0 and (t[j] === '#' || skip2 > 0)) {
            skip2--;
            j--;
        }
        if (i < 0 and j < 0) {
            return true;
        }
        if (i < 0 and j >= 0) {
            return false;
        }
        if (j < 0 and i >= 0) {
            return false;
        }
        if (i >= 0 and j >= 0) {
            if (s[i] !== t[j]) {
                return false;
            }
            i--;
            j--;
        }
    }
    return true;
}
```

```

function backspaceCompare(s: string, t: string): boolean {
    let i = s.length - 1;
    let j = t.length - 1;
    while (i >= 0 || j >= 0) {
        let skip = 0;
        while (i >= 0) {
            if (s[i] === '#') {
                skip++;
            } else if (skip >= 0) {
                skip--;
            } else {
                break;
            }
            i--;
        }
        skip = 0;
        while (j >= 0) {
            if (t[j] === '#') {
                skip++;
            } else if (skip >= 0) {
                skip--;
            } else {
                break;
            }
            j--;
        }
        if (s[i] !== t[j]) {
            return false;
        }
        i--;
        j--;
    }
    return true;
}

```

Rust
Rust

Stack · LeetCode — By Pattern

— 2743 —

LeetCode

```

impl Solution {
    pub fn backspace_compare(s: String, t: String) -> bool {
        let (s, t) = (s.as_bytes(), t.as_bytes());
        let (mut i, mut j) = (s.len(), t.len());
        while i >= 0 || j >= 0 {
            let mut skip = 0;
            while i >= 0 {
                if s[i] == b'#' {
                    skip += 1;
                } else if skip > 0 {
                    skip -= 1;
                } else {
                    break;
                }
                i -= 1;
            }
            skip = 0;
            while j >= 0 {
                if t[j] == b'#' {
                    skip += 1;
                } else if skip > 0 {
                    skip -= 1;
                } else {
                    break;
                }
                j -= 1;
            }
            if i >= 0 && j >= 0 {
                if s[i] != t[j] {
                    return false;
                }
                i -= 1;
                j -= 1;
            } else if i >= 0 || j >= 0 {
                return false;
            }
        }
        return true;
    }
}

```

[*] Stack — Pattern Solution (matched from community answers)
Python

Stack · LeetCode — By Pattern

— 2744 —

LeetCode

```

class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        stack = []
        for c in s:
            if c != '#':
                stack.append(c)
            else:
                stack.pop()
        return stack == backspace(t)

```

#143

MEDIUM

Reorder List

LeetCode · SimplyList · WUJCC · LeetCode · Editorial
Linked List · Two Pointers · Stack · Recursion

Like · 610 · 100

Problem:

You are given the head of a singly linked-list. The list can be represented as:

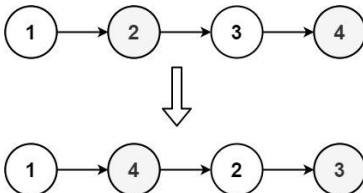
$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n - 1 \rightarrow L_n$

Reorder the list to be on the following form:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Example 1:



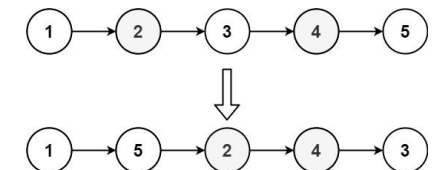
Input: head = [1,2,3,4]
Output: [1,4,2,3]

Example 2:

Stack · LeetCode — By Pattern

— 2745 —

LeetCode



Input: head = [1,2,3,4,5]
Output: [1,5,2,4,3]

Constraints:

- The number of nodes in the list is in the range [1, 5 * 10⁴].
- 1 <= Node.val <= 1000

Python

Python

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: ListNode) -> None:
        if not head or not head.next:
            return

        # Step 1: Find the middle of the linked list using slow and fast pointers.
        slow, fast = head, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # Step 2: Reverse the second half of the list.
        prev, curr = None, slow.next
        slow.next = None
        while curr:
            next_temp = curr.next
            curr.next = prev
            prev = curr
            curr = next_temp

```

Stack · LeetCode — By Pattern

— 2746 —

LeetCode

```

# Step 1: Merge the two halves, alternating nodes.
first, second = head, prev
while second:
    temp1 = first.next
    temp2 = second.next
    first.next = second
    second.next = temp1
    first = temp1
    second = temp2

```

Python

```

class Solution:
    def reorderList(self, head: ListNode) -> None:
        def findMid(head: ListNode):
            pre = None
            slow = head
            fast = head

            while fast and fast.next:
                pre = slow
                slow = slow.next
                fast = fast.next.next
            return slow

        def reverse(head: ListNode) -> ListNode:
            pre = None
            curr = head

            while curr:
                next = curr.next
                curr.next = pre
                pre = curr
                curr = next

            return pre

        def merge(l1: ListNode, l2: ListNode) -> None:
            while l1 and l2:
                next1 = l1.next
                next2 = l2.next
                l1.next = l2
                l2.next = l1
                l1 = next1
                l2 = next2

            if not head or not head.next:
                return
            mid = findMid(head)
            reversed = reverse(mid)
            merge(head, reversed)

```

Python

Stack · LeetCode — By Pattern

— 2747 —

LeetCode

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        cur = slow.next
        slow.next = None
        pre = None

        while cur:
            t = cur.next
            cur.next = pre
            pre = cur
            cur = t

        cur, pre = pre, None

```

Python

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        cur = slow.next
        slow.next = None
        pre = None

        while cur:
            t = cur.next
            cur.next = pre
            pre = cur
            cur = t

        cur, pre = pre, None

```

Stack · LeetCode — By Pattern

— 2748 —

LeetCode

```
Java

/*
 * Definition for singly-linked list.
 */
public class ListNode {
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}

class Solution {
    public void reorderList(ListNode head) {
        ListNode fast = head, slow = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode cur = slow.next;
        slow.next = null;

        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }

        while (pre != null) {
            ListNode t = pre.next;
            pre.next = cur.next;
            cur.next = pre;
            pre = pre.next;
            cur = t;
        }
    }
}

Java
```

Stack · LeetCode · By Pattern

— 2749 —

LeetCode

```
/*
 * Definition for singly-linked list.
 */
public class ListNode {
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}

public class Solution {
    public void reorderList(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode cur = slow.next;
        slow.next = null;

        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }
        cur = head;

        while (pre != null) {
            ListNode t = pre.next;
            pre.next = cur.next;
            cur.next = pre;
            pre = pre.next;
            cur = t;
        }
    }
}

Java
```

Stack · LeetCode · By Pattern

— 2750 —

LeetCode

```
/*
 * Definition for singly-linked list.
 */
public class ListNode {
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}

class Solution {
    public void reorderList(ListNode head) {
        ListNode fast = head, slow = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode cur = slow.next;
        slow.next = null;

        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }

        while (pre != null) {
            ListNode t = pre.next;
            pre.next = cur.next;
            cur.next = pre;
            pre = pre.next;
            cur = t;
        }
    }
}

Java
```

Stack · LeetCode · By Pattern

— 2751 —

LeetCode

```
/*
 * Definition for singly-linked list.
 */
public class ListNode {
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next = next; }
}

public class Solution {
    public void reorderList(ListNode head) {
        ListNode slow = head;
        ListNode fast = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode cur = slow.next;
        slow.next = null;

        ListNode pre = null;
        while (cur != null) {
            ListNode t = cur.next;
            cur.next = pre;
            pre = cur;
            cur = t;
        }
        cur = head;

        while (pre != null) {
            ListNode t = pre.next;
            pre.next = cur.next;
            cur.next = pre;
            pre = pre.next;
            cur = t;
        }
    }
}

JavaScript
```

Stack · LeetCode · By Pattern

— 2752 —

LeetCode

```
/*
 * Definition for singly-linked list.
 */
function ListNode(val, next) {
    this.val = (val===undefined ? 0 : val)
    this.next = (next===undefined ? null : next)
}

/**
 * @param {ListNode} head
 * @return {void} Do not return anything, modify head in-place instead.
 */
var reorderList = function(head) {
    let slow = head;
    let fast = head;
    while (fast.next && fast.next.next) {
        slow = slow.next;
        fast = fast.next.next;
    }

    let cur = slow.next;
    slow.next = null;

    let pre = null;
    while (cur) {
        const t = cur.next;
        cur.next = pre;
        pre = cur;
        cur = t;
    }

    while (pre) {
        const t = pre.next;
        pre.next = cur.next;
        cur.next = pre;
        pre = pre.next;
        cur = t;
    }
};

JavaScript
```

Stack · LeetCode · By Pattern

— 2753 —

LeetCode

```
/*
 * Definition for singly-linked list.
 */
function ListNode(val, next) {
    this.val = (val===undefined ? 0 : val)
    this.next = (next===undefined ? null : next)
}

/**
 * @param {ListNode} head
 * @return {void} Do not return anything, modify head in-place instead.
 */
var reorderList = function(head) {
    let slow = head;
    let fast = head;
    while (fast.next && fast.next.next) {
        slow = slow.next;
        fast = fast.next.next;
    }

    let cur = slow.next;
    slow.next = null;

    let pre = null;
    while (cur) {
        const t = cur.next;
        cur.next = pre;
        pre = cur;
        cur = t;
    }

    while (pre) {
        const t = pre.next;
        pre.next = cur.next;
        cur.next = pre;
        pre = pre.next;
        cur = t;
    }
};

TypeScript
```

Stack · LeetCode · By Pattern

— 2754 —

LeetCode

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

// Do not return anything, modify head in-place instead.
//
function reorderList(head: ListNode | null): void {
    let slow = head;
    let fast = head;
    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;
    }

    let next = slow.next;
    slow.next = null;

    while (next) {
        [next.next, slow, next] = [slow, next, next.next];
    }

    let left = head;
    let right = slow;
    while (right.next) {
        const next = left.next;
        left.next = right;
        right = right.next;
        left.next.next = next;
        left = left.next.next;
    }
}

```

TypeScript

Stack · LeetCode — By Pattern

— 2755 —

LeetCode

```

//
// Definition for singly-linked list.
// class ListNode {
//     val: number
//     next: ListNode | null
//     constructor(val?: number, next?: ListNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.next = (next===undefined ? null : next)
//     }
// }

// Do not return anything, modify head in-place instead.
//
function reorderList(head: ListNode | null): void {
    let slow = head;
    let fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    let next = slow.next;
    slow.next = null;

    while (next != null) {
        [next.next, slow, next] = [slow, next, next.next];
    }

    let left = head;
    let right = slow;
    while (right.next != null) {
        const next = left.next;
        left.next = right;
        right = right.next;
        left.next.next = next;
        left = left.next.next;
    }
}

```

Rust

Rust

Stack · LeetCode — By Pattern

— 2756 —

LeetCode

```

// Definition for singly-linked list.
// #derive(PartialEq, Eq, Clone, Debug)
// pub struct ListNode {
//     pub val: i32,
//     pub next: Option
// }
// impl ListNode {
//     #[inline]
//     fn new(val: i32) -> Self {
//         ListNode {
//             next: None,
//         }
//     }
// }
use std::collections::VecDeque;
impl Solution {
    pub fn reorder_list(head: Option

```

Rust

Stack · LeetCode — By Pattern

— 2757 —

LeetCode

```

// Definition for singly-linked list.
// #derive(PartialEq, Eq, Clone, Debug)
// pub struct ListNode {
//     pub val: i32,
//     pub next: Option
// }
// impl ListNode {
//     #[inline]
//     fn new(val: i32) -> Self {
//         ListNode {
//             next: None,
//         }
//     }
// }
use std::collections::VecDeque;
impl Solution {
    pub fn reorder_list(head: Option

```

[*] Stack — Pattern Solution (stored optimal)

Python

Stack · LeetCode — By Pattern

— 2758 —

LeetCode

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: ListNode) -> None:
        if not head or not head.next:
            return

        # Step 1: Find the middle of the linked list using slow and fast pointers.
        slow, fast = head, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # Step 2: Reverse the second half of the list.
        prev, curr = None, slow.next
        slow.next = None # Join the list into two halves.
        while curr:
            next_temp = curr.next # Save next node.
            curr.next = prev # Reverse current pointer.
            prev = curr # Move prev to current.
            curr = next_temp # Move to the next node.

        # Step 3: Merge the two halves, alternating nodes.
        first, second = head, prev
        while second:
            temp1 = first.next # Save next node from first half.
            temp2 = second.next # Save next node from reversed second half.
            first.next = second # Link first node to second list node.
            second.next = temp1 # Link second node to first half.
            first = temp1 # Move pointer to first half.
            second = temp2 # Move pointer to second half.

```

C++

Stack · LeetCode — By Pattern

— 2759 —

LeetCode

```

// Definition for singly-linked list.
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

class Solution {
public:
    void reorderList(ListNode* head) {
        if (!head || !head->next) return;

        // Step 1: Find the middle of the linked list.
        ListNode* slow = head;
        ListNode* fast = head;
        while (fast && fast->next) {
            slow = slow->next; // Move slow pointer.
            fast = fast->next->next; // Move fast pointer.
        }

        // Step 2: Reverse the second half of the list.
        ListNode* prev = nullptr;
        ListNode* curr = slow->next;
        slow->next = nullptr; // Split the list.
        while (curr) {
            ListNode* next_temp = curr->next; // Save next node.
            curr->next = prev; // Reverse current node.
            prev = curr; // Move prev forward.
            curr = next_temp; // Move to next node.
        }

        // Step 3: Merge the two halves.
        ListNode* first = head;
        ListNode* second = prev;
        while (second) {
            ListNode* temp1 = first->next; // Save next node in first half.
            ListNode* temp2 = second->next; // Save next node in reversed second half.
            first->next = second; // Link first node to second.
            second->next = temp1; // Link second node to next of first.
            first = temp1; // Advance pointer in first half.
            second = temp2; // Advance pointer in second half.
        }
    }
};

```

#150

MEDIUM

Evaluate Reverse Polish Notation

LeetCode · SimpleList · NumCC · LeetCode · Editorial

Array · Math · Stack

List: Grind 169

Problem:

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.

Stack · LeetCode — By Pattern

— 2760 —

LeetCode

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

Input: tokens = ["2","3","+","4","+"]

Output: 9

Explanation: ((2 + 3) + 4) = 9

Example 2:

Input: tokens = ["4","13","3","+","11","*","3","*","+"]

Output: 6

Explanation: (4 + (13 / 3)) = 6

Example 3:

Input: tokens = ["18","6","0","9","3","+","11","*","2","*","13","*","+","5","+"]

Output: 22

Explanation: ((18 + (6 / ((9 + 3) * -11))) + 17) + 5

= ((18 + (6 / (12 * -11))) + 17) + 5

= ((18 + (6 / -132)) + 17) + 5

= ((18 + 0) + 17) + 5

= (0 + 17) + 5

= 17 + 5

= 22

Constraints:

- 1 <= tokens.length <= 104
- tokens[i] is either an operator "+", "-", "*", "/", or an integer in the range [-200, 200].

>> Brute Force (Stack) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        # Brute Force (stack is already optimal) - simulate tokens by token
        stack = []
        ops = {'+': lambda a, b: a+b, '-': lambda a, b: a-b,
              '*': lambda a, b: a*b, '/': lambda a, b: int(a/b)}
        for tok in tokens:
            if tok in ops:
                b, a = stack.pop(), stack.pop()
                stack.append(ops[tok](a, b))
            else:
                stack.append(int(tok))
        return stack[0]
```

Stack · LeetCode — By Pattern

— 2761 —

LeetCode

import operator

```
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        ops = {
            "+": operator.add,
            "-": operator.sub,
            "*": operator.mul,
            "/": operator.truediv,
        }
        s = []
        for token in tokens:
            if token in ops:
                s.append(int(ops[token](s.pop(-2), s.pop(-1))))
            else:
                s.append(int(token))
        return s[0]
```

Python

```
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] /= nums[-1]
                nums.pop()
        return nums[0]
```

Python

```
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] /= nums[-1]
                nums.pop()
        return nums[0]
```

Stack · LeetCode — By Pattern

— 2763 —

LeetCode

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<int> stk;
        for (const t : tokens) {
            if (t.size() > 1 || isdigit(t[0])) {
                stk.push(stoi(t));
            } else {
                int y = stk.top();
                stk.pop();
                int x = stk.top();
                stk.pop();
                if (t[0] == '+')
                    stk.push(x + y);
                else if (t[0] == '-')
                    stk.push(x - y);
                else if (t[0] == '*')
                    stk.push(x * y);
                else
                    stk.push(x / y);
            }
        }
        return stk.top();
    }
};
```

Java

Java

```
class Solution {
    public int evalRPN(String[] tokens) {
        final Map<String, BinaryOperator<Long>> op = Map.of(
            "+", (a, b) -> a + b, "-", (a, b) -> a - b,
            "*", (a, b) -> a * b, "/", (a, b) -> a / b);
        Deque<Long> stack = new ArrayDeque<>();

        for (final String token : tokens) {
            if (op.containsKey(token)) {
                final Long b = stack.pop();
                final Long a = stack.pop();
                stack.push(op.get(token).apply(a, b));
            } else {
                stack.push(Long.parseLong(token));
            }
        }

        return stack.pop().intValue();
    }
}
```

Java

Stack · LeetCode — By Pattern

— 2765 —

LeetCode

Python

Python

```
def evalRPN(tokens):
    # Initialize a stack to hold numbers
    stack = []
    # Loop through each token in the input list
    for token in tokens:
        # If token is "+", "-", "*", "/", then:
        # Push the two nearest values; note the order is important
        b = stack.pop()
        a = stack.pop()
        # Apply the operation based on the token
        if token == "+":
            result = a + b
        elif token == "-":
            result = a - b
        elif token == "*":
            result = a * b
        else:
            result = a / b
        # Use int division to truncate towards zero
        result = int(result)
        # Push the result back onto the stack
        stack.append(result)
    # If token is a number, push its integer value onto the stack
    stack.append(int(token))
    # The final result is the only element left in the stack
    return stack.pop()
```

```
# Example usage:
tokens = ["2","3","+","4","+"]
print(evalRPN(tokens)) # Output: 9
```

Python

```
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        stack = []
        op = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b),
        }
        for token in tokens:
            if token in op:
                b = stack.pop()
                a = stack.pop()
                stack.append(op[token](a, b))
            else:
                stack.append(int(token))
        return stack.pop()
```

Python

Stack · LeetCode — By Pattern

— 2762 —

LeetCode

C++

C++

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<long> stack;
        const unordered_map<string, function<long(long, long)>> op = {
            "+", std::plus<long>{}),
            "-", std::minus<long>{}),
            "*", std::multiplies<long>{}),
            "/", std::divides<long>{});
        for (const string& token : tokens) {
            if (op.contains(token)) {
                const long b = stack.top();
                stack.pop();
                const long a = stack.top();
                stack.pop();
                stack.push(op[token](a, b));
            } else {
                stack.push(stoi(token));
            }
        }
        return stack.top();
    }
};
```

C++

```
class Solution {
public:
    int evalRPN(vector<string>& tokens) {
        stack<int> stk;
        for (const t : tokens) {
            if (t.size() > 1 || isdigit(t[0])) {
                stk.push(stoi(t));
            } else {
                int y = stk.top();
                stk.pop();
                int x = stk.top();
                stk.pop();
                if (t[0] == '+')
                    stk.push(x + y);
                else if (t[0] == '-')
                    stk.push(x - y);
                else if (t[0] == '*')
                    stk.push(x * y);
                else
                    stk.push(x / y);
            }
        }
        return stk.top();
    }
};
```

C++

Stack · LeetCode — By Pattern

— 2764 —

LeetCode

```
class Solution {
    public int evalRPN(String[] tokens) {
        Deque<Integer> stk = new ArrayDeque<>();
        for (String t : tokens) {
            if (t.length() > 1 || Character.isDigit(t.charAt(0))) {
                stk.push(Integer.parseInt(t));
            } else {
                int y = stk.pop();
                int x = stk.pop();
                switch (t) {
                    case "+":
                        stk.push(x + y);
                        break;
                    case "-":
                        stk.push(x - y);
                        break;
                    case "*":
                        stk.push(x * y);
                        break;
                    case "/":
                        stk.push(x / y);
                        break;
                }
            }
        }
        return stk.pop();
    }
}
```

Java

using System.Collections.Generic;

```
public class Solution {
    public int EvalRPN(string[] tokens) {
        var stack = new Stack<int>();
        foreach (var token in tokens) {
            switch (token) {
                case "+":
                    stack.Push(stack.Pop() + stack.Pop());
                    break;
                case "-":
                    stack.Push(-stack.Pop() + stack.Pop());
                    break;
                case "*":
                    stack.Push(stack.Pop() * stack.Pop());
                    break;
                case "/":
                    var right = stack.Pop();
                    stack.Push(stack.Pop() / right);
                    break;
                default:
                    stack.Push(int.Parse(token));
                    break;
            }
        }
    }
}
```

Stack · LeetCode — By Pattern

— 2766 —

LeetCode

```
    }
    return stack.Pop();
}
}

Java

class Solution {
    public int evalRPN(String[] tokens) {
        Deque<Integer> stk = new ArrayDeque<>();
        for (String t : tokens) {
            if (t.length() > 1 || Character.isDigit(t.charAt(0))) {
                stk.push(Integer.parseInt(t));
            } else {
                int y = stk.pop();
                int x = stk.pop();
                switch (t) {
                    case "+":
                        stk.push(x + y);
                        break;
                    case "-":
                        stk.push(x - y);
                        break;
                    case "*":
                        stk.push(x * y);
                        break;
                    case "/":
                        stk.push(x / y);
                        break;
                    default:
                        stk.push((x - y) / x);
                        break;
                }
            }
        }
        return stk.pop();
    }
}

Java
```

Stack · LeetCode — By Pattern

— 2767 —

LeetCode

```
using System.Collections.Generic;

public class Solution {
    public int EvalRPN(string[] tokens) {
        var stack = new Stack<int>();
        foreach (var token in tokens) {
            switch (token) {
                case "+":
                    stack.Push(stack.Pop() + stack.Pop());
                    break;
                case "-":
                    stack.Push(-stack.Pop() + stack.Pop());
                    break;
                case "*":
                    stack.Push(stack.Pop() * stack.Pop());
                    break;
                case "/":
                    var right = stack.Pop();
                    stack.Push(stack.Pop() / right);
                    break;
                default:
                    stack.Push(int.Parse(token));
                    break;
            }
        }
        return stack.Pop();
    }
}

TypeScript
TypeScript
```

Stack · LeetCode — By Pattern

— 2768 —

LeetCode

```
function evalRPN(tokens: string[]): number {
    const stack = [];
    for (const token of tokens) {
        if (/^(\d+)$/i.test(token)) {
            stack.push(Number(token));
        } else {
            const a = stack.pop();
            const b = stack.pop();
            switch (token) {
                case "+":
                    stack.push(b + a);
                    break;
                case "-":
                    stack.push(b - a);
                    break;
                case "*":
                    stack.push(b * a);
                    break;
                case "/":
                    stack.push(Math.floor(b / a));
                    break;
            }
        }
    }
    return stack[0];
}

TypeScript

function evalRPN(tokens: string[]): number {
    const stack = [];
    for (const token of tokens) {
        if (/^(\d+)$/i.test(token)) {
            stack.push(Number(token));
        } else {
            const a = stack.pop();
            const b = stack.pop();
            switch (token) {
                case "+":
                    stack.push(b + a);
                    break;
                case "-":
                    stack.push(b - a);
                    break;
                case "*":
                    stack.push(b * a);
                    break;
                case "/":
                    stack.push(Math.floor(b / a));
                    break;
            }
        }
    }
    return stack[0];
}

Go
```

Stack · LeetCode — By Pattern

— 2769 —

LeetCode

```
Go

func evalRPN(tokens []string) int {
    // tokens: ["tokens", "new", "operator", "push", "arraystack"]
    stk := arraystack.New()
    for _, token := range tokens {
        if len(tokens) > 1 || token[0] == '-' || token[0] == '+' {
            num, _ := strconv.Atoi(token)
            stk.Push(num)
        } else {
            y := popInt(stk)
            x := popInt(stk)
            switch token {
                case "+":
                    stk.Push(x + y)
                    break;
                case "-":
                    stk.Push(x - y)
                    break;
                case "*":
                    stk.Push(x * y)
                    break;
                case "/":
                    stk.Push(x / y)
                    break;
            }
        }
    }
    return popInt(stk)
}

func popInt(stack *arraystack.Stack) int {
    v, _ := stack.Pop()
    return v.(int)
}

Go

func evalRPN(tokens []string) int {
    // tokens: ["tokens", "new", "operator", "push", "arraystack"]
    stk := arraystack.New()
    for _, token := range tokens {
        if len(tokens) > 1 || token[0] == '-' || token[0] == '+' {
            num, _ := strconv.Atoi(token)
            stk.Push(num)
        } else {
            y := popInt(stk)
            x := popInt(stk)
            switch token {
                case "+":
                    stk.Push(x + y)
                    break;
                case "-":
                    stk.Push(x - y)
                    break;
                case "*":
                    stk.Push(x * y)
                    break;
                case "/":
                    stk.Push(x / y)
                    break;
            }
        }
    }
    return popInt(stk)
}

return popInt(stk)
}
```

Stack · LeetCode — By Pattern

— 2770 —

LeetCode

```
func popInt(stack *arraystack.Stack) int {
    v, _ := stack.Pop()
    return v.(int)
}

Rust
Rust

impl Solution {
    pub fn eval_post(tokens: Vec<String>) -> i32 {
        let mut stack = vec![];
        for token in tokens {
            match token.parse() {
                Ok(num) => stack.push(num),
                Err(_) => {
                    let a = stack.pop().unwrap();
                    let b = stack.pop().unwrap();
                    stack.push(match token.as_str() {
                        "+" => b + a,
                        "-" => b - a,
                        "*" => b * a,
                        "/" => b / a,
                        _ => 0,
                    });
                }
            }
        }
        stack[0]
    }
}

Rust

impl Solution {
    pub fn eval_post(tokens: Vec<String>) -> i32 {
        let mut stack = vec![];
        for token in tokens {
            match token.parse() {
                Ok(num) => stack.push(num),
                Err(_) => {
                    let a = stack.pop().unwrap();
                    let b = stack.pop().unwrap();
                    stack.push(match token.as_str() {
                        "+" => b + a,
                        "-" => b - a,
                        "*" => b * a,
                        "/" => b / a,
                        _ => 0,
                    });
                }
            }
        }
        stack[0]
    }
}

[*] Stack — Pattern Solution (matched from community answers)
Python
```

Stack · LeetCode — By Pattern

— 2771 —

LeetCode

```
def evalRPN(tokens):
    # Initialize a stack to hold numbers
    stack = []
    # Loop through each token in the input list
    for token in tokens:
        # If token is "+", "-", "*", or "/"
        if token in ["+", "-", "*", "/"]:
            # Pop the two topmost values; note the order is important
            b = stack.pop()
            a = stack.pop()
            # Apply the operation based on the token
            if token == "+":
                result = a + b
            elif token == "-":
                result = a - b
            elif token == "*":
                result = a * b
            else:
                # Use int division to truncate towards zero
                result = int(a / b)
            # Push the result back onto the stack
            stack.append(result)
        else:
            # Token is a number; push its integer value onto the stack
            stack.append(int(token))
    # The final result is the only element left in the stack
    return stack.pop()

# Example usage:
# tokens = ["2", "1", "+", "3", "*"]
# print(evalRPN(tokens)) # Output: 9
```

#155

MEDIUM

Min Stack

LeetCode · Simplify · WalkCC · LeetCode · Editorial

Stack · Design

Lists Grid 169

Problem:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

```
Input
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[1],[-3],[2],[1],[],[1]]
```

Stack · LeetCode — By Pattern

— 2772 —

LeetCode

```
[1,1,-2],[0],[-3],[1],[1],[1]]

Output
[mult,mult,mult,mult,-3,mult,0,-2]
```

Explanation

```
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top(); // return 0
minStack.getMin(); // return -2
```

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- Methods pop, top and getMin operations will always be called on non-empty stacks.
- At most $3 * 10^4$ calls will be made to push, pop, top, and getMin.

Python

Python

Python Implementation of MinStack with detailed comments

```
class MinStack:
    def __init__(self):
        # Main stack to store all values
        self.stack = []
        # Auxiliary stack to store minimum values
        self.min_stack = []

    def push(self, val: int) -> None:
        # Push value onto main stack
        self.stack.append(val)
        # If min_stack is empty or the new value is less than or equal to current minimum, push it onto
        min_stack
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        # Pop the top value from main stack
        popped_value = self.stack.pop()
        # If the popped value is the current minimum, pop it from min_stack as well
        if popped_value == self.min_stack[-1]:
            self.min_stack.pop()

    def top(self) -> int:
        # Return the top value from main stack without removing it
```

Stack · LeetCode - By Pattern

— 2773 —

LeetCode

```
return self.stack[-1]

def getMin(self) -> int:
    # Return the minimum value from min_stack
    return self.min_stack[-1]

# Example usage:
# minStack = MinStack()
# minStack.push(-2)
# minStack.push(0)
# minStack.push(-3)
# print(minStack.getMin()) # -> -3
# minStack.pop()
# print(minStack.top()) # -> 0
# print(minStack.getMin()) # -> -2
```

Python

```
class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, x: int) -> None:
        m = None if not self.stack else min(self.stack[-1][1], x)
        self.stack.append([x, m])

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]
```

Python

Stack · LeetCode - By Pattern

— 2774 —

LeetCode

```
class MinStack:
    def __init__(self):
        self.stk1 = []
        self.stk2 = []

    def push(self, val: int) -> None:
        self.stk1.append(val)
        self.stk2.append(min(val, self.stk2[-1]))

    def pop(self) -> None:
        self.stk1.pop()
        self.stk2.pop()

    def top(self) -> int:
        return self.stk1[-1]

    def getMin(self) -> int:
        return self.stk2[-1]

# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(val)
# obj.pop()
# param_1 = obj.top()
```

Python

```
class MinStack:
    def __init__(self):
        self.sk = []
        self.minst = []

    def push(self, x: int) -> None:
        self.sk.append(x)
        self.minst.append(min(x, self.minst[-1]))

    def pop(self) -> None:
        if not self.sk:
            return
        self.sk.pop()
        self.minst.pop()

    def top(self) -> int:
        return self.sk[-1]

    def getMin(self) -> int:
        return self.minst[-1]
```

Your MinStack object will be instantiated and called as such:
obj = MinStack()
obj.push(x)

Stack · LeetCode - By Pattern

— 2775 —

LeetCode

```
# obj.pop()

# param_1 = obj.top()
# param_2 = obj.getMin()

#####

class MinStack:
    def __init__(self):
        self.sk = []
        self.minst = []

    def push(self, val: int) -> None:
        self.sk.append(val)

        # empty check
        self.minst.append(val if not self.minst else min(val, self.minst[-1]))

    def pop(self) -> None:
        if not self.sk:
            return
        self.sk.pop()
        self.minst.pop()

    def top(self) -> int:
        return self.sk[-1]

    def getMin(self) -> int:
        return self.minst[-1]

# runtime above, using only 2 stack, with stack element as tuple: (val, its associated min)

class MinStack:
    def __init__(self):
        self._stack = []

    def push(self, x: int) -> None:
        cur_min = self.getMin()
        if x <= cur_min:
            cur_min = x
        self._stack.append((x, cur_min))

    def pop(self) -> None:
        self._stack.pop()

    def top(self) -> int:
        if not self._stack:
            return None
        else:
            return self._stack[-1][0]

    def getMin(self) -> int:
        if not self._stack:
            return float('inf')
```

Java

Stack · LeetCode - By Pattern

— 2776 —

LeetCode

```
Java

class MinStack {
    public void push(int x) {
        if (stack.isEmpty())
            stack.push(new int[] {x, x});
        else
            stack.push(new int[] {x, Math.min(x, stack.peek()[1])});
    }

    public void pop() {
        stack.pop();
    }

    public int top() {
        return stack.peek()[0];
    }

    public int getMin() {
        return stack.peek()[1];
    }

    private Stack<int[]> stack = new Stack<>(); // (x, min)
}
```

Java

```
class MinStack {
    private Deque<Integer> stk1 = new ArrayDeque<>();
    private Deque<Integer> stk2 = new ArrayDeque<>();

    public MinStack() {
        stk2.push(Integer.MAX_VALUE);
    }

    public void push(int val) {
        stk1.push(val);
        stk2.push(Math.min(val, stk2.peek()));
    }

    public void pop() {
        stk1.pop();
        stk2.pop();
    }

    public int top() {
        return stk1.peek();
    }

    public int getMin() {
        return stk2.peek();
    }
}
```

Stack · LeetCode - By Pattern

— 2777 —

LeetCode

```
}

/*
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.push(x);
 * obj.pop();
 * int param_1 = obj.top();
 * int param_2 = obj.getMin();
 */
```

Java

```
public class MinStack {
    private Stack<int> stk1 = new Stack<>();
    private Stack<int> stk2 = new Stack<>();

    public MinStack() {
        stk2.Push(Integer.MAX_VALUE);
    }

    public void Push(int x) {
        stk1.Push(x);
        stk2.Push(Math.Min(x, GetMin()));
    }

    public void Pop() {
        stk1.Pop();
        stk2.Pop();
    }

    public int Top() {
        return stk1.Peek();
    }

    public int GetMin() {
        return stk2.Peek();
    }
}
```

```
/*
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.Push(x);
 * obj.Pop();
 * int param_1 = obj.Top();
 * int param_2 = obj.GetMin();
 */
```

Java

Stack · LeetCode - By Pattern

— 2778 —

LeetCode

```

class MinStack {
    private Queue<Integer> stkl = new ArrayDeque<>();
    private Queue<Integer> stk2 = new ArrayDeque<>();

    public MinStack() {
        stk2.push(Integer.MAX_VALUE);
    }

    public void push(int val) {
        stkl.push(val);
        stk2.push(Math.min(val, stk2.peek()));
    }

    public void pop() {
        stkl.pop();
        stk2.pop();
    }

    public int top() {
        return stkl.peek();
    }

    public int getMin() {
        return stk2.peek();
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.push(val);
 * obj.pop();
 * int param_2 = obj.top();
 * int param_3 = obj.getMin();
 */

```

Java

```

public class MinStack {
    private Stack<int> stkl = new Stack<>();
    private Stack<int> stk2 = new Stack<>();

    public MinStack() {
        stk2.push(Integer.MAX_VALUE);
    }

    public void push(int x) {
        stkl.push(x);
        stk2.push(Math.min(x, GetMin()));
    }

    public void pop() {
        stkl.pop();
        stk2.pop();
    }

    public int top() {
        return stkl.peek();
    }

    public int GetMin() {
        return stk2.Peek();
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.push(x);
 * obj.pop();
 * int param_2 = obj.top();
 * int param_3 = obj.getMin();
 */

```

JavaScript
JavaScript

```

var MinStack = function () {
    this.stkl = [];
    this.stk2 = [Infinity];
};

/**
 * @param {number} val
 * @return {void}
 */
MinStack.prototype.push = function (val) {
    this.stkl.push(val);
    this.stk2.push(Math.min(this.stk2[this.stk2.length - 1], val));
};

/**
 * @return {void}
 */
MinStack.prototype.pop = function () {
    this.stkl.pop();
    this.stk2.pop();
};

/**
 * @return {number}
 */
MinStack.prototype.top = function () {
    return this.stkl[this.stkl.length - 1];
};

/**
 * @return {number}
 */
MinStack.prototype.getMin = function () {
    return this.stk2[this.stk2.length - 1];
};

/**
 * Your MinStack object will be instantiated and called as such:
 * var obj = new MinStack();
 * obj.push(val);
 * obj.pop();
 * var param_2 = obj.top();
 * var param_3 = obj.getMin();
 */

```

JavaScript

```

var MinStack = function () {
    this.stkl = [];
    this.stk2 = [Infinity];
};

/**
 * @param {number} val
 * @return {void}
 */
MinStack.prototype.push = function (val) {
    this.stkl.push(val);
    this.stk2.push(Math.min(this.stk2[this.stk2.length - 1], val));
};

/**
 * @return {void}
 */
MinStack.prototype.pop = function () {
    this.stkl.pop();
    this.stk2.pop();
};

/**
 * @return {number}
 */
MinStack.prototype.top = function () {
    return this.stkl[this.stkl.length - 1];
};

/**
 * @return {number}
 */
MinStack.prototype.getMin = function () {
    return this.stk2[this.stk2.length - 1];
};

/**
 * Your MinStack object will be instantiated and called as such:
 * var obj = new MinStack();
 * obj.push(val);
 * obj.pop();
 * var param_2 = obj.top();
 * var param_3 = obj.getMin();
 */

```

TypeScript
TypeScript

```

class MinStack {
    stkl: number[];
    stk2: number[];

    constructor() {
        this.stkl = [];
        this.stk2 = [Infinity];
    }

    push(val: number): void {
        this.stkl.push(val);
        this.stk2.push(Math.min(val, this.stk2[this.stk2.length - 1]));
    }

    pop(): void {
        this.stkl.pop();
        this.stk2.pop();
    }

    top(): number {
        return this.stkl[this.stkl.length - 1];
    }

    getMin(): number {
        return this.stk2[this.stk2.length - 1];
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * var obj = new MinStack();
 * obj.push(x);
 * obj.pop();
 * var param_2 = obj.top();
 * var param_3 = obj.getMin();
 */

```

TypeScript

```

class MinStack {
    stkl: number[];
    stk2: number[];

    constructor() {
        this.stkl = [];
        this.stk2 = [Infinity];
    }

    push(val: number): void {
        this.stkl.push(val);
        this.stk2.push(Math.min(val, this.stk2[this.stk2.length - 1]));
    }

    pop(): void {
        this.stkl.pop();
        this.stk2.pop();
    }

    top(): number {
        return this.stkl[this.stkl.length - 1];
    }

    getMin(): number {
        return this.stk2[this.stk2.length - 1];
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * var obj = new MinStack();
 * obj.push(x);
 * obj.pop();
 * var param_2 = obj.top();
 * var param_3 = obj.getMin();
 */

```

Go
Go

```
type MinStack struct {
    stk1 []int
    stk2 []int
}

func Constructor() MinStack {
    return MinStack{[]int{}, []int{math.MaxInt32}}
}

func (this *MinStack) Push(val int) {
    this.stk1 = append(this.stk1, val)
    this.stk2 = append(this.stk2, min(val, this.stk2[len(this.stk2)-1]))
}

func (this *MinStack) Pop() {
    this.stk1 = this.stk1[:len(this.stk1)-1]
    this.stk2 = this.stk2[:len(this.stk2)-1]
}

func (this *MinStack) Top() int {
    return this.stk1[len(this.stk1)-1]
}

func (this *MinStack) GetMin() int {
    return this.stk2[len(this.stk2)-1]
}

}
```

```
/*
 * Your MinStack object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Push(val);
 * obj.Pop();
 * param_1 := obj.Top();
 * param_2 := obj.GetMin();
 */
```

Go

Stack · LeetCode — By Pattern

— 2785 —

LeetCode

```
type MinStack struct {
    stk1 []int
    stk2 []int
}

func Constructor() MinStack {
    return MinStack{[]int{}, []int{math.MaxInt32}}
}

func (this *MinStack) Push(val int) {
    this.stk1 = append(this.stk1, val)
    this.stk2 = append(this.stk2, min(val, this.stk2[len(this.stk2)-1]))
}

func (this *MinStack) Pop() {
    this.stk1 = this.stk1[:len(this.stk1)-1]
    this.stk2 = this.stk2[:len(this.stk2)-1]
}

func (this *MinStack) Top() int {
    return this.stk1[len(this.stk1)-1]
}

func (this *MinStack) GetMin() int {
    return this.stk2[len(this.stk2)-1]
}

}
```

```
/*
 * Your MinStack object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Push(val);
 * obj.Pop();
 * param_1 := obj.Top();
 * param_2 := obj.GetMin();
 */
```

Rust

Rust

Stack · LeetCode — By Pattern

— 2786 —

LeetCode

```
use std::collections::VecDeque;

struct MinStack {
    stk1: VecDeque<i32>,
    stk2: VecDeque<i32>,
}

/*
 * 'self' means the method takes an immutable reference.
 * If you need a mutable reference, change it to 'mut self' instead.
 */

impl MinStack {
    fn new() -> Self {
        Self {
            stk1: VecDeque::new(),
            stk2: VecDeque::new(),
        }
    }

    fn push(&mut self, x: i32) {
        self.stk1.push_back(x);
        if self.stk2.is_empty() || self.stk2.back().unwrap() >= x {
            self.stk2.push_back(x);
        }
    }

    fn pop(&mut self) {
        let val = self.stk1.pop_back().unwrap();
        if self.stk2.back().unwrap() == val {
            self.stk2.pop_back();
        }
    }

    fn top(&self) -> i32 {
        *self.stk1.back().unwrap()
    }

    fn get_min(&self) -> i32 {
        *self.stk2.back().unwrap()
    }
}
```

Rust

Stack · LeetCode — By Pattern

— 2787 —

LeetCode

```
use std::collections::VecDeque;

struct MinStack {
    stk1: VecDeque<i32>,
    stk2: VecDeque<i32>,
}

/*
 * 'self' means the method takes an immutable reference.
 * If you need a mutable reference, change it to 'mut self' instead.
 */

impl MinStack {
    fn new() -> Self {
        Self {
            stk1: VecDeque::new(),
            stk2: VecDeque::new()
        }
    }

    fn push(&mut self, x: i32) {
        self.stk1.push_back(x);
        if self.stk2.is_empty() || self.stk2.back().unwrap() >= x {
            self.stk2.push_back(x);
        }
    }

    fn pop(&mut self) {
        let val = self.stk1.pop_back().unwrap();
        if self.stk2.back().unwrap() == val {
            self.stk2.pop_back();
        }
    }

    fn top(&self) -> i32 {
        *self.stk1.back().unwrap()
    }

    fn get_min(&self) -> i32 {
        *self.stk2.back().unwrap()
    }
}
```

```
/*
 * Your MinStack object will be instantiated and called as such:
 * let obj = MinStack::new();
 * obj.push(x);
 * obj.pop();
 * let ret_1: i32 = obj.top();
 * let ret_2: i32 = obj.get_min();
 */
```

[*] Stack — Pattern Solution (matched from community answers)

Python

Stack · LeetCode — By Pattern

— 2788 —

LeetCode

```
# Python implementation of MinStack with detailed comments

class MinStack:
    def __init__(self):
        # MinStack class to store all values
        self.stack = []
        # Auxiliary stack to store minimum values
        self.min_stack = []

    def push(self, val: int) -> None:
        # Push value into main stack
        self.stack.append(val)
        # If min_stack is empty or the new value is less than or equal to current minimum, push it onto min_stack
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        # Pop the top value from main stack
        popped_value = self.stack.pop()
        # If the popped value is the current minimum, pop it from min_stack as well
        if popped_value == self.min_stack[-1]:
            self.min_stack.pop()

    def top(self) -> int:
        # Return the top value from main stack without removing it
        return self.stack[-1]

    def getMin(self) -> int:
        # Return the minimum value from min_stack
        return self.min_stack[-1]

# Example usage:
# minStack = MinStack()
# minStack.push(-2)
# minStack.push(-1)
# minStack.push(-3)
# print(minStack.getMin()) # -> -3
# minStack.pop()
# print(minStack.top()) # -> -1
# print(minStack.getMin()) # -> -2
```

#173

MEDIUM

Binary Search Tree Iterator

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Stack · Tree · Design · Binary Search Tree · Recursion

Like · Denny Zhang

Problem:

Implement the BSTIterator class that represents an iterator over the in-order traversal of a binary search tree (BST):

- BSTIterator(TreeNode root) initializes an object of the BSTIterator class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number

Stack · LeetCode — By Pattern

— 2789 —

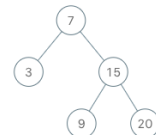
LeetCode

- smaller than any element in the BST.
- boolean hasNext() Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- int next() Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to next() will return the smallest element in the BST.

You may assume that next() calls will always be valid. That is, there will be at least a next number in the in-order traversal when next() is called.

Example 1:



```
Input:
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next", "hasNext"]
[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], []]
Output:
[null, 3, 7, true, 9, true, 15, true, 20, false]
```

Explanation

BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);

bSTIterator.next(); // return 3

bSTIterator.next(); // return 7

bSTIterator.hasNext(); // return True

bSTIterator.next(); // return 9

bSTIterator.hasNext(); // return True

bSTIterator.next(); // return 15

bSTIterator.hasNext(); // return True

bSTIterator.next(); // return 20

bSTIterator.hasNext(); // return False

Constraints:

- The number of nodes in the tree is in the range [1, 105].
- 0 <= Node.val <= 106
- At most 105 calls will be made to hasNext, and next.

Follow up:

Stack · LeetCode — By Pattern

— 2790 —

LeetCode

• Could you implement next() and hasNext() to run in average O(1) time and use O(h) memory, where h is the height of the tree?

Python

```
Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class BSTIterator:
    def __init__(self, root):
        # Initialize an empty stack to simulate in-order traversal
        self.stack = []
        # Push all left nodes starting from the root
        self._push_left_branch(root)

    def _push_left_branch(self, node):
        # Push node and all its left children to the stack
        while node:
            self.stack.append(node)
            node = node.left

    def next(self):
        # Pop the top node which represents the next smallest element
        node = self.stack.pop()
        # If the node has a right child, push its left branch onto the stack
        if node.right:
            self._push_left_branch(node.right)
        # Return the node's value
        return node.val

    def hasNext(self):
        # The iterator has a next element if the stack is not empty
        return len(self.stack) > 0
```

Python

Stack · LeetCode · By Pattern

— 2791 —

LeetCode

```
class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.i = 0
        self.vals = []
        self._inorder(root)

    def next(self) -> int:
        self.i += 1
        return self.vals[self.i - 1]

    def hasNext(self) -> bool:
        return self.i < len(self.vals)

    def _inorder(self, root: TreeNode | None) -> None:
        if not root:
            return
        self._inorder(root.left)
        self.vals.append(root.val)
        self._inorder(root.right)
```

Python

```
class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.stack = []
        self._pushLeftUntilNull(root)

    def next(self) -> int:
        root = self.stack.pop()
        self._pushLeftUntilNull(root.right)
        return root.val

    def hasNext(self) -> bool:
        return self.stack

    def _pushLeftUntilNull(self, root: TreeNode | None) -> None:
        while root:
            self.stack.append(root)
            root = root.left
```

Python

Stack · LeetCode · By Pattern

— 2792 —

LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class BSTIterator:
    def __init__(self, root: TreeNode):
        if root:
            inorder(root.left)
            self.vals.append(root.val)
            inorder(root.right)

        self.cur = 0
        self.vals = []
        inorder(root)

    def next(self) -> int:
        res = self.vals[self.cur]
        self.cur += 1
        return res

    def hasNext(self) -> bool:
        return self.cur < len(self.vals)
```

```
Python
# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()
# param_2 = obj.hasNext()

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class BSTIterator:
    def __init__(self, root: TreeNode):
        self.stack = []
        self._leftmost_inorder(root)

    def _leftmost_inorder(self, node):
        while node:
            self.stack.append(node)
            node = node.left

    def next(self) -> int:
        cur = self.stack.pop()
        node = cur.right
        self._leftmost_inorder(node)
        return cur.val

    def hasNext(self) -> bool:
        return len(self.stack) > 0
```

Stack · LeetCode · By Pattern

— 2793 —

LeetCode

```
# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()
# param_2 = obj.hasNext()

// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class BSTIterator {
    int root;
    Stack<TreeNode> stack;
    root = root.left;

    def next() -> int {
        cur = self.stack.pop()
        node = cur.right
        while node:
            self.stack.append(node)
            node = node.left
        return cur.val
    }

    def hasNext() -> bool {
        return len(self.stack) > 0
    }
}
```

C++

C++

Stack · LeetCode · By Pattern

— 2794 —

LeetCode

```
class BSTIterator {
public:
    BSTIterator(TreeNode* root) {
        inorder(root);
    }

    int next() {
        return vals[car++];
    }

    bool hasNext() {
        return i < vals.size();
    }

private:
    int i = 0;
    vector<int> vals;

    void inorder(TreeNode* root) {
        if (root == nullptr)
            return;
        inorder(root->left);
        vals.push_back(root->val);
        inorder(root->right);
    }
};
```

```
C++
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this->val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this->val = val;
//         this->left = left;
//         this->right = right;
//     }
// };

class BSTIterator {
public:
    vector<int> vals;
    int car;
    BSTIterator(TreeNode* root) {
        car = 0;
        inorder(root);
    }

    int next() {
        return vals[car++];
    }

    bool hasNext() {
        return car < vals.size();
    }
};
```

Stack · LeetCode · By Pattern

— 2795 —

LeetCode

```
return cur = vals.size();

void inorder(TreeNode* root) {
    if (root) {
        inorder(root->left);
        vals.push_back(root->val);
        inorder(root->right);
    }
}

// Your BSTIterator object will be instantiated and called as such:
// BSTIterator obj = new BSTIterator(root);
// int param_1 = obj.next();
// bool param_2 = obj.hasNext();
```

Java

```
class BSTIterator {
    public BSTIterator(TreeNode root) {
        inorder(root);
    }

    public int next() {
        return vals.get(car++);
    }

    public boolean hasNext() {
        return i < vals.size();
    }

    private int i = 0;
    private List<Integer> vals = new ArrayList<>();

    private void inorder(TreeNode root) {
        if (root == null)
            return;
        inorder(root.left);
        vals.add(root.val);
        inorder(root.right);
    }
}
```

Java

Stack · LeetCode · By Pattern

— 2796 —

LeetCode

```

class BSTIterator {
public:
    BSTIterator(TreeNode root) {
        pushLeftUntilNull(root);
    }

    public int next() {
        TreeNode root = stack.pop();
        pushLeftUntilNull(root.right);
        return root.val;
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

private:
    Deque<TreeNode> stack = new ArrayDeque<>();

private:
    void pushLeftUntilNull(TreeNode root) {
        while (root != null) {
            stack.push(root);
            root = root.left;
        }
    }
}
}

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class BSTIterator {
    private int cur = 0;
    private List<Integer> vals = new ArrayList<>();

    public BSTIterator(TreeNode root) {
        inorder(root);
    }

    public int next() {
        return vals.get(cur++);
    }
}

```

```

}

public boolean hasNext() {
    return cur < vals.size();
}

private void inorder(TreeNode root) {
    if (root != null) {
        inorder(root.left);
        vals.add(root.val);
        inorder(root.right);
    }
}
}

//
// Your BSTIterator object will be instantiated and called as such:
// BSTIterator obj = new BSTIterator(root);
// int param_1 = obj.next();
// boolean param_2 = obj.hasNext();
//
}

//
// Definition for binary tree
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) { val = x; }
}

//
// public class BSTIterator {
//     Stack<TreeNode> sk;
//
//     // Write your code here. Important feature, min is always going left branch to leaf
//     public BSTIterator(TreeNode root) {
//         sk = new Stack<>();
//
//         // all the way to leftmost leaf
//     }
// }

```

```

while (root != null) {
    sk.push(root);
    root = root.left;
}

// Returns whether we have a next smallest number */
public boolean hasNext() {
    return !sk.isEmpty();
}

// Returns the next smallest number */
public int next() {
    TreeNode minNode = sk.pop();
    TreeNode current = minNode;
    // nodes above, possible next time min is from its right-then-left branch
    if (current.right != null) {
        current = current.right;
    }

    // some logic for the constructor
    while (current != null) {
        sk.push(current.left);
        current = current.left;
    }

    return minNode.val;
}

//
// Your BSTIterator will be called like this:
// BSTIterator i = new BSTIterator(root);
// while (i.hasNext()) v[i++] = i.next();
//
}

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}
}

```

```

//
// Definition for a binary tree node.
//
function TreeNode(val, left, right) {
    this.val = (val===undefined ? 0 : val);
    this.left = (left===undefined ? null : left);
    this.right = (right===undefined ? null : right);
}

//
// Return [TreeNode] root
//
var BSTIterator = function (root) {
    this.stack = [];
    for (i: root != null; root = root.left) {
        this.stack.push(root);
    }
}

//
// Returns (number)
//
BSTIterator.prototype.next = function () {
    let cur = this.stack.pop();
    let mode = cur.right;
    for (; mode != null; mode = mode.left) {
        this.stack.push(mode);
    }
    return cur.val;
}

//
// Returns (boolean)
//
BSTIterator.prototype.hasNext = function () {
    return this.stack.length > 0;
}

//
// Your BSTIterator object will be instantiated and called as such:
// var obj = new BSTIterator(root);
// var param_1 = obj.next();
// var param_2 = obj.hasNext();
//
}

```

```

//
// Definition for a binary tree node.
//
function TreeNode(val, left, right) {
    this.val = (val===undefined ? 0 : val);
    this.left = (left===undefined ? null : left);
    this.right = (right===undefined ? null : right);
}

//
// Return [TreeNode] root
//
var BSTIterator = function (root) {
    this.stack = [];
    for (i: root != null; root = root.left) {
        this.stack.push(root);
    }
}

//
// Returns (number)
//
BSTIterator.prototype.next = function () {
    let cur = this.stack.pop();
    let mode = cur.right;
    for (; mode != null; mode = mode.left) {
        this.stack.push(mode);
    }
    return cur.val;
}

//
// Returns (boolean)
//
BSTIterator.prototype.hasNext = function () {
    return this.stack.length > 0;
}

//
// Your BSTIterator object will be instantiated and called as such:
// var obj = new BSTIterator(root);
// var param_1 = obj.next();
// var param_2 = obj.hasNext();
//
}

```

```

//
// Definition for a binary tree node.
//
class TreeNode {
    val: number;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
        this.val = (val===undefined ? 0 : val);
        this.left = (left===undefined ? null : left);
        this.right = (right===undefined ? null : right);
    }
}

class BSTIterator {
    private data: number[];
    private index: number;

    constructor(root: TreeNode | null) {
        this.index = 0;
        this.data = [];
        const dfs = (root: TreeNode | null) => {
            if (root == null) {
                return;
            }
            dfs(left);
            this.data.push(val);
            dfs(right);
        };
        dfs(root);

        next(): number {
            return this.data[this.index++];
        }

        hasNext(): boolean {
            return this.index < this.data.length;
        }
    }

    //
    // Your BSTIterator object will be instantiated and called as such:
    // var obj = new BSTIterator(root);
    // var param_1 = obj.next();
    // var param_2 = obj.hasNext();
    //
}

```

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     struct TreeNode *left;
 *     struct TreeNode *right;
 * };
 */
class BSTIterator {
private: stack<TreeNode*> st;

public:
    BSTIterator(TreeNode* root) {
        this->st = {};
        dfs(root);
    }

    int next() {
        if (st.empty()) return -1;
        auto node = st.top();
        st.pop();
        if (node->right) dfs(node->right);
        return node->val;
    }

    bool hasNext() {
        return !st.empty();
    }
};
```

/*
 * Your BSTIterator object will be instantiated and called as such:
 * BSTIterator i(root);
 * while (i.hasNext()) cout << i.next() << " ";
 */

Rust

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     struct TreeNode *left;
 *     struct TreeNode *right;
 * };
 */
class BSTIterator {
private: stack<TreeNode*> st;

public:
    BSTIterator(TreeNode* root) {
        this->st = {};
        dfs(root);
    }

    int next() {
        if (st.empty()) return -1;
        auto node = st.top();
        st.pop();
        if (node->right) dfs(node->right);
        return node->val;
    }

    bool hasNext() {
        return !st.empty();
    }
};
```

/*
 * Your BSTIterator object will be instantiated and called as such:
 * BSTIterator i(root);
 * while (i.hasNext()) cout << i.next() << " ";
 */

```
self.dfs(node.right.take(), &mut self.stack)
}

fn has_next(&self) -> bool {
    !self.stack.is_empty()
}
```

/*
 * Your BSTIterator object will be instantiated and called as such:
 * BSTIterator i(root);
 * while (i.hasNext()) cout << i.next() << " ";
 */

[*] Stack — Pattern Solution (matched from community answers)

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     struct TreeNode *left;
 *     struct TreeNode *right;
 * };
 */
class BSTIterator {
private: stack<TreeNode*> st;

public:
    BSTIterator(TreeNode* root) {
        this->st = {};
        dfs(root);
    }

    int next() {
        if (st.empty()) return -1;
        auto node = st.top();
        st.pop();
        if (node->right) dfs(node->right);
        return node->val;
    }

    bool hasNext() {
        return !st.empty();
    }
};
```

#227 MEDIUM Basic Calculator II

LeetCode · SimplifyLeetCode · WalkCC · LeetCode · Editorial

Math · String · Stack

Like · 100

Problem:
Given a string *s* which represents an expression, evaluate this expression and return its value.
The integer division should truncate toward zero.
You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:
Input: *s* = "3+2*2"
Output: 7

Example 2:
Input: *s* = " 3/2 "
Output: 1

Example 3:
Input: *s* = " 3*5 / 2 "
Output: 5

Constraints:

- $1 \leq s.length \leq 3 \times 10^5$
- s* consists of integers and operators `('+', '-', '*', '/')` separated by some number of spaces.
- s* represents a valid expression.
- All the integers in the expression are non-negative integers in the range $[0, 2^{31} - 1]$.
- The answer is guaranteed to fit in a 32-bit integer.

Python

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     struct TreeNode *left;
 *     struct TreeNode *right;
 * };
 */
class BSTIterator {
private: stack<TreeNode*> st;

public:
    BSTIterator(TreeNode* root) {
        this->st = {};
        dfs(root);
    }

    int next() {
        if (st.empty()) return -1;
        auto node = st.top();
        st.pop();
        if (node->right) dfs(node->right);
        return node->val;
    }

    bool hasNext() {
        return !st.empty();
    }
};
```

Python

```
class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        prevnum = 0
        curnum = 0
        op = '+'

        for i, c in enumerate(s):
            if c.isdigit():
                curnum = curnum * 10 + int(c)
            elif c in '+-*/':
                if op == '+' or op == '-':
                    prevnum = curnum
                elif op == '*':
                    prevnum = prevnum * curnum
                elif op == '/':
                    prevnum = math.floor(prevnum / curnum)
                curnum = 0
                op = c
            elif c == ' ':
                continue
            else:
                raise ValueError('Invalid character')

        return ans + prevnum
```

Python

```
class Solution:
    def calculate(self, s: str) -> int:
        v, n = 0, len(s)
        sign = '+'
        stk = []

        for i, c in enumerate(s):
            if c.isdigit():
                v = v * 10 + int(c)
            elif c in '+-*/':
                match sign:
                    case '+':
                        stk.append(v)
                    case '-':
                        stk.append(-v)
                    case '*':
                        stk.append(stk.pop() * v)
                    case '/':
                        stk.append(int(stk.pop() / v))
                sign = c
                v = 0
            elif c == ' ':
                continue
            else:
                raise ValueError('Invalid character')

        return sum(stk)
```

Python


```

class Solution {
    def calculate(s: String) => Int:
        var n = 0, last()
        sign = '+'
        stk = []
        for i, c in enumerate(s):
            if c.isdigit():
                v = s[i+10 : i+1]
                if i == n - 1 or c != "+-*/":
                    if sign == "+":
                        stk.append(v)
                    # For "-//*", when "-" encountered, our 'sign' is still "+",
                    # so -10 will be pushed to stk before setting sign to "-".
                    elif sign == "-":
                        stk.append(-v)
                    elif sign == "*":
                        stk.append(stk.pop() * v)
                    elif sign == "/":
                        stk.append(int(stk.pop() / v))
                    else:
                        print("operator not supported")
                sign = c if c != "+" else "-"
            else:
                return sum(stk)

```

Java

```

class Solution {
    public int calculate(String s) {
        int ans = 0;
        int curNum = 0;
        int prevNum = 0;
        char op = '+';
        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            if (Character.isDigit(c)) {
                curNum = curNum * 10 + c - '0';
            } else if (Character.isDigit(c) && c != '+' && c != '-' && c != '*' && c != '/') {
                if (op == '+' || op == '-') {
                    ans += prevNum;
                    prevNum = op == '+' ? curNum : -curNum;
                } else if (op == '*' || op == '/') {
                    prevNum = curNum;
                } else if (op == '/') {
                    prevNum /= curNum;
                }
                op = c;
                curNum = 0;
            }
        }
        return ans + prevNum;
    }
}

```

Java

```

class Solution {
    public int calculate(String s) {
        Deque<Integer> stk = new ArrayDeque<>();
        char sign = '+';
        int v = 0;
        for (int i = 0; i < s.length(); ++i) {
            char c = s.charAt(i);
            if (Character.isDigit(c)) {
                v = v * 10 + (c - '0');
            }
            if (i == s.length() - 1 || c == '+' || c == '-' || c == '*' || c == '/') {
                if (sign == '+') {
                    stk.push(v);
                } else if (sign == '-') {
                    stk.push(-v);
                } else if (sign == '*') {
                    stk.push(stk.pop() * v);
                } else {
                    stk.push(stk.pop() / v);
                }
                sign = c;
                v = 0;
            }
        }
        int ans = 0;
        while (!stk.isEmpty()) {
            ans += stk.pop();
        }
        return ans;
    }
}

```

Java

```

using System.Collections.Generic;
using System.Linq;

struct Element
{
    public char Op;
    public int Number;
    public Element(char op, int number)
    {
        Op = op;
        Number = number;
    }
}

public class Solution {
    public int Calculate(string s) {
        var stack = new Stack<Element>();
        var readingNumber = false;
        var number = 0;
        var op = '+';
        foreach (var ch in (IEnumerable<char>)s.Concat(Enumerable.Repeat('+', 1)))
        {
            if (ch == '0' && ch == '9')
            {
                if (!readingNumber)
                {
                    readingNumber = true;
                    number = 0;
                }
                number = (number * 10) + (ch - '0');
            }
            else if (ch != '+')
            {
                readingNumber = false;
                if (op == '+' || op == '-')
                {
                    if (stack.Count == 2)
                    {
                        var prev = stack.Pop();
                        var first = stack.Pop();
                        if (prev.Op == '+')
                        {
                            stack.Push(new Element(first.Op, first.Number + prev.Number));
                        }
                        else // '-'
                        {
                            stack.Push(new Element(first.Op, first.Number - prev.Number));
                        }
                    }
                    stack.Push(new Element(op, number));
                }
            }
        }
        return stack.Pop().Number;
    }
}

```

```

        }
        else
        {
            var prev = stack.Pop();
            if (op == '+')
            {
                stack.Push(new Element(prev.Op, prev.Number + number));
            }
            else // '-'
            {
                stack.Push(new Element(prev.Op, prev.Number - number));
            }
        }
        op = ch;
    }
    if (stack.Count == 2)
    {
        var second = stack.Pop();
        var first = stack.Pop();
        if (second.Op == '+')
        {
            stack.Push(new Element(first.Op, first.Number + second.Number));
        }
        else // '-'
        {
            stack.Push(new Element(first.Op, first.Number - second.Number));
        }
    }
}

```

Go

Go

```

func calculate(s string) int {
    sign := '+'
    stk := []int{}
    v := 0
    for i, c := range s {
        digit := '0' < c && c <= '9'
        if digit {
            v = v * 10 + int(c - '0')
        }
        if i == len(s)-1 || !digit && c != '+' {
            switch sign {
                case '+':
                    stk = append(stk, v)
                case '-':
                    stk = append(stk, -v)
                case '*':
                    stk[len(stk)-1] *= v
                case '/':
                    stk[len(stk)-1] /= v
            }
            sign = c
            v = 0
        }
    }
    ans := 0
    for _, v := range stk {
        ans += v
    }
    return ans
}

```

Go

```

func calculate(s string) int {
    sign := '+'
    stk := []int{}
    v := 0
    for i, c := range s {
        digit := '0' < c && c <= '9'
        if digit {
            v = v * 10 + int(c - '0')
        }
        if i == len(s)-1 || !digit && c != '+' {
            switch sign {
                case '+':
                    stk = append(stk, v)
                case '-':
                    stk = append(stk, -v)
                case '*':
                    stk[len(stk)-1] *= v
                case '/':
                    stk[len(stk)-1] /= v
            }
            sign = c
            v = 0
        }
    }
    ans := 0
    for _, v := range stk {
        ans += v
    }
    return ans
}

```

```

for _, v := range stk {
    ans += v
}
return ans
}

```

[7] Stack — Pattern Solution (matched from community answers)

Python

```

# Python solution for Basic Calculator II

def calculate(s: str) -> int:
    # Initialize the stack, current number, and the last seen operator.
    stack = []
    current_number = 0
    previous_operator = "+"

    # Loop over each character in the string along with one extra iteration to handle the end.
    for i, char in enumerate(s):
        if char.isdigit():
            # Build the current number by shifting previous digits.
            current_number = current_number * 10 + int(char)
        # If the character is an operator (or we're at the end), process the previous operator.
        if char in "+-*/" or i == len(s) - 1:
            if previous_operator == "+":
                stack.append(current_number)
            elif previous_operator == "-":
                stack.append(-current_number)
            elif previous_operator == "*":
                last_number = stack.pop()
                stack.append(last_number * current_number)
            elif previous_operator == "/":
                last_number = stack.pop()
                stack.append(int(last_number / current_number))
            # Python division truncates toward negative infinity by default, so we use int() to
            # truncate toward zero.

            # stack.append(int(last_number / current_number))
            # Reset for the next number and update the operator.
            previous_operator = char
            current_number = 0

    # Sum up all numbers in the stack which gives the final result.
    return sum(stack)

# Example usage:
print(calculate("3+2*2")) # Output: 7
print(calculate("3+2*2+3")) # Output: 13
print(calculate("3+5 / 2 * 3")) # Output: 21

```

#255

MEDIUM

Verify Preorder Sequence in Binary Search Tree

LeetCode · Interview · Basic · LeetCode · Editorial

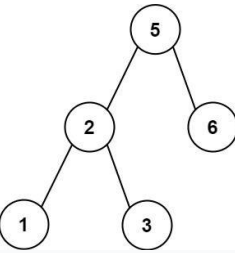
Array · Tree · BST · Stack · Monotonic Stack

Like Premium 98

Problem:

Given an array of unique integers preorder, return true if it is the correct preorder traversal sequence of a binary search tree.

Example 1:



Input: preorder = [5,2,1,3,6]
Output: true

Example 2:

Input: preorder = [5,2,6,1,3]
Output: false

Constraints:

- 1 <= preorder.length <= 104
- 1 <= preorder[i] <= 104

- All the elements of preorder are unique.

Follow up: Could you do it using only constant space complexity?

>> Brute Force [Stack] Interview Prep

Python

Stack · LeetCode — By Pattern

— 2815 —

LeetCode

```
# Brute Force Approach
class Solution:
    def verifyPreorder(self, preorder):
        # Brute Force O(n^2) - simulate without stack via inner loop
        n = len(preorder)
        result = [-1] * n
        for i in range(1, n):
            for j in range(i + 1, n):
                if preorder[j] > preorder[i]:
                    result[i] = preorder[j]
                    break
        return result
```

Python

```
# Python solution using a monotonic stack with detailed comments
def verifyPreorder(preorder):
    lower_bound = float('-inf') # Set initial lower bound to negative infinity
    stack = [] # use a stack to track the BST construction
    for value in preorder:
        # If the current value is less than lower_bound, the BST property is violated
        if value < lower_bound:
            return False
        # If current value is greater than the top of stack, pop elements and update lower_bound
        while stack and value > stack[-1]:
            lower_bound = stack.pop() # update lower_bound as we move to the right subtree
        # Push the current value onto the stack
        stack.append(value)
    return True
```

```
# Example usage:
# print(verifyPreorder([5, 2, 1, 3, 6])) # Expected output: True
```

Python

```
class Solution:
    def verifyPreorder(self, preorder: List[int]) -> bool:
        i = 0

        def dfs(min: int, max: int) -> bool:
            nonlocal i
            if i == len(preorder):
                return True
            if preorder[i] < min or preorder[i] > max:
                return False
            val = preorder[i]
            i += 1
            dfs(min, val)
            dfs(val, max)

        dfs(-math.inf, math.inf)
        return i == len(preorder)
```

Python

Stack · LeetCode — By Pattern

— 2816 —

LeetCode

```
class Solution:
    def verifyPreorder(self, preorder: List[int]) -> bool:
        low = -math.inf
        stack = []

        for p in preorder:
            if p < low:
                return False
            while stack and stack[-1] < p:
                low = stack.pop()
            stack.append(p)

        return True
```

Python

```
class Solution:
    def verifyPreorder(self, preorder: List[int]) -> bool:
        low = -math.inf
        i = 0

        for p in preorder:
            if p < low:
                return False
            while i >= 0 and preorder[i] < p:
                low = preorder[i]
                i -= 1
            i += 1
            preorder[i] = p

        return True
```

Python

```
class Solution:
    def verifyPreorder(self, preorder: List[int]) -> bool:
        stk = []
        last = -inf
        for x in preorder:
            if x < last:
                return False
            while stk and stk[-1] < x:
                last = stk.pop()
            stk.append(x)
            return True
```

Python

Stack · LeetCode — By Pattern

— 2817 —

LeetCode

```
class Solution:
    def verifyPreorder(self, preorder: List[int]) -> bool:
        stk = []
        last = -inf
        for x in preorder:
            if x < last:
                return False
            while stk and stk[-1] < x:
                last = stk.pop()
            stk.append(x)
            return True
```

```
class Solution: # O(N) int modifying input list
    def verifyPreorder(self, preorder: List[int]) -> bool:
        low = -math.inf
        i = -1

        for p in preorder:
            if p < low:
                return False
            while i >= 0 and preorder[i] < p:
                low = preorder[i]
                i -= 1
            i += 1
            preorder[i] = p

        return True
```

```
class Solution(object):
    # O(n) space complexity, O(n) time
    def verifyPreorder(self, preorder):
        (type preorder, List[int])
        (type: bool)
        if len(preorder) == 1:
            return True
        stack, lastlen = [preorder[0]], None
        for i in range(1, len(preorder)):
            if lastlen == preorder[i]:
                return False
            while len(stack) > 0 and preorder[i] > stack[-1]:
                lastlen = stack.pop()
            stack.append(preorder[i])
            return True
```

C++

C++

Stack · LeetCode — By Pattern

— 2818 —

LeetCode

```
class Solution {
public:
    bool verifyPreorder(vector<int>& preorder) {
        int i = 0;
        dfs(preorder, i, INT_MIN, INT_MAX);
        return i == preorder.size();
    }

private:
    void dfs(const vector<int>& preorder, int& i, int min, int max) {
        if (i == preorder.size())
            return;
        if (preorder[i] < min || preorder[i] > max)
            return;

        const int val = preorder[i++];
        dfs(preorder, i, min, val);
        dfs(preorder, i, val, max);
    }
};
```

C++

```
class Solution {
public:
    bool verifyPreorder(vector<int>& preorder) {
        stack<int> stk;
        int last = INT_MIN;
        for (int n : preorder) {
            if (n < last) return false;
            while (!stk.empty() && stk.top() < n) {
                last = stk.top();
                stk.pop();
            }
            stk.push(n);
        }
        return true;
    }
};
```

C++

Stack · LeetCode — By Pattern

— 2819 —

LeetCode

```
// 01: https://leetcode.com/problems/verify-preorder-sequences-in-binary-search-tree/
// Time: O(N)
// Space: O(N)
class Solution {
public:
    bool verifyPreorder(vector<int>& A) {
        stack<int> s;
        int m = INT_MIN;
        for (int a : A) {
            if (a < m) return false;
            while (s.size() && s.top() < a) {
                m = s.top();
                s.pop();
            }
            s.push(a);
        }
        return true;
    }
};
```

Java

Java

```
class Solution {
    public boolean verifyPreorder(int[] preorder) {
        dfs(preorder, Integer.MIN_VALUE, Integer.MAX_VALUE);
        return true;
    }

    private int i = 0;

    private void dfs(int[] preorder, int min, int max) {
        if (i == preorder.length)
            return;
        if (preorder[i] < min || preorder[i] > max)
            return;

        final int val = preorder[i++];
        dfs(preorder, min, val);
        dfs(preorder, val, max);
    }
}
```

Java

Stack · LeetCode — By Pattern

— 2820 —

LeetCode

```

class Solution {
public:
    bool verifyPreorder(int[] preorder) {
        int low = Integer.MIN_VALUE;
        Deque<Integer> stack = new ArrayDeque<>();

        for (final int p : preorder) {
            if (p < low)
                return false;
            while (!stack.isEmpty() && stack.peek() < p)
                low = stack.pop();
            stack.push(p);
        }

        return true;
    }
}

```

Java

```

class Solution {
public:
    bool verifyPreorder(int[] preorder) {
        Deque<Integer> stk = new ArrayDeque<>();
        int last = Integer.MIN_VALUE;
        for (int x : preorder) {
            if (x < last) {
                return false;
            }
            while ((stk.isEmpty() && stk.peek() < x) {
                last = stk.pop();
            }
            stk.push(x);
        }
        return true;
    }
}

```

Java

```

import java.util.Stack;

public class VerifyPreorderSequenceInBinarySearchTree {

    public static void main(String[] args) {
        VerifyPreorderSequenceInBinarySearchTree out = new
        VerifyPreorderSequenceInBinarySearchTree();
        Solution sfs = out.new SolutionDFS();

        System.out.println(sfs.verifyPreorder(new int[]{5,2,6,1,3}));
        System.out.println(sfs.verifyPreorder(new int[]{5,2,1,3,4}));

        Solution sss = out.new Solution();

        System.out.println(sss.verifyPreorder(new int[]{5,2,6,1,3}));
        System.out.println(sss.verifyPreorder(new int[]{5,2,1,3,4}));
    }

    public class Solution {
        public boolean verifyPreorder(int[] preorder) {
            int lowBound = Integer.MIN_VALUE;
            Stack<Integer> st = new Stack<>();
            for (int each : preorder) {
                if (each < lowBound) {
                    return false;
                }
                while (!st.empty() && each > st.peek()) { // st.peek == right value
                    lowBound = st.pop();
                }
                st.push(each);
            }
            return true;
        }
    }

    public class SolutionDFS {
        public boolean verifyPreorder(int[] preorder) {
            return dfs(preorder, 0, preorder.length - 1, Integer.MIN_VALUE, Integer.MAX_VALUE);
        }

        boolean dfs(int[] preorder, int start, int end, int lower, int upper) {
            if (start > end) return true;

            int val = preorder[start];
            if (val <= lower || val >= upper) return false;

            int i = 0;

```

```

        for (i = start + 1; i <= end; i++) {
            if (preorder[i] == val) break; // quote: i stops at root's first right child, same as above
        }
        return dfs(preorder, start + 1, i - 1, lower, val) && dfs(preorder, i, end, val, upper);
    }
}

// =====
class Solution {
public:
    bool verifyPreorder(int[] preorder) {
        Deque<Integer> stk = new ArrayDeque<>();
        int last = Integer.MIN_VALUE;
        for (int x : preorder) {
            if (x < last) {
                return false;
            }
            while ((stk.isEmpty() && stk.peek() < x) {
                last = stk.pop();
            }
            stk.push(x);
        }
        return true;
    }
}

```

Go

```

func verifyPreorder(preorder []int) bool {
    var stk []int
    last := math.MinInt32
    for _, x := range preorder {
        if x < last {
            return false
        }
        for len(stk) > 0 && stk[len(stk)-1] < x {
            last = stk[len(stk)-1]
            stk = stk[:len(stk)-1]
        }
        stk = append(stk, x)
    }
    return true
}

```

Go

```

func verifyPreorder(preorder []int) bool {
    var stk []int
    last := math.MinInt32
    for _, x := range preorder {
        if x < last {
            return false
        }
        for len(stk) > 0 && stk[len(stk)-1] < x {
            last = stk[len(stk)-1]
            stk = stk[:len(stk)-1]
        }
        stk = append(stk, x)
    }
    return true
}

```

Python

```

# Python solution using a monotonic stack with detailed comments
def verifyPreorder(preorder):
    lower_bound = float('-inf') # Set initial lower bound to negative infinity
    stack = [] # Use a stack to track the BST construction
    for value in preorder:
        # If the current value is less than lower_bound, the BST property is violated
        if value < lower_bound:
            return False
        # If current value is greater than the top of stack, pop elements and update lower_bound
        while stack and value < stack[-1]:
            lower_bound = stack.pop() # Update lower_bound as we move to the right subtree
        stack.append(value)
    return True

# Example usage:
# print(verifyPreorder([5,2,1,3,4])) # Expected output: True

```

#394 MEDIUM

Decode String

LeetCode · SimplifyLeetCode · WubCC · LeetCode · Editorial

String · Stack · Recursion

Like · 619 · Dislike · 169

Problem:

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the encoded_string inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like $3a$ or $2[4]$.

The test cases are generated so that the length of the output will never exceed 105.

Example 1:

Input: $s = "3[a]2[b]c"$

Output: "aaabcbcc"

Example 2:

Input: $s = "3[a]c[1]"$

Output: "accaccacc"

Example 3:

Input: $s = "2[abc]3[cd]ef"$

Output: "abccabcccdccdef"

Constraints:

- $1 \leq s.length \leq 30$
- s consists of lowercase English letters, digits, and square brackets $[]$.
- s is guaranteed to be a valid input.
- All the integers in s are in the range $[1, 300]$.

Python

```

def decodeString(s: str) -> str:
    # Index pointer as a list so it can be mutable within recursion
    def helper(i: int) -> (str, int):
        decoded = ""
        while i < len(s):
            if s[i].isdigit():
                # Build the number (it might be more than one digit)
                k = 0
                while i < len(s) and s[i].isdigit():
                    k = k * 10 + int(s[i])
                    i += 1
                # Skip the '[' character
                i += 1
                # Recursively solve the substring inside the brackets
                sub, i = helper(i)
                # Repeat the substring repeated k times
                decoded += sub * k
            elif s[i] == '[':
                # Return the decoded string and new index after '['
                return decoded, i + 1
            else:
                # Accumulate alphabetic characters directly
                decoded += s[i]
                i += 1
        return decoded, i

    result, _ = helper(0)
    return result

# Example usage
print(decodeString("3[a]c[1]")) # Output: "accaccacc"

```

Python

```

class Solution:
    def decodeString(self, s: str) -> str:
        stack = [] # [preStr, repeatCount]
        currStr = ""
        currNum = 0

        for c in s:
            if c.isdigit():
                currNum = currNum * 10 + int(c)
            else:
                if c == '[':
                    stack.append((currStr, currNum))
                    currStr = ""
                    currNum = 0
                elif c == ']':
                    preStr, num = stack.pop()
                    currStr = preStr + num * currStr
                else:
                    currStr += c

        return currStr

```

Python

```

class Solution:
    def decodeString(self, s: str) -> str:
        ans = ""

        while self.i < len(s) and s[self.i] != ']':
            if s[self.i].isdigit():
                k = 0
                while self.i < len(s) and s[self.i].isdigit():
                    k = k * 10 + int(s[self.i])
                    self.i += 1
                self.i += 1
                decodedString = self.decodeString()
                self.i += k + 1
            else:
                ans += s[self.i]
                self.i += 1

        return ans

i = 0

```

Python

```
class Solution:
    def decodeString(self, s: str) -> str:
        st, res = [], ''
        num, res = 0, ''
        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '[':
                st.append(num)
                st.append(res)
                num, res = 0, ''
            elif c == ']':
                res = st.pop() + res * st.pop()
            else:
                res += c
        return res
```

Python

```
class Solution:
    def decodeString(self, s: str) -> str:
        num_stk, str_stk = [], []
        num, res = 0, ''
        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '[':
                num_stk.append(num)
                str_stk.append(res) # initial, res being pushed is empty str ''
                num, res = 0, ''
            elif c == ']':
                res = str_stk.pop() + res * num_stk.pop()
            else:
                res += c
        return res
```

Java

Java

```
class Solution {
    public String decodeString(String s) {
        Stack stack = new Stack<>(); // (prevStr, repeatCount)
        StringBuilder currStr = new StringBuilder();
        int curNum = 0;

        for (final char c : s.toCharArray()) {
            if (Character.isDigit(c)) {
                curNum = curNum * 10 + (c - '0');
            } else {
                if (c == '[') {
                    stack.push(new Pair<>(currStr, curNum));
                    currStr = new StringBuilder();
                    curNum = 0;
                } else if (c == ']') {
                    final Pair<StringBuilder, Integer> pair = stack.pop();
                    final StringBuilder prevStr = pair.getKey();
                    final int n = pair.getValue();
                    currStr = prevStr.append(currStr.toString().repeat(n));
                } else {
                    currStr.append(c);
                }
            }
        }
        return currStr.toString();
    }
}
```

Java

```
class Solution {
    public String decodeString(String s) {
        StringBuilder sb = new StringBuilder();

        while (i < s.length() && s.charAt(i) != ']')
            if (Character.isDigit(s.charAt(i))) {
                int k = 0;
                while (i < s.length() && Character.isDigit(s.charAt(i)))
                    k = k * 10 + (s.charAt(i++) - '0');
                ++i;
                final String decodedString = decodeString(i);
                ++i;
                while (k-- > 0)
                    sb.append(decodedString);
            } else {
                sb.append(s.charAt(i++));
            }
        return sb.toString();
    }
    private int i = 0;
}
```

Java

```
class Solution {
    public String decodeString(String s) {
        Deque<Integer> s1 = new ArrayDeque<>();
        Deque<String> s2 = new ArrayDeque<>();
        int num = 0;
        String res = "";
        for (char c : s.toCharArray()) {
            if (c != '[' && c != ']' && c != '0') {
                num = num * 10 + c - '0';
            } else if (c == '[') {
                s1.push(num);
                num = 0;
                res += "";
            } else if (c == ']') {
                StringBuilder t = new StringBuilder();
                for (int i = 0, n = s1.pop(); i < n; ++i) {
                    t.append(res);
                }
                res = s2.pop() + t.toString();
            } else {
                res += String.valueOf(c);
            }
        }
        return res;
    }
}
```

Java

```
class Solution {
    public String decodeString(String s) {
        Deque<Integer> s1 = new ArrayDeque<>();
        Deque<String> s2 = new ArrayDeque<>();
        int num = 0;
        String res = "";
        for (char c : s.toCharArray()) {
            if (c != '[' && c != ']' && c != '0') {
                num = num * 10 + c - '0';
            } else if (c == '[') {
                s1.push(num);
                s2.push(res);
                num = 0;
                res += "";
            } else if (c == ']') {
                StringBuilder t = new StringBuilder();
                for (int i = 0, n = s1.pop(); i < n; ++i) {
                    t.append(res);
                }
                res = s2.pop() + t.toString();
            } else {
                res += String.valueOf(c);
            }
        }
        return res;
    }
}
```

Java

```
function decodeString(s: string): string {
    let ans = '';
    let stack = [];
    let count = 0; // repeatCount
    for (let cur of s) {
        if (/[\d-]/.test(cur)) {
            count = count * 10 + Number(cur);
        } else if (/[/\d-]/.test(cur)) {
            ans = cur;
        } else if (/[^\d-]/.test(cur)) {
            stack.push([ans, count]);
            // repeat
            ans = '';
            count = 0;
        } else {
            // match
            let [pre, count] = stack.pop();
            ans = pre + ans.repeat(count);
        }
    }
    return ans;
}
```

TypeScript

```
function decodeString(s: string): string {
    let ans = '';
    let stack = [];
    let count = 0; // repeatCount
    for (let cur of s) {
        if (/[\d-]/.test(cur)) {
            count = count * 10 + Number(cur);
        } else if (/[/\d-]/.test(cur)) {
            ans = cur;
        } else if (/[^\d-]/.test(cur)) {
            stack.push([ans, count]);
            // repeat
            ans = '';
            count = 0;
        } else {
            // match
            let [pre, count] = stack.pop();
            ans = pre + ans.repeat(count);
        }
    }
    return ans;
}
```

TypeScript

[*] Stack — Pattern Solution (matched from community answers)

Python

```
class Solution:
    def decodeString(self, s: str) -> str:
        stack = [] # (prevStr, repeatCount)
        curStr = ''
        curNum = 0

        for c in s:
            if c.isdigit():
                curNum = curNum * 10 + int(c)
            else:
                if c == '[':
                    stack.append((curStr, curNum))
                    curStr = ''
                    curNum = 0
                elif c == ']':
                    prevStr, num = stack.pop()
                    curStr = prevStr + curStr * num
                else:
                    curStr += c

        return curStr
```

#439 MEDIUM

Ternary Expression Parser

LeetCode · [Simple](#) · [LeetCode](#) · [WahCCE](#) · [LeetCode](#) · [Editorial](#)

String · Stack · Recursion

List: Premium 98

Problem:

Given a string expression representing arbitrarily nested ternary expressions, evaluate the expression, and return the result of it.

You can always assume that the given expression is valid and only contains digits, 'T', 'F', 'I', and '?' where 'T' is true and 'F' is false. All the numbers in the expression are one-digit numbers (i.e., in the range [0, 9]).

The conditional expressions group right-to-left (as usual in most languages), and the result of the expression will always evaluate to either a digit, 'T' or 'F'.

Example 1:

Input: expression = "T?T?F:5:3"

Output: "2"

Explanation: If true, then result is 2; otherwise result is 3.

Example 2:

Input: expression = "FT?T?T4:5"

Output: "4"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(F ? 1 : (T ? 4 : 5))" ==> "(F ? 1 : 4)" ==> "4"

or "(F ? 1 : (T ? 4 : 5))" ==> "(T ? 4 : 5)" ==> "4"

Example 3:

Input: expression = "T?T?F:5:3"

Output: "2"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(F ? (T ? T : 5) : 3)" ==> "(T ? T : 3)" ==> "3"

"(T ? (T ? T : 5) : 3)" ==> "(T ? T : 5)" ==> "5"

Constraints:

- 5 <= expression.length <= 104
- expression consists of digits, 'T', 'F', 'I', and '?'.
- It is guaranteed that expression is a valid ternary expression and that each number is a one-digit number.

Python

```
def parseTernary(expression: str) -> str:
    # Initialize stack to hold characters/results during evaluation
    stack = []
    # Iterate backwards over the expression
    i = len(expression) - 1
    while i >= 0:
        if expression[i] == 'T':
            # Pop the true and false parts from the stack
            true_expr = stack.pop()
            false_expr = stack.pop()
            i -= 1 # Move to the condition character
            # Check based on the condition
            if expression[i] == 'T':
                stack.append(true_expr)
            else:
                stack.append(false_expr)
        elif expression[i] == '?':
            # Skip the colon as it is only a separator
            pass
        else:
            # One digit, 'F', and 'I' onto stack as is
            stack.append(expression[i])
            i -= 1 # Move to the next character
    return stack[-1]
```

Example usage:
print(parseTernary("T?T?F:5:3")) # Expected output: "2"

Python

```

class Solution:
    def parseTernary(self, expression: str) -> str:
        c = expression[0]
        if self.i + 1 == len(expression) or expression[self.i + 1] == '?':
            self.i += 2
            return str(c)
        self.i += 2
        first = self.parseTernary(expression)
        second = self.parseTernary(expression)
        return first if c == 'T' else second
        i = 0

```

```

Python
class Solution:
    def parseTernary(self, expression: str) -> str:
        stk = []
        cond = False
        for c in expression[1:-1]:
            if c == '?':
                continue
            if c == 'T':
                cond = True
            else:
                if cond:
                    if c == 'T':
                        x = stk.pop()
                        stk.pop()
                        stk.append(x)
                    else:
                        stk.pop()
                        cond = False
                else:
                    stk.append(c)
        return stk[0]

```

Python

```

class Solution:
    def parseTernary(self, expression: str) -> str:
        stk = []
        cond = False
        for c in expression[1:-1]:
            if c == '?':
                continue
            if c == 'T':
                cond = True
            else:
                if cond:
                    if c == 'T':
                        x = stk.pop()
                        stk.pop()
                        stk.append(x)
                    else:
                        stk.pop()
                        cond = False
                else:
                    stk.append(c)
        return stk[0]

```

Java

Java

```

class Solution {
    public String parseTernary(String expression) {
        final char c = expression.charAt(1);
        if (1 < 1 == expression.length() || expression.charAt(1 + 1) == '?') {
            return String.valueOf(c);
        }
        i = 2; // skip '?'
        final String first = parseTernary(expression);
        final String second = parseTernary(expression);
        return c == 'T' ? first : second;
    }

    private int i = 0;
}

```

Java

```

class Solution {
    public String parseTernary(String expression) {
        Deque<Character> stk = new ArrayDeque<>();
        boolean cond = false;
        for (int i = expression.length() - 1; i >= 0; --i) {
            char c = expression.charAt(i);
            if (c == '?') {
                continue;
            }
            if (c == 'T') {
                cond = true;
            } else {
                if (cond) {
                    if (c == 'T') {
                        char x = stk.pop();
                        stk.pop();
                        stk.push(x);
                    } else {
                        stk.pop();
                    }
                } else {
                    cond = false;
                }
            }
            stk.push(c);
        }
        return String.valueOf(stk.peek());
    }
}

```

```

Java
class Solution {
    public String parseTernary(String expression) {
        Deque<Character> stk = new ArrayDeque<>();
        boolean cond = false;
        for (int i = expression.length() - 1; i >= 0; --i) {
            char c = expression.charAt(i);
            if (c == '?') {
                continue;
            }
            if (c == 'T') {
                cond = true;
            } else {
                if (cond) {
                    if (c == 'T') {
                        char x = stk.pop();
                        stk.pop();
                        stk.push(x);
                    } else {
                        stk.pop();
                    }
                } else {
                    cond = false;
                }
            }
            stk.push(c);
        }
        return String.valueOf(stk.peek());
    }
}

```

```

    }
    return String.valueOf(stk.peek());
}

```

[*] Stack — Pattern Solution (matched from community answers)

```

Python
def parseTernary(expression: str) -> str:
    # Initialize stack to hold characters/results during evaluation
    stack = []
    # Iterate backwards over the expression
    i = len(expression) - 1
    while i >= 0:
        if expression[i] == '?':
            # Pop the true and false parts from the stack
            true_expr = stack.pop()
            false_expr = stack.pop()
            # i == 1 & there is the condition character
            # Choose based on the condition
            if expression[i-1] == 'T':
                stack.append(true_expr)
            else:
                stack.append(false_expr)
        elif expression[i] == '?':
            # Only one option as it is only a separator
            pass
        else:
            # Push digits, 'T', and 'F' onto stack as is
            stack.append(expression[i])
            i -= 1 # Move to the next character
    return stack[-1]

```

```

# Example usage:
print(parseTernary("T?T?F")) # Expected output "T"

```

#864

MEDIUM

Find Permutation

LeetCode · Solution · WalkCC · LeetCode · Editorial

Array · String · Stack · Greedy

List: Premium 98

Problem:

A permutation perm of n integers of all the integers in the range [1, n] can be represented as a string s of length n - 1 where:

• s[i] = 'I' if perm[i] < perm[i + 1], and

• s[i] = 'D' if perm[i] > perm[i + 1].

Given a string s, reconstruct the lexicographically smallest permutation perm and return it.

Example 1:

Input: s = "I"

Output: [1,2]

Explanation: [1,2] is the only legal permutation that can be represented by s, where the number 1 and 2 construct an increasing relationship.

Example 2:

Input: s = "DI"

Output: [2,1,3]

Explanation: Both [2,1,3] and [3,1,2] can be represented as "DI", but since we want to find the smallest lexicographical permutation, you should return [2,1,3]

Constraints:

• 1 <= s.length <= 105

• s[i] is either 'I' or 'D'.

>> Brute Force [Stack] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def findPermutation(self, s):
        n = len(s)
        result = [-1] * n
        for i in range(n):
            for j in range(i + 1, n):
                if s[i] > s[j]:
                    result[i], result[j] = result[j], result[i]
                    break
        return result

```

Python

Python

```

# Function to find the lexicographically smallest permutation
def findPermutation(s):
    stack = []
    # Use a stack to hold numbers during a sequence of 'D's
    result = [] # Result list for the final permutation
    # Iterate over the input (includes the process all characters plus one final number.
    for i in range(len(s) + 1):
        # Push the next number (range from 1 to n)
        stack.append(i + 1)
        # If we are at the end of the current character is 'I', flush the stack.
        if i == len(s) or s[i] == 'I':
            while stack:
                result.append(stack.pop())
    return result

```

```

# Example usage:
print(findPermutation("DI"))

```

Python

```

class Solution:
    def findPermutation(self, s: str) -> List[int]:
        ans = []
        stack = []

        for i, c in enumerate(s):
            stack.append(i + 1)
            if c == 'D':
                while stack: # (includes all decreasings)
                    ans.append(stack.pop())
            stack.append(len(s) + 1)

            while stack:
                ans.append(stack.pop())

        return ans

```

Python

```

class Solution:
    def findPermutation(self, s: str) -> List[int]:
        ans = []
        for i in range(1, len(s) + 2):
            # For each De group (e.g., "D"), reverse ans[i..j + 1].
            i = 0
            j = -1

            def getMinIndex(i: str, start: int) -> int:
                for i in range(start, len(s)):
                    if s[i] == 'D':
                        return i
                return len(s)

            while True:
                i = getMinIndex(s, j + 1)
                if i == len(s):
                    break
                j = getMinIndex(s, i + 1)
                ans[i : j + 1] = ans[i : j + 1][::-1]
            return ans

```

Python

```

class Solution:
    def findPermutation(self, s: str) -> List[int]:
        n = len(s)
        ans = list(range(1, n + 2))
        i = 0
        while i < n:
            j = i
            while j < n and s[j] == 'D':
                j += 1
            ans[i : j + 1] = ans[i : j + 1][::-1]
            i = next(i + 1, j)
        return ans

```

Python

```
class Solution:
    def findPermutation(self, s: str) -> list[int]:
        n = len(s)
        ans = list(range(1, n + 2))
        i = 0
        while i < n:
            j = i
            while j < n and s[j] == 'D':
                j += 1
            ans[i : j + 1] = ans[i : j + 1][::-1]
            i = ans[i] + 1, j)
        return ans

C++
class Solution {
public:
    vector<int> findPermutation(string s) {
        vector<int> ans;
        stack<int> stack;
        for (int i = 0; i < s.length(); ++i) {
            stack.push(i + 1);
            if (s[i] == 'I')
                while (!stack.empty())
                    ans.push_back(stack.top()), stack.pop();
        }
        stack.push(s.length() + 1);
        while (!stack.empty())
            ans.push_back(stack.top()), stack.pop();
        return ans;
    }
};

C++
class Solution {
public:
    vector<int> findPermutation(string s) {
        int n = s.size();
        vector<int> ans(n + 1);
        int(n, ans(n), ans.end(), 1);
        int i = 0;
        while (i < n) {
            int j = i;
            while (j < n && s[j] == 'D') {
                ++j;
            }
            reverse(ans.begin() + i, ans.begin() + j + 1);
            i = ans[i + 1, j];
        }
        return ans;
    }
};

C++
```

Stack · LeetCode — By Pattern — 2839 — LeetCode

```
class Solution {
public:
    vector<int> findPermutation(string s) {
        int n = s.size();
        vector<int> ans(n + 1);
        int(n, ans(n), ans.end(), 1);
        int i = 0;
        while (i < n) {
            int j = i;
            while (j < n && s[j] == 'D') {
                ++j;
            }
            reverse(ans.begin() + i, ans.begin() + j + 1);
            i = ans[i + 1, j];
        }
        return ans;
    }
};

Java
class Solution {
    public int[] findPermutation(String s) {
        int[] ans = new int[s.length() + 1];
        int ansIndex = 0;
        Deque<Integer> stack = new ArrayDeque<>();
        for (int i = 0; i < s.length(); ++i) {
            stack.push(i + 1);
            if (s.charAt(i) == 'I')
                while (!stack.isEmpty())
                    ans[ansIndex++] = stack.pop();
        }
        stack.push(s.length() + 1);
        while (!stack.isEmpty())
            ans[ansIndex++] = stack.pop();
        return ans;
    }
}

Java
```

Stack · LeetCode — By Pattern — 2840 — LeetCode

```
class Solution {
    public int[] findPermutation(String s) {
        int n = s.length();
        int[] ans = new int[n + 1];
        for (int i = 0; i < n + 1; ++i) {
            ans[i] = i + 1;
        }
        int i = 0;
        while (i < n) {
            int j = i;
            while (j < n && s.charAt(j) == 'D') {
                ++j;
            }
            reverse(ans, i, j);
            i = Math.max(i + 1, j);
        }
        return ans;
    }

    private void reverse(int[] arr, int i, int j) {
        for (; i < j; ++i, --j) {
            int t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }
    }
}

Java
class Solution {
    public int[] findPermutation(String s) {
        int n = s.length();
        int[] ans = new int[n + 1];
        for (int i = 0; i < n + 1; ++i) {
            ans[i] = i + 1;
        }
        int i = 0;
        while (i < n) {
            int j = i;
            while (j < n && s.charAt(j) == 'D') {
                ++j;
            }
            reverse(ans, i, j);
            i = Math.max(i + 1, j);
        }
        return ans;
    }

    private void reverse(int[] arr, int i, int j) {
        for (; i < j; ++i, --j) {
            int t = arr[i];
            arr[i] = arr[j];
            arr[j] = t;
        }
    }
}

Java
```

Stack · LeetCode — By Pattern — 2841 — LeetCode

```
Go
func findPermutation(s string) []int {
    n := len(s)
    ans := make([]int, n+1)
    for i := range ans {
        ans[i] = i + 1
    }
    i := 0
    for i < n {
        j := i
        for j < n && s[j] == 'D' {
            j++
        }
        reverse(ans, i, j)
        i = ans[i+1, j]
    }
    return ans
}

func reverse(arr []int, i, j int) {
    for i < j {
        i, j = j, i
        arr[i], arr[j] = arr[j], arr[i]
    }
}

Go
func findPermutation(s string) []int {
    n := len(s)
    ans := make([]int, n+1)
    for i := range ans {
        ans[i] = i + 1
    }
    i := 0
    for i < n {
        j := i
        for j < n && s[j] == 'D' {
            j++
        }
        reverse(ans, i, j)
        i = ans[i+1, j]
    }
    return ans
}

func reverse(arr []int, i, j int) {
    for i < j {
        i, j = j, i
        arr[i], arr[j] = arr[j], arr[i]
    }
}

Python
# Brute Force [Stack] Interview Prep
```

Stack · LeetCode — By Pattern — 2842 — LeetCode

```
# Function to find the lexicographically smallest permutation
def findPermutation(s):
    stack = []
    # Use a stack to hold numbers during a sequence of 'D's
    result = []
    # Result list for the final permutation
    # Iterate through the input string to process all characters plus one final number.
    for i in range(len(s) + 1):
        # Push the next number from 1 to n
        stack.append(i + 1)
        # If we are at the end or the current character is 'I', flush the stack.
        if i == len(s) or s[i] == 'I':
            while stack:
                result.append(stack.pop())
    return result

# Example usage
print(findPermutation("DI"))
```

#735 MEDIUM Asteroid Collision

LeetCode · Simplest · MathCC · LeetCode · Editorial

Array · Stack · Simulation

List · Grid 189

Problem:

We are given an array asteroids of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

Input: asteroids = [5,10,-5]

Output: [5,10]

Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.

Example 2:

Input: asteroids = [8,-8]

Output: []

Explanation: The 8 and -8 collide exploding each other.

Example 3:

Input: asteroids = [10,2,-5]

Output: [10]

Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.

Example 4:

Input: asteroids = [3,5,-6,2,-1,4]

Output: [-6,2,4]

Stack · LeetCode — By Pattern — 2843 — LeetCode

Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.

Constraints:

• 2 <= asteroids.length <= 104

• -1000 <= asteroids[i] <= 1000

• asteroids[0] != 0

>> Brute Force [Stack] Interview Prep

Python

```
class Solution:
    def asteroidCollision(self, asteroids):
        # Empty array for 'I' - asteroids without stack via inner loop
        n = len(asteroids)
        result = []
        for i in range(n):
            for j in range(i+1, n):
                if asteroids[i] > asteroids[j]:
                    result[i] = asteroids[i]
                    break
            return result
```

Python

```
def asteroidCollision(asteroids):
    # Initialize an empty list as a stack to hold surviving asteroids
    stack = []
    # Iterate through each asteroid in the input list
    for asteroid in asteroids:
        # Find the index of the current asteroid has exploded
        exploded = False
        # Check for potential collision while there is an asteroid in stack and a collision condition holds (right-moving on stack and left-moving current)
        while stack and asteroid < 0 and stack[-1] > 0:
            # If the incoming left-moving asteroid is larger than the right-moving one on stack, pop the top and continue checking
            if abs(asteroid) > abs(stack[-1]):
                stack.pop()
            # Continue if continue the loop to check next potential collision
            # If they are equal, both explode; pop the stack and mark explosion, break the loop
            elif abs(asteroid) == abs(stack[-1]):
                stack.pop()
            # Mark the incoming asteroid as exploded and break out of the loop
            exploded = True
            break
        # If the asteroid did not explode during collisions, push it onto the stack
        if not exploded:
            stack.append(asteroid)
    # Return the surviving asteroids
    return stack
```

Stack · LeetCode — By Pattern — 2844 — LeetCode

```
# translate vangs:
# print asteroidCollision([5,10,-5]) # Output: [5,10]
```

```
Python
class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stack = []

        for a in asteroids:
            if a > 0:
                stack.append(a)
            else: # a < 0
                # Destroy the previous positive one(s).
                while stack and stack[-1] > 0 and stack[-1] < -a:
                    stack.pop()
                if not stack or stack[-1] < 0:
                    stack.append(a)
                elif stack[-1] == -a:
                    stack.pop() # Both asteroids explode.
                else: # stack[-1] > -a: the current asteroid
                    pass # Destroy the current asteroid, so do nothing.

        return stack
```

```
Python
class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stk = []
        for s in asteroids:
            if s > 0:
                stk.append(s)
            else:
                while stk and stk[-1] > 0 and stk[-1] < -s:
                    stk.pop()
                if stk and stk[-1] == -s:
                    stk.pop()
                elif not stk or stk[-1] < 0:
                    stk.append(s)
                return stk
```

```
Python
class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stk = []
        for s in asteroids:
            if s > 0:
                stk.append(s)
            else:
                while stk and stk[-1] > 0 and stk[-1] < -s:
                    stk.pop()
                if stk and stk[-1] == -s:
                    stk.pop()
                elif not stk or stk[-1] < 0:
                    stk.append(s)
                return stk
```

C++

```
class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        vector<int> stack;

        for (const int a : asteroids)
            if (a > 0) {
                stack.push_back(a);
            } else { // a < 0
                // Destroy the previous positive one(s).
                while (!stack.empty() && stack.back() > 0 && stack.back() < -a)
                    stack.pop_back();
                if (stack.empty() || stack.back() < 0)
                    stack.push_back(a);
                else if (stack.back() == -a)
                    stack.pop_back(); // Both asteroids explode.
                else // stack[-1] > -a: the current asteroid
                    ; // Destroy the current asteroid, so do nothing.
            }

        return stack;
    }
};
```

C++

```
class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        vector<int> stk;
        for (int a : asteroids) {
            if (a > 0) {
                stk.push_back(a);
            } else {
                while (stk.size() && stk.back() > 0 && stk.back() < -a) {
                    stk.pop_back();
                }
                if (stk.size() && stk.back() == -a) {
                    stk.pop_back();
                } else if (stk.empty() || stk.back() < 0) {
                    stk.push_back(a);
                }
            }
        }
        return stk;
    }
};
```

C++

```
class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        vector<int> stk;
        for (int a : asteroids) {
            if (a > 0) {
                stk.push_back(a);
            } else {
                while (stk.size() && stk.back() > 0 && stk.back() < -a) {
                    stk.pop_back();
                }
                if (stk.size() && stk.back() == -a) {
                    stk.pop_back();
                } else if (stk.empty() || stk.back() < 0) {
                    stk.push_back(a);
                }
            }
        }
        return stk;
    }
};
```

Java

```
Java
class Solution {
    public List<Integer> asteroidCollision(List<Integer> asteroids) {
        Stack<Integer> stack = new Stack<>();

        for (final int a : asteroids)
            if (a > 0) {
                stack.push(a);
            } else { // a < 0
                // Destroy the previous positive one(s).
                while (!stack.isEmpty() && stack.peek() > 0 && stack.peek() < -a)
                    stack.pop();
                if (stack.isEmpty() || stack.peek() < 0)
                    stack.push(a);
                else if (stack.peek() == -a)
                    stack.pop(); // Both asteroids explode.
                else // stack[-1] > -a: the current asteroid
                    ; // Destroy the current asteroid, so do nothing.
            }

        return stack.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

Java

```
class Solution {
    public List<Integer> asteroidCollision(List<Integer> asteroids) {
        Deque<Integer> stk = new ArrayDeque<>();
        for (int a : asteroids) {
            if (a > 0) {
                stk.offerLast(a);
            } else {
                while (!stk.isEmpty() && stk.peekLast() > 0 && stk.peekLast() < -a) {
                    stk.pollLast();
                }
                if (!stk.isEmpty() && stk.peekLast() == -a) {
                    stk.pollLast();
                } else if (stk.isEmpty() || stk.peekLast() < 0) {
                    stk.offerLast(a);
                }
            }
        }
        return stk.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

Java

```
class Solution {
    public List<Integer> asteroidCollision(List<Integer> asteroids) {
        Deque<Integer> stk = new ArrayDeque<>();
        for (int a : asteroids) {
            if (a > 0) {
                stk.offerLast(a);
            } else {
                while (!stk.isEmpty() && stk.peekLast() > 0 && stk.peekLast() < -a) {
                    stk.pollLast();
                }
                if (!stk.isEmpty() && stk.peekLast() == -a) {
                    stk.pollLast();
                } else if (stk.isEmpty() || stk.peekLast() < 0) {
                    stk.offerLast(a);
                }
            }
        }
        return stk.stream().mapToInt(Integer::intValue).toArray();
    }
}
```

TypeScript

TypeScript

```
function asteroidCollision(asteroids: number[]): number[] {
    const stk: number[] = [];
    for (const a of asteroids) {
        if (a > 0) {
            stk.push(a);
        } else {
            while (stk.length && stk.at(-1) > 0 && stk.at(-1) < -a) {
                stk.pop();
            }
            if (stk.length && stk.at(-1) === -a) {
                stk.pop();
            } else if (stk.length || stk.at(-1) < 0) {
                stk.push(a);
            }
        }
    }
    return stk;
}
```

TypeScript

```
function asteroidCollision(asteroids: number[]): number[] {
    const stk: number[] = [];
    for (const a of asteroids) {
        if (a > 0) {
            stk.push(a);
        } else {
            while (stk.length && stk.at(-1) > 0 && stk.at(-1) < -a) {
                stk.pop();
            }
            if (stk.length && stk.at(-1) === -a) {
                stk.pop();
            } else if (stk.length || stk.at(-1) < 0) {
                stk.push(a);
            }
        }
    }
    return stk;
}
```

Go

Go

```
func asteroidCollision(asteroids []int) []int {
    for _, a := range asteroids {
        if a > 0 {
            stk = append(stk, a)
        } else {
            for len(stk) > 0 && stk[len(stk)-1] > 0 && stk[len(stk)-1] < -a {
                stk = stk[:len(stk)-1]
            }
            if len(stk) > 0 && stk[len(stk)-1] == -a {
                stk = stk[:len(stk)-1]
            } else if len(stk) == 0 || stk[len(stk)-1] < 0 {
                stk = append(stk, a)
            }
        }
    }
    return stk
}
```

Go

```
func asteroidCollision(asteroids []int) []int {
    for _, a := range asteroids {
        if a > 0 {
            stk = append(stk, a)
        } else {
            for len(stk) > 0 && stk[len(stk)-1] > 0 && stk[len(stk)-1] < -a {
                stk = stk[:len(stk)-1]
            }
            if len(stk) > 0 && stk[len(stk)-1] == -a {
                stk = stk[:len(stk)-1]
            } else if len(stk) == 0 || stk[len(stk)-1] < 0 {
                stk = append(stk, a)
            }
        }
    }
    return stk
}
```

Rust

Rust

```

impl Solution {
    # @return: bool
    pub fn asteroid_collision(asteroids: Vec<i32>) -> Vec<i32> {
        let mut stk = Vec::new();
        for & asteroid in asteroids {
            if x < 0 {
                stk.push(x);
            } else {
                while !stk.is_empty() && stk.last().unwrap() > 0 && stk.last().unwrap() < -x {
                    stk.pop();
                }
                if !stk.is_empty() && stk.last().unwrap() == -x {
                    stk.pop();
                } else if stk.is_empty() || stk.last().unwrap() < 0 {
                    stk.push(x);
                }
            }
        }
        stk
    }
}

```

Rust

```

impl Solution {
    # @return: bool
    pub fn asteroid_collision(asteroids: Vec<i32>) -> Vec<i32> {
        let mut stk = Vec::new();
        for & asteroid in asteroids {
            if x < 0 {
                stk.push(x);
            } else {
                while !stk.is_empty() && stk.last().unwrap() > 0 && stk.last().unwrap() < -x {
                    stk.pop();
                }
                if !stk.is_empty() && stk.last().unwrap() == -x {
                    stk.pop();
                } else if stk.is_empty() || stk.last().unwrap() < 0 {
                    stk.push(x);
                }
            }
        }
        stk
    }
}

```

[*] Stack — Pattern Solution (matched from community answers)

Python

Stack · LeetCode — By Pattern

— 2851 —

LeetCode

```

# Function to simulate asteroid collisions
def asteroidCollision(asteroids):
    # Initialize an empty list as a stack to hold surviving asteroids
    stack = []
    # Iterate through each asteroid in the input list
    for ast in asteroids:
        # Flag to check if the current asteroid has exploded
        exploded = False
        # Check for potential collision while there is an asteroid in stack and a collision condition holds
        # (right-moving on stack and left-moving current)
        while stack and ast < 0 and stack[-1] > 0:
            # If the incoming left-moving asteroid is larger than the right-moving one on stack, pop the
            # top and continue checking
            if abs(ast) > abs(stack[-1]):
                stack.pop()
            # Continue the loop to check next potential collision
            # If they are equal, both explode: pop the stack and mark explosion, break the loop
            elif abs(ast) == abs(stack[-1]):
                stack.pop()
            # Mark the incoming asteroid as exploded and break out of the loop
            exploded = True
            break
        # If the asteroid did not explode during collisions, push it onto the stack
        if not exploded:
            stack.append(ast)
    # Return the surviving asteroids
    return stack

```

Example usage:
print(asteroidCollision([5,10,-5])) # Output: [5,10]

#739

MEDIUM

Daily Temperatures

LeetCode · Synopsis · Discuss · Solution · LeetCode · Editorial

Array · Stack · Monotonic Stack

List: Grid 169

Problem:

Given an array of integers temperatures represents the daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead.

Example 1:

Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]

Example 2:

Input: temperatures = [30,40,50,60]
Output: [1,1,1,0]

Example 3:

Stack · LeetCode — By Pattern

— 2852 —

LeetCode

Input: temperatures = [30,40,90]
Output: [1,1,0]

Constraints:

• 1 <= temperatures.length <= 105
• 30 <= temperatures[i] <= 100

>> Brute Force [Stack] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def dailyTemperatures(self, temperatures):
        # Brute Force O(n^2) - For each day scan forward for warmer day
        n = len(temperatures)
        res = [0] * n
        for i in range(n):
            for j in range(i+1, n):
                if temperatures[j] > temperatures[i]:
                    res[i] = j - i
                    break
        return res

```

Python

Python

```

def dailyTemperatures(temperatures):
    n = len(temperatures)
    # Initialize answer list with zeros
    answer = [0] * n
    # Stack to keep indices of temperatures with unresolved next warmer days
    stack = []
    # Iterate through the temperature list
    for i, temp in enumerate(temperatures):
        # Check if the current temperature resolves any previous indices
        while stack and temperatures[stack[-1]] < temp:
            prev_index = stack.pop() # Get the index of the previous colder day
            answer[prev_index] = i - prev_index # Update the number of days waited
            # Push the current index onto the stack
            stack.append(i)
    # Return answer
    return answer

```

Example usage:
if __name__ == "__main__":
 temps = [73,74,75,71,69,72,76,73]
 print(dailyTemperatures(temps))

Python

Stack · LeetCode — By Pattern

— 2853 —

LeetCode

```

class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        ans = [0] * len(temperatures)
        stack = [] # a decreasing stack
        for i, temperature in enumerate(temperatures):
            while stack and temperature > temperatures[stack[-1]]:
                index = stack.pop()
                ans[index] = i - index
            stack.append(i)
        return ans

```

Python

```

class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        stk = []
        n = len(temperatures)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and temperatures[stk[-1]] <= temperatures[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1] - i
            stk.append(i)
        return ans

```

Python

```

class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        ans = [0] * len(temperatures)
        stk = []
        for i, t in enumerate(temperatures):
            while stk and temperatures[stk[-1]] < t:
                j = stk.pop()
                ans[j] = i - j
            stk.append(i)
        return ans

```

C++

C++

Stack · LeetCode — By Pattern

— 2854 —

LeetCode

```

class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        vector<int> ans(temperatures.size());
        stack<int> stack; // a decreasing stack
        for (int i = 0; i < temperatures.size(); ++i) {
            while (!stack.empty() && temperatures[stack.top()] < temperatures[i]) {
                const int index = stack.top();
                stack.pop();
                ans[index] = i - index;
            }
            stack.push(i);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        int n = temperatures.size();
        stack<int> stk;
        vector<int> ans(n);
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.empty() && temperatures[stk.top()] <= temperatures[i]) {
                stk.pop();
            }
            if (!stk.empty()) {
                ans[i] = stk.top() - i;
            }
            stk.push(i);
        }
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        int n = temperatures.size();
        stack<int> stk;
        vector<int> ans(n);
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.empty() && temperatures[stk.top()] <= temperatures[i]) {
                ans[stk.top()] = i - stk.top();
                stk.pop();
            }
            stk.push(i);
        }
        return ans;
    }
};

```

Stack · LeetCode — By Pattern

— 2855 —

LeetCode

```

// Java
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int[] ans = new int[temperatures.length];
        Deque<Integer> stack = new ArrayDeque<>(); // a decreasing stack
        for (int i = 0; i < temperatures.length; ++i) {
            while (!stack.isEmpty() && temperatures[stack.peek()] < temperatures[i]) {
                final int index = stack.pop();
                ans[index] = i - index;
            }
            stack.push(i);
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] ans = new int[n];
        Deque<Integer> stk = new ArrayDeque<>();
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.isEmpty() && temperatures[stk.peek()] <= temperatures[i]) {
                stk.pop();
            }
            if (!stk.isEmpty()) {
                ans[i] = stk.peek() - i;
            }
            stk.push(i);
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] ans = new int[n];
        Deque<Integer> stk = new ArrayDeque<>();
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.isEmpty() && temperatures[stk.peek()] <= temperatures[i]) {
                int j = stk.pop();
                ans[j] = i - j;
            }
            stk.push(i);
        }
        return ans;
    }
}

```

JavaScript

Stack · LeetCode — By Pattern

— 2856 —

LeetCode


```
JavaScript
/*
 * @param {number[]} temperatures
 * @return {number[]}
 */
var dailyTemperatures = function (temperatures) {
    const n = temperatures.length;
    const ans = new Array(n).fill(0);
    const stk = [];
    for (let i = 0; i < n; i++) {
        while (stk.length && temperatures[stk.at(-1)] <= temperatures[i]) {
            stk.pop();
        }
        if (stk.length) {
            ans[i] = stk.at(-1) - i;
        }
        stk.push(i);
    }
    return ans;
};

TypeScript
/*
 * @param {number[]} temperatures
 * @return {number[]}
 */
var dailyTemperatures = function (temperatures) {
    const n = temperatures.length;
    const ans = new Array(n).fill(0);
    const stk = [];
    for (let i = 0; i < n; i++) {
        while (stk.length && temperatures[stk.at(-1)] <= temperatures[i]) {
            stk.pop();
        }
        if (stk.length) {
            ans[i] = stk.at(-1) - i;
        }
        stk.push(i);
    }
    return ans;
};

TypeScript
TypeScript
```

```
function dailyTemperatures(temperatures: number[]): number[] {
    const n = temperatures.length;
    const ans = new Array(n).fill(0);
    const stk: number[] = [];
    for (let i = 0; i < n; i++) {
        while (stk.length && temperatures[stk.at(-1)] <= temperatures[i]) {
            stk.pop();
        }
        if (stk.length) {
            ans[i] = stk.at(-1) - i;
        }
        stk.push(i);
    }
    return ans;
}

TypeScript
function dailyTemperatures(temperatures: number[]): number[] {
    const n = temperatures.length;
    const ans = new Array(n).fill(0);
    const stk: number[] = [];
    for (let i = 0; i < n; i++) {
        while (stk.length && temperatures[stk.at(-1)] <= temperatures[i]) {
            stk.pop();
        }
        if (stk.length) {
            ans[i] = stk.at(-1) - i;
        }
        stk.push(i);
    }
    return ans;
}

Go
Go
func dailyTemperatures(temperatures []int) []int {
    n := len(temperatures)
    ans := make([]int, n)
    stk := []int{}
    for i, v := range temperatures {
        for len(stk) > 0 && temperatures[stk[len(stk)-1]] <= v {
            prev_index := stk[len(stk)-1]
            ans[prev_index] = i - prev_index
            stk = stk[:len(stk)-1]
        }
        if len(stk) > 0 {
            ans[i] = stk[len(stk)-1] - i
        }
        stk = append(stk, i)
    }
    return ans
}

Go
```

```
func dailyTemperatures(temperatures []int) []int {
    ans := make([]int, len(temperatures))
    var stk []int
    for i, t := range temperatures {
        for len(stk) > 0 && temperatures[stk[len(stk)-1]] <= t {
            j := stk[len(stk)-1]
            ans[j] = i - j
            stk = stk[:len(stk)-1]
        }
        stk = append(stk, i)
    }
    return ans
}

Rust
Rust
impl Solution {
    pub fn daily_temperatures(temperatures: Vec<i32>) -> Vec<i32> {
        let n = temperatures.len();
        let mut stk: Vec<i32> = Vec::new();
        let mut ans = vec![0; n];

        for i in (0..n).rev() {
            while let Some(top) = stk.last() {
                if temperatures[top] <= temperatures[i] {
                    stk.pop();
                } else {
                    break;
                }
            }
            if let Some(top) = stk.last() {
                ans[i] = (top - i) as i32;
            }
            stk.push(i);
        }
        ans
    }
}

Rust
impl Solution {
    pub fn daily_temperatures(temperatures: Vec<i32>) -> Vec<i32> {
        let n = temperatures.len();
        let mut stack = vec![];
        let mut res = vec![0; n];
        for i in 0..n {
            while stack.is_empty() && temperatures[stack.last().unwrap()] < temperatures[i] {
                let j = stack.pop().unwrap();
                res[j] = (i - j) as i32;
            }
            stack.push(i);
        }
        res
    }
}

Rust
```

```
[""] Stack — Pattern Solution (matched from community answers)
Python
def dailyTemperatures(temperatures):
    n = len(temperatures)
    # Initialize answer list with zeros
    answer = [0] * n
    # Stack to keep indices of temperatures with unresolved next warmer days
    stack = []
    # Iterate through the temperature list
    for i, temp in enumerate(temperatures):
        # Check if the current temperature resolves any previous indices
        while stack and temperatures[stack[-1]] < temp:
            prev_index = stack.pop()
            answer[prev_index] = i - prev_index
            # Update the number of days waited
            # Push the current index onto the stack
            stack.append(i)
    return answer

# Example usage:
if __name__ == "__main__":
    temps = [73, 74, 75, 76, 77, 78, 79, 80, 81]
    print(dailyTemperatures(temps))
```

#962 MEDIUM
Maximum Width Ramp
LeetCode · Companies · LeetCode · Editorial
Array · Stack · Monotonic Stack
Like Denny Zhang
Problem:
A ramp in an integer array nums is a pair (i, j) for which i < j and nums[i] <= nums[j]. The width of such a ramp is j - i.
Given an integer array nums, return the maximum width of a ramp in nums. If there is no ramp in nums, return 0.
Example 1:
Input: nums = [6,0,8,2,1,5]
Output: 4
Explanation: The maximum width ramp is achieved at (i, j) = (1, 5): nums[1] = 0 and nums[5] = 5.
Example 2:
Input: nums = [9,8,1,0,1,9,4,0,4,1]
Output: 7
Explanation: The maximum width ramp is achieved at (i, j) = (2, 9): nums[2] = 1 and nums[9] = 1.
Constraints:
• 2 <= nums.length <= 5 * 10⁴
• 0 <= nums[i] <= 5 * 10⁴

```
>> Brute Force [Stack] Interview Prep
Python
# Brute Force Approach
class Solution:
    def maxWidthRamp(self, nums):
        # Brute Force O(n^2) - check every pair (i, j) where i < j and nums[i] <= nums[j]
        res = 0
        for i in range(len(nums)):
            for j in range(i+1, len(nums)):
                if nums[i] <= nums[j]:
                    res = max(res, j - i)
        return res

Python
# Python solution for Maximum Width Ramp
def maxWidthRamp(nums):
    n = len(nums)
    stack = []
    # Build a stack of indices where array values are in decreasing order
    for i in range(n):
        if not stack or nums[i] < nums[stack[-1]]:
            stack.append(i)
    max_width = 0
    # Iterate through indices to find the maximum width ramp
    for j in range(n-1, -1, -1):
        while stack and nums[j] >= nums[stack[-1]]:
            max_width = max(max_width, j - stack.pop())
    return max_width

# Example usage:
nums = [6, 0, 8, 2, 1, 5]
print(maxWidthRamp(nums)) # Output: 4

Python
class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        ans = 0
        stack = []
        for i, num in enumerate(nums):
            if stack == [] or num <= nums[stack[-1]]:
                stack.append(i)
        for i, num in reversed(list(enumerate(nums))):
            while stack and num >= nums[stack[-1]]:
                ans = max(ans, i - stack.pop())
        return ans

Python
```

```
class Solution {
    def maxWidthRamp(self, nums: List[int]) -> int:
        stk = []
        for i, v in enumerate(nums):
            if not stk or nums[stk[-1]] > v:
                stk.append(i)
        ans = 0
        for i in range(len(nums) - 1, -1, -1):
            while stk and nums[stk[-1]] <= nums[i]:
                ans = max(ans, i - stk.pop())
            if not stk:
                break
        return ans
}

Python
class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        stk = []
        for i, v in enumerate(nums):
            if not stk or nums[stk[-1]] > v:
                stk.append(i)
        ans = 0
        for i in range(len(nums) - 1, -1, -1):
            while stk and nums[stk[-1]] <= nums[i]:
                ans = max(ans, i - stk.pop())
            if not stk:
                break
        return ans

C++
C++
class Solution {
public:
    int maxWidthRamp(vector<int>& nums) {
        int ans = 0;
        stack<int> stack;
        for (int i = 0; i < nums.size(); ++i)
            if (stack.empty() || nums[i] <= nums[stack.top()])
                stack.push(i);
        for (int i = nums.size() - 1; i >= 0; --i)
            while (!stack.empty() && nums[i] >= nums[stack.top()])
                ans = max(ans, i - stack.top()), stack.pop();
        return ans;
    }
};

C++
```

```
class Solution {
public:
    int maxWithRange(vector<int>& nums) {
        int n = nums.size();
        stack<int> stk;
        for (int i = 0; i < n; ++i) {
            if (stk.empty() || nums[i] > nums[i]) stk.push(i);
        }
        int ans = 0;
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.empty() && nums[i] <= nums[stk.top()]) {
                ans = max(ans, i - stk.top());
                stk.pop();
            }
            if (stk.empty()) break;
        }
        return ans;
    }
};

C++
class Solution {
public:
    int maxWithRange(vector<int>& nums) {
        int n = nums.size();
        stack<int> stk;
        for (int i = 0; i < n; ++i) {
            if (stk.empty() || nums[i] > nums[stk.top()]) stk.push(i);
        }
        int ans = 0;
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.empty() && nums[i] <= nums[stk.top()]) {
                ans = max(ans, i - stk.top());
                stk.pop();
            }
            if (stk.empty()) break;
        }
        return ans;
    }
};

Java
Java
```

```
class Solution {
    public int maxWithRange(int[] nums) {
        int ans = 0;
        Deque<Integer> stack = new ArrayDeque<>();
        for (int i = 0; i < nums.length; ++i) {
            if (stack.isEmpty() || nums[i] > nums[stack.peek()]) {
                stack.push(i);
            }
        }
        for (int i = nums.length - 1; i >= 0; --i) {
            while (!stack.isEmpty() && nums[i] <= nums[stack.peek()]) {
                ans = Math.max(ans, i - stack.peek());
                stack.pop();
            }
            return ans;
        }
    }
}

Java
class Solution {
    public int maxWithRange(int[] nums) {
        int n = nums.length;
        Deque<Integer> stk = new ArrayDeque<>();
        for (int i = 0; i < n; ++i) {
            if (stk.isEmpty() || nums[i] > nums[stk.peek()]) {
                stk.push(i);
            }
        }
        int ans = 0;
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.isEmpty() && nums[i] <= nums[stk.peek()]) {
                ans = Math.max(ans, i - stk.peek());
            }
            if (stk.isEmpty()) {
                break;
            }
        }
        return ans;
    }
}

Java
```

```
class Solution {
    public int maxWithRange(int[] nums) {
        int n = nums.length;
        Deque<Integer> stk = new ArrayDeque<>();
        for (int i = 0; i < n; ++i) {
            if (stk.isEmpty() || nums[i] > nums[stk.peek()]) {
                stk.push(i);
            }
        }
        int ans = 0;
        for (int i = n - 1; i >= 0; --i) {
            while (!stk.isEmpty() && nums[i] <= nums[stk.peek()]) {
                ans = Math.max(ans, i - stk.peek());
            }
            if (stk.isEmpty()) {
                break;
            }
        }
        return ans;
    }
}

TypeScript
TypeScript
function maxWithRange(nums: number[]): number {
    let [ans, n] = [0, nums.length];
    const stk: number[] = [];
    for (let i = 0; i < n; ++i) {
        if (stk.length === 0 || nums[i] > nums[stk[stk.length - 1]]) {
            stk.push(i);
        }
    }
    for (let i = n - 1; i >= 0; --i) {
        while (stk.length && nums[i] <= nums[stk[stk.length - 1]]) {
            ans = Math.max(ans, i - stk[stk.length - 1]);
        }
        if (stk.length === 0) break;
    }
    return ans;
}

TypeScript
```

```
function maxWithRange(nums: number[]): number {
    let [ans, n] = [0, nums.length];
    const stk: number[] = [];
    for (let i = 0; i < n; ++i) {
        if (stk.length === 0 || nums[i] > nums[stk[stk.length - 1]]) {
            stk.push(i);
        }
    }
    for (let i = n - 1; i >= 0; --i) {
        while (stk.length && nums[i] <= nums[stk[stk.length - 1]]) {
            ans = Math.max(ans, i - stk[stk.length - 1]);
        }
        if (stk.length === 0) break;
    }
    return ans;
}

Go
Go
func maxWithRange(nums []int) int {
    n := len(nums)
    stk := []int{}
    for i, v := range nums {
        if len(stk) == 0 || nums[stk[len(stk)-1]] > v {
            stk = append(stk, i)
        }
    }
    ans := 0
    for i := n - 1; i >= 0; i-- {
        for len(stk) > 0 && nums[stk[len(stk)-1]] <= nums[i] {
            ans = max(ans, i-stk[len(stk)-1])
            stk = stk[:len(stk)-1]
        }
        if len(stk) == 0 {
            break
        }
    }
    return ans
}

Go
```

```
func maxWithRange(nums []int) int {
    n := len(nums)
    stk := []int{}
    for i, v := range nums {
        if len(stk) == 0 || nums[stk[len(stk)-1]] > v {
            stk = append(stk, i)
        }
    }
    ans := 0
    for i := n - 1; i >= 0; i-- {
        for len(stk) > 0 && nums[stk[len(stk)-1]] <= nums[i] {
            ans = max(ans, i-stk[len(stk)-1])
            stk = stk[:len(stk)-1]
        }
        if len(stk) == 0 {
            break
        }
    }
    return ans
}

[""] Stack — Pattern Solution (matched from community answers)
Python
# Python solution for Maximum Width Ramp.
def maxWithRange(nums):
    n = len(nums)
    stack = []
    # Build a stack of indices where array values are in decreasing order
    for i in range(n):
        if not stack or nums[i] < nums[stack[-1]]:
            stack.append(i)
    max_width = 0
    # Iterate backwards to find the maximum width ramp
    for j in range(n-1, -1, -1):
        while stack and nums[j] >= nums[stack[-1]]:
            max_width = max(max_width, j - stack.pop())
    return max_width

# Example usage:
nums = [9, 8, 6, 2, 1, 5]
print(maxWithRange(nums)) # Output: 4
```

#1214 MEDIUM

Two Sum BSTs

[LeetCode](#) · [SimplyLeet](#) · [WuHCC](#) · [LeetCode](#) · [Editorial](#)

Two Pointers · Binary Search · Stack · Tree · DFS · BST · BFS

Like · Premium

Problem:

Given the roots of two binary search trees, root1 and root2, return true if and only if there is a node in the first tree and a node in the second tree whose values sum up to a given integer target.

Example 1:

9	9	8	1
5	6	2	6
8	2	6	4
6	2	2	2

9	9
8	6

Input: root1 = [2,1,4], root2 = [1,0,3], target = 5
Output: true
Explanation: 2 and 3 sum up to 5.

Example 2:

```
graph TD
    0((0)) --> -10((-10))
    0 --> 10((10))
    10 --> 1((1))
    10 --> 7((7))
    1 --> 0((0))
    1 --> 2((2))
```

Input: root1 = [8,-10,10], root2 = [5,1,7,0,2], target = 18
Output: false
Constraints:
• The number of nodes in each tree is in the range [1, 5000].
• -109 <= Node.val, target <= 109

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def twoSumBSTs(root1: TreeNode, root2: TreeNode, target: int) -> bool:
    # Create a set to store values from the first BST
    values_set = set()

    # Helper function to perform DFS and add node values to the set.
    def dfs_collect(node):
        if not node:
            return
        values_set.add(node.val) # Add current node value
        dfs_collect(node.left) # Traverse left subtree
        dfs_collect(node.right) # Traverse right subtree

    # Collect all values from first tree.
    dfs_collect(root1)

    # Helper function to traverse second tree and check for complement.
    def dfs_check(node):
        if not node:
            return False

        # If the complement exists in the first tree values, return True.
        if target - node.val in values_set:
            return True

        # Recursively check left and right children.
        return dfs_check(node.left) or dfs_check(node.right)

    # Check the second tree.
    return dfs_check(root2)

# Example usage:
# Construct trees and call twoSumBSTs as needed.
```

Python

Stack · LeetCode · By Pattern

— 2869 —

LeetCode

```
class BSTIterator:
    def __init__(self, root: TreeNode | None, leftToRight: bool):
        self.stack = []
        self.leftToRight = leftToRight
        self._pushUntilNone(root)

    def hasNext(self) -> bool:
        return len(self.stack) > 0

    def next(self) -> int:
        node = self.stack.pop()
        if self.leftToRight:
            self._pushUntilNone(node.right)
        else:
            self._pushUntilNone(node.left)
        return node.val

    def _pushUntilNone(self, root: TreeNode | None):
        while root:
            self.stack.append(root)
            root = root.left if self.leftToRight else root.right

class Solution:
    def twoSumBSTs(
        self,
        root1: TreeNode | None,
        root2: TreeNode | None,
        target: int,
    ) -> bool:
        bst1 = BSTIterator(root1, True)
        bst2 = BSTIterator(root2, False)

        l = bst1.next()
        r = bst2.next()
        while True:
            ssum = l + r
            if ssum == target:
                return True
            if ssum < target:
                if not bst1.hasNext():
                    return False
                l = bst1.next()
            else:
                if not bst2.hasNext():
                    return False
                r = bst2.next()
```

Python

Stack · LeetCode · By Pattern

— 2870 —

LeetCode

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def twoSumBSTs(
        self, root1: Optional[TreeNode], root2: Optional[TreeNode], target: int
    ) -> bool:
        def dfs(root: Optional[TreeNode], i: int):
            if root is None:
                return
            dfs(root.left, i)
            nums[i].append(root.val)
            dfs(root.right, i)

        nums = [[] for _ in range(1000)]
        dfs(root1, 0)
        dfs(root2, 1)
        for i in range(1000):
            l, j = 0, len(nums[i]) - 1
            while l < len(nums[i]) and j > 0:
                x = nums[i][l] + nums[i][j]
                if x == target:
                    return True
                if x < target:
                    l += 1
                else:
                    j -= 1
            return False
```

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def twoSumBSTs(
        self, root1: Optional[TreeNode], root2: Optional[TreeNode], target: int
    ) -> bool:
        def dfs(root: Optional[TreeNode], i: int):
            if root is None:
                return
            dfs(root.left, i)
            nums[i].append(root.val)
            dfs(root.right, i)

        nums = [[] for _ in range(1000)]
        dfs(root1, 0)
        dfs(root2, 1)
        for i in range(1000):
            l, j = 0, len(nums[i]) - 1
            while l < len(nums[i]) and j > 0:
                x = nums[i][l] + nums[i][j]
                if x == target:
                    return True
                if x < target:
                    l += 1
                else:
                    j -= 1
            return False
```

Stack · LeetCode · By Pattern

— 2871 —

LeetCode

```
if x == target:
    i += 1
else:
    j -= 1
return False
```

C++

C++

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool twoSumBSTs(TreeNode* root1, TreeNode* root2, int target) {
        vector<int> nums[2];
        function<void(TreeNode*, int)> dfs = [&](TreeNode* root, int i) {
            if (!root) return;
            dfs(root->left, i);
            nums[i].push_back(root->val);
            dfs(root->right, i);
        };
        dfs(root1, 0);
        dfs(root2, 1);

        for (int i = 0; i < nums[0].size(); i++) {
            while (i < nums[0].size() && j >= 0) {
                int x = nums[0][i] + nums[1][j];
                if (x == target) {
                    return true;
                }
                if (x < target) {
                    i++;
                } else {
                    j--;
                }
            }
            return false;
        }
    }
};
```

C++

Stack · LeetCode · By Pattern

— 2872 —

LeetCode

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool twoSumBSTs(TreeNode* root1, TreeNode* root2, int target) {
        vector<int> nums[2];
        function<void(TreeNode*, int)> dfs = [&](TreeNode* root, int i) {
            if (!root) return;
            dfs(root->left, i);
            nums[i].push_back(root->val);
            dfs(root->right, i);
        };
        dfs(root1, 0);
        dfs(root2, 1);

        for (int i = 0; i < nums[0].size(); i++) {
            while (i < nums[0].size() && j >= 0) {
                int x = nums[0][i] + nums[1][j];
                if (x == target) {
                    return true;
                }
                if (x < target) {
                    i++;
                } else {
                    j--;
                }
            }
            return false;
        }
    }
};
```

Java

Java

Stack · LeetCode · By Pattern

— 2873 —

LeetCode

```
/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    private List<Integer> nums = new List<>();

    public boolean twoSumBSTs(TreeNode root1, TreeNode root2, int target) {
        Arrays.sort(nums, k -> new ArrayList<>());
        dfs(root1, 0);
        dfs(root2, 1);
        for (int i = 0; i < nums[0].size(); i++) {
            while (i < nums[0].size() && j >= 0) {
                int x = nums[0].get(i) + nums[1].get(j);
                if (x == target) {
                    return true;
                }
                if (x < target) {
                    i++;
                } else {
                    j--;
                }
            }
            return false;
        }
    }

    private void dfs(TreeNode root, int i) {
        if (root == null) return;
        dfs(root.left, i);
        nums[i].add(root.val);
        dfs(root.right, i);
    }
}
```

TypeScript

TypeScript

Stack · LeetCode · By Pattern

— 2874 —

LeetCode

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number
//     left: TreeNode | null
//     right: TreeNode | null
// }
// constructor(val: number, left: TreeNode | null, right: TreeNode | null) {
//     this.val = val;
//     this.left = left;
//     this.right = right;
// }
//

function bstFromPreorder(preorder: number[], root2: TreeNode | null, target: number): boolean {
    const nums: number[] = preorder;
    if (!nums.length) return false;
    const dfs = (root: TreeNode | null, i: number) => {
        if (!root) return false;
        dfs(root.left, i);
        nums[i].push(root.val);
        dfs(root.right, i);
    };
    dfs(root2, 0);
    dfs(root2, 1);
    let i = 0;
    let j = nums[0].length - 1;
    while (i < nums[0].length && j >= 0) {
        const x = nums[0][i] + nums[1][j];
        if (x === target) {
            return true;
        }
        if (x < target) {
            i++;
        } else {
            j--;
        }
    }
    return false;
}

```

Go

Go

Stack · LeetCode · By Pattern

— 2875 —

LeetCode

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number
//     left: TreeNode | null
//     right: TreeNode | null
// }
// constructor(val: number, left: TreeNode | null, right: TreeNode | null) {
//     this.val = val;
//     this.left = left;
//     this.right = right;
// }
//

function bstFromPreorder(preorder: number[], root2: TreeNode | null, target: number): boolean {
    const nums: number[] = preorder;
    if (!nums.length) return false;
    const dfs = (root: TreeNode | null, i: number) => {
        if (!root) return false;
        dfs(root.left, i);
        nums[i].push(root.val);
        dfs(root.right, i);
    };
    dfs(root2, 0);
    dfs(root2, 1);
    let i = 0;
    let j = nums[0].length - 1;
    while (i < nums[0].length && j >= 0) {
        const x = nums[0][i] + nums[1][j];
        if (x === target) {
            return true;
        }
        if (x < target) {
            i++;
        } else {
            j--;
        }
    }
    return false;
}

```

Go

Stack · LeetCode · By Pattern

— 2876 —

LeetCode

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number
//     left: TreeNode | null
//     right: TreeNode | null
// }
// constructor(val: number, left: TreeNode | null, right: TreeNode | null) {
//     this.val = val;
//     this.left = left;
//     this.right = right;
// }
//

function bstFromPreorder(preorder: number[], root2: TreeNode | null, target: number): boolean {
    const nums: number[] = preorder;
    if (!nums.length) return false;
    const dfs = (root: TreeNode | null, i: number) => {
        if (!root) return false;
        dfs(root.left, i);
        nums[i].push(root.val);
        dfs(root.right, i);
    };
    dfs(root2, 0);
    dfs(root2, 1);
    let i = 0;
    let j = nums[0].length - 1;
    while (i < nums[0].length && j >= 0) {
        const x = nums[0][i] + nums[1][j];
        if (x === target) {
            return true;
        }
        if (x < target) {
            i++;
        } else {
            j--;
        }
    }
    return false;
}

```

[*] Stack — Pattern Solution (matched from community answers)

Python

Stack · LeetCode · By Pattern

— 2877 —

LeetCode

```

class BSTIterator:
    def __init__(self, root: TreeNode | None, leftToRight: bool):
        self.stack = []
        self.leftToRight = leftToRight
        self._pushUntilNone(root)

    def hasNext(self) -> bool:
        return len(self.stack) > 0

    def next(self) -> int:
        node = self.stack.pop()
        if self.leftToRight:
            self._pushUntilNone(node.right)
        else:
            self._pushUntilNone(node.left)
        return node.val

    def _pushUntilNone(self, root: TreeNode | None):
        while root:
            self.stack.append(root)
            root = root.left if self.leftToRight else root.right

class Solution:
    def bstFromPreorder(self, preorder: List[int]) -> List[int]:
        self.root1: TreeNode | None = None
        self.root2: TreeNode | None = None
        self.target: int = 0
        ) = bool:
            bst1 = BSTIterator(root1, True)
            bst2 = BSTIterator(root2, False)
            l = bst1.next()
            r = bst2.next()
            while True:
                sum = l + r
                if sum == target:
                    return True
                if sum < target:
                    if not bst1.hasNext():
                        return False
                    l = bst1.next()
                else:
                    if not bst2.hasNext():
                        return False
                    r = bst2.next()

```

#1762 MEDIUM

Buildings With an Ocean View

LeetCode · Solution · WalkCC · LeetCode · Editorial

Array · Stack · Monotonic Stack

List: Premium 98

Problem:

Stack · LeetCode · By Pattern

— 2878 —

LeetCode

There are n buildings in a line. You are given an integer array `heights` of size n that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a smaller height.

Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in increasing order.

Example 1:

Input: `heights = [4,2,3,1]`

Output: `[0,2,3]`

Explanation: Building 1 (0-indexed) does not have an ocean view because building 2 is taller.

Example 2:

Input: `heights = [4,3,2,1]`

Output: `[0,1,2,3]`

Explanation: All the buildings have an ocean view.

Example 3:

Input: `heights = [1,3,2,4]`

Output: `[3]`

Explanation: Only building 3 has an ocean view.

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $1 \leq \text{heights}[i] \leq 109$

>> Brute Force [Stack] Interview Prep

Python

```

# Brute Force Solution
class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        # Brute Force (O(N^2)) - calculate without stack via inner loop
        n = len(heights)
        result = []
        for i in range(n):
            for j in range(i + 1, n):
                if heights[i] > heights[j]:
                    result.append(i)
                    break
        return result

```

Python

Python

Stack · LeetCode · By Pattern

— 2879 —

LeetCode

```

def findBuildings(self, heights: List[int]) -> List[int]:
    n = len(heights)
    result = []
    max_height = 0
    for i in range(n - 1, -1, -1):
        if heights[i] > max_height:
            result.append(i)
            max_height = heights[i]
    # Reverse the result to return indices in increasing order.
    result.reverse()
    return result

```

Example usage:
heights = [4,2,3,1]
print(findBuildings(self, heights)) # Output: [0,2,3]

Python

```

class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        stack = []
        for i, height in enumerate(heights):
            while stack and heights[stack[-1]] <= height:
                stack.pop()
            stack.append(i)
        return stack

```

Python

```

class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        ans = []
        m = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > m:
                ans.append(i)
                m = heights[i]
        return ans[::-1]

```

Python

```

class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        ans = []
        m = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > m:
                ans.append(i)
                m = heights[i]
        return ans[::-1]

```

C++

C++

Stack · LeetCode · By Pattern

— 2880 —

LeetCode

```

class Solution {
public:
    vector<int> findBuildings(vector<int>& heights) {
        vector<int> stack;

        for (int i = 0; i < heights.size(); ++i) {
            while (!stack.empty() && heights[stack.back()] <= heights[i])
                stack.pop_back();
            stack.push_back(i);
        }

        return stack;
    }
};

```

C++

```

class Solution {
public:
    vector<int> findBuildings(vector<int>& heights) {
        vector<int> ans;
        int n = heights.size();
        for (int i = heights.size() - 1; i >= 0; --i) {
            if (heights[i] > ans[i])
                ans.push_back(i);
            else
                ans[i] = heights[i];
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};

```

C++

```

class Solution {
public:
    vector<int> findBuildings(vector<int>& heights) {
        vector<int> ans;
        int n = heights.size();
        for (int i = heights.size() - 1; i >= 0; --i) {
            if (heights[i] > ans[i])
                ans.push_back(i);
            else
                ans[i] = heights[i];
        }
        reverse(ans.begin(), ans.end());
        return ans;
    }
};

```

Java

```

class Solution {
public int[] findBuildings(int[] heights) {
    Queue<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < heights.length; ++i) {
        while (!stack.isEmpty() && heights[stack.peek()] <= heights[i])
            stack.poll();
        stack.add(i);
    }

    int[] ans = new int[stack.size()];
    for (int i = 0; i < ans.length; ++i)
        ans[i] = heights[stack.poll()];
    return ans;
}
}

```

Java

```

class Solution {
public int[] findBuildings(int[] heights) {
    int n = heights.length;
    List<Integer> ans = new ArrayList<>();
    int m = 0;
    for (int i = heights.length - 1; i >= 0; --i) {
        if (heights[i] > m) {
            ans.add(i);
            m = heights[i];
        }
    }
    Collections.reverse(ans);
    return ans.stream().mapToInt(Integer::intValue).toArray();
}
}

```

Java

```

class Solution {
public int[] findBuildings(int[] heights) {
    int n = heights.length;
    List<Integer> ans = new ArrayList<>();
    int m = 0;
    for (int i = heights.length - 1; i >= 0; --i) {
        if (heights[i] > m) {
            ans.add(i);
            m = heights[i];
        }
    }
    Collections.reverse(ans);
    return ans.stream().mapToInt(Integer::intValue).toArray();
}
}

```

JavaScript

```

//
// @param {number[]} heights
// @return {number[]}
//
var findBuildings = function (heights) {
    const ans = [];
    let n = heights.length;
    for (let i = heights.length - 1; i >= 0; --i) {
        if (heights[i] > ans[i])
            ans.push(i);
        else
            ans[i] = heights[i];
    }
    return ans.reverse();
};

```

JavaScript

```

//
// @param {number[]} heights
// @return {number[]}
//
var findBuildings = function (heights) {
    const ans = [];
    let n = heights.length;
    for (let i = heights.length - 1; i >= 0; --i) {
        if (heights[i] > ans[i])
            ans.push(i);
        else
            ans[i] = heights[i];
    }
    return ans.reverse();
};

```

TypeScript

```

function findBuildings(heights: number[]): number[] {
    const ans: number[] = [];
    let n = heights.length;
    for (let i = heights.length - 1; i >= 0; --i) {
        if (heights[i] > ans[i])
            ans.push(i);
        else
            ans[i] = heights[i];
    }
    return ans.reverse();
}

```

```

function findBuildings(heights: number[]): number[] {
    const ans: number[] = [];
    let n = heights.length;
    for (let i = heights.length - 1; i >= 0; --i) {
        if (heights[i] > ans[i])
            ans.push(i);
        else
            ans[i] = heights[i];
    }
    return ans.reverse();
}

```

Go

```

func findBuildings(heights []int) (ans []int) {
    n := len(heights)
    for i := n - 1; i >= 0; i-- {
        if heights[i] > ans[i] {
            ans = append(ans, i)
        } else {
            ans[i] = heights[i]
        }
    }
    return ans.reverse()
}

```

Go

```

func findBuildings(heights []int) (ans []int) {
    n := len(heights)
    for i := n - 1; i >= 0; i-- {
        if heights[i] > ans[i] {
            ans = append(ans, i)
        } else {
            ans[i] = heights[i]
        }
    }
    return ans.reverse()
}

```

#32 **HARD**

Longest Valid Parentheses

LeetCode · Simplify · Waffle · LeetCode · Editorial

String · Dynamic Programming · Stack

Lists · Grid · 169

Problem:

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1:

Input: s = "(()"

Output: 2

Explanation: The longest valid parentheses substring is "()".

Example 2:

Input: s = "()()"

Output: 4

Explanation: The longest valid parentheses substring is "()()".

Example 3:

Input: s = ""

Output: 0

Constraints:

- 0 <= s.length <= 3 * 10⁴
- s[i] is '(' or ')'

```

# Python solution using a stack
def longestValidParentheses(s: str) -> int:
    # Initialize the stack with the base index -1
    stack = [-1]
    max_length = 0

    # Iterate over the string with index
    for i, char in enumerate(s):
        if char == '(':
            # Push the index of '(' onto the stack
            stack.append(i)
        else:
            # Pop the last index for ')'
            stack.pop()
            # If our stack is empty, push current index as the new base
            if not stack:
                stack.append(i)
            # Calculate the length of the current valid substring
            current_length = i - stack[-1]
            max_length = max(max_length, current_length)

    return max_length

# Example usage:
print(longestValidParentheses("()()()")) # Output: 6

```

Python

```

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        # dp[i] -> the length of the longest valid parentheses in the substring s[0:i]
        dp = [0] * n
        for i in range(1, n):
            if s[i] == ')' and s[i-1] == '(':
                dp[i] = dp[i-1] + 2
            elif s[i] == ')' and s[i-2] == '(':
                dp[i] = dp[i-2] + 2
            elif s[i] == ')' and s[i-1] == ')' and s[i-2] == '(':
                dp[i] = dp[i-1] + dp[i-2] + 2
        return max(dp)

```

Python

```

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        # dp[i] -> the length of the longest valid parentheses in the substring s[0:i]
        dp = [0] * n
        for i in range(1, n):
            if s[i] == ')' and s[i-1] == '(':
                dp[i] = dp[i-1] + 2
            elif s[i] == ')' and s[i-2] == '(':
                dp[i] = dp[i-2] + 2
            elif s[i] == ')' and s[i-1] == ')' and s[i-2] == '(':
                dp[i] = dp[i-1] + dp[i-2] + 2
        return max(dp)

```

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        stack = [-1]
        ans = 0
        for i in range(len(s)):
            if s[i] == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:
                    stack.append(i)
                else:
                    ans = max(ans, i - stack[-1])
        return ans
```

Python

```
class Solution:
    def isTargetInRange(self, s: str) -> bool:
        left = right = 0
        res = 0
        for i in s:
            if i == 'r':
                left = right + 1
            elif i == 'l':
                right = left - 1
            else:
                res += 1
        if left == right:
            return res == 0
        elif left > right:
            return res == left - right
        elif left < right:
            return res == right - left
        else:
            return res == 0
```

Stack · LeetCode — By Pattern

— 2887 —

LeetMastery

```

        return res

#####

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        if n < 2:
            return 0
        dp = [0] * n
        for i in range(1, n):
            if s[i] == ')':
                if s[i - 1] == '(':
                    dp[i] = 2 + (dp[i - 2] if i > 1 else 0)
                else:
                    j = i - dp[i - 1] - 1
                    if j >= 0 and s[j] == '(':
                        dp[i] = 2 + dp[i - 1] + dp[j - 1]
            return max(dp)

```

```
>>> s="abcdefg"
>>> [print(i,",",c) for i, c in enumerate(s, 1)]
```

```
1, a
2, b
3, c
4, d
5, e
6, f
7, g
[None, None, None, None, None, None, None]
>>>
...

```

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        f[0] = (n + 1)
        for i, c in enumerate(s, 1): # starting i from 1, not 0
            if c == "(":
                if f[i + 1] and s[i + 1] == ")":
                    f[i] = f[i + 2] + 2
            else:
                j = i - f[i - 1] - 1
                if f[j] and s[j] == "(":
                    f[i] = f[i - 1] + 2 + f[j] - 1
        return max(f)
```

C++

C++

Stack · LeetCode — By Pattern

— 2888 —

LeetMastery

```

class Solution {
public:
    int longestValidParentheses(string s) {
        const string sz2 = "]]";
        // dp[i] = the length of the longest valid parentheses in the substring
        // s[0..i]
        vector<int> dp(sz2.length());

        for (int i = 1; i < sz2.length(); ++i)
            if (sz2[i] == ']' && sz2[i - 1] == '[' ||
                dp[i] == dp[i - 1] + dp[i - 1] - 2) + 2;

        return ranges::max(dp);
    }
};

```

C++

```

class Solution {
public:
    string s;
    int n;
    int left;
    int right;

    Function longestValidParentheses(int s){
        s=s.substr(0,s.length()-1);
        n=s.length();
        array<pushStack, -1>;
        for (int i=0; i<=s.length(); i++){
            if (s[i]!='('){
                array<pushStack, i>;
            }
            array<popStack>;
            if (empty(sStack)){
                array<pushStack, i>;
            }
            else {
                length = i - end(sStack);
                sStack.push_back(s[i]);
            }
        }
        return sStack.length();
    }
};

```

C++

Stack · LeetCode — By Pattern

— 2889 —

LeetMastery

```
class Solution {  
    //  
    = > Given string s  
    = > Return integer  
    //  
  
    function longestValidParentheses(s) {  
        stack = [];  
        sizeLength = s.length - 1;  
  
        array.push(stack, -1);  
  
        for (let i = 0; i < s.length; i++) {  
            if (isLeftChar === '{' ||  
                array.push(stack, i));  
        } else {  
            array.pop(stack);  
  
            if (empty(stack)) {  
                array.push(stack, i);  
            } else {  
                length = i - endOf(stack);  
                sizeLength = max(sizeLength, length);  
            }  
        }  
  
        return sizeLength;  
    }  
}
```

Java

Java

```
class Solution {
    public int longestValidParentheses(String s) {
        final String s2 = "}" + s;
        // dp[i] = the length of the longest valid parentheses in the substring
        // s[0..i-1]
        int dp[] = new int[s2.length()];

        for (int i = 1; i = s2.length(); ++i)
            if (s2.charAt(i) == '}') dp[i] = dp[i - 1] + 1 == '(''
                ? dp[i - 1] + dp[i - dp[i - 1] - 1] + 2 : 1;

        return Arrays.stream(dp).max().getAsInt();
    }
}
```

Java

Stack · LeetCode — By Pattern

— 2890 —

LeetMastery

```

class Solution {
public:
    int longestValidParentheses(String s) {
        int n = s.length();
        int[] f = new int[n + 1];
        int ans = 0;
        for (int i = 2; i <= n; ++i) {
            if (s.charAt(i - 1) == '('') {
                if (s.charAt(i - 2) == '('') {
                    f[i] = f[i - 2] + 2;
                } else {
                    int j = i - f[i - 1] - 1;
                    if (j >= 0 && s.charAt(j) == '('') {
                        f[i] = f[i - 1] + 2 + f[j - 1];
                    }
                }
            }
            ans = Math.max(ans, f[i]);
        }
        return ans;
    }
}

```

Java

```
public class Solution {
    public int LongestValidParentheses(string s) {
        int n = s.Length;
        int[] f = new int[n + 1];
        int ans = 0;
        for (int i = 2; i <= n; ++i) {
            if (s[i - 1] == ')') {
                if (s[i - 2] == '(') {
                    f[i] = f[i - 2] + 2;
                } else {
                    int j = i - f[i - 1] - 1;
                    if (j >= 0 && s[j] == '(') {
                        f[i] = f[i - 1] + 2 + f[j + 1];
                    }
                }
            }
            ans = Math.Max(ans, f[i]);
        }
        return ans;
    }
}
```

java

```
class Solution {
    public int longestValidParentheses(String s) {
        int n = s.length();
        int[] f = new int[n + 1];
        int ans = 0;
        for (int i = 2; i <= n; ++i) {
            if (s.charAt(i - 1) == '('') {
                if (s.charAt(i - 2) == ')') {
                    f[i] = f[i - 2] + 2;
                }
            } else {
                int j = i - 1;
                while (j > 0 && s.charAt(j - 1) == '('') {
                    f[i] = f[i - 1] + 2 + f[j - 1];
                    j = j - 1;
                }
                ans = Math.max(ans, f[i]);
            }
        }
        return ans;
    }
}
```

```
class Solution_noExtraSpace {
```

```
public int longestPalindrome(String s) {
    int res = 0, left = 0, right = 0, n = s.length();

    // from left to right, '()' will never hit left=right
    for (int i = 0; i < n; i++) {
        if (s.charAt(i) == '('') < left;
        else < right;

        if (left == right) res = Math.max(res, 2 + right);
        else if (right < left) left = right = 0;
    }

    // from right to left, '()' will never hit left=right
    left = right = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (s.charAt(i) == '('') < left;
        else < right;

        if (left == right) res = Math.max(res, 2 + left);
        else if (left < right) right = left = 0;
    }

    return res;
}
```

```

//[[[[
class Solution {
public: int longestValidParentheses(string s) {
    Stack<Integer> sk = new Stack<>();
    int start = 0;
    int result = 0;
    for (int i = 0; i < s.length(); i++) {
        if(s.charAt(i) == '(') {
            sk.push(i);
        } else {
            if (sk.empty()) {
                start = i + 1;
            }
            sk.pop();
            result = Math.max(result, sk.isEmpty() ? i - start + 1 : i - sk.pop());
        }
    }
    return result;
}
}

```

```

java
public class Solution {
public: int longestValidParentheses(string s) {
    int n = s.length;
    int[] f = new int[n + 1];
    int ans = 0;
    for (int i = 2; i <= n; ++i) {
        if (s[i-1] == '(') {
            if (s[i-2] == ')') {
                f[i] = f[i-2] + 2;
            } else {
                int j = i - f[i-1] - 1;
                if (j >= 0 && s[j] == '(') {
                    f[i] = f[i-1] + 2 + f[j+1];
                }
            }
            ans = Math.Max(ans, f[i]);
        }
    }
    return ans;
}
}

```

JavaScript
JavaScript

Stack · LeetCode — By Pattern

— 2893 —

LeetCode

```

//[[[[
var longestValidParentheses = function (s) {
    const n = s.length;
    const f = new Array(n + 1).fill(0);
    for (let i = 2; i <= n; ++i) {
        if (s[i-1] == '(') {
            if (s[i-2] == ')') {
                f[i] = f[i-2] + 2;
            } else {
                const j = i - f[i-1] - 1;
                if (j >= 0 && s[j] == '(') {
                    f[i] = f[i-1] + 2 + f[j+1];
                }
            }
        }
    }
    return Math.max(...f);
};

```

JavaScript

```

//[[[[
var longestValidParentheses = function (s) {
    const n = s.length;
    const f = new Array(n + 1).fill(0);
    for (let i = 2; i <= n; ++i) {
        if (s[i-1] == '(') {
            if (s[i-2] == ')') {
                f[i] = f[i-2] + 2;
            } else {
                const j = i - f[i-1] - 1;
                if (j >= 0 && s[j] == '(') {
                    f[i] = f[i-1] + 2 + f[j+1];
                }
            }
        }
    }
    return Math.max(...f);
};

```

TypeScript
TypeScript

Stack · LeetCode — By Pattern

— 2894 —

LeetCode

```

function longestValidParentheses(s: string): number {
    const n = s.length;
    const f: number[] = new Array(n + 1).fill(0);
    for (let i = 2; i <= n; ++i) {
        if (s[i-1] == '(') {
            if (s[i-2] == ')') {
                f[i] = f[i-2] + 2;
            } else {
                const j = i - f[i-1] - 1;
                if (j >= 0 && s[j] == '(') {
                    f[i] = f[i-1] + 2 + f[j+1];
                }
            }
        }
    }
    return Math.max(...f);
}

```

TypeScript

```

function longestValidParentheses(s: string): number {
    let max_length: number = 0;
    const stack: number[] = [];
    for (let i = 0; i < s.length; i++) {
        if (s.charAt(i) == '(') {
            stack.push(i);
        } else {
            stack.pop();
            if (stack.length == 0) {
                stack.push(i);
            } else {
                max_length = Math.max(max_length, i - stack[stack.length - 1]);
            }
        }
    }
    return max_length;
}

```

TypeScript

```

function longestValidParentheses(s: string): number {
    const n = s.length;
    const f: number[] = new Array(n + 1).fill(0);
    for (let i = 2; i <= n; ++i) {
        if (s[i-1] == '(') {
            if (s[i-2] == ')') {
                f[i] = f[i-2] + 2;
            } else {
                const j = i - f[i-1] - 1;
                if (j >= 0 && s[j] == '(') {
                    f[i] = f[i-1] + 2 + f[j+1];
                }
            }
        }
    }
    return Math.max(...f);
}

```

Stack · LeetCode — By Pattern

— 2895 —

LeetCode

```

Go
func longestValidParentheses(s string) int {
    n := len(s)
    f := make([]int, n+1)
    for i := 2; i <= n; i++ {
        if s[i-1] == '(' {
            if s[i-2] == ')' {
                f[i] = f[i-2] + 2
            } else if j := i - f[i-1] - 1; j > 0 && s[j] == '(' {
                f[i] = f[i-1] + 2 + f[j+1]
            }
        }
    }
    return slices.Max(f)
}

```

Go

```

func longestValidParentheses(s string) int {
    ans := 0
    stack := []int{}
    for i, v := range s {
        if v == '(' {
            stack = append(stack, i)
        } else {
            stack = stack[:len(stack)-1]
            if len(stack) == 0 {
                stack = append(stack, i)
            } else {
                if ans < i-stack[len(stack)-1] {
                    ans = i - stack[len(stack)-1]
                }
            }
        }
    }
    return ans
}

```

Go

```

func longestValidParentheses(s string) int {
    n := len(s)
    f := make([]int, n+1)
    for i := 2; i <= n; i++ {
        if s[i-1] == '(') {
            if s[i-2] == ')' {
                f[i] = f[i-2] + 2
            } else if j := i - f[i-1] - 1; j > 0 && s[j] == '(' {
                f[i] = f[i-1] + 2 + f[j+1]
            }
        }
    }
    return slices.Max(f)
}

```

Rust
Rust

Stack · LeetCode — By Pattern

— 2896 —

LeetCode

```

impl Solution {
    pub fn longest_valid_parentheses(s: String) -> i32 {
        let mut ans = 0;
        let mut f = vec![0; s.len() + 1];
        for i in 2..=s.len() {
            if s.chars().nth(i - 1).unwrap() == '(' {
                if s.chars().nth(i - 2).unwrap() == ')' {
                    f[i] = f[i - 2] + 2;
                } else if (i as i32) - f[i - 1] - 1 > 0 {
                    let s.chars().nth((f[i - 1] as usize) - 2).unwrap() == '('
                }
                f[i] = f[i - 1] + 2 + f[(f[i - 1] as usize) - 2];
            }
            ans = ans.max(f[i]);
        }
        ans
    }
}

```

Rust

```

impl Solution {
    pub fn longest_valid_parentheses(s: String) -> i32 {
        let mut ans = 0;
        let mut f = vec![0; s.len() + 1];
        for i in 2..=s.len() {
            if
                s
                    .chars()
                    .nth(i - 2)
                    .unwrap() == ')'
            {
                if
                    s
                        .chars()
                        .nth(i - 2)
                        .unwrap() == '('
                {
                    f[i] = f[i - 2] + 2;
                } else if
                    (i as i32) - f[i - 1] - 1 > 0 &&
                    s
                        .chars()
                        .nth((f[i - 1] as usize) - 2)
                        .unwrap() == '('
                {
                    f[i] = f[i - 1] + 2 + f[(f[i - 1] as usize) - 2];
                }
            }
            ans = ans.max(f[i]);
        }
        ans
    }
}

```

[*] Stack — Pattern Solution (matched from community answers)

Stack · LeetCode — By Pattern

— 2897 —

LeetCode

```

Python
# Python solution using a stack
def longestValidParentheses(s: str) -> int:
    # Initialize the stack with the base index -1
    stack = [-1]
    max_length = 0

    # Iterate over the string with index
    for i, char in enumerate(s):
        if char == '(':
            # Push the index of '(' onto the stack
            stack.append(i)
        else:
            # Pop the last index for ')'
            stack.pop()
            # If not stack:
            # If stack is empty, push current index as the new base
            stack.append(i)
        else:
            # Calculate the length of the current valid substring
            current_length = i - stack[-1]
            max_length = max(max_length, current_length)

    return max_length

# Example usage:
print(longestValidParentheses("()()()")) # Output: 6

```

#42

HARD

Trapping Rain Water

LeetCode · Simplest · Medium · LeetCode · Editorial

Array · Two Pointers · Dynamic Programming · Stack · Monotonic Stack

Like: Grind 169

Problem:

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Stack · LeetCode — By Pattern

— 2898 —

LeetCode

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:
Input: height = [4,2,0,3,2,5]
Output: 9

Constraints:

- n == height.length
- 1 <= n <= 2 * 10⁴
- 0 <= height[i] <= 105

>> Brute Force [Stack] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def trap(self, height: List[int]) -> int:
        # Brute Force O(n^2) - For each bar scan left/right for max heights
        n = len(height)
        ans = 0
        for i in range(n):
            left_max = max(height[:i + 1])
            right_max = max(height[i:])
            res = min(left_max, right_max) - height[i]
            return res
```

Python

Python

Stack · LeetCode - By Pattern

— 2899 —

LeetCode

```
class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0

        ans = 0
        l = 0
        r = len(height) - 1
        maxl = height[l]
        maxr = height[r]

        while l < r:
            if maxl < maxr:
                ans += maxl - height[l]
                l += 1
                maxl = max(maxl, height[l])
            else:
                ans += maxr - height[r]
                r -= 1
                maxr = max(maxr, height[r])

        return ans
```

Python

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i], height[n - i - 1])
        return sum(min(l, r) - h for l, r, h in zip(left, right, height))
```

Python

Stack · LeetCode - By Pattern

— 2901 —

LeetCode

```
# Calculate water trapped on the right side of the max height
right_max = height[-1]
for i in range(len(height) - 2, max_index, -1):
    if right_max < height[i]:
        right_max = height[i]
    total_water += right_max - height[i]
return total_water
```

C++

C++

```
class Solution {
public:
    int trap(vector<int>& height) {
        const int n = height.size();
        int ans = 0;
        vector<int> lmax; // l[i] = max(height[0..i])
        vector<int> rmax; // r[i] = max(height[i..n])

        for (int i = 0; i < n; ++i)
            lmax[i] = i < 1 ? height[0] : max(height[i], lmax[i - 1]);

        for (int i = n - 1; i >= 0; --i)
            rmax[i] = i == n - 1 ? height[n - 1] : max(height[i], rmax[i + 1]);

        for (int i = 0; i < n; ++i)
            ans += min(lmax[i], rmax[i]) - height[i];

        return ans;
    }
};
```

C++

Stack · LeetCode - By Pattern

— 2903 —

LeetCode

```
# Python solution using the two pointers approach
def trap(height):
    # Initialize left and right pointers and water accumulator
    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
    water = 0
```

```
# Traverse the elevation map from both ends
while left < right:
    # Update left_max and right_max as we encounter new bars
    if height[left] < height[right]:
        # Left side is the determining side
        if height[left] > left_max:
            left_max = height[left] # update left_max since current height is higher
        else:
            water += left_max - height[left] # water trapped at left position
            left += 1 # move left pointer to the right
    else:
        # Right side is the determining side
        if height[right] > right_max:
            right_max = height[right] # update right_max since current height is higher
        else:
            water += right_max - height[right] # water trapped at right position
            right -= 1 # move right pointer to the left
    return water
```

```
# Example usage:
print(trap([0,1,0,2,1,0,1,3,2,1,2,1])) # Expected output: 6
```

Python

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        l = [0] * n
        r = [0] * n
        for i in range(1, n):
            l[i] = max(l[i - 1], height[i])
        for i in range(n - 2, -1, -1):
            r[i] = max(r[i + 1], height[i])
        return sum(min(l[i], r[i]) - height[i] for i in range(n))
```

Python

Stack · LeetCode - By Pattern

— 2900 —

LeetCode

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        if n <= 3:
            return 0

        maxl = height[0]
        maxr = height[-1]
        for i in range(1, n):
            maxl = max(maxl, height[i])
            maxr = max(maxr, height[n - i - 1])

        # no negative, worst is min(left, right) is itself
        return sum(min(maxl, maxr) - height[i] for i in range(n))
```

Python

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        left = height[0]
        right = height[-1]
        for i in range(1, n):
            left = max(left, height[i])
            right = max(right, height[n - i - 1])

        return sum(min(l, r) - h for l, r, h in zip(left, right, height))
```

Python

```
class Solution:
    def trap(self, height: List[int]) -> int:
        if not height:
            return 0

        total_water = 0
        max_index = 0
        max_height = float('-inf')

        # Find the index of the max height
        for i in range(len(height)):
            if height[i] > max_height:
                max_height = height[i]
                max_index = i

        # Calculate water trapped on the left side of the max height
        left_max = height[0]
        for i in range(1, max_index):
            if left_max < height[i]:
                left_max = height[i]
            total_water += left_max - height[i]
```

Stack · LeetCode - By Pattern

— 2902 —

LeetCode

```
class Solution {
public:
    int trap(vector<int>& height) {
        if (height.empty())
            return 0;

        int ans = 0;
        int l = 0;
        int r = height.size() - 1;
        int maxl = height[l];
        int maxr = height[r];

        while (l < r)
            if (maxl < maxr) {
                ans += maxl - height[l];
                maxl = max(maxl, height[l + 1]);
            } else {
                ans += maxr - height[r];
                maxr = max(maxr, height[r - 1]);
            }
            return ans;
    }
};
```

C++

C++

```
class Solution {
public:
    int trap(vector<int>& height) {
        int n = height.size();
        int left[0], right[0];
        left[0] = height[0];
        right[n - 1] = height[n - 1];
        for (int i = 1; i < n; ++i) {
            left[i] = max(left[i - 1], height[i]);
            right[n - i - 1] = max(right[n - i], height[n - i - 1]);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans += min(left[i], right[i]) - height[i];
        }
        return ans;
    }
};
```

C++

Stack · LeetCode - By Pattern

— 2904 —

LeetCode


```

class Solution {
    // maxsum Integer[] theight
    // return Integer
    //
    function trap(theight) {
        n = theight.length;

        if (n == 0) {
            return 0;
        }

        start = 0;
        right = n - 1;
        leftMax = 0;
        rightMax = 0;
        sum = 0;

        while (start < right) {
            if (theight[start] < theight[right]) {
                if (theight[start] < leftMax) {
                    leftMax = theight[start];
                } else {
                    sum += leftMax - theight[start];
                }
                start++;
            } else {
                if (theight[right] > rightMax) {
                    rightMax = theight[right];
                } else {
                    sum += rightMax - theight[right];
                }
                right--;
            }
        }
        return sum;
    }
}

// 02. 11/2019 // leetcode.com/problems/trapping-water/
// Time: 0(1)
// Space:
class Solution {
public:
    int trap(vector<int>& A) {
        int N = A.size(), ans = 0;
        int* lefts = new int[N], *rights = new int[N];
        for (int i = 1; i < N; ++i) lefts[i] = max(lefts[i - 1], A[i - 1]);
        for (int i = N - 2; i >= 0; --i) rights[i] = max(rights[i + 1], A[i + 1]);
        for (int i = 1; i < N - 1; ++i) ans += min(lefts[i], rights[i]) - A[i];
        return ans;
    }
}

```

Stack · LeetCode — By Pattern — 2905 — LeetCode

```

class Solution {
    // maxsum Integer[] theight
    // return Integer
    //
    function trap(theight) {
        n = theight.length;

        if (n == 0) {
            return 0;
        }

        start = 0;
        right = n - 1;
        leftMax = 0;
        rightMax = 0;
        sum = 0;

        while (start < right) {
            if (theight[start] < theight[right]) {
                if (theight[start] < leftMax) {
                    leftMax = theight[start];
                } else {
                    sum += leftMax - theight[start];
                }
                start++;
            } else {
                if (theight[right] > rightMax) {
                    rightMax = theight[right];
                } else {
                    sum += rightMax - theight[right];
                }
                right--;
            }
        }
        return sum;
    }
}

Java

```

Stack · LeetCode — By Pattern — 2906 — LeetCode

```

class Solution {
    public int trap(int[] height) {
        int n = height.length;
        int ans = 0;
        int[] l = new int[n]; // l[i] := max(height[0..i])
        int[] r = new int[n]; // r[i] := max(height[i..n])

        for (int i = 0; i < n; ++i)
            l[i] = i == 0 ? height[i] : Math.max(height[i], l[i - 1]);

        for (int i = n - 1; i >= 0; --i)
            r[i] = i == n - 1 ? height[i] : Math.max(height[i], r[i + 1]);

        for (int i = 0; i < n; ++i)
            ans += Math.min(l[i], r[i]) - height[i];

        return ans;
    }
}

Java
class Solution {
    public int trap(int[] height) {
        if (height.length == 0)
            return 0;

        int n = 0;
        int l = 0;
        int r = height.length - 1;
        int maxl = height[l];
        int maxr = height[r];

        while (l < r) {
            if (maxl < maxr) {
                ans += Math.max(0, height[l] - maxl);
                maxl = Math.max(maxl, height[l++]);
            } else {
                ans += Math.max(0, height[r] - maxr);
                maxr = Math.max(maxr, height[r--]);
            }
        }

        return ans;
    }
}

Java

```

Stack · LeetCode — By Pattern — 2907 — LeetCode

```

class Solution {
    public int trap(int[] height) {
        int n = height.length;
        int[] left = new int[n];
        int[] right = new int[n];
        left[0] = height[0];
        left[n - 1] = height[n - 1];
        for (int i = 1; i < n; ++i) {
            left[i] = Math.max(left[i - 1], height[i]);
            right[n - i - 1] = Math.max(right[n - i], height[n - i - 1]);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans += Math.min(left[i], right[i]) - height[i];
        }
        return ans;
    }
}

Java
public class Solution {
    public int trap(int[] height) {
        int n = height.length;
        int[] left = new int[n];
        int[] right = new int[n];
        left[0] = height[0];
        left[n - 1] = height[n - 1];
        for (int i = 1; i < n; ++i) {
            left[i] = Math.max(left[i - 1], height[i]);
            right[n - i - 1] = Math.max(right[n - i], height[n - i - 1]);
        }
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans += Math.min(left[i], right[i]) - height[i];
        }
        return ans;
    }
}

Java

```

Stack · LeetCode — By Pattern — 2908 — LeetCode

```

public class TrappingMainWater {
    public static void main(String[] args) {
        TrappingMainWater wat = new TrappingMainWater();
        SolutionLocalMinimum s = wat.Solution_Local_Minimum();
        System.out.println(s.trapMinw(10, 15, 2, 1, 1, 5));
    }

    // The short board effect, the storage capacity is related to the short board of the bucket.
    // Find the longest slab in the height array, and then iterate from both ends to the long slab. When
    // encountering a shorter one, the storage capacity, and when encountering a longer one, update the edge.
    public class Solution_Minimum {
        public int trapMinw(int[] height) {
            if (height == null || height.length == 0) return 0;

            int sum = 0;
            int minind = 0;
            int max = Integer.MIN_VALUE;

            // Find min index
            for (int i = 0; i < height.length; i++) {
                if (height[i] < max) {
                    max = height[i];
                    minind = i;
                }
            }

            // Left
            int leftMax = height[0];
            for (int i = 1; i < minind; i++) {
                if (leftMax < height[i]) {
                    leftMax = height[i];
                }
            }
            sum += leftMax - height[1];

            // Right
            int rightMax = height[height.length - 1];
            for (int i = height.length - 2; i > minind; i--) {
                if (rightMax < height[i]) {
                    rightMax = height[i];
                }
            }
            sum += rightMax - height[i];

            return sum;
        }
    }
}

```

Stack · LeetCode — By Pattern — 2909 — LeetCode

```

    }

    // local minimum solution is NOT double. like {5,2,1,2,1,5}, there are 2 local minimums with each
    // holding 2 water
    // but they are just part of a global minimum
    class Solution_Local_Minimum {
        public int trapMinw(int[] height) {
            // for each position, probe to left and right, find local minimum

            if (height == null || height.length == 0) return 0;

            int sum = 0;

            let i = 1; // Indeed or indeed-length-1 will not be a local min, since nothing on its left to
            // hold water
            while (i < height.length - 1) {
                // only process local min index
                if (height[i] < height[i - 1] && height[i] < height[i + 1]) {
                    let left = i - 1;
                    while (left - 1 >= 0 && height[left] < height[left - 1]) {
                        left--;
                    }

                    let right = i + 1;
                    while (right + 1 < height.length && height[right] < height[right + 1]) {
                        right++;
                    }

                    sum += (height[right] - height[left]) * (right - left - 1);
                }
            }

            return sum;
        }
    }

    TypeScript

```

Stack · LeetCode — By Pattern — 2910 — LeetCode

```
function trap(height: number[]): number {
    const n = height.length;
    const left: number[] = new Array(n).fill(height[0]);
    const right: number[] = new Array(n).fill(height[n - 1]);
    for (let i = 0; i < n; i++) {
        left[i] = Math.max(left[i - 1], height[i]);
        right[n - i - 1] = Math.max(right[n - i - 1], height[n - i - 1]);
    }
    let ans = 0;
    for (let i = 0; i < n; i++) {
        ans += Math.min(left[i], right[i]) - height[i];
    }
    return ans;
}

TypeScript
function trap(height: number[]): number {
    let ans = 0;
    let left = 0;
    right = height.length - 1;
    let maxLeft = 0;
    maxRight = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            // move left
            if (height[left] >= maxLeft) {
                maxLeft = height[left];
            } else {
                ans += maxLeft - height[left];
            }
            left++;
        } else {
            // move right
            if (height[right] >= maxRight) {
                maxRight = height[right];
            } else {
                ans += maxRight - height[right];
            }
            right--;
        }
    }
    return ans;
}

Go
Go
```

```
func trap(height []int) (ans int) {
    n := len(height)
    left := make([]int, n)
    right := make([]int, n)
    left[0] = height[0]
    for i := 1; i < n; i++ {
        left[i] = max(left[i-1], height[i])
    }
    right[n-1] = height[n-1]
    for i := n-2; i >= 0; i-- {
        right[i] = max(right[i+1], height[i+1])
    }
    for i := 0; i < n; i++ {
        ans += min(left[i], right[i]) - height[i]
    }
    return ans
}

Go
func trap(height []int) int {
    n := len(height)
    if n < 3 {
        return 0
    }
    left, res := make([]int, n), make([]int, n)
    left[0] = height[0]
    for i := 1; i < n; i++ {
        left[i] = max(left[i-1], height[i])
    }
    right, res := make([]int, n), make([]int, n)
    right[n-1] = height[n-1]
    for i := n-2; i >= 0; i-- {
        right[i] = max(right[i+1], height[i+1])
    }
    for i := 0; i < n; i++ {
        res += min(left[i], right[i]) - height[i]
    }
    return res
}

func min(a, b int) int {
    if a < b {
        return a
    }
    return b
}

func min(a, b int) int {
    if a < b {
        return a
    }
    return b
}

Rust
Rust
```

```
impl Solution {
    pub fn trap(height: Vec<i32>) -> i32 {
        let n = height.len();
        let mut left: Vec<i32> = vec![0; n];
        let mut right: Vec<i32> = vec![0; n];

        left[0] = height[0];
        right[n - 1] = height[n - 1];

        // Initialize the left & right vector
        for i in 1..n {
            left[i] = std::cmp::max(left[i - 1], height[i]);
            right[n - i - 1] = std::cmp::max(right[n - i - 1], height[n - i - 1]);
        }

        let mut ans = 0;
        // Calculate the ans
        for i in 0..n {
            ans += std::cmp::min(left[i], right[i]) - height[i];
        }
        ans
    }
}

Rust
impl Solution {
    pub fn trap(height: Vec<i32>) -> i32 {
        let n = height.len();
        let mut left: Vec<i32> = vec![0; n];
        let mut right: Vec<i32> = vec![0; n];

        left[0] = height[0];
        right[n - 1] = height[n - 1];

        // Initialize the left & right vector
        for i in 1..n {
            left[i] = std::cmp::max(left[i - 1], height[i]);
            right[n - i - 1] = std::cmp::max(right[n - i - 1], height[n - i - 1]);
        }

        let mut ans = 0;
        // Calculate the ans
        for i in 0..n {
            ans += std::cmp::min(left[i], right[i]) - height[i];
        }
        ans
    }
}

Rust
```

```
[*] Stack — Pattern Solution (stored optimal)
Python
# Python solution using the two pointers approach
def trap(height):
    # Initialize left and right pointers and water accumulator
    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
    water = 0

    # Traverse the elevation map from both ends
    while left < right:
        # Update left_max and right_max as we encounter new bars
        if height[left] < height[right]:
            # left side is the determining side
            left_max = height[left]
            # update left_max since current height is higher
            water += left_max - height[left]
            left += 1 # move left pointer to the right
        else:
            # right side is the determining side
            right_max = height[right]
            # update right_max since current height is higher
            water += right_max - height[right]
            right -= 1 # move right pointer to the left
    return water

# Example usage:
print(trap([0,1,0,2,1,0,1,3,2,1,2,1])) # Expected output: 6

C++
```

```
// C++ solution using the two pointers approach
#include <iostream>
using namespace std;

class Solution {
public:
    int trap(vector<int>& height) {
        // Initialize pointers and variables to hold the max heights seen so far
        int left = 0, right = height.size() - 1;
        int leftMax = 0, rightMax = 0;
        int water = 0;

        // Use the two pointers to traverse the height array
        while (left < right) {
            if (height[left] < height[right]) {
                // If current left height is less than leftMax, accumulate trapped water
                if (height[left] > leftMax) {
                    leftMax = height[left]; // update leftMax if a new taller bar is found
                } else {
                    water += leftMax - height[left]; // add water trapped at left index
                    left++; // move left pointer to the right
                }
            } else {
                // If current right height is less than rightMax, accumulate trapped water
                if (height[right] > rightMax) {
                    rightMax = height[right]; // update rightMax if a new taller bar is found
                } else {
                    water += rightMax - height[right]; // add water trapped at right index
                    right--; // move right pointer to the left
                }
            }
        }
        return water;
    }
};

// Example usage:
// int main() {
//     vector<int> height = {0,1,0,2,1,0,1,3,2,1,2,1};
//     Solution sol;
//     int result = sol.trap(height); // Expected output: 6
//     return 0;
// }
```

#84 **HARD**

Largest Rectangle in Histogram

LeetCode · [SimplyEasi](#) · [WubCC](#) · [LeetCode](#) · [Editorial](#)

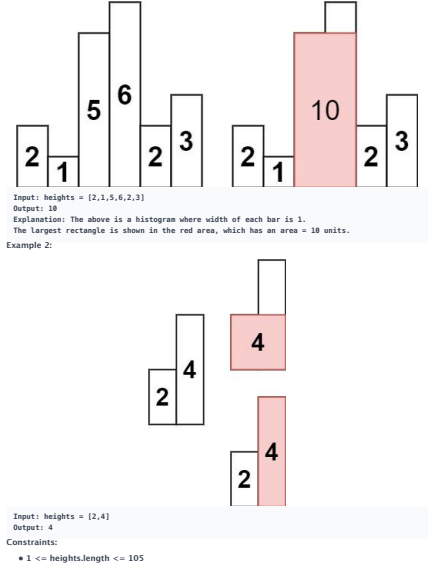
Array · Stack · [Monotonic Stack](#)

Uvic Grid 109

Problem:

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:



```
• 0 <= heights[i] <= 104
```

>> Brute Force (Stack) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def largestRectangleArea(self, heights):
        # Brute Force O(N^2) - fix left bar, expand right, track max height
        n = len(heights)
        res = 0
        for i in range(n):
            min_h = heights[i]
            for j in range(i, n):
                min_h = min(min_h, heights[j])
                res = max(res, min_h * (j - i + 1))
        return res
```

Python

Python

```
# Python solution
def largestRectangleArea(heights):
    # Append a zero to flush out the remaining bars in the stack
    heights.append(0)
    stack = [] # Stack to store indices of bars
    max_area = 0

    # Iterate over each index and bar height
    for i, h in enumerate(heights):
        # Process bars that are taller than the current bar
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()] # Height of the bar at the index popped from the stack
            # Calculate width: if stack is empty, width is the current index; otherwise, it's the
            # difference
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
            stack.append(i)

        return max_area

# Example usage:
if __name__ == "__main__":
    histogram = [2, 1, 5, 6, 2, 3]
    print(largestRectangleArea(histogram))
```

Python

Stack · LeetCode · By Pattern

— 2917 —

LeetCode

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        if not heights:
            return 0
        heights.append(-1) # Done for loop running
        stack = []
        ans = 0
        for i in range(0, len(heights)):
            while stack and heights[i] < heights[stack[-1]]:
                currentLowestBarIndex = stack.pop()
                h = heights[currentLowestBarIndex]
                # stack[-1] is where current bar met i-currentLowestBarIndex
                # because not working for the case in the stack.
                # eg. for [2,1,5,6,2,3], everytime bar popped
                # [2,1,5,6,2,3] h=2 left=1, right=3
                # [2,1,5,6,2,3] h=1 left=0, right=4
                # [2,1,5,6,2,3] h=5 left=2, right=5
                # [2,1,5,6,2,3] h=6 left=3, right=5
                # [2,1,5,6,2,3] h=2 left=1, right=5
                # [2,1,5,6,2,3] h=3 left=5, right=5
                w = i - stack[-1] - 1 if stack else i
                ans = max(ans, h * w)
                stack.append(i)

        heights.pop() # Restore, pop -1
        return ans
```

C++

C++

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        int ans = 0;
        stack<int> stack;

        for (int i = 0; i < heights.size(); ++i) {
            while (!stack.empty() && heights[stack.top()] > heights[i]) {
                const int h = heights[stack.top()];
                stack.pop();
                const int w = stack.empty() ? i : i - stack.top() - 1;
                ans = max(ans, h * w);
            }
            stack.push(i);
        }

        return ans;
    }
};
```

C++

Stack · LeetCode · By Pattern

— 2919 —

LeetCode

```
class Solution {
public:
    int largestRectangleArea(vector<int> heights) {
        int ans = 0;
        Deque<int> stack = new ArrayDeque<>();

        for (int i = 0; i < heights.length; ++i) {
            while (!stack.isEmpty() && heights[stack.peek()] > heights[i]) {
                final int h = heights[stack.pop()];
                final int w = stack.isEmpty() ? i : stack.peek() - 1;
                ans = Math.max(ans, h * w);
            }
            stack.push(i);
        }

        return ans;
    }
}
```

Java

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        int res = 0, n = heights.length;
        Deque<Integer> stk = new ArrayDeque<>();
        int[] left = new int[n];
        int[] right = new int[n];
        Arrays.fill(right, n);
        for (int i = 0; i < n; ++i) {
            while (!stk.isEmpty() && heights[stk.peek()] > heights[i]) {
                right[stk.pop()] = i;
            }
            left[i] = stk.isEmpty() ? -1 : stk.peek();
            stk.push(i);
        }
        for (int i = 0; i < n; ++i) {
            res = Math.max(res, heights[i] * (right[i] - left[i] - 1));
        }
        return res;
    }
}
```

Java

Stack · LeetCode · By Pattern

— 2921 —

LeetCode

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(heights) + 1):
            while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                h = heights[stack.pop()]
                w = i - stack[-1] - 1 if stack else i
                ans = max(ans, h * w)
                stack.append(i)

        return ans
```

Python

```
class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                right[stk[-1]] = i
                stk.pop()
            if stk:
                left[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))
```

Python

```
#####
##### h = (right[i] - left[i] - 1) #####
#####

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                right[stk[-1]] = i
                stk.pop()
            if stk:
                left[i] = stk[-1] # case as below: stk[-1] in 'n - stack[-1] - 1'
            stk.append(i)

        # valid for one element stack [1]
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))

#####
```

Stack · LeetCode · By Pattern

— 2918 —

LeetCode

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        int res = 0, n = heights.size();
        stack<int> stk;
        vector<int> left(n, -1);
        vector<int> right(n, n);
        for (int i = 0; i < n; ++i) {
            while (!stk.empty() && heights[stk.top()] >= heights[i]) {
                right[stk.top()] = i;
                stk.pop();
            }
            if (!stk.empty()) left[i] = stk.top();
            stk.push(i);
        }
        for (int i = 0; i < n; ++i) {
            res = max(res, heights[i] * (right[i] - left[i] - 1));
        }
        return res;
    }
};
```

C++

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        int res = 0, n = heights.size();
        stack<int> stk;
        vector<int> left(n, -1);
        vector<int> right(n, n);
        for (int i = 0; i < n; ++i) {
            while (!stk.empty() && heights[stk.top()] >= heights[i]) {
                right[stk.top()] = i;
                stk.pop();
            }
            if (!stk.empty()) left[i] = stk.top();
            stk.push(i);
        }
        for (int i = 0; i < n; ++i) {
            res = max(res, heights[i] * (right[i] - left[i] - 1));
        }
        return res;
    }
};
```

Java

Java

Stack · LeetCode · By Pattern

— 2920 —

LeetCode

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Solution {
    public int LargestRectangleArea(int[] heights) {
        var stack = new Stack<int>();
        var result = 0;
        var i = 0;
        while (i < heights.Length) {
            if (!stack.Any() || i < heights.Length && heights[stack.Peek()] < heights[i]) {
                stack.Push(i);
                ++i;
            }
            else {
                var previousIndex = stack.Pop();
                var area = heights[previousIndex] * (stack.Any() ? (i - stack.Peek() - 1) : i);
                result = Math.Max(result, area);
            }
        }
        return result;
    }
}
```

Java

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        int res = 0, n = heights.Length;
        Deque<Integer> stk = new ArrayDeque<>();
        int[] left = new int[n];
        int[] right = new int[n];
        Arrays.fill(right, n);
        for (int i = 0; i < n; ++i) {
            while (!stk.isEmpty() && heights[stk.peek()] >= heights[i]) {
                right[stk.pop()] = i;
            }
            left[i] = stk.isEmpty() ? -1 : stk.peek();
            stk.push(i);
        }
        for (int i = 0; i < n; ++i) {
            res = Math.max(res, heights[i] * (right[i] - left[i] - 1));
        }
        return res;
    }
}
```

Java

Stack · LeetCode · By Pattern

— 2922 —

LeetCode

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Solution {
    public int LargestRectangleArea(int[] height) {
        var stack = new Stack<int>();
        var result = 0;
        var i = 0;
        while (i < height.Length) {
            if ((i < height.Length && height[i] < height[stack.Count() - 1]) || (i == height.Length)) {
                stack.Push(i);
                i++;
            } else {
                var previousIndex = stack.Pop();
                var area = height[previousIndex] * (stack.Any() ? (i - stack.Peek() - 1) : i);
                result = Math.Max(result, area);
            }
        }
        return result;
    }
}
```

Go

```
func largestRectangleArea(heights []int) int {
    res, n := 0, len(heights)
    var stk []int
    left, right := make([]int, n), make([]int, n)
    for i := range right {
        right[i] = n
    }
    for i, h := range heights {
        for len(stk) > 0 && heights[stk[len(stk)-1]] >= h {
            right[stk[len(stk)-1]] = i
            stk = stk[:len(stk)-1]
        }
        if len(stk) > 0 {
            left[i] = stk[len(stk)-1]
        } else {
            left[i] = -1
        }
        stk = append(stk, i)
    }
    for i, h := range heights {
        res = max(res, h*(right[i]-left[i]-1))
    }
    return res
}
```

Go

Stack · LeetCode — By Pattern

— 2923 —

LeetCode

```
func largestRectangleArea(heights []int) int {
    res, n := 0, len(heights)
    var stk []int
    left, right := make([]int, n), make([]int, n)
    for i := range right {
        right[i] = n
    }
    for i, h := range heights {
        for len(stk) > 0 && heights[stk[len(stk)-1]] >= h {
            right[stk[len(stk)-1]] = i
            stk = stk[:len(stk)-1]
        }
        if len(stk) > 0 {
            left[i] = stk[len(stk)-1]
        } else {
            left[i] = -1
        }
        stk = append(stk, i)
    }
    for i, h := range heights {
        res = max(res, h*(right[i]-left[i]-1))
    }
    return res
}
```

Rust

Rust

```
impl Solution {
    // @lc=leetcode
    pub fn largest_rectangle_area(heights: Vec<i32>) -> i32 {
        let n = heights.len();
        let mut left = vec![0; n];
        let mut right = vec![n; n];
        let mut stack: Vec<(usize, i32)> = Vec::new();
        let mut res = 0;

        // Build left vector
        for (i, h) in heights.iter().enumerate() {
            while !stack.is_empty() && stack.last().unwrap().1 >= h {
                stack.pop();
            }
            if stack.is_empty() {
                left[i] = -1;
            } else {
                left[i] = stack.last().unwrap().0 as i32;
            }
            stack.push((i, h));
        }

        stack.clear();

        // Build right vector
    }
```

Stack · LeetCode — By Pattern

— 2924 —

LeetCode

```
for (i, h) in heights.iter().enumerate().rev() {
    while !stack.is_empty() && stack.last().unwrap().1 >= h {
        stack.pop();
    }
    if stack.is_empty() {
        right[i] = n as i32;
    } else {
        right[i] = stack.last().unwrap().0 as i32;
    }
    stack.push((i, h));
}

// Calculate the max area
for (i, h) in heights.iter().enumerate() {
    let = std::cmp::max(res, (right[i] - left[i] - 1) * h);
}

res
}
```

[*] Stack — Pattern Solution (matched from community answers)

Python

```
# Python solution
def largestRectangleArea(heights):
    # Return a zero to flush out the remaining bars in the stack
    heights.append(0)
    stack = [] # Stack to store indices of bars
    max_area = 0

    # Iterate over each index and bar height
    for i, h in enumerate(heights):
        # Process bars that are taller than the current bar
        while stack and heights[stack[-1]] >= h:
            height = heights[stack.pop()] # Height of the bar at the index popped from the stack
            # Calculate width: if stack is empty, width is the current index; otherwise, it's the
            # difference
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
            stack.append(i)

        return max_area

# Example usage:
if __name__ == "__main__":
    histogram = [2, 1, 5, 6, 2, 3]
    print(largestRectangleArea(histogram))
```

#224

HARD

Basic Calculator

LeetCode · LeetCode · MySQL · LeetCode · Editorial

Math · String · Stack · Recursion

Lists Grind 169

Stack · LeetCode — By Pattern

— 2925 —

LeetCode

Problem:

Given a string `s` representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: `s = "1 + 1"`
Output: `2`

Example 2:

Input: `s = " 2-1 + 2 "`
Output: `3`

Example 3:

Input: `s = "(1+(4+5+2)-3)+(6+8)"`
Output: `23`

Constraints:

- `s` consists of digits, `+`, `-`, `*`, `/`, `"(",` and `)`.
- `s` represents a valid expression.
- `+` is not used as a unary operation (i.e., `++1` and `-(2 + 3)` is invalid).
- `-` could be used as a unary operation (i.e., `-1` and `-(2 + 3)` is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

Python

Python

Stack · LeetCode — By Pattern

— 2926 —

LeetCode

```
# Python solution for the Basic Calculator problem
def calculate(s: str) -> int:
    # Stack to store pairs of (current result, current sign) before a '+'
    stack = []
    result = 0 # Running total
    number = 0 # Current number being processed
    sign = 1 # Current sign (1 or -1)

    # Iterate through each character in the string
    for char in s:
        if char.isdigit():
            # Build number digit by digit
            number = number * 10 + int(char)
        elif char in "+-":
            # Add the complete number with its sign to result
            result += sign * number
            # Reset number for the next potential number
            number = 0
            # Update the sign based on the operator
            sign = 1 if char == "+" else -1
        elif char == "(":
            # Push the current result and sign onto the stack
            stack.append((result, sign))
            # Reset result and sign for the new sub-expression
            result = 0
            sign = 1
        elif char == ")":
            # Complete the number before the parentheses close
            result += sign * number
            number = 0
            # The top of the stack is the sign before the parenthesis
            prev_sign = stack.pop()
            # The next value on the stack is the result before the parenthesis
            prev_result = stack.pop()
            # Combine the result of the previous expression with the sub-expression evaluated
            result = prev_result + prev_sign * result
            # This never occurs unless we're in a function that is not
            # Add any number left after the loop ends
            result += sign * number
            return result

# Example usage:
print(calculate("1 + 1")) # Expected output: 2
print(calculate(" 2-1 + 2 ")) # Expected output: 3
print(calculate("(1+(4+5+2)-3)+(6+8)")) # Expected output: 23
```

Python

Stack · LeetCode — By Pattern

— 2927 —

LeetCode

```
class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        num = 0
        sign = 1
        stack = [] # stack[-1]: the current environment's sign

        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '+':
                stack.append(sign)
                sign = 1
            elif c == '-':
                stack.append(-1)
                sign = -1
            elif c == '(':
                ans = sign * num
                sign = 1 if c == '+' else -1
                stack.append(sign)
                num = 0
            return ans + sign * num
```

Python

```
class Solution:
    def calculate(self, s: str) -> int:
        stk = []
        ans, sign = 0, 1
        i, n = 0, len(s)
        while i < n:
            if s[i].isdigit():
                x = 0
                j = i + 1
                while j <= n and s[j].isdigit():
                    x = x * 10 + int(s[j])
                    j += 1
                ans = sign * x
                i = j - 1
            elif s[i] == "+":
                sign = 1
            elif s[i] == "-":
                sign = -1
            elif s[i] == "(":
                stk.append(ans)
                stk.append(sign)
                ans, sign = 0, 1
            elif s[i] == ")":
                ans = stk.pop()
                ans = ans + sign * x
                i += 1
            return ans
```

Python

Stack · LeetCode — By Pattern

— 2928 —

LeetCode


```
# Python solution for the Basic Calculator problem
def calculate(s: str) -> int:
    # Stack to store pairs of (current result, current sign) before a '+'
    stack = []
    result = 0 # Running total
    number = 0 # Current number being processed
    sign = 1 # Current sign (1 or -1)

    # Iterate through each character in the string
    for char in s:
        if char.isdigit():
            # Build number digit by digit
            number = number * 10 + int(char)
        elif char in '+-':
            # Add the complete number with its sign to result
            result += sign * number
            # Reset number for the next potential number
            number = 0
            # Update the sign based on the operator
            sign = 1 if char == '+' else -1
        elif char == '(':
            # Push the current result and sign onto the stack
            stack.append((result, sign))
            # Reset result and sign for the new sub-expression
            result = 0
            sign = 1
        elif char == ')':
            # Complete the number before the parentheses close
            result += sign * number
            # The top of the stack is the sign before the parenthesis
            prev_sign = stack.pop()
            # The next value on the stack is the result before the parenthesis
            prev_result = stack.pop()
            # Combine the result of the previous expression with the sub-expression evaluated
            result = prev_result + prev_sign * result
            # Store current result and sign on stack if function is not
            stack.append((result, sign))

    # Add any number left after the loop ends
    result += sign * number
    return result

# Example usage
print(calculate("1+3")) # Expected output: 2
print(calculate("2-3+2")) # Expected output: 3
print(calculate("(2+(3*2)-3+(6*3+1))") # Expected output: 20
```

#239 **HARD**
Sliding Window Maximum
[LeetCode](#) • [SimplyEasiest](#) • [WuJCC](#) • [LeetCode](#) • [Editorial](#)
[Array](#) • [Queue](#) • [Sliding Window](#) • [Monotonic Queue](#)
[Listas Grind 189](#)

Problem:
You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.
Return the max sliding window.

Example 1:
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,5,6,7]
Explanation:
Window position Max

[1 3 -1] -> 3 5 6 7 3
1 [3 -1 -3] -> 5 6 7 3
1 3 [-1 -3 5] -> 6 7 5
1 3 -1 [-3 5 3] -> 6 7 5
1 3 -1 -3 [5 3 6] -> 7 6
1 3 -1 -3 5 [6 7] -> 7

Example 2:
Input: nums = [1], k = 1
Output: [1]

Constraints:
• 1 <= nums.length <= 105
• -104 <= nums[i] <= 104
• 1 <= k <= nums.length

>> Brute Force [Stack] Interview Prep
Python
Brute Force Approach
class Solution:
 def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
 # Brute Force O(N^2) - calculate without stack via inner loop
 n = len(nums)
 result = []
 for i in range(n):
 for j in range(i + 1, n):
 if nums[i] > nums[j]:
 result[i] = nums[i]
 break
 result.append(result[i])
 return result

Python
Python

```
from collections import deque

def maxSlidingWindow(nums, k):
    # Create a deque to store indices of elements in the current window
    dq = deque()
    result = []

    for i, num in enumerate(nums):
        # Remove indices from the front if they are out of the current window
        if dq and dq[0] == i - k:
            dq.popleft()

        # Remove elements from the back while the current number is larger
        # than the number at the indices stored in the deque
        while dq and nums[dq[-1]] < num:
            dq.pop()

        # Append the current index to the deque
        dq.append(i)

        # If we've hit the window size, append the current max to result
        if i >= k - 1:
            result.append(nums[dq[0]])

    return result

# Example usage:
# print(maxSlidingWindow([1,3,-1,-3,5,3,6,7], 3))

Python
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        ans = []
        maoq = collections.deque()

        for i, num in enumerate(nums):
            while maoq and maoq[-1] < num:
                maoq.pop()
            maoq.append(num)
            if i >= k and maoq[0] == i - k: # out-of-bounds
                maoq.popleft()
            if i >= k - 1:
                ans.append(maoq[0])

        return ans
```

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        for i, v in enumerate(nums):
            heapify(q)
            ans = []
            for j in range(i - 1, len(nums)):
                heappush(q, (nums[j], j))
                while q[0][1] < i - k:
                    heappop(q)
            ans.append(q[0][0])
        return ans

Python
from collections import deque

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, v in enumerate(nums):
            if q and i - k > 1 >= q[0]:
                q.popleft() # remove index if out of window left
                while q and nums[q[-1]] >= v: # v: v <= v, not < , to ensure the bigger index stored in deque
                    q.append(i)
                if i >= k - 1:
                    ans.append(nums[q[0]])
        return ans

Python
class Solution(object):
    def maxSlidingWindow(self, nums, k):
        # type: nums: List[int]
        # type: k: int
        # type: List[int]
        ans = []
        maoq = collections.deque()

        for i, num in enumerate(nums):
            while maoq and maoq[-1] < num:
                maoq.pop()
            maoq.append(num)
            if i >= k and maoq[0] == i - k: # out-of-bounds
                maoq.popleft()
            if i >= k - 1:
                ans.append(maoq[0])

        return ans
```

```
if k == 0:
    return []
ans = [0 for _ in range(len(nums) - k + 1)]
stack = collections.deque()
for i in range(k):
    while stack and nums[stack[-1]] < nums[i]:
        stack.pop()
    stack.append(i)
ans[i] = nums[stack[0]]
ids = 0
for i in range(k, len(nums)):
    ids += 1
    if stack and stack[0] == i - k:
        stack.popleft()
    while stack and nums[stack[-1]] < nums[i]:
        stack.pop()
    stack.append(i)
    ans[ids] = nums[stack[0]]
return ans

C++
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        vector<int> ans;
        deque<int> maoq;

        for (int i = 0; i < nums.size(); ++i) {
            while (!maoq.empty() && maoq.back() < nums[i])
                maoq.pop_back();
            maoq.push_back(nums[i]);
            if (i >= k && maoq[0] == i - k) maoq.pop_front(); // out-of-bounds
            if (i >= k - 1)
                ans.push_back(maoq.front());
        }

        return ans;
    }
};

C++
```

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        priority_queue<pair<int, int>> q;
        int n = nums.size();
        for (int i = 0; i < k - 1; ++i) {
            q.push((nums[i], -i));
        }
        vector<int> ans;
        for (int i = k - 1; i < n; ++i) {
            q.push((nums[i], -i));
            while ((q.top()).second >= i - k) {
                q.pop();
            }
            ans.emplace_back(q.top().first);
        }
        return ans;
    }
};

C++
// 01: https://leetcode.com/problems/sliding-window-maximum/
// Time: O(N)
// Space: O(N)
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& A, int k) {
        vector<int> ans;
        deque<int> q;
        for (int i = 0; i < A.size(); ++i) {
            if (q.size() >= k) q.pop_front();
            while (q.size() && A[q.back()] <= A[i]) q.pop_back();
            q.push_back(i);
            if (i >= k - 1) ans.push_back(A[q.front()]);
        }
        return ans;
    }
};

Java
Java
```

```

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int[] ans = new int[nums.length - k + 1];
        Deque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < nums.length; ++i) {
            while (!deque.isEmpty()) && deque.peekLast() < nums[i])
                deque.pollLast();
            deque.offerLast(nums[i]);
            if (i - k < deque.peekFirst()) // out-of-bounds
                deque.pollFirst();
            if (i == k - 1)
                ans[i - k + 1] = deque.peekFirst();
        }
        return ans;
    }
}

```

Java

```

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        PriorityQueue<Integer> q
            = new PriorityQueue<>((a, b) -> a[0] == b[0] ? a[1] - b[1] : b[0] - a[0]);
        int n = nums.length;
        for (int i = 0; i < n - k + 1; ++i) {
            q.offer(new int[] {nums[i], i});
        }
        int[] ans = new int[n - k + 1];
        for (int i = 0; i < n - k + 1; ++i) {
            q.offer(new int[] {nums[i], i});
            while (q.peek()[1] > i - k) {
                q.poll();
            }
            ans[i + 1] = q.peek()[0];
        }
        return ans;
    }
}

```

Java

```

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.PriorityQueue;

public class Sliding_Window_Maximum {

    public static void main(String[] args) {
        Sliding_Window_Maximum sol = new Sliding_Window_Maximum();
        Solution s = sol.new Solution();

        System.out.println(Arrays.toString(s.maxSlidingWindow(new int[] {1,3,-1,-3,5,3,6,7}, 3)));
    }

    /*
    Deque<Integer> dq = new ArrayDeque<>();
    dq.offer("a");
    dq.offer("b");
    dq.offer("c");
    System.out.println(dq.peek()); // a
    System.out.println(dq.pollFirst()); // a
    System.out.println(dq.peekLast()); // c
    */

    class Solution {
        public int[] maxSlidingWindow(int[] nums, int k) {

            if (nums == null || nums.length == 0) {
                return new int[0];
            }

            int n = nums.length;
            int[] result = new int[n - k + 1];
            int resultPointer = 0;

            // store index of nums array
            // q is decreasing values (its indexes)
            // eg. [1,3,5,4,2,3,1]; q will have [1,3,5], then [3,4,3], then [4,3,2], then [3,2,1]
            Deque<Integer> q = new ArrayDeque<>();

            for (int i = 0; i < nums.length; i++) {
                // remove index not in k-window
                while (!q.isEmpty()) && q.peek() < i - k + 1 { // peek() head of queue
                    q.poll();
                }

                // remove elements whose val >
                while (!q.isEmpty() && nums[i] > nums[q.peek()]) { // peekLast() last of queue
                    q.pollLast();
                }

                q.offer(i);
            }

```

```

// start from k-th element, there is a max for window
if (i == k - 1) {
    result[resultPointer] = nums[q.peek()];
    resultPointer++;
}

return result;
}
}

#####

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] ans = new int[n - k + 1];
        Deque<Integer> q = new ArrayDeque<>();
        for (int i = 0; i < n; ++i) {
            if (!q.isEmpty() && i - k + 1 > q.peekFirst()) {
                q.pollFirst();
            }
            while (!q.isEmpty() && nums[q.peekLast()] <= nums[i]) {
                q.pollLast();
            }
            q.offer(i);
            if (i == k - 1) {
                ans[i - k + 1] = nums[q.peekFirst()];
            }
        }
        return ans;
    }
}

```

Java

```

using System.Collections.Generic;

public class Solution {
    public int[] MaxSlidingWindow(int[] nums, int k) {
        if (nums.Length == 0) return new int[0];
        var result = new int[nums.Length - k + 1];
        var descOrderHums = new LinkedList<int>();
        for (var i = 0; i < nums.Length; ++i)
        {
            if (i == k && nums[i - k] == descOrderHums.First.Value)
            {
                descOrderHums.RemoveFirst();
            }
            while (descOrderHums.Count > 0 && nums[i] > descOrderHums.Last.Value)
            {
                descOrderHums.RemoveLast();
            }
            descOrderHums.AddLast(nums[i]);
            if (i == k - 1)
            {
                result[i - k + 1] = descOrderHums.First.Value;
            }
        }
        return result;
    }
}

```

Java

```

}

JavaScript
function maxSlidingWindow(nums, k) {
    const ans = [];
    const q = new Deque();
    for (let i = 0; i < nums.length; ++i) {
        if (!q.isEmpty() && i - q.front() == k) {
            q.popFront();
        }
        while (!q.isEmpty() && nums[q.back()] <= nums[i]) {
            q.popBack();
        }
        q.pushBack(i);
        if (i == k - 1) {
            ans.push(nums[q.front()]);
        }
    }
    return ans;
}

JavaScript
function maxSlidingWindow(nums, k) {
    let ans = [];
    let q = [];
    for (let i = 0; i < nums.length; ++i) {
        if (q[i] < i - k + 1 || q[i] < nums[i]) {
            q.shift();
        }
        while (q && nums[q.length - 1] <= nums[i]) {
            q.pop();
        }
        q.push(i);
        if (i == k - 1) {
            ans.push(nums[q[0]]);
        }
    }
    return ans;
}

TypeScript
TypeScript

```

```

function maxSlidingWindow(nums: number[], k: number): number[] {
    const ans: number[] = [];
    const q = new Deque();
    for (let i = 0; i < nums.length; ++i) {
        if (!q.isEmpty() && i - q.front() == k) {
            q.popFront();
        }
        while (!q.isEmpty() && nums[q.back()] <= nums[i]) {
            q.popBack();
        }
        q.pushBack(i);
        if (i == k - 1) {
            ans.push(nums[q.front()]);
        }
    }
    return ans;
}

Go
func maxSlidingWindow(nums []int, k int) []int {
    q := []int{}
    for i, v := range nums {
        if len(q) > 0 && i-k == q[0] {
            q = q[1:]
        }
        for len(q) > 0 && nums[q[len(q)-1]] <= v {
            q = q[:len(q)-1]
        }
        q = append(q, i)
        if i == k-1 {
            ans = append(ans, nums[q[0]])
        }
    }
    return ans
}

Rust

```

```

use std::collections::VecDeque;

impl Solution {
    pub fn max_sliding_window(nums: Vec<i32>, k: i32) -> Vec<i32> {
        let k = k as usize;
        let mut ans = Vec::new();
        let mut q: VecDeque<usize> = VecDeque::new();

        for i in 0..nums.len() {
            if let Some(qFront) = q.front() {
                if i - qFront == k {
                    q.pop_front();
                }
            }
            while let Some(qBack) = q.back() {
                if nums[qBack] <= nums[i] {
                    q.pop_back();
                } else {
                    break;
                }
            }
            q.push_back(i);
            if i == k - 1 {
                ans.push(nums[q.front().unwrap()]);
            }
        }
        ans
    }
}

Rust
use std::collections::VecDeque;

impl Solution {
    @inline(always)
    pub fn max_sliding_window(nums: Vec<i32>, k: i32) -> Vec<i32> {
        // The deque contains the index of nums
        let mut ans: VecDeque<isize> = VecDeque::new();
        let mut ans_vec: Vec<i32> = Vec::new();

        for i in 0..nums.len() {
            // Check the first element of queue, if it's out of bound
            if !q.is_empty() && i as isize - k + 1 > q.front().unwrap() as isize {
                q.pop_front();
            }

            // Pop back elements not until either the deque is empty
            // or the back element is greater than the current traversed element
            while !q.is_empty() && nums[q.back().unwrap()] <= nums[i] {
                q.pop_back();
            }

            // Push the current index in queue
            q.push_back(i);
            // Check if the condition is satisfied
            if i == (k - 1) as isize {
                ans_vec.push(nums[q.front().unwrap()]);
            }
        }
        ans_vec
    }
}

```

```
    }
    ans_val
}
}
```

[*] Stack — Pattern Solution (matched from community answers)

```
Python
from collections import deque

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, v in enumerate(nums):
            if q and i - k + 1 >= q[0]:
                q.popleft() # remove index if out of window left
            while q and nums[i-1] >= v: # "<=", not "<" , to ensure the bigger index stored in deque
                q.pop()
            q.append(i)
            if i == k - 1:
                ans.append(nums[q[0]])
                return ans

#####
class Solution(object):
    def maxSlidingWindow(self, nums, k):
        (type nums, List[int])
        (type k, int)
        (type List[int])
        ""

        if k == 0:
            return []
        ans = []
        for i in range(len(nums) - k + 1):
            stack = collections.deque()
            for i in range(k):
                while stack and nums[stack[-1]] < nums[i]:
                    stack.pop()
                stack.append(i)
            ans[i] = nums[stack[0]]
            del = 0
            for i in range(k, len(nums)):
                del += 1
                if stack and stack[0] == i - k:
                    stack.popleft()
                while stack and nums[stack[-1]] < nums[i]:
                    stack.pop()
                stack.append(i)
                ans[del] = nums[stack[0]]
            return ans
```

Max Stack

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

Stack · Design · Doubly-Linked List · Ordered Set

Little Denny Zhang

Problem:

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the MaxStack class:

- MaxStack() Initializes the stack object.
- void push(int x) Pushes element x onto the stack.
- int pop() Removes the element on top of the stack and returns it.
- int top() Gets the element on the top of the stack without removing it.
- int peekMax() Retrieves the maximum element in the stack without removing it.
- int popMax() Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the top-most one.

You must come up with a solution that supports O(1) for each top call and O(logn) for each other call.

Example 1:

```
Input
["MaxStack", "push", "push", "push", "top", "popMax", "top", "peekMax", "pop", "top"]
[[], [5], [2], [5], [1, 1, 1, 1, 1, 1, 1]]
Output
[null, null, null, null, 5, 5, 1, 5, 1, 5]
```

Explanation

```
MaxStack stk = new MaxStack();
stk.push(5); // [5] the top of the stack and the maximum number is 5.
stk.push(1); // [5, 1] the top of the stack is 1, but the maximum is 5.
stk.push(2); // [5, 1, 2] the top of the stack is 2, which is also the maximum, because it is the top most one.
stk.top(); // return 2, [5, 1, 2] the stack did not change.
stk.popMax(); // return 5, [5, 1] the stack is changed now, and the top is different from the max.
stk.top(); // return 1, [5, 1] the stack did not change.
stk.peekMax(); // return 5, [5, 1] the stack did not change.
stk.pop(); // return 1, [5] the top of the stack and the max element is now 5.
stk.top(); // return 5, [5] the stack did not change.
```

Constraints:

- -107 <= x <= 107
- At most 105 calls will be made to push, pop, top, peekMax, and popMax.
- There will be at least one element in the stack when pop, top, peekMax, or popMax is called.

Python

Python

```
# Python solution implementing the MaxStack data structure
# we use collections.deque for deque operation
# However, in Python we do not have a built-in balanced BST,
# one can use "sortedcontainers" or manually maintain a heap and dictionary.
# For clarity, we illustrate an approach using only a heap and dictionary.
# In production, one may use "sorteddict" from sortedcontainers.

from collections import defaultdict

class Node:
    def __init__(self, val):
        self.val = val
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        # Create dummy head and tail nodes
        self.head = Node(0)
        self.tail = Node(0)
        self.head.next = self.tail
        self.tail.prev = self.head

    def add(self, node):
        # Add node just before the tail
        node.prev = self.tail.prev
        node.next = self.tail
        self.tail.prev.next = node
        self.tail.prev = node

    def pop(self, node=None):
        # If no node is provided, pop from the end.
        if not node:
            node = self.tail.prev
        node.prev.next = node.next
        node.next.prev = node.prev
        return node

    def top(self):
        # Return the last node's value.
        return self.tail.prev.val

class MaxStack:
    def __init__(self):
        self.dll = DoublyLinkedList()
        # Dictionary mapping values to list of nodes
        self.valToNodes = defaultdict(list)
        # Sorted dict to maintain ordered set of keys.
        self.sortedKeys = []

        # Python solution implementing the MaxStack data structure
        # we use collections.deque for deque operation
        # However, in Python we do not have a built-in balanced BST,
        # one can use "sortedcontainers" or manually maintain a heap and dictionary.
        # For clarity, we illustrate an approach using only a heap and dictionary.
        # In production, one may use "sorteddict" from sortedcontainers.

        from collections import defaultdict

        class Node:
            def __init__(self, val):
                self.val = val
                self.prev = None
                self.next = None

            class DoublyLinkedList:
                def __init__(self):
                    # Create dummy head and tail nodes
                    self.head = Node(0)
                    self.tail = Node(0)
                    self.head.next = self.tail
                    self.tail.prev = self.head

                def add(self, node):
                    # Add node just before the tail
                    node.prev = self.tail.prev
                    node.next = self.tail
                    self.tail.prev.next = node
                    self.tail.prev = node

                def pop(self, node=None):
                    # If no node is provided, pop from the end.
                    if not node:
                        node = self.tail.prev
                    node.prev.next = node.next
                    node.next.prev = node.prev
                    return node

                def top(self):
                    # Return the last node's value.
                    return self.tail.prev.val

            class MaxStack:
                def __init__(self):
                    self.dll = DoublyLinkedList()
                    # Dictionary mapping values to list of nodes
                    self.valToNodes = defaultdict(list)
                    # Sorted dict to maintain ordered set of keys.
                    self.sortedKeys = []
```

```
def push(self, x: int) -> None:
    node = Node(x)
    self.dll.addLast(node)
    self.valToNodes[x].append(node)
    # Insert x into sortedKeys if new, otherwise maintain count.
    if len(self.valToNodes[x]) == 1:
        # binary search insert into sortedKeys
        lo, hi = 0, len(self.sortedKeys)
        while lo < hi:
            mid = (lo + hi) // 2
            if self.sortedKeys[mid] < x:
                lo = mid + 1
            else:
                hi = mid
        self.sortedKeys.insert(lo, x)

def pop(self) -> int:
    node = self.dll.pop()
    x = node.val
    self.valToNodes[x].pop()
    if not self.valToNodes[x]:
        # Remove x from sortedKeys
        self.sortedKeys.remove(x)
    return x

def top(self) -> int:
    return self.dll.top()

def peekMax(self) -> int:
    # The maxKey is the last element in sortedKeys.
```

Python

```
class MaxStack(object):
    def __init__(self):
        # Initialize your data structure here.
        self.stack = []
        self.max_stack = []
        # or the same thing as above
        # self.max_stack = [-math.inf]

    def push(self, x):
        (type x: int)
        (type: void)
        self.stack.append(x)
        self.max_stack.append(max(x, self.max_stack[-1]) if self.max_stack else x)

    def pop(self):
        (type: int)

        if len(self.stack) != 0:
            self.max_stack.pop(-1)

    def top(self):
        return self.stack[-1]

    def peekMax(self):
        return self.max_stack[-1]
```

```
        return self.stack.pop(-1)

    def top(self):
        (type: int)
        return self.stack[-1] if self.stack else None

    def peekMax(self):
        (type: int)
        if len(self.max_stack) != 0:
            return self.max_stack[-1]

    def popMax(self):
        (type: int)
        val = self.peekMax()
        buff = []
        while self.top() != val:
            # Remove node() use on both max_stack and stack
            buff.append(self.pop())
            self.pop()

        while len(buff) != 0:
            # Re-push back() no need to save buffer for max_stack
            self.push(buff.pop(-1))
        return val

#####
from sortedcontainers import SortedList

class Node:
    def __init__(self, val=0):
        self.val = val
        self.prev = UnionNode, None
        self.next = UnionNode, None

class DoublyLinkedList:
    def __init__(self):
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def append(self, val) -> Node:
        node = Node(val)
        node.next = self.tail
        node.prev = self.tail.prev
        self.tail.prev.next = node
        node.prev.next = node
        return node
```

```
C++
C++

class MaxStack {
public:
    void push(int x) {
        list.push_front(x);
        keyToIterators[x].push_back(list.begin());
    }

    int pop() {
        const int x = list.front();
        list.pop_front();
        auto it = keyToIterators[x];
        iterators.pop_back();
        if (iterators.empty())
            keyToIterators.erase(x);
        return x;
    }

    int top() {
        return list.front();
    }

    int peekMax() {
        return keyToIterators.begin()--first;
    }

    int popMax() {
        const int x = peekMax();
        auto it = list.begin();
        auto it = iterators.back();
        list.erase(it);
        iterators.pop_back();
        if (iterators.empty())
            keyToIterators.erase(keyToIterators.begin());
        return x;
    }

private:
    std::list<int> list;
    map<int, vector<std::list<int>::iterator>>, greater> keyToIterators;
};

Java
Java
```



```

class Node {
    public int val;
    public Node prev, next;

    public Node() {
    }

    public Node(int val) {
        this.val = val;
    }
}

class DoublyLinkedList {
    private final Node head = new Node();
    private final Node tail = new Node();

    public DoublyLinkedList() {
        head.next = tail;
        tail.prev = head;
    }

    public Node append(int val) {
        Node node = new Node(val);
        node.next = tail;
        node.prev = tail.prev;
        tail.prev = node;
        node.prev.next = node;
        return node;
    }

    public static Node remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
        return node;
    }

    public Node pop() {
        return remove(tail.prev);
    }

    public int peek() {
        return tail.prev.val;
    }
}

class MaxStack {
    private DoublyLinkedList sll = new DoublyLinkedList();
    private TreeMap<Integer, List<Node>> tm = new TreeMap<>();

    public MaxStack() {
    }
}

```

```

}

public void push(int x) {
    Node node = sll.append(x);
    tm.computeIfAbsent(x, k -> new ArrayList<>()).add(node);
}

public int pop() {
    Node node = sll.pop();
    List<Node> nodes = tm.get(node.val);
    int i = nodes.remove(nodes.size() - 1).val;
    if (nodes.isEmpty()) {
        tm.remove(node.val);
    }
    return x;
}

public int top() {
    return sll.peek();
}

public int peekNext() {
    return tm.lastKey();
}

public int popNext() {
    int x = peekNext();
    List<Node> nodes = tm.get(x);
    Node node = nodes.remove(nodes.size() - 1);
    if (nodes.isEmpty()) {
    }
}

```

[*] Stack — Pattern Solution (matched from community answers)

Python

```

class MaxStack(object):

    def __init__(self):
        """
        Initializing your data structure here.
        """
        self.stack = []
        self.max_stack = []
        # or the same trick is min-stack
        # self.min_stack = [-math.inf]

    def push(self, x):
        """
        :type x: int
        :rtype: void
        """
        self.stack.append(x)
        self.max_stack.append(max(x, self.max_stack[-1]) if self.max_stack else x)

    def pop(self):
        """
        :rtype: int
        """
        if len(self.stack) != 0:
            self.max_stack.pop(-1)
            return self.stack.pop(-1)

    def top(self):
        """
        :rtype: int
        """
        return self.stack[-1] if self.stack else None

    def peekMax(self):
        """
        :rtype: int
        """
        if len(self.max_stack) != 0:
            return self.max_stack[-1]

    def popMax(self):
        """
        :rtype: int
        """
        val = self.peekMax()
        buff = []
        while self.top() != val:
            # remove prev node in both max_stack and stack
            buff.append(self.pop())
            self.pop()

        self.stack.pop()

```

```

while len(buff) != 0:
    # remove prev(); do need to save buffer for max-stack
    self.push(buff.pop(-1))
    return val

#####

from sortedcontainers import SortedList

class Node:
    def __init__(self, val=0):
        self.val = val
        self.prev = Union[Node, None] = None
        self.next = Union[Node, None] = None

class DoublyLinkedList:
    def __init__(self):
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def append(self, val) -> Node:
        node = Node(val)
        node.next = self.tail
        node.prev = self.tail.prev
        self.tail.prev = node
        node.prev.next = node
        return node

```

#772

HARD

Basic Calculator III

LeetCode · Simple · Math · Recursion · Editorial

Math · String · Stack · Recursion

List: Premium 98

Problem:

Implement a basic calculator to evaluate a simple expression string.

The expression string contains only non-negative integers, '+', '-', '*', '/', parentheses, and open '(' and closing parentheses ')'. The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of [-2³¹, 2³¹ - 1].

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Example 1:

Input: s = "1+1"

Output: 2

Example 2:

Input: s = "6+4/2"

Output: 4

Example 3:

Input: s = "2+(5+5*2)/(6/2+8)"

Output: 21

Constraints:

- 1 <= s.length <= 10⁴
- s consists of digits, '+', '-', '*', '/', '(', ')', and space.
- s is a valid expression.

Python

```

def calculate(s: str) -> int:
    # Recursive helper function that processes the expression starting from index 0
    def helper(i):
        stack = []
        num = 0
        sign = '+'
        while i < len(s):
            char = s[i]
            # Build the current number if the character is a digit
            if char.isdigit():
                num = num * 10 + int(char)
            # If it is an operator or closing parenthesis, process the sign and number
            if char in '+-*/' or char == ')':
                if char == '(':
                    num, i = helper(i + 1)
                # Now we have a number, an operator or closing parenthesis, process the sign and number
                if (not char.isdigit() and char != '(') or i == len(s) - 1:
                    if sign == '+':
                        stack.append(num)
                    elif sign == '-':
                        stack.append(-num)
                    elif sign == '*':
                        prev = stack.pop()
                        stack.append(prev * num)
                    elif sign == '/':
                        prev = stack.pop()
                        stack.append(int(prev / num)) # Truncate toward zero
                    num = 0
                    sign = char
                # If a closing parenthesis is reached, and this level of recursion
                if char == ')':
                    return sum(stack), i
                i += 1
                return sum(stack), i
            result, i = helper(i)
        return result

    # Example test
    print(calculate("2+(5+5*2)/(6/2+8)"))

```

Python

```

class Solution:
    def calculate(self, s: str) -> int:
        num = 0
        ops = []

        def calc():
            b = num.pop()
            a = num.pop()
            op = ops.pop()
            if op == '+':
                num.append(a + b)
            elif op == '-':
                num.append(a - b)
            elif op == '*':
                num.append(a * b)
            else: # op == '/'
                num.append(int(a / b))

        def precedes(prev: str, curr: str) -> bool:
            # Returns True if the previous character is an operator and the priority of
            # the previous operator >= the priority of the current character (operator).
            if prev == '(':
                return False
            return prev in '+*/' or curr in '+-'

        return prev in '+*/' or curr in '+-'

        i = 0
        hasPrecedence = False
        while i < len(s):
            c = s[i]
            if c.isdigit():
                while i + 1 < len(s) and s[i + 1].isdigit():
                    num = num * 10 + int(s[i + 1])
                    i += 1
                num.append(int(num))
                hasPrecedence = True
            elif c == '(':
                ops.append('(')
                hasPrecedence = False
            elif c == ')':
                while ops[-1] != '(':
                    calc()
                ops.pop() # Pop '('
                if not hasPrecedence: # Handle input like "-3+(-1)"
                    num.append(0)
                while ops and precedes(ops[-1], c):

```

```

    calc()
    ops.append(c)
    i += 1

while ops:
    calc()

return num.pop()

```

Python

```

class Solution:
    def calculate(self, s: str) -> int:
        def dfs(i):
            num, sign = 0, "+"
            while i:
                c = s.popLeft()
                if c.isdigit():
                    num = num * 10 + int(c)
                if c == "+":
                    num = dfs(i)
                if c != "-"): or not q:
                    match sign:
                        case "+":
                            stk.append(num)
                        case "-":
                            stk.append(-num)
                        case "+":
                            stk.append(stk.pop() + num)
                        case "-":
                            stk.append(int(stk.pop()) - num)
                    num, sign = 0, c
                if c == "(":
                    break
            return num + stk

        return dfs(len(s))

```

Python

Stack · LeetCode - By Pattern

— 2959 —

LeetCode

```

# good one, based on solution of Basic Calculator II
class Solution: # 20, 100%
    def calculate(self, s: str) -> int:
        stack = []
        num = 0
        op = "+"
        i = 0
        while i < len(s):
            c = s[i]
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == "+":
                i = self.findClosing(i)
                num = self.calculate(stack)
                i = i + 1
            elif c == "-":
                i = self.findClosing(i)
                num = self.calculate(stack)
                i = i + 1
            elif c == "(":
                stack.append(num)
                num = 0
                op = "+"
            elif c == ")":
                stack.append(num)
                num = 0
                op = "+"
            else:
                stack[-1] = int(stack[-1] / num)
                op = c

        num = 0
        i = 1
        return sum(stack)

    def findClosing(self, i: str, i: int) -> int:
        level = 0
        while i < len(s):
            if s[i] == "(":
                level += 1
            elif s[i] == ")":
                level -= 1
            if level == 0:
                return i
            i += 1
        return i

#####

class Solution: # 100%
    def calculate(self, s: str) -> int:
        def evaluate_expression(stack):
            res = stack.pop() if stack else 0
            # Evaluate the expression till we get '(' or empty stack

```

Stack · LeetCode - By Pattern

— 2960 —

LeetCode

```

while stack and stack[-1] != '(':
    sign = stack.pop()
    if sign == "+":
        res += stack.pop()
    elif sign == "-":
        res -= stack.pop()
    elif sign == "*":
        res *= stack.pop()
    elif sign == "/":
        res /= stack.pop()
    return res

stack = [] # list of num and operator
n = len(s)
i = 0
while i < n:
    if s[i].isdigit():
        # get the complete number and push it to the stack
        j = i
        while j < n and s[j].isdigit():
            j += 1
        stack.append(int(s[i:j]))
        i = j - 1
    elif s[i] != "(":
        # Evaluate the expression within the parentheses
        res = evaluate_expression(stack)
        stack.pop() # Remove the opening '('
        stack.append(res)

    elif s[i] == "(":
        # Evaluate the expression within the parentheses
        res = evaluate_expression(stack)
        stack.pop() # Remove the opening '('
        stack.append(res)

```

C++

C++

Stack · LeetCode - By Pattern

— 2961 —

LeetCode

```

// 01: https://leetcode.com/problems/basic-calculator-iii/
// Time: O(N)
// Space: O(N)
class Solution {
    stack<long> num;
    stack<char> op;
    unordered_map<char, int> priority { '+', 1, '-', 1, '*', 2, '/', 2 };
    void eval() {
        long b = num.top(); num.pop();
        char c = op.top(); op.pop();
        switch (c) {
            case "+": num.top() += b; break;
            case "-": num.top() -= b; break;
            case "*": num.top() *= b; break;
            case "/": num.top() /= b; break;
        }
    }
public:
    int calculate(string s) {
        for (int i = 0; i < s.size(); i++) {
            if (s[i] == '(') continue;
            if (isDigit(s[i])) {
                long b = 0;
                while (i < s.size() && isDigit(s[i])) b = b * 10 + s[i] - '0';
                num.push(b);
            } else if (s[i] == '(') op.push(s[i]);
            else if (s[i] == ')') {
                while (op.top() != '(') eval();
                op.pop();
            } else {
                while (op.size() && op.top() != '(' && priority[op.top()] > priority[s[i]]) eval();
                op.push(s[i]);
            }
        }
        while (op.size()) eval();
        return num.top();
    }
};

```

Java

Java

Stack · LeetCode - By Pattern

— 2962 —

LeetCode

```

class Solution {
public:
    int calculate(string s) {
        Deque<Integer> num = new ArrayDeque<>();
        Deque<Character> ops = new ArrayDeque<>();
        boolean hasPrecedence = false;

        for (int i = 0; i < s.length(); ++i) {
            final char c = s.charAt(i);
            if (Character.isDigit(c)) {
                int num = 0;
                while (i + 1 < s.length() && Character.isDigit(s.charAt(i + 1)))
                    num = num * 10 + (s.charAt(i + 1) - '0');
                num.push(num);
                hasPrecedence = true;
            } else if (c == '(') {
                ops.push('(');
                hasPrecedence = false;
            } else if (c == ')') {
                while (ops.peek() != '(')
                    calc(num, ops);
                ops.pop(); // pop '('
            } else if (c == '+' || c == '-' || c == '*' || c == '/') {
                if (!hasPrecedence)
                    num.push(0);
                while (num.isEmpty() && precedence(ops.peek(), c))
                    calc(num, ops);
                ops.push(c);
            }
        }
        while (!ops.isEmpty())
            calc(num, ops);
        return num.peek();
    }

    private void calc(Deque<Integer> num, Deque<Character> ops) {
        final int b = num.pop();
        final int a = num.pop();
        final char op = ops.pop();
        if (op == "+")
            num.push(a + b);
        else if (op == "-")
            num.push(a - b);
        else if (op == "*")
            num.push(a * b);
        else // op == "/"
            num.push(a / b);
    }

    // Returns true if the previous character is a operator and the priority of
    // the previous operator == the priority of the current character (operator).
    private boolean precedence(char prev, char curr) {
        if (prev == "(")
            return false;
        return prev == "+" || prev == "-" || curr == "+" || curr == "-" ||
    }
}

```

Stack · LeetCode - By Pattern

— 2963 —

LeetCode

```

Java
public class Basic_Calculator_III {
    // ref: https://leetcode.com/problems/basic-calculator-iii/
    class Solution {
        public int calculate(String s) {
            if (s == null || s.length() == 0)
                return 0;

            // remove leading and trailing spaces and white spaces.
            s = s.trim().replaceAll("\\s+", "");

            if (s == null || s.length() == 0)
                return 0;

            Stack<Character> optStack = new Stack<>();
            Stack<Integer> numStack = new Stack<>();

            int i = 0;
            while (i < s.length()) {
                if (Character.isDigit(s.charAt(i))) {
                    int num = 0;
                    while (i < s.length() && Character.isDigit(s.charAt(i + 1))) {
                        num = num * 10 + Character.getNumericValue(s.charAt(i + 1));
                        i++;
                    }
                    numStack.push(num);
                } else {
                    char op = s.charAt(i);
                    if (optStack.isEmpty()) {
                        numStack.push(0);
                    } else if (op == '+' || op == '-') {
                        char top = optStack.peek();
                        if (top == '+' || top == '-') {
                            numStack.push(top);
                        } else {
                            calculate(numStack, optStack);
                        }
                    } else if (op == '*' || op == '/') {
                        char top = optStack.peek();
                        if (top == '*' || top == '/') {
                            optStack.push(top);
                        } else {
                            calculate(numStack, optStack);
                        }
                    } else if (top == '+' || top == '-') {
                        calculate(numStack, optStack);
                    }
                }
            }
        }
    }
}

```

Stack · LeetCode - By Pattern

— 2964 —

LeetCode

```

        } else if (top == '(' || top == '[') {
            opStack.push(op);
            invs++;
        }
    }
    } else if (top == '(') {
        opStack.push(op);
        invs++;
    }
    } else if (top == '[') {
        while (opStack.peek() != '[') {
            calculate(numStack, opStack);
        }
        opStack.pop();
        invs--;
    }
    }
}

while (!opStack.isEmpty()) {
    calculate(numStack, opStack);
}

return numStack.peek();
}

private void calculate(Stack<Integer> numStack, Stack<Character> opStack) {
    int num0 = numStack.pop();
    int num1 = numStack.pop();
    char op = opStack.pop();
}

```

[*] Stack — Pattern Solution (matched from community answers)

Python

Stack · LeetCodeMastery — By Pattern

— 2965 —

LeetCodeMastery

```

def calculate(s: str) -> int:
    # Recursive helper function that processes the expression starting from index i
    def helper(i):
        stack = []
        num = 0
        sign = '+'
        while i < len(s):
            char = s[i]
            # Build the current number if the character is a digit
            if char.isdigit():
                num = num * 10 + int(char)
            elif char == '+':
                # If '+' is encountered, evaluate the subexpression recursively
                num, i = helper(i + 1)
            elif char == '-':
                # When encountering an operator or closing parenthesis, process the sign and number
                if (not char.isdigit() and char != '(') or i == len(s) - 1:
                    if sign == '+':
                        stack.append(num)
                    elif sign == '-':
                        stack.append(-num)
                    else:
                        stack.append(num)
                    num = 0
                    sign = char
                # If a closing parenthesis is reached, and this level of recursion
                if char == ')':
                    return sum(stack), i + 1
            else:
                # If not char.isdigit() and char != '(' or i == len(s) - 1:
                if sign == '+':
                    stack.append(num)
                elif sign == '-':
                    stack.append(-num)
                else:
                    stack.append(num)
                num = 0
                sign = char
            i += 1
        return sum(stack), i

    result, _ = helper(0)
    return result

# Example test
print(calculate("2*(5+2)*(8+2/4)"))

```

#895

HARD

Maximum Frequency Stack

LeetCode · SimplifyStack · MultiCC · LeetCode · Editorial

Hash Table · Stack · Design

Lists Grind 169

Problem:

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the FreqStack class:

- FreqStack() constructs an empty frequency stack.
- void push(int val) pushes an integer val onto the top of the stack.

Stack · LeetCodeMastery — By Pattern

— 2966 —

LeetCodeMastery

- int pop() removes and returns the most frequent element in the stack. If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

```

Input
["FreqStack", "push", "push", "push", "push", "push", "pop", "pop", "pop", "pop"]
[[], [5], [7], [5], [7], [4], [5], [1], [1], [1], [1]]
Output
[null, null, null, null, null, null, 5, 7, 5, 4]

```

Explanation
 FreqStack freqStack = new FreqStack();
 freqStack.push(5); // The stack is [5]
 freqStack.push(7); // The stack is [5,7]
 freqStack.push(5); // The stack is [5,7,5]
 freqStack.push(7); // The stack is [5,7,5,7]
 freqStack.push(4); // The stack is [5,7,5,7,4]
 freqStack.push(); // The stack is [5,7,5,7,4,5]
 freqStack.pop(); // return 5, as 5 is the most frequent. The stack becomes [5,7,5,7,4].
 freqStack.pop(); // return 7, as 5 and 7 is the most frequent, but 7 is closest to the top.
 The stack becomes [5,7,5,4].
 freqStack.pop(); // return 5, as 5 is the most frequent. The stack becomes [5,7,4].
 freqStack.pop(); // return 4, as 4, 5 and 7 is the most frequent, but 4 is closest to the top. The stack becomes [5,7].

Constraints:

- 0 <= val <= 100
- At most 2 * 10⁴ calls will be made to push and pop.
- It is guaranteed that there will be at least one element in the stack before calling pop.

Python

Python

Stack · LeetCodeMastery — By Pattern

— 2967 —

LeetCodeMastery

```

# Python Implementation of FreqStack

class FreqStack:
    def __init__(self):
        # Dictionary mapping value to its frequency
        self.freq = {}
        # Dictionary mapping frequency to a list (stack) of values
        self.group = {}
        # Variable tracking the current maximum frequency in the stack
        self.maxfreq = 0

    def push(self, val: int) -> None:
        # Update frequency count for the value
        f = self.freq.get(val, 0) + 1
        self.freq[val] = f
        # Update the maximum frequency if necessary
        if f > self.maxfreq:
            self.maxfreq = f
        # Append the value to the appropriate frequency group
        if f not in self.group:
            self.group[f] = []
        self.group[f].append(val)

    def pop(self) -> int:
        # Remove the value from the stack of the current maximum frequency
        val = self.group[self.maxfreq].pop()
        # Decrease the frequency count for the value
        self.freq[val] -= 1
        # If no more elements have the current max frequency, decrease maxfreq
        if not self.group[self.maxfreq]:
            self.maxfreq -= 1
        return val

```

Python

```

class FreqStack:
    def __init__(self):
        self.maxfreq = 0
        self.count = collections.Counter()
        self.countToStack = collections.defaultdict(list)

    def push(self, val: int) -> None:
        self.count[val] += 1
        self.countToStack[self.count[val]].append(val)
        self.maxfreq = max(self.maxfreq, self.count[val])

    def pop(self) -> int:
        count = self.count[self.maxfreq]
        val = self.countToStack[self.maxfreq].pop()
        self.count[val] -= 1
        if (countToStack[self.maxfreq].empty()):
            self.maxfreq -= 1
        return val

```

Python

Stack · LeetCodeMastery — By Pattern

— 2968 —

LeetCodeMastery

```

class FreqStack:
    def __init__(self):
        self.int = defaultdict(int)
        self.q = []
        self.ts = 0

    def push(self, val: int) -> None:
        self.ts += 1
        self.int[val] += 1
        heapq.heappush(self.q, (-self.int[val], -self.ts, val))

    def pop(self) -> int:
        val = heapq.heappop(self.q)[2]
        self.int[val] -= 1
        return val

# Your FreqStack object will be instantiated and called as such:
# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()

```

Python

```

class FreqStack:
    def __init__(self):
        self.int = defaultdict(int)
        self.d = defaultdict(list)
        self.mx = 0

    def push(self, val: int) -> None:
        self.int[val] += 1
        self.d[self.int[val]].append(val)
        self.mx = max(self.mx, self.int[val])

    def pop(self) -> int:
        val = self.d[self.mx].pop()
        self.int[val] -= 1
        if not self.d[self.mx]:
            self.mx -= 1
        return val

# Your FreqStack object will be instantiated and called as such:
# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()

```

C++

C++

Stack · LeetCodeMastery — By Pattern

— 2969 —

LeetCodeMastery

```

class FreqStack {
public:
    void push(int val) {
        countToStack[+count[val]].push(val);
        maxFreq = max(maxFreq, count[val]);
    }

    int pop() {
        const int val = countToStack[maxFreq].top();
        countToStack[maxFreq].pop();
        --count[val];
        if (countToStack[maxFreq].empty())
            --maxFreq;
        return val;
    }

private:
    unordered_map<int, int> count;
    unordered_map<int, stack<int>> countToStack;
};

```

C++

```

class FreqStack {
public:
    FreqStack() {}

    void push(int val) {
        ++cnt[val];
        q.emplace(cnt[val], val);
    }

    int pop() {
        auto [c, v, val] = q.top();
        q.pop();
        --cnt[val];
        return val;
    }

private:
    unordered_map<int, int> cnt;
    priority_queue<tuple<int, int, int>> q;
    int ts = 0;
};

```

```

//
// Your FreqStack object will be instantiated and called as such:
// FreqStack obj = new FreqStack();
// obj.push(val);
// int param_2 = obj.pop();
//

```

C++

Stack · LeetCodeMastery — By Pattern

— 2970 —

LeetCodeMastery

```

class FreqStack {
public:
    FreqStack() {
    }

    void push(int val) {
        ++cnt[val];
        d[cnt[val]].push(val);
        mx = max(mx, cnt[val]);
    }

    int pop() {
        int val = d[mx].top();
        --cnt[val];
        d[mx].pop();
        if (d[mx].empty()) --mx;
        return val;
    }

private:
    unordered_map<int, int> cnt;
    unordered_map<int, stack<int>> d;
    int mx = 0;
};

/*
 * Your FreqStack object will be instantiated and called as such:
 * FreqStack obj = new FreqStack();
 * obj.push(val);
 * int param_2 = obj.pop();
 */

```

Java

Java

```

class FreqStack {
public void push(int val) {
    countMap.put(val, i, Integer::sum);
    countToStack.putIfAbsent(count.get(val), new ArrayDeque<>());
    countToStack.get(count.get(val)).push(val);
    maxFreq = Math.max(maxFreq, count.get(val));
}

public int pop() {
    final int val = countToStack.get(maxFreq).pop();
    countMap.put(val, Integer::sum);
    if (countToStack.get(maxFreq).isEmpty())
        --maxFreq;
    return val;
}

private int maxFreq = 0;
private Map<Integer, Integer> count = new HashMap<>();
private Map<Integer, Deque<Integer>> countToStack = new HashMap<>();
}

```

Java

```

class FreqStack {
    private Map<Integer, Integer> cnt = new HashMap<>();
    private PriorityQueue<int[]> q =
        new PriorityQueue<(a, b) -> a[0] == b[0] ? b[1] - a[1] : b[0] - a[0]);
    private int ts;

    public FreqStack() {
    }

    public void push(int val) {
        cnt.put(val, cnt.getOrDefault(val, 0) + 1);
        q.offer(new int[] {cnt.get(val), ++ts, val});
    }

    public int pop() {
        int val = q.poll()[2];
        cnt.put(val, cnt.get(val) - 1);
        return val;
    }
}

/*
 * Your FreqStack object will be instantiated and called as such:
 * FreqStack obj = new FreqStack();
 * obj.push(val);
 * int param_2 = obj.pop();
 */

```

Java

```

class FreqStack {
    private Map<Integer, Integer> cnt = new HashMap<>();
    private Map<Integer, Deque<Integer>> d = new HashMap<>();
    private int mx;

    public FreqStack() {
    }

    public void push(int val) {
        cnt.put(val, cnt.getOrDefault(val, 0) + 1);
        int t = cnt.get(val);
        d.computeIfAbsent(t, k -> new ArrayDeque<>()).push(val);
        mx = Math.max(mx, t);
    }

    public int pop() {
        int val = d.get(mx).pop();
        cnt.put(val, cnt.get(val) - 1);
        if (d.get(mx).isEmpty()) {
            --mx;
        }
        return val;
    }
}

```

Java

```

/*
 * Your FreqStack object will be instantiated and called as such:
 * FreqStack obj = new FreqStack();
 * obj.push(val);
 * int param_2 = obj.pop();
 */

```

Go

Go

```

type FreqStack struct {
    cnt map[int]int
    d map[int][]int
    mx int
}

func Constructor() FreqStack {
    return FreqStack{cnt:map[int]int{}, map[int][]int{0}, 0}
}

func (this *FreqStack) Push(val int) {
    this.cnt[val]++
    this.d[this.cnt[val]] = append(this.d[this.cnt[val]], val)
    this.mx = max(this.mx, this.cnt[val])
}

func (this *FreqStack) Pop() int {
    val := this.d[this.mx][len(this.d[this.mx])-1]
    this.d[this.mx] = this.d[this.mx][:len(this.d[this.mx])-1]
    this.cnt[val]--
    if len(this.d[this.mx]) == 0 {
        this.mx--
    }
    return val
}

/*
 * Your FreqStack object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Push(val);
 * param_2 := obj.Pop();
 */

```

Go

```

type FreqStack struct {
    cnt map[int]int
    d map[int][]int
    mx int
}

func Constructor() FreqStack {
    return FreqStack{cnt:map[int]int{}, map[int][]int{0}, 0}
}

func (this *FreqStack) Push(val int) {
    this.cnt[val]++
    this.d[this.cnt[val]] = append(this.d[this.cnt[val]], val)
    this.mx = max(this.mx, this.cnt[val])
}

func (this *FreqStack) Pop() int {
    val := this.d[this.mx][len(this.d[this.mx])-1]
    this.d[this.mx] = this.d[this.mx][:len(this.d[this.mx])-1]
    this.cnt[val]--
    if len(this.d[this.mx]) == 0 {
        this.mx--
    }
    return val
}

/*
 * Your FreqStack object will be instantiated and called as such:
 * obj := Constructor();
 * obj.Push(val);
 * param_2 := obj.Pop();
 */

```

[*] Stack — Pattern Solution (stored optimal)

Python

```

# Python implementation of FreqStack

class FreqStack:
    def __init__(self):
        # Dictionary mapping value to its frequency
        self.freq = {}
        # Dictionary mapping frequency to a list (stack) of values
        self.group = {}
        # Variable tracking the current maximum frequency in the stack
        self.maxfreq = 0

    def push(self, val: int) -> None:
        # Update frequency count for the value
        f = self.freq.get(val, 0) + 1
        self.freq[val] = f
        # Update the maximum frequency if necessary
        self.maxfreq = f
        # Append the value to the appropriate frequency group
        if f not in self.group:
            self.group[f] = []
        self.group[f].append(val)

    def pop(self) -> int:
        # Remove the value from the stack of the current maximum frequency

        val = self.group[self.maxfreq].pop()
        # Decrease the frequency count for the value
        self.freq[val] -= 1
        # If no more elements have the current max frequency, decrease maxfreq
        if not self.group[self.maxfreq]:
            self.maxfreq -= 1
        return val

```

C++

```

// C++ implementation of FreqStack

#include <unordered_map>
#include <vector>
using namespace std;

class FreqStack {
public:
    // Constructor initializing max frequency to 0
    FreqStack() : maxFreq(0) {}

    // Push method
    void push(int val) {
        // Increase the occurrence count of val
        int f = ++freq[val];
        // Update max frequency if this value's frequency is greater
        if (f > maxFreq)
            maxFreq = f;
        // Append val to the corresponding frequency group
        group[f].push_back(val);
    }

    // Pop method
    int pop() {
        // Get the most frequent element from the top of the stack for max frequency

        int val = group[maxFreq].back();
        group[maxFreq].pop_back();
        // Decrease the frequency count of the popped value
        freq[val]--;
        // If this group is now empty, decrease the max frequency count
        if (group[maxFreq].empty())
            maxFreq--;
        return val;
    }

private:
    // Mapping from value to frequency count
    unordered_map<int, int> freq;
    // Mapping from frequency to a vector (acting as a stack) of values
    unordered_map<int, vector<int>> group;
    // Current maximum frequency in the stack
    int maxFreq;
};

```

Python

Quick Review — Stack 25 questions · insights · complexity · solution

#20 Valid Parentheses [Easy]

Key Insights

- The problem can be solved using a stack data structure.
- When encountering an opening bracket, push it onto the stack.
- When encountering a closing bracket, check if the top of the stack is its matching opening bracket.
- If the stack is empty before finding a match or if there is a mismatch, the string is invalid.
- At the end, the string is valid only if the stack is empty, meaning all brackets were properly matched.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, since we process each character once.
Space Complexity: $O(n)$ in the worst case for the stack when all characters are opening brackets.

Solution

We use a stack to check for valid parentheses. The algorithm iterates through the string, pushing opening brackets onto the stack and popping them when a corresponding closing bracket is encountered. A mapping of closing to opening brackets helps in validating the matching pairs. By ensuring that the stack is empty at the end, we confirm that every opening bracket was properly closed.

#232 Implement Queue using Stacks [Easy]

Key Insights

- Use two stacks:
- One (input stack) for enqueue operations.
- The other (output stack) for dequeue operations.
- When performing pop or peek, if the output stack is empty, transfer all elements from the input stack to the output stack. This reversal makes the oldest element accessible.
- Each element is moved at most once between the stacks, ensuring an amortized $O(1)$ cost per operation.
- The space complexity is $O(n)$ where n is the number of elements in the queue.

Space & Time

Time Complexity: Amortized $O(1)$ per operation
Space Complexity: $O(n)$

Solution

We maintain two stacks: an input stack for pushing new elements and an output stack for popping or peeking the front element of the queue. When we need to access the front of the queue (either during a pop or peek), we check if the output stack is empty. If it is, we transfer all elements from the input stack to the output stack. This reversal of order allows us to simulate the FIFO nature of the queue. When performing a push, the element is simply added to the input stack.

#844 Backspace String Compare [Easy]

Key Insights

- '#' represents a backspace operation that deletes the previous character.
- A direct simulation using a stack can reconstruct the final string.
- A two-pointer approach allows comparison from the end of the strings while skipping backspace characters.
- The two-pointer technique can achieve $O(n)$ time and $O(1)$ space by processing the strings in reverse.

Space & Time

Stack · LeetCodeMastery — By Pattern

— 297 —

LeetCodeMastery

- A common approach is to use two stacks: one regular stack for all elements and a second stack to keep track of the minimum values.
- When pushing a new element, push it onto the main stack and also update the min stack if the element is less than or equal to the current minimum.
- When popping, if the popped element is equal to the top of the min stack, pop from the min stack as well.
- This design guarantees that all operations (push, pop, top, getMin) run in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per operation (push, pop, top, getMin)
Space Complexity: $O(n)$, where n is the number of elements in the stack, since an additional stack is used to keep track of minima.

Solution

The solution involves using two stacks:

- Main Stack: Stores all the elements.
- Min Stack: Stores the minimum elements encountered so far.

When an element is pushed, we compare it with the top of the min stack. If it is smaller than or equal to the current minimum, we also push it onto the min stack. For the pop operation, if the element being popped is the same as the top of the min stack, we pop from the min stack as well. This approach ensures that the getMin operation always returns the top element of the min stack, which is the current minimum in constant time.

#173 Binary Search Tree Iterator [Medium]

Key Insights

- Use an iterative in-order traversal strategy with a stack.
- In-order traversal for BST yields sorted order.
- Pre-load the stack with all left descendants from the root.
- After returning a node, process its right subtree by pushing its left branch.
- This method maintains an average time complexity of $O(1)$ per operation and uses $O(h)$ memory, where h is the tree height.

Space & Time

Time Complexity: $O(1)$ on average per operation (amortized)
Space Complexity: $O(h)$, where h is the height of the tree

Solution

We use a stack to simulate an in-order traversal. The process involves initially pushing all left-child nodes from the root into the stack. For each call to next(), we pop the top element (the current smallest node) and then push all the left descendants of the node's right child. This ensures that the next smallest element is always at the top of the stack. The hasNext() function then simply checks if the stack is non-empty.

#227 Basic Calculator II [Medium]

Key Insights

- The expression can be processed in one pass while using a stack to handle operator precedence.
- When encountering '*' or '/' operators, the current number is directly added or subtracted (as a positive or negative value).
- For '+' and '-' operators, compute immediately with the last number from the stack and update it.
- Handle spaces and multi-digit numbers by building the number while iterating over the string.
- Division truncation toward zero must be carefully applied, which aligns with typical integer division in most languages.

Space & Time

Time Complexity: $O(n)$ where n is the length of the expression, since the algorithm processes each character once.
Space Complexity: $O(n)$ in the worst-case scenario when all numbers are stored in the stack.

Stack · LeetCodeMastery — By Pattern

— 2979 —

LeetCodeMastery

#439 Ternary Expression Parser [Medium]

Key Insights

- The ternary operator groups right-to-left. This property can be exploited by processing the string from the end to the beginning.
- Using a stack allows us to "collapse" nested ternary expressions as soon as their three components (condition, true branch, false branch) are available.
- An iterative approach is efficient and avoids building an explicit syntax tree for the expression.
- The condition is not stored on the stack; instead, it is encountered immediately preceding the '?' operator when traversing backwards.

Space & Time

Time Complexity: $O(n)$, where n is the length of the expression, as we process each character exactly once.
Space Complexity: $O(n)$, due to the use of a stack for storing characters during evaluation.

Solution

The solution uses an iterative approach with a stack. We traverse the expression string from right to left. For every character:

- If it is a digit, 'T', or 'F' (and not a '?' or part of a pending operator), we push it onto the stack.
- When we encounter a '?' character, we know that the next available items on the stack represent the true and false results of the ternary expression. We then skip the ':' separator that comes between them.
- We then use the condition (the character immediately preceding the '?') to choose which result to push back on the stack. This method "collapses" each ternary expression into a single result, and by the time we finish traversing the string, the stack contains the final evaluated result.

#484 Find Permutation [Medium]

Key Insights

- The answer is a permutation of numbers from 1 to n , where $n = \text{len}(s) + 1$.
- A common strategy is to use a stack to reverse the segments when a decrease (D) is encountered.
- When an 'I' is encountered (or at the end), pop all numbers from the stack to output them in reverse order, ensuring the smallest lexicographical order.
- This greedy approach guarantees that as soon as a rising pattern is detected, we "flush" the pending decreases correctly.

Space & Time

Time Complexity: $O(n)$ · Each element is pushed and popped exactly once.
Space Complexity: $O(n)$ · For the stack and the output array.

Solution

We use a greedy approach with a stack. Iterate from 0 to n (where $n = \text{len}(s) + 1$):

- Push the current number ($i+1$) onto the stack.
- If the current character is 'I' or we have reached the end of the string, pop all elements from the stack and append them to the result.
- By doing so, segments corresponding to series of 'D's are reversed, while 'I's trigger the flush, ensuring that the final permutation is the lexicographically smallest.

#735 Asteroid Collision [Medium]

Key Insights

- Use a stack to keep track of the surviving asteroids.
- Only a collision happens when a right-moving asteroid (positive) is on the stack and a left-moving asteroid (negative) is encountered.
- Compare the absolute values to decide if one or both asteroids explode.
- Continue processing collisions until no further collisions are possible with the current asteroid.

Space & Time

Stack · LeetCodeMastery — By Pattern

— 2981 —

LeetCodeMastery

Time Complexity: $O(n)$ where n is the length of the longer string.

Space Complexity: $O(1)$ using the two-pointer strategy ($O(n)$ if using stacks).

Solution

The solution uses two pointers starting from the end of each string. For each string, maintain a skip counter to account for backspace characters. As you iterate backwards:

- If a '#' is encountered, increment the skip counter.
- If a normal character is encountered while the skip counter is positive, skip that character by decrementing the counter.
- Once a valid character is found, compare the characters from both strings. If all corresponding characters match and both pointers finish processing at the same time, the strings are considered equal.

#143 Reorder List [Medium]

Key Insights

- Find the middle of the linked list using the fast and slow pointer technique.
- Reverse the second half of the list.
- Merge the two halves, alternating nodes from the first half and the reversed second half.
- The challenge is done in-place, ensuring a linear time solution with constant extra space.

Space & Time

Time Complexity: $O(n)$ · Each of the three main steps (finding the middle, reversing, merging) takes $O(n)$ time.
Space Complexity: $O(1)$ · The algorithm is done in-place with only a few pointers.

Solution

The solution involves three key steps:

- Use two pointers (slow and fast) to identify the middle of the list.
- Reverse the second half of the list. This can be done iteratively by adjusting the next pointers.
- Merge the two lists: One starting from the head and the other from the head of the reversed second half. Interleave the nodes by alternating between the two lists until complete. This approach uses in-place pointer manipulation, thus ensuring no extra space is used aside from a few temporary variables.

#150 Evaluate Reverse Polish Notation [Medium]

Key Insights

- Use a stack to process the tokens.
- Push numbers onto the stack.
- When an operator is encountered, pop the top two numbers, perform the operation, and push the result back.
- Handle division carefully to ensure it truncates toward zero.
- The final result remains as the only value in the stack.

Space & Time

Time Complexity: $O(n)$, where n is the number of tokens (each token is processed once).
Space Complexity: $O(n)$, in the worst-case scenario where all tokens are numbers and are stored in the stack.

Solution

We use a stack data structure to evaluate the RPN expression. Iterate through each token:

- If the token is a number, convert and push it onto the stack.
- If the token is an operator, pop the top two numbers (be careful about order), perform the corresponding arithmetic operation, and push the result back onto the stack.
- At the end of processing, the stack contains the result of the expression. Special attention is needed for division to ensure it truncates toward zero (using language-specific functions when necessary).

#155 Min Stack [Medium]

Key Insights

- Use an auxiliary data structure to track the current minimum element.

Stack · LeetCodeMastery — By Pattern

— 2978 —

LeetCodeMastery

Solution

We use a stack-based approach. Iterate over each character of the string while maintaining a variable for the current number and a previous operator (initialized to '+'). When an operator or the end of the string is reached, perform an action based on the previous operator:

- For '+' add the current number to the stack.
- For '-' add the negative of the current number.
- For '*' and '/' pop the last number from the stack, compute the result by performing multiplication or division, and then push the result back to the stack.
- After processing the entire string, sum the elements in the stack to get the final result. This approach ensures that multiplication and division are handled before addition and subtraction.

#255 Verify Preorder Sequence in Binary Search Tree [Medium]

Key Insights

- A BST has the property that every node's left subtree contains values less than the node, and every right subtree contains values greater.
- Preorder traversal processes nodes as: root, left subtree, then right subtree.
- Using a monotonic stack, we can simulate constructing the BST while tracking a lower bound for allowable node values.
- When moving to a right child, update the lower bound to ensure subsequent nodes are valid.

Space & Time

Time Complexity: $O(n)$ · Each element is processed once.
Space Complexity: $O(n)$ · In the worst-case an explicit stack is used; can be optimized to $O(1)$ by reusing the input array.

Solution

We use a monotonic stack to simulate the BST construction from the preorder sequence. The process is:

- Initialize a variable lower_bound to $-\infty$.
- Iterate through each value in the preorder array: if a value is less than lower_bound, it violates BST properties and the sequence is invalid.
- While the stack is not empty and the current value is greater than the element at the top of the stack, pop from the stack and update lower_bound to the popped value. This simulates moving to the right subtree.
- Push the current value onto the stack.
- If the sequence is processed without violations, return true.

This approach efficiently ensures that each value adheres to BST requirements during preorder traversal.

#394 Decode String [Medium]

Key Insights

- Use a stack or recursion to manage nested encoded patterns.
- Process the string character by character, accumulating digits for the repeat count.
- When encountering a 'T', start a new context (recursively or via a new stack frame) for the encoded substring.
- When encountering a 'T', finish the current context and repeat the accumulated substring as needed.
- Careful handling of multi-digit numbers is essential.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.
Space Complexity: $O(n)$, for the recursion stack or the auxiliary stack used.

Solution

We can solve this problem using a recursive approach. When we see a digit, we determine the full repeat count. Upon encountering a 'T', we recursively read the encoded substring until the corresponding 'T'. Each recursive call returns the decoded substring until it hits a 'T', which is then repeated based on the previously computed count. Alternatively, an iterative solution using a stack is also feasible. As we parse the string, push the current string and number onto the stack when encountering 'T', then pop and combine when 'T' is reached. This approach correctly handles nested patterns.

Time Complexity: $O(n)$ where n is the number of asteroids, since each asteroid is pushed and popped at most once.
Space Complexity: $O(n)$ in the worst case, when no collisions occur and all asteroids remain on the stack.

Solution

The solution uses a stack to simulate the asteroid collision process. For each asteroid in the input:

- If it is moving to the right or the stack is empty, simply add it to the stack.
- If it is moving to the left, check the asteroid on top of the stack. If the top asteroid is moving to the right, they might collide. Compare sizes: If the left-moving asteroid is larger (in absolute terms), pop the stack and continue checking. If both are equal, pop the stack and do not add the current asteroid. If the right-moving asteroid is larger, the current asteroid explodes.
- If no collision occurs for the current asteroid, push it onto the stack. Finally, the stack represents the surviving asteroids' order.

#739 Daily Temperatures [Medium]

Key Insights

- Use a monotonic stack to keep track of indices of days with unresolved warmer temperatures.
- As you iterate through the temperatures, resolve previous days that have found a warmer temperature.
- The index differences yield the number of days waited until a warmer temperature.

Space & Time

Time Complexity: $O(n)$, where n is the number of temperatures, because each index is pushed and popped from the stack at most once.
Space Complexity: $O(n)$ for storing the stack and the answer array.

Solution

The solution uses a monotonic stack to track the indices whose next warmer temperature has not been found yet. As you iterate through the temperature list, compare the current temperature with the temperature at the index stored at the top of the stack. If the current temperature is higher, it means you've found a warmer day for the day represented by the index from the stack. Pop that index, calculate the difference between the current index and the popped index, and store the result. Push the current index onto the stack for future comparisons. This approach efficiently finds the answer without nested iterations.

#962 Maximum Width Ramp [Medium]

Key Insights

- We need to identify pairs (i, j) where $i < j$ and $\text{nums}[i] \leq \text{nums}[j]$.
- A monotonic decreasing stack can be constructed to maintain candidate starting indices.
- By traversing from left to right, we push indices onto the stack only when they have a smaller value than the last.
- Then, a backward pass from the end of the array allows us to efficiently match each element with the smallest possible candidate index from the stack.

Space & Time

Time Complexity: $O(n)$
Space Complexity: $O(n)$

Solution

The solution is built in two phases:

- Build a decreasing stack of indices by iterating through the array (only push an index if its value is less than the value at the current top of the stack).
- Iterate through the array backwards. For each index j , check and pop elements from the stack while $\text{nums}[j] \geq \text{nums}[i]$ (greater than or equal to the nums value at the popped index). For each valid ramp found, compute the width ($j - \text{popped index}$) and update the maximum width accordingly. This approach guarantees we examine each element a constant number of times ensuring an efficient solution.

#1214 Two Sum BSTs [Medium]

Key Insights

- Traverse the first tree to collect all node values.
- Traverse the second tree to check for each node if the complement (target - current node value) exists in the first tree's values.
- Using a hash set for the first tree values allows efficient $O(1)$ lookups.
- The BST property is not essential for the hash set solution, but can be exploited if using iterator techniques for a two-pointer approach.

Space & Time

Time Complexity: $O(n + m)$ where n and m are the number of nodes in the first and second trees respectively.
Space Complexity: $O(n)$ for storing the node values from the first tree (in worst-case scenario).

Solution

We perform a DFS (or any tree traversal) on the first BST to collect all its values in a hash set. Then, we traverse the second BST and for each node, we calculate the complement (target - node value) and check whether this complement exists in the hash set. If we find such a pair, we return true. If no such pair exists after traversing the second tree completely, we return false.
This approach is straightforward and leverages extra space (the set) for constant time lookups, yielding an overall linear time complexity relative to the number of nodes.

#1762 Buildings With an Ocean View [Medium]

Key Insights

- Processing the buildings from right to left simplifies the problem because the rightmost building always has an ocean view.
- Keep track of the highest building encountered while iterating from right to left.
- A building has an ocean view if its height is greater than the maximum height seen so far.
- Append indices meeting the criteria, then reverse the order of indices for the solution since they were collected in reverse order.

Space & Time

Time Complexity: $O(n)$ where n is the number of buildings.
Space Complexity: $O(n)$ for storing the result in the worst-case scenario.

Solution

Iterate over the "heights" array from right to left while maintaining a variable to store the maximum height encountered so far. For each building, if its height is greater than this maximum height, add its index to the results list and update the maximum height. Since indices are collected in reverse order, reverse the list before returning it. This approach ensures that each building is visited only once.

#332 Longest Valid Parentheses [Hard]

Key Insights

- Use a stack to keep track of indices where potential valid substrings begin.
- Start by pushing a dummy index (-1) to handle edge cases.
- For every 'Y', push its index onto the stack.
- For every '(', pop from the stack. If the stack becomes empty, push the current index as a new base. Otherwise, compute the length of the valid substring using the current index minus the top index of the stack.
- An alternative is the dynamic programming approach where $dp[i]$ represents the length of the longest valid substring ending at index i .

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.
Space Complexity: $O(n)$ for the stack data structure.

Solution

Stack · LeetCode Mastery — By Pattern

— 2983 —

LeetCode Mastery

The stack-based solution works by maintaining indices for unmatched parentheses. Initially, a dummy index (-1) is pushed onto the stack to serve as the base for calculating subsequent valid substring lengths. As we iterate through the string:

- If the character is 'Y', we push its index onto the stack.
- If the character is '(', we pop from the stack. If the stack is empty afterwards, that means there is no valid starting position, so we push the current index to reset the base. Otherwise, we calculate the length of the current valid substring by subtracting the current index with the new top index of the stack and update the maximum length accordingly.

#42 Trapping Rain Water [Hard]

Key Insights

- The water trapped at any index is determined by the minimum of the maximum height on its left and right minus the height at that index.
- A two-pointers approach can efficiently solve the problem in one pass by maintaining left and right boundaries.
- Alternative approaches include dynamic programming with pre-computed left max and right max arrays, or using a monotonic stack to determine boundaries.

Space & Time

Time Complexity: $O(n)$
Space Complexity: $O(1)$ when using the two-pointer approach

Solution

We use a two-pointers strategy by initializing pointers at the beginning and the end of the array. Keep track of the maximum height seen so far from the left (left_max) and the right (right_max). At each step, move the pointer with the lesser height because the water trapped is determined by the shorter boundary. Compute the water at that position as the difference between the boundary (left_max or right_max) and the current height, accumulating it into the total. This approach ensures linear time scanning with constant extra space.

#84 Largest Rectangle in Histogram [Hard]

Key Insights

- Each bar can potentially extend left and right as long as the neighboring bars are not shorter.
- A monotonic (increasing) stack can be used to efficiently track indices of bars.
- When encountering a bar lower than the one on the top of the stack, pop the stack and calculate the area of the rectangle formed with the popped bar height.
- A dummy bar (of height 0) is appended at the end to ensure all bars are processed.

Space & Time

Time Complexity: $O(n)$
Space Complexity: $O(n)$

Solution

The solution employs a monotonic stack to traverse the histogram once. For each bar, while the current bar's height is less than the height at the top index of the stack, it pops from the stack and calculates the area with the popped height as the smallest bar. The width for the rectangle is determined by the difference between the current index and the index from the stack (or the current index if the stack is empty). This effectively finds the maximum rectangular area possible. A sentinel value (0) is added at the end of the histogram to flush any remaining bars from the stack.

#224 Basic Calculator [Hard]

Key Insights

- Use a stack to save previous results and signs when encountering parentheses.
- Process the string left-to-right by accumulating multi-digit numbers and applying the current sign.
- When encountering a 'Y', push the current result and sign onto the stack and reset them.

Stack · LeetCode Mastery — By Pattern

— 2984 —

LeetCode Mastery

- When encountering a 'Y', complete the current sub-expression then pop the previous state to incorporate the computed value.
- Handle whitespaces by simply skipping them.

Space & Time

Time Complexity: $O(n)$ where n is the length of the input string, since we process each character once.
Space Complexity: $O(n)$ in the worst-case scenario due to the use of a stack for nested parentheses.

Solution

The solution uses an iterative approach with a stack:
- Initialize variables: result to keep the running total, sign to handle addition/subtraction, and index to iterate over the string.
- As you iterate, build multi-digit numbers by checking for consecutive numerical characters.
- Adjust the total by adding the product of the current number and its sign.
- When encountering a 'Y', push the current result and sign onto the stack, then reset results and sign for the new sub-expression.
- When encountering a 'Y', complete the sub-expression by popping the previous result and sign, then combine.
- Skip over spaces.
- The final result is the computed value after the end of the string.
This approach effectively simulates recursion and correctly handles nested expressions and negation.

#239 Sliding Window Maximum [Hard]

Key Insights

- Use a deque (double-ended queue) to maintain the indices of potential maximum elements in a decreasing order.
- Ensure that the deque always has the current window's indices by removing elements that fall outside the current window.
- Only add elements that are greater than the ones at the back of the deque, thus preserving a monotonic decreasing order.
- The maximum for the current window is at the front of the deque.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in nums, since each element is processed at most twice.
Space Complexity: $O(k)$, as the deque stores indices for at most k elements at any time.

Solution

The solution uses a monotonic deque to maintain indices of potential maximum elements in the current window. As the window slides, we:
- Remove indices from the front if they are out of the window.
- Remove indices from the back if the current element is larger, because they cannot be the maximum if the current element is in the window.
- Append the current index.
- After the first k elements, the element at the front of the deque is the maximum for that window. This approach ensures that each element is pushed and popped from the deque only once, achieving an overall $O(n)$ time complexity.

#716 Max Stack [Hard]

Key Insights

- Use a doubly-linked list to support fast insertion and removal from the stack.
- Maintain a balanced tree (or ordered dictionary/multiset) to quickly locate the maximum element in $O(\log n)$ time.
- Map each value to its corresponding node(s) in the doubly-linked list, so that when popMax is called, you can immediately remove the corresponding node from the list.
- The double-linking allows removal of arbitrary nodes in $O(1)$, once you have the reference.

Space & Time

Time Complexity:
- push: $O(\log n)$ due to insertion in the ordered structure.
- pop: $O(1)$ for removal from the doubly-linked list, plus $O(\log n)$ for updating the tree.
- top: $O(1)$

Stack · LeetCode Mastery — By Pattern

— 2985 —

LeetCode Mastery

- peekMax: $O(1)$ or $O(\log n)$ depending on the tree implementation.
- popMax: $O(\log n)$ for looking up and removal from the tree, $O(1)$ for doubly-linked list deletion.

Space Complexity

- $O(n)$ for storing all nodes in the doubly-linked list and the tree structure mapping values to nodes.

Solution

We use two data structures:
- A doubly-linked list to store the elements in stack order. This supports fast push and pop operations and allows $O(1)$ removal when given a reference.
- An ordered structure (like a balanced BST, TreeMap in Java, or SortedList in Python) that maps each value to a list of nodes that hold that value in the linked list. This allows us to quickly determine the maximum element. When there are duplicates, we remove the one closest to the top by storing nodes in order. The trick is to keep these two structures synchronized. When an element is pushed, add its node to both the doubly-linked list and the ordered structure. When an element is removed (either via pop or popMax), remove its node from both structures.

#772 Basic Calculator III [Hard]

Key Insights

- Use recursion to handle nested expressions defined by parentheses.
- A stack can be used to correctly evaluate the expression and respect operator precedence.
- Multiplication and division have higher precedence than addition and subtraction.
- When encountering an open parenthesis, recursively compute its value until a closing parenthesis is found.
- Use immediate calculation for multiplication and division by updating the top of the stack before moving to the next operator.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, as every character is processed.
Space Complexity: $O(n)$ due to the recursion stack or auxiliary stack used for intermediate results.

Solution

We use a recursive approach combined with a stack to solve the problem. As we iterate through the string:
- Convert digits into numbers.
- When encountering an operator or parenthesis, process the previously accumulated number based on the preceding operator.
- For multiplication or division, pop the last number from the stack, apply the operation, and push the result back.
- When a '(' is encountered, recursively evaluate the substring until the corresponding ')' is reached.
- After processing all tokens, sum the values in the stack to get the final result. This method correctly handles both operator precedence and nested expressions.

#895 Maximum Frequency Stack [Hard]

Key Insights

- Use a hash map to count the frequency of each element.
- Maintain another hash map that maps frequencies to stacks (lists) of elements.
- Track the current maximum frequency to enable $O(1)$ retrieval of the most frequent element.
- When pushing an element, update its frequency and add it to the corresponding frequency stack.
- When popping an element, remove it from the stack corresponding to the maximum frequency and update the frequency count; if that stack becomes empty, decrease the current maximum frequency.

Space & Time

Time Complexity: $O(1)$ per push and pop.
Space Complexity: $O(n)$, where n is the number of elements pushed onto the stack.

Solution

The solution involves using two main data structures:

#21 Trees & BST — 37 questions · #21 of 21

#100

EASY

Same Tree

LeetCode · SimplifyLeet · WalkCC · LeetCode · Editorial

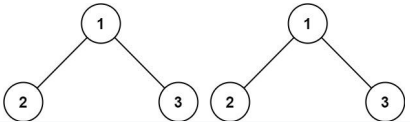
Tree · DFS · BFS

Lists: Grid 169

Problem:

Given the roots of two binary trees p and q , write a function to check if they are the same or not. Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

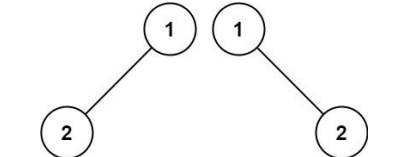
Example 1:



Input: $p = [1, 2, 3]$, $q = [1, 2, 3]$

Output: true

Example 2:



Input: $p = [1, 2]$, $q = [1, m, 1, 2]$

Output: false

Example 3:

Stack · LeetCode Mastery — By Pattern

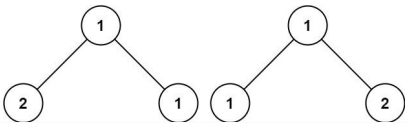
— 2987 —

LeetCode Mastery

Stack · LeetCode Mastery — By Pattern

— 2988 —

LeetCode Mastery



Input: p = [1,2,1], q = [1,1,2]

Output: false

Constraints:

- The number of nodes in both trees is in the range [0, 100].
- -104 <= Node.val <= 104

>> Brute Force (Trees & BST) Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        # Brute Force (DFS) - Recursive structural comparison
        if not p and not q: return True
        if not p or not q or p.val != q.val: return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```

Python

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
    # If both nodes are None, trees are identical at this branch.
    if not p and not q:
        return True
    # If one of the nodes is None, trees are not identical.
    if not p or not q:
        return False
    # Check if current nodes have the same value.
    if p.val != q.val:
        return False
    # Recursively compare the left subtrees and the right subtrees.
    return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```

Python

Trees & BST - LeetCode - By Pattern

— 2889 —

LeetCode

```
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and
                self.isSameTree(p.left, q.left) and
                self.isSameTree(p.right, q.right))
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if p == q:
            return True
        if p is None or q is None or p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right))
```

Python

```
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right))
```

Python

Trees & BST - LeetCode - By Pattern

— 2890 —

LeetCode

```
# Iteration
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if p is None:
            return q is None
        if q is None:
            return p is None
        stack1 = [p]
        stack2 = [q]
        while stack1 and stack2:
            current1 = stack1.pop()
            current2 = stack2.pop()
            if current1 is None and current2 is None:
                continue
            elif current1 is None or current2 is None:
                return False
            elif current1.val != current2.val:
                return False
            stack1.append(current1.left)
            stack1.append(current1.right)
            stack2.append(current2.left)
            stack2.append(current2.right)
        return not stack1 and not stack2
```

Java

Java

```
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null)
            return p == q;
        return p.val == q.val &&
            isSameTree(p.left, q.left) &&
            isSameTree(p.right, q.right);
    }
}
```

Java

Trees & BST - LeetCode - By Pattern

— 2991 —

LeetCode

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == q) return true;
        if (p == null || q == null || p.val != q.val) return false;
        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

Java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q;
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

Java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q;
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

Java

Trees & BST - LeetCode - By Pattern

— 2992 —

LeetCode

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q;
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

JavaScript

Trees & BST - LeetCode - By Pattern

— 2993 —

LeetCode

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q;
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

JavaScript

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q;
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

JavaScript

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null) return p == q;
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

JavaScript

Trees & BST - LeetCode - By Pattern

— 2994 —

LeetCode

```

//
// Definition for a binary tree node.
//
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) { val = x; }
}

boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) {
        return true;
    }
    if (p == null || q == null || p.val != q.val) {
        return false;
    }
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}

```

TypeScript

```

//
// Definition for a binary tree node.
//
class TreeNode {
    val: number;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val: number, left: TreeNode | null, right: TreeNode | null) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

function isSameTree(p: TreeNode | null, q: TreeNode | null): boolean {
    if (p == null && q == null) {
        return true;
    }
    if (p == null || q == null || p.val !== q.val) {
        return false;
    }
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}

```

Rust

Rust

Trees & BST - LeetCode - By Pattern

— 2995 —

LeetCode

```

// Definition for a binary tree node.
//
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q || p->val != q->val) return false;
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}

```

Rust

Trees & BST - LeetCode - By Pattern

— 2996 —

LeetCode

```

// Definition for a binary tree node.
//
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q || p->val != q->val) return false;
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)

Python

Trees & BST - LeetCode - By Pattern

— 2997 —

LeetCode

```

// Definition for a binary tree node.
//
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q || p->val != q->val) return false;
    return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}

```

#101

EASY

Symmetric Tree

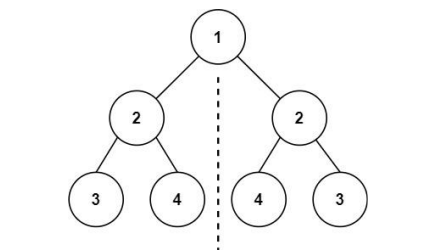
LeetCode - SimplyEasy - WalkCC - LeetCode - Editorial

Tree - DFS - BFS

List Crind 189

Problem: Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

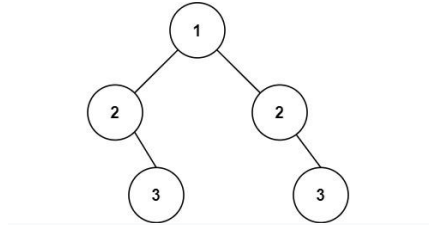
Example 1:



Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]

Output: false

Constraints:

Trees & BST - LeetCode - By Pattern

— 2999 —

LeetCode

• The number of nodes in the tree is in the range [1, 1000].

• -100 <= Node.val <= 100

Follow up: Could you solve it both recursively and iteratively?

Python

```

// Definition for a binary tree node.
//
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

bool isSymmetric(TreeNode* root) {
    return isSymmetric(root, root);
}

bool isSymmetric(TreeNode* t1, TreeNode* t2) {
    if (!t1 && !t2) return true;
    if (!t1 || !t2 || t1->val != t2->val) return false;
    return isSymmetric(t1->left, t2->right) && isSymmetric(t1->right, t2->left);
}

```

Python

```

// Definition for a binary tree node.
//
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

bool isSymmetric(TreeNode* root) {
    return isSymmetric(root, root);
}

bool isSymmetric(TreeNode* t1, TreeNode* t2) {
    if (!t1 && !t2) return true;
    if (!t1 || !t2 || t1->val != t2->val) return false;
    return isSymmetric(t1->left, t2->right) && isSymmetric(t1->right, t2->left);
}

```

Python

Trees & BST - LeetCode - By Pattern

— 3000 —

LeetCode


```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def dfs(root1:TreeNode, root2: Optional[TreeNode]) -> bool:
            if root1 == root2:
                return True
            if root1 is None or root2 is None or root1.val != root2.val:
                return False
            return dfs(root1.left, root2.right) and dfs(root1.right, root2.left)
        return dfs(root, root)

Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def dfs(root1, root2):
            if root1 is None and root2 is None:
                return True
            if root1 is None or root2 is None or root1.val != root2.val:
                return False
            return dfs(root1.left, root2.right) and dfs(root1.right, root2.left)
        return dfs(root, root)

Java
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return isSymmetric(root, root);
    }

    private boolean isSymmetric(TreeNode p, TreeNode q) {
        if (p == null || q == null)
            return p == q;
        return p.val == q.val && isSymmetric(p.left, q.right) && isSymmetric(p.right, q.left);
    }
}

```

Trees & BST - LeetCode - By Pattern

— 3001 —

LeetCode

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return dfs(root.left, root.right);
    }

    private boolean dfs(TreeNode root1, TreeNode root2) {
        if (root1 == root2) {
            return true;
        }
        if (root1 == null || root2 == null || root1.val != root2.val) {
            return false;
        }
        return dfs(root1.left, root2.right) && dfs(root1.right, root2.left);
    }
}

Java
//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return dfs(root, root);
    }

    private boolean dfs(TreeNode root1, TreeNode root2) {
        if (root1 == null && root2 == null) {
            return true;
        }
        if (root1 == null || root2 == null || root1.val != root2.val) {

```

Trees & BST - LeetCode - By Pattern

— 3002 —

LeetCode

```

        return false;
    }
    return dfs(root1.left, root2.right) && dfs(root1.right, root2.left);
}

JavaScript
//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val);
//     this.left = (left===undefined ? null : left);
//     this.right = (right===undefined ? null : right);
// }
//
// @param {TreeNode} root
// @return {boolean}
var isSymmetric = function (root) {
    const dfs = (root1, root2) => {
        if (root1 == root2) {
            return true;
        }
        if ((root1 || root2) && root1.val != root2.val) {
            return false;
        }
        return dfs(root1.left, root2.right) && dfs(root1.right, root2.left);
    };
    return dfs(root.left, root.right);
};

JavaScript
//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val);
//     this.left = (left===undefined ? null : left);
//     this.right = (right===undefined ? null : right);
// }
//
// @param {TreeNode} root
// @return {boolean}
var isSymmetric = function (root) {
    function dfs(root1, root2) {
        if (root1 && root2) return true;
        if ((root1 || root2) && root1.val != root2.val) return false;
        return dfs(root1.left, root2.right) && dfs(root1.right, root2.left);
    };
    return dfs(root, root);
};

TypeScript

```

Trees & BST - LeetCode - By Pattern

— 3003 —

LeetCode

```

TypeScript
//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val);
//         this.left = (left===undefined ? null : left);
//         this.right = (right===undefined ? null : right);
//     }
// }
function isSymmetric(root: TreeNode | null): boolean {
    const dfs = (root1: TreeNode | null, root2: TreeNode | null): boolean => {
        if (root1 == root2) {
            return true;
        }
        if ((root1 || root2) && root1.val != root2.val) {
            return false;
        }
        return dfs(root1.left, root2.right) && dfs(root1.right, root2.left);
    };
    return dfs(root.left, root.right);
}

TypeScript
//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val);
//         this.left = (left===undefined ? null : left);
//         this.right = (right===undefined ? null : right);
//     }
// }
const dfs = (root1: TreeNode | null, root2: TreeNode | null) => {
    if (root1 == root2) {
        return true;
    }
    if (root1 == null || root2 == null || root1.val != root2.val) {
        return false;
    }
    return dfs(root1.left, root2.right) && dfs(root1.right, root2.left);
};

function isSymmetric(root: TreeNode | null): boolean {
    return dfs(root.left, root.right);
}

```

Trees & BST - LeetCode - By Pattern

— 3004 —

LeetCode

```

Rust
Rust
// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<TreeNode>>,
//     pub right: Option<Rc<TreeNode>>,
// }
//
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    pub fn isSymmetric(root: Option<Rc<RefCell<TreeNode>>>) -> bool {
        fn dfs(root1: Option<Rc<RefCell<TreeNode>>>, root2: Option<Rc<RefCell<TreeNode>>>) -> bool {
            match (root1, root2) {
                (Some(node1), Some(node2)) => {
                    let node1 = node1.borrow();
                    let node2 = node2.borrow();
                    node1.val == node2.val
                    && dfs(node1.left.clone(), node2.right.clone())
                    && dfs(node1.right.clone(), node2.left.clone())
                },
                (None, None) => true,
                _ => false,
            }
        }
        match root {
            Some(root) => dfs(root.borrow().left.clone(), root.borrow().right.clone()),
            None => true,
        }
    }
}

```

Trees & BST - LeetCode - By Pattern

— 3005 —

LeetCode

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<RefCell<TreeNode>>>,
//     pub right: Option<Rc<RefCell<TreeNode>>>,
// }
//
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    fn dfs(root1: Option<Rc<RefCell<TreeNode>>>, root2: Option<Rc<RefCell<TreeNode>>>) -> bool {
        if root1.is_none() && root2.is_none() {
            return true;
        }
        if root1.is_none() || root2.is_none() {
            return false;
        }
        let node1 = root1.as_ref().unwrap().borrow();
        let node2 = root2.as_ref().unwrap().borrow();
        node1.val == node2.val &&
            Self::dfs(node1.left.clone(), node2.right.clone()) &&
            Self::dfs(node1.right.clone(), node2.left.clone())
    }
    pub fn isSymmetric(root: Option<Rc<RefCell<TreeNode>>>) -> bool {
        let node = root.as_ref().unwrap().borrow();
        Self::dfs(node.left.clone(), node.right.clone())
    }
}

```

[1] Trees & BST - Pattern Solution (matched from community answers)

Python

Trees & BST - LeetCode - By Pattern

— 3006 —

LeetCode

LeetMastery

Python

LeetMastery

LeetMastery

Python

LeetMastery

C++

LeetMastery

```

class Solution {
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        return build(nums, 0, nums.size() - 1);
    }

private:
    TreeNode* build(const vector<int>& nums, int l, int r) {
        if (l > r)
            return nullptr;
        const int m = (l + r) / 2;
        return new TreeNode(nums[m], build(nums, l, m - 1), build(nums, m + 1, r));
    }
};

```

C++

```

/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        auto dfs = [&](this auto& dfs, int l, int r) -> TreeNode* {
            if (l > r) {
                return nullptr;
            }
            int mid = (l + r) >> 1;
            return new TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r));
        };
        return dfs(0, nums.size() - 1);
    }
};

```

C++

```

/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        function<TreeNode*(int, int)> dfs = [&](int l, int r) -> TreeNode* {
            if (l > r) {
                return nullptr;
            }
            int mid = (l + r) >> 1;
            auto left = dfs(l, mid - 1);
            auto right = dfs(mid + 1, r);
            return new TreeNode(nums[mid], left, right);
        };
        return dfs(0, nums.size() - 1);
    }
};

```

Java

```

/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     public TreeNode(int val) {
 *         this.val = val;
 *         this.left = null;
 *         this.right = null;
 *     }
 * }
 */
public class Solution {
    private int[] nums;

    public TreeNode sortedArrayToBST(int[] nums) {
        this.nums = nums;
        return dfs(0, nums.length - 1);
    }

    private TreeNode dfs(int l, int r) {
        if (l > r) {
            return null;
        }
    }
}

```

```

        int mid = (l + r) >> 1;
        return new TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r));
    }
}

```

Java

```

/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     public TreeNode(int val) {
 *         this.val = val;
 *         this.left = null;
 *         this.right = null;
 *     }
 * }
 */
public class Solution {
    private int[] nums;

    public TreeNode sortedArrayToBST(int[] nums) {
        this.nums = nums;
        return dfs(0, nums.length - 1);
    }

    private TreeNode dfs(int l, int r) {
        if (l > r) {
            return null;
        }

        int mid = (l + r) >> 1;
        return new TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r));
    }
}

```

JavaScript

JavaScript

```

/*
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var sortedArrayToBST = function(nums) {
    const dfs = (l, r) => {
        if (l > r) {
            return null;
        }

        const mid = (l + r) >> 1;
        return new TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r));
    };
    return dfs(0, nums.length - 1);
};

```

JavaScript

```

/*
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {number[]} nums
 * @return {TreeNode}
 */
var sortedArrayToBST = function(nums) {
    const dfs = (l, r) => {
        if (l > r) {
            return null;
        }

        const mid = (l + r) >> 1;
        const left = dfs(l, mid - 1);
        const right = dfs(mid + 1, r);
        return new TreeNode(nums[mid], left, right);
    };
    return dfs(0, nums.length - 1);
};

```

TypeScript

TypeScript

```

/*
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val)
 *         this.left = (left===undefined ? null : left)
 *         this.right = (right===undefined ? null : right)
 *     }
 * }
 */
function sortedArrayToBST(nums: number[]): TreeNode | null {
    const dfs = (l: number, r: number): TreeNode | null => {
        if (l > r) {
            return null;
        }
        const mid = (l + r) >> 1;
        return new TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r));
    };
    return dfs(0, nums.length - 1);
}

```

TypeScript

```

/*
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val)
 *         this.left = (left===undefined ? null : left)
 *         this.right = (right===undefined ? null : right)
 *     }
 * }
 */
function sortedArrayToBST(nums: number[]): TreeNode | null {
    const n = nums.length;
    if (n === 0) {
        return null;
    }
    const mid = n >> 1;
    return new TreeNode(
        nums[mid],
        sortedArrayToBST(nums.slice(0, mid)),
        sortedArrayToBST(nums.slice(mid + 1, n)),
    );
}

```

Go

```

/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *     Val int
 *     Left *TreeNode
 *     Right *TreeNode
 * }
 */
func sortedArrayToBST(nums []int) *TreeNode {
    var dfs func(int, int) *TreeNode
    dfs = func(l, r int) *TreeNode {
        if l > r {
            return nil
        }
        mid := (l + r) >> 1
        return &TreeNode{nums[mid], dfs(l, mid-1), dfs(mid+1, r)}
    }
    return dfs(0, len(nums)-1)
}

```

Go

```

/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *     Val int
 *     Left *TreeNode
 *     Right *TreeNode
 * }
 */
func sortedArrayToBST(nums []int) *TreeNode {
    var dfs func(int, int) *TreeNode
    dfs = func(l, r int) *TreeNode {
        if l > r {
            return nil
        }
        mid := (l + r) >> 1
        left, right := dfs(l, mid-1), dfs(mid+1, r)
        return &TreeNode{nums[mid], left, right}
    }
    return dfs(0, len(nums)-1)
}

```

Rust

Rust

Rust

[^o] Trees & BST — Pattern Solution (matched from community answers)

Python

#110 EASY
Balanced Binary Tree
[LeetCode](#) • [SimplyLeet](#) • [WalkCC](#) • [LeetCode](#) • [Editorial](#)
[Tree](#) • [DFS](#)
Lists: Grind 169

Problem:
Given a binary tree, determine if it is height-balanced.

Example 1:



Example 3:

Input: root = []

Output: true

Constraints:

- The number of nodes in the tree is in the range [0, 5000]
- $-104 \leq \text{Node.val} \leq 104$

```
Python
class Solution:
    def isBalanced(self, root: TreeLinkNode) -> bool:
        if not root:
            return True

        def maxDepth(root: TreeLinkNode) -> int:
            if not root:
                return 0
            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return (abs(maxDepth(root.left) - maxDepth(root.right)) <= 1 and
                self.isBalanced(root.left) and
                self.isBalanced(root.right))
```

```

Python
class Solution:
    def isBalanced(self, root: TreeNode) -> bool:
        def maxdepth(root: TreeNode) -> int:
            # return the height of root if root is balanced, otherwise, return -1
            if not root:
                return 0
            left = maxdepth(root.left)
            if left == -1:
                return -1
            right = maxdepth(root.right)
            if right == -1:
                return -1
            if abs(left - right) > 1:
                return -1
            return 1 + max(maxdepth(root.left), maxdepth(root.right))
        return maxdepth(root) != -1

```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)
        return height(root) == 0
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)
        return height(root) == 0
```

Java

```
Java
class Solution {
    public boolean isBalanced(TreeNode root) {
        if (root == null)
            return true;
        return Math.abs(maxDepth(root.left) - maxDepth(root.right)) <= 1 && //
            isBalanced(root.left) && //
            isBalanced(root.right);
    }

    private int maxDepth(TreeNode root) {
        if (root == null)
            return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

Java

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return maxDepth(root) != -1;
    }

    private boolean ans = true;

    // Returns the height of root and sets ans to false if root is unbalanced.
    public int maxDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }
        final int left = maxDepth(root.left);
        final int right = maxDepth(root.right);
        if (Math.abs(left - right) > 1)
            ans = false;
        return Math.max(left, right) + 1;
    }
}
```

Java

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return maxDepth(root) != -1;
    }

    // Returns the height of root if root is balanced; otherwise, returns -1.
    private int maxDepth(TreeNode root) {
        if (root == null)
            return 0;
        final int left = maxDepth(root.left);
        if (left == -1)
            return -1;
        final int right = maxDepth(root.right);
        if (right == -1)
            return -1;
        if (Math.abs(left - right) > 1)
            return -1;
        return 1 + Math.max(left, right);
    }
}
```

Java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isBalanced(TreeNode root) {
        return height(root) == 0;
    }

    private int height(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int l = height(root.left);
        int r = height(root.right);
        if (l == -1 || r == -1 || Math.abs(l - r) > 1) {
            return -1;
        }
        return 1 + Math.max(l, r);
    }
}
```

Java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isBalanced(TreeNode root) {
        return height(root) == 0;
    }

    private int height(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int l = height(root.left);
        int r = height(root.right);
        if (l == -1 || r == -1 || Math.abs(l - r) > 1) {
            return -1;
        }
        return 1 + Math.max(l, r);
    }
}
```

JavaScript

JavaScript

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isBalanced = function (root) {
    const height = root => {
        if (!root) {
            return 0;
        }
        const l = height(root.left);
        const r = height(root.right);
        if (l == -1 || r == -1 || Math.abs(l - r) > 1) {
            return -1;
        }
        return 1 + Math.max(l, r);
    };
    return height(root) == 0;
};
```

JavaScript

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @return {boolean}
 */
var isBalanced = function (root) {
    const height = root => {
        if (!root) {
            return 0;
        }
        const l = height(root.left);
        const r = height(root.right);
        if (l == -1 || r == -1 || Math.abs(l - r) > 1) {
            return -1;
        }
        return 1 + Math.max(l, r);
    };
    return height(root) == 0;
};
```

TypeScript

```
TypeScript
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val)
 *         this.left = (left===undefined ? null : left)
 *         this.right = (right===undefined ? null : right)
 *     }
 * }
 */
function isBalanced(root: TreeNode | null): boolean {
    const dfs = (root: TreeNode | null) => {
        if (root == null) {
            return 0;
        }
        const left = dfs(root.left);
        const right = dfs(root.right);
        if (left == -1 || right == -1 || Math.abs(left - right) > 1) {
            return -1;
        }
        return 1 + Math.max(left, right);
    };
    return dfs(root) != -1;
}
```

TypeScript

```
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val)
 *         this.left = (left===undefined ? null : left)
 *         this.right = (right===undefined ? null : right)
 *     }
 * }
 */
function isBalanced(root: TreeNode | null): boolean {
    const dfs = (root: TreeNode | null) => {
        if (root == null) {
            return 0;
        }
        const left = dfs(root.left);
        const right = dfs(root.right);
        if (left == -1 || right == -1 || Math.abs(left - right) > 1) {
            return -1;
        }
        return 1 + Math.max(left, right);
    };
    return dfs(root) != -1;
}
```

```

    }
    return dfs(root) + 1;
}

Rust

Rust

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<RefCell<TreeNode>>>,
//     pub right: Option<Rc<RefCell<TreeNode>>>,
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    pub fn is_balanced(root: Option<Rc<RefCell<TreeNode>>>)-> bool {
        Self::dfs(root) == 1
    }

    fn dfs(root: Option<Rc<RefCell<TreeNode>>>)-> i32 {
        if root.is_none() {
            return 0;
        }
        let node = root.as_ref().unwrap().borrow();
        let left = Self::dfs(&node.left);
        let right = Self::dfs(&node.right);
        if left == -1 || right == -1 || (left - right).abs() > 1 {
            return -1;
        }
        1 + left.max(right)
    }
}

Rust

```

Trees & BST - LeetCode - By Pattern

— 3037 —

LeetCode

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<RefCell<TreeNode>>>,
//     pub right: Option<Rc<RefCell<TreeNode>>>,
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    pub fn is_balanced(root: Option<Rc<RefCell<TreeNode>>>)-> bool {
        Self::dfs(root) == 1
    }

    fn dfs(root: Option<Rc<RefCell<TreeNode>>>)-> i32 {
        if root.is_none() {
            return 0;
        }
        let node = root.as_ref().unwrap().borrow();
        let left = Self::dfs(&node.left);
        let right = Self::dfs(&node.right);
        if left == -1 || right == -1 || (left - right).abs() > 1 {
            return -1;
        }
        1 + left.max(right)
    }
}

[+] Trees & BST - Pattern Solution (matched from community answers)
Python

```

Trees & BST - LeetCode - By Pattern

— 3038 —

LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isBalanced(self, root: TreeNode) -> bool:
        # Return true, if return the height of the subtree or -1 if unbalanced.
        def checkHeight(node: TreeNode) -> int:
            # If empty tree has height 0 and is balanced.
            if not node:
                return 0

            # Recursively get the height of the left subtree.
            leftHeight = checkHeight(node.left)
            # If left subtree is unbalanced, return -1.
            if leftHeight == -1:
                return -1

            # Recursively get the height of the right subtree.
            rightHeight = checkHeight(node.right)
            # If right subtree is unbalanced, return -1.
            if rightHeight == -1:
                return -1

            # If the current node's subtrees differ in height by more than 1, it's unbalanced.
            if abs(leftHeight - rightHeight) > 1:
                return -1

            # Return the height of the current node.
            return max(leftHeight, rightHeight) + 1

        # The tree is balanced if checkHeight does not return -1.
        return checkHeight(root) != -1

```

#112

EASY

Path Sum

LeetCode - SimplyEasy - WalkCCC - LeetCode - Editorial

Tree - DFS - BFS

Little Denny Zhang

Problem:

Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum.

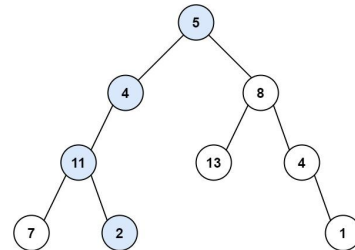
A leaf is a node with no children.

Example 1:

Trees & BST - LeetCode - By Pattern

— 3039 —

LeetCode

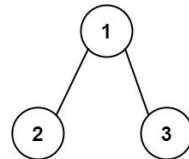


Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22

Output: true

Explanation: The root-to-leaf path with the target sum is shown.

Example 2:



Input: root = [1,2,3], targetSum = 5

Output: false

Explanation: There are two root-to-leaf paths in the tree:

(1 -> 2): The sum is 3.

(1 -> 3): The sum is 4.

There is no root-to-leaf path with sum = 5.

Example 3:

Trees & BST - LeetCode - By Pattern

— 3040 —

LeetCode

Input: root = [], targetSum = 0
Output: false
Explanation: Since the tree is empty, there are no root-to-leaf paths.

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- -1000 <= Node.val <= 1000
- -1000 <= targetSum <= 1000

>> Brute Force [Trees & BST] Interview Prep

Python

```

# Brute Force Approach
class Solution:
    def hasPathSum(self, root, targetSum):
        # Brute Force - collect inorder, then process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)
        collect(root)
        return vals[0] if vals else 0

```

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def hasPathSum(root, targetSum):
    # If the current node is None, no valid path exists
    if not root:
        return False
    # Check if the current node is a leaf node
    if not root.left and not root.right:
        # If the current node's value equals the target sum
        return targetSum == root.val
    # Recursively check the left and right subtrees with the updated target sum
    return hasPathSum(root.left, targetSum - root.val) or hasPathSum(root.right, targetSum - root.val)

# Example usage:
# root = TreeNode(1, TreeNode(4, TreeNode(11, TreeNode(7)), TreeNode(8, TreeNode(13),
#   TreeNode(4, None, TreeNode(1))))
# print(hasPathSum(root, 22)) # Output: True

```

Python

Trees & BST - LeetCode - By Pattern

— 3041 —

LeetCode

```

class Solution:
    def hasPathSum(self, root: TreeNode, sum: int) -> bool:
        if not root:
            return False
        if root.val == sum and not root.left and not root.right:
            return True
        return (self.hasPathSum(root.left, sum - root.val) or
                self.hasPathSum(root.right, sum - root.val))

Python

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        def dfs(node, s):
            if not node:
                return False
            s += node.val
            if not node.left and not node.right and s == targetSum:
                return True
            return dfs(node.left, s) or dfs(node.right, s)
        return dfs(root, 0)

Python

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        def dfs(node, s):
            if not node:
                return False
            s += node.val
            if not node.left and not node.right and s == targetSum:
                return True
            return dfs(node.left, s) or dfs(node.right, s)
        return dfs(root, 0)

Java

Java

```

Trees & BST - LeetCode - By Pattern

— 3042 —

LeetCode

```
class Solution {
public:
    bool hasPathSum(TreeNode* root, int sum) {
        if (root == null)
            return false;
        if (root->val == sum && root->left == null && root->right == null)
            return true;
        return
            hasPathSum(root->left, sum - root->val) ||
            hasPathSum(root->right, sum - root->val);
    }
}

Java

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        return dfs(root, targetSum);
    }

    private boolean dfs(TreeNode root, int s) {
        if (root == null) {
            return false;
        }
        s -= root.val;
        if (root.left == null && root.right == null && s == 0) {
            return true;
        }
        return dfs(root.left, s) || dfs(root.right, s);
    }
}
```

Java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        return dfs(root, targetSum);
    }

    private boolean dfs(TreeNode root, int s) {
        if (root == null) {
            return false;
        }
        s -= root.val;
        if (root.left == null && root.right == null && s == 0) {
            return true;
        }
        return dfs(root.left, s) || dfs(root.right, s);
    }
}
```

JavaScript

```
/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val);
 *     this.left = (left===undefined ? null : left);
 *     this.right = (right===undefined ? null : right);
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} targetSum
 * @return {boolean}
 */
var hasPathSum = function (root, targetSum) {
    function dfs(root, s) {
        if (!root) return false;
        s -= root.val;
        if (!root.left && !root.right && s === targetSum) return true;
        return dfs(root.left, s) || dfs(root.right, s);
    }
    return dfs(root, 0);
};
```

```
JavaScript

/**
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val);
 *     this.left = (left===undefined ? null : left);
 *     this.right = (right===undefined ? null : right);
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} targetSum
 * @return {boolean}
 */
var hasPathSum = function (root, targetSum) {
    function dfs(root, s) {
        if (!root) return false;
        s -= root.val;
        if (!root.left && !root.right && s === targetSum) return true;
        return dfs(root.left, s) || dfs(root.right, s);
    }
    return dfs(root, 0);
};
```

TypeScript

```
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val);
 *         this.left = (left===undefined ? null : left);
 *         this.right = (right===undefined ? null : right);
 *     }
 * }
 */
function hasPathSum(root: TreeNode | null, targetSum: number): boolean {
    if (root === null) {
        return false;
    }
    const { val, left, right } = root;
    if (left === null && right === null) {
        return targetSum - val === 0;
    }
    return hasPathSum(left, targetSum - val) || hasPathSum(right, targetSum - val);
}
```

TypeScript

```
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val);
 *         this.left = (left===undefined ? null : left);
 *         this.right = (right===undefined ? null : right);
 *     }
 * }
 */
function hasPathSum(root: TreeNode | null, targetSum: number): boolean {
    if (root === null) {
        return false;
    }
    const { val, left, right } = root;
    if (left === null && right === null) {
        return targetSum - val === 0;
    }
    return hasPathSum(left, targetSum - val) || hasPathSum(right, targetSum - val);
}
```

Rust

```
Rust

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<RcRefCell<TreeNode>>,
//     pub right: Option<RcRefCell<TreeNode>>,
// }
// impl! TreeNode {
//     # [inline]
//     pub fn is_leaf() -> bool {
//         self.left.is_none() && self.right.is_none()
//     }
//     pub fn is_root() -> bool {
//         self.parent.is_none()
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    pub fn has_path_sum(root: Option<RcRefCell<TreeNode>>, target_sum: i32) -> bool {
        match root {
            None => false,
            Some(node) => {
                let mut node = node.borrow_mut();
                if node.left.is_none() && node.right.is_none() {
                    return target_sum - node.val == 0;
                }
                let val = node.val;
                Self::has_path_sum(node.left.take(), target_sum - val) ||
                    Self::has_path_sum(node.right.take(), target_sum - val)
            }
        }
    }
}
```

```
let mut node = node.borrow_mut();
if node.left.is_none() && node.right.is_none() {
    return target_sum - node.val == 0;
}
let val = node.val;
Self::has_path_sum(node.left.take(), target_sum - val) ||
    Self::has_path_sum(node.right.take(), target_sum - val)
}
}

Rust

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<RcRefCell<TreeNode>>,
//     pub right: Option<RcRefCell<TreeNode>>,
// }
// impl! TreeNode {
//     # [inline]
//     pub fn is_leaf() -> bool {
//         self.left.is_none() && self.right.is_none()
//     }
//     pub fn is_root() -> bool {
//         self.parent.is_none()
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    pub fn has_path_sum(root: Option<RcRefCell<TreeNode>>, target_sum: i32) -> bool {
        match root {
            None => false,
            Some(node) => {
                let mut node = node.borrow_mut();
                if node.left.is_none() && node.right.is_none() {
                    return target_sum - node.val == 0;
                }
                let val = node.val;
                Self::has_path_sum(node.left.take(), target_sum - val) ||
                    Self::has_path_sum(node.right.take(), target_sum - val)
            }
        }
    }
}
```

[1] Trees & BST - Pattern Solution (matched from community answers)

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val: int, left: Optional[TreeNode], right: Optional[TreeNode]) -> None:
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None
        s = root.val
        if root.left is None and root.right is None and s == targetSum:
            return True
        return dfs(root.left, s) or dfs(root.right, s)

        return dfs(root, 0)
```

#226

EASY

Invert Binary Tree

LeetCode - SimplifyLeet - WalkCC - LeetCode - Editorial

Tree - DFS - BFS

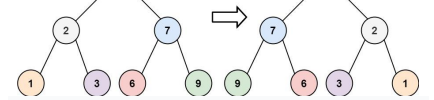
Like: 100

Views: 100

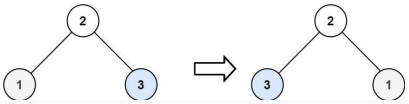
Problem:

Given the root of a binary tree, invert the tree, and return its root.

Example 1:



Example 2:



Input: root = [2,1,3]
Output: [2,3,1]

Example 3:
Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- -100 <= Node.val <= 100

```
Python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = None
        self.right = None

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        # Base case: If the node is None, return None.
        if not root:
            return None
        # Recursively invert the left and right subtrees.
        left_inverted = self.invertTree(root.left)
        right_inverted = self.invertTree(root.right)
        # Swap the left and right subtrees.
        root.left, root.right = right_inverted, left_inverted
        return root

Python
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if not root:
            return None
        left = root.left
        right = root.right
        root.left = self.invertTree(right)
        root.right = self.invertTree(left)
        return root

Python
```

```
Python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = None
        self.right = None

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None
        l, r = self.invertTree(root.left), self.invertTree(root.right)
        root.left, root.right = r, l
        return root

Python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = None
        self.right = None

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def dfs(root):
            if root is None:
                return
            root.left, root.right = root.right, root.left
            dfs(root.left)
            dfs(root.right)
        dfs(root)
        return root

Java
Java
```

```
JavaScript
// Definition for a binary tree node.
function TreeNode(val) {
    this.val = val;
    this.left = null;
    this.right = null;
}

class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) {
            return null;
        }
        TreeNode l = invertTree(root.left);
        TreeNode r = invertTree(root.right);
        root.left = r;
        root.right = l;
        return root;
    }
}

JavaScript
// Definition for a binary tree node.
function TreeNode(val) {
    this.val = val;
    this.left = null;
    this.right = null;
}

class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) {
            return null;
        }
        TreeNode l = invertTree(root.left);
        TreeNode r = invertTree(root.right);
        root.left = r;
        root.right = l;
        return root;
    }
}

JavaScript
JavaScript
```

```
JavaScript
// Definition for a binary tree node.
function TreeNode(val) {
    this.val = val;
    this.left = null;
    this.right = null;
}

var invertTree = function (root) {
    if (!root) {
        return root;
    }
    const l = invertTree(root.left);
    const r = invertTree(root.right);
    root.left = r;
    root.right = l;
    return root;
};

JavaScript
// Definition for a binary tree node.
function TreeNode(val) {
    this.val = val;
    this.left = null;
    this.right = null;
}

var invertTree = function (root) {
    const dfs = root => {
        if (!root) {
            return;
        }
        [root.left, root.right] = [root.right, root.left];
        dfs(root.left);
        dfs(root.right);
    };
    dfs(root);
    return root;
};

TypeScript
TypeScript
```

```
TypeScript
// Definition for a binary tree node.
class TreeNode {
    val: number;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
        this.val = (val === undefined ? 0 : val);
        this.left = (left === undefined ? null : left);
        this.right = (right === undefined ? null : right);
    }
}

function invertTree(root: TreeNode | null): TreeNode | null {
    if (!root) {
        return root;
    }
    const l = invertTree(root.left);
    const r = invertTree(root.right);
    root.left = r;
    root.right = l;
    return root;
}

TypeScript
// Definition for a binary tree node.
class TreeNode {
    val: number;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
        this.val = (val === undefined ? 0 : val);
        this.left = (left === undefined ? null : left);
        this.right = (right === undefined ? null : right);
    }
}

function invertTree(root: TreeNode | null): TreeNode | null {
    const dfs = (root: TreeNode | null) => {
        if (!root) {
            return;
        }
        [root.left, root.right] = [root.right, root.left];
        dfs(root.left);
        dfs(root.right);
    };
    dfs(root);
    return root;
}

Rust
Rust
```

```
Rust
// Definition for a binary tree node.
#[derive(Debug, PartialEq, Eq)]
pub struct TreeNode {
    pub val: i32,
    pub left: Option>,
    pub right: Option>,
}

impl TreeNode {
    fn new(val: i32) -> Self {
        Self {
            val,
            left: None,
            right: None,
        }
    }
}

use std::rc::Rc;
use std::cell::RefCell;

impl Solution {
    pub fn invert_tree(root: Option>) -> Option> {
        if let Some(node) = root.clone() {
            let mut node = node.borrow_mut();
            let left = node.left.take();
            let right = node.right.take();

            let right = Self::invert_tree(right);
            let left = Self::invert_tree(left);

            node.left = right;
            node.right = left;
        }
        Some(Box::new(node))
    }
}

Rust
```

```

// Definition for a binary tree node.
// #include<bits/stdc++.h>
// using namespace std;
// int main() {
//     int n;
//     cin >> n;
//     int arr[n];
//     for (int i = 0; i < n; i++) {
//         cin >> arr[i];
//     }
//     // Insert the nodes
//     int root = arr[0];
//     int left = -1, right = -1;
//     for (int i = 1; i < n; i++) {
//         if (arr[i] < root) {
//             left = arr[i];
//         } else {
//             right = arr[i];
//         }
//     }
//     // Recursively insert the nodes
//     int left_node = left;
//     int right_node = right;
//     if (left != -1) {
//         left = left_node;
//         left_node = left;
//     }
//     if (right != -1) {
//         right = right_node;
//         right_node = right;
//     }
//     // Return the root
//     return root;
// }

```

[1] Trees & BST — Pattern Solution (matched from community answers)

Python

Trees & BST - LeetCode - By Pattern

— 3055 —

LeetCode

```

// Definition for a binary tree node.
// #include<bits/stdc++.h>
// using namespace std;
// int main() {
//     int n;
//     cin >> n;
//     int arr[n];
//     for (int i = 0; i < n; i++) {
//         cin >> arr[i];
//     }
//     // Insert the nodes
//     int root = arr[0];
//     int left = -1, right = -1;
//     for (int i = 1; i < n; i++) {
//         if (arr[i] < root) {
//             left = arr[i];
//         } else {
//             right = arr[i];
//         }
//     }
//     // Recursively insert the nodes
//     int left_node = left;
//     int right_node = right;
//     if (left != -1) {
//         left = left_node;
//         left_node = left;
//     }
//     if (right != -1) {
//         right = right_node;
//         right_node = right;
//     }
//     // Return the root
//     return root;
// }

```

#270

EASY

Closest Binary Search Tree Value

LeetCode - SimplifyLeetCode - WUCC - LeetCode - Editorial

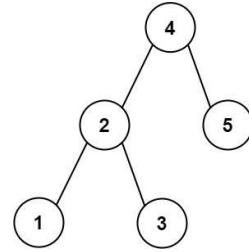
Binary Search - Tree - DFS - BST

LeetCode Premium 30

Problem:

Given the root of a binary search tree and a target value, return the value in the BST that is closest to the target. If there are multiple answers, print the smallest.

Example 1:



Trees & BST - LeetCode - By Pattern

— 3056 —

LeetCode

Input: root = [4,2,5,1,3], target = 3.714286
Output: 4

Example 2:

Input: root = [1], target = 4.428571
Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 0 <= Node.val <= 100
- 109 <= target <= 109

Python

Python

```

// Definition for a binary tree node.
// #include<bits/stdc++.h>
// using namespace std;
// int main() {
//     int n;
//     cin >> n;
//     int arr[n];
//     for (int i = 0; i < n; i++) {
//         cin >> arr[i];
//     }
//     // Insert the nodes
//     int root = arr[0];
//     int left = -1, right = -1;
//     for (int i = 1; i < n; i++) {
//         if (arr[i] < root) {
//             left = arr[i];
//         } else {
//             right = arr[i];
//         }
//     }
//     // Recursively insert the nodes
//     int left_node = left;
//     int right_node = right;
//     if (left != -1) {
//         left = left_node;
//         left_node = left;
//     }
//     if (right != -1) {
//         right = right_node;
//         right_node = right;
//     }
//     // Return the root
//     return root;
// }

```

Python

Trees & BST - LeetCode - By Pattern

— 3057 —

LeetCode

```

class Solution {
public:
    int closestValue(TreeNode* root, float target) {
        if (root == NULL) return 0;
        if (target < root->val) return closestValue(root->left, target);
        if (target > root->val) return closestValue(root->right, target);
        if (target == root->val) return root->val;
    }
};

```

Python

```

// Definition for a binary tree node.
// #include<bits/stdc++.h>
// using namespace std;
// int main() {
//     int n;
//     cin >> n;
//     int arr[n];
//     for (int i = 0; i < n; i++) {
//         cin >> arr[i];
//     }
//     // Insert the nodes
//     int root = arr[0];
//     int left = -1, right = -1;
//     for (int i = 1; i < n; i++) {
//         if (arr[i] < root) {
//             left = arr[i];
//         } else {
//             right = arr[i];
//         }
//     }
//     // Recursively insert the nodes
//     int left_node = left;
//     int right_node = right;
//     if (left != -1) {
//         left = left_node;
//         left_node = left;
//     }
//     if (right != -1) {
//         right = right_node;
//         right_node = right;
//     }
//     // Return the root
//     return root;
// }

```

Python

Trees & BST - LeetCode - By Pattern

— 3058 —

LeetCode

```

// Definition for a binary tree node.
// #include<bits/stdc++.h>
// using namespace std;
// int main() {
//     int n;
//     cin >> n;
//     int arr[n];
//     for (int i = 0; i < n; i++) {
//         cin >> arr[i];
//     }
//     // Insert the nodes
//     int root = arr[0];
//     int left = -1, right = -1;
//     for (int i = 1; i < n; i++) {
//         if (arr[i] < root) {
//             left = arr[i];
//         } else {
//             right = arr[i];
//         }
//     }
//     // Recursively insert the nodes
//     int left_node = left;
//     int right_node = right;
//     if (left != -1) {
//         left = left_node;
//         left_node = left;
//     }
//     if (right != -1) {
//         right = right_node;
//         right_node = right;
//     }
//     // Return the root
//     return root;
// }

```

Java

Java

Trees & BST - LeetCode - By Pattern

— 3059 —

LeetCode

```

class Solution {
public:
    int closestValue(TreeNode* root, double target) {
        if (root == NULL) return 0;
        if (target < root->val) return closestValue(root->left, target);
        if (target > root->val) return closestValue(root->right, target);
        if (target == root->val) return root->val;
    }
};

```

Java

```

// Definition for a binary tree node.
// #include<bits/stdc++.h>
// using namespace std;
// int main() {
//     int n;
//     cin >> n;
//     int arr[n];
//     for (int i = 0; i < n; i++) {
//         cin >> arr[i];
//     }
//     // Insert the nodes
//     int root = arr[0];
//     int left = -1, right = -1;
//     for (int i = 1; i < n; i++) {
//         if (arr[i] < root) {
//             left = arr[i];
//         } else {
//             right = arr[i];
//         }
//     }
//     // Recursively insert the nodes
//     int left_node = left;
//     int right_node = right;
//     if (left != -1) {
//         left = left_node;
//         left_node = left;
//     }
//     if (right != -1) {
//         right = right_node;
//         right_node = right;
//     }
//     // Return the root
//     return root;
// }

```

Java

Java

Trees & BST - LeetCode - By Pattern

— 3060 —

LeetCode

```

private void dfs(TreeNode node) {
    if (node == null) {
        return;
    }
    double xut = Math.abs(node.val - target);
    if (xut < diff || (xut == diff && node.val < ans)) {
        diff = xut;
        ans = node.val;
    }
    node = target < node.val ? node.left : node.right;
    dfs(node);
}
}

```

Java

```

// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    public int closestValue(TreeNode root, double target) {
        int ans = root.val;
        double ai = Number.MAX_VALUE;
        while (root != null) {
            double t = Math.abs(root.val - target);
            if (t < ai || (t == ai && root.val < ans)) {
                ai = t;
                ans = root.val;
            }
            if (root.val > target) {
                root = root.left;
            } else {
                root = root.right;
            }
        }
        return ans;
    }
}

```

JavaScript
JavaScript

Trees & BST - LeetCode - By Pattern

— 3061 —

LeetCode

```

// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @param {number} target
// @return {number}
var closestValue = function (root, target) {
    let ans = 0;
    let diff = Infinity;

    const dfs = node => {
        if (!node) {
            return;
        }

        const xut = Math.abs(target - node.val);
        if (xut < diff || (xut === diff && node.val < ans)) {
            diff = xut;
            ans = node.val;
        }

        node = target < node.val ? node.left : node.right;
        dfs(node);
    };

    dfs(root);
    return ans;
};

```

JavaScript

Trees & BST - LeetCode - By Pattern

— 3062 —

LeetCode

```

// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @param {number} target
// @return {number}
var closestValue = function (root, target) {
    let ans = root.val;
    let ai = Number.MAX_VALUE;
    while (root) {
        const t = Math.abs(root.val - target);
        if (t < ai || (t == ai && root.val < ans)) {
            ai = t;
            ans = root.val;
        }
        if (root.val > target) {
            root = root.left;
        } else {
            root = root.right;
        }
    }
    return ans;
};

```

TypeScript
TypeScript

Trees & BST - LeetCode - By Pattern

— 3063 —

LeetCode

```

// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }
function closestValue(root: TreeNode | null, target: number): number {
    let ans = 0;
    let diff = Number.POSITIVE_INFINITY;

    const dfs = (node: TreeNode | null): void => {
        if (!node) {
            return;
        }

        const xut = Math.abs(target - node.val);
        if (xut < diff || (xut === diff && node.val < ans)) {
            diff = xut;
            ans = node.val;
        }

        node = target < node.val ? node.left : node.right;
        dfs(node);
    };

    dfs(root);
    return ans;
}

```

TypeScript

Trees & BST - LeetCode - By Pattern

— 3064 —

LeetCode

```

// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }
function closestValue(root: TreeNode | null, target: number): number {
    let ans = 0;
    let diff = Number.POSITIVE_INFINITY;

    const dfs = (node: TreeNode | null): void => {
        if (!node) {
            return;
        }

        const xut = Math.abs(target - node.val);
        if (xut < diff || (xut === diff && node.val < ans)) {
            diff = xut;
            ans = node.val;
        }

        node = target < node.val ? node.left : node.right;
        dfs(node);
    };

    dfs(root);
    return ans;
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)
Python

Trees & BST - LeetCode - By Pattern

— 3065 —

LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val: int, left: Optional[TreeNode], right: Optional[TreeNode]):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        def dfs(node: Optional[TreeNode]):
            if node is None:
                return
            xut = abs(target - node.val)
            nonlocal ans, diff
            if xut < diff or (xut == diff and node.val < ans):
                diff = xut
                ans = node.val
            node = node.left if target < node.val else node.right
            dfs(node)

        ans = 0
        diff = inf
        dfs(root)
        return ans

```

#501 EASY
Find Mode in Binary Search Tree
LeetCode - Simplest - MathCC - LeetCode - Editorial

Tree - DFS - BST
List: Denny Zhang

Problem:

Given the root of a binary search tree (BST) with duplicates, return all the mode(s) (i.e., the most frequently occurred element) in it.

If the tree has more than one mode, return them in any order.

Assume a BST is defined as follows:

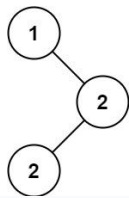
- The left subtree of a node contains only nodes with keys less than or equal to the node's key.
- The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:

Trees & BST - LeetCode - By Pattern

— 3066 —

LeetCode



Input: root = [1,null,2,2]
Output: [2]
Example 2:
Input: root = [0]
Output: [0]
Constraints:
• The number of nodes in the tree is in the range [1, 104].
• -105 <= Node.val <= 105
Follow up: Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

```
>> Brute Force [Trees & BST] Interview Prep
Python
# Brute Force Approach
class Solution:
    def findMode(self, root):
        # Brute Force - collect inorder, then process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)
        collect(root)
        return vals[0] if vals else 0
Python
Python
```

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findMode(self, root):
        # Maintain three variables: prev value, current frequency count, maximum frequency, and nodes
        list = []
        self.prev = None
        self.currentCount = 0
        self.maxCount = 0
        self.nodes = []

        # First In-order traversal to find the maximum frequency (maxCount).
        def inorder_first(node):
            if not node:
                return
            inorder_first(node.left)
            # Update the count for the current node.
            if self.prev is None or node.val != self.prev:
                self.currentCount = 1
            else:
                self.currentCount += 1

            self.prev = node.val
            # Update maxCount if currentCount exceeds it.
            self.maxCount = max(self.maxCount, self.currentCount)
            inorder_first(node.right)

        inorder_first(root)

        # Second In-order traversal to collect all node values with frequency equal to maxCount.
        self.prev = None # Reset previous value.
        self.currentCount = 0 # Reset count.
        def inorder_second(node):
            if not node:
                return
            inorder_second(node.left)
            if self.prev is None or node.val != self.prev:
                self.currentCount = 1
            else:
                self.currentCount += 1
            self.prev = node.val
            if self.currentCount == self.maxCount:
                self.nodes.append(node.val)
            inorder_second(node.right)

        inorder_second(root)

        return self.nodes
Python
```

```
class Solution:
    def findMode(self, root: TreeNode | None) -> List[int]:
        self.ans = []
        self.prev = None
        self.count = 0
        self.maxCount = 0

        def updateCount(root: TreeNode | None) -> None:
            if self.prev and self.prev.val == root.val:
                self.count += 1
            else:
                self.count = 1

            if self.count > self.maxCount:
                self.maxCount = self.count
                self.ans = [root.val]
            elif self.count == self.maxCount:
                self.ans.append(root.val)

            self.prev = root

        def inorder(root: TreeNode | None) -> None:
            if not root:
                return

            inorder(root.left)
            updateCount(root)
            inorder(root.right)

        inorder(root)
        return self.ans
Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findMode(self, root: TreeNode) -> List[int]:
        if root is None:
            return
        maxNode, mx, prev, ans, cnt
        dfs(root.left)
        cnt = cnt + 1 if prev == root.val else 1
        if cnt > mx:
            ans = [root.val]
        mx = cnt
        elif cnt == mx:
            ans.append(root.val)
        prev = root.val
        dfs(root.right)

        prev = None
        mx = cnt = 0
        ans = []
Python
```

```
dfs(root)
return ans
Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findMode(self, root: TreeNode) -> List[int]:
        def dfs(root):
            if root is None:
                return
            maxNode, mx, prev, ans, cnt
            dfs(root.left)
            cnt = cnt + 1 if prev == root.val else 1
            if cnt > mx:
                ans = [root.val]
            mx = cnt
            elif cnt == mx:
                ans.append(root.val)
            prev = root.val
            dfs(root.right)

        prev = None
        mx = cnt = 0
        ans = []

        dfs(root)
        return ans
C++
C++
```

```
//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode* left;
//     TreeNode* right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, int y, int z) : val(x), left(y), right(z) {}
// };

class Solution {
public:
    TreeNode* prev;
    int mx, cnt;
    vector<int> ans;

    vector<int> findMode(TreeNode* root) {
        dfs(root);
        return ans;
    }

    void dfs(TreeNode* root) {
        if (!root) return;
        dfs(root->left);

        cnt = prev != nullptr && prev->val == root->val ? cnt + 1 : 1;
        if (cnt > mx) {
            ans.clear();
            ans.push_back(root->val);
            mx = cnt;
        } else if (cnt == mx) {
            ans.push_back(root->val);
        }
        prev = root;
        dfs(root->right);
    }
};
C++
```

```
//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode* left;
//     TreeNode* right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, int y, int z) : val(x), left(y), right(z) {}
// };

class Solution {
public:
    TreeNode* prev;
    int mx, cnt;
    vector<int> ans;

    vector<int> findMode(TreeNode* root) {
        dfs(root);
        return ans;
    }

    void dfs(TreeNode* root) {
        if (!root) return;
        dfs(root->left);

        cnt = prev != nullptr && prev->val == root->val ? cnt + 1 : 1;
        if (cnt > mx) {
            ans.clear();
            ans.push_back(root->val);
            mx = cnt;
        } else if (cnt == mx) {
            ans.push_back(root->val);
        }
        prev = root;
        dfs(root->right);
    }
};
Java
Java
```

```
public class Solution {
    private int ans;
    private int cnt;
    private TreeNode prev;
    private List<int> res;

    public List<int> findMode(TreeNode root) {
        res = new List<int>();
        Dfs(root);
        List<int> ans = new List<int>(res.Count);
        for (int i = 0; i < res.Count; ++i) {
            ans[i] = res[i];
        }
        return ans;
    }

    private void Dfs(TreeNode root) {
        if (root == null) {
            return;
        }
        Dfs(root.left);
        cnt = prev != null && prev.val == root.val ? cnt + 1 : 1;
        if (cnt == ans) {
            res = new List<int>(new List<int>() { root.val });
            ans = cnt;
        }
        else if (cnt > ans) {
            res.Add(root.val);
        }
        prev = root;
        Dfs(root.right);
    }
}
```

Java

```
//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    private int ans;
    private int cnt;
    private TreeNode prev;
    private List<Integer> res;

    public List<int> findMode(TreeNode root) {
        res = new ArrayList<>();
        Dfs(root);
        List<int> ans = new List<int>(res.size());

        for (int i = 0; i < res.size(); ++i) {
            ans[i] = res.get(i);
        }
        return ans;
    }

    private void dfs(TreeNode root) {
        if (root == null) {
            return;
        }
        dfs(root.left);
        cnt = prev != null && prev.val == root.val ? cnt + 1 : 1;
        if (cnt == ans) {
            res = new ArrayList<>(Arrays.asList(root.val));
            ans = cnt;
        }
        else if (cnt > ans) {
            res.add(root.val);
        }
        prev = root;
        dfs(root.right);
    }
}
```

Java

```
public class Solution {
    private int ans;
    private int cnt;
    private TreeNode prev;
    private List<int> res;

    public List<int> findMode(TreeNode root) {
        res = new List<int>();
        Dfs(root);
        List<int> ans = new List<int>(res.Count);
        for (int i = 0; i < res.Count; ++i) {
            ans[i] = res[i];
        }
        return ans;
    }

    private void Dfs(TreeNode root) {
        if (root == null) {
            return;
        }
        Dfs(root.left);
        cnt = prev != null && prev.val == root.val ? cnt + 1 : 1;
        if (cnt == ans) {
            res = new List<int>(new List<int>() { root.val });
            ans = cnt;
        }
        else if (cnt > ans) {
            res.Add(root.val);
        }
        prev = root;
        Dfs(root.right);
    }
}
```

Go

```
//
// Definition for a binary tree node.
// type TreeNode struct {
//     Val int
//     Left *TreeNode
//     Right *TreeNode
// }
//

func findMode(root *TreeNode) []int {
    ans := []int{}
    var prev *TreeNode
    var ans []int
    var dfs func(*TreeNode) *TreeNode
    dfs = func(root *TreeNode) *TreeNode {
        if root == nil {
            return
        }
        dfs(root.Left)
        if prev != nil && prev.Val == root.Val {
            cnt++
        } else {
            cnt = 1
        }
        if cnt > ans {
            ans = []int{root.Val}
        }
        ans = cnt
    }
    else if cnt == ans {
        ans = append(ans, root.Val)
    }
    prev = root
    dfs(root.Right)
}
dfs(root)
return ans
}
```

Go

```
//
// Definition for a binary tree node.
// type TreeNode struct {
//     Val int
//     Left *TreeNode
//     Right *TreeNode
// }
//

func findMode(root *TreeNode) []int {
    ans := []int{}
    var prev *TreeNode
    var ans []int
    var dfs func(*TreeNode) *TreeNode
    dfs = func(root *TreeNode) *TreeNode {
        if root == nil {
            return
        }
        dfs(root.Left)
        if prev != nil && prev.Val == root.Val {
            cnt++
        } else {
            cnt = 1
        }
        if cnt > ans {
            ans = []int{root.Val}
        }
        ans = cnt
    }
    else if cnt == ans {
        ans = append(ans, root.Val)
    }
    prev = root
    dfs(root.Right)
}
dfs(root)
return ans
}
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val: int, left: Optional[TreeNode], right: Optional[TreeNode]) -> None:
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findMode(self, root: Optional[TreeNode]) -> List[int]:
        # Initialize state variables: prev value, current frequency count, maximum frequency, and modes
        list: List[int] = []
        self.prev = None
        self.currentCount = 0
        self.maxCount = 0
        self.modes = []

        # First in-order traversal to find the maximum frequency (maxCount).
        def inorder_first(node: Optional[TreeNode]) -> None:
            if not node:
                return
            inorder_first(node.left)
            # Update the count for the current node.
            if self.prev is None or node.val != self.prev:
                self.currentCount = 1
            else:
                self.currentCount += 1
            self.prev = node.val
            # Update maxCount if currentCount exceeds it.
            self.maxCount = max(self.maxCount, self.currentCount)
            inorder_first(node.right)

        inorder_first(root)

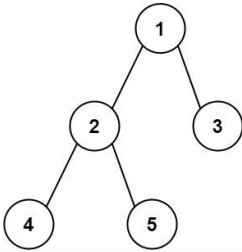
        # Second in-order traversal to collect all node values with frequency equal to maxCount.
        self.prev = None
        self.currentCount = 0
        self.modes = []

        def inorder_second(node: Optional[TreeNode]) -> None:
            if not node:
                return
            inorder_second(node.left)
            if self.prev is None or node.val != self.prev:
                self.currentCount = 1
            else:
                self.currentCount += 1
            self.prev = node.val
            if self.currentCount == self.maxCount:
                self.modes.append(node.val)
            inorder_second(node.right)

        inorder_second(root)

        return self.modes
```

Problem:
 Given the root of a binary tree, return the length of the diameter of the tree.
 The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root.
 The length of a path between two nodes is represented by the number of edges between them.
 Example 1:



Input: root = [1,2,3,4,5]
 Output: 3
 Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].
 Example 2:
 Input: root = [1,2]
 Output: 1
 Constraints:
 • The number of nodes in the tree is in the range [1, 104].
 • -100 <= Node.val <= 100

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        # Global variable to track the maximum diameter found
        self.max_diameter = 0

        def dfs(node):
            # Base case: If node is None, depth is 0.
            if not node:
                return 0

            # Compute the depth of left and right subtrees.
            left_depth = dfs(node.left)
            right_depth = dfs(node.right)

            # Update the global diameter: the path through this node
            self.max_diameter = max(self.max_diameter, left_depth + right_depth)

            # Return the depth of the tree rooted at this node.
            return max(left_depth, right_depth) + 1

        dfs(root)

        return self.max_diameter
    
```

Python

```

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode | None) -> int:
        ans = 0

        def maxDepth(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = maxDepth(root.left)
            r = maxDepth(root.right)
            ans = max(ans, l + r)
            return 1 + max(l, r)

        maxDepth(root)

        return ans
    
```

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return 0
            nonlocal ans
            left, right = dfs(root.left), dfs(root.right)
            ans = max(ans, left + right)
            return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans
    
```

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return 0
            nonlocal ans
            left, right = dfs(root.left), dfs(root.right)
            ans = max(ans, left + right)
            return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans
    
```

Java

Java

```

class Solution {
    public int diameterOfBinaryTree(TreeNode root) {
        maxDepth(root);
        return ans;
    }

    private int ans = 0;

    int maxDepth(TreeNode root) {
        if (root == null)
            return 0;

        final int l = maxDepth(root.left);
        final int r = maxDepth(root.right);
        ans = Math.max(ans, l + r);
        return 1 + Math.max(l, r);
    }
}
    
```

Java

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

class Solution {
    private int ans;

    public int diameterOfBinaryTree(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int l = dfs(root.left);
        int r = dfs(root.right);
        ans = Math.max(ans, l + r);
        return 1 + Math.max(l, r);
    }
}
    
```

Java

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

public class Solution {
    private int ans;

    public int diameterOfBinaryTree(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int l = dfs(root.left);
        int r = dfs(root.right);
        ans = Math.max(ans, l + r);
        return 1 + Math.max(l, r);
    }
}
    
```

Java

```

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

class Solution {
    private int ans;

    public int diameterOfBinaryTree(TreeNode root) {
        ans = 0;
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int left = dfs(root.left);
        int right = dfs(root.right);
        ans = Math.max(ans, left + right);
        return 1 + Math.max(left, right);
    }
}
    
```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     public int val;
//     public TreeNode left;
//     public TreeNode right;
//     public TreeNode(int val, TreeNode left=null, TreeNode right=null) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

public class Solution {
    private int ans;

    public int DiameterOfBinaryTree(TreeNode root) {
        dfs(root);
        return ans;
    }

    private void dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int l = dfs(root.left);
        int r = dfs(root.right);
        ans = Math.Max(ans, 1 + r);
        return 1 + Math.Max(l, r);
    }
}

```

JavaScript

Trees & BST - LeetCode - By Pattern

— 3085 —

LeetCode

```

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @return {number}

var diameterOfBinaryTree = function (root) {
    let ans = 0;
    const dfs = root => {
        if (!root) {
            return 0;
        }
        const [l, r] = [dfs(root.left), dfs(root.right)];
        ans = Math.max(ans, 1 + r);
        return 1 + Math.max(l, r);
    };
    dfs(root);
    return ans;
};

```

JavaScript

```

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @return {number}

var diameterOfBinaryTree = function (root) {
    let ans = 0;
    const dfs = root => {
        if (!root) {
            return 0;
        }
        const [l, r] = [dfs(root.left), dfs(root.right)];
        ans = Math.max(ans, 1 + r);
        return 1 + Math.max(l, r);
    };
    dfs(root);
    return ans;
};

```

TypeScript

TypeScript

Trees & BST - LeetCode - By Pattern

— 3086 —

LeetCode

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }

function diameterOfBinaryTree(root: TreeNode | null): number {
    let ans = 0;
    const dfs = (root: TreeNode | null): number => {
        if (!root) {
            return 0;
        }
        const [l, r] = [dfs(root.left), dfs(root.right)];
        ans = Math.max(ans, 1 + r);
        return 1 + Math.max(l, r);
    };
    dfs(root);
    return ans;
}

```

TypeScript

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }

function diameterOfBinaryTree(root: TreeNode | null): number {
    let ans = 0;
    const dfs = (root: TreeNode | null) => {
        if (!root) {
            return 0;
        }
        const [left, right] = root;
        const l = dfs(left);
        const r = dfs(right);
        res = Math.max(res, 1 + r);
        return Math.max(l, r) + 1;
    };
}

```

Trees & BST - LeetCode - By Pattern

— 3087 —

LeetCode

```

    };
    dfs(root);
    return res;
}

```

Rust

Rust

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Box<TreeNode>>,
//     pub right: Option<Box<TreeNode>>,
// }
//
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }

use std::rc::Rc;
impl Solution {
    pub fn diameter_of_binary_tree(root: Option<Box<TreeNode>> -> i32 {
        let mut ans = 0;
        fn dfs(root: Option<Box<TreeNode>>, ans: &mut i32) -> i32 {
            match root {
                Some(node) => {
                    let node = node.borrow();
                    let l = dfs(node.left.clone(), ans);
                    let r = dfs(node.right.clone(), ans);
                    *ans = (*ans).max(1 + r);
                }
                None => 0,
            }
        }
        dfs(root, &mut ans);
        ans
    }
}

```

Rust

Trees & BST - LeetCode - By Pattern

— 3088 —

LeetCode

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Box<TreeNode>>,
//     pub right: Option<Box<TreeNode>>,
// }
//
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }

use std::rc::Rc;
impl Solution {
    pub fn dfs(root: Option<Box<TreeNode>>, res: &mut i32) -> i32 {
        if root.is_none() {
            return 0;
        }
        let root = root.as_ref().unwrap().as_ref().borrow();
        let left = Self::dfs(root.left, res);
        let right = Self::dfs(root.right, res);
        *res = (*res).max(1 + right);
        left.max(right) + 1
    }

    pub fn diameter_of_binary_tree(root: Option<Box<TreeNode>> -> i32 {
        let mut res = 0;
        Self::dfs(root, &mut res);
        res
    }
}

```

[1] Trees & BST - Pattern Solution (matched from community answers)

Python

Trees & BST - LeetCode - By Pattern

— 3089 —

LeetCode

```

// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }

class Solution {
    def dfs(self, root: TreeNode) -> int:
        # Base case: If node is None, depth is 0.
        if not root:
            return 0

        # Compute the depth of left and right subtrees.
        left_depth = self.dfs(root.left)
        right_depth = self.dfs(root.right)

        # Return the max diameter (the path through this node)
        self.max_diameter = max(self.max_diameter, left_depth + right_depth)

        # Return the depth of the tree rooted at this node.
        return max(left_depth, right_depth) + 1

    def dfs(self):
        return self.max_diameter

```

#572

EASY

Subtree of Another Tree

LeetCode - Simplify - WalkCC - LeetCode - Editorial

Tree - DFS - String Matching

Like (100)

Problem:

Given the roots of two binary trees root and subRoot, return true if there is a subtree of root with the same structure and node values of subRoot and false otherwise.

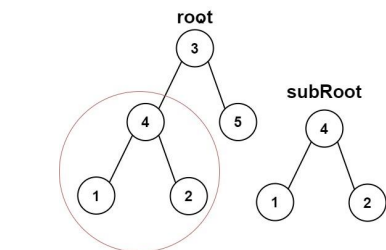
A subtree of a binary tree tree is a tree that consists of a node in tree and all of this node's descendants. The tree tree could also be considered as a subtree of itself.

Example 1:

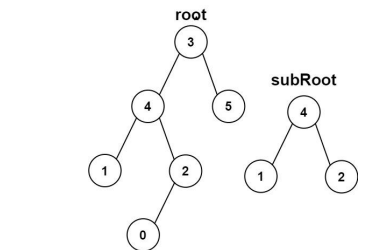
Trees & BST - LeetCode - By Pattern

— 3090 —

LeetCode



Input: root = [3,4,5,1,2], subRoot = [4,1,2]
 Output: true
 Example 2:



Input: root = [3,4,5,1,2,null,null,null,0], subRoot = [4,1,2]
 Output: false

- Constraints:
- The number of nodes in the root tree is in the range [1, 2000].
 - The number of nodes in the subRoot tree is in the range [1, 1000].
 - $-104 \leq \text{root.val} \leq 104$
 - $-104 \leq \text{subRoot.val} \leq 104$

```

Python
# Python solution for checking if subRoot is a subtree of root.
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
        # Helper function to check if two trees are identical.
        def isSameTree(s, t):
            # Both trees are None, trees match.
            if not s and not t:
                return True
            # One node is None or values differ.
            if not s or not t or s.val != t.val:
                return False
            # Recursively check left and right subtrees.
            return isSameTree(s.left, t.left) and isSameTree(s.right, t.right)

        # Base case: if root is None, then no subtree exists.
        if not root:
            return False

        # Check if current subtree is identical to subRoot.
        if isSameTree(root, subRoot):
            return True
        # Recursively check left and right subtrees.
        return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right, subRoot)
  
```

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
        def same(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
            if p is None or q is None:
                return p is q
            return p.val == q.val and same(p.left, q.left) and same(p.right, q.right)

        if root is None:
            return False
        return (
            same(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )

Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
        def dfs(root1: TreeNode):
            if root1 is None and root2 is None:
                return True
            if root1 is None or root2 is None:
                return False
            root1.val == root2.val
            and dfs(root1.left, root2.left)
            and dfs(root1.right, root2.right)

        if root is None:
            return False
        return (
            dfs(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )

Java
Java
  
```

```

class Solution {
    public boolean isSubtree(TreeNode s, TreeNode t) {
        if (s == null)
            return false;
        if (isSameTree(s, t))
            return true;
        return isSubtree(s.left, t) || isSubtree(s.right, t);
    }

    private boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null || q == null)
            return p == q;
        return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}

Java
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    public boolean isSubtree(TreeNode root, TreeNode subRoot) {
        if (root == null) {
            return false;
        }
        return same(root, subRoot) || isSubtree(root.left, subRoot)
            || isSubtree(root.right, subRoot);
    }

    private boolean same(TreeNode p, TreeNode q) {
        if (p == null || q == null) {
            return p == q;
        }
        return p.val == q.val && same(p.left, q.left) && same(p.right, q.right);
    }
}

Java
Java
  
```

```

// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    public boolean isSubtree(TreeNode root, TreeNode subRoot) {
        if (root == null) {
            return false;
        }
        return dfs(root, subRoot) || isSubtree(root.left, subRoot)
            || isSubtree(root.right, subRoot);
    }

    private boolean dfs(TreeNode root1, TreeNode root2) {
        if (root1 == null && root2 == null) {
            return true;
        }
        if (root1 == null || root2 == null) {
            return false;
        }
        return root1.val == root2.val && dfs(root1.left, root2.left)
            && dfs(root1.right, root2.right);
    }
}

JavaScript
JavaScript
  
```

```

// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val);
//     this.left = (left===undefined ? null : left);
//     this.right = (right===undefined ? null : right);
// }
//

/**
 * @param {TreeNode} root
 * @param {TreeNode} subRoot
 * @return {boolean}
 */
var isSubtree = function (root, subRoot) {
    const same = (p, q) => {
        if (!p || !q) {
            return p === q;
        }
        return p.val === q.val && same(p.left, q.left) && same(p.right, q.right);
    };

    if (!root) {
        return false;
    }
    return same(root, subRoot) || isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
};

JavaScript
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val);
//     this.left = (left===undefined ? null : left);
//     this.right = (right===undefined ? null : right);
// }
//

/**
 * @param {TreeNode} root
 * @param {TreeNode} subRoot
 * @return {boolean}
 */
var isSubtree = function (root, subRoot) {
    if (!root) return false;
    let dfs = function (root1, root2) {
        if (!root1 && !root2) {
            return true;
        }
        if (!root1 || !root2) {
            return false;
        }
        return root1.val == root2.val && dfs(root1.left, root2.left) && dfs(root1.right, root2.right);
    };

    return dfs(root, subRoot) || isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
};

TypeScript
  
```



```

TypeScript
//
 * Definition for a binary tree node.
 * class TreeNode {
 *   val: number
 *   left: TreeNode | null
 *   right: TreeNode | null
 *   constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 *   }
 * }
function isSubtree(root: TreeNode | null, subRoot: TreeNode | null): boolean {
  const same = (p: TreeNode | null, q: TreeNode | null): boolean => {
    if (!p || !q) {
      return p === q;
    }
    return p.val === q.val && same(p.left, q.left) && same(p.right, q.right);
  };
  if (!root) {
    return false;
  }
  return same(root, subRoot) || isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
}

```

```

TypeScript
//
 * Definition for a binary tree node.
 * class TreeNode {
 *   val: number
 *   left: TreeNode | null
 *   right: TreeNode | null
 *   constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 *   }
 * }
const dfs = (root: TreeNode | null, subRoot: TreeNode | null) => {
  if (root == null && subRoot == null) {
    return true;
  }
  if (root == null || subRoot == null || root.val !== subRoot.val) {
    return false;
  }
  return dfs(root.left, subRoot.left) && dfs(root.right, subRoot.right);
};
function isSubtree(root: TreeNode | null, subRoot: TreeNode | null): boolean {

```

```

if (root == null) {
  return false;
}
return dfs(root, subRoot) || isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
}

Rust
// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//   pub val: i32;
//   pub left: Option<Box<TreeNode>>;
//   pub right: Option<Box<TreeNode>>;
// }
// impl TreeNode {
//   #[inline]
//   pub fn new(val: i32) -> Self {
//     Self {
//       left: None,
//       right: None,
//     }
//   }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
  fn dfs(root: Option<Rc<RefCell<TreeNode>>, sub_root: Option<Rc<RefCell<TreeNode>>) -> bool {
    if root.is_none() && sub_root.is_none() {
      return true;
    }
  }
}

```

```

if root.is_none() || sub_root.is_none() {
  return false;
}
let root = root.as_ref().unwrap().borrow();
let sub_root = sub_root.as_ref().unwrap().borrow();
root.val == sub_root.val &&
  Self::dfs(root.left, sub_root.left) &&
  Self::dfs(root.right, sub_root.right)
}

fn help(
  root: Option<Rc<RefCell<TreeNode>>,
  sub_root: Option<Rc<RefCell<TreeNode>>
) -> bool {
  if root.is_none() {
    return false;
  }
  Self::dfs(root, sub_root) ||
  Self::help(root.as_ref().unwrap().borrow().left, sub_root) ||
  Self::help(root.as_ref().unwrap().borrow().right, sub_root)
}

pub fn is_subtree(
  root: Option<Rc<RefCell<TreeNode>>,
  sub_root: Option<Rc<RefCell<TreeNode>>
) -> bool {
  Self::help(root, sub_root)
}
}

```

[*] Trees & BST — Pattern Solution (matched from community answers)

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
        def dfs(root1, root2):
            if root1 is None and root2 is None:
                return True
            if root1 is None or root2 is None:
                return False
            return (
                root1.val == root2.val
                and dfs(root1.left, root2.left)
                and dfs(root1.right, root2.right)
            )
        if root is None:
            return False
        return (
            dfs(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )

```

#98

MEDIUM

Validate Binary Search Tree

LeetCode - SimplyLeetCode - WubCC - LeetCode - Editorial

Tree - DFS - BST

Like 100 100

Problem:

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:

```

graph TD
    2((2)) --> 1((1))
    2 --> 3((3))

```

Input: root = [2,1,3]
Output: true

Example 2:

```

graph TD
    5((5)) --> 1((1))
    5 --> 4((4))
    4 --> 3((3))
    4 --> 6((6))

```

Input: root = [5,1,4,null,null,3,6]
Output: false

Explanation: The root node's value is 5 but its right child's value is 4.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 231 <= Node.val <= 231 - 1

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        # Helper function to recursively validate nodes
        def validate(node, low, high):
            if not node:
                return True
            if not (low < node.val < high):
                return False
            return (
                validate(node.left, low, node.val)
                and validate(node.right, node.val, high)
            )
        return validate(root, float("-inf"), float("inf"))

Python
class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        def isValidBST(root: TreeNode | None, minNode: TreeNode | None, maxNode: TreeNode | None) -> bool:
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False
            return (
                isValidBST(root.left, minNode, root)
                and isValidBST(root.right, root, maxNode)
            )
        return isValidBST(root, None, None)

Python
class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        stack = []
        pred = None
        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()
            if pred and pred.val >= root.val:
                return False
            pred = root
            root = root.right
        return True

```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root: Optional[TreeNode]) -> bool:
            if root is None:
                return True
            if not dfs(root.left):
                return False
            nonlocal prev
            if prev < root.val:
                return False
            prev = root.val
            return dfs(root.right)
        prev = -inf
        return dfs(root)
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
```

```
class Solution: # recursive
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root, nl, mx):
            if not root:
                return True
            return nl < root.val < mx and dfs(root.left, nl, root.val) and dfs(root.right, root.val, mx)
        return dfs(root, -math.inf, math.inf)
```

```
class Solution: # iterative
    def isValidBST(self, root: TreeNode) -> bool:
```

```
# store lower, lower, upper: helper is a single queue
queue = [(root, None, None)]

# another option:
# queue = [(root, -math.inf, math.inf)]

while queue:
    node, lower, upper = queue.pop(0)
    if node is None:
        continue
    val = node.val
    if lower is not None and val <= lower:
        return False
    if upper is not None and val >= upper:
        return False
    queue.append((node.right, val, upper))
    queue.append((node.left, lower, val))

return True
```

#####

```
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def isValidBST(root: Optional[TreeNode],
            minNode: Optional[TreeNode], maxNode: Optional[TreeNode]) -> bool:
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False
            return isValidBST(root.left, minNode, root) and isValidBST(root.right, root, maxNode)

        return isValidBST(root, None, None)
```

#####

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: TreeNode) -> bool:

        def dfs(root):
            nonlocal prev
            if root is None:
                return True
            if not dfs(root.left):
```

Java

```
Java
class Solution {
    public boolean isValidBST(TreeNode root) {
        return isValidBST(root, null, null);
    }

    private boolean isValidBST(TreeNode root, TreeNode minNode, TreeNode maxNode) {
        if (root == null)
            return true;
        if (minNode != null && root.val <= minNode.val)
            return false;
        if (maxNode != null && root.val >= maxNode.val)
            return false;
        return //
            isValidBST(root.left, minNode, root) && //
            isValidBST(root.right, root, maxNode);
    }
}
```

Java

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        Queue stack = new ArrayDeque<>();
        TreeNode prev = null;

        while (root != null || !stack.isEmpty()) {
            while (root != null) {
                stack.push(root);
                root = root.left;
            }
            root = stack.pop();
            if (prev != null && prev.val >= root.val)
                return false;
            prev = root;
            root = root.right;
        }
        return true;
    }
}
```

Java

```
Java
//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private boolean prev;
```

```
    public boolean isValidBST(TreeNode root) {
        return dfs(root);
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {
            return true;
        }

        if (dfs(root.left)) {
            return false;
        }
        if (prev != null && prev.val >= root.val) {
            return false;
        }
        prev = root;
        return dfs(root.right);
    }
}
```

Java

```
Java
//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
public class Solution {
    private boolean prev;
```

Java

```
Java
//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private Integer prev;
```

```
    public boolean isValidBST(TreeNode root) {
        prev = null;
        return dfs(root);
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {
            return true;
        }
        if (dfs(root.left)) {
            return false;
        }
        if (prev != null && prev >= root.val) {
            return false;
        }
        prev = root.val;
        return dfs(root.right);
    }
}
```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     public int val;
//     public TreeNode left;
//     public TreeNode right;
//     public TreeNode(int val=0, TreeNode left=null, TreeNode right=null) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
//
public class Solution {
    private TreeNode prev;

    public bool IsValidBST(TreeNode root) {
        prev = null;
        return dfs(root);
    }

    private bool dfs(TreeNode root) {
        if (root == null) {
            return true;
        }

        if (!dfs(root.left)) {
            return false;
        }
        if (prev != null && prev.val == root.val) {
            return false;
        }
        prev = root;
        if (!dfs(root.right)) {
            return false;
        }
        return true;
    }
}

```

JavaScript
JavaScript

```

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @return {boolean}
//
var isValidBST = function (root) {
    let prev = null;
    const dfs = root => {
        if (!root) {
            return true;
        }
        if (!dfs(root.left)) {
            return false;
        }
        if (prev && prev.val == root.val) {
            return false;
        }
        prev = root;
        return dfs(root.right);
    };
    return dfs(root);
};

```

JavaScript

```

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @return {boolean}
//
var isValidBST = function (root) {
    let prev = null;

    let dfs = function (root) {
        if (!root) {
            return true;
        }
        if (!dfs(root.left)) {
            return false;
        }
        if (prev && prev.val == root.val) {
            return false;
        }
        prev = root;
        return dfs(root.right);
    };
    return dfs(root);
};

```

```

        prev = root;
        if (!dfs(root.right)) {
            return false;
        }
        return true;
    };
    return dfs(root);
};

```

TypeScript
TypeScript

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }
//
function isValidBST(root: TreeNode | null): boolean {
    let prev: TreeNode | null = null;
    const dfs = (root: TreeNode | null): boolean => {
        if (!root) {
            return true;
        }
        if (!dfs(root.left)) {
            return false;
        }
        if (prev && prev.val == root.val) {
            return false;
        }
        prev = root;
        return dfs(root.right);
    };
    return dfs(root);
}

```

TypeScript

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }
//
function isValidBST(root: TreeNode | null): boolean {
    let prev: TreeNode | null = null;
    const dfs = (root: TreeNode | null): boolean => {
        if (!root) {
            return true;
        }
        if (!dfs(root.left)) {
            return false;
        }
        if (prev && prev.val == root.val) {
            return false;
        }
        prev = root;
        return dfs(root.right);
    };
    return dfs(root);
}

```

Rust
Rust

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<TreeNode>>,
//     pub right: Option<Rc<TreeNode>>,
// }
//
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
//
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    fn dfs(root: Option<Rc<TreeNode>>, prev: &mut Option<i32>) -> bool {
        if root.is_none() {
            return true;
        }
        let root = root.as_ref().unwrap().borrow();
        if !Self::dfs(&root.left, prev) {
            return false;
        }
        if prev.is_some() && prev.unwrap() == root.val {
            return false;
        }
        *prev = Some(root.val);
        Self::dfs(&root.right, prev)
    }

    pub fn is_valid_bst(root: Option<Rc<TreeNode>>) -> bool {
        Self::dfs(&root, &mut None)
    }
}

```

Rust

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<TreeNode>>,
//     pub right: Option<Rc<TreeNode>>,
// }
//
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
//
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    fn dfs(root: Option<Rc<TreeNode>>, prev: &mut Option<i32>) -> bool {
        if root.is_none() {
            return true;
        }
        let root = root.as_ref().unwrap().borrow();
        if !Self::dfs(&root.left, prev) {
            return false;
        }
        if prev.is_some() && prev.unwrap() == root.val {
            return false;
        }
        *prev = Some(root.val);
        Self::dfs(&root.right, prev)
    }

    pub fn is_valid_bst(root: Option<Rc<TreeNode>>) -> bool {
        Self::dfs(&root, &mut None)
    }
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)
Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root: Optional[TreeNode]) -> bool:
            if root is None:
                return True
            if not dfs(root.left):
                return False
            nonlocal prev
            if prev == root.val:
                return False
            prev = root.val
            return dfs(root.right)
        prev = -1
        return dfs(root)
```

#102 MEDIUM

Binary Tree Level Order Traversal

[LeetCode](#) · [SimpleLine](#) · [WuJieCC](#) · [LeetCode](#) · [Editorial](#)

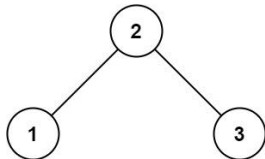
Tree · BFS

List: Grid 169

Problem:

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Trees & BST · LevelMastery · By Pattern

— 3115 —

LevelMastery

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- -1000 <= Node.val <= 1000

Python

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from collections import deque

class Solution:
    def levelOrder(self, root: TreeNode) -> List[List[int]]:
        # Return an empty list if the tree is empty.
        if not root:
            return []

        result = [] # Final result list containing values of each level.
        queue = deque([root]) # Initiation the queue with the root node.

        # Process nodes level by level.
        while queue:
            level_size = len(queue) # Number of nodes at the current level.
            level_nodes = [] # List to hold node values at the current level.

            for _ in range(level_size):
                node = queue.popleft() # Dequeue the next node.

                level_nodes.append(node.val)
                # Enqueue left child if it exists.
                if node.left:
                    queue.append(node.left)
                # Enqueue right child if it exists.
                if node.right:
                    queue.append(node.right)
            # Append the current level's values to the result.
            result.append(level_nodes)

        return result
```

Python

Trees & BST · LevelMastery · By Pattern

— 3116 —

LevelMastery

```
class Solution:
    def levelOrder(self, root: TreeNode | None) -> List[List[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            curLevel = []
            for _ in range(len(q)):
                node = q.popleft()
                curLevel.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(curLevel)

        return ans
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t)
        return ans
```

Python

Trees & BST · LevelMastery · By Pattern

— 3117 —

LevelMastery

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t)
        return ans
```

C++

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<vector<int>> ans;
        queue<TreeNode*> q([root]);

        while (!q.empty()) {
            vector<int> curLevel;
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode* node = q.front();
                q.pop();
                curLevel.push_back(node->val);
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
            ans.push_back(curLevel);
        }

        return ans;
    }
};
```

C++

Trees & BST · LevelMastery · By Pattern

— 3118 —

LevelMastery

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> ans;
        if (!root) return ans;
        queue<TreeNode*> q([root]);
        while (!q.empty()) {
            vector<int> t;
            for (int sz = q.size(); sz > 0; --sz) {
                auto node = q.front();
                q.pop();
                t.push_back(node->val);
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
            ans.push_back(t);
        }
        return ans;
    }
};
```

C++

Trees & BST · LevelMastery · By Pattern

— 3119 —

LevelMastery

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> ans;
        if (!root) return ans;
        queue<TreeNode*> q([root]);
        while (!q.empty()) {
            vector<int> t;
            for (int sz = q.size(); sz > 0; --sz) {
                auto node = q.front();
                q.pop();
                t.push_back(node->val);
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
            ans.push_back(t);
        }
        return ans;
    }
};
```

Java

Java

Trees & BST · LevelMastery · By Pattern

— 3120 —

LevelMastery

```

class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        if (root == null)
            return new ArrayList<>();

        List<List<Integer>> ans = new ArrayList<>();
        Queue<TreeNode> q = new ArrayDeque<>(List.of(root));

        while (!q.isEmpty()) {
            List<Integer> curLevel = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                curLevel.add(node.val);
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            ans.add(curLevel);
        }
        return ans;
    }
}

```

Java

```

/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        if (root == null) {
            return ans;
        }
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        while (!q.isEmpty()) {
            List<Integer> t = new ArrayList<>();

```

```

        for (int n = q.size(); n > 0; --n) {
            TreeNode node = q.poll();
            t.add(node.val);
            if (node.left != null) {
                q.offer(node.left);
            }
            if (node.right != null) {
                q.offer(node.right);
            }
        }
        ans.add(t);
        return ans;
    }
}

```

Java

```

/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        if (root == null) {
            return ans;
        }
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        while (!q.isEmpty()) {
            List<Integer> t = new ArrayList<>();

```

JavaScript

```

/*
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/*
 * @param {TreeNode} root
 * @return {number[][]}
 */
var levelOrder = function (root) {
    const ans = [];
    if (!root) {
        return ans;
    }
    const q = [root];
    while (q.length) {
        const t = [];
        const qq = [];
        for (const { val, left, right } of q) {
            t.push(val);
            left && qq.push(left);
            right && qq.push(right);
        }
        ans.push(t);
        q.splice(0, q.length, ...qq);
    }
    return ans;
};

```

JavaScript

```

/*
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *     this.val = (val===undefined ? 0 : val)
 *     this.left = (left===undefined ? null : left)
 *     this.right = (right===undefined ? null : right)
 * }
 */
/*
 * @param {TreeNode} root
 * @return {number[][]}
 */
var levelOrder = function (root) {
    let ans = [];
    if (!root) {
        return ans;
    }
    let q = [root];
    while (q.length) {
        let t = [];
        for (let n = q.length; n--;) {
            const { val, left, right } = q.shift();
            t.push(val);
            left && q.push(left);
            right && q.push(right);
        }
        ans.push(t);
        return ans;
    }
};

```

TypeScript

TypeScript

```

/*
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number
 *     left: TreeNode | null
 *     right: TreeNode | null
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val)
 *         this.left = (left===undefined ? null : left)
 *         this.right = (right===undefined ? null : right)
 *     }
 * }
 */
function levelOrder(root: TreeNode | null): number[][] {
    const ans: number[][] = [];
    if (!root) {
        return ans;
    }
    const q: TreeNode[] = [root];
    while (q.length) {
        const t: number[] = [];
        const qq: TreeNode[] = [];
        for (const { val, left, right } of q) {
            t.push(val);
            left && qq.push(left);
            right && qq.push(right);
        }
        ans.push(t);
        q.splice(0, q.length, ...qq);
    }
    return ans;
}

```

TypeScript

```

/*
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number
 *     left: TreeNode | null
 *     right: TreeNode | null
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val)
 *         this.left = (left===undefined ? null : left)
 *         this.right = (right===undefined ? null : right)
 *     }
 * }
 */
function levelOrder(root: TreeNode | null): number[][] {
    const res = [];
    if (!root) {
        return res;
    }
    const queue = [root];
    while (queue.length !== 0) {
        const n = queue.length;
        res.push(
            new Array(n).fill(0).map(() => {
                const { val, left, right } = queue.shift();
                left && queue.push(left);
                right && queue.push(right);
                return val;
            })
        );
        return res;
    }
}

```

Go

Go

```

//
// Definition for a binary tree node.
//
type TreeNode struct {
    *Val int
    *Left *TreeNode
    *Right *TreeNode
    *}
//
func levelOrder(root *TreeNode) ans [][]int {
    if root == nil {
        return
    }
    q := []*TreeNode{root}
    for len(q) > 0 {
        t := [][2]int{}
        for a := len(q); a > 0; a-- {
            node := q[0]
            q = q[1:]
            t = append(t, node.Val)
            if node.Left != nil {
                q = append(q, node.Left)
            }
            if node.Right != nil {
                q = append(q, node.Right)
            }
        }
        ans = append(ans, t)
    }
    return
}

```

Go

```

//
// Definition for a binary tree node.
//
type TreeNode struct {
    *Val int
    *Left *TreeNode
    *Right *TreeNode
    *}
//
func levelOrder(root *TreeNode) ans [][]int {
    if root == nil {
        return
    }
    q := []*TreeNode{root}
    for len(q) > 0 {
        t := [][2]int{}
        for a := len(q); a > 0; a-- {
            node := q[0]
            q = q[1:]
            t = append(t, node.Val)
            if node.Left != nil {
                q = append(q, node.Left)
            }
            if node.Right != nil {
                q = append(q, node.Right)
            }
        }
        ans = append(ans, t)
    }
    return
}

```

Trees & BST - LeetCode - By Pattern

— 3127 —

LeetCode

```

}
ans = append(ans, t)
}
return
}

```

Rust

```

// Definition for a binary tree node.
//
#[derive(Debug, PartialEq, Eq)]
pub struct TreeNode {
    pub val: i32,
    pub left: Option<Rc<TreeNode>>,
    pub right: Option<Rc<TreeNode>>,
}
//
impl TreeNode {
    //
    //
    // pub fn new(val: i32) -> Self {
    //     Self {
    //         val,
    //         left: None,
    //         right: None,
    //     }
    // }
    //
    use std::rc::Rc;
    use std::collections::VecDeque;
    impl Solution {
        pub fn level_order(root: Option<Rc<TreeNode>>) -> Vec<Vec<i32>> {
            let mut res = vec![];
            if root.is_none() {

```

Trees & BST - LeetCode - By Pattern

— 3128 —

LeetCode

```

return res;
}
let mut queue: VecDeque<Option<Rc<TreeNode>>> = vec![root.clone().into()];
while !queue.is_empty() {
    let n = queue.pop_front();
    res.push_back(n.clone().into());
    if n.is_some() {
        let n = n.unwrap();
        queue.push_back(n.left.clone().into());
        queue.push_back(n.right.clone().into());
    }
}
res
}
}
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)

Python

```

// Definition for a binary tree node.
//
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
//
from collections import deque
//
class Solution:
    def levelOrder(self, root: TreeNode) -> List[List[int]]:
        # Return an empty list if the tree is empty.
        if not root:
            return []
        result = [] # Final result list containing values of each level.
        queue = deque([root]) # Initialize the queue with the root node.
        # Process nodes level by level.
        while queue:
            level_size = len(queue) # Number of nodes at the current level.
            level_nodes = [] # List to hold node values at the current level.
            for _ in range(level_size):
                node = queue.popleft() # Dequeue the next node.

```

Trees & BST - LeetCode - By Pattern

— 3129 —

LeetCode

```

level_nodes.append(node.val)
# Dequeue left child if it exists.
if node.left:
    queue.append(node.left)
# Dequeue right child if it exists.
if node.right:
    queue.append(node.right)
# Append the current level's values to the result.
result.append(level_nodes)
return result

```

#103

MEDIUM

Binary Tree Zigzag Level Order Traversal

LeetCode - [SimplifyLeetCode](#) - [WuJieCC](#) - [LeetCode](#) - [Editorial](#)

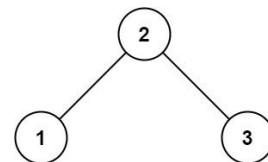
Tree - BFS

Little Grind 169

Problem:

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- -100 <= Node.val <= 100

Trees & BST - LeetCode - By Pattern

— 3130 —

LeetCode

>> Brute Force (Trees & BST) Interview Prep

Python

```

// Brute Force Approach
//
class Solution:
    def zigzagLevelOrder(self, root):
        # Brute Force - collect inorder, then process sorted list
        self.res = []
        def collect(node):
            if not node: return
            collect(node.left)
            self.res.append(node.val)
            collect(node.right)
        collect(root)
        return self.res if self.res else []

```

Python

```

// Definition for a binary tree node.
//
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
//
def zigzagLevelOrder(self, root):
    # If the tree is empty, return an empty list
    if not root:
        return []
    # Initialize the result list and the queue with the root node.
    result = []
    queue = [root]
    left_to_right = True # Flag to indicate the current direction of traversal
    # Process the tree level by level.
    while queue:
        level_size = len(queue)
        current_level = []
        # Process all nodes for the current level.
        for _ in range(level_size):
            node = queue.pop(0) # Dequeue the node from the front of the queue

```

Trees & BST - LeetCode - By Pattern

— 3131 —

LeetCode

```

current_level.append(node.val) # Append the node's value
# Dequeue the left child if it exists.
if node.left:
    queue.append(node.left)
# Dequeue the right child if it exists.
if node.right:
    queue.append(node.right)
# If the current level's order is right-to-left, reverse the list.
if not left_to_right:
    current_level.reverse()
# Append the current level's values to the result.
result.append(current_level)
# Toggle the direction for the next level.
left_to_right = not left_to_right
return result

```

Python

```

// Definition for a binary tree node.
//
class Solution:
    def zigzagLevelOrder(self, root: TreeNode | None) -> List[List[int]]:
        if not root:
            return []
        ans = []
        dq = collections.deque([root])
        isLeftToRight = True
        while dq:
            curLevel = []
            for _ in range(len(dq)):
                if isLeftToRight:
                    node = dq.popleft()
                    curLevel.append(node.val)
                    if node.left:
                        dq.append(node.left)
                    if node.right:
                        dq.append(node.right)
                else:
                    node = dq.pop()
                    curLevel.append(node.val)
                    if node.right:
                        dq.append(node.right)
                    if node.left:
                        dq.append(node.left)
            ans.append(curLevel)
            isLeftToRight = not isLeftToRight
        return ans

```

Python

Trees & BST - LeetCode - By Pattern

— 3132 —

LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        ans = []
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t if len(t) % 2 == 1 else t[::-1])
            left = not left
        return ans
```

```
Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
'''
can also use list.index()
'''
my_list = [2, 3, 4]
''' my_list.index(3, 1) # Insert 3 at the head of the list
''' print(my_list) # Output: [1, 2, 3, 4]

''' a = deque()
''' a
''' a.append(1)
''' a
''' a.append(2)
''' a
''' a.append(2)
''' a
''' a.append(1, 2, 3)
''' a
''' a.append(0, 555)
''' a
```

```
'''
Traverse root, return call back
'''
File "main.py", line 1, in module
TypeError: deque.append() takes exactly one argument (2 given)
''' a = deque([0, 555])
''' a
''' a.append(1, 2, 3)
''' a

from collections import deque

class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        ans = []
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t if len(t) % 2 == 1 else t[::-1])
            left = not left
        return ans
```

```
'''
Traverse root, return call back
'''
File "main.py", line 1, in module
TypeError: deque.append() takes exactly one argument (2 given)
''' a = deque([0, 555])
''' a
''' a.append(1, 2, 3)
''' a
''' a.append(0, 555)
''' a

from collections import deque

class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        ans = []
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t if len(t) % 2 == 1 else t[::-1])
            left = not left
        return ans
```

```
C++
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };

class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<vector<int>> ans;
        deque<TreeNode*> dq([root]);
        bool isLeftToRight = true;

        while (!dq.empty()) {
            vector<int> curLevel;
            for (int sz = dq.size(); sz > 0; --sz) {
                if (!isLeftToRight) {
                    TreeNode* node = dq.front();
                    dq.pop_front();
                    curLevel.push_back(node->val);
                    if (node->left)
                        dq.push_back(node->left);
                    if (node->right)
                        dq.push_back(node->right);
                } else {
                    TreeNode* node = dq.back();
                    dq.pop_back();
                    curLevel.push_back(node->val);
                    if (node->right)
                        dq.push_back(node->right);
                    if (node->left)
                        dq.push_back(node->left);
                }
                ans.push_back(curLevel);
                isLeftToRight = !isLeftToRight;
            }
            return ans;
        }
    }
};
```

```
C++
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };

class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> ans;
        if (!root)
            return ans;
        queue<TreeNode*> q([root]);
        int left = 1;
        while (!q.empty()) {
            vector<int> t;
            for (int sz = q.size(); sz > 0; --sz) {
                auto node = q.front();
                q.pop();
                t.push_back(node->val);
                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }
            if (!left)
                reverse(t.begin(), t.end());
            ans.push_back(t);
            left = !left;
        }
        return ans;
    }
};
```

```
Java
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        Deque<TreeNode> q = new ArrayDeque<>([root]);
        boolean isLeftToRight = true;

        while (!q.isEmpty()) {
            List<Integer> curLevel = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                if (!isLeftToRight)
                    curLevel.add(node.val);
                else
                    curLevel.add(q.offer(node.val));
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            ans.add(curLevel);
            isLeftToRight = !isLeftToRight;
        }

        return ans;
    }
}
```

```
Java
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        if (root == null)
            return ans;
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        boolean left = true;
        while (!q.isEmpty()) {
            List<Integer> curLevel = new ArrayList<>();
            for (int sz = q.size(); sz > 0; --sz) {
                TreeNode node = q.poll();
                if (!left)
                    curLevel.add(node.val);
                else
                    curLevel.add(q.offer(node.val));
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
            ans.add(curLevel);
            left = !left;
        }

        return ans;
    }
}
```

Trees & BST · LeetCode — By Pattern — 3139 — LeetCode

Trees & BST · LeetMastery — By Pattern — 3140 — LeetMastery

Trees & BST · LeetCode — By Pattern — 3141 — LeetCode

Trees & BST · LeetMastery — By Pattern — 3142 — LeetMastery

Trees & BST · LeetCode — By Pattern — 3143 — LeetCode

Trees & BST · LeetMastery — By Pattern — 3144 — LeetMastery


```

//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
// };
//
func zigzagLevelOrder(root *TreeNode) ans [][]int {
    if root == nil {
        return
    }
    q := []*TreeNode{root}
    left := true
    for len(q) > 0 {
        t := (len(q))
        for n := 0; n < t; n++ {
            node := q[0]
            q = q[1:]
            t = append(t, node.Val)
            if node.Left != nil {
                q = append(q, node.Left)
            }
            if node.Right != nil {
                q = append(q, node.Right)
            }
        }
        if left {
            for i, j := 0, len(t)-1; i < j; i, j = i+1, j-1 {
                t[i], t[j] = t[j], t[i]
            }
        }
        ans = append(ans, t)
        left = !left
    }
    return
}

```

Go

```

//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
// };
//
func zigzagLevelOrder(root *TreeNode) ans [][]int {
    if root == nil {
        return
    }
    q := []*TreeNode{root}
    left := true
    for len(q) > 0 {
        t := (len(q))
        for n := 0; n < t; n++ {
            node := q[0]
            q = q[1:]
            t = append(t, node.Val)
            if node.Left != nil {
                q = append(q, node.Left)
            }
            if node.Right != nil {
                q = append(q, node.Right)
            }
        }
        if left {
            for i, j := 0, len(t)-1; i < j; i, j = i+1, j-1 {
                t[i], t[j] = t[j], t[i]
            }
        }
        ans = append(ans, t)
        left = !left
    }
    return
}

```

Rust

Rust

```

// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
// };
//
// #include <bits/stdc++.h>
// using namespace std;
//
// int main() {
//     int n;
//     cin >> n;
//     vector<int> preorder(n);
//     vector<int> inorder(n);
//     for (int i = 0; i < n; i++) {
//         cin >> preorder[i];
//         cin >> inorder[i];
//     }
//     TreeNode* root = buildTree(preorder, inorder);
//     return 0;
// }
//
// TreeNode* buildTree(vector<int> &preorder, vector<int> &inorder) {
//     if (preorder.empty()) return nullptr;
//     int rootVal = preorder[0];
//     int rootIndex = -1;
//     for (int i = 0; i < inorder.size(); i++) {
//         if (inorder[i] == rootVal) {
//             rootIndex = i;
//             break;
//         }
//     }
//     TreeNode* root = new TreeNode(rootVal);
//     vector<int> preLeft(preorder.begin() + 1, preorder.begin() + 1 + rootIndex);
//     vector<int> preRight(preorder.begin() + 1 + rootIndex, preorder.end());
//     vector<int> inLeft(inorder.begin(), inorder.begin() + rootIndex);
//     vector<int> inRight(inorder.begin() + rootIndex + 1, inorder.end());
//     root->left = buildTree(preLeft, inLeft);
//     root->right = buildTree(preRight, inRight);
//     return root;
// }

```

[*] Trees & BST - Pattern Solution (matched from community answers)
Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
#
def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
    # If the tree is empty, return an empty list
    if not root:
        return []
    # Initialize the result list and the queue with the root node.
    result = []
    queue = [root]
    left_to_right = True # Flag to indicate the current direction of traversal
    # Process the tree level by level.
    while queue:
        level_size = len(queue)
        current_level = []
        # Process all nodes for the current level.
        for i in range(level_size):
            node = queue.pop(0) # Remove the node from the front of the queue
            current_level.append(node.val) # Append the node's value
            # Enqueue the left child if it exists.
            if node.left:
                queue.append(node.left)
            # Enqueue the right child if it exists.
            if node.right:
                queue.append(node.right)
        # If the current level's order is right-to-left, reverse the list.
        if not left_to_right:
            current_level.reverse()
        # Append the current level's values to the result.
        result.append(current_level)
        # Toggle the direction for the next level.
        left_to_right = not left_to_right
    return result

```

#105

MEDIUM

Construct Binary Tree from Preorder and Inorder Traversal

LeetCode - Simple - Medium - Easy - LeetCode - Editorial

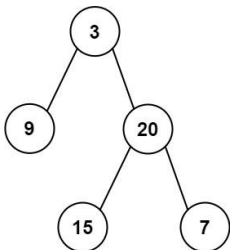
Array - Hash Table - Tree - DFS

Like Grind 109

Problem:

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]
Output: [-1]

Constraints:

- 1 <= preorder.length <= 3000
- inorder.length == preorder.length
- 3000 <= preorder[i], inorder[i] <= 3000
- preorder and inorder consist of unique values.
- Each value of inorder also appears in preorder.
- preorder is guaranteed to be the preorder traversal of the tree.
- inorder is guaranteed to be the inorder traversal of the tree.

Python
Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
#
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> TreeNode:
        # Create a hash map to store the index of each element in the inorder array
        inorder_index = {val: i for i, val in enumerate(inorder)}
        # Recursive helper function to build the tree
        def helper(pre_start, in_left, in_right):
            # Base case: if there are no elements to construct subtree
            if in_left > in_right:
                return None
            # Pick the pre_start element as a root
            root_val = preorder[pre_start]
            root = TreeNode(root_val)
            # Get the inorder index of the root
            in_index = inorder_index[root_val]
            # Number of nodes in left subtree
            left_subtree_size = in_index - in_left
            # Recursively build the left subtree
            root.left = helper(pre_start + 1, in_left, in_index - 1)
            # Recursively build the right subtree
            root.right = helper(pre_start + left_subtree_size + 1, in_index + 1, in_right)
            return root
        # Initiate recursion from the start of preorder and full inorder range
        return helper(0, 0, len(inorder) - 1)

```

Python

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private Map<Integer, Integer> d = new HashMap<>();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        int n = preorder.length;
        this.preorder = preorder;
        for (let i = 0; i < n; i++) {
            d.put(inorder[i], i);
        }

        return dfs(0, 0, n);
    }

    private TreeNode dfs(int l, int j, int n) {
        if (n == 0) {
            return null;
        }
        let v = preorder[l];
        let k = d.get(v);
        TreeNode t = dfs(l + 1, j, k - j);
        TreeNode r = dfs(l + 1 + k - j, k + 1, n - 1 - (k - j));
        return new TreeNode(v, t, r);
    }
}

```

Java

```

// Definition for a binary tree node.
// @param {number} val
// @param {TreeNode} left
// @param {TreeNode} right
// @return {TreeNode}
function TreeNode(val, left, right) {
    this.val = val;
    this.left = left;
    this.right = right;
}

/**
 * @param {number[]} preorder
 * @param {number[]} inorder
 * @return {TreeNode}
 */
var buildTree = function(preorder, inorder) {
    let d = new Map();
    for (let i = 0; i < inorder.length; i++) {
        d.set(inorder[i], i);
    }

    let root = dfs(preorder, 0, 0, preorder.length);
    return root;

    function dfs(preorder, l, r, n) {
        if (n == 0) return null;
        let v = preorder[l];
        let k = d.get(v);
        let left = dfs(preorder, l + 1, j, k - j);
        let right = dfs(preorder, l + 1 + k - j, k + 1, n - 1 - (k - j));
        return new TreeNode(v, left, right);
    }
}

```

JavaScript

JavaScript

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private Map<Integer, Integer> d = new HashMap<>();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        int n = preorder.length;
        for (let i = 0; i < n; i++) {
            d.put(inorder[i], i);
        }

        return dfs(0, 0, n);
    }

    private TreeNode dfs(int l, int j, int n) {
        if (n == 0) {
            return null;
        }
        let v = preorder[l];
        let k = d.get(v);
        let left = dfs(l + 1, j, k - j);
        let right = dfs(l + 1 + k - j, k + 1, n - 1 - (k - j));
        return new TreeNode(v, left, right);
    }
}

```

JavaScript

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private Map<Integer, Integer> d = new HashMap<>();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        int n = preorder.length;
        for (let i = 0; i < n; i++) {
            d.put(inorder[i], i);
        }

        return dfs(0, 0, n);
    }

    private TreeNode dfs(int l, int j, int n) {
        if (n == 0) {
            return null;
        }
        let v = preorder[l];
        let k = d.get(v);
        let left = dfs(l + 1, j, k - j);
        let right = dfs(l + 1 + k - j, k + 1, n - 1 - (k - j));
        return new TreeNode(v, left, right);
    }
}

```

TypeScript

TypeScript

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private Map<Integer, Integer> d = new HashMap<>();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        int n = preorder.length;
        for (let i = 0; i < n; i++) {
            d.put(inorder[i], i);
        }

        return dfs(0, 0, n);
    }

    private TreeNode dfs(int l, int j, int n) {
        if (n == 0) {
            return null;
        }
        let v = preorder[l];
        let k = d.get(v);
        let left = dfs(l + 1, j, k - j);
        let right = dfs(l + 1 + k - j, k + 1, n - 1 - (k - j));
        return new TreeNode(v, left, right);
    }
}

```

TypeScript

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private Map<Integer, Integer> d = new HashMap<>();

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        int n = preorder.length;
        for (let i = 0; i < n; i++) {
            d.put(inorder[i], i);
        }

        return dfs(0, 0, n);
    }

    private TreeNode dfs(int l, int j, int n) {
        if (n == 0) {
            return null;
        }
        let v = preorder[l];
        let k = d.get(v);
        let left = dfs(l + 1, j, k - j);
        let right = dfs(l + 1 + k - j, k + 1, n - 1 - (k - j));
        return new TreeNode(v, left, right);
    }
}

```

Go

Go


```

// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// class Solution {
//     def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
//         ans = []
//         if root is None:
//             return ans
//         q = deque([root])
//         while q:
//             ans.append(q[-1].val) # add last node of previous level traversal results
//             for _ in range(len(q)):
//                 node = q.popleft()
//                 if node.left:
//                     q.append(node.left)
//                 if node.right:
//                     q.append(node.right)
//             return ans
//         return ans
// }

// =====
// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// class Solution {
//     def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
//         if root is None:
//             return []
//         queue = deque([root])
//         while queue:
//             n = queue.pop()
//             if n.right:
//                 queue.append(n.right)
//             if n.left:
//                 queue.append(n.left)
//         return []
// }

```

```

class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        if (root == nullptr)
            return {};

        vector<int> ans;
        queue<TreeNode*> q({root});

        while (!q.empty()) {
            int size = q.size();
            for (int i = 0; i < size; ++i) {
                TreeNode* node = q.front();
                q.pop();

                if (i == size - 1)
                    ans.push_back(node->val);

                if (node->left)
                    q.push(node->left);
                if (node->right)
                    q.push(node->right);
            }

            return ans;
        }
    }
};

```

```

// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };

// class Solution {
// public:
//     vector<int> rightSideView(TreeNode* root) {
//         vector<int> ans;
//         if (!root)
//             return ans;
//         queue<TreeNode*> q({root});
//         while (!q.empty()) {
//             ans.push_back(q.back()->val);
//             for (int i = q.size()-1; i > 0; --i) {
//                 q.pop();
//                 if (q.back()->right)
//                     q.push(q.back()->right);
//                 if (q.back()->left)
//                     q.push(q.back()->left);
//             }
//             return ans;
//         }
//     }
// };

```

```

// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };

// class Solution {
// public:
//     vector<int> rightSideView(TreeNode* root) {
//         vector<int> ans;
//         if (!root)
//             return ans;
//         queue<TreeNode*> q({root});
//         while (!q.empty()) {
//             ans.push_back(q.back()->val);
//             for (int i = q.size()-1; i > 0; --i) {
//                 q.pop();
//                 if (q.back()->right)
//                     q.push(q.back()->right);
//                 if (q.back()->left)
//                     q.push(q.back()->left);
//             }
//             return ans;
//         }
//     }
// };

```

```

class Solution {
public:
    List<Integer> rightSideView(TreeNode* root) {
        if (root == null)
            return new ArrayList<>();

        List<Integer> ans = new ArrayList<>();
        Queue<TreeNode> q = new ArrayDeque<>({root});

        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; ++i) {
                TreeNode node = q.poll();
                if (i == size - 1)
                    ans.add(node.val);
                if (node.left != null)
                    q.offer(node.left);
                if (node.right != null)
                    q.offer(node.right);
            }
        }

        return ans;
    }
}

```

```

// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };

// class Solution {
// public:
//     List<Integer> rightSideView(TreeNode* root) {
//         List<Integer> ans = new ArrayList<>();
//         if (root == null)
//             return ans;
//         Queue<TreeNode> q = new ArrayDeque<>();
//         q.offer(root);
//         while (!q.isEmpty()) {
//             ans.add(q.poll().val);
//             for (int i = q.size()-1; i > 0; --i) {
//                 TreeNode node = q.poll();
//                 if (node.right != null)
//                     q.offer(node.right);
//                 if (node.left != null)
//                     q.offer(node.left);
//             }
//             return ans;
//         }
//     }
// };

```

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> ans = new ArrayList<>();
        if (root == null) {
            return ans;
        }
        Deque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        while (!q.isEmpty()) {
            ans.add(q.getLast().val);

            for (int i = q.size() - 1; i >= 0; --i) {
                TreeNode node = q.poll();
                if (node.left != null) {
                    q.offer(node.left);
                }
                if (node.right != null) {
                    q.offer(node.right);
                }
            }
            return ans;
        }
    }
}

```

JavaScript
JavaScript

```

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @return {number[][]}

var rightSideView = function (root) {
    const ans = [];
    if (!root) {
        return ans;
    }
    const q = [root];
    while (q.length > 0) {
        const nq = [];
        for (const { left, right } of q) {
            if (right) {
                nq.push(right);
            }
        }
        if (left) {
            nq.push(left);
        }
        q.length = 0;
        q.push(...nq);
        return ans;
    }
};

```

JavaScript

```

//
// Definition for a binary tree node.
// function TreeNode(val, left, right) {
//     this.val = (val===undefined ? 0 : val)
//     this.left = (left===undefined ? null : left)
//     this.right = (right===undefined ? null : right)
// }
//
// @param {TreeNode} root
// @return {number[][]}

var rightSideView = function (root) {
    const ans = [];
    if (!root) {
        return ans;
    }
    const q = [root];
    while (q.length > 0) {
        const nq = [];
        for (const { left, right } of q) {
            if (right) {
                nq.push(right);
            }
        }
        if (left) {
            nq.push(left);
        }
        q.length = 0;
        q.push(...nq);
        return ans;
    }
};

```

TypeScript
TypeScript

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }

function rightSideView(root: TreeNode | null): number[] {
    const ans: number[] = [];
    if (!root) {
        return ans;
    }
    const q: TreeNode[] = [root];
    while (q.length > 0) {
        const nq: TreeNode[] = [];
        for (const { left, right } of q) {
            if (right) {
                nq.push(right);
            }
        }
        if (left) {
            nq.push(left);
        }
        q.length = 0;
        q.push(...nq);
        return ans;
    }
}

```

TypeScript

```

//
// Definition for a binary tree node.
// class TreeNode {
//     val: number;
//     left: TreeNode | null;
//     right: TreeNode | null;
//     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//     }
// }

function rightSideView(root: TreeNode | null): number[] {
    const ans = [];
    if (!root) {
        return ans;
    }
    const q = [root];
    while (q.length) {
        const nq = q.length;
        ans.push(q[nq - 1].val);
        for (let i = 0; i < nq; ++i) {
            const { left, right } = q.shift();
            if (left) q.push(left);
            if (right) q.push(right);
        }
        return ans;
    }
}

```

Go

Go

```

//
// Definition for a binary tree node.
// type TreeNode struct {
//     Val int
//     Left *TreeNode
//     Right *TreeNode
// }

func rightSideView(root *TreeNode) []int {
    if root == nil {
        return []int{}
    }
    q := []*TreeNode{root}
    for len(q) > 0 {
        ans := append(ans, q[len(q)-1].Val)
        n := len(q); n--
        for i := 0; i < n; i++ {
            q[i] = q[i+1]
            if q[i+1].Left != nil {
                q = append(q, q[i+1].Left)
            }
            if q[i+1].Right != nil {
                q = append(q, q[i+1].Right)
            }
        }
    }
    return ans
}

```

Go

```

//
// Definition for a binary tree node.
// type TreeNode struct {
//     Val int
//     Left *TreeNode
//     Right *TreeNode
// }

func rightSideView(root *TreeNode) []int {
    if root == nil {
        return []int{}
    }
    q := []*TreeNode{root}
    for len(q) > 0 {
        ans := append(ans, q[len(q)-1].Val)
        n := len(q); n--
        for i := 0; i < n; i++ {
            q[i] = q[i+1]
            if q[i+1].Left != nil {
                q = append(q, q[i+1].Left)
            }
            if q[i+1].Right != nil {
                q = append(q, q[i+1].Right)
            }
        }
    }
    return ans
}

```

```

    }
    return
}

// Rust
// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<RefCell<TreeNode>>>,
//     pub right: Option<Rc<RefCell<TreeNode>>>,
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
use std::collections::VecDeque;
impl Solution {
    pub fn rightSideView(root: Option<Rc<RefCell<TreeNode>>> -> Vec<i32> {
        let mut ans = vec![];
        if root.is_none() {
            return ans;
        }
        let mut q = VecDeque::new();
        q.push_back(root);
        while !q.is_empty() {
            let k = q.len();
            ans.push(q[k-1].as_ref().unwrap().borrow().val);
            for _ in 0..k {
                if let Some(node) = q.pop_front().unwrap() {
                    let mut node = node.borrow_mut();
                    if node.right.is_some() {
                        q.push_back(node.right.take());
                    }
                    if node.left.is_some() {
                        q.push_back(node.left.take());
                    }
                }
            }
        }
        ans
    }
}
// Rust

```

Trees & BST - LeetCode - By Pattern — 3181 — LeetCode

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<RefCell<TreeNode>>>,
//     pub right: Option<Rc<RefCell<TreeNode>>>,
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
use std::collections::VecDeque;
impl Solution {
    pub fn rightSideView(root: Option<Rc<RefCell<TreeNode>>> -> Vec<i32> {
        let mut res = vec![];
        if root.is_none() {
            return res;
        }
        let mut q = VecDeque::new();
        q.push_back(root);
        while !q.is_empty() {
            let k = q.len();
            res.push(q[k-1].as_ref().unwrap().borrow().val);
            for _ in 0..k {
                if let Some(node) = q.pop_front().unwrap() {
                    let mut node = node.borrow_mut();
                    if node.left.is_some() {
                        q.push_back(node.left.take());
                    }
                    if node.right.is_some() {
                        q.push_back(node.right.take());
                    }
                }
            }
        }
        res
    }
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)
Python

Trees & BST - LeetCode - By Pattern — 3182 — LeetCode

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<RefCell<TreeNode>>>,
//     pub right: Option<Rc<RefCell<TreeNode>>>,
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
use std::collections::VecDeque;
impl Solution {
    pub fn rightSideView(root: Option<Rc<RefCell<TreeNode>>> -> Vec<i32> {
        let mut ans = vec![];
        if root.is_none() {
            return ans;
        }
        let mut q = VecDeque::new();
        q.push_back(root);
        while !q.is_empty() {
            let k = q.len();
            ans.push(q[k-1].as_ref().unwrap().borrow().val);
            for _ in 0..k {
                if let Some(node) = q.pop_front().unwrap() {
                    let mut node = node.borrow_mut();
                    if node.left.is_some() {
                        q.push_back(node.left.take());
                    }
                    if node.right.is_some() {
                        q.push_back(node.right.take());
                    }
                }
            }
        }
        ans
    }
}

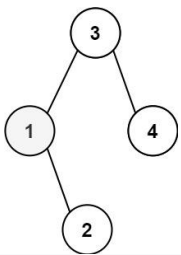
```

#230 MEDIUM
Kth Smallest Element in a BST

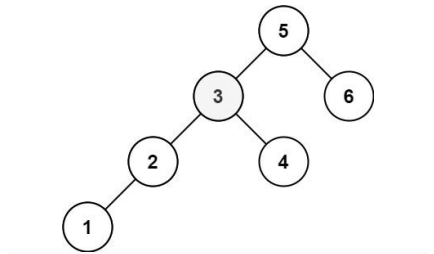
Trees & BST - LeetCode - By Pattern — 3183 — LeetCode

LeetCode - SimplyList - WalkCC - LeetCode - Editorial
Tree - DFS - BST
LeetCode 230

Problem:
Given the root of a binary search tree, and an integer k, return the kth smallest value (1-indexed) of all the values of the nodes in the tree.
Example 1:



Input: root = [3,1,4,null,2], k = 1
Output: 1
Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3
Constraints:
• The number of nodes in the tree is n.
• 1 ≤ k ≤ n ≤ 104
• 0 ≤ Node.val ≤ 104
Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

Python
Python

Trees & BST - LeetCode - By Pattern — 3185 — LeetCode

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32,
//     pub left: Option<Rc<RefCell<TreeNode>>>,
//     pub right: Option<Rc<RefCell<TreeNode>>>,
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }
use std::rc::Rc;
use std::cell::RefCell;
use std::collections::VecDeque;
impl Solution {
    pub fn kthSmallest(root: Option<Rc<RefCell<TreeNode>>>, k: i32) -> i32 {
        let mut ans = None;
        if root.is_none() {
            return ans;
        }
        let mut q = VecDeque::new();
        q.push_back(root);
        while !q.is_empty() {
            let k = q.len();
            ans = q[k-1].as_ref().unwrap().borrow().val;
            for _ in 0..k {
                if let Some(node) = q.pop_front().unwrap() {
                    let mut node = node.borrow_mut();
                    if node.left.is_some() {
                        q.push_back(node.left.take());
                    }
                    if node.right.is_some() {
                        q.push_back(node.right.take());
                    }
                }
            }
        }
        ans
    }
}

```

Python

Trees & BST - LeetCode - By Pattern — 3186 — LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:
            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right

```

```

Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:
            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right

```

```

#####
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):

```

```

# self.val = val
# self.left = left
# self.right = right
class Solution:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
        node = self.build(root)
        return self.dfs(node, k)

    class MyTreeNode:
        def __init__(self, val):
            self.val = val
            self.count = 1 # default 1
            self.left = None
            self.right = None

    def build(self, root: TreeNode) -> MyTreeNode:
        if not root:
            return None
        node = self.MyTreeNode(root.val) # default count already is 1
        node.left = self.build(root.left)
        node.right = self.build(root.right)
        if node.left:
            node.count += node.left.count
        if node.right:
            node.count += node.right.count

        return node

```

```

    def dfs(self, node: MyTreeNode, k: int) -> int:
        if node.left:
            cnt = node.left.count
            if k <= cnt:
                return self.dfs(node.left, k)
            elif k > cnt + 1:
                return self.dfs(node.right, k - 1 - cnt)
            else: # k == cnt + 1
                return node.val
        else:
            if k == 1: # count move to beginning of dfs()
                return node.val
            return self.dfs(node.right, k - 1)

```

Java

Java

```

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        final int leftCount = countNodes(root.left);

        if (leftCount == k - 1)
            return root.val;
        if (leftCount < k)
            return kthSmallest(root.right, k - 1 - leftCount); // leftCount = k
    }

    private int countNodes(TreeNode root) {
        if (root == null)
            return 0;
        return 1 + countNodes(root.left) + countNodes(root.right);
    }
}

```

Java

```

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        traverse(root, k);
        return ans;
    }

    private int ans = -1;
    private int rank = 0;

    private void traverse(TreeNode root, int k) {
        if (root == null)
            return;
        traverse(root.left, k);
        if (++rank == k) {
            ans = root.val;
            return;
        }
        traverse(root.right, k);
    }
}

```

Java

```

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque stack = new ArrayDeque<>();

        while (root != null) {
            stack.push(root);
            root = root.left;
        }

        for (int i = 0; i < k - 1; i++) {
            root = stack.pop();
            root = root.right;
            while (root != null) {
                stack.push(root);
                root = root.left;
            }
        }

        return stack.peek().val;
    }
}

```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }

```

```

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        Deque stk = new ArrayDeque<>();
        while (root != null || !stk.isEmpty()) {
            if (root != null) {
                stk.push(root);
                root = root.left;
            } else {
                root = stk.pop();
                if (--k == 0) {
                    return root.val;
                }
                root = root.right;
            }
        }
        return 0;
    }
}

```

Java

```

import java.util.Comparator;
import java.util.PriorityQueue;

public class Kth_Smallest_Element_in_a_BST {
    //
    // Definition for a binary tree node.
    // public class TreeNode {
    //     int val;
    //     TreeNode left;
    //     TreeNode right;
    //     TreeNode(int x) { val = x; }
    // }
    //
    class Solution {
        PriorityQueue<Integer> heap = new PriorityQueue<>((new Comparator<Integer>()) {
            @Override
            public int compare(Integer o1, Integer o2) {
                return o2 - o1;
            }
        });

        public int kthSmallest(TreeNode root, int k) {
            dfs(root, k);

            return heap.peek();
        }

        private void dfs(TreeNode root, int k) {
            if (root == null) {
                return;
            }

            // maintain heap
            if (heap.size() < k) {
                heap.offer(root.val);
            }

            // following question, heap removal is by object, not index.
            // so if delete operation, just remove element from both tree and heap
        } else {
            int val = root.val;
            if (val < heap.peek()) {
                heap.poll();
                heap.offer(val);
            }
        }
        dfs(root.left, k);
    }
}

```

```

        dfs(root.right, k);
    }
}

class Solution {
    public int kthSmallest(TreeNode root, int k) {
        MyTreeNode node = build(root);
        return dfs(node, k);
    }

    class MyTreeNode {
        int val;
        int count; // key point to add up and find k-th element
        MyTreeNode left;
        MyTreeNode right;
        MyTreeNode(int x) {
            this.val = x;
            this.count = 1;
        }
    }

    MyTreeNode build(TreeNode root) {
        if (root == null) return null;

        MyTreeNode node = new MyTreeNode(root.val);
        node.left = build(root.left);
        node.right = build(root.right);
        if (node.left != null) node.count += node.left.count;
        if (node.right != null) node.count += node.right.count;
    }
}

```

TypeScript

TypeScript


```

//
// Definition for a binary tree node.
//
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) {
        this.val = x;
        this.left = null;
        this.right = null;
    }
}

function kthSmallest(root: TreeNode, k: number): number {
    const dfs = (root: TreeNode) => {
        if (root === null) {
            return -1;
        }
        const { val, left, right } = root;
        const l = dfs(left);
        if (l !== -1) {
            return l;
        }
        //
        if (k === 0) {
            return val;
        }
        return dfs(right);
    };
    return dfs(root);
}

```

TypeScript

Trees & BST - LeetCode - By Pattern

— 3193 —

LeetCode

```

//
// Definition for a binary tree node.
//
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) {
        this.val = x;
        this.left = null;
        this.right = null;
    }
}

function kthSmallest(root: TreeNode, k: number): number {
    const dfs = (root: TreeNode) => {
        if (root === null) {
            return -1;
        }
        const { val, left, right } = root;
        const l = dfs(left);
        if (l !== -1) {
            return l;
        }
        //
        if (k === 0) {
            return val;
        }
        return dfs(right);
    };
    return dfs(root);
}

```

Rust

Rust

Trees & BST - LeetCode - By Pattern

— 3194 —

LeetCode

```

// Definition for a binary tree node.
//
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) {
        this.val = x;
        this.left = null;
        this.right = null;
    }
}

function kthSmallest(root: TreeNode, k: number): number {
    const dfs = (root: TreeNode) => {
        if (root === null) {
            return -1;
        }
        const { val, left, right } = root;
        const l = dfs(left);
        if (l !== -1) {
            return l;
        }
        //
        if (k === 0) {
            return val;
        }
        return dfs(right);
    };
    return dfs(root);
}

```

Rust

Trees & BST - LeetCode - By Pattern

— 3195 —

LeetCode

```

// Definition for a binary tree node.
//
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) {
        this.val = x;
        this.left = null;
        this.right = null;
    }
}

function kthSmallest(root: TreeNode, k: number): number {
    const dfs = (root: TreeNode) => {
        if (root === null) {
            return -1;
        }
        const { val, left, right } = root;
        const l = dfs(left);
        if (l !== -1) {
            return l;
        }
        //
        if (k === 0) {
            return val;
        }
        return dfs(right);
    };
    return dfs(root);
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)

Python

Trees & BST - LeetCode - By Pattern

— 3196 —

LeetCode

```

// Definition for a binary tree node.
//
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) {
        this.val = x;
        this.left = null;
        this.right = null;
    }
}

function kthSmallest(root: TreeNode, k: number): number {
    const dfs = (root: TreeNode) => {
        if (root === null) {
            return -1;
        }
        const { val, left, right } = root;
        const l = dfs(left);
        if (l !== -1) {
            return l;
        }
        //
        if (k === 0) {
            return val;
        }
        return dfs(right);
    };
    return dfs(root);
}

```

#235 MEDIUM
Lowest Common Ancestor of a Binary Search Tree

LeetCode - Binary Search Tree - Medium - Editorial

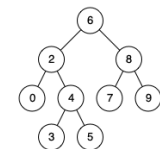
Tree - DFS - BST

LeetCode Grind 109

Problem:
Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

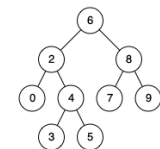
According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
Output: 6
Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
Output: 2
Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [2,1], p = 2, q = 1
Output: 2

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- 109 <= Node.val <= 109
- All Node.val are unique.
- p != q
- p and q will exist in the BST.

Trees & BST - LeetCode - By Pattern

— 3198 —

LeetCode

```
>> Brute Force [Trees & BST] Interview Prep

Python

# Brute Force Approach
class Solution:
    def lowestCommonAncestor(self, root, p, q):
        # Brute Force DFS - find root-to-node paths, find last common node
        def path(node, target):
            path = []
            while node:
                path.append(node)
                if target.val == node.val: node = node.left
                elif target.val == node.val: node = node.right
                else: break
            return path
        p_path = path(root, p)
        q_path = path(root, q)
        p_val = p.val for p in p_path
        for node in reversed(q_path):
            if node.val == p_val:
                return node

Python

def lowestCommonAncestor(root, p, q):
    # Iterate until we find the lowest common ancestor
    while root:
        # If both p and q are less than the current node, move to left subtree
        if p.val < root.val and q.val < root.val:
            root = root.left
        # If both are greater, move to right subtree
        elif p.val > root.val and q.val > root.val:
            root = root.right
        else:
            # Found the split point: return current node as LCA
            return root

Python

class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if root.val > min(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        if root.val < max(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)
        return root

Python

Trees & BST - LeetCode - By Pattern — 3199 — LeetCode
```

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val):
                root = root.left
            elif root.val > max(p.val, q.val):
                root = root.right
            else:
                return root

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val): # no n, so root is p or q is in else block
                root = root.left
            elif root.val > max(p.val, q.val):
                root = root.right
            else:
                return root

# Definition for a binary tree node.
class Solution:
    def lowestCommonAncestor(self, root, p, q):
        if root is None:
            return root
```

```
if root.val < min(p.val, q.val):
    return self.lowestCommonAncestor(root.left, p, q)
elif root.val > max(p.val, q.val):
    return self.lowestCommonAncestor(root.right, p, q)
else:
    return root

# Definition for a binary tree node.
class Solution(object):
    def lowestCommonAncestor(self, root, p, q):
        # Type: TreeNode
        # Type: TreeNode
        # Type: TreeNode
        # Type: TreeNode
        a, b = sorted([p.val, q.val])
        while root and a < root.val < b:
            if a < root.val:
                root = root.left
            else:
                root = root.right
        return root

Java

// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// public class Solution {
//     public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
//         while (true) {
//             if (root.val < Math.Min(p.val, q.val)) {
//                 root = root.left;
//             } else if (root.val > Math.Max(p.val, q.val)) {
//                 root = root.right;
//             } else {
//                 return root;
//             }
//         }
//     }
// }
```

```
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

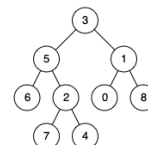
// public class Solution {
//     public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
//         while (true) {
//             if (root.val < Math.Min(p.val, q.val)) {
//                 root = root.left;
//             } else if (root.val > Math.Max(p.val, q.val)) {
//                 root = root.right;
//             } else {
//                 return root;
//             }
//         }
//     }
// }
```

```
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// public class Solution {
//     public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
//         while (true) {
//             if (root.val < Math.Min(p.val, q.val)) {
//                 root = root.left;
//             } else if (root.val > Math.Max(p.val, q.val)) {
//                 root = root.right;
//             } else {
//                 return root;
//             }
//         }
//     }
// }
```

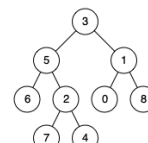
Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.
According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output: 5
Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2
Output: 1
Constraints:

- LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

```

} -> Option<R> def callToTreeNode() = {
  if root.is_name() || root == p || root == q {
    return root;
  }
  let left = Self::lowest_common_ancestor(
    root.as_ref().unwrap().borrow().left.clone(),
    p.clone(),
    q.clone());
  if {
    let right = Self::lowest_common_ancestor(
      root.as_ref().unwrap().borrow().right.clone(),
      p.clone(),
      q.clone());
    if left.is_name() && right.is_name() {
      return root;
    }
    if left.is_name() {
      return left;
    }
    return right;
  }
}
}

// Rust
// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//   pub val: i32,
//   pub left: Option<RustTreeNode>,
//   pub right: Option<RustTreeNode>,
// }
// impl TreeNode {
//   #[inline]
//   pub fn new(val: i32) -> Self {
//     TreeNode {
//       val,
//       left: None,
//       right: None,
//     }
//   }
// }
use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
  fn find(
    root: Option<RustTreeNode>,
    p: Option<RustTreeNode>,
    q: Option<RustTreeNode>
  ) -> Option<RustTreeNode> {
    if root.is_name() || root == p || root == q {
      return root;
    }
    let left = Self::lowest_common_ancestor(
      root.as_ref().unwrap().borrow().left.clone(),
      p.clone(),
      q.clone());
    if {
      let right = Self::lowest_common_ancestor(
        root.as_ref().unwrap().borrow().right.clone(),
        p.clone(),
        q.clone());
      if left.is_name() && right.is_name() {
        return root;
      }
      if left.is_name() {
        return left;
      }
      return right;
    }
  }
}

```

```

} -> Option<R> def callToTreeNode() = {
  if root.is_name() || root == p || root == q {
    return root.clone();
  }
  let node = root.as_ref().unwrap().borrow();
  let left = Self::find(node.left, p, q);
  let right = Self::find(node.right, p, q);
  match (left.is_name(), right.is_name()) {
    (true, false) => left,
    (false, true) => right,
    (false, false) => None,
    (true, true) => root.clone(),
  }
}

pub fn lowest_common_ancestor(
  root: Option<RustTreeNode>,
  p: Option<RustTreeNode>,
  q: Option<RustTreeNode>
) -> Option<RustTreeNode> {
  Self::find(root, p, q)
}

```

[*] Trees & BST — Pattern Solution (matched from community answers)

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # Base case: if the current node is None, return None.
        if root is None:
            return None

        # If the current node is either p or q, return the current node.
        if root == p or root == q:
            return root

        # Search for p and q in the left and right subtrees.
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

        # If both left and right are non-null, the current node is the LCA.
        if left and right:
            return root

        # Otherwise, return the non-null node.
        return left if left is not None else right

```

#250 **MEDIUM**

Count Unival Subtrees

LeetCode - [Simplify](#) - [WuCC](#) - [LeetCode](#) - [Editorial](#)

Tree - DFS

Little Premium 98

Problem:
Given the root of a binary tree, return the number of uni-value subtrees.

A uni-value subtree means all nodes of the subtree have the same value.

Example 1:

```

graph TD
    5((5)) --- 1((1))
    5 --- 5r((5))
    1 --- 5l((5))
    1 --- 5b((5))
    5r --- 5c((5))

```

Input: root = [5,1,5,5,5,null,5]
Output: 4

Example 2:

```

Input: root = []
Output: 0

```

Example 3:

```

Input: root = [5,5,5,5,5,null,5]
Output: 6

```

Constraints:

- The number of the node in the tree will be in the range [0, 1000].
- 1000 <= Node.val <= 1000

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def countUnivalSubtrees(self, root: TreeNode) -> int:
        # Initialization
        self.count = 0

        # Helper function performs postorder DFS
        def isUnival(node):
            # Base case: null nodes are considered unival
            if node is None:
                return True

            # Recursion calls for left and right subtrees
            leftUnival = isUnival(node.left)
            rightUnival = isUnival(node.right)

            # Check if left child exists and its value does not match current node's value
            if node.left and node.left.val != node.val:
                leftUnival = False

            # Check if right child exists and its value does not match current node's value
            if node.right and node.right.val != node.val:
                rightUnival = False

            # If either left or right subtree is not unival, then the current subtree is not unival
            if not leftUnival or not rightUnival:
                return False

            # Increment the count if the subtree rooted at the current node is unival
            self.count += 1
            return True
        else:
            return False

        # Base recursion from root
        isUnival(root)
        return self.count

# Example usage:
# Construct the binary tree: [5,1,5,5,5,null,5]
# root = TreeNode(5)
# root.left = TreeNode(1)
# root.right = TreeNode(5)
# root.left.left = TreeNode(5)
# root.left.right = TreeNode(5)
# root.right.left = TreeNode(5)
# print(Solution().countUnivalSubtrees(root)) # Output: 4

```

Python

```

class Solution:
    def countUnivalSubtrees(self, root: TreeNode | None) -> int:
        ans = 0

        def isUnival(root: TreeNode | None, val: int) -> bool:
            nonlocal ans
            if not root:
                return True
            if isUnival(root.left, root.val) & isUnival(root.right, root.val):
                ans += 1
                return root.val == val
            return False

        isUnival(root, math.inf)
        return ans

Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def dfs(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return True
        l = self.dfs(root.left)
        r = self.dfs(root.right)
        if not l or not r:
            return False
        a = root.val if root.left is None else root.left.val
        b = root.val if root.right is None else root.right.val
        if a == b == root.val:
            nonlocal ans
            ans += 1
            return True
        return False

    def dfs(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        l = self.dfs(root.left)
        r = self.dfs(root.right)
        if not l or not r:
            return 0
        a = root.val if root.left is None else root.left.val
        b = root.val if root.right is None else root.right.val
        if a == b == root.val:
            nonlocal ans
            ans += 1
            return 1 + l + r
        return 0

```

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def countUnivalSubtrees(self, root: TreeNode) -> int:
        count = 0

        def isUnival(node):
            nonlocal count
            if not node:
                return True

            leftUnival = isUnival(node.left)
            rightUnival = isUnival(node.right)

            if leftUnival and rightUnival:
                if node.left and node.val != node.left.val:
                    return False
                if node.right and node.val != node.right.val:
                    return False
                count += 1
                return True
            return False

        return isUnival(root)

class Solution:
    def countUnivalSubtrees(self, root: TreeNode) -> int:
        def isUnival(node):
            if not node:
                return True, 0
            leftUnival, leftCount = isUnival(node.left)
            rightUnival, rightCount = isUnival(node.right)
            if leftUnival and rightUnival:
                if node.left and node.val != node.left.val:
                    return False, 0
                if node.right and node.val != node.right.val:
                    return False, 0
                return True, leftCount + rightCount + 1
            return False, 0

        count = isUnival(root)
        return count

```

Java

```

class Solution {
    public int countUnivalSubtrees(TreeNode root) {
        isUnival(root, Integer.MAX_VALUE);
        return ans;
    }

    private int ans = 0;

    private boolean isUnival(TreeNode root, int val) {
        if (root == null)
            return true;

        if (isUnival(root.left, root.val) & isUnival(root.right, root.val)) {
            ++ans;
            return root.val == val;
        }

        return false;
    }
}

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) { this.val = x; }
}

//
// Definition for a binary tree node.
//
public class Solution {
    private int ans;

    public int countUnivalSubtrees(TreeNode root) {
        dfs(root);
        return ans;
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {

```

```

        return true;
    }

    boolean l = dfs(root.left);
    boolean r = dfs(root.right);
    if (l || r) {
        return false;
    }

    int a = root.left == null ? root.val : root.left.val;
    int b = root.right == null ? root.val : root.right.val;
    if (a == b && b == root.val) {
        ++ans;
        return true;
    }

    return false;
}

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) { this.val = x; }
}

//
// Definition for a binary tree node.
//
public class Solution {
    private int ans;

    public int countUnivalSubtrees(TreeNode root) {
        dfs(root);
        return ans;
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {

```

```

        return true;
    }

    boolean l = dfs(root.left);
    boolean r = dfs(root.right);
    if (l || r) {
        return false;
    }

    int a = root.left == null ? root.val : root.left.val;
    int b = root.right == null ? root.val : root.right.val;
    if (a == b && b == root.val) {
        ++ans;
        return true;
    }

    return false;
}

//
// Definition for a binary tree node.
//
public class Solution {
    private int ans;

    public int countUnivalSubtrees(TreeNode root) {
        dfs(root);
        return ans;
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {

```

```

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) { this.val = x; }
}

//
// Definition for a binary tree node.
//
public class Solution {
    private int ans;

    public int countUnivalSubtrees(TreeNode root) {
        dfs(root);
        return ans;
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {

```

```

//
// Definition for a binary tree node.
//
public class Solution {
    private int ans;

    public int countUnivalSubtrees(TreeNode root) {
        dfs(root);
        return ans;
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {

```

```

//
// Definition for a binary tree node.
//
public class Solution {
    private int ans;

    public int countUnivalSubtrees(TreeNode root) {
        dfs(root);
        return ans;
    }

    private boolean dfs(TreeNode root) {
        if (root == null) {

```

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val: int, left: Optional[TreeNode], right: Optional[TreeNode]) -> None:
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return True
            l, r = dfs(root.left), dfs(root.right)
            if not l or not r:
                return False
            a = root.val if root.left is None else root.left.val
            b = root.val if root.right is None else root.right.val
            if a == b == root.val:
                ans = 1
                return True
            return False

        ans = 0
        dfs(root)
        return ans

```

#285 MEDIUM
Inorder Successor in BST
LeetCode - [SimpleLink](#) - [WikiCC](#) - [LeetCode](#) - [Editorial](#)

Tree - DFS - BST
Lists - Grind 169

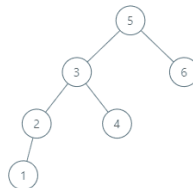
Problem:
Given the root of a binary search tree and a node p in it, return the in-order successor of that node in the BST. If the given node has no in-order successor in the tree, return null.
The successor of a node p is the node with the smallest key greater than p.val.

Example 1:



Input: root = [2,1,3], p = 1
Output: 2
Explanation: 1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.

Example 2:



Input: root = [5,3,6,2,4,null,null,1], p = 6
Output: null
Explanation: There is no in-order successor of the current node, so the answer is null.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 105 <= Node.val <= 105
- All nodes will have unique values.

Python
Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val: int, left: Optional[TreeNode], right: Optional[TreeNode]) -> None:
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> TreeNode:
        # If p has a right subtree, the successor is the left-most node in that subtree.
        if p.right:
            successor = p.right
            while successor.left:
                successor = successor.left
            return successor

        # Otherwise, start from the root and keep track of a potential successor.
        successor = None
        current = root
        while current:
            if p.val < current.val:
                # current is a candidate, so record it and move left to find a smaller candidate
                successor = current
                current = current.left
            else:
                # If p.val is greater or equal, go right.
                current = current.right
        return successor

```

Python
class Solution:
def inorderSuccessor(
self,
root: TreeNode | None,
p: TreeNode | None,
) -> TreeNode | None:
if not root:
return None
if root.val < p.val:
return self.inorderSuccessor(root.right, p)
return self.inorderSuccessor(root.left, p) or root

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val: int, left: Optional[TreeNode], right: Optional[TreeNode]) -> None:
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:
        ans = None
        while root:
            if root.val > p.val:
                ans = root
                root = root.left
            else:
                root = root.right
        return ans

```

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val: int, left: Optional[TreeNode], right: Optional[TreeNode]) -> None:
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:
        ans = None
        while root:
            if root.val > p.val:
                ans = root
                root = root.left
            else:
                root = root.right
        return ans

```

JavaScript
JavaScript

```

/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 *     this.val = val;
 *     this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @return {TreeNode}
 */
var inorderSuccessor = function (root, p) {
    let ans = null;
    while (root) {
        if (root.val > p.val) {
            ans = root;
            root = root.left;
        } else {
            root = root.right;
        }
    }
    return ans;
};

```

JavaScript
/**
 * Definition for a binary tree node.
 * function TreeNode(val) {
 * this.val = val;
 * this.left = this.right = null;
 * }
 */
/**
 * @param {TreeNode} root
 * @param {TreeNode} p
 * @return {TreeNode}
 */
var inorderSuccessor = function (root, p) {
 let ans = null;
 while (root) {
 if (root.val > p.val) {
 ans = root;
 root = root.left;
 } else {
 root = root.right;
 }
 }
 return ans;
};

TypeScript
TypeScript

```

/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val);
 *         this.left = (left===undefined ? null : left);
 *         this.right = (right===undefined ? null : right);
 *     }
 * }
 */
function inorderSuccessor(root: TreeNode | null, p: TreeNode | null): TreeNode | null {
    let ans: TreeNode | null = null;
    while (root) {
        if (root.val > p.val) {
            ans = root;
            root = root.left;
        } else {
            root = root.right;
        }
    }
    return ans;
}

```

TypeScript

```

/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     val: number;
 *     left: TreeNode | null;
 *     right: TreeNode | null;
 *     constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
 *         this.val = (val===undefined ? 0 : val);
 *         this.left = (left===undefined ? null : left);
 *         this.right = (right===undefined ? null : right);
 *     }
 * }
 */
function inorderSuccessor(root: TreeNode | null, p: TreeNode | null): TreeNode | null {
    let ans: TreeNode | null = null;
    while (root) {
        if (root.val > p.val) {
            ans = root;
            root = root.left;
        } else {
            root = root.right;
        }
    }
    return ans;
}

```

[*] Trees & BST — Pattern Solution (matched from community answers)

```
Python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> TreeNode:
        # If p has a right subtree, the successor is the left-most node in that subtree.
        if p.right:
            successor = p.right
            while successor.left:
                successor = successor.left
            return successor

        # Otherwise, start from the root and keep track of a potential successor.
        successor = None
        current = root
        while current:
            if p.val < current.val:
                # current is a candidate, so record it and move left to find a smaller candidate
                successor = current
                current = current.left
            else:
                # If p.val is greater or equal, go right.
                current = current.right
        return successor
```

#298

MEDIUM

Binary Tree Longest Consecutive Sequence

LeetCode · SimplifyLeetCode · WUCC · LeetCode · Editorial

Tree · DFS

Like · Premium 98

Problem:

Given the root of a binary tree, return the length of the longest consecutive sequence path.

A consecutive sequence path is a path where the values increase by one along the path.

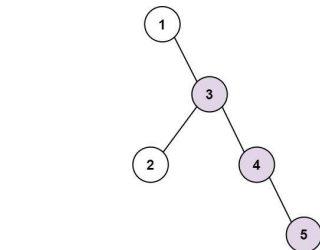
Note that the path can start at any node in the tree, and you cannot go from a node to its parent in the path.

Example 1:

Trees & BST · LeetCodeMastery — By Pattern

— 3229 —

LeetCodeMastery

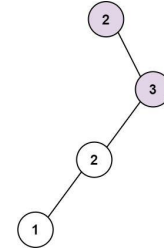


Input: root = [2,null,3,2,4,null,null,null,5]

Output: 3

Explanation: Longest consecutive sequence path is 3-4-5, so return 3.

Example 2:



Trees & BST · LeetCodeMastery — By Pattern

— 3230 —

LeetCodeMastery

Input: root = [2,null,3,2,null,1]

Output: 2

Explanation: Longest consecutive sequence path is 2-3, not 3-2-1, so return 2.

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- -3 * 10⁴ <= Node.val <= 3 * 10⁴

Python

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        # Definition of a consecutive sequence length.
        self.max_length = 0

        # Helper function to perform DFS traversal.
        def dfs(node, parent_val, current_length):
            # If not None, return.
            if not node:
                return

            # Check if the current node's value is consecutive.
            if node.val == parent_val + 1:
                current_length += 1
            else:
                current_length = 1

            # Update the maximum sequence length found so far.
            self.max_length = max(self.max_length, current_length)

            # Recursively traverse the left and right children.
            dfs(node.left, node.val, current_length)
            dfs(node.right, node.val, current_length)

        # Start DFS with an initial condition that forces the first node to count.
        dfs(root, root.val - 1, 0)
        return self.max_length
```

Python

Trees & BST · LeetCodeMastery — By Pattern

— 3231 —

LeetCodeMastery

```
class Solution:
    def longestConsecutive(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        def dfs(root: TreeNode | None, target: int, length: int, max_length: int) -> int:
            if not root:
                return max_length
            if root.val == target:
                length += 1
                max_length = max(max_length, length)
            else:
                length = 1
            return max(dfs(root.left, root.val + 1, length, max_length),
                      dfs(root.right, root.val + 1, length, max_length))
        return dfs(root, root.val, 0, 0)
```

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def longestConsecutive(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        l = dfs(root.left) + 1
        r = dfs(root.right) + 1
        if root.left and root.left.val - root.val != 1:
            l = 1
        if root.right and root.right.val - root.val != 1:
            r = 1
        t = max(l, r)
        maxLen = ans = max(ans, t)
        return t

    ans = 0
    dfs(root)
    return ans
```

Python

Trees & BST · LeetCodeMastery — By Pattern

— 3232 —

LeetCodeMastery

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# DFS
class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        maxlen = 0

        # curLen: consecutive length until parent node
        def dfs(root: TreeNode, lastVal: int, curLen: int) -> None:
            nonlocal maxlen
            if not root:
                return

            curLen = (curLen + 1) if (not lastVal) and root.val == lastVal + 1 else 1
            maxlen = max(maxlen, curLen)

            dfs(root.left, root.val, curLen)
            dfs(root.right, root.val, curLen)

        dfs(root, None, 0)

        return maxlen

#####
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val):
#         self.val = val
#         self.left = None
#         self.right = None

from collections import deque

# iterative bfs
class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        if not root:
            return 0

        max_length = 0
        # queue elements are tuples: (current node, length of consecutive sequence)
        queue = deque([(root, 1)])

        while queue:
            current, length = queue.popleft()
```

Trees & BST · LeetCodeMastery — By Pattern

— 3233 —

LeetCodeMastery

```
max_length = max(max_length, length)

for neighbor in [current.left, current.right]:
    if neighbor:
        # Check if neighbor is consecutive
        if neighbor.val == current.val + 1:
            queue.append((neighbor, length + 1))
        else:
            queue.append((neighbor, 1))

return max_length

#####
class Solution:
    def longestConsecutive(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if not root:
                return 0
            l = dfs(root.left) + 1
            r = dfs(root.right) + 1
            if root.left and root.left.val - root.val != 1:
                l = 1
            if root.right and root.right.val - root.val != 1:
                r = 1
            t = max(l, r)
            maxLen = ans = max(ans, t)
            return t

        ans = 0
        dfs(root)
        return ans
```

Java

Java

```
class Solution {
    public int longestConsecutive(TreeNode root) {
        if (root == null)
            return 0;
        return dfs(root, -1, 0, 1);
    }

    private int dfs(TreeNode root, int target, int length, int mx) {
        if (root == null)
            return mx;
        if (root.val == target)
            mx = Math.max(mx, ++length);
        else
            length = 1;
        return Math.max(dfs(root.left, root.val + 1, length, mx),
                      dfs(root.right, root.val + 1, length, mx));
    }
}
```

Java

Trees & BST · LeetCodeMastery — By Pattern

— 3234 —

LeetCodeMastery

```

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    private int ans;

    public int longestConsecutive(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int l = dfs(root.left) + 1;
        int r = dfs(root.right) + 1;
        if (root.left != null && root.left.val - root.val != 1) {
            l = 1;
        }
        if (root.right != null && root.right.val - root.val != 1) {
            r = 1;
        }
        int t = Math.max(l, r);
        ans = Math.max(ans, t);
        return t;
    }
}

```

Java

```

//
// Definition for a binary tree node.
//
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    private int ans;

    public int longestConsecutive(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int l = dfs(root.left) + 1;
        int r = dfs(root.right) + 1;
        if (root.left != null && root.left.val - root.val != 1) {
            l = 1;
        }
        if (root.right != null && root.right.val - root.val != 1) {
            r = 1;
        }
        int t = Math.max(l, r);
        ans = Math.max(ans, t);
        return t;
    }
}

```

TypeScript
TypeScript

```

//
// Definition for a binary tree node.
//
class TreeNode {
    int val;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
        this.val = (val===undefined ? 0 : val);
        this.left = (left===undefined ? null : left);
        this.right = (right===undefined ? null : right);
    }
}

function longestConsecutive(root: TreeNode | null): number {
    let ans = 0;
    const dfs = (root: TreeNode | null): number => {
        if (root == null) {
            return 0;
        }
        let l = dfs(root.left) + 1;
        let r = dfs(root.right) + 1;
        if (root.left && root.left.val - root.val == 1) {
            l = l;
        }
        if (root.right && root.right.val - root.val == 1) {
            r = r;
        }
        const t = Math.max(l, r);
        ans = Math.max(ans, t);
        return t;
    };
    dfs(root);
    return ans;
}

```

TypeScript

```

//
// Definition for a binary tree node.
//
class TreeNode {
    int val;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
        this.val = (val===undefined ? 0 : val);
        this.left = (left===undefined ? null : left);
        this.right = (right===undefined ? null : right);
    }
}

function longestConsecutive(root: TreeNode | null): number {
    let ans = 0;
    const dfs = (root: TreeNode | null): number => {
        if (root == null) {
            return 0;
        }
        let l = dfs(root.left) + 1;
        let r = dfs(root.right) + 1;
        if (root.left && root.left.val - root.val == 1) {
            l = l;
        }
        if (root.right && root.right.val - root.val == 1) {
            r = r;
        }
        const t = Math.max(l, r);
        ans = Math.max(ans, t);
        return t;
    };
    dfs(root);
    return ans;
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)
Python

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        # Definition is provided to keep track of the maxLen sequence length.
        self.max_length = 0

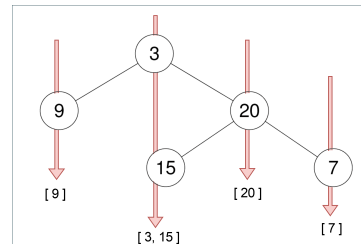
        # Helper function to perform DFS traversal.
        def dfs(node, parent_val, current_length):
            # If node is None, return.
            if not node:
                return
            # Check if the current node's value is consecutive.
            if node.val == parent_val + 1:
                current_length += 1
            else:
                current_length = 1
            # Update the maxLen sequence length found so far.
            self.max_length = max(self.max_length, current_length)
            # Recursively traverse the left and right children.
            dfs(node.left, node.val, current_length)
            dfs(node.right, node.val, current_length)

        # Start DFS with an initial condition that forces the first node to count.
        dfs(root, root.val - 1, 0)
        return self.max_length

```

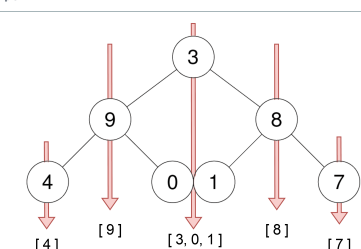
#314 MEDIUM
Binary Tree Vertical Order Traversal
LeetCode - Solution - Hash Table - DFS - Sorting
Hash Table - Tree - DFS - Sorting
List: Premium 98

Problem:
Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column).
If two nodes are in the same row and column, the order should be from left to right.
Example 1:



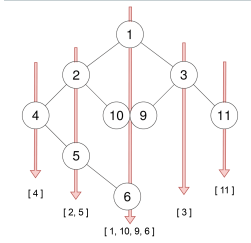
Input: root = [3,9,20,null,null,15,7]
Output: [[9],[3,15],[20],[7]]

Example 2:



Input: root = [3,9,8,4,0,1,7]
Output: [[4],[9],[3,0,1],[8],[7]]

Example 3:



Input: root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,6]
Output: [[4],[2,5],[1,10,9,6],[3],[11]]

- Constraints:
- The number of nodes in the tree is in the range [0, 100].
 - -100 <= Node.val <= 100

>> Brute Force [Trees & BST] Interview Prep

```
Python
# Brute Force Approach
class Solution:
    def verticalOrder(self, root: TreeNode) -> List[List[int]]:
        # Brute Force - collect nodes, then process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)
        collect(root)
        return vals[0] if vals else []
```

Python
Python

```
from collections import deque, defaultdict

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

class Solution:
    def verticalOrder(self, root: TreeNode) -> List[List[int]]:
        # Return empty list if the tree is empty
        if not root:
            return []

        # Dictionary to hold nodes for each column index
        col_table = defaultdict(list)
        # Use deque for efficient queue for BFS; store tuple (node, column)
        queue = deque([(root, 0)])

        while queue:
            node, col = queue.popleft()
            # Append current node value to the list of its column
            col_table[col].append(node.val)
            # If there is a left child, assign column col-1
            if node.left:
                queue.append((node.left, col - 1))
            # If there is a right child, assign column col+1
            if node.right:
                queue.append((node.right, col + 1))

        # Extract the columns in sorted order to build the final result
        sorted_cols = sorted(col_table.keys())
        result = [col_table[col] for col in sorted_cols]
        return result
```

Python

```
class Solution:
    def verticalOrder(self, root: TreeNode) -> List[List[int]]:
        if not root:
            return []

        range_ = [0] * 2

        def getRange(root: TreeNode | None, x: int) -> None:
            if not root:
                return
            range_[0] = min(range_[0], x)
            range_[1] = max(range_[1], x)

            getRange(root.left, x - 1)
            getRange(root.right, x + 1)

        getRange(root, 0) # Get the leftmost and the rightmost x index.

        ans = [[] for _ in range(range_[1] - range_[0] + 1)]
        q = collections.deque([(root, -range_[0])]) # (TreeNode, x)

        while q:
            node, x = q.popleft()
            ans[x + range_[0]].append(node.val)

            if node.left:
                q.append((node.left, x - 1))
            if node.right:
                q.append((node.right, x + 1))

        return ans
```

Python

```
Python
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def dfs(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root, depth, offset):
            if root is None:
                return
            dfs(root.left, depth + 1, offset - 1)
            dfs(root.right, depth + 1, offset + 1)

            d = defaultdict(list)
            dfs(root, 0, 0)
            ans = []
            for x in sorted(d.items()):
                v.sort(key=lambda x: x[0])
                ans.append([v] for x in v)
            return ans
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def dfs(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root, depth, offset):
            if root is None:
                return
            dfs(root.left, depth + 1, offset - 1)
            dfs(root.right, depth + 1, offset + 1)

            d = defaultdict(list)
            dfs(root, 0, 0)
            ans = []
            for x in sorted(d.items()):
                v.sort(key=lambda x: x[0])
                ans.append([v] for x in v)
            return ans
```

```
class Solution:
    def verticalOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if root is None:
            return []
        q = deque([(root, 0)])
        d = defaultdict(list)
        while q:
            for _ in range(len(q)):
                root, offset = q.popleft()
                d[offset].append(root.val)
                if root.left:
                    q.append((root.left, offset - 1))
                if root.right:
                    q.append((root.right, offset + 1))
            return [v for _, v in sorted(d.items())]

        # Similar to above, but use existing data structure
        from sortedcontainers import SortedDict
        from typing import List
        from collections import deque

        class Solution:
            def dfs(self, root: Optional[TreeNode]) -> List[List[int]]:
                def dfs(root, depth, offset):
                    if root is None:
                        return
                    dfs(root.left, depth + 1, offset - 1)
                    dfs(root.right, depth + 1, offset + 1)

                    d = defaultdict(list)
                    dfs(root, 0, 0)
                    ans = []
                    for x in sorted(d.items()):
                        v.sort(key=lambda x: x[0])
                        ans.append([v] for x in v)
                    return ans
```

Python

```
def verticalOrder(self, root: TreeNode) -> List[List[int]]:
    if not root:
        return []

    # Use SortedDict to automatically sort by column index
    sorted_dict = SortedDict()

    # Queue for breadth-first search; stores pairs of (node, column index)
    queue = deque([(root, 0)])

    while queue:
        node, column = queue.popleft()

        if column not in sorted_dict:
            sorted_dict[column] = [node.val]
        else:
            sorted_dict[column].append(node.val)

        # Add left and right children to the queue
        if node.left:
            queue.append((node.left, column - 1))
        if node.right:
            queue.append((node.right, column + 1))

    # Return the values from sorted_dict as a list of lists
    return list(sorted_dict.values())
```

```
#####
# another SortedDict, but different handling on None node, before enqueue vs after enqueue
```

C++
C++

```
/*
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
using pii = pair<int, int>;

class Solution {
public:
    map<int, vector<int>> d;

    vector<vector<int>> verticalOrder(TreeNode* root) {
        dfs(root, 0, 0);
        vector<vector<int>> ans;
        for (auto& [x, v] : d) {
            sort(v.begin(), v.end());
            ans.push_back(v);
        }
        return ans;
    }

    void dfs(TreeNode* root, int depth, int offset) {
        if (!root) return;
        d[offset].push_back(root->val);
        dfs(root->left, depth + 1, offset - 1);
        dfs(root->right, depth + 1, offset + 1);
    }
};
```

C++

Java

LeetMastery

Go

LeetMastery

[*] Trees & BST — Pattern Solution (matched from community answers)

LeetMastery

Problem:
Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

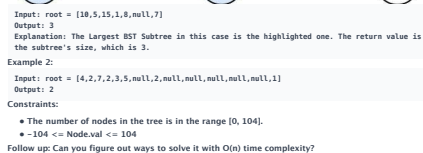
A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1:

LeetMastery



Python

LeetMastery

Python

LeetMastery

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    mx: int # the minimum value in the subtree
    mx: int # the maximum value in the subtree
    size: int # the size of the subtree

class Solution:
    def largestBSTSubtree(self, root: TreeNode) -> int:
        def dfs(root: TreeNode) -> int:
            if not root:
                return 1, math.inf, 0
            l = dfs(root.left)
            r = dfs(root.right)
            if l.mx < root.val < r.mx:
                return 1 + l.size + r.size
            # Mark one as invalid, but still record the size of children.
            # Return (mx, mx, size) if l.mx >= root.val or <= root.val.
            return 1 + math.inf, math.inf, max(l.size, r.size)

        return dfs(root).size

```

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0, -inf, 0
            lval, lmx, lsz = dfs(root.left)
            rval, rmx, rsz = dfs(root.right)
            ans = 0
            if lmx < root.val < rmx:
                ans = max(lsz, rsz + 1)
                return min(lval, root.val), max(rmx, root.val), lsz + rsz + 1
            return -inf, -inf, 0
        ans = 0
        dfs(root)
        return ans

```

Python

```

...
// dfs(int* left)
int dfs(int* left) {
    // dfs(int* left) -> math.inf
    // ...
    // Definition for a binary tree node.
    // class TreeNode:
    //     def __init__(self, val=0, left=None, right=None):
    //         self.val = val
    //         self.left = left
    //         self.right = right
    class Solution:
        def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
            def dfs(root):
                if root is None:
                    return 0, -inf, 0
                lval, lmx, lsz = dfs(root.left)
                rval, rmx, rsz = dfs(root.right)
                ans = 0
                # this if can check if left/right is a bst
                # eq. if left is not bst, the returned lval-inf and lmx-inf, so 'lval < root.val < rval' is false
                if lmx < root.val < rmx:
                    ans = max(lsz, rsz + 1)
                    # but, is lval/rmx correct, except when left/right is None
                    return min(lval, root.val), max(rmx, root.val), lsz + rsz + 1
                return -inf, -inf, 0 # if this one not bst, then all parents will not be bst

            ans = 0
            dfs(root)
            return ans

#####
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val):
#         self.val = val
#         self.left = None
#         self.right = None
class Solution:
    def largestBSTSubtree(self, root:
        ...
        type root: TreeNode
        type int
        ...

```

```

def helper(root):
    if not root:
        return (0, 0, float('inf'), float('-inf'), 0, numBST, numValid, min, max)
    lnumBST, lnumValid, lmx, lmax = helper(root.left)
    rnumBST, rnumValid, rmx, rmax = helper(root.right)
    numBST += 1
    if lmx < root.val < rmx and lnumBST != -1 and rnumBST != -1:
        numBST += 1 + lnumBST + rnumBST
    numValid += max(l, rnumValid) + max(lnumBST, rnumBST)
    return numBST, numValid, min(lmx, root.val), max(lmax, rmax, root.val)

return helper(root)[1]

```

C++

```

//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };
class Solution {
public:
    int ans;

    int largestBSTSubtree(TreeNode* root) {
        ans = 0;
        dfs(root);
        return ans;
    }

    vector<int> dfs(TreeNode* root) {
        if (!root) return {INT_MAX, INT_MIN, 0};
        auto left = dfs(root->left);
        auto right = dfs(root->right);

        if (left[1] < root->val && root->val < right[0]) {
            ans = max(ans, left[2] + right[2] + 1);
            return {min(root->val, left[0]), max(root->val, right[1]), left[2] + right[2] + 1};
        }
        return {INT_MAX, INT_MIN, 0};
    }
};

```

C++

```

//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode *left;
//     TreeNode *right;
//     TreeNode() : val(0), left(nullptr), right(nullptr) {}
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
// };
class Solution {
public:
    int ans;

    int largestBSTSubtree(TreeNode* root) {
        ans = 0;
        dfs(root);
        return ans;
    }

    vector<int> dfs(TreeNode* root) {
        if (!root) return {INT_MAX, INT_MIN, 0};
        auto left = dfs(root->left);
        auto right = dfs(root->right);

        if (left[1] < root->val && root->val < right[0]) {
            ans = max(ans, left[2] + right[2] + 1);
            return {min(root->val, left[0]), max(root->val, right[1]), left[2] + right[2] + 1};
        }
        return {INT_MAX, INT_MIN, 0};
    }
};

```

Java

Java

```

class Solution {
    public int largestBSTSubtree(TreeNode root) {
        return dfs(root).size;
    }

    private T dfs(TreeNode root) {
        if (root == null)
            return new T(Integer.MAX_VALUE, Integer.MIN_VALUE, 0);
        T l = dfs(root.left);
        T r = dfs(root.right);

        if (l.mx < root.val && root.val < r.mx)
            return new T(Math.min(l.mx, root.val), Math.max(r.mx, root.val), 1 + l.size + r.size);

        // Mark one as invalid, but still record the size of children.
        // Return (mx, mx, size) if l.mx >= root.val or <= root.val.
        return new T(Integer.MAX_VALUE, Integer.MAX_VALUE, Math.max(l.size, r.size));
    }

    private record T(int mx, // the minimum value in the subtree
                    int mx, // the maximum value in the subtree
                    int size // the size of the subtree
    ) {}
}

```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private int ans;

    public int largestBSTSubtree(TreeNode root) {
        ans = 0;
        dfs(root);
        return ans;
    }

    private int[] dfs(TreeNode root) {

```

```

    if (root == null) {
        return new int[] {Integer.MAX_VALUE, Integer.MIN_VALUE, 0};
    }
    int[] left = dfs(root.left);
    int[] right = dfs(root.right);
    if (left[1] < root.val && root.val < right[0]) {
        ans = Math.max(ans, left[2] + right[2] + 1);
        return new int[] {
            Math.min(root.val, left[0]), Math.max(root.val, right[1]), left[2] + right[2] + 1;
        };
    }
    return new int[] {Integer.MAX_VALUE, Integer.MIN_VALUE, 0};
}

```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int val) { this.val = val; }
//     TreeNode(int val, TreeNode left, TreeNode right) {
//         this.val = val;
//         this.left = left;
//         this.right = right;
//     }
// }
class Solution {
    private int ans;

    public int largestBSTSubtree(TreeNode root) {
        ans = 0;
        dfs(root);
        return ans;
    }

    private int[] dfs(TreeNode root) {
        if (root == null) {
            return new int[] {Integer.MAX_VALUE, Integer.MIN_VALUE, 0};
        }
        int[] left = dfs(root.left);
        int[] right = dfs(root.right);
        if (left[1] < root.val && root.val < right[0]) {
            ans = Math.max(ans, left[2] + right[2] + 1);
            return new int[] {
                Math.min(root.val, left[0]), Math.max(root.val, right[1]), left[2] + right[2] + 1;
            };
        }
        return new int[] {Integer.MAX_VALUE, Integer.MIN_VALUE, 0};
    }
}

```

Go

Go

```

/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *     int val;
 *     *Left *TreeNode;
 *     *Right *TreeNode;
 * }
 */
func largestBSTSubtree(root *TreeNode) int {
    ans := 0
    var dfs func(root *TreeNode) []int
    dfs = func(root *TreeNode) []int {
        if root == nil {
            return []int{math.MaxInt32, math.MinInt32, 0}
        }
        left := dfs(root.Left)
        right := dfs(root.Right)
        if left[1] < root.Val && root.Val < right[0] {
            ans = max(ans, left[2]+right[2]+1)
            return []int{min(root.Val, left[0]), max(root.Val, right[1]), left[2] + right[2] + 1}
        }
        return []int{math.MaxInt32, math.MinInt32, 0}
    }
    dfs(root)
    return ans
}

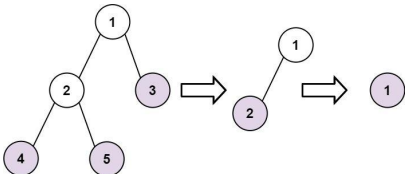
```

Go

```

/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *     int val;
 *     *Left *TreeNode;
 *     *Right *TreeNode;
 * }
 */
func largestBSTSubtree(root *TreeNode) int {
    ans := 0
    var dfs func(root *TreeNode) []int
    dfs = func(root *TreeNode) []int {
        if root == nil {
            return []int{math.MaxInt32, math.MinInt32, 0}
        }
        left := dfs(root.Left)
        right := dfs(root.Right)
        if left[1] < root.Val && root.Val < right[0] {
            ans = max(ans, left[2]+right[2]+1)
            return []int{min(root.Val, left[0]), max(root.Val, right[1]), left[2] + right[2] + 1}
        }
        return []int{math.MaxInt32, math.MinInt32, 0}
    }
    dfs(root)
    return ans
}

```



Input: root = [1,2,3,4,5]
Output: [4,5,3],[2],[1]
Explanation: [3,5,4],[2],[1]] and [3,4,5],[2],[1]] are also considered correct answers since per each level it does not matter the order on which elements are returned.

Example 2:

Input: root = [1]
Output: [1]

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- 100 <= Node.val <= 100

>> Brute Force (Trees & BST) Interview Prep

Python

```

# Brute Force Solution
class Solution:
    def findLeaves(self, root: TreeNode):
        # Brute Force - collect leaves, then process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            collect(node.right)
            collect(node)
            return vals[0] if vals else 0
        collect(root)
        return vals[0] if vals else 0

```

Python

Python

```

# Definition for a binary tree node.
# type TreeNode struct {
#     int val;
#     *Left *TreeNode;
#     *Right *TreeNode;
# }
class Solution {
    def findLeaves(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            l = dfs(root.left)
            r = dfs(root.right)
            h = max(l, r)
            if l == r:
                ans.append(root.val)
                return h + 1
            else:
                ans.append(root.val)
                return h + 1
        ans = []
        dfs(root)
        return ans

```

Python

```

# Definition for a binary tree node.
# type TreeNode struct {
#     int val;
#     *Left *TreeNode;
#     *Right *TreeNode;
# }
class Solution {
    def findLeaves(self, root: TreeNode) -> List[List[int]]:
        def dfs(root, prev, t):
            if root is None:
                return
            if root.left is None and root.right is None:
                ans.append(root.val)
                prev.left = root
            else:
                prev.right = None
                dfs(root.left, root, t)
                dfs(root.right, root, t)
        res = []
        prev = TreeNode(-1)
        while prev.left:
            res.append(prev.val)
            prev = prev.left
        res.append(t)
        return res

```

C++

C++

[*] Trees & BST - Pattern Solution (matched from community answers)

Python

```

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    ans: int # the minimum value in the subtree
    mx: int # the maximum value in the subtree
    size: int # the size of the subtree

class Solution:
    def largestBSTSubtree(self, root: TreeNode) -> int:
        def dfs(root: TreeNode) -> T:
            if not root:
                return T(math.inf, -math.inf, 0)
            l = dfs(root.left)
            r = dfs(root.right)
            if l.mx < root.val < r.mx:
                return T(min(l.mx, root.val), max(r.mx, root.val), l.size + r.size)
            # Mark one as invalid, but still record the size of children
            # Return (inf, -inf) because no node will be > inf or < -inf.
            return T(-math.inf, math.inf, max(l.size, r.size))
        return dfs(root).size

```

#366

MEDIUM

Find Leaves of Binary Tree

LeetCode - Interview - LeetCode - LeetCode - Editorial

Tree - DFS - Sorting

Like Premium 98

Problem:

Given the root of a binary tree, collect a tree's nodes as if you were doing this:

- Collect all the leaf nodes.
- Remove all the leaf nodes.
- Repeat until the tree is empty.

Example 1:

```

# Definition for a binary tree node.
# type TreeNode struct {
#     int val;
#     *Left *TreeNode;
#     *Right *TreeNode;
# }
class Solution {
    def findLeaves(self, root: TreeNode):
        result = []
        def getheight(node):
            # Base case: when the node is None, return 0.
            if not node:
                return 0
            # Recursively find the height of left and right subtrees.
            left_height = getheight(node.left)
            right_height = getheight(node.right)
            # Current node's height is one of left/right heights plus one.
            current_height = max(left_height, right_height) + 1
            # Remove the result list has a sublist for the current height.
            if len(result) < current_height:
                result.append([])
            # Append the current node value to the corresponding height bucket.
            result[current_height - 1].append(node.val)
            return current_height
        getheight(root)
        return result

```

Python

```

class Solution:
    def findLeaves(self, root: TreeNode) -> List[List[int]]:
        ans = []
        def depth(root: TreeNode) -> int:
            # Return the depth of the root (0-indexed)
            if not root:
                return -1
            l = depth(root.left)
            r = depth(root.right)
            h = 1 + max(l, r)
            if len(ans) == h:
                ans.append([])
            ans[h].append(root.val)
            return h
        depth(root)
        return ans

```

Python

```

class Solution {
public:
    vector<vector<int>> findLeaves(TreeNode root) {
        vector<vector<int>> ans;
        depth(root, ans);
        return ans;
    }
private:
    // Return the depth of the root (0-indexed)
    int depth(TreeNode root, vector<vector<int>> ans) {
        if (root == nullptr)
            return -1;
        const int l = depth(root->left, ans);
        const int r = depth(root->right, ans);
        const int h = 1 + max(l, r);
        if (ans.size() == h)
            ans.push_back({});
        ans[h].push_back(root->val);
        return h;
    }
};

```

C++

```

/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *     int val;
 *     *Left *TreeNode;
 *     *Right *TreeNode;
 * }
 */
class Solution {
public:
    vector<vector<int>> findLeaves(TreeNode root) {
        vector<vector<int>> ans;
        function<int(TreeNode)> dfs = [&](TreeNode root) {
            if (root == nullptr)
                return 0;
            int l = dfs(root->left);
            int r = dfs(root->right);
            int h = max(l, r) + 1;
            if (ans.size() == h)
                ans.push_back({});
            ans[h-1].push_back(root->val);
            return h;
        };
        dfs(root);
        return ans;
    }
};

```

```

        ans[h].push_back(root->val);
        return h + 1;
    }
    dfs(root);
    return ans;
}

C++
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode* left;
//     TreeNode* right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, int y, int z) : val(x), left(y), right(z) {}
// };

class Solution {
public:
    vector<vector<int>> findLeaves(TreeNode* root) {
        vector<vector<int>> res;
        TreeNode* prev = new TreeNode(0, root, nullptr);
        while (prev->left) {
            vector<int> t;
            dfs(prev->left, prev, t);
            res.push_back(t);
        }
        return res;
    }

    void dfs(TreeNode* root, TreeNode* prev, vector<int>& t) {
        if (!root) return;
        if ((root->left && root->right) ||
            !root->left && root->right) {
            t.push_back(root->val);
            if (prev->left == root)
                prev->left = nullptr;
            else
                prev->right = nullptr;
            dfs(root->left, root, t);
            dfs(root->right, root, t);
        }
    }
};

Java
Java

```

```

class Solution {
    public List<List<Integer>> findLeaves(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        dfs(root, ans);
        return ans;
    }

    // Returns the depth of the root (0-indexed).
    private int depth(TreeNode root, List<List<Integer>> ans) {
        if (root == null)
            return -1;

        final int l = depth(root.left, ans);
        final int r = depth(root.right, ans);
        final int h = 1 + Math.max(l, r);
        if (ans.size() == h) // Root is leaf.
            ans.add(new ArrayList<>());
        ans.get(h).add(root.val);
        return h;
    }
}

Java
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { this.val = x; }
// }

// Returns the depth of the root (0-indexed).
// private int depth(TreeNode root, List<List<Integer>> ans) {
//     if (root == null)
//         return -1;
//     final int l = depth(root.left, ans);
//     final int r = depth(root.right, ans);
//     final int h = 1 + Math.max(l, r);
//     if (ans.size() == h) // Root is leaf.
//         ans.add(new ArrayList<>());
//     ans.get(h).add(root.val);
//     return h;
// }

```

```

        return 0;
    }
    int l = dfs(root.left);
    int r = dfs(root.right);
    int h = Math.max(l, r);
    if (ans.size() == h) {
        ans.add(new ArrayList<>());
    }
    ans.get(h).add(root.val);
    return h + 1;
}

Java
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// Returns the depth of the root (0-indexed).
// private int depth(TreeNode root, List<List<Integer>> ans) {
//     if (root == null)
//         return -1;
//     final int l = depth(root.left, ans);
//     final int r = depth(root.right, ans);
//     final int h = 1 + Math.max(l, r);
//     if (ans.size() == h) // Root is leaf.
//         ans.add(new ArrayList<>());
//     ans.get(h).add(root.val);
//     return h;
// }

```

```

// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// Returns the depth of the root (0-indexed).
// private int depth(TreeNode root, List<List<Integer>> ans) {
//     if (root == null)
//         return -1;
//     final int l = depth(root.left, ans);
//     final int r = depth(root.right, ans);
//     final int h = 1 + Math.max(l, r);
//     if (ans.size() == h) // Root is leaf.
//         ans.add(new ArrayList<>());
//     ans.get(h).add(root.val);
//     return h;
// }

```

```

// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// Returns the depth of the root (0-indexed).
// private int depth(TreeNode root, List<List<Integer>> ans) {
//     if (root == null)
//         return -1;
//     final int l = depth(root.left, ans);
//     final int r = depth(root.right, ans);
//     final int h = 1 + Math.max(l, r);
//     if (ans.size() == h) // Root is leaf.
//         ans.add(new ArrayList<>());
//     ans.get(h).add(root.val);
//     return h;
// }

```

TypeScript

```

// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

// Returns the depth of the root (0-indexed).
// private int depth(TreeNode root, List<List<Integer>> ans) {
//     if (root == null)
//         return -1;
//     final int l = depth(root.left, ans);
//     final int r = depth(root.right, ans);
//     final int h = 1 + Math.max(l, r);
//     if (ans.size() == h) // Root is leaf.
//         ans.add(new ArrayList<>());
//     ans.get(h).add(root.val);
//     return h;
// }

```

TypeScript


```

//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
// };
//
// class Solution {
// public:
//     int pathSum(TreeNode* root, int targetSum) {
//         unordered_map<long, long> cnt;
//         cnt[0] = 1;
//         auto dfs = [&](this auto& dfs, TreeNode* node, long long s) -> int {
//             if (!node) {
//                 return 0;
//             }
//             s += node->val;
//             int ans = cnt[s - targetSum];
//             ++cnt[s];
//             ans += dfs(node->left, s) + dfs(node->right, s);
//             --cnt[s];
//             return ans;
//         };
//         return dfs(root, 0);
//     };
// };

```

```

//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
//     TreeNode(int x, TreeNode* left, TreeNode* right) : val(x), left(left), right(right) {}
// };
//
// class Solution {
// public:
//     int pathSum(TreeNode* root, int targetSum) {
//         unordered_map<long, long> cnt;
//         cnt[0] = 1;
//         function<int(TreeNode*, long)> dfs = [&](TreeNode* node, long s) -> int {
//             if (!node) return 0;
//             s += node->val;
//             int ans = cnt[s - targetSum];
//             ++cnt[s];
//             ans += dfs(node->left, s) + dfs(node->right, s);
//             --cnt[s];
//             return ans;
//         };
//         return dfs(root, 0);
//     };
// };

```

```

        return dfs(root, 0);
    }
};

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) : val(x), left=null, right=null {}
//     TreeNode(int x, TreeNode left, TreeNode right) {
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }
//
// class Solution {
//     private Map<Long, Integer> cnt = new HashMap<>();
//     private int targetSum;
//
//     public int pathSum(TreeNode root, int targetSum) {
//         cnt.put(0, 1);
//         this.targetSum = targetSum;
//         return dfs(root, 0);
//     }
// }

```

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) : val(x), left=null, right=null {}
//     TreeNode(int x, TreeNode left, TreeNode right) {
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }
//
// class Solution {
//     private Map<Long, Integer> cnt = new HashMap<>();
//     private int targetSum;
//
//     public int pathSum(TreeNode root, int targetSum) {
//         cnt.put(0, 1);
//         this.targetSum = targetSum;
//         return dfs(root, 0);
//     }
// }

```

```

private int dfs(TreeNode node, long s) {
    if (node == null) {
        return 0;
    }
    s += node.val;
    int ans = cnt.getOrDefault(s - targetSum, 0);
    cnt.merge(s, 1, Integer::sum);
    ans += dfs(node.left, s);
    ans += dfs(node.right, s);
    cnt.merge(s, -1, Integer::sum);
    return ans;
}
}

```

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) : val(x), left=null, right=null {}
//     TreeNode(int x, TreeNode left, TreeNode right) {
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }
//
// class Solution {
// public:
//     int pathSum(TreeNode* root, int targetSum) {
//         Dictionary<long, int> cnt = new Dictionary<long, int>();
//
//         int dfs(TreeNode* node, long s) {
//             if (!node) return 0;
//             s += node.val;
//             int ans = cnt.GetValueOrDefault(s - targetSum, 0);
//             cnt[s] = cnt.GetValueOrDefault(s, 0) + 1;
//             ans += dfs(node.left, s);
//
//             ans += dfs(node.right, s);
//             cnt[s]--;
//             return ans;
//         };
//         cnt[0] = 1;
//         return dfs(root, 0);
//     };
// };

```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) : val(x), left=null, right=null {}
//     TreeNode(int x, TreeNode left, TreeNode right) {
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }
//
// class Solution {
//     private Map<Long, Integer> cnt = new HashMap<>();
//     private int targetSum;
//
//     public int pathSum(TreeNode root, int targetSum) {
//         cnt.put(0, 1);
//         this.targetSum = targetSum;
//         return dfs(root, 0);
//     }
// }
//
// private int dfs(TreeNode node, long s) {
//     if (node == null) {
//         return 0;
//     }
//     s += node.val;
//     int ans = cnt.getOrDefault(s - targetSum, 0);
//     cnt.merge(s, 1, Integer::sum);
//     ans += dfs(node.left, s);
//     ans += dfs(node.right, s);
//     cnt.merge(s, -1, Integer::sum);
//     return ans;
// }
}

```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) : val(x), left=null, right=null {}
//     TreeNode(int x, TreeNode left, TreeNode right) {
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }
//
// class Solution {
// public:
//     int pathSum(TreeNode* root, int targetSum) {
//         Dictionary<long, int> cnt = new Dictionary<long, int>();
//
//         int dfs(TreeNode* node, long s) {
//             if (!node) return 0;
//             s += node.val;
//             int ans = cnt.GetValueOrDefault(s - targetSum, 0);
//             cnt[s] = cnt.GetValueOrDefault(s, 0) + 1;
//             ans += dfs(node.left, s);
//
//             ans += dfs(node.right, s);
//             cnt[s]--;
//             return ans;
//         };
//         cnt[0] = 1;
//         return dfs(root, 0);
//     };
// };

```

Java

```

// Definition for a binary tree node.
// @param {TreeNode} root
// @param {number} targetSum
// @return {number}
//
// class Solution {
//     public int pathSum(TreeNode root, int targetSum) {
//         Map<Long, Integer> cnt = new HashMap<>();
//         cnt.put(0, 1);
//         return dfs(root, 0, targetSum, cnt);
//     }
//
//     private int dfs(TreeNode node, long s, int targetSum, Map<Long, Integer> cnt) {
//         if (node == null) {
//             return 0;
//         }
//         s += node.val;
//         int ans = cnt.getOrDefault(s - targetSum, 0);
//         cnt.put(s, cnt.getOrDefault(s, 0) + 1);
//         ans += dfs(node.left, s, targetSum, cnt);
//         ans += dfs(node.right, s, targetSum, cnt);
//         cnt.put(s, cnt.get(s) - 1);
//         return ans;
//     }
// }

```

JavaScript

JavaScript

```

/*
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *   this.val = (val===undefined ? 0 : val)
 *   this.left = (left===undefined ? null : left)
 *   this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} targetSum
 * @return {number}
 */
var pathSum = function (root, targetSum) {
    const cnt = new Map();
    const dfs = (node, s) => {
        if (!node)
            return 0;
        s += node.val;
        let ans = cnt.get(s - targetSum) || 0;
        cnt.set(s, (cnt.get(s) || 0) + 1);
        ans += dfs(node.left, s);
        ans += dfs(node.right, s);
        cnt.set(s, cnt.get(s) - 1);

        return ans;
    };
    cnt.set(0, 1);
    return dfs(root, 0);
};

```

```

//
 * Definition for a binary tree node.
 * function TreeNode(val, left, right) {
 *   this.val = (val===undefined ? 0 : val)
 *   this.left = (left===undefined ? null : left)
 *   this.right = (right===undefined ? null : right)
 * }
 */
/**
 * @param {TreeNode} root
 * @param {number} targetSum
 * @return {number}
 */
var pathSum = function (root, targetSum) {
    const cnt = new Map();
    const dfs = (node, s) => {
        if (!node)
            return 0;
        s += node.val;
        let ans = cnt.get(s - targetSum) || 0;
        cnt.set(s, (cnt.get(s) || 0) + 1);
        ans += dfs(node.left, s);
        ans += dfs(node.right, s);
        cnt.set(s, cnt.get(s) - 1);
    };
    cnt.set(0, 1);
    return dfs(root, 0);
};

```

```

return ans;
};
cnt.set(0, 1);
return dfs(root, 0);
};

TypeScript
TypeScript
/*
 * Definition for a binary tree node.
 * class TreeNode {
 *   val: number;
 *   left: TreeNode | null;
 *   right: TreeNode | null;
 * }
 * constructor(val: number, left?: TreeNode | null, right?: TreeNode | null) {
 *   this.val = (val===undefined ? 0 : val)
 *   this.left = (left===undefined ? null : left)
 *   this.right = (right===undefined ? null : right)
 * }
 */
function pathSum(root: TreeNode | null, targetSum: number): number {
    const cnt: Map<number, number> = new Map();
    const dfs = (node: TreeNode | null, s: number): number => {
        if (!node)
            return 0;
        s += node.val;
        let ans = cnt.get(s - targetSum) ?? 0;
        cnt.set(s, (cnt.get(s) ?? 0) + 1);
        ans += dfs(node.left, s);
        ans += dfs(node.right, s);

        cnt.set(s, (cnt.get(s) ?? 0) - 1);

        return ans;
    };
    cnt.set(0, 1);
    return dfs(root, 0);
}

```

```

/*
 * Definition for a binary tree node.
 * class TreeNode {
 *   val: number;
 *   left: TreeNode | null;
 *   right: TreeNode | null;
 * }
 * constructor(val: number, left?: TreeNode | null, right?: TreeNode | null) {
 *   this.val = (val===undefined ? 0 : val)
 *   this.left = (left===undefined ? null : left)
 *   this.right = (right===undefined ? null : right)
 * }
 */
function pathSum(root: TreeNode | null, targetSum: number): number {
    const cnt: Map<number, number> = new Map();
    const dfs = (node: TreeNode | null, s: number): number => {
        if (!node)
            return 0;
        s += node.val;
        let ans = cnt.get(s - targetSum) ?? 0;
        cnt.set(s, (cnt.get(s) ?? 0) + 1);
        ans += dfs(node.left, s);
        ans += dfs(node.right, s);

        cnt.set(s, (cnt.get(s) ?? 0) - 1);

        return ans;
    };
    cnt.set(0, 1);
    return dfs(root, 0);
}

```

```

/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *   Val int
 *   Left *TreeNode
 *   Right *TreeNode
 * }
 */
func pathSum(root *TreeNode, targetSum int) int {
    cnt := map[int]int{0: 1}
    var dfs func(*TreeNode, int) int
    dfs = func(node *TreeNode, s int) int {
        if node == nil {
            return 0
        }
        s += node.Val
        ans := cnt[s-targetSum]
        cnt[s]++
        ans += dfs(node.Left, s) + dfs(node.Right, s)
        cnt[s]--
        return ans
    }
    return dfs(root, 0)
}

```

```

Go
/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *   Val int
 *   Left *TreeNode
 *   Right *TreeNode
 * }
 */
func pathSum(root *TreeNode, targetSum int) int {
    cnt := map[int]int{0: 1}
    var dfs func(*TreeNode, int) int
    dfs = func(node *TreeNode, s int) int {
        if node == nil {
            return 0
        }
        s += node.Val
        ans := cnt[s-targetSum]
        cnt[s]++
        ans += dfs(node.Left, s) + dfs(node.Right, s)
        cnt[s]--
        return ans
    }
    return dfs(root, 0)
}

```

```

# Python solution for Path Sum III
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # value of the node
        self.left = left # left child
        self.right = right # right child

class Solution:
    def pathSum(self, root: TreeNode, targetSum: int) -> int:
        # Helper function to perform DFS traversal
        def dfs(node, current_sum, prefix_sums):
            if not node:
                return 0

            # Update current sum by including the current node's value
            current_sum += node.val

            # Check if the current sum (current_sum) equals the targetSum
            num_paths = prefix_sums.get(current_sum - targetSum, 0)

            # Update the prefix sum hash map with the current sum
            prefix_sums[current_sum] = prefix_sums.get(current_sum, 0) + 1

            # Recursively search in the left and right subtrees
            num_paths += dfs(node.left, current_sum, prefix_sums)
            num_paths += dfs(node.right, current_sum, prefix_sums)

            # Backtrack: remove the current sum count before returning to the parent node
            prefix_sums[current_sum] -= 1
            return num_paths

        # Initial prefix sum hash map with 0 sum seen once (empty path)
        prefix_sums = {0: 1}
        return dfs(root, 0, prefix_sums)

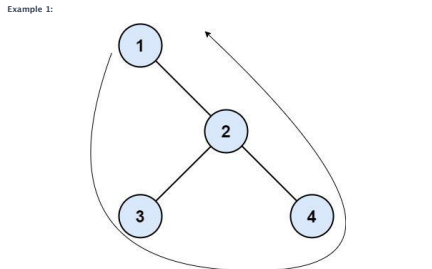
```

#545 MEDIUM
Boundary of Binary Tree
[LeetCode](#) - [SimplifyLeet](#) - [WalkCC](#) - [LeetCode](#) - [Editorial](#)
 Tree - DFS
 LeetCode Premium 98

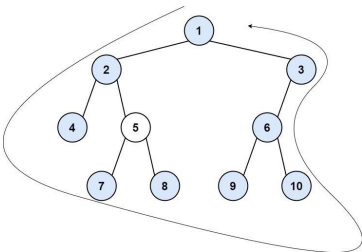
Problem:
 The boundary of a binary tree is the concatenation of the root, the left boundary, the leaves ordered from left-to-right, and the reverse order of the right boundary.
 The left boundary is the set of nodes defined by the following:

- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is empty.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has no left child, but has a right child, then the right child is in the left boundary.
- The leftmost leaf is not in the left boundary.

The right boundary is similar to the left boundary, except it is the right side of the root's right subtree. Again, the leaf is not part of the right boundary, and the right boundary is empty if the root does not have a right child.
 The leaves are nodes that do not have any children. For this problem, the root is not a leaf.
 Given the root of a binary tree, return the values of its boundary.



Example 2:



Input: root = [1,2,3,4,5,6,null,null,7,8,9,10]
Output: [1,2,4,7,8,9,10,6,3]
Explanation:
- The left boundary follows the path starting from the root's left child 2 → 4.
4 is a leaf, so the left boundary is [2].
- The right boundary follows the path starting from the root's right child 3 → 6 → 10.
10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3].
- The leaves from left to right are [2,7,8,9,10].
Concatenating everything results in [1] + [2] + [4,7,8,9,10] + [6,3] = [1,2,4,7,8,9,10,6,3].
Constraints:
• The number of nodes in the tree is in the range [1, 104].
• -1000 ≤ Node.val ≤ 1000

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode) -> List:
        if not root:
            return []

        # Check if a node is a leaf.
        def isLeaf(node):
            return node and not node.left and not node.right

        # Add left boundary nodes (excluding leaves).
        def addLeftBoundary(node, boundary):
            while node:
                if not isLeaf(node):
                    boundary.append(node.val)
                # Preorder visit node, if not present then right child.
                if node.left:
                    node = node.left
                else:
                    node = node.right

        # Add leaf nodes using BFS.
        def addLeaves(node, leaves):
            if not node:
                return
            if isLeaf(node):
                leaves.append(node.val)
            else:
                addLeaves(node.left, leaves)
                addLeaves(node.right, leaves)

        # Add right boundary nodes in reverse order (excluding leaves).
        def addRightBoundary(node, boundary):
            temp = []
            while node:
                if not isLeaf(node):
                    temp.append(node.val)
                if node.right:
                    node = node.right
                else:
                    node = node.left
            # Append in reverse order.
            while temp:
                boundary.append(temp.pop())
```

```
boundary = []
# Add root value if it is not a leaf.
if not isLeaf(root):
    boundary.append(root.val)
addLeftBoundary(root.left, boundary)
addLeaves(root, boundary)
addRightBoundary(root.right, boundary)
return boundary

# Example usage:
# Construct a binary tree and call boundaryOfBinaryTree(root)

Python
class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode | None) -> List[int]:
        if not root:
            return []
        ans = [root.val]

        def dfs(root: TreeNode | None, lb: bool, rb: bool):
            # 1. root left is left boundary if root is left boundary.
            # root right is left boundary if root left is None.
            # 2. Same applies for right boundary.
            # 3. If root is left boundary, add it before 2 children - preorder.
            # If root is right boundary, add it after 2 children - postorder.
            # 4. A leaf node is neither left/right boundary belongs to the bottom.
            if not root:
                return
            if lb:
                ans.append(root.val)
            if not lb and not rb and not root.left and not root.right:
                ans.append(root.val)
            dfs(root.left, lb, rb) and not root.right:
            dfs(root.right, lb and not root.left, rb)

            if rb:
                ans.append(root.val)
            dfs(root.left, True, False)
            dfs(root.right, False, True)
            return ans
```

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def boundaryOfBinaryTree(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(node: List[int], root: Optional[TreeNode], i: int):
            if root is None:
                return
            if i == 0:
                if root.left != root.right:
                    node.append(root.val)
                if root.left:
                    dfs(node, root.left, i)
                else:
                    dfs(node, root.right, i)
            elif i == 1:
                if root.left == root.right:
                    node.append(root.val)
            else:
                dfs(node, root.left, i)
                dfs(node, root.right, i)

            if root.left != root.right:
                node.append(root.val)
            if root.right:
                dfs(node, root.right, i)
            else:
                dfs(node, root.left, i)

        ans = [root.val]
        if root.left == root.right:
            return ans
        left, leaves, right = [], [], []
        dfs(left, root.left, 0)
        dfs(leaves, root, 1)
        dfs(right, root.right, 2)
        ans = left + leaves + right[::-1]
        return ans
```

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode) -> List[int]:
        self.res = []
        if not root:
            return self.res
        if not self.isLeaf(root):
            self.res.append(root.val)
        # left boundary
        t = root.left
        while t:
            if not self.isLeaf(t):
                self.res.append(t.val)
            t = t.left if t.left else t.right
        # leaves
        self.add_leaves(root)
        # right boundary (reverse order)
        s = []
        t = root.right
        while t:
            if not self.isLeaf(t):
                s.append(t.val)
            t = t.right if t.right else t.left
        while s:
            self.res.append(s.pop())
        # output
        return self.res

    def add_leaves(self, root):
        if not root:
            return
        self.res.append(root.val)
        if root.left:
            self.add_leaves(root.left)
        if root.right:
            self.add_leaves(root.right)

    def isLeaf(self, node) -> bool:
        return node and node.left is None and node.right is None
```

C++

```
class Solution {
public:
    vector<int> boundaryOfBinaryTree(TreeNode* root) {
        if (root == nullptr)
            return {};
        vector<int> ans(root->val);
        dfs(root->left, true, false, ans);
        dfs(root->right, false, true, ans);
        return ans;
    }

private:
    // 1. root-left is left boundary if root is left boundary.
    // root-right is left boundary if root-left == nullptr.
    // 2. Same applies for right boundary.
    // 3. If root is left boundary, add it before 2 children - preorder.
    // If root is right boundary, add it after 2 children - postorder.
    // 4. A leaf node is neither left/right boundary belongs to the bottom.
    void dfs(TreeNode* root, bool lb, bool rb, vector<int>& ans) {
        if (root == nullptr)
            return;
        if (lb)
            ans.push_back(root->val);
        if ((lb && rb && root->left == nullptr && root->right != nullptr)
            || ans.push_back(root->val);
        dfs(root->left, lb, rb && root->right == nullptr, ans);
        dfs(root->right, lb && root->left == nullptr, rb, ans);
        if (rb)
            ans.push_back(root->val);
    }
};
```

C++

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

LeetMastery

```
        dfs(nums, root.left, i);
    }
}

dfs(left, root.left, 0);
dfs(leaves, root, 1);
dfs(right, root.right, 2);

return ans.concat(lefts).concat(leaves).concat(right.reverse());
}

Go
Go

/*
 * Definition for a binary tree node.
 * type TreeNode struct {
 *     Val int
 *     Left *TreeNode
 *     Right *TreeNode
 * }
 */
func boundaryOfBinaryTree(root *TreeNode) []int {
    ans := []int{root.Val}
    if root.Left != root.Right {
        return ans
    }

    left, leaves, right := []int{}, []int{}, []int{}

    var dfs func(node *TreeNode, i int)
    dfs = func(node *TreeNode, i int) {
        if root == nil {
            return
        }
        if i == 0 {
            if root.Left != root.Right {
                ans = append(ans, root.Val)
            }
            if root.Left != nil {
                dfs(root.Left, 1)
            }
        }
    }

    dfs(left, root.Left, 0)
    dfs(leaves, root, 1)
    dfs(right, root.Right, 2)

    ans = append(ans, left...)
    ans = append(ans, leaves...)
    for i := len(right) - 1; i >= 0; i-- {
        ans = append(ans, right[i])
    }

    return ans
}
```

```
        dfs(nums, root.left, 1)
    } else {
        dfs(nums, root.Right, 1)
    }
}

} else if i == 1 {
    if root.Left == root.Right {
        ans = append(ans, root.Val)
    } else {
        dfs(nums, root.Left, 1)
        dfs(nums, root.Right, 1)
    }
} else {
    if root.Left != root.Right {
        ans = append(ans, root.Val)
        if root.Right != nil {
            dfs(nums, root.Right, 1)
        } else {
            dfs(nums, root.Left, 1)
        }
    }
}

dfs(left, root.Left, 0)

dfs(leaves, root, 1)
dfs(right, root.Right, 2)

ans = append(ans, left...)
ans = append(ans, leaves...)
for i := len(right) - 1; i >= 0; i-- {
    ans = append(ans, right[i])
}

return ans
}
```

[*] Trees & BST - Pattern Solution (matched from community answers)

Python

```
class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = [root.val]

        def dfs(root: TreeNode | None, lb: bool, rb: bool):
            1. root.left is left boundary if root is left boundary.
            2. root.right is left boundary if root.left is None.
            3. Same applies for right boundary.
            4. If root is left boundary, add it before 2 children - preorder.
            5. If root is right boundary, add it after 2 children - postorder.
            6. A leaf node is neither left/right boundary belongs to the bottom.

            if not root:
                return
            if lb:
                ans.append(root.val)
            if not lb and not rb and not root.left and not root.right:
                ans.append(root.val)
            dfs(root.left, lb, rb and not root.right)
            dfs(root.right, lb and not root.left, rb)

            if rb:
                ans.append(root.val)

        dfs(root.left, True, False)
        dfs(root.right, False, True)
        return ans
```

#549

MEDIUM

Binary Tree Longest Consecutive Sequence II

LeetCode - SimplyLeetCode - WubCC - LeetCode - Editorial

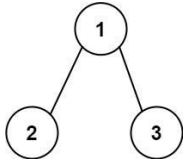
Tree - DFS
Little Premium 98

Problem:

Given the root of a binary tree, return the length of the longest consecutive path in the tree. A consecutive path is a path where the values of the consecutive nodes in the path differ by one. This path can be either increasing or decreasing.

For example, [1,2,3,4] and [4,3,2,1] are both considered valid, but the path [1,2,4,3] is not valid. On the other hand, the path can be in the child-Parent-child order, where not necessarily be parent-child order.

Example 1:

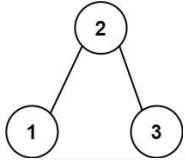


Input: root = [1,2,3]

Output: 2

Explanation: The longest consecutive path is [1, 2] or [2, 1].

Example 2:



Input: root = [2,1,3]

Output: 3

Explanation: The longest consecutive path is [1, 2, 3] or [3, 2, 1].

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- 3 * 10⁴ <= Node.val <= 3 * 10⁴

Python

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        # Definition of the maximum path length variable
        self.max_length = 0

        def dfs(node):
            if not node:
                return (0, 0) # (increasing, decreasing)

            inc = dec = 1 # Each node counts as length 1

            # Process left child
            if node.left:
                left_inc, left_dec = dfs(node.left)
                # If node and left child are consecutive increasing with respect to node
                if node.left.val == node.val + 1:
                    inc = max(inc, left_inc + 1)
                # If left child's value is consecutive decreasing with respect to node
                elif node.left.val == node.val - 1:
                    dec = max(dec, left_dec + 1)

            # Process right child
            if node.right:
                right_inc, right_dec = dfs(node.right)
                # If right child's value is consecutive increasing with respect to node
                if node.right.val == node.val + 1:
                    inc = max(inc, right_inc + 1)
                # If right child's value is consecutive decreasing with respect to node
                elif node.right.val == node.val - 1:
                    dec = max(dec, right_dec + 1)

            # Update the global maximum path by combining increasing and decreasing paths
            self.max_length = max(self.max_length, inc + dec - 1)
            return (inc, dec)

        dfs(root)
        return self.max_length
```

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return (0, 0)

            inc = dec = 1
            li, di = dfs(root.left)
            ri, dr = dfs(root.right)

            if root.left:
                if root.left.val == root.val + 1:
                    inc = li + 1
                if root.left.val == root.val - 1:
                    dec = di + 1

            if root.right:
                if root.right.val == root.val + 1:
                    inc = max(inc, ri + 1)
                if root.right.val == root.val - 1:
                    dec = max(dec, dr + 1)

            ans = max(inc, dec)
            return (inc, dec)

        ans = 0
        dfs(root)
        return ans
```

Python

Trees & BST · LeetMastery — By Pattern — 3307 — LeetMastery

Trees & BST · LeetMastery — By Pattern — 3308 — LeetMastery

Trees & BST · LeetMastery — By Pattern — 3309 — LeetMastery

Trees & BST · LeetMastery — By Pattern — 3310 — LeetMastery

Trees & BST · LeetCode — By Pattern — 3311 — LeetCode

Trees & BST · LeetMastery — By Pattern — 3312 — LeetMastery


```
class Solution {
    private Map<Integer, List<Integer>> g = new HashMap<>();
    private List<Integer> ans = new ArrayList<>();

    public List<Integer> killProcess(List<Integer> pid, List<Integer> ppid, int kill) {
        int n = pid.size();
        for (int i = 0; i < n; ++i) {
            g.computeIfAbsent(pid.get(i), k -> new ArrayList<>()).add(pid.get(i));
        }
        dfs(kill);
        return ans;
    }

    private void dfs(int i) {
        ans.add(i);
        for (int j : g.getOrDefault(i, Collections.emptyList())) {
            dfs(j);
        }
    }
}

Java

use id: collections: HashMap;

impl Solution {
    pub fn kill_process(pid: Vec<i32>, ppid: Vec<i32>, kill: i32) -> Vec<i32> {
        let mut g: HashMap<i32, Vec<i32>> = HashMap::new();
        let mut ans: Vec<i32> = Vec::new();

        let n = pid.len();
        for i in 0..n {
            g.entry(pid[i]).or_insert(Vec::new()).push(pid[i]);
        }

        Self.dfs(&mut ans, &g, kill);
        ans
    }

    fn dfs(ans: &mut Vec<i32>, g: &HashMap<i32, Vec<i32>>, i: i32) {
        ans.push(i);
        if let Some(children) = g.get(&i) {
            for &j in children {
                Self.dfs(ans, g, *j);
            }
        }
    }
}

Java
```

```
class Solution {
    private Map<Integer, List<Integer>> g = new HashMap<>();
    private List<Integer> ans = new ArrayList<>();

    public List<Integer> killProcess(List<Integer> pid, List<Integer> ppid, int kill) {
        int n = pid.size();
        for (int i = 0; i < n; ++i) {
            g.computeIfAbsent(pid.get(i), k -> new ArrayList<>()).add(pid.get(i));
        }
        dfs(kill);
        return ans;
    }

    private void dfs(int i) {
        ans.add(i);
        for (int j : g.getOrDefault(i, Collections.emptyList())) {
            dfs(j);
        }
    }
}

JavaScript

use id: collections: HashMap;

impl Solution {
    pub fn kill_process(pid: Vec<i32>, ppid: Vec<i32>, kill: i32) -> Vec<i32> {
        let mut g: HashMap<i32, Vec<i32>> = HashMap::new();
        let mut ans: Vec<i32> = Vec::new();

        let n = pid.len();
        for i in 0..n {
            g.entry(pid[i]).or_insert(Vec::new()).push(pid[i]);
        }

        Self.dfs(&mut ans, &g, kill);
        ans
    }

    fn dfs(ans: &mut Vec<i32>, g: &HashMap<i32, Vec<i32>>, i: i32) {
        ans.push(i);
        if let Some(children) = g.get(&i) {
            for &j in children {
                Self.dfs(ans, g, *j);
            }
        }
    }
}

TypeScript
TypeScript
```

```
function killProcess(pid, ppid, number) {
    const g = Map<number, number[]> = new Map();
    for (let i = 0; i < pid.length; ++i) {
        if (!g.has(ppid[i])) {
            g.set(ppid[i], []);
        }
        g.get(ppid[i]).push(pid[i]);
    }
    const ans = number[] = [];
    const dfs = (i: number) => {
        ans.push(i);
        for (const j of g.get(i) ?? []) {
            dfs(j);
        }
    };
    dfs(kill);
    return ans;
}

TypeScript

function killProcess(pid: number[], ppid: number[], kill: number): number[] {
    const g: Map<number, number[]> = new Map();
    for (let i = 0; i < pid.length; ++i) {
        if (!g.has(ppid[i])) {
            g.set(ppid[i], []);
        }
        g.get(ppid[i]).push(pid[i]);
    }
    const ans = number[] = [];
    const dfs = (i: number) => {
        ans.push(i);
        for (const j of g.get(i) ?? []) {
            dfs(j);
        }
    };
    dfs(kill);
    return ans;
}

Go

func killProcess(pid []int, ppid []int, kill int) ans []int {
    g := map[int][]int{}
    for i, p := range ppid {
        g[p] = append(g[p], pid[i])
    }
    var dfs func(int)
    dfs = func(i int) {
        ans = append(ans, i)
        for j := range g[i] {
            dfs(j)
        }
    }
    dfs(kill)
    return ans
}

Go

func killProcess(pid []int, ppid []int, kill int) ans []int {
    g := map[int][]int{}
    for i, p := range ppid {
        g[p] = append(g[p], pid[i])
    }
    var dfs func(int)
    dfs = func(i int) {
        ans = append(ans, i)
        for j := range g[i] {
            dfs(j)
        }
    }
    dfs(kill)
    return ans
}

Go
```

```
Go

func killProcess(pid []int, ppid []int, kill int) ans []int {
    g := map[int][]int{}
    for i, p := range ppid {
        g[p] = append(g[p], pid[i])
    }
    var dfs func(int)
    dfs = func(i int) {
        ans = append(ans, i)
        for j := range g[i] {
            dfs(j)
        }
    }
    dfs(kill)
    return ans
}

Python

class Solution:
    def killProcess(
        self,
        pid: List[int],
        ppid: List[int],
        kill: int
    ) -> List[int]:
        ans = []
        tree = collections.defaultdict(list)

        for u, p in zip(pid, ppid):
            if u == 0:
                continue
            tree[p].append(u)

        def dfs(i: int) -> None:
            ans.append(i)
            for v in tree.get(i, []):
                dfs(v)

        dfs(kill)
        return ans

#662 MEDIUM
Maximum Width of Binary Tree
LeetCode - Solution - WalkCC - LeetCode - Editorial
Tree - BFS
List: Grind 169

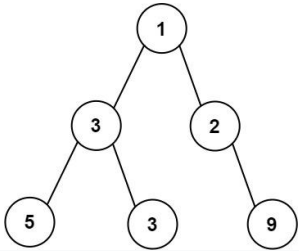
Problem:
Given the root of a binary tree, return the maximum width of the given tree.
The maximum width of a tree is the maximum width among all levels.

Python

class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        queue = deque([(root, 0)])
        max_width = 0
        while queue:
            level_size = len(queue)
            first_index = None
            last_index = None
            for i in range(level_size):
                node, index = queue.popleft()
                if i == 0:
                    first_index = index
                if i == level_size - 1:
                    last_index = index
                if node.left:
                    queue.append((node.left, index * 2))
                if node.right:
                    queue.append((node.right, index * 2 + 1))
            max_width = max(max_width, last_index - first_index + 1)
        return max_width
```

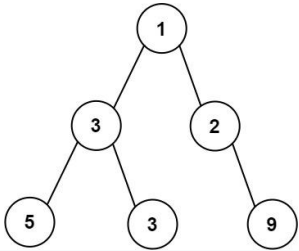
The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation. It is guaranteed that the answer will in the range of a 32-bit signed integer.

Example 1:

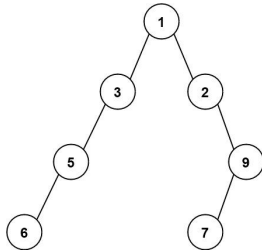


```
Input: root = [1,3,2,5,3,null,9]
Output: 4
Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).
```

Example 2:

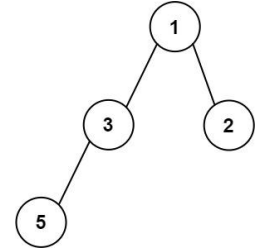


```
Input: root = [1,3,2,5,3,null,9]
Output: 4
Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).
```



```
Input: root = [1,3,2,5,null,null,9,6,null,7]
Output: 7
Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,7).
```

Example 3:



Input: root = [1,3,2,5]
Output: 2
Explanation: The maximum width exists in the second level with length 2 ([3,2]).

- Constraints:
- The number of nodes in the tree is in the range [1, 3000].
 - -100 <= Node.val <= 100

Python

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from collections import deque

class Solution:
    def widthOfBinaryTree(self, root: TreeNode) -> int:
        if not root:
            return 0
        ans_width = 0
        # Initialize queue with a tuple of (node, index)
        queue = deque([(root, 1)])

        while queue:
            level_length = len(queue)
            # Get the index of first node at current level
            first_index = queue[0][1]
            # Process nodes at current level
            for i in range(level_length):
                node, index = queue.popleft()

                # Check and add left child with index 2 * index
                if node.left:
                    queue.append((node.left, 2 * index))
                # Check and add right child with index 2 * index + 1
                if node.right:
                    queue.append((node.right, 2 * index + 1))
            # For the last node, calculate width of the level
            if i == level_length - 1:
                current_width = index - first_index + 1
                ans_width = max(ans_width, current_width)
            return ans_width
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        ans = 0
        q = deque([(root, 1)])
        while q:
            ans = max(ans, q[-1][1] - q[0][1] + 1)
            for i in range(len(q)):
                root, i = q.popleft()
                if root.left:
                    q.append((root.left, i + 1))
                if root.right:
                    q.append((root.right, i + 1 | 1))
            return ans
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        ans = 0
        q = deque([(root, 1)])
        while q:
            ans = max(ans, q[-1][1] - q[0][1] + 1)
            for i in range(len(q)):
                root, i = q.popleft()
                if root.left:
                    q.append((root.left, i + 1))
                if root.right:
                    q.append((root.right, i + 1 | 1))
            return ans
```

Java

Java

```
class Solution {
    public int widthOfBinaryTree(TreeNode root) {
        if (root == null)
            return 0;

        int ans = 0;
        // Queue: Pair<TreeNode, Integer> q = new ArrayDeque<>();
        Deque<Pair<TreeNode, Integer>> q = new ArrayDeque<>();
        q.offer(new Pair<>((root, 1)));

        while (!q.isEmpty()) {
            final int offset = q.peekFirst().getValue() + 2;
            ans = Math.max(ans, q.peekLast().getValue() - q.peekFirst().getValue() + 1);
            for (int sz = q.size(); sz > 0; --sz) {
                final TreeNode node = q.pollFirst().getValue();
                final int index = q.pollFirst().getValue();
                if (node.left != null)
                    q.offer(new Pair<>((node.left, index + 2 - offset)));
                if (node.right != null)
                    q.offer(new Pair<>((node.right, index + 2 + 1 - offset)));
            }
        }

        return ans;
    }
}
```

Java

```
/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

class Solution {
    public int widthOfBinaryTree(TreeNode root) {
        Deque<Pair<TreeNode, Integer>> q = new ArrayDeque<>();
        q.offer(new Pair<>((root, 1)));

        int ans = 0;
        while (!q.isEmpty()) {
            ans = Math.max(ans, q.peekLast().getValue() - q.peekFirst().getValue() + 1);
            for (int n = q.size(); n > 0; --n) {
                var p = q.pollFirst();
                root = p.getValue();
            }
        }

        return ans;
    }
}
```

```
int i = p.getValue();
if (root.left != null) {
    q.offer(new Pair<>((root.left, i + 1)));
}
if (root.right != null) {
    q.offer(new Pair<>((root.right, i + 1 | 1)));
}
return ans;
}
}
```

Java

```
/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

class Solution {
    public int widthOfBinaryTree(TreeNode root) {
        Deque<Pair<TreeNode, Integer>> q = new ArrayDeque<>();
        q.offer(new Pair<>((root, 1)));

        int ans = 0;
        while (!q.isEmpty()) {
            ans = Math.max(ans, q.peekLast().getValue() - q.peekFirst().getValue() + 1);
            for (int n = q.size(); n > 0; --n) {
                var p = q.pollFirst();
                root = p.getValue();
            }
        }

        return ans;
    }
}
```

[+] Trees & BST - Pattern Solution (matched from community answers)

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from collections import deque

class Solution:
    def widthOfBinaryTree(self, root: TreeNode) -> int:
        if not root:
            return 0
        ans_width = 0
        # Initialize queue with a tuple of (node, index)
        queue = deque([(root, 1)])

        while queue:
            level_length = len(queue)
            # Get the index of first node at current level
            first_index = queue[0][1]
            # Process nodes at current level
            for i in range(level_length):
                node, index = queue.popleft()

                # Check and add left child with index 2 * index
                if node.left:
                    queue.append((node.left, 2 * index))
                # Check and add right child with index 2 * index + 1
                if node.right:
                    queue.append((node.right, 2 * index + 1))
            # For the last node, calculate width of the level
            if i == level_length - 1:
                current_width = index - first_index + 1
                ans_width = max(ans_width, current_width)
            return ans_width
```

#863

MEDIUM

All Nodes Distance K in Binary Tree

LeetCode - SimplifyLeetCode - LeetCode - Editorial

Hash Table - Tree - DFS - BFS

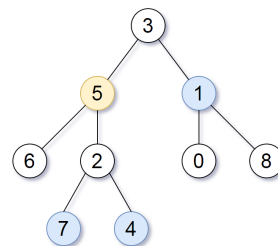
Uppis Grind 109

Problem:

Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:

Input: root = [1], target = 1, k = 3

Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 500].
- 0 <= Node.val <= 500
- All the values Node.val are unique.
- target is the value of one of the nodes in the tree.
- 0 <= k <= 1000

>> Brute Force [Trees & BST] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def distanceK(self, root: TreeNode, target: int, k: int):
        # Brute Force - collect inorder, then process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)
        collect(root)
        return vals[k] if len(vals) > k else []

Python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def distanceK(root: TreeNode, target: int, k: int):
    from collections import defaultdict, deque

    # Build an adjacency list to represent the graph
    graph = defaultdict(list)

    # Helper function to build the graph using DFS
    def buildGraph(node, parent):
        if node is None:
            return
        if parent is not None:
            # Add undirected connection between node and parent
            graph[node.val].append(parent.val)
            graph[parent.val].append(node.val)
        buildGraph(node.left, node)
        buildGraph(node.right, node)

    buildGraph(root, None)
```

```
# DFS from target node
visited = set([target])
queue = deque([target])
current_distance = 0

while queue:
    if current_distance == k:
        # Return all nodes at distance k
        return list(queue)
    size = len(queue)
    for _ in range(size):
        current = queue.popleft()
        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    current_distance += 1
    return [] # If no nodes at distance k

# Example usage:
# Construct tree to use a tree builder before calling distanceK function.
```

```
Python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]:
        def dfs1(root, fa):
            if root is None:
                return
            g[root] = fa
            dfs1(root.left, root)
            dfs1(root.right, root)

        def dfs2(root, fa, k):
            if root is None:
                return
            if k == 0:
                ans.append(root.val)
            for node in (root.left, root.right, g[root]):
                if node != fa:
                    dfs2(node, root, k - 1)

        g = {}
        dfs1(root, None)
        ans = []
        dfs2(target, None, k)
        return ans
```

```
Python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]:
        def parents(root, prev):
            nonlocal p
            if root is None:
                return
            p[root] = prev
            parents(root.left, root)
            parents(root.right, root)

        def dfs1(root, k):
            nonlocal ans, vis
            if root is None or root.val in vis:
                return
            vis.add(root.val)
            if k == 0:
                ans.append(root.val)

            return
            dfs1(root.left, k - 1)
            dfs1(root.right, k - 1)
            dfs1(g[root], k - 1)

        p = {}
        parents(root, None)
        ans = []
        vis = set()
        dfs1(target, k)
        return ans

C++
C++
```

```
class Solution {
public:
    vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
        vector<int> ans;
        unordered_map<TreeNode*, int> nodeToDist; // (node, distance to target)

        getDist(root, target, nodeToDist);
        dfs1(root, k, nodeToDist, ans);
        return ans;
    }

private:
    void getDist(TreeNode* root, TreeNode* target,
        unordered_map<TreeNode*, int>& nodeToDist) {
        if (root == nullptr)
            return;
        if (root == target) {
            nodeToDist[root] = 0;
            return;
        }

        getDist(root->left, target, nodeToDist);
        if (const auto it = nodeToDist.find(root->left); it != nodeToDist.cend()) {
            // The target is in the left subtree
            nodeToDist[root] = it->second + 1;
        }

        getDist(root->right, target, nodeToDist);
        if (const auto it = nodeToDist.find(root->right); it != nodeToDist.cend()) {
            // The target is in the right subtree
            nodeToDist[root] = it->second + 1;
        }

        void dfs(TreeNode* root, int k, int dist,
            unordered_map<TreeNode*, int>& nodeToDist, vector<int>& ans) {
            if (root == nullptr)
                return;
            if (const auto it = nodeToDist.find(root); it != nodeToDist.cend()) {
                dist = it->second;
                if (dist == k)
                    ans.push_back(root->val);

                dfs(root->left, k, dist + 1, nodeToDist, ans);
                dfs(root->right, k, dist + 1, nodeToDist, ans);
            }
        }
};

C++
```

```
//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode* left;
//     TreeNode* right;
//     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
// };

class Solution {
public:
    vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
        unordered_map<TreeNode*, TreeNode*> g;
        vector<int> ans;

        auto dfs = [&](this auto& dfs, TreeNode* node, TreeNode* fa) {
            if (!node) return;
            g[node] = fa;
            dfs(node->left, node);
            dfs(node->right, node);
        };

        auto dfs2 = [&](this auto& dfs2, TreeNode* node, TreeNode* fa, int k) {
            if (!node) return;
            if (k == 0) {
                ans.push_back(node->val);
                return;
            }
            for (auto& next : {node->left, node->right, g[node]}) {
                if (next != fa) {
                    dfs2(next, node, k - 1);
                }
            }
        };

        dfs1(root, nullptr);
        dfs2(target, nullptr, k);
        return ans;
    }
};

C++
```

```
//
// Definition for a binary tree node.
// struct TreeNode {
//     int val;
//     TreeNode* left;
//     TreeNode* right;
//     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
// };

class Solution {
public:
    unordered_map<TreeNode*, TreeNode*> p;
    unordered_map<int, int> vis;
    vector<int> ans;

    void parents(TreeNode* root, TreeNode* prev) {
        if (!root) return;
        p[root] = prev;
        parents(root->left, root);
        parents(root->right, root);
    }

    void dfs(TreeNode* root, int k) {
        if (!root || vis.count(root->val)) return;
        vis.insert(root->val);
        if (k == 0) {
            ans.push_back(root->val);
            return;
        }
        dfs(root->left, k - 1);
        dfs(root->right, k - 1);
        dfs(g[root], k - 1);
    }
};

Java
Java
```



```

class Solution {
    public List<Integer> distance(TreeNode root, TreeNode target, int k) {
        List<Integer> ans = new ArrayList<>();
        Map<TreeNode, Integer> nodeToDist = new HashMap<>(); // node: distance to target

        getDist(root, target, nodeToDist);
        dfs(root, k, 0, nodeToDist, ans);

        return ans;
    }

    private void getDist(TreeNode root, TreeNode target, Map<TreeNode, Integer> nodeToDist) {
        if (root == null)
            return;
        if (root == target) {
            nodeToDist.put(root, 0);
            return;
        }
        getDist(root.left, target, nodeToDist);
        if (nodeToDist.containsKey(root.left)) {
            // the target is in the left subtree
            nodeToDist.put(root, nodeToDist.get(root.left) + 1);
            return;
        }
        getDist(root.right, target, nodeToDist);
        if (nodeToDist.containsKey(root.right)) {
            // the target is in the right subtree
            nodeToDist.put(root, nodeToDist.get(root.right) + 1);
        }

        private void dfs(TreeNode root, int k, int dist, Map<TreeNode, Integer> nodeToDist,
            List<Integer> ans) {
            if (root == null)
                return;
            if (nodeToDist.containsKey(root))
                dist = nodeToDist.get(root);
            if (dist == k)
                ans.add(root.val);

            dfs(root.left, k, dist + 1, nodeToDist, ans);
            dfs(root.right, k, dist + 1, nodeToDist, ans);
        }
    }
}

```

Java

```

//
// Definition for a binary tree node.
// public class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

class Solution {
    private Map<TreeNode, Integer> p;
    private Set<Integer> vis;
    private List<Integer> ans;

    public List<Integer> distance(TreeNode root, TreeNode target, int k) {
        p = new HashMap<>();
        vis = new HashSet<>();
        ans = new ArrayList<>();
        parents(root, null);
        dfs(target, k);
        return ans;
    }

    private void parents(TreeNode root, TreeNode prev) {
        if (root == null) {
            return;
        }
        p.put(root, prev);
        parents(root.left, root);
        parents(root.right, root);
    }

    private void dfs(TreeNode root, int k) {
        if (root == null || vis.contains(root.val)) {
            return;
        }
        vis.add(root.val);
        if (k == 0) {
            ans.add(root.val);
            return;
        }
        dfs(root.left, k - 1);
        dfs(root.right, k - 1);
        dfs(p.get(root), k - 1);
    }
}

```

TypeScript
TypeScript

```

//
// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

function distance(root: TreeNode | null, target: TreeNode | null, k: number): number[] {
    if (!root) return [];

    const g: Record<number, number[]> = {};

    const dfs = (node: TreeNode | null, parent: TreeNode | null = null) => {
        if (!node) return;

        g[node.val] ??= [];
        if (parent) g[node.val].push(parent.val);
        if (node.left) g[node.val].push(node.left.val);

        if (node.right) g[node.val].push(node.right.val);

        dfs(node.left, node);
        dfs(node.right, node);
    };

    const vis = new Set<number>();
    let q = [target.val];

    while (q.length) {
        if (k-- === 0) return q;

        const next: number[] = [];
        for (const x of q) {
            if (vis.has(x)) continue;
            vis.add(x);
            next.push(...g[x].filter(x => !vis.has(x)));
        }
        q = next;
    }

    return [];
}

```

Go

```

func distance(root *TreeNode, target *TreeNode, k int) []int {
    g := make(map[*TreeNode][]*TreeNode)
    ans = []int{}

    var dfs func(node, fa *TreeNode)
    dfs = func(node, fa *TreeNode) {
        if node == nil {
            return
        }
        g[node] = fa
        dfs(node.Left, node)
        dfs(node.Right, node)
    }

    var dfs2 func(node, fa *TreeNode, k int)
    dfs2 = func(node, fa *TreeNode, k int) {
        if node == nil {
            return
        }
        if k == 0 {
            ans = append(ans, node.Val)
            return
        }
        for _, x := range []{*TreeNode, *TreeNode, *TreeNode} {
            if x == fa {
                continue
            }
            dfs2(x, node, k-1)
        }
    }

    dfs(root, nil)
    dfs2(target, nil, k)

    return ans
}

```

Go

```

//
// Definition for a binary tree node.
// type TreeNode struct {
//     Val int
//     Left *TreeNode
//     Right *TreeNode
// }

func distance(root *TreeNode, target *TreeNode, k int) []int {
    p := make(map[*TreeNode]*TreeNode)
    vis := make(map[int]bool)

    var ans []int
    var parents func(root, prev *TreeNode)
    parents = func(root, prev *TreeNode) {
        if root == nil {
            return
        }
        p[root] = prev
        parents(root.Left, root)
        parents(root.Right, root)
    }

    parents(root, nil)
    var dfs func(root *TreeNode, k int)
    dfs = func(root *TreeNode, k int) {
        if root == nil || vis[root.Val] {
            return
        }
        vis[root.Val] = true
        if k == 0 {
            ans = append(ans, root.Val)
            return
        }
        dfs(root.Left, k-1)
        dfs(root.Right, k-1)
        dfs(p[root], k-1)
    }

    dfs(target, k)
    return ans
}

```

[*] Trees & BST - Pattern Solution (matched from community answers)

Python

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def distance(self, root: TreeNode, target: TreeNode, k: int) -> List[int]:
        def dfs(root, fa):
            if root is None:
                return
            g[root] = fa
            dfs(root.left, root)
            dfs(root.right, root)

        def dfs2(root, fa, k):
            if root is None:
                return
            if k == 0:
                ans.append(root.val)
                return
            for x in (root.left, root.right, g[root]):
                if x == fa:
                    continue
                dfs2(x, root, k - 1)

        g = {}
        dfs(root, None)
        ans = []
        dfs2(target, None, k)
        return ans

```

#1120 MEDIUM
Maximum Average Subtree
LeetCode - Simplify - [LeetCode](#) - [LeetCode](#) - [LeetCode](#)

Tree - DFS

LeetCode Premium 98

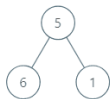
Problem:

Given the root of a binary tree, return the maximum average value of a subtree of that tree. Answers within 10⁻⁵ of the actual answer will be accepted.

A subtree of a tree is any node of that tree plus all its descendants.

The average value of a tree is the sum of its values, divided by the number of nodes.

Example 1:



Input: root = [5,6,1]
Output: 6.00000
Explanation:
For the node with value = 5 we have an average of (5 + 6 + 1) / 3 = 4.
For the node with value = 6 we have an average of 6 / 1 = 6.
For the node with value = 1 we have an average of 1 / 1 = 1.
So the answer is 6 which is the maximum.

Example 2:

Input: root = [0,null,1]
Output: 1.00000

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 0 <= Node.val <= 105

>> Brute Force [Trees & BST] Interview Prep

```
Python
# Define DFS Approach
class Solution:
    def maximumAverageSubtree(self, root):
        # Base Case - return 0, because then process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)
        collect(root)
        return vals[0] if vals else 0
```

Python

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # value of the node
        self.left = left # left child
        self.right = right # right child

class Solution:
    def maximumAverageSubtree(self, root):
        # Initialize the answer average to negative infinity
        self.maxAvg = float('-inf')

        # Recursive DFS function that returns a tuple (subtree sum, node count)
        def dfs(node):
            if not node:
                return (0, 0) # return sum 0 and count 0 if node is null
            leftSum, leftCount = dfs(node.left) # traverse left subtree
            rightSum, rightCount = dfs(node.right) # traverse right subtree
            currentSum = node.val + leftSum + rightSum # total sum at current node
            currentCount = 1 + leftCount + rightCount # total node count
            currentAvg = currentSum / currentCount # average value for current subtree
            self.maxAvg = max(self.maxAvg, currentAvg) # update maximum average if necessary
            return (currentSum, currentCount)

        dfs(root)

        return self.maxAvg
```

Python

```
from dataclasses import dataclass

@dataclass(frozen=True)
class FS:
    sum: int
    count: int
    maxAverage: int

class Solution:
    def maximumAverageSubtree(self, root: TreeNode | None) -> float:
        def dfs(node: TreeNode | None) -> FS:
            if not node:
                return FS(0, 0, 0)
            left = dfs(node.left)
            right = dfs(node.right)
            sum = root.val + left.sum + right.sum
            count = 1 + left.count + right.count
            maxAverage = max(sum / count, left.maxAverage, right.maxAverage)
            return FS(sum, count, maxAverage)

        return dfs(root).maxAverage
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maximumAverageSubtree(self, root: Optional[TreeNode]) -> float:
        def dfs(root):
            if root is None:
                return 0, 0
            ls, lsCnt = dfs(root.left)
            rs, rsCnt = dfs(root.right)
            s = root.val + ls + rs
            n = 1 + lsCnt + rsCnt
            ans = Math.max(ans, s / n)
            return s, n

        ans = 0
        dfs(root)
        return ans
```

Python

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maximumAverageSubtree(self, root: Optional[TreeNode]) -> float:
        def dfs(node):
            if not node:
                return 0, 0
            ls, lsCnt = dfs(node.left)
            rs, rsCnt = dfs(node.right)
            s = node.val + ls + rs
            n = 1 + lsCnt + rsCnt
            ans = Math.max(ans, s / n)
            return s, n

        ans = 0
        dfs(root)
        return ans
```

Java

Java

```
class Solution {
    public double maximumAverageSubtree(TreeNode root) {
        return maximumAverageSubtree(root, maxAverage);
    }

    private record T(int sum, int count, double maxAverage) {}

    private T maximumAverageSubtree(TreeNode root) {
        if (root == null)
            return new T(0, 0, 0.0);

        T left = maximumAverageSubtree(root.left);
        T right = maximumAverageSubtree(root.right);

        final int sum = root.val + left.sum + right.sum;
        final int count = 1 + left.count + right.count;
        final double maxAverage =
            Math.max(sum / (double) count, Math.max(left.maxAverage, right.maxAverage));
        return new T(sum, count, maxAverage);
    }
}
```

Java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    private double ans;

    public double maximumAverageSubtree(TreeNode root) {
        dfs(root);
        return ans;
    }

    private void dfs(TreeNode root) {
        if (root == null) {

```

```
        return new List<T>();
    }

    var l = dfs(root.left);
    var r = dfs(root.right);
    int s = root.val + l.sum + r.sum;
    int n = 1 + l.cnt + r.cnt;
    ans = Math.max(ans, s * 1.0 / n);
    return new List<T>() { s, n };
}
```

Java

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    private double ans;

    public double maximumAverageSubtree(TreeNode root) {
        dfs(root);
        return ans;
    }

    private List<int> dfs(TreeNode root) {
        if (root == null) {
            return new List<int>();
        }
        var l = dfs(root.left);
        var r = dfs(root.right);
        int s = root.val + l[0] + r[0];
        int n = 1 + l[1] + r[1];
        ans = Math.max(ans, s * 1.0 / n);
        return new List<int>() { s, n };
    }
}
```

[*] Trees & BST - Pattern Solution (matched from community answers)

Python

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val # value of the node
        self.left = left # left child
        self.right = right # right child

class Solution:
    def maximumAverageSubtree(self, root):
        # Initialize the answer average to negative infinity
        self.maxAvg = float('-inf')

        # Recursive DFS function that returns a tuple (subtree sum, node count)
        def dfs(node):
            if not node:
                return (0, 0) # return sum 0 and count 0 if node is null
            leftSum, leftCount = dfs(node.left) # traverse left subtree
            rightSum, rightCount = dfs(node.right) # traverse right subtree
            currentSum = node.val + leftSum + rightSum # total sum at current node
            currentCount = 1 + leftCount + rightCount # total node count
            currentAvg = currentSum / currentCount # average value for current subtree
            self.maxAvg = max(self.maxAvg, currentAvg) # update maximum average if necessary
            return (currentSum, currentCount)

        dfs(root)

        return self.maxAvg
```

#1490

MEDIUM

Clone N-ary Tree

LeetCode - SimplyList - WuhCC - LeetCode - Editorial

Hash Table - Tree - DFS - BFS

Like Premium 98

Problem:

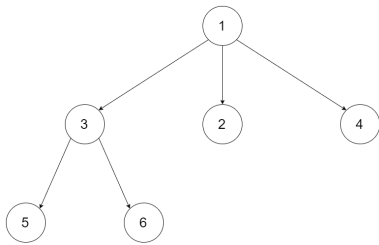
Given a root of an N-ary tree, return a deep copy (clone) of the tree.

Each node in the n-ary tree contains a val (int) and a List(ListNode) of its children.

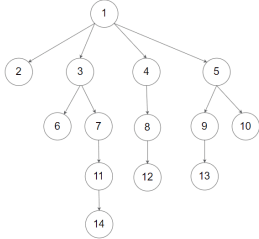
```
class Node {
    public int val;
    public List<Node> children;
}
```

N-ary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

Example 1:



Input: root = [1,null,3,2,4,null,5,6]
Output: [1,null,3,2,4,null,5,6]
Example 2:



Input: root =
[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Output:
[1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [0, 104].

Follow up: Can your solution work for the graph problem?

>> Brute Force [Trees & BST] Interview Prep

Python

```
# Brute Force Approach
class Solution:
    def cloneTree(self, root):
        # Brute Force - collect inorder, then process sorted list
        vals = []
        def collect(nodes):
            if not nodes: return
            collect(nodes.left)
            vals.append(nodes.val)
            collect(nodes.right)
            collect(root)
        return vals[0] if vals else 0
```

Python

```
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []

def cloneTree(root):
    # Base case: if root is None, return None.
    if root is None:
        return None

    # Create a clone for the current node.
    cloned_node = Node(root.val)

    # Recursively clone each child and add it to the cloned_node's children list.
    for child in root.children:
        cloned_child = cloneTree(child)
        cloned_node.children.append(cloned_child)

    # Return the cloned node.
    return cloned_node

# Example usage:
# original_tree = Node(1, [Node(2), Node(3), Node(4), Node(5)])
# cloned_tree = cloneTree(original_tree)
```

Python

```
...
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
...

class Solution:
    def cloneTree(self, root: 'Node') -> 'Node':
        if root is None:
            return None
        children = [self.cloneTree(child) for child in root.children]
        return Node(root.val, children)
```

Python

```
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
...

class Solution:
    def cloneTree(self, root: 'Node') -> 'Node':
        if root is None:
            return None
        children = [self.cloneTree(child) for child in root.children]
        return Node(root.val, children)
```

C++

C++

```
class Solution {
public:
    Node* cloneTree(Node* root) {
        if (root == nullptr)
            return nullptr;
        if (root->val == 0) return root;
        Node* newRoot = new Node(root->val);
        map[root] = newRoot;
        for (Node* child : root->children)
            newRoot->children.push_back(cloneTree(child));
        return newRoot;
    }
private:
    unordered_map<Node*, Node*> map;
};
```

C++

```
// Definition for a Node.
class Node {
public:
    int val;
    vector<Node*> children;
    Node() {}
    Node(int val) {
        this->val = val;
    }
    Node(int val, vector<Node*> children) {
        this->val = val;
        this->children = children;
    }
};

class Solution {
public:
    Node* cloneTree(Node* root) {
        if (!root)
            return root;

        vector<Node*> children;
        for (Node* child : root->children) {
            children.push_back(cloneTree(child));
        }
        return new Node(root->val, children);
    }
};
```

C++

```
// Definition for a Node.
class Node {
public:
    int val;
    vector<Node*> children;
    Node() {}
    Node(int val) {
        this->val = val;
    }
    Node(int val, vector<Node*> children) {
        this->val = val;
        this->children = children;
    }
};

class Solution {
public:
    Node* cloneTree(Node* root) {
        if (!root)
            return root;

        vector<Node*> children;
        for (Node* child : root->children) {
            children.push_back(cloneTree(child));
        }
        return new Node(root->val, children);
    }
};
```

Java

Java

```
class Solution {
    public Node cloneTree(Node root) {
        if (root == null)
            return null;
        if (map.containsKey(root))
            return map.get(root);
        Node newRoot = new Node(root.val);
        map.put(root, newRoot);
        for (Node child : root.children)
            newRoot.children.add(cloneTree(child));
        return newRoot;
    }
    private Map<Node, Node> map = new HashMap<>();
}
```

Java

```
// Definition for a Node.
class Node {
public:
    int val;
    List<Node*> children;
    Node() {}
    Node(int val) {
        this->val = val;
    }
    Node(int val, List<Node*> children) {
        this->val = val;
        this->children = children;
    }
};

class Solution {
    public Node cloneTree(Node root) {
        if (root == null)
            return null;
        ArrayList<Node*> children = new ArrayList<>();
        for (Node child : root.children) {
            children.add(cloneTree(child));
        }
        return new Node(root.val, children);
    }
}
```

Java

Java

```
class Node {
    public int val;
    public List<Node*> children;
}
```

Java

```
// Definition for a Node.
class Node {
public: int val;
public: List<Node> children;

public Node() {
    children = new ArrayList<Node>();
}

public Node(int val) {
    val = val;
    children = new ArrayList<Node>();
}

public Node(int val, ArrayList<Node> children) {
    val = val;
    children = children;
}
};

class Solution {
public: Node cloneTree(Node root) {

    if (root == null) {
        return null;
    }
    ArrayList<Node> children = new ArrayList<>();
    for (Node child : root.children) {
        children.add(cloneTree(child));
    }
    return new Node(root.val, children);
}
}
```

[*] Trees & BST — Pattern Solution (matched from community answers)
Python

```
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []

def cloneTree(root):
    # Base case: if root is None, return None.
    if root is None:
        return None

    # Create a clone for the current node.
    cloned_node = Node(root.val)

    # Recursively clone each child and add it to the cloned_node's children list.
    for child in root.children:
        cloned_child = cloneTree(child)
        cloned_node.children.append(cloned_child)

    # Return the cloned node.
    return cloned_node

# Example usage:
# original_tree = Node(1, [Node(3), Node(1), Node(2), Node(4)])
# cloned_tree = cloneTree(original_tree)
```

#1522

MEDIUM

Diameter of N-Ary Tree

LeetCode — [SimpleLink](#) — [WikiCC](#) — [LeetCode](#) — [Editorial](#)

Tree — DFS

List: Premium 98

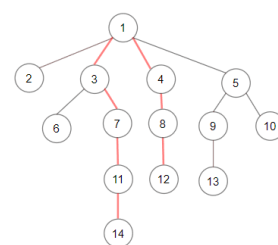
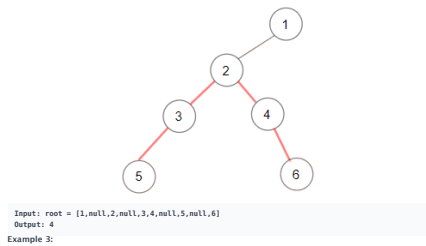
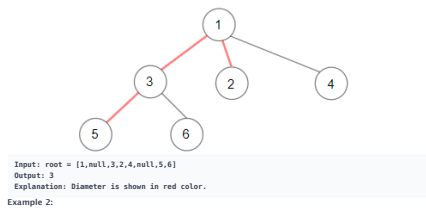
Problem:

Given a root of an N-ary tree, you need to compute the length of the diameter of the tree.

The diameter of an N-ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.

(N-ary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value.)

Example 1:



- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [1, 1004].

Python

Python

```
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []

class Solution:
    def diameter(self, root: 'Node') -> int:
        # Global variable to keep track of the maximum diameter found so far.
        self.max_diameter = 0

        def dfs(node):
            if not node:
                return 0 # Base case: if the node is None, return depth 0

            # Variables to hold the top two maximum depths from the children.
            max_depth1, max_depth2 = 0, 0

            # Traverse over each child to compute their maximum depths.
            for child in node.children:
                child_depth = dfs(child) # Recurse on the child
                # Update the top two maximum depths.
                if child_depth > max_depth1:
                    max_depth2 = max_depth1
                    max_depth1 = child_depth
                elif child_depth > max_depth2:
                    max_depth2 = child_depth

            # Update the maximum diameter. The candidate is the sum of the two deepest paths.
            self.max_diameter = max(self.max_diameter, max_depth1 + max_depth2)

            # Return the maximum depth from this node.
            return max_depth1 + 1

        dfs(root)
        return self.max_diameter
```

Python

```
class Solution:
    def diameter(self, root: 'Node') -> int:
        ans = 0

        def maxDepth(root: 'Node') -> int:
            """Returns the maximum depth of the subtree rooted at 'root'."""
            nonlocal ans
            maxDepth1 = 0
            maxDepth2 = 0
            for child in root.children:
                maxSubDepth = maxDepth(child)
                if maxSubDepth > maxDepth1:
                    maxSubDepth2 = maxSubDepth1
                    maxSubDepth1 = maxSubDepth
                elif maxSubDepth > maxSubDepth2:
                    maxSubDepth2 = maxSubDepth
            ans = max(ans, maxSubDepth1 + maxSubDepth2)
            return 1 + maxSubDepth1

        maxDepth(root)
        return ans
```

Python

```
...
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
...

class Solution:
    def diameter(self, root: 'Node') -> int:
        ...
        type root: 'Node'
        /-type: int
        ...

        def dfs(root):
            if root is None:
                return 0
            maxLeft, ans = 0, 0
            maxRight, ans = 0, 0
            for child in root.children:
                t = dfs(child)
                if t > maxLeft:
                    maxLeft, ans = t, t
                elif t > maxRight:
                    maxRight, ans = t, ans
            return 1 + max(maxLeft, maxRight)

        ans = 0
        dfs(root)
        return ans
```

Python

```

...
// Definition for a Node.
class Node {
    def __init__(self, val:Node, children:Node):
        self.val = val
        self.children = children if children is not None else []
...

class Solution:
    def diameter(self, root: 'Node') -> int:
        """
        :type root: 'Node'
        :rtype: int
        """
        def dfs(node):
            if root is None:
                return 0
            maxLeft, ans
            m1 = m2 = 0
            for child in root.children:
                t = dfs(child)
                if t > m1:
                    m1, m2 = m1, t
            elif 0 <= m2:
                m2 = t
            ans = max(ans, m1 + m2)
            return 1 + m1
        ans = 0
        dfs(root)
        return ans

```

C++

C++

```

// Definition for a Node.
class Node {
public:
    int val;
    vector<Node*> children;

    Node() {}

    Node(int _val) {
        val = _val;
    }

    Node(int _val, vector<Node*> _children) {
        val = _val;
        children = _children;
    }
};

class Solution {
public:
    int ans;

    int diameter(Node* root) {
        ans = 0;
        dfs(root);
        return ans;
    }

    int dfs(Node* root) {
        if (!root) return 0;
        int m1 = 0, m2 = 0;
        for (Node* child : root->children) {
            int t = dfs(child);
            if (t > m1) {
                m2 = m1;
                m1 = t;
            } else if (t > m2) {
                m2 = t;
            }
        }
        ans = max(ans, m1 + m2);
        return 1 + m1;
    }
};

```

C++

```

// Definition for a Node.
class Node {
public:
    int val;
    vector<Node*> children;

    Node() {}

    Node(int _val) {
        val = _val;
    }

    Node(int _val, vector<Node*> _children) {
        val = _val;
        children = _children;
    }
};

class Solution {
public:
    int ans;

    int diameter(Node* root) {
        ans = 0;
        dfs(root);
        return ans;
    }

    int dfs(Node* root) {
        if (!root) return 0;
        int m1 = 0, m2 = 0;
        for (Node* child : root->children) {
            int t = dfs(child);
            if (t > m1) {
                m2 = m1;
                m1 = t;
            } else if (t > m2) {
                m2 = t;
            }
        }
        ans = max(ans, m1 + m2);
        return 1 + m1;
    }
};

```

Java

Java

```

class Solution {
    public int diameter(Node root) {
        maxDepth(root);
        return ans;
    }

    private int ans = 0;

    // Returns the maximum depth of the subtree rooted at 'root'.
    private int maxDepth(Node root) {
        int maxSubDepth1 = 0;
        int maxSubDepth2 = 0;
        for (Node child : root.children) {
            final int maxSubDepth = maxDepth(child);
            if (maxSubDepth > maxSubDepth1) {
                maxSubDepth1 = maxSubDepth;
            } else if (maxSubDepth > maxSubDepth2) {
                maxSubDepth2 = maxSubDepth;
            }
        }
        ans = Math.max(ans, maxSubDepth1 + maxSubDepth2);
        return 1 + maxSubDepth1;
    }
}

// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {
        children = new ArrayList<Node>();
    }

    public Node(int _val) {
        val = _val;
        children = new ArrayList<Node>();
    }

    public Node(int _val, ArrayList<Node> _children) {
        val = _val;
        children = _children;
    }
};

class Solution {
    public int ans;
}

```

Java

```

public int diameter(Node root) {
    ans = 0;
    dfs(root);
    return ans;
}

private int dfs(Node root) {
    if (root == null) {
        return 0;
    }
    int m1 = 0, m2 = 0;
    for (Node child : root.children) {
        int t = dfs(child);
        if (t > m1) {
            m2 = m1;
            m1 = t;
        } else if (t > m2) {
            m2 = t;
        }
    }
    ans = Math.max(ans, m1 + m2);
    return 1 + m1;
}
}

// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {
        children = new ArrayList<Node>();
    }

    public Node(int _val) {
        val = _val;
        children = new ArrayList<Node>();
    }

    public Node(int _val, ArrayList<Node> _children) {
        val = _val;
        children = _children;
    }
};

class Solution {
    public int ans;
}

```

Java

```

public int diameter(Node root) {
    ans = 0;
    dfs(root);
    return ans;
}

private int dfs(Node root) {
    if (root == null) {
        return 0;
    }
    int m1 = 0, m2 = 0;
    for (Node child : root.children) {
        int t = dfs(child);
        if (t > m1) {
            m2 = m1;
            m1 = t;
        } else if (t > m2) {
            m2 = t;
        }
    }
    ans = Math.max(ans, m1 + m2);
    return 1 + m1;
}
}

// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {
        children = new ArrayList<Node>();
    }

    public Node(int _val) {
        val = _val;
        children = new ArrayList<Node>();
    }

    public Node(int _val, ArrayList<Node> _children) {
        val = _val;
        children = _children;
    }
};

class Solution {
    public int ans;
}

```

Python

Definition for a Node.

class Node:

def __init__(self, val:Node, children:Node):

self.val = val

self.children = children if children is not None else []

class Solution:

def diameter(self, root: 'Node') -> int:

Returns the maximum depth of the subtree rooted at 'root'.

self.max_diameter = 0

def dfs(self, node):

if not node:

return 0

Base case: If the node is None, return depth 0

Variables to hold the two maximum depths from the children.

max_depth1, max_depth2 = 0, 0

Traverse over each child to compute their maximum depths.

for child in node.children:

child_depth = dfs(child)

Recurse on the child

Update the two largest depths

if child_depth > max_depth1:

max_depth1 = max_depth2

max_depth2 = child_depth

```

        elif child_depth > max_depth:
            max_depth = child_depth

        # Update the maximum diameter. The candidate is the sum of the two deepest paths.
        self.max_diameter = max(self.max_diameter, max_depth1 + max_depth2)

        # Return the maximum depth from this node.
        return max_depth1 + 1

    def dfs(self, root):
        return self.dfs_max_diameter

```

#1650 MEDIUM LeetCode · Simple to Hard · MultiLang · LastCode · Editorial

Hash Table · Tree · Two Pointers
 Lists Premium 98

Problem:

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).

Each node will have a reference to its parent node. The definition for Node is below:

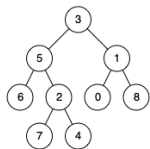
```

class Node {
public:
    Node left;
    Node right;
    Node parent;
};

```

According to the definition of LCA on Wikipedia: "The lowest common ancestor of two nodes p and q in a tree T is the lowest node that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1:



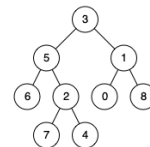
Trees & BST · LeastMastery · By Pattern

— 3367 —

LeastMastery

Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
 Output: 3
 Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
 Output: 5
 Explanation: The LCA of nodes 5 and 4 is 5 since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2
 Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- 109 <= Node.val <= 109
- All Node.val are unique.
- p != q
- p and q exist in the tree.

>> Brute Force [Trees & BST] Interview Prep

```

Python
# Brute Force Approach
class Solution:
    def lowestCommonAncestor(self, p, q):
        # Brute Force - collect inorder, then process sorted list
        vals = []
        def collect(node):
            if not node: return
            collect(node.left)
            vals.append(node.val)
            collect(node.right)
            collect(p)
            return vals[i] if vals else 0

```

Trees & BST · LeastMastery · By Pattern

— 3368 —

LeastMastery

Python
 Python

```

# Definition for a Node.
class Node:
    def __init__(self, val=0, left=None, right=None, parent=None):
        self.val = val
        self.left = left
        self.right = right
        self.parent = parent

def lowestCommonAncestor(p, q):
    # Generate the depth for each node
    def getDepth(node):
        depth = 0
        while node:
            depth += 1
            node = node.parent
        return depth

    depthP = getDepth(p)
    depthQ = getDepth(q)

    # Bring both nodes to the same depth level
    while depthP > depthQ:
        p = p.parent
        depthP -= 1
    while depthQ > depthP:
        q = q.parent
        depthQ -= 1

    # Traverse up until we find the common ancestor
    while p != q:
        p = p.parent
        q = q.parent

    return p

```

```

Python
class Solution:
    # Same as 150. Intersection of Two Linked Lists
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        a = p
        b = q

        while a != b:
            a = a.parent if a else q
            b = b.parent if b else p

        return a

```

Python

Trees & BST · LeastMastery · By Pattern

— 3369 —

LeastMastery

```

...
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
...

class Solution:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        vis = set()
        node = p
        while node:
            vis.add(node)
            node = node.parent
        node = q
        while node not in vis:
            node = node.parent
        return node

```

```

Python
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
...

class Solution:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        a, b = p, q
        while a != b:
            a = a.parent if a.parent else q
            b = b.parent if b.parent else p
        return a

#####
# ref: https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree-iii/solutions/3376c
def lowestCommonAncestor(self, a: 'Node', b: 'Node') -> 'Node':
    ancestors = set()

```

Trees & BST · LeastMastery · By Pattern

— 3370 —

LeastMastery

```

while a is not None:
    ancestors.add(a)
    a = a.parent

while b is not None:
    if b in ancestors:
        return b
    b = b.parent

# or another one
def lowestCommonAncestor(self, a: 'Node', b: 'Node') -> 'Node':
    pointerA, pointerB = a, b

    while pointerA != pointerB:
        pointerA = pointerA.parent if pointerA else b
        pointerB = pointerB.parent if pointerB else a

    return pointerA

```

Java

```

Java
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
};

class Solution {
    public Node lowestCommonAncestor(Node p, Node q) {
        Set<Node> vis = new HashSet<>();
        for (Node node = p; node != null; node = node.parent) {
            vis.add(node);
        }
        for (Node node = q; node != null; node = node.parent) {
            if (vis.contains(node)) {
                return node;
            }
        }
    }
}

```

Java

```

class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
}

```

Java

Trees & BST · LeastMastery · By Pattern

— 3371 —

LeastMastery

```

// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
};

class Solution {
    public Node lowestCommonAncestor(Node p, Node q) {
        Set<Node> vis = new HashSet<>();
        Node temp = p;
        while (temp != null) {
            vis.add(temp);
            temp = temp.parent;
        }
        temp = q;
        while (temp != null) {
            if (vis.contains(temp)) {
                break;
            }
            temp = temp.parent;
        }
        return temp;
    }
}

#####
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node parent;
};

class Solution {
    public Node lowestCommonAncestor(Node p, Node q) {
        Node a = p, b = q;
        while (a != b) {
            a = a.parent == null ? q : a.parent;
            b = b.parent == null ? p : b.parent;
        }
        return a;
    }
}

```

TypeScript

TypeScript

Trees & BST · LeastMastery · By Pattern

— 3372 —

LeastMastery

```

//
// Definition for a binary tree node.
// class Node {
//     val: number
//     left: Node | null
//     right: Node | null
//     parent: Node | null
//     constructor(val: number, left?: Node | null, right?: Node | null, parent?: Node | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//         this.parent = (parent===undefined ? null : parent)
//     }
// }

function lowestCommonAncestor(p: Node | null, q: Node | null): Node | null {
    const vis = Set<Node>().new Set({});
    for (let node = p; node = node.parent) {
        vis.add(node);
    }
    for (let node = q; node = node.parent) {
        if (vis.has(node)) {
            return node;
        }
    }
}

```

TimeComplex

```

//
// Definition for a binary tree node.
// class Node {
//     val: number
//     left: Node | null
//     right: Node | null
//     parent: Node | null
//     constructor(val: number, left?: Node | null, right?: Node | null, parent?: Node | null) {
//         this.val = (val===undefined ? 0 : val)
//         this.left = (left===undefined ? null : left)
//         this.right = (right===undefined ? null : right)
//         this.parent = (parent===undefined ? null : parent)
//     }
// }

function lowestCommonAncestor(p: Node | null, q: Node | null): Node | null {
    const vis = Set<Node>().new Set({});
    for (let node = p; node = node.parent) {
        vis.add(node);
    }
    for (let node = q; node = node.parent) {
        if (vis.has(node)) {
            return node;
        }
    }
}

```

```

Go
//
// Definition for a Node.
// type Node struct {
//     val int
//     left *Node
//     right *Node
//     parent *Node
// }
//

func lowestCommonAncestor(p *Node, q *Node) *Node {
    vis := map[*Node]bool{}
    for node := p; node != nil; node = node.Parent {
        vis[node] = true
    }
    for node := q; node = node.Parent {
        if vis[node] {
            return node
        }
    }
}

```

[1] Trees & BST — Pattern Solution (matched from community answers)

Python

```

// Definition for a Node.
// class Node:
//     def __init__(self, val=0, left=None, right=None, parent=None):
//         self.val = val
//         self.left = left
//         self.right = right
//         self.parent = parent

def lowestCommonAncestor(p, q):
    # Compute the depth for node p
    def getDepth(node):
        depth = 0
        while node:
            depth += 1
            node = node.parent
        return depth

    depthP = getDepth(p)
    depthQ = getDepth(q)

    # Bring both nodes to the same depth level
    while depthP > depthQ:
        p = p.parent
        depthP -= 1
    while depthQ > depthP:

```

```

q = q.parent
depthQ += 1

# Traverse up until we find the common ancestor
while p != q:
    p = p.parent
    q = q.parent

return p

```

#124

HARD

Binary Tree Maximum Path Sum

LeetCode - SimplifyLeetCode - WsLCC - LeetCode - Editorial

Dynamic Programming - Tree - DFS

Lists Grid 169

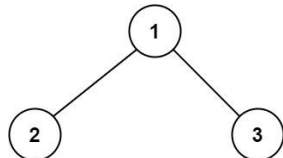
Problem:

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

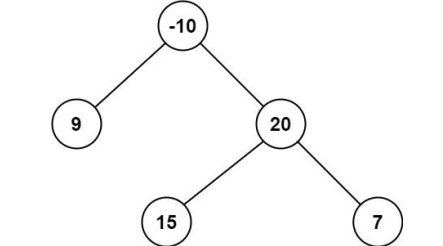
Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:



Input: root = [1,2,3]
Output: 6
Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.

Example 2:



Input: root = [-10,9,20,null,null,15,7]
Output: 42
Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.
Constraints:
• The number of nodes in the tree is in the range [1, 3 * 10⁴].
• -1000 <= Node.val <= 1000

Python

Python

```

// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int x) { val = x; }
// }

class Solution {
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        # Initialize global maximum path sum with a very small number
        self.global_max = float('-inf')

        def dfs(node: Optional[TreeNode]) -> int:
            if not node:
                return 0

            # Recursively calculate the max gain from left and right children, discard negative gains
            left_gain = max(dfs(node.left), 0)
            right_gain = max(dfs(node.right), 0)

            # Current maximum including both left and right gains
            current_max = node.val + left_gain + right_gain

            # Update global maximum path sum if current_max is higher
            self.global_max = max(self.global_max, current_max)

        # Return the node's value plus the max of one child gain (only one side can be chosen for parent path)
        return node.val + max(left_gain, right_gain)

    def dfs(self):
        return self.global_max

```

Python

```

class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        ans = -math.inf

        def maxPathSumFromRoot(root: Optional[TreeNode]) -> int:
            # Returns the maximum path sum starting from the current root, where
            # root.val is always included.
            ...

            nonlocal ans
            if not root:
                return 0

            l = max(0, maxPathSumFromRoot(root.left))
            r = max(0, maxPathSumFromRoot(root.right))
            ans = max(ans, root.val + l + r)
            return root.val + max(l, r)

        maxPathSumFromRoot(root)
        return ans

```

```

// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode() {}
//     TreeNode(int x) { val = x; }
// }

class Solution {
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        left = max(0, dfs(root.left))
        right = max(0, dfs(root.right))
        nonlocal ans
        ans = max(ans, root.val + left + right)
        return root.val + max(left, right)

        ans = -inf
        dfs(root)
        return ans

    def dfs(self, root: Optional[TreeNode]) -> int:
        if root is None:
            return 0
        left = max(0, dfs(root.left))
        right = max(0, dfs(root.right))
        nonlocal ans
        ans = max(ans, root.val + left + right)
        return root.val + max(left, right)

        ans = -inf
        dfs(root)
        return ans

```

Java

Java

```

class Solution {
    public int maxPathSum(TreeNode root) {
        maxPathSumFrom(root);
        return ans;
    }

    private int ans = Integer.MIN_VALUE;

    // Returns the maximum path sum starting from the current root, where
    // root.val is always included
    private int maxPathSumFrom(TreeNode root) {
        if (root == null)
            return 0;

        final int l = Math.max(maxPathSumFrom(root.left), 0);
        final int r = Math.max(maxPathSumFrom(root.right), 0);
        ans = Math.max(ans, root.val + l + r);
        return root.val + Math.max(l, r);
    }
}

Java

/**
 * Definition for a binary tree node.
 */
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    private int ans = -1001;

    public int maxPathSum(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int left = Math.max(0, dfs(root.left));
        int right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, root.val + left + right);
        return root.val + Math.max(left, right);
    }
}

Java

```

```

/**
 * Definition for a binary tree node.
 */
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

public class Solution {
    private int ans = -1001;

    public int maxPathSum(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int left = Math.max(0, dfs(root.left));
        int right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, left + right + root.val);
        return root.val + Math.max(left, right);
    }
}

Java

```

```

/**
 * Definition for a binary tree node.
 */
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

class Solution {
    private int ans = -1001;

    public int maxPathSum(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int left = Math.max(0, dfs(root.left));
        int right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, root.val + left + right);
        return root.val + Math.max(left, right);
    }
}

Java

```

```

/**
 * Definition for a binary tree node.
 */
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}

public class Solution {
    private int ans = -1001;

    public int maxPathSum(TreeNode root) {
        dfs(root);
        return ans;
    }

    private int dfs(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int left = Math.max(0, dfs(root.left));
        int right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, left + right + root.val);
        return root.val + Math.max(left, right);
    }
}

JavaScript
JavaScript

```

```

/**
 * Definition for a binary tree node.
 */
function TreeNode(val, left, right) {
    this.val = (val===undefined ? 0 : val)
    this.left = (left===undefined ? null : left)
    this.right = (right===undefined ? null : right)
}

/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxPathSum = function (root) {
    let ans = -1001;
    const dfs = root => {
        if (!root) {
            return 0;
        }
        const left = Math.max(0, dfs(root.left));
        const right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, left + right + root.val);
        return Math.max(left, right) + root.val;
    };
    dfs(root);
    return ans;
};

JavaScript

/**
 * Definition for a binary tree node.
 */
function TreeNode(val, left, right) {
    this.val = (val===undefined ? 0 : val)
    this.left = (left===undefined ? null : left)
    this.right = (right===undefined ? null : right)
}

/**
 * @param {TreeNode} root
 * @return {number}
 */
var maxPathSum = function (root) {
    let ans = -1001;
    const dfs = root => {
        if (!root) {
            return 0;
        }
        const left = Math.max(0, dfs(root.left));
        const right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, left + right + root.val);
        return Math.max(left, right) + root.val;
    };
    dfs(root);
    return ans;
};

TypeScript

```

```

TypeScript

/**
 * Definition for a binary tree node.
 */
class TreeNode {
    val: number;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
        this.val = (val===undefined ? 0 : val)
        this.left = (left===undefined ? null : left)
        this.right = (right===undefined ? null : right)
    }
}

function maxPathSum(root: TreeNode | null): number {
    let ans = -1001;
    const dfs = (root: TreeNode | null): number => {
        if (!root) {
            return 0;
        }
        const left = Math.max(0, dfs(root.left));
        const right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, left + right + root.val);
        return Math.max(left, right) + root.val;
    };
    dfs(root);
    return ans;
}

TypeScript

/**
 * Definition for a binary tree node.
 */
class TreeNode {
    val: number;
    left: TreeNode | null;
    right: TreeNode | null;
    constructor(val?: number, left?: TreeNode | null, right?: TreeNode | null) {
        this.val = (val===undefined ? 0 : val)
        this.left = (left===undefined ? null : left)
        this.right = (right===undefined ? null : right)
    }
}

function maxPathSum(root: TreeNode | null): number {
    let ans = -1001;
    const dfs = (root: TreeNode | null): number => {
        if (!root) {
            return 0;
        }
        const left = Math.max(0, dfs(root.left));
        const right = Math.max(0, dfs(root.right));
        ans = Math.max(ans, left + right + root.val);
        return Math.max(left, right) + root.val;
    };
    dfs(root);
    return ans;
}

```



```

def(root):
    return ans;
}

Rust
// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32;
//     pub left: Option<Rc<RefCell<TreeNode>>>;
//     pub right: Option<Rc<RefCell<TreeNode>>>;
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }

use std::cell::RefCell;
use std::rc::Rc;
impl Solution {
    fn dfs(root: Option<Rc<RefCell<TreeNode>>>, res: &mut i32) -> i32 {
        if root.is_none() {
            return 0;
        }

        let node = root.as_ref().unwrap().borrow();
        let left = (0..max(Self::dfs(&node.left, res))).max();
        let right = (0..max(Self::dfs(&node.right, res))).max();
        res = (node.val + left + right).max(res);
        node.val + left.max(right)
    }

    pub fn max_path_sum(root: Option<Rc<RefCell<TreeNode>>>) -> i32 {
        let mut res = -1000;
        Self::dfs(&root, &mut res);
        res
    }
}
Rust

```

Trees & BST - LeetCode - By Pattern

— 3385 —

LeetCode

```

// Definition for a binary tree node.
// #[derive(Debug, PartialEq, Eq)]
// pub struct TreeNode {
//     pub val: i32;
//     pub left: Option<Rc<RefCell<TreeNode>>>;
//     pub right: Option<Rc<RefCell<TreeNode>>>;
// }
// impl TreeNode {
//     #[inline]
//     pub fn new(val: i32) -> Self {
//         TreeNode {
//             val,
//             left: None,
//             right: None,
//         }
//     }
// }

use std::rc::Rc;
use std::cell::RefCell;
impl Solution {
    fn dfs(root: Option<Rc<RefCell<TreeNode>>>, res: &mut i32) -> i32 {
        if root.is_none() {
            return 0;
        }

        let node = root.as_ref().unwrap().borrow();
        let left = (0..max(Self::dfs(&node.left, res))).max();
        let right = (0..max(Self::dfs(&node.right, res))).max();
        res = (node.val + left + right).max(res);
        node.val + left.max(right)
    }

    pub fn max_path_sum(root: Option<Rc<RefCell<TreeNode>>>) -> i32 {
        let mut res = -1000;
        Self::dfs(&root, &mut res);
        res
    }
}
Python

```

[1] Trees & BST - Pattern Solution (matched from community answers)

Python

Trees & BST - LeetCode - By Pattern

— 3386 —

LeetCode

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxPathSum(self, root: TreeNode) -> int:
        # Initialize global maximum path sum with a very small number
        self.global_max = float('-inf')

        def dfs(node: TreeNode) -> int:
            if not node:
                return 0

            # Recursively calculate max gain from left and right children, discard negative gains
            left_gain = max(dfs(node.left), 0)
            right_gain = max(dfs(node.right), 0)

            # Current maximum including both left and right gains
            current_max = node.val + left_gain + right_gain

            # Update global maximum
            self.global_max = max(self.global_max, current_max)

        # Return the node's value plus the max of new child gain (only one side can be chosen for parent path)
        return node.val + max(left_gain, right_gain)

    def dfs(self, root: TreeNode):
        return self.global_max

```

297 HARD Serialize and Deserialize Binary Tree

LeetCode - Implementation - Python3 - LeetCode - Editorial

String - Tree - DFS - BFS - Design

Lists Grind 189

Problem:

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

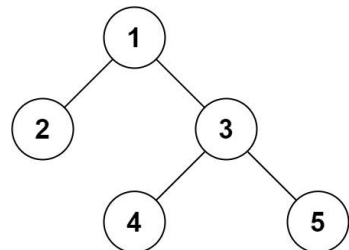
Clarification: The input/output format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:

Trees & BST - LeetCode - By Pattern

— 3387 —

LeetCode



Input: root = [1,2,3,null,null,4,5]
Output: ["1,2,3,null,null,4,5"]

Example 2:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- 1000 <= Node.val <= 1000

Python

Python

Trees & BST - LeetCode - By Pattern

— 3388 —

LeetCode

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Codec:
    # Encodes a tree to a single string using BFS.
    def serialize(self, root):
        if not root:
            return ""
        queue = [root]
        while queue:
            node = queue.pop(0)
            if node:
                result.append(str(node.val))
                queue.append(node.left)
                queue.append(node.right)
            else:
                result.append("null")
        return ",".join(result)

    # Decodes the encoded data to reconstruct the tree using BFS.
    def deserialize(self, data):
        if not data:
            return None
        nodes = data.split(",")
        root = TreeNode(int(nodes[0]))
        queue = [root]
        i = 1
        while queue:
            current = queue.pop(0)
            if i < len(nodes) and nodes[i] != "null":
                current.left = TreeNode(int(nodes[i]))
                queue.append(current.left)
            i += 1
            if i < len(nodes) and nodes[i] != "null":
                current.right = TreeNode(int(nodes[i]))
                queue.append(current.right)
            i += 1
        return root

```

Python

Trees & BST - LeetCode - By Pattern

— 3389 —

LeetCode

```

class Codec:
    def serialize(self, root: 'TreeNode') -> str:
        """Encodes a tree to a single string.
        """
        if not root:
            return ""
        s = ""
        q = collections.deque([root])
        while q:
            node = q.popleft()
            if node:
                s += str(node.val) + ","
                q.append(node.left)
                q.append(node.right)
            else:
                s += "n"
        return s

    def deserialize(self, data: str) -> 'TreeNode':
        """Decodes your encoded data to tree.
        """
        if not data:
            return None

        vals = data.split(',')
        root = TreeNode(int(vals[0]))
        q = collections.deque([root])
        for i in range(1, len(vals), 2):
            node = q.popleft()
            if vals[i] != "n":
                node.left = TreeNode(int(vals[i]))
                q.append(node.left)
            if vals[i+1] != "n":
                node.right = TreeNode(int(vals[i+1]))
                q.append(node.right)
        return root

```

Python

Trees & BST - LeetCode - By Pattern

— 3390 —

LeetCode

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str

        If root is None:
            return ""
        q = deque([root])
        ans = []
        while q:
            node = q.popleft()
            if node:
                ans.append(str(node.val))
                q.append(node.left)
                q.append(node.right)
            else:
                ans.append("#")
        return "".join(ans)

    def deserialize(self, data):
        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode

        If not data:
            return None
        vals = data.split("#")
        root = TreeNode(int(vals[0]))
        q = deque([root])
        i = 1
        while q:
            node = q.popleft()
            if vals[i] != "#":
                node.left = TreeNode(int(vals[i]))
                q.append(node.left)
            i += 1
            if vals[i] != "#":
                node.right = TreeNode(int(vals[i]))
                q.append(node.right)
            i += 1
        return root

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# ans = codec.deserialize(codec.serialize(root))

```

```

Python
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str

        If root is None:
            return ""
        res = []
        def preorder(root):
            if root is None:
                res.append("#")
            else:
                res.append(str(root.val) + ",")
                preorder(root.left)
                preorder(root.right)
        preorder(root)
        return "".join(res)

    def deserialize(self, data):
        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode

        If not data:
            return None
        vals = data.split(",")

        def inner():
            first = vals.pop(0)
            if first == "#":
                return None
            return TreeNode(int(first), inner(), inner())
        # some using a constructor __init__(val, left, right)
        return inner()

```

```

# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# ans = deser.deserialize(ser.serialize(root))

#####
from collections import deque
# bfs, each level based
class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str

        ret = []
        queue = deque([root])
        while queue:
            top = queue.popleft()
            if not top:
                ret.append("None")
                continue
            else:
                ret.append(str(top.val))
                queue.append(top.left)
                queue.append(top.right)
        return "".join(ret)

Java
Java

```

```

public class Codec {
    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        preorder(root, sb);
        return sb.toString();
    }

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        final String[] vals = data.split("#");
        Queue<String> q = new ArrayDeque<>(List.of(vals));
        return preorder(q);
    }

    private void preorder(TreeNode root, StringBuilder sb) {
        if (root == null) {
            sb.append("#");
            return;
        }
        sb.append(root.val).append("#");
        preorder(root.left, sb);
        preorder(root.right, sb);
    }

    private TreeNode preorder(Queue<String> q) {
        final String s = q.poll();
        if (s.equals("#"))
            return null;
        TreeNode root = new TreeNode(Integer.parseInt(s));
        root.left = preorder(q);
        root.right = preorder(q);
        return root;
    }
}

Java

```

```

/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
public class Codec {
    private static final String NULL = "#";
    private static final String SEP = ",";

    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        if (root == null) {
            return "";
        }
        StringBuilder sb = new StringBuilder();
        preorder(root, sb);
        return sb.toString();
    }

    private void preorder(TreeNode root, StringBuilder sb) {
        if (root == null) {
            sb.append(NULL + SEP);
            return;
        }
        sb.append(root.val + SEP);
        preorder(root.left, sb);
        preorder(root.right, sb);
    }

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        if (data == null || "".equals(data)) {
            return null;
        }
        List<String> vals = new LinkedList<>();
        for (String s : data.split(SEP)) {
            vals.add(s);
        }
        return deserialize(vals);
    }

    private TreeNode deserialize(List<String> vals) {
        String first = vals.remove(0);
        if (NULL.equals(first)) {
            return null;
        }
    }
}

```

```

        TreeNode root = new TreeNode(Integer.parseInt(first));
        root.left = deserialize(vals);
        root.right = deserialize(vals);
        return root;
    }
}

// Your Codec object will be instantiated and called as such:
// Codec ser = new Codec();
// Codec deser = new Codec();
// TreeNode ans = deser.deserialize(ser.serialize(root));

Java
/*
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
public class Codec {
    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        if (root == null) {
            return null;
        }
        List<String> ans = new List<>();
        Queue<TreeNode> q = new Queue<>();
        q.add(root);
        while (q.Count > 0) {
            TreeNode node = q.Dequeue();
            if (node != null) {
                ans.Add(node.val.ToString());
                q.Enqueue(node.left);
                q.Enqueue(node.right);
            }
        }
    }
}

```

```

    } else {
      ans.Add("F");
    }
  }
  return string.Join(",", ans);
}

// Decodes your encoded data to tree.
public TTreeNode Deserialize(string data) {
  if (data == null) {
    return null;
  }
  string[] vals = data.Split(',');
  int i = 0;
  TTreeNode root = new TTreeNode(int.Parse(vals[i]));
  Queue<TTreeNode> q = new Queue<TTreeNode>();
  q.Enqueue(root);
  while (q.Count > 0) {
    TTreeNode node = q.Dequeue();
    if (vals[i+1] != "N") {
      node.Left = new TTreeNode(int.Parse(vals[i+1]));
      q.Enqueue(node.Left);
    }
    i++;
    if (vals[i] != "N") {
      node.Right = new TTreeNode(int.Parse(vals[i+1]));
      q.Enqueue(node.Right);
    }
    i++;
  }
  return root;
}
}

// Your Codec object will be instantiated and called as such:
// Codec codec = new Codec();
// codec.Deserialize(codec.Serialize(root));

```

JavaScript
JavaScript

```

//
// Definition for a binary tree node.
//
function TTreeNode(val) {
  this.val = val;
  this.left = this.right = null;
}

//
// Encodes a tree to a single string.
//
var serialize = function(root) {
  return rserialize(root, "");
};

//
// Decodes your encoded data to tree.
//
var deserialize = function(data) {
  const dataArray = data.split(',');
  return ddeserialize(dataArray);
};

const rserialize = (root, str) => {
  if (root === null) {
    str += 'N';
  } else {
    str += root.val + "," + ",";
    str = rserialize(root.left, str);
    str = rserialize(root.right, str);
  }
  return str;
};

const ddeserialize = dataList => {
  if (dataList[0] === "N") {
    return null;
  }

  const root = new TTreeNode(parseInt(dataList[0]));
  root.left = ddeserialize(dataList);
  root.right = ddeserialize(dataList);
};

```

```

    return root;
  }
}

//
// Your functions will be called as such:
// deserialize(serialize(root));
//

```

TypeScript
TypeScript

```

//
// Definition for a binary tree node.
//
class TTreeNode {
  val: number;
  left: TTreeNode | null;
  right: TTreeNode | null;
  constructor(val: number, left: TTreeNode | null, right: TTreeNode | null) {
    this.val = val;
    this.left = left;
    this.right = right;
  }
}

//
// Encodes a tree to a single string.
//
function serialize(root: TTreeNode | null): string {
  if (root === null) {
    return "N";
  }
  const { val, left, right } = root;
  return `${val},${serialize(left)},${serialize(right)};`;
}

```

```

//
// Decodes your encoded data to tree.
//
function deserialize(data: string): TTreeNode | null {
  const s = data.length;
  if (s === 0) {
    return null;
  }
  const vals = data.split(',').reverse();
  const rnode = () => {
    const val = vals.pop();
    if (val === null || val === "N") {
      return null;
    }
    return new TTreeNode(Number(val), rnode(), rnode());
  };
  return rnode();
}

//
// Your functions will be called as such:
// deserialize(serialize(root));
//

```

Go
Go

```

//
// Definition for a binary tree node.
//
type TTreeNode struct {
  Val int
  Left *TTreeNode
  Right *TTreeNode
}

//
// Your functions will be called as such:
// deserialize(serialize(root));
//

```

```

q = q[1];
if node is nil {
  ans = append(ans, strconv.Itoa(node.Val))
  q = append(q, node.Left)
  q = append(q, node.Right)
} else {
  ans = append(ans, "F")
}
return string.Join(ans, ",")
}

// Serializes your encoded data to tree.
func (this *Codec) Deserialize(data string) *TTreeNode {
  if data == "" {
    return nil
  }
  vals := strings.Split(data, ",")
  v, _ := strconv.Atoi(vals[0])
  i := 1
  root := &TTreeNode{Val: v}
  q := []*TTreeNode{root}
  for len(q) > 0 {
    node := q[0]
    q = q[1:]
    if v, err := strconv.Atoi(vals[i]); err == nil {
      node.Left = &TTreeNode{Val: v}
      q = append(q, node.Left)
    }
    i++
    if v, err := strconv.Atoi(vals[i]); err == nil {
      node.Right = &TTreeNode{Val: v}
      q = append(q, node.Right)
    }
    i++
  }
  return root
}

//
// Your Codec object will be instantiated and called as such:
// codec := Constructor();
// data := ser.Serialize(root);
// ans := deser.Deserialize(data);
//

```

Rust
Rust

```

// Definition for a binary tree node.
//
#[derive(Debug, PartialEq, Eq)]
pub struct TTreeNode {
  pub val: i32,
  pub left: Option<Rc<RefCell<TTreeNode>>>,
  pub right: Option<Rc<RefCell<TTreeNode>>>,
}

//
// Impl TTreeNode {
//   #[inline]
//   pub fn new(val: i32) -> Self {
//     TTreeNode {
//       val,
//       left: None,
//       right: None,
//     }
//   }
// }

use std::rc::Rc;
use std::cell::RefCell;
struct Codec {
  //
  // Self means the method takes an immutable reference.
  // If you need a mutable reference, change it to 'Self' instead.
}

impl Codec {
  fn new() -> Self {
    Codec {}
  }

  fn serialize(&self, root: Option<Rc<RefCell<TTreeNode>>>) -> String {
    if root.is_none() {
      return String::from("N");
    }
    let mut node = root.as_ref().unwrap().borrow_mut();
    let left = node.left.take();
    let right = node.right.take();
    format!("{},{},".self.serialize(right), self.serialize(left), node.val)
  }

  fn deserialize(&self, data: String) -> Option<Rc<RefCell<TTreeNode>>> {
    if data.is_empty() {
      return None;
    }
    Self::remove(data.split(",").collect())
  }

  fn remove(&self, mut Vec<String>) -> Option<Rc<RefCell<TTreeNode>>> {
    let val = val.pop().unwrap_or("N");
  }
}

```

```
if val == "N" {
    return None
}

Some {
    val node =
        Node(val: new TreeNode {
            val: val.parse().toInt(),
            left: Self::recurse(left),
            right: Self::recurse(right)
        })
}
}

// ==
// Your Codec object will be instantiated and called as such:
// val obj = Codec()
// let data = "(1,2,3,null,null,4,5)"
// let ans = Solution.deserialize(obj.serialize(data))
//
```

[1] Trees & BST — Pattern Solution (matched from community answers)

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        if root is None:
            return ""
        res = []

        def preorder(root):
            if root is None:
                res.append("N")
            else:
                res.append(str(root.val) + ",")

        preorder(root)

        return str(res)

    def deserialize(self, data):
        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode
        """
        if not data:
            return None
        vals = data.split(',')

        def inner():
            first = vals.pop(0)
            if first == "N":
                return None
            return TreeNode(int(first), inner(), inner())

        return inner()

# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# ans = deser.deserialize(ser.serialize(root))

# =====
from collections import deque

# BFS, queue level based
class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        res = []
        queue = deque([root])
        while queue:
            top = queue.popleft()
            if not top:
                res.append("None")
                continue
            else:
                res.append(str(top.val))
                queue.append(top.left)
                queue.append(top.right)
        return ",".join(res)
```

```
preorder(root, left)
preorder(root, right)

preorder(root)
return ".".join(res)

def deserialize(self, data):
    """Decodes your encoded data to tree.

    :type data: str
    :rtype: TreeNode
    """
    if not data:
        return None
    vals = data.split(',')

    def inner():
        first = vals.pop(0)
        if first == "N":
            return None
        return TreeNode(int(first), inner(), inner())

    # some thing = constructor ...last ...val, left, right

    return inner()

# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# ans = deser.deserialize(ser.serialize(root))

# =====
from collections import deque

# BFS, queue level based
class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        res = []
        queue = deque([root])
        while queue:
            top = queue.popleft()
            if not top:
                res.append("None")
                continue
            else:
                res.append(str(top.val))
                queue.append(top.left)
                queue.append(top.right)
        return ",".join(res)
```

Quick Review — Trees & BST 37 questions · insights · solution

#100 Same Tree [Easy]

Key Insights

- Use recursion to compare nodes in both trees.
- If both nodes being compared are null, they are the same.
- If one node is null while the other is not, the trees differ.
- Check if current nodes' values match, then recursively verify the left and right subtrees.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the trees, since every node is visited once.
Space Complexity: $O(n)$ in the worst-case (unbalanced tree), due to recursion call stack usage. In a balanced tree, the space would be $O(\log n)$.

Solution

The problem can be solved using a recursive approach (Depth-First Search) where we compare the current nodes from both trees. If the current nodes are both null, they match. If one is null, then the trees are not identical. For non-null nodes, check if the values are equal. Then, recursively apply the same logic on the left and right children of the nodes. This approach ensures that both the structure and the node values are identical across the two trees.

#101 Symmetric Tree [Easy]

Key Insights

- Use recursion (or iteration) to compare pairs of nodes.
- For two trees (or two nodes) to be mirror images:
 - Their root values must be the same.
 - The left subtree of one must match the right subtree of the other.
 - The right subtree of one must match the left subtree of the other.
- Alternatively, iterative approaches using a queue can be applied to compare the nodes in pairs.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the trees, since we traverse each node once.
Space Complexity: $O(h)$ for recursive call stack (worst-case $O(n)$ for a skewed tree) or $O(n)$ for the iterative approach using a queue.

Solution

We solve the problem using recursion by defining a helper function that takes two nodes and checks if they are mirrors of each other. The helper function:

- Returns true if both nodes are null.
- Returns false if one node is null or if their values differ.
- Recursively compares the left child of the first node with the right child of the second node, and vice-versa. This method ensures each comparison verifies symmetry at every level. A similar logic can be applied iteratively by using a queue to compare pairs of nodes.

#104 Maximum Depth of Binary Tree [Easy]

Key Insights

- Use a recursive strategy (DFS) to traverse the binary tree.
- At each node, compute the maximum depth of its left and right subtree.
- The maximum depth at the current node is 1 (current node) plus the maximum of the left and right subtree depths.
- Alternatively, an iterative approach using BFS (level-order traversal) can be used.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since every node is visited once.
Space Complexity: $O(h)$ for recursion stack in DFS, where h is the height of the tree (worst-case $O(n)$ for a skewed tree); $O(n)$ for BFS in the worst-case scenario if the tree is balanced.

Solution

The problem is approached using Depth-First Search (DFS) with recursion. The idea is to visit every node and calculate the maximum depth of the left and right subtrees, then add 1 for the current node. This method naturally handles the base case when a null node (empty tree) is reached by returning 0. Alternatively, a Breadth-First Search (BFS) approach can determine the tree level by level, with the total number of levels equalling the maximum depth. Key data structures include recursion call stack for DFS or a queue for BFS.

#108 Convert Sorted Array to Binary Search Tree [Easy]

Key Insights

- The array is sorted in ascending order, which allows the use of a divide and conquer strategy.
- Choosing the middle element of the array as the root ensures that the left and right subtrees are roughly equal in size, yielding a balanced BST.
- A recursive approach is natural for building the tree: the base case is when the subarray is empty.
- The algorithm splits the array into two halves to recursively build the left and right subtrees.

Space & Time

Time Complexity: $O(n)$ - Every element is processed exactly once.
Space Complexity: $O(n)$ - $O(n)$ space is used to store the BST, and additional $O(n)$ space may be used for the recursion stack.

Solution

We use a divide and conquer approach with recursion. The main idea is to select the middle element of the current subarray as the root node so that the left subarray elements form the left subtree and the right subarray elements form the right subtree, ensuring the tree is balanced. The recursion continues until the subarray is empty, which serves as the base case. This strategy naturally maintains the BST property since all elements in the left subarray are smaller than the root, and all in the right subarray are larger.

#110 Balanced Binary Tree [Easy]

Key Insights

- Use a bottom-up DFS (Depth-First Search) traversal to compute the height of each subtree.
- If any subtree is found unbalanced, propagate an error indicator (e.g., -1) upward, allowing early termination.
- An empty tree is considered balanced.
- The key check is to ensure that for each node, the absolute difference between the heights of its left and right subtrees is no more than 1.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the tree, since we visit each node once.
Space Complexity: $O(h)$ where h is the height of the tree, which corresponds to the recursion stack ($O(n)$ in the worst-case of a skewed tree).

Solution

The solution uses a recursive DFS approach. A helper function recursively computes the height of a subtree. If a subtree is found to be unbalanced (the height difference between its left and right subtrees exceeds 1), the helper returns a special value (e.g., -1) to indicate that the subtree is unbalanced. This value is then propagated up the recursion to terminate further unnecessary computations. The main function checks the result of the helper function and returns true if the tree is balanced (i.e., the helper did not return -1) and false otherwise.

#112 Path Sum [Easy]

Key Insights

- Use a depth-first search (DFS) strategy to traverse the tree from the root to the leaves.
- At each node, subtract the node's value from the targetSum.
- Check if the node is a leaf; if yes and the remaining targetSum equals the node's value then a valid path is found.
- If the tree is empty, return false as no path exists.

Space & Time

Time Complexity: $O(N)$ where N is the number of nodes, since every node may need to be visited.
Space Complexity: $O(h)$ where h is the height of the tree, accounting for the recursion stack space in a skewed tree.

Solution

The approach uses a recursive depth-first search. The algorithm subtracts the current node's value from targetSum as it moves down the tree. When it reaches a leaf, it checks if the modified targetSum is equal to the leaf's value. If yes, it returns true. The recursion continues until either a valid path is found or all paths are exhausted. The primary data structure used here is the call stack for recursion.

#226 Invert Binary Tree [Easy]

Key Insights

- The problem can be solved using recursion (depth-first search) by swapping the children of every node.
- Alternatively, an iterative approach (using a queue for breadth-first search) can be used.
- The recursion terminates when a null node is encountered.
- The operation is performed in-place, modifying the original tree structure.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since each node is visited once.
Space Complexity: $O(h)$ where h is the height of the tree, which is the recursion stack space in a recursive approach. In worst-case (skewed tree), this could be $O(n)$.

Solution

We solve the problem using recursion (depth-first search). The approach is to:

- Check the base case: if the node is null, simply return null.
- Recursively invert the left subtree and the right subtree.
- Swap the left and right children of the current node.
- Return the current node after the swap.

The recursive approach elegantly handles all nodes and ensures the inversion process covers the complete tree.

#270 Closest Binary Search Tree Value [Easy]

Key Insights

- BST property allows us to traverse the tree efficiently by comparing the target with node values.
- We can iteratively traverse the tree, moving left if the target is smaller than the current node's value and moving right if greater.
- At each node, calculate the absolute difference between node's value and the target, and update the answer if the difference is smaller—or equal with a smaller candidate value.
- This approach minimizes the search area by leveraging the inherent ordering in BSTs.

Space & Time

Time Complexity: $O(h)$ on average, where h is the height of the tree ($O(\log n)$ for a balanced tree, and $O(n)$ in the worst-case scenario for a skewed tree).
Space Complexity: $O(1)$ for an iterative solution (or $O(h)$ if using recursion due to the call stack).

Solution

We solve the problem by performing an iterative traversal of the BST. Starting at the root, we maintain a variable to store the value closest to the target. For every node encountered, we:

- Compute the absolute difference between the node's value and the target.
- Update the stored closest value if the computed difference is less than the current smallest difference; if the difference is the same, choose the smaller node value.
- Depending on whether the target is less than or greater than the current node's value, move to the left or right child accordingly. This approach efficiently defines a search path that is consistent with the properties of a BST.

#501 Find Mode in Binary Search Tree [Easy]

Key Insights

- The BST in-order traversal yields a sorted order, which helps in counting consecutive duplicates.
- A two-pass in-order traversal can be used: one pass to determine the maximum frequency (maxCount) and a second pass to collect all values that match this frequency.
- The solution can be implemented without extra space (apart from recursion stack) by leveraging constant space counters and state variables.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is processed twice (once in each traversal).
Space Complexity: $O(h)$ where h is the height of the tree due to the recursion stack. $O(1)$ extra space is used aside from this.

Solution

We perform an in-order traversal of the BST twice. The first traversal determines the maximum frequency of any value in the BST by comparing the current node's value with the previously visited node. Since the in-order traversal processes nodes in sorted order, duplicate values are consecutive, making the counting simple. After calculating maxCount, we reset our state and perform a second in-order traversal to collect all node values with frequency equal to maxCount. The key trick is to leverage the BST property (sorted in-order traversal) to count frequencies in one pass without using additional data structures like hash maps.

#543 Diameter of Binary Tree [Easy]

Key Insights

- Use depth-first search (DFS) to explore the tree.
- The diameter at a node is the sum of the depths of its left and right subtrees.
- Track the maximum diameter found during the DFS traversal.
- Recursively calculating the depth of subtrees provides an efficient solution.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(n)$, in the worst-case scenario due to the recursion stack (skewed tree).

Solution

The solution employs a DFS traversal of the binary tree. At every node in the tree, compute the depth of the left and right subtrees. The sum of these depths gives the potential diameter (longest path through that node). Update a global maximum diameter variable if this sum exceeds the current maximum. Finally, return the maximum diameter after completing the DFS.

#572 Subtree of Another Tree [Easy]

Key Insights

- Use Depth-First Search (DFS) to visit each node in the main tree.
- At each node, check if the subtree starting from that node is identical to subRoot.
- A helper function is used to compare the structure and node values of two trees.
- Consider recursion as the primary approach to traverse and compare trees.
- Early exit when a mismatch is found can optimize average-case performance.

Space & Time
Time Complexity: $O(m \cdot n)$ in the worst case, where m is the number of nodes in the main tree and n is the number of nodes in the subtree trees.
Space Complexity: $O(\max(\text{depth of main tree, depth of subtree}))$ due to recursive call stack.

Solution

We solve the problem by performing a DFS traversal on the main tree. For each visited node, a helper function checks whether the subtree starting at that node is identical to subtree by comparing node values and recursively checking both left and right subtrees. This method leverages recursion effectively and provides early termination if mismatches occur, allowing for efficient traversal and comparison.

#98 Validate Binary Search Tree [Medium]

Key Insights

- Every node must adhere to a range: all nodes in the left subtree should be less than the current node, and all nodes in the right subtree should be greater.
- A recursive depth-first search approach works well by passing down the valid range (lower and upper bounds) for each node.
- Using the limits (negative and positive infinity) simplifies handling edge cases at the tree root.
- An in-order traversal would alternatively yield a sorted order for node values, but updating ranges directly is more intuitive for recursive implementation.

Space & Time
Time Complexity: $O(N)$, where N is the number of nodes, as each node is visited once.
Space Complexity: $O(H)$, where H is the height of the tree, due to the recursion stack.

Solution

We use a recursive approach to validate the BST property. We maintain two variables, low and high, representing the valid range for the current node's value. For the left child, the valid range becomes (low, node.val) and for the right child, it becomes (node.val, high). By checking if each node's value falls within its respective range, we can ensure that the tree satisfies the BST properties.

#102 Binary Tree Level Order Traversal [Medium]

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree.
- A queue data structure is ideal for maintaining the order of nodes at each level.
- For every level, process all nodes currently in the queue and add their children for the next level.

Space & Time
Time Complexity: $O(N)$, where N is the number of nodes, because each node is processed once.
Space Complexity: $O(N)$, which accounts for the space used by the queue in the worst-case scenario.

Solution
This solution leverages a BFS approach using a queue. The algorithm starts by enqueueing the root node. Then, it enters a loop that continues until the queue is empty. For each iteration—representing one level of the tree—it computes the number of nodes at that level (levelSize), processes these nodes by dequeuing them, and collects their values. Their non-null children are then enqueued. The list of values for each level is added to the final result list. Key data structures include the queue (for managing nodes) and a list (to store level values).

#103 Binary Tree Zigzag Level Order Traversal [Medium]

Key Insights

- Use a breadth-first search (BFS) approach to traverse the tree level by level.
- Alternate the direction of traversal on each level to achieve the zigzag pattern.
- A flag or counter can be used to determine the order of nodes for the current level.

Trees & BST - LeetCodeMastery - By Pattern — 3409 — LeetCodeMastery

In both approaches, appropriate data structures (a queue for BFS or recursion with a list for DFS) are used to store nodes and track the traversal state.

#230 Kth Smallest Element in a BST [Medium]

Key Insights

- An in-order traversal of a BST visits nodes in increasing order.
- Use a counter to keep track of the number of nodes visited.
- As soon as the counter reaches k , the current node's value is the k th smallest.
- For frequent modifications (insertions/deletions) and queries, consider augmenting the BST with subtree counts.

Space & Time
Time Complexity: $O(N)$ in the worst-case scenario (when $k = N$), but on average the traversal may stop early.
Space Complexity: $O(H)$ where H is the height of the tree ($O(N)$ in the worst-case for a skewed tree).

Solution
We perform an in-order traversal of the BST which visits nodes in ascending order. During the traversal, we maintain a counter (or decrement k) to check when we have encountered the k th smallest element. When the counter reaches 0 (or k becomes 0), we capture the current node's value as our answer and return it. This approach is both simple and efficient for the given constraints.

#235 Lowest Common Ancestor of a Binary Search Tree [Medium]

Key Insights

- Utilize the BST property: for any node, values in the left subtree are less and values in the right subtree are greater.
- If both p and q have values less than the current node, the LCA must lie in the left subtree.
- If both p and q have values greater than the current node, the LCA must lie in the right subtree.
- When one node value is less than and the other is greater than (or equal to) the current node, the current node is the lowest common ancestor.
- An iterative approach is efficient and uses constant extra space.

Space & Time
Time Complexity: $O(H)$, where H is the height of the tree ($O(\log n)$ for balanced trees and $O(n)$ in the worst-case skewed tree).
Space Complexity: $O(1)$ with an iterative approach ($O(n)$ if using recursion due to call stack).

Solution
The solution leverages the BST properties. Algorithm:
- Start at the root node. - While the current node exists: if both p and q have values less than the current node's value, move to the left child. if both p and q have values greater than the current node's value, move to the right child. Otherwise, the current node is the split point where p and q diverge, so it is the lowest common ancestor. - Return the current node.
This approach efficiently finds the LCA using a single traversal from the root.

#236 Lowest Common Ancestor of a Binary Tree [Medium]

Key Insights

- Use a depth-first search (DFS) traversal to explore the tree.
- When a node's value is either p or q , it could be the LCA.
- Recursively find p and q in the left and right subtrees.
- If both left and right recursive calls return a non-null value, then the current node is the LCA.
- If only one side returns a non-null value, propagate that value up the recursion.

Space & Time
Time Complexity: $O(N)$, where N is the number of nodes in the tree since each node is visited once.

Trees & BST - LeetCodeMastery - By Pattern — 3411 — LeetCodeMastery

- Pass the current node's value and the length of the current consecutive sequence in the recursive call.
- When a node's value is exactly one greater than its parent's value, increment the sequence length; otherwise, reset it to 1.
- Keep track of the maximum sequence length found during the traversal.

Space & Time
Time Complexity: $O(N)$, where N is the number of nodes in the tree, since each node is visited once.
Space Complexity: $O(H)$, where H is the height of the tree. In the worst-case of a skewed tree, the recursion stack can be $O(N)$.

Solution
The solution involves using DFS to explore every node in the binary tree while keeping track of the length of the consecutive sequence. For each node, if the node's value is one greater than its parent's value, we increment the sequence length; otherwise, we reset the sequence length to 1. During the traversal, a global variable is updated if the current sequence length exceeds the previously recorded maximum. This approach ensures that every possible consecutive path is considered.

#314 Binary Tree Vertical Order Traversal [Medium]

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree level by level.
- Track the column index for each node where root is column 0, left child is column-1, and right child is column+1.
- Group nodes by their column indices using a hash table (or dictionary).
- BFS traversal naturally handles left-to-right ordering within the same level.
- Finally, sort the keys (column indices) to generate results from leftmost column to rightmost column.

Space & Time
Time Complexity: $O(n \log n)$ in the worst-case due to sorting the column indices in nodes in the worst-case if each node is on a unique column.
Space Complexity: $O(n)$ for storing the node information and the resultant structure.

Solution
The solution uses BFS with a queue that stores nodes along with their corresponding column index. For each visited node, add its value to a dictionary keyed by its column index. When processing children, update the column index appropriately (left child: col-1, right child: col+1). After the traversal, sort the keys of the dictionary and compile the node values in order from smallest to largest column index. This approach guarantees that nodes at the same level will be processed in left-to-right order, matching the required output.

#333 Largest BST Subtree [Medium]

Key Insights

- Use a bottom-up DFS (post-order traversal) to process each node.
- For each node, determine whether its left and right subtrees are BSTs.
- Maintain additional data per subtree: size (number of nodes), minimum value, and maximum value.
- A subtree rooted at the current node is a BST if:
 - Its left subtree is a BST.
 - Its right subtree is a BST.
 - The maximum value in the left subtree is less than the current node's value.
 - The current node's value is less than the minimum value in the right subtree.
- Track the size of the largest BST identified during the traversal.

Space & Time
Time Complexity: $O(n)$ (each node is visited once)
Space Complexity: $O(H)$, where H is the height of the tree (due to recursion stack)

Trees & BST - LeetCodeMastery - By Pattern — 3413 — LeetCodeMastery

- Using a double-ended data structure or simply reversing the list at every alternate level is an effective approach.
- Space & Time**
Time Complexity: $O(n)$, where n is the number of nodes, because each node is visited exactly once.
Space Complexity: $O(n)$, as in the worst-case scenario (e.g., a complete binary tree) the queue used for BFS may hold $O(n)$ nodes at once.

Solution

We can solve this problem using a breadth-first search (BFS). Start by initializing a queue to hold nodes for each level. For each level:
- Determine the number of nodes at the current level. - Iterate over these nodes, appending their children for the next level. - Depending on a flag (or level count), either append node values in the natural order (left to right) or reverse the order (right to left) before adding to the result. Key data structures:
- A queue for BFS traversal. - A list (or similar container) to store current level values. The main trick is toggling the order of insertion or reversing the list at alternate levels to achieve the "zigzag" pattern.

#105 Construct Binary Tree from Preorder and Inorder Traversal [Medium]

Key Insights

- Preorder traversal always starts with the root node.
- Inorder traversal splits the tree into left subtree and right subtree based on the root.
- A hash table (map/dictionary) can be used to quickly locate the index of the root in the inorder array.
- Use recursion to construct left and right subtrees.

Space & Time
Time Complexity: $O(n)$ since we process every node exactly once.
Space Complexity: $O(n)$ due to the recursion stack and the hash map used for inorder indices.

Solution

We solve the problem using a recursive divide-and-conquer approach. First, create a hash map to store each value's index in the inorder array for quick lookups. The preorder array always provides the current root. Use the inorder array to divide the tree into left and right subtrees. Recursively construct the subtrees by adjusting the preorder index and the boundaries of the inorder array. The algorithm leverages recursion and a hash map to achieve $O(n)$ time complexity.

#199 Binary Tree Right Side View [Medium]

Key Insights

- The problem is essentially about capturing the last node at each level for the rightmost node in a binary tree.
- A Breadth-First Search (BFS) approach naturally works by processing nodes level by level.
- Alternatively, Depth-First Search (DFS) can be adapted by prioritizing the right subtree first, ensuring that the first node encountered at each depth is the visible one.
- The number of nodes is capped at 100, so both approaches are efficient.

Space & Time
Time Complexity: $O(n)$ - visit every node in the tree.
Space Complexity: $O(n)$ - in the worst case, the queue (or call stack for DFS) may hold all nodes of the tree.

Solution

We can solve this problem using BFS (level-order traversal). The idea is to traverse the tree level by level using a queue. For each level, the last element added to the level represents the rightmost view from that level. We then append that element's value to our result list. Alternatively, a DFS approach can be used where we visit the right subtree first. By keeping track of the current depth and checking if it's the first time we've encountered this depth, we can add the first node visited at that depth (which will be the rightmost one) to the result.

Trees & BST - LeetCodeMastery - By Pattern — 3410 — LeetCodeMastery

Space Complexity: $O(H)$, where H is the height of the tree due to the recursion stack ($O(n)$ in the worst case of a skewed tree).

Solution

We solve the problem using recursion with depth-first search. The algorithm checks if the current node matches p or q . If then recursively searches the left and right subtrees. If both subtrees contain one of p or q , the current node is the LCA. Otherwise, the algorithm forwards the non-null result up the recursion. This approach naturally handles cases where one node is an ancestor of the other.

#250 Count Unvalue Subtrees [Medium]

Key Insights

- Perform a postorder traversal (left, right, root) to determine whether each subtree is unvalue.
- Treat null nodes as unvalue to ease the recursion.
- A node's subtree is unvalue if both left and right subtrees are unvalue and, if they exist, their values equal the current node's value.
- Use a global or external counter that increments every time a unvalue subtree is confirmed.

Space & Time
Time Complexity: $O(N)$, where N is the number of nodes, since each node is visited once.
Space Complexity: $O(H)$, where H is the height of the tree, due to the recursion stack.

Solution

We use a recursive DFS approach (postorder traversal) to solve the problem. For each node, we recursively check its left and right subtrees to decide if they are unvalue. By default, if a child is null, it is considered unvalue. Then, we verify the current node's value against its non-null children. If both conditions hold (subtrees are unvalue and matching values), we count the current node's subtree as unvalue. This DFS checks in a bottom-up manner, ensuring that the properties required for a subtree to be unvalue are maintained. The key trick is to return a boolean indicating if the subtree at each node is unvalue and, while backtracking, increment the counter appropriately.

#285 Inorder Successor in BST [Medium]

Key Insights

- In a BST, an in-order traversal yields nodes in ascending order.
- If the node p has a right subtree, the in-order successor is the left-most node in that right subtree.
- If p has no right subtree, the successor is one of its ancestors: traverse from the root towards p while keeping track of the last node that is greater than p .
- An iterative approach is efficient and avoids recursion stack overhead.

Space & Time
Time Complexity: $O(H)$, where H is the height of the tree. In the worst-case (skewed tree), it can be $O(n)$.
Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We solve the problem using two cases:
- If node p has a right subtree, we simply find the left-most node in that right subtree. - If node p does not have a right subtree, initiate a search from the BST root. As we traverse: If the current node's value is greater than p val, the current node is a candidate for the successor; move to the left subtree to look for a smaller candidate. Otherwise, move to the right subtree. By the end of the traversal, the candidate (if exists) is the in-order successor. This approach benefits from the BST property, limiting traversal to $O(H)$ time.

#298 Binary Tree Longest Consecutive Sequence [Medium]

Key Insights

- Use a Depth-First Search (DFS) to traverse the binary tree.

Trees & BST - LeetCodeMastery - By Pattern — 3412 — LeetCodeMastery

Solution
Perform a post-order traversal of the binary tree. For each node, return a tuple (or object in some languages) that provides:
- Whether the subtree rooted at this node is a BST. - The size of the subtree if it is a BST (or the size of the largest BST found within it). - The minimum value in the subtree. - The maximum value in the subtree.
At each node, if both left and right children form valid BSTs and the current node's value fits the BST condition (greater than left's max and less than right's min), then the subtree rooted at the node is also a BST. Calculate its size and update the current global maximum if necessary. If the condition fails, return the maximum BST size found in its subtrees.

#366 Find Leaves of Binary Tree [Medium]

Key Insights

- Use a postorder traversal (bottom-up approach) to process the tree.
- Determine the "height" of each node (distance from the deepest leaf) where leaves have a height of 1.
- Group nodes by their computed height to simulate the layer-wise removal of leaves.
- The same node may become a leaf after its children are removed.

Space & Time
Time Complexity: $O(n)$ - Each node is visited exactly once.
Space Complexity: $O(n)$ - Space used for the recursion stack and the result list.

Solution

The solution uses a recursive helper function that performs a postorder traversal of the tree. For each node, it computes its height as $\max(\text{height of left, height of right}) + 1$. Nodes with the same height are collected together in a list. This idea mirrors the process of removing leaves level by level, where leaves (height 1) are removed first, then their parents become new leaves, and so on. Data Structures used:
- An array or list to maintain the groups of node values by their height. Algorithmic Approach: - Use Depth-First Search (DFS) via recursion. - Postorder traversal ensures that leaf nodes (base cases) are processed before their parents. - Append the node's value to the correct list based on the computed height.

#437 Path Sum III [Medium]

Key Insights

- A path can start and end at any nodes, as long as it goes downwards.
- Using DFS helps traverse the tree and explore all possible paths.
- A prefix sum technique with a hash map allows quick calculation of the sum of any subpath.
- Backtracking is used to remove a path's prefix sum when moving back up the recursion tree.
- Although a brute-force approach exists, it is inefficient compared to the prefix sum method.

Space & Time
Time Complexity: $O(n)$ on average (where n is the number of nodes), though worst-case can be $O(n^2)$ in a skewed tree.
Space Complexity: $O(n)$ due to recursion stack and the hash map that stores prefix sums.

Solution

The solution employs a DFS traversal of the tree along with a hash map that records the frequency of prefix sums encountered so far. At each node, the current prefix sum is updated by adding the node's value. We then check the hash map for how many times each current prefix sum - targetSum has occurred, since this value indicates the existence of a subpath summing to targetSum. We update the hash map with the current prefix sum, continue the DFS to traverse child nodes, and then backtrack by decrementing the current prefix sum's count before returning to the parent node. This ensures that sums from other branches are not incorrectly combined.

#545 Boundary of Binary Tree [Medium]

Trees & BST - LeetCodeMastery - By Pattern — 3414 — LeetCodeMastery

Key Insights

• Break down the problem into three parts:

- Left boundary (excluding leaves).
- All leaves (do a DFS to find leaves).
- Right boundary (excluding leaves, then reverse the collected list).
- Special handling is needed when the tree doesn't have a left or right subtree.
- The root is always included and is never considered as a leaf, even if it has no children.
- Avoid duplication of leaf nodes in the left/right boundaries.

Space & Time

Time Complexity: O(n) - Every node is visited at most once.

Space Complexity: O(n) - O(n) space is used for the output list and recursion stack (in worst case tree is skewed).

Solution

The solution uses a Depth-First Search (DFS) strategy to traverse the tree and collect the boundary nodes in three steps:

- For the left boundary, start at root.left and traverse down, adding nodes (skipping leaves).
- For the leaves, perform a DFS from the root and add any node that has no children.
- For the right boundary, start at root.right and traverse down, adding nodes (skipping leaves).
- Since the right boundary must be added in reverse order, we reverse the collected list after traversal.
- Finally, concatenate these lists in the order: root, left boundary, leaves, reversed right boundary.

The approach makes use of helper functions to clearly separate the logic for gathering left boundary, leaves, and right boundary. This modular approach simplifies reasoning and avoids edge case mistakes such as including duplicate leaf nodes.

#549 Binary Tree Longest Consecutive Sequence II [Medium]

Key Insights

• Use Depth-First Search (DFS) to visit every node.

- For each node, compute:
- The longest increasing consecutive path starting from that node.
- The longest decreasing consecutive path starting from that node.
- Combine the increasing and decreasing paths (subtracting one to avoid double counting the current node) to potentially form a longer consecutive sequence through the node.
- Update a global maximum to record the longest path seen.

Space & Time

Time Complexity: O(n), where n is the number of nodes since every node is visited once.

Space Complexity: O(h), where h is the height of the tree, due to the recursion stack.

Solution

The solution employs a DFS traversal. At every node, we calculate two values:

- inc - the length of the longest increasing consecutive sequence starting from that node.
- dec - the length of the longest decreasing consecutive sequence starting from that node.

For each child of the current node:

- If the child's value is node.val + 1, update the increasing sequence.
- If the child's value is node.val - 1, update the decreasing sequence.

Then, the longest path through the current node is inc + dec - 1 (subtracting one to avoid counting the current node twice). A global variable maintains the maximum length encountered. This process works regardless of the path direction because it combines both increasing and decreasing parts.

#582 Kill Process [Medium]

Trees & BST - LeetCode - By Pattern

— 3415 —

LeetCode

- Build an undirected graph representation of the binary tree using a DFS. For each node, add the node's value as a key in an adjacency list and connect it with its children (if any). This creates bidirectional links between parent and child.

- Use a BFS starting from the target node and explore nodes level-by-level. Keep a counter for the current distance, and once the counter reaches k, return all node values found at that level. To avoid processing nodes more than once, maintain a set of visited nodes.

#1120 Maximum Average Subtree [Medium]

Key Insights

- Use a depth-first search (DFS) to traverse every subtree.
- At each node, compute the sum and count of nodes in its subtree.
- Calculate the average for the current subtree and update a global maximum if needed.
- Processing every node exactly once yields an efficient solution.

Space & Time

Time Complexity: O(n), where n is the number of nodes (each node is visited once).

Space Complexity: O(h), where h is the height of the tree (space for recursion stack).

Solution

We perform a post-order DFS traversal, calculating at each node both the cumulative sum and the count of nodes in its subtree. This enables us to compute the average value in constant time for any subtree. We maintain a global variable to track the maximum average encountered during the traversal. The primary technique is recursive DFS combined with tuple (or pair) aggregation. The simplicity of the recursion and in-place aggregation ensures an efficient and clear solution.

#1490 Clone N-ary Tree [Medium]

Key Insights

- The tree is N-ary, meaning every node can have multiple children.
- A deep copy requires creating new nodes for each node in the tree.
- Both DFS (recursive) or BFS (iterative) traversals can be used to clone the tree.
- The overall time complexity is O(N) since each node is visited once.
- We can follow similar cloning techniques used in cloning graphs, using either recursion or an auxiliary data structure like a queue.

Space & Time

Time Complexity: O(N), where N is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: O(N), accounting for the recursive call stack (or an explicit queue in BFS) and the space needed to store the clone.

Solution

We perform a DFS traversal starting from the root node. For each node we encounter, we:

- Create a new node with the same value.
- Recursively clone all its children and attach the cloned children to the new node.

This approach ensures that every original node is copied into a new node and that the parent-child relationships are preserved. Edge cases such as an empty tree (null root) are also handled by returning null immediately.

#1522 Diameter of N-Ary Tree [Medium]

Key Insights

- The diameter of the tree can be found by considering the two largest depths from any node among its children.
- Use a depth-first search (DFS) to compute the maximum depth from each node.
- For each node, the potential diameter is the sum of the two longest depths among its children.
- Update a global variable tracking the maximum diameter as you recurse through the tree.

Space & Time

Time Complexity: O(n) for both serialization and deserialization, where n is the number of nodes in the tree.

Space Complexity: O(n) due to the storage used for the serialized string and additional data structures (queue) during processing.

Solution

This solution uses level-order traversal (BFS) for both serialization and deserialization. During serialization, each node is processed sequentially with missing children recorded as "null". For deserialization, the string is split by commas, and a queue is used to rebuild the tree level by level. The root is created from the first token, then subsequent tokens are assigned as left and right children accordingly, ensuring an accurate reconstruction of the original tree.

Key Insights

- The tree can be traversed using either depth-first search (DFS) or breadth-first search (BFS).
- Using BFS (level-order traversal) simplifies reconstruction since nodes are processed level-by-level.
- Placeholder markers (e.g., "null") are used to indicate missing children.
- Both serialization and deserialization processes should handle empty trees and edge cases gracefully.
- Ensuring a one-to-one mapping between the tree structure and the serialized string is critical.

Space & Time

Time Complexity: O(n) for both serialization and deserialization, where n is the number of nodes in the tree.

Space Complexity: O(n) due to the storage used for the serialized string and additional data structures (queue) during processing.

Solution

This solution uses level-order traversal (BFS) for both serialization and deserialization. During serialization, each node is processed sequentially with missing children recorded as "null". For deserialization, the string is split by commas, and a queue is used to rebuild the tree level by level. The root is created from the first token, then subsequent tokens are assigned as left and right children accordingly, ensuring an accurate reconstruction of the original tree.

Trees & BST - LeetCode - By Pattern

— 3419 —

LeetCode

Key Insights

- Build a mapping (dictionary/hash table) from a parent process ID to its list of child process IDs.
- Use Depth-First Search (DFS) or Breadth-First Search (BFS) starting from the kill process to traverse the tree and collect all descendant processes.
- The problem requires handling a tree structure that may have multiple children per node.

Space & Time

Time Complexity: O(n), where n is the number of processes (both building the mapping and the tree traversal take linear time).

Space Complexity: O(n), for storing the mapping and the traversal queue/stack.

Solution

We first construct a mapping from each process's parent ID to its list of child IDs. This allows us to quickly access all children of any process. Once the mapping is built, we perform a tree traversal (either BFS or DFS) starting from the process to kill. During the traversal, we record every process encountered. This collected list represents all processes that will be terminated. The data structures used are a dictionary (for the mapping) and either a stack (for DFS) or a queue (for BFS), making the solution efficient and straightforward.

#662 Maximum Width of Binary Tree [Medium]

Key Insights

- Perform a level order traversal (BFS) of the tree.
- Assign an index to each node as if the tree were complete (e.g., root is index 1, left child is 2
- 1 , right child is 2
- $i+1$).
- For each level, compute the width as (last_index - first_index + 1).
- Using indices allows capturing gaps between nodes due to nulls in between.
- Edge cases include handling very skewed trees.

Space & Time

Time Complexity: O(N), where N is the number of nodes in the tree.

Space Complexity: O(N), due to the queue used for BFS which in the worst case holds nodes of the last level.

Solution

We use a Breadth-First Search (BFS) approach with a queue to traverse the tree level by level. At each level, we pair each node with an index representing its position assuming a complete binary tree. This allows us to compute the width of each level using the formula: width = last_index - first_index + 1. We then track the maximum width seen throughout the traversal. The use of indices handles cases where there are null gaps between nodes.

#863 All Nodes Distance K in Binary Tree [Medium]

Key Insights

- Convert the binary tree into an undirected graph so that we can easily traverse both down to the children and up to the parent.
- Use a DFS or simple recursive traversal to create an adjacency list representing the graph.
- Perform a BFS starting from the target node to explore nodes level-by-level until reaching distance k.
- Use a visited set (or equivalent) to avoid revisiting nodes and to prevent cycles.

Space & Time

Time Complexity: O(N), where N is the number of nodes in the tree. Each node is visited once when constructing the graph and once in the BFS.

Space Complexity: O(N), for storing the graph and the additional data structures used in BFS and DFS.

Solution

The solution involves two main steps:

Trees & BST - LeetCode - By Pattern

— 3418 —

LeetCode

Time Complexity: O(n), where n is the number of nodes, since each node is visited once.

Space Complexity: O(h), where h is the height of the tree, due to the recursion stack (worst-case O(n) for a skewed tree).

Solution

Use a DFS approach to traverse the tree. At each node, calculate the longest and second longest depth among its children. The sum of these two depths gives the candidate diameter through that node. Maintain a global variable to track the maximum diameter found so far. Return the maximum depth from the node i.e., one plus the largest depth among its children) to be used in parent's calculations.

#1650 Lowest Common Ancestor of a Binary Tree III [Medium]

Key Insights

- Each node has a parent pointer, so you can traverse from any node to the root.
- By determining the depth of each node, you can "align" the two nodes to the same depth.
- When both nodes are aligned, traverse upward simultaneously until they meet.
- This approach guarantees finding the LCA with O(h) time complexity, where h is the height of the tree.

Space & Time

Time Complexity: O(h), where h is the height of the tree.

Space Complexity: O(1)

Solution

We first compute the depth of both nodes by moving up their parent pointers. If one node is deeper, move it upward until both nodes are at the same level. Then, move both nodes upward one step at a time until they meet; the meeting point is the lowest common ancestor. This approach uses constant extra space and leverages the parent pointer for an efficient solution.

#124 Binary Tree Maximum Path Sum [Hard]

Key Insights

- Use Depth-First Search (DFS) to explore the tree in a post-order fashion.
- For each node, calculate the maximum gain including that node and optionally one of its subtrees.
- A node's best contribution to its parent can be either just its own value or its value plus the maximum gain from either the left or right child.
- Update a global maximum path sum that considers the possibility of combining both left and right gains with the current node.
- Handle negative values by comparing gains against 0 to avoid decreasing the path sum.

Space & Time

Time Complexity: O(n), where n is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: O(n) in the worst-case scenario due to the recursion stack (O(h) where h is the height of the tree, which can be O(n) for skewed trees).

Solution

We use a recursive DFS approach (post-order traversal) to compute the maximum gain from each subtree. For every node:

- Recursively compute the maximum gain from the left and right child, ensuring that negative gains are treated as 0.
- The maximum gain that can be provided to the node's parent is the node's value plus the larger gain from its left or right child.
- Simultaneously, the potential maximum path sum including the current node as the highest (or root) node is the node's value plus both left and right gains.
- Update a global variable that holds the maximum path sum found so far.
- Return the maximum gain (node value plus one best child to be used by the node's parent).

This strategy ensures that every possible path sum is considered while avoiding double-counting nodes.

#297 Serialize and Deserialize Binary Tree [Hard]

Trees & BST - LeetCode - By Pattern

— 3418 —

LeetCode